XMTPS: X-Message Telemetry Processing Service

0.6

Generated on Sat Oct 24 2026 14:53:30 for XMTPS: X-Message Telemetry Processing Service by Doxygen 1.11.0

Sat Oct 24 2026 14:53:30

1 XMTPS — X-Message Telemetry Processing Service 1
1.1 How it works . . . 1
1.1.1 Source layout . . . 1
1.2 Building . . . 2
1.3 Configuration . . . 2
1.3.1 How options are read . . . 2
1.3.2 Command-line only options . . . 2
1.3.3 Configuration file options . . . 3
1.3.4 Example configuration file . . . 4
1.3.5 Float metadata . . . 5
1.4 Running . . . 5
1.4.1 Output files . . . 5
1.5 Database schema . . . 5
1.5.1 `XFloat` . . . 6
1.5.2 `XFloatMetadata` . . . 6
1.5.3 `XFloatDive` . . . 6
1.5.4 `XGPS` . . . 7
1.5.5 `XFloatMail` . . . 7
1.6 Deployment . . . 7
1.7 Python tools (`bin/py`) . . . 7
1.7.1 Command-line options . . . 8
1.8 Changelog . . . 9
1.8.1 2026-10-24 . . . 9
1.8.2 2026-10-23 . . . 9
1.8.3 2026-08-11 . . . 9
1.8.4 2026-08-06 . . . 10
1.8.5 2026-08-05 . . . 10
1.9 Author and license . . . 10

2 Libraries and Database Schema 11
2.1 Libraries . . . 11
2.1.1 Boost C++ Libraries . . . 11
2.1.1.1 program_options . . . 11
2.1.1.2 filesystem . . . 11
2.1.1.3 log . . . 11
2.1.1.4 thread . . . 11
2.1.2 MySQL C++ Connector . . . 11
2.1.3 VMime MIME & Mail Library for C++ . . . 12
2.1.4 libkml . . . 12
2.2 SQL Database Schema . . . 12
2.2.1 Table XFloat . . . 13
2.2.2 Table XFloatMetadata . . . 14
2.2.3 Table XFloatDive . . . 14
2.2.4 Table XGPS . . . 14
2.2.5 Table XFloatMail . . . 15

3 Deprecated List 17

4 Todo List 19

5 Namespace Index 23
5.1 Namespace List . . . 23

6 Hierarchical Index 25

6.1 Class Hierarchy . . . 25

7 Class Index **29**

7.1 Class List . . . 29

8 File Index **33**

8.1 File List . . . 33

9 Namespace Documentation **37**

9.1 argo Namespace Reference . . . 37
9.1.1 Detailed Description . . . 37
9.2 argo::iridium Namespace Reference . . . 37
9.2.1 Detailed Description . . . 37
9.3 argo::iridium::xfloat Namespace Reference . . . 37
9.3.1 Detailed Description . . . 38
9.3.2 Typedef Documentation . . . 39
9.3.2.1 XMailSbdByte_t . . . 39
9.3.2.2 XMailSbdData_t . . . 39
9.3.3 Enumeration Type Documentation . . . 39
9.3.3.1 XFloatMailRawFileFormat_t . . . 39
9.3.4 Function Documentation . . . 39
9.3.4.1 operator<<() [1/2] . . . 39
9.3.4.2 operator<<() [2/2] . . . 39
9.3.4.3 operator>>() [1/2] . . . 40
9.3.4.4 operator>>() [2/2] . . . 40
9.4 boost Namespace Reference . . . 41
9.4.1 Detailed Description . . . 41
9.5 boost::container Namespace Reference . . . 41
9.5.1 Detailed Description . . . 41
9.6 util Namespace Reference . . . 41
9.6.1 Detailed Description . . . 42
9.7 util::config Namespace Reference . . . 42
9.7.1 Detailed Description . . . 42
9.8 util::date Namespace Reference . . . 42
9.8.1 Detailed Description . . . 43
9.9 util::file Namespace Reference . . . 43
9.9.1 Detailed Description . . . 43
9.10 util::ftp Namespace Reference . . . 43
9.10.1 Detailed Description . . . 43
9.11 util::log Namespace Reference . . . 43
9.11.1 Detailed Description . . . 44
9.11.2 Typedef Documentation . . . 44
9.11.2.1 sev_logger_mt . . . 44
9.11.3 Enumeration Type Documentation . . . 44
9.11.3.1 LogLevel_t . . . 44
9.12 util::mail Namespace Reference . . . 44
9.12.1 Detailed Description . . . 44
9.13 util::math Namespace Reference . . . 45
9.13.1 Detailed Description . . . 45
9.14 util::mysql Namespace Reference . . . 45
9.14.1 Detailed Description . . . 46
9.14.2 Enumeration Type Documentation . . . 46
9.14.2.1 ColumDataType_t . . . 46

9.15 util::position Namespace Reference . . . 46
9.15.1 Detailed Description . . . 46
9.16 util::sig Namespace Reference . . . 47
9.16.1 Detailed Description . . . 47
9.17 util::stream Namespace Reference . . . 47
9.17.1 Detailed Description . . . 47
9.18 util::string Namespace Reference . . . 47
9.18.1 Detailed Description . . . 47
9.19 util::sys Namespace Reference . . . 47
9.19.1 Detailed Description . . . 47
9.20 util::thread Namespace Reference . . . 48
9.20.1 Detailed Description . . . 48
9.21 util::types Namespace Reference . . . 48
9.21.1 Detailed Description . . . 48
9.22 util::xmsg Namespace Reference . . . 48
9.22.1 Detailed Description . . . 54
9.22.2 Enumeration Type Documentation . . . 54
9.22.2.1 XFallRiseDataPackFormat_t . . . 54
9.22.2.2 XMsgTypeId_t . . . 55
9.22.2.3 XPhaseCode_t . . . 58
9.22.2.4 XPumpPackingFormat_t . . . 59
9.22.2.5 XSensorDataFormat_t . . . 59
9.22.2.6 XWaveMode_t . . . 59
9.23 vmime Namespace Reference . . . 59
9.23.1 Detailed Description . . . 59

10 Class Documentation **61**

10.1 util::xmsg::AbstractXComposite< T > Class Template Reference . . . 61
10.1.1 Detailed Description . . . 62
10.1.2 Member Typedef Documentation . . . 62
10.1.2.1 Element_t . . . 62
10.1.3 Constructor & Destructor Documentation . . . 62
10.1.3.1 AbstractXComposite() . . . 62
10.1.3.2 ~AbstractXComposite() . . . 62
10.1.4 Member Function Documentation . . . 62
10.1.4.1 AppendAllElements() [1/2] . . . 62
10.1.4.2 AppendAllElements() [2/2] . . . 63
10.1.4.3 AppendElement() . . . 63
10.1.4.4 AssignElement() . . . 63
10.1.4.5 elements() . . . 64
10.1.4.6 num_elements() . . . 64
10.1.5 Member Data Documentation . . . 64
10.1.5.1 elements_ . . . 64
10.2 util::xmsg::AbstractXFallRise Class Reference . . . 65
10.2.1 Detailed Description . . . 66
10.2.2 Constructor & Destructor Documentation . . . 66
10.2.2.1 AbstractXFallRise() . . . 66
10.2.2.2 ~AbstractXFallRise() . . . 67
10.2.3 Member Function Documentation . . . 67
10.2.3.1 CheckXMsgData() . . . 67
10.2.3.2 data_pack_format() . . . 68
10.2.3.3 DecodeXMsgData() . . . 68

10.2.3.4 GenerateCurrentUtcTimeIso8601String() . . . 69
10.2.3.5 GenerateIso8601String() . . . 69
10.2.3.6 GenerateJulianDay() . . . 70
10.2.3.7 GenerateListingComment() . . . 71
10.2.3.8 GetCurrentUtcTime() . . . 71
10.2.3.9 id_dive() . . . 72
10.2.3.10 id_instrument() . . . 73
10.2.3.11 id_xmsg_type() . . . 73
10.2.3.12 num_samples() . . . 74
10.2.3.13 sensor_data() . . . 74
10.2.3.14 set_data_pack_format() . . . 75
10.2.3.15 set_id_dive() . . . 75
10.2.3.16 set_id_instrument() . . . 76
10.2.3.17 set_id_xmsg_type() . . . 76
10.2.3.18 set_start_time() . . . 77
10.2.3.19 set_xmsg_index() . . . 78
10.2.3.20 set_xmsg_size_bytes() . . . 79
10.2.3.21 start_time() . . . 80
10.2.3.22 UnpackData() . . . 80
10.2.3.23 xmsg_index() . . . 81
10.2.3.24 xmsg_size_bytes() . . . 82
10.2.4 Member Data Documentation . . . 82
10.2.4.1 data_pack_format_ . . . 82
10.2.4.2 id_dive_ . . . 83
10.2.4.3 id_instrument_ . . . 83
10.2.4.4 id_xmsg_type_ . . . 83
10.2.4.5 sensor_data_ . . . 83
10.2.4.6 start_time_ . . . 83
10.2.4.7 xmsg_index_ . . . 83
10.2.4.8 xmsg_size_bytes_ . . . 83
10.3 util::xmsg::AbstractXSensorData Class Reference . . . 84
10.3.1 Detailed Description . . . 85
10.3.2 Constructor & Destructor Documentation . . . 86
10.3.2.1 AbstractXSensorData() [1/2] . . . 86
10.3.2.2 ∼AbstractXSensorData() . . . 86
10.3.2.3 AbstractXSensorData() [2/2] . . . 86
10.3.3 Member Function Documentation . . . 86
10.3.3.1 CheckXMsgData() . . . 86
10.3.3.2 DecodeXMsgData() . . . 87
10.3.3.3 id_dive() . . . 88
10.3.3.4 id_instrument() . . . 89
10.3.3.5 id_xmsg_type() . . . 89
10.3.3.6 num_samples() . . . 90
10.3.3.7 raw_sensor_data() . . . 90
10.3.3.8 Sample() . . . 90
10.3.3.9 SampleValue() . . . 91
10.3.3.10 SampleValueAsPressure() . . . 91
10.3.3.11 SampleValueAsSalinity() . . . 92
10.3.3.12 SampleValueAsTemperature() . . . 92
10.3.3.13 sensor_data_format() . . . 93
10.3.3.14 set_id_dive() . . . 93
10.3.3.15 set_id_instrument() . . . 93

10.3.3.16 set_id_xmsg_type() . . . 94
10.3.3.17 set_raw_sensor_data() . . . 95
10.3.3.18 set_sensor_data_format() . . . 96
10.3.3.19 set_xmsg_index() . . . 96
10.3.3.20 set_xmsg_size_bytes() . . . 97
10.3.3.21 UnpackSensorData() . . . 98
10.3.3.22 xmsg_index() . . . 99
10.3.3.23 xmsg_size_bytes() . . . 100
10.3.4 Member Data Documentation . . . 101
10.3.4.1 id_dive_ . . . 101
10.3.4.2 id_instrument_ . . . 101
10.3.4.3 id_xmsg_type_ . . . 101
10.3.4.4 raw_sensor_data_ . . . 102
10.3.4.5 sensor_data_format_ . . . 102
10.3.4.6 xmsg_index_ . . . 102
10.3.4.7 xmsg_size_bytes_ . . . 102
10.4 util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_↩ MSGS > Class Template Reference . . . 103
10.4.1 Detailed Description . . . 105
10.4.2 Member Typedef Documentation . . . 105
10.4.2.1 PresMsgPtr_t . . . 105
10.4.2.2 PresMsgVector_t . . . 105
10.4.2.3 SalnMsgPtr_t . . . 105
10.4.2.4 SalnMsgVector_t . . . 106
10.4.2.5 TempMsgPtr_t . . . 106
10.4.2.6 TempMsgVector_t . . . 106
10.4.3 Constructor & Destructor Documentation . . . 106
10.4.3.1 AbstractXSensorDataComposite() . . . 106
10.4.3.2 ∼AbstractXSensorDataComposite() . . . 106
10.4.4 Member Function Documentation . . . 106
10.4.4.1 AggregateSensorData() . . . 106
10.4.4.2 AssignAuxTemperatureMessage() . . . 108
10.4.4.3 AssignPressureMessage() . . . 108
10.4.4.4 AssignSalinityMessage() . . . 109
10.4.4.5 AssignTemperatureMessage() . . . 110
10.4.4.6 GenerateListingString() . . . 111
10.4.4.7 HasContiguousMessageIndices() . . . 112
10.4.4.8 is_aggregated_data_valid() . . . 113
10.4.4.9 IsDataComplete() . . . 113
10.4.4.10 IsEmpty() . . . 114
10.4.4.11 LockObject() . . . 115
10.4.4.12 max_num_samples() . . . 115
10.4.4.13 num_pres_samples() . . . 115
10.4.4.14 num_saln_samples() . . . 116
10.4.4.15 num_temp_samples() . . . 117
10.4.4.16 Reset() . . . 118
10.4.4.17 SamplePres() . . . 119
10.4.4.18 SamplePresAsFloat() . . . 119
10.4.4.19 SampleSaln() . . . 120
10.4.4.20 SampleSalnAsFloat() . . . 121
10.4.4.21 SampleTemp() . . . 121
10.4.4.22 SampleTempAsFloat() . . . 122

10.4.4.23 set_aggregated_data_valid() . . . 122
10.4.4.24 UnlockObject() . . . 123
10.4.5 Member Data Documentation . . . 123
10.4.5.1 aggregated_data_valid_ . . . 123
10.4.5.2 aggregated_pres_data_ . . . 124
10.4.5.3 aggregated_saln_data_ . . . 124
10.4.5.4 aggregated_temp_data_ . . . 124
10.4.5.5 mutex_ . . . 125
10.4.5.6 pres_messages_ . . . 126
10.4.5.7 saln_messages_ . . . 126
10.4.5.8 temp_messages_ . . . 127
10.5 util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS > Class Template Reference . . . 127
10.5.1 Detailed Description . . . 129
10.5.2 Member Function Documentation . . . 130
10.5.2.1 AggregateSensorData() . . . 130
10.5.2.2 AssignMessage() . . . 130
10.5.2.3 is_aggregated_data_valid() . . . 131
10.5.2.4 IsEmpty() . . . 132
10.5.2.5 Reset() . . . 132
10.5.2.6 ResetAggregatedData() . . . 132
10.5.2.7 SamplePres() . . . 133
10.5.2.8 SamplePresAsFloat() . . . 133
10.5.2.9 SampleSaln() . . . 134
10.5.2.10 SampleSalnAsFloat() . . . 134
10.5.2.11 SampleTemp() . . . 135
10.5.2.12 SampleTempAsFloat() . . . 135
10.5.2.13 set_aggregated_data_valid() . . . 136
10.5.3 Member Data Documentation . . . 136
10.5.3.1 aggregated_data_valid_ . . . 136
10.6 util::xmsg::AuxiliarySensorData Class Reference . . . 137
10.6.1 Detailed Description . . . 138
10.6.2 Constructor & Destructor Documentation . . . 139
10.6.2.1 AuxiliarySensorData() . . . 139
10.6.2.2 ~AuxiliarySensorData() . . . 139
10.6.3 Member Function Documentation . . . 139
10.6.3.1 CheckXMsgData() . . . 139
10.6.3.2 DecodeXMsgData() . . . 140
10.6.3.3 id_dive() . . . 140
10.6.3.4 id_instrument() . . . 141
10.6.3.5 id_xmsg_type() . . . 142
10.6.3.6 num_samples() . . . 143
10.6.3.7 raw_sensor_data() . . . 143
10.6.3.8 Sample() . . . 143
10.6.3.9 SampleValue() . . . 143
10.6.3.10 SampleValueAsPressure() . . . 144
10.6.3.11 SampleValueAsSalinity() . . . 144
10.6.3.12 SampleValueAsTemperature() . . . 145
10.6.3.13 sensor_data_format() . . . 146
10.6.3.14 set_id_dive() . . . 146
10.6.3.15 set_id_instrument() . . . 146
10.6.3.16 set_id_xmsg_type() . . . 147
10.6.3.17 set_raw_sensor_data() . . . 148

10.6.3.18 set_sensor_data_format() . . . 148
10.6.3.19 set_xmsg_index() . . . 149
10.6.3.20 set_xmsg_size_bytes() . . . 149
10.6.3.21 UnpackSensorData() . . . 151
10.6.3.22 xmsg_index() . . . 152
10.6.3.23 xmsg_size_bytes() . . . 152
10.6.4 Member Data Documentation . . . 153
10.6.4.1 id_dive_ . . . 153
10.6.4.2 id_instrument_ . . . 153
10.6.4.3 id_xmsg_type_ . . . 153
10.6.4.4 raw_sensor_data_ . . . 154
10.6.4.5 sensor_data_format_ . . . 154
10.6.4.6 xmsg_index_ . . . 154
10.6.4.7 xmsg_size_bytes_ . . . 154
10.7 util::mysql::Column_bool< C > Class Template Reference . . . 154
10.7.1 Detailed Description . . . 155
10.7.2 Constructor & Destructor Documentation . . . 155
10.7.2.1 Column_bool() . . . 155
10.7.3 Member Function Documentation . . . 156
10.7.3.1 column_index() . . . 156
10.7.3.2 column_name() . . . 157
10.7.3.3 column_select_name() . . . 158
10.7.3.4 data_type() . . . 158
10.7.3.5 GetValueFromObject() . . . 158
10.7.3.6 is_primary_key() . . . 158
10.7.3.7 set_column_index() . . . 158
10.7.3.8 SetObjectValue() . . . 159
10.7.3.9 SetObjectValueFromResultSet() . . . 159
10.7.3.10 SetPreparedStatementValueFromObject() . . . 159
10.7.4 Member Data Documentation . . . 160
10.7.4.1 accessor_ . . . 160
10.7.4.2 column_index_ . . . 160
10.7.4.3 column_name_ . . . 160
10.7.4.4 data_type_ . . . 160
10.7.4.5 is_primary_key_ . . . 160
10.7.4.6 mutator_ . . . 161
10.8 util::mysql::Column_char< C > Class Template Reference . . . 161
10.8.1 Detailed Description . . . 162
10.8.2 Constructor & Destructor Documentation . . . 162
10.8.2.1 Column_char() . . . 162
10.8.3 Member Function Documentation . . . 163
10.8.3.1 column_index() . . . 163
10.8.3.2 column_name() . . . 164
10.8.3.3 column_select_name() . . . 164
10.8.3.4 data_type() . . . 164
10.8.3.5 GetValueFromObject() . . . 164
10.8.3.6 is_primary_key() . . . 165
10.8.3.7 set_column_index() . . . 165
10.8.3.8 SetObjectValue() . . . 165
10.8.3.9 SetObjectValueFromResultSet() . . . 165
10.8.3.10 SetPreparedStatementValueFromObject() . . . 166
10.8.4 Member Data Documentation . . . 166

10.8.4.1 accessor_ . . . 166
10.8.4.2 column_index_ . . . 166
10.8.4.3 column_name_ . . . 167
10.8.4.4 data_type_ . . . 167
10.8.4.5 is_primary_key_ . . . 167
10.8.4.6 mutator_ . . . 167
10.9 util::mysql::Column_double< C > Class Template Reference . . . 167
10.9.1 Detailed Description . . . 168
10.9.2 Constructor & Destructor Documentation . . . 168
10.9.2.1 Column_double() . . . 168
10.9.3 Member Function Documentation . . . 169
10.9.3.1 column_index() . . . 169
10.9.3.2 column_name() . . . 170
10.9.3.3 column_select_name() . . . 171
10.9.3.4 data_type() . . . 171
10.9.3.5 GetValueFromObject() . . . 171
10.9.3.6 is_primary_key() . . . 171
10.9.3.7 set_column_index() . . . 171
10.9.3.8 SetObjectValue() . . . 172
10.9.3.9 SetObjectValueFromResultSet() . . . 172
10.9.3.10 SetPreparedStatementValueFromObject() . . . 172
10.9.4 Member Data Documentation . . . 173
10.9.4.1 accessor_ . . . 173
10.9.4.2 column_index_ . . . 173
10.9.4.3 column_name_ . . . 173
10.9.4.4 data_type_ . . . 173
10.9.4.5 is_primary_key_ . . . 173
10.9.4.6 mutator_ . . . 174
10.10 util::mysql::Column_filesystem_path< C > Class Template Reference . . . 174
10.10.1 Detailed Description . . . 175
10.10.2 Constructor & Destructor Documentation . . . 175
10.10.2.1 Column_filesystem_path() [1/2] . . . 175
10.10.2.2 Column_filesystem_path() [2/2] . . . 175
10.10.3 Member Function Documentation . . . 176
10.10.3.1 column_index() . . . 176
10.10.3.2 column_name() . . . 177
10.10.3.3 column_select_name() . . . 178
10.10.3.4 data_type() . . . 178
10.10.3.5 GetValueFromObject() . . . 178
10.10.3.6 is_primary_key() . . . 178
10.10.3.7 set_column_index() . . . 178
10.10.3.8 SetObjectValue() . . . 179
10.10.3.9 SetObjectValueFromResultSet() . . . 179
10.10.3.10 SetPreparedStatementValueFromObject() . . . 179
10.10.4 Member Data Documentation . . . 180
10.10.4.1 accessor_ . . . 180
10.10.4.2 column_index_ . . . 180
10.10.4.3 column_name_ . . . 180
10.10.4.4 data_type_ . . . 180
10.10.4.5 is_primary_key_ . . . 180
10.10.4.6 mutator_ . . . 181
10.11 util::mysql::Column_float< C > Class Template Reference . . . 181

10.11.1 Detailed Description . . . 182
10.11.2 Constructor & Destructor Documentation . . . 182
10.11.2.1 Column_float() . . . 182
10.11.3 Member Function Documentation . . . 183
10.11.3.1 column_index() . . . 183
10.11.3.2 column_name() . . . 184
10.11.3.3 column_select_name() . . . 184
10.11.3.4 data_type() . . . 184
10.11.3.5 GetValueFromObject() . . . 184
10.11.3.6 is_primary_key() . . . 185
10.11.3.7 set_column_index() . . . 185
10.11.3.8 SetObjectValue() . . . 185
10.11.3.9 SetObjectValueFromResultSet() . . . 185
10.11.3.10 SetPreparedStatementValueFromObject() . . . 186
10.11.4 Member Data Documentation . . . 186
10.11.4.1 accessor_ . . . 186
10.11.4.2 column_index_ . . . 186
10.11.4.3 column_name_ . . . 187
10.11.4.4 data_type_ . . . 187
10.11.4.5 is_primary_key_ . . . 187
10.11.4.6 mutator_ . . . 187
10.12 util::mysql::Column_int< C > Class Template Reference . . . 187
10.12.1 Detailed Description . . . 188
10.12.2 Constructor & Destructor Documentation . . . 188
10.12.2.1 Column_int() . . . 188
10.12.3 Member Function Documentation . . . 189
10.12.3.1 column_index() . . . 189
10.12.3.2 column_name() . . . 190
10.12.3.3 column_select_name() . . . 191
10.12.3.4 data_type() . . . 191
10.12.3.5 GetValueFromObject() . . . 191
10.12.3.6 is_primary_key() . . . 191
10.12.3.7 set_column_index() . . . 191
10.12.3.8 SetObjectValue() . . . 192
10.12.3.9 SetObjectValueFromResultSet() . . . 192
10.12.3.10 SetPreparedStatementValueFromObject() . . . 192
10.12.4 Member Data Documentation . . . 193
10.12.4.1 accessor_ . . . 193
10.12.4.2 column_index_ . . . 193
10.12.4.3 column_name_ . . . 193
10.12.4.4 data_type_ . . . 193
10.12.4.5 is_primary_key_ . . . 193
10.12.4.6 mutator_ . . . 194
10.13 util::mysql::Column_int16_t< C > Class Template Reference . . . 194
10.13.1 Detailed Description . . . 195
10.13.2 Constructor & Destructor Documentation . . . 195
10.13.2.1 Column_int16_t() . . . 195
10.13.3 Member Function Documentation . . . 196
10.13.3.1 column_index() . . . 196
10.13.3.2 column_name() . . . 197
10.13.3.3 column_select_name() . . . 197
10.13.3.4 data_type() . . . 197

10.13.3.5 GetValueFromObject() . . . 197
10.13.3.6 is_primary_key() . . . 198
10.13.3.7 set_column_index() . . . 198
10.13.3.8 SetObjectValue() . . . 198
10.13.3.9 SetObjectValueFromResultSet() . . . 198
10.13.3.10 SetPreparedStatementValueFromObject() . . . 199
10.13.4 Member Data Documentation . . . 199
10.13.4.1 accessor_ . . . 199
10.13.4.2 column_index_ . . . 199
10.13.4.3 column_name_ . . . 200
10.13.4.4 data_type_ . . . 200
10.13.4.5 is_primary_key_ . . . 200
10.13.4.6 mutator_ . . . 200
10.14 util::mysql::Column_long< C > Class Template Reference . . . 200
10.14.1 Detailed Description . . . 201
10.14.2 Constructor & Destructor Documentation . . . 201
10.14.2.1 Column_long() . . . 201
10.14.3 Member Function Documentation . . . 202
10.14.3.1 column_index() . . . 202
10.14.3.2 column_name() . . . 203
10.14.3.3 column_select_name() . . . 204
10.14.3.4 data_type() . . . 204
10.14.3.5 GetValueFromObject() . . . 204
10.14.3.6 is_primary_key() . . . 204
10.14.3.7 set_column_index() . . . 204
10.14.3.8 SetObjectValue() . . . 205
10.14.3.9 SetObjectValueFromResultSet() . . . 205
10.14.3.10 SetPreparedStatementValueFromObject() . . . 205
10.14.4 Member Data Documentation . . . 206
10.14.4.1 accessor_ . . . 206
10.14.4.2 column_index_ . . . 206
10.14.4.3 column_name_ . . . 206
10.14.4.4 data_type_ . . . 206
10.14.4.5 is_primary_key_ . . . 206
10.14.4.6 mutator_ . . . 207
10.15 util::mysql::Column_posix_time< C > Class Template Reference . . . 207
10.15.1 Detailed Description . . . 208
10.15.2 Constructor & Destructor Documentation . . . 208
10.15.2.1 Column_posix_time() [1/2] . . . 208
10.15.2.2 Column_posix_time() [2/2] . . . 208
10.15.3 Member Function Documentation . . . 209
10.15.3.1 column_index() . . . 209
10.15.3.2 column_name() . . . 210
10.15.3.3 column_select_name() . . . 211
10.15.3.4 data_type() . . . 211
10.15.3.5 GetValueFromObject() . . . 211
10.15.3.6 is_primary_key() . . . 211
10.15.3.7 set_column_index() . . . 211
10.15.3.8 SetObjectValue() . . . 212
10.15.3.9 SetObjectValueFromResultSet() . . . 212
10.15.3.10 SetPreparedStatementValueFromObject() . . . 212
10.15.4 Member Data Documentation . . . 213

10.15.4.1 accessor_ . . . 213
10.15.4.2 column_index_ . . . 213
10.15.4.3 column_name_ . . . 213
10.15.4.4 data_type_ . . . 213
10.15.4.5 is_primary_key_ . . . 213
10.15.4.6 mutator_ . . . 214
10.16 util::mysql::Column_string< C > Class Template Reference . . . 214
10.16.1 Detailed Description . . . 215
10.16.2 Constructor & Destructor Documentation . . . 215
10.16.2.1 Column_string() [1/2] . . . 215
10.16.2.2 Column_string() [2/2] . . . 215
10.16.3 Member Function Documentation . . . 216
10.16.3.1 column_index() . . . 216
10.16.3.2 column_name() . . . 217
10.16.3.3 column_select_name() . . . 218
10.16.3.4 data_type() . . . 218
10.16.3.5 GetValueFromObject() . . . 218
10.16.3.6 is_primary_key() . . . 218
10.16.3.7 set_column_index() . . . 218
10.16.3.8 SetObjectValue() . . . 219
10.16.3.9 SetObjectValueFromResultSet() . . . 219
10.16.3.10 SetPreparedStatementValueFromObject() . . . 219
10.16.4 Member Data Documentation . . . 220
10.16.4.1 accessor_ . . . 220
10.16.4.2 column_index_ . . . 220
10.16.4.3 column_name_ . . . 220
10.16.4.4 data_type_ . . . 220
10.16.4.5 is_primary_key_ . . . 220
10.16.4.6 mutator_ . . . 221
10.17 util::mysql::Column_uint16_t< C > Class Template Reference . . . 221
10.17.1 Detailed Description . . . 222
10.17.2 Constructor & Destructor Documentation . . . 222
10.17.2.1 Column_uint16_t() . . . 222
10.17.3 Member Function Documentation . . . 223
10.17.3.1 column_index() . . . 223
10.17.3.2 column_name() . . . 224
10.17.3.3 column_select_name() . . . 224
10.17.3.4 data_type() . . . 224
10.17.3.5 GetValueFromObject() . . . 224
10.17.3.6 is_primary_key() . . . 225
10.17.3.7 set_column_index() . . . 225
10.17.3.8 SetObjectValue() . . . 225
10.17.3.9 SetObjectValueFromResultSet() . . . 225
10.17.3.10 SetPreparedStatementValueFromObject() . . . 226
10.17.4 Member Data Documentation . . . 226
10.17.4.1 accessor_ . . . 226
10.17.4.2 column_index_ . . . 226
10.17.4.3 column_name_ . . . 227
10.17.4.4 data_type_ . . . 227
10.17.4.5 is_primary_key_ . . . 227
10.17.4.6 mutator_ . . . 227
10.18 util::mysql::Column_uint8_t< C > Class Template Reference . . . 227

10.18.1 Detailed Description . . . 228
10.18.2 Constructor & Destructor Documentation . . . 228
10.18.2.1 Column_uint8_t() . . . 228
10.18.3 Member Function Documentation . . . 229
10.18.3.1 column_index() . . . 229
10.18.3.2 column_name() . . . 230
10.18.3.3 column_select_name() . . . 231
10.18.3.4 data_type() . . . 231
10.18.3.5 GetValueFromObject() . . . 231
10.18.3.6 is_primary_key() . . . 231
10.18.3.7 set_column_index() . . . 231
10.18.3.8 SetObjectValue() . . . 232
10.18.3.9 SetObjectValueFromResultSet() . . . 232
10.18.3.10 SetPreparedStatementValueFromObject() . . . 232
10.18.4 Member Data Documentation . . . 233
10.18.4.1 accessor_ . . . 233
10.18.4.2 column_index_ . . . 233
10.18.4.3 column_name_ . . . 233
10.18.4.4 data_type_ . . . 233
10.18.4.5 is_primary_key_ . . . 233
10.18.4.6 mutator_ . . . 234
10.19 util::mysql::Column_vector_uint8_t< C > Class Template Reference . . . 234
10.19.1 Detailed Description . . . 235
10.19.2 Constructor & Destructor Documentation . . . 235
10.19.2.1 Column_vector_uint8_t() . . . 235
10.19.3 Member Function Documentation . . . 236
10.19.3.1 column_index() . . . 236
10.19.3.2 column_name() . . . 237
10.19.3.3 column_select_name() . . . 237
10.19.3.4 data_type() . . . 237
10.19.3.5 GetValueFromObject() . . . 237
10.19.3.6 htoi() . . . 238
10.19.3.7 is_primary_key() . . . 238
10.19.3.8 set_column_index() . . . 238
10.19.3.9 SetObjectValue() . . . 238
10.19.3.10 SetObjectValueFromResultSet() . . . 239
10.19.3.11 SetPreparedStatementValueFromObject() . . . 239
10.19.4 Member Data Documentation . . . 240
10.19.4.1 accessor_ . . . 240
10.19.4.2 column_index_ . . . 240
10.19.4.3 column_name_ . . . 240
10.19.4.4 data_type_ . . . 240
10.19.4.5 is_primary_key_ . . . 240
10.19.4.6 mutator_ . . . 241
10.20 util::stream::DataBuf< CharT, TraitsT > Class Template Reference . . . 241
10.20.1 Detailed Description . . . 241
10.20.2 Constructor & Destructor Documentation . . . 242
10.20.2.1 DataBuf() . . . 242
10.21 util::date::DateUtils Class Reference . . . 243
10.21.1 Detailed Description . . . 243
10.21.2 Member Function Documentation . . . 244
10.21.2.1 GenerateCurrentUtcTimeIso8601String() . . . 244

10.21.2.2 GenerateIso8601String() . . . 244
10.21.2.3 GenerateJulianDay() . . . 245
10.21.2.4 GetCurrentUtcTime() . . . 246
10.22 util::file::FileSystemUtils Class Reference . . . 246
10.22.1 Detailed Description . . . 247
10.22.2 Member Function Documentation . . . 247
10.22.2.1 GetDirectoryPath() . . . 247
10.22.2.2 GetDirectoryPathFromString() . . . 247
10.22.2.3 ListDirectories() . . . 248
10.22.2.4 ListFiles() . . . 249
10.22.2.5 MoveFileToDirectory() . . . 250
10.23 boost::container::flat_map< C, D > Class Template Reference . . . 251
10.23.1 Detailed Description . . . 251
10.23.2 Member Data Documentation . . . 251
10.23.2.1 elements . . . 251
10.23.2.2 index . . . 252
10.24 argo::iridium::Iridium Class Reference . . . 252
10.24.1 Detailed Description . . . 253
10.24.2 Constructor & Destructor Documentation . . . 253
10.24.2.1 Iridium() [1/2] . . . 253
10.24.2.2 Iridium() [2/2] . . . 253
10.24.2.3 ~Iridium() . . . 254
10.24.3 Member Function Documentation . . . 254
10.24.3.1 id_dac() . . . 254
10.24.3.2 id_imei() . . . 254
10.24.3.3 id_instrument() . . . 255
10.24.3.4 id_wmo() . . . 255
10.24.3.5 LockObject() . . . 256
10.24.3.6 set_id_dac() . . . 256
10.24.3.7 set_id_imei() . . . 256
10.24.3.8 set_id_instrument() . . . 257
10.24.3.9 set_id_wmo() . . . 258
10.24.3.10 UnlockObject() . . . 258
10.24.4 Member Data Documentation . . . 258
10.24.4.1 id_dac_ . . . 258
10.24.4.2 id_imei_ . . . 258
10.24.4.3 id_instrument_ . . . 259
10.24.4.4 id_wmo_ . . . 259
10.24.4.5 mutex_ . . . 259
10.25 argo::iridium::IridiumPosition Class Reference . . . 260
10.25.1 Detailed Description . . . 263
10.25.2 Constructor & Destructor Documentation . . . 263
10.25.2.1 IridiumPosition() . . . 263
10.25.2.2 ~IridiumPosition() . . . 263
10.25.3 Member Function Documentation . . . 263
10.25.3.1 CalculateEllipsoidHeight() . . . 263
10.25.3.2 CalculateGeographicDistance() . . . 264
10.25.3.3 degToRad() . . . 265
10.25.3.4 GenerateCurrentUtcTimeIso8601String() . . . 265
10.25.3.5 GenerateIso6709String() . . . 266
10.25.3.6 GenerateIso8601String() . . . 266
10.25.3.7 GenerateJulianDay() . . . 267

10.25.3.8 GetCurrentUtcTime() . . . 268
10.25.3.9 iridium_position_as_iso6709_string() . . . 268
10.25.3.10 iridium_position_cep_radius() . . . 269
10.25.3.11 iridium_position_datetime() . . . 269
10.25.3.12 iridium_position_datetime_as_iso8601_string() . . . 270
10.25.3.13 iridium_position_is_valid() . . . 270
10.25.3.14 iridium_position_latitude() . . . 271
10.25.3.15 iridium_position_longitude() . . . 271
10.25.3.16 IsPositionValid() . . . 271
10.25.3.17 position_datetime() . . . 272
10.25.3.18 position_datetime_as_iso8601_string() . . . 272
10.25.3.19 position_is_valid() . . . 273
10.25.3.20 position_latitude() . . . 274
10.25.3.21 position_longitude() . . . 274
10.25.3.22 postition_as_iso6709_string() . . . 275
10.25.3.23 set_gps_position_datetime() . . . 275
10.25.3.24 set_iridium_position_cep_radius() . . . 276
10.25.3.25 set_iridium_position_datetime() . . . 276
10.25.3.26 set_iridium_position_is_valid() . . . 277
10.25.3.27 set_iridium_position_latitude() . . . 277
10.25.3.28 set_iridium_position_longitude() . . . 278
10.25.3.29 set_position_datetime() . . . 278
10.25.3.30 set_position_is_valid() . . . 278
10.25.3.31 set_position_latitude() . . . 279
10.25.3.32 set_position_longitude() . . . 280
10.25.4 Member Data Documentation . . . 281
10.25.4.1 EARTH_RADIUS_KM_EQUATORIAL . . . 281
10.25.4.2 EARTH_RADIUS_KM_POLAR . . . 281
10.25.4.3 iridium_position_cep_radius_ . . . 281
10.25.4.4 position_datetime_ . . . 282
10.25.4.5 position_is_valid_ . . . 282
10.25.4.6 position_latitude_ . . . 282
10.25.4.7 position_longitude_ . . . 282
10.26 util::kml::KmlUtil Class Reference . . . 282
10.26.1 Detailed Description . . . 283
10.26.2 Member Function Documentation . . . 284
10.26.2.1 CloseKmlFactory() . . . 284
10.26.2.2 GetDefaltKmlPathColor() . . . 284
10.26.2.3 GetDefaltKmlPathStyleId() . . . 284
10.26.2.4 GetDefaltKmlPlacemarkStyleId() . . . 285
10.26.2.5 GetKmlFactory() . . . 285
10.26.2.6 GetSmallKmlPlacemarkStyleId() . . . 285
10.26.2.7 GetTriangleKmlPlacemarkStyleId() . . . 286
10.26.2.8 SetAllDefaultKmlSyles() . . . 286
10.26.2.9 SetDefaultKmlStyle() . . . 287
10.26.3 Member Data Documentation . . . 288
10.26.3.1 kml_default_path_colors_ . . . 288
10.26.3.2 kml_default_path_style_ids_ . . . 288
10.26.3.3 kml_default_placemark_style_ids_ . . . 289
10.26.3.4 kml_factory_ . . . 289
10.26.3.5 kml_small_placemark_style_ids_ . . . 289
10.26.3.6 kml_triangle_placemark_style_ids_ . . . 289

10.27 util::config::LocalConfiguration Class Reference . . . 290
10.27.1 Detailed Description . . . 291
10.27.2 Constructor & Destructor Documentation . . . 291
10.27.2.1 LocalConfiguration() . . . 291
10.27.2.2 ~LocalConfiguration() . . . 291
10.27.3 Member Function Documentation . . . 291
10.27.3.1 AcquireRuntimeLock() . . . 291
10.27.3.2 CheckRuntimeLock() . . . 292
10.27.3.3 configuration() . . . 292
10.27.3.4 GenerateCommandLineOptionsDescription() . . . 293
10.27.3.5 GenerateConfigurationFileOptionsDescription() . . . 293
10.27.3.6 GetDirectoryPath() . . . 293
10.27.3.7 GetDirectoryPathFromString() . . . 294
10.27.3.8 InitializeConfiguration() . . . 295
10.27.3.9 is_daemon() . . . 296
10.27.3.10 ListDirectories() . . . 296
10.27.3.11 ListFiles() . . . 296
10.27.3.12 MoveFileToDirectory() . . . 297
10.27.3.13 ParseProgramOptions() . . . 297
10.27.3.14 ReleaseRuntimeLock() . . . 298
10.27.3.15 splash_lis() . . . 298
10.27.4 Member Data Documentation . . . 298
10.27.4.1 configuration_ . . . 298
10.27.4.2 configuration_file_path_ . . . 299
10.27.4.3 is_daemon_ . . . 299
10.27.4.4 pid_file_path_ . . . 299
10.27.4.5 SPLASH . . . 299
10.27.4.6 SPLASH_LIS . . . 300
10.28 util::mysql::LocalMysqlConfiguration Class Reference . . . 300
10.28.1 Detailed Description . . . 302
10.28.2 Constructor & Destructor Documentation . . . 302
10.28.2.1 LocalMysqlConfiguration() . . . 302
10.28.2.2 ~LocalMysqlConfiguration() . . . 302
10.28.3 Member Function Documentation . . . 302
10.28.3.1 CleanupSqlConnection() . . . 302
10.28.3.2 CleanupSqlDriver() . . . 302
10.28.3.3 CloseSqlConnection() . . . 303
10.28.3.4 ExecuteQuery() . . . 303
10.28.3.5 executeQuery() . . . 304
10.28.3.6 executeSelectQuery() . . . 305
10.28.3.7 ExecuteUpdate() . . . 305
10.28.3.8 executeUpdate() . . . 306
10.28.3.9 GeneratePreparedStatement() . . . 307
10.28.3.10 generatePreparedStatement() . . . 308
10.28.3.11 GenerateSqlDeleteStr() . . . 309
10.28.3.12 GenerateSqlSelectStr() . . . 309
10.28.3.13 GetSqlConnection() . . . 309
10.28.3.14 getSqlConnection() . . . 310
10.28.3.15 GetSqlDriver() . . . 311
10.28.4 Member Data Documentation . . . 311
10.28.4.1 address_ . . . 311
10.28.4.2 password_ . . . 312

10.28.4.3 port_ . . . 312
10.28.4.4 protocol_ . . . 312
10.28.4.5 schema_ . . . 312
10.28.4.6 sql_connection_ . . . 312
10.28.4.7 sql_driver_ . . . 312
10.28.4.8 user_ . . . 313
10.29 util::mysql::LocalPersistentEntity Class Reference . . . 313
10.29.1 Detailed Description . . . 315
10.29.2 Constructor & Destructor Documentation . . . 315
10.29.2.1 LocalPersistentEntity() . . . 315
10.29.2.2 ∼LocalPersistentEntity() . . . 315
10.29.3 Member Function Documentation . . . 315
10.29.3.1 CleanupSqlConnection() . . . 315
10.29.3.2 CleanupSqlDriver() . . . 315
10.29.3.3 CloseSqlConnection() . . . 315
10.29.3.4 ExecuteQuery() . . . 316
10.29.3.5 executeQuery() . . . 317
10.29.3.6 executeSelectQuery() . . . 317
10.29.3.7 ExecuteUpdate() . . . 318
10.29.3.8 executeUpdate() . . . 319
10.29.3.9 GeneratePreparedStatement() . . . 319
10.29.3.10 generatePreparedStatement() . . . 320
10.29.3.11 GenerateSqlDeleteStr() . . . 320
10.29.3.12 GenerateSqlSelectStr() . . . 321
10.29.3.13 GetSqlConnection() . . . 321
10.29.3.14 getSqlConnection() . . . 322
10.29.3.15 GetSqlDriver() . . . 323
10.29.3.16 SaveObjectToDatabase() . . . 323
10.29.4 Member Data Documentation . . . 324
10.29.4.1 address_ . . . 324
10.29.4.2 password_ . . . 324
10.29.4.3 port_ . . . 324
10.29.4.4 protocol_ . . . 324
10.29.4.5 schema_ . . . 324
10.29.4.6 sql_connection_ . . . 324
10.29.4.7 sql_driver_ . . . 325
10.29.4.8 user_ . . . 325
10.30 util::thread::Lockable Class Reference . . . 325
10.30.1 Detailed Description . . . 326
10.30.2 Constructor & Destructor Documentation . . . 326
10.30.2.1 Lockable() . . . 326
10.30.2.2 ∼Lockable() . . . 326
10.30.3 Member Function Documentation . . . 326
10.30.3.1 LockObject() . . . 326
10.30.3.2 UnlockObject() . . . 326
10.30.4 Member Data Documentation . . . 327
10.30.4.1 mutex_ . . . 327
10.31 util::log::Log Class Reference . . . 327
10.31.1 Detailed Description . . . 328
10.31.2 Constructor & Destructor Documentation . . . 328
10.31.2.1 Log() . . . 328
10.31.2.2 ∼Log() . . . 328

10.31.3 Member Function Documentation . . . 329
10.31.3.1 Flush() [1/2] . . . 329
10.31.3.2 Flush() [2/2] . . . 329
10.31.3.3 FlushGlobalLogger() . . . 330
10.31.3.4 InitializeLog() . . . 330
10.31.3.5 log_reporting_level() . . . 330
10.31.3.6 operator<<() . . . 330
10.31.3.7 set_log_reporting_level() . . . 331
10.31.4 Member Data Documentation . . . 331
10.31.4.1 global_logger_ . . . 331
10.31.4.2 log_directory_path_ . . . 331
10.31.4.3 log_msg_severity_level_ . . . 331
10.31.4.4 log_msg_stream_ . . . 332
10.31.4.5 log_reporting_level_ . . . 332
10.32 util::math::MathUtils Class Reference . . . 332
10.32.1 Detailed Description . . . 332
10.32.2 Member Function Documentation . . . 332
10.32.2.1 degToRad() . . . 332
10.33 util::log::MessageException Class Reference . . . 333
10.33.1 Detailed Description . . . 334
10.33.2 Constructor & Destructor Documentation . . . 334
10.33.2.1 MessageException() [1/3] . . . 334
10.33.2.2 MessageException() [2/3] . . . 334
10.33.2.3 MessageException() [3/3] . . . 334
10.33.2.4 ∼MessageException() . . . 334
10.33.3 Member Function Documentation . . . 335
10.33.3.1 what() . . . 335
10.33.4 Member Data Documentation . . . 335
10.33.4.1 message_ . . . 335
10.34 util::mysql::MysqlUtils Class Reference . . . 335
10.34.1 Detailed Description . . . 336
10.34.2 Member Function Documentation . . . 336
10.34.2.1 CleanupSqlConnection() . . . 336
10.34.2.2 CleanupSqlDriver() . . . 336
10.34.2.3 CloseSqlConnection() . . . 337
10.34.2.4 ExecuteQuery() . . . 337
10.34.2.5 ExecuteUpdate() . . . 338
10.34.2.6 GeneratePreparedStatement() . . . 339
10.34.2.7 GenerateSqlDeleteStr() . . . 339
10.34.2.8 GenerateSqlSelectStr() . . . 340
10.34.2.9 GetSqlConnection() . . . 340
10.34.2.10 GetSqlDriver() . . . 341
10.34.3 Member Data Documentation . . . 342
10.34.3.1 sql_connection_ . . . 342
10.34.3.2 sql_driver_ . . . 342
10.35 util::position::Position Class Reference . . . 342
10.35.1 Detailed Description . . . 344
10.35.2 Constructor & Destructor Documentation . . . 344
10.35.2.1 Position() . . . 344
10.35.2.2 ∼Position() . . . 344
10.35.3 Member Function Documentation . . . 344
10.35.3.1 CalculateEllipsoidHeight() . . . 344

10.35.3.2 CalculateGeographicDistance() . . . 345
10.35.3.3 degToRad() . . . 346
10.35.3.4 GenerateCurrentUtcTimeIso8601String() . . . 346
10.35.3.5 GenerateIso6709String() . . . 347
10.35.3.6 GenerateIso8601String() . . . 347
10.35.3.7 GenerateJulianDay() . . . 348
10.35.3.8 GetCurrentUtcTime() . . . 349
10.35.3.9 IsPositionValid() . . . 349
10.35.3.10 position_datetime() . . . 350
10.35.3.11 position_datetime_as_iso8601_string() . . . 350
10.35.3.12 position_is_valid() . . . 351
10.35.3.13 position_latitude() . . . 352
10.35.3.14 position_longitude() . . . 352
10.35.3.15 postition_as_iso6709_string() . . . 353
10.35.3.16 set_gps_position_datetime() . . . 353
10.35.3.17 set_position_datetime() . . . 354
10.35.3.18 set_position_is_valid() . . . 354
10.35.3.19 set_position_latitude() . . . 355
10.35.3.20 set_position_longitude() . . . 356
10.35.4 Member Data Documentation . . . 357
10.35.4.1 EARTH_RADIUS_KM_EQUATORIAL . . . 357
10.35.4.2 EARTH_RADIUS_KM_POLAR . . . 357
10.35.4.3 position_datetime_ . . . 357
10.35.4.4 position_is_valid_ . . . 357
10.35.4.5 position_latitude_ . . . 357
10.35.4.6 position_longitude_ . . . 358
10.36 util::position::PositionUtils Class Reference . . . 358
10.36.1 Detailed Description . . . 359
10.36.2 Member Function Documentation . . . 359
10.36.2.1 CalculateEllipsoidHeight() . . . 359
10.36.2.2 CalculateGeographicDistance() . . . 360
10.36.2.3 degToRad() . . . 360
10.36.2.4 GenerateCurrentUtcTimeIso8601String() . . . 361
10.36.2.5 GenerateIso6709String() . . . 361
10.36.2.6 GenerateIso8601String() . . . 362
10.36.2.7 GenerateJulianDay() . . . 363
10.36.2.8 GetCurrentUtcTime() . . . 364
10.36.2.9 IsPositionValid() . . . 364
10.36.3 Member Data Documentation . . . 364
10.36.3.1 EARTH_RADIUS_KM_EQUATORIAL . . . 364
10.36.3.2 EARTH_RADIUS_KM_POLAR . . . 365
10.37 util::thread::RecursiveLockable Class Reference . . . 365
10.37.1 Detailed Description . . . 366
10.37.2 Constructor & Destructor Documentation . . . 366
10.37.2.1 RecursiveLockable() . . . 366
10.37.2.2 ∼RecursiveLockable() . . . 366
10.37.3 Member Function Documentation . . . 366
10.37.3.1 LockObject() . . . 366
10.37.3.2 UnlockObject() . . . 366
10.37.4 Member Data Documentation . . . 367
10.37.4.1 mutex_ . . . 367
10.38 boost::shared_ptr< A > Class Template Reference . . . 368

10.38.1 Detailed Description . . . 369
10.38.2 Member Data Documentation . . . 369
10.38.2.1 element . . . 369
10.39 vmime::shared_ptr< F > Class Template Reference . . . 370
10.39.1 Detailed Description . . . 370
10.39.2 Member Data Documentation . . . 370
10.39.2.1 element . . . 370
10.40 util::sig::SignalHandler Class Reference . . . 371
10.40.1 Detailed Description . . . 371
10.40.2 Member Function Documentation . . . 371
10.40.2.1 ExitHandler() . . . 371
10.40.2.2 RegisterSignalHandlers() . . . 371
10.41 util::mysql::SqlTableColumnDescription< C, T > Class Template Reference . . . 372
10.41.1 Detailed Description . . . 373
10.41.2 Constructor & Destructor Documentation . . . 373
10.41.2.1 SqlTableColumnDescription() [1/3] . . . 373
10.41.2.2 SqlTableColumnDescription() [2/3] . . . 374
10.41.2.3 SqlTableColumnDescription() [3/3] . . . 374
10.41.3 Member Function Documentation . . . 374
10.41.3.1 column_index() . . . 374
10.41.3.2 column_name() . . . 375
10.41.3.3 column_select_name() . . . 376
10.41.3.4 data_type() . . . 376
10.41.3.5 GetValueFromObject() . . . 376
10.41.3.6 is_primary_key() . . . 376
10.41.3.7 set_column_index() . . . 376
10.41.3.8 SetObjectValue() . . . 377
10.41.4 Member Data Documentation . . . 377
10.41.4.1 accessor_ . . . 377
10.41.4.2 column_index_ . . . 377
10.41.4.3 column_name_ . . . 377
10.41.4.4 data_type_ . . . 377
10.41.4.5 is_primary_key_ . . . 378
10.41.4.6 mutator_ . . . 378
10.42 util::mysql::SqlTableColumnDescriptionBase Class Reference . . . 378
10.42.1 Detailed Description . . . 379
10.42.2 Constructor & Destructor Documentation . . . 379
10.42.2.1 SqlTableColumnDescriptionBase() . . . 379
10.42.2.2 ~SqlTableColumnDescriptionBase() . . . 379
10.42.3 Member Function Documentation . . . 380
10.42.3.1 column_index() . . . 380
10.42.3.2 column_name() . . . 381
10.42.3.3 column_select_name() . . . 381
10.42.3.4 data_type() . . . 381
10.42.3.5 is_primary_key() . . . 381
10.42.3.6 set_column_index() . . . 381
10.42.4 Member Data Documentation . . . 382
10.42.4.1 column_index_ . . . 382
10.42.4.2 column_name_ . . . 382
10.42.4.3 data_type_ . . . 382
10.42.4.4 is_primary_key_ . . . 382
10.43 util::mysql::SqlTableDescription< T > Class Template Reference . . . 382

10.43.1 Detailed Description . . . 385
10.43.2 Member Typedef Documentation . . . 385
10.43.2.1 cbol . . . 385
10.43.2.2 cchar . . . 385
10.43.2.3 cdbl . . . 385
10.43.2.4 cflt . . . 385
10.43.2.5 ci16 . . . 386
10.43.2.6 cint . . . 386
10.43.2.7 clng . . . 386
10.43.2.8 cpth . . . 386
10.43.2.9 cptm . . . 386
10.43.2.10 cstr . . . 386
10.43.2.11 cu16 . . . 386
10.43.2.12 cu8 . . . 386
10.43.2.13 cvu8 . . . 386
10.43.3 Constructor & Destructor Documentation . . . 386
10.43.3.1 SqlTableDescription() . . . 386
10.43.3.2 ~SqlTableDescription() . . . 387
10.43.4 Member Function Documentation . . . 387
10.43.4.1 CleanupSqlConnection() . . . 387
10.43.4.2 CleanupSqlDriver() . . . 387
10.43.4.3 CloseSqlConnection() . . . 388
10.43.4.4 ExecuteQuery() . . . 388
10.43.4.5 executeQuery() . . . 389
10.43.4.6 executeSelectQuery() . . . 390
10.43.4.7 ExecuteUpdate() . . . 391
10.43.4.8 executeUpdate() . . . 391
10.43.4.9 generateInsertStatment() . . . 392
10.43.4.10 GenerateObjectList() . . . 392
10.43.4.11 GeneratePreparedStatement() . . . 393
10.43.4.12 generatePreparedStatement() . . . 394
10.43.4.13 generateSelectAllStatement() . . . 395
10.43.4.14 GenerateSqlDeleteStr() . . . 396
10.43.4.15 GenerateSqlSelectStr() . . . 396
10.43.4.16 getColumnNames() . . . 397
10.43.4.17 getColumns() . . . 397
10.43.4.18 getColumnSelectNames() . . . 397
10.43.4.19 GetSqlConnection() . . . 398
10.43.4.20 getSqlConnection() . . . 398
10.43.4.21 GetSqlDriver() . . . 399
10.43.4.22 htoi() . . . 399
10.43.4.23 SaveObjectToDatabase() . . . 400
10.43.4.24 SelectAllObjectsFromDatabase() . . . 400
10.43.4.25 SelectObjectFromDatabase() . . . 401
10.43.4.26 SelectObjectsFromDatabase() . . . 402
10.43.4.27 SelectValueFromDatabase() . . . 403
10.43.4.28 values() . . . 404
10.43.5 Member Data Documentation . . . 404
10.43.5.1 address_ . . . 404
10.43.5.2 insert_on_duplicate_key_update_statement_ . . . 405
10.43.5.3 password_ . . . 405
10.43.5.4 port_ . . . 405

10.43.5.5 protocol_ . . . 405
10.43.5.6 schema_ . . . 405
10.43.5.7 select_statement_ . . . 405
10.43.5.8 sql_connection_ . . . 405
10.43.5.9 sql_driver_ . . . 406
10.43.5.10 table_columns_ . . . 406
10.43.5.11 table_name_ . . . 406
10.43.5.12 user_ . . . 406
10.44 boost::container::static_vector< E > Class Template Reference . . . 407
10.44.1 Detailed Description . . . 407
10.44.2 Member Data Documentation . . . 408
10.44.2.1 elements . . . 408
10.45 util::stream::StreamUtils Class Reference . . . 408
10.45.1 Detailed Description . . . 408
10.45.2 Member Function Documentation . . . 408
10.45.2.1 htoi() . . . 408
10.46 util::thread::SynchronousQueue< T > Class Template Reference . . . 409
10.46.1 Detailed Description . . . 410
10.46.2 Constructor & Destructor Documentation . . . 410
10.46.2.1 SynchronousQueue() . . . 410
10.46.3 Member Function Documentation . . . 410
10.46.3.1 empty() . . . 410
10.46.3.2 full() . . . 410
10.46.3.3 pop() . . . 410
10.46.3.4 push() [1/2] . . . 411
10.46.3.5 push() [2/2] . . . 411
10.46.3.6 size() . . . 411
10.46.4 Member Data Documentation . . . 412
10.46.4.1 buffer . . . 412
10.46.4.2 bufferNotEmpty . . . 412
10.46.4.3 bufferNotFull . . . 412
10.46.4.4 mutex . . . 412
10.47 boost::thread_specific_ptr< B > Class Template Reference . . . 412
10.47.1 Detailed Description . . . 413
10.47.2 Member Data Documentation . . . 413
10.47.2.1 element . . . 413
10.48 util::thread::ThreadUtils Class Reference . . . 413
10.48.1 Detailed Description . . . 414
10.48.2 Member Function Documentation . . . 414
10.48.2.1 executeAndPipe() [1/2] . . . 414
10.48.2.2 executeAndPipe() [2/2] . . . 414
10.49 util::stream::VectorBuf< CharT, TraitsT > Class Template Reference . . . 415
10.49.1 Detailed Description . . . 416
10.49.2 Constructor & Destructor Documentation . . . 416
10.49.2.1 VectorBuf() . . . 416
10.50 util::stream::VectorBuf_c< CharT > Class Template Reference . . . 417
10.50.1 Detailed Description . . . 417
10.50.2 Constructor & Destructor Documentation . . . 417
10.50.2.1 VectorBuf_c() . . . 417
10.51 util::xmsg::XAcousticCacst Class Reference . . . 418
10.51.1 Detailed Description . . . 419
10.51.2 Constructor & Destructor Documentation . . . 419

10.51.2.1 XAcousticCacst() [1/2] . . . 419
10.51.2.2 XAcousticCacst() [2/2] . . . 419
10.51.2.3 ~XAcousticCacst() . . . 419
10.51.3 Member Function Documentation . . . 419
10.51.3.1 DecodeXMsgData() . . . 419
10.51.4 Member Data Documentation . . . 420
10.51.4.1 mfd_power_ . . . 420
10.51.4.2 psk_error_code_ . . . 420
10.51.4.3 snr_in_ . . . 420
10.51.4.4 snr_out_ . . . 420
10.51.4.5 src_ . . . 420
10.51.4.6 toa_sec_ . . . 420
10.51.4.7 toa_usec_ . . . 421
10.52 util::xmsg::XAcousticPosition Class Reference . . . 421
10.52.1 Detailed Description . . . 422
10.52.2 Constructor & Destructor Documentation . . . 422
10.52.2.1 XAcousticPosition() . . . 422
10.52.2.2 ~XAcousticPosition() . . . 423
10.52.3 Member Function Documentation . . . 423
10.52.3.1 AssignWindowIndices() . . . 423
10.52.3.2 CheckXMsgData() . . . 423
10.52.3.3 DecodeXMsgData() . . . 424
10.52.3.4 GenerateListingString() . . . 425
10.52.3.5 id_dive() . . . 425
10.52.3.6 id_instrument() . . . 426
10.52.3.7 id_xmsg_type() . . . 427
10.52.3.8 set_id_dive() . . . 428
10.52.3.9 set_id_instrument() . . . 428
10.52.3.10 set_id_xmsg_type() . . . 428
10.52.3.11 set_xmsg_size_bytes() . . . 429
10.52.3.12 xmsg_size_bytes() . . . 431
10.52.4 Member Data Documentation . . . 431
10.52.4.1 id_dive_ . . . 431
10.52.4.2 id_instrument_ . . . 432
10.52.4.3 id_xmsg_type_ . . . 432
10.52.4.4 xmsg_size_bytes_ . . . 432
10.53 util::xmsg::XAcousticWindow Class Reference . . . 432
10.53.1 Detailed Description . . . 433
10.53.2 Constructor & Destructor Documentation . . . 434
10.53.2.1 XAcousticWindow() [1/2] . . . 434
10.53.2.2 XAcousticWindow() [2/2] . . . 434
10.53.2.3 ~XAcousticWindow() . . . 434
10.53.3 Member Function Documentation . . . 434
10.53.3.1 DecodeXMsgData() . . . 434
10.53.4 Member Data Documentation . . . 435
10.53.4.1 p_end_ . . . 435
10.53.4.2 p_start_ . . . 435
10.53.4.3 ts_end_ . . . 435
10.54 util::xmsg::XAlamoBit Class Reference . . . 435
10.54.1 Detailed Description . . . 438
10.54.2 Constructor & Destructor Documentation . . . 438
10.54.2.1 XAlamoBit() . . . 438

10.54.2.2 ~XAlamoBit() . . . 438
10.54.3 Member Function Documentation . . . 439
10.54.3.1 bit_status_code() . . . 439
10.54.3.2 bit_version() . . . 439
10.54.3.3 DecodeXMsgData() . . . 439
10.54.3.4 float_software_version() . . . 440
10.54.3.5 GenerateListingString() . . . 440
10.54.3.6 set_bit_status_code() . . . 441
10.54.3.7 set_bit_version() . . . 442
10.54.3.8 set_float_software_version() . . . 442
10.54.4 Member Data Documentation . . . 443
10.54.4.1 bit_status_code_ . . . 443
10.54.4.2 bit_version_ . . . 443
10.54.4.3 float_software_version_ . . . 443
10.54.4.4 pres_offset_ . . . 443
10.55 util::xmsg::XAlamoBit2 Class Reference . . . 443
10.55.1 Detailed Description . . . 447
10.55.2 Constructor & Destructor Documentation . . . 447
10.55.2.1 XAlamoBit2() . . . 447
10.55.2.2 ~XAlamoBit2() . . . 448
10.55.3 Member Function Documentation . . . 448
10.55.3.1 bit_version() . . . 448
10.55.3.2 DecodeXMsgData() . . . 448
10.55.3.3 float_software_version() . . . 449
10.55.3.4 GenerateCurrentUtcTimeIso8601String() . . . 449
10.55.3.5 GenerateIso8601String() . . . 450
10.55.3.6 GenerateJulianDay() . . . 451
10.55.3.7 GenerateListingString() . . . 451
10.55.3.8 GetCurrentUtcTime() . . . 452
10.55.3.9 set_bit_passed() . . . 452
10.55.3.10 set_bit_status_code() . . . 452
10.55.3.11 set_bit_version() . . . 453
10.55.3.12 set_float_software_version() . . . 453
10.55.4 Member Data Documentation . . . 454
10.55.4.1 air_pump_curr_max_ . . . 454
10.55.4.2 air_pump_cycle_count_ . . . 454
10.55.4.3 air_pump_errors_ . . . 454
10.55.4.4 air_pump_test_duration_ . . . 454
10.55.4.5 air_pump_v12_curr_max_ . . . 454
10.55.4.6 air_pump_volt_max_ . . . 454
10.55.4.7 air_valve_cal_open_ . . . 454
10.55.4.8 air_valve_errors_ . . . 454
10.55.4.9 air_valve_open_ . . . 455
10.55.4.10 bat_volt_start_ . . . 455
10.55.4.11 bit_passed_ . . . 455
10.55.4.12 bit_status_code_ . . . 455
10.55.4.13 bit_version_ . . . 455
10.55.4.14 case_vac_start_ . . . 455
10.55.4.15 dt_end_ . . . 455
10.55.4.16 dt_end_epoch_ . . . 455
10.55.4.17 dt_start_ . . . 455
10.55.4.18 dt_start_epoch_ . . . 456

10.55.4.19 duration_sec_ . . . 456
10.55.4.20 float_software_version_ . . . 456
10.55.4.21 gps_errors_ . . . 456
10.55.4.22 oil_pump_curr_max_ . . . 456
10.55.4.23 oil_pump_errors_ . . . 456
10.55.4.24 oil_pump_test_duration_ . . . 456
10.55.4.25 oil_pump_v12_curr_max_ . . . 456
10.55.4.26 oil_pump_volt_max_ . . . 456
10.55.4.27 oil_valve_cal_open_ . . . 457
10.55.4.28 oil_valve_errors_ . . . 457
10.55.4.29 oil_valve_open_ . . . 457
10.55.4.30 sensor_type_ . . . 457
10.55.4.31 type_ . . . 457
10.56 util::xmsg::XAlamoEngineering Class Reference . . . 457
10.56.1 Detailed Description . . . 459
10.56.2 Constructor & Destructor Documentation . . . 459
10.56.2.1 XAlamoEngineering() . . . 459
10.56.2.2 ~XAlamoEngineering() . . . 459
10.56.3 Member Function Documentation . . . 459
10.56.3.1 CheckXMsgData() . . . 459
10.56.3.2 DecodeXMsgData() . . . 460
10.56.3.3 engineering_version() . . . 461
10.56.3.4 engineering_version_as_float() . . . 461
10.56.3.5 GenerateCurrentUtcTimeIso8601String() . . . 461
10.56.3.6 GenerateIso8601String() . . . 462
10.56.3.7 GenerateJulianDay() . . . 463
10.56.3.8 GetCurrentUtcTime() . . . 464
10.56.3.9 id_dive() . . . 464
10.56.3.10 id_instrument() . . . 465
10.56.3.11 id_xmsg_type() . . . 466
10.56.3.12 set_engineering_version() . . . 467
10.56.3.13 set_id_dive() . . . 467
10.56.3.14 set_id_instrument() . . . 467
10.56.3.15 set_id_xmsg_type() . . . 468
10.56.3.16 set_start_time() . . . 469
10.56.3.17 set_xmsg_size_bytes() . . . 470
10.56.3.18 start_time() . . . 471
10.56.3.19 xmsg_size_bytes() . . . 472
10.56.4 Member Data Documentation . . . 473
10.56.4.1 engineering_version_ . . . 473
10.56.4.2 id_dive_ . . . 473
10.56.4.3 id_instrument_ . . . 473
10.56.4.4 id_xmsg_type_ . . . 474
10.56.4.5 start_time_ . . . 474
10.56.4.6 xmsg_size_bytes_ . . . 474
10.57 util::xmsg::XAlamoMissionConfiguration Class Reference . . . 474
10.57.1 Detailed Description . . . 484
10.57.2 Constructor & Destructor Documentation . . . 484
10.57.2.1 XAlamoMissionConfiguration() [1/2] . . . 484
10.57.2.2 XAlamoMissionConfiguration() [2/2] . . . 484
10.57.2.3 ~XAlamoMissionConfiguration() . . . 485
10.57.3 Member Function Documentation . . . 485

10.57.3.1 DecodeXMsgData() . . . 485
10.57.3.2 GenerateCurrentUtcTimeIso8601String() . . . 486
10.57.3.3 GenerateIso8601String() . . . 486
10.57.3.4 GenerateJulianDay() . . . 487
10.57.3.5 GenerateListingString() . . . 488
10.57.3.6 GetCurrentUtcTime() . . . 488
10.57.4 Member Data Documentation . . . 489
10.57.4.1 add_ . . . 489
10.57.4.2 adt_ . . . 489
10.57.4.3 apf_ . . . 489
10.57.4.4 apn_ . . . 489
10.57.4.5 apt_ . . . 489
10.57.4.6 arr_ . . . 489
10.57.4.7 ascent_start_time_ . . . 489
10.57.4.8 asd_ . . . 489
10.57.4.9 asi_ . . . 489
10.57.4.10 asn_ . . . 490
10.57.4.11 btm_ . . . 490
10.57.4.12 cpb_ . . . 490
10.57.4.13 cpn_ . . . 490
10.57.4.14 cpp_ . . . 490
10.57.4.15 cps_ . . . 490
10.57.4.16 csp_ . . . 490
10.57.4.17 cst_ . . . 490
10.57.4.18 ctf_ . . . 491
10.57.4.19 ctl_ . . . 491
10.57.4.20 ctu_ . . . 491
10.57.4.21 ddd_ . . . 491
10.57.4.22 ddt_ . . . 491
10.57.4.23 descend_start_time_ . . . 491
10.57.4.24 dft_ . . . 492
10.57.4.25 dns_ . . . 492
10.57.4.26 dsd_ . . . 492
10.57.4.27 dsi_ . . . 492
10.57.4.28 dst_ . . . 492
10.57.4.29 dtd_ . . . 492
10.57.4.30 fw_version_ . . . 492
10.57.4.31 mam_ . . . 492
10.57.4.32 mat_ . . . 492
10.57.4.33 mds_ . . . 493
10.57.4.34 msg_version_ . . . 493
10.57.4.35 msn_ . . . 493
10.57.4.36 mtd_ . . . 493
10.57.4.37 mtt_ . . . 493
10.57.4.38 park_start_time_ . . . 493
10.57.4.39 pdt_ . . . 493
10.57.4.40 pfd_ . . . 493
10.57.4.41 pfm_ . . . 493
10.57.4.42 pns_ . . . 494
10.57.4.43 psa_ . . . 494
10.57.4.44 psi_ . . . 494
10.57.4.45 pth_ . . . 494

10.57.4.46 pto_ . . . 494
10.57.4.47 sbm_ . . . 494
10.57.4.48 sbt_ . . . 494
10.57.4.49 sdc_ . . . 494
10.57.4.50 sds_ . . . 494
10.57.4.51 sensor_type_ . . . 495
10.57.4.52 set_ . . . 495
10.57.4.53 sfn_ . . . 495
10.57.4.54 sgt_ . . . 495
10.57.4.55 ssi_ . . . 495
10.57.4.56 surface_start_time_ . . . 495
10.57.4.57 xas_ . . . 495
10.58 util::xmsg::XArgo Class Reference . . . 496
10.58.1 Detailed Description . . . 500
10.58.2 Constructor & Destructor Documentation . . . 500
10.58.2.1 XArgo() . . . 500
10.58.2.2 ∼XArgo() . . . 501
10.58.3 Member Function Documentation . . . 501
10.58.3.1 argo_instrument_type() . . . 501
10.58.3.2 ascent_state_start_time() . . . 502
10.58.3.3 CheckXMsgData() . . . 502
10.58.3.4 DecodeXMsgData() . . . 503
10.58.3.5 descent_state_start_time() . . . 504
10.58.3.6 drift_state_start_time() . . . 504
10.58.3.7 gain_pressure() . . . 505
10.58.3.8 gain_pressure_surface() . . . 505
10.58.3.9 gain_salinity() . . . 505
10.58.3.10 gain_salinity_surface() . . . 506
10.58.3.11 gain_temperature() . . . 506
10.58.3.12 gain_temperature_surface() . . . 506
10.58.3.13 GenerateCurrentUtcTimeIso8601String() . . . 507
10.58.3.14 GenerateIso8601String() . . . 507
10.58.3.15 GenerateJulianDay() . . . 508
10.58.3.16 GenerateListingString() . . . 509
10.58.3.17 GetCurrentUtcTime() . . . 509
10.58.3.18 id_dive() . . . 510
10.58.3.19 id_instrument() . . . 511
10.58.3.20 id_xmsg_type() . . . 511
10.58.3.21 max_ascent_duration() . . . 512
10.58.3.22 max_fall_to_drift_duration() . . . 512
10.58.3.23 max_fall_to_profle_duration() . . . 513
10.58.3.24 max_surface_duration() . . . 513
10.58.3.25 mission_phase() . . . 513
10.58.3.26 num_seeks_to_drift_pressure() . . . 514
10.58.3.27 offset_pressure() . . . 514
10.58.3.28 offset_pressure_surface() . . . 514
10.58.3.29 offset_salinity() . . . 515
10.58.3.30 offset_salinity_surface() . . . 515
10.58.3.31 offset_temperature() . . . 515
10.58.3.32 offset_temperature_surface() . . . 516
10.58.3.33 seek_to_drift_pressure_interval_duration() . . . 516
10.58.3.34 set_argo_instrument_type() . . . 516

10.58.3.35 set_ascent_state_start_time() . . . 517
10.58.3.36 set_descent_state_start_time() . . . 517
10.58.3.37 set_drift_state_start_time() . . . 517
10.58.3.38 set_gain_pressure() . . . 518
10.58.3.39 set_gain_pressure_surface() . . . 518
10.58.3.40 set_gain_salinity() . . . 518
10.58.3.41 set_gain_salinity_surface() . . . 519
10.58.3.42 set_gain_temperature() . . . 519
10.58.3.43 set_gain_temperature_surface() . . . 519
10.58.3.44 set_id_dive() . . . 520
10.58.3.45 set_id_instrument() . . . 520
10.58.3.46 set_id_xmsg_type() . . . 520
10.58.3.47 set_max_ascent_duration() . . . 521
10.58.3.48 set_max_fall_to_drift_duration() . . . 522
10.58.3.49 set_max_fall_to_profle_duration() . . . 522
10.58.3.50 set_max_surface_duration() . . . 522
10.58.3.51 set_mission_phase() . . . 523
10.58.3.52 set_num_seeks_to_drift_pressure() . . . 523
10.58.3.53 set_offset_pressure() . . . 523
10.58.3.54 set_offset_pressure_surface() . . . 524
10.58.3.55 set_offset_salinity() . . . 524
10.58.3.56 set_offset_salinity_surface() . . . 524
10.58.3.57 set_offset_temperature() . . . 525
10.58.3.58 set_offset_temperature_surface() . . . 525
10.58.3.59 set_seek_to_drift_pressure_interval_duration() . . . 525
10.58.3.60 set_surface_state_start_time() . . . 526
10.58.3.61 set_target_ascent_rate() . . . 526
10.58.3.62 set_target_drift_duration() . . . 526
10.58.3.63 set_target_drift_pressure() . . . 527
10.58.3.64 set_target_profile_pressure() . . . 527
10.58.3.65 set_x_version_major() . . . 527
10.58.3.66 set_x_version_minor() . . . 528
10.58.3.67 set_xmsg_size_bytes() . . . 528
10.58.3.68 surface_state_start_time() . . . 529
10.58.3.69 target_ascent_rate() . . . 530
10.58.3.70 target_drift_duration() . . . 530
10.58.3.71 target_drift_pressure() . . . 530
10.58.3.72 target_profile_pressure() . . . 531
10.58.3.73 x_version_float() . . . 531
10.58.3.74 x_version_major() . . . 532
10.58.3.75 x_version_minor() . . . 532
10.58.3.76 xmsg_size_bytes() . . . 532
10.58.4 Member Data Documentation . . . 533
10.58.4.1 argo_instrument_type_ . . . 533
10.58.4.2 ascent_state_start_time_ . . . 533
10.58.4.3 descent_state_start_time_ . . . 534
10.58.4.4 drift_state_start_time_ . . . 534
10.58.4.5 gain_pressure_ . . . 534
10.58.4.6 gain_pressure_surface_ . . . 534
10.58.4.7 gain_salinity_ . . . 534
10.58.4.8 gain_salinity_surface_ . . . 535
10.58.4.9 gain_temperature_ . . . 535

10.58.4.10 gain_temperature_surface_ . . . 535
10.58.4.11 id_dive_ . . . 535
10.58.4.12 id_instrument_ . . . 535
10.58.4.13 id_xmsg_type_ . . . 535
10.58.4.14 max_ascent_duration_ . . . 536
10.58.4.15 max_fall_to_drift_duration_ . . . 536
10.58.4.16 max_fall_to_profle_duration_ . . . 536
10.58.4.17 max_surface_duration_ . . . 536
10.58.4.18 mission_phase_ . . . 536
10.58.4.19 num_seeks_to_drift_pressure_ . . . 536
10.58.4.20 offset_pressure_ . . . 536
10.58.4.21 offset_pressure_surface_ . . . 537
10.58.4.22 offset_salinity_ . . . 537
10.58.4.23 offset_salinity_surface_ . . . 537
10.58.4.24 offset_temperature_ . . . 537
10.58.4.25 offset_temperature_surface_ . . . 537
10.58.4.26 seek_to_drift_pressure_interval_duration_ . . . 538
10.58.4.27 surface_state_start_time_ . . . 538
10.58.4.28 target_ascent_rate_ . . . 538
10.58.4.29 target_drift_duration_ . . . 538
10.58.4.30 target_drift_pressure_ . . . 538
10.58.4.31 target_profile_pressure_ . . . 538
10.58.4.32 x_version_major_ . . . 538
10.58.4.33 x_version_minor_ . . . 539
10.58.4.34 xmsg_size_bytes_ . . . 539
10.59 util::xmsg::XCompositeAuxTemp Class Reference . . . 539
10.59.1 Detailed Description . . . 540
10.59.2 Member Typedef Documentation . . . 540
10.59.2.1 Element_t . . . 540
10.59.3 Member Function Documentation . . . 540
10.59.3.1 AppendAllElements() [1/2] . . . 540
10.59.3.2 AppendAllElements() [2/2] . . . 541
10.59.3.3 AppendElement() . . . 541
10.59.3.4 AssignElement() . . . 541
10.59.3.5 elements() . . . 541
10.59.3.6 GenerateListingString() . . . 542
10.59.3.7 num_elements() . . . 542
10.59.4 Member Data Documentation . . . 542
10.59.4.1 elements_ . . . 542
10.60 util::xmsg::XCompositeDrift Class Reference . . . 543
10.60.1 Detailed Description . . . 545
10.60.2 Member Typedef Documentation . . . 545
10.60.2.1 PresMsgPtr_t . . . 545
10.60.2.2 PresMsgVector_t . . . 545
10.60.2.3 SalnMsgPtr_t . . . 545
10.60.2.4 SalnMsgVector_t . . . 545
10.60.2.5 TempMsgPtr_t . . . 546
10.60.2.6 TempMsgVector_t . . . 546
10.60.3 Constructor & Destructor Documentation . . . 546
10.60.3.1 XCompositeDrift() . . . 546
10.60.3.2 ~XCompositeDrift() . . . 546
10.60.4 Member Function Documentation . . . 546

10.60.4.1 AggregateSensorData() . . . 546
10.60.4.2 AssignAuxTemperatureMessage() . . . 546
10.60.4.3 AssignPressureMessage() . . . 547
10.60.4.4 AssignSalinityMessage() . . . 547
10.60.4.5 AssignTemperatureMessage() . . . 548
10.60.4.6 GenerateListingString() . . . 548
10.60.4.7 GeneratePhyString() . . . 549
10.60.4.8 HasContiguousMessageIndices() . . . 550
10.60.4.9 is_aggregated_data_valid() . . . 551
10.60.4.10 IsDataComplete() . . . 551
10.60.4.11 IsEmpty() . . . 551
10.60.4.12 LockObject() . . . 551
10.60.4.13 max_num_samples() . . . 552
10.60.4.14 num_pres_samples() . . . 552
10.60.4.15 num_saln_samples() . . . 552
10.60.4.16 num_temp_samples() . . . 553
10.60.4.17 Reset() . . . 553
10.60.4.18 SamplePres() . . . 554
10.60.4.19 SamplePresAsFloat() . . . 554
10.60.4.20 SampleSaln() . . . 555
10.60.4.21 SampleSalnAsFloat() . . . 555
10.60.4.22 SampleTemp() . . . 555
10.60.4.23 SampleTempAsFloat() . . . 556
10.60.4.24 set_aggregated_data_valid() . . . 556
10.60.4.25 UnlockObject() . . . 557
10.60.5 Member Data Documentation . . . 557
10.60.5.1 aggregated_data_valid_ . . . 557
10.60.5.2 aggregated_pres_data_ . . . 557
10.60.5.3 aggregated_saln_data_ . . . 557
10.60.5.4 aggregated_temp_data_ . . . 558
10.60.5.5 mutex_ . . . 558
10.60.5.6 pres_messages_ . . . 559
10.60.5.7 saln_messages_ . . . 560
10.60.5.8 temp_messages_ . . . 560
10.61 util::xmsg::XCompositeFall Class Reference . . . 560
10.61.1 Detailed Description . . . 561
10.61.2 Member Typedef Documentation . . . 561
10.61.2.1 Element_t . . . 561
10.61.3 Constructor & Destructor Documentation . . . 561
10.61.3.1 XCompositeFall() . . . 561
10.61.3.2 ~XCompositeFall() . . . 562
10.61.4 Member Function Documentation . . . 562
10.61.4.1 AppendAllElements() [1/2] . . . 562
10.61.4.2 AppendAllElements() [2/2] . . . 562
10.61.4.3 AppendElement() . . . 562
10.61.4.4 AssignElement() . . . 562
10.61.4.5 elements() . . . 563
10.61.4.6 GenerateListingString() . . . 563
10.61.4.7 num_elements() . . . 564
10.61.5 Member Data Documentation . . . 564
10.61.5.1 elements_ . . . 564
10.62 util::xmsg::XCompositeHiResProfile Class Reference . . . 564

10.62.1 Detailed Description . . . 566
10.62.2 Member Typedef Documentation . . . 566
10.62.2.1 PresMsgPtr_t . . . 566
10.62.2.2 PresMsgVector_t . . . 567
10.62.2.3 SalnMsgPtr_t . . . 567
10.62.2.4 SalnMsgVector_t . . . 567
10.62.2.5 TempMsgPtr_t . . . 567
10.62.2.6 TempMsgVector_t . . . 567
10.62.3 Constructor & Destructor Documentation . . . 567
10.62.3.1 XCompositeHiResProfile() . . . 567
10.62.3.2 ~XCompositeHiResProfile() . . . 567
10.62.4 Member Function Documentation . . . 567
10.62.4.1 AggregateSensorData() . . . 567
10.62.4.2 AssignAuxTemperatureMessage() . . . 568
10.62.4.3 AssignPressureMessage() . . . 568
10.62.4.4 AssignSalinityMessage() . . . 568
10.62.4.5 AssignTemperatureMessage() . . . 569
10.62.4.6 GenerateListingString() . . . 569
10.62.4.7 GeneratePhyString() . . . 570
10.62.4.8 HasContiguousMessageIndices() . . . 571
10.62.4.9 is_aggregated_data_valid() . . . 572
10.62.4.10 IsDataComplete() . . . 572
10.62.4.11 IsEmpty() . . . 573
10.62.4.12 LockObject() . . . 573
10.62.4.13 max_num_samples() . . . 573
10.62.4.14 num_pres_samples() . . . 573
10.62.4.15 num_saln_samples() . . . 574
10.62.4.16 num_temp_samples() . . . 574
10.62.4.17 Reset() . . . 575
10.62.4.18 SamplePres() . . . 575
10.62.4.19 SamplePresAsFloat() . . . 575
10.62.4.20 SampleSaln() . . . 576
10.62.4.21 SampleSalnAsFloat() . . . 576
10.62.4.22 SampleTemp() . . . 577
10.62.4.23 SampleTempAsFloat() . . . 577
10.62.4.24 set_aggregated_data_valid() . . . 578
10.62.4.25 UnlockObject() . . . 578
10.62.5 Member Data Documentation . . . 578
10.62.5.1 aggregated_data_valid_ . . . 578
10.62.5.2 aggregated_pres_data_ . . . 579
10.62.5.3 aggregated_saln_data_ . . . 579
10.62.5.4 aggregated_temp_data_ . . . 579
10.62.5.5 mutex_ . . . 579
10.62.5.6 pres_messages_ . . . 581
10.62.5.7 saln_messages_ . . . 581
10.62.5.8 temp_messages_ . . . 581
10.63 util::xmsg::XCompositeIridiumPosition Class Reference . . . 581
10.63.1 Detailed Description . . . 584
10.63.2 Member Typedef Documentation . . . 584
10.63.2.1 Element_t . . . 584
10.63.3 Constructor & Destructor Documentation . . . 584
10.63.3.1 XCompositeIridiumPosition() . . . 584

10.63.3.2 ~XCompositeIridiumPosition() . . . 584
10.63.4 Member Function Documentation . . . 584
10.63.4.1 AppendAllElements() [1/2] . . . 584
10.63.4.2 AppendAllElements() [2/2] . . . 584
10.63.4.3 AppendElement() . . . 584
10.63.4.4 AssignElement() . . . 585
10.63.4.5 CalculateEllipsoidHeight() . . . 585
10.63.4.6 CalculateGeographicDistance() . . . 586
10.63.4.7 ComputeAverageIrdiumPosition() . . . 586
10.63.4.8 degToRad() . . . 587
10.63.4.9 elements() . . . 588
10.63.4.10 GenerateCurrentUtcTimeIso8601String() . . . 588
10.63.4.11 GenerateIso6709String() . . . 588
10.63.4.12 GenerateIso8601String() . . . 589
10.63.4.13 GenerateJulianDay() . . . 590
10.63.4.14 GenerateListingComment() . . . 591
10.63.4.15 GenerateListingString() . . . 591
10.63.4.16 GetCurrentUtcTime() . . . 592
10.63.4.17 IsPositionValid() . . . 592
10.63.4.18 num_elements() . . . 593
10.63.4.19 position_datetime() . . . 593
10.63.4.20 position_datetime_as_iso8601_string() . . . 593
10.63.4.21 position_is_valid() . . . 594
10.63.4.22 position_latitude() . . . 595
10.63.4.23 position_longitude() . . . 595
10.63.4.24 position_standard_error() . . . 596
10.63.4.25 position_variance() . . . 596
10.63.4.26 postition_as_iso6709_string() . . . 596
10.63.4.27 set_gps_position_datetime() . . . 597
10.63.4.28 set_position_datetime() . . . 597
10.63.4.29 set_position_is_valid() . . . 598
10.63.4.30 set_position_latitude() . . . 598
10.63.4.31 set_position_longitude() . . . 599
10.63.4.32 set_position_standard_error() . . . 599
10.63.4.33 set_position_variance() . . . 600
10.63.4.34 set_total_satellite_comm_time_seconds() . . . 600
10.63.4.35 total_satellite_comm_time_seconds() . . . 600
10.63.5 Member Data Documentation . . . 601
10.63.5.1 EARTH_RADIUS_KM_EQUATORIAL . . . 601
10.63.5.2 EARTH_RADIUS_KM_POLAR . . . 601
10.63.5.3 elements_ . . . 601
10.63.5.4 position_datetime_ . . . 601
10.63.5.5 position_is_valid_ . . . 601
10.63.5.6 position_latitude_ . . . 601
10.63.5.7 position_longitude_ . . . 601
10.63.5.8 position_standard_error_ . . . 602
10.63.5.9 position_variance_ . . . 602
10.63.5.10 total_satellite_comm_time_seconds_ . . . 602
10.64 util::xmsg::XCompositeProfile Class Reference . . . 602
10.64.1 Detailed Description . . . 604
10.64.2 Member Typedef Documentation . . . 604
10.64.2.1 PresMsgPtr_t . . . 604

10.64.2.2 PresMsgVector_t . . . 605
10.64.2.3 SalnMsgPtr_t . . . 605
10.64.2.4 SalnMsgVector_t . . . 605
10.64.2.5 TempMsgPtr_t . . . 605
10.64.2.6 TempMsgVector_t . . . 605
10.64.3 Constructor & Destructor Documentation . . . 605
10.64.3.1 XCompositeProfile() . . . 605
10.64.3.2 ~XCompositeProfile() . . . 605
10.64.4 Member Function Documentation . . . 605
10.64.4.1 AggregateSensorData() . . . 605
10.64.4.2 AssignAuxTemperatureMessage() . . . 606
10.64.4.3 AssignPressureMessage() . . . 606
10.64.4.4 AssignSalinityMessage() . . . 606
10.64.4.5 AssignTemperatureMessage() . . . 607
10.64.4.6 GenerateListingString() . . . 607
10.64.4.7 GeneratePhyString() . . . 608
10.64.4.8 HasContiguousMessageIndices() . . . 609
10.64.4.9 is_aggregated_data_valid() . . . 610
10.64.4.10 IsDataComplete() . . . 610
10.64.4.11 IsEmpty() . . . 611
10.64.4.12 LockObject() . . . 611
10.64.4.13 max_num_samples() . . . 611
10.64.4.14 num_pres_samples() . . . 611
10.64.4.15 num_saln_samples() . . . 612
10.64.4.16 num_temp_samples() . . . 612
10.64.4.17 Reset() . . . 613
10.64.4.18 SamplePres() . . . 613
10.64.4.19 SamplePresAsFloat() . . . 613
10.64.4.20 SampleSaln() . . . 614
10.64.4.21 SampleSalnAsFloat() . . . 614
10.64.4.22 SampleTemp() . . . 615
10.64.4.23 SampleTempAsFloat() . . . 615
10.64.4.24 set_aggregated_data_valid() . . . 616
10.64.4.25 UnlockObject() . . . 616
10.64.5 Member Data Documentation . . . 616
10.64.5.1 aggregated_data_valid_ . . . 616
10.64.5.2 aggregated_pres_data_ . . . 617
10.64.5.3 aggregated_saln_data_ . . . 617
10.64.5.4 aggregated_temp_data_ . . . 617
10.64.5.5 mutex_ . . . 617
10.64.5.6 pres_messages_ . . . 619
10.64.5.7 saln_messages_ . . . 619
10.64.5.8 temp_messages_ . . . 619
10.65 util::xmsg::XCompositePump Class Reference . . . 619
10.65.1 Detailed Description . . . 620
10.65.2 Member Typedef Documentation . . . 621
10.65.2.1 Element_t . . . 621
10.65.3 Constructor & Destructor Documentation . . . 621
10.65.3.1 XCompositePump() . . . 621
10.65.3.2 ~XCompositePump() . . . 621
10.65.4 Member Function Documentation . . . 621
10.65.4.1 AppendAllElements() [1/2] . . . 621

10.65.4.2 AppendAllElements() [2/2] . . . 621
10.65.4.3 AppendElement() . . . 621
10.65.4.4 AssignElement() . . . 621
10.65.4.5 elements() . . . 622
10.65.4.6 GenerateListingString() . . . 622
10.65.4.7 num_elements() . . . 623
10.65.5 Member Data Documentation . . . 623
10.65.5.1 elements_ . . . 623
10.66 util::xmsg::XCompositeRise Class Reference . . . 623
10.66.1 Detailed Description . . . 624
10.66.2 Member Typedef Documentation . . . 624
10.66.2.1 Element_t . . . 624
10.66.3 Constructor & Destructor Documentation . . . 624
10.66.3.1 XCompositeRise() . . . 624
10.66.3.2 ~XCompositeRise() . . . 625
10.66.4 Member Function Documentation . . . 625
10.66.4.1 AppendAllElements() [1/2] . . . 625
10.66.4.2 AppendAllElements() [2/2] . . . 625
10.66.4.3 AppendElement() . . . 625
10.66.4.4 AssignElement() . . . 625
10.66.4.5 elements() . . . 626
10.66.4.6 GenerateListingString() . . . 626
10.66.4.7 num_elements() . . . 627
10.66.5 Member Data Documentation . . . 627
10.66.5.1 elements_ . . . 627
10.67 util::xmsg::XCompositeSurfaceData Class Reference . . . 627
10.67.1 Detailed Description . . . 629
10.67.2 Constructor & Destructor Documentation . . . 629
10.67.2.1 XCompositeSurfaceData() . . . 629
10.67.2.2 ~XCompositeSurfaceData() . . . 629
10.67.3 Member Function Documentation . . . 630
10.67.3.1 AggregateSensorData() . . . 630
10.67.3.2 AssignMessage() . . . 630
10.67.3.3 GenerateListingString() . . . 630
10.67.3.4 is_aggregated_data_valid() . . . 631
10.67.3.5 IsEmpty() . . . 632
10.67.3.6 Reset() . . . 632
10.67.3.7 ResetAggregatedData() . . . 633
10.67.3.8 SamplePres() . . . 633
10.67.3.9 SamplePresAsFloat() . . . 633
10.67.3.10 SampleSaln() . . . 634
10.67.3.11 SampleSalnAsFloat() . . . 634
10.67.3.12 SampleTemp() . . . 634
10.67.3.13 SampleTempAsFloat() . . . 635
10.67.3.14 set_aggregated_data_valid() . . . 635
10.67.4 Member Data Documentation . . . 636
10.67.4.1 aggregated_data_valid_ . . . 636
10.68 util::xmsg::XCompositeValve Class Reference . . . 636
10.68.1 Detailed Description . . . 637
10.68.2 Member Typedef Documentation . . . 637
10.68.2.1 Element_t . . . 637
10.68.3 Member Function Documentation . . . 637

10.68.3.1 AppendAllElements() [1/2] . . . 637
10.68.3.2 AppendAllElements() [2/2] . . . 638
10.68.3.3 AppendElement() . . . 638
10.68.3.4 AssignElement() . . . 638
10.68.3.5 elements() . . . 638
10.68.3.6 num_elements() . . . 638
10.68.4 Member Data Documentation . . . 639
10.68.4.1 elements_ . . . 639
10.69 util::xmsg::XDriftPressure Class Reference . . . 639
10.69.1 Detailed Description . . . 641
10.69.2 Constructor & Destructor Documentation . . . 641
10.69.2.1 XDriftPressure() . . . 641
10.69.2.2 ~XDriftPressure() . . . 641
10.69.3 Member Function Documentation . . . 641
10.69.3.1 CheckXMsgData() . . . 641
10.69.3.2 DecodeXMsgData() . . . 642
10.69.3.3 id_dive() . . . 643
10.69.3.4 id_instrument() . . . 644
10.69.3.5 id_xmsg_type() . . . 645
10.69.3.6 num_samples() . . . 646
10.69.3.7 raw_sensor_data() . . . 646
10.69.3.8 Sample() . . . 646
10.69.3.9 SampleValue() . . . 646
10.69.3.10 SampleValueAsPressure() . . . 647
10.69.3.11 SampleValueAsSalinity() . . . 647
10.69.3.12 SampleValueAsTemperature() . . . 648
10.69.3.13 sensor_data_format() . . . 649
10.69.3.14 set_id_dive() . . . 649
10.69.3.15 set_id_instrument() . . . 649
10.69.3.16 set_id_xmsg_type() . . . 650
10.69.3.17 set_raw_sensor_data() . . . 651
10.69.3.18 set_sensor_data_format() . . . 651
10.69.3.19 set_xmsg_index() . . . 652
10.69.3.20 set_xmsg_size_bytes() . . . 652
10.69.3.21 UnpackSensorData() . . . 654
10.69.3.22 xmsg_index() . . . 655
10.69.3.23 xmsg_size_bytes() . . . 655
10.69.4 Member Data Documentation . . . 656
10.69.4.1 id_dive_ . . . 656
10.69.4.2 id_instrument_ . . . 656
10.69.4.3 id_xmsg_type_ . . . 657
10.69.4.4 raw_sensor_data_ . . . 657
10.69.4.5 sensor_data_format_ . . . 657
10.69.4.6 xmsg_index_ . . . 657
10.69.4.7 xmsg_size_bytes_ . . . 657
10.70 util::xmsg::XDriftSalinity Class Reference . . . 658
10.70.1 Detailed Description . . . 659
10.70.2 Constructor & Destructor Documentation . . . 659
10.70.2.1 XDriftSalinity() . . . 659
10.70.2.2 ~XDriftSalinity() . . . 660
10.70.3 Member Function Documentation . . . 660
10.70.3.1 CheckXMsgData() . . . 660

10.70.3.2 DecodeXMsgData() . . . 661
10.70.3.3 id_dive() . . . 662
10.70.3.4 id_instrument() . . . 663
10.70.3.5 id_xmsg_type() . . . 663
10.70.3.6 num_samples() . . . 664
10.70.3.7 raw_sensor_data() . . . 664
10.70.3.8 Sample() . . . 664
10.70.3.9 SampleValue() . . . 665
10.70.3.10 SampleValueAsPressure() . . . 665
10.70.3.11 SampleValueAsSalinity() . . . 666
10.70.3.12 SampleValueAsTemperature() . . . 666
10.70.3.13 sensor_data_format() . . . 667
10.70.3.14 set_id_dive() . . . 667
10.70.3.15 set_id_instrument() . . . 667
10.70.3.16 set_id_xmsg_type() . . . 668
10.70.3.17 set_raw_sensor_data() . . . 669
10.70.3.18 set_sensor_data_format() . . . 670
10.70.3.19 set_xmsg_index() . . . 670
10.70.3.20 set_xmsg_size_bytes() . . . 671
10.70.3.21 UnpackSensorData() . . . 672
10.70.3.22 xmsg_index() . . . 673
10.70.3.23 xmsg_size_bytes() . . . 674
10.70.4 Member Data Documentation . . . 675
10.70.4.1 id_dive_ . . . 675
10.70.4.2 id_instrument_ . . . 675
10.70.4.3 id_xmsg_type_ . . . 675
10.70.4.4 raw_sensor_data_ . . . 676
10.70.4.5 sensor_data_format_ . . . 676
10.70.4.6 xmsg_index_ . . . 676
10.70.4.7 xmsg_size_bytes_ . . . 676
10.71 util::xmsg::XDriftTemperature Class Reference . . . 677
10.71.1 Detailed Description . . . 678
10.71.2 Constructor & Destructor Documentation . . . 678
10.71.2.1 XDriftTemperature() . . . 678
10.71.2.2 ∼XDriftTemperature() . . . 679
10.71.3 Member Function Documentation . . . 679
10.71.3.1 CheckXMsgData() . . . 679
10.71.3.2 DecodeXMsgData() . . . 680
10.71.3.3 id_dive() . . . 681
10.71.3.4 id_instrument() . . . 682
10.71.3.5 id_xmsg_type() . . . 682
10.71.3.6 num_samples() . . . 683
10.71.3.7 raw_sensor_data() . . . 683
10.71.3.8 Sample() . . . 683
10.71.3.9 SampleValue() . . . 684
10.71.3.10 SampleValueAsPressure() . . . 684
10.71.3.11 SampleValueAsSalinity() . . . 685
10.71.3.12 SampleValueAsTemperature() . . . 685
10.71.3.13 sensor_data_format() . . . 686
10.71.3.14 set_id_dive() . . . 686
10.71.3.15 set_id_instrument() . . . 686
10.71.3.16 set_id_xmsg_type() . . . 687

10.71.3.17 set_raw_sensor_data() . . . 688
10.71.3.18 set_sensor_data_format() . . . 689
10.71.3.19 set_xmsg_index() . . . 689
10.71.3.20 set_xmsg_size_bytes() . . . 690
10.71.3.21 UnpackSensorData() . . . 691
10.71.3.22 xmsg_index() . . . 692
10.71.3.23 xmsg_size_bytes() . . . 693
10.71.4 Member Data Documentation . . . 694
10.71.4.1 id_dive_ . . . 694
10.71.4.2 id_instrument_ . . . 694
10.71.4.3 id_xmsg_type_ . . . 694
10.71.4.4 raw_sensor_data_ . . . 695
10.71.4.5 sensor_data_format_ . . . 695
10.71.4.6 xmsg_index_ . . . 695
10.71.4.7 xmsg_size_bytes_ . . . 695
10.72 util::xmsg::XEngAbort Class Reference . . . 696
10.72.1 Detailed Description . . . 697
10.72.2 Constructor & Destructor Documentation . . . 697
10.72.2.1 XEngAbort() . . . 697
10.72.2.2 ∼XEngAbort() . . . 698
10.72.3 Member Function Documentation . . . 698
10.72.3.1 CheckXMsgData() . . . 698
10.72.3.2 DecodeXMsgData() . . . 698
10.72.3.3 GenerateListingString() . . . 699
10.72.3.4 id_dive() . . . 699
10.72.3.5 id_instrument() . . . 700
10.72.3.6 id_xmsg_type() . . . 701
10.72.3.7 set_id_dive() . . . 702
10.72.3.8 set_id_instrument() . . . 702
10.72.3.9 set_id_xmsg_type() . . . 702
10.72.3.10 set_xmsg_size_bytes() . . . 703
10.72.3.11 xmsg_size_bytes() . . . 705
10.72.4 Member Data Documentation . . . 705
10.72.4.1 id_dive_ . . . 705
10.72.4.2 id_instrument_ . . . 706
10.72.4.3 id_xmsg_type_ . . . 706
10.72.4.4 xmsg_size_bytes_ . . . 706
10.73 util::xmsg::XEngAscent Class Reference . . . 706
10.73.1 Detailed Description . . . 708
10.73.2 Constructor & Destructor Documentation . . . 708
10.73.2.1 XEngAscent() . . . 708
10.73.2.2 ∼XEngAscent() . . . 709
10.73.3 Member Function Documentation . . . 709
10.73.3.1 CheckXMsgData() . . . 709
10.73.3.2 DecodeXMsgData() . . . 710
10.73.3.3 engineering_version() . . . 711
10.73.3.4 engineering_version_as_float() . . . 711
10.73.3.5 GenerateCurrentUtcTimeIso8601String() . . . 711
10.73.3.6 GenerateIso8601String() . . . 712
10.73.3.7 GenerateJulianDay() . . . 713
10.73.3.8 GenerateListingString() . . . 714
10.73.3.9 GetCurrentUtcTime() . . . 714

10.73.3.10 id_dive() . . . 715
10.73.3.11 id_instrument() . . . 716
10.73.3.12 id_xmsg_type() . . . 717
10.73.3.13 set_engineering_version() . . . 718
10.73.3.14 set_id_dive() . . . 718
10.73.3.15 set_id_instrument() . . . 718
10.73.3.16 set_id_xmsg_type() . . . 719
10.73.3.17 set_start_time() . . . 720
10.73.3.18 set_xmsg_size_bytes() . . . 721
10.73.3.19 start_time() . . . 722
10.73.3.20 system_current_a() . . . 723
10.73.3.21 xmsg_size_bytes() . . . 723
10.73.4 Member Data Documentation . . . 724
10.73.4.1 case_vac_at_end_cinhg_ . . . 724
10.73.4.2 case_vac_at_start_cinhg_ . . . 724
10.73.4.3 engineering_version_ . . . 725
10.73.4.4 id_dive_ . . . 725
10.73.4.5 id_instrument_ . . . 725
10.73.4.6 id_xmsg_type_ . . . 725
10.73.4.7 pump_run_duration_ . . . 725
10.73.4.8 start_time_ . . . 725
10.73.4.9 system_current_ma_ . . . 725
10.73.4.10 xmsg_size_bytes_ . . . 726
10.74 util::xmsg::XEngBIT Class Reference . . . 726
10.74.1 Detailed Description . . . 727
10.74.2 Constructor & Destructor Documentation . . . 727
10.74.2.1 XEngBIT() . . . 727
10.74.3 Member Function Documentation . . . 728
10.74.3.1 CheckXMsgData() . . . 728
10.74.3.2 DecodeXMsgData() . . . 728
10.74.3.3 GenerateListingString() . . . 729
10.74.3.4 id_dive() . . . 730
10.74.3.5 id_instrument() . . . 731
10.74.3.6 id_xmsg_type() . . . 732
10.74.3.7 set_alamo_bit2_msg() . . . 733
10.74.3.8 set_alamo_bit_msg() . . . 733
10.74.3.9 set_id_dive() . . . 733
10.74.3.10 set_id_instrument() . . . 733
10.74.3.11 set_id_xmsg_type() . . . 734
10.74.3.12 set_xmsg_size_bytes() . . . 735
10.74.3.13 xmsg_size_bytes() . . . 736
10.74.4 Member Data Documentation . . . 736
10.74.4.1 alamo_bit2_msg_ . . . 736
10.74.4.2 alamo_bit_msg_ . . . 737
10.74.4.3 id_dive_ . . . 737
10.74.4.4 id_instrument_ . . . 737
10.74.4.5 id_xmsg_type_ . . . 737
10.74.4.6 xmsg_size_bytes_ . . . 737
10.75 util::xmsg::XEngBITFail Class Reference . . . 738
10.75.1 Detailed Description . . . 739
10.75.2 Constructor & Destructor Documentation . . . 739
10.75.2.1 XEngBITFail() . . . 739

10.75.2.2 ~XEngBITFail() . . . 739
10.75.3 Member Function Documentation . . . 740
10.75.3.1 CheckXMsgData() . . . 740
10.75.3.2 DecodeXMsgData() . . . 740
10.75.3.3 GenerateListingString() . . . 741
10.75.3.4 id_dive() . . . 742
10.75.3.5 id_instrument() . . . 743
10.75.3.6 id_xmsg_type() . . . 744
10.75.3.7 set_alamo_bit2_msg() . . . 745
10.75.3.8 set_alamo_bit_msg() . . . 745
10.75.3.9 set_id_dive() . . . 745
10.75.3.10 set_id_instrument() . . . 745
10.75.3.11 set_id_xmsg_type() . . . 746
10.75.3.12 set_xmsg_size_bytes() . . . 747
10.75.3.13 xmsg_size_bytes() . . . 748
10.75.4 Member Data Documentation . . . 748
10.75.4.1 alamo_bit2_msg_ . . . 748
10.75.4.2 alamo_bit_msg_ . . . 749
10.75.4.3 id_dive_ . . . 749
10.75.4.4 id_instrument_ . . . 749
10.75.4.5 id_xmsg_type_ . . . 749
10.75.4.6 xmsg_size_bytes_ . . . 749
10.76 util::xmsg::XEngBITPass Class Reference . . . 750
10.76.1 Detailed Description . . . 751
10.76.2 Constructor & Destructor Documentation . . . 751
10.76.2.1 XEngBITPass() . . . 751
10.76.2.2 ~XEngBITPass() . . . 752
10.76.3 Member Function Documentation . . . 752
10.76.3.1 CheckXMsgData() . . . 752
10.76.3.2 DecodeXMsgData() . . . 752
10.76.3.3 GenerateListingString() . . . 753
10.76.3.4 id_dive() . . . 754
10.76.3.5 id_instrument() . . . 755
10.76.3.6 id_xmsg_type() . . . 756
10.76.3.7 set_alamo_bit2_msg() . . . 757
10.76.3.8 set_alamo_bit_msg() . . . 757
10.76.3.9 set_id_dive() . . . 757
10.76.3.10 set_id_instrument() . . . 757
10.76.3.11 set_id_xmsg_type() . . . 758
10.76.3.12 set_xmsg_size_bytes() . . . 759
10.76.3.13 xmsg_size_bytes() . . . 760
10.76.4 Member Data Documentation . . . 760
10.76.4.1 alamo_bit2_msg_ . . . 760
10.76.4.2 alamo_bit_msg_ . . . 761
10.76.4.3 id_dive_ . . . 761
10.76.4.4 id_instrument_ . . . 761
10.76.4.5 id_xmsg_type_ . . . 761
10.76.4.6 xmsg_size_bytes_ . . . 761
10.77 util::xmsg::XEngDescent Class Reference . . . 762
10.77.1 Detailed Description . . . 764
10.77.2 Constructor & Destructor Documentation . . . 765
10.77.2.1 XEngDescent() . . . 765

10.77.2.2 ~XEngDescent() . . . 765
10.77.3 Member Function Documentation . . . 765
10.77.3.1 case_vac_inhg() . . . 765
10.77.3.2 CheckXMsgData() . . . 766
10.77.3.3 DecodeXMsgData() . . . 766
10.77.3.4 engineering_version() . . . 767
10.77.3.5 engineering_version_as_float() . . . 768
10.77.3.6 GenerateCurrentUtcTimeIso8601String() . . . 768
10.77.3.7 GenerateIso8601String() . . . 768
10.77.3.8 GenerateJulianDay() . . . 769
10.77.3.9 GenerateListingString() . . . 770
10.77.3.10 GetCurrentUtcTime() . . . 771
10.77.3.11 id_dive() . . . 771
10.77.3.12 id_instrument() . . . 772
10.77.3.13 id_xmsg_type() . . . 773
10.77.3.14 pump_current_a() . . . 774
10.77.3.15 set_engineering_version() . . . 774
10.77.3.16 set_id_dive() . . . 774
10.77.3.17 set_id_instrument() . . . 775
10.77.3.18 set_id_xmsg_type() . . . 775
10.77.3.19 set_start_time() . . . 776
10.77.3.20 set_xmsg_size_bytes() . . . 777
10.77.3.21 start_time() . . . 778
10.77.3.22 system_current_a() . . . 779
10.77.3.23 system_voltage_v() . . . 779
10.77.3.24 xmsg_size_bytes() . . . 780
10.77.4 Member Data Documentation . . . 780
10.77.4.1 air_valve_middle_position_at_surface_mv_ . . . 780
10.77.4.2 case_vac_at_end_cinhg_ . . . 781
10.77.4.3 case_vac_at_start_cinhg_ . . . 781
10.77.4.4 case_vac_cinhg_ . . . 781
10.77.4.5 engineering_version_ . . . 781
10.77.4.6 first_pump_duration_adjustment_s_ . . . 781
10.77.4.7 id_dive_ . . . 781
10.77.4.8 id_instrument_ . . . 782
10.77.4.9 id_xmsg_type_ . . . 782
10.77.4.10 num_seek_attempts_ . . . 782
10.77.4.11 oil_valve_closed_position_during_fall_pump_mv_ . . . 782
10.77.4.12 oil_valve_open_position_at_surface_mv_ . . . 782
10.77.4.13 pump_current_ma_ . . . 782
10.77.4.14 pump_duration_sec_ . . . 783
10.77.4.15 start_time_ . . . 783
10.77.4.16 system_current_ma_ . . . 783
10.77.4.17 system_voltage_mv_ . . . 783
10.77.4.18 xmsg_size_bytes_ . . . 783
10.78 util::xmsg::XEngDiagnostic Class Reference . . . 784
10.78.1 Detailed Description . . . 785
10.78.2 Constructor & Destructor Documentation . . . 785
10.78.2.1 XEngDiagnostic() . . . 785
10.78.2.2 ~XEngDiagnostic() . . . 786
10.78.3 Member Function Documentation . . . 786
10.78.3.1 CheckXMsgData() . . . 786

10.78.3.2 DecodeXMsgData() . . . 786
10.78.3.3 GenerateListingString() . . . 787
10.78.3.4 id_dive() . . . 787
10.78.3.5 id_instrument() . . . 788
10.78.3.6 id_xmsg_type() . . . 789
10.78.3.7 set_id_dive() . . . 790
10.78.3.8 set_id_instrument() . . . 790
10.78.3.9 set_id_xmsg_type() . . . 790
10.78.3.10 set_xmsg_size_bytes() . . . 791
10.78.3.11 xmsg_size_bytes() . . . 793
10.78.4 Member Data Documentation . . . 793
10.78.4.1 id_dive_ . . . 793
10.78.4.2 id_instrument_ . . . 794
10.78.4.3 id_xmsg_type_ . . . 794
10.78.4.4 xmsg_size_bytes_ . . . 794
10.79 util::xmsg::XEngDrift Class Reference . . . 794
10.79.1 Detailed Description . . . 796
10.79.2 Constructor & Destructor Documentation . . . 797
10.79.2.1 XEngDrift() . . . 797
10.79.2.2 ∼XEngDrift() . . . 797
10.79.3 Member Function Documentation . . . 797
10.79.3.1 CheckXMsgData() . . . 797
10.79.3.2 DecodeXMsgData() . . . 798
10.79.3.3 engineering_version() . . . 799
10.79.3.4 engineering_version_as_float() . . . 799
10.79.3.5 GenerateCurrentUtcTimeIso8601String() . . . 800
10.79.3.6 GenerateIso8601String() . . . 800
10.79.3.7 GenerateJulianDay() . . . 801
10.79.3.8 GenerateListingString() . . . 802
10.79.3.9 GetCurrentUtcTime() . . . 803
10.79.3.10 id_dive() . . . 803
10.79.3.11 id_instrument() . . . 804
10.79.3.12 id_xmsg_type() . . . 805
10.79.3.13 set_engineering_version() . . . 806
10.79.3.14 set_id_dive() . . . 806
10.79.3.15 set_id_instrument() . . . 806
10.79.3.16 set_id_xmsg_type() . . . 807
10.79.3.17 set_start_time() . . . 808
10.79.3.18 set_xmsg_size_bytes() . . . 809
10.79.3.19 start_time() . . . 810
10.79.3.20 xmsg_size_bytes() . . . 811
10.79.4 Member Data Documentation . . . 812
10.79.4.1 average_drift_pressure_counts_ . . . 812
10.79.4.2 average_drift_temperature_counts_ . . . 812
10.79.4.3 case_vac_at_end_cinhg_ . . . 812
10.79.4.4 case_vac_at_start_cinhg_ . . . 813
10.79.4.5 engineering_version_ . . . 813
10.79.4.6 id_dive_ . . . 813
10.79.4.7 id_instrument_ . . . 813
10.79.4.8 id_xmsg_type_ . . . 813
10.79.4.9 start_of_fall_to_profile_pressure_counts_ . . . 813
10.79.4.10 start_of_fall_to_profile_pressure_time_ . . . 813

10.79.4.11 start_time_ . . . 814
10.79.4.12 xmsg_size_bytes_ . . . 814
10.80 util::xmsg::XEngSurface::XEngErrorCode Class Reference . . . 814
10.80.1 Detailed Description . . . 815
10.80.2 Constructor & Destructor Documentation . . . 815
10.80.2.1 XEngErrorCode() . . . 815
10.80.2.2 ~XEngErrorCode() . . . 816
10.80.3 Member Function Documentation . . . 816
10.80.3.1 date_time() . . . 816
10.80.3.2 DateTimeAs8601String() . . . 816
10.80.3.3 error_code() . . . 817
10.80.3.4 GenerateCurrentUtcTimeIso8601String() . . . 817
10.80.3.5 GenerateIso8601String() . . . 817
10.80.3.6 GenerateJulianDay() . . . 818
10.80.3.7 GetCurrentUtcTime() . . . 819
10.80.3.8 set_date_time() . . . 819
10.80.3.9 set_error_code() . . . 819
10.80.4 Member Data Documentation . . . 820
10.80.4.1 date_time_ . . . 820
10.80.4.2 error_code_ . . . 820
10.81 util::xmsg::XEngIce Class Reference . . . 820
10.81.1 Detailed Description . . . 822
10.81.2 Constructor & Destructor Documentation . . . 823
10.81.2.1 XEngIce() . . . 823
10.81.2.2 ~XEngIce() . . . 823
10.81.3 Member Function Documentation . . . 823
10.81.3.1 CheckXMsgData() . . . 823
10.81.3.2 DecodeXMsgData() . . . 824
10.81.3.3 engineering_version() . . . 825
10.81.3.4 engineering_version_as_float() . . . 826
10.81.3.5 GenerateCurrentUtcTimeIso8601String() . . . 826
10.81.3.6 GenerateIso8601String() . . . 826
10.81.3.7 GenerateJulianDay() . . . 827
10.81.3.8 GetCurrentUtcTime() . . . 828
10.81.3.9 id_dive() . . . 828
10.81.3.10 id_instrument() . . . 829
10.81.3.11 id_xmsg_type() . . . 830
10.81.3.12 set_engineering_version() . . . 831
10.81.3.13 set_id_dive() . . . 831
10.81.3.14 set_id_instrument() . . . 831
10.81.3.15 set_id_xmsg_type() . . . 832
10.81.3.16 set_start_time() . . . 833
10.81.3.17 set_xmsg_size_bytes() . . . 834
10.81.3.18 start_time() . . . 835
10.81.3.19 xmsg_size_bytes() . . . 836
10.81.4 Member Data Documentation . . . 837
10.81.4.1 engineering_version_ . . . 837
10.81.4.2 error_codes_ . . . 837
10.81.4.3 id_dive_ . . . 837
10.81.4.4 id_instrument_ . . . 837
10.81.4.5 id_xmsg_type_ . . . 837
10.81.4.6 start_time_ . . . 837

10.81.4.7 xmsg_size_bytes_ . . . 837
10.82 util::xmsg::XEngIce::XEngIceErrorCode Class Reference . . . 838
10.82.1 Detailed Description . . . 839
10.82.2 Constructor & Destructor Documentation . . . 839
10.82.2.1 XEngIceErrorCode() . . . 839
10.82.2.2 ~XEngIceErrorCode() . . . 839
10.82.3 Member Function Documentation . . . 839
10.82.3.1 date_time() . . . 839
10.82.3.2 DateTimeAs8601String() . . . 840
10.82.3.3 error_code() . . . 840
10.82.3.4 GenerateCurrentUtcTimeIso8601String() . . . 841
10.82.3.5 GenerateIso8601String() . . . 841
10.82.3.6 GenerateJulianDay() . . . 842
10.82.3.7 GetCurrentUtcTime() . . . 843
10.82.3.8 set_date_time() . . . 843
10.82.3.9 set_error_code() . . . 843
10.82.4 Member Data Documentation . . . 844
10.82.4.1 date_time_ . . . 844
10.82.4.2 error_code_ . . . 844
10.83 util::xmsg::XEngStandard Class Reference . . . 844
10.83.1 Detailed Description . . . 846
10.83.2 Constructor & Destructor Documentation . . . 846
10.83.2.1 XEngStandard() . . . 846
10.83.2.2 ~XEngStandard() . . . 846
10.83.3 Member Function Documentation . . . 846
10.83.3.1 CheckXMsgData() . . . 846
10.83.3.2 DecodeXMsgData() [1/2] . . . 847
10.83.3.3 DecodeXMsgData() [2/2] . . . 848
10.83.3.4 GenerateListingString() . . . 848
10.83.3.5 id_dive() . . . 848
10.83.3.6 id_instrument() . . . 849
10.83.3.7 id_xmsg_type() . . . 850
10.83.3.8 set_id_dive() . . . 851
10.83.3.9 set_id_instrument() . . . 851
10.83.3.10 set_id_xmsg_type() . . . 851
10.83.3.11 set_xmsg_size_bytes() . . . 852
10.83.3.12 xmsg_size_bytes() . . . 854
10.83.4 Member Data Documentation . . . 854
10.83.4.1 id_dive_ . . . 854
10.83.4.2 id_instrument_ . . . 855
10.83.4.3 id_xmsg_type_ . . . 855
10.83.4.4 xmsg_size_bytes_ . . . 855
10.84 util::xmsg::XEngSurface Class Reference . . . 855
10.84.1 Detailed Description . . . 860
10.84.2 Constructor & Destructor Documentation . . . 860
10.84.2.1 XEngSurface() . . . 860
10.84.2.2 ~XEngSurface() . . . 860
10.84.3 Member Function Documentation . . . 861
10.84.3.1 battery_voltage_v() . . . 861
10.84.3.2 case_vacuum_inhg() . . . 861
10.84.3.3 CheckXMsgData() . . . 861
10.84.3.4 DecodeXMsgData() . . . 862

10.84.3.5 engineering_version() . . . 863
10.84.3.6 engineering_version_as_float() . . . 863
10.84.3.7 GenerateCurrentUtcTimeIso8601String() . . . 864
10.84.3.8 GenerateIso8601String() . . . 864
10.84.3.9 GenerateJulianDay() . . . 865
10.84.3.10 GenerateListingString() . . . 866
10.84.3.11 GetCurrentUtcTime() . . . 866
10.84.3.12 id_dive() . . . 867
10.84.3.13 id_instrument() . . . 868
10.84.3.14 id_xmsg_type() . . . 869
10.84.3.15 set_engineering_version() . . . 870
10.84.3.16 set_id_dive() . . . 870
10.84.3.17 set_id_instrument() . . . 870
10.84.3.18 set_id_xmsg_type() . . . 871
10.84.3.19 set_start_time() . . . 872
10.84.3.20 set_xmsg_size_bytes() . . . 873
10.84.3.21 start_time() . . . 874
10.84.3.22 system_current_a() . . . 875
10.84.3.23 xmsg_size_bytes() . . . 875
10.84.4 Member Data Documentation . . . 876
10.84.4.1 air_bladder_max_pres_mpsi_ . . . 876
10.84.4.2 air_pump_current_ma_ . . . 876
10.84.4.3 air_pump_runtime_s_ . . . 877
10.84.4.4 air_valve_closed_cal_position_mv_ . . . 877
10.84.4.5 air_valve_mid_cal_position_mv_ . . . 877
10.84.4.6 air_valve_mid_position_at_end_mv_ . . . 877
10.84.4.7 air_valve_open_cal_position_mv_ . . . 877
10.84.4.8 air_valve_open_position_at_start_mv_ . . . 877
10.84.4.9 battery_voltage_mv_ . . . 878
10.84.4.10 case_vac_at_end_cinhg_ . . . 878
10.84.4.11 case_vac_at_start_cinhg_ . . . 878
10.84.4.12 case_vac_cingh_ . . . 878
10.84.4.13 ctd_status_code_ . . . 878
10.84.4.14 engineering_version_ . . . 878
10.84.4.15 error_codes_ . . . 879
10.84.4.16 id_dive_ . . . 879
10.84.4.17 id_instrument_ . . . 879
10.84.4.18 id_xmsg_type_ . . . 879
10.84.4.19 num_errors_ . . . 879
10.84.4.20 num_sbd_packets_for_dive_ . . . 879
10.84.4.21 num_transmit_attempts_for_previous_dive_ . . . 879
10.84.4.22 oil_valve_closed_cal_position_mv_ . . . 879
10.84.4.23 oil_valve_closed_position_at_end_mv_ . . . 880
10.84.4.24 oil_valve_open_cal_position_mv_ . . . 880
10.84.4.25 oil_valve_open_position_during_air_assist_mv_ . . . 880
10.84.4.26 pressure_offset_reading_ . . . 880
10.84.4.27 start_time_ . . . 880
10.84.4.28 system_current_ma_ . . . 880
10.84.4.29 xmsg_size_bytes_ . . . 880
10.85 util::xmsg::XFail Class Reference . . . 881
10.85.1 Detailed Description . . . 882
10.85.2 Constructor & Destructor Documentation . . . 882

10.85.2.1 XFall() . . . 882
10.85.2.2 ~XFall() . . . 883
10.85.3 Member Function Documentation . . . 883
10.85.3.1 CheckXMsgData() . . . 883
10.85.3.2 data_pack_format() . . . 884
10.85.3.3 DecodeXMsgData() . . . 884
10.85.3.4 GenerateCurrentUtcTimeIso8601String() . . . 885
10.85.3.5 GenerateIso8601String() . . . 886
10.85.3.6 GenerateJulianDay() . . . 887
10.85.3.7 GenerateListingComment() . . . 887
10.85.3.8 GenerateListingString() . . . 887
10.85.3.9 GetCurrentUtcTime() . . . 888
10.85.3.10 id_dive() . . . 889
10.85.3.11 id_instrument() . . . 890
10.85.3.12 id_xmsg_type() . . . 891
10.85.3.13 num_samples() . . . 892
10.85.3.14 sensor_data() . . . 892
10.85.3.15 set_data_pack_format() . . . 892
10.85.3.16 set_id_dive() . . . 893
10.85.3.17 set_id_instrument() . . . 893
10.85.3.18 set_id_xmsg_type() . . . 894
10.85.3.19 set_start_time() . . . 895
10.85.3.20 set_xmsg_index() . . . 896
10.85.3.21 set_xmsg_size_bytes() . . . 897
10.85.3.22 start_time() . . . 898
10.85.3.23 UnpackData() . . . 898
10.85.3.24 xmsg_index() . . . 899
10.85.3.25 xmsg_size_bytes() . . . 900
10.85.4 Member Data Documentation . . . 900
10.85.4.1 data_pack_format_ . . . 900
10.85.4.2 id_dive_ . . . 901
10.85.4.3 id_instrument_ . . . 901
10.85.4.4 id_xmsg_type_ . . . 901
10.85.4.5 sensor_data_ . . . 901
10.85.4.6 start_time_ . . . 901
10.85.4.7 xmsg_index_ . . . 901
10.85.4.8 xmsg_size_bytes_ . . . 901
10.86 argo::iridium::xfloat::XFloat Class Reference . . . 902
10.86.1 Detailed Description . . . 910
10.86.2 Constructor & Destructor Documentation . . . 910
10.86.2.1 XFloat() [1/2] . . . 910
10.86.2.2 XFloat() [2/2] . . . 911
10.86.2.3 ~XFloat() . . . 911
10.86.3 Member Function Documentation . . . 911
10.86.3.1 argo_instrument_type() . . . 911
10.86.3.2 CalculateEllipsoidHeight() . . . 911
10.86.3.3 CalculateGeographicDistance() . . . 912
10.86.3.4 CleanupSqlConnection() . . . 913
10.86.3.5 CleanupSqlDriver() . . . 913
10.86.3.6 CloseKmlFactory() . . . 913
10.86.3.7 CloseSqlConnection() . . . 914
10.86.3.8 degToRad() . . . 914

10.86.3.9 ExecuteQuery() . . . 915
10.86.3.10 executeQuery() . . . 915
10.86.3.11 executeSelectQuery() . . . 916
10.86.3.12 ExecuteUpdate() . . . 917
10.86.3.13 executeUpdate() . . . 917
10.86.3.14 fw_version() . . . 918
10.86.3.15 gain_pressure() . . . 918
10.86.3.16 gain_pressure_surface() . . . 919
10.86.3.17 gain_salinity() . . . 919
10.86.3.18 gain_salinity_surface() . . . 919
10.86.3.19 gain_temperature() . . . 920
10.86.3.20 gain_temperature_surface() . . . 920
10.86.3.21 GenerateCurrentUtcTimeIso8601String() . . . 920
10.86.3.22 GenerateIso6709String() . . . 921
10.86.3.23 GenerateIso8601String() . . . 921
10.86.3.24 GenerateJulianDay() . . . 922
10.86.3.25 GenerateKmlMasterDocument() . . . 923
10.86.3.26 GenerateKmlMasterString() . . . 923
10.86.3.27 GenerateKmlString() . . . 924
10.86.3.28 GeneratePreparedStatement() . . . 924
10.86.3.29 generatePreparedStatement() . . . 924
10.86.3.30 GenerateSqlDeleteStr() . . . 925
10.86.3.31 GenerateSqlSelectStr() . . . 925
10.86.3.32 GetAllCachedFloats() . . . 926
10.86.3.33 GetAllDiveData() . . . 926
10.86.3.34 GetCurrentUtcTime() . . . 927
10.86.3.35 GetDefaltKmlPathColor() . . . 927
10.86.3.36 GetDefaltKmlPathStyleId() . . . 928
10.86.3.37 GetDefaltKmlPlacemarkStyleId() . . . 928
10.86.3.38 GetDirectoryPath() . . . 928
10.86.3.39 GetDirectoryPathFromString() . . . 929
10.86.3.40 GetDiveData() . . . 930
10.86.3.41 GetFloat() . . . 931
10.86.3.42 GetKmlFactory() . . . 932
10.86.3.43 GetListingDirectoryPath() . . . 932
10.86.3.44 GetSmallKmlPlacemarkStyleId() . . . 932
10.86.3.45 GetSqlConnection() . . . 932
10.86.3.46 getSqlConnection() . . . 933
10.86.3.47 GetSqlDriver() . . . 934
10.86.3.48 GetTriangleKmlPlacemarkStyleId() . . . 934
10.86.3.49 id_dac() . . . 935
10.86.3.50 id_imei() . . . 935
10.86.3.51 id_instrument() . . . 936
10.86.3.52 id_wmo() . . . 936
10.86.3.53 instrument_ids_to_disable_phy_generation() . . . 937
10.86.3.54 instrument_ids_to_enable_phy_generation() . . . 937
10.86.3.55 IsPositionValid() . . . 937
10.86.3.56 ListDirectories() . . . 938
10.86.3.57 ListFiles() . . . 938
10.86.3.58 LoadLocalCacheFromDatabase() . . . 938
10.86.3.59 LoadMetadataFromFiles() . . . 939
10.86.3.60 LockObject() . . . 939

10.86.3.61 metadata() . . . 940
10.86.3.62 MoveFileToDirectory() . . . 940
10.86.3.63 offset_pressure() . . . 941
10.86.3.64 offset_pressure_surface() . . . 941
10.86.3.65 offset_salinity() . . . 942
10.86.3.66 offset_salinity_surface() . . . 942
10.86.3.67 offset_temperature() . . . 942
10.86.3.68 offset_temperature_surface() . . . 943
10.86.3.69 phy_file_generation_is_enabled() . . . 943
10.86.3.70 ResetCachedDiveData() . . . 943
10.86.3.71 ResetDiveData() . . . 944
10.86.3.72 SaveLocalCacheToDatabase() . . . 944
10.86.3.73 SaveObjectToDatabase() . . . 944
10.86.3.74 SelectAllObjectsFromDatabase() . . . 944
10.86.3.75 SelectObjectFromDatabase() . . . 945
10.86.3.76 SelectObjectsFromDatabase() . . . 945
10.86.3.77 SelectValueFromDatabase() . . . 946
10.86.3.78 set_argo_instrument_type() . . . 947
10.86.3.79 set_fw_version() . . . 947
10.86.3.80 set_gain_pressure() . . . 947
10.86.3.81 set_gain_pressure_surface() . . . 948
10.86.3.82 set_gain_salinity() . . . 948
10.86.3.83 set_gain_salinity_surface() . . . 948
10.86.3.84 set_gain_temperature() . . . 949
10.86.3.85 set_gain_temperature_surface() . . . 949
10.86.3.86 set_id_dac() . . . 949
10.86.3.87 set_id_imei() . . . 950
10.86.3.88 set_id_instrument() . . . 950
10.86.3.89 set_id_wmo() . . . 951
10.86.3.90 set_metadata() . . . 952
10.86.3.91 set_offset_pressure() . . . 952
10.86.3.92 set_offset_pressure_surface() . . . 952
10.86.3.93 set_offset_salinity() . . . 953
10.86.3.94 set_offset_salinity_surface() . . . 953
10.86.3.95 set_offset_temperature() . . . 953
10.86.3.96 set_offset_temperature_surface() . . . 954
10.86.3.97 set_phy_file_generation_is_enabled() . . . 954
10.86.3.98 set_start_of_mission_datetime() . . . 954
10.86.3.99 set_xmsg_version_major() . . . 955
10.86.3.100 set_xmsg_version_minor() . . . 955
10.86.3.101 SetAllDefaultKmlSyles() . . . 955
10.86.3.102 SetDefaultKmlStyle() . . . 956
10.86.3.103 start_of_mission_datetime() . . . 957
10.86.3.104 StartupTasks() . . . 957
10.86.3.105 UnlockObject() . . . 957
10.86.3.106 WriteKmlFile() . . . 958
10.86.3.107 WriteKmlMasterFile() . . . 958
10.86.3.108 WriteStatusFile() . . . 958
10.86.3.109 xmsg_version_major() . . . 959
10.86.3.110 xmsg_version_minor() . . . 959
10.86.4 Friends And Related Symbol Documentation . . . 959
10.86.4.1 Initialize . . . 959

10.86.4.2 util::config::LocalConfiguration::GenerateCommandLineOptionsDescription 960
10.86.4.3 util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription 960
10.86.5 Member Data Documentation 960
10.86.5.1 address_ 960
10.86.5.2 argo_instrument_type_ 960
10.86.5.3 dive2data_map_ 960
10.86.5.4 EARTH_RADIUS_KM_EQUATORIAL 961
10.86.5.5 EARTH_RADIUS_KM_POLAR 961
10.86.5.6 fw_version_ 961
10.86.5.7 gain_pressure_ 961
10.86.5.8 gain_pressure_surface_ 961
10.86.5.9 gain_salinity_ 961
10.86.5.10 gain_salinity_surface_ 962
10.86.5.11 gain_temperature_ 962
10.86.5.12 gain_temperature_surface_ 962
10.86.5.13 id2float_map_ 962
10.86.5.14 id2float_map_mutex_ 962
10.86.5.15 id_dac_ 963
10.86.5.16 id_imei_ 963
10.86.5.17 id_instrument_ 963
10.86.5.18 id_wmo_ 963
10.86.5.19 instrument_ids_to_disable_phy_generation_ 963
10.86.5.20 instrument_ids_to_enable_phy_generation_ 963
10.86.5.21 kml_default_path_colors_ 964
10.86.5.22 kml_default_path_style_ids_ 964
10.86.5.23 kml_default_placemark_style_ids_ 964
10.86.5.24 kml_directory_path_ 964
10.86.5.25 kml_document_ 965
10.86.5.26 kml_document_updated_ 965
10.86.5.27 kml_factory_ 965
10.86.5.28 kml_file_generation_is_enabled_ 965
10.86.5.29 kml_file_path_ 965
10.86.5.30 kml_file_updated_ 965
10.86.5.31 kml_small_placemark_style_ids_ 966
10.86.5.32 kml_triangle_placemark_style_ids_ 966
10.86.5.33 latest_gps_position_datetime_ 966
10.86.5.34 latest_gps_position_dive_number_ 966
10.86.5.35 latest_gps_position_latitude_ 966
10.86.5.36 latest_gps_position_longitude_ 966
10.86.5.37 latest_iridium_position_cep_radius_ 967
10.86.5.38 latest_iridium_position_datetime_ 967
10.86.5.39 latest_iridium_position_dive_number_ 967
10.86.5.40 latest_iridium_position_latitude_ 967
10.86.5.41 latest_iridium_position_longitude_ 967
10.86.5.42 latest_iridium_sbd_momsn_ 967
10.86.5.43 latest_iridium_sbd_mtmsn_ 967
10.86.5.44 latest_iridium_startup_position_datetime_ 967
10.86.5.45 latest_iridium_startup_sbd_momsn_ 967
10.86.5.46 metadata_ 968
10.86.5.47 mutex_ 968
10.86.5.48 netcdf_file_generation_is_enabled_ 969
10.86.5.49 offset_pressure_ 969

10.86.5.50 offset_pressure_surface_ . . . 970
10.86.5.51 offset_salinity_ . . . 970
10.86.5.52 offset_salinity_surface_ . . . 970
10.86.5.53 offset_temperature_ . . . 970
10.86.5.54 offset_temperature_surface_ . . . 970
10.86.5.55 password_ . . . 970
10.86.5.56 phy_file_generation_is_enabled_ . . . 971
10.86.5.57 port_ . . . 971
10.86.5.58 protocol_ . . . 971
10.86.5.59 schema_ . . . 971
10.86.5.60 sql_connection_ . . . 971
10.86.5.61 sql_driver_ . . . 971
10.86.5.62 sql_table_description_ . . . 972
10.86.5.63 start_of_mission_datetime_ . . . 972
10.86.5.64 status_directory_path_ . . . 972
10.86.5.65 status_file_generation_is_enabled_ . . . 972
10.86.5.66 status_file_path_ . . . 972
10.86.5.67 status_file_updated_ . . . 972
10.86.5.68 user_ . . . 973
10.86.5.69 xmsg_version_major_ . . . 973
10.86.5.70 xmsg_version_minor_ . . . 973
10.87 argo::iridium::xfloat::XFloatDiveData Class Reference . . . 973
10.87.1 Detailed Description . . . 981
10.87.2 Constructor & Destructor Documentation . . . 982
10.87.2.1 XFloatDiveData() [1/2] . . . 982
10.87.2.2 XFloatDiveData() [2/2] . . . 982
10.87.2.3 ~XFloatDiveData() . . . 982
10.87.3 Member Function Documentation . . . 983
10.87.3.1 argo_instrument_type() . . . 983
10.87.3.2 AssignArgoData() . . . 983
10.87.3.3 AssignAuxTemperatureBinned() . . . 983
10.87.3.4 AssignDriftPressure() . . . 983
10.87.3.5 AssignDriftSalinity() . . . 984
10.87.3.6 AssignDriftTemperatre() . . . 984
10.87.3.7 AssignEngineeringAbort() . . . 984
10.87.3.8 AssignEngineeringAscent() . . . 985
10.87.3.9 AssignEngineeringBitFail() . . . 985
10.87.3.10 AssignEngineeringBitPass() . . . 985
10.87.3.11 AssignEngineeringDescent() . . . 985
10.87.3.12 AssignEngineeringDiagnostic() . . . 986
10.87.3.13 AssignEngineeringDrift() . . . 986
10.87.3.14 AssignEngineeringIce() . . . 986
10.87.3.15 AssignEngineeringStandard() . . . 986
10.87.3.16 AssignEngineeringSurface() . . . 987
10.87.3.17 AssignFallTimeSeries() . . . 987
10.87.3.18 AssignGps() . . . 987
10.87.3.19 AssignMailMessage() . . . 988
10.87.3.20 AssignMissionCmdEchoback() . . . 988
10.87.3.21 AssignParentInstrument() . . . 988
10.87.3.22 AssignProfilePressureBinned() . . . 989
10.87.3.23 AssignProfilePressureUnbinned() . . . 989
10.87.3.24 AssignProfileSalinityBinned() . . . 990

10.87.3.25 AssignProfileSalinityUnbinned() . . . 990
10.87.3.26 AssignProfileTemperatureBinned() . . . 991
10.87.3.27 AssignProfileTemperatureUnbinned() . . . 991
10.87.3.28 AssignPumpTimeSeries() . . . 991
10.87.3.29 AssignRiseTimeSeries() . . . 992
10.87.3.30 AssignSurfaceTimeSeries() . . . 992
10.87.3.31 AssignValveData() . . . 992
10.87.3.32 CheckCompleteness() . . . 993
10.87.3.33 CleanupSqlConnection() . . . 993
10.87.3.34 CleanupSqlDriver() . . . 994
10.87.3.35 CloseSqlConnection() . . . 994
10.87.3.36 ComputePressureOffsetFromSurfaceTimeSeries() . . . 995
10.87.3.37 DecodeMailMessageData() . . . 995
10.87.3.38 DecodeXMsgData() . . . 996
10.87.3.39 DeleteFromDatabase() . . . 997
10.87.3.40 DeleteObjectsFromDatabase() . . . 997
10.87.3.41 ExecuteQuery() . . . 997
10.87.3.42 executeQuery() . . . 998
10.87.3.43 executeSelectQuery() . . . 999
10.87.3.44 ExecuteUpdate() . . . 1000
10.87.3.45 executeUpdate() . . . 1000
10.87.3.46 GenerateCompositeFallTimeSeriesData() . . . 1001
10.87.3.47 GenerateCompositeIridiumPosition() . . . 1002
10.87.3.48 GenerateCompositePumpTimeSeriesData() . . . 1002
10.87.3.49 GenerateCompositeRiseTimeSeriesData() . . . 1002
10.87.3.50 GenerateCurrentUtcTimeIso8601String() . . . 1003
10.87.3.51 GenerateIso8601String() . . . 1003
10.87.3.52 GenerateJulianDay() . . . 1004
10.87.3.53 GenerateListingString() . . . 1005
10.87.3.54 GenerateOrderedMomsnList() . . . 1005
10.87.3.55 GenerateOrderedPidList() . . . 1006
10.87.3.56 GeneratePhyString() . . . 1006
10.87.3.57 GeneratePreparedStatement() . . . 1007
10.87.3.58 generatePreparedStatement() . . . 1008
10.87.3.59 GenerateReadableHex() . . . 1008
10.87.3.60 GenerateSqlDeleteStr() . . . 1009
10.87.3.61 GenerateSqlSelectStr() . . . 1009
10.87.3.62 GetCurrentUtcTime() . . . 1010
10.87.3.63 GetDirectoryPath() . . . 1010
10.87.3.64 GetDirectoryPathFromString() . . . 1010
10.87.3.65 GetDiveDataMailMessagesFromDatabase() . . . 1011
10.87.3.66 GetDiveDataObjectFromDatabase() . . . 1012
10.87.3.67 GetGainsAndOffsets() . . . 1012
10.87.3.68 GetGpsPositionEndOfDive() . . . 1013
10.87.3.69 GetSqlConnection() . . . 1013
10.87.3.70 getSqlConnection() . . . 1014
10.87.3.71 GetSqlDriver() . . . 1015
10.87.3.72 GetSurfaceGainsAndOffsets() . . . 1015
10.87.3.73 HasBinnedProfileData() . . . 1016
10.87.3.74 HasCompleteBinnedProfileData() . . . 1017
10.87.3.75 HasCompleteProfileData() . . . 1017
10.87.3.76 HasCompleteUnbinnedProfileData() . . . 1018

10.87.3.77 HasContiguousMomsns() . . . 1019
10.87.3.78 HasContiguousPids() . . . 1019
10.87.3.79 HasDiveData() . . . 1020
10.87.3.80 HasDriftData() . . . 1021
10.87.3.81 HasEngineeringData() . . . 1021
10.87.3.82 HasFallRiseData() . . . 1021
10.87.3.83 HasPositionData() . . . 1022
10.87.3.84 HasUnbinnedProfileData() . . . 1022
10.87.3.85 HasValidGpsPosition() . . . 1022
10.87.3.86 HasValidIridiumPosition() . . . 1023
10.87.3.87 HasValidPosition() . . . 1023
10.87.3.88 id_dac() . . . 1024
10.87.3.89 id_dive() . . . 1024
10.87.3.90 id_imei() . . . 1025
10.87.3.91 id_instrument() . . . 1025
10.87.3.92 id_wmo() . . . 1026
10.87.3.93 intermediate_listing_file_path() . . . 1026
10.87.3.94 ListDirectories() . . . 1026
10.87.3.95 ListFiles() . . . 1026
10.87.3.96 LockObject() . . . 1027
10.87.3.97 MoveFileToDirectory() . . . 1027
10.87.3.98 phy_file_path() . . . 1028
10.87.3.99 ResetData() . . . 1028
10.87.3.100 SaveObjectToDatabase() . . . 1029
10.87.3.101 SelectObjectFromDatabase() . . . 1029
10.87.3.102 SelectObjectsFromDatabase() . . . 1029
10.87.3.103 SelectValueFromDatabase() . . . 1030
10.87.3.104 SensorGainsAndOffsetsAreKnown() . . . 1031
10.87.3.105 set_argo_instrument_type() . . . 1031
10.87.3.106 set_id_dac() . . . 1031
10.87.3.107 set_id_dive() . . . 1032
10.87.3.108 set_id_imei() . . . 1032
10.87.3.109 set_id_instrument() . . . 1032
10.87.3.110 set_id_wmo() . . . 1033
10.87.3.111 set_intermediate_listing_file_path() . . . 1033
10.87.3.112 set_phy_file_path() . . . 1034
10.87.3.113 UnlockObject() . . . 1034
10.87.3.114 UpdateDerivedSurfaceDataFields() . . . 1034
10.87.3.115 WriteListingFile() . . . 1035
10.87.3.116 WritePhyFile() . . . 1036
10.87.4 Friends And Related Symbol Documentation . . . 1037
10.87.4.1 std::make_shared . . . 1037
10.87.4.2 util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription . . . 1037
10.87.5 Member Data Documentation . . . 1037
10.87.5.1 address_ . . . 1037
10.87.5.2 argo_instrument_type_ . . . 1037
10.87.5.3 complete_ . . . 1038
10.87.5.4 id_dac_ . . . 1038
10.87.5.5 id_dive_ . . . 1038
10.87.5.6 id_imei_ . . . 1038
10.87.5.7 id_instrument_ . . . 1038
10.87.5.8 id_wmo_ . . . 1038

10.87.5.9 intermediate_listing_directory_path_ . . . 1039
10.87.5.10 intermediate_listing_file_generation_is_enabled_ . . . 1039
10.87.5.11 iridium_position_datetime_first_ . . . 1039
10.87.5.12 iridium_position_datetime_last_ . . . 1039
10.87.5.13 listing_file_path_ . . . 1039
10.87.5.14 listing_file_updated_ . . . 1039
10.87.5.15 mail_messages_ . . . 1040
10.87.5.16 mutex_ . . . 1040
10.87.5.17 netcdf_directory_path_ . . . 1041
10.87.5.18 netcdf_file_generation_is_enabled_ . . . 1041
10.87.5.19 netcdf_file_path_ . . . 1042
10.87.5.20 netcdf_file_updated_ . . . 1042
10.87.5.21 netcdf_reference_datetime_ . . . 1042
10.87.5.22 num_sbd_msgs_ . . . 1042
10.87.5.23 parent_instrument_ . . . 1042
10.87.5.24 password_ . . . 1042
10.87.5.25 phy_directory_path_ . . . 1043
10.87.5.26 phy_file_generation_is_enabled_ . . . 1043
10.87.5.27 phy_file_path_ . . . 1043
10.87.5.28 phy_file_updated_ . . . 1043
10.87.5.29 port_ . . . 1043
10.87.5.30 pres_offs_ . . . 1043
10.87.5.31 pres_offs_num_dives_ . . . 1044
10.87.5.32 pres_offs_num_samples_ . . . 1044
10.87.5.33 pres_offs_valid_ . . . 1044
10.87.5.34 pressure_offset_correction_enabled_list_file_ . . . 1044
10.87.5.35 pressure_offset_correction_enabled_netcdf_file_ . . . 1044
10.87.5.36 pressure_offset_correction_enabled_phy_file_ . . . 1044
10.87.5.37 protocol_ . . . 1045
10.87.5.38 schema_ . . . 1045
10.87.5.39 sql_connection_ . . . 1045
10.87.5.40 sql_driver_ . . . 1045
10.87.5.41 sql_table_description_ . . . 1045
10.87.5.42 surface_pres_max_ . . . 1045
10.87.5.43 surface_pres_mean_ . . . 1046
10.87.5.44 surface_pres_min_ . . . 1046
10.87.5.45 surface_pres_num_samples_ . . . 1046
10.87.5.46 surface_pres_valid_ . . . 1046
10.87.5.47 surface_pres_variance_ . . . 1046
10.87.5.48 user_ . . . 1046
10.87.5.49 xmsg_acoustic_positions_ . . . 1046
10.87.5.50 xmsg_argo_data_ . . . 1046
10.87.5.51 xmsg_composite_drift_data_ . . . 1047
10.87.5.52 xmsg_composite_profile_binned_ . . . 1047
10.87.5.53 xmsg_composite_profile_unbinned_ . . . 1047
10.87.5.54 xmsg_engineering_data_abort_ . . . 1047
10.87.5.55 xmsg_engineering_data_ascent_ . . . 1047
10.87.5.56 xmsg_engineering_data_bit_fail_ . . . 1048
10.87.5.57 xmsg_engineering_data_bit_passed_ . . . 1048
10.87.5.58 xmsg_engineering_data_descent_ . . . 1048
10.87.5.59 xmsg_engineering_data_diagnostic_dive_ . . . 1048
10.87.5.60 xmsg_engineering_data_drift_ . . . 1048

10.87.5.61 xmsg_engineering_data_ice_ . . . 1048
10.87.5.62 xmsg_engineering_data_standard_dive_ . . . 1049
10.87.5.63 xmsg_engineering_data_surface_ . . . 1049
10.87.5.64 xmsg_fall_time_series_ . . . 1049
10.87.5.65 xmsg_gps_positions_ . . . 1049
10.87.5.66 xmsg_mission_cfg_ . . . 1049
10.87.5.67 xmsg_mission_cmd_echoback_ . . . 1050
10.87.5.68 xmsg_pump_time_series_ . . . 1050
10.87.5.69 xmsg_rise_time_series_ . . . 1050
10.87.5.70 xmsg_surface_time_series_ . . . 1050
10.87.5.71 xmsg_valve_data_ . . . 1050
10.87.5.72 xmsg_wave_field_ . . . 1051
10.88 argo::iridium::xfloat::XFloatMailMessage Class Reference . . . 1051
10.88.1 Detailed Description . . . 1058
10.88.2 Constructor & Destructor Documentation . . . 1059
10.88.2.1 XFloatMailMessage() [1/2] . . . 1059
10.88.2.2 XFloatMailMessage() [2/2] . . . 1059
10.88.2.3 ~XFloatMailMessage() . . . 1059
10.88.3 Member Function Documentation . . . 1059
10.88.3.1 CalculateEllipsoidHeight() . . . 1059
10.88.3.2 CalculateGeographicDistance() . . . 1060
10.88.3.3 CleanupSqlConnection() . . . 1061
10.88.3.4 CleanupSqlDriver() . . . 1061
10.88.3.5 CloseSqlConnection() . . . 1062
10.88.3.6 degToRad() . . . 1063
10.88.3.7 ExecuteQuery() . . . 1063
10.88.3.8 executeQuery() . . . 1064
10.88.3.9 executeSelectQuery() . . . 1065
10.88.3.10 ExecuteUpdate() . . . 1066
10.88.3.11 executeUpdate() . . . 1066
10.88.3.12 GenerateCurrentUtcTimeIso8601String() . . . 1067
10.88.3.13 GenerateIso6709String() . . . 1067
10.88.3.14 GenerateIso8601String() . . . 1068
10.88.3.15 GenerateJulianDay() . . . 1069
10.88.3.16 GenerateListingComment() . . . 1070
10.88.3.17 GenerateListingString() . . . 1070
10.88.3.18 GeneratePreparedStatement() . . . 1070
10.88.3.19 generatePreparedStatement() . . . 1071
10.88.3.20 GenerateSqlDeleteStr() . . . 1072
10.88.3.21 GenerateSqlSelectStr() . . . 1072
10.88.3.22 GetCurrentUtcTime() . . . 1073
10.88.3.23 GetDirectoryPath() . . . 1073
10.88.3.24 GetDirectoryPathFromString() . . . 1073
10.88.3.25 GetSqlConnection() . . . 1074
10.88.3.26 getSqlConnection() . . . 1075
10.88.3.27 GetSqlDriver() . . . 1076
10.88.3.28 id_dac() . . . 1076
10.88.3.29 id_dive() . . . 1077
10.88.3.30 id_imei() . . . 1077
10.88.3.31 id_instrument() . . . 1078
10.88.3.32 id_msg() . . . 1078
10.88.3.33 id_msg_uid() . . . 1079

10.88.3.34 id_sbd_momsn() . . . 1079
10.88.3.35 id_sbd_mtmsn() . . . 1080
10.88.3.36 id_sbd_packet() . . . 1080
10.88.3.37 id_wmo() . . . 1081
10.88.3.38 iridium_position_as_iso6709_string() . . . 1081
10.88.3.39 iridium_position_cep_radius() . . . 1082
10.88.3.40 iridium_position_datetime() . . . 1082
10.88.3.41 iridium_position_datetime_as_iso8601_string() . . . 1083
10.88.3.42 iridium_position_is_valid() . . . 1083
10.88.3.43 iridium_position_latitude() . . . 1084
10.88.3.44 iridium_position_longitude() . . . 1084
10.88.3.45 IsPositionValid() . . . 1084
10.88.3.46 ListDirectories() . . . 1085
10.88.3.47 ListFiles() . . . 1085
10.88.3.48 LockObject() . . . 1086
10.88.3.49 MoveFileToDirectory() . . . 1086
10.88.3.50 msg_body_is_valid() . . . 1087
10.88.3.51 msg_body_text() . . . 1088
10.88.3.52 msg_header_datetime() . . . 1088
10.88.3.53 msg_header_from() . . . 1088
10.88.3.54 msg_header_is_valid() . . . 1089
10.88.3.55 msg_header_raw_text() . . . 1089
10.88.3.56 msg_header_subject() . . . 1089
10.88.3.57 msg_is_valid() . . . 1090
10.88.3.58 msg_raw_text() . . . 1090
10.88.3.59 ParseMailMsg() [1/2] . . . 1090
10.88.3.60 ParseMailMsg() [2/2] . . . 1091
10.88.3.61 ParseMailMsgBody() . . . 1092
10.88.3.62 ParseMailMsgHeader() . . . 1093
10.88.3.63 ParseMailMsgSbdAttachment() . . . 1094
10.88.3.64 position_datetime() . . . 1095
10.88.3.65 position_datetime_as_iso8601_string() . . . 1095
10.88.3.66 position_is_valid() . . . 1096
10.88.3.67 position_latitude() . . . 1097
10.88.3.68 position_longitude() . . . 1097
10.88.3.69 postition_as_iso6709_string() . . . 1098
10.88.3.70 raw_file_local_path() . . . 1098
10.88.3.71 SaveObjectToDatabase() . . . 1098
10.88.3.72 sbd_data() . . . 1099
10.88.3.73 sbd_data_is_valid() . . . 1099
10.88.3.74 sbd_file_local_path() . . . 1100
10.88.3.75 sbd_file_name() . . . 1100
10.88.3.76 sbd_file_size_bytes() . . . 1100
10.88.3.77 sbd_session_status_code() . . . 1101
10.88.3.78 sbd_session_status_string() . . . 1101
10.88.3.79 SelectAllObjectsFromDatabase() . . . 1101
10.88.3.80 SelectObjectFromDatabase() . . . 1102
10.88.3.81 SelectObjectsFromDatabase() . . . 1103
10.88.3.82 SelectValueFromDatabase() . . . 1104
10.88.3.83 set_gps_position_datetime() . . . 1104
10.88.3.84 set_id_dac() . . . 1105
10.88.3.85 set_id_dive() . . . 1105

10.88.3.86 set_id_imei() . . . 1106
10.88.3.87 set_id_instrument() . . . 1106
10.88.3.88 set_id_msg() . . . 1107
10.88.3.89 set_id_msg_uid() . . . 1108
10.88.3.90 set_id_sbd_momsn() . . . 1108
10.88.3.91 set_id_sbd_mtmsn() . . . 1108
10.88.3.92 set_id_sbd_packet() . . . 1108
10.88.3.93 set_id_wmo() . . . 1109
10.88.3.94 set_iridium_position_cep_radius() . . . 1109
10.88.3.95 set_iridium_position_datetime() . . . 1110
10.88.3.96 set_iridium_position_is_valid() . . . 1110
10.88.3.97 set_iridium_position_latitude() . . . 1111
10.88.3.98 set_iridium_position_longitude() . . . 1111
10.88.3.99 set_msg_body_is_valid() . . . 1112
10.88.3.100 set_msg_body_text() . . . 1112
10.88.3.101 set_msg_header_datetime() . . . 1113
10.88.3.102 set_msg_header_from() . . . 1113
10.88.3.103 set_msg_header_is_valid() . . . 1114
10.88.3.104 set_msg_header_raw_text() . . . 1114
10.88.3.105 set_msg_header_subject() . . . 1114
10.88.3.106 set_msg_is_valid() . . . 1114
10.88.3.107 set_msg_raw_text() . . . 1115
10.88.3.108 set_position_datetime() . . . 1115
10.88.3.109 set_position_is_valid() . . . 1115
10.88.3.110 set_position_latitude() . . . 1116
10.88.3.111 set_position_longitude() . . . 1117
10.88.3.112 set_raw_file_local_path() . . . 1117
10.88.3.113 set_sbd_data() . . . 1118
10.88.3.114 set_sbd_data_is_valid() . . . 1119
10.88.3.115 set_sbd_file_local_path() . . . 1119
10.88.3.116 set_sbd_file_name() . . . 1120
10.88.3.117 set_sbd_file_size_bytes() . . . 1120
10.88.3.118 set_sbd_session_status_code() . . . 1120
10.88.3.119 set_sbd_session_status_string() . . . 1120
10.88.3.120 UnlockObject() . . . 1120
10.88.3.121 WriteFiles() . . . 1121
10.88.3.122 WriteRawFile() . . . 1122
10.88.3.123 WriteSbdFile() . . . 1123
10.88.4 Friends And Related Symbol Documentation . . . 1124
10.88.4.1 operator<< . . . 1124
10.88.4.2 operator>> . . . 1124
10.88.4.3 util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription . . . 1124
10.88.5 Member Data Documentation . . . 1125
10.88.5.1 address_ . . . 1125
10.88.5.2 EARTH_RADIUS_KM_EQUATORIAL . . . 1125
10.88.5.3 EARTH_RADIUS_KM_POLAR . . . 1125
10.88.5.4 id_dac_ . . . 1125
10.88.5.5 id_dive_ . . . 1125
10.88.5.6 id_imei_ . . . 1125
10.88.5.7 id_instrument_ . . . 1126
10.88.5.8 id_msg_ . . . 1126
10.88.5.9 id_msg_uid_ . . . 1126

10.88.5.10 id_sbd_momsn_ . . . 1126
10.88.5.11 id_sbd_mtmsn_ . . . 1127
10.88.5.12 id_sbd_packet_ . . . 1127
10.88.5.13 id_wmo_ . . . 1127
10.88.5.14 iridium_position_cep_radius_ . . . 1127
10.88.5.15 msg_body_is_valid_ . . . 1127
10.88.5.16 msg_body_text_ . . . 1128
10.88.5.17 msg_header_datetime_ . . . 1128
10.88.5.18 msg_header_from_ . . . 1128
10.88.5.19 msg_header_is_valid_ . . . 1128
10.88.5.20 msg_header_raw_text_ . . . 1129
10.88.5.21 msg_header_subject_ . . . 1129
10.88.5.22 msg_header_valid_sender_regex_ . . . 1129
10.88.5.23 msg_header_valid_subject_regex_ . . . 1129
10.88.5.24 msg_is_valid_ . . . 1129
10.88.5.25 msg_raw_text_ . . . 1130
10.88.5.26 mutex_ . . . 1130
10.88.5.27 password_ . . . 1131
10.88.5.28 port_ . . . 1131
10.88.5.29 position_datetime_ . . . 1131
10.88.5.30 position_is_valid_ . . . 1132
10.88.5.31 position_latitude_ . . . 1132
10.88.5.32 position_longitude_ . . . 1132
10.88.5.33 protocol_ . . . 1132
10.88.5.34 raw_directory_path_ . . . 1132
10.88.5.35 raw_file_format_ . . . 1132
10.88.5.36 raw_file_generation_is_enabled_ . . . 1133
10.88.5.37 raw_file_local_path_ . . . 1133
10.88.5.38 sbd_data_ . . . 1133
10.88.5.39 sbd_data_is_valid_ . . . 1133
10.88.5.40 sbd_directory_path_ . . . 1133
10.88.5.41 sbd_file_generation_is_enabled_ . . . 1134
10.88.5.42 sbd_file_local_path_ . . . 1134
10.88.5.43 sbd_file_name_ . . . 1134
10.88.5.44 sbd_file_size_bytes_ . . . 1134
10.88.5.45 sbd_file_valid_name_regex_ . . . 1134
10.88.5.46 sbd_session_status_code_ . . . 1135
10.88.5.47 sbd_session_status_string_ . . . 1135
10.88.5.48 schema_ . . . 1135
10.88.5.49 sql_connection_ . . . 1135
10.88.5.50 sql_driver_ . . . 1135
10.88.5.51 sql_table_description_ . . . 1136
10.88.5.52 user_ . . . 1136
10.89 argo::iridium::xfloat::XFloatMailProcessingService Class Reference . . . 1136
10.89.1 Detailed Description . . . 1138
10.89.2 Constructor & Destructor Documentation . . . 1139
10.89.2.1 XFloatMailProcessingService() . . . 1139
10.89.2.2 ~XFloatMailProcessingService() . . . 1139
10.89.3 Member Function Documentation . . . 1139
10.89.3.1 CleanupSqlConnection() . . . 1139
10.89.3.2 CleanupSqlDriver() . . . 1139
10.89.3.3 CloseSqlConnection() . . . 1139

10.89.3.4 ExecuteQuery() . . . 1140
10.89.3.5 executeQuery() . . . 1141
10.89.3.6 executeSelectQuery() . . . 1141
10.89.3.7 ExecuteUpdate() . . . 1142
10.89.3.8 executeUpdate() . . . 1143
10.89.3.9 GeneratePreparedStatement() . . . 1143
10.89.3.10 generatePreparedStatement() . . . 1144
10.89.3.11 GenerateSqlDeleteStr() . . . 1144
10.89.3.12 GenerateSqlSelectStr() . . . 1145
10.89.3.13 GetSqlConnection() . . . 1145
10.89.3.14 getSqlConnection() . . . 1146
10.89.3.15 GetSqlDriver() . . . 1147
10.89.3.16 MsgProcessingQueuePop() . . . 1147
10.89.3.17 MsgProcessingQueuePush() . . . 1148
10.89.3.18 MsgProcessingQueuePushAll() . . . 1148
10.89.3.19 num_threads() . . . 1149
10.89.3.20 operator()() . . . 1149
10.89.3.21 ProcessMailMsg() . . . 1150
10.89.3.22 set_termination_signaled() . . . 1151
10.89.3.23 StartWorkerThreads() . . . 1151
10.89.3.24 TerminateWorkerThreads() . . . 1151
10.89.4 Friends And Related Symbol Documentation . . . 1152
10.89.4.1 util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription . . . 1152
10.89.5 Member Data Documentation . . . 1152
10.89.5.1 address_ . . . 1152
10.89.5.2 msg_processing_queue_ . . . 1152
10.89.5.3 num_threads_ . . . 1152
10.89.5.4 password_ . . . 1153
10.89.5.5 port_ . . . 1153
10.89.5.6 protocol_ . . . 1153
10.89.5.7 schema_ . . . 1153
10.89.5.8 sql_connection_ . . . 1153
10.89.5.9 sql_driver_ . . . 1153
10.89.5.10 termination_signaled_ . . . 1154
10.89.5.11 thread_group_ . . . 1154
10.89.5.12 thread_objects_ . . . 1154
10.89.5.13 user_ . . . 1154
10.90 argo::iridium::xfloat::XFloatMailRetrievalService Class Reference . . . 1155
10.90.1 Detailed Description . . . 1158
10.90.2 Constructor & Destructor Documentation . . . 1158
10.90.2.1 XFloatMailRetrievalService() . . . 1158
10.90.2.2 ∼XFloatMailRetrievalService() . . . 1158
10.90.3 Member Function Documentation . . . 1158
10.90.3.1 CleanupSqlConnection() . . . 1158
10.90.3.2 CleanupSqlDriver() . . . 1159
10.90.3.3 CloseSqlConnection() . . . 1159
10.90.3.4 ConnectToMailServer() . . . 1160
10.90.3.5 DisconnectFromMailServer() . . . 1160
10.90.3.6 ExecuteQuery() . . . 1161
10.90.3.7 executeQuery() . . . 1161
10.90.3.8 executeSelectQuery() . . . 1162
10.90.3.9 ExecuteUpdate() . . . 1163

10.90.3.10 executeUpdate() . . . 1163
10.90.3.11 FetchMailMessagesFromFileSystem() . . . 1164
10.90.3.12 FetchMailMessagesFromServer() . . . 1165
10.90.3.13 GenerateMailServerUrlString() . . . 1165
10.90.3.14 GeneratePreparedStatement() . . . 1165
10.90.3.15 generatePreparedStatement() . . . 1166
10.90.3.16 GenerateSqlDeleteStr() . . . 1167
10.90.3.17 GenerateSqlSelectStr() . . . 1167
10.90.3.18 GetDirectoryPath() . . . 1167
10.90.3.19 GetDirectoryPathFromString() . . . 1168
10.90.3.20 GetSqlConnection() . . . 1169
10.90.3.21 getSqlConnection() . . . 1170
10.90.3.22 GetSqlDriver() . . . 1171
10.90.3.23 ListDirectories() . . . 1171
10.90.3.24 ListFiles() . . . 1171
10.90.3.25 ListRawMailFiles() . . . 1172
10.90.3.26 mail_session() . . . 1173
10.90.3.27 MoveFileToDirectory() . . . 1173
10.90.3.28 OpenMailInboxFolder() . . . 1173
10.90.3.29 operator()() . . . 1174
10.90.3.30 set_termination_signaled() . . . 1175
10.90.3.31 StartWorkerThread() . . . 1175
10.90.3.32 TerminateWorkerThread() . . . 1176
10.90.4 Friends And Related Symbol Documentation . . . 1176
10.90.4.1 util::config::LocalConfiguration::GenerateCommandLineOptionsDescription . . . 1176
10.90.4.2 util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription . . . 1176
10.90.5 Member Data Documentation . . . 1176
10.90.5.1 address_ . . . 1176
10.90.5.2 inbox_folder_ . . . 1177
10.90.5.3 is_reprocess_all_ . . . 1177
10.90.5.4 mail_fetch_mode_ . . . 1177
10.90.5.5 mail_server_host_ . . . 1177
10.90.5.6 mail_server_inbox_folder_path_ . . . 1177
10.90.5.7 mail_server_password_ . . . 1178
10.90.5.8 mail_server_protocol_ . . . 1178
10.90.5.9 mail_server_user_ . . . 1178
10.90.5.10 mail_session_ . . . 1178
10.90.5.11 mail_store_ . . . 1178
10.90.5.12 max_num_msgs_to_fetch_ . . . 1178
10.90.5.13 message_start_index_ . . . 1179
10.90.5.14 move_mail_messages_ . . . 1179
10.90.5.15 password_ . . . 1179
10.90.5.16 port_ . . . 1179
10.90.5.17 processing_thread_propagation_enabled_ . . . 1179
10.90.5.18 protocol_ . . . 1179
10.90.5.19 reprocess_by_inst_ids_ . . . 1180
10.90.5.20 schema_ . . . 1180
10.90.5.21 sleep_seconds_if_no_new_msgs_ . . . 1180
10.90.5.22 sql_connection_ . . . 1180
10.90.5.23 sql_driver_ . . . 1180
10.90.5.24 termination_signaled_ . . . 1181
10.90.5.25 thread_ . . . 1181

10.90.5.26 thread_object_ . . . 1181
10.90.5.27 user_ . . . 1181
10.91 argo::iridium::xfloat::XFloatMetadata Class Reference . . . 1182
10.91.1 Detailed Description . . . 1187
10.91.2 Constructor & Destructor Documentation . . . 1187
10.91.2.1 XFloatMetadata() [1/2] . . . 1187
10.91.2.2 XFloatMetadata() [2/2] . . . 1187
10.91.2.3 ~XFloatMetadata() . . . 1188
10.91.3 Member Function Documentation . . . 1188
10.91.3.1 CalculateEllipsoidHeight() . . . 1188
10.91.3.2 CalculateGeographicDistance() . . . 1189
10.91.3.3 CleanupSqlConnection() . . . 1189
10.91.3.4 CleanupSqlDriver() . . . 1190
10.91.3.5 CloseSqlConnection() . . . 1190
10.91.3.6 comment() . . . 1191
10.91.3.7 degToRad() . . . 1191
10.91.3.8 deployer() . . . 1191
10.91.3.9 deployer_address() . . . 1192
10.91.3.10 deployment_cruise_id() . . . 1192
10.91.3.11 deployment_datetime() . . . 1192
10.91.3.12 deployment_datetime_status() . . . 1192
10.91.3.13 deployment_latitude() . . . 1193
10.91.3.14 deployment_longitude() . . . 1193
10.91.3.15 deployment_platform() . . . 1193
10.91.3.16 deployment_position_status() . . . 1194
10.91.3.17 deployment_type() . . . 1194
10.91.3.18 ExecuteQuery() . . . 1194
10.91.3.19 executeQuery() . . . 1194
10.91.3.20 executeSelectQuery() . . . 1195
10.91.3.21 ExecuteUpdate() . . . 1196
10.91.3.22 executeUpdate() . . . 1197
10.91.3.23 GenerateCurrentUtcTimeIso8601String() . . . 1197
10.91.3.24 GenerateIso6709String() . . . 1198
10.91.3.25 GenerateIso8601String() . . . 1198
10.91.3.26 GenerateJulianDay() . . . 1199
10.91.3.27 GeneratePreparedStatement() . . . 1199
10.91.3.28 generatePreparedStatement() . . . 1200
10.91.3.29 GenerateSqlDeleteStr() . . . 1201
10.91.3.30 GenerateSqlSelectStr() . . . 1201
10.91.3.31 GetCurrentUtcTime() . . . 1202
10.91.3.32 GetDirectoryPath() . . . 1202
10.91.3.33 GetDirectoryPathFromString() . . . 1202
10.91.3.34 GetSqlConnection() . . . 1203
10.91.3.35 getSqlConnection() . . . 1204
10.91.3.36 GetSqlDriver() . . . 1205
10.91.3.37 id_dac() . . . 1205
10.91.3.38 id_imei() . . . 1206
10.91.3.39 id_instrument() . . . 1206
10.91.3.40 id_wmo() . . . 1207
10.91.3.41 IsPositionValid() . . . 1207
10.91.3.42 ListDirectories() . . . 1207
10.91.3.43 ListFiles() . . . 1208

10.91.3.44 ListMetadataFiles() . . . 1208
10.91.3.45 LoadFile() . . . 1209
10.91.3.46 LoadMetadataFromFiles() . . . 1209
10.91.3.47 local_path() . . . 1210
10.91.3.48 LockObject() . . . 1210
10.91.3.49 manufacturer() . . . 1211
10.91.3.50 MoveFileToDirectory() . . . 1211
10.91.3.51 position_datetime() . . . 1212
10.91.3.52 position_datetime_as_iso8601_string() . . . 1212
10.91.3.53 position_is_valid() . . . 1213
10.91.3.54 position_latitude() . . . 1213
10.91.3.55 position_longitude() . . . 1214
10.91.3.56 postition_as_iso6709_string() . . . 1214
10.91.3.57 principal_investigators() . . . 1215
10.91.3.58 principal_investigators_address() . . . 1215
10.91.3.59 principal_investigators_originating_country() . . . 1215
10.91.3.60 SaveObjectToDatabase() . . . 1215
10.91.3.61 set_comment() . . . 1215
10.91.3.62 set_deployer() . . . 1216
10.91.3.63 set_deployer_address() . . . 1216
10.91.3.64 set_deployment_cruise_id() . . . 1216
10.91.3.65 set_deployment_datetime() . . . 1217
10.91.3.66 set_deployment_datetime_status() . . . 1217
10.91.3.67 set_deployment_latitude() . . . 1217
10.91.3.68 set_deployment_longitude() . . . 1218
10.91.3.69 set_deployment_platform() . . . 1219
10.91.3.70 set_deployment_position_status() . . . 1219
10.91.3.71 set_deployment_type() . . . 1219
10.91.3.72 set_gps_position_datetime() . . . 1219
10.91.3.73 set_id_dac() . . . 1220
10.91.3.74 set_id_imei() . . . 1220
10.91.3.75 set_id_instrument() . . . 1221
10.91.3.76 set_id_wmo() . . . 1222
10.91.3.77 set_local_path() . . . 1222
10.91.3.78 set_manufacturer() . . . 1223
10.91.3.79 set_position_datetime() . . . 1223
10.91.3.80 set_position_is_valid() . . . 1223
10.91.3.81 set_position_latitude() . . . 1224
10.91.3.82 set_position_longitude() . . . 1224
10.91.3.83 set_principal_investigators() . . . 1225
10.91.3.84 set_principal_investigators_address() . . . 1225
10.91.3.85 set_principal_investigators_originating_country() . . . 1225
10.91.3.86 set_wmo_instrument_type() . . . 1226
10.91.3.87 set_wmo_recorder_type() . . . 1226
10.91.3.88 UnlockObject() . . . 1226
10.91.3.89 wmo_instrument_type() . . . 1226
10.91.3.90 wmo_recorder_type() . . . 1227
10.91.4 Friends And Related Symbol Documentation . . . 1227
10.91.4.1 operator>> . . . 1227
10.91.4.2 util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription . . . 1227
10.91.5 Member Data Documentation . . . 1227
10.91.5.1 address_ . . . 1227

10.91.5.2 comment_ . . . 1227
10.91.5.3 deployer_ . . . 1228
10.91.5.4 deployer_address_ . . . 1228
10.91.5.5 deployment_cruiseId_ . . . 1228
10.91.5.6 deployment_datetime_status_ . . . 1228
10.91.5.7 deployment_platform_ . . . 1228
10.91.5.8 deployment_position_status_ . . . 1228
10.91.5.9 deployment_type_ . . . 1228
10.91.5.10 EARTH_RADIUS_KM_EQUATORIAL . . . 1229
10.91.5.11 EARTH_RADIUS_KM_POLAR . . . 1229
10.91.5.12 id_dac_ . . . 1229
10.91.5.13 id_imei_ . . . 1229
10.91.5.14 id_instrument_ . . . 1229
10.91.5.15 id_wmo_ . . . 1229
10.91.5.16 local_path_ . . . 1229
10.91.5.17 manufacturer_ . . . 1230
10.91.5.18 metadata_directory_path_ . . . 1230
10.91.5.19 mutex_ . . . 1230
10.91.5.20 password_ . . . 1231
10.91.5.21 port_ . . . 1232
10.91.5.22 position_datetime_ . . . 1232
10.91.5.23 position_is_valid_ . . . 1232
10.91.5.24 position_latitude_ . . . 1232
10.91.5.25 position_longitude_ . . . 1232
10.91.5.26 principal_investigators_ . . . 1232
10.91.5.27 principal_investigators_address_ . . . 1232
10.91.5.28 principal_investigators_originating_country_ . . . 1233
10.91.5.29 protocol_ . . . 1233
10.91.5.30 schema_ . . . 1233
10.91.5.31 sql_connection_ . . . 1233
10.91.5.32 sql_driver_ . . . 1233
10.91.5.33 sql_table_description_ . . . 1233
10.91.5.34 user_ . . . 1234
10.91.5.35 wmo_instrument_type_ . . . 1234
10.91.5.36 wmo_recorder_type_ . . . 1234
10.92 argo::iridium::xfloat::XFloatPostProcessingService Class Reference . . . 1234
10.92.1 Detailed Description . . . 1236
10.92.2 Constructor & Destructor Documentation . . . 1236
10.92.2.1 XFloatPostProcessingService() . . . 1236
10.92.2.2 ∼XFloatPostProcessingService() . . . 1236
10.92.3 Member Function Documentation . . . 1236
10.92.3.1 CloseKmlFactory() . . . 1236
10.92.3.2 GetDefaltKmlPathColor() . . . 1236
10.92.3.3 GetDefaltKmlPathStyleId() . . . 1237
10.92.3.4 GetDefaltKmlPlacemarkStyleId() . . . 1237
10.92.3.5 GetKmlFactory() . . . 1237
10.92.3.6 GetSmallKmlPlacemarkStyleId() . . . 1238
10.92.3.7 GetTriangleKmlPlacemarkStyleId() . . . 1238
10.92.3.8 operator()() . . . 1239
10.92.3.9 SetAllDefaultKmlSyles() . . . 1239
10.92.3.10 SetDefaultKmlStyle() . . . 1240
10.92.3.11 StartWorkerThread() . . . 1241

10.92.3.12 TerminateWorkerThread() . . . 1241
10.92.4 Friends And Related Symbol Documentation . . . 1241
10.92.4.1 util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription . . . 1241
10.92.5 Member Data Documentation . . . 1241
10.92.5.1 idle_sleep_seconds_ . . . 1241
10.92.5.2 kml_default_path_colors_ . . . 1242
10.92.5.3 kml_default_path_style_ids_ . . . 1242
10.92.5.4 kml_default_placemark_style_ids_ . . . 1242
10.92.5.5 kml_factory_ . . . 1242
10.92.5.6 kml_small_placemark_style_ids_ . . . 1243
10.92.5.7 kml_triangle_placemark_style_ids_ . . . 1243
10.92.5.8 termination_signaled_ . . . 1243
10.92.5.9 thread_ . . . 1243
10.92.5.10 thread_object_ . . . 1244
10.93 argo::iridium::xfloat::XFloatTelemetryDecoderService Class Reference . . . 1244
10.93.1 Detailed Description . . . 1246
10.93.2 Constructor & Destructor Documentation . . . 1246
10.93.2.1 XFloatTelemetryDecoderService() . . . 1246
10.93.2.2 ∼XFloatTelemetryDecoderService() . . . 1246
10.93.3 Member Function Documentation . . . 1246
10.93.3.1 CleanupSqlConnection() . . . 1246
10.93.3.2 CleanupSqlDriver() . . . 1246
10.93.3.3 CloseSqlConnection() . . . 1247
10.93.3.4 DecodeMessage() . . . 1248
10.93.3.5 ExecuteQuery() . . . 1249
10.93.3.6 ExecuteUpdate() . . . 1249
10.93.3.7 GeneratePreparedStatement() . . . 1250
10.93.3.8 GenerateSqlDeleteStr() . . . 1250
10.93.3.9 GenerateSqlSelectStr() . . . 1251
10.93.3.10 GetSqlConnection() . . . 1251
10.93.3.11 GetSqlDriver() . . . 1252
10.93.3.12 MsgDecodingQueuePush() . . . 1253
10.93.3.13 MsgProcessingQueuePop() . . . 1253
10.93.3.14 num_threads() . . . 1254
10.93.3.15 operator()() . . . 1254
10.93.3.16 ProcessDiveData() . . . 1254
10.93.3.17 set_termination_signaled() . . . 1255
10.93.3.18 StartWorkerThreads() . . . 1255
10.93.3.19 TerminateWorkerThreads() . . . 1256
10.93.4 Friends And Related Symbol Documentation . . . 1256
10.93.4.1 util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription . . . 1256
10.93.5 Member Data Documentation . . . 1256
10.93.5.1 msg_decoding_queue_ . . . 1256
10.93.5.2 num_threads_ . . . 1257
10.93.5.3 sql_connection_ . . . 1257
10.93.5.4 sql_driver_ . . . 1257
10.93.5.5 termination_signaled_ . . . 1257
10.93.5.6 thread_group_ . . . 1257
10.93.5.7 thread_objects_ . . . 1258
10.94 util::xmsg::XGPS Class Reference . . . 1258
10.94.1 Detailed Description . . . 1263
10.94.2 Constructor & Destructor Documentation . . . 1263

10.94.2.1 XGPS() [1/2] . . . 1263
10.94.2.2 XGPS() [2/2] . . . 1263
10.94.2.3 ~XGPS() . . . 1264
10.94.3 Member Function Documentation . . . 1264
10.94.3.1 CalculateEllipsoidHeight() . . . 1264
10.94.3.2 CalculateGeographicDistance() . . . 1264
10.94.3.3 CheckXMsgData() . . . 1265
10.94.3.4 CleanupSqlConnection() . . . 1266
10.94.3.5 CleanupSqlDriver() . . . 1266
10.94.3.6 CloseKmlFactory() . . . 1267
10.94.3.7 CloseSqlConnection() . . . 1267
10.94.3.8 DecodeXMsgData() . . . 1267
10.94.3.9 degToRad() . . . 1268
10.94.3.10 DeleteFromDatabase() [1/2] . . . 1269
10.94.3.11 DeleteFromDatabase() [2/2] . . . 1269
10.94.3.12 DeleteObjectsFromDatabase() . . . 1269
10.94.3.13 ExecuteQuery() . . . 1270
10.94.3.14 executeQuery() . . . 1270
10.94.3.15 executeSelectQuery() . . . 1271
10.94.3.16 ExecuteUpdate() . . . 1272
10.94.3.17 executeUpdate() . . . 1273
10.94.3.18 GenerateCurrentUtcTimeIso8601String() . . . 1273
10.94.3.19 GenerateIso6709String() . . . 1274
10.94.3.20 GenerateIso8601String() . . . 1274
10.94.3.21 GenerateJulianDay() . . . 1275
10.94.3.22 GenerateListingComment() . . . 1276
10.94.3.23 GenerateListingString() . . . 1276
10.94.3.24 GeneratePhyString() . . . 1276
10.94.3.25 GeneratePreparedStatement() . . . 1277
10.94.3.26 generatePreparedStatement() . . . 1277
10.94.3.27 GenerateSqlDeleteStr() . . . 1278
10.94.3.28 GenerateSqlSelectStr() . . . 1278
10.94.3.29 GetCurrentUtcTime() . . . 1279
10.94.3.30 GetDefaltKmlPathColor() . . . 1279
10.94.3.31 GetDefaltKmlPathStyleId() . . . 1280
10.94.3.32 GetDefaltKmlPlacemarkStyleId() . . . 1280
10.94.3.33 GetKmlFactory() . . . 1281
10.94.3.34 GetSmallKmlPlacemarkStyleId() . . . 1282
10.94.3.35 GetSqlConnection() . . . 1282
10.94.3.36 getSqlConnection() . . . 1283
10.94.3.37 GetSqlDriver() . . . 1284
10.94.3.38 GetTriangleKmlPlacemarkStyleId() . . . 1284
10.94.3.39 id_dive() . . . 1285
10.94.3.40 id_instrument() . . . 1286
10.94.3.41 id_xmsg_type() . . . 1287
10.94.3.42 IsPositionValid() . . . 1288
10.94.3.43 position_datetime() . . . 1288
10.94.3.44 position_datetime_as_iso8601_string() . . . 1289
10.94.3.45 position_is_valid() . . . 1289
10.94.3.46 position_latitude() . . . 1290
10.94.3.47 position_longitude() . . . 1290
10.94.3.48 postition_as_iso6709_string() . . . 1291

10.94.3.49 SaveObjectToDatabase() . . . 1291
10.94.3.50 SelectAllObjectsFromDatabase() . . . 1291
10.94.3.51 SelectObjectFromDatabase() . . . 1292
10.94.3.52 SelectObjectsFromDatabase() . . . 1292
10.94.3.53 SelectValueFromDatabase() . . . 1293
10.94.3.54 set_gps_position_datetime() . . . 1294
10.94.3.55 set_id_dive() . . . 1294
10.94.3.56 set_id_instrument() . . . 1295
10.94.3.57 set_id_xmsg_type() . . . 1295
10.94.3.58 set_position_datetime() . . . 1296
10.94.3.59 set_position_is_valid() . . . 1297
10.94.3.60 set_position_latitude() . . . 1297
10.94.3.61 set_position_longitude() . . . 1298
10.94.3.62 set_xmsg_size_bytes() . . . 1298
10.94.3.63 SetAllDefaultKmlSyles() . . . 1299
10.94.3.64 SetDefaultKmlStyle() . . . 1300
10.94.3.65 xmsg_size_bytes() . . . 1301
10.94.4 Friends And Related Symbol Documentation . . . 1302
10.94.4.1 std::make_shared . . . 1302
10.94.5 Member Data Documentation . . . 1302
10.94.5.1 address_ . . . 1302
10.94.5.2 EARTH_RADIUS_KM_EQUATORIAL . . . 1302
10.94.5.3 EARTH_RADIUS_KM_POLAR . . . 1302
10.94.5.4 horizontal_dilution_of_precision_ . . . 1302
10.94.5.5 id_dive_ . . . 1302
10.94.5.6 id_instrument_ . . . 1302
10.94.5.7 id_xmsg_type_ . . . 1303
10.94.5.8 kml_default_path_colors_ . . . 1303
10.94.5.9 kml_default_path_style_ids_ . . . 1303
10.94.5.10 kml_default_placemark_style_ids_ . . . 1303
10.94.5.11 kml_factory_ . . . 1304
10.94.5.12 kml_small_placemark_style_ids_ . . . 1304
10.94.5.13 kml_triangle_placemark_style_ids_ . . . 1304
10.94.5.14 num_satellites_ . . . 1304
10.94.5.15 password_ . . . 1305
10.94.5.16 port_ . . . 1305
10.94.5.17 position_datetime_ . . . 1305
10.94.5.18 position_is_valid_ . . . 1305
10.94.5.19 position_latitude_ . . . 1305
10.94.5.20 position_longitude_ . . . 1305
10.94.5.21 protocol_ . . . 1305
10.94.5.22 schema_ . . . 1306
10.94.5.23 signal_to_noise_max_ . . . 1306
10.94.5.24 signal_to_noise_mean_ . . . 1306
10.94.5.25 signal_to_noise_min_ . . . 1306
10.94.5.26 sql_connection_ . . . 1306
10.94.5.27 sql_driver_ . . . 1306
10.94.5.28 time_to_acquire_fix_ . . . 1306
10.94.5.29 user_ . . . 1307
10.94.5.30 xmsg_size_bytes_ . . . 1307
10.95 util::xmsg::XHiResPressure Class Reference . . . 1307
10.95.1 Detailed Description . . . 1309

10.95.2 Constructor & Destructor Documentation . . . 1309
10.95.2.1 XHiResPressure() . . . 1309
10.95.2.2 ~XHiResPressure() . . . 1310
10.95.3 Member Function Documentation . . . 1310
10.95.3.1 CheckXMsgData() . . . 1310
10.95.3.2 DecodeXMsgData() . . . 1310
10.95.3.3 id_dive() . . . 1311
10.95.3.4 id_instrument() . . . 1312
10.95.3.5 id_xmsg_type() . . . 1313
10.95.3.6 num_samples() . . . 1314
10.95.3.7 raw_sensor_data() . . . 1314
10.95.3.8 Sample() . . . 1314
10.95.3.9 SampleValue() . . . 1314
10.95.3.10 SampleValueAsPressure() . . . 1315
10.95.3.11 SampleValueAsSalinity() . . . 1315
10.95.3.12 SampleValueAsTemperature() . . . 1316
10.95.3.13 sensor_data_format() . . . 1317
10.95.3.14 set_id_dive() . . . 1317
10.95.3.15 set_id_instrument() . . . 1317
10.95.3.16 set_id_xmsg_type() . . . 1318
10.95.3.17 set_raw_sensor_data() . . . 1319
10.95.3.18 set_sensor_data_format() . . . 1319
10.95.3.19 set_xmsg_index() . . . 1320
10.95.3.20 set_xmsg_size_bytes() . . . 1320
10.95.3.21 UnpackSensorData() . . . 1322
10.95.3.22 xmsg_index() . . . 1323
10.95.3.23 xmsg_size_bytes() . . . 1323
10.95.4 Member Data Documentation . . . 1324
10.95.4.1 id_dive_ . . . 1324
10.95.4.2 id_instrument_ . . . 1324
10.95.4.3 id_xmsg_type_ . . . 1325
10.95.4.4 raw_sensor_data_ . . . 1325
10.95.4.5 sensor_data_format_ . . . 1325
10.95.4.6 xmsg_index_ . . . 1325
10.95.4.7 xmsg_size_bytes_ . . . 1325
10.96 util::xmsg::XHiResSalinity Class Reference . . . 1326
10.96.1 Detailed Description . . . 1327
10.96.2 Constructor & Destructor Documentation . . . 1327
10.96.2.1 XHiResSalinity() . . . 1327
10.96.2.2 ~XHiResSalinity() . . . 1328
10.96.3 Member Function Documentation . . . 1328
10.96.3.1 CheckXMsgData() . . . 1328
10.96.3.2 DecodeXMsgData() . . . 1329
10.96.3.3 id_dive() . . . 1330
10.96.3.4 id_instrument() . . . 1331
10.96.3.5 id_xmsg_type() . . . 1331
10.96.3.6 num_samples() . . . 1332
10.96.3.7 raw_sensor_data() . . . 1332
10.96.3.8 Sample() . . . 1332
10.96.3.9 SampleValue() . . . 1333
10.96.3.10 SampleValueAsPressure() . . . 1333
10.96.3.11 SampleValueAsSalinity() . . . 1334

10.96.3.12 SampleValueAsTemperature() . . . 1334
10.96.3.13 sensor_data_format() . . . 1335
10.96.3.14 set_id_dive() . . . 1335
10.96.3.15 set_id_instrument() . . . 1335
10.96.3.16 set_id_xmsg_type() . . . 1336
10.96.3.17 set_raw_sensor_data() . . . 1337
10.96.3.18 set_sensor_data_format() . . . 1338
10.96.3.19 set_xmsg_index() . . . 1338
10.96.3.20 set_xmsg_size_bytes() . . . 1339
10.96.3.21 UnpackSensorData() . . . 1340
10.96.3.22 xmsg_index() . . . 1341
10.96.3.23 xmsg_size_bytes() . . . 1342
10.96.4 Member Data Documentation . . . 1343
10.96.4.1 id_dive_ . . . 1343
10.96.4.2 id_instrument_ . . . 1343
10.96.4.3 id_xmsg_type_ . . . 1343
10.96.4.4 raw_sensor_data_ . . . 1344
10.96.4.5 sensor_data_format_ . . . 1344
10.96.4.6 xmsg_index_ . . . 1344
10.96.4.7 xmsg_size_bytes_ . . . 1344
10.97 util::xmsg::XHiResTemperature Class Reference . . . 1345
10.97.1 Detailed Description . . . 1346
10.97.2 Constructor & Destructor Documentation . . . 1346
10.97.2.1 XHiResTemperature() . . . 1346
10.97.2.2 ∼XHiResTemperature() . . . 1347
10.97.3 Member Function Documentation . . . 1347
10.97.3.1 CheckXMsgData() . . . 1347
10.97.3.2 DecodeXMsgData() . . . 1348
10.97.3.3 id_dive() . . . 1349
10.97.3.4 id_instrument() . . . 1350
10.97.3.5 id_xmsg_type() . . . 1350
10.97.3.6 num_samples() . . . 1351
10.97.3.7 raw_sensor_data() . . . 1351
10.97.3.8 Sample() . . . 1351
10.97.3.9 SampleValue() . . . 1352
10.97.3.10 SampleValueAsPressure() . . . 1352
10.97.3.11 SampleValueAsSalinity() . . . 1353
10.97.3.12 SampleValueAsTemperature() . . . 1353
10.97.3.13 sensor_data_format() . . . 1354
10.97.3.14 set_id_dive() . . . 1354
10.97.3.15 set_id_instrument() . . . 1354
10.97.3.16 set_id_xmsg_type() . . . 1355
10.97.3.17 set_raw_sensor_data() . . . 1356
10.97.3.18 set_sensor_data_format() . . . 1357
10.97.3.19 set_xmsg_index() . . . 1357
10.97.3.20 set_xmsg_size_bytes() . . . 1358
10.97.3.21 UnpackSensorData() . . . 1359
10.97.3.22 xmsg_index() . . . 1360
10.97.3.23 xmsg_size_bytes() . . . 1361
10.97.4 Member Data Documentation . . . 1362
10.97.4.1 id_dive_ . . . 1362
10.97.4.2 id_instrument_ . . . 1362

10.97.4.3 id_xmsg_type_ . . . 1362
10.97.4.4 raw_sensor_data_ . . . 1363
10.97.4.5 sensor_data_format_ . . . 1363
10.97.4.6 xmsg_index_ . . . 1363
10.97.4.7 xmsg_size_bytes_ . . . 1363
10.98 util::xmsg::XMessage Class Reference . . . 1364
10.98.1 Detailed Description . . . 1365
10.98.2 Constructor & Destructor Documentation . . . 1365
10.98.2.1 XMessage() [1/2] . . . 1365
10.98.2.2 XMessage() [2/2] . . . 1365
10.98.2.3 ~XMessage() . . . 1366
10.98.3 Member Function Documentation . . . 1366
10.98.3.1 CheckXMsgData() . . . 1366
10.98.3.2 DecodeXMsgData() . . . 1367
10.98.3.3 id_dive() . . . 1367
10.98.3.4 id_instrument() . . . 1368
10.98.3.5 id_xmsg_type() . . . 1369
10.98.3.6 set_id_dive() . . . 1370
10.98.3.7 set_id_instrument() . . . 1370
10.98.3.8 set_id_xmsg_type() . . . 1370
10.98.3.9 set_xmsg_size_bytes() . . . 1371
10.98.3.10 xmsg_size_bytes() . . . 1373
10.98.4 Member Data Documentation . . . 1373
10.98.4.1 id_dive_ . . . 1373
10.98.4.2 id_instrument_ . . . 1374
10.98.4.3 id_xmsg_type_ . . . 1374
10.98.4.4 xmsg_size_bytes_ . . . 1374
10.99 util::xmsg::XMessageIndexable Class Reference . . . 1374
10.99.1 Detailed Description . . . 1375
10.99.2 Constructor & Destructor Documentation . . . 1375
10.99.2.1 XMessageIndexable() [1/2] . . . 1375
10.99.2.2 XMessageIndexable() [2/2] . . . 1376
10.99.2.3 ~XMessageIndexable() . . . 1376
10.99.3 Member Function Documentation . . . 1376
10.99.3.1 CheckXMsgData() . . . 1376
10.99.3.2 DecodeXMsgData() . . . 1377
10.99.3.3 id_dive() . . . 1377
10.99.3.4 id_instrument() . . . 1378
10.99.3.5 id_xmsg_type() . . . 1379
10.99.3.6 set_id_dive() . . . 1380
10.99.3.7 set_id_instrument() . . . 1380
10.99.3.8 set_id_xmsg_type() . . . 1380
10.99.3.9 set_xmsg_index() . . . 1381
10.99.3.10 set_xmsg_size_bytes() . . . 1382
10.99.3.11 xmsg_index() . . . 1383
10.99.3.12 xmsg_size_bytes() . . . 1384
10.99.4 Member Data Documentation . . . 1385
10.99.4.1 id_dive_ . . . 1385
10.99.4.2 id_instrument_ . . . 1385
10.99.4.3 id_xmsg_type_ . . . 1385
10.99.4.4 xmsg_index_ . . . 1386
10.99.4.5 xmsg_size_bytes_ . . . 1386

10.100 util::xmsg::XMfaMissionCfg Class Reference . . . 1386
10.100.1 Detailed Description . . . 1398
10.100.2 Constructor & Destructor Documentation . . . 1398
10.100.2.1 XMfaMissionCfg() . . . 1398
10.100.2.2 ~XMfaMissionCfg() . . . 1398
10.100.3 Member Function Documentation . . . 1399
10.100.3.1 AssignMessage() . . . 1399
10.100.3.2 DecodeXMsgData() . . . 1400
10.100.3.3 GenerateCurrentUtcTimeIso8601String() . . . 1400
10.100.3.4 GenerateIso8601String() . . . 1401
10.100.3.5 GenerateJulianDay() . . . 1402
10.100.3.6 GenerateListingString() . . . 1402
10.100.3.7 GetCurrentUtcTime() . . . 1403
10.100.4 Member Data Documentation . . . 1403
10.100.4.1 aca_ . . . 1403
10.100.4.2 acg_ . . . 1403
10.100.4.3 acr_ . . . 1404
10.100.4.4 add_ . . . 1404
10.100.4.5 adt_ . . . 1404
10.100.4.6 apf_ . . . 1404
10.100.4.7 apn_ . . . 1404
10.100.4.8 apt_ . . . 1404
10.100.4.9 arr_ . . . 1404
10.100.4.10 ascent_start_time_ . . . 1404
10.100.4.11 asd_ . . . 1404
10.100.4.12 asi_ . . . 1405
10.100.4.13 asn_ . . . 1405
10.100.4.14 btm_ . . . 1405
10.100.4.15 cms_ . . . 1405
10.100.4.16 cpb_ . . . 1405
10.100.4.17 cpn_ . . . 1405
10.100.4.18 cpp_ . . . 1405
10.100.4.19 cps_ . . . 1405
10.100.4.20 crd_ . . . 1405
10.100.4.21 crs_ . . . 1406
10.100.4.22 csp_ . . . 1406
10.100.4.23 cst_ . . . 1406
10.100.4.24 ctc_ . . . 1406
10.100.4.25 ctf_ . . . 1406
10.100.4.26 ctl_ . . . 1406
10.100.4.27 ctu_ . . . 1407
10.100.4.28 ddb_ . . . 1407
10.100.4.29 ddd_ . . . 1407
10.100.4.30 ddt_ . . . 1407
10.100.4.31 descend_start_time_ . . . 1407
10.100.4.32 dft_ . . . 1407
10.100.4.33 dns_ . . . 1407
10.100.4.34 dsd_ . . . 1407
10.100.4.35 dsi_ . . . 1408
10.100.4.36 dst_ . . . 1408
10.100.4.37 dtd_ . . . 1408
10.100.4.38 fw_version_ . . . 1408

10.100.4.39 iat_ . . . 1408
10.100.4.40 ica_ . . . 1408
10.100.4.41 icb_ . . . 1408
10.100.4.42 icd_ . . . 1408
10.100.4.43 icf_ . . . 1408
10.100.4.44 icg_ . . . 1409
10.100.4.45 icm_ . . . 1409
10.100.4.46 icn_ . . . 1409
10.100.4.47 ico_ . . . 1409
10.100.4.48 icr_ . . . 1409
10.100.4.49 ics_ . . . 1409
10.100.4.50 ict_ . . . 1409
10.100.4.51 icw_ . . . 1409
10.100.4.52 icx_ . . . 1409
10.100.4.53 ilt_ . . . 1410
10.100.4.54 imm_ . . . 1410
10.100.4.55 imt_ . . . 1410
10.100.4.56 isa_ . . . 1410
10.100.4.57 isd_ . . . 1410
10.100.4.58 ism_ . . . 1410
10.100.4.59 iso_ . . . 1410
10.100.4.60 isp_ . . . 1410
10.100.4.61 iss_ . . . 1410
10.100.4.62 ist_ . . . 1411
10.100.4.63 isv_ . . . 1411
10.100.4.64 isw_ . . . 1411
10.100.4.65 iue_ . . . 1411
10.100.4.66 mam_ . . . 1411
10.100.4.67 mat_ . . . 1411
10.100.4.68 mds_ . . . 1411
10.100.4.69 mnm_ . . . 1411
10.100.4.70 mns_ . . . 1411
10.100.4.71 mpc_ . . . 1412
10.100.4.72 mps_ . . . 1412
10.100.4.73 msg_data_ . . . 1412
10.100.4.74 msg_version_ . . . 1412
10.100.4.75 msn_ . . . 1412
10.100.4.76 mtd_ . . . 1412
10.100.4.77 mtt_ . . . 1412
10.100.4.78 park_start_time_ . . . 1412
10.100.4.79 pdt_ . . . 1412
10.100.4.80 pfd_ . . . 1413
10.100.4.81 pfm_ . . . 1413
10.100.4.82 pns_ . . . 1413
10.100.4.83 psa_ . . . 1413
10.100.4.84 psi_ . . . 1413
10.100.4.85 pth_ . . . 1413
10.100.4.86 pto_ . . . 1413
10.100.4.87 sbm_ . . . 1413
10.100.4.88 sbt_ . . . 1413
10.100.4.89 sdc_ . . . 1414
10.100.4.90 sds_ . . . 1414

10.100.4.91 sensor_type_ . . . 1414
10.100.4.92 set_ . . . 1414
10.100.4.93 sfn_ . . . 1414
10.100.4.94 sgt_ . . . 1414
10.100.4.95 ssi_ . . . 1414
10.100.4.96 surface_start_time_ . . . 1414
10.100.4.97 wmt_ . . . 1414
10.100.4.98 xas_ . . . 1415
10.101 util::xmsg::XMissionCmdEcho Class Reference . . . 1415
10.101.1 Detailed Description . . . 1416
10.101.2 Constructor & Destructor Documentation . . . 1416
10.101.2.1 XMissionCmdEcho() . . . 1416
10.101.2.2 ∼XMissionCmdEcho() . . . 1417
10.101.3 Member Function Documentation . . . 1417
10.101.3.1 CheckXMsgData() . . . 1417
10.101.3.2 DecodeXMsgData() . . . 1417
10.101.3.3 GenerateListingString() . . . 1418
10.101.3.4 id_dive() . . . 1419
10.101.3.5 id_instrument() . . . 1420
10.101.3.6 id_xmsg_type() . . . 1420
10.101.3.7 set_id_dive() . . . 1421
10.101.3.8 set_id_instrument() . . . 1421
10.101.3.9 set_id_xmsg_type() . . . 1421
10.101.3.10 set_xmsg_size_bytes() . . . 1422
10.101.3.11 xmsg_size_bytes() . . . 1424
10.101.4 Member Data Documentation . . . 1424
10.101.4.1 id_dive_ . . . 1424
10.101.4.2 id_instrument_ . . . 1425
10.101.4.3 id_xmsg_type_ . . . 1425
10.101.4.4 xmsg_size_bytes_ . . . 1425
10.102 util::xmsg::XMissionConfiguration Class Reference . . . 1425
10.102.1 Detailed Description . . . 1426
10.102.2 Constructor & Destructor Documentation . . . 1426
10.102.2.1 XMissionConfiguration() . . . 1426
10.102.2.2 ∼XMissionConfiguration() . . . 1427
10.102.3 Member Function Documentation . . . 1427
10.102.3.1 CheckXMsgData() . . . 1427
10.102.3.2 DecodeXMsgData() . . . 1428
10.102.3.3 GenerateListingString() . . . 1429
10.102.3.4 id_dive() . . . 1429
10.102.3.5 id_instrument() . . . 1430
10.102.3.6 id_xmsg_type() . . . 1431
10.102.3.7 set_id_dive() . . . 1432
10.102.3.8 set_id_instrument() . . . 1432
10.102.3.9 set_id_xmsg_type() . . . 1432
10.102.3.10 set_xmsg_index() . . . 1433
10.102.3.11 set_xmsg_size_bytes() . . . 1434
10.102.3.12 xmsg_index() . . . 1435
10.102.3.13 xmsg_size_bytes() . . . 1436
10.102.4 Member Data Documentation . . . 1437
10.102.4.1 alamo_mission_configuration_ . . . 1437
10.102.4.2 id_dive_ . . . 1437

10.102.4.3 id_instrument_ . . . 1438
10.102.4.4 id_xmsg_type_ . . . 1438
10.102.4.5 xmsg_index_ . . . 1438
10.102.4.6 xmsg_size_bytes_ . . . 1438
10.103 util::xmsg::XPresCodeCounts Class Reference . . . 1438
10.103.1 Detailed Description . . . 1439
10.103.2 Constructor & Destructor Documentation . . . 1439
10.103.2.1 XPresCodeCounts() . . . 1439
10.103.2.2 ~XPresCodeCounts() . . . 1440
10.103.3 Member Function Documentation . . . 1440
10.103.3.1 elapsed_seconds_since_reference_time() . . . 1440
10.103.3.2 GenerateListingString() . . . 1440
10.103.3.3 GetPhaseCodeName() . . . 1441
10.103.3.4 phase_code() . . . 1441
10.103.3.5 phase_code_name() . . . 1442
10.103.3.6 PressureValue() . . . 1442
10.103.4 Member Data Documentation . . . 1443
10.103.4.1 elapsed_seconds_since_reference_time_ . . . 1443
10.103.4.2 phase_code_ . . . 1443
10.103.4.3 phase_code_names_ . . . 1443
10.103.4.4 pressure_counts_ . . . 1443
10.104 util::xmsg::XPresCounts Class Reference . . . 1444
10.104.1 Detailed Description . . . 1445
10.104.2 Constructor & Destructor Documentation . . . 1445
10.104.2.1 XPresCounts() . . . 1445
10.104.2.2 ~XPresCounts() . . . 1445
10.104.3 Member Function Documentation . . . 1445
10.104.3.1 elapsed_seconds_since_reference_time() . . . 1445
10.104.3.2 GenerateListingString() . . . 1445
10.104.3.3 PressureValue() . . . 1446
10.104.4 Member Data Documentation . . . 1447
10.104.4.1 elapsed_seconds_since_reference_time_ . . . 1447
10.104.4.2 pressure_counts_ . . . 1447
10.105 util::xmsg::XProfilePressure Class Reference . . . 1447
10.105.1 Detailed Description . . . 1449
10.105.2 Constructor & Destructor Documentation . . . 1449
10.105.2.1 XProfilePressure() . . . 1449
10.105.2.2 ~XProfilePressure() . . . 1450
10.105.3 Member Function Documentation . . . 1450
10.105.3.1 CheckXMsgData() . . . 1450
10.105.3.2 DecodeXMsgData() . . . 1450
10.105.3.3 id_dive() . . . 1451
10.105.3.4 id_instrument() . . . 1452
10.105.3.5 id_xmsg_type() . . . 1453
10.105.3.6 num_samples() . . . 1454
10.105.3.7 raw_sensor_data() . . . 1454
10.105.3.8 Sample() . . . 1454
10.105.3.9 SampleValue() . . . 1454
10.105.3.10 SampleValueAsPressure() . . . 1455
10.105.3.11 SampleValueAsSalinity() . . . 1455
10.105.3.12 SampleValueAsTemperature() . . . 1456
10.105.3.13 sensor_data_format() . . . 1457

10.105.3.14 set_id_dive() . . . 1457
10.105.3.15 set_id_instrument() . . . 1457
10.105.3.16 set_id_xmsg_type() . . . 1458
10.105.3.17 set_raw_sensor_data() . . . 1459
10.105.3.18 set_sensor_data_format() . . . 1459
10.105.3.19 set_xmsg_index() . . . 1460
10.105.3.20 set_xmsg_size_bytes() . . . 1460
10.105.3.21 UnpackSensorData() . . . 1462
10.105.3.22 xmsg_index() . . . 1463
10.105.3.23 xmsg_size_bytes() . . . 1463
10.105.4 Member Data Documentation . . . 1464
10.105.4.1 id_dive_ . . . 1464
10.105.4.2 id_instrument_ . . . 1464
10.105.4.3 id_xmsg_type_ . . . 1465
10.105.4.4 raw_sensor_data_ . . . 1465
10.105.4.5 sensor_data_format_ . . . 1465
10.105.4.6 xmsg_index_ . . . 1465
10.105.4.7 xmsg_size_bytes_ . . . 1465
10.106 util::xmsg::XProfileSalinity Class Reference . . . 1466
10.106.1 Detailed Description . . . 1467
10.106.2 Constructor & Destructor Documentation . . . 1467
10.106.2.1 XProfileSalinity() . . . 1467
10.106.2.2 ∼XProfileSalinity() . . . 1468
10.106.3 Member Function Documentation . . . 1468
10.106.3.1 CheckXMsgData() . . . 1468
10.106.3.2 DecodeXMsgData() . . . 1469
10.106.3.3 id_dive() . . . 1470
10.106.3.4 id_instrument() . . . 1471
10.106.3.5 id_xmsg_type() . . . 1471
10.106.3.6 num_samples() . . . 1472
10.106.3.7 raw_sensor_data() . . . 1472
10.106.3.8 Sample() . . . 1472
10.106.3.9 SampleValue() . . . 1473
10.106.3.10 SampleValueAsPressure() . . . 1473
10.106.3.11 SampleValueAsSalinity() . . . 1474
10.106.3.12 SampleValueAsTemperature() . . . 1474
10.106.3.13 sensor_data_format() . . . 1475
10.106.3.14 set_id_dive() . . . 1475
10.106.3.15 set_id_instrument() . . . 1475
10.106.3.16 set_id_xmsg_type() . . . 1476
10.106.3.17 set_raw_sensor_data() . . . 1477
10.106.3.18 set_sensor_data_format() . . . 1478
10.106.3.19 set_xmsg_index() . . . 1478
10.106.3.20 set_xmsg_size_bytes() . . . 1479
10.106.3.21 UnpackSensorData() . . . 1480
10.106.3.22 xmsg_index() . . . 1481
10.106.3.23 xmsg_size_bytes() . . . 1482
10.106.4 Member Data Documentation . . . 1483
10.106.4.1 id_dive_ . . . 1483
10.106.4.2 id_instrument_ . . . 1483
10.106.4.3 id_xmsg_type_ . . . 1483
10.106.4.4 raw_sensor_data_ . . . 1484

10.106.4.5 sensor_data_format_ . . . 1484
10.106.4.6 xmsg_index_ . . . 1484
10.106.4.7 xmsg_size_bytes_ . . . 1484
10.107 util::xmsg::XProfileTemperature Class Reference . . . 1485
10.107.1 Detailed Description . . . 1486
10.107.2 Constructor & Destructor Documentation . . . 1486
10.107.2.1 XProfileTemperature() . . . 1486
10.107.2.2 ~XProfileTemperature() . . . 1487
10.107.3 Member Function Documentation . . . 1487
10.107.3.1 CheckXMsgData() . . . 1487
10.107.3.2 DecodeXMsgData() . . . 1488
10.107.3.3 id_dive() . . . 1489
10.107.3.4 id_instrument() . . . 1490
10.107.3.5 id_xmsg_type() . . . 1490
10.107.3.6 num_samples() . . . 1491
10.107.3.7 raw_sensor_data() . . . 1491
10.107.3.8 Sample() . . . 1491
10.107.3.9 SampleValue() . . . 1492
10.107.3.10 SampleValueAsPressure() . . . 1492
10.107.3.11 SampleValueAsSalinity() . . . 1493
10.107.3.12 SampleValueAsTemperature() . . . 1493
10.107.3.13 sensor_data_format() . . . 1494
10.107.3.14 set_id_dive() . . . 1494
10.107.3.15 set_id_instrument() . . . 1494
10.107.3.16 set_id_xmsg_type() . . . 1495
10.107.3.17 set_raw_sensor_data() . . . 1496
10.107.3.18 set_sensor_data_format() . . . 1497
10.107.3.19 set_xmsg_index() . . . 1497
10.107.3.20 set_xmsg_size_bytes() . . . 1498
10.107.3.21 UnpackSensorData() . . . 1499
10.107.3.22 xmsg_index() . . . 1500
10.107.3.23 xmsg_size_bytes() . . . 1501
10.107.4 Member Data Documentation . . . 1502
10.107.4.1 id_dive_ . . . 1502
10.107.4.2 id_instrument_ . . . 1502
10.107.4.3 id_xmsg_type_ . . . 1502
10.107.4.4 raw_sensor_data_ . . . 1503
10.107.4.5 sensor_data_format_ . . . 1503
10.107.4.6 xmsg_index_ . . . 1503
10.107.4.7 xmsg_size_bytes_ . . . 1503
10.108 util::xmsg::XPump Class Reference . . . 1503
10.108.1 Detailed Description . . . 1505
10.108.2 Member Typedef Documentation . . . 1505
10.108.2.1 Element_t . . . 1505
10.108.3 Constructor & Destructor Documentation . . . 1505
10.108.3.1 XPump() . . . 1505
10.108.3.2 ~XPump() . . . 1506
10.108.4 Member Function Documentation . . . 1506
10.108.4.1 AppendAllElements() [1/2] . . . 1506
10.108.4.2 AppendAllElements() [2/2] . . . 1506
10.108.4.3 AppendElement() . . . 1506
10.108.4.4 AssignElement() . . . 1506

10.108.4.5 CheckXMsgData() . . . 1507
10.108.4.6 DecodeXMsgData() . . . 1507
10.108.4.7 elements() . . . 1508
10.108.4.8 GenerateListingComment() . . . 1509
10.108.4.9 GenerateListingString() . . . 1509
10.108.4.10 id_dive() . . . 1510
10.108.4.11 id_instrument() . . . 1511
10.108.4.12 id_xmsg_type() . . . 1511
10.108.4.13 num_elements() . . . 1512
10.108.4.14 pump_packing_format() . . . 1512
10.108.4.15 set_id_dive() . . . 1512
10.108.4.16 set_id_instrument() . . . 1513
10.108.4.17 set_id_xmsg_type() . . . 1513
10.108.4.18 set_xmsg_index() . . . 1514
10.108.4.19 set_xmsg_size_bytes() . . . 1515
10.108.4.20 xmsg_index() . . . 1516
10.108.4.21 xmsg_size_bytes() . . . 1517
10.108.5 Member Data Documentation . . . 1518
10.108.5.1 elements_ . . . 1518
10.108.5.2 id_dive_ . . . 1518
10.108.5.3 id_instrument_ . . . 1518
10.108.5.4 id_xmsg_type_ . . . 1518
10.108.5.5 pump_packing_format_ . . . 1518
10.108.5.6 xmsg_index_ . . . 1518
10.108.5.7 xmsg_size_bytes_ . . . 1518
10.109 util::xmsg::XPumpCounts Class Reference . . . 1519
10.109.1 Detailed Description . . . 1520
10.109.2 Constructor & Destructor Documentation . . . 1520
10.109.2.1 XPumpCounts() . . . 1520
10.109.2.2 ~XPumpCounts() . . . 1520
10.109.3 Member Function Documentation . . . 1520
10.109.3.1 case_vac_after_pumping_cinhg() . . . 1520
10.109.3.2 case_vac_after_pumping_inhg() . . . 1521
10.109.3.3 case_vac_before_pumping_cinhg() . . . 1521
10.109.3.4 case_vac_before_pumping_inhg() . . . 1522
10.109.3.5 GenerateListingString() . . . 1522
10.109.3.6 PressureValue() . . . 1523
10.109.3.7 pump_current_a() . . . 1523
10.109.3.8 pump_voltage_v() . . . 1524
10.109.4 Member Data Documentation . . . 1524
10.109.4.1 case_vac_after_pumping_cinhg_ . . . 1524
10.109.4.2 case_vac_before_pumping_cinhg_ . . . 1524
10.109.4.3 phase_code_ . . . 1524
10.109.4.4 pressure_counts_ . . . 1524
10.109.4.5 pump_current_ma_ . . . 1524
10.109.4.6 pump_duration_sec_ . . . 1525
10.109.4.7 pump_voltage_cv_ . . . 1525
10.110 util::xmsg::XRise Class Reference . . . 1525
10.110.1 Detailed Description . . . 1527
10.110.2 Constructor & Destructor Documentation . . . 1527
10.110.2.1 XRise() . . . 1527
10.110.2.2 ~XRise() . . . 1527

10.110.3 Member Function Documentation . . . 1528
10.110.3.1 CheckXMsgData() . . . 1528
10.110.3.2 data_pack_format() . . . 1529
10.110.3.3 DecodeXMsgData() . . . 1529
10.110.3.4 GenerateCurrentUtcTimeIso8601String() . . . 1530
10.110.3.5 GenerateIso8601String() . . . 1530
10.110.3.6 GenerateJulianDay() . . . 1531
10.110.3.7 GenerateListingComment() . . . 1532
10.110.3.8 GenerateListingString() . . . 1532
10.110.3.9 GetCurrentUtcTime() . . . 1533
10.110.3.10 id_dive() . . . 1534
10.110.3.11 id_instrument() . . . 1535
10.110.3.12 id_xmsg_type() . . . 1535
10.110.3.13 num_samples() . . . 1536
10.110.3.14 sensor_data() . . . 1536
10.110.3.15 set_data_pack_format() . . . 1537
10.110.3.16 set_id_dive() . . . 1537
10.110.3.17 set_id_instrument() . . . 1538
10.110.3.18 set_id_xmsg_type() . . . 1538
10.110.3.19 set_start_time() . . . 1539
10.110.3.20 set_xmsg_index() . . . 1540
10.110.3.21 set_xmsg_size_bytes() . . . 1541
10.110.3.22 start_time() . . . 1542
10.110.3.23 UnpackData() . . . 1542
10.110.3.24 xmsg_index() . . . 1543
10.110.3.25 xmsg_size_bytes() . . . 1544
10.110.4 Member Data Documentation . . . 1544
10.110.4.1 data_pack_format_ . . . 1544
10.110.4.2 id_dive_ . . . 1545
10.110.4.3 id_instrument_ . . . 1545
10.110.4.4 id_xmsg_type_ . . . 1545
10.110.4.5 sensor_data_ . . . 1545
10.110.4.6 start_time_ . . . 1545
10.110.4.7 xmsg_index_ . . . 1545
10.110.4.8 xmsg_size_bytes_ . . . 1545
10.111 util::xmsg::XSensorTripletCodeCounts Class Reference . . . 1546
10.111.1 Detailed Description . . . 1547
10.111.2 Constructor & Destructor Documentation . . . 1547
10.111.2.1 XSensorTripletCodeCounts() . . . 1547
10.111.2.2 ∼XSensorTripletCodeCounts() . . . 1548
10.111.3 Member Function Documentation . . . 1548
10.111.3.1 elapsed_seconds_since_reference_time() . . . 1548
10.111.3.2 GenerateListingString() [1/2] . . . 1548
10.111.3.3 GenerateListingString() [2/2] . . . 1549
10.111.3.4 GetPhaseCodeName() . . . 1549
10.111.3.5 phase_code() . . . 1550
10.111.3.6 phase_code_name() . . . 1550
10.111.3.7 PressureValue() . . . 1551
10.111.3.8 SalinityValue() . . . 1551
10.111.3.9 TemperatureValue() . . . 1551
10.111.4 Member Data Documentation . . . 1552
10.111.4.1 elapsed_seconds_since_reference_time_ . . . 1552

10.111.4.2 phase_code_ . . . 1552
10.111.4.3 phase_code_names_ . . . 1552
10.111.4.4 pressure_counts_ . . . 1553
10.111.4.5 salinity_counts_ . . . 1553
10.111.4.6 temperature_counts_ . . . 1553
10.112 util::xmsg::XSurfaceData Class Reference . . . 1553
10.112.1 Detailed Description . . . 1555
10.112.2 Constructor & Destructor Documentation . . . 1555
10.112.2.1 XSurfaceData() . . . 1555
10.112.2.2 ∼XSurfaceData() . . . 1555
10.112.3 Member Function Documentation . . . 1556
10.112.3.1 CheckXMsgData() . . . 1556
10.112.3.2 data_pack_format() . . . 1557
10.112.3.3 DecodeXMsgData() . . . 1557
10.112.3.4 GenerateCurrentUtcTimeIso8601String() . . . 1558
10.112.3.5 GenerateIso8601String() . . . 1558
10.112.3.6 GenerateJulianDay() . . . 1559
10.112.3.7 GenerateListingComment() . . . 1560
10.112.3.8 GenerateListingString() . . . 1560
10.112.3.9 GetCurrentUtcTime() . . . 1561
10.112.3.10 id_dive() . . . 1562
10.112.3.11 id_instrument() . . . 1563
10.112.3.12 id_xmsg_type() . . . 1563
10.112.3.13 num_samples() . . . 1564
10.112.3.14 sensor_data() . . . 1564
10.112.3.15 set_data_pack_format() . . . 1565
10.112.3.16 set_id_dive() . . . 1565
10.112.3.17 set_id_instrument() . . . 1566
10.112.3.18 set_id_xmsg_type() . . . 1566
10.112.3.19 set_start_time() . . . 1567
10.112.3.20 set_xmsg_index() . . . 1568
10.112.3.21 set_xmsg_size_bytes() . . . 1569
10.112.3.22 start_time() . . . 1570
10.112.3.23 UnpackData() . . . 1570
10.112.3.24 xmsg_index() . . . 1571
10.112.3.25 xmsg_size_bytes() . . . 1572
10.112.4 Member Data Documentation . . . 1572
10.112.4.1 data_pack_format_ . . . 1572
10.112.4.2 id_dive_ . . . 1573
10.112.4.3 id_instrument_ . . . 1573
10.112.4.4 id_xmsg_type_ . . . 1573
10.112.4.5 sensor_data_ . . . 1573
10.112.4.6 start_time_ . . . 1573
10.112.4.7 xmsg_index_ . . . 1573
10.112.4.8 xmsg_size_bytes_ . . . 1573
10.113 util::xmsg::XValve Class Reference . . . 1574
10.113.1 Detailed Description . . . 1575
10.113.2 Member Typedef Documentation . . . 1575
10.113.2.1 Element_t . . . 1575
10.113.3 Constructor & Destructor Documentation . . . 1575
10.113.3.1 XValve() . . . 1575
10.113.4 Member Function Documentation . . . 1576

10.113.4.1 AppendAllElements() [1/2] . . . 1576
10.113.4.2 AppendAllElements() [2/2] . . . 1576
10.113.4.3 AppendElement() . . . 1576
10.113.4.4 AssignElement() . . . 1577
10.113.4.5 CheckXMsgData() . . . 1577
10.113.4.6 DecodeXMsgData() . . . 1578
10.113.4.7 elements() . . . 1578
10.113.4.8 GenerateListingComment() . . . 1579
10.113.4.9 id_dive() . . . 1579
10.113.4.10 id_instrument() . . . 1580
10.113.4.11 id_xmsg_type() . . . 1581
10.113.4.12 num_elements() . . . 1582
10.113.4.13 set_id_dive() . . . 1582
10.113.4.14 set_id_instrument() . . . 1582
10.113.4.15 set_id_xmsg_type() . . . 1582
10.113.4.16 set_xmsg_index() . . . 1583
10.113.4.17 set_xmsg_size_bytes() . . . 1584
10.113.4.18 xmsg_index() . . . 1585
10.113.4.19 xmsg_size_bytes() . . . 1586
10.113.5 Member Data Documentation . . . 1587
10.113.5.1 elements_ . . . 1587
10.113.5.2 id_dive_ . . . 1587
10.113.5.3 id_instrument_ . . . 1587
10.113.5.4 id_xmsg_type_ . . . 1588
10.113.5.5 xmsg_index_ . . . 1588
10.113.5.6 xmsg_size_bytes_ . . . 1588
10.114 util::xmsg::XValveCounts Class Reference . . . 1588
10.114.1 Detailed Description . . . 1589
10.114.2 Constructor & Destructor Documentation . . . 1589
10.114.2.1 XValveCounts() . . . 1589
10.114.2.2 ~XValveCounts() . . . 1589
10.114.3 Member Function Documentation . . . 1590
10.114.3.1 case_vac_after_pumping_inhg() . . . 1590
10.114.3.2 case_vac_before_pumping_inhg() . . . 1590
10.114.3.3 PressureValue() . . . 1590
10.114.4 Member Data Documentation . . . 1590
10.114.4.1 case_vac_after_valve_cinhg_ . . . 1590
10.114.4.2 case_vac_before_valve_cinhg_ . . . 1590
10.114.4.3 phase_code_ . . . 1591
10.114.4.4 pressure_counts_ . . . 1591
10.114.4.5 valve_duration_sec_ . . . 1591
10.115 util::xmsg::XWave Class Reference . . . 1591
10.115.1 Detailed Description . . . 1593
10.115.2 Constructor & Destructor Documentation . . . 1593
10.115.2.1 XWave() . . . 1593
10.115.2.2 ~XWave() . . . 1594
10.115.3 Member Function Documentation . . . 1594
10.115.3.1 CheckXMsgData() . . . 1594
10.115.3.2 DecodeXMsgData() . . . 1594
10.115.3.3 FixFuckup9000() . . . 1595
10.115.3.4 GenerateListingString() . . . 1595
10.115.3.5 id_dive() . . . 1596

10.115.3.6 id_instrument() . . . 1597
10.115.3.7 id_xmsg_type() . . . 1598
10.115.3.8 set_id_dive() . . . 1599
10.115.3.9 set_id_instrument() . . . 1599
10.115.3.10 set_id_xmsg_type() . . . 1599
10.115.3.11 set_xmsg_index() . . . 1600
10.115.3.12 set_xmsg_size_bytes() . . . 1601
10.115.3.13 xmsg_index() . . . 1602
10.115.3.14 xmsg_size_bytes() . . . 1603
10.115.4 Member Data Documentation . . . 1604
10.115.4.1 assf_ . . . 1604
10.115.4.2 ASSF_RAW . . . 1604
10.115.4.3 ASSF_SPECTRAL_V1 . . . 1604
10.115.4.4 ASSF_SPECTRAL_V2 . . . 1605
10.115.4.5 FSAM . . . 1605
10.115.4.6 gain_accel_q3_ . . . 1605
10.115.4.7 GSSF . . . 1605
10.115.4.8 id_dive_ . . . 1605
10.115.4.9 id_instrument_ . . . 1605
10.115.4.10 id_xmsg_type_ . . . 1605
10.115.4.11 MSSF . . . 1605
10.115.4.12 sample_mode_ . . . 1606
10.115.4.13 sample_rate_base_ . . . 1606
10.115.4.14 sample_rate_div_ . . . 1606
10.115.4.15 sample_rate_effective_ . . . 1606
10.115.4.16 segment_len_ . . . 1606
10.115.4.17 segment_len_pow_ . . . 1606
10.115.4.18 wave_data_spectral_ . . . 1606
10.115.4.19 xmsg_index_ . . . 1606
10.115.4.20 xmsg_size_bytes_ . . . 1607
10.116 util::xmsg::XWaveRawSample Class Reference . . . 1607
10.116.1 Detailed Description . . . 1608
10.116.2 Constructor & Destructor Documentation . . . 1608
10.116.2.1 XWaveRawSample() . . . 1608
10.116.2.2 ∼XWaveRawSample() . . . 1608
10.116.3 Member Function Documentation . . . 1608
10.116.3.1 GenerateListingString() . . . 1608
10.116.4 Member Data Documentation . . . 1608
10.116.4.1 accel . . . 1608
10.116.4.2 magnt . . . 1609
10.116.4.3 omega . . . 1609
10.117 util::xmsg::XWaveSpectralSample Class Reference . . . 1609
10.117.1 Detailed Description . . . 1610

11 File Documentation **1611**

11.1 AdditionalDocumentation.h File Reference . . . 1611
11.1.1 Detailed Description . . . 1612
11.2 AdditionalDocumentation.h . . . 1613
11.3 config/LocalConfiguration.cpp File Reference . . . 1615
11.3.1 Detailed Description . . . 1615
11.4 config/LocalConfiguration.h File Reference . . . 1616
11.4.1 Detailed Description . . . 1617

11.4.2 Macro Definition Documentation . . . 1618
11.4.2.1 CFG_DEFAULT_ISDAEMON . . . 1618
11.4.2.2 CFG_DEFAULT_MAIL_CHK_DLY . . . 1618
11.4.2.3 CFG_DEFAULT_MAX_FETCH . . . 1618
11.4.2.4 CFG_DEFAULT_MIN_UID . . . 1618
11.4.2.5 CFG_DEFAULT_NDECO . . . 1618
11.4.2.6 CFG_DEFAULT_NPROC . . . 1618
11.4.2.7 CFG_DEFAULT_PATH . . . 1618
11.4.2.8 CFG_DEFAULT_PATH_PID . . . 1619
11.4.2.9 CFG_DEFAULT_REPROCESS_ALL . . . 1619
11.4.2.10 CFG_FMT_DESC_LEN . . . 1619
11.4.2.11 CFG_FMT_LINE_LEN . . . 1619
11.4.2.12 PROG_DESC . . . 1619
11.4.2.13 PROG_ID . . . 1619
11.4.2.14 PROG_MAJOR_VERS . . . 1619
11.4.2.15 PROG_MINOR_VERS . . . 1619
11.4.2.16 PROG_NAME . . . 1620
11.4.2.17 PROG_VERS . . . 1620
11.4.2.18 PROG_XVERS . . . 1620
11.5 LocalConfiguration.h . . . 1620
11.6 config/LocalMysqlConfiguration.cpp File Reference . . . 1625
11.6.1 Detailed Description . . . 1625
11.7 config/LocalMysqlConfiguration.h File Reference . . . 1626
11.7.1 Detailed Description . . . 1626
11.7.2 Macro Definition Documentation . . . 1627
11.7.2.1 CFG_MYSQL_DFLT_ADDRESS . . . 1627
11.7.2.2 CFG_MYSQL_DFLT_PORT . . . 1627
11.7.2.3 CFG_MYSQL_DFLT_PROTOCOL . . . 1627
11.7.2.4 CFG_MYSQL_DFLT_SCHEMA . . . 1627
11.8 LocalMysqlConfiguration.h . . . 1627
11.9 config/LocalPersistentEntity.h File Reference . . . 1628
11.9.1 Detailed Description . . . 1629
11.10 LocalPersistentEntity.h . . . 1630
11.11 data/Iridium.h File Reference . . . 1630
11.11.1 Detailed Description . . . 1631
11.12 Iridium.h . . . 1632
11.13 data/IridiumDefinitions.h File Reference . . . 1634
11.13.1 Detailed Description . . . 1635
11.13.2 Macro Definition Documentation . . . 1635
11.13.2.1 IRI_DIVE_UNKNOWN . . . 1635
11.13.2.2 IRI_GAIN_INVALID . . . 1635
11.13.2.3 IRI_ID_UNKNOWN . . . 1635
11.13.2.4 IRI_OFFS_INVALID . . . 1635
11.13.2.5 IRI_POS_INVALID_CEP . . . 1635
11.13.2.6 IRI_POS_INVALID_LAT . . . 1636
11.13.2.7 IRI_POS_INVALID_LON . . . 1636
11.14 IridiumDefinitions.h . . . 1636
11.15 data/IridiumPosition.h File Reference . . . 1637
11.15.1 Detailed Description . . . 1638
11.16 IridiumPosition.h . . . 1638
11.17 data/XFloat.cpp File Reference . . . 1640
11.17.1 Detailed Description . . . 1641

11.18 data/XFloat.h File Reference . . . 1641
11.18.1 Detailed Description . . . 1642
11.19 XFloat.h . . . 1643
11.20 data/XFloatDefinitions.h File Reference . . . 1658
11.20.1 Detailed Description . . . 1660
11.20.2 Macro Definition Documentation . . . 1660
11.20.2.1 XFLT_DAEMON_FLUSH_LOG_SEC . . . 1660
11.20.2.2 XFLT_DIVE_UNKNOWN . . . 1660
11.20.2.3 XFLT_GAIN_INVALID . . . 1660
11.20.2.4 XFLT_ID_UNKNOWN . . . 1660
11.20.2.5 XFLT_OFFS_INVALID . . . 1661
11.20.2.6 XFLT_PID_INVALID . . . 1661
11.20.2.7 XFLT_POS_INVALID_LAT . . . 1661
11.20.2.8 XFLT_POS_INVALID_LON . . . 1661
11.20.2.9 XFLT_QUEUE_SIZE_MAIL_PROCESSING . . . 1661
11.20.2.10 XFLT_QUEUE_SIZE_TELEMETRY . . . 1661
11.20.2.11 XFLT_SBD_SESSION_STATUS_CODE_UNKNOWN . . . 1661
11.20.2.12 XFLT_SBD_SIZE_BYTES_UNKNOWN . . . 1661
11.20.2.13 XFLT_THREAD_SLEEP_MS . . . 1662
11.21 XFloatDefinitions.h . . . 1662
11.22 data/XFloatDiveData.cpp File Reference . . . 1663
11.22.1 Detailed Description . . . 1664
11.23 data/XFloatDiveData.h File Reference . . . 1665
11.23.1 Detailed Description . . . 1665
11.24 XFloatDiveData.h . . . 1666
11.25 data/XFloatMailMessage.cpp File Reference . . . 1684
11.25.1 Detailed Description . . . 1685
11.26 data/XFloatMailMessage.h File Reference . . . 1685
11.26.1 Detailed Description . . . 1687
11.26.2 Macro Definition Documentation . . . 1687
11.26.2.1 XMAIL_DFLT_MSG_HEADER_VALID_SNDR_REGEX . . . 1687
11.26.2.2 XMAIL_DFLT_MSG_HEADER_VALID_SUBJ_REGEX . . . 1688
11.26.2.3 XMAIL_DFLT_RAW_FILE_FORMAT . . . 1688
11.26.2.4 XMAIL_DFLT_RAW_FILE_GENERATRION_ENABLE . . . 1688
11.26.2.5 XMAIL_DFLT_SBD_FILE_GENERATRION_ENABLE . . . 1688
11.26.2.6 XMAIL_DFLT_SBD_FILE_VALID_NAME_REGEX . . . 1688
11.27 XFloatMailMessage.h . . . 1689
11.28 data/XFloatMetadata.cpp File Reference . . . 1706
11.28.1 Detailed Description . . . 1707
11.29 data/XFloatMetadata.h File Reference . . . 1707
11.29.1 Detailed Description . . . 1708
11.30 XFloatMetadata.h . . . 1709
11.31 data/xmsg/AbstractXComposite.h File Reference . . . 1717
11.31.1 Detailed Description . . . 1717
11.32 AbstractXComposite.h . . . 1718
11.33 data/xmsg/AbstractXFallRise.h File Reference . . . 1719
11.33.1 Detailed Description . . . 1720
11.34 AbstractXFallRise.h . . . 1721
11.35 data/xmsg/AbstractXSensorData.h File Reference . . . 1724
11.35.1 Detailed Description . . . 1725
11.36 AbstractXSensorData.h . . . 1726
11.37 data/xmsg/AbstractXSensorDataComposite.h File Reference . . . 1733

11.37.1 Detailed Description . . . 1734
11.38 AbstractXSensorDataComposite.h . . . 1734
11.39 data/xmsg/AbstractXTimeSeriesDataComposite.h File Reference . . . 1745
11.39.1 Detailed Description . . . 1746
11.40 AbstractXTimeSeriesDataComposite.h . . . 1747
11.41 data/xmsg/AuxiliarySensorData.cpp File Reference . . . 1757
11.41.1 Detailed Description . . . 1758
11.42 data/xmsg/AuxiliarySensorData.h File Reference . . . 1759
11.42.1 Detailed Description . . . 1760
11.43 AuxiliarySensorData.h . . . 1760
11.44 data/xmsg/common/XCommon.h File Reference . . . 1765
11.44.1 Detailed Description . . . 1765
11.45 XCommon.h . . . 1766
11.46 data/xmsg/common/XDefinitions.h File Reference . . . 1766
11.46.1 Detailed Description . . . 1769
11.46.2 Macro Definition Documentation . . . 1770
11.46.2.1 i08 . . . 1770
11.46.2.2 i16 . . . 1770
11.46.2.3 i24 . . . 1770
11.46.2.4 i32 . . . 1770
11.46.2.5 s1970 . . . 1770
11.46.2.6 s2000 . . . 1771
11.46.2.7 u08 . . . 1771
11.46.2.8 u16 . . . 1771
11.46.2.9 u24 . . . 1771
11.46.2.10 u32 . . . 1771
11.46.2.11 XMSG_BAD_DBL . . . 1772
11.46.2.12 XMSG_BAD_FLT . . . 1772
11.46.2.13 XMSG_BAD_I08 . . . 1772
11.46.2.14 XMSG_BAD_I16 . . . 1772
11.46.2.15 XMSG_BAD_I32 . . . 1772
11.46.2.16 XMSG_BAD_U08 . . . 1772
11.46.2.17 XMSG_BAD_U16 . . . 1772
11.46.2.18 XMSG_BAD_U32 . . . 1772
11.46.2.19 XMSG_DEBUG_AGGREGATE_TIMESERIES_DATA . . . 1772
11.46.2.20 XMSG_DEBUG_ALAMO_MISS_GFG . . . 1773
11.46.2.21 XMSG_DEBUG_BIST . . . 1773
11.46.2.22 XMSG_DEBUG_ENG_SURF . . . 1773
11.46.2.23 XMSG_DEBUG_MFA_MISS_GFG . . . 1773
11.46.2.24 XMSG_DEBUG_SENSOR_CURVATURE . . . 1773
11.46.2.25 XMSG_DEBUG_SURF_PRES_AGGR_DATA . . . 1773
11.46.2.26 XMSG_DIVE_UNKNOWN . . . 1773
11.46.2.27 XMSG_ENABLE_MULTI_PROF . . . 1774
11.46.2.28 XMSG_ENG_BAD_VER . . . 1774
11.46.2.29 XMSG_GAIN_INVALID . . . 1774
11.46.2.30 XMSG_HEADER . . . 1774
11.46.2.31 XMSG_HEADER2 . . . 1774
11.46.2.32 XMSG_ID_UNKNOWN . . . 1775
11.46.2.33 XMSG_INIT_SAM_N_RESERVE . . . 1775
11.46.2.34 XMSG_MAX_DRFT . . . 1775
11.46.2.35 XMSG_MAX_FALL . . . 1775
11.46.2.36 XMSG_MAX_GPS . . . 1775

11.46.2.37 XMSG_MAX_MAIL . . . 1775
11.46.2.38 XMSG_MAX_MFA_MISS . . . 1775
11.46.2.39 XMSG_MAX_MISS . . . 1775
11.46.2.40 XMSG_MAX_PROF . . . 1776
11.46.2.41 XMSG_MAX_PROF_HIRES . . . 1776
11.46.2.42 XMSG_MAX_PUMP . . . 1776
11.46.2.43 XMSG_MAX_RISE . . . 1776
11.46.2.44 XMSG_MAX_SURF_PRES . . . 1776
11.46.2.45 XMSG_MAX_VALVE . . . 1776
11.46.2.46 XMSG_MAX_WAVE . . . 1776
11.46.2.47 XMSG_OFFS_INVALID . . . 1776
11.46.2.48 XMSG_PHY . . . 1777
11.46.2.49 XMSG_SBD_MIN_BYTES . . . 1777
11.46.2.50 XMSG_SECTION . . . 1777
11.46.2.51 XMSG_SECTION_BREAK . . . 1777
11.46.2.52 XMSG_SENS_BAD . . . 1777
11.46.2.53 XMSG_SENS_BAD_PRES . . . 1778
11.46.2.54 XMSG_SENS_BAD_SALN . . . 1778
11.46.2.55 XMSG_SENS_BAD_TEMP . . . 1778
11.46.2.56 XMSG_SENS_MAX_PRES . . . 1778
11.46.2.57 XMSG_SENS_MAX_SALN . . . 1778
11.46.2.58 XMSG_SENS_MAX_SAMPLES . . . 1778
11.46.2.59 XMSG_SENS_MAX_SUB . . . 1778
11.46.2.60 XMSG_SENS_MAX_TEMP . . . 1779
11.46.2.61 XMSG_SENS_MIN_PRES . . . 1779
11.46.2.62 XMSG_SENS_MIN_SALN . . . 1779
11.46.2.63 XMSG_SENS_MIN_TEMP . . . 1779
11.46.2.64 XMSG_VERSION_UNKNOWN . . . 1779
11.47 XDefinitions.h . . . 1779
11.48 data/xmsg/common/XPresCodeCounts.cpp File Reference . . . 1783
11.48.1 Detailed Description . . . 1784
11.49 data/xmsg/common/XPresCodeCounts.h File Reference . . . 1784
11.49.1 Detailed Description . . . 1785
11.50 XPresCodeCounts.h . . . 1786
11.51 data/xmsg/common/XPresCounts.h File Reference . . . 1788
11.51.1 Detailed Description . . . 1788
11.52 XPresCounts.h . . . 1789
11.53 data/xmsg/common/XPumpCounts.h File Reference . . . 1790
11.53.1 Detailed Description . . . 1791
11.54 XPumpCounts.h . . . 1792
11.55 data/xmsg/common/XSensorTripletCodeCounts.h File Reference . . . 1794
11.55.1 Detailed Description . . . 1795
11.56 XSensorTripletCodeCounts.h . . . 1795
11.57 data/xmsg/common/XValveCounts.cpp File Reference . . . 1797
11.57.1 Detailed Description . . . 1797
11.58 data/xmsg/common/XValveCounts.h File Reference . . . 1798
11.58.1 Detailed Description . . . 1799
11.59 XValveCounts.h . . . 1799
11.60 data/xmsg/common/XWaveRawSample.h File Reference . . . 1801
11.60.1 Detailed Description . . . 1802
11.61 XWaveRawSample.h . . . 1802
11.62 data/xmsg/common/XWaveSpectralSample.cpp File Reference . . . 1803

11.62.1 Detailed Description . . . 1804
11.63 data/xmsg/common/XWaveSpectralSample.h File Reference . . . 1805
11.63.1 Detailed Description . . . 1805
11.64 XWaveSpectralSample.h . . . 1806
11.65 data/xmsg/XAcousticPosition.h File Reference . . . 1808
11.65.1 Detailed Description . . . 1809
11.66 XAcousticPosition.h . . . 1809
11.67 data/xmsg/XAlamoBit.h File Reference . . . 1816
11.67.1 Detailed Description . . . 1817
11.68 XAlamoBit.h . . . 1817
11.69 XAlamoBit2.h . . . 1828
11.70 data/xmsg/XAlamoEngineering.h File Reference . . . 1841
11.70.1 Detailed Description . . . 1842
11.71 XAlamoEngineering.h . . . 1843
11.72 data/xmsg/XAlamoMissionConfiguration.h File Reference . . . 1844
11.72.1 Detailed Description . . . 1845
11.73 XAlamoMissionConfiguration.h . . . 1845
11.74 data/xmsg/XArgo.h File Reference . . . 1901
11.74.1 Detailed Description . . . 1902
11.74.2 Enumeration Type Documentation . . . 1903
11.74.2.1 XArgoInstrumentType_t . . . 1903
11.75 XArgo.h . . . 1904
11.76 data/xmsg/XCompositeAuxTemp.h File Reference . . . 1915
11.76.1 Detailed Description . . . 1916
11.77 XCompositeAuxTemp.h . . . 1917
11.78 data/xmsg/XCompositeDrift.h File Reference . . . 1918
11.78.1 Detailed Description . . . 1918
11.79 XCompositeDrift.h . . . 1919
11.80 data/xmsg/XCompositeFall.h File Reference . . . 1921
11.80.1 Detailed Description . . . 1922
11.81 XCompositeFall.h . . . 1922
11.82 data/xmsg/XCompositeHiResProfile.h File Reference . . . 1923
11.82.1 Detailed Description . . . 1924
11.83 XCompositeHiResProfile.h . . . 1924
11.84 data/xmsg/XCompositeIridiumPosition.h File Reference . . . 1926
11.84.1 Detailed Description . . . 1928
11.84.2 Macro Definition Documentation . . . 1928
11.84.2.1 XMSG_POS_INVALID_LAT . . . 1928
11.84.2.2 XMSG_POS_INVALID_LON . . . 1928
11.84.2.3 XMSG_POS_INVALID_STDERR . . . 1928
11.84.2.4 XMSG_POS_INVALID_VAR . . . 1928
11.85 XCompositeIridiumPosition.h . . . 1929
11.86 data/xmsg/XCompositeProfile.h File Reference . . . 1931
11.86.1 Detailed Description . . . 1931
11.87 XCompositeProfile.h . . . 1932
11.88 data/xmsg/XCompositePump.h File Reference . . . 1934
11.88.1 Detailed Description . . . 1935
11.89 XCompositePump.h . . . 1936
11.90 data/xmsg/XCompositeRise.h File Reference . . . 1937
11.90.1 Detailed Description . . . 1938
11.91 XCompositeRise.h . . . 1938
11.92 data/xmsg/XCompositeSurfaceData.cpp File Reference . . . 1939

11.92.1 Detailed Description . . . 1940
11.93 data/xmsg/XCompositeSurfaceData.h File Reference . . . 1940
11.93.1 Detailed Description . . . 1941
11.94 XCompositeSurfaceData.h . . . 1941
11.95 data/xmsg/XCompositeValve.cpp File Reference . . . 1946
11.95.1 Detailed Description . . . 1947
11.96 data/xmsg/XCompositeValve.h File Reference . . . 1947
11.96.1 Detailed Description . . . 1948
11.97 XCompositeValve.h . . . 1948
11.98 data/xmsg/XDriftPressure.h File Reference . . . 1949
11.98.1 Detailed Description . . . 1950
11.99 XDriftPressure.h . . . 1951
11.100 data/xmsg/XDriftSalinity.h File Reference . . . 1951
11.100.1 Detailed Description . . . 1952
11.101 XDriftSalinity.h . . . 1953
11.102 data/xmsg/XDriftTemperature.h File Reference . . . 1953
11.102.1 Detailed Description . . . 1954
11.103 XDriftTemperature.h . . . 1955
11.104 data/xmsg/XEngAbort.h File Reference . . . 1955
11.104.1 Detailed Description . . . 1956
11.105 XEngAbort.h . . . 1957
11.106 data/xmsg/XEngAscent.h File Reference . . . 1958
11.106.1 Detailed Description . . . 1959
11.107 XEngAscent.h . . . 1959
11.108 XEngBIT.h . . . 1963
11.109 data/xmsg/XEngBITFail.h File Reference . . . 1964
11.109.1 Detailed Description . . . 1965
11.110 XEngBITFail.h . . . 1966
11.111 data/xmsg/XEngBITPass.h File Reference . . . 1967
11.111.1 Detailed Description . . . 1967
11.112 XEngBITPass.h . . . 1968
11.113 data/xmsg/XEngDescent.h File Reference . . . 1969
11.113.1 Detailed Description . . . 1969
11.114 XEngDescent.h . . . 1970
11.115 data/xmsg/XEngDiagnostic.h File Reference . . . 1974
11.115.1 Detailed Description . . . 1975
11.116 XEngDiagnostic.h . . . 1976
11.117 data/xmsg/XEngDrift.h File Reference . . . 1977
11.117.1 Detailed Description . . . 1978
11.118 XEngDrift.h . . . 1978
11.119 data/xmsg/XEngIce.cpp File Reference . . . 1983
11.119.1 Detailed Description . . . 1983
11.120 data/xmsg/XEngIce.h File Reference . . . 1984
11.120.1 Detailed Description . . . 1985
11.121 XEngIce.h . . . 1985
11.122 data/xmsg/XEngStandard.h File Reference . . . 1991
11.122.1 Detailed Description . . . 1992
11.123 XEngStandard.h . . . 1993
11.124 data/xmsg/XEngSurface.h File Reference . . . 1994
11.124.1 Detailed Description . . . 1995
11.125 XEngSurface.h . . . 1995
11.126 data/xmsg/XFall.h File Reference . . . 2012

11.126.1 Detailed Description . . . 2013
11.127 XFall.h . . . 2014
11.128 data/xmsg/XGPS.cpp File Reference . . . 2015
11.128.1 Detailed Description . . . 2016
11.129 data/xmsg/XGPS.h File Reference . . . 2017
11.129.1 Detailed Description . . . 2018
11.130 XGPS.h . . . 2018
11.131 data/xmsg/XHiResPressure.h File Reference . . . 2023
11.131.1 Detailed Description . . . 2024
11.132 XHiResPressure.h . . . 2025
11.133 data/xmsg/XHiResSalinity.h File Reference . . . 2025
11.133.1 Detailed Description . . . 2026
11.134 XHiResSalinity.h . . . 2027
11.135 data/xmsg/XHiResTemperature.h File Reference . . . 2027
11.135.1 Detailed Description . . . 2028
11.136 XHiResTemperature.h . . . 2029
11.137 data/xmsg/XMessage.h File Reference . . . 2029
11.137.1 Detailed Description . . . 2033
11.138 XMessage.h . . . 2033
11.139 data/xmsg/XMessageIndexable.h File Reference . . . 2038
11.139.1 Detailed Description . . . 2039
11.140 XMessageIndexable.h . . . 2039
11.141 data/xmsg/XMfaMissionCfg.cpp File Reference . . . 2040
11.141.1 Detailed Description . . . 2041
11.142 data/xmsg/XMfaMissionCfg.h File Reference . . . 2042
11.142.1 Detailed Description . . . 2042
11.143 XMfaMissionCfg.h . . . 2043
11.144 data/xmsg/XMissionCmdEcho.h File Reference . . . 2054
11.144.1 Detailed Description . . . 2055
11.145 XMissionCmdEcho.h . . . 2056
11.146 data/xmsg/XMissionConfiguration.h File Reference . . . 2057
11.146.1 Detailed Description . . . 2057
11.147 XMissionConfiguration.h . . . 2058
11.148 data/xmsg/XProfilePressure.h File Reference . . . 2059
11.148.1 Detailed Description . . . 2060
11.149 XProfilePressure.h . . . 2060
11.150 data/xmsg/XProfileSalinity.h File Reference . . . 2061
11.150.1 Detailed Description . . . 2062
11.151 XProfileSalinity.h . . . 2063
11.152 data/xmsg/XProfileTemperature.h File Reference . . . 2063
11.152.1 Detailed Description . . . 2064
11.153 XProfileTemperature.h . . . 2065
11.154 data/xmsg/XPump.h File Reference . . . 2065
11.154.1 Detailed Description . . . 2067
11.155 XPump.h . . . 2067
11.156 data/xmsg/XRise.h File Reference . . . 2070
11.156.1 Detailed Description . . . 2071
11.157 XRise.h . . . 2071
11.158 data/xmsg/XSurfaceData.h File Reference . . . 2072
11.158.1 Detailed Description . . . 2074
11.159 XSurfaceData.h . . . 2074
11.160 data/xmsg/XValve.cpp File Reference . . . 2075

11.160.1 Detailed Description . . . 2076
11.161 data/xmsg/XValve.h File Reference . . . 2077
11.161.1 Detailed Description . . . 2077
11.162 XValve.h . . . 2078
11.163 data/xmsg/XWave.cpp File Reference . . . 2080
11.163.1 Detailed Description . . . 2080
11.164 data/xmsg/XWave.h File Reference . . . 2081
11.164.1 Detailed Description . . . 2082
11.164.2 Macro Definition Documentation . . . 2083
11.164.2.1 XWAVE_INVALID_SF . . . 2083
11.164.2.2 XWAVE_SIZE_ASSF_RAW . . . 2083
11.164.2.3 XWAVE_SIZE_ASSF_SPECTRAL . . . 2083
11.164.2.4 XWAVE_SIZE_FSAM . . . 2083
11.164.2.5 XWAVE_SIZE_GSSF . . . 2083
11.164.2.6 XWAVE_SIZE_MSSF . . . 2083
11.165 XWave.h . . . 2084
11.166 Main.cpp File Reference . . . 2086
11.166.1 Detailed Description . . . 2087
11.166.2 Function Documentation . . . 2087
11.166.2.1 Initialize() . . . 2087
11.166.2.2 main() . . . 2088
11.166.2.3 StartWorkerThreads() . . . 2089
11.166.2.4 TerminateWorkerThreads() . . . 2089
11.167 Main.h File Reference . . . 2090
11.167.1 Detailed Description . . . 2091
11.167.2 Function Documentation . . . 2091
11.167.2.1 Initialize() . . . 2091
11.167.2.2 StartWorkerThreads() . . . 2092
11.167.2.3 TerminateWorkerThreads() . . . 2092
11.168 Main.h . . . 2093
11.169 stage1_MailRetrievalThread.cpp File Reference . . . 2093
11.169.1 Detailed Description . . . 2094
11.170 stage1_MailRetrievalThread.h File Reference . . . 2094
11.170.1 Detailed Description . . . 2095
11.171 stage1_MailRetrievalThread.h . . . 2096
11.172 stage2_MailProcessingThread.cpp File Reference . . . 2099
11.172.1 Detailed Description . . . 2100
11.173 stage2_MailProcessingThread.h File Reference . . . 2101
11.173.1 Detailed Description . . . 2101
11.174 stage2_MailProcessingThread.h . . . 2102
11.175 stage3_TelemetryDecoderThread.cpp File Reference . . . 2104
11.175.1 Detailed Description . . . 2104
11.176 stage3_TelemetryDecoderThread.h File Reference . . . 2105
11.176.1 Detailed Description . . . 2105
11.176.2 Macro Definition Documentation . . . 2106
11.176.2.1 DBG_X_DECODE . . . 2106
11.177 stage3_TelemetryDecoderThread.h . . . 2106
11.178 stage4b_PostProcessingThread.cpp File Reference . . . 2113
11.178.1 Detailed Description . . . 2113
11.179 stage4b_PostProcessingThread.h File Reference . . . 2114
11.179.1 Detailed Description . . . 2114
11.180 stage4b_PostProcessingThread.h . . . 2115

11.181 ThreadDefinitions.h File Reference . . . 2118
11.181.1 Detailed Description . . . 2119
11.181.2 Macro Definition Documentation . . . 2119
11.181.2.1 XMTPS_DEBUG_FETCH_MAIL . . . 2119
11.181.2.2 XMTPS_THREAD_JOIN_SECONDS . . . 2119
11.182 ThreadDefinitions.h . . . 2120
11.183 utility/ContainerUtils.h File Reference . . . 2120
11.183.1 Detailed Description . . . 2121
11.184 ContainerUtils.h . . . 2121
11.185 utility/DataBuf.h File Reference . . . 2122
11.185.1 Detailed Description . . . 2122
11.186 DataBuf.h . . . 2123
11.187 utility/DateUtils.h File Reference . . . 2124
11.187.1 Detailed Description . . . 2124
11.188 DateUtils.h . . . 2125
11.189 utility/FileSystemUtils.h File Reference . . . 2126
11.189.1 Detailed Description . . . 2127
11.190 FileSystemUtils.h . . . 2127
11.191 utility/KmlUtil.cpp File Reference . . . 2129
11.191.1 Detailed Description . . . 2130
11.192 utility/KmlUtil.h File Reference . . . 2130
11.192.1 Detailed Description . . . 2131
11.193 KmlUtil.h . . . 2131
11.194 utility/Lockable.h File Reference . . . 2135
11.194.1 Detailed Description . . . 2135
11.195 Lockable.h . . . 2136
11.196 utility/Log.cpp File Reference . . . 2137
11.196.1 Detailed Description . . . 2137
11.197 utility/Log.h File Reference . . . 2138
11.197.1 Detailed Description . . . 2140
11.197.2 Macro Definition Documentation . . . 2140
11.197.2.1 CATCH_BLOCK . . . 2140
11.197.2.2 CATCH_NOTHROW . . . 2142
11.197.2.3 dbg1 . . . 2142
11.197.2.4 dbg2 . . . 2142
11.197.2.5 dbg3 . . . 2142
11.197.2.6 DEFAULT_LOG_LEVEL . . . 2142
11.197.2.7 exc_catchAny . . . 2142
11.197.2.8 exc_catchAny_noThrow . . . 2143
11.197.2.9 exc_catchStandard . . . 2143
11.197.2.10 exc_catchStandard_noThrow . . . 2143
11.197.2.11 exc_diag . . . 2143
11.197.2.12 EXC_THROW . . . 2144
11.197.2.13 ldbg . . . 2144
11.197.2.14 lerr . . . 2145
11.197.2.15 lfat . . . 2145
11.197.2.16 linf . . . 2145
11.197.2.17 llog . . . 2145
11.197.2.18 LOG . . . 2145
11.197.2.19 lout . . . 2145
11.197.2.20 lwrn . . . 2146
11.197.2.21 N_LOG_LEVEL_NAMES . . . 2146

11.197.2.22 PP_STR . . . 2146
11.197.2.23 X_PP_STR . . . 2146
11.198 Log.h . . . 2146
11.199 utility/MathUtils.h File Reference . . . 2153
11.199.1 Detailed Description . . . 2154
11.200 MathUtils.h . . . 2154
11.201 utility/MessageException.h File Reference . . . 2155
11.201.1 Detailed Description . . . 2156
11.201.2 Macro Definition Documentation . . . 2156
11.201.2.1 ATTR_NO_EXCEPTIONS . . . 2156
11.202 MessageException.h . . . 2156
11.203 utility/MysqlUtils.cpp File Reference . . . 2157
11.203.1 Detailed Description . . . 2158
11.204 utility/MysqlUtils.h File Reference . . . 2158
11.204.1 Detailed Description . . . 2159
11.205 MysqlUtils.h . . . 2159
11.206 utility/Position.h File Reference . . . 2163
11.206.1 Detailed Description . . . 2164
11.207 Position.h . . . 2164
11.208 utility/PositionUtils.cpp File Reference . . . 2166
11.208.1 Detailed Description . . . 2166
11.209 utility/PositionUtils.h File Reference . . . 2167
11.209.1 Detailed Description . . . 2168
11.209.2 Macro Definition Documentation . . . 2168
11.209.2.1 UTIL_POS_INVALID_LAT . . . 2168
11.209.2.2 UTIL_POS_INVALID_LON . . . 2168
11.209.2.3 UTIL_POS_INVALID_STDERR . . . 2168
11.209.2.4 UTIL_POS_INVALID_VAR . . . 2169
11.210 PositionUtils.h . . . 2169
11.211 utility/RecursiveLockable.h File Reference . . . 2170
11.211.1 Detailed Description . . . 2171
11.212 RecursiveLockable.h . . . 2172
11.213 utility/SignalHandler.h File Reference . . . 2173
11.213.1 Detailed Description . . . 2173
11.214 SignalHandler.h . . . 2174
11.215 utility/SqlTableColumnDescription.h File Reference . . . 2175
11.215.1 Detailed Description . . . 2177
11.215.2 Macro Definition Documentation . . . 2177
11.215.2.1 sql_ci16 . . . 2177
11.215.2.2 sql_colb . . . 2177
11.215.2.3 sql_cold . . . 2177
11.215.2.4 sql_colf . . . 2177
11.215.2.5 sql_coli . . . 2178
11.215.2.6 sql_coll . . . 2178
11.215.2.7 sql_colp . . . 2178
11.215.2.8 sql_cols . . . 2178
11.215.2.9 sql_colt . . . 2178
11.215.2.10 sql_colv . . . 2178
11.215.2.11 sql_cou8 . . . 2178
11.215.2.12 sql_cu16 . . . 2179
11.216 SqlTableColumnDescription.h . . . 2179
11.217 utility/SqlTableDescription.cpp File Reference . . . 2189

11.217.1 Detailed Description . . . 2190
11.218 utility/SqlTableDescription.h File Reference . . . 2190
11.218.1 Detailed Description . . . 2191
11.219 SqlTableDescription.h . . . 2192
11.220 utility/StreamUtils.h File Reference . . . 2199
11.220.1 Detailed Description . . . 2200
11.221 StreamUtils.h . . . 2200
11.222 utility/StringUtils.h File Reference . . . 2201
11.222.1 Detailed Description . . . 2201
11.223 StringUtils.h . . . 2202
11.224 utility/SynchronousQueue.h File Reference . . . 2202
11.224.1 Detailed Description . . . 2203
11.225 SynchronousQueue.h . . . 2203
11.226 utility/SystemUtils.h File Reference . . . 2205
11.226.1 Detailed Description . . . 2206
11.226.2 Macro Definition Documentation . . . 2207
11.226.2.1 ATTR_DEPRECATED . . . 2207
11.227 SystemUtils.h . . . 2207
11.228 utility/ThreadUtils.h File Reference . . . 2207
11.228.1 Detailed Description . . . 2208
11.229 ThreadUtils.h . . . 2208
11.230 utility/VectorBuf.h File Reference . . . 2210
11.230.1 Detailed Description . . . 2210
11.231 VectorBuf.h . . . 2211

Index **2213**

Chapter 1

XMTPS — X-Message Telemetry Processing Service

XMTPS is the WHOI Float Group daemon that receives Iridium Short Burst Data (SBD) telemetry from ALAMO and ALTO profiling floats, decodes it, stores it in MySQL, and writes the listing, PHY, KML, status and RAW files used downstream. A set of Python tools in `bin/py` turns the listing files into JSON and generates the per-float plot pages and the fleet summary page.

1.1 How it works

Iridium delivers each SBD message as an email with a binary `.sbd` attachment. XMTPS processes them in a four-stage pipeline of worker threads, connected by bounded queues:

Stage	Source	Role
1. Mail retrieval	`src/stage1_MailRetrieval↩ Thread.*`	Fetches mail from the IMAP server (or RAW mail files from a directory) and queues it. Optionally reprocesses stored messages from the database at startup.
2. Mail processing	`src/stage2_MailProcessing↩ Thread.*`	Validates the header and SBD attachment, resolves the float's identifiers, writes the RAW/SBD files, and saves the message to MySQL.
3. Telemetry decoding	`src/stage3_TelemetryDecoder↩ Thread.*`	Decodes the SBD data into X-messages, assigns them to the float's dive, and writes listing/PHY files and dive records when a dive is complete.
4. Post-processing	`src/stage4b_PostProcessing↩ Thread.*`	Periodically writes each float's KML and status files and the master KML file.

The main thread (`src/Main.cpp`) parses the configuration, takes the run-time PID lock, loads the cached float data and metadata, starts the workers, and shuts them down in pipeline order when it receives a signal. Each stage drains its queue before exiting.

1.1.1 Source layout

```
src/
  Main.cpp, stage*_*.cpp/.h   daemon entry point and pipeline stages
  config/                     command-line/config-file options, MySQL connection settings
  data/                       floats (XFloat), dives (XFloatDiveData), mail messages, metadata
  data/xmsg/                  X-message decoders (GPS, profiles, drift, engineering, ice, wave, ...)
  utility/                    logging, MySQL/SQL table helpers, KML, queues, file-system helpers
bin/
  py/                         JSON conversion, plotting and summary-page tools
  xmtps-alamo.service         systemd unit for the decoder daemon
  xmtps-alamo-plot.service    systemd unit for the plot-generation loop
build/                        build-number script and counter
```

1.2 Building

XMTPS is a C++11 project built with Eclipse CDT managed builds (`.cproject`, `Debug/` and `Release/` makefiles) and g++ on Linux. It links against:

- Boost: `chrono`, `date_time`, `filesystem`, `iostreams`, `log`, `log_setup`, `program_options`, `regex`, `thread`
- `VMime` (IMAP and MIME parsing)
- MySQL Connector/C++ (`mysqlcppconn`)
- libkml (`kmlbase`, `kmldom`, `kmlengine`)
- pthreads

The build number embedded in the binary comes from `build/buildNo.txt`, which `build/incrementBuild.sh` bumps as a build step.

The code uses POSIX APIs (`open`/`O_EXCL`, `getpid`, signals), so it builds on Linux only.

1.3 Configuration

1.3.1 How options are read

- Every option can be given on the command line. The options in the "Configuration file options" tables below can also be set in the configuration file.
- The configuration file (`-c PATH`, default `../cfg/options.cfg`, relative to the working directory) must exist; the program stops if it is missing or contains an unknown option.
- Command-line values take precedence: the configuration file is read after the command line and only fills in options that were not given there.
- In the configuration file, options in a table's section are written as `name = value` under a `[section]` header (for example `host` under `[mail]`); on the command line they are written with the section as a prefix (for example `--mail.host maila.whoi.edu`).
- Boolean values are written as `0`/`1` or `false`/`true`, except the switches `-D` and `-R`, which take no value.
- `xmtps --help` prints all options with their defaults and exits.

1.3.2 Command-line only options

Option	Default	Description
`-c`, `--config PATH`	`../cfg/options.cfg`	Path to the configuration file.
`-D`, `--daemon`	off	Run as a daemon: keep running until a termination signal or until the run-time lock is lost. Without it, the program starts the pipeline, waits briefly, and exits.
`-h`, `--help`		Print all options with their defaults, then exit.
`-V`, `--version`		Print the version banner, then exit.
`-d`, `--debug`		Log at debug level.
`-v`, `--verbose`		Log informational messages as well (used if `-d` is not given).
`-q`, `--quiet`		Log only warnings and errors (used if neither `-d` nor `-v` is given).
`-r`, `--reprocess ID ...`	none	Reprocess all stored mail messages for the given instrument serial numbers before normal operation: their dive data is deleted and rebuilt from the database. Repeatable, and accepts several IDs.
`-R`, `--reprocess-all`	off	Reprocess the stored mail messages of every cached instrument at startup.

Option	Default	Description
-A, --reprocess-active 0|1	1	Reprocess every instrument that reported in the last 90 days at startup. **On by default.**
--phy-enable ID	none	Enable PHY file generation for an instrument (repeatable). The instrument is also reprocessed at startup.
--phy-disable ID	none	Disable PHY file generation for an instrument (repeatable).

With no logging option, the normal log level is used: operational messages, warnings and errors.

1.3.3 Configuration file options

General:

Option	Default	Description
-m, maxFetch N	127	Maximum number of mail messages fetched from the server, or files read from dir.mail, per batch.
-M, msgStartIdx N	1	Index of the first message on the mail server to fetch (server mode only).
-n, nProcThrd N	1	Number of mail processing threads.
-N, nDecoThrd N	1	Number of telemetry decoder threads.
-p, pid-file PATH	/var/lock/xmtps/lock.pid	Run-time lock (PID) file. Only one instance can hold it.

[dir] — directories (relative paths are relative to the working directory):

Option	Default	Description
dir.kml	kml	KML files.
dir.lis	lis	Intermediate listing files.
dir.log	log	Log files.
dir.meta	meta	Float metadata (.meta) files.
dir.phy	phy	PHY listing files.
dir.raw	raw	RAW mail message output files.
dir.mail	raw/incoming	RAW mail input files (file-system fetch mode).
dir.mail.processed	raw/processed	Processed RAW mail input files. Accepted but not currently used.
dir.sbd	sbd	Binary SBD message files.
dir.stat	stat	Status files.

[gen] — which output files to write:

Option	Default	Description
gen.kml	1	Google Earth KML files.
gen.lis	1	Intermediate listing files.
gen.phy	1	PHY listing files.
gen.raw	1	RAW mail message files.
gen.raw.format	1	RAW file format: 0 standard, 1 extended (adds headers, Iridium session details and an SBD hex dump).
gen.sbd	0	Binary SBD message files.
gen.stat	0	Status files.

[poc] — surface pressure offset correction (applied only for sensor types that need it; see Float metadata):

Option	Default	Description
poc.lis	1	Apply the correction in listing files.
poc.phy	1	Apply the correction in PHY files.

[mail] — mail server and message validation:

Option	Default	Description
mail.host	maila.whoi.edu	Mail server host name or IP address.
mail.protocol	imap	Mail server protocol.
mail.user	alamo	Mail account user name.
mail.password	none	Mail account password.
mail.folder	empty (server default)	Folder to read messages from, e.g. INBOX.
mail.fetchMode	0	0 fetch from the mail server; 1 read RAW mail files from dir.mail (renamed to .err if they can't be parsed).
mail.moveMessages	0	After processing, move server messages to the folder INBOX.<YYYYMM> for their header date.
mail.xDlyS	60	Seconds to wait before checking again when no new messages are found.
mail.validSenderRegex	(sbdservice@sbd.↩ iridium.↩ com|service@sbd.pac.↩ disa.mil)	Senders whose messages are accepted.
mail.validSubjectRegex	SBD Msg From Unit↩ : (\d+)	Accepted subjects; the captured number is the IMEI.
mail.validSbdFilename↩ Regex	(\d+)_(\d{6})\.sbd	Accepted SBD attachment file names.

The mail user and password may contain any characters; they are percent-encoded when the server URL is built.

[sql] — MySQL server:

Option	Default	Description
sql.host	127.0.↩ 0.1	Server host name or IP address.
sql.port	3306	Server port.
sql.protocol	tcp	Connection protocol.
sql.schema	float_db	Database schema name.
sql.user	none	Account user name.
sql.password	none	Account password.

1.3.4 Example configuration file

Credentials omitted:

```
maxFetch                = 16
nProcThrd               = 1
nDecoThrd               = 1

[dir]
meta                    = /data1/alamo/meta/dac
lis                     = /data1/alamo/data/list
raw                     = /data1/alamo/data/raw

[gen]
kml                     = true
stat                    = true

[mail]
host                    = maila.whoi.edu
protocol                = imap
folder                  = INBOX
fetchMode               = 0
validSenderRegex        = (sbdservice@sbd\.iridium\.com|service@sbd\.pac\.disa\.mil)
validSubjectRegex       = SBD Msg From Unit: (\d+)

[sql]
host                    = 127.0.0.1
port                    = 3306
schema                  = xmtps_alamo
```

Keep real configuration files, with their credentials, out of version control (cfg/ is ignored).

1.3.5 Float metadata

Each float's metadata is read at startup from `<dir.meta>/<DAC-ID>_<serial>.meta`, a plain-text file with the field name in the first 40 columns and the value after it. Files must be plain ASCII text (not RTF). The daemon uses, among others, the WMO ID, WMO instrument type, launch time (`dd mm yyyy hh mm`) and launch position (`lat latm lon lonm`, with the minutes carrying the same sign as the degrees). A field that fails to parse is logged and skipped. DAC ID `9999` marks a placeholder file and is ignored.

Supported WMO instrument types: 866, 867, 868 (ALAMO) and 873, 875, 876 (ALTO). The surface pressure offset correction is applied for RBR conductivity sensors (867, 875) and for floats without one (866), and is skipped for SBE sensors (868, 876) and for 873.

1.4 Running

```
xmtps --config /data1/alamo/etc/xmtps.cfg --daemon --pid-file /data1/alamo/var/lock/xmtps.pid
```

See Configuration for every option. Note that `--reprocess-active` is on by default, so each start reprocesses the floats active in the last 90 days; pass `-A 0` to skip it.

The PID file is created exclusively at startup, so only one instance can run at a time. It is removed on shutdown only by the process that owns it.

Logs are written to `<dir.log>/YYYY-MM-DD.log`, with warnings and errors also in `<dir.log>/YYYY-MM-DD.error.log`.

1.4.1 Output files

Output	Location
RAW mail	`<dir.raw>/<serial>/raw<serial>-<MOMSN>-<dive>-<packet>.txt`
Listings	`<dir.lis>/<serial>/lis<serial>-<dive>.txt`
PHY, KML, status, SBD	under `dir.phy`, `dir.kml`, `dir.stat`, `dir.sbd`

Decoded data, messages, floats and dives are also stored in the MySQL schema set by `sql.schema`.

1.5 Database schema

XMTPS stores its state in five InnoDB tables in the MySQL schema named by `sql.schema` (production: `xmtps_alamo`). Each table is mapped to one C++ class through a static `util::mysql::SqlTableDescription`. The description lists the column names and the getter and setter used to read and write each column, and generates the `SELECT`, `DELETE` and `INSERT ... ON DUPLICATE KEY UPDATE` statements. Saving an object therefore inserts a new row or overwrites the existing row with the same primary key.

Table	Class	Primary key	One row per	Written by
`XFloat`	`XFloat`	`id_instrument`	instrument	startup, stage 3 and shutdown
`XFloat_Metadata`	`XFloat_Metadata`	`id_instrument`	instrument, from its `.meta` file	saved with `XFloat`
`XFloatDive`	`XFloatDive_Data`	`id_instrument`, `id_dive`	dive cycle	stage 3
`XGPS`	`XGPS`	`id_instrument`, `id_dive`, `datetime`	GPS fix	stage 3, saved with the dive
`XFloatMail`	`XFloatMail_Message`	`id_dive`, `id_imei`, `id_sbd_momsn`, `id_sbd_packet`, `id_msg`	mail message (one SBD packet)	stage 2

Relationships (foreign keys):

```
XFloat (id_instrument)
 +- XFloatMetadata.id_instrument              fk_XFloatMetadata_1
 `- XFloatDive.id_instrument                  fk_XFloatDive_1
      `- XGPS (id_instrument, id_dive)        fk_XGPS_1
XFloatMail                                    no foreign key; linked by id_instrument / id_imei
```

Because of these keys a float's `XFloat` row must exist before its dives are saved, and a dive's row before its GPS fixes; `XFloatDiveData::SaveObjectToDatabase()` saves the parent float first. `XFloatMail` has no foreign key, so a message can be stored before its float is known (stage 2).

Common identifier columns:

Column	Meaning
id_instrument	Float serial number (the instrument ID used throughout XMTPS).
id_imei	IMEI of the float's Iridium modem.
id_wmo	WMO platform number (-1 if not assigned).
id_dac	Argo data assembly center ID.
id_dive	Dive (profile) number.

Unset values use sentinel defaults rather than NULL: -1 for IDs, 1400-01-01 00:00:00 for dates, 99.↩ 99999/999.99999 for latitude/longitude, and 9999 for gains and offsets.

1.5.1 XFloat

The current state of each float, reloaded at startup so that processing resumes where it left off.

- Identifiers: id_instrument, id_imei, id_wmo, id_dac, and argo_instrument_type (Argo instrument type code).
- Calibration: gain_* and offset_* for pressure, salinity and temperature, with separate *_surface values for the surface sample.
- Latest positions: latest_gps_position_* and latest_iridium_position_* (date/time, dive number, latitude, longitude, and Iridium CEP radius), used for the KML and status files.
- Iridium sequence numbers: latest_sbd_momsn and latest_sbd_mtmsn, the most recent mobile-originated and mobile-terminated message sequence numbers.
- Output switches: phy_file_generation_is_enabled and netcdf_file_generation_is_↩ enabled.
- Mission: start_of_mission_datetime, and xmsg_version_major/xmsg_version_minor (the X-Message format version the float reports).
- latest_iridium_startup_position_datetime and latest_iridium_startup_sbd_↩ momsn exist in the production table but are not read or written by the current code.

1.5.2 XFloatMetadata

Deployment metadata parsed from the float's .meta file (see Float metadata).

- Deployment: deployment_datetime, deployment_latitude/deployment_longitude, deployment_position_valid, the deployment_*_status fields, deployment_platform, deployment_cruise_id and deployment_type.
- People: deployer, deployer_address, principal_investigators, principal_investigators↩ _address, principal_investigators_originating_country.
- Instrument: manufacturer, wmo_instrument_type (for example 873, 875 or 876 for ALTO floats), and wmo_recorder_type.
- Other: comment, and local_path, the path of the .meta file.

1.5.3 XFloatDive

One dive cycle, built from all the SBD messages received for it.

- complete: the dive has a complete profile, known sensor gains and offsets, contiguous message IDs and a valid position.
- num_sbd_msgs: the number of SBD messages received for the dive.
- iridium_position_datetime_first/_last: the first and last Iridium fix times for the dive.
- intermediate_listing_file_path and phy_file_path: the output files written for the dive.
- surface_pres_*: statistics of the surface pressure samples (min, max, mean, variance, valid).
- pres_offs*: the pressure offset applied to the profile, with the number of dives and samples it was computed from and whether it is valid.

1.5.4 XGPS

One GPS fix reported in an X-Message.

- datetime, latitude, longitude and valid.
- id_type: the GPS message type.
- Fix quality: hdop, num_satellites, snr_min, snr_mean, snr_max, and time_to_fix.

1.5.5 XFloatMail

Every Iridium SBD mail message received, so that a float can be reprocessed from the database with -r/--reprocess.

- Keys: id_imei, id_sbd_momsn, id_sbd_packet and id_dive identify the packet; id_msg is the mail Message-ID; id_msg_uid is the IMAP UID; id_sbd_mtmsn is the MTMSN.
- Mail: msg_header_datetime, msg_header_from, msg_header_subject, msg_header_raw↩ _text, msg_body_text and msg_raw_text, each with an *_is_valid flag, and msg_is_valid for the whole message.
- Iridium session: iridium_position_* (latitude, longitude, CEP radius, date/time, valid), and sbd_↩ session_status_code/sbd_session_status_string.
- Attachment: sbd_data (the raw SBD bytes), sbd_data_is_valid, sbd_file_name and sbd_file↩ _size_bytes.
- Files written: sbd_file_local_path and raw_file_local_path.
- id_dac and id_wmo exist in the production table but are not written by the current code, so they keep their default of -1.

Reprocessing (-r ID) deletes the float's XGPS and XFloatDive rows and rebuilds them from its XFloatMail rows. The full CREATE TABLE statements are in the Doxygen documentation (the "Libraries and Database Schema" page in doc/xmtps.pdf).

1.6 Deployment

The bin/*.service files are systemd units:

- xmtps-alamo.service runs the decoder daemon with Restart=always and RestartSec=30, so a startup failure (for example MySQL being unavailable) keeps retrying without hitting systemd's start limit.
- xmtps-alamo-plot.service runs bin/py/update-plots-service.sh.

After installing or changing a unit file, run sudo systemctl daemon-reload.

1.7 Python tools (bin/py)

Requires Python 3 with numpy, scipy, pandas, plotly (plus kaleido for PNG export), jsonpickle, geographiclib, pytz, python-dateutil, Pillow and yattag.

Script	Purpose
xmtps-list2json.py	Converts listing files into per-dive JSON (<out>/<serial>/dives/<serial>_↩ <dive>.json), adding density and σ_0 from RhoLUT.py and wave statistics from FloatClasses.py.
xmtps-aggregate-json.py	Aggregates a float's dive JSON into per-float JSON files (<serial>_profile.json, _wave_spectra.json, _↩ summary.json, ...) and, with -p DIR, writes the plot images and the float's HTML page.
xmtps-generate-summary-page.py	Builds the fleet summary page from the per-float _summary.↩ json files.
update-plots-service.sh	Loop used by the plot service: regenerates JSON and plots for floats with recent listings, then the summary page.

All three scripts share JSON version 0.11; JSON files with a different version are ignored.

1.7.1 Command-line options

Each script takes one or more input paths (directories and/or files) as positional arguments. Paths matching an include pattern are tested with a regular-expression search on the full path (using / separators), so a pattern only needs to match part of it.

Options common to all three scripts:

Option	Default	Description
-d, --debug		Print debug output.
-v, --verbose		Print informational output (used if -d is not given).
-q, --quiet		Print only errors (used if neither -d nor -v is given).
-H, --halt-on-error	off	Stop at the first error instead of logging it and continuing.
-i, --include REGEX	all files	Process only files whose path matches REGEX. Repeatable; a file is processed if it matches any pattern.
-o, --output DIR	.	Output directory (see each script).

With no logging option, only warnings and errors are printed.

xmtps-list2json.py [options] PATH ... — converts listing files (PATH may be listing files or directories containing them) into per-dive JSON files in <output>/<serial>/dives/:

Option	Default	Description
-r, --recursive	off	Descend into subdirectories of the input directories.
-x, --exclude REGEX	none	Skip directories and files whose path matches REGEX. Repeatable.
-f, --force	off	Rewrite every dive JSON file, even if it is newer than its listing file.
-s, --skip	off	Skip listing files whose dive JSON already exists, even if the listing is newer.
-S, --strip-path REGEX	.*/alamo/	Remove the matching part of the listing file path before storing it in the JSON.

Without -f or -s, a listing file is converted only if its dive JSON doesn't exist yet or is older than the listing file.

xmtps-aggregate-json.py [options] PATH ... — reads dive JSON files (PATH is usually a float's dives directory) and writes the per-float aggregate JSON files to <output>/<serial>/:

Option	Default	Description
-p, --plots DIR	none	Also write the plot images, plot HTML files and the float's index.html page to DIR/<serial>/. DIR must exist.
-r, --recursive	off	Descend into subdirectories of the input directories.
-x, --exclude REGEX	none	Skip subdirectories whose path matches REGEX (applies to directories only). Repeatable.
-f, --force	off	Process every directory, ignoring its .lastprocessed file.
-S, --strip-path REGEX	.*/alamo/	Remove the matching part of file paths before storing them in the JSON and the plot pages.
-D, --display-figures	off	Also open each figure in a web browser as it is generated (interactive use).
-s, --skip		Accepted but has no effect in this script.

Without -f, a directory is skipped when none of its included files is newer than the time in its .lastprocessed file. That file is written only after all aggregate JSON and plot output has been written, so a failed or interrupted run is retried next time. Use -f after changing RhoLUT.py or FloatClasses.py.

xmtps-generate-summary-page.py [options] PATH ... — reads the per-float <serial>_summary.json files (searching input directories recursively) and writes the fleet summary page to <output>/index.html. DIR for -o must exist. It takes only the common options above, and always rebuilds the page from all matching summary files.

Example (as run by update-plots-service.sh for one float):

```
xmtps-list2json.py -i "/data1/alamo/data/list/[0-9]+/lis[0-9]+-(-?)[0-9]+.txt" \
    -x "/\..*" -o /data1/alamo/data/json /data1/alamo/data/list/9308
xmtps-aggregate-json.py -i "/data1/alamo/data/json/[0-9]+/dives/[0-9]+_-?[0-9]+.json" \
    -o /data1/alamo/data/json -p /data1/alamo/data/plots2 /data1/alamo/data/json/9308/dives
xmtps-generate-summary-page.py -i "/data1/alamo/data/json/[0-9]+/[0-9]+_summary.json" \
    -o /data1/alamo/data/plots2 /data1/alamo/data/json
```

1.8 Changelog

1.8.1 2026-10-24

- Added this README.

1.8.2 2026-10-23

Decoder daemon:

- Fixed the mail retrieval thread busy-looping on an unparseable mail file (now renamed to `.err`), and replaying a whole batch when one server message failed.
- Worker threads now drain their queues on shutdown, and termination flags are atomic.
- Fixed the latest Iridium position being compared against the GPS time instead of the Iridium time.
- Fixed composite Iridium positions: latitude/longitude were swapped in the variance calculation, and longitudes are now averaged correctly across ±180°.
- Mail parsing: "Message Size" is now parsed, the Date header timezone is converted to UTC correctly, and SBD attachments must be at least 8 bytes.
- The mail fetch limit is applied after filtering for `.raw`/`.txt` files, and the mail user and password are percent-encoded in the server URL.
- Status files now follow `gen.stat` instead of `gen.kml`; archived floats go into the KML "archive" folder; floats with a single GPS fix get a latest-position marker.
- Metadata: launch times at hour or minute 0 are accepted, a bad field no longer stops parsing of the rest of the file, launch positions accept any whitespace, and DAC ID 9999 placeholders are ignored.
- The run-time PID lock is created exclusively and removed only by its owner.
- The post-processing thread keeps going when one float's KML or status file fails.
- A float's record is saved before its dives (fixes foreign-key failures for new floats), and the shutdown save continues past a failing float.
- Added ALTO WMO instrument types 873, 875 and 876.
- Fixed ice engineering (v0x11) messages being rejected whenever they reported error codes.
- Identical retransmitted SBD packets are no longer decoded twice.
- Stopped tracking the Debug/Release binaries; build number 3287.

Python tools:

- Updated to JSON version 0.11.
- Fixed the RhoLUT maximum temperature (39 °C, was 34 °C).
- Fixed drift fill values, a crash on 0 V pump records, and `.lastprocessed` being written before output succeeded.
- Wave directional coefficients now use the displacement formulation; `r1` and `r2` are distinct and lie in [0, 1].
- Fixed the drift σ_0 average plot title, wave plots not being linked when a float has one wave dive, and plot path handling.

Operations:

- The decoder service waits 30 s between restarts (`RestartSec=30`).

1.8.3 2026-08-11

- The post-processing thread closes its SQL connection after each pass, avoiding "lost connection to MySQL server" errors.

1.8.4 2026-08-06

- Added mission parameter support for firmware 0x0A0C0001 (10.12.0.1, RBR).

1.8.5 2026-08-05

- Initial commit.

1.9 Author and license

Written and maintained by Alexander K. Ekholm, WHOI Float Group, Department of Physical Oceanography, Woods Hole Oceanographic Institution. Released under the GNU General Public License.

Chapter 2

Libraries and Database Schema

This page describes the external libraries XMTPS uses and its MySQL database schema. For an overview, building, configuration and the command-line options, see the main page (XMTPS — X-Message Telemetry Processing Service).

2.1 Libraries

2.1.1 Boost C++ Libraries

Several of the Boost C++ libraries are utilized by xmtps.

See http://www.boost.org/.

2.1.1.1 program_options

The program_options library allows program developers to obtain program options, that is (name, value) pairs from the user, via conventional methods such as command line and config file.

See http://www.boost.org/doc/libs/1_55_0/doc/html/program_options.html.

2.1.1.2 filesystem

The Boost filesystem library provides facilities to manipulate files and directories, and the paths that identify them.

See http://www.boost.org/doc/libs/1_55_0/libs/filesystem/doc/index.htm.

2.1.1.3 log

The Boost log library provides facilities for logging, formatting, and manipulating logger attributes.

See http://www.boost.org/doc/libs/1_55_0/libs/log/doc/html/log/tutorial.html.

2.1.1.4 thread

Boost.Thread enables the use of multiple threads of execution with shared data in portable C++ code. It provides classes and functions for managing the threads themselves, along with others for synchronizing data between the threads or providing separate copies of data specific to individual threads.

Seehttp://www.boost.org/doc/libs/1_56_0/doc/html/thread.html.

2.1.2 MySQL C++ Connector

The MySQL C++ connector provides an API for accessing and manipulating a MySQL database. MySQL is used as a back-end database solution for storing and retrieving persistent data that is needed by the application.

See http://dev.mysql.com/doc/connector-cpp/en/.

2.1.3 VMime MIME & Mail Library for C++

VMime provides the IMAP connection to the mail server and parsing of the mail messages and their SBD attachments.

See http://www.vmime.org/.

2.1.4 libkml

libkml is used to generate the Google Earth KML files for each instrument and the master KML file.

See https://github.com/libkml/libkml.

A MySQL server is used for persistent backend storage.

2.2 SQL Database Schema

XMTPS stores its state in five InnoDB tables in the MySQL schema named by `sql.schema` (production: `xmtps_alamo`). Each table is mapped to one class by a static util::mysql::SqlTableDescription member, which lists each column with its getter and setter and generates the SELECT, DELETE and INSERT ... ON DUPLICATE KEY UPDATE statements:

- argo::iridium::xfloat::XFloat - table XFloat, one row per instrument.
- argo::iridium::xfloat::XFloatMetadata - table XFloatMetadata, instrument metadata parsed from the metadata file.
- argo::iridium::xfloat::XFloatDiveData - table XFloatDive, all telemetered data for one dive cycle.
- util::xmsg::XGPS - table XGPS, one GPS fix.
- argo::iridium::xfloat::XFloatMailMessage - table XFloatMail, one mail message.

Foreign keys: XFloatMetadata and XFloatDive reference XFloat (id_instrument), and XGPS references XFloatDive (id_instrument, id_dive). XFloatMail has no foreign key, so messages can be stored before their float is known. The column-by-column description is in the "Database schema" section of the main page (XMTPS — X-Message Telemetry Processing Service). The figure below shows the schema; the CREATE TABLE statements that follow are taken from the production database (October 2026) and take precedence over the figure.

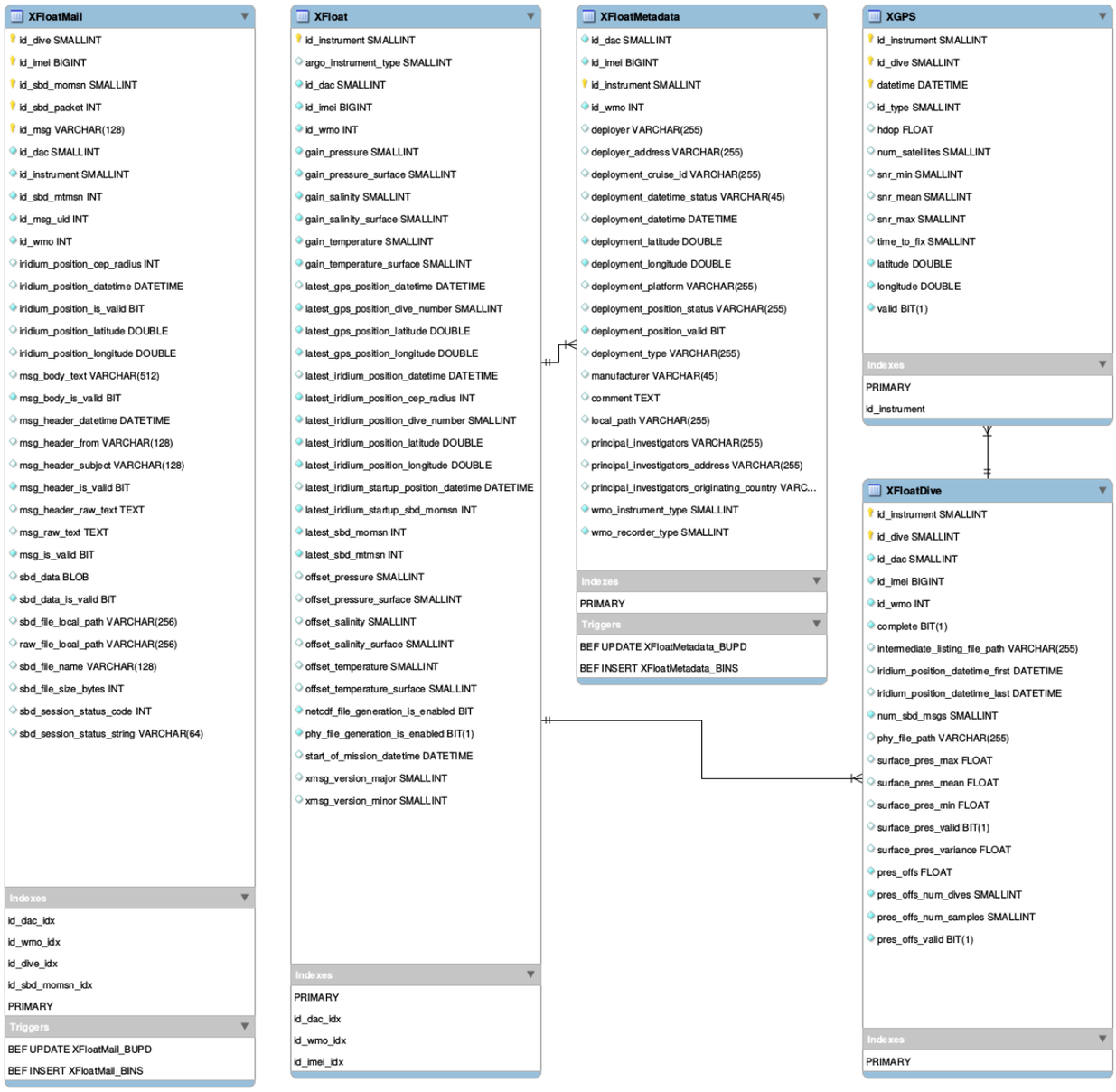


Figure 2.1 SQL Database Schema

2.2.1 Table XFloat

The current state of each float (identifiers, calibration, latest positions and SBD sequence numbers). Mapped to argo::iridium::xfloat::XFloat.

```
CREATE TABLE XFloat (
  id_instrument smallint NOT NULL,
  argo_instrument_type smallint unsigned DEFAULT '99',
  id_dac smallint NOT NULL DEFAULT '-1',
  id_imei bigint NOT NULL DEFAULT '-1',
  id_wmo int NOT NULL DEFAULT '-1',
  gain_pressure smallint unsigned NOT NULL DEFAULT '9999',
  gain_pressure_surface smallint unsigned NOT NULL DEFAULT '9999',
  gain_salinity smallint unsigned NOT NULL DEFAULT '9999',
  gain_salinity_surface smallint unsigned NOT NULL DEFAULT '9999',
  gain_temperature smallint unsigned NOT NULL DEFAULT '9999',
  gain_temperature_surface smallint unsigned NOT NULL DEFAULT '9999',
  latest_gps_position_datetime datetime NOT NULL DEFAULT '1400-01-01 00:00:00',
  latest_gps_position_dive_number smallint NOT NULL DEFAULT '-999',
  latest_gps_position_latitude double NOT NULL DEFAULT '99.99999',
  latest_gps_position_longitude double NOT NULL DEFAULT '999.99999',
  latest_iridium_position_datetime datetime NOT NULL DEFAULT '1400-01-01 00:00:00',
  latest_iridium_position_cep_radius int NOT NULL DEFAULT '-1',
  latest_iridium_position_dive_number smallint NOT NULL DEFAULT '-999',
  latest_iridium_position_latitude double NOT NULL DEFAULT '99.99999',
  latest_iridium_position_longitude double NOT NULL DEFAULT '999.99999',
  latest_iridium_startup_position_datetime datetime NOT NULL DEFAULT '1400-01-01 00:00:00',
  latest_iridium_startup_sbd_momsn int NOT NULL DEFAULT '-1',
  latest_sbd_momsn int NOT NULL DEFAULT '-1',
  latest_sbd_mtmsn int NOT NULL DEFAULT '-1',
  offset_pressure smallint unsigned DEFAULT '9999',
  offset_pressure_surface smallint unsigned DEFAULT '9999',
  offset_salinity smallint unsigned DEFAULT '9999',
  offset_salinity_surface smallint unsigned DEFAULT '999',
  offset_temperature smallint unsigned DEFAULT '9999',
  offset_temperature_surface smallint unsigned DEFAULT '9999',
  netcdf_file_generation_is_enabled bit(1) NOT NULL DEFAULT b'0',
  phy_file_generation_is_enabled bit(1) DEFAULT b'0',
```

```
  start_of_mission_datetime datetime NOT NULL DEFAULT '1400-01-01 00:00:00',
  xmsg_version_major smallint unsigned DEFAULT '255',
  xmsg_version_minor smallint unsigned DEFAULT '255',
  PRIMARY KEY (id_instrument),
  KEY id_dac_idx (id_dac),
  KEY id_wmo_idx (id_wmo),
  KEY id_imei_idx (id_imei)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb3;
```

2.2.2 Table XFloatMetadata

Deployment metadata parsed from the float's `.meta` file. Mapped to argo::iridium::xfloat::XFloatMetadata.

```
CREATE TABLE XFloatMetadata (
  id_dac smallint NOT NULL DEFAULT '-1',
  id_imei bigint NOT NULL DEFAULT '-1',
  id_instrument smallint NOT NULL,
  id_wmo int NOT NULL DEFAULT '-1',
  deployer varchar(255) DEFAULT NULL,
  deployer_address varchar(255) DEFAULT NULL,
  deployment_cruise_id varchar(255) DEFAULT NULL,
  deployment_datetime_status varchar(45) DEFAULT NULL,
  deployment_datetime datetime DEFAULT '1400-01-01 00:00:00',
  deployment_latitude double NOT NULL DEFAULT '99.99999',
  deployment_longitude double NOT NULL DEFAULT '999.99999',
  deployment_platform varchar(255) DEFAULT NULL,
  deployment_position_status varchar(255) DEFAULT NULL,
  deployment_position_valid bit(1) NOT NULL DEFAULT b'0',
  deployment_type varchar(255) DEFAULT NULL,
  manufacturer varchar(45) DEFAULT NULL,
  comment text,
  local_path varchar(255) DEFAULT NULL,
  principal_investigators varchar(255) DEFAULT NULL,
  principal_investigators_address varchar(255) DEFAULT 'WHOI, Woods Hole, MA',
  principal_investigators_originating_country varchar(255) DEFAULT 'USA',
  wmo_instrument_type smallint NOT NULL DEFAULT '-1',
  wmo_recorder_type smallint NOT NULL DEFAULT '-1',
  PRIMARY KEY (id_instrument),
  CONSTRAINT fk_XFloatMetadata_1 FOREIGN KEY (id_instrument) REFERENCES XFloat (id_instrument)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb3;
```

2.2.3 Table XFloatDive

One row per dive cycle. Mapped to argo::iridium::xfloat::XFloatDiveData.

```
CREATE TABLE XFloatDive (
  id_instrument smallint NOT NULL,
  id_dive smallint NOT NULL,
  id_dac smallint NOT NULL DEFAULT '-1',
  id_imei bigint NOT NULL DEFAULT '-1',
  id_wmo int NOT NULL DEFAULT '-1',
  complete bit(1) NOT NULL DEFAULT b'0',
  intermediate_listing_file_path varchar(255) DEFAULT NULL,
  iridium_position_datetime_first datetime DEFAULT '1400-01-01 00:00:00',
  iridium_position_datetime_last datetime DEFAULT '1400-01-01 00:00:00',
  num_sbd_msgs smallint unsigned NOT NULL DEFAULT '0',
  phy_file_path varchar(255) DEFAULT NULL,
  surface_pres_max float DEFAULT '-9.999',
  surface_pres_mean float DEFAULT '-9.999',
  surface_pres_min float DEFAULT '-9.999',
  surface_pres_valid bit(1) DEFAULT b'0',
  surface_pres_variance float DEFAULT '-9.999',
  pres_offs float NOT NULL DEFAULT '0',
  pres_offs_num_dives smallint NOT NULL DEFAULT '-1',
  pres_offs_num_samples smallint NOT NULL DEFAULT '-1',
  pres_offs_valid bit(1) NOT NULL DEFAULT b'0',
  PRIMARY KEY (id_instrument,id_dive),
  CONSTRAINT fk_XFloatDive_1 FOREIGN KEY (id_instrument) REFERENCES XFloat (id_instrument)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb3;
```

2.2.4 Table XGPS

One row per GPS fix. Mapped to util::xmsg::XGPS.

```
CREATE TABLE XGPS (
  id_instrument smallint NOT NULL,
  id_dive smallint NOT NULL,
  datetime datetime NOT NULL DEFAULT '1400-01-01 00:00:00',
  id_type smallint DEFAULT NULL,
  hdop float DEFAULT NULL,
  num_satellites smallint unsigned DEFAULT NULL,
  snr_min smallint unsigned DEFAULT NULL,
  snr_mean smallint unsigned DEFAULT NULL,
  snr_max smallint unsigned DEFAULT NULL,
  time_to_fix smallint unsigned DEFAULT NULL,
  latitude double NOT NULL DEFAULT '99.99999',
  longitude double NOT NULL DEFAULT '999.99999',
  valid bit(1) NOT NULL DEFAULT b'0',
  PRIMARY KEY (id_instrument,id_dive,datetime),
  KEY id_instrument (id_instrument),
  CONSTRAINT fk_XGPS_1 FOREIGN KEY (id_instrument, id_dive) REFERENCES XFloatDive (id_instrument, id_dive)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb3;
```

2.2.5 Table XFloatMail

One row per received Iridium SBD mail message; the source for reprocessing. Mapped to argo::iridium::xfloat::XFloatMailMessage.

```
CREATE TABLE XFloatMail (
  id_dive smallint NOT NULL DEFAULT '-99',
  id_imei bigint unsigned NOT NULL,
  id_sbd_momsn smallint NOT NULL DEFAULT '-1',
  id_sbd_packet int NOT NULL DEFAULT '-1',
  id_msg varchar(128) NOT NULL,
  id_dac smallint NOT NULL DEFAULT '-1',
  id_instrument smallint NOT NULL DEFAULT '-1',
  id_sbd_mtmsn int NOT NULL DEFAULT '-1',
  id_msg_uid int NOT NULL DEFAULT '-1',
  id_wmo int NOT NULL DEFAULT '-1',
  iridium_position_cep_radius int DEFAULT NULL,
  iridium_position_datetime datetime DEFAULT NULL,
  iridium_position_is_valid bit(1) NOT NULL DEFAULT b'0',
  iridium_position_latitude double DEFAULT NULL,
  iridium_position_longitude double DEFAULT NULL,
  msg_body_text varchar(512) DEFAULT NULL,
  msg_body_is_valid bit(1) NOT NULL DEFAULT b'0',
  msg_header_datetime datetime DEFAULT NULL,
  msg_header_from varchar(128) DEFAULT NULL,
  msg_header_subject varchar(128) DEFAULT NULL,
  msg_header_is_valid bit(1) NOT NULL DEFAULT b'0',
  msg_header_raw_text text,
  msg_raw_text text,
  msg_is_valid bit(1) NOT NULL DEFAULT b'0',
  sbd_data blob,
  sbd_data_is_valid bit(1) NOT NULL DEFAULT b'0',
  sbd_file_local_path varchar(256) DEFAULT NULL,
  raw_file_local_path varchar(256) DEFAULT NULL,
  sbd_file_name varchar(128) DEFAULT NULL,
  sbd_file_size_bytes int DEFAULT NULL,
  sbd_session_status_code int DEFAULT NULL,
  sbd_session_status_string varchar(64) DEFAULT NULL,
  PRIMARY KEY (id_dive,id_imei,id_sbd_momsn,id_sbd_packet,id_msg),
  KEY id_dac_idx (id_dac),
  KEY id_wmo_idx (id_wmo),
  KEY id_dive_idx (id_dive),
  KEY id_sbd_momsn_idx (id_sbd_momsn),
  KEY id_instrument (id_instrument)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb3;
```

Chapter 3

Deprecated List

Member N_LOG_LEVEL_NAMES

Log names are no long used.

Namespace util::ftp

FTP client integration features have been scrapped. This namespace is no longer utilized.

Chapter 4

Todo List

Member argo::iridium::xfloat::operator>> (istream &in, XFloatMetadata &metadata)

Implement the functionality for parsing additional metadata field from stream

Class argo::iridium::xfloat::XFloat

Devise a method to actively check for (1) changes in the metadata files and (2) newly created metadata files. Load the metadata from the new or modified files and assign them to the corresponding float object. If necessary, retroactively process the mail messages for these instruments. Persist updates to the SQL database as necessary.

Member argo::iridium::xfloat::XFloat::start_of_mission_datetime_

Implement an alternative method to assign the value of this field if the date/time reported by the start-of-mission GPS position is invalid or unavailable.

Class argo::iridium::xfloat::XFloatDiveData

Devise a scheme to ensure that the dive data objects do not remain in memory indefinitely if one or more message for the dive or cycle are not received and the dive data object is not completed.

Member argo::iridium::xfloat::XFloatDiveData::DecodeXMsgData (const vector< uint8_t > &data, shared_ptr< vector< uint8_t > > xmsg_id_types=shared_ptr< vector< uint8_t > >()) ATTR_NO_EXCEPTIONS

Verify the X-Message checksum value during the decoding of the X-Message data.

Member argo::iridium::xfloat::XFloatDiveData::GeneratePhyString () const

Implement the functionality for generate a PHY string representation of a dive data object for a Solo-II float.

Implement the functionality for generate a PHY string representation of a dive data object for a Deep Argo float.

Member argo::iridium::xfloat::XFloatDiveData::HasValidIridiumPosition () const

Implement the functionality to determine if a dive data object has a valid Iridium position that accurately represents the position and date/time of the profile data for this dive cycle.

Member argo::iridium::xfloat::XFloatDiveData::ResetData ()

Implement NC variable generation for XFloatDiveData

Class argo::iridium::xfloat::XFloatMetadata

Devise a method to actively check for (1) changes in the metadata files and (2) newly created metadata files. Load the metadata from the new or modified files and assign them to the corresponding float object. If necessary, retroactively process the mail messages for these instruments. Persist updates to the SQL database as necessary.

Member util::mysql::SqlTableDescription< T >::SelectValueFromDatabase (const string &what, const string &where="", const string &groupBy="", const string &orderBy="", const unsigned int &offset=0) const

Implement the functionality to parse values of type vector<uint8_t> from the result set of an SQL query.

Member util::xmsg::XAcousticPosition::DecodeXMsgData (const vector< uint8_t > &data)

Implement the functionality for decoding the acoustic position message.

Member util::xmsg::XAcousticPosition::GenerateListingString () const

Implement the functionality for generating a listing string for the acoustic position message.

Member util::xmsg::XAlamoMissionConfiguration::ctf_

Determine gain for profile cutoff pressure field.

Member util::xmsg::XAlamoMissionConfiguration::ctl_

Determine gain for pressure sensor offset reset lower pressure limit field.

Member util::xmsg::XAlamoMissionConfiguration::ctu_

Determine gain for pressure sensor offset reset lower pressure limit field.

Member util::xmsg::XAlamoMissionConfiguration::DecodeXMsgData (const vector< uint8_t > &data)

Implement ALAMO mission parameter: SSP.

Implement ALAMO mission parameter: SSP.

Class util::xmsg::XArgo

Add support for the Solo-II-type Argo data message.

Add support for the Deep-Argo-type Argo data message.

Member util::xmsg::XArgo::DecodeXMsgData (const vector< uint8_t > &data)

Implement the functionality to decode the argo data message for Solo-II floats.

Implement the functionality to decode the argo data message for Deep Argo floats.

Member util::xmsg::XArgo::GenerateListingString () const

Implement the functionality for generating a listing string for a Solo-II-type Argo data message.

Implement the functionality for generating a listing string for a Deep-Argo-type Argo data message.

Member util::xmsg::XEngAbort::DecodeXMsgData (const vector< uint8_t > &data)

Implement the functionality to decode the abort/beacon mode engineering X-Message from the binary data.

Member util::xmsg::XEngAbort::GenerateListingString () const

Implement the functionality for generating a listing string for the Solo-II abort/beacon mode engineering data message.

Class util::xmsg::XEngBITPass

Add support for Solo-II type bit passed engineering data message.

Member util::xmsg::XEngDescent::oil_valve_closed_position_during_fall_pump_mv_

Clarify the description of the "oil valve position during fall pump" field.

Member util::xmsg::XEngDiagnostic::DecodeXMsgData (const vector< uint8_t > &data)

Implement the functionality for decoding the Solo-II diagnostic engineering data message.

Member util::xmsg::XEngDiagnostic::GenerateListingString () const

Implement the functionality for generating a listing string for the Solo-II diagnostic engineering data message.

Member util::xmsg::XEngStandard::DecodeXMsgData (const std::vector< uint8_t > &data)

Implement the functionality for decoding the Solo-II standard dive/cycle engineering data message.

Member util::xmsg::XEngStandard::GenerateListingString () const

Implement the functionality for generating a listing string for the Solo-II standard dive engineering data message.

Member util::xmsg::XEngSurface::ctd_status_code_

Define status code values for the ALAMO surface engineering data message.

Member util::xmsg::XMfaMissionCfg::ctf_

Determine gain for profile cutoff pressure field.

Member util::xmsg::XMfaMissionCfg::ctl_

Determine gain for pressure sensor offset reset lower pressure limit field.

Member util::xmsg::XMfaMissionCfg::ctu_

Determine gain for pressure sensor offset reset lower pressure limit field.

Member util::xmsg::XMissionCmdEcho::DecodeXMsgData (const vector< uint8_t > &data)

Implement the functionality to decode the ground station command echo-back X-Message from the binary message data.

Member util::xmsg::XMissionCmdEcho::GenerateListingString () const

Implement the functionality for generating a listing string for the ground station upload command echo-back message.

Chapter 5

Namespace Index

5.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

argo
The Argo instrument namespace . . . 37
argo::iridium
The Argo Iridium instrument namespace . . . 37
argo::iridium::xfloat
The X-Float instrument namespace . . . 37
boost
The Boost C++ library namespace . . . 41
boost::container
Boost container library namespace . . . 41
util
The utility namespace . . . 41
util::config
The configuration utility namespace . . . 42
util::date
Date utility namespace . . . 42
util::file
The filesystem utility namespace . . . 43
util::ftp
The ftp utility namespace . . . 43
util::log
The logging utility namespace . . . 43
util::mail
The mail utility namespace . . . 44
util::math
The math utility namespace . . . 45
util::mysql
The MySQL utility namespace . . . 45
util::position
The position utility namespace . . . 46
util::sig
The system signals utility namespace . . . 47
util::stream
The stream utility namespace . . . 47
util::string
The string utility namespace . . . 47
util::sys
The system utility namespace . . . 47
util::thread
The thread utility namespace . . . 48
util::types
The types utility namespace . . . 48
util::xmsg
The X-Message utility namespace . . . 48
vmime
VMime library namespace . . . 59

Chapter 6

Hierarchical Index

6.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

util::xmsg::AbstractXComposite< T > . . . 61
util::xmsg::AbstractXComposite< shared_ptr< argo::iridium::xfloat::XFloatMailMessage > > . . . 61
util::xmsg::XCompositeIridiumPosition . . . 581
util::xmsg::AbstractXComposite< shared_ptr< XFall > > . . . 61
util::xmsg::XCompositeFall . . . 560
util::xmsg::AbstractXComposite< shared_ptr< XProfileTemperature > > . . . 61
util::xmsg::XCompositeAuxTemp . . . 539
util::xmsg::AbstractXComposite< shared_ptr< XPump > > . . . 61
util::xmsg::XCompositePump . . . 619
util::xmsg::AbstractXComposite< shared_ptr< XRise > > . . . 61
util::xmsg::XCompositeRise . . . 623
util::xmsg::AbstractXComposite< shared_ptr< XValve > > . . . 61
util::xmsg::XCompositeValve . . . 636
util::xmsg::AbstractXComposite< XPumpCounts > . . . 61
util::xmsg::XPump . . . 1503
util::xmsg::AbstractXComposite< XValveCounts > . . . 61
util::xmsg::XValve . . . 1574
util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS > . . . 127
util::xmsg::AbstractXTimeSeriesDataComposite< XSurfaceData > . . . 127
util::xmsg::XCompositeSurfaceData . . . 627
util::date::DateUtils . . . 243
argo::iridium::xfloat::XFloatDiveData . . . 973
util::position::PositionUtils . . . 358
argo::iridium::xfloat::XFloat . . . 902
util::position::Position . . . 342
argo::iridium::IridiumPosition . . . 260
argo::iridium::xfloat::XFloatMailMessage . . . 1051
argo::iridium::xfloat::XFloatMetadata . . . 1182
util::xmsg::XCompositeIridiumPosition . . . 581
util::xmsg::XGPS . . . 1258
util::xmsg::AbstractXFallRise . . . 65
util::xmsg::XFall . . . 881
util::xmsg::XRise . . . 1525
util::xmsg::XSurfaceData . . . 1553
util::xmsg::XAlamoBit2 . . . 443
util::xmsg::XAlamoEngineering . . . 457
util::xmsg::XEngAscent . . . 706
util::xmsg::XEngDescent . . . 762
util::xmsg::XEngDrift . . . 794
util::xmsg::XEngIce . . . 820
util::xmsg::XEngSurface . . . 855
util::xmsg::XAlamoMissionConfiguration . . . 474
util::xmsg::XArgo . . . 496
util::xmsg::XEngIce::XEngIceErrorCode . . . 838
util::xmsg::XEngSurface::XEngErrorCode . . . 814
util::xmsg::XMfaMissionCfg . . . 1386

boost::exception
util::log::MessageException . . . 333
std::exception
util::log::MessageException . . . 333
util::file::FileSystemUtils . . . 246
argo::iridium::xfloat::XFloat . . . 902
argo::iridium::xfloat::XFloatDiveData . . . 973
argo::iridium::xfloat::XFloatMailMessage . . . 1051
argo::iridium::xfloat::XFloatMailRetrievalService . . . 1155
argo::iridium::xfloat::XFloatMetadata . . . 1182
util::config::LocalConfiguration . . . 290
boost::container::flat_map< C, D > . . . 251
boost::container::flat_map< int, shared_ptr< argo::iridium::xfloat::XFloatDiveData > > . . . 251
boost::container::flat_map< int, std::shared_ptr< argo::iridium::xfloat::XFloat > > . . . 251
util::kml::KmlUtil . . . 282
argo::iridium::xfloat::XFloat . . . 902
argo::iridium::xfloat::XFloatPostProcessingService . . . 1234
util::xmsg::XGPS . . . 1258
util::thread::Lockable . . . 325
util::log::Log . . . 327
util::math::MathUtils . . . 332
util::position::PositionUtils . . . 358
util::mysql::MysqlUtils . . . 335
argo::iridium::xfloat::XFloatTelemetryDecoderService . . . 1244
util::mysql::LocalMysqlConfiguration . . . 300
util::mysql::SqlTableDescription< argo::iridium::xfloat::XFloat > . . . 382
util::mysql::SqlTableDescription< argo::iridium::xfloat::XFloatDiveData > . . . 382
util::mysql::SqlTableDescription< argo::iridium::xfloat::XFloatMailMessage > . . . 382
util::mysql::SqlTableDescription< argo::iridium::xfloat::XFloatMetadata > . . . 382
util::mysql::SqlTableDescription< util::xmsg::XGPS > . . . 382
argo::iridium::xfloat::XFloatMailProcessingService . . . 1136
argo::iridium::xfloat::XFloatMailRetrievalService . . . 1155
util::mysql::LocalPersistentEntity . . . 313
argo::iridium::xfloat::XFloat . . . 902
argo::iridium::xfloat::XFloatDiveData . . . 973
argo::iridium::xfloat::XFloatMailMessage . . . 1051
argo::iridium::xfloat::XFloatMetadata . . . 1182
util::xmsg::XGPS . . . 1258
util::mysql::SqlTableDescription< T > . . . 382
util::thread::RecursiveLockable . . . 365
util::xmsg::AbstractXSensorDataComposite< XDriftPressure, XDriftSalinity, XDriftTemperature, XMSG_↩ MAX_DRFT > . . . 103
util::xmsg::XCompositeDrift . . . 543
util::xmsg::AbstractXSensorDataComposite< XHiResPressure, XHiResSalinity, XHiResTemperature, XMSG_MAX_PROF_HIRES > . . . 103
util::xmsg::XCompositeHiResProfile . . . 564
util::xmsg::AbstractXSensorDataComposite< XProfilePressure, XProfileSalinity, XProfileTemperature, XMSG_MAX_PROF > . . . 103
util::xmsg::XCompositeProfile . . . 602
argo::iridium::Iridium . . . 252
argo::iridium::xfloat::XFloat . . . 902
argo::iridium::xfloat::XFloatDiveData . . . 973
argo::iridium::xfloat::XFloatMailMessage . . . 1051
argo::iridium::xfloat::XFloatMetadata . . . 1182
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_↩ MSGS > . . . 103
boost::shared_ptr< A > . . . 368
vmime::shared_ptr< F > . . . 370
boost::shared_ptr< argo::iridium::xfloat::XFloatDiveData > . . . 368
boost::shared_ptr< argo::iridium::xfloat::XFloatMailMessage > . . . 368
boost::shared_ptr< util::xmsg::XFall > . . . 368
boost::shared_ptr< util::xmsg::XGPS > . . . 368
boost::shared_ptr< util::xmsg::XPump > . . . 368
boost::shared_ptr< util::xmsg::XRise > . . . 368
boost::shared_ptr< util::xmsg::XValve > . . . 368
vmime::shared_ptr< vmime::net::folder > . . . 370
vmime::shared_ptr< vmime::net::session > . . . 370
vmime::shared_ptr< vmime::net::store > . . . 370
util::sig::SignalHandler . . . 371
util::mysql::SqlTableColumnDescriptionBase . . . 378

util::mysql::SqlTableColumnDescription< C, bool > . . . 372
util::mysql::Column_bool< C > . . . 154
util::mysql::SqlTableColumnDescription< C, char > . . . 372
util::mysql::Column_char< C > . . . 161
util::mysql::SqlTableColumnDescription< C, double > . . . 372
util::mysql::Column_double< C > . . . 167
util::mysql::SqlTableColumnDescription< C, path > . . . 372
util::mysql::Column_filesystem_path< C > . . . 174
util::mysql::SqlTableColumnDescription< C, float > . . . 372
util::mysql::Column_float< C > . . . 181
util::mysql::SqlTableColumnDescription< C, int > . . . 372
util::mysql::Column_int< C > . . . 187
util::mysql::SqlTableColumnDescription< C, int16_t > . . . 372
util::mysql::Column_int16_t< C > . . . 194
util::mysql::SqlTableColumnDescription< C, long > . . . 372
util::mysql::Column_long< C > . . . 200
util::mysql::SqlTableColumnDescription< C, ptime > . . . 372
util::mysql::Column_posix_time< C > . . . 207
util::mysql::SqlTableColumnDescription< C, string > . . . 372
util::mysql::Column_string< C > . . . 214
util::mysql::SqlTableColumnDescription< C, uint16_t > . . . 372
util::mysql::Column_uint16_t< C > . . . 221
util::mysql::SqlTableColumnDescription< C, uint8_t > . . . 372
util::mysql::Column_uint8_t< C > . . . 227
util::mysql::SqlTableColumnDescription< C, vector< uint8_t > > . . . 372
util::mysql::Column_vector_uint8_t< C > . . . 234
util::mysql::SqlTableColumnDescription< C, T > . . . 372
boost::container::static_vector< E > . . . 407
boost::container::static_vector< PresMsgPtr_t, MAX_MSGS > . . . 407
boost::container::static_vector< SalnMsgPtr_t, MAX_MSGS > . . . 407
boost::container::static_vector< shared_ptr< argo::iridium::xfloat::XFloatMailMessage >, XMSG_MAX_MAIL > . . . 407
boost::container::static_vector< shared_ptr< util::xmsg::XFall >, XMSG_MAX_FALL > . . . 407
boost::container::static_vector< shared_ptr< util::xmsg::XGPS >, XMSG_MAX_GPS > . . . 407
boost::container::static_vector< shared_ptr< util::xmsg::XPump >, XMSG_MAX_PUMP > . . . 407
boost::container::static_vector< shared_ptr< util::xmsg::XRise >, XMSG_MAX_RISE > . . . 407
boost::container::static_vector< shared_ptr< util::xmsg::XValve >, XMSG_MAX_VALVE > . . . 407
boost::container::static_vector< TempMsgPtr_t, MAX_MSGS > . . . 407
boost::container::static_vector< vector< uint8_t >, XMSG_MAX_MFA_MISS > . . . 407
util::stream::StreamUtils . . . 408
util::mysql::SqlTableDescription< argo::iridium::xfloat::XFloat > . . . 382
util::mysql::SqlTableDescription< argo::iridium::xfloat::XFloatDiveData > . . . 382
util::mysql::SqlTableDescription< argo::iridium::xfloat::XFloatMailMessage > . . . 382
util::mysql::SqlTableDescription< argo::iridium::xfloat::XFloatMetadata > . . . 382
util::mysql::SqlTableDescription< util::xmsg::XGPS > . . . 382
util::mysql::Column_vector_uint8_t< C > . . . 234
util::mysql::SqlTableDescription< T > . . . 382
util::thread::SynchronousQueue< T > . . . 409
util::thread::SynchronousQueue< shared_ptr< argo::iridium::xfloat::XFloatMailMessage > > . . . 409
boost::thread_specific_ptr< B > . . . 412
boost::thread_specific_ptr< Connection > . . . 412
boost::thread_specific_ptr< KmlFactory > . . . 412
boost::thread_specific_ptr< MySQL_Driver > . . . 412
util::thread::ThreadUtils . . . 413
util::xmsg::XAcousticCacst . . . 418
util::xmsg::XAcousticWindow . . . 432
util::xmsg::XAlamoBit . . . 435
util::xmsg::XMessage . . . 1364
util::xmsg::XAcousticPosition . . . 421
util::xmsg::XAlamoEngineering . . . 457
util::xmsg::XArgo . . . 496
util::xmsg::XEngAbort . . . 696
util::xmsg::XEngBIT . . . 726
util::xmsg::XEngBITFail . . . 738
util::xmsg::XEngBITPass . . . 750
util::xmsg::XEngDiagnostic . . . 784
util::xmsg::XEngStandard . . . 844
util::xmsg::XGPS . . . 1258
util::xmsg::XMessageIndexable . . . 1374
util::xmsg::AbstractXFallRise . . . 65

util::xmsg::AbstractXSensorData . . . 84
util::xmsg::AuxiliarySensorData . . . 137
util::xmsg::XDriftPressure . . . 639
util::xmsg::XDriftSalinity . . . 658
util::xmsg::XDriftTemperature . . . 677
util::xmsg::XHiResPressure . . . 1307
util::xmsg::XHiResSalinity . . . 1326
util::xmsg::XHiResTemperature . . . 1345
util::xmsg::XProfilePressure . . . 1447
util::xmsg::XProfileSalinity . . . 1466
util::xmsg::XProfileTemperature . . . 1485
util::xmsg::XMissionConfiguration . . . 1425
util::xmsg::XPump . . . 1503
util::xmsg::XValve . . . 1574
util::xmsg::XWave . . . 1591
util::xmsg::XMissionCmdEcho . . . 1415
util::xmsg::XPresCounts . . . 1444
util::xmsg::XPresCodeCounts . . . 1438
util::xmsg::XSensorTripletCodeCounts . . . 1546
util::xmsg::XPumpCounts . . . 1519
util::xmsg::XValveCounts . . . 1588
util::xmsg::XWaveRawSample . . . 1607
util::xmsg::XWaveSpectralSample . . . 1609
basic_streambuf
util::stream::DataBuf< CharT, std::char_traits< CharT > > . . . 241
util::stream::DataBuf< char > . . . 241
util::stream::VectorBuf_c< CharT > . . . 417
util::stream::DataBuf< CharT, TraitsT > . . . 241
util::stream::VectorBuf< CharT, TraitsT > . . . 415
bool . . . **??**
circular_buffer< shared_ptr< argo::iridium::xfloat::XFloatMailMessage > > . . . **??**
folder * . . . **??**
int64_t . . . **??**
PresMsgPtr_t . . . **??**
SalnMsgPtr_t . . . **??**
session * . . . **??**
store * . . . **??**
TempMsgPtr_t . . . **??**

Chapter 7

Class Index

7.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

util::xmsg::AbstractXComposite< T >
An abstract class representing XFloat dive data that consists of multiple X-Messages 61
util::xmsg::AbstractXFallRise
An abstract class representing a fall, rise, or surface pressure time-series data X-message 65
util::xmsg::AbstractXSensorData
The abstract sensor data class 84
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >
The AbstractXSensorDataComposite class 103
util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >
The abstract time-series data composite class template 127
util::xmsg::AuxiliarySensorData
The auxiliary sensor data class 137
util::mysql::Column_bool< C >
Template column description class for value of type bool 154
util::mysql::Column_char< C >
Template column description class for value of type double 161
util::mysql::Column_double< C >
Template column description class for value of type double 167
util::mysql::Column_filesystem_path< C >
Template column description class for value of type path 174
util::mysql::Column_float< C >
Template column description class for value of type float 181
util::mysql::Column_int< C >
Template column description class for value of type int 187
util::mysql::Column_int16_t< C >
Template column description class for value of type int16_t 194
util::mysql::Column_long< C >
Template column description class for value of type long 200
util::mysql::Column_posix_time< C >
Template column description class for value of type boost::posix_time::ptime 207
util::mysql::Column_string< C >
Template column description class for value of type string 214
util::mysql::Column_uint16_t< C >
Template column description class for value of type uint16_t 221
util::mysql::Column_uint8_t< C >
Template column description class for value of type uint8_t 227
util::mysql::Column_vector_uint8_t< C >
Template column description class for value of type vector<uint8_t> 234
util::stream::DataBuf< CharT, TraitsT >
Template class for wrapping an arbitrary contiguous section of data as a std::basic_streambuf object 241
util::date::DateUtils
Class providing library of useful date and time functions 243
util::file::FileSystemUtils
Class providing library of useful filesystem functions 246
boost::container::flat_map< C, D >
Dummy template class used for configuring Doxygen to interpret boost::container::flat_map correctly 251
argo::iridium::Iridium
The Iridium instrument class 252
argo::iridium::IridiumPosition
The Iridium position class 260

util::kml::KmlUtil
The KML utility class . . . 282
util::config::LocalConfiguration
Class representing all program run-time configuration options . . . 290
util::mysql::LocalMysqlConfiguration
Class providing useful functions for accessing and manipulating the MySQL database, given the local run-time configuration settings . . . 300
util::mysql::LocalPersistentEntity
An abstract class representing a persistent database entity in the database defined by the config::LocalConfiguration . . . 313
util::thread::Lockable
A lockable class . . . 325
util::log::Log
The logger class . . . 327
util::math::MathUtils
Class containing commonly-used mathematical functions . . . 332
util::log::MessageException
Exception class with associated message . . . 333
util::mysql::MysqlUtils
Class provides a library of function for accessing and manipulating a MySQL database . . . 335
util::position::Position
Abstract class representing a position, date, and time . . . 342
util::position::PositionUtils
Class providing library of useful functions for manipulating and processing geographic positions . . . 358
util::thread::RecursiveLockable
A recursively lockable class . . . 365
boost::shared_ptr< A >
Dummy template class used for configuring Doxygen to interpret boost::shared_ptr objects as plain pointers . . . 368
vmime::shared_ptr< F >
Dummy template class used for configuring Doxygen to interpret vmime::shared_ptr objects as plain pointers . . . 370
util::sig::SignalHandler
Class providing function for handling POSIX style system signals . . . 371
util::mysql::SqlTableColumnDescription< C, T >
The `SqlTableColumnDescription` class is a template class used to describe an SQL table column for class `C` and data member of type `T` . . . 372
util::mysql::SqlTableColumnDescriptionBase
An abstract base class for the description of an SQL database table column . . . 378
util::mysql::SqlTableDescription< T >
Template class representing the description of an SQL database table . . . 382
boost::container::static_vector< E >
Dummy template class used for configuring Doxygen to interpret boost::container::static_vector correctly . . . 407
util::stream::StreamUtils
Class providing useful functions for manipulating I/O streams . . . 408
util::thread::SynchronousQueue< T >
Synchronous queue template . . . 409
boost::thread_specific_ptr< B >
Dummy template class used for configuring Doxygen to interpret boost::thread_specific_ptr objects as plain pointers . . . 412
util::thread::ThreadUtils
Class providing useful functions for manipulating threads and processes . . . 413
util::stream::VectorBuf< CharT, TraitsT >
Template class for wrapping a vector<T> as a std::basic_streambuf<T> . . . 415
util::stream::VectorBuf_c< CharT >
Template class for wrapping a vector<T> as a std::basic_streambuf<char> . . . 417
util::xmsg::XAcousticCacst
An acoustic reception (CACST) record . . . 418
util::xmsg::XAcousticPosition
Class representing an acoustic position X-Message . . . 421
util::xmsg::XAcousticWindow
An acoustic listening window . . . 432
util::xmsg::XAlamoBit
Class representing the BIT test of an ALAMO float . . . 435
util::xmsg::XAlamoBit2
The ALAMO Built-in-Test (BIT) message, version 2 . . . 443
util::xmsg::XAlamoEngineering
Abstract class representing a version controlled engineering message for ALAMO-type instruments . . . 457
util::xmsg::XAlamoMissionConfiguration
Class representing an ALAMO Mission Configuration message . . . 474

util::xmsg::XArgo
Class representing Argo X message . . . 496
util::xmsg::XCompositeAuxTemp
The composite auxiliary temperature class . . . 539
util::xmsg::XCompositeDrift
Class representing the aggregate drift data for a single dive, composed from one or more drift messages . . . 543
util::xmsg::XCompositeFall
Class representing the aggregate fall time-series data for a single dive, composed from one or more fall time-series messages . . . 560
util::xmsg::XCompositeHiResProfile
Class representing the aggregate raw/high-resolution/unbinned profile data for a single dive, composed from one or more raw profile data messages . . . 564
util::xmsg::XCompositeIridiumPosition
Class representing the aggregate Iridium position for a single dive, composed from one or more Iridium message positions . . . 581
util::xmsg::XCompositeProfile
Class representing the aggregate bin-averaged profile data for a single dive, composed from one or more bin-averaged profile data messages . . . 602
util::xmsg::XCompositePump
Class representing the aggregate pump time-series data for a single dive, composed from one or more pump time-series messages . . . 619
util::xmsg::XCompositeRise
Class representing the aggregate pump data for a single dive, composed from one or more pump data messages . . . 623
util::xmsg::XCompositeSurfaceData
Class representing the aggregate surface pressure data for a single dive, composed from one or more surface pressure messages . . . 627
util::xmsg::XCompositeValve
The composite valve data class . . . 636
util::xmsg::XDriftPressure
Class representing a drift pressure time-series X-Message . . . 639
util::xmsg::XDriftSalinity
Class representing a drift salinity time-series X-Message . . . 658
util::xmsg::XDriftTemperature
Class reprenting a drift temperature X-Message . . . 677
util::xmsg::XEngAbort
Class representing an abort/beacon mode engineering X-Message . . . 696
util::xmsg::XEngAscent
Class representing an ascend state engineering X-Message . . . 706
util::xmsg::XEngBIT
The ALAMO Built-in-Test (BIT) engineering message . . . 726
util::xmsg::XEngBITFail
Class representing an engineering startup message for BIT failed . . . 738
util::xmsg::XEngBITPass
Class representing an engineering startup message for BIT passed . . . 750
util::xmsg::XEngDescent
Class representing an descend state engineering X-Message . . . 762
util::xmsg::XEngDiagnostic
Class representing a diagnostic dive engineering message . . . 784
util::xmsg::XEngDrift
Class representing an drift/park state engineering X-Message . . . 794
util::xmsg::XEngSurface::XEngErrorCode
The engineering surface data error code class . . . 814
util::xmsg::XEngIce
The ALAMO engineering (ice) message class . . . 820
util::xmsg::XEngIce::XEngIceErrorCode
The engineering (ice) error code class . . . 838
util::xmsg::XEngStandard
Class representing an engineering message for a standard dive cycle . . . 844
util::xmsg::XEngSurface
Class representing an surface state engineering X-Message . . . 855
util::xmsg::XFall
Class representing a fall time-series data X-Message . . . 881
argo::iridium::xfloat::XFloat
The instrument or float class . . . 902
argo::iridium::xfloat::XFloatDiveData
The instrument or float dive data class . . . 973
argo::iridium::xfloat::XFloatMailMessage
The mail message class . . . 1051
argo::iridium::xfloat::XFloatMailProcessingService
The mail processing thread class . . . 1136

argo::iridium::xfloat::XFloatMailRetrievalService
The mail retrieval thread class . . . 1155
argo::iridium::xfloat::XFloatMetadata
The instrument metadata class . . . 1182
argo::iridium::xfloat::XFloatPostProcessingService
The post-processing analysis thread class . . . 1234
argo::iridium::xfloat::XFloatTelemetryDecoderService
The telemetry decoder thread class . . . 1244
util::xmsg::XGPS
Class representing a GPS type X-Message . . . 1258
util::xmsg::XHiResPressure
Class representing high-resolution/raw/unbinned profile pressure X-Message . . . 1307
util::xmsg::XHiResSalinity
Class representing high-resolution/raw/unbinned profile salinity X-Message . . . 1326
util::xmsg::XHiResTemperature
Class representing high-resolution/raw/unbinned profile temperature X-Message . . . 1345
util::xmsg::XMessage
Class representing a message following the X format specification . . . 1364
util::xmsg::XMessageIndexable
Class representing an indexable X-Message . . . 1374
util::xmsg::XMfaMissionCfg
The ALAMO mission configuration (mission parameters) message class . . . 1386
util::xmsg::XMissionCmdEcho
Class representing a ground station command echo-back X-Message . . . 1415
util::xmsg::XMissionConfiguration
Class representing a mission configuration X-Message . . . 1425
util::xmsg::XPresCodeCounts
Class representing a raw pressure measurement and phase code for a rise or fall data X-Message 1438
util::xmsg::XPresCounts
Class representing a raw pressure measurement for a rise or fall data X-Message . . . 1444
util::xmsg::XProfilePressure
Class representing a profile pressure data X-Message . . . 1447
util::xmsg::XProfileSalinity
Class representing a profile salinity X-Message . . . 1466
util::xmsg::XProfileTemperature
Class representing a profile temperature X-Message . . . 1485
util::xmsg::XPump
Class representing a pump data X-Message . . . 1503
util::xmsg::XPumpCounts
Class representing a single pump data measurement for a pump data time-series X-Message . . . 1519
util::xmsg::XRise
Class representing a rise data X-Message . . . 1525
util::xmsg::XSensorTripletCodeCounts
The sensor triplet and phase code class . . . 1546
util::xmsg::XSurfaceData
The surface time-series data X-Message class . . . 1553
util::xmsg::XValve
The valve data message class . . . 1574
util::xmsg::XValveCounts
The valve counts class . . . 1588
util::xmsg::XWave
Class representing measured ocean surface wave data X message . . . 1591
util::xmsg::XWaveRawSample
Class representing a single raw motion measurement in the surface time series wave data X-message . . . 1607
util::xmsg::XWaveSpectralSample
The wave spectral sample class . . . 1609

Chapter 8

File Index

8.1 File List

Here is a list of all documented files with brief descriptions:

AdditionalDocumentation.h
: Additional Doxygen documentation . . . 1611

Main.cpp
: The main() function source file . . . 2086

Main.h
: The main() function header file . . . 2090

stage1_MailRetrievalThread.cpp
: The argo::iridium::xfloat::XFloatMailRetrievalService class source file . . . 2093

stage1_MailRetrievalThread.h
: The argo::iridium::xfloat::XFloatMailRetrievalService class header file . . . 2094

stage2_MailProcessingThread.cpp
: The argo::iridium::xfloat::XFloatMailProcessingService class source file . . . 2099

stage2_MailProcessingThread.h
: The argo::iridium::xfloat::XFloatMailProcessingService class header file . . . 2101

stage3_TelemetryDecoderThread.cpp
: The argo::iridium::xfloat::XFloatTelemetryDecoderService class source file . . . 2104

stage3_TelemetryDecoderThread.h
: The argo::iridium::xfloat::XFloatTelemetryDecoderService class header file . . . 2105

stage4b_PostProcessingThread.cpp
: The argo::iridium::xfloat::XFloatPostProcessingService class source file . . . 2113

stage4b_PostProcessingThread.h
: The argo::iridium::xfloat::XFloatPostProcessingService class header file . . . 2114

ThreadDefinitions.h
: The thread definitions header file . . . 2118

config/LocalConfiguration.cpp
: The util::config::LocalConfiguration class source file . . . 1615

config/LocalConfiguration.h
: The util::config::LocalConfiguration class header file . . . 1616

config/LocalMysqlConfiguration.cpp
: The util::mysql::LocalMysqlConfiguration class source file . . . 1625

config/LocalMysqlConfiguration.h
: The util::mysql::LocalMysqlConfiguration class header file . . . 1626

config/LocalPersistentEntity.h
: The util::mysql::LocalPersistentEntity class header file . . . 1628

data/Iridium.h
: The argo::iridium::Iridium class header file . . . 1630

data/IridiumDefinitions.h
: The Iridium Definitions header file . . . 1634

data/IridiumPosition.h
: The argo::iridium::IridiumPosition class header file . . . 1637

data/XFloat.cpp
: The argo::iridium::xfloat::XFloat class source file . . . 1640

data/XFloat.h
: The argo::iridium::xfloat::XFloat class header file . . . 1641

data/XFloatDefinitions.h
: The XFloat Definitions header file . . . 1658

data/XFloatDiveData.cpp
: The argo::iridium::xfloat::XFloatDiveData class source file . . . 1663

data/XFloatDiveData.h
: The argo::iridium::xfloat::XFloatDiveData class header file . . . 1665

data/XFloatMailMessage.cpp
The argo::iridium::xfloat::XFloatMailMessage class source file . . . 1684
data/XFloatMailMessage.h
The argo::iridium::xfloat::XFloatMailMessage class header file . . . 1685
data/XFloatMetadata.cpp
The argo::iridium::xfloat::XFloatMetadata class source file . . . 1706
data/XFloatMetadata.h
The argo::iridium::xfloat::XFloatMetadata class header file . . . 1707
data/xmsg/AbstractXComposite.h
The util::xmsg::AbstractXComposite class source file . . . 1717
data/xmsg/AbstractXFallRise.h
The util::xmsg::AbstractXFallRise class header file . . . 1719
data/xmsg/AbstractXSensorData.h
The util::xmsg::AbstractXSensorData class header file . . . 1724
data/xmsg/AbstractXSensorDataComposite.h
The util::xmsg::AbstractXSensorDataComposite class header file . . . 1733
data/xmsg/AbstractXTimeSeriesDataComposite.h
The util::xmsg::AbstractXTimeSeriesDataComposite class header file . . . 1745
data/xmsg/AuxiliarySensorData.cpp
The util::xmsg::AuxiliarySensorData class source file . . . 1757
data/xmsg/AuxiliarySensorData.h
The util::xmsg::AuxiliarySensorData class header file . . . 1759
data/xmsg/XAcousticPosition.h
The util::xmsg::XAcousticPosition class header file . . . 1808
data/xmsg/XAlamoBit.h
The util::xmsg::XAlamoBit class header file . . . 1816
data/xmsg/XAlamoBit2.h . . . 1828
data/xmsg/XAlamoEngineering.h
The util::xmsg::XAlamoEngineering class header file . . . 1841
data/xmsg/XAlamoMissionConfiguration.h
The util::xmsg::XAlamoMissionConfiguration class header file . . . 1844
data/xmsg/XArgo.h
The util::xmsg::XArgo class header file . . . 1901
data/xmsg/XCompositeAuxTemp.h
The util::xmsg::XCompositeAuxTemp class header file . . . 1915
data/xmsg/XCompositeDrift.h
The util::xmsg::XCompositeDrift class header file . . . 1918
data/xmsg/XCompositeFall.h
The util::xmsg::XCompositeFall class header file . . . 1921
data/xmsg/XCompositeHiResProfile.h
The util::xmsg::XCompositeHiResProfile class header file . . . 1923
data/xmsg/XCompositeIridiumPosition.h
The util::xmsg::XCompositeIridiumPosition class header file . . . 1926
data/xmsg/XCompositeProfile.h
The util::xmsg::XCompositeProfile class header file . . . 1931
data/xmsg/XCompositePump.h
The util::xmsg::XCompositePump class header file . . . 1934
data/xmsg/XCompositeRise.h
The util::xmsg::XCompositeRise class header file . . . 1937
data/xmsg/XCompositeSurfaceData.cpp
The util::xmsg::XCompositeSurfaceData class source file . . . 1939
data/xmsg/XCompositeSurfaceData.h
The util::xmsg::XCompositeSurfaceData class header file . . . 1940
data/xmsg/XCompositeValve.cpp
The util::xmsg::XCompositeValve class source file . . . 1946
data/xmsg/XCompositeValve.h
The util::xmsg::XCompositeValve class header file . . . 1947
data/xmsg/XDriftPressure.h
The util::xmsg::XDriftPressure class source file . . . 1949
data/xmsg/XDriftSalinity.h
The util::xmsg::XDriftSalinity class source file . . . 1951
data/xmsg/XDriftTemperature.h
The util::xmsg::XDriftTemperature class source file . . . 1953
data/xmsg/XEngAbort.h
The util::xmsg::XEngAbort class header file . . . 1955
data/xmsg/XEngAscent.h
The util::xmsg::XEngAscent class header file . . . 1958
data/xmsg/XEngBIT.h . . . 1963
data/xmsg/XEngBITFail.h
The util::xmsg::XEngBITFail class header file . . . 1964
data/xmsg/XEngBITPass.h
The util::xmsg::XEngBITPass class header file . . . 1967

data/xmsg/XEngDescent.h
The util::xmsg::XEngDescent class header file . . . 1969
data/xmsg/XEngDiagnostic.h
The util::xmsg::XEngDiagnostic class source file . . . 1974
data/xmsg/XEngDrift.h
The util::xmsg::XEngDrift class header file . . . 1977
data/xmsg/XEngIce.cpp
The util::xmsg::XEngIce class source file . . . 1983
data/xmsg/XEngIce.h
The util::xmsg::XEngIce class header file . . . 1984
data/xmsg/XEngStandard.h
The util::xmsg::XEngStandard class source file . . . 1991
data/xmsg/XEngSurface.h
The util::xmsg::XEngSurface class header file . . . 1994
data/xmsg/XFall.h
The util::xmsg::XFall class header file . . . 2012
data/xmsg/XGPS.cpp
The util::xmsg::XGPS class source file . . . 2015
data/xmsg/XGPS.h
The util::xmsg::XGPS class header file . . . 2017
data/xmsg/XHiResPressure.h
The util::xmsg::XHiResPressure class header file . . . 2023
data/xmsg/XHiResSalinity.h
The util::xmsg::XHiResSalinity class header file . . . 2025
data/xmsg/XHiResTemperature.h
The util::xmsg::XHiResTemperature class header file . . . 2027
data/xmsg/XMessage.h
The util::xmsg::XMessage class header file . . . 2029
data/xmsg/XMessageIndexable.h
The util::xmsg::XMessageIndexable class source file . . . 2038
data/xmsg/XMfaMissionCfg.cpp
The util::xmsg::XMfaMissionCfg class source file . . . 2040
data/xmsg/XMfaMissionCfg.h
The util::xmsg::XMfaMissionCfg class header file . . . 2042
data/xmsg/XMissionCmdEcho.h
The util::xmsg::XMissionCmdEcho class source file . . . 2054
data/xmsg/XMissionConfiguration.h
The util::xmsg::XMissionConfiguration class header file . . . 2057
data/xmsg/XProfilePressure.h
The util::xmsg::XProfilePressure class header file . . . 2059
data/xmsg/XProfileSalinity.h
The util::xmsg::XProfileSalinity class header file . . . 2061
data/xmsg/XProfileTemperature.h
The util::xmsg::XProfileTemperature class header file . . . 2063
data/xmsg/XPump.h
The util::xmsg::XPump class header file . . . 2065
data/xmsg/XRise.h
The util::xmsg::XRise class header file . . . 2070
data/xmsg/XSurfaceData.h
The util::xmsg::XSurfaceData class header file . . . 2072
data/xmsg/XValve.cpp
The util::xmsg::XValve class source file . . . 2075
data/xmsg/XValve.h
The util::xmsg::XValve class header file . . . 2077
data/xmsg/XWave.cpp
The util::xmsg::XWave class source file . . . 2080
data/xmsg/XWave.h
The util::xmsg::XWave class header file . . . 2081
data/xmsg/common/XCommon.h
The XCommon source file . . . 1765
data/xmsg/common/XDefinitions.h
The XDefinitions header file . . . 1766
data/xmsg/common/XPresCodeCounts.cpp
The util::xmsg::XPresCodeCounts class source file . . . 1783
data/xmsg/common/XPresCodeCounts.h
The util::xmsg::XPresCodeCounts class header file . . . 1784
data/xmsg/common/XPresCounts.h
The util::xmsg::XPresCounts class header file . . . 1788
data/xmsg/common/XPumpCounts.h
The util::xmsg::XPumpCounts class source file . . . 1790
data/xmsg/common/XSensorTripletCodeCounts.h
The util::xmsg::XSensorTripletCodeCounts class header file . . . 1794

data/xmsg/common/XValveCounts.cpp
The util::xmsg::XValveCounts class source file . . . 1797
data/xmsg/common/XValveCounts.h
The util::xmsg::XValveCounts class header file . . . 1798
data/xmsg/common/XWaveRawSample.h
The util::xmsg::XWaveRawSample class header file . . . 1801
data/xmsg/common/XWaveSpectralSample.cpp
The util::xmsg::XWaveSpectralSample class source file . . . 1803
data/xmsg/common/XWaveSpectralSample.h
The util::xmsg::XWaveSpectralSample class header file . . . 1805
utility/ContainerUtils.h
The ContainerUtils class header file . . . 2120
utility/DataBuf.h
The util::stream::DataBuf class header file . . . 2122
utility/DateUtils.h
The util::date::DateUtils class header file . . . 2124
utility/FileSystemUtils.h
The util::file::FileSystemUtils class header file . . . 2126
utility/KmlUtil.cpp
The util::kml::KmlUtil class source file . . . 2129
utility/KmlUtil.h
The util::kml::KmlUtil class header file . . . 2130
utility/Lockable.h
The util::thread::Lockable class header file . . . 2135
utility/Log.cpp
The util::log::Log class source file . . . 2137
utility/Log.h
The util::log::Log class header file . . . 2138
utility/MathUtils.h
The util::math::MathUtils class header file . . . 2153
utility/MessageException.h
The util::log::MessageException class header file . . . 2155
utility/MysqlUtils.cpp
The util::mysql::MysqlUtils class source file . . . 2157
utility/MysqlUtils.h
The util::mysql::MysqlUtils class header file . . . 2158
utility/Position.h
The util::position::Position class header file . . . 2163
utility/PositionUtils.cpp
The util::position::PositionUtils class source file . . . 2166
utility/PositionUtils.h
The util::position::PositionUtils class header file . . . 2167
utility/RecursiveLockable.h
The util::thread::RecursiveLockable class header file . . . 2170
utility/SignalHandler.h
The The util::sig::SignalHandler class header file . . . 2173
utility/SqlTableColumnDescription.h
The util::mysql::SqlTableColumnDescription class header file . . . 2175
utility/SqlTableDescription.cpp
The util::mysql::SqlTableDescription class source file . . . 2189
utility/SqlTableDescription.h
The util::mysql::SqlTableDescription class header file . . . 2190
utility/StreamUtils.h
The util::stream::StreamUtils class source file . . . 2199
utility/StringUtils.h
The StringUtils class header file . . . 2201
utility/SynchronousQueue.h
The The util::thread::SynchronousQueue class header file . . . 2202
utility/SystemUtils.h
The SystemUtils class header file . . . 2205
utility/ThreadUtils.h
The util::thread::ThreadUtils class header file . . . 2207
utility/VectorBuf.h
The util::stream::VectorBuf template class header file . . . 2210

Chapter 9

Namespace Documentation

9.1 argo Namespace Reference

The Argo instrument namespace.

Namespaces

- namespace iridium
  The Argo Iridium instrument namespace.

9.1.1 Detailed Description

The Argo instrument namespace.

9.2 argo::iridium Namespace Reference

The Argo Iridium instrument namespace.

Namespaces

- namespace xfloat
  The X-Float instrument namespace.

Classes

- class Iridium
  The Iridium instrument class.
- class IridiumPosition
  The Iridium position class.

9.2.1 Detailed Description

The Argo Iridium instrument namespace.

9.3 argo::iridium::xfloat Namespace Reference

The X-Float instrument namespace.

Classes

- class XFloat

  The instrument or float class.
- class XFloatDiveData

  The instrument or float dive data class.
- class XFloatMailMessage

  The mail message class.
- class XFloatMailProcessingService

  The mail processing thread class.
- class XFloatMailRetrievalService

  The mail retrieval thread class.
- class XFloatMetadata

  The instrument metadata class.
- class XFloatPostProcessingService

  The post-processing analysis thread class.
- class XFloatTelemetryDecoderService

  The telemetry decoder thread class.

Typedefs

- typedef uint8_t XMailSbdByte_t

  The SBD byte type.
- typedef vector< XMailSbdByte_t > XMailSbdData_t

  The SBD data type.

Enumerations

- enum **XFloatMailFetchMode_t** {
  MAIL_FETCH_SERVER = 0 ,
  MAIL_FETCH_FILESYSTEM = 1 }
- enum XFloatMailRawFileFormat_t {
  XMAIL_RAW_FORMAT_STANDARD = 0 ,
  XMAIL_RAW_FORMAT_EXTENDED = 1 }

  The mail message raw file format type identifier enumeration.

Functions

- ostream & operator<< (ostream &os, const XFloatMailMessage &mail)

  The stream insertion operator overload function for XFloatMailMessage.
- ostream & operator<< (ostream &os, const XMailSbdData_t &data)

  The stream insertion operator for the SBD data type.
- istream & operator>> (istream &in, XFloatMetadata &metadata)

  Stream extraction operator overload function for the XFloatMetadata class.
- istream & operator>> (istream &is, XFloatMailMessage &mail)

  The stream extraction operator overload function for XFloatMailMessage.

9.3.1 Detailed Description

The X-Float instrument namespace.

Note

This namespace contains all object that represents individual X-Floats (including ALAMOs and Solo-IIs), their dives and mail messages, and all of the constituents that make up these objects.

9.3.2 Typedef Documentation

9.3.2.1 XMailSbdByte_t

```
typedef uint8_t argo::iridium::xfloat::XMailSbdByte_t
```

The SBD byte type.

This type definition determines the type that is used to represent a single byte the Iridium Short Burst Data (SBD) message. Generally, this field should be a single signed or unsigned byte type.

9.3.2.2 XMailSbdData_t

```
typedef vector<XMailSbdByte_t> argo::iridium::xfloat::XMailSbdData_t
```

The SBD data type.

This type definition determines the type that is used to represent the Iridium Short Burst Data (SBD) message. It is defined to be a vector of bytes of type XMailSbdByte_t.

9.3.3 Enumeration Type Documentation

9.3.3.1 XFloatMailRawFileFormat_t

```
enum argo::iridium::xfloat::XFloatMailRawFileFormat_t
```

The mail message raw file format type identifier enumeration.

This enumeration lists the supported formats for the raw mail message file. The format of RAW mail message files that are written to the local files system will be specified by one of the values of this enumeration.

Enumerator

XMAIL_RAW_FORMAT_STANDARD	The standard or legacy format for raw mail message files. This format represents the standard raw mail message file format in which the attachment data is encoded in base-64. This format is currently used by the Solo-II decoding software.
XMAIL_RAW_FORMAT_EXTENDED	The extended format for raw mail message files. This format is an extended version of the raw file format in which the attachment data is encoded in hex, rather than base-64.

9.3.4 Function Documentation

9.3.4.1 operator<<() [1/2]

```
ostream & argo::iridium::xfloat::operator<< (
            ostream & os,
            const XFloatMailMessage & mail)
```

The stream insertion operator overload function for XFloatMailMessage.

Generates and inserts a RAW mail string representation of the mail message.

Exceptions

util::log::MessageException	of an error occurs.

Parameters

in	*os*	The ostream object.
in	*mail*	The mail message object.

Returns

The ostream object, os.

9.3.4.2 operator<<() [2/2]

```
ostream & argo::iridium::xfloat::operator<< (
            ostream & os,
            const XMailSbdData_t & data)
```

The stream insertion operator for the SBD data type.

Exceptions

util::log::MessageException	of an error occurs.

Parameters

in	*os*	The ostream object.
in	*data*	The SBD data.

Returns

The ostream object, os.

References CATCH_BLOCK.

9.3.4.3 operator>>() [1/2]

```
istream & argo::iridium::xfloat::operator>> (
            istream & in,
            XFloatMetadata & metadata)
```

Stream extraction operator overload function for the XFloatMetadata class.

Parameters

in	*in*	The input stream.
in	*metadata*	The metadata object.

Exceptions

util::log::MessageException	if the stream extraction fails.

Returns

The reference to the input stream, istream.

Todo Implement the functionality for parsing additional metadata field from stream

9.3.4.4 operator>>() [2/2]

```
istream & argo::iridium::xfloat::operator>> (
            istream & is,
            XFloatMailMessage & mail)
```

The stream extraction operator overload function for XFloatMailMessage.

Parses mail message from istream.

Exceptions

util::log::MessageException	of an error occurs.

Parameters

in	*is*	The istream object.
in	*mail*	The mail message object.

Returns

The istream object, is.

9.4 boost Namespace Reference

The Boost C++ library namespace.

Namespaces

- namespace container

  Boost container library namespace.

Classes

- class shared_ptr

  Dummy template class used for configuring Doxygen to interpret boost::shared_ptr objects as plain pointers.
- class thread_specific_ptr

  Dummy template class used for configuring Doxygen to interpret boost::thread_specific_ptr objects as plain pointers.

9.4.1 Detailed Description

The Boost C++ library namespace.

9.5 boost::container Namespace Reference

Boost container library namespace.

Classes

- class flat_map

  Dummy template class used for configuring Doxygen to interpret boost::container::flat_map correctly.
- class static_vector

  Dummy template class used for configuring Doxygen to interpret boost::container::static_vector correctly.

9.5.1 Detailed Description

Boost container library namespace.

9.6 util Namespace Reference

The utility namespace.

Namespaces

- namespace config
  The configuration utility namespace.
- namespace date
  Date utility namespace.
- namespace file
  The filesystem utility namespace.
- namespace ftp
  The ftp utility namespace.
- namespace log
  The logging utility namespace.
- namespace mail
  The mail utility namespace.
- namespace math
  The math utility namespace.
- namespace mysql
  The MySQL utility namespace.
- namespace position
  The position utility namespace.
- namespace sig
  The system signals utility namespace.
- namespace stream
  The stream utility namespace.
- namespace string
  The string utility namespace.
- namespace sys
  The system utility namespace.
- namespace thread
  The thread utility namespace.
- namespace types
  The types utility namespace.
- namespace xmsg
  The X-Message utility namespace.

9.6.1 Detailed Description

The utility namespace.

9.7 util::config Namespace Reference

The configuration utility namespace.

Classes

- class LocalConfiguration
  Class representing all program run-time configuration options.

9.7.1 Detailed Description

The configuration utility namespace.

9.8 util::date Namespace Reference

Date utility namespace.

Classes

- class DateUtils
  Class providing library of useful date and time functions.

9.8.1 Detailed Description

Date utility namespace.

9.9 util::file Namespace Reference

The filesystem utility namespace.

Classes

- class FileSystemUtils
  Class providing library of useful filesystem functions.

9.9.1 Detailed Description

The filesystem utility namespace.

9.10 util::ftp Namespace Reference

The ftp utility namespace.

9.10.1 Detailed Description

The ftp utility namespace.

Deprecated FTP client integration features have been scrapped. This namespace is no longer utilized.

9.11 util::log Namespace Reference

The logging utility namespace.

Classes

- class Log
  The logger class.
- class MessageException
  Exception class with associated message.

Typedefs

- typedef blg::sources::severity_logger_mt< blg::trivial::severity_level > sev_logger_mt
  A thread-safe severity logger object type.

Enumerations

- enum LogLevel_t {
  LOG_FAT = 0 ,
  LOG_ERR = 1 ,
  LOG_WRN = 2 ,
  LOG_LOG = 3 ,
  LOG_INF = 4 ,
  LOG_DBG = 5 ,
  LOG_DBG1 = 6 ,
  LOG_DBG2 = 7 ,
  LOG_DBG3 = 8 }

  Logging levels enumeration.

9.11.1 Detailed Description

The logging utility namespace.

9.11.2 Typedef Documentation

9.11.2.1 sev_logger_mt

```
typedef blg::sources::severity_logger_mt<blg::trivial::severity_level> util::log::sev_logger_mt
```

A thread-safe severity logger object type.

Note

This logger type uses the severity levels defined by blg::trivial::severity_level in the boost log library.

9.11.3 Enumeration Type Documentation

9.11.3.1 LogLevel_t

```
enum util::log::LogLevel_t
```

Logging levels enumeration.

Enumerator

LOG_FAT	Logging level 0: Fatal errors only.
LOG_ERR	Logging level 1: Errors only.
LOG_WRN	Logging level 2: Warnings, i.e. quiet.
LOG_LOG	Logging level 3: Standard/Default output logging level.
LOG_INF	Logging level 4: Informational messages, i.e. verbose.
LOG_DBG	Logging level 5: Debugging messages.
LOG_DBG1	Logging level 6: More debugging messages.
LOG_DBG2	Logging level 7: Even more debugging messages.
LOG_DBG3	Logging level 8: Most debugging messages.

9.12 util::mail Namespace Reference

The mail utility namespace.

9.12.1 Detailed Description

The mail utility namespace.

9.13 util::math Namespace Reference

The math utility namespace.

Classes

- class MathUtils

  Class containing commonly-used mathematical functions.

9.13.1 Detailed Description

The math utility namespace.

9.14 util::mysql Namespace Reference

The MySQL utility namespace.

Classes

- class Column_bool

  Template column description class for value of type bool.
- class Column_char

  Template column description class for value of type double.
- class Column_double

  Template column description class for value of type double.
- class Column_filesystem_path

  Template column description class for value of type path.
- class Column_float

  Template column description class for value of type float.
- class Column_int

  Template column description class for value of type int.
- class Column_int16_t

  Template column description class for value of type int16_t.
- class Column_long

  Template column description class for value of type long.
- class Column_posix_time

  Template column description class for value of type boost::posix_time::ptime.
- class Column_string

  Template column description class for value of type string.
- class Column_uint16_t

  Template column description class for value of type uint16_t.
- class Column_uint8_t

  Template column description class for value of type uint8_t.
- class Column_vector_uint8_t

  Template column description class for value of type vector<uint8_t>.
- class LocalMysqlConfiguration

  Class providing useful functions for accessing and manipulating the MySQL database, given the local run-time configuration settings.
- class LocalPersistentEntity

  An abstract class representing a persistent database entity in the database defined by the config::LocalConfiguration.
- class MysqlUtils

  Class provides a library of function for accessing and manipulating a MySQL database.
- class SqlTableColumnDescription

  The `SqlTableColumnDescription` class is a template class used to describe an SQL table column for class `C` and data member of type `T`.
- class SqlTableColumnDescriptionBase

  An abstract base class for the description of an SQL database table column.
- class SqlTableDescription

  Template class representing the description of an SQL database table.

Enumerations

- enum ColumDataType_t {
  COLUMN_DATA_TYPE_UNKOWN ,
  COLUMN_DATA_TYPE_BOOL ,
  COLUMN_DATA_TYPE_CHAR ,
  COLUMN_DATA_TYPE_DOUBLE ,
  COLUMN_DATA_TYPE_FILESYSTEM_PATH ,
  COLUMN_DATA_TYPE_FLOAT ,
  COLUMN_DATA_TYPE_INT ,
  COLUMN_DATA_TYPE_INT16_T ,
  COLUMN_DATA_TYPE_LONG ,
  COLUMN_DATA_TYPE_POSIX_TIME ,
  COLUMN_DATA_TYPE_STRING ,
  COLUMN_DATA_TYPE_TEST ,
  COLUMN_DATA_TYPE_UINT16_T ,
  COLUMN_DATA_TYPE_UINT8_T ,
  COLUMN_DATA_TYPE_VECTOR_UINT8_T }

  The ColumnDescription class is used to describe the attributes of a particular SQL table column including its name, type, and whether or not the column is part of the primary key for the SQL table.

Variables

- thread_specific_ptr< Connection > MysqlUtils::sql_connection_ & **CleanupSqlConnection**
- thread_specific_ptr< MySQL_Driver > MysqlUtils::sql_driver_ & **CleanupSqlDriver**

9.14.1 Detailed Description

The MySQL utility namespace.

9.14.2 Enumeration Type Documentation

9.14.2.1 ColumDataType_t

`enum util::mysql::ColumDataType_t`

The ColumnDescription class is used to describe the attributes of a particular SQL table column including its name, type, and whether or not the column is part of the primary key for the SQL table.

A collection of ColumnDescrition object may be used to fully describe the attributes of one or more SQL database tables.

9.15 util::position Namespace Reference

The position utility namespace.

Classes

- class Position

  Abstract class representing a position, date, and time.
- class PositionUtils

  Class providing library of useful functions for manipulating and processing geographic positions.

9.15.1 Detailed Description

The position utility namespace.

9.16 util::sig Namespace Reference

The system signals utility namespace.

Classes

- class SignalHandler

  Class providing function for handling POSIX style system signals.

9.16.1 Detailed Description

The system signals utility namespace.

9.17 util::stream Namespace Reference

The stream utility namespace.

Classes

- class DataBuf

  Template class for wrapping an arbitrary contiguous section of data as a std::basic_streambuf object.
- class StreamUtils

  Class providing useful functions for manipulating I/O streams.
- class VectorBuf

  Template class for wrapping a vector<T> as a std::basic_streambuf<T>.
- class VectorBuf_c

  Template class for wrapping a vector<T> as a std::basic_streambuf<char>.

9.17.1 Detailed Description

The stream utility namespace.

9.18 util::string Namespace Reference

The string utility namespace.

9.18.1 Detailed Description

The string utility namespace.

9.19 util::sys Namespace Reference

The system utility namespace.

9.19.1 Detailed Description

The system utility namespace.

9.20 util::thread Namespace Reference

The thread utility namespace.

Classes

- class Lockable

  A lockable class.
- class RecursiveLockable

  A recursively lockable class.
- class SynchronousQueue

  Synchronous queue template.
- class ThreadUtils

  Class providing useful functions for manipulating threads and processes.

9.20.1 Detailed Description

The thread utility namespace.

9.21 util::types Namespace Reference

The types utility namespace.

9.21.1 Detailed Description

The types utility namespace.

9.22 util::xmsg Namespace Reference

The X-Message utility namespace.

Classes

- class AbstractXComposite

  An abstract class representing XFloat dive data that consists of multiple X-Messages.
- class AbstractXFallRise

  An abstract class representing a fall, rise, or surface pressure time-series data X-message.
- class AbstractXSensorData

  The abstract sensor data class.
- class AbstractXSensorDataComposite

  The AbstractXSensorDataComposite class.
- class AbstractXTimeSeriesDataComposite

  The abstract time-series data composite class template.
- class AuxiliarySensorData

  The auxiliary sensor data class.
- class XAcousticCacst

  An acoustic reception (CACST) record.
- class XAcousticPosition

  Class representing an acoustic position X-Message.
- class XAcousticWindow

  An acoustic listening window.
- class XAlamoBit

  Class representing the BIT test of an ALAMO float.
- class XAlamoBit2

  The ALAMO Built-in-Test (BIT) message, version 2.

- class XAlamoEngineering

  Abstract class representing a version controlled engineering message for ALAMO-type instruments.
- class XAlamoMissionConfiguration

  Class representing an ALAMO Mission Configuration message.
- class XArgo

  Class representing Argo X message.
- class XCompositeAuxTemp

  The composite auxiliary temperature class.
- class XCompositeDrift

  Class representing the aggregate drift data for a single dive, composed from one or more drift messages.
- class XCompositeFall

  Class representing the aggregate fall time-series data for a single dive, composed from one or more fall time-series messages.
- class XCompositeHiResProfile

  Class representing the aggregate raw/high-resolution/unbinned profile data for a single dive, composed from one or more raw profile data messages.
- class XCompositeIridiumPosition

  Class representing the aggregate Iridium position for a single dive, composed from one or more Iridium message positions.
- class XCompositeProfile

  Class representing the aggregate bin-averaged profile data for a single dive, composed from one or more bin-averaged profile data messages.
- class XCompositePump

  Class representing the aggregate pump time-series data for a single dive, composed from one or more pump time-series messages.
- class XCompositeRise

  Class representing the aggregate pump data for a single dive, composed from one or more pump data messages.
- class XCompositeSurfaceData

  Class representing the aggregate surface pressure data for a single dive, composed from one or more surface pressure messages.
- class XCompositeValve

  The composite valve data class.
- class XDriftPressure

  Class representing a drift pressure time-series X-Message.
- class XDriftSalinity

  Class representing a drift salinity time-series X-Message.
- class XDriftTemperature

  Class reprenting a drift temperature X-Message.
- class XEngAbort

  Class representing an abort/beacon mode engineering X-Message.
- class XEngAscent

  Class representing an ascend state engineering X-Message.
- class XEngBIT

  The ALAMO Built-in-Test (BIT) engineering message.
- class XEngBITFail

  Class representing an engineering startup message for BIT failed.
- class XEngBITPass

  Class representing an engineering startup message for BIT passed.
- class XEngDescent

  Class representing an descend state engineering X-Message.
- class XEngDiagnostic

  Class representing a diagnostic dive engineering message.
- class XEngDrift

  Class representing an drift/park state engineering X-Message.
- class XEngIce

  The ALAMO engineering (ice) message class.
- class XEngStandard

  Class representing an engineering message for a standard dive cycle.
- class XEngSurface

  Class representing an surface state engineering X-Message.
- class XFall

  Class representing a fall time-series data X-Message.
- class XGPS

  Class representing a GPS type X-Message.
- class XHiResPressure

Class representing high-resolution/raw/unbinned profile pressure X-Message.

- class XHiResSalinity

  Class representing high-resolution/raw/unbinned profile salinity X-Message.
- class XHiResTemperature

  Class representing high-resolution/raw/unbinned profile temperature X-Message.
- class XMessage

  Class representing a message following the X format specification.
- class XMessageIndexable

  Class representing an indexable X-Message.
- class XMfaMissionCfg

  The ALAMO mission configuration (mission parameters) message class.
- class XMissionCmdEcho

  Class representing a ground station command echo-back X-Message.
- class XMissionConfiguration

  Class representing a mission configuration X-Message.
- class XPresCodeCounts

  Class representing a raw pressure measurement and phase code for a rise or fall data X-Message.
- class XPresCounts

  Class representing a raw pressure measurement for a rise or fall data X-Message.
- class XProfilePressure

  Class representing a profile pressure data X-Message.
- class XProfileSalinity

  Class representing a profile salinity X-Message.
- class XProfileTemperature

  Class representing a profile temperature X-Message.
- class XPump

  Class representing a pump data X-Message.
- class XPumpCounts

  Class representing a single pump data measurement for a pump data time-series X-Message.
- class XRise

  Class representing a rise data X-Message.
- class XSensorTripletCodeCounts

  The sensor triplet and phase code class.
- class XSurfaceData

  The surface time-series data X-Message class.
- class XValve

  The valve data message class.
- class XValveCounts

  The valve counts class.
- class XWave

  Class representing measured ocean surface wave data X message.
- class XWaveRawSample

  Class representing a single raw motion measurement in the surface time series wave data X-message.
- class XWaveSpectralSample

  The wave spectral sample class.

Enumerations

- enum **XAlamoBIT2Type_t** {
  UNKNOWN = -1 ,
  NO_SKY_VIEW = 0 ,
  SKY_VIEW = 1 }
- enum XFallRiseDataPackFormat_t {
  XRISEFALL_FMT_LEGACY = 0 ,
  XRISEFALL_FMT_STNDRD = 1 ,
  XRISEFALL_FMT_STNDRD_WITH_TEMP_AND_SALN = 2 ,
  XRISEFALL_FMT_UNKNOWN = 0xFF }

  Packing format type enumeration for rise and fall data X-Messages.

- enum XMsgTypeId_t {
  XMSG_TYPE_ID_GPS_DIAGNOSTIC = 0x00 ,
  XMSG_TYPE_ID_GPS_STARTOFDIVE = 0x01 ,
  XMSG_TYPE_ID_GPS_ENDOFDIVE = 0x02 ,
  XMSG_TYPE_ID_GPS_ABORT = 0x03 ,
  XMSG_TYPE_ID_GPS_BITEST = 0x05 ,
  XMSG_TYPE_ID_PRES0 = 0x10 ,
  XMSG_TYPE_ID_PRES1 = 0x11 ,
  XMSG_TYPE_ID_PRES2 = 0x12 ,
  XMSG_TYPE_ID_PRES3 = 0x13 ,
  XMSG_TYPE_ID_PRES4 = 0x14 ,
  XMSG_TYPE_ID_PRES5 = 0x15 ,
  XMSG_TYPE_ID_PRES6 = 0x16 ,
  XMSG_TYPE_ID_PRES7 = 0x17 ,
  XMSG_TYPE_ID_PRES8 = 0x18 ,
  XMSG_TYPE_ID_PRES9 = 0x19 ,
  XMSG_TYPE_ID_PRESA = 0x1A ,
  XMSG_TYPE_ID_PRESB = 0x1B ,
  XMSG_TYPE_ID_PRESC = 0x1C ,
  XMSG_TYPE_ID_PRESD = 0x1D ,
  XMSG_TYPE_ID_PRESE = 0x1E ,
  XMSG_TYPE_ID_PRESF = 0x1F ,
  XMSG_TYPE_ID_TEMP0 = 0x20 ,
  XMSG_TYPE_ID_TEMP1 = 0x21 ,
  XMSG_TYPE_ID_TEMP2 = 0x22 ,
  XMSG_TYPE_ID_TEMP3 = 0x23 ,
  XMSG_TYPE_ID_TEMP4 = 0x24 ,
  XMSG_TYPE_ID_TEMP5 = 0x25 ,
  XMSG_TYPE_ID_TEMP6 = 0x26 ,
  XMSG_TYPE_ID_TEMP7 = 0x27 ,
  XMSG_TYPE_ID_TEMP8 = 0x28 ,
  XMSG_TYPE_ID_TEMP9 = 0x29 ,
  XMSG_TYPE_ID_TEMPA = 0x2A ,
  XMSG_TYPE_ID_TEMPB = 0x2B ,
  XMSG_TYPE_ID_TEMPC = 0x2C ,
  XMSG_TYPE_ID_TEMPD = 0x2D ,
  XMSG_TYPE_ID_TEMPE = 0x2E ,
  XMSG_TYPE_ID_TEMPF = 0x2F ,
  XMSG_TYPE_ID_SALN0 = 0x30 ,
  XMSG_TYPE_ID_SALN1 = 0x31 ,
  XMSG_TYPE_ID_SALN2 = 0x32 ,
  XMSG_TYPE_ID_SALN3 = 0x33 ,
  XMSG_TYPE_ID_SALN4 = 0x34 ,
  XMSG_TYPE_ID_SALN5 = 0x35 ,
  XMSG_TYPE_ID_SALN6 = 0x36 ,
  XMSG_TYPE_ID_SALN7 = 0x37 ,
  XMSG_TYPE_ID_SALN8 = 0x38 ,
  XMSG_TYPE_ID_SALN9 = 0x39 ,
  XMSG_TYPE_ID_SALNA = 0x3A ,
  XMSG_TYPE_ID_SALNB = 0x3B ,
  XMSG_TYPE_ID_SALNC = 0x3C ,
  XMSG_TYPE_ID_SALND = 0x3D ,
  XMSG_TYPE_ID_SALNE = 0x3E ,
  XMSG_TYPE_ID_SALNF = 0x3F ,
  XMSG_TYPE_ID_FALL0 = 0x40 ,
  XMSG_TYPE_ID_FALL1 = 0x41 ,
  XMSG_TYPE_ID_FALL2 = 0x42 ,
  XMSG_TYPE_ID_FALL3 = 0x43 ,
  XMSG_TYPE_ID_FALL4 = 0x44 ,
  XMSG_TYPE_ID_FALL5 = 0x45 ,
  XMSG_TYPE_ID_FALL6 = 0x46 ,
  XMSG_TYPE_ID_FALL7 = 0x47 ,
  XMSG_TYPE_ID_FALL8 = 0x48 ,
  XMSG_TYPE_ID_FALL9 = 0x49 ,
  XMSG_TYPE_ID_FALLA = 0x4A ,
  XMSG_TYPE_ID_FALLB = 0x4B ,
  XMSG_TYPE_ID_FALLC = 0x4C ,
  XMSG_TYPE_ID_FALLD = 0x4D ,
  XMSG_TYPE_ID_FALLE = 0x4E ,
  XMSG_TYPE_ID_FALLF = 0x4F ,
  XMSG_TYPE_ID_RISE0 = 0x50 ,
  XMSG_TYPE_ID_RISE1 = 0x51 ,

XMSG_TYPE_ID_RISE2 = 0x52 ,
XMSG_TYPE_ID_RISE3 = 0x53 ,
XMSG_TYPE_ID_RISE4 = 0x54 ,
XMSG_TYPE_ID_RISE5 = 0x55 ,
XMSG_TYPE_ID_RISE6 = 0x56 ,
XMSG_TYPE_ID_RISE7 = 0x57 ,
XMSG_TYPE_ID_RISE8 = 0x58 ,
XMSG_TYPE_ID_RISE9 = 0x59 ,
XMSG_TYPE_ID_RISEA = 0x5A ,
XMSG_TYPE_ID_RISEB = 0x5B ,
XMSG_TYPE_ID_RISEC = 0x5C ,
XMSG_TYPE_ID_RISED = 0x5D ,
XMSG_TYPE_ID_RISEE = 0x5E ,
XMSG_TYPE_ID_RISEF = 0x5F ,
XMSG_TYPE_ID_PUMP0 = 0x60 ,
XMSG_TYPE_ID_PUMP1 = 0x61 ,
XMSG_TYPE_ID_PUMP2 = 0x62 ,
XMSG_TYPE_ID_PUMP3 = 0x63 ,
XMSG_TYPE_ID_PUMP4 = 0x64 ,
XMSG_TYPE_ID_PUMP5 = 0x65 ,
XMSG_TYPE_ID_PUMP6 = 0x66 ,
XMSG_TYPE_ID_PUMP7 = 0x67 ,
XMSG_TYPE_ID_PUMP8 = 0x68 ,
XMSG_TYPE_ID_PUMP9 = 0x69 ,
XMSG_TYPE_ID_PUMPA = 0x6A ,
XMSG_TYPE_ID_PUMPB = 0x6B ,
XMSG_TYPE_ID_PUMPC = 0x6C ,
XMSG_TYPE_ID_PUMPD = 0x6D ,
XMSG_TYPE_ID_PUMPE = 0x6E ,
XMSG_TYPE_ID_PUMPF = 0x6F ,
XMSG_TYPE_ID_PRES_SURF0 = 0x80 ,
XMSG_TYPE_ID_PRES_SURF1 = 0x81 ,
XMSG_TYPE_ID_PRES_SURF2 = 0x82 ,
XMSG_TYPE_ID_PRES_SURF3 = 0x83 ,
XMSG_TYPE_ID_PRES_SURF4 = 0x84 ,
XMSG_TYPE_ID_PRES_SURF5 = 0x85 ,
XMSG_TYPE_ID_PRES_SURF6 = 0x86 ,
XMSG_TYPE_ID_PRES_SURF7 = 0x87 ,
XMSG_TYPE_ID_PRES_SURF8 = 0x88 ,
XMSG_TYPE_ID_PRES_SURF9 = 0x89 ,
XMSG_TYPE_ID_PRES_SURFA = 0x8A ,
XMSG_TYPE_ID_PRES_SURFB = 0x8B ,
XMSG_TYPE_ID_PRES_SURFC = 0x8C ,
XMSG_TYPE_ID_PRES_SURFD = 0x8D ,
XMSG_TYPE_ID_PRES_SURFE = 0x8E ,
XMSG_TYPE_ID_PRES_SURFF = 0x8F ,
XMSG_TYPE_ID_PRES_HI_RES0 = 0x90 ,
XMSG_TYPE_ID_PRES_HI_RES1 = 0x91 ,
XMSG_TYPE_ID_PRES_HI_RES2 = 0x92 ,
XMSG_TYPE_ID_PRES_HI_RES3 = 0x93 ,
XMSG_TYPE_ID_PRES_HI_RES4 = 0x94 ,
XMSG_TYPE_ID_PRES_HI_RES5 = 0x95 ,
XMSG_TYPE_ID_PRES_HI_RES6 = 0x96 ,
XMSG_TYPE_ID_PRES_HI_RES7 = 0x97 ,
XMSG_TYPE_ID_PRES_DRIFT8 = 0x98 ,
XMSG_TYPE_ID_PRES_DRIFT9 = 0x99 ,
XMSG_TYPE_ID_PRES_DRIFTA = 0x9A ,
XMSG_TYPE_ID_PRES_DRIFTB = 0x9B ,
XMSG_TYPE_ID_PRES_DRIFTC = 0x9C ,
XMSG_TYPE_ID_PRES_DRIFTD = 0x9D ,
XMSG_TYPE_ID_PRES_DRIFTE = 0x9E ,
XMSG_TYPE_ID_PRES_DRIFTF = 0x9F ,
XMSG_TYPE_ID_TEMP_HI_RES0 = 0xA0 ,
XMSG_TYPE_ID_TEMP_HI_RES1 = 0xA1 ,
XMSG_TYPE_ID_TEMP_HI_RES2 = 0xA2 ,
XMSG_TYPE_ID_TEMP_HI_RES3 = 0xA3 ,
XMSG_TYPE_ID_TEMP_HI_RES4 = 0xA4 ,
XMSG_TYPE_ID_TEMP_HI_RES5 = 0xA5 ,
XMSG_TYPE_ID_TEMP_HI_RES6 = 0xA6 ,
XMSG_TYPE_ID_TEMP_HI_RES7 = 0xA7 ,
XMSG_TYPE_ID_TEMP_DRIFT8 = 0xA8 ,
XMSG_TYPE_ID_TEMP_DRIFT9 = 0xA9 ,

XMSG_TYPE_ID_TEMP_DRIFTA = 0xAA ,
XMSG_TYPE_ID_TEMP_DRIFTB = 0xAB ,
XMSG_TYPE_ID_TEMP_DRIFTC = 0xAC ,
XMSG_TYPE_ID_TEMP_DRIFTD = 0xAD ,
XMSG_TYPE_ID_TEMP_DRIFTE = 0xAE ,
XMSG_TYPE_ID_TEMP_DRIFTF = 0xAF ,
XMSG_TYPE_ID_SALN_HI_RES0 = 0xB0 ,
XMSG_TYPE_ID_SALN_HI_RES1 = 0xB1 ,
XMSG_TYPE_ID_SALN_HI_RES2 = 0xB2 ,
XMSG_TYPE_ID_SALN_HI_RES3 = 0xB3 ,
XMSG_TYPE_ID_SALN_HI_RES4 = 0xB4 ,
XMSG_TYPE_ID_SALN_HI_RES5 = 0xB5 ,
XMSG_TYPE_ID_SALN_HI_RES6 = 0xB6 ,
XMSG_TYPE_ID_SALN_HI_RES7 = 0xB7 ,
XMSG_TYPE_ID_SALN_DRIFT8 = 0xB8 ,
XMSG_TYPE_ID_SALN_DRIFT9 = 0xB9 ,
XMSG_TYPE_ID_SALN_DRIFTA = 0xBA ,
XMSG_TYPE_ID_SALN_DRIFTB = 0xBB ,
XMSG_TYPE_ID_SALN_DRIFTC = 0xBC ,
XMSG_TYPE_ID_SALN_DRIFTD = 0xBD ,
XMSG_TYPE_ID_SALN_DRIFTE = 0xBE ,
XMSG_TYPE_ID_SALN_DRIFTF = 0xBF ,
XMSG_TYPE_ID_ALAMO_WAVE_C1 = 0xC1 ,
XMSG_TYPE_ID_AUX_SENSOR_DATA = 0xC2 ,
XMSG_TYPE_ID_ALAMO_WAVE_C3 = 0xC3 ,
XMSG_TYPE_ID_MIZ_ACOUSTIC = 0xCA ,
XMSG_TYPE_ID_MISS_CONF0 = 0xD0 ,
XMSG_TYPE_ID_MISS_CONF1 = 0xD1 ,
XMSG_TYPE_ID_MISS_CONF2 = 0xD2 ,
XMSG_TYPE_ID_MISS_CONF3 = 0xD3 ,
XMSG_TYPE_ID_MISS_CONF4 = 0xD4 ,
XMSG_TYPE_ID_MISS_CONF5 = 0xD5 ,
XMSG_TYPE_ID_MISS_CONF6 = 0xD6 ,
XMSG_TYPE_ID_MISS_CONF7 = 0xD7 ,
XMSG_TYPE_ID_MISS_CMD_ECHO = 0xDE ,
XMSG_TYPE_ID_ENG_DIAGNOSTIC = 0xE0 ,
XMSG_TYPE_ID_ENG_DIVE = 0xE2 ,
XMSG_TYPE_ID_ENG_ABORT = 0xE3 ,
XMSG_TYPE_ID_ENG_BIT_PASS = 0xE5 ,
XMSG_TYPE_ID_ENG_BIT_FAIL = 0xE6 ,
XMSG_TYPE_ID_ENG_DESCENT = 0xE7 ,
XMSG_TYPE_ID_ENG_PARK = 0xE8 ,
XMSG_TYPE_ID_ENG_ASCENT = 0xE9 ,
XMSG_TYPE_ID_ENG_SURFACE = 0xEA ,
XMSG_TYPE_ID_ENG_VALVE = 0xEB ,
XMSG_TYPE_ID_ENG_ICE = 0xEC ,
XMSG_TYPE_ID_ARGO_DATA = 0xF0 ,
XMSG_TYPE_ID_TEST = 0xF1 ,
XMSG_TYPE_ID_UNKNOWN = 0xFF }

Enumeration of the unique X message identifiers.

- enum XPhaseCode_t {
  XPHASE_CODE_UNKNOWN = 0 ,
  XPHASE_CODE_START_OF_SINK = 1 ,
  XPHASE_CODE_BUOYANCY_AT_100DBAR = 2 ,
  XPHASE_CODE_SEEK = 3 ,
  XPHASE_CODE_BEGINNING_OF_DRIFT = 4 ,
  XPHASE_CODE_SEEK_DURING_DRIFT = 5 ,
  XPHASE_CODE_BEGINING_OF_FALL_TO_PROFILE = 6 ,
  XPHASE_CODE_START_OF_RISE = 7 ,
  XPHASE_CODE_END_OF_RISE = 8 ,
  XPHASE_CODE_ICE_TURNAROUND = 9 ,
  XPHASE_CODE_SINKING = 10 ,
  XPHASE_CODE_DRIFTING = 11 ,
  XPHASE_CODE_FALLING_TO_PROFILE = 12 ,
  XPHASE_CODE_RISING = 13 ,
  XPHASE_CODE_SURFACE = 14 ,
  XPHASE_CODE_INVALID = 15 }

  The X-message phase codes enumeration.
- enum XPumpPackingFormat_t {
  XPUMP_PACK_LEGACY = 0 ,
  XPUMP_PACK_STANDARD = 1 ,
  XPUMP_PACK_STANDARD2 = 2 ,

XPUMP_PACK_UNKNOWN = 99 }

Enumeration type definition of pump data X-Message packing formats.

- enum XSensorDataFormat_t {
  XSENS_FMT_DIFFERENCE = 0 ,
  XSENS_FMT_CURVATURE = 1 ,
  XSENS_FMT_CURVATURE_AUX_TEMP_CHANNEL = 3 ,
  XSENS_FMT_UNKNOWN = 0xFF }

  The X-Message sensor data packing format enumeration.
- enum XWaveMode_t {
  XWAVE_MODE_RAW = 0 ,
  XWAVE_MODE_SPECTRAL = 1 ,
  XWAVE_MODE_UNKNOWN = 99 }

  Wave data modes enumeration.
- enum **XWaveSpectralAxis_t** {
  XWAVE_SPECTRAL_AXIS_UNDEFINED = 0 ,
  XWAVE_SPECTRAL_AXIS_Z_PRIME = 1 ,
  XWAVE_SPECTRAL_AXIS_Y_PRIME = 2 ,
  XWAVE_SPECTRAL_AXIS_X_PRIME = 3 ,
  XWAVE_SPECTRAL_AXIS_Z = 4 ,
  XWAVE_SPECTRAL_AXIS_ZZ = 5 ,
  XWAVE_SPECTRAL_AXIS_ZX = 6 ,
  XWAVE_SPECTRAL_AXIS_ZY = 7 ,
  XWAVE_SPECTRAL_AXIS_X = 8 ,
  XWAVE_SPECTRAL_AXIS_XZ = 9 ,
  XWAVE_SPECTRAL_AXIS_XX = 10 ,
  XWAVE_SPECTRAL_AXIS_XY = 11 ,
  XWAVE_SPECTRAL_AXIS_Y = 12 ,
  XWAVE_SPECTRAL_AXIS_YZ = 13 ,
  XWAVE_SPECTRAL_AXIS_YX = 14 ,
  XWAVE_SPECTRAL_AXIS_YY = 15 ,
  XWAVE_NUM_SPECTRAL_AXES = 16 }
- enum **XWaveSpectralType_t** {
  XWAVE_SPECTRAL_TYPE_UNDEFINED = 0 ,
  XWAVE_SPECTRAL_TYPE_CO = 1 ,
  XWAVE_SPECTRAL_TYPE_QU = 2 ,
  XWAVE_SPECTRAL_TYPE_RE = 3 ,
  XWAVE_SPECTRAL_TYPE_IM = 4 ,
  XWAVE_SPECTRAL_TYPE_MAG = 5 ,
  XWAVE_SPECTRAL_TYPE_PHASE = 6 ,
  XWAVE_NUM_SPECTRAL_TYPES = 7 }

9.22.1 Detailed Description

The X-Message utility namespace.

9.22.2 Enumeration Type Documentation

9.22.2.1 XFallRiseDataPackFormat_t

`enum util::xmsg::XFallRiseDataPackFormat_t`

Packing format type enumeration for rise and fall data X-Messages.

Enumerator

XRISEFALL_FMT_LEGACY	The legacy 4-byte packing format. **Note** This format is only used by instrument previous to X-Message format version 2.0.
XRISEFALL_FMT_STNDRD	The standard 5-byte packing format. **Note** This packing format was add in X-Message version 2.0.

Enumerator

XRISEFALL_FMT_STNDRD_WITH_TEMP_AND_↩ SALN	The standard 9-byte packing format with temperature and salinity. **Note** This packing format was add in X-Message version 2.2.
XRISEFALL_FMT_UNKNOWN	Unknown, unset, or unspecified packing format.

9.22.2.2 XMsgTypeId_t

`enum util::xmsg::XMsgTypeId_t`

Enumeration of the unique X message identifiers.

Enumerator

XMSG_TYPE_ID_GPS_DIAGNOSTIC	GPS0.
XMSG_TYPE_ID_GPS_STARTOFDIVE	GPS1.
XMSG_TYPE_ID_GPS_ENDOFDIVE	GPS2.
XMSG_TYPE_ID_GPS_ABORT	GPS3.
XMSG_TYPE_ID_GPS_BITEST	GPS5.
XMSG_TYPE_ID_PRES0	PRES0.
XMSG_TYPE_ID_PRES1	PRES1.
XMSG_TYPE_ID_PRES2	PRES2.
XMSG_TYPE_ID_PRES3	PRES3.
XMSG_TYPE_ID_PRES4	PRES4.
XMSG_TYPE_ID_PRES5	PRES5.
XMSG_TYPE_ID_PRES6	PRES6.
XMSG_TYPE_ID_PRES7	PRES7.
XMSG_TYPE_ID_PRES8	PRES8.
XMSG_TYPE_ID_PRES9	PRES9.
XMSG_TYPE_ID_PRESA	PRESA.
XMSG_TYPE_ID_PRESB	PRESB.
XMSG_TYPE_ID_PRESC	PRESC.
XMSG_TYPE_ID_PRESD	PRESD.
XMSG_TYPE_ID_PRESE	PRESE.
XMSG_TYPE_ID_PRESF	PRESF.
XMSG_TYPE_ID_TEMP0	TEMP0.
XMSG_TYPE_ID_TEMP1	TEMP1.
XMSG_TYPE_ID_TEMP2	TEMP2.
XMSG_TYPE_ID_TEMP3	TEMP3.
XMSG_TYPE_ID_TEMP4	TEMP4.
XMSG_TYPE_ID_TEMP5	TEMP5.
XMSG_TYPE_ID_TEMP6	TEMP6.
XMSG_TYPE_ID_TEMP7	TEMP7.
XMSG_TYPE_ID_TEMP8	TEMP8.
XMSG_TYPE_ID_TEMP9	TEMP9.
XMSG_TYPE_ID_TEMPA	TEMPA.
XMSG_TYPE_ID_TEMPB	TEMPB.
XMSG_TYPE_ID_TEMPC	TEMPC.
XMSG_TYPE_ID_TEMPD	TEMPD.
XMSG_TYPE_ID_TEMPE	TEMPE.
XMSG_TYPE_ID_TEMPF	TEMPF.
XMSG_TYPE_ID_SALN0	SALN0.
XMSG_TYPE_ID_SALN1	SALN1.
XMSG_TYPE_ID_SALN2	SALN2.
XMSG_TYPE_ID_SALN3	SALN3.
XMSG_TYPE_ID_SALN4	SALN4.
XMSG_TYPE_ID_SALN5	SALN5.
XMSG_TYPE_ID_SALN6	SALN6.

Enumerator

XMSG_TYPE_ID_SALN7	SALN7.
XMSG_TYPE_ID_SALN8	SALN8.
XMSG_TYPE_ID_SALN9	SALN9.
XMSG_TYPE_ID_SALNA	SALNA.
XMSG_TYPE_ID_SALNB	SALNB.
XMSG_TYPE_ID_SALNC	SALNC.
XMSG_TYPE_ID_SALND	SALND.
XMSG_TYPE_ID_SALNE	SALNE.
XMSG_TYPE_ID_SALNF	SALNF.
XMSG_TYPE_ID_FALL0	FALL0.
XMSG_TYPE_ID_FALL1	FALL1.
XMSG_TYPE_ID_FALL2	FALL2.
XMSG_TYPE_ID_FALL3	FALL3.
XMSG_TYPE_ID_FALL4	FALL4.
XMSG_TYPE_ID_FALL5	FALL5.
XMSG_TYPE_ID_FALL6	FALL6.
XMSG_TYPE_ID_FALL7	FALL7.
XMSG_TYPE_ID_FALL8	FALL8.
XMSG_TYPE_ID_FALL9	FALL9.
XMSG_TYPE_ID_FALLA	FALLA.
XMSG_TYPE_ID_FALLB	FALLB.
XMSG_TYPE_ID_FALLC	FALLC.
XMSG_TYPE_ID_FALLD	FALLD.
XMSG_TYPE_ID_FALLE	FALLE.
XMSG_TYPE_ID_FALLF	FALLF.
XMSG_TYPE_ID_RISE0	XMSG_TYPE_ID_RISE0.
XMSG_TYPE_ID_RISE1	XMSG_TYPE_ID_RISE1.
XMSG_TYPE_ID_RISE2	XMSG_TYPE_ID_RISE2.
XMSG_TYPE_ID_RISE3	XMSG_TYPE_ID_RISE3.
XMSG_TYPE_ID_RISE4	XMSG_TYPE_ID_RISE4.
XMSG_TYPE_ID_RISE5	XMSG_TYPE_ID_RISE5.
XMSG_TYPE_ID_RISE6	XMSG_TYPE_ID_RISE6.
XMSG_TYPE_ID_RISE7	XMSG_TYPE_ID_RISE7.
XMSG_TYPE_ID_RISE8	XMSG_TYPE_ID_RISE8.
XMSG_TYPE_ID_RISE9	XMSG_TYPE_ID_RISE9.
XMSG_TYPE_ID_RISEA	XMSG_TYPE_ID_RISEA.
XMSG_TYPE_ID_RISEB	XMSG_TYPE_ID_RISEB.
XMSG_TYPE_ID_RISEC	XMSG_TYPE_ID_RISEC.
XMSG_TYPE_ID_RISED	XMSG_TYPE_ID_RISED.
XMSG_TYPE_ID_RISEE	XMSG_TYPE_ID_RISEE.
XMSG_TYPE_ID_RISEF	XMSG_TYPE_ID_RISEF.
XMSG_TYPE_ID_PUMP0	XMSG_TYPE_ID_PUMP0.
XMSG_TYPE_ID_PUMP1	XMSG_TYPE_ID_PUMP1.
XMSG_TYPE_ID_PUMP2	XMSG_TYPE_ID_PUMP2.
XMSG_TYPE_ID_PUMP3	XMSG_TYPE_ID_PUMP3.
XMSG_TYPE_ID_PUMP4	XMSG_TYPE_ID_PUMP4.
XMSG_TYPE_ID_PUMP5	XMSG_TYPE_ID_PUMP5.
XMSG_TYPE_ID_PUMP6	XMSG_TYPE_ID_PUMP6.
XMSG_TYPE_ID_PUMP7	XMSG_TYPE_ID_PUMP7.
XMSG_TYPE_ID_PUMP8	XMSG_TYPE_ID_PUMP8.
XMSG_TYPE_ID_PUMP9	XMSG_TYPE_ID_PUMP9.
XMSG_TYPE_ID_PUMPA	XMSG_TYPE_ID_PUMPA.
XMSG_TYPE_ID_PUMPB	XMSG_TYPE_ID_PUMPB.
XMSG_TYPE_ID_PUMPC	XMSG_TYPE_ID_PUMPC.
XMSG_TYPE_ID_PUMPD	XMSG_TYPE_ID_PUMPD.
XMSG_TYPE_ID_PUMPE	XMSG_TYPE_ID_PUMPE.
XMSG_TYPE_ID_PUMPF	XMSG_TYPE_ID_PUMPF.
XMSG_TYPE_ID_PRES_SURF0	XMSG_TYPE_ID_PRES_SURF0.
XMSG_TYPE_ID_PRES_SURF1	XMSG_TYPE_ID_PRES_SURF1.
XMSG_TYPE_ID_PRES_SURF2	XMSG_TYPE_ID_PRES_SURF2.
XMSG_TYPE_ID_PRES_SURF3	XMSG_TYPE_ID_PRES_SURF3.

Enumerator

XMSG_TYPE_ID_PRES_SURF4	XMSG_TYPE_ID_PRES_SURF4.
XMSG_TYPE_ID_PRES_SURF5	XMSG_TYPE_ID_PRES_SURF5.
XMSG_TYPE_ID_PRES_SURF6	XMSG_TYPE_ID_PRES_SURF6.
XMSG_TYPE_ID_PRES_SURF7	XMSG_TYPE_ID_PRES_SURF7.
XMSG_TYPE_ID_PRES_SURF8	XMSG_TYPE_ID_PRES_SURF8.
XMSG_TYPE_ID_PRES_SURF9	XMSG_TYPE_ID_PRES_SURF9.
XMSG_TYPE_ID_PRES_SURFA	XMSG_TYPE_ID_PRES_SURFA.
XMSG_TYPE_ID_PRES_SURFB	XMSG_TYPE_ID_PRES_SURFB.
XMSG_TYPE_ID_PRES_SURFC	XMSG_TYPE_ID_PRES_SURFC.
XMSG_TYPE_ID_PRES_SURFD	XMSG_TYPE_ID_PRES_SURFD.
XMSG_TYPE_ID_PRES_SURFE	XMSG_TYPE_ID_PRES_SURFE.
XMSG_TYPE_ID_PRES_SURFF	XMSG_TYPE_ID_PRES_SURFF.
XMSG_TYPE_ID_PRES_HI_RES0	XMSG_TYPE_ID_PRES_HI_RES0.
XMSG_TYPE_ID_PRES_HI_RES1	XMSG_TYPE_ID_PRES_HI_RES1.
XMSG_TYPE_ID_PRES_HI_RES2	XMSG_TYPE_ID_PRES_HI_RES2.
XMSG_TYPE_ID_PRES_HI_RES3	XMSG_TYPE_ID_PRES_HI_RES3.
XMSG_TYPE_ID_PRES_HI_RES4	XMSG_TYPE_ID_PRES_HI_RES4.
XMSG_TYPE_ID_PRES_HI_RES5	XMSG_TYPE_ID_PRES_HI_RES5.
XMSG_TYPE_ID_PRES_HI_RES6	XMSG_TYPE_ID_PRES_HI_RES6.
XMSG_TYPE_ID_PRES_HI_RES7	XMSG_TYPE_ID_PRES_HI_RES7.
XMSG_TYPE_ID_PRES_DRIFT8	XMSG_TYPE_ID_PRES_DRIFT8.
XMSG_TYPE_ID_PRES_DRIFT9	XMSG_TYPE_ID_PRES_DRIFT9.
XMSG_TYPE_ID_PRES_DRIFTA	XMSG_TYPE_ID_PRES_DRIFTA.
XMSG_TYPE_ID_PRES_DRIFTB	XMSG_TYPE_ID_PRES_DRIFTB.
XMSG_TYPE_ID_PRES_DRIFTC	XMSG_TYPE_ID_PRES_DRIFTC.
XMSG_TYPE_ID_PRES_DRIFTD	XMSG_TYPE_ID_PRES_DRIFTD.
XMSG_TYPE_ID_PRES_DRIFTE	XMSG_TYPE_ID_PRES_DRIFTE.
XMSG_TYPE_ID_PRES_DRIFTF	XMSG_TYPE_ID_PRES_DRIFTF.
XMSG_TYPE_ID_TEMP_HI_RES0	XMSG_TYPE_ID_TEMP_HI_RES0.
XMSG_TYPE_ID_TEMP_HI_RES1	XMSG_TYPE_ID_TEMP_HI_RES1.
XMSG_TYPE_ID_TEMP_HI_RES2	XMSG_TYPE_ID_TEMP_HI_RES2.
XMSG_TYPE_ID_TEMP_HI_RES3	XMSG_TYPE_ID_TEMP_HI_RES3.
XMSG_TYPE_ID_TEMP_HI_RES4	XMSG_TYPE_ID_TEMP_HI_RES4.
XMSG_TYPE_ID_TEMP_HI_RES5	XMSG_TYPE_ID_TEMP_HI_RES5.
XMSG_TYPE_ID_TEMP_HI_RES6	XMSG_TYPE_ID_TEMP_HI_RES6.
XMSG_TYPE_ID_TEMP_HI_RES7	XMSG_TYPE_ID_TEMP_HI_RES7.
XMSG_TYPE_ID_TEMP_DRIFT8	XMSG_TYPE_ID_TEMP_DRIFT8.
XMSG_TYPE_ID_TEMP_DRIFT9	XMSG_TYPE_ID_TEMP_DRIFT9.
XMSG_TYPE_ID_TEMP_DRIFTA	XMSG_TYPE_ID_TEMP_DRIFTA.
XMSG_TYPE_ID_TEMP_DRIFTB	XMSG_TYPE_ID_TEMP_DRIFTB.
XMSG_TYPE_ID_TEMP_DRIFTC	XMSG_TYPE_ID_TEMP_DRIFTC.
XMSG_TYPE_ID_TEMP_DRIFTD	XMSG_TYPE_ID_TEMP_DRIFTD.
XMSG_TYPE_ID_TEMP_DRIFTE	XMSG_TYPE_ID_TEMP_DRIFTE.
XMSG_TYPE_ID_TEMP_DRIFTF	XMSG_TYPE_ID_TEMP_DRIFTF.
XMSG_TYPE_ID_SALN_HI_RES0	XMSG_TYPE_ID_SALN_HI_RES0.
XMSG_TYPE_ID_SALN_HI_RES1	XMSG_TYPE_ID_SALN_HI_RES1.
XMSG_TYPE_ID_SALN_HI_RES2	XMSG_TYPE_ID_SALN_HI_RES2.
XMSG_TYPE_ID_SALN_HI_RES3	XMSG_TYPE_ID_SALN_HI_RES3.
XMSG_TYPE_ID_SALN_HI_RES4	XMSG_TYPE_ID_SALN_HI_RES4.
XMSG_TYPE_ID_SALN_HI_RES5	XMSG_TYPE_ID_SALN_HI_RES5.
XMSG_TYPE_ID_SALN_HI_RES6	XMSG_TYPE_ID_SALN_HI_RES6.
XMSG_TYPE_ID_SALN_HI_RES7	XMSG_TYPE_ID_SALN_HI_RES7.
XMSG_TYPE_ID_SALN_DRIFT8	XMSG_TYPE_ID_SALN_DRIFT8.
XMSG_TYPE_ID_SALN_DRIFT9	XMSG_TYPE_ID_SALN_DRIFT9.
XMSG_TYPE_ID_SALN_DRIFTA	XMSG_TYPE_ID_SALN_DRIFTA.
XMSG_TYPE_ID_SALN_DRIFTB	XMSG_TYPE_ID_SALN_DRIFTB.
XMSG_TYPE_ID_SALN_DRIFTC	XMSG_TYPE_ID_SALN_DRIFTC.
XMSG_TYPE_ID_SALN_DRIFTD	XMSG_TYPE_ID_SALN_DRIFTD.
XMSG_TYPE_ID_SALN_DRIFTE	XMSG_TYPE_ID_SALN_DRIFTE.
XMSG_TYPE_ID_SALN_DRIFTF	XMSG_TYPE_ID_SALN_DRIFTF.
XMSG_TYPE_ID_ALAMO_WAVE_C1	ALAMO_WAVE.

Enumerator

XMSG_TYPE_ID_AUX_SENSOR_DATA	ALAMO_WAVE.
XMSG_TYPE_ID_ALAMO_WAVE_C3	ALAMO_WAVE.
XMSG_TYPE_ID_MIZ_ACOUSTIC	MIZ_ACOUSTIC.
XMSG_TYPE_ID_MISS_CONF0	MISS_CONF0.
XMSG_TYPE_ID_MISS_CONF1	MISS_CONF1.
XMSG_TYPE_ID_MISS_CONF2	MISS_CONF2.
XMSG_TYPE_ID_MISS_CONF3	MISS_CONF3.
XMSG_TYPE_ID_MISS_CONF4	MISS_CONF4.
XMSG_TYPE_ID_MISS_CONF5	MISS_CONF5.
XMSG_TYPE_ID_MISS_CONF6	MISS_CONF6.
XMSG_TYPE_ID_MISS_CONF7	MISS_CONF7.
XMSG_TYPE_ID_MISS_CMD_ECHO	MISS_CMD_ECHO.
XMSG_TYPE_ID_ENG_DIAGNOSTIC	ENG_DIAGNOSTIC.
XMSG_TYPE_ID_ENG_DIVE	ENG_DIVE.
XMSG_TYPE_ID_ENG_ABORT	ENG_MISS_START.
XMSG_TYPE_ID_ENG_BIT_PASS	ENG_BIT_PASS.
XMSG_TYPE_ID_ENG_BIT_FAIL	ENG_BIT_FAIL.
XMSG_TYPE_ID_ARGO_DATA	ARGO_DATA.
XMSG_TYPE_ID_TEST	TEST.
XMSG_TYPE_ID_UNKNOWN	UNKNOWN.

9.22.2.3 XPhaseCode_t

```
enum util::xmsg::XPhaseCode_t
```

The X-message phase codes enumeration.

The phase code is used to indicate the context of a measurement i.e. what phase of the dive cycle the measurement was taken.

Enumerator

XPHASE_CODE_UNKNOWN	Phase code 0: unknown mission phase. This code is only used for instruments with an X-Message format previous to version 2.0.
XPHASE_CODE_START_OF_SINK	Phase code 1: start of the descent-from-surface phase of the dive cycle.
XPHASE_CODE_BUOYANCY_AT_100DBAR	Phase code 2: first pump (Tlastx) at 100 dBar.
XPHASE_CODE_SEEK	Phase code 3: seek during the descent-to-drift/park-pressure.
XPHASE_CODE_BEGINNING_OF_DRIFT	Phase code 4: the beginning of the drift phase of the dive cycle.
XPHASE_CODE_SEEK_DURING_DRIFT	Phase code 5: seek to drift/park pressure during the drift phase of the dive cycle.
XPHASE_CODE_BEGINING_OF_FALL_TO_PROFILE	Phase code 6: beginning of the descent from drift/park depth to profile depth.
XPHASE_CODE_START_OF_RISE	Phase code 7: start of the ascent/profile/rise phase of the dive cycle.
XPHASE_CODE_END_OF_RISE	Phase code 8: end of the ascent/profile/rise phase of the dive cycle.
XPHASE_CODE_ICE_TURNAROUND	Phase code 9: turn-around due to the ice detection algorithm.
XPHASE_CODE_SINKING	Phase code 10: the descent-from-surface phase of the dive cycle.
XPHASE_CODE_DRIFTING	Phase code 11: the drift phase of the dive cycle.
XPHASE_CODE_FALLING_TO_PROFILE	Phase code 12: the descent from drift/park depth to profile depth.
XPHASE_CODE_RISING	Phase code 13: the ascent/profile/rise phase of the dive cycle.
XPHASE_CODE_SURFACE	Phase code 14: the surface phase of the dive cycle.
XPHASE_CODE_INVALID	Phase code 15: invalid. This code is not used.

9.22.2.4 XPumpPackingFormat_t

`enum util::xmsg::XPumpPackingFormat_t`

Enumeration type definition of pump data X-Message packing formats.

Enumerator

XPUMP_PACK_LEGACY	X-Message pump data packing format 0: The legacy packing format identifier.
XPUMP_PACK_STANDARD	X-Message pump data packing format 1: The standard packing format identifier.
XPUMP_PACK_STANDARD2	X-Message pump data packing format 2: The (tentative) packing format identifier.
XPUMP_PACK_UNKNOWN	X-Message pump data packing format 99: The unknown/unset/invalid packing format identifier.

9.22.2.5 XSensorDataFormat_t

`enum util::xmsg::XSensorDataFormat_t`

The X-Message sensor data packing format enumeration.

This type enumerates the different packing format types that may be used by the class AbstractXSensorData and the classes that inherit from it. The manner in which the sensor data is decoded is determined by the data packing formats enumerated here.

Enumerator

XSENS_FMT_DIFFERENCE	The X-Message sensor data packing format 0: Legacy first-difference packing format.
XSENS_FMT_CURVATURE	The X-Message sensor data packing format 1: Standard curvature packing format. **Note** The packing format was added in X version 2.0.
XSENS_FMT_UNKNOWN	The X-Message sensor data packing format 255: Unknown packing format.

9.22.2.6 XWaveMode_t

`enum util::xmsg::XWaveMode_t`

Wave data modes enumeration.

Note

Currently, only raw wave motion measurements are supported.

Enumerator

XWAVE_MODE_RAW	Raw data mode.
XWAVE_MODE_SPECTRAL	Spectral data mode.
XWAVE_MODE_UNKNOWN	Unknown/invalid data mode.

9.23 vmime Namespace Reference

VMime library namespace.

Classes

- class shared_ptr

  Dummy template class used for configuring Doxygen to interpret vmime::shared_ptr objects as plain pointers.

9.23.1 Detailed Description

VMime library namespace.

Chapter 10

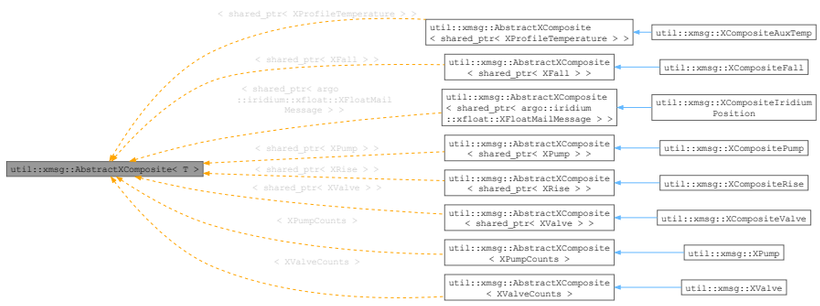
Class Documentation

10.1 util::xmsg::AbstractXComposite< T > Class Template Reference

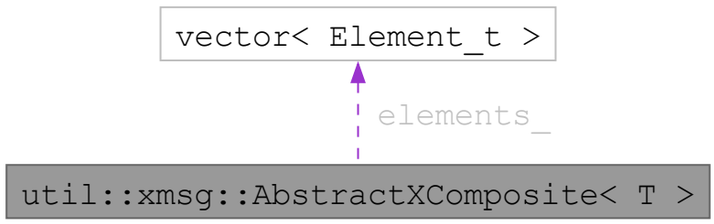
An abstract class representing XFloat dive data that consists of multiple X-Messages.

`#include <AbstractXComposite.h>`

Inheritance diagram for util::xmsg::AbstractXComposite< T >:



Collaboration diagram for util::xmsg::AbstractXComposite< T >:



Public Member Functions

- AbstractXComposite ()

  The default constructor.
- virtual ~AbstractXComposite ()

  The destructor.
- template<int I>
  void AppendAllElements (const static_vector< Element_t, I > &elements)

  Append a vector of elements to the vector of individual X-Messages elements that compose this composite object.
- virtual void AssignElement (const Element_t &element)

  Assign an element to this composite object.
- const vector< Element_t > & elements () const

  Get the vector of individual X-Messages elements that compose this composite object.
- virtual bool **IsEmpty** () const
- virtual size_t num_elements () const

  Get the number of individual elements that compose this composite object.

Protected Member Functions

- void AppendAllElements (const vector< Element_t > &elements)

  Append a vector of elements to the vector of individual X-Messages elements that compose this composite object.
- void AppendElement (const Element_t &element)

  Append one element to the vector of individual X-Messages elements that compose this composite object.

Private Types

- typedef T Element_t

  Type definition for the element aggregated by this object.
- typedef ElementVec_t::const_iterator **ElementConstIter_t**
- typedef ElementVec_t::iterator **ElementIter_t**
- typedef vector< Element_t > **ElementVec_t**

Private Attributes

- ElementVec_t elements_

  The vector of individual X-Messages elements that compose this composite object.

10.1.1 Detailed Description

template<typename T>
class util::xmsg::AbstractXComposite< T >

An abstract class representing XFloat dive data that consists of multiple X-Messages.

10.1.2 Member Typedef Documentation

10.1.2.1 Element_t

```
template<typename T >
typedef T util::xmsg::AbstractXComposite< T >::Element_t  [private]
```

Type definition for the element aggregated by this object.

10.1.3 Constructor & Destructor Documentation

10.1.3.1 AbstractXComposite()

```
template<typename T >
util::xmsg::AbstractXComposite< T >::AbstractXComposite ()  [inline]
```

The default constructor.

10.1.3.2 ~AbstractXComposite()

```
template<typename T >
virtual util::xmsg::AbstractXComposite< T >::~AbstractXComposite ()  [inline], [virtual]
```

The destructor.

10.1.4 Member Function Documentation

10.1.4.1 AppendAllElements() [1/2]

```
template<typename T >
template<int I>
void util::xmsg::AbstractXComposite< T >::AppendAllElements (
            const static_vector< Element_t, I > & elements)  [inline]
```

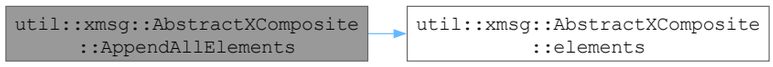
Append a vector of elements to the vector of individual X-Messages elements that compose this composite object.

Parameters

in	*elements*	The vector of elements to append.

References util::xmsg::AbstractXComposite< T >::elements(), and util::xmsg::AbstractXComposite< T >::elements_.

Here is the call graph for this function:



10.1.4.2 AppendAllElements() [2/2]

```
template<typename T >
void util::xmsg::AbstractXComposite< T >::AppendAllElements (
            const vector< Element_t > & elements)  [inline], [protected]
```

Append a vector of elements to the vector of individual X-Messages elements that compose this composite object.

Parameters

in	*elements*	The vector of elements to append.

References util::xmsg::AbstractXComposite< T >::elements(), and util::xmsg::AbstractXComposite< T >::elements_.

Here is the call graph for this function:

10.1.4.3 AppendElement()

```
template<typename T >
void util::xmsg::AbstractXComposite< T >::AppendElement (
            const Element_t & element)  [inline], [protected]
```

Append one element to the vector of individual X-Messages elements that compose this composite object.

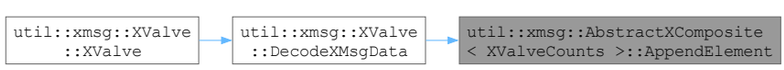
Parameters

in	*element*	The element to append.

References util::xmsg::AbstractXComposite< T >::elements_.

Referenced by util::xmsg::AbstractXComposite< T >::AssignElement().

Here is the caller graph for this function:



10.1.4.4 AssignElement()

```
template<typename T >
virtual void util::xmsg::AbstractXComposite< T >::AssignElement (
            const Element_t & element)  [inline], [virtual]
```

Assign an element to this composite object.

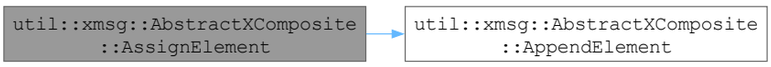
The default implementation appends the element; derived classes may override it to insert elements in order or reject duplicates.

Parameters

in	*element*	The element to assign.

References util::xmsg::AbstractXComposite< T >::AppendElement().

Here is the call graph for this function:



10.1.4.5 elements()

```
template<typename T >
const vector< Element_t > & util::xmsg::AbstractXComposite< T >::elements () const  [inline]
```

Get the vector of individual X-Messages elements that compose this composite object.

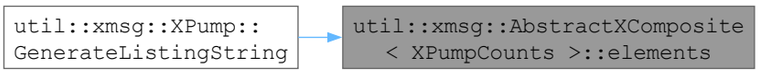
Returns

The vector of individual X-Messages elements that compose this composite object.

References util::xmsg::AbstractXComposite< T >::elements_.

Referenced by util::xmsg::AbstractXComposite< T >::AppendAllElements(), and util::xmsg::AbstractXComposite< T >::AppendAllEleme

Here is the caller graph for this function:



10.1.4.6 num_elements()

```
template<typename T >
virtual size_t util::xmsg::AbstractXComposite< T >::num_elements () const  [inline], [virtual]
```

Get the number of individual elements that compose this composite object.

Returns

The number of individual elements that compose this composite object.

References util::xmsg::AbstractXComposite< T >::elements_.

10.1.5 Member Data Documentation

10.1.5.1 elements_

```
template<typename T >
ElementVec_t util::xmsg::AbstractXComposite< T >::elements_  [private]
```

The vector of individual X-Messages elements that compose this composite object.

Referenced by util::xmsg::AbstractXComposite< T >::AppendAllElements(), util::xmsg::AbstractXComposite< T >::AppendAllElements(), util::xmsg::AbstractXComposite< T >::AppendElement(), util::xmsg::AbstractXComposite< T >::elements(), and util::xmsg::AbstractXComposite< T >::num_elements().

The documentation for this class was generated from the following file:

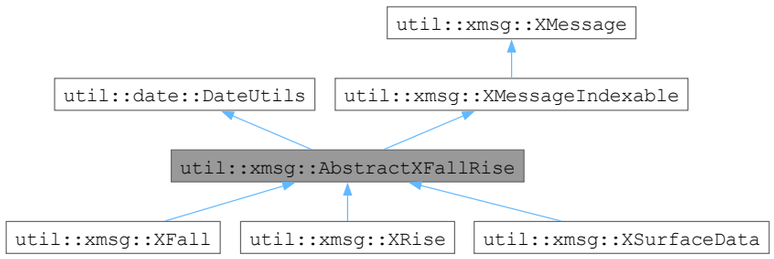
- data/xmsg/AbstractXComposite.h

10.2 util::xmsg::AbstractXFallRise Class Reference

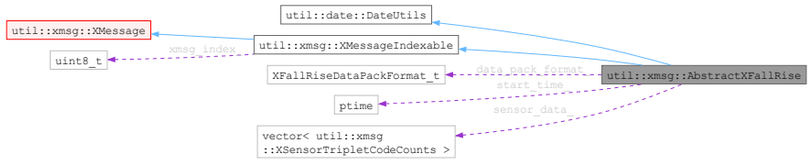
An abstract class representing a fall, rise, or surface pressure time-series data X-message.

`#include <AbstractXFallRise.h>`

Inheritance diagram for util::xmsg::AbstractXFallRise:



Collaboration diagram for util::xmsg::AbstractXFallRise:



Public Member Functions

- AbstractXFallRise (int16_t dive, uint16_t whoi, const vector< uint8_t > &data) ATTR_NO_EXCEPTIONS

  The parameterized constructor.
- virtual ~AbstractXFallRise ()

  The destructor.
- const uint8_t * CheckXMsgData (const vector< uint8_t > &data, int id, int minBytes, bool parseSize=true)

  Check the X-message for the correct ID, minimum size, and message terminator.
- XFallRiseDataPackFormat_t data_pack_format () const

  Get the enumerated data packing format type.
- virtual void DecodeXMsgData (const vector< uint8_t > &data)=0

  Decode the X message.
- int16_t id_dive () const

  Get the dive or cycle number of this X-Message.
- uint16_t id_instrument () const

  Get the unique instrument identifier of this X-Message.
- XMsgTypeId_t id_xmsg_type () const

  Get the enumerated X-Message type identifier.
- size_t num_samples () const

  Get the number of samples in this message.
- const vector< XSensorTripletCodeCounts > & sensor_data () const

  Get the vector of sensor data triplets.
- const ptime & start_time () const

  Get the start time of this message.
- uint8_t xmsg_index () const

  Get the index of this X-Message.
- uint16_t xmsg_size_bytes () const

  Get the size of the X-Message, in bytes.

Static Public Member Functions

- static string GenerateCurrentUtcTimeIso8601String ()

  Generate an ISO8601 format date/time string representation of the current time, in UTC.
- static string GenerateIso8601String (const ptime &p, bool extended=true)

  Generate an ISO8601 format date/time string representation of a POSIX time object.
- static double GenerateJulianDay (const ptime &dt, const ptime &ref_dt)

  Convert a date/time to fractional days since a reference date/time.
- static const string GenerateListingComment ()

  Get the listing entry comment string.
- static ptime GetCurrentUtcTime ()

  Generate a POSIX time object representing the current date and time (rounded to the nearest second).

Protected Member Functions

- void set_data_pack_format (XFallRiseDataPackFormat_t data_pack_format)

  Set the enumerated data packing format value for this message.
- void set_id_dive (int16_t id_dive)

  Set the dive or cycle number of this message.
- void set_id_instrument (uint16_t id_instrument)

  Set the unique instrument identifier.
- void set_id_xmsg_type (XMsgTypeId_t id_xmsg_type)

  Set the X-Message type identifier.
- void set_start_time (const ptime &start_time)

  Set the start time of this message.
- void set_xmsg_index (uint8_t xmsg_index)

  Set the index of this X-Message.
- void set_xmsg_size_bytes (uint16_t xmsg_size_bytes)

  Set the size of the X-Message, in bytes.
- vector< XSensorTripletCodeCounts > UnpackData (const vector< uint8_t > &data)

  Unpack the fall/rise data from the binary message.

Private Attributes

- XFallRiseDataPackFormat_t data_pack_format_

  The packing format type identifier.
- int16_t id_dive_

  The dive or cycle number of this message.
- uint16_t id_instrument_

  The unique WHOI identifier of this instrument.
- XMsgTypeId_t id_xmsg_type_

  The X-message type identifier.
- vector< XSensorTripletCodeCounts > sensor_data_

  The raw counts of measured fall, rise, or surface time-series data.
- ptime start_time_

  The start time reference.
- uint8_t xmsg_index_

  The index of this X-Message.
- uint16_t xmsg_size_bytes_

  The size of the X message, in bytes.

10.2.1 Detailed Description

An abstract class representing a fall, rise, or surface pressure time-series data X-message.

10.2.2 Constructor & Destructor Documentation

10.2.2.1 AbstractXFallRise()

```
util::xmsg::AbstractXFallRise::AbstractXFallRise (
            int16_t dive,
            uint16_t whoi,
            const vector< uint8_t > & data)  [inline]
```

The parameterized constructor.

Parameters

in	*dive*	The dive or cycle number of this message.
in	*whoi*	The unique WHOI identifier.
in	*data*	Binary X-message data to decode.

Exceptions

None.	Never throws.

10.2.2.2 ~AbstractXFallRise()

```
virtual util::xmsg::AbstractXFallRise::~AbstractXFallRise ()  [inline], [virtual]
```

The destructor.

10.2.3 Member Function Documentation

10.2.3.1 CheckXMsgData()

```
const uint8_t * util::xmsg::XMessage::CheckXMsgData (
            const vector< uint8_t > & data,
            int id,
            int minBytes,
            bool parseSize = true)  [inline], [inherited]
```

Check the X-message for the correct ID, minimum size, and message terminator.

Parameters

in	*data*	The X-message binary data to check.
in	*id*	The expected ID of the X-message. If this field is less that 0, then the ID check will be skipped.
in	*minBytes*	The minimum number of bytes expected in a valid message.
in	*parseSize*	If TRUE, then the message size field will be parsed from the X-Message. Otherwise, the size field will not be parsed.

Exceptions

util::log::MessageException	if • the message does not contain the expected ID, • the message does not contain the minimum number of expected bytes, • the message does not contain a terminator (';') in the correct location, or, • any other error occurs.

Returns

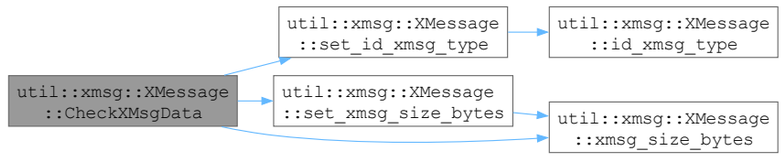
A pointer to the underlying data buffer of data.

References CATCH_BLOCK, EXC_THROW, lwrn, util::xmsg::XMessage::set_id_xmsg_type(), util::xmsg::XMessage::set_xmsg_size_byte
u16, and util::xmsg::XMessage::xmsg_size_bytes().

Referenced by util::xmsg::XAcousticPosition::DecodeXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XDriftPressure::Deco
util::xmsg::XDriftSalinity::DecodeXMsgData(), util::xmsg::XDriftTemperature::DecodeXMsgData(), util::xmsg::XEngAbort::DecodeXMsgDa
util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngBIT::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(),
util::xmsg::XEngDiagnostic::DecodeXMsgData(), util::xmsg::XEngDrift::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(),
util::xmsg::XEngStandard::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XGPS::DecodeXMsgData(),
util::xmsg::XHiResPressure::DecodeXMsgData(), util::xmsg::XHiResSalinity::DecodeXMsgData(), util::xmsg::XHiResTemperature::Decod
util::xmsg::XMissionCmdEcho::DecodeXMsgData(), util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XProfilePressure::

util::xmsg::XProfileSalinity::DecodeXMsgData(), util::xmsg::XProfileTemperature::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgD
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), and UnpackData().

Here is the call graph for this function:



10.2.3.2 data_pack_format()

```
XFallRiseDataPackFormat_t util::xmsg::AbstractXFallRise::data_pack_format () const  [inline]
```

Get the enumerated data packing format type.

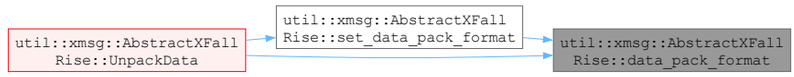
Returns

The enumerated data packing format type.

References data_pack_format_.

Referenced by set_data_pack_format(), and UnpackData().

Here is the caller graph for this function:



10.2.3.3 DecodeXMsgData()

```
virtual void util::xmsg::AbstractXFallRise::DecodeXMsgData (
            const vector< uint8_t > & data)  [pure virtual]
```

Decode the X message.

Parameters

in	*data*	The encoded X message to decode.

Exceptions

util::log::MessageException	if • an invalid X-Message type identifier is encountered. • the message is of insufficient length, • an invalid packing format is encountered, or • the message terminator is in an invalid location.

Implements util::xmsg::XMessageIndexable.

Implemented in util::xmsg::XFall, util::xmsg::XRise, and util::xmsg::XSurfaceData.

10.2.3.4 GenerateCurrentUtcTimeIso8601String()

```
static string util::date::DateUtils::GenerateCurrentUtcTimeIso8601String ()  [inline], [static],
[inherited]
```

Generate an ISO8601 format date/time string representation of the current time, in UTC.

Note

All date/time objects use the UTC time-zone.

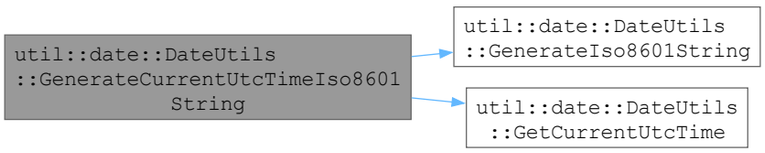
Returns

An ISO8601 format date/time string representation of the current time.

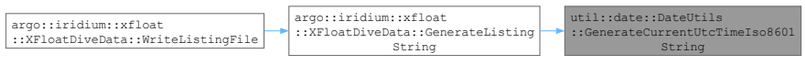
References util::date::DateUtils::GenerateIso8601String(), and util::date::DateUtils::GetCurrentUtcTime().

Referenced by argo::iridium::xfloat::XFloatDiveData::GenerateListingString().

Here is the call graph for this function:



Here is the caller graph for this function:



10.2.3.5 GenerateIso8601String()

```
static string util::date::DateUtils::GenerateIso8601String (
            const ptime & p,
            bool extended = true)  [inline], [static], [inherited]
```

Generate an ISO8601 format date/time string representation of a POSIX time object.

Note

All date/time objects use the UTC time-zone.

Parameters

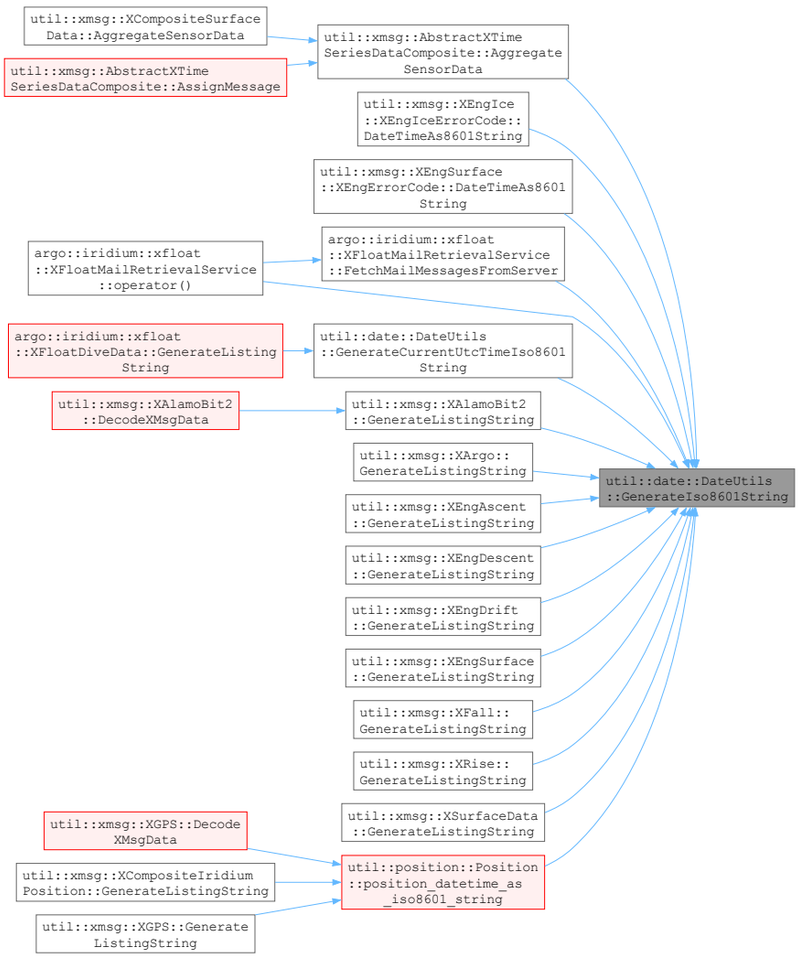
in	*p*	POSIX time from which to generate string.
in	*extended*	If TRUE, use the ISO 8601 extended format (YYYY-MM-DDThh:mm:ssZ); otherwise, use the basic format (YYYYMMDDThhmmss).

Returns

An ISO8601 format date/time string representation of the POSIX time object. If p is equal to boost::posix_↩ time::not_a_date_time, then the string "0000-00-00T00:00:00Z" (extended) or "00000000000000" (basic) will be returned.

Referenced by util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::AggregateSensorData(), util::xmsg::XEngIce::XEngIce
util::xmsg::XEngSurface::XEngErrorCode::DateTimeAs8601String(), argo::iridium::xfloat::XFloatMailRetrievalService::FetchMailMessages
util::date::DateUtils::GenerateCurrentUtcTimeIso8601String(), util::xmsg::XAlamoBit2::GenerateListingString(), util::xmsg::XArgo::Genera
util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListing
util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XRise::GenerateListingString(),
util::xmsg::XSurfaceData::GenerateListingString(), argo::iridium::xfloat::XFloatMailRetrievalService::operator()(), and
util::position::Position::position_datetime_as_iso8601_string().

Here is the caller graph for this function:



10.2.3.6 GenerateJulianDay()

```
static double util::date::DateUtils::GenerateJulianDay (
            const ptime & dt,
            const ptime & ref_dt)  [inline], [static], [inherited]
```

Convert a date/time to fractional days since a reference date/time.

Parameters

in	*dt*	The date/time to convert.
in	*ref*↩ *_dt*	The reference date/time (day 0).

Exceptions

util::log::MessageException	if either date/time is not valid.

Returns

The number of days (with millisecond resolution) from `ref_dt` to `dt`.

References CATCH_BLOCK, and EXC_THROW.

10.2.3.7 GenerateListingComment()

```
static const string util::xmsg::AbstractXFallRise::GenerateListingComment ()  [inline], [static]
```

Get the listing entry comment string.

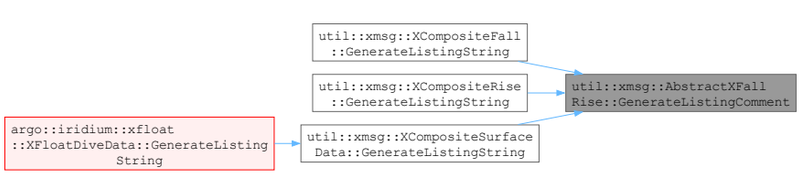
Returns

The listing entry comment string.

References CATCH_BLOCK.

Referenced by util::xmsg::XCompositeFall::GenerateListingString(), util::xmsg::XCompositeRise::GenerateListingString(), and util::xmsg::XCompositeSurfaceData::GenerateListingString().

Here is the caller graph for this function:



10.2.3.8 GetCurrentUtcTime()

```
static ptime util::date::DateUtils::GetCurrentUtcTime ()  [inline], [static], [inherited]
```

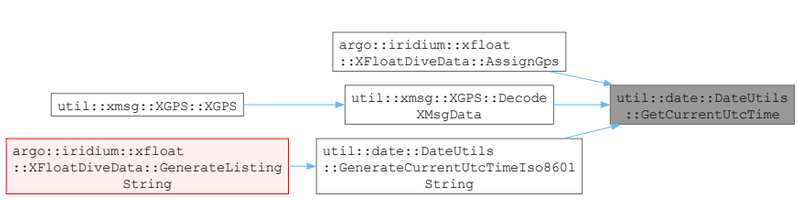
Generate a POSIX time object representing the current date and time (rounded to the nearest second).

Returns

A POSIX time object representing the current date and time (rounded to the nearest second).

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignGps(), util::xmsg::XGPS::DecodeXMsgData(), and util::date::DateUtils::GenerateCurrentUtcTimeIso8601String().

Here is the caller graph for this function:



10.2.3.9 id_dive()

```
int16_t util::xmsg::XMessage::id_dive () const  [inline], [inherited]
```

Get the dive or cycle number of this X-Message.

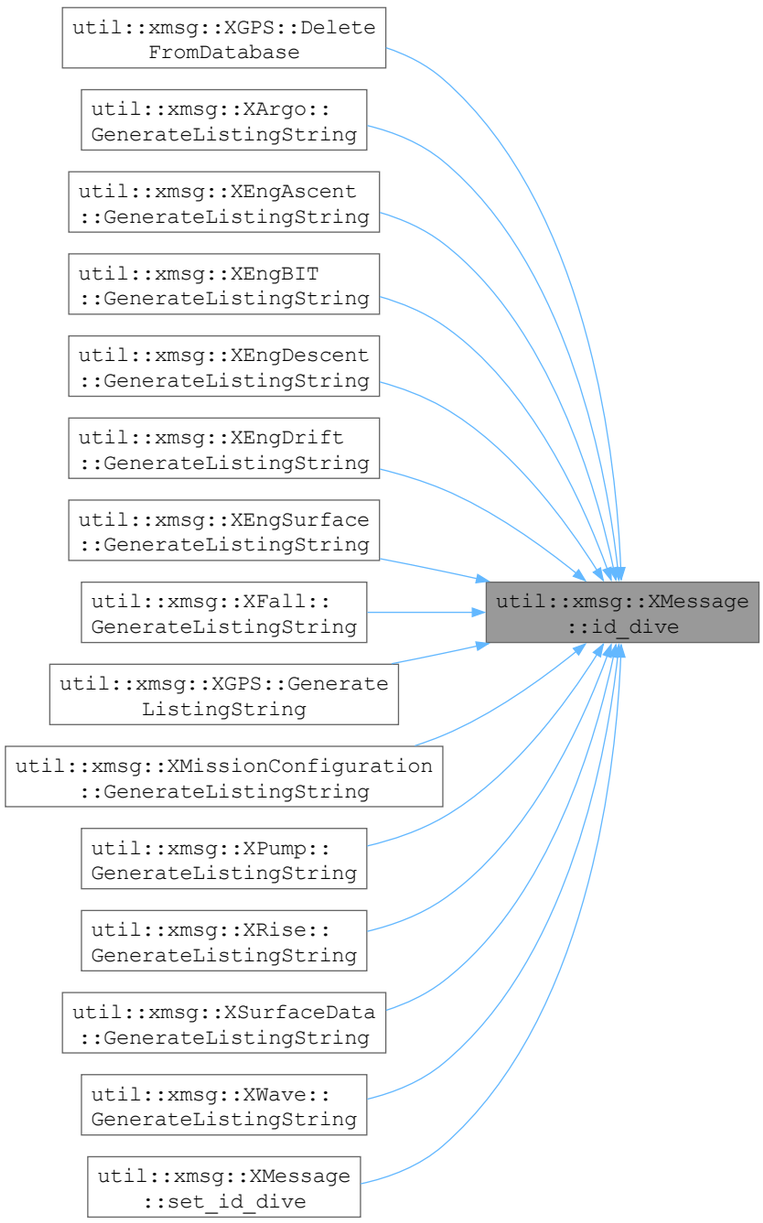
Returns

The dive or cycle number of this X-Message.

References util::xmsg::XMessage::id_dive_.

Referenced by util::xmsg::XGPS::DeleteFromDatabase(), util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateLis
util::xmsg::XEngBIT::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingStr
util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(),
util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing
util::xmsg::XSurfaceData::GenerateListingString(), util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_dive().

Here is the caller graph for this function:



10.2.3.10 id_instrument()

```
uint16_t util::xmsg::XMessage::id_instrument () const  [inline], [inherited]
```

Get the unique instrument identifier of this X-Message.

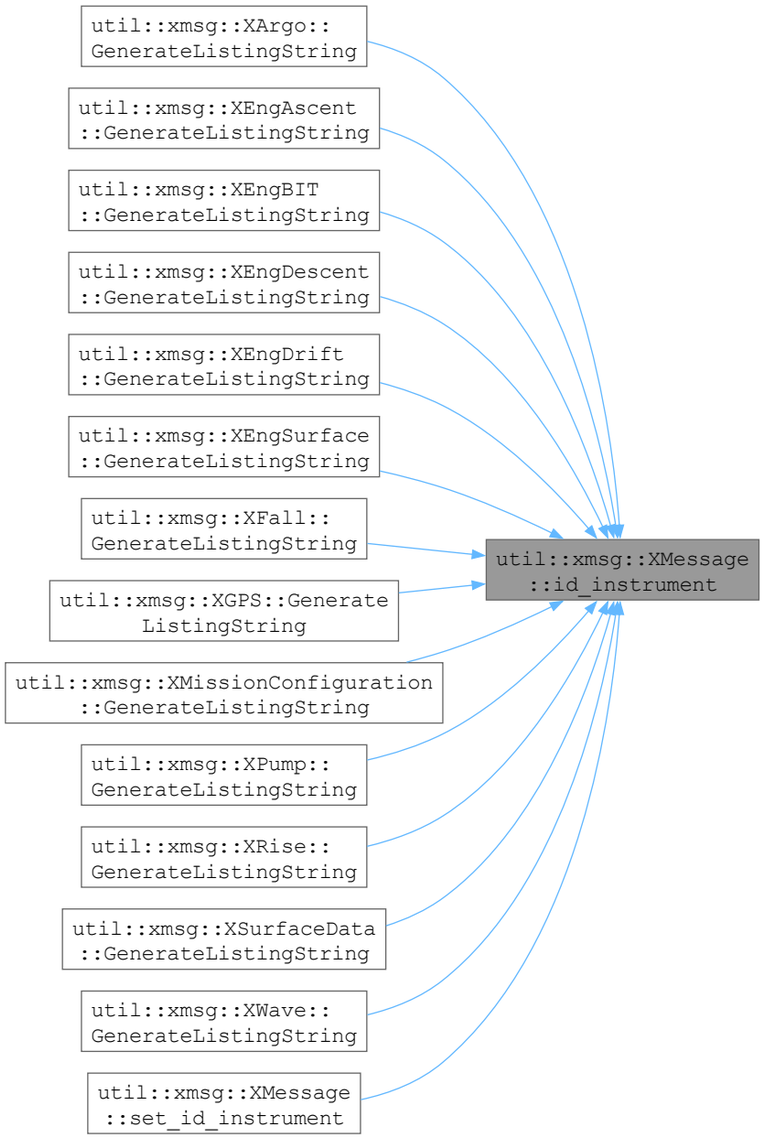
Returns

The unique instrument identifier of this X-Message.

References util::xmsg::XMessage::id_instrument_.

Referenced by util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngBIT::Generate
util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), util::xmsg::XEngSurface::GenerateListin
util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingS
util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(),
util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_instrument().

Here is the caller graph for this function:



10.2.3.11 id_xmsg_type()

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type () const  [inline], [inherited]
```

Get the enumerated X-Message type identifier.

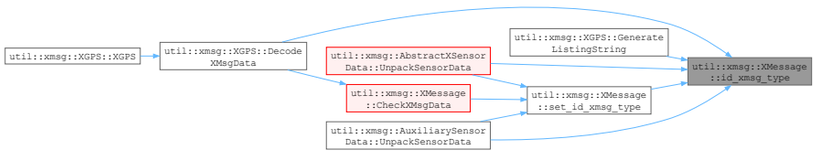
Returns

The enumerated X-Message type identifier.

References util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMessage::set_id_xmsg_typ
util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the caller graph for this function:



10.2.3.12 num_samples()

```
size_t util::xmsg::AbstractXFallRise::num_samples () const  [inline]
```

Get the number of samples in this message.

Returns

The number of samples in this message.

References sensor_data_.

Referenced by util::xmsg::XFall::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), and util::xmsg::XSurfaceData::Genera

Here is the caller graph for this function:

10.2.3.13 sensor_data()

```
const vector< XSensorTripletCodeCounts > & util::xmsg::AbstractXFallRise::sensor_data () const
[inline]
```

Get the vector of sensor data triplets.

Returns

The vector of sensor data triplets.

References sensor_data_.

Referenced by util::xmsg::XFall::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), and util::xmsg::XSurfaceData::Genera

Here is the caller graph for this function:

10.2.3.14 set_data_pack_format()

```
void util::xmsg::AbstractXFallRise::set_data_pack_format (
            XFallRiseDataPackFormat_t data_pack_format)  [inline], [protected]
```

Set the enumerated data packing format value for this message.

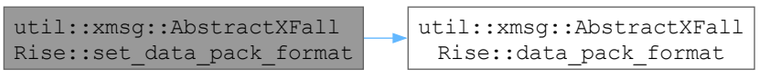
Parameters

in	*data_pack_format*	The enumerated data packing format value for this message to set.

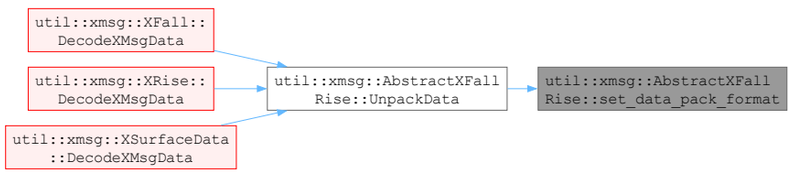
References data_pack_format(), and data_pack_format_.

Referenced by UnpackData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.2.3.15 set_id_dive()

```
void util::xmsg::XMessage::set_id_dive (
            int16_t id_dive)  [inline], [protected], [inherited]
```

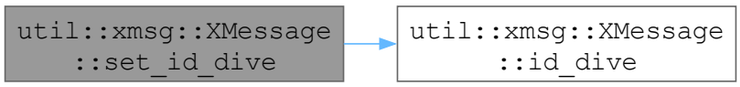
Set the dive or cycle number of this message.

Parameters

in	*id_dive*	The dive or cycle number of this message to set.

References util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::id_dive_.

Here is the call graph for this function:



10.2.3.16 set_id_instrument()

```
void util::xmsg::XMessage::set_id_instrument (
            uint16_t id_instrument)  [inline], [protected], [inherited]
```

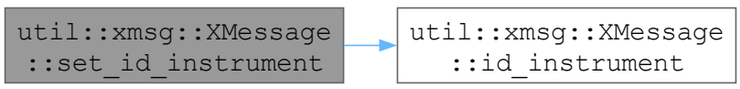
Set the unique instrument identifier.

Parameters

in	*id_instrument*	The unique instrument identifier to set.

References util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::id_instrument_.

Here is the call graph for this function:



10.2.3.17 set_id_xmsg_type()

```
void util::xmsg::XMessage::set_id_xmsg_type (
            XMsgTypeId_t id_xmsg_type)  [inline], [protected], [inherited]
```

Set the X-Message type identifier.

Parameters

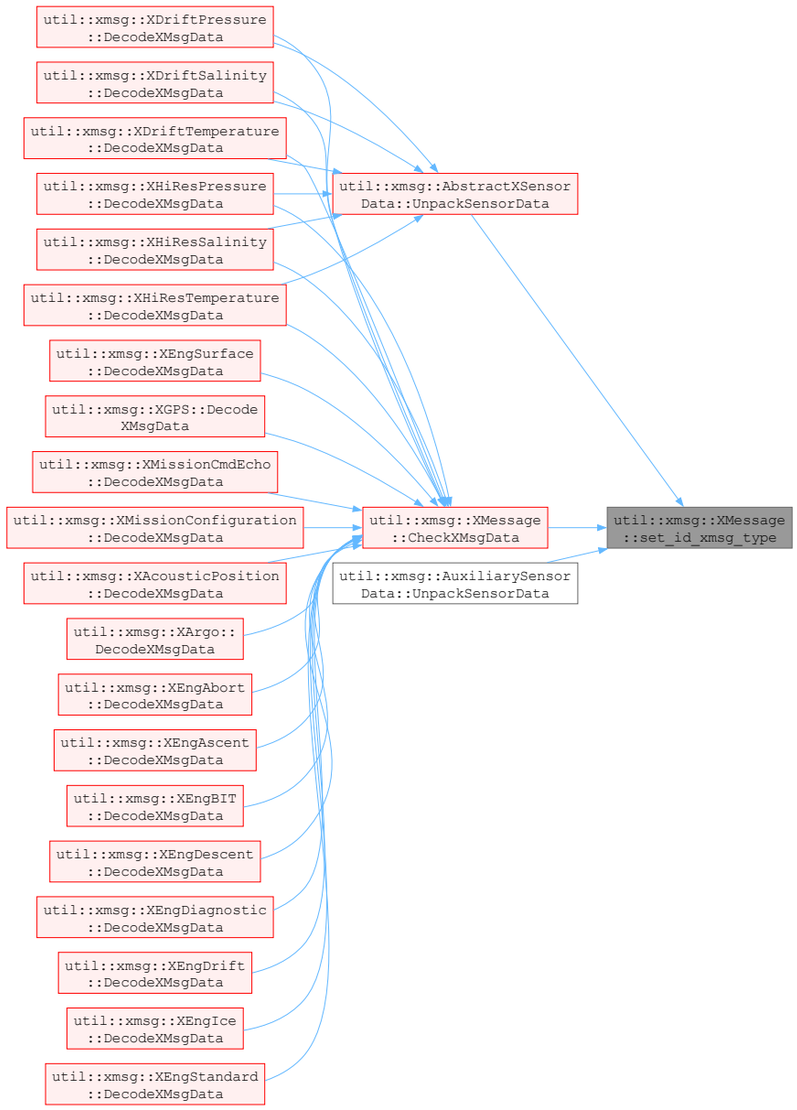
in	*id_xmsg_type*	The X-Message type identifier to set.

References util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:

Here is the caller graph for this function:



10.2.3.18 set_start_time()

```
void util::xmsg::AbstractXFallRise::set_start_time (
            const ptime & start_time)  [inline], [protected]
```

Set the start time of this message.

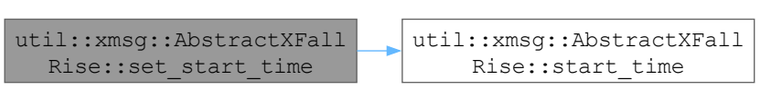
Parameters

in	*start_time*	The start time to set.

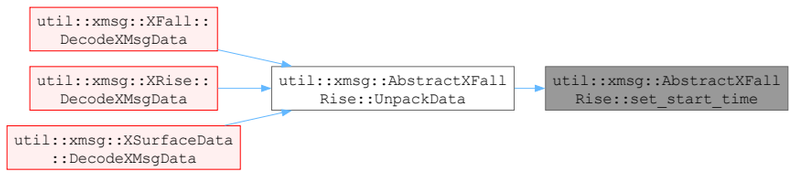
References start_time(), and start_time_.

Referenced by UnpackData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.2.3.19 set_xmsg_index()

```
void util::xmsg::XMessageIndexable::set_xmsg_index (
            uint8_t xmsg_index)  [inline], [protected], [inherited]
```

Set the index of this X-Message.

Parameters

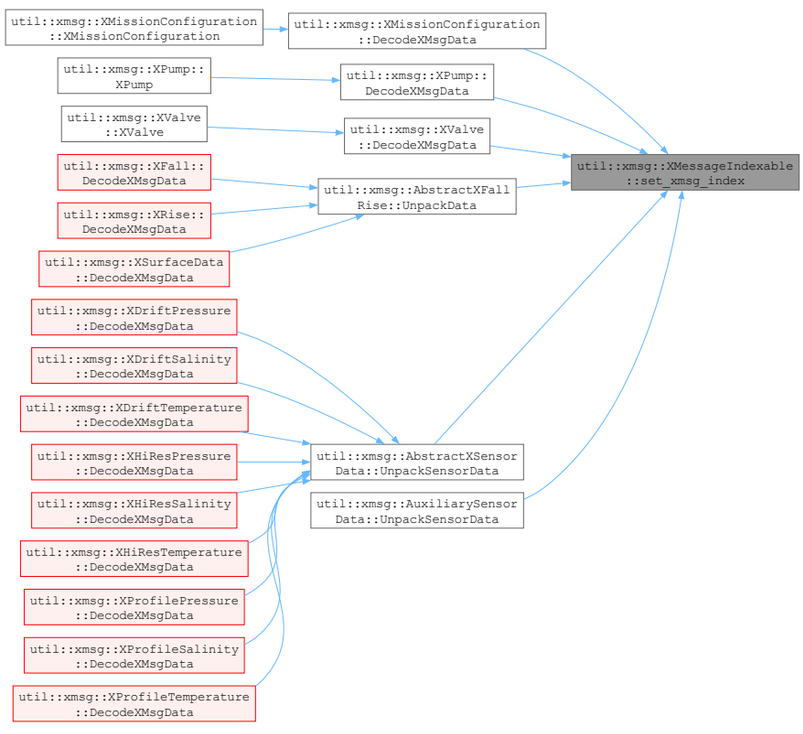
in	*xmsg_index*	The index of this X-Message.

References util::xmsg::XMessageIndexable::xmsg_index(), and util::xmsg::XMessageIndexable::xmsg_index_.

Referenced by util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(), util::xmsg::XValve::DecodeXMsgData(), UnpackData(), util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:

Here is the caller graph for this function:



10.2.3.20 set_xmsg_size_bytes()

```
void util::xmsg::XMessage::set_xmsg_size_bytes (
            uint16_t xmsg_size_bytes)  [inline], [protected], [inherited]
```

Set the size of the X-Message, in bytes.

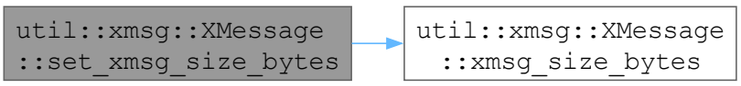
Parameters

in	*xmsg_size_bytes*	The size of the X-Message to set, in bytes.

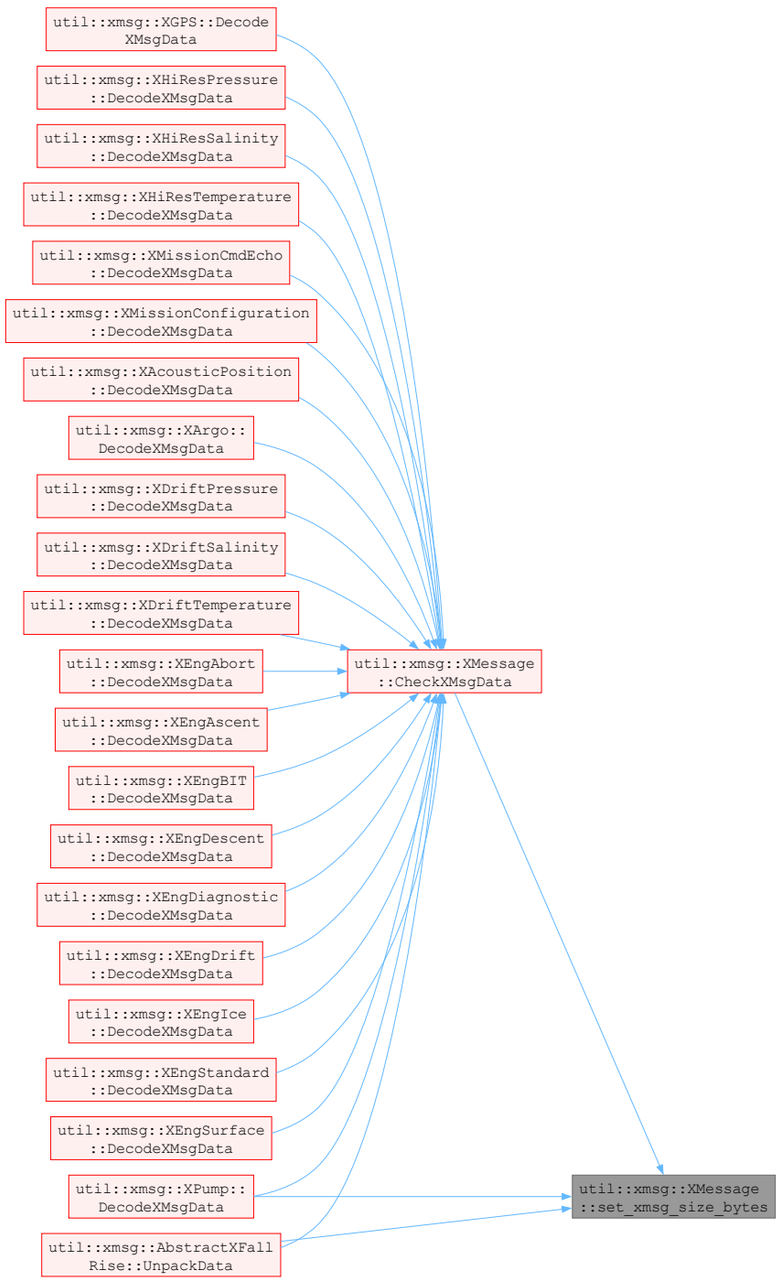
References util::xmsg::XMessage::xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XPump::DecodeXMsgData(), and UnpackData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.2.3.21 start_time()

```
const ptime & util::xmsg::AbstractXFallRise::start_time () const  [inline]
```

Get the start time of this message.

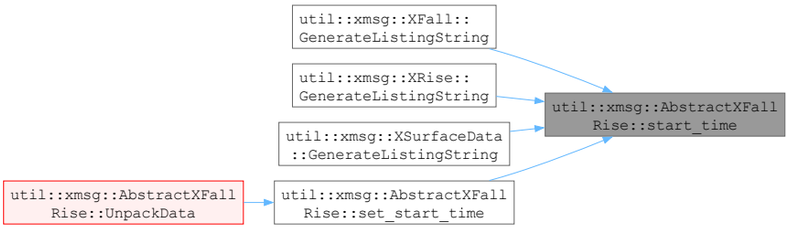
Returns

The start time of this message.

References start_time_.

Referenced by util::xmsg::XFall::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateLi and set_start_time().

Here is the caller graph for this function:



10.2.3.22 UnpackData()

```
vector< XSensorTripletCodeCounts > util::xmsg::AbstractXFallRise::UnpackData (
            const vector< uint8_t > & data)  [inline], [protected]
```

Unpack the fall/rise data from the binary message.

Parameters

in	*data*	The binary message to unpack.

Exceptions

util::log::MessageException	if • the message is of insufficient length, • an invalid packing format is encountered, or • the message terminator is in an invalid location.

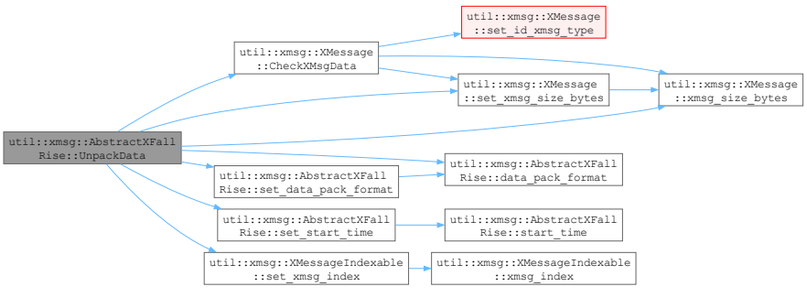
Returns

The raw counts of measured fall or rise time-series data.

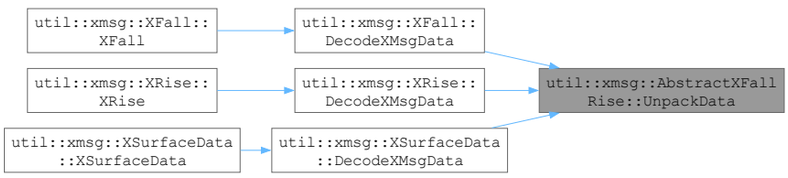
References CATCH_BLOCK, util::xmsg::XMessage::CheckXMsgData(), data_pack_format(), EXC_THROW, lwrn, s2000, sensor_data_, set_data_pack_format(), set_start_time(), util::xmsg::XMessageIndexable::set_xmsg_index(), util::xmsg::XMessage::set_xmsg_size_bytes(), XMSG_SENS_BAD, util::xmsg::XMessage::xmsg_size_bytes(), util::xmsg::XPHASE_CODE_UNKNOWN, util::xmsg::XRISEFALL_FMT_LEGACY, util::xmsg::XRISEFALL_FMT_STNDRD, util::xmsg::XRISEFALL_FMT_STNDRD_WITH_TEMP_AND_SALN, and util::xmsg::XRISEFALL_FMT_UNKNOWN.

Referenced by util::xmsg::XFall::DecodeXMsgData(), util::xmsg::XRise::DecodeXMsgData(), and util::xmsg::XSurfaceData::DecodeXMsgD

Here is the call graph for this function:



Here is the caller graph for this function:



10.2.3.23 xmsg_index()

```
uint8_t util::xmsg::XMessageIndexable::xmsg_index () const  [inline], [inherited]
```

Get the index of this X-Message.

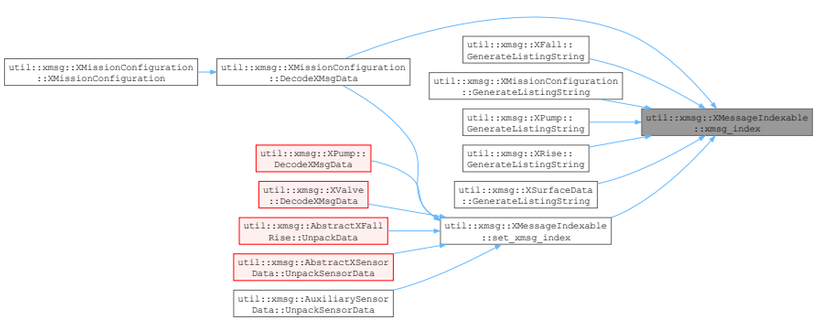
Returns

The index of this X-Message.

References util::xmsg::XMessageIndexable::xmsg_index_.

Referenced by util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing util::xmsg::XSurfaceData::GenerateListingString(), and util::xmsg::XMessageIndexable::set_xmsg_index().

Here is the caller graph for this function:



10.2.3.24 xmsg_size_bytes()

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes () const  [inline], [inherited]
```

Get the size of the X-Message, in bytes.

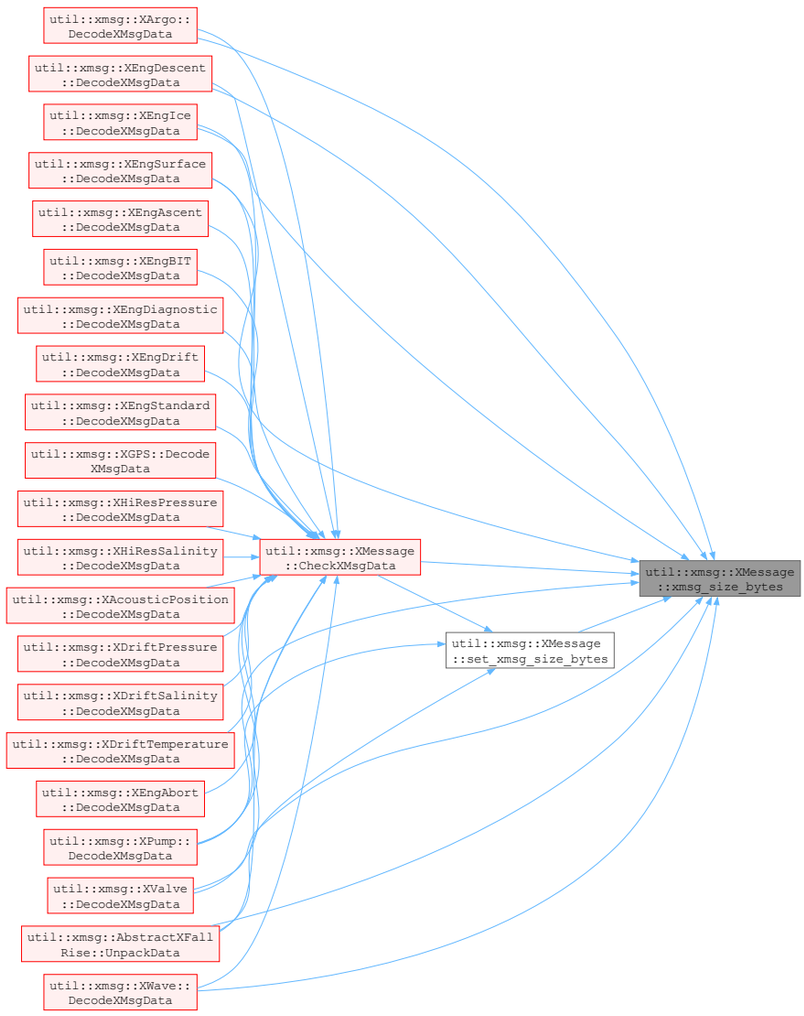
Returns

The size of the X-Message, in bytes.

References util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgI
util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(),
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), util::xmsg::XMessage::set_xmsg_size_bytes(),
and UnpackData().

Here is the caller graph for this function:



10.2.4 Member Data Documentation

10.2.4.1 data_pack_format_

```
XFallRiseDataPackFormat_t util::xmsg::AbstractXFallRise::data_pack_format_  [private]
```

The packing format type identifier.

Referenced by data_pack_format(), and set_data_pack_format().

10.2.4.2 id_dive_

```
int16_t util::xmsg::XMessage::id_dive_  [private], [inherited]
```

The dive or cycle number of this message.

Referenced by util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::set_id_dive().

10.2.4.3 id_instrument_

```
uint16_t util::xmsg::XMessage::id_instrument_  [private], [inherited]
```

The unique WHOI identifier of this instrument.

Referenced by util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::set_id_instrument().

10.2.4.4 id_xmsg_type_

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type_  [private], [inherited]
```

The X-message type identifier.

Referenced by util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::set_id_xmsg_type().

10.2.4.5 sensor_data_

```
vector<XSensorTripletCodeCounts> util::xmsg::AbstractXFallRise::sensor_data_  [private]
```

The raw counts of measured fall, rise, or surface time-series data.

Referenced by num_samples(), sensor_data(), and UnpackData().

10.2.4.6 start_time_

```
ptime util::xmsg::AbstractXFallRise::start_time_  [private]
```

The start time reference.

Referenced by set_start_time(), and start_time().

10.2.4.7 xmsg_index_

```
uint8_t util::xmsg::XMessageIndexable::xmsg_index_  [private], [inherited]
```

The index of this X-Message.

Referenced by util::xmsg::XMessageIndexable::set_xmsg_index(), and util::xmsg::XMessageIndexable::xmsg_index().

10.2.4.8 xmsg_size_bytes_

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes_  [private], [inherited]
```

The size of the X message, in bytes.

Referenced by util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes().

The documentation for this class was generated from the following file:

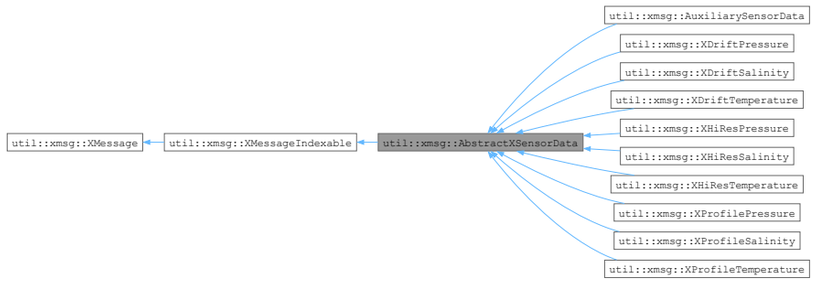
- data/xmsg/AbstractXFallRise.h

10.3 util::xmsg::AbstractXSensorData Class Reference

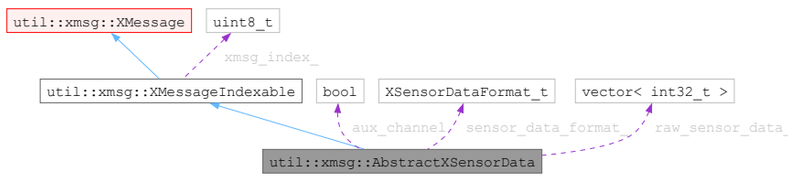
The abstract sensor data class.

`#include <AbstractXSensorData.h>`

Inheritance diagram for util::xmsg::AbstractXSensorData:



Collaboration diagram for util::xmsg::AbstractXSensorData:



Public Member Functions

- AbstractXSensorData (int16_t dive, uint16_t whoi, const vector< uint8_t > &data)

  Parameterized constructor.
- virtual ~AbstractXSensorData ()

  Destructor.
- const uint8_t * CheckXMsgData (const vector< uint8_t > &data, int id, int minBytes, bool parseSize=true)

  Check the X-message for the correct ID, minimum size, and message terminator.
- virtual void DecodeXMsgData (const vector< uint8_t > &data)=0

  Decode the X message.
- int16_t id_dive () const

  Get the dive or cycle number of this X-Message.
- uint16_t id_instrument () const

  Get the unique instrument identifier of this X-Message.
- XMsgTypeId_t id_xmsg_type () const

  Get the enumerated X-Message type identifier.
- bool **is_aux_channel** () const
- size_t num_samples () const

  Get the number of samples in this message.
- const vector< int32_t > & raw_sensor_data () const

  Get the raw sensor data vector.
- int32_t Sample (size_t i) const

  Get the raw integer value of the i^{th} sample.
- template<typename T = double>
  T SampleValue (size_t i, int16_t gain, int16_t offs) const

  Get the i^{th} sensor value in scientific units.
- template<typename T = double>
  T SampleValueAsPressure (size_t i, int16_t gain, int16_t offs) const

  Get the i^{th} pressure sensor value, in dBar.
- template<typename T = double>
  T SampleValueAsSalinity (size_t i, int16_t gain, int16_t offs) const

  Get the i^{th} salinity sensor value, in PSU.

- template<typename T = double>
  T SampleValueAsTemperature (size_t i, int16_t gain, int16_t offs) const

  Get the i^{th} temperature sensor value, in degC.
- XSensorDataFormat_t sensor_data_format () const

  Get the enumerated sensor data format value.
- uint8_t xmsg_index () const

  Get the index of this X-Message.
- uint16_t xmsg_size_bytes () const

  Get the size of the X-Message, in bytes.

Protected Member Functions

- AbstractXSensorData ()

  Default constructor.
- void set_id_dive (int16_t id_dive)

  Set the dive or cycle number of this message.
- void set_id_instrument (uint16_t id_instrument)

  Set the unique instrument identifier.
- void set_id_xmsg_type (XMsgTypeId_t id_xmsg_type)

  Set the X-Message type identifier.
- void set_raw_sensor_data (const vector< int32_t > &raw_sensor_data)

  Set raw integer counts of the measured sensor data.
- void set_sensor_data_format (XSensorDataFormat_t sensor_data_format)

  Set the sensor data format.
- void set_xmsg_index (uint8_t xmsg_index)

  Set the index of this X-Message.
- void set_xmsg_size_bytes (uint16_t xmsg_size_bytes)

  Set the size of the X-Message, in bytes.
- virtual vector< int32_t > UnpackSensorData (const vector< uint8_t > &data)

  Unpack the sensor data from the binary message.

Protected Attributes

- bool **aux_channel**
- vector< int32_t > raw_sensor_data_

  The raw integer counts of the measured sensor data.
- XSensorDataFormat_t sensor_data_format_

  The enumerated sensor data packing format.

Private Attributes

- int16_t id_dive_

  The dive or cycle number of this message.
- uint16_t id_instrument_

  The unique WHOI identifier of this instrument.
- XMsgTypeId_t id_xmsg_type_

  The X-message type identifier.
- uint8_t xmsg_index_

  The index of this X-Message.
- uint16_t xmsg_size_bytes_

  The size of the X message, in bytes.

10.3.1 Detailed Description

The abstract sensor data class.

The abstract sensor data class provides the functionality to decode sensor data messages using either first-difference or curvature encoding. It is used to decode and represent the sensor data returned from the instrument that is contained within a single message.

The classes that inherit from this class represent profile or drift data taken from a single sensor e.g. pressure, temperature, or salinity. The sensor data is stored in a vector of raw integer counts, raw_sensor_data_. The data from the sensor is recovered in scientific units by applying the appropriate offset and gain corrections, as specified in the Argo data message.

10.3.2 Constructor & Destructor Documentation

10.3.2.1 AbstractXSensorData() [1/2]

```
util::xmsg::AbstractXSensorData::AbstractXSensorData (
            int16_t dive,
            uint16_t whoi,
            const vector< uint8_t > & data)  [inline]
```

Parameterized constructor.

Parameters

in	*dive*	The dive or cycle number of this message.
in	*whoi*	The unique WHOI identifier.
in	*data*	Binary X-message data to decode.

Exceptions

util::log::MessageException	if • an invalid X-Message type identifier is encountered. • the message is of insufficient length, • an invalid packing format is encountered, or • the message terminator is in an invalid location.

10.3.2.2 ~AbstractXSensorData()

```
virtual util::xmsg::AbstractXSensorData::~AbstractXSensorData ()  [inline], [virtual]
```

Destructor.

10.3.2.3 AbstractXSensorData() [2/2]

```
util::xmsg::AbstractXSensorData::AbstractXSensorData ()  [inline], [protected]
```

Default constructor.

Constructs an empty sensor data message with an unknown sensor data format.

10.3.3 Member Function Documentation

10.3.3.1 CheckXMsgData()

```
const uint8_t * util::xmsg::XMessage::CheckXMsgData (
            const vector< uint8_t > & data,
            int id,
            int minBytes,
            bool parseSize = true)  [inline], [inherited]
```

Check the X-message for the correct ID, minimum size, and message terminator.

Parameters

in	*data*	The X-message binary data to check.
in	*id*	The expected ID of the X-message. If this field is less that 0, then the ID check will be skipped.
in	*minBytes*	The minimum number of bytes expected in a valid message.
in	*parseSize*	If TRUE, then the message size field will be parsed from the X-Message. Otherwise, the size field will not be parsed.

Exceptions

util::log::MessageException	if • the message does not contain the expected ID, • the message does not contain the minimum number of expected bytes, • the message does not contain a terminator (';') in the correct location, or, • any other error occurs.

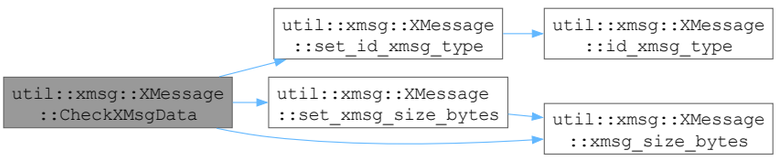
Returns

A pointer to the underlying data buffer of data.

References CATCH_BLOCK, EXC_THROW, lwrn, util::xmsg::XMessage::set_id_xmsg_type(), util::xmsg::XMessage::set_xmsg_size_byte u16, and util::xmsg::XMessage::xmsg_size_bytes().

Referenced by util::xmsg::XAcousticPosition::DecodeXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XDriftPressure::Dec util::xmsg::XDriftSalinity::DecodeXMsgData(), util::xmsg::XDriftTemperature::DecodeXMsgData(), util::xmsg::XEngAbort::DecodeXMsgDa util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngBIT::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(), util::xmsg::XEngDiagnostic::DecodeXMsgData(), util::xmsg::XEngDrift::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngStandard::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XHiResPressure::DecodeXMsgData(), util::xmsg::XHiResSalinity::DecodeXMsgData(), util::xmsg::XHiResTemperature::Decod util::xmsg::XMissionCmdEcho::DecodeXMsgData(), util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XProfilePressure: util::xmsg::XProfileSalinity::DecodeXMsgData(), util::xmsg::XProfileTemperature::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgD util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the call graph for this function:



10.3.3.2 DecodeXMsgData()

```
virtual void util::xmsg::AbstractXSensorData::DecodeXMsgData (
            const vector< uint8_t > & data)  [pure virtual]
```

Decode the X message.

Parameters

in	*data*	The encoded X message to decode.

Implements util::xmsg::XMessageIndexable.

Implemented in util::xmsg::XDriftPressure, util::xmsg::XDriftSalinity, util::xmsg::XDriftTemperature, util::xmsg::XHiResPressure, util::xmsg::XHiResSalinity, util::xmsg::XHiResTemperature, util::xmsg::XProfilePressure, util::xmsg::XProfileSalinity, and util::xmsg::XProfileTemperature.

10.3.3.3 id_dive()

```
int16_t util::xmsg::XMessage::id_dive () const  [inline], [inherited]
```

Get the dive or cycle number of this X-Message.

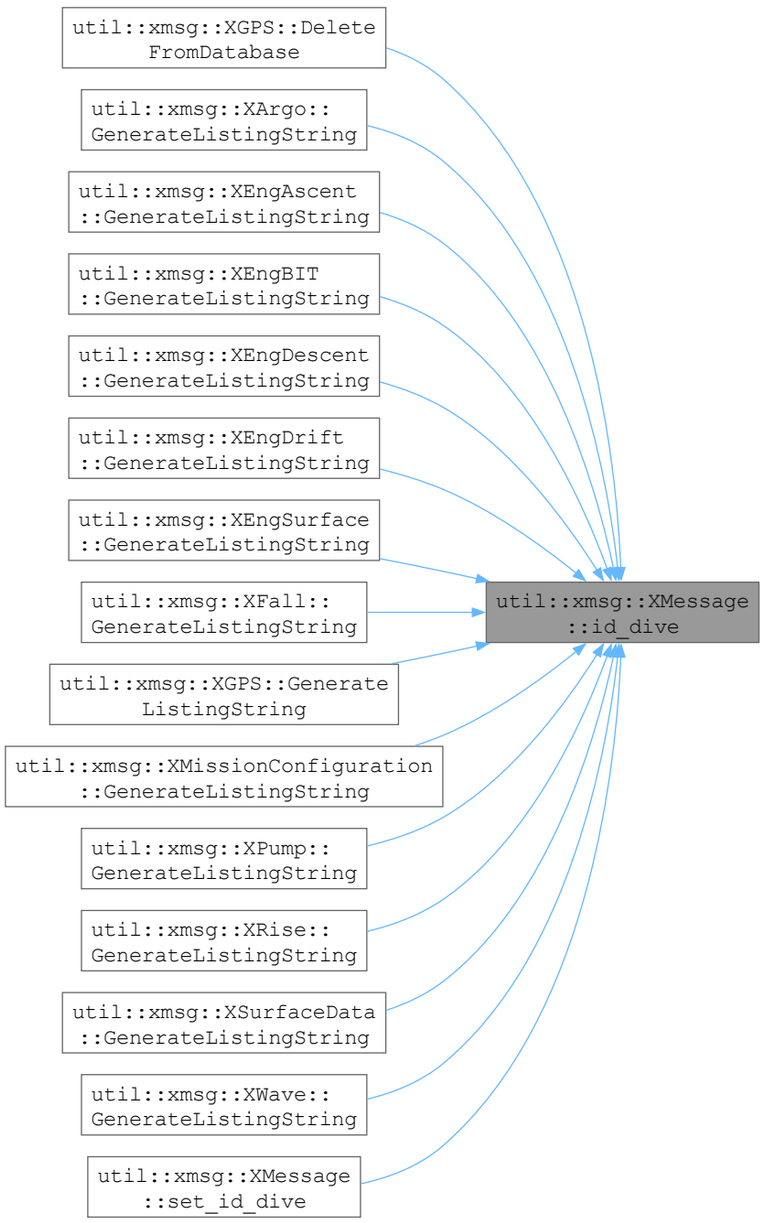
Returns

The dive or cycle number of this X-Message.

References util::xmsg::XMessage::id_dive_.

Referenced by util::xmsg::XGPS::DeleteFromDatabase(), util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateLis
util::xmsg::XEngBIT::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingStr
util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(),
util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing
util::xmsg::XSurfaceData::GenerateListingString(), util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_dive().

Here is the caller graph for this function:



10.3.3.4 id_instrument()

```
uint16_t util::xmsg::XMessage::id_instrument () const  [inline], [inherited]
```

Get the unique instrument identifier of this X-Message.

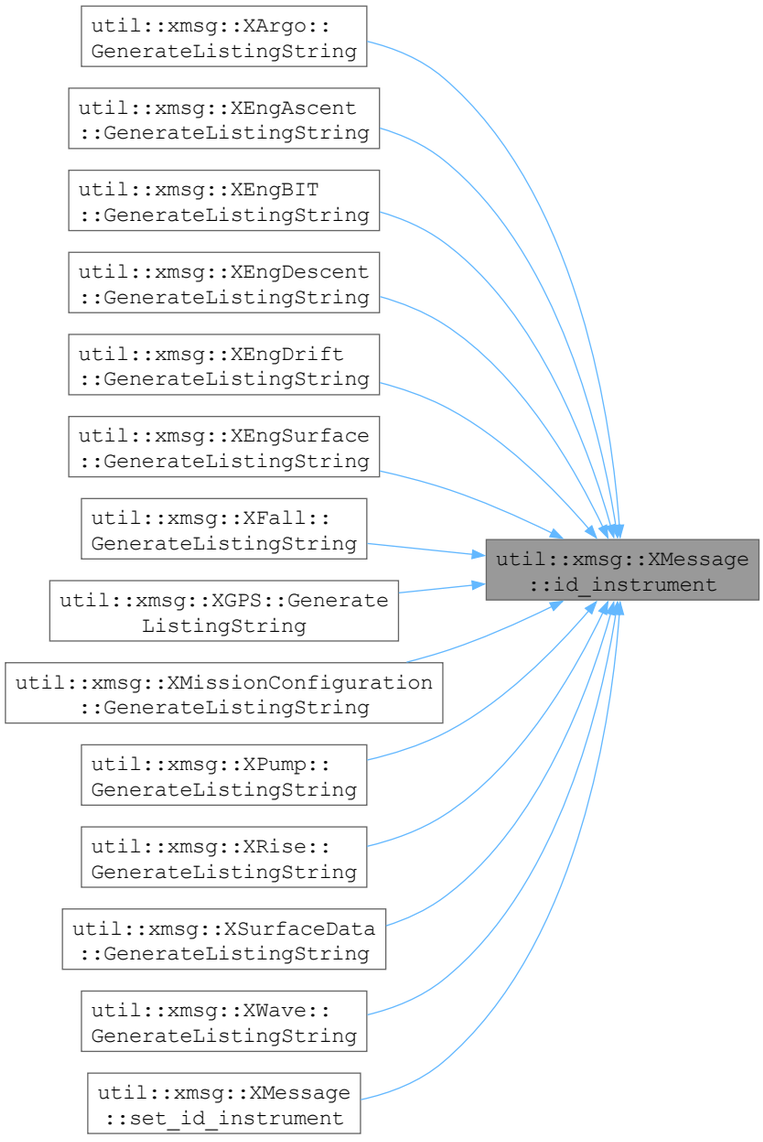
Returns

The unique instrument identifier of this X-Message.

References util::xmsg::XMessage::id_instrument_.

Referenced by util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngBIT::Generate
util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), util::xmsg::XEngSurface::GenerateListin
util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingS
util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(),
util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_instrument().

Here is the caller graph for this function:



10.3.3.5 id_xmsg_type()

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type () const  [inline], [inherited]
```

Get the enumerated X-Message type identifier.

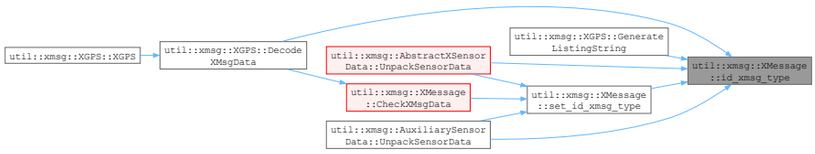
Returns

The enumerated X-Message type identifier.

References util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMessage::set_id_xmsg_typ
UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the caller graph for this function:



10.3.3.6 num_samples()

```
size_t util::xmsg::AbstractXSensorData::num_samples () const  [inline]
```

Get the number of samples in this message.

Returns

The number of samples in this message.

References raw_sensor_data_.

10.3.3.7 raw_sensor_data()

```
const vector< int32_t > & util::xmsg::AbstractXSensorData::raw_sensor_data () const  [inline]
```

Get the raw sensor data vector.

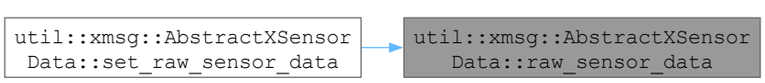
Returns

The raw sensor data vector.

References raw_sensor_data_.

Referenced by set_raw_sensor_data().

Here is the caller graph for this function:



10.3.3.8 Sample()

```
int32_t util::xmsg::AbstractXSensorData::Sample (
            size_t i) const  [inline]
```

Get the raw integer value of the i^{th} sample.

Parameters

in	*i*	The index of the sensor value.

Returns

The raw integer value of the i^{th} ith sample.

References raw_sensor_data_.

10.3.3.9 SampleValue()

```
template<typename T = double>
T util::xmsg::AbstractXSensorData::SampleValue (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline]
```

Get the i^{th} sensor value in scientific units.

Parameters

in	*i*	The index of the sensor value.
in	*gain*	The sensor sensitivity scale factor.
in	*offs*	The sensor offset.

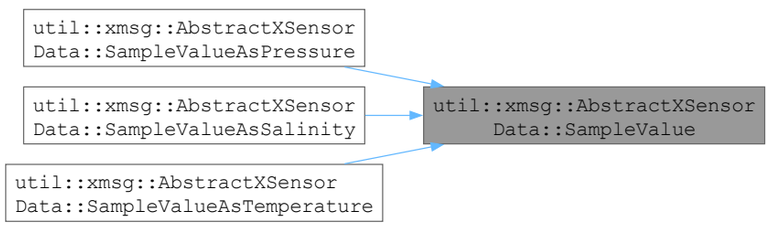
Returns

The i^{th} sensor value in scientific units.

References CATCH_BLOCK, and raw_sensor_data_.

Referenced by SampleValueAsPressure(), SampleValueAsSalinity(), and SampleValueAsTemperature().

Here is the caller graph for this function:



10.3.3.10 SampleValueAsPressure()

```
template<typename T = double>
T util::xmsg::AbstractXSensorData::SampleValueAsPressure (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline]
```

Get the i^{th} pressure sensor value, in dBar.

Parameters

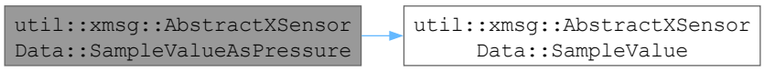
in	*i*	The index of the sensor value.
in	*gain*	The pressure sensor sensitivity scale factor, in LSB/dBar.
in	*offs*	The pressure sensor offset, in dBar.

Returns

The i^{th} pressure sensor value, in dBar.

References SampleValue(), XMSG_SENS_BAD_PRES, and XMSG_SENS_MAX_PRES.

Here is the call graph for this function:



10.3.3.11 SampleValueAsSalinity()

```
template<typename T = double>
T util::xmsg::AbstractXSensorData::SampleValueAsSalinity (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline]
```

Get the i^{th} salinity sensor value, in PSU.

Parameters

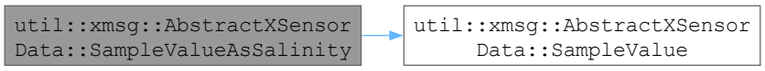
in	*i*	The index of the sensor value.
in	*gain*	The salinity sensor sensitivity scale factor, in LSB/PSU.
in	*offs*	The salinity sensor offset, in PSU.

Returns

The i^{th} salinity sensor value, in PSU.

References SampleValue(), XMSG_SENS_BAD_SALN, and XMSG_SENS_MAX_SALN.

Here is the call graph for this function:



10.3.3.12 SampleValueAsTemperature()

```
template<typename T = double>
T util::xmsg::AbstractXSensorData::SampleValueAsTemperature (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline]
```

Get the i^{th} temperature sensor value, in degC.

Parameters

in	*i*	The index of the sensor value.
in	*gain*	The temperature sensor sensitivity scale factor, in LSB/degC.
in	*offs*	The temperature sensor offset, in degC.

Returns

The i^{th} temperature sensor value, in degC.

References SampleValue(), XMSG_SENS_BAD_TEMP, and XMSG_SENS_MAX_TEMP.

Here is the call graph for this function:

10.3.3.13 sensor_data_format()

```
XSensorDataFormat_t util::xmsg::AbstractXSensorData::sensor_data_format () const  [inline]
```

Get the enumerated sensor data format value.

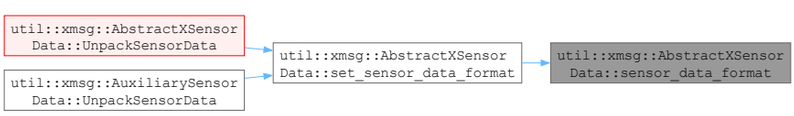
Returns

The enumerated sensor data format value.

References sensor_data_format_.

Referenced by set_sensor_data_format().

Here is the caller graph for this function:



10.3.3.14 set_id_dive()

```
void util::xmsg::XMessage::set_id_dive (
            int16_t id_dive)  [inline], [protected], [inherited]
```

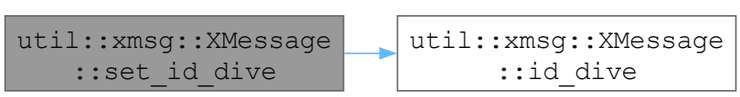
Set the dive or cycle number of this message.

Parameters

in	*id_dive*	The dive or cycle number of this message to set.

References util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::id_dive_.

Here is the call graph for this function:



10.3.3.15 set_id_instrument()

```
void util::xmsg::XMessage::set_id_instrument (
            uint16_t id_instrument)  [inline], [protected], [inherited]
```

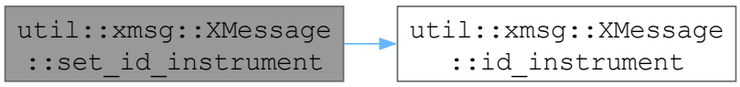
Set the unique instrument identifier.

Parameters

in	*id_instrument*	The unique instrument identifier to set.

References util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::id_instrument_.

Here is the call graph for this function:



10.3.3.16 set_id_xmsg_type()

```
void util::xmsg::XMessage::set_id_xmsg_type (
            XMsgTypeId_t id_xmsg_type)  [inline], [protected], [inherited]
```

Set the X-Message type identifier.

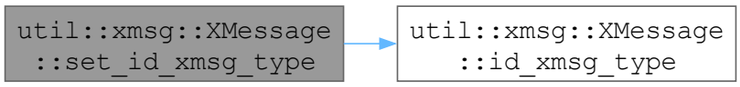
Parameters

in	*id_xmsg_type*	The X-Message type identifier to set.

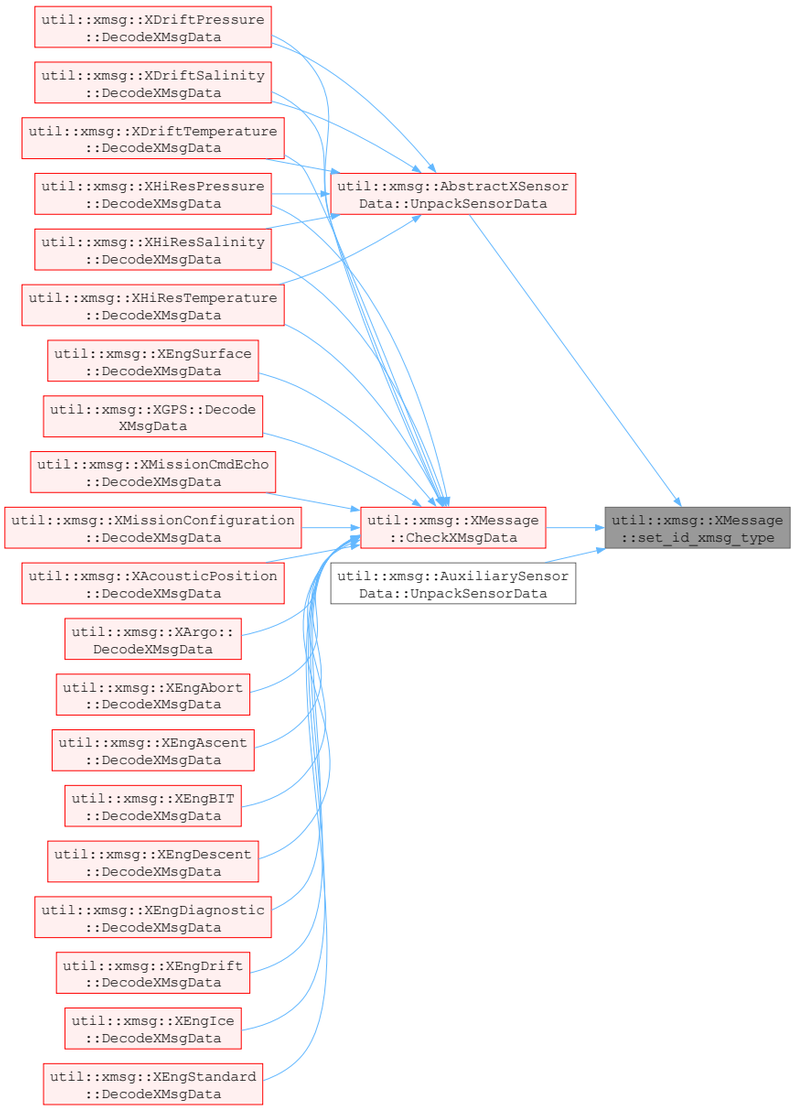
References util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData(

Here is the call graph for this function:



Here is the caller graph for this function:



10.3.3.17 set_raw_sensor_data()

```
void util::xmsg::AbstractXSensorData::set_raw_sensor_data (
            const vector< int32_t > & raw_sensor_data)  [inline], [protected]
```

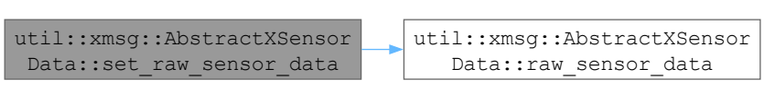
Set raw integer counts of the measured sensor data.

Parameters

in	*raw_sensor_data*	The data to set.

References raw_sensor_data(), and raw_sensor_data_.

Here is the call graph for this function:



10.3.3.18 set_sensor_data_format()

```
void util::xmsg::AbstractXSensorData::set_sensor_data_format (
            XSensorDataFormat_t sensor_data_format)  [inline], [protected]
```

Set the sensor data format.

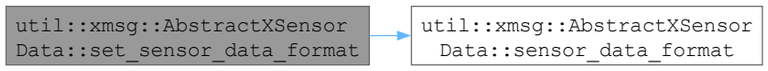
Parameters

in	*sensor_data_format*	The sensor data format to set.

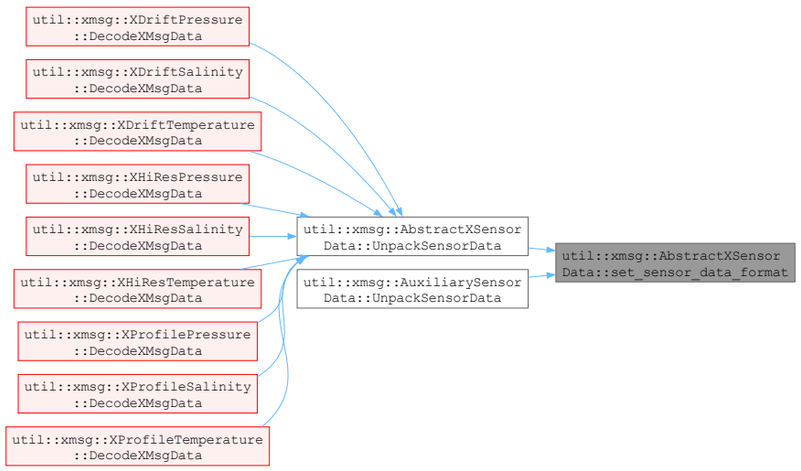
References sensor_data_format(), and sensor_data_format_.

Referenced by UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.3.3.19 set_xmsg_index()

```
void util::xmsg::XMessageIndexable::set_xmsg_index (
            uint8_t xmsg_index)  [inline], [protected], [inherited]
```

Set the index of this X-Message.

Parameters

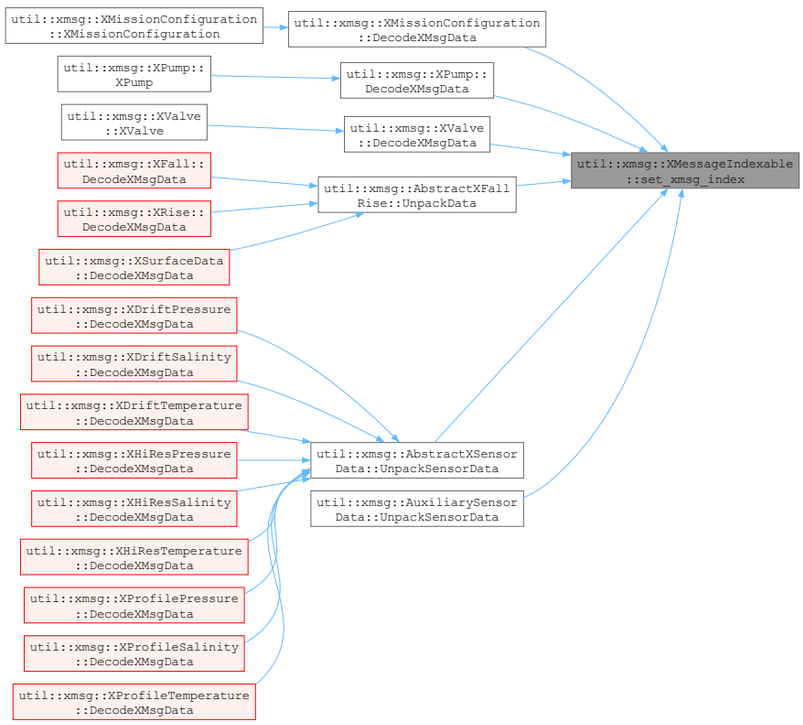
in	*xmsg_index*	The index of this X-Message.

References util::xmsg::XMessageIndexable::xmsg_index(), and util::xmsg::XMessageIndexable::xmsg_index_.

Referenced by util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(), util::xmsg::XValve::DecodeXMsgData(), util::xmsg::AbstractXFallRise::UnpackData(), UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:

Here is the caller graph for this function:



10.3.3.20 set_xmsg_size_bytes()

```
void util::xmsg::XMessage::set_xmsg_size_bytes (
            uint16_t xmsg_size_bytes)  [inline], [protected], [inherited]
```

Set the size of the X-Message, in bytes.

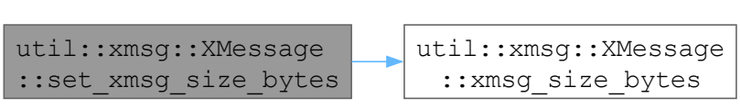
Parameters

in	*xmsg_size_bytes*	The size of the X-Message to set, in bytes.

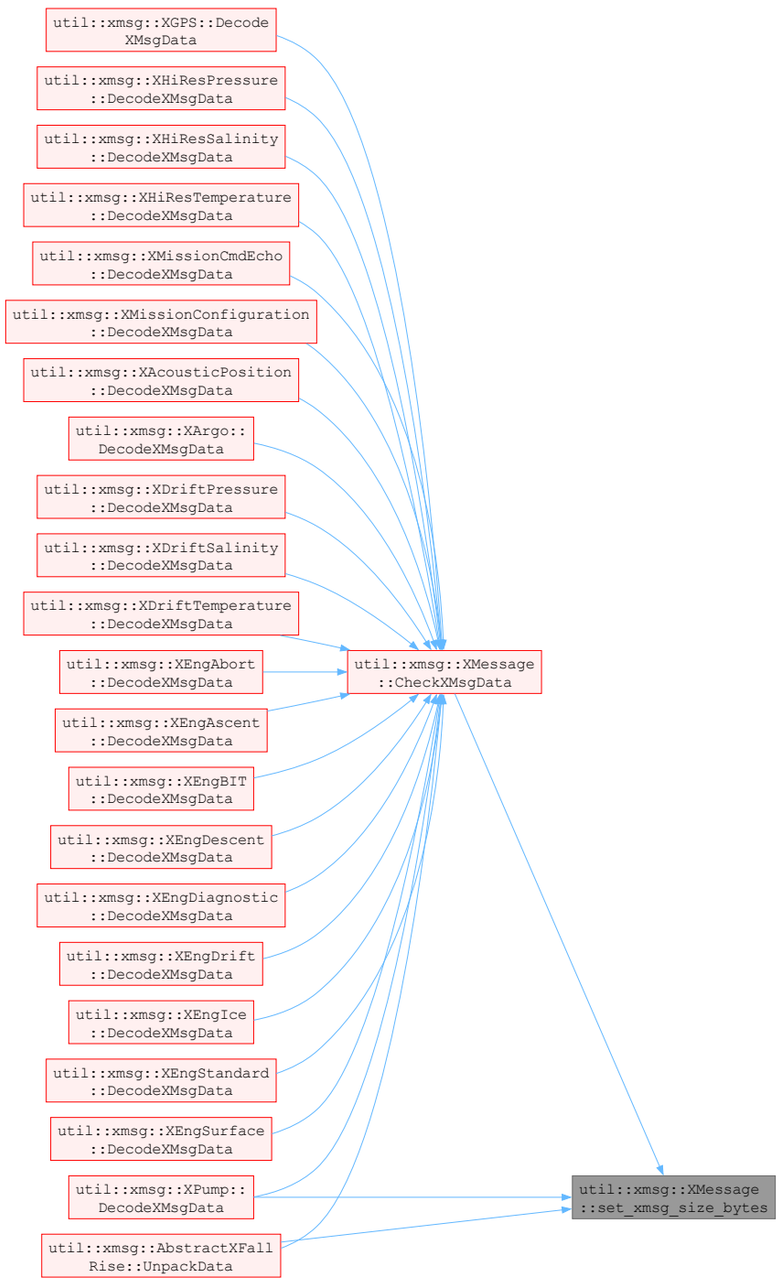
References util::xmsg::XMessage::xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XPump::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::Unpa

Here is the call graph for this function:



Here is the caller graph for this function:



10.3.3.21 UnpackSensorData()

```
virtual vector< int32_t > util::xmsg::AbstractXSensorData::UnpackSensorData (
            const vector< uint8_t > & data)  [inline], [protected], [virtual]
```

Unpack the sensor data from the binary message.

This field unpacks the encoded sensor data from the message and stores the decoded sensor values in raw_sensor_data_. First the function checks the size of the message, and the data packing format. The message data is decoded according to the packing format type. Both first-difference and curvature encoding are supported. The encoding method is determined from the value of the high nibble of byte `data`[1].

Once this function completes successfully, then the data may be recovered in scientific units from the raw sensor values by applying the appropriate sensor gain and offset values.

Parameters

in	*data*	The binary message to unpack, in a vector of unsigned bytes.

Warning

The function assumes the message data is valid. Message validity should be ensured before this function is called.

Exceptions

util::log::MessageException	if • an error occurs while decoding the message, • the message is of insufficient length, or • an invalid packing format is encountered.

Returns

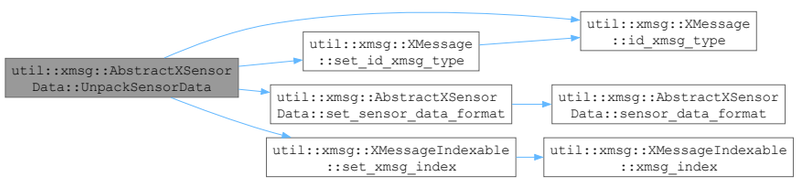
The sensor data raw counts, in a vector of signed integers.

Reimplemented in util::xmsg::AuxiliarySensorData.

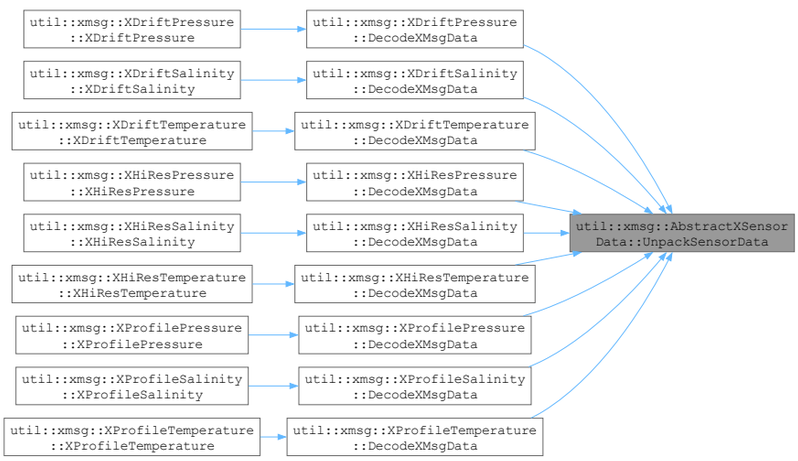
References CATCH_BLOCK, CATCH_NOTHROW, EXC_THROW, i24, util::xmsg::XMessage::id_xmsg_type(), ldbg, lwrn, raw_sensor_data_, util::xmsg::XMessage::set_id_xmsg_type(), set_sensor_data_format(), util::xmsg::XMessageIndexable::set_xms u16, u24, XMSG_SENS_MAX_SUB, util::xmsg::XMSG_TYPE_ID_PRES0, util::xmsg::XMSG_TYPE_ID_PRES_DRIFT8, util::xmsg::XMSG_TYPE_ID_PRES_HI_RES0, util::xmsg::XMSG_TYPE_ID_SALN0, util::xmsg::XMSG_TYPE_ID_SALN_DRIFT8, util::xmsg::XMSG_TYPE_ID_SALN_HI_RES0, util::xmsg::XMSG_TYPE_ID_TEMP0, util::xmsg::XMSG_TYPE_ID_TEMP_DRIFT8, util::xmsg::XMSG_TYPE_ID_TEMP_HI_RES0, util::xmsg::XSENS_FMT_CURVATURE, and util::xmsg::XSENS_FMT_DIFFERENCE.

Referenced by util::xmsg::XDriftPressure::DecodeXMsgData(), util::xmsg::XDriftSalinity::DecodeXMsgData(), util::xmsg::XDriftTemperatur util::xmsg::XHiResPressure::DecodeXMsgData(), util::xmsg::XHiResSalinity::DecodeXMsgData(), util::xmsg::XHiResTemperature::Decod util::xmsg::XProfilePressure::DecodeXMsgData(), util::xmsg::XProfileSalinity::DecodeXMsgData(), and util::xmsg::XProfileTemperature::D

Here is the call graph for this function:



Here is the caller graph for this function:



10.3.3.22 xmsg_index()

```
uint8_t util::xmsg::XMessageIndexable::xmsg_index () const  [inline], [inherited]
```

Get the index of this X-Message.

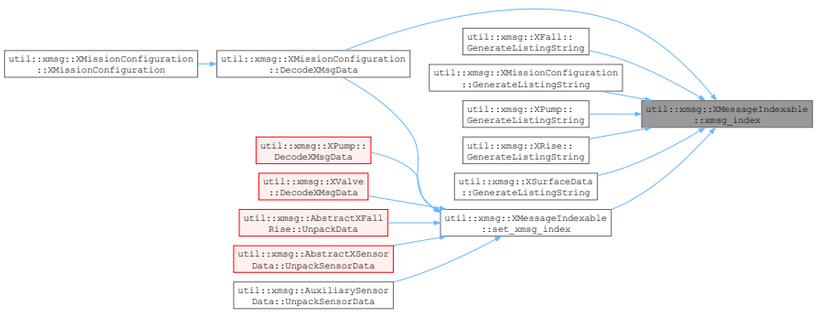
Returns

The index of this X-Message.

References util::xmsg::XMessageIndexable::xmsg_index_.

Referenced by util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing util::xmsg::XSurfaceData::GenerateListingString(), and util::xmsg::XMessageIndexable::set_xmsg_index().

Here is the caller graph for this function:



10.3.3.23 xmsg_size_bytes()

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes () const  [inline], [inherited]
```

Get the size of the X-Message, in bytes.

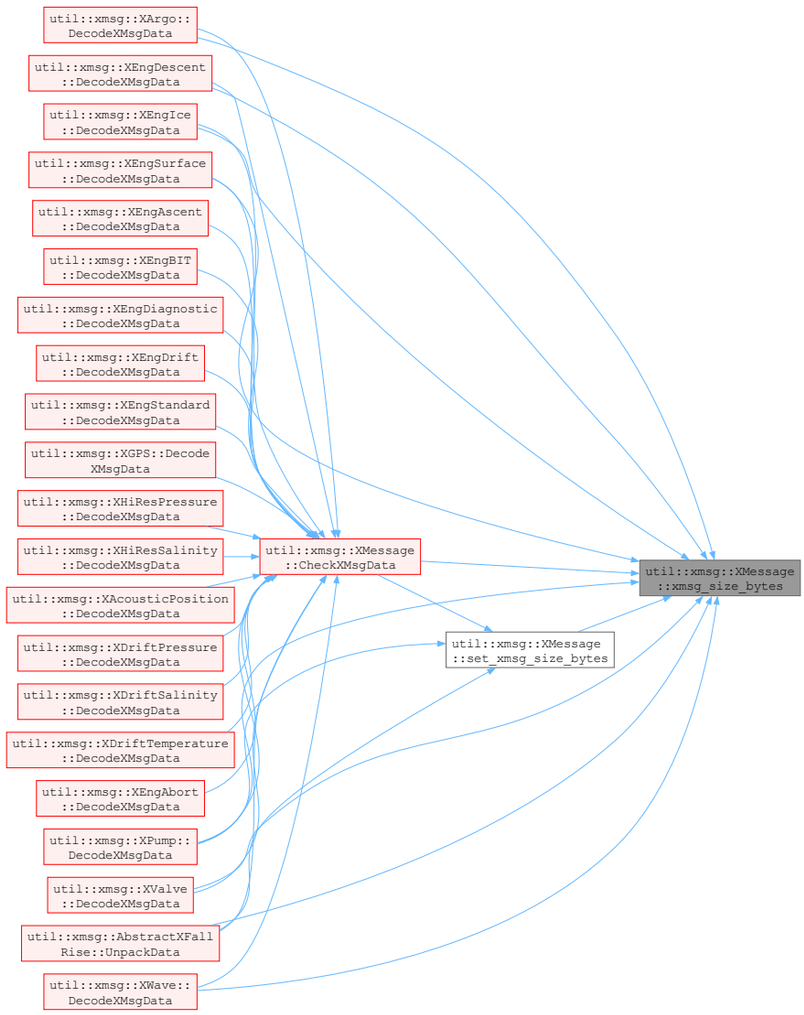
Returns

The size of the X-Message, in bytes.

References util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgI util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(), util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the caller graph for this function:



10.3.4 Member Data Documentation

10.3.4.1 id_dive_

```
int16_t util::xmsg::XMessage::id_dive_  [private], [inherited]
```

The dive or cycle number of this message.

Referenced by util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::set_id_dive().

10.3.4.2 id_instrument_

```
uint16_t util::xmsg::XMessage::id_instrument_  [private], [inherited]
```

The unique WHOI identifier of this instrument.

Referenced by util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::set_id_instrument().

10.3.4.3 id_xmsg_type_

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type_  [private], [inherited]
```

The X-message type identifier.

Referenced by util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::set_id_xmsg_type().

10.3.4.4 raw_sensor_data_

```
vector<int32_t> util::xmsg::AbstractXSensorData::raw_sensor_data_  [protected]
```

The raw integer counts of the measured sensor data.

This vector of integers represents the "raw" sample values of the message. That is, by applying the appropriate gain and offset corrections, the sensor values may be recovered in scientific units. This data may represent pressure, temperature, salinity, or any other data that is recorded during the drift or profile phases of the dive cycle.

Referenced by num_samples(), raw_sensor_data(), Sample(), SampleValue(), set_raw_sensor_data(), UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

10.3.4.5 sensor_data_format_

```
XSensorDataFormat_t util::xmsg::AbstractXSensorData::sensor_data_format_  [protected]
```

The enumerated sensor data packing format.

This field indicates the format of encoded sensor data. The manner in which the sensor data is decoded is determined by the format of this field.

Referenced by sensor_data_format(), and set_sensor_data_format().

10.3.4.6 xmsg_index_

```
uint8_t util::xmsg::XMessageIndexable::xmsg_index_  [private], [inherited]
```

The index of this X-Message.

Referenced by util::xmsg::XMessageIndexable::set_xmsg_index(), and util::xmsg::XMessageIndexable::xmsg_index().

10.3.4.7 xmsg_size_bytes_

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes_  [private], [inherited]
```

The size of the X message, in bytes.

Referenced by util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes().

The documentation for this class was generated from the following file:

- data/xmsg/AbstractXSensorData.h

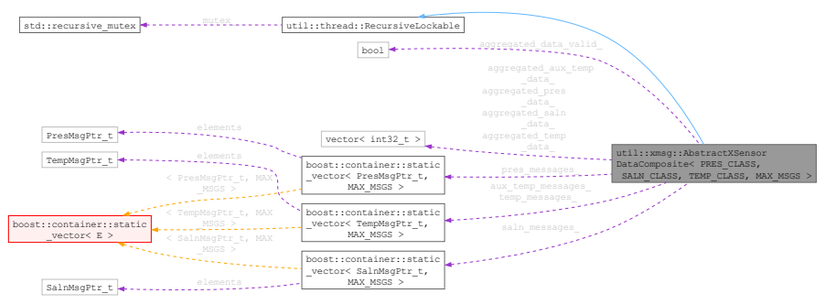
10.4 util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS > Class Template Reference

The AbstractXSensorDataComposite class.

`#include <AbstractXSensorDataComposite.h>`

Inheritance diagram for util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >:

Collaboration diagram for util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >:



Public Member Functions

- AbstractXSensorDataComposite ()

  The constructor.
- virtual ~AbstractXSensorDataComposite ()

  The destructor.
- void AssignAuxTemperatureMessage (const TempMsgPtr_t &x)

  Assign an auxiliary-channel temperature message and re-aggregate the sensor data.
- void AssignPressureMessage (const PresMsgPtr_t &x)

  Assign a pressure data message to this composite data object.
- void AssignSalinityMessage (const SalnMsgPtr_t &x)

  Assign a salinity data message to this composite data object.
- void AssignTemperatureMessage (const TempMsgPtr_t &x)

  Assign a temperature data message to this composite data object.
- template<typename T = double>
  void **CopyPressureDataToArray** (T dest[], size_t max_len, int16_t gain, int16_t offs, float pres_correction, T fill_value=XMSG_SENS_BAD_PRES) const
- template<typename T = double>
  void **CopySalinityDataToArray** (T dest[], size_t max_len, int16_t gain, int16_t offs, T fill_value=XMSG_SENS_BAD_SALN) const
- template<typename T = double>
  void **CopyTemperatureDataToArray** (T dest[], size_t max_len, int16_t gain, int16_t offs, T fill_↩ value=XMSG_SENS_BAD_TEMP) const
- virtual const string GenerateListingString (int id_inst, int id_dive, uint16_t pg, uint16_t po, uint16_t sg, uint16_t so, uint16_t tg, uint16_t to, float pc) const =0

  Generate a listing string representation of the this composite sensor data object.
- bool **HasAuxTempData** () const
- bool HasContiguousMessageIndices () const

  Do the sensor data messages assigned to this composite data object have contiguous message indices?
- bool is_aggregated_data_valid () const

Is the aggregated data valid?

- bool IsDataComplete () const

  Is the sensor data for this composite date message complete?

- bool IsEmpty () const

  Is this composite data object empty?

- bool LockObject (bool block)

  Lock the object.

- size_t max_num_samples () const

  Return the maximum of the number of pressure, temperature, or salinity sensor samples in the sensor data messages assigned to this composite sensor data object.

- size_t **num_aux_temp_samples** () const
- size_t num_pres_samples () const

  Get the total number of pressure sensor samples contained in the pressure data messages assigned to this composite data object.

- size_t num_saln_samples () const

  Get the total number of salinity sensor samples contained in the salinity data messages assigned to this composite data object.

- size_t num_temp_samples () const

  Get the total number of temperature sensor samples contained in the temperature data messages assigned to this composite data object.

- void Reset ()

  Remove the assigned pressure, temperature and salinity messages and their aggregated data.

- template<typename T = double>
  T **SampleAuxTempAsFloat** (size_t i, int16_t gain, int16_t offs) const
- int32_t SamplePres (size_t i) const

  Get the i^{th} pressure sensor value, in raw counts.

- template<typename T = double>
  T SamplePresAsFloat (size_t i, int16_t pg, int16_t po, float pc) const

  Get the i^{th} pressure sensor value, in dBar.

- int32_t SampleSaln (size_t i) const

  Get the i^{th} salinity sensor value, in raw counts.

- template<typename T = double>
  T SampleSalnAsFloat (size_t i, int16_t gain, int16_t offs) const

  Get the i^{th} salinity sensor value, in PSU.

- int32_t SampleTemp (size_t i) const

  Get the i^{th} temperature sensor value, in raw counts.

- template<typename T = double>
  T SampleTempAsFloat (size_t i, int16_t gain, int16_t offs) const

  Get the i^{th} temperature sensor value, in degC.

- void UnlockObject ()

  Unlock the object.

Protected Member Functions

- void AggregateSensorData () ATTR_NO_EXCEPTIONS

  Aggregate the sensor data for this composite sensor data object.

- void set_aggregated_data_valid (bool valid)

  Set the value of the aggregated_data_valid_ field.

Protected Attributes

- recursive_mutex mutex_

  Recursive mutual exclusion lock for synchronizing operations.

Private Types

- typedef shared_ptr< PRES_CLASS > PresMsgPtr_t

  Type definition for a shared pointer to the pressure data message.

- typedef static_vector< PresMsgPtr_t, MAX_MSGS > PresMsgVector_t

  Type definition for a static vector (of size MAX_MSGS) of shared pointers to the pressure data message.

- typedef shared_ptr< SALN_CLASS > SalnMsgPtr_t

  Type definition for a shared pointer to the salinity data message.

- typedef static_vector< SalnMsgPtr_t, MAX_MSGS > SalnMsgVector_t

  Type definition for a static vector (of size MAX_MSGS) of shared pointers to the salinity data message.

- typedef shared_ptr< TEMP_CLASS > TempMsgPtr_t

  Type definition for a shared pointer to the temperature data message.

- typedef static_vector< TempMsgPtr_t, MAX_MSGS > TempMsgVector_t

  Type definition for a static vector (of size MAX_MSGS) of shared pointers to the temperature data message.

Private Attributes

- vector< int32_t > **aggregated_aux_temp_data_**
- bool aggregated_data_valid_

  The aggregated data is valid flag.
- vector< int32_t > aggregated_pres_data_

  A vector of the aggregated pressure data samples, in raw counts.
- vector< int32_t > aggregated_saln_data_

  A vector of the aggregated salinity data samples, in raw counts.
- vector< int32_t > aggregated_temp_data_

  A vector of the aggregated temperature data samples, in raw counts.
- TempMsgVector_t **aux_temp_messages_**
- PresMsgVector_t pres_messages_

  A static vector (of size MAX_MSGS) containing pointers the pressure sensor data messages assigned to this composite object.
- SalnMsgVector_t saln_messages_

  A static vector (of size MAX_MSGS) containing pointers the salinity sensor data messages assigned to this composite object.
- TempMsgVector_t temp_messages_

  A static vector (of size MAX_MSGS) containing pointers the temperature sensor data messages assigned to this composite object.

10.4.1 Detailed Description

template<class PRES_CLASS, class SALN_CLASS, class TEMP_CLASS, unsigned int MAX_MSGS>
class util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >

The AbstractXSensorDataComposite class.

This abstract, template class represents the composite sensor data for a single dive cycle. It provided the functionality for aggregating sensor data messages into a single object.

10.4.2 Member Typedef Documentation

10.4.2.1 PresMsgPtr_t

```
template<class PRES_CLASS , class SALN_CLASS , class TEMP_CLASS , unsigned int MAX_MSGS>
typedef shared_ptr<PRES_CLASS> util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS,
TEMP_CLASS, MAX_MSGS >::PresMsgPtr_t  [private]
```

Type definition for a shared pointer to the pressure data message.

10.4.2.2 PresMsgVector_t

```
template<class PRES_CLASS , class SALN_CLASS , class TEMP_CLASS , unsigned int MAX_MSGS>
typedef static_vector<PresMsgPtr_t, MAX_MSGS> util::xmsg::AbstractXSensorDataComposite< PRES_↩
CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::PresMsgVector_t  [private]
```

Type definition for a static vector (of size MAX_MSGS) of shared pointers to the pressure data message.

10.4.2.3 SalnMsgPtr_t

```
template<class PRES_CLASS , class SALN_CLASS , class TEMP_CLASS , unsigned int MAX_MSGS>
typedef shared_ptr<SALN_CLASS> util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS,
TEMP_CLASS, MAX_MSGS >::SalnMsgPtr_t  [private]
```

Type definition for a shared pointer to the salinity data message.

10.4.2.4 SalnMsgVector_t

```
template<class PRES_CLASS , class SALN_CLASS , class TEMP_CLASS , unsigned int MAX_MSGS>
typedef static_vector<SalnMsgPtr_t, MAX_MSGS> util::xmsg::AbstractXSensorDataComposite< PRES_↩
CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SalnMsgVector_t  [private]
```

Type definition for a static vector (of size MAX_MSGS) of shared pointers to the salinity data message.

10.4.2.5 TempMsgPtr_t

```
template<class PRES_CLASS , class SALN_CLASS , class TEMP_CLASS , unsigned int MAX_MSGS>
typedef shared_ptr<TEMP_CLASS> util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS,
TEMP_CLASS, MAX_MSGS >::TempMsgPtr_t  [private]
```

Type definition for a shared pointer to the temperature data message.

10.4.2.6 TempMsgVector_t

```
template<class PRES_CLASS , class SALN_CLASS , class TEMP_CLASS , unsigned int MAX_MSGS>
typedef static_vector<TempMsgPtr_t, MAX_MSGS> util::xmsg::AbstractXSensorDataComposite< PRES_↩
CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::TempMsgVector_t  [private]
```

Type definition for a static vector (of size MAX_MSGS) of shared pointers to the temperature data message.

10.4.3 Constructor & Destructor Documentation

10.4.3.1 AbstractXSensorDataComposite()

```
template<class PRES_CLASS , class SALN_CLASS , class TEMP_CLASS , unsigned int MAX_MSGS>
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::↩
AbstractXSensorDataComposite ()  [inline]
```

The constructor.

The field aggregated_data_valid_ initialized to FALSE, and the aggregated sensor data vectors reserve XMSG_INIT_SAM_N_RESERVE samples.

References util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::aggregated_p
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::aggregated_saln_data_,
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::aggregated_temp_data_,
and XMSG_INIT_SAM_N_RESERVE.

10.4.3.2 ~AbstractXSensorDataComposite()

```
template<class PRES_CLASS , class SALN_CLASS , class TEMP_CLASS , unsigned int MAX_MSGS>
virtual util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS
>::~AbstractXSensorDataComposite ()  [inline], [virtual]
```

The destructor.

10.4.4 Member Function Documentation

10.4.4.1 AggregateSensorData()

```
template<class PRES_CLASS , class SALN_CLASS , class TEMP_CLASS , unsigned int MAX_MSGS>
void util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::↩
AggregateSensorData ()  [inline], [protected]
```

Aggregate the sensor data for this composite sensor data object.

This function iterates over each of the sensor data messages assigned to this composite data object, and copies of the sensor data values into the corresponding sensor data vector.

If aggregated_data_valid_ is TRUE, then this function will have no effect. Otherwise, each of the aggregegated sensor data vectors will be cleared and the appropriate number of samples will be reserved.

Then, the sensor data message will be iterated over in order of increasing message indices, and the sensor data values will be copied into the corresponding sensor data vector in the appropriate order.

Finally, aggregated_data_valid_ will be set to FALSE to ensure that subsequent calls to this function will have no effect, until another sensor data message object is assigned to this composite object and aggregated_data_valid_ is set to TRUE.

Exceptions

util::log::MessageException	if an error occurs while aggregating the sensor data.

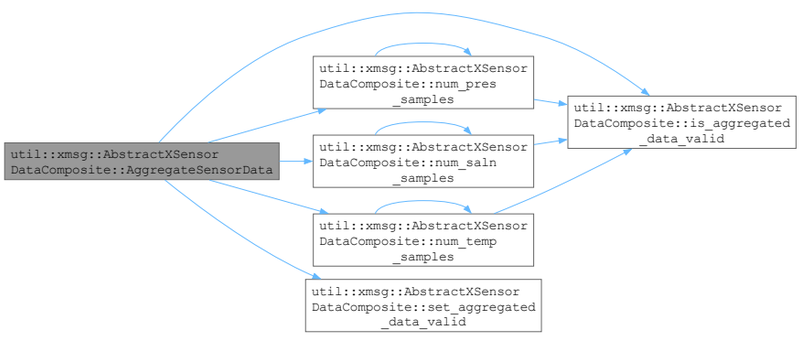
Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

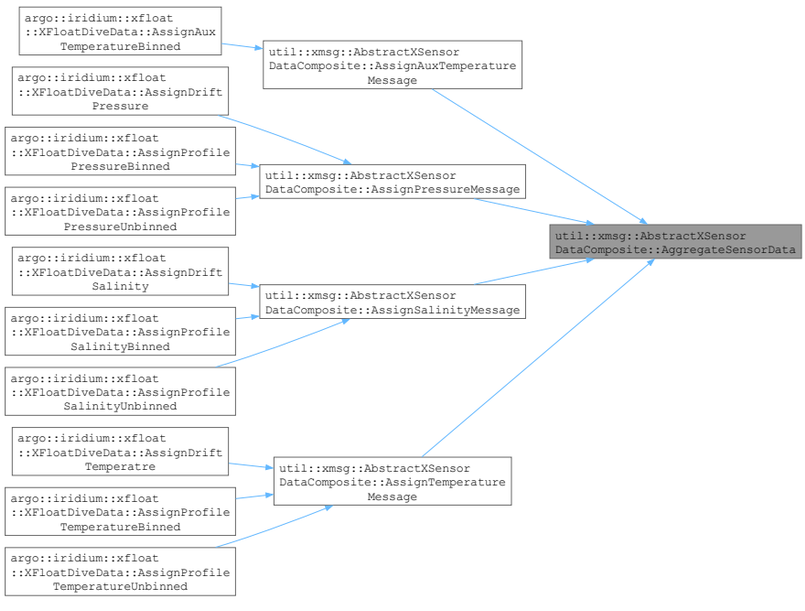
References util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::aggregated_p
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::aggregated_saln_data_,
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::aggregated_temp_data_,
CATCH_BLOCK, util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::is_aggre
util::thread::RecursiveLockable::mutex_, util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, M
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::num_saln_samples(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::num_temp_samples(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::pres_messages_,
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::saln_messages_,
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::set_aggregated_data_vali
and util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::temp_messages_.

Referenced by util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignAux
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignPressureMessage(
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignSalinityMessage(),
and util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignTemperatureM

Here is the call graph for this function:



Here is the caller graph for this function:



10.4.4.2 AssignAuxTemperatureMessage()

```
template<class PRES_CLASS , class SALN_CLASS , class TEMP_CLASS , unsigned int MAX_MSGS>
void util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::↩
AssignAuxTemperatureMessage (
            const TempMsgPtr_t & x)  [inline]
```

Assign an auxiliary-channel temperature message and re-aggregate the sensor data.

The message is inserted in order of message index, and the aggregated sensor data is rebuilt.

Parameters

in	*x*	The auxiliary-channel temperature message to assign.

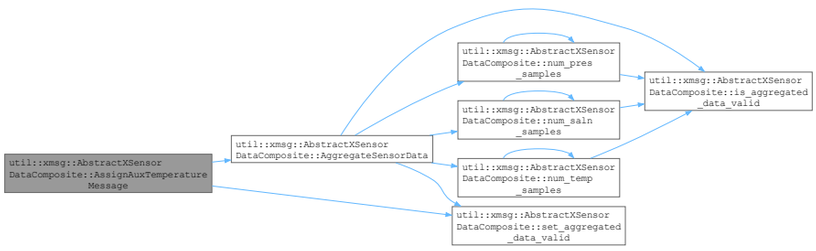
Exceptions

util::log::MessageException	if x is NULL or an auxiliary temperature message with the same index has already been assigned.

References util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AggregateSer
CATCH_BLOCK, EXC_THROW, ldbg, util::thread::RecursiveLockable::mutex_, and util::xmsg::AbstractXSensorDataComposite< PRES_

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignAuxTemperatureBinned().

Here is the call graph for this function:



Here is the caller graph for this function:

10.4.4.3 AssignPressureMessage()

```
template<class PRES_CLASS , class SALN_CLASS , class TEMP_CLASS , unsigned int MAX_MSGS>
void util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::↩
AssignPressureMessage (
            const PresMsgPtr_t & x)  [inline]
```

Assign a pressure data message to this composite data object.

The pressure data message will be emplaced into the sensor data message vector in order of increasing message indices. If a message with the same index is as x is already contained in the vector, then an exception will be thrown.

Parameters

in	*x*	A shared pointer to the pressure data message.

Exceptions

util::log::MessageException	if x is a null pointer, a duplicate message index is encountered, or any other error occurs.

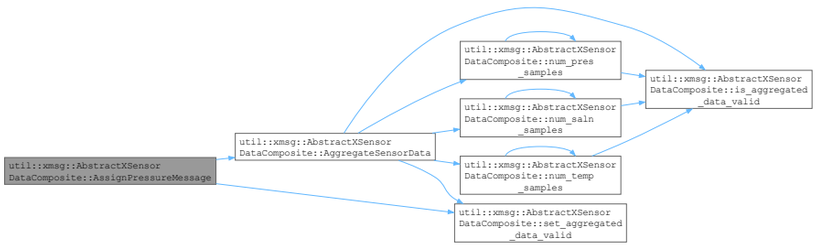
Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

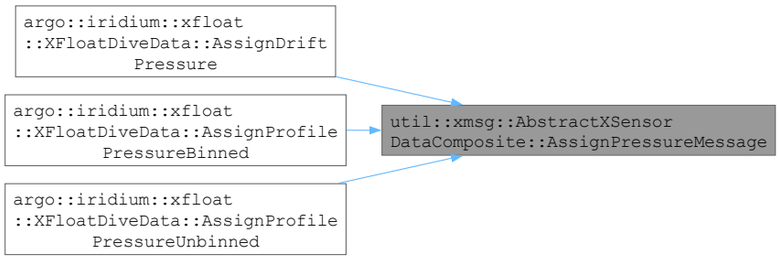
References util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AggregateSer CATCH_BLOCK, EXC_THROW, ldbg, util::thread::RecursiveLockable::mutex_, util::xmsg::AbstractXSensorDataComposite< PRES_CLA and util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::set_aggregated_data_

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignDriftPressure(), argo::iridium::xfloat::XFloatDiveData::AssignProfilePressureBi and argo::iridium::xfloat::XFloatDiveData::AssignProfilePressureUnbinned().

Here is the call graph for this function:



Here is the caller graph for this function:



10.4.4.4 AssignSalinityMessage()

```
template<class PRES_CLASS , class SALN_CLASS , class TEMP_CLASS , unsigned int MAX_MSGS>
void util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::↩
AssignSalinityMessage (
            const SalnMsgPtr_t & x)  [inline]
```

Assign a salinity data message to this composite data object.

The salinity data message will be emplaced into the sensor data message vector in order of increasing message indices. If a message with the same index is as x is already contained in the vector, then an exception will be thrown.

Parameters

in	*x*	A shared pointer to the salinity data message.

Exceptions

util::log::MessageException	if x is a null pointer, a duplicate message index is encountered, or any other error occurs.

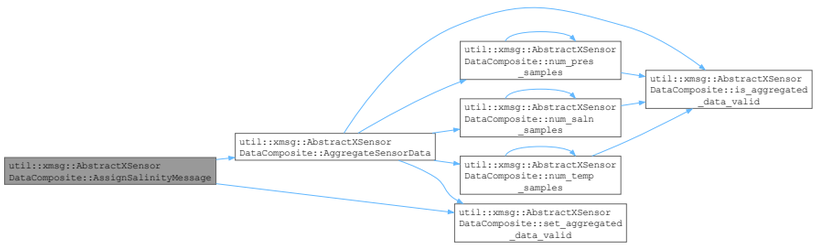
Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

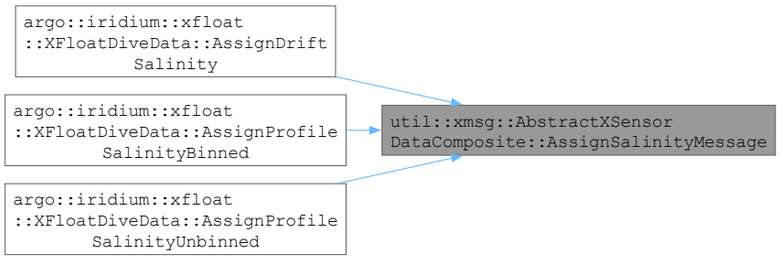
References util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AggregateSer
CATCH_BLOCK, EXC_THROW, ldbg, util::thread::RecursiveLockable::mutex_, util::xmsg::AbstractXSensorDataComposite< PRES_CLA
and util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::set_aggregated_data_

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignDriftSalinity(), argo::iridium::xfloat::XFloatDiveData::AssignProfileSalinityBinne
and argo::iridium::xfloat::XFloatDiveData::AssignProfileSalinityUnbinned().

Here is the call graph for this function:



Here is the caller graph for this function:



10.4.4.5 AssignTemperatureMessage()

```
template<class PRES_CLASS , class SALN_CLASS , class TEMP_CLASS , unsigned int MAX_MSGS>
void util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::↩
AssignTemperatureMessage (
            const TempMsgPtr_t & x)  [inline]
```

Assign a temperature data message to this composite data object.

The temperature data message will be emplaced into the sensor data message vector in order of increasing message indices. If a message with the same index is as x is already contained in the vector, then an exception will be thrown.

Parameters

in	*x*	A shared pointer to the temperature data message.

Exceptions

util::log::MessageException	if x is a null pointer, a duplicate message index is encountered, or any other error occurs.

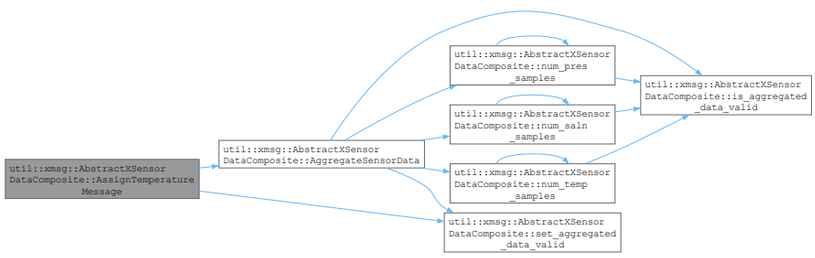
Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

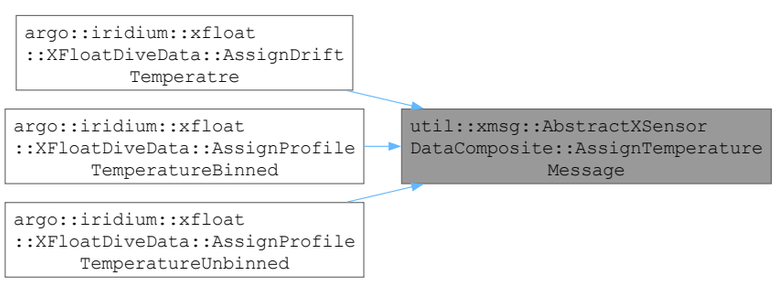
References util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AggregateSer CATCH_BLOCK, EXC_THROW, ldbg, util::thread::RecursiveLockable::mutex_, util::xmsg::AbstractXSensorDataComposite< PRES_CLA and util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::temp_messages_.

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignDriftTemperatre(), argo::iridium::xfloat::XFloatDiveData::AssignProfileTempera and argo::iridium::xfloat::XFloatDiveData::AssignProfileTemperatureUnbinned().

Here is the call graph for this function:



Here is the caller graph for this function:



10.4.4.6 GenerateListingString()

```
template<class PRES_CLASS , class SALN_CLASS , class TEMP_CLASS , unsigned int MAX_MSGS>
virtual const string util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_↩
CLASS, MAX_MSGS >::GenerateListingString (
            int id_inst,
            int id_dive,
            uint16_t pg,
            uint16_t po,
            uint16_t sg,
            uint16_t so,
            uint16_t tg,
            uint16_t to,
            float pc) const  [pure virtual]
```

Generate a listing string representation of the this composite sensor data object.

Parameters

in	*id_inst*	The unique instrument identifier for this composite sensor data object.
in	*id_dive*	The dive or cycle number for this composite data object.
in	*pg*	The pressure sensor sensitivity scale factor, in LSB/dBar.
in	*po*	The pressure sensor offset, in dBar.
in	*sg*	The salinity sensor sensitivity scale factor, in LSB/PSU.
in	*so*	The salinity sensor offset, in PSU.
in	*tg*	The temperature sensor sensitivity scale factor, in LSB/degC.
in	*to*	The temperature sensor offset, in degC.
in	*pc*	The pressure correction, in dBar, to be subtracted computed pressure value.

Exceptions

log::MessageException	if an error occurs while generating the listing string.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

A listing string representation of this composite sensor data object.

Implemented in util::xmsg::XCompositeDrift, util::xmsg::XCompositeHiResProfile, and util::xmsg::XCompositeProfile.

10.4.4.7 HasContiguousMessageIndices()

```
template<class PRES_CLASS , class SALN_CLASS , class TEMP_CLASS , unsigned int MAX_MSGS>
bool util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::↩
HasContiguousMessageIndices () const  [inline]
```

Do the sensor data messages assigned to this composite data object have contiguous message indices?

This function is used, in part, to determine the sensor data is complete.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

TRUE if the sensor data messages have contiguous message indices or if there is less than two of each sensor data message, or FALSE otherwise.

References CATCH_BLOCK, ldbg, util::thread::RecursiveLockable::mutex_, util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::saln_messages_, and util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::temp_messages_.

Referenced by util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::IsDataCom

Here is the caller graph for this function:

10.4.4.8 is_aggregated_data_valid()

```
template<class PRES_CLASS , class SALN_CLASS , class TEMP_CLASS , unsigned int MAX_MSGS>
bool util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >↩
::is_aggregated_data_valid () const  [inline]
```

Is the aggregated data valid?

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

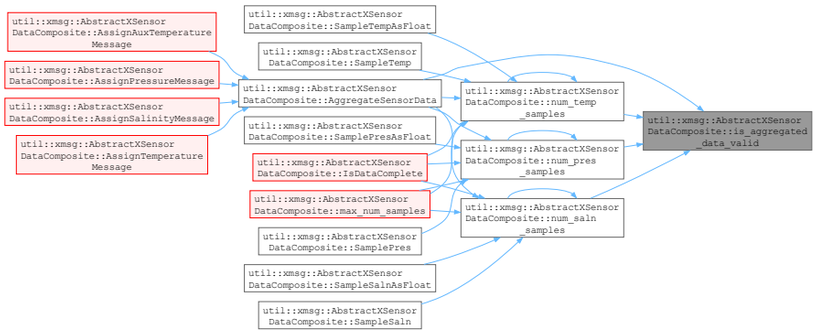
Returns

TRUE if the aggregated sensor is valid, or FALSE otherwise.

References util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::aggregated_d
and util::thread::RecursiveLockable::mutex_.

Referenced by util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AggregateS
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::num_pres_samples(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::num_saln_samples(),
and util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::num_temp_samples()

Here is the caller graph for this function:



10.4.4.9 IsDataComplete()

```
template<class PRES_CLASS , class SALN_CLASS , class TEMP_CLASS , unsigned int MAX_MSGS>
bool util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::↩
IsDataComplete () const  [inline]
```

Is the sensor data for this composite date message complete?

This function returns true if

- this composite sensor data object is not empty,
- the sensor data messages assigned to this composite data object have contiguous message indices,
- this sensor data object contains the same number of temperature and pressure sensor values, and
- the number of salinity data samples is either zero or equal to the number of pressure data samples.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

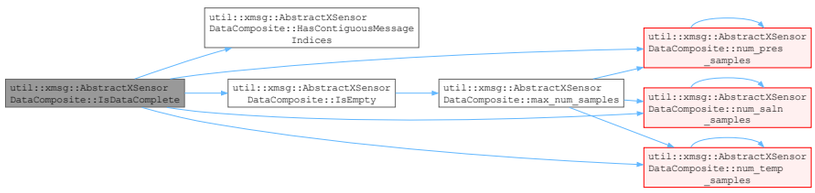
Returns

TRUE if the sensor data is complete, or FALSE otherwise.

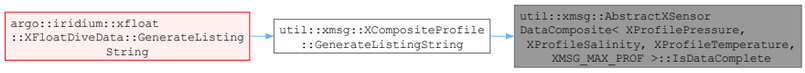
References CATCH_BLOCK, util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::IsEmpty(),
util::thread::RecursiveLockable::mutex_, util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, M
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::num_saln_samples(),
and util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::num_temp_samples()

Referenced by argo::iridium::xfloat::XFloatDiveData::HasCompleteBinnedProfileData(), and argo::iridium::xfloat::XFloatDiveData::HasCon

Here is the call graph for this function:



Here is the caller graph for this function:



10.4.4.10 IsEmpty()

```
template<class PRES_CLASS , class SALN_CLASS , class TEMP_CLASS , unsigned int MAX_MSGS>
bool util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::↩
IsEmpty () const  [inline]
```

Is this composite data object empty?

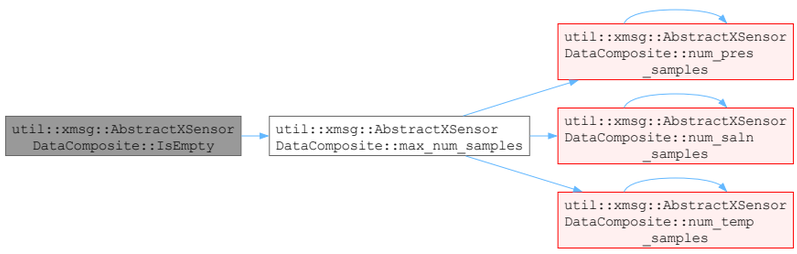
Returns

TRUE if empty, or FALSE otherwise.

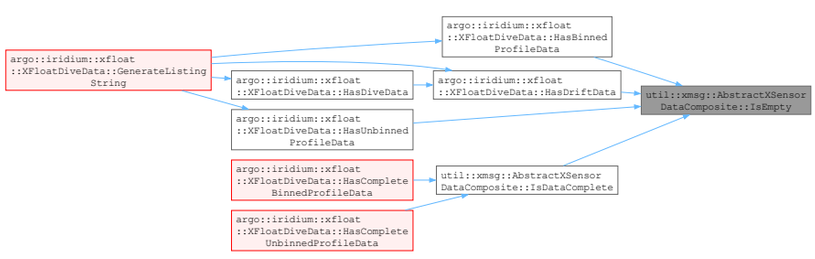
References util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::max_num_sa

Referenced by argo::iridium::xfloat::XFloatDiveData::HasBinnedProfileData(), argo::iridium::xfloat::XFloatDiveData::HasDriftData(),
argo::iridium::xfloat::XFloatDiveData::HasUnbinnedProfileData(), and util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN

Here is the call graph for this function:



Here is the caller graph for this function:



10.4.4.11 LockObject()

```
bool util::thread::RecursiveLockable::LockObject (
            bool block)  [inline], [inherited]
```

Lock the object.

Parameters

in	*block*	If TRUE, then the calling thread will block until the lock becomes available.

Exceptions

util::log::MessageException	if an error occurs.

Returns

TRUE if the lock was successfully obtained, or FALSE if block is FALSE and another thread owns the lock on the mutex.

References CATCH_BLOCK, and util::thread::RecursiveLockable::mutex_.

10.4.4.12 max_num_samples()

```
template<class PRES_CLASS , class SALN_CLASS , class TEMP_CLASS , unsigned int MAX_MSGS>
size_t util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >↩
::max_num_samples () const  [inline]
```

Return the maximum of the number of pressure, temperature, or salinity sensor samples in the sensor data messages assigned to this composite sensor data object.

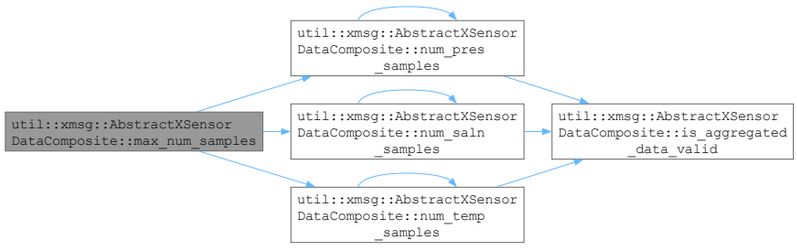
Returns

The maximum of the number of pressure, temperature, or salinity sensor samples.

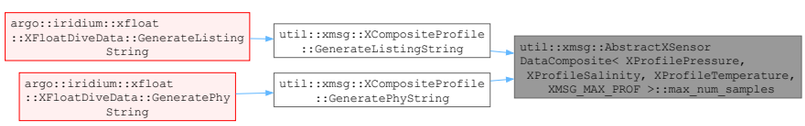
References util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::num_pres_sa
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::num_saln_samples(),
and util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::num_temp_samples()

Referenced by util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::IsEmpty().

Here is the call graph for this function:



Here is the caller graph for this function:



10.4.4.13 num_pres_samples()

```
template<class PRES_CLASS , class SALN_CLASS , class TEMP_CLASS , unsigned int MAX_MSGS>
size_t util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >↩
::num_pres_samples () const  [inline]
```

Get the total number of pressure sensor samples contained in the pressure data messages assigned to this composite data object.

Exceptions

util::log::MessageException	if an error occurs.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

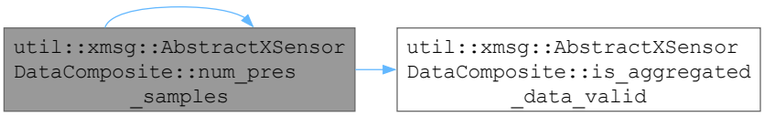
Returns

The total number of pressure sensor samples contained in the pressure data messages assigned to this composite data object.

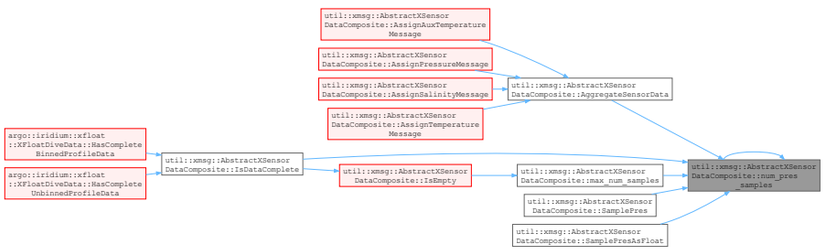
References util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::aggregated_p
CATCH_BLOCK, util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::is_aggre
util::thread::RecursiveLockable::mutex_, util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, M
and util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::pres_messages_.

Referenced by util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AggregateS
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::IsDataComplete(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::max_num_samples(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::num_pres_samples(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SamplePres(),
and util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SamplePresAsFloat()

Here is the call graph for this function:



Here is the caller graph for this function:



10.4.4.14 num_saln_samples()

```
template<class PRES_CLASS , class SALN_CLASS , class TEMP_CLASS , unsigned int MAX_MSGS>
size_t util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >↩
::num_saln_samples () const  [inline]
```

Get the total number of salinity sensor samples contained in the salinity data messages assigned to this composite data object.

Exceptions

util::log::MessageException	if an error occurs.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

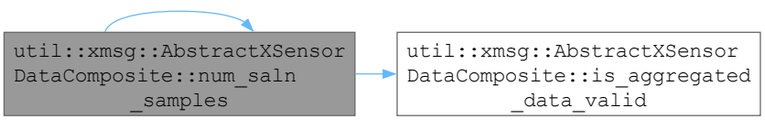
Returns

The total number of pressure sensor samples contained in the pressure data messages assigned to this composite data object.

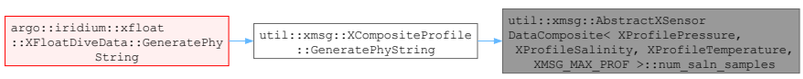
References util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::aggregated_s
CATCH_BLOCK, util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::is_aggre
util::thread::RecursiveLockable::mutex_, util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, M
and util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::saln_messages_.

Referenced by util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AggregateS
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::IsDataComplete(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::max_num_samples(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::num_saln_samples(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleSaln(),
and util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleSalnAsFloat().

Here is the call graph for this function:



Here is the caller graph for this function:



10.4.4.15 num_temp_samples()

```
template<class PRES_CLASS , class SALN_CLASS , class TEMP_CLASS , unsigned int MAX_MSGS>
size_t util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >↩
::num_temp_samples () const  [inline]
```

Get the total number of temperature sensor samples contained in the temperature data messages assigned to this composite data object.

Exceptions

util::log::MessageException	if an error occurs.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

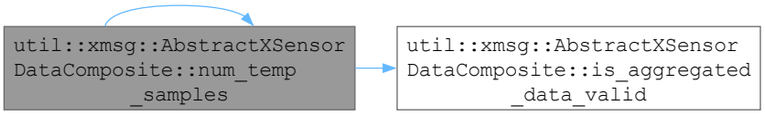
Returns

The total number of pressure sensor samples contained in the pressure data messages assigned to this composite data object.

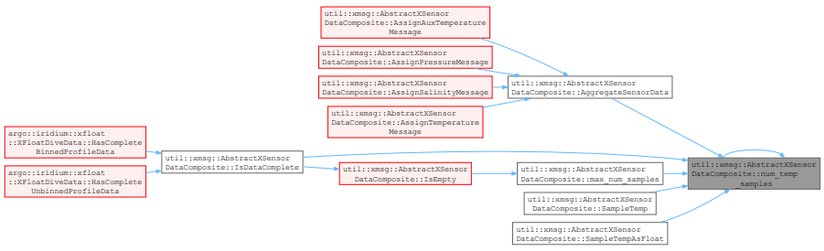
References util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::aggregated_te
CATCH_BLOCK, util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::is_aggre
util::thread::RecursiveLockable::mutex_, util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MA
and util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::temp_messages_.

Referenced by util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AggregateS
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::IsDataComplete(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::max_num_samples(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::num_temp_samples(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleTemp(),
and util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleTempAsFloat(

Here is the call graph for this function:



Here is the caller graph for this function:



10.4.4.16 Reset()

```
template<class PRES_CLASS , class SALN_CLASS , class TEMP_CLASS , unsigned int MAX_MSGS>
void util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::↩
Reset ()  [inline]
```

Remove the assigned pressure, temperature and salinity messages and their aggregated data.

Warning

The auxiliary temperature messages and aggregated auxiliary temperature data are not cleared, so re-assigning them after a reset fails with a duplicate index.

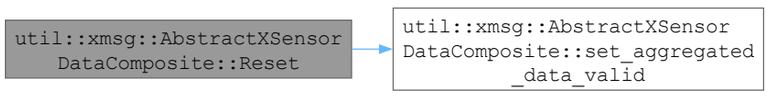
Exceptions

util::log::MessageException	if an error occurs.

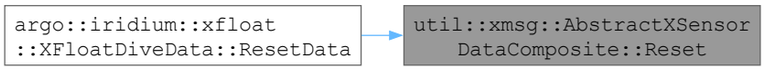
References util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::aggregated_p
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::aggregated_saln_data_,
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::aggregated_temp_data_,
CATCH_BLOCK, util::thread::RecursiveLockable::mutex_, util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, T
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::saln_messages_,
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::set_aggregated_data_vali
and util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::temp_messages_.

Referenced by argo::iridium::xfloat::XFloatDiveData::ResetData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.4.4.17 SamplePres()

```
template<class PRES_CLASS , class SALN_CLASS , class TEMP_CLASS , unsigned int MAX_MSGS>
int32_t util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS
>::SamplePres (
            size_t i) const  [inline]
```

Get the i^{th} pressure sensor value, in raw counts.

Parameters

in	*i*	The index of the pressure sensor sample to return. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD will be returned.

Note

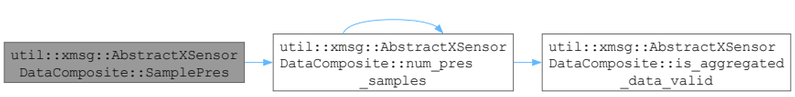
This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The i^{th} pressure sensor value, in raw counts, or XMSG_SENS_BAD if the index `i` exceeds the number of pressure sensor samples in the aggregated data vector.

References util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::aggregated_p util::thread::RecursiveLockable::mutex_, util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, M and XMSG_SENS_BAD.

Here is the call graph for this function:



10.4.4.18 SamplePresAsFloat()

```
template<class PRES_CLASS , class SALN_CLASS , class TEMP_CLASS , unsigned int MAX_MSGS>
template<typename T = double>
T util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::↩
SamplePresAsFloat (
            size_t i,
            int16_t pg,
            int16_t po,
            float pc) const  [inline]
```

Get the i^{th} pressure sensor value, in dBar.

Parameters

in	*i*	The index of the sensor value. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD_PRES will be returned.
in	*pg*	The pressure sensor sensitivity scale factor, in LSB/dBar.
in	*po*	The pressure sensor offset, in dBar.
in	*pc*	The pressure correction, in dBar, to be subtracted computed pressure value.

Note

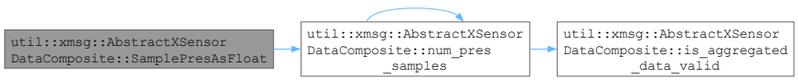
This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The i^{th} pressure sensor value, in dBar. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD_PRES will be returned.

References util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::aggregated_p
CATCH_BLOCK, util::thread::RecursiveLockable::mutex_, util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, T
XMSG_SENS_BAD_PRES, and XMSG_SENS_MAX_PRES.

Here is the call graph for this function:



10.4.4.19 SampleSaln()

```
template<class PRES_CLASS , class SALN_CLASS , class TEMP_CLASS , unsigned int MAX_MSGS>
int32_t util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS
>::SampleSaln (
            size_t i) const  [inline]
```

Get the i^{th} salinity sensor value, in raw counts.

Parameters

in	*i*	The index of the salinity sensor sample to return. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD will be returned.

Note

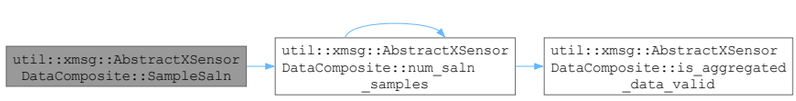
This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The i^{th} salinity sensor value, in raw counts, or XMSG_SENS_BAD if the index i exceeds the number of pressure sensor samples in the aggregated data vector.

References util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::aggregated_s
util::thread::RecursiveLockable::mutex_, util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, M
and XMSG_SENS_BAD.

Here is the call graph for this function:



10.4.4.20 SampleSalnAsFloat()

```
template<class PRES_CLASS , class SALN_CLASS , class TEMP_CLASS , unsigned int MAX_MSGS>
template<typename T = double>
T util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::↩
SampleSalnAsFloat (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline]
```

Get the i^{th} salinity sensor value, in PSU.

Parameters

in	*i*	The index of the sensor value. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD_SALN will be returned.
in	*gain*	The salinity sensor sensitivity scale factor, in LSB/PSU.
in	*offs*	The salinity sensor offset, in PSU.

Note

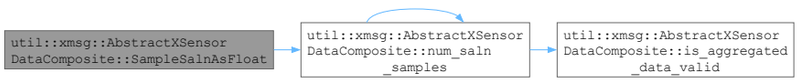
This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The i^{th} salinity sensor value, in PSU. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD_SALN will be returned.

References util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::aggregated_s
CATCH_BLOCK, util::thread::RecursiveLockable::mutex_, util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, T
XMSG_SENS_BAD_SALN, and XMSG_SENS_MAX_SALN.

Here is the call graph for this function:



10.4.4.21 SampleTemp()

```
template<class PRES_CLASS , class SALN_CLASS , class TEMP_CLASS , unsigned int MAX_MSGS>
int32_t util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS
>::SampleTemp (
            size_t i) const  [inline]
```

Get the i^{th} temperature sensor value, in raw counts.

Parameters

in	*i*	The index of the temperature sensor sample to return. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD will be returned.

Note

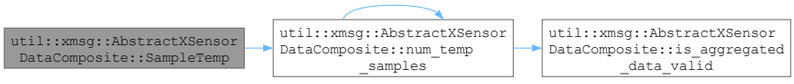
This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The i^{th} temperature sensor value, in raw counts, or XMSG_SENS_BAD if the index `i` exceeds the number of pressure sensor samples in the aggregated data vector.

References util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::aggregated_te
util::thread::RecursiveLockable::mutex_, util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MA
and XMSG_SENS_BAD.

Here is the call graph for this function:



10.4.4.22 SampleTempAsFloat()

```
template<class PRES_CLASS , class SALN_CLASS , class TEMP_CLASS , unsigned int MAX_MSGS>
template<typename T = double>
T util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::↩
SampleTempAsFloat (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline]
```

Get the i^{th} temperature sensor value, in degC.

Parameters

in	*i*	The index of the sensor value. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD_SALN will be returned.
in	*gain*	The temperature sensor sensitivity scale factor, in LSB/degC.
in	*offs*	The temperature sensor offset, in degC.

Note

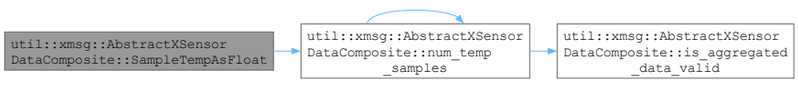
This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The i^{th} temperature sensor value, in degC. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD_TEMP will be returned.

References util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::aggregated_te
CATCH_BLOCK, util::thread::RecursiveLockable::mutex_, util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, T
XMSG_SENS_BAD_TEMP, and XMSG_SENS_MAX_TEMP.

Here is the call graph for this function:



10.4.4.23 set_aggregated_data_valid()

```
template<class PRES_CLASS , class SALN_CLASS , class TEMP_CLASS , unsigned int MAX_MSGS>
void util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >↩
::set_aggregated_data_valid (
            bool valid)  [inline], [protected]
```

Set the value of the aggregated_data_valid_ field.

Parameters

in	*valid*	TRUE if the aggregated data is valid, or FALSE otherwise.

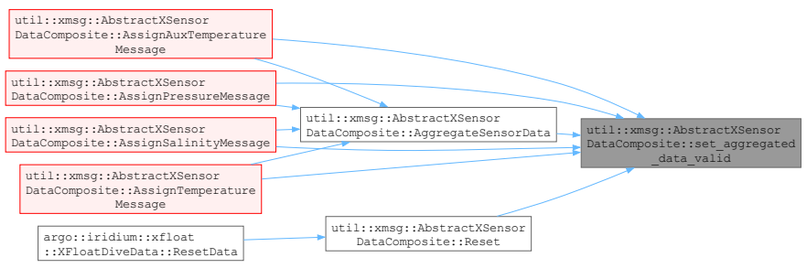
Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

References util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::aggregated_d and util::thread::RecursiveLockable::mutex_.

Referenced by util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AggregateS util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignAuxTemperatureMe util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignPressureMessage(util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignSalinityMessage(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignTemperatureMessa and util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::Reset().

Here is the caller graph for this function:



10.4.4.24 UnlockObject()

```
void util::thread::RecursiveLockable::UnlockObject ()  [inline], [inherited]
```

Unlock the object.

Exceptions

util::log::MessageException	if an error occurs.

References CATCH_BLOCK, and util::thread::RecursiveLockable::mutex_.

10.4.5 Member Data Documentation

10.4.5.1 aggregated_data_valid_

```
template<class PRES_CLASS , class SALN_CLASS , class TEMP_CLASS , unsigned int MAX_MSGS>
bool util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >↩
::aggregated_data_valid_  [private]
```

The aggregated data is valid flag.

This flag indicates the status of the aggregated sensor data. It is used to ensure the composite sensor data is aggregated only once following each assignment of sensor data to the composite data message.

This field is set to FALSE during initialization, and is set to FALSE each time a sensor data message is assigned to the composite data message. Each time AggregateSensorData() is called, the data will be aggregated, and this field will be set to TRUE.

Referenced by util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::is_aggrega and util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::set_aggregated_data_

10.4.5.2 aggregated_pres_data_

```
template<class PRES_CLASS , class SALN_CLASS , class TEMP_CLASS , unsigned int MAX_MSGS>
vector<int32_t> util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS,
MAX_MSGS >::aggregated_pres_data_  [private]
```

A vector of the aggregated pressure data samples, in raw counts.

Each time the data is aggregated, the sensor data samples from the sensor data messages assigned to this composite objects are stored in this vector.

Referenced by util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AbstractXS
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AggregateSensorData(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::num_pres_samples(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::Reset(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SamplePres(),
and util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SamplePresAsFloat()

10.4.5.3 aggregated_saln_data_

```
template<class PRES_CLASS , class SALN_CLASS , class TEMP_CLASS , unsigned int MAX_MSGS>
vector<int32_t> util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS,
MAX_MSGS >::aggregated_saln_data_  [private]
```

A vector of the aggregated salinity data samples, in raw counts.

Each time the data is aggregated, the sensor data samples from the sensor data messages assigned to this composite objects are stored in this vector.

Referenced by util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AbstractXS
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AggregateSensorData(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::num_saln_samples(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::Reset(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleSaln(),
and util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleSalnAsFloat().

10.4.5.4 aggregated_temp_data_

```
template<class PRES_CLASS , class SALN_CLASS , class TEMP_CLASS , unsigned int MAX_MSGS>
vector<int32_t> util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS,
MAX_MSGS >::aggregated_temp_data_  [private]
```

A vector of the aggregated temperature data samples, in raw counts.

Each time the data is aggregated, the sensor data samples from the sensor data messages assigned to this composite object are stored in this vector.

Referenced by util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AbstractXS
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AggregateSensorData(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::num_temp_samples(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::Reset(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleTemp(),
and util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleTempAsFloat(

10.4.5.5 mutex_

recursive_mutex util::thread::RecursiveLockable::mutex_ [mutable], [protected], [inherited]

Recursive mutual exclusion lock for synchronizing operations.

Referenced by util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AggregateS
argo::iridium::xfloat::XFloat::argo_instrument_type(), argo::iridium::xfloat::XFloatDiveData::argo_instrument_type(),
argo::iridium::xfloat::XFloatDiveData::AssignArgoData(), argo::iridium::xfloat::XFloatDiveData::AssignAuxTemperatureBinned(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignAuxTemperatureMe
argo::iridium::xfloat::XFloatDiveData::AssignDriftPressure(), argo::iridium::xfloat::XFloatDiveData::AssignDriftSalinity(),
argo::iridium::xfloat::XFloatDiveData::AssignDriftTemperatre(), argo::iridium::xfloat::XFloatDiveData::AssignEngineeringAbort(),
argo::iridium::xfloat::XFloatDiveData::AssignEngineeringAscent(), argo::iridium::xfloat::XFloatDiveData::AssignEngineeringBitFail(),
argo::iridium::xfloat::XFloatDiveData::AssignEngineeringBitPass(), argo::iridium::xfloat::XFloatDiveData::AssignEngineeringDescent(),
argo::iridium::xfloat::XFloatDiveData::AssignEngineeringDiagnostic(), argo::iridium::xfloat::XFloatDiveData::AssignEngineeringDrift(),
argo::iridium::xfloat::XFloatDiveData::AssignEngineeringIce(), argo::iridium::xfloat::XFloatDiveData::AssignEngineeringStandard(),
argo::iridium::xfloat::XFloatDiveData::AssignEngineeringSurface(), argo::iridium::xfloat::XFloatDiveData::AssignFallTimeSeries(),
argo::iridium::xfloat::XFloatDiveData::AssignGps(), argo::iridium::xfloat::XFloatDiveData::AssignMailMessage(),
argo::iridium::xfloat::XFloatDiveData::AssignMissionCmdEchoback(), argo::iridium::xfloat::XFloatDiveData::AssignParentInstrument(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignPressureMessage(
argo::iridium::xfloat::XFloatDiveData::AssignProfilePressureBinned(), argo::iridium::xfloat::XFloatDiveData::AssignProfilePressureUnbinn
argo::iridium::xfloat::XFloatDiveData::AssignProfileSalinityBinned(), argo::iridium::xfloat::XFloatDiveData::AssignProfileSalinityUnbinned()
argo::iridium::xfloat::XFloatDiveData::AssignProfileTemperatureBinned(), argo::iridium::xfloat::XFloatDiveData::AssignProfileTemperatureU
argo::iridium::xfloat::XFloatDiveData::AssignPumpTimeSeries(), argo::iridium::xfloat::XFloatDiveData::AssignRiseTimeSeries(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignSalinityMessage(),
argo::iridium::xfloat::XFloatDiveData::AssignSurfaceTimeSeries(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_C
argo::iridium::xfloat::XFloatDiveData::AssignValveData(), argo::iridium::xfloat::XFloatDiveData::CheckCompleteness(),
argo::iridium::xfloat::XFloatMetadata::comment(), argo::iridium::xfloat::XFloatDiveData::ComputePressureOffsetFromSurfaceTimeSeries(
argo::iridium::xfloat::XFloatMetadata::deployer(), argo::iridium::xfloat::XFloatMetadata::deployer_address(), argo::iridium::xfloat::XFloatMe
argo::iridium::xfloat::XFloatMetadata::deployment_datetime(), argo::iridium::xfloat::XFloatMetadata::deployment_datetime_status(),
argo::iridium::xfloat::XFloatMetadata::deployment_latitude(), argo::iridium::xfloat::XFloatMetadata::deployment_longitude(),
argo::iridium::xfloat::XFloatMetadata::deployment_platform(), argo::iridium::xfloat::XFloatMetadata::deployment_position_status(),
argo::iridium::xfloat::XFloatMetadata::deployment_type(), argo::iridium::xfloat::XFloat::fw_version(), argo::iridium::xfloat::XFloat::gain_pres
argo::iridium::xfloat::XFloat::gain_pressure_surface(), argo::iridium::xfloat::XFloat::gain_salinity(), argo::iridium::xfloat::XFloat::gain_salinit
argo::iridium::xfloat::XFloat::gain_temperature(), argo::iridium::xfloat::XFloat::gain_temperature_surface(), argo::iridium::xfloat::XFloatDive
argo::iridium::xfloat::XFloatDiveData::GenerateCompositeIridiumPosition(), argo::iridium::xfloat::XFloatDiveData::GenerateCompositePun
argo::iridium::xfloat::XFloatDiveData::GenerateCompositeRiseTimeSeriesData(), argo::iridium::xfloat::XFloatDiveData::GenerateListingSt
util::xmsg::XCompositeDrift::GenerateListingString(), util::xmsg::XCompositeHiResProfile::GenerateListingString(),
util::xmsg::XCompositeProfile::GenerateListingString(), argo::iridium::xfloat::XFloatDiveData::GenerateOrderedMomsnList(),
argo::iridium::xfloat::XFloatDiveData::GeneratePhyString(), util::xmsg::XCompositeDrift::GeneratePhyString(), util::xmsg::XCompositeHiR
util::xmsg::XCompositeProfile::GeneratePhyString(), argo::iridium::xfloat::XFloatDiveData::GetDiveDataMailMessagesFromDatabase(),
argo::iridium::xfloat::XFloatDiveData::GetGainsAndOffsets(), argo::iridium::xfloat::XFloatDiveData::GetGpsPositionEndOfDive(),
argo::iridium::xfloat::XFloatDiveData::GetSurfaceGainsAndOffsets(), argo::iridium::xfloat::XFloatDiveData::HasBinnedProfileData(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::HasContiguousMessageIn
argo::iridium::xfloat::XFloatDiveData::HasContiguousMomsns(), argo::iridium::xfloat::XFloatDiveData::HasContiguousPids(),
argo::iridium::xfloat::XFloatDiveData::HasDiveData(), argo::iridium::xfloat::XFloatDiveData::HasDriftData(), argo::iridium::xfloat::XFloatDiv
argo::iridium::xfloat::XFloatDiveData::HasFallRiseData(), argo::iridium::xfloat::XFloatDiveData::HasPositionData(),
argo::iridium::xfloat::XFloatDiveData::HasUnbinnedProfileData(), argo::iridium::xfloat::XFloatDiveData::HasValidGpsPosition(),
argo::iridium::Iridium::id_dac(), argo::iridium::xfloat::XFloatDiveData::id_dive(), argo::iridium::xfloat::XFloatMailMessage::id_dive(),
argo::iridium::Iridium::id_imei(), argo::iridium::Iridium::id_instrument(), argo::iridium::xfloat::XFloatMailMessage::id_msg(),
argo::iridium::xfloat::XFloatMailMessage::id_msg_uid(), argo::iridium::xfloat::XFloatMailMessage::id_sbd_momsn(),
argo::iridium::xfloat::XFloatMailMessage::id_sbd_mtmsn(), argo::iridium::xfloat::XFloatMailMessage::id_sbd_packet(),
argo::iridium::Iridium::id_wmo(), argo::iridium::xfloat::XFloatDiveData::intermediate_listing_file_path(), util::xmsg::AbstractXSensorDataCo
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::IsDataComplete(),
argo::iridium::xfloat::XFloatMetadata::local_path(), util::thread::RecursiveLockable::LockObject(), argo::iridium::xfloat::XFloatMetadata::ma
argo::iridium::xfloat::XFloat::metadata(), argo::iridium::xfloat::XFloatMailMessage::msg_body_is_valid(), argo::iridium::xfloat::XFloatMailM
argo::iridium::xfloat::XFloatMailMessage::msg_header_datetime(), argo::iridium::xfloat::XFloatMailMessage::msg_header_from(),
argo::iridium::xfloat::XFloatMailMessage::msg_header_is_valid(), argo::iridium::xfloat::XFloatMailMessage::msg_header_raw_text(),
argo::iridium::xfloat::XFloatMailMessage::msg_header_subject(), argo::iridium::xfloat::XFloatMailMessage::msg_is_valid(),
argo::iridium::xfloat::XFloatMailMessage::msg_raw_text(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, T
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::num_saln_samples(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::num_temp_samples(),
argo::iridium::xfloat::XFloat::offset_pressure(), argo::iridium::xfloat::XFloat::offset_pressure_surface(), argo::iridium::xfloat::XFloat::offset_
argo::iridium::xfloat::XFloat::offset_salinity_surface(), argo::iridium::xfloat::XFloat::offset_temperature(), argo::iridium::xfloat::XFloat::offset
argo::iridium::xfloat::XFloatMailMessage::ParseMailMsg(), argo::iridium::xfloat::XFloatMailMessage::ParseMailMsg(),
argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgBody(), argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgHeader(),
argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgSbdAttachment(), argo::iridium::xfloat::XFloat::phy_file_generation_is_enabled(),
argo::iridium::xfloat::XFloatDiveData::phy_file_path(), argo::iridium::xfloat::XFloatMetadata::principal_investigators(),
argo::iridium::xfloat::XFloatMetadata::principal_investigators_address(), argo::iridium::xfloat::XFloatMetadata::principal_investigators_orig
argo::iridium::xfloat::XFloatMailMessage::raw_file_local_path(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLA
argo::iridium::xfloat::XFloatDiveData::ResetData(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_C
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SamplePresAsFloat(),

util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleSaln(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleSalnAsFloat(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleTemp(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleTempAsFloat(), argo::iridium::xfloat::XFloatDiveData::SaveObjectToDatabase(), argo::iridium::xfloat::XFloatMailMessage::SaveObjectToDatabase(), argo::iridium::xfloat::XFloatMetadata::SaveObjectToDatabase(), argo::iridium::xfloat::XFloatMailMessage::sbd_data(), argo::iridium::xfloat::XFloatMailMessage::sbd_data_is_valid(), argo::iridium::xfloat::XFloatMailMessage::sbd_file_local_path(), argo::iridium::xfloat::XFloatMailMessage::sbd_file_name(), argo::iridium::xfloat::XFloatMailMessage::sbd_file_size_bytes(), argo::iridium::xfloat::XFloatMailMessage::sbd_session_status_code(), argo::iridium::xfloat::XFloatMailMessage::sbd_session_status_strir argo::iridium::xfloat::XFloatDiveData::SensorGainsAndOffsetsAreKnown(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, argo::iridium::xfloat::XFloat::set_argo_instrument_type(), argo::iridium::xfloat::XFloatDiveData::set_argo_instrument_type(), argo::iridium::xfloat::XFloatMetadata::set_comment(), argo::iridium::xfloat::XFloatMetadata::set_deployer(), argo::iridium::xfloat::XFloatMe argo::iridium::xfloat::XFloatMetadata::set_deployment_cruise_id(), argo::iridium::xfloat::XFloatMetadata::set_deployment_datetime(), argo::iridium::xfloat::XFloatMetadata::set_deployment_datetime_status(), argo::iridium::xfloat::XFloatMetadata::set_deployment_latitude(argo::iridium::xfloat::XFloatMetadata::set_deployment_longitude(), argo::iridium::xfloat::XFloatMetadata::set_deployment_platform(), argo::iridium::xfloat::XFloatMetadata::set_deployment_position_status(), argo::iridium::xfloat::XFloatMetadata::set_deployment_type(), argo::iridium::xfloat::XFloat::set_fw_version(), argo::iridium::xfloat::XFloat::set_gain_pressure(), argo::iridium::xfloat::XFloat::set_gain_pre argo::iridium::xfloat::XFloat::set_gain_salinity(), argo::iridium::xfloat::XFloat::set_gain_salinity_surface(), argo::iridium::xfloat::XFloat::set_ argo::iridium::xfloat::XFloat::set_gain_temperature_surface(), argo::iridium::Iridium::set_id_dac(), argo::iridium::xfloat::XFloatDiveData::se argo::iridium::xfloat::XFloatMailMessage::set_id_dive(), argo::iridium::Iridium::set_id_imei(), argo::iridium::Iridium::set_id_instrument(), argo::iridium::xfloat::XFloatMailMessage::set_id_msg(), argo::iridium::xfloat::XFloatMailMessage::set_id_msg_uid(), argo::iridium::xfloat::XFloatMailMessage::set_id_sbd_momsn(), argo::iridium::xfloat::XFloatMailMessage::set_id_sbd_mtmsn(), argo::iridium::xfloat::XFloatMailMessage::set_id_sbd_packet(), argo::iridium::Iridium::set_id_wmo(), argo::iridium::xfloat::XFloatDiveData: argo::iridium::xfloat::XFloatMetadata::set_local_path(), argo::iridium::xfloat::XFloatMetadata::set_manufacturer(), argo::iridium::xfloat::XFloat::set_metadata(), argo::iridium::xfloat::XFloatMailMessage::set_msg_body_is_valid(), argo::iridium::xfloat::XFloatMailMessage::set_msg_body_text(), argo::iridium::xfloat::XFloatMailMessage::set_msg_header_datetime(), argo::iridium::xfloat::XFloatMailMessage::set_msg_header_from(), argo::iridium::xfloat::XFloatMailMessage::set_msg_header_is_valid(), argo::iridium::xfloat::XFloatMailMessage::set_msg_header_raw_text(), argo::iridium::xfloat::XFloatMailMessage::set_msg_header_subjec argo::iridium::xfloat::XFloatMailMessage::set_msg_is_valid(), argo::iridium::xfloat::XFloatMailMessage::set_msg_raw_text(), argo::iridium::xfloat::XFloat::set_offset_pressure(), argo::iridium::xfloat::XFloat::set_offset_pressure_surface(), argo::iridium::xfloat::XFloa argo::iridium::xfloat::XFloat::set_offset_salinity_surface(), argo::iridium::xfloat::XFloat::set_offset_temperature(), argo::iridium::xfloat::XFloat::set_offset_temperature_surface(), argo::iridium::xfloat::XFloat::set_phy_file_generation_is_enabled(), argo::iridium::xfloat::XFloatDiveData::set_phy_file_path(), argo::iridium::xfloat::XFloatMetadata::set_principal_investigators(), argo::iridium::xfloat::XFloatMetadata::set_principal_investigators_address(), argo::iridium::xfloat::XFloatMetadata::set_principal_investiga argo::iridium::xfloat::XFloatMailMessage::set_raw_file_local_path(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_data(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_data_is_valid(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_file_local_path(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_file_name(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_file_size_bytes(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_session_status_code(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_session_sta argo::iridium::xfloat::XFloat::set_start_of_mission_datetime(), argo::iridium::xfloat::XFloatMetadata::set_wmo_instrument_type(), argo::iridium::xfloat::XFloatMetadata::set_wmo_recorder_type(), argo::iridium::xfloat::XFloat::set_xmsg_version_major(), argo::iridium::xfloat::XFloat::set_xmsg_version_minor(), argo::iridium::xfloat::XFloat::start_of_mission_datetime(), util::thread::RecursiveLockable::UnlockObject(), argo::iridium::xfloat::XFloatDiveData::UpdateDerivedSurfaceDataFields(), argo::iridium::xfloat::XFloatMetadata::wmo_instrument_type(), argo::iridium::xfloat::XFloatMetadata::wmo_recorder_type(), argo::iridium::xfloat::XFloatDiveData::WriteListingFile(), argo::iridium::xfloat::XFloatDiveData::WritePhyFile(), argo::iridium::xfloat::XFloatM argo::iridium::xfloat::XFloat::xmsg_version_major(), and argo::iridium::xfloat::XFloat::xmsg_version_minor().

10.4.5.6 pres_messages_

```
template<class PRES_CLASS , class SALN_CLASS , class TEMP_CLASS , unsigned int MAX_MSGS>
PresMsgVector_t util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS,
MAX_MSGS >::pres_messages_  [private]
```

A static vector (of size MAX_MSGS) containing pointers the pressure sensor data messages assigned to this composite object.

Each time a pressure sensor data message is assigned to the this object, it will be emplaced in this vector in order of increasing message indices.

Referenced by util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AggregateS util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignPressureMessage(util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::HasContiguousMessageIr util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::num_pres_samples(), and util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::Reset().

10.4.5.7 saln_messages_

```
template<class PRES_CLASS , class SALN_CLASS , class TEMP_CLASS , unsigned int MAX_MSGS>
SalnMsgVector_t util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS,
MAX_MSGS >::saln_messages_  [private]
```

A static vector (of size MAX_MSGS) containing pointers the salinity sensor data messages assigned to this composite object.

Each time a salinity sensor data message is assigned to the this object, it will be emplaced in this vector in order of increasing message indices.

Referenced by util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AggregateS
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignSalinityMessage(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::HasContiguousMessageIr
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::num_saln_samples(),
and util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::Reset().

10.4.5.8 temp_messages_

```
template<class PRES_CLASS , class SALN_CLASS , class TEMP_CLASS , unsigned int MAX_MSGS>
TempMsgVector_t util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS,
MAX_MSGS >::temp_messages_  [private]
```

A static vector (of size MAX_MSGS) containing pointers the temperature sensor data messages assigned to this composite object.

Each time a temperature sensor data message is assigned to the this object, it will be emplaced in this vector in order of increasing message indices.

Referenced by util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AggregateS
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignTemperatureMessa
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::HasContiguousMessageIr
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::num_temp_samples(),
and util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::Reset().

The documentation for this class was generated from the following file:

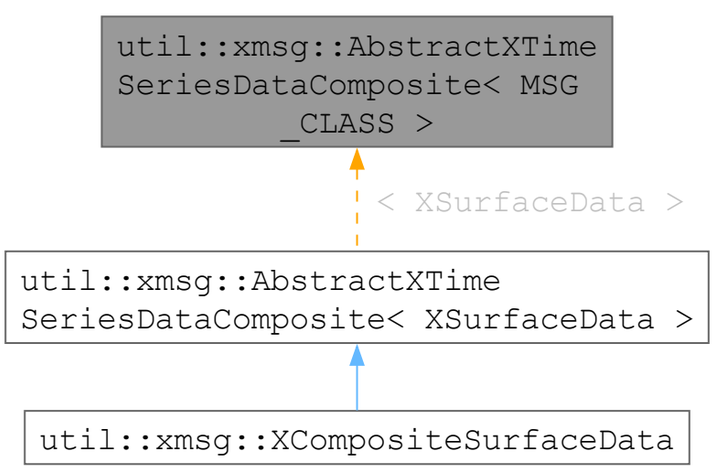
- data/xmsg/AbstractXSensorDataComposite.h

10.5 util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS > Class Template Reference

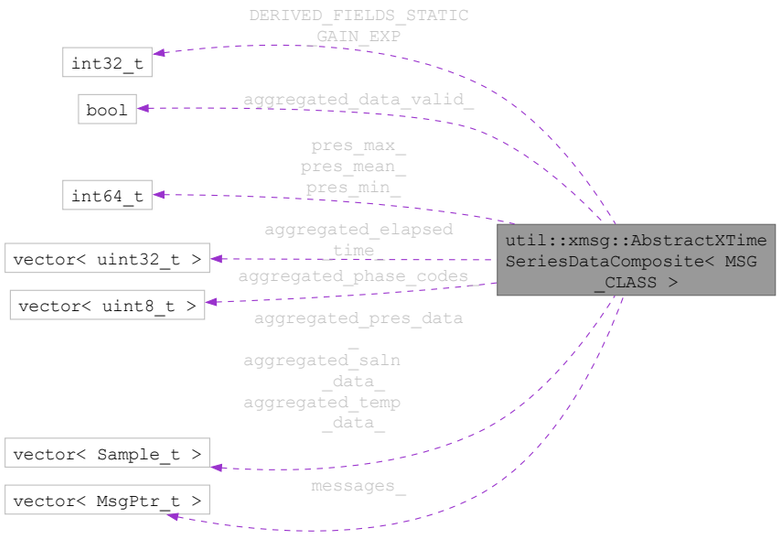
The abstract time-series data composite class template.

```
#include <AbstractXTimeSeriesDataComposite.h>
```

Inheritance diagram for util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >:



Collaboration diagram for util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >:



Public Types

- typedef int64_t **DerivedSample_t**
- typedef vector< uint32_t > **ElapsedTimeVector_t**
- typedef shared_ptr< MSG_CLASS > **MsgPtr_t**
- typedef vector< MsgPtr_t > **MsgVector_t**
- typedef MsgVector_t::iterator **MsgVectorIter_t**
- typedef MsgVector_t::const_iterator **MsgVectorIterConst_t**
- typedef vector< uint8_t > **PhaseCodeVector_t**
- typedef int32_t **Sample_t**
- typedef vector< Sample_t > **SampleVector_t**

Public Member Functions

- virtual void AssignMessage (const MsgPtr_t &x)

  Assign a time-series message to this composite and re-aggregate the time series.
- const int32_t **DERIVED_FIELDS_STATIC_GAIN_EXP**
- virtual const string **GenerateListingString** (uint16_t &pg, uint16_t &po, uint16_t &sg, uint16_t &so, uint16_t &tg, uint16_t &to, float pres_correction) const
- virtual bool **HasContiguousMessageIndices** () const
- bool is_aggregated_data_valid () const

  Is the aggregated data valid?
- bool **IsDataComplete** () const
- bool IsEmpty () const

  Is this composite data object empty?
- size_t **NumPresSamples** () const
- size_t **NumSalnSamples** () const
- size_t **NumTempSamples** () const
- DerivedSample_t **pres_max** () const
- DerivedSample_t **pres_mean** () const
- DerivedSample_t **pres_min** () const
- template<typename T = float>
  T **PresMaxAsFloat** (uint16_t gain, uint16_t offs, float pres_correction=0) const
- template<typename T = float>
  T **PresMeanAsFloat** (uint16_t gain, uint16_t offs, float pres_correction=0) const
- template<typename T = float>
  T **PresMinAsFloat** (uint16_t gain, uint16_t offs, float pres_correction=0) const
- void Reset ()

  Remove all assigned messages and clear the aggregated data.
- void ResetAggregatedData ()

  Clear the aggregated time series and its derived pressure statistics.
- virtual size_t **SampleCount** () const

- int32_t SamplePres (size_t i) const

  Get the i^{th} pressure sensor value, in raw counts.
- template<typename T = float>
  T SamplePresAsFloat (size_t i, int16_t pg, int16_t po, float pc) const

  Get the i^{th} pressure sensor value, in dBar.
- int32_t SampleSaln (size_t i) const

  Get the i^{th} salinity sensor value, in raw counts.
- template<typename T = float>
  T SampleSalnAsFloat (size_t i, int16_t gain, int16_t offs) const

  Get the i^{th} salinity sensor value, in PSU.
- int32_t SampleTemp (size_t i) const

  Get the i^{th} temperature sensor value, in raw counts.
- template<typename T = float>
  T SampleTempAsFloat (size_t i, int16_t gain, int16_t offs) const

  Get the i^{th} temperature sensor value, in degC.

Static Public Member Functions

- static int32_t **derived_fields_static_gain_exp** ()

Static Public Attributes

- static const int32_t **DERIVED_FIELDS_STATIC_GAIN_EXP**

Protected Member Functions

- ElapsedTimeVector_t & **aggregated_elapsed_time** ()
- PhaseCodeVector_t & **aggregated_phase_codes** ()
- SampleVector_t & **aggregated_pres_data** ()
- SampleVector_t & **aggregated_saln_data** ()
- SampleVector_t & **aggregated_temp_data** ()
- virtual void AggregateSensorData ()

  Combine the assigned time-series messages into one aggregated time series.
- const MsgVector_t & **messages** () const
- void set_aggregated_data_valid (bool valid)

  Set the value of the aggregated_data_valid_ field.
- void **set_pres_max** (DerivedSample_t pres_max)
- void **set_pres_mean** (DerivedSample_t pres_mean)
- void **set_pres_min** (DerivedSample_t pres_min)

Private Attributes

- bool aggregated_data_valid_

  The aggregated data is valid flag.
- ElapsedTimeVector_t **aggregated_elapsed_time_**
- PhaseCodeVector_t **aggregated_phase_codes_**
- SampleVector_t **aggregated_pres_data_**
- SampleVector_t **aggregated_saln_data_**
- SampleVector_t **aggregated_temp_data_**
- MsgVector_t **messages_**
- DerivedSample_t **pres_max_**
- DerivedSample_t **pres_mean_**
- DerivedSample_t **pres_min_**

10.5.1 Detailed Description

template<class MSG_CLASS>
class util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >

The abstract time-series data composite class template.

This template aggregates the indexed time-series X-messages of type `MSG_CLASS` received for a single dive into one time series. It is used for the surface time-series data (see XCompositeSurfaceData).

10.5.2 Member Function Documentation

10.5.2.1 AggregateSensorData()

```
template<class MSG_CLASS >
virtual void util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::AggregateSensorData ()
[inline], [protected], [virtual]
```

Combine the assigned time-series messages into one aggregated time series.

Does nothing if the aggregated data is already valid. Otherwise the aggregated data is cleared and rebuilt from the messages, in message-index order: the pressure counts and phase codes, plus temperature and salinity counts for messages in the XRISEFALL_FMT_STNDRD_WITH_TEMP_AND_SALN format. Salinity data is dropped if it contains no valid samples (temperature-only instruments).

The pressure maximum, minimum and mean are computed from the valid samples (ignoring XMSG_SENS_BAD and 0xFFFF), after applying the static gain (derived_fields_static_gain_exp). The aggregated data is marked valid only if there is at least one valid pressure sample.

Exceptions

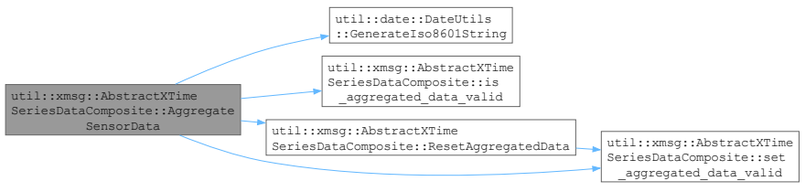
util::log::MessageException	if no messages are assigned or the message format is unsupported.

Reimplemented in util::xmsg::XCompositeSurfaceData.

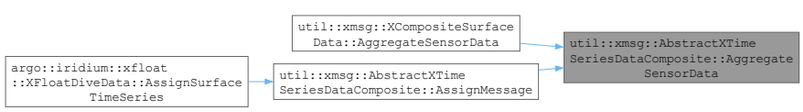
References CATCH_BLOCK, EXC_THROW, util::date::DateUtils::GenerateIso8601String(), util::xmsg::AbstractXTimeSeriesDataCompos
ldbg, lwrn, util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::ResetAggregatedData(), util::xmsg::AbstractXTimeSeriesD
XMSG_SENS_BAD, util::xmsg::XRISEFALL_FMT_LEGACY, util::xmsg::XRISEFALL_FMT_STNDRD, util::xmsg::XRISEFALL_FMT_STN
and util::xmsg::XRISEFALL_FMT_UNKNOWN.

Referenced by util::xmsg::XCompositeSurfaceData::AggregateSensorData(), and util::xmsg::AbstractXTimeSeriesDataComposite< MSG

Here is the call graph for this function:



Here is the caller graph for this function:



10.5.2.2 AssignMessage()

```
template<class MSG_CLASS >
virtual void util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::AssignMessage (
            const MsgPtr_t & x)  [inline], [virtual]
```

Assign a time-series message to this composite and re-aggregate the time series.

The message is inserted in order of message index, and AggregateSensorData() is called to rebuild the combined time series.

Parameters

in	*x*	The time-series message to assign.

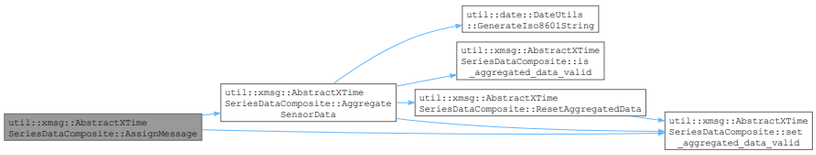
Exceptions

util::log::MessageException	if x is NULL or a message with the same index has already been assigned.

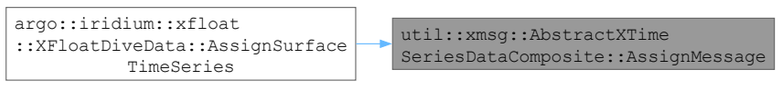
References util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::AggregateSensorData(), CATCH_BLOCK, EXC_THROW, ldbg, and util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::set_aggregated_data_valid().

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignSurfaceTimeSeries().

Here is the call graph for this function:



Here is the caller graph for this function:



10.5.2.3 is_aggregated_data_valid()

```
template<class MSG_CLASS >
bool util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::is_aggregated_data_valid () const
[inline]
```

Is the aggregated data valid?

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

TRUE if the aggregated sensor is valid, or FALSE otherwise.

References util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::aggregated_data_valid_.

Referenced by util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::AggregateSensorData().

Here is the caller graph for this function:

10.5.2.4 IsEmpty()

```
template<class MSG_CLASS >
bool util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::IsEmpty () const  [inline]
```

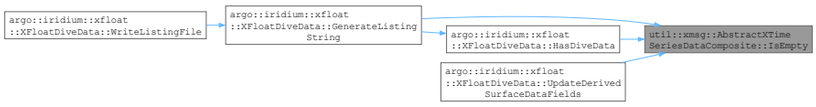
Is this composite data object empty?

Returns

TRUE if empty, or FALSE otherwise.

Referenced by argo::iridium::xfloat::XFloatDiveData::GenerateListingString(), argo::iridium::xfloat::XFloatDiveData::HasDiveData(), and argo::iridium::xfloat::XFloatDiveData::UpdateDerivedSurfaceDataFields().

Here is the caller graph for this function:



10.5.2.5 Reset()

```
template<class MSG_CLASS >
void util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::Reset ()  [inline]
```

Remove all assigned messages and clear the aggregated data.

Exceptions

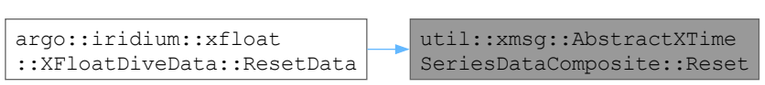
util::log::MessageException	if an error occurs.

References CATCH_BLOCK, and util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::ResetAggregatedData().

Referenced by argo::iridium::xfloat::XFloatDiveData::ResetData().

Here is the call graph for this function:

Here is the caller graph for this function:



10.5.2.6 ResetAggregatedData()

```
template<class MSG_CLASS >
void util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::ResetAggregatedData ()  [inline]
```

Clear the aggregated time series and its derived pressure statistics.

The assigned messages are kept; the next call to `AggregateSensorData()` rebuilds the aggregated data from them.

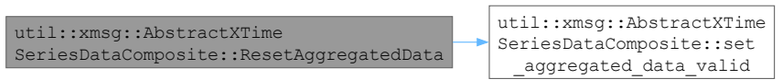
Exceptions

util::log::MessageException	if an error occurs.

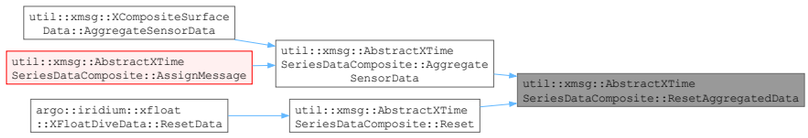
References CATCH_BLOCK, util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::set_aggregated_data_valid(), and XMSG_SENS_BAD.

Referenced by util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::AggregateSensorData(), and util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::Reset().

Here is the call graph for this function:



Here is the caller graph for this function:



10.5.2.7 SamplePres()

```
template<class MSG_CLASS >
int32_t util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::SamplePres (
            size_t i) const  [inline]
```

Get the i^{th} pressure sensor value, in raw counts.

Parameters

in	*i*	The index of the pressure sensor sample to return. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD will be returned.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The i^{th} pressure sensor value, in raw counts, or XMSG_SENS_BAD if the index `i` exceeds the number of pressure sensor samples in the aggregated data vector.

References XMSG_SENS_BAD.

10.5.2.8 SamplePresAsFloat()

```
template<class MSG_CLASS >
template<typename T = float>
T util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::SamplePresAsFloat (
            size_t i,
            int16_t pg,
            int16_t po,
            float pc) const  [inline]
```

Get the i^{th} pressure sensor value, in dBar.

Parameters

in	*i*	The index of the sensor value. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD_PRES will be returned.
in	*pg*	The pressure sensor sensitivity scale factor, in LSB/dBar.
in	*po*	The pressure sensor offset, in dBar.
in	*pc*	The pressure correction, in dBar, to be subtracted computed pressure value.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The i^{th} pressure sensor value, in dBar. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD_PRES will be returned.

References CATCH_BLOCK, XMSG_SENS_BAD_PRES, and XMSG_SENS_MAX_PRES.

10.5.2.9 SampleSaln()

```
template<class MSG_CLASS >
int32_t util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::SampleSaln (
            size_t i) const  [inline]
```

Get the i^{th} salinity sensor value, in raw counts.

Parameters

in	*i*	The index of the salinity sensor sample to return. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD will be returned.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The i^{th} salinity sensor value, in raw counts, or XMSG_SENS_BAD if the index `i` exceeds the number of pressure sensor samples in the aggregated data vector.

References XMSG_SENS_BAD.

10.5.2.10 SampleSalnAsFloat()

```
template<class MSG_CLASS >
template<typename T = float>
T util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::SampleSalnAsFloat (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline]
```

Get the i^{th} salinity sensor value, in PSU.

Parameters

in	*i*	The index of the sensor value. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD_SALN will be returned.
in	*gain*	The salinity sensor sensitivity scale factor, in LSB/PSU.
in	*offs*	The salinity sensor offset, in PSU.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The i^{th} salinity sensor value, in PSU. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD_SALN will be returned.

References CATCH_BLOCK, XMSG_SENS_BAD_SALN, and XMSG_SENS_MAX_SALN.

10.5.2.11 SampleTemp()

```
template<class MSG_CLASS >
int32_t util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::SampleTemp (
            size_t i) const  [inline]
```

Get the i^{th} temperature sensor value, in raw counts.

Parameters

in	*i*	The index of the temperature sensor sample to return. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD will be returned.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The i^{th} temperature sensor value, in raw counts, or XMSG_SENS_BAD if the index `i` exceeds the number of pressure sensor samples in the aggregated data vector.

References XMSG_SENS_BAD.

10.5.2.12 SampleTempAsFloat()

```
template<class MSG_CLASS >
template<typename T = float>
T util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::SampleTempAsFloat (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline]
```

Get the i^{th} temperature sensor value, in degC.

Parameters

in	*i*	The index of the sensor value. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD_SALN will be returned.
in	*gain*	The temperature sensor sensitivity scale factor, in LSB/degC.
in	*offs*	The temperature sensor offset, in degC.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The i^{th} temperature sensor value, in degC. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD_TEMP will be returned.

References CATCH_BLOCK, XMSG_SENS_BAD_TEMP, and XMSG_SENS_MAX_TEMP.

10.5.2.13 set_aggregated_data_valid()

```
template<class MSG_CLASS >
void util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::set_aggregated_data_valid (
            bool valid)  [inline], [protected]
```

Set the value of the aggregated_data_valid_ field.

Parameters

in	*valid*	TRUE if the aggregated data is valid, or FALSE otherwise.

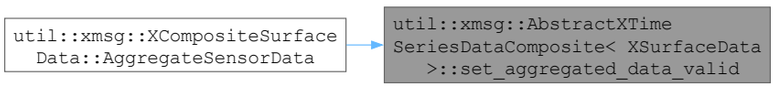
Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

References util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::aggregated_data_valid_.

Referenced by util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::AggregateSensorData(), util::xmsg::AbstractXTimeSer
and util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::ResetAggregatedData().

Here is the caller graph for this function:



10.5.3 Member Data Documentation

10.5.3.1 aggregated_data_valid_

```
template<class MSG_CLASS >
bool util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::aggregated_data_valid_  [private]
```

The aggregated data is valid flag.

This flag indicates the status of the aggregated sensor data. It is used to ensure the composite sensor data is aggregated only once following each assignment of sensor data to the composite data message.

This field is set to FALSE during initialization, and is set to FALSE each time a sensor data message is assigned to the composite data message. Each time `AggregateSensorData()` is called, the data will be aggregated, and this field will be set to TRUE.

Referenced by util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::is_aggregated_data_valid(), and util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::set_aggregated_data_valid().

The documentation for this class was generated from the following file:

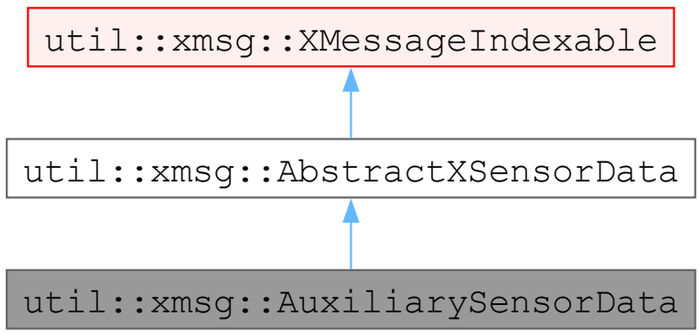
- data/xmsg/AbstractXTimeSeriesDataComposite.h

10.6 util::xmsg::AuxiliarySensorData Class Reference

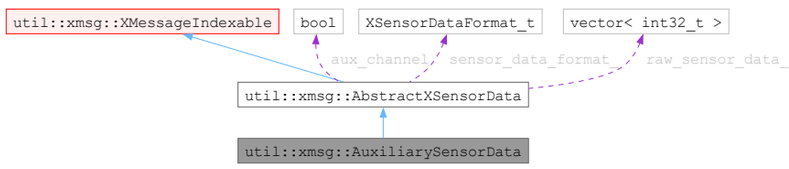
The auxiliary sensor data class.

`#include <AuxiliarySensorData.h>`

Inheritance diagram for util::xmsg::AuxiliarySensorData:



Collaboration diagram for util::xmsg::AuxiliarySensorData:



Public Member Functions

- AuxiliarySensorData ()
  Default constructor.
- virtual ~AuxiliarySensorData ()
  Destructor.
- const uint8_t * CheckXMsgData (const vector< uint8_t > &data, int id, int minBytes, bool parseSize=true)
  Check the X-message for the correct ID, minimum size, and message terminator.
- virtual void DecodeXMsgData (const vector< uint8_t > &data)=0
  Decode the X message.
- int16_t id_dive () const
  Get the dive or cycle number of this X-Message.
- uint16_t id_instrument () const
  Get the unique instrument identifier of this X-Message.
- XMsgTypeId_t id_xmsg_type () const
  Get the enumerated X-Message type identifier.
- bool **is_aux_channel** () const
- size_t num_samples () const
  Get the number of samples in this message.
- const vector< int32_t > & raw_sensor_data () const
  Get the raw sensor data vector.
- int32_t Sample (size_t i) const
  Get the raw integer value of the i^{th} sample.
- template<typename T = double>
  T SampleValue (size_t i, int16_t gain, int16_t offs) const
  Get the i^{th} sensor value in scientific units.
- template<typename T = double>
  T SampleValueAsPressure (size_t i, int16_t gain, int16_t offs) const

Get the i^{th} pressure sensor value, in dBar.

- template<typename T = double>
  T SampleValueAsSalinity (size_t i, int16_t gain, int16_t offs) const

  Get the i^{th} salinity sensor value, in PSU.
- template<typename T = double>
  T SampleValueAsTemperature (size_t i, int16_t gain, int16_t offs) const

  Get the i^{th} temperature sensor value, in degC.
- XSensorDataFormat_t sensor_data_format () const

  Get the enumerated sensor data format value.
- uint8_t xmsg_index () const

  Get the index of this X-Message.
- uint16_t xmsg_size_bytes () const

  Get the size of the X-Message, in bytes.

Protected Member Functions

- void set_id_dive (int16_t id_dive)

  Set the dive or cycle number of this message.
- void set_id_instrument (uint16_t id_instrument)

  Set the unique instrument identifier.
- void set_id_xmsg_type (XMsgTypeId_t id_xmsg_type)

  Set the X-Message type identifier.
- void set_raw_sensor_data (const vector< int32_t > &raw_sensor_data)

  Set raw integer counts of the measured sensor data.
- void set_sensor_data_format (XSensorDataFormat_t sensor_data_format)

  Set the sensor data format.
- void set_xmsg_index (uint8_t xmsg_index)

  Set the index of this X-Message.
- void set_xmsg_size_bytes (uint16_t xmsg_size_bytes)

  Set the size of the X-Message, in bytes.
- vector< int32_t > UnpackSensorData (const vector< uint8_t > &data)

  Unpack the sensor data from the binary message.

Protected Attributes

- bool **aux_channel**
- vector< int32_t > raw_sensor_data_

  The raw integer counts of the measured sensor data.
- XSensorDataFormat_t sensor_data_format_

  The enumerated sensor data packing format.

Private Attributes

- int16_t id_dive_

  The dive or cycle number of this message.
- uint16_t id_instrument_

  The unique WHOI identifier of this instrument.
- XMsgTypeId_t id_xmsg_type_

  The X-message type identifier.
- uint8_t xmsg_index_

  The index of this X-Message.
- uint16_t xmsg_size_bytes_

  The size of the X message, in bytes.

10.6.1 Detailed Description

The auxiliary sensor data class.

An instance of this class represents sensor data from an instrument's auxiliary sensor channel, decoded in the same way as the primary sensor data (see AbstractXSensorData).

10.6.2 Constructor & Destructor Documentation

10.6.2.1 AuxiliarySensorData()

```
util::xmsg::AuxiliarySensorData::AuxiliarySensorData ()  [inline]
```

Default constructor.

10.6.2.2 ~AuxiliarySensorData()

```
virtual util::xmsg::AuxiliarySensorData::~AuxiliarySensorData ()  [inline], [virtual]
```

Destructor.

10.6.3 Member Function Documentation

10.6.3.1 CheckXMsgData()

```
const uint8_t * util::xmsg::XMessage::CheckXMsgData (
            const vector< uint8_t > & data,
            int id,
            int minBytes,
            bool parseSize = true)  [inline], [inherited]
```

Check the X-message for the correct ID, minimum size, and message terminator.

Parameters

in	*data*	The X-message binary data to check.
in	*id*	The expected ID of the X-message. If this field is less that 0, then the ID check will be skipped.
in	*minBytes*	The minimum number of bytes expected in a valid message.
in	*parseSize*	If TRUE, then the message size field will be parsed from the X-Message. Otherwise, the size field will not be parsed.

Exceptions

util::log::MessageException	if • the message does not contain the expected ID, • the message does not contain the minimum number of expected bytes, • the message does not contain a terminator (';') in the correct location, or, • any other error occurs.

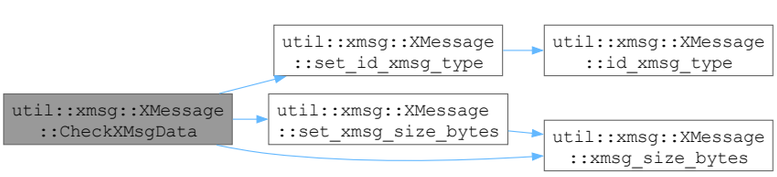
Returns

A pointer to the underlying data buffer of data.

References CATCH_BLOCK, EXC_THROW, lwrn, util::xmsg::XMessage::set_id_xmsg_type(), util::xmsg::XMessage::set_xmsg_size_byte
u16, and util::xmsg::XMessage::xmsg_size_bytes().

Referenced by util::xmsg::XAcousticPosition::DecodeXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XDriftPressure::Dec
util::xmsg::XDriftSalinity::DecodeXMsgData(), util::xmsg::XDriftTemperature::DecodeXMsgData(), util::xmsg::XEngAbort::DecodeXMsgDa
util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngBIT::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(),
util::xmsg::XEngDiagnostic::DecodeXMsgData(), util::xmsg::XEngDrift::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(),
util::xmsg::XEngStandard::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XGPS::DecodeXMsgData(),
util::xmsg::XHiResPressure::DecodeXMsgData(), util::xmsg::XHiResSalinity::DecodeXMsgData(), util::xmsg::XHiResTemperature::Decod
util::xmsg::XMissionCmdEcho::DecodeXMsgData(), util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XProfilePressure:
util::xmsg::XProfileSalinity::DecodeXMsgData(), util::xmsg::XProfileTemperature::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgD
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the call graph for this function:



10.6.3.2 DecodeXMsgData()

```
virtual void util::xmsg::AbstractXSensorData::DecodeXMsgData (
            const vector< uint8_t > & data)  [pure virtual], [inherited]
```

Decode the X message.

Parameters

in	*data*	The encoded X message to decode.

Implements util::xmsg::XMessageIndexable.

Implemented in util::xmsg::XDriftPressure, util::xmsg::XDriftSalinity, util::xmsg::XDriftTemperature, util::xmsg::XHiResPressure, util::xmsg::XHiResSalinity, util::xmsg::XHiResTemperature, util::xmsg::XProfilePressure, util::xmsg::XProfileSalinity, and util::xmsg::XProfileTemperature.

10.6.3.3 id_dive()

```
int16_t util::xmsg::XMessage::id_dive () const  [inline], [inherited]
```

Get the dive or cycle number of this X-Message.

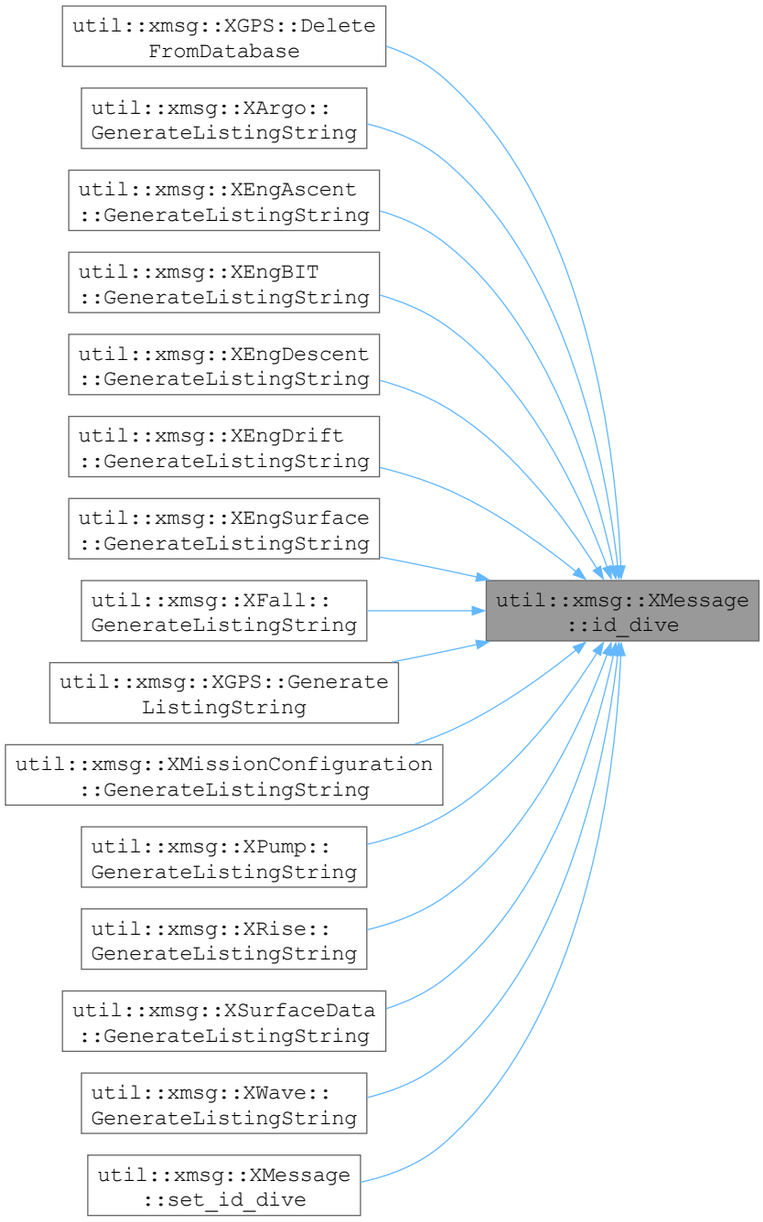
Returns

The dive or cycle number of this X-Message.

References util::xmsg::XMessage::id_dive_.

Referenced by util::xmsg::XGPS::DeleteFromDatabase(), util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateLis
util::xmsg::XEngBIT::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingStr
util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(),
util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing
util::xmsg::XSurfaceData::GenerateListingString(), util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_dive().

Here is the caller graph for this function:



10.6.3.4 id_instrument()

```
uint16_t util::xmsg::XMessage::id_instrument () const  [inline], [inherited]
```

Get the unique instrument identifier of this X-Message.

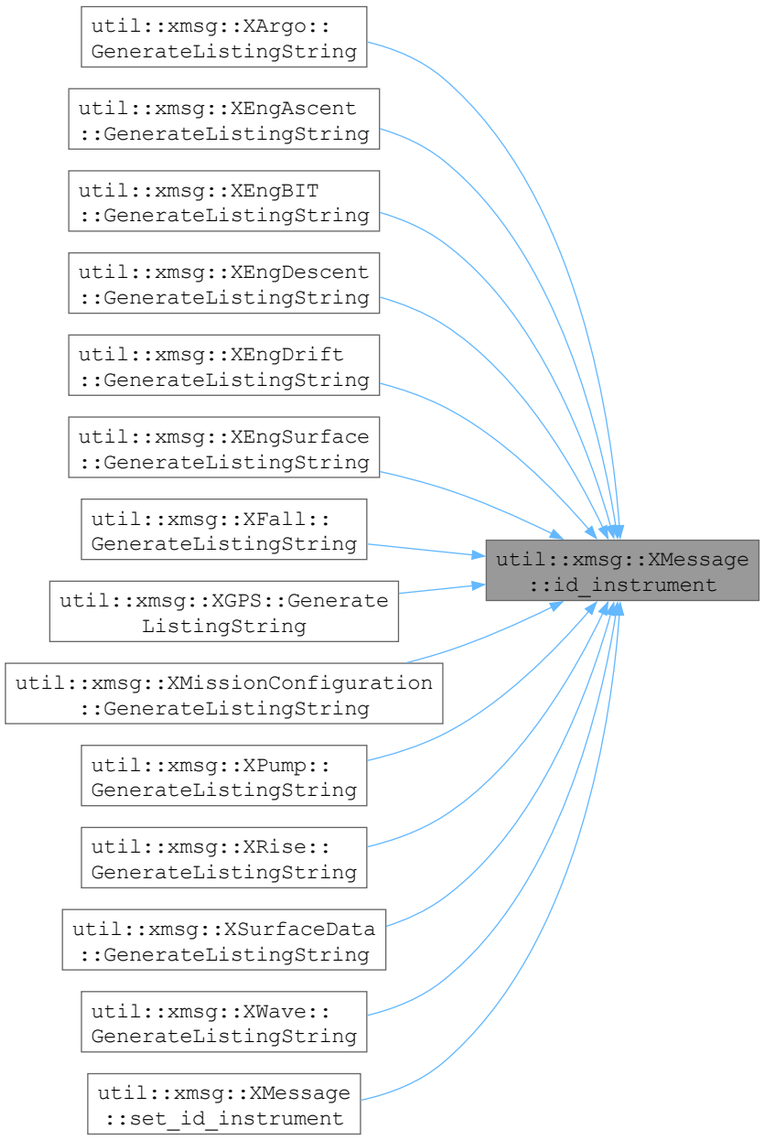
Returns

The unique instrument identifier of this X-Message.

References util::xmsg::XMessage::id_instrument_.

Referenced by util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngBIT::Generate
util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), util::xmsg::XEngSurface::GenerateListin
util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingS
util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(),
util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_instrument().

Here is the caller graph for this function:



10.6.3.5 id_xmsg_type()

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type () const  [inline], [inherited]
```

Get the enumerated X-Message type identifier.

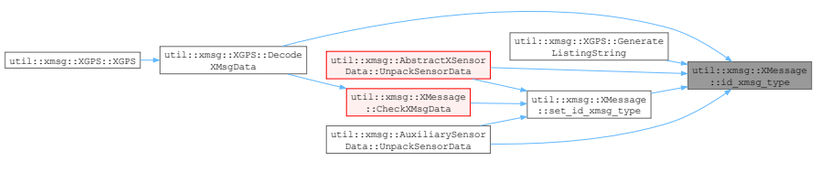
Returns

The enumerated X-Message type identifier.

References util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMessage::set_id_xmsg_typ
util::xmsg::AbstractXSensorData::UnpackSensorData(), and UnpackSensorData().

Here is the caller graph for this function:



10.6.3.6 num_samples()

```
size_t util::xmsg::AbstractXSensorData::num_samples () const  [inline], [inherited]
```

Get the number of samples in this message.

Returns

The number of samples in this message.

References util::xmsg::AbstractXSensorData::raw_sensor_data_.

10.6.3.7 raw_sensor_data()

```
const vector< int32_t > & util::xmsg::AbstractXSensorData::raw_sensor_data () const  [inline],
[inherited]
```

Get the raw sensor data vector.

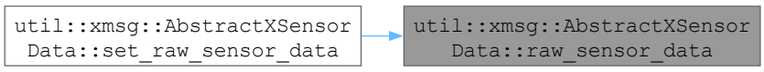
Returns

The raw sensor data vector.

References util::xmsg::AbstractXSensorData::raw_sensor_data_.

Referenced by util::xmsg::AbstractXSensorData::set_raw_sensor_data().

Here is the caller graph for this function:



10.6.3.8 Sample()

```
int32_t util::xmsg::AbstractXSensorData::Sample (
            size_t i) const  [inline], [inherited]
```

Get the raw integer value of the i^{th} sample.

Parameters

in	*i*	The index of the sensor value.

Returns

The raw integer value of the i^{th} ith sample.

References util::xmsg::AbstractXSensorData::raw_sensor_data_.

10.6.3.9 SampleValue()

```
template<typename T = double>
T util::xmsg::AbstractXSensorData::SampleValue (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline], [inherited]
```

Get the i^{th} sensor value in scientific units.

Parameters

in	*i*	The index of the sensor value.
in	*gain*	The sensor sensitivity scale factor.
in	*offs*	The sensor offset.

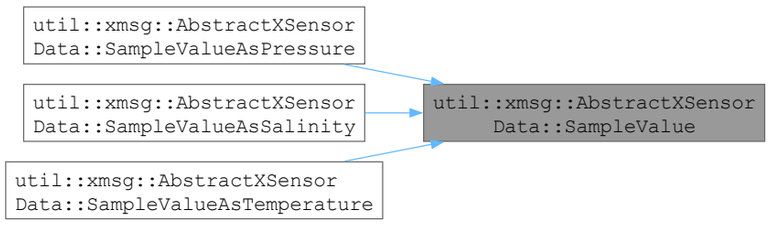
Returns

The i^{th} sensor value in scientific units.

References CATCH_BLOCK, and util::xmsg::AbstractXSensorData::raw_sensor_data_.

Referenced by util::xmsg::AbstractXSensorData::SampleValueAsPressure(), util::xmsg::AbstractXSensorData::SampleValueAsSalinity(), and util::xmsg::AbstractXSensorData::SampleValueAsTemperature().

Here is the caller graph for this function:



10.6.3.10 SampleValueAsPressure()

```
template<typename T = double>
T util::xmsg::AbstractXSensorData::SampleValueAsPressure (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline], [inherited]
```

Get the i^{th} pressure sensor value, in dBar.

Parameters

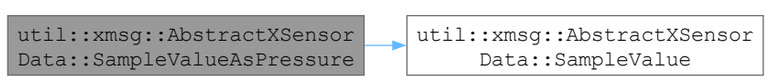
in	*i*	The index of the sensor value.
in	*gain*	The pressure sensor sensitivity scale factor, in LSB/dBar.
in	*offs*	The pressure sensor offset, in dBar.

Returns

The i^{th} pressure sensor value, in dBar.

References util::xmsg::AbstractXSensorData::SampleValue(), XMSG_SENS_BAD_PRES, and XMSG_SENS_MAX_PRES.

Here is the call graph for this function:



10.6.3.11 SampleValueAsSalinity()

```
template<typename T = double>
T util::xmsg::AbstractXSensorData::SampleValueAsSalinity (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline], [inherited]
```

Get the i^{th} salinity sensor value, in PSU.

Parameters

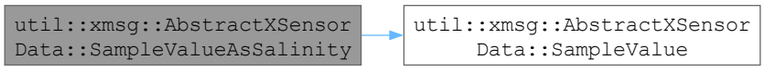
in	*i*	The index of the sensor value.
in	*gain*	The salinity sensor sensitivity scale factor, in LSB/PSU.
in	*offs*	The salinity sensor offset, in PSU.

Returns

The i^{th} salinity sensor value, in PSU.

References util::xmsg::AbstractXSensorData::SampleValue(), XMSG_SENS_BAD_SALN, and XMSG_SENS_MAX_SALN.

Here is the call graph for this function:



10.6.3.12 SampleValueAsTemperature()

```
template<typename T = double>
T util::xmsg::AbstractXSensorData::SampleValueAsTemperature (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline], [inherited]
```

Get the i^{th} temperature sensor value, in degC.

Parameters

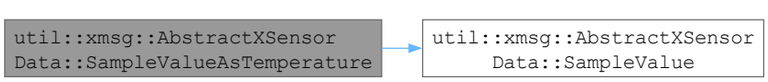
in	*i*	The index of the sensor value.
in	*gain*	The temperature sensor sensitivity scale factor, in LSB/degC.
in	*offs*	The temperature sensor offset, in degC.

Returns

The i^{th} temperature sensor value, in degC.

References util::xmsg::AbstractXSensorData::SampleValue(), XMSG_SENS_BAD_TEMP, and XMSG_SENS_MAX_TEMP.

Here is the call graph for this function:



10.6.3.13 sensor_data_format()

```
XSensorDataFormat_t util::xmsg::AbstractXSensorData::sensor_data_format () const  [inline], [inherited]
```

Get the enumerated sensor data format value.

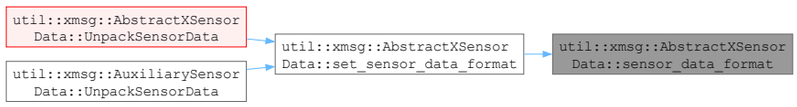
Returns

The enumerated sensor data format value.

References util::xmsg::AbstractXSensorData::sensor_data_format_.

Referenced by util::xmsg::AbstractXSensorData::set_sensor_data_format().

Here is the caller graph for this function:



10.6.3.14 set_id_dive()

```
void util::xmsg::XMessage::set_id_dive (
            int16_t id_dive)  [inline], [protected], [inherited]
```

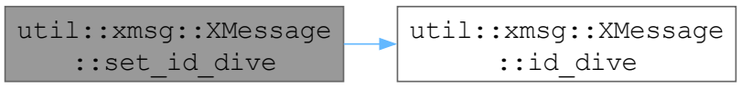
Set the dive or cycle number of this message.

Parameters

in	*id_dive*	The dive or cycle number of this message to set.

References util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::id_dive_.

Here is the call graph for this function:



10.6.3.15 set_id_instrument()

```
void util::xmsg::XMessage::set_id_instrument (
            uint16_t id_instrument)  [inline], [protected], [inherited]
```

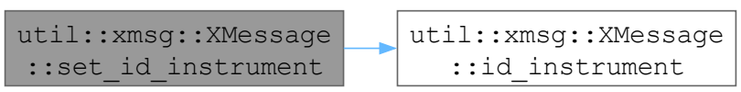
Set the unique instrument identifier.

Parameters

in	*id_instrument*	The unique instrument identifier to set.

References util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::id_instrument_.

Here is the call graph for this function:



10.6.3.16 set_id_xmsg_type()

```
void util::xmsg::XMessage::set_id_xmsg_type (
            XMsgTypeId_t id_xmsg_type)  [inline], [protected], [inherited]
```

Set the X-Message type identifier.

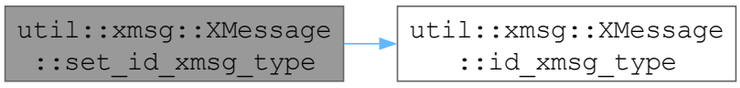
Parameters

in	*id_xmsg_type*	The X-Message type identifier to set.

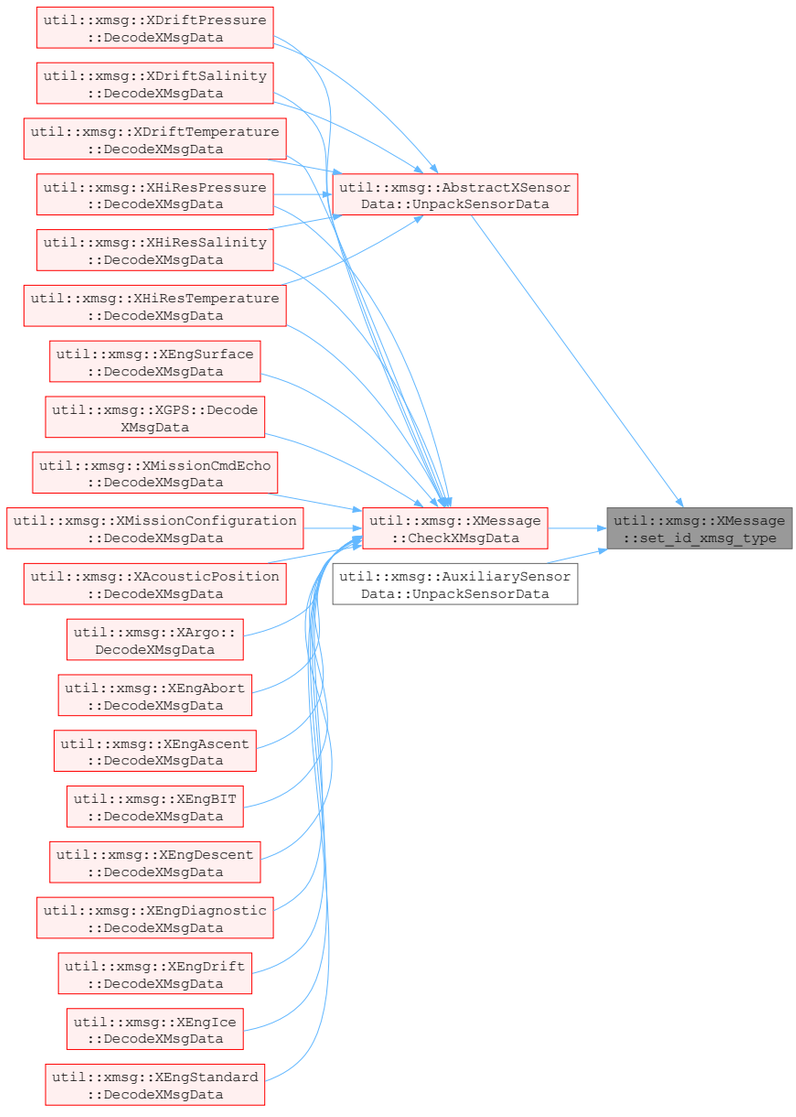
References util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::AbstractXSensorData::UnpackSensorData(), and UnpackSensorData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.6.3.17 set_raw_sensor_data()

```
void util::xmsg::AbstractXSensorData::set_raw_sensor_data (
            const vector< int32_t > & raw_sensor_data)  [inline], [protected], [inherited]
```

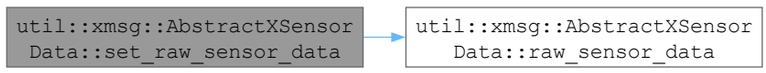
Set raw integer counts of the measured sensor data.

Parameters

in	*raw_sensor_data*	The data to set.

References util::xmsg::AbstractXSensorData::raw_sensor_data(), and util::xmsg::AbstractXSensorData::raw_sensor_data_.

Here is the call graph for this function:



10.6.3.18 set_sensor_data_format()

```
void util::xmsg::AbstractXSensorData::set_sensor_data_format (
            XSensorDataFormat_t sensor_data_format)  [inline], [protected], [inherited]
```

Set the sensor data format.

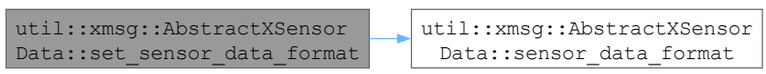
Parameters

in	*sensor_data_format*	The sensor data format to set.

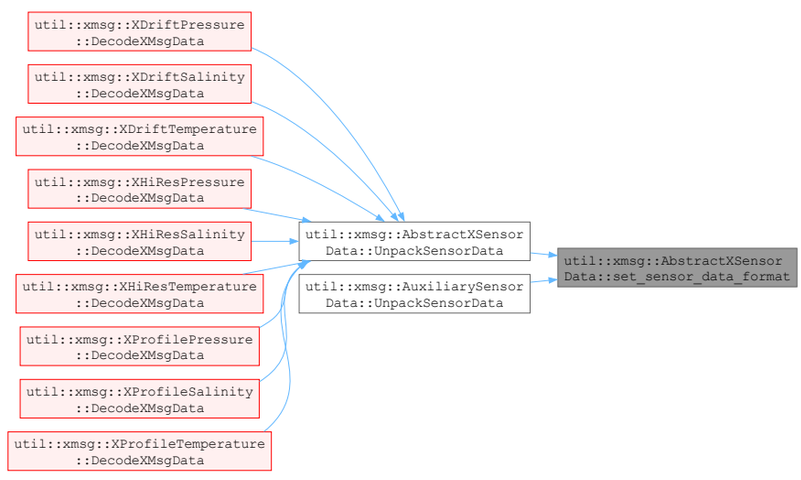
References util::xmsg::AbstractXSensorData::sensor_data_format(), and util::xmsg::AbstractXSensorData::sensor_data_format_.

Referenced by util::xmsg::AbstractXSensorData::UnpackSensorData(), and UnpackSensorData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.6.3.19 set_xmsg_index()

```
void util::xmsg::XMessageIndexable::set_xmsg_index (
            uint8_t xmsg_index)  [inline], [protected], [inherited]
```

Set the index of this X-Message.

Parameters

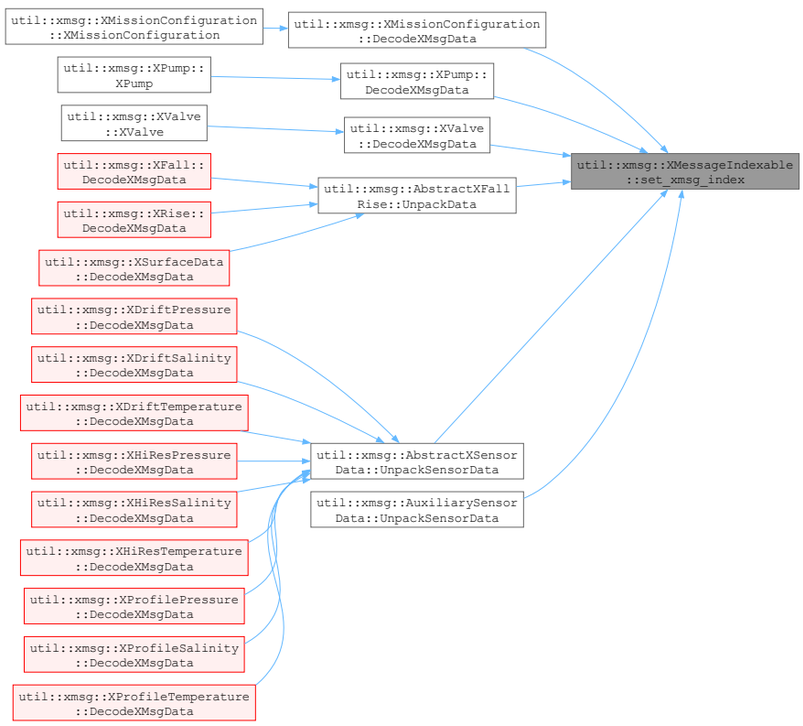
in	*xmsg_index*	The index of this X-Message.

References util::xmsg::XMessageIndexable::xmsg_index(), and util::xmsg::XMessageIndexable::xmsg_index_.

Referenced by util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(), util::xmsg::XValve::DecodeXMsgData(), util::xmsg::AbstractXFallRise::UnpackData(), util::xmsg::AbstractXSensorData::UnpackSensorDa and UnpackSensorData().

Here is the call graph for this function:

Here is the caller graph for this function:



10.6.3.20 set_xmsg_size_bytes()

```
void util::xmsg::XMessage::set_xmsg_size_bytes (
            uint16_t xmsg_size_bytes)  [inline], [protected], [inherited]
```

Set the size of the X-Message, in bytes.

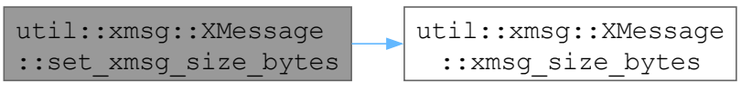
Parameters

in	*xmsg_size_bytes*	The size of the X-Message to set, in bytes.

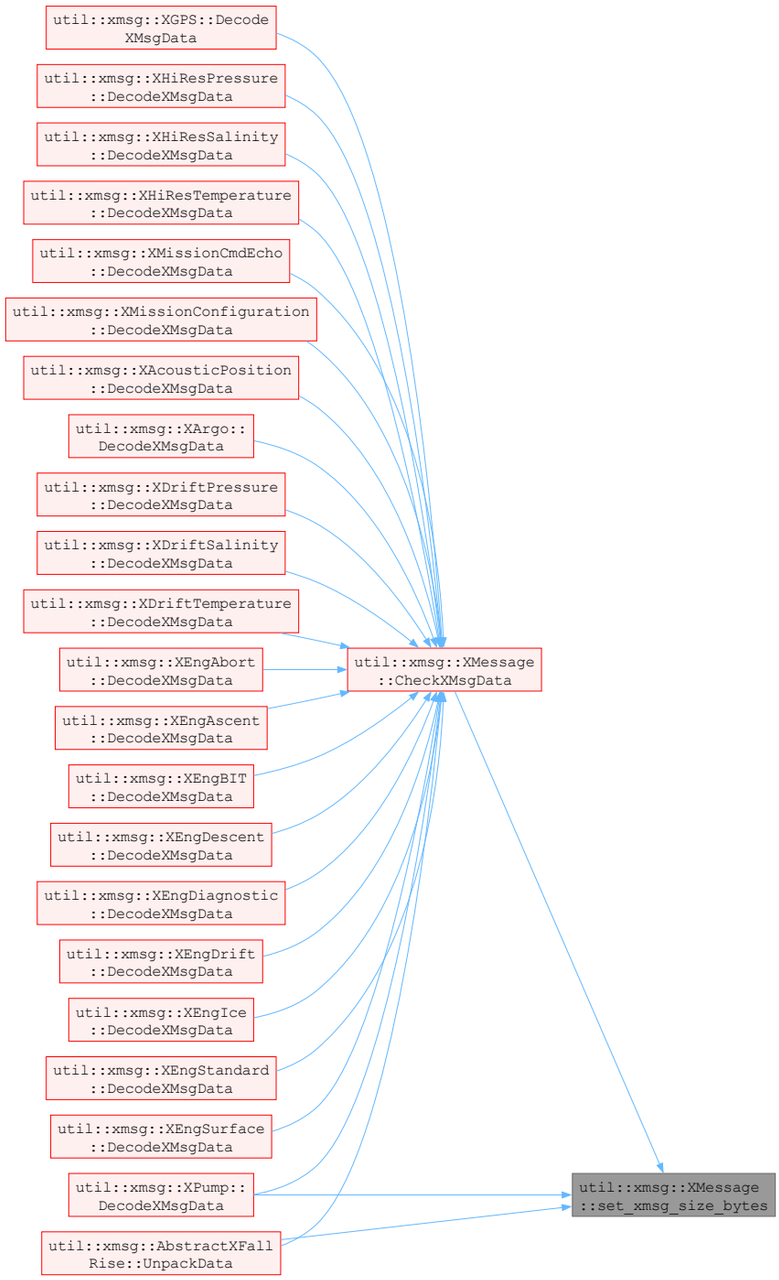
References util::xmsg::XMessage::xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XPump::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::Unpa

Here is the call graph for this function:



Here is the caller graph for this function:



10.6.3.21 UnpackSensorData()

```
vector< int32_t > util::xmsg::AuxiliarySensorData::UnpackSensorData (
            const vector< uint8_t > & data)  [inline], [protected], [virtual]
```

Unpack the sensor data from the binary message.

This field unpacks the encoded sensor data from the message and stores the decoded sensor values in raw_sensor_data_. First the function checks the size of the message, and the data packing format. The message data is decoded according to the packing format type. Both first-difference and curvature encoding are supported. The encoding method is determined from the value of the high nibble of byte `data`[1].

Once this function completes successfully, then the data may be recovered in scientific units from the raw sensor values by applying the appropriate sensor gain and offset values.

Parameters

in	*data*	The binary message to unpack, in a vector of unsigned bytes.

Warning

The function assumes the message data is valid. Message validity should be ensured before this function is called.

Exceptions

util::log::MessageException	if • an error occurs while decoding the message, • the message is of insufficient length, or • an invalid packing format is encountered.

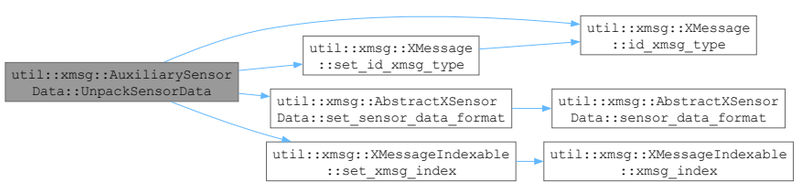
Returns

The sensor data raw counts, in a vector of signed integers.

Reimplemented from util::xmsg::AbstractXSensorData.

References CATCH_BLOCK, CATCH_NOTHROW, EXC_THROW, i24, util::xmsg::XMessage::id_xmsg_type(), ldbg, util::xmsg::AbstractXSensorData::raw_sensor_data_, util::xmsg::XMessage::set_id_xmsg_type(), util::xmsg::AbstractXSensorData::set_s util::xmsg::XMessageIndexable::set_xmsg_index(), u16, u24, XMSG_SENS_MAX_SUB, util::xmsg::XMSG_TYPE_ID_PRES0, util::xmsg::XMSG_TYPE_ID_PRES_DRIFT8, util::xmsg::XMSG_TYPE_ID_PRES_HI_RES0, util::xmsg::XMSG_TYPE_ID_SALN0, util::xmsg::XMSG_TYPE_ID_SALN_DRIFT8, util::xmsg::XMSG_TYPE_ID_SALN_HI_RES0, util::xmsg::XMSG_TYPE_ID_TEMP0, util::xmsg::XMSG_TYPE_ID_TEMP_DRIFT8, util::xmsg::XMSG_TYPE_ID_TEMP_HI_RES0, util::xmsg::XSENS_FMT_CURVATURE, and util::xmsg::XSENS_FMT_DIFFERENCE.

Here is the call graph for this function:



10.6.3.22 xmsg_index()

```
uint8_t util::xmsg::XMessageIndexable::xmsg_index () const  [inline], [inherited]
```

Get the index of this X-Message.

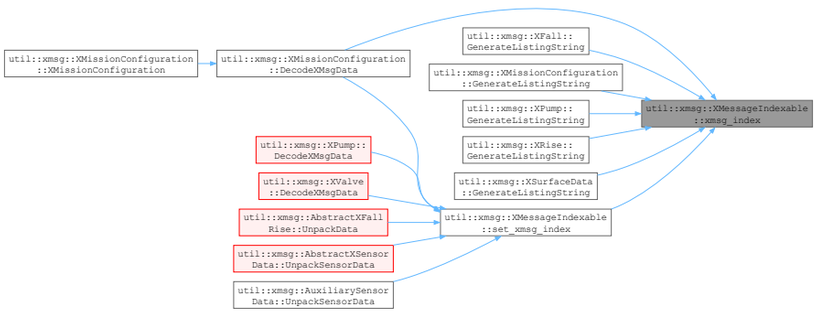
Returns

The index of this X-Message.

References util::xmsg::XMessageIndexable::xmsg_index_.

Referenced by util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing util::xmsg::XSurfaceData::GenerateListingString(), and util::xmsg::XMessageIndexable::set_xmsg_index().

Here is the caller graph for this function:



10.6.3.23 xmsg_size_bytes()

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes () const  [inline], [inherited]
```

Get the size of the X-Message, in bytes.

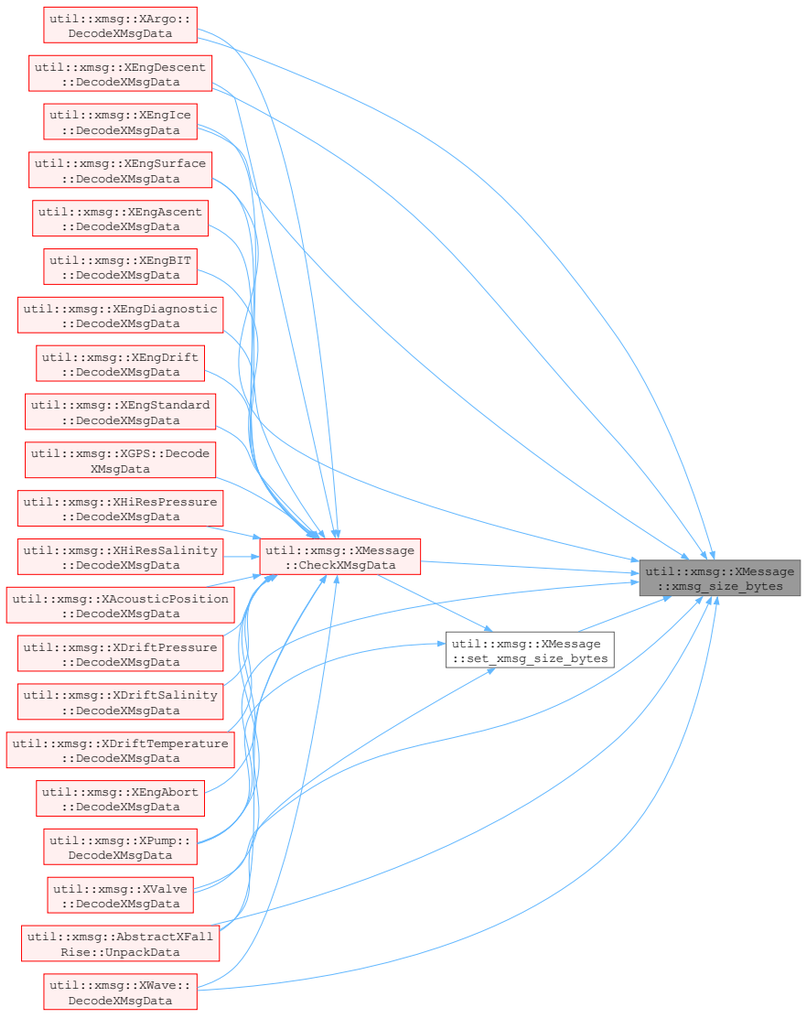
Returns

The size of the X-Message, in bytes.

References util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgI util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(), util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the caller graph for this function:



10.6.4 Member Data Documentation

10.6.4.1 id_dive_

```
int16_t util::xmsg::XMessage::id_dive_  [private], [inherited]
```

The dive or cycle number of this message.

Referenced by util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::set_id_dive().

10.6.4.2 id_instrument_

```
uint16_t util::xmsg::XMessage::id_instrument_  [private], [inherited]
```

The unique WHOI identifier of this instrument.

Referenced by util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::set_id_instrument().

10.6.4.3 id_xmsg_type_

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type_  [private], [inherited]
```

The X-message type identifier.

Referenced by util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::set_id_xmsg_type().

10.6.4.4 raw_sensor_data_

```
vector<int32_t> util::xmsg::AbstractXSensorData::raw_sensor_data_  [protected], [inherited]
```

The raw integer counts of the measured sensor data.

This vector of integers represents the "raw" sample values of the message. That is, by applying the appropriate gain and offset corrections, the sensor values may be recovered in scientific units. This data may represent pressure, temperature, salinity, or any other data that is recorded during the drift or profile phases of the dive cycle.

Referenced by util::xmsg::AbstractXSensorData::num_samples(), util::xmsg::AbstractXSensorData::raw_sensor_data(), util::xmsg::AbstractXSensorData::Sample(), util::xmsg::AbstractXSensorData::SampleValue(), util::xmsg::AbstractXSensorData::set_raw_ util::xmsg::AbstractXSensorData::UnpackSensorData(), and UnpackSensorData().

10.6.4.5 sensor_data_format_

```
XSensorDataFormat_t util::xmsg::AbstractXSensorData::sensor_data_format_  [protected], [inherited]
```

The enumerated sensor data packing format.

This field indicates the format of encoded sensor data. The manner in which the sensor data is decoded is determined by the format of this field.

Referenced by util::xmsg::AbstractXSensorData::sensor_data_format(), and util::xmsg::AbstractXSensorData::set_sensor_data_format().

10.6.4.6 xmsg_index_

```
uint8_t util::xmsg::XMessageIndexable::xmsg_index_  [private], [inherited]
```

The index of this X-Message.

Referenced by util::xmsg::XMessageIndexable::set_xmsg_index(), and util::xmsg::XMessageIndexable::xmsg_index().

10.6.4.7 xmsg_size_bytes_

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes_  [private], [inherited]
```

The size of the X message, in bytes.

Referenced by util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes().

The documentation for this class was generated from the following file:

- data/xmsg/AuxiliarySensorData.h

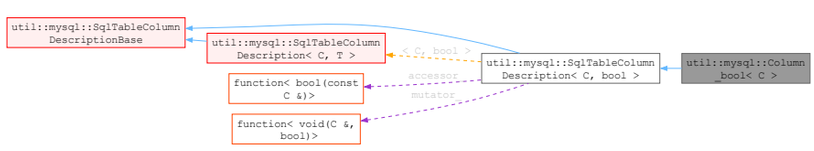
10.7 util::mysql::Column_bool< C > Class Template Reference

Template column description class for value of type bool.

```
#include <SqlTableColumnDescription.h>
```

Inheritance diagram for util::mysql::Column_bool< C >:

Collaboration diagram for util::mysql::Column_bool< C >:



Public Member Functions

- Column_bool (string column_name, bool(C::*accessor)() const =NULL, void(C::*mutator)(bool)=NULL, bool is_primary_key=false)

  The parameterized constructor.
- size_t column_index () const

  Get the index of the SQL table column.
- const string & column_name () const

  Get the name of the SQL table column.
- string column_select_name () const

  Get the name of the SQL table column to be used in a `SELECT` query.
- ColumDataType_t data_type () const

  Get the the enumerated SQL column data type.
- bool is_primary_key () const

  Is this column a primary key for the corresponding SQL database table?
- void set_column_index (size_t idx)

  Set the index of this SQL table column.
- void SetObjectValueFromResultSet (C &obj, const ResultSet &results) const

  Set the value in the field of the corresponding object from an SQL ResultSet Object.
- void SetPreparedStatementValueFromObject (const C &obj, PreparedStatement &stmt) const

  Set the value for the SQL PreparedStatement object from the value of the field of the corresponding object.

Protected Member Functions

- bool GetValueFromObject (const C &obj) const

  Get the object member value from the object.
- void SetObjectValue (C &obj, const bool &value) const

  Set the object member value.

Private Attributes

- function< bool(const C &)> accessor_

  The accessor function object wrapper.
- size_t column_index_

  The index of the SQL table column.
- string column_name_

  The name of the SQL table column.
- ColumDataType_t data_type_

  The enumerated SQL table column data type.
- bool is_primary_key_

  Indicates whether the column is part of the primary key for the SQL table.
- function< void(C &, bool)> mutator_

  The mutator function object wrapper.

10.7.1 Detailed Description

template<class C>
class util::mysql::Column_bool< C >

Template column description class for value of type bool.

10.7.2 Constructor & Destructor Documentation

10.7.2.1 Column_bool()

```
template<class C >
util::mysql::Column_bool< C >::Column_bool (
            string column_name,
            bool(C::* accessor )() const = NULL,
            void(C::* mutator )(bool) = NULL,
            bool is_primary_key = false)  [inline]
```

The parameterized constructor.

Parameters

in	*column_name*	The name of the SQL table column.
in	*accessor*	A pointer to the accessor method for acquiring the value of the corresponding object field.
in	*mutator*	A pointer to the mutator method for setting the value of the corresponding object field.
in	*is_primary_key*	TRUE if this column is a primary key for the SQL table, or FALSE otherwise.

10.7.3 Member Function Documentation

10.7.3.1 column_index()

```
size_t util::mysql::SqlTableColumnDescriptionBase::column_index () const  [inline], [inherited]
```

Get the index of the SQL table column.

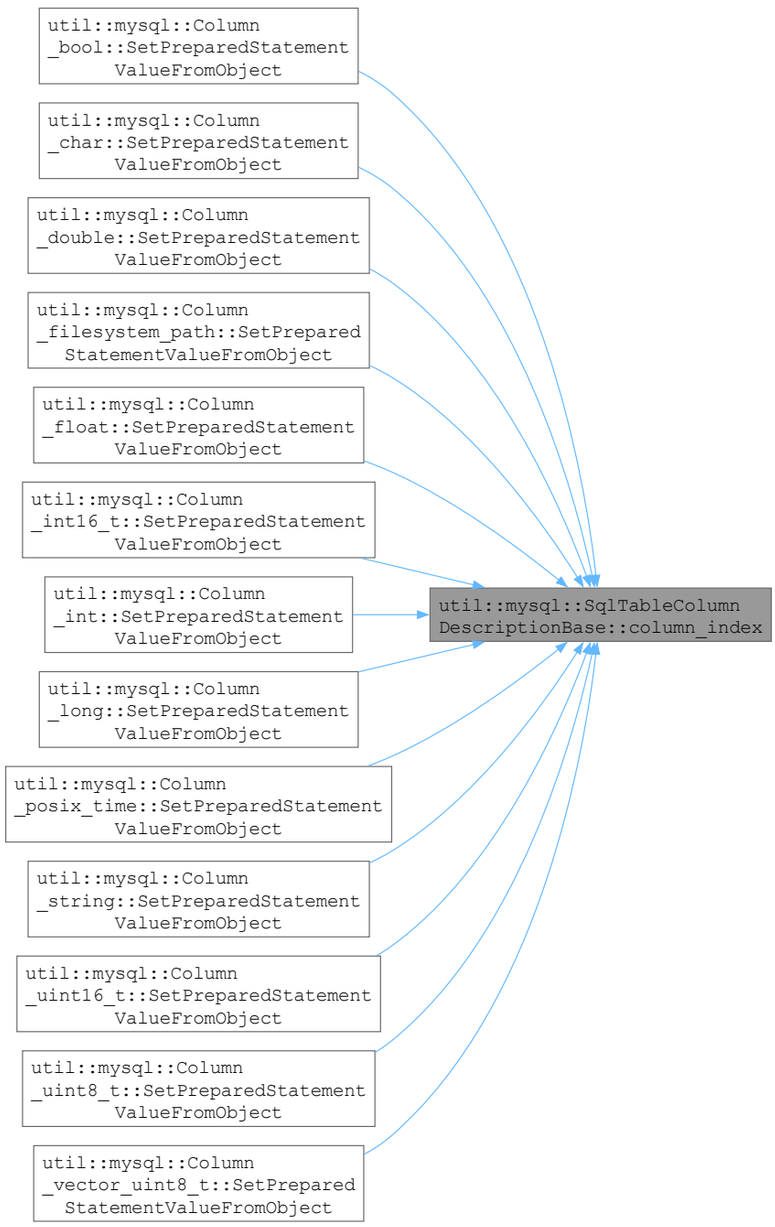
Returns

The index of the SQL table column.

References util::mysql::SqlTableColumnDescriptionBase::column_index_.

Referenced by util::mysql::Column_bool< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_char< C >::SetPreparedS
util::mysql::Column_double< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_filesystem_path< C >::SetPreparedS
util::mysql::Column_float< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_int16_t< C >::SetPreparedStatementVa
util::mysql::Column_int< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_long< C >::SetPreparedStatementValueF
util::mysql::Column_posix_time< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_string< C >::SetPreparedStateme
util::mysql::Column_uint16_t< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_uint8_t< C >::SetPreparedStatemer
and util::mysql::Column_vector_uint8_t< C >::SetPreparedStatementValueFromObject().

Here is the caller graph for this function:



10.7.3.2 column_name()

```
const string & util::mysql::SqlTableColumnDescriptionBase::column_name () const  [inline], [inherited]
```

Get the name of the SQL table column.

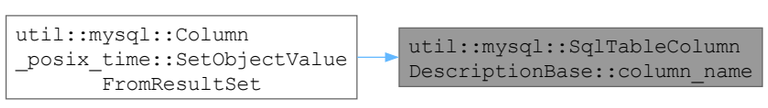
Returns

The name of the SQL table column.

References util::mysql::SqlTableColumnDescriptionBase::column_name_.

Referenced by util::mysql::Column_posix_time< C >::SetObjectValueFromResultSet().

Here is the caller graph for this function:



10.7.3.3 column_select_name()

```
string util::mysql::SqlTableColumnDescriptionBase::column_select_name () const  [inline], [inherited]
```

Get the name of the SQL table column to be used in a `SELECT` query.

Returns

The name of the SQL table column to be used in a `SELECT` query.

References util::mysql::SqlTableColumnDescriptionBase::column_name_, and util::mysql::SqlTableColumnDescriptionBase::data_type_.

10.7.3.4 data_type()

```
ColumDataType_t util::mysql::SqlTableColumnDescriptionBase::data_type () const  [inline], [inherited]
```

Get the the enumerated SQL column data type.

Returns

The the enumerated SQL column data type.

References util::mysql::SqlTableColumnDescriptionBase::data_type_.

10.7.3.5 GetValueFromObject()

```
bool util::mysql::SqlTableColumnDescription< C, bool >::GetValueFromObject (
            const C & obj) const  [inline], [protected], [inherited]
```

Get the object member value from the object.

Parameters

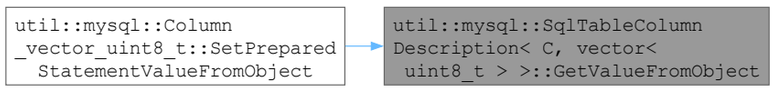
in	*obj*	The object of type `C`.

Returns

The value of type `T` of the object member corresponding to this SQL column.

Referenced by util::mysql::Column_bool< C >::SetPreparedStatementValueFromObject().

Here is the caller graph for this function:



10.7.3.6 is_primary_key()

```
bool util::mysql::SqlTableColumnDescriptionBase::is_primary_key () const  [inline], [inherited]
```

Is this column a primary key for the corresponding SQL database table?

Returns

TRUE if this column is a primary key for the corresponding SQL database table, or FALSE otherwise.

References util::mysql::SqlTableColumnDescriptionBase::is_primary_key_.

10.7.3.7 set_column_index()

```
void util::mysql::SqlTableColumnDescriptionBase::set_column_index (
            size_t idx)  [inline], [inherited]
```

Set the index of this SQL table column.

Parameters

in	*idx*	The index to set.

References util::mysql::SqlTableColumnDescriptionBase::column_index_.

10.7.3.8 SetObjectValue()

```
void util::mysql::SqlTableColumnDescription< C, bool >::SetObjectValue (
            C & obj,
            const bool & value) const  [inline], [protected], [inherited]
```

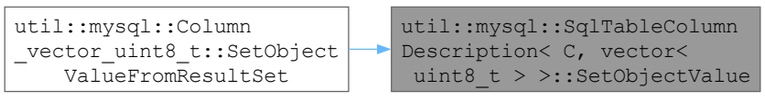
Set the object member value.

Parameters

in	*obj*	The object of type C.
in	*value*	The value of type T, to set.

Referenced by util::mysql::Column_bool< C >::SetObjectValueFromResultSet().

Here is the caller graph for this function:



10.7.3.9 SetObjectValueFromResultSet()

```
template<class C >
void util::mysql::Column_bool< C >::SetObjectValueFromResultSet (
            C & obj,
            const ResultSet & results) const  [inline]
```

Set the value in the field of the corresponding object from an SQL ResultSet Object.

Parameters

in,out	*obj*	A reference to the object whose field value will be set.
in	*results*	A reference to the ResultSet object.

References util::mysql::SqlTableColumnDescription< C, bool >::SetObjectValue().

Here is the call graph for this function:

10.7.3.10 SetPreparedStatementValueFromObject()

```
template<class C >
void util::mysql::Column_bool< C >::SetPreparedStatementValueFromObject (
            const C & obj,
            PreparedStatement & stmt) const  [inline]
```

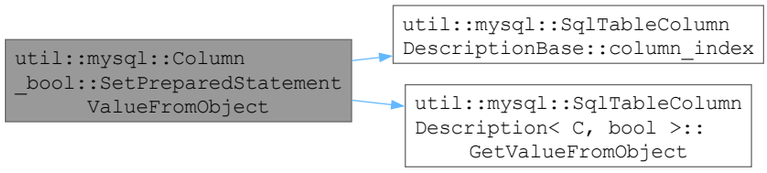
Set the value for the SQL PreparedStatement object from the value of the field of the corresponding object.

Parameters

in	*obj*	A reference to the object whose field value will be read.
in,out	*stmt*	A reference to the SQL PreparedStatement object whose column value will be set.

References util::mysql::SqlTableColumnDescriptionBase::column_index(), and util::mysql::SqlTableColumnDescription< C, bool >::GetVa

Here is the call graph for this function:



10.7.4 Member Data Documentation

10.7.4.1 accessor_

`function<bool(const C&)> util::mysql::SqlTableColumnDescription< C, bool >::accessor_ [private], [inherited]`

The accessor function object wrapper.

This object is used as a wrapper for the accessor method of the template class C. It is used to retrieve the value of the SQL table column from the object for generating an INSERT or UPDATE statement.

10.7.4.2 column_index_

`size_t util::mysql::SqlTableColumnDescriptionBase::column_index_ [private], [inherited]`

The index of the SQL table column.

Referenced by util::mysql::SqlTableColumnDescriptionBase::column_index(), and util::mysql::SqlTableColumnDescriptionBase::set_colum

10.7.4.3 column_name_

`string util::mysql::SqlTableColumnDescriptionBase::column_name_ [private], [inherited]`

The name of the SQL table column.

Referenced by util::mysql::SqlTableColumnDescriptionBase::column_name(), and util::mysql::SqlTableColumnDescriptionBase::column_s

10.7.4.4 data_type_

`ColumDataType_t util::mysql::SqlTableColumnDescriptionBase::data_type_ [private], [inherited]`

The enumerated SQL table column data type.

Referenced by util::mysql::SqlTableColumnDescriptionBase::column_select_name(), and util::mysql::SqlTableColumnDescriptionBase::da

10.7.4.5 is_primary_key_

`bool util::mysql::SqlTableColumnDescriptionBase::is_primary_key_ [private], [inherited]`

Indicates whether the column is part of the primary key for the SQL table.

Referenced by util::mysql::SqlTableColumnDescriptionBase::is_primary_key().

10.7.4.6 mutator_

`function<void(C&, bool)> util::mysql::SqlTableColumnDescription< C, bool >::mutator_ [private], [inherited]`

The mutator function object wrapper.

This object is used as a wrapper for the mutator method of the template class C. It is used to set the value of an object from the result set of an SQL SELECT query statement.

The documentation for this class was generated from the following file:

- utility/SqlTableColumnDescription.h

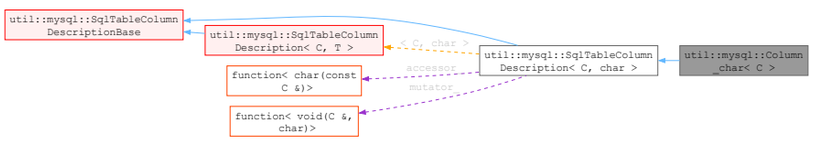
10.8 util::mysql::Column_char< C > Class Template Reference

Template column description class for value of type double.

`#include <SqlTableColumnDescription.h>`

Inheritance diagram for util::mysql::Column_char< C >:

Collaboration diagram for util::mysql::Column_char< C >:



Public Member Functions

- Column_char (string column_name, double(C::∗accessor)() const =NULL, void(C::∗mutator)(double)=NULL, bool is_primary_key=false)

  The parameterized constructor.
- size_t column_index () const

  Get the index of the SQL table column.
- const string & column_name () const

  Get the name of the SQL table column.
- string column_select_name () const

  Get the name of the SQL table column to be used in a `SELECT` query.
- ColumDataType_t data_type () const

  Get the the enumerated SQL column data type.
- bool is_primary_key () const

  Is this column a primary key for the corresponding SQL database table?
- void set_column_index (size_t idx)

  Set the index of this SQL table column.
- void SetObjectValueFromResultSet (C &obj, const ResultSet &results) const

  Set the value in the field of the corresponding object from an SQL ResultSet Object.
- void SetPreparedStatementValueFromObject (const C &obj, PreparedStatement &stmt) const

  Set the value for the SQL PreparedStatement object from the value of the field of the corresponding object.

Protected Member Functions

- char GetValueFromObject (const C &obj) const

  Get the object member value from the object.
- void SetObjectValue (C &obj, const char &value) const

  Set the object member value.

Private Attributes

- function< char(const C &)> accessor_

  The accessor function object wrapper.
- size_t column_index_

  The index of the SQL table column.
- string column_name_

  The name of the SQL table column.
- ColumDataType_t data_type_

  The enumerated SQL table column data type.
- bool is_primary_key_

  Indicates whether the column is part of the primary key for the SQL table.
- function< void(C &, char)> mutator_

  The mutator function object wrapper.

10.8.1 Detailed Description

template<class C>
class util::mysql::Column_char< C >

Template column description class for value of type double.

10.8.2 Constructor & Destructor Documentation

10.8.2.1 Column_char()

```
template<class C >
util::mysql::Column_char< C >::Column_char (
            string column_name,
            double(C::* accessor )() const = NULL,
            void(C::* mutator )(double) = NULL,
            bool is_primary_key = false)  [inline]
```

The parameterized constructor.

Parameters

in	*column_name*	The name of the SQL table column.
in	*accessor*	A pointer to the accessor method for acquiring the value of the corresponding object field.
in	*mutator*	A pointer to the mutator method for setting the value of the corresponding object field.
in	*is_primary_key*	TRUE if this column is a primary key for the SQL table, or FALSE otherwise.

10.8.3 Member Function Documentation

10.8.3.1 column_index()

```
size_t util::mysql::SqlTableColumnDescriptionBase::column_index () const  [inline], [inherited]
```

Get the index of the SQL table column.

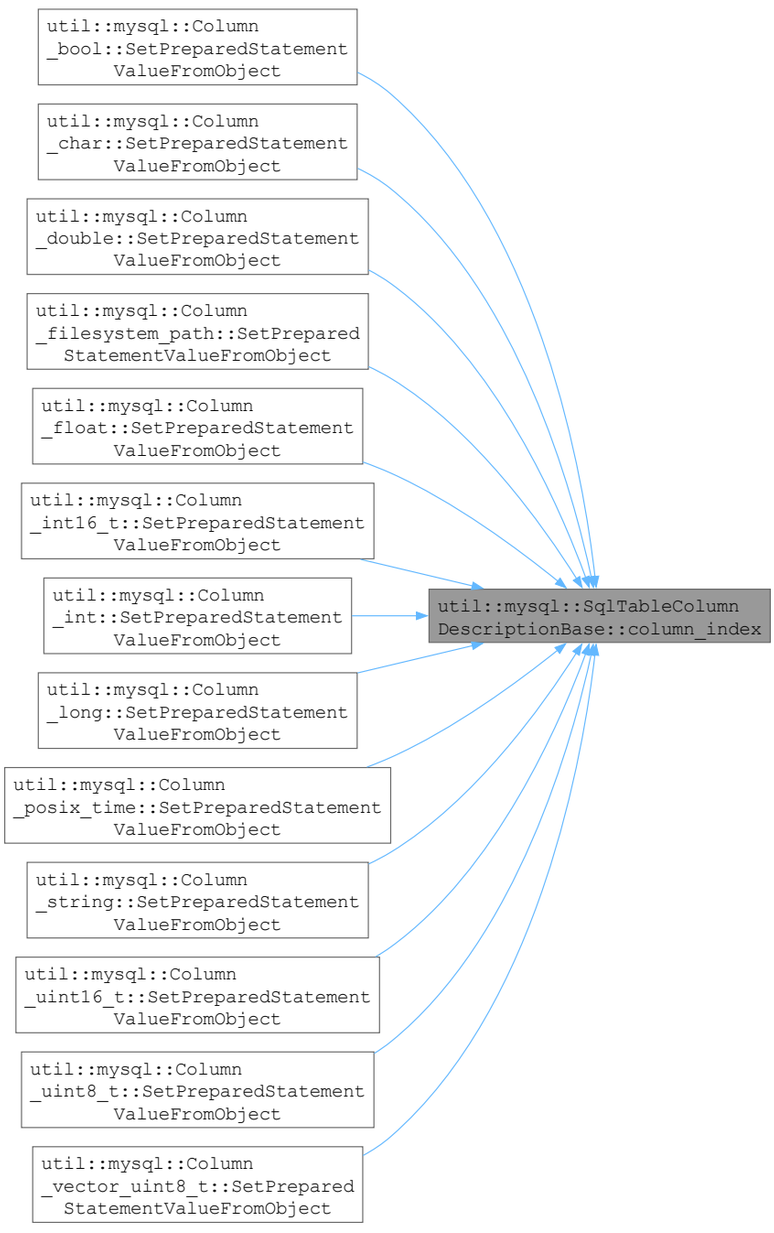
Returns

The index of the SQL table column.

References util::mysql::SqlTableColumnDescriptionBase::column_index_.

Referenced by util::mysql::Column_bool< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_char< C >::SetPreparedS
util::mysql::Column_double< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_filesystem_path< C >::SetPreparedS
util::mysql::Column_float< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_int16_t< C >::SetPreparedStatementVa
util::mysql::Column_int< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_long< C >::SetPreparedStatementValueF
util::mysql::Column_posix_time< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_string< C >::SetPreparedStateme
util::mysql::Column_uint16_t< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_uint8_t< C >::SetPreparedStatemer
and util::mysql::Column_vector_uint8_t< C >::SetPreparedStatementValueFromObject().

Here is the caller graph for this function:



10.8.3.2 column_name()

```
const string & util::mysql::SqlTableColumnDescriptionBase::column_name () const  [inline], [inherited]
```

Get the name of the SQL table column.

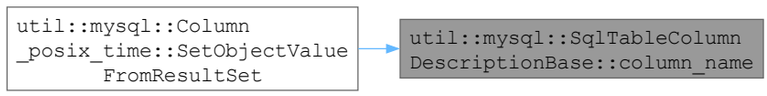
Returns

The name of the SQL table column.

References util::mysql::SqlTableColumnDescriptionBase::column_name_.

Referenced by util::mysql::Column_posix_time< C >::SetObjectValueFromResultSet().

Here is the caller graph for this function:



10.8.3.3 column_select_name()

```
string util::mysql::SqlTableColumnDescriptionBase::column_select_name () const  [inline], [inherited]
```

Get the name of the SQL table column to be used in a `SELECT` query.

Returns

The name of the SQL table column to be used in a `SELECT` query.

References util::mysql::SqlTableColumnDescriptionBase::column_name_, and util::mysql::SqlTableColumnDescriptionBase::data_type_.

10.8.3.4 data_type()

```
ColumDataType_t util::mysql::SqlTableColumnDescriptionBase::data_type () const  [inline], [inherited]
```

Get the the enumerated SQL column data type.

Returns

The the enumerated SQL column data type.

References util::mysql::SqlTableColumnDescriptionBase::data_type_.

10.8.3.5 GetValueFromObject()

```
char util::mysql::SqlTableColumnDescription< C, char >::GetValueFromObject (
            const C & obj) const  [inline], [protected], [inherited]
```

Get the object member value from the object.

Parameters

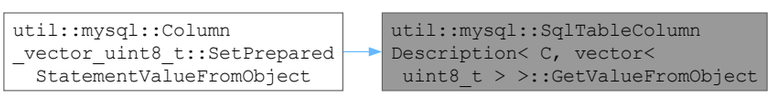
in	*obj*	The object of type C.

Returns

The value of type T of the object member corresponding to this SQL column.

Referenced by util::mysql::Column_char< C >::SetPreparedStatementValueFromObject().

Here is the caller graph for this function:



10.8.3.6 is_primary_key()

```
bool util::mysql::SqlTableColumnDescriptionBase::is_primary_key () const  [inline], [inherited]
```

Is this column a primary key for the corresponding SQL database table?

Returns

TRUE if this column is a primary key for the corresponding SQL database table, or FALSE otherwise.

References util::mysql::SqlTableColumnDescriptionBase::is_primary_key_.

10.8.3.7 set_column_index()

```
void util::mysql::SqlTableColumnDescriptionBase::set_column_index (
            size_t idx)  [inline], [inherited]
```

Set the index of this SQL table column.

Parameters

in	*idx*	The index to set.

References util::mysql::SqlTableColumnDescriptionBase::column_index_.

10.8.3.8 SetObjectValue()

```
void util::mysql::SqlTableColumnDescription< C, char >::SetObjectValue (
            C & obj,
            const char & value) const  [inline], [protected], [inherited]
```

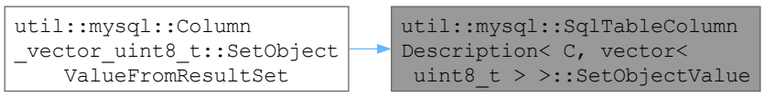
Set the object member value.

Parameters

in	*obj*	The object of type C.
in	*value*	The value of type T, to set.

Referenced by util::mysql::Column_char< C >::SetObjectValueFromResultSet().

Here is the caller graph for this function:



10.8.3.9 SetObjectValueFromResultSet()

```
template<class C >
void util::mysql::Column_char< C >::SetObjectValueFromResultSet (
            C & obj,
            const ResultSet & results) const  [inline]
```

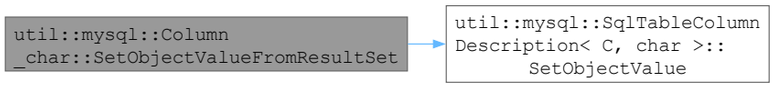
Set the value in the field of the corresponding object from an SQL ResultSet Object.

Parameters

in,out	*obj*	A reference to the object whose field value will be set.
in	*results*	A reference to the ResultSet object.

References util::mysql::SqlTableColumnDescription< C, char >::SetObjectValue().

Here is the call graph for this function:



10.8.3.10 SetPreparedStatementValueFromObject()

```
template<class C >
void util::mysql::Column_char< C >::SetPreparedStatementValueFromObject (
            const C & obj,
            PreparedStatement & stmt) const  [inline]
```

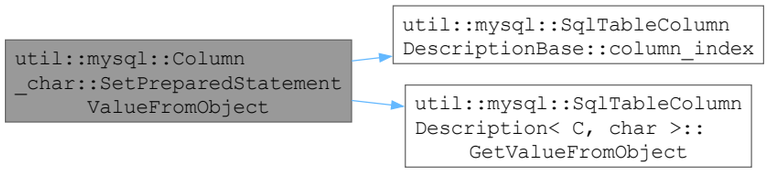
Set the value for the SQL PreparedStatement object from the value of the field of the corresponding object.

Parameters

in	*obj*	A reference to the object whose field value will be read.
in,out	*stmt*	A reference to the SQL PreparedStatement object whose column value will be set.

References util::mysql::SqlTableColumnDescriptionBase::column_index(), and util::mysql::SqlTableColumnDescription< C, char >::GetVa

Here is the call graph for this function:



10.8.4 Member Data Documentation

10.8.4.1 accessor_

```
function<char(const C&)> util::mysql::SqlTableColumnDescription< C, char >::accessor_  [private],
[inherited]
```

The accessor function object wrapper.

This object is used as a wrapper for the accessor method of the template class C. It is used to retrieve the value of the SQL table column from the object for generating an INSERT or UPDATE statement.

10.8.4.2 column_index_

```
size_t util::mysql::SqlTableColumnDescriptionBase::column_index_  [private], [inherited]
```

The index of the SQL table column.

Referenced by util::mysql::SqlTableColumnDescriptionBase::column_index(), and util::mysql::SqlTableColumnDescriptionBase::set_colum

10.8.4.3 column_name_

```
string util::mysql::SqlTableColumnDescriptionBase::column_name_  [private], [inherited]
```

The name of the SQL table column.

Referenced by util::mysql::SqlTableColumnDescriptionBase::column_name(), and util::mysql::SqlTableColumnDescriptionBase::column_s

10.8.4.4 data_type_

```
ColumDataType_t util::mysql::SqlTableColumnDescriptionBase::data_type_  [private], [inherited]
```

The enumerated SQL table column data type.

Referenced by util::mysql::SqlTableColumnDescriptionBase::column_select_name(), and util::mysql::SqlTableColumnDescriptionBase::da

10.8.4.5 is_primary_key_

```
bool util::mysql::SqlTableColumnDescriptionBase::is_primary_key_  [private], [inherited]
```

Indicates whether the column is part of the primary key for the SQL table.

Referenced by util::mysql::SqlTableColumnDescriptionBase::is_primary_key().

10.8.4.6 mutator_

```
function<void(C&, char)> util::mysql::SqlTableColumnDescription< C, char >::mutator_  [private],
[inherited]
```

The mutator function object wrapper.

This object is used as a wrapper for the mutator method of the template class C. It is used to set the value of an object from the result set of an SQL SELECT query statement.

The documentation for this class was generated from the following file:

- utility/SqlTableColumnDescription.h

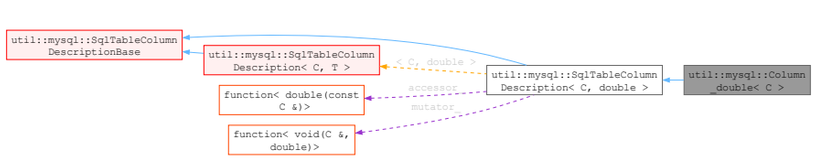
10.9 util::mysql::Column_double< C > Class Template Reference

Template column description class for value of type double.

```
#include <SqlTableColumnDescription.h>
```

Inheritance diagram for util::mysql::Column_double< C >:

Collaboration diagram for util::mysql::Column_double< C >:



Public Member Functions

- Column_double (string column_name, double(C::∗accessor)() const =NULL, void(C::∗mutator)(double)=NULL, bool is_primary_key=false)

  The parameterized constructor.
- size_t column_index () const

  Get the index of the SQL table column.
- const string & column_name () const

  Get the name of the SQL table column.
- string column_select_name () const

  Get the name of the SQL table column to be used in a `SELECT` query.
- ColumDataType_t data_type () const

  Get the the enumerated SQL column data type.
- bool is_primary_key () const

  Is this column a primary key for the corresponding SQL database table?
- void set_column_index (size_t idx)

  Set the index of this SQL table column.
- void SetObjectValueFromResultSet (C &obj, const ResultSet &results) const

  Set the value in the field of the corresponding object from an SQL ResultSet Object.
- void SetPreparedStatementValueFromObject (const C &obj, PreparedStatement &stmt) const

  Set the value for the SQL PreparedStatement object from the value of the field of the corresponding object.

Protected Member Functions

- double GetValueFromObject (const C &obj) const

  Get the object member value from the object.
- void SetObjectValue (C &obj, const double &value) const

  Set the object member value.

Private Attributes

- function< double(const C &)> accessor_

  The accessor function object wrapper.
- size_t column_index_

  The index of the SQL table column.
- string column_name_

  The name of the SQL table column.
- ColumDataType_t data_type_

  The enumerated SQL table column data type.
- bool is_primary_key_

  Indicates whether the column is part of the primary key for the SQL table.
- function< void(C &, double)> mutator_

  The mutator function object wrapper.

10.9.1 Detailed Description

template<class C>
class util::mysql::Column_double< C >

Template column description class for value of type double.

10.9.2 Constructor & Destructor Documentation

10.9.2.1 Column_double()

```
template<class C >
util::mysql::Column_double< C >::Column_double (
            string column_name,
            double(C::* accessor )() const = NULL,
            void(C::* mutator )(double) = NULL,
            bool is_primary_key = false)  [inline]
```

The parameterized constructor.

Parameters

in	*column_name*	The name of the SQL table column.
in	*accessor*	A pointer to the accessor method for acquiring the value of the corresponding object field.
in	*mutator*	A pointer to the mutator method for setting the value of the corresponding object field.
in	*is_primary_key*	TRUE if this column is a primary key for the SQL table, or FALSE otherwise.

10.9.3 Member Function Documentation

10.9.3.1 column_index()

```
size_t util::mysql::SqlTableColumnDescriptionBase::column_index () const  [inline], [inherited]
```

Get the index of the SQL table column.

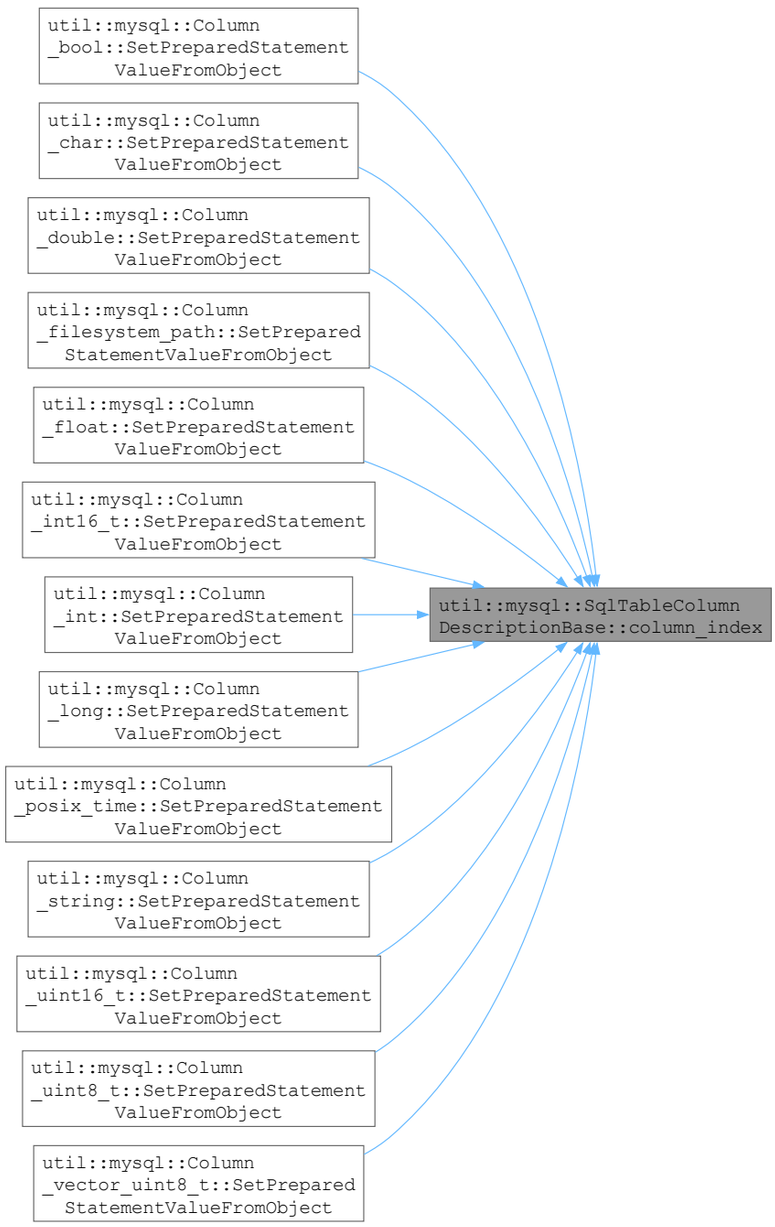
Returns

The index of the SQL table column.

References util::mysql::SqlTableColumnDescriptionBase::column_index_.

Referenced by util::mysql::Column_bool< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_char< C >::SetPreparedS
util::mysql::Column_double< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_filesystem_path< C >::SetPreparedS
util::mysql::Column_float< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_int16_t< C >::SetPreparedStatementVa
util::mysql::Column_int< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_long< C >::SetPreparedStatementValueF
util::mysql::Column_posix_time< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_string< C >::SetPreparedStateme
util::mysql::Column_uint16_t< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_uint8_t< C >::SetPreparedStatemer
and util::mysql::Column_vector_uint8_t< C >::SetPreparedStatementValueFromObject().

Here is the caller graph for this function:



10.9.3.2 column_name()

```
const string & util::mysql::SqlTableColumnDescriptionBase::column_name () const  [inline], [inherited]
```

Get the name of the SQL table column.

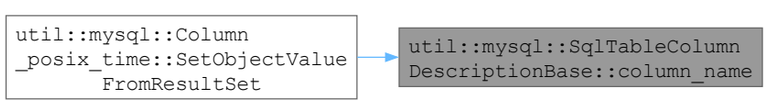
Returns

The name of the SQL table column.

References util::mysql::SqlTableColumnDescriptionBase::column_name_.

Referenced by util::mysql::Column_posix_time< C >::SetObjectValueFromResultSet().

Here is the caller graph for this function:



10.9.3.3 column_select_name()

```
string util::mysql::SqlTableColumnDescriptionBase::column_select_name () const  [inline], [inherited]
```

Get the name of the SQL table column to be used in a SELECT query.

Returns

The name of the SQL table column to be used in a SELECT query.

References util::mysql::SqlTableColumnDescriptionBase::column_name_, and util::mysql::SqlTableColumnDescriptionBase::data_type_.

10.9.3.4 data_type()

```
ColumDataType_t util::mysql::SqlTableColumnDescriptionBase::data_type () const  [inline], [inherited]
```

Get the the enumerated SQL column data type.

Returns

The the enumerated SQL column data type.

References util::mysql::SqlTableColumnDescriptionBase::data_type_.

10.9.3.5 GetValueFromObject()

```
double util::mysql::SqlTableColumnDescription< C, double >::GetValueFromObject (
            const C & obj) const  [inline], [protected], [inherited]
```

Get the object member value from the object.

Parameters

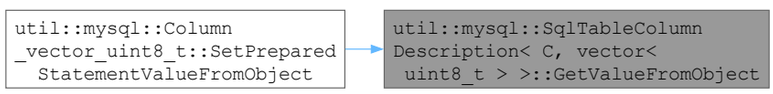
in	*obj*	The object of type C.

Returns

The value of type T of the object member corresponding to this SQL column.

Referenced by util::mysql::Column_double< C >::SetPreparedStatementValueFromObject().

Here is the caller graph for this function:



10.9.3.6 is_primary_key()

```
bool util::mysql::SqlTableColumnDescriptionBase::is_primary_key () const  [inline], [inherited]
```

Is this column a primary key for the corresponding SQL database table?

Returns

TRUE if this column is a primary key for the corresponding SQL database table, or FALSE otherwise.

References util::mysql::SqlTableColumnDescriptionBase::is_primary_key_.

10.9.3.7 set_column_index()

```
void util::mysql::SqlTableColumnDescriptionBase::set_column_index (
            size_t idx)  [inline], [inherited]
```

Set the index of this SQL table column.

Parameters

in	*idx*	The index to set.

References util::mysql::SqlTableColumnDescriptionBase::column_index_.

10.9.3.8 SetObjectValue()

```
void util::mysql::SqlTableColumnDescription< C, double >::SetObjectValue (
            C & obj,
            const double & value) const  [inline], [protected], [inherited]
```

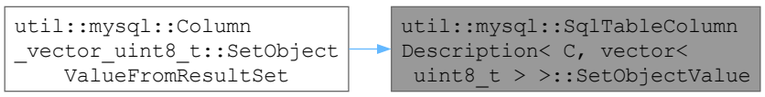
Set the object member value.

Parameters

in	*obj*	The object of type C.
in	*value*	The value of type T, to set.

Referenced by util::mysql::Column_double< C >::SetObjectValueFromResultSet().

Here is the caller graph for this function:



10.9.3.9 SetObjectValueFromResultSet()

```
template<class C >
void util::mysql::Column_double< C >::SetObjectValueFromResultSet (
            C & obj,
            const ResultSet & results) const  [inline]
```

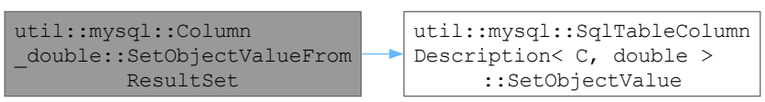
Set the value in the field of the corresponding object from an SQL ResultSet Object.

Parameters

in,out	*obj*	A reference to the object whose field value will be set.
in	*results*	A reference to the ResultSet object.

References util::mysql::SqlTableColumnDescription< C, double >::SetObjectValue().

Here is the call graph for this function:



10.9.3.10 SetPreparedStatementValueFromObject()

```
template<class C >
void util::mysql::Column_double< C >::SetPreparedStatementValueFromObject (
            const C & obj,
            PreparedStatement & stmt) const  [inline]
```

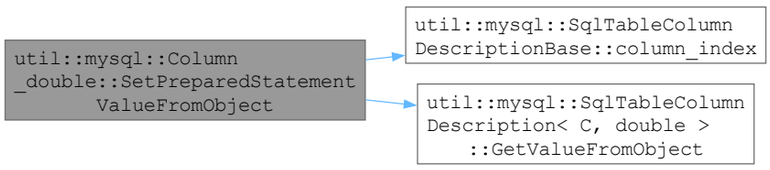
Set the value for the SQL PreparedStatement object from the value of the field of the corresponding object.

Parameters

in	*obj*	A reference to the object whose field value will be read.
in,out	*stmt*	A reference to the SQL PreparedStatement object whose column value will be set.

References util::mysql::SqlTableColumnDescriptionBase::column_index(), and util::mysql::SqlTableColumnDescription< C, double >::Get

Here is the call graph for this function:



10.9.4 Member Data Documentation

10.9.4.1 accessor_

function<double(const C&)> util::mysql::SqlTableColumnDescription< C, double >::accessor_↩
[private], [inherited]

The accessor function object wrapper.

This object is used as a wrapper for the accessor method of the template class C. It is used to retrieve the value of the SQL table column from the object for generating an INSERT or UPDATE statement.

10.9.4.2 column_index_

size_t util::mysql::SqlTableColumnDescriptionBase::column_index_ [private], [inherited]

The index of the SQL table column.

Referenced by util::mysql::SqlTableColumnDescriptionBase::column_index(), and util::mysql::SqlTableColumnDescriptionBase::set_colum

10.9.4.3 column_name_

string util::mysql::SqlTableColumnDescriptionBase::column_name_ [private], [inherited]

The name of the SQL table column.

Referenced by util::mysql::SqlTableColumnDescriptionBase::column_name(), and util::mysql::SqlTableColumnDescriptionBase::column_s

10.9.4.4 data_type_

ColumDataType_t util::mysql::SqlTableColumnDescriptionBase::data_type_ [private], [inherited]

The enumerated SQL table column data type.

Referenced by util::mysql::SqlTableColumnDescriptionBase::column_select_name(), and util::mysql::SqlTableColumnDescriptionBase::da

10.9.4.5 is_primary_key_

bool util::mysql::SqlTableColumnDescriptionBase::is_primary_key_ [private], [inherited]

Indicates whether the column is part of the primary key for the SQL table.

Referenced by util::mysql::SqlTableColumnDescriptionBase::is_primary_key().

10.9.4.6 mutator_

`function<void(C&, double)> util::mysql::SqlTableColumnDescription< C, double >::mutator_ [private], [inherited]`

The mutator function object wrapper.

This object is used as a wrapper for the mutator method of the template class C. It is used to set the value of an object from the result set of an SQL SELECT query statement.

The documentation for this class was generated from the following file:

- utility/SqlTableColumnDescription.h

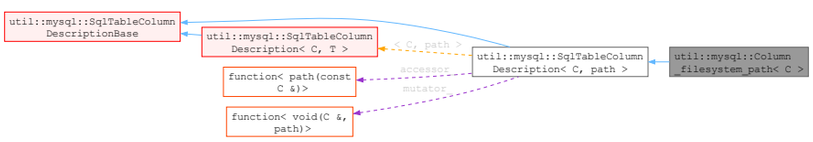
10.10 util::mysql::Column_filesystem_path< C > Class Template Reference

Template column description class for value of type path.

`#include <SqlTableColumnDescription.h>`

Inheritance diagram for util::mysql::Column_filesystem_path< C >:

Collaboration diagram for util::mysql::Column_filesystem_path< C >:



Public Member Functions

- Column_filesystem_path (string column_name, const path &(C::∗accessor)() const =NULL, void(C::∗mutator)(const path &)=NULL, bool is_primary_key=false)

  The parameterized constructor.
- Column_filesystem_path (string column_name, path(C::∗accessor)() const =NULL, void(C::∗mutator)(const path &)=NULL, bool is_primary_key=false)

  The parameterized constructor.
- size_t column_index () const

  Get the index of the SQL table column.
- const string & column_name () const

  Get the name of the SQL table column.
- string column_select_name () const

  Get the name of the SQL table column to be used in a `SELECT` query.
- ColumDataType_t data_type () const

  Get the the enumerated SQL column data type.
- bool is_primary_key () const

  Is this column a primary key for the corresponding SQL database table?
- void set_column_index (size_t idx)

  Set the index of this SQL table column.
- void SetObjectValueFromResultSet (C &obj, const ResultSet &results) const

  Set the value in the field of the corresponding object from an SQL ResultSet Object.
- void SetPreparedStatementValueFromObject (const C &obj, PreparedStatement &stmt) const

  Set the value for the SQL PreparedStatement object from the value of the field of the corresponding object.

Protected Member Functions

- path GetValueFromObject (const C &obj) const
  Get the object member value from the object.
- void SetObjectValue (C &obj, const path &value) const
  Set the object member value.

Private Attributes

- function< path(const C &)> accessor_
  The accessor function object wrapper.
- size_t column_index_
  The index of the SQL table column.
- string column_name_
  The name of the SQL table column.
- ColumDataType_t data_type_
  The enumerated SQL table column data type.
- bool is_primary_key_
  Indicates whether the column is part of the primary key for the SQL table.
- function< void(C &, path)> mutator_
  The mutator function object wrapper.

10.10.1 Detailed Description

template<class C>
class util::mysql::Column_filesystem_path< C >

Template column description class for value of type path.

10.10.2 Constructor & Destructor Documentation

10.10.2.1 Column_filesystem_path() [1/2]

```
template<class C >
util::mysql::Column_filesystem_path< C >::Column_filesystem_path (
            string column_name,
            const path &(C::* accessor )() const = NULL,
            void(C::* mutator )(const path &) = NULL,
            bool is_primary_key = false)  [inline]
```

The parameterized constructor.

Parameters

in	*column_name*	The name of the SQL table column.
in	*accessor*	A pointer to the accessor method for acquiring the value of the corresponding object field.
in	*mutator*	A pointer to the mutator method for setting the value of the corresponding object field.
in	*is_primary_key*	TRUE if this column is a primary key for the SQL table, or FALSE otherwise.

10.10.2.2 Column_filesystem_path() [2/2]

```
template<class C >
util::mysql::Column_filesystem_path< C >::Column_filesystem_path (
            string column_name,
            path(C::* accessor )() const = NULL,
            void(C::* mutator )(const path &) = NULL,
            bool is_primary_key = false)  [inline]
```

The parameterized constructor.

Parameters

in	*column_name*	The name of the SQL table column.
in	*accessor*	A pointer to the accessor method for acquiring the value of the corresponding object field.
in	*mutator*	A pointer to the mutator method for setting the value of the corresponding object field.
in	*is_primary_key*	TRUE if this column is a primary key for the SQL table, or FALSE otherwise.

10.10.3 Member Function Documentation

10.10.3.1 column_index()

```
size_t util::mysql::SqlTableColumnDescriptionBase::column_index () const  [inline], [inherited]
```

Get the index of the SQL table column.

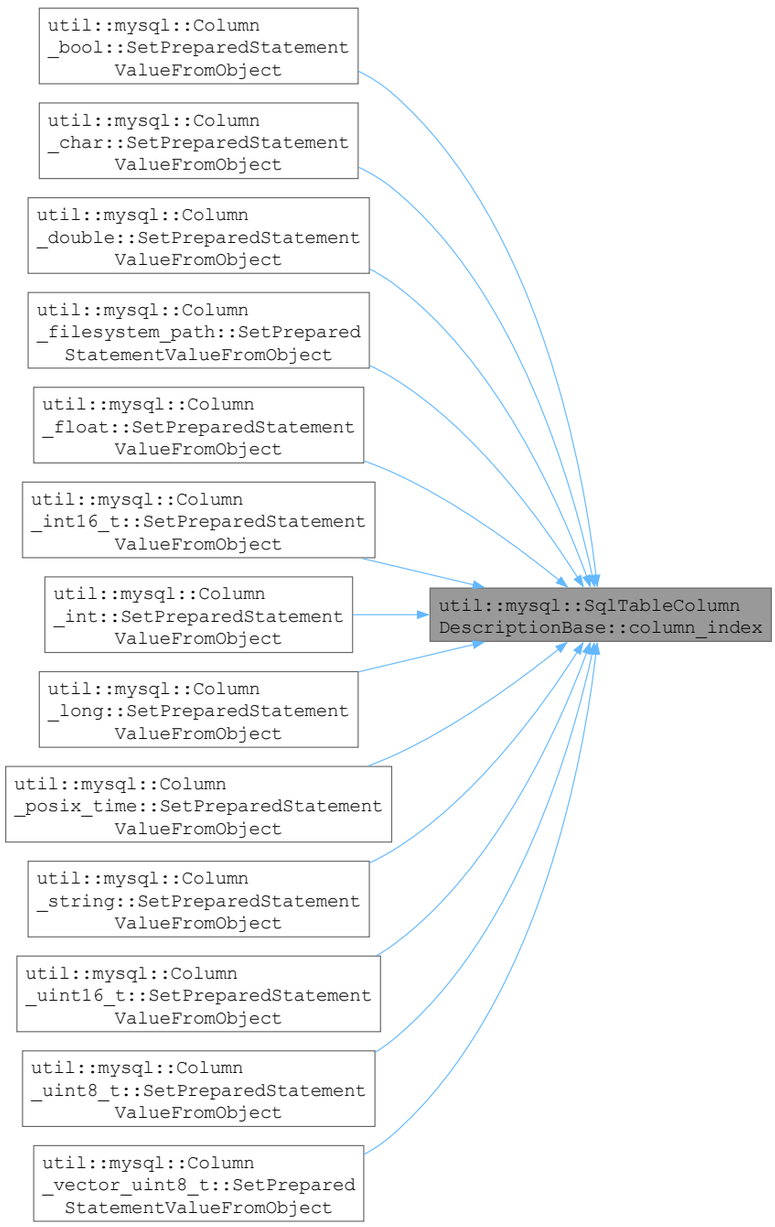
Returns

The index of the SQL table column.

References util::mysql::SqlTableColumnDescriptionBase::column_index_.

Referenced by util::mysql::Column_bool< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_char< C >::SetPreparedS
util::mysql::Column_double< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_filesystem_path< C >::SetPreparedS
util::mysql::Column_float< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_int16_t< C >::SetPreparedStatementVa
util::mysql::Column_int< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_long< C >::SetPreparedStatementValueF
util::mysql::Column_posix_time< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_string< C >::SetPreparedStateme
util::mysql::Column_uint16_t< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_uint8_t< C >::SetPreparedStatemer
and util::mysql::Column_vector_uint8_t< C >::SetPreparedStatementValueFromObject().

Here is the caller graph for this function:



10.10.3.2 column_name()

```
const string & util::mysql::SqlTableColumnDescriptionBase::column_name () const  [inline], [inherited]
```

Get the name of the SQL table column.

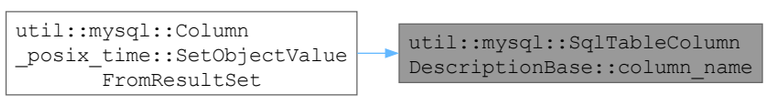
Returns

The name of the SQL table column.

References util::mysql::SqlTableColumnDescriptionBase::column_name_.

Referenced by util::mysql::Column_posix_time< C >::SetObjectValueFromResultSet().

Here is the caller graph for this function:



10.10.3.3 column_select_name()

```
string util::mysql::SqlTableColumnDescriptionBase::column_select_name () const  [inline], [inherited]
```

Get the name of the SQL table column to be used in a `SELECT` query.

Returns

The name of the SQL table column to be used in a `SELECT` query.

References util::mysql::SqlTableColumnDescriptionBase::column_name_, and util::mysql::SqlTableColumnDescriptionBase::data_type_.

10.10.3.4 data_type()

```
ColumDataType_t util::mysql::SqlTableColumnDescriptionBase::data_type () const  [inline], [inherited]
```

Get the the enumerated SQL column data type.

Returns

The the enumerated SQL column data type.

References util::mysql::SqlTableColumnDescriptionBase::data_type_.

10.10.3.5 GetValueFromObject()

```
path util::mysql::SqlTableColumnDescription< C, path >::GetValueFromObject (
            const C & obj) const  [inline], [protected], [inherited]
```

Get the object member value from the object.

Parameters

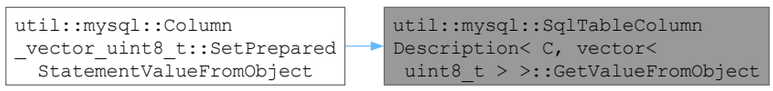
in	*obj*	The object of type `C`.

Returns

The value of type `T` of the object member corresponding to this SQL column.

Referenced by util::mysql::Column_filesystem_path< C >::SetPreparedStatementValueFromObject().

Here is the caller graph for this function:



10.10.3.6 is_primary_key()

```
bool util::mysql::SqlTableColumnDescriptionBase::is_primary_key () const  [inline], [inherited]
```

Is this column a primary key for the corresponding SQL database table?

Returns

TRUE if this column is a primary key for the corresponding SQL database table, or FALSE otherwise.

References util::mysql::SqlTableColumnDescriptionBase::is_primary_key_.

10.10.3.7 set_column_index()

```
void util::mysql::SqlTableColumnDescriptionBase::set_column_index (
            size_t idx)  [inline], [inherited]
```

Set the index of this SQL table column.

Parameters

in	*idx*	The index to set.

References util::mysql::SqlTableColumnDescriptionBase::column_index_.

10.10.3.8 SetObjectValue()

```
void util::mysql::SqlTableColumnDescription< C, path >::SetObjectValue (
            C & obj,
            const path & value) const  [inline], [protected], [inherited]
```

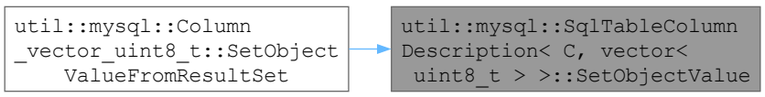
Set the object member value.

Parameters

in	*obj*	The object of type C.
in	*value*	The value of type T, to set.

Referenced by util::mysql::Column_filesystem_path< C >::SetObjectValueFromResultSet().

Here is the caller graph for this function:



10.10.3.9 SetObjectValueFromResultSet()

```
template<class C >
void util::mysql::Column_filesystem_path< C >::SetObjectValueFromResultSet (
            C & obj,
            const ResultSet & results) const  [inline]
```

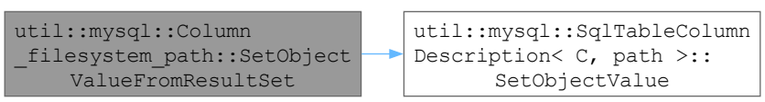
Set the value in the field of the corresponding object from an SQL ResultSet Object.

Parameters

in,out	*obj*	A reference to the object whose field value will be set.
in	*results*	A reference to the ResultSet object.

References util::mysql::SqlTableColumnDescription< C, path >::SetObjectValue().

Here is the call graph for this function:



10.10.3.10 SetPreparedStatementValueFromObject()

```
template<class C >
void util::mysql::Column_filesystem_path< C >::SetPreparedStatementValueFromObject (
            const C & obj,
            PreparedStatement & stmt) const  [inline]
```

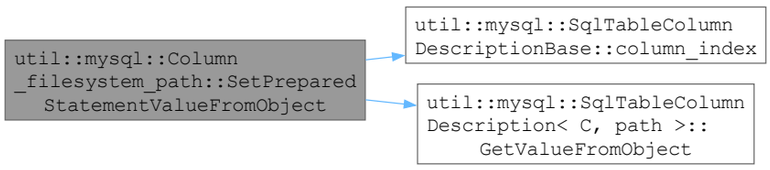
Set the value for the SQL PreparedStatement object from the value of the field of the corresponding object.

Parameters

in	*obj*	A reference to the object whose field value will be read.
in,out	*stmt*	A reference to the SQL PreparedStatement object whose column value will be set.

References util::mysql::SqlTableColumnDescriptionBase::column_index(), and util::mysql::SqlTableColumnDescription< C, path >::GetVa

Here is the call graph for this function:



10.10.4 Member Data Documentation

10.10.4.1 accessor_

```
function<path(const C&)> util::mysql::SqlTableColumnDescription< C, path >::accessor_  [private],
[inherited]
```

The accessor function object wrapper.

This object is used as a wrapper for the accessor method of the template class C. It is used to retrieve the value of the SQL table column from the object for generating an INSERT or UPDATE statement.

10.10.4.2 column_index_

```
size_t util::mysql::SqlTableColumnDescriptionBase::column_index_  [private], [inherited]
```

The index of the SQL table column.

Referenced by util::mysql::SqlTableColumnDescriptionBase::column_index(), and util::mysql::SqlTableColumnDescriptionBase::set_colum

10.10.4.3 column_name_

```
string util::mysql::SqlTableColumnDescriptionBase::column_name_  [private], [inherited]
```

The name of the SQL table column.

Referenced by util::mysql::SqlTableColumnDescriptionBase::column_name(), and util::mysql::SqlTableColumnDescriptionBase::column_s

10.10.4.4 data_type_

```
ColumDataType_t util::mysql::SqlTableColumnDescriptionBase::data_type_  [private], [inherited]
```

The enumerated SQL table column data type.

Referenced by util::mysql::SqlTableColumnDescriptionBase::column_select_name(), and util::mysql::SqlTableColumnDescriptionBase::da

10.10.4.5 is_primary_key_

```
bool util::mysql::SqlTableColumnDescriptionBase::is_primary_key_  [private], [inherited]
```

Indicates whether the column is part of the primary key for the SQL table.

Referenced by util::mysql::SqlTableColumnDescriptionBase::is_primary_key().

10.10.4.6 mutator_

`function<void(C&, path)> util::mysql::SqlTableColumnDescription< C, path >::mutator_ [private], [inherited]`

The mutator function object wrapper.

This object is used as a wrapper for the mutator method of the template class C. It is used to set the value of an object from the result set of an SQL SELECT query statement.

The documentation for this class was generated from the following file:

- utility/SqlTableColumnDescription.h

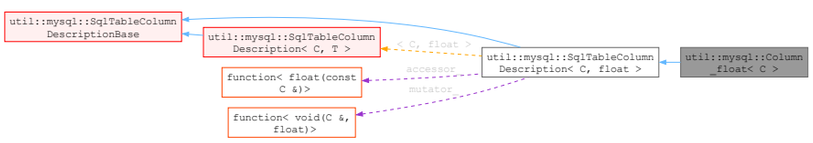
10.11 util::mysql::Column_float< C > Class Template Reference

Template column description class for value of type float.

`#include <SqlTableColumnDescription.h>`

Inheritance diagram for util::mysql::Column_float< C >:

Collaboration diagram for util::mysql::Column_float< C >:



Public Member Functions

- Column_float (string column_name, float(C::∗accessor)() const =NULL, void(C::∗mutator)(float)=NULL, bool is_primary_key=false)

  The parameterized constructor.
- size_t column_index () const

  Get the index of the SQL table column.
- const string & column_name () const

  Get the name of the SQL table column.
- string column_select_name () const

  Get the name of the SQL table column to be used in a `SELECT` query.
- ColumDataType_t data_type () const

  Get the the enumerated SQL column data type.
- bool is_primary_key () const

  Is this column a primary key for the corresponding SQL database table?
- void set_column_index (size_t idx)

  Set the index of this SQL table column.
- void SetObjectValueFromResultSet (C &obj, const ResultSet &results) const

  Set the value in the field of the corresponding object from an SQL ResultSet Object.
- void SetPreparedStatementValueFromObject (const C &obj, PreparedStatement &stmt) const

  Set the value for the SQL PreparedStatement object from the value of the field of the corresponding object.

Protected Member Functions

- float GetValueFromObject (const C &obj) const

  Get the object member value from the object.
- void SetObjectValue (C &obj, const float &value) const

  Set the object member value.

Private Attributes

- function< float(const C &)> accessor_

  The accessor function object wrapper.
- size_t column_index_

  The index of the SQL table column.
- string column_name_

  The name of the SQL table column.
- ColumDataType_t data_type_

  The enumerated SQL table column data type.
- bool is_primary_key_

  Indicates whether the column is part of the primary key for the SQL table.
- function< void(C &, float)> mutator_

  The mutator function object wrapper.

10.11.1 Detailed Description

template<class C>
class util::mysql::Column_float< C >

Template column description class for value of type float.

10.11.2 Constructor & Destructor Documentation

10.11.2.1 Column_float()

```
template<class C >
util::mysql::Column_float< C >::Column_float (
            string column_name,
            float(C::* accessor )() const = NULL,
            void(C::* mutator )(float) = NULL,
            bool is_primary_key = false)  [inline]
```

The parameterized constructor.

Parameters

in	*column_name*	The name of the SQL table column.
in	*accessor*	A pointer to the accessor method for acquiring the value of the corresponding object field.
in	*mutator*	A pointer to the mutator method for setting the value of the corresponding object field.
in	*is_primary_key*	TRUE if this column is a primary key for the SQL table, or FALSE otherwise.

10.11.3 Member Function Documentation

10.11.3.1 column_index()

```
size_t util::mysql::SqlTableColumnDescriptionBase::column_index () const  [inline], [inherited]
```

Get the index of the SQL table column.

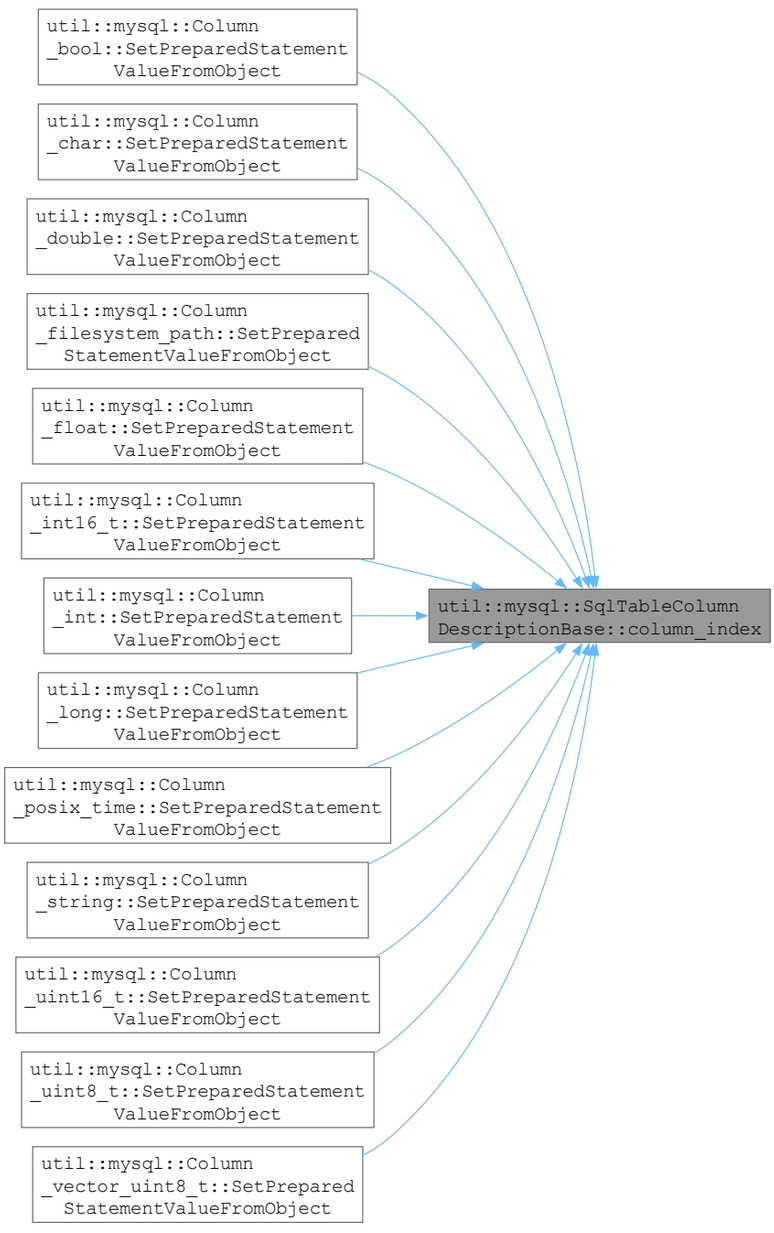
Returns

The index of the SQL table column.

References util::mysql::SqlTableColumnDescriptionBase::column_index_.

Referenced by util::mysql::Column_bool< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_char< C >::SetPreparedS
util::mysql::Column_double< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_filesystem_path< C >::SetPreparedS
util::mysql::Column_float< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_int16_t< C >::SetPreparedStatementVa
util::mysql::Column_int< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_long< C >::SetPreparedStatementValueF
util::mysql::Column_posix_time< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_string< C >::SetPreparedStateme
util::mysql::Column_uint16_t< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_uint8_t< C >::SetPreparedStatemer
and util::mysql::Column_vector_uint8_t< C >::SetPreparedStatementValueFromObject().

Here is the caller graph for this function:



10.11.3.2 column_name()

```
const string & util::mysql::SqlTableColumnDescriptionBase::column_name () const  [inline], [inherited]
```

Get the name of the SQL table column.

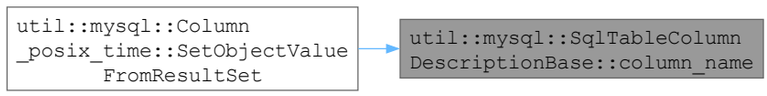
Returns

The name of the SQL table column.

References util::mysql::SqlTableColumnDescriptionBase::column_name_.

Referenced by util::mysql::Column_posix_time< C >::SetObjectValueFromResultSet().

Here is the caller graph for this function:



10.11.3.3 column_select_name()

```
string util::mysql::SqlTableColumnDescriptionBase::column_select_name () const  [inline], [inherited]
```

Get the name of the SQL table column to be used in a `SELECT` query.

Returns

The name of the SQL table column to be used in a `SELECT` query.

References util::mysql::SqlTableColumnDescriptionBase::column_name_, and util::mysql::SqlTableColumnDescriptionBase::data_type_.

10.11.3.4 data_type()

```
ColumDataType_t util::mysql::SqlTableColumnDescriptionBase::data_type () const  [inline], [inherited]
```

Get the the enumerated SQL column data type.

Returns

The the enumerated SQL column data type.

References util::mysql::SqlTableColumnDescriptionBase::data_type_.

10.11.3.5 GetValueFromObject()

```
float util::mysql::SqlTableColumnDescription< C, float >::GetValueFromObject (
            const C & obj) const  [inline], [protected], [inherited]
```

Get the object member value from the object.

Parameters

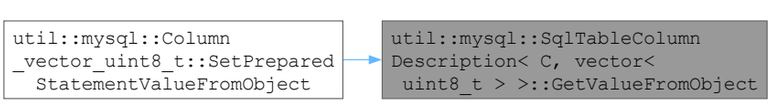
in	*obj*	The object of type C.

Returns

The value of type T of the object member corresponding to this SQL column.

Referenced by util::mysql::Column_float< C >::SetPreparedStatementValueFromObject().

Here is the caller graph for this function:



10.11.3.6 is_primary_key()

```
bool util::mysql::SqlTableColumnDescriptionBase::is_primary_key () const  [inline], [inherited]
```

Is this column a primary key for the corresponding SQL database table?

Returns

TRUE if this column is a primary key for the corresponding SQL database table, or FALSE otherwise.

References util::mysql::SqlTableColumnDescriptionBase::is_primary_key_.

10.11.3.7 set_column_index()

```
void util::mysql::SqlTableColumnDescriptionBase::set_column_index (
            size_t idx)  [inline], [inherited]
```

Set the index of this SQL table column.

Parameters

in	*idx*	The index to set.

References util::mysql::SqlTableColumnDescriptionBase::column_index_.

10.11.3.8 SetObjectValue()

```
void util::mysql::SqlTableColumnDescription< C, float >::SetObjectValue (
            C & obj,
            const float & value) const  [inline], [protected], [inherited]
```

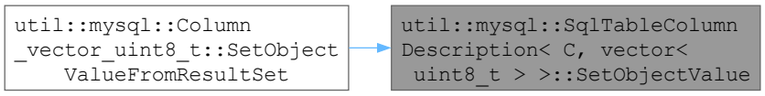
Set the object member value.

Parameters

in	*obj*	The object of type C.
in	*value*	The value of type T, to set.

Referenced by util::mysql::Column_float< C >::SetObjectValueFromResultSet().

Here is the caller graph for this function:



10.11.3.9 SetObjectValueFromResultSet()

```
template<class C >
void util::mysql::Column_float< C >::SetObjectValueFromResultSet (
            C & obj,
            const ResultSet & results) const  [inline]
```

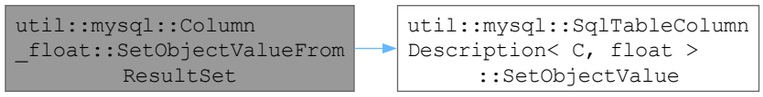
Set the value in the field of the corresponding object from an SQL ResultSet Object.

Parameters

in,out	*obj*	A reference to the object whose field value will be set.
in	*results*	A reference to the ResultSet object.

References util::mysql::SqlTableColumnDescription< C, float >::SetObjectValue().

Here is the call graph for this function:



10.11.3.10 SetPreparedStatementValueFromObject()

```
template<class C >
void util::mysql::Column_float< C >::SetPreparedStatementValueFromObject (
            const C & obj,
            PreparedStatement & stmt) const  [inline]
```

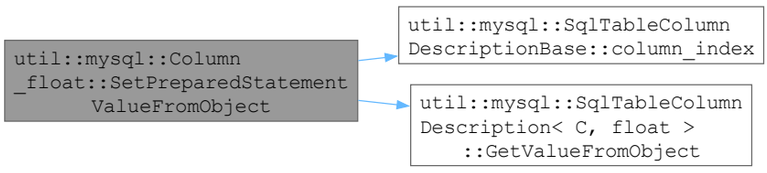
Set the value for the SQL PreparedStatement object from the value of the field of the corresponding object.

Parameters

in	*obj*	A reference to the object whose field value will be read.
in,out	*stmt*	A reference to the SQL PreparedStatement object whose column value will be set.

References util::mysql::SqlTableColumnDescriptionBase::column_index(), and util::mysql::SqlTableColumnDescription< C, float >::GetVa

Here is the call graph for this function:



10.11.4 Member Data Documentation

10.11.4.1 accessor_

```
function<float(const C&)> util::mysql::SqlTableColumnDescription< C, float >::accessor_  [private],
[inherited]
```

The accessor function object wrapper.

This object is used as a wrapper for the accessor method of the template class C. It is used to retrieve the value of the SQL table column from the object for generating an INSERT or UPDATE statement.

10.11.4.2 column_index_

```
size_t util::mysql::SqlTableColumnDescriptionBase::column_index_  [private], [inherited]
```

The index of the SQL table column.

Referenced by util::mysql::SqlTableColumnDescriptionBase::column_index(), and util::mysql::SqlTableColumnDescriptionBase::set_colum

10.11.4.3 column_name_

```
string util::mysql::SqlTableColumnDescriptionBase::column_name_  [private], [inherited]
```

The name of the SQL table column.

Referenced by util::mysql::SqlTableColumnDescriptionBase::column_name(), and util::mysql::SqlTableColumnDescriptionBase::column_s

10.11.4.4 data_type_

```
ColumDataType_t util::mysql::SqlTableColumnDescriptionBase::data_type_  [private], [inherited]
```

The enumerated SQL table column data type.

Referenced by util::mysql::SqlTableColumnDescriptionBase::column_select_name(), and util::mysql::SqlTableColumnDescriptionBase::da

10.11.4.5 is_primary_key_

```
bool util::mysql::SqlTableColumnDescriptionBase::is_primary_key_  [private], [inherited]
```

Indicates whether the column is part of the primary key for the SQL table.

Referenced by util::mysql::SqlTableColumnDescriptionBase::is_primary_key().

10.11.4.6 mutator_

```
function<void(C&, float)> util::mysql::SqlTableColumnDescription< C, float >::mutator_  [private],
[inherited]
```

The mutator function object wrapper.

This object is used as a wrapper for the mutator method of the template class C. It is used to set the value of an object from the result set of an SQL SELECT query statement.

The documentation for this class was generated from the following file:

- utility/SqlTableColumnDescription.h

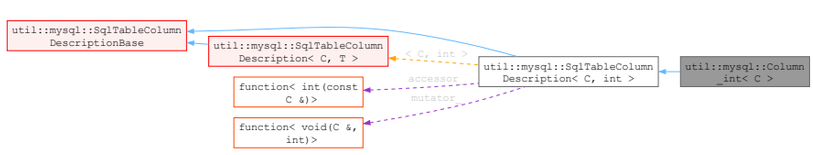
10.12 util::mysql::Column_int< C > Class Template Reference

Template column description class for value of type int.

```
#include <SqlTableColumnDescription.h>
```

Inheritance diagram for util::mysql::Column_int< C >:

Collaboration diagram for util::mysql::Column_int< C >:



Public Member Functions

- Column_int (string column_name, int(C::∗accessor)() const =NULL, void(C::∗mutator)(int)=NULL, bool is_primary_key=false)
  The parameterized constructor.
- size_t column_index () const
  Get the index of the SQL table column.
- const string & column_name () const
  Get the name of the SQL table column.
- string column_select_name () const
  Get the name of the SQL table column to be used in a `SELECT` query.
- ColumDataType_t data_type () const
  Get the the enumerated SQL column data type.
- bool is_primary_key () const
  Is this column a primary key for the corresponding SQL database table?
- void set_column_index (size_t idx)
  Set the index of this SQL table column.
- void SetObjectValueFromResultSet (C &obj, const ResultSet &results) const
  Set the value in the field of the corresponding object from an SQL ResultSet Object.
- void SetPreparedStatementValueFromObject (const C &obj, PreparedStatement &stmt) const
  Set the value for the SQL PreparedStatement object from the value of the field of the corresponding object.

Protected Member Functions

- int GetValueFromObject (const C &obj) const
  Get the object member value from the object.
- void SetObjectValue (C &obj, const int &value) const
  Set the object member value.

Private Attributes

- function< int(const C &)> accessor_
  The accessor function object wrapper.
- size_t column_index_
  The index of the SQL table column.
- string column_name_
  The name of the SQL table column.
- ColumDataType_t data_type_
  The enumerated SQL table column data type.
- bool is_primary_key_
  Indicates whether the column is part of the primary key for the SQL table.
- function< void(C &, int)> mutator_
  The mutator function object wrapper.

10.12.1 Detailed Description

template<class C>
class util::mysql::Column_int< C >

Template column description class for value of type int.

10.12.2 Constructor & Destructor Documentation

10.12.2.1 Column_int()

```
template<class C >
util::mysql::Column_int< C >::Column_int (
            string column_name,
            int(C::* accessor )() const = NULL,
            void(C::* mutator )(int) = NULL,
            bool is_primary_key = false)  [inline]
```

The parameterized constructor.

Parameters

in	*column_name*	The name of the SQL table column.
in	*accessor*	A pointer to the accessor method for acquiring the value of the corresponding object field.
in	*mutator*	A pointer to the mutator method for setting the value of the corresponding object field.
in	*is_primary_key*	TRUE if this column is a primary key for the SQL table, or FALSE otherwise.

10.12.3 Member Function Documentation

10.12.3.1 column_index()

```
size_t util::mysql::SqlTableColumnDescriptionBase::column_index () const  [inline], [inherited]
```

Get the index of the SQL table column.

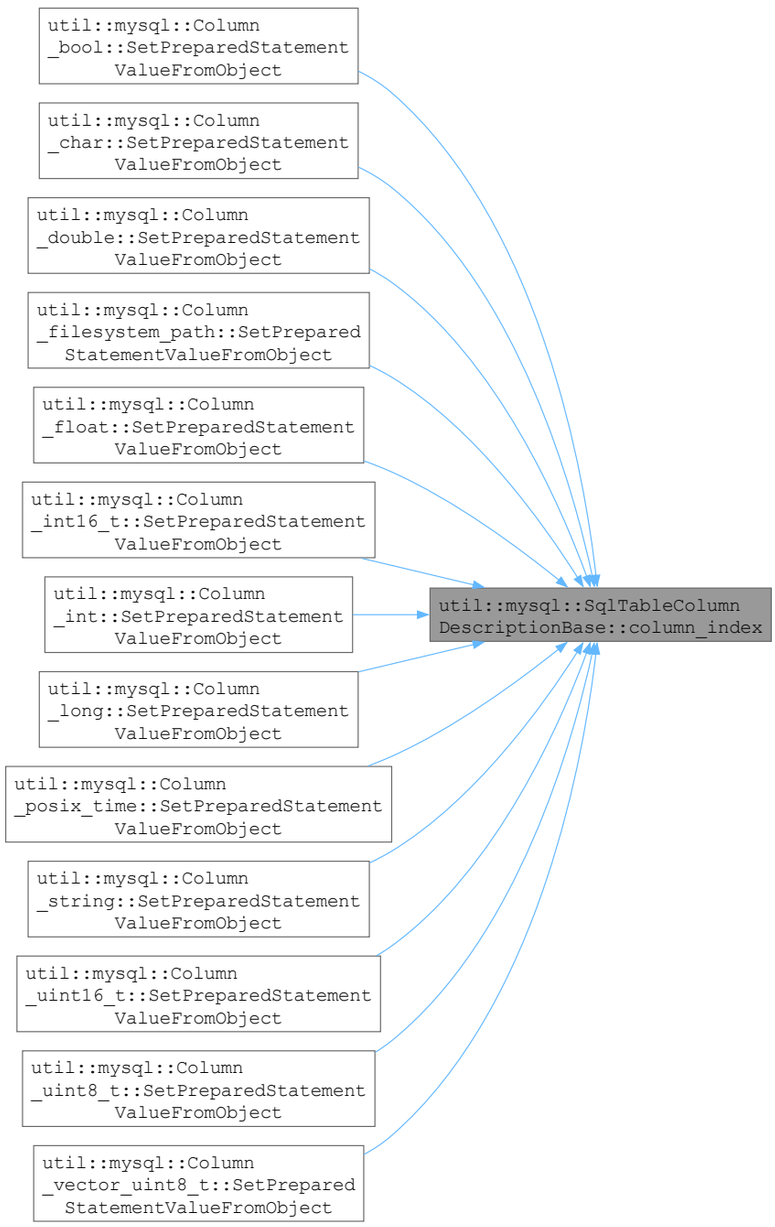
Returns

The index of the SQL table column.

References util::mysql::SqlTableColumnDescriptionBase::column_index_.

Referenced by util::mysql::Column_bool< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_char< C >::SetPreparedS
util::mysql::Column_double< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_filesystem_path< C >::SetPreparedS
util::mysql::Column_float< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_int16_t< C >::SetPreparedStatementVa
util::mysql::Column_int< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_long< C >::SetPreparedStatementValueF
util::mysql::Column_posix_time< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_string< C >::SetPreparedStateme
util::mysql::Column_uint16_t< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_uint8_t< C >::SetPreparedStatemer
and util::mysql::Column_vector_uint8_t< C >::SetPreparedStatementValueFromObject().

Here is the caller graph for this function:



10.12.3.2 column_name()

```
const string & util::mysql::SqlTableColumnDescriptionBase::column_name () const  [inline], [inherited]
```

Get the name of the SQL table column.

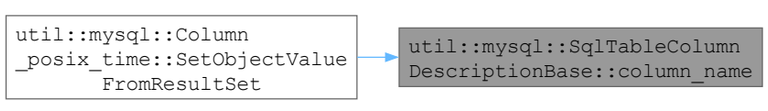
Returns

The name of the SQL table column.

References util::mysql::SqlTableColumnDescriptionBase::column_name_.

Referenced by util::mysql::Column_posix_time< C >::SetObjectValueFromResultSet().

Here is the caller graph for this function:



10.12.3.3 column_select_name()

```
string util::mysql::SqlTableColumnDescriptionBase::column_select_name () const  [inline], [inherited]
```

Get the name of the SQL table column to be used in a SELECT query.

Returns

The name of the SQL table column to be used in a SELECT query.

References util::mysql::SqlTableColumnDescriptionBase::column_name_, and util::mysql::SqlTableColumnDescriptionBase::data_type_.

10.12.3.4 data_type()

```
ColumDataType_t util::mysql::SqlTableColumnDescriptionBase::data_type () const  [inline], [inherited]
```

Get the the enumerated SQL column data type.

Returns

The the enumerated SQL column data type.

References util::mysql::SqlTableColumnDescriptionBase::data_type_.

10.12.3.5 GetValueFromObject()

```
int util::mysql::SqlTableColumnDescription< C, int >::GetValueFromObject (
            const C & obj) const  [inline], [protected], [inherited]
```

Get the object member value from the object.

Parameters

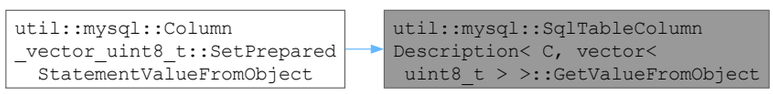
in	*obj*	The object of type C.

Returns

The value of type T of the object member corresponding to this SQL column.

Referenced by util::mysql::Column_int< C >::SetPreparedStatementValueFromObject().

Here is the caller graph for this function:



10.12.3.6 is_primary_key()

```
bool util::mysql::SqlTableColumnDescriptionBase::is_primary_key () const  [inline], [inherited]
```

Is this column a primary key for the corresponding SQL database table?

Returns

TRUE if this column is a primary key for the corresponding SQL database table, or FALSE otherwise.

References util::mysql::SqlTableColumnDescriptionBase::is_primary_key_.

10.12.3.7 set_column_index()

```
void util::mysql::SqlTableColumnDescriptionBase::set_column_index (
            size_t idx)  [inline], [inherited]
```

Set the index of this SQL table column.

Parameters

in	*idx*	The index to set.

References util::mysql::SqlTableColumnDescriptionBase::column_index_.

10.12.3.8 SetObjectValue()

```
void util::mysql::SqlTableColumnDescription< C, int >::SetObjectValue (
            C & obj,
            const int & value) const  [inline], [protected], [inherited]
```

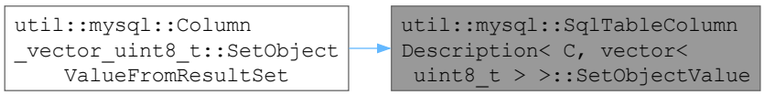
Set the object member value.

Parameters

in	*obj*	The object of type C.
in	*value*	The value of type T, to set.

Referenced by util::mysql::Column_int< C >::SetObjectValueFromResultSet().

Here is the caller graph for this function:



10.12.3.9 SetObjectValueFromResultSet()

```
template<class C >
void util::mysql::Column_int< C >::SetObjectValueFromResultSet (
            C & obj,
            const ResultSet & results) const  [inline]
```

Set the value in the field of the corresponding object from an SQL ResultSet Object.

Parameters

in,out	*obj*	A reference to the object whose field value will be set.
in	*results*	A reference to the ResultSet object.

References util::mysql::SqlTableColumnDescription< C, int >::SetObjectValue().

Here is the call graph for this function:

10.12.3.10 SetPreparedStatementValueFromObject()

```
template<class C >
void util::mysql::Column_int< C >::SetPreparedStatementValueFromObject (
            const C & obj,
            PreparedStatement & stmt) const  [inline]
```

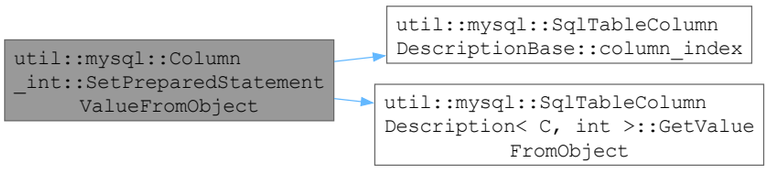
Set the value for the SQL PreparedStatement object from the value of the field of the corresponding object.

Parameters

in	*obj*	A reference to the object whose field value will be read.
in,out	*stmt*	A reference to the SQL PreparedStatement object whose column value will be set.

References util::mysql::SqlTableColumnDescriptionBase::column_index(), and util::mysql::SqlTableColumnDescription< C, int >::GetValu

Here is the call graph for this function:



10.12.4 Member Data Documentation

10.12.4.1 accessor_

```
function<int(const C&)> util::mysql::SqlTableColumnDescription< C, int >::accessor_  [private],
[inherited]
```

The accessor function object wrapper.

This object is used as a wrapper for the accessor method of the template class C. It is used to retrieve the value of the SQL table column from the object for generating an INSERT or UPDATE statement.

10.12.4.2 column_index_

```
size_t util::mysql::SqlTableColumnDescriptionBase::column_index_  [private], [inherited]
```

The index of the SQL table column.

Referenced by util::mysql::SqlTableColumnDescriptionBase::column_index(), and util::mysql::SqlTableColumnDescriptionBase::set_colum

10.12.4.3 column_name_

```
string util::mysql::SqlTableColumnDescriptionBase::column_name_  [private], [inherited]
```

The name of the SQL table column.

Referenced by util::mysql::SqlTableColumnDescriptionBase::column_name(), and util::mysql::SqlTableColumnDescriptionBase::column_s

10.12.4.4 data_type_

```
ColumDataType_t util::mysql::SqlTableColumnDescriptionBase::data_type_  [private], [inherited]
```

The enumerated SQL table column data type.

Referenced by util::mysql::SqlTableColumnDescriptionBase::column_select_name(), and util::mysql::SqlTableColumnDescriptionBase::da

10.12.4.5 is_primary_key_

```
bool util::mysql::SqlTableColumnDescriptionBase::is_primary_key_  [private], [inherited]
```

Indicates whether the column is part of the primary key for the SQL table.

Referenced by util::mysql::SqlTableColumnDescriptionBase::is_primary_key().

10.12.4.6 mutator_

`function<void(C&, int)> util::mysql::SqlTableColumnDescription< C, int >::mutator_ [private], [inherited]`

The mutator function object wrapper.

This object is used as a wrapper for the mutator method of the template class C. It is used to set the value of an object from the result set of an SQL SELECT query statement.

The documentation for this class was generated from the following file:

- utility/SqlTableColumnDescription.h

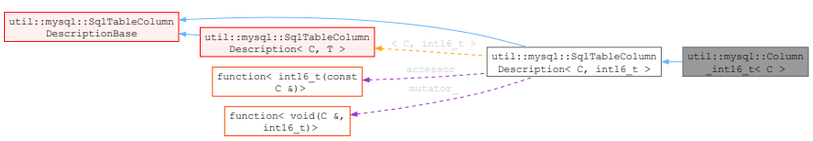
10.13 util::mysql::Column_int16_t< C > Class Template Reference

Template column description class for value of type int16_t.

`#include <SqlTableColumnDescription.h>`

Inheritance diagram for util::mysql::Column_int16_t< C >:

Collaboration diagram for util::mysql::Column_int16_t< C >:



Public Member Functions

- Column_int16_t (string column_name, int16_t(C::∗accessor)() const =NULL, void(C::∗mutator)(int16_t)=NULL, bool is_primary_key=false)

  The parameterized constructor.
- size_t column_index () const

  Get the index of the SQL table column.
- const string & column_name () const

  Get the name of the SQL table column.
- string column_select_name () const

  Get the name of the SQL table column to be used in a `SELECT` query.
- ColumDataType_t data_type () const

  Get the the enumerated SQL column data type.
- bool is_primary_key () const

  Is this column a primary key for the corresponding SQL database table?
- void set_column_index (size_t idx)

  Set the index of this SQL table column.
- void SetObjectValueFromResultSet (C &obj, const ResultSet &results) const

  Set the value in the field of the corresponding object from an SQL ResultSet Object.
- void SetPreparedStatementValueFromObject (const C &obj, PreparedStatement &stmt) const

  Set the value for the SQL PreparedStatement object from the value of the field of the corresponding object.

Protected Member Functions

- int16_t GetValueFromObject (const C &obj) const

  Get the object member value from the object.
- void SetObjectValue (C &obj, const int16_t &value) const

  Set the object member value.

Private Attributes

- function< int16_t(const C &)> accessor_

  The accessor function object wrapper.
- size_t column_index_

  The index of the SQL table column.
- string column_name_

  The name of the SQL table column.
- ColumDataType_t data_type_

  The enumerated SQL table column data type.
- bool is_primary_key_

  Indicates whether the column is part of the primary key for the SQL table.
- function< void(C &, int16_t)> mutator_

  The mutator function object wrapper.

10.13.1 Detailed Description

template<class C>
class util::mysql::Column_int16_t< C >

Template column description class for value of type int16_t.

10.13.2 Constructor & Destructor Documentation

10.13.2.1 Column_int16_t()

```
template<class C >
util::mysql::Column_int16_t< C >::Column_int16_t (
            string column_name,
            int16_t(C::* accessor )() const = NULL,
            void(C::* mutator )(int16_t) = NULL,
            bool is_primary_key = false)  [inline]
```

The parameterized constructor.

Parameters

in	*column_name*	The name of the SQL table column.
in	*accessor*	A pointer to the accessor method for acquiring the value of the corresponding object field.
in	*mutator*	A pointer to the mutator method for setting the value of the corresponding object field.
in	*is_primary_key*	TRUE if this column is a primary key for the SQL table, or FALSE otherwise.

10.13.3 Member Function Documentation

10.13.3.1 column_index()

```
size_t util::mysql::SqlTableColumnDescriptionBase::column_index () const  [inline], [inherited]
```

Get the index of the SQL table column.

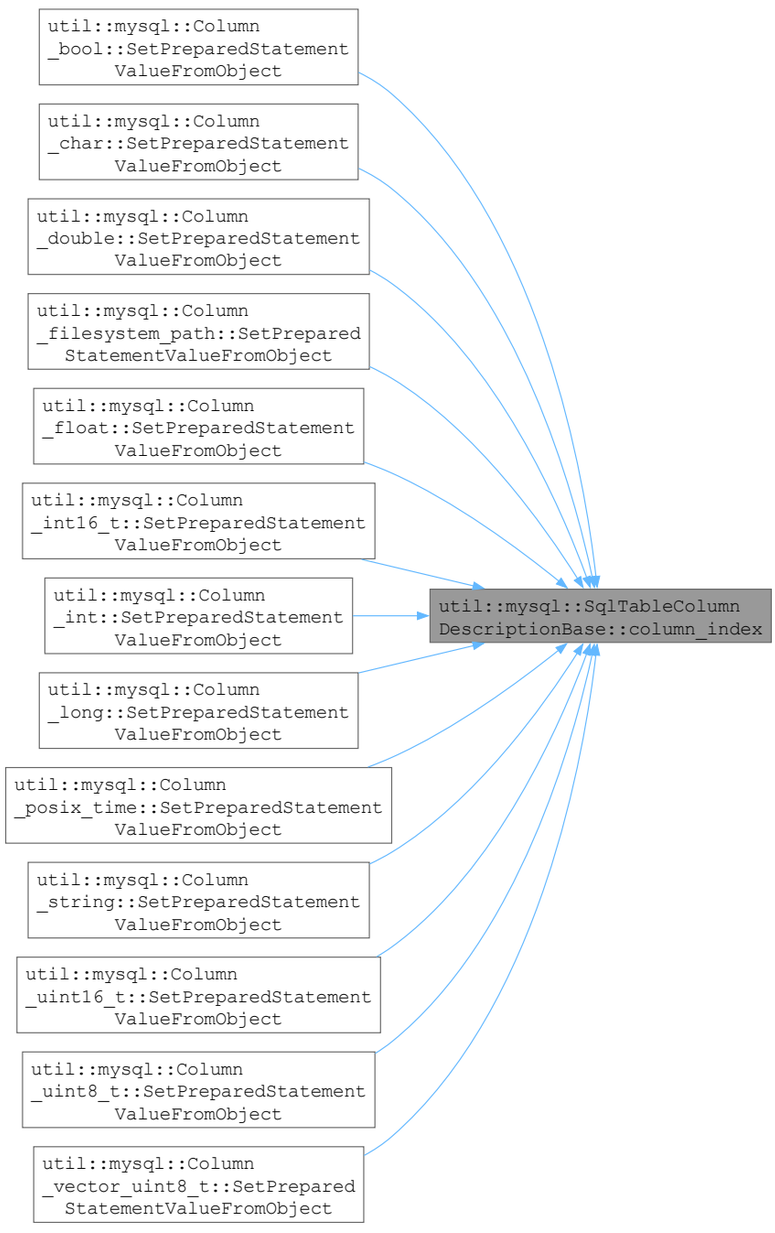
Returns

The index of the SQL table column.

References util::mysql::SqlTableColumnDescriptionBase::column_index_.

Referenced by util::mysql::Column_bool< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_char< C >::SetPreparedS
util::mysql::Column_double< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_filesystem_path< C >::SetPreparedS
util::mysql::Column_float< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_int16_t< C >::SetPreparedStatementVa
util::mysql::Column_int< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_long< C >::SetPreparedStatementValueF
util::mysql::Column_posix_time< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_string< C >::SetPreparedStateme
util::mysql::Column_uint16_t< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_uint8_t< C >::SetPreparedStatemer
and util::mysql::Column_vector_uint8_t< C >::SetPreparedStatementValueFromObject().

Here is the caller graph for this function:



10.13.3.2 column_name()

```
const string & util::mysql::SqlTableColumnDescriptionBase::column_name () const  [inline], [inherited]
```

Get the name of the SQL table column.

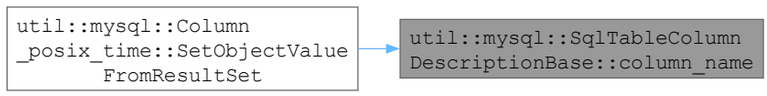
Returns

The name of the SQL table column.

References util::mysql::SqlTableColumnDescriptionBase::column_name_.

Referenced by util::mysql::Column_posix_time< C >::SetObjectValueFromResultSet().

Here is the caller graph for this function:



10.13.3.3 column_select_name()

```
string util::mysql::SqlTableColumnDescriptionBase::column_select_name () const  [inline], [inherited]
```

Get the name of the SQL table column to be used in a SELECT query.

Returns

The name of the SQL table column to be used in a SELECT query.

References util::mysql::SqlTableColumnDescriptionBase::column_name_, and util::mysql::SqlTableColumnDescriptionBase::data_type_.

10.13.3.4 data_type()

```
ColumDataType_t util::mysql::SqlTableColumnDescriptionBase::data_type () const  [inline], [inherited]
```

Get the the enumerated SQL column data type.

Returns

The the enumerated SQL column data type.

References util::mysql::SqlTableColumnDescriptionBase::data_type_.

10.13.3.5 GetValueFromObject()

```
int16_t util::mysql::SqlTableColumnDescription< C, int16_t >::GetValueFromObject (
            const C & obj) const  [inline], [protected], [inherited]
```

Get the object member value from the object.

Parameters

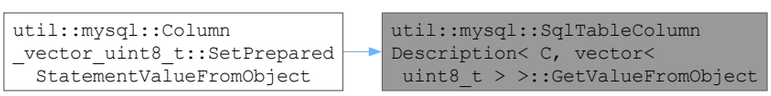
in	*obj*	The object of type C.

Returns

The value of type T of the object member corresponding to this SQL column.

Referenced by util::mysql::Column_int16_t< C >::SetPreparedStatementValueFromObject().

Here is the caller graph for this function:



10.13.3.6 is_primary_key()

```
bool util::mysql::SqlTableColumnDescriptionBase::is_primary_key () const  [inline], [inherited]
```

Is this column a primary key for the corresponding SQL database table?

Returns

TRUE if this column is a primary key for the corresponding SQL database table, or FALSE otherwise.

References util::mysql::SqlTableColumnDescriptionBase::is_primary_key_.

10.13.3.7 set_column_index()

```
void util::mysql::SqlTableColumnDescriptionBase::set_column_index (
            size_t idx)  [inline], [inherited]
```

Set the index of this SQL table column.

Parameters

in	*idx*	The index to set.

References util::mysql::SqlTableColumnDescriptionBase::column_index_.

10.13.3.8 SetObjectValue()

```
void util::mysql::SqlTableColumnDescription< C, int16_t >::SetObjectValue (
            C & obj,
            const int16_t & value) const  [inline], [protected], [inherited]
```

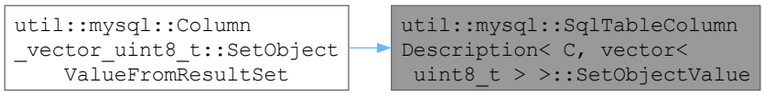
Set the object member value.

Parameters

in	*obj*	The object of type C.
in	*value*	The value of type T, to set.

Referenced by util::mysql::Column_int16_t< C >::SetObjectValueFromResultSet().

Here is the caller graph for this function:



10.13.3.9 SetObjectValueFromResultSet()

```
template<class C >
void util::mysql::Column_int16_t< C >::SetObjectValueFromResultSet (
            C & obj,
            const ResultSet & results) const  [inline]
```

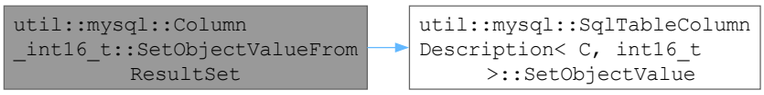
Set the value in the field of the corresponding object from an SQL ResultSet Object.

Parameters

in,out	*obj*	A reference to the object whose field value will be set.
in	*results*	A reference to the ResultSet object.

References util::mysql::SqlTableColumnDescription< C, int16_t >::SetObjectValue().

Here is the call graph for this function:



10.13.3.10 SetPreparedStatementValueFromObject()

```
template<class C >
void util::mysql::Column_int16_t< C >::SetPreparedStatementValueFromObject (
            const C & obj,
            PreparedStatement & stmt) const  [inline]
```

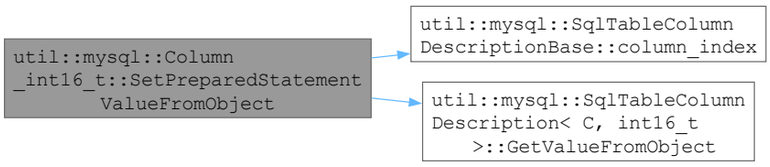
Set the value for the SQL PreparedStatement object from the value of the field of the corresponding object.

Parameters

in	*obj*	A reference to the object whose field value will be read.
in,out	*stmt*	A reference to the SQL PreparedStatement object whose column value will be set.

References util::mysql::SqlTableColumnDescriptionBase::column_index(), and util::mysql::SqlTableColumnDescription< C, int16_t >::Get

Here is the call graph for this function:



10.13.4 Member Data Documentation

10.13.4.1 accessor_

```
function<int16_t(const C&)> util::mysql::SqlTableColumnDescription< C, int16_t >::accessor_↩
[private], [inherited]
```

The accessor function object wrapper.

This object is used as a wrapper for the accessor method of the template class C. It is used to retrieve the value of the SQL table column from the object for generating an INSERT or UPDATE statement.

10.13.4.2 column_index_

```
size_t util::mysql::SqlTableColumnDescriptionBase::column_index_  [private], [inherited]
```

The index of the SQL table column.

Referenced by util::mysql::SqlTableColumnDescriptionBase::column_index(), and util::mysql::SqlTableColumnDescriptionBase::set_colum

10.13.4.3 column_name_

```
string util::mysql::SqlTableColumnDescriptionBase::column_name_  [private], [inherited]
```

The name of the SQL table column.

Referenced by util::mysql::SqlTableColumnDescriptionBase::column_name(), and util::mysql::SqlTableColumnDescriptionBase::column_s

10.13.4.4 data_type_

```
ColumDataType_t util::mysql::SqlTableColumnDescriptionBase::data_type_  [private], [inherited]
```

The enumerated SQL table column data type.

Referenced by util::mysql::SqlTableColumnDescriptionBase::column_select_name(), and util::mysql::SqlTableColumnDescriptionBase::da

10.13.4.5 is_primary_key_

```
bool util::mysql::SqlTableColumnDescriptionBase::is_primary_key_  [private], [inherited]
```

Indicates whether the column is part of the primary key for the SQL table.

Referenced by util::mysql::SqlTableColumnDescriptionBase::is_primary_key().

10.13.4.6 mutator_

```
function<void(C&, int16_t)> util::mysql::SqlTableColumnDescription< C, int16_t >::mutator_↩
[private], [inherited]
```

The mutator function object wrapper.

This object is used as a wrapper for the mutator method of the template class C. It is used to set the value of an object from the result set of an SQL SELECT query statement.

The documentation for this class was generated from the following file:

- utility/SqlTableColumnDescription.h

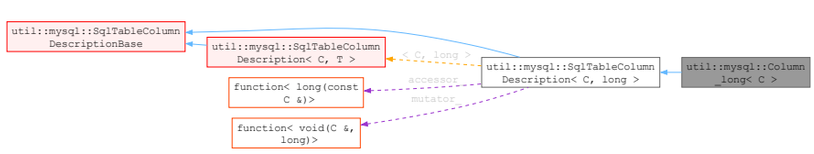
10.14 util::mysql::Column_long< C > Class Template Reference

Template column description class for value of type long.

```
#include <SqlTableColumnDescription.h>
```

Inheritance diagram for util::mysql::Column_long< C >:

Collaboration diagram for util::mysql::Column_long< C >:



Public Member Functions

- Column_long (string column_name, long(C::*accessor)() const =NULL, void(C::*mutator)(long)=NULL, bool is_primary_key=false)
  The parameterized constructor.
- size_t column_index () const
  Get the index of the SQL table column.
- const string & column_name () const
  Get the name of the SQL table column.
- string column_select_name () const
  Get the name of the SQL table column to be used in a `SELECT` query.
- ColumDataType_t data_type () const
  Get the the enumerated SQL column data type.
- bool is_primary_key () const
  Is this column a primary key for the corresponding SQL database table?
- void set_column_index (size_t idx)
  Set the index of this SQL table column.
- void SetObjectValueFromResultSet (C &obj, const ResultSet &results) const
  Set the value in the field of the corresponding object from an SQL ResultSet Object.
- void SetPreparedStatementValueFromObject (const C &obj, PreparedStatement &stmt) const
  Set the value for the SQL PreparedStatement object from the value of the field of the corresponding object.

Protected Member Functions

- long GetValueFromObject (const C &obj) const
  Get the object member value from the object.
- void SetObjectValue (C &obj, const long &value) const
  Set the object member value.

Private Attributes

- function< long(const C &)> accessor_
  The accessor function object wrapper.
- size_t column_index_
  The index of the SQL table column.
- string column_name_
  The name of the SQL table column.
- ColumDataType_t data_type_
  The enumerated SQL table column data type.
- bool is_primary_key_
  Indicates whether the column is part of the primary key for the SQL table.
- function< void(C &, long)> mutator_
  The mutator function object wrapper.

10.14.1 Detailed Description

template<class C>
class util::mysql::Column_long< C >

Template column description class for value of type long.

10.14.2 Constructor & Destructor Documentation

10.14.2.1 Column_long()

```
template<class C >
util::mysql::Column_long< C >::Column_long (
            string column_name,
            long(C::* accessor )() const = NULL,
            void(C::* mutator )(long) = NULL,
            bool is_primary_key = false)  [inline]
```

The parameterized constructor.

Parameters

in	*column_name*	The name of the SQL table column.
in	*accessor*	A pointer to the accessor method for acquiring the value of the corresponding object field.
in	*mutator*	A pointer to the mutator method for setting the value of the corresponding object field.
in	*is_primary_key*	TRUE if this column is a primary key for the SQL table, or FALSE otherwise.

10.14.3 Member Function Documentation

10.14.3.1 column_index()

```
size_t util::mysql::SqlTableColumnDescriptionBase::column_index () const  [inline], [inherited]
```

Get the index of the SQL table column.

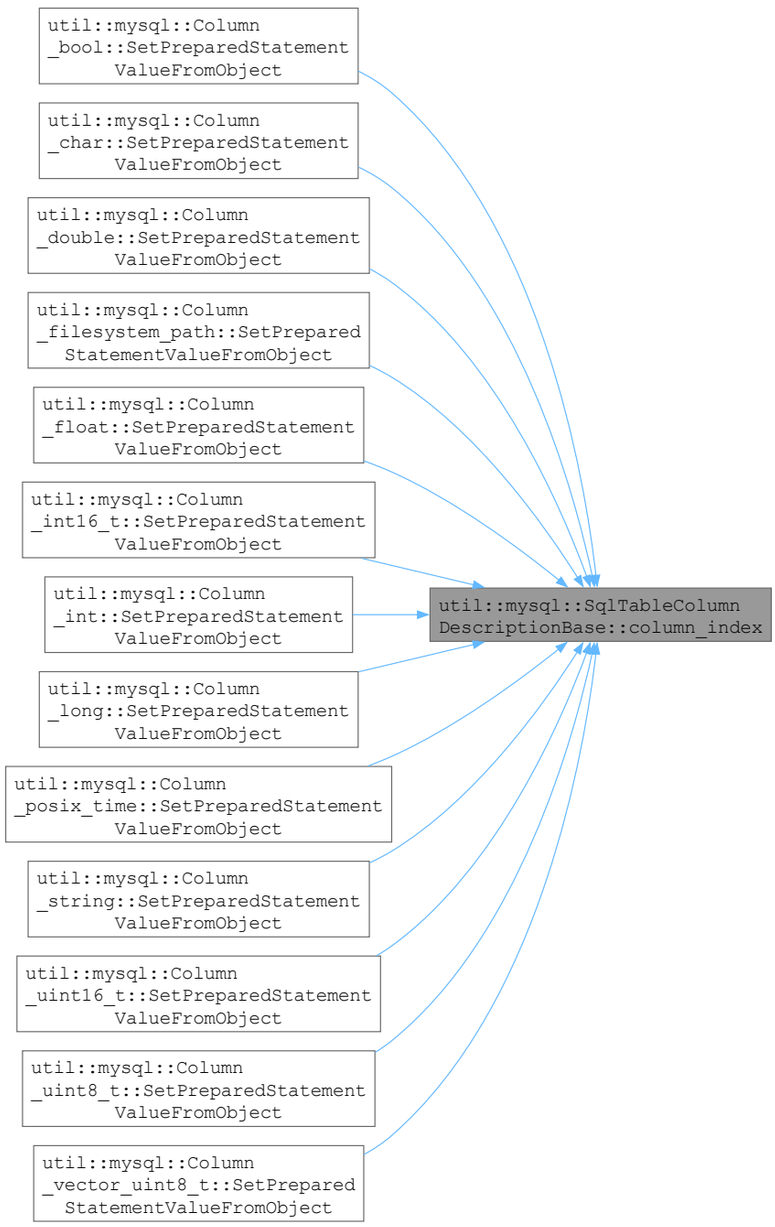
Returns

The index of the SQL table column.

References util::mysql::SqlTableColumnDescriptionBase::column_index_.

Referenced by util::mysql::Column_bool< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_char< C >::SetPreparedS
util::mysql::Column_double< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_filesystem_path< C >::SetPreparedS
util::mysql::Column_float< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_int16_t< C >::SetPreparedStatementVa
util::mysql::Column_int< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_long< C >::SetPreparedStatementValueF
util::mysql::Column_posix_time< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_string< C >::SetPreparedStateme
util::mysql::Column_uint16_t< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_uint8_t< C >::SetPreparedStatemer
and util::mysql::Column_vector_uint8_t< C >::SetPreparedStatementValueFromObject().

Here is the caller graph for this function:



10.14.3.2 column_name()

```
const string & util::mysql::SqlTableColumnDescriptionBase::column_name () const  [inline], [inherited]
```

Get the name of the SQL table column.

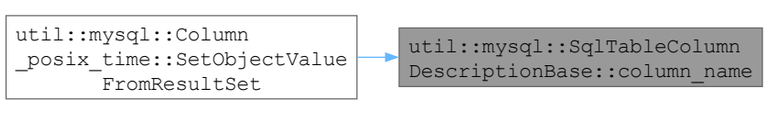
Returns

The name of the SQL table column.

References util::mysql::SqlTableColumnDescriptionBase::column_name_.

Referenced by util::mysql::Column_posix_time< C >::SetObjectValueFromResultSet().

Here is the caller graph for this function:



10.14.3.3 column_select_name()

```
string util::mysql::SqlTableColumnDescriptionBase::column_select_name () const  [inline], [inherited]
```

Get the name of the SQL table column to be used in a `SELECT` query.

Returns

The name of the SQL table column to be used in a `SELECT` query.

References util::mysql::SqlTableColumnDescriptionBase::column_name_, and util::mysql::SqlTableColumnDescriptionBase::data_type_.

10.14.3.4 data_type()

```
ColumDataType_t util::mysql::SqlTableColumnDescriptionBase::data_type () const  [inline], [inherited]
```

Get the the enumerated SQL column data type.

Returns

The the enumerated SQL column data type.

References util::mysql::SqlTableColumnDescriptionBase::data_type_.

10.14.3.5 GetValueFromObject()

```
long util::mysql::SqlTableColumnDescription< C, long >::GetValueFromObject (
            const C & obj) const  [inline], [protected], [inherited]
```

Get the object member value from the object.

Parameters

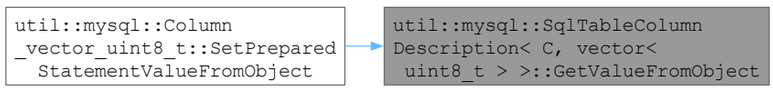
in	*obj*	The object of type `C`.

Returns

The value of type `T` of the object member corresponding to this SQL column.

Referenced by util::mysql::Column_long< C >::SetPreparedStatementValueFromObject().

Here is the caller graph for this function:



10.14.3.6 is_primary_key()

```
bool util::mysql::SqlTableColumnDescriptionBase::is_primary_key () const  [inline], [inherited]
```

Is this column a primary key for the corresponding SQL database table?

Returns

TRUE if this column is a primary key for the corresponding SQL database table, or FALSE otherwise.

References util::mysql::SqlTableColumnDescriptionBase::is_primary_key_.

10.14.3.7 set_column_index()

```
void util::mysql::SqlTableColumnDescriptionBase::set_column_index (
            size_t idx)  [inline], [inherited]
```

Set the index of this SQL table column.

Parameters

in	*idx*	The index to set.

References util::mysql::SqlTableColumnDescriptionBase::column_index_.

10.14.3.8 SetObjectValue()

```
void util::mysql::SqlTableColumnDescription< C, long >::SetObjectValue (
            C & obj,
            const long & value) const  [inline], [protected], [inherited]
```

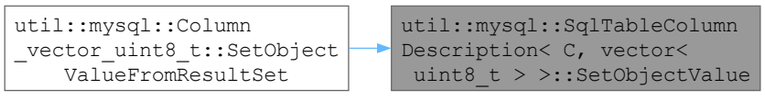
Set the object member value.

Parameters

in	*obj*	The object of type C.
in	*value*	The value of type T, to set.

Referenced by util::mysql::Column_long< C >::SetObjectValueFromResultSet().

Here is the caller graph for this function:



10.14.3.9 SetObjectValueFromResultSet()

```
template<class C >
void util::mysql::Column_long< C >::SetObjectValueFromResultSet (
            C & obj,
            const ResultSet & results) const  [inline]
```

Set the value in the field of the corresponding object from an SQL ResultSet Object.

Parameters

in,out	*obj*	A reference to the object whose field value will be set.
in	*results*	A reference to the ResultSet object.

References util::mysql::SqlTableColumnDescription< C, long >::SetObjectValue().

Here is the call graph for this function:

10.14.3.10 SetPreparedStatementValueFromObject()

```
template<class C >
void util::mysql::Column_long< C >::SetPreparedStatementValueFromObject (
            const C & obj,
            PreparedStatement & stmt) const  [inline]
```

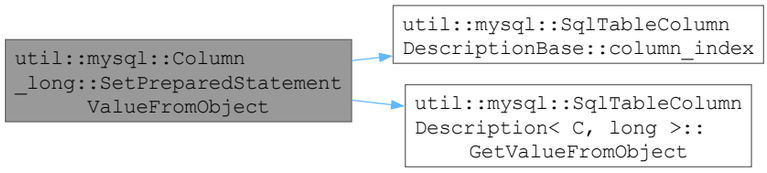
Set the value for the SQL PreparedStatement object from the value of the field of the corresponding object.

Parameters

in	*obj*	A reference to the object whose field value will be read.
in,out	*stmt*	A reference to the SQL PreparedStatement object whose column value will be set.

References util::mysql::SqlTableColumnDescriptionBase::column_index(), and util::mysql::SqlTableColumnDescription< C, long >::GetVa

Here is the call graph for this function:



10.14.4 Member Data Documentation

10.14.4.1 accessor_

```
function<long(const C&)> util::mysql::SqlTableColumnDescription< C, long >::accessor_  [private],
[inherited]
```

The accessor function object wrapper.

This object is used as a wrapper for the accessor method of the template class C. It is used to retrieve the value of the SQL table column from the object for generating an INSERT or UPDATE statement.

10.14.4.2 column_index_

```
size_t util::mysql::SqlTableColumnDescriptionBase::column_index_  [private], [inherited]
```

The index of the SQL table column.

Referenced by util::mysql::SqlTableColumnDescriptionBase::column_index(), and util::mysql::SqlTableColumnDescriptionBase::set_colum

10.14.4.3 column_name_

```
string util::mysql::SqlTableColumnDescriptionBase::column_name_  [private], [inherited]
```

The name of the SQL table column.

Referenced by util::mysql::SqlTableColumnDescriptionBase::column_name(), and util::mysql::SqlTableColumnDescriptionBase::column_s

10.14.4.4 data_type_

```
ColumDataType_t util::mysql::SqlTableColumnDescriptionBase::data_type_  [private], [inherited]
```

The enumerated SQL table column data type.

Referenced by util::mysql::SqlTableColumnDescriptionBase::column_select_name(), and util::mysql::SqlTableColumnDescriptionBase::da

10.14.4.5 is_primary_key_

```
bool util::mysql::SqlTableColumnDescriptionBase::is_primary_key_  [private], [inherited]
```

Indicates whether the column is part of the primary key for the SQL table.

Referenced by util::mysql::SqlTableColumnDescriptionBase::is_primary_key().

10.14.4.6 mutator_

`function<void(C&, long)> util::mysql::SqlTableColumnDescription< C, long >::mutator_ [private], [inherited]`

The mutator function object wrapper.

This object is used as a wrapper for the mutator method of the template class C. It is used to set the value of an object from the result set of an SQL SELECT query statement.

The documentation for this class was generated from the following file:

- utility/SqlTableColumnDescription.h

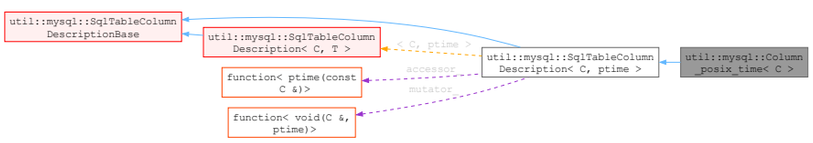
10.15 util::mysql::Column_posix_time< C > Class Template Reference

Template column description class for value of type boost::posix_time::ptime.

`#include <SqlTableColumnDescription.h>`

Inheritance diagram for util::mysql::Column_posix_time< C >:

Collaboration diagram for util::mysql::Column_posix_time< C >:



Public Member Functions

- Column_posix_time (string column_name, const ptime &(C::∗accessor)() const =NULL, void(C::∗mutator)(const ptime &)=NULL, bool is_primary_key=false)

  The parameterized constructor.
- Column_posix_time (string column_name, ptime(C::∗accessor)() const =NULL, void(C::∗mutator)(const ptime &)=NULL, bool is_primary_key=false)

  The parameterized constructor.
- size_t column_index () const

  Get the index of the SQL table column.
- const string & column_name () const

  Get the name of the SQL table column.
- string column_select_name () const

  Get the name of the SQL table column to be used in a `SELECT` query.
- ColumDataType_t data_type () const

  Get the the enumerated SQL column data type.
- bool is_primary_key () const

  Is this column a primary key for the corresponding SQL database table?
- void set_column_index (size_t idx)

  Set the index of this SQL table column.
- void SetObjectValueFromResultSet (C &obj, const ResultSet &results) const

  Set the value in the field of the corresponding object from an SQL ResultSet Object.
- void SetPreparedStatementValueFromObject (const C &obj, PreparedStatement &stmt) const

  Set the value for the SQL PreparedStatement object from the value of the field of the corresponding object.

Protected Member Functions

- ptime GetValueFromObject (const C &obj) const

  Get the object member value from the object.
- void SetObjectValue (C &obj, const ptime &value) const

  Set the object member value.

Private Attributes

- function< ptime(const C &)> accessor_

  The accessor function object wrapper.
- size_t column_index_

  The index of the SQL table column.
- string column_name_

  The name of the SQL table column.
- ColumDataType_t data_type_

  The enumerated SQL table column data type.
- bool is_primary_key_

  Indicates whether the column is part of the primary key for the SQL table.
- function< void(C &, ptime)> mutator_

  The mutator function object wrapper.

10.15.1 Detailed Description

template<class C>
class util::mysql::Column_posix_time< C >

Template column description class for value of type boost::posix_time::ptime.

10.15.2 Constructor & Destructor Documentation

10.15.2.1 Column_posix_time() [1/2]

```
template<class C >
util::mysql::Column_posix_time< C >::Column_posix_time (
            string column_name,
            ptime(C::* accessor )() const = NULL,
            void(C::* mutator )(const ptime &) = NULL,
            bool is_primary_key = false)  [inline]
```

The parameterized constructor.

Parameters

in	*column_name*	The name of the SQL table column.
in	*accessor*	A pointer to the accessor method for acquiring the value of the corresponding object field.
in	*mutator*	A pointer to the mutator method for setting the value of the corresponding object field.
in	*is_primary_key*	TRUE if this column is a primary key for the SQL table, or FALSE otherwise.

10.15.2.2 Column_posix_time() [2/2]

```
template<class C >
util::mysql::Column_posix_time< C >::Column_posix_time (
            string column_name,
            const ptime &(C::* accessor )() const = NULL,
            void(C::* mutator )(const ptime &) = NULL,
            bool is_primary_key = false)  [inline]
```

The parameterized constructor.

Parameters

in	*column_name*	The name of the SQL table column.
in	*accessor*	A pointer to the accessor method for acquiring the value of the corresponding object field.
in	*mutator*	A pointer to the mutator method for setting the value of the corresponding object field.
in	*is_primary_key*	TRUE if this column is a primary key for the SQL table, or FALSE otherwise.

10.15.3 Member Function Documentation

10.15.3.1 column_index()

```
size_t util::mysql::SqlTableColumnDescriptionBase::column_index () const  [inline], [inherited]
```

Get the index of the SQL table column.

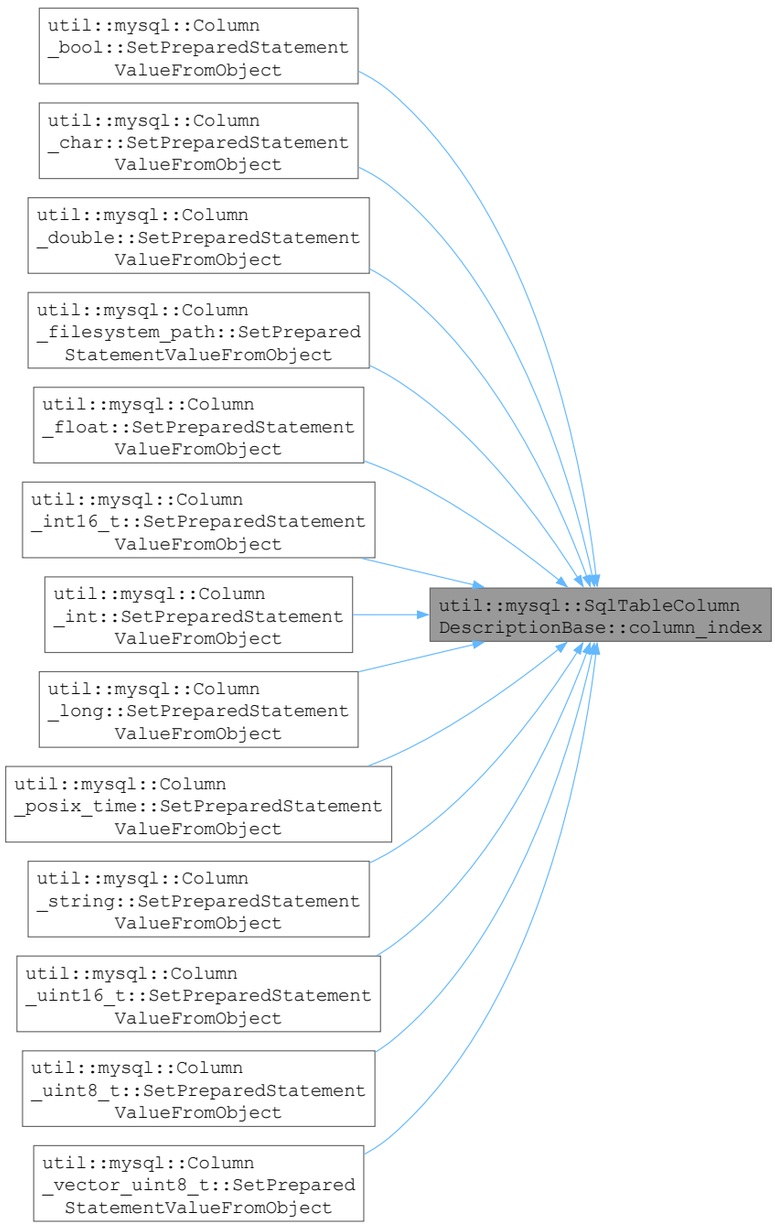
Returns

The index of the SQL table column.

References util::mysql::SqlTableColumnDescriptionBase::column_index_.

Referenced by util::mysql::Column_bool< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_char< C >::SetPreparedS
util::mysql::Column_double< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_filesystem_path< C >::SetPreparedS
util::mysql::Column_float< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_int16_t< C >::SetPreparedStatementVa
util::mysql::Column_int< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_long< C >::SetPreparedStatementValueF
util::mysql::Column_posix_time< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_string< C >::SetPreparedStateme
util::mysql::Column_uint16_t< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_uint8_t< C >::SetPreparedStatemer
and util::mysql::Column_vector_uint8_t< C >::SetPreparedStatementValueFromObject().

Here is the caller graph for this function:



10.15.3.2 column_name()

```
const string & util::mysql::SqlTableColumnDescriptionBase::column_name () const  [inline], [inherited]
```

Get the name of the SQL table column.

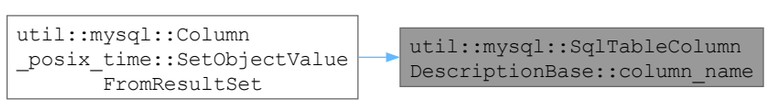
Returns

The name of the SQL table column.

References util::mysql::SqlTableColumnDescriptionBase::column_name_.

Referenced by util::mysql::Column_posix_time< C >::SetObjectValueFromResultSet().

Here is the caller graph for this function:



10.15.3.3 column_select_name()

```
string util::mysql::SqlTableColumnDescriptionBase::column_select_name () const  [inline], [inherited]
```

Get the name of the SQL table column to be used in a SELECT query.

Returns

The name of the SQL table column to be used in a SELECT query.

References util::mysql::SqlTableColumnDescriptionBase::column_name_, and util::mysql::SqlTableColumnDescriptionBase::data_type_.

10.15.3.4 data_type()

```
ColumDataType_t util::mysql::SqlTableColumnDescriptionBase::data_type () const  [inline], [inherited]
```

Get the the enumerated SQL column data type.

Returns

The the enumerated SQL column data type.

References util::mysql::SqlTableColumnDescriptionBase::data_type_.

10.15.3.5 GetValueFromObject()

```
ptime util::mysql::SqlTableColumnDescription< C, ptime >::GetValueFromObject (
            const C & obj) const  [inline], [protected], [inherited]
```

Get the object member value from the object.

Parameters

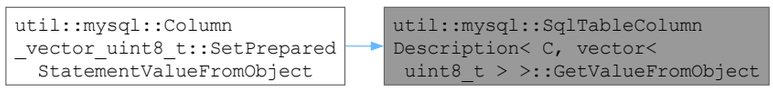
in	*obj*	The object of type C.

Returns

The value of type T of the object member corresponding to this SQL column.

Referenced by util::mysql::Column_posix_time< C >::SetPreparedStatementValueFromObject().

Here is the caller graph for this function:



10.15.3.6 is_primary_key()

```
bool util::mysql::SqlTableColumnDescriptionBase::is_primary_key () const  [inline], [inherited]
```

Is this column a primary key for the corresponding SQL database table?

Returns

TRUE if this column is a primary key for the corresponding SQL database table, or FALSE otherwise.

References util::mysql::SqlTableColumnDescriptionBase::is_primary_key_.

10.15.3.7 set_column_index()

```
void util::mysql::SqlTableColumnDescriptionBase::set_column_index (
            size_t idx)  [inline], [inherited]
```

Set the index of this SQL table column.

Parameters

in	*idx*	The index to set.

References util::mysql::SqlTableColumnDescriptionBase::column_index_.

10.15.3.8 SetObjectValue()

```
void util::mysql::SqlTableColumnDescription< C, ptime >::SetObjectValue (
            C & obj,
            const ptime & value) const  [inline], [protected], [inherited]
```

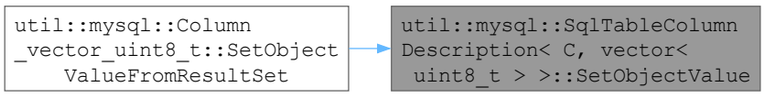
Set the object member value.

Parameters

in	*obj*	The object of type C.
in	*value*	The value of type T, to set.

Referenced by util::mysql::Column_posix_time< C >::SetObjectValueFromResultSet().

Here is the caller graph for this function:



10.15.3.9 SetObjectValueFromResultSet()

```
template<class C >
void util::mysql::Column_posix_time< C >::SetObjectValueFromResultSet (
            C & obj,
            const ResultSet & results) const  [inline]
```

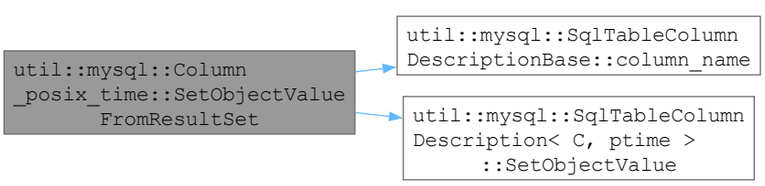
Set the value in the field of the corresponding object from an SQL ResultSet Object.

Parameters

in,out	*obj*	A reference to the object whose field value will be set.
in	*results*	A reference to the ResultSet object.

References util::mysql::SqlTableColumnDescriptionBase::column_name(), and util::mysql::SqlTableColumnDescription< C, ptime >::SetO

Here is the call graph for this function:



10.15.3.10 SetPreparedStatementValueFromObject()

```
template<class C >
void util::mysql::Column_posix_time< C >::SetPreparedStatementValueFromObject (
            const C & obj,
            PreparedStatement & stmt) const  [inline]
```

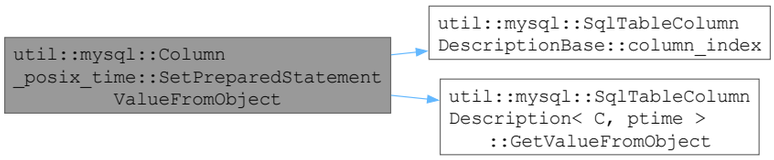
Set the value for the SQL PreparedStatement object from the value of the field of the corresponding object.

Parameters

in	*obj*	A reference to the object whose field value will be read.
in,out	*stmt*	A reference to the SQL PreparedStatement object whose column value will be set.

References util::mysql::SqlTableColumnDescriptionBase::column_index(), and util::mysql::SqlTableColumnDescription< C, ptime >::GetV

Here is the call graph for this function:



10.15.4 Member Data Documentation

10.15.4.1 accessor_

function<ptime(const C&)> util::mysql::SqlTableColumnDescription< C, ptime >::accessor_ [private], [inherited]

The accessor function object wrapper.

This object is used as a wrapper for the accessor method of the template class C. It is used to retrieve the value of the SQL table column from the object for generating an INSERT or UPDATE statement.

10.15.4.2 column_index_

size_t util::mysql::SqlTableColumnDescriptionBase::column_index_ [private], [inherited]

The index of the SQL table column.

Referenced by util::mysql::SqlTableColumnDescriptionBase::column_index(), and util::mysql::SqlTableColumnDescriptionBase::set_colum

10.15.4.3 column_name_

string util::mysql::SqlTableColumnDescriptionBase::column_name_ [private], [inherited]

The name of the SQL table column.

Referenced by util::mysql::SqlTableColumnDescriptionBase::column_name(), and util::mysql::SqlTableColumnDescriptionBase::column_s

10.15.4.4 data_type_

ColumDataType_t util::mysql::SqlTableColumnDescriptionBase::data_type_ [private], [inherited]

The enumerated SQL table column data type.

Referenced by util::mysql::SqlTableColumnDescriptionBase::column_select_name(), and util::mysql::SqlTableColumnDescriptionBase::da

10.15.4.5 is_primary_key_

bool util::mysql::SqlTableColumnDescriptionBase::is_primary_key_ [private], [inherited]

Indicates whether the column is part of the primary key for the SQL table.

Referenced by util::mysql::SqlTableColumnDescriptionBase::is_primary_key().

10.15.4.6 mutator_

`function<void(C&, ptime)> util::mysql::SqlTableColumnDescription< C, ptime >::mutator_ [private], [inherited]`

The mutator function object wrapper.

This object is used as a wrapper for the mutator method of the template class C. It is used to set the value of an object from the result set of an SQL SELECT query statement.

The documentation for this class was generated from the following file:

- utility/SqlTableColumnDescription.h

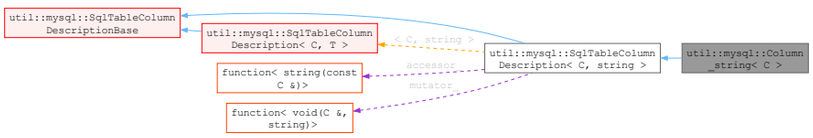
10.16 util::mysql::Column_string< C > Class Template Reference

Template column description class for value of type string.

`#include <SqlTableColumnDescription.h>`

Inheritance diagram for util::mysql::Column_string< C >:

Collaboration diagram for util::mysql::Column_string< C >:



Public Member Functions

- Column_string (string column_name, const string &(C::∗accessor)() const =NULL, void(C::∗mutator)(const string &)=NULL, bool is_primary_key=false) ATTR_DEPRECATED

  The parameterized constructor.
- Column_string (string column_name, string(C::∗accessor)() const =NULL, void(C::∗mutator)(const string &)=NULL, bool is_primary_key=false)

  The parameterized constructor.
- size_t column_index () const

  Get the index of the SQL table column.
- const string & column_name () const

  Get the name of the SQL table column.
- string column_select_name () const

  Get the name of the SQL table column to be used in a `SELECT` query.
- ColumDataType_t data_type () const

  Get the the enumerated SQL column data type.
- bool is_primary_key () const

  Is this column a primary key for the corresponding SQL database table?
- void set_column_index (size_t idx)

  Set the index of this SQL table column.
- void SetObjectValueFromResultSet (C &obj, const ResultSet &results) const

  Set the value in the field of the corresponding object from an SQL ResultSet Object.
- void SetPreparedStatementValueFromObject (const C &obj, PreparedStatement &stmt) const

  Set the value for the SQL PreparedStatement object from the value of the field of the corresponding object.

Protected Member Functions

- string GetValueFromObject (const C &obj) const
  Get the object member value from the object.
- void SetObjectValue (C &obj, const string &value) const
  Set the object member value.

Private Attributes

- function< string(const C &)> accessor_
  The accessor function object wrapper.
- size_t column_index_
  The index of the SQL table column.
- string column_name_
  The name of the SQL table column.
- ColumDataType_t data_type_
  The enumerated SQL table column data type.
- bool is_primary_key_
  Indicates whether the column is part of the primary key for the SQL table.
- function< void(C &, string)> mutator_
  The mutator function object wrapper.

10.16.1 Detailed Description

template<class C>
class util::mysql::Column_string< C >

Template column description class for value of type string.

10.16.2 Constructor & Destructor Documentation

10.16.2.1 Column_string() [1/2]

```
template<class C >
util::mysql::Column_string< C >::Column_string (
            string column_name,
            string(C::* accessor )() const = NULL,
            void(C::* mutator )(const string &) = NULL,
            bool is_primary_key = false)  [inline]
```

The parameterized constructor.

Parameters

in	*column_name*	The name of the SQL table column.
in	*accessor*	A pointer to the accessor method for acquiring the value of the corresponding object field.
in	*mutator*	A pointer to the mutator method for setting the value of the corresponding object field.
in	*is_primary_key*	TRUE if this column is a primary key for the SQL table, or FALSE otherwise.

10.16.2.2 Column_string() [2/2]

```
template<class C >
util::mysql::Column_string< C >::Column_string (
            string column_name,
            const string &(C::* accessor )() const = NULL,
            void(C::* mutator )(const string &) = NULL,
            bool is_primary_key = false)  [inline]
```

The parameterized constructor.

Parameters

in	*column_name*	The name of the SQL table column.
in	*accessor*	A pointer to the accessor method for acquiring the value of the corresponding object field.
in	*mutator*	A pointer to the mutator method for setting the value of the corresponding object field.
in	*is_primary_key*	TRUE if this column is a primary key for the SQL table, or FALSE otherwise.

10.16.3 Member Function Documentation

10.16.3.1 column_index()

```
size_t util::mysql::SqlTableColumnDescriptionBase::column_index () const  [inline], [inherited]
```

Get the index of the SQL table column.

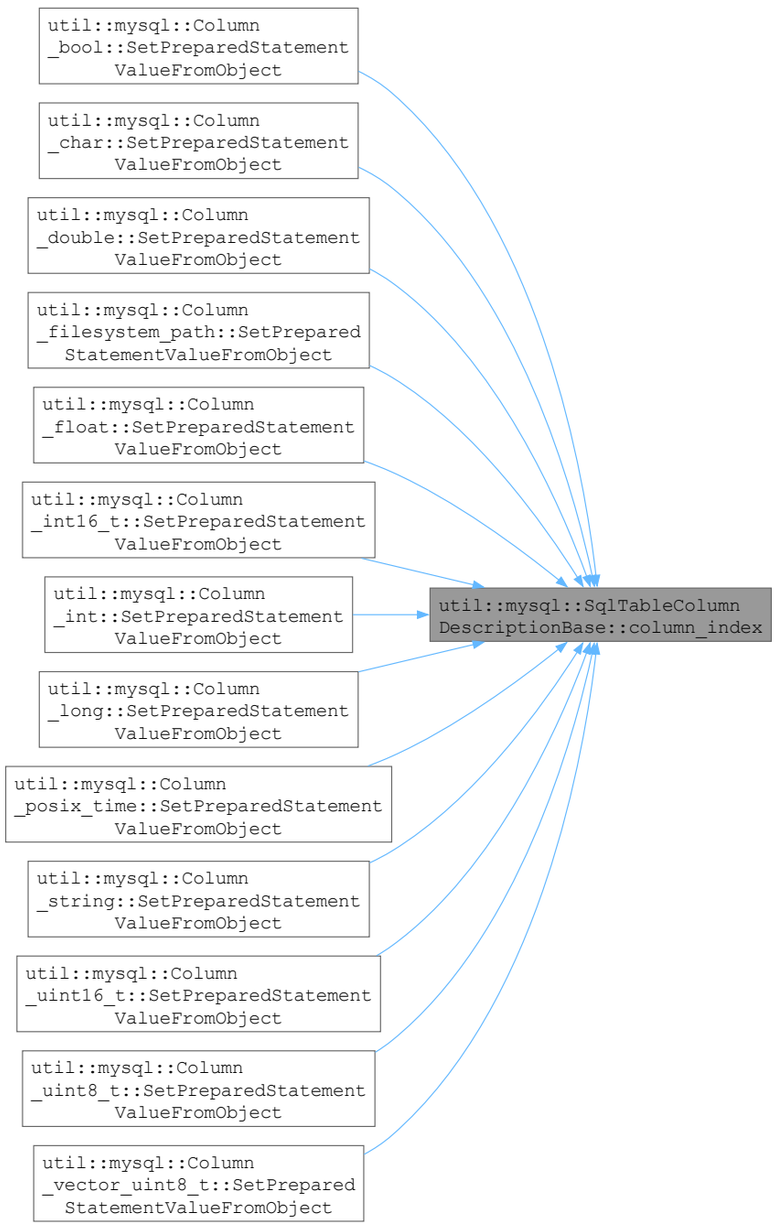
Returns

The index of the SQL table column.

References util::mysql::SqlTableColumnDescriptionBase::column_index_.

Referenced by util::mysql::Column_bool< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_char< C >::SetPreparedS
util::mysql::Column_double< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_filesystem_path< C >::SetPreparedS
util::mysql::Column_float< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_int16_t< C >::SetPreparedStatementVa
util::mysql::Column_int< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_long< C >::SetPreparedStatementValueF
util::mysql::Column_posix_time< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_string< C >::SetPreparedStateme
util::mysql::Column_uint16_t< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_uint8_t< C >::SetPreparedStatemer
and util::mysql::Column_vector_uint8_t< C >::SetPreparedStatementValueFromObject().

Here is the caller graph for this function:



10.16.3.2 column_name()

```
const string & util::mysql::SqlTableColumnDescriptionBase::column_name () const  [inline], [inherited]
```

Get the name of the SQL table column.

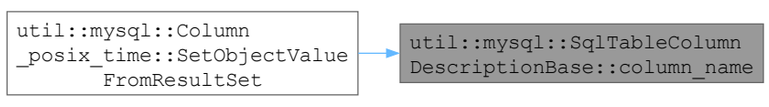
Returns

The name of the SQL table column.

References util::mysql::SqlTableColumnDescriptionBase::column_name_.

Referenced by util::mysql::Column_posix_time< C >::SetObjectValueFromResultSet().

Here is the caller graph for this function:



10.16.3.3 column_select_name()

```
string util::mysql::SqlTableColumnDescriptionBase::column_select_name () const  [inline], [inherited]
```

Get the name of the SQL table column to be used in a `SELECT` query.

Returns

The name of the SQL table column to be used in a `SELECT` query.

References util::mysql::SqlTableColumnDescriptionBase::column_name_, and util::mysql::SqlTableColumnDescriptionBase::data_type_.

10.16.3.4 data_type()

```
ColumDataType_t util::mysql::SqlTableColumnDescriptionBase::data_type () const  [inline], [inherited]
```

Get the the enumerated SQL column data type.

Returns

The the enumerated SQL column data type.

References util::mysql::SqlTableColumnDescriptionBase::data_type_.

10.16.3.5 GetValueFromObject()

```
string util::mysql::SqlTableColumnDescription< C, string >::GetValueFromObject (
            const C & obj) const  [inline], [protected], [inherited]
```

Get the object member value from the object.

Parameters

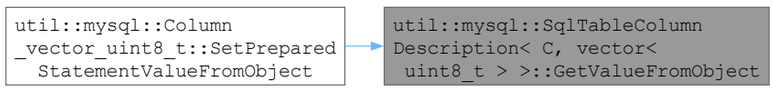
in	*obj*	The object of type C.

Returns

The value of type T of the object member corresponding to this SQL column.

Referenced by util::mysql::Column_string< C >::SetPreparedStatementValueFromObject().

Here is the caller graph for this function:



10.16.3.6 is_primary_key()

```
bool util::mysql::SqlTableColumnDescriptionBase::is_primary_key () const  [inline], [inherited]
```

Is this column a primary key for the corresponding SQL database table?

Returns

TRUE if this column is a primary key for the corresponding SQL database table, or FALSE otherwise.

References util::mysql::SqlTableColumnDescriptionBase::is_primary_key_.

10.16.3.7 set_column_index()

```
void util::mysql::SqlTableColumnDescriptionBase::set_column_index (
            size_t idx)  [inline], [inherited]
```

Set the index of this SQL table column.

Parameters

in	*idx*	The index to set.

References util::mysql::SqlTableColumnDescriptionBase::column_index_.

10.16.3.8 SetObjectValue()

```
void util::mysql::SqlTableColumnDescription< C, string >::SetObjectValue (
            C & obj,
            const string & value) const  [inline], [protected], [inherited]
```

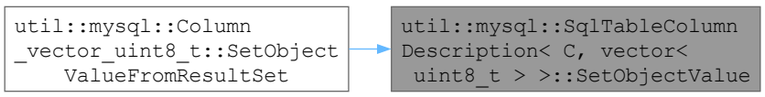
Set the object member value.

Parameters

in	*obj*	The object of type C.
in	*value*	The value of type T, to set.

Referenced by util::mysql::Column_string< C >::SetObjectValueFromResultSet().

Here is the caller graph for this function:



10.16.3.9 SetObjectValueFromResultSet()

```
template<class C >
void util::mysql::Column_string< C >::SetObjectValueFromResultSet (
            C & obj,
            const ResultSet & results) const  [inline]
```

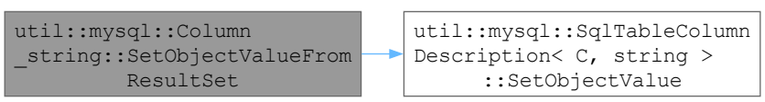
Set the value in the field of the corresponding object from an SQL ResultSet Object.

Parameters

in,out	*obj*	A reference to the object whose field value will be set.
in	*results*	A reference to the ResultSet object.

References util::mysql::SqlTableColumnDescription< C, string >::SetObjectValue().

Here is the call graph for this function:



10.16.3.10 SetPreparedStatementValueFromObject()

```
template<class C >
void util::mysql::Column_string< C >::SetPreparedStatementValueFromObject (
            const C & obj,
            PreparedStatement & stmt) const  [inline]
```

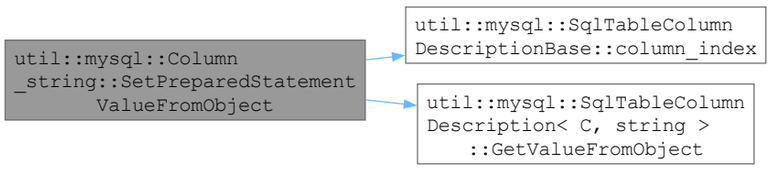
Set the value for the SQL PreparedStatement object from the value of the field of the corresponding object.

Parameters

in	*obj*	A reference to the object whose field value will be read.
in,out	*stmt*	A reference to the SQL PreparedStatement object whose column value will be set.

References util::mysql::SqlTableColumnDescriptionBase::column_index(), and util::mysql::SqlTableColumnDescription< C, string >::GetV

Here is the call graph for this function:



10.16.4 Member Data Documentation

10.16.4.1 accessor_

```
function<string(const C&)> util::mysql::SqlTableColumnDescription< C, string >::accessor_↩
[private], [inherited]
```

The accessor function object wrapper.

This object is used as a wrapper for the accessor method of the template class C. It is used to retrieve the value of the SQL table column from the object for generating an INSERT or UPDATE statement.

10.16.4.2 column_index_

```
size_t util::mysql::SqlTableColumnDescriptionBase::column_index_ [private], [inherited]
```

The index of the SQL table column.

Referenced by util::mysql::SqlTableColumnDescriptionBase::column_index(), and util::mysql::SqlTableColumnDescriptionBase::set_colum

10.16.4.3 column_name_

```
string util::mysql::SqlTableColumnDescriptionBase::column_name_ [private], [inherited]
```

The name of the SQL table column.

Referenced by util::mysql::SqlTableColumnDescriptionBase::column_name(), and util::mysql::SqlTableColumnDescriptionBase::column_s

10.16.4.4 data_type_

```
ColumDataType_t util::mysql::SqlTableColumnDescriptionBase::data_type_ [private], [inherited]
```

The enumerated SQL table column data type.

Referenced by util::mysql::SqlTableColumnDescriptionBase::column_select_name(), and util::mysql::SqlTableColumnDescriptionBase::da

10.16.4.5 is_primary_key_

```
bool util::mysql::SqlTableColumnDescriptionBase::is_primary_key_ [private], [inherited]
```

Indicates whether the column is part of the primary key for the SQL table.

Referenced by util::mysql::SqlTableColumnDescriptionBase::is_primary_key().

10.16.4.6 mutator_

`function<void(C&, string)> util::mysql::SqlTableColumnDescription< C, string >::mutator_ [private], [inherited]`

The mutator function object wrapper.

This object is used as a wrapper for the mutator method of the template class C. It is used to set the value of an object from the result set of an SQL SELECT query statement.

The documentation for this class was generated from the following file:

- utility/SqlTableColumnDescription.h

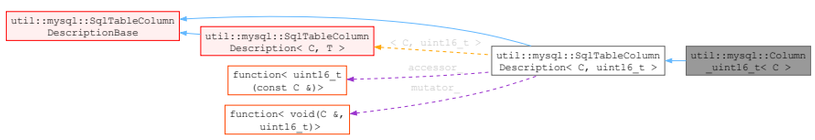
10.17 util::mysql::Column_uint16_t< C > Class Template Reference

Template column description class for value of type uint16_t.

`#include <SqlTableColumnDescription.h>`

Inheritance diagram for util::mysql::Column_uint16_t< C >:

Collaboration diagram for util::mysql::Column_uint16_t< C >:



Public Member Functions

- Column_uint16_t (string column_name, uint16_t(C::∗accessor)() const =NULL, void(C::∗mutator)(uint16_t)=NULL, bool is_primary_key=false)

  The parameterized constructor.
- size_t column_index () const

  Get the index of the SQL table column.
- const string & column_name () const

  Get the name of the SQL table column.
- string column_select_name () const

  Get the name of the SQL table column to be used in a `SELECT` query.
- ColumDataType_t data_type () const

  Get the the enumerated SQL column data type.
- bool is_primary_key () const

  Is this column a primary key for the corresponding SQL database table?
- void set_column_index (size_t idx)

  Set the index of this SQL table column.
- void SetObjectValueFromResultSet (C &obj, const ResultSet &results) const

  Set the value in the field of the corresponding object from an SQL ResultSet Object.
- void SetPreparedStatementValueFromObject (const C &obj, PreparedStatement &stmt) const

  Set the value for the SQL PreparedStatement object from the value of the field of the corresponding object.

Protected Member Functions

- uint16_t GetValueFromObject (const C &obj) const

  Get the object member value from the object.
- void SetObjectValue (C &obj, const uint16_t &value) const

  Set the object member value.

Private Attributes

- function< uint16_t(const C &)> accessor_

  The accessor function object wrapper.
- size_t column_index_

  The index of the SQL table column.
- string column_name_

  The name of the SQL table column.
- ColumDataType_t data_type_

  The enumerated SQL table column data type.
- bool is_primary_key_

  Indicates whether the column is part of the primary key for the SQL table.
- function< void(C &, uint16_t)> mutator_

  The mutator function object wrapper.

10.17.1 Detailed Description

template<class C>
class util::mysql::Column_uint16_t< C >

Template column description class for value of type uint16_t.

10.17.2 Constructor & Destructor Documentation

10.17.2.1 Column_uint16_t()

```
template<class C >
util::mysql::Column_uint16_t< C >::Column_uint16_t (
            string column_name,
            uint16_t(C::* accessor )() const = NULL,
            void(C::* mutator )(uint16_t) = NULL,
            bool is_primary_key = false)  [inline]
```

The parameterized constructor.

Parameters

in	*column_name*	The name of the SQL table column.
in	*accessor*	A pointer to the accessor method for acquiring the value of the corresponding object field.
in	*mutator*	A pointer to the mutator method for setting the value of the corresponding object field.
in	*is_primary_key*	TRUE if this column is a primary key for the SQL table, or FALSE otherwise.

10.17.3 Member Function Documentation

10.17.3.1 column_index()

```
size_t util::mysql::SqlTableColumnDescriptionBase::column_index () const  [inline], [inherited]
```

Get the index of the SQL table column.

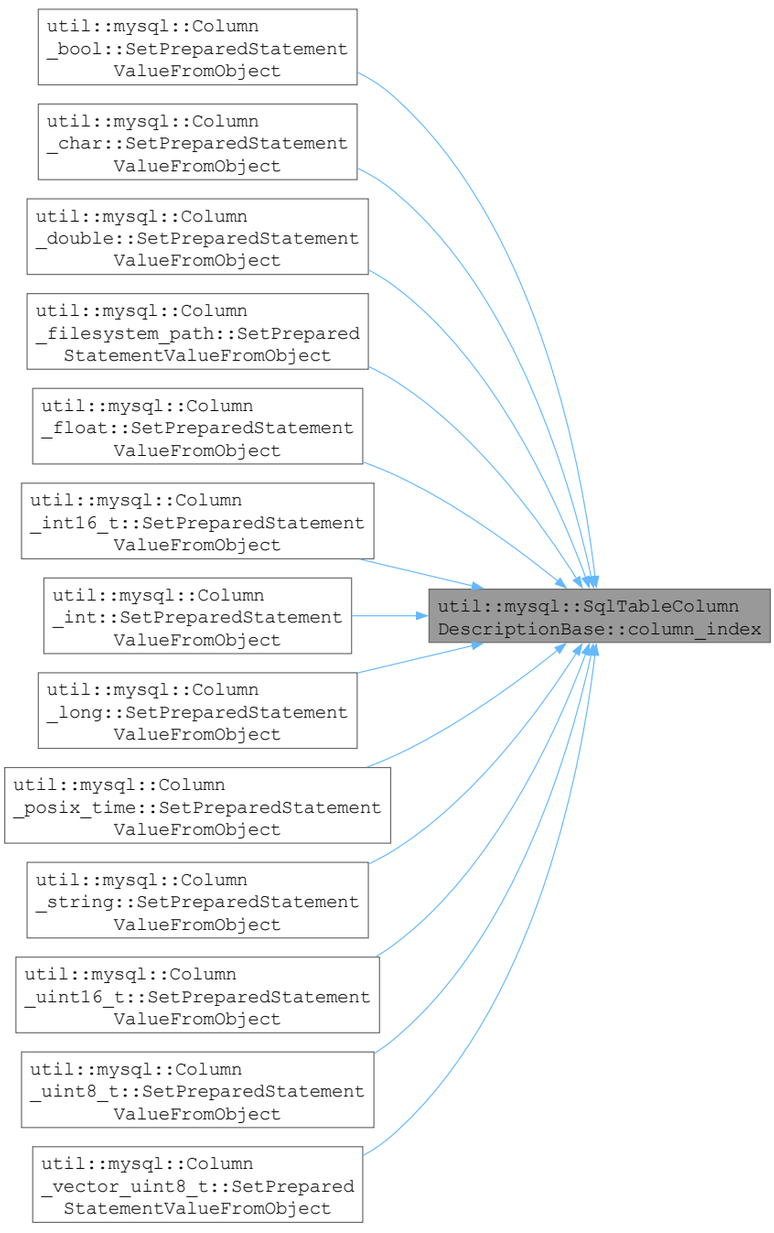
Returns

The index of the SQL table column.

References util::mysql::SqlTableColumnDescriptionBase::column_index_.

Referenced by util::mysql::Column_bool< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_char< C >::SetPreparedS
util::mysql::Column_double< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_filesystem_path< C >::SetPreparedS
util::mysql::Column_float< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_int16_t< C >::SetPreparedStatementVa
util::mysql::Column_int< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_long< C >::SetPreparedStatementValueF
util::mysql::Column_posix_time< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_string< C >::SetPreparedStateme
util::mysql::Column_uint16_t< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_uint8_t< C >::SetPreparedStatemer
and util::mysql::Column_vector_uint8_t< C >::SetPreparedStatementValueFromObject().

Here is the caller graph for this function:



10.17.3.2 column_name()

```
const string & util::mysql::SqlTableColumnDescriptionBase::column_name () const  [inline], [inherited]
```

Get the name of the SQL table column.

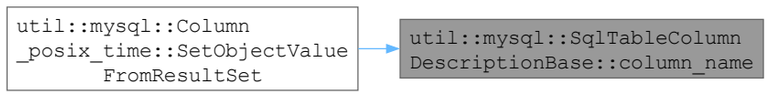
Returns

The name of the SQL table column.

References util::mysql::SqlTableColumnDescriptionBase::column_name_.

Referenced by util::mysql::Column_posix_time< C >::SetObjectValueFromResultSet().

Here is the caller graph for this function:



10.17.3.3 column_select_name()

```
string util::mysql::SqlTableColumnDescriptionBase::column_select_name () const  [inline], [inherited]
```

Get the name of the SQL table column to be used in a `SELECT` query.

Returns

The name of the SQL table column to be used in a `SELECT` query.

References util::mysql::SqlTableColumnDescriptionBase::column_name_, and util::mysql::SqlTableColumnDescriptionBase::data_type_.

10.17.3.4 data_type()

```
ColumDataType_t util::mysql::SqlTableColumnDescriptionBase::data_type () const  [inline], [inherited]
```

Get the the enumerated SQL column data type.

Returns

The the enumerated SQL column data type.

References util::mysql::SqlTableColumnDescriptionBase::data_type_.

10.17.3.5 GetValueFromObject()

```
uint16_t util::mysql::SqlTableColumnDescription< C, uint16_t >::GetValueFromObject (
            const C & obj) const  [inline], [protected], [inherited]
```

Get the object member value from the object.

Parameters

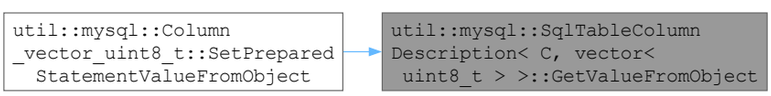
in	*obj*	The object of type `C`.

Returns

The value of type `T` of the object member corresponding to this SQL column.

Referenced by util::mysql::Column_uint16_t< C >::SetPreparedStatementValueFromObject().

Here is the caller graph for this function:



10.17.3.6 is_primary_key()

```
bool util::mysql::SqlTableColumnDescriptionBase::is_primary_key () const  [inline], [inherited]
```

Is this column a primary key for the corresponding SQL database table?

Returns

TRUE if this column is a primary key for the corresponding SQL database table, or FALSE otherwise.

References util::mysql::SqlTableColumnDescriptionBase::is_primary_key_.

10.17.3.7 set_column_index()

```
void util::mysql::SqlTableColumnDescriptionBase::set_column_index (
            size_t idx)  [inline], [inherited]
```

Set the index of this SQL table column.

Parameters

in	*idx*	The index to set.

References util::mysql::SqlTableColumnDescriptionBase::column_index_.

10.17.3.8 SetObjectValue()

```
void util::mysql::SqlTableColumnDescription< C, uint16_t >::SetObjectValue (
            C & obj,
            const uint16_t & value) const  [inline], [protected], [inherited]
```

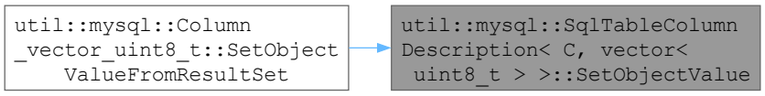
Set the object member value.

Parameters

in	*obj*	The object of type C.
in	*value*	The value of type T, to set.

Referenced by util::mysql::Column_uint16_t< C >::SetObjectValueFromResultSet().

Here is the caller graph for this function:



10.17.3.9 SetObjectValueFromResultSet()

```
template<class C >
void util::mysql::Column_uint16_t< C >::SetObjectValueFromResultSet (
            C & obj,
            const ResultSet & results) const  [inline]
```

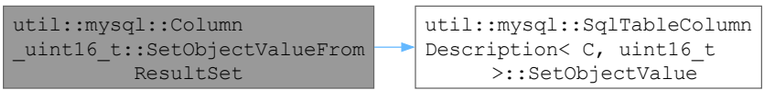
Set the value in the field of the corresponding object from an SQL ResultSet Object.

Parameters

in,out	*obj*	A reference to the object whose field value will be set.
in	*results*	A reference to the ResultSet object.

References util::mysql::SqlTableColumnDescription< C, uint16_t >::SetObjectValue().

Here is the call graph for this function:



10.17.3.10 SetPreparedStatementValueFromObject()

```
template<class C >
void util::mysql::Column_uint16_t< C >::SetPreparedStatementValueFromObject (
            const C & obj,
            PreparedStatement & stmt) const  [inline]
```

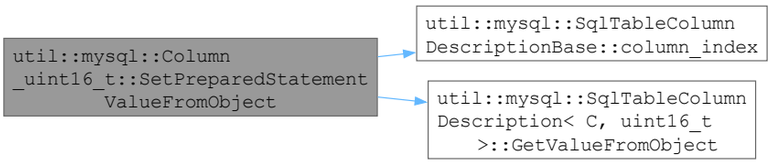
Set the value for the SQL PreparedStatement object from the value of the field of the corresponding object.

Parameters

in	*obj*	A reference to the object whose field value will be read.
in,out	*stmt*	A reference to the SQL PreparedStatement object whose column value will be set.

References util::mysql::SqlTableColumnDescriptionBase::column_index(), and util::mysql::SqlTableColumnDescription< C, uint16_t >::G

Here is the call graph for this function:



10.17.4 Member Data Documentation

10.17.4.1 accessor_

```
function<uint16_t(const C&)> util::mysql::SqlTableColumnDescription< C, uint16_t >::accessor_↩
[private], [inherited]
```

The accessor function object wrapper.

This object is used as a wrapper for the accessor method of the template class C. It is used to retrieve the value of the SQL table column from the object for generating an INSERT or UPDATE statement.

10.17.4.2 column_index_

```
size_t util::mysql::SqlTableColumnDescriptionBase::column_index_  [private], [inherited]
```

The index of the SQL table column.

Referenced by util::mysql::SqlTableColumnDescriptionBase::column_index(), and util::mysql::SqlTableColumnDescriptionBase::set_colum

10.17.4.3 column_name_

```
string util::mysql::SqlTableColumnDescriptionBase::column_name_  [private], [inherited]
```

The name of the SQL table column.

Referenced by util::mysql::SqlTableColumnDescriptionBase::column_name(), and util::mysql::SqlTableColumnDescriptionBase::column_s

10.17.4.4 data_type_

```
ColumDataType_t util::mysql::SqlTableColumnDescriptionBase::data_type_  [private], [inherited]
```

The enumerated SQL table column data type.

Referenced by util::mysql::SqlTableColumnDescriptionBase::column_select_name(), and util::mysql::SqlTableColumnDescriptionBase::da

10.17.4.5 is_primary_key_

```
bool util::mysql::SqlTableColumnDescriptionBase::is_primary_key_  [private], [inherited]
```

Indicates whether the column is part of the primary key for the SQL table.

Referenced by util::mysql::SqlTableColumnDescriptionBase::is_primary_key().

10.17.4.6 mutator_

```
function<void(C&, uint16_t)> util::mysql::SqlTableColumnDescription< C, uint16_t >::mutator_↩
[private], [inherited]
```

The mutator function object wrapper.

This object is used as a wrapper for the mutator method of the template class C. It is used to set the value of an object from the result set of an SQL SELECT query statement.

The documentation for this class was generated from the following file:

- utility/SqlTableColumnDescription.h

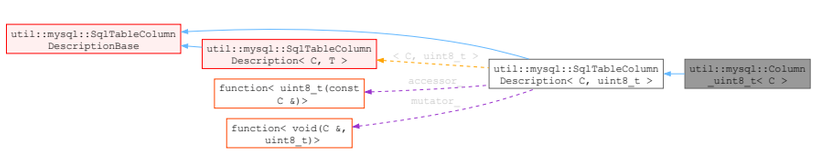
10.18 util::mysql::Column_uint8_t< C > Class Template Reference

Template column description class for value of type uint8_t.

```
#include <SqlTableColumnDescription.h>
```

Inheritance diagram for util::mysql::Column_uint8_t< C >:

Collaboration diagram for util::mysql::Column_uint8_t< C >:



Public Member Functions

- Column_uint8_t (string column_name, uint8_t(C::∗accessor)() const =NULL, void(C::∗mutator)(uint8_t)=NULL, bool is_primary_key=false)

  The parameterized constructor.
- size_t column_index () const

  Get the index of the SQL table column.
- const string & column_name () const

  Get the name of the SQL table column.
- string column_select_name () const

  Get the name of the SQL table column to be used in a `SELECT` query.
- ColumDataType_t data_type () const

  Get the the enumerated SQL column data type.
- bool is_primary_key () const

  Is this column a primary key for the corresponding SQL database table?
- void set_column_index (size_t idx)

  Set the index of this SQL table column.
- void SetObjectValueFromResultSet (C &obj, const ResultSet &results) const

  Set the value in the field of the corresponding object from an SQL ResultSet Object.
- void SetPreparedStatementValueFromObject (const C &obj, PreparedStatement &stmt) const

  Set the value for the SQL PreparedStatement object from the value of the field of the corresponding object.

Protected Member Functions

- uint8_t GetValueFromObject (const C &obj) const

  Get the object member value from the object.
- void SetObjectValue (C &obj, const uint8_t &value) const

  Set the object member value.

Private Attributes

- function< uint8_t(const C &)> accessor_

  The accessor function object wrapper.
- size_t column_index_

  The index of the SQL table column.
- string column_name_

  The name of the SQL table column.
- ColumDataType_t data_type_

  The enumerated SQL table column data type.
- bool is_primary_key_

  Indicates whether the column is part of the primary key for the SQL table.
- function< void(C &, uint8_t)> mutator_

  The mutator function object wrapper.

10.18.1 Detailed Description

template<class C>
class util::mysql::Column_uint8_t< C >

Template column description class for value of type uint8_t.

10.18.2 Constructor & Destructor Documentation

10.18.2.1 Column_uint8_t()

```
template<class C >
util::mysql::Column_uint8_t< C >::Column_uint8_t (
            string column_name,
            uint8_t(C::* accessor )() const = NULL,
            void(C::* mutator )(uint8_t) = NULL,
            bool is_primary_key = false)  [inline]
```

The parameterized constructor.

Parameters

in	*column_name*	The name of the SQL table column.
in	*accessor*	A pointer to the accessor method for acquiring the value of the corresponding object field.
in	*mutator*	A pointer to the mutator method for setting the value of the corresponding object field.
in	*is_primary_key*	TRUE if this column is a primary key for the SQL table, or FALSE otherwise.

10.18.3 Member Function Documentation

10.18.3.1 column_index()

```
size_t util::mysql::SqlTableColumnDescriptionBase::column_index () const  [inline], [inherited]
```

Get the index of the SQL table column.

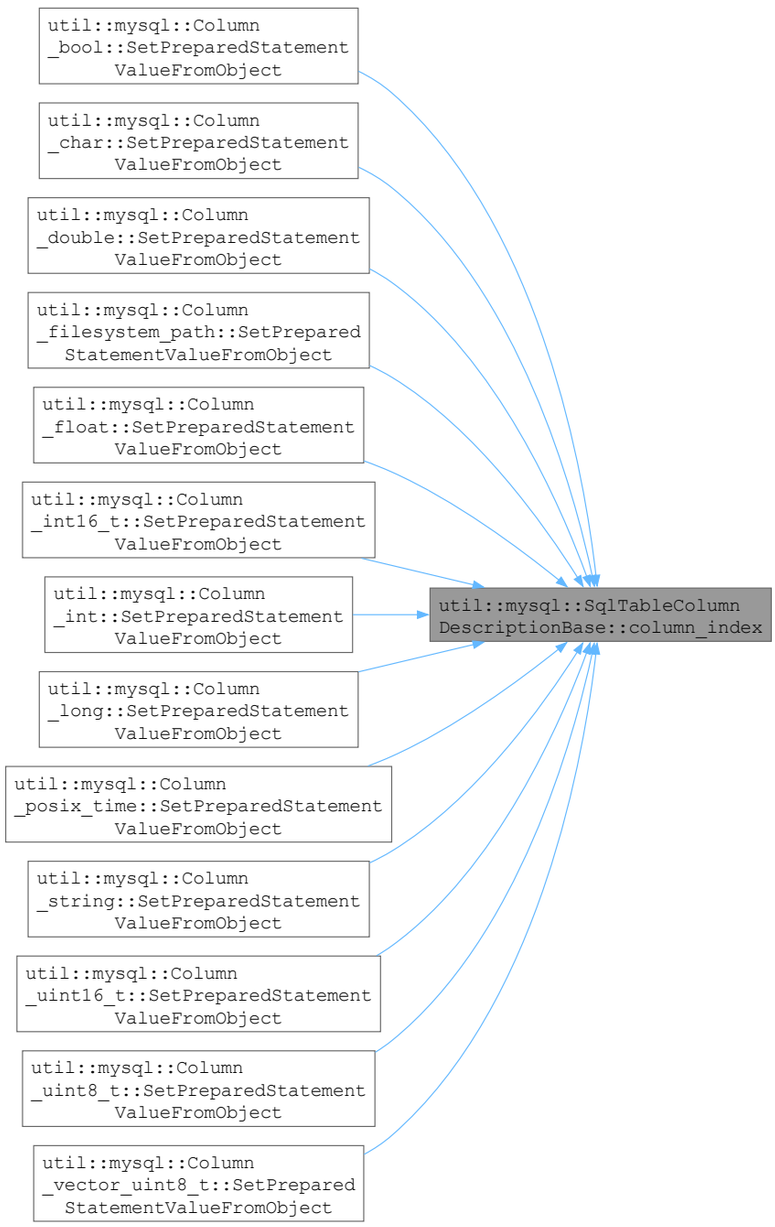
Returns

The index of the SQL table column.

References util::mysql::SqlTableColumnDescriptionBase::column_index_.

Referenced by util::mysql::Column_bool< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_char< C >::SetPreparedS
util::mysql::Column_double< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_filesystem_path< C >::SetPreparedS
util::mysql::Column_float< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_int16_t< C >::SetPreparedStatementVa
util::mysql::Column_int< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_long< C >::SetPreparedStatementValueF
util::mysql::Column_posix_time< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_string< C >::SetPreparedStateme
util::mysql::Column_uint16_t< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_uint8_t< C >::SetPreparedStatemer
and util::mysql::Column_vector_uint8_t< C >::SetPreparedStatementValueFromObject().

Here is the caller graph for this function:



10.18.3.2 column_name()

```
const string & util::mysql::SqlTableColumnDescriptionBase::column_name () const  [inline], [inherited]
```

Get the name of the SQL table column.

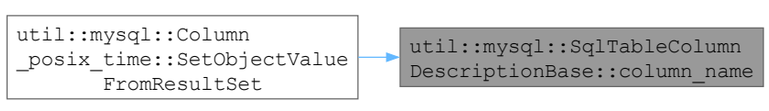
Returns

The name of the SQL table column.

References util::mysql::SqlTableColumnDescriptionBase::column_name_.

Referenced by util::mysql::Column_posix_time< C >::SetObjectValueFromResultSet().

Here is the caller graph for this function:



10.18.3.3 column_select_name()

```
string util::mysql::SqlTableColumnDescriptionBase::column_select_name () const  [inline], [inherited]
```

Get the name of the SQL table column to be used in a SELECT query.

Returns

The name of the SQL table column to be used in a SELECT query.

References util::mysql::SqlTableColumnDescriptionBase::column_name_, and util::mysql::SqlTableColumnDescriptionBase::data_type_.

10.18.3.4 data_type()

```
ColumDataType_t util::mysql::SqlTableColumnDescriptionBase::data_type () const  [inline], [inherited]
```

Get the the enumerated SQL column data type.

Returns

The the enumerated SQL column data type.

References util::mysql::SqlTableColumnDescriptionBase::data_type_.

10.18.3.5 GetValueFromObject()

```
uint8_t util::mysql::SqlTableColumnDescription< C, uint8_t >::GetValueFromObject (
            const C & obj) const  [inline], [protected], [inherited]
```

Get the object member value from the object.

Parameters

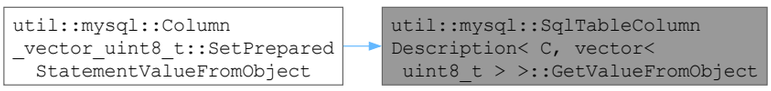
in	*obj*	The object of type C.

Returns

The value of type T of the object member corresponding to this SQL column.

Referenced by util::mysql::Column_uint8_t< C >::SetPreparedStatementValueFromObject().

Here is the caller graph for this function:



10.18.3.6 is_primary_key()

```
bool util::mysql::SqlTableColumnDescriptionBase::is_primary_key () const  [inline], [inherited]
```

Is this column a primary key for the corresponding SQL database table?

Returns

TRUE if this column is a primary key for the corresponding SQL database table, or FALSE otherwise.

References util::mysql::SqlTableColumnDescriptionBase::is_primary_key_.

10.18.3.7 set_column_index()

```
void util::mysql::SqlTableColumnDescriptionBase::set_column_index (
            size_t idx)  [inline], [inherited]
```

Set the index of this SQL table column.

Parameters

in	*idx*	The index to set.

References util::mysql::SqlTableColumnDescriptionBase::column_index_.

10.18.3.8 SetObjectValue()

```
void util::mysql::SqlTableColumnDescription< C, uint8_t >::SetObjectValue (
            C & obj,
            const uint8_t & value) const  [inline], [protected], [inherited]
```

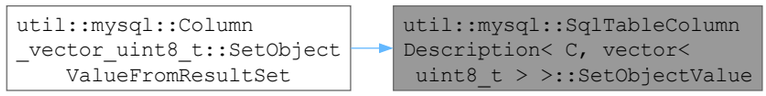
Set the object member value.

Parameters

in	*obj*	The object of type C.
in	*value*	The value of type T, to set.

Referenced by util::mysql::Column_uint8_t< C >::SetObjectValueFromResultSet().

Here is the caller graph for this function:



10.18.3.9 SetObjectValueFromResultSet()

```
template<class C >
void util::mysql::Column_uint8_t< C >::SetObjectValueFromResultSet (
            C & obj,
            const ResultSet & results) const  [inline]
```

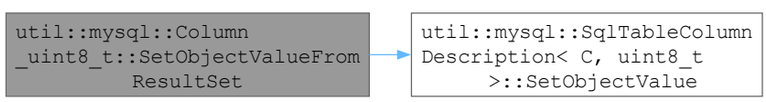
Set the value in the field of the corresponding object from an SQL ResultSet Object.

Parameters

in,out	*obj*	A reference to the object whose field value will be set.
in	*results*	A reference to the ResultSet object.

References util::mysql::SqlTableColumnDescription< C, uint8_t >::SetObjectValue().

Here is the call graph for this function:



10.18.3.10 SetPreparedStatementValueFromObject()

```
template<class C >
void util::mysql::Column_uint8_t< C >::SetPreparedStatementValueFromObject (
            const C & obj,
            PreparedStatement & stmt) const  [inline]
```

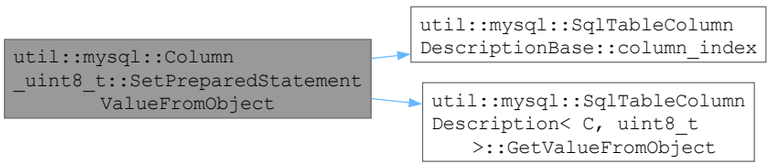
Set the value for the SQL PreparedStatement object from the value of the field of the corresponding object.

Parameters

in	*obj*	A reference to the object whose field value will be read.
in,out	*stmt*	A reference to the SQL PreparedStatement object whose column value will be set.

References util::mysql::SqlTableColumnDescriptionBase::column_index(), and util::mysql::SqlTableColumnDescription< C, uint8_t >::Get

Here is the call graph for this function:



10.18.4 Member Data Documentation

10.18.4.1 accessor_

function<uint8_t(const C&)> util::mysql::SqlTableColumnDescription< C, uint8_t >::accessor_↩
[private], [inherited]

The accessor function object wrapper.

This object is used as a wrapper for the accessor method of the template class C. It is used to retrieve the value of the SQL table column from the object for generating an INSERT or UPDATE statement.

10.18.4.2 column_index_

size_t util::mysql::SqlTableColumnDescriptionBase::column_index_ [private], [inherited]

The index of the SQL table column.

Referenced by util::mysql::SqlTableColumnDescriptionBase::column_index(), and util::mysql::SqlTableColumnDescriptionBase::set_colum

10.18.4.3 column_name_

string util::mysql::SqlTableColumnDescriptionBase::column_name_ [private], [inherited]

The name of the SQL table column.

Referenced by util::mysql::SqlTableColumnDescriptionBase::column_name(), and util::mysql::SqlTableColumnDescriptionBase::column_s

10.18.4.4 data_type_

ColumDataType_t util::mysql::SqlTableColumnDescriptionBase::data_type_ [private], [inherited]

The enumerated SQL table column data type.

Referenced by util::mysql::SqlTableColumnDescriptionBase::column_select_name(), and util::mysql::SqlTableColumnDescriptionBase::da

10.18.4.5 is_primary_key_

bool util::mysql::SqlTableColumnDescriptionBase::is_primary_key_ [private], [inherited]

Indicates whether the column is part of the primary key for the SQL table.

Referenced by util::mysql::SqlTableColumnDescriptionBase::is_primary_key().

10.18.4.6 mutator_

`function<void(C&, uint8_t)> util::mysql::SqlTableColumnDescription< C, uint8_t >::mutator_↩ [private], [inherited]`

The mutator function object wrapper.

This object is used as a wrapper for the mutator method of the template class C. It is used to set the value of an object from the result set of an SQL SELECT query statement.

The documentation for this class was generated from the following file:

- utility/SqlTableColumnDescription.h

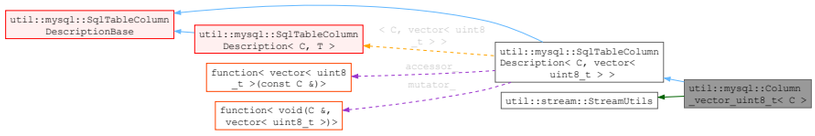
10.19 util::mysql::Column_vector_uint8_t< C > Class Template Reference

Template column description class for value of type vector<uint8_t>.

`#include <SqlTableColumnDescription.h>`

Inheritance diagram for util::mysql::Column_vector_uint8_t< C >:

Collaboration diagram for util::mysql::Column_vector_uint8_t< C >:



Public Member Functions

- Column_vector_uint8_t (string column_name, vector< uint8_t >(C::∗accessor)() const =NULL, void(C↩ ::∗mutator)(const vector< uint8_t > &)=NULL, bool is_primary_key=false)

  The parameterized constructor.
- size_t column_index () const

  Get the index of the SQL table column.
- const string & column_name () const

  Get the name of the SQL table column.
- string column_select_name () const

  Get the name of the SQL table column to be used in a `SELECT` query.
- ColumDataType_t data_type () const

  Get the the enumerated SQL column data type.
- bool is_primary_key () const

  Is this column a primary key for the corresponding SQL database table?
- void set_column_index (size_t idx)

  Set the index of this SQL table column.
- void SetObjectValueFromResultSet (C &obj, const ResultSet &results) const

  Set the value in the field of the corresponding object from an SQL ResultSet Object.
- void SetPreparedStatementValueFromObject (const C &obj, PreparedStatement &stmt) const

  Set the value for the SQL PreparedStatement object from the value of the field of the corresponding object.

Protected Member Functions

- vector< uint8_t > GetValueFromObject (const C &obj) const

  Get the object member value from the object.
- void SetObjectValue (C &obj, const vector< uint8_t > &value) const

  Set the object member value.

Static Protected Member Functions

- static uint8_t htoi (char c)

  Get the integer value of a hexadecimal character.

Private Attributes

- function< vector< uint8_t >(const C &)> accessor_

  The accessor function object wrapper.
- size_t column_index_

  The index of the SQL table column.
- string column_name_

  The name of the SQL table column.
- ColumDataType_t data_type_

  The enumerated SQL table column data type.
- bool is_primary_key_

  Indicates whether the column is part of the primary key for the SQL table.
- function< void(C &, vector< uint8_t >)> mutator_

  The mutator function object wrapper.

10.19.1 Detailed Description

template<class C>
class util::mysql::Column_vector_uint8_t< C >

Template column description class for value of type vector<uint8_t>.

10.19.2 Constructor & Destructor Documentation

10.19.2.1 Column_vector_uint8_t()

```
template<class C >
util::mysql::Column_vector_uint8_t< C >::Column_vector_uint8_t (
            string column_name,
            vector< uint8_t >(C::* accessor )() const = NULL,
            void(C::* mutator )(const vector< uint8_t > &) = NULL,
            bool is_primary_key = false)  [inline]
```

The parameterized constructor.

Parameters

in	*column_name*	The name of the SQL table column.
in	*accessor*	A pointer to the accessor method for acquiring the value of the corresponding object field.
in	*mutator*	A pointer to the mutator method for setting the value of the corresponding object field.
in	*is_primary_key*	TRUE if this column is a primary key for the SQL table, or FALSE otherwise.

10.19.3 Member Function Documentation

10.19.3.1 column_index()

```
size_t util::mysql::SqlTableColumnDescriptionBase::column_index () const  [inline], [inherited]
```

Get the index of the SQL table column.

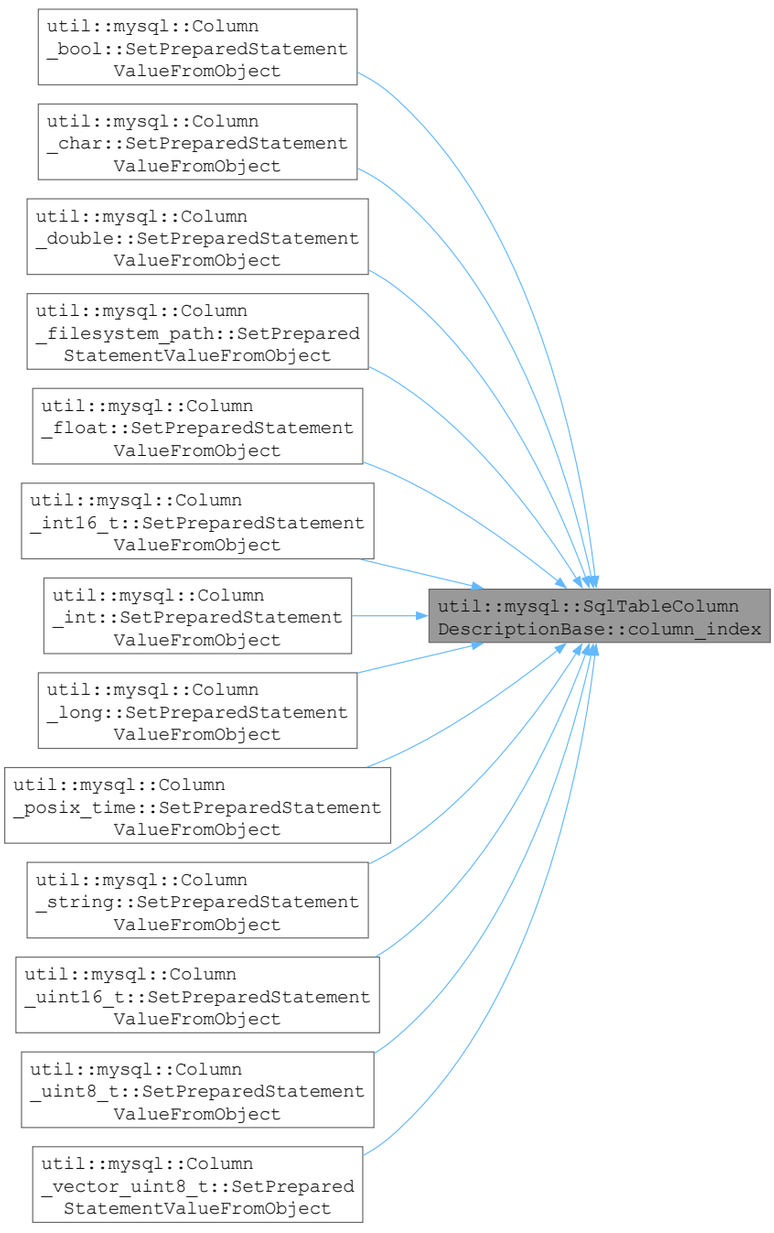
Returns

The index of the SQL table column.

References util::mysql::SqlTableColumnDescriptionBase::column_index_.

Referenced by util::mysql::Column_bool< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_char< C >::SetPreparedS
util::mysql::Column_double< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_filesystem_path< C >::SetPreparedS
util::mysql::Column_float< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_int16_t< C >::SetPreparedStatementVa
util::mysql::Column_int< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_long< C >::SetPreparedStatementValueF
util::mysql::Column_posix_time< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_string< C >::SetPreparedStateme
util::mysql::Column_uint16_t< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_uint8_t< C >::SetPreparedStatemer
and util::mysql::Column_vector_uint8_t< C >::SetPreparedStatementValueFromObject().

Here is the caller graph for this function:



10.19.3.2 column_name()

```
const string & util::mysql::SqlTableColumnDescriptionBase::column_name () const  [inline], [inherited]
```

Get the name of the SQL table column.

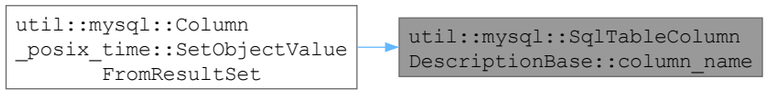
Returns

The name of the SQL table column.

References util::mysql::SqlTableColumnDescriptionBase::column_name_.

Referenced by util::mysql::Column_posix_time< C >::SetObjectValueFromResultSet().

Here is the caller graph for this function:



10.19.3.3 column_select_name()

```
string util::mysql::SqlTableColumnDescriptionBase::column_select_name () const  [inline], [inherited]
```

Get the name of the SQL table column to be used in a SELECT query.

Returns

The name of the SQL table column to be used in a SELECT query.

References util::mysql::SqlTableColumnDescriptionBase::column_name_, and util::mysql::SqlTableColumnDescriptionBase::data_type_.

10.19.3.4 data_type()

```
ColumDataType_t util::mysql::SqlTableColumnDescriptionBase::data_type () const  [inline], [inherited]
```

Get the the enumerated SQL column data type.

Returns

The the enumerated SQL column data type.

References util::mysql::SqlTableColumnDescriptionBase::data_type_.

10.19.3.5 GetValueFromObject()

```
vector< uint8_t > util::mysql::SqlTableColumnDescription< C, vector< uint8_t > >::GetValueFrom↩
Object (
            const C & obj) const  [inline], [protected], [inherited]
```

Get the object member value from the object.

Parameters

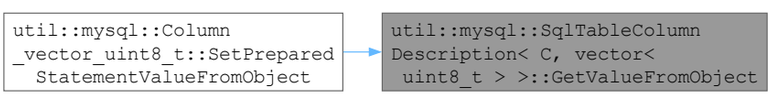
in	*obj*	The object of type C.

Returns

The value of type T of the object member corresponding to this SQL column.

Referenced by util::mysql::Column_vector_uint8_t< C >::SetPreparedStatementValueFromObject().

Here is the caller graph for this function:



10.19.3.6 htoi()

```
static uint8_t util::stream::StreamUtils::htoi (
            char c)  [inline], [static], [inherited]
```

Get the integer value of a hexadecimal character.

Parameters

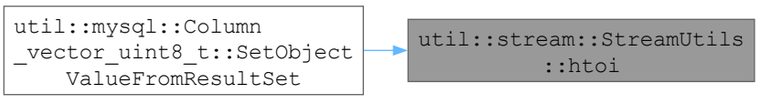
in	*c*	The hexadecimal character.

Returns

The integer value of a hexadecimal character, or 0 if c is not a hexadecimal character.

Referenced by util::mysql::Column_vector_uint8_t< C >::SetObjectValueFromResultSet().

Here is the caller graph for this function:



10.19.3.7 is_primary_key()

```
bool util::mysql::SqlTableColumnDescriptionBase::is_primary_key () const  [inline], [inherited]
```

Is this column a primary key for the corresponding SQL database table?

Returns

TRUE if this column is a primary key for the corresponding SQL database table, or FALSE otherwise.

References util::mysql::SqlTableColumnDescriptionBase::is_primary_key_.

10.19.3.8 set_column_index()

```
void util::mysql::SqlTableColumnDescriptionBase::set_column_index (
            size_t idx)  [inline], [inherited]
```

Set the index of this SQL table column.

Parameters

in	*idx*	The index to set.

References util::mysql::SqlTableColumnDescriptionBase::column_index_.

10.19.3.9 SetObjectValue()

```
void util::mysql::SqlTableColumnDescription< C, vector< uint8_t > >::SetObjectValue (
            C & obj,
            const vector< uint8_t > & value) const  [inline], [protected], [inherited]
```

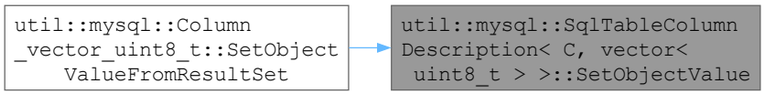
Set the object member value.

Parameters

in	*obj*	The object of type C.
in	*value*	The value of type T, to set.

Referenced by util::mysql::Column_vector_uint8_t< C >::SetObjectValueFromResultSet().

Here is the caller graph for this function:



10.19.3.10 SetObjectValueFromResultSet()

```
template<class C >
void util::mysql::Column_vector_uint8_t< C >::SetObjectValueFromResultSet (
            C & obj,
            const ResultSet & results) const  [inline]
```

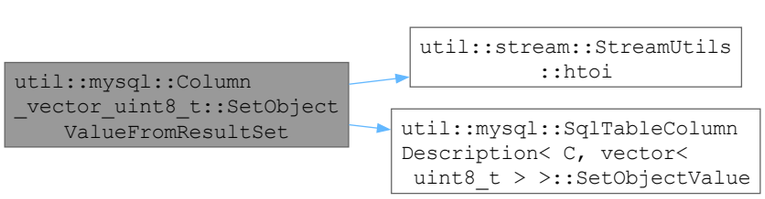
Set the value in the field of the corresponding object from an SQL ResultSet Object.

Parameters

in,out	*obj*	A reference to the object whose field value will be set.
in	*results*	A reference to the ResultSet object.

References util::stream::StreamUtils::htoi(), and util::mysql::SqlTableColumnDescription< C, vector< uint8_t > >::SetObjectValue().

Here is the call graph for this function:



10.19.3.11 SetPreparedStatementValueFromObject()

```
template<class C >
void util::mysql::Column_vector_uint8_t< C >::SetPreparedStatementValueFromObject (
            const C & obj,
            PreparedStatement & stmt) const  [inline]
```

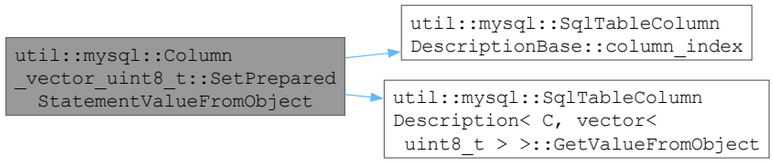
Set the value for the SQL PreparedStatement object from the value of the field of the corresponding object.

Parameters

in	*obj*	A reference to the object whose field value will be read.
in,out	*stmt*	A reference to the SQL PreparedStatement object whose column value will be set.

References util::mysql::SqlTableColumnDescriptionBase::column_index(), and util::mysql::SqlTableColumnDescription< C, vector< uint8_

Here is the call graph for this function:



10.19.4 Member Data Documentation

10.19.4.1 accessor_

function<vector< uint8_t >(const C&)> util::mysql::SqlTableColumnDescription< C, vector< uint8_t > >::accessor_ [private], [inherited]

The accessor function object wrapper.

This object is used as a wrapper for the accessor method of the template class C. It is used to retrieve the value of the SQL table column from the object for generating an INSERT or UPDATE statement.

10.19.4.2 column_index_

size_t util::mysql::SqlTableColumnDescriptionBase::column_index_ [private], [inherited]

The index of the SQL table column.

Referenced by util::mysql::SqlTableColumnDescriptionBase::column_index(), and util::mysql::SqlTableColumnDescriptionBase::set_colum

10.19.4.3 column_name_

string util::mysql::SqlTableColumnDescriptionBase::column_name_ [private], [inherited]

The name of the SQL table column.

Referenced by util::mysql::SqlTableColumnDescriptionBase::column_name(), and util::mysql::SqlTableColumnDescriptionBase::column_s

10.19.4.4 data_type_

ColumDataType_t util::mysql::SqlTableColumnDescriptionBase::data_type_ [private], [inherited]

The enumerated SQL table column data type.

Referenced by util::mysql::SqlTableColumnDescriptionBase::column_select_name(), and util::mysql::SqlTableColumnDescriptionBase::da

10.19.4.5 is_primary_key_

bool util::mysql::SqlTableColumnDescriptionBase::is_primary_key_ [private], [inherited]

Indicates whether the column is part of the primary key for the SQL table.

Referenced by util::mysql::SqlTableColumnDescriptionBase::is_primary_key().

10.19.4.6 mutator_

```
function<void(C&, vector< uint8_t >)> util::mysql::SqlTableColumnDescription< C, vector< uint8_t >
_t > >::mutator_  [private], [inherited]
```

The mutator function object wrapper.

This object is used as a wrapper for the mutator method of the template class C. It is used to set the value of an object from the result set of an SQL SELECT query statement.

The documentation for this class was generated from the following file:

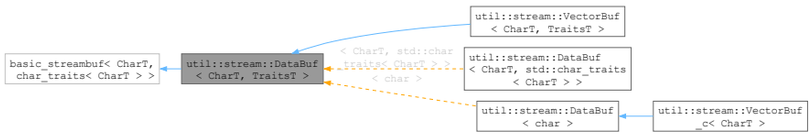
- utility/SqlTableColumnDescription.h

10.20 util::stream::DataBuf< CharT, TraitsT > Class Template Reference

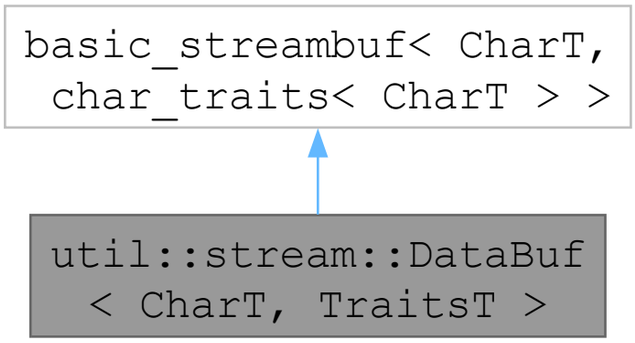
Template class for wrapping an arbitrary contiguous section of data as a std::basic_streambuf object.

```
#include <DataBuf.h>
```

Inheritance diagram for util::stream::DataBuf< CharT, TraitsT >:



Collaboration diagram for util::stream::DataBuf< CharT, TraitsT >:



Public Member Functions

- DataBuf (const CharT *data, size_t size)

  Parameterized constructor.

10.20.1 Detailed Description

template<typename CharT, typename TraitsT = char_traits<CharT>>
class util::stream::DataBuf< CharT, TraitsT >

Template class for wrapping an arbitrary contiguous section of data as a std::basic_streambuf object.

10.20.2 Constructor & Destructor Documentation

10.20.2.1 DataBuf()

```
template<typename CharT , typename TraitsT = char_traits<CharT>>
util::stream::DataBuf< CharT, TraitsT >::DataBuf (
            const CharT * data,
            size_t size)  [inline]
```

Parameterized constructor.

Parameters

in	*data*	A pointer to the start of the memory section of interest.
in	*size*	The size of the memory section, in bytes.

The documentation for this class was generated from the following file:

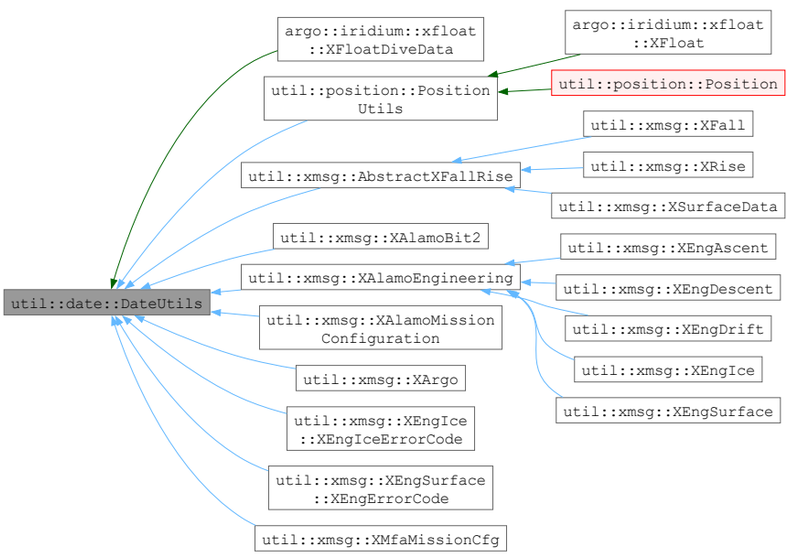
- utility/DataBuf.h

10.21 util::date::DateUtils Class Reference

Class providing library of useful date and time functions.

`#include <DateUtils.h>`

Inheritance diagram for util::date::DateUtils:



Static Public Member Functions

- static string GenerateCurrentUtcTimeIso8601String ()

  Generate an ISO8601 format date/time string representation of the current time, in UTC.
- static string GenerateIso8601String (const ptime &p, bool extended=true)

  Generate an ISO8601 format date/time string representation of a POSIX time object.
- static double GenerateJulianDay (const ptime &dt, const ptime &ref_dt)

  Convert a date/time to fractional days since a reference date/time.
- static ptime GetCurrentUtcTime ()

  Generate a POSIX time object representing the current date and time (rounded to the nearest second).

10.21.1 Detailed Description

Class providing library of useful date and time functions.

10.21.2 Member Function Documentation

10.21.2.1 GenerateCurrentUtcTimeIso8601String()

```
static string util::date::DateUtils::GenerateCurrentUtcTimeIso8601String ()  [inline], [static]
```

Generate an ISO8601 format date/time string representation of the current time, in UTC.

Note

All date/time objects use the UTC time-zone.

Returns

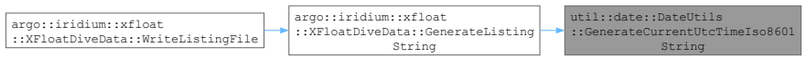
An ISO8601 format date/time string representation of the current time.

References GenerateIso8601String(), and GetCurrentUtcTime().

Referenced by argo::iridium::xfloat::XFloatDiveData::GenerateListingString().

Here is the call graph for this function:

Here is the caller graph for this function:



10.21.2.2 GenerateIso8601String()

```
static string util::date::DateUtils::GenerateIso8601String (
            const ptime & p,
            bool extended = true)  [inline], [static]
```

Generate an ISO8601 format date/time string representation of a POSIX time object.

Note

All date/time objects use the UTC time-zone.

Parameters

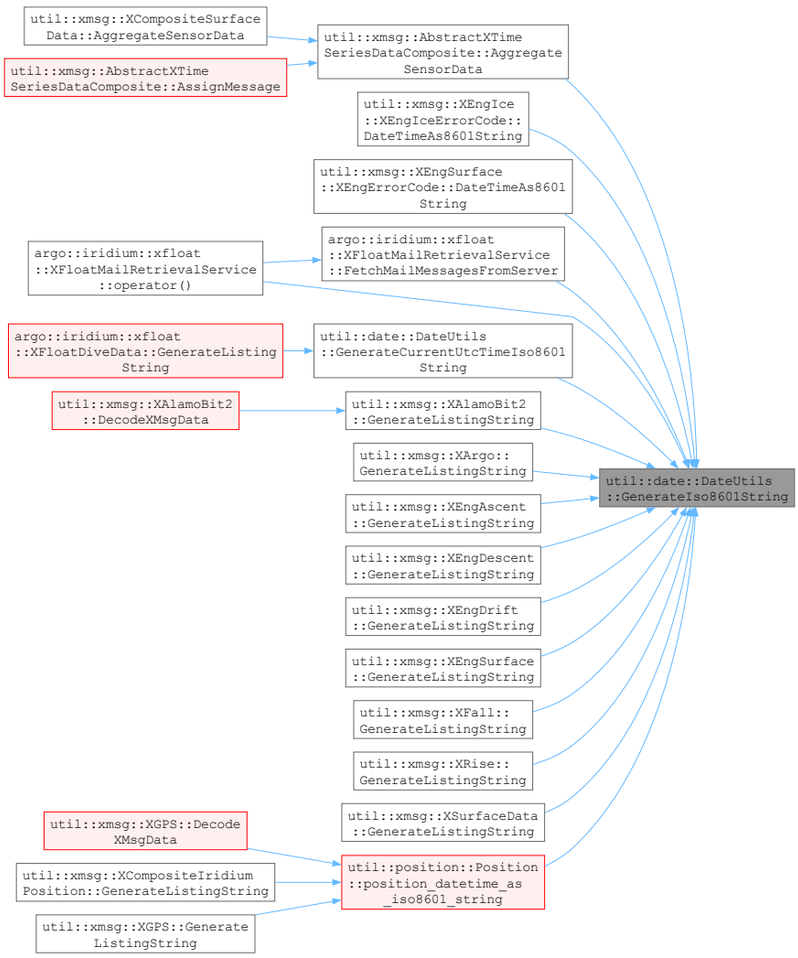
in	*p*	POSIX time from which to generate string.
in	*extended*	If TRUE, use the ISO 8601 extended format (YYYY-MM-DDThh:mm:ssZ); otherwise, use the basic format (YYYYMMDDThhmmss).

Returns

An ISO8601 format date/time string representation of the POSIX time object. If p is equal to boost::posix_↩ time::not_a_date_time, then the string "0000-00-00T00:00:00Z" (extended) or "00000000000000" (basic) will be returned.

Referenced by util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::AggregateSensorData(), util::xmsg::XEngIce::XEngIce
util::xmsg::XEngSurface::XEngErrorCode::DateTimeAs8601String(), argo::iridium::xfloat::XFloatMailRetrievalService::FetchMailMessages
GenerateCurrentUtcTimeIso8601String(), util::xmsg::XAlamoBit2::GenerateListingString(), util::xmsg::XArgo::GenerateListingString(),
util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListing
util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XRise::GenerateListingString(),
util::xmsg::XSurfaceData::GenerateListingString(), argo::iridium::xfloat::XFloatMailRetrievalService::operator()(), and
util::position::Position::position_datetime_as_iso8601_string().

Here is the caller graph for this function:



10.21.2.3 GenerateJulianDay()

```
static double util::date::DateUtils::GenerateJulianDay (
            const ptime & dt,
            const ptime & ref_dt)  [inline], [static]
```

Convert a date/time to fractional days since a reference date/time.

Parameters

in	*dt*	The date/time to convert.
in	*ref*↩ *_dt*	The reference date/time (day 0).

Exceptions

util::log::MessageException	if either date/time is not valid.

Returns

The number of days (with millisecond resolution) from `ref_dt` to `dt`.

References CATCH_BLOCK, and EXC_THROW.

10.21.2.4 GetCurrentUtcTime()

```
static ptime util::date::DateUtils::GetCurrentUtcTime ()  [inline], [static]
```

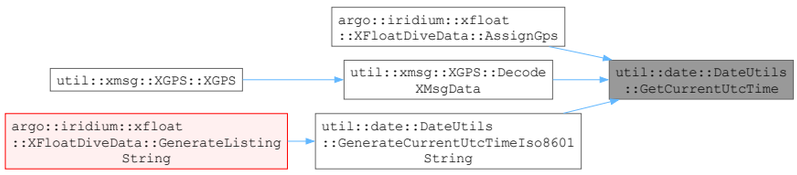
Generate a POSIX time object representing the current date and time (rounded to the nearest second).

Returns

A POSIX time object representing the current date and time (rounded to the nearest second).

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignGps(), util::xmsg::XGPS::DecodeXMsgData(), and GenerateCurrentUtcTimeIso8601String().

Here is the caller graph for this function:



The documentation for this class was generated from the following file:

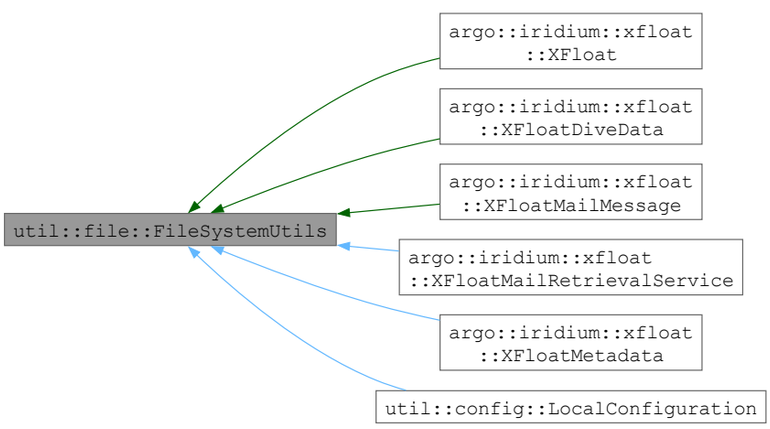
- utility/DateUtils.h

10.22 util::file::FileSystemUtils Class Reference

Class providing library of useful filesystem functions.

```
#include <FileSystemUtils.h>
```

Inheritance diagram for util::file::FileSystemUtils:



Static Public Member Functions

- static const path & GetDirectoryPath (const path &dir, bool create=false)
  Get a directory path.
- static path GetDirectoryPathFromString (const string &dir, bool create=false)
  Generate a boost::filesystem::path object from the directory path string.
- static vector< path > ListDirectories (const path &dir)
  List all directories contained in the given directory.
- static vector< path > ListFiles (path dir, size_t n=0)
  List all regular files contained in the given directory.
- static path MoveFileToDirectory (const path &file, const path &dir_dst, bool create_dir=false)
  Copy a file into a directory.

10.22.1 Detailed Description

Class providing library of useful filesystem functions.

10.22.2 Member Function Documentation

10.22.2.1 GetDirectoryPath()

```
static const path & util::file::FileSystemUtils::GetDirectoryPath (
            const path & dir,
            bool create = false)  [inline], [static]
```

Get a directory path.

Exceptions

MessageException	if create == FALSE and the directory specified by path does not exist.

Parameters

in	*dir*	Directory path.
in	*create*	If TRUE, the directory specified by path will be created if it does not already exists. If FALSE, an exception will be thrown if the directory does not exist.

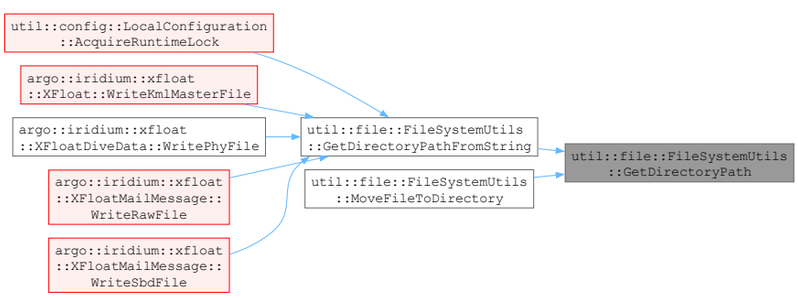
Returns

dir

References CATCH_BLOCK, EXC_THROW, and ldbg.

Referenced by GetDirectoryPathFromString(), and MoveFileToDirectory().

Here is the caller graph for this function:



10.22.2.2 GetDirectoryPathFromString()

```
static path util::file::FileSystemUtils::GetDirectoryPathFromString (
            const string & dir,
            bool create = false)  [inline], [static]
```

Generate a boost::filesystem::path object from the directory path string.

Exceptions

MessageException	if create == FALSE and the directory specified by path does not exist.

Parameters

in	*dir*	Directory path string.
in	*create*	If TRUE, the directory specified by path will be created if it does not already exists. If FALSE, an exception will be thrown if the directory does not exist.

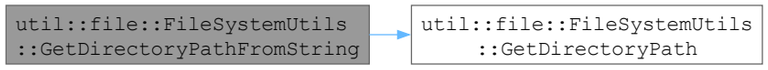
Returns

A boost::filesystem::path object from the directory path string.

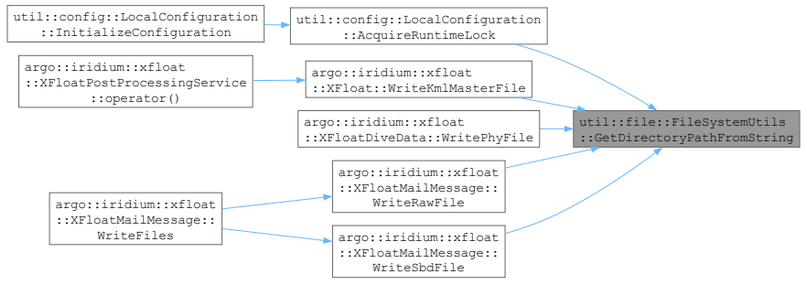
References CATCH_BLOCK, and GetDirectoryPath().

Referenced by util::config::LocalConfiguration::AcquireRuntimeLock(), argo::iridium::xfloat::XFloat::WriteKmlMasterFile(), argo::iridium::xfloat::XFloatDiveData::WritePhyFile(), argo::iridium::xfloat::XFloatMailMessage::WriteRawFile(), and argo::iridium::xfloat::XFloatMailMessage::WriteSbdFile().

Here is the call graph for this function:



Here is the caller graph for this function:



10.22.2.3 ListDirectories()

```
static vector< path > util::file::FileSystemUtils::ListDirectories (
            const path & dir)  [inline], [static]
```

List all directories contained in the given directory.

Parameters

in	*dir*	The path to the directory of interest.

Exceptions

log::MessageException	if the directory does not exist, is not a directory, or an error occurs.

Returns

A vector of paths to all the directories contained in the given directory.

References CATCH_BLOCK, and EXC_THROW.

10.22.2.4 ListFiles()

```
static vector< path > util::file::FileSystemUtils::ListFiles (
            path dir,
            size_t n = 0)  [inline], [static]
```

List all regular files contained in the given directory.

Parameters

in	*dir*	The path to the directory of interest.
in	*n*	If n>0, the maximum number of paths to return, else no effect.

Exceptions

log::MessageException	if the directory does not exist, is not a directory, or an error occurs.

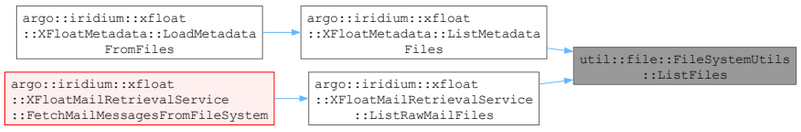
Returns

A vector of paths to all the regular files contained in the given directory.

References CATCH_BLOCK, and EXC_THROW.

Referenced by argo::iridium::xfloat::XFloatMetadata::ListMetadataFiles(), and argo::iridium::xfloat::XFloatMailRetrievalService::ListRawMa

Here is the caller graph for this function:



10.22.2.5 MoveFileToDirectory()

```
static path util::file::FileSystemUtils::MoveFileToDirectory (
            const path & file,
            const path & dir_dst,
            bool create_dir = false)  [inline], [static]
```

Copy a file into a directory.

Warning

Despite its name, this function copies the file and does not remove the source. If a file with the same name already exists in the destination directory, it is left unchanged. It is not currently called.

Parameters

in	*file*	The file to copy.
in	*dir_dst*	The destination directory.
in	*create_dir*	If TRUE, create the destination directory if it does not exist.

Exceptions

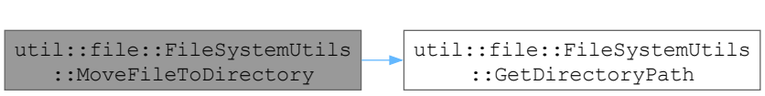
util::log::MessageException	if the file does not exist, is not a regular file, or cannot be copied.

Returns

The path of the file in the destination directory.

References CATCH_BLOCK, EXC_THROW, and GetDirectoryPath().

Here is the call graph for this function:



The documentation for this class was generated from the following file:

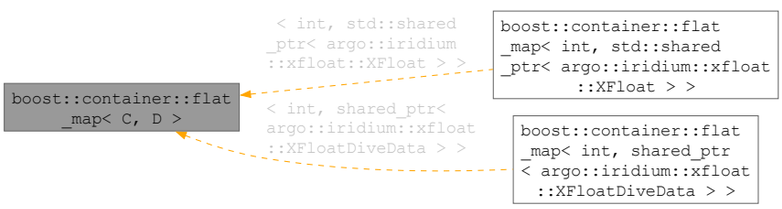
- utility/FileSystemUtils.h

10.23 boost::container::flat_map< C, D > Class Template Reference

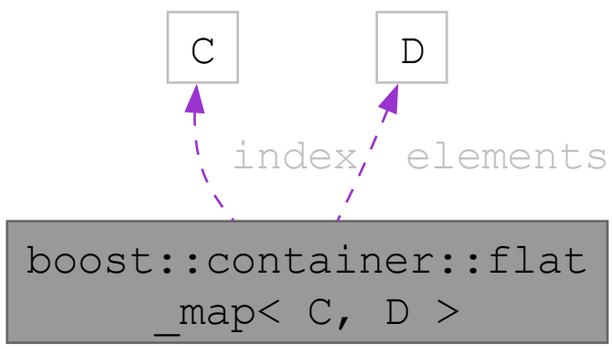
Dummy template class used for configuring Doxygen to interpret boost::container::flat_map correctly.

`#include <AdditionalDocumentation.h>`

Inheritance diagram for boost::container::flat_map< C, D >:



Collaboration diagram for boost::container::flat_map< C, D >:



Private Attributes

- D elements [ ]

  The elements.
- C index [ ]

  The index.

10.23.1 Detailed Description

template<class C, class D>
class boost::container::flat_map< C, D >

Dummy template class used for configuring Doxygen to interpret boost::container::flat_map correctly.

10.23.2 Member Data Documentation

10.23.2.1 elements

```
template<class C , class D >
D boost::container::flat_map< C, D >::elements[]  [private]
```

The elements.

10.23.2.2 index

```
template<class C , class D >
C boost::container::flat_map< C, D >::index[]  [private]
```

The index.

The documentation for this class was generated from the following file:

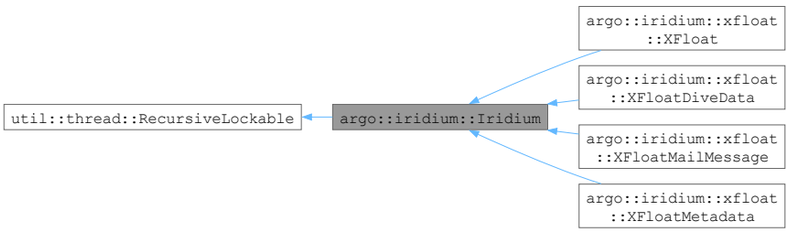
- AdditionalDocumentation.h

10.24 argo::iridium::Iridium Class Reference

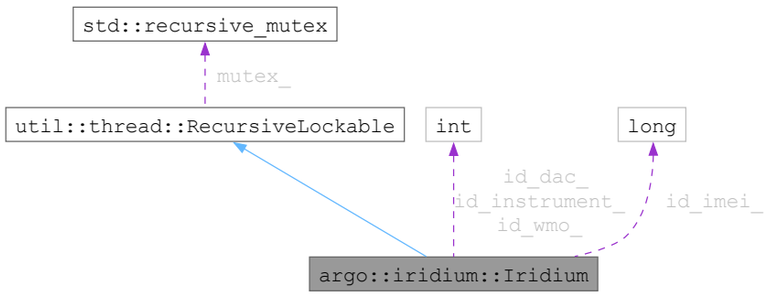
The Iridium instrument class.

```
#include <Iridium.h>
```

Inheritance diagram for argo::iridium::Iridium:



Collaboration diagram for argo::iridium::Iridium:



Public Member Functions

- Iridium (int whoi, long imei=IRI_ID_UNKNOWN, int wmo=IRI_ID_UNKNOWN, int dac=IRI_ID_UNKNOWN)

  The parameterized constructor.
- virtual ~Iridium ()

  The destructor.
- int id_dac () const

  Get the unique Data Assembly Center (DAC) identifier.
- long id_imei () const

  Get the unique International Mobile Equipment Identifier (IMEI).
- int id_instrument () const

  Get the unique instrument identifier or instrument serial number.
- int id_wmo () const

  Get the unique World Meteorological Organization (WMO) identifier.
- bool LockObject (bool block)

  Lock the object.
- void UnlockObject ()

  Unlock the object.

Protected Member Functions

- Iridium ()

  The default constructor.
- void set_id_dac (int id_dac)

  Set the unique Data Assembly Center (DAC) identifier.
- void set_id_imei (long id_imei)

  Set the unique International Mobile Equipment Identifier (IMEI).
- void set_id_instrument (int id_instrument)

  Set the unique instrument identifier or instrument serial number.
- void set_id_wmo (int id_wmo)

  Set the unique World Meteorological Organization (WMO) identifier.

Protected Attributes

- recursive_mutex mutex_

  Recursive mutual exclusion lock for synchronizing operations.

Private Attributes

- int id_dac_

  The unique Data Assembly Center (DAC) identifier.
- long id_imei_

  The unique International Mobile Equipment Identifier (IMEI).
- int id_instrument_

  The unique instrument identifier or instrument serial number.
- int id_wmo_

  The unique World Meteorological Organization (WMO) identifier.

10.24.1 Detailed Description

The Iridium instrument class.

This class represents an individual instrument and its associated unique identifiers. This class is not used explicitly. Rather, all other classes that are associated with an instrument's unique identifiers inherit from this class.

Note

This object is lockable. It contains a recursive mutual exclusion (mutex) lock in order to synchronize access to and modification of objects of this type by various worker threads. Once the object is locked, any other thread that attempts to access or modify the object will block until the initial lock is released. Since the mutex is recursive, the object may be locked multiple times by the same thread without causing the thread to block indefinitely.

10.24.2 Constructor & Destructor Documentation

10.24.2.1 Iridium() [1/2]

```
argo::iridium::Iridium::Iridium ()  [inline], [protected]
```

The default constructor.

10.24.2.2 Iridium() [2/2]

```
argo::iridium::Iridium::Iridium (
            int whoi,
            long imei = IRI_ID_UNKNOWN,
            int wmo = IRI_ID_UNKNOWN,
            int dac = IRI_ID_UNKNOWN)  [inline]
```

The parameterized constructor.

Parameters

in	*imei*	The unique International Mobile Equipment Identifier (IMEI).
in	*whoi*	The unique WHOI identifier.
in	*wmo*	The unique World Meteorological Organization (WMO) identifier.
in	*dac*	The unique Data Assembly Center (DAC) identifier.

10.24.2.3 ~Iridium()

```
virtual argo::iridium::Iridium::~Iridium ()  [inline], [virtual]
```

The destructor.

10.24.3 Member Function Documentation

10.24.3.1 id_dac()

```
int argo::iridium::Iridium::id_dac () const  [inline]
```

Get the unique Data Assembly Center (DAC) identifier.

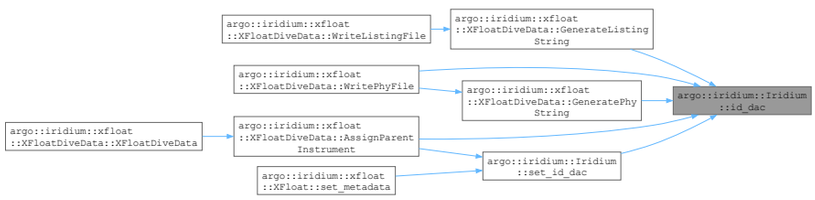
Returns

The unique Data Assembly Center (DAC) identifier.

References id_dac_, and util::thread::RecursiveLockable::mutex_.

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignParentInstrument(), argo::iridium::xfloat::XFloatDiveData::GenerateListingStrir argo::iridium::xfloat::XFloatDiveData::GeneratePhyString(), set_id_dac(), and argo::iridium::xfloat::XFloatDiveData::WritePhyFile().

Here is the caller graph for this function:



10.24.3.2 id_imei()

```
long argo::iridium::Iridium::id_imei () const  [inline]
```

Get the unique International Mobile Equipment Identifier (IMEI).

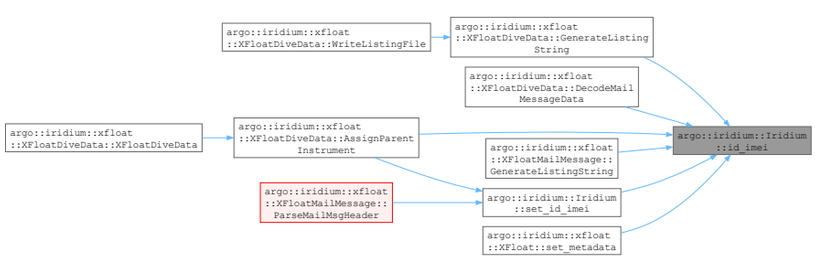
Returns

The unique International Mobile Equipment Identifier (IMEI).

References id_imei_, and util::thread::RecursiveLockable::mutex_.

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignParentInstrument(), argo::iridium::xfloat::XFloatDiveData::DecodeMailMessag argo::iridium::xfloat::XFloatDiveData::GenerateListingString(), argo::iridium::xfloat::XFloatMailMessage::GenerateListingString(), set_id_imei(), and argo::iridium::xfloat::XFloat::set_metadata().

Here is the caller graph for this function:



10.24.3.3 id_instrument()

```
int argo::iridium::Iridium::id_instrument () const  [inline]
```

Get the unique instrument identifier or instrument serial number.

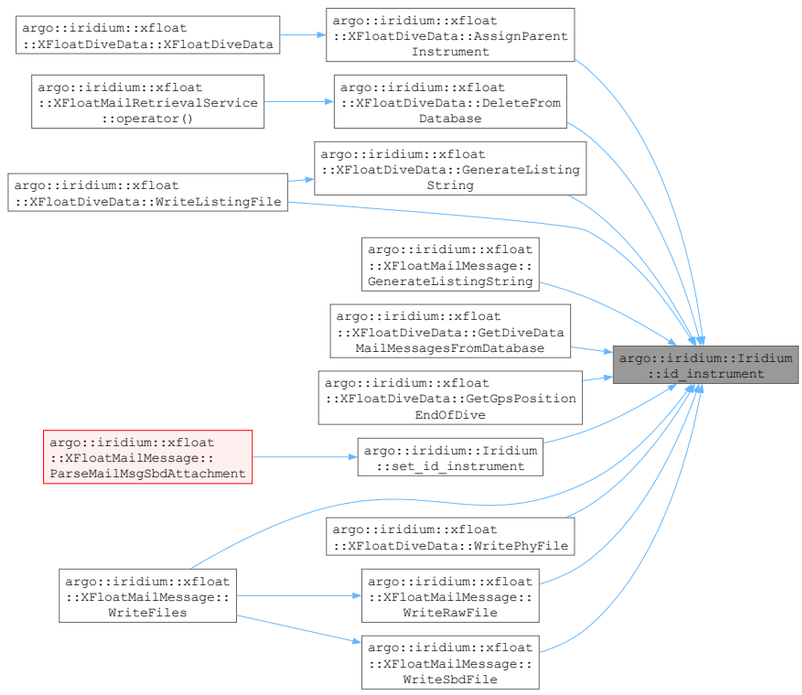
Returns

The unique instrument identifier or instrument serial number.

References id_instrument_, and util::thread::RecursiveLockable::mutex_.

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignParentInstrument(), argo::iridium::xfloat::XFloatDiveData::DeleteFromDatabas argo::iridium::xfloat::XFloatDiveData::GenerateListingString(), argo::iridium::xfloat::XFloatMailMessage::GenerateListingString(), argo::iridium::xfloat::XFloatDiveData::GetDiveDataMailMessagesFromDatabase(), argo::iridium::xfloat::XFloatDiveData::GetGpsPositionE set_id_instrument(), argo::iridium::xfloat::XFloatMailMessage::WriteFiles(), argo::iridium::xfloat::XFloatDiveData::WriteListingFile(), argo::iridium::xfloat::XFloatDiveData::WritePhyFile(), argo::iridium::xfloat::XFloatMailMessage::WriteRawFile(), and argo::iridium::xfloat::XFloatMailMessage::WriteSbdFile().

Here is the caller graph for this function:



10.24.3.4 id_wmo()

```
int argo::iridium::Iridium::id_wmo () const  [inline]
```

Get the unique World Meteorological Organization (WMO) identifier.

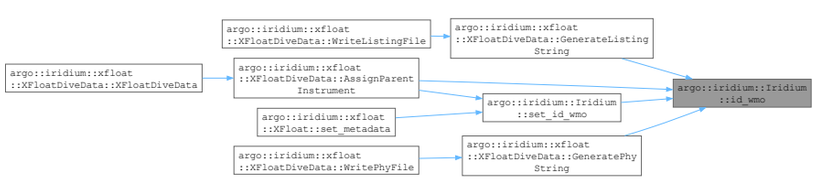
Returns

The unique World Meteorological Organization (WMO) identifier.

References id_wmo_, and util::thread::RecursiveLockable::mutex_.

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignParentInstrument(), argo::iridium::xfloat::XFloatDiveData::GenerateListingStrir argo::iridium::xfloat::XFloatDiveData::GeneratePhyString(), and set_id_wmo().

Here is the caller graph for this function:



10.24.3.5 LockObject()

```
bool util::thread::RecursiveLockable::LockObject (
            bool block)  [inline], [inherited]
```

Lock the object.

Parameters

in	*block*	If TRUE, then the calling thread will block until the lock becomes available.

Exceptions

util::log::MessageException	if an error occurs.

Returns

TRUE if the lock was successfully obtained, or FALSE if block is FALSE and another thread owns the lock on the mutex.

References CATCH_BLOCK, and util::thread::RecursiveLockable::mutex_.

10.24.3.6 set_id_dac()

```
void argo::iridium::Iridium::set_id_dac (
            int id_dac)  [inline], [protected]
```

Set the unique Data Assembly Center (DAC) identifier.

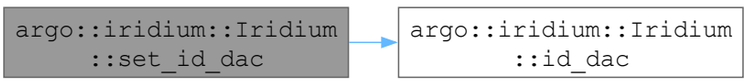
Parameters

in	*id_dac*	The unique Data Assembly Center (DAC) identifier.

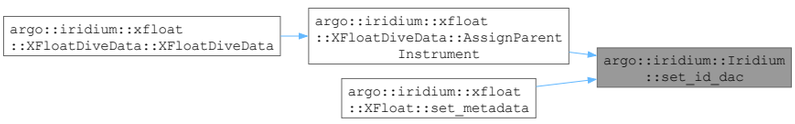
References id_dac(), id_dac_, and util::thread::RecursiveLockable::mutex_.

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignParentInstrument(), and argo::iridium::xfloat::XFloat::set_metadata().

Here is the call graph for this function:



Here is the caller graph for this function:



10.24.3.7 set_id_imei()

```
void argo::iridium::Iridium::set_id_imei (
            long id_imei)  [inline], [protected]
```

Set the unique International Mobile Equipment Identifier (IMEI).

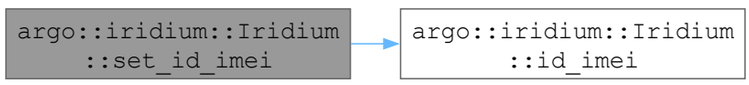
Parameters

in	*id_imei*	The unique International Mobile Equipment Identifier (IMEI).

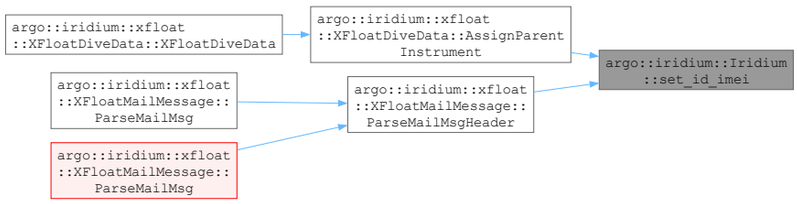
References id_imei(), id_imei_, and util::thread::RecursiveLockable::mutex_.

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignParentInstrument(), and argo::iridium::xfloat::XFloatMailMessage::ParseMailM

Here is the call graph for this function:



Here is the caller graph for this function:



10.24.3.8 set_id_instrument()

```
void argo::iridium::Iridium::set_id_instrument (
            int id_instrument)  [inline], [protected]
```

Set the unique instrument identifier or instrument serial number.

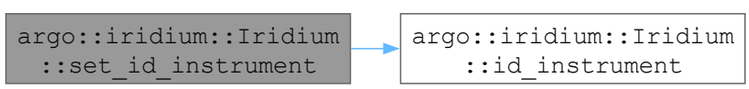
Parameters

in	*id_instrument*	The unique instrument identifier or instrument serial number

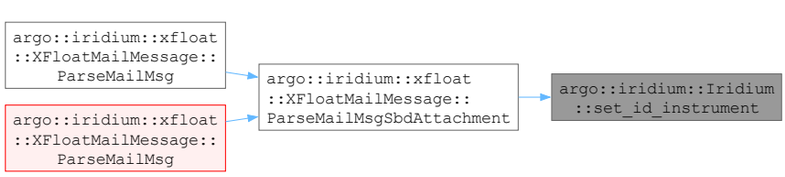
References id_instrument(), id_instrument_, and util::thread::RecursiveLockable::mutex_.

Referenced by argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgSbdAttachment().

Here is the call graph for this function:



Here is the caller graph for this function:



10.24.3.9 set_id_wmo()

```
void argo::iridium::Iridium::set_id_wmo (
            int id_wmo)  [inline], [protected]
```

Set the unique World Meteorological Organization (WMO) identifier.

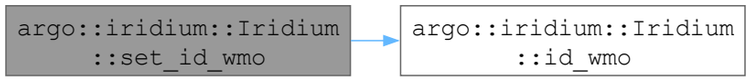
Parameters

in	*id_wmo*	The unique World Meteorological Organization (WMO) identifier.

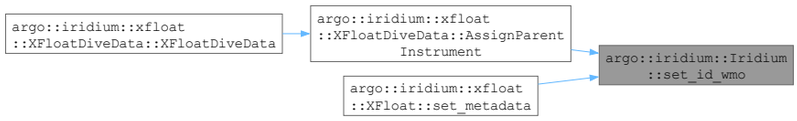
References id_wmo(), id_wmo_, and util::thread::RecursiveLockable::mutex_.

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignParentInstrument(), and argo::iridium::xfloat::XFloat::set_metadata().

Here is the call graph for this function:



Here is the caller graph for this function:



10.24.3.10 UnlockObject()

```
void util::thread::RecursiveLockable::UnlockObject ()  [inline], [inherited]
```

Unlock the object.

Exceptions

util::log::MessageException	if an error occurs.

References CATCH_BLOCK, and util::thread::RecursiveLockable::mutex_.

10.24.4 Member Data Documentation

10.24.4.1 id_dac_

```
int argo::iridium::Iridium::id_dac_  [private]
```

The unique Data Assembly Center (DAC) identifier.

Referenced by id_dac(), and set_id_dac().

10.24.4.2 id_imei_

```
long argo::iridium::Iridium::id_imei_  [private]
```

The unique International Mobile Equipment Identifier (IMEI).

Referenced by id_imei(), and set_id_imei().

10.24.4.3 id_instrument_

```
int argo::iridium::Iridium::id_instrument_  [private]
```

The unique instrument identifier or instrument serial number.

Referenced by id_instrument(), and set_id_instrument().

10.24.4.4 id_wmo_

```
int argo::iridium::Iridium::id_wmo_  [private]
```

The unique World Meteorological Organization (WMO) identifier.

Referenced by id_wmo(), and set_id_wmo().

10.24.4.5 mutex_

```
recursive_mutex util::thread::RecursiveLockable::mutex_  [mutable], [protected], [inherited]
```

Recursive mutual exclusion lock for synchronizing operations.

Referenced by util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AggregateS
argo::iridium::xfloat::XFloat::argo_instrument_type(), argo::iridium::xfloat::XFloatDiveData::argo_instrument_type(),
argo::iridium::xfloat::XFloatDiveData::AssignArgoData(), argo::iridium::xfloat::XFloatDiveData::AssignAuxTemperatureBinned(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignAuxTemperatureMe
argo::iridium::xfloat::XFloatDiveData::AssignDriftPressure(), argo::iridium::xfloat::XFloatDiveData::AssignDriftSalinity(),
argo::iridium::xfloat::XFloatDiveData::AssignDriftTemperatre(), argo::iridium::xfloat::XFloatDiveData::AssignEngineeringAbort(),
argo::iridium::xfloat::XFloatDiveData::AssignEngineeringAscent(), argo::iridium::xfloat::XFloatDiveData::AssignEngineeringBitFail(),
argo::iridium::xfloat::XFloatDiveData::AssignEngineeringBitPass(), argo::iridium::xfloat::XFloatDiveData::AssignEngineeringDescent(),
argo::iridium::xfloat::XFloatDiveData::AssignEngineeringDiagnostic(), argo::iridium::xfloat::XFloatDiveData::AssignEngineeringDrift(),
argo::iridium::xfloat::XFloatDiveData::AssignEngineeringIce(), argo::iridium::xfloat::XFloatDiveData::AssignEngineeringStandard(),
argo::iridium::xfloat::XFloatDiveData::AssignEngineeringSurface(), argo::iridium::xfloat::XFloatDiveData::AssignFallTimeSeries(),
argo::iridium::xfloat::XFloatDiveData::AssignGps(), argo::iridium::xfloat::XFloatDiveData::AssignMailMessage(),
argo::iridium::xfloat::XFloatDiveData::AssignMissionCmdEchoback(), argo::iridium::xfloat::XFloatDiveData::AssignParentInstrument(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignPressureMessage(
argo::iridium::xfloat::XFloatDiveData::AssignProfilePressureBinned(), argo::iridium::xfloat::XFloatDiveData::AssignProfilePressureUnbinn
argo::iridium::xfloat::XFloatDiveData::AssignProfileSalinityBinned(), argo::iridium::xfloat::XFloatDiveData::AssignProfileSalinityUnbinned()
argo::iridium::xfloat::XFloatDiveData::AssignProfileTemperatureBinned(), argo::iridium::xfloat::XFloatDiveData::AssignProfileTemperatureU
argo::iridium::xfloat::XFloatDiveData::AssignPumpTimeSeries(), argo::iridium::xfloat::XFloatDiveData::AssignRiseTimeSeries(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignSalinityMessage(),
argo::iridium::xfloat::XFloatDiveData::AssignSurfaceTimeSeries(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_C
argo::iridium::xfloat::XFloatDiveData::AssignValveData(), argo::iridium::xfloat::XFloatDiveData::CheckCompleteness(),
argo::iridium::xfloat::XFloatMetadata::comment(), argo::iridium::xfloat::XFloatDiveData::ComputePressureOffsetFromSurfaceTimeSeries(
argo::iridium::xfloat::XFloatMetadata::deployer(), argo::iridium::xfloat::XFloatMetadata::deployer_address(), argo::iridium::xfloat::XFloatMe
argo::iridium::xfloat::XFloatMetadata::deployment_datetime(), argo::iridium::xfloat::XFloatMetadata::deployment_datetime_status(),
argo::iridium::xfloat::XFloatMetadata::deployment_latitude(), argo::iridium::xfloat::XFloatMetadata::deployment_longitude(),
argo::iridium::xfloat::XFloatMetadata::deployment_platform(), argo::iridium::xfloat::XFloatMetadata::deployment_position_status(),
argo::iridium::xfloat::XFloatMetadata::deployment_type(), argo::iridium::xfloat::XFloat::fw_version(), argo::iridium::xfloat::XFloat::gain_pres
argo::iridium::xfloat::XFloat::gain_pressure_surface(), argo::iridium::xfloat::XFloat::gain_salinity(), argo::iridium::xfloat::XFloat::gain_salinit
argo::iridium::xfloat::XFloat::gain_temperature(), argo::iridium::xfloat::XFloat::gain_temperature_surface(), argo::iridium::xfloat::XFloatDive
argo::iridium::xfloat::XFloatDiveData::GenerateCompositeIridiumPosition(), argo::iridium::xfloat::XFloatDiveData::GenerateCompositePun
argo::iridium::xfloat::XFloatDiveData::GenerateCompositeRiseTimeSeriesData(), argo::iridium::xfloat::XFloatDiveData::GenerateListingSt
util::xmsg::XCompositeDrift::GenerateListingString(), util::xmsg::XCompositeHiResProfile::GenerateListingString(),
util::xmsg::XCompositeProfile::GenerateListingString(), argo::iridium::xfloat::XFloatDiveData::GenerateOrderedMomsnList(),
argo::iridium::xfloat::XFloatDiveData::GeneratePhyString(), util::xmsg::XCompositeDrift::GeneratePhyString(), util::xmsg::XCompositeHiR
util::xmsg::XCompositeProfile::GeneratePhyString(), argo::iridium::xfloat::XFloatDiveData::GetDiveDataMailMessagesFromDatabase(),
argo::iridium::xfloat::XFloatDiveData::GetGainsAndOffsets(), argo::iridium::xfloat::XFloatDiveData::GetGpsPositionEndOfDive(),
argo::iridium::xfloat::XFloatDiveData::GetSurfaceGainsAndOffsets(), argo::iridium::xfloat::XFloatDiveData::HasBinnedProfileData(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::HasContiguousMessageI
argo::iridium::xfloat::XFloatDiveData::HasContiguousMomsns(), argo::iridium::xfloat::XFloatDiveData::HasContiguousPids(),
argo::iridium::xfloat::XFloatDiveData::HasDiveData(), argo::iridium::xfloat::XFloatDiveData::HasDriftData(), argo::iridium::xfloat::XFloatDiv
argo::iridium::xfloat::XFloatDiveData::HasFallRiseData(), argo::iridium::xfloat::XFloatDiveData::HasPositionData(),
argo::iridium::xfloat::XFloatDiveData::HasUnbinnedProfileData(), argo::iridium::xfloat::XFloatDiveData::HasValidGpsPosition(),
id_dac(), argo::iridium::xfloat::XFloatDiveData::id_dive(), argo::iridium::xfloat::XFloatMailMessage::id_dive(), id_imei(),
id_instrument(), argo::iridium::xfloat::XFloatMailMessage::id_msg(), argo::iridium::xfloat::XFloatMailMessage::id_msg_uid(),
argo::iridium::xfloat::XFloatMailMessage::id_sbd_momsn(), argo::iridium::xfloat::XFloatMailMessage::id_sbd_mtmsn(),
argo::iridium::xfloat::XFloatMailMessage::id_sbd_packet(), id_wmo(), argo::iridium::xfloat::XFloatDiveData::intermediate_listing_file_path(

util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::is_aggregated_data_valid
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::IsDataComplete(),
argo::iridium::xfloat::XFloatMetadata::local_path(), util::thread::RecursiveLockable::LockObject(), argo::iridium::xfloat::XFloatMetadata::ma
argo::iridium::xfloat::XFloat::metadata(), argo::iridium::xfloat::XFloatMailMessage::msg_body_is_valid(), argo::iridium::xfloat::XFloatMailM
argo::iridium::xfloat::XFloatMailMessage::msg_header_datetime(), argo::iridium::xfloat::XFloatMailMessage::msg_header_from(),
argo::iridium::xfloat::XFloatMailMessage::msg_header_is_valid(), argo::iridium::xfloat::XFloatMailMessage::msg_header_raw_text(),
argo::iridium::xfloat::XFloatMailMessage::msg_header_subject(), argo::iridium::xfloat::XFloatMailMessage::msg_is_valid(),
argo::iridium::xfloat::XFloatMailMessage::msg_raw_text(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, T
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::num_saln_samples(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::num_temp_samples(),
argo::iridium::xfloat::XFloat::offset_pressure(), argo::iridium::xfloat::XFloat::offset_pressure_surface(), argo::iridium::xfloat::XFloat::offset_
argo::iridium::xfloat::XFloat::offset_salinity_surface(), argo::iridium::xfloat::XFloat::offset_temperature(), argo::iridium::xfloat::XFloat::offset
argo::iridium::xfloat::XFloatMailMessage::ParseMailMsg(), argo::iridium::xfloat::XFloatMailMessage::ParseMailMsg(),
argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgBody(), argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgHeader(),
argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgSbdAttachment(), argo::iridium::xfloat::XFloat::phy_file_generation_is_enabled(),
argo::iridium::xfloat::XFloatDiveData::phy_file_path(), argo::iridium::xfloat::XFloatMetadata::principal_investigators(),
argo::iridium::xfloat::XFloatMetadata::principal_investigators_address(), argo::iridium::xfloat::XFloatMetadata::principal_investigators_orig
argo::iridium::xfloat::XFloatMailMessage::raw_file_local_path(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLA
argo::iridium::xfloat::XFloatDiveData::ResetData(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_C
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SamplePresAsFloat(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleSaln(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleSalnAsFloat(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleTemp(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleTempAsFloat(),
argo::iridium::xfloat::XFloatDiveData::SaveObjectToDatabase(), argo::iridium::xfloat::XFloatMailMessage::SaveObjectToDatabase(),
argo::iridium::xfloat::XFloatMetadata::SaveObjectToDatabase(), argo::iridium::xfloat::XFloatMailMessage::sbd_data(),
argo::iridium::xfloat::XFloatMailMessage::sbd_data_is_valid(), argo::iridium::xfloat::XFloatMailMessage::sbd_file_local_path(),
argo::iridium::xfloat::XFloatMailMessage::sbd_file_name(), argo::iridium::xfloat::XFloatMailMessage::sbd_file_size_bytes(),
argo::iridium::xfloat::XFloatMailMessage::sbd_session_status_code(), argo::iridium::xfloat::XFloatMailMessage::sbd_session_status_strir
argo::iridium::xfloat::XFloatDiveData::SensorGainsAndOffsetsAreKnown(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS,
argo::iridium::xfloat::XFloat::set_argo_instrument_type(), argo::iridium::xfloat::XFloatDiveData::set_argo_instrument_type(),
argo::iridium::xfloat::XFloatMetadata::set_comment(), argo::iridium::xfloat::XFloatMetadata::set_deployer(), argo::iridium::xfloat::XFloatMe
argo::iridium::xfloat::XFloatMetadata::set_deployment_cruise_id(), argo::iridium::xfloat::XFloatMetadata::set_deployment_datetime(),
argo::iridium::xfloat::XFloatMetadata::set_deployment_datetime_status(), argo::iridium::xfloat::XFloatMetadata::set_deployment_latitude(
argo::iridium::xfloat::XFloatMetadata::set_deployment_longitude(), argo::iridium::xfloat::XFloatMetadata::set_deployment_platform(),
argo::iridium::xfloat::XFloatMetadata::set_deployment_position_status(), argo::iridium::xfloat::XFloatMetadata::set_deployment_type(),
argo::iridium::xfloat::XFloat::set_fw_version(), argo::iridium::xfloat::XFloat::set_gain_pressure(), argo::iridium::xfloat::XFloat::set_gain_pre
argo::iridium::xfloat::XFloat::set_gain_salinity(), argo::iridium::xfloat::XFloat::set_gain_salinity_surface(), argo::iridium::xfloat::XFloat::set_g
argo::iridium::xfloat::XFloat::set_gain_temperature_surface(), set_id_dac(), argo::iridium::xfloat::XFloatDiveData::set_id_dive(),
argo::iridium::xfloat::XFloatMailMessage::set_id_dive(), set_id_imei(), set_id_instrument(), argo::iridium::xfloat::XFloatMailMessage::set_i
argo::iridium::xfloat::XFloatMailMessage::set_id_msg_uid(), argo::iridium::xfloat::XFloatMailMessage::set_id_sbd_momsn(),
argo::iridium::xfloat::XFloatMailMessage::set_id_sbd_mtmsn(), argo::iridium::xfloat::XFloatMailMessage::set_id_sbd_packet(),
set_id_wmo(), argo::iridium::xfloat::XFloatDiveData::set_intermediate_listing_file_path(), argo::iridium::xfloat::XFloatMetadata::set_local_
argo::iridium::xfloat::XFloatMetadata::set_manufacturer(), argo::iridium::xfloat::XFloat::set_metadata(), argo::iridium::xfloat::XFloatMailMe
argo::iridium::xfloat::XFloatMailMessage::set_msg_body_text(), argo::iridium::xfloat::XFloatMailMessage::set_msg_header_datetime(),
argo::iridium::xfloat::XFloatMailMessage::set_msg_header_from(), argo::iridium::xfloat::XFloatMailMessage::set_msg_header_is_valid(),
argo::iridium::xfloat::XFloatMailMessage::set_msg_header_raw_text(), argo::iridium::xfloat::XFloatMailMessage::set_msg_header_subjec
argo::iridium::xfloat::XFloatMailMessage::set_msg_is_valid(), argo::iridium::xfloat::XFloatMailMessage::set_msg_raw_text(),
argo::iridium::xfloat::XFloat::set_offset_pressure(), argo::iridium::xfloat::XFloat::set_offset_pressure_surface(), argo::iridium::xfloat::XFloat
argo::iridium::xfloat::XFloat::set_offset_salinity_surface(), argo::iridium::xfloat::XFloat::set_offset_temperature(),
argo::iridium::xfloat::XFloat::set_offset_temperature_surface(), argo::iridium::xfloat::XFloat::set_phy_file_generation_is_enabled(),
argo::iridium::xfloat::XFloatDiveData::set_phy_file_path(), argo::iridium::xfloat::XFloatMetadata::set_principal_investigators(),
argo::iridium::xfloat::XFloatMetadata::set_principal_investigators_address(), argo::iridium::xfloat::XFloatMetadata::set_principal_investiga
argo::iridium::xfloat::XFloatMailMessage::set_raw_file_local_path(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_data(),
argo::iridium::xfloat::XFloatMailMessage::set_sbd_data_is_valid(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_file_local_path(),
argo::iridium::xfloat::XFloatMailMessage::set_sbd_file_name(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_file_size_bytes(),
argo::iridium::xfloat::XFloatMailMessage::set_sbd_session_status_code(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_session_sta
argo::iridium::xfloat::XFloat::set_start_of_mission_datetime(), argo::iridium::xfloat::XFloatMetadata::set_wmo_instrument_type(),
argo::iridium::xfloat::XFloatMetadata::set_wmo_recorder_type(), argo::iridium::xfloat::XFloat::set_xmsg_version_major(),
argo::iridium::xfloat::XFloat::set_xmsg_version_minor(), argo::iridium::xfloat::XFloat::start_of_mission_datetime(),
util::thread::RecursiveLockable::UnlockObject(), argo::iridium::xfloat::XFloatDiveData::UpdateDerivedSurfaceDataFields(),
argo::iridium::xfloat::XFloatMetadata::wmo_instrument_type(), argo::iridium::xfloat::XFloatMetadata::wmo_recorder_type(),
argo::iridium::xfloat::XFloatDiveData::WriteListingFile(), argo::iridium::xfloat::XFloatDiveData::WritePhyFile(), argo::iridium::xfloat::XFloatM
argo::iridium::xfloat::XFloat::xmsg_version_major(), and argo::iridium::xfloat::XFloat::xmsg_version_minor().

The documentation for this class was generated from the following file:

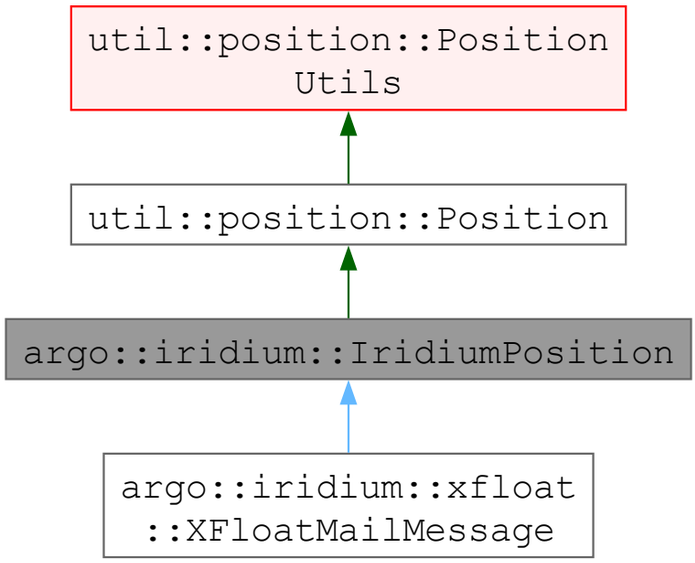
- data/Iridium.h

10.25 argo::iridium::IridiumPosition Class Reference

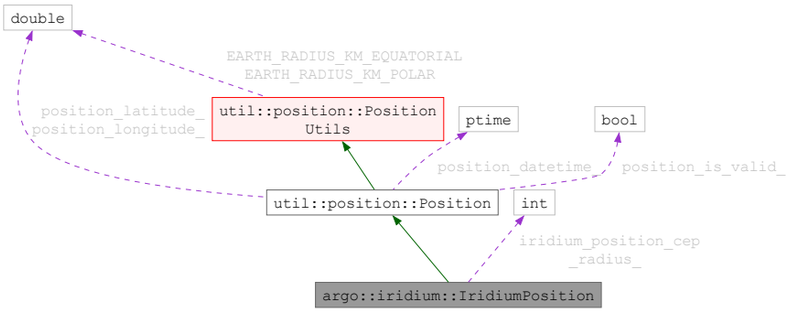
The Iridium position class.

`#include <IridiumPosition.h>`

Inheritance diagram for argo::iridium::IridiumPosition:



Collaboration diagram for argo::iridium::IridiumPosition:



Public Member Functions

- IridiumPosition ()

  The default constructor.
- ~IridiumPosition ()

  The destructor.
- string iridium_position_as_iso6709_string () const

  Get a formatted string representing the Iridium position composed using the ISO 6709 standard.
- int iridium_position_cep_radius () const

  Get the Iridium position Circular Error Probable (CEP) radius, in kilometers.
- ptime iridium_position_datetime () const

  Get the Iridium position date/time.
- string iridium_position_datetime_as_iso8601_string () const

  Get a formatted string representing the Iridium position date/time composed using the ISO 8601 standard.
- bool iridium_position_is_valid () const

  Get the Iridium position is valid flag.
- double iridium_position_latitude () const

  Get the Iridium position latitude.
- double iridium_position_longitude () const

  Get the Iridium position longitude.

Protected Member Functions

- ptime position_datetime () const

  Get the date/time of the position.
- string position_datetime_as_iso8601_string () const

  Get a formatted string representing the position date/time composed using the ISO 8601 standard.
- bool position_is_valid () const

  Is the position valid?
- double position_latitude () const

  Get the latitude of the position, in decimal degrees.
- double position_longitude () const

  Get the longitude of the position, in decimal degrees.
- string postition_as_iso6709_string () const

  Get a formatted string representing the position composed using the ISO 6709 standard.
- void set_gps_position_datetime (unsigned int gpsWeek, unsigned int gpsDay, unsigned int gpsHour=0, unsigned int gpsMinute=0, unsigned int gpsSecond=0)

  Set the date and time of this position from the given GPS week, day, and time.
- void set_iridium_position_cep_radius (int iridium_position_cep_radius)

  Set the Iridium position Circular Error Probable (CEP) radius, in kilometers.
- void set_iridium_position_datetime (const ptime &iridium_position_datetime)

  Set the Iridium position date/time.
- void set_iridium_position_is_valid (bool iridium_position_is_valid)

  Set the Iridium position is valid flag.
- void set_iridium_position_latitude (double iridium_position_latitude)

  Set the Iridium position latitude, in decimal degrees.
- void set_iridium_position_longitude (double iridium_position_longitude)

  Set the Iridium position longitude, in decimal degrees.
- void set_position_datetime (const ptime &datetime)

  Set the date/time of the position.
- void set_position_is_valid (bool is_valid=true)

  Set the position is valid flag.
- void set_position_latitude (double lat)

  Set the latitude of the position, in decimal degrees.
- void set_position_longitude (double lon)

  Set the longitude of the position, in decimal degrees.

Static Protected Member Functions

- static float **bitcastUint32ToFloat** (uint32_t u)
- template<typename T >
  static double CalculateEllipsoidHeight (T lat)

  Get the approximate surface altitude or height of earth for a given latitude, in kilometers.
- template<typename T >
  static double CalculateGeographicDistance (T lat1, T lon1, T lat0, T lon0)

  Get the geographical distance between two position, in kilometers, from the positions' locations, in decimal degrees.
- template<typename T >
  static double degToRad (T deg)

  Convert angular units from from degrees to decimal radians.
- static string GenerateCurrentUtcTimeIso8601String ()

  Generate an ISO8601 format date/time string representation of the current time, in UTC.
- template<typename T >
  static std::string GenerateIso6709String (T lat, T lon)

  Generate an IS06709 string representation of the position given by (`lat`, `lon`).
- static string GenerateIso8601String (const ptime &p, bool extended=true)

  Generate an ISO8601 format date/time string representation of a POSIX time object.
- static double GenerateJulianDay (const ptime &dt, const ptime &ref_dt)

  Convert a date/time to fractional days since a reference date/time.
- static ptime GetCurrentUtcTime ()

  Generate a POSIX time object representing the current date and time (rounded to the nearest second).
- template<typename T >
  static bool IsPositionValid (T lat, T lon)

  Check position validity.

Private Attributes

- int iridium_position_cep_radius_

  Iridium position Circular Error Probable (CEP) radius, in kilometers.
- ptime position_datetime_

  The date and time of the position.
- bool position_is_valid_

  The valid position indicator.
- double position_latitude_

  The latitude of the position, in decimal degrees.
- double position_longitude_

  The longitude of the position, in decimal degrees.

Static Private Attributes

- static const double EARTH_RADIUS_KM_EQUATORIAL = 6378.1

  The equatorial radius of the earth, in kilometers.
- static const double EARTH_RADIUS_KM_POLAR = 6356.1

  The polar radius of the earth, in kilometers.

10.25.1 Detailed Description

The Iridium position class.

This class represents a position that was acquired through the Iridium system. It represents an estimate of the position of the instrument and is provided in the body of an email message received from the Iridium system along the an Iridium Short Burst Data (SBD) message attachment.

10.25.2 Constructor & Destructor Documentation

10.25.2.1 IridiumPosition()

```
argo::iridium::IridiumPosition::IridiumPosition ()  [inline]
```

The default constructor.

10.25.2.2 ~IridiumPosition()

```
argo::iridium::IridiumPosition::~IridiumPosition ()  [inline]
```

The destructor.

10.25.3 Member Function Documentation

10.25.3.1 CalculateEllipsoidHeight()

```
template<typename T >
static double util::position::PositionUtils::CalculateEllipsoidHeight (
            T lat)  [inline], [static], [inherited]
```

Get the approximate surface altitude or height of earth for a given latitude, in kilometers.

Warning

This method provides approximate surface height estimates only! This approximation is calculated from a reference ellipsoid – a mathematically-defined surface that approximates the geoid, the truer figure of the Earth.

Parameters

in	*lat*	The geographic latitude of interest, in decimal degrees.

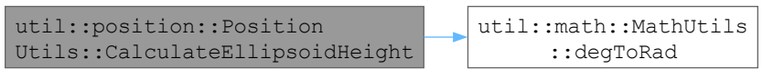
Returns

The approximate surface altitude or height of earth for the given latitude, in kilometers.

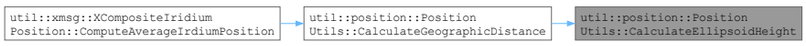
References util::math::MathUtils::degToRad(), util::position::PositionUtils::EARTH_RADIUS_KM_EQUATORIAL, and util::position::PositionUtils::EARTH_RADIUS_KM_POLAR.

Referenced by util::position::PositionUtils::CalculateGeographicDistance().

Here is the call graph for this function:



Here is the caller graph for this function:



10.25.3.2 CalculateGeographicDistance()

```
template<typename T >
static double util::position::PositionUtils::CalculateGeographicDistance (
            T lat1,
            T lon1,
            T lat0,
            T lon0)  [inline], [static], [inherited]
```

Get the geographical distance between two position, in kilometers, from the positions' locations, in decimal degrees.

Warning

This method provides approximate geographic estimates only! This approximation is calculated from a reference ellipsoid – a mathematically-defined surface that approximates the geoid, the truer figure of the Earth.

Note

This method uses the 'haversine' formula to calculate the great-circle distance between two points – that is, the shortest distance over the earth's surface – giving an 'as-the-crow-flies' distance between the points (ignoring any hills they fly over, of course!).

Parameters

in	*lat1*	The latitude of the geographic final position, in decimal degrees North.
in	*lon1*	The longitude of the geographic final position, in decimal degrees East.
in	*lat0*	The latitude of the geographic initial position, in decimal degrees North.
in	*lon0*	The longitude of the geographic initial position, in decimal degrees East.

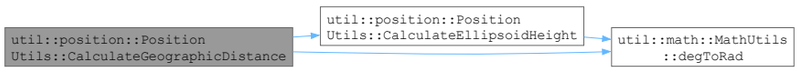
Returns

The geographical distance between the two given positions, in kilometers.

References util::position::PositionUtils::CalculateEllipsoidHeight(), and util::math::MathUtils::degToRad().

Referenced by util::xmsg::XCompositeIridiumPosition::ComputeAverageIrdiumPosition().

Here is the call graph for this function:



Here is the caller graph for this function:

10.25.3.3 degToRad()

```
template<typename T >
static double util::math::MathUtils::degToRad (
            T deg)  [inline], [static], [inherited]
```

Convert angular units from from degrees to decimal radians.

Parameters

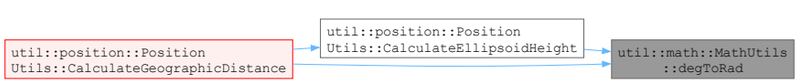
in	*deg*	Angle in integer or decimal degrees to convert to radians.

Returns

Angle in decimal radians.

Referenced by util::position::PositionUtils::CalculateEllipsoidHeight(), and util::position::PositionUtils::CalculateGeographicDistance().

Here is the caller graph for this function:



10.25.3.4 GenerateCurrentUtcTimeIso8601String()

```
static string util::date::DateUtils::GenerateCurrentUtcTimeIso8601String ()  [inline], [static],
[inherited]
```

Generate an ISO8601 format date/time string representation of the current time, in UTC.

Note

All date/time objects use the UTC time-zone.

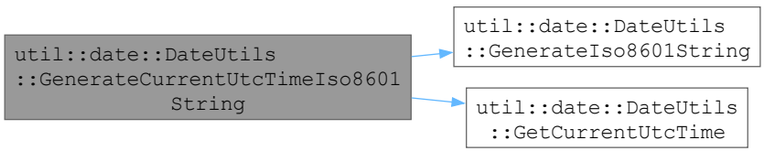
Returns

An ISO8601 format date/time string representation of the current time.

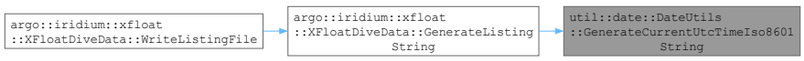
References util::date::DateUtils::GenerateIso8601String(), and util::date::DateUtils::GetCurrentUtcTime().

Referenced by argo::iridium::xfloat::XFloatDiveData::GenerateListingString().

Here is the call graph for this function:



Here is the caller graph for this function:



10.25.3.5 GenerateIso6709String()

```
template<typename T >
static std::string util::position::PositionUtils::GenerateIso6709String (
            T lat,
            T lon)  [inline], [static], [inherited]
```

Generate an IS06709 string representation of the position given by (`lat`, `lon`).

Parameters

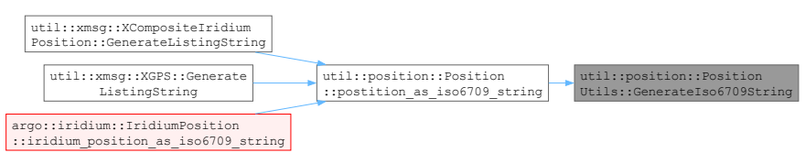
in	*lat*	The latitude of the position, in degrees North.
in	*lon*	The longitude of the position, in degrees East.

Returns

An IS06709 string representation of the position given by (`lat`, `lon`).

Referenced by util::position::Position::postition_as_iso6709_string().

Here is the caller graph for this function:



10.25.3.6 GenerateIso8601String()

```
static string util::date::DateUtils::GenerateIso8601String (
            const ptime & p,
            bool extended = true)  [inline], [static], [inherited]
```

Generate an ISO8601 format date/time string representation of a POSIX time object.

Note

All date/time objects use the UTC time-zone.

Parameters

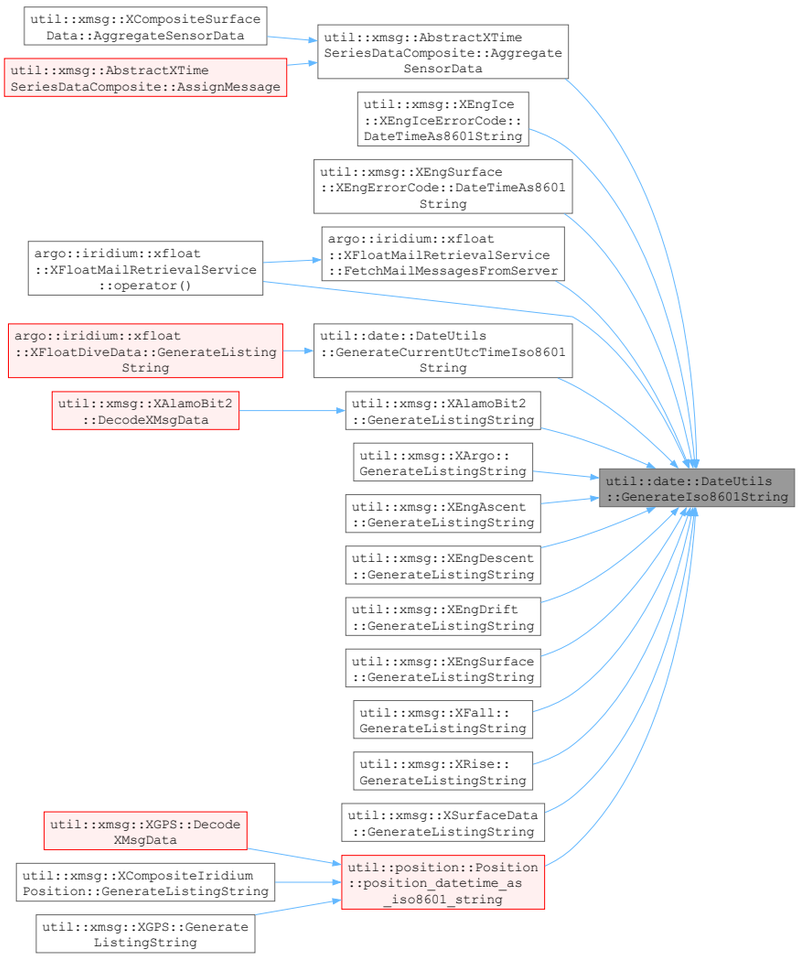
in	*p*	POSIX time from which to generate string.
in	*extended*	If TRUE, use the ISO 8601 extended format (YYYY-MM-DDThh:mm:ssZ); otherwise, use the basic format (YYYYMMDDThhmmss).

Returns

An ISO8601 format date/time string representation of the POSIX time object. If p is equal to boost::posix_↩ time::not_a_date_time, then the string "0000-00-00T00:00:00Z" (extended) or "00000000000000" (basic) will be returned.

Referenced by util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::AggregateSensorData(), util::xmsg::XEngIce::XEngIce util::xmsg::XEngSurface::XEngErrorCode::DateTimeAs8601String(), argo::iridium::xfloat::XFloatMailRetrievalService::FetchMailMessages util::date::DateUtils::GenerateCurrentUtcTimeIso8601String(), util::xmsg::XAlamoBit2::GenerateListingString(), util::xmsg::XArgo::Genera util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListing util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(), argo::iridium::xfloat::XFloatMailRetrievalService::operator()(), and util::position::Position::position_datetime_as_iso8601_string().

Here is the caller graph for this function:



10.25.3.7 GenerateJulianDay()

```
static double util::date::DateUtils::GenerateJulianDay (
            const ptime & dt,
            const ptime & ref_dt)  [inline], [static], [inherited]
```

Convert a date/time to fractional days since a reference date/time.

Parameters

in	*dt*	The date/time to convert.
in	*ref_dt*	The reference date/time (day 0).

Exceptions

util::log::MessageException	if either date/time is not valid.

Returns

The number of days (with millisecond resolution) from `ref_dt` to `dt`.

References CATCH_BLOCK, and EXC_THROW.

10.25.3.8 GetCurrentUtcTime()

```
static ptime util::date::DateUtils::GetCurrentUtcTime ()  [inline], [static], [inherited]
```

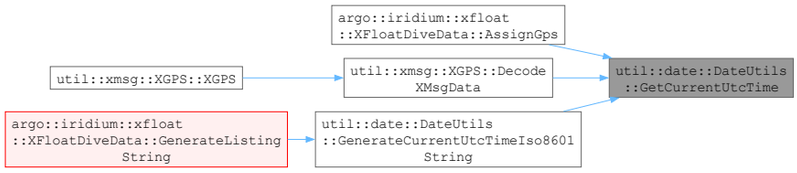
Generate a POSIX time object representing the current date and time (rounded to the nearest second).

Returns

A POSIX time object representing the current date and time (rounded to the nearest second).

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignGps(), util::xmsg::XGPS::DecodeXMsgData(), and util::date::DateUtils::GenerateCurrentUtcTimeIso8601String().

Here is the caller graph for this function:



10.25.3.9 iridium_position_as_iso6709_string()

```
string argo::iridium::IridiumPosition::iridium_position_as_iso6709_string () const  [inline]
```

Get a formatted string representing the Iridium position composed using the ISO 6709 standard.

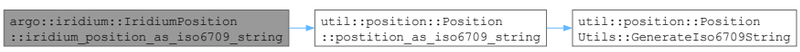
Returns

The formatted string representation of the Iridium position.

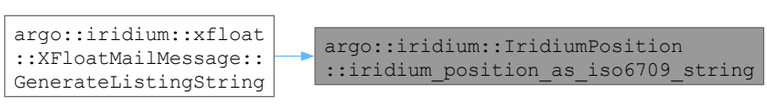
References util::position::Position::postition_as_iso6709_string().

Referenced by argo::iridium::xfloat::XFloatMailMessage::GenerateListingString().

Here is the call graph for this function:



Here is the caller graph for this function:



10.25.3.10 iridium_position_cep_radius()

```
int argo::iridium::IridiumPosition::iridium_position_cep_radius () const  [inline]
```

Get the Iridium position Circular Error Probable (CEP) radius, in kilometers.

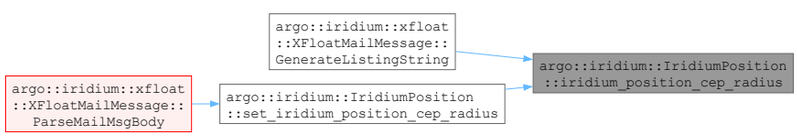
Returns

The Iridium position Circular Error Probable (CEP) radius, in kilometers

References iridium_position_cep_radius_.

Referenced by argo::iridium::xfloat::XFloatMailMessage::GenerateListingString(), and set_iridium_position_cep_radius().

Here is the caller graph for this function:



10.25.3.11 iridium_position_datetime()

```
ptime argo::iridium::IridiumPosition::iridium_position_datetime () const  [inline]
```

Get the Iridium position date/time.

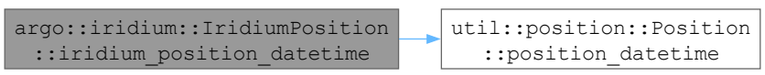
Returns

The Iridium position date/time.

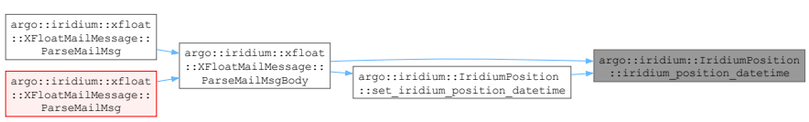
References util::position::Position::position_datetime().

Referenced by argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgBody(), and set_iridium_position_datetime().

Here is the call graph for this function:



Here is the caller graph for this function:



10.25.3.12 iridium_position_datetime_as_iso8601_string()

```
string argo::iridium::IridiumPosition::iridium_position_datetime_as_iso8601_string () const  [inline]
```

Get a formatted string representing the Iridium position date/time composed using the ISO 8601 standard.

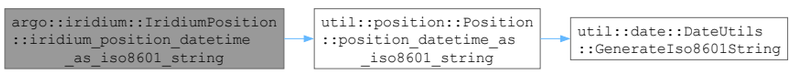
Returns

The formatted string representation of the Iridium position date/time.

References util::position::Position::position_datetime_as_iso8601_string().

Referenced by argo::iridium::xfloat::XFloatMailMessage::GenerateListingString().

Here is the call graph for this function:



Here is the caller graph for this function:

10.25.3.13 iridium_position_is_valid()

```
bool argo::iridium::IridiumPosition::iridium_position_is_valid () const  [inline]
```

Get the Iridium position is valid flag.

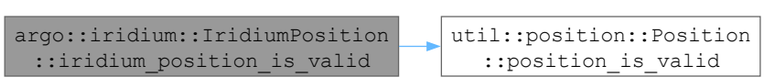
Returns

TRUE if the Iridium position is valid, or FALSE otherwise.

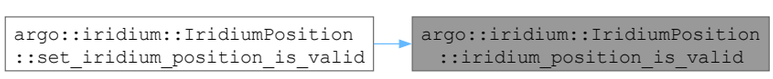
References util::position::Position::position_is_valid().

Referenced by set_iridium_position_is_valid().

Here is the call graph for this function:



Here is the caller graph for this function:



10.25.3.14 iridium_position_latitude()

```
double argo::iridium::IridiumPosition::iridium_position_latitude () const  [inline]
```

Get the Iridium position latitude.

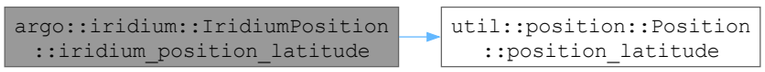
Returns

The Iridium position latitude.

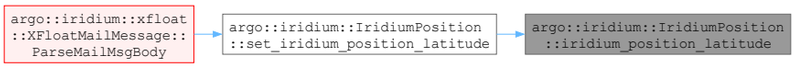
References util::position::Position::position_latitude().

Referenced by set_iridium_position_latitude().

Here is the call graph for this function:



Here is the caller graph for this function:



10.25.3.15 iridium_position_longitude()

```
double argo::iridium::IridiumPosition::iridium_position_longitude () const  [inline]
```

Get the Iridium position longitude.

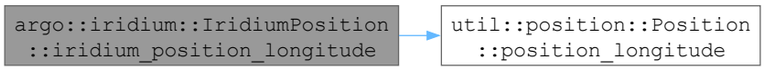
Returns

The Iridium position longitude.

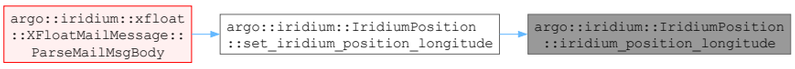
References util::position::Position::position_longitude().

Referenced by set_iridium_position_longitude().

Here is the call graph for this function:



Here is the caller graph for this function:



10.25.3.16 IsPositionValid()

```
template<typename T >
static bool util::position::PositionUtils::IsPositionValid (
            T lat,
            T lon)  [inline], [static], [inherited]
```

Check position validity.

Parameters

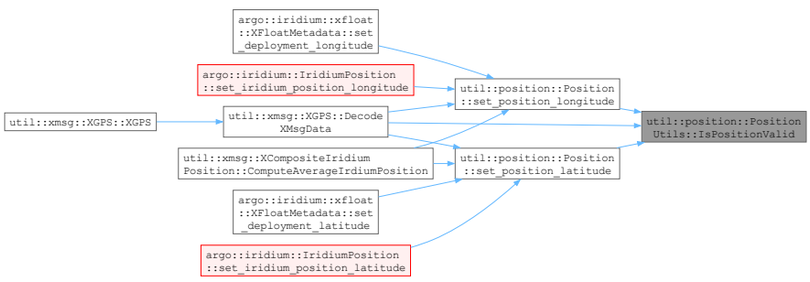
in	*lat*	Latitude of position.
in	*lon*	Longitude of position.

Returns

TRUE if (lat, lon) represents a valid geographical position, or FALSE otherwise.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), util::position::Position::set_position_latitude(), and util::position::Position::set_positi

Here is the caller graph for this function:



10.25.3.17 position_datetime()

```
ptime util::position::Position::position_datetime () const  [inline], [inherited]
```

Get the date/time of the position.

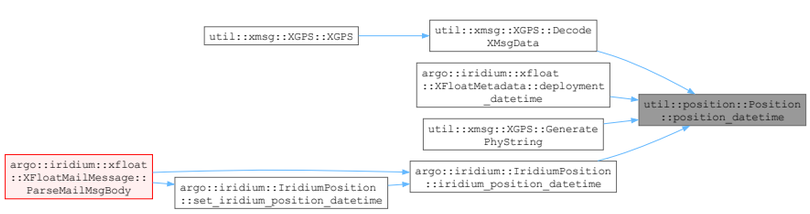
Returns

The date/time of the position.

References util::position::Position::position_datetime_.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), argo::iridium::xfloat::XFloatMetadata::deployment_datetime(), util::xmsg::XGPS::GeneratePhyString(), and iridium_position_datetime().

Here is the caller graph for this function:



10.25.3.18 position_datetime_as_iso8601_string()

```
string util::position::Position::position_datetime_as_iso8601_string () const  [inline], [inherited]
```

Get a formatted string representing the position date/time composed using the ISO 8601 standard.

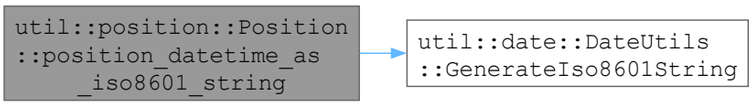
Returns

The formatted string representation of the position date/time.

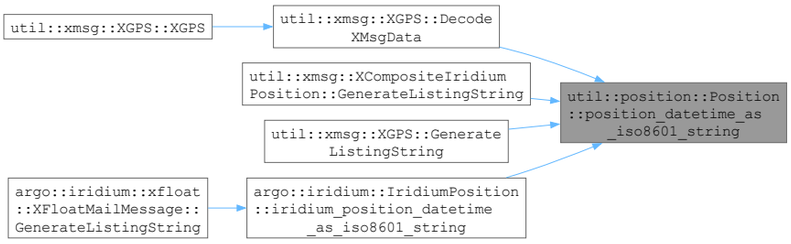
References util::date::DateUtils::GenerateIso8601String(), and util::position::Position::position_datetime_.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XCompositeIridiumPosition::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), and iridium_position_datetime_as_iso8601_string().

Here is the call graph for this function:



Here is the caller graph for this function:



10.25.3.19 position_is_valid()

```
bool util::position::Position::position_is_valid () const  [inline], [inherited]
```

Is the position valid?

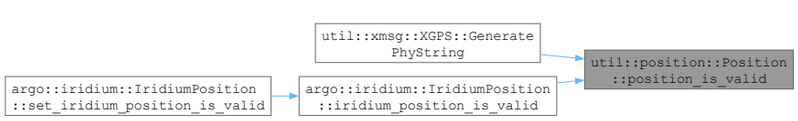
Returns

TRUE if the position is valid, or false otherwise.

References util::position::Position::position_is_valid_.

Referenced by util::xmsg::XGPS::GeneratePhyString(), and iridium_position_is_valid().

Here is the caller graph for this function:



10.25.3.20 position_latitude()

```
double util::position::Position::position_latitude () const  [inline], [inherited]
```

Get the latitude of the position, in decimal degrees.

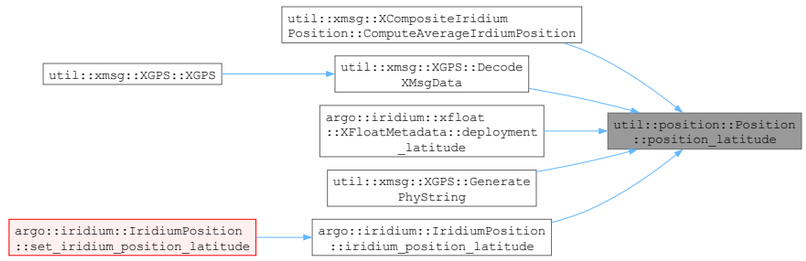
Returns

The latitude of the position, in decimal degrees.

References util::position::Position::position_latitude_.

Referenced by util::xmsg::XCompositeIridiumPosition::ComputeAverageIrdiumPosition(), util::xmsg::XGPS::DecodeXMsgData(), argo::iridium::xfloat::XFloatMetadata::deployment_latitude(), util::xmsg::XGPS::GeneratePhyString(), and iridium_position_latitude().

Here is the caller graph for this function:



10.25.3.21 position_longitude()

```
double util::position::Position::position_longitude () const  [inline], [inherited]
```

Get the longitude of the position, in decimal degrees.

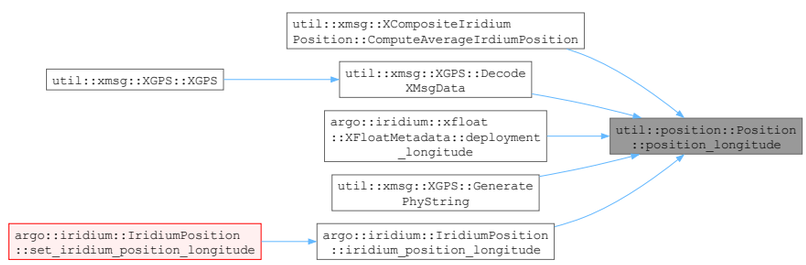
Returns

The longitude of the position, in decimal degrees.

References util::position::Position::position_longitude_.

Referenced by util::xmsg::XCompositeIridiumPosition::ComputeAverageIrdiumPosition(), util::xmsg::XGPS::DecodeXMsgData(), argo::iridium::xfloat::XFloatMetadata::deployment_longitude(), util::xmsg::XGPS::GeneratePhyString(), and iridium_position_longitude().

Here is the caller graph for this function:



10.25.3.22 postition_as_iso6709_string()

```
string util::position::Position::postition_as_iso6709_string () const  [inline], [inherited]
```

Get a formatted string representing the position composed using the ISO 6709 standard.

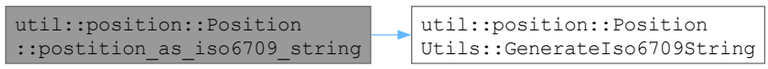
Returns

The formatted string representation of the position.

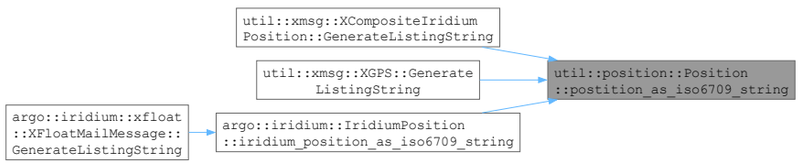
References util::position::PositionUtils::GenerateIso6709String(), util::position::Position::position_latitude_, and util::position::Position::position_longitude_.

Referenced by util::xmsg::XCompositeIridiumPosition::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), and iridium_position_as_iso6709_string().

Here is the call graph for this function:



Here is the caller graph for this function:



10.25.3.23 set_gps_position_datetime()

```
void util::position::Position::set_gps_position_datetime (
            unsigned int gpsWeek,
            unsigned int gpsDay,
            unsigned int gpsHour = 0,
            unsigned int gpsMinute = 0,
            unsigned int gpsSecond = 0)  [inline], [protected], [inherited]
```

Set the date and time of this position from the given GPS week, day, and time.

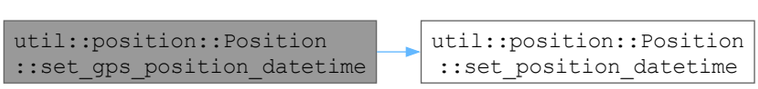
Parameters

in	*gpsWeek*	The GPS week, in weeks since midnight of January 6, 1980.
in	*gpsDay*	The GPS day of week (0 - 6).
in	*gpsHour*	The UTC hour of day (0 - 23).
in	*gpsMinute*	The UTC minute of hour (0 - 59)..
in	*gpsSecond*	The UTC second of minute (0 - 59).

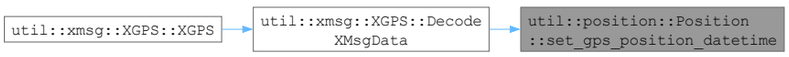
References util::position::Position::set_position_datetime().

Referenced by util::xmsg::XGPS::DecodeXMsgData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.25.3.24 set_iridium_position_cep_radius()

```
void argo::iridium::IridiumPosition::set_iridium_position_cep_radius (
            int iridium_position_cep_radius)  [inline], [protected]
```

Set the Iridium position Circular Error Probable (CEP) radius, in kilometers.

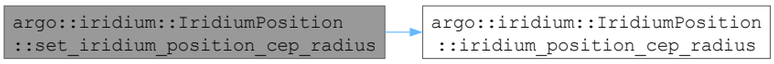
Parameters

in	*iridium_position_cep_radius*	The Iridium position Circular Error Probable (CEP) radius to set, in kilometers.

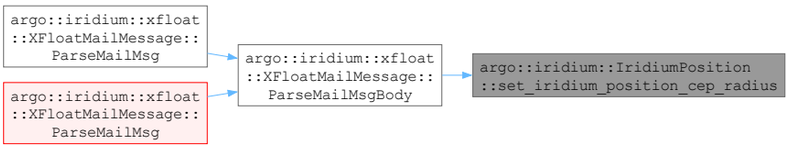
References iridium_position_cep_radius(), and iridium_position_cep_radius_.

Referenced by argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgBody().

Here is the call graph for this function:



Here is the caller graph for this function:



10.25.3.25 set_iridium_position_datetime()

```
void argo::iridium::IridiumPosition::set_iridium_position_datetime (
            const ptime & iridium_position_datetime)  [inline], [protected]
```

Set the Iridium position date/time.

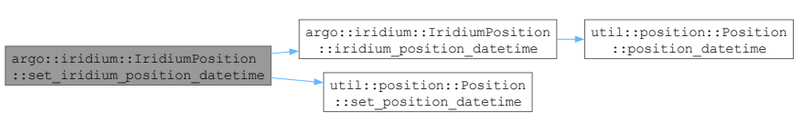
Parameters

in	*iridium_position_datetime*	The Iridium position date/time to set.

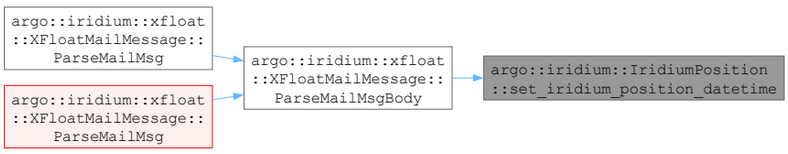
References iridium_position_datetime(), and util::position::Position::set_position_datetime().

Referenced by argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgBody().

Here is the call graph for this function:



Here is the caller graph for this function:



10.25.3.26 set_iridium_position_is_valid()

```
void argo::iridium::IridiumPosition::set_iridium_position_is_valid (
            bool iridium_position_is_valid)  [inline], [protected]
```

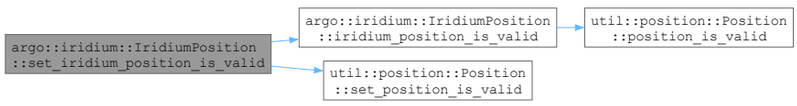
Set the Iridium position is valid flag.

Parameters

in	*iridium_position_is_valid*	The Iridium position is valid flag to set.

References iridium_position_is_valid(), and util::position::Position::set_position_is_valid().

Here is the call graph for this function:



10.25.3.27 set_iridium_position_latitude()

```
void argo::iridium::IridiumPosition::set_iridium_position_latitude (
            double iridium_position_latitude)  [inline], [protected]
```

Set the Iridium position latitude, in decimal degrees.

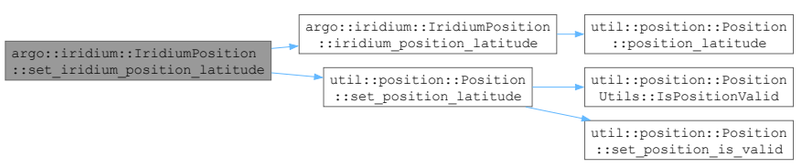
Parameters

in	*iridium_position_latitude*	The Iridium position latitude to set, in decimal degrees.

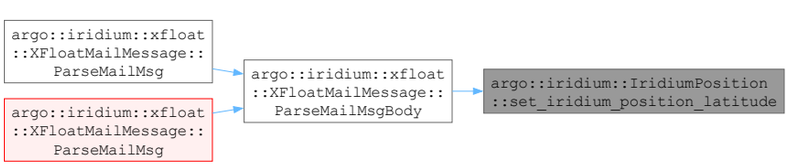
References iridium_position_latitude(), and util::position::Position::set_position_latitude().

Referenced by argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgBody().

Here is the call graph for this function:



Here is the caller graph for this function:



10.25.3.28 set_iridium_position_longitude()

```
void argo::iridium::IridiumPosition::set_iridium_position_longitude (
            double iridium_position_longitude)  [inline], [protected]
```

Set the Iridium position longitude, in decimal degrees.

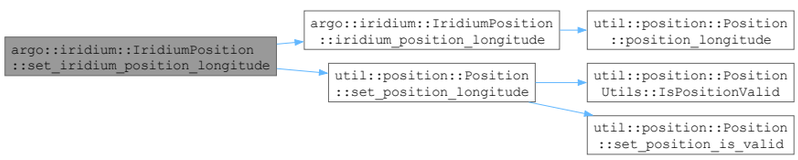
Parameters

in	*iridium_position_longitude*	The Iridium position longitude to set, in decimal degrees.

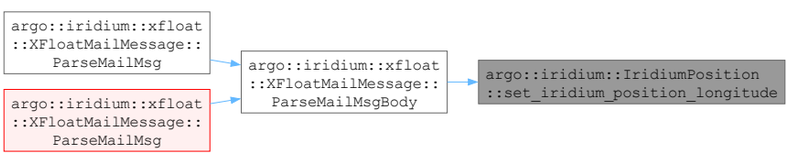
References iridium_position_longitude(), and util::position::Position::set_position_longitude().

Referenced by argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgBody().

Here is the call graph for this function:



Here is the caller graph for this function:



10.25.3.29 set_position_datetime()

```
void util::position::Position::set_position_datetime (
            const ptime & datetime)  [inline], [protected], [inherited]
```

Set the date/time of the position.

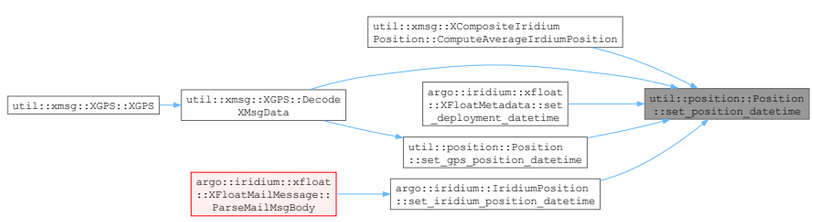
Parameters

in	*datetime*	The date/time of the position to set.

References util::position::Position::position_datetime_.

Referenced by util::xmsg::XCompositeIridiumPosition::ComputeAverageIrdiumPosition(), util::xmsg::XGPS::DecodeXMsgData(), argo::iridium::xfloat::XFloatMetadata::set_deployment_datetime(), util::position::Position::set_gps_position_datetime(), and set_iridium_position_datetime().

Here is the caller graph for this function:



10.25.3.30 set_position_is_valid()

```
void util::position::Position::set_position_is_valid (
            bool is_valid = true)  [inline], [protected], [inherited]
```

Set the position is valid flag.

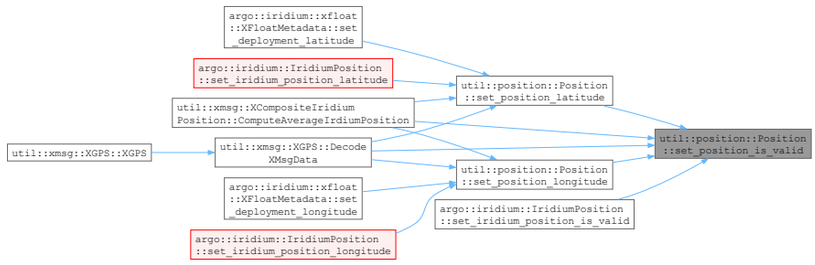
Parameters

in	*is_valid*	TRUE if the position is valid, or FALSE otherwise.

References util::position::Position::position_is_valid_.

Referenced by util::xmsg::XCompositeIridiumPosition::ComputeAverageIrdiumPosition(), util::xmsg::XGPS::DecodeXMsgData(), set_iridium_position_is_valid(), util::position::Position::set_position_latitude(), and util::position::Position::set_position_longitude().

Here is the caller graph for this function:



10.25.3.31 set_position_latitude()

```
void util::position::Position::set_position_latitude (
            double lat)  [inline], [protected], [inherited]
```

Set the latitude of the position, in decimal degrees.

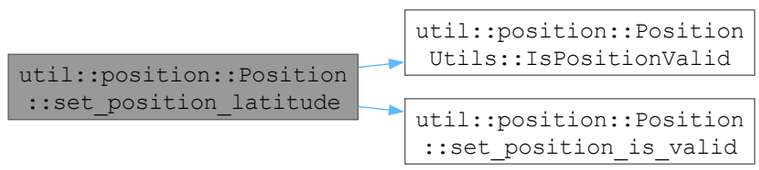
Parameters

in	*lat*	The latitude of the position, in decimal degrees.

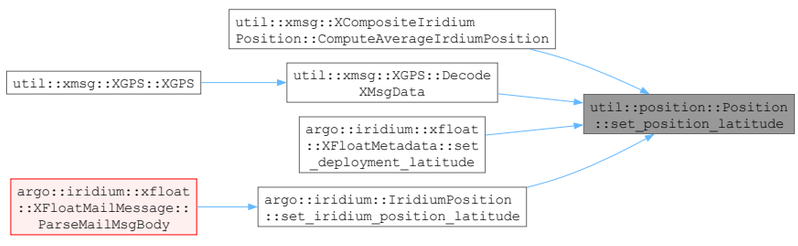
References util::position::PositionUtils::IsPositionValid(), util::position::Position::position_latitude_, util::position::Position::position_longitud and util::position::Position::set_position_is_valid().

Referenced by util::xmsg::XCompositeIridiumPosition::ComputeAverageIrdiumPosition(), util::xmsg::XGPS::DecodeXMsgData(), argo::iridium::xfloat::XFloatMetadata::set_deployment_latitude(), and set_iridium_position_latitude().

Here is the call graph for this function:



Here is the caller graph for this function:



10.25.3.32 set_position_longitude()

```
void util::position::Position::set_position_longitude (
            double lon)  [inline], [protected], [inherited]
```

Set the longitude of the position, in decimal degrees.

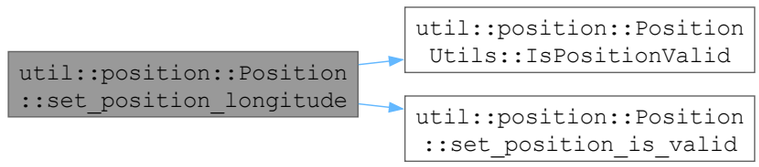
Parameters

in	*lon*	The longitude of the position, in decimal degrees.

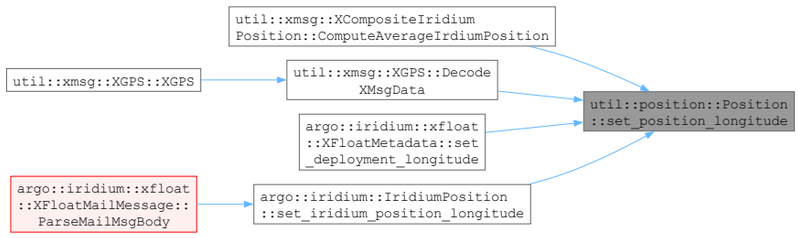
References util::position::PositionUtils::IsPositionValid(), util::position::Position::position_latitude_, util::position::Position::position_longitud and util::position::Position::set_position_is_valid().

Referenced by util::xmsg::XCompositeIridiumPosition::ComputeAverageIrdiumPosition(), util::xmsg::XGPS::DecodeXMsgData(), argo::iridium::xfloat::XFloatMetadata::set_deployment_longitude(), and set_iridium_position_longitude().

Here is the call graph for this function:



Here is the caller graph for this function:



10.25.4 Member Data Documentation

10.25.4.1 EARTH_RADIUS_KM_EQUATORIAL

```
const double util::position::PositionUtils::EARTH_RADIUS_KM_EQUATORIAL = 6378.1  [static], [private],
[inherited]
```

The equatorial radius of the earth, in kilometers.

Referenced by util::position::PositionUtils::CalculateEllipsoidHeight().

10.25.4.2 EARTH_RADIUS_KM_POLAR

```
const double util::position::PositionUtils::EARTH_RADIUS_KM_POLAR = 6356.1  [static], [private],
[inherited]
```

The polar radius of the earth, in kilometers.

Referenced by util::position::PositionUtils::CalculateEllipsoidHeight().

10.25.4.3 iridium_position_cep_radius_

```
int argo::iridium::IridiumPosition::iridium_position_cep_radius_  [private]
```

Iridium position Circular Error Probable (CEP) radius, in kilometers.

This field defines the estimate of an error of the Iridium position. It is provided by the Iridium system, along with the Iridium position, in the body of the email containing the Iridium SBD message.

Since multiple Iridium messages are transmitted for each dive cycle, a better estimate of the instrument position is provided by a composite position, constructed from two or more individual positions. To compute this composite position, each position is weighted proportionally to the inverse of the CEP radius.

Referenced by iridium_position_cep_radius(), and set_iridium_position_cep_radius().

10.25.4.4 position_datetime_

```
ptime util::position::Position::position_datetime_  [private], [inherited]
```

The date and time of the position.

Referenced by util::position::Position::position_datetime(), util::position::Position::position_datetime_as_iso8601_string(), and util::position::Position::set_position_datetime().

10.25.4.5 position_is_valid_

```
bool util::position::Position::position_is_valid_  [private], [inherited]
```

The valid position indicator.

Referenced by util::position::Position::position_is_valid(), and util::position::Position::set_position_is_valid().

10.25.4.6 position_latitude_

```
double util::position::Position::position_latitude_  [private], [inherited]
```

The latitude of the position, in decimal degrees.

Referenced by util::position::Position::position_latitude(), util::position::Position::postition_as_iso6709_string(), util::position::Position::set_position_latitude(), and util::position::Position::set_position_longitude().

10.25.4.7 position_longitude_

```
double util::position::Position::position_longitude_  [private], [inherited]
```

The longitude of the position, in decimal degrees.

Referenced by util::position::Position::position_longitude(), util::position::Position::postition_as_iso6709_string(), util::position::Position::set_position_latitude(), and util::position::Position::set_position_longitude().

The documentation for this class was generated from the following file:

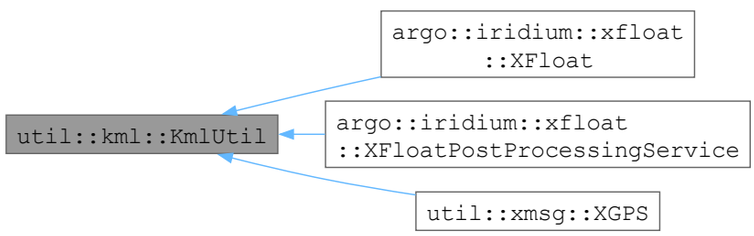
- data/IridiumPosition.h

10.26 util::kml::KmlUtil Class Reference

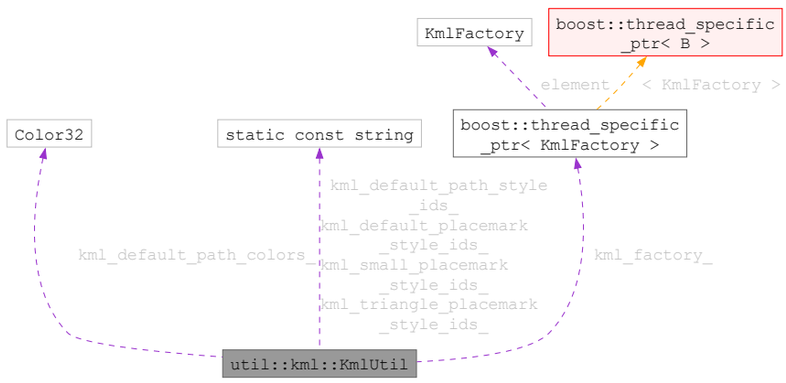
The KML utility class.

```
#include <KmlUtil.h>
```

Inheritance diagram for util::kml::KmlUtil:



Collaboration diagram for util::kml::KmlUtil:



Static Public Member Functions

- static void CloseKmlFactory ()

  Release this thread's KML factory pointer.
- static const Color32 & GetDefaltKmlPathColor (int id)

  Get the palette colour for an index (an instrument ID, for example), modulo the palette size.
- static const string & GetDefaltKmlPathStyleId (int id)

  Get the path (line) style ID for an index, modulo the palette size.
- static const string & GetDefaltKmlPlacemarkStyleId (int id)

  Get the large placemark style ID for an index, modulo the palette size.
- static thread_specific_ptr< KmlFactory > & GetKmlFactory ()

  Get the libkml factory, initializing it for this thread if necessary.
- static const string & GetSmallKmlPlacemarkStyleId (int id)

  Get the small placemark style ID for an index, modulo the palette size.
- static const string & GetTriangleKmlPlacemarkStyleId (int id)

  Get the triangle placemark style ID for an index, modulo the palette size.
- static void SetAllDefaultKmlSyles (const DocumentPtr &document)

  Add the default styles for every palette colour to a KML document.
- static void SetDefaultKmlStyle (const DocumentPtr &document, unsigned int style_idx)

  Add the four default styles for one palette colour to a KML document.

Static Private Attributes

- static const Color32 kml_default_path_colors_ [KML_NUM_DFLT_PATH_COLORS]

  The colour palette for instrument paths and placemarks, indexed by style index.
- static const string kml_default_path_style_ids_ [KML_NUM_DFLT_PATH_COLORS]

  The KML style IDs of the path (line) styles, one per palette colour.
- static const string kml_default_placemark_style_ids_ [KML_NUM_DFLT_PATH_COLORS]

  The KML style IDs of the large placemark styles, one per palette colour.
- static thread_specific_ptr< KmlFactory > kml_factory_

  The libkml factory used to create KML elements, held per thread.
- static const string kml_small_placemark_style_ids_ [KML_NUM_DFLT_PATH_COLORS]

  The KML style IDs of the small placemark styles, one per palette colour.
- static const string kml_triangle_placemark_style_ids_ [KML_NUM_DFLT_PATH_COLORS]

  The KML style IDs of the triangle placemark styles, one per palette colour.

10.26.1 Detailed Description

The KML utility class.

This class provides some useful functions for creating Google Earth KML files.

10.26.2 Member Function Documentation

10.26.2.1 CloseKmlFactory()

```
static void util::kml::KmlUtil::CloseKmlFactory ()  [inline], [static]
```

Release this thread's KML factory pointer.

Warning

This deletes libkml's shared factory (see kml_factory_). It is not currently called.

Exceptions

util::log::MessageException	if an error occurs.

References CATCH_BLOCK, and kml_factory_.

10.26.2.2 GetDefaltKmlPathColor()

```
static const Color32 & util::kml::KmlUtil::GetDefaltKmlPathColor (
            int id)  [inline], [static]
```

Get the palette colour for an index (an instrument ID, for example), modulo the palette size.

Parameters

in	*id*	The index.

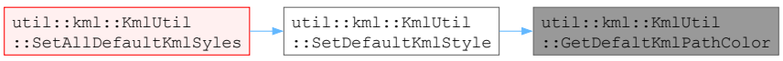
Returns

The colour.

References kml_default_path_colors_.

Referenced by SetDefaultKmlStyle().

Here is the caller graph for this function:



10.26.2.3 GetDefaltKmlPathStyleId()

```
static const string & util::kml::KmlUtil::GetDefaltKmlPathStyleId (
            int id)  [inline], [static]
```

Get the path (line) style ID for an index, modulo the palette size.

Parameters

in	*id*	The index.

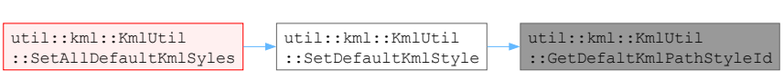
Returns

The KML style ID.

References kml_default_path_style_ids_.

Referenced by SetDefaultKmlStyle().

Here is the caller graph for this function:



10.26.2.4 GetDefaltKmlPlacemarkStyleId()

```
static const string & util::kml::KmlUtil::GetDefaltKmlPlacemarkStyleId (
            int id)  [inline], [static]
```

Get the large placemark style ID for an index, modulo the palette size.

Parameters

in	*id*	The index.

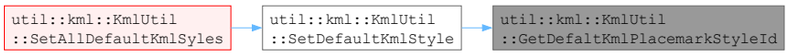
Returns

The KML style ID.

References kml_default_placemark_style_ids_.

Referenced by SetDefaultKmlStyle().

Here is the caller graph for this function:



10.26.2.5 GetKmlFactory()

```
static thread_specific_ptr< KmlFactory > & util::kml::KmlUtil::GetKmlFactory ()  [inline], [static]
```

Get the libkml factory, initializing it for this thread if necessary.

Exceptions

util::log::MessageException	if the factory cannot be obtained.

Returns

The thread's KML factory pointer (see kml_factory_).

References CATCH_BLOCK, and kml_factory_.

Referenced by argo::iridium::xfloat::XFloat::GenerateKmlMasterDocument(), argo::iridium::xfloat::XFloat::GenerateKmlMasterString(), and SetDefaultKmlStyle().

Here is the caller graph for this function:

10.26.2.6 GetSmallKmlPlacemarkStyleId()

```
static const string & util::kml::KmlUtil::GetSmallKmlPlacemarkStyleId (
            int id)  [inline], [static]
```

Get the small placemark style ID for an index, modulo the palette size.

Parameters

in	*id*	The index.

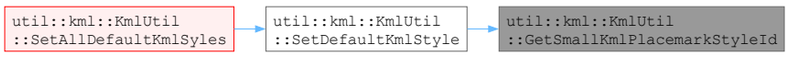
Returns

The KML style ID.

References kml_small_placemark_style_ids_.

Referenced by SetDefaultKmlStyle().

Here is the caller graph for this function:



10.26.2.7 GetTriangleKmlPlacemarkStyleId()

```
static const string & util::kml::KmlUtil::GetTriangleKmlPlacemarkStyleId (
            int id)  [inline], [static]
```

Get the triangle placemark style ID for an index, modulo the palette size.

Parameters

in	*id*	The index.

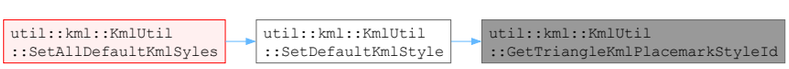
Returns

The KML style ID.

References kml_triangle_placemark_style_ids_.

Referenced by SetDefaultKmlStyle().

Here is the caller graph for this function:



10.26.2.8 SetAllDefaultKmlSyles()

```
static void util::kml::KmlUtil::SetAllDefaultKmlSyles (
            const DocumentPtr & document)  [inline], [static]
```

Add the default styles for every palette colour to a KML document.

Parameters

in	*document*	The KML document to add the styles to.

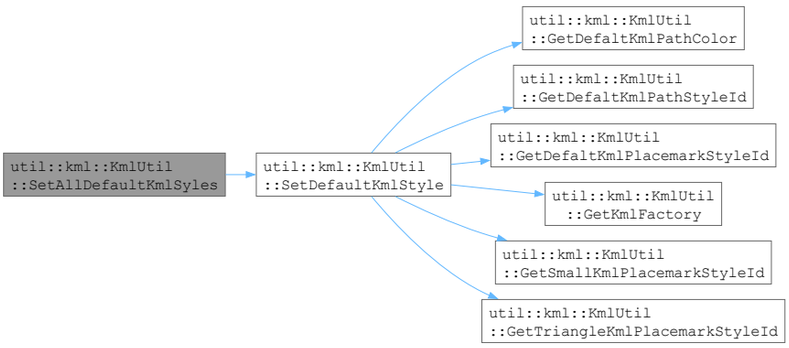
Exceptions

util::log::MessageException	if an error occurs.

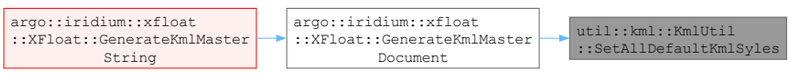
References CATCH_BLOCK, and SetDefaultKmlStyle().

Referenced by argo::iridium::xfloat::XFloat::GenerateKmlMasterDocument().

Here is the call graph for this function:



Here is the caller graph for this function:



10.26.2.9 SetDefaultKmlStyle()

```
static void util::kml::KmlUtil::SetDefaultKmlStyle (
            const DocumentPtr & document,
            unsigned int style_idx)  [inline], [static]
```

Add the four default styles for one palette colour to a KML document.

Adds a 5-pixel path (line) style, a large dot placemark style, a small dot placemark style and a triangle placemark style, all in the colour `style_idx` (modulo the palette size). The icons are loaded from `http://argo.whoi.edu/kml/img/`. Errors are logged, not thrown.

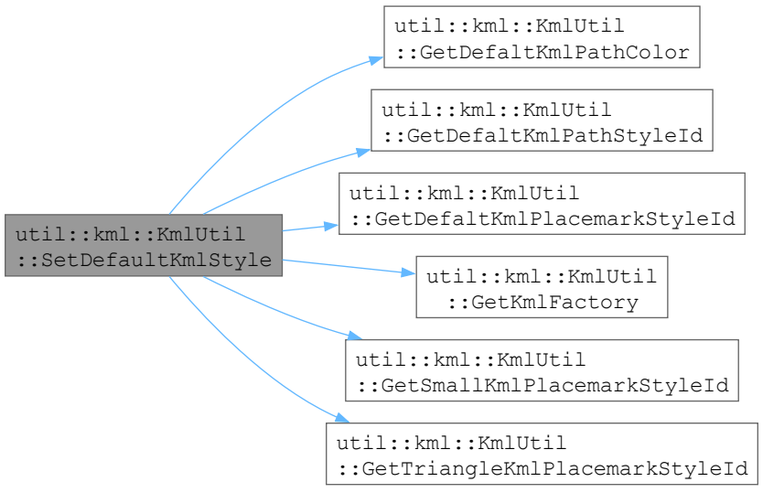
Parameters

in	*document*	The KML document to add the styles to.
in	*style_idx*	The palette index of the colour.

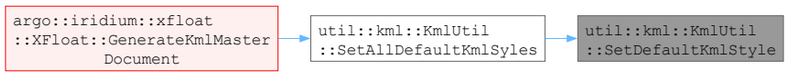
References CATCH_NOTHROW, GetDefaltKmlPathColor(), GetDefaltKmlPathStyleId(), GetDefaltKmlPlacemarkStyleId(), GetKmlFactory(), GetSmallKmlPlacemarkStyleId(), and GetTriangleKmlPlacemarkStyleId().

Referenced by SetAllDefaultKmlSyles().

Here is the call graph for this function:



Here is the caller graph for this function:



10.26.3 Member Data Documentation

10.26.3.1 kml_default_path_colors_

```
const Color32 util::kml::KmlUtil::kml_default_path_colors_  [static], [private]
```

Initial value:

```
= {
        Color32(0xB0, 0xFF, 0x00, 0x00),
        Color32(0xB0, 0xFF, 0x80, 0x00),
        Color32(0xB0, 0xFF, 0xFF, 0x00),
        Color32(0xB0, 0x80, 0xFF, 0x00),
        Color32(0xB0, 0x00, 0xFF, 0x00),
        Color32(0xB0, 0x00, 0xFF, 0x80),
        Color32(0xB0, 0x00, 0xFF, 0xFF),
        Color32(0xB0, 0x00, 0x80, 0xFF),
        Color32(0xB0, 0x00, 0x00, 0xFF),
        Color32(0xB0, 0x80, 0x00, 0xFF),
        Color32(0xB0, 0xFF, 0x00, 0xFF),
        Color32(0xB0, 0xFF, 0x00, 0x80),
        Color32(0xB0, 0x80, 0x80, 0x080),
 }
```

The colour palette for instrument paths and placemarks, indexed by style index.

Referenced by GetDefaltKmlPathColor().

10.26.3.2 kml_default_path_style_ids_

```
const string util::kml::KmlUtil::kml_default_path_style_ids_  [static], [private]
```

Initial value:

```
= {
        "path1",
        "path2",
        "path3",
        "path4",
        "path5",
        "path6",
        "path7",
        "path8",
        "path9",
        "pathA",
        "pathB",
        "pathC",
        "pathD",}
```

The KML style IDs of the path (line) styles, one per palette colour.

Referenced by GetDefaltKmlPathStyleId().

10.26.3.3 kml_default_placemark_style_ids_

```
const string util::kml::KmlUtil::kml_default_placemark_style_ids_  [static], [private]
```

Initial value:

```
= {
        "dot1_1",
        "dot1_2",
        "dot1_3",
        "dot1_4",
        "dot1_5",
        "dot1_6",
        "dot1_7",
        "dot1_8",
        "dot1_9",
        "dot1_A",
        "dot1_B",
        "dot1_C",
        "dot1_D", }
```

The KML style IDs of the large placemark styles, one per palette colour.

Referenced by GetDefaltKmlPlacemarkStyleId().

10.26.3.4 kml_factory_

```
thread_specific_ptr< KmlFactory > util::kml::KmlUtil::kml_factory_  [static], [private]
```

The libkml factory used to create KML elements, held per thread.

Warning

This holds libkml's process-wide factory (`KmlFactory::GetFactory()`), and the default cleanup deletes it when the owning thread exits or CloseKmlFactory() is called. Only the post-processing thread uses KML, so this currently happens only at shutdown.

Referenced by CloseKmlFactory(), and GetKmlFactory().

10.26.3.5 kml_small_placemark_style_ids_

```
const string util::kml::KmlUtil::kml_small_placemark_style_ids_  [static], [private]
```

Initial value:

```
= {
        "dot1s_1",
        "dot1s_2",
        "dot1s_3",
        "dot1s_4",
        "dot1s_5",
        "dot1s_6",
        "dot1s_7",
        "dot1s_8",
        "dot1s_9",
        "dot1s_A",
        "dot1s_B",
        "dot1s_C",
        "dot1s_D", }
```

The KML style IDs of the small placemark styles, one per palette colour.

Referenced by GetSmallKmlPlacemarkStyleId().

10.26.3.6 kml_triangle_placemark_style_ids_

```
const string util::kml::KmlUtil::kml_triangle_placemark_style_ids_  [static], [private]
```

Initial value:

```
= {
        "tri1_1",
        "tri1_2",
        "tri1_3",
        "tri1_4",
        "tri1_5",
        "tri1_6",
        "tri1_7",
        "tri1_8",
        "tri1_9",
        "tri1_A",
        "tri1_B",
        "tri1_C",
        "tri1_D", }
```

The KML style IDs of the triangle placemark styles, one per palette colour.

Referenced by GetTriangleKmlPlacemarkStyleId().

The documentation for this class was generated from the following files:

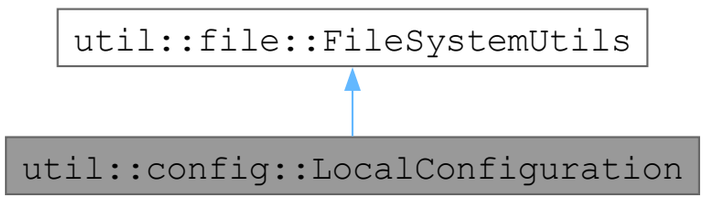
- utility/KmlUtil.h
- utility/KmlUtil.cpp

10.27 util::config::LocalConfiguration Class Reference

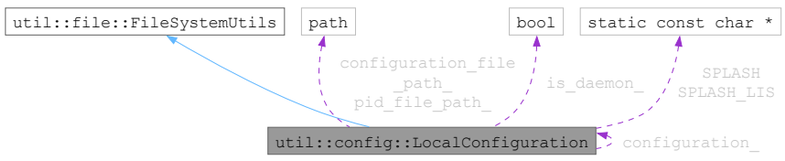
Class representing all program run-time configuration options.

`#include <LocalConfiguration.h>`

Inheritance diagram for util::config::LocalConfiguration:



Collaboration diagram for util::config::LocalConfiguration:



Public Member Functions

- LocalConfiguration ()
  The default constructor.
- ~LocalConfiguration ()
  The destructor.

Static Public Member Functions

- static __pid_t AcquireRuntimeLock () ATTR_NO_EXCEPTIONS
  Acquire a run-time lock.
- static void CheckRuntimeLock ()
  Check the run-time lock.
- static const LocalConfiguration & configuration ()
  Get the local configuration run-time settings object.
- static options_description GenerateCommandLineOptionsDescription () ATTR_NO_EXCEPTIONS
  Get the command-line interface configuration options' description.
- static options_description GenerateConfigurationFileOptionsDescription () ATTR_NO_EXCEPTIONS
  Get configuration file options' description.
- static const path & GetDirectoryPath (const path &dir, bool create=false)
  Get a directory path.
- static path GetDirectoryPathFromString (const string &dir, bool create=false)
  Generate a boost::filesystem::path object from the directory path string.
- static const LocalConfiguration & InitializeConfiguration (int argc, char **argv) ATTR_NO_EXCEPTIONS
  Configure the application run-time settings and acquire a run-time lock.
- static bool is_daemon ()
  Is the application configured to run as a daemon process?
- static vector< path > ListDirectories (const path &dir)
  List all directories contained in the given directory.
- static vector< path > ListFiles (path dir, size_t n=0)
  List all regular files contained in the given directory.
- static path MoveFileToDirectory (const path &file, const path &dir_dst, bool create_dir=false)
  Copy a file into a directory.
- static void ReleaseRuntimeLock ()
  Release the run-time lock.
- static const char * splash_lis ()
  Get the splash listing text string.

Private Member Functions

- void ParseProgramOptions (int argc, char **argv) ATTR_NO_EXCEPTIONS

  Parse the run-time configuration options from the command-line arguments and/or the configuration file.

Static Private Attributes

- static LocalConfiguration configuration_

  Statically-allocated global run-time configuration options object.
- static path configuration_file_path_

  Configuration file path string.
- static bool is_daemon_

  Indicates whether the program is to run as a daemon.
- static path pid_file_path_

  The path to the PID file on the local filesystem.
- static const char * SPLASH

  The splash string.
- static const char * SPLASH_LIS

  The listing splash string.

10.27.1 Detailed Description

Class representing all program run-time configuration options.

This class defines all run-time configuration options, descriptions, and default values. The boost::program_options library is used parse run-time configuration options from a configuration file and/or Command-Line Interface (CLI) arguments.

10.27.2 Constructor & Destructor Documentation

10.27.2.1 LocalConfiguration()

```
util::config::LocalConfiguration::LocalConfiguration () [inline]
```

The default constructor.

10.27.2.2 ~LocalConfiguration()

```
util::config::LocalConfiguration::~LocalConfiguration () [inline]
```

The destructor.

10.27.3 Member Function Documentation

10.27.3.1 AcquireRuntimeLock()

```
static __pid_t util::config::LocalConfiguration::AcquireRuntimeLock () [inline], [static]
```

Acquire a run-time lock.

A PID file is created upon successful acquisition of a run-time lock. This PID file is used to ensure that only a single instance of the program is running at any time. The file contains a single ASCII integer corresponding to the PID of the running instance. It is created during program initialization, and is removed upon termination. The file is checked periodically during execution to ensure that it still exists and that the PID is correct.

Exceptions

util::log::MessageException	if acquisition of the run-time lock fails e.g. another instance of the program has already acquired a run-time lock.

Note

To release the run-time lock, call the releaseRuntimeLock() function.

Returns

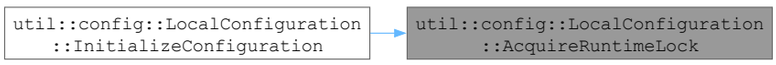
The PID of the process acquiring the run-time lock.

References CATCH_BLOCK, CheckRuntimeLock(), EXC_THROW, util::file::FileSystemUtils::GetDirectoryPathFromString(), and pid_file_path_.

Referenced by InitializeConfiguration().

Here is the call graph for this function:

Here is the caller graph for this function:



10.27.3.2 CheckRuntimeLock()

```
static void util::config::LocalConfiguration::CheckRuntimeLock ()  [inline], [static]
```

Check the run-time lock.

This method checks the run-time lock file to ensure that this instance of the program retains the run-time lock acquired using the acquireRuntimeLock() function. This means that the lock file exists, is accessible, and contains the PID of this instance of the program.

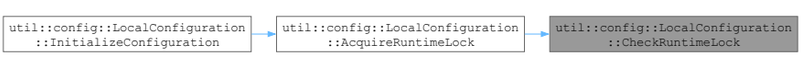
Exceptions

log::MessageException	if the run-time lock check fails e.g. the run-time lock file does not exist, is not accessible, or contains the incorrect PID.

References CATCH_BLOCK, EXC_THROW, and pid_file_path_.

Referenced by AcquireRuntimeLock().

Here is the caller graph for this function:



10.27.3.3 configuration()

```
static const LocalConfiguration & util::config::LocalConfiguration::configuration ()  [inline],
[static]
```

Get the local configuration run-time settings object.

Returns

The local configuration run-time settings object.

References configuration_.

10.27.3.4 GenerateCommandLineOptionsDescription()

```
options_description util::config::LocalConfiguration::GenerateCommandLineOptionsDescription ()
[static]
```

Get the command-line interface configuration options' description.

This function fully describes the configuration options that may be passed to the application via the command-line. It includes the short- and long-hand versions of the options, the value types and destinations, the default values, and a brief description of each option.

Exceptions

None.	Never throws.

Returns

Command-line interface configuration options description.

References CATCH_BLOCK, CFG_DEFAULT_ISDAEMON, CFG_DEFAULT_PATH, CFG_DEFAULT_REPROCESS_ALL, CFG_FMT_DESC_LEN, CFG_FMT_LINE_LEN, configuration_file_path_, and is_daemon_.

Referenced by ParseProgramOptions().

Here is the caller graph for this function:

10.27.3.5 GenerateConfigurationFileOptionsDescription()

```
options_description util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription
() [static]
```

Get configuration file options' description.

This function fully describes the configuration options that may be passed to the application via the configuration file. It includes the short- and long-hand versions of the options, the value types and destinations, the default values, and a brief description of each option.

Exceptions

None.	Never throws.

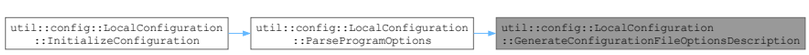
Returns

Configuration file options description.

References CATCH_BLOCK, CFG_DEFAULT_MAIL_CHK_DLY, CFG_DEFAULT_MAX_FETCH, CFG_DEFAULT_NDECO, CFG_DEFAULT_NPROC, CFG_DEFAULT_PATH_PID, CFG_FMT_DESC_LEN, CFG_FMT_LINE_LEN, CFG_MYSQL_DFLT_ADDRESS, CFG_MYSQL_DFLT_PORT, CFG_MYSQL_DFLT_PROTOCOL, CFG_MYSQL_DFLT_SCHEMA, pid_file_path_, XMAIL_DFLT_MSG_HEADER_VALID_SNDR_REGEX, XMAIL_DFLT_MSG_HEADER_VALID_SUBJ_REGEX, and XMAIL_DFLT_SBD_FILE_VALID_NAME_REGEX.

Referenced by ParseProgramOptions().

Here is the caller graph for this function:



10.27.3.6 GetDirectoryPath()

```
static const path & util::file::FileSystemUtils::GetDirectoryPath (
            const path & dir,
            bool create = false)  [inline], [static], [inherited]
```

Get a directory path.

Exceptions

MessageException	if create == FALSE and the directory specified by path does not exist.

Parameters

in	*dir*	Directory path.
in	*create*	If TRUE, the directory specified by path will be created if it does not already exists. If FALSE, an exception will be thrown if the directory does not exist.

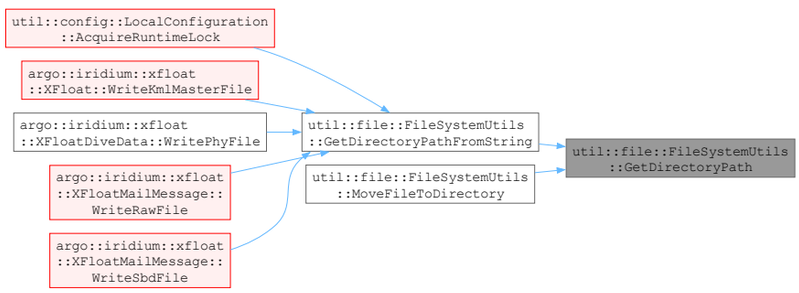
Returns

dir

References CATCH_BLOCK, EXC_THROW, and ldbg.

Referenced by util::file::FileSystemUtils::GetDirectoryPathFromString(), and util::file::FileSystemUtils::MoveFileToDirectory().

Here is the caller graph for this function:



10.27.3.7 GetDirectoryPathFromString()

```
static path util::file::FileSystemUtils::GetDirectoryPathFromString (
            const string & dir,
            bool create = false)  [inline], [static], [inherited]
```

Generate a boost::filesystem::path object from the directory path string.

Exceptions

MessageException	if create == FALSE and the directory specified by path does not exist.

Parameters

in	*dir*	Directory path string.
in	*create*	If TRUE, the directory specified by path will be created if it does not already exists. If FALSE, an exception will be thrown if the directory does not exist.

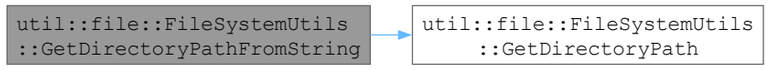
Returns

A boost::filesystem::path object from the directory path string.

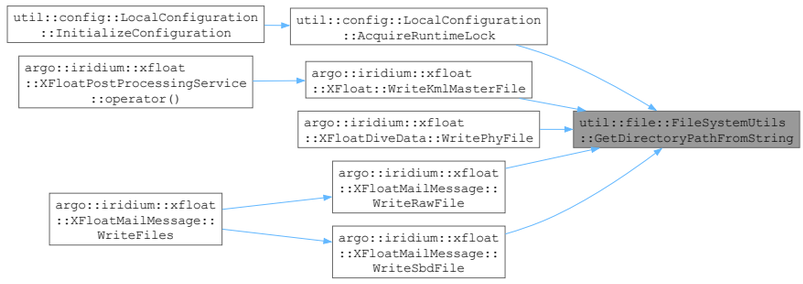
References CATCH_BLOCK, and util::file::FileSystemUtils::GetDirectoryPath().

Referenced by AcquireRuntimeLock(), argo::iridium::xfloat::XFloat::WriteKmlMasterFile(), argo::iridium::xfloat::XFloatDiveData::WritePhyF
argo::iridium::xfloat::XFloatMailMessage::WriteRawFile(), and argo::iridium::xfloat::XFloatMailMessage::WriteSbdFile().

Here is the call graph for this function:



Here is the caller graph for this function:



10.27.3.8 InitializeConfiguration()

```
static const LocalConfiguration & util::config::LocalConfiguration::InitializeConfiguration (
            int argc,
            char ** argv)  [inline], [static]
```

Configure the application run-time settings and acquire a run-time lock.

Parse the configuration options, then acquire a run-time lock. Verify the run-time lock before returning.

Parameters

in	*argc*	The number of command-line arguments.
in	*argv*	The command-line argument list.

Exceptions

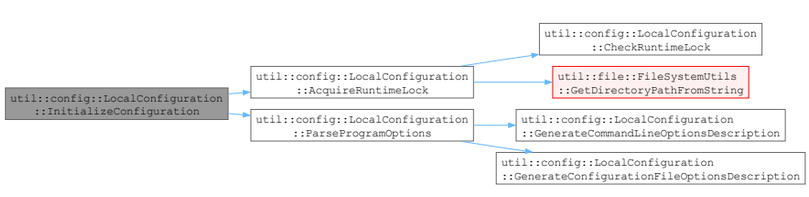
log::MessageException	if an error occurs while parsing the run-time configuration options or acquiring the run-time lock.

Returns

A const reference to the configuration_ object.

References AcquireRuntimeLock(), CATCH_BLOCK, configuration_, ParseProgramOptions(), and SPLASH.

Here is the call graph for this function:



10.27.3.9 is_daemon()

```
static bool util::config::LocalConfiguration::is_daemon ()  [inline], [static]
```

Is the application configured to run as a daemon process?

Returns

TRUE if the application is configured to run as a daemon process, or FALSE otherwise.

References is_daemon_.

10.27.3.10 ListDirectories()

```
static vector< path > util::file::FileSystemUtils::ListDirectories (
            const path & dir)  [inline], [static], [inherited]
```

List all directories contained in the given directory.

Parameters

in	*dir*	The path to the directory of interest.

Exceptions

log::MessageException	if the directory does not exist, is not a directory, or an error occurs.

Returns

A vector of paths to all the directories contained in the given directory.

References CATCH_BLOCK, and EXC_THROW.

10.27.3.11 ListFiles()

```
static vector< path > util::file::FileSystemUtils::ListFiles (
            path dir,
            size_t n = 0)  [inline], [static], [inherited]
```

List all regular files contained in the given directory.

Parameters

in	*dir*	The path to the directory of interest.
in	*n*	If n>0, the maximum number of paths to return, else no effect.

Exceptions

log::MessageException	if the directory does not exist, is not a directory, or an error occurs.

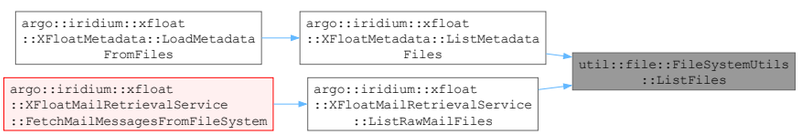
Returns

A vector of paths to all the regular files contained in the given directory.

References CATCH_BLOCK, and EXC_THROW.

Referenced by argo::iridium::xfloat::XFloatMetadata::ListMetadataFiles(), and argo::iridium::xfloat::XFloatMailRetrievalService::ListRawMa

Here is the caller graph for this function:



10.27.3.12 MoveFileToDirectory()

```
static path util::file::FileSystemUtils::MoveFileToDirectory (
            const path & file,
            const path & dir_dst,
            bool create_dir = false)  [inline], [static], [inherited]
```

Copy a file into a directory.

Warning

Despite its name, this function copies the file and does not remove the source. If a file with the same name already exists in the destination directory, it is left unchanged. It is not currently called.

Parameters

in	*file*	The file to copy.
in	*dir_dst*	The destination directory.
in	*create_dir*	If TRUE, create the destination directory if it does not exist.

Exceptions

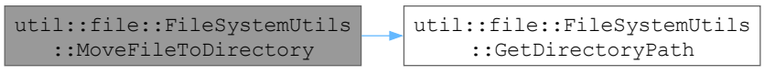
util::log::MessageException	if the file does not exist, is not a regular file, or cannot be copied.

Returns

The path of the file in the destination directory.

References CATCH_BLOCK, EXC_THROW, and util::file::FileSystemUtils::GetDirectoryPath().

Here is the call graph for this function:



10.27.3.13 ParseProgramOptions()

```
void util::config::LocalConfiguration::ParseProgramOptions (
            int argc,
            char ** argv)  [inline], [private]
```

Parse the run-time configuration options from the command-line arguments and/or the configuration file.

Populates the configuration options object.

Parameters

in	*argc*	The number of command-line arguments.
in	*argv*	The command-line argument list.

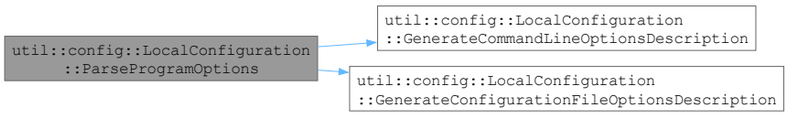
Exceptions

log::MessageException	if an error occurs while parsing the command-line arguments or the configuration file.

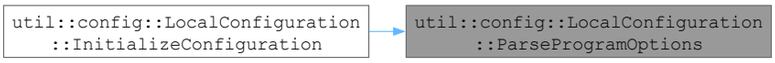
References CATCH_BLOCK, configuration_file_path_, DEFAULT_LOG_LEVEL, EXC_THROW, GenerateCommandLineOptionsDescriptio
GenerateConfigurationFileOptionsDescription(), util::log::LOG_DBG3, util::log::LOG_INF, and util::log::LOG_WRN.

Referenced by InitializeConfiguration().

Here is the call graph for this function:



Here is the caller graph for this function:



10.27.3.14 ReleaseRuntimeLock()

```
static void util::config::LocalConfiguration::ReleaseRuntimeLock ()  [inline], [static]
```

Release the run-time lock.

This method releases the run-time lock acquired by the AcquireRuntimeLock() function. It deletes the lock file from the local filesystem

References CATCH_BLOCK, EXC_THROW, ldbg, linf, lout, lwrn, and pid_file_path_.

10.27.3.15 splash_lis()

```
static const char * util::config::LocalConfiguration::splash_lis ()  [inline], [static]
```

Get the splash listing text string.

Returns

The splash listing text string.

References SPLASH_LIS.

10.27.4 Member Data Documentation

10.27.4.1 configuration_

```
LocalConfiguration util::config::LocalConfiguration::configuration_  [static], [private]
```

Statically-allocated global run-time configuration options object.

All program run-time configuration settings are stored here.

Referenced by configuration(), and InitializeConfiguration().

10.27.4.2 configuration_file_path_

```
path util::config::LocalConfiguration::configuration_file_path_  [static], [private]
```

Configuration file path string.

Referenced by GenerateCommandLineOptionsDescription(), and ParseProgramOptions().

10.27.4.3 is_daemon_

```
bool util::config::LocalConfiguration::is_daemon_  [static], [private]
```

Indicates whether the program is to run as a daemon.

Referenced by GenerateCommandLineOptionsDescription(), and is_daemon().

10.27.4.4 pid_file_path_

```
path util::config::LocalConfiguration::pid_file_path_  [static], [private]
```

The path to the PID file on the local filesystem.

This PID file is used to ensure that only a single instance of the program is running at any time. The file contains a single ASCII integer corresponding to the PID of the running instance. It is created during program initialization, and is removed upon termination. The file is checked periodically during execution to ensure that it still exists and that the PID is correct.

Referenced by AcquireRuntimeLock(), CheckRuntimeLock(), GenerateConfigurationFileOptionsDescription(), and ReleaseRuntimeLock().

10.27.4.5 SPLASH

```
const char * util::config::LocalConfiguration::SPLASH  [static], [private]
```

Initial value:
=

```
"================================================================================================\n"
"\n"
"  X-Message Mail-Telemetry Processing Service\n"
"  v" PROG_VERS " (built " BUILD_DATETIME ")\n"
"  Alexander K. Ekholm, Research Engineer, Float Group\n"
"  Physical Oceanography\n"
"  Woods Hole Oceanographic Institution\n"
"  Woods Hole, MA, USA\n"
"\n"
"================================================================================================\n"
""
```

The splash string.

This c-string is printed to standard output upon program startup,

Referenced by InitializeConfiguration().

10.27.4.6 SPLASH_LIS

```
const char * util::config::LocalConfiguration::SPLASH_LIS  [static], [private]
```

Initial value:

```
=
"#=============================================================================================================\n"
"#        _       ____   ______  ____   _________              __      ______\n"
"#       | |     / / / / / __ \\/  _/  / ____/ /___  ____ _/ /_    / ____/_________  __  ______\n"
"#       | | /| / / /_/ / / / // /    / /_   / / __ \\/ __ `/ __/   / / __/ ___/ __ \\/ / / / __ \\\n"
"#       | |/ |/ / __  / /_/ // /    / __/ / / /_/ / /_/ / /_    / /_/ / /  / /_/ / /_/ / /_/ /\n"
"#       |__/|__/_/ /_/\\____/___/   /_/    /_/\\____/\\__,_/\\__/    \\____/_/    \\____/\\__,_/  .___/\n"
"#                                                                                        /_/\n"
"#=============================================================================================================\n"
"#\n"
"# This data was produced and distributed as part of projects that were planned and executed by or\n"
"# in collaboration with the WHOI Float Group.\n"
"#\n"
"# This file was created by:\n"
"#\n"
"#   The X-Message Telemetry Processing Service (XMTPS)\n"
"#   version " PROG_VERS " (built " BUILD_DATETIME ")\n"
"#\n"
"# XMTPS was created and is maintained by:\n"
"#\n"
"#   Alexander K. Ekholm <aekholm@whoi.edu>\n"
"#   Research Engineer, Float Group\n"
"#   Department of Physical Oceanography\n"
"#   Woods Hole Oceanographic Institution\n"
"#   Woods Hole, MA, USA\n"
"#\n"
```

The listing splash string.

The file header for created ASCII files such as RAW mail messages, and intermediate listing files.

Referenced by splash_lis().

The documentation for this class was generated from the following files:

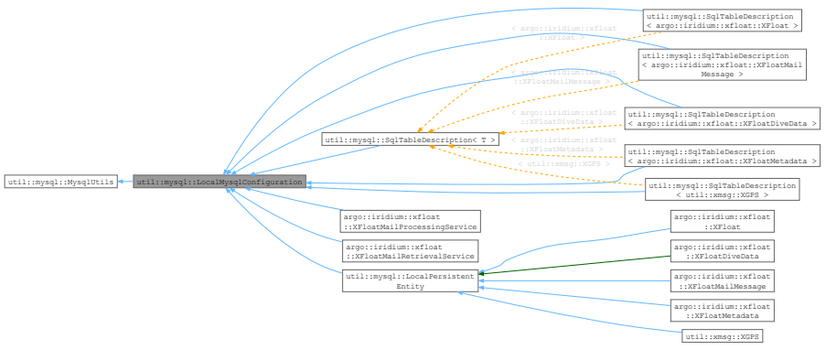
- config/LocalConfiguration.h
- config/LocalConfiguration.cpp
- Main.cpp

10.28 util::mysql::LocalMysqlConfiguration Class Reference

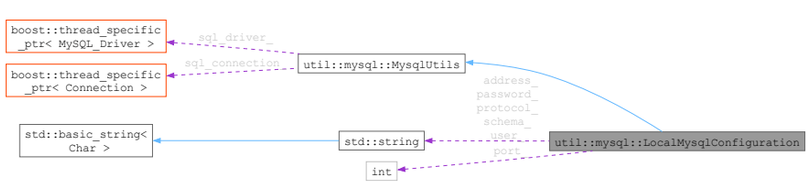
Class providing useful functions for accessing and manipulating the MySQL database, given the local run-time configuration settings.

`#include <LocalMysqlConfiguration.h>`

Inheritance diagram for util::mysql::LocalMysqlConfiguration:



Collaboration diagram for util::mysql::LocalMysqlConfiguration:



Public Member Functions

- LocalMysqlConfiguration ()
  Default constructor.
- virtual ~LocalMysqlConfiguration ()
  Destructor.

Static Public Member Functions

- static void CloseSqlConnection ()
  Close the thread-specific MySQL connection if it is currently open.
- static shared_ptr< sql::ResultSet > **executeDeleteQuery** (const string &from, const string &where="")
- static shared_ptr< ResultSet > ExecuteQuery (Connection *con, const string &query)
  Execute an SQL query and return a shared pointer to the ResultSet object.
- static shared_ptr< sql::ResultSet > executeQuery (const string &query)
  Execute an SQL query and return a shared pointer to the ResultSet object.
- static shared_ptr< sql::ResultSet > executeSelectQuery (const string &what, const string &from, const string &where="", const string &groupBy="", const string &orderBy="", unsigned int limit=0, unsigned int offset=0)
  Execute an SQL select query and return a shared pointer to the ResultSet object.
- static int ExecuteUpdate (Connection *con, const string &update)
  Execute an SQL update and return the status code.
- static int executeUpdate (const string &update)
  Execute an SQL update and return the status code.
- static shared_ptr< PreparedStatement > GeneratePreparedStatement (Connection *con, const string &statement)
  Generate a prepared SQL statement object for the given SQL statement.
- static shared_ptr< sql::PreparedStatement > generatePreparedStatement (const string &statement)
  Generate a prepared SQL statement object for the given SQL statement.
- static string GenerateSqlDeleteStr (const string &from, const string &where="")
  Generate an SQL DELETE statement.
- static string GenerateSqlSelectStr (const string &what, const string &from, const string &where="", const string &groupBy="", const string &orderBy="", unsigned int limit=0, unsigned int offset=0)
  Generate an SQL select statement.
- static thread_specific_ptr< Connection > & GetSqlConnection (string url, string user, string password, string schema)
  Get a thread-specific instance of a MySQL connection object.
- static thread_specific_ptr< MySQL_Driver > & GetSqlDriver ()
  Get a thread-specific instance of a MySQL driver object.

Static Protected Member Functions

- static sql::Connection * getSqlConnection ()
  Generate a thread-specific instance of a MySQL connection object.

Static Private Member Functions

- static void CleanupSqlConnection (Connection *con)
  Cleanup an SQL connection.
- static void CleanupSqlDriver (MySQL_Driver *driver)
  Function for memory cleanup of MySQL driver on destruct.

Static Private Attributes

- static string address_
  The SQL server qualified name or IP address string.
- static string password_
  The SQL server account password string.
- static unsigned int port_
  The SQL server port number.
- static string protocol_
  The protocol to use in the SQL server connection e.g.
- static string schema_
  The SQL server schema name.
- static thread_specific_ptr< Connection > sql_connection_
  Tthe thread-specific pointer to MySQL connection object.
- static thread_specific_ptr< MySQL_Driver > sql_driver_
  The thread-specific pointer to MySQL driver object.
- static string user_
  The SQL server account username string.

Friends

- class **config::LocalConfiguration**

10.28.1 Detailed Description

Class providing useful functions for accessing and manipulating the MySQL database, given the local run-time configuration settings.

10.28.2 Constructor & Destructor Documentation

10.28.2.1 LocalMysqlConfiguration()

```
util::mysql::LocalMysqlConfiguration::LocalMysqlConfiguration ()
```

Default constructor.

10.28.2.2 ~LocalMysqlConfiguration()

```
util::mysql::LocalMysqlConfiguration::~LocalMysqlConfiguration ()  [virtual]
```

Destructor.

10.28.3 Member Function Documentation

10.28.3.1 CleanupSqlConnection()

```
static void util::mysql::MysqlUtils::CleanupSqlConnection (
            Connection * con)  [inline], [static], [private], [inherited]
```

Cleanup an SQL connection.

This function for memory cleanup of the SQL connection on destruction. The thread-specific SQL connection object will be deleted if it exists.

Parameters

in	*con*	The SQL connection to cleanup.

Exceptions

log::MessageException	if an error occurs while cleaning up the SQL connection object.

References CATCH_BLOCK.

10.28.3.2 CleanupSqlDriver()

```
static void util::mysql::MysqlUtils::CleanupSqlDriver (
            MySQL_Driver * driver)  [inline], [static], [private], [inherited]
```

Function for memory cleanup of MySQL driver on destruct.

Signals the SQL driver to end operation, if it exists.

Parameters

in	*driver*	Driver to cleanup.

Exceptions

log::MessageException	if an error occurs while cleaning up the SQL driver object.

References CATCH_BLOCK.

10.28.3.3 CloseSqlConnection()

```
static void util::mysql::MysqlUtils::CloseSqlConnection ()  [inline], [static], [inherited]
```

Close the thread-specific MySQL connection if it is currently open.

If the thread local MySQL connection is not currently open, then this function will have no effect.

Note

The MySQL connection should be closed when idle to prevent the connection from timing out and throwing exceptions.

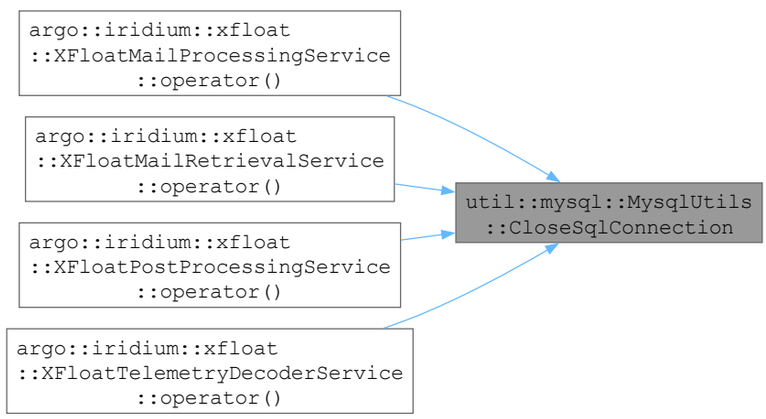
Exceptions

log::MessageException	if an error occurs while attempting to close the MySQL connection.

References CATCH_BLOCK, and util::mysql::MysqlUtils::sql_connection_.

Referenced by argo::iridium::xfloat::XFloatMailProcessingService::operator()(), argo::iridium::xfloat::XFloatMailRetrievalService::operator(
argo::iridium::xfloat::XFloatPostProcessingService::operator()(), and argo::iridium::xfloat::XFloatTelemetryDecoderService::operator()().

Here is the caller graph for this function:



10.28.3.4 ExecuteQuery()

```
static shared_ptr< ResultSet > util::mysql::MysqlUtils::ExecuteQuery (
            Connection * con,
            const string & query)  [inline], [static], [inherited]
```

Execute an SQL query and return a shared pointer to the ResultSet object.

Parameters

in	*con*	A pointer to the SQL connection object. This must not be NULL.
in	*query*	The SQL query to execute. This must be a well-formatted SQL query string.

Exceptions

log::MessageException	if 'con' is null, 'query' is a malfored SQL string, or any other error occurs.

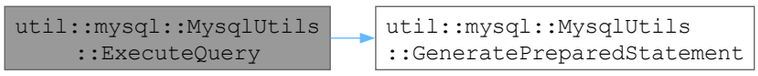
Returns

A shared pointer to the query ResultSet object.

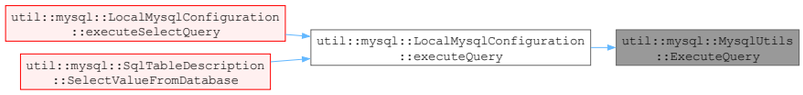
References CATCH_BLOCK, and util::mysql::MysqlUtils::GeneratePreparedStatement().

Referenced by executeQuery().

Here is the call graph for this function:



Here is the caller graph for this function:



10.28.3.5 executeQuery()

```
shared_ptr< sql::ResultSet > util::mysql::LocalMysqlConfiguration::executeQuery (
            const string & query)  [static]
```

Execute an SQL query and return a shared pointer to the ResultSet object.

Parameters

in	*query*	The SQL query to execute. This must be a well-formatted SQL query string.

Exceptions

log::MessageException	'query' is a malfored SQL string or any other error occurs while executing the query.

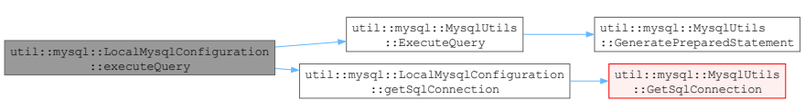
Returns

A shared pointer to the query ResultSet object.

References util::mysql::MysqlUtils::ExecuteQuery(), and getSqlConnection().

Referenced by executeSelectQuery(), and util::mysql::SqlTableDescription< T >::SelectValueFromDatabase().

Here is the call graph for this function:



Here is the caller graph for this function:

10.28.3.6 executeSelectQuery()

```
shared_ptr< sql::ResultSet > util::mysql::LocalMysqlConfiguration::executeSelectQuery (
            const string & what,
            const string & from,
            const string & where = "",
            const string & groupBy = "",
            const string & orderBy = "",
            unsigned int limit = 0,
            unsigned int offset = 0)  [static]
```

Execute an SQL select query and return a shared pointer to the ResultSet object.

Parameters

in	*what*	The SQL select statement select-expression clause.
in	*from*	The SQL select statement table–references clause.
in	*where*	The SQL select statement where-condition clause.
in	*groupBy*	The SQL select statement group-by clause.
in	*orderBy*	The SQL select statement order-by clause.
in	*limit*	The SQL select statement result set row limit value.
in	*offset*	The SQL select statement result set row offset.

Exceptions

log::MessageException	if an error occurs while executing the query.

Returns

A shared pointer to the ResultSet object.

References CATCH_BLOCK, executeQuery(), and util::mysql::MysqlUtils::GenerateSqlSelectStr().

Referenced by util::mysql::SqlTableDescription< T >::SelectObjectsFromDatabase().

Here is the call graph for this function:

Here is the caller graph for this function:

10.28.3.7 ExecuteUpdate()

```
static int util::mysql::MysqlUtils::ExecuteUpdate (
            Connection * con,
            const string & update)  [inline], [static], [inherited]
```

Execute an SQL update and return the status code.

Parameters

in	*con*	A pointer to the SQL connection object. This must not be NULL.
in	*update*	The SQL update to execute. This must be a well-formatted SQL update string.

Exceptions

log::MessageException	if 'con' is null, 'update' is a malformed SQL string, or any other error occurs.

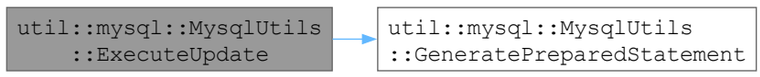
Returns

The update status code.

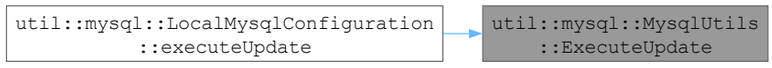
References CATCH_BLOCK, and util::mysql::MysqlUtils::GeneratePreparedStatement().

Referenced by executeUpdate().

Here is the call graph for this function:



Here is the caller graph for this function:



10.28.3.8 executeUpdate()

```
int util::mysql::LocalMysqlConfiguration::executeUpdate (
            const string & update)  [static]
```

Execute an SQL update and return the status code.

Parameters

in	*update*	The SQL update to execute. This must be a well-formatted SQL update string.

Exceptions

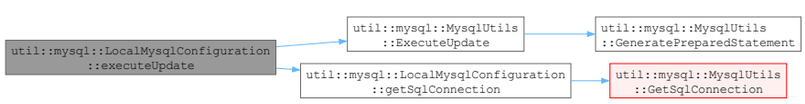
log::MessageException	if 'update' is a malformed SQL string, or any other error occurs while executing the update.

Returns

The update status code.

References util::mysql::MysqlUtils::ExecuteUpdate(), and getSqlConnection().

Here is the call graph for this function:



10.28.3.9 GeneratePreparedStatement()

```
static shared_ptr< PreparedStatement > util::mysql::MysqlUtils::GeneratePreparedStatement (
            Connection * con,
            const string & statement)  [inline], [static], [inherited]
```

Generate a prepared SQL statement object for the given SQL statement.

Parameters

in	*con*	A pointer to the SQL connection object. This must not be NULL.
in	*statement*	The SQL statement to prepare. This must be a well-formed SQL string.

Exceptions

log::MessageException	if 'con' is NULL, 'statement' is not a valid SQL string, or if an error occurs.

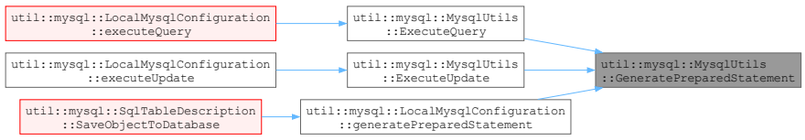
Returns

A prepared SQL statement object generated from the given SQL statement.

References CATCH_BLOCK, and EXC_THROW.

Referenced by util::mysql::MysqlUtils::ExecuteQuery(), util::mysql::MysqlUtils::ExecuteUpdate(), and generatePreparedStatement().

Here is the caller graph for this function:



10.28.3.10 generatePreparedStatement()

```
std::shared_ptr< sql::PreparedStatement > util::mysql::LocalMysqlConfiguration::generatePrepared↩
Statement (
            const string & statement)  [static]
```

Generate a prepared SQL statement object for the given SQL statement.

Parameters

in	*statement*	The SQL statement to prepare. This must be a well-formed SQL string.

Exceptions

log::MessageException	if 'statement' is not a valid SQL string or if an error occurs while generating the prepared statement object..

Returns

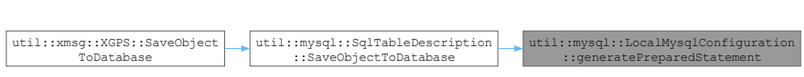
A prepared SQL statement object generated from the given SQL statement.

References util::mysql::MysqlUtils::GeneratePreparedStatement(), and getSqlConnection().

Referenced by util::mysql::SqlTableDescription< T >::SaveObjectToDatabase().

Here is the call graph for this function:

Here is the caller graph for this function:



10.28.3.11 GenerateSqlDeleteStr()

```
static string util::mysql::MysqlUtils::GenerateSqlDeleteStr (
            const string & from,
            const string & where = "")  [inline], [static], [inherited]
```

Generate an SQL DELETE statement.

Warning

Without a `where` clause, the statement deletes every row in the table.

Parameters

in	*from*	The name of the table to delete from.
in	*where*	The SQL "where-condition" clause (empty = all rows).

Returns

The SQL statement.

10.28.3.12 GenerateSqlSelectStr()

```
static string util::mysql::MysqlUtils::GenerateSqlSelectStr (
            const string & what,
            const string & from,
            const string & where = "",
            const string & groupBy = "",
            const string & orderBy = "",
            unsigned int limit = 0,
            unsigned int offset = 0)  [inline], [static], [inherited]
```

Generate an SQL select statement.

Parameters

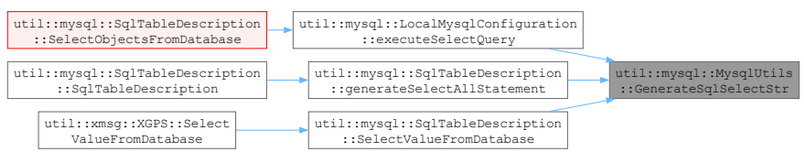
in	*what*	The SQL select statement select-expression clause.
in	*from*	The SQL select statement table-references clause.
in	*where*	The SQL select statement where-condition clause.
in	*groupBy*	The SQL select statement group-by clause.
in	*orderBy*	The SQL select statement order-by clause.
in	*limit*	The SQL select statement result set row limit value.
in	*offset*	The SQL select statement result set row offset.

Returns

The SQL select statement.

Referenced by executeSelectQuery(), util::mysql::SqlTableDescription< T >::generateSelectAllStatement(), and util::mysql::SqlTableDescription< T >::SelectValueFromDatabase().

Here is the caller graph for this function:



10.28.3.13 GetSqlConnection()

```
static thread_specific_ptr< Connection > & util::mysql::MysqlUtils::GetSqlConnection (
            string url,
            string user,
            string password,
            string schema)  [inline], [static], [inherited]
```

Get a thread-specific instance of a MySQL connection object.

A new connection will be created, if necessary.

Parameters

in	*url*	The MySQL host server qualified address.
in	*user*	The MySQL account username.
in	*password*	The MySQL account password.
in	*schema*	The MySQL account schema name.

Exceptions

log::MessageException	if an error occurs while creating a new thread-specific instance of the SQL connection object

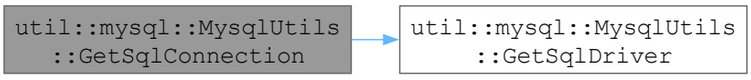
Returns

A thread-specific instance of a MySQL connection object.

References CATCH_BLOCK, util::mysql::MysqlUtils::GetSqlDriver(), and util::mysql::MysqlUtils::sql_connection_.

Referenced by getSqlConnection().

Here is the call graph for this function:



Here is the caller graph for this function:

10.28.3.14 getSqlConnection()

```
sql::Connection * util::mysql::LocalMysqlConfiguration::getSqlConnection ()  [static], [protected]
```

Generate a thread-specific instance of a MySQL connection object.

A new connection will be created, if necessary.

Exceptions

log::MessageException	if an error occurs while generating the SQL connection object.

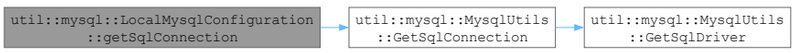
Returns

A thread-specific instance of a MySQL connection object.

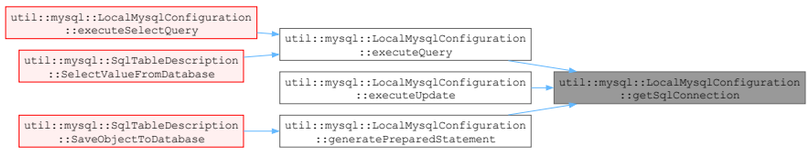
References address_, CATCH_BLOCK, util::mysql::MysqlUtils::GetSqlConnection(), password_, port_, protocol_, schema_, and user_.

Referenced by executeQuery(), executeUpdate(), and generatePreparedStatement().

Here is the call graph for this function:



Here is the caller graph for this function:



10.28.3.15 GetSqlDriver()

```
static thread_specific_ptr< MySQL_Driver > & util::mysql::MysqlUtils::GetSqlDriver ()  [inline], [static], [inherited]
```

Get a thread-specific instance of a MySQL driver object.

A new driver will created, if necessary.

Exceptions

log::MessageException	if an error occurs while creating a new thread-specific instance of the SQL driver object

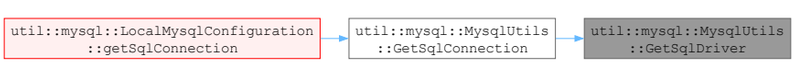
Returns

A thread-specific instance of a MySQL driver object.

References CATCH_BLOCK, and util::mysql::MysqlUtils::sql_driver_.

Referenced by util::mysql::MysqlUtils::GetSqlConnection().

Here is the caller graph for this function:



10.28.4 Member Data Documentation

10.28.4.1 address_

```
std::string util::mysql::LocalMysqlConfiguration::address_  [static], [private]
```

The SQL server qualified name or IP address string.

Referenced by getSqlConnection().

10.28.4.2 password_

`std::string util::mysql::LocalMysqlConfiguration::password_ [static], [private]`

The SQL server account password string.

Referenced by getSqlConnection().

10.28.4.3 port_

`unsigned int util::mysql::LocalMysqlConfiguration::port_ [static], [private]`

The SQL server port number.

Referenced by getSqlConnection().

10.28.4.4 protocol_

`std::string util::mysql::LocalMysqlConfiguration::protocol_ [static], [private]`

The protocol to use in the SQL server connection e.g.

"tcp".

Referenced by getSqlConnection().

10.28.4.5 schema_

`std::string util::mysql::LocalMysqlConfiguration::schema_ [static], [private]`

The SQL server schema name.

Referenced by getSqlConnection().

10.28.4.6 sql_connection_

`thread_specific_ptr<Connection> util::mysql::MysqlUtils::sql_connection_ [static], [private], [inherited]`

Tthe thread-specific pointer to MySQL connection object.

Thread-specific MySQL connection, released by `MysqlUtils::CleanupSqlConnection()` when the thread exits.

Referenced by util::mysql::MysqlUtils::CloseSqlConnection(), and util::mysql::MysqlUtils::GetSqlConnection().

10.28.4.7 sql_driver_

`thread_specific_ptr<MySQL_Driver> util::mysql::MysqlUtils::sql_driver_ [static], [private], [inherited]`

The thread-specific pointer to MySQL driver object.

Thread-specific reference to the MySQL driver.

Its cleanup function, `MysqlUtils::CleanupSqlDriver()`, does not delete the shared driver instance.

Referenced by util::mysql::MysqlUtils::GetSqlDriver().

10.28.4.8 user_

```
std::string util::mysql::LocalMysqlConfiguration::user_  [static], [private]
```

The SQL server account username string.

Referenced by getSqlConnection().

The documentation for this class was generated from the following files:

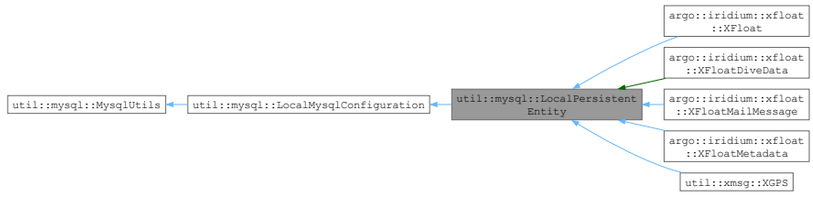
- config/LocalMysqlConfiguration.h
- config/LocalMysqlConfiguration.cpp

10.29 util::mysql::LocalPersistentEntity Class Reference

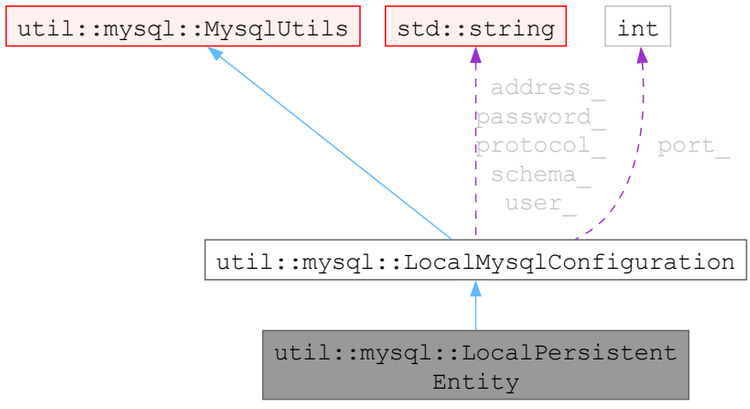
An abstract class representing a persistent database entity in the database defined by the config::LocalConfiguration.

```
#include <LocalPersistentEntity.h>
```

Inheritance diagram for util::mysql::LocalPersistentEntity:



Collaboration diagram for util::mysql::LocalPersistentEntity:



Public Member Functions

- LocalPersistentEntity ()

  Default constructor.
- virtual ~LocalPersistentEntity ()

  The destructor.
- virtual void SaveObjectToDatabase () const =0

  Persist this database entity.

Static Public Member Functions

- static void CloseSqlConnection ()

  Close the thread-specific MySQL connection if it is currently open.
- static shared_ptr< sql::ResultSet > **executeDeleteQuery** (const string &from, const string &where="")
- static shared_ptr< ResultSet > ExecuteQuery (Connection *con, const string &query)

  Execute an SQL query and return a shared pointer to the ResultSet object.
- static shared_ptr< sql::ResultSet > executeQuery (const string &query)

  Execute an SQL query and return a shared pointer to the ResultSet object.
- static shared_ptr< sql::ResultSet > executeSelectQuery (const string &what, const string &from, const string &where="", const string &groupBy="", const string &orderBy="", unsigned int limit=0, unsigned int offset=0)

  Execute an SQL select query and return a shared pointer to the ResultSet object.
- static int ExecuteUpdate (Connection *con, const string &update)

  Execute an SQL update and return the status code.
- static int executeUpdate (const string &update)

  Execute an SQL update and return the status code.
- static shared_ptr< PreparedStatement > GeneratePreparedStatement (Connection *con, const string &statement)

  Generate a prepared SQL statement object for the given SQL statement.
- static shared_ptr< sql::PreparedStatement > generatePreparedStatement (const string &statement)

  Generate a prepared SQL statement object for the given SQL statement.
- static string GenerateSqlDeleteStr (const string &from, const string &where="")

  Generate an SQL DELETE statement.
- static string GenerateSqlSelectStr (const string &what, const string &from, const string &where="", const string &groupBy="", const string &orderBy="", unsigned int limit=0, unsigned int offset=0)

  Generate an SQL select statement.
- static thread_specific_ptr< Connection > & GetSqlConnection (string url, string user, string password, string schema)

  Get a thread-specific instance of a MySQL connection object.
- static thread_specific_ptr< MySQL_Driver > & GetSqlDriver ()

  Get a thread-specific instance of a MySQL driver object.

Static Protected Member Functions

- static sql::Connection * getSqlConnection ()

  Generate a thread-specific instance of a MySQL connection object.

Static Private Member Functions

- static void CleanupSqlConnection (Connection *con)

  Cleanup an SQL connection.
- static void CleanupSqlDriver (MySQL_Driver *driver)

  Function for memory cleanup of MySQL driver on destruct.

Static Private Attributes

- static string address_

  The SQL server qualified name or IP address string.
- static string password_

  The SQL server account password string.
- static unsigned int port_

  The SQL server port number.
- static string protocol_

  The protocol to use in the SQL server connection e.g.
- static string schema_

  The SQL server schema name.
- static thread_specific_ptr< Connection > sql_connection_

  Tthe thread-specific pointer to MySQL connection object.
- static thread_specific_ptr< MySQL_Driver > sql_driver_

  The thread-specific pointer to MySQL driver object.
- static string user_

  The SQL server account username string.

10.29.1 Detailed Description

An abstract class representing a persistent database entity in the database defined by the config::LocalConfiguration.

10.29.2 Constructor & Destructor Documentation

10.29.2.1 LocalPersistentEntity()

```
util::mysql::LocalPersistentEntity::LocalPersistentEntity ()  [inline]
```

Default constructor.

10.29.2.2 ~LocalPersistentEntity()

```
virtual util::mysql::LocalPersistentEntity::~LocalPersistentEntity ()  [inline], [virtual]
```

The destructor.

10.29.3 Member Function Documentation

10.29.3.1 CleanupSqlConnection()

```
static void util::mysql::MysqlUtils::CleanupSqlConnection (
            Connection * con)  [inline], [static], [private], [inherited]
```

Cleanup an SQL connection.

This function for memory cleanup of the SQL connection on destruction. The thread-specific SQL connection object will be deleted if it exists.

Parameters

in	*con*	The SQL connection to cleanup.

Exceptions

log::MessageException	if an error occurs while cleaning up the SQL connection object.

References CATCH_BLOCK.

10.29.3.2 CleanupSqlDriver()

```
static void util::mysql::MysqlUtils::CleanupSqlDriver (
            MySQL_Driver * driver)  [inline], [static], [private], [inherited]
```

Function for memory cleanup of MySQL driver on destruct.

Signals the SQL driver to end operation, if it exists.

Parameters

in	*driver*	Driver to cleanup.

Exceptions

log::MessageException	if an error occurs while cleaning up the SQL driver object.

References CATCH_BLOCK.

10.29.3.3 CloseSqlConnection()

```
static void util::mysql::MysqlUtils::CloseSqlConnection ()  [inline], [static], [inherited]
```

Close the thread-specific MySQL connection if it is currently open.

If the thread local MySQL connection is not currently open, then this function will have no effect.

Note

The MySQL connection should be closed when idle to prevent the connection from timing out and throwing exceptions.

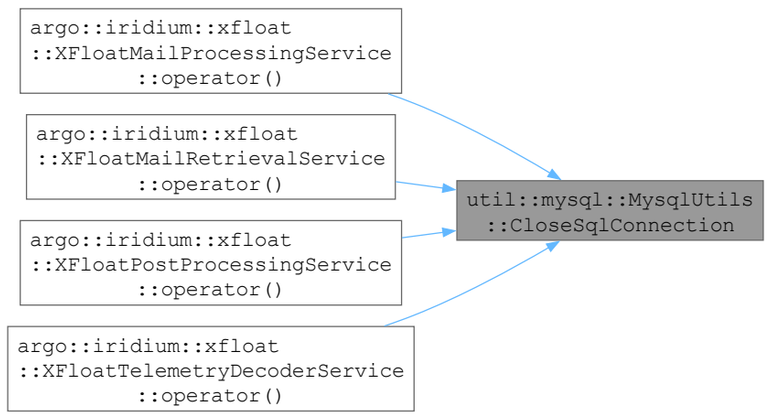
Exceptions

log::MessageException	if an error occurs while attempting to close the MySQL connection.

References CATCH_BLOCK, and util::mysql::MysqlUtils::sql_connection_.

Referenced by argo::iridium::xfloat::XFloatMailProcessingService::operator()(), argo::iridium::xfloat::XFloatMailRetrievalService::operator(
argo::iridium::xfloat::XFloatPostProcessingService::operator()(), and argo::iridium::xfloat::XFloatTelemetryDecoderService::operator()().

Here is the caller graph for this function:



10.29.3.4 ExecuteQuery()

```
static shared_ptr< ResultSet > util::mysql::MysqlUtils::ExecuteQuery (
            Connection * con,
            const string & query)  [inline], [static], [inherited]
```

Execute an SQL query and return a shared pointer to the ResultSet object.

Parameters

in	*con*	A pointer to the SQL connection object. This must not be NULL.
in	*query*	The SQL query to execute. This must be a well-formatted SQL query string.

Exceptions

log::MessageException	if 'con' is null, 'query' is a malfored SQL string, or any other error occurs.

Returns

A shared pointer to the query ResultSet object.

References CATCH_BLOCK, and util::mysql::MysqlUtils::GeneratePreparedStatement().

Referenced by util::mysql::LocalMysqlConfiguration::executeQuery().

Here is the call graph for this function:

Here is the caller graph for this function:

10.29.3.5 executeQuery()

```
shared_ptr< sql::ResultSet > util::mysql::LocalMysqlConfiguration::executeQuery (
            const string & query)  [static], [inherited]
```

Execute an SQL query and return a shared pointer to the ResultSet object.

Parameters

in	*query*	The SQL query to execute. This must be a well-formatted SQL query string.

Exceptions

log::MessageException	'query' is a malfored SQL string or any other error occurs while executing the query.

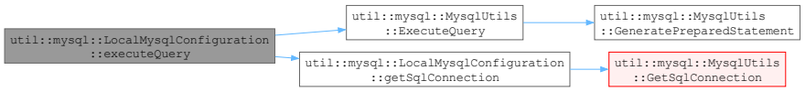
Returns

A shared pointer to the query ResultSet object.

References util::mysql::MysqlUtils::ExecuteQuery(), and util::mysql::LocalMysqlConfiguration::getSqlConnection().

Referenced by util::mysql::LocalMysqlConfiguration::executeSelectQuery(), and util::mysql::SqlTableDescription< T >::SelectValueFromD

Here is the call graph for this function:



Here is the caller graph for this function:

10.29.3.6 executeSelectQuery()

```
shared_ptr< sql::ResultSet > util::mysql::LocalMysqlConfiguration::executeSelectQuery (
            const string & what,
            const string & from,
            const string & where = "",
            const string & groupBy = "",
            const string & orderBy = "",
            unsigned int limit = 0,
            unsigned int offset = 0)  [static], [inherited]
```

Execute an SQL select query and return a shared pointer to the ResultSet object.

Parameters

in	*what*	The SQL select statement select-expression clause.
in	*from*	The SQL select statement table–references clause.
in	*where*	The SQL select statement where-condition clause.
in	*groupBy*	The SQL select statement group-by clause.
in	*orderBy*	The SQL select statement order-by clause.
in	*limit*	The SQL select statement result set row limit value.
in	*offset*	The SQL select statement result set row offset.

Exceptions

log::MessageException	if an error occurs while executing the query.

Returns

A shared pointer to the ResultSet object.

References CATCH_BLOCK, util::mysql::LocalMysqlConfiguration::executeQuery(), and util::mysql::MysqlUtils::GenerateSqlSelectStr().

Referenced by util::mysql::SqlTableDescription< T >::SelectObjectsFromDatabase().

Here is the call graph for this function:

Here is the caller graph for this function:

10.29.3.7 ExecuteUpdate()

```
static int util::mysql::MysqlUtils::ExecuteUpdate (
            Connection * con,
            const string & update)  [inline], [static], [inherited]
```

Execute an SQL update and return the status code.

Parameters

in	*con*	A pointer to the SQL connection object. This must not be NULL.
in	*update*	The SQL update to execute. This must be a well-formatted SQL update string.

Exceptions

log::MessageException	if 'con' is null, 'update' is a malformed SQL string, or any other error occurs.

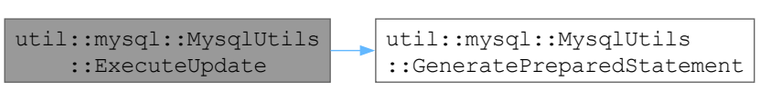
Returns

The update status code.

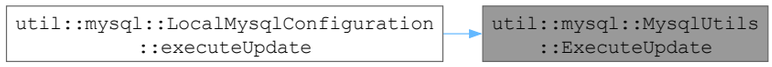
References CATCH_BLOCK, and util::mysql::MysqlUtils::GeneratePreparedStatement().

Referenced by util::mysql::LocalMysqlConfiguration::executeUpdate().

Here is the call graph for this function:



Here is the caller graph for this function:



10.29.3.8 executeUpdate()

```
int util::mysql::LocalMysqlConfiguration::executeUpdate (
            const string & update)  [static], [inherited]
```

Execute an SQL update and return the status code.

Parameters

in	*update*	The SQL update to execute. This must be a well-formatted SQL update string.

Exceptions

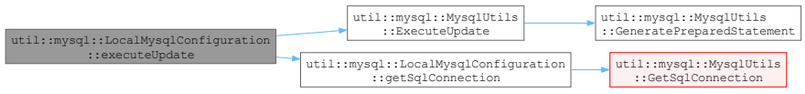
log::MessageException	if 'update' is a malformed SQL string, or any other error occurs while executing the update.

Returns

The update status code.

References util::mysql::MysqlUtils::ExecuteUpdate(), and util::mysql::LocalMysqlConfiguration::getSqlConnection().

Here is the call graph for this function:



10.29.3.9 GeneratePreparedStatement()

```
static shared_ptr< PreparedStatement > util::mysql::MysqlUtils::GeneratePreparedStatement (
            Connection * con,
            const string & statement)  [inline], [static], [inherited]
```

Generate a prepared SQL statement object for the given SQL statement.

Parameters

in	*con*	A pointer to the SQL connection object. This must not be NULL.
in	*statement*	The SQL statement to prepare. This must be a well-formed SQL string.

Exceptions

log::MessageException	if 'con' is NULL, 'statement' is not a valid SQL string, or if an error occurs.

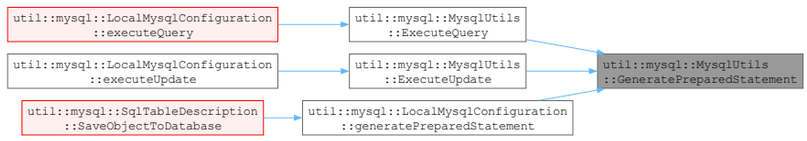
Returns

A prepared SQL statement object generated from the given SQL statement.

References CATCH_BLOCK, and EXC_THROW.

Referenced by util::mysql::MysqlUtils::ExecuteQuery(), util::mysql::MysqlUtils::ExecuteUpdate(), and util::mysql::LocalMysqlConfiguration:

Here is the caller graph for this function:



10.29.3.10 generatePreparedStatement()

```
std::shared_ptr< sql::PreparedStatement > util::mysql::LocalMysqlConfiguration::generatePrepared↩
Statement (
            const string & statement)  [static], [inherited]
```

Generate a prepared SQL statement object for the given SQL statement.

Parameters

in	*statement*	The SQL statement to prepare. This must be a well-formed SQL string.

Exceptions

log::MessageException	if 'statement' is not a valid SQL string or if an error occurs while generating the prepared statement object..

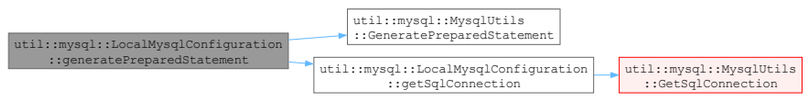
Returns

A prepared SQL statement object generated from the given SQL statement.

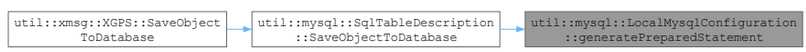
References util::mysql::MysqlUtils::GeneratePreparedStatement(), and util::mysql::LocalMysqlConfiguration::getSqlConnection().

Referenced by util::mysql::SqlTableDescription< T >::SaveObjectToDatabase().

Here is the call graph for this function:



Here is the caller graph for this function:



10.29.3.11 GenerateSqlDeleteStr()

```
static string util::mysql::MysqlUtils::GenerateSqlDeleteStr (
            const string & from,
            const string & where = "")  [inline], [static], [inherited]
```

Generate an SQL DELETE statement.

Warning

Without a `where` clause, the statement deletes every row in the table.

Parameters

in	*from*	The name of the table to delete from.
in	*where*	The SQL "where-condition" clause (empty = all rows).

Returns

The SQL statement.

10.29.3.12 GenerateSqlSelectStr()

```
static string util::mysql::MysqlUtils::GenerateSqlSelectStr (
            const string & what,
            const string & from,
            const string & where = "",
            const string & groupBy = "",
            const string & orderBy = "",
            unsigned int limit = 0,
            unsigned int offset = 0)  [inline], [static], [inherited]
```

Generate an SQL select statement.

Parameters

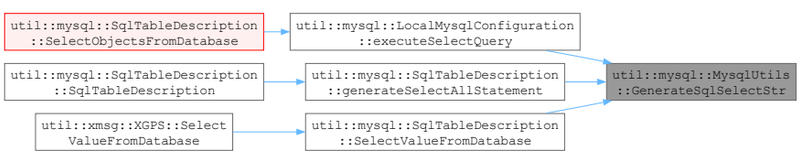
in	*what*	The SQL select statement select-expression clause.
in	*from*	The SQL select statement table-references clause.
in	*where*	The SQL select statement where-condition clause.
in	*groupBy*	The SQL select statement group-by clause.
in	*orderBy*	The SQL select statement order-by clause.
in	*limit*	The SQL select statement result set row limit value.
in	*offset*	The SQL select statement result set row offset.

Returns

The SQL select statement.

Referenced by util::mysql::LocalMysqlConfiguration::executeSelectQuery(), util::mysql::SqlTableDescription< T >::generateSelectAllState
and util::mysql::SqlTableDescription< T >::SelectValueFromDatabase().

Here is the caller graph for this function:



10.29.3.13 GetSqlConnection()

```
static thread_specific_ptr< Connection > & util::mysql::MysqlUtils::GetSqlConnection (
            string url,
            string user,
            string password,
            string schema)  [inline], [static], [inherited]
```

Get a thread-specific instance of a MySQL connection object.

A new connection will be created, if necessary.

Parameters

in	*url*	The MySQL host server qualified address.
in	*user*	The MySQL account username.
in	*password*	The MySQL account password.
in	*schema*	The MySQL account schema name.

Exceptions

log::MessageException	if an error occurs while creating a new thread-specific instance of the SQL connection object

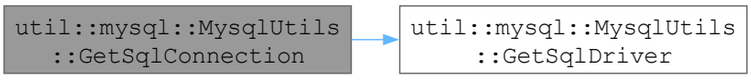
Returns

A thread-specific instance of a MySQL connection object.

References CATCH_BLOCK, util::mysql::MysqlUtils::GetSqlDriver(), and util::mysql::MysqlUtils::sql_connection_.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

Here is the call graph for this function:



Here is the caller graph for this function:

10.29.3.14 getSqlConnection()

`sql::Connection * util::mysql::LocalMysqlConfiguration::getSqlConnection () [static], [protected], [inherited]`

Generate a thread-specific instance of a MySQL connection object.

A new connection will be created, if necessary.

Exceptions

log::MessageException	if an error occurs while generating the SQL connection object.

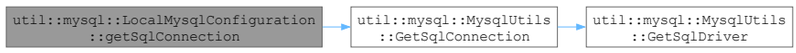
Returns

A thread-specific instance of a MySQL connection object.

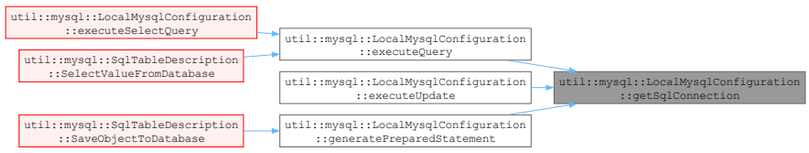
References util::mysql::LocalMysqlConfiguration::address_, CATCH_BLOCK, util::mysql::MysqlUtils::GetSqlConnection(), util::mysql::LocalMysqlConfiguration::password_, util::mysql::LocalMysqlConfiguration::port_, util::mysql::LocalMysqlConfiguration::protoc util::mysql::LocalMysqlConfiguration::schema_, and util::mysql::LocalMysqlConfiguration::user_.

Referenced by util::mysql::LocalMysqlConfiguration::executeQuery(), util::mysql::LocalMysqlConfiguration::executeUpdate(), and util::mysql::LocalMysqlConfiguration::generatePreparedStatement().

Here is the call graph for this function:



Here is the caller graph for this function:



10.29.3.15 GetSqlDriver()

```
static thread_specific_ptr< MySQL_Driver > & util::mysql::MysqlUtils::GetSqlDriver ()  [inline], [static], [inherited]
```

Get a thread-specific instance of a MySQL driver object.

A new driver will created, if necessary.

Exceptions

log::MessageException	if an error occurs while creating a new thread-specific instance of the SQL driver object

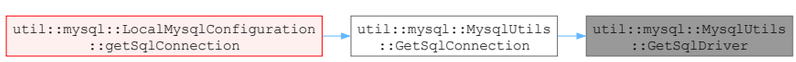
Returns

A thread-specific instance of a MySQL driver object.

References CATCH_BLOCK, and util::mysql::MysqlUtils::sql_driver_.

Referenced by util::mysql::MysqlUtils::GetSqlConnection().

Here is the caller graph for this function:



10.29.3.16 SaveObjectToDatabase()

```
virtual void util::mysql::LocalPersistentEntity::SaveObjectToDatabase () const  [pure virtual]
```

Persist this database entity.

Implemented in argo::iridium::xfloat::XFloat, argo::iridium::xfloat::XFloatDiveData, argo::iridium::xfloat::XFloatMailMessage, argo::iridium::xfloat::XFloatMetadata, and util::xmsg::XGPS.

10.29.4 Member Data Documentation

10.29.4.1 address_

```
std::string util::mysql::LocalMysqlConfiguration::address_  [static], [private], [inherited]
```

The SQL server qualified name or IP address string.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.29.4.2 password_

```
std::string util::mysql::LocalMysqlConfiguration::password_  [static], [private], [inherited]
```

The SQL server account password string.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.29.4.3 port_

```
unsigned int util::mysql::LocalMysqlConfiguration::port_  [static], [private], [inherited]
```

The SQL server port number.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.29.4.4 protocol_

```
std::string util::mysql::LocalMysqlConfiguration::protocol_  [static], [private], [inherited]
```

The protocol to use in the SQL server connection e.g.

"tcp".

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.29.4.5 schema_

```
std::string util::mysql::LocalMysqlConfiguration::schema_  [static], [private], [inherited]
```

The SQL server schema name.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.29.4.6 sql_connection_

```
thread_specific_ptr<Connection> util::mysql::MysqlUtils::sql_connection_  [static], [private],
[inherited]
```

Tthe thread-specific pointer to MySQL connection object.

Thread-specific MySQL connection, released by `MysqlUtils::CleanupSqlConnection()` when the thread exits.

Referenced by util::mysql::MysqlUtils::CloseSqlConnection(), and util::mysql::MysqlUtils::GetSqlConnection().

10.29.4.7 sql_driver_

thread_specific_ptr<MySQL_Driver> util::mysql::MysqlUtils::sql_driver_ [static], [private], [inherited]

The thread-specific pointer to MySQL driver object.

Thread-specific reference to the MySQL driver.

Its cleanup function, MysqlUtils::CleanupSqlDriver(), does not delete the shared driver instance.

Referenced by util::mysql::MysqlUtils::GetSqlDriver().

10.29.4.8 user_

std::string util::mysql::LocalMysqlConfiguration::user_ [static], [private], [inherited]

The SQL server account username string.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

The documentation for this class was generated from the following file:

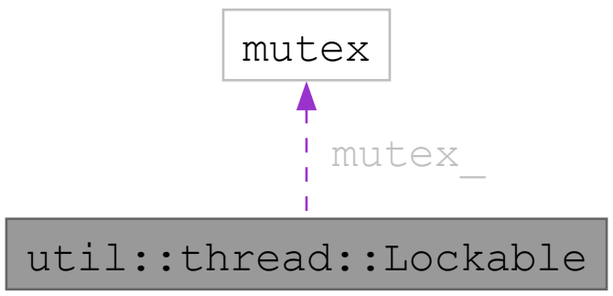
- config/LocalPersistentEntity.h

10.30 util::thread::Lockable Class Reference

A lockable class.

#include <Lockable.h>

Collaboration diagram for util::thread::Lockable:



Public Member Functions

- Lockable ()
  The constructor.
- virtual ~Lockable ()
  The destructor.
- bool LockObject (bool block)
  Lock the object.
- void UnlockObject ()
  Unlock the object.

Protected Attributes

- mutex mutex_

  Mutual exclusion lock for synchronizing operations.

10.30.1 Detailed Description

A lockable class.

Note

The mutex used for locking the object is not recursive, which means that it may be lock at most one time, even by the same thread.

10.30.2 Constructor & Destructor Documentation

10.30.2.1 Lockable()

```
util::thread::Lockable::Lockable ()  [inline]
```

The constructor.

10.30.2.2 ~Lockable()

```
virtual util::thread::Lockable::~Lockable ()  [virtual]
```

The destructor.

10.30.3 Member Function Documentation

10.30.3.1 LockObject()

```
bool util::thread::Lockable::LockObject (
            bool block)  [inline]
```

Lock the object.

Parameters

in	*block*	If TRUE, then the calling thread will block until the lock becomes available. The default value is TRUE.

Exceptions

util::log::MessageException	if an error occurs.

Returns

TRUE if the lock was successfully obtained, or FALSE if block is FALSE and the mutex is already locked.

References CATCH_BLOCK, and mutex_.

10.30.3.2 UnlockObject()

```
void util::thread::Lockable::UnlockObject ()  [inline]
```

Unlock the object.

Exceptions

util::log::MessageException	if an error occurs.

References CATCH_BLOCK, and mutex_.

10.30.4 Member Data Documentation

10.30.4.1 mutex_

```
mutex util::thread::Lockable::mutex_  [mutable], [protected]
```

Mutual exclusion lock for synchronizing operations.

Referenced by LockObject(), and UnlockObject().

The documentation for this class was generated from the following file:

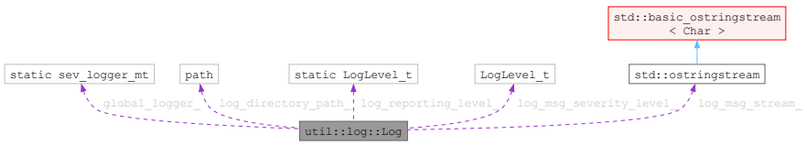
- utility/Lockable.h

10.31 util::log::Log Class Reference

The logger class.

```
#include <Log.h>
```

Collaboration diagram for util::log::Log:



Public Member Functions

- Log (LogLevel_t level) ATTR_NO_EXCEPTIONS
  The default constructor.
- ~Log ()
  The destructor.
- template<typename T = sev_logger_mt>
  ostream & Flush (LogLevel_t lev, T &log=global_logger_) ATTR_NO_EXCEPTIONS
  Flush the log's message buffer buffer.
- template<typename T = sev_logger_mt>
  ostream & Flush (T &log=global_logger_) ATTR_NO_EXCEPTIONS
  Flush the log's message buffer buffer.
- template<typename T >
  ostream & operator<< (const T &x) ATTR_NO_EXCEPTIONS
  The template input stream operator overload function.

Static Public Member Functions

- static void FlushGlobalLogger () ATTR_NO_EXCEPTIONS
  Flush the global logger object.
- static void InitializeLog () ATTR_NO_EXCEPTIONS
  Initialize the global logger.
- static LogLevel_t log_reporting_level ()
  Get the global logger message severity reporting level.

Static Private Member Functions

- static void set_log_reporting_level (LogLevel_t level)
  Set the reporting level of the global logger.

Private Attributes

- LogLevel_t log_msg_severity_level_
  The severity level of this log message.
- ostringstream log_msg_stream_
  The log message output stream.

Static Private Attributes

- static sev_logger_mt global_logger_
  The global severity logger object.
- static path log_directory_path_
  The log directory path.
- static LogLevel_t log_reporting_level_ = DEFAULT_LOG_LEVEL
  The global reporting level.

Friends

- class **config::LocalConfiguration**

10.31.1 Detailed Description

The logger class.

This class is a wrapper for the boost thread-safe severity logger object. It provides facilities for formatted console and file logging and supports filtering by message severity and other attributes.

10.31.2 Constructor & Destructor Documentation

10.31.2.1 Log()

```
util::log::Log::Log (
            LogLevel_t level)  [inline]
```

The default constructor.

Parameters

in	*level*	The message severity level.

10.31.2.2 ~Log()

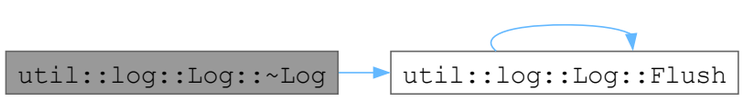
```
util::log::Log::~Log ()  [inline]
```

The destructor.

Flushes the global severity logger's output stream.

References Flush(), and log_msg_severity_level_.

Here is the call graph for this function:



10.31.3 Member Function Documentation

10.31.3.1 Flush() [1/2]

```
template<typename T = sev_logger_mt>
ostream & util::log::Log::Flush (
            LogLevel_t lev,
            T & log = global_logger_)  [inline]
```

Flush the log's message buffer buffer.

Exceptions

None.	Never throws.

Parameters

in	*lev*	The severity level at which to flush the log.
in	*log*	The logger.

Returns

A const reference to this log's output stream.

References exc_diag, util::log::LOG_DBG, util::log::LOG_DBG1, util::log::LOG_DBG2, util::log::LOG_DBG3, util::log::LOG_ERR, util::log::LOG_FAT, util::log::LOG_INF, util::log::LOG_LOG, log_msg_stream_, log_reporting_level_, and util::log::LOG_WRN.

10.31.3.2 Flush() [2/2]

```
template<typename T = sev_logger_mt>
ostream & util::log::Log::Flush (
            T & log = global_logger_)  [inline]
```

Flush the log's message buffer buffer.

Exceptions

None.	Never throws.

Parameters

in	*log*	The logger.

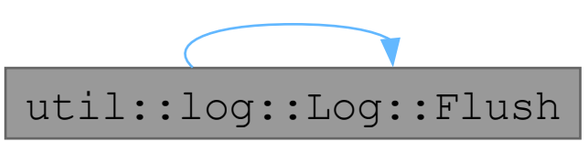
Returns

A const reference to this log's output stream.

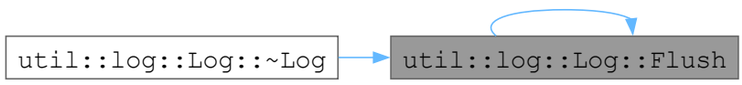
References Flush(), and log_msg_severity_level_.

Referenced by ~Log(), and Flush().

Here is the call graph for this function:



Here is the caller graph for this function:



10.31.3.3 FlushGlobalLogger()

```
static void util::log::Log::FlushGlobalLogger ()  [inline], [static]
```

Flush the global logger object.

Exceptions

None.	Never throws.

10.31.3.4 InitializeLog()

```
static void util::log::Log::InitializeLog ()  [inline], [static]
```

Initialize the global logger.

Adds common attributes to the global logger and defines the format of the log output.

Adds a log file at logPath/Y-m-d.%3N.log that is rotated at midnight (local time) or every time each log file exceeds 10MB.

Also adds a console log.

Adds a float-specific log file.

Exceptions

MessageException	if the directory specified by logPath does not exist, or an error occurs while initializing the global logger.

References CATCH_BLOCK, EXC_THROW, and log_directory_path_.

10.31.3.5 log_reporting_level()

```
static LogLevel_t util::log::Log::log_reporting_level ()  [inline], [static]
```

Get the global logger message severity reporting level.

Returns

The global logger message severity reporting level.

References log_reporting_level_.

10.31.3.6 operator<<()

```
template<typename T >
ostream & util::log::Log::operator<< (
            const T & x)  [inline]
```

The template input stream operator overload function.

Exceptions

None.	Never throws.

Parameters

in	*x*	A const reference to the object invoking the input stream operation. This may be any object that supports the input stream operation.

Returns

A const reference to this log's output stream.

References log_msg_stream_.

10.31.3.7 set_log_reporting_level()

```
static void util::log::Log::set_log_reporting_level (
            LogLevel_t level)  [inline], [static], [private]
```

Set the reporting level of the global logger.

Parameters

in	*level*	The reporting level of the global logger to set.

References log_reporting_level_.

10.31.4 Member Data Documentation

10.31.4.1 global_logger_

```
sev_logger_mt util::log::Log::global_logger_  [static], [private]
```

The global severity logger object.

This object is used as a global logger for all log messages generated by the application. It includes support for log message attributes that are used for filtering and formatting of the log messages. It automatically filters the destination (log files, standard output, etc.) of log messages, and handles daily rotation of log files.

10.31.4.2 log_directory_path_

```
path util::log::Log::log_directory_path_  [static], [private]
```

The log directory path.

Log files generated as an output of this application will be written to this directory as well as sub-directories of this directory.

This field is set as a result of parsing the configuration options during initialization. During initialization, the application checks to ensure that this directory exists, and throws an error if it does not. Thus, during the subsequent duration of execution of the application, this directory is assumed to exist on the local filesystem.

Referenced by InitializeLog().

10.31.4.3 log_msg_severity_level_

```
LogLevel_t util::log::Log::log_msg_severity_level_  [private]
```

The severity level of this log message.

Referenced by ~Log(), and Flush().

10.31.4.4 log_msg_stream_

```
ostringstream util::log::Log::log_msg_stream_  [private]
```

The log message output stream.

Referenced by Flush(), and operator<<().

10.31.4.5 log_reporting_level_

```
LogLevel_t util::log::Log::log_reporting_level_ = DEFAULT_LOG_LEVEL  [static], [private]
```

The global reporting level.

This field is used to filter the log messages by severity. Only log message with a severity that is less than or equal to this value will be reported by the global logger.

This field is set a result of parsing the configuration options during initialization of the application. If the log level is unspecified in the configuration options, then it will default to a value of DEFAULT_LOG_LEVEL.

Referenced by Flush(), log_reporting_level(), and set_log_reporting_level().

The documentation for this class was generated from the following files:

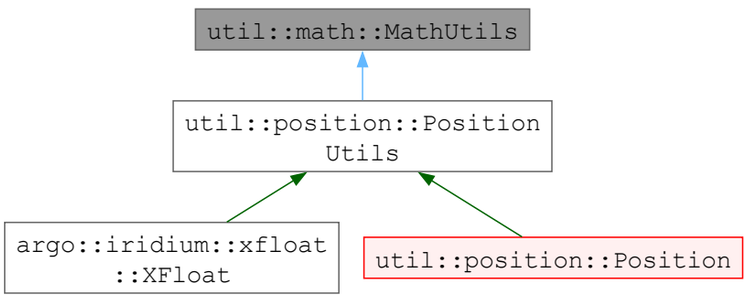
- utility/Log.h
- utility/Log.cpp

10.32 util::math::MathUtils Class Reference

Class containing commonly-used mathematical functions.

`#include <MathUtils.h>`

Inheritance diagram for util::math::MathUtils:



Static Public Member Functions

- static float **bitcastUint32ToFloat** (uint32_t u)
- template<typename T >
  static double degToRad (T deg)

  Convert angular units from from degrees to decimal radians.

10.32.1 Detailed Description

Class containing commonly-used mathematical functions.

10.32.2 Member Function Documentation

10.32.2.1 degToRad()

```
template<typename T >
static double util::math::MathUtils::degToRad (
            T deg)  [inline], [static]
```

Convert angular units from from degrees to decimal radians.

Parameters

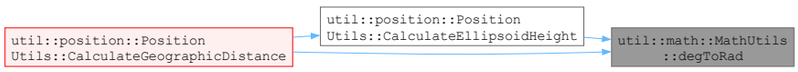
in	*deg*	Angle in integer or decimal degrees to convert to radians.

Returns

Angle in decimal radians.

Referenced by util::position::PositionUtils::CalculateEllipsoidHeight(), and util::position::PositionUtils::CalculateGeographicDistance().

Here is the caller graph for this function:



The documentation for this class was generated from the following file:

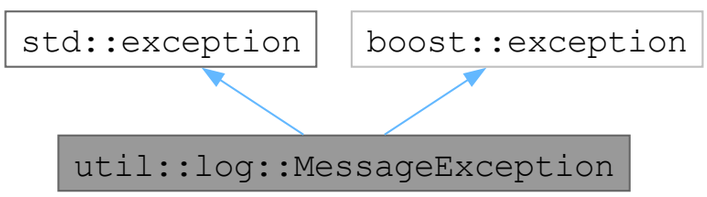
- utility/MathUtils.h

10.33 util::log::MessageException Class Reference

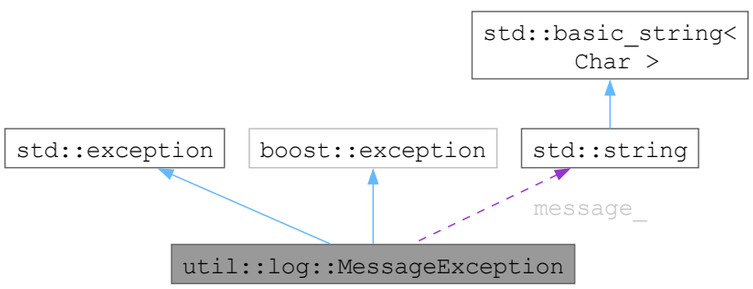
Exception class with associated message.

`#include <MessageException.h>`

Inheritance diagram for util::log::MessageException:



Collaboration diagram for util::log::MessageException:



Public Member Functions

- MessageException ()
  Default Constructor.
- MessageException (const ostringstream &message)
  Parameterized constructor.
- MessageException (const string &message)
  Parameterized constructor.
- virtual ~MessageException () ATTR_NO_EXCEPTIONS
  Destructor.
- virtual const char * what () const ATTR_NO_EXCEPTIONS
  Return C string representing message exception.

Private Attributes

- string message_
  The exception message.

10.33.1 Detailed Description

Exception class with associated message.

10.33.2 Constructor & Destructor Documentation

10.33.2.1 MessageException() [1/3]

```
util::log::MessageException::MessageException ()  [inline]
```

Default Constructor.

10.33.2.2 MessageException() [2/3]

```
util::log::MessageException::MessageException (
            const string & message)  [inline]
```

Parameterized constructor.

Parameters

in	*message*	Exception message string.

10.33.2.3 MessageException() [3/3]

```
util::log::MessageException::MessageException (
            const ostringstream & message)  [inline]
```

Parameterized constructor.

Parameters

in	*message*	Exception message string.

10.33.2.4 ~MessageException()

```
virtual util::log::MessageException::~MessageException ()  [inline], [virtual]
```

Destructor.

10.33.3 Member Function Documentation

10.33.3.1 what()

```
virtual const char * util::log::MessageException::what () const  [inline], [virtual]
```

Return C string representing message exception.

Returns

C string representing message exception.

References message_.

10.33.4 Member Data Documentation

10.33.4.1 message_

```
string util::log::MessageException::message_  [private]
```

The exception message.

Referenced by what().

The documentation for this class was generated from the following file:

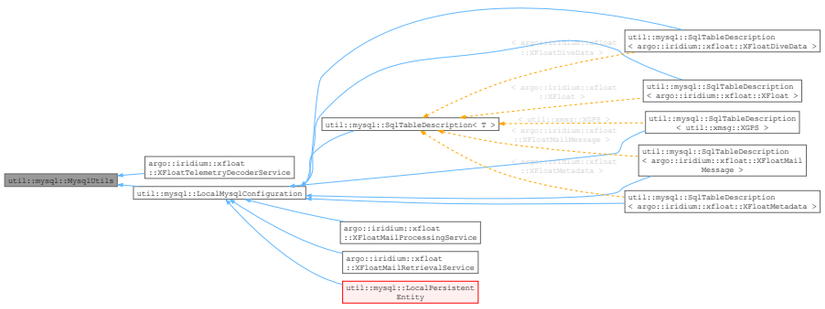
- utility/MessageException.h

10.34 util::mysql::MysqlUtils Class Reference

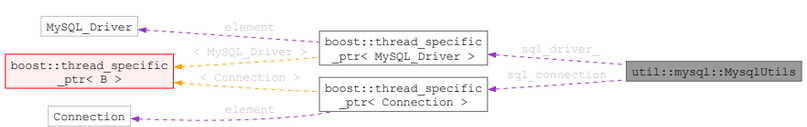
Class provides a library of function for accessing and manipulating a MySQL database.

```
#include <MysqlUtils.h>
```

Inheritance diagram for util::mysql::MysqlUtils:



Collaboration diagram for util::mysql::MysqlUtils:



Static Public Member Functions

- static void CloseSqlConnection ()

  Close the thread-specific MySQL connection if it is currently open.
- static shared_ptr< ResultSet > ExecuteQuery (Connection *con, const string &query)

  Execute an SQL query and return a shared pointer to the ResultSet object.
- static int ExecuteUpdate (Connection *con, const string &update)

  Execute an SQL update and return the status code.
- static shared_ptr< PreparedStatement > GeneratePreparedStatement (Connection *con, const string &statement)

  Generate a prepared SQL statement object for the given SQL statement.
- static string GenerateSqlDeleteStr (const string &from, const string &where="")

  Generate an SQL DELETE statement.
- static string GenerateSqlSelectStr (const string &what, const string &from, const string &where="", const string &groupBy="", const string &orderBy="", unsigned int limit=0, unsigned int offset=0)

  Generate an SQL select statement.
- static thread_specific_ptr< Connection > & GetSqlConnection (string url, string user, string password, string schema)

  Get a thread-specific instance of a MySQL connection object.
- static thread_specific_ptr< MySQL_Driver > & GetSqlDriver ()

  Get a thread-specific instance of a MySQL driver object.

Static Private Member Functions

- static void CleanupSqlConnection (Connection *con)

  Cleanup an SQL connection.
- static void CleanupSqlDriver (MySQL_Driver *driver)

  Function for memory cleanup of MySQL driver on destruct.

Static Private Attributes

- static thread_specific_ptr< Connection > sql_connection_

  Tthe thread-specific pointer to MySQL connection object.
- static thread_specific_ptr< MySQL_Driver > sql_driver_

  The thread-specific pointer to MySQL driver object.

10.34.1 Detailed Description

Class provides a library of function for accessing and manipulating a MySQL database.

10.34.2 Member Function Documentation

10.34.2.1 CleanupSqlConnection()

```
static void util::mysql::MysqlUtils::CleanupSqlConnection (
            Connection * con)  [inline], [static], [private]
```

Cleanup an SQL connection.

This function for memory cleanup of the SQL connection on destruction. The thread-specific SQL connection object will be deleted if it exists.

Parameters

in	*con*	The SQL connection to cleanup.

Exceptions

log::MessageException	if an error occurs while cleaning up the SQL connection object.

References CATCH_BLOCK.

10.34.2.2 CleanupSqlDriver()

```
static void util::mysql::MysqlUtils::CleanupSqlDriver (
            MySQL_Driver * driver)  [inline], [static], [private]
```

Function for memory cleanup of MySQL driver on destruct.

Signals the SQL driver to end operation, if it exists.

Parameters

in	*driver*	Driver to cleanup.

Exceptions

log::MessageException	if an error occurs while cleaning up the SQL driver object.

References CATCH_BLOCK.

10.34.2.3 CloseSqlConnection()

```
static void util::mysql::MysqlUtils::CloseSqlConnection ()  [inline], [static]
```

Close the thread-specific MySQL connection if it is currently open.

If the thread local MySQL connection is not currently open, then this function will have no effect.

Note

The MySQL connection should be closed when idle to prevent the connection from timing out and throwing exceptions.

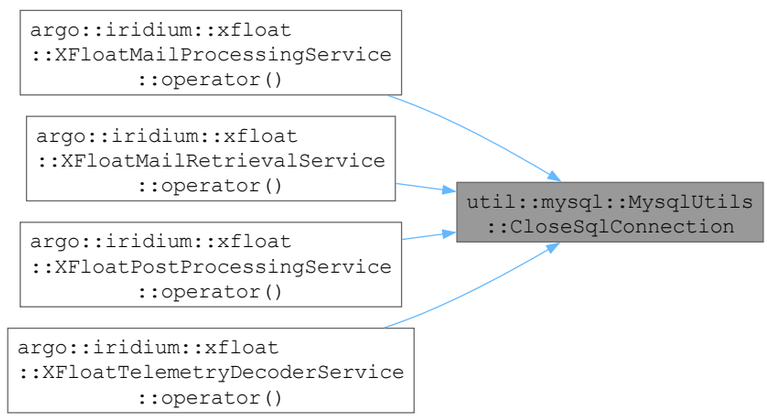
Exceptions

log::MessageException	if an error occurs while attempting to close the MySQL connection.

References CATCH_BLOCK, and sql_connection_.

Referenced by argo::iridium::xfloat::XFloatMailProcessingService::operator()(), argo::iridium::xfloat::XFloatMailRetrievalService::operator(
argo::iridium::xfloat::XFloatPostProcessingService::operator()(), and argo::iridium::xfloat::XFloatTelemetryDecoderService::operator()().

Here is the caller graph for this function:



10.34.2.4 ExecuteQuery()

```
static shared_ptr< ResultSet > util::mysql::MysqlUtils::ExecuteQuery (
            Connection * con,
            const string & query)  [inline], [static]
```

Execute an SQL query and return a shared pointer to the ResultSet object.

Parameters

in	*con*	A pointer to the SQL connection object. This must not be NULL.
in	*query*	The SQL query to execute. This must be a well-formatted SQL query string.

Exceptions

log::MessageException	if 'con' is null, 'query' is a malfored SQL string, or any other error occurs.

Returns

A shared pointer to the query ResultSet object.

References CATCH_BLOCK, and GeneratePreparedStatement().

Referenced by util::mysql::LocalMysqlConfiguration::executeQuery().

Here is the call graph for this function:

Here is the caller graph for this function:

10.34.2.5 ExecuteUpdate()

```
static int util::mysql::MysqlUtils::ExecuteUpdate (
            Connection * con,
            const string & update)  [inline], [static]
```

Execute an SQL update and return the status code.

Parameters

in	*con*	A pointer to the SQL connection object. This must not be NULL.
in	*update*	The SQL update to execute. This must be a well-formatted SQL update string.

Exceptions

log::MessageException	if 'con' is null, 'update' is a malformed SQL string, or any other error occurs.

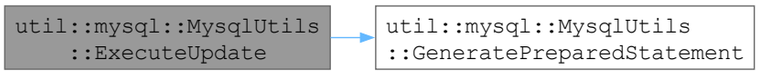
Returns

The update status code.

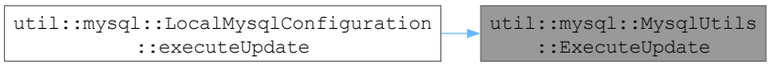
References CATCH_BLOCK, and GeneratePreparedStatement().

Referenced by util::mysql::LocalMysqlConfiguration::executeUpdate().

Here is the call graph for this function:



Here is the caller graph for this function:



10.34.2.6 GeneratePreparedStatement()

```
static shared_ptr< PreparedStatement > util::mysql::MysqlUtils::GeneratePreparedStatement (
            Connection * con,
            const string & statement)  [inline], [static]
```

Generate a prepared SQL statement object for the given SQL statement.

Parameters

in	*con*	A pointer to the SQL connection object. This must not be NULL.
in	*statement*	The SQL statement to prepare. This must be a well-formed SQL string.

Exceptions

log::MessageException	if 'con' is NULL, 'statement' is not a valid SQL string, or if an error occurs.

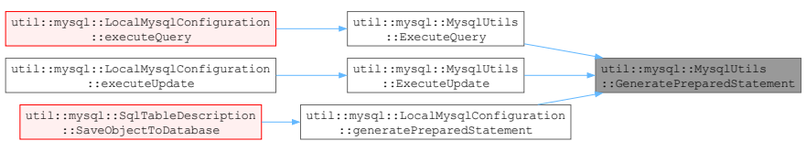
Returns

A prepared SQL statement object generated from the given SQL statement.

References CATCH_BLOCK, and EXC_THROW.

Referenced by ExecuteQuery(), ExecuteUpdate(), and util::mysql::LocalMysqlConfiguration::generatePreparedStatement().

Here is the caller graph for this function:



10.34.2.7 GenerateSqlDeleteStr()

```
static string util::mysql::MysqlUtils::GenerateSqlDeleteStr (
            const string & from,
            const string & where = "")  [inline], [static]
```

Generate an SQL DELETE statement.

Warning

Without a `where` clause, the statement deletes every row in the table.

Parameters

in	*from*	The name of the table to delete from.
in	*where*	The SQL "where-condition" clause (empty = all rows).

Returns

The SQL statement.

10.34.2.8 GenerateSqlSelectStr()

```
static string util::mysql::MysqlUtils::GenerateSqlSelectStr (
            const string & what,
            const string & from,
            const string & where = "",
            const string & groupBy = "",
            const string & orderBy = "",
            unsigned int limit = 0,
            unsigned int offset = 0)  [inline], [static]
```

Generate an SQL select statement.

Parameters

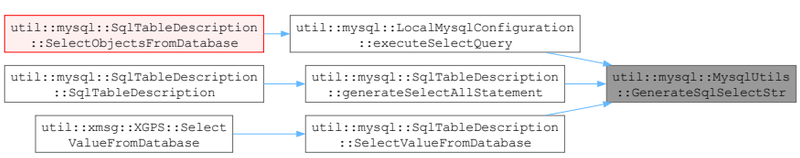
in	*what*	The SQL select statement select-expression clause.
in	*from*	The SQL select statement table-references clause.
in	*where*	The SQL select statement where-condition clause.
in	*groupBy*	The SQL select statement group-by clause.
in	*orderBy*	The SQL select statement order-by clause.
in	*limit*	The SQL select statement result set row limit value.
in	*offset*	The SQL select statement result set row offset.

Returns

The SQL select statement.

Referenced by util::mysql::LocalMysqlConfiguration::executeSelectQuery(), util::mysql::SqlTableDescription< T >::generateSelectAllState
and util::mysql::SqlTableDescription< T >::SelectValueFromDatabase().

Here is the caller graph for this function:



10.34.2.9 GetSqlConnection()

```
static thread_specific_ptr< Connection > & util::mysql::MysqlUtils::GetSqlConnection (
            string url,
            string user,
            string password,
            string schema)  [inline], [static]
```

Get a thread-specific instance of a MySQL connection object.

A new connection will be created, if necessary.

Parameters

in	*url*	The MySQL host server qualified address.
in	*user*	The MySQL account username.
in	*password*	The MySQL account password.
in	*schema*	The MySQL account schema name.

Exceptions

log::MessageException	if an error occurs while creating a new thread-specific instance of the SQL connection object

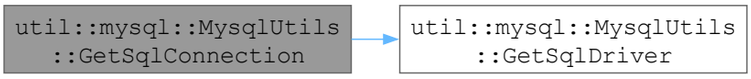
Returns

A thread-specific instance of a MySQL connection object.

References CATCH_BLOCK, GetSqlDriver(), and sql_connection_.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

Here is the call graph for this function:



Here is the caller graph for this function:

10.34.2.10 GetSqlDriver()

```
static thread_specific_ptr< MySQL_Driver > & util::mysql::MysqlUtils::GetSqlDriver ()  [inline],
[static]
```

Get a thread-specific instance of a MySQL driver object.

A new driver will created, if necessary.

Exceptions

log::MessageException	if an error occurs while creating a new thread-specific instance of the SQL driver object

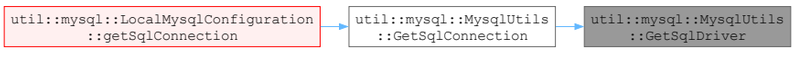
Returns

A thread-specific instance of a MySQL driver object.

References CATCH_BLOCK, and sql_driver_.

Referenced by GetSqlConnection().

Here is the caller graph for this function:



10.34.3 Member Data Documentation

10.34.3.1 sql_connection_

`thread_specific_ptr<Connection> util::mysql::MysqlUtils::sql_connection_ [static], [private]`

Tthe thread-specific pointer to MySQL connection object.

Thread-specific MySQL connection, released by `MysqlUtils::CleanupSqlConnection()` when the thread exits.

Referenced by CloseSqlConnection(), and GetSqlConnection().

10.34.3.2 sql_driver_

`thread_specific_ptr<MySQL_Driver> util::mysql::MysqlUtils::sql_driver_ [static], [private]`

The thread-specific pointer to MySQL driver object.

Thread-specific reference to the MySQL driver.

Its cleanup function, `MysqlUtils::CleanupSqlDriver()`, does not delete the shared driver instance.

Referenced by GetSqlDriver().

The documentation for this class was generated from the following files:

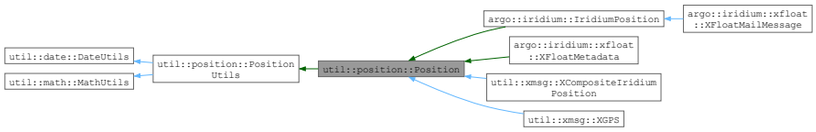
- utility/MysqlUtils.h
- utility/MysqlUtils.cpp

10.35 util::position::Position Class Reference

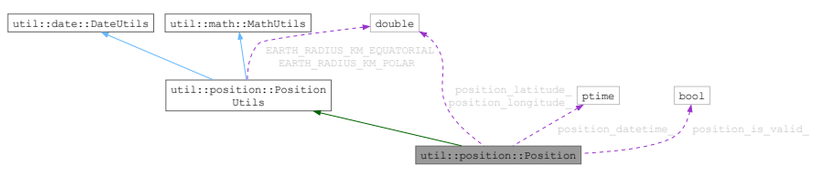
Abstract class representing a position, date, and time.

`#include <Position.h>`

Inheritance diagram for util::position::Position:



Collaboration diagram for util::position::Position:



Public Member Functions

- Position () ATTR_NO_EXCEPTIONS

  The default constructor.
- virtual ~Position ()

  The destructor.
- ptime position_datetime () const

  Get the date/time of the position.
- string position_datetime_as_iso8601_string () const

  Get a formatted string representing the position date/time composed using the ISO 8601 standard.
- bool position_is_valid () const

  Is the position valid?
- double position_latitude () const

  Get the latitude of the position, in decimal degrees.
- double position_longitude () const

  Get the longitude of the position, in decimal degrees.
- string postition_as_iso6709_string () const

  Get a formatted string representing the position composed using the ISO 6709 standard.

Protected Member Functions

- void set_gps_position_datetime (unsigned int gpsWeek, unsigned int gpsDay, unsigned int gpsHour=0, unsigned int gpsMinute=0, unsigned int gpsSecond=0)

  Set the date and time of this position from the given GPS week, day, and time.
- void set_position_datetime (const ptime &datetime)

  Set the date/time of the position.
- void set_position_is_valid (bool is_valid=true)

  Set the position is valid flag.
- void set_position_latitude (double lat)

  Set the latitude of the position, in decimal degrees.
- void set_position_longitude (double lon)

  Set the longitude of the position, in decimal degrees.

Static Protected Member Functions

- static float **bitcastUint32ToFloat** (uint32_t u)
- template<typename T >
  static double CalculateEllipsoidHeight (T lat)

  Get the approximate surface altitude or height of earth for a given latitude, in kilometers.
- template<typename T >
  static double CalculateGeographicDistance (T lat1, T lon1, T lat0, T lon0)

  Get the geographical distance between two position, in kilometers, from the positions' locations, in decimal degrees.
- template<typename T >
  static double degToRad (T deg)

  Convert angular units from from degrees to decimal radians.
- static string GenerateCurrentUtcTimeIso8601String ()

  Generate an ISO8601 format date/time string representation of the current time, in UTC.
- template<typename T >
  static std::string GenerateIso6709String (T lat, T lon)

  Generate an IS06709 string representation of the position given by (`lat`, `lon`).
- static string GenerateIso8601String (const ptime &p, bool extended=true)

  Generate an ISO8601 format date/time string representation of a POSIX time object.
- static double GenerateJulianDay (const ptime &dt, const ptime &ref_dt)

  Convert a date/time to fractional days since a reference date/time.
- static ptime GetCurrentUtcTime ()

  Generate a POSIX time object representing the current date and time (rounded to the nearest second).
- template<typename T >
  static bool IsPositionValid (T lat, T lon)

  Check position validity.

Private Attributes

- ptime position_datetime_

  The date and time of the position.
- bool position_is_valid_

  The valid position indicator.
- double position_latitude_

  The latitude of the position, in decimal degrees.
- double position_longitude_

  The longitude of the position, in decimal degrees.

Static Private Attributes

- static const double EARTH_RADIUS_KM_EQUATORIAL = 6378.1

  The equatorial radius of the earth, in kilometers.
- static const double EARTH_RADIUS_KM_POLAR = 6356.1

  The polar radius of the earth, in kilometers.

10.35.1 Detailed Description

Abstract class representing a position, date, and time.

10.35.2 Constructor & Destructor Documentation

10.35.2.1 Position()

```
util::position::Position::Position ()  [inline]
```

The default constructor.

Exceptions

None.	Never throws.

10.35.2.2 ~Position()

```
virtual util::position::Position::~Position ()  [inline], [virtual]
```

The destructor.

10.35.3 Member Function Documentation

10.35.3.1 CalculateEllipsoidHeight()

```
template<typename T >
static double util::position::PositionUtils::CalculateEllipsoidHeight (
            T lat)  [inline], [static], [inherited]
```

Get the approximate surface altitude or height of earth for a given latitude, in kilometers.

Warning

This method provides approximate surface height estimates only! This approximation is calculated from a reference ellipsoid – a mathematically-defined surface that approximates the geoid, the truer figure of the Earth.

Parameters

in	*lat*	The geographic latitude of interest, in decimal degrees.

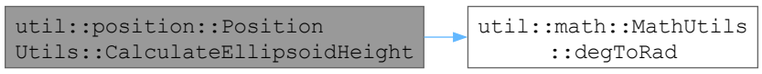
Returns

The approximate surface altitude or height of earth for the given latitude, in kilometers.

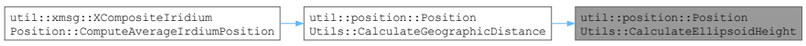
References util::math::MathUtils::degToRad(), util::position::PositionUtils::EARTH_RADIUS_KM_EQUATORIAL, and util::position::PositionUtils::EARTH_RADIUS_KM_POLAR.

Referenced by util::position::PositionUtils::CalculateGeographicDistance().

Here is the call graph for this function:



Here is the caller graph for this function:



10.35.3.2 CalculateGeographicDistance()

```
template<typename T >
static double util::position::PositionUtils::CalculateGeographicDistance (
            T lat1,
            T lon1,
            T lat0,
            T lon0)  [inline], [static], [inherited]
```

Get the geographical distance between two position, in kilometers, from the positions' locations, in decimal degrees.

Warning

This method provides approximate geographic estimates only! This approximation is calculated from a reference ellipsoid – a mathematically-defined surface that approximates the geoid, the truer figure of the Earth.

Note

This method uses the 'haversine' formula to calculate the great-circle distance between two points – that is, the shortest distance over the earth's surface – giving an 'as-the-crow-flies' distance between the points (ignoring any hills they fly over, of course!).

Parameters

in	*lat1*	The latitude of the geographic final position, in decimal degrees North.
in	*lon1*	The longitude of the geographic final position, in decimal degrees East.
in	*lat0*	The latitude of the geographic initial position, in decimal degrees North.
in	*lon0*	The longitude of the geographic initial position, in decimal degrees East.

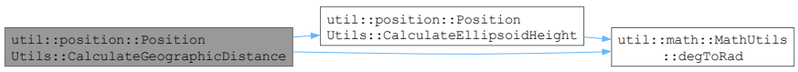
Returns

The geographical distance between the two given positions, in kilometers.

References util::position::PositionUtils::CalculateEllipsoidHeight(), and util::math::MathUtils::degToRad().

Referenced by util::xmsg::XCompositeIridiumPosition::ComputeAverageIrdiumPosition().

Here is the call graph for this function:



Here is the caller graph for this function:

10.35.3.3 degToRad()

```
template<typename T >
static double util::math::MathUtils::degToRad (
            T deg)  [inline], [static], [inherited]
```

Convert angular units from from degrees to decimal radians.

Parameters

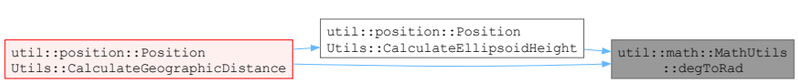
in	*deg*	Angle in integer or decimal degrees to convert to radians.

Returns

Angle in decimal radians.

Referenced by util::position::PositionUtils::CalculateEllipsoidHeight(), and util::position::PositionUtils::CalculateGeographicDistance().

Here is the caller graph for this function:



10.35.3.4 GenerateCurrentUtcTimeIso8601String()

```
static string util::date::DateUtils::GenerateCurrentUtcTimeIso8601String ()  [inline], [static],
[inherited]
```

Generate an ISO8601 format date/time string representation of the current time, in UTC.

Note

All date/time objects use the UTC time-zone.

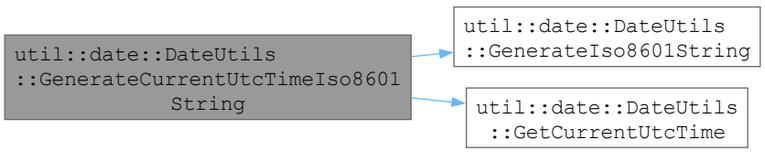
Returns

An ISO8601 format date/time string representation of the current time.

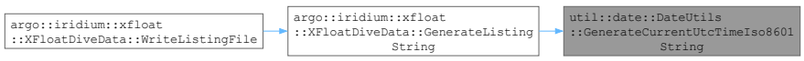
References util::date::DateUtils::GenerateIso8601String(), and util::date::DateUtils::GetCurrentUtcTime().

Referenced by argo::iridium::xfloat::XFloatDiveData::GenerateListingString().

Here is the call graph for this function:



Here is the caller graph for this function:



10.35.3.5 GenerateIso6709String()

```
template<typename T >
static std::string util::position::PositionUtils::GenerateIso6709String (
            T lat,
            T lon)  [inline], [static], [inherited]
```

Generate an IS06709 string representation of the position given by (`lat`, `lon`).

Parameters

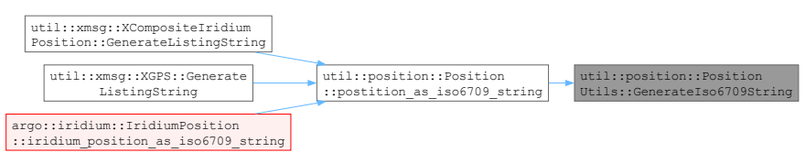
in	*lat*	The latitude of the position, in degrees North.
in	*lon*	The longitude of the position, in degrees East.

Returns

An IS06709 string representation of the position given by (`lat`, `lon`).

Referenced by postition_as_iso6709_string().

Here is the caller graph for this function:



10.35.3.6 GenerateIso8601String()

```
static string util::date::DateUtils::GenerateIso8601String (
            const ptime & p,
            bool extended = true)  [inline], [static], [inherited]
```

Generate an ISO8601 format date/time string representation of a POSIX time object.

Note

All date/time objects use the UTC time-zone.

Parameters

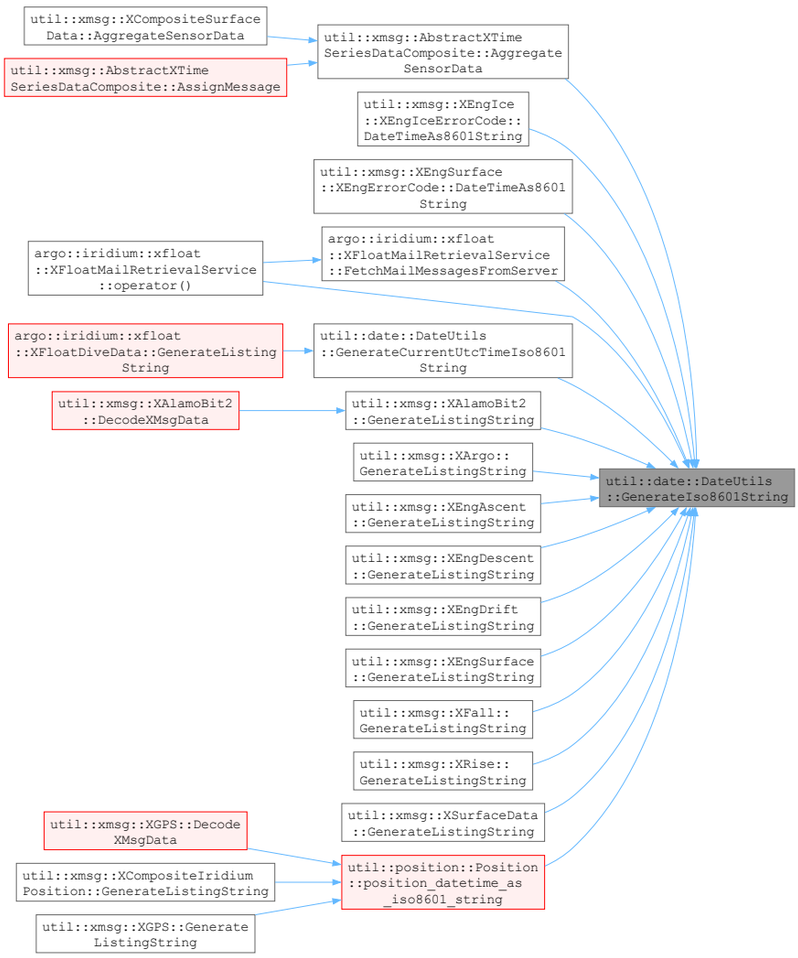
in	*p*	POSIX time from which to generate string.
in	*extended*	If TRUE, use the ISO 8601 extended format (YYYY-MM-DDThh:mm:ssZ); otherwise, use the basic format (YYYYMMDDThhmmss).

Returns

An ISO8601 format date/time string representation of the POSIX time object. If p is equal to boost::posix_↩ time::not_a_date_time, then the string "0000-00-00T00:00:00Z" (extended) or "00000000000000" (basic) will be returned.

Referenced by util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::AggregateSensorData(), util::xmsg::XEngIce::XEngIce util::xmsg::XEngSurface::XEngErrorCode::DateTimeAs8601String(), argo::iridium::xfloat::XFloatMailRetrievalService::FetchMailMessages util::date::DateUtils::GenerateCurrentUtcTimeIso8601String(), util::xmsg::XAlamoBit2::GenerateListingString(), util::xmsg::XArgo::Genera util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListing util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(), argo::iridium::xfloat::XFloatMailRetrievalService::operator()(), and position_datetime_as_iso8601_string().

Here is the caller graph for this function:



10.35.3.7 GenerateJulianDay()

```
static double util::date::DateUtils::GenerateJulianDay (
            const ptime & dt,
            const ptime & ref_dt)  [inline], [static], [inherited]
```

Convert a date/time to fractional days since a reference date/time.

Parameters

in	*dt*	The date/time to convert.
in	*ref_dt*	The reference date/time (day 0).

Exceptions

util::log::MessageException	if either date/time is not valid.

Returns

The number of days (with millisecond resolution) from `ref_dt` to `dt`.

References CATCH_BLOCK, and EXC_THROW.

10.35.3.8 GetCurrentUtcTime()

```
static ptime util::date::DateUtils::GetCurrentUtcTime ()  [inline], [static], [inherited]
```

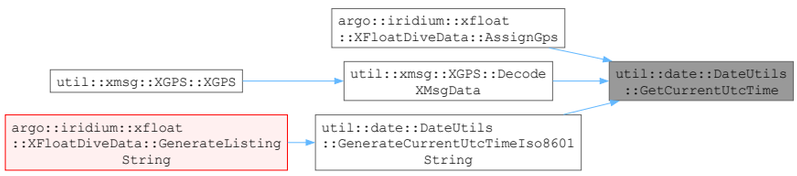
Generate a POSIX time object representing the current date and time (rounded to the nearest second).

Returns

A POSIX time object representing the current date and time (rounded to the nearest second).

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignGps(), util::xmsg::XGPS::DecodeXMsgData(), and util::date::DateUtils::GenerateCurrentUtcTimeIso8601String().

Here is the caller graph for this function:



10.35.3.9 IsPositionValid()

```
template<typename T >
static bool util::position::PositionUtils::IsPositionValid (
            T lat,
            T lon)  [inline], [static], [inherited]
```

Check position validity.

Parameters

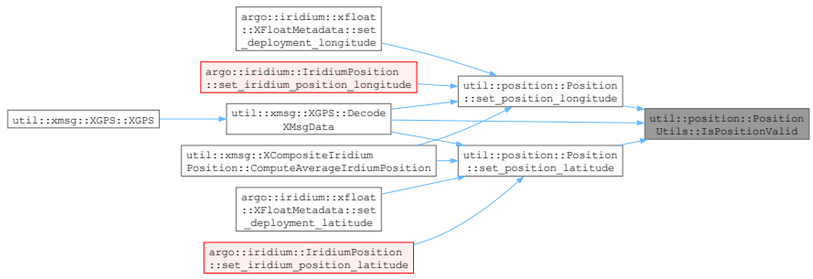
in	*lat*	Latitude of position.
in	*lon*	Longitude of position.

Returns

TRUE if (lat, lon) represents a valid geographical position, or FALSE otherwise.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), set_position_latitude(), and set_position_longitude().

Here is the caller graph for this function:



10.35.3.10 position_datetime()

```
ptime util::position::Position::position_datetime () const  [inline]
```

Get the date/time of the position.

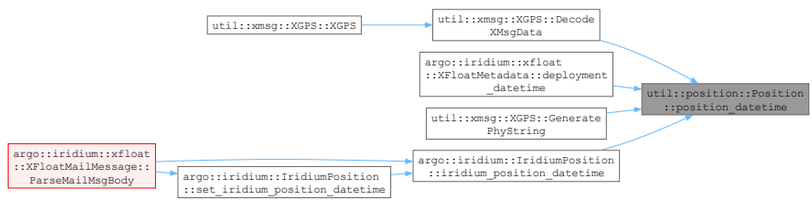
Returns

The date/time of the position.

References position_datetime_.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), argo::iridium::xfloat::XFloatMetadata::deployment_datetime(), util::xmsg::XGPS::GeneratePhyString(), and argo::iridium::IridiumPosition::iridium_position_datetime().

Here is the caller graph for this function:



10.35.3.11 position_datetime_as_iso8601_string()

```
string util::position::Position::position_datetime_as_iso8601_string () const  [inline]
```

Get a formatted string representing the position date/time composed using the ISO 8601 standard.

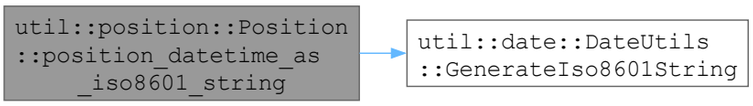
Returns

The formatted string representation of the position date/time.

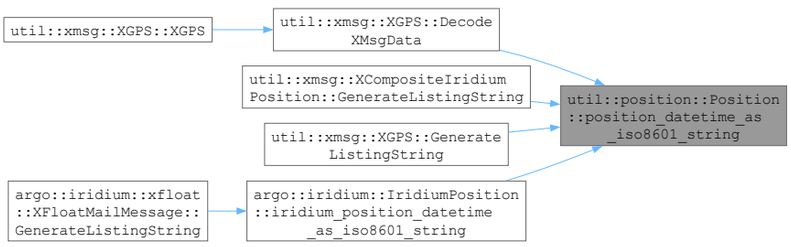
References util::date::DateUtils::GenerateIso8601String(), and position_datetime_.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XCompositeIridiumPosition::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), and argo::iridium::IridiumPosition::iridium_position_datetime_as_iso8601_string().

Here is the call graph for this function:



Here is the caller graph for this function:



10.35.3.12 position_is_valid()

```
bool util::position::Position::position_is_valid () const  [inline]
```

Is the position valid?

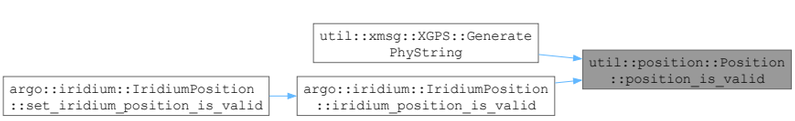
Returns

TRUE if the position is valid, or false otherwise.

References position_is_valid_.

Referenced by util::xmsg::XGPS::GeneratePhyString(), and argo::iridium::IridiumPosition::iridium_position_is_valid().

Here is the caller graph for this function:



10.35.3.13 position_latitude()

```
double util::position::Position::position_latitude () const  [inline]
```

Get the latitude of the position, in decimal degrees.

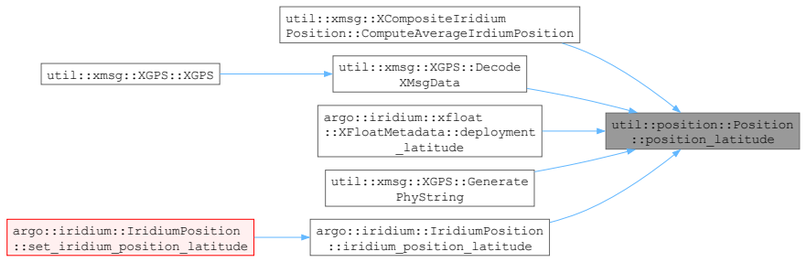
Returns

The latitude of the position, in decimal degrees.

References position_latitude_.

Referenced by util::xmsg::XCompositeIridiumPosition::ComputeAverageIrdiumPosition(), util::xmsg::XGPS::DecodeXMsgData(), argo::iridium::xfloat::XFloatMetadata::deployment_latitude(), util::xmsg::XGPS::GeneratePhyString(), and argo::iridium::IridiumPosition::iri

Here is the caller graph for this function:



10.35.3.14 position_longitude()

```
double util::position::Position::position_longitude () const  [inline]
```

Get the longitude of the position, in decimal degrees.

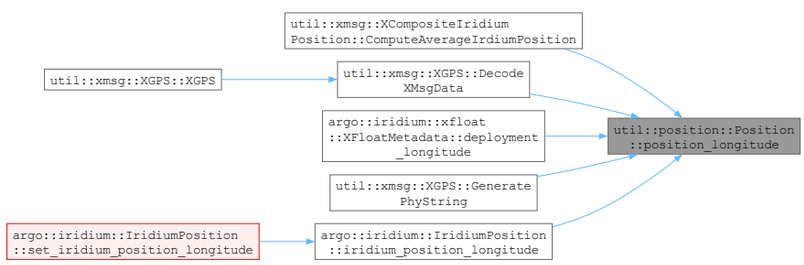
Returns

The longitude of the position, in decimal degrees.

References position_longitude_.

Referenced by util::xmsg::XCompositeIridiumPosition::ComputeAverageIrdiumPosition(), util::xmsg::XGPS::DecodeXMsgData(), argo::iridium::xfloat::XFloatMetadata::deployment_longitude(), util::xmsg::XGPS::GeneratePhyString(), and argo::iridium::IridiumPosition::

Here is the caller graph for this function:



10.35.3.15 postition_as_iso6709_string()

```
string util::position::Position::postition_as_iso6709_string () const  [inline]
```

Get a formatted string representing the position composed using the ISO 6709 standard.

Returns

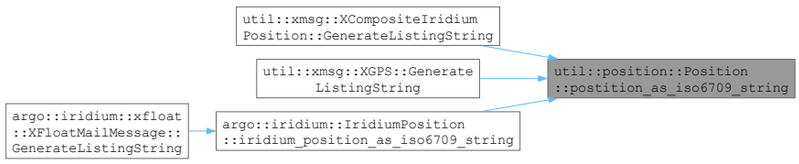
The formatted string representation of the position.

References util::position::PositionUtils::GenerateIso6709String(), position_latitude_, and position_longitude_.

Referenced by util::xmsg::XCompositeIridiumPosition::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), and argo::iridium::IridiumPosition::iridium_position_as_iso6709_string().

Here is the call graph for this function:

Here is the caller graph for this function:



10.35.3.16 set_gps_position_datetime()

```
void util::position::Position::set_gps_position_datetime (
            unsigned int gpsWeek,
            unsigned int gpsDay,
            unsigned int gpsHour = 0,
            unsigned int gpsMinute = 0,
            unsigned int gpsSecond = 0)  [inline], [protected]
```

Set the date and time of this position from the given GPS week, day, and time.

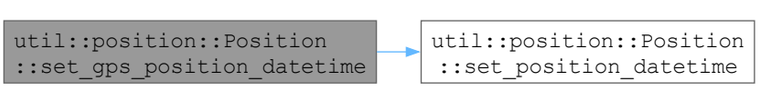
Parameters

in	*gpsWeek*	The GPS week, in weeks since midnight of January 6, 1980.
in	*gpsDay*	The GPS day of week (0 - 6).
in	*gpsHour*	The UTC hour of day (0 - 23).
in	*gpsMinute*	The UTC minute of hour (0 - 59)..
in	*gpsSecond*	The UTC second of minute (0 - 59).

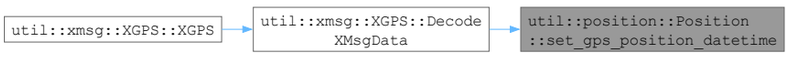
References set_position_datetime().

Referenced by util::xmsg::XGPS::DecodeXMsgData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.35.3.17 set_position_datetime()

```
void util::position::Position::set_position_datetime (
            const ptime & datetime)  [inline], [protected]
```

Set the date/time of the position.

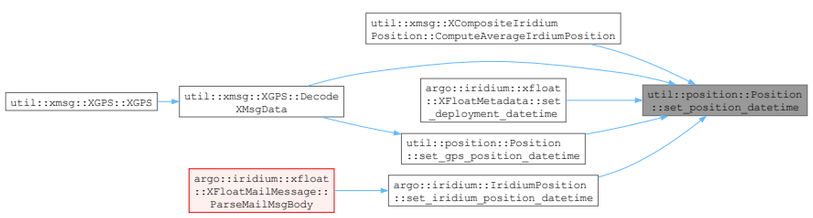
Parameters

in	*datetime*	The date/time of the position to set.

References position_datetime_.

Referenced by util::xmsg::XCompositeIridiumPosition::ComputeAverageIrdiumPosition(), util::xmsg::XGPS::DecodeXMsgData(), argo::iridium::xfloat::XFloatMetadata::set_deployment_datetime(), set_gps_position_datetime(), and argo::iridium::IridiumPosition::set_iric

Here is the caller graph for this function:



10.35.3.18 set_position_is_valid()

```
void util::position::Position::set_position_is_valid (
            bool is_valid = true)  [inline], [protected]
```

Set the position is valid flag.

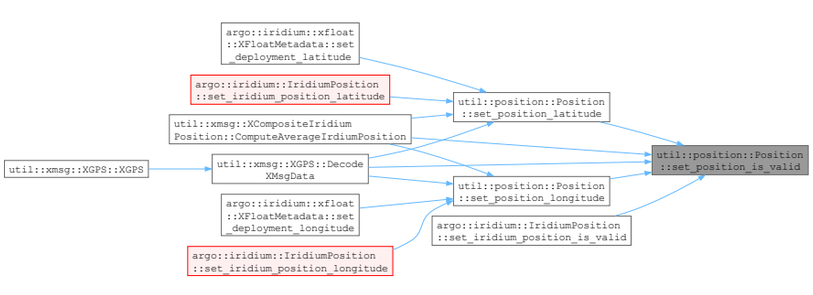
Parameters

in	*is_valid*	TRUE if the position is valid, or FALSE otherwise.

References position_is_valid_.

Referenced by util::xmsg::XCompositeIridiumPosition::ComputeAverageIrdiumPosition(), util::xmsg::XGPS::DecodeXMsgData(), argo::iridium::IridiumPosition::set_iridium_position_is_valid(), set_position_latitude(), and set_position_longitude().

Here is the caller graph for this function:



10.35.3.19 set_position_latitude()

```
void util::position::Position::set_position_latitude (
            double lat)  [inline], [protected]
```

Set the latitude of the position, in decimal degrees.

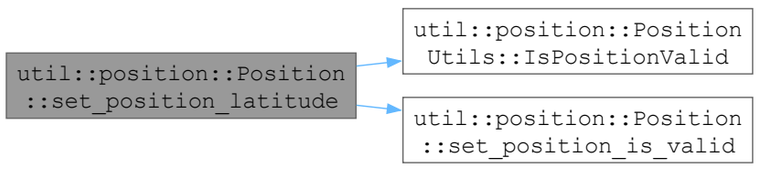
Parameters

in	*lat*	The latitude of the position, in decimal degrees.

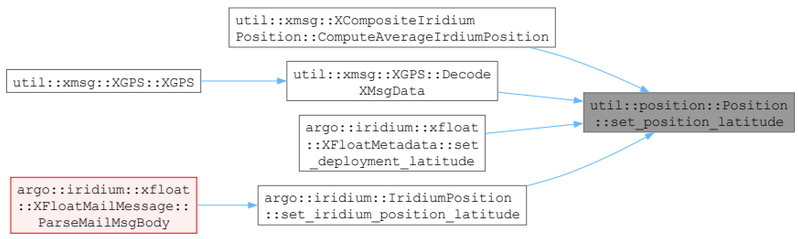
References util::position::PositionUtils::IsPositionValid(), position_latitude_, position_longitude_, and set_position_is_valid().

Referenced by util::xmsg::XCompositeIridiumPosition::ComputeAverageIrdiumPosition(), util::xmsg::XGPS::DecodeXMsgData(), argo::iridium::xfloat::XFloatMetadata::set_deployment_latitude(), and argo::iridium::IridiumPosition::set_iridium_position_latitude().

Here is the call graph for this function:



Here is the caller graph for this function:



10.35.3.20 set_position_longitude()

```
void util::position::Position::set_position_longitude (
            double lon)  [inline], [protected]
```

Set the longitude of the position, in decimal degrees.

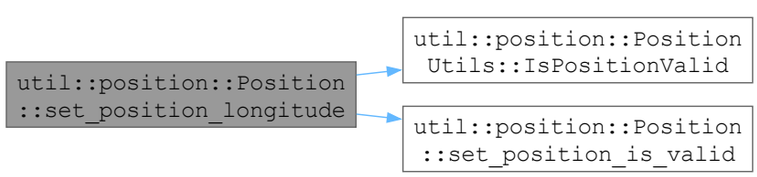
Parameters

in	*lon*	The longitude of the position, in decimal degrees.

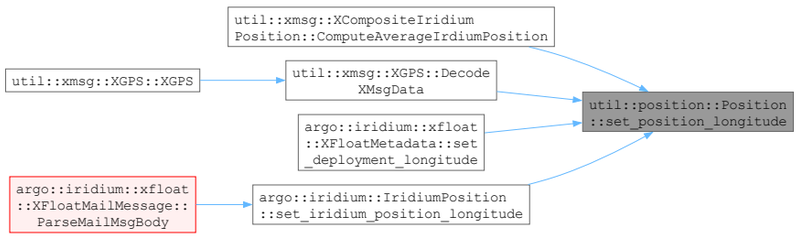
References util::position::PositionUtils::IsPositionValid(), position_latitude_, position_longitude_, and set_position_is_valid().

Referenced by util::xmsg::XCompositeIridiumPosition::ComputeAverageIrdiumPosition(), util::xmsg::XGPS::DecodeXMsgData(), argo::iridium::xfloat::XFloatMetadata::set_deployment_longitude(), and argo::iridium::IridiumPosition::set_iridium_position_longitude().

Here is the call graph for this function:



Here is the caller graph for this function:



10.35.4 Member Data Documentation

10.35.4.1 EARTH_RADIUS_KM_EQUATORIAL

```
const double util::position::PositionUtils::EARTH_RADIUS_KM_EQUATORIAL = 6378.1  [static], [private],
[inherited]
```

The equatorial radius of the earth, in kilometers.

Referenced by util::position::PositionUtils::CalculateEllipsoidHeight().

10.35.4.2 EARTH_RADIUS_KM_POLAR

```
const double util::position::PositionUtils::EARTH_RADIUS_KM_POLAR = 6356.1  [static], [private],
[inherited]
```

The polar radius of the earth, in kilometers.

Referenced by util::position::PositionUtils::CalculateEllipsoidHeight().

10.35.4.3 position_datetime_

```
ptime util::position::Position::position_datetime_  [private]
```

The date and time of the position.

Referenced by position_datetime(), position_datetime_as_iso8601_string(), and set_position_datetime().

10.35.4.4 position_is_valid_

```
bool util::position::Position::position_is_valid_  [private]
```

The valid position indicator.

Referenced by position_is_valid(), and set_position_is_valid().

10.35.4.5 position_latitude_

```
double util::position::Position::position_latitude_  [private]
```

The latitude of the position, in decimal degrees.

Referenced by position_latitude(), postition_as_iso6709_string(), set_position_latitude(), and set_position_longitude().

10.35.4.6 position_longitude_

```
double util::position::Position::position_longitude_  [private]
```

The longitude of the position, in decimal degrees.

Referenced by position_longitude(), postition_as_iso6709_string(), set_position_latitude(), and set_position_longitude().

The documentation for this class was generated from the following file:

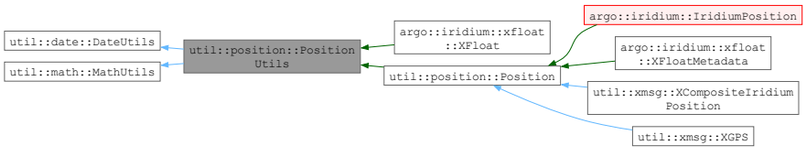
- utility/Position.h

10.36 util::position::PositionUtils Class Reference

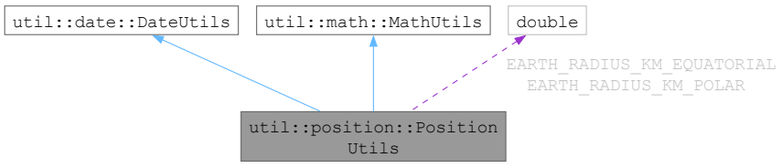
Class providing library of useful functions for manipulating and processing geographic positions.

```
#include <PositionUtils.h>
```

Inheritance diagram for util::position::PositionUtils:



Collaboration diagram for util::position::PositionUtils:



Static Public Member Functions

- static float **bitcastUint32ToFloat** (uint32_t u)
- template<typename T >
  static double CalculateEllipsoidHeight (T lat)

  Get the approximate surface altitude or height of earth for a given latitude, in kilometers.
- template<typename T >
  static double CalculateGeographicDistance (T lat1, T lon1, T lat0, T lon0)

  Get the geographical distance between two position, in kilometers, from the positions' locations, in decimal degrees.
- template<typename T >
  static double degToRad (T deg)

  Convert angular units from from degrees to decimal radians.
- static string GenerateCurrentUtcTimeIso8601String ()

  Generate an ISO8601 format date/time string representation of the current time, in UTC.
- template<typename T >
  static std::string GenerateIso6709String (T lat, T lon)

  Generate an IS06709 string representation of the position given by (`lat, lon`).
- static string GenerateIso8601String (const ptime &p, bool extended=true)

  Generate an ISO8601 format date/time string representation of a POSIX time object.
- static double GenerateJulianDay (const ptime &dt, const ptime &ref_dt)

  Convert a date/time to fractional days since a reference date/time.
- static ptime GetCurrentUtcTime ()

  Generate a POSIX time object representing the current date and time (rounded to the nearest second).
- template<typename T >
  static bool IsPositionValid (T lat, T lon)

  Check position validity.

Static Private Attributes

- static const double EARTH_RADIUS_KM_EQUATORIAL = 6378.1

  The equatorial radius of the earth, in kilometers.
- static const double EARTH_RADIUS_KM_POLAR = 6356.1

  The polar radius of the earth, in kilometers.

10.36.1 Detailed Description

Class providing library of useful functions for manipulating and processing geographic positions.

10.36.2 Member Function Documentation

10.36.2.1 CalculateEllipsoidHeight()

```
template<typename T >
static double util::position::PositionUtils::CalculateEllipsoidHeight (
            T lat)  [inline], [static]
```

Get the approximate surface altitude or height of earth for a given latitude, in kilometers.

Warning

This method provides approximate surface height estimates only! This approximation is calculated from a reference ellipsoid – a mathematically-defined surface that approximates the geoid, the truer figure of the Earth.

Parameters

in	*lat*	The geographic latitude of interest, in decimal degrees.

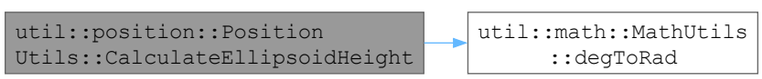
Returns

The approximate surface altitude or height of earth for the given latitude, in kilometers.

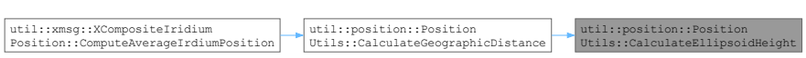
References util::math::MathUtils::degToRad(), EARTH_RADIUS_KM_EQUATORIAL, and EARTH_RADIUS_KM_POLAR.

Referenced by CalculateGeographicDistance().

Here is the call graph for this function:



Here is the caller graph for this function:



10.36.2.2 CalculateGeographicDistance()

```
template<typename T >
static double util::position::PositionUtils::CalculateGeographicDistance (
            T lat1,
            T lon1,
            T lat0,
            T lon0)  [inline], [static]
```

Get the geographical distance between two position, in kilometers, from the positions' locations, in decimal degrees.

Warning

This method provides approximate geographic estimates only! This approximation is calculated from a reference ellipsoid – a mathematically-defined surface that approximates the geoid, the truer figure of the Earth.

Note

This method uses the 'haversine' formula to calculate the great-circle distance between two points – that is, the shortest distance over the earth's surface – giving an 'as-the-crow-flies' distance between the points (ignoring any hills they fly over, of course!).

Parameters

in	*lat1*	The latitude of the geographic final position, in decimal degrees North.
in	*lon1*	The longitude of the geographic final position, in decimal degrees East.
in	*lat0*	The latitude of the geographic initial position, in decimal degrees North.
in	*lon0*	The longitude of the geographic initial position, in decimal degrees East.

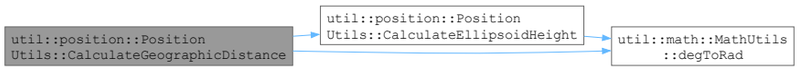
Returns

The geographical distance between the two given positions, in kilometers.

References CalculateEllipsoidHeight(), and util::math::MathUtils::degToRad().

Referenced by util::xmsg::XCompositeIridiumPosition::ComputeAverageIrdiumPosition().

Here is the call graph for this function:



Here is the caller graph for this function:

10.36.2.3 degToRad()

```
template<typename T >
static double util::math::MathUtils::degToRad (
            T deg)  [inline], [static], [inherited]
```

Convert angular units from from degrees to decimal radians.

Parameters

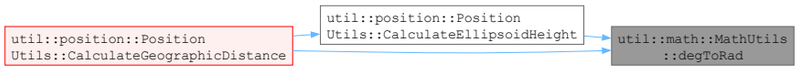
in	*deg*	Angle in integer or decimal degrees to convert to radians.

Returns

Angle in decimal radians.

Referenced by CalculateEllipsoidHeight(), and CalculateGeographicDistance().

Here is the caller graph for this function:



10.36.2.4 GenerateCurrentUtcTimeIso8601String()

```
static string util::date::DateUtils::GenerateCurrentUtcTimeIso8601String ()  [inline], [static],
[inherited]
```

Generate an ISO8601 format date/time string representation of the current time, in UTC.

Note

All date/time objects use the UTC time-zone.

Returns

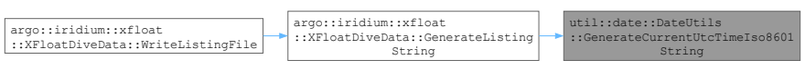
An ISO8601 format date/time string representation of the current time.

References util::date::DateUtils::GenerateIso8601String(), and util::date::DateUtils::GetCurrentUtcTime().

Referenced by argo::iridium::xfloat::XFloatDiveData::GenerateListingString().

Here is the call graph for this function:

Here is the caller graph for this function:



10.36.2.5 GenerateIso6709String()

```
template<typename T >
static std::string util::position::PositionUtils::GenerateIso6709String (
            T lat,
            T lon)  [inline], [static]
```

Generate an IS06709 string representation of the position given by (`lat`, `lon`).

Parameters

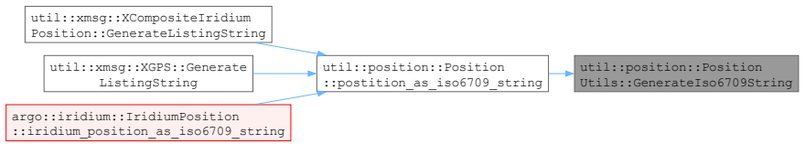
in	*lat*	The latitude of the position, in degrees North.
in	*lon*	The longitude of the position, in degrees East.

Returns

An IS06709 string representation of the position given by (`lat`, `lon`).

Referenced by util::position::Position::postition_as_iso6709_string().

Here is the caller graph for this function:



10.36.2.6 GenerateIso8601String()

```
static string util::date::DateUtils::GenerateIso8601String (
            const ptime & p,
            bool extended = true)  [inline], [static], [inherited]
```

Generate an ISO8601 format date/time string representation of a POSIX time object.

Note

All date/time objects use the UTC time-zone.

Parameters

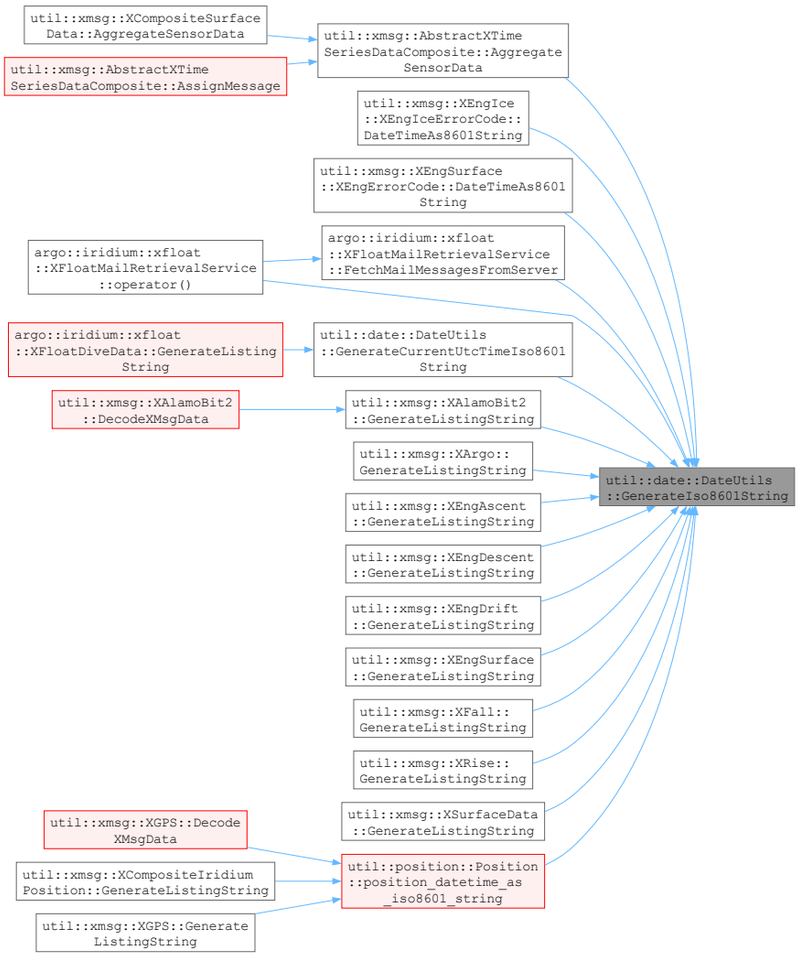
in	*p*	POSIX time from which to generate string.
in	*extended*	If TRUE, use the ISO 8601 extended format (YYYY-MM-DDThh:mm:ssZ); otherwise, use the basic format (YYYYMMDDThhmmss).

Returns

An ISO8601 format date/time string representation of the POSIX time object. If p is equal to boost::posix_↩ time::not_a_date_time, then the string "0000-00-00T00:00:00Z" (extended) or "00000000000000" (basic) will be returned.

Referenced by util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::AggregateSensorData(), util::xmsg::XEngIce::XEngIce
util::xmsg::XEngSurface::XEngErrorCode::DateTimeAs8601String(), argo::iridium::xfloat::XFloatMailRetrievalService::FetchMailMessages
util::date::DateUtils::GenerateCurrentUtcTimeIso8601String(), util::xmsg::XAlamoBit2::GenerateListingString(), util::xmsg::XArgo::Genera
util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListing
util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XRise::GenerateListingString(),
util::xmsg::XSurfaceData::GenerateListingString(), argo::iridium::xfloat::XFloatMailRetrievalService::operator()(), and
util::position::Position::position_datetime_as_iso8601_string().

Here is the caller graph for this function:



10.36.2.7 GenerateJulianDay()

```
static double util::date::DateUtils::GenerateJulianDay (
            const ptime & dt,
            const ptime & ref_dt)  [inline], [static], [inherited]
```

Convert a date/time to fractional days since a reference date/time.

Parameters

in	*dt*	The date/time to convert.
in	*ref↩ _dt*	The reference date/time (day 0).

Exceptions

util::log::MessageException	if either date/time is not valid.

Returns

The number of days (with millisecond resolution) from `ref_dt` to `dt`.

References CATCH_BLOCK, and EXC_THROW.

10.36.2.8 GetCurrentUtcTime()

```
static ptime util::date::DateUtils::GetCurrentUtcTime ()  [inline], [static], [inherited]
```

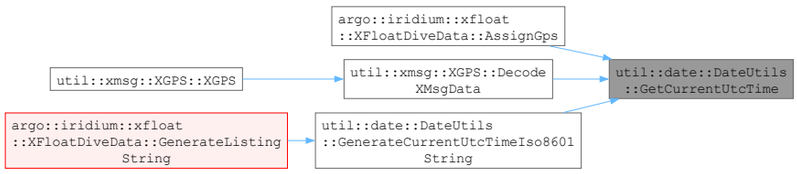
Generate a POSIX time object representing the current date and time (rounded to the nearest second).

Returns

A POSIX time object representing the current date and time (rounded to the nearest second).

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignGps(), util::xmsg::XGPS::DecodeXMsgData(), and util::date::DateUtils::GenerateCurrentUtcTimeIso8601String().

Here is the caller graph for this function:



10.36.2.9 IsPositionValid()

```
template<typename T >
static bool util::position::PositionUtils::IsPositionValid (
            T lat,
            T lon)  [inline], [static]
```

Check position validity.

Parameters

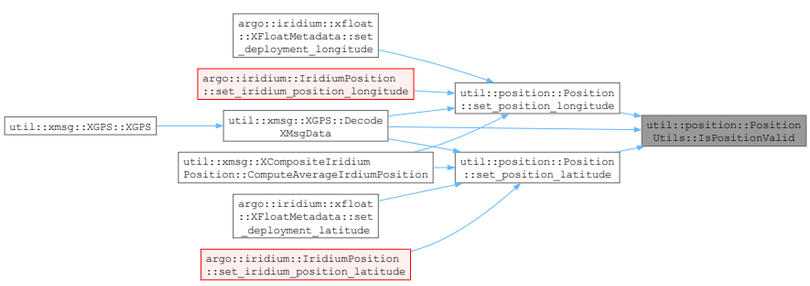
in	*lat*	Latitude of position.
in	*lon*	Longitude of position.

Returns

TRUE if (lat, lon) represents a valid geographical position, or FALSE otherwise.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), util::position::Position::set_position_latitude(), and util::position::Position::set_positi

Here is the caller graph for this function:



10.36.3 Member Data Documentation

10.36.3.1 EARTH_RADIUS_KM_EQUATORIAL

```
const double util::position::PositionUtils::EARTH_RADIUS_KM_EQUATORIAL = 6378.1  [static], [private]
```

The equatorial radius of the earth, in kilometers.

Referenced by CalculateEllipsoidHeight().

10.36.3.2 EARTH_RADIUS_KM_POLAR

```
const double util::position::PositionUtils::EARTH_RADIUS_KM_POLAR = 6356.1  [static], [private]
```

The polar radius of the earth, in kilometers.

Referenced by CalculateEllipsoidHeight().

The documentation for this class was generated from the following files:

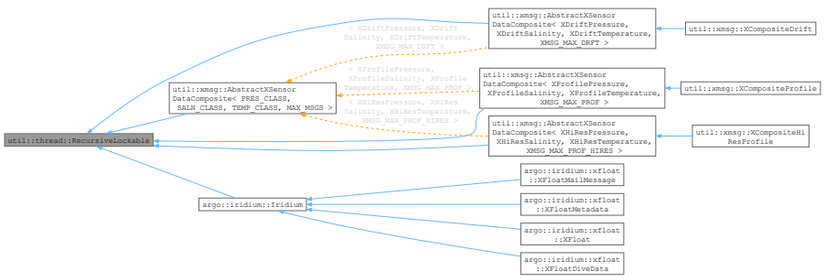
- utility/PositionUtils.h
- utility/PositionUtils.cpp

10.37 util::thread::RecursiveLockable Class Reference

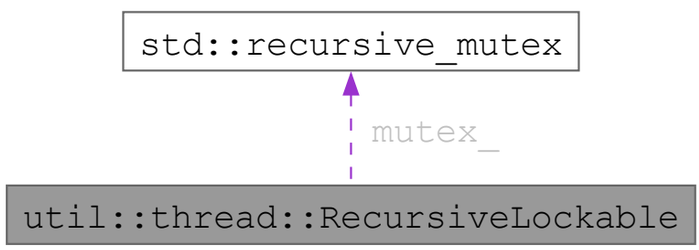
A recursively lockable class.

```
#include <RecursiveLockable.h>
```

Inheritance diagram for util::thread::RecursiveLockable:



Collaboration diagram for util::thread::RecursiveLockable:



Public Member Functions

- RecursiveLockable ()
  Default constructor.
- virtual ~RecursiveLockable ()
  Destructor.
- bool LockObject (bool block)
  Lock the object.
- void UnlockObject ()
  Unlock the object.

Protected Attributes

- recursive_mutex mutex_

  Recursive mutual exclusion lock for synchronizing operations.

10.37.1 Detailed Description

A recursively lockable class.

Note

This class uses a recursive mutex which means that it may be locked multiple times by the same thread.

10.37.2 Constructor & Destructor Documentation

10.37.2.1 RecursiveLockable()

```
util::thread::RecursiveLockable::RecursiveLockable ()  [inline]
```

Default constructor.

10.37.2.2 ~RecursiveLockable()

```
virtual util::thread::RecursiveLockable::~RecursiveLockable ()  [inline], [virtual]
```

Destructor.

10.37.3 Member Function Documentation

10.37.3.1 LockObject()

```
bool util::thread::RecursiveLockable::LockObject (
            bool block)  [inline]
```

Lock the object.

Parameters

in	*block*	If TRUE, then the calling thread will block until the lock becomes available.

Exceptions

util::log::MessageException	if an error occurs.

Returns

TRUE if the lock was successfully obtained, or FALSE if block is FALSE and another thread owns the lock on the mutex.

References CATCH_BLOCK, and mutex_.

10.37.3.2 UnlockObject()

```
void util::thread::RecursiveLockable::UnlockObject ()  [inline]
```

Unlock the object.

Exceptions

util::log::MessageException	if an error occurs.

References CATCH_BLOCK, and mutex_.

10.37.4 Member Data Documentation

10.37.4.1 mutex_

```
recursive_mutex util::thread::RecursiveLockable::mutex_  [mutable], [protected]
```

Recursive mutual exclusion lock for synchronizing operations.

Referenced by util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AggregateS
argo::iridium::xfloat::XFloat::argo_instrument_type(), argo::iridium::xfloat::XFloatDiveData::argo_instrument_type(),
argo::iridium::xfloat::XFloatDiveData::AssignArgoData(), argo::iridium::xfloat::XFloatDiveData::AssignAuxTemperatureBinned(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignAuxTemperatureMe
argo::iridium::xfloat::XFloatDiveData::AssignDriftPressure(), argo::iridium::xfloat::XFloatDiveData::AssignDriftSalinity(),
argo::iridium::xfloat::XFloatDiveData::AssignDriftTemperatre(), argo::iridium::xfloat::XFloatDiveData::AssignEngineeringAbort(),
argo::iridium::xfloat::XFloatDiveData::AssignEngineeringAscent(), argo::iridium::xfloat::XFloatDiveData::AssignEngineeringBitFail(),
argo::iridium::xfloat::XFloatDiveData::AssignEngineeringBitPass(), argo::iridium::xfloat::XFloatDiveData::AssignEngineeringDescent(),
argo::iridium::xfloat::XFloatDiveData::AssignEngineeringDiagnostic(), argo::iridium::xfloat::XFloatDiveData::AssignEngineeringDrift(),
argo::iridium::xfloat::XFloatDiveData::AssignEngineeringIce(), argo::iridium::xfloat::XFloatDiveData::AssignEngineeringStandard(),
argo::iridium::xfloat::XFloatDiveData::AssignEngineeringSurface(), argo::iridium::xfloat::XFloatDiveData::AssignFallTimeSeries(),
argo::iridium::xfloat::XFloatDiveData::AssignGps(), argo::iridium::xfloat::XFloatDiveData::AssignMailMessage(),
argo::iridium::xfloat::XFloatDiveData::AssignMissionCmdEchoback(), argo::iridium::xfloat::XFloatDiveData::AssignParentInstrument(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignPressureMessage(
argo::iridium::xfloat::XFloatDiveData::AssignProfilePressureBinned(), argo::iridium::xfloat::XFloatDiveData::AssignProfilePressureUnbinn
argo::iridium::xfloat::XFloatDiveData::AssignProfileSalinityBinned(), argo::iridium::xfloat::XFloatDiveData::AssignProfileSalinityUnbinned()
argo::iridium::xfloat::XFloatDiveData::AssignProfileTemperatureBinned(), argo::iridium::xfloat::XFloatDiveData::AssignProfileTemperature
argo::iridium::xfloat::XFloatDiveData::AssignPumpTimeSeries(), argo::iridium::xfloat::XFloatDiveData::AssignRiseTimeSeries(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignSalinityMessage(),
argo::iridium::xfloat::XFloatDiveData::AssignSurfaceTimeSeries(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_C
argo::iridium::xfloat::XFloatDiveData::AssignValveData(), argo::iridium::xfloat::XFloatDiveData::CheckCompleteness(),
argo::iridium::xfloat::XFloatMetadata::comment(), argo::iridium::xfloat::XFloatDiveData::ComputePressureOffsetFromSurfaceTimeSeries(
argo::iridium::xfloat::XFloatMetadata::deployer(), argo::iridium::xfloat::XFloatMetadata::deployer_address(), argo::iridium::xfloat::XFloatMe
argo::iridium::xfloat::XFloatMetadata::deployment_datetime(), argo::iridium::xfloat::XFloatMetadata::deployment_datetime_status(),
argo::iridium::xfloat::XFloatMetadata::deployment_latitude(), argo::iridium::xfloat::XFloatMetadata::deployment_longitude(),
argo::iridium::xfloat::XFloatMetadata::deployment_platform(), argo::iridium::xfloat::XFloatMetadata::deployment_position_status(),
argo::iridium::xfloat::XFloatMetadata::deployment_type(), argo::iridium::xfloat::XFloat::fw_version(), argo::iridium::xfloat::XFloat::gain_pres
argo::iridium::xfloat::XFloat::gain_pressure_surface(), argo::iridium::xfloat::XFloat::gain_salinity(), argo::iridium::xfloat::XFloat::gain_salinit
argo::iridium::xfloat::XFloat::gain_temperature(), argo::iridium::xfloat::XFloat::gain_temperature_surface(), argo::iridium::xfloat::XFloatDive
argo::iridium::xfloat::XFloatDiveData::GenerateCompositeIridiumPosition(), argo::iridium::xfloat::XFloatDiveData::GenerateCompositePun
argo::iridium::xfloat::XFloatDiveData::GenerateCompositeRiseTimeSeriesData(), argo::iridium::xfloat::XFloatDiveData::GenerateListingSt
util::xmsg::XCompositeDrift::GenerateListingString(), util::xmsg::XCompositeHiResProfile::GenerateListingString(),
util::xmsg::XCompositeProfile::GenerateListingString(), argo::iridium::xfloat::XFloatDiveData::GenerateOrderedMomsnList(),
argo::iridium::xfloat::XFloatDiveData::GeneratePhyString(), util::xmsg::XCompositeDrift::GeneratePhyString(), util::xmsg::XCompositeHiR
util::xmsg::XCompositeProfile::GeneratePhyString(), argo::iridium::xfloat::XFloatDiveData::GetDiveDataMailMessagesFromDatabase(),
argo::iridium::xfloat::XFloatDiveData::GetGainsAndOffsets(), argo::iridium::xfloat::XFloatDiveData::GetGpsPositionEndOfDive(),
argo::iridium::xfloat::XFloatDiveData::GetSurfaceGainsAndOffsets(), argo::iridium::xfloat::XFloatDiveData::HasBinnedProfileData(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::HasContiguousMessageIr
argo::iridium::xfloat::XFloatDiveData::HasContiguousMomsns(), argo::iridium::xfloat::XFloatDiveData::HasContiguousPids(),
argo::iridium::xfloat::XFloatDiveData::HasDiveData(), argo::iridium::xfloat::XFloatDiveData::HasDriftData(), argo::iridium::xfloat::XFloatDiv
argo::iridium::xfloat::XFloatDiveData::HasFallRiseData(), argo::iridium::xfloat::XFloatDiveData::HasPositionData(),
argo::iridium::xfloat::XFloatDiveData::HasUnbinnedProfileData(), argo::iridium::xfloat::XFloatDiveData::HasValidGpsPosition(),
argo::iridium::Iridium::id_dac(), argo::iridium::xfloat::XFloatDiveData::id_dive(), argo::iridium::xfloat::XFloatMailMessage::id_dive(),
argo::iridium::Iridium::id_imei(), argo::iridium::Iridium::id_instrument(), argo::iridium::xfloat::XFloatMailMessage::id_msg(),
argo::iridium::xfloat::XFloatMailMessage::id_msg_uid(), argo::iridium::xfloat::XFloatMailMessage::id_sbd_momsn(),
argo::iridium::xfloat::XFloatMailMessage::id_sbd_mtmsn(), argo::iridium::xfloat::XFloatMailMessage::id_sbd_packet(),
argo::iridium::Iridium::id_wmo(), argo::iridium::xfloat::XFloatDiveData::intermediate_listing_file_path(), util::xmsg::AbstractXSensorDataCo
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::IsDataComplete(),
argo::iridium::xfloat::XFloatMetadata::local_path(), LockObject(), argo::iridium::xfloat::XFloatMetadata::manufacturer(),
argo::iridium::xfloat::XFloat::metadata(), argo::iridium::xfloat::XFloatMailMessage::msg_body_is_valid(), argo::iridium::xfloat::XFloatMailM
argo::iridium::xfloat::XFloatMailMessage::msg_header_datetime(), argo::iridium::xfloat::XFloatMailMessage::msg_header_from(),
argo::iridium::xfloat::XFloatMailMessage::msg_header_is_valid(), argo::iridium::xfloat::XFloatMailMessage::msg_header_raw_text(),
argo::iridium::xfloat::XFloatMailMessage::msg_header_subject(), argo::iridium::xfloat::XFloatMailMessage::msg_is_valid(),
argo::iridium::xfloat::XFloatMailMessage::msg_raw_text(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, T

util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::num_saln_samples(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::num_temp_samples(), argo::iridium::xfloat::XFloat::offset_pressure(), argo::iridium::xfloat::XFloat::offset_pressure_surface(), argo::iridium::xfloat::XFloat::offset_ argo::iridium::xfloat::XFloat::offset_salinity_surface(), argo::iridium::xfloat::XFloat::offset_temperature(), argo::iridium::xfloat::XFloat::offset argo::iridium::xfloat::XFloatMailMessage::ParseMailMsg(), argo::iridium::xfloat::XFloatMailMessage::ParseMailMsg(), argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgBody(), argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgHeader(), argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgSbdAttachment(), argo::iridium::xfloat::XFloat::phy_file_generation_is_enabled(), argo::iridium::xfloat::XFloatDiveData::phy_file_path(), argo::iridium::xfloat::XFloatMetadata::principal_investigators(), argo::iridium::xfloat::XFloatMetadata::principal_investigators_address(), argo::iridium::xfloat::XFloatMetadata::principal_investigators_orig argo::iridium::xfloat::XFloatMailMessage::raw_file_local_path(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLA argo::iridium::xfloat::XFloatDiveData::ResetData(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_ util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SamplePresAsFloat(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleSaln(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleSalnAsFloat(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleTemp(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleTempAsFloat(), argo::iridium::xfloat::XFloatDiveData::SaveObjectToDatabase(), argo::iridium::xfloat::XFloatMailMessage::SaveObjectToDatabase(), argo::iridium::xfloat::XFloatMetadata::SaveObjectToDatabase(), argo::iridium::xfloat::XFloatMailMessage::sbd_data(), argo::iridium::xfloat::XFloatMailMessage::sbd_data_is_valid(), argo::iridium::xfloat::XFloatMailMessage::sbd_file_local_path(), argo::iridium::xfloat::XFloatMailMessage::sbd_file_name(), argo::iridium::xfloat::XFloatMailMessage::sbd_file_size_bytes(), argo::iridium::xfloat::XFloatMailMessage::sbd_session_status_code(), argo::iridium::xfloat::XFloatMailMessage::sbd_session_status_strir argo::iridium::xfloat::XFloatDiveData::SensorGainsAndOffsetsAreKnown(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, argo::iridium::xfloat::XFloat::set_argo_instrument_type(), argo::iridium::xfloat::XFloatDiveData::set_argo_instrument_type(), argo::iridium::xfloat::XFloatMetadata::set_comment(), argo::iridium::xfloat::XFloatMetadata::set_deployer(), argo::iridium::xfloat::XFloatMe argo::iridium::xfloat::XFloatMetadata::set_deployment_cruise_id(), argo::iridium::xfloat::XFloatMetadata::set_deployment_datetime(), argo::iridium::xfloat::XFloatMetadata::set_deployment_datetime_status(), argo::iridium::xfloat::XFloatMetadata::set_deployment_latitude(argo::iridium::xfloat::XFloatMetadata::set_deployment_longitude(), argo::iridium::xfloat::XFloatMetadata::set_deployment_platform(), argo::iridium::xfloat::XFloatMetadata::set_deployment_position_status(), argo::iridium::xfloat::XFloatMetadata::set_deployment_type(), argo::iridium::xfloat::XFloat::set_fw_version(), argo::iridium::xfloat::XFloat::set_gain_pressure(), argo::iridium::xfloat::XFloat::set_gain_pre argo::iridium::xfloat::XFloat::set_gain_salinity(), argo::iridium::xfloat::XFloat::set_gain_salinity_surface(), argo::iridium::xfloat::XFloat::set_ argo::iridium::xfloat::XFloat::set_gain_temperature_surface(), argo::iridium::Iridium::set_id_dac(), argo::iridium::xfloat::XFloatDiveData::se argo::iridium::xfloat::XFloatMailMessage::set_id_dive(), argo::iridium::Iridium::set_id_imei(), argo::iridium::Iridium::set_id_instrument(), argo::iridium::xfloat::XFloatMailMessage::set_id_msg(), argo::iridium::xfloat::XFloatMailMessage::set_id_msg_uid(), argo::iridium::xfloat::XFloatMailMessage::set_id_sbd_momsn(), argo::iridium::xfloat::XFloatMailMessage::set_id_sbd_mtmsn(), argo::iridium::xfloat::XFloatMailMessage::set_id_sbd_packet(), argo::iridium::Iridium::set_id_wmo(), argo::iridium::xfloat::XFloatDiveData: argo::iridium::xfloat::XFloatMetadata::set_local_path(), argo::iridium::xfloat::XFloatMetadata::set_manufacturer(), argo::iridium::xfloat::XFloat::set_metadata(), argo::iridium::xfloat::XFloatMailMessage::set_msg_body_is_valid(), argo::iridium::xfloat::XFloatMailMessage::set_msg_body_text(), argo::iridium::xfloat::XFloatMailMessage::set_msg_header_datetime(), argo::iridium::xfloat::XFloatMailMessage::set_msg_header_from(), argo::iridium::xfloat::XFloatMailMessage::set_msg_header_is_valid(), argo::iridium::xfloat::XFloatMailMessage::set_msg_header_raw_text(), argo::iridium::xfloat::XFloatMailMessage::set_msg_header_subjec argo::iridium::xfloat::XFloatMailMessage::set_msg_is_valid(), argo::iridium::xfloat::XFloatMailMessage::set_msg_raw_text(), argo::iridium::xfloat::XFloat::set_offset_pressure(), argo::iridium::xfloat::XFloat::set_offset_pressure_surface(), argo::iridium::xfloat::XFloa argo::iridium::xfloat::XFloat::set_offset_salinity_surface(), argo::iridium::xfloat::XFloat::set_offset_temperature(), argo::iridium::xfloat::XFloat::set_offset_temperature_surface(), argo::iridium::xfloat::XFloat::set_phy_file_generation_is_enabled(), argo::iridium::xfloat::XFloatDiveData::set_phy_file_path(), argo::iridium::xfloat::XFloatMetadata::set_principal_investigators(), argo::iridium::xfloat::XFloatMetadata::set_principal_investigators_address(), argo::iridium::xfloat::XFloatMetadata::set_principal_investiga argo::iridium::xfloat::XFloatMailMessage::set_raw_file_local_path(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_data(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_data_is_valid(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_file_local_path(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_file_name(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_file_size_bytes(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_session_status_code(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_session_sta argo::iridium::xfloat::XFloat::set_start_of_mission_datetime(), argo::iridium::xfloat::XFloatMetadata::set_wmo_instrument_type(), argo::iridium::xfloat::XFloatMetadata::set_wmo_recorder_type(), argo::iridium::xfloat::XFloat::set_xmsg_version_major(), argo::iridium::xfloat::XFloat::set_xmsg_version_minor(), argo::iridium::xfloat::XFloat::start_of_mission_datetime(), UnlockObject(), argo::iridium::xfloat::XFloatDiveData::UpdateDerivedSurfaceDataFields(), argo::iridium::xfloat::XFloatMetadata::wmo_inst argo::iridium::xfloat::XFloatMetadata::wmo_recorder_type(), argo::iridium::xfloat::XFloatDiveData::WriteListingFile(), argo::iridium::xfloat::XFloatDiveData::WritePhyFile(), argo::iridium::xfloat::XFloatMailMessage::WriteSbdFile(), argo::iridium::xfloat::XFloat and argo::iridium::xfloat::XFloat::xmsg_version_minor().

The documentation for this class was generated from the following file:

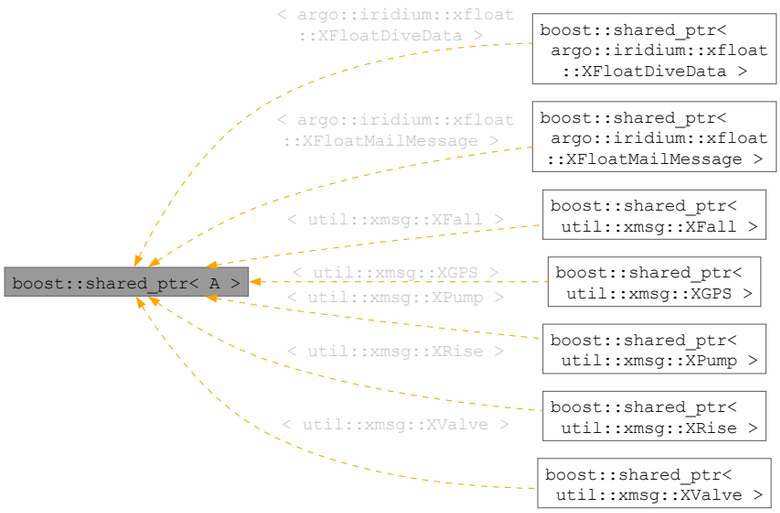
- utility/RecursiveLockable.h

10.38 boost::shared_ptr< A > Class Template Reference

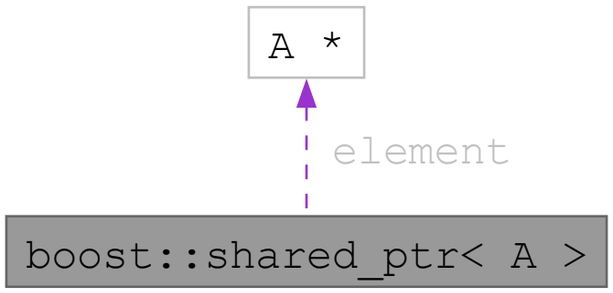
Dummy template class used for configuring Doxygen to interpret boost::shared_ptr objects as plain pointers.

`#include <AdditionalDocumentation.h>`

Inheritance diagram for boost::shared_ptr< A >:



Collaboration diagram for boost::shared_ptr< A >:



Private Attributes

- A * element

  The pointer to the element.

10.38.1 Detailed Description

template<class A>
class boost::shared_ptr< A >

Dummy template class used for configuring Doxygen to interpret boost::shared_ptr objects as plain pointers.

10.38.2 Member Data Documentation

10.38.2.1 element

```
template<class A >
A* boost::shared_ptr< A >::element  [private]
```

The pointer to the element.

The documentation for this class was generated from the following file:

- AdditionalDocumentation.h

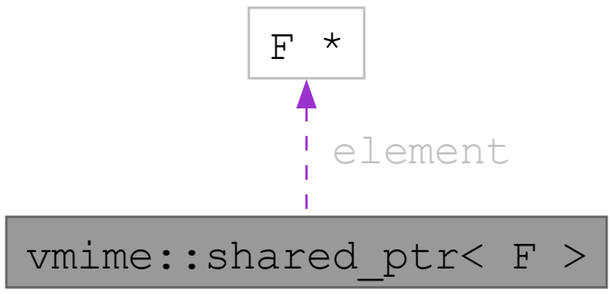
10.39 vmime::shared_ptr< F > Class Template Reference

Dummy template class used for configuring Doxygen to interpret vmime::shared_ptr objects as plain pointers.

`#include <AdditionalDocumentation.h>`

Inheritance diagram for vmime::shared_ptr< F >:

Collaboration diagram for vmime::shared_ptr< F >:



Private Attributes

- F * element

  The pointer to the element.

10.39.1 Detailed Description

template<class F>
class vmime::shared_ptr< F >

Dummy template class used for configuring Doxygen to interpret vmime::shared_ptr objects as plain pointers.

10.39.2 Member Data Documentation

10.39.2.1 element

```
template<class F >
F* vmime::shared_ptr< F >::element  [private]
```

The pointer to the element.

The documentation for this class was generated from the following file:

- AdditionalDocumentation.h

10.40 util::sig::SignalHandler Class Reference

Class providing function for handling POSIX style system signals.

`#include <SignalHandler.h>`

Static Public Member Functions

- static void ExitHandler (int signo)
  The exit handler.
- static void RegisterSignalHandlers () ATTR_NO_EXCEPTIONS
  Register signal handlers functions for UNIX-style system signals.

10.40.1 Detailed Description

Class providing function for handling POSIX style system signals.

This class provides functions for graceful shutdown, triggered by catchable system signals.

10.40.2 Member Function Documentation

10.40.2.1 ExitHandler()

```
void util::sig::SignalHandler::ExitHandler (
            int signo)  [static]
```

The exit handler.

This function is responsible for handling signals received from the local system. It is invoked when a signal is caught, but may also be invoked "manually" anywhere in the execution of the primary daemon thread to terminate the application, for example, at the bequest of a user.

This function attempts an orderly shutdown of the application. First, the worker threads are terminated. Then, locally cached data is saved to the database. Next, run-time lock is released. Finally, the globally logger is flushed and the application is terminated.

Parameters

in	*signo*	The signal number.

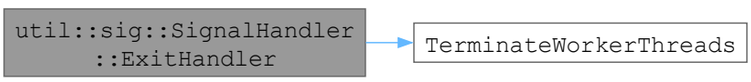
Exceptions

None.	Never throws.

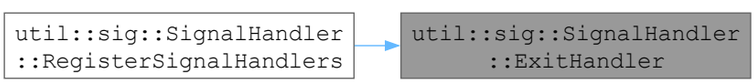
References CATCH_NOTHROW, Iout, and TerminateWorkerThreads().

Referenced by RegisterSignalHandlers().

Here is the call graph for this function:



Here is the caller graph for this function:



10.40.2.2 RegisterSignalHandlers()

```
static void util::sig::SignalHandler::RegisterSignalHandlers ()  [inline], [static]
```

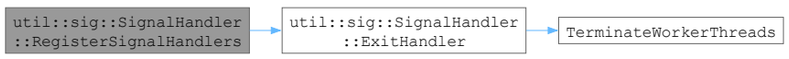
Register signal handlers functions for UNIX-style system signals.

Exceptions

None.	Never throws.

References CATCH_BLOCK, EXC_THROW, ExitHandler(), ldbg, linf, and lout.

Here is the call graph for this function:



The documentation for this class was generated from the following files:

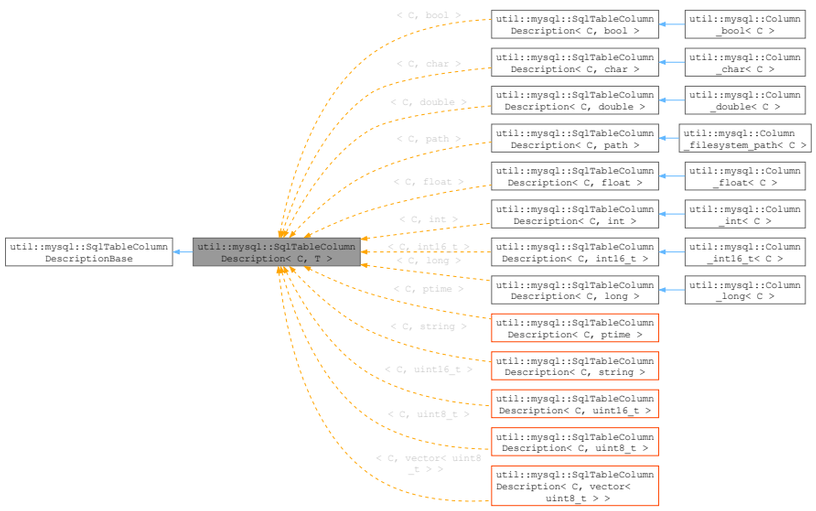
- utility/SignalHandler.h
- Main.cpp

10.41 util::mysql::SqlTableColumnDescription< C, T > Class Template Reference

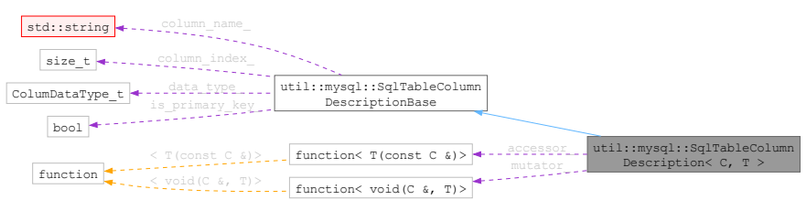
The `SqlTableColumnDescription` class is a template class used to describe an SQL table column for class `C` and data member of type `T`.

`#include <SqlTableColumnDescription.h>`

Inheritance diagram for util::mysql::SqlTableColumnDescription< C, T >:



Collaboration diagram for util::mysql::SqlTableColumnDescription< C, T >:



Public Member Functions

- SqlTableColumnDescription (string column_name, ColumDataType_t data_type, const T &(C::*accessor)() const =NULL, void(C::*mutator)(const T &)=NULL, bool is_primary_key=false)

  The parameterized constructor.
- SqlTableColumnDescription (string column_name, ColumDataType_t data_type, T(C::*accessor)() const =NULL, void(C::*mutator)(const T &)=NULL, bool is_primary_key=false)

  The parameterized constructor.
- SqlTableColumnDescription (string column_name, ColumDataType_t data_type, T(C::*accessor)() const =NULL, void(C::*mutator)(T)=NULL, bool is_primary_key=false)

  The parameterized constructor.
- size_t column_index () const

  Get the index of the SQL table column.
- const string & column_name () const

  Get the name of the SQL table column.
- string column_select_name () const

  Get the name of the SQL table column to be used in a `SELECT` query.
- ColumDataType_t data_type () const

  Get the the enumerated SQL column data type.
- bool is_primary_key () const

  Is this column a primary key for the corresponding SQL database table?
- void set_column_index (size_t idx)

  Set the index of this SQL table column.

Protected Member Functions

- T GetValueFromObject (const C &obj) const

  Get the object member value from the object.
- void SetObjectValue (C &obj, const T &value) const

  Set the object member value.

Private Attributes

- function< T(const C &)> accessor_

  The accessor function object wrapper.
- size_t column_index_

  The index of the SQL table column.
- string column_name_

  The name of the SQL table column.
- ColumDataType_t data_type_

  The enumerated SQL table column data type.
- bool is_primary_key_

  Indicates whether the column is part of the primary key for the SQL table.
- function< void(C &, T)> mutator_

  The mutator function object wrapper.

10.41.1 Detailed Description

template<class C, typename T>
class util::mysql::SqlTableColumnDescription< C, T >

The `SqlTableColumnDescription` class is a template class used to describe an SQL table column for class `C` and data member of type `T`.

10.41.2 Constructor & Destructor Documentation

10.41.2.1 SqlTableColumnDescription() [1/3]

```
template<class C , typename T >
util::mysql::SqlTableColumnDescription< C, T >::SqlTableColumnDescription (
            string column_name,
            ColumDataType_t data_type,
            T(C::* accessor )() const = NULL,
            void(C::* mutator )(T) = NULL,
            bool is_primary_key = false)  [inline]
```

The parameterized constructor.

Parameters

in	*column_name*	The name of the SQL table column.
in	*data_type*	They enumerated SQL table column type.
in	*accessor*	A pointer to the accessor method for acquiring the value of the corresponding object field.
in	*mutator*	A pointer to the mutator method for setting the value of the corresponding object field.
in	*is_primary_key*	TRUE if this column is a primary key for the SQL table, or FALSE otherwise.

10.41.2.2 SqlTableColumnDescription() [2/3]

```
template<class C , typename T >
util::mysql::SqlTableColumnDescription< C, T >::SqlTableColumnDescription (
            string column_name,
            ColumDataType_t data_type,
            T(C::* accessor )() const = NULL,
            void(C::* mutator )(const T &) = NULL,
            bool is_primary_key = false)  [inline]
```

The parameterized constructor.

Parameters

in	*column_name*	The name of the SQL table column.
in	*data_type*	They enumerated SQL table column type.
in	*accessor*	A pointer to the accessor method for acquiring the value of the corresponding object field.
in	*mutator*	A pointer to the mutator method for setting the value of the corresponding object field.
in	*is_primary_key*	TRUE if this column is a primary key for the SQL table, or FALSE otherwise.

10.41.2.3 SqlTableColumnDescription() [3/3]

```
template<class C , typename T >
util::mysql::SqlTableColumnDescription< C, T >::SqlTableColumnDescription (
            string column_name,
            ColumDataType_t data_type,
            const T &(C::* accessor )() const = NULL,
            void(C::* mutator )(const T &) = NULL,
            bool is_primary_key = false)  [inline]
```

The parameterized constructor.

Parameters

in	*column_name*	The name of the SQL table column.
in	*data_type*	They enumerated SQL table column type.
in	*accessor*	A pointer to the accessor method for acquiring the value of the corresponding object field.
in	*mutator*	A pointer to the mutator method for setting the value of the corresponding object field.
in	*is_primary_key*	TRUE if this column is a primary key for the SQL table, or FALSE otherwise.

10.41.3 Member Function Documentation

10.41.3.1 column_index()

```
size_t util::mysql::SqlTableColumnDescriptionBase::column_index () const  [inline], [inherited]
```

Get the index of the SQL table column.

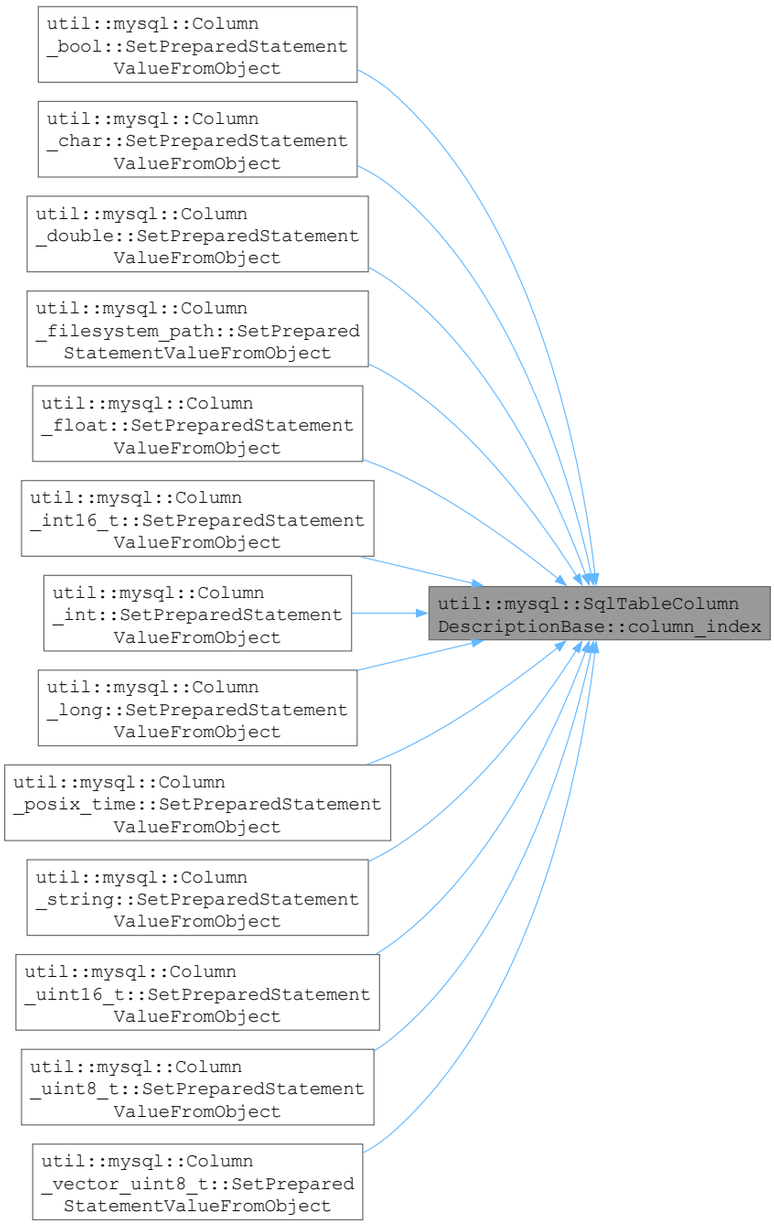
Returns

The index of the SQL table column.

References util::mysql::SqlTableColumnDescriptionBase::column_index_.

Referenced by util::mysql::Column_bool< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_char< C >::SetPreparedS
util::mysql::Column_double< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_filesystem_path< C >::SetPreparedS
util::mysql::Column_float< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_int16_t< C >::SetPreparedStatementVa
util::mysql::Column_int< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_long< C >::SetPreparedStatementValueF
util::mysql::Column_posix_time< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_string< C >::SetPreparedStateme
util::mysql::Column_uint16_t< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_uint8_t< C >::SetPreparedStatemer
and util::mysql::Column_vector_uint8_t< C >::SetPreparedStatementValueFromObject().

Here is the caller graph for this function:



10.41.3.2 column_name()

```
const string & util::mysql::SqlTableColumnDescriptionBase::column_name () const  [inline], [inherited]
```

Get the name of the SQL table column.

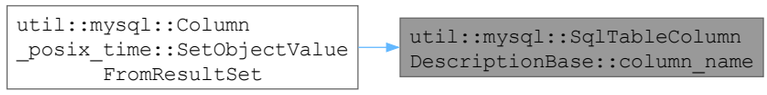
Returns

The name of the SQL table column.

References util::mysql::SqlTableColumnDescriptionBase::column_name_.

Referenced by util::mysql::Column_posix_time< C >::SetObjectValueFromResultSet().

Here is the caller graph for this function:



10.41.3.3 column_select_name()

```
string util::mysql::SqlTableColumnDescriptionBase::column_select_name () const  [inline], [inherited]
```

Get the name of the SQL table column to be used in a `SELECT` query.

Returns

The name of the SQL table column to be used in a `SELECT` query.

References util::mysql::SqlTableColumnDescriptionBase::column_name_, and util::mysql::SqlTableColumnDescriptionBase::data_type_.

10.41.3.4 data_type()

```
ColumDataType_t util::mysql::SqlTableColumnDescriptionBase::data_type () const  [inline], [inherited]
```

Get the the enumerated SQL column data type.

Returns

The the enumerated SQL column data type.

References util::mysql::SqlTableColumnDescriptionBase::data_type_.

10.41.3.5 GetValueFromObject()

```
template<class C , typename T >
T util::mysql::SqlTableColumnDescription< C, T >::GetValueFromObject (
            const C & obj) const  [inline], [protected]
```

Get the object member value from the object.

Parameters

in	*obj*	The object of type `C`.

Returns

The value of type `T` of the object member corresponding to this SQL column.

References util::mysql::SqlTableColumnDescription< C, T >::accessor_.

10.41.3.6 is_primary_key()

```
bool util::mysql::SqlTableColumnDescriptionBase::is_primary_key () const  [inline], [inherited]
```

Is this column a primary key for the corresponding SQL database table?

Returns

TRUE if this column is a primary key for the corresponding SQL database table, or FALSE otherwise.

References util::mysql::SqlTableColumnDescriptionBase::is_primary_key_.

10.41.3.7 set_column_index()

```
void util::mysql::SqlTableColumnDescriptionBase::set_column_index (
            size_t idx)  [inline], [inherited]
```

Set the index of this SQL table column.

Parameters

in	*idx*	The index to set.

References util::mysql::SqlTableColumnDescriptionBase::column_index_.

10.41.3.8 SetObjectValue()

```
template<class C , typename T >
void util::mysql::SqlTableColumnDescription< C, T >::SetObjectValue (
            C & obj,
            const T & value) const  [inline], [protected]
```

Set the object member value.

Parameters

in	*obj*	The object of type C.
in	*value*	The value of type T, to set.

References util::mysql::SqlTableColumnDescription< C, T >::mutator_.

10.41.4 Member Data Documentation

10.41.4.1 accessor_

```
template<class C , typename T >
function<T(const C&)> util::mysql::SqlTableColumnDescription< C, T >::accessor_  [private]
```

The accessor function object wrapper.

This object is used as a wrapper for the accessor method of the template class C. It is used to retrieve the value of the SQL table column from the object for generating an INSERT or UPDATE statement.

Referenced by util::mysql::SqlTableColumnDescription< C, T >::GetValueFromObject().

10.41.4.2 column_index_

```
size_t util::mysql::SqlTableColumnDescriptionBase::column_index_  [private], [inherited]
```

The index of the SQL table column.

Referenced by util::mysql::SqlTableColumnDescriptionBase::column_index(), and util::mysql::SqlTableColumnDescriptionBase::set_colum

10.41.4.3 column_name_

```
string util::mysql::SqlTableColumnDescriptionBase::column_name_  [private], [inherited]
```

The name of the SQL table column.

Referenced by util::mysql::SqlTableColumnDescriptionBase::column_name(), and util::mysql::SqlTableColumnDescriptionBase::column_s

10.41.4.4 data_type_

```
ColumDataType_t util::mysql::SqlTableColumnDescriptionBase::data_type_  [private], [inherited]
```

The enumerated SQL table column data type.

Referenced by util::mysql::SqlTableColumnDescriptionBase::column_select_name(), and util::mysql::SqlTableColumnDescriptionBase::da

10.41.4.5 is_primary_key_

```
bool util::mysql::SqlTableColumnDescriptionBase::is_primary_key_  [private], [inherited]
```

Indicates whether the column is part of the primary key for the SQL table.

Referenced by util::mysql::SqlTableColumnDescriptionBase::is_primary_key().

10.41.4.6 mutator_

```
template<class C , typename T >
function<void(C&, T)> util::mysql::SqlTableColumnDescription< C, T >::mutator_  [private]
```

The mutator function object wrapper.

This object is used as a wrapper for the mutator method of the template class C. It is used to set the value of an object from the result set of an SQL SELECT query statement.

Referenced by util::mysql::SqlTableColumnDescription< C, T >::SetObjectValue().

The documentation for this class was generated from the following file:

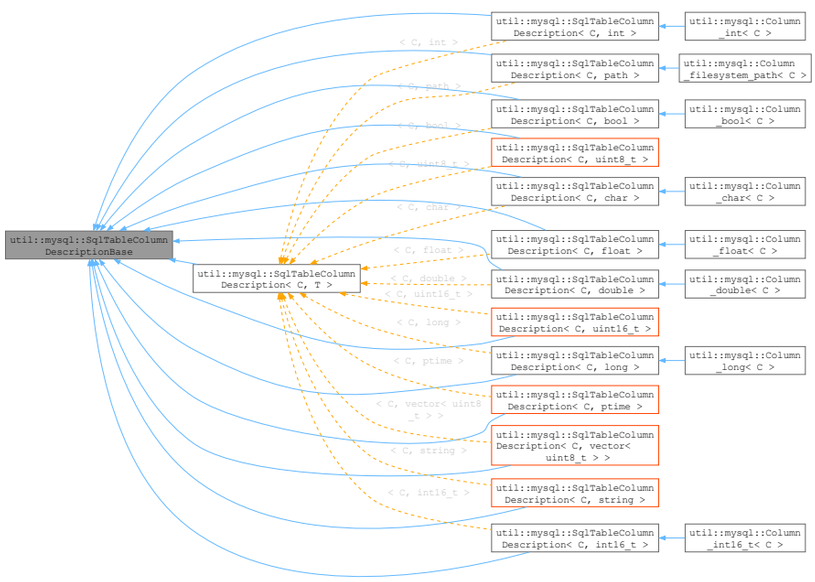
- utility/SqlTableColumnDescription.h

10.42 util::mysql::SqlTableColumnDescriptionBase Class Reference

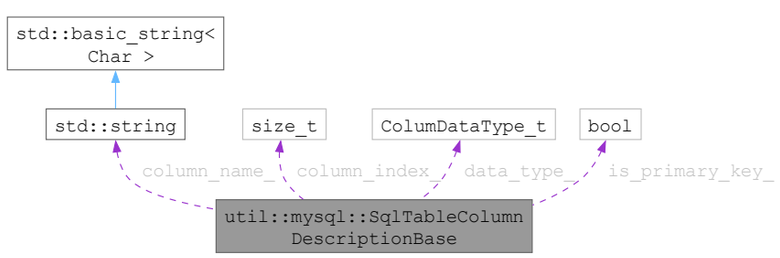
An abstract base class for the description of an SQL database table column.

```
#include <SqlTableColumnDescription.h>
```

Inheritance diagram for util::mysql::SqlTableColumnDescriptionBase:



Collaboration diagram for util::mysql::SqlTableColumnDescriptionBase:



Public Member Functions

- SqlTableColumnDescriptionBase (string column_name, ColumDataType_t type, bool is_primary_key)

  The parameterized constructor.
- virtual ~SqlTableColumnDescriptionBase ()

  The destructor.
- size_t column_index () const

  Get the index of the SQL table column.
- const string & column_name () const

  Get the name of the SQL table column.
- string column_select_name () const

  Get the name of the SQL table column to be used in a `SELECT` query.
- ColumDataType_t data_type () const

  Get the the enumerated SQL column data type.
- bool is_primary_key () const

  Is this column a primary key for the corresponding SQL database table?
- void set_column_index (size_t idx)

  Set the index of this SQL table column.

Private Attributes

- size_t column_index_

  The index of the SQL table column.
- string column_name_

  The name of the SQL table column.
- ColumDataType_t data_type_

  The enumerated SQL table column data type.
- bool is_primary_key_

  Indicates whether the column is part of the primary key for the SQL table.

10.42.1 Detailed Description

An abstract base class for the description of an SQL database table column.

This base class is necessary for utilizing polymorphism such that a pointers to classes that inherit from this class may be stored in a single vector.

10.42.2 Constructor & Destructor Documentation

10.42.2.1 SqlTableColumnDescriptionBase()

```
util::mysql::SqlTableColumnDescriptionBase::SqlTableColumnDescriptionBase (
            string column_name,
            ColumDataType_t type,
            bool is_primary_key)  [inline]
```

The parameterized constructor.

Parameters

in	*column_name*	The name of the SQL table column.
in	*type*	They enumerated SQL table column type.
in	*is_primary_key*	TRUE if this column is a primary key for the SQL table, or FALSE otherwise.

10.42.2.2 ~SqlTableColumnDescriptionBase()

```
virtual util::mysql::SqlTableColumnDescriptionBase::~SqlTableColumnDescriptionBase ()  [inline],
[virtual]
```

The destructor.

10.42.3 Member Function Documentation

10.42.3.1 column_index()

```
size_t util::mysql::SqlTableColumnDescriptionBase::column_index () const  [inline]
```

Get the index of the SQL table column.

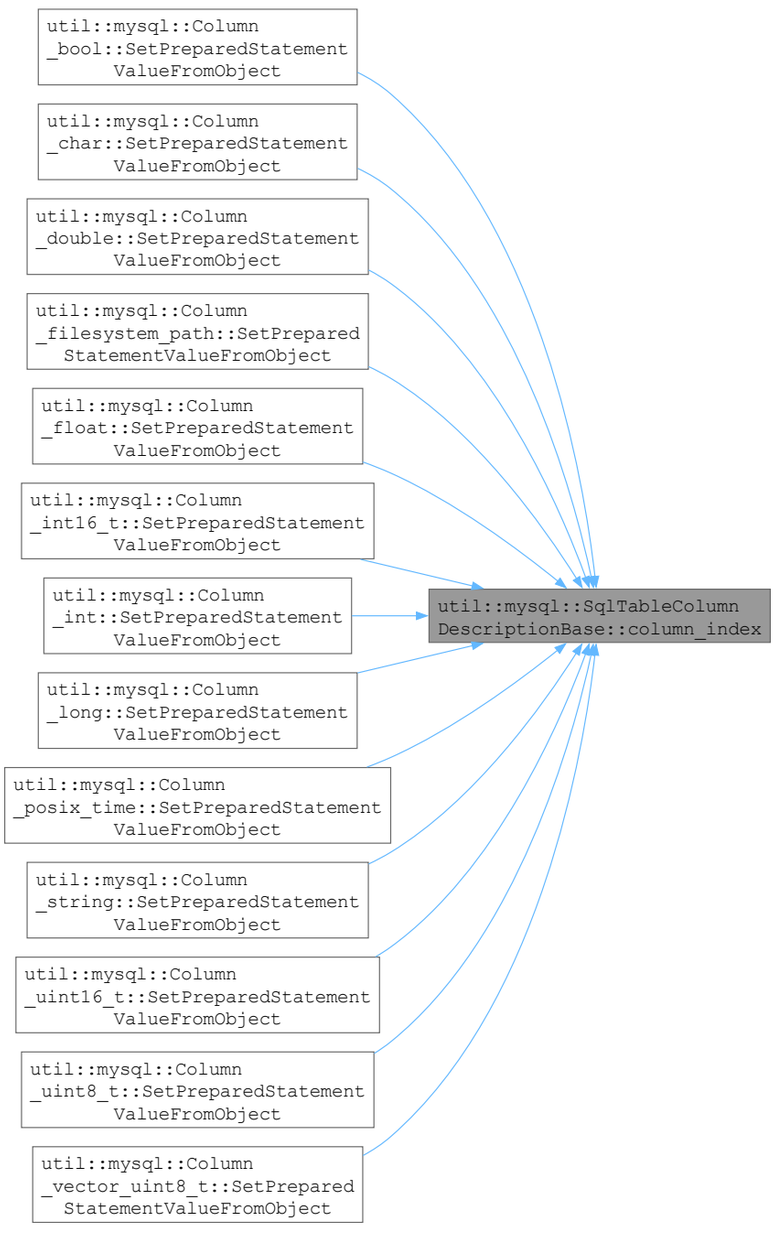
Returns

The index of the SQL table column.

References column_index_.

Referenced by util::mysql::Column_bool< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_char< C >::SetPreparedS
util::mysql::Column_double< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_filesystem_path< C >::SetPreparedS
util::mysql::Column_float< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_int16_t< C >::SetPreparedStatementVa
util::mysql::Column_int< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_long< C >::SetPreparedStatementValueF
util::mysql::Column_posix_time< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_string< C >::SetPreparedStateme
util::mysql::Column_uint16_t< C >::SetPreparedStatementValueFromObject(), util::mysql::Column_uint8_t< C >::SetPreparedStatemer
and util::mysql::Column_vector_uint8_t< C >::SetPreparedStatementValueFromObject().

Here is the caller graph for this function:



10.42.3.2 column_name()

```
const string & util::mysql::SqlTableColumnDescriptionBase::column_name () const  [inline]
```

Get the name of the SQL table column.

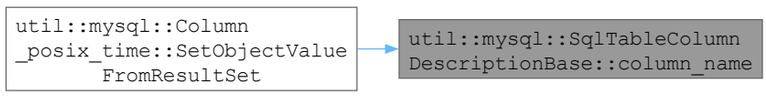
Returns

The name of the SQL table column.

References column_name_.

Referenced by util::mysql::Column_posix_time< C >::SetObjectValueFromResultSet().

Here is the caller graph for this function:



10.42.3.3 column_select_name()

```
string util::mysql::SqlTableColumnDescriptionBase::column_select_name () const  [inline]
```

Get the name of the SQL table column to be used in a `SELECT` query.

Returns

The name of the SQL table column to be used in a `SELECT` query.

References column_name_, and data_type_.

10.42.3.4 data_type()

```
ColumDataType_t util::mysql::SqlTableColumnDescriptionBase::data_type () const  [inline]
```

Get the the enumerated SQL column data type.

Returns

The the enumerated SQL column data type.

References data_type_.

10.42.3.5 is_primary_key()

```
bool util::mysql::SqlTableColumnDescriptionBase::is_primary_key () const  [inline]
```

Is this column a primary key for the corresponding SQL database table?

Returns

TRUE if this column is a primary key for the corresponding SQL database table, or FALSE otherwise.

References is_primary_key_.

10.42.3.6 set_column_index()

```
void util::mysql::SqlTableColumnDescriptionBase::set_column_index (
            size_t idx)  [inline]
```

Set the index of this SQL table column.

Parameters

in	*idx*	The index to set.

References column_index_.

10.42.4 Member Data Documentation

10.42.4.1 column_index_

```
size_t util::mysql::SqlTableColumnDescriptionBase::column_index_  [private]
```

The index of the SQL table column.

Referenced by column_index(), and set_column_index().

10.42.4.2 column_name_

```
string util::mysql::SqlTableColumnDescriptionBase::column_name_  [private]
```

The name of the SQL table column.

Referenced by column_name(), and column_select_name().

10.42.4.3 data_type_

```
ColumDataType_t util::mysql::SqlTableColumnDescriptionBase::data_type_  [private]
```

The enumerated SQL table column data type.

Referenced by column_select_name(), and data_type().

10.42.4.4 is_primary_key_

```
bool util::mysql::SqlTableColumnDescriptionBase::is_primary_key_  [private]
```

Indicates whether the column is part of the primary key for the SQL table.

Referenced by is_primary_key().

The documentation for this class was generated from the following file:

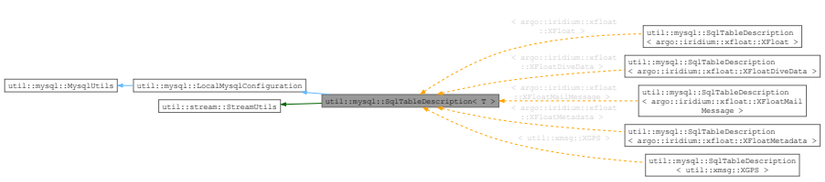
- utility/SqlTableColumnDescription.h

10.43 util::mysql::SqlTableDescription< T > Class Template Reference

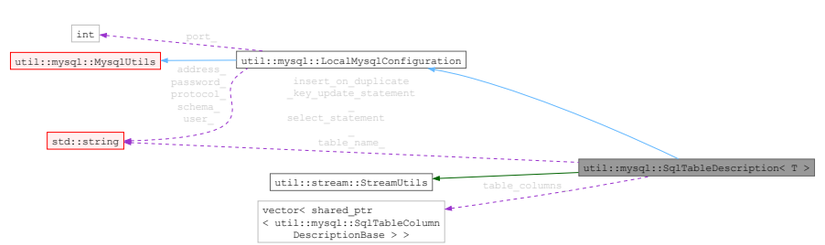
Template class representing the description of an SQL database table.

`#include <SqlTableDescription.h>`

Inheritance diagram for util::mysql::SqlTableDescription< T >:



Collaboration diagram for util::mysql::SqlTableDescription< T >:



Public Member Functions

- SqlTableDescription (const string &name, const vector< shared_ptr< SqlTableColumnDescriptionBase > > &columns)

  The parameterized constructor.
- ~SqlTableDescription ()

  The destructor.
- void **DeleteObjectsFromDatabase** (const string &where="") const
- string generateInsertStatment ()

  Generate an SQL INSERT statement for this SQL table.
- string generateSelectAllStatement ()

  Generate the "select all" SQL statement for this table.
- string getColumnNames () const

  Generate a string containing comma-separated, ordered list of column names for this SQL table from the column descriptions.
- vector< shared_ptr< SqlTableColumnDescriptionBase > > getColumns () const

  Get the vector of shared pointers to ColumnDescription objects describing the columns of this SQL table.
- string getColumnSelectNames () const

  Generate a string containing comma-separated, ordered list of column names used for select statements for this SQL table from the column descriptions.
- void SaveObjectToDatabase (const T &obj) const

  Save a persistent entity to the SQL table.
- vector< shared_ptr< T > > SelectAllObjectsFromDatabase (const string &groupBy="", const string &orderBy="", const unsigned int &limit=0, const unsigned int &offset=0) const

  Select all objects from this SQL database table.
- shared_ptr< T > SelectObjectFromDatabase (const string &where="", const string &groupBy="", const string &orderBy="", const unsigned int &offset=0) const

  Get a single object from the database.
- vector< shared_ptr< T > > SelectObjectsFromDatabase (const string &where="", const string &groupBy="", const string &orderBy="", const unsigned int &limit=0, const unsigned int &offset=0) const

  Select objects from this SQL database table.
- template<typename TT >
  TT SelectValueFromDatabase (const string &what, const string &where="", const string &groupBy="", const string &orderBy="", const unsigned int &offset=0) const

  Select a value from the SQL database table represented by this object.

Static Public Member Functions

- static void CloseSqlConnection ()

  Close the thread-specific MySQL connection if it is currently open.
- static shared_ptr< sql::ResultSet > **executeDeleteQuery** (const string &from, const string &where="")
- static shared_ptr< ResultSet > ExecuteQuery (Connection *con, const string &query)

  Execute an SQL query and return a shared pointer to the ResultSet object.
- static shared_ptr< sql::ResultSet > executeQuery (const string &query)

  Execute an SQL query and return a shared pointer to the ResultSet object.
- static shared_ptr< sql::ResultSet > executeSelectQuery (const string &what, const string &from, const string &where="", const string &groupBy="", const string &orderBy="", unsigned int limit=0, unsigned int offset=0)

  Execute an SQL select query and return a shared pointer to the ResultSet object.
- static int ExecuteUpdate (Connection *con, const string &update)

  Execute an SQL update and return the status code.
- static int executeUpdate (const string &update)

  Execute an SQL update and return the status code.
- static shared_ptr< PreparedStatement > GeneratePreparedStatement (Connection *con, const string &statement)

  Generate a prepared SQL statement object for the given SQL statement.
- static shared_ptr< sql::PreparedStatement > generatePreparedStatement (const string &statement)

  Generate a prepared SQL statement object for the given SQL statement.
- static string GenerateSqlDeleteStr (const string &from, const string &where="")

  Generate an SQL DELETE statement.
- static string GenerateSqlSelectStr (const string &what, const string &from, const string &where="", const string &groupBy="", const string &orderBy="", unsigned int limit=0, unsigned int offset=0)

  Generate an SQL select statement.
- static thread_specific_ptr< Connection > & GetSqlConnection (string url, string user, string password, string schema)

  Get a thread-specific instance of a MySQL connection object.
- static thread_specific_ptr< MySQL_Driver > & GetSqlDriver ()

  Get a thread-specific instance of a MySQL driver object.
- static string values (size_t n)

  Generate a values string component of an SQL insert statement with n values.

Static Protected Member Functions

- static sql::Connection * getSqlConnection ()

  Generate a thread-specific instance of a MySQL connection object.
- static uint8_t htoi (char c)

  Get the integer value of a hexadecimal character.

Private Types

- typedef Column_bool< T > cbol

  A column of type `bool`.
- typedef Column_char< T > cchar

  A column of type `char`.
- typedef Column_double< T > cdbl

  A column of type `double`.
- typedef Column_float< T > cflt

  A column of type `float`.
- typedef Column_int16_t< T > ci16

  A column of type `int16_t`.
- typedef Column_int< T > cint

  A column of type `int`.
- typedef Column_long< T > clng

  A column of type `long`.
- typedef Column_filesystem_path< T > cpth

  A column of type `boost::filesystem::path`.
- typedef Column_posix_time< T > cptm

  A column of type `boost::posix_time::ptime`.
- typedef Column_string< T > cstr

  A column of type `std::string`.
- typedef Column_uint16_t< T > cu16

  A column of type `uint16_t`.
- typedef Column_uint8_t< T > cu8

  A column of type `uint8_t`.
- typedef Column_vector_uint8_t< T > cvu8

  A column of type `std::vector<uint8_t>`.

Private Member Functions

- vector< shared_ptr< T > > GenerateObjectList (shared_ptr< ResultSet > res) const

  Generate a vector of object T from the given result set.

Static Private Member Functions

- static void CleanupSqlConnection (Connection *con)

  Cleanup an SQL connection.
- static void CleanupSqlDriver (MySQL_Driver *driver)

  Function for memory cleanup of MySQL driver on destruct.

Private Attributes

- string insert_on_duplicate_key_update_statement_

  The insert statement for this SQL table.
- string select_statement_

  The select all statement for this SQL table.
- vector< shared_ptr< SqlTableColumnDescriptionBase > > table_columns_

  A vector of shared pointers to SqlTableColumnDescription objects describing the columns of this SQL table.
- string table_name_

  The name of this SQL table.

Static Private Attributes

- static string address_
  The SQL server qualified name or IP address string.
- static string password_
  The SQL server account password string.
- static unsigned int port_
  The SQL server port number.
- static string protocol_
  The protocol to use in the SQL server connection e.g.
- static string schema_
  The SQL server schema name.
- static thread_specific_ptr< Connection > sql_connection_
  Tthe thread-specific pointer to MySQL connection object.
- static thread_specific_ptr< MySQL_Driver > sql_driver_
  The thread-specific pointer to MySQL driver object.
- static string user_
  The SQL server account username string.

10.43.1 Detailed Description

template<class T>
class util::mysql::SqlTableDescription< T >

Template class representing the description of an SQL database table.

The template parameter T represents the class that the SQL database table represents.

10.43.2 Member Typedef Documentation

10.43.2.1 cbol

```
template<class T >
typedef Column_bool<T> util::mysql::SqlTableDescription< T >::cbol  [private]
```

A column of type `bool`.

10.43.2.2 cchar

```
template<class T >
typedef Column_char<T> util::mysql::SqlTableDescription< T >::cchar  [private]
```

A column of type `char`.

10.43.2.3 cdbl

```
template<class T >
typedef Column_double<T> util::mysql::SqlTableDescription< T >::cdbl  [private]
```

A column of type `double`.

10.43.2.4 cflt

```
template<class T >
typedef Column_float<T> util::mysql::SqlTableDescription< T >::cflt  [private]
```

A column of type `float`.

10.43.2.5 ci16

```
template<class T >
typedef Column_int16_t<T> util::mysql::SqlTableDescription< T >::ci16  [private]
```

A column of type `int16_t`.

10.43.2.6 cint

```
template<class T >
typedef Column_int<T> util::mysql::SqlTableDescription< T >::cint  [private]
```

A column of type `int`.

10.43.2.7 clng

```
template<class T >
typedef Column_long<T> util::mysql::SqlTableDescription< T >::clng  [private]
```

A column of type `long`.

10.43.2.8 cpth

```
template<class T >
typedef Column_filesystem_path<T> util::mysql::SqlTableDescription< T >::cpth  [private]
```

A column of type `boost::filesystem::path`.

10.43.2.9 cptm

```
template<class T >
typedef Column_posix_time<T> util::mysql::SqlTableDescription< T >::cptm  [private]
```

A column of type `boost::posix_time::ptime`.

10.43.2.10 cstr

```
template<class T >
typedef Column_string<T> util::mysql::SqlTableDescription< T >::cstr  [private]
```

A column of type `std::string`.

10.43.2.11 cu16

```
template<class T >
typedef Column_uint16_t<T> util::mysql::SqlTableDescription< T >::cu16  [private]
```

A column of type `uint16_t`.

10.43.2.12 cu8

```
template<class T >
typedef Column_uint8_t<T> util::mysql::SqlTableDescription< T >::cu8  [private]
```

A column of type `uint8_t`.

10.43.2.13 cvu8

```
template<class T >
typedef Column_vector_uint8_t<T> util::mysql::SqlTableDescription< T >::cvu8  [private]
```

A column of type `std::vector<uint8_t>`.

10.43.3 Constructor & Destructor Documentation

10.43.3.1 SqlTableDescription()

```
template<class T >
util::mysql::SqlTableDescription< T >::SqlTableDescription (
            const string & name,
            const vector< shared_ptr< SqlTableColumnDescriptionBase > > & columns)  [inline]
```

The parameterized constructor.

Parameters

in	*name*	The name of the SQL database table.
in	*columns*	Vector of shared pointers to ColumnDescription objects describing the columns of this SQL table.

References util::mysql::SqlTableDescription< T >::generateInsertStatment(), util::mysql::SqlTableDescription< T >::generateSelectAllSta
and util::mysql::SqlTableDescription< T >::table_columns_.

Here is the call graph for this function:

10.43.3.2 ~SqlTableDescription()

```
template<class T >
util::mysql::SqlTableDescription< T >::~SqlTableDescription ()  [inline]
```

The destructor.

10.43.4 Member Function Documentation

10.43.4.1 CleanupSqlConnection()

```
static void util::mysql::MysqlUtils::CleanupSqlConnection (
            Connection * con)  [inline], [static], [private], [inherited]
```

Cleanup an SQL connection.

This function for memory cleanup of the SQL connection on destruction. The thread-specific SQL connection object will be deleted if it exists.

Parameters

in	*con*	The SQL connection to cleanup.

Exceptions

log::MessageException	if an error occurs while cleaning up the SQL connection object.

References CATCH_BLOCK.

10.43.4.2 CleanupSqlDriver()

```
static void util::mysql::MysqlUtils::CleanupSqlDriver (
            MySQL_Driver * driver)  [inline], [static], [private], [inherited]
```

Function for memory cleanup of MySQL driver on destruct.

Signals the SQL driver to end operation, if it exists.

Parameters

in	*driver*	Driver to cleanup.

Exceptions

log::MessageException	if an error occurs while cleaning up the SQL driver object.

References CATCH_BLOCK.

10.43.4.3 CloseSqlConnection()

```
static void util::mysql::MysqlUtils::CloseSqlConnection ()  [inline], [static], [inherited]
```

Close the thread-specific MySQL connection if it is currently open.

If the thread local MySQL connection is not currently open, then this function will have no effect.

Note

The MySQL connection should be closed when idle to prevent the connection from timing out and throwing exceptions.

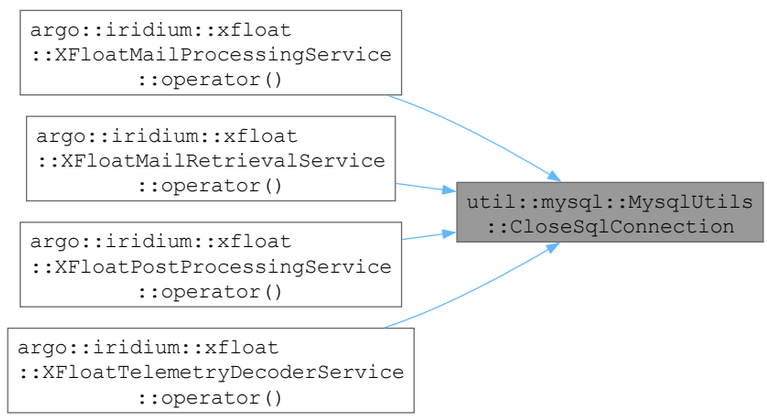
Exceptions

log::MessageException	if an error occurs while attempting to close the MySQL connection.

References CATCH_BLOCK, and util::mysql::MysqlUtils::sql_connection_.

Referenced by argo::iridium::xfloat::XFloatMailProcessingService::operator()(), argo::iridium::xfloat::XFloatMailRetrievalService::operator(
argo::iridium::xfloat::XFloatPostProcessingService::operator()(), and argo::iridium::xfloat::XFloatTelemetryDecoderService::operator()().

Here is the caller graph for this function:



10.43.4.4 ExecuteQuery()

```
static shared_ptr< ResultSet > util::mysql::MysqlUtils::ExecuteQuery (
            Connection * con,
            const string & query)  [inline], [static], [inherited]
```

Execute an SQL query and return a shared pointer to the ResultSet object.

Parameters

in	*con*	A pointer to the SQL connection object. This must not be NULL.
in	*query*	The SQL query to execute. This must be a well-formatted SQL query string.

Exceptions

log::MessageException	if 'con' is null, 'query' is a malfored SQL string, or any other error occurs.

Returns

A shared pointer to the query ResultSet object.

References CATCH_BLOCK, and util::mysql::MysqlUtils::GeneratePreparedStatement().

Referenced by util::mysql::LocalMysqlConfiguration::executeQuery().

Here is the call graph for this function:

Here is the caller graph for this function:

10.43.4.5 executeQuery()

```
shared_ptr< sql::ResultSet > util::mysql::LocalMysqlConfiguration::executeQuery (
            const string & query)  [static], [inherited]
```

Execute an SQL query and return a shared pointer to the ResultSet object.

Parameters

in	*query*	The SQL query to execute. This must be a well-formatted SQL query string.

Exceptions

log::MessageException	'query' is a malfored SQL string or any other error occurs while executing the query.

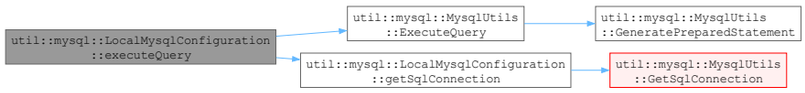
Returns

A shared pointer to the query ResultSet object.

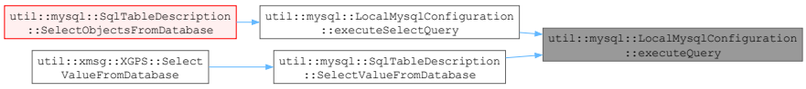
References util::mysql::MysqlUtils::ExecuteQuery(), and util::mysql::LocalMysqlConfiguration::getSqlConnection().

Referenced by util::mysql::LocalMysqlConfiguration::executeSelectQuery(), and util::mysql::SqlTableDescription< T >::SelectValueFromD

Here is the call graph for this function:



Here is the caller graph for this function:



10.43.4.6 executeSelectQuery()

```
shared_ptr< sql::ResultSet > util::mysql::LocalMysqlConfiguration::executeSelectQuery (
            const string & what,
            const string & from,
            const string & where = "",
            const string & groupBy = "",
            const string & orderBy = "",
            unsigned int limit = 0,
            unsigned int offset = 0)  [static], [inherited]
```

Execute an SQL select query and return a shared pointer to the ResultSet object.

Parameters

in	*what*	The SQL select statement select-expression clause.
in	*from*	The SQL select statement table–references clause.
in	*where*	The SQL select statement where-condition clause.
in	*groupBy*	The SQL select statement group-by clause.
in	*orderBy*	The SQL select statement order-by clause.
in	*limit*	The SQL select statement result set row limit value.
in	*offset*	The SQL select statement result set row offset.

Exceptions

log::MessageException	if an error occurs while executing the query.

Returns

A shared pointer to the ResultSet object.

References CATCH_BLOCK, util::mysql::LocalMysqlConfiguration::executeQuery(), and util::mysql::MysqlUtils::GenerateSqlSelectStr().

Referenced by util::mysql::SqlTableDescription< T >::SelectObjectsFromDatabase().

Here is the call graph for this function:

Here is the caller graph for this function:

10.43.4.7 ExecuteUpdate()

```
static int util::mysql::MysqlUtils::ExecuteUpdate (
            Connection * con,
            const string & update)  [inline], [static], [inherited]
```

Execute an SQL update and return the status code.

Parameters

in	*con*	A pointer to the SQL connection object. This must not be NULL.
in	*update*	The SQL update to execute. This must be a well-formatted SQL update string.

Exceptions

log::MessageException	if 'con' is null, 'update' is a malformed SQL string, or any other error occurs.

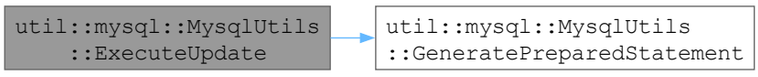
Returns

The update status code.

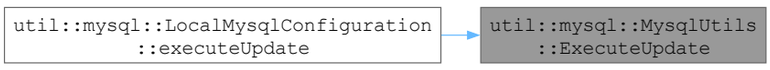
References CATCH_BLOCK, and util::mysql::MysqlUtils::GeneratePreparedStatement().

Referenced by util::mysql::LocalMysqlConfiguration::executeUpdate().

Here is the call graph for this function:



Here is the caller graph for this function:



10.43.4.8 executeUpdate()

```
int util::mysql::LocalMysqlConfiguration::executeUpdate (
            const string & update)  [static], [inherited]
```

Execute an SQL update and return the status code.

Parameters

in	*update*	The SQL update to execute. This must be a well-formatted SQL update string.

Exceptions

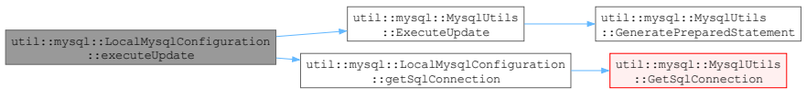
log::MessageException	if 'update' is a malformed SQL string, or any other error occurs while executing the update.

Returns

The update status code.

References util::mysql::MysqlUtils::ExecuteUpdate(), and util::mysql::LocalMysqlConfiguration::getSqlConnection().

Here is the call graph for this function:



10.43.4.9 generateInsertStatment()

```
template<class T >
string util::mysql::SqlTableDescription< T >::generateInsertStatment ()  [inline]
```

Generate an SQL INSERT statement for this SQL table.

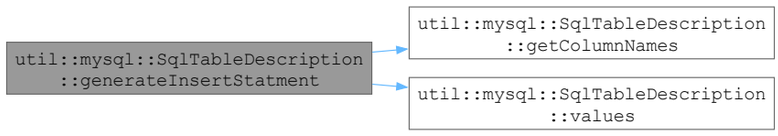
Returns

An SQL INSERT statement for this SQL table.

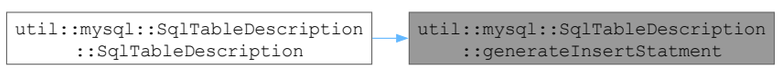
References util::mysql::SqlTableDescription< T >::getColumnNames(), util::mysql::SqlTableDescription< T >::insert_on_duplicate_key_ util::mysql::SqlTableDescription< T >::table_columns_, util::mysql::SqlTableDescription< T >::table_name_, and util::mysql::SqlTableDescription< T >::values().

Referenced by util::mysql::SqlTableDescription< T >::SqlTableDescription().

Here is the call graph for this function:



Here is the caller graph for this function:



10.43.4.10 GenerateObjectList()

```
template<class T >
vector< shared_ptr< T > > util::mysql::SqlTableDescription< T >::GenerateObjectList (
            shared_ptr< ResultSet > res) const  [inline], [private]
```

Generate a vector of object T from the given result set.

Parameters

in	*res*	A shared pointer to the result set from which the vector of objects is to be constructed.

Note

The columns of the result set must correspond in name, type, and order to column descriptions contained in SqlTableDescription<T>::table_columns_.

Exceptions

log::MessageException	if an error occurs while constructing the result vector.

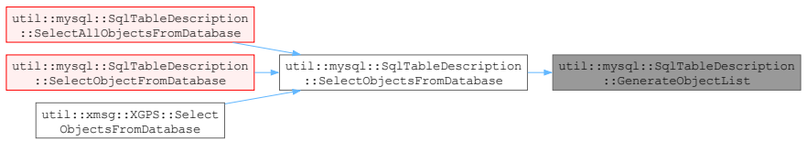
Returns

A vector of object T constructed from the given result set.

References EXC_THROW, and util::mysql::SqlTableDescription< T >::table_columns_.

Referenced by util::mysql::SqlTableDescription< T >::SelectObjectsFromDatabase().

Here is the caller graph for this function:



10.43.4.11 GeneratePreparedStatement()

```
static shared_ptr< PreparedStatement > util::mysql::MysqlUtils::GeneratePreparedStatement (
            Connection * con,
            const string & statement)  [inline], [static], [inherited]
```

Generate a prepared SQL statement object for the given SQL statement.

Parameters

in	*con*	A pointer to the SQL connection object. This must not be NULL.
in	*statement*	The SQL statement to prepare. This must be a well-formed SQL string.

Exceptions

log::MessageException	if 'con' is NULL, 'statement' is not a valid SQL string, or if an error occurs.

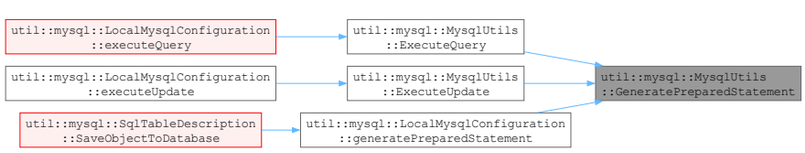
Returns

A prepared SQL statement object generated from the given SQL statement.

References CATCH_BLOCK, and EXC_THROW.

Referenced by util::mysql::MysqlUtils::ExecuteQuery(), util::mysql::MysqlUtils::ExecuteUpdate(), and util::mysql::LocalMysqlConfiguration:

Here is the caller graph for this function:



10.43.4.12 generatePreparedStatement()

```
std::shared_ptr< sql::PreparedStatement > util::mysql::LocalMysqlConfiguration::generatePrepared↩
Statement (
            const string & statement)  [static], [inherited]
```

Generate a prepared SQL statement object for the given SQL statement.

Parameters

in	*statement*	The SQL statement to prepare. This must be a well-formed SQL string.

Exceptions

log::MessageException	if 'statement' is not a valid SQL string or if an error occurs while generating the prepared statement object..

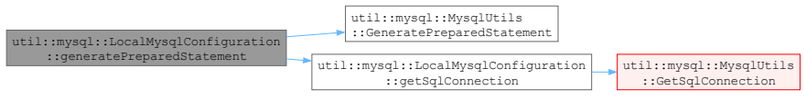
Returns

A prepared SQL statement object generated from the given SQL statement.

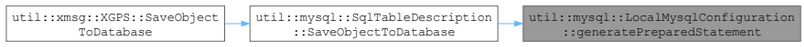
References util::mysql::MysqlUtils::GeneratePreparedStatement(), and util::mysql::LocalMysqlConfiguration::getSqlConnection().

Referenced by util::mysql::SqlTableDescription< T >::SaveObjectToDatabase().

Here is the call graph for this function:



Here is the caller graph for this function:



10.43.4.13 generateSelectAllStatement()

```
template<class T >
string util::mysql::SqlTableDescription< T >::generateSelectAllStatement ()  [inline]
```

Generate the "select all" SQL statement for this table.

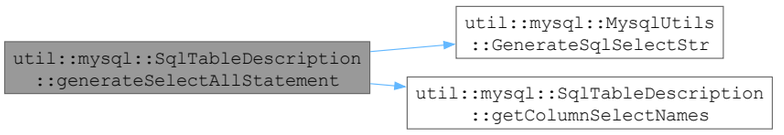
Returns

The "select all" SQL statement for this table.

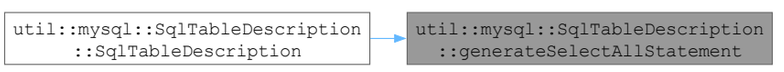
References util::mysql::MysqlUtils::GenerateSqlSelectStr(), util::mysql::SqlTableDescription< T >::getColumnSelectNames(), util::mysql::SqlTableDescription< T >::select_statement_, and util::mysql::SqlTableDescription< T >::table_name_.

Referenced by util::mysql::SqlTableDescription< T >::SqlTableDescription().

Here is the call graph for this function:



Here is the caller graph for this function:



10.43.4.14 GenerateSqlDeleteStr()

```
static string util::mysql::MysqlUtils::GenerateSqlDeleteStr (
            const string & from,
            const string & where = "")  [inline], [static], [inherited]
```

Generate an SQL DELETE statement.

Warning

Without a `where` clause, the statement deletes every row in the table.

Parameters

in	*from*	The name of the table to delete from.
in	*where*	The SQL "where-condition" clause (empty = all rows).

Returns

The SQL statement.

10.43.4.15 GenerateSqlSelectStr()

```
static string util::mysql::MysqlUtils::GenerateSqlSelectStr (
            const string & what,
            const string & from,
            const string & where = "",
            const string & groupBy = "",
            const string & orderBy = "",
            unsigned int limit = 0,
            unsigned int offset = 0)  [inline], [static], [inherited]
```

Generate an SQL select statement.

Parameters

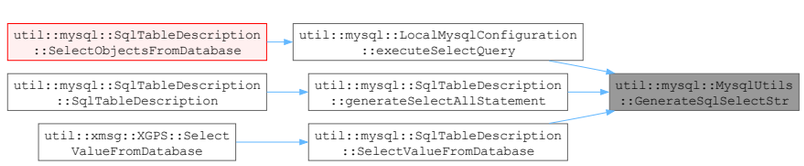
in	*what*	The SQL select statement select-expression clause.
in	*from*	The SQL select statement table-references clause.
in	*where*	The SQL select statement where-condition clause.
in	*groupBy*	The SQL select statement group-by clause.
in	*orderBy*	The SQL select statement order-by clause.
in	*limit*	The SQL select statement result set row limit value.
in	*offset*	The SQL select statement result set row offset.

Returns

The SQL select statement.

Referenced by util::mysql::LocalMysqlConfiguration::executeSelectQuery(), util::mysql::SqlTableDescription< T >::generateSelectAllState
and util::mysql::SqlTableDescription< T >::SelectValueFromDatabase().

Here is the caller graph for this function:



10.43.4.16 getColumnNames()

```
template<class T >
string util::mysql::SqlTableDescription< T >::getColumnNames () const  [inline]
```

Generate a string containing comma-separated, ordered list of column names for this SQL table from the column descriptions.

Note

This function is used to generate both INSERT and SELECT query statements for the SQL table.

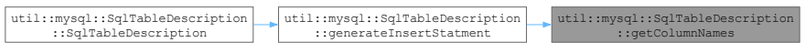
Returns

A string containing comma-separated, ordered list of column names for this SQL table.

References util::mysql::SqlTableDescription< T >::table_columns_.

Referenced by util::mysql::SqlTableDescription< T >::generateInsertStatment().

Here is the caller graph for this function:



10.43.4.17 getColumns()

```
template<class T >
vector< shared_ptr< SqlTableColumnDescriptionBase > > util::mysql::SqlTableDescription< T >↩
::getColumns () const  [inline]
```

Get the vector of shared pointers to ColumnDescription objects describing the columns of this SQL table.

Returns

The vector of shared pointers to ColumnDescription objects describing the columns of this SQL table.

References util::mysql::SqlTableDescription< T >::table_columns_.

10.43.4.18 getColumnSelectNames()

```
template<class T >
string util::mysql::SqlTableDescription< T >::getColumnSelectNames () const  [inline]
```

Generate a string containing comma-separated, ordered list of column names used for select statements for this SQL table from the column descriptions.

Note

This function is used to generate both INSERT and SELECT query statements for the SQL table.

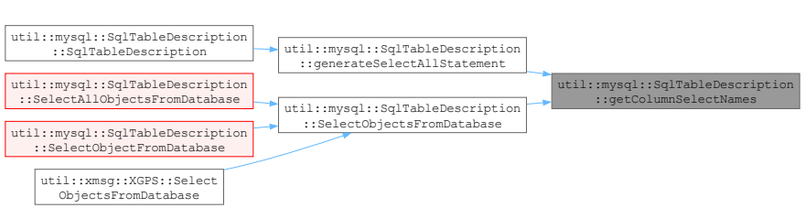
Returns

A string containing comma-separated, ordered list of column names for this SQL table.

References util::mysql::SqlTableDescription< T >::table_columns_.

Referenced by util::mysql::SqlTableDescription< T >::generateSelectAllStatement(), and util::mysql::SqlTableDescription< T >::SelectOb

Here is the caller graph for this function:



10.43.4.19 GetSqlConnection()

```
static thread_specific_ptr< Connection > & util::mysql::MysqlUtils::GetSqlConnection (
            string url,
            string user,
            string password,
            string schema)  [inline], [static], [inherited]
```

Get a thread-specific instance of a MySQL connection object.

A new connection will be created, if necessary.

Parameters

in	*url*	The MySQL host server qualified address.
in	*user*	The MySQL account username.
in	*password*	The MySQL account password.
in	*schema*	The MySQL account schema name.

Exceptions

log::MessageException	if an error occurs while creating a new thread-specific instance of the SQL connection object

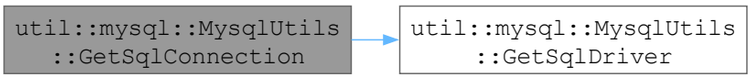
Returns

A thread-specific instance of a MySQL connection object.

References CATCH_BLOCK, util::mysql::MysqlUtils::GetSqlDriver(), and util::mysql::MysqlUtils::sql_connection_.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

Here is the call graph for this function:



Here is the caller graph for this function:

10.43.4.20 getSqlConnection()

```
sql::Connection * util::mysql::LocalMysqlConfiguration::getSqlConnection ()  [static], [protected],
[inherited]
```

Generate a thread-specific instance of a MySQL connection object.

A new connection will be created, if necessary.

Exceptions

log::MessageException	if an error occurs while generating the SQL connection object.

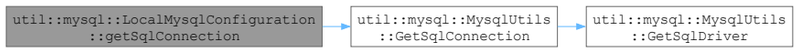
Returns

A thread-specific instance of a MySQL connection object.

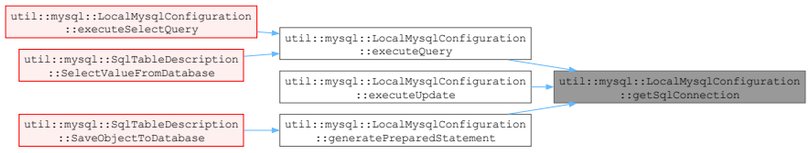
References util::mysql::LocalMysqlConfiguration::address_, CATCH_BLOCK, util::mysql::MysqlUtils::GetSqlConnection(), util::mysql::LocalMysqlConfiguration::password_, util::mysql::LocalMysqlConfiguration::port_, util::mysql::LocalMysqlConfiguration::protoc util::mysql::LocalMysqlConfiguration::schema_, and util::mysql::LocalMysqlConfiguration::user_.

Referenced by util::mysql::LocalMysqlConfiguration::executeQuery(), util::mysql::LocalMysqlConfiguration::executeUpdate(), and util::mysql::LocalMysqlConfiguration::generatePreparedStatement().

Here is the call graph for this function:



Here is the caller graph for this function:



10.43.4.21 GetSqlDriver()

```
static thread_specific_ptr< MySQL_Driver > & util::mysql::MysqlUtils::GetSqlDriver ()  [inline],
[static], [inherited]
```

Get a thread-specific instance of a MySQL driver object.

A new driver will created, if necessary.

Exceptions

log::MessageException	if an error occurs while creating a new thread-specific instance of the SQL driver object

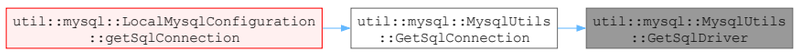
Returns

A thread-specific instance of a MySQL driver object.

References CATCH_BLOCK, and util::mysql::MysqlUtils::sql_driver_.

Referenced by util::mysql::MysqlUtils::GetSqlConnection().

Here is the caller graph for this function:



10.43.4.22 htoi()

```
static uint8_t util::stream::StreamUtils::htoi (
             char c)  [inline], [static], [inherited]
```

Get the integer value of a hexadecimal character.

Parameters

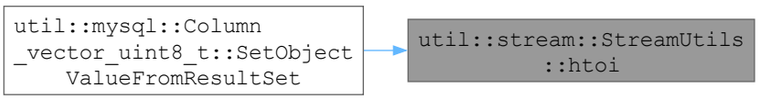
in	*c*	The hexadecimal character.

Returns

The integer value of a hexadecimal character, or 0 if c is not a hexadecimal character.

Referenced by util::mysql::Column_vector_uint8_t< C >::SetObjectValueFromResultSet().

Here is the caller graph for this function:



10.43.4.23 SaveObjectToDatabase()

```
template<class T >
void util::mysql::SqlTableDescription< T >::SaveObjectToDatabase (
            const T & obj) const  [inline]
```

Save a persistent entity to the SQL table.

Exceptions

util::log::MessageException	if an error occurs.

Parameters

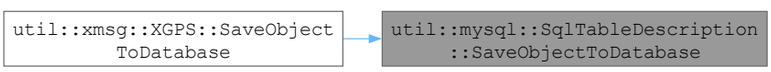
in	*obj*	The instance of the persistent entity to save to the SQL database table.

References CATCH_BLOCK, CATCH_NOTHROW, EXC_THROW, util::mysql::LocalMysqlConfiguration::generatePreparedStatement(), util::mysql::SqlTableDescription< T >::insert_on_duplicate_key_update_statement_, and util::mysql::SqlTableDescription< T >::table_co

Referenced by util::xmsg::XGPS::SaveObjectToDatabase().

Here is the call graph for this function:

Here is the caller graph for this function:



10.43.4.24 SelectAllObjectsFromDatabase()

```
template<class T >
vector< shared_ptr< T > > util::mysql::SqlTableDescription< T >::SelectAllObjectsFromDatabase (
            const string & groupBy = "",
            const string & orderBy = "",
            const unsigned int & limit = 0,
            const unsigned int & offset = 0) const  [inline]
```

Select all objects from this SQL database table.

Parameters

in	*groupBy*	The SQL select statement group-by clause.
in	*orderBy*	The SQL select statement order-by clause.
in	*limit*	The SQL select statement result set row limit value.
in	*offset*	The SQL select statement result set row offset.

Exceptions

util::log::MessageException	if an error occurs while querying the database.

Returns

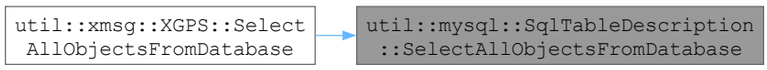
A vector of shared pointers to the object selected from the database.

References util::mysql::SqlTableDescription< T >::SelectObjectsFromDatabase().

Referenced by util::xmsg::XGPS::SelectAllObjectsFromDatabase().

Here is the call graph for this function:

Here is the caller graph for this function:



10.43.4.25 SelectObjectFromDatabase()

```
template<class T >
shared_ptr< T > util::mysql::SqlTableDescription< T >::SelectObjectFromDatabase (
            const string & where = "",
            const string & groupBy = "",
            const string & orderBy = "",
            const unsigned int & offset = 0) const  [inline]
```

Get a single object from the database.

Parameters

in	*where*	The SQL select statement where-condition clause.
in	*groupBy*	The SQL select statement group-by clause.
in	*orderBy*	The SQL select statement order-by clause.
in	*offset*	The SQL select statement result set row offset.

Exceptions

util::log::MessageException	if an error occurs while querying the database.

Returns

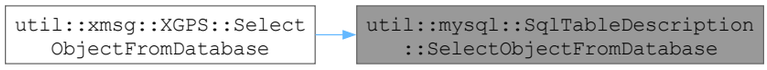
A shared pointer to the object selected from the database.

References CATCH_BLOCK, and util::mysql::SqlTableDescription< T >::SelectObjectsFromDatabase().

Referenced by util::xmsg::XGPS::SelectObjectFromDatabase().

Here is the call graph for this function:

Here is the caller graph for this function:



10.43.4.26 SelectObjectsFromDatabase()

```
template<class T >
vector< shared_ptr< T > > util::mysql::SqlTableDescription< T >::SelectObjectsFromDatabase (
            const string & where = "",
            const string & groupBy = "",
            const string & orderBy = "",
            const unsigned int & limit = 0,
            const unsigned int & offset = 0) const  [inline]
```

Select objects from this SQL database table.

Parameters

in	*where*	The SQL select statement where-condition clause.
in	*groupBy*	The SQL select statement group-by clause.
in	*orderBy*	The SQL select statement order-by clause.
in	*limit*	The SQL select statement result set row limit value.
in	*offset*	The SQL select statement result set row offset.

Exceptions

util::log::MessageException	if an error occurs while querying the database.

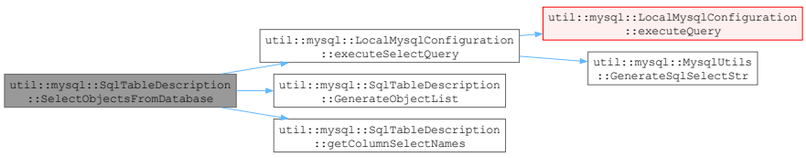
Returns

A vector of shared pointers to the object selected from the database.

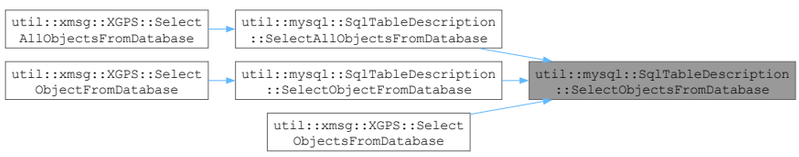
References CATCH_BLOCK, util::mysql::LocalMysqlConfiguration::executeSelectQuery(), util::mysql::SqlTableDescription< T >::Genera
util::mysql::SqlTableDescription< T >::getColumnSelectNames(), and util::mysql::SqlTableDescription< T >::table_name_.

Referenced by util::mysql::SqlTableDescription< T >::SelectAllObjectsFromDatabase(), util::mysql::SqlTableDescription< T >::SelectObj
and util::xmsg::XGPS::SelectObjectsFromDatabase().

Here is the call graph for this function:



Here is the caller graph for this function:



10.43.4.27 SelectValueFromDatabase()

```
template<class T >
template<typename TT >
TT util::mysql::SqlTableDescription< T >::SelectValueFromDatabase (
            const string & what,
            const string & where = "",
            const string & groupBy = "",
            const string & orderBy = "",
            const unsigned int & offset = 0) const  [inline]
```

Select a value from the SQL database table represented by this object.

Warning

The "select expression" clause must name a singular element when querying a singular value from the database.

Note

The result set of this query is always limited to a single result.

Parameters

in	*what*	The SQL select statement select-expression clause.
in	*where*	The SQL select statement where-condition clause.
in	*groupBy*	The SQL select statement group-by clause.
in	*orderBy*	The SQL select statement order-by clause.
in	*offset*	The SQL select statement result set row offset.

Exceptions

log::MessageException	if an error occurs while querying the database or if con == NULL.

Returns

The selected value from the SQL database table represented by this object.

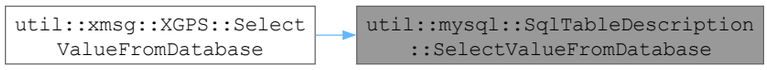
Todo Implement the functionality to parse values of type vector<uint8_t> from the result set of an SQL query.

References CATCH_BLOCK, EXC_THROW, util::mysql::LocalMysqlConfiguration::executeQuery(), util::mysql::MysqlUtils::GenerateSqlSe
util::mysql::SqlTableDescription< T >::table_columns_, and util::mysql::SqlTableDescription< T >::table_name_.

Referenced by util::xmsg::XGPS::SelectValueFromDatabase().

Here is the call graph for this function:

Here is the caller graph for this function:



10.43.4.28 values()

```
template<class T >
static string util::mysql::SqlTableDescription< T >::values (
             size_t n)  [inline], [static]
```

Generate a values string component of an SQL insert statement with n values.

Parameters

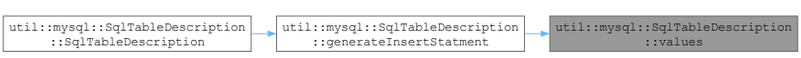
in	*n*	The number of values in the SQL insert statement.

Returns

A values string component of an SQL insert statement with n values.

Referenced by util::mysql::SqlTableDescription< T >::generateInsertStatment().

Here is the caller graph for this function:



10.43.5 Member Data Documentation

10.43.5.1 address_

```
std::string util::mysql::LocalMysqlConfiguration::address_  [static], [private], [inherited]
```

The SQL server qualified name or IP address string.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.43.5.2 insert_on_duplicate_key_update_statement_

```
template<class T >
string util::mysql::SqlTableDescription< T >::insert_on_duplicate_key_update_statement_  [private]
```

The insert statement for this SQL table.

Referenced by util::mysql::SqlTableDescription< T >::generateInsertStatment(), and util::mysql::SqlTableDescription< T >::SaveObjectT

10.43.5.3 password_

```
std::string util::mysql::LocalMysqlConfiguration::password_  [static], [private], [inherited]
```

The SQL server account password string.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.43.5.4 port_

```
unsigned int util::mysql::LocalMysqlConfiguration::port_  [static], [private], [inherited]
```

The SQL server port number.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.43.5.5 protocol_

```
std::string util::mysql::LocalMysqlConfiguration::protocol_  [static], [private], [inherited]
```

The protocol to use in the SQL server connection e.g.

"tcp".

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.43.5.6 schema_

```
std::string util::mysql::LocalMysqlConfiguration::schema_  [static], [private], [inherited]
```

The SQL server schema name.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.43.5.7 select_statement_

```
template<class T >
string util::mysql::SqlTableDescription< T >::select_statement_  [private]
```

The select all statement for this SQL table.

Referenced by util::mysql::SqlTableDescription< T >::generateSelectAllStatement().

10.43.5.8 sql_connection_

```
thread_specific_ptr<Connection> util::mysql::MysqlUtils::sql_connection_  [static], [private],
[inherited]
```

Tthe thread-specific pointer to MySQL connection object.

Thread-specific MySQL connection, released by `MysqlUtils::CleanupSqlConnection()` when the thread exits.

Referenced by util::mysql::MysqlUtils::CloseSqlConnection(), and util::mysql::MysqlUtils::GetSqlConnection().

10.43.5.9 sql_driver_

```
thread_specific_ptr<MySQL_Driver> util::mysql::MysqlUtils::sql_driver_  [static], [private],
[inherited]
```

The thread-specific pointer to MySQL driver object.

Thread-specific reference to the MySQL driver.

Its cleanup function, `MysqlUtils::CleanupSqlDriver()`, does not delete the shared driver instance.

Referenced by util::mysql::MysqlUtils::GetSqlDriver().

10.43.5.10 table_columns_

```
template<class T >
vector<shared_ptr<SqlTableColumnDescriptionBase> > util::mysql::SqlTableDescription< T >::table↩
_columns_  [private]
```

A vector of shared pointers to SqlTableColumnDescription objects describing the columns of this SQL table.

Referenced by util::mysql::SqlTableDescription< T >::SqlTableDescription(), util::mysql::SqlTableDescription< T >::generateInsertStatme
util::mysql::SqlTableDescription< T >::GenerateObjectList(), util::mysql::SqlTableDescription< T >::getColumnNames(),
util::mysql::SqlTableDescription< T >::getColumns(), util::mysql::SqlTableDescription< T >::getColumnSelectNames(),
util::mysql::SqlTableDescription< T >::SaveObjectToDatabase(), and util::mysql::SqlTableDescription< T >::SelectValueFromDatabase()

10.43.5.11 table_name_

```
template<class T >
string util::mysql::SqlTableDescription< T >::table_name_  [private]
```

The name of this SQL table.

Referenced by util::mysql::SqlTableDescription< T >::generateInsertStatment(), util::mysql::SqlTableDescription< T >::generateSelectAl
util::mysql::SqlTableDescription< T >::SelectObjectsFromDatabase(), and util::mysql::SqlTableDescription< T >::SelectValueFromDatab

10.43.5.12 user_

```
std::string util::mysql::LocalMysqlConfiguration::user_  [static], [private], [inherited]
```

The SQL server account username string.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

The documentation for this class was generated from the following file:

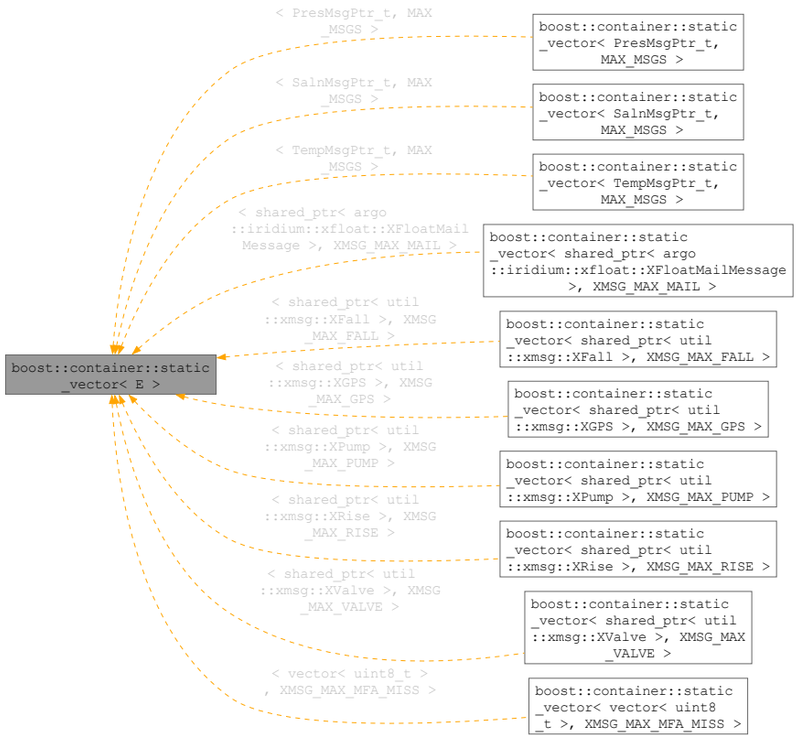
- utility/SqlTableDescription.h

10.44 boost::container::static_vector< E > Class Template Reference

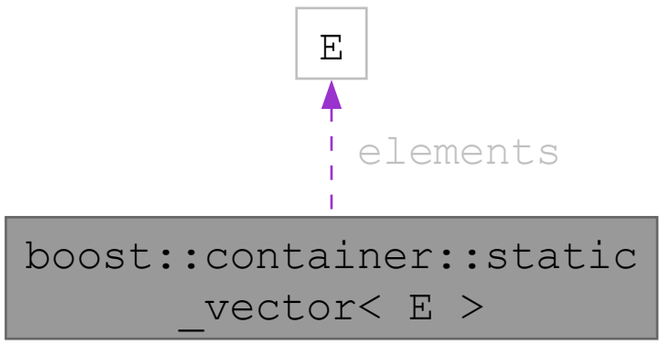
Dummy template class used for configuring Doxygen to interpret boost::container::static_vector correctly.

`#include <AdditionalDocumentation.h>`

Inheritance diagram for boost::container::static_vector< E >:



Collaboration diagram for boost::container::static_vector< E >:



Private Attributes

- E elements [ ]

  The elements.

10.44.1 Detailed Description

template<class E>
class boost::container::static_vector< E >

Dummy template class used for configuring Doxygen to interpret boost::container::static_vector correctly.

10.44.2 Member Data Documentation

10.44.2.1 elements

```
template<class E >
E boost::container::static_vector< E >::elements[]  [private]
```

The elements.

The documentation for this class was generated from the following file:

- AdditionalDocumentation.h

10.45 util::stream::StreamUtils Class Reference

Class providing useful functions for manipulating I/O streams.

`#include <StreamUtils.h>`

Inheritance diagram for util::stream::StreamUtils:

Static Public Member Functions

- static uint8_t htoi (char c)

  Get the integer value of a hexadecimal character.

10.45.1 Detailed Description

Class providing useful functions for manipulating I/O streams.

10.45.2 Member Function Documentation

10.45.2.1 htoi()

```
static uint8_t util::stream::StreamUtils::htoi (
            char c)  [inline], [static]
```

Get the integer value of a hexadecimal character.

Parameters

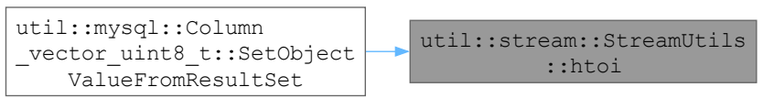
in	*c*	The hexadecimal character.

Returns

The integer value of a hexadecimal character, or 0 if c is not a hexadecimal character.

Referenced by util::mysql::Column_vector_uint8_t< C >::SetObjectValueFromResultSet().

Here is the caller graph for this function:



The documentation for this class was generated from the following file:

- utility/StreamUtils.h

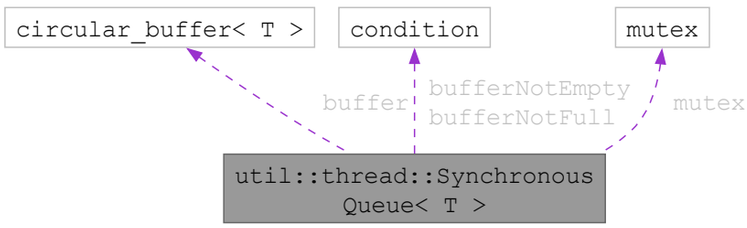
10.46 util::thread::SynchronousQueue< T > Class Template Reference

Synchronous queue template.

`#include <SynchronousQueue.h>`

Inheritance diagram for util::thread::SynchronousQueue< T >:

Collaboration diagram for util::thread::SynchronousQueue< T >:



Public Member Functions

- SynchronousQueue (size_t n)
  Constructor.
- bool **contains** (const T &elem) ATTR_DEPRECATED
- bool empty () const ATTR_DEPRECATED
  Is the queue empty?
- bool full () const ATTR_DEPRECATED
  Is the queue full?
- bool pop (T &elem, bool block)
  Pop element out of queue.
- bool push (const T &elem, bool block)
  Push element into queue.
- template<typename Iter >
  void push (Iter first, Iter last)
  Push multiple elements into queue.
- bool **pushIfNotContained** (const T &elem, bool block) ATTR_DEPRECATED
- size_t size () const ATTR_DEPRECATED
  Get the number of elements currently in the queue.

Private Attributes

- boost::circular_buffer< T > buffer

  Circular buffer for synchronized data storage.
- boost::condition bufferNotEmpty

  Conditional variable indicating that the buffer is not empty.
- boost::condition bufferNotFull

  Conditional variable indicating that the buffer is not full.
- boost::mutex mutex

  Mutual exclusion lock for synchronizing operations.

10.46.1 Detailed Description

template<typename T>
class util::thread::SynchronousQueue< T >

Synchronous queue template.

This template provides synchronized access to the underlying circular buffer and may be implemented for data of arbitrary type.

10.46.2 Constructor & Destructor Documentation

10.46.2.1 SynchronousQueue()

```
template<typename T >
util::thread::SynchronousQueue< T >::SynchronousQueue (
            size_t n)  [inline]
```

Constructor.

Parameters

in	*n*	Capacity of the underlying circular buffer to set.

10.46.3 Member Function Documentation

10.46.3.1 empty()

```
template<typename T >
bool util::thread::SynchronousQueue< T >::empty () const  [inline]
```

Is the queue empty?

Returns

TRUE if the buffer is empty, or FALSE otherwise.

References util::thread::SynchronousQueue< T >::buffer.

10.46.3.2 full()

```
template<typename T >
bool util::thread::SynchronousQueue< T >::full () const  [inline]
```

Is the queue full?

Returns

TRUE if the buffer is full, or FALSE otherwise.

References util::thread::SynchronousQueue< T >::buffer.

10.46.3.3 pop()

```
template<typename T >
bool util::thread::SynchronousQueue< T >::pop (
            T & elem,
            bool block)  [inline]
```

Pop element out of queue.

Parameters

out	*elem*	The destination of element to pop out of the queue.
in	*block*	If TRUE, this function will block until the queue is not empty. If FALSE, this function will return FALSE if the queue is empty (and no element will be popped out of the queue).

Returns

TRUE if the element was successfully popped out of the queue, or FALSE otherwise.

References util::thread::SynchronousQueue< T >::buffer, util::thread::SynchronousQueue< T >::bufferNotEmpty, and util::thread::SynchronousQueue< T >::bufferNotFull.

10.46.3.4 push() [1/2]

```
template<typename T >
bool util::thread::SynchronousQueue< T >::push (
            const T & elem,
            bool block)  [inline]
```

Push element into queue.

Parameters

in	*elem*	Element to push into queue.
in	*block*	If TRUE, this function will block until the queue is not full. If FALSE, this function will return FALSE if the queue is full (and the element will not be pushed into the queue).

Returns

TRUE if the element was successfully pushed into the queue, or FALSE otherwise.

References util::thread::SynchronousQueue< T >::buffer, util::thread::SynchronousQueue< T >::bufferNotEmpty, and util::thread::SynchronousQueue< T >::bufferNotFull.

10.46.3.5 push() [2/2]

```
template<typename T >
template<typename Iter >
void util::thread::SynchronousQueue< T >::push (
            Iter first,
            Iter last)  [inline]
```

Push multiple elements into queue.

Parameters

in	*first,last*	Input iterators to the initial and final positions in a range. The range used is [first,last), which includes all the elements between first and last, including the element pointed by first but not the element pointed by last.

References util::thread::SynchronousQueue< T >::buffer, util::thread::SynchronousQueue< T >::bufferNotEmpty, and util::thread::SynchronousQueue< T >::bufferNotFull.

10.46.3.6 size()

```
template<typename T >
size_t util::thread::SynchronousQueue< T >::size () const  [inline]
```

Get the number of elements currently in the queue.

Returns

The number of elements currently in the queue.

References util::thread::SynchronousQueue< T >::buffer.

10.46.4 Member Data Documentation

10.46.4.1 buffer

```
template<typename T >
boost::circular_buffer<T> util::thread::SynchronousQueue< T >::buffer  [private]
```

Circular buffer for synchronized data storage.

Referenced by util::thread::SynchronousQueue< T >::empty(), util::thread::SynchronousQueue< T >::full(), util::thread::SynchronousQu
util::thread::SynchronousQueue< T >::push(), util::thread::SynchronousQueue< T >::push(), and util::thread::SynchronousQueue< T >

10.46.4.2 bufferNotEmpty

```
template<typename T >
boost::condition util::thread::SynchronousQueue< T >::bufferNotEmpty  [private]
```

Conditional variable indicating that the buffer is not empty.

Referenced by util::thread::SynchronousQueue< T >::pop(), util::thread::SynchronousQueue< T >::push(), and util::thread::SynchronousQueue< T >::push().

10.46.4.3 bufferNotFull

```
template<typename T >
boost::condition util::thread::SynchronousQueue< T >::bufferNotFull  [private]
```

Conditional variable indicating that the buffer is not full.

Referenced by util::thread::SynchronousQueue< T >::pop(), util::thread::SynchronousQueue< T >::push(), and util::thread::SynchronousQueue< T >::push().

10.46.4.4 mutex

```
template<typename T >
boost::mutex util::thread::SynchronousQueue< T >::mutex  [mutable], [private]
```

Mutual exclusion lock for synchronizing operations.

The documentation for this class was generated from the following file:

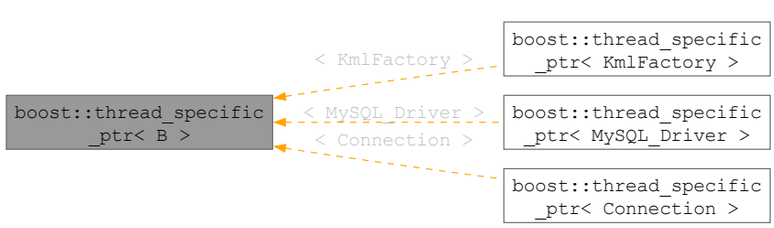
- utility/SynchronousQueue.h

10.47 boost::thread_specific_ptr< B > Class Template Reference

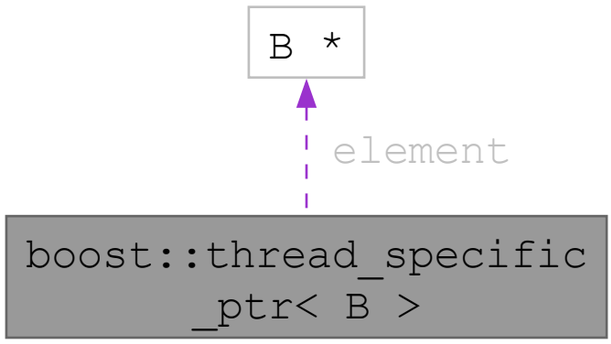
Dummy template class used for configuring Doxygen to interpret boost::thread_specific_ptr objects as plain pointers.

```
#include <AdditionalDocumentation.h>
```

Inheritance diagram for boost::thread_specific_ptr< B >:



Collaboration diagram for boost::thread_specific_ptr< B >:



Private Attributes

- B * element
  The pointer to the element.

10.47.1 Detailed Description

template<class B>
class boost::thread_specific_ptr< B >

Dummy template class used for configuring Doxygen to interpret boost::thread_specific_ptr objects as plain pointers.

10.47.2 Member Data Documentation

10.47.2.1 element

```
template<class B >
B* boost::thread_specific_ptr< B >::element  [private]
```

The pointer to the element.

The documentation for this class was generated from the following file:

- AdditionalDocumentation.h

10.48 util::thread::ThreadUtils Class Reference

Class providing useful functions for manipulating threads and processes.

```
#include <ThreadUtils.h>
```

Static Public Member Functions

- static std::string executeAndPipe (const char *cmd)
  Invoke an executable on the local filesystem, pipe the output of the command back to this application .
- static std::string executeAndPipe (const std::string &cmd)
  Invoke an executable on the local filesystem, pipe the output of the command back to this application .

10.48.1 Detailed Description

Class providing useful functions for manipulating threads and processes.

10.48.2 Member Function Documentation

10.48.2.1 executeAndPipe() [1/2]

```
static std::string util::thread::ThreadUtils::executeAndPipe (
            const char * cmd)  [inline], [static]
```

Invoke an executable on the local filesystem, pipe the output of the command back to this application .

Parameters

in	*cmd*	A pointer to the C-string containing the local system command to execute.

Exceptions

log::MessageException	if the pointer to the command string is null, the command string is zero length, or an error occurs while executing the external command or piping its output back to this application.

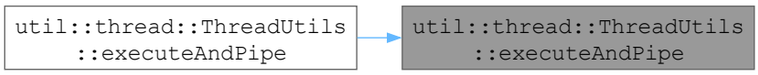
Returns

The output of the executed command.

References CATCH_BLOCK, dbg1, and EXC_THROW.

Referenced by executeAndPipe().

Here is the caller graph for this function:



10.48.2.2 executeAndPipe() [2/2]

```
static std::string util::thread::ThreadUtils::executeAndPipe (
            const std::string & cmd)  [inline], [static]
```

Invoke an executable on the local filesystem, pipe the output of the command back to this application .

Parameters

in	*cmd*	A reference to the string containing the local system command to execute.

Exceptions

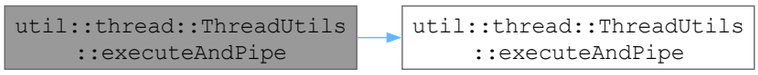
log::MessageException	if the pointer to the command string is null, the command string is zero length, or an error occurs while executing the external command or piping its output back to this application.

Returns

The output of the executed command.

References executeAndPipe().

Here is the call graph for this function:



The documentation for this class was generated from the following file:

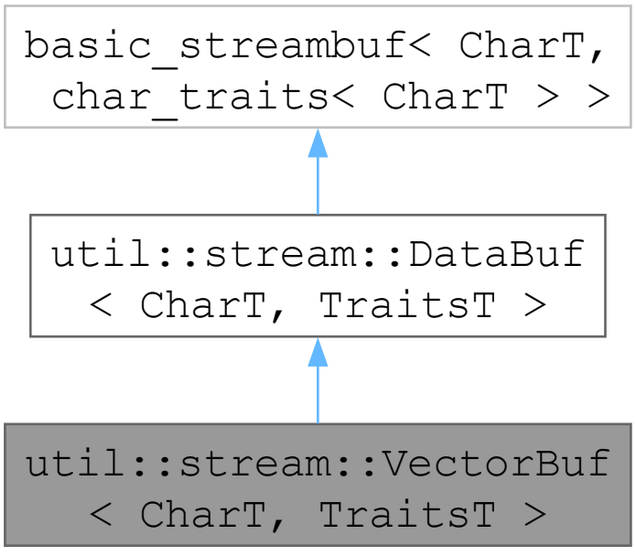
- utility/ThreadUtils.h

10.49 util::stream::VectorBuf< CharT, TraitsT > Class Template Reference

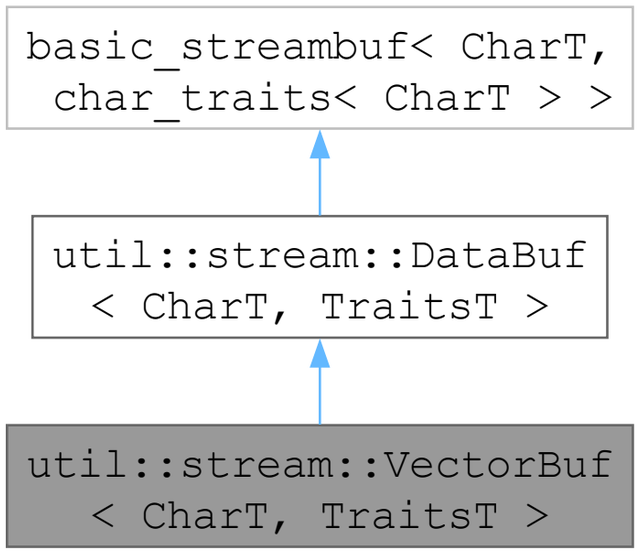
Template class for wrapping a vector<T> as a std::basic_streambuf<T>.

`#include <VectorBuf.h>`

Inheritance diagram for util::stream::VectorBuf< CharT, TraitsT >:



Collaboration diagram for util::stream::VectorBuf< CharT, TraitsT >:



Public Member Functions

- VectorBuf (const std::vector< CharT > &vec)
  Parameterized constructor.

10.49.1 Detailed Description

template<typename CharT, typename TraitsT = std::char_traits<CharT>>
class util::stream::VectorBuf< CharT, TraitsT >

Template class for wrapping a vector<T> as a std::basic_streambuf<T>.

10.49.2 Constructor & Destructor Documentation

10.49.2.1 VectorBuf()

```
template<typename CharT , typename TraitsT = std::char_traits<CharT>>
util::stream::VectorBuf< CharT, TraitsT >::VectorBuf (
            const std::vector< CharT > & vec)  [inline]
```

Parameterized constructor.

Parameters

in	*vec*	Vector to wrap.

The documentation for this class was generated from the following file:

- utility/VectorBuf.h

10.50 util::stream::VectorBuf_c< CharT > Class Template Reference

Template class for wrapping a vector<T> as a std::basic_streambuf<char>.

`#include <VectorBuf.h>`

Inheritance diagram for util::stream::VectorBuf_c< CharT >:

Collaboration diagram for util::stream::VectorBuf_c< CharT >:

Public Member Functions

- VectorBuf_c (const std::vector< CharT > &vec)
  Parameterized constructor.

10.50.1 Detailed Description

template<typename CharT>
class util::stream::VectorBuf_c< CharT >

Template class for wrapping a vector<T> as a std::basic_streambuf<char>.

10.50.2 Constructor & Destructor Documentation

10.50.2.1 VectorBuf_c()

```
template<typename CharT >
util::stream::VectorBuf_c< CharT >::VectorBuf_c (
            const std::vector< CharT > & vec)  [inline]
```

Parameterized constructor.

Parameters

in	*vec*	Vector to wrap.

The documentation for this class was generated from the following file:

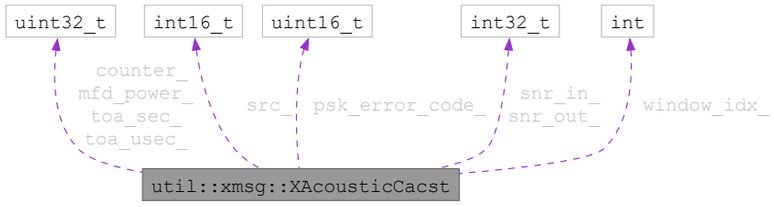
- utility/VectorBuf.h

10.51 util::xmsg::XAcousticCacst Class Reference

An acoustic reception (CACST) record.

`#include <XAcousticPosition.h>`

Collaboration diagram for util::xmsg::XAcousticCacst:



Public Member Functions

- XAcousticCacst ()
  Default constructor.
- XAcousticCacst (const vector< uint8_t > &data)
  Construct from message data.
- ~XAcousticCacst ()
  Destructor.
- uint32_t **counter** () const
- void DecodeXMsgData (const vector< uint8_t > &data)
  Decode the acoustic position CACST X-Message from the binary message data.
- uint32_t **mfd_power** () const
- uint16_t **psk_error_code** () const
- void **set_counter** (uint32_t counter)
- void **set_mfd_power** (uint32_t mfd_power)
- void **set_psk_error_code** (uint16_t psk_error_code)
- void **set_snr_in** (int32_t snr_in)
- void **set_snr_out** (int32_t snr_out)
- void **set_src** (int16_t src)
- void **set_toa_sec** (uint32_t toa_sec)
- void **set_toa_usec** (uint32_t toa_usec)
- void **set_window_idx** (int idx)
- int32_t **snr_in** () const
- float **snr_in_as_float** ()
- int32_t **snr_out** () const
- float **snr_out_as_float** ()
- int16_t **src** () const
- string **toa_iso8601_string** () const
- ptime **toa_ptime** () const
- uint32_t **toa_sec** () const
- uint32_t **toa_usec** () const
- int **window_idx** () const

Private Attributes

- uint32_t **counter_**
- uint32_t mfd_power_
  Matched filter peak value, in dB.
- uint16_t psk_error_code_
  PSK error code enumeration.
- int32_t snr_in_
  Input SNR, in 1/10 dB.
- int32_t snr_out_
  SNR at output of equalizer, in 1/10 dB.
- int16_t src_
  Acoustic message source ID.
- uint32_t toa_sec_
  Time of arrival, in UNIX time.
- uint32_t toa_usec_
  Microsecond of arrival time.
- int **window_idx_**

Friends

- class **XAcousticPosition**

10.51.1 Detailed Description

An acoustic reception (CACST) record.

An instance of this class represents a single acoustic signal received by the instrument, decoded from an acoustic position message (see XAcousticPosition): its counter, time of arrival (seconds and microseconds), source identifier, signal power, PSK error code, input and output signal-to-noise ratios, and the index of the listening window it was received in.

10.51.2 Constructor & Destructor Documentation

10.51.2.1 XAcousticCacst() [1/2]

```
util::xmsg::XAcousticCacst::XAcousticCacst ()  [inline]
```

Default constructor.

10.51.2.2 XAcousticCacst() [2/2]

```
util::xmsg::XAcousticCacst::XAcousticCacst (
            const vector< uint8_t > & data)  [inline]
```

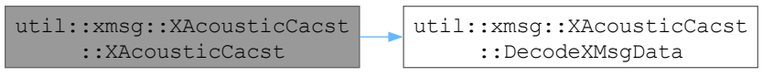
Construct from message data.

Parameters

in	*data*	The binary message data to decode.

References DecodeXMsgData().

Here is the call graph for this function:



10.51.2.3 ~XAcousticCacst()

```
util::xmsg::XAcousticCacst::~XAcousticCacst ()  [inline]
```

Destructor.

10.51.3 Member Function Documentation

10.51.3.1 DecodeXMsgData()

```
void util::xmsg::XAcousticCacst::DecodeXMsgData (
            const vector< uint8_t > & data)  [inline]
```

Decode the acoustic position CACST X-Message from the binary message data.

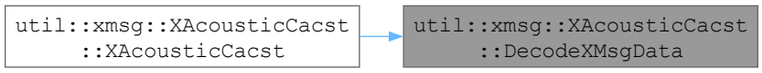
Parameters

in	*data*	The binary message data to decode.

References CATCH_BLOCK, u16, and u32.

Referenced by XAcousticCacst().

Here is the caller graph for this function:



10.51.4 Member Data Documentation

10.51.4.1 mfd_power_

```
uint32_t util::xmsg::XAcousticCacst::mfd_power_  [private]
```

Matched filter peak value, in dB.

10.51.4.2 psk_error_code_

```
uint16_t util::xmsg::XAcousticCacst::psk_error_code_  [private]
```

PSK error code enumeration.

10.51.4.3 snr_in_

```
int32_t util::xmsg::XAcousticCacst::snr_in_  [private]
```

Input SNR, in 1/10 dB.

10.51.4.4 snr_out_

```
int32_t util::xmsg::XAcousticCacst::snr_out_  [private]
```

SNR at output of equalizer, in 1/10 dB.

10.51.4.5 src_

```
int16_t util::xmsg::XAcousticCacst::src_  [private]
```

Acoustic message source ID.

10.51.4.6 toa_sec_

```
uint32_t util::xmsg::XAcousticCacst::toa_sec_  [private]
```

Time of arrival, in UNIX time.

10.51.4.7 toa_usec_

```
uint32_t util::xmsg::XAcousticCacst::toa_usec_  [private]
```

Microsecond of arrival time.

The documentation for this class was generated from the following file:

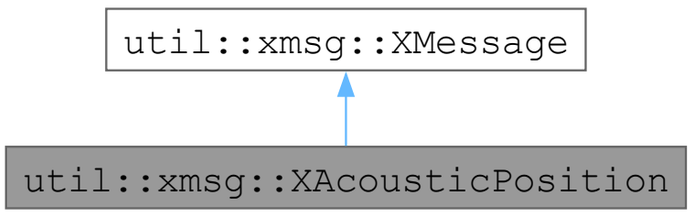
- data/xmsg/XAcousticPosition.h

10.52 util::xmsg::XAcousticPosition Class Reference

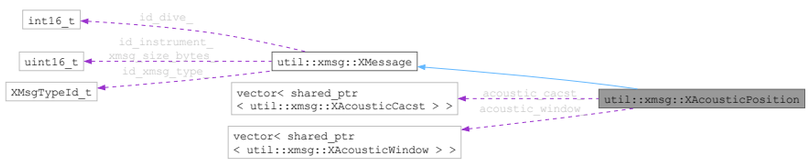
Class representing an acoustic position X-Message.

```
#include <XAcousticPosition.h>
```

Inheritance diagram for util::xmsg::XAcousticPosition:



Collaboration diagram for util::xmsg::XAcousticPosition:



Public Member Functions

- XAcousticPosition (int16_t dive, uint16_t whoi, const vector< uint8_t > &data)

  Parameterized constructor.
- ~XAcousticPosition ()

  Destructor.
- void AssignWindowIndices (void)

  Number the acoustic listening windows and assign each ping to its window.
- const uint8_t * CheckXMsgData (const vector< uint8_t > &data, int id, int minBytes, bool parseSize=true)

  Check the X-message for the correct ID, minimum size, and message terminator.
- void DecodeXMsgData (const vector< uint8_t > &data)

  Decode the acoustic position X-Message from the binary message data.
- const string GenerateListingString () const

  Generate a listing string representation of the acoustic position message.
- int16_t id_dive () const

  Get the dive or cycle number of this X-Message.
- uint16_t id_instrument () const

  Get the unique instrument identifier of this X-Message.
- XMsgTypeId_t id_xmsg_type () const

  Get the enumerated X-Message type identifier.
- uint16_t xmsg_size_bytes () const

  Get the size of the X-Message, in bytes.

Protected Member Functions

- void set_id_dive (int16_t id_dive)

  Set the dive or cycle number of this message.
- void set_id_instrument (uint16_t id_instrument)

  Set the unique instrument identifier.
- void set_id_xmsg_type (XMsgTypeId_t id_xmsg_type)

  Set the X-Message type identifier.
- void set_xmsg_size_bytes (uint16_t xmsg_size_bytes)

  Set the size of the X-Message, in bytes.

Private Attributes

- vector< shared_ptr< XAcousticCacst > > **acoustic_cacst_**
- vector< shared_ptr< XAcousticWindow > > **acoustic_window_**
- int16_t id_dive_

  The dive or cycle number of this message.
- uint16_t id_instrument_

  The unique WHOI identifier of this instrument.
- XMsgTypeId_t id_xmsg_type_

  The X-message type identifier.
- uint16_t xmsg_size_bytes_

  The size of the X message, in bytes.

10.52.1 Detailed Description

Class representing an acoustic position X-Message.

Note

This message type was added in X-Message version 2.0.

This message type is used by marginal ice-zone (MIZ) floats only.

10.52.2 Constructor & Destructor Documentation

10.52.2.1 XAcousticPosition()

```
util::xmsg::XAcousticPosition::XAcousticPosition (
            int16_t dive,
            uint16_t whoi,
            const vector< uint8_t > & data)  [inline]
```

Parameterized constructor.

Parameters

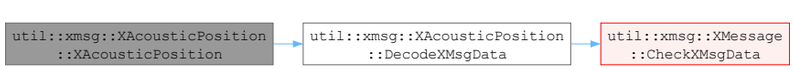
in	*dive*	The dive or cycle number of this message.
in	*whoi*	The unique WHOI identifier.
in	*data*	The binary X-message data to decode.

Exceptions

util::log::MessageException	if an decode error occurs.

References DecodeXMsgData().

Here is the call graph for this function:



10.52.2.2 ~XAcousticPosition()

```
util::xmsg::XAcousticPosition::~XAcousticPosition ()  [inline]
```

Destructor.

10.52.3 Member Function Documentation

10.52.3.1 AssignWindowIndices()

```
void util::xmsg::XAcousticPosition::AssignWindowIndices (
            void )  [inline]
```

Number the acoustic listening windows and assign each ping to its window.

Windows are numbered in order, starting at 0. Each ping (CACST record) whose time of arrival falls within a window's start and end times is given that window's index, and the window's receive count is incremented. A ping outside every window keeps its window index unchanged.

Exceptions

util::log::MessageException	if an error occurs.

References CATCH_BLOCK.

10.52.3.2 CheckXMsgData()

```
const uint8_t * util::xmsg::XMessage::CheckXMsgData (
            const vector< uint8_t > & data,
            int id,
            int minBytes,
            bool parseSize = true)  [inline], [inherited]
```

Check the X-message for the correct ID, minimum size, and message terminator.

Parameters

in	*data*	The X-message binary data to check.
in	*id*	The expected ID of the X-message. If this field is less that 0, then the ID check will be skipped.
in	*minBytes*	The minimum number of bytes expected in a valid message.
in	*parseSize*	If TRUE, then the message size field will be parsed from the X-Message. Otherwise, the size field will not be parsed.

Exceptions

util::log::MessageException	if • the message does not contain the expected ID, • the message does not contain the minimum number of expected bytes, • the message does not contain a terminator (';') in the correct location, or, • any other error occurs.

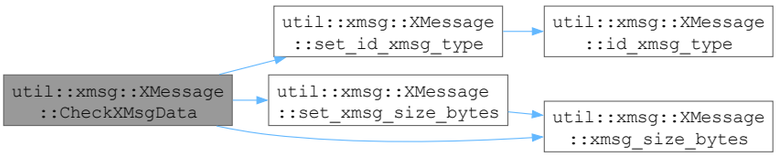
Returns

A pointer to the underlying data buffer of data.

References CATCH_BLOCK, EXC_THROW, lwrn, util::xmsg::XMessage::set_id_xmsg_type(), util::xmsg::XMessage::set_xmsg_size_byte
u16, and util::xmsg::XMessage::xmsg_size_bytes().

Referenced by DecodeXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XDriftPressure::DecodeXMsgData(),
util::xmsg::XDriftSalinity::DecodeXMsgData(), util::xmsg::XDriftTemperature::DecodeXMsgData(), util::xmsg::XEngAbort::DecodeXMsgDa
util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngBIT::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(),
util::xmsg::XEngDiagnostic::DecodeXMsgData(), util::xmsg::XEngDrift::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(),
util::xmsg::XEngStandard::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XGPS::DecodeXMsgData(),
util::xmsg::XHiResPressure::DecodeXMsgData(), util::xmsg::XHiResSalinity::DecodeXMsgData(), util::xmsg::XHiResTemperature::Decod
util::xmsg::XMissionCmdEcho::DecodeXMsgData(), util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XProfilePressure:
util::xmsg::XProfileSalinity::DecodeXMsgData(), util::xmsg::XProfileTemperature::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgD
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the call graph for this function:



10.52.3.3 DecodeXMsgData()

```
void util::xmsg::XAcousticPosition::DecodeXMsgData (
            const vector< uint8_t > & data)  [inline], [virtual]
```

Decode the acoustic position X-Message from the binary message data.

Parameters

in	*data*	The binary message data to decode.

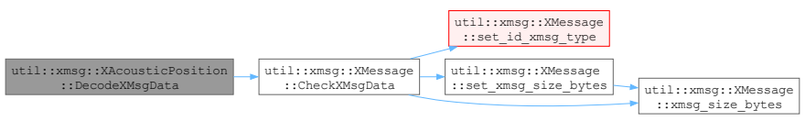
Todo Implement the functionality for decoding the acoustic position message.

Implements util::xmsg::XMessage.

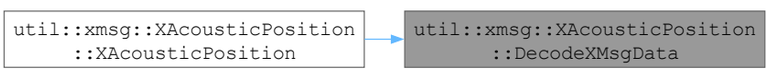
References CATCH_BLOCK, util::xmsg::XMessage::CheckXMsgData(), u08, and util::xmsg::XMSG_TYPE_ID_MIZ_ACOUSTIC.

Referenced by XAcousticPosition().

Here is the call graph for this function:



Here is the caller graph for this function:



10.52.3.4 GenerateListingString()

```
const string util::xmsg::XAcousticPosition::GenerateListingString () const  [inline]
```

Generate a listing string representation of the acoustic position message.

Returns

A listing string representation of the acoustic position message.

Todo Implement the functionality for generating a listing string for the acoustic position message.

References CATCH_BLOCK.

10.52.3.5 id_dive()

```
int16_t util::xmsg::XMessage::id_dive () const  [inline], [inherited]
```

Get the dive or cycle number of this X-Message.

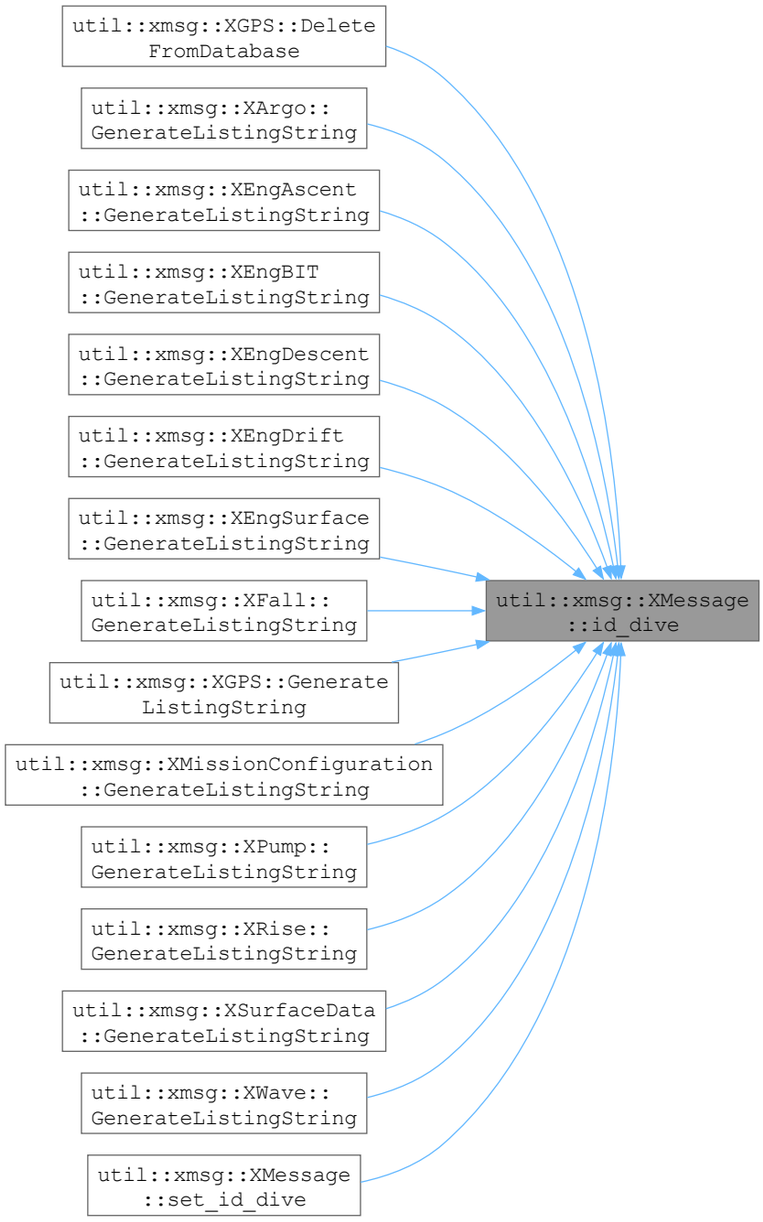
Returns

The dive or cycle number of this X-Message.

References util::xmsg::XMessage::id_dive_.

Referenced by util::xmsg::XGPS::DeleteFromDatabase(), util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateLis
util::xmsg::XEngBIT::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingStr
util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(),
util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing
util::xmsg::XSurfaceData::GenerateListingString(), util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_dive().

Here is the caller graph for this function:



10.52.3.6 id_instrument()

```
uint16_t util::xmsg::XMessage::id_instrument () const  [inline], [inherited]
```

Get the unique instrument identifier of this X-Message.

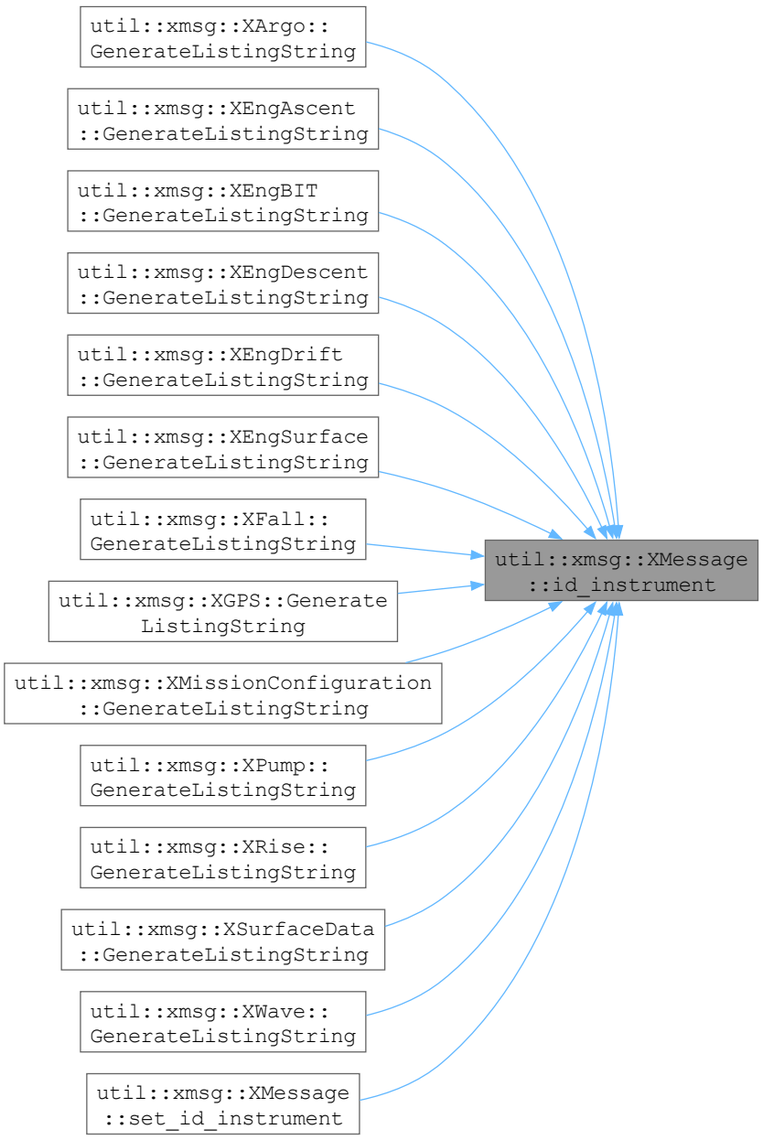
Returns

The unique instrument identifier of this X-Message.

References util::xmsg::XMessage::id_instrument_.

Referenced by util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngBIT::Generate
util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), util::xmsg::XEngSurface::GenerateListin
util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingS
util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(),
util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_instrument().

Here is the caller graph for this function:



10.52.3.7 id_xmsg_type()

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type () const  [inline], [inherited]
```

Get the enumerated X-Message type identifier.

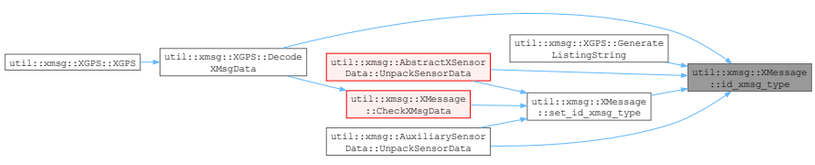
Returns

The enumerated X-Message type identifier.

References util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMessage::set_id_xmsg_typ
util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the caller graph for this function:



10.52.3.8 set_id_dive()

```
void util::xmsg::XMessage::set_id_dive (
            int16_t id_dive)  [inline], [protected], [inherited]
```

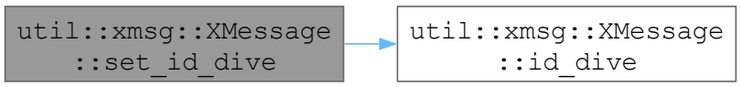
Set the dive or cycle number of this message.

Parameters

in	*id_dive*	The dive or cycle number of this message to set.

References util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::id_dive_.

Here is the call graph for this function:



10.52.3.9 set_id_instrument()

```
void util::xmsg::XMessage::set_id_instrument (
            uint16_t id_instrument)  [inline], [protected], [inherited]
```

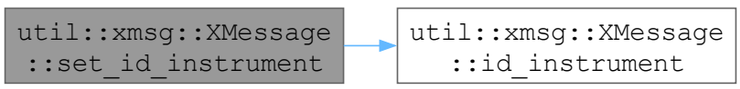
Set the unique instrument identifier.

Parameters

in	*id_instrument*	The unique instrument identifier to set.

References util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::id_instrument_.

Here is the call graph for this function:



10.52.3.10 set_id_xmsg_type()

```
void util::xmsg::XMessage::set_id_xmsg_type (
            XMsgTypeId_t id_xmsg_type)  [inline], [protected], [inherited]
```

Set the X-Message type identifier.

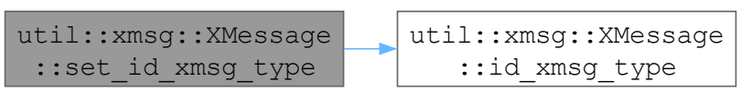
Parameters

in	*id_xmsg_type*	The X-Message type identifier to set.

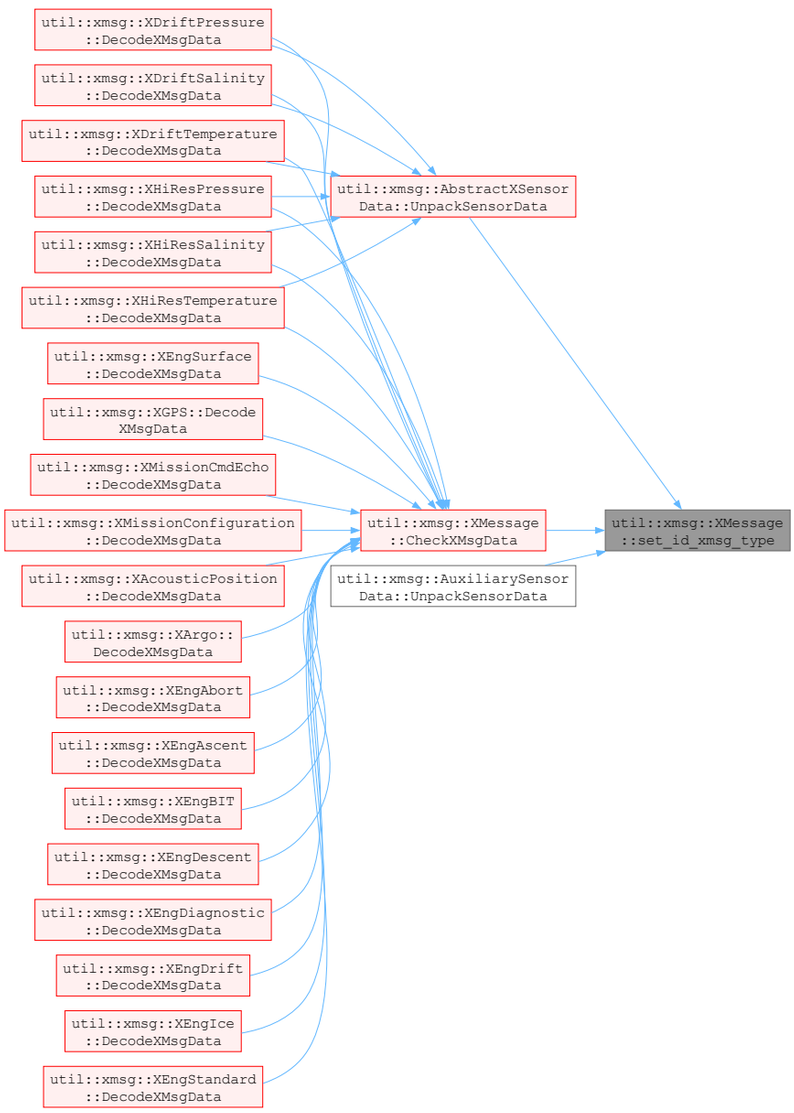
References util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.52.3.11 set_xmsg_size_bytes()

```
void util::xmsg::XMessage::set_xmsg_size_bytes (
            uint16_t xmsg_size_bytes)  [inline], [protected], [inherited]
```

Set the size of the X-Message, in bytes.

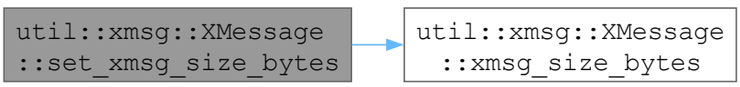
Parameters

in	*xmsg_size_bytes*	The size of the X-Message to set, in bytes.

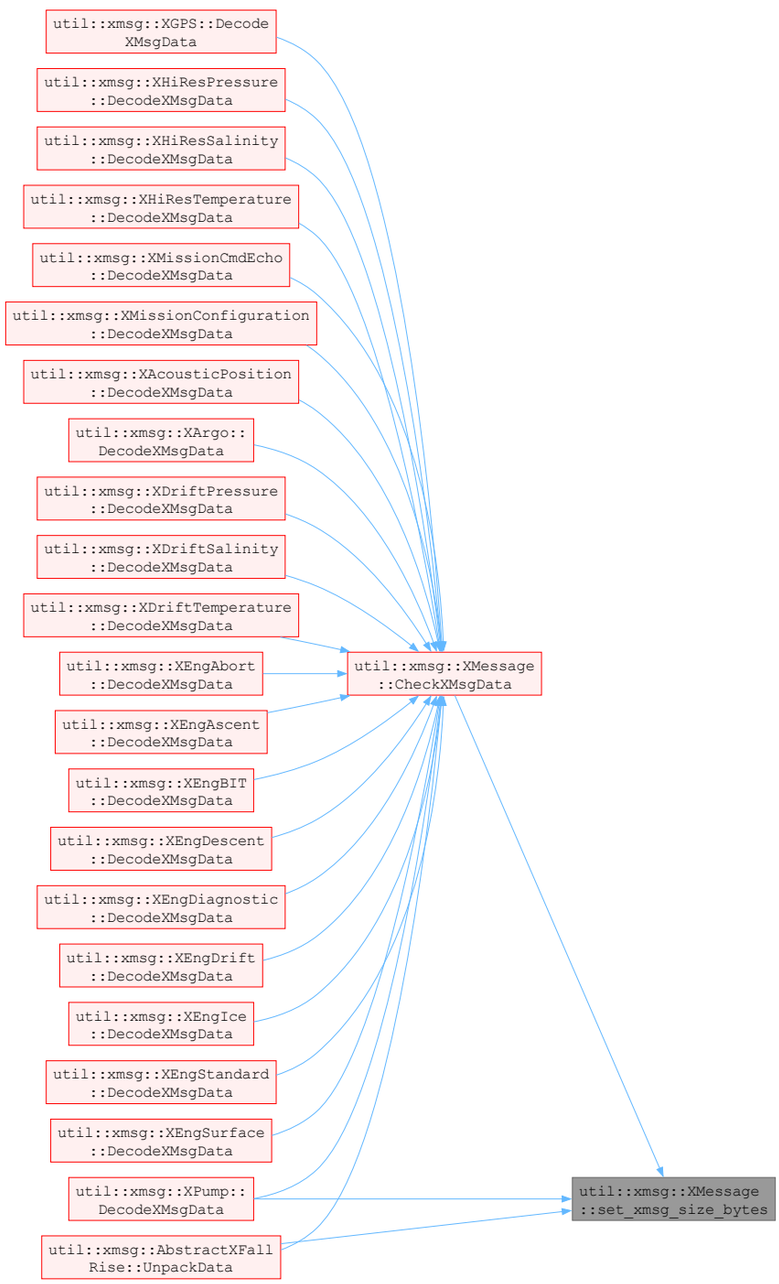
References util::xmsg::XMessage::xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XPump::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::Unpa

Here is the call graph for this function:



Here is the caller graph for this function:



10.52.3.12 xmsg_size_bytes()

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes () const  [inline], [inherited]
```

Get the size of the X-Message, in bytes.

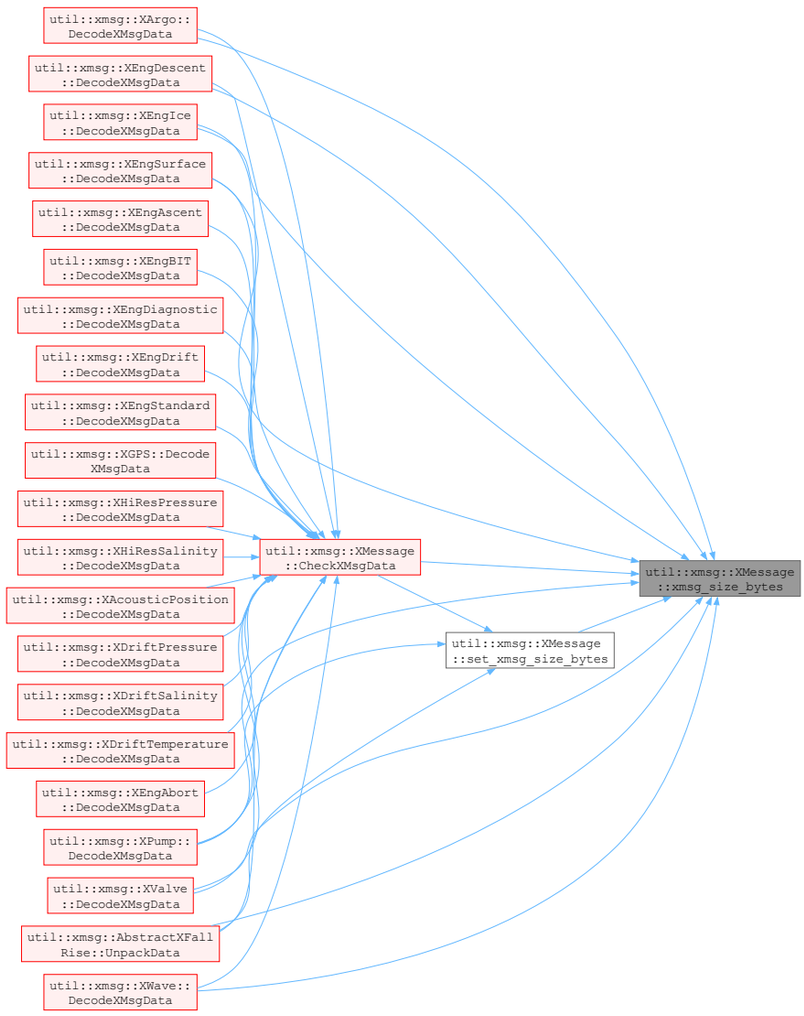
Returns

The size of the X-Message, in bytes.

References util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgI
util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(),
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), util::xmsg::XMessage::set_xmsg_size_bytes(),
and util::xmsg::AbstractXFallRise::UnpackData().

Here is the caller graph for this function:



10.52.4 Member Data Documentation

10.52.4.1 id_dive_

```
int16_t util::xmsg::XMessage::id_dive_  [private], [inherited]
```

The dive or cycle number of this message.

Referenced by util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::set_id_dive().

10.52.4.2 id_instrument_

```
uint16_t util::xmsg::XMessage::id_instrument_  [private], [inherited]
```

The unique WHOI identifier of this instrument.

Referenced by util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::set_id_instrument().

10.52.4.3 id_xmsg_type_

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type_  [private], [inherited]
```

The X-message type identifier.

Referenced by util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::set_id_xmsg_type().

10.52.4.4 xmsg_size_bytes_

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes_  [private], [inherited]
```

The size of the X message, in bytes.

Referenced by util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes().

The documentation for this class was generated from the following file:

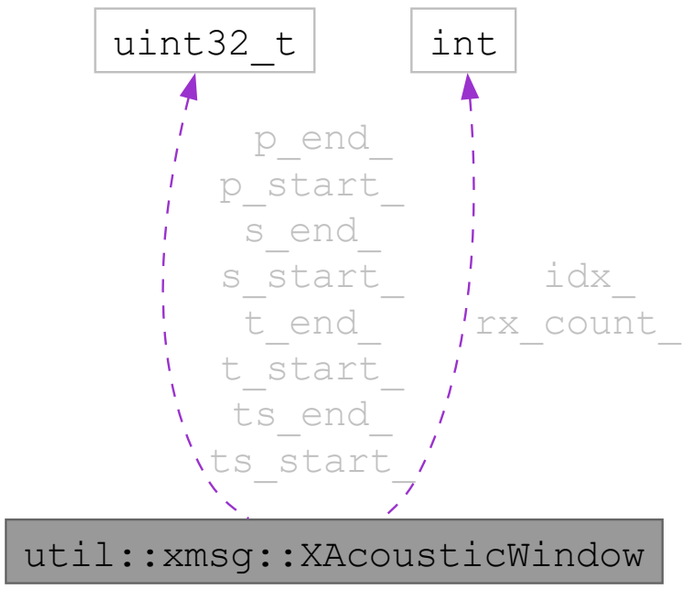
- data/xmsg/XAcousticPosition.h

10.53 util::xmsg::XAcousticWindow Class Reference

An acoustic listening window.

```
#include <XAcousticPosition.h>
```

Collaboration diagram for util::xmsg::XAcousticWindow:



Public Member Functions

- XAcousticWindow ()
  Default constructor.
- XAcousticWindow (const vector< uint8_t > &data)
  Construct from message data.
- ~XAcousticWindow ()
  Destructor.
- void DecodeXMsgData (const vector< uint8_t > &data)
  Decode the acoustic position CACST X-Message from the binary message data.
- int **idx** () const
- uint32_t **p_end** () const
- uint32_t **p_start** () const
- int **rx_count** () const
- uint32_t **s_end** () const
- uint32_t **s_start** () const
- void **set_idx** (int idx)
- void **set_p_end** (uint32_t p_end)
- void **set_p_start** (uint32_t p_start)
- void **set_rx_count** (int rx_count)
- void **set_s_end** (uint32_t s_end)
- void **set_s_start** (uint32_t s_start)
- void **set_t_end** (uint32_t t_end)
- void **set_t_start** (uint32_t t_start)
- void **set_ts_end** (uint32_t ts_end)
- void **set_ts_start** (uint32_t ts_start)
- uint32_t **t_end** () const
- uint32_t **t_start** () const
- uint32_t **ts_end** () const
- string **ts_end_iso8601_string** () const
- ptime **ts_end_ptime** () const
- uint32_t **ts_start** () const
- string **ts_start_iso8601_string** () const
- ptime **ts_start_ptime** () const

Private Attributes

- int **idx_**
- uint32_t p_end_
  CTD triplet at end of window.
- uint32_t p_start_
  CTD triplet at start of window.
- int **rx_count_**
- uint32_t **s_end_**
- uint32_t **s_start_**
- uint32_t **t_end_**
- uint32_t **t_start_**
- uint32_t ts_end_
  Start and end of window timestamps, in UNIX time.
- uint32_t **ts_start_**

Friends

- class **XAcousticPosition**

10.53.1 Detailed Description

An acoustic listening window.

An instance of this class represents a period during which the instrument listened for acoustic signals, decoded from an acoustic position message (see XAcousticPosition): its start and end times, the pressure, temperature and salinity at the start and end, its index, and the number of receptions (see XAcousticCacst) that fall within it.

10.53.2 Constructor & Destructor Documentation

10.53.2.1 XAcousticWindow() [1/2]

```
util::xmsg::XAcousticWindow::XAcousticWindow ()  [inline]
```

Default constructor.

10.53.2.2 XAcousticWindow() [2/2]

```
util::xmsg::XAcousticWindow::XAcousticWindow (
            const vector< uint8_t > & data)  [inline]
```

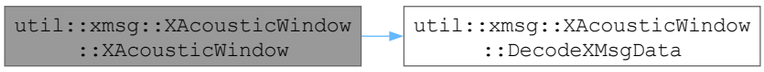
Construct from message data.

Parameters

in	*data*	The binary message data to decode.

References DecodeXMsgData().

Here is the call graph for this function:



10.53.2.3 ~XAcousticWindow()

```
util::xmsg::XAcousticWindow::~XAcousticWindow ()  [inline]
```

Destructor.

10.53.3 Member Function Documentation

10.53.3.1 DecodeXMsgData()

```
void util::xmsg::XAcousticWindow::DecodeXMsgData (
            const vector< uint8_t > & data)  [inline]
```

Decode the acoustic position CACST X-Message from the binary message data.

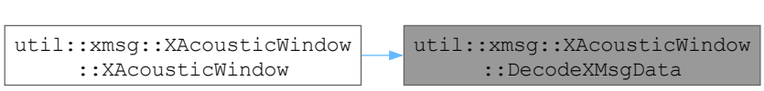
Parameters

in	*data*	The binary message data to decode.

References CATCH_BLOCK, and u32.

Referenced by XAcousticWindow().

Here is the caller graph for this function:



10.53.4 Member Data Documentation

10.53.4.1 p_end_

```
uint32_t util::xmsg::XAcousticWindow::p_end_  [private]
```

CTD triplet at end of window.

10.53.4.2 p_start_

```
uint32_t util::xmsg::XAcousticWindow::p_start_  [private]
```

CTD triplet at start of window.

10.53.4.3 ts_end_

```
uint32_t util::xmsg::XAcousticWindow::ts_end_  [private]
```

Start and end of window timestamps, in UNIX time.

The documentation for this class was generated from the following file:

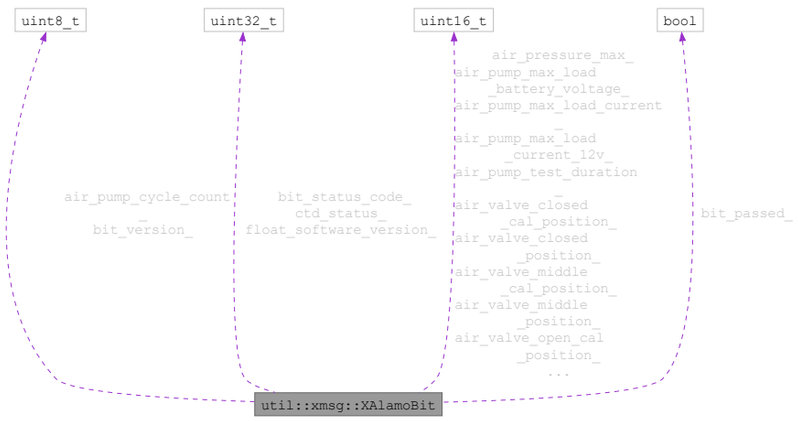
- data/xmsg/XAcousticPosition.h

10.54 util::xmsg::XAlamoBit Class Reference

Class representing the BIT test of an ALAMO float.

```
#include <XAlamoBit.h>
```

Collaboration diagram for util::xmsg::XAlamoBit:



Public Member Functions

- XAlamoBit (const vector< uint8_t > &data)

  The parameterized constructor.
- ~XAlamoBit ()

  The destructor.
- uint16_t **air_pressure_max** () const
- uint8_t **air_pump_cycle_count** () const
- uint16_t **air_pump_max_load_battery_voltage** () const
- uint16_t **air_pump_max_load_current** () const
- uint16_t **air_pump_max_load_current_12v** () const
- uint16_t **air_pump_test_duration** () const
- uint16_t **air_valve_closed_cal_position** () const
- uint16_t **air_valve_closed_position** () const
- uint16_t **air_valve_middle_cal_position** () const
- uint16_t **air_valve_middle_position** () const
- uint16_t **air_valve_open_cal_position** () const
- uint16_t **air_valve_open_position** () const
- uint16_t **battery_voltage_at_end_of_bit** () const
- uint16_t **battery_voltage_at_start_of_bit** () const
- uint16_t **bit_duration** () const
- uint16_t **bit_pressure** () const
- uint16_t **bit_salinity** () const
- uint32_t bit_status_code () const

  Get the Built-in-Test (BIT) status code.
- uint16_t **bit_temperature** () const
- uint8_t bit_version () const

  Get the Built-in-Test (BIT) version.
- uint32_t **ctd_status** () const
- void DecodeXMsgData (const vector< uint8_t > &data)

  Decode the ALAMO BIT test engineering message from the binary message data.
- uint32_t float_software_version () const

  Get the instrument software version.
- const string GenerateListingString (int whoi, int dive, uint16_t pg, uint16_t po, uint16_t sg, uint16_t so, uint16_t tg, uint16_t to) const

  Generate a listing entry string representation of the the ALAMO BIT test engineering message.
- template<typename T = double>
  T **GetDecodedBatteryVoltageAtEndOfBit** () const
- template<typename T = double>
  T **GetDecodedBatteryVoltageAtStartOfBit** () const
- template<typename T = double>
  T **GetDecodedBitPressure** (int16_t gain, int16_t offs) const
- template<typename T = double>
  T **GetDecodedBitPressureAfterOffset** (int16_t gain, int16_t offs) const
- template<typename T = double>
  T **GetDecodedBitPressureBeforeOffset** (int16_t gain, int16_t offs) const
- template<typename T = double>
  T **GetDecodedBitPressureOffset** () const
- template<typename T = double>
  T **GetDecodedBitSalinity** (int16_t gain, int16_t offs) const
- template<typename T = double>
  T **GetDecodedBitTemperature** (int16_t gain, int16_t offs) const
- template<typename T = double>
  T **GetDecodedVacAtEndOfBit** () const
- template<typename T = double>
  T **GetDecodedVacAtStartOfBit** () const
- bool **is_bit_passed** () const
- uint16_t **oil_pump_max_load_battery_voltage** () const
- uint16_t **oil_pump_max_load_current** () const
- uint16_t **oil_pump_max_load_current_12v** () const
- uint16_t **oil_pump_test_duration** () const
- uint16_t **oil_valve_closed_cal_position** () const
- uint16_t **oil_valve_closed_position** () const
- uint16_t **oil_valve_open_cal_position** () const
- uint16_t **oil_valve_open_position** () const
- uint16_t **pres_counts_after_offset_reset** () const
- uint16_t **pres_counts_before_offset_reset** () const
- uint16_t **pres_offset** () const
- uint16_t **vac_at_end_of_bit** () const
- uint16_t **vac_at_start_of_bit** () const

Protected Member Functions

- void **set_air_pressure_max** (uint16_t air_pressure_max)
- void **set_air_pump_cycle_count** (uint8_t air_pump_cycle_count)
- void **set_air_pump_max_load_battery_voltage** (uint16_t air_pump_max_load_battery_voltage)
- void **set_air_pump_max_load_current** (uint16_t air_pump_max_load_current)
- void **set_air_pump_max_load_current_12v** (uint16_t air_pump_max_load_current_12v)
- void **set_air_pump_test_duration** (uint16_t air_pump_test_duration)
- void **set_air_valve_closed_cal_position** (uint16_t air_valve_closed_cal_position)
- void **set_air_valve_closed_position** (uint16_t air_valve_closed_position)
- void **set_air_valve_middle_cal_position** (uint16_t air_valve_middle_cal_position)
- void **set_air_valve_middle_position** (uint16_t air_valve_middle_position)
- void **set_air_valve_open_cal_position** (uint16_t air_valve_open_cal_position)
- void **set_air_valve_open_position** (uint16_t air_valve_open_position)
- void **set_battery_voltage_at_end_of_bit** (uint16_t battery_voltage_at_end_of_bit)
- void **set_battery_voltage_at_start_of_bit** (uint16_t battery_voltage_at_start_of_bit)
- void **set_bit_duration** (uint16_t bit_duration)
- void **set_bit_passed** (bool bit_passed)
- void **set_bit_pressure** (uint16_t bit_pressure)
- void **set_bit_salinity** (uint16_t bit_salinity)
- void set_bit_status_code (uint32_t bit_status_code)

  Set the Built-in-Test (BIT) status code.
- void **set_bit_temperature** (uint16_t bit_temperature)
- void set_bit_version (uint8_t bit_version)

  Set the Built-in-Test (BIT) version.
- void **set_ctd_status** (uint32_t ctd_status)
- void set_float_software_version (uint32_t float_software_version)

  Set the instrument software version.
- void **set_oil_pump_max_load_battery_voltage** (uint16_t oil_pump_max_load_battery_voltage)
- void **set_oil_pump_max_load_current** (uint16_t oil_pump_max_load_current)
- void **set_oil_pump_max_load_current_12v** (uint16_t oil_pump_max_load_current_12v)
- void **set_oil_pump_test_duration** (uint16_t oil_pump_test_duration)
- void **set_oil_valve_closed_cal_position** (uint16_t oil_valve_closed_cal_position)
- void **set_oil_valve_closed_position** (uint16_t oil_valve_closed_position)
- void **set_oil_valve_open_cal_position** (uint16_t oil_valve_open_cal_position)
- void **set_oil_valve_open_position** (uint16_t oil_valve_open_position)
- void **set_pres_counts_after_offset_reset** (uint16_t pres_counts_after_offset_reset)
- void **set_pres_counts_before_offset_reset** (uint16_t pres_counts_before_offset_reset)
- void **set_pres_offset** (uint16_t pres_offset)
- void **set_vac_at_end_of_bit** (uint16_t vac_at_end_of_bit)
- void **set_vac_at_start_of_bit** (uint16_t vac_at_start_of_bit)

Private Attributes

- uint16_t **air_pressure_max_**
- uint8_t **air_pump_cycle_count_**
- uint16_t **air_pump_max_load_battery_voltage_**
- uint16_t **air_pump_max_load_current_**
- uint16_t **air_pump_max_load_current_12v_**
- uint16_t **air_pump_test_duration_**
- uint16_t **air_valve_closed_cal_position_**
- uint16_t **air_valve_closed_position_**
- uint16_t **air_valve_middle_cal_position_**
- uint16_t **air_valve_middle_position_**
- uint16_t **air_valve_open_cal_position_**
- uint16_t **air_valve_open_position_**
- uint16_t **battery_voltage_at_end_of_bit_**
- uint16_t **battery_voltage_at_start_of_bit_**
- uint16_t **bit_duration_**
- bool **bit_passed_**
- uint16_t **bit_pressure_**
- uint16_t **bit_salinity_**
- uint32_t bit_status_code_

  The Built-in-Test (BIT) status code.
- uint16_t **bit_temperature_**
- uint8_t bit_version_

The Built-in-Test (BIT) version identifier.

- uint32_t **ctd_status_**
- uint32_t float_software_version_

  The software version identifier.
- uint16_t **oil_pump_max_load_battery_voltage_**
- uint16_t **oil_pump_max_load_current_**
- uint16_t **oil_pump_max_load_current_12v_**
- uint16_t **oil_pump_test_duration_**
- uint16_t **oil_valve_closed_cal_position_**
- uint16_t **oil_valve_closed_position_**
- uint16_t **oil_valve_open_cal_position_**
- uint16_t **oil_valve_open_position_**
- uint16_t **pres_counts_after_offset_reset_**
- uint16_t **pres_counts_before_offset_reset_**
- uint16_t pres_offset_

  The measured pressure offset, in 1/1000 dBar.
- uint16_t **vac_at_end_of_bit_**
- uint16_t **vac_at_start_of_bit_**

10.54.1 Detailed Description

Class representing the BIT test of an ALAMO float.

10.54.2 Constructor & Destructor Documentation

10.54.2.1 XAlamoBit()

```
util::xmsg::XAlamoBit::XAlamoBit (
            const vector< uint8_t > & data)  [inline]
```

The parameterized constructor.

Parameters

in	*data*	The binary data to decode.

Exceptions

log::MessageException	if an error occurs while decoding.

References DecodeXMsgData().

Here is the call graph for this function:

10.54.2.2 ~XAlamoBit()

```
util::xmsg::XAlamoBit::~XAlamoBit ()  [inline]
```

The destructor.

10.54.3 Member Function Documentation

10.54.3.1 bit_status_code()

```
uint32_t util::xmsg::XAlamoBit::bit_status_code () const  [inline]
```

Get the Built-in-Test (BIT) status code.

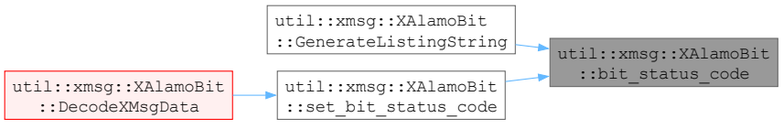
Returns

The Built-in-Test (BIT) status code.

References bit_status_code_.

Referenced by GenerateListingString(), and set_bit_status_code().

Here is the caller graph for this function:



10.54.3.2 bit_version()

```
uint8_t util::xmsg::XAlamoBit::bit_version () const  [inline]
```

Get the Built-in-Test (BIT) version.

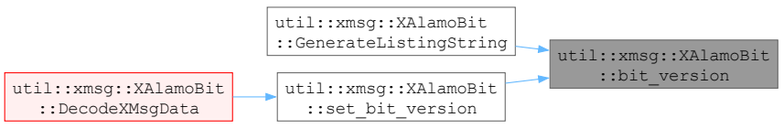
Returns

The Built-in-Test (BIT) version.

References bit_version_.

Referenced by GenerateListingString(), and set_bit_version().

Here is the caller graph for this function:



10.54.3.3 DecodeXMsgData()

```
void util::xmsg::XAlamoBit::DecodeXMsgData (
            const vector< uint8_t > & data)  [inline]
```

Decode the ALAMO BIT test engineering message from the binary message data.

Parameters

in	*data*	The binary data to decode.

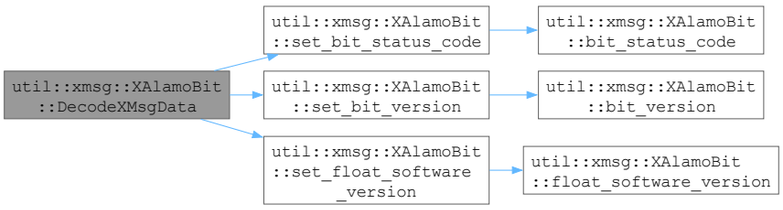
Exceptions

log::MessageException	if an error occurs while decoding.

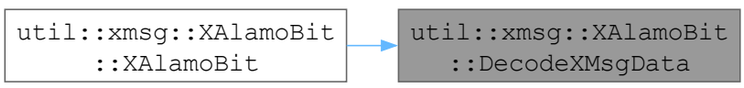
References bit_version_, CATCH_BLOCK, EXC_THROW, set_bit_status_code(), set_bit_version(), set_float_software_version(), u08, u16, and u32.

Referenced by XAlamoBit().

Here is the call graph for this function:



Here is the caller graph for this function:



10.54.3.4 float_software_version()

```
uint32_t util::xmsg::XAlamoBit::float_software_version () const  [inline]
```

Get the instrument software version.

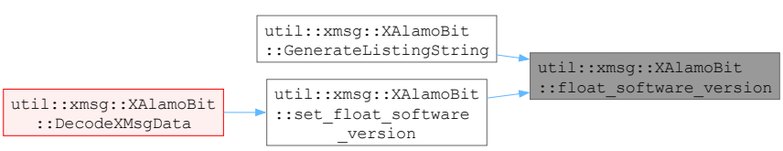
Returns

The instrument software version.

References float_software_version_.

Referenced by GenerateListingString(), and set_float_software_version().

Here is the caller graph for this function:



10.54.3.5 GenerateListingString()

```
const string util::xmsg::XAlamoBit::GenerateListingString (
            int whoi,
            int dive,
            uint16_t pg,
            uint16_t po,
            uint16_t sg,
            uint16_t so,
            uint16_t tg,
            uint16_t to) const  [inline]
```

Generate a listing entry string representation of the the ALAMO BIT test engineering message.

Parameters

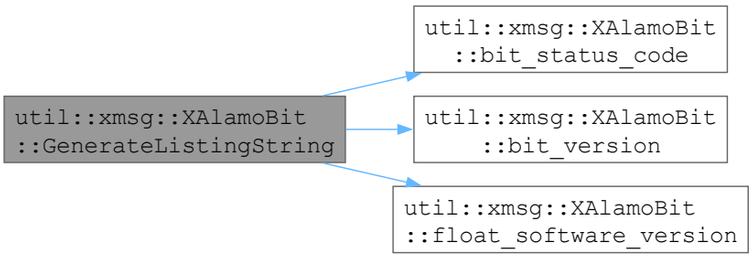
in	*whoi*	The instrument serial number.
in	*dive*	The dive number of this ALAMO BIT test engineering message.
in	*pg*	The pressure sensor sensitivity scale factor, in LSB/dBar.
in	*po*	The pressure sensor offset, in dBar.
in	*sg*	The salinity sensor sensitivity scale factor, in LSB/PSU.
in	*so*	The salinity sensor offset, in PSU.
in	*tg*	The temperature sensor sensitivity scale factor, in LSB/degC.
in	*to*	The temperatUre sensor offset, in degC.

Returns

A listing entry string representation of the the ALAMO BIT test engineering message.

References bit_status_code(), bit_version(), bit_version_, EXC_THROW, and float_software_version().

Here is the call graph for this function:



10.54.3.6 set_bit_status_code()

```
void util::xmsg::XAlamoBit::set_bit_status_code (
            uint32_t bit_status_code)  [inline], [protected]
```

Set the Built-in-Test (BIT) status code.

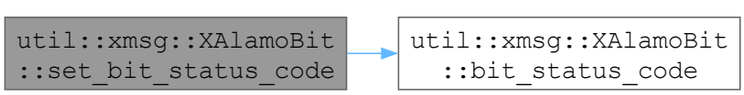
Parameters

in	*bit_status_code*	The Built-in-Test (BIT) status code to set.

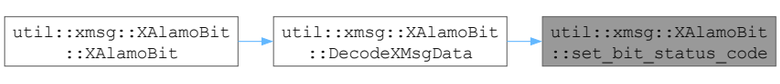
References bit_status_code(), and bit_status_code_.

Referenced by DecodeXMsgData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.54.3.7 set_bit_version()

```
void util::xmsg::XAlamoBit::set_bit_version (
            uint8_t bit_version)  [inline], [protected]
```

Set the Built-in-Test (BIT) version.

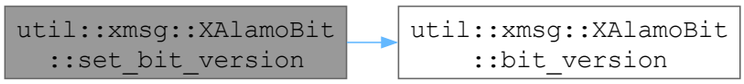
Parameters

in	*bit_version*	The Built-in-Test (BIT) version to set.

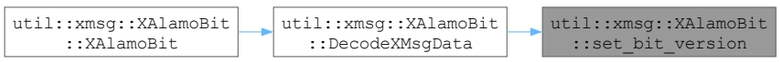
References bit_version(), and bit_version_.

Referenced by DecodeXMsgData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.54.3.8 set_float_software_version()

```
void util::xmsg::XAlamoBit::set_float_software_version (
            uint32_t float_software_version)  [inline], [protected]
```

Set the instrument software version.

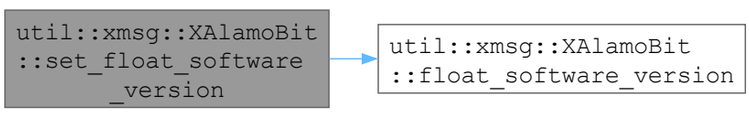
Parameters

in	*float_software_version*	The instrument software version to set.

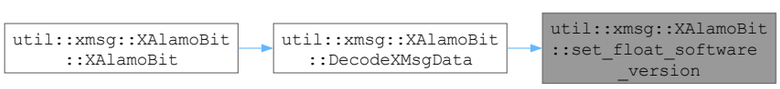
References float_software_version(), and float_software_version_.

Referenced by DecodeXMsgData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.54.4 Member Data Documentation

10.54.4.1 bit_status_code_

```
uint32_t util::xmsg::XAlamoBit::bit_status_code_  [private]
```

The Built-in-Test (BIT) status code.

Referenced by bit_status_code(), and set_bit_status_code().

10.54.4.2 bit_version_

```
uint8_t util::xmsg::XAlamoBit::bit_version_  [private]
```

The Built-in-Test (BIT) version identifier.

Referenced by bit_version(), DecodeXMsgData(), GenerateListingString(), and set_bit_version().

10.54.4.3 float_software_version_

```
uint32_t util::xmsg::XAlamoBit::float_software_version_  [private]
```

The software version identifier.

Referenced by float_software_version(), and set_float_software_version().

10.54.4.4 pres_offset_

```
uint16_t util::xmsg::XAlamoBit::pres_offset_  [private]
```

The measured pressure offset, in 1/1000 dBar.

The documentation for this class was generated from the following file:

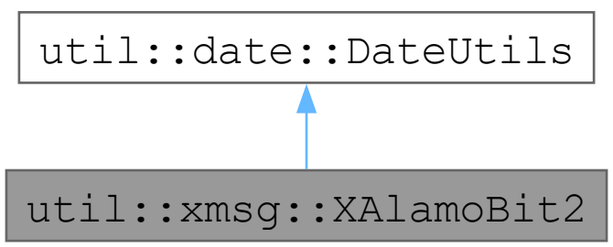
- data/xmsg/XAlamoBit.h

10.55 util::xmsg::XAlamoBit2 Class Reference

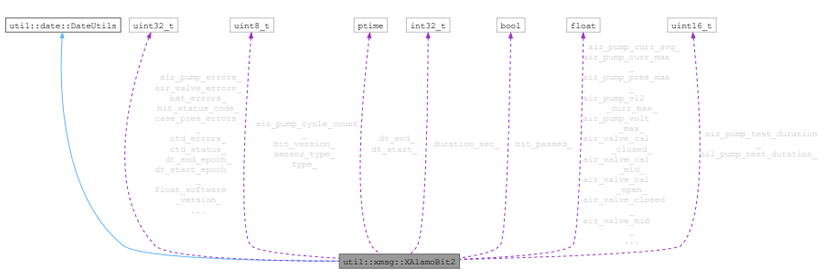
The ALAMO Built-in-Test (BIT) message, version 2.

`#include <XAlamoBit2.h>`

Inheritance diagram for util::xmsg::XAlamoBit2:



Collaboration diagram for util::xmsg::XAlamoBit2:



Public Member Functions

- XAlamoBit2 (const vector< uint8_t > &data)

  The parameterized constructor.
- ~XAlamoBit2 ()

  The destructor.
- float **air_pump_curr_avg** () const
- float **air_pump_curr_max** () const
- uint8_t **air_pump_cycle_count** () const
- uint32_t **air_pump_errors** () const
- float **air_pump_pres_max** () const
- uint16_t **air_pump_test_duration** () const
- float **air_pump_v12_curr_max** () const
- float **air_pump_volt_max** () const
- float **air_valve_cal_closed** () const
- float **air_valve_cal_mid** () const
- float **air_valve_cal_open** () const
- float **air_valve_closed** () const
- uint32_t **air_valve_errors** () const
- float **air_valve_mid** () const
- float **air_valve_open** () const
- uint32_t **bat_errors** () const
- float **bat_volt_end** () const
- float **bat_volt_start** () const
- bool **bit_passed** () const
- uint32_t **bit_status_code** () const
- uint8_t bit_version () const

  Get the Built-in-Test (BIT) version.
- uint32_t **case_pres_errors** () const
- float **case_vac_end** () const
- float **case_vac_start** () const
- uint32_t **ctd_errors** () const
- uint32_t **ctd_status** () const
- void DecodeXMsgData (const vector< uint8_t > &data)

  Decode the ALAMO BIT test engineering message from the binary message data.
- const ptime & **dt_end** () const
- uint32_t **dt_end_epoch** () const
- const ptime & **dt_start** () const
- uint32_t **dt_start_epoch** () const
- int32_t **duration_sec** () const
- uint32_t float_software_version () const

  Get the instrument software version.
- const string GenerateListingString (int whoi, int dive) const

  Generate a listing entry string representation of the the ALAMO BIT test engineering message.
- uint32_t **gps_errors** () const
- uint32_t **iridum_errors** () const
- float **oil_pump_curr_avg** () const
- float **oil_pump_curr_max** () const
- uint32_t **oil_pump_errors** () const
- uint16_t **oil_pump_test_duration** () const
- float **oil_pump_v12_curr_max** () const
- float **oil_pump_volt_max** () const
- float **oil_valve_cal_closed** () const
- float **oil_valve_cal_open** () const
- float **oil_valve_closed** () const
- uint32_t **oil_valve_errors** () const
- float **oil_valve_open** () const
- float **pres_after_cal** () const
- float **pres_before_cal** () const
- float **pres_offset** () const
- float **saln** () const
- uint8_t **sensor_type** () const
- float **temp** () const
- uint32_t **traveler_errors** () const
- uint8_t **type** () const

Static Public Member Functions

- static string GenerateCurrentUtcTimeIso8601String ()

  Generate an ISO8601 format date/time string representation of the current time, in UTC.
- static string GenerateIso8601String (const ptime &p, bool extended=true)

  Generate an ISO8601 format date/time string representation of a POSIX time object.
- static double GenerateJulianDay (const ptime &dt, const ptime &ref_dt)

  Convert a date/time to fractional days since a reference date/time.
- static ptime GetCurrentUtcTime ()

  Generate a POSIX time object representing the current date and time (rounded to the nearest second).

Protected Member Functions

- void **set_air_pump_curr_avg** (float air_pump_curr_avg)
- void **set_air_pump_curr_max** (float air_pump_curr_max)
- void **set_air_pump_cycle_count** (uint8_t air_pump_cycle_count)
- void **set_air_pump_errors** (uint32_t air_pump_errors)
- void **set_air_pump_pres_max** (float air_pump_pres_max)
- void **set_air_pump_test_duration** (uint16_t air_pump_test_duration)
- void **set_air_pump_v12_curr_max** (float air_pump_v12_curr_max)
- void **set_air_pump_volt_max** (float air_pump_volt_max)
- void **set_air_valve_cal_closed** (float air_valve_cal_closed)
- void **set_air_valve_cal_mid** (float air_valve_cal_mid)
- void **set_air_valve_cal_open** (float air_valve_cal_open)
- void **set_air_valve_closed** (float air_valve_closed)
- void **set_air_valve_errors** (uint32_t air_valve_errors)
- void **set_air_valve_mid** (float air_valve_mid)
- void **set_air_valve_open** (float air_valve_open)
- void **set_bat_errors** (uint32_t bat_errors)
- void **set_bat_volt_end** (float bat_volt_end)
- void **set_bat_volt_start** (float bat_volt_start)
- void set_bit_passed (bool bit_passed)

  Set the BIT passed flag.
- void set_bit_status_code (uint32_t bit_status_code)

  Set the Built-in-Test (BIT) status code.
- void set_bit_version (uint8_t bit_version)

  Set the Built-in-Test (BIT) version.
- void **set_case_pres_errors** (uint32_t case_pres_errors)
- void **set_case_vac_end** (float case_vac_end)
- void **set_case_vac_start** (float case_vac_start)
- void **set_ctd_errors** (uint32_t ctd_errors)
- void **set_ctd_status** (uint32_t ctd_status)
- void **set_dt_end** (const ptime &dt_end)
- void **set_dt_end_epoch** (uint32_t dt_end_epoch)
- void **set_dt_start** (const ptime &dt_start)
- void **set_dt_start_epoch** (uint32_t dt_start_epoch)
- void **set_duration_sec** (int32_t duration_sec)
- void set_float_software_version (uint32_t float_software_version)

  Set the instrument software version.
- void **set_gps_errors** (uint32_t gps_errors)
- void **set_iridum_errors** (uint32_t iridum_errors)
- void **set_oil_pump_curr_avg** (float oil_pump_curr_avg)
- void **set_oil_pump_curr_max** (float oil_pump_curr_max)
- void **set_oil_pump_errors** (uint32_t oil_pump_errors)
- void **set_oil_pump_test_duration** (uint16_t oil_pump_test_duration)
- void **set_oil_pump_v12_curr_max** (float oil_pump_v12_curr_max)
- void **set_oil_pump_volt_max** (float oil_pump_volt_max)
- void **set_oil_valve_cal_closed** (float oil_valve_cal_closed)
- void **set_oil_valve_cal_open** (float oil_valve_cal_open)
- void **set_oil_valve_closed** (float oil_valve_closed)
- void **set_oil_valve_errors** (uint32_t oil_valve_errors)
- void **set_oil_valve_open** (float oil_valve_open)
- void **set_pres_after_cal** (float pres_after_cal)
- void **set_pres_before_cal** (float pres_before_cal)
- void **set_pres_offset** (float pres_offset)
- void **set_saln** (float saln)
- void **set_sensor_type** (uint8_t sensor_type)
- void **set_temp** (float temp)
- void **set_traveler_errors** (uint32_t travelerErrors)
- void **set_type** (uint8_t type)

Private Attributes

- float **air_pump_curr_avg_**
- float air_pump_curr_max_

  Max.
- uint8_t air_pump_cycle_count_

  Air pump cycle count.
- uint32_t air_pump_errors_

  Air pump errors bitmask.
- float **air_pump_pres_max_**
- uint16_t air_pump_test_duration_

  Air pump test duration, in seconds.
- float air_pump_v12_curr_max_

  Max.
- float air_pump_volt_max_

  Max.
- float **air_valve_cal_closed_**
- float **air_valve_cal_mid_**
- float air_valve_cal_open_

  Air valve open, closed, and middle calibration positions, in volts.
- float **air_valve_closed_**
- uint32_t air_valve_errors_

  Air valve errors bitmask.
- float **air_valve_mid_**
- float air_valve_open_

  Air valve open, closed, and middle positions, in volts.
- uint32_t **bat_errors_**
- float **bat_volt_end_**
- float bat_volt_start_

  Battery voltages at start and end of BIT, in volts.
- bool bit_passed_

  The BIT passed flag.
- uint32_t bit_status_code_

  The Built-in-Test (BIT) status code.
- uint8_t bit_version_

  The Built-in-Test (BIT) version identifier.
- uint32_t **case_pres_errors_**
- float **case_vac_end_**
- float case_vac_start_

  Case vac at start and end of BIT, in inHg.
- uint32_t **ctd_errors_**
- uint32_t **ctd_status_**
- ptime dt_end_

  BIT End time.
- uint32_t dt_end_epoch_

  BIT End time, in seconds since 1970-01-01T00:00:00Z.
- ptime dt_start_

  BIT Start time.
- uint32_t dt_start_epoch_

  BIT start time, in seconds since 1970-01-01T00:00:00Z.
- int32_t duration_sec_

  BIT duration in seconds.
- uint32_t float_software_version_

  The software version identifier.
- uint32_t gps_errors_

  GPS errors bitmask.
- uint32_t **iridum_errors_**
- float **oil_pump_curr_avg_**
- float oil_pump_curr_max_

  Max.
- uint32_t oil_pump_errors_

  Oil pump errors bitmask.
- uint16_t oil_pump_test_duration_

  Oil pump test duration, in seconds.

- float oil_pump_v12_curr_max_

  Max 12V rail current during oil pump test, in amps.
- float oil_pump_volt_max_

  Max battery voltage during oil pump test, in volts.
- float **oil_valve_cal_closed_**
- float oil_valve_cal_open_

  Oil valve open and clased calibration positions, in volts.
- float **oil_valve_closed_**
- uint32_t oil_valve_errors_

  Oil valve errors bitmask.
- float oil_valve_open_

  Oil valve open and closed positions, in volts.
- float **pres_after_cal_**
- float **pres_before_cal_**
- float **pres_offset_**
- float **saln_**
- uint8_t sensor_type_

  Sensor type enum.
- float **temp_**
- uint32_t **traveler_errors_**
- uint8_t type_

  BIT test type enum.

10.55.1 Detailed Description

The ALAMO Built-in-Test (BIT) message, version 2.

An instance of this class represents a decoded BIT message from ALAMO instruments with firmware version 10.x or later (BIT formats 9, 10 and 11; see XEngBIT). Earlier instruments use XAlamoBit.

10.55.2 Constructor & Destructor Documentation

10.55.2.1 XAlamoBit2()

```
util::xmsg::XAlamoBit2::XAlamoBit2 (
            const vector< uint8_t > & data)  [inline]
```

The parameterized constructor.

Parameters

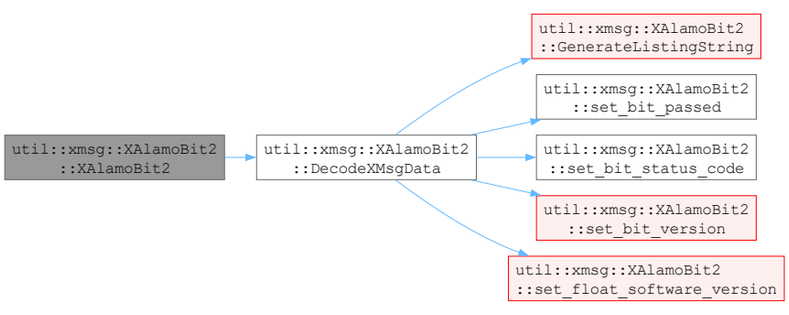
in	*data*	The binary data to decode.

Exceptions

log::MessageException	if an error occurs while decoding.

References DecodeXMsgData().

Here is the call graph for this function:



10.55.2.2 ~XAlamoBit2()

```
util::xmsg::XAlamoBit2::~XAlamoBit2 ()  [inline]
```

The destructor.

10.55.3 Member Function Documentation

10.55.3.1 bit_version()

```
uint8_t util::xmsg::XAlamoBit2::bit_version () const  [inline]
```

Get the Built-in-Test (BIT) version.

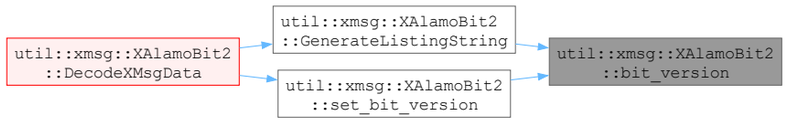
Returns

The Built-in-Test (BIT) version.

References bit_version_.

Referenced by GenerateListingString(), and set_bit_version().

Here is the caller graph for this function:



10.55.3.2 DecodeXMsgData()

```
void util::xmsg::XAlamoBit2::DecodeXMsgData (
            const vector< uint8_t > & data)  [inline]
```

Decode the ALAMO BIT test engineering message from the binary message data.

Parameters

in	*data*	The binary data to decode.

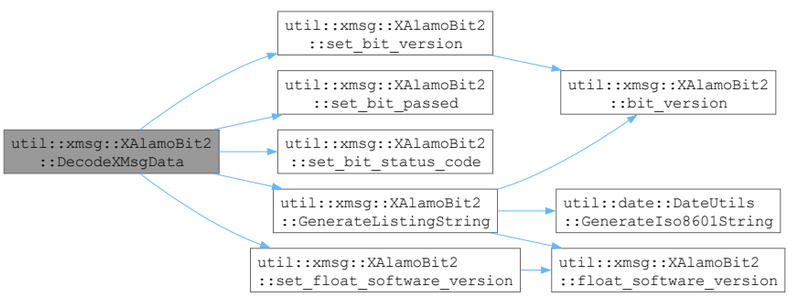
Exceptions

log::MessageException	if an error occurs while decoding.

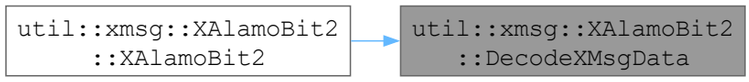
References bit_version_, CATCH_BLOCK, EXC_THROW, GenerateListingString(), ldbg, lwrn, s1970, set_bit_passed(), set_bit_status_code(), set_bit_version(), set_float_software_version(), u08, u16, and u32.

Referenced by XAlamoBit2().

Here is the call graph for this function:



Here is the caller graph for this function:



10.55.3.3 float_software_version()

```
uint32_t util::xmsg::XAlamoBit2::float_software_version () const  [inline]
```

Get the instrument software version.

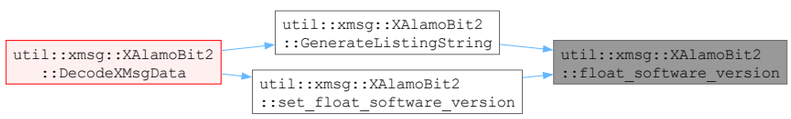
Returns

The instrument software version.

References float_software_version_.

Referenced by GenerateListingString(), and set_float_software_version().

Here is the caller graph for this function:



10.55.3.4 GenerateCurrentUtcTimeIso8601String()

```
static string util::date::DateUtils::GenerateCurrentUtcTimeIso8601String ()  [inline], [static],
[inherited]
```

Generate an ISO8601 format date/time string representation of the current time, in UTC.

Note

All date/time objects use the UTC time-zone.

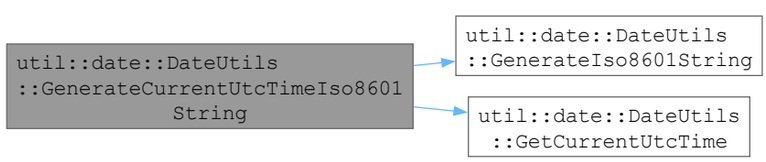
Returns

An ISO8601 format date/time string representation of the current time.

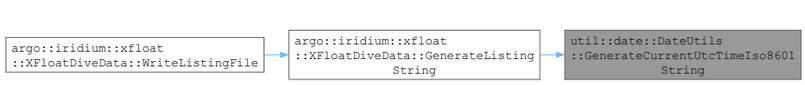
References util::date::DateUtils::GenerateIso8601String(), and util::date::DateUtils::GetCurrentUtcTime().

Referenced by argo::iridium::xfloat::XFloatDiveData::GenerateListingString().

Here is the call graph for this function:



Here is the caller graph for this function:



10.55.3.5 GenerateIso8601String()

```
static string util::date::DateUtils::GenerateIso8601String (
            const ptime & p,
            bool extended = true)  [inline], [static], [inherited]
```

Generate an ISO8601 format date/time string representation of a POSIX time object.

Note

All date/time objects use the UTC time-zone.

Parameters

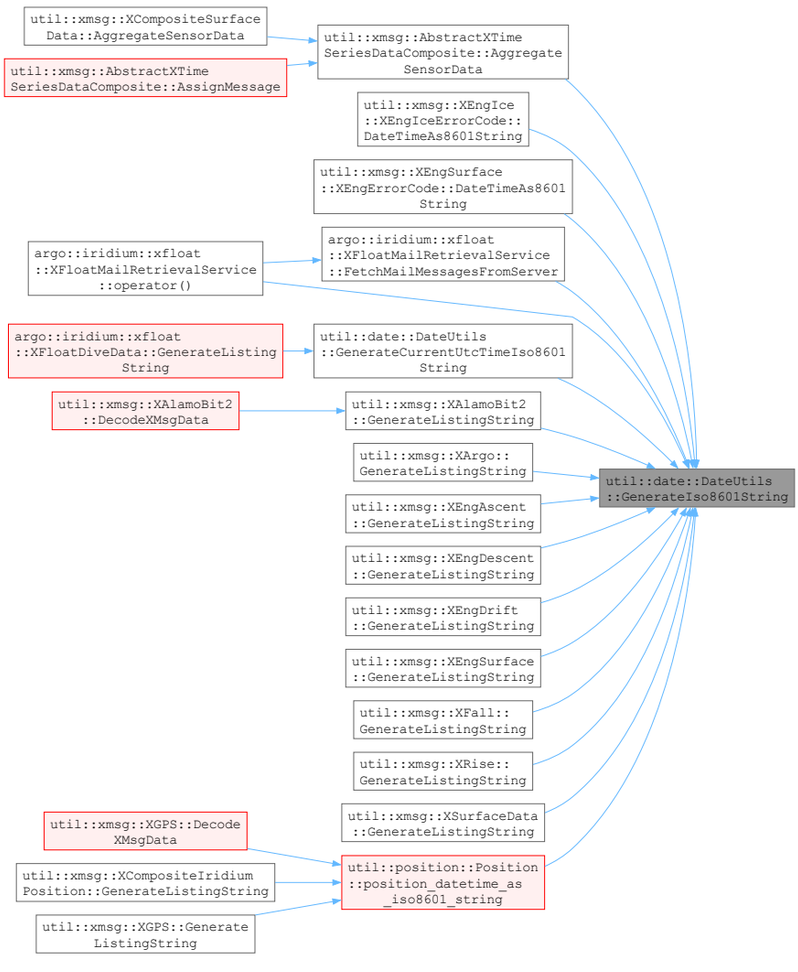
in	*p*	POSIX time from which to generate string.
in	*extended*	If TRUE, use the ISO 8601 extended format (YYYY-MM-DDThh:mm:ssZ); otherwise, use the basic format (YYYYMMDDThhmmss).

Returns

An ISO8601 format date/time string representation of the POSIX time object. If p is equal to boost::posix_↩ time::not_a_date_time, then the string "0000-00-00T00:00:00Z" (extended) or "00000000000000" (basic) will be returned.

Referenced by util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::AggregateSensorData(), util::xmsg::XEngIce::XEngIce
util::xmsg::XEngSurface::XEngErrorCode::DateTimeAs8601String(), argo::iridium::xfloat::XFloatMailRetrievalService::FetchMailMessages
util::date::DateUtils::GenerateCurrentUtcTimeIso8601String(), GenerateListingString(), util::xmsg::XArgo::GenerateListingString(),
util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListing
util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XRise::GenerateListingString(),
util::xmsg::XSurfaceData::GenerateListingString(), argo::iridium::xfloat::XFloatMailRetrievalService::operator()(), and
util::position::Position::position_datetime_as_iso8601_string().

Here is the caller graph for this function:



10.55.3.6 GenerateJulianDay()

```
static double util::date::DateUtils::GenerateJulianDay (
            const ptime & dt,
            const ptime & ref_dt)  [inline], [static], [inherited]
```

Convert a date/time to fractional days since a reference date/time.

Parameters

in	*dt*	The date/time to convert.
in	*ref_dt*	The reference date/time (day 0).

Exceptions

util::log::MessageException	if either date/time is not valid.

Returns

The number of days (with millisecond resolution) from `ref_dt` to `dt`.

References CATCH_BLOCK, and EXC_THROW.

10.55.3.7 GenerateListingString()

```
const string util::xmsg::XAlamoBit2::GenerateListingString (
            int whoi,
            int dive) const  [inline]
```

Generate a listing entry string representation of the the ALAMO BIT test engineering message.

Parameters

in	*whoi*	The instrument serial number.
in	*dive*	The dive number of this ALAMO BIT test engineering message.

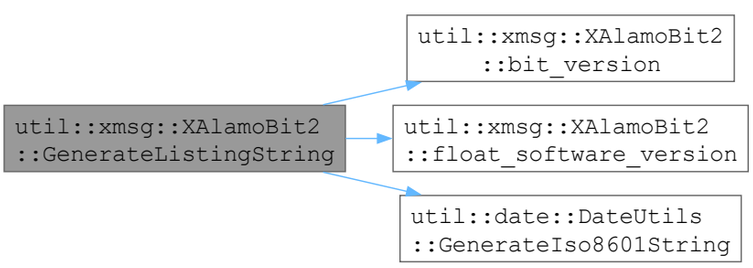
Returns

A listing entry string representation of the the ALAMO BIT test engineering message.

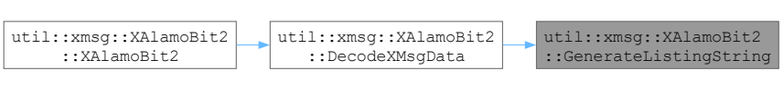
References bit_version(), EXC_THROW, float_software_version(), and util::date::DateUtils::GenerateIso8601String().

Referenced by DecodeXMsgData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.55.3.8 GetCurrentUtcTime()

```
static ptime util::date::DateUtils::GetCurrentUtcTime ()  [inline], [static], [inherited]
```

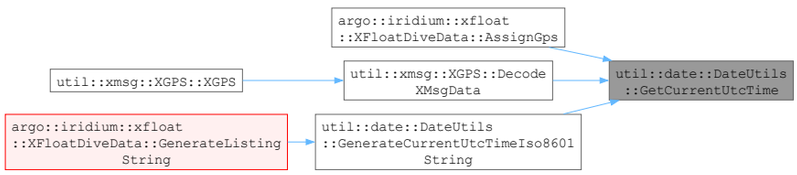
Generate a POSIX time object representing the current date and time (rounded to the nearest second).

Returns

A POSIX time object representing the current date and time (rounded to the nearest second).

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignGps(), util::xmsg::XGPS::DecodeXMsgData(), and util::date::DateUtils::GenerateCurrentUtcTimeIso8601String().

Here is the caller graph for this function:



10.55.3.9 set_bit_passed()

```
void util::xmsg::XAlamoBit2::set_bit_passed (
            bool bit_passed)  [inline], [protected]
```

Set the BIT passed flag.

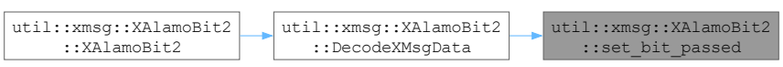
Parameters

in	*bit_passed*	BIT passed flag to set.

References bit_passed_.

Referenced by DecodeXMsgData().

Here is the caller graph for this function:



10.55.3.10 set_bit_status_code()

```
void util::xmsg::XAlamoBit2::set_bit_status_code (
            uint32_t bit_status_code)  [inline], [protected]
```

Set the Built-in-Test (BIT) status code.

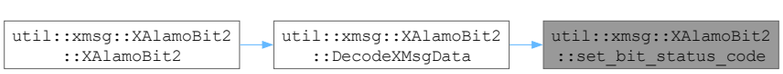
Parameters

in	*bit_status_code*	The Built-in-Test (BIT) status code to set.

References bit_status_code_.

Referenced by DecodeXMsgData().

Here is the caller graph for this function:



10.55.3.11 set_bit_version()

```
void util::xmsg::XAlamoBit2::set_bit_version (
            uint8_t bit_version)  [inline], [protected]
```

Set the Built-in-Test (BIT) version.

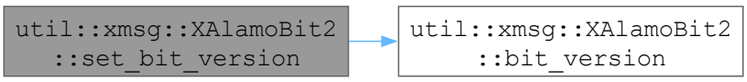
Parameters

in	*bit_version*	The Built-in-Test (BIT) version to set.

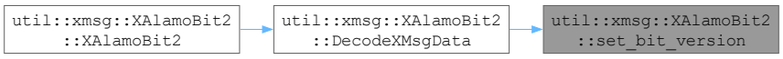
References bit_version(), and bit_version_.

Referenced by DecodeXMsgData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.55.3.12 set_float_software_version()

```
void util::xmsg::XAlamoBit2::set_float_software_version (
            uint32_t float_software_version)  [inline], [protected]
```

Set the instrument software version.

Parameters

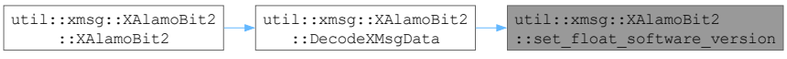
in	*float_software_version*	The instrument software version to set.

References float_software_version(), and float_software_version_.

Referenced by DecodeXMsgData().

Here is the call graph for this function:

Here is the caller graph for this function:



10.55.4 Member Data Documentation

10.55.4.1 air_pump_curr_max_

```
float util::xmsg::XAlamoBit2::air_pump_curr_max_  [private]
```

Max.

and mean current during air pump test, in amps.

10.55.4.2 air_pump_cycle_count_

```
uint8_t util::xmsg::XAlamoBit2::air_pump_cycle_count_  [private]
```

Air pump cycle count.

10.55.4.3 air_pump_errors_

```
uint32_t util::xmsg::XAlamoBit2::air_pump_errors_  [private]
```

Air pump errors bitmask.

10.55.4.4 air_pump_test_duration_

```
uint16_t util::xmsg::XAlamoBit2::air_pump_test_duration_  [private]
```

Air pump test duration, in seconds.

10.55.4.5 air_pump_v12_curr_max_

```
float util::xmsg::XAlamoBit2::air_pump_v12_curr_max_  [private]
```

Max.

12V rail current during air pump test, in amps.

10.55.4.6 air_pump_volt_max_

```
float util::xmsg::XAlamoBit2::air_pump_volt_max_  [private]
```

Max.

voltage during air pump test, in volts.

10.55.4.7 air_valve_cal_open_

```
float util::xmsg::XAlamoBit2::air_valve_cal_open_  [private]
```

Air valve open, closed, and middle calibration positions, in volts.

10.55.4.8 air_valve_errors_

```
uint32_t util::xmsg::XAlamoBit2::air_valve_errors_  [private]
```

Air valve errors bitmask.

10.55.4.9 air_valve_open_

```
float util::xmsg::XAlamoBit2::air_valve_open_  [private]
```

Air valve open, closed, and middle positions, in volts.

10.55.4.10 bat_volt_start_

```
float util::xmsg::XAlamoBit2::bat_volt_start_  [private]
```

Battery voltages at start and end of BIT, in volts.

10.55.4.11 bit_passed_

```
bool util::xmsg::XAlamoBit2::bit_passed_  [private]
```

The BIT passed flag.

Referenced by set_bit_passed().

10.55.4.12 bit_status_code_

```
uint32_t util::xmsg::XAlamoBit2::bit_status_code_  [private]
```

The Built-in-Test (BIT) status code.

Referenced by set_bit_status_code().

10.55.4.13 bit_version_

```
uint8_t util::xmsg::XAlamoBit2::bit_version_  [private]
```

The Built-in-Test (BIT) version identifier.

Referenced by bit_version(), DecodeXMsgData(), and set_bit_version().

10.55.4.14 case_vac_start_

```
float util::xmsg::XAlamoBit2::case_vac_start_  [private]
```

Case vac at start and end of BIT, in inHg.

10.55.4.15 dt_end_

```
ptime util::xmsg::XAlamoBit2::dt_end_  [private]
```

BIT End time.

10.55.4.16 dt_end_epoch_

```
uint32_t util::xmsg::XAlamoBit2::dt_end_epoch_  [private]
```

BIT End time, in seconds since 1970-01-01T00:00:00Z.

10.55.4.17 dt_start_

```
ptime util::xmsg::XAlamoBit2::dt_start_  [private]
```

BIT Start time.

10.55.4.18 dt_start_epoch_

```
uint32_t util::xmsg::XAlamoBit2::dt_start_epoch_  [private]
```

BIT start time, in seconds since 1970-01-01T00:00:00Z.

10.55.4.19 duration_sec_

```
int32_t util::xmsg::XAlamoBit2::duration_sec_  [private]
```

BIT duration in seconds.

10.55.4.20 float_software_version_

```
uint32_t util::xmsg::XAlamoBit2::float_software_version_  [private]
```

The software version identifier.

Referenced by float_software_version(), and set_float_software_version().

10.55.4.21 gps_errors_

```
uint32_t util::xmsg::XAlamoBit2::gps_errors_  [private]
```

GPS errors bitmask.

10.55.4.22 oil_pump_curr_max_

```
float util::xmsg::XAlamoBit2::oil_pump_curr_max_  [private]
```

Max.

and mean oil pump current during test, in amps.

10.55.4.23 oil_pump_errors_

```
uint32_t util::xmsg::XAlamoBit2::oil_pump_errors_  [private]
```

Oil pump errors bitmask.

10.55.4.24 oil_pump_test_duration_

```
uint16_t util::xmsg::XAlamoBit2::oil_pump_test_duration_  [private]
```

Oil pump test duration, in seconds.

10.55.4.25 oil_pump_v12_curr_max_

```
float util::xmsg::XAlamoBit2::oil_pump_v12_curr_max_  [private]
```

Max 12V rail current during oil pump test, in amps.

10.55.4.26 oil_pump_volt_max_

```
float util::xmsg::XAlamoBit2::oil_pump_volt_max_  [private]
```

Max battery voltage during oil pump test, in volts.

10.55.4.27 oil_valve_cal_open_

```
float util::xmsg::XAlamoBit2::oil_valve_cal_open_  [private]
```

Oil valve open and clased calibration positions, in volts.

10.55.4.28 oil_valve_errors_

```
uint32_t util::xmsg::XAlamoBit2::oil_valve_errors_  [private]
```

Oil valve errors bitmask.

10.55.4.29 oil_valve_open_

```
float util::xmsg::XAlamoBit2::oil_valve_open_  [private]
```

Oil valve open and closed positions, in volts.

10.55.4.30 sensor_type_

```
uint8_t util::xmsg::XAlamoBit2::sensor_type_  [private]
```

Sensor type enum.

10.55.4.31 type_

```
uint8_t util::xmsg::XAlamoBit2::type_  [private]
```

BIT test type enum.

The documentation for this class was generated from the following file:

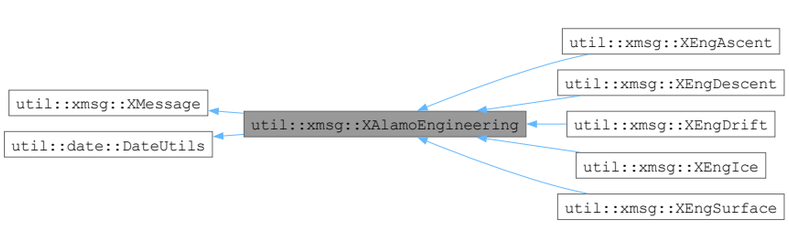
- data/xmsg/XAlamoBit2.h

10.56 util::xmsg::XAlamoEngineering Class Reference

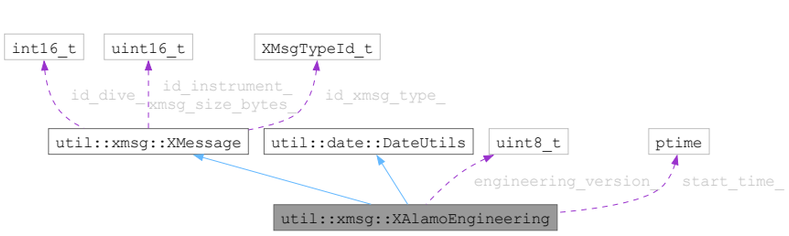
Abstract class representing a version controlled engineering message for ALAMO-type instruments.

`#include <XAlamoEngineering.h>`

Inheritance diagram for util::xmsg::XAlamoEngineering:



Collaboration diagram for util::xmsg::XAlamoEngineering:



Public Member Functions

- XAlamoEngineering (int16_t dive, uint16_t whoi, const vector< uint8_t > &data)
  Parameterized constructor.
- virtual ~XAlamoEngineering ()
  Destructor.
- const uint8_t * CheckXMsgData (const vector< uint8_t > &data, int id, int minBytes, bool parseSize=true)
  Check the X-message for the correct ID, minimum size, and message terminator.
- virtual void DecodeXMsgData (const vector< uint8_t > &data)=0
  Decode the X-Message data.
- uint8_t engineering_version () const
  Get the engineering message version identifier.
- float engineering_version_as_float () const
  Get the engineering message verion identifier as a floating point number.
- int16_t id_dive () const
  Get the dive or cycle number of this X-Message.
- uint16_t id_instrument () const
  Get the unique instrument identifier of this X-Message.
- XMsgTypeId_t id_xmsg_type () const
  Get the enumerated X-Message type identifier.
- const ptime & start_time () const
  Get the engineering message state start time.
- uint16_t xmsg_size_bytes () const
  Get the size of the X-Message, in bytes.

Static Public Member Functions

- static string GenerateCurrentUtcTimeIso8601String ()
  Generate an ISO8601 format date/time string representation of the current time, in UTC.
- static string GenerateIso8601String (const ptime &p, bool extended=true)
  Generate an ISO8601 format date/time string representation of a POSIX time object.
- static double GenerateJulianDay (const ptime &dt, const ptime &ref_dt)
  Convert a date/time to fractional days since a reference date/time.
- static ptime GetCurrentUtcTime ()
  Generate a POSIX time object representing the current date and time (rounded to the nearest second).

Protected Member Functions

- void set_engineering_version (uint8_t engineering_version)
  Set the engineering message version identifier.
- void set_id_dive (int16_t id_dive)
  Set the dive or cycle number of this message.
- void set_id_instrument (uint16_t id_instrument)
  Set the unique instrument identifier.
- void set_id_xmsg_type (XMsgTypeId_t id_xmsg_type)
  Set the X-Message type identifier.
- void set_start_time (const ptime &start_time)
  Set the engineering message state start time.
- void set_xmsg_size_bytes (uint16_t xmsg_size_bytes)
  Set the size of the X-Message, in bytes.

Private Attributes

- uint8_t engineering_version_
  The engineering message version identifier.
- int16_t id_dive_
  The dive or cycle number of this message.
- uint16_t id_instrument_
  The unique WHOI identifier of this instrument.
- XMsgTypeId_t id_xmsg_type_
  The X-message type identifier.
- ptime start_time_
  The engineering message state start time.
- uint16_t xmsg_size_bytes_
  The size of the X message, in bytes.

10.56.1 Detailed Description

Abstract class representing a version controlled engineering message for ALAMO-type instruments.

Note

The X-Message types extended from this class are used by ALAMO-type instruments only.

The X-Message types extended from this class were introduced in the X-Message format specification version 2.1.

10.56.2 Constructor & Destructor Documentation

10.56.2.1 XAlamoEngineering()

```
util::xmsg::XAlamoEngineering::XAlamoEngineering (
            int16_t dive,
            uint16_t whoi,
            const vector< uint8_t > & data)  [inline]
```

Parameterized constructor.

Parameters

in	*dive*	The dive or cycle number of this message.
in	*whoi*	The unique WHOI identifier.
in	*data*	The binary X-message data to decode.

10.56.2.2 ~XAlamoEngineering()

```
virtual util::xmsg::XAlamoEngineering::~XAlamoEngineering ()  [inline], [virtual]
```

Destructor.

10.56.3 Member Function Documentation

10.56.3.1 CheckXMsgData()

```
const uint8_t * util::xmsg::XMessage::CheckXMsgData (
            const vector< uint8_t > & data,
            int id,
            int minBytes,
            bool parseSize = true)  [inline], [inherited]
```

Check the X-message for the correct ID, minimum size, and message terminator.

Parameters

in	*data*	The X-message binary data to check.
in	*id*	The expected ID of the X-message. If this field is less that 0, then the ID check will be skipped.
in	*minBytes*	The minimum number of bytes expected in a valid message.
in	*parseSize*	If TRUE, then the message size field will be parsed from the X-Message. Otherwise, the size field will not be parsed.

Exceptions

util::log::MessageException	if • the message does not contain the expected ID, • the message does not contain the minimum number of expected bytes, • the message does not contain a terminator (';') in the correct location, or, • any other error occurs.

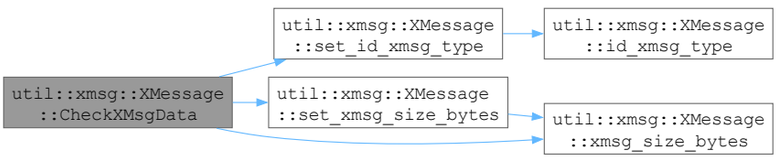
Returns

A pointer to the underlying data buffer of data.

References CATCH_BLOCK, EXC_THROW, lwrn, util::xmsg::XMessage::set_id_xmsg_type(), util::xmsg::XMessage::set_xmsg_size_byte
u16, and util::xmsg::XMessage::xmsg_size_bytes().

Referenced by util::xmsg::XAcousticPosition::DecodeXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XDriftPressure::Dec
util::xmsg::XDriftSalinity::DecodeXMsgData(), util::xmsg::XDriftTemperature::DecodeXMsgData(), util::xmsg::XEngAbort::DecodeXMsgDa
util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngBIT::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(),
util::xmsg::XEngDiagnostic::DecodeXMsgData(), util::xmsg::XEngDrift::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(),
util::xmsg::XEngStandard::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XGPS::DecodeXMsgData(),
util::xmsg::XHiResPressure::DecodeXMsgData(), util::xmsg::XHiResSalinity::DecodeXMsgData(), util::xmsg::XHiResTemperature::Decod
util::xmsg::XMissionCmdEcho::DecodeXMsgData(), util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XProfilePressure:
util::xmsg::XProfileSalinity::DecodeXMsgData(), util::xmsg::XProfileTemperature::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgD
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the call graph for this function:



10.56.3.2 DecodeXMsgData()

```
virtual void util::xmsg::XMessage::DecodeXMsgData (
            const vector< uint8_t > & data)  [pure virtual], [inherited]
```

Decode the X-Message data.

Parameters

in	*data*	The encoded X message to decode.

Implemented in util::xmsg::AbstractXFallRise, util::xmsg::AbstractXSensorData, util::xmsg::XAcousticPosition, util::xmsg::XArgo, util::xmsg::XDriftPressure, util::xmsg::XDriftSalinity, util::xmsg::XDriftTemperature, util::xmsg::XEngAbort, util::xmsg::XEngAscent, util::xmsg::XEngBIT, util::xmsg::XEngDescent, util::xmsg::XEngDiagnostic, util::xmsg::XEngDrift, util::xmsg::XEngIce, util::xmsg::XEngSurface, util::xmsg::XFall, util::xmsg::XGPS, util::xmsg::XHiResPressure, util::xmsg::XHiResSalinity, util::xmsg::XHiResTemperature, util::xmsg::XMessageIndexable, util::xmsg::XMissionCmdEcho, util::xmsg::XMissionConfiguration, util::xmsg::XProfilePressure, util::xmsg::XProfileSalinity, util::xmsg::XProfileTemperature, util::xmsg::XPump, util::xmsg::XRise, util::xmsg::XSurfaceData, util::xmsg::XValve, and util::xmsg::XWave.

10.56.3.3 engineering_version()

```
uint8_t util::xmsg::XAlamoEngineering::engineering_version () const  [inline]
```

Get the engineering message version identifier.

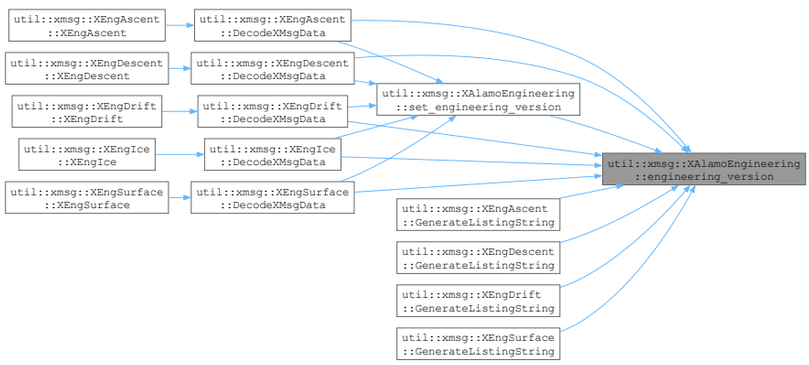
Returns

The engineering message version identifier.

References engineering_version_.

Referenced by util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(), util::xmsg::XEngDrift::Decode
util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XEngAscent::GenerateListingString(),
util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), util::xmsg::XEngSurface::GenerateListin
and set_engineering_version().

Here is the caller graph for this function:



10.56.3.4 engineering_version_as_float()

```
float util::xmsg::XAlamoEngineering::engineering_version_as_float () const  [inline]
```

Get the engineering message verion identifier as a floating point number.

Returns

The engineering message verion identifier as a floating point number.

References engineering_version_.

10.56.3.5 GenerateCurrentUtcTimeIso8601String()

```
static string util::date::DateUtils::GenerateCurrentUtcTimeIso8601String ()  [inline], [static],
[inherited]
```

Generate an ISO8601 format date/time string representation of the current time, in UTC.

Note

All date/time objects use the UTC time-zone.

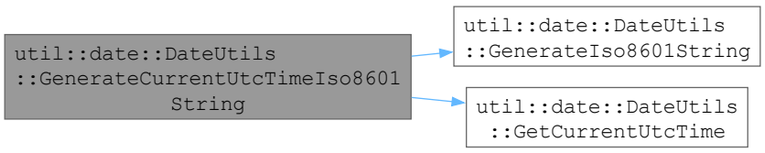
Returns

An ISO8601 format date/time string representation of the current time.

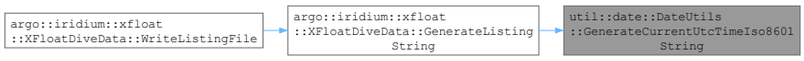
References util::date::DateUtils::GenerateIso8601String(), and util::date::DateUtils::GetCurrentUtcTime().

Referenced by argo::iridium::xfloat::XFloatDiveData::GenerateListingString().

Here is the call graph for this function:



Here is the caller graph for this function:



10.56.3.6 GenerateIso8601String()

```
static string util::date::DateUtils::GenerateIso8601String (
            const ptime & p,
            bool extended = true)  [inline], [static], [inherited]
```

Generate an ISO8601 format date/time string representation of a POSIX time object.

Note

All date/time objects use the UTC time-zone.

Parameters

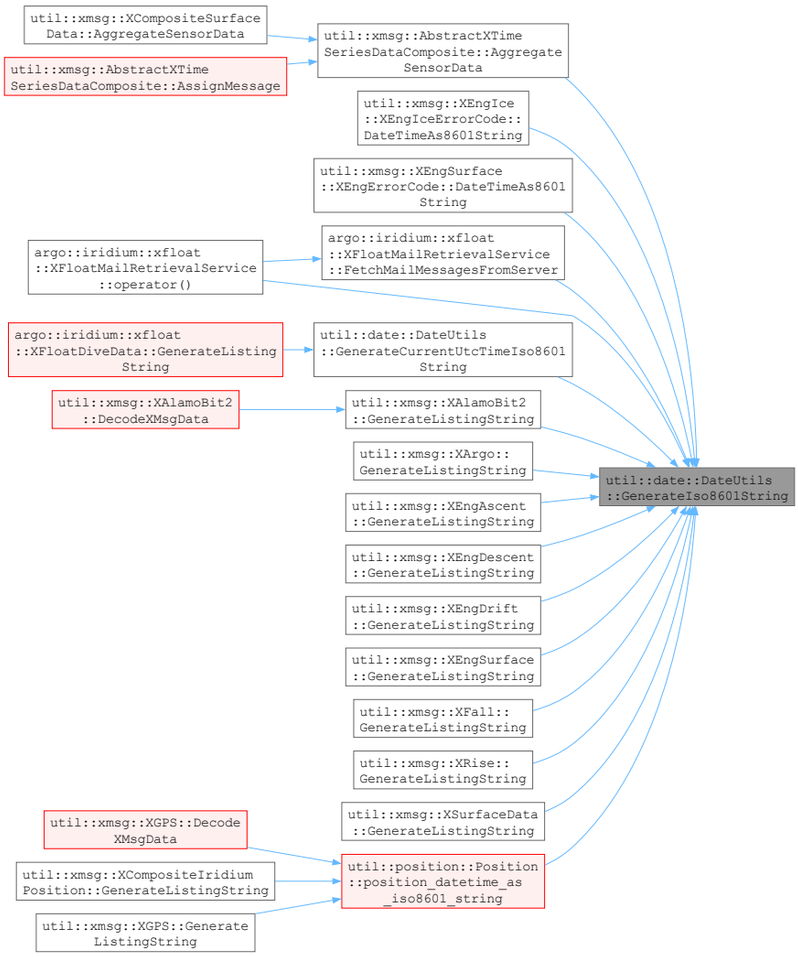
in	*p*	POSIX time from which to generate string.
in	*extended*	If TRUE, use the ISO 8601 extended format (YYYY-MM-DDThh:mm:ssZ); otherwise, use the basic format (YYYYMMDDThhmmss).

Returns

An ISO8601 format date/time string representation of the POSIX time object. If p is equal to boost::posix_↩ time::not_a_date_time, then the string "0000-00-00T00:00:00Z" (extended) or "00000000000000" (basic) will be returned.

Referenced by util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::AggregateSensorData(), util::xmsg::XEngIce::XEngIce
util::xmsg::XEngSurface::XEngErrorCode::DateTimeAs8601String(), argo::iridium::xfloat::XFloatMailRetrievalService::FetchMailMessages
util::date::DateUtils::GenerateCurrentUtcTimeIso8601String(), util::xmsg::XAlamoBit2::GenerateListingString(), util::xmsg::XArgo::Genera
util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListing
util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XRise::GenerateListingString(),
util::xmsg::XSurfaceData::GenerateListingString(), argo::iridium::xfloat::XFloatMailRetrievalService::operator()(), and
util::position::Position::position_datetime_as_iso8601_string().

Here is the caller graph for this function:



10.56.3.7 GenerateJulianDay()

```
static double util::date::DateUtils::GenerateJulianDay (
            const ptime & dt,
            const ptime & ref_dt)  [inline], [static], [inherited]
```

Convert a date/time to fractional days since a reference date/time.

Parameters

in	*dt*	The date/time to convert.
in	*ref↩ _dt*	The reference date/time (day 0).

Exceptions

util::log::MessageException	if either date/time is not valid.

Returns

The number of days (with millisecond resolution) from `ref_dt` to `dt`.

References CATCH_BLOCK, and EXC_THROW.

10.56.3.8 GetCurrentUtcTime()

```
static ptime util::date::DateUtils::GetCurrentUtcTime ()  [inline], [static], [inherited]
```

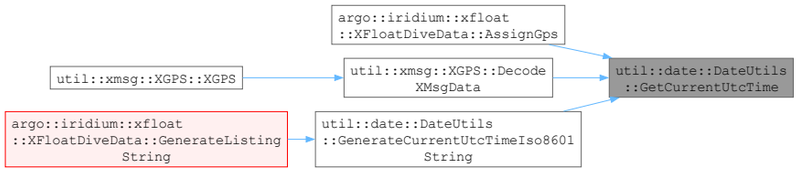
Generate a POSIX time object representing the current date and time (rounded to the nearest second).

Returns

A POSIX time object representing the current date and time (rounded to the nearest second).

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignGps(), util::xmsg::XGPS::DecodeXMsgData(), and util::date::DateUtils::GenerateCurrentUtcTimeIso8601String().

Here is the caller graph for this function:



10.56.3.9 id_dive()

```
int16_t util::xmsg::XMessage::id_dive () const  [inline], [inherited]
```

Get the dive or cycle number of this X-Message.

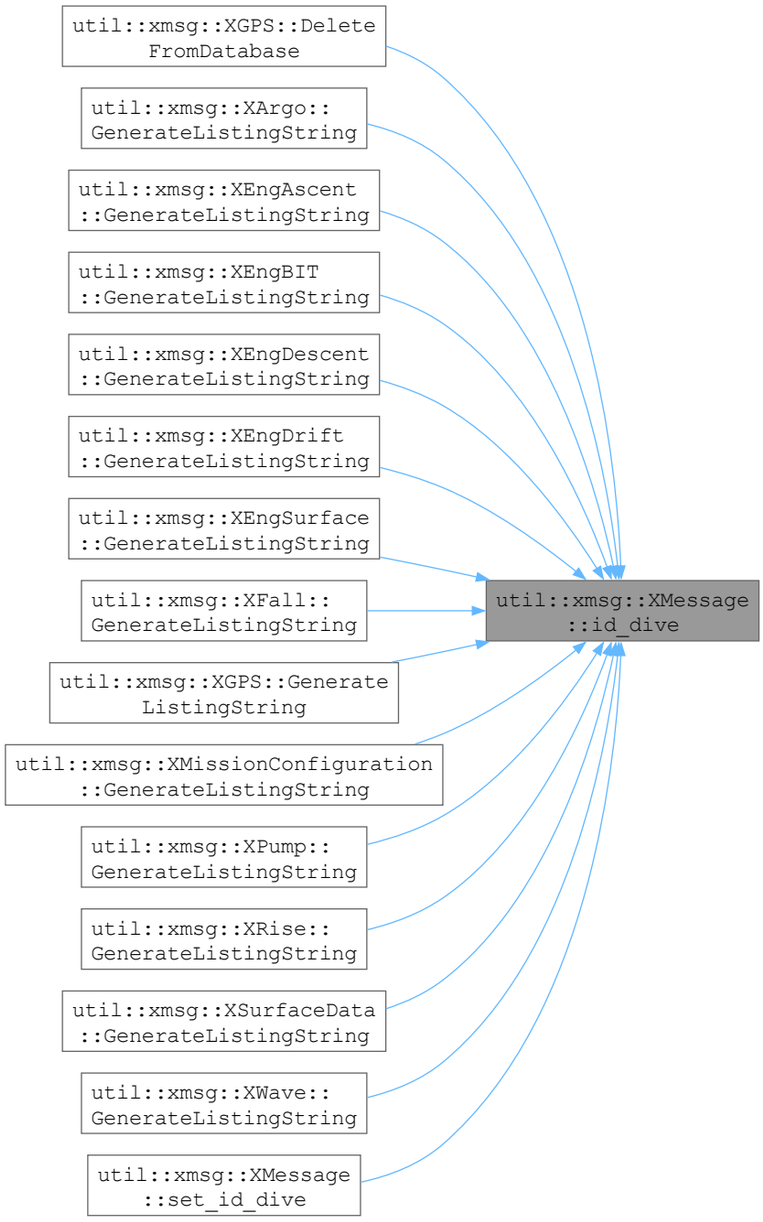
Returns

The dive or cycle number of this X-Message.

References util::xmsg::XMessage::id_dive_.

Referenced by util::xmsg::XGPS::DeleteFromDatabase(), util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateLis
util::xmsg::XEngBIT::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingStr
util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(),
util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing
util::xmsg::XSurfaceData::GenerateListingString(), util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_dive().

Here is the caller graph for this function:



10.56.3.10 id_instrument()

```
uint16_t util::xmsg::XMessage::id_instrument () const  [inline], [inherited]
```

Get the unique instrument identifier of this X-Message.

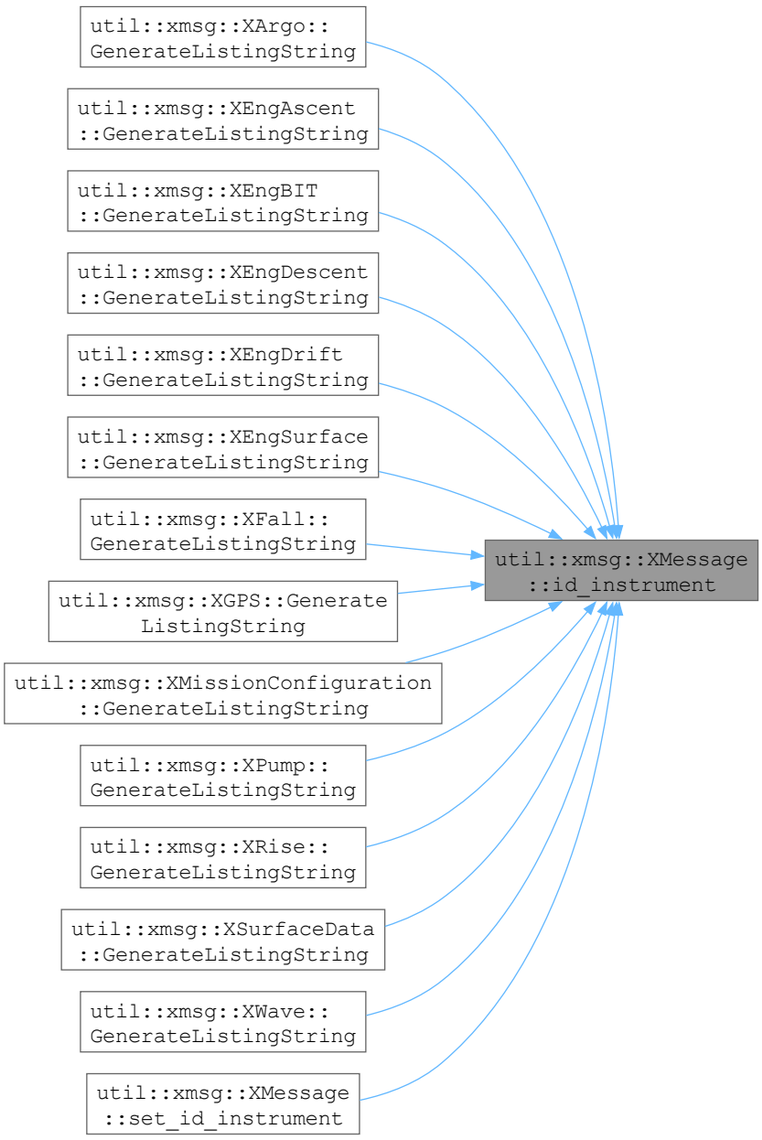
Returns

The unique instrument identifier of this X-Message.

References util::xmsg::XMessage::id_instrument_.

Referenced by util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngBIT::Generate
util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), util::xmsg::XEngSurface::GenerateListin
util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingS
util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(),
util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_instrument().

Here is the caller graph for this function:



10.56.3.11 id_xmsg_type()

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type () const  [inline], [inherited]
```

Get the enumerated X-Message type identifier.

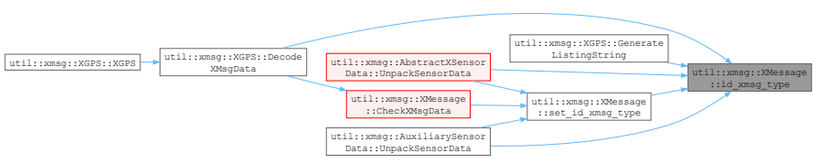
Returns

The enumerated X-Message type identifier.

References util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMessage::set_id_xmsg_typ
util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the caller graph for this function:



10.56.3.12 set_engineering_version()

```
void util::xmsg::XAlamoEngineering::set_engineering_version (
            uint8_t engineering_version)  [inline], [protected]
```

Set the engineering message version identifier.

Parameters

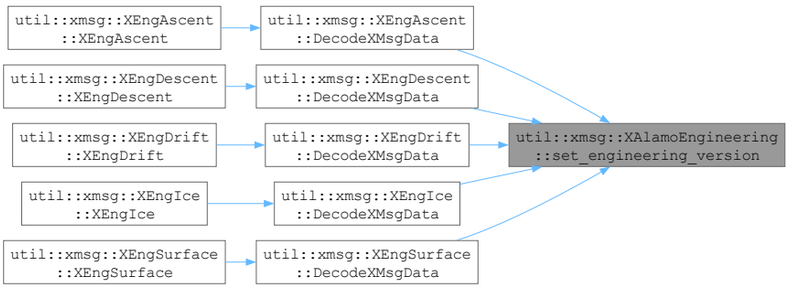
in	*engineering_version*	The engineering message version identifier.

References engineering_version(), and engineering_version_.

Referenced by util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(), util::xmsg::XEngDrift::Decode
util::xmsg::XEngIce::DecodeXMsgData(), and util::xmsg::XEngSurface::DecodeXMsgData().

Here is the call graph for this function:

Here is the caller graph for this function:



10.56.3.13 set_id_dive()

```
void util::xmsg::XMessage::set_id_dive (
            int16_t id_dive)  [inline], [protected], [inherited]
```

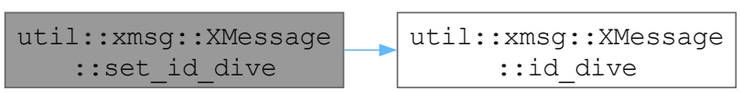
Set the dive or cycle number of this message.

Parameters

in	*id_dive*	The dive or cycle number of this message to set.

References util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::id_dive_.

Here is the call graph for this function:



10.56.3.14 set_id_instrument()

```
void util::xmsg::XMessage::set_id_instrument (
            uint16_t id_instrument)  [inline], [protected], [inherited]
```

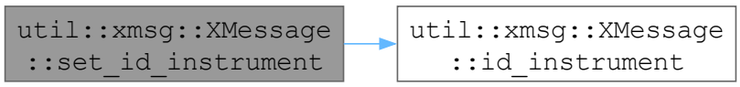
Set the unique instrument identifier.

Parameters

in	*id_instrument*	The unique instrument identifier to set.

References util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::id_instrument_.

Here is the call graph for this function:



10.56.3.15 set_id_xmsg_type()

```
void util::xmsg::XMessage::set_id_xmsg_type (
            XMsgTypeId_t id_xmsg_type)  [inline], [protected], [inherited]
```

Set the X-Message type identifier.

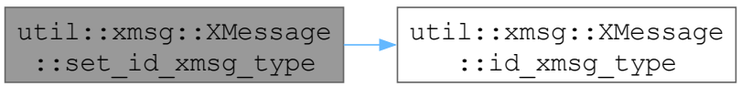
Parameters

in	*id_xmsg_type*	The X-Message type identifier to set.

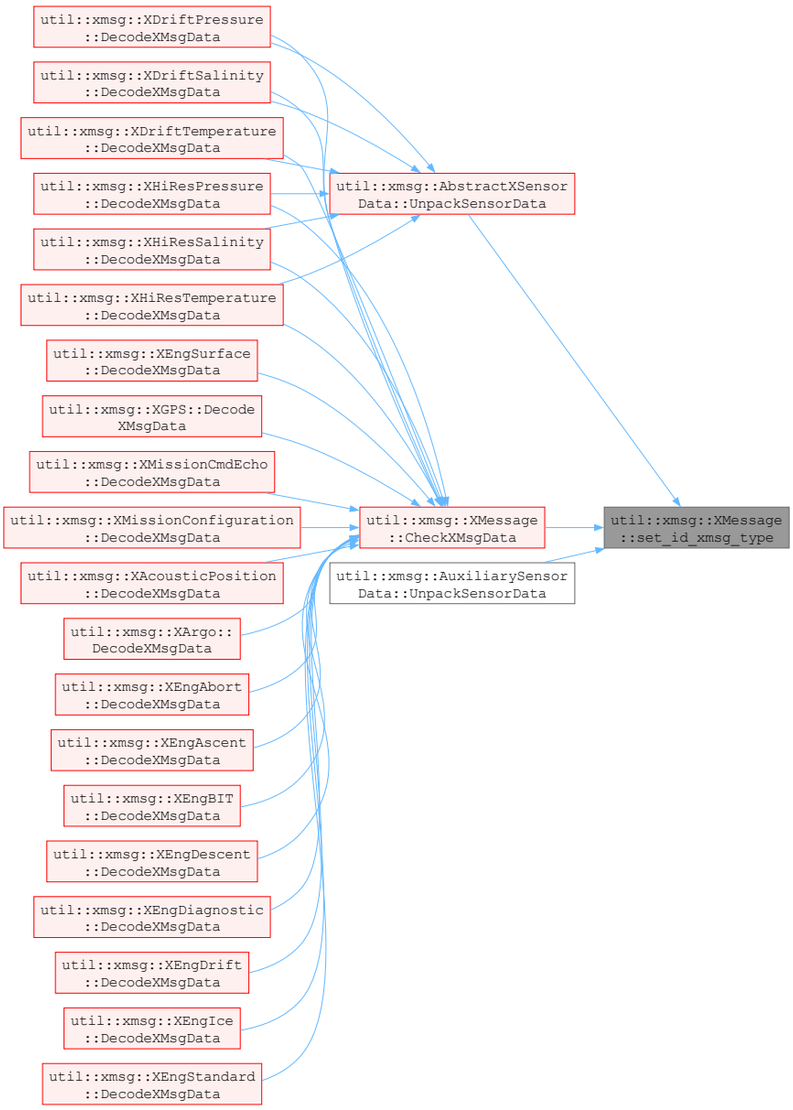
References util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.56.3.16 set_start_time()

```
void util::xmsg::XAlamoEngineering::set_start_time (
            const ptime & start_time)  [inline], [protected]
```

Set the engineering message state start time.

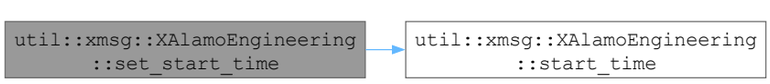
Parameters

in	*start_time*	The engineering message state start time.

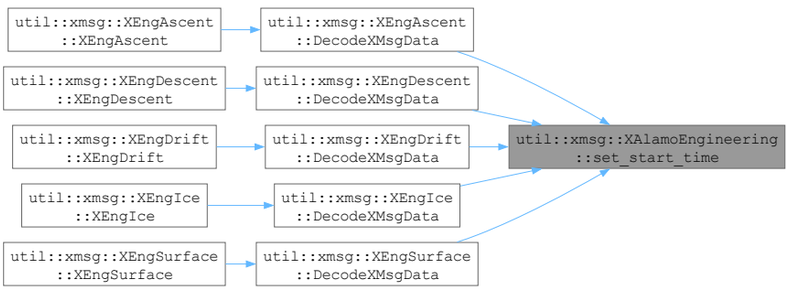
References start_time(), and start_time_.

Referenced by util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(), util::xmsg::XEngDrift::Decode
util::xmsg::XEngIce::DecodeXMsgData(), and util::xmsg::XEngSurface::DecodeXMsgData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.56.3.17 set_xmsg_size_bytes()

```
void util::xmsg::XMessage::set_xmsg_size_bytes (
            uint16_t xmsg_size_bytes)  [inline], [protected], [inherited]
```

Set the size of the X-Message, in bytes.

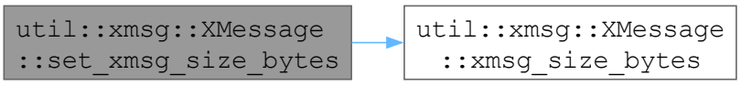
Parameters

in	*xmsg_size_bytes*	The size of the X-Message to set, in bytes.

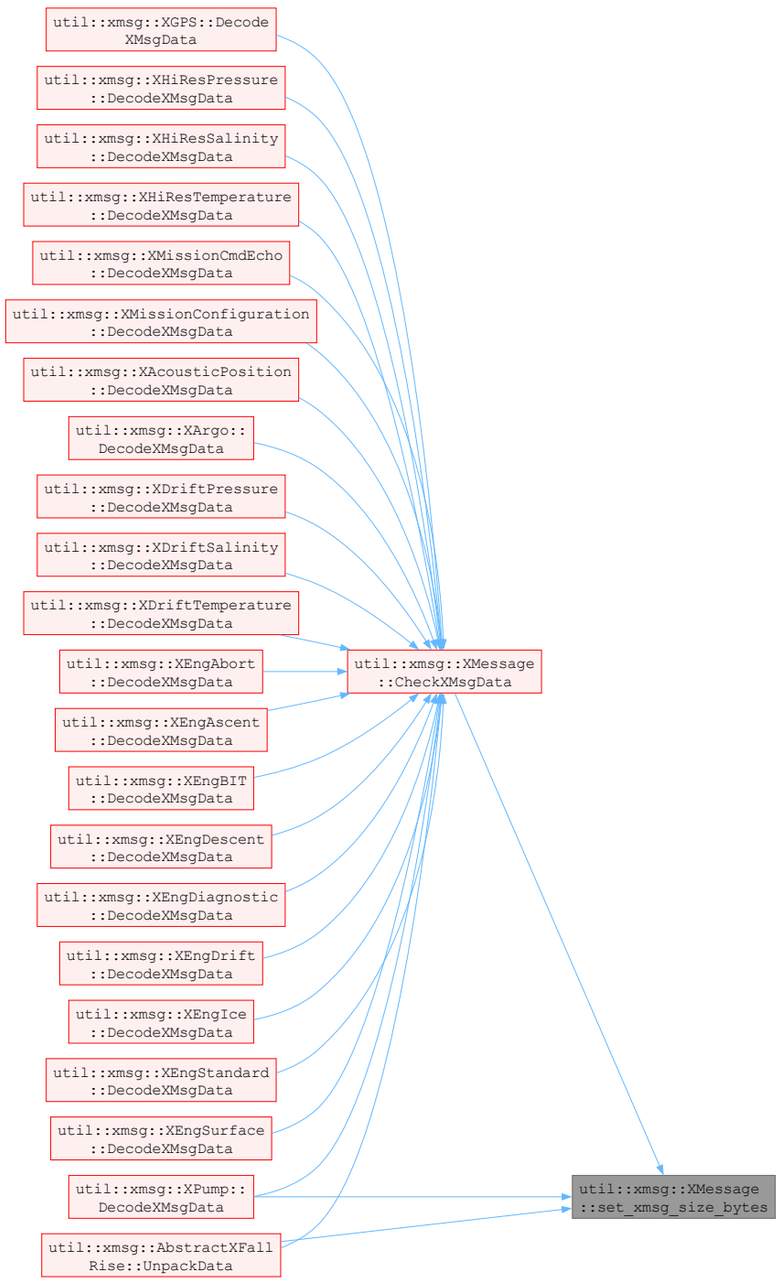
References util::xmsg::XMessage::xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XPump::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::Unpa

Here is the call graph for this function:



Here is the caller graph for this function:



10.56.3.18 start_time()

```
const ptime & util::xmsg::XAlamoEngineering::start_time () const  [inline]
```

Get the engineering message state start time.

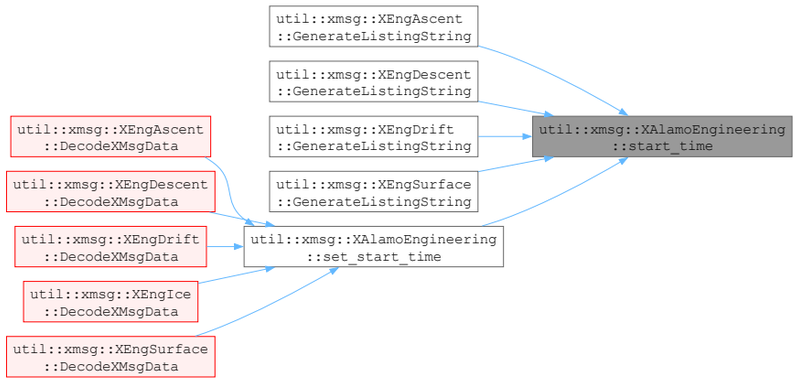
Returns

The engineering message state start time.

References start_time_.

Referenced by util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), util::xmsg::XEngSurface::GenerateListingString(), and set_start_time().

Here is the caller graph for this function:



10.56.3.19 xmsg_size_bytes()

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes () const  [inline], [inherited]
```

Get the size of the X-Message, in bytes.

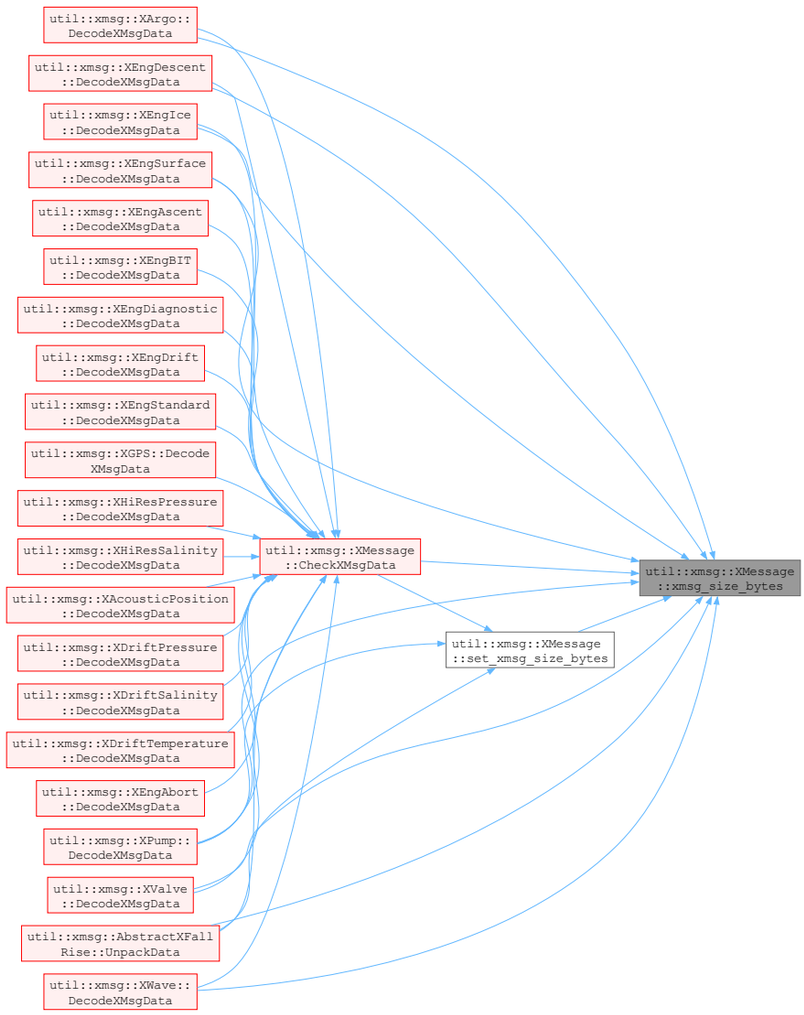
Returns

The size of the X-Message, in bytes.

References util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgI
util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(),
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), util::xmsg::XMessage::set_xmsg_size_bytes(),
and util::xmsg::AbstractXFallRise::UnpackData().

Here is the caller graph for this function:



10.56.4 Member Data Documentation

10.56.4.1 engineering_version_

```
uint8_t util::xmsg::XAlamoEngineering::engineering_version_  [private]
```

The engineering message version identifier.

Referenced by engineering_version(), engineering_version_as_float(), and set_engineering_version().

10.56.4.2 id_dive_

```
int16_t util::xmsg::XMessage::id_dive_  [private], [inherited]
```

The dive or cycle number of this message.

Referenced by util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::set_id_dive().

10.56.4.3 id_instrument_

```
uint16_t util::xmsg::XMessage::id_instrument_  [private], [inherited]
```

The unique WHOI identifier of this instrument.

Referenced by util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::set_id_instrument().

10.56.4.4 id_xmsg_type_

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type_  [private], [inherited]
```

The X-message type identifier.

Referenced by util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::set_id_xmsg_type().

10.56.4.5 start_time_

```
ptime util::xmsg::XAlamoEngineering::start_time_  [private]
```

The engineering message state start time.

Referenced by set_start_time(), and start_time().

10.56.4.6 xmsg_size_bytes_

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes_  [private], [inherited]
```

The size of the X message, in bytes.

Referenced by util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes().

The documentation for this class was generated from the following file:

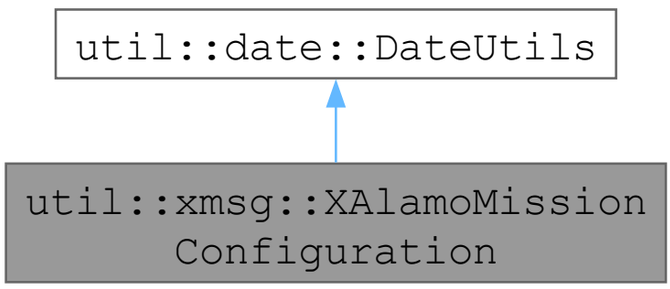
- data/xmsg/XAlamoEngineering.h

10.57 util::xmsg::XAlamoMissionConfiguration Class Reference

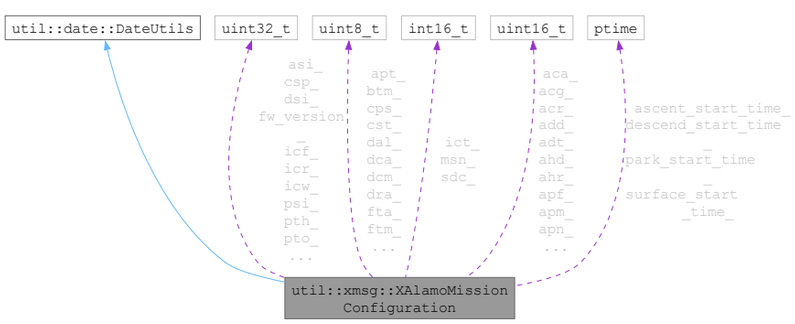
Class representing an ALAMO Mission Configuration message.

```
#include <XAlamoMissionConfiguration.h>
```

Inheritance diagram for util::xmsg::XAlamoMissionConfiguration:



Collaboration diagram for util::xmsg::XAlamoMissionConfiguration:



Public Member Functions

- XAlamoMissionConfiguration (const vector< uint8_t > &data)

  Parameterized Constructor.
- ~XAlamoMissionConfiguration ()

  Destructor.
- uint16_t **aca** () const
- uint16_t **acg** () const
- uint16_t **acr** () const
- uint16_t **add** () const
- uint16_t **adt** () const
- uint16_t **ahd** () const
- uint16_t **ahr** () const
- template<typename T = float>
  T **ahr_m_per_s** () const
- uint16_t **apf** () const
- uint16_t **apm** () const
- uint16_t **apn** () const
- unsigned int **apt** () const
- uint16_t **arc** () const
- uint16_t **arr** () const
- template<typename T = float>
  T **arr_m_per_s** () const
- const ptime & **ascent_start_time** () const
- uint16_t **asd** () const
- uint32_t **asi** () const
- template<typename T = float>
  T **asi_sec** () const
- uint16_t **asn** () const
- uint8_t **btm** () const
- uint16_t **cpb1** () const
- uint16_t **cpb2** () const
- uint16_t **cpb3** () const
- uint16_t **cpn1** () const
- template<typename T = float>
  T **cpn1_dbar** () const
- uint16_t **cpn2** () const
- template<typename T = float>
  T **cpn2_dbar** () const
- uint16_t **cpn3** () const
- template<typename T = float>
  T **cpn3_dbar** () const
- uint16_t **cpp1** () const
- template<typename T = float>
  T **cpp1_sec** () const
- uint16_t **cpp2** () const
- template<typename T = float>
  T **cpp2_sec** () const
- uint16_t **cpp3** () const
- template<typename T = float>
  T **cpp3_sec** () const
- uint8_t **cps** () const
- uint32_t **csp** () const
- template<typename T = float>
  T **csp_sec** () const
- uint8_t **cst** () const
- uint16_t **ctf** () const
- uint16_t **ctl** () const
- uint16_t **ctu** () const
- uint8_t **dal** () const
- uint8_t **dca** () const
- uint16_t **dcd** () const
- uint16_t **dcf** () const
- uint8_t **dcm** () const
- uint16_t **dcp** () const
- uint16_t **dcr** () const
- uint16_t **dct** () const
- uint16_t **dcx** () const

- uint16_t **ddc** () const
- uint16_t **ddd** () const
- uint16_t **ddf** () const
- uint16_t **ddp** () const
- uint16_t **dds** () const
- uint16_t **ddt** () const
- void DecodeXMsgData (const vector< uint8_t > &data)

  Decode the ALAMO mission configuration message from the binary message data.
- const ptime & **descend_start_time** () const
- uint16_t **dft** () const
- uint16_t **dns** () const
- uint16_t **dpd** () const
- uint16_t **dpt** () const
- uint8_t **dra** () const
- template<typename T = float>
  T **dra_m_per_s** () const
- uint16_t **drc** () const
- uint16_t **drt** () const
- uint16_t **dsd** () const
- uint32_t **dsi** () const
- template<typename T = float>
  T **dsi_sec** () const
- uint16_t **dst** () const
- uint16_t **dsw** () const
- uint16_t **dtd** () const
- uint16_t **duc** () const
- uint16_t **dut** () const
- uint16_t **dvd** () const
- uint16_t **dvt** () const
- uint16_t **dvw** () const
- uint16_t **fos** () const
- uint8_t **fta** () const
- uint8_t **ftm** () const
- uint16_t **fto** () const
- uint32_t **fw_version** () const
- const string GenerateListingString (int whoi, int dive, int idx) const

  Generate a listing entry string from this ALAMO mission configuration message object.
- uint16_t **iat** () const
- uint16_t **ica0** () const
- uint16_t **ica1** () const
- uint16_t **icb0** () const
- uint16_t **icb1** () const
- uint16_t **icd0** () const
- uint16_t **icd1** () const
- uint32_t **icf** () const
- uint8_t **icg0** () const
- uint8_t **icg1** () const
- uint8_t **icm** () const
- uint16_t **icn** () const
- uint16_t **ico0** () const
- uint16_t **ico1** () const
- uint32_t **icr0** () const
- uint32_t **icr1** () const
- uint16_t **ics** () const
- int16_t **ict** () const
- template<typename T = float>
  T **ict_degc** () const
- uint32_t **icw** () const
- uint16_t **icx** () const
- uint16_t **ilt0** () const
- uint16_t **ilt1** () const
- uint16_t **imm** () const
- uint16_t **imt0** () const
- uint16_t **imt1** () const
- uint8_t **isa** () const
- uint16_t **isd** () const
- uint8_t **ism** () const

- uint16_t **iso** () const
- uint8_t **isp** () const
- uint8_t **iss0** () const
- uint8_t **iss1** () const
- uint16_t **ist** () const
- template<typename T = float>
  T **ist_psu** () const
- uint16_t **isv** () const
- uint16_t **isw** () const
- uint8_t **iue0** () const
- uint8_t **iue1** () const
- uint8_t **mam** () const
- uint16_t **mat** () const
- uint16_t **mds** () const
- unsigned int **msg_version** () const
- int16_t **msn** () const
- uint16_t **mtd** () const
- uint16_t **mtt** () const
- template<typename T = float>
  T **mtt_sec** () const
- const ptime & **park_start_time** () const
- uint16_t **pbh** () const
- uint16_t **pbl** () const
- uint16_t **pc0** () const
- uint16_t **pc1** () const
- uint16_t **pdt** () const
- uint16_t **pfc** () const
- uint16_t **pfd** () const
- uint8_t **pfm** () const
- uint16_t **pfr** () const
- uint16_t **pft** () const
- uint16_t **pfv** () const
- uint16_t **pnb** () const
- template<typename T = float>
  T **pnb_v** () const
- uint16_t **pnc** () const
- template<typename T = float>
  T **pnc_psi** () const
- uint16_t **pns** () const
- uint16_t **pp0** () const
- uint16_t **pp1** () const
- uint8_t **psa** () const
- uint16_t **psd** () const
- uint32_t **psi** () const
- template<typename T = float>
  T **psi_sec** () const
- uint8_t **psm** () const
- uint16_t **pst** () const
- uint16_t **psv** () const
- uint16_t **psw** () const
- uint32_t **pth** () const
- uint32_t **pto** () const
- uint16_t **pv0** () const
- uint16_t **pv5** () const
- uint16_t **pvh** () const
- uint16_t **pvl** () const
- uint16_t **pxc** () const
- template<typename T = float>
  T **pxc_psi** () const
- uint8_t **sam** () const
- uint16_t **san** () const
- template<typename T = float>
  T **san_psi** () const
- uint16_t **sao** () const
- uint16_t **sap** () const
- uint16_t **sax** () const
- template<typename T = float>
  T **sax_psi** () const

- uint8_t **sbm** () const
- uint16_t **sbt** () const
- int16_t **sdc** () const
- uint16_t **sds** () const
- uint32_t **sdt** () const
- int **sensor_type** () const
- uint16_t **set** () const
- uint16_t **sfn** () const
- uint16_t **sgt** () const
- uint16_t **spt** () const
- uint16_t **ssa** () const
- uint32_t **ssi** () const
- template<typename T = float>
  T **ssi_sec** () const
- uint16_t **ssp** () const
- const ptime & **surface_start_time** () const
- int **xas** () const

Static Public Member Functions

- static string GenerateCurrentUtcTimeIso8601String ()

  Generate an ISO8601 format date/time string representation of the current time, in UTC.
- static string GenerateIso8601String (const ptime &p, bool extended=true)

  Generate an ISO8601 format date/time string representation of a POSIX time object.
- static double GenerateJulianDay (const ptime &dt, const ptime &ref_dt)

  Convert a date/time to fractional days since a reference date/time.
- static ptime GetCurrentUtcTime ()

  Generate a POSIX time object representing the current date and time (rounded to the nearest second).

Protected Member Functions

- void **set_aca** (uint16_t aca)
- void **set_acg** (uint16_t acg)
- void **set_acr** (uint16_t acr)
- void **set_add** (uint16_t add)
- void **set_adt** (uint16_t adt)
- void **set_ahd** (uint16_t ahd)
- void **set_ahr** (uint16_t ahr)
- void **set_apf** (uint16_t apf)
- void **set_apm** (uint16_t apm)
- void **set_apn** (uint16_t apn)
- void **set_apt** (uint8_t apt)
- void **set_arc** (uint16_t arc)
- void **set_arr** (uint16_t arr)
- void **set_ascent_start_time** (const ptime &ascent_start_time)
- void **set_asd** (uint16_t asd)
- void **set_asi** (uint32_t asi)
- void **set_asn** (uint16_t asn)
- void **set_btm** (uint8_t btm)
- void **set_cpb1** (uint16_t cpb)
- void **set_cpb2** (uint16_t cpb)
- void **set_cpb3** (uint16_t cpb)
- void **set_cpn1** (uint16_t cpn)
- void **set_cpn2** (uint16_t cpn)
- void **set_cpn3** (uint16_t cpn)
- void **set_cpp1** (uint16_t cpp)
- void **set_cpp2** (uint16_t cpp)
- void **set_cpp3** (uint16_t cpp)
- void **set_cps** (uint8_t cps)
- void **set_csp** (uint32_t csp)
- void **set_cst** (uint8_t cst)
- void **set_ctf** (uint16_t ctf)
- void **set_ctl** (uint16_t ctl)
- void **set_ctu** (uint16_t ctu)
- void **set_dal** (uint8_t dal)

- void **set_dca** (uint8_t dca)
- void **set_dcd** (uint16_t dcd)
- void **set_dcf** (uint16_t dcf)
- void **set_dcm** (uint8_t dcm)
- void **set_dcp** (uint16_t dcp)
- void **set_dcr** (uint16_t dcr)
- void **set_dct** (uint16_t dct)
- void **set_dcx** (uint16_t dcx)
- void **set_ddc** (uint16_t ddc)
- void **set_ddd** (uint16_t ddd)
- void **set_ddf** (uint16_t ddf)
- void **set_ddp** (uint16_t ddp)
- void **set_dds** (uint16_t dds)
- void **set_ddt** (uint16_t ddt)
- void **set_descend_start_time** (const ptime &descend_start_time)
- void **set_dft** (uint16_t dft)
- void **set_dns** (uint16_t dns)
- void **set_dpd** (uint16_t dpd)
- void **set_dpt** (uint16_t dpt)
- void **set_dra** (uint8_t dra)
- void **set_drc** (uint16_t drc)
- void **set_drt** (uint16_t drt)
- void **set_dsd** (uint16_t dsd)
- void **set_dsi** (uint32_t dsi)
- void **set_dst** (uint16_t dst)
- void **set_dsw** (uint16_t dsw)
- void **set_dtd** (uint16_t dtd)
- void **set_duc** (uint16_t duc)
- void **set_dut** (uint16_t dut)
- void **set_dvd** (uint16_t dvd)
- void **set_dvt** (uint16_t dvt)
- void **set_dvw** (uint16_t dvw)
- void **set_fos** (uint16_t fos)
- void **set_fta** (uint8_t fta)
- void **set_ftm** (uint8_t ftm)
- void **set_fto** (uint16_t fto)
- void **set_fw_version** (uint32_t fw_version)
- void **set_iat** (uint16_t iat)
- void **set_ica0** (uint16_t ica)
- void **set_ica1** (uint16_t ica)
- void **set_icb0** (uint16_t icb)
- void **set_icb1** (uint16_t icb)
- void **set_icd0** (uint16_t icd)
- void **set_icd1** (uint16_t icd)
- void **set_icf** (uint32_t icf)
- void **set_icg0** (uint8_t icg)
- void **set_icg1** (uint8_t icg)
- void **set_icm** (uint8_t icm)
- void **set_icn** (uint16_t icn)
- void **set_ico0** (uint16_t ico)
- void **set_ico1** (uint16_t ico)
- void **set_icr0** (uint32_t icr)
- void **set_icr1** (uint32_t icr)
- void **set_ics** (uint16_t ics)
- void **set_ict** (int16_t ict)
- void **set_icw** (uint32_t icw)
- void **set_icx** (uint16_t icx)
- void **set_ilt0** (uint16_t ilt)
- void **set_ilt1** (uint16_t ilt)
- void **set_imm** (uint16_t imm)
- void **set_imt0** (uint16_t imt)
- void **set_imt1** (uint16_t imt)
- void **set_isa** (uint8_t isa)
- void **set_isd** (uint16_t isd)
- void **set_ism** (uint8_t ism)
- void **set_iso** (uint16_t iso)
- void **set_isp** (uint8_t isp)
- void **set_iss0** (uint8_t iss)

- void **set_iss1** (uint8_t iss)
- void **set_ist** (uint16_t ist)
- void **set_isv** (uint16_t isv)
- void **set_isw** (uint16_t isw)
- void **set_iue0** (uint8_t iue)
- void **set_iue1** (uint8_t iue)
- void **set_mam** (uint8_t mam)
- void **set_mat** (uint16_t mat)
- void **set_mds** (uint16_t mds)
- void **set_msg_version** (uint8_t msg_version)
- void **set_msn** (int16_t msn)
- void **set_mtd** (uint16_t mtd)
- void **set_mtt** (uint16_t mtt)
- void **set_park_start_time** (const ptime &park_start_time)
- void **set_pbh** (uint16_t pbh)
- void **set_pbl** (uint16_t pbl)
- void **set_pc0** (uint16_t pc0)
- void **set_pc1** (uint16_t pc1)
- void **set_pdt** (uint16_t pdt)
- void **set_pfc** (uint16_t pfc)
- void **set_pfd** (uint16_t pfd)
- void **set_pfm** (uint8_t pfm)
- void **set_pfr** (uint16_t pfr)
- void **set_pft** (uint16_t pft)
- void **set_pfv** (uint16_t pfv)
- void **set_pnb** (uint16_t pnb)
- void **set_pnc** (uint16_t pnc)
- void **set_pns** (uint16_t pns)
- void **set_pp0** (uint16_t pp0)
- void **set_pp1** (uint16_t pp1)
- void **set_psa** (uint8_t psa)
- void **set_psd** (uint16_t psd)
- void **set_psi** (uint32_t psi)
- void **set_psm** (uint8_t psm)
- void **set_pst** (uint16_t pst)
- void **set_psv** (uint16_t psv)
- void **set_psw** (uint16_t psw)
- void **set_pth** (uint32_t pth)
- void **set_pto** (uint32_t pto)
- void **set_pv0** (uint16_t pv0)
- void **set_pv5** (uint16_t pv5)
- void **set_pvh** (uint16_t pvh)
- void **set_pvl** (uint16_t pvl)
- void **set_pxc** (uint16_t pxc)
- void **set_sam** (uint8_t sam)
- void **set_san** (uint16_t san)
- void **set_sao** (uint16_t sao)
- void **set_sap** (uint16_t sap)
- void **set_sax** (uint16_t sax)
- void **set_sbm** (uint8_t sbm)
- void **set_sbt** (uint16_t sbt)
- void **set_sdc** (int16_t sdc)
- void **set_sds** (uint16_t sds)
- void **set_sdt** (uint32_t sdt)
- void **set_sensor_type** (uint8_t sensor_type)
- void **set_set** (uint16_t set)
- void **set_sfn** (uint16_t sfn)
- void **set_sgt** (uint16_t sgt)
- void **set_spt** (uint16_t spt)
- void **set_ssa** (uint16_t ssa)
- void **set_ssi** (uint32_t ssi)
- void **set_ssp** (uint16_t ssp)
- void **set_surface_start_time** (const ptime &surface_start_time)
- void **set_xas** (uint8_t xas)

Private Member Functions

- XAlamoMissionConfiguration (void)
  Default constructor.

Private Attributes

- uint16_t **aca_**
- uint16_t **acg_**
- uint16_t **acr_**
- uint16_t add_
  The ascent state data storage pressure interval, in dBar.
- uint16_t adt_
  The ascent state data storage interval timeout, in seconds.
- uint16_t **ahd_**
- uint16_t **ahr_**
- uint16_t apf_
  The ascent state pump off duration/interval/timeout, in seconds.
- uint16_t **apm_**
- uint16_t apn_
  The ascent state pump on duration/interval/timeout, in seconds.
- uint8_t apt_
  The auto-transmit mission configuration message flag.
- uint16_t **arc_**
- uint16_t arr_
  The ascent state rise rate, in centimeters per second.
- ptime ascent_start_time_
  The start time of the ascent state of the dive cycle.
- uint16_t asd_
  The ascent state surface threshold pressure, in dBar.
- uint32_t asi_
  The ascent state sampling interval/timeout, in milliseconds.
- uint16_t asn_
  The ascent number.
- uint8_t btm_
  The bluetooth mode.
- uint16_t cpb_ [3]
  The CTD sampling period pressure boundaries, in dBar.
- uint16_t cpn_ [3]
  The CTD sampling period bin sizes, in 0.1 dBar.
- uint16_t cpp_ [3]
  The CTD sampling period interval, in milliseconds.
- uint8_t cps_
  The CTD profiling mode.
- uint32_t csp_
  The CTD continuous sampling period, in milliseconds.
- uint8_t cst_
  The CTD sampling type.
- uint16_t ctf_
  The profile cutoff pressure.
- uint16_t ctl_
  The pressure sensor offset reset lower pressure limit.
- uint16_t ctu_
  The pressure sensor offset reset lower pressure limit.
- uint8_t **dal_**
- uint8_t **dca_**
- uint16_t **dcd_**
- uint16_t **dcf_**
- uint8_t **dcm_**
- uint16_t **dcp_**
- uint16_t **dcr_**
- uint16_t **dct_**
- uint16_t **dcx_**
- uint16_t **ddc_**

- uint16_t ddd_

  The descent state data storage pressure interval, in dBar.
- uint16_t **ddf_**
- uint16_t **ddp_**
- uint16_t **dds_**
- uint16_t ddt_

  The descent state data storage interval, in seconds.
- ptime descend_start_time_

  The start time of the descend state of the dive cycle.
- uint16_t dft_

  The descent state fall timeout, in seconds.
- uint16_t dns_

  The descent number.
- uint16_t **dpd_**
- uint16_t **dpt_**
- uint8_t **dra_**
- uint16_t **drc_**
- uint16_t **drt_**
- uint16_t dsd_

  The descent state start pressure, in dBar.
- uint32_t dsi_

  The descent state sampling interval, in milliseconds.
- uint16_t dst_

  The descent state start timeout, in seconds.
- uint16_t **dsw_**
- uint16_t dtd_

  The target descent/drift/park pressure, in dBar.
- uint16_t **duc_**
- uint16_t **dut_**
- uint16_t **dvd_**
- uint16_t **dvt_**
- uint16_t **dvw_**
- uint16_t **fos_**
- uint8_t **fta_**
- uint8_t **ftm_**
- uint16_t **fto_**
- uint32_t fw_version_

  The software version identifier.
- uint16_t **iat_**
- uint16_t **ica_** [2]
- uint16_t **icb_** [2]
- uint16_t **icd_** [2]
- uint32_t **icf_**
- uint8_t **icg_** [2]
- uint8_t **icm_**
- uint16_t **icn_**
- uint16_t **ico_** [2]
- uint32_t **icr_** [2]
- uint16_t **ics_**
- int16_t **ict_**
- uint32_t **icw_**
- uint16_t **icx_**
- uint16_t **ilt_** [2]
- uint16_t **imm_**
- uint16_t **imt_** [2]
- uint8_t **isa_**
- uint16_t **isd_**
- uint8_t **ism_**
- uint16_t **iso_**
- uint8_t **isp_**
- uint8_t **iss_** [2]
- uint16_t **ist_**
- uint16_t **isv_**
- uint16_t **isw_**
- uint8_t **iue_** [2]
- uint8_t mam_

The motion processing unit (MPU) data acquisition operational mode.

- uint16_t mat_

  The motion processing unit (MPU) sampling duration, in seconds.

- uint16_t mds_

  The maximum descent/fall/profile pressure, in dBar.

- uint8_t msg_version_

  The message version identifier.

- int16_t msn_

  The mission, dive, or cycle number for this mission configuration.

- uint16_t mtd_

  The initial trigger depth, in dBar.

- uint16_t mtt_

  The initial trigger timeout/interval, in milliseconds.

- ptime park_start_time_

  The start time of the park/drift state of the dive cycle.

- uint16_t **pbh_**
- uint16_t **pbl_**
- uint16_t **pc0_**
- uint16_t **pc1_**
- uint16_t pdt_

  The park/drift state data storage interval/timeout, in seconds.

- uint16_t **pfc_**
- uint16_t pfd_

  The park/drift state fall depth (i.e.

- uint8_t pfm_

  The park/drift state fall mode.

- uint16_t **pfr_**
- uint16_t **pft_**
- uint16_t **pfv_**
- uint16_t **pnb_**
- uint16_t **pnc_**
- uint16_t pns_

  The park/drift number.

- uint16_t **pp0_**
- uint16_t **pp1_**
- uint8_t psa_

  The park/drift state sampling mode.

- uint16_t **psd_**
- uint32_t psi_

  The park/drift state sample interval/timeout, in milliseconds.

- uint8_t **psm_**
- uint16_t **pst_**
- uint16_t **psv_**
- uint16_t **psw_**
- uint32_t pth_

  The bit-encoded park/drift state hour timeout trigger(s).

- uint32_t pto_

  The park/drift state timeout, in seconds.

- uint16_t **pv0_**
- uint16_t **pv5_**
- uint16_t **pvh_**
- uint16_t **pvl_**
- uint16_t **pxc_**
- uint8_t **sam_**
- uint16_t **san_**
- uint16_t **sao_**
- uint16_t **sap_**
- uint16_t **sax_**
- uint8_t sbm_

  The surface state beacon mode flag.

- uint16_t sbt_

  The surface state beacon interval timeout, in seconds.

- int16_t sdc_

  The surface state sensor depth compensation offset, in dBar.

- uint16_t sds_

The surface state sampling storage duration/interval/timeout, in seconds.

- uint32_t **sdt_**
- uint8_t sensor_type_

  The sensor type identifier.
- uint16_t set_

  The surface state Iridium session timeout, in minutes.
- uint16_t sfn_

  The surface number.
- uint16_t sgt_

  The surface state GPS timeout, in minutes.
- uint16_t **spt_**
- uint16_t **ssa_**
- uint32_t ssi_

  The surface state sampling interval timeout, in milliseconds.
- uint16_t **ssp_**
- ptime surface_start_time_

  The start time of the surface state of the dive cycle.
- uint8_t xas_

  The profile data encode method.

10.57.1 Detailed Description

Class representing an ALAMO Mission Configuration message.

10.57.2 Constructor & Destructor Documentation

10.57.2.1 XAlamoMissionConfiguration() [1/2]

```
util::xmsg::XAlamoMissionConfiguration::XAlamoMissionConfiguration (
            void )  [inline], [private]
```

Default constructor.

10.57.2.2 XAlamoMissionConfiguration() [2/2]

```
util::xmsg::XAlamoMissionConfiguration::XAlamoMissionConfiguration (
            const vector< uint8_t > & data)  [inline]
```

Parameterized Constructor.

Parameters

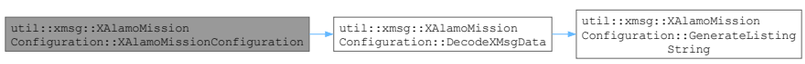
in	*data*	The binary data to decode.

Exceptions

log::MessageException	if the message is invalid or an error occurs while decoding.

References DecodeXMsgData().

Here is the call graph for this function:



10.57.2.3 ~XAlamoMissionConfiguration()

```
util::xmsg::XAlamoMissionConfiguration::~XAlamoMissionConfiguration ()  [inline]
```

Destructor.

10.57.3 Member Function Documentation

10.57.3.1 DecodeXMsgData()

```
void util::xmsg::XAlamoMissionConfiguration::DecodeXMsgData (
            const vector< uint8_t > & data)  [inline]
```

Decode the ALAMO mission configuration message from the binary message data.

Parameters

in	*data*	The binary data to decode.

Exceptions

log::MessageException	if the message is invalid or an error occurs while decoding.

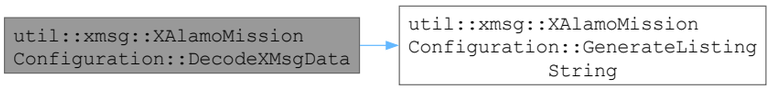
Todo Implement ALAMO mission parameter: SSP.

Todo Implement ALAMO mission parameter: SSP.

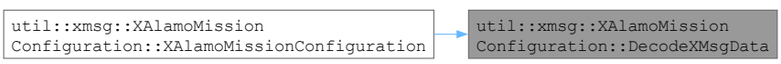
References CATCH_BLOCK, cpb_, cpn_, cpp_, EXC_THROW, GenerateListingString(), i16, ldbg, lwrn, s2000, u16, u32, and XMSG_BAD_U16.

Referenced by XAlamoMissionConfiguration().

Here is the call graph for this function:



Here is the caller graph for this function:



10.57.3.2 GenerateCurrentUtcTimeIso8601String()

```
static string util::date::DateUtils::GenerateCurrentUtcTimeIso8601String ()  [inline], [static],
[inherited]
```

Generate an ISO8601 format date/time string representation of the current time, in UTC.

Note

All date/time objects use the UTC time-zone.

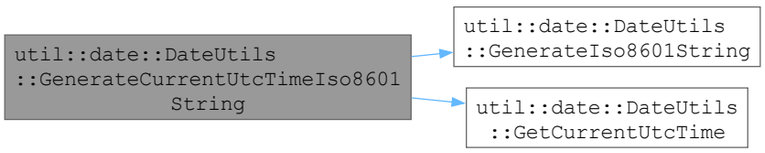
Returns

An ISO8601 format date/time string representation of the current time.

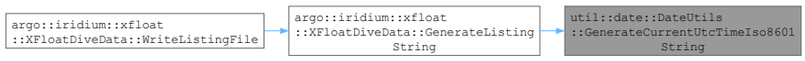
References util::date::DateUtils::GenerateIso8601String(), and util::date::DateUtils::GetCurrentUtcTime().

Referenced by argo::iridium::xfloat::XFloatDiveData::GenerateListingString().

Here is the call graph for this function:



Here is the caller graph for this function:



10.57.3.3 GenerateIso8601String()

```
static string util::date::DateUtils::GenerateIso8601String (
            const ptime & p,
            bool extended = true)  [inline], [static], [inherited]
```

Generate an ISO8601 format date/time string representation of a POSIX time object.

Note

All date/time objects use the UTC time-zone.

Parameters

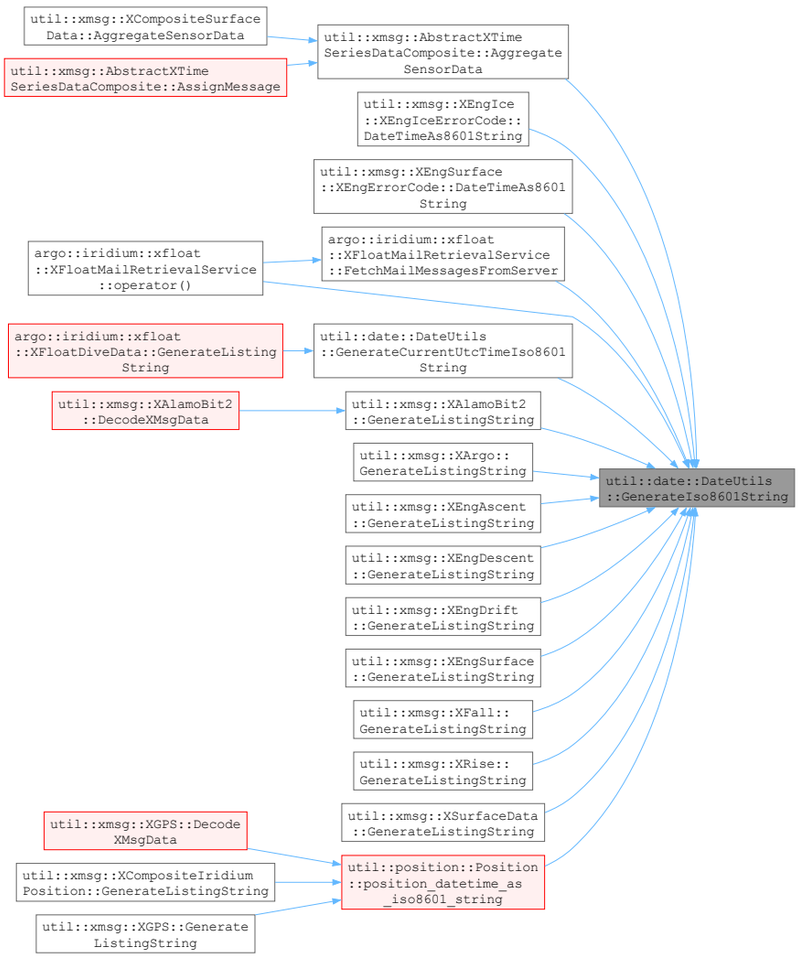
in	*p*	POSIX time from which to generate string.
in	*extended*	If TRUE, use the ISO 8601 extended format (YYYY-MM-DDThh:mm:ssZ); otherwise, use the basic format (YYYYMMDDThhmmss).

Returns

An ISO8601 format date/time string representation of the POSIX time object. If p is equal to boost::posix_↩ time::not_a_date_time, then the string "0000-00-00T00:00:00Z" (extended) or "00000000000000" (basic) will be returned.

Referenced by util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::AggregateSensorData(), util::xmsg::XEngIce::XEngIce
util::xmsg::XEngSurface::XEngErrorCode::DateTimeAs8601String(), argo::iridium::xfloat::XFloatMailRetrievalService::FetchMailMessages
util::date::DateUtils::GenerateCurrentUtcTimeIso8601String(), util::xmsg::XAlamoBit2::GenerateListingString(), util::xmsg::XArgo::Genera
util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListing
util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XRise::GenerateListingString(),
util::xmsg::XSurfaceData::GenerateListingString(), argo::iridium::xfloat::XFloatMailRetrievalService::operator()(), and
util::position::Position::position_datetime_as_iso8601_string().

Here is the caller graph for this function:



10.57.3.4 GenerateJulianDay()

```
static double util::date::DateUtils::GenerateJulianDay (
            const ptime & dt,
            const ptime & ref_dt)  [inline], [static], [inherited]
```

Convert a date/time to fractional days since a reference date/time.

Parameters

in	*dt*	The date/time to convert.
in	*ref*↩ *_dt*	The reference date/time (day 0).

Exceptions

util::log::MessageException	if either date/time is not valid.

Returns

The number of days (with millisecond resolution) from `ref_dt` to `dt`.

References CATCH_BLOCK, and EXC_THROW.

10.57.3.5 GenerateListingString()

```
const string util::xmsg::XAlamoMissionConfiguration::GenerateListingString (
            int whoi,
            int dive,
            int idx) const  [inline]
```

Generate a listing entry string from this ALAMO mission configuration message object.

Parameters

in	*whoi*	The unique WHOI instrument identifier or serial number.
in	*dive*	The dive or cycle number of this ALAMO mission configuration message object.
in	*idx*	The X-Message index of this ALAMO mission configuration message object.

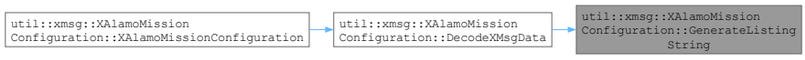
Returns

A listing entry string from this ALAMO mission configuration message object.

References CATCH_BLOCK, and EXC_THROW.

Referenced by DecodeXMsgData().

Here is the caller graph for this function:



10.57.3.6 GetCurrentUtcTime()

```
static ptime util::date::DateUtils::GetCurrentUtcTime ()  [inline], [static], [inherited]
```

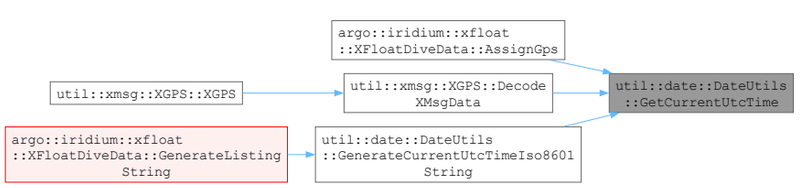
Generate a POSIX time object representing the current date and time (rounded to the nearest second).

Returns

A POSIX time object representing the current date and time (rounded to the nearest second).

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignGps(), util::xmsg::XGPS::DecodeXMsgData(), and util::date::DateUtils::GenerateCurrentUtcTimeIso8601String().

Here is the caller graph for this function:



10.57.4 Member Data Documentation

10.57.4.1 add_

```
uint16_t util::xmsg::XAlamoMissionConfiguration::add_  [private]
```

The ascent state data storage pressure interval, in dBar.

10.57.4.2 adt_

```
uint16_t util::xmsg::XAlamoMissionConfiguration::adt_  [private]
```

The ascent state data storage interval timeout, in seconds.

10.57.4.3 apf_

```
uint16_t util::xmsg::XAlamoMissionConfiguration::apf_  [private]
```

The ascent state pump off duration/interval/timeout, in seconds.

10.57.4.4 apn_

```
uint16_t util::xmsg::XAlamoMissionConfiguration::apn_  [private]
```

The ascent state pump on duration/interval/timeout, in seconds.

10.57.4.5 apt_

```
uint8_t util::xmsg::XAlamoMissionConfiguration::apt_  [private]
```

The auto-transmit mission configuration message flag.

10.57.4.6 arr_

```
uint16_t util::xmsg::XAlamoMissionConfiguration::arr_  [private]
```

The ascent state rise rate, in centimeters per second.

10.57.4.7 ascent_start_time_

```
ptime util::xmsg::XAlamoMissionConfiguration::ascent_start_time_  [private]
```

The start time of the ascent state of the dive cycle.

10.57.4.8 asd_

```
uint16_t util::xmsg::XAlamoMissionConfiguration::asd_  [private]
```

The ascent state surface threshold pressure, in dBar.

10.57.4.9 asi_

```
uint32_t util::xmsg::XAlamoMissionConfiguration::asi_  [private]
```

The ascent state sampling interval/timeout, in milliseconds.

10.57.4.10 asn_

```
uint16_t util::xmsg::XAlamoMissionConfiguration::asn_  [private]
```

The ascent number.

10.57.4.11 btm_

```
uint8_t util::xmsg::XAlamoMissionConfiguration::btm_  [private]
```

The bluetooth mode.

10.57.4.12 cpb_

```
uint16_t util::xmsg::XAlamoMissionConfiguration::cpb_[3]  [private]
```

The CTD sampling period pressure boundaries, in dBar.

Referenced by DecodeXMsgData().

10.57.4.13 cpn_

```
uint16_t util::xmsg::XAlamoMissionConfiguration::cpn_[3]  [private]
```

The CTD sampling period bin sizes, in 0.1 dBar.

Referenced by DecodeXMsgData().

10.57.4.14 cpp_

```
uint16_t util::xmsg::XAlamoMissionConfiguration::cpp_[3]  [private]
```

The CTD sampling period interval, in milliseconds.

Referenced by DecodeXMsgData().

10.57.4.15 cps_

```
uint8_t util::xmsg::XAlamoMissionConfiguration::cps_  [private]
```

The CTD profiling mode.

10.57.4.16 csp_

```
uint32_t util::xmsg::XAlamoMissionConfiguration::csp_  [private]
```

The CTD continuous sampling period, in milliseconds.

10.57.4.17 cst_

```
uint8_t util::xmsg::XAlamoMissionConfiguration::cst_  [private]
```

The CTD sampling type.

10.57.4.18 ctf_

```
uint16_t util::xmsg::XAlamoMissionConfiguration::ctf_  [private]
```

The profile cutoff pressure.

Todo Determine gain for profile cutoff pressure field.

Note

This field was added in firmware version 0x02006900.

10.57.4.19 ctl_

```
uint16_t util::xmsg::XAlamoMissionConfiguration::ctl_  [private]
```

The pressure sensor offset reset lower pressure limit.

Todo Determine gain for pressure sensor offset reset lower pressure limit field.

Note

This field was added in firmware version 0x02006900.

10.57.4.20 ctu_

```
uint16_t util::xmsg::XAlamoMissionConfiguration::ctu_  [private]
```

The pressure sensor offset reset lower pressure limit.

Todo Determine gain for pressure sensor offset reset lower pressure limit field.

Note

This field was added in firmware version 0x02006900.

10.57.4.21 ddd_

```
uint16_t util::xmsg::XAlamoMissionConfiguration::ddd_  [private]
```

The descent state data storage pressure interval, in dBar.

10.57.4.22 ddt_

```
uint16_t util::xmsg::XAlamoMissionConfiguration::ddt_  [private]
```

The descent state data storage interval, in seconds.

10.57.4.23 descend_start_time_

```
ptime util::xmsg::XAlamoMissionConfiguration::descend_start_time_  [private]
```

The start time of the descend state of the dive cycle.

10.57.4.24 dft_

```
uint16_t util::xmsg::XAlamoMissionConfiguration::dft_  [private]
```

The descent state fall timeout, in seconds.

10.57.4.25 dns_

```
uint16_t util::xmsg::XAlamoMissionConfiguration::dns_  [private]
```

The descent number.

10.57.4.26 dsd_

```
uint16_t util::xmsg::XAlamoMissionConfiguration::dsd_  [private]
```

The descent state start pressure, in dBar.

10.57.4.27 dsi_

```
uint32_t util::xmsg::XAlamoMissionConfiguration::dsi_  [private]
```

The descent state sampling interval, in milliseconds.

10.57.4.28 dst_

```
uint16_t util::xmsg::XAlamoMissionConfiguration::dst_  [private]
```

The descent state start timeout, in seconds.

10.57.4.29 dtd_

```
uint16_t util::xmsg::XAlamoMissionConfiguration::dtd_  [private]
```

The target descent/drift/park pressure, in dBar.

10.57.4.30 fw_version_

```
uint32_t util::xmsg::XAlamoMissionConfiguration::fw_version_  [private]
```

The software version identifier.

10.57.4.31 mam_

```
uint8_t util::xmsg::XAlamoMissionConfiguration::mam_  [private]
```

The motion processing unit (MPU) data acquisition operational mode.

10.57.4.32 mat_

```
uint16_t util::xmsg::XAlamoMissionConfiguration::mat_  [private]
```

The motion processing unit (MPU) sampling duration, in seconds.

10.57.4.33 mds_

```
uint16_t util::xmsg::XAlamoMissionConfiguration::mds_  [private]
```

The maximum descent/fall/profile pressure, in dBar.

10.57.4.34 msg_version_

```
uint8_t util::xmsg::XAlamoMissionConfiguration::msg_version_  [private]
```

The message version identifier.

10.57.4.35 msn_

```
int16_t util::xmsg::XAlamoMissionConfiguration::msn_  [private]
```

The mission, dive, or cycle number for this mission configuration.

10.57.4.36 mtd_

```
uint16_t util::xmsg::XAlamoMissionConfiguration::mtd_  [private]
```

The initial trigger depth, in dBar.

10.57.4.37 mtt_

```
uint16_t util::xmsg::XAlamoMissionConfiguration::mtt_  [private]
```

The initial trigger timeout/interval, in milliseconds.

10.57.4.38 park_start_time_

```
ptime util::xmsg::XAlamoMissionConfiguration::park_start_time_  [private]
```

The start time of the park/drift state of the dive cycle.

10.57.4.39 pdt_

```
uint16_t util::xmsg::XAlamoMissionConfiguration::pdt_  [private]
```

The park/drift state data storage interval/timeout, in seconds.

10.57.4.40 pfd_

```
uint16_t util::xmsg::XAlamoMissionConfiguration::pfd_  [private]
```

The park/drift state fall depth (i.e.

profile depth), in dBar.

10.57.4.41 pfm_

```
uint8_t util::xmsg::XAlamoMissionConfiguration::pfm_  [private]
```

The park/drift state fall mode.

10.57.4.42 pns_

```
uint16_t util::xmsg::XAlamoMissionConfiguration::pns_  [private]
```

The park/drift number.

10.57.4.43 psa_

```
uint8_t util::xmsg::XAlamoMissionConfiguration::psa_  [private]
```

The park/drift state sampling mode.

10.57.4.44 psi_

```
uint32_t util::xmsg::XAlamoMissionConfiguration::psi_  [private]
```

The park/drift state sample interval/timeout, in milliseconds.

10.57.4.45 pth_

```
uint32_t util::xmsg::XAlamoMissionConfiguration::pth_  [private]
```

The bit-encoded park/drift state hour timeout trigger(s).

10.57.4.46 pto_

```
uint32_t util::xmsg::XAlamoMissionConfiguration::pto_  [private]
```

The park/drift state timeout, in seconds.

10.57.4.47 sbm_

```
uint8_t util::xmsg::XAlamoMissionConfiguration::sbm_  [private]
```

The surface state beacon mode flag.

10.57.4.48 sbt_

```
uint16_t util::xmsg::XAlamoMissionConfiguration::sbt_  [private]
```

The surface state beacon interval timeout, in seconds.

10.57.4.49 sdc_

```
int16_t util::xmsg::XAlamoMissionConfiguration::sdc_  [private]
```

The surface state sensor depth compensation offset, in dBar.

10.57.4.50 sds_

```
uint16_t util::xmsg::XAlamoMissionConfiguration::sds_  [private]
```

The surface state sampling storage duration/interval/timeout, in seconds.

10.57.4.51 sensor_type_

```
uint8_t util::xmsg::XAlamoMissionConfiguration::sensor_type_  [private]
```

The sensor type identifier.

10.57.4.52 set_

```
uint16_t util::xmsg::XAlamoMissionConfiguration::set_  [private]
```

The surface state Iridium session timeout, in minutes.

10.57.4.53 sfn_

```
uint16_t util::xmsg::XAlamoMissionConfiguration::sfn_  [private]
```

The surface number.

10.57.4.54 sgt_

```
uint16_t util::xmsg::XAlamoMissionConfiguration::sgt_  [private]
```

The surface state GPS timeout, in minutes.

10.57.4.55 ssi_

```
uint32_t util::xmsg::XAlamoMissionConfiguration::ssi_  [private]
```

The surface state sampling interval timeout, in milliseconds.

10.57.4.56 surface_start_time_

```
ptime util::xmsg::XAlamoMissionConfiguration::surface_start_time_  [private]
```

The start time of the surface state of the dive cycle.

10.57.4.57 xas_

```
uint8_t util::xmsg::XAlamoMissionConfiguration::xas_  [private]
```

The profile data encode method.

The documentation for this class was generated from the following file:

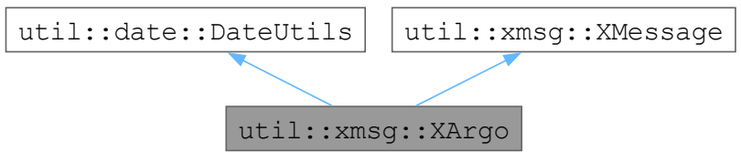
- data/xmsg/XAlamoMissionConfiguration.h

10.58 util::xmsg::XArgo Class Reference

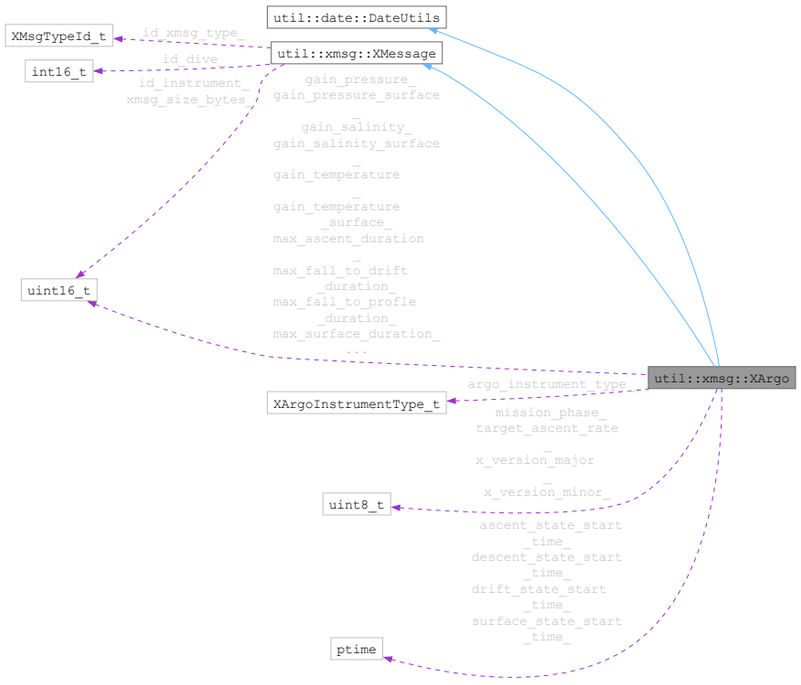
Class representing Argo X message.

`#include <XArgo.h>`

Inheritance diagram for util::xmsg::XArgo:



Collaboration diagram for util::xmsg::XArgo:



Public Member Functions

- XArgo (int16_t dive, uint16_t whoi, const vector< uint8_t > &data)

  The constructor.
- ~XArgo ()

  The destructor.
- XArgoInstrumentType_t argo_instrument_type () const

  Get the Argo instrument type identifier.
- const ptime & ascent_state_start_time () const

  Get the start time of the ascent state of the dive cycle.
- const uint8_t * CheckXMsgData (const vector< uint8_t > &data, int id, int minBytes, bool parseSize=true)

  Check the X-message for the correct ID, minimum size, and message terminator.
- void DecodeXMsgData (const vector< uint8_t > &data)

  Decode the Argo X message.
- const ptime & descent_state_start_time () const

  Get the start time of the descent state of the dive cycle.
- const ptime & drift_state_start_time () const

  Get the start time of the drift/park state of the dive cycle.
- uint16_t gain_pressure () const

Get the pressure sensor gain value, in LSB/dBar.

- uint16_t gain_pressure_surface () const

  Get the pressure sensor gain value for surface data, in LSB/dBar.

- uint16_t gain_salinity () const

  Get the salinity sensor gain value, in LSB/PSU.

- uint16_t gain_salinity_surface () const

  Get the salinity sensor gain value for surface data, in LSB/PSU.

- uint16_t gain_temperature () const

  Get the temperature sensor gain value, in LSB/degC.

- uint16_t gain_temperature_surface () const

  Get the temperature sensor gain value for surface data, in LSB/degC.

- const string GenerateListingString () const

  Generate a listing entry string representation of this Argo data message.

- int16_t id_dive () const

  Get the dive or cycle number of this X-Message.

- uint16_t id_instrument () const

  Get the unique instrument identifier of this X-Message.

- XMsgTypeId_t id_xmsg_type () const

  Get the enumerated X-Message type identifier.

- uint16_t max_ascent_duration () const

  Get the maximum duration of the ascent state of the dive cycle, in minutes.

- uint16_t max_fall_to_drift_duration () const

  Get the maximum duration of the descent-to-drift/park-pressure phase of the dive cycle.

- uint16_t max_fall_to_profle_duration () const

  Get the maximum duration of the descent-to-profile-pressure phase of the dive cycle, in seconds.

- uint16_t max_surface_duration () const

  Get the maximum duration of the surface state of the dive cycle, in seconds.

- uint8_t mission_phase () const

  Get the mission phase.

- uint16_t num_seeks_to_drift_pressure () const

  Get the maximum number of seeks to drift/park pressure.

- uint16_t offset_pressure () const

  Get the pressure sensor offset, in dBar.

- uint16_t offset_pressure_surface () const

  Get the pressure sensor offset for surface data, in dBar.

- uint16_t offset_salinity () const

  Get the salinity sensor offset, in PSU.

- uint16_t offset_salinity_surface () const

  Get the salinity sensor offset for surface data, in PSU.

- uint16_t offset_temperature () const

  Get the temperature sensor offset, in degC.

- uint16_t offset_temperature_surface () const

  Get the temperature sensor offset for surface data, in degC.

- uint16_t seek_to_drift_pressure_interval_duration () const

  Get the seek-to-drift/park-pressure interval duration, in minutes.

- const ptime & surface_state_start_time () const

  Get the start time of the surface state of the dive cycle.

- uint8_t target_ascent_rate () const

  Get the target ascent/profile rate, in cm/sec.

- uint16_t target_drift_duration () const

  Get the target druration of the drift/park phase of the dive cycle, in minutes.

- uint16_t target_drift_pressure () const

  Get the target drift/park pressure, in dBar.

- uint16_t target_profile_pressure () const

  Get the target profile pressure, in dBar.

- float x_version_float () const

  Get the X-Message version as a floating-point value.

- uint8_t x_version_major () const

  Get the X-Message major version indicator.

- uint8_t x_version_minor () const

  Get the X-Message minor version indicator.

- uint16_t xmsg_size_bytes () const

  Get the size of the X-Message, in bytes.

Static Public Member Functions

- static string GenerateCurrentUtcTimeIso8601String ()

  Generate an ISO8601 format date/time string representation of the current time, in UTC.
- static string GenerateIso8601String (const ptime &p, bool extended=true)

  Generate an ISO8601 format date/time string representation of a POSIX time object.
- static double GenerateJulianDay (const ptime &dt, const ptime &ref_dt)

  Convert a date/time to fractional days since a reference date/time.
- static ptime GetCurrentUtcTime ()

  Generate a POSIX time object representing the current date and time (rounded to the nearest second).

Protected Member Functions

- void set_argo_instrument_type (XArgoInstrumentType_t type)

  Set the Argo instrument type identifier.
- void set_ascent_state_start_time (const ptime &time)

  Set the start time of the ascent state of the dive cycle.
- void set_descent_state_start_time (const ptime &time)

  Set the start time of the descent state of the dive cycle.
- void set_drift_state_start_time (const ptime &time)

  Set the start time of the drift state of the dive cycle.
- void set_gain_pressure (uint16_t gain)

  Set the pressure sensor gain, in LSB/dBar.
- void set_gain_pressure_surface (uint16_t gain)

  Set the pressure sensor gain for surface data, in LSB/dBar.
- void set_gain_salinity (uint16_t gain)

  Set the salinity sensor gain, in LSB/PSU.
- void set_gain_salinity_surface (uint16_t gain)

  Set the salinity sensor gain for surface data, in PSU/PSU.
- void set_gain_temperature (uint16_t gain)

  Set the temperature sensor gain, in PSU/degC.
- void set_gain_temperature_surface (uint16_t gain)

  Set the temperature sensor gain for surface data, in PSU/degC.
- void set_id_dive (int16_t id_dive)

  Set the dive or cycle number of this message.
- void set_id_instrument (uint16_t id_instrument)

  Set the unique instrument identifier.
- void set_id_xmsg_type (XMsgTypeId_t id_xmsg_type)

  Set the X-Message type identifier.
- void set_max_ascent_duration (uint16_t duration)

  Set the maximum duration for the ascent state of the dive cycle, in minutes.
- void set_max_fall_to_drift_duration (uint16_t duration)

  Set the maximum duration for the descent-to-drift/park-pressure state of the dive cycle, in minutes.
- void set_max_fall_to_profle_duration (uint16_t duration)

  Set the maximum duration for the descent-to-profile-pressure state of the dive cycle, in minutes.
- void set_max_surface_duration (uint16_t duration)

  Set the maximum duration for the surface state of the dive cycle, in seconds.
- void set_mission_phase (uint8_t phase)

  Set the mission phase number.
- void set_num_seeks_to_drift_pressure (uint16_t seeks)

  Set the maximum number of seeks to drift/park pressure.
- void set_offset_pressure (uint16_t offs)

  Set the pressure sensor offset, in dBar.
- void set_offset_pressure_surface (uint16_t offs)

  Set the pressure sensor offset for surface data, in dBar.
- void set_offset_salinity (uint16_t offs)

  Set the salinity sensor offset, in PSU.
- void set_offset_salinity_surface (uint16_t offs)

  Set the salinity sensor offset for surface data, in PSU.
- void set_offset_temperature (uint16_t offs)

  Set the temperature sensor offset, in degC.
- void set_offset_temperature_surface (uint16_t offs)

Set the temperature sensor offset for surface data, in degC.

- void set_seek_to_drift_pressure_interval_duration (uint16_t duration)

  Set the seek-to-drift/park-pressure interval duration, in minutes.
- void set_surface_state_start_time (const ptime &time)

  Set the start time of the surface state of the dive cycle.
- void set_target_ascent_rate (uint8_t rate)

  Set the target ascent rate, in cm/s.
- void set_target_drift_duration (uint16_t duration)

  Set the target duration of the drift/park phase of the dive cycle, in minutes.
- void set_target_drift_pressure (uint16_t pres)

  Set the target drift/park pressure, in dBar.
- void set_target_profile_pressure (uint16_t pres)

  Set the target profile pressure, in dBar.
- void set_x_version_major (uint8_t vers)

  Set the X-Message major version identifier.
- void set_x_version_minor (uint8_t vers)

  Set the X-Message minor version identifier.
- void set_xmsg_size_bytes (uint16_t xmsg_size_bytes)

  Set the size of the X-Message, in bytes.

Private Attributes

- XArgoInstrumentType_t argo_instrument_type_

  The Argo instrument type identifier.
- ptime ascent_state_start_time_

  The ascend state start time.
- ptime descent_state_start_time_

  The descend state start time.
- ptime drift_state_start_time_

  The park state start time.
- uint16_t gain_pressure_

  The pressure sensitivity gain factor, in LSB/dBar.
- uint16_t gain_pressure_surface_

  The pressure sensitivity gain factor for surface data, in LSB/dBar.
- uint16_t gain_salinity_

  The salinity sensitivity scale factor, in LSB/PSU.
- uint16_t gain_salinity_surface_

  The salinity sensitivity scale factor for surface data, in LSB/PSU.
- uint16_t gain_temperature_

  The temperature sensitivity scale factor, in LSB/degC.
- uint16_t gain_temperature_surface_

  The temperature sensitivity scale factor for surface data, in LSB/degC.
- int16_t id_dive_

  The dive or cycle number of this message.
- uint16_t id_instrument_

  The unique WHOI identifier of this instrument.
- XMsgTypeId_t id_xmsg_type_

  The X-message type identifier.
- uint16_t max_ascent_duration_

  The maximum rise/ascend/profile time, in minutes.
- uint16_t max_fall_to_drift_duration_

  The maximum time to fall from the surface to drift/park depth, in minutes.
- uint16_t max_fall_to_profle_duration_

  The maximum time to fall from the drift/park depth to profile depth, in seconds.
- uint16_t max_surface_duration_

  The maximum duration at the surface, in seconds.
- uint8_t mission_phase_

  The mission phase number of this dive.
- uint16_t num_seeks_to_drift_pressure_

  The number of seeks to drift depth.
- uint16_t offset_pressure_

  The pressure offset, in dBar.

- uint16_t offset_pressure_surface_

  The pressure offset for surface data, in dBar.
- uint16_t offset_salinity_

  The salinity offset, in PSU.
- uint16_t offset_salinity_surface_

  The salinity offset for surface data, in PSU.
- uint16_t offset_temperature_

  The temperature offset, in degC.
- uint16_t offset_temperature_surface_

  The temperature offset for surface data, in degC.
- uint16_t seek_to_drift_pressure_interval_duration_

  The interval duration between subsequent seeks to drift/park depth, in minutes.
- ptime surface_state_start_time_

  The surface state start time.
- uint8_t target_ascent_rate_

  The target ascent rate, in cm/s.
- uint16_t target_drift_duration_

  The target duration of the drift/park phase of the dive cycle, in minutes.
- uint16_t target_drift_pressure_

  The target parking or drift depth (DTD), in dBar.
- uint16_t target_profile_pressure_

  The target profile depth (PFD), in dBar.
- uint8_t x_version_major_

  The X message format major version number.
- uint8_t x_version_minor_

  The X Message format minor version number.
- uint16_t xmsg_size_bytes_

  The size of the X message, in bytes.

10.58.1 Detailed Description

Class representing Argo X message.

This message contains information about the characteristics of the dive cycle.

Todo Add support for the Solo-II-type Argo data message.

Add support for the Deep-Argo-type Argo data message.

10.58.2 Constructor & Destructor Documentation

10.58.2.1 XArgo()

```
util::xmsg::XArgo::XArgo (
            int16_t dive,
            uint16_t whoi,
            const vector< uint8_t > & data)  [inline]
```

The constructor.

Parameters

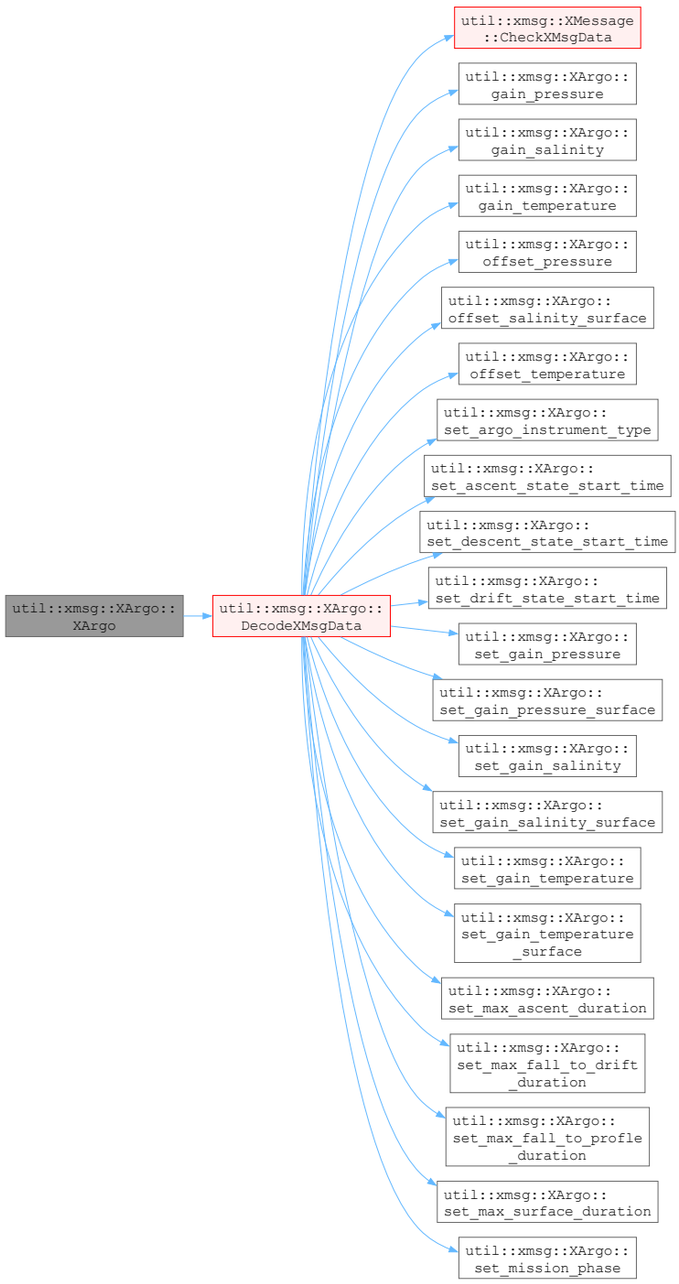
in	*dive*	The dive or cycle number of this message.
in	*whoi*	The unique WHOI identifier.
in	*data*	The binary Argo data X-message data to decode.

Exceptions

util::log::MessageException	if an decode error occurs.

References DecodeXMsgData().

Here is the call graph for this function:



10.58.2.2 ~XArgo()

```
util::xmsg::XArgo::~XArgo ()  [inline]
```

The destructor.

10.58.3 Member Function Documentation

10.58.3.1 argo_instrument_type()

```
XArgoInstrumentType_t util::xmsg::XArgo::argo_instrument_type () const  [inline]
```

Get the Argo instrument type identifier.

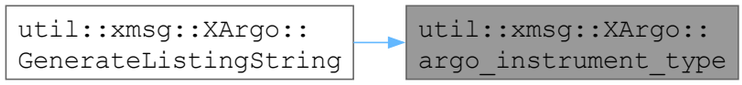
Returns

The Argo instrument type identifier.

References argo_instrument_type_.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.58.3.2 ascent_state_start_time()

```
const ptime & util::xmsg::XArgo::ascent_state_start_time () const  [inline]
```

Get the start time of the ascent state of the dive cycle.

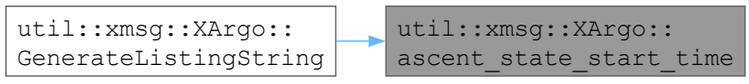
Returns

The start time of the ascent state of the dive cycle.

References ascent_state_start_time_.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.58.3.3 CheckXMsgData()

```
const uint8_t * util::xmsg::XMessage::CheckXMsgData (
            const vector< uint8_t > & data,
            int id,
            int minBytes,
            bool parseSize = true)  [inline], [inherited]
```

Check the X-message for the correct ID, minimum size, and message terminator.

Parameters

in	*data*	The X-message binary data to check.
in	*id*	The expected ID of the X-message. If this field is less that 0, then the ID check will be skipped.
in	*minBytes*	The minimum number of bytes expected in a valid message.
in	*parseSize*	If TRUE, then the message size field will be parsed from the X-Message. Otherwise, the size field will not be parsed.

Exceptions

util::log::MessageException	if • the message does not contain the expected ID, • the message does not contain the minimum number of expected bytes, • the message does not contain a terminator (';') in the correct location, or, • any other error occurs.

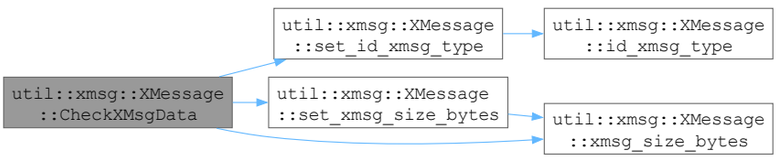
Returns

A pointer to the underlying data buffer of data.

References CATCH_BLOCK, EXC_THROW, lwrn, util::xmsg::XMessage::set_id_xmsg_type(), util::xmsg::XMessage::set_xmsg_size_byte
u16, and util::xmsg::XMessage::xmsg_size_bytes().

Referenced by util::xmsg::XAcousticPosition::DecodeXMsgData(), DecodeXMsgData(), util::xmsg::XDriftPressure::DecodeXMsgData(),
util::xmsg::XDriftSalinity::DecodeXMsgData(), util::xmsg::XDriftTemperature::DecodeXMsgData(), util::xmsg::XEngAbort::DecodeXMsgDa
util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngBIT::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(),
util::xmsg::XEngDiagnostic::DecodeXMsgData(), util::xmsg::XEngDrift::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(),
util::xmsg::XEngStandard::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XGPS::DecodeXMsgData(),
util::xmsg::XHiResPressure::DecodeXMsgData(), util::xmsg::XHiResSalinity::DecodeXMsgData(), util::xmsg::XHiResTemperature::Decod
util::xmsg::XMissionCmdEcho::DecodeXMsgData(), util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XProfilePressure:
util::xmsg::XProfileSalinity::DecodeXMsgData(), util::xmsg::XProfileTemperature::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgD
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the call graph for this function:



10.58.3.4 DecodeXMsgData()

```
void util::xmsg::XArgo::DecodeXMsgData (
            const vector< uint8_t > & data)  [inline], [virtual]
```

Decode the Argo X message.

Parameters

in	*data*	The data to decode.

Exceptions

util::log::MessageException	if an error occurs while decoding the message.

Todo Implement the functionality to decode the argo data message for Solo-II floats.

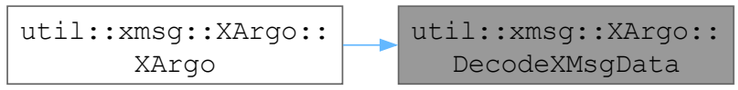
Todo Implement the functionality to decode the argo data message for Deep Argo floats.

Implements util::xmsg::XMessage.

References CATCH_BLOCK, util::xmsg::XMessage::CheckXMsgData(), gain_pressure(), gain_salinity(), gain_temperature(), offset_pressure(), offset_salinity_surface(), offset_temperature(), s2000, set_argo_instrument_type(), set_ascent_state_start_time(), set_descent_state_start_time(), set_drift_state_start_time(), set_gain_pressure(), set_gain_pressure_surface(), set_gain_salinity(), set_gain_salinity_surface(), set_gain_temperature(), set_gain_temperature_surface(), set_max_ascent_duration(), set_max_fall_to_drift_duration(), set_max_fall_to_profle_duration(), set_max_surface_duration(), set_mission_phase(), set_num_seeks_to_drift_pressure(), set_offset_pressure(), set_offset_pressure_surface(), set_offset_salinity(), set_offset_salinity_surface(), set_offset_temperature(), set_offset_temperature_surface(), set_seek_to_drift_pressure_interval_duration() set_surface_state_start_time(), set_target_ascent_rate(), set_target_drift_duration(), set_target_drift_pressure(), set_target_profile_pressure(), set_x_version_major(), set_x_version_minor(), u08, u16, x_version_major(), x_version_minor(), XARGO_INSRUMENT_TYPE_ALAMO, and util::xmsg::XMessage::xmsg_size_bytes().

Referenced by XArgo().

Here is the caller graph for this function:



10.58.3.5 descent_state_start_time()

```
const ptime & util::xmsg::XArgo::descent_state_start_time () const  [inline]
```

Get the start time of the descent state of the dive cycle.

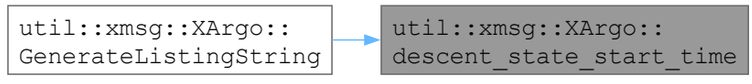
Returns

The start time of the descent state of the dive cycle.

References descent_state_start_time_.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.58.3.6 drift_state_start_time()

```
const ptime & util::xmsg::XArgo::drift_state_start_time () const  [inline]
```

Get the start time of the drift/park state of the dive cycle.

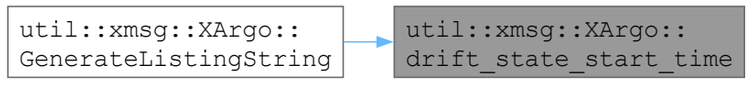
Returns

The start time of the drift/park state of the dive cycle.

References drift_state_start_time_.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.58.3.7 gain_pressure()

```
uint16_t util::xmsg::XArgo::gain_pressure () const  [inline]
```

Get the pressure sensor gain value, in LSB/dBar.

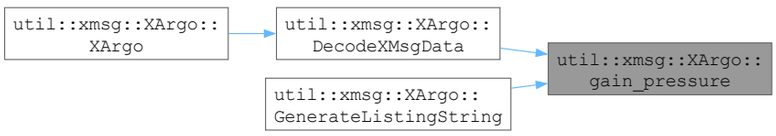
Returns

The pressure sensor gain value, in LSB/dBar.

References gain_pressure_.

Referenced by DecodeXMsgData(), and GenerateListingString().

Here is the caller graph for this function:



10.58.3.8 gain_pressure_surface()

```
uint16_t util::xmsg::XArgo::gain_pressure_surface () const  [inline]
```

Get the pressure sensor gain value for surface data, in LSB/dBar.

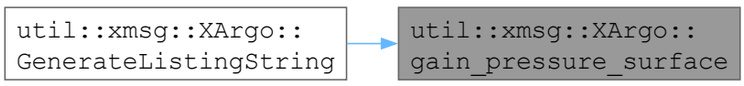
Returns

The pressure sensor gain value, in LSB/dBar.

References gain_pressure_surface_.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.58.3.9 gain_salinity()

```
uint16_t util::xmsg::XArgo::gain_salinity () const  [inline]
```

Get the salinity sensor gain value, in LSB/PSU.

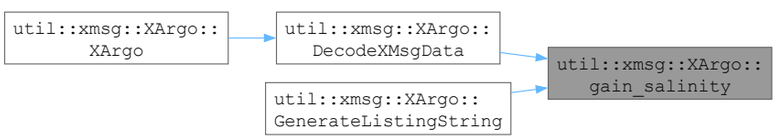
Returns

The salinity sensor gain value, in LSB/PSU.

References gain_salinity_.

Referenced by DecodeXMsgData(), and GenerateListingString().

Here is the caller graph for this function:



10.58.3.10 gain_salinity_surface()

```
uint16_t util::xmsg::XArgo::gain_salinity_surface () const  [inline]
```

Get the salinity sensor gain value for surface data, in LSB/PSU.

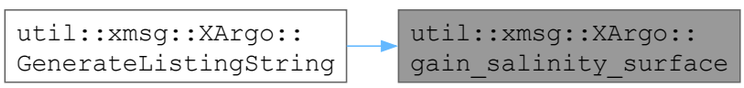
Returns

The salinity sensor gain value, in LSB/PSU.

References gain_salinity_surface_.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.58.3.11 gain_temperature()

```
uint16_t util::xmsg::XArgo::gain_temperature () const  [inline]
```

Get the temperature sensor gain value, in LSB/degC.

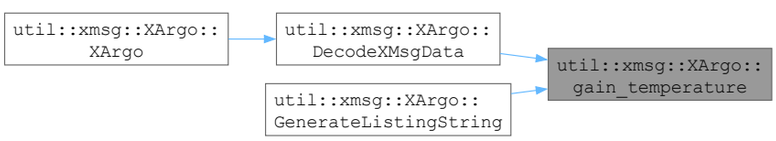
Returns

The temperature sensor gain value, in LSB/degC.

References gain_temperature_.

Referenced by DecodeXMsgData(), and GenerateListingString().

Here is the caller graph for this function:



10.58.3.12 gain_temperature_surface()

```
uint16_t util::xmsg::XArgo::gain_temperature_surface () const  [inline]
```

Get the temperature sensor gain value for surface data, in LSB/degC.

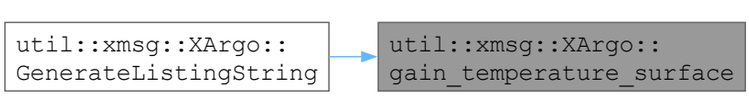
Returns

The temperature sensor gain value, in LSB/degC.

References gain_temperature_surface_.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.58.3.13 GenerateCurrentUtcTimeIso8601String()

```
static string util::date::DateUtils::GenerateCurrentUtcTimeIso8601String ()  [inline], [static],
[inherited]
```

Generate an ISO8601 format date/time string representation of the current time, in UTC.

Note

All date/time objects use the UTC time-zone.

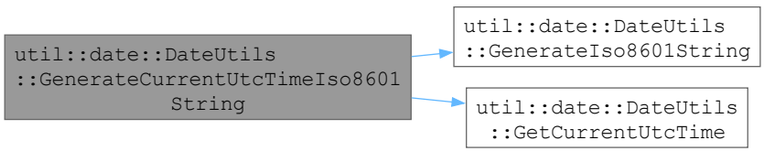
Returns

An ISO8601 format date/time string representation of the current time.

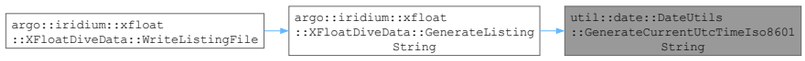
References util::date::DateUtils::GenerateIso8601String(), and util::date::DateUtils::GetCurrentUtcTime().

Referenced by argo::iridium::xfloat::XFloatDiveData::GenerateListingString().

Here is the call graph for this function:



Here is the caller graph for this function:



10.58.3.14 GenerateIso8601String()

```
static string util::date::DateUtils::GenerateIso8601String (
            const ptime & p,
            bool extended = true)  [inline], [static], [inherited]
```

Generate an ISO8601 format date/time string representation of a POSIX time object.

Note

All date/time objects use the UTC time-zone.

Parameters

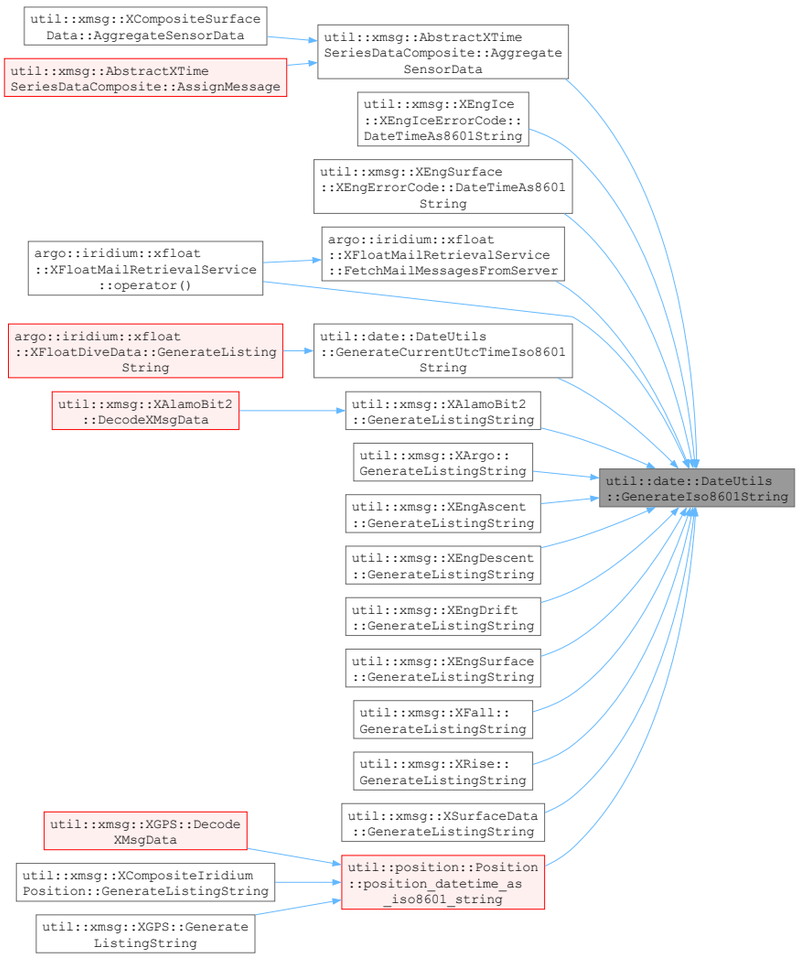
in	*p*	POSIX time from which to generate string.
in	*extended*	If TRUE, use the ISO 8601 extended format (YYYY-MM-DDThh:mm:ssZ); otherwise, use the basic format (YYYYMMDDThhmmss).

Returns

An ISO8601 format date/time string representation of the POSIX time object. If p is equal to boost::posix_↩ time::not_a_date_time, then the string "0000-00-00T00:00:00Z" (extended) or "00000000000000" (basic) will be returned.

Referenced by util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::AggregateSensorData(), util::xmsg::XEngIce::XEngIce
util::xmsg::XEngSurface::XEngErrorCode::DateTimeAs8601String(), argo::iridium::xfloat::XFloatMailRetrievalService::FetchMailMessages
util::date::DateUtils::GenerateCurrentUtcTimeIso8601String(), util::xmsg::XAlamoBit2::GenerateListingString(), GenerateListingString(),
util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListing
util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XRise::GenerateListingString(),
util::xmsg::XSurfaceData::GenerateListingString(), argo::iridium::xfloat::XFloatMailRetrievalService::operator()(), and
util::position::Position::position_datetime_as_iso8601_string().

Here is the caller graph for this function:



10.58.3.15 GenerateJulianDay()

```
static double util::date::DateUtils::GenerateJulianDay (
            const ptime & dt,
            const ptime & ref_dt)  [inline], [static], [inherited]
```

Convert a date/time to fractional days since a reference date/time.

Parameters

in	*dt*	The date/time to convert.
in	*ref*↩ *_dt*	The reference date/time (day 0).

Exceptions

util::log::MessageException	if either date/time is not valid.

Returns

The number of days (with millisecond resolution) from `ref_dt` to `dt`.

References CATCH_BLOCK, and EXC_THROW.

10.58.3.16 GenerateListingString()

```
const string util::xmsg::XArgo::GenerateListingString () const  [inline]
```

Generate a listing entry string representation of this Argo data message.

Returns

A listing entry string representation of this Argo data message.

Todo Implement the functionality for generating a listing string for a Solo-II-type Argo data message.

Todo Implement the functionality for generating a listing string for a Deep-Argo-type Argo data message.

References argo_instrument_type(), ascent_state_start_time(), CATCH_BLOCK, descent_state_start_time(), drift_state_start_time(), gain_pressure(), gain_pressure_surface(), gain_salinity(), gain_salinity_surface(), gain_temperature(), gain_temperature_surface(), util::date::DateUtils::GenerateIso8601String(), util::xmsg::XMessage::id_dive(), util::xmsg::XMessage::id_ins max_ascent_duration(), max_fall_to_drift_duration(), max_fall_to_profle_duration(), max_surface_duration(), mission_phase(), num_seeks_to_drift_pressure(), offset_pressure(), offset_pressure_surface(), offset_salinity(), offset_salinity_surface(), offset_temperature(), offset_temperature_surface(), seek_to_drift_pressure_interval_duration(), surface_state_start_time(), target_ascent_rate(), target_drift_duration(), target_drift_pressure(), target_profile_pressure(), x_version_float(), XARGO_INSRUMENT_TYPE_ALAMO, XARGO_INSRUMENT_TYPE_DEEP_ARGO, XARGO_INSRUMENT_TYPE_SOLO2, and XARGO_INSRUMENT_TYPE_UNKNOWN.

10.58.3.17 GetCurrentUtcTime()

```
static ptime util::date::DateUtils::GetCurrentUtcTime ()  [inline], [static], [inherited]
```

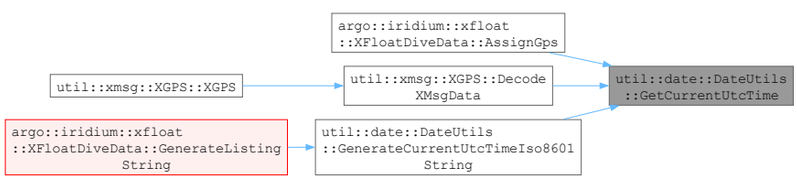
Generate a POSIX time object representing the current date and time (rounded to the nearest second).

Returns

A POSIX time object representing the current date and time (rounded to the nearest second).

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignGps(), util::xmsg::XGPS::DecodeXMsgData(), and util::date::DateUtils::GenerateCurrentUtcTimeIso8601String().

Here is the caller graph for this function:



10.58.3.18 id_dive()

```
int16_t util::xmsg::XMessage::id_dive () const  [inline], [inherited]
```

Get the dive or cycle number of this X-Message.

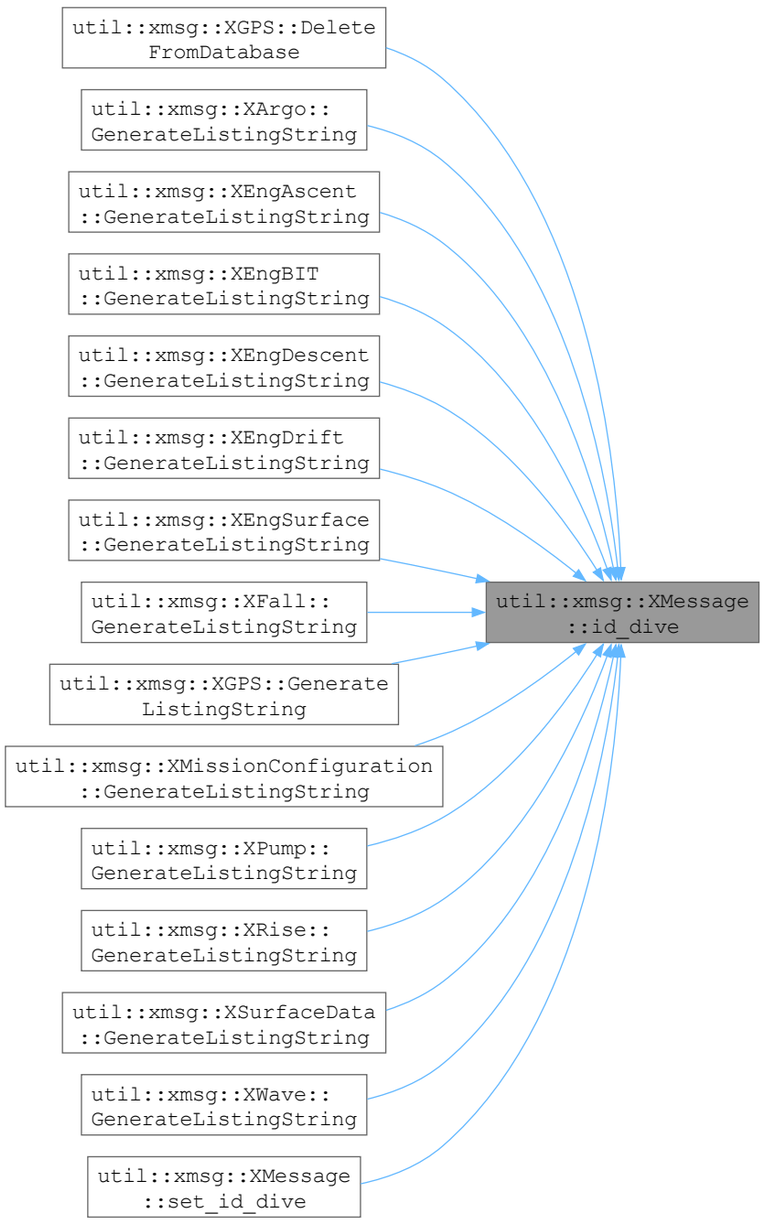
Returns

The dive or cycle number of this X-Message.

References util::xmsg::XMessage::id_dive_.

Referenced by util::xmsg::XGPS::DeleteFromDatabase(), GenerateListingString(), util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngBIT::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingStr util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing util::xmsg::XSurfaceData::GenerateListingString(), util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_dive().

Here is the caller graph for this function:



10.58.3.19 id_instrument()

```
uint16_t util::xmsg::XMessage::id_instrument () const  [inline], [inherited]
```

Get the unique instrument identifier of this X-Message.

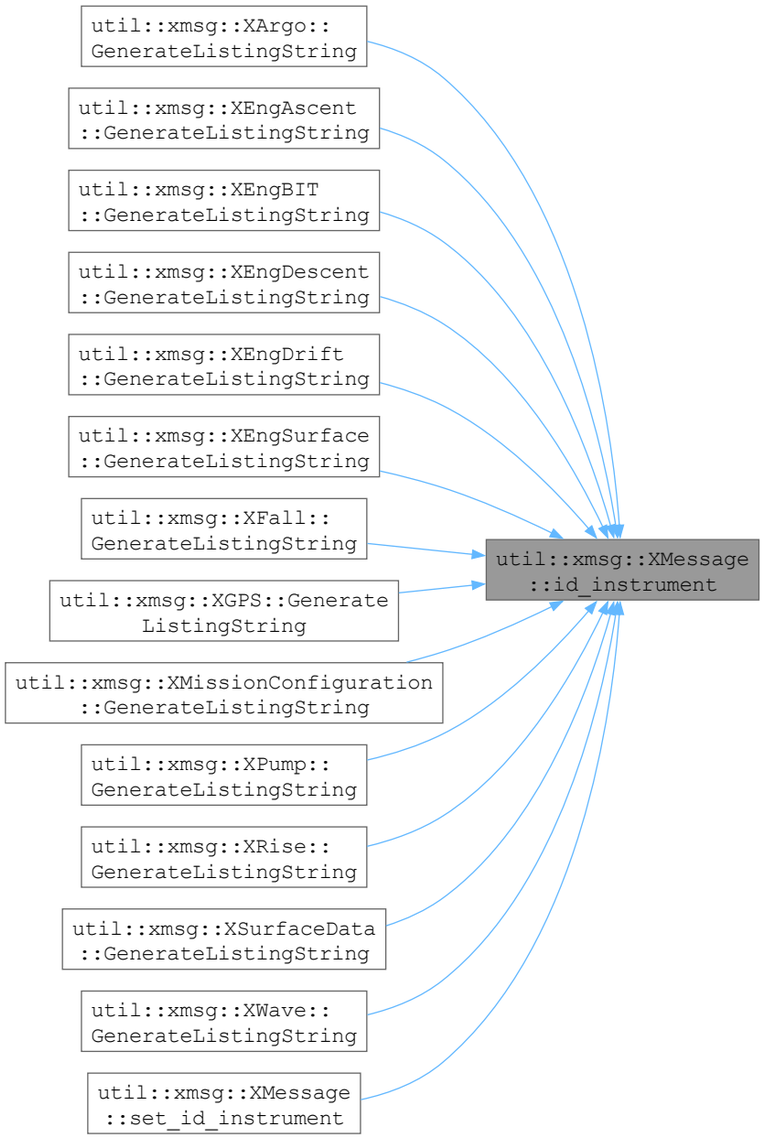
Returns

The unique instrument identifier of this X-Message.

References util::xmsg::XMessage::id_instrument_.

Referenced by GenerateListingString(), util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngBIT::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), util::xmsg::XEngSurface::GenerateListin util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingS util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(), util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_instrument().

Here is the caller graph for this function:



10.58.3.20 id_xmsg_type()

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type () const  [inline], [inherited]
```

Get the enumerated X-Message type identifier.

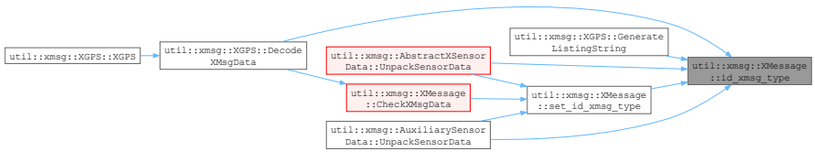
Returns

The enumerated X-Message type identifier.

References util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMessage::set_id_xmsg_typ
util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the caller graph for this function:



10.58.3.21 max_ascent_duration()

```
uint16_t util::xmsg::XArgo::max_ascent_duration () const  [inline]
```

Get the maximum duration of the ascent state of the dive cycle, in minutes.

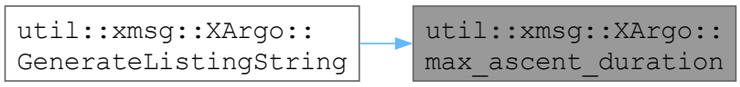
Returns

The maximum duration of the ascent state of the dive cycle, in minutes.

References max_ascent_duration_.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.58.3.22 max_fall_to_drift_duration()

```
uint16_t util::xmsg::XArgo::max_fall_to_drift_duration () const  [inline]
```

Get the maximum duration of the descent-to-drift/park-pressure phase of the dive cycle.

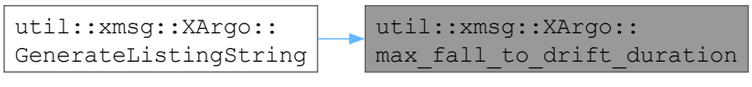
Returns

The maximum duration of the descent-to-drift/park-pressure phase of the dive cycle.

References max_fall_to_drift_duration_.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.58.3.23 max_fall_to_profle_duration()

```
uint16_t util::xmsg::XArgo::max_fall_to_profle_duration () const  [inline]
```

Get the maximum duration of the descent-to-profile-pressure phase of the dive cycle, in seconds.

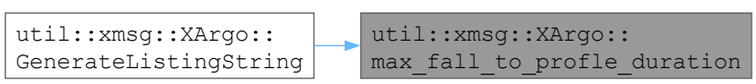
Returns

The maximum duration of the descent-to-profile-pressure phase of the dive cycle, in seconds.

References max_fall_to_profle_duration_.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.58.3.24 max_surface_duration()

```
uint16_t util::xmsg::XArgo::max_surface_duration () const  [inline]
```

Get the maximum duration of the surface state of the dive cycle, in seconds.

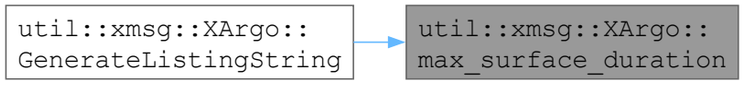
Returns

The maximum duration of the surface state of the dive cycle, in seconds.

References max_surface_duration_.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.58.3.25 mission_phase()

```
uint8_t util::xmsg::XArgo::mission_phase () const  [inline]
```

Get the mission phase.

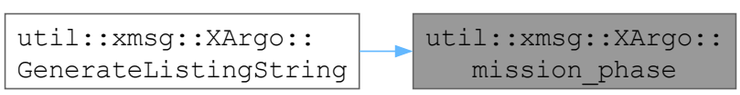
Returns

The mission phase.

References mission_phase_.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.58.3.26 num_seeks_to_drift_pressure()

```
uint16_t util::xmsg::XArgo::num_seeks_to_drift_pressure () const  [inline]
```

Get the maximum number of seeks to drift/park pressure.

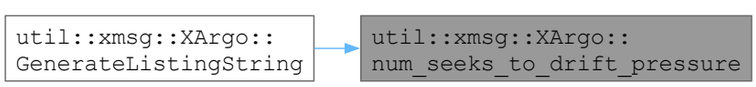
Returns

The maximum number of seeks to drift/park pressure.

References num_seeks_to_drift_pressure_.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.58.3.27 offset_pressure()

```
uint16_t util::xmsg::XArgo::offset_pressure () const  [inline]
```

Get the pressure sensor offset, in dBar.

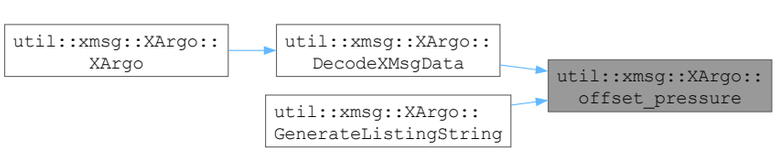
Returns

The pressure sensor offset, in dBar.

References offset_pressure_.

Referenced by DecodeXMsgData(), and GenerateListingString().

Here is the caller graph for this function:



10.58.3.28 offset_pressure_surface()

```
uint16_t util::xmsg::XArgo::offset_pressure_surface () const  [inline]
```

Get the pressure sensor offset for surface data, in dBar.

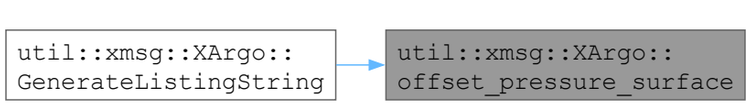
Returns

The pressure sensor offset, in dBar.

References offset_pressure_surface_.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.58.3.29 offset_salinity()

```
uint16_t util::xmsg::XArgo::offset_salinity () const  [inline]
```

Get the salinity sensor offset, in PSU.

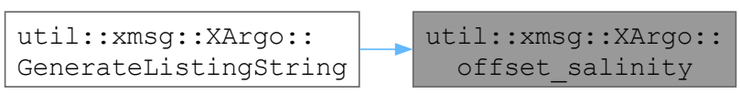
Returns

The salinity sensor offset, in PSU.

References offset_salinity_.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.58.3.30 offset_salinity_surface()

```
uint16_t util::xmsg::XArgo::offset_salinity_surface () const  [inline]
```

Get the salinity sensor offset for surface data, in PSU.

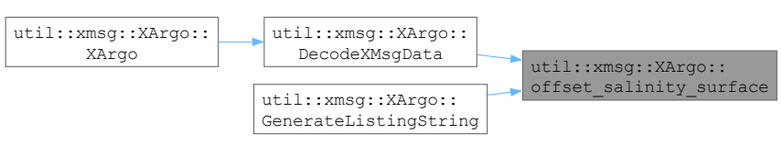
Returns

The salinity sensor offset, in PSU.

References offset_salinity_surface_.

Referenced by DecodeXMsgData(), and GenerateListingString().

Here is the caller graph for this function:



10.58.3.31 offset_temperature()

```
uint16_t util::xmsg::XArgo::offset_temperature () const  [inline]
```

Get the temperature sensor offset, in degC.

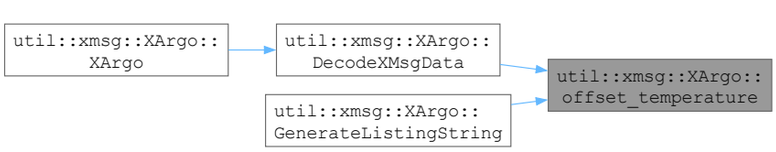
Returns

The temperature sensor offset, in degC.

References offset_temperature_.

Referenced by DecodeXMsgData(), and GenerateListingString().

Here is the caller graph for this function:



10.58.3.32 offset_temperature_surface()

```
uint16_t util::xmsg::XArgo::offset_temperature_surface () const  [inline]
```

Get the temperature sensor offset for surface data, in degC.

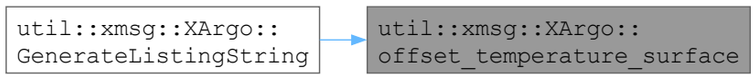
Returns

The temperature sensor offset, in degC.

References offset_temperature_surface_.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.58.3.33 seek_to_drift_pressure_interval_duration()

```
uint16_t util::xmsg::XArgo::seek_to_drift_pressure_interval_duration () const  [inline]
```

Get the seek-to-drift/park-pressure interval duration, in minutes.

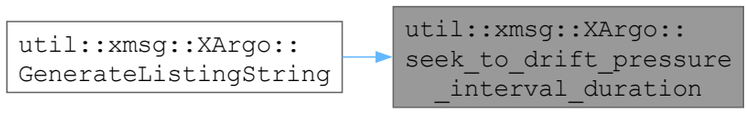
Returns

The seek-to-drift/park-pressure interval duration, in minutes.

References seek_to_drift_pressure_interval_duration_.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.58.3.34 set_argo_instrument_type()

```
void util::xmsg::XArgo::set_argo_instrument_type (
            XArgoInstrumentType_t type)  [inline], [protected]
```

Set the Argo instrument type identifier.

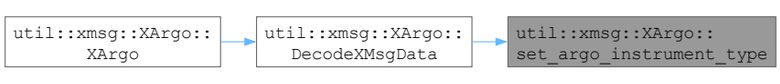
Parameters

in	*type*	The Argo instrument type identifier.

References argo_instrument_type_.

Referenced by DecodeXMsgData().

Here is the caller graph for this function:



10.58.3.35 set_ascent_state_start_time()

```
void util::xmsg::XArgo::set_ascent_state_start_time (
            const ptime & time)  [inline], [protected]
```

Set the start time of the ascent state of the dive cycle.

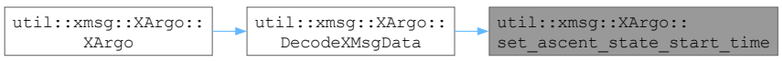
Parameters

in	*time*	The start time.

References ascent_state_start_time_.

Referenced by DecodeXMsgData().

Here is the caller graph for this function:



10.58.3.36 set_descent_state_start_time()

```
void util::xmsg::XArgo::set_descent_state_start_time (
            const ptime & time)  [inline], [protected]
```

Set the start time of the descent state of the dive cycle.

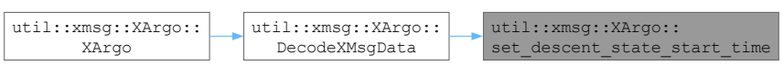
Parameters

in	*time*	The start time.

References descent_state_start_time_.

Referenced by DecodeXMsgData().

Here is the caller graph for this function:



10.58.3.37 set_drift_state_start_time()

```
void util::xmsg::XArgo::set_drift_state_start_time (
            const ptime & time)  [inline], [protected]
```

Set the start time of the drift state of the dive cycle.

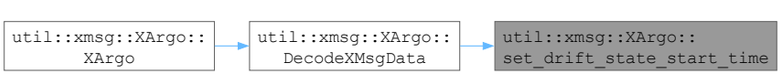
Parameters

in	*time*	The start time.

References drift_state_start_time_.

Referenced by DecodeXMsgData().

Here is the caller graph for this function:



10.58.3.38 set_gain_pressure()

```
void util::xmsg::XArgo::set_gain_pressure (
            uint16_t gain)  [inline], [protected]
```

Set the pressure sensor gain, in LSB/dBar.

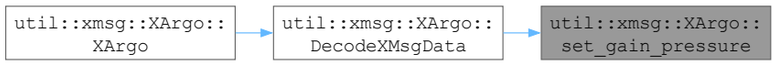
Parameters

in	*gain*	The sensor gain value.

References gain_pressure_.

Referenced by DecodeXMsgData().

Here is the caller graph for this function:



10.58.3.39 set_gain_pressure_surface()

```
void util::xmsg::XArgo::set_gain_pressure_surface (
            uint16_t gain)  [inline], [protected]
```

Set the pressure sensor gain for surface data, in LSB/dBar.

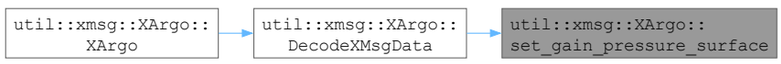
Parameters

in	*gain*	The sensor gain value.

References gain_pressure_surface_.

Referenced by DecodeXMsgData().

Here is the caller graph for this function:



10.58.3.40 set_gain_salinity()

```
void util::xmsg::XArgo::set_gain_salinity (
            uint16_t gain)  [inline], [protected]
```

Set the salinity sensor gain, in LSB/PSU.

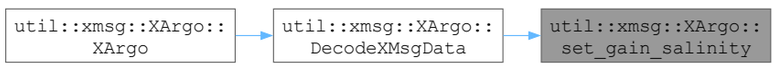
Parameters

in	*gain*	The sensor gain value.

References gain_salinity_.

Referenced by DecodeXMsgData().

Here is the caller graph for this function:



10.58.3.41 set_gain_salinity_surface()

```
void util::xmsg::XArgo::set_gain_salinity_surface (
            uint16_t gain)  [inline], [protected]
```

Set the salinity sensor gain for surface data, in PSU/PSU.

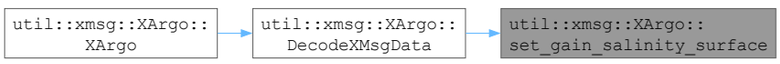
Parameters

in	*gain*	The sensor gain value.

References gain_salinity_surface_.

Referenced by DecodeXMsgData().

Here is the caller graph for this function:



10.58.3.42 set_gain_temperature()

```
void util::xmsg::XArgo::set_gain_temperature (
            uint16_t gain)  [inline], [protected]
```

Set the temperature sensor gain, in PSU/degC.

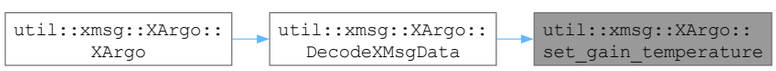
Parameters

in	*gain*	The sensor gain value.

References gain_temperature_.

Referenced by DecodeXMsgData().

Here is the caller graph for this function:



10.58.3.43 set_gain_temperature_surface()

```
void util::xmsg::XArgo::set_gain_temperature_surface (
            uint16_t gain)  [inline], [protected]
```

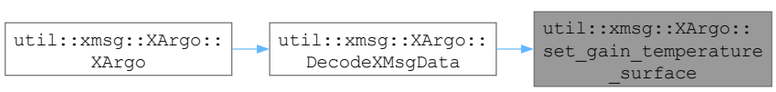
Set the temperature sensor gain for surface data, in PSU/degC.

Parameters

in	*gain*	The sensor gain value.

Referenced by DecodeXMsgData().

Here is the caller graph for this function:



10.58.3.44 set_id_dive()

```
void util::xmsg::XMessage::set_id_dive (
            int16_t id_dive)  [inline], [protected], [inherited]
```

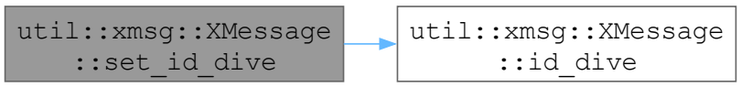
Set the dive or cycle number of this message.

Parameters

in	*id_dive*	The dive or cycle number of this message to set.

References util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::id_dive_.

Here is the call graph for this function:



10.58.3.45 set_id_instrument()

```
void util::xmsg::XMessage::set_id_instrument (
            uint16_t id_instrument)  [inline], [protected], [inherited]
```

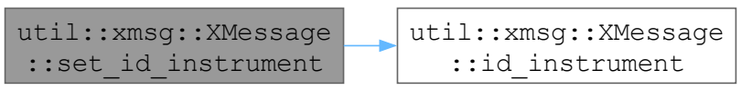
Set the unique instrument identifier.

Parameters

in	*id_instrument*	The unique instrument identifier to set.

References util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::id_instrument_.

Here is the call graph for this function:



10.58.3.46 set_id_xmsg_type()

```
void util::xmsg::XMessage::set_id_xmsg_type (
            XMsgTypeId_t id_xmsg_type)  [inline], [protected], [inherited]
```

Set the X-Message type identifier.

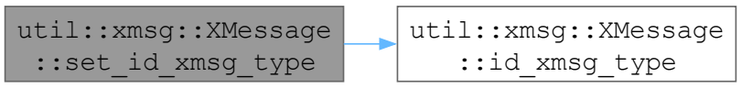
Parameters

in	*id_xmsg_type*	The X-Message type identifier to set.

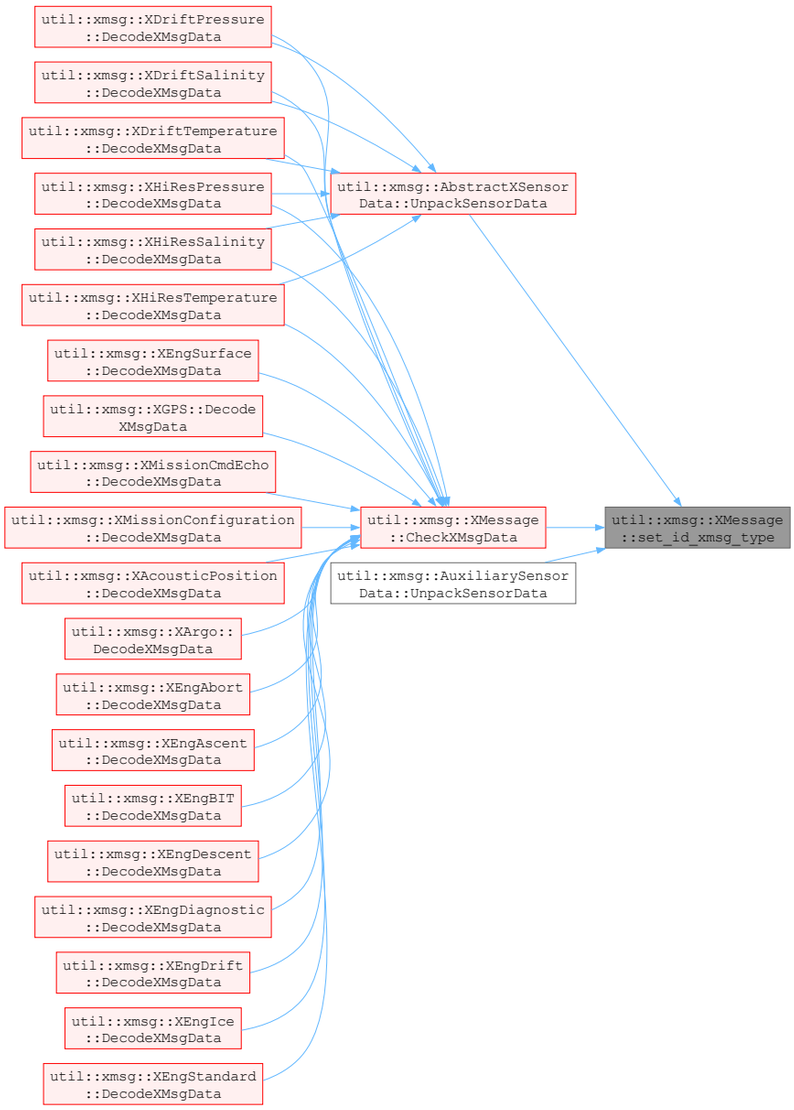
References util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.58.3.47 set_max_ascent_duration()

```
void util::xmsg::XArgo::set_max_ascent_duration (
            uint16_t duration)  [inline], [protected]
```

Set the maximum duration for the ascent state of the dive cycle, in minutes.

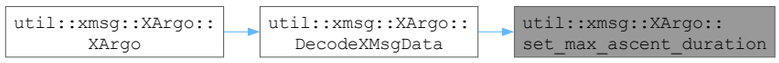
Parameters

in	*duration*	The maximum duration.

References max_ascent_duration_.

Referenced by DecodeXMsgData().

Here is the caller graph for this function:



10.58.3.48 set_max_fall_to_drift_duration()

```
void util::xmsg::XArgo::set_max_fall_to_drift_duration (
            uint16_t duration)  [inline], [protected]
```

Set the maximum duration for the descent-to-drift/park-pressure state of the dive cycle, in minutes.

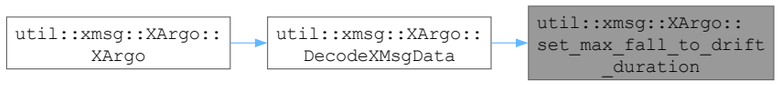
Parameters

in	*duration*	The maximum duration.

References max_fall_to_drift_duration_.

Referenced by DecodeXMsgData().

Here is the caller graph for this function:



10.58.3.49 set_max_fall_to_profle_duration()

```
void util::xmsg::XArgo::set_max_fall_to_profle_duration (
            uint16_t duration)  [inline], [protected]
```

Set the maximum duration for the descent-to-profile-pressure state of the dive cycle, in minutes.

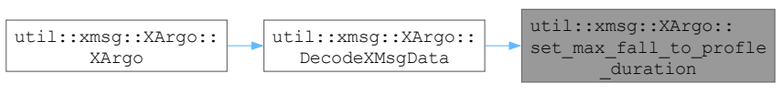
Parameters

in	*duration*	The maximum duration.

References max_fall_to_profle_duration_.

Referenced by DecodeXMsgData().

Here is the caller graph for this function:



10.58.3.50 set_max_surface_duration()

```
void util::xmsg::XArgo::set_max_surface_duration (
            uint16_t duration)  [inline], [protected]
```

Set the maximum duration for the surface state of the dive cycle, in seconds.

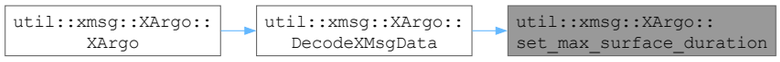
Parameters

in	*duration*	The maximum duration.

References max_surface_duration_.

Referenced by DecodeXMsgData().

Here is the caller graph for this function:



10.58.3.51 set_mission_phase()

```
void util::xmsg::XArgo::set_mission_phase (
            uint8_t phase)  [inline], [protected]
```

Set the mission phase number.

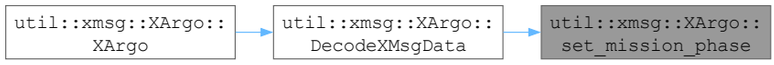
Parameters

in	*phase*	The mission phase number.

References mission_phase_.

Referenced by DecodeXMsgData().

Here is the caller graph for this function:



10.58.3.52 set_num_seeks_to_drift_pressure()

```
void util::xmsg::XArgo::set_num_seeks_to_drift_pressure (
            uint16_t seeks)  [inline], [protected]
```

Set the maximum number of seeks to drift/park pressure.

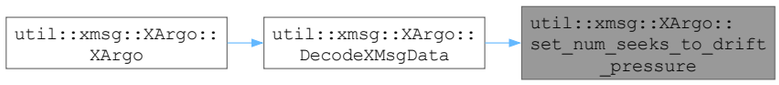
Parameters

in	*seeks*	The number of seeks.

References num_seeks_to_drift_pressure_.

Referenced by DecodeXMsgData().

Here is the caller graph for this function:



10.58.3.53 set_offset_pressure()

```
void util::xmsg::XArgo::set_offset_pressure (
            uint16_t offs)  [inline], [protected]
```

Set the pressure sensor offset, in dBar.

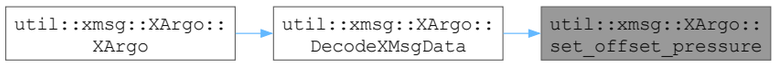
Parameters

in	*offs*	The sensor offset.

References offset_pressure_.

Referenced by DecodeXMsgData().

Here is the caller graph for this function:



10.58.3.54 set_offset_pressure_surface()

```
void util::xmsg::XArgo::set_offset_pressure_surface (
            uint16_t offs)  [inline], [protected]
```

Set the pressure sensor offset for surface data, in dBar.

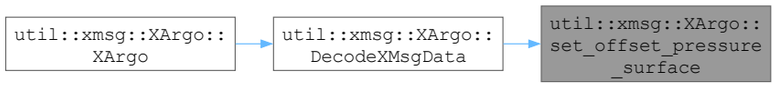
Parameters

in	*offs*	The sensor offset.

References offset_pressure_surface_.

Referenced by DecodeXMsgData().

Here is the caller graph for this function:



10.58.3.55 set_offset_salinity()

```
void util::xmsg::XArgo::set_offset_salinity (
            uint16_t offs)  [inline], [protected]
```

Set the salinity sensor offset, in PSU.

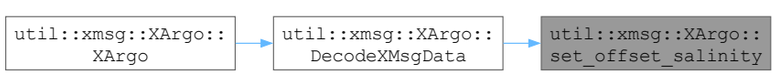
Parameters

in	*offs*	The sensor offset.

References offset_salinity_.

Referenced by DecodeXMsgData().

Here is the caller graph for this function:



10.58.3.56 set_offset_salinity_surface()

```
void util::xmsg::XArgo::set_offset_salinity_surface (
            uint16_t offs)  [inline], [protected]
```

Set the salinity sensor offset for surface data, in PSU.

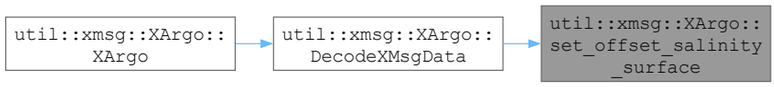
Parameters

in	*offs*	The sensor offset.

References offset_salinity_surface_.

Referenced by DecodeXMsgData().

Here is the caller graph for this function:



10.58.3.57 set_offset_temperature()

```
void util::xmsg::XArgo::set_offset_temperature (
            uint16_t offs)  [inline], [protected]
```

Set the temperature sensor offset, in degC.

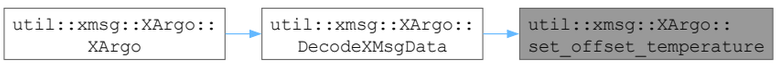
Parameters

in	*offs*	The sensor offset.

References offset_temperature_.

Referenced by DecodeXMsgData().

Here is the caller graph for this function:



10.58.3.58 set_offset_temperature_surface()

```
void util::xmsg::XArgo::set_offset_temperature_surface (
            uint16_t offs)  [inline], [protected]
```

Set the temperature sensor offset for surface data, in degC.

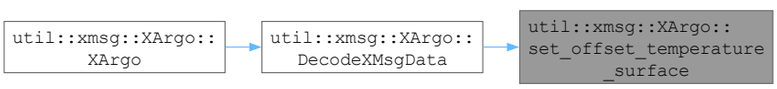
Parameters

in	*offs*	The sensor offset.

References offset_temperature_surface_.

Referenced by DecodeXMsgData().

Here is the caller graph for this function:



10.58.3.59 set_seek_to_drift_pressure_interval_duration()

```
void util::xmsg::XArgo::set_seek_to_drift_pressure_interval_duration (
            uint16_t duration)  [inline], [protected]
```

Set the seek-to-drift/park-pressure interval duration, in minutes.

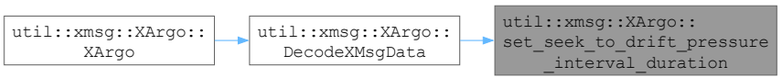
Parameters

in	*duration*	The interval duration.

References seek_to_drift_pressure_interval_duration_.

Referenced by DecodeXMsgData().

Here is the caller graph for this function:



10.58.3.60 set_surface_state_start_time()

```
void util::xmsg::XArgo::set_surface_state_start_time (
            const ptime & time)  [inline], [protected]
```

Set the start time of the surface state of the dive cycle.

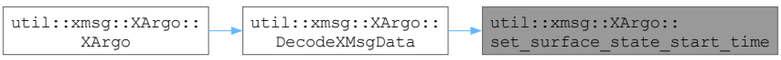
Parameters

in	*time*	The start time.

References surface_state_start_time_.

Referenced by DecodeXMsgData().

Here is the caller graph for this function:



10.58.3.61 set_target_ascent_rate()

```
void util::xmsg::XArgo::set_target_ascent_rate (
            uint8_t rate)  [inline], [protected]
```

Set the target ascent rate, in cm/s.

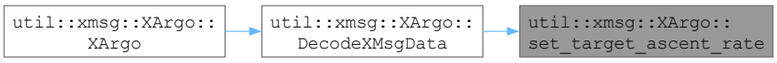
Parameters

in	*rate*	The target rate.

References target_ascent_rate_.

Referenced by DecodeXMsgData().

Here is the caller graph for this function:



10.58.3.62 set_target_drift_duration()

```
void util::xmsg::XArgo::set_target_drift_duration (
            uint16_t duration)  [inline], [protected]
```

Set the target duration of the drift/park phase of the dive cycle, in minutes.

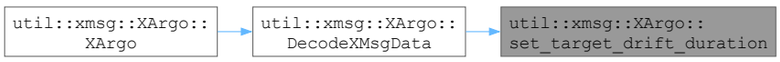
Parameters

in	*duration*	The target duration.

References target_drift_duration_.

Referenced by DecodeXMsgData().

Here is the caller graph for this function:



10.58.3.63 set_target_drift_pressure()

```
void util::xmsg::XArgo::set_target_drift_pressure (
            uint16_t pres)  [inline], [protected]
```

Set the target drift/park pressure, in dBar.

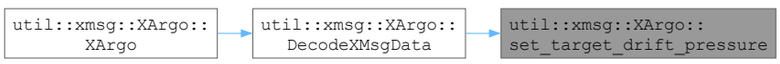
Parameters

in	*pres*	The target pressure.

References target_drift_pressure_.

Referenced by DecodeXMsgData().

Here is the caller graph for this function:



10.58.3.64 set_target_profile_pressure()

```
void util::xmsg::XArgo::set_target_profile_pressure (
            uint16_t pres)  [inline], [protected]
```

Set the target profile pressure, in dBar.

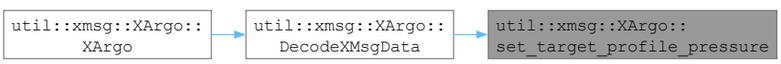
Parameters

in	*pres*	The target pressure.

References target_profile_pressure_.

Referenced by DecodeXMsgData().

Here is the caller graph for this function:



10.58.3.65 set_x_version_major()

```
void util::xmsg::XArgo::set_x_version_major (
            uint8_t vers)  [inline], [protected]
```

Set the X-Message major version identifier.

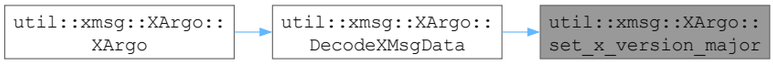
Parameters

in	*vers*	The version.

References x_version_major_.

Referenced by DecodeXMsgData().

Here is the caller graph for this function:



10.58.3.66 set_x_version_minor()

```
void util::xmsg::XArgo::set_x_version_minor (
            uint8_t vers)  [inline], [protected]
```

Set the X-Message minor version identifier.

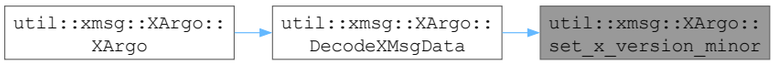
Parameters

in	*vers*	The version.

References x_version_minor_.

Referenced by DecodeXMsgData().

Here is the caller graph for this function:



10.58.3.67 set_xmsg_size_bytes()

```
void util::xmsg::XMessage::set_xmsg_size_bytes (
            uint16_t xmsg_size_bytes)  [inline], [protected], [inherited]
```

Set the size of the X-Message, in bytes.

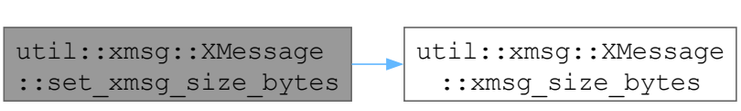
Parameters

in	*xmsg_size_bytes*	The size of the X-Message to set, in bytes.

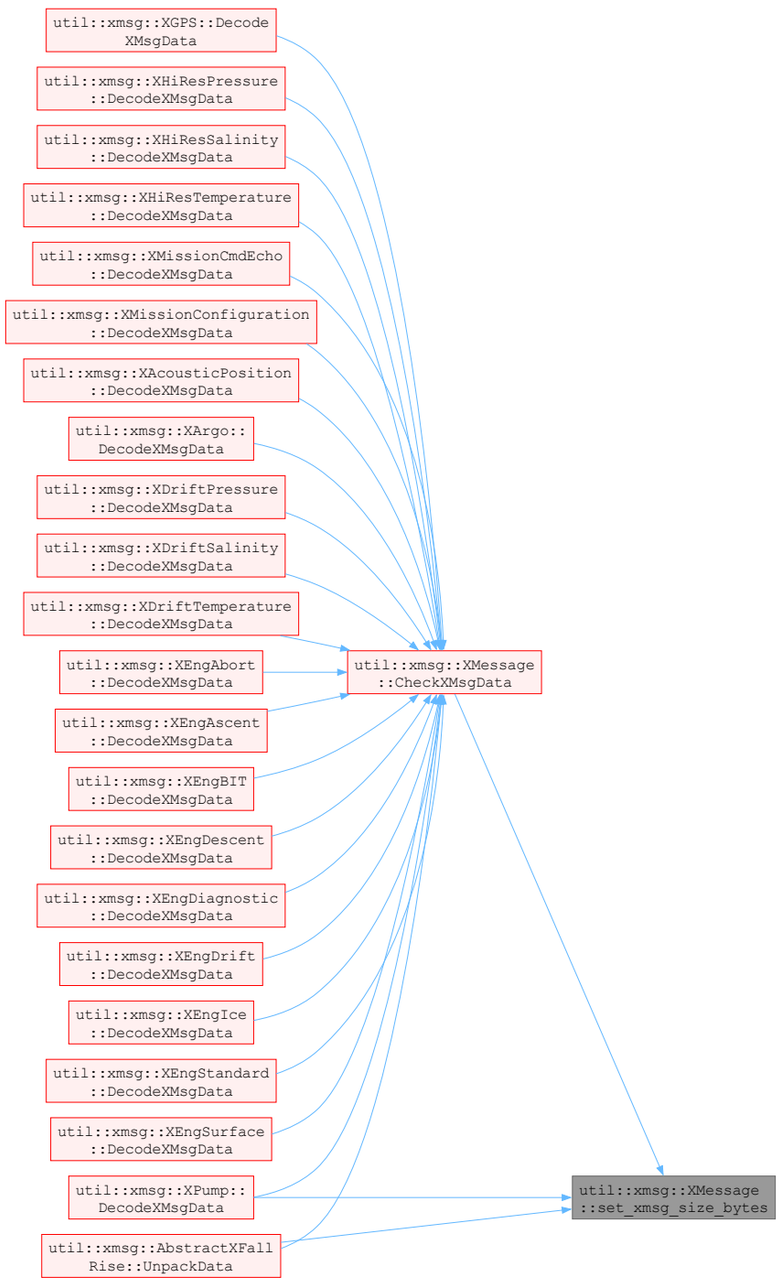
References util::xmsg::XMessage::xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XPump::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::Unpa

Here is the call graph for this function:



Here is the caller graph for this function:



10.58.3.68 surface_state_start_time()

```
const ptime & util::xmsg::XArgo::surface_state_start_time () const  [inline]
```

Get the start time of the surface state of the dive cycle.

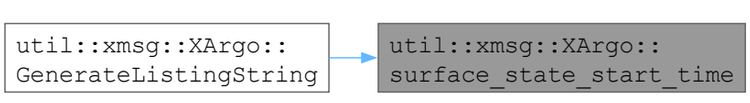
Returns

The start time of the surface state of the dive cycle.

References surface_state_start_time_.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.58.3.69 target_ascent_rate()

```
uint8_t util::xmsg::XArgo::target_ascent_rate () const  [inline]
```

Get the target ascent/profile rate, in cm/sec.

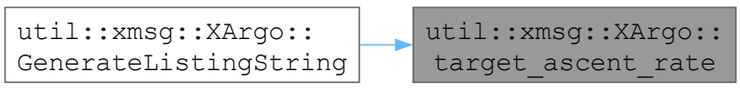
Returns

The target ascent/profile rate, in cm/sec.

References target_ascent_rate_.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.58.3.70 target_drift_duration()

```
uint16_t util::xmsg::XArgo::target_drift_duration () const  [inline]
```

Get the target druration of the drift/park phase of the dive cycle, in minutes.

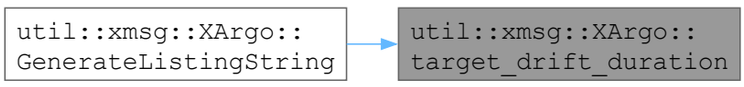
Returns

The target druration of the drift/park phase of the dive cycle, in minutes.

References target_drift_duration_.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.58.3.71 target_drift_pressure()

```
uint16_t util::xmsg::XArgo::target_drift_pressure () const  [inline]
```

Get the target drift/park pressure, in dBar.

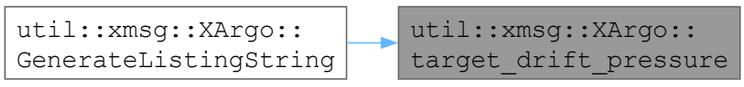
Returns

The target drift/park pressure, in dBar.

References target_drift_pressure_.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.58.3.72 target_profile_pressure()

```
uint16_t util::xmsg::XArgo::target_profile_pressure () const  [inline]
```

Get the target profile pressure, in dBar.

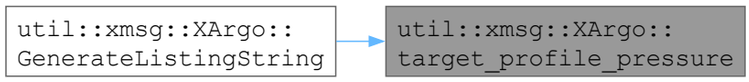
Returns

The target profile pressure, in dBar.

References target_profile_pressure_.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.58.3.73 x_version_float()

```
float util::xmsg::XArgo::x_version_float () const  [inline]
```

Get the X-Message version as a floating-point value.

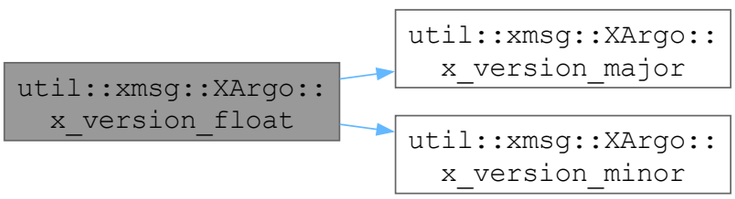
Returns

The X-Message version as a floating-point value.

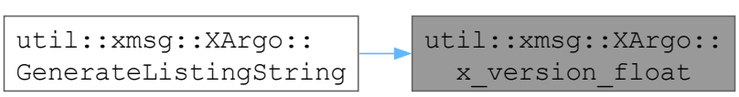
References x_version_major(), and x_version_minor().

Referenced by GenerateListingString().

Here is the call graph for this function:



Here is the caller graph for this function:



10.58.3.74 x_version_major()

```
uint8_t util::xmsg::XArgo::x_version_major () const  [inline]
```

Get the X-Message major version indicator.

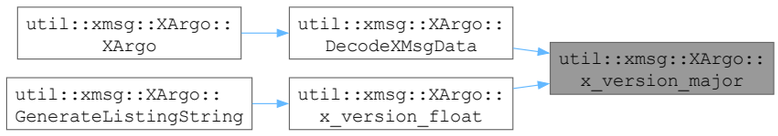
Returns

The X-Message major version indicator.

References x_version_major_.

Referenced by DecodeXMsgData(), and x_version_float().

Here is the caller graph for this function:



10.58.3.75 x_version_minor()

```
uint8_t util::xmsg::XArgo::x_version_minor () const  [inline]
```

Get the X-Message minor version indicator.

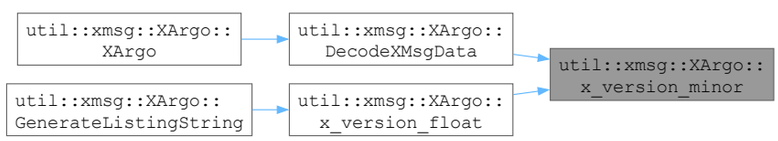
Returns

The X-Message minor version indicator.

References x_version_minor_.

Referenced by DecodeXMsgData(), and x_version_float().

Here is the caller graph for this function:



10.58.3.76 xmsg_size_bytes()

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes () const  [inline], [inherited]
```

Get the size of the X-Message, in bytes.

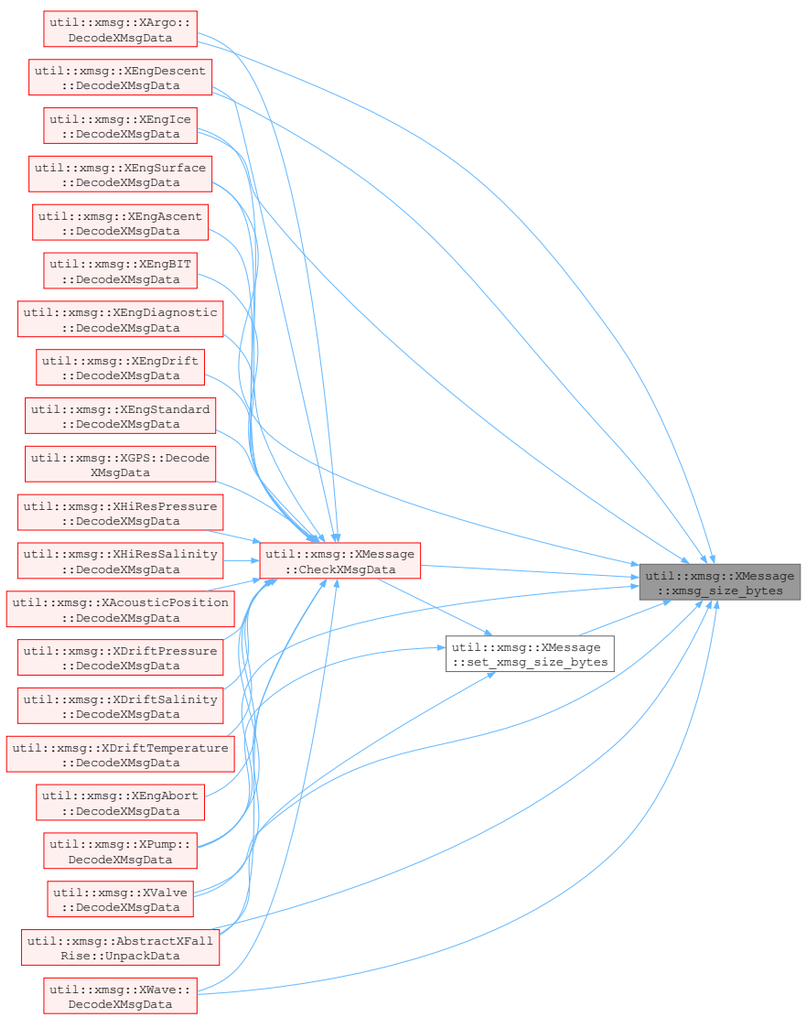
Returns

The size of the X-Message, in bytes.

References util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(), util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the caller graph for this function:



10.58.4 Member Data Documentation

10.58.4.1 argo_instrument_type_

```
XArgoInstrumentType_t util::xmsg::XArgo::argo_instrument_type_  [private]
```

The Argo instrument type identifier.

Referenced by argo_instrument_type(), and set_argo_instrument_type().

10.58.4.2 ascent_state_start_time_

```
ptime util::xmsg::XArgo::ascent_state_start_time_  [private]
```

The ascend state start time.

Note

This field is used for ALAMO-type floats only.

This field was added in X-Message format version 2.1.

Referenced by ascent_state_start_time(), and set_ascent_state_start_time().

10.58.4.3 descent_state_start_time_

```
ptime util::xmsg::XArgo::descent_state_start_time_  [private]
```

The descend state start time.

Note

This field is used for ALAMO-type floats only.

This field was added in X-Message format version 2.1.

Referenced by descent_state_start_time(), and set_descent_state_start_time().

10.58.4.4 drift_state_start_time_

```
ptime util::xmsg::XArgo::drift_state_start_time_  [private]
```

The park state start time.

Note

This field is used for ALAMO-type floats only.

This field was added in X-Message format version 2.1.

Referenced by drift_state_start_time(), and set_drift_state_start_time().

10.58.4.5 gain_pressure_

```
uint16_t util::xmsg::XArgo::gain_pressure_  [private]
```

The pressure sensitivity gain factor, in LSB/dBar.

Note

This field was added in X-Message format version 1.2.

Referenced by gain_pressure(), and set_gain_pressure().

10.58.4.6 gain_pressure_surface_

```
uint16_t util::xmsg::XArgo::gain_pressure_surface_  [private]
```

The pressure sensitivity gain factor for surface data, in LSB/dBar.

Note

This field was added in X-Message format version 2.2.

Referenced by gain_pressure_surface(), and set_gain_pressure_surface().

10.58.4.7 gain_salinity_

```
uint16_t util::xmsg::XArgo::gain_salinity_  [private]
```

The salinity sensitivity scale factor, in LSB/PSU.

Note

This field was added in X-Message format version 1.2.

Referenced by gain_salinity(), and set_gain_salinity().

10.58.4.8 gain_salinity_surface_

```
uint16_t util::xmsg::XArgo::gain_salinity_surface_  [private]
```

The salinity sensitivity scale factor for surface data, in LSB/PSU.

Note

This field was added in X-Message format version 2.2.

Referenced by gain_salinity_surface(), and set_gain_salinity_surface().

10.58.4.9 gain_temperature_

```
uint16_t util::xmsg::XArgo::gain_temperature_  [private]
```

The temperature sensitivity scale factor, in LSB/degC.

Note

This field was added in X-Message format version 1.2.

Referenced by gain_temperature(), and set_gain_temperature().

10.58.4.10 gain_temperature_surface_

```
uint16_t util::xmsg::XArgo::gain_temperature_surface_  [private]
```

The temperature sensitivity scale factor for surface data, in LSB/degC.

Note

This field was added in X-Message format version 2.2.

Referenced by gain_temperature_surface().

10.58.4.11 id_dive_

```
int16_t util::xmsg::XMessage::id_dive_  [private], [inherited]
```

The dive or cycle number of this message.

Referenced by util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::set_id_dive().

10.58.4.12 id_instrument_

```
uint16_t util::xmsg::XMessage::id_instrument_  [private], [inherited]
```

The unique WHOI identifier of this instrument.

Referenced by util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::set_id_instrument().

10.58.4.13 id_xmsg_type_

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type_  [private], [inherited]
```

The X-message type identifier.

Referenced by util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::set_id_xmsg_type().

10.58.4.14 max_ascent_duration_

```
uint16_t util::xmsg::XArgo::max_ascent_duration_  [private]
```

The maximum rise/ascend/profile time, in minutes.

Referenced by max_ascent_duration(), and set_max_ascent_duration().

10.58.4.15 max_fall_to_drift_duration_

```
uint16_t util::xmsg::XArgo::max_fall_to_drift_duration_  [private]
```

The maximum time to fall from the surface to drift/park depth, in minutes.

Referenced by max_fall_to_drift_duration(), and set_max_fall_to_drift_duration().

10.58.4.16 max_fall_to_profle_duration_

```
uint16_t util::xmsg::XArgo::max_fall_to_profle_duration_  [private]
```

The maximum time to fall from the drift/park depth to profile depth, in seconds.

Referenced by max_fall_to_profle_duration(), and set_max_fall_to_profle_duration().

10.58.4.17 max_surface_duration_

```
uint16_t util::xmsg::XArgo::max_surface_duration_  [private]
```

The maximum duration at the surface, in seconds.

Referenced by max_surface_duration(), and set_max_surface_duration().

10.58.4.18 mission_phase_

```
uint8_t util::xmsg::XArgo::mission_phase_  [private]
```

The mission phase number of this dive.

Referenced by mission_phase(), and set_mission_phase().

10.58.4.19 num_seeks_to_drift_pressure_

```
uint16_t util::xmsg::XArgo::num_seeks_to_drift_pressure_  [private]
```

The number of seeks to drift depth.

Referenced by num_seeks_to_drift_pressure(), and set_num_seeks_to_drift_pressure().

10.58.4.20 offset_pressure_

```
uint16_t util::xmsg::XArgo::offset_pressure_  [private]
```

The pressure offset, in dBar.

Note

This field was added in X-Message format version 1.2.

Referenced by offset_pressure(), and set_offset_pressure().

10.58.4.21 offset_pressure_surface_

```
uint16_t util::xmsg::XArgo::offset_pressure_surface_  [private]
```

The pressure offset for surface data, in dBar.

Note

This field was added in X-Message format version 2.2.

Referenced by offset_pressure_surface(), and set_offset_pressure_surface().

10.58.4.22 offset_salinity_

```
uint16_t util::xmsg::XArgo::offset_salinity_  [private]
```

The salinity offset, in PSU.

Note

This field was added in X-Message format version 1.2.

Referenced by offset_salinity(), and set_offset_salinity().

10.58.4.23 offset_salinity_surface_

```
uint16_t util::xmsg::XArgo::offset_salinity_surface_  [private]
```

The salinity offset for surface data, in PSU.

Note

This field was added in X-Message format version 2.2.

Referenced by offset_salinity_surface(), and set_offset_salinity_surface().

10.58.4.24 offset_temperature_

```
uint16_t util::xmsg::XArgo::offset_temperature_  [private]
```

The temperature offset, in degC.

Note

This field was added in X-Message format version 1.2.

Referenced by offset_temperature(), and set_offset_temperature().

10.58.4.25 offset_temperature_surface_

```
uint16_t util::xmsg::XArgo::offset_temperature_surface_  [private]
```

The temperature offset for surface data, in degC.

Note

This field was added in X-Message format version 2.2.

Referenced by offset_temperature_surface(), and set_offset_temperature_surface().

10.58.4.26 seek_to_drift_pressure_interval_duration_

```
uint16_t util::xmsg::XArgo::seek_to_drift_pressure_interval_duration_  [private]
```

The interval duration between subsequent seeks to drift/park depth, in minutes.

Referenced by seek_to_drift_pressure_interval_duration(), and set_seek_to_drift_pressure_interval_duration().

10.58.4.27 surface_state_start_time_

```
ptime util::xmsg::XArgo::surface_state_start_time_  [private]
```

The surface state start time.

Note

This field is used for ALAMO-type floats only.
This field was added in X-Message format version 2.1.

Referenced by set_surface_state_start_time(), and surface_state_start_time().

10.58.4.28 target_ascent_rate_

```
uint8_t util::xmsg::XArgo::target_ascent_rate_  [private]
```

The target ascent rate, in cm/s.

Referenced by set_target_ascent_rate(), and target_ascent_rate().

10.58.4.29 target_drift_duration_

```
uint16_t util::xmsg::XArgo::target_drift_duration_  [private]
```

The target duration of the drift/park phase of the dive cycle, in minutes.

Referenced by set_target_drift_duration(), and target_drift_duration().

10.58.4.30 target_drift_pressure_

```
uint16_t util::xmsg::XArgo::target_drift_pressure_  [private]
```

The target parking or drift depth (DTD), in dBar.

Referenced by set_target_drift_pressure(), and target_drift_pressure().

10.58.4.31 target_profile_pressure_

```
uint16_t util::xmsg::XArgo::target_profile_pressure_  [private]
```

The target profile depth (PFD), in dBar.

Referenced by set_target_profile_pressure(), and target_profile_pressure().

10.58.4.32 x_version_major_

```
uint8_t util::xmsg::XArgo::x_version_major_  [private]
```

The X message format major version number.

Referenced by set_x_version_major(), and x_version_major().

10.58.4.33 x_version_minor_

```
uint8_t util::xmsg::XArgo::x_version_minor_  [private]
```

The X Message format minor version number.

Referenced by set_x_version_minor(), and x_version_minor().

10.58.4.34 xmsg_size_bytes_

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes_  [private], [inherited]
```

The size of the X message, in bytes.

Referenced by util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes().

The documentation for this class was generated from the following file:

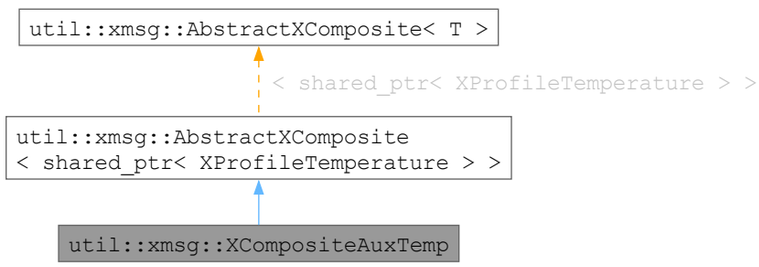
- data/xmsg/XArgo.h

10.59 util::xmsg::XCompositeAuxTemp Class Reference

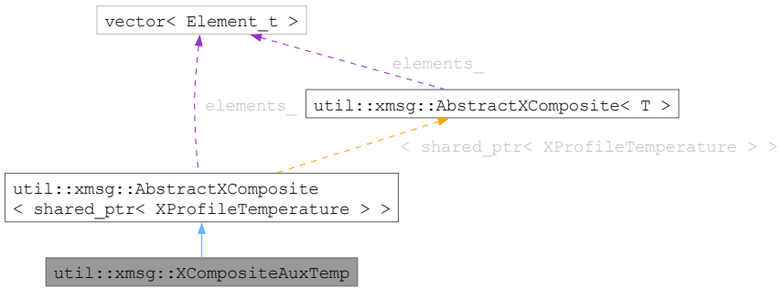
The composite auxiliary temperature class.

```
#include <XCompositeAuxTemp.h>
```

Inheritance diagram for util::xmsg::XCompositeAuxTemp:



Collaboration diagram for util::xmsg::XCompositeAuxTemp:



Public Member Functions

- void AppendAllElements (const static_vector< Element_t, I > &elements)

  Append a vector of elements to the vector of individual X-Messages elements that compose this composite object.
- virtual void AssignElement (const Element_t &element)

  Assign an element to this composite object.
- const vector< Element_t > & elements () const

  Get the vector of individual X-Messages elements that compose this composite object.
- const string GenerateListingString (int id_inst, int id_dive, uint16_t tGain, uint16_t tOff)

  Generate the listing file section for the auxiliary temperature data.
- virtual bool **IsEmpty** () const
- virtual size_t num_elements () const

  Get the number of individual elements that compose this composite object.

Protected Member Functions

- void AppendAllElements (const vector< Element_t > &elements)

  Append a vector of elements to the vector of individual X-Messages elements that compose this composite object.
- void AppendElement (const Element_t &element)

  Append one element to the vector of individual X-Messages elements that compose this composite object.

Private Types

- typedef shared_ptr< XProfileTemperature > Element_t

  Type definition for the element aggregated by this object.
- typedef ElementVec_t::const_iterator **ElementConstIter_t**
- typedef ElementVec_t::iterator **ElementIter_t**
- typedef vector< Element_t > **ElementVec_t**

Private Attributes

- ElementVec_t elements_

  The vector of individual X-Messages elements that compose this composite object.

10.59.1 Detailed Description

The composite auxiliary temperature class.

An instance of this class aggregates the auxiliary-channel profile temperature messages (see XProfileTemperature) received for a single dive.

10.59.2 Member Typedef Documentation

10.59.2.1 Element_t

```
typedef shared_ptr< XProfileTemperature > util::xmsg::AbstractXComposite< shared_ptr< XProfileTemperature
> >::Element_t  [private], [inherited]
```

Type definition for the element aggregated by this object.

10.59.3 Member Function Documentation

10.59.3.1 AppendAllElements() [1/2]

```
void util::xmsg::AbstractXComposite< shared_ptr< XProfileTemperature > >::AppendAllElements (
            const static_vector< Element_t, I > & elements)  [inline], [inherited]
```

Append a vector of elements to the vector of individual X-Messages elements that compose this composite object.

Parameters

in	*elements*	The vector of elements to append.

10.59.3.2 AppendAllElements() [2/2]

```
void util::xmsg::AbstractXComposite< shared_ptr< XProfileTemperature > >::AppendAllElements (
            const vector< Element_t > & elements)  [inline], [protected], [inherited]
```

Append a vector of elements to the vector of individual X-Messages elements that compose this composite object.

Parameters

in	*elements*	The vector of elements to append.

10.59.3.3 AppendElement()

```
void util::xmsg::AbstractXComposite< shared_ptr< XProfileTemperature > >::AppendElement (
            const Element_t & element)  [inline], [protected], [inherited]
```

Append one element to the vector of individual X-Messages elements that compose this composite object.

Parameters

in	*element*	The element to append.

10.59.3.4 AssignElement()

```
virtual void util::xmsg::AbstractXComposite< shared_ptr< XProfileTemperature > >::AssignElement
(
            const Element_t & element)  [inline], [virtual], [inherited]
```

Assign an element to this composite object.

The default implementation appends the element; derived classes may override it to insert elements in order or reject duplicates.

Parameters

in	*element*	The element to assign.

10.59.3.5 elements()

```
const vector< Element_t > & util::xmsg::AbstractXComposite< shared_ptr< XProfileTemperature >
>::elements () const  [inline], [inherited]
```

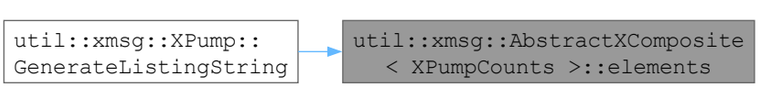
Get the vector of individual X-Messages elements that compose this composite object.

Returns

The vector of individual X-Messages elements that compose this composite object.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.59.3.6 GenerateListingString()

```
const string util::xmsg::XCompositeAuxTemp::GenerateListingString (
            int id_inst,
            int id_dive,
            uint16_t tGain,
            uint16_t tOff)  [inline]
```

Generate the listing file section for the auxiliary temperature data.

Writes an `AXT` header line, then one `axt` line per sample of each auxiliary temperature message, with the raw counts and the temperature in degrees Celsius.

Parameters

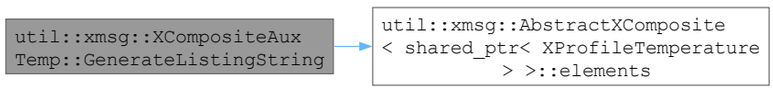
in	*id_inst*	The unique instrument identifier.
in	*id_dive*	The dive or cycle number.
in	*tGain*	The temperature gain used to convert counts.
in	*tOff*	The temperature offset used to convert counts.

Returns

The listing file section.

References util::xmsg::AbstractXComposite< shared_ptr< XProfileTemperature > >::elements().

Here is the call graph for this function:



10.59.3.7 num_elements()

```
virtual size_t util::xmsg::AbstractXComposite< shared_ptr< XProfileTemperature > >::num_elements
() const  [inline], [virtual], [inherited]
```

Get the number of individual elements that compose this composite object.

Returns

The number of individual elements that compose this composite object.

10.59.4 Member Data Documentation

10.59.4.1 elements_

```
ElementVec_t util::xmsg::AbstractXComposite< shared_ptr< XProfileTemperature > >::elements_↩
[private], [inherited]
```

The vector of individual X-Messages elements that compose this composite object.

The documentation for this class was generated from the following file:

- data/xmsg/XCompositeAuxTemp.h

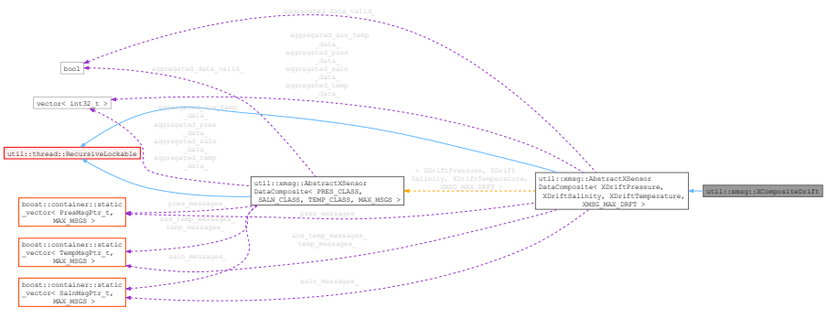
10.60 util::xmsg::XCompositeDrift Class Reference

Class representing the aggregate drift data for a single dive, composed from one or more drift messages.

`#include <XCompositeDrift.h>`

Inheritance diagram for util::xmsg::XCompositeDrift:

Collaboration diagram for util::xmsg::XCompositeDrift:



Public Member Functions

- XCompositeDrift ()

  The default constructor.
- ~XCompositeDrift ()

  The destructor.
- void AssignAuxTemperatureMessage (const TempMsgPtr_t &x)

  Assign an auxiliary-channel temperature message and re-aggregate the sensor data.
- void AssignPressureMessage (const PresMsgPtr_t &x)

  Assign a pressure data message to this composite data object.
- void AssignSalinityMessage (const SalnMsgPtr_t &x)

  Assign a salinity data message to this composite data object.
- void AssignTemperatureMessage (const TempMsgPtr_t &x)

  Assign a temperature data message to this composite data object.
- void **CopyPressureDataToArray** (T dest[], size_t max_len, int16_t gain, int16_t offs, float pres_correction, T fill_value=XMSG_SENS_BAD_PRES) const
- void **CopySalinityDataToArray** (T dest[], size_t max_len, int16_t gain, int16_t offs, T fill_value=XMSG_SENS_BAD_SALN) const
- void **CopyTemperatureDataToArray** (T dest[], size_t max_len, int16_t gain, int16_t offs, T fill_↩ value=XMSG_SENS_BAD_TEMP) const
- const string GenerateListingString (int id_inst, int id_dive, uint16_t pg, uint16_t po, uint16_t sg, uint16_t so, uint16_t tg, uint16_t to, float pc) const

  Generate a listing string representation of the this composite sensor data object.
- const string GeneratePhyString (uint16_t pg, uint16_t po, uint16_t sg, uint16_t so, uint16_t tg, uint16_t to, float pc) const

  Generate a PHY entry string from this composite drift data object.
- bool **HasAuxTempData** () const
- bool HasContiguousMessageIndices () const

  Do the sensor data messages assigned to this composite data object have contiguous message indices?
- bool is_aggregated_data_valid () const

  Is the aggregated data valid?
- bool IsDataComplete () const

  Is the sensor data for this composite date message complete?
- bool IsEmpty () const

  Is this composite data object empty?
- bool LockObject (bool block)

  Lock the object.

- size_t max_num_samples () const

  Return the maximum of the number of pressure, temperature, or salinity sensor samples in the sensor data messages assigned to this composite sensor data object.
- size_t **num_aux_temp_samples** () const
- size_t num_pres_samples () const

  Get the total number of pressure sensor samples contained in the pressure data messages assigned to this composite data object.
- size_t num_saln_samples () const

  Get the total number of salinity sensor samples contained in the salinity data messages assigned to this composite data object.
- size_t num_temp_samples () const

  Get the total number of temperature sensor samples contained in the temperature data messages assigned to this composite data object.
- void Reset ()

  Remove the assigned pressure, temperature and salinity messages and their aggregated data.
- T **SampleAuxTempAsFloat** (size_t i, int16_t gain, int16_t offs) const
- int32_t SamplePres (size_t i) const

  Get the i^{th} pressure sensor value, in raw counts.
- T SamplePresAsFloat (size_t i, int16_t pg, int16_t po, float pc) const

  Get the i^{th} pressure sensor value, in dBar.
- int32_t SampleSaln (size_t i) const

  Get the i^{th} salinity sensor value, in raw counts.
- T SampleSalnAsFloat (size_t i, int16_t gain, int16_t offs) const

  Get the i^{th} salinity sensor value, in PSU.
- int32_t SampleTemp (size_t i) const

  Get the i^{th} temperature sensor value, in raw counts.
- T SampleTempAsFloat (size_t i, int16_t gain, int16_t offs) const

  Get the i^{th} temperature sensor value, in degC.
- void UnlockObject ()

  Unlock the object.

Protected Member Functions

- void AggregateSensorData () ATTR_NO_EXCEPTIONS

  Aggregate the sensor data for this composite sensor data object.
- void set_aggregated_data_valid (bool valid)

  Set the value of the aggregated_data_valid_ field.

Protected Attributes

- recursive_mutex mutex_

  Recursive mutual exclusion lock for synchronizing operations.

Private Types

- typedef shared_ptr< XDriftPressure > PresMsgPtr_t

  Type definition for a shared pointer to the pressure data message.
- typedef static_vector< PresMsgPtr_t, MAX_MSGS > PresMsgVector_t

  Type definition for a static vector (of size MAX_MSGS) of shared pointers to the pressure data message.
- typedef shared_ptr< XDriftSalinity > SalnMsgPtr_t

  Type definition for a shared pointer to the salinity data message.
- typedef static_vector< SalnMsgPtr_t, MAX_MSGS > SalnMsgVector_t

  Type definition for a static vector (of size MAX_MSGS) of shared pointers to the salinity data message.
- typedef shared_ptr< XDriftTemperature > TempMsgPtr_t

  Type definition for a shared pointer to the temperature data message.
- typedef static_vector< TempMsgPtr_t, MAX_MSGS > TempMsgVector_t

  Type definition for a static vector (of size MAX_MSGS) of shared pointers to the temperature data message.

Private Attributes

- vector< int32_t > **aggregated_aux_temp_data_**
- bool aggregated_data_valid_

  The aggregated data is valid flag.
- vector< int32_t > aggregated_pres_data_

  A vector of the aggregated pressure data samples, in raw counts.
- vector< int32_t > aggregated_saln_data_

  A vector of the aggregated salinity data samples, in raw counts.
- vector< int32_t > aggregated_temp_data_

  A vector of the aggregated temperature data samples, in raw counts.
- TempMsgVector_t **aux_temp_messages_**
- PresMsgVector_t pres_messages_

  A static vector (of size MAX_MSGS) containing pointers the pressure sensor data messages assigned to this composite object.
- SalnMsgVector_t saln_messages_

  A static vector (of size MAX_MSGS) containing pointers the salinity sensor data messages assigned to this composite object.
- TempMsgVector_t temp_messages_

  A static vector (of size MAX_MSGS) containing pointers the temperature sensor data messages assigned to this composite object.

10.60.1 Detailed Description

Class representing the aggregate drift data for a single dive, composed from one or more drift messages.

10.60.2 Member Typedef Documentation

10.60.2.1 PresMsgPtr_t

```
typedef shared_ptr<XDriftPressure> util::xmsg::AbstractXSensorDataComposite< XDriftPressure,
XDriftSalinity, XDriftTemperature, MAX_MSGS >::PresMsgPtr_t  [private], [inherited]
```

Type definition for a shared pointer to the pressure data message.

10.60.2.2 PresMsgVector_t

```
typedef static_vector<PresMsgPtr_t, MAX_MSGS> util::xmsg::AbstractXSensorDataComposite< XDriftPressure,
XDriftSalinity, XDriftTemperature, MAX_MSGS >::PresMsgVector_t  [private], [inherited]
```

Type definition for a static vector (of size MAX_MSGS) of shared pointers to the pressure data message.

10.60.2.3 SalnMsgPtr_t

```
typedef shared_ptr<XDriftSalinity> util::xmsg::AbstractXSensorDataComposite< XDriftPressure,
XDriftSalinity, XDriftTemperature, MAX_MSGS >::SalnMsgPtr_t  [private], [inherited]
```

Type definition for a shared pointer to the salinity data message.

10.60.2.4 SalnMsgVector_t

```
typedef static_vector<SalnMsgPtr_t, MAX_MSGS> util::xmsg::AbstractXSensorDataComposite< XDriftPressure,
XDriftSalinity, XDriftTemperature, MAX_MSGS >::SalnMsgVector_t  [private], [inherited]
```

Type definition for a static vector (of size MAX_MSGS) of shared pointers to the salinity data message.

10.60.2.5 TempMsgPtr_t

typedef shared_ptr<XDriftTemperature> util::xmsg::AbstractXSensorDataComposite< XDriftPressure, XDriftSalinity, XDriftTemperature, MAX_MSGS >::TempMsgPtr_t [private], [inherited]

Type definition for a shared pointer to the temperature data message.

10.60.2.6 TempMsgVector_t

typedef static_vector<TempMsgPtr_t, MAX_MSGS> util::xmsg::AbstractXSensorDataComposite< XDriftPressure, XDriftSalinity, XDriftTemperature, MAX_MSGS >::TempMsgVector_t [private], [inherited]

Type definition for a static vector (of size MAX_MSGS) of shared pointers to the temperature data message.

10.60.3 Constructor & Destructor Documentation

10.60.3.1 XCompositeDrift()

util::xmsg::XCompositeDrift::XCompositeDrift () [inline]

The default constructor.

10.60.3.2 ~XCompositeDrift()

util::xmsg::XCompositeDrift::~XCompositeDrift () [inline]

The destructor.

10.60.4 Member Function Documentation

10.60.4.1 AggregateSensorData()

void util::xmsg::AbstractXSensorDataComposite< XDriftPressure, XDriftSalinity, XDriftTemperature, MAX_MSGS >::AggregateSensorData () [inline], [protected], [inherited]

Aggregate the sensor data for this composite sensor data object.

This function iterates over each of the sensor data messages assigned to this composite data object, and copies of the sensor data values into the corresponding sensor data vector.

If aggregated_data_valid_ is TRUE, then this function will have no effect. Otherwise, each of the aggregegated sensor data vectors will be cleared and the appropriate number of samples will be reserved.

Then, the sensor data message will be iterated over in order of increasing message indices, and the sensor data values will be copied into the corresponding sensor data vector in the appropriate order.

Finally, aggregated_data_valid_ will be set to FALSE to ensure that subsequent calls to this function will have no effect, until another sensor data message object is assigned to this composite object and aggregated_data_valid_ is set to TRUE.

Exceptions

util::log::MessageException	if an error occurs while aggregating the sensor data.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

10.60.4.2 AssignAuxTemperatureMessage()

void util::xmsg::AbstractXSensorDataComposite< XDriftPressure, XDriftSalinity, XDriftTemperature, MAX_MSGS >::AssignAuxTemperatureMessage (
const TempMsgPtr_t & x) [inline], [inherited]

Assign an auxiliary-channel temperature message and re-aggregate the sensor data.

The message is inserted in order of message index, and the aggregated sensor data is rebuilt.

Parameters

in	*x*	The auxiliary-channel temperature message to assign.

Exceptions

util::log::MessageException	if x is NULL or an auxiliary temperature message with the same index has already been assigned.

10.60.4.3 AssignPressureMessage()

```
void util::xmsg::AbstractXSensorDataComposite< XDriftPressure, XDriftSalinity, XDriftTemperature,
MAX_MSGS >::AssignPressureMessage (
            const PresMsgPtr_t & x)  [inline], [inherited]
```

Assign a pressure data message to this composite data object.

The pressure data message will be emplaced into the sensor data message vector in order of increasing message indices. If a message with the same index is as x is already contained in the vector, then an exception will be thrown.

Parameters

in	*x*	A shared pointer to the pressure data message.

Exceptions

util::log::MessageException	if x is a null pointer, a duplicate message index is encountered, or any other error occurs.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

10.60.4.4 AssignSalinityMessage()

```
void util::xmsg::AbstractXSensorDataComposite< XDriftPressure, XDriftSalinity, XDriftTemperature,
MAX_MSGS >::AssignSalinityMessage (
            const SalnMsgPtr_t & x)  [inline], [inherited]
```

Assign a salinity data message to this composite data object.

The salinity data message will be emplaced into the sensor data message vector in order of increasing message indices. If a message with the same index is as x is already contained in the vector, then an exception will be thrown.

Parameters

in	*x*	A shared pointer to the salinity data message.

Exceptions

util::log::MessageException	if x is a null pointer, a duplicate message index is encountered, or any other error occurs.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

10.60.4.5 AssignTemperatureMessage()

```
void util::xmsg::AbstractXSensorDataComposite< XDriftPressure, XDriftSalinity, XDriftTemperature,
MAX_MSGS >::AssignTemperatureMessage (
            const TempMsgPtr_t & x)  [inline], [inherited]
```

Assign a temperature data message to this composite data object.

The temperature data message will be emplaced into the sensor data message vector in order of increasing message indices. If a message with the same index is as `x` is already contained in the vector, then an exception will be thrown.

Parameters

in	*x*	A shared pointer to the temperature data message.

Exceptions

util::log::MessageException	if `x` is a null pointer, a duplicate message index is encountered, or any other error occurs.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

10.60.4.6 GenerateListingString()

```
const string util::xmsg::XCompositeDrift::GenerateListingString (
            int id_inst,
            int id_dive,
            uint16_t pg,
            uint16_t po,
            uint16_t sg,
            uint16_t so,
            uint16_t tg,
            uint16_t to,
            float pc) const  [inline], [virtual]
```

Generate a listing string representation of the this composite sensor data object.

Parameters

in	*id_inst*	The unique instrument identifier for this composite sensor data object.
in	*id_dive*	The dive or cycle number for this composite data object.
in	*pg*	The pressure sensor sensitivity scale factor, in LSB/dBar.
in	*po*	The pressure sensor offset, in dBar.
in	*sg*	The salinity sensor sensitivity scale factor, in LSB/PSU.
in	*so*	The salinity sensor offset, in PSU.
in	*tg*	The temperature sensor sensitivity scale factor, in LSB/degC.
in	*to*	The temperature sensor offset, in degC.
in	*pc*	The pressure correction, in dBar, to be subtracted computed pressure value.

Exceptions

log::MessageException	if an error occurs while generating the listing string.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

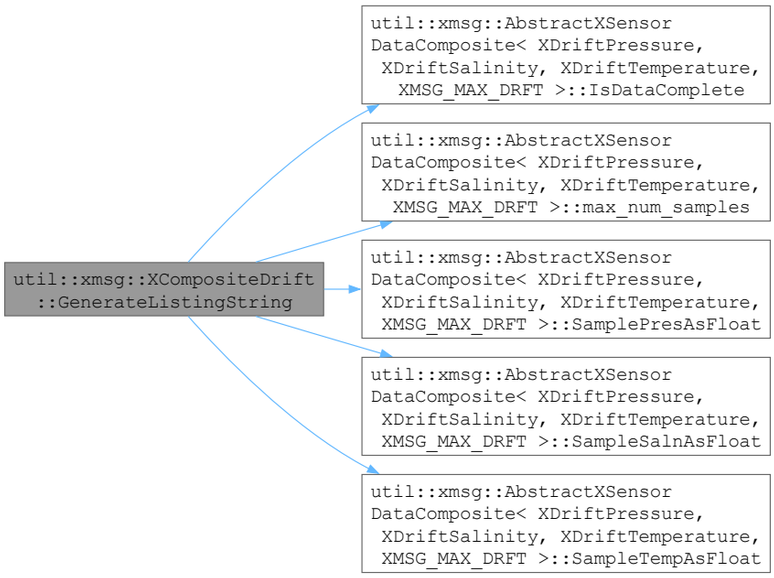
A listing string representation of this composite sensor data object.

Implements util::xmsg::AbstractXSensorDataComposite< XDriftPressure, XDriftSalinity, XDriftTemperature, XMSG_MAX_DRFT >.

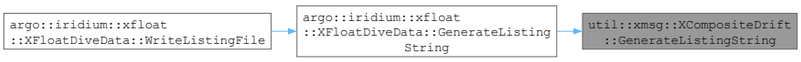
References CATCH_BLOCK, util::xmsg::AbstractXSensorDataComposite< XDriftPressure, XDriftSalinity, XDriftTemperature, XMSG_MA
util::xmsg::AbstractXSensorDataComposite< XDriftPressure, XDriftSalinity, XDriftTemperature, XMSG_MAX_DRFT >::max_num_sampl
util::thread::RecursiveLockable::mutex_, util::xmsg::AbstractXSensorDataComposite< XDriftPressure, XDriftSalinity, XDriftTemperature, X
util::xmsg::AbstractXSensorDataComposite< XDriftPressure, XDriftSalinity, XDriftTemperature, XMSG_MAX_DRFT >::SampleSalnAsFl
and util::xmsg::AbstractXSensorDataComposite< XDriftPressure, XDriftSalinity, XDriftTemperature, XMSG_MAX_DRFT >::SampleTemp

Referenced by argo::iridium::xfloat::XFloatDiveData::GenerateListingString().

Here is the call graph for this function:



Here is the caller graph for this function:



10.60.4.7 GeneratePhyString()

```
const string util::xmsg::XCompositeDrift::GeneratePhyString (
            uint16_t pg,
            uint16_t po,
            uint16_t sg,
            uint16_t so,
            uint16_t tg,
            uint16_t to,
            float pc) const  [inline]
```

Generate a PHY entry string from this composite drift data object.

Parameters

in	*pg*	The pressure sensor sensitivity scale factor, in LSB/dBar.
in	*po*	The pressure sensor offset, in dBar.
in	*sg*	The salinity sensor sensitivity scale factor, in LSB/PSU.
in	*so*	The salinity sensor offset, in PSU.
in	*tg*	The temperature sensor sensitivity scale factor, in LSB/degC.
in	*to*	The temperature sensor offset, in degC.
in	*pc*	The pressure correction, in dBar, to be subtracted computed pressure value.

Exceptions

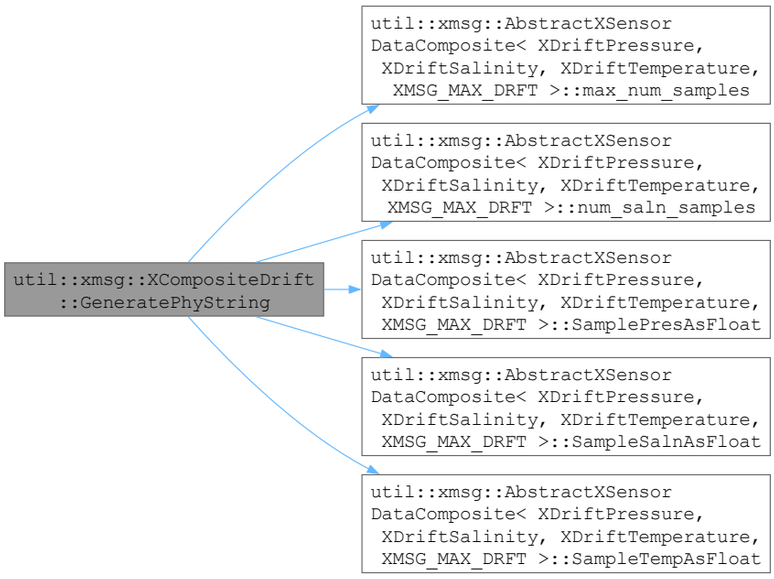
log::MessageException	if an error occurs while generating the PHY string.

Returns

A PHY entry string representation of this composite drift dataobject.

References CATCH_BLOCK, util::xmsg::AbstractXSensorDataComposite< XDriftPressure, XDriftSalinity, XDriftTemperature, XMSG_MA
util::thread::RecursiveLockable::mutex_, util::xmsg::AbstractXSensorDataComposite< XDriftPressure, XDriftSalinity, XDriftTemperature, X
util::xmsg::AbstractXSensorDataComposite< XDriftPressure, XDriftSalinity, XDriftTemperature, XMSG_MAX_DRFT >::SamplePresAsFl
util::xmsg::AbstractXSensorDataComposite< XDriftPressure, XDriftSalinity, XDriftTemperature, XMSG_MAX_DRFT >::SampleSalnAsFl
and util::xmsg::AbstractXSensorDataComposite< XDriftPressure, XDriftSalinity, XDriftTemperature, XMSG_MAX_DRFT >::SampleTemp

Here is the call graph for this function:



10.60.4.8 HasContiguousMessageIndices()

```
bool util::xmsg::AbstractXSensorDataComposite< XDriftPressure, XDriftSalinity, XDriftTemperature,
MAX_MSGS >::HasContiguousMessageIndices () const  [inline], [inherited]
```

Do the sensor data messages assigned to this composite data object have contiguous message indices?

This function is used, in part, to determine the sensor data is complete.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

TRUE if the sensor data messages have contiguous message indices or if there is less than two of each sensor data message, or FALSE otherwise.

10.60.4.9 is_aggregated_data_valid()

bool util::xmsg::AbstractXSensorDataComposite< XDriftPressure, XDriftSalinity, XDriftTemperature, MAX_MSGS >::is_aggregated_data_valid () const [inline], [inherited]

Is the aggregated data valid?

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

TRUE if the aggregated sensor is valid, or FALSE otherwise.

10.60.4.10 IsDataComplete()

bool util::xmsg::AbstractXSensorDataComposite< XDriftPressure, XDriftSalinity, XDriftTemperature, MAX_MSGS >::IsDataComplete () const [inline], [inherited]

Is the sensor data for this composite date message complete?

This function returns true if

- this composite sensor data object is not empty,
- the sensor data messages assigned to this composite data object have contiguous message indices,
- this sensor data object contains the same number of temperature and pressure sensor values, and
- the number of salinity data samples is either zero or equal to the number of pressure data samples.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

TRUE if the sensor data is complete, or FALSE otherwise.

Referenced by GenerateListingString().

Here is the caller graph for this function:

10.60.4.11 IsEmpty()

bool util::xmsg::AbstractXSensorDataComposite< XDriftPressure, XDriftSalinity, XDriftTemperature, MAX_MSGS >::IsEmpty () const [inline], [inherited]

Is this composite data object empty?

Returns

TRUE if empty, or FALSE otherwise.

10.60.4.12 LockObject()

bool util::thread::RecursiveLockable::LockObject (
bool *block*) [inline], [inherited]

Lock the object.

Parameters

in	*block*	If TRUE, then the calling thread will block until the lock becomes available.

Exceptions

util::log::MessageException	if an error occurs.

Returns

TRUE if the lock was successfully obtained, or FALSE if block is FALSE and another thread owns the lock on the mutex.

References CATCH_BLOCK, and util::thread::RecursiveLockable::mutex_.

10.60.4.13 max_num_samples()

`size_t util::xmsg::AbstractXSensorDataComposite< XDriftPressure, XDriftSalinity, XDriftTemperature, MAX_MSGS >::max_num_samples () const [inline], [inherited]`

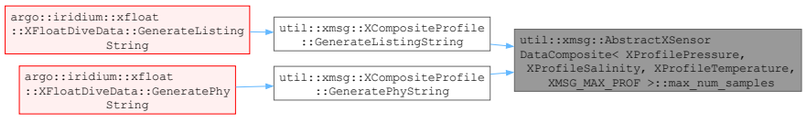
Return the maximum of the number of pressure, temperature, or salinity sensor samples in the sensor data messages assigned to this composite sensor data object.

Returns

The maximum of the number of pressure, temperature, or salinity sensor samples.

Referenced by GenerateListingString(), and GeneratePhyString().

Here is the caller graph for this function:



10.60.4.14 num_pres_samples()

`size_t util::xmsg::AbstractXSensorDataComposite< XDriftPressure, XDriftSalinity, XDriftTemperature, MAX_MSGS >::num_pres_samples () const [inline], [inherited]`

Get the total number of pressure sensor samples contained in the pressure data messages assigned to this composite data object.

Exceptions

util::log::MessageException	if an error occurs.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The total number of pressure sensor samples contained in the pressure data messages assigned to this composite data object.

10.60.4.15 num_saln_samples()

`size_t util::xmsg::AbstractXSensorDataComposite< XDriftPressure, XDriftSalinity, XDriftTemperature, MAX_MSGS >::num_saln_samples () const [inline], [inherited]`

Get the total number of salinity sensor samples contained in the salinity data messages assigned to this composite data object.

Exceptions

util::log::MessageException	if an error occurs.

Note

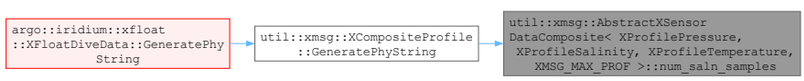
This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The total number of pressure sensor samples contained in the pressure data messages assigned to this composite data object.

Referenced by GeneratePhyString().

Here is the caller graph for this function:



10.60.4.16 num_temp_samples()

```
size_t util::xmsg::AbstractXSensorDataComposite< XDriftPressure, XDriftSalinity, XDriftTemperature,
MAX_MSGS >::num_temp_samples () const  [inline], [inherited]
```

Get the total number of temperature sensor samples contained in the temperature data messages assigned to this composite data object.

Exceptions

util::log::MessageException	if an error occurs.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The total number of pressure sensor samples contained in the pressure data messages assigned to this composite data object.

10.60.4.17 Reset()

```
void util::xmsg::AbstractXSensorDataComposite< XDriftPressure, XDriftSalinity, XDriftTemperature,
MAX_MSGS >::Reset ()  [inline], [inherited]
```

Remove the assigned pressure, temperature and salinity messages and their aggregated data.

Warning

The auxiliary temperature messages and aggregated auxiliary temperature data are not cleared, so re-assigning them after a reset fails with a duplicate index.

Exceptions

util::log::MessageException	if an error occurs.

10.60.4.18 SamplePres()

```
int32_t util::xmsg::AbstractXSensorDataComposite< XDriftPressure, XDriftSalinity, XDriftTemperature,
MAX_MSGS >::SamplePres (
            size_t i) const  [inline], [inherited]
```

Get the i^{th} pressure sensor value, in raw counts.

Parameters

in	*i*	The index of the pressure sensor sample to return. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD will be returned.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The i^{th} pressure sensor value, in raw counts, or XMSG_SENS_BAD if the index `i` exceeds the number of pressure sensor samples in the aggregated data vector.

10.60.4.19 SamplePresAsFloat()

```
T util::xmsg::AbstractXSensorDataComposite< XDriftPressure, XDriftSalinity, XDriftTemperature,
MAX_MSGS >::SamplePresAsFloat (
            size_t i,
            int16_t pg,
            int16_t po,
            float pc) const  [inline], [inherited]
```

Get the i^{th} pressure sensor value, in dBar.

Parameters

in	*i*	The index of the sensor value. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD_PRES will be returned.
in	*pg*	The pressure sensor sensitivity scale factor, in LSB/dBar.
in	*po*	The pressure sensor offset, in dBar.
in	*pc*	The pressure correction, in dBar, to be subtracted computed pressure value.

Note

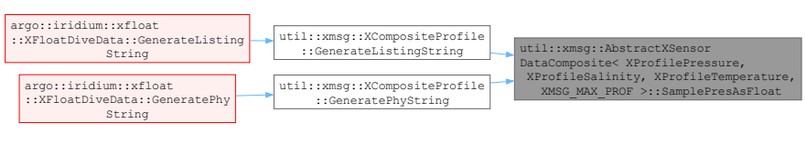
This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The i^{th} pressure sensor value, in dBar. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD_PRES will be returned.

Referenced by GenerateListingString(), and GeneratePhyString().

Here is the caller graph for this function:



10.60.4.20 SampleSaln()

```
int32_t util::xmsg::AbstractXSensorDataComposite< XDriftPressure, XDriftSalinity, XDriftTemperature,
MAX_MSGS >::SampleSaln (
            size_t i) const  [inline], [inherited]
```

Get the i^{th} salinity sensor value, in raw counts.

Parameters

in	*i*	The index of the salinity sensor sample to return. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD will be returned.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The i^{th} salinity sensor value, in raw counts, or XMSG_SENS_BAD if the index `i` exceeds the number of pressure sensor samples in the aggregated data vector.

10.60.4.21 SampleSalnAsFloat()

```
T util::xmsg::AbstractXSensorDataComposite< XDriftPressure, XDriftSalinity, XDriftTemperature,
MAX_MSGS >::SampleSalnAsFloat (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline], [inherited]
```

Get the i^{th} salinity sensor value, in PSU.

Parameters

in	*i*	The index of the sensor value. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD_SALN will be returned.
in	*gain*	The salinity sensor sensitivity scale factor, in LSB/PSU.
in	*offs*	The salinity sensor offset, in PSU.

Note

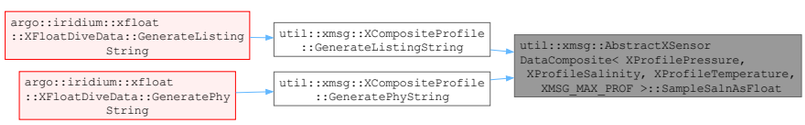
This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The i^{th} salinity sensor value, in PSU. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD_SALN will be returned.

Referenced by GenerateListingString(), and GeneratePhyString().

Here is the caller graph for this function:



10.60.4.22 SampleTemp()

```
int32_t util::xmsg::AbstractXSensorDataComposite< XDriftPressure, XDriftSalinity, XDriftTemperature,
MAX_MSGS >::SampleTemp (
            size_t i) const  [inline], [inherited]
```

Get the i^{th} temperature sensor value, in raw counts.

Parameters

in	*i*	The index of the temperature sensor sample to return. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD will be returned.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The i^{th} temperature sensor value, in raw counts, or XMSG_SENS_BAD if the index `i` exceeds the number of pressure sensor samples in the aggregated data vector.

10.60.4.23 SampleTempAsFloat()

```
T util::xmsg::AbstractXSensorDataComposite< XDriftPressure, XDriftSalinity, XDriftTemperature,
MAX_MSGS >::SampleTempAsFloat (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline], [inherited]
```

Get the i^{th} temperature sensor value, in degC.

Parameters

in	*i*	The index of the sensor value. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD_SALN will be returned.
in	*gain*	The temperature sensor sensitivity scale factor, in LSB/degC.
in	*offs*	The temperature sensor offset, in degC.

Note

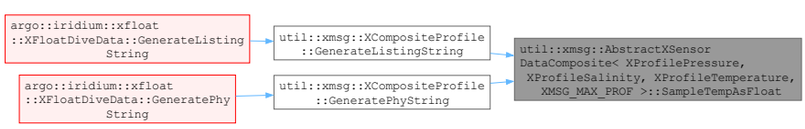
This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The i^{th} temperature sensor value, in degC. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD_TEMP will be returned.

Referenced by GenerateListingString(), and GeneratePhyString().

Here is the caller graph for this function:



10.60.4.24 set_aggregated_data_valid()

```
void util::xmsg::AbstractXSensorDataComposite< XDriftPressure, XDriftSalinity, XDriftTemperature,
MAX_MSGS >::set_aggregated_data_valid (
            bool valid)  [inline], [protected], [inherited]
```

Set the value of the aggregated_data_valid_ field.

Parameters

in	*valid*	TRUE if the aggregated data is valid, or FALSE otherwise.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

10.60.4.25 UnlockObject()

```
void util::thread::RecursiveLockable::UnlockObject ()  [inline], [inherited]
```

Unlock the object.

Exceptions

util::log::MessageException	if an error occurs.

References CATCH_BLOCK, and util::thread::RecursiveLockable::mutex_.

10.60.5 Member Data Documentation

10.60.5.1 aggregated_data_valid_

```
bool util::xmsg::AbstractXSensorDataComposite< XDriftPressure, XDriftSalinity, XDriftTemperature,
MAX_MSGS >::aggregated_data_valid_  [private], [inherited]
```

The aggregated data is valid flag.

This flag indicates the status of the aggregated sensor data. It is used to ensure the composite sensor data is aggregated only once following each assignment of sensor data to the composite data message.

This field is set to FALSE during initialization, and is set to FALSE each time a sensor data message is assigned to the composite data message. Each time AggregateSensorData() is called, the data will be aggregated, and this field will be set to TRUE.

10.60.5.2 aggregated_pres_data_

```
vector<int32_t> util::xmsg::AbstractXSensorDataComposite< XDriftPressure, XDriftSalinity, XDriftTemperature,
MAX_MSGS >::aggregated_pres_data_  [private], [inherited]
```

A vector of the aggregated pressure data samples, in raw counts.

Each time the data is aggregated, the sensor data samples from the sensor data messages assigned to this composite objects are stored in this vector.

10.60.5.3 aggregated_saln_data_

```
vector<int32_t> util::xmsg::AbstractXSensorDataComposite< XDriftPressure, XDriftSalinity, XDriftTemperature,
MAX_MSGS >::aggregated_saln_data_  [private], [inherited]
```

A vector of the aggregated salinity data samples, in raw counts.

Each time the data is aggregated, the sensor data samples from the sensor data messages assigned to this composite objects are stored in this vector.

10.60.5.4 aggregated_temp_data_

vector<int32_t> util::xmsg::AbstractXSensorDataComposite< XDriftPressure, XDriftSalinity, XDriftTemperature, MAX_MSGS >::aggregated_temp_data_ [private], [inherited]

A vector of the aggregated temperature data samples, in raw counts.

Each time the data is aggregated, the sensor data samples from the sensor data messages assigned to this composite object are stored in this vector.

10.60.5.5 mutex_

recursive_mutex util::thread::RecursiveLockable::mutex_ [mutable], [protected], [inherited]

Recursive mutual exclusion lock for synchronizing operations.

Referenced by util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AggregateS
argo::iridium::xfloat::XFloat::argo_instrument_type(), argo::iridium::xfloat::XFloatDiveData::argo_instrument_type(),
argo::iridium::xfloat::XFloatDiveData::AssignArgoData(), argo::iridium::xfloat::XFloatDiveData::AssignAuxTemperatureBinned(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignAuxTemperatureMe
argo::iridium::xfloat::XFloatDiveData::AssignDriftPressure(), argo::iridium::xfloat::XFloatDiveData::AssignDriftSalinity(),
argo::iridium::xfloat::XFloatDiveData::AssignDriftTemperatre(), argo::iridium::xfloat::XFloatDiveData::AssignEngineeringAbort(),
argo::iridium::xfloat::XFloatDiveData::AssignEngineeringAscent(), argo::iridium::xfloat::XFloatDiveData::AssignEngineeringBitFail(),
argo::iridium::xfloat::XFloatDiveData::AssignEngineeringBitPass(), argo::iridium::xfloat::XFloatDiveData::AssignEngineeringDescent(),
argo::iridium::xfloat::XFloatDiveData::AssignEngineeringDiagnostic(), argo::iridium::xfloat::XFloatDiveData::AssignEngineeringDrift(),
argo::iridium::xfloat::XFloatDiveData::AssignEngineeringIce(), argo::iridium::xfloat::XFloatDiveData::AssignEngineeringStandard(),
argo::iridium::xfloat::XFloatDiveData::AssignEngineeringSurface(), argo::iridium::xfloat::XFloatDiveData::AssignFallTimeSeries(),
argo::iridium::xfloat::XFloatDiveData::AssignGps(), argo::iridium::xfloat::XFloatDiveData::AssignMailMessage(),
argo::iridium::xfloat::XFloatDiveData::AssignMissionCmdEchoback(), argo::iridium::xfloat::XFloatDiveData::AssignParentInstrument(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignPressureMessage(
argo::iridium::xfloat::XFloatDiveData::AssignProfilePressureBinned(), argo::iridium::xfloat::XFloatDiveData::AssignProfilePressureUnbinn
argo::iridium::xfloat::XFloatDiveData::AssignProfileSalinityBinned(), argo::iridium::xfloat::XFloatDiveData::AssignProfileSalinityUnbinned()
argo::iridium::xfloat::XFloatDiveData::AssignProfileTemperatureBinned(), argo::iridium::xfloat::XFloatDiveData::AssignProfileTemperatureU
argo::iridium::xfloat::XFloatDiveData::AssignPumpTimeSeries(), argo::iridium::xfloat::XFloatDiveData::AssignRiseTimeSeries(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignSalinityMessage(),
argo::iridium::xfloat::XFloatDiveData::AssignSurfaceTimeSeries(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_C
argo::iridium::xfloat::XFloatDiveData::AssignValveData(), argo::iridium::xfloat::XFloatDiveData::CheckCompleteness(),
argo::iridium::xfloat::XFloatMetadata::comment(), argo::iridium::xfloat::XFloatDiveData::ComputePressureOffsetFromSurfaceTimeSeries(
argo::iridium::xfloat::XFloatMetadata::deployer(), argo::iridium::xfloat::XFloatMetadata::deployer_address(), argo::iridium::xfloat::XFloatMe
argo::iridium::xfloat::XFloatMetadata::deployment_datetime(), argo::iridium::xfloat::XFloatMetadata::deployment_datetime_status(),
argo::iridium::xfloat::XFloatMetadata::deployment_latitude(), argo::iridium::xfloat::XFloatMetadata::deployment_longitude(),
argo::iridium::xfloat::XFloatMetadata::deployment_platform(), argo::iridium::xfloat::XFloatMetadata::deployment_position_status(),
argo::iridium::xfloat::XFloatMetadata::deployment_type(), argo::iridium::xfloat::XFloat::fw_version(), argo::iridium::xfloat::XFloat::gain_pres
argo::iridium::xfloat::XFloat::gain_pressure_surface(), argo::iridium::xfloat::XFloat::gain_salinity(), argo::iridium::xfloat::XFloat::gain_salinit
argo::iridium::xfloat::XFloat::gain_temperature(), argo::iridium::xfloat::XFloat::gain_temperature_surface(), argo::iridium::xfloat::XFloatDive
argo::iridium::xfloat::XFloatDiveData::GenerateCompositeIridiumPosition(), argo::iridium::xfloat::XFloatDiveData::GenerateCompositePun
argo::iridium::xfloat::XFloatDiveData::GenerateCompositeRiseTimeSeriesData(), argo::iridium::xfloat::XFloatDiveData::GenerateListingSt
GenerateListingString(), util::xmsg::XCompositeHiResProfile::GenerateListingString(), util::xmsg::XCompositeProfile::GenerateListingStri
argo::iridium::xfloat::XFloatDiveData::GenerateOrderedMomsnList(), argo::iridium::xfloat::XFloatDiveData::GeneratePhyString(),
GeneratePhyString(), util::xmsg::XCompositeHiResProfile::GeneratePhyString(), util::xmsg::XCompositeProfile::GeneratePhyString(),
argo::iridium::xfloat::XFloatDiveData::GetDiveDataMailMessagesFromDatabase(), argo::iridium::xfloat::XFloatDiveData::GetGainsAndOffs
argo::iridium::xfloat::XFloatDiveData::GetGpsPositionEndOfDive(), argo::iridium::xfloat::XFloatDiveData::GetSurfaceGainsAndOffsets(),
argo::iridium::xfloat::XFloatDiveData::HasBinnedProfileData(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLAS
argo::iridium::xfloat::XFloatDiveData::HasContiguousMomsns(), argo::iridium::xfloat::XFloatDiveData::HasContiguousPids(),
argo::iridium::xfloat::XFloatDiveData::HasDiveData(), argo::iridium::xfloat::XFloatDiveData::HasDriftData(), argo::iridium::xfloat::XFloatDiv
argo::iridium::xfloat::XFloatDiveData::HasFallRiseData(), argo::iridium::xfloat::XFloatDiveData::HasPositionData(),
argo::iridium::xfloat::XFloatDiveData::HasUnbinnedProfileData(), argo::iridium::xfloat::XFloatDiveData::HasValidGpsPosition(),
argo::iridium::Iridium::id_dac(), argo::iridium::xfloat::XFloatDiveData::id_dive(), argo::iridium::xfloat::XFloatMailMessage::id_dive(),
argo::iridium::Iridium::id_imei(), argo::iridium::Iridium::id_instrument(), argo::iridium::xfloat::XFloatMailMessage::id_msg(),
argo::iridium::xfloat::XFloatMailMessage::id_msg_uid(), argo::iridium::xfloat::XFloatMailMessage::id_sbd_momsn(),
argo::iridium::xfloat::XFloatMailMessage::id_sbd_mtmsn(), argo::iridium::xfloat::XFloatMailMessage::id_sbd_packet(),
argo::iridium::Iridium::id_wmo(), argo::iridium::xfloat::XFloatDiveData::intermediate_listing_file_path(), util::xmsg::AbstractXSensorDataCo
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::IsDataComplete(),
argo::iridium::xfloat::XFloatMetadata::local_path(), util::thread::RecursiveLockable::LockObject(), argo::iridium::xfloat::XFloatMetadata::ma
argo::iridium::xfloat::XFloat::metadata(), argo::iridium::xfloat::XFloatMailMessage::msg_body_is_valid(), argo::iridium::xfloat::XFloatMailM
argo::iridium::xfloat::XFloatMailMessage::msg_header_datetime(), argo::iridium::xfloat::XFloatMailMessage::msg_header_from(),
argo::iridium::xfloat::XFloatMailMessage::msg_header_is_valid(), argo::iridium::xfloat::XFloatMailMessage::msg_header_raw_text(),
argo::iridium::xfloat::XFloatMailMessage::msg_header_subject(), argo::iridium::xfloat::XFloatMailMessage::msg_is_valid(),
argo::iridium::xfloat::XFloatMailMessage::msg_raw_text(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, T
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::num_saln_samples(),

util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::num_temp_samples(), argo::iridium::xfloat::XFloat::offset_pressure(), argo::iridium::xfloat::XFloat::offset_pressure_surface(), argo::iridium::xfloat::XFloat::offset_ argo::iridium::xfloat::XFloat::offset_salinity_surface(), argo::iridium::xfloat::XFloat::offset_temperature(), argo::iridium::xfloat::XFloat::offset argo::iridium::xfloat::XFloatMailMessage::ParseMailMsg(), argo::iridium::xfloat::XFloatMailMessage::ParseMailMsg(), argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgBody(), argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgHeader(), argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgSbdAttachment(), argo::iridium::xfloat::XFloat::phy_file_generation_is_enabled(), argo::iridium::xfloat::XFloatDiveData::phy_file_path(), argo::iridium::xfloat::XFloatMetadata::principal_investigators(), argo::iridium::xfloat::XFloatMetadata::principal_investigators_address(), argo::iridium::xfloat::XFloatMetadata::principal_investigators_orig argo::iridium::xfloat::XFloatMailMessage::raw_file_local_path(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLA argo::iridium::xfloat::XFloatDiveData::ResetData(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_ util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SamplePresAsFloat(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleSaln(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleSalnAsFloat(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleTemp(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleTempAsFloat(), argo::iridium::xfloat::XFloatDiveData::SaveObjectToDatabase(), argo::iridium::xfloat::XFloatMailMessage::SaveObjectToDatabase(), argo::iridium::xfloat::XFloatMetadata::SaveObjectToDatabase(), argo::iridium::xfloat::XFloatMailMessage::sbd_data(), argo::iridium::xfloat::XFloatMailMessage::sbd_data_is_valid(), argo::iridium::xfloat::XFloatMailMessage::sbd_file_local_path(), argo::iridium::xfloat::XFloatMailMessage::sbd_file_name(), argo::iridium::xfloat::XFloatMailMessage::sbd_file_size_bytes(), argo::iridium::xfloat::XFloatMailMessage::sbd_session_status_code(), argo::iridium::xfloat::XFloatMailMessage::sbd_session_status_strir argo::iridium::xfloat::XFloatDiveData::SensorGainsAndOffsetsAreKnown(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, argo::iridium::xfloat::XFloat::set_argo_instrument_type(), argo::iridium::xfloat::XFloatDiveData::set_argo_instrument_type(), argo::iridium::xfloat::XFloatMetadata::set_comment(), argo::iridium::xfloat::XFloatMetadata::set_deployer(), argo::iridium::xfloat::XFloatMe argo::iridium::xfloat::XFloatMetadata::set_deployment_cruise_id(), argo::iridium::xfloat::XFloatMetadata::set_deployment_datetime(), argo::iridium::xfloat::XFloatMetadata::set_deployment_datetime_status(), argo::iridium::xfloat::XFloatMetadata::set_deployment_latitude(argo::iridium::xfloat::XFloatMetadata::set_deployment_longitude(), argo::iridium::xfloat::XFloatMetadata::set_deployment_platform(), argo::iridium::xfloat::XFloatMetadata::set_deployment_position_status(), argo::iridium::xfloat::XFloatMetadata::set_deployment_type(), argo::iridium::xfloat::XFloat::set_fw_version(), argo::iridium::xfloat::XFloat::set_gain_pressure(), argo::iridium::xfloat::XFloat::set_gain_pre argo::iridium::xfloat::XFloat::set_gain_salinity(), argo::iridium::xfloat::XFloat::set_gain_salinity_surface(), argo::iridium::xfloat::XFloat::set_ argo::iridium::xfloat::XFloat::set_gain_temperature_surface(), argo::iridium::Iridium::set_id_dac(), argo::iridium::xfloat::XFloatDiveData::se argo::iridium::xfloat::XFloatMailMessage::set_id_dive(), argo::iridium::Iridium::set_id_imei(), argo::iridium::Iridium::set_id_instrument(), argo::iridium::xfloat::XFloatMailMessage::set_id_msg(), argo::iridium::xfloat::XFloatMailMessage::set_id_msg_uid(), argo::iridium::xfloat::XFloatMailMessage::set_id_sbd_momsn(), argo::iridium::xfloat::XFloatMailMessage::set_id_sbd_mtmsn(), argo::iridium::xfloat::XFloatMailMessage::set_id_sbd_packet(), argo::iridium::Iridium::set_id_wmo(), argo::iridium::xfloat::XFloatDiveData: argo::iridium::xfloat::XFloatMetadata::set_local_path(), argo::iridium::xfloat::XFloatMetadata::set_manufacturer(), argo::iridium::xfloat::XFloat::set_metadata(), argo::iridium::xfloat::XFloatMailMessage::set_msg_body_is_valid(), argo::iridium::xfloat::XFloatMailMessage::set_msg_body_text(), argo::iridium::xfloat::XFloatMailMessage::set_msg_header_datetime(), argo::iridium::xfloat::XFloatMailMessage::set_msg_header_from(), argo::iridium::xfloat::XFloatMailMessage::set_msg_header_is_valid(), argo::iridium::xfloat::XFloatMailMessage::set_msg_header_raw_text(), argo::iridium::xfloat::XFloatMailMessage::set_msg_header_subjec argo::iridium::xfloat::XFloatMailMessage::set_msg_is_valid(), argo::iridium::xfloat::XFloatMailMessage::set_msg_raw_text(), argo::iridium::xfloat::XFloat::set_offset_pressure(), argo::iridium::xfloat::XFloat::set_offset_pressure_surface(), argo::iridium::xfloat::XFloa argo::iridium::xfloat::XFloat::set_offset_salinity_surface(), argo::iridium::xfloat::XFloat::set_offset_temperature(), argo::iridium::xfloat::XFloat::set_offset_temperature_surface(), argo::iridium::xfloat::XFloat::set_phy_file_generation_is_enabled(), argo::iridium::xfloat::XFloatDiveData::set_phy_file_path(), argo::iridium::xfloat::XFloatMetadata::set_principal_investigators(), argo::iridium::xfloat::XFloatMetadata::set_principal_investigators_address(), argo::iridium::xfloat::XFloatMetadata::set_principal_investiga argo::iridium::xfloat::XFloatMailMessage::set_raw_file_local_path(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_data(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_data_is_valid(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_file_local_path(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_file_name(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_file_size_bytes(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_session_status_code(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_session_sta argo::iridium::xfloat::XFloat::set_start_of_mission_datetime(), argo::iridium::xfloat::XFloatMetadata::set_wmo_instrument_type(), argo::iridium::xfloat::XFloatMetadata::set_wmo_recorder_type(), argo::iridium::xfloat::XFloat::set_xmsg_version_major(), argo::iridium::xfloat::XFloat::set_xmsg_version_minor(), argo::iridium::xfloat::XFloat::start_of_mission_datetime(), util::thread::RecursiveLockable::UnlockObject(), argo::iridium::xfloat::XFloatDiveData::UpdateDerivedSurfaceDataFields(), argo::iridium::xfloat::XFloatMetadata::wmo_instrument_type(), argo::iridium::xfloat::XFloatMetadata::wmo_recorder_type(), argo::iridium::xfloat::XFloatDiveData::WriteListingFile(), argo::iridium::xfloat::XFloatDiveData::WritePhyFile(), argo::iridium::xfloat::XFloatM argo::iridium::xfloat::XFloat::xmsg_version_major(), and argo::iridium::xfloat::XFloat::xmsg_version_minor().

10.60.5.6 pres_messages_

```
PresMsgVector_t util::xmsg::AbstractXSensorDataComposite< XDriftPressure, XDriftSalinity, XDriftTemperature,
MAX_MSGS >::pres_messages_  [private], [inherited]
```

A static vector (of size MAX_MSGS) containing pointers the pressure sensor data messages assigned to this composite object.

Each time a pressure sensor data message is assigned to the this object, it will be emplaced in this vector in order of increasing message indices.

10.60.5.7 saln_messages_

SalnMsgVector_t util::xmsg::AbstractXSensorDataComposite< XDriftPressure, XDriftSalinity, XDriftTemperature, MAX_MSGS >::saln_messages_ [private], [inherited]

A static vector (of size MAX_MSGS) containing pointers the salinity sensor data messages assigned to this composite object.

Each time a salinity sensor data message is assigned to the this object, it will be emplaced in this vector in order of increasing message indices.

10.60.5.8 temp_messages_

TempMsgVector_t util::xmsg::AbstractXSensorDataComposite< XDriftPressure, XDriftSalinity, XDriftTemperature, MAX_MSGS >::temp_messages_ [private], [inherited]

A static vector (of size MAX_MSGS) containing pointers the temperature sensor data messages assigned to this composite object.

Each time a temperature sensor data message is assigned to the this object, it will be emplaced in this vector in order of increasing message indices.

The documentation for this class was generated from the following file:

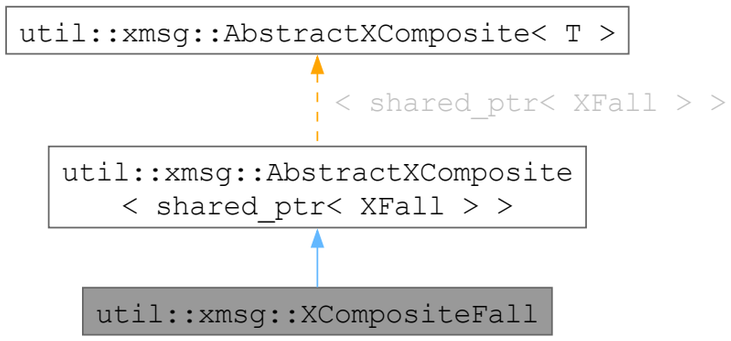
- data/xmsg/XCompositeDrift.h

10.61 util::xmsg::XCompositeFall Class Reference

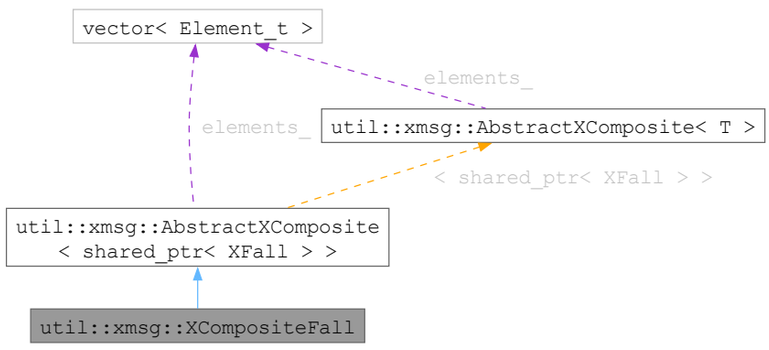
Class representing the aggregate fall time-series data for a single dive, composed from one or more fall time-series messages.

#include <XCompositeFall.h>

Inheritance diagram for util::xmsg::XCompositeFall:



Collaboration diagram for util::xmsg::XCompositeFall:



Public Member Functions

- XCompositeFall ()

  The default constructor.
- ~XCompositeFall ()

  The destructor.
- void AppendAllElements (const static_vector< Element_t, I > &elements)

  Append a vector of elements to the vector of individual X-Messages elements that compose this composite object.
- virtual void AssignElement (const Element_t &element)

  Assign an element to this composite object.
- const vector< Element_t > & elements () const

  Get the vector of individual X-Messages elements that compose this composite object.
- const string GenerateListingString (uint16_t pg, uint16_t po, uint16_t sg, uint16_t so, uint16_t tg, uint16_t to, float pc) const

  Generate a listing entry string from this composite fall time-series data object.
- virtual bool **IsEmpty** () const
- virtual size_t num_elements () const

  Get the number of individual elements that compose this composite object.

Protected Member Functions

- void AppendAllElements (const vector< Element_t > &elements)

  Append a vector of elements to the vector of individual X-Messages elements that compose this composite object.
- void AppendElement (const Element_t &element)

  Append one element to the vector of individual X-Messages elements that compose this composite object.

Private Types

- typedef shared_ptr< XFall > Element_t

  Type definition for the element aggregated by this object.
- typedef ElementVec_t::const_iterator **ElementConstIter_t**
- typedef ElementVec_t::iterator **ElementIter_t**
- typedef vector< Element_t > **ElementVec_t**

Private Attributes

- ElementVec_t elements_

  The vector of individual X-Messages elements that compose this composite object.

10.61.1 Detailed Description

Class representing the aggregate fall time-series data for a single dive, composed from one or more fall time-series messages.

10.61.2 Member Typedef Documentation

10.61.2.1 Element_t

```
typedef shared_ptr< XFall > util::xmsg::AbstractXComposite< shared_ptr< XFall > >::Element_↩
t  [private], [inherited]
```

Type definition for the element aggregated by this object.

10.61.3 Constructor & Destructor Documentation

10.61.3.1 XCompositeFall()

```
util::xmsg::XCompositeFall::XCompositeFall ()  [inline]
```

The default constructor.

10.61.3.2 ~XCompositeFall()

```
util::xmsg::XCompositeFall::~XCompositeFall ()  [inline]
```

The destructor.

10.61.4 Member Function Documentation

10.61.4.1 AppendAllElements() [1/2]

```
void util::xmsg::AbstractXComposite< shared_ptr< XFall > >::AppendAllElements (
            const static_vector< Element_t, I > & elements)  [inline], [inherited]
```

Append a vector of elements to the vector of individual X-Messages elements that compose this composite object.

Parameters

in	*elements*	The vector of elements to append.

10.61.4.2 AppendAllElements() [2/2]

```
void util::xmsg::AbstractXComposite< shared_ptr< XFall > >::AppendAllElements (
            const vector< Element_t > & elements)  [inline], [protected], [inherited]
```

Append a vector of elements to the vector of individual X-Messages elements that compose this composite object.

Parameters

in	*elements*	The vector of elements to append.

10.61.4.3 AppendElement()

```
void util::xmsg::AbstractXComposite< shared_ptr< XFall > >::AppendElement (
            const Element_t & element)  [inline], [protected], [inherited]
```

Append one element to the vector of individual X-Messages elements that compose this composite object.

Parameters

in	*element*	The element to append.

10.61.4.4 AssignElement()

```
virtual void util::xmsg::AbstractXComposite< shared_ptr< XFall > >::AssignElement (
            const Element_t & element)  [inline], [virtual], [inherited]
```

Assign an element to this composite object.

The default implementation appends the element; derived classes may override it to insert elements in order or reject duplicates.

Parameters

in	*element*	The element to assign.

10.61.4.5 elements()

```
const vector< Element_t > & util::xmsg::AbstractXComposite< shared_ptr< XFall > >::elements ()
const  [inline], [inherited]
```

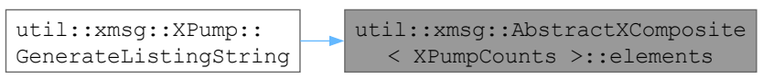
Get the vector of individual X-Messages elements that compose this composite object.

Returns

The vector of individual X-Messages elements that compose this composite object.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.61.4.6 GenerateListingString()

```
const string util::xmsg::XCompositeFall::GenerateListingString (
            uint16_t pg,
            uint16_t po,
            uint16_t sg,
            uint16_t so,
            uint16_t tg,
            uint16_t to,
            float pc) const  [inline]
```

Generate a listing entry string from this composite fall time-series data object.

Parameters

in	*pg*	The pressure sensor sensitivity scale factor, in LSB/dBar.
in	*po*	The pressure sensor offset, in dBar.
in	*sg*	The salinity sensor sensitivity scale factor, in LSB/PSU.
in	*so*	The salinity sensor offset, in PSU.
in	*tg*	The temperature sensor sensitivity scale factor, in LSB/degC.
in	*to*	The temperature sensor offset, in degC.
in	*pc*	The pressure correction, in dBar, to be subtracted computed pressure value.

Exceptions

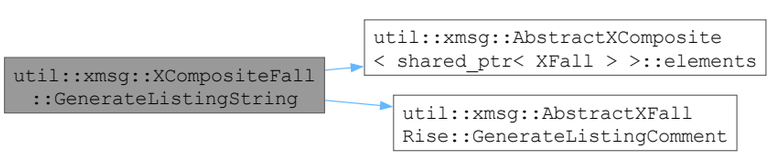
log::MessageException	if an error occurs while generating the listing string.

Returns

A listing entry string representation of this composite fall time-series data object.

References CATCH_BLOCK, util::xmsg::AbstractXComposite< shared_ptr< XFall > >::elements(), util::xmsg::AbstractXFallRise::Gener
and XMSG_SECTION_BREAK.

Here is the call graph for this function:



10.61.4.7 num_elements()

```
virtual size_t util::xmsg::AbstractXComposite< shared_ptr< XFall > >::num_elements () const
[inline], [virtual], [inherited]
```

Get the number of individual elements that compose this composite object.

Returns

The number of individual elements that compose this composite object.

10.61.5 Member Data Documentation

10.61.5.1 elements_

```
ElementVec_t util::xmsg::AbstractXComposite< shared_ptr< XFall > >::elements_  [private], [inherited]
```

The vector of individual X-Messages elements that compose this composite object.

The documentation for this class was generated from the following file:

- data/xmsg/XCompositeFall.h

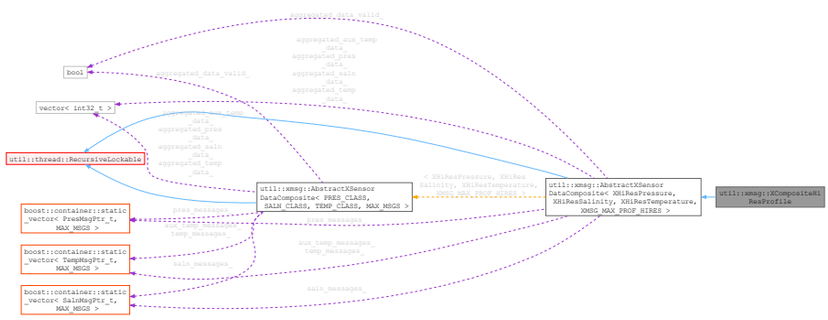
10.62 util::xmsg::XCompositeHiResProfile Class Reference

Class representing the aggregate raw/high-resolution/unbinned profile data for a single dive, composed from one or more raw profile data messages.

```
#include <XCompositeHiResProfile.h>
```

Inheritance diagram for util::xmsg::XCompositeHiResProfile:

Collaboration diagram for util::xmsg::XCompositeHiResProfile:



Public Member Functions

- XCompositeHiResProfile ()

  The default constructor.
- ~XCompositeHiResProfile ()

  The destructor.
- void AssignAuxTemperatureMessage (const TempMsgPtr_t &x)

  Assign an auxiliary-channel temperature message and re-aggregate the sensor data.
- void AssignPressureMessage (const PresMsgPtr_t &x)

  Assign a pressure data message to this composite data object.
- void AssignSalinityMessage (const SalnMsgPtr_t &x)

  Assign a salinity data message to this composite data object.
- void AssignTemperatureMessage (const TempMsgPtr_t &x)

  Assign a temperature data message to this composite data object.
- void **CopyPressureDataToArray** (T dest[ ], size_t max_len, int16_t gain, int16_t offs, float pres_correction, T fill_value=XMSG_SENS_BAD_PRES) const
- void **CopySalinityDataToArray** (T dest[ ], size_t max_len, int16_t gain, int16_t offs, T fill_value=XMSG_SENS_BAD_SALN) const
- void **CopyTemperatureDataToArray** (T dest[ ], size_t max_len, int16_t gain, int16_t offs, T fill_↩ value=XMSG_SENS_BAD_TEMP) const
- const string GenerateListingString (int id_inst, int id_dive, uint16_t pg, uint16_t po, uint16_t sg, uint16_t so, uint16_t tg, uint16_t to, float pc) const

  Generate a listing string representation of the this composite sensor data object.
- const string GeneratePhyString (uint16_t pg, uint16_t po, uint16_t sg, uint16_t so, uint16_t tg, uint16_t to, float pc) const

  Generate a PHY entry string from this composite bin-averaged profile data object.
- bool **HasAuxTempData** () const
- bool HasContiguousMessageIndices () const

  Do the sensor data messages assigned to this composite data object have contiguous message indices?
- bool is_aggregated_data_valid () const

  Is the aggregated data valid?
- bool IsDataComplete () const

  Is the sensor data for this composite date message complete?
- bool IsEmpty () const

  Is this composite data object empty?
- bool LockObject (bool block)

  Lock the object.
- size_t max_num_samples () const

  Return the maximum of the number of pressure, temperature, or salinity sensor samples in the sensor data messages assigned to this composite sensor data object.
- size_t **num_aux_temp_samples** () const
- size_t num_pres_samples () const

  Get the total number of pressure sensor samples contained in the pressure data messages assigned to this composite data object.
- size_t num_saln_samples () const

  Get the total number of salinity sensor samples contained in the salinity data messages assigned to this composite data object.
- size_t num_temp_samples () const

  Get the total number of temperature sensor samples contained in the temperature data messages assigned to this composite data object.
- void Reset ()

  Remove the assigned pressure, temperature and salinity messages and their aggregated data.
- T **SampleAuxTempAsFloat** (size_t i, int16_t gain, int16_t offs) const
- int32_t SamplePres (size_t i) const

  Get the i^{th} pressure sensor value, in raw counts.
- T SamplePresAsFloat (size_t i, int16_t pg, int16_t po, float pc) const

  Get the i^{th} pressure sensor value, in dBar.
- int32_t SampleSaln (size_t i) const

  Get the i^{th} salinity sensor value, in raw counts.
- T SampleSalnAsFloat (size_t i, int16_t gain, int16_t offs) const

  Get the i^{th} salinity sensor value, in PSU.
- int32_t SampleTemp (size_t i) const

  Get the i^{th} temperature sensor value, in raw counts.
- T SampleTempAsFloat (size_t i, int16_t gain, int16_t offs) const

  Get the i^{th} temperature sensor value, in degC.
- void UnlockObject ()

  Unlock the object.

Protected Member Functions

- void AggregateSensorData () ATTR_NO_EXCEPTIONS

  Aggregate the sensor data for this composite sensor data object.
- void set_aggregated_data_valid (bool valid)

  Set the value of the aggregated_data_valid_ field.

Protected Attributes

- recursive_mutex mutex_

  Recursive mutual exclusion lock for synchronizing operations.

Private Types

- typedef shared_ptr< XHiResPressure > PresMsgPtr_t

  Type definition for a shared pointer to the pressure data message.
- typedef static_vector< PresMsgPtr_t, MAX_MSGS > PresMsgVector_t

  Type definition for a static vector (of size MAX_MSGS) of shared pointers to the pressure data message.
- typedef shared_ptr< XHiResSalinity > SalnMsgPtr_t

  Type definition for a shared pointer to the salinity data message.
- typedef static_vector< SalnMsgPtr_t, MAX_MSGS > SalnMsgVector_t

  Type definition for a static vector (of size MAX_MSGS) of shared pointers to the salinity data message.
- typedef shared_ptr< XHiResTemperature > TempMsgPtr_t

  Type definition for a shared pointer to the temperature data message.
- typedef static_vector< TempMsgPtr_t, MAX_MSGS > TempMsgVector_t

  Type definition for a static vector (of size MAX_MSGS) of shared pointers to the temperature data message.

Private Attributes

- vector< int32_t > **aggregated_aux_temp_data_**
- bool aggregated_data_valid_

  The aggregated data is valid flag.
- vector< int32_t > aggregated_pres_data_

  A vector of the aggregated pressure data samples, in raw counts.
- vector< int32_t > aggregated_saln_data_

  A vector of the aggregated salinity data samples, in raw counts.
- vector< int32_t > aggregated_temp_data_

  A vector of the aggregated temperature data samples, in raw counts.
- TempMsgVector_t **aux_temp_messages_**
- PresMsgVector_t pres_messages_

  A static vector (of size MAX_MSGS) containing pointers the pressure sensor data messages assigned to this composite object.
- SalnMsgVector_t saln_messages_

  A static vector (of size MAX_MSGS) containing pointers the salinity sensor data messages assigned to this composite object.
- TempMsgVector_t temp_messages_

  A static vector (of size MAX_MSGS) containing pointers the temperature sensor data messages assigned to this composite object.

10.62.1 Detailed Description

Class representing the aggregate raw/high-resolution/unbinned profile data for a single dive, composed from one or more raw profile data messages.

10.62.2 Member Typedef Documentation

10.62.2.1 PresMsgPtr_t

```
typedef shared_ptr<XHiResPressure> util::xmsg::AbstractXSensorDataComposite< XHiResPressure,
XHiResSalinity, XHiResTemperature, MAX_MSGS >::PresMsgPtr_t  [private], [inherited]
```

Type definition for a shared pointer to the pressure data message.

10.62.2.2 PresMsgVector_t

typedef static_vector<PresMsgPtr_t, MAX_MSGS> util::xmsg::AbstractXSensorDataComposite< XHiResPressure, XHiResSalinity, XHiResTemperature, MAX_MSGS >::PresMsgVector_t [private], [inherited]

Type definition for a static vector (of size MAX_MSGS) of shared pointers to the pressure data message.

10.62.2.3 SalnMsgPtr_t

typedef shared_ptr<XHiResSalinity> util::xmsg::AbstractXSensorDataComposite< XHiResPressure, XHiResSalinity, XHiResTemperature, MAX_MSGS >::SalnMsgPtr_t [private], [inherited]

Type definition for a shared pointer to the salinity data message.

10.62.2.4 SalnMsgVector_t

typedef static_vector<SalnMsgPtr_t, MAX_MSGS> util::xmsg::AbstractXSensorDataComposite< XHiResPressure, XHiResSalinity, XHiResTemperature, MAX_MSGS >::SalnMsgVector_t [private], [inherited]

Type definition for a static vector (of size MAX_MSGS) of shared pointers to the salinity data message.

10.62.2.5 TempMsgPtr_t

typedef shared_ptr<XHiResTemperature> util::xmsg::AbstractXSensorDataComposite< XHiResPressure, XHiResSalinity, XHiResTemperature, MAX_MSGS >::TempMsgPtr_t [private], [inherited]

Type definition for a shared pointer to the temperature data message.

10.62.2.6 TempMsgVector_t

typedef static_vector<TempMsgPtr_t, MAX_MSGS> util::xmsg::AbstractXSensorDataComposite< XHiResPressure, XHiResSalinity, XHiResTemperature, MAX_MSGS >::TempMsgVector_t [private], [inherited]

Type definition for a static vector (of size MAX_MSGS) of shared pointers to the temperature data message.

10.62.3 Constructor & Destructor Documentation

10.62.3.1 XCompositeHiResProfile()

util::xmsg::XCompositeHiResProfile::XCompositeHiResProfile () [inline]

The default constructor.

10.62.3.2 ~XCompositeHiResProfile()

util::xmsg::XCompositeHiResProfile::~XCompositeHiResProfile () [inline]

The destructor.

10.62.4 Member Function Documentation

10.62.4.1 AggregateSensorData()

void util::xmsg::AbstractXSensorDataComposite< XHiResPressure, XHiResSalinity, XHiResTemperature, MAX_MSGS >::AggregateSensorData () [inline], [protected], [inherited]

Aggregate the sensor data for this composite sensor data object.

This function iterates over each of the sensor data messages assigned to this composite data object, and copies of the sensor data values into the corresponding sensor data vector.

If aggregated_data_valid_ is TRUE, then this function will have no effect. Otherwise, each of the aggregegated sensor data vectors will be cleared and the appropriate number of samples will be reserved.

Then, the sensor data message will be iterated over in order of increasing message indices, and the sensor data values will be copied into the corresponding sensor data vector in the appropriate order.

Finally, aggregated_data_valid_ will be set to FALSE to ensure that subsequent calls to this function will have no effect, until another sensor data message object is assigned to this composite object and aggregated_data_valid_ is set to TRUE.

Exceptions

util::log::MessageException	if an error occurs while aggregating the sensor data.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

10.62.4.2 AssignAuxTemperatureMessage()

```
void util::xmsg::AbstractXSensorDataComposite< XHiResPressure, XHiResSalinity, XHiResTemperature,
MAX_MSGS >::AssignAuxTemperatureMessage (
            const TempMsgPtr_t & x)  [inline], [inherited]
```

Assign an auxiliary-channel temperature message and re-aggregate the sensor data.

The message is inserted in order of message index, and the aggregated sensor data is rebuilt.

Parameters

in	*x*	The auxiliary-channel temperature message to assign.

Exceptions

util::log::MessageException	if x is NULL or an auxiliary temperature message with the same index has already been assigned.

10.62.4.3 AssignPressureMessage()

```
void util::xmsg::AbstractXSensorDataComposite< XHiResPressure, XHiResSalinity, XHiResTemperature,
MAX_MSGS >::AssignPressureMessage (
            const PresMsgPtr_t & x)  [inline], [inherited]
```

Assign a pressure data message to this composite data object.

The pressure data message will be emplaced into the sensor data message vector in order of increasing message indices. If a message with the same index is as x is already contained in the vector, then an exception will be thrown.

Parameters

in	*x*	A shared pointer to the pressure data message.

Exceptions

util::log::MessageException	if x is a null pointer, a duplicate message index is encountered, or any other error occurs.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

10.62.4.4 AssignSalinityMessage()

```
void util::xmsg::AbstractXSensorDataComposite< XHiResPressure, XHiResSalinity, XHiResTemperature,
MAX_MSGS >::AssignSalinityMessage (
            const SalnMsgPtr_t & x)  [inline], [inherited]
```

Assign a salinity data message to this composite data object.

The salinity data message will be emplaced into the sensor data message vector in order of increasing message indices. If a message with the same index is as x is already contained in the vector, then an exception will be thrown.

Parameters

in	*x*	A shared pointer to the salinity data message.

Exceptions

util::log::MessageException	if x is a null pointer, a duplicate message index is encountered, or any other error occurs.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

10.62.4.5 AssignTemperatureMessage()

```
void util::xmsg::AbstractXSensorDataComposite< XHiResPressure, XHiResSalinity, XHiResTemperature,
MAX_MSGS >::AssignTemperatureMessage (
            const TempMsgPtr_t & x)  [inline], [inherited]
```

Assign a temperature data message to this composite data object.

The temperature data message will be emplaced into the sensor data message vector in order of increasing message indices. If a message with the same index is as x is already contained in the vector, then an exception will be thrown.

Parameters

in	*x*	A shared pointer to the temperature data message.

Exceptions

util::log::MessageException	if x is a null pointer, a duplicate message index is encountered, or any other error occurs.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

10.62.4.6 GenerateListingString()

```
const string util::xmsg::XCompositeHiResProfile::GenerateListingString (
            int id_inst,
            int id_dive,
            uint16_t pg,
            uint16_t po,
            uint16_t sg,
            uint16_t so,
            uint16_t tg,
            uint16_t to,
            float pc) const  [inline], [virtual]
```

Generate a listing string representation of the this composite sensor data object.

Parameters

in	*id_inst*	The unique instrument identifier for this composite sensor data object.
in	*id_dive*	The dive or cycle number for this composite data object.
in	*pg*	The pressure sensor sensitivity scale factor, in LSB/dBar.
in	*po*	The pressure sensor offset, in dBar.
in	*sg*	The salinity sensor sensitivity scale factor, in LSB/PSU.
in	*so*	The salinity sensor offset, in PSU.
in	*tg*	The temperature sensor sensitivity scale factor, in LSB/degC.
in	*to*	The temperature sensor offset, in degC.
in	*pc*	The pressure correction, in dBar, to be subtracted computed pressure value.

Exceptions

log::MessageException	if an error occurs while generating the listing string.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

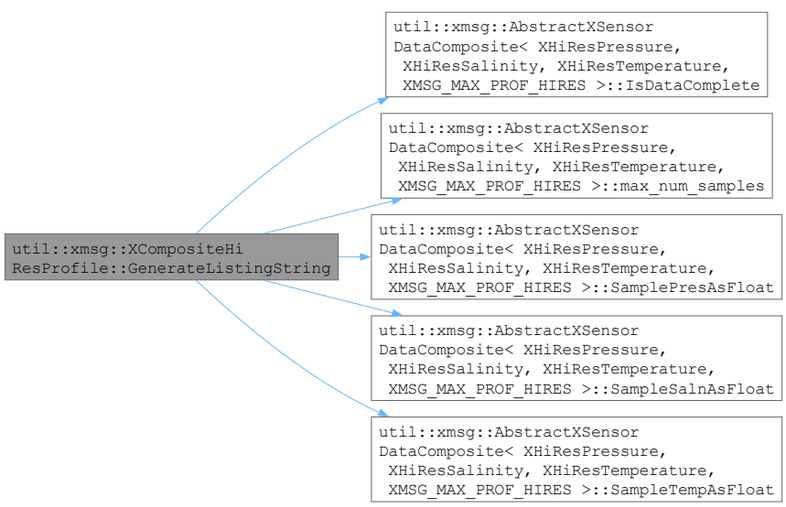
A listing string representation of this composite sensor data object.

Implements util::xmsg::AbstractXSensorDataComposite< XHiResPressure, XHiResSalinity, XHiResTemperature, XMSG_MAX_PROF_H

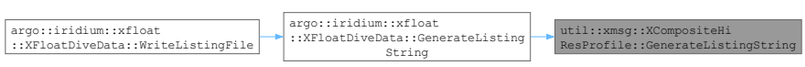
References CATCH_BLOCK, util::xmsg::AbstractXSensorDataComposite< XHiResPressure, XHiResSalinity, XHiResTemperature, XMS
util::xmsg::AbstractXSensorDataComposite< XHiResPressure, XHiResSalinity, XHiResTemperature, XMSG_MAX_PROF_HIRES >::ma
util::thread::RecursiveLockable::mutex_, util::xmsg::AbstractXSensorDataComposite< XHiResPressure, XHiResSalinity, XHiResTempera
util::xmsg::AbstractXSensorDataComposite< XHiResPressure, XHiResSalinity, XHiResTemperature, XMSG_MAX_PROF_HIRES >::Sa
and util::xmsg::AbstractXSensorDataComposite< XHiResPressure, XHiResSalinity, XHiResTemperature, XMSG_MAX_PROF_HIRES >

Referenced by argo::iridium::xfloat::XFloatDiveData::GenerateListingString().

Here is the call graph for this function:



Here is the caller graph for this function:



10.62.4.7 GeneratePhyString()

```
const string util::xmsg::XCompositeHiResProfile::GeneratePhyString (
            uint16_t pg,
            uint16_t po,
            uint16_t sg,
            uint16_t so,
            uint16_t tg,
            uint16_t to,
            float pc) const  [inline]
```

Generate a PHY entry string from this composite bin-averaged profile data object.

Parameters

in	*pg*	The pressure sensor sensitivity scale factor, in LSB/dBar.
in	*po*	The pressure sensor offset, in dBar.
in	*sg*	The salinity sensor sensitivity scale factor, in LSB/PSU.
in	*so*	The salinity sensor offset, in PSU.
in	*tg*	The temperature sensor sensitivity scale factor, in LSB/degC.
in	*to*	The temperature sensor offset, in degC.
in	*pc*	The pressure correction, in dBar, to be subtracted computed pressure value.

Exceptions

log::MessageException	if an error occurs while generating the PHY string.

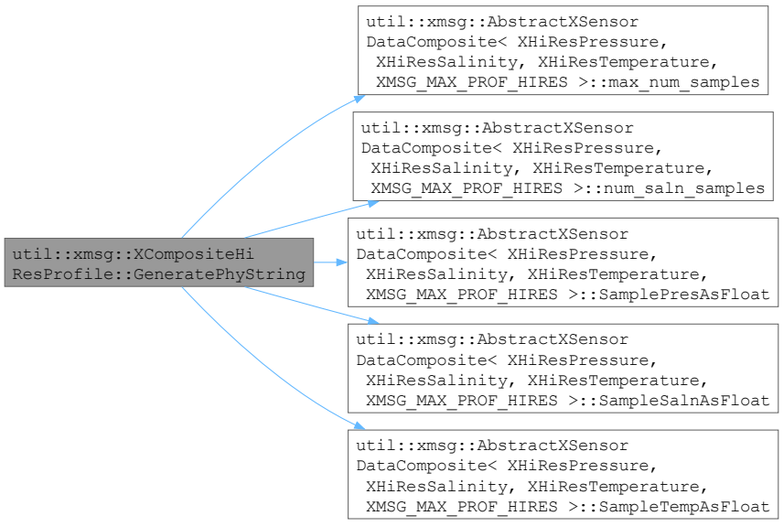
Returns

A PHY entry string representation of this composite raw profile data object.

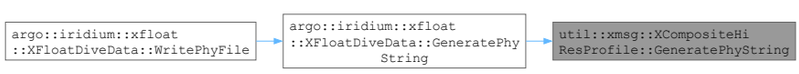
References CATCH_BLOCK, util::xmsg::AbstractXSensorDataComposite< XHiResPressure, XHiResSalinity, XHiResTemperature, XMSG
util::thread::RecursiveLockable::mutex_, util::xmsg::AbstractXSensorDataComposite< XHiResPressure, XHiResSalinity, XHiResTempera
util::xmsg::AbstractXSensorDataComposite< XHiResPressure, XHiResSalinity, XHiResTemperature, XMSG_MAX_PROF_HIRES >::Sa
util::xmsg::AbstractXSensorDataComposite< XHiResPressure, XHiResSalinity, XHiResTemperature, XMSG_MAX_PROF_HIRES >::Sa
and util::xmsg::AbstractXSensorDataComposite< XHiResPressure, XHiResSalinity, XHiResTemperature, XMSG_MAX_PROF_HIRES >

Referenced by argo::iridium::xfloat::XFloatDiveData::GeneratePhyString().

Here is the call graph for this function:



Here is the caller graph for this function:



10.62.4.8 HasContiguousMessageIndices()

```
bool util::xmsg::AbstractXSensorDataComposite< XHiResPressure, XHiResSalinity, XHiResTemperature,
MAX_MSGS >::HasContiguousMessageIndices () const  [inline], [inherited]
```

Do the sensor data messages assigned to this composite data object have contiguous message indices?

This function is used, in part, to determine the sensor data is complete.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

TRUE if the sensor data messages have contiguous message indices or if there is less than two of each sensor data message, or FALSE otherwise.

10.62.4.9 is_aggregated_data_valid()

`bool util::xmsg::AbstractXSensorDataComposite< XHiResPressure, XHiResSalinity, XHiResTemperature, MAX_MSGS >::is_aggregated_data_valid () const [inline], [inherited]`

Is the aggregated data valid?

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

TRUE if the aggregated sensor is valid, or FALSE otherwise.

10.62.4.10 IsDataComplete()

`bool util::xmsg::AbstractXSensorDataComposite< XHiResPressure, XHiResSalinity, XHiResTemperature, MAX_MSGS >::IsDataComplete () const [inline], [inherited]`

Is the sensor data for this composite date message complete?

This function returns true if

- this composite sensor data object is not empty,
- the sensor data messages assigned to this composite data object have contiguous message indices,
- this sensor data object contains the same number of temperature and pressure sensor values, and
- the number of salinity data samples is either zero or equal to the number of pressure data samples.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

TRUE if the sensor data is complete, or FALSE otherwise.

Referenced by GenerateListingString().

Here is the caller graph for this function:

10.62.4.11 IsEmpty()

bool util::xmsg::AbstractXSensorDataComposite< XHiResPressure, XHiResSalinity, XHiResTemperature, MAX_MSGS >::IsEmpty () const [inline], [inherited]

Is this composite data object empty?

Returns

TRUE if empty, or FALSE otherwise.

10.62.4.12 LockObject()

bool util::thread::RecursiveLockable::LockObject (bool *block*) [inline], [inherited]

Lock the object.

Parameters

in	*block*	If TRUE, then the calling thread will block until the lock becomes available.

Exceptions

util::log::MessageException	if an error occurs.

Returns

TRUE if the lock was successfully obtained, or FALSE if block is FALSE and another thread owns the lock on the mutex.

References CATCH_BLOCK, and util::thread::RecursiveLockable::mutex_.

10.62.4.13 max_num_samples()

size_t util::xmsg::AbstractXSensorDataComposite< XHiResPressure, XHiResSalinity, XHiResTemperature, MAX_MSGS >::max_num_samples () const [inline], [inherited]

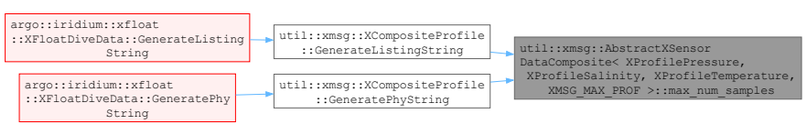
Return the maximum of the number of pressure, temperature, or salinity sensor samples in the sensor data messages assigned to this composite sensor data object.

Returns

The maximum of the number of pressure, temperature, or salinity sensor samples.

Referenced by GenerateListingString(), and GeneratePhyString().

Here is the caller graph for this function:



10.62.4.14 num_pres_samples()

size_t util::xmsg::AbstractXSensorDataComposite< XHiResPressure, XHiResSalinity, XHiResTemperature, MAX_MSGS >::num_pres_samples () const [inline], [inherited]

Get the total number of pressure sensor samples contained in the pressure data messages assigned to this composite data object.

Exceptions

util::log::MessageException	if an error occurs.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The total number of pressure sensor samples contained in the pressure data messages assigned to this composite data object.

10.62.4.15 num_saln_samples()

`size_t util::xmsg::AbstractXSensorDataComposite< XHiResPressure, XHiResSalinity, XHiResTemperature, MAX_MSGS >::num_saln_samples () const [inline], [inherited]`

Get the total number of salinity sensor samples contained in the salinity data messages assigned to this composite data object.

Exceptions

util::log::MessageException	if an error occurs.

Note

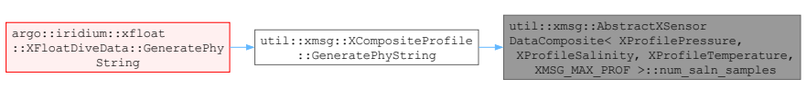
This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The total number of pressure sensor samples contained in the pressure data messages assigned to this composite data object.

Referenced by GeneratePhyString().

Here is the caller graph for this function:



10.62.4.16 num_temp_samples()

`size_t util::xmsg::AbstractXSensorDataComposite< XHiResPressure, XHiResSalinity, XHiResTemperature, MAX_MSGS >::num_temp_samples () const [inline], [inherited]`

Get the total number of temperature sensor samples contained in the temperature data messages assigned to this composite data object.

Exceptions

util::log::MessageException	if an error occurs.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The total number of pressure sensor samples contained in the pressure data messages assigned to this composite data object.

10.62.4.17 Reset()

```
void util::xmsg::AbstractXSensorDataComposite< XHiResPressure, XHiResSalinity, XHiResTemperature,
MAX_MSGS >::Reset ()  [inline], [inherited]
```

Remove the assigned pressure, temperature and salinity messages and their aggregated data.

Warning

The auxiliary temperature messages and aggregated auxiliary temperature data are not cleared, so re-assigning them after a reset fails with a duplicate index.

Exceptions

util::log::MessageException	if an error occurs.

10.62.4.18 SamplePres()

```
int32_t util::xmsg::AbstractXSensorDataComposite< XHiResPressure, XHiResSalinity, XHiResTemperature,
MAX_MSGS >::SamplePres (
            size_t i) const  [inline], [inherited]
```

Get the i^{th} pressure sensor value, in raw counts.

Parameters

in	*i*	The index of the pressure sensor sample to return. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD will be returned.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The i^{th} pressure sensor value, in raw counts, or XMSG_SENS_BAD if the index `i` exceeds the number of pressure sensor samples in the aggregated data vector.

10.62.4.19 SamplePresAsFloat()

```
T util::xmsg::AbstractXSensorDataComposite< XHiResPressure, XHiResSalinity, XHiResTemperature,
MAX_MSGS >::SamplePresAsFloat (
            size_t i,
            int16_t pg,
            int16_t po,
            float pc) const  [inline], [inherited]
```

Get the i^{th} pressure sensor value, in dBar.

Parameters

in	*i*	The index of the sensor value. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD_PRES will be returned.
in	*pg*	The pressure sensor sensitivity scale factor, in LSB/dBar.
in	*po*	The pressure sensor offset, in dBar.
in	*pc*	The pressure correction, in dBar, to be subtracted computed pressure value.

Note

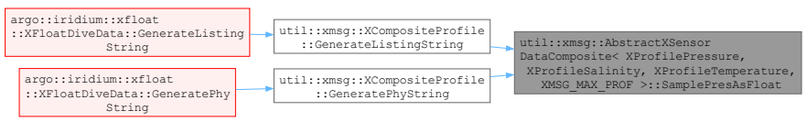
This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The i^{th} pressure sensor value, in dBar. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD_PRES will be returned.

Referenced by GenerateListingString(), and GeneratePhyString().

Here is the caller graph for this function:



10.62.4.20 SampleSaln()

```
int32_t util::xmsg::AbstractXSensorDataComposite< XHiResPressure, XHiResSalinity, XHiResTemperature,
MAX_MSGS >::SampleSaln (
            size_t i) const  [inline], [inherited]
```

Get the i^{th} salinity sensor value, in raw counts.

Parameters

in	*i*	The index of the salinity sensor sample to return. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD will be returned.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The i^{th} salinity sensor value, in raw counts, or XMSG_SENS_BAD if the index `i` exceeds the number of pressure sensor samples in the aggregated data vector.

10.62.4.21 SampleSalnAsFloat()

```
T util::xmsg::AbstractXSensorDataComposite< XHiResPressure, XHiResSalinity, XHiResTemperature,
MAX_MSGS >::SampleSalnAsFloat (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline], [inherited]
```

Get the i^{th} salinity sensor value, in PSU.

Parameters

in	*i*	The index of the sensor value. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD_SALN will be returned.
in	*gain*	The salinity sensor sensitivity scale factor, in LSB/PSU.
in	*offs*	The salinity sensor offset, in PSU.

Note

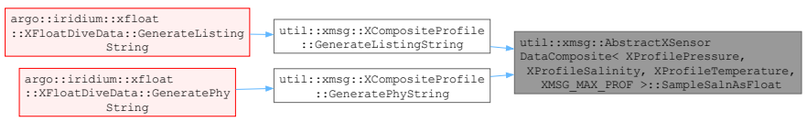
This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The i^{th} salinity sensor value, in PSU. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD_SALN will be returned.

Referenced by GenerateListingString(), and GeneratePhyString().

Here is the caller graph for this function:



10.62.4.22 SampleTemp()

```
int32_t util::xmsg::AbstractXSensorDataComposite< XHiResPressure, XHiResSalinity, XHiResTemperature,
MAX_MSGS >::SampleTemp (
            size_t i) const  [inline], [inherited]
```

Get the i^{th} temperature sensor value, in raw counts.

Parameters

in	*i*	The index of the temperature sensor sample to return. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD will be returned.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The i^{th} temperature sensor value, in raw counts, or XMSG_SENS_BAD if the index `i` exceeds the number of pressure sensor samples in the aggregated data vector.

10.62.4.23 SampleTempAsFloat()

```
T util::xmsg::AbstractXSensorDataComposite< XHiResPressure, XHiResSalinity, XHiResTemperature,
MAX_MSGS >::SampleTempAsFloat (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline], [inherited]
```

Get the i^{th} temperature sensor value, in degC.

Parameters

in	*i*	The index of the sensor value. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD_SALN will be returned.
in	*gain*	The temperature sensor sensitivity scale factor, in LSB/degC.
in	*offs*	The temperature sensor offset, in degC.

Note

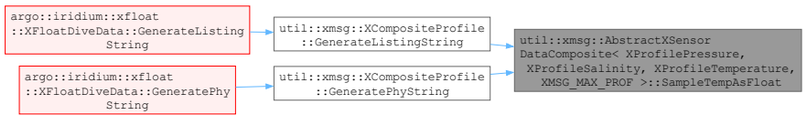
This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The i^{th} temperature sensor value, in degC. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD_TEMP will be returned.

Referenced by GenerateListingString(), and GeneratePhyString().

Here is the caller graph for this function:



10.62.4.24 set_aggregated_data_valid()

```
void util::xmsg::AbstractXSensorDataComposite< XHiResPressure, XHiResSalinity, XHiResTemperature,
MAX_MSGS >::set_aggregated_data_valid (
            bool valid)  [inline], [protected], [inherited]
```

Set the value of the aggregated_data_valid_ field.

Parameters

in	*valid*	TRUE if the aggregated data is valid, or FALSE otherwise.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

10.62.4.25 UnlockObject()

```
void util::thread::RecursiveLockable::UnlockObject ()  [inline], [inherited]
```

Unlock the object.

Exceptions

util::log::MessageException	if an error occurs.

References CATCH_BLOCK, and util::thread::RecursiveLockable::mutex_.

10.62.5 Member Data Documentation

10.62.5.1 aggregated_data_valid_

```
bool util::xmsg::AbstractXSensorDataComposite< XHiResPressure, XHiResSalinity, XHiResTemperature,
MAX_MSGS >::aggregated_data_valid_  [private], [inherited]
```

The aggregated data is valid flag.

This flag indicates the status of the aggregated sensor data. It is used to ensure the composite sensor data is aggregated only once following each assignment of sensor data to the composite data message.

This field is set to FALSE during initialization, and is set to FALSE each time a sensor data message is assigned to the composite data message. Each time AggregateSensorData() is called, the data will be aggregated, and this field will be set to TRUE.

10.62.5.2 aggregated_pres_data_

vector<int32_t> util::xmsg::AbstractXSensorDataComposite< XHiResPressure, XHiResSalinity, XHiResTemperature, MAX_MSGS >::aggregated_pres_data_ [private], [inherited]

A vector of the aggregated pressure data samples, in raw counts.

Each time the data is aggregated, the sensor data samples from the sensor data messages assigned to this composite objects are stored in this vector.

10.62.5.3 aggregated_saln_data_

vector<int32_t> util::xmsg::AbstractXSensorDataComposite< XHiResPressure, XHiResSalinity, XHiResTemperature, MAX_MSGS >::aggregated_saln_data_ [private], [inherited]

A vector of the aggregated salinity data samples, in raw counts.

Each time the data is aggregated, the sensor data samples from the sensor data messages assigned to this composite objects are stored in this vector.

10.62.5.4 aggregated_temp_data_

vector<int32_t> util::xmsg::AbstractXSensorDataComposite< XHiResPressure, XHiResSalinity, XHiResTemperature, MAX_MSGS >::aggregated_temp_data_ [private], [inherited]

A vector of the aggregated temperature data samples, in raw counts.

Each time the data is aggregated, the sensor data samples from the sensor data messages assigned to this composite object are stored in this vector.

10.62.5.5 mutex_

recursive_mutex util::thread::RecursiveLockable::mutex_ [mutable], [protected], [inherited]

Recursive mutual exclusion lock for synchronizing operations.

Referenced by util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AggregateS
argo::iridium::xfloat::XFloat::argo_instrument_type(), argo::iridium::xfloat::XFloatDiveData::argo_instrument_type(),
argo::iridium::xfloat::XFloatDiveData::AssignArgoData(), argo::iridium::xfloat::XFloatDiveData::AssignAuxTemperatureBinned(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignAuxTemperatureMe
argo::iridium::xfloat::XFloatDiveData::AssignDriftPressure(), argo::iridium::xfloat::XFloatDiveData::AssignDriftSalinity(),
argo::iridium::xfloat::XFloatDiveData::AssignDriftTemperatre(), argo::iridium::xfloat::XFloatDiveData::AssignEngineeringAbort(),
argo::iridium::xfloat::XFloatDiveData::AssignEngineeringAscent(), argo::iridium::xfloat::XFloatDiveData::AssignEngineeringBitFail(),
argo::iridium::xfloat::XFloatDiveData::AssignEngineeringBitPass(), argo::iridium::xfloat::XFloatDiveData::AssignEngineeringDescent(),
argo::iridium::xfloat::XFloatDiveData::AssignEngineeringDiagnostic(), argo::iridium::xfloat::XFloatDiveData::AssignEngineeringDrift(),
argo::iridium::xfloat::XFloatDiveData::AssignEngineeringIce(), argo::iridium::xfloat::XFloatDiveData::AssignEngineeringStandard(),
argo::iridium::xfloat::XFloatDiveData::AssignEngineeringSurface(), argo::iridium::xfloat::XFloatDiveData::AssignFallTimeSeries(),
argo::iridium::xfloat::XFloatDiveData::AssignGps(), argo::iridium::xfloat::XFloatDiveData::AssignMailMessage(),
argo::iridium::xfloat::XFloatDiveData::AssignMissionCmdEchoback(), argo::iridium::xfloat::XFloatDiveData::AssignParentInstrument(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignPressureMessage(
argo::iridium::xfloat::XFloatDiveData::AssignProfilePressureBinned(), argo::iridium::xfloat::XFloatDiveData::AssignProfilePressureUnbinn
argo::iridium::xfloat::XFloatDiveData::AssignProfileSalinityBinned(), argo::iridium::xfloat::XFloatDiveData::AssignProfileSalinityUnbinned()
argo::iridium::xfloat::XFloatDiveData::AssignProfileTemperatureBinned(), argo::iridium::xfloat::XFloatDiveData::AssignProfileTemperature
argo::iridium::xfloat::XFloatDiveData::AssignPumpTimeSeries(), argo::iridium::xfloat::XFloatDiveData::AssignRiseTimeSeries(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignSalinityMessage(),
argo::iridium::xfloat::XFloatDiveData::AssignSurfaceTimeSeries(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_C
argo::iridium::xfloat::XFloatDiveData::AssignValveData(), argo::iridium::xfloat::XFloatDiveData::CheckCompleteness(),
argo::iridium::xfloat::XFloatMetadata::comment(), argo::iridium::xfloat::XFloatDiveData::ComputePressureOffsetFromSurfaceTimeSeries(
argo::iridium::xfloat::XFloatMetadata::deployer(), argo::iridium::xfloat::XFloatMetadata::deployer_address(), argo::iridium::xfloat::XFloatMe
argo::iridium::xfloat::XFloatMetadata::deployment_datetime(), argo::iridium::xfloat::XFloatMetadata::deployment_datetime_status(),
argo::iridium::xfloat::XFloatMetadata::deployment_latitude(), argo::iridium::xfloat::XFloatMetadata::deployment_longitude(),
argo::iridium::xfloat::XFloatMetadata::deployment_platform(), argo::iridium::xfloat::XFloatMetadata::deployment_position_status(),
argo::iridium::xfloat::XFloatMetadata::deployment_type(), argo::iridium::xfloat::XFloat::fw_version(), argo::iridium::xfloat::XFloat::gain_pres
argo::iridium::xfloat::XFloat::gain_pressure_surface(), argo::iridium::xfloat::XFloat::gain_salinity(), argo::iridium::xfloat::XFloat::gain_salinit
argo::iridium::xfloat::XFloat::gain_temperature(), argo::iridium::xfloat::XFloat::gain_temperature_surface(), argo::iridium::xfloat::XFloatDive
argo::iridium::xfloat::XFloatDiveData::GenerateCompositeIridiumPosition(), argo::iridium::xfloat::XFloatDiveData::GenerateCompositePun
argo::iridium::xfloat::XFloatDiveData::GenerateCompositeRiseTimeSeriesData(), argo::iridium::xfloat::XFloatDiveData::GenerateListingSt

util::xmsg::XCompositeDrift::GenerateListingString(), GenerateListingString(), util::xmsg::XCompositeProfile::GenerateListingString(), argo::iridium::xfloat::XFloatDiveData::GenerateOrderedMomsnList(), argo::iridium::xfloat::XFloatDiveData::GeneratePhyString(), util::xmsg::XCompositeDrift::GeneratePhyString(), GeneratePhyString(), util::xmsg::XCompositeProfile::GeneratePhyString(), argo::iridium::xfloat::XFloatDiveData::GetDiveDataMailMessagesFromDatabase(), argo::iridium::xfloat::XFloatDiveData::GetGainsAndOffs argo::iridium::xfloat::XFloatDiveData::GetGpsPositionEndOfDive(), argo::iridium::xfloat::XFloatDiveData::GetSurfaceGainsAndOffsets(), argo::iridium::xfloat::XFloatDiveData::HasBinnedProfileData(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLAS argo::iridium::xfloat::XFloatDiveData::HasContiguousMomsns(), argo::iridium::xfloat::XFloatDiveData::HasContiguousPids(), argo::iridium::xfloat::XFloatDiveData::HasDiveData(), argo::iridium::xfloat::XFloatDiveData::HasDriftData(), argo::iridium::xfloat::XFloatDiv argo::iridium::xfloat::XFloatDiveData::HasFallRiseData(), argo::iridium::xfloat::XFloatDiveData::HasPositionData(), argo::iridium::xfloat::XFloatDiveData::HasUnbinnedProfileData(), argo::iridium::xfloat::XFloatDiveData::HasValidGpsPosition(), argo::iridium::Iridium::id_dac(), argo::iridium::xfloat::XFloatDiveData::id_dive(), argo::iridium::xfloat::XFloatMailMessage::id_dive(), argo::iridium::Iridium::id_imei(), argo::iridium::Iridium::id_instrument(), argo::iridium::xfloat::XFloatMailMessage::id_msg(), argo::iridium::xfloat::XFloatMailMessage::id_msg_uid(), argo::iridium::xfloat::XFloatMailMessage::id_sbd_momsn(), argo::iridium::xfloat::XFloatMailMessage::id_sbd_mtmsn(), argo::iridium::xfloat::XFloatMailMessage::id_sbd_packet(), argo::iridium::Iridium::id_wmo(), argo::iridium::xfloat::XFloatDiveData::intermediate_listing_file_path(), util::xmsg::AbstractXSensorDataCo util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::IsDataComplete(), argo::iridium::xfloat::XFloatMetadata::local_path(), util::thread::RecursiveLockable::LockObject(), argo::iridium::xfloat::XFloatMetadata::ma argo::iridium::xfloat::XFloat::metadata(), argo::iridium::xfloat::XFloatMailMessage::msg_body_is_valid(), argo::iridium::xfloat::XFloatMailM argo::iridium::xfloat::XFloatMailMessage::msg_header_datetime(), argo::iridium::xfloat::XFloatMailMessage::msg_header_from(), argo::iridium::xfloat::XFloatMailMessage::msg_header_is_valid(), argo::iridium::xfloat::XFloatMailMessage::msg_header_raw_text(), argo::iridium::xfloat::XFloatMailMessage::msg_header_subject(), argo::iridium::xfloat::XFloatMailMessage::msg_is_valid(), argo::iridium::xfloat::XFloatMailMessage::msg_raw_text(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, T util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::num_saln_samples(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::num_temp_samples(), argo::iridium::xfloat::XFloat::offset_pressure(), argo::iridium::xfloat::XFloat::offset_pressure_surface(), argo::iridium::xfloat::XFloat::offset_ argo::iridium::xfloat::XFloat::offset_salinity_surface(), argo::iridium::xfloat::XFloat::offset_temperature(), argo::iridium::xfloat::XFloat::offset argo::iridium::xfloat::XFloatMailMessage::ParseMailMsg(), argo::iridium::xfloat::XFloatMailMessage::ParseMailMsg(), argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgBody(), argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgHeader(), argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgSbdAttachment(), argo::iridium::xfloat::XFloat::phy_file_generation_is_enabled(), argo::iridium::xfloat::XFloatDiveData::phy_file_path(), argo::iridium::xfloat::XFloatMetadata::principal_investigators(), argo::iridium::xfloat::XFloatMetadata::principal_investigators_address(), argo::iridium::xfloat::XFloatMetadata::principal_investigators_orig argo::iridium::xfloat::XFloatMailMessage::raw_file_local_path(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLA argo::iridium::xfloat::XFloatDiveData::ResetData(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_C util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SamplePresAsFloat(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleSaln(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleSalnAsFloat(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleTemp(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleTempAsFloat(), argo::iridium::xfloat::XFloatDiveData::SaveObjectToDatabase(), argo::iridium::xfloat::XFloatMailMessage::SaveObjectToDatabase(), argo::iridium::xfloat::XFloatMetadata::SaveObjectToDatabase(), argo::iridium::xfloat::XFloatMailMessage::sbd_data(), argo::iridium::xfloat::XFloatMailMessage::sbd_data_is_valid(), argo::iridium::xfloat::XFloatMailMessage::sbd_file_local_path(), argo::iridium::xfloat::XFloatMailMessage::sbd_file_name(), argo::iridium::xfloat::XFloatMailMessage::sbd_file_size_bytes(), argo::iridium::xfloat::XFloatMailMessage::sbd_session_status_code(), argo::iridium::xfloat::XFloatMailMessage::sbd_session_status_strir argo::iridium::xfloat::XFloatDiveData::SensorGainsAndOffsetsAreKnown(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, argo::iridium::xfloat::XFloat::set_argo_instrument_type(), argo::iridium::xfloat::XFloatDiveData::set_argo_instrument_type(), argo::iridium::xfloat::XFloatMetadata::set_comment(), argo::iridium::xfloat::XFloatMetadata::set_deployer(), argo::iridium::xfloat::XFloatMe argo::iridium::xfloat::XFloatMetadata::set_deployment_cruise_id(), argo::iridium::xfloat::XFloatMetadata::set_deployment_datetime(), argo::iridium::xfloat::XFloatMetadata::set_deployment_datetime_status(), argo::iridium::xfloat::XFloatMetadata::set_deployment_latitude() argo::iridium::xfloat::XFloatMetadata::set_deployment_longitude(), argo::iridium::xfloat::XFloatMetadata::set_deployment_platform(), argo::iridium::xfloat::XFloatMetadata::set_deployment_position_status(), argo::iridium::xfloat::XFloatMetadata::set_deployment_type(), argo::iridium::xfloat::XFloat::set_fw_version(), argo::iridium::xfloat::XFloat::set_gain_pressure(), argo::iridium::xfloat::XFloat::set_gain_pre argo::iridium::xfloat::XFloat::set_gain_salinity(), argo::iridium::xfloat::XFloat::set_gain_salinity_surface(), argo::iridium::xfloat::XFloat::set_ argo::iridium::xfloat::XFloat::set_gain_temperature_surface(), argo::iridium::Iridium::set_id_dac(), argo::iridium::xfloat::XFloatDiveData::se argo::iridium::xfloat::XFloatMailMessage::set_id_dive(), argo::iridium::Iridium::set_id_imei(), argo::iridium::Iridium::set_id_instrument(), argo::iridium::xfloat::XFloatMailMessage::set_id_msg(), argo::iridium::xfloat::XFloatMailMessage::set_id_msg_uid(), argo::iridium::xfloat::XFloatMailMessage::set_id_sbd_momsn(), argo::iridium::xfloat::XFloatMailMessage::set_id_sbd_mtmsn(), argo::iridium::xfloat::XFloatMailMessage::set_id_sbd_packet(), argo::iridium::Iridium::set_id_wmo(), argo::iridium::xfloat::XFloatDiveData: argo::iridium::xfloat::XFloatMetadata::set_local_path(), argo::iridium::xfloat::XFloatMetadata::set_manufacturer(), argo::iridium::xfloat::XFloat::set_metadata(), argo::iridium::xfloat::XFloatMailMessage::set_msg_body_is_valid(), argo::iridium::xfloat::XFloatMailMessage::set_msg_body_text(), argo::iridium::xfloat::XFloatMailMessage::set_msg_header_datetime(), argo::iridium::xfloat::XFloatMailMessage::set_msg_header_from(), argo::iridium::xfloat::XFloatMailMessage::set_msg_header_is_valid(), argo::iridium::xfloat::XFloatMailMessage::set_msg_header_raw_text(), argo::iridium::xfloat::XFloatMailMessage::set_msg_header_subjec argo::iridium::xfloat::XFloatMailMessage::set_msg_is_valid(), argo::iridium::xfloat::XFloatMailMessage::set_msg_raw_text(), argo::iridium::xfloat::XFloat::set_offset_pressure(), argo::iridium::xfloat::XFloat::set_offset_pressure_surface(), argo::iridium::xfloat::XFloa argo::iridium::xfloat::XFloat::set_offset_salinity_surface(), argo::iridium::xfloat::XFloat::set_offset_temperature(), argo::iridium::xfloat::XFloat::set_offset_temperature_surface(), argo::iridium::xfloat::XFloat::set_phy_file_generation_is_enabled(), argo::iridium::xfloat::XFloatDiveData::set_phy_file_path(), argo::iridium::xfloat::XFloatMetadata::set_principal_investigators(), argo::iridium::xfloat::XFloatMetadata::set_principal_investigators_address(), argo::iridium::xfloat::XFloatMetadata::set_principal_investiga argo::iridium::xfloat::XFloatMailMessage::set_raw_file_local_path(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_data(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_data_is_valid(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_file_local_path(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_file_name(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_file_size_bytes(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_session_status_code(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_session_sta

argo::iridium::xfloat::XFloat::set_start_of_mission_datetime(), argo::iridium::xfloat::XFloatMetadata::set_wmo_instrument_type(), argo::iridium::xfloat::XFloatMetadata::set_wmo_recorder_type(), argo::iridium::xfloat::XFloat::set_xmsg_version_major(), argo::iridium::xfloat::XFloat::set_xmsg_version_minor(), argo::iridium::xfloat::XFloat::start_of_mission_datetime(), util::thread::RecursiveLockable::UnlockObject(), argo::iridium::xfloat::XFloatDiveData::UpdateDerivedSurfaceDataFields(), argo::iridium::xfloat::XFloatMetadata::wmo_instrument_type(), argo::iridium::xfloat::XFloatMetadata::wmo_recorder_type(), argo::iridium::xfloat::XFloatDiveData::WriteListingFile(), argo::iridium::xfloat::XFloatDiveData::WritePhyFile(), argo::iridium::xfloat::XFloatM argo::iridium::xfloat::XFloat::xmsg_version_major(), and argo::iridium::xfloat::XFloat::xmsg_version_minor().

10.62.5.6 pres_messages_

PresMsgVector_t util::xmsg::AbstractXSensorDataComposite< XHiResPressure, XHiResSalinity, XHiResTemperature, MAX_MSGS >::pres_messages_ [private], [inherited]

A static vector (of size MAX_MSGS) containing pointers the pressure sensor data messages assigned to this composite object.

Each time a pressure sensor data message is assigned to the this object, it will be emplaced in this vector in order of increasing message indices.

10.62.5.7 saln_messages_

SalnMsgVector_t util::xmsg::AbstractXSensorDataComposite< XHiResPressure, XHiResSalinity, XHiResTemperature, MAX_MSGS >::saln_messages_ [private], [inherited]

A static vector (of size MAX_MSGS) containing pointers the salinity sensor data messages assigned to this composite object.

Each time a salinity sensor data message is assigned to the this object, it will be emplaced in this vector in order of increasing message indices.

10.62.5.8 temp_messages_

TempMsgVector_t util::xmsg::AbstractXSensorDataComposite< XHiResPressure, XHiResSalinity, XHiResTemperature, MAX_MSGS >::temp_messages_ [private], [inherited]

A static vector (of size MAX_MSGS) containing pointers the temperature sensor data messages assigned to this composite object.

Each time a temperature sensor data message is assigned to the this object, it will be emplaced in this vector in order of increasing message indices.

The documentation for this class was generated from the following file:

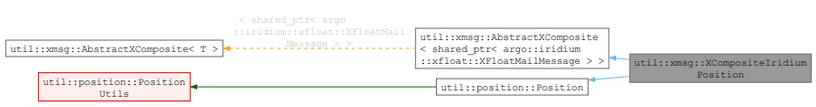
- data/xmsg/XCompositeHiResProfile.h

10.63 util::xmsg::XCompositeIridiumPosition Class Reference

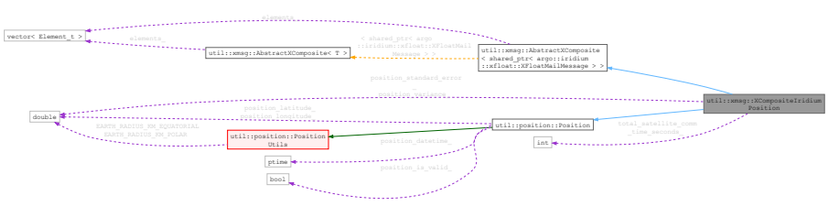
Class representing the aggregate Iridium position for a single dive, composed from one or more Iridium message positions.

#include <XCompositeIridiumPosition.h>

Inheritance diagram for util::xmsg::XCompositeIridiumPosition:



Collaboration diagram for util::xmsg::XCompositeIridiumPosition:



Public Member Functions

- XCompositeIridiumPosition ()

  The default constructor.
- ~XCompositeIridiumPosition ()

  The destructor.
- void AppendAllElements (const static_vector< Element_t, I > &elements)

  Append a vector of elements to the vector of individual X-Messages elements that compose this composite object.
- virtual void AssignElement (const Element_t &element)

  Assign an element to this composite object.
- void ComputeAverageIridiumPosition ()

  Compute the weighted average position from the Iridium positions for this dive.
- const vector< Element_t > & elements () const

  Get the vector of individual X-Messages elements that compose this composite object.
- const string GenerateListingString (int whoi, int dive) const

  Generate a listing entry string from this composite Iridum position object.
- virtual bool **IsEmpty** () const
- virtual size_t num_elements () const

  Get the number of individual elements that compose this composite object.
- ptime position_datetime () const

  Get the date/time of the position.
- string position_datetime_as_iso8601_string () const

  Get a formatted string representing the position date/time composed using the ISO 8601 standard.
- bool position_is_valid () const

  Is the position valid?
- double position_latitude () const

  Get the latitude of the position, in decimal degrees.
- double position_longitude () const

  Get the longitude of the position, in decimal degrees.
- double position_standard_error () const

  Get the calculated variance of this composite Iridium position, in km^2.
- double position_variance () const

  Get the calculated variance of this composite Iridium position, in km^2.
- string postition_as_iso6709_string () const

  Get a formatted string representing the position composed using the ISO 6709 standard.
- int total_satellite_comm_time_seconds () const

  Get the Iridium session communication time, in seconds.

Static Public Member Functions

- static const string GenerateListingComment ()

  Generate the listing entry comment.

Protected Member Functions

- void AppendAllElements (const vector< Element_t > &elements)

  Append a vector of elements to the vector of individual X-Messages elements that compose this composite object.
- void AppendElement (const Element_t &element)

  Append one element to the vector of individual X-Messages elements that compose this composite object.
- void set_gps_position_datetime (unsigned int gpsWeek, unsigned int gpsDay, unsigned int gpsHour=0, unsigned int gpsMinute=0, unsigned int gpsSecond=0)

  Set the date and time of this position from the given GPS week, day, and time.
- void set_position_datetime (const ptime &datetime)

  Set the date/time of the position.
- void set_position_is_valid (bool is_valid=true)

  Set the position is valid flag.
- void set_position_latitude (double lat)

  Set the latitude of the position, in decimal degrees.
- void set_position_longitude (double lon)

  Set the longitude of the position, in decimal degrees.
- void set_position_standard_error (double position_standard_error)

  Set the calculated standard error of this composite Iridium position, in km.
- void set_position_variance (double position_variance)

  Set the calculated variance of this composite Iridium position, in km^2.
- void set_total_satellite_comm_time_seconds (int time)

  Set the Iridium session communication time, in seconds.

Static Protected Member Functions

- static float **bitcastUint32ToFloat** (uint32_t u)
- template<typename T >
  static double CalculateEllipsoidHeight (T lat)

  Get the approximate surface altitude or height of earth for a given latitude, in kilometers.
- template<typename T >
  static double CalculateGeographicDistance (T lat1, T lon1, T lat0, T lon0)

  Get the geographical distance between two position, in kilometers, from the positions' locations, in decimal degrees.
- template<typename T >
  static double degToRad (T deg)

  Convert angular units from from degrees to decimal radians.
- static string GenerateCurrentUtcTimeIso8601String ()

  Generate an ISO8601 format date/time string representation of the current time, in UTC.
- template<typename T >
  static std::string GenerateIso6709String (T lat, T lon)

  Generate an IS06709 string representation of the position given by (`lat`, `lon`).
- static string GenerateIso8601String (const ptime &p, bool extended=true)

  Generate an ISO8601 format date/time string representation of a POSIX time object.
- static double GenerateJulianDay (const ptime &dt, const ptime &ref_dt)

  Convert a date/time to fractional days since a reference date/time.
- static ptime GetCurrentUtcTime ()

  Generate a POSIX time object representing the current date and time (rounded to the nearest second).
- template<typename T >
  static bool IsPositionValid (T lat, T lon)

  Check position validity.

Private Types

- typedef shared_ptr< argo::iridium::xfloat::XFloatMailMessage > Element_t

  Type definition for the element aggregated by this object.
- typedef ElementVec_t::const_iterator **ElementConstIter_t**
- typedef ElementVec_t::iterator **ElementIter_t**
- typedef vector< Element_t > **ElementVec_t**

Private Attributes

- ElementVec_t elements_

  The vector of individual X-Messages elements that compose this composite object.
- ptime position_datetime_

  The date and time of the position.
- bool position_is_valid_

  The valid position indicator.
- double position_latitude_

  The latitude of the position, in decimal degrees.
- double position_longitude_

  The longitude of the position, in decimal degrees.
- double position_standard_error_

  The calculated standard error of this composite Iridium position, in km.
- double position_variance_

  The calculated variance of this composite Iridium position, in km^2.
- int total_satellite_comm_time_seconds_

  The Iridium session communication time, in seconds.

Static Private Attributes

- static const double EARTH_RADIUS_KM_EQUATORIAL = 6378.1

  The equatorial radius of the earth, in kilometers.
- static const double EARTH_RADIUS_KM_POLAR = 6356.1

  The polar radius of the earth, in kilometers.

10.63.1 Detailed Description

Class representing the aggregate Iridium position for a single dive, composed from one or more Iridium message positions.

10.63.2 Member Typedef Documentation

10.63.2.1 Element_t

```
typedef shared_ptr< argo::iridium::xfloat::XFloatMailMessage > util::xmsg::AbstractXComposite<
shared_ptr< argo::iridium::xfloat::XFloatMailMessage > >::Element_t  [private], [inherited]
```

Type definition for the element aggregated by this object.

10.63.3 Constructor & Destructor Documentation

10.63.3.1 XCompositeIridiumPosition()

```
util::xmsg::XCompositeIridiumPosition::XCompositeIridiumPosition ()  [inline]
```

The default constructor.

10.63.3.2 ~XCompositeIridiumPosition()

```
util::xmsg::XCompositeIridiumPosition::~XCompositeIridiumPosition ()  [inline]
```

The destructor.

10.63.4 Member Function Documentation

10.63.4.1 AppendAllElements() [1/2]

```
void util::xmsg::AbstractXComposite< shared_ptr< argo::iridium::xfloat::XFloatMailMessage > >↩
::AppendAllElements (
            const static_vector< Element_t, I > & elements)  [inline], [inherited]
```

Append a vector of elements to the vector of individual X-Messages elements that compose this composite object.

Parameters

in	*elements*	The vector of elements to append.

10.63.4.2 AppendAllElements() [2/2]

```
void util::xmsg::AbstractXComposite< shared_ptr< argo::iridium::xfloat::XFloatMailMessage > >↩
::AppendAllElements (
            const vector< Element_t > & elements)  [inline], [protected], [inherited]
```

Append a vector of elements to the vector of individual X-Messages elements that compose this composite object.

Parameters

in	*elements*	The vector of elements to append.

10.63.4.3 AppendElement()

```
void util::xmsg::AbstractXComposite< shared_ptr< argo::iridium::xfloat::XFloatMailMessage > >↩
::AppendElement (
            const Element_t & element)  [inline], [protected], [inherited]
```

Append one element to the vector of individual X-Messages elements that compose this composite object.

Parameters

in	*element*	The element to append.

10.63.4.4 AssignElement()

```
virtual void util::xmsg::AbstractXComposite< shared_ptr< argo::iridium::xfloat::XFloatMailMessage
> >::AssignElement (
            const Element_t & element)  [inline], [virtual], [inherited]
```

Assign an element to this composite object.

The default implementation appends the element; derived classes may override it to insert elements in order or reject duplicates.

Parameters

in	*element*	The element to assign.

10.63.4.5 CalculateEllipsoidHeight()

```
template<typename T >
static double util::position::PositionUtils::CalculateEllipsoidHeight (
            T lat)  [inline], [static], [inherited]
```

Get the approximate surface altitude or height of earth for a given latitude, in kilometers.

Warning

This method provides approximate surface height estimates only! This approximation is calculated from a reference ellipsoid – a mathematically-defined surface that approximates the geoid, the truer figure of the Earth.

Parameters

in	*lat*	The geographic latitude of interest, in decimal degrees.

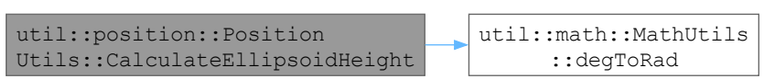
Returns

The approximate surface altitude or height of earth for the given latitude, in kilometers.

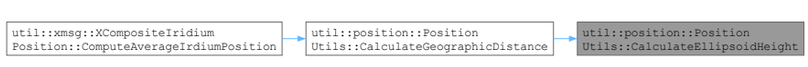
References util::math::MathUtils::degToRad(), util::position::PositionUtils::EARTH_RADIUS_KM_EQUATORIAL, and util::position::PositionUtils::EARTH_RADIUS_KM_POLAR.

Referenced by util::position::PositionUtils::CalculateGeographicDistance().

Here is the call graph for this function:



Here is the caller graph for this function:



10.63.4.6 CalculateGeographicDistance()

```
template<typename T >
static double util::position::PositionUtils::CalculateGeographicDistance (
            T lat1,
            T lon1,
            T lat0,
            T lon0)  [inline], [static], [inherited]
```

Get the geographical distance between two position, in kilometers, from the positions' locations, in decimal degrees.

Warning

This method provides approximate geographic estimates only! This approximation is calculated from a reference ellipsoid – a mathematically-defined surface that approximates the geoid, the truer figure of the Earth.

Note

This method uses the 'haversine' formula to calculate the great-circle distance between two points – that is, the shortest distance over the earth's surface – giving an 'as-the-crow-flies' distance between the points (ignoring any hills they fly over, of course!).

Parameters

in	*lat1*	The latitude of the geographic final position, in decimal degrees North.
in	*lon1*	The longitude of the geographic final position, in decimal degrees East.
in	*lat0*	The latitude of the geographic initial position, in decimal degrees North.
in	*lon0*	The longitude of the geographic initial position, in decimal degrees East.

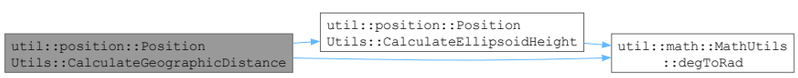
Returns

The geographical distance between the two given positions, in kilometers.

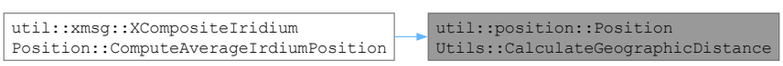
References util::position::PositionUtils::CalculateEllipsoidHeight(), and util::math::MathUtils::degToRad().

Referenced by ComputeAverageIrdiumPosition().

Here is the call graph for this function:



Here is the caller graph for this function:



10.63.4.7 ComputeAverageIrdiumPosition()

```
void util::xmsg::XCompositeIridiumPosition::ComputeAverageIrdiumPosition ()
```

Compute the weighted average position from the Iridium positions for this dive.

Note

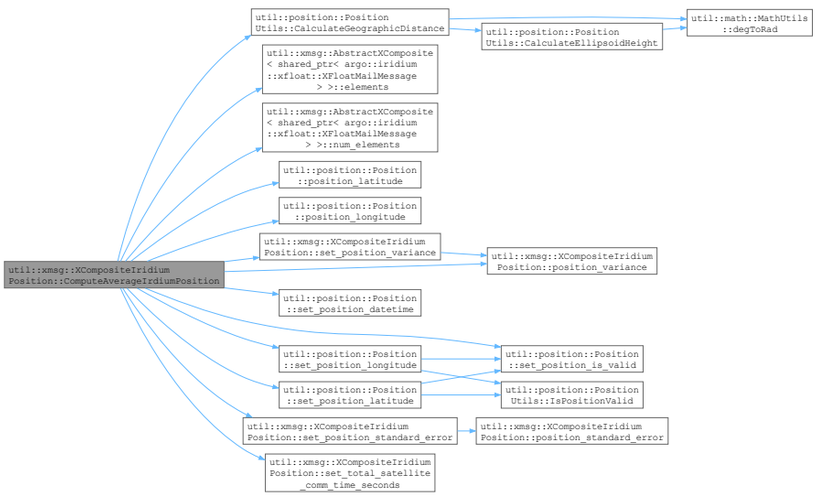
Each Iridium position is assigned a weight of CEP^-2 when computing the average postion, where CEP is the circular error probable radius reported with each Iridium postion.

Exceptions

log::MessageException	if an error occurs while computing the average position.

References util::position::PositionUtils::CalculateGeographicDistance(), CATCH_BLOCK, util::xmsg::AbstractXComposite< shared_ptr< a
util::xmsg::AbstractXComposite< shared_ptr< argo::iridium::xfloat::XFloatMailMessage > >::num_elements(), util::position::Position::po
util::position::Position::position_longitude(), position_variance(), util::position::Position::set_position_datetime(),
util::position::Position::set_position_is_valid(), util::position::Position::set_position_latitude(), util::position::Position::set_position_longitude
set_position_standard_error(), set_position_variance(), set_total_satellite_comm_time_seconds(), XMSG_POS_INVALID_LAT,
XMSG_POS_INVALID_LON, XMSG_POS_INVALID_STDERR, and XMSG_POS_INVALID_VAR.

Here is the call graph for this function:



10.63.4.8 degToRad()

```
template<typename T >
static double util::math::MathUtils::degToRad (
            T deg)  [inline], [static], [inherited]
```

Convert angular units from from degrees to decimal radians.

Parameters

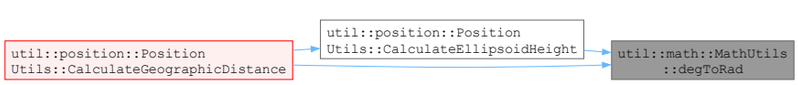
in	*deg*	Angle in integer or decimal degrees to convert to radians.

Returns

Angle in decimal radians.

Referenced by util::position::PositionUtils::CalculateEllipsoidHeight(), and util::position::PositionUtils::CalculateGeographicDistance().

Here is the caller graph for this function:



10.63.4.9 elements()

```
const vector< Element_t > & util::xmsg::AbstractXComposite< shared_ptr< argo::iridium::xfloat::XFloatMailMessage
> >::elements () const  [inline], [inherited]
```

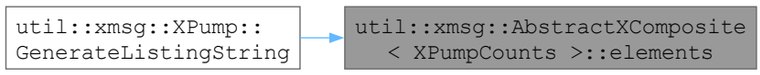
Get the vector of individual X-Messages elements that compose this composite object.

Returns

The vector of individual X-Messages elements that compose this composite object.

Referenced by ComputeAverageIrdiumPosition(), and GenerateListingString().

Here is the caller graph for this function:



10.63.4.10 GenerateCurrentUtcTimeIso8601String()

```
static string util::date::DateUtils::GenerateCurrentUtcTimeIso8601String ()  [inline], [static],
[inherited]
```

Generate an ISO8601 format date/time string representation of the current time, in UTC.

Note

All date/time objects use the UTC time-zone.

Returns

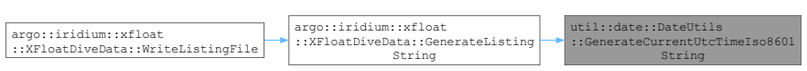
An ISO8601 format date/time string representation of the current time.

References util::date::DateUtils::GenerateIso8601String(), and util::date::DateUtils::GetCurrentUtcTime().

Referenced by argo::iridium::xfloat::XFloatDiveData::GenerateListingString().

Here is the call graph for this function:

Here is the caller graph for this function:



10.63.4.11 GenerateIso6709String()

```
template<typename T >
static std::string util::position::PositionUtils::GenerateIso6709String (
            T lat,
            T lon)  [inline], [static], [inherited]
```

Generate an IS06709 string representation of the position given by (`lat`, `lon`).

Parameters

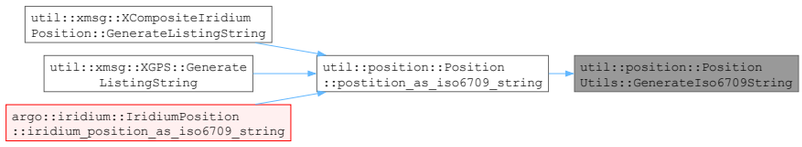
in	*lat*	The latitude of the position, in degrees North.
in	*lon*	The longitude of the position, in degrees East.

Returns

An IS06709 string representation of the position given by (`lat`, `lon`).

Referenced by util::position::Position::postition_as_iso6709_string().

Here is the caller graph for this function:



10.63.4.12 GenerateIso8601String()

```
static string util::date::DateUtils::GenerateIso8601String (
            const ptime & p,
            bool extended = true)  [inline], [static], [inherited]
```

Generate an ISO8601 format date/time string representation of a POSIX time object.

Note

All date/time objects use the UTC time-zone.

Parameters

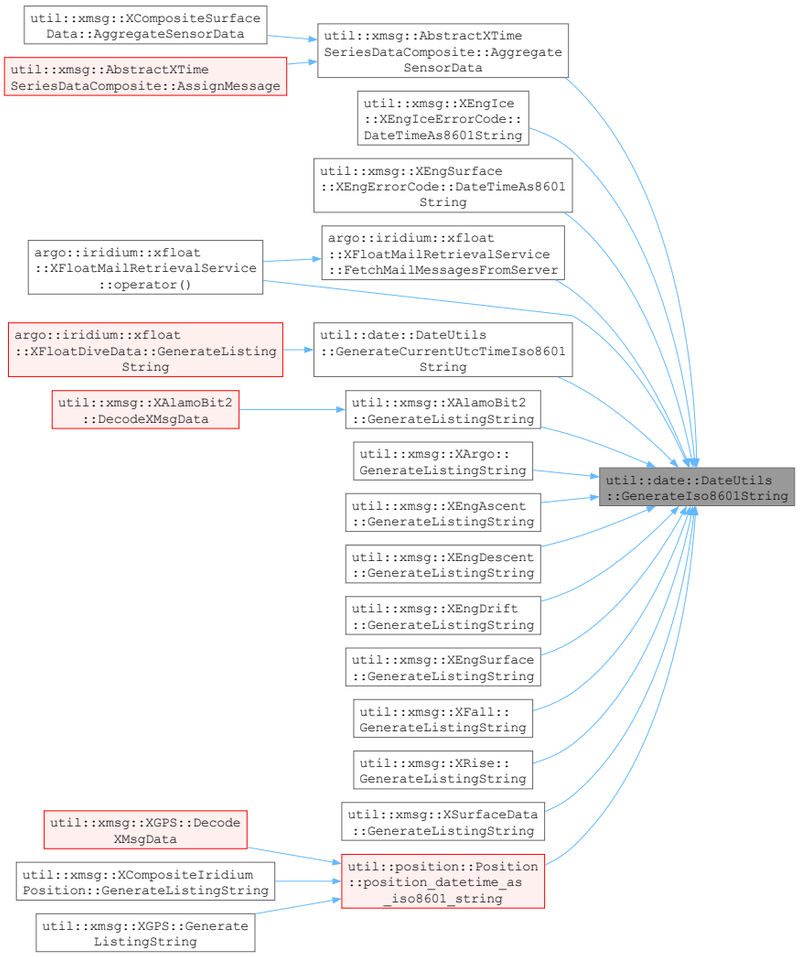
in	*p*	POSIX time from which to generate string.
in	*extended*	If TRUE, use the ISO 8601 extended format (YYYY-MM-DDThh:mm:ssZ); otherwise, use the basic format (YYYYMMDDThhmmss).

Returns

An ISO8601 format date/time string representation of the POSIX time object. If p is equal to boost::posix_↩ time::not_a_date_time, then the string "0000-00-00T00:00:00Z" (extended) or "00000000000000" (basic) will be returned.

Referenced by util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::AggregateSensorData(), util::xmsg::XEngIce::XEngIce
util::xmsg::XEngSurface::XEngErrorCode::DateTimeAs8601String(), argo::iridium::xfloat::XFloatMailRetrievalService::FetchMailMessages
util::date::DateUtils::GenerateCurrentUtcTimeIso8601String(), util::xmsg::XAlamoBit2::GenerateListingString(), util::xmsg::XArgo::Genera
util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListing
util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XRise::GenerateListingString(),
util::xmsg::XSurfaceData::GenerateListingString(), argo::iridium::xfloat::XFloatMailRetrievalService::operator()(), and
util::position::Position::position_datetime_as_iso8601_string().

Here is the caller graph for this function:



10.63.4.13 GenerateJulianDay()

```
static double util::date::DateUtils::GenerateJulianDay (
            const ptime & dt,
            const ptime & ref_dt)  [inline], [static], [inherited]
```

Convert a date/time to fractional days since a reference date/time.

Parameters

in	*dt*	The date/time to convert.
in	*ref↩ _dt*	The reference date/time (day 0).

Exceptions

util::log::MessageException	if either date/time is not valid.

Returns

The number of days (with millisecond resolution) from `ref_dt` to `dt`.

References CATCH_BLOCK, and EXC_THROW.

10.63.4.14 GenerateListingComment()

```
static const string util::xmsg::XCompositeIridiumPosition::GenerateListingComment ()  [inline],
[static]
```

Generate the listing entry comment.

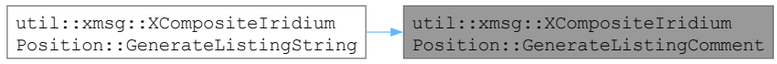
Returns

The listing entry comment.

References CATCH_BLOCK.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.63.4.15 GenerateListingString()

```
const string util::xmsg::XCompositeIridiumPosition::GenerateListingString (
            int whoi,
            int dive) const  [inline]
```

Generate a listing entry string from this composite Iridum position object.

Parameters

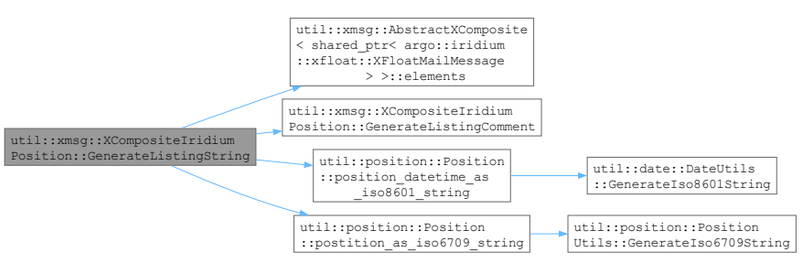
in	*whoi*	The unique WHOI instrument identifier or serial number.
in	*dive*	The dive or cycle number of this composite Iridum position object.

Returns

A listing entry string from this composite Iridum position object.

References CATCH_BLOCK, util::xmsg::AbstractXComposite< shared_ptr< argo::iridium::xfloat::XFloatMailMessage > >::elements(), GenerateListingComment(), util::position::Position::position_datetime_as_iso8601_string(), position_standard_error_, position_variance_, util::position::Position::postition_as_iso6709_string(), and total_satellite_comm_time_seconds_.

Here is the call graph for this function:



10.63.4.16 GetCurrentUtcTime()

```
static ptime util::date::DateUtils::GetCurrentUtcTime ()  [inline], [static], [inherited]
```

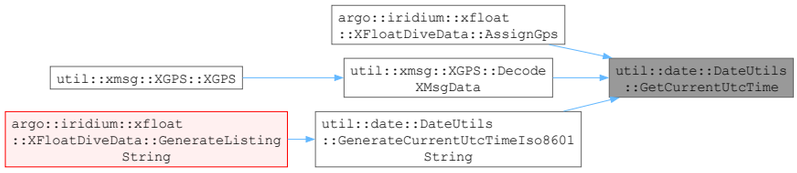
Generate a POSIX time object representing the current date and time (rounded to the nearest second).

Returns

A POSIX time object representing the current date and time (rounded to the nearest second).

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignGps(), util::xmsg::XGPS::DecodeXMsgData(), and util::date::DateUtils::GenerateCurrentUtcTimeIso8601String().

Here is the caller graph for this function:



10.63.4.17 IsPositionValid()

```
template<typename T >
static bool util::position::PositionUtils::IsPositionValid (
            T lat,
            T lon)  [inline], [static], [inherited]
```

Check position validity.

Parameters

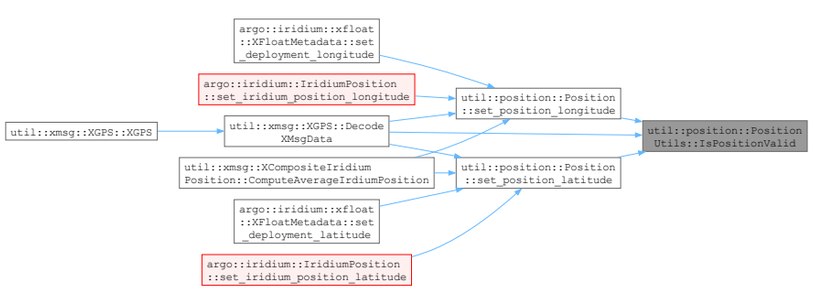
in	*lat*	Latitude of position.
in	*lon*	Longitude of position.

Returns

TRUE if (lat, lon) represents a valid geographical position, or FALSE otherwise.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), util::position::Position::set_position_latitude(), and util::position::Position::set_positi

Here is the caller graph for this function:



10.63.4.18 num_elements()

```
virtual size_t util::xmsg::AbstractXComposite< shared_ptr< argo::iridium::xfloat::XFloatMailMessage
> >::num_elements () const  [inline], [virtual], [inherited]
```

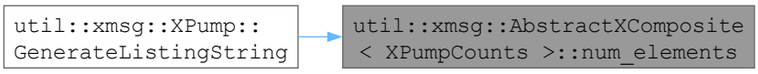
Get the number of individual elements that compose this composite object.

Returns

The number of individual elements that compose this composite object.

Referenced by ComputeAverageIrdiumPosition().

Here is the caller graph for this function:



10.63.4.19 position_datetime()

```
ptime util::position::Position::position_datetime () const  [inline], [inherited]
```

Get the date/time of the position.

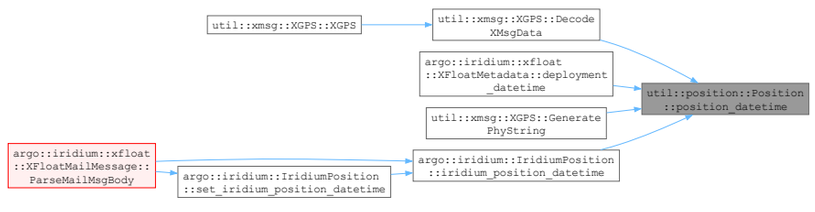
Returns

The date/time of the position.

References util::position::Position::position_datetime_.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), argo::iridium::xfloat::XFloatMetadata::deployment_datetime(), util::xmsg::XGPS::GeneratePhyString(), and argo::iridium::IridiumPosition::iridium_position_datetime().

Here is the caller graph for this function:



10.63.4.20 position_datetime_as_iso8601_string()

```
string util::position::Position::position_datetime_as_iso8601_string () const  [inline], [inherited]
```

Get a formatted string representing the position date/time composed using the ISO 8601 standard.

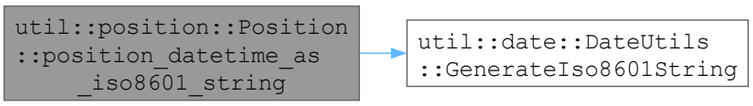
Returns

The formatted string representation of the position date/time.

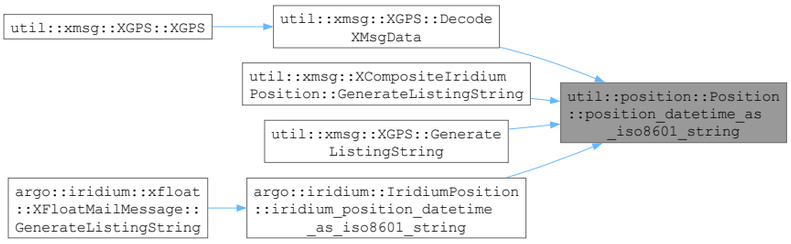
References util::date::DateUtils::GenerateIso8601String(), and util::position::Position::position_datetime_.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), and argo::iridium::IridiumPosition::iridium_position_datetime_as_iso8601_string().

Here is the call graph for this function:



Here is the caller graph for this function:



10.63.4.21 position_is_valid()

```
bool util::position::Position::position_is_valid () const  [inline], [inherited]
```

Is the position valid?

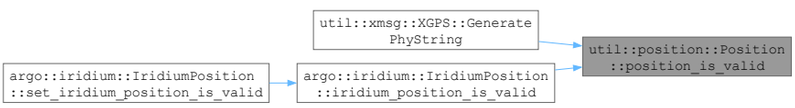
Returns

TRUE if the position is valid, or false otherwise.

References util::position::Position::position_is_valid_.

Referenced by util::xmsg::XGPS::GeneratePhyString(), and argo::iridium::IridiumPosition::iridium_position_is_valid().

Here is the caller graph for this function:



10.63.4.22 position_latitude()

```
double util::position::Position::position_latitude () const  [inline], [inherited]
```

Get the latitude of the position, in decimal degrees.

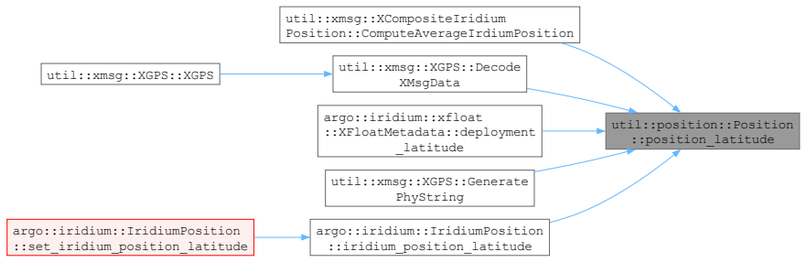
Returns

The latitude of the position, in decimal degrees.

References util::position::Position::position_latitude_.

Referenced by ComputeAverageIridiumPosition(), util::xmsg::XGPS::DecodeXMsgData(), argo::iridium::xfloat::XFloatMetadata::deploymer
util::xmsg::XGPS::GeneratePhyString(), and argo::iridium::IridiumPosition::iridium_position_latitude().

Here is the caller graph for this function:



10.63.4.23 position_longitude()

```
double util::position::Position::position_longitude () const  [inline], [inherited]
```

Get the longitude of the position, in decimal degrees.

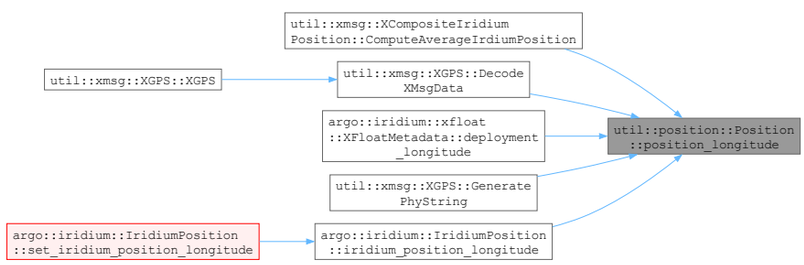
Returns

The longitude of the position, in decimal degrees.

References util::position::Position::position_longitude_.

Referenced by ComputeAverageIridiumPosition(), util::xmsg::XGPS::DecodeXMsgData(), argo::iridium::xfloat::XFloatMetadata::deploymer
util::xmsg::XGPS::GeneratePhyString(), and argo::iridium::IridiumPosition::iridium_position_longitude().

Here is the caller graph for this function:



10.63.4.24 position_standard_error()

```
double util::xmsg::XCompositeIridiumPosition::position_standard_error () const  [inline]
```

Get the calculated variance of this composite Iridium position, in km^2.

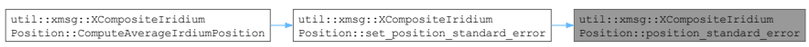
Returns

The calculated variance of this composite Iridium position, in km^2.

References position_standard_error_.

Referenced by set_position_standard_error().

Here is the caller graph for this function:



10.63.4.25 position_variance()

```
double util::xmsg::XCompositeIridiumPosition::position_variance () const  [inline]
```

Get the calculated variance of this composite Iridium position, in km^2.

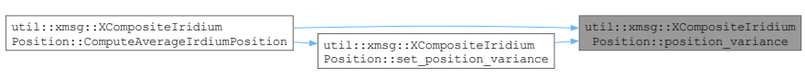
Returns

The calculated variance of this composite Iridium position, in km^2.

References position_variance_.

Referenced by ComputeAverageIrdiumPosition(), and set_position_variance().

Here is the caller graph for this function:



10.63.4.26 postition_as_iso6709_string()

```
string util::position::Position::postition_as_iso6709_string () const  [inline], [inherited]
```

Get a formatted string representing the position composed using the ISO 6709 standard.

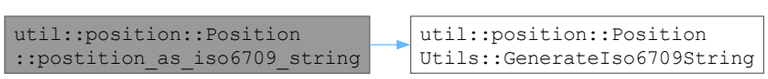
Returns

The formatted string representation of the position.

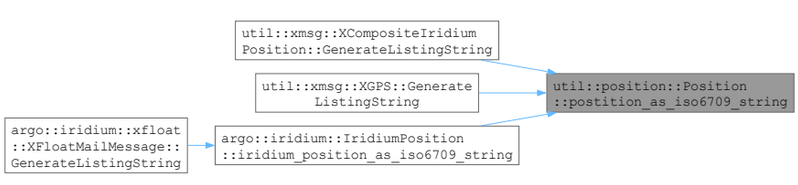
References util::position::PositionUtils::GenerateIso6709String(), util::position::Position::position_latitude_, and util::position::Position::position_longitude_.

Referenced by GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), and argo::iridium::IridiumPosition::iridium_position_as_

Here is the call graph for this function:



Here is the caller graph for this function:



10.63.4.27 set_gps_position_datetime()

```
void util::position::Position::set_gps_position_datetime (
            unsigned int gpsWeek,
            unsigned int gpsDay,
            unsigned int gpsHour = 0,
            unsigned int gpsMinute = 0,
            unsigned int gpsSecond = 0)  [inline], [protected], [inherited]
```

Set the date and time of this position from the given GPS week, day, and time.

Parameters

in	*gpsWeek*	The GPS week, in weeks since midnight of January 6, 1980.
in	*gpsDay*	The GPS day of week (0 - 6).
in	*gpsHour*	The UTC hour of day (0 - 23).
in	*gpsMinute*	The UTC minute of hour (0 - 59)..
in	*gpsSecond*	The UTC second of minute (0 - 59).

References util::position::Position::set_position_datetime().

Referenced by util::xmsg::XGPS::DecodeXMsgData().

Here is the call graph for this function:

Here is the caller graph for this function:

10.63.4.28 set_position_datetime()

```
void util::position::Position::set_position_datetime (
            const ptime & datetime)  [inline], [protected], [inherited]
```

Set the date/time of the position.

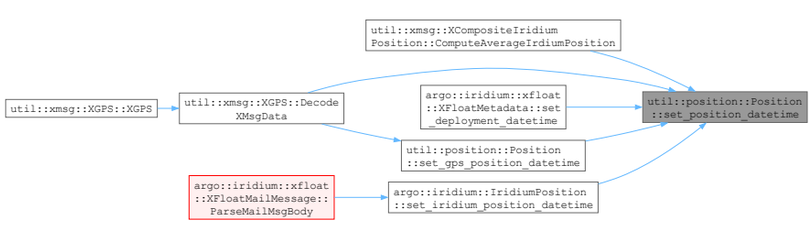
Parameters

in	*datetime*	The date/time of the position to set.

References util::position::Position::position_datetime_.

Referenced by ComputeAverageIrdiumPosition(), util::xmsg::XGPS::DecodeXMsgData(), argo::iridium::xfloat::XFloatMetadata::set_deploy
util::position::Position::set_gps_position_datetime(), and argo::iridium::IridiumPosition::set_iridium_position_datetime().

Here is the caller graph for this function:



10.63.4.29 set_position_is_valid()

```
void util::position::Position::set_position_is_valid (
            bool is_valid = true)  [inline], [protected], [inherited]
```

Set the position is valid flag.

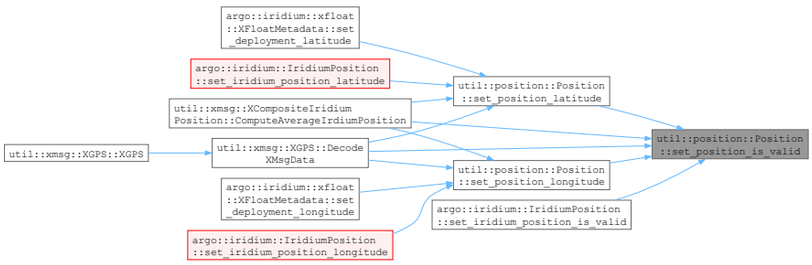
Parameters

in	*is_valid*	TRUE if the position is valid, or FALSE otherwise.

References util::position::Position::position_is_valid_.

Referenced by ComputeAverageIrdiumPosition(), util::xmsg::XGPS::DecodeXMsgData(), argo::iridium::IridiumPosition::set_iridium_positio util::position::Position::set_position_latitude(), and util::position::Position::set_position_longitude().

Here is the caller graph for this function:



10.63.4.30 set_position_latitude()

```
void util::position::Position::set_position_latitude (
            double lat)  [inline], [protected], [inherited]
```

Set the latitude of the position, in decimal degrees.

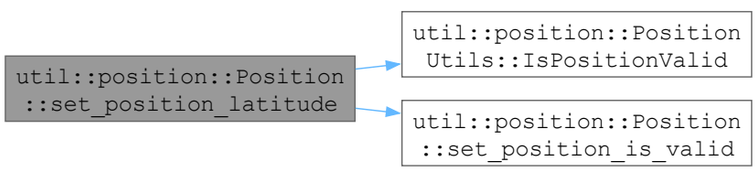
Parameters

in	*lat*	The latitude of the position, in decimal degrees.

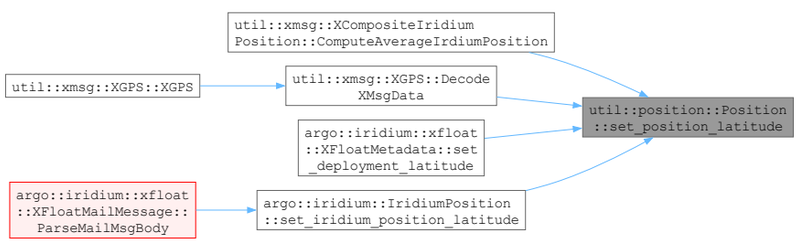
References util::position::PositionUtils::IsPositionValid(), util::position::Position::position_latitude_, util::position::Position::position_longitud and util::position::Position::set_position_is_valid().

Referenced by ComputeAverageIrdiumPosition(), util::xmsg::XGPS::DecodeXMsgData(), argo::iridium::xfloat::XFloatMetadata::set_deploy and argo::iridium::IridiumPosition::set_iridium_position_latitude().

Here is the call graph for this function:



Here is the caller graph for this function:



10.63.4.31 set_position_longitude()

```
void util::position::Position::set_position_longitude (
            double lon)  [inline], [protected], [inherited]
```

Set the longitude of the position, in decimal degrees.

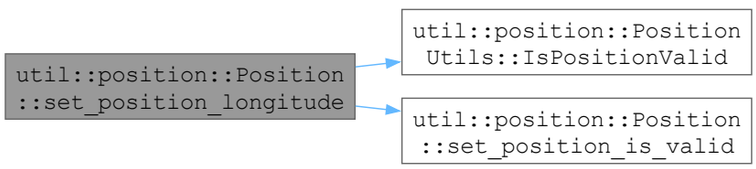
Parameters

in	*lon*	The longitude of the position, in decimal degrees.

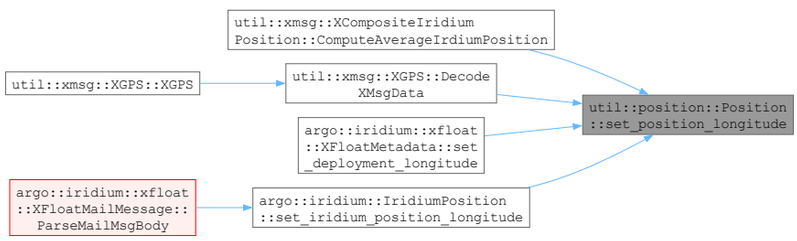
References util::position::PositionUtils::IsPositionValid(), util::position::Position::position_latitude_, util::position::Position::position_longitud and util::position::Position::set_position_is_valid().

Referenced by ComputeAverageIrdiumPosition(), util::xmsg::XGPS::DecodeXMsgData(), argo::iridium::xfloat::XFloatMetadata::set_deploy and argo::iridium::IridiumPosition::set_iridium_position_longitude().

Here is the call graph for this function:



Here is the caller graph for this function:



10.63.4.32 set_position_standard_error()

```
void util::xmsg::XCompositeIridiumPosition::set_position_standard_error (
            double position_standard_error)  [inline], [protected]
```

Set the calculated standard error of this composite Iridium position, in km.

Parameters

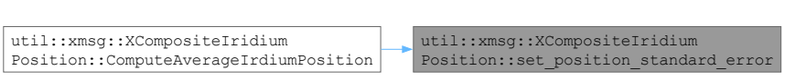
in	*position_standard_error*	The calculated standard error of this composite Iridium position, in km

References position_standard_error(), and position_standard_error_.

Referenced by ComputeAverageIrdiumPosition().

Here is the call graph for this function:

Here is the caller graph for this function:



10.63.4.33 set_position_variance()

```
void util::xmsg::XCompositeIridiumPosition::set_position_variance (
            double position_variance)  [inline], [protected]
```

Set the calculated variance of this composite Iridium position, in km^2.

Parameters

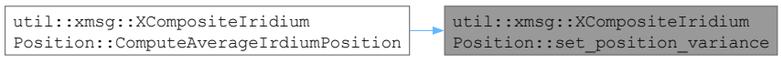
in	*position_variance*	The calculated variance of this composite Iridium position, in km^2

References position_variance(), and position_variance_.

Referenced by ComputeAverageIrdiumPosition().

Here is the call graph for this function:

Here is the caller graph for this function:



10.63.4.34 set_total_satellite_comm_time_seconds()

```
void util::xmsg::XCompositeIridiumPosition::set_total_satellite_comm_time_seconds (
            int time)  [inline], [protected]
```

Set the Iridium session communication time, in seconds.

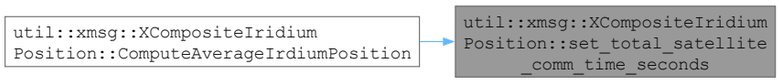
Parameters

in	*time*	The Iridium session communication time, in seconds.

References total_satellite_comm_time_seconds_.

Referenced by ComputeAverageIrdiumPosition().

Here is the caller graph for this function:



10.63.4.35 total_satellite_comm_time_seconds()

```
int util::xmsg::XCompositeIridiumPosition::total_satellite_comm_time_seconds () const  [inline]
```

Get the Iridium session communication time, in seconds.

Returns

The Iridium session communication time, in seconds.

References total_satellite_comm_time_seconds_.

10.63.5 Member Data Documentation

10.63.5.1 EARTH_RADIUS_KM_EQUATORIAL

```
const double util::position::PositionUtils::EARTH_RADIUS_KM_EQUATORIAL = 6378.1  [static], [private],
[inherited]
```

The equatorial radius of the earth, in kilometers.

Referenced by util::position::PositionUtils::CalculateEllipsoidHeight().

10.63.5.2 EARTH_RADIUS_KM_POLAR

```
const double util::position::PositionUtils::EARTH_RADIUS_KM_POLAR = 6356.1  [static], [private],
[inherited]
```

The polar radius of the earth, in kilometers.

Referenced by util::position::PositionUtils::CalculateEllipsoidHeight().

10.63.5.3 elements_

```
ElementVec_t util::xmsg::AbstractXComposite< shared_ptr< argo::iridium::xfloat::XFloatMailMessage
> >::elements_  [private], [inherited]
```

The vector of individual X-Messages elements that compose this composite object.

10.63.5.4 position_datetime_

```
ptime util::position::Position::position_datetime_  [private], [inherited]
```

The date and time of the position.

Referenced by util::position::Position::position_datetime(), util::position::Position::position_datetime_as_iso8601_string(), and util::position::Position::set_position_datetime().

10.63.5.5 position_is_valid_

```
bool util::position::Position::position_is_valid_  [private], [inherited]
```

The valid position indicator.

Referenced by util::position::Position::position_is_valid(), and util::position::Position::set_position_is_valid().

10.63.5.6 position_latitude_

```
double util::position::Position::position_latitude_  [private], [inherited]
```

The latitude of the position, in decimal degrees.

Referenced by util::position::Position::position_latitude(), util::position::Position::postition_as_iso6709_string(), util::position::Position::set_position_latitude(), and util::position::Position::set_position_longitude().

10.63.5.7 position_longitude_

```
double util::position::Position::position_longitude_  [private], [inherited]
```

The longitude of the position, in decimal degrees.

Referenced by util::position::Position::position_longitude(), util::position::Position::postition_as_iso6709_string(), util::position::Position::set_position_latitude(), and util::position::Position::set_position_longitude().

10.63.5.8 position_standard_error_

```
double util::xmsg::XCompositeIridiumPosition::position_standard_error_  [private]
```

The calculated standard error of this composite Iridium position, in km.

Referenced by GenerateListingString(), position_standard_error(), and set_position_standard_error().

10.63.5.9 position_variance_

```
double util::xmsg::XCompositeIridiumPosition::position_variance_  [private]
```

The calculated variance of this composite Iridium position, in km^2.

Referenced by GenerateListingString(), position_variance(), and set_position_variance().

10.63.5.10 total_satellite_comm_time_seconds_

```
int util::xmsg::XCompositeIridiumPosition::total_satellite_comm_time_seconds_  [private]
```

The Iridium session communication time, in seconds.

Note

This value is calculated from the difference in time between the first and last Iridium messages that are included in this composite Iridium position.

Referenced by GenerateListingString(), set_total_satellite_comm_time_seconds(), and total_satellite_comm_time_seconds().

The documentation for this class was generated from the following files:

- data/xmsg/XCompositeIridiumPosition.h
- data/XFloatMailMessage.cpp

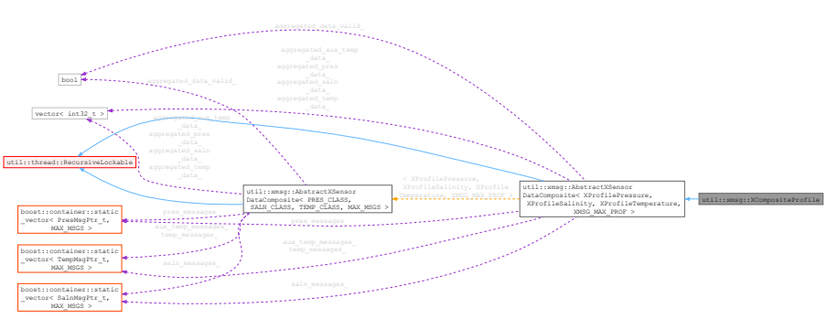
10.64 util::xmsg::XCompositeProfile Class Reference

Class representing the aggregate bin-averaged profile data for a single dive, composed from one or more bin-averaged profile data messages.

```
#include <XCompositeProfile.h>
```

Inheritance diagram for util::xmsg::XCompositeProfile:

Collaboration diagram for util::xmsg::XCompositeProfile:



Public Member Functions

- XCompositeProfile ()

  The default constructor.
- ~XCompositeProfile ()

  The destructor.
- void AssignAuxTemperatureMessage (const TempMsgPtr_t &x)

  Assign an auxiliary-channel temperature message and re-aggregate the sensor data.
- void AssignPressureMessage (const PresMsgPtr_t &x)

  Assign a pressure data message to this composite data object.
- void AssignSalinityMessage (const SalnMsgPtr_t &x)

  Assign a salinity data message to this composite data object.
- void AssignTemperatureMessage (const TempMsgPtr_t &x)

  Assign a temperature data message to this composite data object.
- void **CopyPressureDataToArray** (T dest[ ], size_t max_len, int16_t gain, int16_t offs, float pres_correction, T fill_value=XMSG_SENS_BAD_PRES) const
- void **CopySalinityDataToArray** (T dest[ ], size_t max_len, int16_t gain, int16_t offs, T fill_value=XMSG_SENS_BAD_SALN) const
- void **CopyTemperatureDataToArray** (T dest[ ], size_t max_len, int16_t gain, int16_t offs, T fill_↩ value=XMSG_SENS_BAD_TEMP) const
- const string GenerateListingString (int id_inst, int id_dive, uint16_t pg, uint16_t po, uint16_t sg, uint16_t so, uint16_t tg, uint16_t to, float pc) const

  Generate a listing string representation of the this composite sensor data object.
- const string GeneratePhyString (uint16_t pg, uint16_t po, uint16_t sg, uint16_t so, uint16_t tg, uint16_t to, float pc) const

  Generate a PHY entry string from this composite bin-averaged profile data object.
- bool **HasAuxTempData** () const
- bool HasContiguousMessageIndices () const

  Do the sensor data messages assigned to this composite data object have contiguous message indices?
- bool is_aggregated_data_valid () const

  Is the aggregated data valid?
- bool IsDataComplete () const

  Is the sensor data for this composite date message complete?
- bool IsEmpty () const

  Is this composite data object empty?
- bool LockObject (bool block)

  Lock the object.
- size_t max_num_samples () const

  Return the maximum of the number of pressure, temperature, or salinity sensor samples in the sensor data messages assigned to this composite sensor data object.
- size_t **num_aux_temp_samples** () const
- size_t num_pres_samples () const

  Get the total number of pressure sensor samples contained in the pressure data messages assigned to this composite data object.
- size_t num_saln_samples () const

  Get the total number of salinity sensor samples contained in the salinity data messages assigned to this composite data object.
- size_t num_temp_samples () const

  Get the total number of temperature sensor samples contained in the temperature data messages assigned to this composite data object.
- void Reset ()

  Remove the assigned pressure, temperature and salinity messages and their aggregated data.
- T **SampleAuxTempAsFloat** (size_t i, int16_t gain, int16_t offs) const
- int32_t SamplePres (size_t i) const

  Get the i^{th} pressure sensor value, in raw counts.
- T SamplePresAsFloat (size_t i, int16_t pg, int16_t po, float pc) const

  Get the i^{th} pressure sensor value, in dBar.
- int32_t SampleSaln (size_t i) const

  Get the i^{th} salinity sensor value, in raw counts.
- T SampleSalnAsFloat (size_t i, int16_t gain, int16_t offs) const

  Get the i^{th} salinity sensor value, in PSU.
- int32_t SampleTemp (size_t i) const

  Get the i^{th} temperature sensor value, in raw counts.
- T SampleTempAsFloat (size_t i, int16_t gain, int16_t offs) const

  Get the i^{th} temperature sensor value, in degC.
- void UnlockObject ()

  Unlock the object.

Protected Member Functions

- void AggregateSensorData () ATTR_NO_EXCEPTIONS

  Aggregate the sensor data for this composite sensor data object.
- void set_aggregated_data_valid (bool valid)

  Set the value of the aggregated_data_valid_ field.

Protected Attributes

- recursive_mutex mutex_

  Recursive mutual exclusion lock for synchronizing operations.

Private Types

- typedef shared_ptr< XProfilePressure > PresMsgPtr_t

  Type definition for a shared pointer to the pressure data message.
- typedef static_vector< PresMsgPtr_t, MAX_MSGS > PresMsgVector_t

  Type definition for a static vector (of size MAX_MSGS) of shared pointers to the pressure data message.
- typedef shared_ptr< XProfileSalinity > SalnMsgPtr_t

  Type definition for a shared pointer to the salinity data message.
- typedef static_vector< SalnMsgPtr_t, MAX_MSGS > SalnMsgVector_t

  Type definition for a static vector (of size MAX_MSGS) of shared pointers to the salinity data message.
- typedef shared_ptr< XProfileTemperature > TempMsgPtr_t

  Type definition for a shared pointer to the temperature data message.
- typedef static_vector< TempMsgPtr_t, MAX_MSGS > TempMsgVector_t

  Type definition for a static vector (of size MAX_MSGS) of shared pointers to the temperature data message.

Private Attributes

- vector< int32_t > **aggregated_aux_temp_data_**
- bool aggregated_data_valid_

  The aggregated data is valid flag.
- vector< int32_t > aggregated_pres_data_

  A vector of the aggregated pressure data samples, in raw counts.
- vector< int32_t > aggregated_saln_data_

  A vector of the aggregated salinity data samples, in raw counts.
- vector< int32_t > aggregated_temp_data_

  A vector of the aggregated temperature data samples, in raw counts.
- TempMsgVector_t **aux_temp_messages_**
- PresMsgVector_t pres_messages_

  A static vector (of size MAX_MSGS) containing pointers the pressure sensor data messages assigned to this composite object.
- SalnMsgVector_t saln_messages_

  A static vector (of size MAX_MSGS) containing pointers the salinity sensor data messages assigned to this composite object.
- TempMsgVector_t temp_messages_

  A static vector (of size MAX_MSGS) containing pointers the temperature sensor data messages assigned to this composite object.

10.64.1 Detailed Description

Class representing the aggregate bin-averaged profile data for a single dive, composed from one or more bin-averaged profile data messages.

10.64.2 Member Typedef Documentation

10.64.2.1 PresMsgPtr_t

```
typedef shared_ptr<XProfilePressure> util::xmsg::AbstractXSensorDataComposite< XProfilePressure,
XProfileSalinity, XProfileTemperature, MAX_MSGS >::PresMsgPtr_t  [private], [inherited]
```

Type definition for a shared pointer to the pressure data message.

10.64.2.2 PresMsgVector_t

typedef static_vector<PresMsgPtr_t, MAX_MSGS> util::xmsg::AbstractXSensorDataComposite< XProfilePressure, XProfileSalinity, XProfileTemperature, MAX_MSGS >::PresMsgVector_t [private], [inherited]

Type definition for a static vector (of size MAX_MSGS) of shared pointers to the pressure data message.

10.64.2.3 SalnMsgPtr_t

typedef shared_ptr<XProfileSalinity> util::xmsg::AbstractXSensorDataComposite< XProfilePressure, XProfileSalinity, XProfileTemperature, MAX_MSGS >::SalnMsgPtr_t [private], [inherited]

Type definition for a shared pointer to the salinity data message.

10.64.2.4 SalnMsgVector_t

typedef static_vector<SalnMsgPtr_t, MAX_MSGS> util::xmsg::AbstractXSensorDataComposite< XProfilePressure, XProfileSalinity, XProfileTemperature, MAX_MSGS >::SalnMsgVector_t [private], [inherited]

Type definition for a static vector (of size MAX_MSGS) of shared pointers to the salinity data message.

10.64.2.5 TempMsgPtr_t

typedef shared_ptr<XProfileTemperature> util::xmsg::AbstractXSensorDataComposite< XProfilePressure, XProfileSalinity, XProfileTemperature, MAX_MSGS >::TempMsgPtr_t [private], [inherited]

Type definition for a shared pointer to the temperature data message.

10.64.2.6 TempMsgVector_t

typedef static_vector<TempMsgPtr_t, MAX_MSGS> util::xmsg::AbstractXSensorDataComposite< XProfilePressure, XProfileSalinity, XProfileTemperature, MAX_MSGS >::TempMsgVector_t [private], [inherited]

Type definition for a static vector (of size MAX_MSGS) of shared pointers to the temperature data message.

10.64.3 Constructor & Destructor Documentation

10.64.3.1 XCompositeProfile()

util::xmsg::XCompositeProfile::XCompositeProfile () [inline]

The default constructor.

10.64.3.2 ~XCompositeProfile()

util::xmsg::XCompositeProfile::~XCompositeProfile () [inline]

The destructor.

10.64.4 Member Function Documentation

10.64.4.1 AggregateSensorData()

void util::xmsg::AbstractXSensorDataComposite< XProfilePressure, XProfileSalinity, XProfileTemperature, MAX_MSGS >::AggregateSensorData () [inline], [protected], [inherited]

Aggregate the sensor data for this composite sensor data object.

This function iterates over each of the sensor data messages assigned to this composite data object, and copies of the sensor data values into the corresponding sensor data vector.

If aggregated_data_valid_ is TRUE, then this function will have no effect. Otherwise, each of the aggregegated sensor data vectors will be cleared and the appropriate number of samples will be reserved.

Then, the sensor data message will be iterated over in order of increasing message indices, and the sensor data values will be copied into the corresponding sensor data vector in the appropriate order.

Finally, aggregated_data_valid_ will be set to FALSE to ensure that subsequent calls to this function will have no effect, until another sensor data message object is assigned to this composite object and aggregated_data_valid_ is set to TRUE.

Exceptions

util::log::MessageException	if an error occurs while aggregating the sensor data.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

10.64.4.2 AssignAuxTemperatureMessage()

```
void util::xmsg::AbstractXSensorDataComposite< XProfilePressure, XProfileSalinity, XProfileTemperature,
MAX_MSGS >::AssignAuxTemperatureMessage (
            const TempMsgPtr_t & x)  [inline], [inherited]
```

Assign an auxiliary-channel temperature message and re-aggregate the sensor data.

The message is inserted in order of message index, and the aggregated sensor data is rebuilt.

Parameters

in	*x*	The auxiliary-channel temperature message to assign.

Exceptions

util::log::MessageException	if x is NULL or an auxiliary temperature message with the same index has already been assigned.

10.64.4.3 AssignPressureMessage()

```
void util::xmsg::AbstractXSensorDataComposite< XProfilePressure, XProfileSalinity, XProfileTemperature,
MAX_MSGS >::AssignPressureMessage (
            const PresMsgPtr_t & x)  [inline], [inherited]
```

Assign a pressure data message to this composite data object.

The pressure data message will be emplaced into the sensor data message vector in order of increasing message indices. If a message with the same index is as x is already contained in the vector, then an exception will be thrown.

Parameters

in	*x*	A shared pointer to the pressure data message.

Exceptions

util::log::MessageException	if x is a null pointer, a duplicate message index is encountered, or any other error occurs.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

10.64.4.4 AssignSalinityMessage()

```
void util::xmsg::AbstractXSensorDataComposite< XProfilePressure, XProfileSalinity, XProfileTemperature,
MAX_MSGS >::AssignSalinityMessage (
            const SalnMsgPtr_t & x)  [inline], [inherited]
```

Assign a salinity data message to this composite data object.

The salinity data message will be emplaced into the sensor data message vector in order of increasing message indices. If a message with the same index is as x is already contained in the vector, then an exception will be thrown.

Parameters

in	*x*	A shared pointer to the salinity data message.

Exceptions

util::log::MessageException	if x is a null pointer, a duplicate message index is encountered, or any other error occurs.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

10.64.4.5 AssignTemperatureMessage()

```
void util::xmsg::AbstractXSensorDataComposite< XProfilePressure, XProfileSalinity, XProfileTemperature,
MAX_MSGS >::AssignTemperatureMessage (
            const TempMsgPtr_t & x)  [inline], [inherited]
```

Assign a temperature data message to this composite data object.

The temperature data message will be emplaced into the sensor data message vector in order of increasing message indices. If a message with the same index is as x is already contained in the vector, then an exception will be thrown.

Parameters

in	*x*	A shared pointer to the temperature data message.

Exceptions

util::log::MessageException	if x is a null pointer, a duplicate message index is encountered, or any other error occurs.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

10.64.4.6 GenerateListingString()

```
const string util::xmsg::XCompositeProfile::GenerateListingString (
            int id_inst,
            int id_dive,
            uint16_t pg,
            uint16_t po,
            uint16_t sg,
            uint16_t so,
            uint16_t tg,
            uint16_t to,
            float pc) const  [inline], [virtual]
```

Generate a listing string representation of the this composite sensor data object.

Parameters

in	*id_inst*	The unique instrument identifier for this composite sensor data object.
in	*id_dive*	The dive or cycle number for this composite data object.
in	*pg*	The pressure sensor sensitivity scale factor, in LSB/dBar.
in	*po*	The pressure sensor offset, in dBar.
in	*sg*	The salinity sensor sensitivity scale factor, in LSB/PSU.
in	*so*	The salinity sensor offset, in PSU.
in	*tg*	The temperature sensor sensitivity scale factor, in LSB/degC.
in	*to*	The temperature sensor offset, in degC.
in	*pc*	The pressure correction, in dBar, to be subtracted computed pressure value.

Exceptions

log::MessageException	if an error occurs while generating the listing string.
util::log::MessageException	if x if an error occurs while generating a listing string representation of this composite sensor data object.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

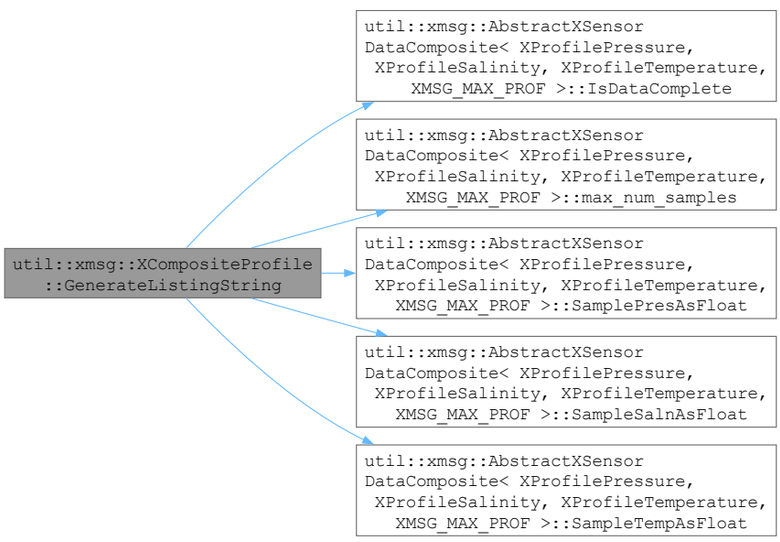
A listing string representation of this composite sensor data object.

Implements util::xmsg::AbstractXSensorDataComposite< XProfilePressure, XProfileSalinity, XProfileTemperature, XMSG_MAX_PROF >

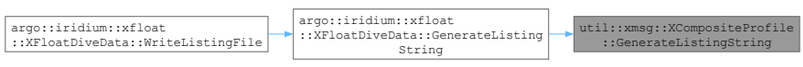
References CATCH_BLOCK, util::xmsg::AbstractXSensorDataComposite< XProfilePressure, XProfileSalinity, XProfileTemperature, XMS util::xmsg::AbstractXSensorDataComposite< XProfilePressure, XProfileSalinity, XProfileTemperature, XMSG_MAX_PROF >::max_num util::thread::RecursiveLockable::mutex_, util::xmsg::AbstractXSensorDataComposite< XProfilePressure, XProfileSalinity, XProfileTemper util::xmsg::AbstractXSensorDataComposite< XProfilePressure, XProfileSalinity, XProfileTemperature, XMSG_MAX_PROF >::SampleSa and util::xmsg::AbstractXSensorDataComposite< XProfilePressure, XProfileSalinity, XProfileTemperature, XMSG_MAX_PROF >::Samp

Referenced by argo::iridium::xfloat::XFloatDiveData::GenerateListingString().

Here is the call graph for this function:



Here is the caller graph for this function:



10.64.4.7 GeneratePhyString()

```
const string util::xmsg::XCompositeProfile::GeneratePhyString (
            uint16_t pg,
            uint16_t po,
            uint16_t sg,
            uint16_t so,
            uint16_t tg,
            uint16_t to,
            float pc) const  [inline]
```

Generate a PHY entry string from this composite bin-averaged profile data object.

Parameters

in	*pg*	The pressure sensor sensitivity scale factor, in LSB/dBar.
in	*po*	The pressure sensor offset, in dBar.
in	*sg*	The salinity sensor sensitivity scale factor, in LSB/PSU.
in	*so*	The salinity sensor offset, in PSU.
in	*tg*	The temperature sensor sensitivity scale factor, in LSB/degC.
in	*to*	The temperature sensor offset, in degC.
in	*pc*	The pressure correction, in dBar, to be subtracted computed pressure value.

Exceptions

log::MessageException	if an error occurs while generating the PHY string.

Returns

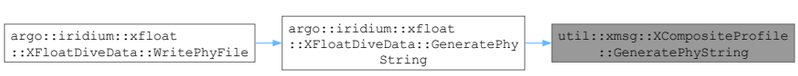
A PHY entry string representation of this composite bin-averaged profile data object.

References CATCH_BLOCK, util::xmsg::AbstractXSensorDataComposite< XProfilePressure, XProfileSalinity, XProfileTemperature, XMS
util::thread::RecursiveLockable::mutex_, util::xmsg::AbstractXSensorDataComposite< XProfilePressure, XProfileSalinity, XProfileTemper
util::xmsg::AbstractXSensorDataComposite< XProfilePressure, XProfileSalinity, XProfileTemperature, XMSG_MAX_PROF >::SamplePr
util::xmsg::AbstractXSensorDataComposite< XProfilePressure, XProfileSalinity, XProfileTemperature, XMSG_MAX_PROF >::SampleSa
and util::xmsg::AbstractXSensorDataComposite< XProfilePressure, XProfileSalinity, XProfileTemperature, XMSG_MAX_PROF >::Samp

Referenced by argo::iridium::xfloat::XFloatDiveData::GeneratePhyString().

Here is the call graph for this function:

Here is the caller graph for this function:



10.64.4.8 HasContiguousMessageIndices()

```
bool util::xmsg::AbstractXSensorDataComposite< XProfilePressure, XProfileSalinity, XProfileTemperature,
MAX_MSGS >::HasContiguousMessageIndices () const  [inline], [inherited]
```

Do the sensor data messages assigned to this composite data object have contiguous message indices?

This function is used, in part, to determine the sensor data is complete.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

TRUE if the sensor data messages have contiguous message indices or if there is less than two of each sensor data message, or FALSE otherwise.

10.64.4.9 is_aggregated_data_valid()

`bool util::xmsg::AbstractXSensorDataComposite< XProfilePressure, XProfileSalinity, XProfileTemperature, MAX_MSGS >::is_aggregated_data_valid () const [inline], [inherited]`

Is the aggregated data valid?

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

TRUE if the aggregated sensor is valid, or FALSE otherwise.

10.64.4.10 IsDataComplete()

`bool util::xmsg::AbstractXSensorDataComposite< XProfilePressure, XProfileSalinity, XProfileTemperature, MAX_MSGS >::IsDataComplete () const [inline], [inherited]`

Is the sensor data for this composite date message complete?

This function returns true if

- this composite sensor data object is not empty,
- the sensor data messages assigned to this composite data object have contiguous message indices,
- this sensor data object contains the same number of temperature and pressure sensor values, and
- the number of salinity data samples is either zero or equal to the number of pressure data samples.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

TRUE if the sensor data is complete, or FALSE otherwise.

Referenced by GenerateListingString().

Here is the caller graph for this function:

10.64.4.11 IsEmpty()

bool util::xmsg::AbstractXSensorDataComposite< XProfilePressure, XProfileSalinity, XProfileTemperature, MAX_MSGS >::IsEmpty () const [inline], [inherited]

Is this composite data object empty?

Returns

TRUE if empty, or FALSE otherwise.

10.64.4.12 LockObject()

bool util::thread::RecursiveLockable::LockObject (bool *block*) [inline], [inherited]

Lock the object.

Parameters

in	*block*	If TRUE, then the calling thread will block until the lock becomes available.

Exceptions

util::log::MessageException	if an error occurs.

Returns

TRUE if the lock was successfully obtained, or FALSE if block is FALSE and another thread owns the lock on the mutex.

References CATCH_BLOCK, and util::thread::RecursiveLockable::mutex_.

10.64.4.13 max_num_samples()

size_t util::xmsg::AbstractXSensorDataComposite< XProfilePressure, XProfileSalinity, XProfileTemperature, MAX_MSGS >::max_num_samples () const [inline], [inherited]

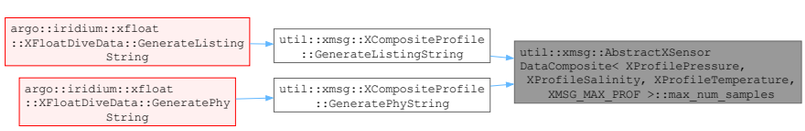
Return the maximum of the number of pressure, temperature, or salinity sensor samples in the sensor data messages assigned to this composite sensor data object.

Returns

The maximum of the number of pressure, temperature, or salinity sensor samples.

Referenced by GenerateListingString(), and GeneratePhyString().

Here is the caller graph for this function:



10.64.4.14 num_pres_samples()

size_t util::xmsg::AbstractXSensorDataComposite< XProfilePressure, XProfileSalinity, XProfileTemperature, MAX_MSGS >::num_pres_samples () const [inline], [inherited]

Get the total number of pressure sensor samples contained in the pressure data messages assigned to this composite data object.

Exceptions

util::log::MessageException	if an error occurs.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The total number of pressure sensor samples contained in the pressure data messages assigned to this composite data object.

10.64.4.15 num_saln_samples()

`size_t util::xmsg::AbstractXSensorDataComposite< XProfilePressure, XProfileSalinity, XProfileTemperature, MAX_MSGS >::num_saln_samples () const [inline], [inherited]`

Get the total number of salinity sensor samples contained in the salinity data messages assigned to this composite data object.

Exceptions

util::log::MessageException	if an error occurs.

Note

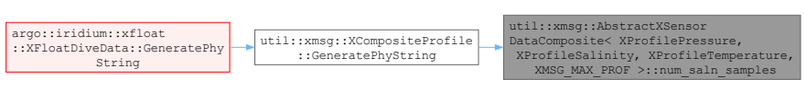
This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The total number of pressure sensor samples contained in the pressure data messages assigned to this composite data object.

Referenced by GeneratePhyString().

Here is the caller graph for this function:



10.64.4.16 num_temp_samples()

`size_t util::xmsg::AbstractXSensorDataComposite< XProfilePressure, XProfileSalinity, XProfileTemperature, MAX_MSGS >::num_temp_samples () const [inline], [inherited]`

Get the total number of temperature sensor samples contained in the temperature data messages assigned to this composite data object.

Exceptions

util::log::MessageException	if an error occurs.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The total number of pressure sensor samples contained in the pressure data messages assigned to this composite data object.

10.64.4.17 Reset()

```
void util::xmsg::AbstractXSensorDataComposite< XProfilePressure, XProfileSalinity, XProfileTemperature,
MAX_MSGS >::Reset ()  [inline], [inherited]
```

Remove the assigned pressure, temperature and salinity messages and their aggregated data.

Warning

The auxiliary temperature messages and aggregated auxiliary temperature data are not cleared, so re-assigning them after a reset fails with a duplicate index.

Exceptions

util::log::MessageException	if an error occurs.

10.64.4.18 SamplePres()

```
int32_t util::xmsg::AbstractXSensorDataComposite< XProfilePressure, XProfileSalinity, XProfileTemperature,
MAX_MSGS >::SamplePres (
            size_t i) const  [inline], [inherited]
```

Get the i^{th} pressure sensor value, in raw counts.

Parameters

in	*i*	The index of the pressure sensor sample to return. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD will be returned.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The i^{th} pressure sensor value, in raw counts, or XMSG_SENS_BAD if the index `i` exceeds the number of pressure sensor samples in the aggregated data vector.

10.64.4.19 SamplePresAsFloat()

```
T util::xmsg::AbstractXSensorDataComposite< XProfilePressure, XProfileSalinity, XProfileTemperature,
MAX_MSGS >::SamplePresAsFloat (
            size_t i,
            int16_t pg,
            int16_t po,
            float pc) const  [inline], [inherited]
```

Get the i^{th} pressure sensor value, in dBar.

Parameters

in	*i*	The index of the sensor value. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD_PRES will be returned.
in	*pg*	The pressure sensor sensitivity scale factor, in LSB/dBar.
in	*po*	The pressure sensor offset, in dBar.
in	*pc*	The pressure correction, in dBar, to be subtracted computed pressure value.

Note

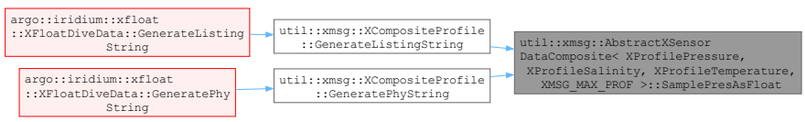
This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The i^{th} pressure sensor value, in dBar. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD_PRES will be returned.

Referenced by GenerateListingString(), and GeneratePhyString().

Here is the caller graph for this function:



10.64.4.20 SampleSaln()

```
int32_t util::xmsg::AbstractXSensorDataComposite< XProfilePressure, XProfileSalinity, XProfileTemperature,
MAX_MSGS >::SampleSaln (
            size_t i) const  [inline], [inherited]
```

Get the i^{th} salinity sensor value, in raw counts.

Parameters

in	*i*	The index of the salinity sensor sample to return. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD will be returned.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The i^{th} salinity sensor value, in raw counts, or XMSG_SENS_BAD if the index `i` exceeds the number of pressure sensor samples in the aggregated data vector.

10.64.4.21 SampleSalnAsFloat()

```
T util::xmsg::AbstractXSensorDataComposite< XProfilePressure, XProfileSalinity, XProfileTemperature,
MAX_MSGS >::SampleSalnAsFloat (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline], [inherited]
```

Get the i^{th} salinity sensor value, in PSU.

Parameters

in	*i*	The index of the sensor value. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD_SALN will be returned.
in	*gain*	The salinity sensor sensitivity scale factor, in LSB/PSU.
in	*offs*	The salinity sensor offset, in PSU.

Note

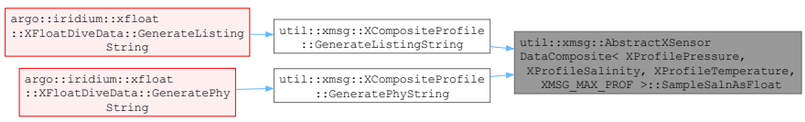
This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The i^{th} salinity sensor value, in PSU. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD_SALN will be returned.

Referenced by GenerateListingString(), and GeneratePhyString().

Here is the caller graph for this function:



10.64.4.22 SampleTemp()

```
int32_t util::xmsg::AbstractXSensorDataComposite< XProfilePressure, XProfileSalinity, XProfileTemperature,
MAX_MSGS >::SampleTemp (
            size_t i) const  [inline], [inherited]
```

Get the i^{th} temperature sensor value, in raw counts.

Parameters

in	*i*	The index of the temperature sensor sample to return. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD will be returned.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The i^{th} temperature sensor value, in raw counts, or XMSG_SENS_BAD if the index `i` exceeds the number of pressure sensor samples in the aggregated data vector.

10.64.4.23 SampleTempAsFloat()

```
T util::xmsg::AbstractXSensorDataComposite< XProfilePressure, XProfileSalinity, XProfileTemperature,
MAX_MSGS >::SampleTempAsFloat (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline], [inherited]
```

Get the i^{th} temperature sensor value, in degC.

Parameters

in	*i*	The index of the sensor value. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD_SALN will be returned.
in	*gain*	The temperature sensor sensitivity scale factor, in LSB/degC.
in	*offs*	The temperature sensor offset, in degC.

Note

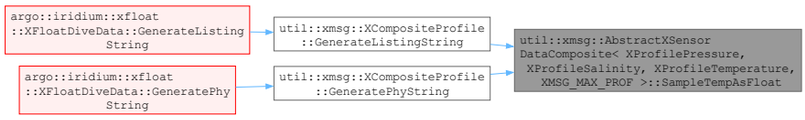
This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The i^{th} temperature sensor value, in degC. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD_TEMP will be returned.

Referenced by GenerateListingString(), and GeneratePhyString().

Here is the caller graph for this function:



10.64.4.24 set_aggregated_data_valid()

```
void util::xmsg::AbstractXSensorDataComposite< XProfilePressure, XProfileSalinity, XProfileTemperature,
MAX_MSGS >::set_aggregated_data_valid (
            bool valid)  [inline], [protected], [inherited]
```

Set the value of the aggregated_data_valid_ field.

Parameters

in	*valid*	TRUE if the aggregated data is valid, or FALSE otherwise.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

10.64.4.25 UnlockObject()

```
void util::thread::RecursiveLockable::UnlockObject ()  [inline], [inherited]
```

Unlock the object.

Exceptions

util::log::MessageException	if an error occurs.

References CATCH_BLOCK, and util::thread::RecursiveLockable::mutex_.

10.64.5 Member Data Documentation

10.64.5.1 aggregated_data_valid_

```
bool util::xmsg::AbstractXSensorDataComposite< XProfilePressure, XProfileSalinity, XProfileTemperature,
MAX_MSGS >::aggregated_data_valid_  [private], [inherited]
```

The aggregated data is valid flag.

This flag indicates the status of the aggregated sensor data. It is used to ensure the composite sensor data is aggregated only once following each assignment of sensor data to the composite data message.

This field is set to FALSE during initialization, and is set to FALSE each time a sensor data message is assigned to the composite data message. Each time AggregateSensorData() is called, the data will be aggregated, and this field will be set to TRUE.

10.64.5.2 aggregated_pres_data_

vector<int32_t> util::xmsg::AbstractXSensorDataComposite< XProfilePressure, XProfileSalinity, XProfileTemperature, MAX_MSGS >::aggregated_pres_data_ [private], [inherited]

A vector of the aggregated pressure data samples, in raw counts.

Each time the data is aggregated, the sensor data samples from the sensor data messages assigned to this composite objects are stored in this vector.

10.64.5.3 aggregated_saln_data_

vector<int32_t> util::xmsg::AbstractXSensorDataComposite< XProfilePressure, XProfileSalinity, XProfileTemperature, MAX_MSGS >::aggregated_saln_data_ [private], [inherited]

A vector of the aggregated salinity data samples, in raw counts.

Each time the data is aggregated, the sensor data samples from the sensor data messages assigned to this composite objects are stored in this vector.

10.64.5.4 aggregated_temp_data_

vector<int32_t> util::xmsg::AbstractXSensorDataComposite< XProfilePressure, XProfileSalinity, XProfileTemperature, MAX_MSGS >::aggregated_temp_data_ [private], [inherited]

A vector of the aggregated temperature data samples, in raw counts.

Each time the data is aggregated, the sensor data samples from the sensor data messages assigned to this composite object are stored in this vector.

10.64.5.5 mutex_

recursive_mutex util::thread::RecursiveLockable::mutex_ [mutable], [protected], [inherited]

Recursive mutual exclusion lock for synchronizing operations.

Referenced by util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AggregateS
argo::iridium::xfloat::XFloat::argo_instrument_type(), argo::iridium::xfloat::XFloatDiveData::argo_instrument_type(),
argo::iridium::xfloat::XFloatDiveData::AssignArgoData(), argo::iridium::xfloat::XFloatDiveData::AssignAuxTemperatureBinned(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignAuxTemperatureMe
argo::iridium::xfloat::XFloatDiveData::AssignDriftPressure(), argo::iridium::xfloat::XFloatDiveData::AssignDriftSalinity(),
argo::iridium::xfloat::XFloatDiveData::AssignDriftTemperatre(), argo::iridium::xfloat::XFloatDiveData::AssignEngineeringAbort(),
argo::iridium::xfloat::XFloatDiveData::AssignEngineeringAscent(), argo::iridium::xfloat::XFloatDiveData::AssignEngineeringBitFail(),
argo::iridium::xfloat::XFloatDiveData::AssignEngineeringBitPass(), argo::iridium::xfloat::XFloatDiveData::AssignEngineeringDescent(),
argo::iridium::xfloat::XFloatDiveData::AssignEngineeringDiagnostic(), argo::iridium::xfloat::XFloatDiveData::AssignEngineeringDrift(),
argo::iridium::xfloat::XFloatDiveData::AssignEngineeringIce(), argo::iridium::xfloat::XFloatDiveData::AssignEngineeringStandard(),
argo::iridium::xfloat::XFloatDiveData::AssignEngineeringSurface(), argo::iridium::xfloat::XFloatDiveData::AssignFallTimeSeries(),
argo::iridium::xfloat::XFloatDiveData::AssignGps(), argo::iridium::xfloat::XFloatDiveData::AssignMailMessage(),
argo::iridium::xfloat::XFloatDiveData::AssignMissionCmdEchoback(), argo::iridium::xfloat::XFloatDiveData::AssignParentInstrument(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignPressureMessage(
argo::iridium::xfloat::XFloatDiveData::AssignProfilePressureBinned(), argo::iridium::xfloat::XFloatDiveData::AssignProfilePressureUnbinn
argo::iridium::xfloat::XFloatDiveData::AssignProfileSalinityBinned(), argo::iridium::xfloat::XFloatDiveData::AssignProfileSalinityUnbinned()
argo::iridium::xfloat::XFloatDiveData::AssignProfileTemperatureBinned(), argo::iridium::xfloat::XFloatDiveData::AssignProfileTemperatureU
argo::iridium::xfloat::XFloatDiveData::AssignPumpTimeSeries(), argo::iridium::xfloat::XFloatDiveData::AssignRiseTimeSeries(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignSalinityMessage(),
argo::iridium::xfloat::XFloatDiveData::AssignSurfaceTimeSeries(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_C
argo::iridium::xfloat::XFloatDiveData::AssignValveData(), argo::iridium::xfloat::XFloatDiveData::CheckCompleteness(),
argo::iridium::xfloat::XFloatMetadata::comment(), argo::iridium::xfloat::XFloatDiveData::ComputePressureOffsetFromSurfaceTimeSeries(
argo::iridium::xfloat::XFloatMetadata::deployer(), argo::iridium::xfloat::XFloatMetadata::deployer_address(), argo::iridium::xfloat::XFloatMe
argo::iridium::xfloat::XFloatMetadata::deployment_datetime(), argo::iridium::xfloat::XFloatMetadata::deployment_datetime_status(),
argo::iridium::xfloat::XFloatMetadata::deployment_latitude(), argo::iridium::xfloat::XFloatMetadata::deployment_longitude(),
argo::iridium::xfloat::XFloatMetadata::deployment_platform(), argo::iridium::xfloat::XFloatMetadata::deployment_position_status(),
argo::iridium::xfloat::XFloatMetadata::deployment_type(), argo::iridium::xfloat::XFloat::fw_version(), argo::iridium::xfloat::XFloat::gain_pres
argo::iridium::xfloat::XFloat::gain_pressure_surface(), argo::iridium::xfloat::XFloat::gain_salinity(), argo::iridium::xfloat::XFloat::gain_salinit
argo::iridium::xfloat::XFloat::gain_temperature(), argo::iridium::xfloat::XFloat::gain_temperature_surface(), argo::iridium::xfloat::XFloatDive
argo::iridium::xfloat::XFloatDiveData::GenerateCompositeIridiumPosition(), argo::iridium::xfloat::XFloatDiveData::GenerateCompositePun
argo::iridium::xfloat::XFloatDiveData::GenerateCompositeRiseTimeSeriesData(), argo::iridium::xfloat::XFloatDiveData::GenerateListingSt

util::xmsg::XCompositeDrift::GenerateListingString(), util::xmsg::XCompositeHiResProfile::GenerateListingString(), GenerateListingString(), argo::iridium::xfloat::XFloatDiveData::GenerateOrderedMomsnList(), argo::iridium::xfloat::XFloatDiveData::Gener
util::xmsg::XCompositeDrift::GeneratePhyString(), util::xmsg::XCompositeHiResProfile::GeneratePhyString(), GeneratePhyString(),
argo::iridium::xfloat::XFloatDiveData::GetDiveDataMailMessagesFromDatabase(), argo::iridium::xfloat::XFloatDiveData::GetGainsAndOffs
argo::iridium::xfloat::XFloatDiveData::GetGpsPositionEndOfDive(), argo::iridium::xfloat::XFloatDiveData::GetSurfaceGainsAndOffsets(),
argo::iridium::xfloat::XFloatDiveData::HasBinnedProfileData(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLAS
argo::iridium::xfloat::XFloatDiveData::HasContiguousMomsns(), argo::iridium::xfloat::XFloatDiveData::HasContiguousPids(),
argo::iridium::xfloat::XFloatDiveData::HasDiveData(), argo::iridium::xfloat::XFloatDiveData::HasDriftData(), argo::iridium::xfloat::XFloatDiv
argo::iridium::xfloat::XFloatDiveData::HasFallRiseData(), argo::iridium::xfloat::XFloatDiveData::HasPositionData(),
argo::iridium::xfloat::XFloatDiveData::HasUnbinnedProfileData(), argo::iridium::xfloat::XFloatDiveData::HasValidGpsPosition(),
argo::iridium::Iridium::id_dac(), argo::iridium::xfloat::XFloatDiveData::id_dive(), argo::iridium::xfloat::XFloatMailMessage::id_dive(),
argo::iridium::Iridium::id_imei(), argo::iridium::Iridium::id_instrument(), argo::iridium::xfloat::XFloatMailMessage::id_msg(),
argo::iridium::xfloat::XFloatMailMessage::id_msg_uid(), argo::iridium::xfloat::XFloatMailMessage::id_sbd_momsn(),
argo::iridium::xfloat::XFloatMailMessage::id_sbd_mtmsn(), argo::iridium::xfloat::XFloatMailMessage::id_sbd_packet(),
argo::iridium::Iridium::id_wmo(), argo::iridium::xfloat::XFloatDiveData::intermediate_listing_file_path(), util::xmsg::AbstractXSensorDataCo
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::IsDataComplete(),
argo::iridium::xfloat::XFloatMetadata::local_path(), util::thread::RecursiveLockable::LockObject(), argo::iridium::xfloat::XFloatMetadata::ma
argo::iridium::xfloat::XFloat::metadata(), argo::iridium::xfloat::XFloatMailMessage::msg_body_is_valid(), argo::iridium::xfloat::XFloatMailM
argo::iridium::xfloat::XFloatMailMessage::msg_header_datetime(), argo::iridium::xfloat::XFloatMailMessage::msg_header_from(),
argo::iridium::xfloat::XFloatMailMessage::msg_header_is_valid(), argo::iridium::xfloat::XFloatMailMessage::msg_header_raw_text(),
argo::iridium::xfloat::XFloatMailMessage::msg_header_subject(), argo::iridium::xfloat::XFloatMailMessage::msg_is_valid(),
argo::iridium::xfloat::XFloatMailMessage::msg_raw_text(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, T
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::num_saln_samples(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::num_temp_samples(),
argo::iridium::xfloat::XFloat::offset_pressure(), argo::iridium::xfloat::XFloat::offset_pressure_surface(), argo::iridium::xfloat::XFloat::offset_
argo::iridium::xfloat::XFloat::offset_salinity_surface(), argo::iridium::xfloat::XFloat::offset_temperature(), argo::iridium::xfloat::XFloat::offset
argo::iridium::xfloat::XFloatMailMessage::ParseMailMsg(), argo::iridium::xfloat::XFloatMailMessage::ParseMailMsg(),
argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgBody(), argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgHeader(),
argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgSbdAttachment(), argo::iridium::xfloat::XFloat::phy_file_generation_is_enabled(),
argo::iridium::xfloat::XFloatDiveData::phy_file_path(), argo::iridium::xfloat::XFloatMetadata::principal_investigators(),
argo::iridium::xfloat::XFloatMetadata::principal_investigators_address(), argo::iridium::xfloat::XFloatMetadata::principal_investigators_orig
argo::iridium::xfloat::XFloatMailMessage::raw_file_local_path(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLA
argo::iridium::xfloat::XFloatDiveData::ResetData(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_C
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SamplePresAsFloat(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleSaln(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleSalnAsFloat(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleTemp(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleTempAsFloat(),
argo::iridium::xfloat::XFloatDiveData::SaveObjectToDatabase(), argo::iridium::xfloat::XFloatMailMessage::SaveObjectToDatabase(),
argo::iridium::xfloat::XFloatMetadata::SaveObjectToDatabase(), argo::iridium::xfloat::XFloatMailMessage::sbd_data(),
argo::iridium::xfloat::XFloatMailMessage::sbd_data_is_valid(), argo::iridium::xfloat::XFloatMailMessage::sbd_file_local_path(),
argo::iridium::xfloat::XFloatMailMessage::sbd_file_name(), argo::iridium::xfloat::XFloatMailMessage::sbd_file_size_bytes(),
argo::iridium::xfloat::XFloatMailMessage::sbd_session_status_code(), argo::iridium::xfloat::XFloatMailMessage::sbd_session_status_strin
argo::iridium::xfloat::XFloatDiveData::SensorGainsAndOffsetsAreKnown(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS,
argo::iridium::xfloat::XFloat::set_argo_instrument_type(), argo::iridium::xfloat::XFloatDiveData::set_argo_instrument_type(),
argo::iridium::xfloat::XFloatMetadata::set_comment(), argo::iridium::xfloat::XFloatMetadata::set_deployer(), argo::iridium::xfloat::XFloatMe
argo::iridium::xfloat::XFloatMetadata::set_deployment_cruise_id(), argo::iridium::xfloat::XFloatMetadata::set_deployment_datetime(),
argo::iridium::xfloat::XFloatMetadata::set_deployment_datetime_status(), argo::iridium::xfloat::XFloatMetadata::set_deployment_latitude()
argo::iridium::xfloat::XFloatMetadata::set_deployment_longitude(), argo::iridium::xfloat::XFloatMetadata::set_deployment_platform(),
argo::iridium::xfloat::XFloatMetadata::set_deployment_position_status(), argo::iridium::xfloat::XFloatMetadata::set_deployment_type(),
argo::iridium::xfloat::XFloat::set_fw_version(), argo::iridium::xfloat::XFloat::set_gain_pressure(), argo::iridium::xfloat::XFloat::set_gain_pre
argo::iridium::xfloat::XFloat::set_gain_salinity(), argo::iridium::xfloat::XFloat::set_gain_salinity_surface(), argo::iridium::xfloat::XFloat::set_g
argo::iridium::xfloat::XFloat::set_gain_temperature_surface(), argo::iridium::Iridium::set_id_dac(), argo::iridium::xfloat::XFloatDiveData::se
argo::iridium::xfloat::XFloatMailMessage::set_id_dive(), argo::iridium::Iridium::set_id_imei(), argo::iridium::Iridium::set_id_instrument(),
argo::iridium::xfloat::XFloatMailMessage::set_id_msg(), argo::iridium::xfloat::XFloatMailMessage::set_id_msg_uid(),
argo::iridium::xfloat::XFloatMailMessage::set_id_sbd_momsn(), argo::iridium::xfloat::XFloatMailMessage::set_id_sbd_mtmsn(),
argo::iridium::xfloat::XFloatMailMessage::set_id_sbd_packet(), argo::iridium::Iridium::set_id_wmo(), argo::iridium::xfloat::XFloatDiveData:
argo::iridium::xfloat::XFloatMetadata::set_local_path(), argo::iridium::xfloat::XFloatMetadata::set_manufacturer(),
argo::iridium::xfloat::XFloat::set_metadata(), argo::iridium::xfloat::XFloatMailMessage::set_msg_body_is_valid(),
argo::iridium::xfloat::XFloatMailMessage::set_msg_body_text(), argo::iridium::xfloat::XFloatMailMessage::set_msg_header_datetime(),
argo::iridium::xfloat::XFloatMailMessage::set_msg_header_from(), argo::iridium::xfloat::XFloatMailMessage::set_msg_header_is_valid(),
argo::iridium::xfloat::XFloatMailMessage::set_msg_header_raw_text(), argo::iridium::xfloat::XFloatMailMessage::set_msg_header_subjec
argo::iridium::xfloat::XFloatMailMessage::set_msg_is_valid(), argo::iridium::xfloat::XFloatMailMessage::set_msg_raw_text(),
argo::iridium::xfloat::XFloat::set_offset_pressure(), argo::iridium::xfloat::XFloat::set_offset_pressure_surface(), argo::iridium::xfloat::XFloa
argo::iridium::xfloat::XFloat::set_offset_salinity_surface(), argo::iridium::xfloat::XFloat::set_offset_temperature(),
argo::iridium::xfloat::XFloat::set_offset_temperature_surface(), argo::iridium::xfloat::XFloat::set_phy_file_generation_is_enabled(),
argo::iridium::xfloat::XFloatDiveData::set_phy_file_path(), argo::iridium::xfloat::XFloatMetadata::set_principal_investigators(),
argo::iridium::xfloat::XFloatMetadata::set_principal_investigators_address(), argo::iridium::xfloat::XFloatMetadata::set_principal_investiga
argo::iridium::xfloat::XFloatMailMessage::set_raw_file_local_path(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_data(),
argo::iridium::xfloat::XFloatMailMessage::set_sbd_data_is_valid(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_file_local_path(),
argo::iridium::xfloat::XFloatMailMessage::set_sbd_file_name(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_file_size_bytes(),
argo::iridium::xfloat::XFloatMailMessage::set_sbd_session_status_code(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_session_sta

argo::iridium::xfloat::XFloat::set_start_of_mission_datetime(), argo::iridium::xfloat::XFloatMetadata::set_wmo_instrument_type(), argo::iridium::xfloat::XFloatMetadata::set_wmo_recorder_type(), argo::iridium::xfloat::XFloat::set_xmsg_version_major(), argo::iridium::xfloat::XFloat::set_xmsg_version_minor(), argo::iridium::xfloat::XFloat::start_of_mission_datetime(), util::thread::RecursiveLockable::UnlockObject(), argo::iridium::xfloat::XFloatDiveData::UpdateDerivedSurfaceDataFields(), argo::iridium::xfloat::XFloatMetadata::wmo_instrument_type(), argo::iridium::xfloat::XFloatMetadata::wmo_recorder_type(), argo::iridium::xfloat::XFloatDiveData::WriteListingFile(), argo::iridium::xfloat::XFloatDiveData::WritePhyFile(), argo::iridium::xfloat::XFloatM argo::iridium::xfloat::XFloat::xmsg_version_major(), and argo::iridium::xfloat::XFloat::xmsg_version_minor().

10.64.5.6 pres_messages_

PresMsgVector_t util::xmsg::AbstractXSensorDataComposite< XProfilePressure, XProfileSalinity, XProfileTemperature, MAX_MSGS >::pres_messages_ [private], [inherited]

A static vector (of size MAX_MSGS) containing pointers the pressure sensor data messages assigned to this composite object.

Each time a pressure sensor data message is assigned to the this object, it will be emplaced in this vector in order of increasing message indices.

10.64.5.7 saln_messages_

SalnMsgVector_t util::xmsg::AbstractXSensorDataComposite< XProfilePressure, XProfileSalinity, XProfileTemperature, MAX_MSGS >::saln_messages_ [private], [inherited]

A static vector (of size MAX_MSGS) containing pointers the salinity sensor data messages assigned to this composite object.

Each time a salinity sensor data message is assigned to the this object, it will be emplaced in this vector in order of increasing message indices.

10.64.5.8 temp_messages_

TempMsgVector_t util::xmsg::AbstractXSensorDataComposite< XProfilePressure, XProfileSalinity, XProfileTemperature, MAX_MSGS >::temp_messages_ [private], [inherited]

A static vector (of size MAX_MSGS) containing pointers the temperature sensor data messages assigned to this composite object.

Each time a temperature sensor data message is assigned to the this object, it will be emplaced in this vector in order of increasing message indices.

The documentation for this class was generated from the following file:

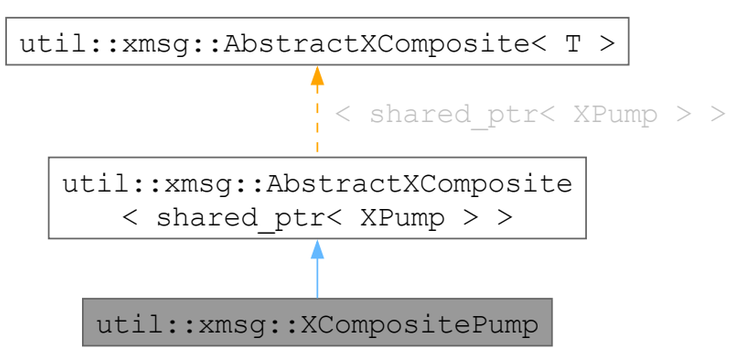
- data/xmsg/XCompositeProfile.h

10.65 util::xmsg::XCompositePump Class Reference

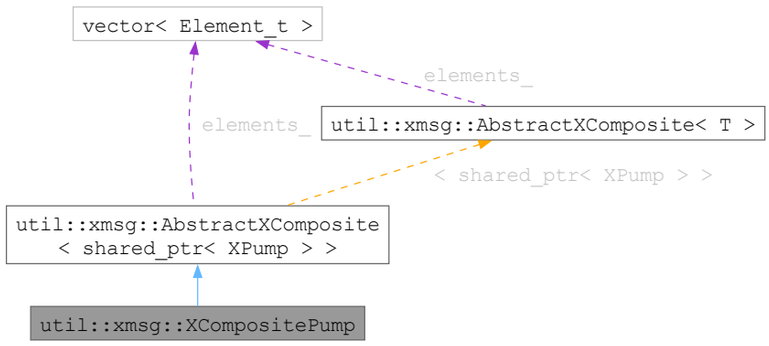
Class representing the aggregate pump time-series data for a single dive, composed from one or more pump time-series messages.

`#include <XCompositePump.h>`

Inheritance diagram for util::xmsg::XCompositePump:



Collaboration diagram for util::xmsg::XCompositePump:



Public Member Functions

- XCompositePump ()

  The default constructor.
- ~XCompositePump ()

  The destructor.
- void AppendAllElements (const static_vector< Element_t, I > &elements)

  Append a vector of elements to the vector of individual X-Messages elements that compose this composite object.
- virtual void AssignElement (const Element_t &element)

  Assign an element to this composite object.
- const vector< Element_t > & elements () const

  Get the vector of individual X-Messages elements that compose this composite object.
- const string GenerateListingString (uint16_t pg, uint16_t po, float pc) const

  Generate a listing entry string from this composite pump data object.
- virtual bool **IsEmpty** () const
- virtual size_t num_elements () const

  Get the number of individual elements that compose this composite object.

Protected Member Functions

- void AppendAllElements (const vector< Element_t > &elements)

  Append a vector of elements to the vector of individual X-Messages elements that compose this composite object.
- void AppendElement (const Element_t &element)

  Append one element to the vector of individual X-Messages elements that compose this composite object.

Private Types

- typedef shared_ptr< XPump > Element_t

  Type definition for the element aggregated by this object.
- typedef ElementVec_t::const_iterator **ElementConstIter_t**
- typedef ElementVec_t::iterator **ElementIter_t**
- typedef vector< Element_t > **ElementVec_t**

Private Attributes

- ElementVec_t elements_

  The vector of individual X-Messages elements that compose this composite object.

10.65.1 Detailed Description

Class representing the aggregate pump time-series data for a single dive, composed from one or more pump time-series messages.

10.65.2 Member Typedef Documentation

10.65.2.1 Element_t

```
typedef shared_ptr< XPump > util::xmsg::AbstractXComposite< shared_ptr< XPump > >::Element_↩
t  [private], [inherited]
```

Type definition for the element aggregated by this object.

10.65.3 Constructor & Destructor Documentation

10.65.3.1 XCompositePump()

```
util::xmsg::XCompositePump::XCompositePump ()  [inline]
```

The default constructor.

10.65.3.2 ~XCompositePump()

```
util::xmsg::XCompositePump::~XCompositePump ()  [inline]
```

The destructor.

10.65.4 Member Function Documentation

10.65.4.1 AppendAllElements() [1/2]

```
void util::xmsg::AbstractXComposite< shared_ptr< XPump > >::AppendAllElements (
            const static_vector< Element_t, I > & elements)  [inline], [inherited]
```

Append a vector of elements to the vector of individual X-Messages elements that compose this composite object.

Parameters

in	*elements*	The vector of elements to append.

10.65.4.2 AppendAllElements() [2/2]

```
void util::xmsg::AbstractXComposite< shared_ptr< XPump > >::AppendAllElements (
            const vector< Element_t > & elements)  [inline], [protected], [inherited]
```

Append a vector of elements to the vector of individual X-Messages elements that compose this composite object.

Parameters

in	*elements*	The vector of elements to append.

10.65.4.3 AppendElement()

```
void util::xmsg::AbstractXComposite< shared_ptr< XPump > >::AppendElement (
            const Element_t & element)  [inline], [protected], [inherited]
```

Append one element to the vector of individual X-Messages elements that compose this composite object.

Parameters

in	*element*	The element to append.

10.65.4.4 AssignElement()

```
virtual void util::xmsg::AbstractXComposite< shared_ptr< XPump > >::AssignElement (
            const Element_t & element)  [inline], [virtual], [inherited]
```

Assign an element to this composite object.

The default implementation appends the element; derived classes may override it to insert elements in order or reject duplicates.

Parameters

in	*element*	The element to assign.

10.65.4.5 elements()

```
const vector< Element_t > & util::xmsg::AbstractXComposite< shared_ptr< XPump > >::elements ()
const  [inline], [inherited]
```

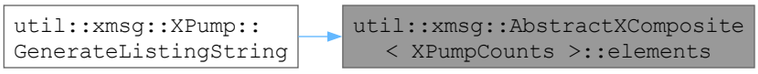
Get the vector of individual X-Messages elements that compose this composite object.

Returns

The vector of individual X-Messages elements that compose this composite object.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.65.4.6 GenerateListingString()

```
const string util::xmsg::XCompositePump::GenerateListingString (
            uint16_t pg,
            uint16_t po,
            float pc) const  [inline]
```

Generate a listing entry string from this composite pump data object.

Parameters

in	*pg*	The pressure sensor sensitivity scale factor, in LSB/dBar.
in	*po*	The pressure sensor offset, in dBar.
in	*pc*	The pressure correction, in dBar, to be subtracted computed pressure value.

Exceptions

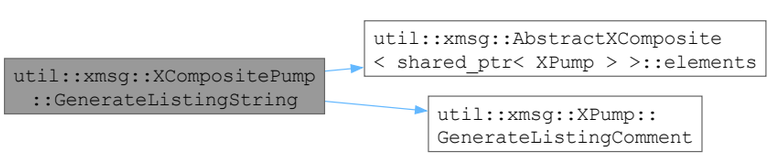
log::MessageException	if an error occurs while generating the listing string.

Returns

A listing entry string representation of this composite pump data object.

References CATCH_BLOCK, util::xmsg::AbstractXComposite< shared_ptr< XPump > >::elements(), util::xmsg::XPump::GenerateListin
and XMSG_SECTION_BREAK.

Here is the call graph for this function:



10.65.4.7 num_elements()

```
virtual size_t util::xmsg::AbstractXComposite< shared_ptr< XPump > >::num_elements () const
[inline], [virtual], [inherited]
```

Get the number of individual elements that compose this composite object.

Returns

The number of individual elements that compose this composite object.

10.65.5 Member Data Documentation

10.65.5.1 elements_

```
ElementVec_t util::xmsg::AbstractXComposite< shared_ptr< XPump > >::elements_  [private], [inherited]
```

The vector of individual X-Messages elements that compose this composite object.

The documentation for this class was generated from the following file:

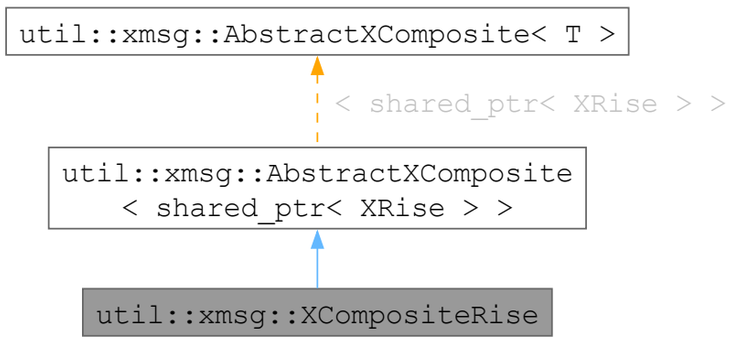
- data/xmsg/XCompositePump.h

10.66 util::xmsg::XCompositeRise Class Reference

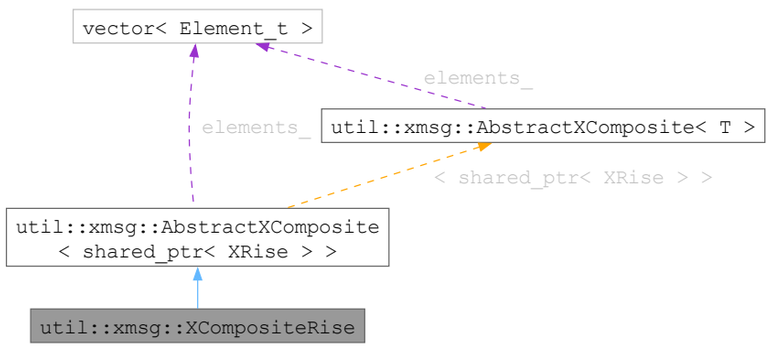
Class representing the aggregate pump data for a single dive, composed from one or more pump data messages.

```
#include <XCompositeRise.h>
```

Inheritance diagram for util::xmsg::XCompositeRise:



Collaboration diagram for util::xmsg::XCompositeRise:



Public Member Functions

- XCompositeRise ()

  The default constructor.
- ~XCompositeRise ()

  The destructor.
- void AppendAllElements (const static_vector< Element_t, I > &elements)

  Append a vector of elements to the vector of individual X-Messages elements that compose this composite object.
- virtual void AssignElement (const Element_t &element)

  Assign an element to this composite object.
- const vector< Element_t > & elements () const

  Get the vector of individual X-Messages elements that compose this composite object.
- const string GenerateListingString (uint16_t pg, uint16_t po, uint16_t sg, uint16_t so, uint16_t tg, uint16_t to, float pc) const

  Generate a listing entry string from this composite rise time-series data object.
- virtual bool **IsEmpty** () const
- virtual size_t num_elements () const

  Get the number of individual elements that compose this composite object.

Protected Member Functions

- void AppendAllElements (const vector< Element_t > &elements)

  Append a vector of elements to the vector of individual X-Messages elements that compose this composite object.
- void AppendElement (const Element_t &element)

  Append one element to the vector of individual X-Messages elements that compose this composite object.

Private Types

- typedef shared_ptr< XRise > Element_t

  Type definition for the element aggregated by this object.
- typedef ElementVec_t::const_iterator **ElementConstIter_t**
- typedef ElementVec_t::iterator **ElementIter_t**
- typedef vector< Element_t > **ElementVec_t**

Private Attributes

- ElementVec_t elements_

  The vector of individual X-Messages elements that compose this composite object.

10.66.1 Detailed Description

Class representing the aggregate pump data for a single dive, composed from one or more pump data messages.

10.66.2 Member Typedef Documentation

10.66.2.1 Element_t

```
typedef shared_ptr< XRise > util::xmsg::AbstractXComposite< shared_ptr< XRise > >::Element_↩
t  [private], [inherited]
```

Type definition for the element aggregated by this object.

10.66.3 Constructor & Destructor Documentation

10.66.3.1 XCompositeRise()

```
util::xmsg::XCompositeRise::XCompositeRise ()  [inline]
```

The default constructor.

10.66.3.2 ~XCompositeRise()

```
util::xmsg::XCompositeRise::~XCompositeRise ()  [inline]
```

The destructor.

10.66.4 Member Function Documentation

10.66.4.1 AppendAllElements() [1/2]

```
void util::xmsg::AbstractXComposite< shared_ptr< XRise > >::AppendAllElements (
            const static_vector< Element_t, I > & elements)  [inline], [inherited]
```

Append a vector of elements to the vector of individual X-Messages elements that compose this composite object.

Parameters

in	*elements*	The vector of elements to append.

10.66.4.2 AppendAllElements() [2/2]

```
void util::xmsg::AbstractXComposite< shared_ptr< XRise > >::AppendAllElements (
            const vector< Element_t > & elements)  [inline], [protected], [inherited]
```

Append a vector of elements to the vector of individual X-Messages elements that compose this composite object.

Parameters

in	*elements*	The vector of elements to append.

10.66.4.3 AppendElement()

```
void util::xmsg::AbstractXComposite< shared_ptr< XRise > >::AppendElement (
            const Element_t & element)  [inline], [protected], [inherited]
```

Append one element to the vector of individual X-Messages elements that compose this composite object.

Parameters

in	*element*	The element to append.

10.66.4.4 AssignElement()

```
virtual void util::xmsg::AbstractXComposite< shared_ptr< XRise > >::AssignElement (
            const Element_t & element)  [inline], [virtual], [inherited]
```

Assign an element to this composite object.

The default implementation appends the element; derived classes may override it to insert elements in order or reject duplicates.

Parameters

in	*element*	The element to assign.

10.66.4.5 elements()

```
const vector< Element_t > & util::xmsg::AbstractXComposite< shared_ptr< XRise > >::elements ()
const  [inline], [inherited]
```

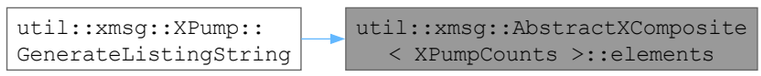
Get the vector of individual X-Messages elements that compose this composite object.

Returns

The vector of individual X-Messages elements that compose this composite object.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.66.4.6 GenerateListingString()

```
const string util::xmsg::XCompositeRise::GenerateListingString (
            uint16_t pg,
            uint16_t po,
            uint16_t sg,
            uint16_t so,
            uint16_t tg,
            uint16_t to,
            float pc) const  [inline]
```

Generate a listing entry string from this composite rise time-series data object.

Parameters

in	*pg*	The pressure sensor sensitivity scale factor, in LSB/dBar.
in	*po*	The pressure sensor offset, in dBar.
in	*sg*	The salinity sensor sensitivity scale factor, in LSB/PSU.
in	*so*	The salinity sensor offset, in PSU.
in	*tg*	The temperature sensor sensitivity scale factor, in LSB/degC.
in	*to*	The temperature sensor offset, in degC.
in	*pc*	The pressure correction, in dBar, to be subtracted computed pressure value.

Exceptions

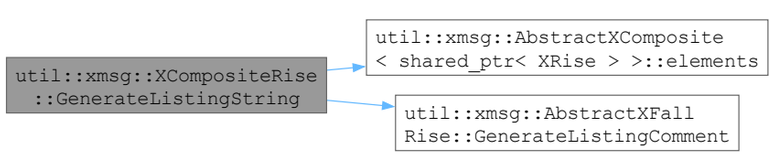
log::MessageException	if an error occurs while generating the listing string.

Returns

A listing entry string representation of this composite rise time-series data object.

References CATCH_BLOCK, util::xmsg::AbstractXComposite< shared_ptr< XRise > >::elements(), util::xmsg::AbstractXFallRise::Gene
and XMSG_SECTION_BREAK.

Here is the call graph for this function:



10.66.4.7 num_elements()

```
virtual size_t util::xmsg::AbstractXComposite< shared_ptr< XRise > >::num_elements () const
[inline], [virtual], [inherited]
```

Get the number of individual elements that compose this composite object.

Returns

The number of individual elements that compose this composite object.

10.66.5 Member Data Documentation

10.66.5.1 elements_

```
ElementVec_t util::xmsg::AbstractXComposite< shared_ptr< XRise > >::elements_  [private], [inherited]
```

The vector of individual X-Messages elements that compose this composite object.

The documentation for this class was generated from the following file:

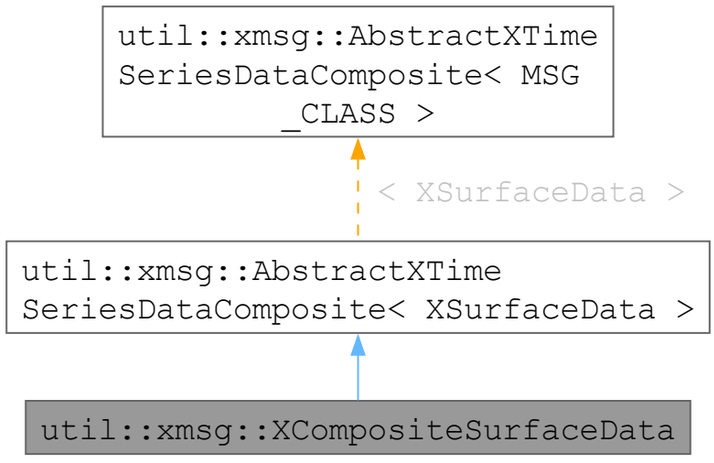
- data/xmsg/XCompositeRise.h

10.67 util::xmsg::XCompositeSurfaceData Class Reference

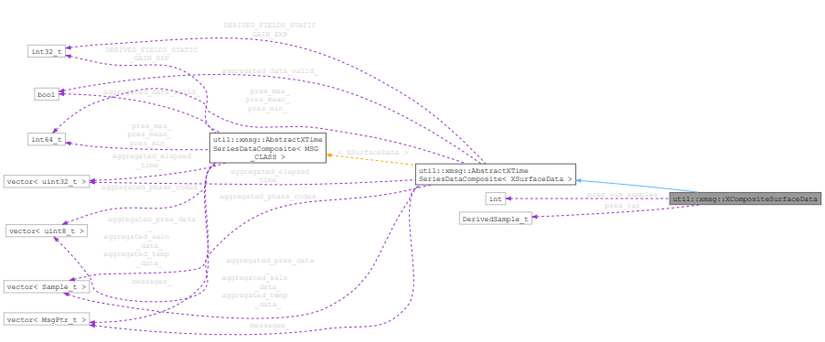
Class representing the aggregate surface pressure data for a single dive, composed from one or more surface pressure messages.

```
#include <XCompositeSurfaceData.h>
```

Inheritance diagram for util::xmsg::XCompositeSurfaceData:



Collaboration diagram for util::xmsg::XCompositeSurfaceData:



Public Types

- typedef int64_t **DerivedSample_t**
- typedef vector< uint32_t > **ElapsedTimeVector_t**
- typedef shared_ptr< XSurfaceData > **MsgPtr_t**
- typedef vector< MsgPtr_t > **MsgVector_t**
- typedef MsgVector_t::iterator **MsgVectorIter_t**
- typedef MsgVector_t::const_iterator **MsgVectorIterConst_t**
- typedef vector< uint8_t > **PhaseCodeVector_t**
- typedef int32_t **Sample_t**
- typedef vector< Sample_t > **SampleVector_t**

Public Member Functions

- XCompositeSurfaceData ()

  Default constructor.
- ~XCompositeSurfaceData ()

  Destructor.
- void AggregateSensorData ()

  Aggregate the surface time series and compute outlier-filtered pressure statistics.
- virtual void AssignMessage (const MsgPtr_t &x)

  Assign a time-series message to this composite and re-aggregate the time series.
- const int32_t **DERIVED_FIELDS_STATIC_GAIN_EXP**
- const string **GenerateDerivedFieldsListingString** (uint16_t pg, uint16_t po, float pres_correction) const
- virtual const string **GenerateListingString** (uint16_t &pg, uint16_t &po, uint16_t &sg, uint16_t &so, uint16_t &tg, uint16_t &to, float pres_correction) const
- const string GenerateListingString (uint16_t pg, uint16_t po, uint16_t sg, uint16_t so, uint16_t tg, uint16_t to, float pc) const

  Generate a listing entry string from this composite surface pressure time-series data object.
- virtual bool **HasContiguousMessageIndices** () const
- bool is_aggregated_data_valid () const

  Is the aggregated data valid?
- bool **IsDataComplete** () const
- bool IsEmpty () const

  Is this composite data object empty?
- size_t **NumPresSamples** () const
- size_t **NumSalnSamples** () const
- size_t **NumTempSamples** () const
- DerivedSample_t **pres_max** () const
- DerivedSample_t **pres_mean** () const
- DerivedSample_t **pres_min** () const
- int **pres_num_samples** () const
- DerivedSample_t **pres_var** () const
- T **PresMaxAsFloat** (uint16_t gain, uint16_t offs, float pres_correction=0) const
- T **PresMeanAsFloat** (uint16_t gain, uint16_t offs, float pres_correction=0) const
- T **PresMinAsFloat** (uint16_t gain, uint16_t offs, float pres_correction=0) const
- template<typename T = float>
  T **PresVarAsFloat** (uint16_t gain) const
- void Reset ()

  Remove all assigned messages and clear the aggregated data.
- void ResetAggregatedData ()

  Clear the aggregated time series and its derived pressure statistics.
- virtual size_t **SampleCount** () const
- int32_t SamplePres (size_t i) const

  Get the i^{th} pressure sensor value, in raw counts.
- T SamplePresAsFloat (size_t i, int16_t pg, int16_t po, float pc) const

  Get the i^{th} pressure sensor value, in dBar.
- int32_t SampleSaln (size_t i) const

  Get the i^{th} salinity sensor value, in raw counts.
- T SampleSalnAsFloat (size_t i, int16_t gain, int16_t offs) const

  Get the i^{th} salinity sensor value, in PSU.
- int32_t SampleTemp (size_t i) const

  Get the i^{th} temperature sensor value, in raw counts.
- T SampleTempAsFloat (size_t i, int16_t gain, int16_t offs) const

  Get the i^{th} temperature sensor value, in degC.

Static Public Member Functions

- static int32_t **derived_fields_static_gain_exp** ()

Static Public Attributes

- static const int32_t **DERIVED_FIELDS_STATIC_GAIN_EXP**

Protected Member Functions

- ElapsedTimeVector_t & **aggregated_elapsed_time** ()
- PhaseCodeVector_t & **aggregated_phase_codes** ()
- SampleVector_t & **aggregated_pres_data** ()
- SampleVector_t & **aggregated_saln_data** ()
- SampleVector_t & **aggregated_temp_data** ()
- const MsgVector_t & **messages** () const
- void set_aggregated_data_valid (bool valid)

  Set the value of the aggregated_data_valid_ field.
- void **set_pres_max** (DerivedSample_t pres_max)
- void **set_pres_mean** (DerivedSample_t pres_mean)
- void **set_pres_min** (DerivedSample_t pres_min)
- void **set_pres_num_samples** (int pres_num_samples)
- void **set_pres_var** (DerivedSample_t pres_var)

Private Attributes

- bool aggregated_data_valid_

  The aggregated data is valid flag.
- ElapsedTimeVector_t **aggregated_elapsed_time_**
- PhaseCodeVector_t **aggregated_phase_codes_**
- SampleVector_t **aggregated_pres_data_**
- SampleVector_t **aggregated_saln_data_**
- SampleVector_t **aggregated_temp_data_**
- MsgVector_t **messages_**
- DerivedSample_t **pres_max_**
- DerivedSample_t **pres_mean_**
- DerivedSample_t **pres_min_**
- int **pres_num_samples_**
- DerivedSample_t **pres_var_**

10.67.1 Detailed Description

Class representing the aggregate surface pressure data for a single dive, composed from one or more surface pressure messages.

10.67.2 Constructor & Destructor Documentation

10.67.2.1 XCompositeSurfaceData()

```
util::xmsg::XCompositeSurfaceData::XCompositeSurfaceData () [inline]
```

Default constructor.

10.67.2.2 ~XCompositeSurfaceData()

```
util::xmsg::XCompositeSurfaceData::~XCompositeSurfaceData () [inline]
```

Destructor.

10.67.3 Member Function Documentation

10.67.3.1 AggregateSensorData()

```
void util::xmsg::XCompositeSurfaceData::AggregateSensorData ()  [inline], [virtual]
```

Aggregate the surface time series and compute outlier-filtered pressure statistics.

First aggregates the messages as in AbstractXTimeSeriesDataComposite::AggregateSensorData(), then refines the surface pressure statistics in three passes over the valid samples (after the static gain):

1. compute the pressure variance about the mean;
2. discard samples more than two standard deviations from the mean, and recompute the maximum, minimum and mean from the rest;
3. recompute the variance from the remaining samples.

These statistics drive the surface pressure offset correction (see `XFloatDiveData::UpdateDerived↩SurfaceDataFields()`).

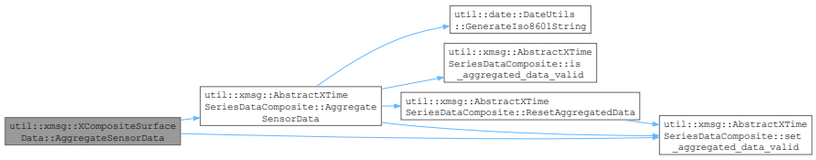
Exceptions

util::log::MessageException	if a pass finds no valid samples.

Reimplemented from util::xmsg::AbstractXTimeSeriesDataComposite< XSurfaceData >.

References util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::AggregateSensorData(), CATCH_BLOCK, EXC_THROW, ldbg, lwrn, util::xmsg::AbstractXTimeSeriesDataComposite< XSurfaceData >::set_aggregated_data_valid(), and XMSG_SENS_BAD.

Here is the call graph for this function:



10.67.3.2 AssignMessage()

```
virtual void util::xmsg::AbstractXTimeSeriesDataComposite< XSurfaceData >::AssignMessage (
            const MsgPtr_t & x)  [inline], [virtual], [inherited]
```

Assign a time-series message to this composite and re-aggregate the time series.

The message is inserted in order of message index, and `AggregateSensorData()` is called to rebuild the combined time series.

Parameters

in	*x*	The time-series message to assign.

Exceptions

util::log::MessageException	if `x` is NULL or a message with the same index has already been assigned.

10.67.3.3 GenerateListingString()

```
const string util::xmsg::XCompositeSurfaceData::GenerateListingString (
            uint16_t pg,
            uint16_t po,
            uint16_t sg,
            uint16_t so,
            uint16_t tg,
            uint16_t to,
            float pc) const  [inline]
```

Generate a listing entry string from this composite surface pressure time-series data object.

Parameters

in	*pg*	The pressure sensor sensitivity scale factor, in LSB/dBar.
in	*po*	The pressure sensor offset, in dBar.
in	*sg*	The salinity sensor sensitivity scale factor, in LSB/PSU.
in	*so*	The salinity sensor offset, in PSU.
in	*tg*	The temperature sensor sensitivity scale factor, in LSB/degC.
in	*to*	The temperature sensor offset, in degC.
in	*pc*	The pressure correction, in dBar, to be subtracted computed pressure value.

Exceptions

log::MessageException	if an error occurs while generating the listing string.

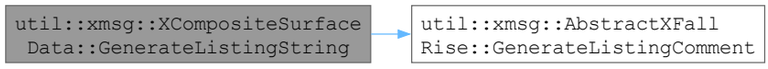
Returns

A listing entry string representation of this composite surface pressure time-series data object.

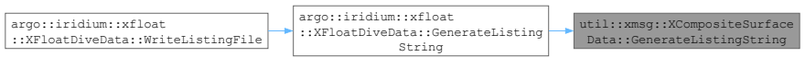
References CATCH_BLOCK, util::xmsg::AbstractXFallRise::GenerateListingComment(), and XMSG_SECTION.

Referenced by argo::iridium::xfloat::XFloatDiveData::GenerateListingString().

Here is the call graph for this function:



Here is the caller graph for this function:



10.67.3.4 is_aggregated_data_valid()

```
bool util::xmsg::AbstractXTimeSeriesDataComposite< XSurfaceData >::is_aggregated_data_valid ()
const  [inline], [inherited]
```

Is the aggregated data valid?

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

TRUE if the aggregated sensor is valid, or FALSE otherwise.

10.67.3.5 IsEmpty()

```
bool util::xmsg::AbstractXTimeSeriesDataComposite< XSurfaceData >::IsEmpty () const  [inline],
[inherited]
```

Is this composite data object empty?

Returns

TRUE if empty, or FALSE otherwise.

10.67.3.6 Reset()

```
void util::xmsg::AbstractXTimeSeriesDataComposite< XSurfaceData >::Reset ()  [inline], [inherited]
```

Remove all assigned messages and clear the aggregated data.

Exceptions

util::log::MessageException	if an error occurs.

10.67.3.7 ResetAggregatedData()

```
void util::xmsg::AbstractXTimeSeriesDataComposite< XSurfaceData >::ResetAggregatedData ()  [inline],
[inherited]
```

Clear the aggregated time series and its derived pressure statistics.

The assigned messages are kept; the next call to `AggregateSensorData()` rebuilds the aggregated data from them.

Exceptions

util::log::MessageException	if an error occurs.

10.67.3.8 SamplePres()

```
int32_t util::xmsg::AbstractXTimeSeriesDataComposite< XSurfaceData >::SamplePres (
            size_t i) const  [inline], [inherited]
```

Get the i^{th} pressure sensor value, in raw counts.

Parameters

in	*i*	The index of the pressure sensor sample to return. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD will be returned.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The i^{th} pressure sensor value, in raw counts, or XMSG_SENS_BAD if the index `i` exceeds the number of pressure sensor samples in the aggregated data vector.

10.67.3.9 SamplePresAsFloat()

```
T util::xmsg::AbstractXTimeSeriesDataComposite< XSurfaceData >::SamplePresAsFloat (
            size_t i,
            int16_t pg,
            int16_t po,
            float pc) const  [inline], [inherited]
```

Get the i^{th} pressure sensor value, in dBar.

Parameters

in	*i*	The index of the sensor value. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD_PRES will be returned.
in	*pg*	The pressure sensor sensitivity scale factor, in LSB/dBar.
in	*po*	The pressure sensor offset, in dBar.
in	*pc*	The pressure correction, in dBar, to be subtracted computed pressure value.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The i^{th} pressure sensor value, in dBar. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD_PRES will be returned.

10.67.3.10 SampleSaln()

```
int32_t util::xmsg::AbstractXTimeSeriesDataComposite< XSurfaceData >::SampleSaln (
            size_t i) const  [inline], [inherited]
```

Get the i^{th} salinity sensor value, in raw counts.

Parameters

in	*i*	The index of the salinity sensor sample to return. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD will be returned.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The i^{th} salinity sensor value, in raw counts, or XMSG_SENS_BAD if the index `i` exceeds the number of pressure sensor samples in the aggregated data vector.

10.67.3.11 SampleSalnAsFloat()

```
T util::xmsg::AbstractXTimeSeriesDataComposite< XSurfaceData >::SampleSalnAsFloat (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline], [inherited]
```

Get the i^{th} salinity sensor value, in PSU.

Parameters

in	*i*	The index of the sensor value. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD_SALN will be returned.
in	*gain*	The salinity sensor sensitivity scale factor, in LSB/PSU.
in	*offs*	The salinity sensor offset, in PSU.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The i^{th} salinity sensor value, in PSU. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD_SALN will be returned.

10.67.3.12 SampleTemp()

```
int32_t util::xmsg::AbstractXTimeSeriesDataComposite< XSurfaceData >::SampleTemp (
            size_t i) const  [inline], [inherited]
```

Get the i^{th} temperature sensor value, in raw counts.

Parameters

in	*i*	The index of the temperature sensor sample to return. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD will be returned.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The i^{th} temperature sensor value, in raw counts, or XMSG_SENS_BAD if the index `i` exceeds the number of pressure sensor samples in the aggregated data vector.

10.67.3.13 SampleTempAsFloat()

```
T util::xmsg::AbstractXTimeSeriesDataComposite< XSurfaceData >::SampleTempAsFloat (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline], [inherited]
```

Get the i^{th} temperature sensor value, in degC.

Parameters

in	*i*	The index of the sensor value. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD_SALN will be returned.
in	*gain*	The temperature sensor sensitivity scale factor, in LSB/degC.
in	*offs*	The temperature sensor offset, in degC.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The i^{th} temperature sensor value, in degC. If the index exceeds the number of values in the aggregated data vector, then XMSG_SENS_BAD_TEMP will be returned.

10.67.3.14 set_aggregated_data_valid()

```
void util::xmsg::AbstractXTimeSeriesDataComposite< XSurfaceData >::set_aggregated_data_valid (
            bool valid)  [inline], [protected], [inherited]
```

Set the value of the aggregated_data_valid_ field.

Parameters

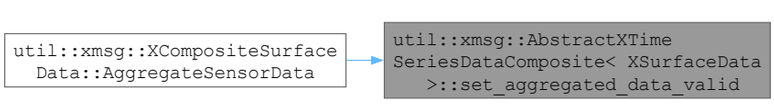
in	*valid*	TRUE if the aggregated data is valid, or FALSE otherwise.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Referenced by AggregateSensorData().

Here is the caller graph for this function:



10.67.4 Member Data Documentation

10.67.4.1 aggregated_data_valid_

bool util::xmsg::AbstractXTimeSeriesDataComposite< XSurfaceData >::aggregated_data_valid_ [private], [inherited]

The aggregated data is valid flag.

This flag indicates the status of the aggregated sensor data. It is used to ensure the composite sensor data is aggregated only once following each assignment of sensor data to the composite data message.

This field is set to FALSE during initialization, and is set to FALSE each time a sensor data message is assigned to the composite data message. Each time AggregateSensorData() is called, the data will be aggregated, and this field will be set to TRUE.

The documentation for this class was generated from the following file:

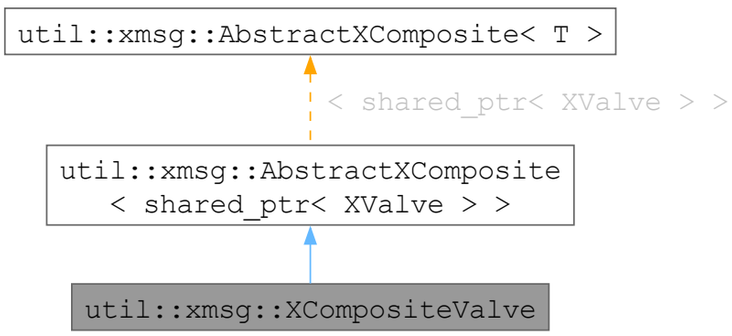
- data/xmsg/XCompositeSurfaceData.h

10.68 util::xmsg::XCompositeValve Class Reference

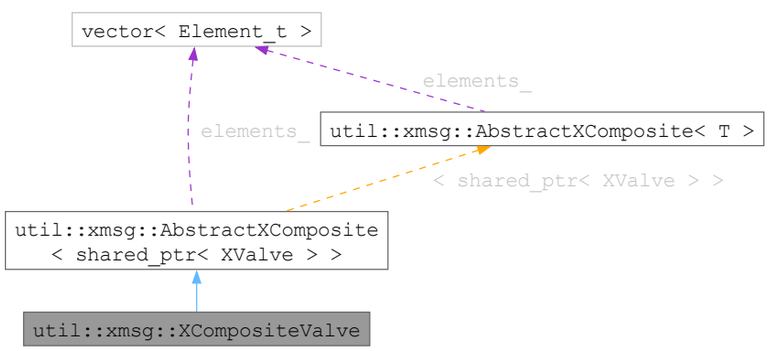
The composite valve data class.

#include <XCompositeValve.h>

Inheritance diagram for util::xmsg::XCompositeValve:



Collaboration diagram for util::xmsg::XCompositeValve:



Public Member Functions

- void AppendAllElements (const static_vector< Element_t, I > &elements)

  Append a vector of elements to the vector of individual X-Messages elements that compose this composite object.
- virtual void AssignElement (const Element_t &element)

  Assign an element to this composite object.
- const vector< Element_t > & elements () const

  Get the vector of individual X-Messages elements that compose this composite object.
- const string **GenerateListingString** (uint16_t pGain, uint16_t pOff, float pres_correction) const
- virtual bool **IsEmpty** () const
- virtual size_t num_elements () const

  Get the number of individual elements that compose this composite object.

Protected Member Functions

- void AppendAllElements (const vector< Element_t > &elements)

  Append a vector of elements to the vector of individual X-Messages elements that compose this composite object.
- void AppendElement (const Element_t &element)

  Append one element to the vector of individual X-Messages elements that compose this composite object.

Private Types

- typedef shared_ptr< XValve > Element_t

  Type definition for the element aggregated by this object.
- typedef ElementVec_t::const_iterator **ElementConstIter_t**
- typedef ElementVec_t::iterator **ElementIter_t**
- typedef vector< Element_t > **ElementVec_t**

Private Attributes

- ElementVec_t elements_

  The vector of individual X-Messages elements that compose this composite object.

10.68.1 Detailed Description

The composite valve data class.

An instance of this class aggregates the valve data messages (see XValve) received for a single dive.

10.68.2 Member Typedef Documentation

10.68.2.1 Element_t

```
typedef shared_ptr< XValve > util::xmsg::AbstractXComposite< shared_ptr< XValve > >::Element_t
[private], [inherited]
```

Type definition for the element aggregated by this object.

10.68.3 Member Function Documentation

10.68.3.1 AppendAllElements() [1/2]

```
void util::xmsg::AbstractXComposite< shared_ptr< XValve > >::AppendAllElements (
            const static_vector< Element_t, I > & elements)  [inline], [inherited]
```

Append a vector of elements to the vector of individual X-Messages elements that compose this composite object.

Parameters

in	*elements*	The vector of elements to append.

10.68.3.2 AppendAllElements() [2/2]

```
void util::xmsg::AbstractXComposite< shared_ptr< XValve > >::AppendAllElements (
            const vector< Element_t > & elements)  [inline], [protected], [inherited]
```

Append a vector of elements to the vector of individual X-Messages elements that compose this composite object.

Parameters

in	*elements*	The vector of elements to append.

10.68.3.3 AppendElement()

```
void util::xmsg::AbstractXComposite< shared_ptr< XValve > >::AppendElement (
            const Element_t & element)  [inline], [protected], [inherited]
```

Append one element to the vector of individual X-Messages elements that compose this composite object.

Parameters

in	*element*	The element to append.

10.68.3.4 AssignElement()

```
virtual void util::xmsg::AbstractXComposite< shared_ptr< XValve > >::AssignElement (
            const Element_t & element)  [inline], [virtual], [inherited]
```

Assign an element to this composite object.

The default implementation appends the element; derived classes may override it to insert elements in order or reject duplicates.

Parameters

in	*element*	The element to assign.

10.68.3.5 elements()

```
const vector< Element_t > & util::xmsg::AbstractXComposite< shared_ptr< XValve > >::elements ()
const  [inline], [inherited]
```

Get the vector of individual X-Messages elements that compose this composite object.

Returns

The vector of individual X-Messages elements that compose this composite object.

10.68.3.6 num_elements()

```
virtual size_t util::xmsg::AbstractXComposite< shared_ptr< XValve > >::num_elements () const
[inline], [virtual], [inherited]
```

Get the number of individual elements that compose this composite object.

Returns

The number of individual elements that compose this composite object.

10.68.4 Member Data Documentation

10.68.4.1 elements_

`ElementVec_t util::xmsg::AbstractXComposite< shared_ptr< XValve > >::elements_ [private], [inherited]`

The vector of individual X-Messages elements that compose this composite object.

The documentation for this class was generated from the following file:

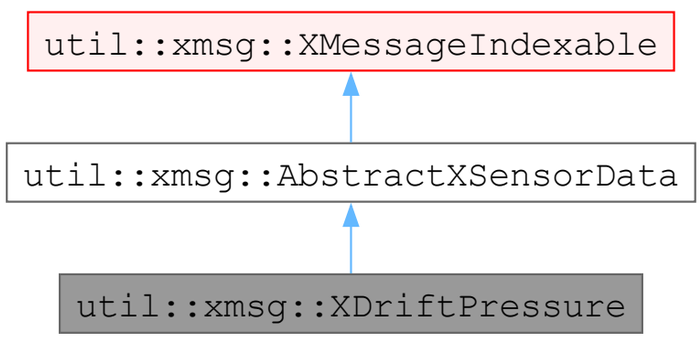
- data/xmsg/XCompositeValve.h

10.69 util::xmsg::XDriftPressure Class Reference

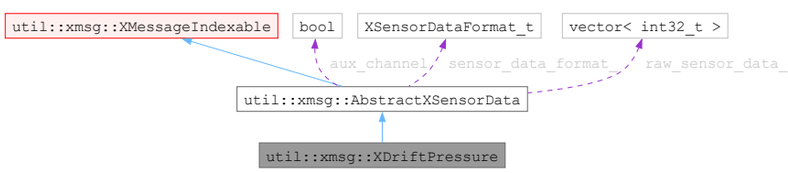
Class representing a drift pressure time-series X-Message.

`#include <XDriftPressure.h>`

Inheritance diagram for util::xmsg::XDriftPressure:



Collaboration diagram for util::xmsg::XDriftPressure:



Public Member Functions

- XDriftPressure (int16_t dive, uint16_t whoi, const vector< uint8_t > &data)
  Parameterized constructor.
- ~XDriftPressure ()
  Destructor.
- const uint8_t * CheckXMsgData (const vector< uint8_t > &data, int id, int minBytes, bool parseSize=true)
  Check the X-message for the correct ID, minimum size, and message terminator.
- void DecodeXMsgData (const vector< uint8_t > &data)
  Decode the drift pressure X-Message from binary data.
- int16_t id_dive () const
  Get the dive or cycle number of this X-Message.
- uint16_t id_instrument () const
  Get the unique instrument identifier of this X-Message.

- XMsgTypeId_t id_xmsg_type () const
  Get the enumerated X-Message type identifier.
- bool **is_aux_channel** () const
- size_t num_samples () const
  Get the number of samples in this message.
- const vector< int32_t > & raw_sensor_data () const
  Get the raw sensor data vector.
- int32_t Sample (size_t i) const
  Get the raw integer value of the i^{th} sample.
- template<typename T = double>
  T SampleValue (size_t i, int16_t gain, int16_t offs) const
  Get the i^{th} sensor value in scientific units.
- template<typename T = double>
  T SampleValueAsPressure (size_t i, int16_t gain, int16_t offs) const
  Get the i^{th} pressure sensor value, in dBar.
- template<typename T = double>
  T SampleValueAsSalinity (size_t i, int16_t gain, int16_t offs) const
  Get the i^{th} salinity sensor value, in PSU.
- template<typename T = double>
  T SampleValueAsTemperature (size_t i, int16_t gain, int16_t offs) const
  Get the i^{th} temperature sensor value, in degC.
- XSensorDataFormat_t sensor_data_format () const
  Get the enumerated sensor data format value.
- uint8_t xmsg_index () const
  Get the index of this X-Message.
- uint16_t xmsg_size_bytes () const
  Get the size of the X-Message, in bytes.

Protected Member Functions

- void set_id_dive (int16_t id_dive)
  Set the dive or cycle number of this message.
- void set_id_instrument (uint16_t id_instrument)
  Set the unique instrument identifier.
- void set_id_xmsg_type (XMsgTypeId_t id_xmsg_type)
  Set the X-Message type identifier.
- void set_raw_sensor_data (const vector< int32_t > &raw_sensor_data)
  Set raw integer counts of the measured sensor data.
- void set_sensor_data_format (XSensorDataFormat_t sensor_data_format)
  Set the sensor data format.
- void set_xmsg_index (uint8_t xmsg_index)
  Set the index of this X-Message.
- void set_xmsg_size_bytes (uint16_t xmsg_size_bytes)
  Set the size of the X-Message, in bytes.
- virtual vector< int32_t > UnpackSensorData (const vector< uint8_t > &data)
  Unpack the sensor data from the binary message.

Protected Attributes

- bool **aux_channel**
- vector< int32_t > raw_sensor_data_
  The raw integer counts of the measured sensor data.
- XSensorDataFormat_t sensor_data_format_
  The enumerated sensor data packing format.

Private Attributes

- int16_t id_dive_
  The dive or cycle number of this message.
- uint16_t id_instrument_
  The unique WHOI identifier of this instrument.
- XMsgTypeId_t id_xmsg_type_
  The X-message type identifier.
- uint8_t xmsg_index_
  The index of this X-Message.
- uint16_t xmsg_size_bytes_
  The size of the X message, in bytes.

10.69.1 Detailed Description

Class representing a drift pressure time-series X-Message.

10.69.2 Constructor & Destructor Documentation

10.69.2.1 XDriftPressure()

```
util::xmsg::XDriftPressure::XDriftPressure (
            int16_t dive,
            uint16_t whoi,
            const vector< uint8_t > & data)  [inline]
```

Parameterized constructor.

Parameters

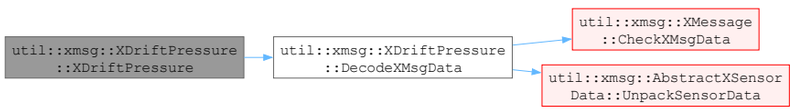
in	*dive*	The dive or cycle number of this message.
in	*whoi*	The unique WHOI identifier.
in	*data*	The binary X-message data to decode.

Exceptions

util::log::MessageException	if an decode error occurs.

References DecodeXMsgData().

Here is the call graph for this function:



10.69.2.2 ~XDriftPressure()

```
util::xmsg::XDriftPressure::~XDriftPressure ()  [inline]
```

Destructor.

10.69.3 Member Function Documentation

10.69.3.1 CheckXMsgData()

```
const uint8_t * util::xmsg::XMessage::CheckXMsgData (
            const vector< uint8_t > & data,
            int id,
            int minBytes,
            bool parseSize = true)  [inline], [inherited]
```

Check the X-message for the correct ID, minimum size, and message terminator.

Parameters

in	*data*	The X-message binary data to check.
in	*id*	The expected ID of the X-message. If this field is less that 0, then the ID check will be skipped.
in	*minBytes*	The minimum number of bytes expected in a valid message.
in	*parseSize*	If TRUE, then the message size field will be parsed from the X-Message. Otherwise, the size field will not be parsed.

Exceptions

util::log::MessageException	if • the message does not contain the expected ID, • the message does not contain the minimum number of expected bytes, • the message does not contain a terminator (';') in the correct location, or, • any other error occurs.

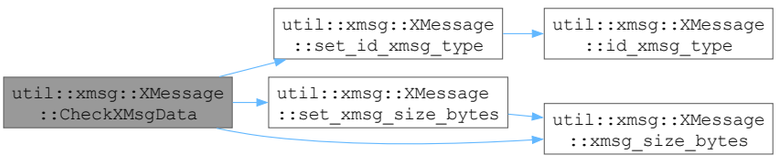
Returns

A pointer to the underlying data buffer of data.

References CATCH_BLOCK, EXC_THROW, lwrn, util::xmsg::XMessage::set_id_xmsg_type(), util::xmsg::XMessage::set_xmsg_size_byte
u16, and util::xmsg::XMessage::xmsg_size_bytes().

Referenced by util::xmsg::XAcousticPosition::DecodeXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), DecodeXMsgData(),
util::xmsg::XDriftSalinity::DecodeXMsgData(), util::xmsg::XDriftTemperature::DecodeXMsgData(), util::xmsg::XEngAbort::DecodeXMsgDa
util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngBIT::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(),
util::xmsg::XEngDiagnostic::DecodeXMsgData(), util::xmsg::XEngDrift::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(),
util::xmsg::XEngStandard::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XGPS::DecodeXMsgData(),
util::xmsg::XHiResPressure::DecodeXMsgData(), util::xmsg::XHiResSalinity::DecodeXMsgData(), util::xmsg::XHiResTemperature::Decod
util::xmsg::XMissionCmdEcho::DecodeXMsgData(), util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XProfilePressure::
util::xmsg::XProfileSalinity::DecodeXMsgData(), util::xmsg::XProfileTemperature::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgD
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the call graph for this function:



10.69.3.2 DecodeXMsgData()

```
void util::xmsg::XDriftPressure::DecodeXMsgData (
            const vector< uint8_t > & data)  [inline], [virtual]
```

Decode the drift pressure X-Message from binary data.

Parameters

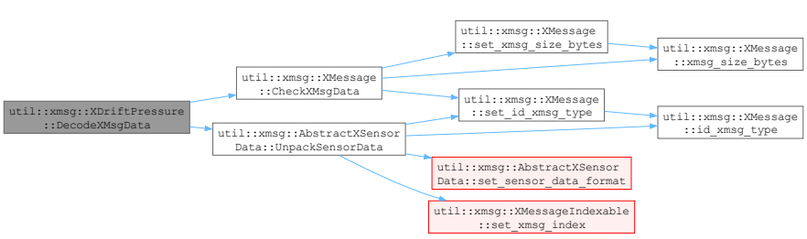
in	*data*	The binary data to decode.

Implements util::xmsg::AbstractXSensorData.

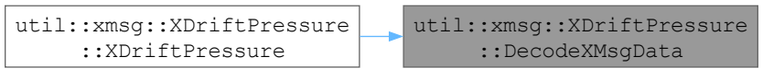
References CATCH_BLOCK, util::xmsg::XMessage::CheckXMsgData(), EXC_THROW, util::xmsg::AbstractXSensorData::UnpackSensorD
and util::xmsg::XMSG_TYPE_ID_PRES_DRIFT8.

Referenced by XDriftPressure().

Here is the call graph for this function:



Here is the caller graph for this function:



10.69.3.3 id_dive()

```
int16_t util::xmsg::XMessage::id_dive () const  [inline], [inherited]
```

Get the dive or cycle number of this X-Message.

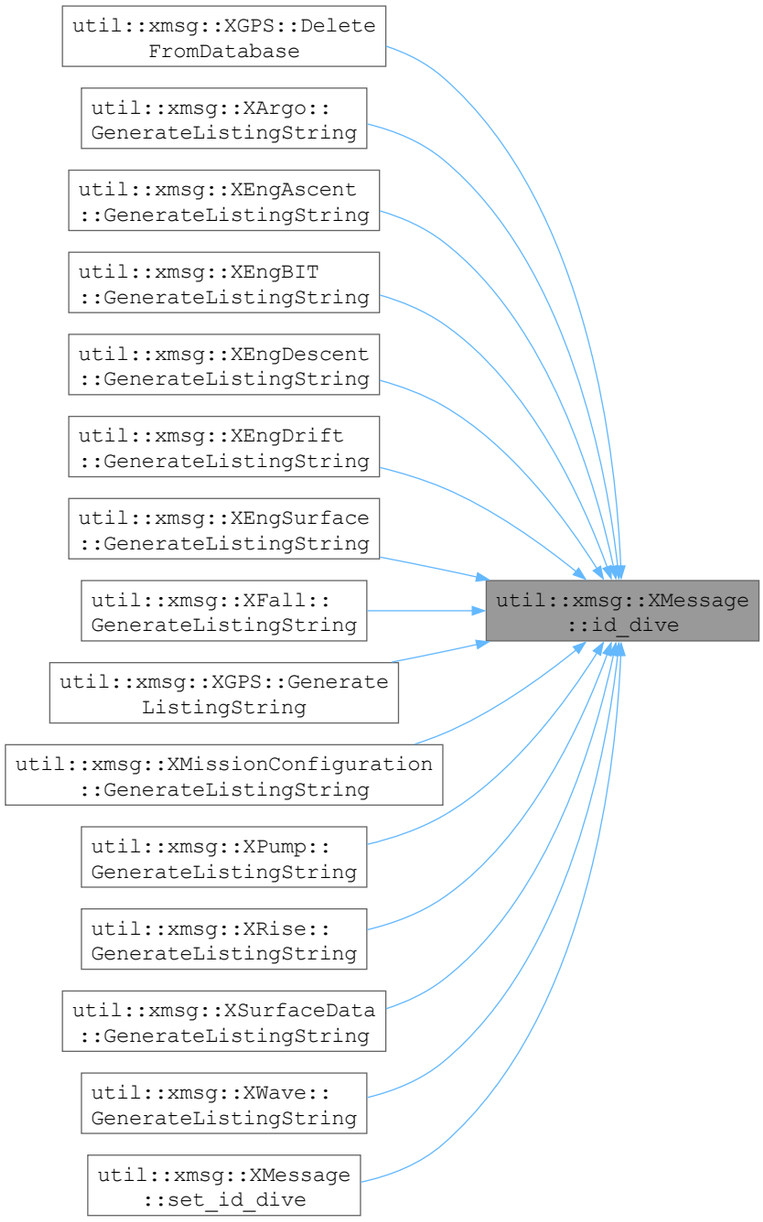
Returns

The dive or cycle number of this X-Message.

References util::xmsg::XMessage::id_dive_.

Referenced by util::xmsg::XGPS::DeleteFromDatabase(), util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateLis
util::xmsg::XEngBIT::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingStr
util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(),
util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing
util::xmsg::XSurfaceData::GenerateListingString(), util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_dive().

Here is the caller graph for this function:



10.69.3.4 id_instrument()

```
uint16_t util::xmsg::XMessage::id_instrument () const  [inline], [inherited]
```

Get the unique instrument identifier of this X-Message.

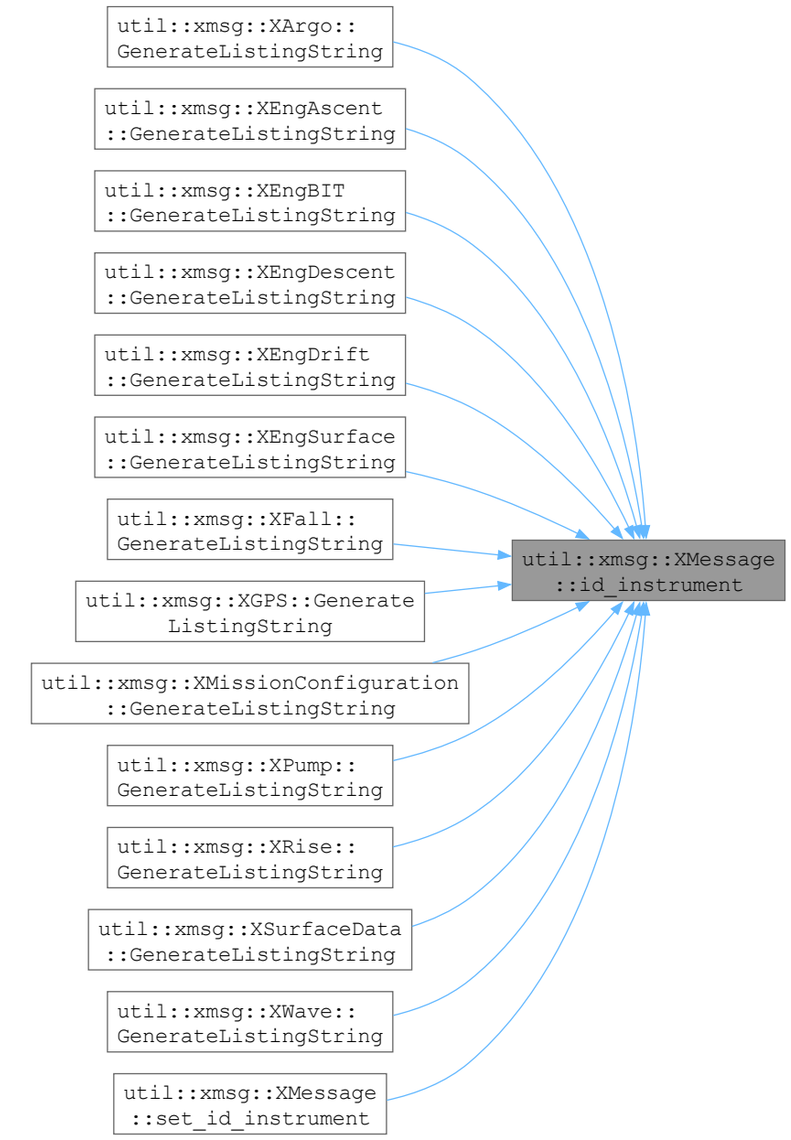
Returns

The unique instrument identifier of this X-Message.

References util::xmsg::XMessage::id_instrument_.

Referenced by util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngBIT::Generate
util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), util::xmsg::XEngSurface::GenerateListin
util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingS
util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(),
util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_instrument().

Here is the caller graph for this function:



10.69.3.5 id_xmsg_type()

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type () const  [inline], [inherited]
```

Get the enumerated X-Message type identifier.

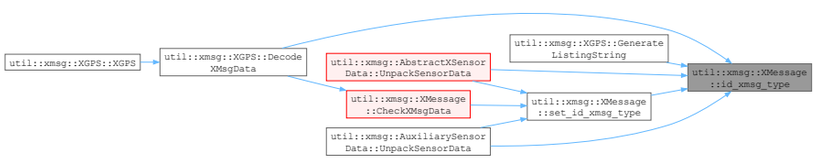
Returns

The enumerated X-Message type identifier.

References util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMessage::set_id_xmsg_typ
util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the caller graph for this function:



10.69.3.6 num_samples()

```
size_t util::xmsg::AbstractXSensorData::num_samples () const  [inline], [inherited]
```

Get the number of samples in this message.

Returns

The number of samples in this message.

References util::xmsg::AbstractXSensorData::raw_sensor_data_.

10.69.3.7 raw_sensor_data()

```
const vector< int32_t > & util::xmsg::AbstractXSensorData::raw_sensor_data () const  [inline],
[inherited]
```

Get the raw sensor data vector.

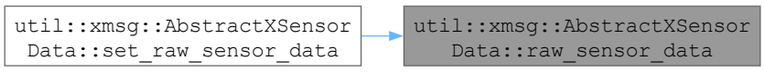
Returns

The raw sensor data vector.

References util::xmsg::AbstractXSensorData::raw_sensor_data_.

Referenced by util::xmsg::AbstractXSensorData::set_raw_sensor_data().

Here is the caller graph for this function:



10.69.3.8 Sample()

```
int32_t util::xmsg::AbstractXSensorData::Sample (
            size_t i) const  [inline], [inherited]
```

Get the raw integer value of the i^{th} sample.

Parameters

in	*i*	The index of the sensor value.

Returns

The raw integer value of the i^{th} ith sample.

References util::xmsg::AbstractXSensorData::raw_sensor_data_.

10.69.3.9 SampleValue()

```
template<typename T = double>
T util::xmsg::AbstractXSensorData::SampleValue (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline], [inherited]
```

Get the i^{th} sensor value in scientific units.

Parameters

in	*i*	The index of the sensor value.
in	*gain*	The sensor sensitivity scale factor.
in	*offs*	The sensor offset.

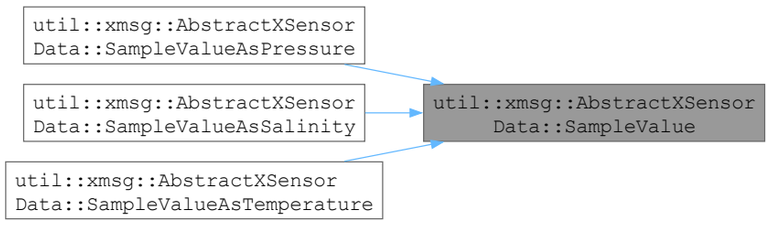
Returns

The i^{th} sensor value in scientific units.

References CATCH_BLOCK, and util::xmsg::AbstractXSensorData::raw_sensor_data_.

Referenced by util::xmsg::AbstractXSensorData::SampleValueAsPressure(), util::xmsg::AbstractXSensorData::SampleValueAsSalinity(), and util::xmsg::AbstractXSensorData::SampleValueAsTemperature().

Here is the caller graph for this function:



10.69.3.10 SampleValueAsPressure()

```
template<typename T = double>
T util::xmsg::AbstractXSensorData::SampleValueAsPressure (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline], [inherited]
```

Get the i^{th} pressure sensor value, in dBar.

Parameters

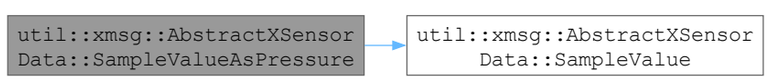
in	*i*	The index of the sensor value.
in	*gain*	The pressure sensor sensitivity scale factor, in LSB/dBar.
in	*offs*	The pressure sensor offset, in dBar.

Returns

The i^{th} pressure sensor value, in dBar.

References util::xmsg::AbstractXSensorData::SampleValue(), XMSG_SENS_BAD_PRES, and XMSG_SENS_MAX_PRES.

Here is the call graph for this function:



10.69.3.11 SampleValueAsSalinity()

```
template<typename T = double>
T util::xmsg::AbstractXSensorData::SampleValueAsSalinity (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline], [inherited]
```

Get the i^{th} salinity sensor value, in PSU.

Parameters

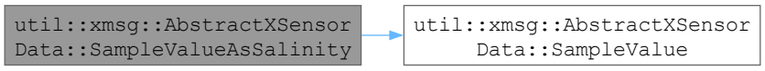
in	*i*	The index of the sensor value.
in	*gain*	The salinity sensor sensitivity scale factor, in LSB/PSU.
in	*offs*	The salinity sensor offset, in PSU.

Returns

The i^{th} salinity sensor value, in PSU.

References util::xmsg::AbstractXSensorData::SampleValue(), XMSG_SENS_BAD_SALN, and XMSG_SENS_MAX_SALN.

Here is the call graph for this function:



10.69.3.12 SampleValueAsTemperature()

```
template<typename T = double>
T util::xmsg::AbstractXSensorData::SampleValueAsTemperature (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline], [inherited]
```

Get the i^{th} temperature sensor value, in degC.

Parameters

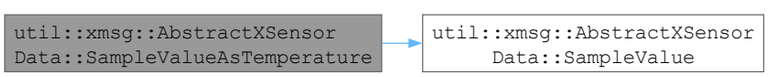
in	*i*	The index of the sensor value.
in	*gain*	The temperature sensor sensitivity scale factor, in LSB/degC.
in	*offs*	The temperature sensor offset, in degC.

Returns

The i^{th} temperature sensor value, in degC.

References util::xmsg::AbstractXSensorData::SampleValue(), XMSG_SENS_BAD_TEMP, and XMSG_SENS_MAX_TEMP.

Here is the call graph for this function:



10.69.3.13 sensor_data_format()

```
XSensorDataFormat_t util::xmsg::AbstractXSensorData::sensor_data_format () const  [inline], [inherited]
```

Get the enumerated sensor data format value.

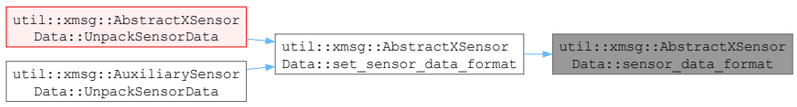
Returns

The enumerated sensor data format value.

References util::xmsg::AbstractXSensorData::sensor_data_format_.

Referenced by util::xmsg::AbstractXSensorData::set_sensor_data_format().

Here is the caller graph for this function:



10.69.3.14 set_id_dive()

```
void util::xmsg::XMessage::set_id_dive (
            int16_t id_dive)  [inline], [protected], [inherited]
```

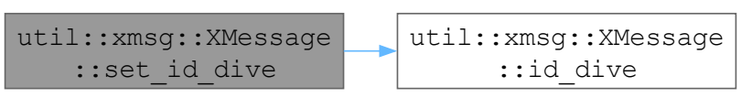
Set the dive or cycle number of this message.

Parameters

in	*id_dive*	The dive or cycle number of this message to set.

References util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::id_dive_.

Here is the call graph for this function:



10.69.3.15 set_id_instrument()

```
void util::xmsg::XMessage::set_id_instrument (
            uint16_t id_instrument)  [inline], [protected], [inherited]
```

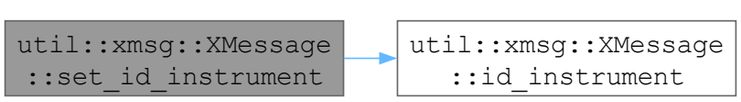
Set the unique instrument identifier.

Parameters

in	*id_instrument*	The unique instrument identifier to set.

References util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::id_instrument_.

Here is the call graph for this function:



10.69.3.16 set_id_xmsg_type()

```
void util::xmsg::XMessage::set_id_xmsg_type (
            XMsgTypeId_t id_xmsg_type)  [inline], [protected], [inherited]
```

Set the X-Message type identifier.

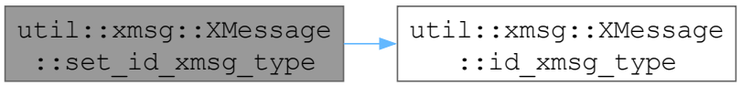
Parameters

in	*id_xmsg_type*	The X-Message type identifier to set.

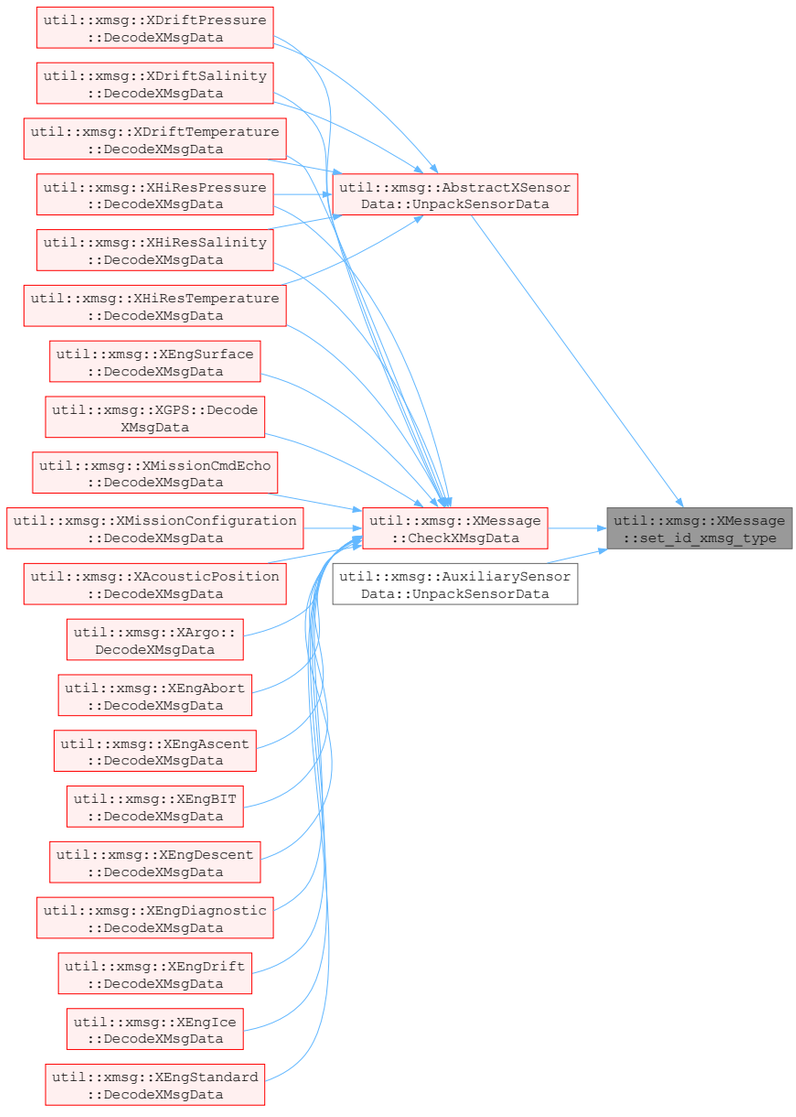
References util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.69.3.17 set_raw_sensor_data()

```
void util::xmsg::AbstractXSensorData::set_raw_sensor_data (
            const vector< int32_t > & raw_sensor_data)  [inline], [protected], [inherited]
```

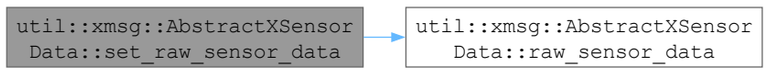
Set raw integer counts of the measured sensor data.

Parameters

in	*raw_sensor_data*	The data to set.

References util::xmsg::AbstractXSensorData::raw_sensor_data(), and util::xmsg::AbstractXSensorData::raw_sensor_data_.

Here is the call graph for this function:



10.69.3.18 set_sensor_data_format()

```
void util::xmsg::AbstractXSensorData::set_sensor_data_format (
            XSensorDataFormat_t sensor_data_format)  [inline], [protected], [inherited]
```

Set the sensor data format.

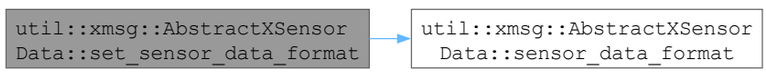
Parameters

in	*sensor_data_format*	The sensor data format to set.

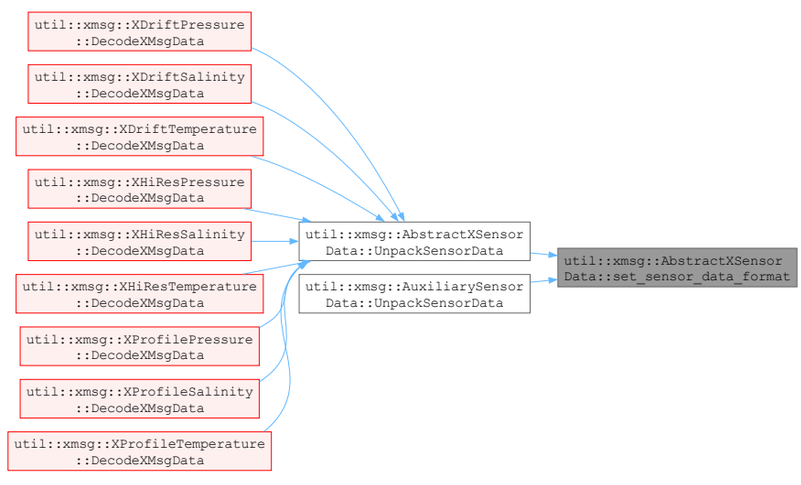
References util::xmsg::AbstractXSensorData::sensor_data_format(), and util::xmsg::AbstractXSensorData::sensor_data_format_.

Referenced by util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.69.3.19 set_xmsg_index()

```
void util::xmsg::XMessageIndexable::set_xmsg_index (
            uint8_t xmsg_index)  [inline], [protected], [inherited]
```

Set the index of this X-Message.

Parameters

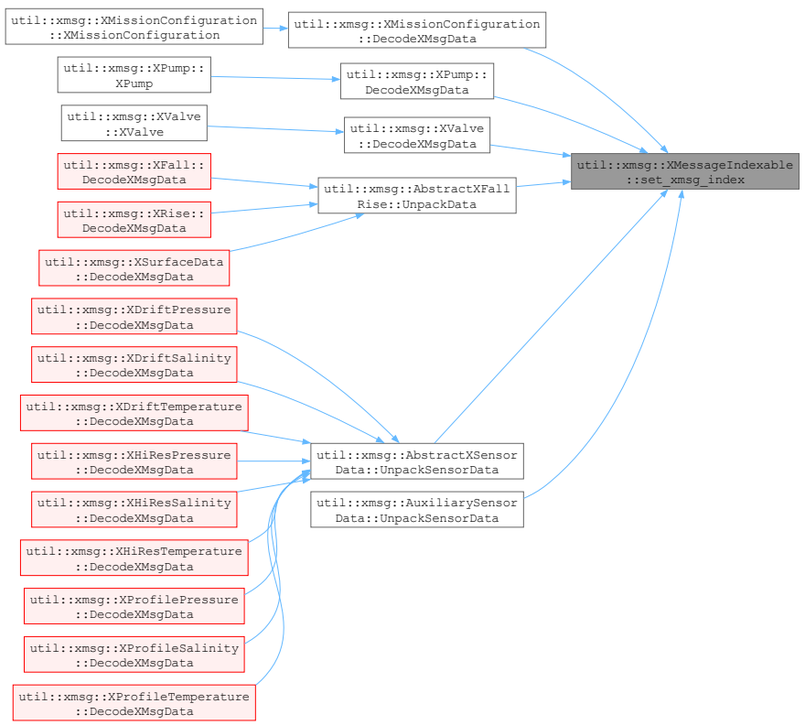
in	*xmsg_index*	The index of this X-Message.

References util::xmsg::XMessageIndexable::xmsg_index(), and util::xmsg::XMessageIndexable::xmsg_index_.

Referenced by util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(), util::xmsg::XValve::DecodeXMsgData(), util::xmsg::AbstractXFallRise::UnpackData(), util::xmsg::AbstractXSensorData::UnpackSensorDa and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:

Here is the caller graph for this function:



10.69.3.20 set_xmsg_size_bytes()

```
void util::xmsg::XMessage::set_xmsg_size_bytes (
            uint16_t xmsg_size_bytes)  [inline], [protected], [inherited]
```

Set the size of the X-Message, in bytes.

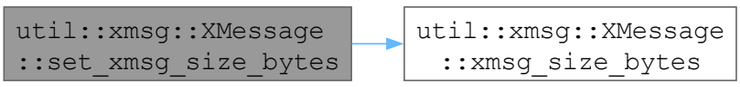
Parameters

in	*xmsg_size_bytes*	The size of the X-Message to set, in bytes.

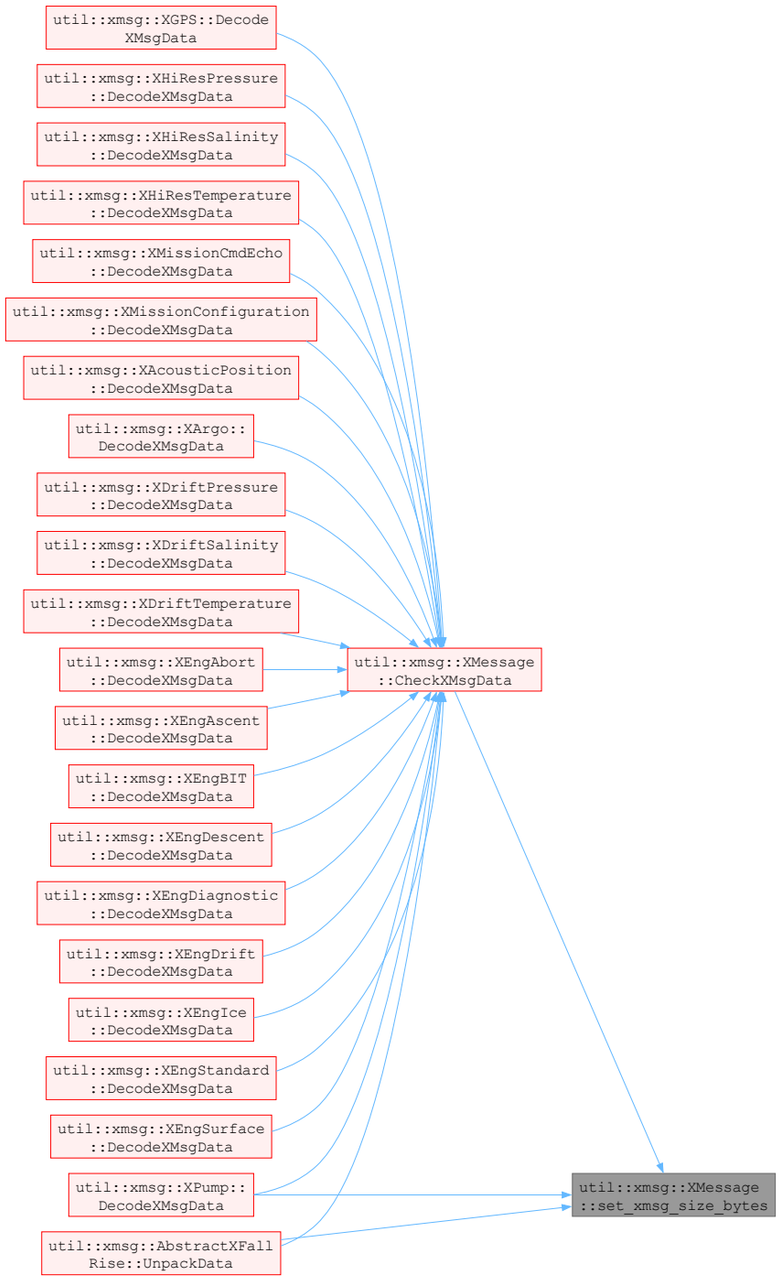
References util::xmsg::XMessage::xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XPump::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::Unpa

Here is the call graph for this function:



Here is the caller graph for this function:



10.69.3.21 UnpackSensorData()

```
virtual vector< int32_t > util::xmsg::AbstractXSensorData::UnpackSensorData (
            const vector< uint8_t > & data)  [inline], [protected], [virtual], [inherited]
```

Unpack the sensor data from the binary message.

This field unpacks the encoded sensor data from the message and stores the decoded sensor values in raw_sensor_data_. First the function checks the size of the message, and the data packing format. The message data is decoded according to the packing format type. Both first-difference and curvature encoding are supported. The encoding method is determined from the value of the high nibble of byte `data`[1].

Once this function completes successfully, then the data may be recovered in scientific units from the raw sensor values by applying the appropriate sensor gain and offset values.

Parameters

in	*data*	The binary message to unpack, in a vector of unsigned bytes.

Warning

The function assumes the message data is valid. Message validity should be ensured before this function is called.

Exceptions

util::log::MessageException	if • an error occurs while decoding the message, • the message is of insufficient length, or • an invalid packing format is encountered.

Returns

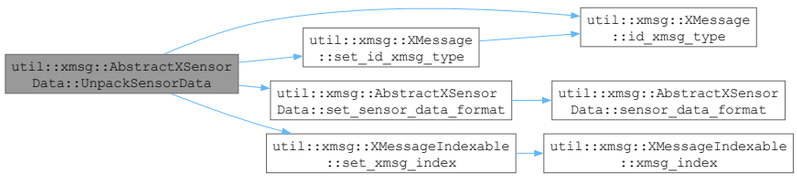
The sensor data raw counts, in a vector of signed integers.

Reimplemented in util::xmsg::AuxiliarySensorData.

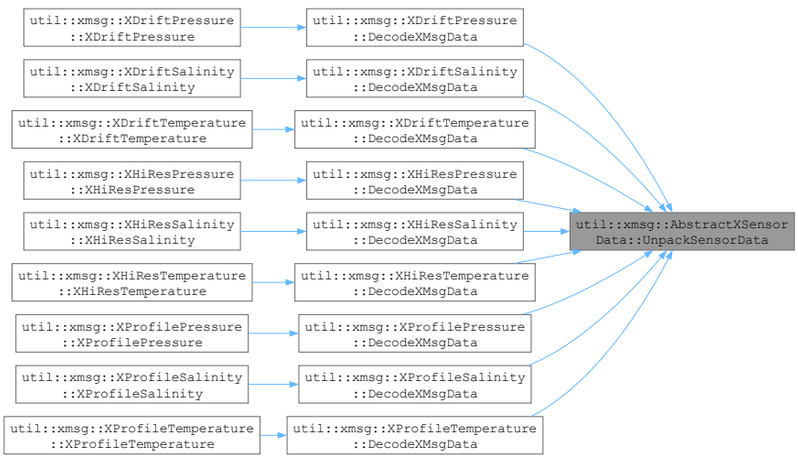
References CATCH_BLOCK, CATCH_NOTHROW, EXC_THROW, i24, util::xmsg::XMessage::id_xmsg_type(), ldbg, lwrn, util::xmsg::AbstractXSensorData::raw_sensor_data_, util::xmsg::XMessage::set_id_xmsg_type(), util::xmsg::AbstractXSensorData::s util::xmsg::XMessageIndexable::set_xmsg_index(), u16, u24, XMSG_SENS_MAX_SUB, util::xmsg::XMSG_TYPE_ID_PRES0, util::xmsg::XMSG_TYPE_ID_PRES_DRIFT8, util::xmsg::XMSG_TYPE_ID_PRES_HI_RES0, util::xmsg::XMSG_TYPE_ID_SALN0, util::xmsg::XMSG_TYPE_ID_SALN_DRIFT8, util::xmsg::XMSG_TYPE_ID_SALN_HI_RES0, util::xmsg::XMSG_TYPE_ID_TEMP0, util::xmsg::XMSG_TYPE_ID_TEMP_DRIFT8, util::xmsg::XMSG_TYPE_ID_TEMP_HI_RES0, util::xmsg::XSENS_FMT_CURVATURE, and util::xmsg::XSENS_FMT_DIFFERENCE.

Referenced by DecodeXMsgData(), util::xmsg::XDriftSalinity::DecodeXMsgData(), util::xmsg::XDriftTemperature::DecodeXMsgData(), util::xmsg::XHiResPressure::DecodeXMsgData(), util::xmsg::XHiResSalinity::DecodeXMsgData(), util::xmsg::XHiResTemperature::Decod util::xmsg::XProfilePressure::DecodeXMsgData(), util::xmsg::XProfileSalinity::DecodeXMsgData(), and util::xmsg::XProfileTemperature::D

Here is the call graph for this function:



Here is the caller graph for this function:



10.69.3.22 xmsg_index()

```
uint8_t util::xmsg::XMessageIndexable::xmsg_index () const  [inline], [inherited]
```

Get the index of this X-Message.

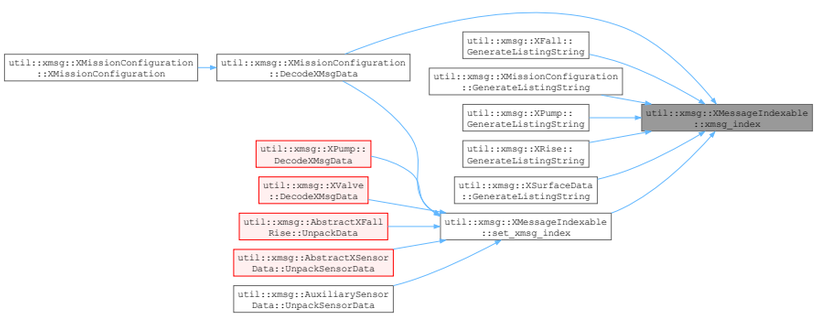
Returns

The index of this X-Message.

References util::xmsg::XMessageIndexable::xmsg_index_.

Referenced by util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing util::xmsg::XSurfaceData::GenerateListingString(), and util::xmsg::XMessageIndexable::set_xmsg_index().

Here is the caller graph for this function:



10.69.3.23 xmsg_size_bytes()

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes () const  [inline], [inherited]
```

Get the size of the X-Message, in bytes.

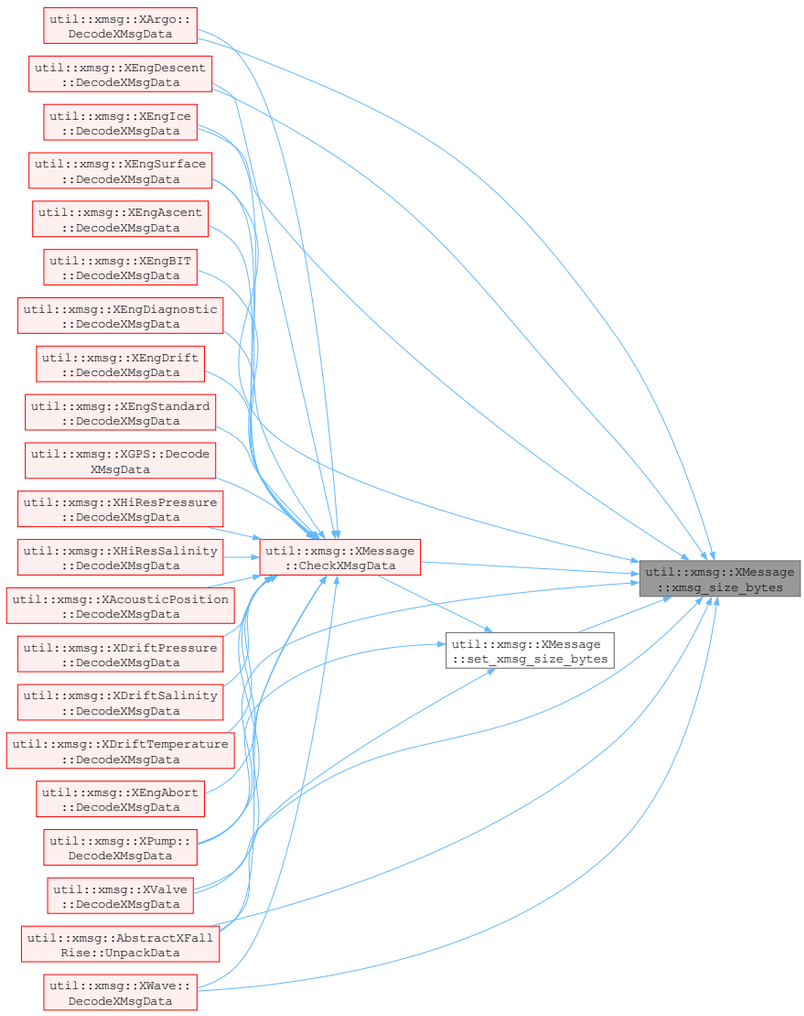
Returns

The size of the X-Message, in bytes.

References util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgI
util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(),
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), util::xmsg::XMessage::set_xmsg_size_bytes(),
and util::xmsg::AbstractXFallRise::UnpackData().

Here is the caller graph for this function:



10.69.4 Member Data Documentation

10.69.4.1 id_dive_

```
int16_t util::xmsg::XMessage::id_dive_  [private], [inherited]
```

The dive or cycle number of this message.

Referenced by util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::set_id_dive().

10.69.4.2 id_instrument_

```
uint16_t util::xmsg::XMessage::id_instrument_  [private], [inherited]
```

The unique WHOI identifier of this instrument.

Referenced by util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::set_id_instrument().

10.69.4.3 id_xmsg_type_

XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type_ [private], [inherited]

The X-message type identifier.

Referenced by util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::set_id_xmsg_type().

10.69.4.4 raw_sensor_data_

vector<int32_t> util::xmsg::AbstractXSensorData::raw_sensor_data_ [protected], [inherited]

The raw integer counts of the measured sensor data.

This vector of integers represents the "raw" sample values of the message. That is, by applying the appropriate gain and offset corrections, the sensor values may be recovered in scientific units. This data may represent pressure, temperature, salinity, or any other data that is recorded during the drift or profile phases of the dive cycle.

Referenced by util::xmsg::AbstractXSensorData::num_samples(), util::xmsg::AbstractXSensorData::raw_sensor_data(), util::xmsg::AbstractXSensorData::Sample(), util::xmsg::AbstractXSensorData::SampleValue(), util::xmsg::AbstractXSensorData::set_raw_ util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

10.69.4.5 sensor_data_format_

XSensorDataFormat_t util::xmsg::AbstractXSensorData::sensor_data_format_ [protected], [inherited]

The enumerated sensor data packing format.

This field indicates the format of encoded sensor data. The manner in which the sensor data is decoded is determined by the format of this field.

Referenced by util::xmsg::AbstractXSensorData::sensor_data_format(), and util::xmsg::AbstractXSensorData::set_sensor_data_format().

10.69.4.6 xmsg_index_

uint8_t util::xmsg::XMessageIndexable::xmsg_index_ [private], [inherited]

The index of this X-Message.

Referenced by util::xmsg::XMessageIndexable::set_xmsg_index(), and util::xmsg::XMessageIndexable::xmsg_index().

10.69.4.7 xmsg_size_bytes_

uint16_t util::xmsg::XMessage::xmsg_size_bytes_ [private], [inherited]

The size of the X message, in bytes.

Referenced by util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes().

The documentation for this class was generated from the following file:

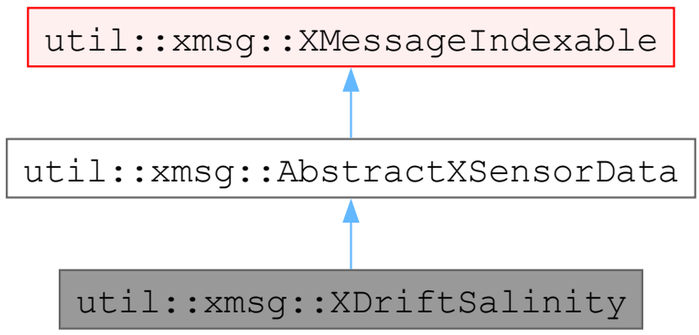
- data/xmsg/XDriftPressure.h

10.70 util::xmsg::XDriftSalinity Class Reference

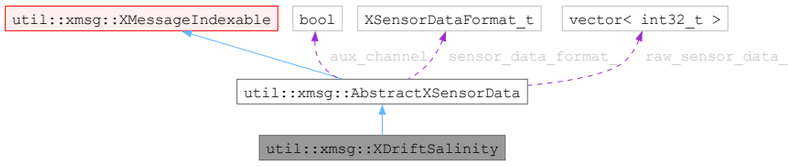
Class representing a drift salinity time-series X-Message.

`#include <XDriftSalinity.h>`

Inheritance diagram for util::xmsg::XDriftSalinity:



Collaboration diagram for util::xmsg::XDriftSalinity:



Public Member Functions

- XDriftSalinity (int16_t dive, uint16_t whoi, const vector< uint8_t > &data)
  The parameterized constructor.
- ~XDriftSalinity ()
  Destructor.
- const uint8_t * CheckXMsgData (const vector< uint8_t > &data, int id, int minBytes, bool parseSize=true)
  Check the X-message for the correct ID, minimum size, and message terminator.
- void DecodeXMsgData (const vector< uint8_t > &data)
  Decode the drift salinity X-Message from binary data.
- int16_t id_dive () const
  Get the dive or cycle number of this X-Message.
- uint16_t id_instrument () const
  Get the unique instrument identifier of this X-Message.
- XMsgTypeId_t id_xmsg_type () const
  Get the enumerated X-Message type identifier.
- bool **is_aux_channel** () const
- size_t num_samples () const
  Get the number of samples in this message.
- const vector< int32_t > & raw_sensor_data () const
  Get the raw sensor data vector.
- int32_t Sample (size_t i) const
  Get the raw integer value of the i^{th} sample.
- template<typename T = double>
  T SampleValue (size_t i, int16_t gain, int16_t offs) const
  Get the i^{th} sensor value in scientific units.
- template<typename T = double>
  T SampleValueAsPressure (size_t i, int16_t gain, int16_t offs) const

Get the i^{th} pressure sensor value, in dBar.

- template<typename T = double>
  T SampleValueAsSalinity (size_t i, int16_t gain, int16_t offs) const

  Get the i^{th} salinity sensor value, in PSU.
- template<typename T = double>
  T SampleValueAsTemperature (size_t i, int16_t gain, int16_t offs) const

  Get the i^{th} temperature sensor value, in degC.
- XSensorDataFormat_t sensor_data_format () const

  Get the enumerated sensor data format value.
- uint8_t xmsg_index () const

  Get the index of this X-Message.
- uint16_t xmsg_size_bytes () const

  Get the size of the X-Message, in bytes.

Protected Member Functions

- void set_id_dive (int16_t id_dive)

  Set the dive or cycle number of this message.
- void set_id_instrument (uint16_t id_instrument)

  Set the unique instrument identifier.
- void set_id_xmsg_type (XMsgTypeId_t id_xmsg_type)

  Set the X-Message type identifier.
- void set_raw_sensor_data (const vector< int32_t > &raw_sensor_data)

  Set raw integer counts of the measured sensor data.
- void set_sensor_data_format (XSensorDataFormat_t sensor_data_format)

  Set the sensor data format.
- void set_xmsg_index (uint8_t xmsg_index)

  Set the index of this X-Message.
- void set_xmsg_size_bytes (uint16_t xmsg_size_bytes)

  Set the size of the X-Message, in bytes.
- virtual vector< int32_t > UnpackSensorData (const vector< uint8_t > &data)

  Unpack the sensor data from the binary message.

Protected Attributes

- bool **aux_channel**
- vector< int32_t > raw_sensor_data_

  The raw integer counts of the measured sensor data.
- XSensorDataFormat_t sensor_data_format_

  The enumerated sensor data packing format.

Private Attributes

- int16_t id_dive_

  The dive or cycle number of this message.
- uint16_t id_instrument_

  The unique WHOI identifier of this instrument.
- XMsgTypeId_t id_xmsg_type_

  The X-message type identifier.
- uint8_t xmsg_index_

  The index of this X-Message.
- uint16_t xmsg_size_bytes_

  The size of the X message, in bytes.

10.70.1 Detailed Description

Class representing a drift salinity time-series X-Message.

10.70.2 Constructor & Destructor Documentation

10.70.2.1 XDriftSalinity()

```
util::xmsg::XDriftSalinity::XDriftSalinity (
            int16_t dive,
            uint16_t whoi,
            const vector< uint8_t > & data)  [inline]
```

The parameterized constructor.

Parameters

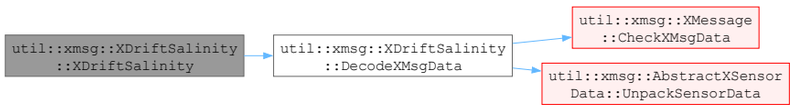
in	*dive*	The dive or cycle number of this message.
in	*whoi*	The unique WHOI identifier.
in	*data*	The binary X-message data to decode.

Exceptions

util::log::MessageException	if an decode error occurs.

References DecodeXMsgData().

Here is the call graph for this function:



10.70.2.2 ~XDriftSalinity()

```
util::xmsg::XDriftSalinity::~XDriftSalinity ()  [inline]
```

Destructor.

10.70.3 Member Function Documentation

10.70.3.1 CheckXMsgData()

```
const uint8_t * util::xmsg::XMessage::CheckXMsgData (
            const vector< uint8_t > & data,
            int id,
            int minBytes,
            bool parseSize = true)  [inline], [inherited]
```

Check the X-message for the correct ID, minimum size, and message terminator.

Parameters

in	*data*	The X-message binary data to check.
in	*id*	The expected ID of the X-message. If this field is less that 0, then the ID check will be skipped.
in	*minBytes*	The minimum number of bytes expected in a valid message.
in	*parseSize*	If TRUE, then the message size field will be parsed from the X-Message. Otherwise, the size field will not be parsed.

Exceptions

util::log::MessageException	if • the message does not contain the expected ID, • the message does not contain the minimum number of expected bytes, • the message does not contain a terminator (';') in the correct location, or, • any other error occurs.

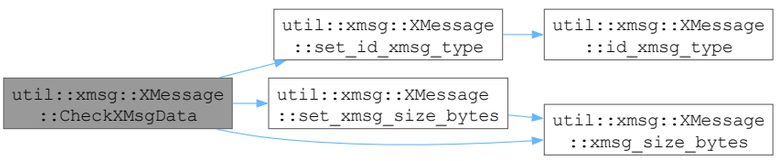
Returns

A pointer to the underlying data buffer of data.

References CATCH_BLOCK, EXC_THROW, lwrn, util::xmsg::XMessage::set_id_xmsg_type(), util::xmsg::XMessage::set_xmsg_size_byte
u16, and util::xmsg::XMessage::xmsg_size_bytes().

Referenced by util::xmsg::XAcousticPosition::DecodeXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XDriftPressure::Dec
DecodeXMsgData(), util::xmsg::XDriftTemperature::DecodeXMsgData(), util::xmsg::XEngAbort::DecodeXMsgData(),
util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngBIT::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(),
util::xmsg::XEngDiagnostic::DecodeXMsgData(), util::xmsg::XEngDrift::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(),
util::xmsg::XEngStandard::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XGPS::DecodeXMsgData(),
util::xmsg::XHiResPressure::DecodeXMsgData(), util::xmsg::XHiResSalinity::DecodeXMsgData(), util::xmsg::XHiResTemperature::Decod
util::xmsg::XMissionCmdEcho::DecodeXMsgData(), util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XProfilePressure:
util::xmsg::XProfileSalinity::DecodeXMsgData(), util::xmsg::XProfileTemperature::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgD
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the call graph for this function:



10.70.3.2 DecodeXMsgData()

```
void util::xmsg::XDriftSalinity::DecodeXMsgData (
            const vector< uint8_t > & data)  [inline], [virtual]
```

Decode the drift salinity X-Message from binary data.

Parameters

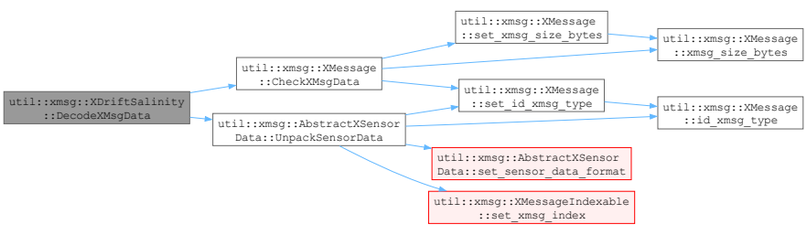
in	*data*	The binary data to decode.

Implements util::xmsg::AbstractXSensorData.

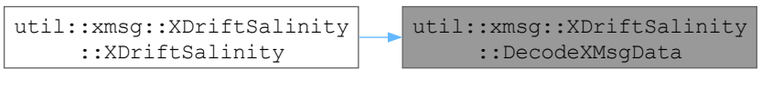
References CATCH_BLOCK, util::xmsg::XMessage::CheckXMsgData(), EXC_THROW, util::xmsg::AbstractXSensorData::UnpackSensorD
and util::xmsg::XMSG_TYPE_ID_SALN_DRIFT8.

Referenced by XDriftSalinity().

Here is the call graph for this function:



Here is the caller graph for this function:



10.70.3.3 id_dive()

```
int16_t util::xmsg::XMessage::id_dive () const  [inline], [inherited]
```

Get the dive or cycle number of this X-Message.

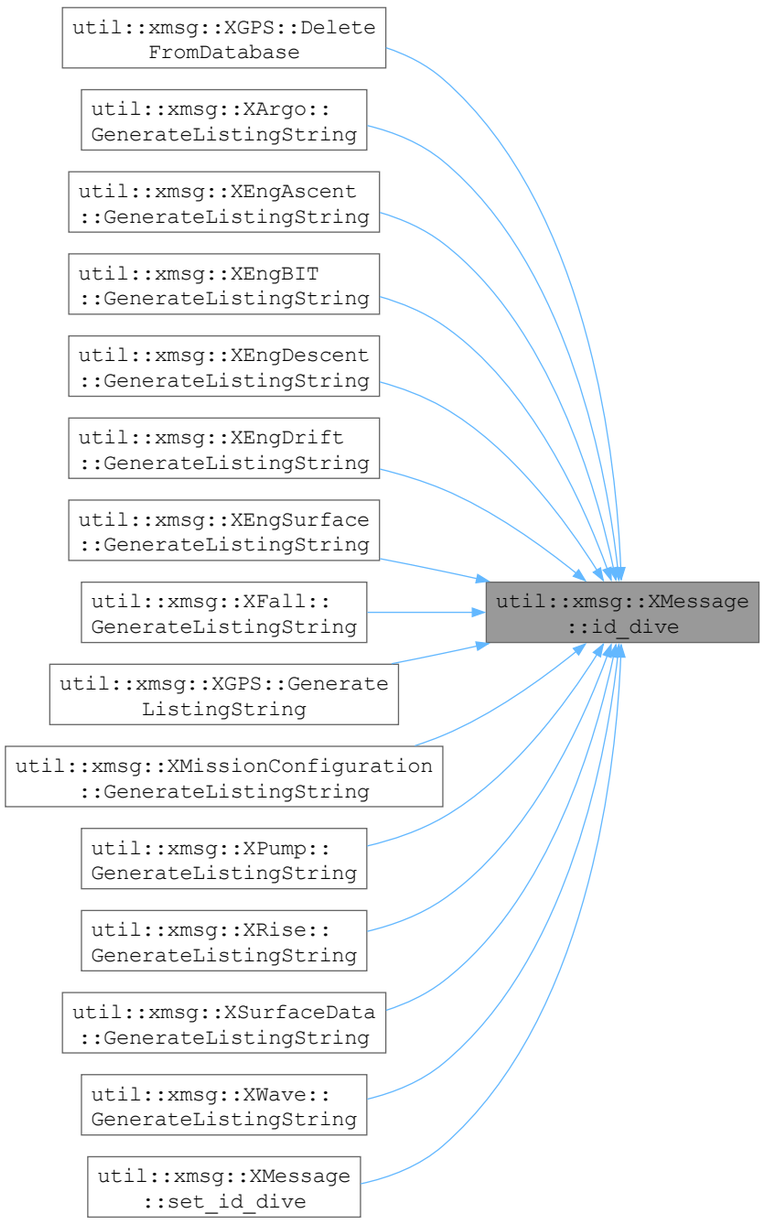
Returns

The dive or cycle number of this X-Message.

References util::xmsg::XMessage::id_dive_.

Referenced by util::xmsg::XGPS::DeleteFromDatabase(), util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateLis
util::xmsg::XEngBIT::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingStr
util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(),
util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing
util::xmsg::XSurfaceData::GenerateListingString(), util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_dive().

Here is the caller graph for this function:



10.70.3.4 id_instrument()

```
uint16_t util::xmsg::XMessage::id_instrument () const  [inline], [inherited]
```

Get the unique instrument identifier of this X-Message.

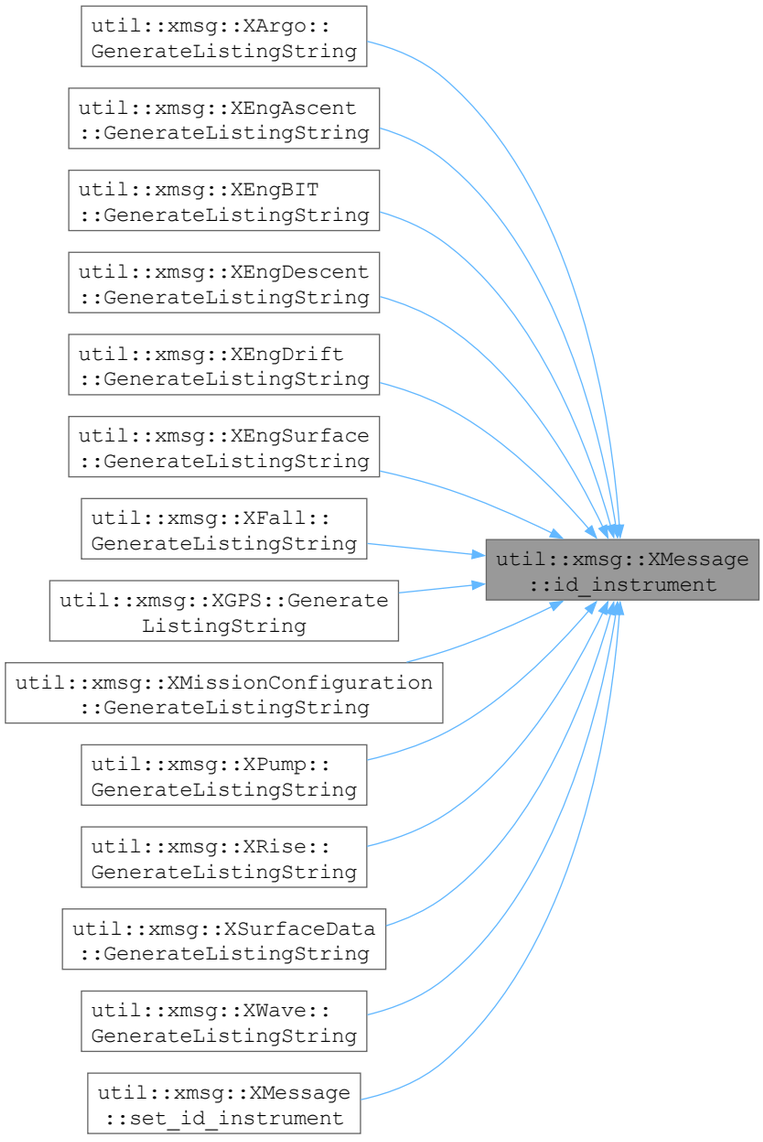
Returns

The unique instrument identifier of this X-Message.

References util::xmsg::XMessage::id_instrument_.

Referenced by util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngBIT::Generate
util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), util::xmsg::XEngSurface::GenerateListin
util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingS
util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(),
util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_instrument().

Here is the caller graph for this function:



10.70.3.5 id_xmsg_type()

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type () const  [inline], [inherited]
```

Get the enumerated X-Message type identifier.

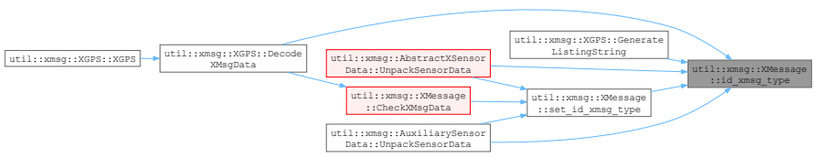
Returns

The enumerated X-Message type identifier.

References util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMessage::set_id_xmsg_typ
util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the caller graph for this function:



10.70.3.6 num_samples()

```
size_t util::xmsg::AbstractXSensorData::num_samples () const  [inline], [inherited]
```

Get the number of samples in this message.

Returns

The number of samples in this message.

References util::xmsg::AbstractXSensorData::raw_sensor_data_.

10.70.3.7 raw_sensor_data()

```
const vector< int32_t > & util::xmsg::AbstractXSensorData::raw_sensor_data () const  [inline],
[inherited]
```

Get the raw sensor data vector.

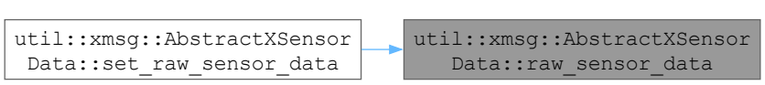
Returns

The raw sensor data vector.

References util::xmsg::AbstractXSensorData::raw_sensor_data_.

Referenced by util::xmsg::AbstractXSensorData::set_raw_sensor_data().

Here is the caller graph for this function:



10.70.3.8 Sample()

```
int32_t util::xmsg::AbstractXSensorData::Sample (
            size_t i) const  [inline], [inherited]
```

Get the raw integer value of the i^{th} sample.

Parameters

in	*i*	The index of the sensor value.

Returns

The raw integer value of the i^{th} ith sample.

References util::xmsg::AbstractXSensorData::raw_sensor_data_.

10.70.3.9 SampleValue()

```
template<typename T = double>
T util::xmsg::AbstractXSensorData::SampleValue (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline], [inherited]
```

Get the i^{th} sensor value in scientific units.

Parameters

in	*i*	The index of the sensor value.
in	*gain*	The sensor sensitivity scale factor.
in	*offs*	The sensor offset.

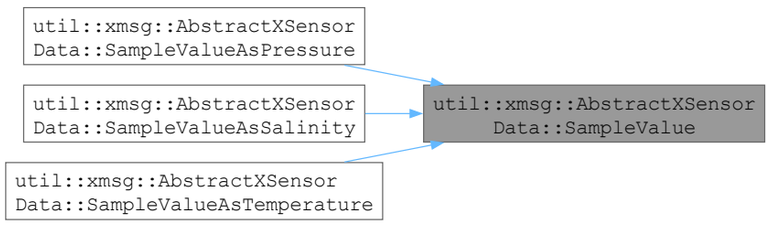
Returns

The i^{th} sensor value in scientific units.

References CATCH_BLOCK, and util::xmsg::AbstractXSensorData::raw_sensor_data_.

Referenced by util::xmsg::AbstractXSensorData::SampleValueAsPressure(), util::xmsg::AbstractXSensorData::SampleValueAsSalinity(), and util::xmsg::AbstractXSensorData::SampleValueAsTemperature().

Here is the caller graph for this function:



10.70.3.10 SampleValueAsPressure()

```
template<typename T = double>
T util::xmsg::AbstractXSensorData::SampleValueAsPressure (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline], [inherited]
```

Get the i^{th} pressure sensor value, in dBar.

Parameters

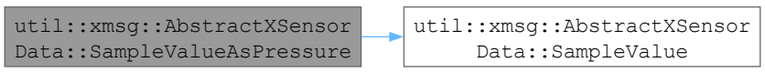
in	*i*	The index of the sensor value.
in	*gain*	The pressure sensor sensitivity scale factor, in LSB/dBar.
in	*offs*	The pressure sensor offset, in dBar.

Returns

The i^{th} pressure sensor value, in dBar.

References util::xmsg::AbstractXSensorData::SampleValue(), XMSG_SENS_BAD_PRES, and XMSG_SENS_MAX_PRES.

Here is the call graph for this function:



10.70.3.11 SampleValueAsSalinity()

```
template<typename T = double>
T util::xmsg::AbstractXSensorData::SampleValueAsSalinity (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline], [inherited]
```

Get the i^{th} salinity sensor value, in PSU.

Parameters

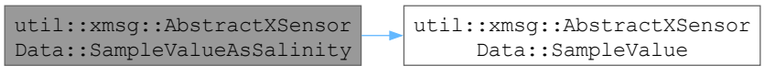
in	*i*	The index of the sensor value.
in	*gain*	The salinity sensor sensitivity scale factor, in LSB/PSU.
in	*offs*	The salinity sensor offset, in PSU.

Returns

The i^{th} salinity sensor value, in PSU.

References util::xmsg::AbstractXSensorData::SampleValue(), XMSG_SENS_BAD_SALN, and XMSG_SENS_MAX_SALN.

Here is the call graph for this function:



10.70.3.12 SampleValueAsTemperature()

```
template<typename T = double>
T util::xmsg::AbstractXSensorData::SampleValueAsTemperature (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline], [inherited]
```

Get the i^{th} temperature sensor value, in degC.

Parameters

in	*i*	The index of the sensor value.
in	*gain*	The temperature sensor sensitivity scale factor, in LSB/degC.
in	*offs*	The temperature sensor offset, in degC.

Returns

The i^{th} temperature sensor value, in degC.

References util::xmsg::AbstractXSensorData::SampleValue(), XMSG_SENS_BAD_TEMP, and XMSG_SENS_MAX_TEMP.

Here is the call graph for this function:

10.70.3.13 sensor_data_format()

```
XSensorDataFormat_t util::xmsg::AbstractXSensorData::sensor_data_format () const  [inline], [inherited]
```

Get the enumerated sensor data format value.

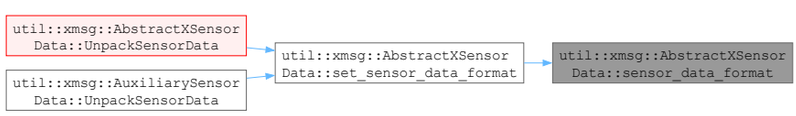
Returns

The enumerated sensor data format value.

References util::xmsg::AbstractXSensorData::sensor_data_format_.

Referenced by util::xmsg::AbstractXSensorData::set_sensor_data_format().

Here is the caller graph for this function:



10.70.3.14 set_id_dive()

```
void util::xmsg::XMessage::set_id_dive (
            int16_t id_dive)  [inline], [protected], [inherited]
```

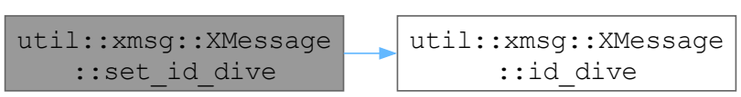
Set the dive or cycle number of this message.

Parameters

in	*id_dive*	The dive or cycle number of this message to set.

References util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::id_dive_.

Here is the call graph for this function:



10.70.3.15 set_id_instrument()

```
void util::xmsg::XMessage::set_id_instrument (
            uint16_t id_instrument)  [inline], [protected], [inherited]
```

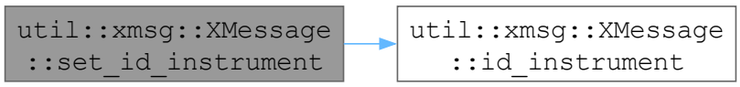
Set the unique instrument identifier.

Parameters

in	*id_instrument*	The unique instrument identifier to set.

References util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::id_instrument_.

Here is the call graph for this function:



10.70.3.16 set_id_xmsg_type()

```
void util::xmsg::XMessage::set_id_xmsg_type (
            XMsgTypeId_t id_xmsg_type)  [inline], [protected], [inherited]
```

Set the X-Message type identifier.

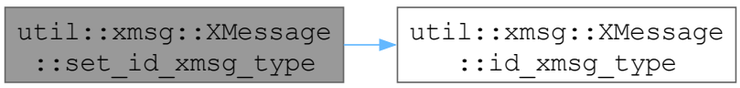
Parameters

in	*id_xmsg_type*	The X-Message type identifier to set.

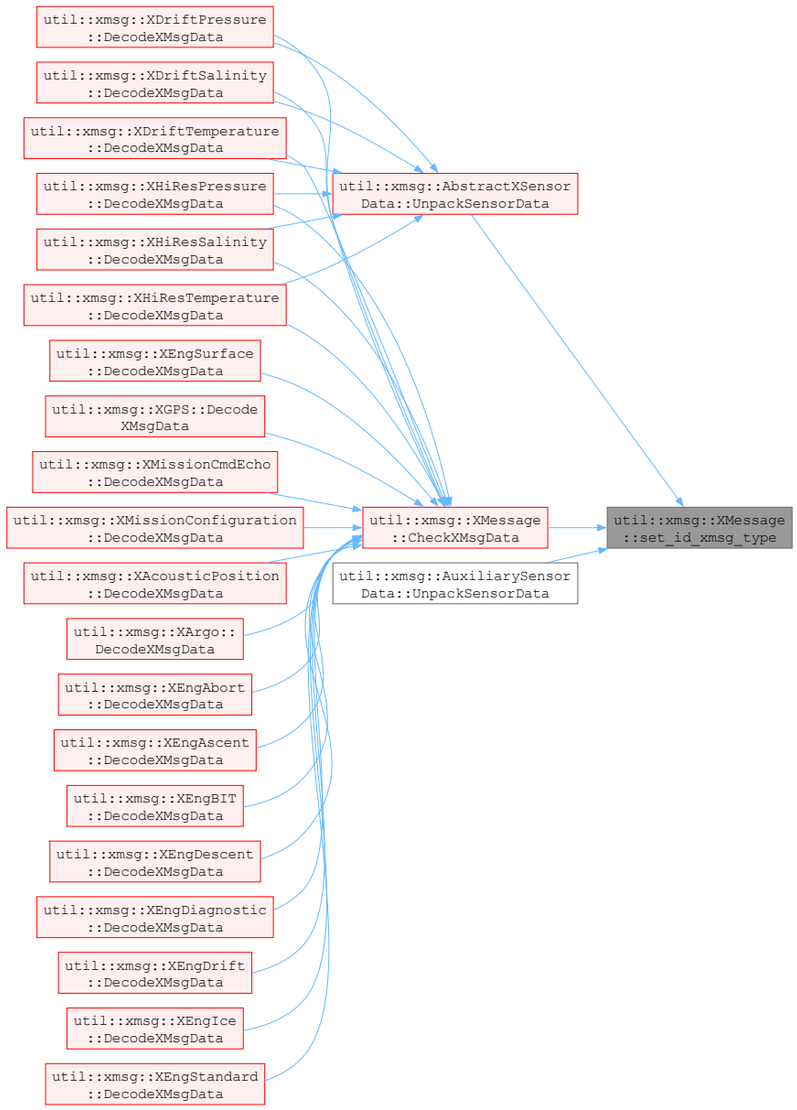
References util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.70.3.17 set_raw_sensor_data()

```
void util::xmsg::AbstractXSensorData::set_raw_sensor_data (
            const vector< int32_t > & raw_sensor_data)  [inline], [protected], [inherited]
```

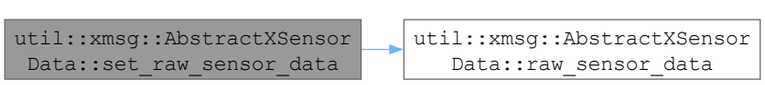
Set raw integer counts of the measured sensor data.

Parameters

in	*raw_sensor_data*	The data to set.

References util::xmsg::AbstractXSensorData::raw_sensor_data(), and util::xmsg::AbstractXSensorData::raw_sensor_data_.

Here is the call graph for this function:



10.70.3.18 set_sensor_data_format()

```
void util::xmsg::AbstractXSensorData::set_sensor_data_format (
            XSensorDataFormat_t sensor_data_format)  [inline], [protected], [inherited]
```

Set the sensor data format.

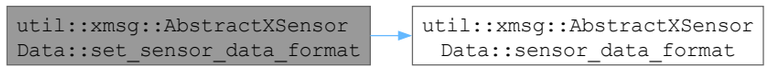
Parameters

in	*sensor_data_format*	The sensor data format to set.

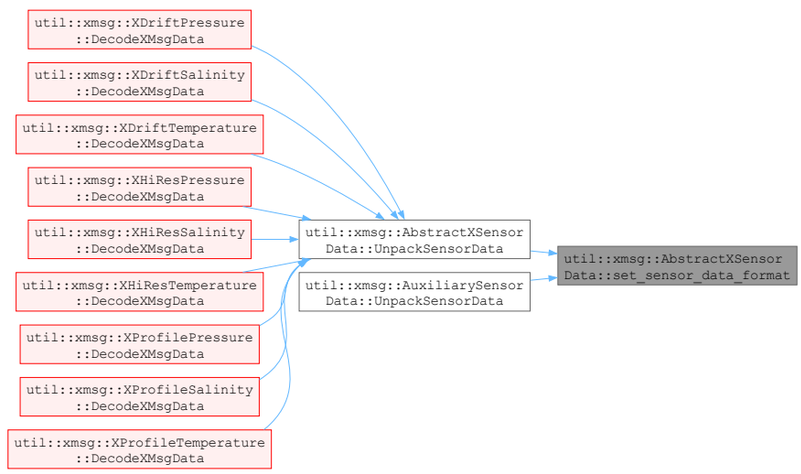
References util::xmsg::AbstractXSensorData::sensor_data_format(), and util::xmsg::AbstractXSensorData::sensor_data_format_.

Referenced by util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.70.3.19 set_xmsg_index()

```
void util::xmsg::XMessageIndexable::set_xmsg_index (
            uint8_t xmsg_index)  [inline], [protected], [inherited]
```

Set the index of this X-Message.

Parameters

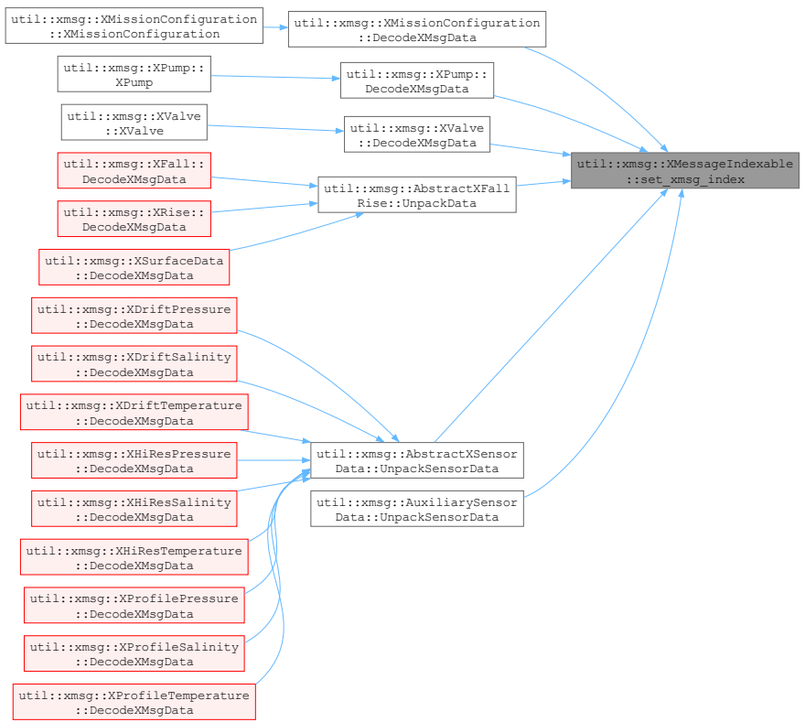
in	*xmsg_index*	The index of this X-Message.

References util::xmsg::XMessageIndexable::xmsg_index(), and util::xmsg::XMessageIndexable::xmsg_index_.

Referenced by util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(), util::xmsg::XValve::DecodeXMsgData(), util::xmsg::AbstractXFallRise::UnpackData(), util::xmsg::AbstractXSensorData::UnpackSensorDa and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:

Here is the caller graph for this function:



10.70.3.20 set_xmsg_size_bytes()

```
void util::xmsg::XMessage::set_xmsg_size_bytes (
            uint16_t xmsg_size_bytes)  [inline], [protected], [inherited]
```

Set the size of the X-Message, in bytes.

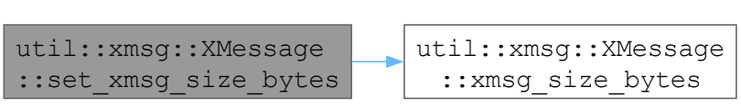
Parameters

in	*xmsg_size_bytes*	The size of the X-Message to set, in bytes.

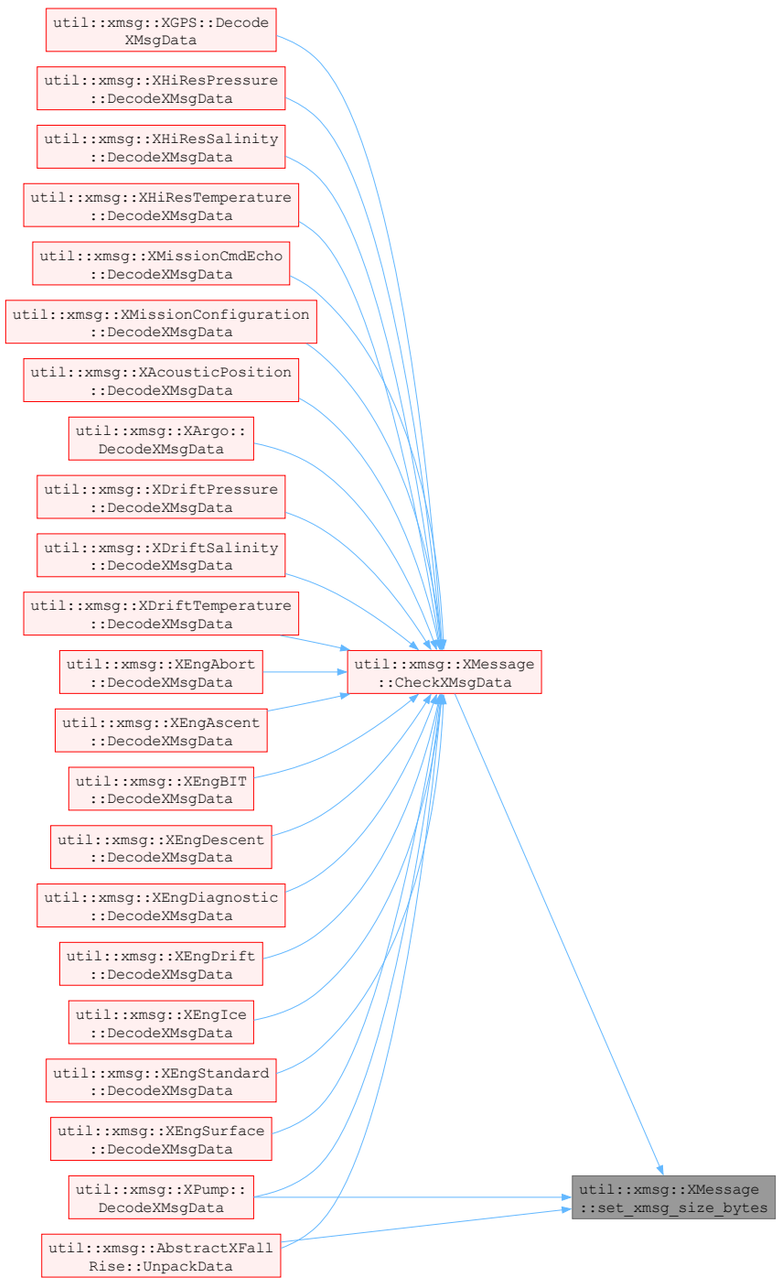
References util::xmsg::XMessage::xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XPump::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::Unpa

Here is the call graph for this function:



Here is the caller graph for this function:



10.70.3.21 UnpackSensorData()

```
virtual vector< int32_t > util::xmsg::AbstractXSensorData::UnpackSensorData (
            const vector< uint8_t > & data)  [inline], [protected], [virtual], [inherited]
```

Unpack the sensor data from the binary message.

This field unpacks the encoded sensor data from the message and stores the decoded sensor values in raw_sensor_data_. First the function checks the size of the message, and the data packing format. The message data is decoded according to the packing format type. Both first-difference and curvature encoding are supported. The encoding method is determined from the value of the high nibble of byte `data`[1].

Once this function completes successfully, then the data may be recovered in scientific units from the raw sensor values by applying the appropriate sensor gain and offset values.

Parameters

in	*data*	The binary message to unpack, in a vector of unsigned bytes.

Warning

The function assumes the message data is valid. Message validity should be ensured before this function is called.

Exceptions

util::log::MessageException	if • an error occurs while decoding the message, • the message is of insufficient length, or • an invalid packing format is encountered.

Returns

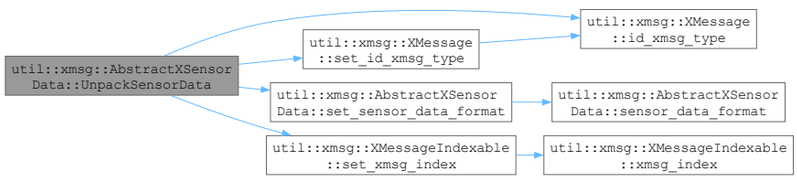
The sensor data raw counts, in a vector of signed integers.

Reimplemented in util::xmsg::AuxiliarySensorData.

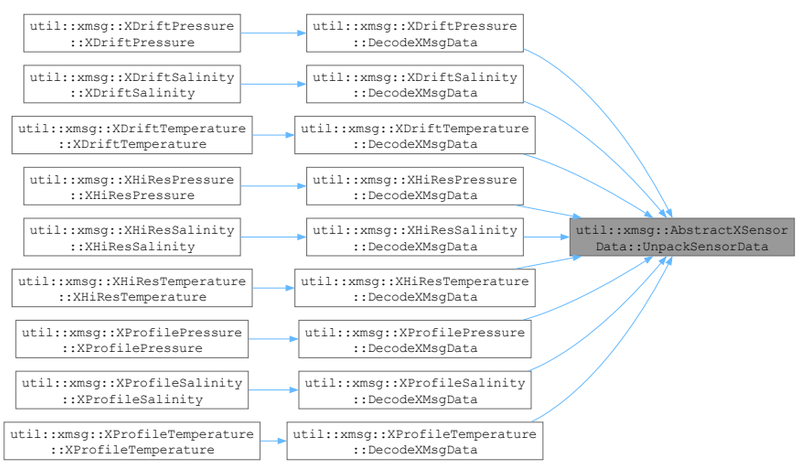
References CATCH_BLOCK, CATCH_NOTHROW, EXC_THROW, i24, util::xmsg::XMessage::id_xmsg_type(), ldbg, lwrn, util::xmsg::AbstractXSensorData::raw_sensor_data_, util::xmsg::XMessage::set_id_xmsg_type(), util::xmsg::AbstractXSensorData::s util::xmsg::XMessageIndexable::set_xmsg_index(), u16, u24, XMSG_SENS_MAX_SUB, util::xmsg::XMSG_TYPE_ID_PRES0, util::xmsg::XMSG_TYPE_ID_PRES_DRIFT8, util::xmsg::XMSG_TYPE_ID_PRES_HI_RES0, util::xmsg::XMSG_TYPE_ID_SALN0, util::xmsg::XMSG_TYPE_ID_SALN_DRIFT8, util::xmsg::XMSG_TYPE_ID_SALN_HI_RES0, util::xmsg::XMSG_TYPE_ID_TEMP0, util::xmsg::XMSG_TYPE_ID_TEMP_DRIFT8, util::xmsg::XMSG_TYPE_ID_TEMP_HI_RES0, util::xmsg::XSENS_FMT_CURVATURE, and util::xmsg::XSENS_FMT_DIFFERENCE.

Referenced by util::xmsg::XDriftPressure::DecodeXMsgData(), DecodeXMsgData(), util::xmsg::XDriftTemperature::DecodeXMsgData(), util::xmsg::XHiResPressure::DecodeXMsgData(), util::xmsg::XHiResSalinity::DecodeXMsgData(), util::xmsg::XHiResTemperature::Decod util::xmsg::XProfilePressure::DecodeXMsgData(), util::xmsg::XProfileSalinity::DecodeXMsgData(), and util::xmsg::XProfileTemperature::D

Here is the call graph for this function:



Here is the caller graph for this function:



10.70.3.22 xmsg_index()

```
uint8_t util::xmsg::XMessageIndexable::xmsg_index () const  [inline], [inherited]
```

Get the index of this X-Message.

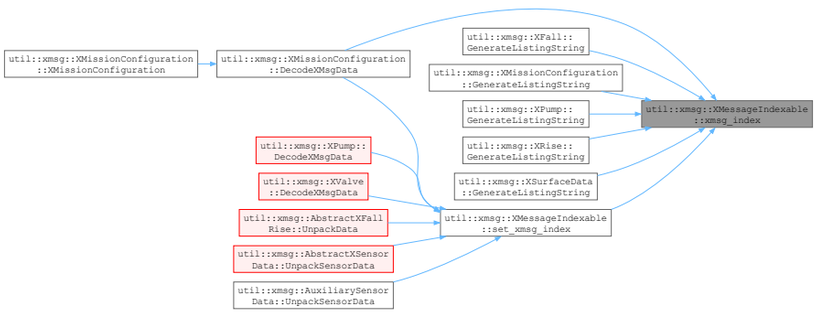
Returns

The index of this X-Message.

References util::xmsg::XMessageIndexable::xmsg_index_.

Referenced by util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing util::xmsg::XSurfaceData::GenerateListingString(), and util::xmsg::XMessageIndexable::set_xmsg_index().

Here is the caller graph for this function:



10.70.3.23 xmsg_size_bytes()

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes () const  [inline], [inherited]
```

Get the size of the X-Message, in bytes.

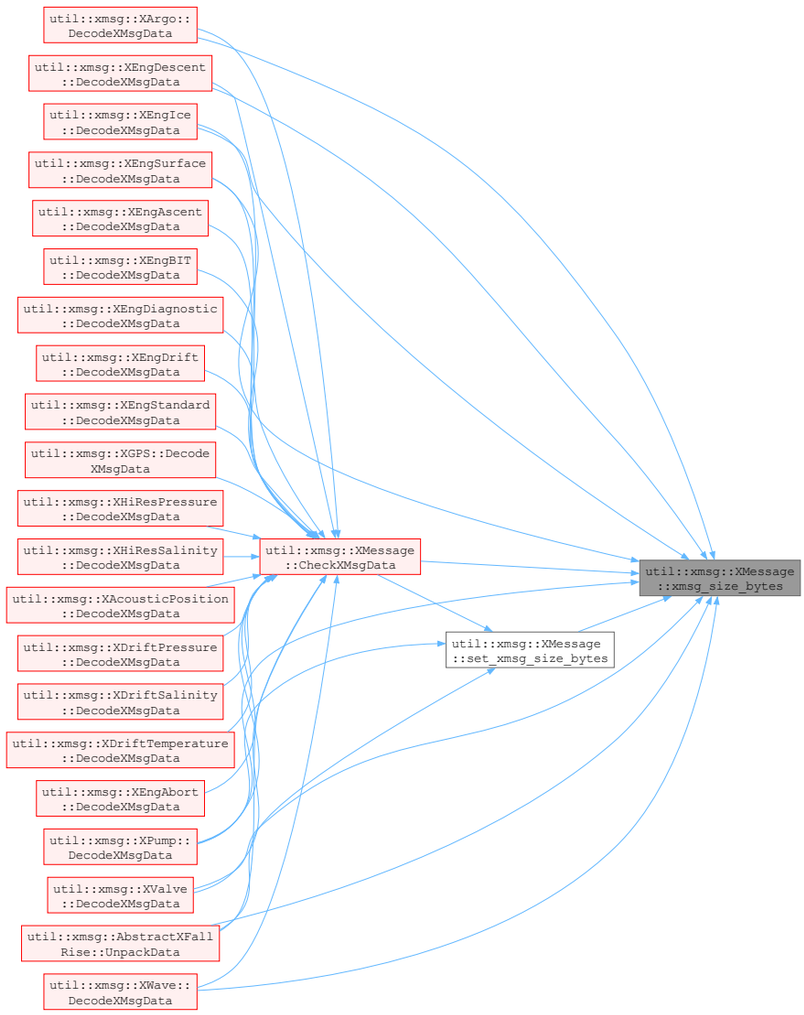
Returns

The size of the X-Message, in bytes.

References util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgI util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(), util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the caller graph for this function:



10.70.4 Member Data Documentation

10.70.4.1 id_dive_

```
int16_t util::xmsg::XMessage::id_dive_  [private], [inherited]
```

The dive or cycle number of this message.

Referenced by util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::set_id_dive().

10.70.4.2 id_instrument_

```
uint16_t util::xmsg::XMessage::id_instrument_  [private], [inherited]
```

The unique WHOI identifier of this instrument.

Referenced by util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::set_id_instrument().

10.70.4.3 id_xmsg_type_

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type_  [private], [inherited]
```

The X-message type identifier.

Referenced by util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::set_id_xmsg_type().

10.70.4.4 raw_sensor_data_

```
vector<int32_t> util::xmsg::AbstractXSensorData::raw_sensor_data_  [protected], [inherited]
```

The raw integer counts of the measured sensor data.

This vector of integers represents the "raw" sample values of the message. That is, by applying the appropriate gain and offset corrections, the sensor values may be recovered in scientific units. This data may represent pressure, temperature, salinity, or any other data that is recorded during the drift or profile phases of the dive cycle.

Referenced by util::xmsg::AbstractXSensorData::num_samples(), util::xmsg::AbstractXSensorData::raw_sensor_data(), util::xmsg::AbstractXSensorData::Sample(), util::xmsg::AbstractXSensorData::SampleValue(), util::xmsg::AbstractXSensorData::set_raw_ util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

10.70.4.5 sensor_data_format_

```
XSensorDataFormat_t util::xmsg::AbstractXSensorData::sensor_data_format_  [protected], [inherited]
```

The enumerated sensor data packing format.

This field indicates the format of encoded sensor data. The manner in which the sensor data is decoded is determined by the format of this field.

Referenced by util::xmsg::AbstractXSensorData::sensor_data_format(), and util::xmsg::AbstractXSensorData::set_sensor_data_format().

10.70.4.6 xmsg_index_

```
uint8_t util::xmsg::XMessageIndexable::xmsg_index_  [private], [inherited]
```

The index of this X-Message.

Referenced by util::xmsg::XMessageIndexable::set_xmsg_index(), and util::xmsg::XMessageIndexable::xmsg_index().

10.70.4.7 xmsg_size_bytes_

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes_  [private], [inherited]
```

The size of the X message, in bytes.

Referenced by util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes().

The documentation for this class was generated from the following file:

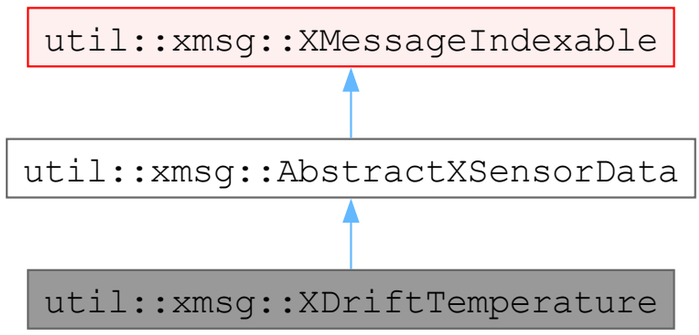
- data/xmsg/XDriftSalinity.h

10.71 util::xmsg::XDriftTemperature Class Reference

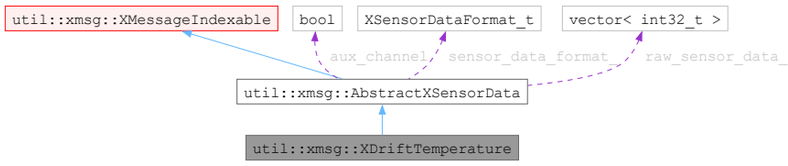
Class reprenting a drift temperature X-Message.

`#include <XDriftTemperature.h>`

Inheritance diagram for util::xmsg::XDriftTemperature:



Collaboration diagram for util::xmsg::XDriftTemperature:



Public Member Functions

- XDriftTemperature (int16_t dive, uint16_t whoi, const vector< uint8_t > &data)
  The parameterized constructor.
- ~XDriftTemperature ()
  The destructor.
- const uint8_t * CheckXMsgData (const vector< uint8_t > &data, int id, int minBytes, bool parseSize=true)
  Check the X-message for the correct ID, minimum size, and message terminator.
- void DecodeXMsgData (const vector< uint8_t > &data)
  Decode the drift temperature X-Message from binary data.
- int16_t id_dive () const
  Get the dive or cycle number of this X-Message.
- uint16_t id_instrument () const
  Get the unique instrument identifier of this X-Message.
- XMsgTypeId_t id_xmsg_type () const
  Get the enumerated X-Message type identifier.
- bool **is_aux_channel** () const
- size_t num_samples () const
  Get the number of samples in this message.
- const vector< int32_t > & raw_sensor_data () const
  Get the raw sensor data vector.
- int32_t Sample (size_t i) const
  Get the raw integer value of the i^{th} sample.
- template<typename T = double>
  T SampleValue (size_t i, int16_t gain, int16_t offs) const
  Get the i^{th} sensor value in scientific units.
- template<typename T = double>
  T SampleValueAsPressure (size_t i, int16_t gain, int16_t offs) const

Get the i^{th} pressure sensor value, in dBar.

- template<typename T = double>
  T SampleValueAsSalinity (size_t i, int16_t gain, int16_t offs) const

  Get the i^{th} salinity sensor value, in PSU.
- template<typename T = double>
  T SampleValueAsTemperature (size_t i, int16_t gain, int16_t offs) const

  Get the i^{th} temperature sensor value, in degC.
- XSensorDataFormat_t sensor_data_format () const

  Get the enumerated sensor data format value.
- uint8_t xmsg_index () const

  Get the index of this X-Message.
- uint16_t xmsg_size_bytes () const

  Get the size of the X-Message, in bytes.

Protected Member Functions

- void set_id_dive (int16_t id_dive)

  Set the dive or cycle number of this message.
- void set_id_instrument (uint16_t id_instrument)

  Set the unique instrument identifier.
- void set_id_xmsg_type (XMsgTypeId_t id_xmsg_type)

  Set the X-Message type identifier.
- void set_raw_sensor_data (const vector< int32_t > &raw_sensor_data)

  Set raw integer counts of the measured sensor data.
- void set_sensor_data_format (XSensorDataFormat_t sensor_data_format)

  Set the sensor data format.
- void set_xmsg_index (uint8_t xmsg_index)

  Set the index of this X-Message.
- void set_xmsg_size_bytes (uint16_t xmsg_size_bytes)

  Set the size of the X-Message, in bytes.
- virtual vector< int32_t > UnpackSensorData (const vector< uint8_t > &data)

  Unpack the sensor data from the binary message.

Protected Attributes

- bool **aux_channel**
- vector< int32_t > raw_sensor_data_

  The raw integer counts of the measured sensor data.
- XSensorDataFormat_t sensor_data_format_

  The enumerated sensor data packing format.

Private Attributes

- int16_t id_dive_

  The dive or cycle number of this message.
- uint16_t id_instrument_

  The unique WHOI identifier of this instrument.
- XMsgTypeId_t id_xmsg_type_

  The X-message type identifier.
- uint8_t xmsg_index_

  The index of this X-Message.
- uint16_t xmsg_size_bytes_

  The size of the X message, in bytes.

10.71.1 Detailed Description

Class reprenting a drift temperature X-Message.

10.71.2 Constructor & Destructor Documentation

10.71.2.1 XDriftTemperature()

```
util::xmsg::XDriftTemperature::XDriftTemperature (
            int16_t dive,
            uint16_t whoi,
            const vector< uint8_t > & data)  [inline]
```

The parameterized constructor.

Parameters

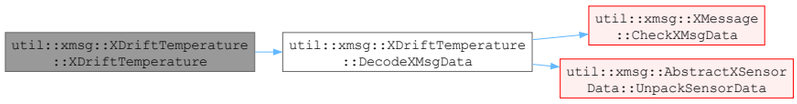
in	*dive*	The dive or cycle number of this message.
in	*whoi*	The unique WHOI identifier.
in	*data*	The binary X-message data to decode.

Exceptions

util::log::MessageException	if an decode error occurs.

References DecodeXMsgData().

Here is the call graph for this function:



10.71.2.2 ~XDriftTemperature()

```
util::xmsg::XDriftTemperature::~XDriftTemperature ()  [inline]
```

The destructor.

10.71.3 Member Function Documentation

10.71.3.1 CheckXMsgData()

```
const uint8_t * util::xmsg::XMessage::CheckXMsgData (
            const vector< uint8_t > & data,
            int id,
            int minBytes,
            bool parseSize = true)  [inline], [inherited]
```

Check the X-message for the correct ID, minimum size, and message terminator.

Parameters

in	*data*	The X-message binary data to check.
in	*id*	The expected ID of the X-message. If this field is less that 0, then the ID check will be skipped.
in	*minBytes*	The minimum number of bytes expected in a valid message.
in	*parseSize*	If TRUE, then the message size field will be parsed from the X-Message. Otherwise, the size field will not be parsed.

Exceptions

util::log::MessageException	if • the message does not contain the expected ID, • the message does not contain the minimum number of expected bytes, • the message does not contain a terminator (';') in the correct location, or, • any other error occurs.

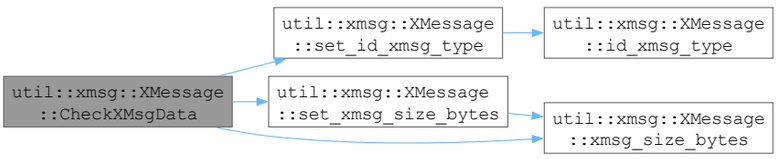
Returns

A pointer to the underlying data buffer of data.

References CATCH_BLOCK, EXC_THROW, lwrn, util::xmsg::XMessage::set_id_xmsg_type(), util::xmsg::XMessage::set_xmsg_size_byte
u16, and util::xmsg::XMessage::xmsg_size_bytes().

Referenced by util::xmsg::XAcousticPosition::DecodeXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XDriftPressure::Dec
util::xmsg::XDriftSalinity::DecodeXMsgData(), DecodeXMsgData(), util::xmsg::XEngAbort::DecodeXMsgData(),
util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngBIT::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(),
util::xmsg::XEngDiagnostic::DecodeXMsgData(), util::xmsg::XEngDrift::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(),
util::xmsg::XEngStandard::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XGPS::DecodeXMsgData(),
util::xmsg::XHiResPressure::DecodeXMsgData(), util::xmsg::XHiResSalinity::DecodeXMsgData(), util::xmsg::XHiResTemperature::Decod
util::xmsg::XMissionCmdEcho::DecodeXMsgData(), util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XProfilePressure:
util::xmsg::XProfileSalinity::DecodeXMsgData(), util::xmsg::XProfileTemperature::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgD
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the call graph for this function:



10.71.3.2 DecodeXMsgData()

```
void util::xmsg::XDriftTemperature::DecodeXMsgData (
            const vector< uint8_t > & data)  [inline], [virtual]
```

Decode the drift temperature X-Message from binary data.

Parameters

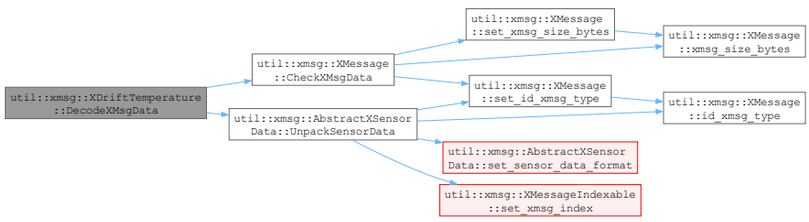
in	*data*	The binary data to decode.

Implements util::xmsg::AbstractXSensorData.

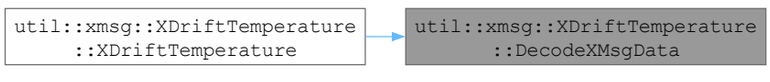
References CATCH_BLOCK, util::xmsg::XMessage::CheckXMsgData(), EXC_THROW, util::xmsg::AbstractXSensorData::UnpackSensorD
and util::xmsg::XMSG_TYPE_ID_TEMP_DRIFT8.

Referenced by XDriftTemperature().

Here is the call graph for this function:



Here is the caller graph for this function:



10.71.3.3 id_dive()

```
int16_t util::xmsg::XMessage::id_dive () const  [inline], [inherited]
```

Get the dive or cycle number of this X-Message.

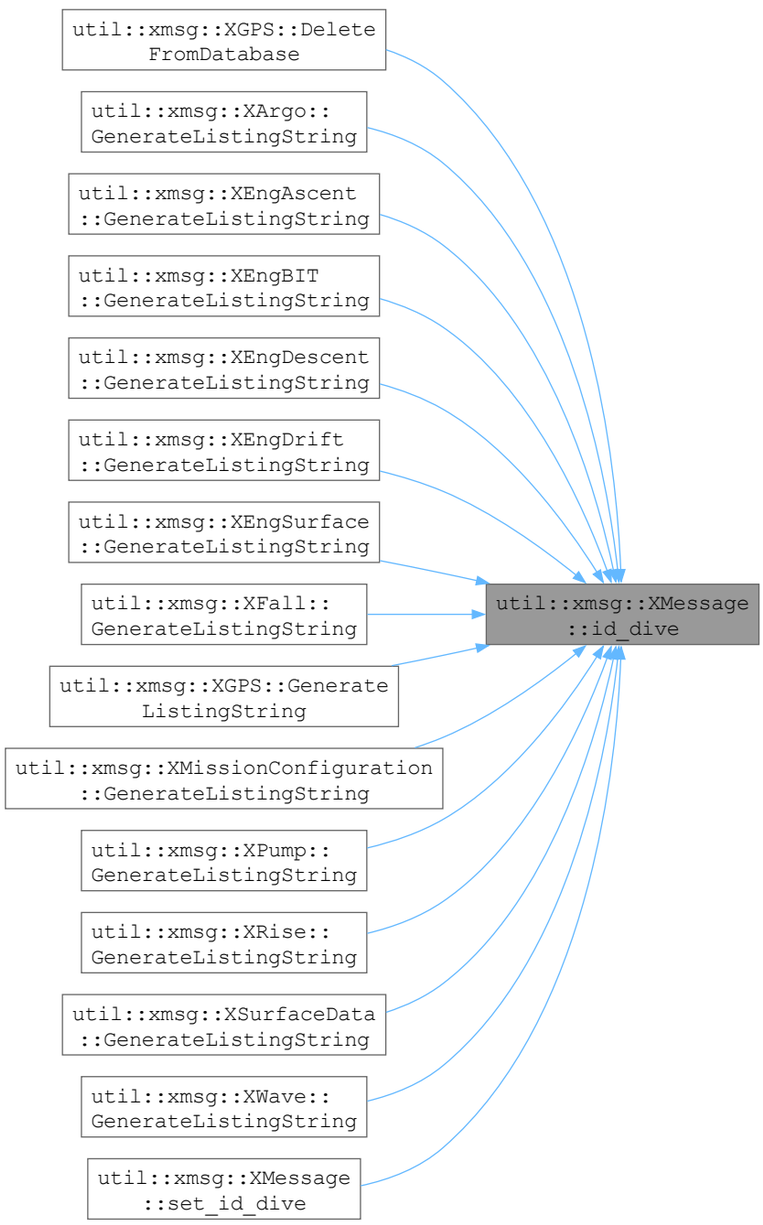
Returns

The dive or cycle number of this X-Message.

References util::xmsg::XMessage::id_dive_.

Referenced by util::xmsg::XGPS::DeleteFromDatabase(), util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateLis
util::xmsg::XEngBIT::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingStr
util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(),
util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing
util::xmsg::XSurfaceData::GenerateListingString(), util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_dive().

Here is the caller graph for this function:



10.71.3.4 id_instrument()

```
uint16_t util::xmsg::XMessage::id_instrument () const  [inline], [inherited]
```

Get the unique instrument identifier of this X-Message.

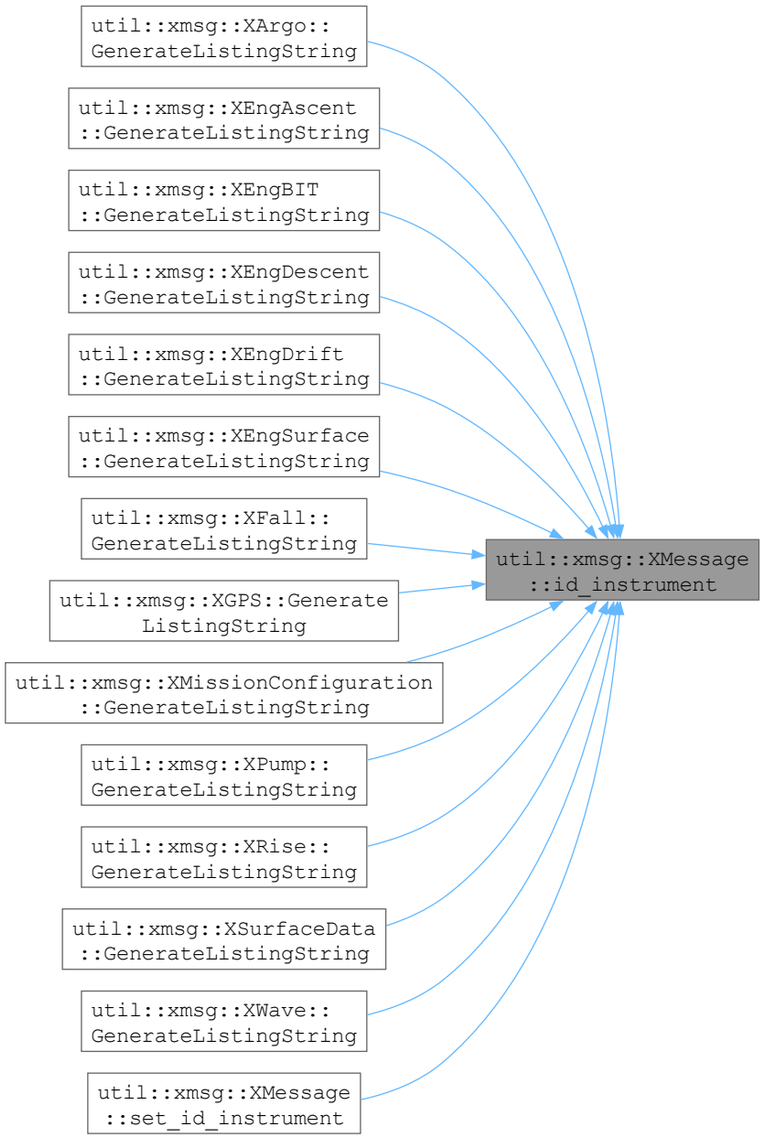
Returns

The unique instrument identifier of this X-Message.

References util::xmsg::XMessage::id_instrument_.

Referenced by util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngBIT::Generate
util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), util::xmsg::XEngSurface::GenerateListin
util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingS
util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(),
util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_instrument().

Here is the caller graph for this function:



10.71.3.5 id_xmsg_type()

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type () const  [inline], [inherited]
```

Get the enumerated X-Message type identifier.

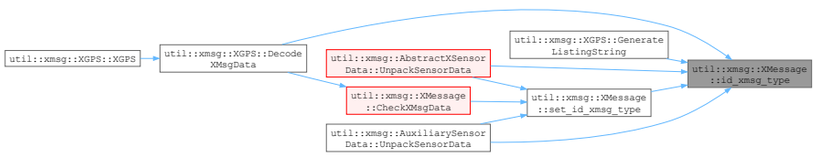
Returns

The enumerated X-Message type identifier.

References util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMessage::set_id_xmsg_typ
util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the caller graph for this function:



10.71.3.6 num_samples()

```
size_t util::xmsg::AbstractXSensorData::num_samples () const  [inline], [inherited]
```

Get the number of samples in this message.

Returns

The number of samples in this message.

References util::xmsg::AbstractXSensorData::raw_sensor_data_.

10.71.3.7 raw_sensor_data()

```
const vector< int32_t > & util::xmsg::AbstractXSensorData::raw_sensor_data () const  [inline],
[inherited]
```

Get the raw sensor data vector.

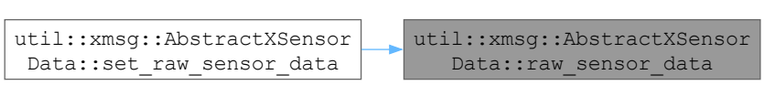
Returns

The raw sensor data vector.

References util::xmsg::AbstractXSensorData::raw_sensor_data_.

Referenced by util::xmsg::AbstractXSensorData::set_raw_sensor_data().

Here is the caller graph for this function:



10.71.3.8 Sample()

```
int32_t util::xmsg::AbstractXSensorData::Sample (
            size_t i) const  [inline], [inherited]
```

Get the raw integer value of the i^{th} sample.

Parameters

in	*i*	The index of the sensor value.

Returns

The raw integer value of the i^{th} ith sample.

References util::xmsg::AbstractXSensorData::raw_sensor_data_.

10.71.3.9 SampleValue()

```
template<typename T = double>
T util::xmsg::AbstractXSensorData::SampleValue (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline], [inherited]
```

Get the i^{th} sensor value in scientific units.

Parameters

in	*i*	The index of the sensor value.
in	*gain*	The sensor sensitivity scale factor.
in	*offs*	The sensor offset.

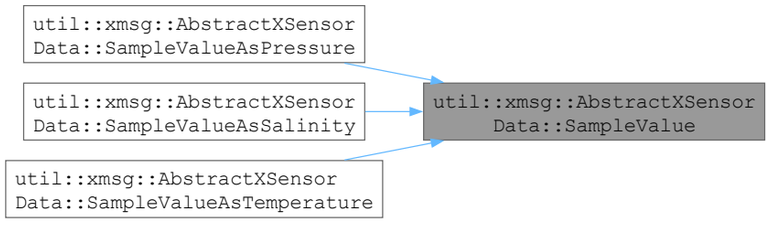
Returns

The i^{th} sensor value in scientific units.

References CATCH_BLOCK, and util::xmsg::AbstractXSensorData::raw_sensor_data_.

Referenced by util::xmsg::AbstractXSensorData::SampleValueAsPressure(), util::xmsg::AbstractXSensorData::SampleValueAsSalinity(), and util::xmsg::AbstractXSensorData::SampleValueAsTemperature().

Here is the caller graph for this function:



10.71.3.10 SampleValueAsPressure()

```
template<typename T = double>
T util::xmsg::AbstractXSensorData::SampleValueAsPressure (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline], [inherited]
```

Get the i^{th} pressure sensor value, in dBar.

Parameters

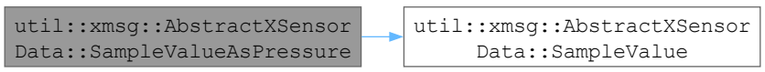
in	*i*	The index of the sensor value.
in	*gain*	The pressure sensor sensitivity scale factor, in LSB/dBar.
in	*offs*	The pressure sensor offset, in dBar.

Returns

The i^{th} pressure sensor value, in dBar.

References util::xmsg::AbstractXSensorData::SampleValue(), XMSG_SENS_BAD_PRES, and XMSG_SENS_MAX_PRES.

Here is the call graph for this function:



10.71.3.11 SampleValueAsSalinity()

```
template<typename T = double>
T util::xmsg::AbstractXSensorData::SampleValueAsSalinity (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline], [inherited]
```

Get the i^{th} salinity sensor value, in PSU.

Parameters

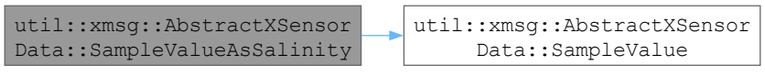
in	*i*	The index of the sensor value.
in	*gain*	The salinity sensor sensitivity scale factor, in LSB/PSU.
in	*offs*	The salinity sensor offset, in PSU.

Returns

The i^{th} salinity sensor value, in PSU.

References util::xmsg::AbstractXSensorData::SampleValue(), XMSG_SENS_BAD_SALN, and XMSG_SENS_MAX_SALN.

Here is the call graph for this function:



10.71.3.12 SampleValueAsTemperature()

```
template<typename T = double>
T util::xmsg::AbstractXSensorData::SampleValueAsTemperature (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline], [inherited]
```

Get the i^{th} temperature sensor value, in degC.

Parameters

in	*i*	The index of the sensor value.
in	*gain*	The temperature sensor sensitivity scale factor, in LSB/degC.
in	*offs*	The temperature sensor offset, in degC.

Returns

The i^{th} temperature sensor value, in degC.

References util::xmsg::AbstractXSensorData::SampleValue(), XMSG_SENS_BAD_TEMP, and XMSG_SENS_MAX_TEMP.

Here is the call graph for this function:

10.71.3.13 sensor_data_format()

```
XSensorDataFormat_t util::xmsg::AbstractXSensorData::sensor_data_format () const  [inline], [inherited]
```

Get the enumerated sensor data format value.

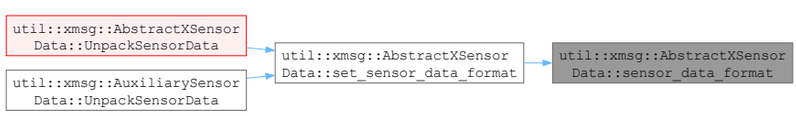
Returns

The enumerated sensor data format value.

References util::xmsg::AbstractXSensorData::sensor_data_format_.

Referenced by util::xmsg::AbstractXSensorData::set_sensor_data_format().

Here is the caller graph for this function:



10.71.3.14 set_id_dive()

```
void util::xmsg::XMessage::set_id_dive (
            int16_t id_dive)  [inline], [protected], [inherited]
```

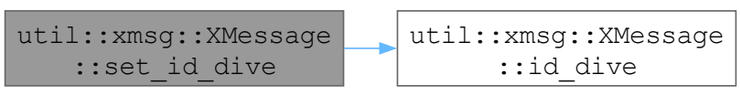
Set the dive or cycle number of this message.

Parameters

in	*id_dive*	The dive or cycle number of this message to set.

References util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::id_dive_.

Here is the call graph for this function:



10.71.3.15 set_id_instrument()

```
void util::xmsg::XMessage::set_id_instrument (
            uint16_t id_instrument)  [inline], [protected], [inherited]
```

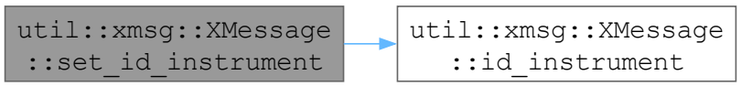
Set the unique instrument identifier.

Parameters

in	*id_instrument*	The unique instrument identifier to set.

References util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::id_instrument_.

Here is the call graph for this function:



10.71.3.16 set_id_xmsg_type()

```
void util::xmsg::XMessage::set_id_xmsg_type (
            XMsgTypeId_t id_xmsg_type)  [inline], [protected], [inherited]
```

Set the X-Message type identifier.

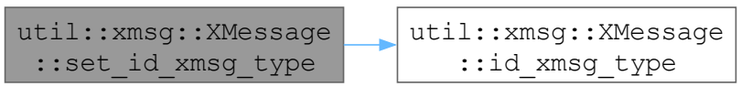
Parameters

in	*id_xmsg_type*	The X-Message type identifier to set.

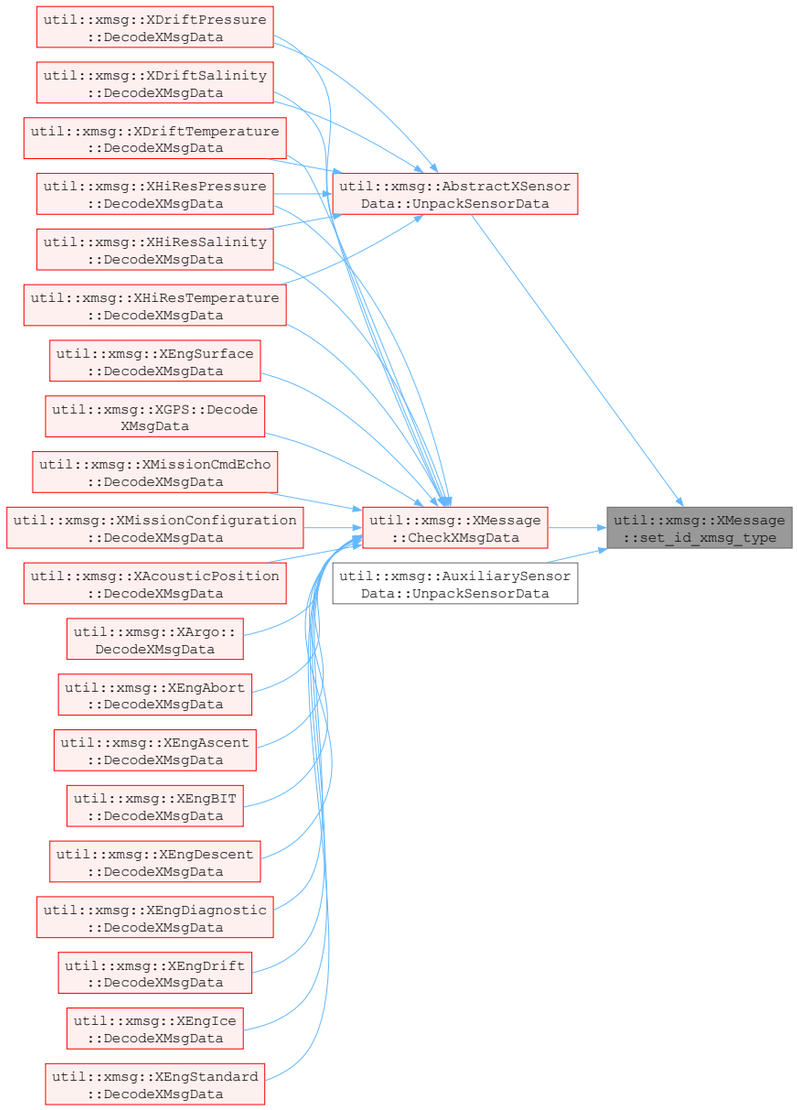
References util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.71.3.17 set_raw_sensor_data()

```
void util::xmsg::AbstractXSensorData::set_raw_sensor_data (
            const vector< int32_t > & raw_sensor_data)  [inline], [protected], [inherited]
```

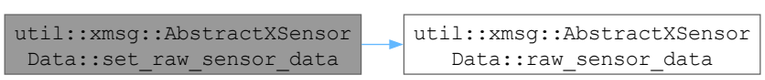
Set raw integer counts of the measured sensor data.

Parameters

in	*raw_sensor_data*	The data to set.

References util::xmsg::AbstractXSensorData::raw_sensor_data(), and util::xmsg::AbstractXSensorData::raw_sensor_data_.

Here is the call graph for this function:



10.71.3.18 set_sensor_data_format()

```
void util::xmsg::AbstractXSensorData::set_sensor_data_format (
            XSensorDataFormat_t sensor_data_format)  [inline], [protected], [inherited]
```

Set the sensor data format.

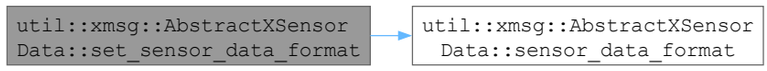
Parameters

in	*sensor_data_format*	The sensor data format to set.

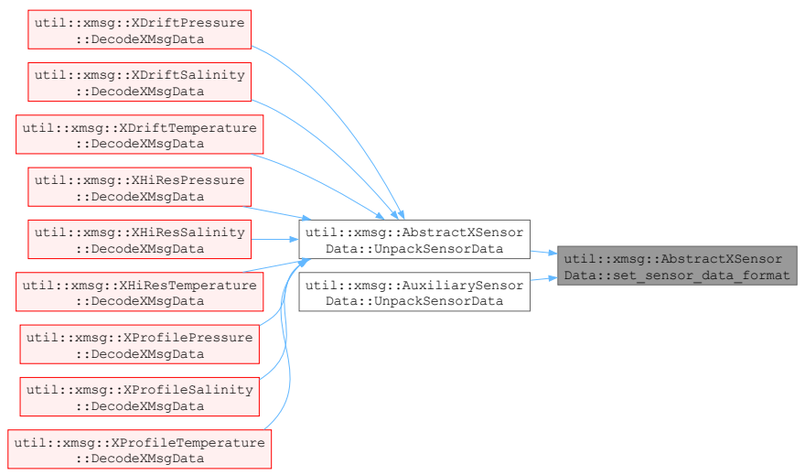
References util::xmsg::AbstractXSensorData::sensor_data_format(), and util::xmsg::AbstractXSensorData::sensor_data_format_.

Referenced by util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.71.3.19 set_xmsg_index()

```
void util::xmsg::XMessageIndexable::set_xmsg_index (
            uint8_t xmsg_index)  [inline], [protected], [inherited]
```

Set the index of this X-Message.

Parameters

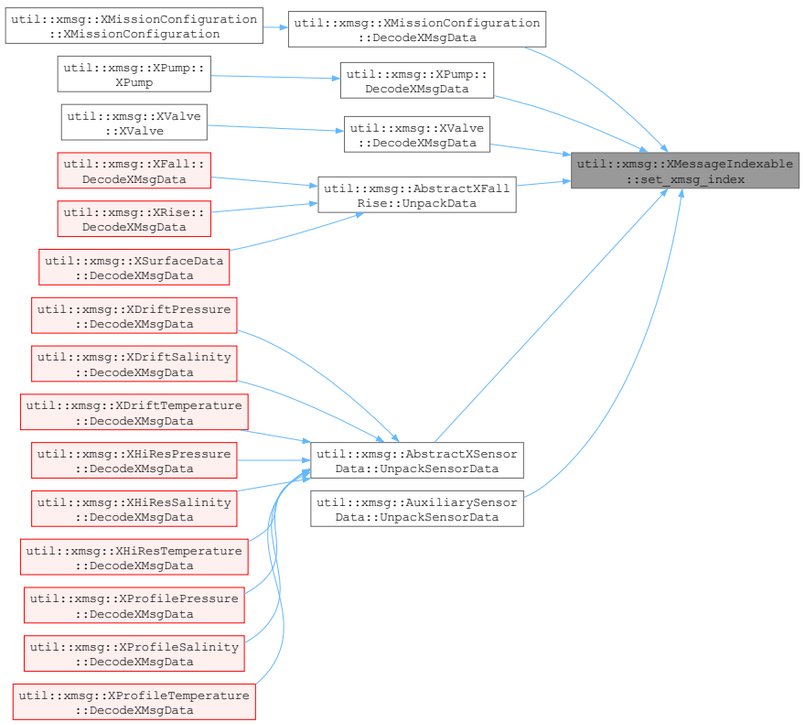
in	*xmsg_index*	The index of this X-Message.

References util::xmsg::XMessageIndexable::xmsg_index(), and util::xmsg::XMessageIndexable::xmsg_index_.

Referenced by util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(), util::xmsg::XValve::DecodeXMsgData(), util::xmsg::AbstractXFallRise::UnpackData(), util::xmsg::AbstractXSensorData::UnpackSensorDa and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:

Here is the caller graph for this function:



10.71.3.20 set_xmsg_size_bytes()

```
void util::xmsg::XMessage::set_xmsg_size_bytes (
            uint16_t xmsg_size_bytes)  [inline], [protected], [inherited]
```

Set the size of the X-Message, in bytes.

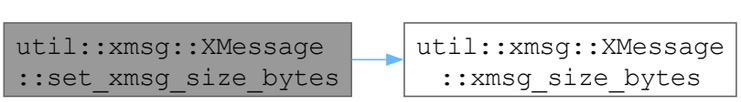
Parameters

in	*xmsg_size_bytes*	The size of the X-Message to set, in bytes.

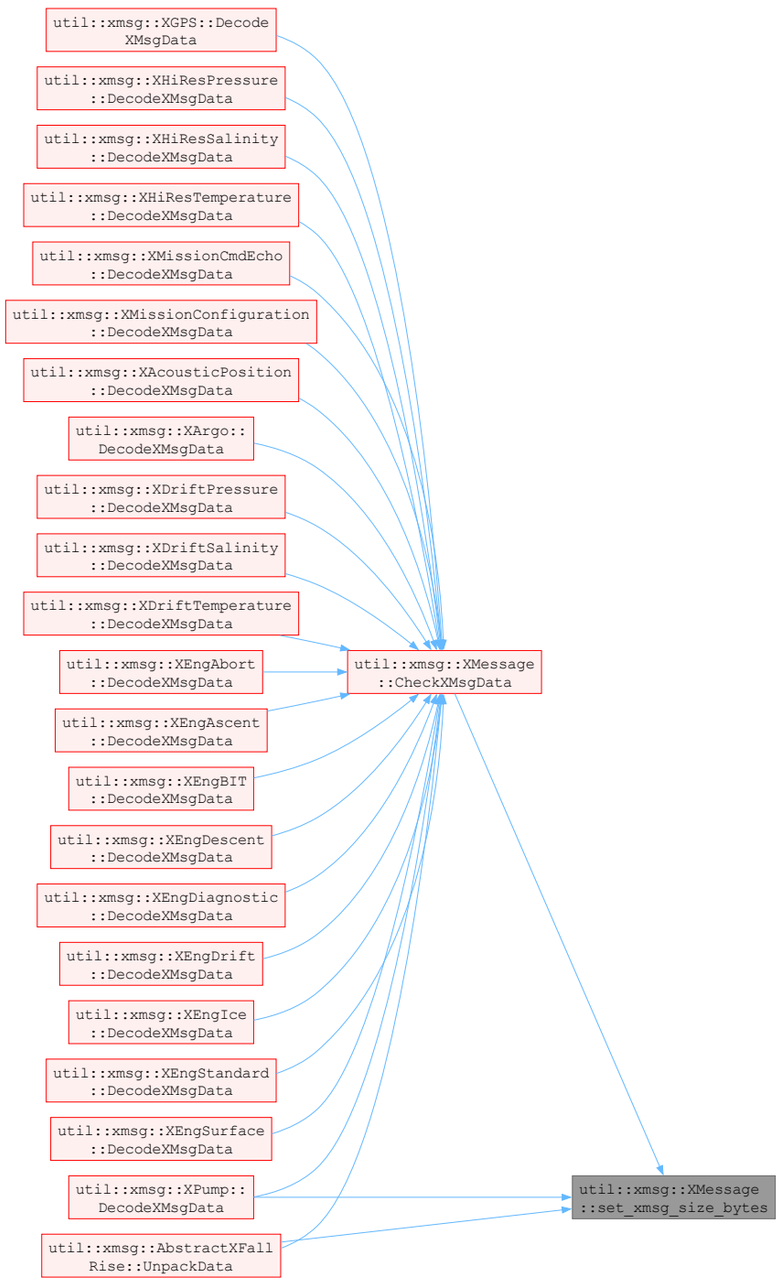
References util::xmsg::XMessage::xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XPump::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::Unpa

Here is the call graph for this function:



Here is the caller graph for this function:



10.71.3.21 UnpackSensorData()

```
virtual vector< int32_t > util::xmsg::AbstractXSensorData::UnpackSensorData (
            const vector< uint8_t > & data)  [inline], [protected], [virtual], [inherited]
```

Unpack the sensor data from the binary message.

This field unpacks the encoded sensor data from the message and stores the decoded sensor values in raw_sensor_data_. First the function checks the size of the message, and the data packing format. The message data is decoded according to the packing format type. Both first-difference and curvature encoding are supported. The encoding method is determined from the value of the high nibble of byte `data`[1].

Once this function completes successfully, then the data may be recovered in scientific units from the raw sensor values by applying the appropriate sensor gain and offset values.

Parameters

`in`	*data*	The binary message to unpack, in a vector of unsigned bytes.

Warning

The function assumes the message data is valid. Message validity should be ensured before this function is called.

Exceptions

util::log::MessageException	if • an error occurs while decoding the message, • the message is of insufficient length, or • an invalid packing format is encountered.

Returns

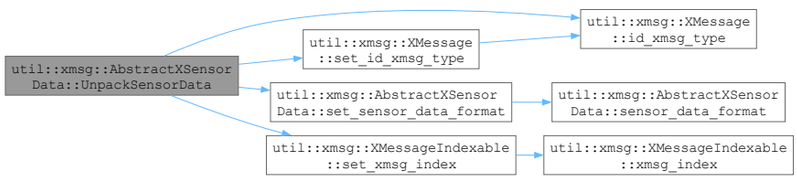
The sensor data raw counts, in a vector of signed integers.

Reimplemented in util::xmsg::AuxiliarySensorData.

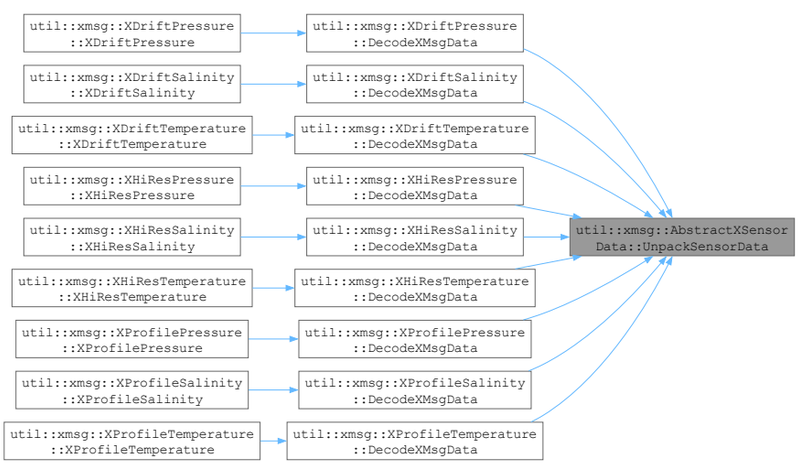
References CATCH_BLOCK, CATCH_NOTHROW, EXC_THROW, i24, util::xmsg::XMessage::id_xmsg_type(), ldbg, lwrn, util::xmsg::AbstractXSensorData::raw_sensor_data_, util::xmsg::XMessage::set_id_xmsg_type(), util::xmsg::AbstractXSensorData::s util::xmsg::XMessageIndexable::set_xmsg_index(), u16, u24, XMSG_SENS_MAX_SUB, util::xmsg::XMSG_TYPE_ID_PRES0, util::xmsg::XMSG_TYPE_ID_PRES_DRIFT8, util::xmsg::XMSG_TYPE_ID_PRES_HI_RES0, util::xmsg::XMSG_TYPE_ID_SALN0, util::xmsg::XMSG_TYPE_ID_SALN_DRIFT8, util::xmsg::XMSG_TYPE_ID_SALN_HI_RES0, util::xmsg::XMSG_TYPE_ID_TEMP0, util::xmsg::XMSG_TYPE_ID_TEMP_DRIFT8, util::xmsg::XMSG_TYPE_ID_TEMP_HI_RES0, util::xmsg::XSENS_FMT_CURVATURE, and util::xmsg::XSENS_FMT_DIFFERENCE.

Referenced by util::xmsg::XDriftPressure::DecodeXMsgData(), util::xmsg::XDriftSalinity::DecodeXMsgData(), DecodeXMsgData(), util::xmsg::XHiResPressure::DecodeXMsgData(), util::xmsg::XHiResSalinity::DecodeXMsgData(), util::xmsg::XHiResTemperature::Decod util::xmsg::XProfilePressure::DecodeXMsgData(), util::xmsg::XProfileSalinity::DecodeXMsgData(), and util::xmsg::XProfileTemperature::D

Here is the call graph for this function:



Here is the caller graph for this function:



10.71.3.22 xmsg_index()

```
uint8_t util::xmsg::XMessageIndexable::xmsg_index () const  [inline], [inherited]
```

Get the index of this X-Message.

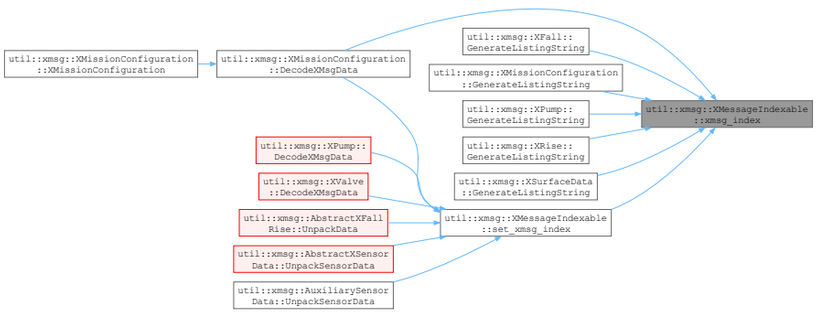
Returns

The index of this X-Message.

References util::xmsg::XMessageIndexable::xmsg_index_.

Referenced by util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing util::xmsg::XSurfaceData::GenerateListingString(), and util::xmsg::XMessageIndexable::set_xmsg_index().

Here is the caller graph for this function:



10.71.3.23 xmsg_size_bytes()

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes () const  [inline], [inherited]
```

Get the size of the X-Message, in bytes.

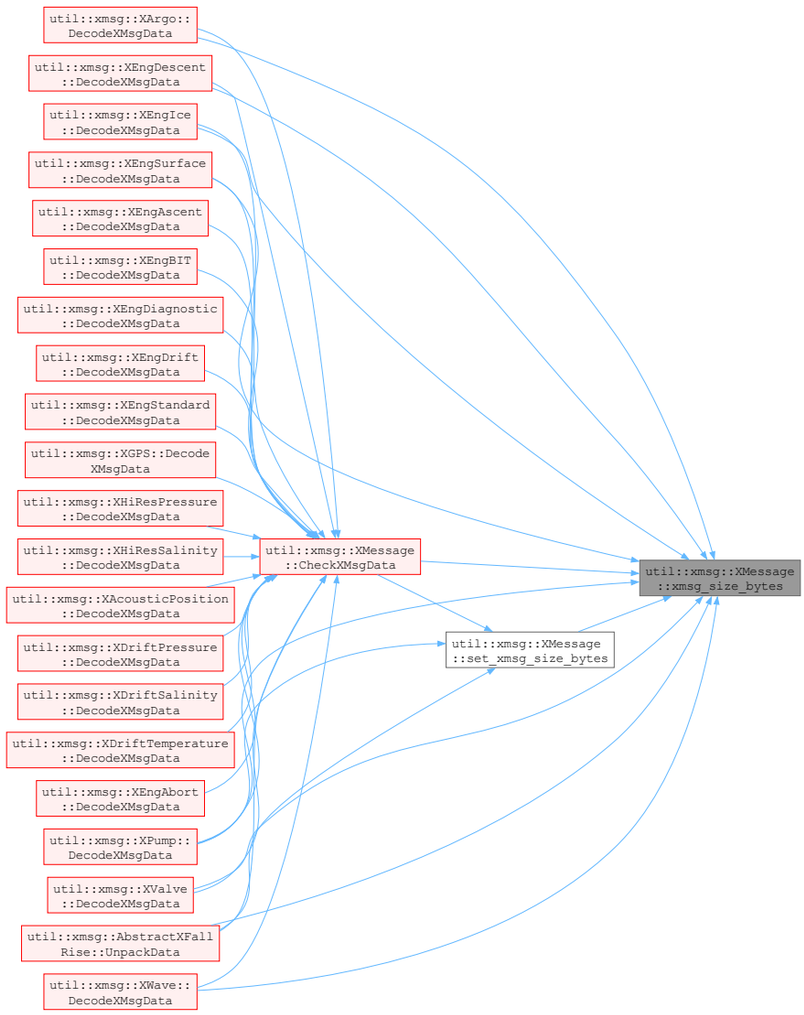
Returns

The size of the X-Message, in bytes.

References util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgI util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(), util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the caller graph for this function:



10.71.4 Member Data Documentation

10.71.4.1 id_dive_

```
int16_t util::xmsg::XMessage::id_dive_  [private], [inherited]
```

The dive or cycle number of this message.

Referenced by util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::set_id_dive().

10.71.4.2 id_instrument_

```
uint16_t util::xmsg::XMessage::id_instrument_  [private], [inherited]
```

The unique WHOI identifier of this instrument.

Referenced by util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::set_id_instrument().

10.71.4.3 id_xmsg_type_

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type_  [private], [inherited]
```

The X-message type identifier.

Referenced by util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::set_id_xmsg_type().

10.71.4.4 raw_sensor_data_

```
vector<int32_t> util::xmsg::AbstractXSensorData::raw_sensor_data_  [protected], [inherited]
```

The raw integer counts of the measured sensor data.

This vector of integers represents the "raw" sample values of the message. That is, by applying the appropriate gain and offset corrections, the sensor values may be recovered in scientific units. This data may represent pressure, temperature, salinity, or any other data that is recorded during the drift or profile phases of the dive cycle.

Referenced by util::xmsg::AbstractXSensorData::num_samples(), util::xmsg::AbstractXSensorData::raw_sensor_data(), util::xmsg::AbstractXSensorData::Sample(), util::xmsg::AbstractXSensorData::SampleValue(), util::xmsg::AbstractXSensorData::set_raw_ util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

10.71.4.5 sensor_data_format_

```
XSensorDataFormat_t util::xmsg::AbstractXSensorData::sensor_data_format_  [protected], [inherited]
```

The enumerated sensor data packing format.

This field indicates the format of encoded sensor data. The manner in which the sensor data is decoded is determined by the format of this field.

Referenced by util::xmsg::AbstractXSensorData::sensor_data_format(), and util::xmsg::AbstractXSensorData::set_sensor_data_format().

10.71.4.6 xmsg_index_

```
uint8_t util::xmsg::XMessageIndexable::xmsg_index_  [private], [inherited]
```

The index of this X-Message.

Referenced by util::xmsg::XMessageIndexable::set_xmsg_index(), and util::xmsg::XMessageIndexable::xmsg_index().

10.71.4.7 xmsg_size_bytes_

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes_  [private], [inherited]
```

The size of the X message, in bytes.

Referenced by util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes().

The documentation for this class was generated from the following file:

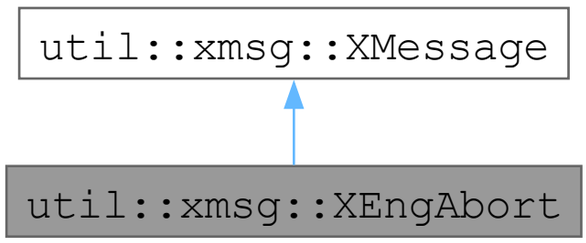
- data/xmsg/XDriftTemperature.h

10.72 util::xmsg::XEngAbort Class Reference

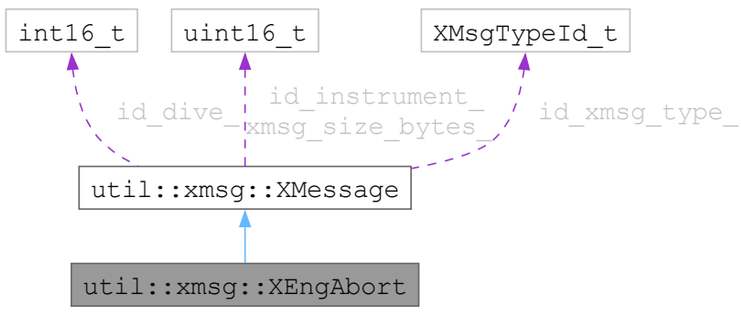
Class representing an abort/beacon mode engineering X-Message.

`#include <XEngAbort.h>`

Inheritance diagram for util::xmsg::XEngAbort:



Collaboration diagram for util::xmsg::XEngAbort:



Public Member Functions

- XEngAbort (int16_t dive, uint16_t whoi, const vector< uint8_t > &data)

  Parameterized constructor.
- ~XEngAbort ()

  Destructor.
- const uint8_t * CheckXMsgData (const vector< uint8_t > &data, int id, int minBytes, bool parseSize=true)

  Check the X-message for the correct ID, minimum size, and message terminator.
- void DecodeXMsgData (const vector< uint8_t > &data)

  Decode the abort/beacon mode engineering X-Message from the binary data.
- const string GenerateListingString () const

  Generate a listing string representation of the abort/beacon mode engineering data message.
- int16_t id_dive () const

  Get the dive or cycle number of this X-Message.
- uint16_t id_instrument () const

  Get the unique instrument identifier of this X-Message.
- XMsgTypeId_t id_xmsg_type () const

  Get the enumerated X-Message type identifier.
- uint16_t xmsg_size_bytes () const

  Get the size of the X-Message, in bytes.

Protected Member Functions

- void set_id_dive (int16_t id_dive)

  Set the dive or cycle number of this message.
- void set_id_instrument (uint16_t id_instrument)

  Set the unique instrument identifier.
- void set_id_xmsg_type (XMsgTypeId_t id_xmsg_type)

  Set the X-Message type identifier.
- void set_xmsg_size_bytes (uint16_t xmsg_size_bytes)

  Set the size of the X-Message, in bytes.

Private Attributes

- int16_t id_dive_

  The dive or cycle number of this message.
- uint16_t id_instrument_

  The unique WHOI identifier of this instrument.
- XMsgTypeId_t id_xmsg_type_

  The X-message type identifier.
- uint16_t xmsg_size_bytes_

  The size of the X message, in bytes.

10.72.1 Detailed Description

Class representing an abort/beacon mode engineering X-Message.

10.72.2 Constructor & Destructor Documentation

10.72.2.1 XEngAbort()

```
util::xmsg::XEngAbort::XEngAbort (
            int16_t dive,
            uint16_t whoi,
            const vector< uint8_t > & data)  [inline]
```

Parameterized constructor.

Parameters

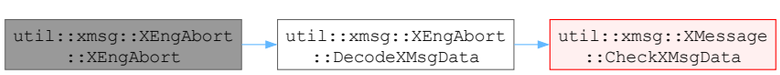
in	*dive*	The dive or cycle number of this message.
in	*whoi*	The unique WHOI identifier.
in	*data*	The binary X-message data to decode.

Exceptions

util::log::MessageException	if an decode error occurs.

References DecodeXMsgData().

Here is the call graph for this function:



10.72.2.2 ~XEngAbort()

```
util::xmsg::XEngAbort::~XEngAbort ()  [inline]
```

Destructor.

10.72.3 Member Function Documentation

10.72.3.1 CheckXMsgData()

```
const uint8_t * util::xmsg::XMessage::CheckXMsgData (
            const vector< uint8_t > & data,
            int id,
            int minBytes,
            bool parseSize = true)  [inline], [inherited]
```

Check the X-message for the correct ID, minimum size, and message terminator.

Parameters

in	*data*	The X-message binary data to check.
in	*id*	The expected ID of the X-message. If this field is less that 0, then the ID check will be skipped.
in	*minBytes*	The minimum number of bytes expected in a valid message.
in	*parseSize*	If TRUE, then the message size field will be parsed from the X-Message. Otherwise, the size field will not be parsed.

Exceptions

util::log::MessageException	if • the message does not contain the expected ID, • the message does not contain the minimum number of expected bytes, • the message does not contain a terminator (';') in the correct location, or, • any other error occurs.

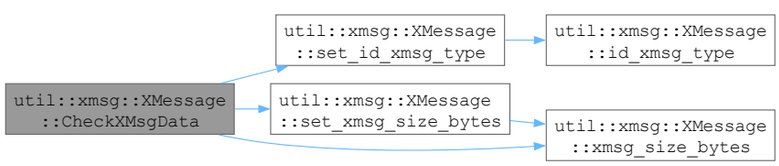
Returns

A pointer to the underlying data buffer of data.

References CATCH_BLOCK, EXC_THROW, lwrn, util::xmsg::XMessage::set_id_xmsg_type(), util::xmsg::XMessage::set_xmsg_size_byte
u16, and util::xmsg::XMessage::xmsg_size_bytes().

Referenced by util::xmsg::XAcousticPosition::DecodeXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XDriftPressure::Dec
util::xmsg::XDriftSalinity::DecodeXMsgData(), util::xmsg::XDriftTemperature::DecodeXMsgData(), DecodeXMsgData(),
util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngBIT::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(),
util::xmsg::XEngDiagnostic::DecodeXMsgData(), util::xmsg::XEngDrift::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(),
util::xmsg::XEngStandard::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XGPS::DecodeXMsgData(),
util::xmsg::XHiResPressure::DecodeXMsgData(), util::xmsg::XHiResSalinity::DecodeXMsgData(), util::xmsg::XHiResTemperature::Decod
util::xmsg::XMissionCmdEcho::DecodeXMsgData(), util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XProfilePressure:
util::xmsg::XProfileSalinity::DecodeXMsgData(), util::xmsg::XProfileTemperature::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgD
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the call graph for this function:



10.72.3.2 DecodeXMsgData()

```
void util::xmsg::XEngAbort::DecodeXMsgData (
            const vector< uint8_t > & data)  [inline], [virtual]
```

Decode the abort/beacon mode engineering X-Message from the binary data.

Parameters

in	*data*	The binary message to decode.

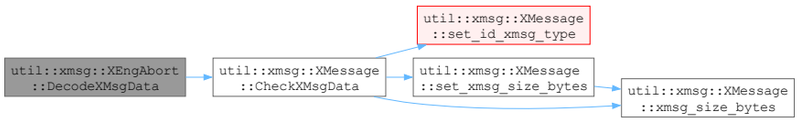
Todo Implement the functionality to decode the abort/beacon mode engineering X-Message from the binary data.

Implements util::xmsg::XMessage.

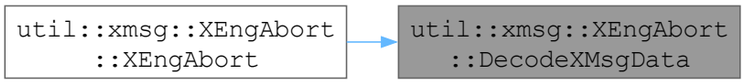
References CATCH_BLOCK, util::xmsg::XMessage::CheckXMsgData(), and util::xmsg::XMSG_TYPE_ID_ENG_ABORT.

Referenced by XEngAbort().

Here is the call graph for this function:



Here is the caller graph for this function:



10.72.3.3 GenerateListingString()

```
const string util::xmsg::XEngAbort::GenerateListingString () const  [inline]
```

Generate a listing string representation of the abort/beacon mode engineering data message.

Returns

A listing string representation of the abort/beacon mode engineering data message.

Todo Implement the functionality for generating a listing string for the Solo-II abort/beacon mode engineering data message.

References CATCH_BLOCK.

10.72.3.4 id_dive()

```
int16_t util::xmsg::XMessage::id_dive () const  [inline], [inherited]
```

Get the dive or cycle number of this X-Message.

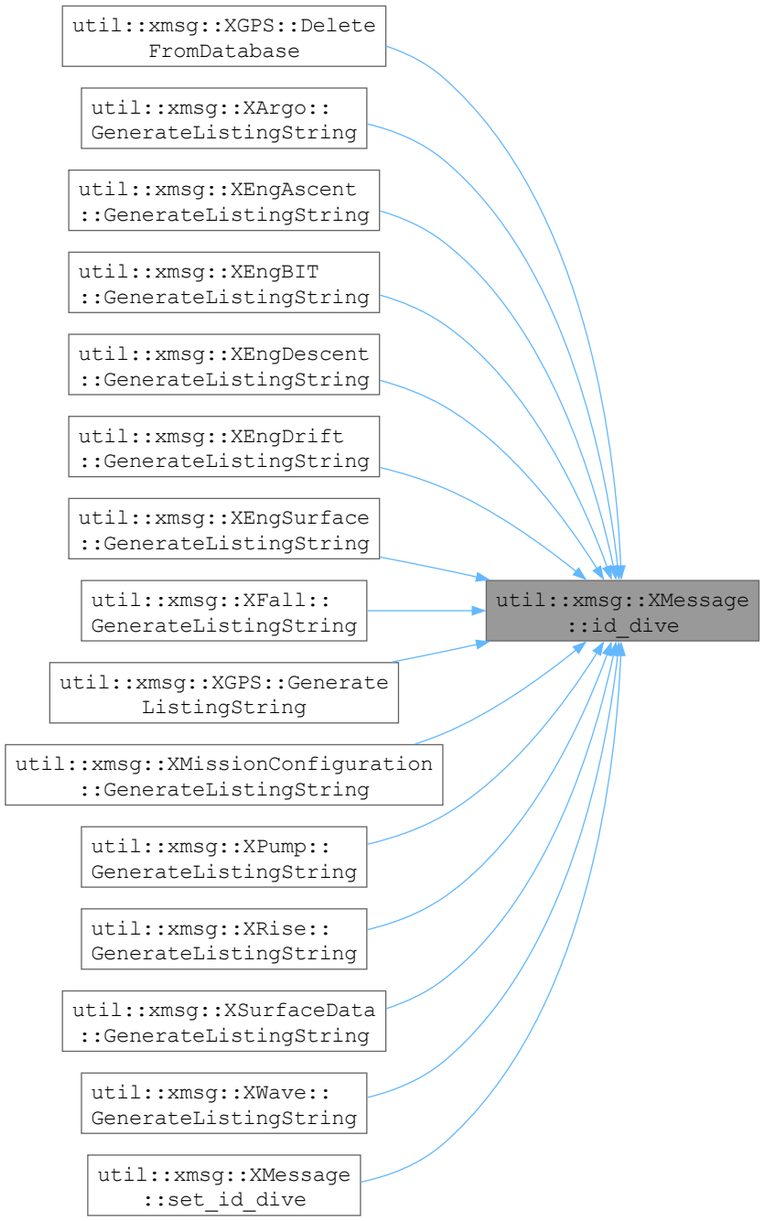
Returns

The dive or cycle number of this X-Message.

References util::xmsg::XMessage::id_dive_.

Referenced by util::xmsg::XGPS::DeleteFromDatabase(), util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateLis
util::xmsg::XEngBIT::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingStr
util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(),
util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing
util::xmsg::XSurfaceData::GenerateListingString(), util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_dive().

Here is the caller graph for this function:



10.72.3.5 id_instrument()

```
uint16_t util::xmsg::XMessage::id_instrument () const  [inline], [inherited]
```

Get the unique instrument identifier of this X-Message.

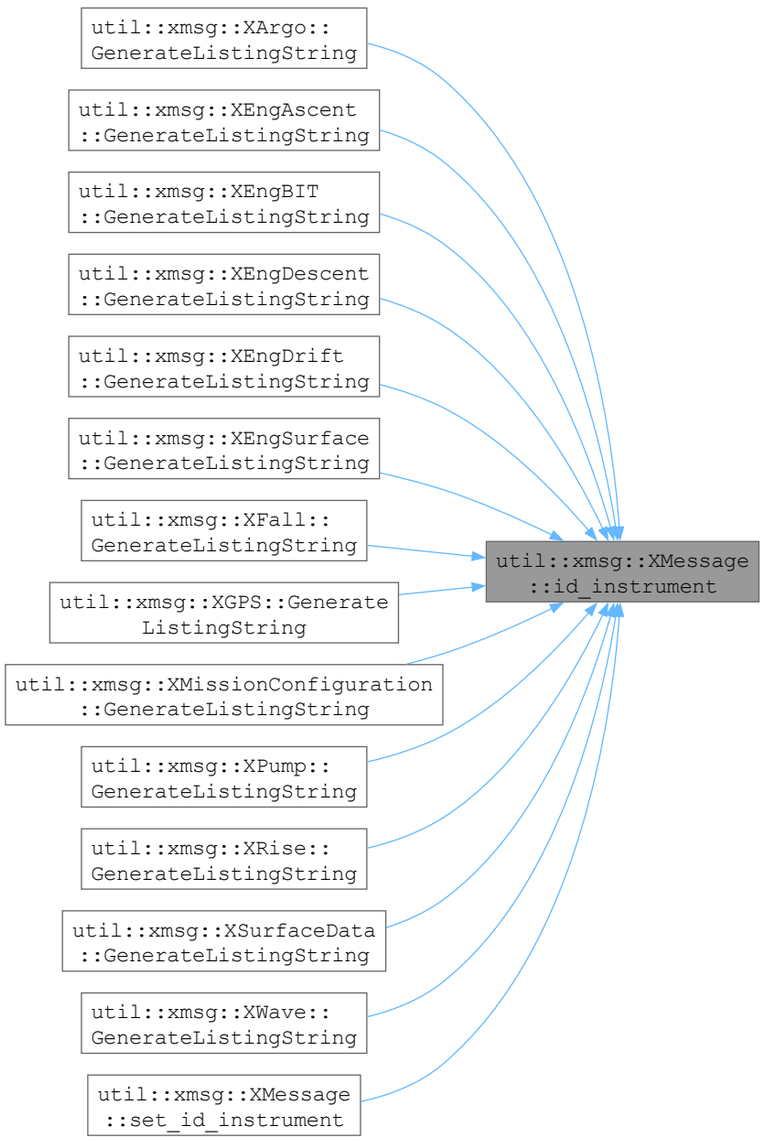
Returns

The unique instrument identifier of this X-Message.

References util::xmsg::XMessage::id_instrument_.

Referenced by util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngBIT::Generate
util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), util::xmsg::XEngSurface::GenerateListin
util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingS
util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(),
util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_instrument().

Here is the caller graph for this function:



10.72.3.6 id_xmsg_type()

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type () const  [inline], [inherited]
```

Get the enumerated X-Message type identifier.

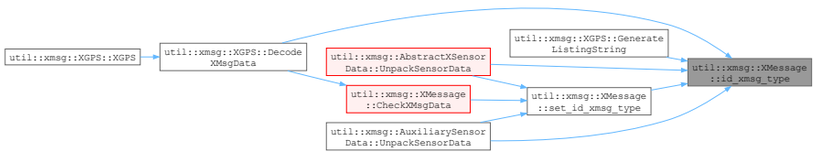
Returns

The enumerated X-Message type identifier.

References util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMessage::set_id_xmsg_typ
util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the caller graph for this function:



10.72.3.7 set_id_dive()

```
void util::xmsg::XMessage::set_id_dive (
            int16_t id_dive)  [inline], [protected], [inherited]
```

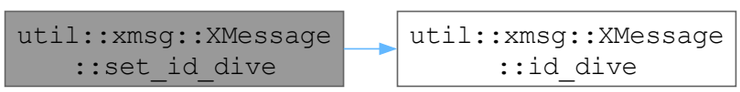
Set the dive or cycle number of this message.

Parameters

in	*id_dive*	The dive or cycle number of this message to set.

References util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::id_dive_.

Here is the call graph for this function:



10.72.3.8 set_id_instrument()

```
void util::xmsg::XMessage::set_id_instrument (
            uint16_t id_instrument)  [inline], [protected], [inherited]
```

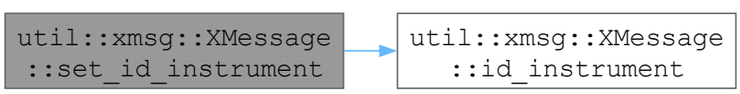
Set the unique instrument identifier.

Parameters

in	*id_instrument*	The unique instrument identifier to set.

References util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::id_instrument_.

Here is the call graph for this function:



10.72.3.9 set_id_xmsg_type()

```
void util::xmsg::XMessage::set_id_xmsg_type (
            XMsgTypeId_t id_xmsg_type)  [inline], [protected], [inherited]
```

Set the X-Message type identifier.

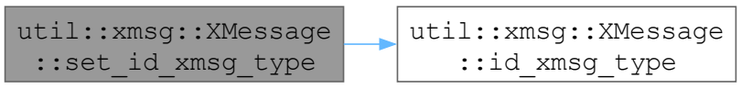
Parameters

in	*id_xmsg_type*	The X-Message type identifier to set.

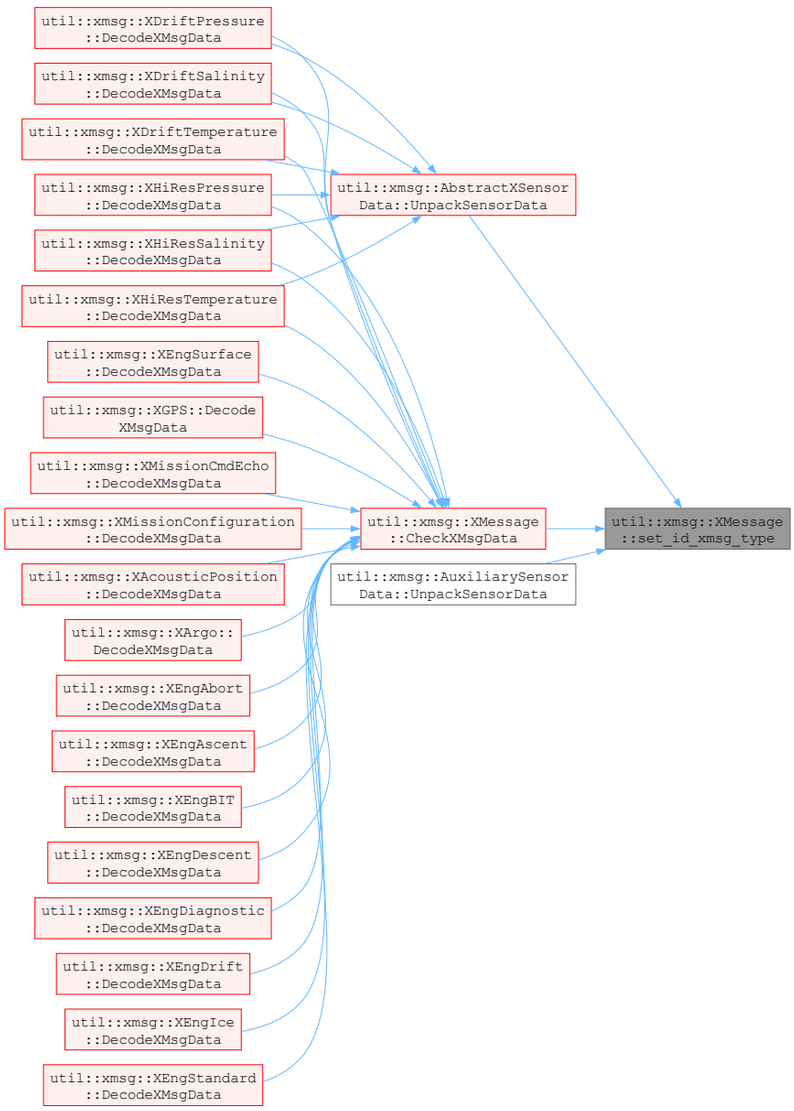
References util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.72.3.10 set_xmsg_size_bytes()

```
void util::xmsg::XMessage::set_xmsg_size_bytes (
            uint16_t xmsg_size_bytes)  [inline], [protected], [inherited]
```

Set the size of the X-Message, in bytes.

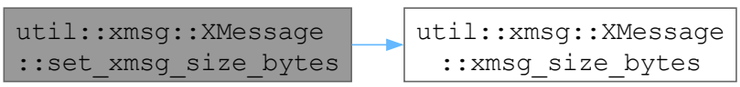
Parameters

in	*xmsg_size_bytes*	The size of the X-Message to set, in bytes.

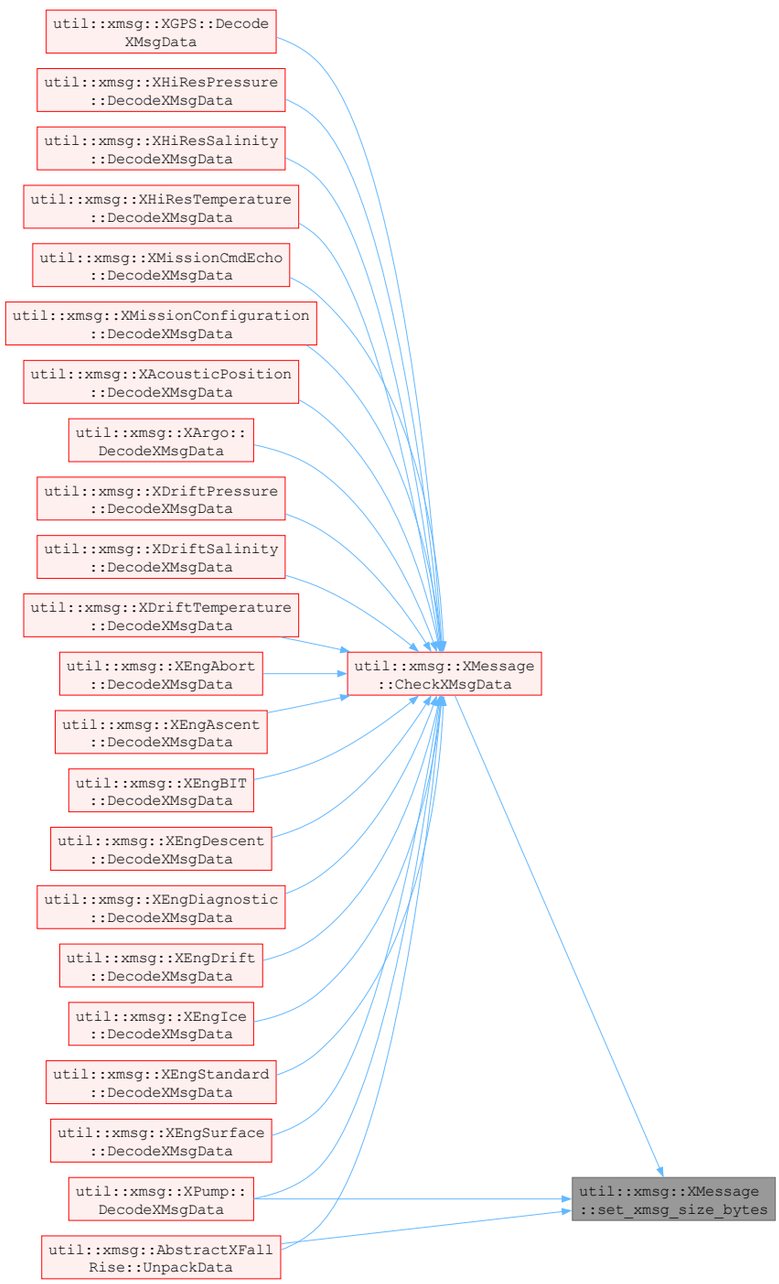
References util::xmsg::XMessage::xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XPump::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::Unpa

Here is the call graph for this function:



Here is the caller graph for this function:



10.72.3.11 xmsg_size_bytes()

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes () const  [inline], [inherited]
```

Get the size of the X-Message, in bytes.

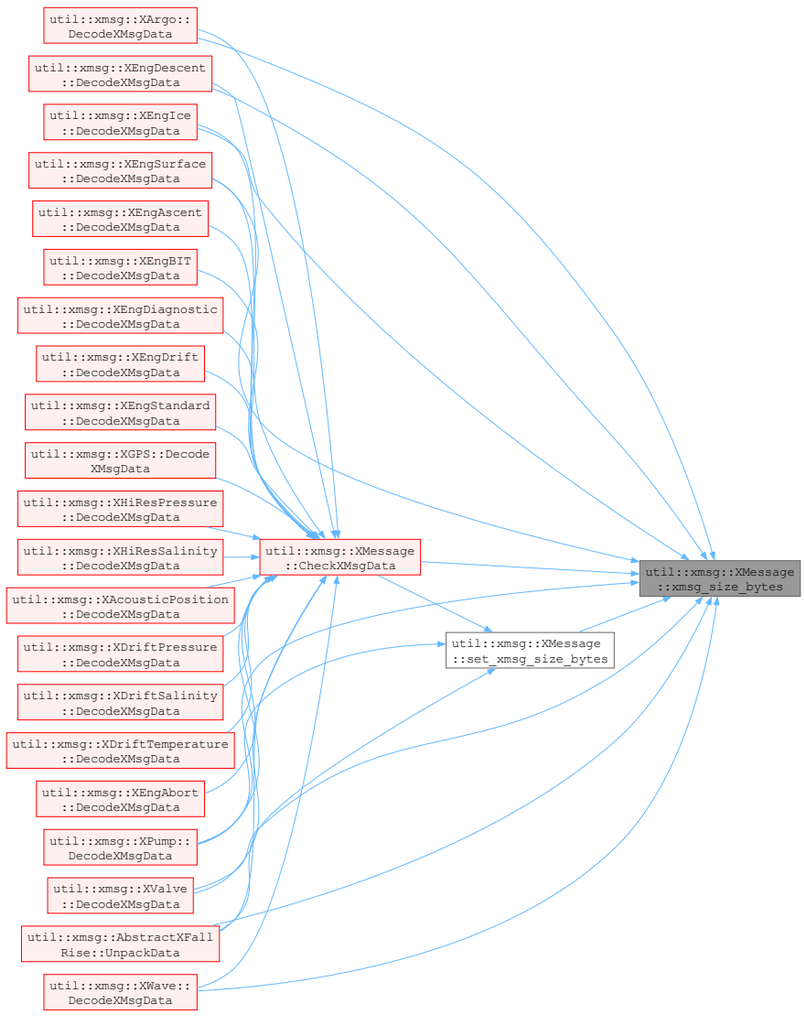
Returns

The size of the X-Message, in bytes.

References util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgI
util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(),
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), util::xmsg::XMessage::set_xmsg_size_bytes(),
and util::xmsg::AbstractXFallRise::UnpackData().

Here is the caller graph for this function:



10.72.4 Member Data Documentation

10.72.4.1 id_dive_

```
int16_t util::xmsg::XMessage::id_dive_  [private], [inherited]
```

The dive or cycle number of this message.

Referenced by util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::set_id_dive().

10.72.4.2 id_instrument_

`uint16_t util::xmsg::XMessage::id_instrument_ [private], [inherited]`

The unique WHOI identifier of this instrument.

Referenced by util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::set_id_instrument().

10.72.4.3 id_xmsg_type_

`XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type_ [private], [inherited]`

The X-message type identifier.

Referenced by util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::set_id_xmsg_type().

10.72.4.4 xmsg_size_bytes_

`uint16_t util::xmsg::XMessage::xmsg_size_bytes_ [private], [inherited]`

The size of the X message, in bytes.

Referenced by util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes().

The documentation for this class was generated from the following file:

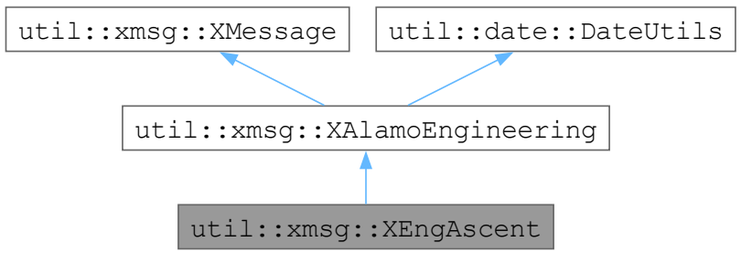
- data/xmsg/XEngAbort.h

10.73 util::xmsg::XEngAscent Class Reference

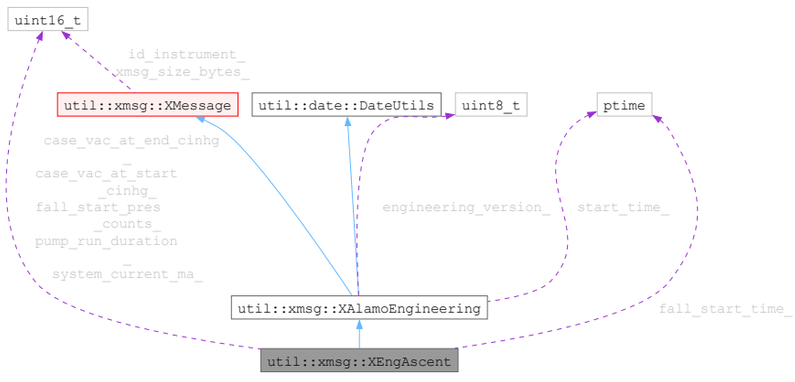
Class representing an ascend state engineering X-Message.

`#include <XEngAscent.h>`

Inheritance diagram for util::xmsg::XEngAscent:



Collaboration diagram for util::xmsg::XEngAscent:



Public Member Functions

- XEngAscent (int16_t dive, uint16_t whoi, const vector< uint8_t > &data)

  Parameterized constructor.
- ~XEngAscent ()

  Destructor.
- uint16_t **case_vac_at_end_cinhg** () const
- template<typename T = float>
  T **case_vac_at_end_inhg** () const
- uint16_t **case_vac_at_start_cinhg** () const
- template<typename T = float>
  T **case_vac_at_start_inhg** () const
- const uint8_t * CheckXMsgData (const vector< uint8_t > &data, int id, int minBytes, bool parseSize=true)

  Check the X-message for the correct ID, minimum size, and message terminator.
- void DecodeXMsgData (const vector< uint8_t > &data)

  Decode the ascent state engineering X-Message from the binary data.
- uint8_t engineering_version () const

  Get the engineering message version identifier.
- float engineering_version_as_float () const

  Get the engineering message verion identifier as a floating point number.
- uint16_t **fall_start_pres_counts** () const
- template<typename T = float>
  T **fall_start_pres_dbar** (int16_t gain, int16_t offs) const
- const ptime & **fall_start_time** () const
- const string GenerateListingString (uint16_t pg, uint16_t po, uint16_t sg, uint16_t so, uint16_t tg, uint16_t to) const

  Get the listing entry string for this ascent engineering data X-Message.
- int16_t id_dive () const

  Get the dive or cycle number of this X-Message.
- uint16_t id_instrument () const

  Get the unique instrument identifier of this X-Message.
- XMsgTypeId_t id_xmsg_type () const

  Get the enumerated X-Message type identifier.
- uint16_t **pump_run_duration** () const
- const ptime & start_time () const

  Get the engineering message state start time.
- template<typename T = float>
  T system_current_a () const

  Get the system current, in amps.
- uint16_t **system_current_ma** () const
- uint16_t xmsg_size_bytes () const

  Get the size of the X-Message, in bytes.

Static Public Member Functions

- static string GenerateCurrentUtcTimeIso8601String ()

  Generate an ISO8601 format date/time string representation of the current time, in UTC.
- static string GenerateIso8601String (const ptime &p, bool extended=true)

  Generate an ISO8601 format date/time string representation of a POSIX time object.
- static double GenerateJulianDay (const ptime &dt, const ptime &ref_dt)

  Convert a date/time to fractional days since a reference date/time.
- static ptime GetCurrentUtcTime ()

  Generate a POSIX time object representing the current date and time (rounded to the nearest second).

Protected Member Functions

- void **set_case_vac_at_end_cinhg** (uint16_t case_vac_at_end_cinhg)
- void **set_case_vac_at_start_cinhg** (uint16_t case_vac_at_start_cinhg)
- void set_engineering_version (uint8_t engineering_version)

  Set the engineering message version identifier.
- void **set_fall_start_pres_counts** (uint16_t fall_start_pres)
- void **set_fall_start_time** (const ptime &fall_start_time)
- void set_id_dive (int16_t id_dive)

  Set the dive or cycle number of this message.
- void set_id_instrument (uint16_t id_instrument)

  Set the unique instrument identifier.
- void set_id_xmsg_type (XMsgTypeId_t id_xmsg_type)

  Set the X-Message type identifier.
- void **set_pump_run_duration** (uint16_t pump_run_duration)
- void set_start_time (const ptime &start_time)

  Set the engineering message state start time.
- void **set_system_current_ma** (uint16_t system_current_ma)
- void set_xmsg_size_bytes (uint16_t xmsg_size_bytes)

  Set the size of the X-Message, in bytes.

Private Attributes

- uint16_t case_vac_at_end_cinhg_

  The measured internal pres case vacuum, in 0.01 inHg, at the end of the ascent/profile phase of the dive cycle.
- uint16_t case_vac_at_start_cinhg_

  The measured internal pres case vacuum, in 0.01 inHg, at the start of the ascent/profile phase of the dive cycle.
- uint8_t engineering_version_

  The engineering message version identifier.
- uint16_t **fall_start_pres_counts_**
- ptime **fall_start_time_**
- int16_t id_dive_

  The dive or cycle number of this message.
- uint16_t id_instrument_

  The unique WHOI identifier of this instrument.
- XMsgTypeId_t id_xmsg_type_

  The X-message type identifier.
- uint16_t pump_run_duration_

  The cumulative pump run time during the ascent phase of the dive cycle, in seconds.
- ptime start_time_

  The engineering message state start time.
- uint16_t system_current_ma_

  The measured system current, in mA.
- uint16_t xmsg_size_bytes_

  The size of the X message, in bytes.

10.73.1 Detailed Description

Class representing an ascend state engineering X-Message.

Note

This X-Message is only used by ALAMO-type instruments.

This message type was added in X-Message version 2.1.

10.73.2 Constructor & Destructor Documentation

10.73.2.1 XEngAscent()

```
util::xmsg::XEngAscent::XEngAscent (
            int16_t dive,
            uint16_t whoi,
            const vector< uint8_t > & data)  [inline]
```

Parameterized constructor.

Parameters

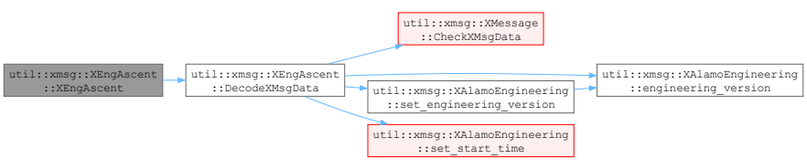
in	*dive*	The dive or cycle number of this message.
in	*whoi*	The unique WHOI identifier.
in	*data*	The binary X-message data to decode.

Exceptions

util::log::MessageException	if an decode error occurs.

References DecodeXMsgData().

Here is the call graph for this function:



10.73.2.2 ~XEngAscent()

```
util::xmsg::XEngAscent::~XEngAscent ()  [inline]
```

Destructor.

10.73.3 Member Function Documentation

10.73.3.1 CheckXMsgData()

```
const uint8_t * util::xmsg::XMessage::CheckXMsgData (
            const vector< uint8_t > & data,
            int id,
            int minBytes,
            bool parseSize = true)  [inline], [inherited]
```

Check the X-message for the correct ID, minimum size, and message terminator.

Parameters

in	*data*	The X-message binary data to check.
in	*id*	The expected ID of the X-message. If this field is less that 0, then the ID check will be skipped.
in	*minBytes*	The minimum number of bytes expected in a valid message.
in	*parseSize*	If TRUE, then the message size field will be parsed from the X-Message. Otherwise, the size field will not be parsed.

Exceptions

util::log::MessageException	if • the message does not contain the expected ID, • the message does not contain the minimum number of expected bytes, • the message does not contain a terminator (';') in the correct location, or, • any other error occurs.

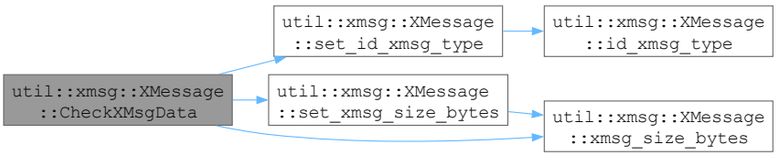
Returns

A pointer to the underlying data buffer of data.

References CATCH_BLOCK, EXC_THROW, lwrn, util::xmsg::XMessage::set_id_xmsg_type(), util::xmsg::XMessage::set_xmsg_size_byte
u16, and util::xmsg::XMessage::xmsg_size_bytes().

Referenced by util::xmsg::XAcousticPosition::DecodeXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XDriftPressure::Dec
util::xmsg::XDriftSalinity::DecodeXMsgData(), util::xmsg::XDriftTemperature::DecodeXMsgData(), util::xmsg::XEngAbort::DecodeXMsgDa
DecodeXMsgData(), util::xmsg::XEngBIT::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(),
util::xmsg::XEngDiagnostic::DecodeXMsgData(), util::xmsg::XEngDrift::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(),
util::xmsg::XEngStandard::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XGPS::DecodeXMsgData(),
util::xmsg::XHiResPressure::DecodeXMsgData(), util::xmsg::XHiResSalinity::DecodeXMsgData(), util::xmsg::XHiResTemperature::Decod
util::xmsg::XMissionCmdEcho::DecodeXMsgData(), util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XProfilePressure:
util::xmsg::XProfileSalinity::DecodeXMsgData(), util::xmsg::XProfileTemperature::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgD
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the call graph for this function:



10.73.3.2 DecodeXMsgData()

```
void util::xmsg::XEngAscent::DecodeXMsgData (
            const vector< uint8_t > & data)  [inline], [virtual]
```

Decode the ascent state engineering X-Message from the binary data.

Parameters

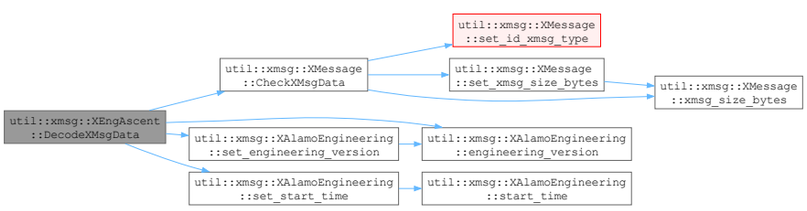
in	*data*	The binary message to decode.

Implements util::xmsg::XMessage.

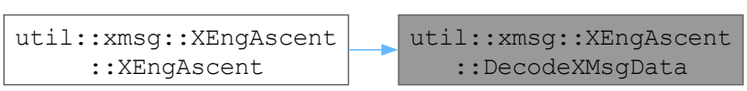
References CATCH_BLOCK, util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XAlamoEngineering::engineering_version(),
EXC_THROW, s2000, util::xmsg::XAlamoEngineering::set_engineering_version(), util::xmsg::XAlamoEngineering::set_start_time(),
and u16.

Referenced by XEngAscent().

Here is the call graph for this function:



Here is the caller graph for this function:



10.73.3.3 engineering_version()

```
uint8_t util::xmsg::XAlamoEngineering::engineering_version () const  [inline], [inherited]
```

Get the engineering message version identifier.

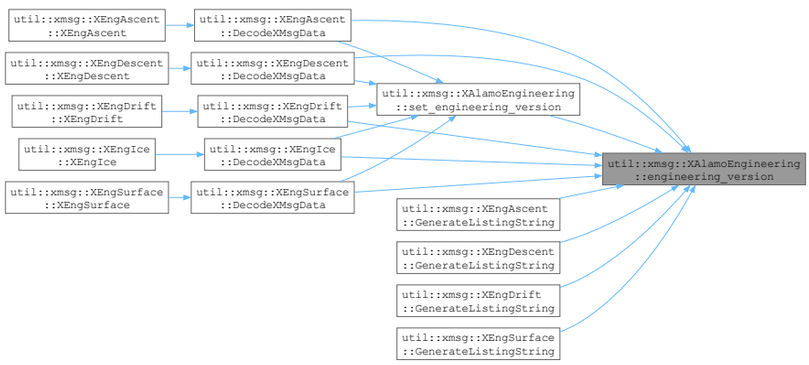
Returns

The engineering message version identifier.

References util::xmsg::XAlamoEngineering::engineering_version_.

Referenced by DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(), util::xmsg::XEngDrift::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), util::xmsg::XEngSurface::GenerateListin and util::xmsg::XAlamoEngineering::set_engineering_version().

Here is the caller graph for this function:



10.73.3.4 engineering_version_as_float()

```
float util::xmsg::XAlamoEngineering::engineering_version_as_float () const  [inline], [inherited]
```

Get the engineering message verion identifier as a floating point number.

Returns

The engineering message verion identifier as a floating point number.

References util::xmsg::XAlamoEngineering::engineering_version_.

10.73.3.5 GenerateCurrentUtcTimeIso8601String()

```
static string util::date::DateUtils::GenerateCurrentUtcTimeIso8601String ()  [inline], [static],
[inherited]
```

Generate an ISO8601 format date/time string representation of the current time, in UTC.

Note

All date/time objects use the UTC time-zone.

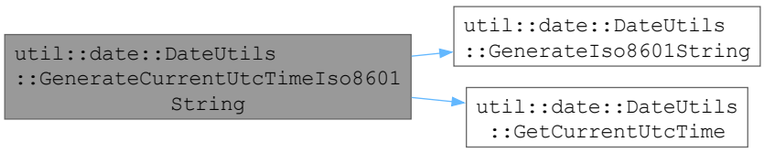
Returns

An ISO8601 format date/time string representation of the current time.

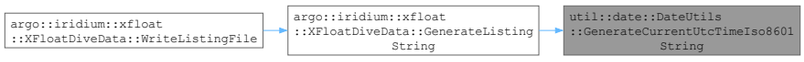
References util::date::DateUtils::GenerateIso8601String(), and util::date::DateUtils::GetCurrentUtcTime().

Referenced by argo::iridium::xfloat::XFloatDiveData::GenerateListingString().

Here is the call graph for this function:



Here is the caller graph for this function:



10.73.3.6 GenerateIso8601String()

```
static string util::date::DateUtils::GenerateIso8601String (
            const ptime & p,
            bool extended = true)  [inline], [static], [inherited]
```

Generate an ISO8601 format date/time string representation of a POSIX time object.

Note

All date/time objects use the UTC time-zone.

Parameters

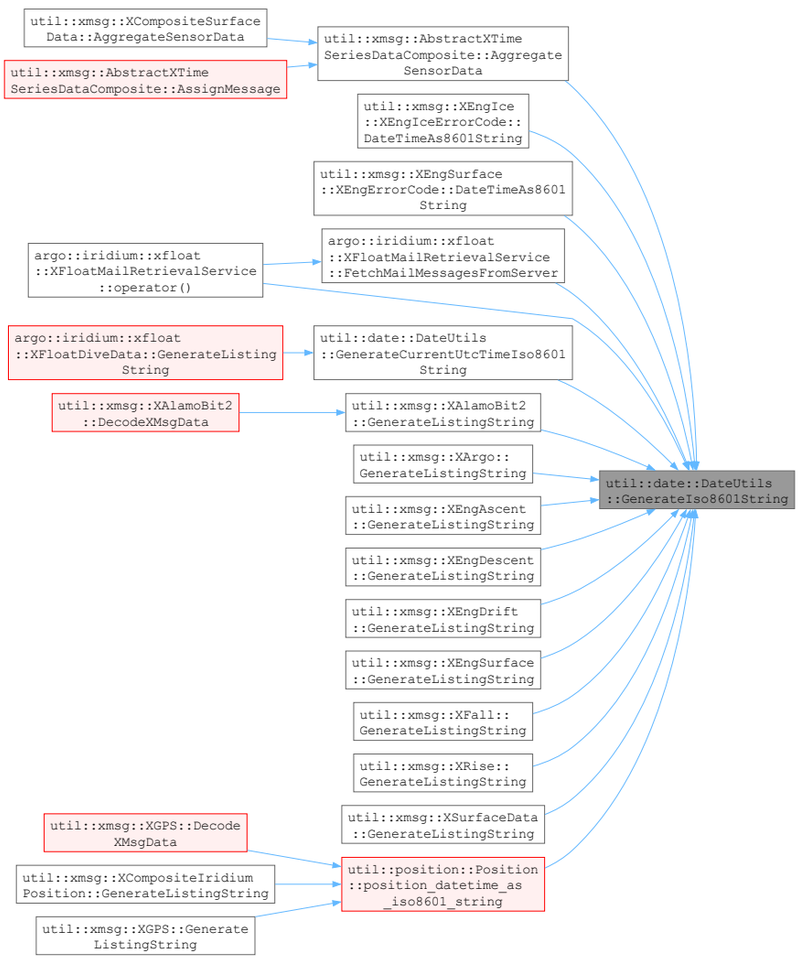
in	*p*	POSIX time from which to generate string.
in	*extended*	If TRUE, use the ISO 8601 extended format (YYYY-MM-DDThh:mm:ssZ); otherwise, use the basic format (YYYYMMDDThhmmss).

Returns

An ISO8601 format date/time string representation of the POSIX time object. If p is equal to boost::posix_↩ time::not_a_date_time, then the string "0000-00-00T00:00:00Z" (extended) or "00000000000000" (basic) will be returned.

Referenced by util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::AggregateSensorData(), util::xmsg::XEngIce::XEngIce
util::xmsg::XEngSurface::XEngErrorCode::DateTimeAs8601String(), argo::iridium::xfloat::XFloatMailRetrievalService::FetchMailMessages
util::date::DateUtils::GenerateCurrentUtcTimeIso8601String(), util::xmsg::XAlamoBit2::GenerateListingString(), util::xmsg::XArgo::Genera
GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(),
util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XRise::GenerateListingString(),
util::xmsg::XSurfaceData::GenerateListingString(), argo::iridium::xfloat::XFloatMailRetrievalService::operator()(), and
util::position::Position::position_datetime_as_iso8601_string().

Here is the caller graph for this function:



10.73.3.7 GenerateJulianDay()

```
static double util::date::DateUtils::GenerateJulianDay (
            const ptime & dt,
            const ptime & ref_dt)  [inline], [static], [inherited]
```

Convert a date/time to fractional days since a reference date/time.

Parameters

in	*dt*	The date/time to convert.
in	*ref↩ _dt*	The reference date/time (day 0).

Exceptions

util::log::MessageException	if either date/time is not valid.

Returns

The number of days (with millisecond resolution) from `ref_dt` to `dt`.

References CATCH_BLOCK, and EXC_THROW.

10.73.3.8 GenerateListingString()

```
const string util::xmsg::XEngAscent::GenerateListingString (
            uint16_t pg,
            uint16_t po,
            uint16_t sg,
            uint16_t so,
            uint16_t tg,
            uint16_t to) const  [inline]
```

Get the listing entry string for this ascent engineering data X-Message.

Parameters

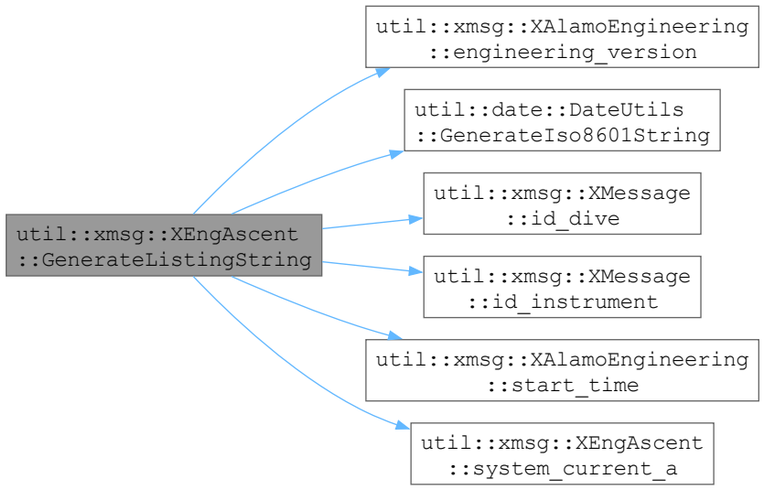
in	*pg*	The pressure sensor sensitivity scale factor, in LSB/dBar.
in	*po*	The pressure sensor offset, in dBar.
in	*sg*	The salinity sensor sensitivity scale factor, in LSB/PSU.
in	*so*	The salinity sensor offset, in PSU.
in	*tg*	The temperature sensor sensitivity scale factor, in LSB/degC.
in	*to*	The temperature sensor offset, in degC.

Returns

The listing entry string for this ascent engineering data X-Message

References util::xmsg::XAlamoEngineering::engineering_version(), EXC_THROW, util::date::DateUtils::GenerateIso8601String(), util::xmsg::XMessage::id_dive(), util::xmsg::XMessage::id_instrument(), util::xmsg::XAlamoEngineering::start_time(), and system_current_a().

Here is the call graph for this function:



10.73.3.9 GetCurrentUtcTime()

```
static ptime util::date::DateUtils::GetCurrentUtcTime ()  [inline], [static], [inherited]
```

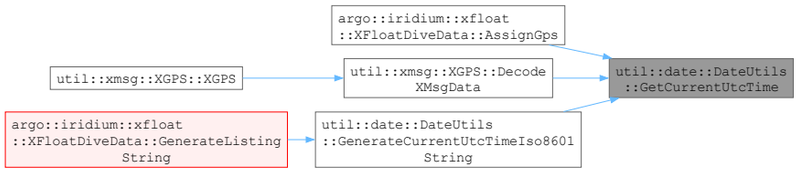
Generate a POSIX time object representing the current date and time (rounded to the nearest second).

Returns

A POSIX time object representing the current date and time (rounded to the nearest second).

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignGps(), util::xmsg::XGPS::DecodeXMsgData(), and util::date::DateUtils::GenerateCurrentUtcTimeIso8601String().

Here is the caller graph for this function:



10.73.3.10 id_dive()

```
int16_t util::xmsg::XMessage::id_dive () const  [inline], [inherited]
```

Get the dive or cycle number of this X-Message.

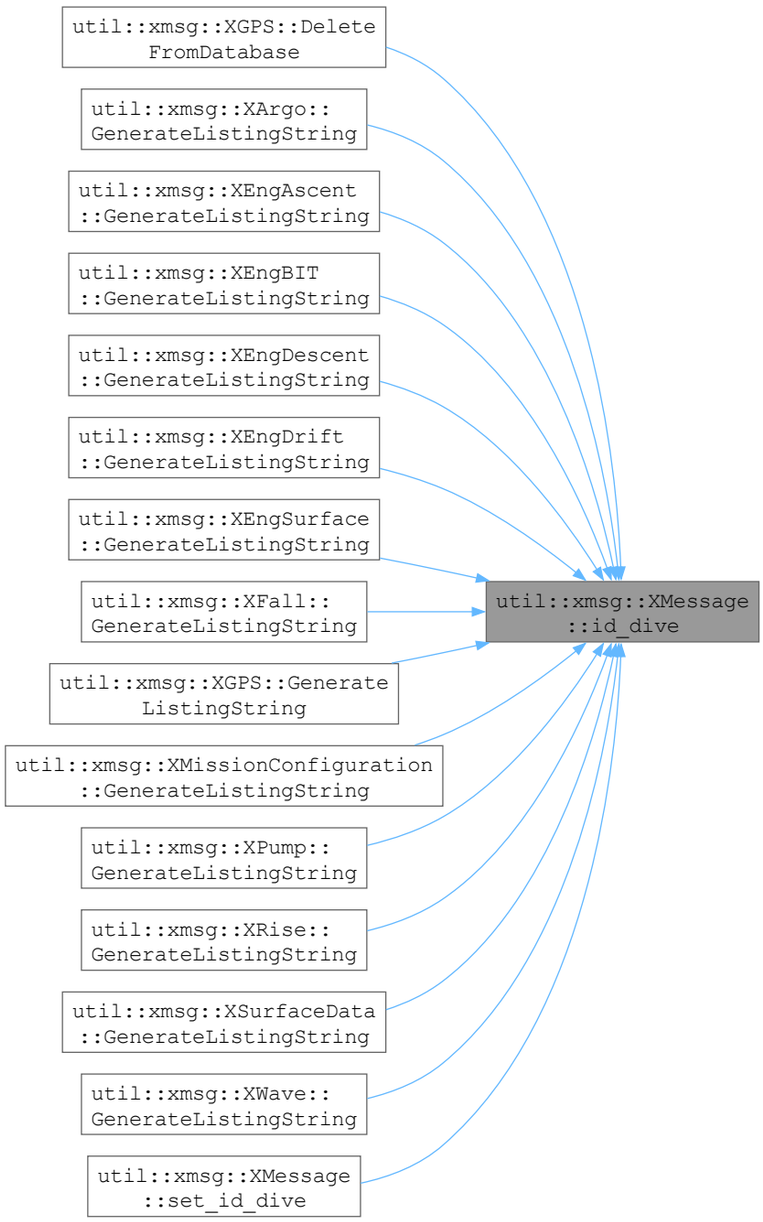
Returns

The dive or cycle number of this X-Message.

References util::xmsg::XMessage::id_dive_.

Referenced by util::xmsg::XGPS::DeleteFromDatabase(), util::xmsg::XArgo::GenerateListingString(), GenerateListingString(), util::xmsg::XEngBIT::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingStr util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing util::xmsg::XSurfaceData::GenerateListingString(), util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_dive().

Here is the caller graph for this function:



10.73.3.11 id_instrument()

```
uint16_t util::xmsg::XMessage::id_instrument () const  [inline], [inherited]
```

Get the unique instrument identifier of this X-Message.

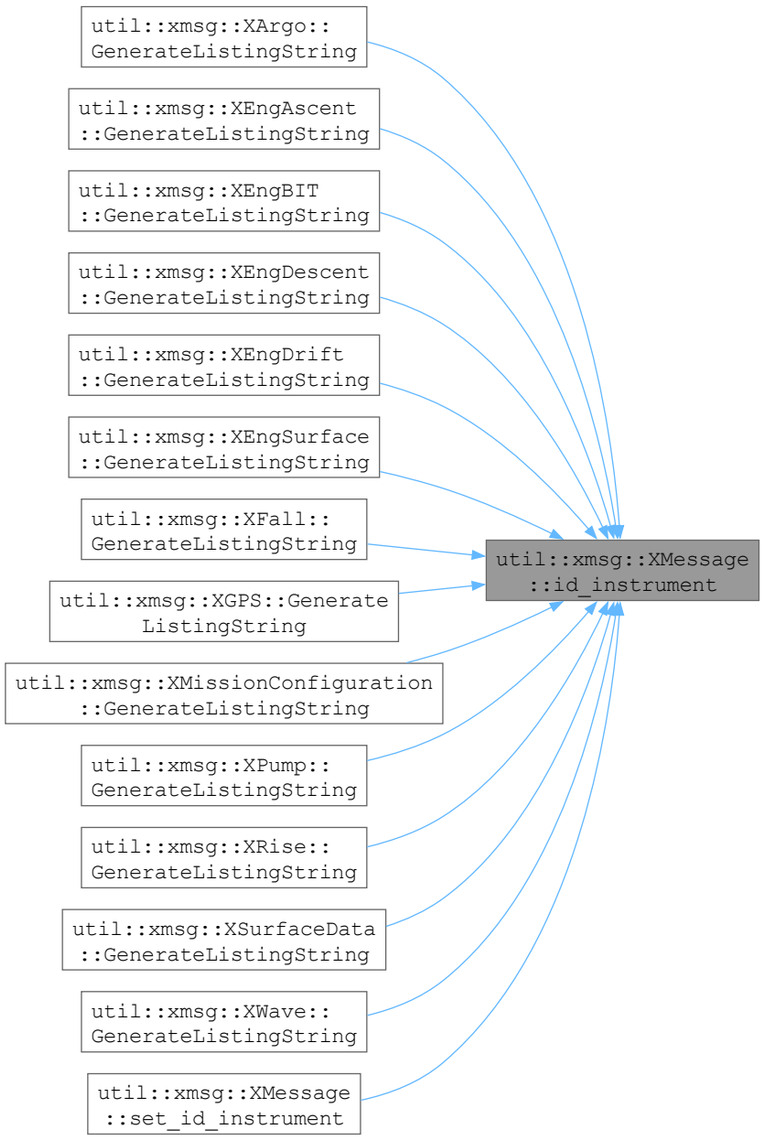
Returns

The unique instrument identifier of this X-Message.

References util::xmsg::XMessage::id_instrument_.

Referenced by util::xmsg::XArgo::GenerateListingString(), GenerateListingString(), util::xmsg::XEngBIT::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), util::xmsg::XEngSurface::GenerateListin util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingS util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(), util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_instrument().

Here is the caller graph for this function:



10.73.3.12 id_xmsg_type()

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type () const  [inline], [inherited]
```

Get the enumerated X-Message type identifier.

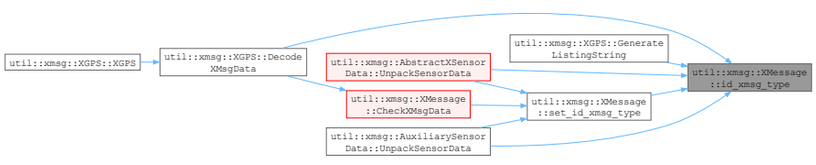
Returns

The enumerated X-Message type identifier.

References util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMessage::set_id_xmsg_typ
util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the caller graph for this function:



10.73.3.13 set_engineering_version()

```
void util::xmsg::XAlamoEngineering::set_engineering_version (
            uint8_t engineering_version)  [inline], [protected], [inherited]
```

Set the engineering message version identifier.

Parameters

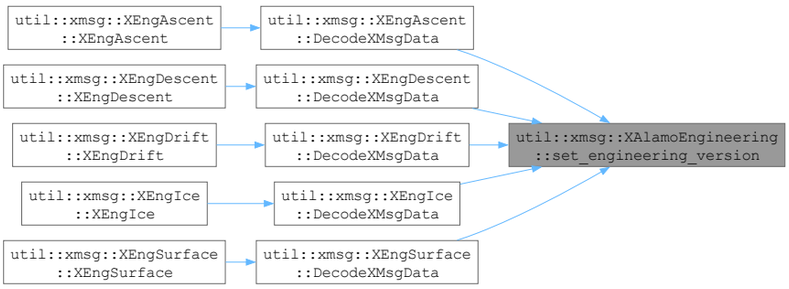
in	*engineering_version*	The engineering message version identifier.

References util::xmsg::XAlamoEngineering::engineering_version(), and util::xmsg::XAlamoEngineering::engineering_version_.

Referenced by DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(), util::xmsg::XEngDrift::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(), and util::xmsg::XEngSurface::DecodeXMsgData().

Here is the call graph for this function:

Here is the caller graph for this function:



10.73.3.14 set_id_dive()

```
void util::xmsg::XMessage::set_id_dive (
            int16_t id_dive)  [inline], [protected], [inherited]
```

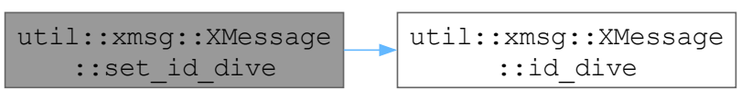
Set the dive or cycle number of this message.

Parameters

in	*id_dive*	The dive or cycle number of this message to set.

References util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::id_dive_.

Here is the call graph for this function:



10.73.3.15 set_id_instrument()

```
void util::xmsg::XMessage::set_id_instrument (
            uint16_t id_instrument)  [inline], [protected], [inherited]
```

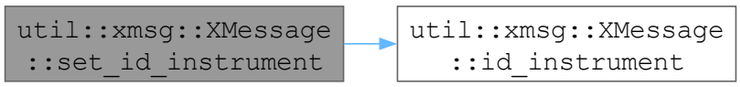
Set the unique instrument identifier.

Parameters

in	*id_instrument*	The unique instrument identifier to set.

References util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::id_instrument_.

Here is the call graph for this function:



10.73.3.16 set_id_xmsg_type()

```
void util::xmsg::XMessage::set_id_xmsg_type (
            XMsgTypeId_t id_xmsg_type)  [inline], [protected], [inherited]
```

Set the X-Message type identifier.

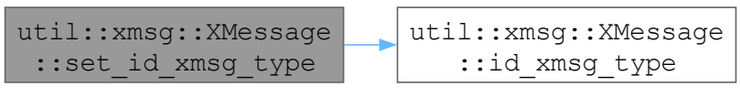
Parameters

in	*id_xmsg_type*	The X-Message type identifier to set.

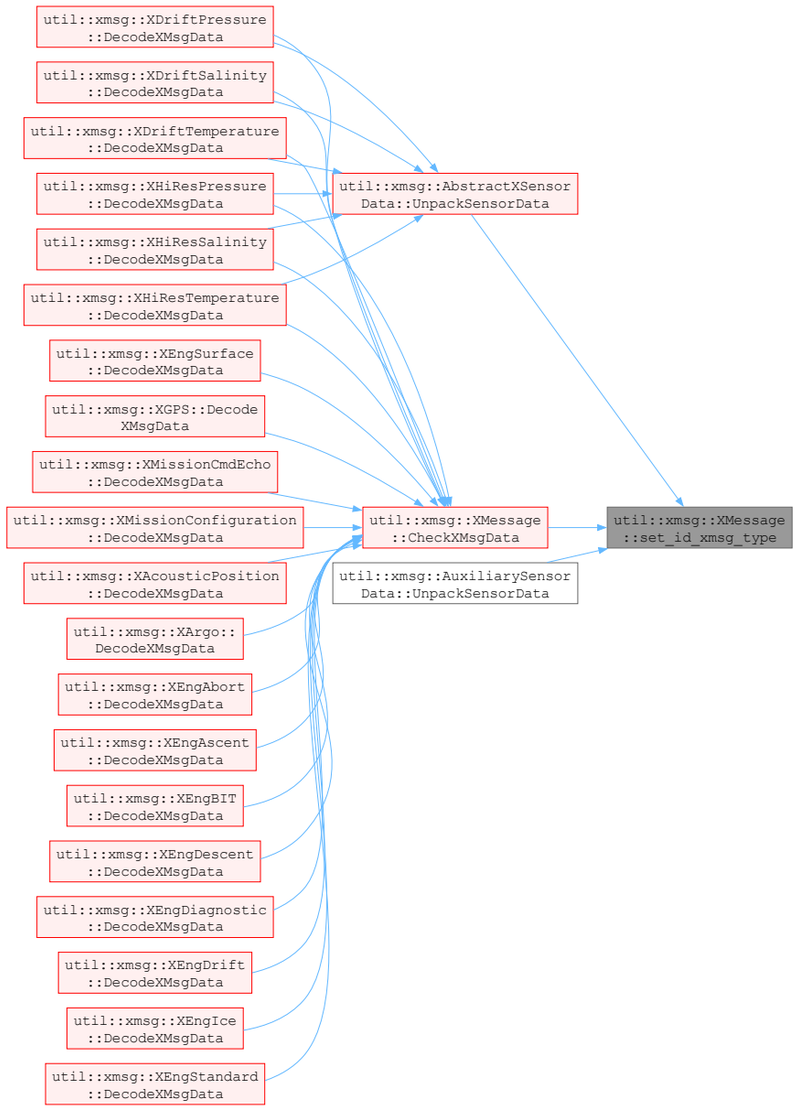
References util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.73.3.17 set_start_time()

```
void util::xmsg::XAlamoEngineering::set_start_time (
            const ptime & start_time)  [inline], [protected], [inherited]
```

Set the engineering message state start time.

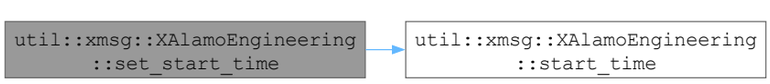
Parameters

in	*start_time*	The engineering message state start time.

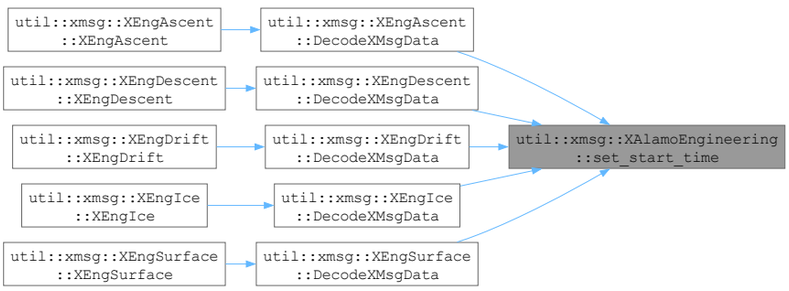
References util::xmsg::XAlamoEngineering::start_time(), and util::xmsg::XAlamoEngineering::start_time_.

Referenced by DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(), util::xmsg::XEngDrift::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(), and util::xmsg::XEngSurface::DecodeXMsgData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.73.3.18 set_xmsg_size_bytes()

```
void util::xmsg::XMessage::set_xmsg_size_bytes (
            uint16_t xmsg_size_bytes)  [inline], [protected], [inherited]
```

Set the size of the X-Message, in bytes.

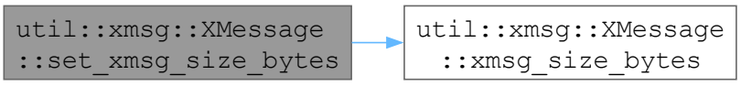
Parameters

in	*xmsg_size_bytes*	The size of the X-Message to set, in bytes.

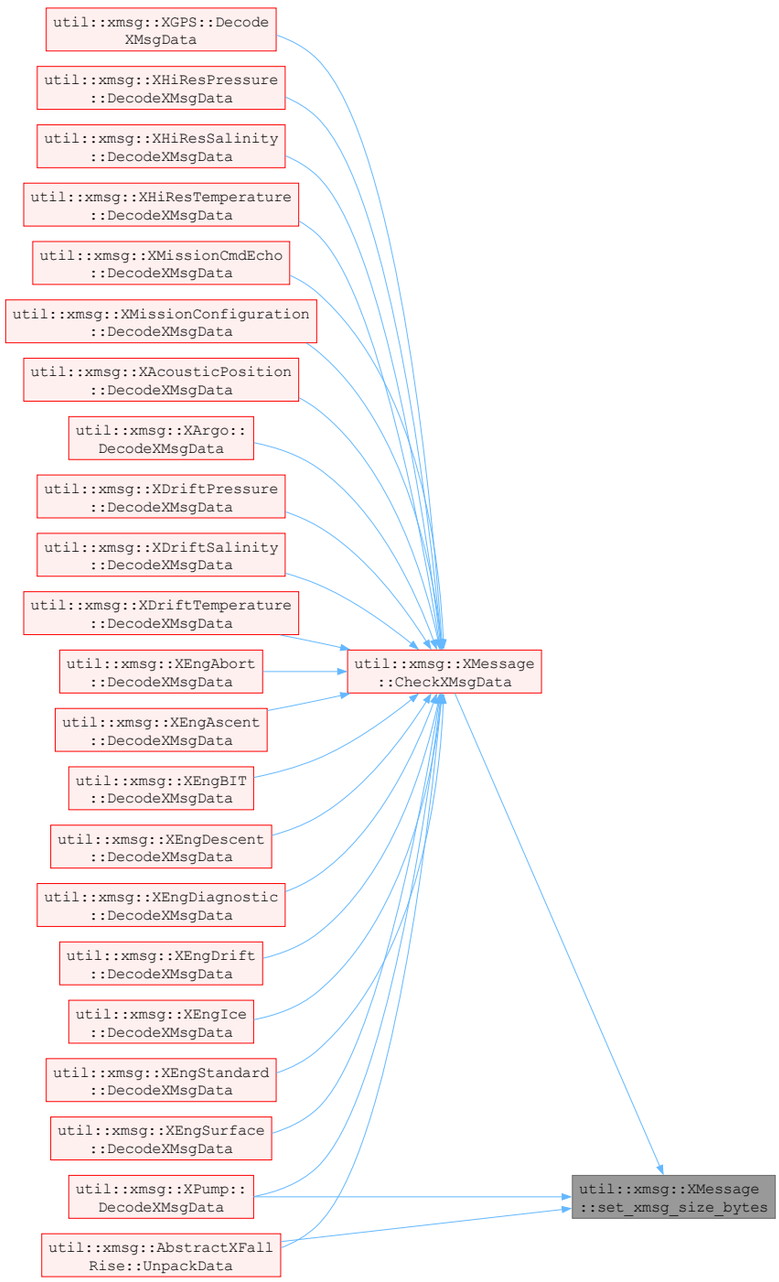
References util::xmsg::XMessage::xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XPump::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::Unpa

Here is the call graph for this function:



Here is the caller graph for this function:



10.73.3.19 start_time()

```
const ptime & util::xmsg::XAlamoEngineering::start_time () const  [inline], [inherited]
```

Get the engineering message state start time.

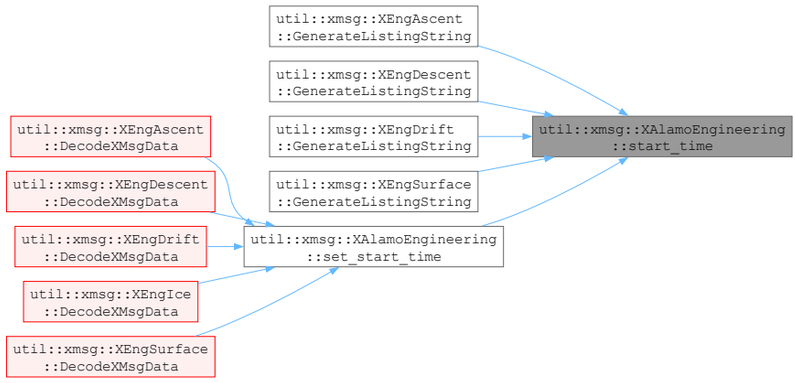
Returns

The engineering message state start time.

References util::xmsg::XAlamoEngineering::start_time_.

Referenced by GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), util::xmsg::XEngSurface::GenerateListingString(), and util::xmsg::XAlamoEngineering::set_start_time().

Here is the caller graph for this function:



10.73.3.20 system_current_a()

```
template<typename T = float>
T util::xmsg::XEngAscent::system_current_a () const  [inline]
```

Get the system current, in amps.

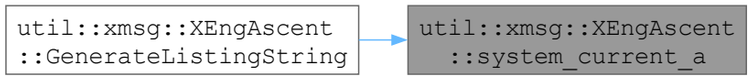
Returns

The system current, in amps.

References system_current_ma_.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.73.3.21 xmsg_size_bytes()

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes () const  [inline], [inherited]
```

Get the size of the X-Message, in bytes.

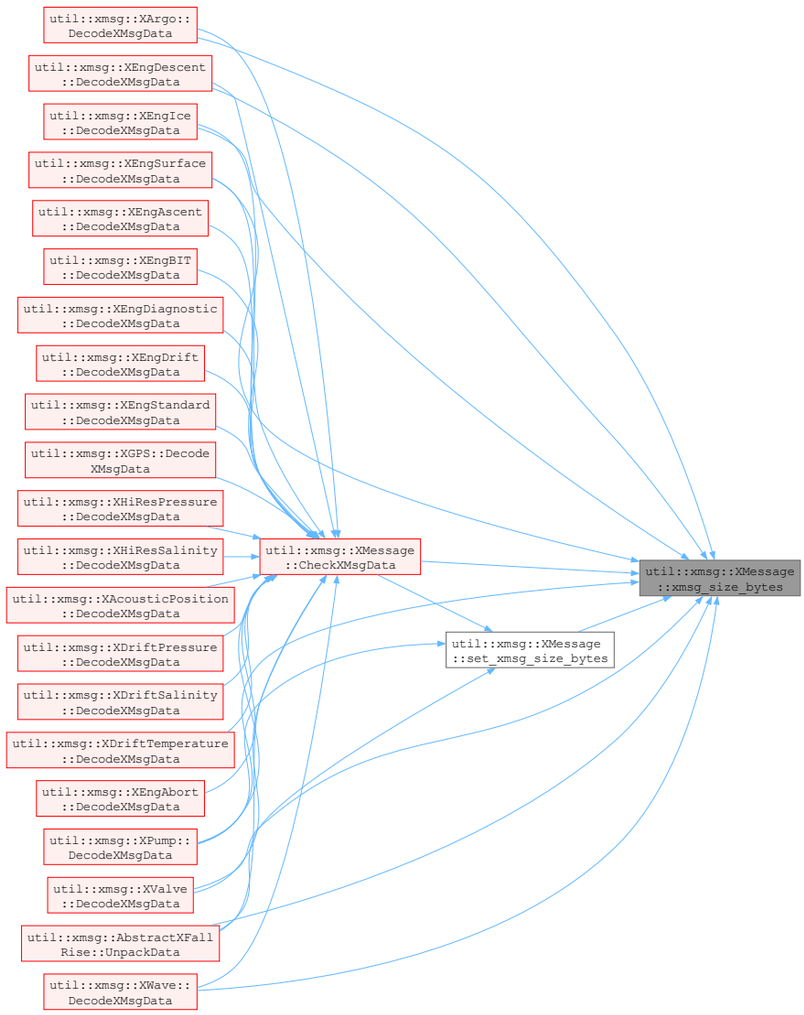
Returns

The size of the X-Message, in bytes.

References util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgI
util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(),
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), util::xmsg::XMessage::set_xmsg_size_bytes(),
and util::xmsg::AbstractXFallRise::UnpackData().

Here is the caller graph for this function:



10.73.4 Member Data Documentation

10.73.4.1 case_vac_at_end_cinhg_

```
uint16_t util::xmsg::XEngAscent::case_vac_at_end_cinhg_  [private]
```

The measured internal pres case vacuum, in 0.01 inHg, at the end of the ascent/profile phase of the dive cycle.

Note

This field was added in version 0x11.

10.73.4.2 case_vac_at_start_cinhg_

```
uint16_t util::xmsg::XEngAscent::case_vac_at_start_cinhg_  [private]
```

The measured internal pres case vacuum, in 0.01 inHg, at the start of the ascent/profile phase of the dive cycle.

Note

This field was added in version 0x11.

10.73.4.3 engineering_version_

```
uint8_t util::xmsg::XAlamoEngineering::engineering_version_  [private], [inherited]
```

The engineering message version identifier.

Referenced by util::xmsg::XAlamoEngineering::engineering_version(), util::xmsg::XAlamoEngineering::engineering_version_as_float(), and util::xmsg::XAlamoEngineering::set_engineering_version().

10.73.4.4 id_dive_

```
int16_t util::xmsg::XMessage::id_dive_  [private], [inherited]
```

The dive or cycle number of this message.

Referenced by util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::set_id_dive().

10.73.4.5 id_instrument_

```
uint16_t util::xmsg::XMessage::id_instrument_  [private], [inherited]
```

The unique WHOI identifier of this instrument.

Referenced by util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::set_id_instrument().

10.73.4.6 id_xmsg_type_

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type_  [private], [inherited]
```

The X-message type identifier.

Referenced by util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::set_id_xmsg_type().

10.73.4.7 pump_run_duration_

```
uint16_t util::xmsg::XEngAscent::pump_run_duration_  [private]
```

The cumulative pump run time during the ascent phase of the dive cycle, in seconds.

Note

This field is only used for message version 0x10. It was removed in version 0x11.

10.73.4.8 start_time_

```
ptime util::xmsg::XAlamoEngineering::start_time_  [private], [inherited]
```

The engineering message state start time.

Referenced by util::xmsg::XAlamoEngineering::set_start_time(), and util::xmsg::XAlamoEngineering::start_time().

10.73.4.9 system_current_ma_

```
uint16_t util::xmsg::XEngAscent::system_current_ma_  [private]
```

The measured system current, in mA.

Note

This field is only used for message version 0x10. It was removed in version 0x11.

Referenced by system_current_a().

10.73.4.10 xmsg_size_bytes_

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes_  [private], [inherited]
```

The size of the X message, in bytes.

Referenced by util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes().

The documentation for this class was generated from the following file:

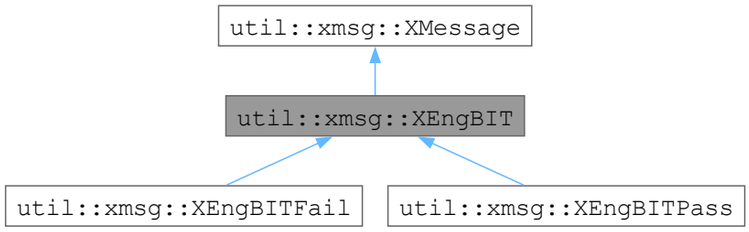
- data/xmsg/XEngAscent.h

10.74 util::xmsg::XEngBIT Class Reference

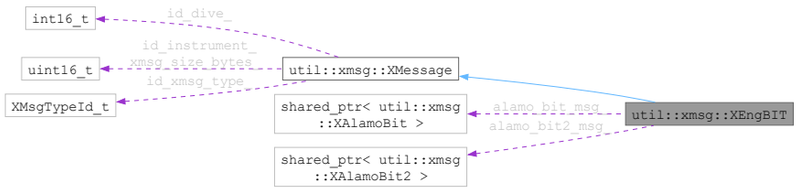
The ALAMO Built-in-Test (BIT) engineering message.

```
#include <XEngBIT.h>
```

Inheritance diagram for util::xmsg::XEngBIT:



Collaboration diagram for util::xmsg::XEngBIT:



Public Member Functions

- XEngBIT (int16_t dive, uint16_t whoi, const vector< uint8_t > data)

  Parameterized constructor.
- const uint8_t * CheckXMsgData (const vector< uint8_t > &data, int id, int minBytes, bool parseSize=true)

  Check the X-message for the correct ID, minimum size, and message terminator.
- void DecodeXMsgData (const vector< uint8_t > &data)

  Decode the BIT engineering X-Message from the binary data.
- const string GenerateListingString (uint16_t pg, uint16_t po, uint16_t sg, uint16_t so, uint16_t tg, uint16_t to) const

  Return a listing entry string representation of the BIT engineering X-Message.
- int16_t id_dive () const

  Get the dive or cycle number of this X-Message.
- uint16_t id_instrument () const

  Get the unique instrument identifier of this X-Message.
- XMsgTypeId_t id_xmsg_type () const

  Get the enumerated X-Message type identifier.
- uint16_t xmsg_size_bytes () const

  Get the size of the X-Message, in bytes.

Protected Member Functions

- void set_alamo_bit2_msg (const shared_ptr< XAlamoBit2 > &alamo_bit2_msg)
  Set the ALAMO Built-in-Test (BIT) v2 message.
- void set_alamo_bit_msg (const shared_ptr< XAlamoBit > &alamo_bit_msg)
  Set the ALALMO Built-in-Test (BIT) v1 message.
- void set_id_dive (int16_t id_dive)
  Set the dive or cycle number of this message.
- void set_id_instrument (uint16_t id_instrument)
  Set the unique instrument identifier.
- void set_id_xmsg_type (XMsgTypeId_t id_xmsg_type)
  Set the X-Message type identifier.
- void set_xmsg_size_bytes (uint16_t xmsg_size_bytes)
  Set the size of the X-Message, in bytes.

Private Attributes

- shared_ptr< XAlamoBit2 > alamo_bit2_msg_
  A shared pointer to the ALALMO Built-in-Test (BIT) message.
- shared_ptr< XAlamoBit > alamo_bit_msg_
  A shared pointer to the ALALMO Built-in-Test (BIT) message.
- int16_t id_dive_
  The dive or cycle number of this message.
- uint16_t id_instrument_
  The unique WHOI identifier of this instrument.
- XMsgTypeId_t id_xmsg_type_
  The X-message type identifier.
- uint16_t xmsg_size_bytes_
  The size of the X message, in bytes.

10.74.1 Detailed Description

The ALAMO Built-in-Test (BIT) engineering message.

An instance of this class represents a BIT engineering X-message. Depending on the BIT format in byte 3, the message is decoded as an XAlamoBit (formats 6-8, firmware before 10.x) or an XAlamoBit2 (formats 9-11, firmware 10.x and later).

10.74.2 Constructor & Destructor Documentation

10.74.2.1 XEngBIT()

```
util::xmsg::XEngBIT::XEngBIT (
            int16_t dive,
            uint16_t whoi,
            const vector< uint8_t > data)  [inline]
```

Parameterized constructor.

Parameters

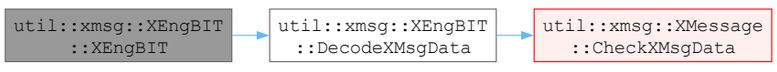
in	*dive*	The dive or cycle number of this message.
in	*whoi*	The unique WHOI identifier.
in	*data*	The binary X-message data to decode.

Exceptions

util::log::MessageException	if an decode error occurs.

References DecodeXMsgData().

Here is the call graph for this function:



10.74.3 Member Function Documentation

10.74.3.1 CheckXMsgData()

```
const uint8_t * util::xmsg::XMessage::CheckXMsgData (
            const vector< uint8_t > & data,
            int id,
            int minBytes,
            bool parseSize = true)  [inline], [inherited]
```

Check the X-message for the correct ID, minimum size, and message terminator.

Parameters

in	*data*	The X-message binary data to check.
in	*id*	The expected ID of the X-message. If this field is less that 0, then the ID check will be skipped.
in	*minBytes*	The minimum number of bytes expected in a valid message.
in	*parseSize*	If TRUE, then the message size field will be parsed from the X-Message. Otherwise, the size field will not be parsed.

Exceptions

util::log::MessageException	if • the message does not contain the expected ID, • the message does not contain the minimum number of expected bytes, • the message does not contain a terminator (';') in the correct location, or, • any other error occurs.

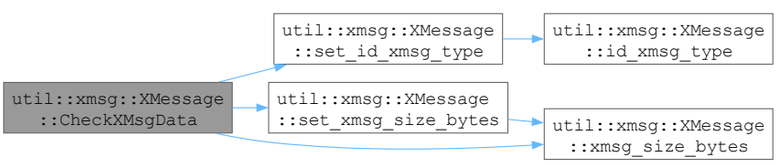
Returns

A pointer to the underlying data buffer of data.

References CATCH_BLOCK, EXC_THROW, lwrn, util::xmsg::XMessage::set_id_xmsg_type(), util::xmsg::XMessage::set_xmsg_size_byte
u16, and util::xmsg::XMessage::xmsg_size_bytes().

Referenced by util::xmsg::XAcousticPosition::DecodeXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XDriftPressure::Dec
util::xmsg::XDriftSalinity::DecodeXMsgData(), util::xmsg::XDriftTemperature::DecodeXMsgData(), util::xmsg::XEngAbort::DecodeXMsgDa
util::xmsg::XEngAscent::DecodeXMsgData(), DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(),
util::xmsg::XEngDiagnostic::DecodeXMsgData(), util::xmsg::XEngDrift::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(),
util::xmsg::XEngStandard::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XGPS::DecodeXMsgData(),
util::xmsg::XHiResPressure::DecodeXMsgData(), util::xmsg::XHiResSalinity::DecodeXMsgData(), util::xmsg::XHiResTemperature::Decod
util::xmsg::XMissionCmdEcho::DecodeXMsgData(), util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XProfilePressure:
util::xmsg::XProfileSalinity::DecodeXMsgData(), util::xmsg::XProfileTemperature::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgD
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the call graph for this function:



10.74.3.2 DecodeXMsgData()

```
void util::xmsg::XEngBIT::DecodeXMsgData (
            const vector< uint8_t > & data)  [inline], [virtual]
```

Decode the BIT engineering X-Message from the binary data.

Parameters

in	*data*	The binary message to decode.

Exceptions

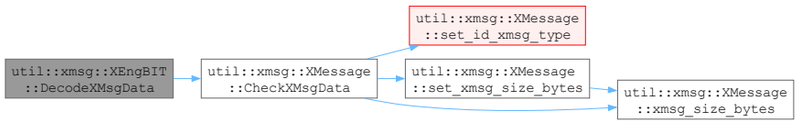
util::log::MessageException	if an error occurs while decoding.

Implements util::xmsg::XMessage.

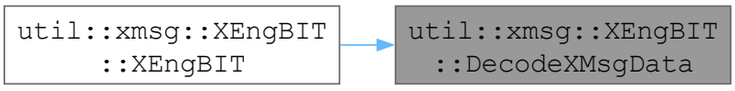
References alamo_bit2_msg_, alamo_bit_msg_, CATCH_BLOCK, util::xmsg::XMessage::CheckXMsgData(), and EXC_THROW.

Referenced by XEngBIT().

Here is the call graph for this function:



Here is the caller graph for this function:



10.74.3.3 GenerateListingString()

```
const string util::xmsg::XEngBIT::GenerateListingString (
            uint16_t pg,
            uint16_t po,
            uint16_t sg,
            uint16_t so,
            uint16_t tg,
            uint16_t to) const  [inline]
```

Return a listing entry string representation of the BIT engineering X-Message.

Parameters

in	*pg*	The pressure sensor sensitivity (gain) scale factor, in LSB/dBar.
in	*po*	The pressure sensor offset, in dBar.
in	*sg*	The salinity sensor sensitivity (gain) scale factor, in LSB/PSU.
in	*so*	The salinity sensor offset, in PSU.
in	*tg*	The temperature sensor sensitivity (gain) scale factor, in LSB/degC.
in	*to*	The temperature sensor offset, in degC.

Exceptions

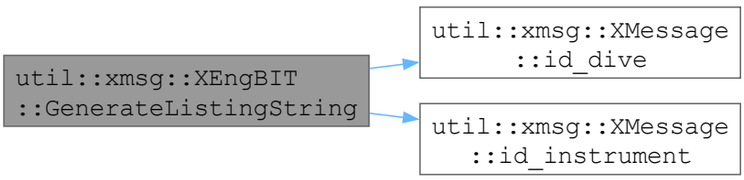
util::log::MessageException	if an error occurs while generating the listing entry string representation.

Returns

A listing entry string representation of the BIT engineering X-Message.

References alamo_bit2_msg_, alamo_bit_msg_, CATCH_BLOCK, util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::id_instrume

Here is the call graph for this function:



10.74.3.4 id_dive()

```
int16_t util::xmsg::XMessage::id_dive () const  [inline], [inherited]
```

Get the dive or cycle number of this X-Message.

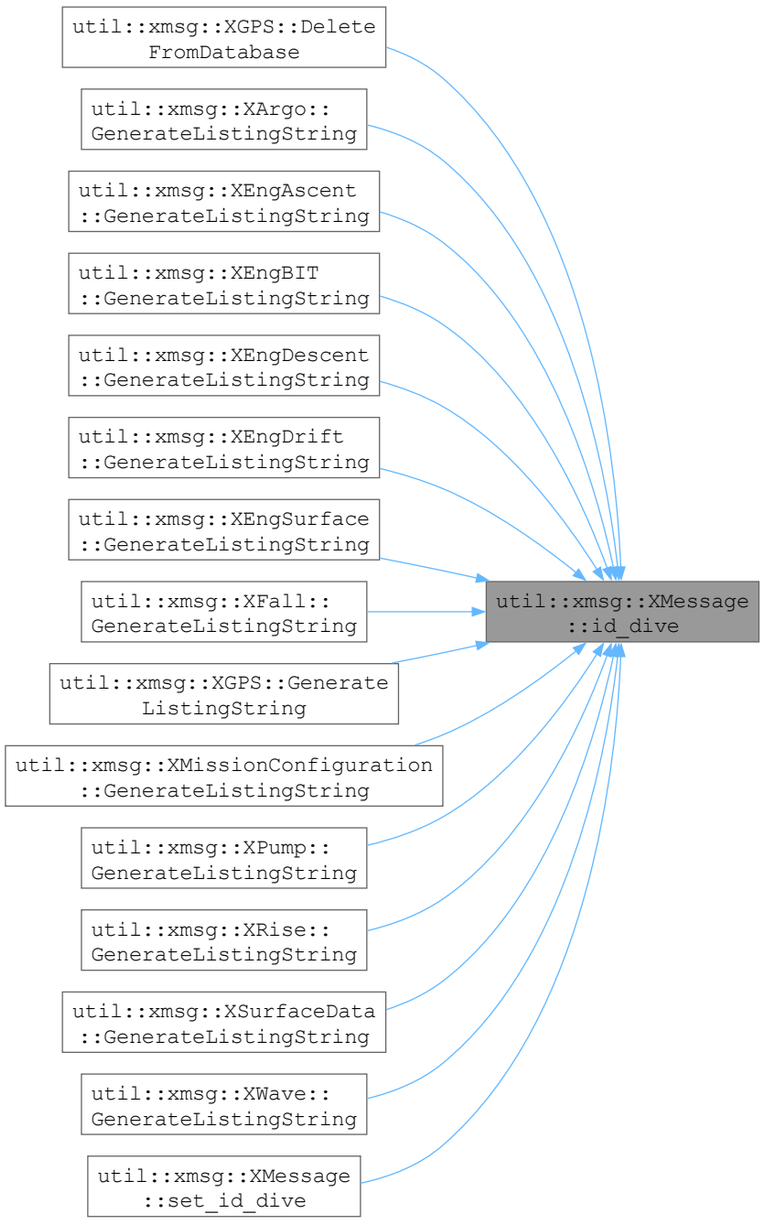
Returns

The dive or cycle number of this X-Message.

References util::xmsg::XMessage::id_dive_.

Referenced by util::xmsg::XGPS::DeleteFromDatabase(), util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateLis
GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(),
util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(),
util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing
util::xmsg::XSurfaceData::GenerateListingString(), util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_dive().

Here is the caller graph for this function:



10.74.3.5 id_instrument()

```
uint16_t util::xmsg::XMessage::id_instrument () const  [inline], [inherited]
```

Get the unique instrument identifier of this X-Message.

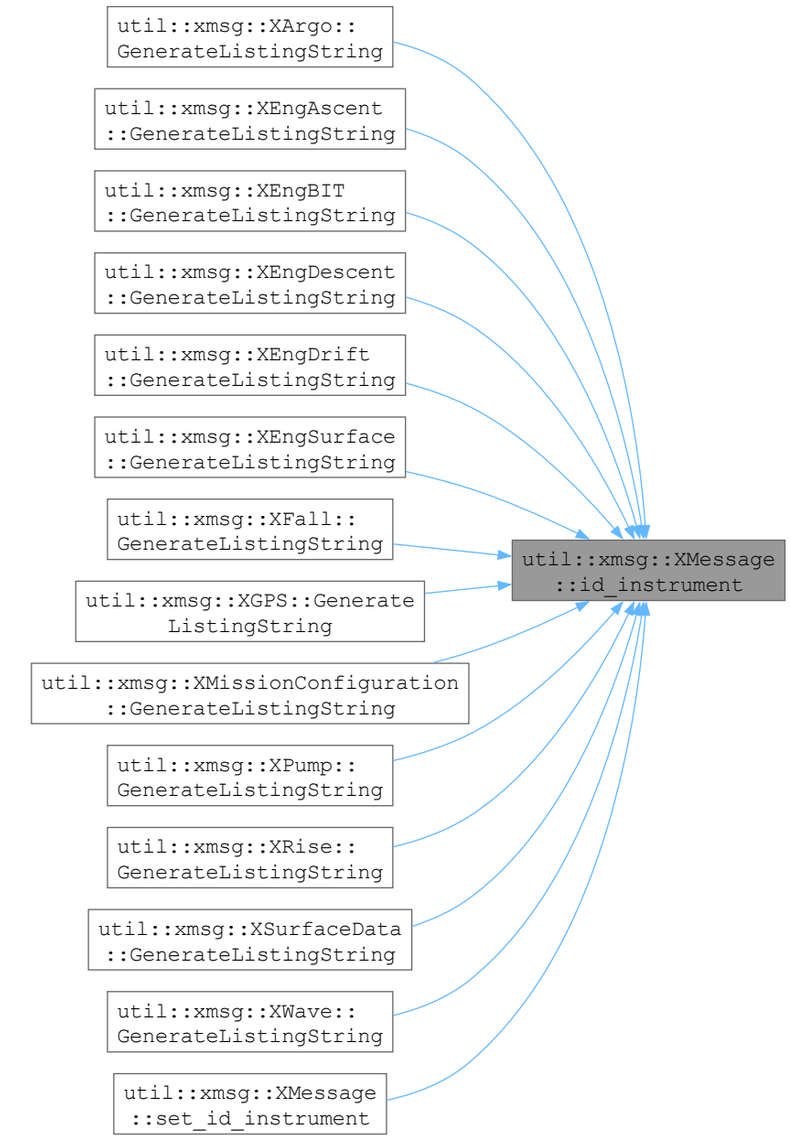
Returns

The unique instrument identifier of this X-Message.

References util::xmsg::XMessage::id_instrument_.

Referenced by util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateListingString(), GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), util::xmsg::XEngSurface::GenerateListin util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingS util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(), util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_instrument().

Here is the caller graph for this function:



10.74.3.6 id_xmsg_type()

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type () const  [inline], [inherited]
```

Get the enumerated X-Message type identifier.

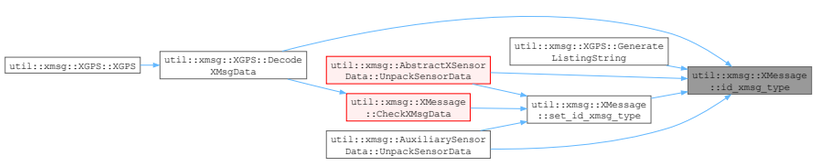
Returns

The enumerated X-Message type identifier.

References util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMessage::set_id_xmsg_typ
util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the caller graph for this function:



10.74.3.7 set_alamo_bit2_msg()

```
void util::xmsg::XEngBIT::set_alamo_bit2_msg (
            const shared_ptr< XAlamoBit2 > & alamo_bit2_msg)  [inline], [protected]
```

Set the ALAMO Built-in-Test (BIT) v2 message.

Parameters

in	*alamo_bit2_msg*	A shared pointer to the ALAMO Built-in-Test (BIT) v2 message to set.

References alamo_bit2_msg_.

10.74.3.8 set_alamo_bit_msg()

```
void util::xmsg::XEngBIT::set_alamo_bit_msg (
            const shared_ptr< XAlamoBit > & alamo_bit_msg)  [inline], [protected]
```

Set the ALALMO Built-in-Test (BIT) v1 message.

Parameters

in	*alamo_bit_msg*	A shared pointer to the ALALMO Built-in-Test (BIT) message to set.

References alamo_bit_msg_.

10.74.3.9 set_id_dive()

```
void util::xmsg::XMessage::set_id_dive (
            int16_t id_dive)  [inline], [protected], [inherited]
```

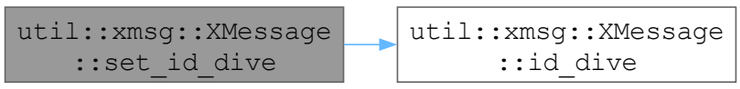
Set the dive or cycle number of this message.

Parameters

in	*id_dive*	The dive or cycle number of this message to set.

References util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::id_dive_.

Here is the call graph for this function:



10.74.3.10 set_id_instrument()

```
void util::xmsg::XMessage::set_id_instrument (
            uint16_t id_instrument)  [inline], [protected], [inherited]
```

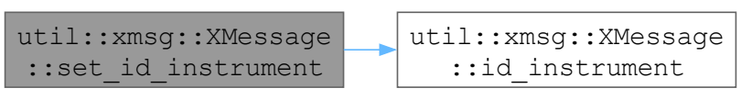
Set the unique instrument identifier.

Parameters

in	*id_instrument*	The unique instrument identifier to set.

References util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::id_instrument_.

Here is the call graph for this function:



10.74.3.11 set_id_xmsg_type()

```
void util::xmsg::XMessage::set_id_xmsg_type (
            XMsgTypeId_t id_xmsg_type)  [inline], [protected], [inherited]
```

Set the X-Message type identifier.

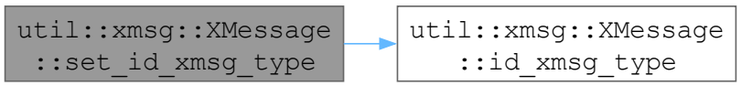
Parameters

in	*id_xmsg_type*	The X-Message type identifier to set.

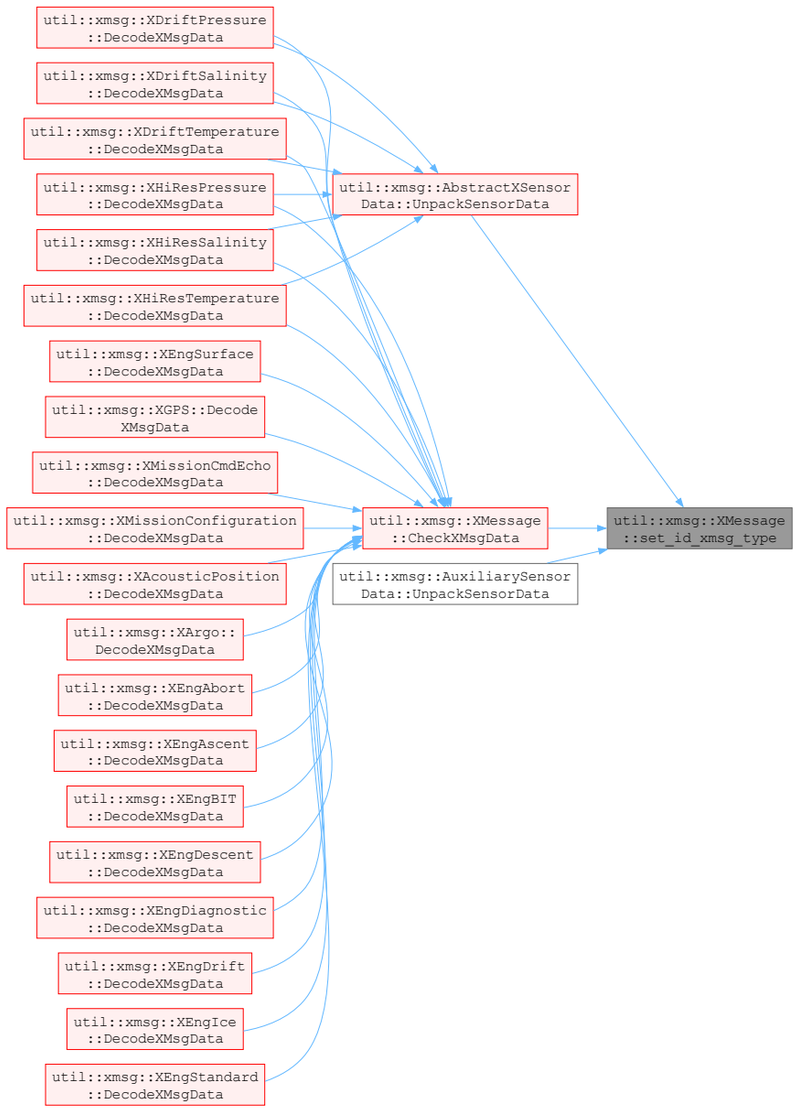
References util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.74.3.12 set_xmsg_size_bytes()

```
void util::xmsg::XMessage::set_xmsg_size_bytes (
            uint16_t xmsg_size_bytes)  [inline], [protected], [inherited]
```

Set the size of the X-Message, in bytes.

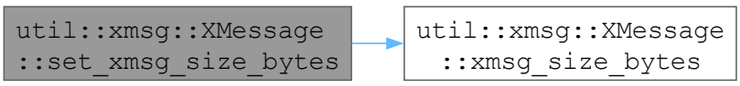
Parameters

in	*xmsg_size_bytes*	The size of the X-Message to set, in bytes.

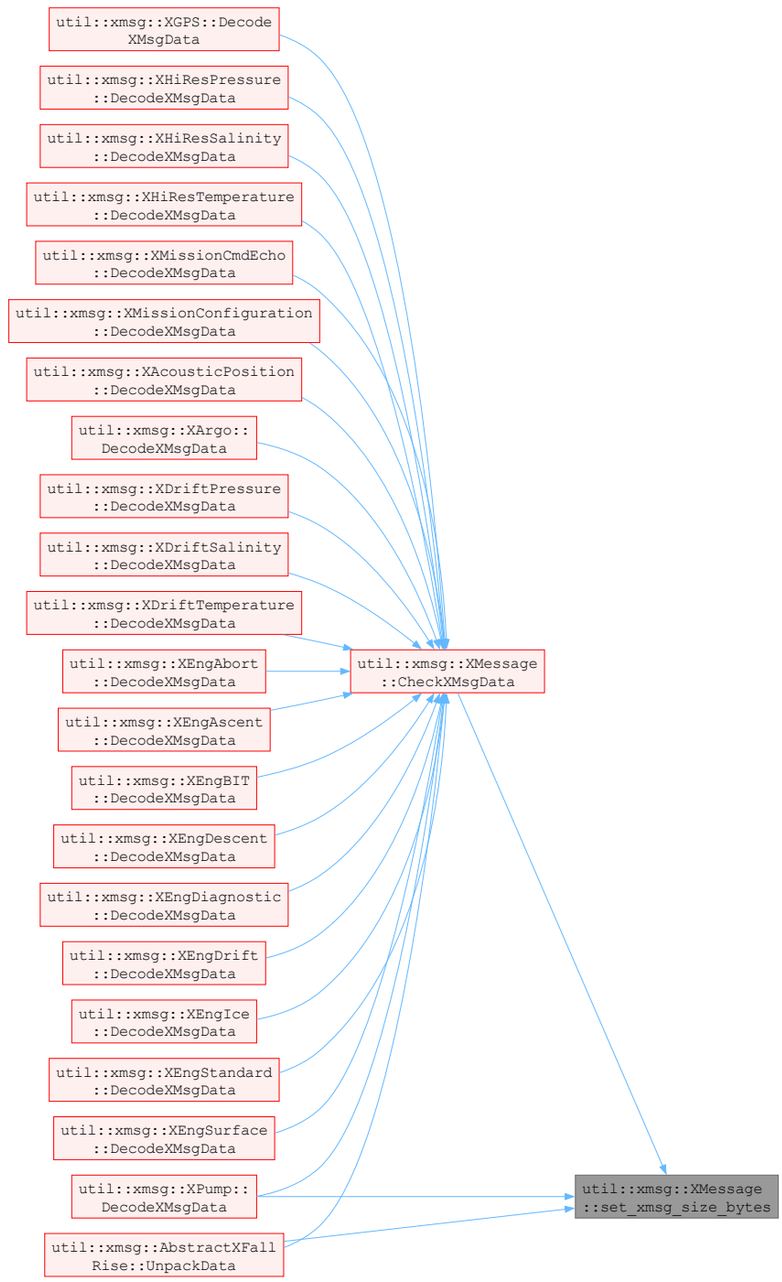
References util::xmsg::XMessage::xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XPump::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::Unpa

Here is the call graph for this function:



Here is the caller graph for this function:



10.74.3.13 xmsg_size_bytes()

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes () const  [inline], [inherited]
```

Get the size of the X-Message, in bytes.

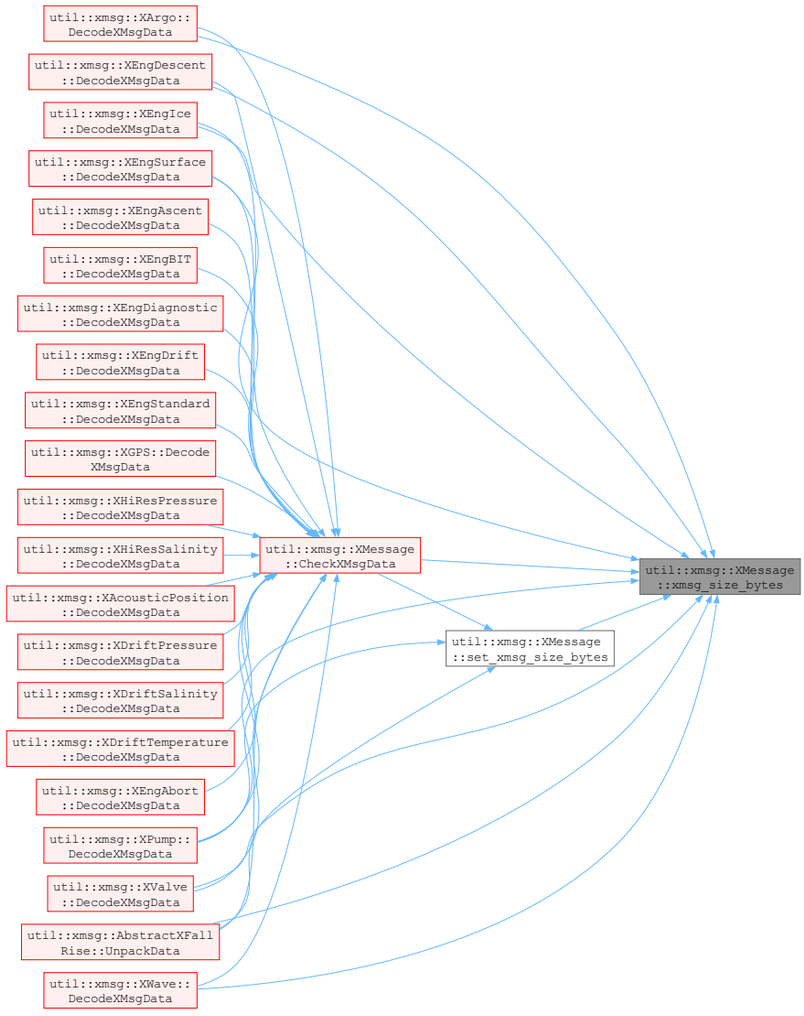
Returns

The size of the X-Message, in bytes.

References util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgI
util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(),
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), util::xmsg::XMessage::set_xmsg_size_bytes(),
and util::xmsg::AbstractXFallRise::UnpackData().

Here is the caller graph for this function:



10.74.4 Member Data Documentation

10.74.4.1 alamo_bit2_msg_

```
shared_ptr<XAlamoBit2> util::xmsg::XEngBIT::alamo_bit2_msg_  [private]
```

A shared pointer to the ALALMO Built-in-Test (BIT) message.

Note

This member is only used for ALAMO-type instruments starting with firmware version 10.x.

Referenced by DecodeXMsgData(), GenerateListingString(), and set_alamo_bit2_msg().

10.74.4.2 alamo_bit_msg_

```
shared_ptr<XAlamoBit> util::xmsg::XEngBIT::alamo_bit_msg_  [private]
```

A shared pointer to the ALALMO Built-in-Test (BIT) message.

Note

This member is only used for ALAMO-type instruments prior to firmware version 10.x.

Referenced by DecodeXMsgData(), GenerateListingString(), and set_alamo_bit_msg().

10.74.4.3 id_dive_

```
int16_t util::xmsg::XMessage::id_dive_  [private], [inherited]
```

The dive or cycle number of this message.

Referenced by util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::set_id_dive().

10.74.4.4 id_instrument_

```
uint16_t util::xmsg::XMessage::id_instrument_  [private], [inherited]
```

The unique WHOI identifier of this instrument.

Referenced by util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::set_id_instrument().

10.74.4.5 id_xmsg_type_

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type_  [private], [inherited]
```

The X-message type identifier.

Referenced by util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::set_id_xmsg_type().

10.74.4.6 xmsg_size_bytes_

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes_  [private], [inherited]
```

The size of the X message, in bytes.

Referenced by util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes().

The documentation for this class was generated from the following file:

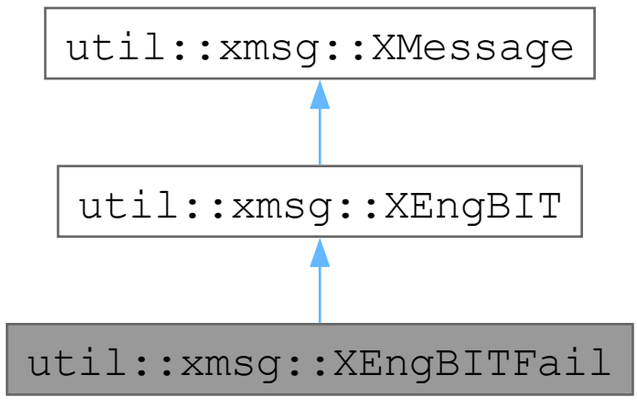
- data/xmsg/XEngBIT.h

10.75 util::xmsg::XEngBITFail Class Reference

Class representing an engineering startup message for BIT failed.

`#include <XEngBITFail.h>`

Inheritance diagram for util::xmsg::XEngBITFail:



Collaboration diagram for util::xmsg::XEngBITFail:

Public Member Functions

- XEngBITFail (int16_t dive, uint16_t whoi, const vector< uint8_t > data)

  Parameterized constructor.
- ~XEngBITFail ()

  The destructor.
- const uint8_t * CheckXMsgData (const vector< uint8_t > &data, int id, int minBytes, bool parseSize=true)

  Check the X-message for the correct ID, minimum size, and message terminator.
- void DecodeXMsgData (const vector< uint8_t > &data)

  Decode the BIT engineering X-Message from the binary data.
- const string GenerateListingString (uint16_t pg, uint16_t po, uint16_t sg, uint16_t so, uint16_t tg, uint16_t to) const

  Return a listing entry string representation of the BIT engineering X-Message.
- int16_t id_dive () const

  Get the dive or cycle number of this X-Message.
- uint16_t id_instrument () const

  Get the unique instrument identifier of this X-Message.
- XMsgTypeId_t id_xmsg_type () const

  Get the enumerated X-Message type identifier.
- uint16_t xmsg_size_bytes () const

  Get the size of the X-Message, in bytes.

Protected Member Functions

- void set_alamo_bit2_msg (const shared_ptr< XAlamoBit2 > &alamo_bit2_msg)
  Set the ALAMO Built-in-Test (BIT) v2 message.
- void set_alamo_bit_msg (const shared_ptr< XAlamoBit > &alamo_bit_msg)
  Set the ALALMO Built-in-Test (BIT) v1 message.
- void set_id_dive (int16_t id_dive)
  Set the dive or cycle number of this message.
- void set_id_instrument (uint16_t id_instrument)
  Set the unique instrument identifier.
- void set_id_xmsg_type (XMsgTypeId_t id_xmsg_type)
  Set the X-Message type identifier.
- void set_xmsg_size_bytes (uint16_t xmsg_size_bytes)
  Set the size of the X-Message, in bytes.

Private Attributes

- shared_ptr< XAlamoBit2 > alamo_bit2_msg_
  A shared pointer to the ALALMO Built-in-Test (BIT) message.
- shared_ptr< XAlamoBit > alamo_bit_msg_
  A shared pointer to the ALALMO Built-in-Test (BIT) message.
- int16_t id_dive_
  The dive or cycle number of this message.
- uint16_t id_instrument_
  The unique WHOI identifier of this instrument.
- XMsgTypeId_t id_xmsg_type_
  The X-message type identifier.
- uint16_t xmsg_size_bytes_
  The size of the X message, in bytes.

10.75.1 Detailed Description

Class representing an engineering startup message for BIT failed.

10.75.2 Constructor & Destructor Documentation

10.75.2.1 XEngBITFail()

```
util::xmsg::XEngBITFail::XEngBITFail (
            int16_t dive,
            uint16_t whoi,
            const vector< uint8_t > data)  [inline]
```

Parameterized constructor.

Parameters

in	*dive*	The dive or cycle number of this message.
in	*whoi*	The unique WHOI identifier.
in	*data*	The binary X-message data to decode.

Exceptions

util::log::MessageException	if an decode error occurs.

10.75.2.2 ~XEngBITFail()

```
util::xmsg::XEngBITFail::~XEngBITFail ()  [inline]
```

The destructor.

10.75.3 Member Function Documentation

10.75.3.1 CheckXMsgData()

```
const uint8_t * util::xmsg::XMessage::CheckXMsgData (
            const vector< uint8_t > & data,
            int id,
            int minBytes,
            bool parseSize = true)  [inline], [inherited]
```

Check the X-message for the correct ID, minimum size, and message terminator.

Parameters

in	*data*	The X-message binary data to check.
in	*id*	The expected ID of the X-message. If this field is less that 0, then the ID check will be skipped.
in	*minBytes*	The minimum number of bytes expected in a valid message.
in	*parseSize*	If TRUE, then the message size field will be parsed from the X-Message. Otherwise, the size field will not be parsed.

Exceptions

util::log::MessageException	if • the message does not contain the expected ID, • the message does not contain the minimum number of expected bytes, • the message does not contain a terminator (';') in the correct location, or, • any other error occurs.

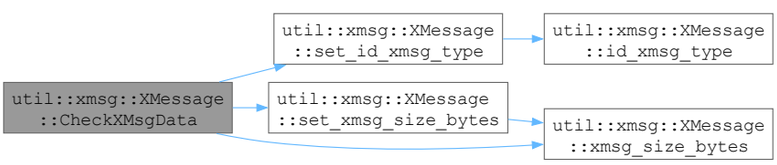
Returns

A pointer to the underlying data buffer of data.

References CATCH_BLOCK, EXC_THROW, lwrn, util::xmsg::XMessage::set_id_xmsg_type(), util::xmsg::XMessage::set_xmsg_size_byte
u16, and util::xmsg::XMessage::xmsg_size_bytes().

Referenced by util::xmsg::XAcousticPosition::DecodeXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XDriftPressure::Dec
util::xmsg::XDriftSalinity::DecodeXMsgData(), util::xmsg::XDriftTemperature::DecodeXMsgData(), util::xmsg::XEngAbort::DecodeXMsgDa
util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngBIT::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(),
util::xmsg::XEngDiagnostic::DecodeXMsgData(), util::xmsg::XEngDrift::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(),
util::xmsg::XEngStandard::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XGPS::DecodeXMsgData(),
util::xmsg::XHiResPressure::DecodeXMsgData(), util::xmsg::XHiResSalinity::DecodeXMsgData(), util::xmsg::XHiResTemperature::Decod
util::xmsg::XMissionCmdEcho::DecodeXMsgData(), util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XProfilePressure::
util::xmsg::XProfileSalinity::DecodeXMsgData(), util::xmsg::XProfileTemperature::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgD
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the call graph for this function:



10.75.3.2 DecodeXMsgData()

```
void util::xmsg::XEngBIT::DecodeXMsgData (
            const vector< uint8_t > & data)  [inline], [virtual], [inherited]
```

Decode the BIT engineering X-Message from the binary data.

Parameters

in	*data*	The binary message to decode.

Exceptions

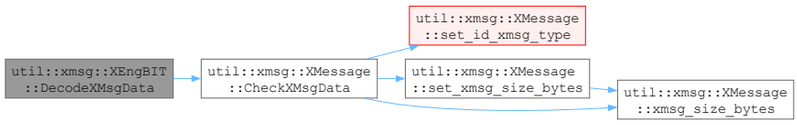
util::log::MessageException	if an error occurs while decoding.

Implements util::xmsg::XMessage.

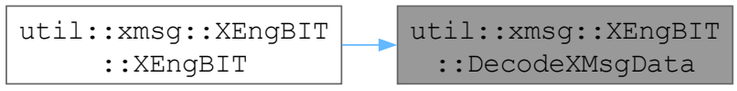
References util::xmsg::XEngBIT::alamo_bit2_msg_, util::xmsg::XEngBIT::alamo_bit_msg_, CATCH_BLOCK, util::xmsg::XMessage::Chec and EXC_THROW.

Referenced by util::xmsg::XEngBIT::XEngBIT().

Here is the call graph for this function:



Here is the caller graph for this function:



10.75.3.3 GenerateListingString()

```
const string util::xmsg::XEngBIT::GenerateListingString (
            uint16_t pg,
            uint16_t po,
            uint16_t sg,
            uint16_t so,
            uint16_t tg,
            uint16_t to) const  [inline], [inherited]
```

Return a listing entry string representation of the BIT engineering X-Message.

Parameters

in	*pg*	The pressure sensor sensitivity (gain) scale factor, in LSB/dBar.
in	*po*	The pressure sensor offset, in dBar.
in	*sg*	The salinity sensor sensitivity (gain) scale factor, in LSB/PSU.
in	*so*	The salinity sensor offset, in PSU.
in	*tg*	The temperature sensor sensitivity (gain) scale factor, in LSB/degC.
in	*to*	The temperature sensor offset, in degC.

Exceptions

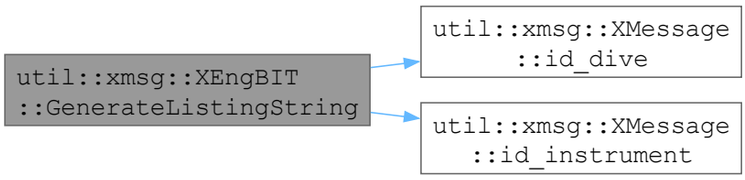
util::log::MessageException	if an error occurs while generating the listing entry string representation.

Returns

A listing entry string representation of the BIT engineering X-Message.

References util::xmsg::XEngBIT::alamo_bit2_msg_, util::xmsg::XEngBIT::alamo_bit_msg_, CATCH_BLOCK, util::xmsg::XMessage::id_div
and util::xmsg::XMessage::id_instrument().

Here is the call graph for this function:



10.75.3.4 id_dive()

```
int16_t util::xmsg::XMessage::id_dive () const  [inline], [inherited]
```

Get the dive or cycle number of this X-Message.

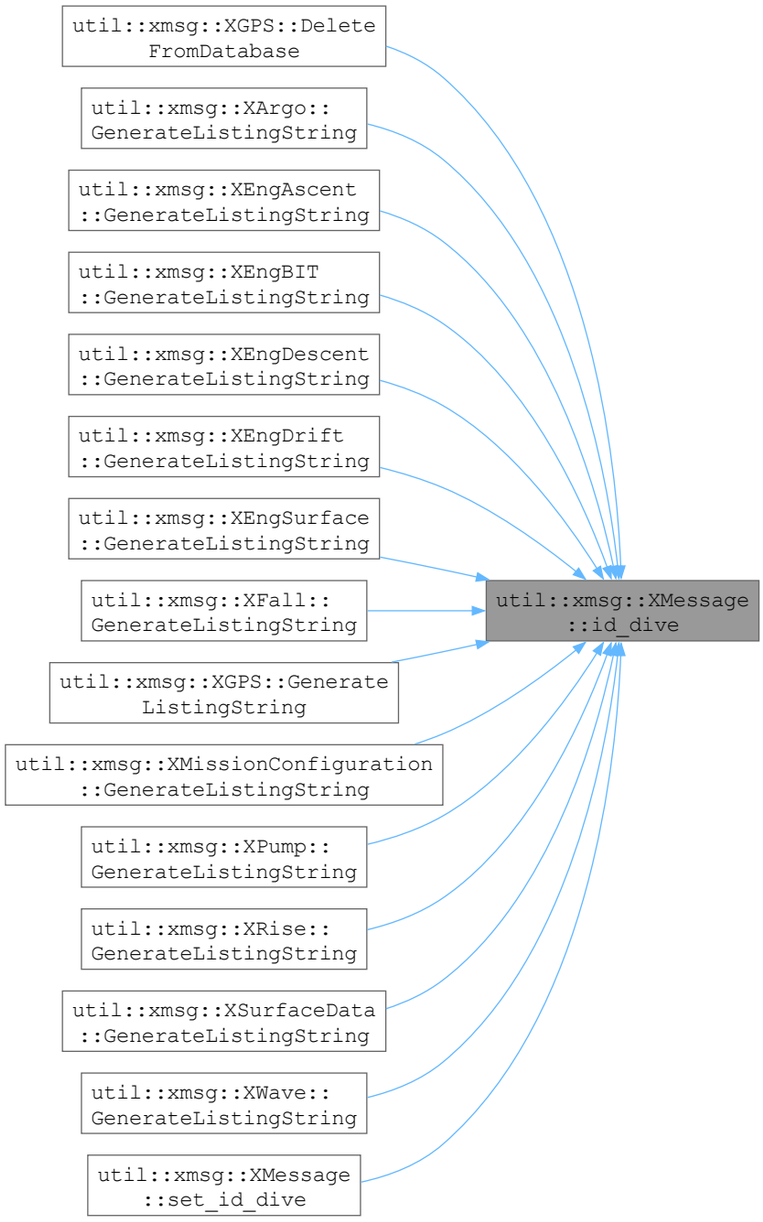
Returns

The dive or cycle number of this X-Message.

References util::xmsg::XMessage::id_dive_.

Referenced by util::xmsg::XGPS::DeleteFromDatabase(), util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateLis
util::xmsg::XEngBIT::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingStr
util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(),
util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing
util::xmsg::XSurfaceData::GenerateListingString(), util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_dive().

Here is the caller graph for this function:



10.75.3.5 id_instrument()

```
uint16_t util::xmsg::XMessage::id_instrument () const  [inline], [inherited]
```

Get the unique instrument identifier of this X-Message.

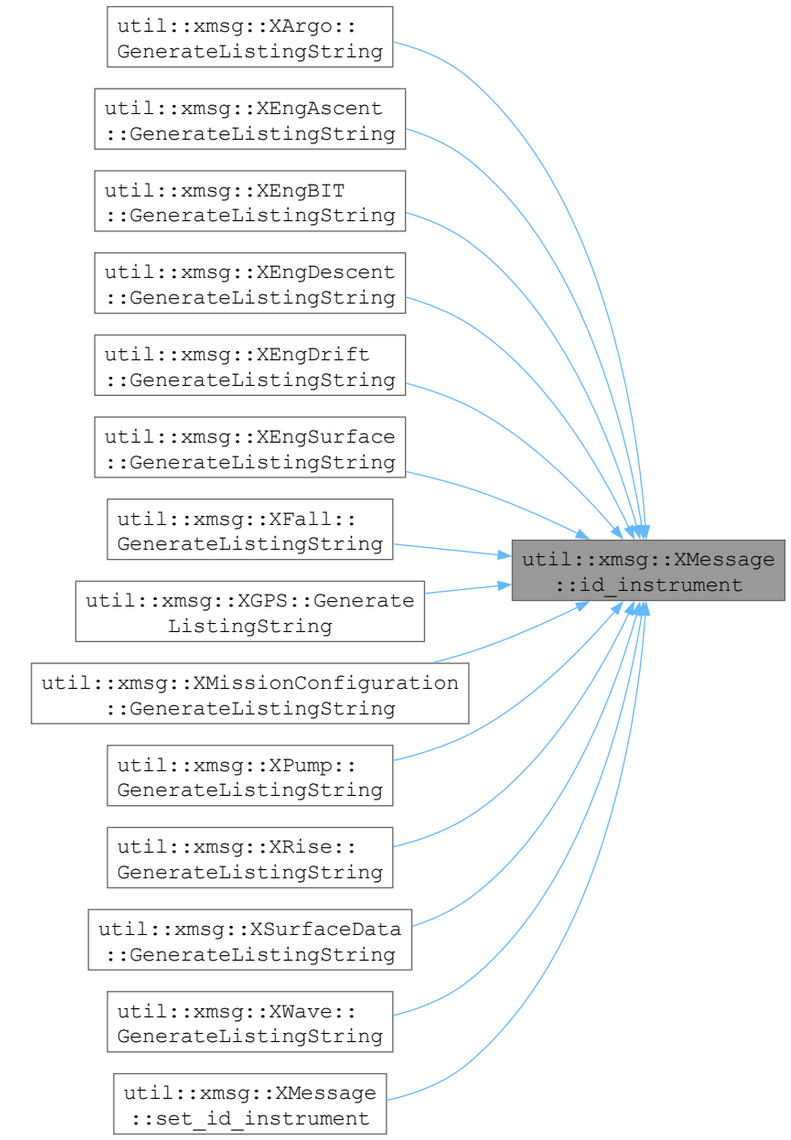
Returns

The unique instrument identifier of this X-Message.

References util::xmsg::XMessage::id_instrument_.

Referenced by util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngBIT::Generate
util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), util::xmsg::XEngSurface::GenerateListin
util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingS
util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(),
util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_instrument().

Here is the caller graph for this function:



10.75.3.6 id_xmsg_type()

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type () const  [inline], [inherited]
```

Get the enumerated X-Message type identifier.

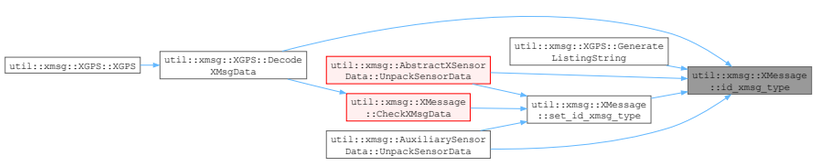
Returns

The enumerated X-Message type identifier.

References util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMessage::set_id_xmsg_typ
util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the caller graph for this function:



10.75.3.7 set_alamo_bit2_msg()

```
void util::xmsg::XEngBIT::set_alamo_bit2_msg (
            const shared_ptr< XAlamoBit2 > & alamo_bit2_msg)  [inline], [protected], [inherited]
```

Set the ALAMO Built-in-Test (BIT) v2 message.

Parameters

in	*alamo_bit2_msg*	A shared pointer to the ALAMO Built-in-Test (BIT) v2 message to set.

References util::xmsg::XEngBIT::alamo_bit2_msg_.

10.75.3.8 set_alamo_bit_msg()

```
void util::xmsg::XEngBIT::set_alamo_bit_msg (
            const shared_ptr< XAlamoBit > & alamo_bit_msg)  [inline], [protected], [inherited]
```

Set the ALALMO Built-in-Test (BIT) v1 message.

Parameters

in	*alamo_bit_msg*	A shared pointer to the ALALMO Built-in-Test (BIT) message to set.

References util::xmsg::XEngBIT::alamo_bit_msg_.

10.75.3.9 set_id_dive()

```
void util::xmsg::XMessage::set_id_dive (
            int16_t id_dive)  [inline], [protected], [inherited]
```

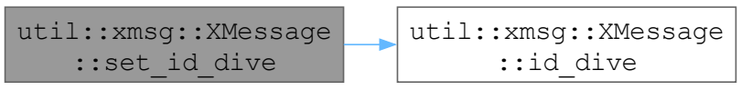
Set the dive or cycle number of this message.

Parameters

in	*id_dive*	The dive or cycle number of this message to set.

References util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::id_dive_.

Here is the call graph for this function:



10.75.3.10 set_id_instrument()

```
void util::xmsg::XMessage::set_id_instrument (
            uint16_t id_instrument)  [inline], [protected], [inherited]
```

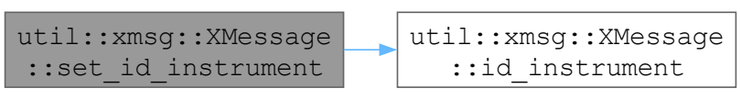
Set the unique instrument identifier.

Parameters

in	*id_instrument*	The unique instrument identifier to set.

References util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::id_instrument_.

Here is the call graph for this function:



10.75.3.11 set_id_xmsg_type()

```
void util::xmsg::XMessage::set_id_xmsg_type (
            XMsgTypeId_t id_xmsg_type)  [inline], [protected], [inherited]
```

Set the X-Message type identifier.

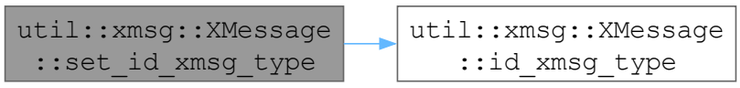
Parameters

in	*id_xmsg_type*	The X-Message type identifier to set.

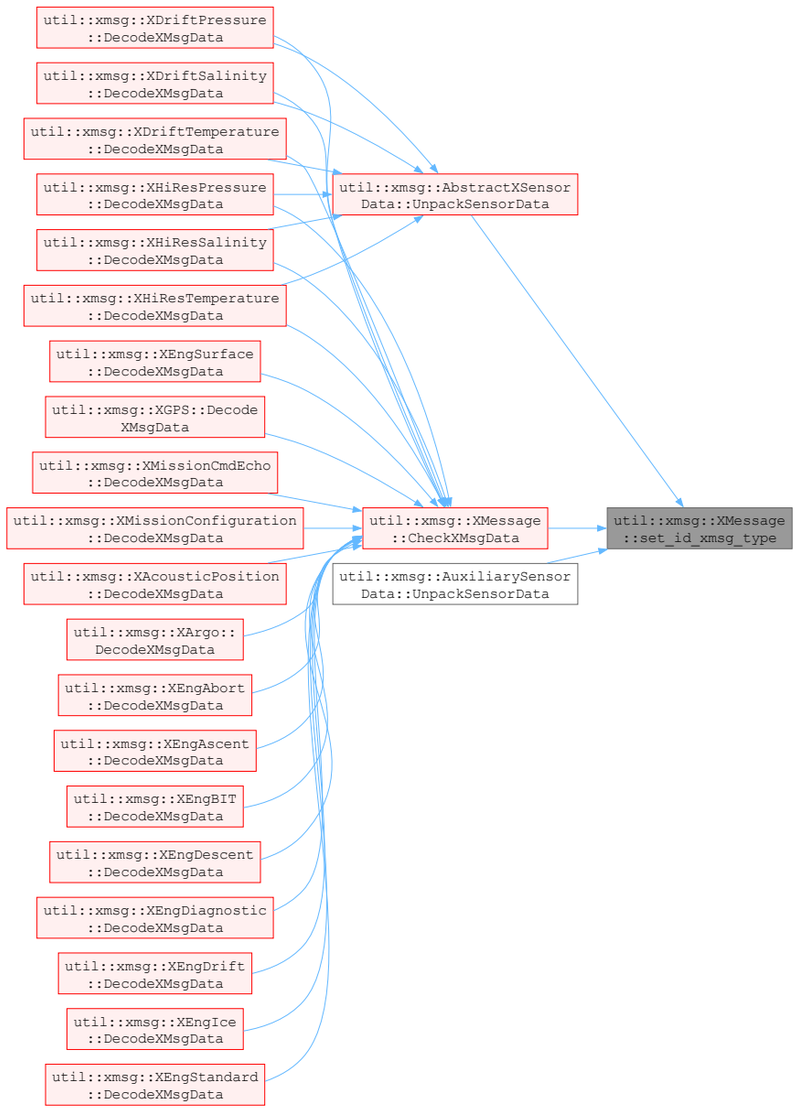
References util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.75.3.12 set_xmsg_size_bytes()

```
void util::xmsg::XMessage::set_xmsg_size_bytes (
            uint16_t xmsg_size_bytes)  [inline], [protected], [inherited]
```

Set the size of the X-Message, in bytes.

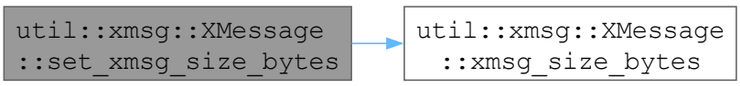
Parameters

in	*xmsg_size_bytes*	The size of the X-Message to set, in bytes.

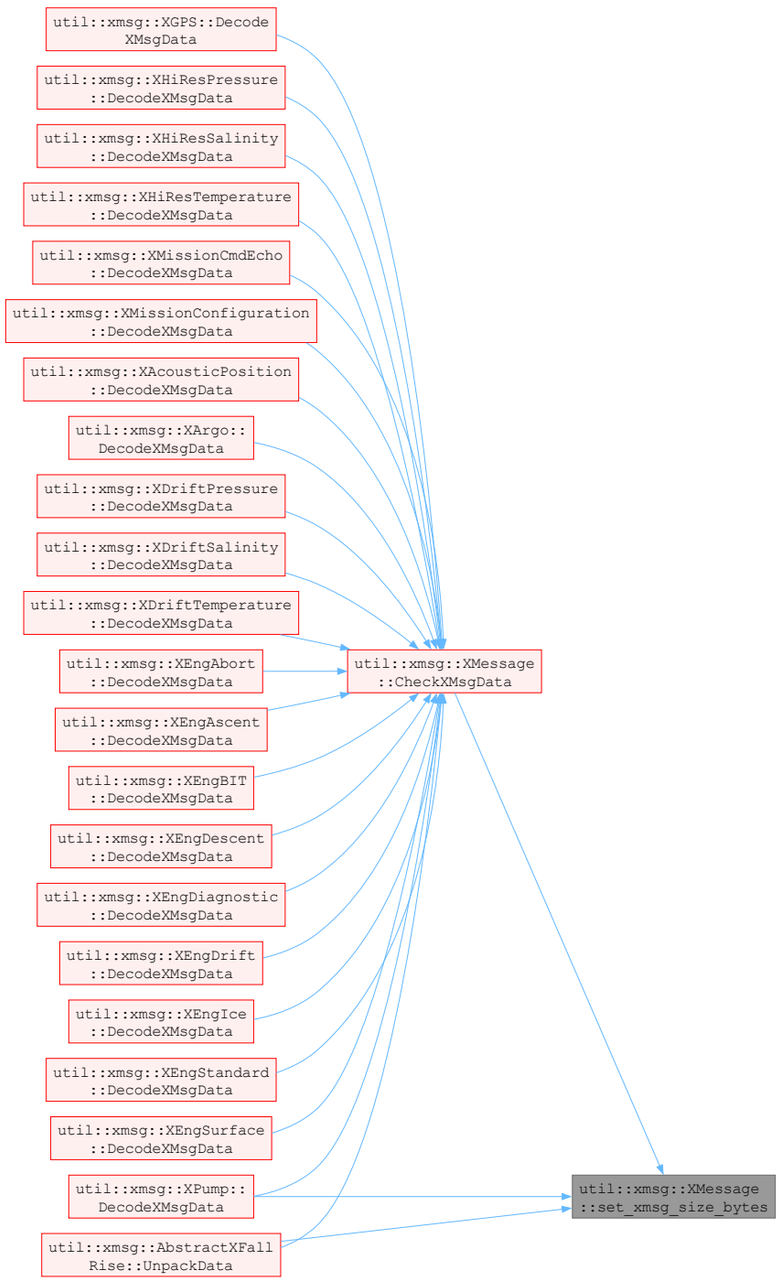
References util::xmsg::XMessage::xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XPump::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::Unpa

Here is the call graph for this function:



Here is the caller graph for this function:



10.75.3.13 xmsg_size_bytes()

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes () const  [inline], [inherited]
```

Get the size of the X-Message, in bytes.

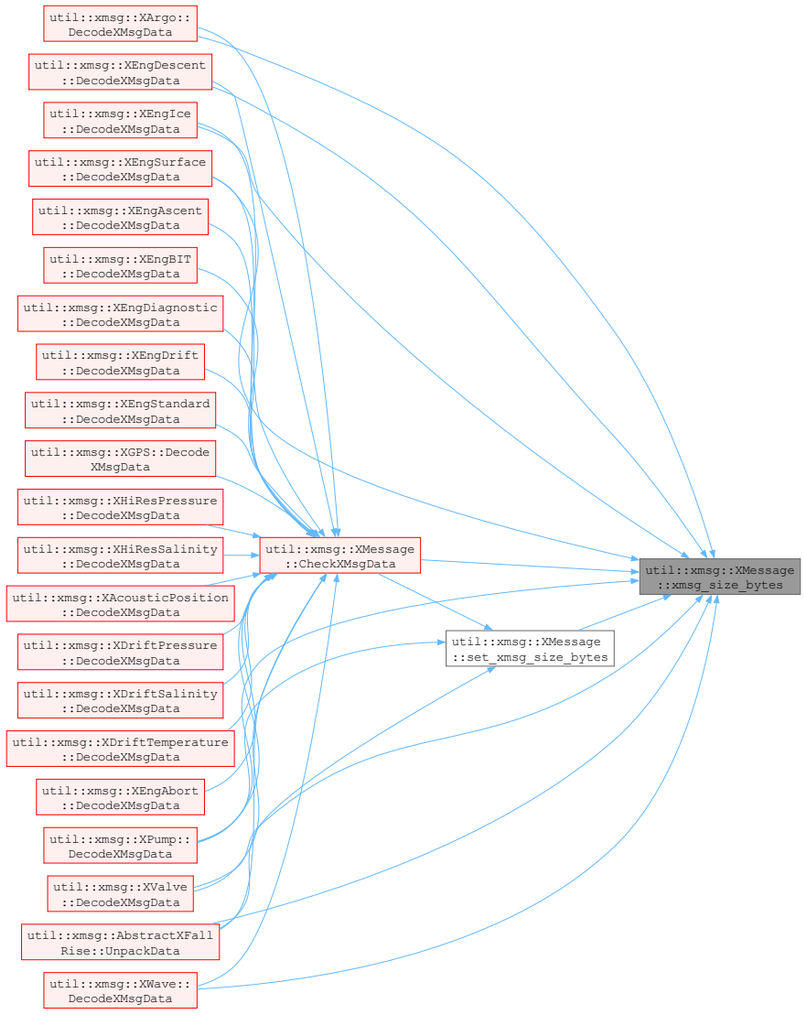
Returns

The size of the X-Message, in bytes.

References util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgI
util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(),
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), util::xmsg::XMessage::set_xmsg_size_bytes(),
and util::xmsg::AbstractXFallRise::UnpackData().

Here is the caller graph for this function:



10.75.4 Member Data Documentation

10.75.4.1 alamo_bit2_msg_

```
shared_ptr<XAlamoBit2> util::xmsg::XEngBIT::alamo_bit2_msg_  [private], [inherited]
```

A shared pointer to the ALALMO Built-in-Test (BIT) message.

Note

This member is only used for ALAMO-type instruments starting with firmware version 10.x.

Referenced by util::xmsg::XEngBIT::DecodeXMsgData(), util::xmsg::XEngBIT::GenerateListingString(), and util::xmsg::XEngBIT::set_alam

10.75.4.2 alamo_bit_msg_

```
shared_ptr<XAlamoBit> util::xmsg::XEngBIT::alamo_bit_msg_  [private], [inherited]
```

A shared pointer to the ALALMO Built-in-Test (BIT) message.

Note

This member is only used for ALAMO-type instruments prior to firmware version 10.x.

Referenced by util::xmsg::XEngBIT::DecodeXMsgData(), util::xmsg::XEngBIT::GenerateListingString(), and util::xmsg::XEngBIT::set_alam

10.75.4.3 id_dive_

```
int16_t util::xmsg::XMessage::id_dive_  [private], [inherited]
```

The dive or cycle number of this message.

Referenced by util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::set_id_dive().

10.75.4.4 id_instrument_

```
uint16_t util::xmsg::XMessage::id_instrument_  [private], [inherited]
```

The unique WHOI identifier of this instrument.

Referenced by util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::set_id_instrument().

10.75.4.5 id_xmsg_type_

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type_  [private], [inherited]
```

The X-message type identifier.

Referenced by util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::set_id_xmsg_type().

10.75.4.6 xmsg_size_bytes_

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes_  [private], [inherited]
```

The size of the X message, in bytes.

Referenced by util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes().

The documentation for this class was generated from the following file:

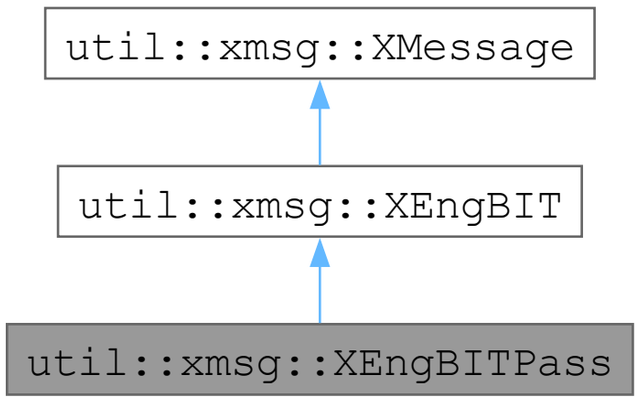
- data/xmsg/XEngBITFail.h

10.76 util::xmsg::XEngBITPass Class Reference

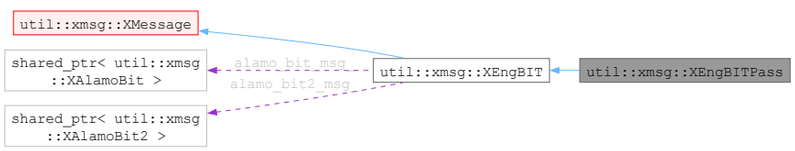
Class representing an engineering startup message for BIT passed.

`#include <XEngBITPass.h>`

Inheritance diagram for util::xmsg::XEngBITPass:



Collaboration diagram for util::xmsg::XEngBITPass:



Public Member Functions

- XEngBITPass (int16_t dive, uint16_t whoi, const vector< uint8_t > data)

  Parameterized constructor.
- ~XEngBITPass ()

  Destructor.
- const uint8_t * CheckXMsgData (const vector< uint8_t > &data, int id, int minBytes, bool parseSize=true)

  Check the X-message for the correct ID, minimum size, and message terminator.
- void DecodeXMsgData (const vector< uint8_t > &data)

  Decode the BIT engineering X-Message from the binary data.
- const string GenerateListingString (uint16_t pg, uint16_t po, uint16_t sg, uint16_t so, uint16_t tg, uint16_t to) const

  Return a listing entry string representation of the BIT engineering X-Message.
- int16_t id_dive () const

  Get the dive or cycle number of this X-Message.
- uint16_t id_instrument () const

  Get the unique instrument identifier of this X-Message.
- XMsgTypeId_t id_xmsg_type () const

  Get the enumerated X-Message type identifier.
- uint16_t xmsg_size_bytes () const

  Get the size of the X-Message, in bytes.

Protected Member Functions

- void set_alamo_bit2_msg (const shared_ptr< XAlamoBit2 > &alamo_bit2_msg)

  Set the ALAMO Built-in-Test (BIT) v2 message.
- void set_alamo_bit_msg (const shared_ptr< XAlamoBit > &alamo_bit_msg)

  Set the ALALMO Built-in-Test (BIT) v1 message.
- void set_id_dive (int16_t id_dive)

  Set the dive or cycle number of this message.
- void set_id_instrument (uint16_t id_instrument)

  Set the unique instrument identifier.
- void set_id_xmsg_type (XMsgTypeId_t id_xmsg_type)

  Set the X-Message type identifier.
- void set_xmsg_size_bytes (uint16_t xmsg_size_bytes)

  Set the size of the X-Message, in bytes.

Private Attributes

- shared_ptr< XAlamoBit2 > alamo_bit2_msg_

  A shared pointer to the ALALMO Built-in-Test (BIT) message.
- shared_ptr< XAlamoBit > alamo_bit_msg_

  A shared pointer to the ALALMO Built-in-Test (BIT) message.
- int16_t id_dive_

  The dive or cycle number of this message.
- uint16_t id_instrument_

  The unique WHOI identifier of this instrument.
- XMsgTypeId_t id_xmsg_type_

  The X-message type identifier.
- uint16_t xmsg_size_bytes_

  The size of the X message, in bytes.

10.76.1 Detailed Description

Class representing an engineering startup message for BIT passed.

Todo Add support for Solo-II type bit passed engineering data message.

10.76.2 Constructor & Destructor Documentation

10.76.2.1 XEngBITPass()

```
util::xmsg::XEngBITPass::XEngBITPass (
            int16_t dive,
            uint16_t whoi,
            const vector< uint8_t > data)  [inline]
```

Parameterized constructor.

Parameters

in	*dive*	The dive or cycle number of this message.
in	*whoi*	The unique WHOI identifier.
in	*data*	The binary X-message data to decode.

Exceptions

util::log::MessageException	if an decode error occurs.

10.76.2.2 ~XEngBITPass()

```
util::xmsg::XEngBITPass::~XEngBITPass ()  [inline]
```

Destructor.

10.76.3 Member Function Documentation

10.76.3.1 CheckXMsgData()

```
const uint8_t * util::xmsg::XMessage::CheckXMsgData (
            const vector< uint8_t > & data,
            int id,
            int minBytes,
            bool parseSize = true)  [inline], [inherited]
```

Check the X-message for the correct ID, minimum size, and message terminator.

Parameters

in	*data*	The X-message binary data to check.
in	*id*	The expected ID of the X-message. If this field is less that 0, then the ID check will be skipped.
in	*minBytes*	The minimum number of bytes expected in a valid message.
in	*parseSize*	If TRUE, then the message size field will be parsed from the X-Message. Otherwise, the size field will not be parsed.

Exceptions

util::log::MessageException	if • the message does not contain the expected ID, • the message does not contain the minimum number of expected bytes, • the message does not contain a terminator (';') in the correct location, or, • any other error occurs.

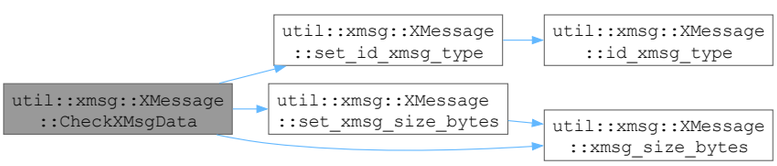
Returns

A pointer to the underlying data buffer of data.

References CATCH_BLOCK, EXC_THROW, lwrn, util::xmsg::XMessage::set_id_xmsg_type(), util::xmsg::XMessage::set_xmsg_size_byte
u16, and util::xmsg::XMessage::xmsg_size_bytes().

Referenced by util::xmsg::XAcousticPosition::DecodeXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XDriftPressure::Dec
util::xmsg::XDriftSalinity::DecodeXMsgData(), util::xmsg::XDriftTemperature::DecodeXMsgData(), util::xmsg::XEngAbort::DecodeXMsgDa
util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngBIT::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(),
util::xmsg::XEngDiagnostic::DecodeXMsgData(), util::xmsg::XEngDrift::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(),
util::xmsg::XEngStandard::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XGPS::DecodeXMsgData(),
util::xmsg::XHiResPressure::DecodeXMsgData(), util::xmsg::XHiResSalinity::DecodeXMsgData(), util::xmsg::XHiResTemperature::Decod
util::xmsg::XMissionCmdEcho::DecodeXMsgData(), util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XProfilePressure:
util::xmsg::XProfileSalinity::DecodeXMsgData(), util::xmsg::XProfileTemperature::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgD
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the call graph for this function:



10.76.3.2 DecodeXMsgData()

```
void util::xmsg::XEngBIT::DecodeXMsgData (
            const vector< uint8_t > & data)  [inline], [virtual], [inherited]
```

Decode the BIT engineering X-Message from the binary data.

Parameters

in	*data*	The binary message to decode.

Exceptions

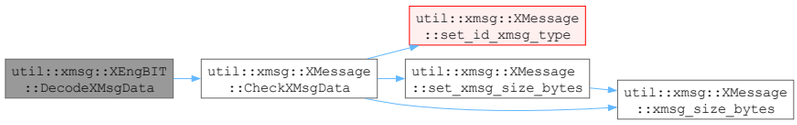
util::log::MessageException	if an error occurs while decoding.

Implements util::xmsg::XMessage.

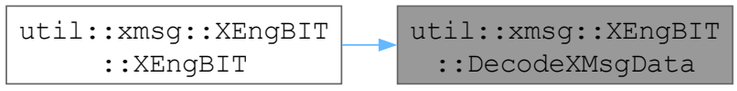
References util::xmsg::XEngBIT::alamo_bit2_msg_, util::xmsg::XEngBIT::alamo_bit_msg_, CATCH_BLOCK, util::xmsg::XMessage::Chec and EXC_THROW.

Referenced by util::xmsg::XEngBIT::XEngBIT().

Here is the call graph for this function:



Here is the caller graph for this function:



10.76.3.3 GenerateListingString()

```
const string util::xmsg::XEngBIT::GenerateListingString (
            uint16_t pg,
            uint16_t po,
            uint16_t sg,
            uint16_t so,
            uint16_t tg,
            uint16_t to) const  [inline], [inherited]
```

Return a listing entry string representation of the BIT engineering X-Message.

Parameters

in	*pg*	The pressure sensor sensitivity (gain) scale factor, in LSB/dBar.
in	*po*	The pressure sensor offset, in dBar.
in	*sg*	The salinity sensor sensitivity (gain) scale factor, in LSB/PSU.
in	*so*	The salinity sensor offset, in PSU.
in	*tg*	The temperature sensor sensitivity (gain) scale factor, in LSB/degC.
in	*to*	The temperature sensor offset, in degC.

Exceptions

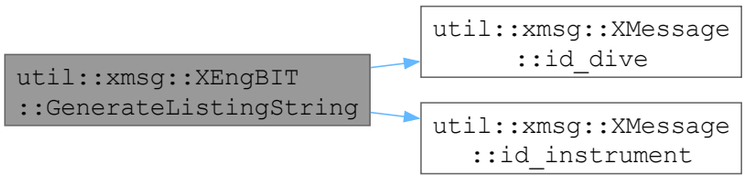
util::log::MessageException	if an error occurs while generating the listing entry string representation.

Returns

A listing entry string representation of the BIT engineering X-Message.

References util::xmsg::XEngBIT::alamo_bit2_msg_, util::xmsg::XEngBIT::alamo_bit_msg_, CATCH_BLOCK, util::xmsg::XMessage::id_div
and util::xmsg::XMessage::id_instrument().

Here is the call graph for this function:



10.76.3.4 id_dive()

```
int16_t util::xmsg::XMessage::id_dive () const  [inline], [inherited]
```

Get the dive or cycle number of this X-Message.

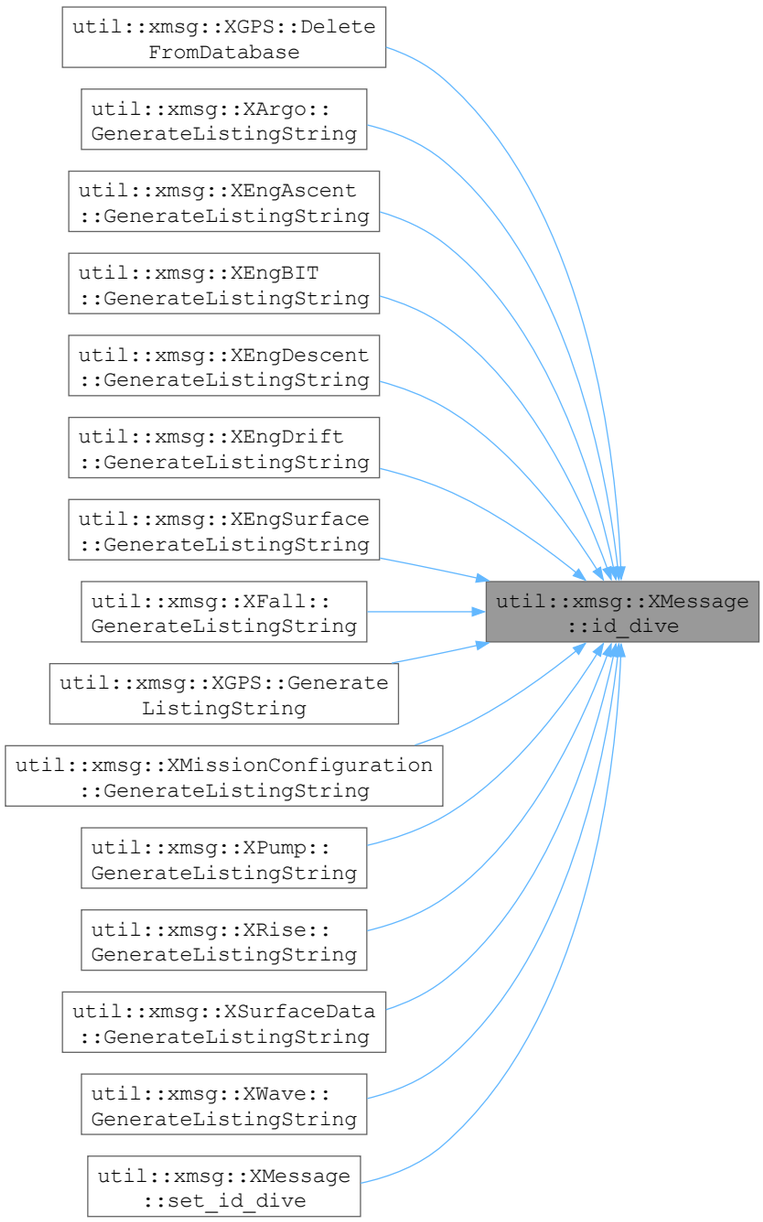
Returns

The dive or cycle number of this X-Message.

References util::xmsg::XMessage::id_dive_.

Referenced by util::xmsg::XGPS::DeleteFromDatabase(), util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateLis
util::xmsg::XEngBIT::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingStr
util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(),
util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing
util::xmsg::XSurfaceData::GenerateListingString(), util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_dive().

Here is the caller graph for this function:



10.76.3.5 id_instrument()

```
uint16_t util::xmsg::XMessage::id_instrument () const  [inline], [inherited]
```

Get the unique instrument identifier of this X-Message.

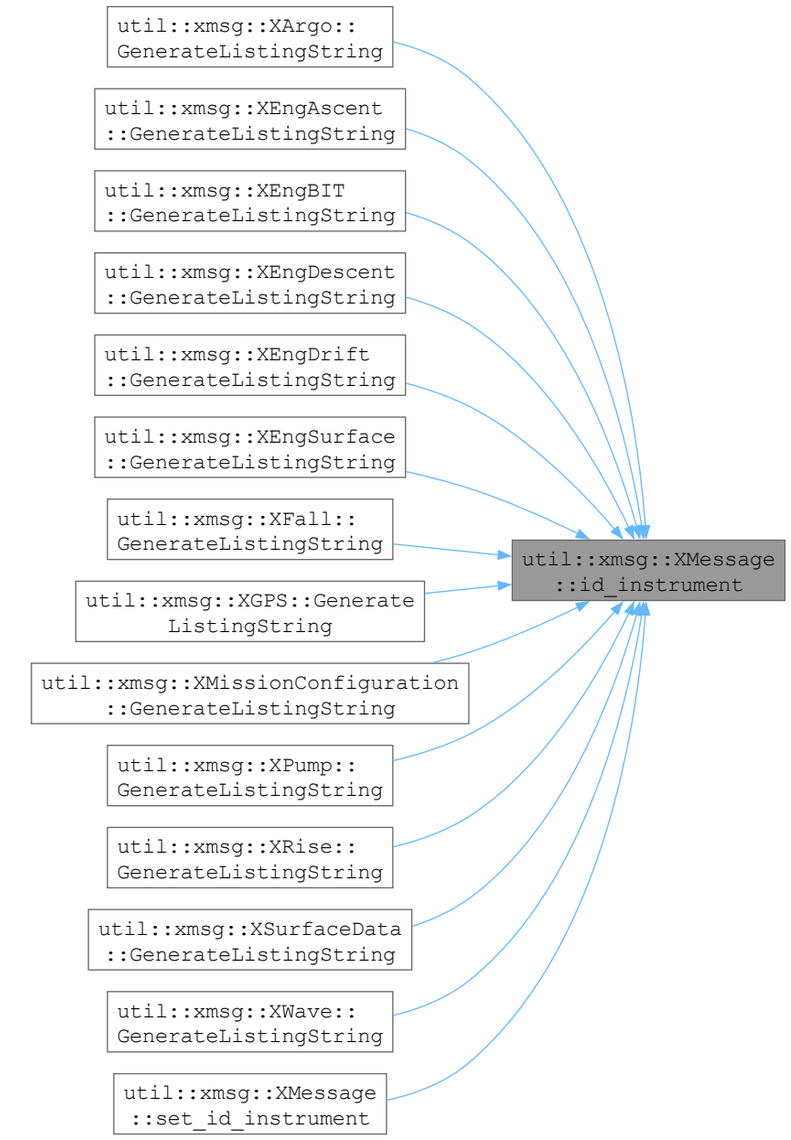
Returns

The unique instrument identifier of this X-Message.

References util::xmsg::XMessage::id_instrument_.

Referenced by util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngBIT::Generate
util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), util::xmsg::XEngSurface::GenerateListin
util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingS
util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(),
util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_instrument().

Here is the caller graph for this function:



10.76.3.6 id_xmsg_type()

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type () const  [inline], [inherited]
```

Get the enumerated X-Message type identifier.

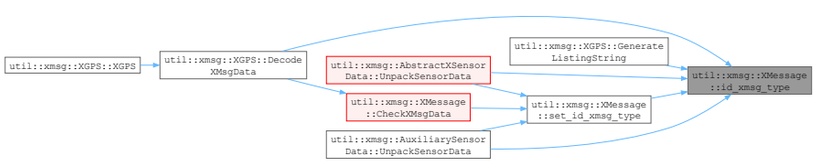
Returns

The enumerated X-Message type identifier.

References util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMessage::set_id_xmsg_typ
util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the caller graph for this function:



10.76.3.7 set_alamo_bit2_msg()

```
void util::xmsg::XEngBIT::set_alamo_bit2_msg (
            const shared_ptr< XAlamoBit2 > & alamo_bit2_msg)  [inline], [protected], [inherited]
```

Set the ALAMO Built-in-Test (BIT) v2 message.

Parameters

in	*alamo_bit2_msg*	A shared pointer to the ALAMO Built-in-Test (BIT) v2 message to set.

References util::xmsg::XEngBIT::alamo_bit2_msg_.

10.76.3.8 set_alamo_bit_msg()

```
void util::xmsg::XEngBIT::set_alamo_bit_msg (
            const shared_ptr< XAlamoBit > & alamo_bit_msg)  [inline], [protected], [inherited]
```

Set the ALALMO Built-in-Test (BIT) v1 message.

Parameters

in	*alamo_bit_msg*	A shared pointer to the ALALMO Built-in-Test (BIT) message to set.

References util::xmsg::XEngBIT::alamo_bit_msg_.

10.76.3.9 set_id_dive()

```
void util::xmsg::XMessage::set_id_dive (
            int16_t id_dive)  [inline], [protected], [inherited]
```

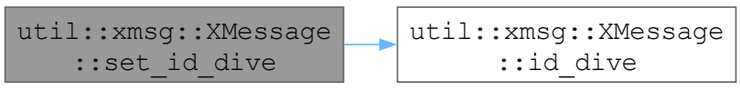
Set the dive or cycle number of this message.

Parameters

in	*id_dive*	The dive or cycle number of this message to set.

References util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::id_dive_.

Here is the call graph for this function:



10.76.3.10 set_id_instrument()

```
void util::xmsg::XMessage::set_id_instrument (
            uint16_t id_instrument)  [inline], [protected], [inherited]
```

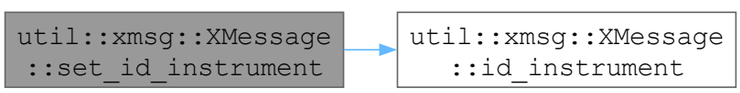
Set the unique instrument identifier.

Parameters

in	*id_instrument*	The unique instrument identifier to set.

References util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::id_instrument_.

Here is the call graph for this function:



10.76.3.11 set_id_xmsg_type()

```
void util::xmsg::XMessage::set_id_xmsg_type (
            XMsgTypeId_t id_xmsg_type)  [inline], [protected], [inherited]
```

Set the X-Message type identifier.

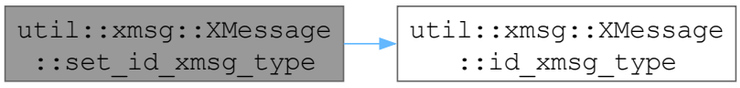
Parameters

in	*id_xmsg_type*	The X-Message type identifier to set.

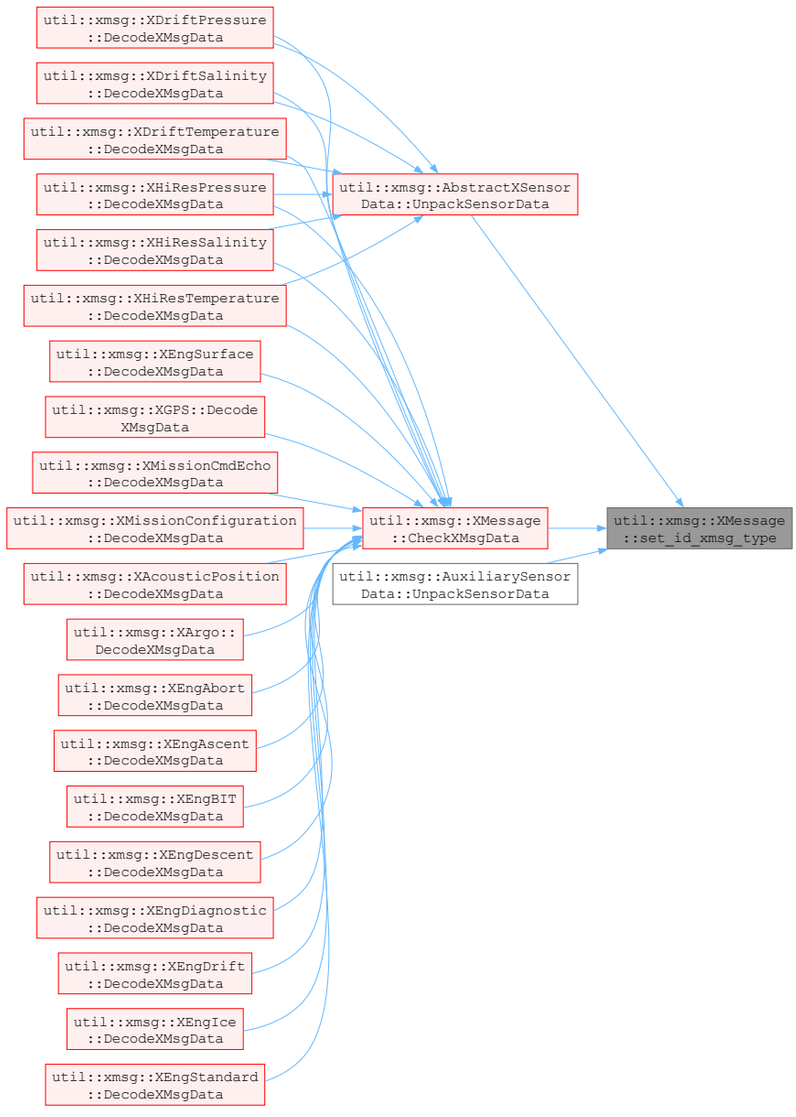
References util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.76.3.12 set_xmsg_size_bytes()

```
void util::xmsg::XMessage::set_xmsg_size_bytes (
            uint16_t xmsg_size_bytes)  [inline], [protected], [inherited]
```

Set the size of the X-Message, in bytes.

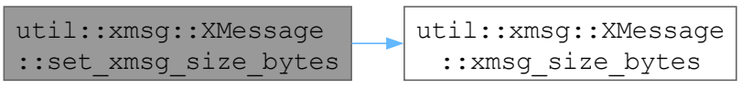
Parameters

in	*xmsg_size_bytes*	The size of the X-Message to set, in bytes.

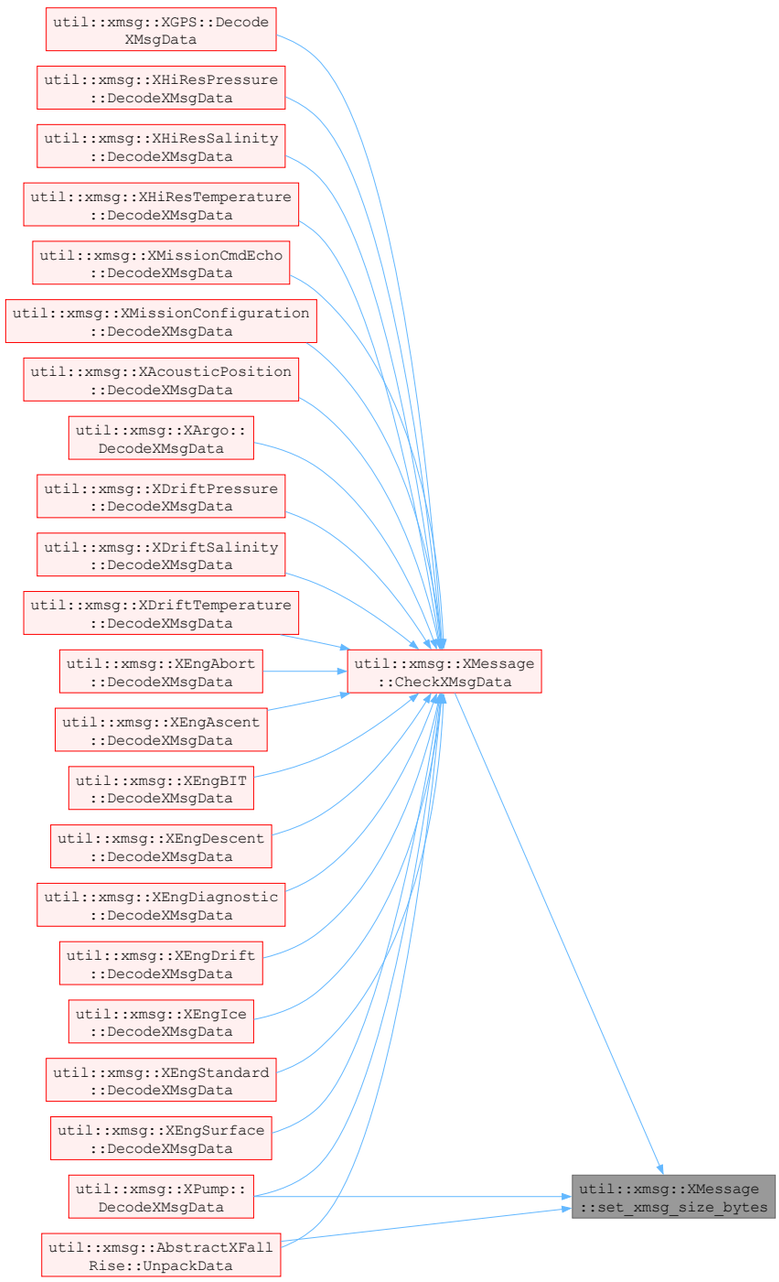
References util::xmsg::XMessage::xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XPump::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::Unpa

Here is the call graph for this function:



Here is the caller graph for this function:



10.76.3.13 xmsg_size_bytes()

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes () const  [inline], [inherited]
```

Get the size of the X-Message, in bytes.

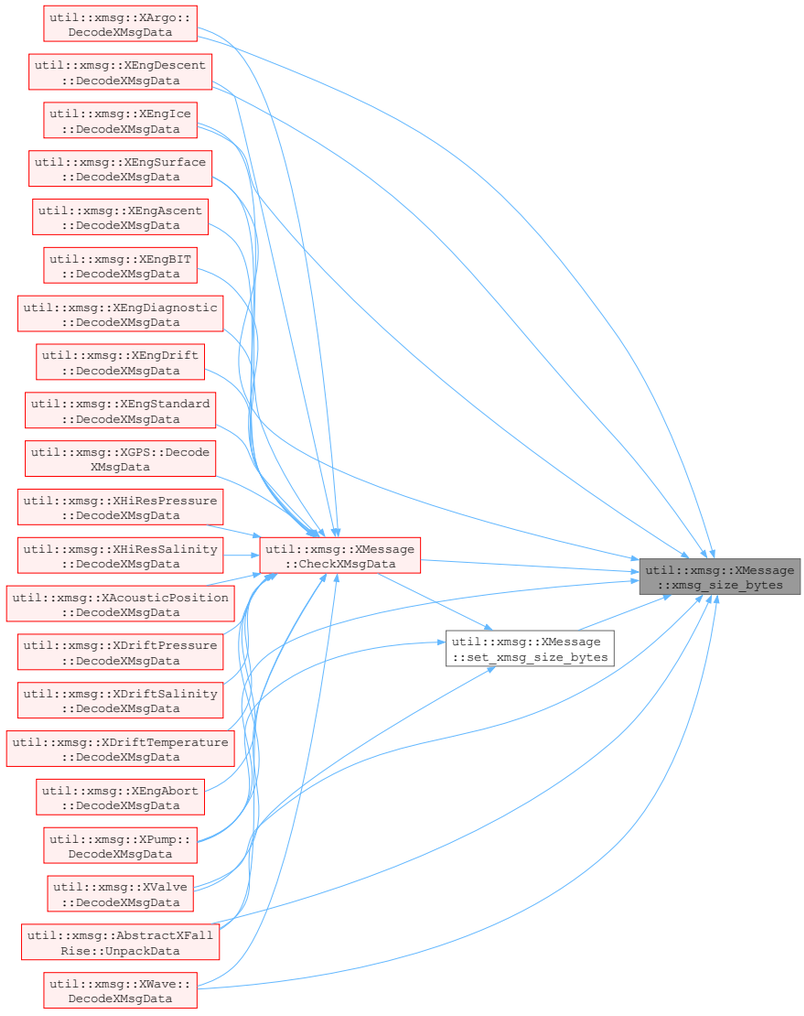
Returns

The size of the X-Message, in bytes.

References util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgI
util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(),
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), util::xmsg::XMessage::set_xmsg_size_bytes(),
and util::xmsg::AbstractXFallRise::UnpackData().

Here is the caller graph for this function:



10.76.4 Member Data Documentation

10.76.4.1 alamo_bit2_msg_

```
shared_ptr<XAlamoBit2> util::xmsg::XEngBIT::alamo_bit2_msg_  [private], [inherited]
```

A shared pointer to the ALALMO Built-in-Test (BIT) message.

Note

This member is only used for ALAMO-type instruments starting with firmware version 10.x.

Referenced by util::xmsg::XEngBIT::DecodeXMsgData(), util::xmsg::XEngBIT::GenerateListingString(), and util::xmsg::XEngBIT::set_alam

10.76.4.2 alamo_bit_msg_

```
shared_ptr<XAlamoBit> util::xmsg::XEngBIT::alamo_bit_msg_  [private], [inherited]
```

A shared pointer to the ALALMO Built-in-Test (BIT) message.

Note

This member is only used for ALAMO-type instruments prior to firmware version 10.x.

Referenced by util::xmsg::XEngBIT::DecodeXMsgData(), util::xmsg::XEngBIT::GenerateListingString(), and util::xmsg::XEngBIT::set_alam

10.76.4.3 id_dive_

```
int16_t util::xmsg::XMessage::id_dive_  [private], [inherited]
```

The dive or cycle number of this message.

Referenced by util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::set_id_dive().

10.76.4.4 id_instrument_

```
uint16_t util::xmsg::XMessage::id_instrument_  [private], [inherited]
```

The unique WHOI identifier of this instrument.

Referenced by util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::set_id_instrument().

10.76.4.5 id_xmsg_type_

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type_  [private], [inherited]
```

The X-message type identifier.

Referenced by util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::set_id_xmsg_type().

10.76.4.6 xmsg_size_bytes_

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes_  [private], [inherited]
```

The size of the X message, in bytes.

Referenced by util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes().

The documentation for this class was generated from the following file:

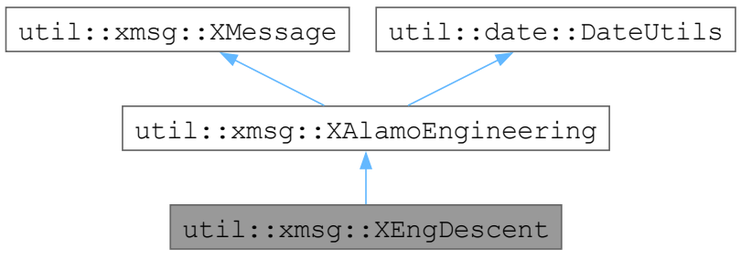
- data/xmsg/XEngBITPass.h

10.77 util::xmsg::XEngDescent Class Reference

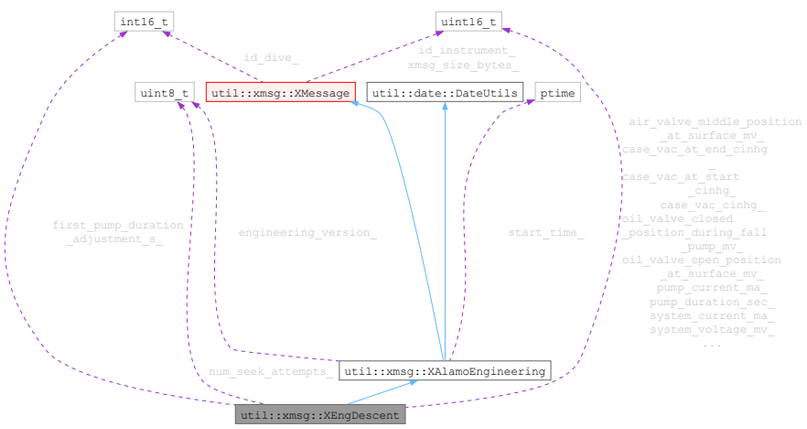
Class representing an descend state engineering X-Message.

`#include <XEngDescent.h>`

Inheritance diagram for util::xmsg::XEngDescent:



Collaboration diagram for util::xmsg::XEngDescent:



Public Member Functions

- XEngDescent (int16_t dive, uint16_t whoi, const vector< uint8_t > &data)

  The parameterized constructor.
- ~XEngDescent ()

  The destructor.
- uint16_t **air_valve_middle_position_at_surface_mv** () const
- template<typename T = float>
  T **air_valve_middle_position_at_surface_v** () const
- uint16_t **case_vac_at_end_cinhg** () const
- template<typename T = float>
  T **case_vac_at_end_inhg** () const
- uint16_t **case_vac_at_start_cinhg** () const
- template<typename T = float>
  T **case_vac_at_start_inhg** () const
- uint16_t **case_vac_cinhg** () const
- template<typename T = float>
  T case_vac_inhg () const

  Get the measured pressure case vacuum, in inches of mercury.
- const uint8_t * CheckXMsgData (const vector< uint8_t > &data, int id, int minBytes, bool parseSize=true)

  Check the X-message for the correct ID, minimum size, and message terminator.
- void DecodeXMsgData (const vector< uint8_t > &data)

  Decode the descent state engineering X-Message from the binary data.
- uint8_t engineering_version () const

  Get the engineering message version identifier.

- float engineering_version_as_float () const

  Get the engineering message verion identifier as a floating point number.
- int16_t **first_pump_duration_adjustment_s** () const
- const string GenerateListingString () const

  Get the listing entry string for this descent engineering data X-Message.
- int16_t id_dive () const

  Get the dive or cycle number of this X-Message.
- uint16_t id_instrument () const

  Get the unique instrument identifier of this X-Message.
- XMsgTypeId_t id_xmsg_type () const

  Get the enumerated X-Message type identifier.
- uint8_t **num_seek_attempts** () const
- uint16_t **oil_valve_closed_position_during_fall_pump_mv** () const
- template<typename T = float>
  T **oil_valve_closed_position_during_fall_pump_v** () const
- uint16_t **oil_valve_open_position_at_surface_mv** () const
- template<typename T = float>
  T **oil_valve_open_position_at_surface_v** () const
- template<typename T = float>
  T pump_current_a () const

  Get the measured pump current, in amperes.
- uint16_t **pump_current_ma** () const
- uint16_t **pump_duration_sec** () const
- const ptime & start_time () const

  Get the engineering message state start time.
- template<typename T = float>
  T system_current_a () const

  Get the measured system current, in amperes.
- uint16_t **system_current_ma** () const
- uint16_t **system_voltage_mv** () const
- template<typename T = float>
  T system_voltage_v () const

  Get the measured system voltage, in volts.
- uint16_t xmsg_size_bytes () const

  Get the size of the X-Message, in bytes.

Static Public Member Functions

- static string GenerateCurrentUtcTimeIso8601String ()

  Generate an ISO8601 format date/time string representation of the current time, in UTC.
- static string GenerateIso8601String (const ptime &p, bool extended=true)

  Generate an ISO8601 format date/time string representation of a POSIX time object.
- static double GenerateJulianDay (const ptime &dt, const ptime &ref_dt)

  Convert a date/time to fractional days since a reference date/time.
- static ptime GetCurrentUtcTime ()

  Generate a POSIX time object representing the current date and time (rounded to the nearest second).

Protected Member Functions

- void **set_air_valve_middle_position_at_surface_mv** (uint16_t air_valve_middle_position_at_surface_mv)
- void **set_case_vac_at_end_cinhg** (uint16_t case_vac_at_end_cinhg)
- void **set_case_vac_at_start_cinhg** (uint16_t case_vac_at_start_cinhg)
- void **set_case_vac_cinhg** (uint16_t case_vac_cinhg)
- void set_engineering_version (uint8_t engineering_version)

  Set the engineering message version identifier.
- void **set_first_pump_duration_adjustment_s** (int16_t first_pump_duration_adjustment_s)
- void set_id_dive (int16_t id_dive)

  Set the dive or cycle number of this message.
- void set_id_instrument (uint16_t id_instrument)

  Set the unique instrument identifier.
- void set_id_xmsg_type (XMsgTypeId_t id_xmsg_type)

  Set the X-Message type identifier.
- void **set_num_seek_attempts** (uint8_t num_seek_attempts)

- void **set_oil_valve_closed_position_during_fall_pump_mv** (uint16_t oil_valve_closed_position_during_fall↩ _pump_mv)
- void **set_oil_valve_open_position_at_surface_mv** (uint16_t oil_valve_open_position_at_surface_mv)
- void **set_pump_current_ma** (uint16_t pump_current_ma)
- void **set_pump_duration_sec** (uint16_t pump_duration_sec)
- void set_start_time (const ptime &start_time)

  Set the engineering message state start time.
- void **set_system_current_ma** (uint16_t system_current_ma)
- void **set_system_voltage_mv** (uint16_t system_voltage_mv)
- void set_xmsg_size_bytes (uint16_t xmsg_size_bytes)

  Set the size of the X-Message, in bytes.

Private Attributes

- uint16_t air_valve_middle_position_at_surface_mv_

  The air valve middle position at surface, in .001 V.
- uint16_t case_vac_at_end_cinhg_

  The measured internal pressure case vacuum, in 0.01 inHg, at the end of the descent-to-drift/park-pressure phase of the dive cycle.
- uint16_t case_vac_at_start_cinhg_

  The measured internal pressure case vacuum, in 0.01 inHg, at the start of the descent-to-drift/park-pressure phase of the dive cycle.
- uint16_t case_vac_cinhg_

  The measured internal pressure case vacuum, in 0.01 inHg.
- uint8_t engineering_version_

  The engineering message version identifier.
- int16_t first_pump_duration_adjustment_s_

  The the first-pump run time adjustment, in seconds.
- int16_t id_dive_

  The dive or cycle number of this message.
- uint16_t id_instrument_

  The unique WHOI identifier of this instrument.
- XMsgTypeId_t id_xmsg_type_

  The X-message type identifier.
- uint8_t num_seek_attempts_

  The number of seek attempts.
- uint16_t oil_valve_closed_position_during_fall_pump_mv_

  The oil valve closed position during fall pump, in .001 V.
- uint16_t oil_valve_open_position_at_surface_mv_

  The oil valve open position at surface, in .001 V.
- uint16_t pump_current_ma_

  The measured pump current, in milliamps.
- uint16_t pump_duration_sec_

  The cumulative pump run time for the descent state of the dive cycle, in seconds.
- ptime start_time_

  The engineering message state start time.
- uint16_t system_current_ma_

  The measured system current, in milliamps.
- uint16_t system_voltage_mv_

  The measured system current, in millivolts.
- uint16_t xmsg_size_bytes_

  The size of the X message, in bytes.

10.77.1 Detailed Description

Class representing an descend state engineering X-Message.

Note

This X-Message is only used by ALAMO-type instruments.

This message type was added in X-Message version 2.1.

10.77.2 Constructor & Destructor Documentation

10.77.2.1 XEngDescent()

```
util::xmsg::XEngDescent::XEngDescent (
            int16_t dive,
            uint16_t whoi,
            const vector< uint8_t > & data)  [inline]
```

The parameterized constructor.

Parameters

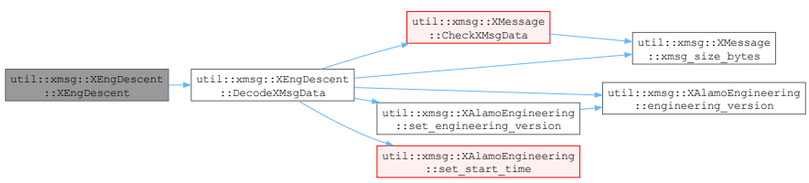
in	*dive*	The dive or cycle number of this message.
in	*whoi*	The unique WHOI identifier.
in	*data*	The binary X-message data to decode.

Exceptions

util::log::MessageException	if an decode error occurs.

References DecodeXMsgData().

Here is the call graph for this function:



10.77.2.2 ~XEngDescent()

```
util::xmsg::XEngDescent::~XEngDescent ()  [inline]
```

The destructor.

10.77.3 Member Function Documentation

10.77.3.1 case_vac_inhg()

```
template<typename T = float>
T util::xmsg::XEngDescent::case_vac_inhg () const  [inline]
```

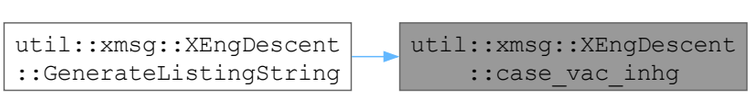
Get the measured pressure case vacuum, in inches of mercury.

Returns

The measured pressure case vacuum, in inches of mercury.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.77.3.2 CheckXMsgData()

```
const uint8_t * util::xmsg::XMessage::CheckXMsgData (
            const vector< uint8_t > & data,
            int id,
            int minBytes,
            bool parseSize = true)  [inline], [inherited]
```

Check the X-message for the correct ID, minimum size, and message terminator.

Parameters

in	*data*	The X-message binary data to check.
in	*id*	The expected ID of the X-message. If this field is less that 0, then the ID check will be skipped.
in	*minBytes*	The minimum number of bytes expected in a valid message.
in	*parseSize*	If TRUE, then the message size field will be parsed from the X-Message. Otherwise, the size field will not be parsed.

Exceptions

util::log::MessageException	if • the message does not contain the expected ID, • the message does not contain the minimum number of expected bytes, • the message does not contain a terminator (';') in the correct location, or, • any other error occurs.

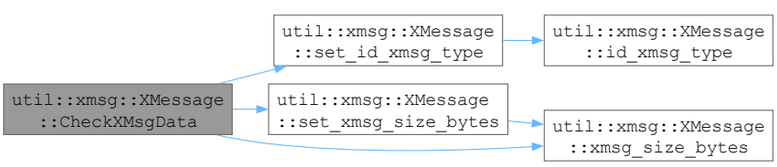
Returns

A pointer to the underlying data buffer of data.

References CATCH_BLOCK, EXC_THROW, lwrn, util::xmsg::XMessage::set_id_xmsg_type(), util::xmsg::XMessage::set_xmsg_size_byte
u16, and util::xmsg::XMessage::xmsg_size_bytes().

Referenced by util::xmsg::XAcousticPosition::DecodeXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XDriftPressure::Dec
util::xmsg::XDriftSalinity::DecodeXMsgData(), util::xmsg::XDriftTemperature::DecodeXMsgData(), util::xmsg::XEngAbort::DecodeXMsgDa
util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngBIT::DecodeXMsgData(), DecodeXMsgData(), util::xmsg::XEngDiagnostic::
util::xmsg::XEngDrift::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngStandard::DecodeXMsgData(),
util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XHiResPressure::DecodeXMsgData(),
util::xmsg::XHiResSalinity::DecodeXMsgData(), util::xmsg::XHiResTemperature::DecodeXMsgData(), util::xmsg::XMissionCmdEcho::Dec
util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XProfilePressure::DecodeXMsgData(), util::xmsg::XProfileSalinity::Dec
util::xmsg::XProfileTemperature::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(), util::xmsg::XValve::DecodeXMsgData(),
util::xmsg::XWave::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the call graph for this function:



10.77.3.3 DecodeXMsgData()

```
void util::xmsg::XEngDescent::DecodeXMsgData (
            const vector< uint8_t > & data)  [inline], [virtual]
```

Decode the descent state engineering X-Message from the binary data.

Parameters

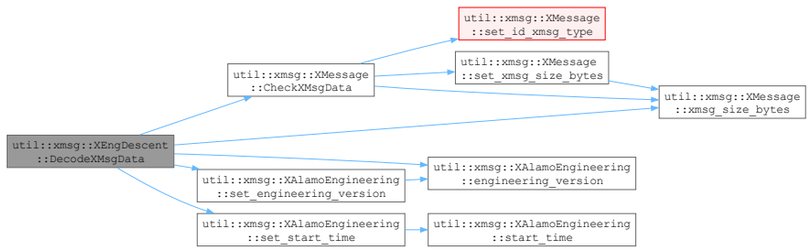
in	*data*	The binary message to decode.

Implements util::xmsg::XMessage.

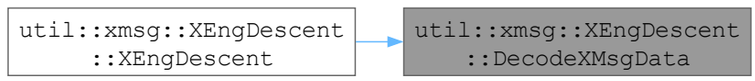
References CATCH_BLOCK, util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XAlamoEngineering::engineering_version(), EXC_THROW, i16, s2000, util::xmsg::XAlamoEngineering::set_engineering_version(), util::xmsg::XAlamoEngineering::set_start_time(), u08, u16, and util::xmsg::XMessage::xmsg_size_bytes().

Referenced by XEngDescent().

Here is the call graph for this function:



Here is the caller graph for this function:



10.77.3.4 engineering_version()

```
uint8_t util::xmsg::XAlamoEngineering::engineering_version () const  [inline], [inherited]
```

Get the engineering message version identifier.

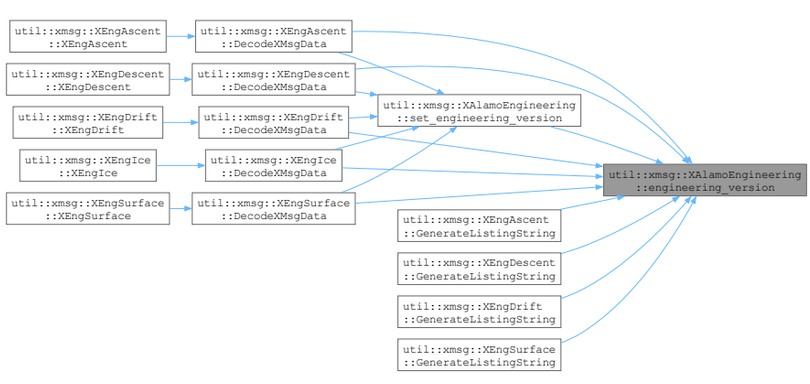
Returns

The engineering message version identifier.

References util::xmsg::XAlamoEngineering::engineering_version_.

Referenced by util::xmsg::XEngAscent::DecodeXMsgData(), DecodeXMsgData(), util::xmsg::XEngDrift::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XEngAscent::GenerateListingString(), GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), util::xmsg::XEngSurface::GenerateListingString(), and util::xmsg::XAlamoEngineering::set_engineering_version().

Here is the caller graph for this function:



10.77.3.5 engineering_version_as_float()

```
float util::xmsg::XAlamoEngineering::engineering_version_as_float () const  [inline], [inherited]
```

Get the engineering message verion identifier as a floating point number.

Returns

The engineering message verion identifier as a floating point number.

References util::xmsg::XAlamoEngineering::engineering_version_.

10.77.3.6 GenerateCurrentUtcTimeIso8601String()

```
static string util::date::DateUtils::GenerateCurrentUtcTimeIso8601String ()  [inline], [static],
[inherited]
```

Generate an ISO8601 format date/time string representation of the current time, in UTC.

Note

All date/time objects use the UTC time-zone.

Returns

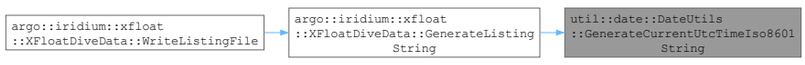
An ISO8601 format date/time string representation of the current time.

References util::date::DateUtils::GenerateIso8601String(), and util::date::DateUtils::GetCurrentUtcTime().

Referenced by argo::iridium::xfloat::XFloatDiveData::GenerateListingString().

Here is the call graph for this function:

Here is the caller graph for this function:



10.77.3.7 GenerateIso8601String()

```
static string util::date::DateUtils::GenerateIso8601String (
            const ptime & p,
            bool extended = true)  [inline], [static], [inherited]
```

Generate an ISO8601 format date/time string representation of a POSIX time object.

Note

All date/time objects use the UTC time-zone.

Parameters

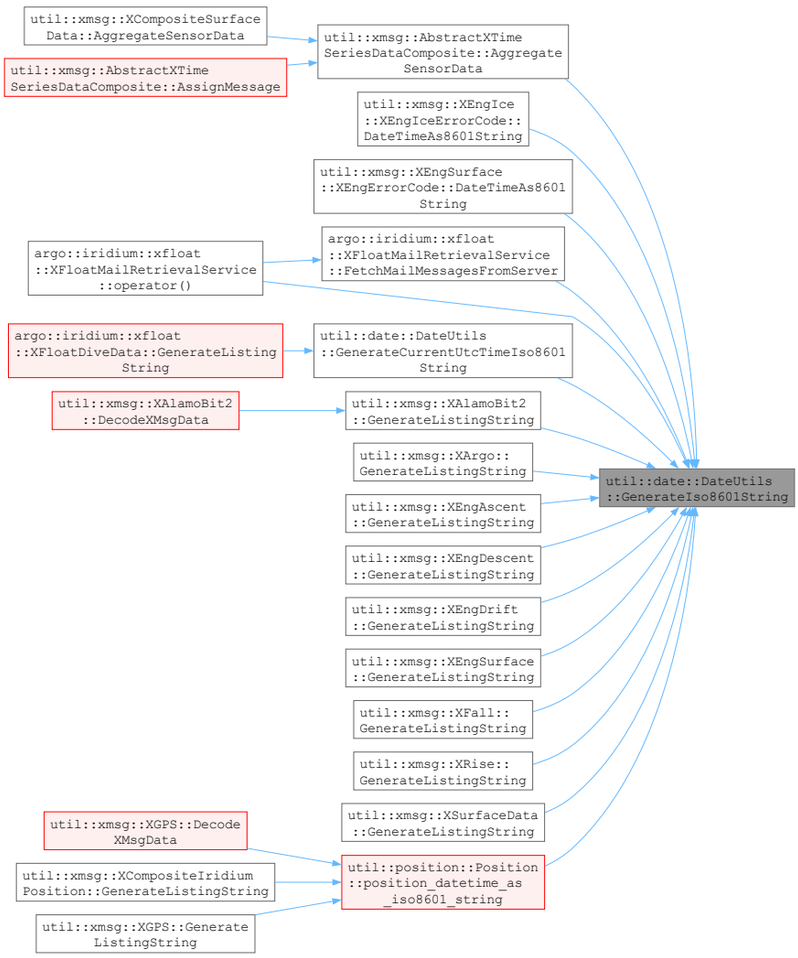
in	*p*	POSIX time from which to generate string.
in	*extended*	If TRUE, use the ISO 8601 extended format (YYYY-MM-DDThh:mm:ssZ); otherwise, use the basic format (YYYYMMDDThhmmss).

Returns

An ISO8601 format date/time string representation of the POSIX time object. If p is equal to boost::posix_↩ time::not_a_date_time, then the string "0000-00-00T00:00:00Z" (extended) or "00000000000000" (basic) will be returned.

Referenced by util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::AggregateSensorData(), util::xmsg::XEngIce::XEngIce util::xmsg::XEngSurface::XEngErrorCode::DateTimeAs8601String(), argo::iridium::xfloat::XFloatMailRetrievalService::FetchMailMessages util::date::DateUtils::GenerateCurrentUtcTimeIso8601String(), util::xmsg::XAlamoBit2::GenerateListingString(), util::xmsg::XArgo::Genera util::xmsg::XEngAscent::GenerateListingString(), GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(), argo::iridium::xfloat::XFloatMailRetrievalService::operator()(), and util::position::Position::position_datetime_as_iso8601_string().

Here is the caller graph for this function:



10.77.3.8 GenerateJulianDay()

```
static double util::date::DateUtils::GenerateJulianDay (
            const ptime & dt,
            const ptime & ref_dt)  [inline], [static], [inherited]
```

Convert a date/time to fractional days since a reference date/time.

Parameters

in	*dt*	The date/time to convert.
in	*ref↩ _dt*	The reference date/time (day 0).

Exceptions

util::log::MessageException	if either date/time is not valid.

Returns

The number of days (with millisecond resolution) from `ref_dt` to `dt`.

References CATCH_BLOCK, and EXC_THROW.

10.77.3.9 GenerateListingString()

```
const string util::xmsg::XEngDescent::GenerateListingString () const  [inline]
```

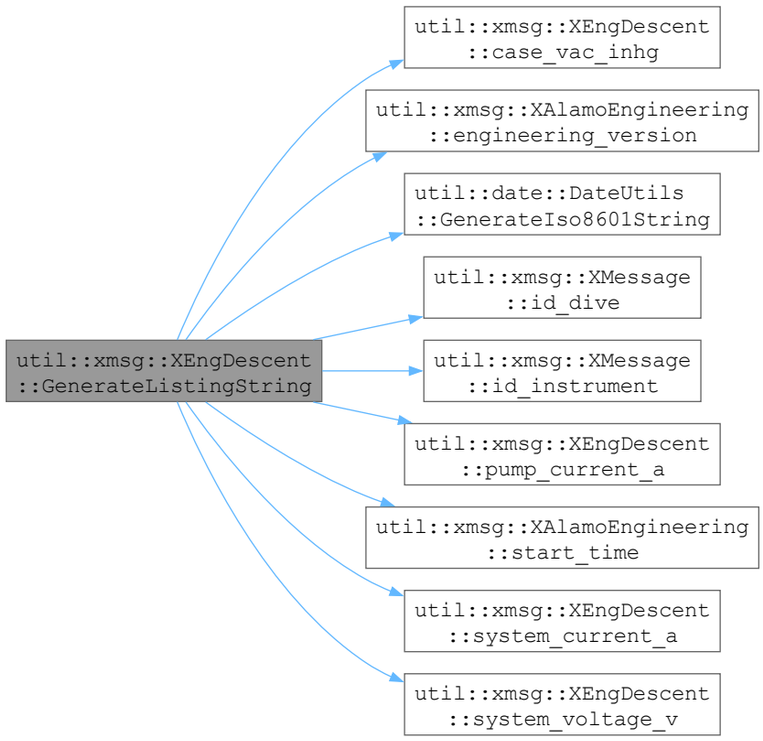
Get the listing entry string for this descent engineering data X-Message.

Returns

The listing entry string for this descent engineering data X-Message

References case_vac_inhg(), CATCH_BLOCK, util::xmsg::XAlamoEngineering::engineering_version(), EXC_THROW, util::date::DateUtils::GenerateIso8601String(), util::xmsg::XMessage::id_dive(), util::xmsg::XMessage::id_instrument(), pump_current_a(), util::xmsg::XAlamoEngineering::start_time(), system_current_a(), and system_voltage_v().

Here is the call graph for this function:



10.77.3.10 GetCurrentUtcTime()

```
static ptime util::date::DateUtils::GetCurrentUtcTime ()  [inline], [static], [inherited]
```

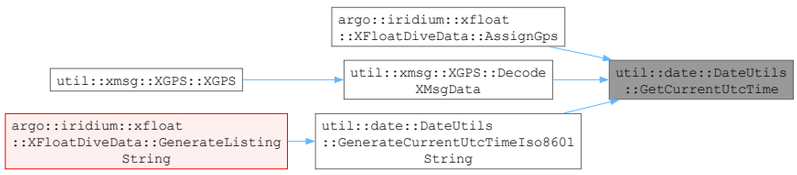
Generate a POSIX time object representing the current date and time (rounded to the nearest second).

Returns

A POSIX time object representing the current date and time (rounded to the nearest second).

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignGps(), util::xmsg::XGPS::DecodeXMsgData(), and util::date::DateUtils::GenerateCurrentUtcTimeIso8601String().

Here is the caller graph for this function:



10.77.3.11 id_dive()

```
int16_t util::xmsg::XMessage::id_dive () const  [inline], [inherited]
```

Get the dive or cycle number of this X-Message.

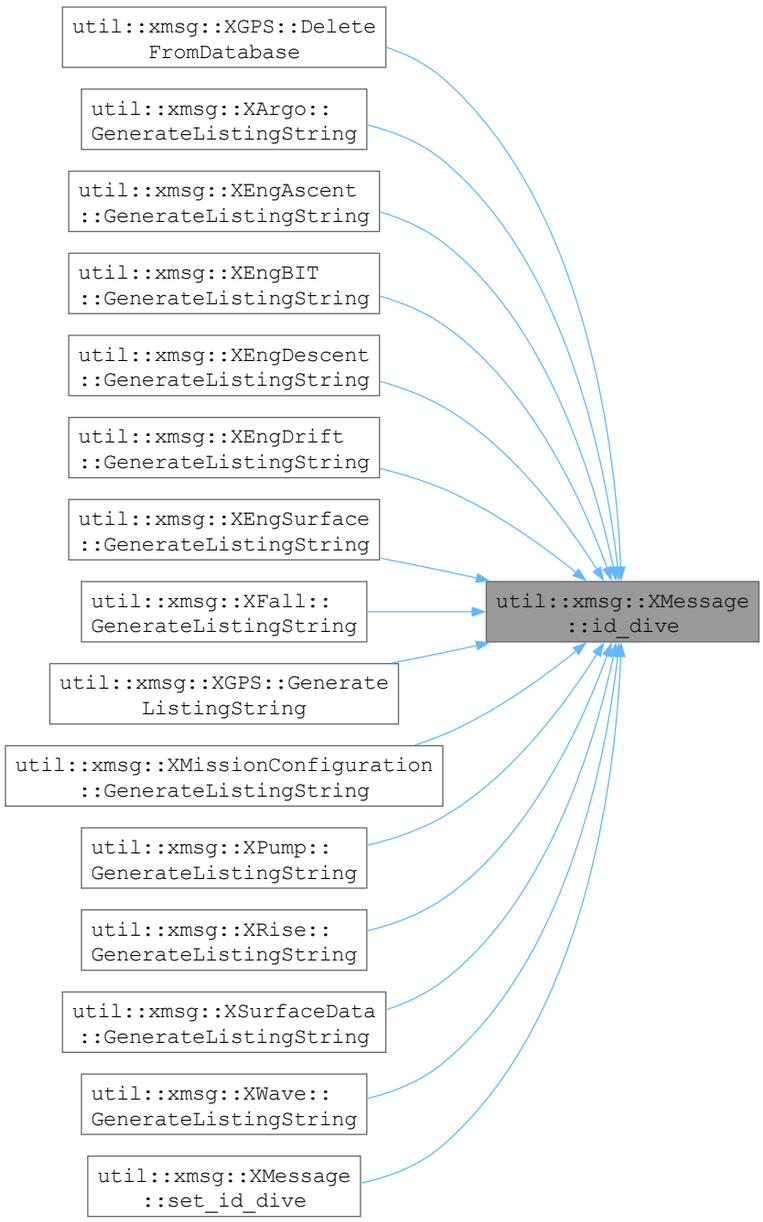
Returns

The dive or cycle number of this X-Message.

References util::xmsg::XMessage::id_dive_.

Referenced by util::xmsg::XGPS::DeleteFromDatabase(), util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateLis
util::xmsg::XEngBIT::GenerateListingString(), GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(),
util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(),
util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing
util::xmsg::XSurfaceData::GenerateListingString(), util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_dive().

Here is the caller graph for this function:



10.77.3.12 id_instrument()

```
uint16_t util::xmsg::XMessage::id_instrument () const  [inline], [inherited]
```

Get the unique instrument identifier of this X-Message.

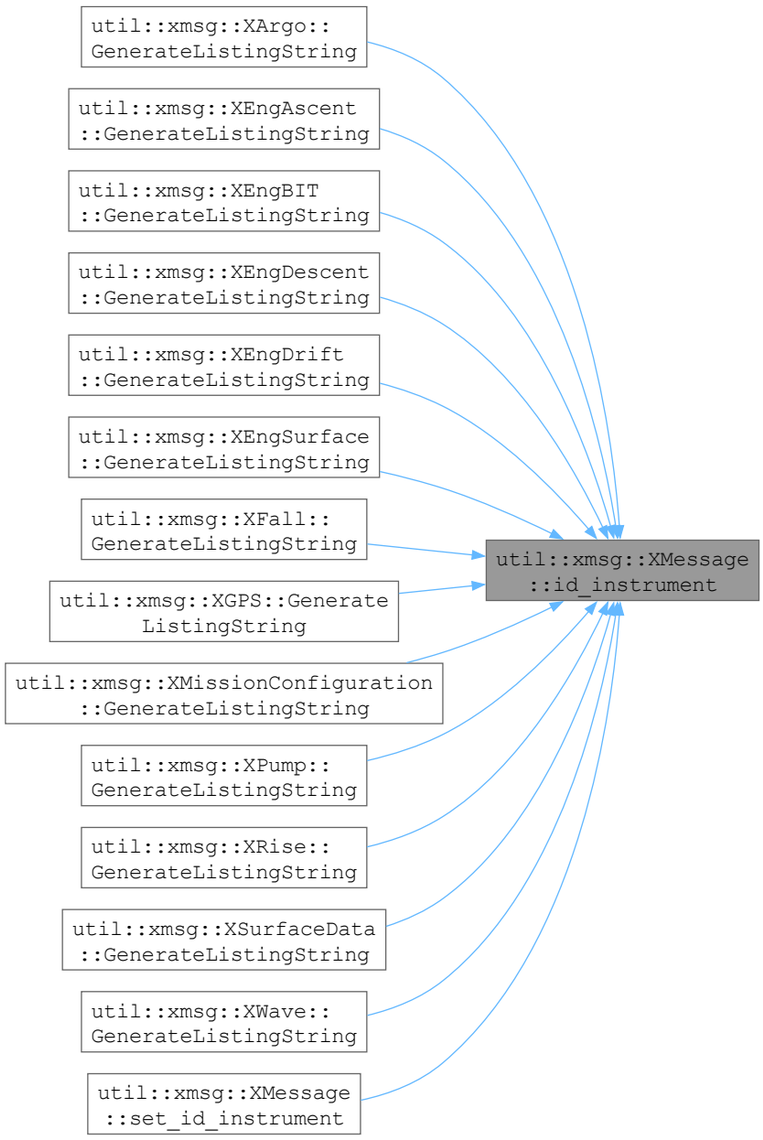
Returns

The unique instrument identifier of this X-Message.

References util::xmsg::XMessage::id_instrument_.

Referenced by util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngBIT::Generate GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingS util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(), util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_instrument().

Here is the caller graph for this function:



10.77.3.13 id_xmsg_type()

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type () const  [inline], [inherited]
```

Get the enumerated X-Message type identifier.

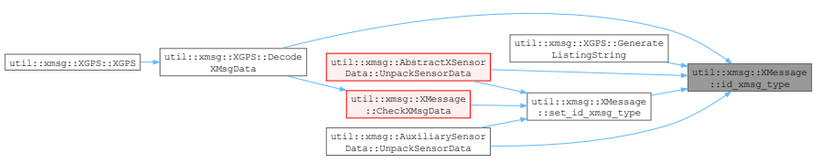
Returns

The enumerated X-Message type identifier.

References util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMessage::set_id_xmsg_typ
util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the caller graph for this function:



10.77.3.14 pump_current_a()

```
template<typename T = float>
T util::xmsg::XEngDescent::pump_current_a () const  [inline]
```

Get the measured pump current, in amperes.

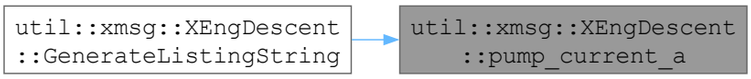
Returns

The measured pump current, in amperes.

References pump_current_ma_.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.77.3.15 set_engineering_version()

```
void util::xmsg::XAlamoEngineering::set_engineering_version (
            uint8_t engineering_version)  [inline], [protected], [inherited]
```

Set the engineering message version identifier.

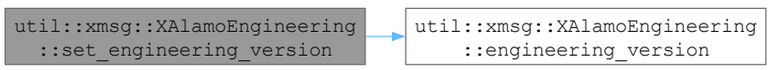
Parameters

in	*engineering_version*	The engineering message version identifier.

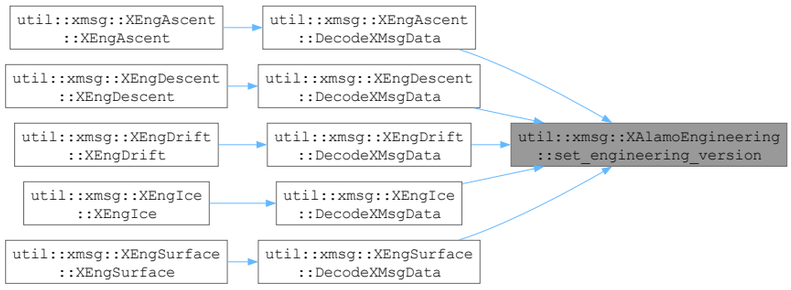
References util::xmsg::XAlamoEngineering::engineering_version(), and util::xmsg::XAlamoEngineering::engineering_version_.

Referenced by util::xmsg::XEngAscent::DecodeXMsgData(), DecodeXMsgData(), util::xmsg::XEngDrift::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(), and util::xmsg::XEngSurface::DecodeXMsgData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.77.3.16 set_id_dive()

```
void util::xmsg::XMessage::set_id_dive (
            int16_t id_dive)  [inline], [protected], [inherited]
```

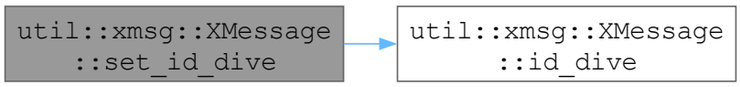
Set the dive or cycle number of this message.

Parameters

in	*id_dive*	The dive or cycle number of this message to set.

References util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::id_dive_.

Here is the call graph for this function:



10.77.3.17 set_id_instrument()

```
void util::xmsg::XMessage::set_id_instrument (
            uint16_t id_instrument)  [inline], [protected], [inherited]
```

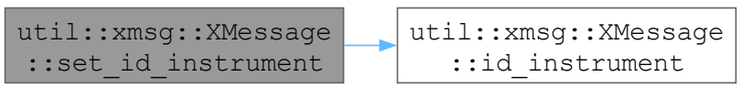
Set the unique instrument identifier.

Parameters

in	*id_instrument*	The unique instrument identifier to set.

References util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::id_instrument_.

Here is the call graph for this function:



10.77.3.18 set_id_xmsg_type()

```
void util::xmsg::XMessage::set_id_xmsg_type (
            XMsgTypeId_t id_xmsg_type)  [inline], [protected], [inherited]
```

Set the X-Message type identifier.

Parameters

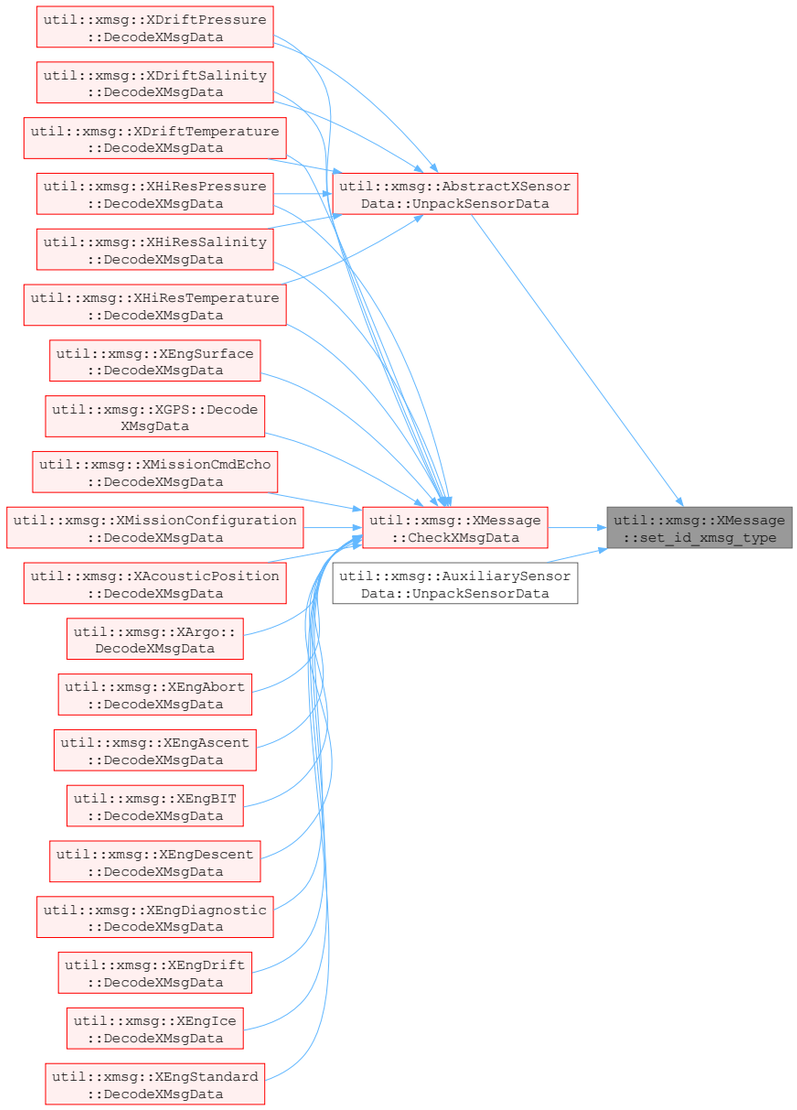
in	*id_xmsg_type*	The X-Message type identifier to set.

References util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:

Here is the caller graph for this function:



10.77.3.19 set_start_time()

```
void util::xmsg::XAlamoEngineering::set_start_time (
            const ptime & start_time)  [inline], [protected], [inherited]
```

Set the engineering message state start time.

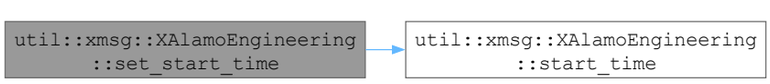
Parameters

in	*start_time*	The engineering message state start time.

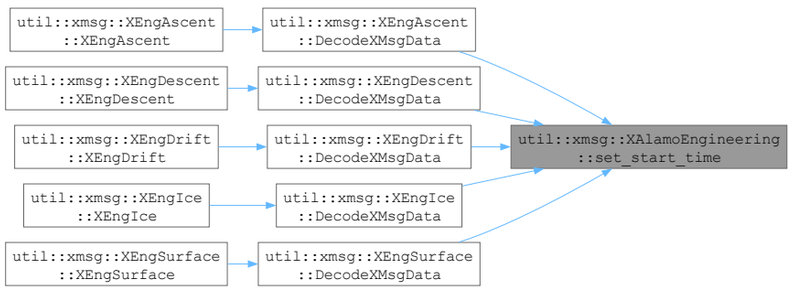
References util::xmsg::XAlamoEngineering::start_time(), and util::xmsg::XAlamoEngineering::start_time_.

Referenced by util::xmsg::XEngAscent::DecodeXMsgData(), DecodeXMsgData(), util::xmsg::XEngDrift::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(), and util::xmsg::XEngSurface::DecodeXMsgData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.77.3.20 set_xmsg_size_bytes()

```
void util::xmsg::XMessage::set_xmsg_size_bytes (
            uint16_t xmsg_size_bytes)  [inline], [protected], [inherited]
```

Set the size of the X-Message, in bytes.

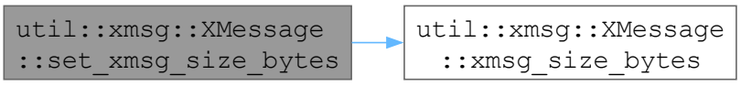
Parameters

in	*xmsg_size_bytes*	The size of the X-Message to set, in bytes.

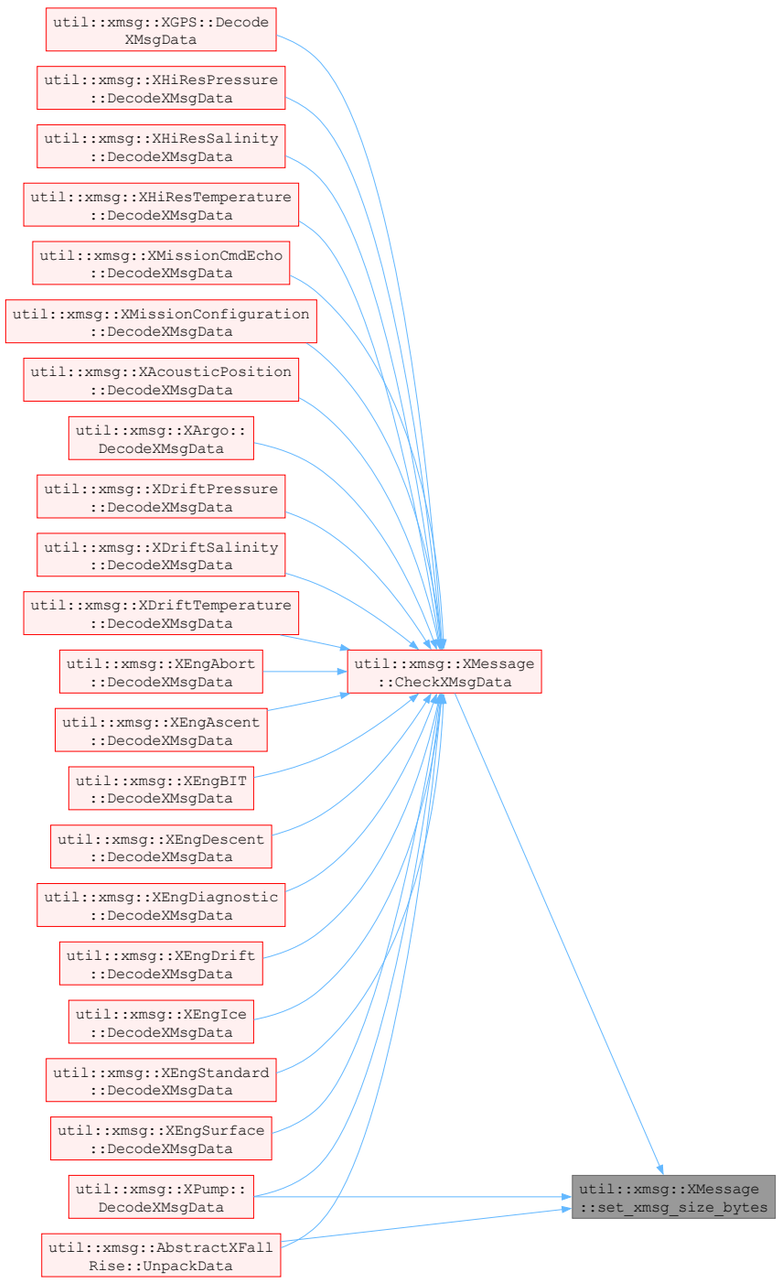
References util::xmsg::XMessage::xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XPump::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::Unpa

Here is the call graph for this function:



Here is the caller graph for this function:



10.77.3.21 start_time()

```
const ptime & util::xmsg::XAlamoEngineering::start_time () const  [inline], [inherited]
```

Get the engineering message state start time.

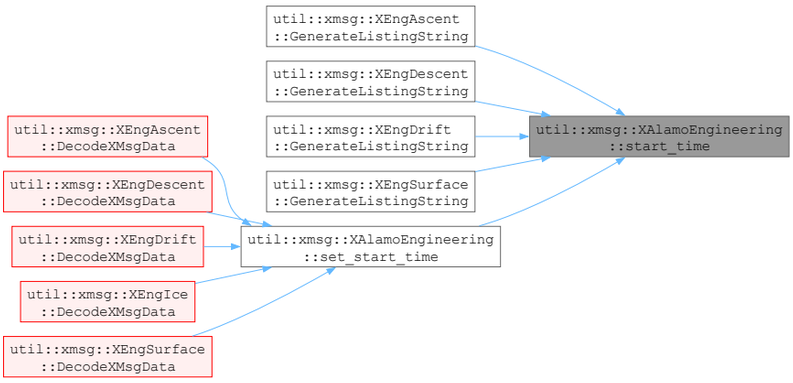
Returns

The engineering message state start time.

References util::xmsg::XAlamoEngineering::start_time_.

Referenced by util::xmsg::XEngAscent::GenerateListingString(), GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), util::xmsg::XEngSurface::GenerateListingString(), and util::xmsg::XAlamoEngineering::set_start_time().

Here is the caller graph for this function:



10.77.3.22 system_current_a()

```
template<typename T = float>
T util::xmsg::XEngDescent::system_current_a () const  [inline]
```

Get the measured system current, in amperes.

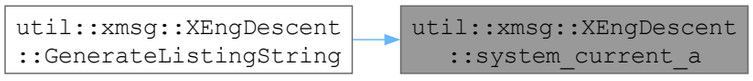
Returns

The measured system current, in amperes.

References system_current_ma_.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.77.3.23 system_voltage_v()

```
template<typename T = float>
T util::xmsg::XEngDescent::system_voltage_v () const  [inline]
```

Get the measured system voltage, in volts.

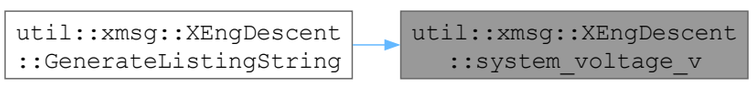
Returns

The measured system voltage, in volts.

References system_voltage_mv_.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.77.3.24 xmsg_size_bytes()

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes () const  [inline], [inherited]
```

Get the size of the X-Message, in bytes.

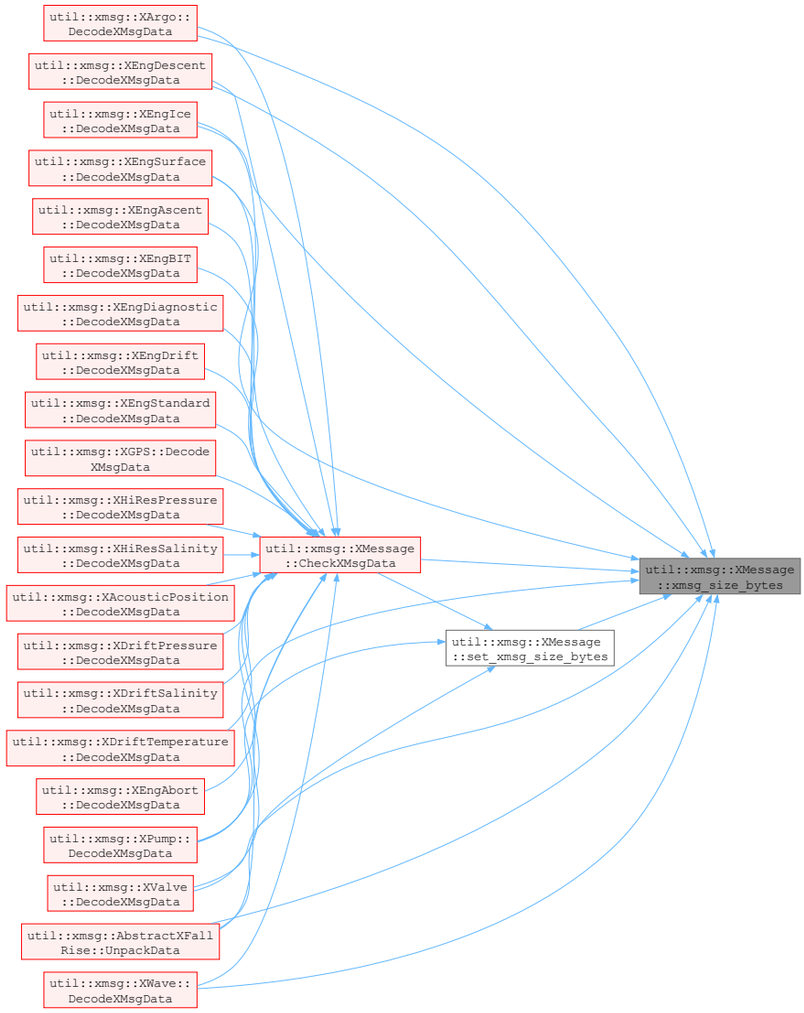
Returns

The size of the X-Message, in bytes.

References util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(), util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the caller graph for this function:



10.77.4 Member Data Documentation

10.77.4.1 air_valve_middle_position_at_surface_mv_

```
uint16_t util::xmsg::XEngDescent::air_valve_middle_position_at_surface_mv_  [private]
```

The air valve middle position at surface, in .001 V.

Note

This field was added in version 0x11.

10.77.4.2 case_vac_at_end_cinhg_

```
uint16_t util::xmsg::XEngDescent::case_vac_at_end_cinhg_  [private]
```

The measured internal pressure case vacuum, in 0.01 inHg, at the end of the descent-to-drift/park-pressure phase of the dive cycle.

Note

This field was added in version 0x11.

10.77.4.3 case_vac_at_start_cinhg_

```
uint16_t util::xmsg::XEngDescent::case_vac_at_start_cinhg_  [private]
```

The measured internal pressure case vacuum, in 0.01 inHg, at the start of the descent-to-drift/park-pressure phase of the dive cycle.

Note

This field was added in version 0x11.

10.77.4.4 case_vac_cinhg_

```
uint16_t util::xmsg::XEngDescent::case_vac_cinhg_  [private]
```

The measured internal pressure case vacuum, in 0.01 inHg.

Note

This field is only used for message version 0x10. It was removed in version 0x11.

10.77.4.5 engineering_version_

```
uint8_t util::xmsg::XAlamoEngineering::engineering_version_  [private], [inherited]
```

The engineering message version identifier.

Referenced by util::xmsg::XAlamoEngineering::engineering_version(), util::xmsg::XAlamoEngineering::engineering_version_as_float(), and util::xmsg::XAlamoEngineering::set_engineering_version().

10.77.4.6 first_pump_duration_adjustment_s_

```
int16_t util::xmsg::XEngDescent::first_pump_duration_adjustment_s_  [private]
```

The the first-pump run time adjustment, in seconds.

10.77.4.7 id_dive_

```
int16_t util::xmsg::XMessage::id_dive_  [private], [inherited]
```

The dive or cycle number of this message.

Referenced by util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::set_id_dive().

10.77.4.8 id_instrument_

```
uint16_t util::xmsg::XMessage::id_instrument_  [private], [inherited]
```

The unique WHOI identifier of this instrument.

Referenced by util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::set_id_instrument().

10.77.4.9 id_xmsg_type_

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type_  [private], [inherited]
```

The X-message type identifier.

Referenced by util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::set_id_xmsg_type().

10.77.4.10 num_seek_attempts_

```
uint8_t util::xmsg::XEngDescent::num_seek_attempts_  [private]
```

The number of seek attempts.

10.77.4.11 oil_valve_closed_position_during_fall_pump_mv_

```
uint16_t util::xmsg::XEngDescent::oil_valve_closed_position_during_fall_pump_mv_  [private]
```

The oil valve closed position during fall pump, in .001 V.

Todo Clarify the description of the "oil valve position during fall pump" field.

Note

This field was added in version 0x11.

10.77.4.12 oil_valve_open_position_at_surface_mv_

```
uint16_t util::xmsg::XEngDescent::oil_valve_open_position_at_surface_mv_  [private]
```

The oil valve open position at surface, in .001 V.

Note

This field was added in version 0x11.

10.77.4.13 pump_current_ma_

```
uint16_t util::xmsg::XEngDescent::pump_current_ma_  [private]
```

The measured pump current, in milliamps.

Note

This field is only used for message version 0x10. It was removed in version 0x11.

Referenced by pump_current_a().

10.77.4.14 pump_duration_sec_

```
uint16_t util::xmsg::XEngDescent::pump_duration_sec_  [private]
```

The cumulative pump run time for the descent state of the dive cycle, in seconds.

Note

This field is only used for message version 0x10. It was removed in version 0x11.

10.77.4.15 start_time_

```
ptime util::xmsg::XAlamoEngineering::start_time_  [private], [inherited]
```

The engineering message state start time.

Referenced by util::xmsg::XAlamoEngineering::set_start_time(), and util::xmsg::XAlamoEngineering::start_time().

10.77.4.16 system_current_ma_

```
uint16_t util::xmsg::XEngDescent::system_current_ma_  [private]
```

The measured system current, in milliamps.

Note

This field is only used for message version 0x10. It was removed in version 0x11.

Referenced by system_current_a().

10.77.4.17 system_voltage_mv_

```
uint16_t util::xmsg::XEngDescent::system_voltage_mv_  [private]
```

The measured system current, in millivolts.

Note

This field is only used for message version 0x10. It was removed in version 0x11.

Referenced by system_voltage_v().

10.77.4.18 xmsg_size_bytes_

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes_  [private], [inherited]
```

The size of the X message, in bytes.

Referenced by util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes().

The documentation for this class was generated from the following file:

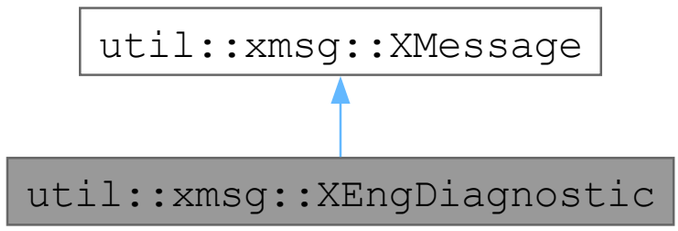
- data/xmsg/XEngDescent.h

10.78 util::xmsg::XEngDiagnostic Class Reference

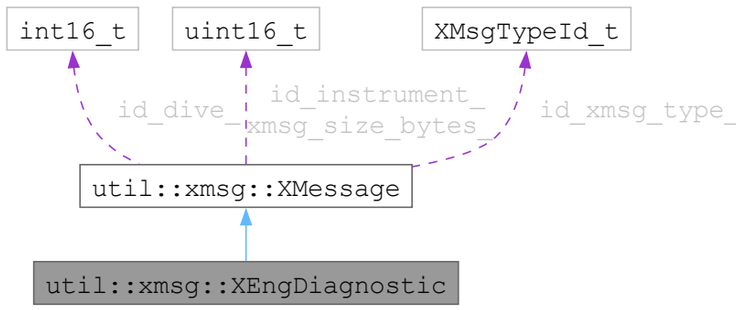
Class representing a diagnostic dive engineering message.

`#include <XEngDiagnostic.h>`

Inheritance diagram for util::xmsg::XEngDiagnostic:



Collaboration diagram for util::xmsg::XEngDiagnostic:



Public Member Functions

- XEngDiagnostic (int16_t dive, uint16_t whoi, const vector< uint8_t > &data)
  Parameterized constructor.
- ~XEngDiagnostic ()
  Destructor.
- const uint8_t * CheckXMsgData (const vector< uint8_t > &data, int id, int minBytes, bool parseSize=true)
  Check the X-message for the correct ID, minimum size, and message terminator.
- void DecodeXMsgData (const vector< uint8_t > &data)
  Decode the diagnostic dive engineering X-Message from the binary data.
- const string GenerateListingString () const
  Generate a listing string representation of the diagnostic dive engineering data message.
- int16_t id_dive () const
  Get the dive or cycle number of this X-Message.
- uint16_t id_instrument () const
  Get the unique instrument identifier of this X-Message.
- XMsgTypeId_t id_xmsg_type () const
  Get the enumerated X-Message type identifier.
- uint16_t xmsg_size_bytes () const
  Get the size of the X-Message, in bytes.

Protected Member Functions

- void set_id_dive (int16_t id_dive)

  Set the dive or cycle number of this message.
- void set_id_instrument (uint16_t id_instrument)

  Set the unique instrument identifier.
- void set_id_xmsg_type (XMsgTypeId_t id_xmsg_type)

  Set the X-Message type identifier.
- void set_xmsg_size_bytes (uint16_t xmsg_size_bytes)

  Set the size of the X-Message, in bytes.

Private Attributes

- int16_t id_dive_

  The dive or cycle number of this message.
- uint16_t id_instrument_

  The unique WHOI identifier of this instrument.
- XMsgTypeId_t id_xmsg_type_

  The X-message type identifier.
- uint16_t xmsg_size_bytes_

  The size of the X message, in bytes.

10.78.1 Detailed Description

Class representing a diagnostic dive engineering message.

10.78.2 Constructor & Destructor Documentation

10.78.2.1 XEngDiagnostic()

```
util::xmsg::XEngDiagnostic::XEngDiagnostic (
            int16_t dive,
            uint16_t whoi,
            const vector< uint8_t > & data)  [inline]
```

Parameterized constructor.

Parameters

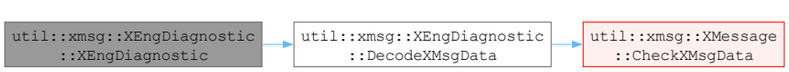
in	*dive*	The dive or cycle number of this message.
in	*whoi*	The unique WHOI identifier.
in	*data*	The binary X-message data to decode.

Exceptions

util::log::MessageException	if an decode error occurs.

References DecodeXMsgData().

Here is the call graph for this function:



10.78.2.2 ~XEngDiagnostic()

```
util::xmsg::XEngDiagnostic::~XEngDiagnostic ()  [inline]
```

Destructor.

10.78.3 Member Function Documentation

10.78.3.1 CheckXMsgData()

```
const uint8_t * util::xmsg::XMessage::CheckXMsgData (
            const vector< uint8_t > & data,
            int id,
            int minBytes,
            bool parseSize = true)  [inline], [inherited]
```

Check the X-message for the correct ID, minimum size, and message terminator.

Parameters

in	*data*	The X-message binary data to check.
in	*id*	The expected ID of the X-message. If this field is less that 0, then the ID check will be skipped.
in	*minBytes*	The minimum number of bytes expected in a valid message.
in	*parseSize*	If TRUE, then the message size field will be parsed from the X-Message. Otherwise, the size field will not be parsed.

Exceptions

util::log::MessageException	if • the message does not contain the expected ID, • the message does not contain the minimum number of expected bytes, • the message does not contain a terminator (';') in the correct location, or, • any other error occurs.

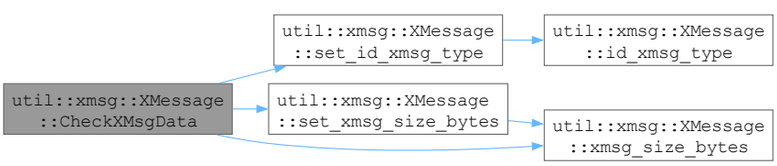
Returns

A pointer to the underlying data buffer of data.

References CATCH_BLOCK, EXC_THROW, lwrn, util::xmsg::XMessage::set_id_xmsg_type(), util::xmsg::XMessage::set_xmsg_size_byte
u16, and util::xmsg::XMessage::xmsg_size_bytes().

Referenced by util::xmsg::XAcousticPosition::DecodeXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XDriftPressure::Dec
util::xmsg::XDriftSalinity::DecodeXMsgData(), util::xmsg::XDriftTemperature::DecodeXMsgData(), util::xmsg::XEngAbort::DecodeXMsgDa
util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngBIT::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(),
DecodeXMsgData(), util::xmsg::XEngDrift::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngStandard::Dec
util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XHiResPressure::DecodeXMsgData(),
util::xmsg::XHiResSalinity::DecodeXMsgData(), util::xmsg::XHiResTemperature::DecodeXMsgData(), util::xmsg::XMissionCmdEcho::Dec
util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XProfilePressure::DecodeXMsgData(), util::xmsg::XProfileSalinity::Dec
util::xmsg::XProfileTemperature::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(), util::xmsg::XValve::DecodeXMsgData(),
util::xmsg::XWave::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the call graph for this function:



10.78.3.2 DecodeXMsgData()

```
void util::xmsg::XEngDiagnostic::DecodeXMsgData (
            const vector< uint8_t > & data)  [inline], [virtual]
```

Decode the diagnostic dive engineering X-Message from the binary data.

Parameters

in	*data*	The binary message to decode.

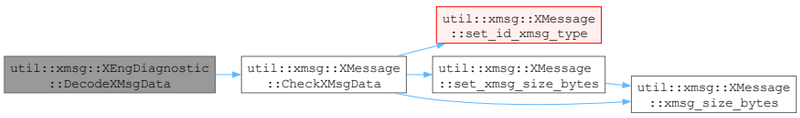
Todo Implement the functionality for decoding the Solo-II diagnostic engineering data message.

Implements util::xmsg::XMessage.

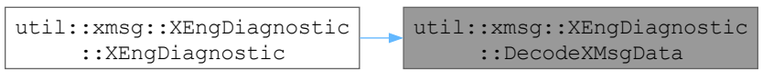
References CATCH_BLOCK, util::xmsg::XMessage::CheckXMsgData(), and util::xmsg::XMSG_TYPE_ID_ENG_DIAGNOSTIC.

Referenced by XEngDiagnostic().

Here is the call graph for this function:



Here is the caller graph for this function:



10.78.3.3 GenerateListingString()

```
const string util::xmsg::XEngDiagnostic::GenerateListingString () const  [inline]
```

Generate a listing string representation of the diagnostic dive engineering data message.

Returns

A listing string representation of the diagnostic dive engineering data message.

Todo Implement the functionality for generating a listing string for the Solo-II diagnostic engineering data message.

References CATCH_BLOCK.

10.78.3.4 id_dive()

```
int16_t util::xmsg::XMessage::id_dive () const  [inline], [inherited]
```

Get the dive or cycle number of this X-Message.

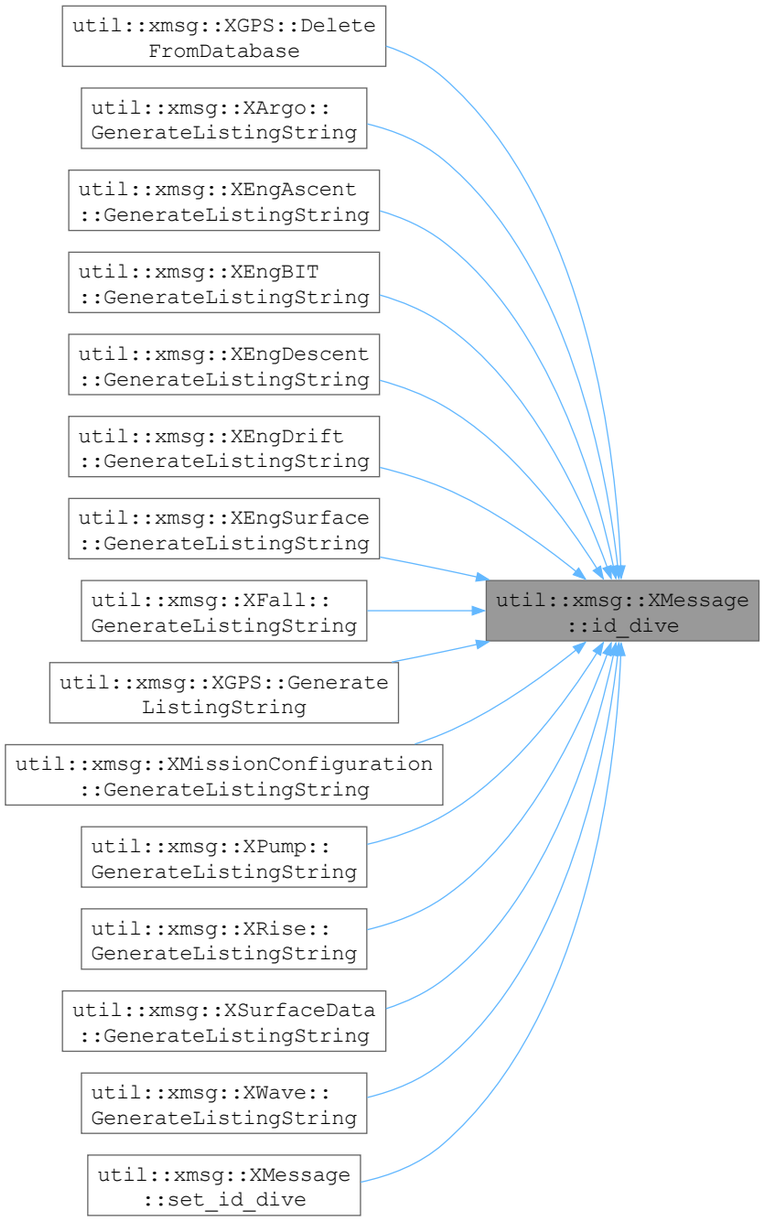
Returns

The dive or cycle number of this X-Message.

References util::xmsg::XMessage::id_dive_.

Referenced by util::xmsg::XGPS::DeleteFromDatabase(), util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateLis
util::xmsg::XEngBIT::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingStr
util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(),
util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing
util::xmsg::XSurfaceData::GenerateListingString(), util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_dive().

Here is the caller graph for this function:



10.78.3.5 id_instrument()

```
uint16_t util::xmsg::XMessage::id_instrument () const  [inline], [inherited]
```

Get the unique instrument identifier of this X-Message.

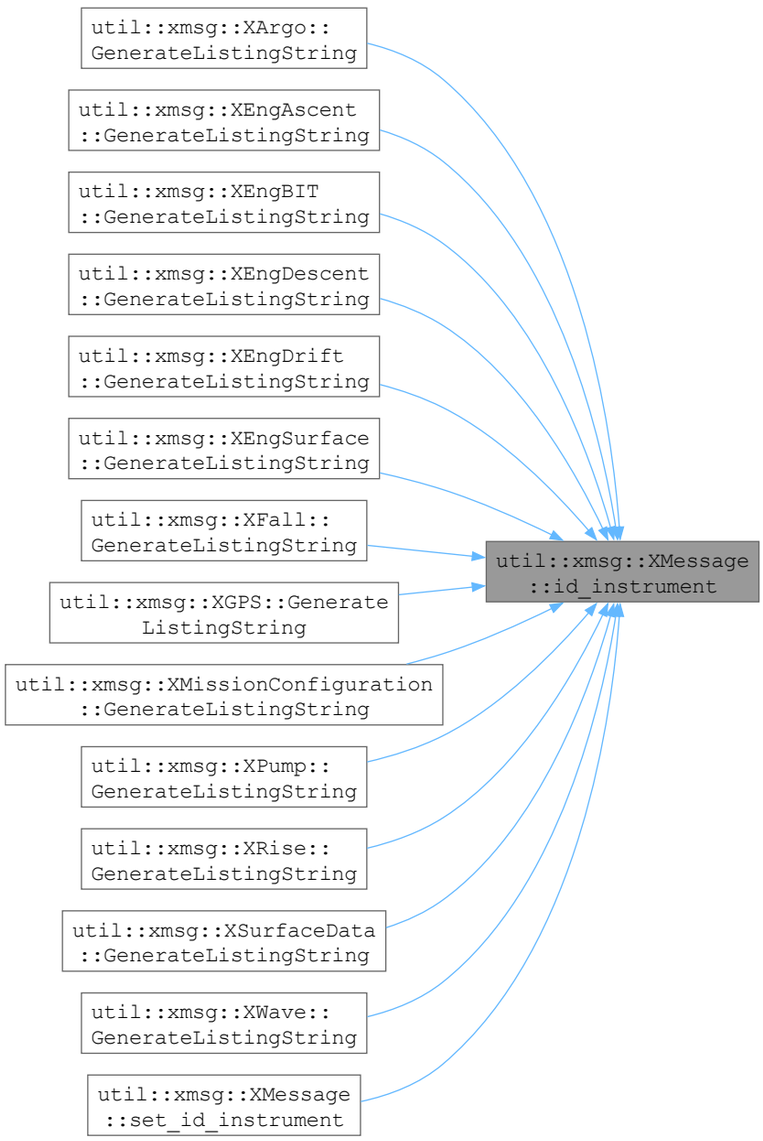
Returns

The unique instrument identifier of this X-Message.

References util::xmsg::XMessage::id_instrument_.

Referenced by util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngBIT::Generate
util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), util::xmsg::XEngSurface::GenerateListin
util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingS
util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(),
util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_instrument().

Here is the caller graph for this function:



10.78.3.6 id_xmsg_type()

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type () const  [inline], [inherited]
```

Get the enumerated X-Message type identifier.

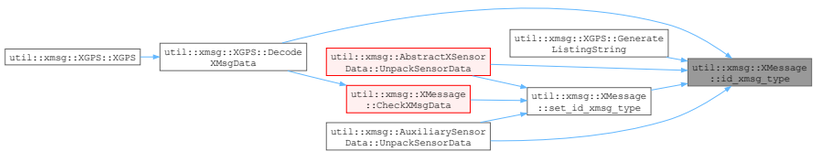
Returns

The enumerated X-Message type identifier.

References util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMessage::set_id_xmsg_typ
util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the caller graph for this function:



10.78.3.7 set_id_dive()

```
void util::xmsg::XMessage::set_id_dive (
            int16_t id_dive)  [inline], [protected], [inherited]
```

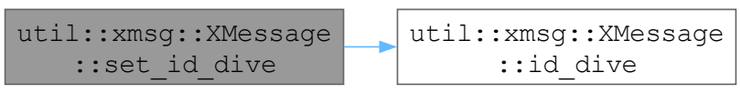
Set the dive or cycle number of this message.

Parameters

in	*id_dive*	The dive or cycle number of this message to set.

References util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::id_dive_.

Here is the call graph for this function:



10.78.3.8 set_id_instrument()

```
void util::xmsg::XMessage::set_id_instrument (
            uint16_t id_instrument)  [inline], [protected], [inherited]
```

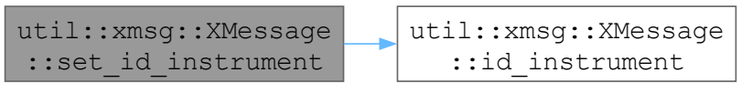
Set the unique instrument identifier.

Parameters

in	*id_instrument*	The unique instrument identifier to set.

References util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::id_instrument_.

Here is the call graph for this function:



10.78.3.9 set_id_xmsg_type()

```
void util::xmsg::XMessage::set_id_xmsg_type (
            XMsgTypeId_t id_xmsg_type)  [inline], [protected], [inherited]
```

Set the X-Message type identifier.

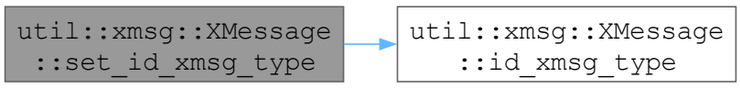
Parameters

in	*id_xmsg_type*	The X-Message type identifier to set.

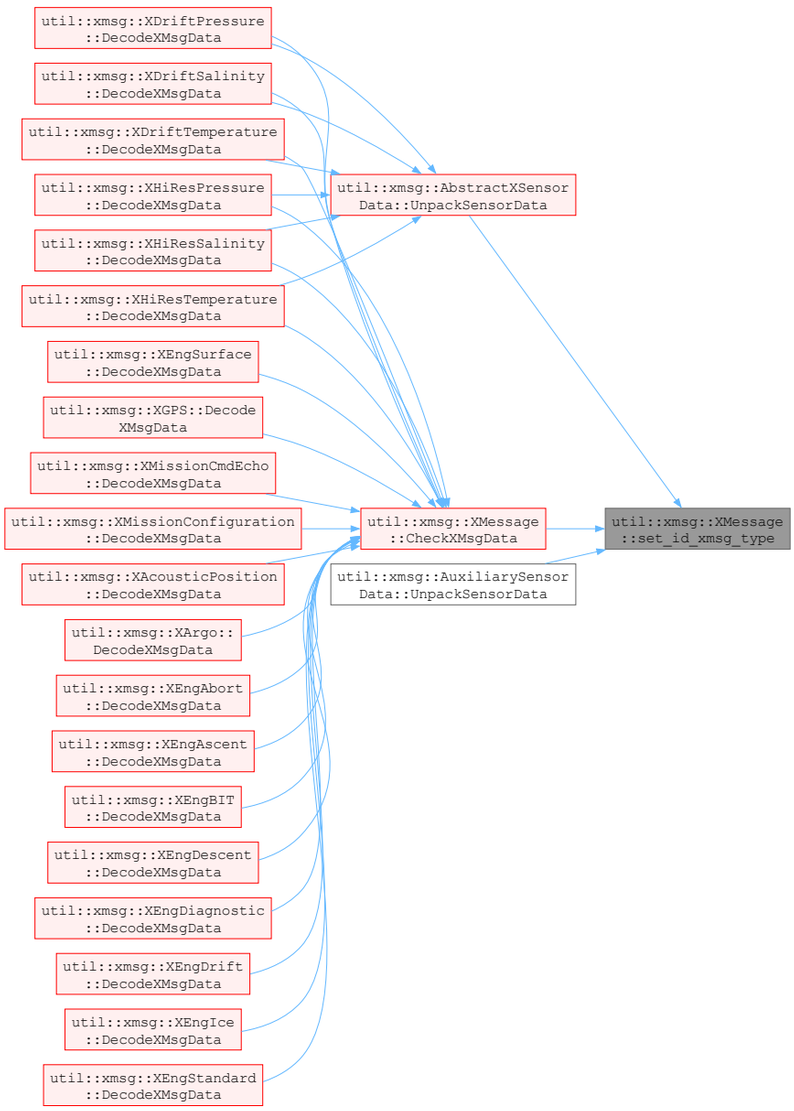
References util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.78.3.10 set_xmsg_size_bytes()

```
void util::xmsg::XMessage::set_xmsg_size_bytes (
            uint16_t xmsg_size_bytes)  [inline], [protected], [inherited]
```

Set the size of the X-Message, in bytes.

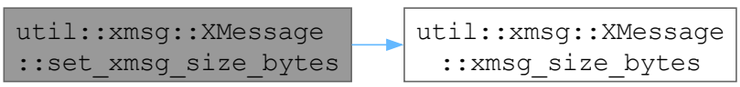
Parameters

in	*xmsg_size_bytes*	The size of the X-Message to set, in bytes.

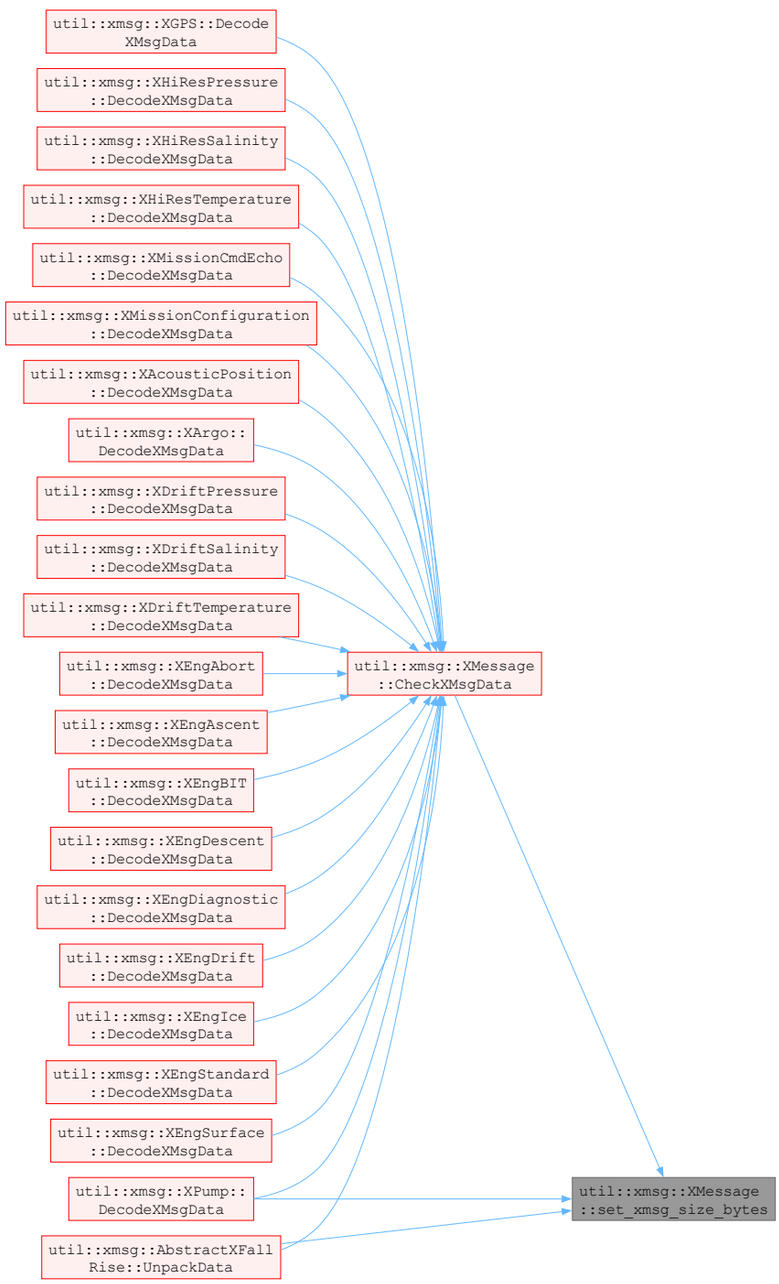
References util::xmsg::XMessage::xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XPump::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::Unpa

Here is the call graph for this function:



Here is the caller graph for this function:



10.78.3.11 xmsg_size_bytes()

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes () const  [inline], [inherited]
```

Get the size of the X-Message, in bytes.

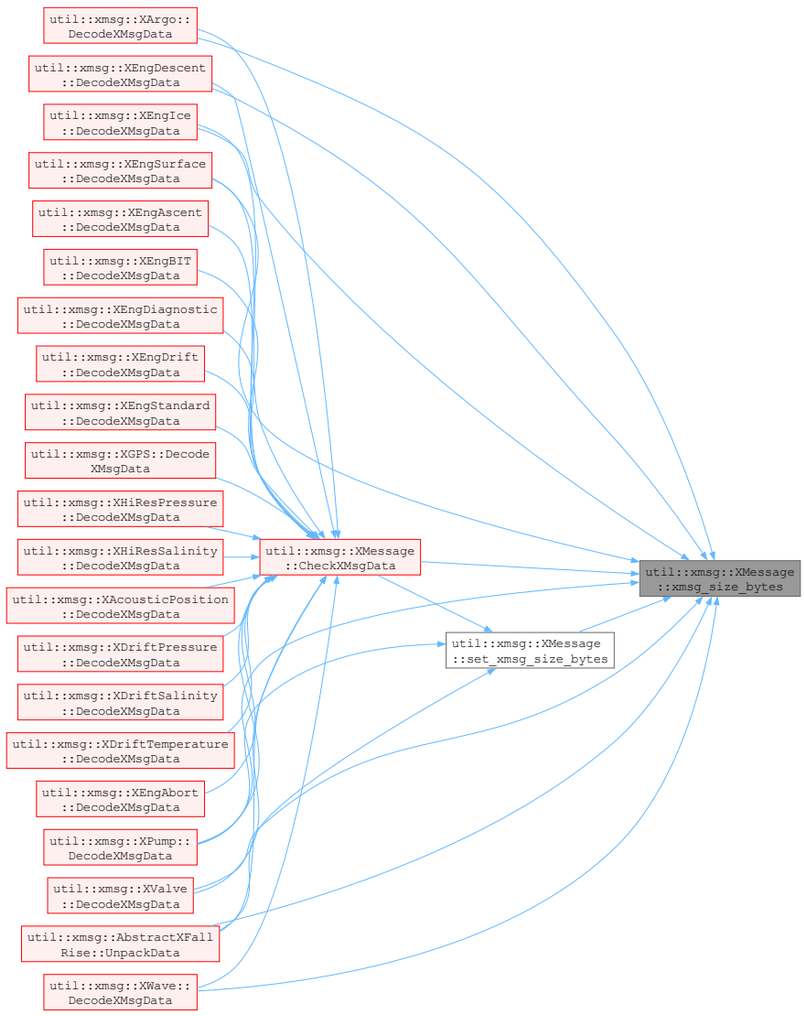
Returns

The size of the X-Message, in bytes.

References util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgI
util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(),
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), util::xmsg::XMessage::set_xmsg_size_bytes(),
and util::xmsg::AbstractXFallRise::UnpackData().

Here is the caller graph for this function:



10.78.4 Member Data Documentation

10.78.4.1 id_dive_

```
int16_t util::xmsg::XMessage::id_dive_  [private], [inherited]
```

The dive or cycle number of this message.

Referenced by util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::set_id_dive().

10.78.4.2 id_instrument_

```
uint16_t util::xmsg::XMessage::id_instrument_  [private], [inherited]
```

The unique WHOI identifier of this instrument.

Referenced by util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::set_id_instrument().

10.78.4.3 id_xmsg_type_

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type_  [private], [inherited]
```

The X-message type identifier.

Referenced by util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::set_id_xmsg_type().

10.78.4.4 xmsg_size_bytes_

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes_  [private], [inherited]
```

The size of the X message, in bytes.

Referenced by util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes().

The documentation for this class was generated from the following file:

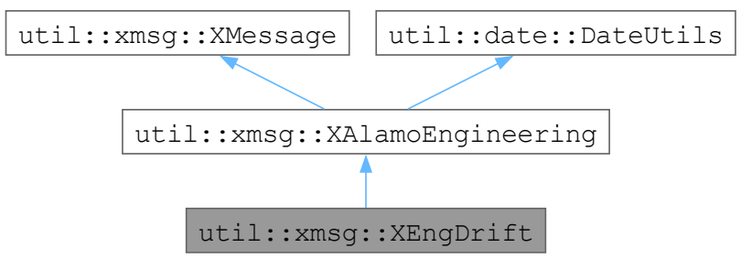
- data/xmsg/XEngDiagnostic.h

10.79 util::xmsg::XEngDrift Class Reference

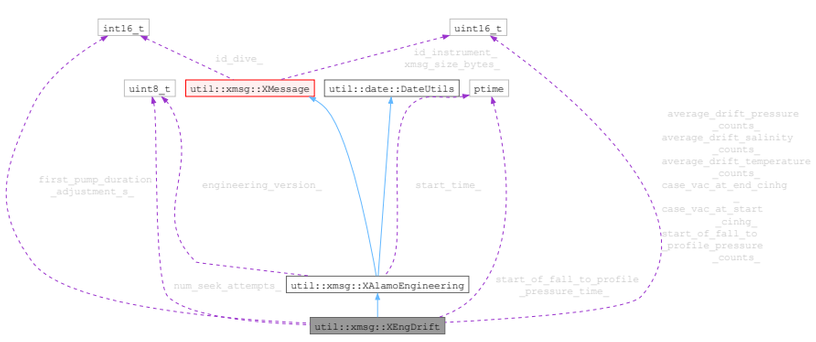
Class representing an drift/park state engineering X-Message.

```
#include <XEngDrift.h>
```

Inheritance diagram for util::xmsg::XEngDrift:



Collaboration diagram for util::xmsg::XEngDrift:



Public Member Functions

- XEngDrift (int16_t dive, uint16_t whoi, const vector< uint8_t > &data)

  Parameterized constructor.
- ~XEngDrift ()

  Destructor.
- uint16_t **average_drift_pressure_counts** () const
- template<typename T = double>
  T **average_drift_pressure_dbar** (int16_t gain, int16_t offs) const
- uint16_t **average_drift_salinity_counts** () const
- template<typename T = double>
  T **average_drift_salinity_psu** (int16_t gain, int16_t offs) const
- template<typename T = double>
  T **average_drift_temperature_c** (int16_t gain, int16_t offs) const
- uint16_t **average_drift_temperature_counts** () const
- uint16_t **case_vac_at_end_cinhg** () const
- template<typename T = float>
  T **case_vac_at_end_inhg** () const
- uint16_t **case_vac_at_start_cinhg** () const
- template<typename T = float>
  T **case_vac_at_start_inhg** () const
- const uint8_t * CheckXMsgData (const vector< uint8_t > &data, int id, int minBytes, bool parseSize=true)

  Check the X-message for the correct ID, minimum size, and message terminator.
- void DecodeXMsgData (const vector< uint8_t > &data)

  Decode the drift/park state engineering X-Message from the binary data.
- uint8_t engineering_version () const

  Get the engineering message version identifier.
- float engineering_version_as_float () const

  Get the engineering message verion identifier as a floating point number.
- int16_t **first_pump_duration_adjustment_s** () const
- const string GenerateListingString (uint16_t pg, uint16_t po, uint16_t sg, uint16_t so, uint16_t tg, uint16_t to) const

  Get the listing entry string for this drift/park engineering data X-Message.
- int16_t id_dive () const

  Get the dive or cycle number of this X-Message.
- uint16_t id_instrument () const

  Get the unique instrument identifier of this X-Message.
- XMsgTypeId_t id_xmsg_type () const

  Get the enumerated X-Message type identifier.
- uint8_t **num_seek_attempts** () const
- uint16_t **start_of_fall_to_profile_pressure_counts** () const
- template<typename T = double>
  T **start_of_fall_to_profile_pressure_dbar** (int16_t gain, int16_t offs) const
- const ptime & **start_of_fall_to_profile_pressure_time** () const
- const ptime & start_time () const

  Get the engineering message state start time.
- uint16_t xmsg_size_bytes () const

  Get the size of the X-Message, in bytes.

Static Public Member Functions

- static string GenerateCurrentUtcTimeIso8601String ()

  Generate an ISO8601 format date/time string representation of the current time, in UTC.
- static string GenerateIso8601String (const ptime &p, bool extended=true)

  Generate an ISO8601 format date/time string representation of a POSIX time object.
- static double GenerateJulianDay (const ptime &dt, const ptime &ref_dt)

  Convert a date/time to fractional days since a reference date/time.
- static ptime GetCurrentUtcTime ()

  Generate a POSIX time object representing the current date and time (rounded to the nearest second).

Protected Member Functions

- void **set_average_drift_pressure_counts** (uint16_t average_drift_pressure_counts)
- void **set_average_drift_salinity_counts** (uint16_t average_drift_salinity_counts)
- void **set_average_drift_temperature_counts** (uint16_t average_drift_temperature_counts)
- void **set_case_vac_at_end_cinhg** (uint16_t case_vac_at_end_cinhg)
- void **set_case_vac_at_start_cinhg** (uint16_t case_vac_at_start_cinhg)
- void set_engineering_version (uint8_t engineering_version)

  Set the engineering message version identifier.
- void **set_first_pump_duration_adjustment_s** (int16_t first_pump_duration_adjustment_s)
- void set_id_dive (int16_t id_dive)

  Set the dive or cycle number of this message.
- void set_id_instrument (uint16_t id_instrument)

  Set the unique instrument identifier.
- void set_id_xmsg_type (XMsgTypeId_t id_xmsg_type)

  Set the X-Message type identifier.
- void **set_num_seek_attempts** (uint8_t num_seek_attempts)
- void **set_start_of_fall_to_profile_pressure_counts** (uint16_t start_of_fall_to_profile_pressure_counts)
- void **set_start_of_fall_to_profile_pressure_time** (const ptime &start_of_fall_to_profile_pressure_time)
- void set_start_time (const ptime &start_time)

  Set the engineering message state start time.
- void set_xmsg_size_bytes (uint16_t xmsg_size_bytes)

  Set the size of the X-Message, in bytes.

Private Attributes

- uint16_t average_drift_pressure_counts_

  The average pressure during the drift/park phase of the dive cycle, in raw pressure counts.
- uint16_t **average_drift_salinity_counts_**
- uint16_t average_drift_temperature_counts_

  The average temperature during the drift/park phase of the dive cycle, in raw temperature counts.
- uint16_t case_vac_at_end_cinhg_

  The measured internal pressure case vacuum, in 0.01 inHg, at the end of the drift/park phase of the dive cycle.
- uint16_t case_vac_at_start_cinhg_

  The measured internal pressure case vacuum, in 0.01 inHg, at the start of the drift/park phase of the dive cycle.
- uint8_t engineering_version_

  The engineering message version identifier.
- int16_t **first_pump_duration_adjustment_s_**
- int16_t id_dive_

  The dive or cycle number of this message.
- uint16_t id_instrument_

  The unique WHOI identifier of this instrument.
- XMsgTypeId_t id_xmsg_type_

  The X-message type identifier.
- uint8_t **num_seek_attempts_**
- uint16_t start_of_fall_to_profile_pressure_counts_

  The measured pressure at the start of the fall-to-profile phase of the dive cycle, in raw pressure counts.
- ptime start_of_fall_to_profile_pressure_time_

  The start time of the fall-to-profile phase of the dive cycle.
- ptime start_time_

  The engineering message state start time.
- uint16_t xmsg_size_bytes_

  The size of the X message, in bytes.

10.79.1 Detailed Description

Class representing an drift/park state engineering X-Message.

Note

This X-Message is only used by ALAMO-type instruments.

This message type was introduced in the X-Message format specification version 2.1.

10.79.2 Constructor & Destructor Documentation

10.79.2.1 XEngDrift()

```
util::xmsg::XEngDrift::XEngDrift (
            int16_t dive,
            uint16_t whoi,
            const vector< uint8_t > & data)  [inline]
```

Parameterized constructor.

Parameters

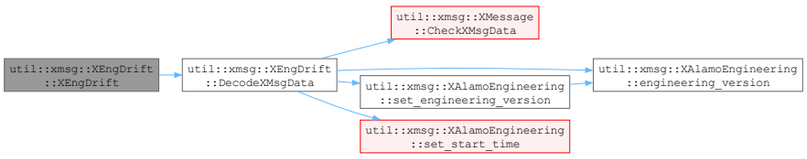
in	*dive*	The dive or cycle number of this message.
in	*whoi*	The unique WHOI identifier.
in	*data*	The binary X-message data to decode.

Exceptions

util::log::MessageException	if an decode error occurs.

References DecodeXMsgData().

Here is the call graph for this function:



10.79.2.2 ~XEngDrift()

```
util::xmsg::XEngDrift::~XEngDrift ()  [inline]
```

Destructor.

10.79.3 Member Function Documentation

10.79.3.1 CheckXMsgData()

```
const uint8_t * util::xmsg::XMessage::CheckXMsgData (
            const vector< uint8_t > & data,
            int id,
            int minBytes,
            bool parseSize = true)  [inline], [inherited]
```

Check the X-message for the correct ID, minimum size, and message terminator.

Parameters

in	*data*	The X-message binary data to check.
in	*id*	The expected ID of the X-message. If this field is less that 0, then the ID check will be skipped.
in	*minBytes*	The minimum number of bytes expected in a valid message.
in	*parseSize*	If TRUE, then the message size field will be parsed from the X-Message. Otherwise, the size field will not be parsed.

Exceptions

util::log::MessageException	if • the message does not contain the expected ID, • the message does not contain the minimum number of expected bytes, • the message does not contain a terminator (';') in the correct location, or, • any other error occurs.

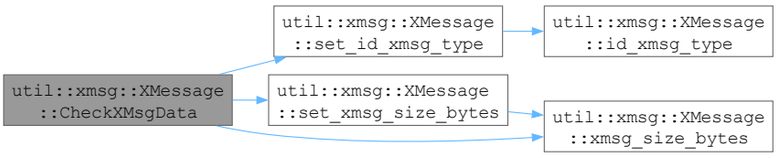
Returns

A pointer to the underlying data buffer of data.

References CATCH_BLOCK, EXC_THROW, lwrn, util::xmsg::XMessage::set_id_xmsg_type(), util::xmsg::XMessage::set_xmsg_size_byte u16, and util::xmsg::XMessage::xmsg_size_bytes().

Referenced by util::xmsg::XAcousticPosition::DecodeXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XDriftPressure::Dec util::xmsg::XDriftSalinity::DecodeXMsgData(), util::xmsg::XDriftTemperature::DecodeXMsgData(), util::xmsg::XEngAbort::DecodeXMsgDa util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngBIT::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(), util::xmsg::XEngDiagnostic::DecodeXMsgData(), DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngStandard::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XHiResPressure::DecodeXMsgData(), util::xmsg::XHiResSalinity::DecodeXMsgData(), util::xmsg::XHiResTemperature::Decod util::xmsg::XMissionCmdEcho::DecodeXMsgData(), util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XProfilePressure: util::xmsg::XProfileSalinity::DecodeXMsgData(), util::xmsg::XProfileTemperature::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgD util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the call graph for this function:



10.79.3.2 DecodeXMsgData()

```
void util::xmsg::XEngDrift::DecodeXMsgData (
            const vector< uint8_t > & data)  [inline], [virtual]
```

Decode the drift/park state engineering X-Message from the binary data.

Parameters

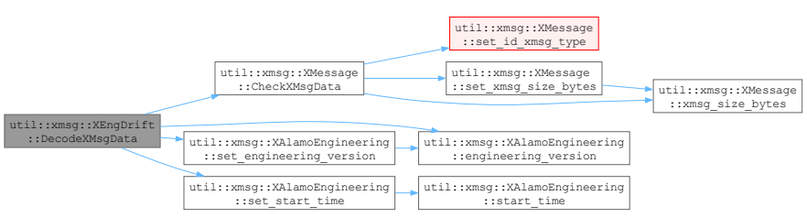
in	*data*	The binary message to decode.

Implements util::xmsg::XMessage.

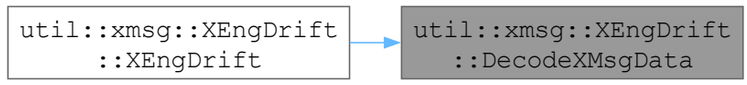
References CATCH_BLOCK, util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XAlamoEngineering::engineering_version(), EXC_THROW, s2000, util::xmsg::XAlamoEngineering::set_engineering_version(), util::xmsg::XAlamoEngineering::set_start_time(), u08, and u16.

Referenced by XEngDrift().

Here is the call graph for this function:



Here is the caller graph for this function:



10.79.3.3 engineering_version()

```
uint8_t util::xmsg::XAlamoEngineering::engineering_version () const  [inline], [inherited]
```

Get the engineering message version identifier.

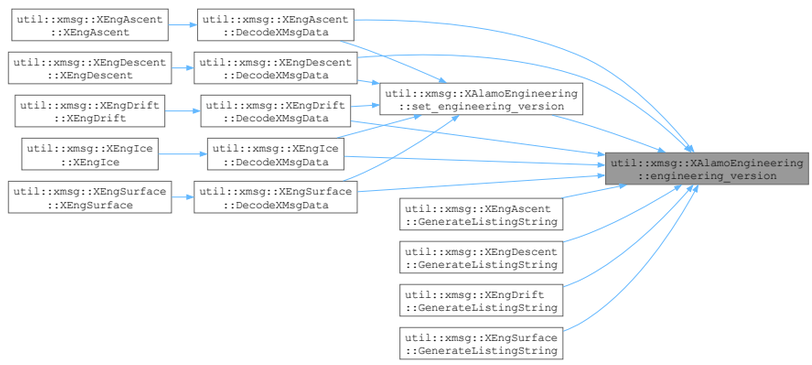
Returns

The engineering message version identifier.

References util::xmsg::XAlamoEngineering::engineering_version_.

Referenced by util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(), DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), GenerateListingString(), util::xmsg::XEngSurface::GenerateListingString(), and util::xmsg::XAlamoEngineering::set_engineering_version().

Here is the caller graph for this function:



10.79.3.4 engineering_version_as_float()

```
float util::xmsg::XAlamoEngineering::engineering_version_as_float () const  [inline], [inherited]
```

Get the engineering message verion identifier as a floating point number.

Returns

The engineering message verion identifier as a floating point number.

References util::xmsg::XAlamoEngineering::engineering_version_.

10.79.3.5 GenerateCurrentUtcTimeIso8601String()

```
static string util::date::DateUtils::GenerateCurrentUtcTimeIso8601String ()  [inline], [static],
[inherited]
```

Generate an ISO8601 format date/time string representation of the current time, in UTC.

Note

All date/time objects use the UTC time-zone.

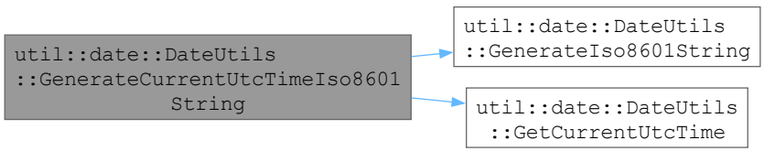
Returns

An ISO8601 format date/time string representation of the current time.

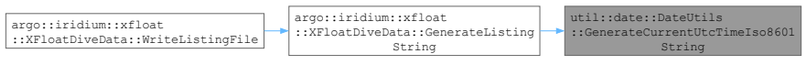
References util::date::DateUtils::GenerateIso8601String(), and util::date::DateUtils::GetCurrentUtcTime().

Referenced by argo::iridium::xfloat::XFloatDiveData::GenerateListingString().

Here is the call graph for this function:



Here is the caller graph for this function:



10.79.3.6 GenerateIso8601String()

```
static string util::date::DateUtils::GenerateIso8601String (
            const ptime & p,
            bool extended = true)  [inline], [static], [inherited]
```

Generate an ISO8601 format date/time string representation of a POSIX time object.

Note

All date/time objects use the UTC time-zone.

Parameters

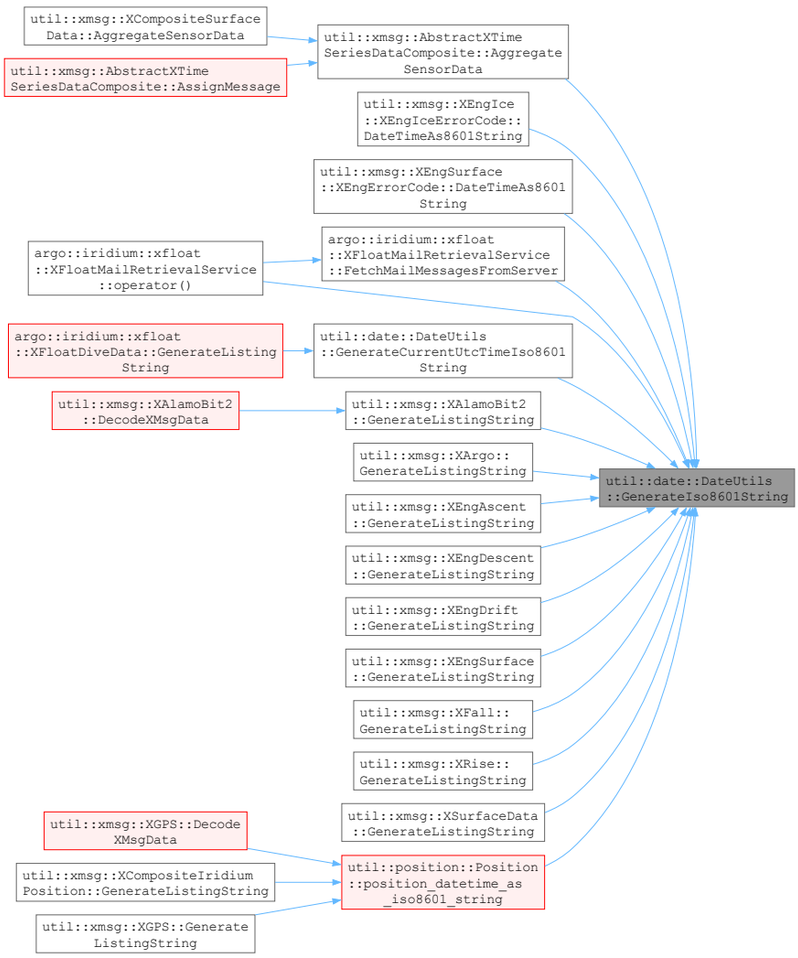
in	*p*	POSIX time from which to generate string.
in	*extended*	If TRUE, use the ISO 8601 extended format (YYYY-MM-DDThh:mm:ssZ); otherwise, use the basic format (YYYYMMDDThhmmss).

Returns

An ISO8601 format date/time string representation of the POSIX time object. If p is equal to boost::posix_↩ time::not_a_date_time, then the string "0000-00-00T00:00:00Z" (extended) or "00000000000000" (basic) will be returned.

Referenced by util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::AggregateSensorData(), util::xmsg::XEngIce::XEngIce
util::xmsg::XEngSurface::XEngErrorCode::DateTimeAs8601String(), argo::iridium::xfloat::XFloatMailRetrievalService::FetchMailMessages
util::date::DateUtils::GenerateCurrentUtcTimeIso8601String(), util::xmsg::XAlamoBit2::GenerateListingString(), util::xmsg::XArgo::Genera
util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), GenerateListingString(),
util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XRise::GenerateListingString(),
util::xmsg::XSurfaceData::GenerateListingString(), argo::iridium::xfloat::XFloatMailRetrievalService::operator()(), and
util::position::Position::position_datetime_as_iso8601_string().

Here is the caller graph for this function:



10.79.3.7 GenerateJulianDay()

```
static double util::date::DateUtils::GenerateJulianDay (
            const ptime & dt,
            const ptime & ref_dt)  [inline], [static], [inherited]
```

Convert a date/time to fractional days since a reference date/time.

Parameters

in	*dt*	The date/time to convert.
in	*ref*↩ *_dt*	The reference date/time (day 0).

Exceptions

util::log::MessageException	if either date/time is not valid.

Returns

The number of days (with millisecond resolution) from `ref_dt` to `dt`.

References CATCH_BLOCK, and EXC_THROW.

10.79.3.8 GenerateListingString()

```
const string util::xmsg::XEngDrift::GenerateListingString (
            uint16_t pg,
            uint16_t po,
            uint16_t sg,
            uint16_t so,
            uint16_t tg,
            uint16_t to) const  [inline]
```

Get the listing entry string for this drift/park engineering data X-Message.

Parameters

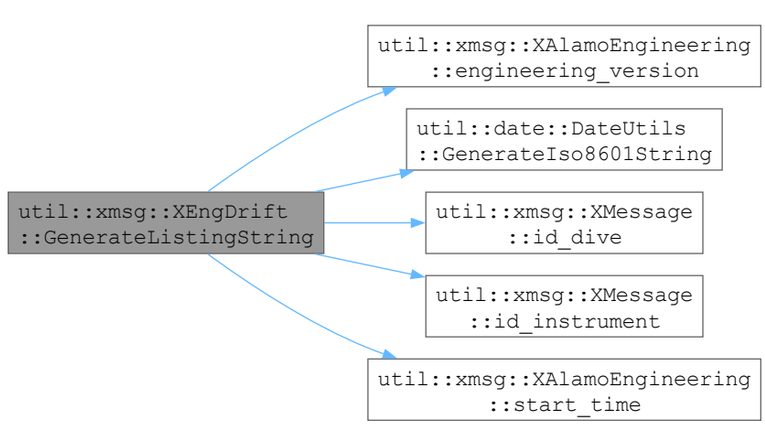
in	*pg*	The pressure sensor sensitivity scale factor, in LSB/dBar.
in	*po*	The pressure sensor offset, in dBar.
in	*sg*	The salinity sensor sensitivity scale factor, in LSB/PSU.
in	*so*	The salinity sensor offset, in PSU.
in	*tg*	The temperature sensor sensitivity scale factor, in LSB/degC.
in	*to*	The temperatyre sensor offset, in degC.

Returns

The listing entry string for this drift/park engineering data X-Message.

References CATCH_BLOCK, util::xmsg::XAlamoEngineering::engineering_version(), EXC_THROW, util::date::DateUtils::GenerateIso860 util::xmsg::XMessage::id_dive(), util::xmsg::XMessage::id_instrument(), start_of_fall_to_profile_pressure_time_, and util::xmsg::XAlamoEngineering::start_time().

Here is the call graph for this function:



10.79.3.9 GetCurrentUtcTime()

```
static ptime util::date::DateUtils::GetCurrentUtcTime ()  [inline], [static], [inherited]
```

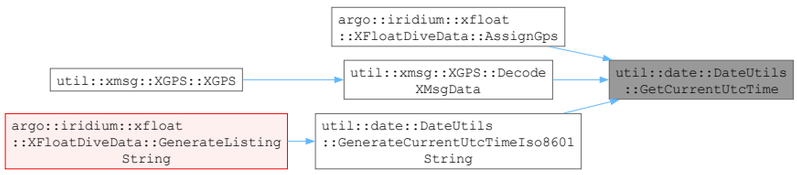
Generate a POSIX time object representing the current date and time (rounded to the nearest second).

Returns

A POSIX time object representing the current date and time (rounded to the nearest second).

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignGps(), util::xmsg::XGPS::DecodeXMsgData(), and util::date::DateUtils::GenerateCurrentUtcTimeIso8601String().

Here is the caller graph for this function:



10.79.3.10 id_dive()

```
int16_t util::xmsg::XMessage::id_dive () const  [inline], [inherited]
```

Get the dive or cycle number of this X-Message.

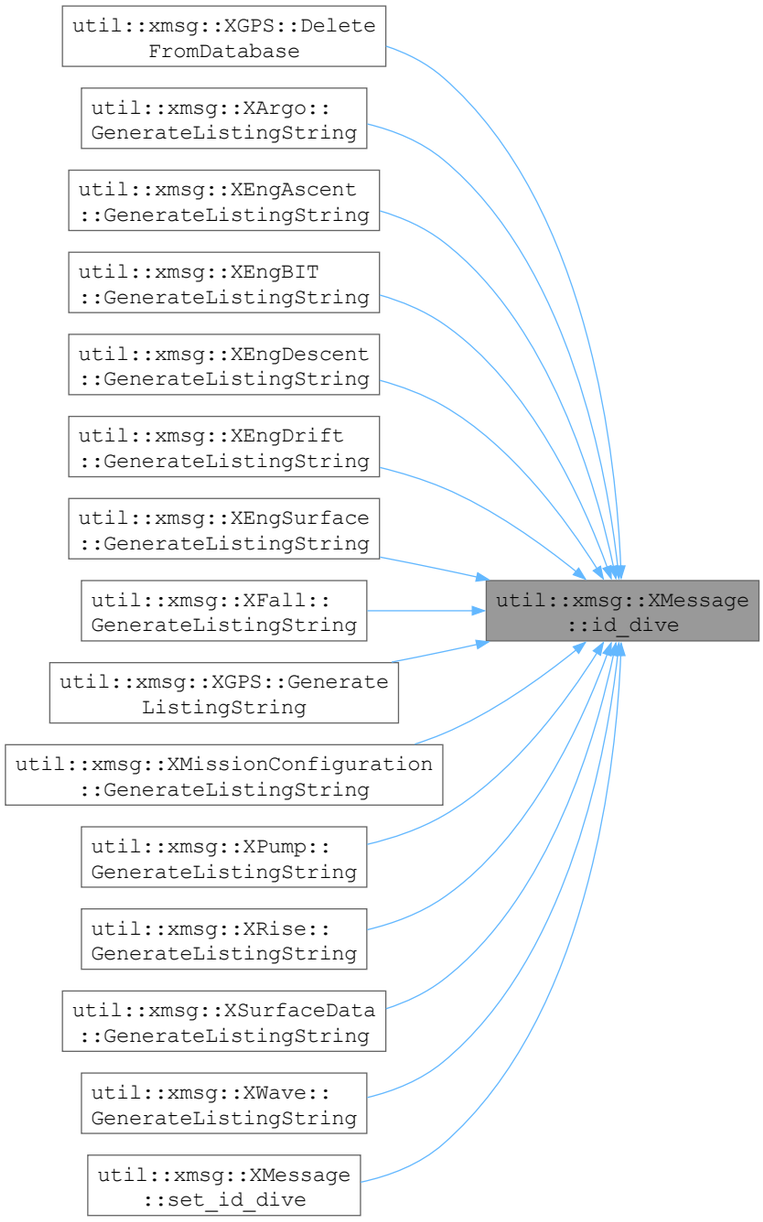
Returns

The dive or cycle number of this X-Message.

References util::xmsg::XMessage::id_dive_.

Referenced by util::xmsg::XGPS::DeleteFromDatabase(), util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateLis
util::xmsg::XEngBIT::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), GenerateListingString(),
util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(),
util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing
util::xmsg::XSurfaceData::GenerateListingString(), util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_dive().

Here is the caller graph for this function:



10.79.3.11 id_instrument()

```
uint16_t util::xmsg::XMessage::id_instrument () const  [inline], [inherited]
```

Get the unique instrument identifier of this X-Message.

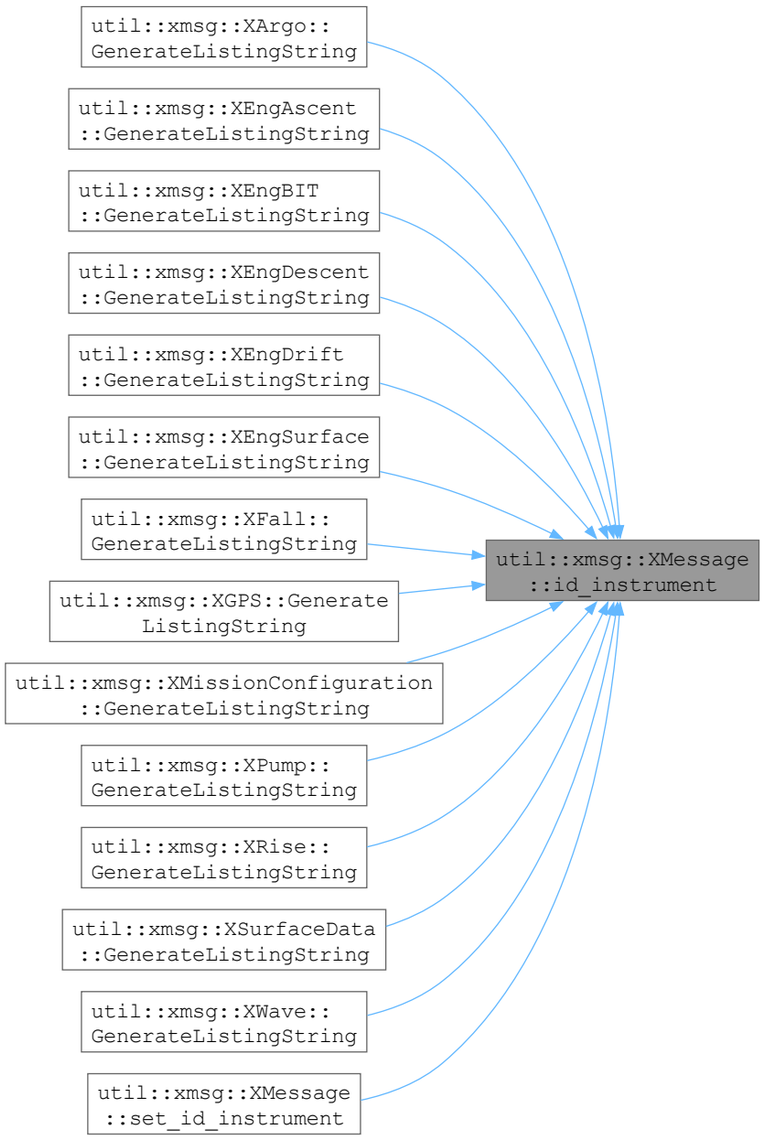
Returns

The unique instrument identifier of this X-Message.

References util::xmsg::XMessage::id_instrument_.

Referenced by util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngBIT::Generate
util::xmsg::XEngDescent::GenerateListingString(), GenerateListingString(), util::xmsg::XEngSurface::GenerateListingString(),
util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingS
util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(),
util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_instrument().

Here is the caller graph for this function:



10.79.3.12 id_xmsg_type()

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type () const  [inline], [inherited]
```

Get the enumerated X-Message type identifier.

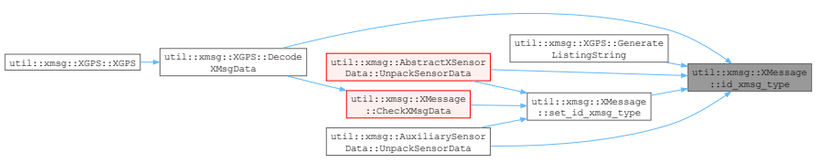
Returns

The enumerated X-Message type identifier.

References util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMessage::set_id_xmsg_typ
util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the caller graph for this function:



10.79.3.13 set_engineering_version()

```
void util::xmsg::XAlamoEngineering::set_engineering_version (
            uint8_t engineering_version)  [inline], [protected], [inherited]
```

Set the engineering message version identifier.

Parameters

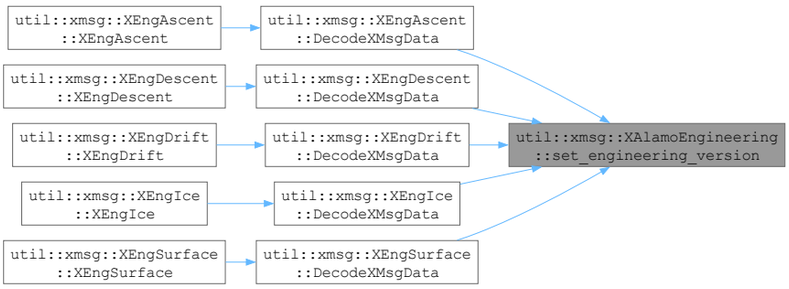
in	*engineering_version*	The engineering message version identifier.

References util::xmsg::XAlamoEngineering::engineering_version(), and util::xmsg::XAlamoEngineering::engineering_version_.

Referenced by util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(), DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(), and util::xmsg::XEngSurface::DecodeXMsgData().

Here is the call graph for this function:

Here is the caller graph for this function:



10.79.3.14 set_id_dive()

```
void util::xmsg::XMessage::set_id_dive (
            int16_t id_dive)  [inline], [protected], [inherited]
```

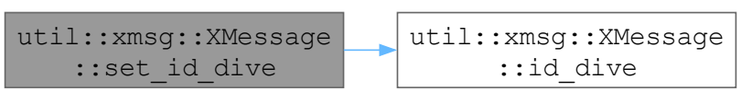
Set the dive or cycle number of this message.

Parameters

in	*id_dive*	The dive or cycle number of this message to set.

References util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::id_dive_.

Here is the call graph for this function:



10.79.3.15 set_id_instrument()

```
void util::xmsg::XMessage::set_id_instrument (
            uint16_t id_instrument)  [inline], [protected], [inherited]
```

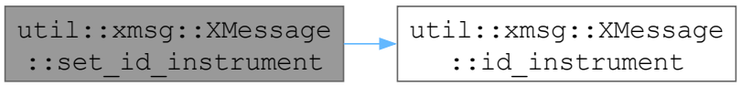
Set the unique instrument identifier.

Parameters

in	*id_instrument*	The unique instrument identifier to set.

References util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::id_instrument_.

Here is the call graph for this function:



10.79.3.16 set_id_xmsg_type()

```
void util::xmsg::XMessage::set_id_xmsg_type (
            XMsgTypeId_t id_xmsg_type)  [inline], [protected], [inherited]
```

Set the X-Message type identifier.

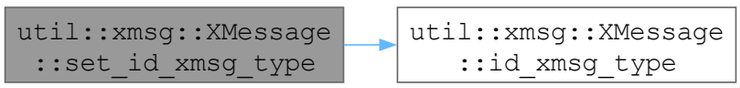
Parameters

in	*id_xmsg_type*	The X-Message type identifier to set.

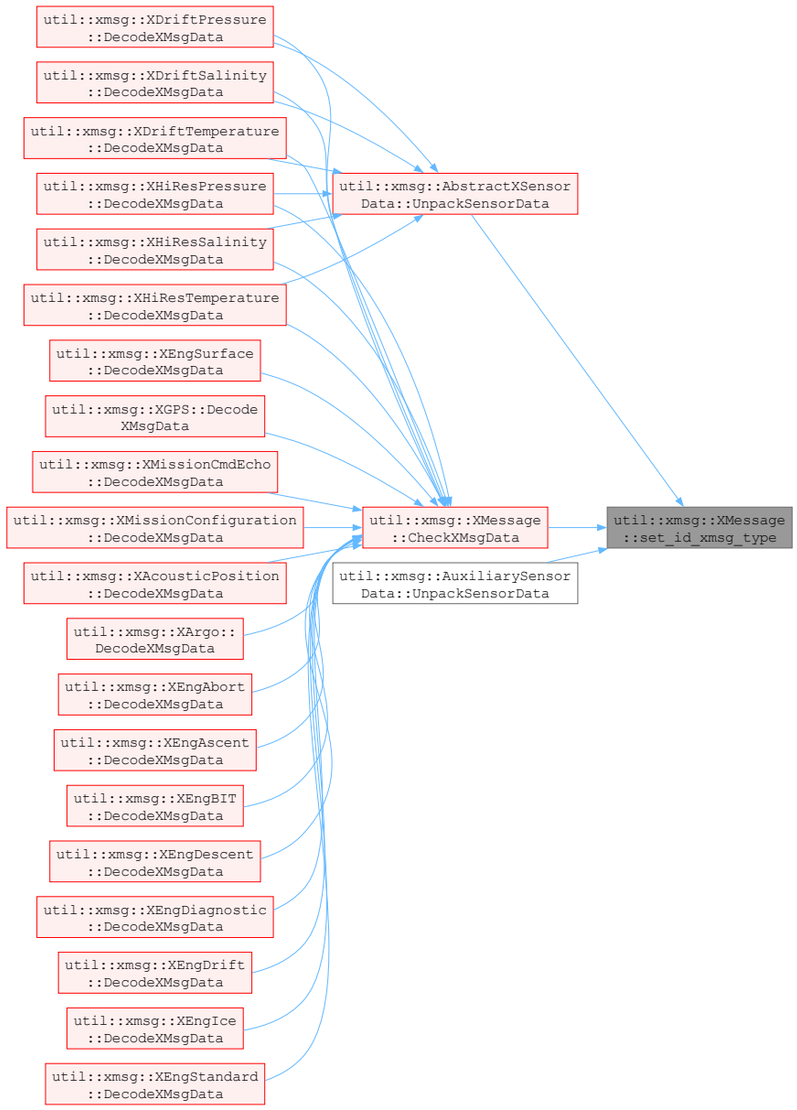
References util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.79.3.17 set_start_time()

```
void util::xmsg::XAlamoEngineering::set_start_time (
            const ptime & start_time)  [inline], [protected], [inherited]
```

Set the engineering message state start time.

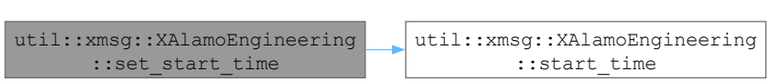
Parameters

in	*start_time*	The engineering message state start time.

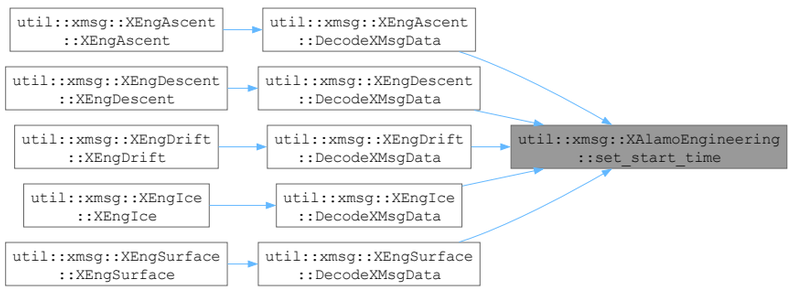
References util::xmsg::XAlamoEngineering::start_time(), and util::xmsg::XAlamoEngineering::start_time_.

Referenced by util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(), DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(), and util::xmsg::XEngSurface::DecodeXMsgData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.79.3.18 set_xmsg_size_bytes()

```
void util::xmsg::XMessage::set_xmsg_size_bytes (
            uint16_t xmsg_size_bytes)  [inline], [protected], [inherited]
```

Set the size of the X-Message, in bytes.

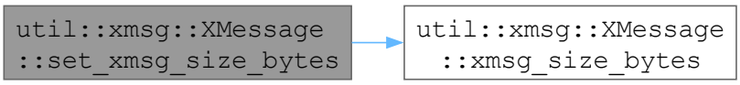
Parameters

in	*xmsg_size_bytes*	The size of the X-Message to set, in bytes.

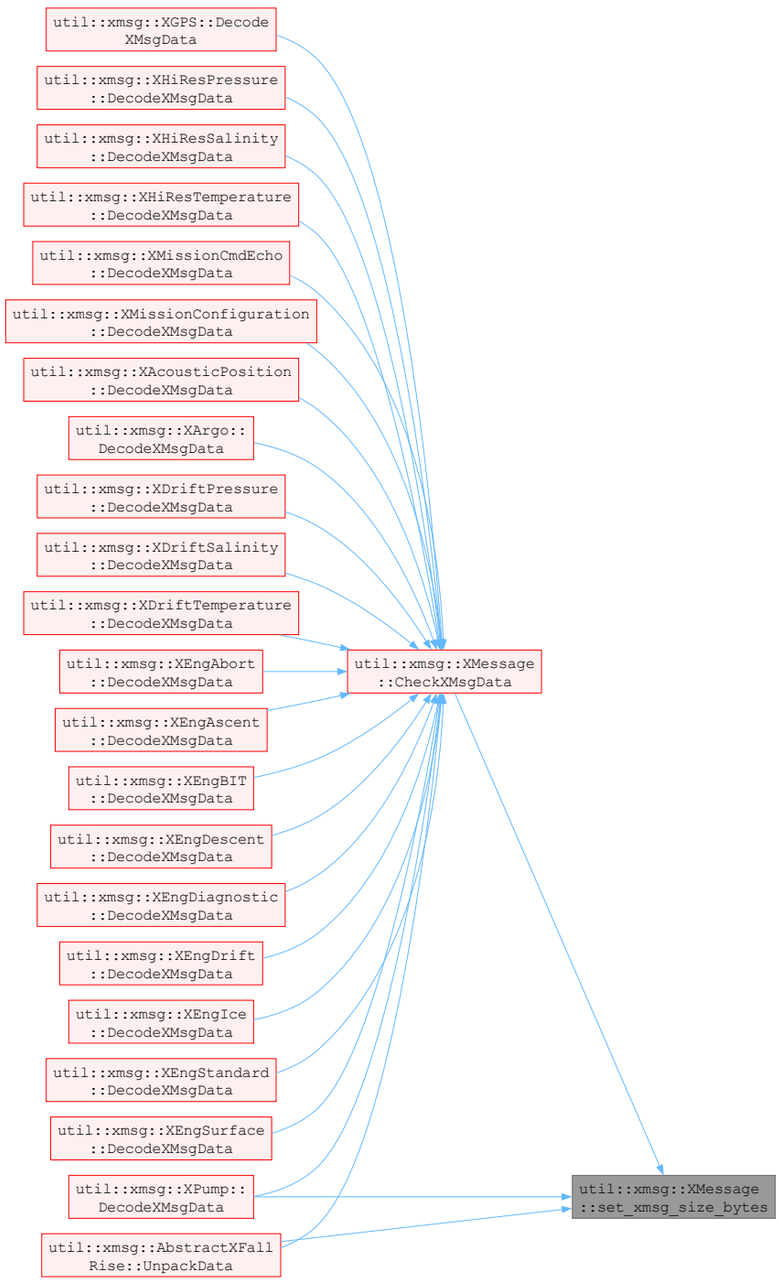
References util::xmsg::XMessage::xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XPump::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::Unpa

Here is the call graph for this function:



Here is the caller graph for this function:



10.79.3.19 start_time()

```
const ptime & util::xmsg::XAlamoEngineering::start_time () const  [inline], [inherited]
```

Get the engineering message state start time.

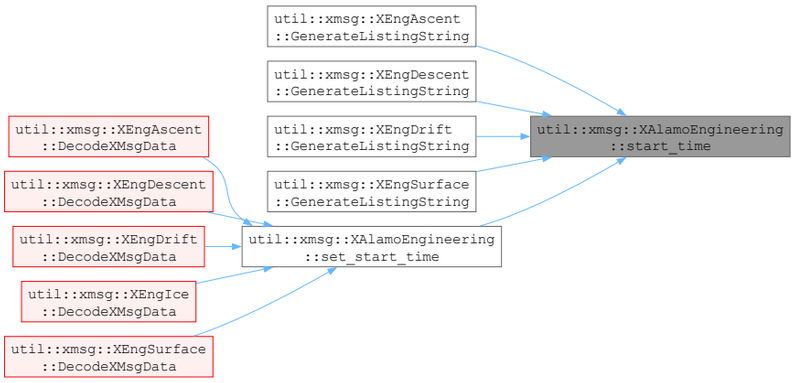
Returns

The engineering message state start time.

References util::xmsg::XAlamoEngineering::start_time_.

Referenced by util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), GenerateListingString(), util::xmsg::XEngSurface::GenerateListingString(), and util::xmsg::XAlamoEngineering::set_start_time().

Here is the caller graph for this function:



10.79.3.20 xmsg_size_bytes()

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes () const  [inline], [inherited]
```

Get the size of the X-Message, in bytes.

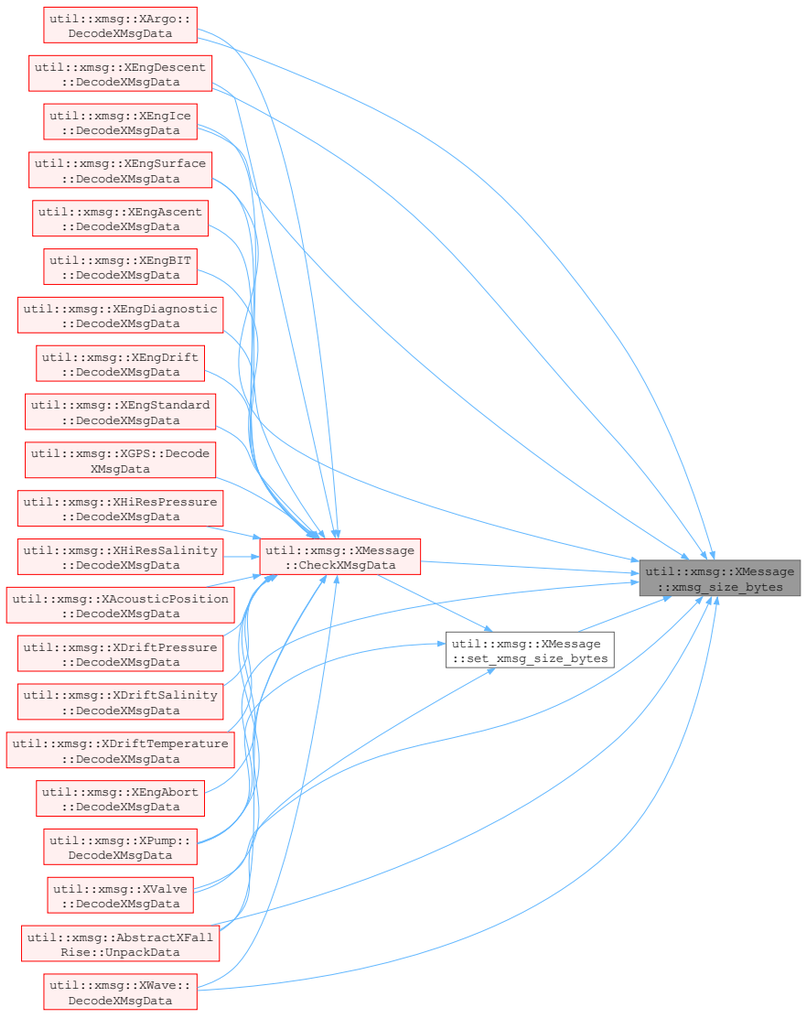
Returns

The size of the X-Message, in bytes.

References util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgI
util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(),
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), util::xmsg::XMessage::set_xmsg_size_bytes(),
and util::xmsg::AbstractXFallRise::UnpackData().

Here is the caller graph for this function:



10.79.4 Member Data Documentation

10.79.4.1 average_drift_pressure_counts_

```
uint16_t util::xmsg::XEngDrift::average_drift_pressure_counts_  [private]
```

The average pressure during the drift/park phase of the dive cycle, in raw pressure counts.

10.79.4.2 average_drift_temperature_counts_

```
uint16_t util::xmsg::XEngDrift::average_drift_temperature_counts_  [private]
```

The average temperature during the drift/park phase of the dive cycle, in raw temperature counts.

10.79.4.3 case_vac_at_end_cinhg_

```
uint16_t util::xmsg::XEngDrift::case_vac_at_end_cinhg_  [private]
```

The measured internal pressure case vacuum, in 0.01 inHg, at the end of the drift/park phase of the dive cycle.

Note

This field was added in version 0x11.

10.79.4.4 case_vac_at_start_cinhg_

```
uint16_t util::xmsg::XEngDrift::case_vac_at_start_cinhg_  [private]
```

The measured internal pressure case vacuum, in 0.01 inHg, at the start of the drift/park phase of the dive cycle.

Note

This field was added in version 0x11.

10.79.4.5 engineering_version_

```
uint8_t util::xmsg::XAlamoEngineering::engineering_version_  [private], [inherited]
```

The engineering message version identifier.

Referenced by util::xmsg::XAlamoEngineering::engineering_version(), util::xmsg::XAlamoEngineering::engineering_version_as_float(), and util::xmsg::XAlamoEngineering::set_engineering_version().

10.79.4.6 id_dive_

```
int16_t util::xmsg::XMessage::id_dive_  [private], [inherited]
```

The dive or cycle number of this message.

Referenced by util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::set_id_dive().

10.79.4.7 id_instrument_

```
uint16_t util::xmsg::XMessage::id_instrument_  [private], [inherited]
```

The unique WHOI identifier of this instrument.

Referenced by util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::set_id_instrument().

10.79.4.8 id_xmsg_type_

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type_  [private], [inherited]
```

The X-message type identifier.

Referenced by util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::set_id_xmsg_type().

10.79.4.9 start_of_fall_to_profile_pressure_counts_

```
uint16_t util::xmsg::XEngDrift::start_of_fall_to_profile_pressure_counts_  [private]
```

The measured pressure at the start of the fall-to-profile phase of the dive cycle, in raw pressure counts.

10.79.4.10 start_of_fall_to_profile_pressure_time_

```
ptime util::xmsg::XEngDrift::start_of_fall_to_profile_pressure_time_  [private]
```

The start time of the fall-to-profile phase of the dive cycle.

Referenced by GenerateListingString().

10.79.4.11 start_time_

```
ptime util::xmsg::XAlamoEngineering::start_time_  [private], [inherited]
```

The engineering message state start time.

Referenced by util::xmsg::XAlamoEngineering::set_start_time(), and util::xmsg::XAlamoEngineering::start_time().

10.79.4.12 xmsg_size_bytes_

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes_  [private], [inherited]
```

The size of the X message, in bytes.

Referenced by util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes().

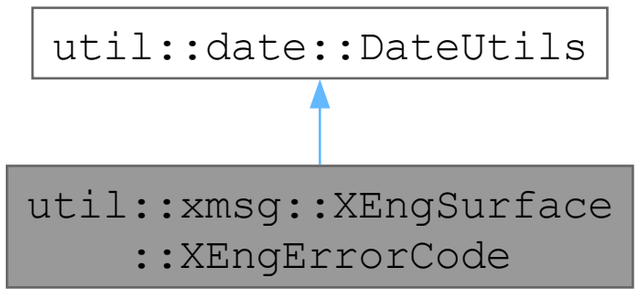
The documentation for this class was generated from the following file:

- data/xmsg/XEngDrift.h

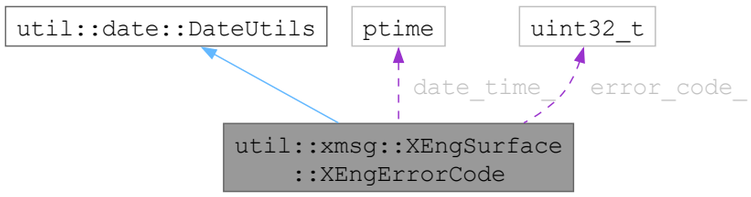
10.80 util::xmsg::XEngSurface::XEngErrorCode Class Reference

The engineering surface data error code class.

Inheritance diagram for util::xmsg::XEngSurface::XEngErrorCode:



Collaboration diagram for util::xmsg::XEngSurface::XEngErrorCode:



Public Member Functions

- XEngErrorCode (ptime date_time, uint32_t error_code)
  The parameterized constructor.
- ~XEngErrorCode ()
  The destructor.
- const ptime & date_time () const
  Get the timestamp for the error code.
- string DateTimeAs8601String () const
  Generate an ISO8601 format date/time string representation of the error date/time.
- uint32_t error_code () const
  Get the error code.

Static Public Member Functions

- static string GenerateCurrentUtcTimeIso8601String ()
  Generate an ISO8601 format date/time string representation of the current time, in UTC.
- static string GenerateIso8601String (const ptime &p, bool extended=true)
  Generate an ISO8601 format date/time string representation of a POSIX time object.
- static double GenerateJulianDay (const ptime &dt, const ptime &ref_dt)
  Convert a date/time to fractional days since a reference date/time.
- static ptime GetCurrentUtcTime ()
  Generate a POSIX time object representing the current date and time (rounded to the nearest second).

Protected Member Functions

- void set_date_time (const ptime &date_time)
  Set the timestamp for the error code.
- void set_error_code (uint32_t error_code)
  Set the error code.

Private Attributes

- ptime date_time_
  The timestamp for the error code.
- uint32_t error_code_
  The error code.

10.80.1 Detailed Description

The engineering surface data error code class.

An instance of this object represents a single error code reported in the ALAMO engineering (surface) data message. Each error consists of a timestamp and a

10.80.2 Constructor & Destructor Documentation

10.80.2.1 XEngErrorCode()

```
util::xmsg::XEngSurface::XEngErrorCode::XEngErrorCode (
            ptime date_time,
            uint32_t error_code)  [inline]
```

The parameterized constructor.

Parameters

in	*date_time*	The timestamp of the error code.
in	*error_code*	The error code.

10.80.2.2 ~XEngErrorCode()

```
util::xmsg::XEngSurface::XEngErrorCode::~XEngErrorCode ()  [inline]
```

The destructor.

10.80.3 Member Function Documentation

10.80.3.1 date_time()

```
const ptime & util::xmsg::XEngSurface::XEngErrorCode::date_time () const  [inline]
```

Get the timestamp for the error code.

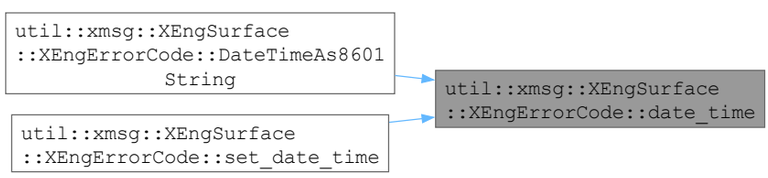
Returns

The timestamp for the error code.

References date_time_.

Referenced by DateTimeAs8601String(), and set_date_time().

Here is the caller graph for this function:



10.80.3.2 DateTimeAs8601String()

```
string util::xmsg::XEngSurface::XEngErrorCode::DateTimeAs8601String () const  [inline]
```

Generate an ISO8601 format date/time string representation of the error date/time.

Note

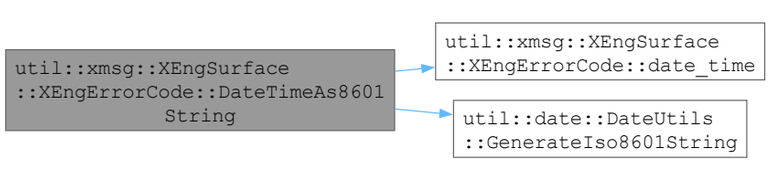
All date/time objects use the UTC time-zone.

Returns

An ISO8601 format date/time string representation of the error date/time. If date_time() returns boost::posix_↩ time::not_a_date_time, then the string "0000-00-00T00:00:00Z" will be returned.

References date_time(), and util::date::DateUtils::GenerateIso8601String().

Here is the call graph for this function:



10.80.3.3 error_code()

```
uint32_t util::xmsg::XEngSurface::XEngErrorCode::error_code () const  [inline]
```

Get the error code.

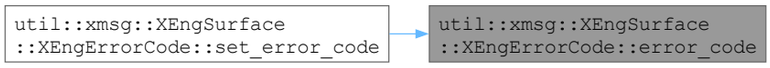
Returns

The error code.

References error_code_.

Referenced by set_error_code().

Here is the caller graph for this function:



10.80.3.4 GenerateCurrentUtcTimeIso8601String()

```
static string util::date::DateUtils::GenerateCurrentUtcTimeIso8601String ()  [inline], [static],
[inherited]
```

Generate an ISO8601 format date/time string representation of the current time, in UTC.

Note

All date/time objects use the UTC time-zone.

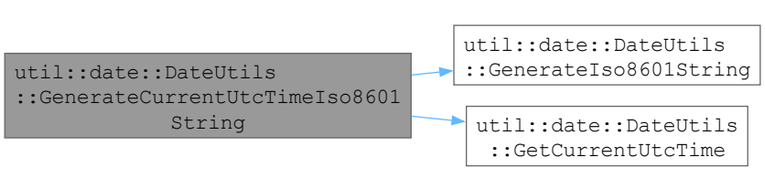
Returns

An ISO8601 format date/time string representation of the current time.

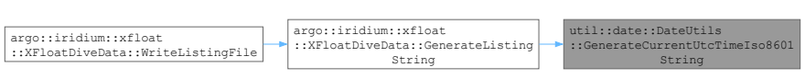
References util::date::DateUtils::GenerateIso8601String(), and util::date::DateUtils::GetCurrentUtcTime().

Referenced by argo::iridium::xfloat::XFloatDiveData::GenerateListingString().

Here is the call graph for this function:



Here is the caller graph for this function:



10.80.3.5 GenerateIso8601String()

```
static string util::date::DateUtils::GenerateIso8601String (
            const ptime & p,
            bool extended = true)  [inline], [static], [inherited]
```

Generate an ISO8601 format date/time string representation of a POSIX time object.

Note

All date/time objects use the UTC time-zone.

Parameters

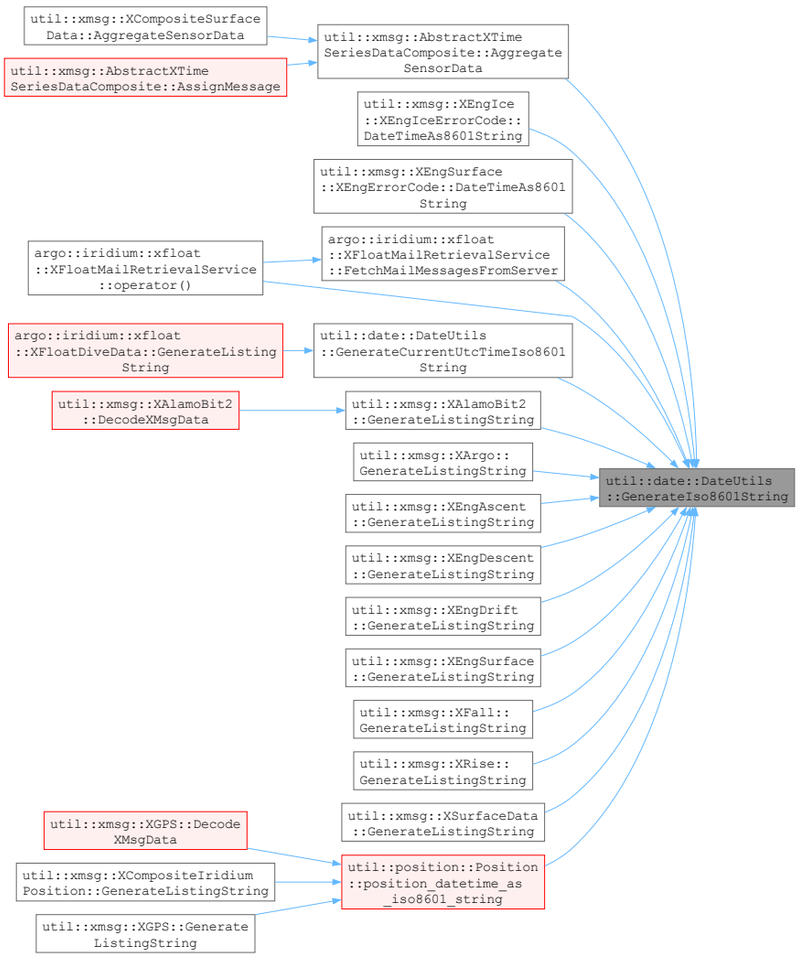
in	*p*	POSIX time from which to generate string.
in	*extended*	If TRUE, use the ISO 8601 extended format (YYYY-MM-DDThh:mm:ssZ); otherwise, use the basic format (YYYYMMDDThhmmss).

Returns

An ISO8601 format date/time string representation of the POSIX time object. If p is equal to boost::posix_↩ time::not_a_date_time, then the string "0000-00-00T00:00:00Z" (extended) or "00000000000000" (basic) will be returned.

Referenced by util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::AggregateSensorData(), util::xmsg::XEngIce::XEngIce DateTimeAs8601String(), argo::iridium::xfloat::XFloatMailRetrievalService::FetchMailMessagesFromServer(), util::date::DateUtils::Genera util::xmsg::XAlamoBit2::GenerateListingString(), util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateListingStrin util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), util::xmsg::XEngSurface::GenerateListin util::xmsg::XFall::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(), argo::iridium::xfloat::XFloatMailRetrievalService::operator()(), and util::position::Position::position_datetime_as_iso8601_string().

Here is the caller graph for this function:



10.80.3.6 GenerateJulianDay()

```
static double util::date::DateUtils::GenerateJulianDay (
            const ptime & dt,
            const ptime & ref_dt)  [inline], [static], [inherited]
```

Convert a date/time to fractional days since a reference date/time.

Parameters

in	*dt*	The date/time to convert.
in	*ref_dt*	The reference date/time (day 0).

Exceptions

util::log::MessageException	if either date/time is not valid.

Returns

The number of days (with millisecond resolution) from `ref_dt` to `dt`.

References CATCH_BLOCK, and EXC_THROW.

10.80.3.7 GetCurrentUtcTime()

```
static ptime util::date::DateUtils::GetCurrentUtcTime ()  [inline], [static], [inherited]
```

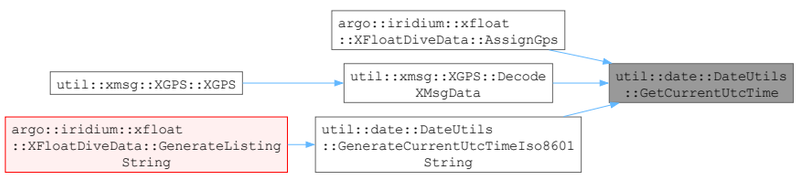
Generate a POSIX time object representing the current date and time (rounded to the nearest second).

Returns

A POSIX time object representing the current date and time (rounded to the nearest second).

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignGps(), util::xmsg::XGPS::DecodeXMsgData(), and util::date::DateUtils::GenerateCurrentUtcTimeIso8601String().

Here is the caller graph for this function:



10.80.3.8 set_date_time()

```
void util::xmsg::XEngSurface::XEngErrorCode::set_date_time (
             const ptime & date_time)  [inline], [protected]
```

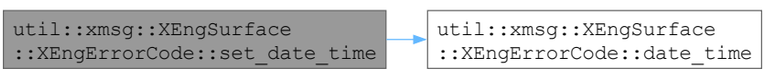
Set the timestamp for the error code.

Parameters

in	*date_time*	The timestamp for the error code.

References date_time(), and date_time_.

Here is the call graph for this function:



10.80.3.9 set_error_code()

```
void util::xmsg::XEngSurface::XEngErrorCode::set_error_code (
             uint32_t error_code)  [inline], [protected]
```

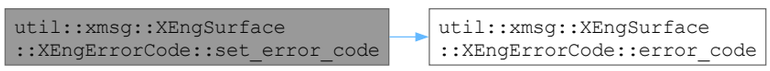
Set the error code.

Parameters

in	*error_code*	The error code.

References error_code(), and error_code_.

Here is the call graph for this function:



10.80.4 Member Data Documentation

10.80.4.1 date_time_

`ptime util::xmsg::XEngSurface::XEngErrorCode::date_time_ [private]`

The timestamp for the error code.

Referenced by date_time(), and set_date_time().

10.80.4.2 error_code_

`uint32_t util::xmsg::XEngSurface::XEngErrorCode::error_code_ [private]`

The error code.

Referenced by error_code(), and set_error_code().

The documentation for this class was generated from the following file:

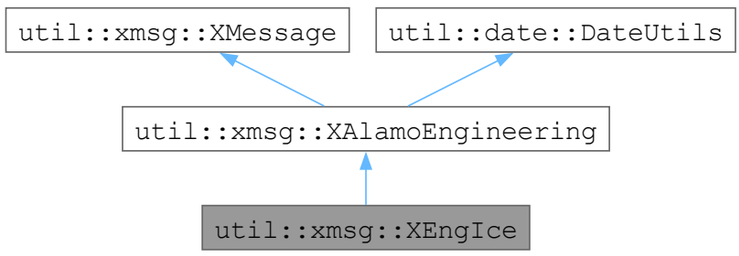
- data/xmsg/XEngSurface.h

10.81 util::xmsg::XEngIce Class Reference

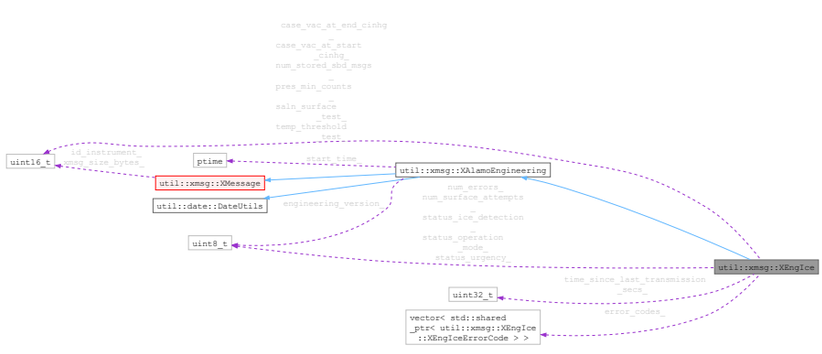
The ALAMO engineering (ice) message class.

`#include <XEngIce.h>`

Inheritance diagram for util::xmsg::XEngIce:



Collaboration diagram for util::xmsg::XEngIce:



Classes

- class XEngIceErrorCode

  The engineering (ice) error code class.

Public Member Functions

- XEngIce (int16_t dive, uint16_t whoi, const vector< uint8_t > &data)

  The parameterized constructor.
- ~XEngIce ()

  The destructor.
- uint16_t **case_vac_at_end_cinhg** () const
- uint16_t **case_vac_at_start_cinhg** () const
- const uint8_t * CheckXMsgData (const vector< uint8_t > &data, int id, int minBytes, bool parseSize=true)

  Check the X-message for the correct ID, minimum size, and message terminator.
- void DecodeXMsgData (const vector< uint8_t > &data)

  Decode an engineering (ice) X-message.
- uint8_t engineering_version () const

  Get the engineering message version identifier.
- float engineering_version_as_float () const

  Get the engineering message verion identifier as a floating point number.
- const vector< shared_ptr< XEngIceErrorCode > > & **error_codes** () const
- const string **GenerateListingString** (uint16_t pg, uint16_t po, uint16_t sg, uint16_t so, uint16_t tg, uint16_t to) const
- template<typename T = float>
  T **GetDecodedCaseVacAtEnd** () const
- template<typename T = float>
  T **GetDecodedCaseVacAtStart** () const
- template<typename T = double>
  T **GetDecodedPresMin** (int16_t gain, int16_t offs) const
- template<typename T = double>
  T **GetDecodedSalnSurfaceTest** (int16_t gain, int16_t offs) const
- template<typename T = double>
  T **GetDecodedTempThresholdTest** (int16_t gain, int16_t offs) const
- int16_t id_dive () const

  Get the dive or cycle number of this X-Message.
- uint16_t id_instrument () const

  Get the unique instrument identifier of this X-Message.
- XMsgTypeId_t id_xmsg_type () const

  Get the enumerated X-Message type identifier.
- uint8_t **num_errors** () const
- uint16_t **num_stored_sbd_msgs** () const
- uint8_t **num_surface_attempts** () const
- uint16_t **pres_min_counts** () const
- uint16_t **saln_surface_test** () const
- const ptime & start_time () const

  Get the engineering message state start time.
- uint8_t **status_ice_detection** () const
- uint8_t **status_operation_mode** () const
- uint8_t **status_urgency** () const
- uint16_t **temp_threshold_test** () const
- uint32_t **time_since_last_transmission_secs** () const
- uint16_t xmsg_size_bytes () const

  Get the size of the X-Message, in bytes.

Static Public Member Functions

- static string GenerateCurrentUtcTimeIso8601String ()

  Generate an ISO8601 format date/time string representation of the current time, in UTC.
- static string GenerateIso8601String (const ptime &p, bool extended=true)

  Generate an ISO8601 format date/time string representation of a POSIX time object.
- static double GenerateJulianDay (const ptime &dt, const ptime &ref_dt)

  Convert a date/time to fractional days since a reference date/time.
- static ptime GetCurrentUtcTime ()

  Generate a POSIX time object representing the current date and time (rounded to the nearest second).

Protected Member Functions

- void **set_case_vac_at_end_cinhg** (uint16_t case_vac_at_end_cinhg)
- void **set_case_vac_at_start_cinhg** (uint16_t case_vac_at_start_cinhg)
- void set_engineering_version (uint8_t engineering_version)

  Set the engineering message version identifier.
- void set_id_dive (int16_t id_dive)

  Set the dive or cycle number of this message.
- void set_id_instrument (uint16_t id_instrument)

  Set the unique instrument identifier.
- void set_id_xmsg_type (XMsgTypeId_t id_xmsg_type)

  Set the X-Message type identifier.
- void **set_num_errors** (uint8_t num_errors)
- void **set_num_stored_sbd_msgs** (uint16_t num_stored_sbd_msgs)
- void **set_num_surface_attempts** (uint8_t num_surface_attempts)
- void **set_pres_min_counts** (uint16_t pres_min_counts)
- void **set_saln_surface_test** (uint16_t saln_surface_test)
- void set_start_time (const ptime &start_time)

  Set the engineering message state start time.
- void **set_status_ice_detection** (uint8_t status_ice_detection)
- void **set_status_operation_mode** (uint8_t status_operation_mode)
- void **set_status_urgency** (uint8_t status_urgency)
- void **set_temp_threshold_test** (uint16_t temp_threshold_test)
- void **set_time_since_last_transmission_secs** (uint32_t time_since_last_transmission_secs)
- void set_xmsg_size_bytes (uint16_t xmsg_size_bytes)

  Set the size of the X-Message, in bytes.

Private Attributes

- uint16_t **case_vac_at_end_cinhg_**
- uint16_t **case_vac_at_start_cinhg_**
- uint8_t engineering_version_

  The engineering message version identifier.
- vector< std::shared_ptr< XEngIceErrorCode > > error_codes_

  The error code values for this dive, if any.
- int16_t id_dive_

  The dive or cycle number of this message.
- uint16_t id_instrument_

  The unique WHOI identifier of this instrument.
- XMsgTypeId_t id_xmsg_type_

  The X-message type identifier.
- uint8_t **num_errors_**
- uint16_t **num_stored_sbd_msgs_**
- uint8_t **num_surface_attempts_**
- uint16_t **pres_min_counts_**
- uint16_t **saln_surface_test_**
- ptime start_time_

  The engineering message state start time.
- uint8_t **status_ice_detection_**
- uint8_t **status_operation_mode_**
- uint8_t **status_urgency_**
- uint16_t **temp_threshold_test_**
- uint32_t **time_since_last_transmission_secs_**
- uint16_t xmsg_size_bytes_

  The size of the X message, in bytes.

10.81.1 Detailed Description

The ALAMO engineering (ice) message class.

An instance of this class represents a decoded ALAMO engineering (ice detection) X-message, which reports the ice detection status, surfacing attempts, threshold test values and any error codes recorded by the instrument.

10.81.2 Constructor & Destructor Documentation

10.81.2.1 XEngIce()

```
util::xmsg::XEngIce::XEngIce (
            int16_t dive,
            uint16_t whoi,
            const vector< uint8_t > & data)  [inline]
```

The parameterized constructor.

Parameters

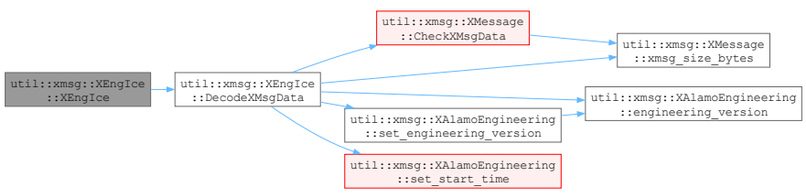
in	*dive*	The dive or cycle number of this message.
in	*whoi*	The unique WHOI identifier.
in	*data*	The binary X-message data to decode.

Exceptions

util::log::MessageException	if an decode error occurs.

References DecodeXMsgData().

Here is the call graph for this function:



10.81.2.2 ~XEngIce()

```
util::xmsg::XEngIce::~XEngIce ()  [inline]
```

The destructor.

10.81.3 Member Function Documentation

10.81.3.1 CheckXMsgData()

```
const uint8_t * util::xmsg::XMessage::CheckXMsgData (
            const vector< uint8_t > & data,
            int id,
            int minBytes,
            bool parseSize = true)  [inline], [inherited]
```

Check the X-message for the correct ID, minimum size, and message terminator.

Parameters

in	*data*	The X-message binary data to check.
in	*id*	The expected ID of the X-message. If this field is less that 0, then the ID check will be skipped.
in	*minBytes*	The minimum number of bytes expected in a valid message.
in	*parseSize*	If TRUE, then the message size field will be parsed from the X-Message. Otherwise, the size field will not be parsed.

Exceptions

util::log::MessageException	if • the message does not contain the expected ID, • the message does not contain the minimum number of expected bytes, • the message does not contain a terminator (';') in the correct location, or, • any other error occurs.

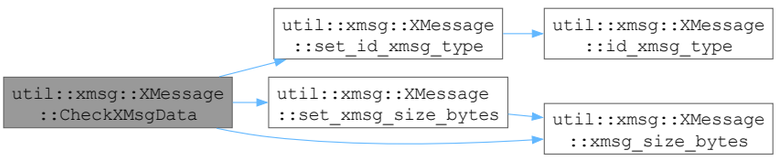
Returns

A pointer to the underlying data buffer of data.

References CATCH_BLOCK, EXC_THROW, lwrn, util::xmsg::XMessage::set_id_xmsg_type(), util::xmsg::XMessage::set_xmsg_size_byte
u16, and util::xmsg::XMessage::xmsg_size_bytes().

Referenced by util::xmsg::XAcousticPosition::DecodeXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XDriftPressure::Dec
util::xmsg::XDriftSalinity::DecodeXMsgData(), util::xmsg::XDriftTemperature::DecodeXMsgData(), util::xmsg::XEngAbort::DecodeXMsgDa
util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngBIT::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(),
util::xmsg::XEngDiagnostic::DecodeXMsgData(), util::xmsg::XEngDrift::DecodeXMsgData(), DecodeXMsgData(),
util::xmsg::XEngStandard::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XGPS::DecodeXMsgData(),
util::xmsg::XHiResPressure::DecodeXMsgData(), util::xmsg::XHiResSalinity::DecodeXMsgData(), util::xmsg::XHiResTemperature::Decod
util::xmsg::XMissionCmdEcho::DecodeXMsgData(), util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XProfilePressure:
util::xmsg::XProfileSalinity::DecodeXMsgData(), util::xmsg::XProfileTemperature::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgD
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the call graph for this function:



10.81.3.2 DecodeXMsgData()

```
void util::xmsg::XEngIce::DecodeXMsgData (
            const vector< uint8_t > & data)  [inline], [virtual]
```

Decode an engineering (ice) X-message.

Byte 3 holds the message version. Versions 0x10 and 0x11 share the fields in bytes 4-19: operation mode, ice detection status, urgency, number of surfacing attempts, number of stored SBD messages, time since the last transmission, minimum pressure, temperature threshold test and surface salinity test. Version 0x11 adds error codes; its layout is described in the 0x11 case below.

Parameters

in	*data*	The binary X-message data to decode.

Exceptions

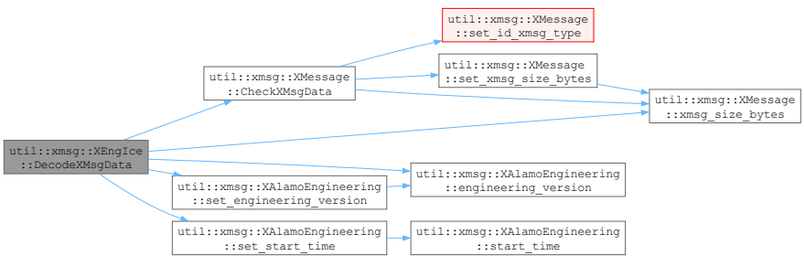
util::log::MessageException	if the message type, size or terminator is invalid, the version is unsupported, or (version 0x11) the error count does not match the number of error code entries.

Implements util::xmsg::XMessage.

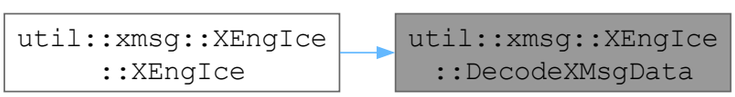
References CATCH_BLOCK, util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XAlamoEngineering::engineering_version(), error_codes_, EXC_THROW, lwrn, s2000, util::xmsg::XAlamoEngineering::set_engineering_version(), util::xmsg::XAlamoEngineering::se u08, u16, u32, and util::xmsg::XMessage::xmsg_size_bytes().

Referenced by XEngIce().

Here is the call graph for this function:



Here is the caller graph for this function:



10.81.3.3 engineering_version()

```
uint8_t util::xmsg::XAlamoEngineering::engineering_version () const  [inline], [inherited]
```

Get the engineering message version identifier.

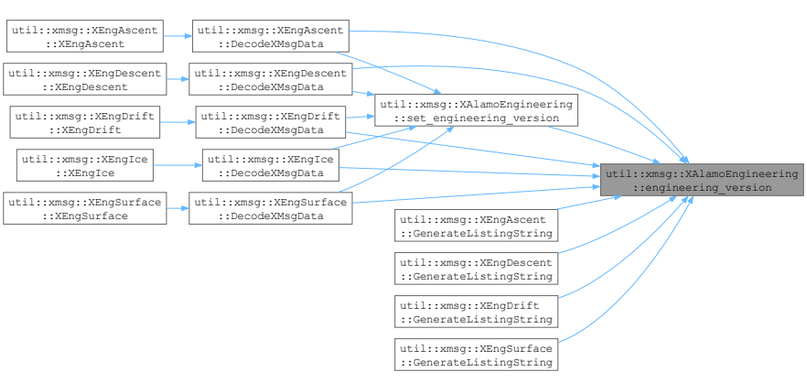
Returns

The engineering message version identifier.

References util::xmsg::XAlamoEngineering::engineering_version_.

Referenced by util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(), util::xmsg::XEngDrift::Decode DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), util::xmsg::XEngSurface::GenerateListin and util::xmsg::XAlamoEngineering::set_engineering_version().

Here is the caller graph for this function:



10.81.3.4 engineering_version_as_float()

```
float util::xmsg::XAlamoEngineering::engineering_version_as_float () const  [inline], [inherited]
```

Get the engineering message verion identifier as a floating point number.

Returns

The engineering message verion identifier as a floating point number.

References util::xmsg::XAlamoEngineering::engineering_version_.

10.81.3.5 GenerateCurrentUtcTimeIso8601String()

```
static string util::date::DateUtils::GenerateCurrentUtcTimeIso8601String ()  [inline], [static],
[inherited]
```

Generate an ISO8601 format date/time string representation of the current time, in UTC.

Note

All date/time objects use the UTC time-zone.

Returns

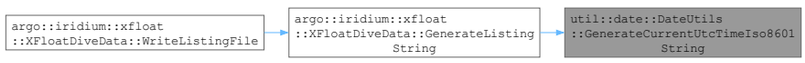
An ISO8601 format date/time string representation of the current time.

References util::date::DateUtils::GenerateIso8601String(), and util::date::DateUtils::GetCurrentUtcTime().

Referenced by argo::iridium::xfloat::XFloatDiveData::GenerateListingString().

Here is the call graph for this function:

Here is the caller graph for this function:



10.81.3.6 GenerateIso8601String()

```
static string util::date::DateUtils::GenerateIso8601String (
            const ptime & p,
            bool extended = true)  [inline], [static], [inherited]
```

Generate an ISO8601 format date/time string representation of a POSIX time object.

Note

All date/time objects use the UTC time-zone.

Parameters

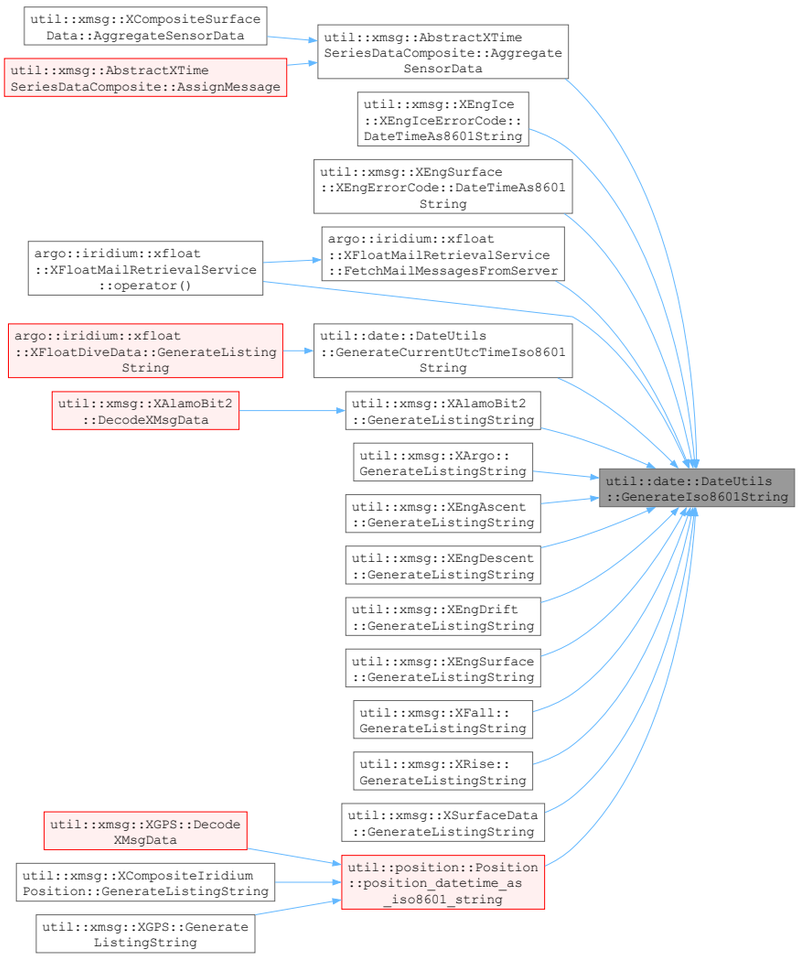
in	*p*	POSIX time from which to generate string.
in	*extended*	If TRUE, use the ISO 8601 extended format (YYYY-MM-DDThh:mm:ssZ); otherwise, use the basic format (YYYYMMDDThhmmss).

Returns

An ISO8601 format date/time string representation of the POSIX time object. If p is equal to boost::posix_↩ time::not_a_date_time, then the string "0000-00-00T00:00:00Z" (extended) or "00000000000000" (basic) will be returned.

Referenced by util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::AggregateSensorData(), util::xmsg::XEngIce::XEngIce util::xmsg::XEngSurface::XEngErrorCode::DateTimeAs8601String(), argo::iridium::xfloat::XFloatMailRetrievalService::FetchMailMessages util::date::DateUtils::GenerateCurrentUtcTimeIso8601String(), util::xmsg::XAlamoBit2::GenerateListingString(), util::xmsg::XArgo::Genera util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListing util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(), argo::iridium::xfloat::XFloatMailRetrievalService::operator()(), and util::position::Position::position_datetime_as_iso8601_string().

Here is the caller graph for this function:



10.81.3.7 GenerateJulianDay()

```
static double util::date::DateUtils::GenerateJulianDay (
            const ptime & dt,
            const ptime & ref_dt)  [inline], [static], [inherited]
```

Convert a date/time to fractional days since a reference date/time.

Parameters

in	*dt*	The date/time to convert.
in	*ref_dt*	The reference date/time (day 0).

Exceptions

util::log::MessageException	if either date/time is not valid.

Returns

The number of days (with millisecond resolution) from `ref_dt` to `dt`.

References CATCH_BLOCK, and EXC_THROW.

10.81.3.8 GetCurrentUtcTime()

```
static ptime util::date::DateUtils::GetCurrentUtcTime ()  [inline], [static], [inherited]
```

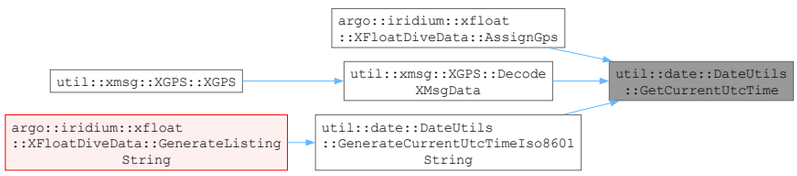
Generate a POSIX time object representing the current date and time (rounded to the nearest second).

Returns

A POSIX time object representing the current date and time (rounded to the nearest second).

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignGps(), util::xmsg::XGPS::DecodeXMsgData(), and util::date::DateUtils::GenerateCurrentUtcTimeIso8601String().

Here is the caller graph for this function:



10.81.3.9 id_dive()

```
int16_t util::xmsg::XMessage::id_dive () const  [inline], [inherited]
```

Get the dive or cycle number of this X-Message.

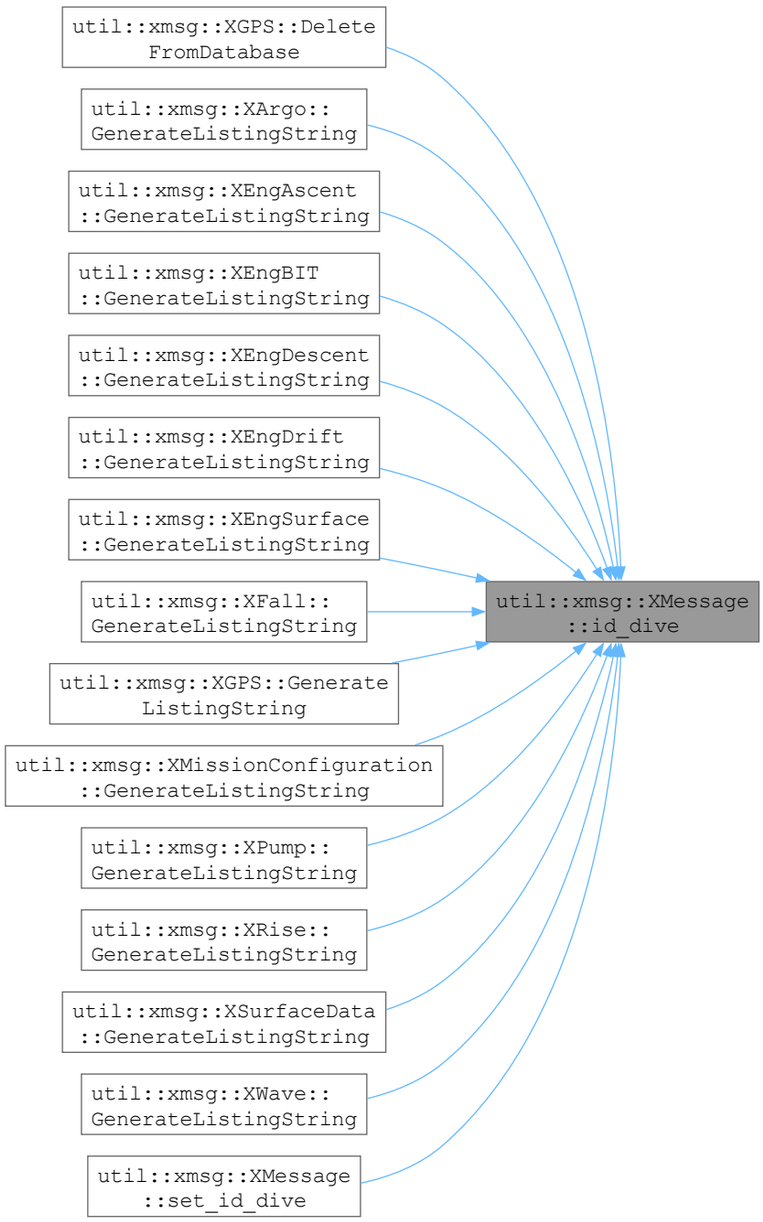
Returns

The dive or cycle number of this X-Message.

References util::xmsg::XMessage::id_dive_.

Referenced by util::xmsg::XGPS::DeleteFromDatabase(), util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateLis
util::xmsg::XEngBIT::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingStr
util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(),
util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing
util::xmsg::XSurfaceData::GenerateListingString(), util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_dive().

Here is the caller graph for this function:



10.81.3.10 id_instrument()

```
uint16_t util::xmsg::XMessage::id_instrument () const  [inline], [inherited]
```

Get the unique instrument identifier of this X-Message.

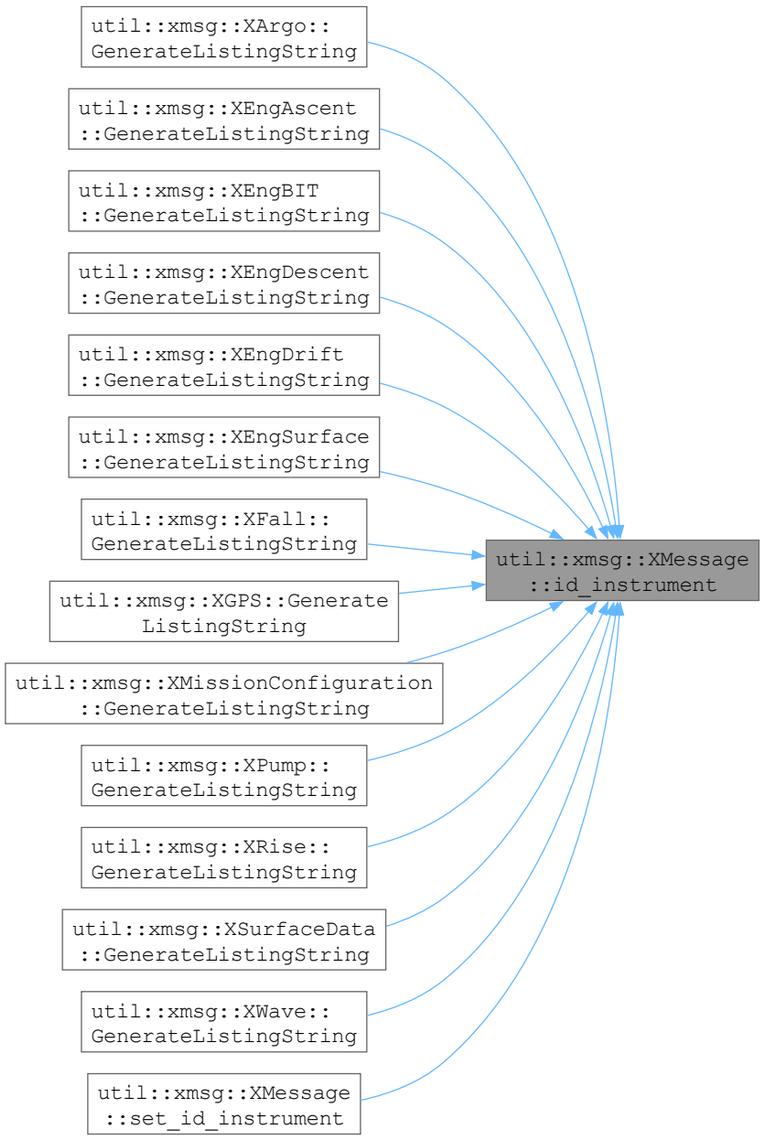
Returns

The unique instrument identifier of this X-Message.

References util::xmsg::XMessage::id_instrument_.

Referenced by util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngBIT::Generate
util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), util::xmsg::XEngSurface::GenerateListin
util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingS
util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(),
util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_instrument().

Here is the caller graph for this function:



10.81.3.11 id_xmsg_type()

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type () const  [inline], [inherited]
```

Get the enumerated X-Message type identifier.

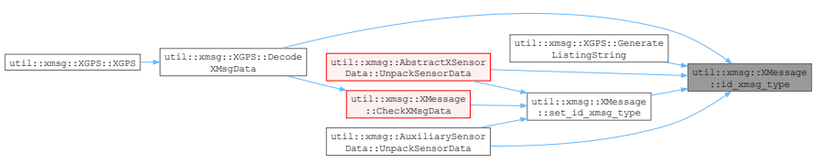
Returns

The enumerated X-Message type identifier.

References util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMessage::set_id_xmsg_typ
util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the caller graph for this function:



10.81.3.12 set_engineering_version()

```
void util::xmsg::XAlamoEngineering::set_engineering_version (
            uint8_t engineering_version)  [inline], [protected], [inherited]
```

Set the engineering message version identifier.

Parameters

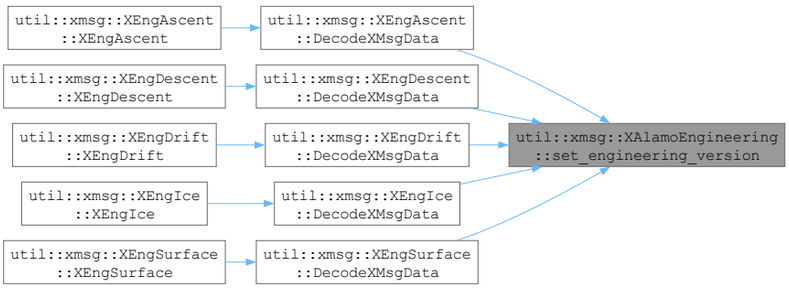
in	*engineering_version*	The engineering message version identifier.

References util::xmsg::XAlamoEngineering::engineering_version(), and util::xmsg::XAlamoEngineering::engineering_version_.

Referenced by util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(), util::xmsg::XEngDrift::Decode
DecodeXMsgData(), and util::xmsg::XEngSurface::DecodeXMsgData().

Here is the call graph for this function:

Here is the caller graph for this function:



10.81.3.13 set_id_dive()

```
void util::xmsg::XMessage::set_id_dive (
            int16_t id_dive)  [inline], [protected], [inherited]
```

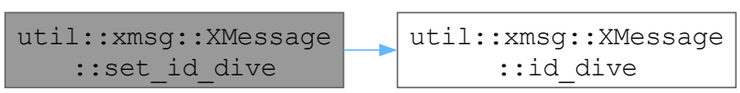
Set the dive or cycle number of this message.

Parameters

in	*id_dive*	The dive or cycle number of this message to set.

References util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::id_dive_.

Here is the call graph for this function:



10.81.3.14 set_id_instrument()

```
void util::xmsg::XMessage::set_id_instrument (
            uint16_t id_instrument)  [inline], [protected], [inherited]
```

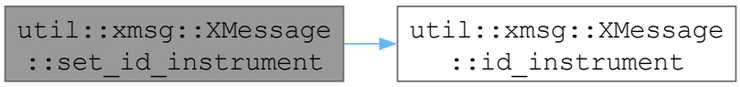
Set the unique instrument identifier.

Parameters

in	*id_instrument*	The unique instrument identifier to set.

References util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::id_instrument_.

Here is the call graph for this function:



10.81.3.15 set_id_xmsg_type()

```
void util::xmsg::XMessage::set_id_xmsg_type (
            XMsgTypeId_t id_xmsg_type)  [inline], [protected], [inherited]
```

Set the X-Message type identifier.

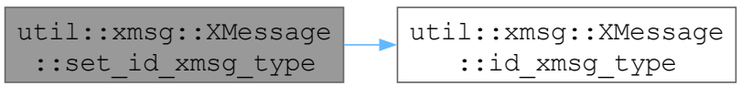
Parameters

in	*id_xmsg_type*	The X-Message type identifier to set.

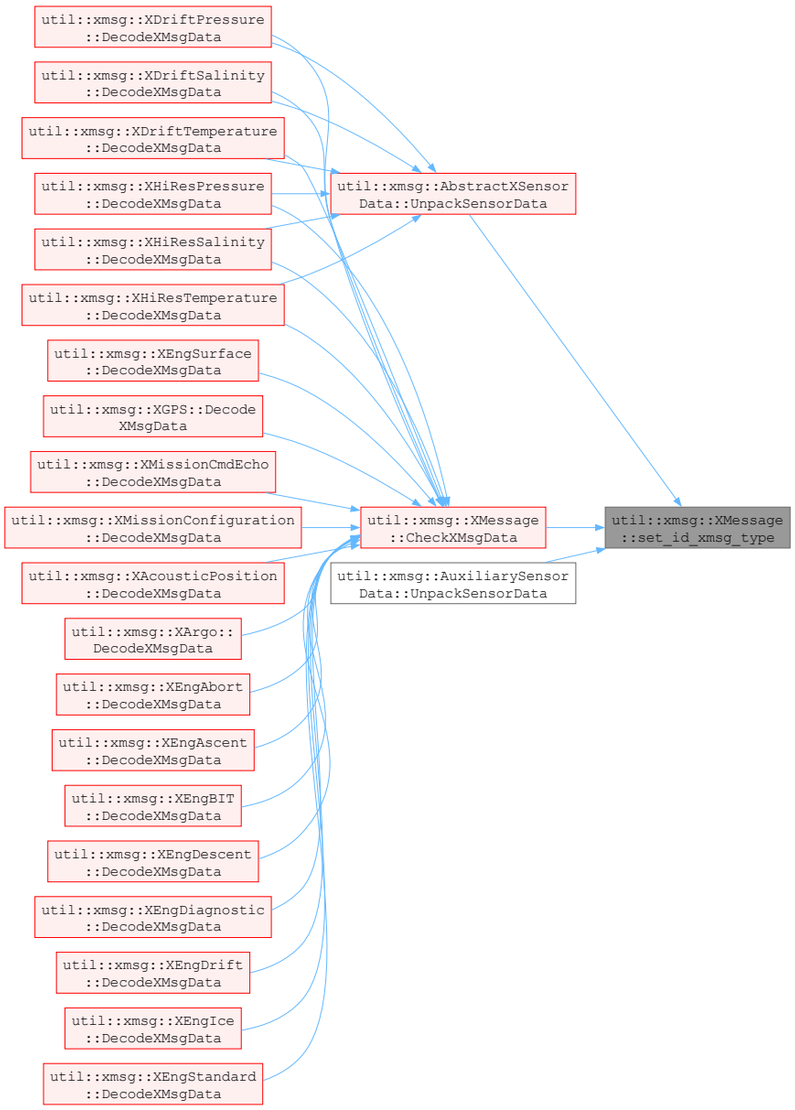
References util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.81.3.16 set_start_time()

```
void util::xmsg::XAlamoEngineering::set_start_time (
            const ptime & start_time)  [inline], [protected], [inherited]
```

Set the engineering message state start time.

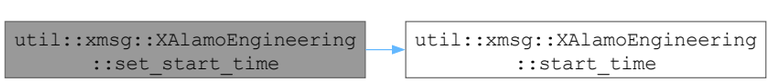
Parameters

in	*start_time*	The engineering message state start time.

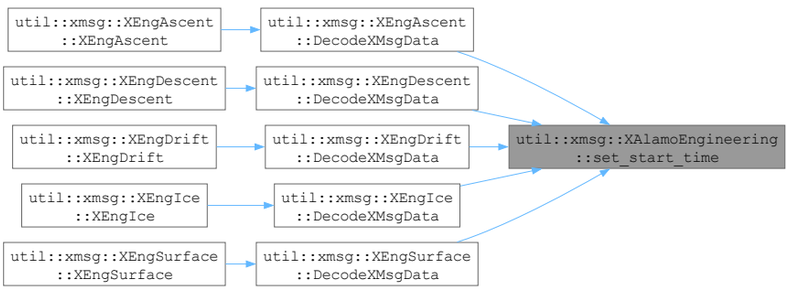
References util::xmsg::XAlamoEngineering::start_time(), and util::xmsg::XAlamoEngineering::start_time_.

Referenced by util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(), util::xmsg::XEngDrift::Decode DecodeXMsgData(), and util::xmsg::XEngSurface::DecodeXMsgData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.81.3.17 set_xmsg_size_bytes()

```
void util::xmsg::XMessage::set_xmsg_size_bytes (
            uint16_t xmsg_size_bytes)  [inline], [protected], [inherited]
```

Set the size of the X-Message, in bytes.

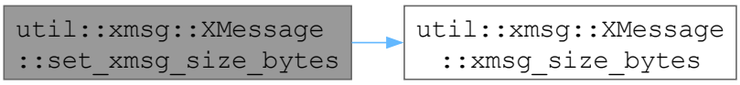
Parameters

in	*xmsg_size_bytes*	The size of the X-Message to set, in bytes.

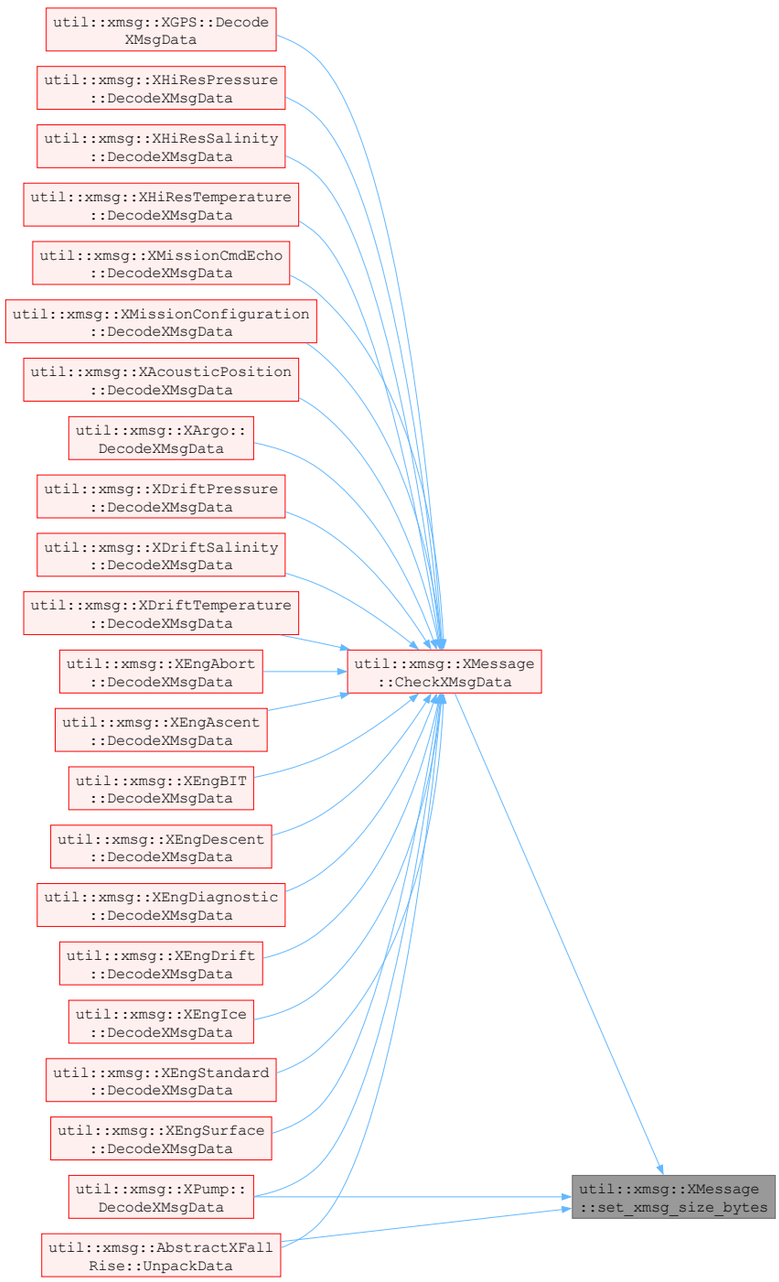
References util::xmsg::XMessage::xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XPump::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::Unpa

Here is the call graph for this function:



Here is the caller graph for this function:



10.81.3.18 start_time()

```
const ptime & util::xmsg::XAlamoEngineering::start_time () const  [inline], [inherited]
```

Get the engineering message state start time.

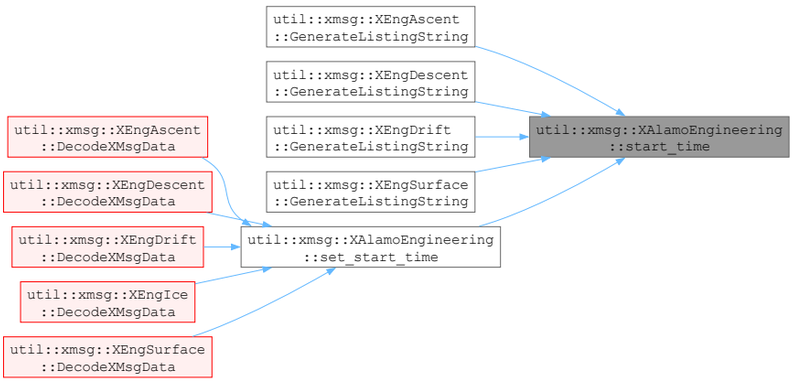
Returns

The engineering message state start time.

References util::xmsg::XAlamoEngineering::start_time_.

Referenced by util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), util::xmsg::XEngSurface::GenerateListingString(), and util::xmsg::XAlamoEngineering::set_

Here is the caller graph for this function:



10.81.3.19 xmsg_size_bytes()

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes () const  [inline], [inherited]
```

Get the size of the X-Message, in bytes.

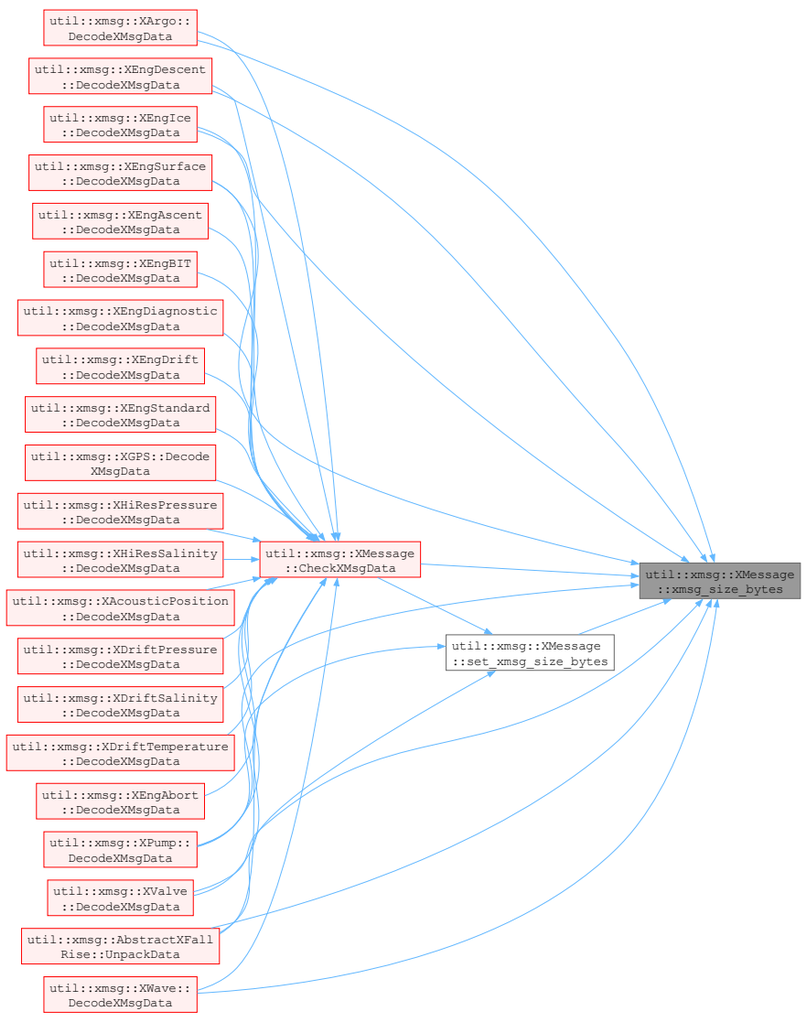
Returns

The size of the X-Message, in bytes.

References util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgI DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(), util::xmsg::XValve::DecodeXI util::xmsg::XWave::DecodeXMsgData(), util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the caller graph for this function:



10.81.4 Member Data Documentation

10.81.4.1 engineering_version_

```
uint8_t util::xmsg::XAlamoEngineering::engineering_version_  [private], [inherited]
```

The engineering message version identifier.

Referenced by util::xmsg::XAlamoEngineering::engineering_version(), util::xmsg::XAlamoEngineering::engineering_version_as_float(), and util::xmsg::XAlamoEngineering::set_engineering_version().

10.81.4.2 error_codes_

```
vector<std::shared_ptr<XEngIceErrorCode> > util::xmsg::XEngIce::error_codes_  [private]
```

The error code values for this dive, if any.

Referenced by DecodeXMsgData().

10.81.4.3 id_dive_

```
int16_t util::xmsg::XMessage::id_dive_  [private], [inherited]
```

The dive or cycle number of this message.

Referenced by util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::set_id_dive().

10.81.4.4 id_instrument_

```
uint16_t util::xmsg::XMessage::id_instrument_  [private], [inherited]
```

The unique WHOI identifier of this instrument.

Referenced by util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::set_id_instrument().

10.81.4.5 id_xmsg_type_

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type_  [private], [inherited]
```

The X-message type identifier.

Referenced by util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::set_id_xmsg_type().

10.81.4.6 start_time_

```
ptime util::xmsg::XAlamoEngineering::start_time_  [private], [inherited]
```

The engineering message state start time.

Referenced by util::xmsg::XAlamoEngineering::set_start_time(), and util::xmsg::XAlamoEngineering::start_time().

10.81.4.7 xmsg_size_bytes_

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes_  [private], [inherited]
```

The size of the X message, in bytes.

Referenced by util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes().

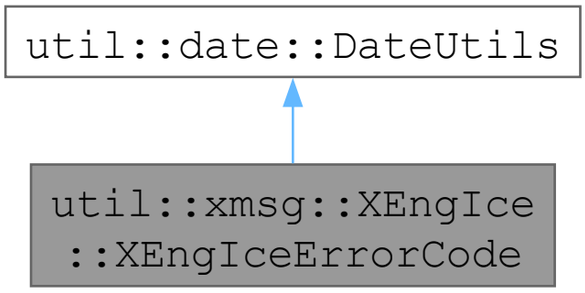
The documentation for this class was generated from the following file:

- data/xmsg/XEngIce.h

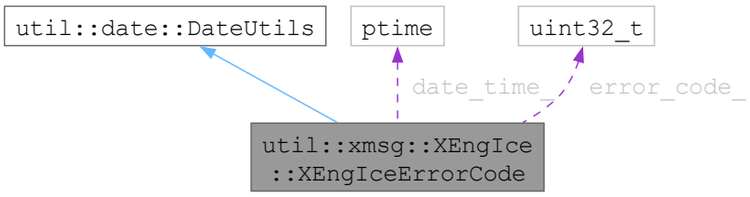
10.82 util::xmsg::XEngIce::XEngIceErrorCode Class Reference

The engineering (ice) error code class.

Inheritance diagram for util::xmsg::XEngIce::XEngIceErrorCode:



Collaboration diagram for util::xmsg::XEngIce::XEngIceErrorCode:



Public Member Functions

- XEngIceErrorCode (ptime date_time, uint32_t error_code)
  The parameterized constructor.
- ~XEngIceErrorCode ()
  The destructor.
- const ptime & date_time () const
  Get the timestamp for the error code.
- string DateTimeAs8601String () const
  Generate an ISO8601 format date/time string representation of the error date/time.
- uint32_t error_code () const
  Get the error code.

Static Public Member Functions

- static string GenerateCurrentUtcTimeIso8601String ()
  Generate an ISO8601 format date/time string representation of the current time, in UTC.
- static string GenerateIso8601String (const ptime &p, bool extended=true)
  Generate an ISO8601 format date/time string representation of a POSIX time object.
- static double GenerateJulianDay (const ptime &dt, const ptime &ref_dt)
  Convert a date/time to fractional days since a reference date/time.
- static ptime GetCurrentUtcTime ()
  Generate a POSIX time object representing the current date and time (rounded to the nearest second).

Protected Member Functions

- void set_date_time (const ptime &date_time)
  Set the timestamp for the error code.
- void set_error_code (uint32_t error_code)
  Set the error code.

Private Attributes

- ptime date_time_
  The timestamp for the error code.
- uint32_t error_code_
  The error code.

10.82.1 Detailed Description

The engineering (ice) error code class.

An instance of this object represents a single error code reported in the ALAMO engineering (ice) data message. Each error consists of a timestamp and a 16-bit error code.

10.82.2 Constructor & Destructor Documentation

10.82.2.1 XEngIceErrorCode()

```
util::xmsg::XEngIce::XEngIceErrorCode::XEngIceErrorCode (
            ptime date_time,
            uint32_t error_code)  [inline]
```

The parameterized constructor.

Parameters

in	*date_time*	The timestamp of the error code.
in	*error_code*	The error code.

10.82.2.2 ~XEngIceErrorCode()

```
util::xmsg::XEngIce::XEngIceErrorCode::~XEngIceErrorCode ()  [inline]
```

The destructor.

10.82.3 Member Function Documentation

10.82.3.1 date_time()

```
const ptime & util::xmsg::XEngIce::XEngIceErrorCode::date_time () const  [inline]
```

Get the timestamp for the error code.

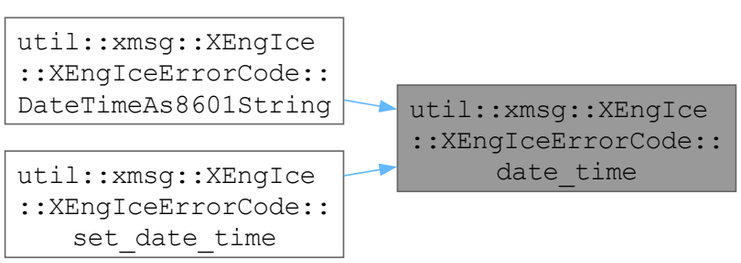
Returns

The timestamp for the error code.

References date_time_.

Referenced by DateTimeAs8601String(), and set_date_time().

Here is the caller graph for this function:



10.82.3.2 DateTimeAs8601String()

```
string util::xmsg::XEngIce::XEngIceErrorCode::DateTimeAs8601String () const  [inline]
```

Generate an ISO8601 format date/time string representation of the error date/time.

Note

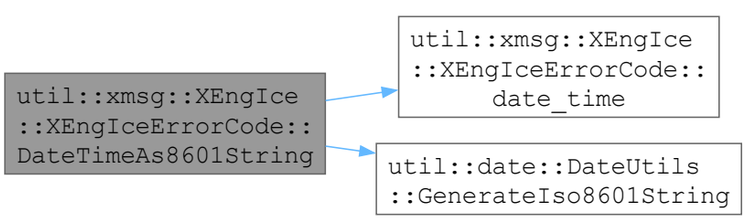
All date/time objects use the UTC time-zone.

Returns

An ISO8601 format date/time string representation of the error date/time. If date_time() returns boost::posix_↩ time::not_a_date_time, then the string "0000-00-00T00:00:00Z" will be returned.

References date_time(), and util::date::DateUtils::GenerateIso8601String().

Here is the call graph for this function:



10.82.3.3 error_code()

```
uint32_t util::xmsg::XEngIce::XEngIceErrorCode::error_code () const  [inline]
```

Get the error code.

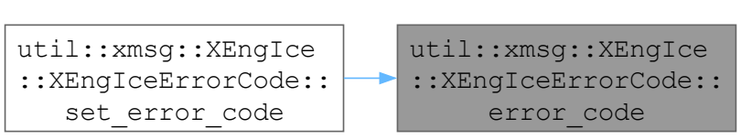
Returns

The error code.

References error_code_.

Referenced by set_error_code().

Here is the caller graph for this function:



10.82.3.4 GenerateCurrentUtcTimeIso8601String()

```
static string util::date::DateUtils::GenerateCurrentUtcTimeIso8601String ()  [inline], [static],
[inherited]
```

Generate an ISO8601 format date/time string representation of the current time, in UTC.

Note

All date/time objects use the UTC time-zone.

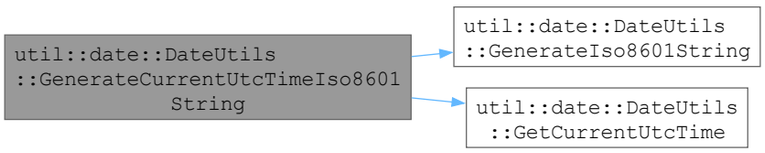
Returns

An ISO8601 format date/time string representation of the current time.

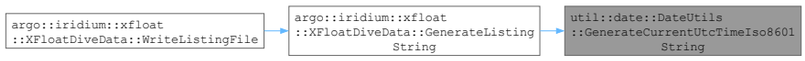
References util::date::DateUtils::GenerateIso8601String(), and util::date::DateUtils::GetCurrentUtcTime().

Referenced by argo::iridium::xfloat::XFloatDiveData::GenerateListingString().

Here is the call graph for this function:



Here is the caller graph for this function:



10.82.3.5 GenerateIso8601String()

```
static string util::date::DateUtils::GenerateIso8601String (
            const ptime & p,
            bool extended = true)  [inline], [static], [inherited]
```

Generate an ISO8601 format date/time string representation of a POSIX time object.

Note

All date/time objects use the UTC time-zone.

Parameters

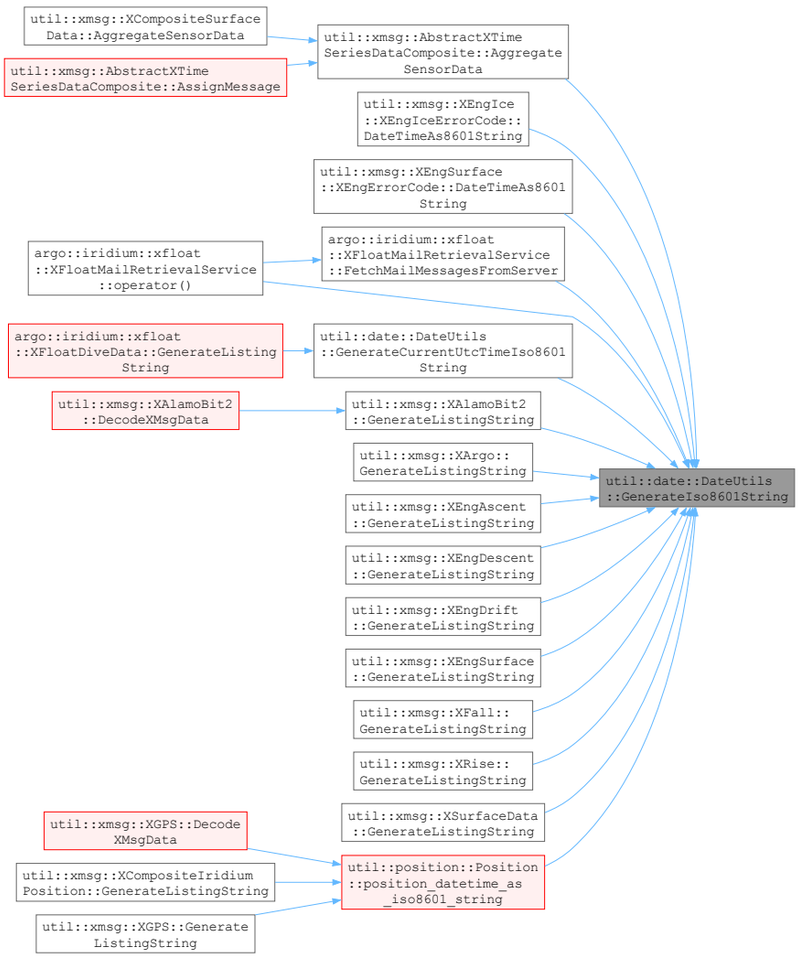
in	*p*	POSIX time from which to generate string.
in	*extended*	If TRUE, use the ISO 8601 extended format (YYYY-MM-DDThh:mm:ssZ); otherwise, use the basic format (YYYYMMDDThhmmss).

Returns

An ISO8601 format date/time string representation of the POSIX time object. If p is equal to boost::posix_↩ time::not_a_date_time, then the string "0000-00-00T00:00:00Z" (extended) or "00000000000000" (basic) will be returned.

Referenced by util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::AggregateSensorData(), DateTimeAs8601String(), util::xmsg::XEngSurface::XEngErrorCode::DateTimeAs8601String(), argo::iridium::xfloat::XFloatMailRetrievalService::FetchMailMessages util::date::DateUtils::GenerateCurrentUtcTimeIso8601String(), util::xmsg::XAlamoBit2::GenerateListingString(), util::xmsg::XArgo::Genera util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListing util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(), argo::iridium::xfloat::XFloatMailRetrievalService::operator()(), and util::position::Position::position_datetime_as_iso8601_string().

Here is the caller graph for this function:



10.82.3.6 GenerateJulianDay()

```
static double util::date::DateUtils::GenerateJulianDay (
            const ptime & dt,
            const ptime & ref_dt)  [inline], [static], [inherited]
```

Convert a date/time to fractional days since a reference date/time.

Parameters

in	*dt*	The date/time to convert.
in	*ref↩ _dt*	The reference date/time (day 0).

Exceptions

util::log::MessageException	if either date/time is not valid.

Returns

The number of days (with millisecond resolution) from `ref_dt` to `dt`.

References CATCH_BLOCK, and EXC_THROW.

10.82.3.7 GetCurrentUtcTime()

```
static ptime util::date::DateUtils::GetCurrentUtcTime ()  [inline], [static], [inherited]
```

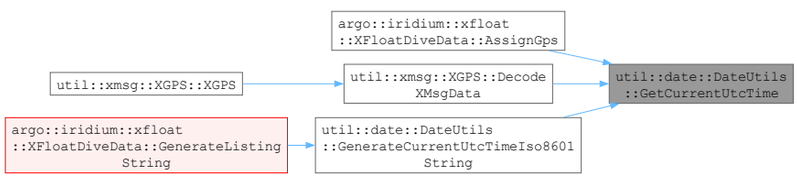
Generate a POSIX time object representing the current date and time (rounded to the nearest second).

Returns

A POSIX time object representing the current date and time (rounded to the nearest second).

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignGps(), util::xmsg::XGPS::DecodeXMsgData(), and util::date::DateUtils::GenerateCurrentUtcTimeIso8601String().

Here is the caller graph for this function:



10.82.3.8 set_date_time()

```
void util::xmsg::XEngIce::XEngIceErrorCode::set_date_time (
            const ptime & date_time)  [inline], [protected]
```

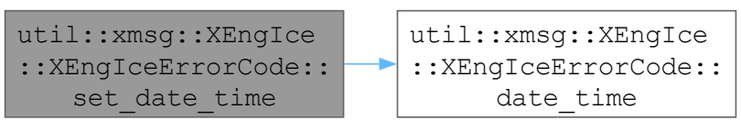
Set the timestamp for the error code.

Parameters

in	*date_time*	The timestamp for the error code.

References date_time(), and date_time_.

Here is the call graph for this function:



10.82.3.9 set_error_code()

```
void util::xmsg::XEngIce::XEngIceErrorCode::set_error_code (
            uint32_t error_code)  [inline], [protected]
```

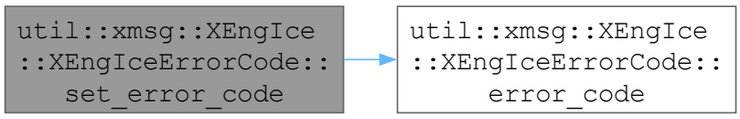
Set the error code.

Parameters

in	*error_code*	The error code.

References error_code(), and error_code_.

Here is the call graph for this function:



10.82.4 Member Data Documentation

10.82.4.1 date_time_

```
ptime util::xmsg::XEngIce::XEngIceErrorCode::date_time_  [private]
```

The timestamp for the error code.

Referenced by date_time(), and set_date_time().

10.82.4.2 error_code_

```
uint32_t util::xmsg::XEngIce::XEngIceErrorCode::error_code_  [private]
```

The error code.

Referenced by error_code(), and set_error_code().

The documentation for this class was generated from the following file:

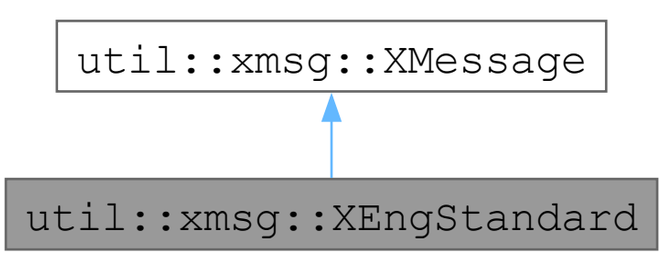
- data/xmsg/XEngIce.h

10.83 util::xmsg::XEngStandard Class Reference

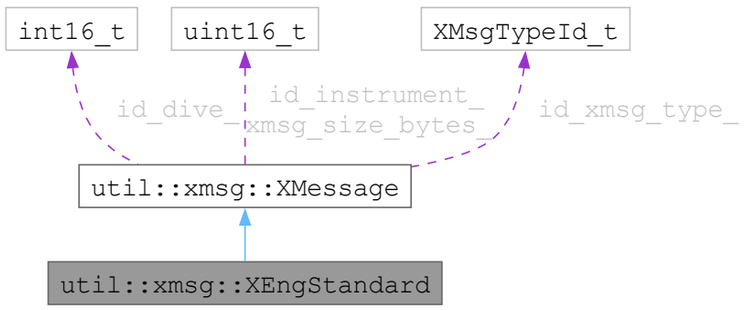
Class representing an engineering message for a standard dive cycle.

```
#include <XEngStandard.h>
```

Inheritance diagram for util::xmsg::XEngStandard:



Collaboration diagram for util::xmsg::XEngStandard:



Public Member Functions

- XEngStandard (uint16_t dive, uint16_t whoi, const std::vector< uint8_t > &data)

  Parameterized constructor.
- ~XEngStandard ()

  Destructor.
- const uint8_t * CheckXMsgData (const vector< uint8_t > &data, int id, int minBytes, bool parseSize=true)

  Check the X-message for the correct ID, minimum size, and message terminator.
- void DecodeXMsgData (const std::vector< uint8_t > &data)

  Decode the standard dive engineering X-Message from the binary data.
- virtual void DecodeXMsgData (const vector< uint8_t > &data)=0

  Decode the X-Message data.
- const string GenerateListingString () const

  Generate a listing string representation of the standard dive engineering data message.
- int16_t id_dive () const

  Get the dive or cycle number of this X-Message.
- uint16_t id_instrument () const

  Get the unique instrument identifier of this X-Message.
- XMsgTypeId_t id_xmsg_type () const

  Get the enumerated X-Message type identifier.
- uint16_t xmsg_size_bytes () const

  Get the size of the X-Message, in bytes.

Protected Member Functions

- void set_id_dive (int16_t id_dive)

  Set the dive or cycle number of this message.
- void set_id_instrument (uint16_t id_instrument)

  Set the unique instrument identifier.
- void set_id_xmsg_type (XMsgTypeId_t id_xmsg_type)

  Set the X-Message type identifier.
- void set_xmsg_size_bytes (uint16_t xmsg_size_bytes)

  Set the size of the X-Message, in bytes.

Private Attributes

- int16_t id_dive_

  The dive or cycle number of this message.
- uint16_t id_instrument_

  The unique WHOI identifier of this instrument.
- XMsgTypeId_t id_xmsg_type_

  The X-message type identifier.
- uint16_t xmsg_size_bytes_

  The size of the X message, in bytes.

10.83.1 Detailed Description

Class representing an engineering message for a standard dive cycle.

10.83.2 Constructor & Destructor Documentation

10.83.2.1 XEngStandard()

```
util::xmsg::XEngStandard::XEngStandard (
            uint16_t dive,
            uint16_t whoi,
            const std::vector< uint8_t > & data)  [inline]
```

Parameterized constructor.

Parameters

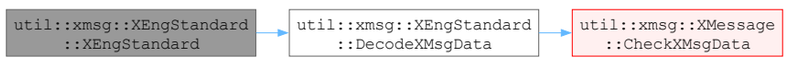
in	*dive*	The dive or cycle number of this message.
in	*whoi*	The unique WHOI identifier.
in	*data*	The binary X-message data to decode.

Exceptions

util::log::MessageException	if an decode error occurs.

References DecodeXMsgData().

Here is the call graph for this function:



10.83.2.2 ~XEngStandard()

```
util::xmsg::XEngStandard::~XEngStandard ()  [inline]
```

Destructor.

10.83.3 Member Function Documentation

10.83.3.1 CheckXMsgData()

```
const uint8_t * util::xmsg::XMessage::CheckXMsgData (
            const vector< uint8_t > & data,
            int id,
            int minBytes,
            bool parseSize = true)  [inline], [inherited]
```

Check the X-message for the correct ID, minimum size, and message terminator.

Parameters

in	*data*	The X-message binary data to check.
in	*id*	The expected ID of the X-message. If this field is less that 0, then the ID check will be skipped.
in	*minBytes*	The minimum number of bytes expected in a valid message.
in	*parseSize*	If TRUE, then the message size field will be parsed from the X-Message. Otherwise, the size field will not be parsed.

Exceptions

util::log::MessageException	if • the message does not contain the expected ID, • the message does not contain the minimum number of expected bytes, • the message does not contain a terminator (';') in the correct location, or, • any other error occurs.

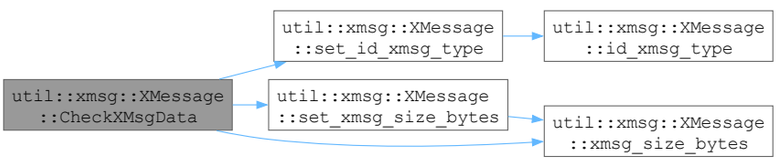
Returns

A pointer to the underlying data buffer of data.

References CATCH_BLOCK, EXC_THROW, lwrn, util::xmsg::XMessage::set_id_xmsg_type(), util::xmsg::XMessage::set_xmsg_size_byte
u16, and util::xmsg::XMessage::xmsg_size_bytes().

Referenced by util::xmsg::XAcousticPosition::DecodeXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XDriftPressure::Dec
util::xmsg::XDriftSalinity::DecodeXMsgData(), util::xmsg::XDriftTemperature::DecodeXMsgData(), util::xmsg::XEngAbort::DecodeXMsgDa
util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngBIT::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(),
util::xmsg::XEngDiagnostic::DecodeXMsgData(), util::xmsg::XEngDrift::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(),
DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XHiResPressure::D
util::xmsg::XHiResSalinity::DecodeXMsgData(), util::xmsg::XHiResTemperature::DecodeXMsgData(), util::xmsg::XMissionCmdEcho::Dec
util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XProfilePressure::DecodeXMsgData(), util::xmsg::XProfileSalinity::Dec
util::xmsg::XProfileTemperature::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(), util::xmsg::XValve::DecodeXMsgData(),
util::xmsg::XWave::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the call graph for this function:



10.83.3.2 DecodeXMsgData() [1/2]

```
void util::xmsg::XEngStandard::DecodeXMsgData (
            const std::vector< uint8_t > & data)  [inline]
```

Decode the standard dive engineering X-Message from the binary data.

Parameters

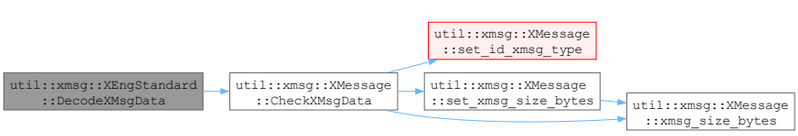
in	*data*	The binary message to decode.

Todo Implement the functionality for decoding the Solo-II standard dive/cycle engineering data message.

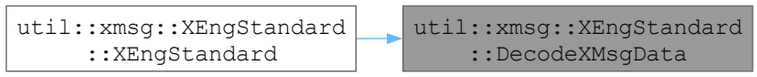
References CATCH_BLOCK, util::xmsg::XMessage::CheckXMsgData(), and util::xmsg::XMSG_TYPE_ID_ENG_DIVE.

Referenced by XEngStandard().

Here is the call graph for this function:



Here is the caller graph for this function:



10.83.3.3 DecodeXMsgData() [2/2]

```
virtual void util::xmsg::XMessage::DecodeXMsgData (
            const vector< uint8_t > & data)  [pure virtual], [inherited]
```

Decode the X-Message data.

Parameters

in	*data*	The encoded X message to decode.

Implemented in util::xmsg::AbstractXFallRise, util::xmsg::AbstractXSensorData, util::xmsg::XAcousticPosition, util::xmsg::XArgo, util::xmsg::XDriftPressure, util::xmsg::XDriftSalinity, util::xmsg::XDriftTemperature, util::xmsg::XEngAbort, util::xmsg::XEngAscent, util::xmsg::XEngBIT, util::xmsg::XEngDescent, util::xmsg::XEngDiagnostic, util::xmsg::XEngDrift, util::xmsg::XEngIce, util::xmsg::XEngSurface, util::xmsg::XFall, util::xmsg::XGPS, util::xmsg::XHiResPressure, util::xmsg::XHiResSalinity, util::xmsg::XHiResTemperature, util::xmsg::XMessageIndexable, util::xmsg::XMissionCmdEcho, util::xmsg::XMissionConfiguration, util::xmsg::XProfilePressure, util::xmsg::XProfileSalinity, util::xmsg::XProfileTemperature, util::xmsg::XPump, util::xmsg::XRise, util::xmsg::XSurfaceData, util::xmsg::XValve, and util::xmsg::XWave.

10.83.3.4 GenerateListingString()

```
const string util::xmsg::XEngStandard::GenerateListingString () const  [inline]
```

Generate a listing string representation of the standard dive engineering data message.

Returns

A listing string representation of the standard dive engineering data message.

Todo Implement the functionality for generating a listing string for the Solo-II standard dive engineering data message.

References CATCH_BLOCK.

10.83.3.5 id_dive()

```
int16_t util::xmsg::XMessage::id_dive () const  [inline], [inherited]
```

Get the dive or cycle number of this X-Message.

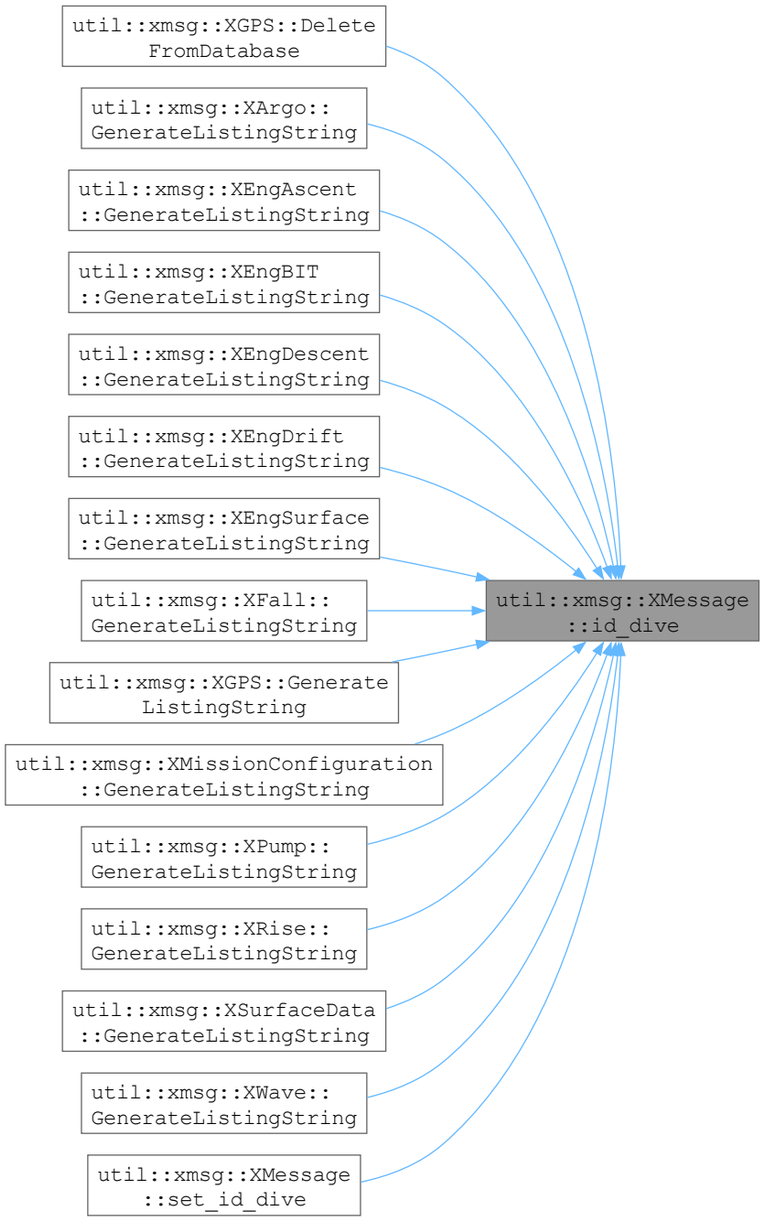
Returns

The dive or cycle number of this X-Message.

References util::xmsg::XMessage::id_dive_.

Referenced by util::xmsg::XGPS::DeleteFromDatabase(), util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateLis
util::xmsg::XEngBIT::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingStr
util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(),
util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing
util::xmsg::XSurfaceData::GenerateListingString(), util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_dive().

Here is the caller graph for this function:



10.83.3.6 id_instrument()

```
uint16_t util::xmsg::XMessage::id_instrument () const  [inline], [inherited]
```

Get the unique instrument identifier of this X-Message.

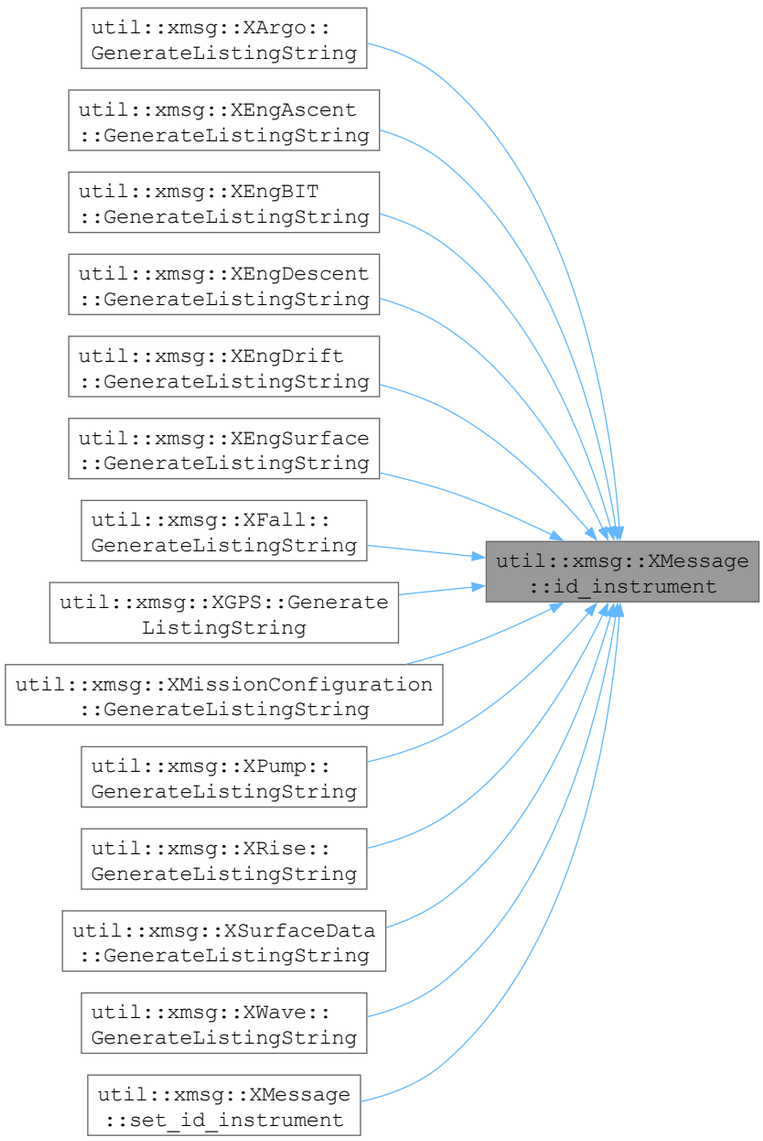
Returns

The unique instrument identifier of this X-Message.

References util::xmsg::XMessage::id_instrument_.

Referenced by util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngBIT::Generate
util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), util::xmsg::XEngSurface::GenerateListin
util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingS
util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(),
util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_instrument().

Here is the caller graph for this function:



10.83.3.7 id_xmsg_type()

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type () const  [inline], [inherited]
```

Get the enumerated X-Message type identifier.

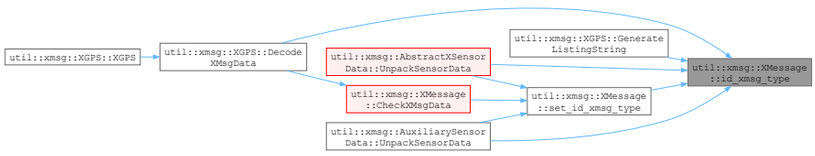
Returns

The enumerated X-Message type identifier.

References util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMessage::set_id_xmsg_typ
util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the caller graph for this function:



10.83.3.8 set_id_dive()

```
void util::xmsg::XMessage::set_id_dive (
            int16_t id_dive)  [inline], [protected], [inherited]
```

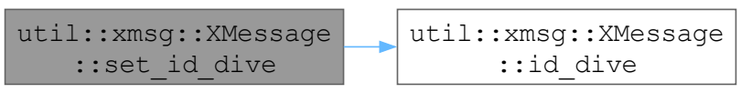
Set the dive or cycle number of this message.

Parameters

in	*id_dive*	The dive or cycle number of this message to set.

References util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::id_dive_.

Here is the call graph for this function:



10.83.3.9 set_id_instrument()

```
void util::xmsg::XMessage::set_id_instrument (
            uint16_t id_instrument)  [inline], [protected], [inherited]
```

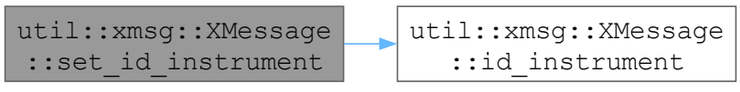
Set the unique instrument identifier.

Parameters

in	*id_instrument*	The unique instrument identifier to set.

References util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::id_instrument_.

Here is the call graph for this function:



10.83.3.10 set_id_xmsg_type()

```
void util::xmsg::XMessage::set_id_xmsg_type (
            XMsgTypeId_t id_xmsg_type)  [inline], [protected], [inherited]
```

Set the X-Message type identifier.

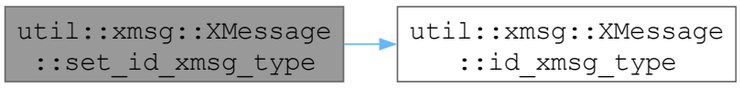
Parameters

in	*id_xmsg_type*	The X-Message type identifier to set.

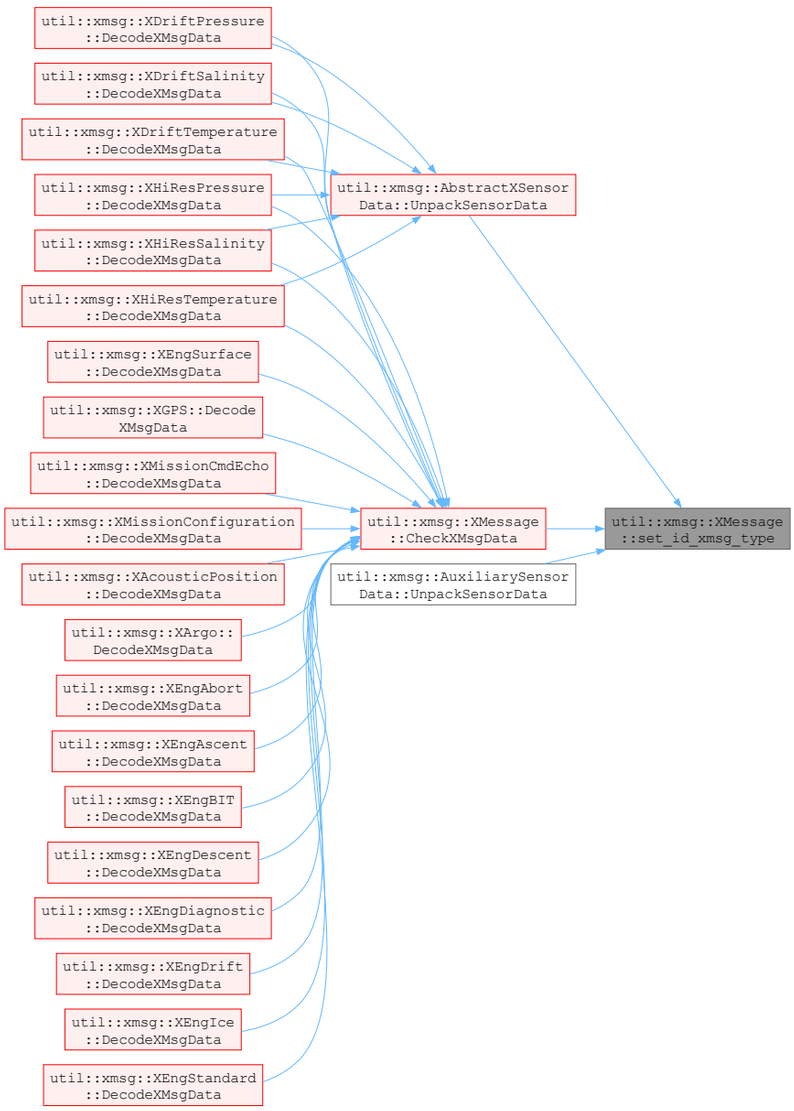
References util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.83.3.11 set_xmsg_size_bytes()

```
void util::xmsg::XMessage::set_xmsg_size_bytes (
            uint16_t xmsg_size_bytes)  [inline], [protected], [inherited]
```

Set the size of the X-Message, in bytes.

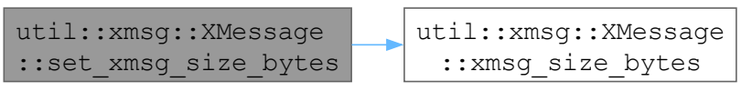
Parameters

in	*xmsg_size_bytes*	The size of the X-Message to set, in bytes.

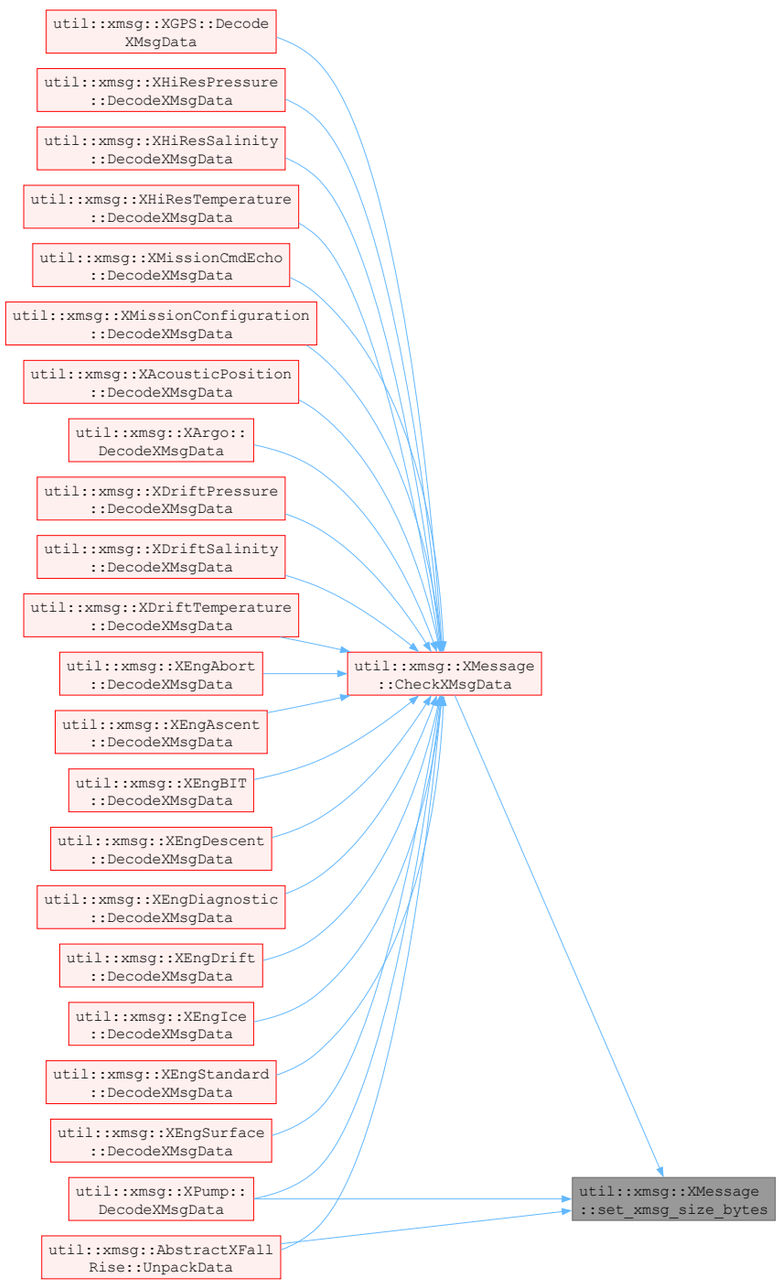
References util::xmsg::XMessage::xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XPump::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::Unpa

Here is the call graph for this function:



Here is the caller graph for this function:



10.83.3.12 xmsg_size_bytes()

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes () const  [inline], [inherited]
```

Get the size of the X-Message, in bytes.

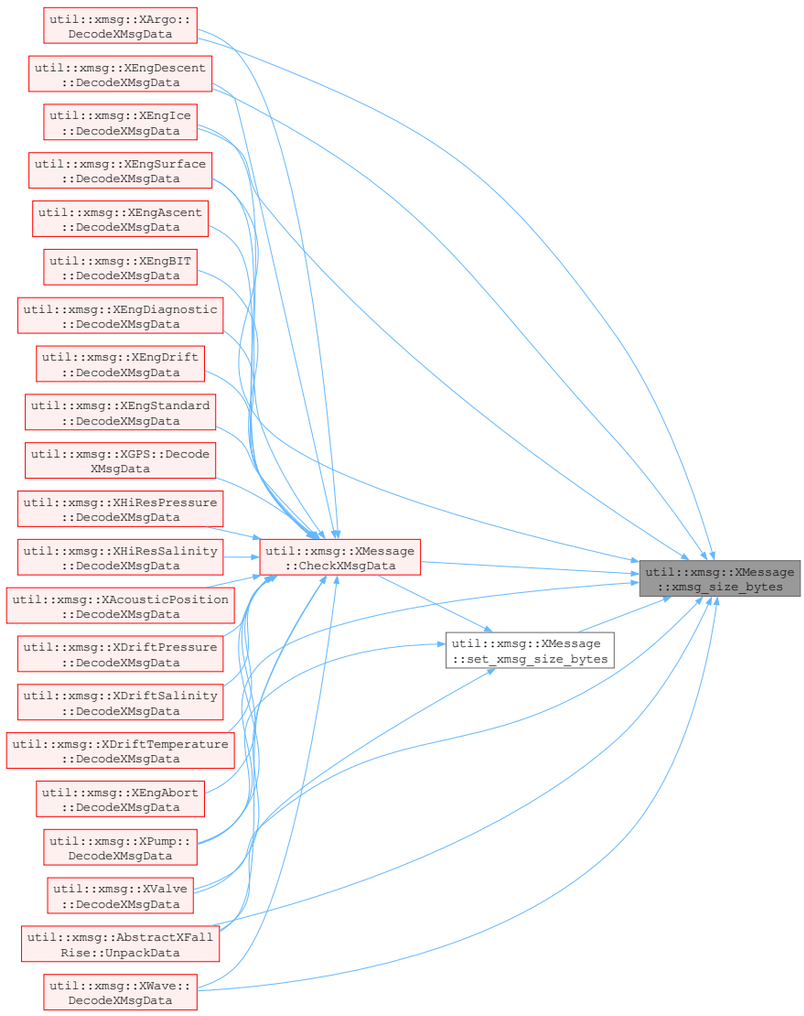
Returns

The size of the X-Message, in bytes.

References util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgI
util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(),
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), util::xmsg::XMessage::set_xmsg_size_bytes(),
and util::xmsg::AbstractXFallRise::UnpackData().

Here is the caller graph for this function:



10.83.4 Member Data Documentation

10.83.4.1 id_dive_

```
int16_t util::xmsg::XMessage::id_dive_  [private], [inherited]
```

The dive or cycle number of this message.

Referenced by util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::set_id_dive().

10.83.4.2 id_instrument_

`uint16_t util::xmsg::XMessage::id_instrument_ [private], [inherited]`

The unique WHOI identifier of this instrument.

Referenced by util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::set_id_instrument().

10.83.4.3 id_xmsg_type_

`XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type_ [private], [inherited]`

The X-message type identifier.

Referenced by util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::set_id_xmsg_type().

10.83.4.4 xmsg_size_bytes_

`uint16_t util::xmsg::XMessage::xmsg_size_bytes_ [private], [inherited]`

The size of the X message, in bytes.

Referenced by util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes().

The documentation for this class was generated from the following file:

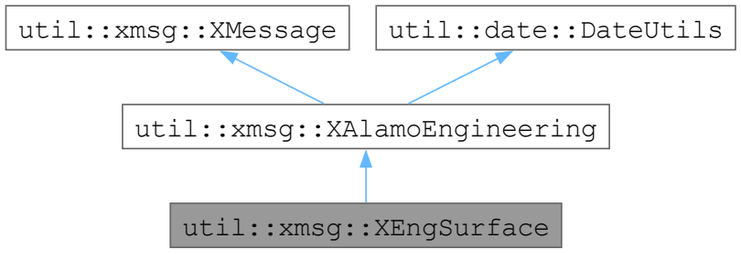
- data/xmsg/XEngStandard.h

10.84 util::xmsg::XEngSurface Class Reference

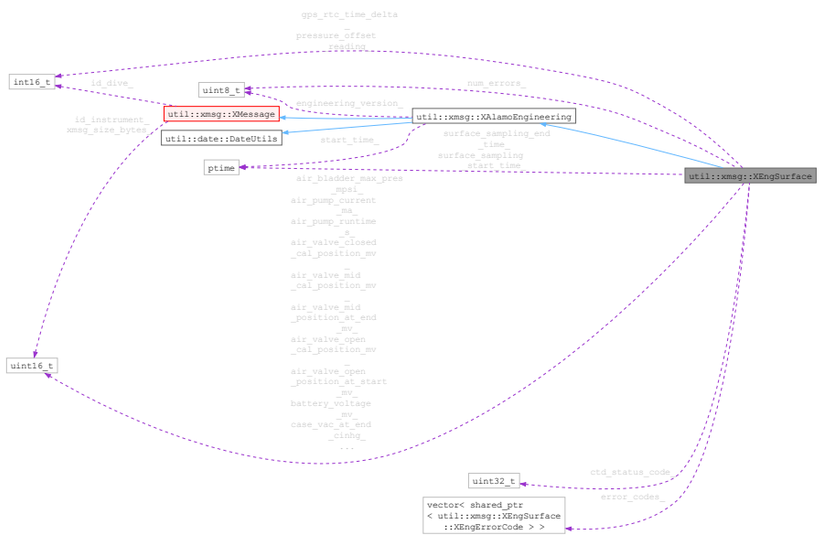
Class representing an surface state engineering X-Message.

`#include <XEngSurface.h>`

Inheritance diagram for util::xmsg::XEngSurface:



Collaboration diagram for util::xmsg::XEngSurface:



Classes

- class XEngErrorCode

  The engineering surface data error code class.

Public Member Functions

- XEngSurface (int16_t dive, uint16_t whoi, const vector< uint8_t > &data)

  The parameterized constructor.
- ~XEngSurface ()

  The destructor.
- uint16_t **air_bladder_max_pres_mpsi** () const
- template<typename T = float>
  T **air_bladder_max_pres_psi** () const
- template<typename T = float>
  T **air_pump_current_a** () const
- uint16_t **air_pump_current_ma** () const
- uint16_t **air_pump_runtime_s** () const
- uint16_t **air_valve_closed_cal_position_mv** () const
- template<typename T = float>
  T **air_valve_closed_cal_position_v** () const
- uint16_t **air_valve_mid_cal_position_mv** () const
- template<typename T = float>
  T **air_valve_mid_cal_position_v** () const
- uint16_t **air_valve_mid_position_at_end_mv** () const
- template<typename T = float>
  T **air_valve_mid_position_at_end_v** () const
- uint16_t **air_valve_open_cal_position_mv** () const
- template<typename T = float>
  T **air_valve_open_cal_position_v** () const
- uint16_t **air_valve_open_position_at_start_mv** () const
- template<typename T = float>
  T **air_valve_open_position_at_start_v** () const
- uint16_t **battery_voltage_mv** () const
- template<typename T = float>
  T battery_voltage_v () const

  Get the measured system battery voltage, in volts.
- uint16_t **case_vac_at_end_cinhg** () const
- template<typename T = float>
  T **case_vac_at_end_inhg** () const
- uint16_t **case_vac_at_start_cinhg** () const
- template<typename T = float>
  T **case_vac_at_start_inhg** () const
- uint16_t **case_vac_cingh** () const
- template<typename T = float>
  T case_vacuum_inhg () const

  Get the measured internal pressure case vacuum, in inches of Mercury.
- const uint8_t * CheckXMsgData (const vector< uint8_t > &data, int id, int minBytes, bool parseSize=true)

  Check the X-message for the correct ID, minimum size, and message terminator.
- uint32_t **ctd_status_code** () const
- void DecodeXMsgData (const vector< uint8_t > &data)

  Decode the surface state engineering X-Message from the binary data.
- uint8_t engineering_version () const

  Get the engineering message version identifier.
- float engineering_version_as_float () const

  Get the engineering message verion identifier as a floating point number.
- const vector< shared_ptr< XEngErrorCode > > & **error_codes** () const
- const string GenerateListingString (uint16_t pg, uint16_t po, uint16_t sg, uint16_t so, uint16_t tg, uint16_t to) const

  Generate the listing entry string for this surface engineering data X-Message.
- template<typename T = double>
  T **GetDecodedBitPressureOffset** () const
- int16_t **gps_rtc_time_delta** () const
- int16_t id_dive () const

  Get the dive or cycle number of this X-Message.
- uint16_t id_instrument () const

Get the unique instrument identifier of this X-Message.

- XMsgTypeId_t id_xmsg_type () const

  Get the enumerated X-Message type identifier.
- uint8_t **num_errors** () const
- uint16_t **num_sbd_packets_for_dive** () const
- uint16_t **num_transmit_attempts_for_previous_dive** () const
- uint16_t **oil_valve_closed_cal_position_mv** () const
- template<typename T = float>
  T **oil_valve_closed_cal_position_v** () const
- uint16_t **oil_valve_closed_position_at_end_mv** () const
- template<typename T = float>
  T **oil_valve_closed_position_at_end_v** () const
- uint16_t **oil_valve_open_cal_position_mv** () const
- template<typename T = float>
  T **oil_valve_open_cal_position_v** () const
- uint16_t **oil_valve_open_position_during_air_assist_mv** () const
- template<typename T = float>
  T **oil_valve_open_position_during_air_assist_v** () const
- int16_t **pressure_offset_reading** () const
- uint16_t **prof_num_bins** () const
- uint16_t **prof_num_bins_empty** () const
- uint16_t **prof_num_raw_samples** () const
- uint16_t **prof_raw_sample_first_pres_counts** () const
- template<typename T = float>
  T **prof_raw_sample_first_pres_dbar** (int16_t gain, int16_t offs) const
- uint16_t **prof_raw_sample_first_saln_counts** () const
- template<typename T = float>
  T **prof_raw_sample_first_saln_psu** (int16_t gain, int16_t offs) const
- uint16_t **prof_raw_sample_first_temp_counts** () const
- template<typename T = float>
  T **prof_raw_sample_first_temp_degc** (int16_t gain, int16_t offs) const
- uint16_t **prof_raw_sample_last_pres_counts** () const
- template<typename T = float>
  T **prof_raw_sample_last_pres_dbar** (int16_t gain, int16_t offs) const
- uint16_t **prof_raw_sample_last_saln_counts** () const
- template<typename T = float>
  T **prof_raw_sample_last_saln_psu** (int16_t gain, int16_t offs) const
- uint16_t **prof_raw_sample_last_temp_counts** () const
- template<typename T = float>
  T **prof_raw_sample_last_temp_degc** (int16_t gain, int16_t offs) const
- const ptime & start_time () const

  Get the engineering message state start time.
- const ptime & **surface_sampling_end_time** () const
- const ptime & **surface_sampling_start_time** () const
- template<typename T = float>
  T system_current_a () const

  Get the measured system current, in amperes.
- uint16_t **system_current_ma** () const
- uint16_t xmsg_size_bytes () const

  Get the size of the X-Message, in bytes.

Static Public Member Functions

- static string GenerateCurrentUtcTimeIso8601String ()

  Generate an ISO8601 format date/time string representation of the current time, in UTC.
- static string GenerateIso8601String (const ptime &p, bool extended=true)

  Generate an ISO8601 format date/time string representation of a POSIX time object.
- static double GenerateJulianDay (const ptime &dt, const ptime &ref_dt)

  Convert a date/time to fractional days since a reference date/time.
- static ptime GetCurrentUtcTime ()

  Generate a POSIX time object representing the current date and time (rounded to the nearest second).

Protected Member Functions

- void **set_air_bladder_max_pres_mpsi** (uint16_t air_bladder_max_pres_mpsi)
- void **set_air_pump_current_ma** (uint16_t air_pump_current_ma)
- void **set_air_pump_runtime_s** (uint16_t air_pump_runtime_s)
- void **set_air_valve_closed_cal_position_mv** (uint16_t air_valve_closed_cal_position_mv)
- void **set_air_valve_mid_cal_position_mv** (uint16_t air_valve_mid_cal_position_mv)
- void **set_air_valve_mid_position_at_end_mv** (uint16_t air_valve_mid_position_at_end_mv)
- void **set_air_valve_open_cal_position_mv** (uint16_t air_valve_open_cal_position_mv)
- void **set_air_valve_open_position_at_start_mv** (uint16_t air_valve_open_position_at_start_mv)
- void **set_battery_voltage_mv** (uint16_t battery_voltage_mv)
- void **set_case_vac_at_end_cinhg** (uint16_t case_vac_at_end_cinhg)
- void **set_case_vac_at_start_cinhg** (uint16_t case_vac_at_start_cinhg)
- void **set_case_vac_cingh** (uint16_t case_vac_cingh)
- void **set_ctd_status_code** (uint32_t ctd_status_code)
- void set_engineering_version (uint8_t engineering_version)

  Set the engineering message version identifier.
- void **set_error_codes** (const vector< shared_ptr< XEngErrorCode > > &error_codes)
- void **set_gps_rtc_time_delta** (int16_t gps_rtc_time_delta)
- void set_id_dive (int16_t id_dive)

  Set the dive or cycle number of this message.
- void set_id_instrument (uint16_t id_instrument)

  Set the unique instrument identifier.
- void set_id_xmsg_type (XMsgTypeId_t id_xmsg_type)

  Set the X-Message type identifier.
- void **set_num_errors** (uint8_t num_errors)
- void **set_num_sbd_packets_for_dive** (uint16_t num_sbd_packets_for_dive)
- void **set_num_transmit_attempts_for_previous_dive** (uint16_t num_transmit_attempts_for_previous_dive)
- void **set_oil_valve_closed_cal_position_mv** (uint16_t oil_valve_closed_cal_position_mv)
- void **set_oil_valve_closed_position_at_end_mv** (uint16_t oil_valve_closed_position_at_end_mv)
- void **set_oil_valve_open_cal_position_mv** (uint16_t oil_valve_open_cal_position_mv)
- void **set_oil_valve_open_position_during_air_assist_mv** (uint16_t oil_valve_open_position_during_air_↩ assist_mv)
- void **set_pressure_offset_reading** (int16_t pressure_offset_reading)
- void **set_prof_num_bins** (uint16_t prof_num_bins)
- void **set_prof_num_bins_empty** (uint16_t prof_num_bins_empty)
- void **set_prof_num_raw_samples** (uint16_t prof_num_raw_samples)
- void **set_prof_raw_sample_first_pres** (uint16_t prof_raw_sample_first_pres)
- void **set_prof_raw_sample_first_saln** (uint16_t prof_raw_sample_first_saln)
- void **set_prof_raw_sample_first_temp** (uint16_t prof_raw_sample_first_temp)
- void **set_prof_raw_sample_last_pres** (uint16_t prof_raw_sample_last_pres)
- void **set_prof_raw_sample_last_saln** (uint16_t prof_raw_sample_last_saln)
- void **set_prof_raw_sample_last_temp** (uint16_t prof_raw_sample_last_temp)
- void set_start_time (const ptime &start_time)

  Set the engineering message state start time.
- void **set_surface_sampling_end_time** (const ptime &surface_sampling_end_time)
- void **set_surface_sampling_start_time** (const ptime &surface_sampling_start_time)
- void **set_system_current_ma** (uint16_t system_current_ma)
- void set_xmsg_size_bytes (uint16_t xmsg_size_bytes)

  Set the size of the X-Message, in bytes.

Private Attributes

- uint16_t air_bladder_max_pres_mpsi_

  The maximum measured air bladder pressure during the air pump, in .001 PSI.
- uint16_t air_pump_current_ma_

  The measured air pump current, in mA.
- uint16_t air_pump_runtime_s_

  The measured air pump run time, in seconds.
- uint16_t air_valve_closed_cal_position_mv_

  The air valve closed calibration position, in mV.
- uint16_t air_valve_mid_cal_position_mv_

  The air valve mid calibration position, in mV.
- uint16_t air_valve_mid_position_at_end_mv_

  The air valve mid position at the end of the surface phase of the dive cycle, in mV.

- uint16_t air_valve_open_cal_position_mv_

  The air valve open calibration position, in mV.
- uint16_t air_valve_open_position_at_start_mv_

  The air valve open position at the start of the surface phase of the dive cycle, in mV.
- uint16_t battery_voltage_mv_

  The measured system battery voltage, in mV.
- uint16_t case_vac_at_end_cinhg_

  The measured internal pressure case vacuum, in 0.01 inHg, at the end of the surface phase of the dive cycle.
- uint16_t case_vac_at_start_cinhg_

  The measured internal pressure case vacuum, in 0.01 inHg, at the start of the surface phase of the dive cycle.
- uint16_t case_vac_cingh_

  The measured internal pressure case vacuum, in 0.01 inHg.
- uint32_t ctd_status_code_

  The CTD status code.
- uint8_t engineering_version_

  The engineering message version identifier.
- vector< shared_ptr< XEngErrorCode > > error_codes_

  The error code values for this dive, if any.
- int16_t **gps_rtc_time_delta_**
- int16_t id_dive_

  The dive or cycle number of this message.
- uint16_t id_instrument_

  The unique WHOI identifier of this instrument.
- XMsgTypeId_t id_xmsg_type_

  The X-message type identifier.
- uint8_t num_errors_

  The number of error codes reported for this dive cycle.
- uint16_t num_sbd_packets_for_dive_

  The number of packets to be transmitted for the current dive.
- uint16_t num_transmit_attempts_for_previous_dive_

  The number of attempts to transmit in the previous Iridium session.
- uint16_t oil_valve_closed_cal_position_mv_

  The oil valve closed calibration position, in mV.
- uint16_t oil_valve_closed_position_at_end_mv_

  The oil valve closed position at the end of the surface phase of the dive cycle, in mV.
- uint16_t oil_valve_open_cal_position_mv_

  The oil valve open calibration position at the end of the surface phase of the dive cycle, in mV.
- uint16_t oil_valve_open_position_during_air_assist_mv_

  The oil valve open position during air pump, in mV.
- int16_t pressure_offset_reading_

  The measured pressure sensor offset at the surface, in 1/1000 dBar.
- uint16_t **prof_num_bins_**
- uint16_t **prof_num_bins_empty_**
- uint16_t **prof_num_raw_samples_**
- uint16_t **prof_raw_sample_first_pres_**
- uint16_t **prof_raw_sample_first_saln_**
- uint16_t **prof_raw_sample_first_temp_**
- uint16_t **prof_raw_sample_last_pres_**
- uint16_t **prof_raw_sample_last_saln_**
- uint16_t **prof_raw_sample_last_temp_**
- ptime start_time_

  The engineering message state start time.
- ptime **surface_sampling_end_time_**
- ptime **surface_sampling_start_time_**
- uint16_t system_current_ma_

  The measured system current, in mA.
- uint16_t xmsg_size_bytes_

  The size of the X message, in bytes.

10.84.1 Detailed Description

Class representing an surface state engineering X-Message.

Note

This X-Message is only used by ALAMO-type instruments.

This message type was added in X-Message version 2.1.

10.84.2 Constructor & Destructor Documentation

10.84.2.1 XEngSurface()

```
util::xmsg::XEngSurface::XEngSurface (
            int16_t dive,
            uint16_t whoi,
            const vector< uint8_t > & data)  [inline]
```

The parameterized constructor.

Parameters

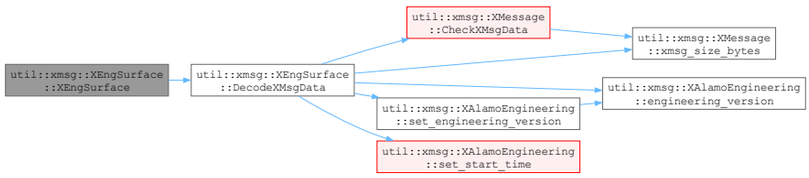
in	*dive*	The dive or cycle number of this message.
in	*whoi*	The unique WHOI identifier.
in	*data*	The binary X-message data to decode.

Exceptions

util::log::MessageException	if an decode error occurs.

References DecodeXMsgData().

Here is the call graph for this function:



10.84.2.2 ~XEngSurface()

```
util::xmsg::XEngSurface::~XEngSurface ()  [inline]
```

The destructor.

10.84.3 Member Function Documentation

10.84.3.1 battery_voltage_v()

```
template<typename T = float>
T util::xmsg::XEngSurface::battery_voltage_v () const  [inline]
```

Get the measured system battery voltage, in volts.

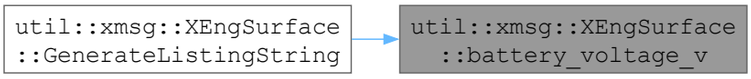
Returns

The measured system battery voltage, in volts.

References battery_voltage_mv_.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.84.3.2 case_vacuum_inhg()

```
template<typename T = float>
T util::xmsg::XEngSurface::case_vacuum_inhg () const  [inline]
```

Get the measured internal pressure case vacuum, in inches of Mercury.

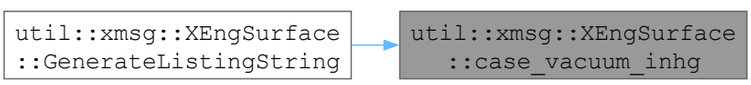
Returns

The measured internal pressure case vacuum, in inches of Mercury.

References case_vac_cingh_.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.84.3.3 CheckXMsgData()

```
const uint8_t * util::xmsg::XMessage::CheckXMsgData (
            const vector< uint8_t > & data,
            int id,
            int minBytes,
            bool parseSize = true)  [inline], [inherited]
```

Check the X-message for the correct ID, minimum size, and message terminator.

Parameters

in	*data*	The X-message binary data to check.
in	*id*	The expected ID of the X-message. If this field is less that 0, then the ID check will be skipped.
in	*minBytes*	The minimum number of bytes expected in a valid message.
in	*parseSize*	If TRUE, then the message size field will be parsed from the X-Message. Otherwise, the size field will not be parsed.

Exceptions

util::log::MessageException	if • the message does not contain the expected ID, • the message does not contain the minimum number of expected bytes, • the message does not contain a terminator (';') in the correct location, or, • any other error occurs.

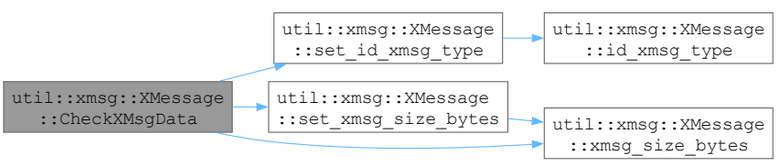
Returns

A pointer to the underlying data buffer of data.

References CATCH_BLOCK, EXC_THROW, lwrn, util::xmsg::XMessage::set_id_xmsg_type(), util::xmsg::XMessage::set_xmsg_size_byte
u16, and util::xmsg::XMessage::xmsg_size_bytes().

Referenced by util::xmsg::XAcousticPosition::DecodeXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XDriftPressure::Dec
util::xmsg::XDriftSalinity::DecodeXMsgData(), util::xmsg::XDriftTemperature::DecodeXMsgData(), util::xmsg::XEngAbort::DecodeXMsgDa
util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngBIT::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(),
util::xmsg::XEngDiagnostic::DecodeXMsgData(), util::xmsg::XEngDrift::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(),
util::xmsg::XEngStandard::DecodeXMsgData(), DecodeXMsgData(), util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XHiResPressure:
util::xmsg::XHiResSalinity::DecodeXMsgData(), util::xmsg::XHiResTemperature::DecodeXMsgData(), util::xmsg::XMissionCmdEcho::Dec
util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XProfilePressure::DecodeXMsgData(), util::xmsg::XProfileSalinity::Dec
util::xmsg::XProfileTemperature::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(), util::xmsg::XValve::DecodeXMsgData(),
util::xmsg::XWave::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the call graph for this function:



10.84.3.4 DecodeXMsgData()

```
void util::xmsg::XEngSurface::DecodeXMsgData (
            const vector< uint8_t > & data)  [inline], [virtual]
```

Decode the surface state engineering X-Message from the binary data.

Parameters

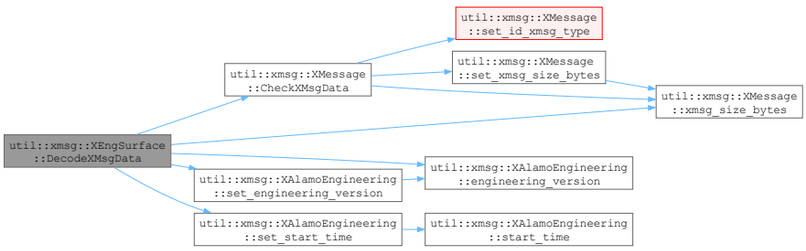
in	*data*	The binary message to decode.

Implements util::xmsg::XMessage.

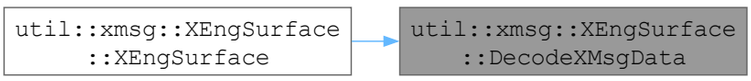
References CATCH_BLOCK, util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XAlamoEngineering::engineering_version(),
error_codes_, EXC_THROW, i16, lwrn, s2000, util::xmsg::XAlamoEngineering::set_engineering_version(), util::xmsg::XAlamoEngineering
u08, u16, u32, and util::xmsg::XMessage::xmsg_size_bytes().

Referenced by XEngSurface().

Here is the call graph for this function:



Here is the caller graph for this function:



10.84.3.5 engineering_version()

```
uint8_t util::xmsg::XAlamoEngineering::engineering_version () const  [inline], [inherited]
```

Get the engineering message version identifier.

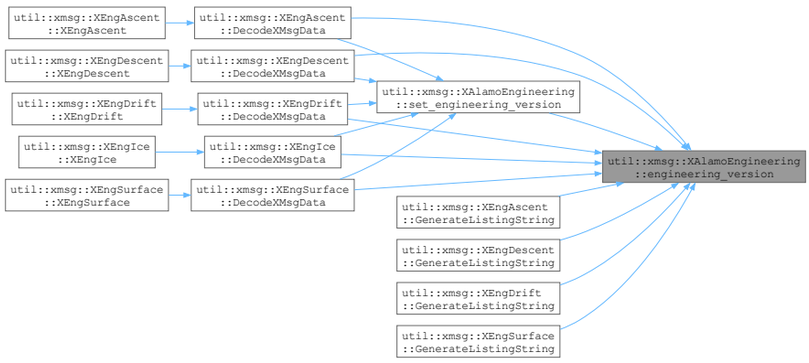
Returns

The engineering message version identifier.

References util::xmsg::XAlamoEngineering::engineering_version_.

Referenced by util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(), util::xmsg::XEngDrift::Decode util::xmsg::XEngIce::DecodeXMsgData(), DecodeXMsgData(), util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), GenerateListingString(), and util::xmsg::XAlamoEngineering::set_engineering_version().

Here is the caller graph for this function:



10.84.3.6 engineering_version_as_float()

```
float util::xmsg::XAlamoEngineering::engineering_version_as_float () const  [inline], [inherited]
```

Get the engineering message verion identifier as a floating point number.

Returns

The engineering message verion identifier as a floating point number.

References util::xmsg::XAlamoEngineering::engineering_version_.

10.84.3.7 GenerateCurrentUtcTimeIso8601String()

```
static string util::date::DateUtils::GenerateCurrentUtcTimeIso8601String ()  [inline], [static],
[inherited]
```

Generate an ISO8601 format date/time string representation of the current time, in UTC.

Note

All date/time objects use the UTC time-zone.

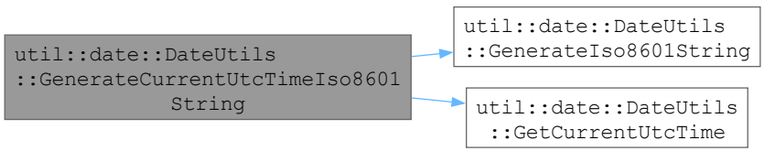
Returns

An ISO8601 format date/time string representation of the current time.

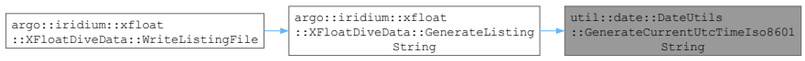
References util::date::DateUtils::GenerateIso8601String(), and util::date::DateUtils::GetCurrentUtcTime().

Referenced by argo::iridium::xfloat::XFloatDiveData::GenerateListingString().

Here is the call graph for this function:



Here is the caller graph for this function:



10.84.3.8 GenerateIso8601String()

```
static string util::date::DateUtils::GenerateIso8601String (
            const ptime & p,
            bool extended = true)  [inline], [static], [inherited]
```

Generate an ISO8601 format date/time string representation of a POSIX time object.

Note

All date/time objects use the UTC time-zone.

Parameters

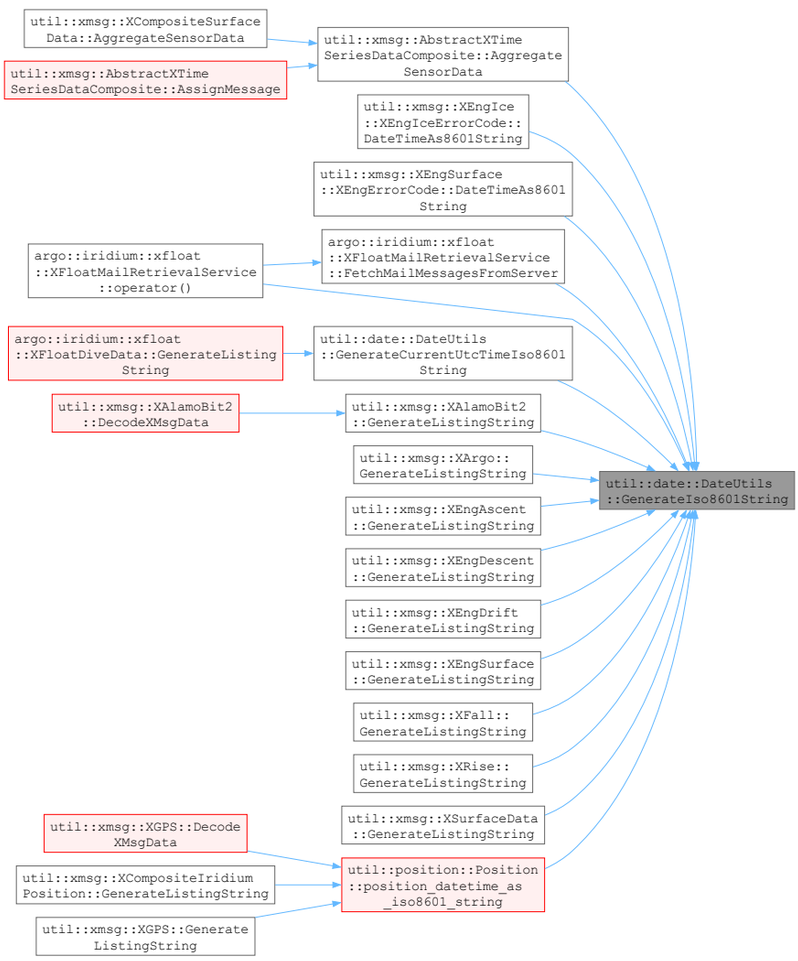
in	*p*	POSIX time from which to generate string.
in	*extended*	If TRUE, use the ISO 8601 extended format (YYYY-MM-DDThh:mm:ssZ); otherwise, use the basic format (YYYYMMDDThhmmss).

Returns

An ISO8601 format date/time string representation of the POSIX time object. If p is equal to boost::posix_↩ time::not_a_date_time, then the string "0000-00-00T00:00:00Z" (extended) or "00000000000000" (basic) will be returned.

Referenced by util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::AggregateSensorData(), util::xmsg::XEngIce::XEngIce
util::xmsg::XEngSurface::XEngErrorCode::DateTimeAs8601String(), argo::iridium::xfloat::XFloatMailRetrievalService::FetchMailMessages
util::date::DateUtils::GenerateCurrentUtcTimeIso8601String(), util::xmsg::XAlamoBit2::GenerateListingString(), util::xmsg::XArgo::Genera
util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListing
GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::G
argo::iridium::xfloat::XFloatMailRetrievalService::operator()(), and util::position::Position::position_datetime_as_iso8601_string().

Here is the caller graph for this function:



10.84.3.9 GenerateJulianDay()

```
static double util::date::DateUtils::GenerateJulianDay (
            const ptime & dt,
            const ptime & ref_dt)  [inline], [static], [inherited]
```

Convert a date/time to fractional days since a reference date/time.

Parameters

in	*dt*	The date/time to convert.
in	*ref*↩ *_dt*	The reference date/time (day 0).

Exceptions

util::log::MessageException	if either date/time is not valid.

Returns

The number of days (with millisecond resolution) from `ref_dt` to `dt`.

References CATCH_BLOCK, and EXC_THROW.

10.84.3.10 GenerateListingString()

```
const string util::xmsg::XEngSurface::GenerateListingString (
            uint16_t pg,
            uint16_t po,
            uint16_t sg,
            uint16_t so,
            uint16_t tg,
            uint16_t to) const  [inline]
```

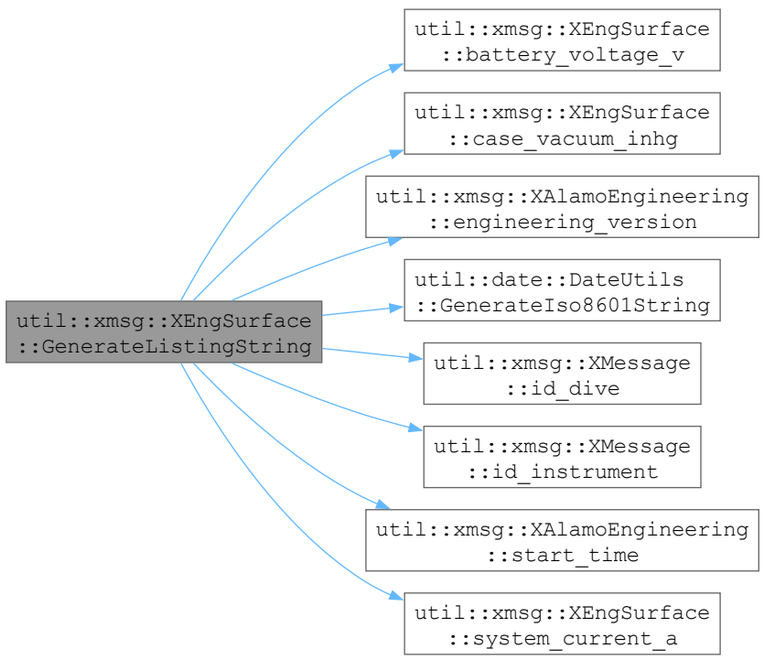
Generate the listing entry string for this surface engineering data X-Message.

Returns

The listing entry string for this surface engineering data X-Message.

References battery_voltage_v(), case_vacuum_inhg(), CATCH_BLOCK, util::xmsg::XAlamoEngineering::engineering_version(), error_codes_, EXC_THROW, util::date::DateUtils::GenerateIso8601String(), util::xmsg::XMessage::id_dive(), util::xmsg::XMessage::id_instrument(), util::xmsg::XAlamoEngineering::start_time(), and system_current_a().

Here is the call graph for this function:



10.84.3.11 GetCurrentUtcTime()

```
static ptime util::date::DateUtils::GetCurrentUtcTime ()  [inline], [static], [inherited]
```

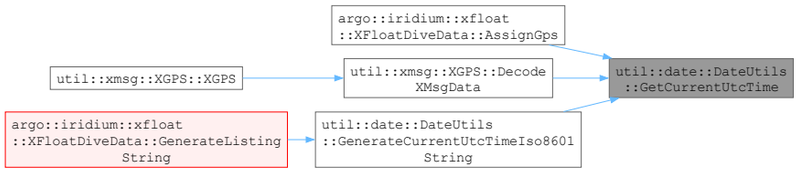
Generate a POSIX time object representing the current date and time (rounded to the nearest second).

Returns

A POSIX time object representing the current date and time (rounded to the nearest second).

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignGps(), util::xmsg::XGPS::DecodeXMsgData(), and util::date::DateUtils::GenerateCurrentUtcTimeIso8601String().

Here is the caller graph for this function:



10.84.3.12 id_dive()

```
int16_t util::xmsg::XMessage::id_dive () const  [inline], [inherited]
```

Get the dive or cycle number of this X-Message.

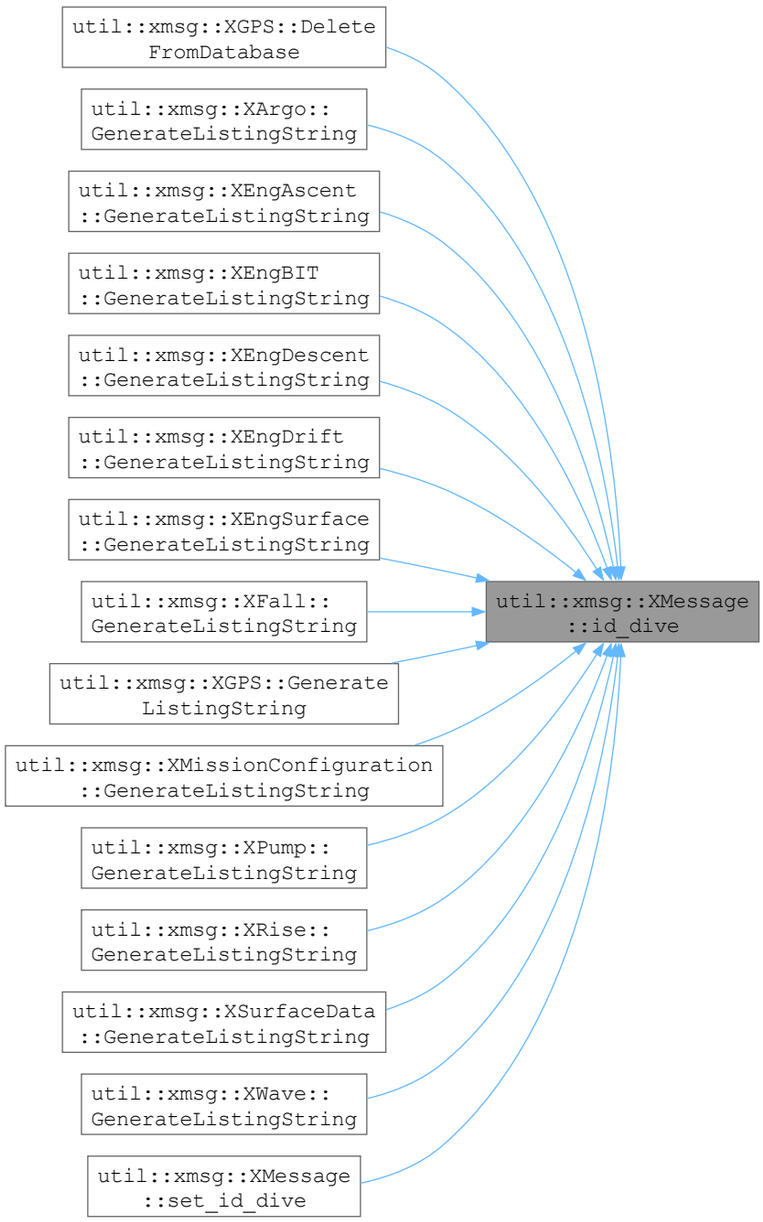
Returns

The dive or cycle number of this X-Message.

References util::xmsg::XMessage::id_dive_.

Referenced by util::xmsg::XGPS::DeleteFromDatabase(), util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateLis
util::xmsg::XEngBIT::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingStr
GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMissionConfigu
util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(),
util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_dive().

Here is the caller graph for this function:



10.84.3.13 id_instrument()

```
uint16_t util::xmsg::XMessage::id_instrument () const  [inline], [inherited]
```

Get the unique instrument identifier of this X-Message.

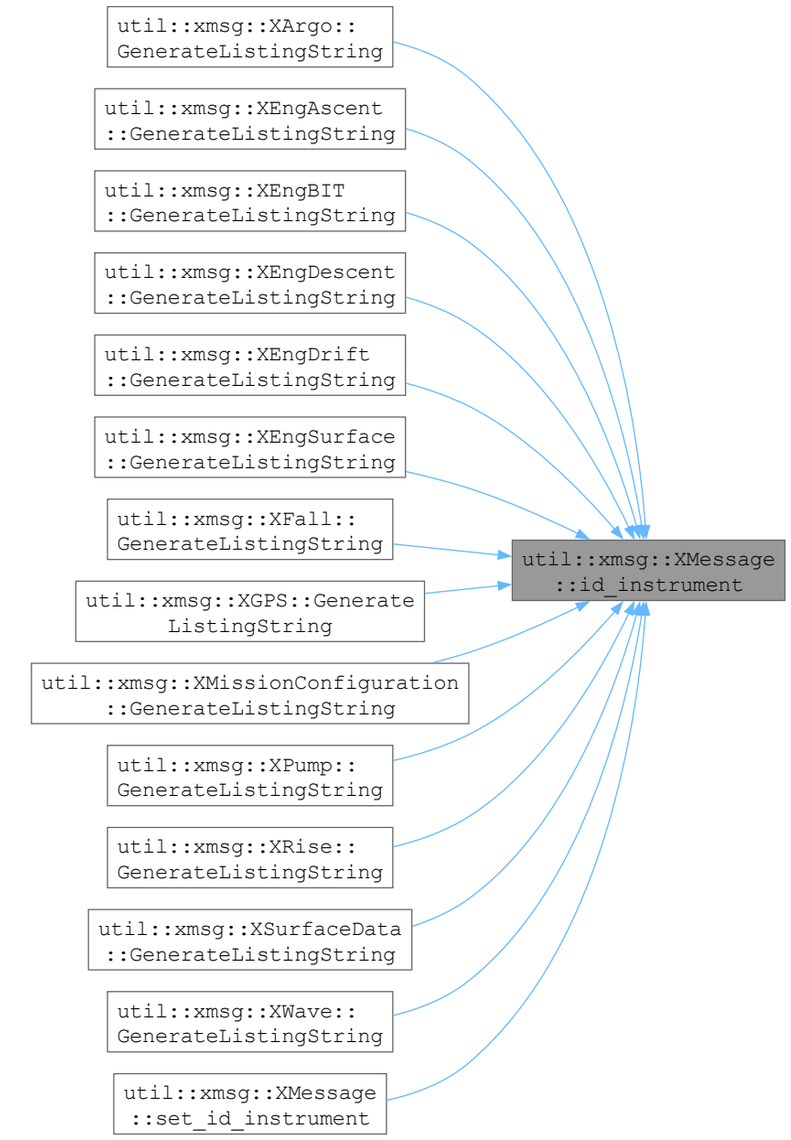
Returns

The unique instrument identifier of this X-Message.

References util::xmsg::XMessage::id_instrument_.

Referenced by util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngBIT::Generate
util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), GenerateListingString(),
util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingS
util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(),
util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_instrument().

Here is the caller graph for this function:



10.84.3.14 id_xmsg_type()

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type () const  [inline], [inherited]
```

Get the enumerated X-Message type identifier.

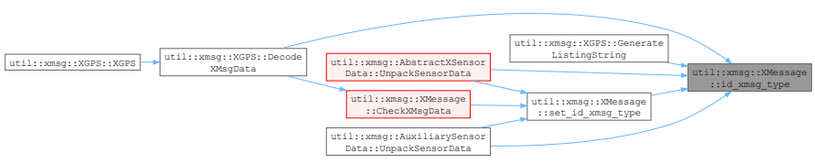
Returns

The enumerated X-Message type identifier.

References util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMessage::set_id_xmsg_typ
util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the caller graph for this function:



10.84.3.15 set_engineering_version()

```
void util::xmsg::XAlamoEngineering::set_engineering_version (
            uint8_t engineering_version)  [inline], [protected], [inherited]
```

Set the engineering message version identifier.

Parameters

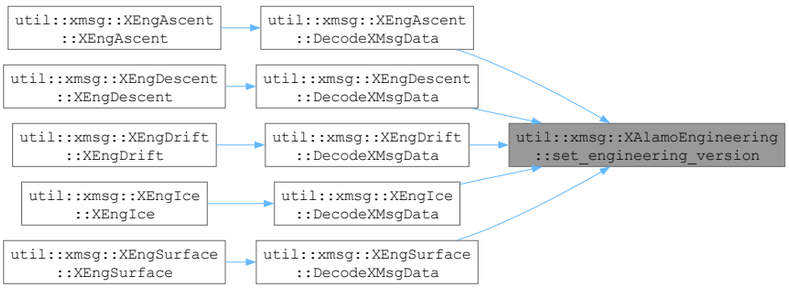
in	*engineering_version*	The engineering message version identifier.

References util::xmsg::XAlamoEngineering::engineering_version(), and util::xmsg::XAlamoEngineering::engineering_version_.

Referenced by util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(), util::xmsg::XEngDrift::Decode
util::xmsg::XEngIce::DecodeXMsgData(), and DecodeXMsgData().

Here is the call graph for this function:

Here is the caller graph for this function:



10.84.3.16 set_id_dive()

```
void util::xmsg::XMessage::set_id_dive (
            int16_t id_dive)  [inline], [protected], [inherited]
```

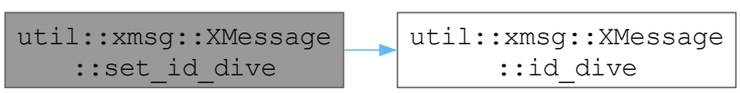
Set the dive or cycle number of this message.

Parameters

in	*id_dive*	The dive or cycle number of this message to set.

References util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::id_dive_.

Here is the call graph for this function:



10.84.3.17 set_id_instrument()

```
void util::xmsg::XMessage::set_id_instrument (
            uint16_t id_instrument)  [inline], [protected], [inherited]
```

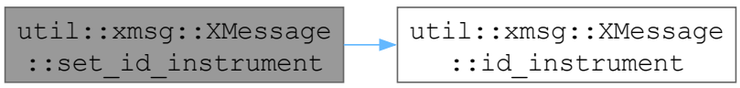
Set the unique instrument identifier.

Parameters

in	*id_instrument*	The unique instrument identifier to set.

References util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::id_instrument_.

Here is the call graph for this function:



10.84.3.18 set_id_xmsg_type()

```
void util::xmsg::XMessage::set_id_xmsg_type (
            XMsgTypeId_t id_xmsg_type)  [inline], [protected], [inherited]
```

Set the X-Message type identifier.

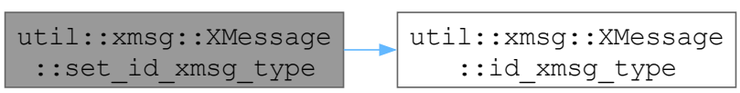
Parameters

in	*id_xmsg_type*	The X-Message type identifier to set.

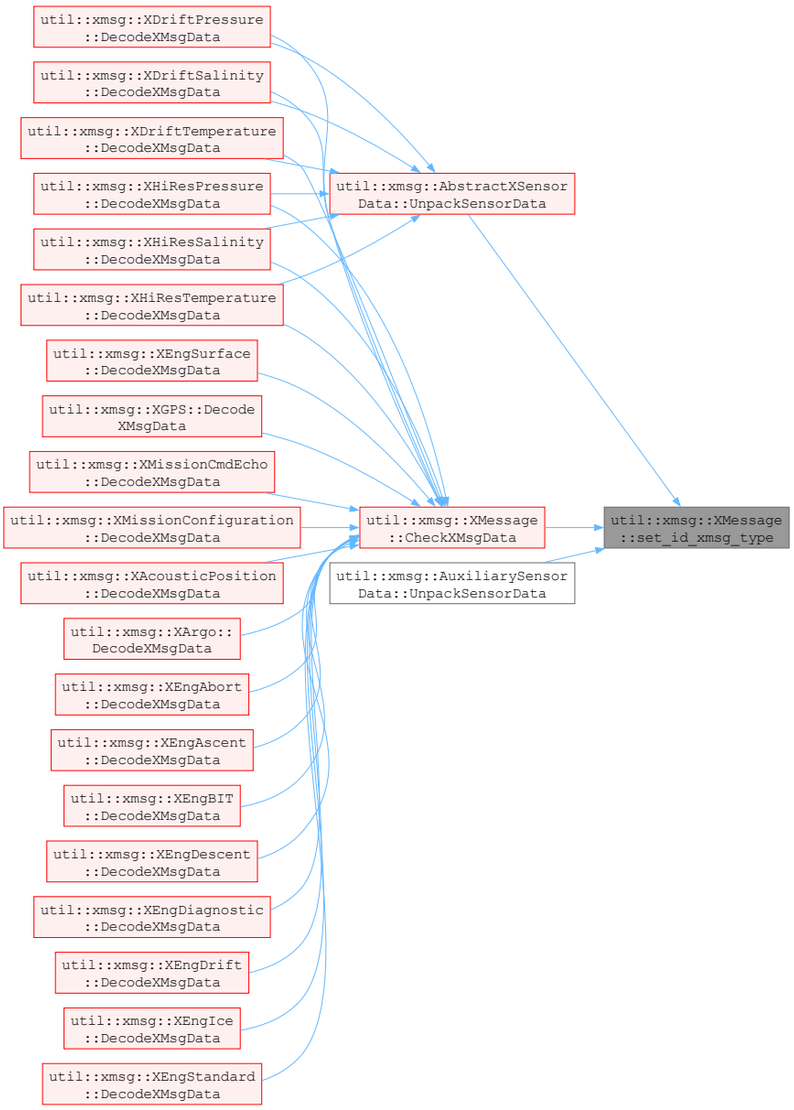
References util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.84.3.19 set_start_time()

```
void util::xmsg::XAlamoEngineering::set_start_time (
            const ptime & start_time)  [inline], [protected], [inherited]
```

Set the engineering message state start time.

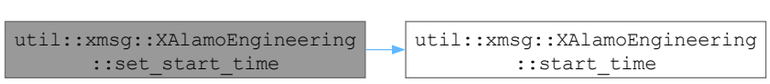
Parameters

in	*start_time*	The engineering message state start time.

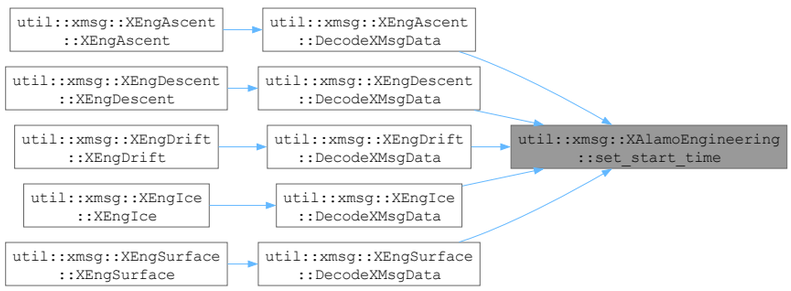
References util::xmsg::XAlamoEngineering::start_time(), and util::xmsg::XAlamoEngineering::start_time_.

Referenced by util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(), util::xmsg::XEngDrift::Decode
util::xmsg::XEngIce::DecodeXMsgData(), and DecodeXMsgData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.84.3.20 set_xmsg_size_bytes()

```
void util::xmsg::XMessage::set_xmsg_size_bytes (
            uint16_t xmsg_size_bytes)  [inline], [protected], [inherited]
```

Set the size of the X-Message, in bytes.

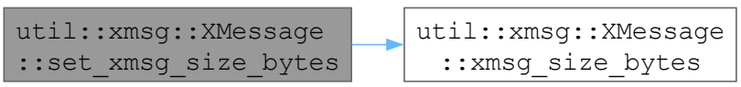
Parameters

in	*xmsg_size_bytes*	The size of the X-Message to set, in bytes.

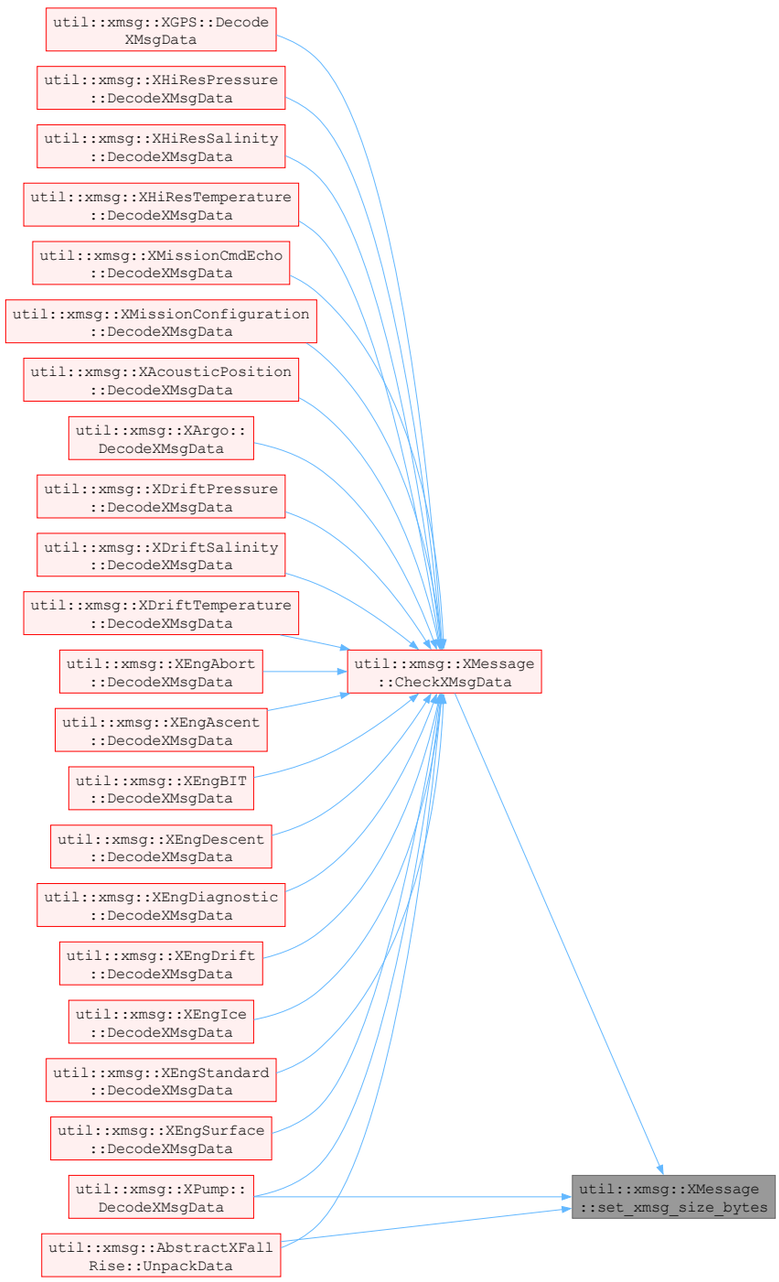
References util::xmsg::XMessage::xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XPump::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::Unpa

Here is the call graph for this function:



Here is the caller graph for this function:



10.84.3.21 start_time()

```
const ptime & util::xmsg::XAlamoEngineering::start_time () const  [inline], [inherited]
```

Get the engineering message state start time.

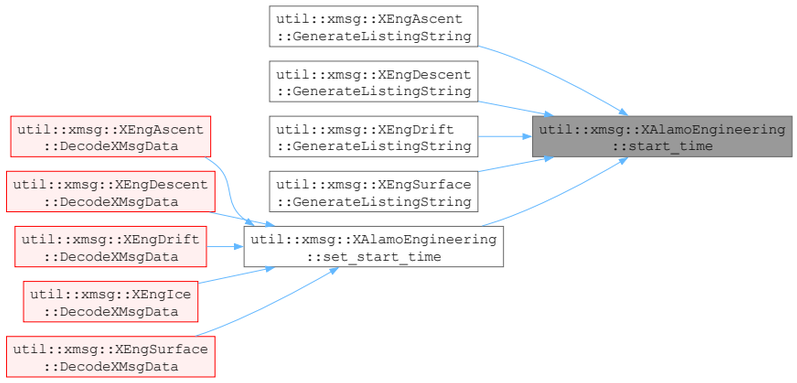
Returns

The engineering message state start time.

References util::xmsg::XAlamoEngineering::start_time_.

Referenced by util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), GenerateListingString(), and util::xmsg::XAlamoEngineering::set_start_time().

Here is the caller graph for this function:



10.84.3.22 system_current_a()

```
template<typename T = float>
T util::xmsg::XEngSurface::system_current_a () const  [inline]
```

Get the measured system current, in amperes.

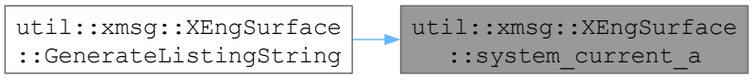
Returns

The measured system current, in amperes.

References system_current_ma_.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.84.3.23 xmsg_size_bytes()

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes () const  [inline], [inherited]
```

Get the size of the X-Message, in bytes.

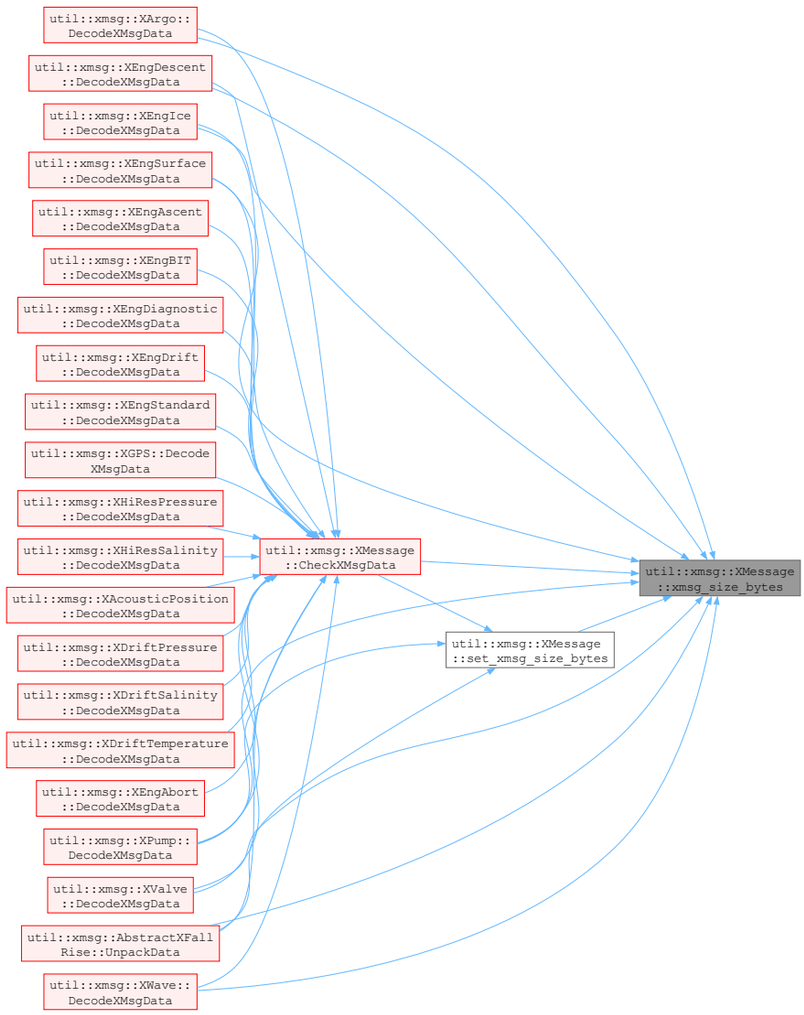
Returns

The size of the X-Message, in bytes.

References util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgI
util::xmsg::XEngIce::DecodeXMsgData(), DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(), util::xmsg::XValve::DecodeXMsgD
util::xmsg::XWave::DecodeXMsgData(), util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the caller graph for this function:



10.84.4 Member Data Documentation

10.84.4.1 air_bladder_max_pres_mpsi_

```
uint16_t util::xmsg::XEngSurface::air_bladder_max_pres_mpsi_  [private]
```

The maximum measured air bladder pressure during the air pump, in .001 PSI.

Note

This field was added in version 0x11.

10.84.4.2 air_pump_current_ma_

```
uint16_t util::xmsg::XEngSurface::air_pump_current_ma_  [private]
```

The measured air pump current, in mA.

Note

This field was added in version 0x11.

10.84.4.3 air_pump_runtime_s_

```
uint16_t util::xmsg::XEngSurface::air_pump_runtime_s_  [private]
```

The measured air pump run time, in seconds.

Note

This field was added in version 0x11.

10.84.4.4 air_valve_closed_cal_position_mv_

```
uint16_t util::xmsg::XEngSurface::air_valve_closed_cal_position_mv_  [private]
```

The air valve closed calibration position, in mV.

Note

This field was added in version 0x11.

10.84.4.5 air_valve_mid_cal_position_mv_

```
uint16_t util::xmsg::XEngSurface::air_valve_mid_cal_position_mv_  [private]
```

The air valve mid calibration position, in mV.

Note

This field was added in version 0x11.

10.84.4.6 air_valve_mid_position_at_end_mv_

```
uint16_t util::xmsg::XEngSurface::air_valve_mid_position_at_end_mv_  [private]
```

The air valve mid position at the end of the surface phase of the dive cycle, in mV.

Note

This field was added in version 0x11.

10.84.4.7 air_valve_open_cal_position_mv_

```
uint16_t util::xmsg::XEngSurface::air_valve_open_cal_position_mv_  [private]
```

The air valve open calibration position, in mV.

Note

This field was added in version 0x11.

10.84.4.8 air_valve_open_position_at_start_mv_

```
uint16_t util::xmsg::XEngSurface::air_valve_open_position_at_start_mv_  [private]
```

The air valve open position at the start of the surface phase of the dive cycle, in mV.

Note

This field was added in version 0x11.

10.84.4.9 battery_voltage_mv_

```
uint16_t util::xmsg::XEngSurface::battery_voltage_mv_  [private]
```

The measured system battery voltage, in mV.

Note

This field is only used for message version 0x10. It was removed in version 0x11.

Referenced by battery_voltage_v().

10.84.4.10 case_vac_at_end_cinhg_

```
uint16_t util::xmsg::XEngSurface::case_vac_at_end_cinhg_  [private]
```

The measured internal pressure case vacuum, in 0.01 inHg, at the end of the surface phase of the dive cycle.

Note

This field was added in version 0x11.

10.84.4.11 case_vac_at_start_cinhg_

```
uint16_t util::xmsg::XEngSurface::case_vac_at_start_cinhg_  [private]
```

The measured internal pressure case vacuum, in 0.01 inHg, at the start of the surface phase of the dive cycle.

Note

This field was added in version 0x11.

10.84.4.12 case_vac_cingh_

```
uint16_t util::xmsg::XEngSurface::case_vac_cingh_  [private]
```

The measured internal pressure case vacuum, in 0.01 inHg.

Note

This field is only used for message version 0x10. It was removed in version 0x11.

Referenced by case_vacuum_inhg().

10.84.4.13 ctd_status_code_

```
uint32_t util::xmsg::XEngSurface::ctd_status_code_  [private]
```

The CTD status code.

Todo Define status code values for the ALAMO surface engineering data message.

10.84.4.14 engineering_version_

```
uint8_t util::xmsg::XAlamoEngineering::engineering_version_  [private], [inherited]
```

The engineering message version identifier.

Referenced by util::xmsg::XAlamoEngineering::engineering_version(), util::xmsg::XAlamoEngineering::engineering_version_as_float(), and util::xmsg::XAlamoEngineering::set_engineering_version().

10.84.4.15 error_codes_

```
vector<shared_ptr<XEngErrorCode> > util::xmsg::XEngSurface::error_codes_  [private]
```

The error code values for this dive, if any.

Referenced by DecodeXMsgData(), and GenerateListingString().

10.84.4.16 id_dive_

```
int16_t util::xmsg::XMessage::id_dive_  [private], [inherited]
```

The dive or cycle number of this message.

Referenced by util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::set_id_dive().

10.84.4.17 id_instrument_

```
uint16_t util::xmsg::XMessage::id_instrument_  [private], [inherited]
```

The unique WHOI identifier of this instrument.

Referenced by util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::set_id_instrument().

10.84.4.18 id_xmsg_type_

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type_  [private], [inherited]
```

The X-message type identifier.

Referenced by util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::set_id_xmsg_type().

10.84.4.19 num_errors_

```
uint8_t util::xmsg::XEngSurface::num_errors_  [private]
```

The number of error codes reported for this dive cycle.

10.84.4.20 num_sbd_packets_for_dive_

```
uint16_t util::xmsg::XEngSurface::num_sbd_packets_for_dive_  [private]
```

The number of packets to be transmitted for the current dive.

10.84.4.21 num_transmit_attempts_for_previous_dive_

```
uint16_t util::xmsg::XEngSurface::num_transmit_attempts_for_previous_dive_  [private]
```

The number of attempts to transmit in the previous Iridium session.

10.84.4.22 oil_valve_closed_cal_position_mv_

```
uint16_t util::xmsg::XEngSurface::oil_valve_closed_cal_position_mv_  [private]
```

The oil valve closed calibration position, in mV.

10.84.4.23 oil_valve_closed_position_at_end_mv_

```
uint16_t util::xmsg::XEngSurface::oil_valve_closed_position_at_end_mv_  [private]
```

The oil valve closed position at the end of the surface phase of the dive cycle, in mV.

10.84.4.24 oil_valve_open_cal_position_mv_

```
uint16_t util::xmsg::XEngSurface::oil_valve_open_cal_position_mv_  [private]
```

The oil valve open calibration position at the end of the surface phase of the dive cycle, in mV.

10.84.4.25 oil_valve_open_position_during_air_assist_mv_

```
uint16_t util::xmsg::XEngSurface::oil_valve_open_position_during_air_assist_mv_  [private]
```

The oil valve open position during air pump, in mV.

10.84.4.26 pressure_offset_reading_

```
int16_t util::xmsg::XEngSurface::pressure_offset_reading_  [private]
```

The measured pressure sensor offset at the surface, in 1/1000 dBar.

10.84.4.27 start_time_

```
ptime util::xmsg::XAlamoEngineering::start_time_  [private], [inherited]
```

The engineering message state start time.

Referenced by util::xmsg::XAlamoEngineering::set_start_time(), and util::xmsg::XAlamoEngineering::start_time().

10.84.4.28 system_current_ma_

```
uint16_t util::xmsg::XEngSurface::system_current_ma_  [private]
```

The measured system current, in mA.

Note

This field is only used for message version 0x10. It was removed in version 0x11.

Referenced by system_current_a().

10.84.4.29 xmsg_size_bytes_

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes_  [private], [inherited]
```

The size of the X message, in bytes.

Referenced by util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes().

The documentation for this class was generated from the following file:

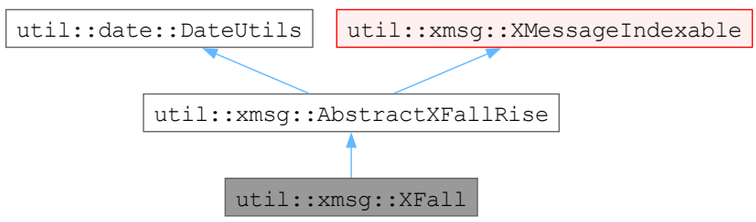
- data/xmsg/XEngSurface.h

10.85 util::xmsg::XFall Class Reference

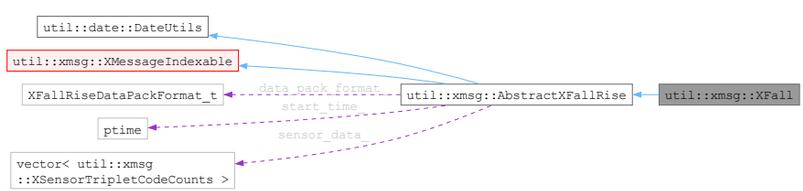
Class representing a fall time-series data X-Message.

`#include <XFall.h>`

Inheritance diagram for util::xmsg::XFall:



Collaboration diagram for util::xmsg::XFall:



Public Member Functions

- XFall (int16_t dive, uint16_t whoi, const vector< uint8_t > &data)

  Parameterized constructor.
- ~XFall ()

  Destructor.
- const uint8_t * CheckXMsgData (const vector< uint8_t > &data, int id, int minBytes, bool parseSize=true)

  Check the X-message for the correct ID, minimum size, and message terminator.
- XFallRiseDataPackFormat_t data_pack_format () const

  Get the enumerated data packing format type.
- void DecodeXMsgData (const vector< uint8_t > &data)

  Decode the fall data X-Message from the binary message data.
- const string GenerateListingString (uint16_t pg, uint16_t po, uint16_t sg, uint16_t so, uint16_t tg, uint16_t to, float pc) const

  Get the listing entry string for this fall data X-Message.
- int16_t id_dive () const

  Get the dive or cycle number of this X-Message.
- uint16_t id_instrument () const

  Get the unique instrument identifier of this X-Message.
- XMsgTypeId_t id_xmsg_type () const

  Get the enumerated X-Message type identifier.
- size_t num_samples () const

  Get the number of samples in this message.
- const vector< XSensorTripletCodeCounts > & sensor_data () const

  Get the vector of sensor data triplets.
- const ptime & start_time () const

  Get the start time of this message.
- uint8_t xmsg_index () const

  Get the index of this X-Message.
- uint16_t xmsg_size_bytes () const

  Get the size of the X-Message, in bytes.

Static Public Member Functions

- static string GenerateCurrentUtcTimeIso8601String ()

  Generate an ISO8601 format date/time string representation of the current time, in UTC.
- static string GenerateIso8601String (const ptime &p, bool extended=true)

  Generate an ISO8601 format date/time string representation of a POSIX time object.
- static double GenerateJulianDay (const ptime &dt, const ptime &ref_dt)

  Convert a date/time to fractional days since a reference date/time.
- static const string GenerateListingComment ()

  Get the listing entry comment string.
- static ptime GetCurrentUtcTime ()

  Generate a POSIX time object representing the current date and time (rounded to the nearest second).

Protected Member Functions

- void set_data_pack_format (XFallRiseDataPackFormat_t data_pack_format)

  Set the enumerated data packing format value for this message.
- void set_id_dive (int16_t id_dive)

  Set the dive or cycle number of this message.
- void set_id_instrument (uint16_t id_instrument)

  Set the unique instrument identifier.
- void set_id_xmsg_type (XMsgTypeId_t id_xmsg_type)

  Set the X-Message type identifier.
- void set_start_time (const ptime &start_time)

  Set the start time of this message.
- void set_xmsg_index (uint8_t xmsg_index)

  Set the index of this X-Message.
- void set_xmsg_size_bytes (uint16_t xmsg_size_bytes)

  Set the size of the X-Message, in bytes.
- vector< XSensorTripletCodeCounts > UnpackData (const vector< uint8_t > &data)

  Unpack the fall/rise data from the binary message.

Private Attributes

- XFallRiseDataPackFormat_t data_pack_format_

  The packing format type identifier.
- int16_t id_dive_

  The dive or cycle number of this message.
- uint16_t id_instrument_

  The unique WHOI identifier of this instrument.
- XMsgTypeId_t id_xmsg_type_

  The X-message type identifier.
- vector< XSensorTripletCodeCounts > sensor_data_

  The raw counts of measured fall, rise, or surface time-series data.
- ptime start_time_

  The start time reference.
- uint8_t xmsg_index_

  The index of this X-Message.
- uint16_t xmsg_size_bytes_

  The size of the X message, in bytes.

10.85.1 Detailed Description

Class representing a fall time-series data X-Message.

10.85.2 Constructor & Destructor Documentation

10.85.2.1 XFall()

```
util::xmsg::XFall::XFall (
            int16_t dive,
            uint16_t whoi,
            const vector< uint8_t > & data)  [inline]
```

Parameterized constructor.

Parameters

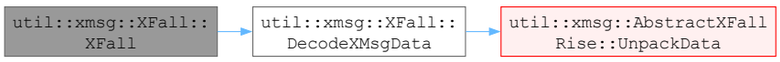
in	*dive*	The dive or cycle number of this message.
in	*whoi*	The unique WHOI identifier.
in	*data*	The binary fall rate X-message data to decode.

Exceptions

log::MessageException	if an decode error occurs.

References DecodeXMsgData().

Here is the call graph for this function:



10.85.2.2 ~XFall()

```
util::xmsg::XFall::~XFall ()  [inline]
```

Destructor.

10.85.3 Member Function Documentation

10.85.3.1 CheckXMsgData()

```
const uint8_t * util::xmsg::XMessage::CheckXMsgData (
            const vector< uint8_t > & data,
            int id,
            int minBytes,
            bool parseSize = true)  [inline], [inherited]
```

Check the X-message for the correct ID, minimum size, and message terminator.

Parameters

in	*data*	The X-message binary data to check.
in	*id*	The expected ID of the X-message. If this field is less that 0, then the ID check will be skipped.
in	*minBytes*	The minimum number of bytes expected in a valid message.
in	*parseSize*	If TRUE, then the message size field will be parsed from the X-Message. Otherwise, the size field will not be parsed.

Exceptions

util::log::MessageException	if • the message does not contain the expected ID, • the message does not contain the minimum number of expected bytes, • the message does not contain a terminator (';') in the correct location, or, • any other error occurs.

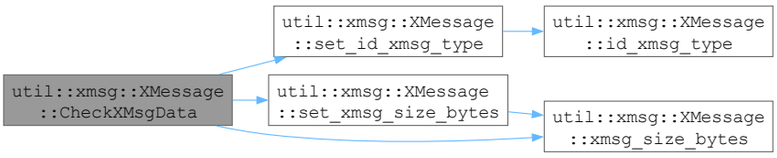
Returns

A pointer to the underlying data buffer of data.

References CATCH_BLOCK, EXC_THROW, lwrn, util::xmsg::XMessage::set_id_xmsg_type(), util::xmsg::XMessage::set_xmsg_size_byte
u16, and util::xmsg::XMessage::xmsg_size_bytes().

Referenced by util::xmsg::XAcousticPosition::DecodeXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XDriftPressure::Dec
util::xmsg::XDriftSalinity::DecodeXMsgData(), util::xmsg::XDriftTemperature::DecodeXMsgData(), util::xmsg::XEngAbort::DecodeXMsgDa
util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngBIT::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(),
util::xmsg::XEngDiagnostic::DecodeXMsgData(), util::xmsg::XEngDrift::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(),
util::xmsg::XEngStandard::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XGPS::DecodeXMsgData(),
util::xmsg::XHiResPressure::DecodeXMsgData(), util::xmsg::XHiResSalinity::DecodeXMsgData(), util::xmsg::XHiResTemperature::Decod
util::xmsg::XMissionCmdEcho::DecodeXMsgData(), util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XProfilePressure::
util::xmsg::XProfileSalinity::DecodeXMsgData(), util::xmsg::XProfileTemperature::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgD
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the call graph for this function:



10.85.3.2 data_pack_format()

```
XFallRiseDataPackFormat_t util::xmsg::AbstractXFallRise::data_pack_format () const  [inline],
[inherited]
```

Get the enumerated data packing format type.

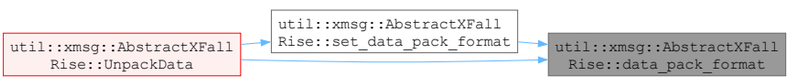
Returns

The enumerated data packing format type.

References util::xmsg::AbstractXFallRise::data_pack_format_.

Referenced by util::xmsg::AbstractXFallRise::set_data_pack_format(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the caller graph for this function:



10.85.3.3 DecodeXMsgData()

```
void util::xmsg::XFall::DecodeXMsgData (
            const vector< uint8_t > & data)  [inline], [virtual]
```

Decode the fall data X-Message from the binary message data.

Parameters

in	*data*	The binary message data to decode.

Exceptions

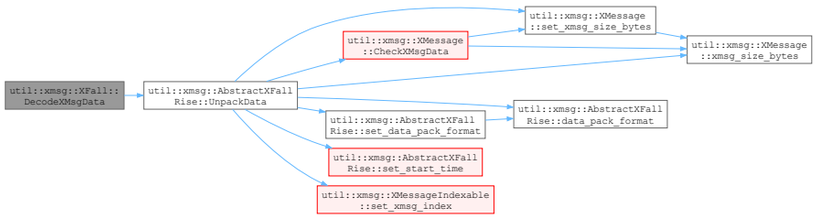
log::MessageException	if the message is invalid or an error occurs during decoding.

Implements util::xmsg::AbstractXFallRise.

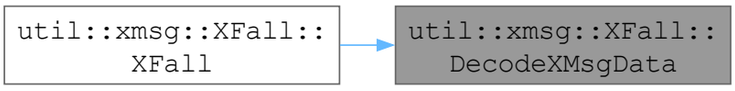
References CATCH_BLOCK, EXC_THROW, util::xmsg::AbstractXFallRise::UnpackData(), and util::xmsg::XMSG_TYPE_ID_FALL0.

Referenced by XFall().

Here is the call graph for this function:



Here is the caller graph for this function:



10.85.3.4 GenerateCurrentUtcTimeIso8601String()

`static string util::date::DateUtils::GenerateCurrentUtcTimeIso8601String () [inline], [static], [inherited]`

Generate an ISO8601 format date/time string representation of the current time, in UTC.

Note

All date/time objects use the UTC time-zone.

Returns

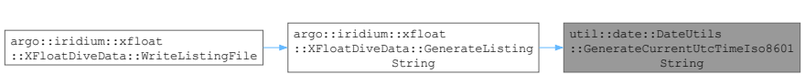
An ISO8601 format date/time string representation of the current time.

References util::date::DateUtils::GenerateIso8601String(), and util::date::DateUtils::GetCurrentUtcTime().

Referenced by argo::iridium::xfloat::XFloatDiveData::GenerateListingString().

Here is the call graph for this function:

Here is the caller graph for this function:



10.85.3.5 GenerateIso8601String()

```
static string util::date::DateUtils::GenerateIso8601String (
            const ptime & p,
            bool extended = true)  [inline], [static], [inherited]
```

Generate an ISO8601 format date/time string representation of a POSIX time object.

Note

All date/time objects use the UTC time-zone.

Parameters

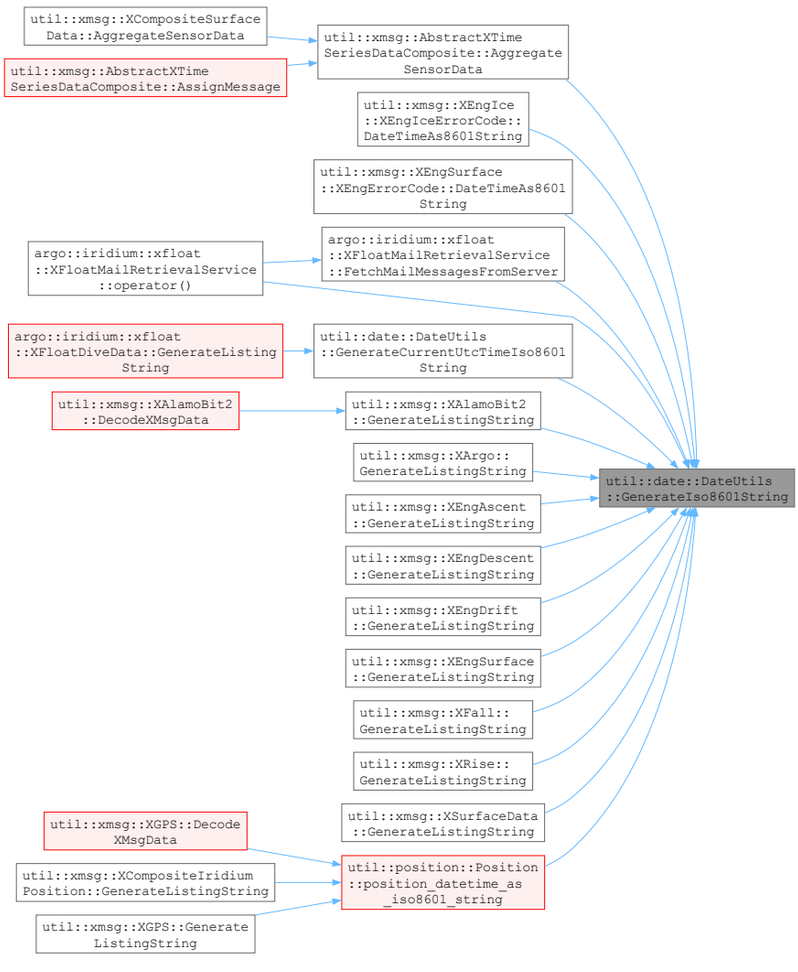
in	*p*	POSIX time from which to generate string.
in	*extended*	If TRUE, use the ISO 8601 extended format (YYYY-MM-DDThh:mm:ssZ); otherwise, use the basic format (YYYYMMDDThhmmss).

Returns

An ISO8601 format date/time string representation of the POSIX time object. If p is equal to boost::posix_↩ time::not_a_date_time, then the string "0000-00-00T00:00:00Z" (extended) or "00000000000000" (basic) will be returned.

Referenced by util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::AggregateSensorData(), util::xmsg::XEngIce::XEngIce
util::xmsg::XEngSurface::XEngErrorCode::DateTimeAs8601String(), argo::iridium::xfloat::XFloatMailRetrievalService::FetchMailMessages
util::date::DateUtils::GenerateCurrentUtcTimeIso8601String(), util::xmsg::XAlamoBit2::GenerateListingString(), util::xmsg::XArgo::Genera
util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListing
util::xmsg::XEngSurface::GenerateListingString(), GenerateListingString(), util::xmsg::XRise::GenerateListingString(),
util::xmsg::XSurfaceData::GenerateListingString(), argo::iridium::xfloat::XFloatMailRetrievalService::operator()(), and
util::position::Position::position_datetime_as_iso8601_string().

Here is the caller graph for this function:



10.85.3.6 GenerateJulianDay()

```
static double util::date::DateUtils::GenerateJulianDay (
            const ptime & dt,
            const ptime & ref_dt)  [inline], [static], [inherited]
```

Convert a date/time to fractional days since a reference date/time.

Parameters

in	*dt*	The date/time to convert.
in	*ref↩ _dt*	The reference date/time (day 0).

Exceptions

util::log::MessageException	if either date/time is not valid.

Returns

The number of days (with millisecond resolution) from `ref_dt` to `dt`.

References CATCH_BLOCK, and EXC_THROW.

10.85.3.7 GenerateListingComment()

```
static const string util::xmsg::AbstractXFallRise::GenerateListingComment ()  [inline], [static],
[inherited]
```

Get the listing entry comment string.

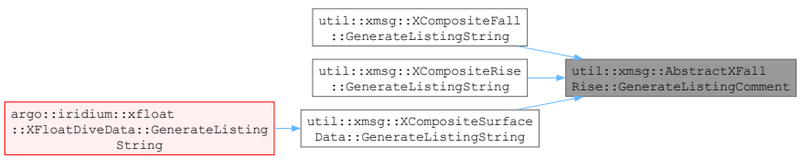
Returns

The listing entry comment string.

References CATCH_BLOCK.

Referenced by util::xmsg::XCompositeFall::GenerateListingString(), util::xmsg::XCompositeRise::GenerateListingString(), and util::xmsg::XCompositeSurfaceData::GenerateListingString().

Here is the caller graph for this function:



10.85.3.8 GenerateListingString()

```
const string util::xmsg::XFall::GenerateListingString (
            uint16_t pg,
            uint16_t po,
            uint16_t sg,
            uint16_t so,
            uint16_t tg,
            uint16_t to,
            float pc) const  [inline]
```

Get the listing entry string for this fall data X-Message.

Parameters

in	*pg*	The pressure sensor sensitivity scale factor, in LSB/dBar.
in	*po*	The pressure sensor offset, in dBar.
in	*sg*	The salinity sensor sensitivity scale factor, in LSB/PSU.
in	*so*	The salinity sensor offset, in PSU.
in	*tg*	The temperature sensor sensitivity scale factor, in LSB/degC.
in	*to*	The temperature sensor offset, in degC.
in	*pc*	The pressure correction, in dBar, to be subtracted computed pressure value.

Exceptions

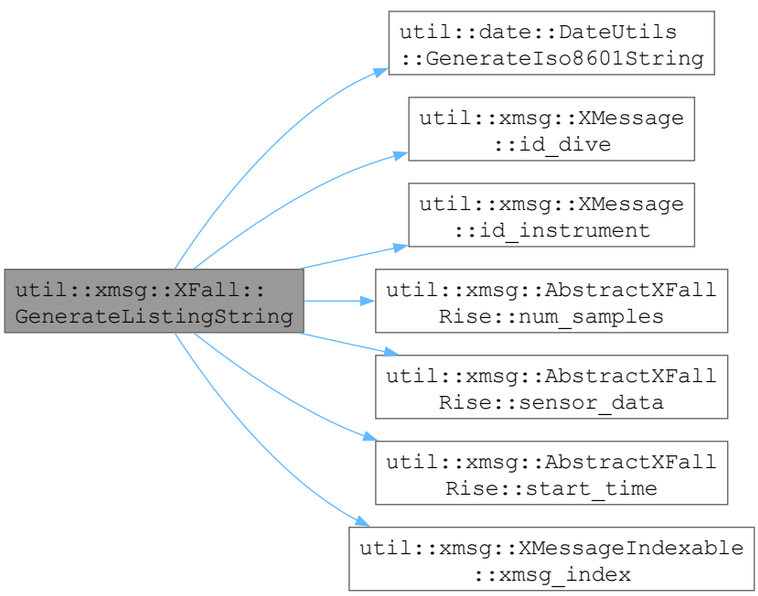
log::MessageException	if an error occurs while generating the listing string.

Returns

The listing entry string for this fall data X-Message

References CATCH_BLOCK, util::date::DateUtils::GenerateIso8601String(), util::xmsg::XMessage::id_dive(), util::xmsg::XMessage::id_ins
util::xmsg::AbstractXFallRise::num_samples(), util::xmsg::AbstractXFallRise::sensor_data(), util::xmsg::AbstractXFallRise::start_time(),
and util::xmsg::XMessageIndexable::xmsg_index().

Here is the call graph for this function:



10.85.3.9 GetCurrentUtcTime()

```
static ptime util::date::DateUtils::GetCurrentUtcTime ()  [inline], [static], [inherited]
```

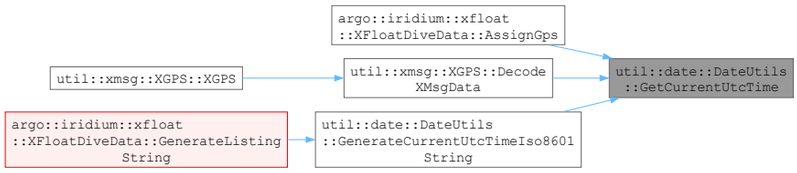
Generate a POSIX time object representing the current date and time (rounded to the nearest second).

Returns

A POSIX time object representing the current date and time (rounded to the nearest second).

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignGps(), util::xmsg::XGPS::DecodeXMsgData(), and util::date::DateUtils::GenerateCurrentUtcTimeIso8601String().

Here is the caller graph for this function:



10.85.3.10 id_dive()

```
int16_t util::xmsg::XMessage::id_dive () const  [inline], [inherited]
```

Get the dive or cycle number of this X-Message.

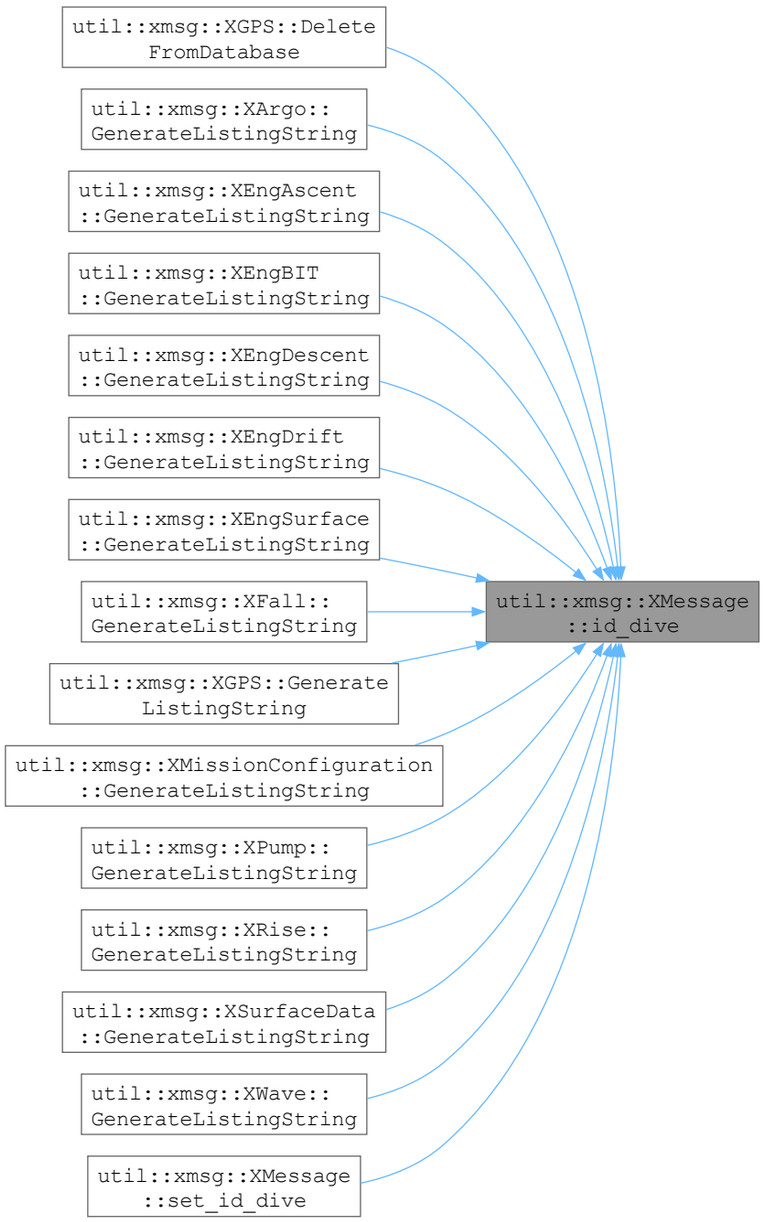
Returns

The dive or cycle number of this X-Message.

References util::xmsg::XMessage::id_dive_.

Referenced by util::xmsg::XGPS::DeleteFromDatabase(), util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateLis
util::xmsg::XEngBIT::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingStr
util::xmsg::XEngSurface::GenerateListingString(), GenerateListingString(), util::xmsg::XGPS::GenerateListingString(),
util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing
util::xmsg::XSurfaceData::GenerateListingString(), util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_dive().

Here is the caller graph for this function:



10.85.3.11 id_instrument()

```
uint16_t util::xmsg::XMessage::id_instrument () const  [inline], [inherited]
```

Get the unique instrument identifier of this X-Message.

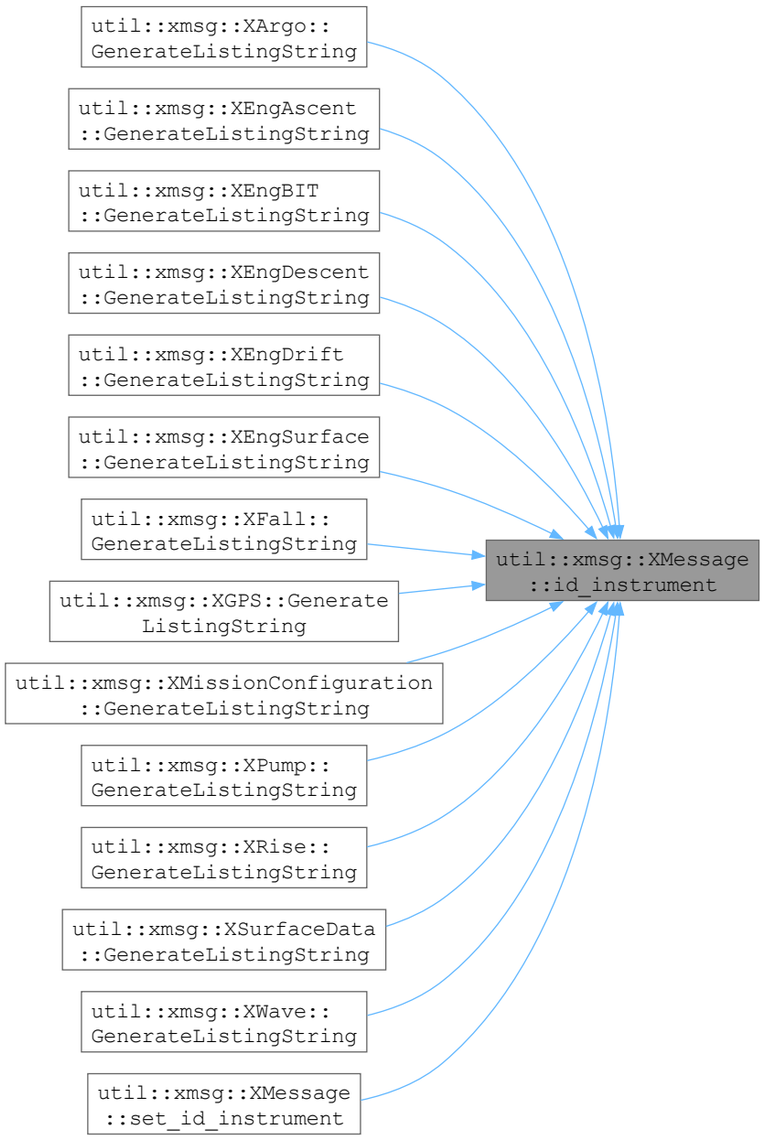
Returns

The unique instrument identifier of this X-Message.

References util::xmsg::XMessage::id_instrument_.

Referenced by util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngBIT::Generate
util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), util::xmsg::XEngSurface::GenerateListin
GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingString(),
util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(),
util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_instrument().

Here is the caller graph for this function:



10.85.3.12 id_xmsg_type()

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type () const  [inline], [inherited]
```

Get the enumerated X-Message type identifier.

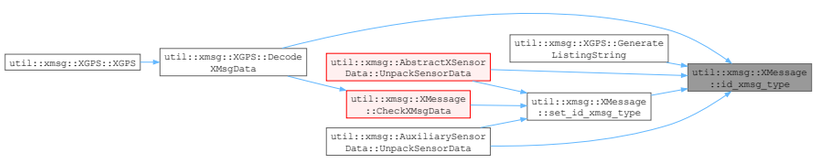
Returns

The enumerated X-Message type identifier.

References util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMessage::set_id_xmsg_typ
util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the caller graph for this function:



10.85.3.13 num_samples()

```
size_t util::xmsg::AbstractXFallRise::num_samples () const  [inline], [inherited]
```

Get the number of samples in this message.

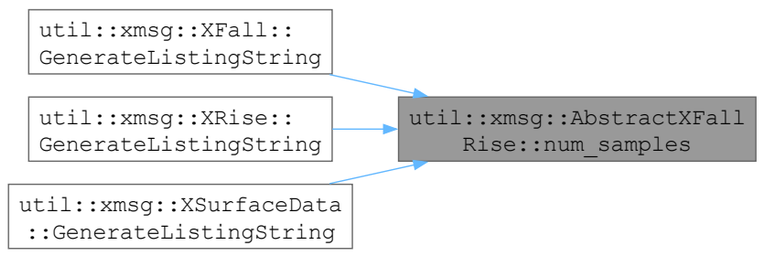
Returns

The number of samples in this message.

References util::xmsg::AbstractXFallRise::sensor_data_.

Referenced by GenerateListingString(), util::xmsg::XRise::GenerateListingString(), and util::xmsg::XSurfaceData::GenerateListingString().

Here is the caller graph for this function:



10.85.3.14 sensor_data()

```
const vector< XSensorTripletCodeCounts > & util::xmsg::AbstractXFallRise::sensor_data () const
[inline], [inherited]
```

Get the vector of sensor data triplets.

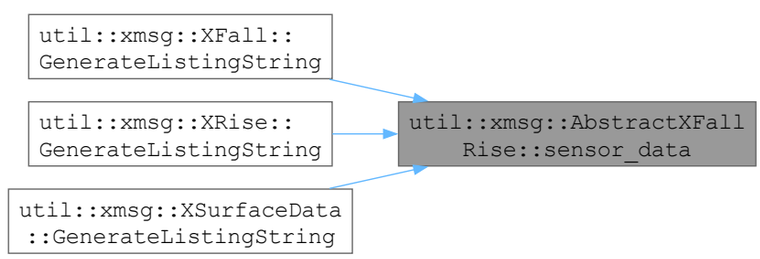
Returns

The vector of sensor data triplets.

References util::xmsg::AbstractXFallRise::sensor_data_.

Referenced by GenerateListingString(), util::xmsg::XRise::GenerateListingString(), and util::xmsg::XSurfaceData::GenerateListingString().

Here is the caller graph for this function:



10.85.3.15 set_data_pack_format()

```
void util::xmsg::AbstractXFallRise::set_data_pack_format (
            XFallRiseDataPackFormat_t data_pack_format)  [inline], [protected], [inherited]
```

Set the enumerated data packing format value for this message.

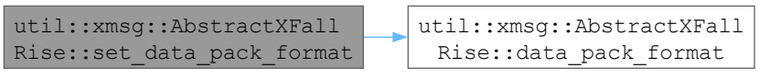
Parameters

in	*data_pack_format*	The enumerated data packing format value for this message to set.

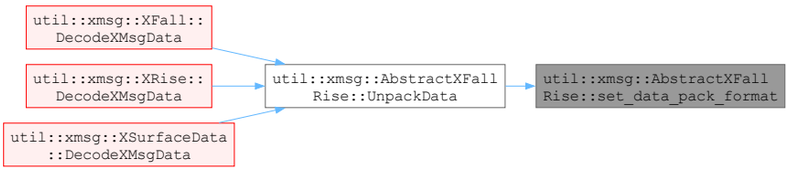
References util::xmsg::AbstractXFallRise::data_pack_format(), and util::xmsg::AbstractXFallRise::data_pack_format_.

Referenced by util::xmsg::AbstractXFallRise::UnpackData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.85.3.16 set_id_dive()

```
void util::xmsg::XMessage::set_id_dive (
            int16_t id_dive)  [inline], [protected], [inherited]
```

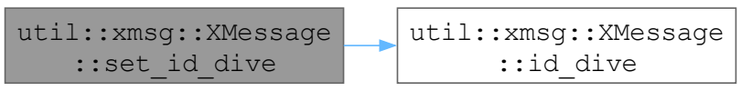
Set the dive or cycle number of this message.

Parameters

in	*id_dive*	The dive or cycle number of this message to set.

References util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::id_dive_.

Here is the call graph for this function:



10.85.3.17 set_id_instrument()

```
void util::xmsg::XMessage::set_id_instrument (
            uint16_t id_instrument)  [inline], [protected], [inherited]
```

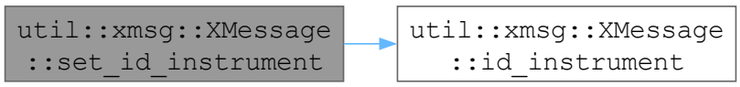
Set the unique instrument identifier.

Parameters

in	*id_instrument*	The unique instrument identifier to set.

References util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::id_instrument_.

Here is the call graph for this function:



10.85.3.18 set_id_xmsg_type()

```
void util::xmsg::XMessage::set_id_xmsg_type (
            XMsgTypeId_t id_xmsg_type)  [inline], [protected], [inherited]
```

Set the X-Message type identifier.

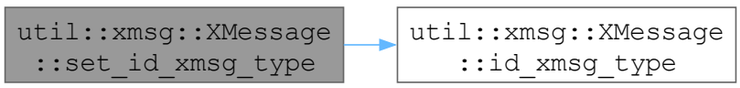
Parameters

in	*id_xmsg_type*	The X-Message type identifier to set.

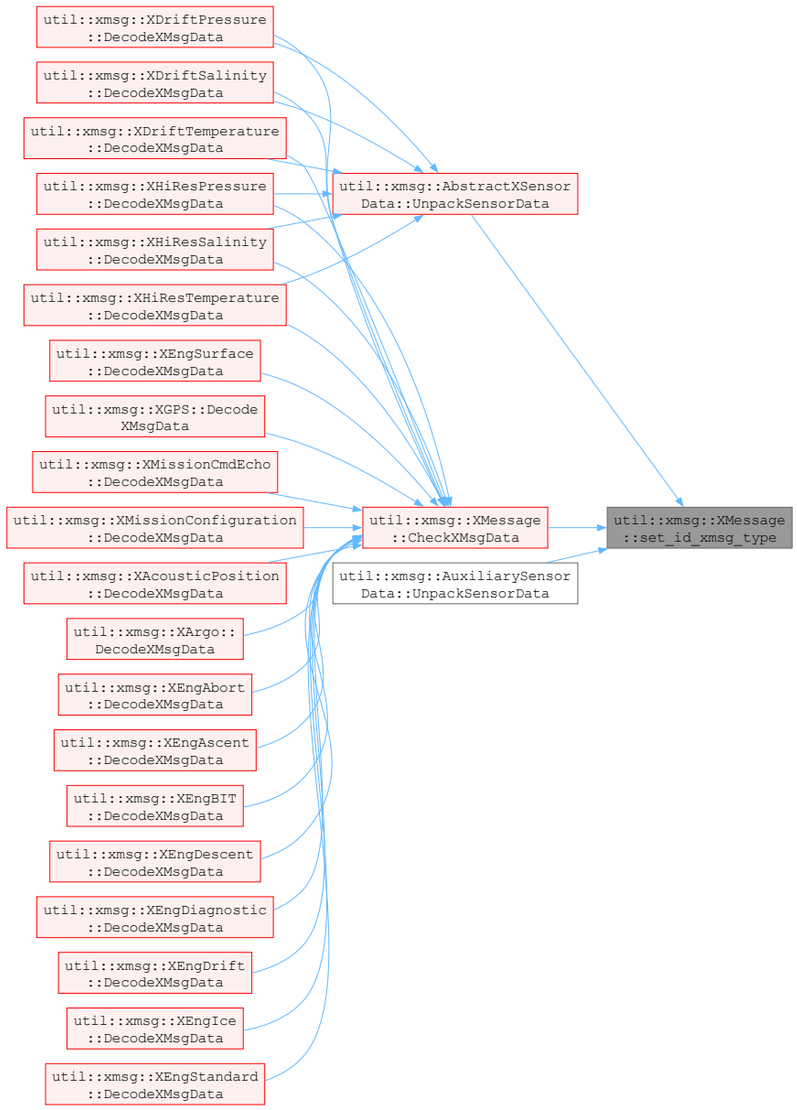
References util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.85.3.19 set_start_time()

```
void util::xmsg::AbstractXFallRise::set_start_time (
            const ptime & start_time)  [inline], [protected], [inherited]
```

Set the start time of this message.

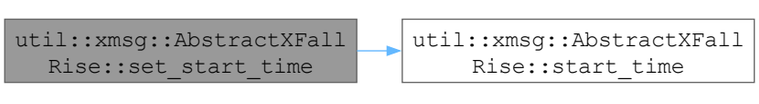
Parameters

in	*start_time*	The start time to set.

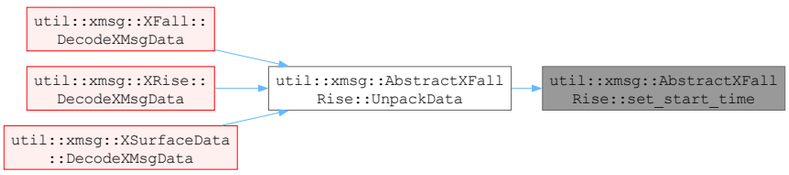
References util::xmsg::AbstractXFallRise::start_time(), and util::xmsg::AbstractXFallRise::start_time_.

Referenced by util::xmsg::AbstractXFallRise::UnpackData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.85.3.20 set_xmsg_index()

```
void util::xmsg::XMessageIndexable::set_xmsg_index (
            uint8_t xmsg_index)  [inline], [protected], [inherited]
```

Set the index of this X-Message.

Parameters

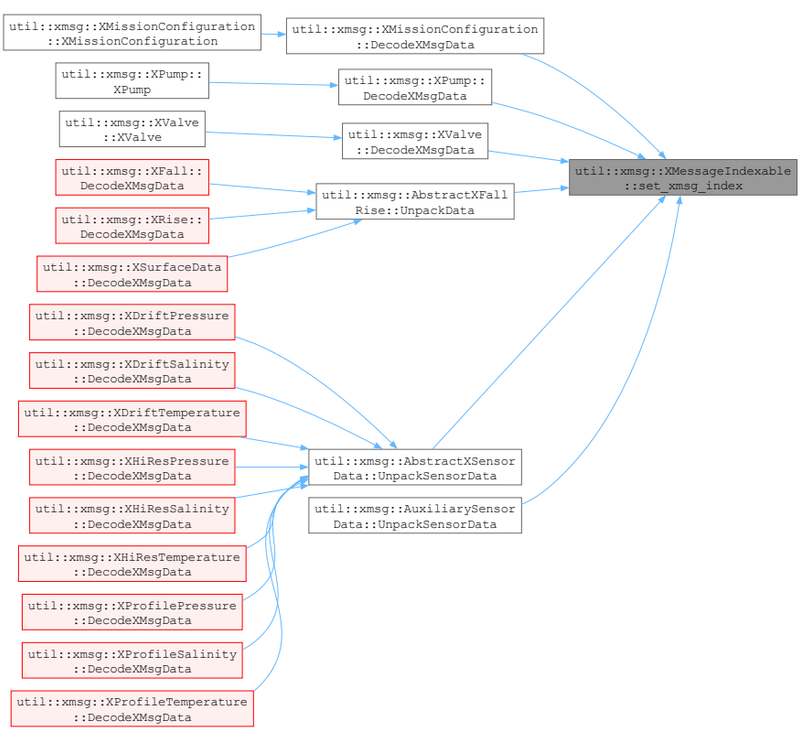
in	*xmsg_index*	The index of this X-Message.

References util::xmsg::XMessageIndexable::xmsg_index(), and util::xmsg::XMessageIndexable::xmsg_index_.

Referenced by util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(), util::xmsg::XValve::DecodeXMsgData(), util::xmsg::AbstractXFallRise::UnpackData(), util::xmsg::AbstractXSensorData::UnpackSensorDa and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:

Here is the caller graph for this function:



10.85.3.21 set_xmsg_size_bytes()

```
void util::xmsg::XMessage::set_xmsg_size_bytes (
            uint16_t xmsg_size_bytes)  [inline], [protected], [inherited]
```

Set the size of the X-Message, in bytes.

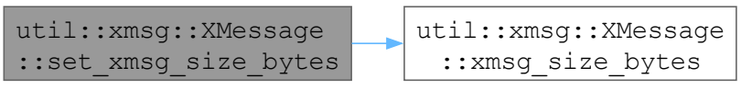
Parameters

in	*xmsg_size_bytes*	The size of the X-Message to set, in bytes.

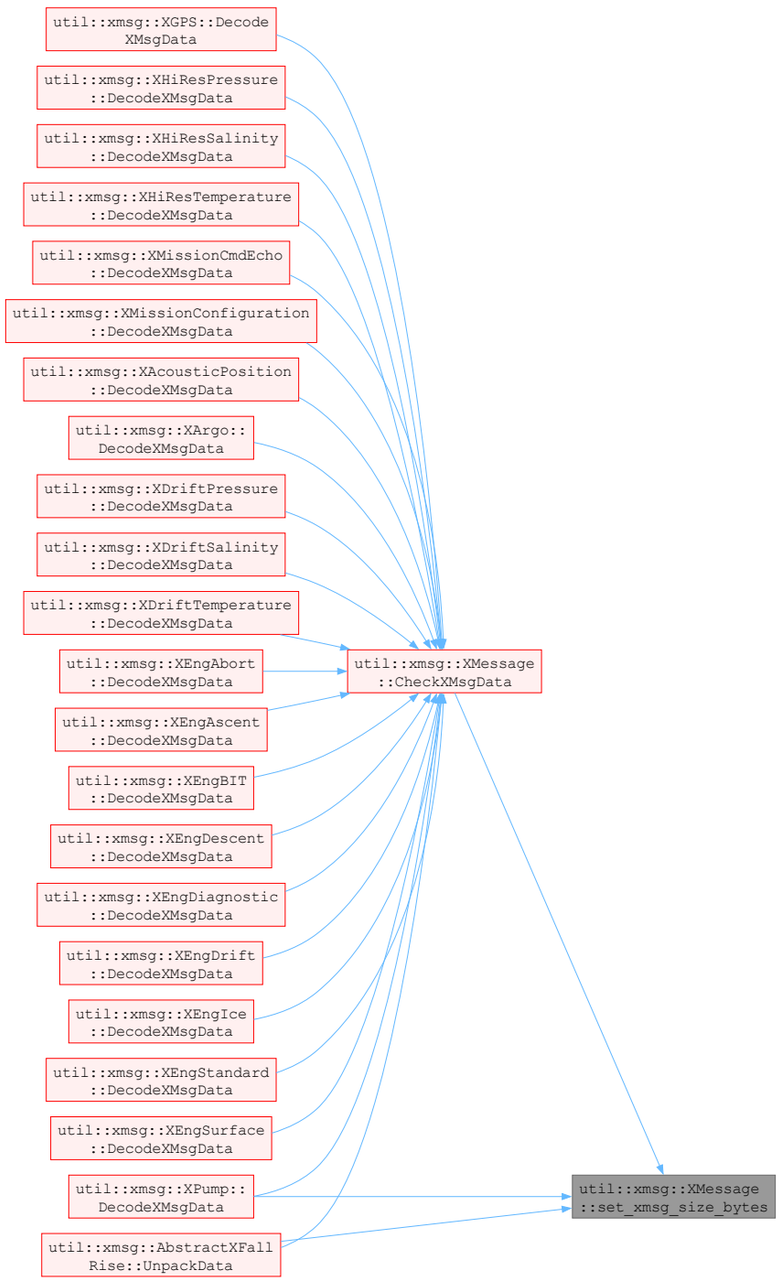
References util::xmsg::XMessage::xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XPump::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::Unpa

Here is the call graph for this function:



Here is the caller graph for this function:



10.85.3.22 start_time()

```
const ptime & util::xmsg::AbstractXFallRise::start_time () const  [inline], [inherited]
```

Get the start time of this message.

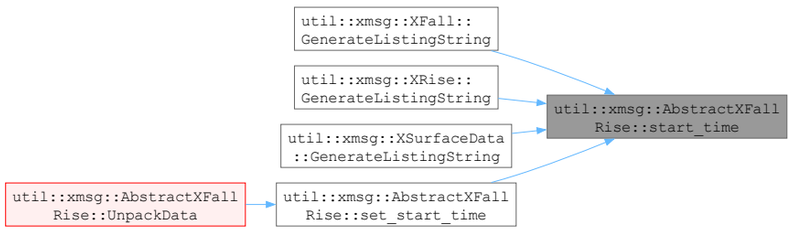
Returns

The start time of this message.

References util::xmsg::AbstractXFallRise::start_time_.

Referenced by GenerateListingString(), util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(), and util::xmsg::AbstractXFallRise::set_start_time().

Here is the caller graph for this function:



10.85.3.23 UnpackData()

```
vector< XSensorTripletCodeCounts > util::xmsg::AbstractXFallRise::UnpackData (
            const vector< uint8_t > & data)  [inline], [protected], [inherited]
```

Unpack the fall/rise data from the binary message.

Parameters

in	*data*	The binary message to unpack.

Exceptions

util::log::MessageException	if • the message is of insufficient length, • an invalid packing format is encountered, or • the message terminator is in an invalid location.

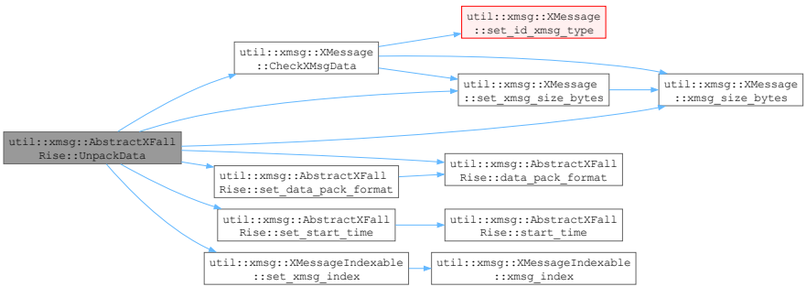
Returns

The raw counts of measured fall or rise time-series data.

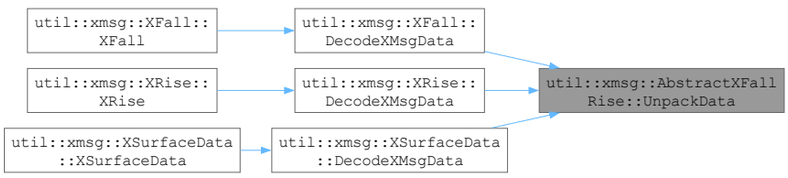
References CATCH_BLOCK, util::xmsg::XMessage::CheckXMsgData(), util::xmsg::AbstractXFallRise::data_pack_format(), EXC_THROW, lwrn, s2000, util::xmsg::AbstractXFallRise::sensor_data_, util::xmsg::AbstractXFallRise::set_data_pack_format(), util::xmsg::AbstractXFallRise::set_start_time(), util::xmsg::XMessageIndexable::set_xmsg_index(), util::xmsg::XMessage::set_xmsg_size_ XMSG_SENS_BAD, util::xmsg::XMessage::xmsg_size_bytes(), util::xmsg::XPHASE_CODE_UNKNOWN, util::xmsg::XRISEFALL_FMT_ util::xmsg::XRISEFALL_FMT_STNDRD, util::xmsg::XRISEFALL_FMT_STNDRD_WITH_TEMP_AND_SALN, and util::xmsg::XRISEFALL_FMT_UNKNOWN.

Referenced by DecodeXMsgData(), util::xmsg::XRise::DecodeXMsgData(), and util::xmsg::XSurfaceData::DecodeXMsgData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.85.3.24 xmsg_index()

```
uint8_t util::xmsg::XMessageIndexable::xmsg_index () const  [inline], [inherited]
```

Get the index of this X-Message.

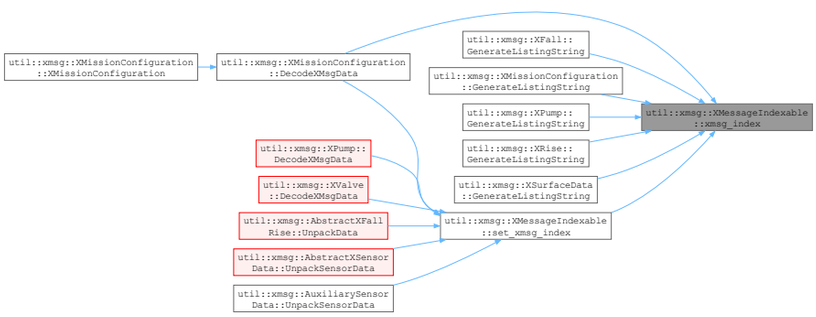
Returns

The index of this X-Message.

References util::xmsg::XMessageIndexable::xmsg_index_.

Referenced by util::xmsg::XMissionConfiguration::DecodeXMsgData(), GenerateListingString(), util::xmsg::XMissionConfiguration::Genera
util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(),
and util::xmsg::XMessageIndexable::set_xmsg_index().

Here is the caller graph for this function:



10.85.3.25 xmsg_size_bytes()

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes () const  [inline], [inherited]
```

Get the size of the X-Message, in bytes.

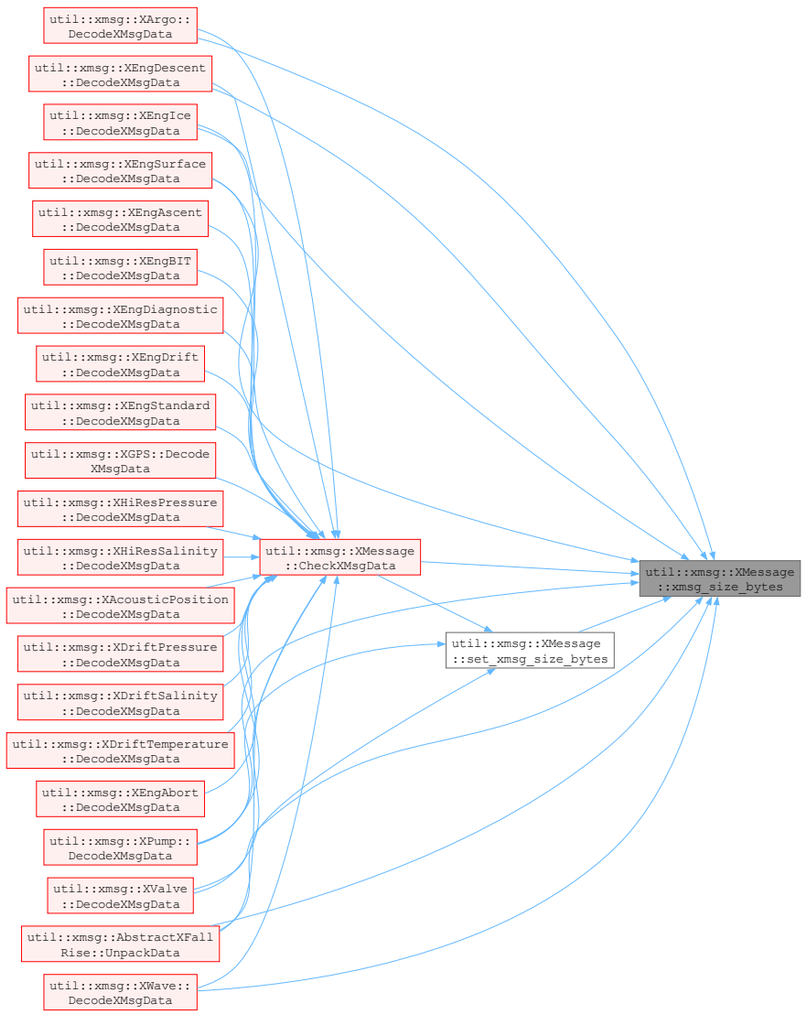
Returns

The size of the X-Message, in bytes.

References util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgI
util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(),
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), util::xmsg::XMessage::set_xmsg_size_bytes(),
and util::xmsg::AbstractXFallRise::UnpackData().

Here is the caller graph for this function:



10.85.4 Member Data Documentation

10.85.4.1 data_pack_format_

```
XFallRiseDataPackFormat_t util::xmsg::AbstractXFallRise::data_pack_format_  [private], [inherited]
```

The packing format type identifier.

Referenced by util::xmsg::AbstractXFallRise::data_pack_format(), and util::xmsg::AbstractXFallRise::set_data_pack_format().

10.85.4.2 id_dive_

int16_t util::xmsg::XMessage::id_dive_ [private], [inherited]

The dive or cycle number of this message.

Referenced by util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::set_id_dive().

10.85.4.3 id_instrument_

uint16_t util::xmsg::XMessage::id_instrument_ [private], [inherited]

The unique WHOI identifier of this instrument.

Referenced by util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::set_id_instrument().

10.85.4.4 id_xmsg_type_

XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type_ [private], [inherited]

The X-message type identifier.

Referenced by util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::set_id_xmsg_type().

10.85.4.5 sensor_data_

vector<XSensorTripletCodeCounts> util::xmsg::AbstractXFallRise::sensor_data_ [private], [inherited]

The raw counts of measured fall, rise, or surface time-series data.

Referenced by util::xmsg::AbstractXFallRise::num_samples(), util::xmsg::AbstractXFallRise::sensor_data(), and util::xmsg::AbstractXFallRise::UnpackData().

10.85.4.6 start_time_

ptime util::xmsg::AbstractXFallRise::start_time_ [private], [inherited]

The start time reference.

Referenced by util::xmsg::AbstractXFallRise::set_start_time(), and util::xmsg::AbstractXFallRise::start_time().

10.85.4.7 xmsg_index_

uint8_t util::xmsg::XMessageIndexable::xmsg_index_ [private], [inherited]

The index of this X-Message.

Referenced by util::xmsg::XMessageIndexable::set_xmsg_index(), and util::xmsg::XMessageIndexable::xmsg_index().

10.85.4.8 xmsg_size_bytes_

uint16_t util::xmsg::XMessage::xmsg_size_bytes_ [private], [inherited]

The size of the X message, in bytes.

Referenced by util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes().

The documentation for this class was generated from the following file:

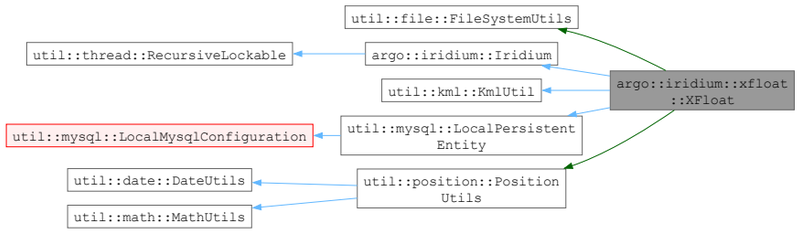
- data/xmsg/XFall.h

10.86 argo::iridium::xfloat::XFloat Class Reference

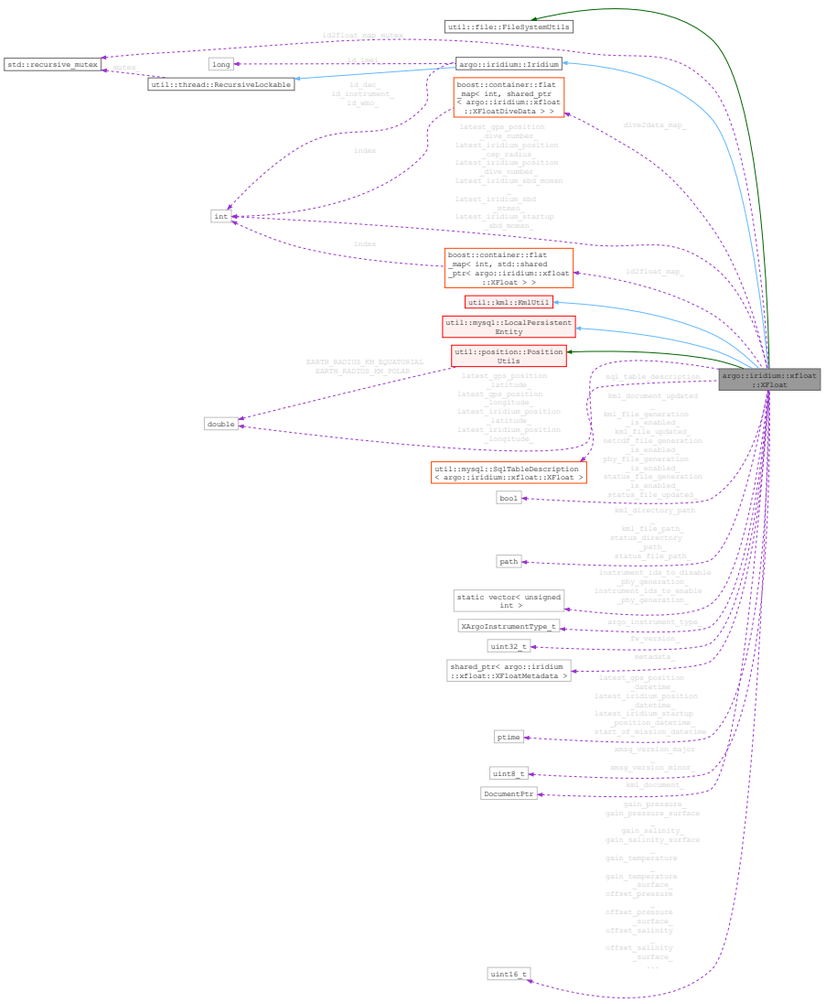
The instrument or float class.

`#include <XFloat.h>`

Inheritance diagram for argo::iridium::xfloat::XFloat:



Collaboration diagram for argo::iridium::xfloat::XFloat:



Public Member Functions

- XFloat ()

  The default constructor.
- XFloat (int whoi, long long imei=XFLT_ID_UNKNOWN, int wmo=XFLT_ID_UNKNOWN, int dac=XFLT_ID_UNKNOWN)

  The parameterized constructor.
- ~XFloat ()

  The destructor.
- uint8_t argo_instrument_type () const

Get the enumerated Argo instrument type identifier.

- uint32_t fw_version () const

  Get the firmware version.

- uint16_t gain_pressure () const ATTR_DEPRECATED

  Get the pressure gain value, in dBar/LSB.

- uint16_t gain_pressure_surface () const ATTR_DEPRECATED

  Get the pressure gain value for surface data, in dBar/LSB.

- uint16_t gain_salinity () const ATTR_DEPRECATED

  Get the salinity gain value, in PSU/LSB.

- uint16_t gain_salinity_surface () const ATTR_DEPRECATED

  Get the salinity gain value for surface data, in PSU/LSB.

- uint16_t gain_temperature () const ATTR_DEPRECATED

  Get the temperature gain value, in degC/LSB.

- uint16_t gain_temperature_surface () const ATTR_DEPRECATED

  Get the temperature gain value for surface data, in degC/LSB.

- vector< shared_ptr< XFloatDiveData > > GetAllDiveData ()

  Get all dive data objects for this float.

- shared_ptr< XFloatDiveData > GetDiveData (int dive, bool load_from_local_cache_only=false)

  Get the dive data object from the dive or cycle number.

- vector< shared_ptr< x::XGPS > > **GetGpsPositions** () const
- path GetListingDirectoryPath () const

  Get the listing directory path for this float.

- int id_dac () const

  Get the unique Data Assembly Center (DAC) identifier.

- long id_imei () const

  Get the unique International Mobile Equipment Identifier (IMEI).

- int id_instrument () const

  Get the unique instrument identifier or instrument serial number.

- int id_wmo () const

  Get the unique World Meteorological Organization (WMO) identifier.

- bool **is_kml_document_updated** () const
- bool **is_kml_file_updated** () const
- bool **is_status_file_updated** () const
- const kmldom::DocumentPtr & **kml_document** () const
- const path & **kml_file_path** () const
- string **latest_gps_position_as_iso6709_string** () const
- ptime **latest_gps_position_datetime** () const
- int **latest_gps_position_dive_number** () const
- double **latest_gps_position_latitude** () const
- double **latest_gps_position_longitude** () const
- string **latest_iridium_position_as_iso6709_string** () const
- int **latest_iridium_position_cep_radius** () const
- ptime **latest_iridium_position_datetime** () const
- int **latest_iridium_position_dive_number** () const
- double **latest_iridium_position_latitude** () const
- double **latest_iridium_position_longitude** () const
- const ptime & **latest_iridium_startup_position_datetime** () const
- int **latest_iridium_startup_sbd_momsn** () const
- int **latest_sbd_momsn** () const
- int **latest_sbd_mtmsn** () const
- bool LockObject (bool block)

  Lock the object.

- const shared_ptr< XFloatMetadata > & metadata () const

  Get the metadata object for this instrument.

- bool **netcdf_file_generation_is_enabled** () const
- uint16_t offset_pressure () const ATTR_DEPRECATED

  Get the pressure offset value for this instrument, in dBar.

- uint16_t offset_pressure_surface () const ATTR_DEPRECATED

  Get the pressure offset value for surface data, in dBar.

- uint16_t offset_salinity () const ATTR_DEPRECATED

  Get the salinity offset value for this instrument, in PSU.

- uint16_t offset_salinity_surface () const ATTR_DEPRECATED

  Get the salinity offset value for surface data, in PSU.

- uint16_t offset_temperature () const ATTR_DEPRECATED

Get the temperature offset value for this instrument, in degC.

- uint16_t offset_temperature_surface () const ATTR_DEPRECATED

  Get the temperature offset value for surface data, in degC.

- bool phy_file_generation_is_enabled () const

  Is PHY file generation enabled?

- void ResetCachedDiveData (int dive) ATTR_NO_EXCEPTIONS

  Reset the dive data object for the given dive or cycle number.

- void ResetDiveData ()

  Reset the dive data objects for this float.

- ptime start_of_mission_datetime () const

  Get the date/time of the start-of-mission for this instrument.

- const path & **status_file_path** () const
- void UnlockObject ()

  Unlock the object.

- uint8_t xmsg_version_major () const

  Get the X-Message major version identifier for this instrument.

- uint8_t xmsg_version_minor () const

  Get the X-Message minor version identifier for this instrument.

Static Public Member Functions

- static void CloseKmlFactory ()

  Release this thread's KML factory pointer.

- static void CloseSqlConnection ()

  Close the thread-specific MySQL connection if it is currently open.

- static shared_ptr< sql::ResultSet > **executeDeleteQuery** (const string &from, const string &where="")
- static shared_ptr< ResultSet > ExecuteQuery (Connection *con, const string &query)

  Execute an SQL query and return a shared pointer to the ResultSet object.

- static shared_ptr< sql::ResultSet > executeQuery (const string &query)

  Execute an SQL query and return a shared pointer to the ResultSet object.

- static shared_ptr< sql::ResultSet > executeSelectQuery (const string &what, const string &from, const string &where="", const string &groupBy="", const string &orderBy="", unsigned int limit=0, unsigned int offset=0)

  Execute an SQL select query and return a shared pointer to the ResultSet object.

- static int ExecuteUpdate (Connection *con, const string &update)

  Execute an SQL update and return the status code.

- static int executeUpdate (const string &update)

  Execute an SQL update and return the status code.

- static shared_ptr< PreparedStatement > GeneratePreparedStatement (Connection *con, const string &statement)

  Generate a prepared SQL statement object for the given SQL statement.

- static shared_ptr< sql::PreparedStatement > generatePreparedStatement (const string &statement)

  Generate a prepared SQL statement object for the given SQL statement.

- static string GenerateSqlDeleteStr (const string &from, const string &where="")

  Generate an SQL DELETE statement.

- static string GenerateSqlSelectStr (const string &what, const string &from, const string &where="", const string &groupBy="", const string &orderBy="", unsigned int limit=0, unsigned int offset=0)

  Generate an SQL select statement.

- static vector< shared_ptr< XFloat > > GetAllCachedFloats ()

  Get all float objects currently stored in the cache.

- static const Color32 & GetDefaltKmlPathColor (int id)

  Get the palette colour for an index (an instrument ID, for example), modulo the palette size.

- static const string & GetDefaltKmlPathStyleId (int id)

  Get the path (line) style ID for an index, modulo the palette size.

- static const string & GetDefaltKmlPlacemarkStyleId (int id)

  Get the large placemark style ID for an index, modulo the palette size.

- static shared_ptr< XFloat > GetFloat (int instrument_id)

  Get a float object from the instrument identifier.

- static thread_specific_ptr< KmlFactory > & GetKmlFactory ()

  Get the libkml factory, initializing it for this thread if necessary.

- static const string & GetSmallKmlPlacemarkStyleId (int id)

  Get the small placemark style ID for an index, modulo the palette size.

- static thread_specific_ptr< Connection > & GetSqlConnection (string url, string user, string password, string schema)

Get a thread-specific instance of a MySQL connection object.

- static thread_specific_ptr< MySQL_Driver > & GetSqlDriver ()

  Get a thread-specific instance of a MySQL driver object.
- static const string & GetTriangleKmlPlacemarkStyleId (int id)

  Get the triangle placemark style ID for an index, modulo the palette size.
- static const vector< unsigned int > & instrument_ids_to_disable_phy_generation ()

  Get the floats that the user has specified to disable PHY file generation.
- static const vector< unsigned int > & instrument_ids_to_enable_phy_generation ()

  Get the floats that the user has specified to enable PHY file generation.
- static void LoadLocalCacheFromDatabase ()

  Load all persistent instrument data from the database and use this data to populate the local cache.
- static void LoadMetadataFromFiles ()

  Load the instrument metadata.
- static void SaveLocalCacheToDatabase ()

  Save the local cache of X-Float objects to the database.
- static vector< shared_ptr< XFloat > > SelectAllObjectsFromDatabase (const string &groupBy="", const string &orderBy="", const unsigned int &limit=0, const unsigned int &offset=0)

  Select all XFloat objects from this SQL database table.
- static shared_ptr< XFloat > SelectObjectFromDatabase (const string &where="", const string &groupBy="", const string &orderBy="", const unsigned int &offset=0)

  Get a single XFloat object from the database.
- static vector< shared_ptr< XFloat > > SelectObjectsFromDatabase (const string &where="", const string &groupBy="", const string &orderBy="", const unsigned int &limit=0, const unsigned int &offset=0)

  Query float objects from the SQL database.
- template<typename TT >
  static TT SelectValueFromDatabase (const string &what, const string &where="", const string &groupBy="", const string &orderBy="", const unsigned int &offset=0)

  Select a value from the SQL database table that contains the persisted float objects.
- static void SetAllDefaultKmlSyles (const DocumentPtr &document)

  Add the default styles for every palette colour to a KML document.
- static void SetDefaultKmlStyle (const DocumentPtr &document, unsigned int style_idx)

  Add the four default styles for one palette colour to a KML document.
- static void StartupTasks ()

  Perform XFloat startup tasks.

Protected Member Functions

- void set_id_dac (int id_dac)

  Set the unique Data Assembly Center (DAC) identifier.
- void set_id_imei (long id_imei)

  Set the unique International Mobile Equipment Identifier (IMEI).
- void set_id_instrument (int id_instrument)

  Set the unique instrument identifier or instrument serial number.
- void set_id_wmo (int id_wmo)

  Set the unique World Meteorological Organization (WMO) identifier.

Static Protected Member Functions

- static float **bitcastUint32ToFloat** (uint32_t u)
- template<typename T >
  static double CalculateEllipsoidHeight (T lat)

  Get the approximate surface altitude or height of earth for a given latitude, in kilometers.
- template<typename T >
  static double CalculateGeographicDistance (T lat1, T lon1, T lat0, T lon0)

  Get the geographical distance between two position, in kilometers, from the positions' locations, in decimal degrees.
- template<typename T >
  static double degToRad (T deg)

  Convert angular units from from degrees to decimal radians.
- static string GenerateCurrentUtcTimeIso8601String ()

  Generate an ISO8601 format date/time string representation of the current time, in UTC.
- template<typename T >
  static std::string GenerateIso6709String (T lat, T lon)

  Generate an IS06709 string representation of the position given by (`lat`, `lon`).

- static string GenerateIso8601String (const ptime &p, bool extended=true)

  Generate an ISO8601 format date/time string representation of a POSIX time object.
- static double GenerateJulianDay (const ptime &dt, const ptime &ref_dt)

  Convert a date/time to fractional days since a reference date/time.
- static ptime GetCurrentUtcTime ()

  Generate a POSIX time object representing the current date and time (rounded to the nearest second).
- static const path & GetDirectoryPath (const path &dir, bool create=false)

  Get a directory path.
- static path GetDirectoryPathFromString (const string &dir, bool create=false)

  Generate a boost::filesystem::path object from the directory path string.
- static sql::Connection * getSqlConnection ()

  Generate a thread-specific instance of a MySQL connection object.
- template<typename T >
  static bool IsPositionValid (T lat, T lon)

  Check position validity.
- static vector< path > ListDirectories (const path &dir)

  List all directories contained in the given directory.
- static vector< path > ListFiles (path dir, size_t n=0)

  List all regular files contained in the given directory.
- static path MoveFileToDirectory (const path &file, const path &dir_dst, bool create_dir=false)

  Copy a file into a directory.

Protected Attributes

- recursive_mutex mutex_

  Recursive mutual exclusion lock for synchronizing operations.

Private Member Functions

- kmldom::DocumentPtr **GenerateKmlDocument** (bool set_default_kml_style=true, bool visibility=true) const
- string GenerateKmlString ()

  Generate the Google Earth KML string for this float.
- string **GenerateStatusString** () const
- void SaveObjectToDatabase () const

  Save this float object to the database.
- void set_argo_instrument_type (uint8_t argo_instrument_type)

  Set the enumerated Argo instrument type identifier.
- void set_fw_version (uint32_t fw_version)

  Set the instrument firmware version.
- void set_gain_pressure (uint16_t gain)

  Set the instrument pressure gain value, in dBar/LSB.
- void set_gain_pressure_surface (uint16_t gain)

  Set the instrument pressure gain value for surface data, in dBar/LSB.
- void set_gain_salinity (uint16_t gain)

  Set the instrument salinity gain value, in PSU/LSB.
- void set_gain_salinity_surface (uint16_t gain)

  Set the instrument salinity gain value for surface data, in PSU/LSB.
- void set_gain_temperature (uint16_t gain)

  Set the instrument temperature gain value, in degC/LSB.
- void set_gain_temperature_surface (uint16_t gain)

  Set the instrument temperature gain value for surface data, in degC/LSB.
- void **set_kml_document** (const kmldom::DocumentPtr &kml_document)
- void **set_kml_document_updated** (bool kml_document_updated=true)
- void **set_kml_file_path** (const path &kml_file_path)
- void **set_kml_file_updated** (bool kml_file_updated=true)
- void **set_latest_gps_position_datetime** (const ptime &latest_gps_position_datetime)
- void **set_latest_gps_position_dive_number** (int latest_gps_position_dive_number)
- void **set_latest_gps_position_latitude** (double latest_gps_position_latitude)
- void **set_latest_gps_position_longitude** (double latest_gps_position_longitude)
- void **set_latest_iridium_position_cep_radius** (int latest_iridium_position_cep_radius)
- void **set_latest_iridium_position_datetime** (const ptime &latest_iridium_position_datetime)
- void **set_latest_iridium_position_dive_number** (int latest_iridium_position_dive_number)
- void **set_latest_iridium_position_latitude** (double latest_iridium_position_latitude)

- void **set_latest_iridium_position_longitude** (double latest_iridium_position_longitude)
- void **set_latest_iridium_startup_position_datetime** (const ptime &latest_iridium_startup_position_datetime)
- void **set_latest_iridium_startup_sbd_momsn** (int latest_iridium_startup_sbd_momsn)
- void **set_latest_sbd_momsn** (int latest_sbd_momsn)
- void **set_latest_sbd_mtmsn** (int latest_sbd_mtmsn)
- void set_metadata (const shared_ptr< XFloatMetadata > &metadata)

  Assign an instrument metadata object to this X-Float object.
- void **set_netcdf_file_generation_is_enabled** (bool netcdf_file_generation_is_enabled)
- void set_offset_pressure (uint16_t offs)

  Set the instrument pressure offset value, in dBar.
- void set_offset_pressure_surface (uint16_t offs)

  Set the instrument pressure offset value for surface data, in dBar.
- void set_offset_salinity (uint16_t offs)

  Set the instrument salinity offset value, in PSU.
- void set_offset_salinity_surface (uint16_t offs)

  Set the instrument salinity offset value for surface data, in PSU.
- void set_offset_temperature (uint16_t offs)

  Set the instrument temperature offset value, in degC.
- void set_offset_temperature_surface (uint16_t offs)

  Set the instrument temperature offset value for surface data, in degC.
- void set_phy_file_generation_is_enabled (bool is_enabled)

  Set the PHY-file-generation-is-enabled flag.
- void set_start_of_mission_datetime (const ptime &datetime)

  Set the date/time of the start-of-mission for this instrument.
- void **set_status_file_path** (const path &path)
- void **set_status_file_updated** (bool status_file_updated=true)
- void set_xmsg_version_major (uint8_t xmsg_version_major)

  Set the X-Message major version number.
- void set_xmsg_version_minor (uint8_t xmsg_version_minor)

  Set the X-Message minor version number.
- bool WriteKmlFile ()

  Write the Google Earth KML file for this float.
- void WriteStatusFile ()

  Write the status file for this float.

Static Private Member Functions

- static void CleanupSqlConnection (Connection *con)

  Cleanup an SQL connection.
- static void CleanupSqlDriver (MySQL_Driver *driver)

  Function for memory cleanup of MySQL driver on destruct.
- static kmldom::DocumentPtr GenerateKmlMasterDocument ()

  Generate the Google Earth KML master document.
- static string GenerateKmlMasterString ()

  Generate the Google Earth KML master string.
- static void WriteKmlMasterFile ()

  Write the KML master file.

Private Attributes

- XArgoInstrumentType_t argo_instrument_type_

  The enumerated Argo instrument type identifier.
- flat_map< int, shared_ptr< XFloatDiveData > > dive2data_map_

  The dive-number-to-dive-data-object map.
- uint32_t fw_version_

  The reported instrument firmware version.
- uint16_t gain_pressure_

  The pressure sensitivity gain factor, in dBar/LSB.
- uint16_t gain_pressure_surface_

  The pressure sensitivity gain factor for surface data, in dBar/LSB.
- uint16_t gain_salinity_

  The salinity sensitivity scale factor, in PSU/LSB.

- uint16_t gain_salinity_surface_

  The salinity sensitivity scale factor for surface data, in PSU/LSB.
- uint16_t gain_temperature_

  The temperature sensitivity gain factor, in degC/LSB.
- uint16_t gain_temperature_surface_

  The temperature sensitivity gain factor for surface data, in degC/LSB.
- int id_dac_

  The unique Data Assembly Center (DAC) identifier.
- long id_imei_

  The unique International Mobile Equipment Identifier (IMEI).
- int id_instrument_

  The unique instrument identifier or instrument serial number.
- int id_wmo_

  The unique World Meteorological Organization (WMO) identifier.
- kmldom::DocumentPtr kml_document_

  The GE KML document.
- bool kml_document_updated_

  The KML document is updated flag.
- path kml_file_path_

  The KMLfile path.
- bool kml_file_updated_

  The KML file is updated flag.
- ptime latest_gps_position_datetime_

  The latest GPS position date/time.
- int latest_gps_position_dive_number_

  The latest GPS position dive number.
- double latest_gps_position_latitude_

  The latest GPS position latitude.
- double latest_gps_position_longitude_

  The latest GPS position longitude.
- int latest_iridium_position_cep_radius_

  The latest iridium position circular error probable radius.
- ptime latest_iridium_position_datetime_

  The latest iridium position date/time.
- int latest_iridium_position_dive_number_

  The latest iridium position dive number.
- double latest_iridium_position_latitude_

  The latest iridium position latitude.
- double latest_iridium_position_longitude_

  The latest iridium position longitude.
- int latest_iridium_sbd_momsn_

  The latest iridium position Mobile-Originated Message Number (MOMSN).
- int latest_iridium_sbd_mtmsn_

  The latest iridium position Mobile-Terminated Message Number (MTMSN).
- ptime latest_iridium_startup_position_datetime_

  The latest startup iridium position date/time.
- int latest_iridium_startup_sbd_momsn_

  The latest startup iridium position Mobile-Originated Message Number (MOMSN).
- shared_ptr< XFloatMetadata > metadata_

  The instrument metadata.
- bool netcdf_file_generation_is_enabled_

  The NetCDF-file-generation-is-enabled flag.
- uint16_t offset_pressure_

  The pressure offset, in dBar.
- uint16_t offset_pressure_surface_

  The pressure offset for surface data, in dBar.
- uint16_t offset_salinity_

  The salinity offset, in PSU.
- uint16_t offset_salinity_surface_

  The salinity offset for surface data, in PSU.
- uint16_t offset_temperature_

  The temperature offset, in degC.
- uint16_t offset_temperature_surface_

The temperature offset for surface data, in degC.

- bool phy_file_generation_is_enabled_

  The PHY-file-generation-enabled flag.
- ptime start_of_mission_datetime_

  The date/time of startup of the instrument.
- path status_file_path_

  The status file path.
- bool status_file_updated_

  The status-file-is-updated flag.
- uint8_t xmsg_version_major_

  The X-Message format major version number.
- uint8_t xmsg_version_minor_

  The X-Message format minor version number.

Static Private Attributes

- static string address_

  The SQL server qualified name or IP address string.
- static const double EARTH_RADIUS_KM_EQUATORIAL = 6378.1

  The equatorial radius of the earth, in kilometers.
- static const double EARTH_RADIUS_KM_POLAR = 6356.1

  The polar radius of the earth, in kilometers.
- static flat_map< int, std::shared_ptr< XFloat > > id2float_map_

  The instrument-identifier-to-instrument-object map.
- static recursive_mutex id2float_map_mutex_

  The mutex lock for synchronizing access to the instrument-identifier-to-instrument-object map.
- static vector< unsigned int > instrument_ids_to_disable_phy_generation_

  A list of instrument identifiers for which to disable PHY file generation.
- static vector< unsigned int > instrument_ids_to_enable_phy_generation_

  A list of instrument identifiers for which to enable PHY file generation.
- static const Color32 kml_default_path_colors_ [KML_NUM_DFLT_PATH_COLORS]

  The colour palette for instrument paths and placemarks, indexed by style index.
- static const string kml_default_path_style_ids_ [KML_NUM_DFLT_PATH_COLORS]

  The KML style IDs of the path (line) styles, one per palette colour.
- static const string kml_default_placemark_style_ids_ [KML_NUM_DFLT_PATH_COLORS]

  The KML style IDs of the large placemark styles, one per palette colour.
- static path kml_directory_path_

  The KML file directory.
- static thread_specific_ptr< KmlFactory > kml_factory_

  The libkml factory used to create KML elements, held per thread.
- static bool kml_file_generation_is_enabled_

  The global KML-file-generation-is-enabled flag.
- static const string kml_small_placemark_style_ids_ [KML_NUM_DFLT_PATH_COLORS]

  The KML style IDs of the small placemark styles, one per palette colour.
- static const string kml_triangle_placemark_style_ids_ [KML_NUM_DFLT_PATH_COLORS]

  The KML style IDs of the triangle placemark styles, one per palette colour.
- static string password_

  The SQL server account password string.
- static unsigned int port_

  The SQL server port number.
- static string protocol_

  The protocol to use in the SQL server connection e.g.
- static string schema_

  The SQL server schema name.
- static thread_specific_ptr< Connection > sql_connection_

  Tthe thread-specific pointer to MySQL connection object.
- static thread_specific_ptr< MySQL_Driver > sql_driver_

  The thread-specific pointer to MySQL driver object.
- static const util::mysql::SqlTableDescription< XFloat > sql_table_description_

  The SQL table description object.
- static path status_directory_path_

  The global status file directory path.
- static bool status_file_generation_is_enabled_

  The global status-file-generation-is-enabled flag.
- static string user_

  The SQL server account username string.

Friends

- const util::config::LocalConfiguration & Initialize (int argc, char **argv)

  Initialize the application.
- options_description util::config::LocalConfiguration::GenerateCommandLineOptionsDescription ()

  Friend function for populating some of the static members of this class from the command-line options.
- options_description util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription ()

  Friend function for populating some of the static members of this class from the configuration file options.
- class **XFloatDiveData**
- class **XFloatMailProcessingService**
- class **XFloatPostProcessingService**
- class **XFloatTelemetryDecoderService**

10.86.1 Detailed Description

The instrument or float class.

This class represents a single instrument or float. Each instance of this contains information about an instrument including unique identifiers, sensor gains and offsets, metadata, and the instrument type. It also contains a dive-or-cycle-number-to-dive-data-object, dive2data_map_, map that contains all of the incomplete dive data objects that belong to the instrument, uniquely indexed by dive or cycle number. As the application receives the telemetry data from the instrument, the data is decoded and assigned to the corresponding dive data object. Once it is determined that the data is complete for a single dive, the intermediate and PHY listing files are written to the local filesystem, the dive data object is persisted to the SQL database, and then the shared reference to the dive data object is released, thereby purging the dive data from the program's memory.

At run-time, at most one float object will exist for each instrument. Each float object is accessed and/or manipulated by the worker threads through a shared reference. A shared reference to all float objects is contained in the instrument-identifier-to-instrument-object map, id2float_map_, which is indexed by the unique instrument identifier. During application initialization, the map is populated as a result of querying all the known instruments from the SQL database. Each time a float object is created, it is added to the map. During application termination, the daemon thread iterates over the contents of the map and persists each of the float objects to the SQL database.

Note

This object is lockable. It contains a recursive mutual exclusion (mutex) lock in order to synchronize access to and modification of objects of this type by various worker threads. Once the object is locked, any other thread that attempts to access or modify the object will block until the initial lock is released. Since the mutex is recursive, the object may be locked multiple times by the same thread without causing the thread to block indefinitely.

This class is mapped to an SQL database table by the sql_table_description_ object. This allows some or all of the field of this class to be persisted to the SQL database. Likewise, an object of this type may be constructed from the results of an SQL query to the corresponding table.

Todo Devise a method to actively check for (1) changes in the metadata files and (2) newly created metadata files. Load the metadata from the new or modified files and assign them to the corresponding float object. If necessary, retroactively process the mail messages for these instruments. Persist updates to the SQL database as necessary.

10.86.2 Constructor & Destructor Documentation

10.86.2.1 XFloat() [1/2]

```
argo::iridium::xfloat::XFloat::XFloat (
            int whoi,
            long long imei = XFLT_ID_UNKNOWN,
            int wmo = XFLT_ID_UNKNOWN,
            int dac = XFLT_ID_UNKNOWN)
```

The parameterized constructor.

Parameters

in	*imei*	The unique International Mobile Equipment Identifier (IMEI).
in	*whoi*	The unique WHOI identifier.
in	*wmo*	The unique World Meteorological Organization (WMO) identifier.
in	*dac*	The unique Data Assembly Center (DAC) identifier.

10.86.2.2 XFloat() [2/2]

```
argo::iridium::xfloat::XFloat::XFloat ()
```

The default constructor.

10.86.2.3 ~XFloat()

```
argo::iridium::xfloat::XFloat::~XFloat ()
```

The destructor.

10.86.3 Member Function Documentation

10.86.3.1 argo_instrument_type()

```
uint8_t argo::iridium::xfloat::XFloat::argo_instrument_type () const  [inline]
```

Get the enumerated Argo instrument type identifier.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The enumerated Argo instrument type identifier.

References argo_instrument_type_, and util::thread::RecursiveLockable::mutex_.

Referenced by set_argo_instrument_type().

Here is the caller graph for this function:

10.86.3.2 CalculateEllipsoidHeight()

```
template<typename T >
static double util::position::PositionUtils::CalculateEllipsoidHeight (
            T lat)  [inline], [static], [inherited]
```

Get the approximate surface altitude or height of earth for a given latitude, in kilometers.

Warning

This method provides approximate surface height estimates only! This approximation is calculated from a reference ellipsoid – a mathematically-defined surface that approximates the geoid, the truer figure of the Earth.

Parameters

in	*lat*	The geographic latitude of interest, in decimal degrees.

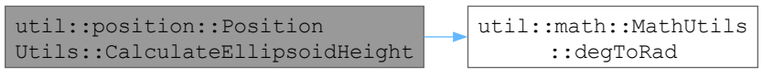
Returns

The approximate surface altitude or height of earth for the given latitude, in kilometers.

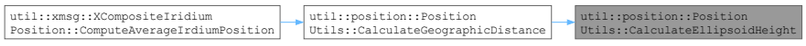
References util::math::MathUtils::degToRad(), util::position::PositionUtils::EARTH_RADIUS_KM_EQUATORIAL, and util::position::PositionUtils::EARTH_RADIUS_KM_POLAR.

Referenced by util::position::PositionUtils::CalculateGeographicDistance().

Here is the call graph for this function:



Here is the caller graph for this function:



10.86.3.3 CalculateGeographicDistance()

```
template<typename T >
static double util::position::PositionUtils::CalculateGeographicDistance (
            T lat1,
            T lon1,
            T lat0,
            T lon0)  [inline], [static], [inherited]
```

Get the geographical distance between two position, in kilometers, from the positions' locations, in decimal degrees.

Warning

This method provides approximate geographic estimates only! This approximation is calculated from a reference ellipsoid – a mathematically-defined surface that approximates the geoid, the truer figure of the Earth.

Note

This method uses the 'haversine' formula to calculate the great-circle distance between two points – that is, the shortest distance over the earth's surface – giving an 'as-the-crow-flies' distance between the points (ignoring any hills they fly over, of course!).

Parameters

in	*lat1*	The latitude of the geographic final position, in decimal degrees North.
in	*lon1*	The longitude of the geographic final position, in decimal degrees East.
in	*lat0*	The latitude of the geographic initial position, in decimal degrees North.
in	*lon0*	The longitude of the geographic initial position, in decimal degrees East.

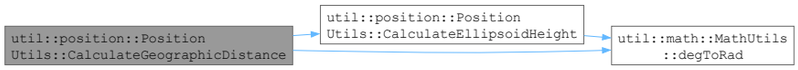
Returns

The geographical distance between the two given positions, in kilometers.

References util::position::PositionUtils::CalculateEllipsoidHeight(), and util::math::MathUtils::degToRad().

Referenced by util::xmsg::XCompositeIridiumPosition::ComputeAverageIrdiumPosition().

Here is the call graph for this function:



Here is the caller graph for this function:

10.86.3.4 CleanupSqlConnection()

```
static void util::mysql::MysqlUtils::CleanupSqlConnection (
            Connection * con)  [inline], [static], [private], [inherited]
```

Cleanup an SQL connection.

This function for memory cleanup of the SQL connection on destruction. The thread-specific SQL connection object will be deleted if it exists.

Parameters

in	*con*	The SQL connection to cleanup.

Exceptions

log::MessageException	if an error occurs while cleaning up the SQL connection object.

References CATCH_BLOCK.

10.86.3.5 CleanupSqlDriver()

```
static void util::mysql::MysqlUtils::CleanupSqlDriver (
            MySQL_Driver * driver)  [inline], [static], [private], [inherited]
```

Function for memory cleanup of MySQL driver on destruct.

Signals the SQL driver to end operation, if it exists.

Parameters

in	*driver*	Driver to cleanup.

Exceptions

log::MessageException	if an error occurs while cleaning up the SQL driver object.

References CATCH_BLOCK.

10.86.3.6 CloseKmlFactory()

```
static void util::kml::KmlUtil::CloseKmlFactory ()  [inline], [static], [inherited]
```

Release this thread's KML factory pointer.

Warning

This deletes libkml's shared factory (see kml_factory_). It is not currently called.

Exceptions

util::log::MessageException	if an error occurs.

References CATCH_BLOCK, and util::kml::KmlUtil::kml_factory_.

10.86.3.7 CloseSqlConnection()

```
static void util::mysql::MysqlUtils::CloseSqlConnection ()  [inline], [static], [inherited]
```

Close the thread-specific MySQL connection if it is currently open.

If the thread local MySQL connection is not currently open, then this function will have no effect.

Note

The MySQL connection should be closed when idle to prevent the connection from timing out and throwing exceptions.

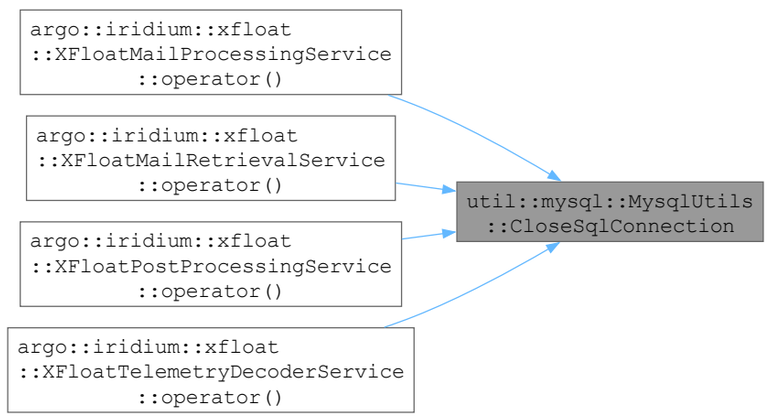
Exceptions

log::MessageException	if an error occurs while attempting to close the MySQL connection.

References CATCH_BLOCK, and util::mysql::MysqlUtils::sql_connection_.

Referenced by argo::iridium::xfloat::XFloatMailProcessingService::operator()(), argo::iridium::xfloat::XFloatMailRetrievalService::operator(argo::iridium::xfloat::XFloatPostProcessingService::operator()(), and argo::iridium::xfloat::XFloatTelemetryDecoderService::operator()().

Here is the caller graph for this function:



10.86.3.8 degToRad()

```
template<typename T >
static double util::math::MathUtils::degToRad (
             T deg)  [inline], [static], [inherited]
```

Convert angular units from from degrees to decimal radians.

Parameters

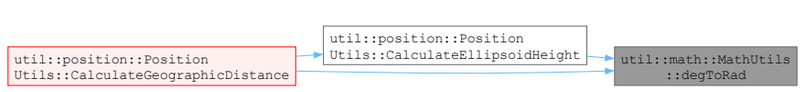
in	*deg*	Angle in integer or decimal degrees to convert to radians.

Returns

Angle in decimal radians.

Referenced by util::position::PositionUtils::CalculateEllipsoidHeight(), and util::position::PositionUtils::CalculateGeographicDistance().

Here is the caller graph for this function:



10.86.3.9 ExecuteQuery()

```
static shared_ptr< ResultSet > util::mysql::MysqlUtils::ExecuteQuery (
            Connection * con,
            const string & query)  [inline], [static], [inherited]
```

Execute an SQL query and return a shared pointer to the ResultSet object.

Parameters

in	*con*	A pointer to the SQL connection object. This must not be NULL.
in	*query*	The SQL query to execute. This must be a well-formatted SQL query string.

Exceptions

log::MessageException	if 'con' is null, 'query' is a malfored SQL string, or any other error occurs.

Returns

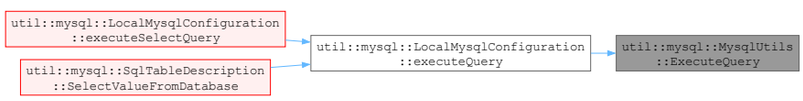
A shared pointer to the query ResultSet object.

References CATCH_BLOCK, and util::mysql::MysqlUtils::GeneratePreparedStatement().

Referenced by util::mysql::LocalMysqlConfiguration::executeQuery().

Here is the call graph for this function:

Here is the caller graph for this function:



10.86.3.10 executeQuery()

```
shared_ptr< sql::ResultSet > util::mysql::LocalMysqlConfiguration::executeQuery (
            const string & query)  [static], [inherited]
```

Execute an SQL query and return a shared pointer to the ResultSet object.

Parameters

in	*query*	The SQL query to execute. This must be a well-formatted SQL query string.

Exceptions

log::MessageException	'query' is a malfored SQL string or any other error occurs while executing the query.

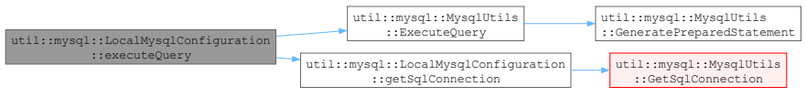
Returns

A shared pointer to the query ResultSet object.

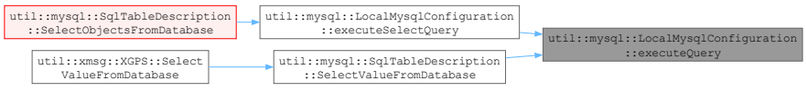
References util::mysql::MysqlUtils::ExecuteQuery(), and util::mysql::LocalMysqlConfiguration::getSqlConnection().

Referenced by util::mysql::LocalMysqlConfiguration::executeSelectQuery(), and util::mysql::SqlTableDescription< T >::SelectValueFromD

Here is the call graph for this function:



Here is the caller graph for this function:



10.86.3.11 executeSelectQuery()

```
shared_ptr< sql::ResultSet > util::mysql::LocalMysqlConfiguration::executeSelectQuery (
            const string & what,
            const string & from,
            const string & where = "",
            const string & groupBy = "",
            const string & orderBy = "",
            unsigned int limit = 0,
            unsigned int offset = 0)  [static], [inherited]
```

Execute an SQL select query and return a shared pointer to the ResultSet object.

Parameters

in	*what*	The SQL select statement select-expression clause.
in	*from*	The SQL select statement table–references clause.
in	*where*	The SQL select statement where-condition clause.
in	*groupBy*	The SQL select statement group-by clause.
in	*orderBy*	The SQL select statement order-by clause.
in	*limit*	The SQL select statement result set row limit value.
in	*offset*	The SQL select statement result set row offset.

Exceptions

log::MessageException	if an error occurs while executing the query.

Returns

A shared pointer to the ResultSet object.

References CATCH_BLOCK, util::mysql::LocalMysqlConfiguration::executeQuery(), and util::mysql::MysqlUtils::GenerateSqlSelectStr().

Referenced by util::mysql::SqlTableDescription< T >::SelectObjectsFromDatabase().

Here is the call graph for this function:

Here is the caller graph for this function:

10.86.3.12 ExecuteUpdate()

```
static int util::mysql::MysqlUtils::ExecuteUpdate (
            Connection * con,
            const string & update)  [inline], [static], [inherited]
```

Execute an SQL update and return the status code.

Parameters

in	*con*	A pointer to the SQL connection object. This must not be NULL.
in	*update*	The SQL update to execute. This must be a well-formatted SQL update string.

Exceptions

log::MessageException	if 'con' is null, 'update' is a malformed SQL string, or any other error occurs.

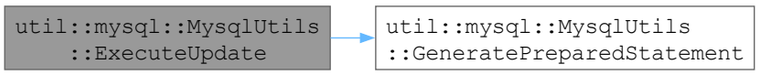
Returns

The update status code.

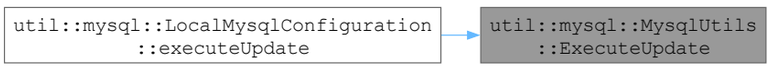
References CATCH_BLOCK, and util::mysql::MysqlUtils::GeneratePreparedStatement().

Referenced by util::mysql::LocalMysqlConfiguration::executeUpdate().

Here is the call graph for this function:



Here is the caller graph for this function:



10.86.3.13 executeUpdate()

```
int util::mysql::LocalMysqlConfiguration::executeUpdate (
            const string & update)  [static], [inherited]
```

Execute an SQL update and return the status code.

Parameters

in	*update*	The SQL update to execute. This must be a well-formatted SQL update string.

Exceptions

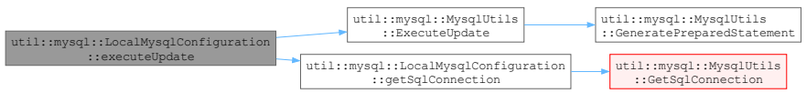
log::MessageException	if 'update' is a malformed SQL string, or any other error occurs while executing the update.

Returns

The update status code.

References util::mysql::MysqlUtils::ExecuteUpdate(), and util::mysql::LocalMysqlConfiguration::getSqlConnection().

Here is the call graph for this function:



10.86.3.14 fw_version()

```
uint32_t argo::iridium::xfloat::XFloat::fw_version () const  [inline]
```

Get the firmware version.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

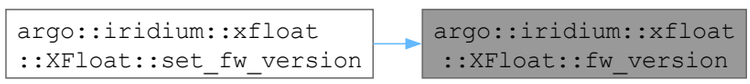
Returns

The firmware version.

References fw_version_, and util::thread::RecursiveLockable::mutex_.

Referenced by set_fw_version().

Here is the caller graph for this function:



10.86.3.15 gain_pressure()

```
uint16_t argo::iridium::xfloat::XFloat::gain_pressure () const  [inline]
```

Get the pressure gain value, in dBar/LSB.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The pressure gain value, in dBar/LSB.

References gain_pressure_, and util::thread::RecursiveLockable::mutex_.

10.86.3.16 gain_pressure_surface()

```
uint16_t argo::iridium::xfloat::XFloat::gain_pressure_surface () const  [inline]
```

Get the pressure gain value for surface data, in dBar/LSB.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The pressure gain value, in dBar/LSB.

References gain_pressure_surface_, and util::thread::RecursiveLockable::mutex_.

10.86.3.17 gain_salinity()

```
uint16_t argo::iridium::xfloat::XFloat::gain_salinity () const  [inline]
```

Get the salinity gain value, in PSU/LSB.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The salinity gain value, in PSU/LSB.

References gain_salinity_, and util::thread::RecursiveLockable::mutex_.

10.86.3.18 gain_salinity_surface()

```
uint16_t argo::iridium::xfloat::XFloat::gain_salinity_surface () const  [inline]
```

Get the salinity gain value for surface data, in PSU/LSB.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The salinity gain value, in PSU/LSB.

References gain_salinity_surface_, and util::thread::RecursiveLockable::mutex_.

10.86.3.19 gain_temperature()

```
uint16_t argo::iridium::xfloat::XFloat::gain_temperature () const  [inline]
```

Get the temperature gain value, in degC/LSB.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The temperature gain value, in degC/LSB.

References gain_temperature_, and util::thread::RecursiveLockable::mutex_.

10.86.3.20 gain_temperature_surface()

```
uint16_t argo::iridium::xfloat::XFloat::gain_temperature_surface () const  [inline]
```

Get the temperature gain value for surface data, in degC/LSB.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The temperature gain value, in degC/LSB.

References gain_temperature_surface_, and util::thread::RecursiveLockable::mutex_.

10.86.3.21 GenerateCurrentUtcTimeIso8601String()

```
static string util::date::DateUtils::GenerateCurrentUtcTimeIso8601String ()  [inline], [static],
[inherited]
```

Generate an ISO8601 format date/time string representation of the current time, in UTC.

Note

All date/time objects use the UTC time-zone.

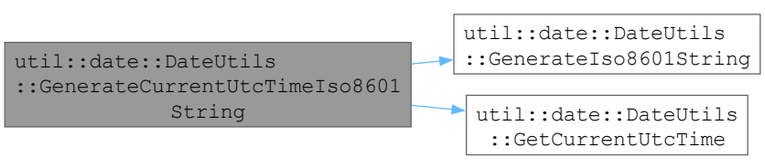
Returns

An ISO8601 format date/time string representation of the current time.

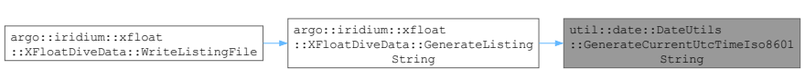
References util::date::DateUtils::GenerateIso8601String(), and util::date::DateUtils::GetCurrentUtcTime().

Referenced by argo::iridium::xfloat::XFloatDiveData::GenerateListingString().

Here is the call graph for this function:



Here is the caller graph for this function:



10.86.3.22 GenerateIso6709String()

```
template<typename T >
static std::string util::position::PositionUtils::GenerateIso6709String (
            T lat,
            T lon)  [inline], [static], [inherited]
```

Generate an IS06709 string representation of the position given by (`lat`, `lon`).

Parameters

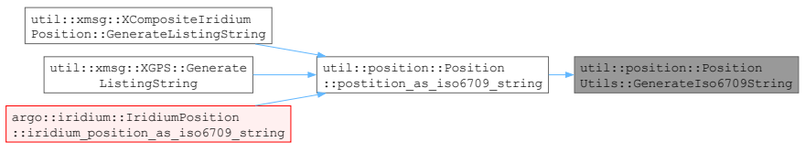
in	*lat*	The latitude of the position, in degrees North.
in	*lon*	The longitude of the position, in degrees East.

Returns

An IS06709 string representation of the position given by (`lat`, `lon`).

Referenced by util::position::Position::postition_as_iso6709_string().

Here is the caller graph for this function:



10.86.3.23 GenerateIso8601String()

```
static string util::date::DateUtils::GenerateIso8601String (
            const ptime & p,
            bool extended = true)  [inline], [static], [inherited]
```

Generate an ISO8601 format date/time string representation of a POSIX time object.

Note

All date/time objects use the UTC time-zone.

Parameters

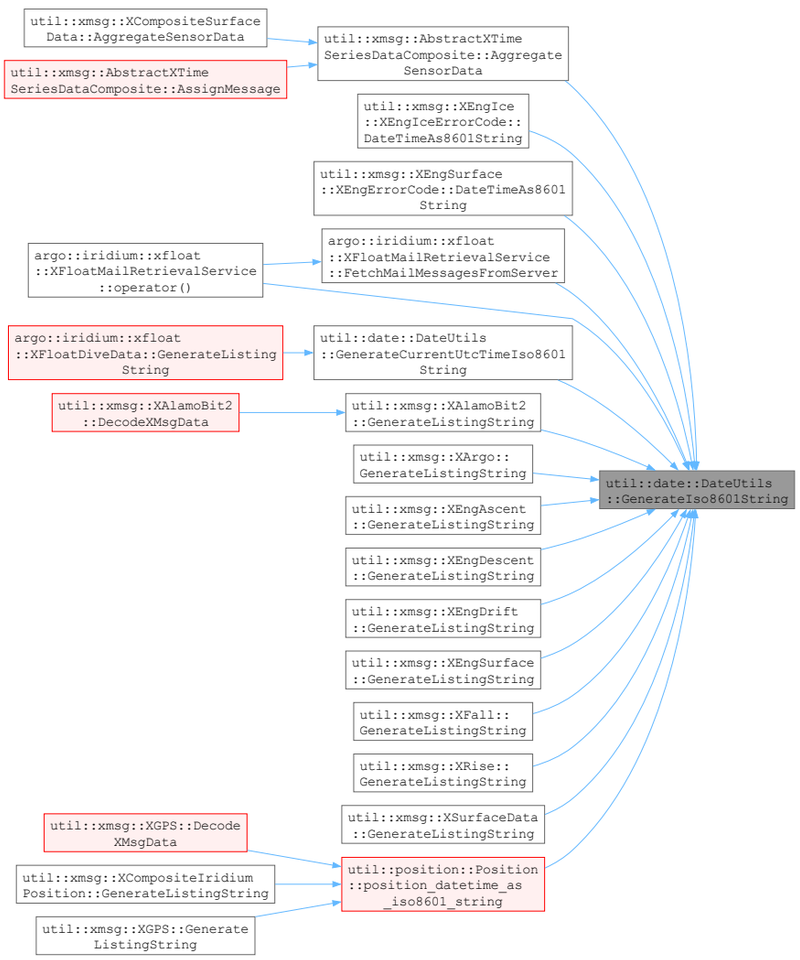
in	*p*	POSIX time from which to generate string.
in	*extended*	If TRUE, use the ISO 8601 extended format (YYYY-MM-DDThh:mm:ssZ); otherwise, use the basic format (YYYYMMDDThhmmss).

Returns

An ISO8601 format date/time string representation of the POSIX time object. If p is equal to boost::posix_↩ time::not_a_date_time, then the string "0000-00-00T00:00:00Z" (extended) or "00000000000000" (basic) will be returned.

Referenced by util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::AggregateSensorData(), util::xmsg::XEngIce::XEngIce
util::xmsg::XEngSurface::XEngErrorCode::DateTimeAs8601String(), argo::iridium::xfloat::XFloatMailRetrievalService::FetchMailMessages
util::date::DateUtils::GenerateCurrentUtcTimeIso8601String(), util::xmsg::XAlamoBit2::GenerateListingString(), util::xmsg::XArgo::Genera
util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListing
util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XRise::GenerateListingString(),
util::xmsg::XSurfaceData::GenerateListingString(), argo::iridium::xfloat::XFloatMailRetrievalService::operator()(), and
util::position::Position::position_datetime_as_iso8601_string().

Here is the caller graph for this function:



10.86.3.24 GenerateJulianDay()

```
static double util::date::DateUtils::GenerateJulianDay (
            const ptime & dt,
            const ptime & ref_dt)  [inline], [static], [inherited]
```

Convert a date/time to fractional days since a reference date/time.

Parameters

in	*dt*	The date/time to convert.
in	*ref↩ _dt*	The reference date/time (day 0).

Exceptions

util::log::MessageException	if either date/time is not valid.

Returns

The number of days (with millisecond resolution) from `ref_dt` to `dt`.

References CATCH_BLOCK, and EXC_THROW.

10.86.3.25 GenerateKmlMasterDocument()

```
kmldom::DocumentPtr argo::iridium::xfloat::XFloat::GenerateKmlMasterDocument ()  [static], [private]
```

Generate the Google Earth KML master document.

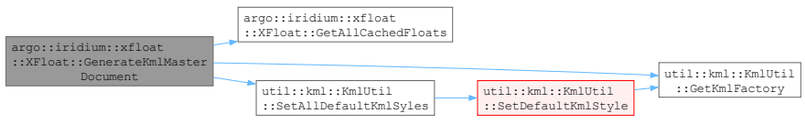
Returns

The Google Earth KML master document.

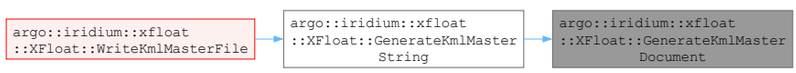
References CATCH_BLOCK, GetAllCachedFloats(), util::kml::KmlUtil::GetKmlFactory(), and util::kml::KmlUtil::SetAllDefaultKmlSyles().

Referenced by GenerateKmlMasterString().

Here is the call graph for this function:



Here is the caller graph for this function:



10.86.3.26 GenerateKmlMasterString()

```
string argo::iridium::xfloat::XFloat::GenerateKmlMasterString ()  [static], [private]
```

Generate the Google Earth KML master string.

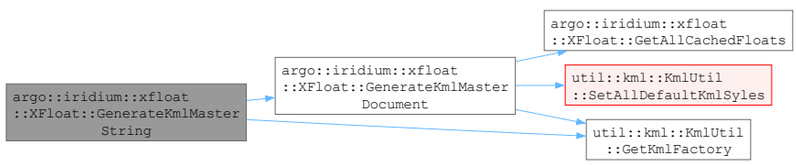
Returns

The Google Earth KML master string.

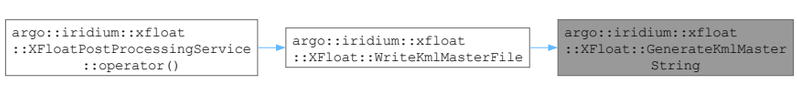
References CATCH_BLOCK, GenerateKmlMasterDocument(), and util::kml::KmlUtil::GetKmlFactory().

Referenced by WriteKmlMasterFile().

Here is the call graph for this function:



Here is the caller graph for this function:



10.86.3.27 GenerateKmlString()

```
string argo::iridium::xfloat::XFloat::GenerateKmlString ()  [private]
```

Generate the Google Earth KML string for this float.

Returns

The Google Earth KML string for this float.

References CATCH_BLOCK.

10.86.3.28 GeneratePreparedStatement()

```
static shared_ptr< PreparedStatement > util::mysql::MysqlUtils::GeneratePreparedStatement (
            Connection * con,
            const string & statement)  [inline], [static], [inherited]
```

Generate a prepared SQL statement object for the given SQL statement.

Parameters

in	*con*	A pointer to the SQL connection object. This must not be NULL.
in	*statement*	The SQL statement to prepare. This must be a well-formed SQL string.

Exceptions

log::MessageException	if 'con' is NULL, 'statement' is not a valid SQL string, or if an error occurs.

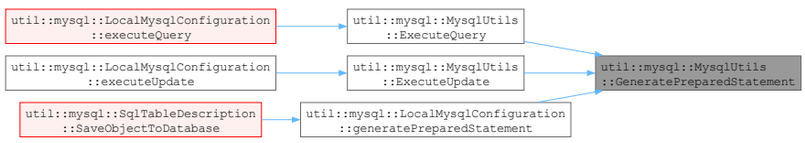
Returns

A prepared SQL statement object generated from the given SQL statement.

References CATCH_BLOCK, and EXC_THROW.

Referenced by util::mysql::MysqlUtils::ExecuteQuery(), util::mysql::MysqlUtils::ExecuteUpdate(), and util::mysql::LocalMysqlConfiguration:

Here is the caller graph for this function:



10.86.3.29 generatePreparedStatement()

```
std::shared_ptr< sql::PreparedStatement > util::mysql::LocalMysqlConfiguration::generatePrepared↩
Statement (
            const string & statement)  [static], [inherited]
```

Generate a prepared SQL statement object for the given SQL statement.

Parameters

in	*statement*	The SQL statement to prepare. This must be a well-formed SQL string.

Exceptions

log::MessageException	if 'statement' is not a valid SQL string or if an error occurs while generating the prepared statement object..

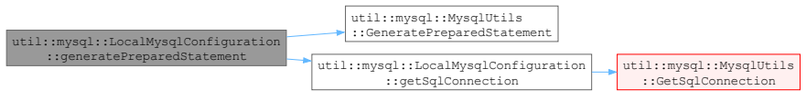
Returns

A prepared SQL statement object generated from the given SQL statement.

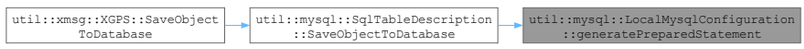
References util::mysql::MysqlUtils::GeneratePreparedStatement(), and util::mysql::LocalMysqlConfiguration::getSqlConnection().

Referenced by util::mysql::SqlTableDescription< T >::SaveObjectToDatabase().

Here is the call graph for this function:



Here is the caller graph for this function:



10.86.3.30 GenerateSqlDeleteStr()

```
static string util::mysql::MysqlUtils::GenerateSqlDeleteStr (
            const string & from,
            const string & where = "")  [inline], [static], [inherited]
```

Generate an SQL DELETE statement.

Warning

Without a `where` clause, the statement deletes every row in the table.

Parameters

`in`	*from*	The name of the table to delete from.
`in`	*where*	The SQL "where-condition" clause (empty = all rows).

Returns

The SQL statement.

10.86.3.31 GenerateSqlSelectStr()

```
static string util::mysql::MysqlUtils::GenerateSqlSelectStr (
            const string & what,
            const string & from,
            const string & where = "",
            const string & groupBy = "",
            const string & orderBy = "",
            unsigned int limit = 0,
            unsigned int offset = 0)  [inline], [static], [inherited]
```

Generate an SQL select statement.

Parameters

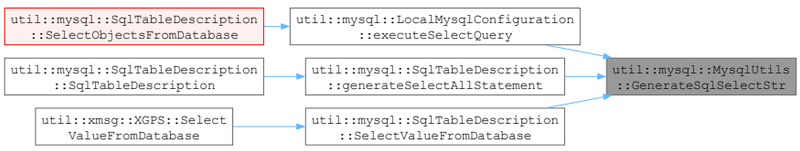
in	*what*	The SQL select statement select-expression clause.
in	*from*	The SQL select statement table-references clause.
in	*where*	The SQL select statement where-condition clause.
in	*groupBy*	The SQL select statement group-by clause.
in	*orderBy*	The SQL select statement order-by clause.
in	*limit*	The SQL select statement result set row limit value.
in	*offset*	The SQL select statement result set row offset.

Returns

The SQL select statement.

Referenced by util::mysql::LocalMysqlConfiguration::executeSelectQuery(), util::mysql::SqlTableDescription< T >::generateSelectAllState and util::mysql::SqlTableDescription< T >::SelectValueFromDatabase().

Here is the caller graph for this function:



10.86.3.32 GetAllCachedFloats()

```
vector< shared_ptr< XFloat > > argo::iridium::xfloat::XFloat::GetAllCachedFloats ()  [static]
```

Get all float objects currently stored in the cache.

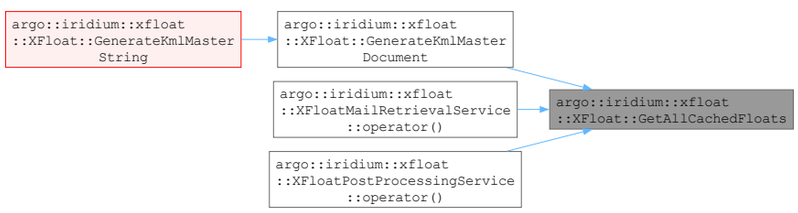
Returns

A vector of shared pointers to the cached float objects.

References CATCH_BLOCK, id2float_map_, and id2float_map_mutex_.

Referenced by GenerateKmlMasterDocument(), argo::iridium::xfloat::XFloatMailRetrievalService::operator()(), and argo::iridium::xfloat::XFloatPostProcessingService::operator()().

Here is the caller graph for this function:



10.86.3.33 GetAllDiveData()

```
vector< shared_ptr< XFloatDiveData > > argo::iridium::xfloat::XFloat::GetAllDiveData ()
```

Get all dive data objects for this float.

Returns

A vector of shared pointers to all dive data objects for this float.

References CATCH_BLOCK.

10.86.3.34 GetCurrentUtcTime()

```
static ptime util::date::DateUtils::GetCurrentUtcTime ()  [inline], [static], [inherited]
```

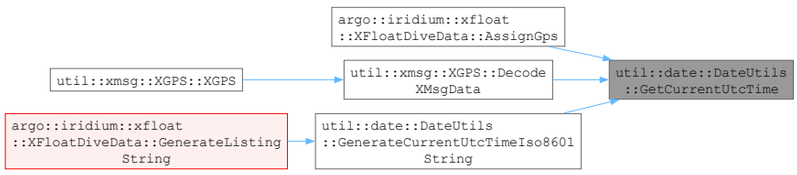
Generate a POSIX time object representing the current date and time (rounded to the nearest second).

Returns

A POSIX time object representing the current date and time (rounded to the nearest second).

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignGps(), util::xmsg::XGPS::DecodeXMsgData(), and util::date::DateUtils::GenerateCurrentUtcTimeIso8601String().

Here is the caller graph for this function:



10.86.3.35 GetDefaltKmlPathColor()

```
static const Color32 & util::kml::KmlUtil::GetDefaltKmlPathColor (
            int id)  [inline], [static], [inherited]
```

Get the palette colour for an index (an instrument ID, for example), modulo the palette size.

Parameters

in	*id*	The index.

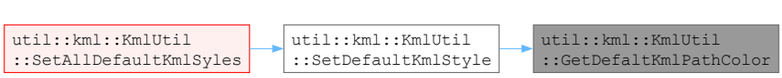
Returns

The colour.

References util::kml::KmlUtil::kml_default_path_colors_.

Referenced by util::kml::KmlUtil::SetDefaultKmlStyle().

Here is the caller graph for this function:



10.86.3.36 GetDefaltKmlPathStyleId()

```
static const string & util::kml::KmlUtil::GetDefaltKmlPathStyleId (
            int id)  [inline], [static], [inherited]
```

Get the path (line) style ID for an index, modulo the palette size.

Parameters

in	*id*	The index.

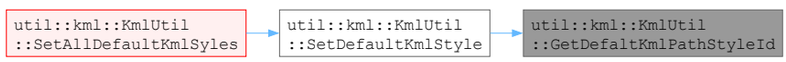
Returns

The KML style ID.

References util::kml::KmlUtil::kml_default_path_style_ids_.

Referenced by util::kml::KmlUtil::SetDefaultKmlStyle().

Here is the caller graph for this function:



10.86.3.37 GetDefaltKmlPlacemarkStyleId()

```
static const string & util::kml::KmlUtil::GetDefaltKmlPlacemarkStyleId (
            int id)  [inline], [static], [inherited]
```

Get the large placemark style ID for an index, modulo the palette size.

Parameters

in	*id*	The index.

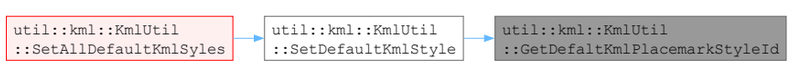
Returns

The KML style ID.

References util::kml::KmlUtil::kml_default_placemark_style_ids_.

Referenced by util::kml::KmlUtil::SetDefaultKmlStyle().

Here is the caller graph for this function:



10.86.3.38 GetDirectoryPath()

```
static const path & util::file::FileSystemUtils::GetDirectoryPath (
            const path & dir,
            bool create = false)  [inline], [static], [inherited]
```

Get a directory path.

Exceptions

MessageException	if create == FALSE and the directory specified by path does not exist.

Parameters

in	*dir*	Directory path.
in	*create*	If TRUE, the directory specified by path will be created if it does not already exists. If FALSE, an exception will be thrown if the directory does not exist.

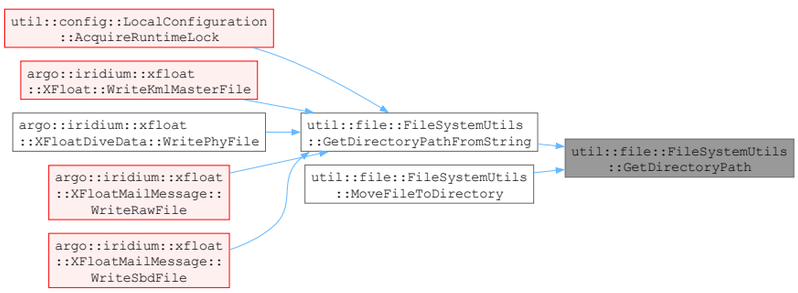
Returns

dir

References CATCH_BLOCK, EXC_THROW, and ldbg.

Referenced by util::file::FileSystemUtils::GetDirectoryPathFromString(), and util::file::FileSystemUtils::MoveFileToDirectory().

Here is the caller graph for this function:



10.86.3.39 GetDirectoryPathFromString()

```
static path util::file::FileSystemUtils::GetDirectoryPathFromString (
            const string & dir,
            bool create = false)  [inline], [static], [inherited]
```

Generate a boost::filesystem::path object from the directory path string.

Exceptions

MessageException	if create == FALSE and the directory specified by path does not exist.

Parameters

in	*dir*	Directory path string.
in	*create*	If TRUE, the directory specified by path will be created if it does not already exists. If FALSE, an exception will be thrown if the directory does not exist.

Returns

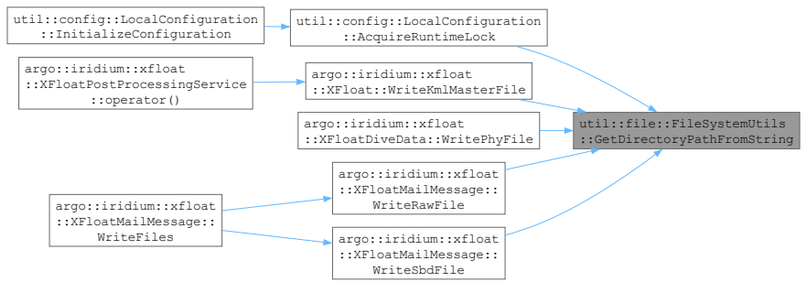
A boost::filesystem::path object from the directory path string.

References CATCH_BLOCK, and util::file::FileSystemUtils::GetDirectoryPath().

Referenced by util::config::LocalConfiguration::AcquireRuntimeLock(), WriteKmlMasterFile(), argo::iridium::xfloat::XFloatDiveData::WriteP argo::iridium::xfloat::XFloatMailMessage::WriteRawFile(), and argo::iridium::xfloat::XFloatMailMessage::WriteSbdFile().

Here is the call graph for this function:

Here is the caller graph for this function:



10.86.3.40 GetDiveData()

```
shared_ptr< XFloatDiveData > argo::iridium::xfloat::XFloat::GetDiveData (
            int dive,
            bool local_only = false)
```

Get the dive data object from the dive or cycle number.

This function returns a shared reference to the dive data object for this instrument identified by the unique dive or cycle number. First, the function checks if the dive data object exists in the dive-or-cycle-number-to-dive-data-object map, dive2data_map_. If it does, then a shared reference to the object is returned.

Otherwise, if the dive data object does not exist in the map, the function attempts to query the SQL database, and populate a dive data object from the result of the SQL query. If successful, then the newly populated object is added to the map and a shared reference to the object is returned.

Finally, if the dive data object also does not exist in the SQL database, then a new dive data object will be constructed. The newly constructed dive data object is added to the dive-or-cycle-number-to-dive-data-object map, dive2data_map_, and then persisted to the SQL database. A shared reference to the new object is returned.

Exceptions

util::log::MessageException	if an error occurs while attempting to retrieve or construct the dive data object.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Parameters

in	*dive*	The cycle or dive number of interest.
in	*local_only*	If TRUE, then the dive data object will only be loaded from the local cache. Otherwise, the dive data object will be fetched from the SQL database if necessary.

Returns

The dive data object for a given dive.

References CATCH_BLOCK, dbg1, and ldbg.

10.86.3.41 GetFloat()

```
shared_ptr< XFloat > argo::iridium::xfloat::XFloat::GetFloat (
            int instrument_id)  [static]
```

Get a float object from the instrument identifier.

This function returns a shared reference to the float object identified by the unique instrument identifier. First, the function checks to see if the instrument-identifier-to-instrument-object map, id2float_map_, is empty. It the map is empty, then the query the known float object from the SQL database and used the result of the query to populate the map.

Next, the function checks to see if the map contains the float object with the given instrument identifier. If it does not, then a new float object is created and added to the map. Finally, the shared reference to float object is returned.

Exceptions

util::log::MessageException	if an error occurs while attempting to retrieve the float object with the give instrument identifier.

Note

This function places a lock guard on the instrument-identifier-to-instrument-object map mutual exclusion (mutex) lock, id2float_map_mutex_, thereby locking the instrument-identifier-to-instrument-object map, id2float_map_, and preventing access or modification of the map by any other thread. Any thread that does attempt to access or modify the map will block until the lock is released.

Parameters

in	*instrument↩ _id*	The unique instrument identifier.

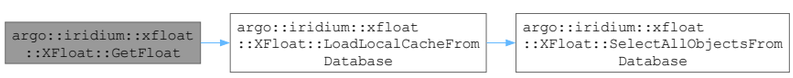
Returns

A shared reference to the float object with the given instrument identifier, `instrument_id`.

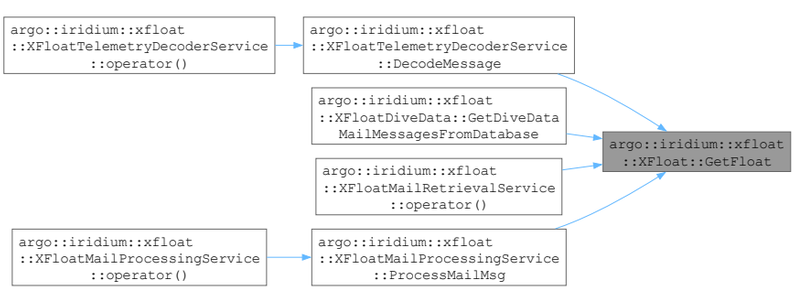
References CATCH_BLOCK, id2float_map_, id2float_map_mutex_, and LoadLocalCacheFromDatabase().

Referenced by argo::iridium::xfloat::XFloatTelemetryDecoderService::DecodeMessage(), argo::iridium::xfloat::XFloatDiveData::GetDiveDa
argo::iridium::xfloat::XFloatMailRetrievalService::operator()(), and argo::iridium::xfloat::XFloatMailProcessingService::ProcessMailMsg().

Here is the call graph for this function:



Here is the caller graph for this function:



10.86.3.42 GetKmlFactory()

```
static thread_specific_ptr< KmlFactory > & util::kml::KmlUtil::GetKmlFactory ()  [inline], [static],
[inherited]
```

Get the libkml factory, initializing it for this thread if necessary.

Exceptions

util::log::MessageException	if the factory cannot be obtained.

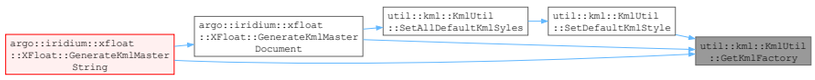
Returns

The thread's KML factory pointer (see kml_factory_).

References CATCH_BLOCK, and util::kml::KmlUtil::kml_factory_.

Referenced by GenerateKmlMasterDocument(), GenerateKmlMasterString(), and util::kml::KmlUtil::SetDefaultKmlStyle().

Here is the caller graph for this function:



10.86.3.43 GetListingDirectoryPath()

```
path argo::iridium::xfloat::XFloat::GetListingDirectoryPath () const
```

Get the listing directory path for this float.

Returns

The listing directory path for this float.

References CATCH_BLOCK.

10.86.3.44 GetSmallKmlPlacemarkStyleId()

```
static const string & util::kml::KmlUtil::GetSmallKmlPlacemarkStyleId (
            int id)  [inline], [static], [inherited]
```

Get the small placemark style ID for an index, modulo the palette size.

Parameters

in	*id*	The index.

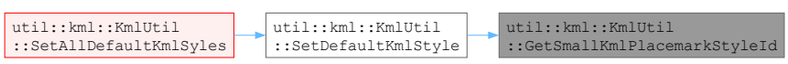
Returns

The KML style ID.

References util::kml::KmlUtil::kml_small_placemark_style_ids_.

Referenced by util::kml::KmlUtil::SetDefaultKmlStyle().

Here is the caller graph for this function:



10.86.3.45 GetSqlConnection()

```
static thread_specific_ptr< Connection > & util::mysql::MysqlUtils::GetSqlConnection (
            string url,
            string user,
            string password,
            string schema)  [inline], [static], [inherited]
```

Get a thread-specific instance of a MySQL connection object.

A new connection will be created, if necessary.

Parameters

in	*url*	The MySQL host server qualified address.
in	*user*	The MySQL account username.
in	*password*	The MySQL account password.
in	*schema*	The MySQL account schema name.

Exceptions

log::MessageException	if an error occurs while creating a new thread-specific instance of the SQL connection object

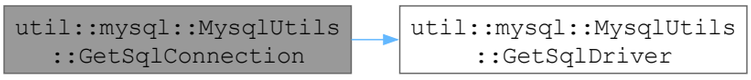
Returns

A thread-specific instance of a MySQL connection object.

References CATCH_BLOCK, util::mysql::MysqlUtils::GetSqlDriver(), and util::mysql::MysqlUtils::sql_connection_.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

Here is the call graph for this function:



Here is the caller graph for this function:

10.86.3.46 getSqlConnection()

`sql::Connection * util::mysql::LocalMysqlConfiguration::getSqlConnection () [static], [protected], [inherited]`

Generate a thread-specific instance of a MySQL connection object.

A new connection will be created, if necessary.

Exceptions

log::MessageException	if an error occurs while generating the SQL connection object.

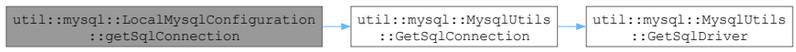
Returns

A thread-specific instance of a MySQL connection object.

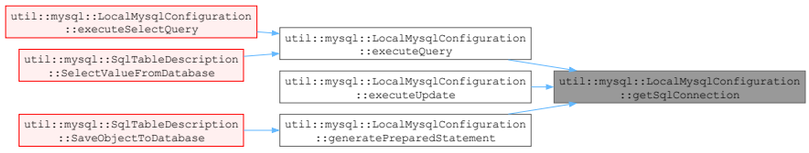
References util::mysql::LocalMysqlConfiguration::address_, CATCH_BLOCK, util::mysql::MysqlUtils::GetSqlConnection(), util::mysql::LocalMysqlConfiguration::password_, util::mysql::LocalMysqlConfiguration::port_, util::mysql::LocalMysqlConfiguration::protoc util::mysql::LocalMysqlConfiguration::schema_, and util::mysql::LocalMysqlConfiguration::user_.

Referenced by util::mysql::LocalMysqlConfiguration::executeQuery(), util::mysql::LocalMysqlConfiguration::executeUpdate(), and util::mysql::LocalMysqlConfiguration::generatePreparedStatement().

Here is the call graph for this function:



Here is the caller graph for this function:



10.86.3.47 GetSqlDriver()

```
static thread_specific_ptr< MySQL_Driver > & util::mysql::MysqlUtils::GetSqlDriver ()  [inline],
[static], [inherited]
```

Get a thread-specific instance of a MySQL driver object.

A new driver will created, if necessary.

Exceptions

log::MessageException	if an error occurs while creating a new thread-specific instance of the SQL driver object

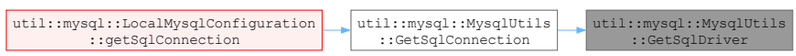
Returns

A thread-specific instance of a MySQL driver object.

References CATCH_BLOCK, and util::mysql::MysqlUtils::sql_driver_.

Referenced by util::mysql::MysqlUtils::GetSqlConnection().

Here is the caller graph for this function:



10.86.3.48 GetTriangleKmlPlacemarkStyleId()

```
static const string & util::kml::KmlUtil::GetTriangleKmlPlacemarkStyleId (
            int id)  [inline], [static], [inherited]
```

Get the triangle placemark style ID for an index, modulo the palette size.

Parameters

in	*id*	The index.

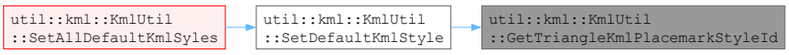
Returns

The KML style ID.

References util::kml::KmlUtil::kml_triangle_placemark_style_ids_.

Referenced by util::kml::KmlUtil::SetDefaultKmlStyle().

Here is the caller graph for this function:



10.86.3.49 id_dac()

```
int argo::iridium::Iridium::id_dac () const  [inline], [inherited]
```

Get the unique Data Assembly Center (DAC) identifier.

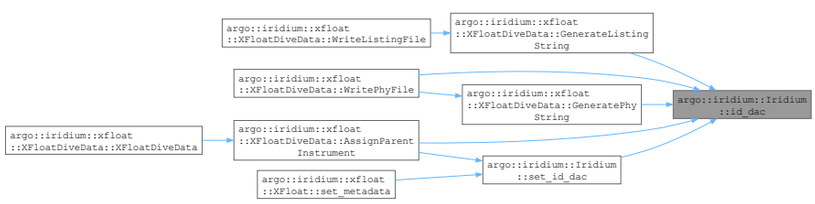
Returns

The unique Data Assembly Center (DAC) identifier.

References argo::iridium::Iridium::id_dac_, and util::thread::RecursiveLockable::mutex_.

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignParentInstrument(), argo::iridium::xfloat::XFloatDiveData::GenerateListingStrin
argo::iridium::xfloat::XFloatDiveData::GeneratePhyString(), argo::iridium::Iridium::set_id_dac(), and argo::iridium::xfloat::XFloatDiveData::

Here is the caller graph for this function:



10.86.3.50 id_imei()

```
long argo::iridium::Iridium::id_imei () const  [inline], [inherited]
```

Get the unique International Mobile Equipment Identifier (IMEI).

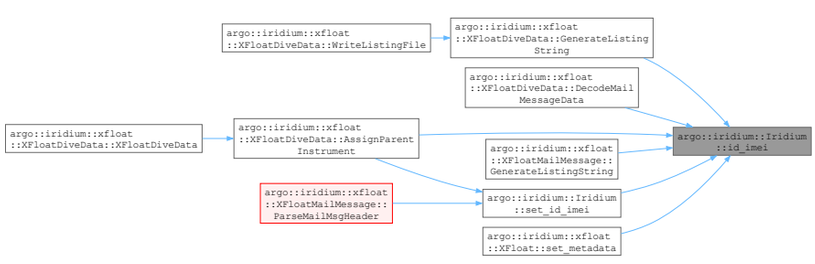
Returns

The unique International Mobile Equipment Identifier (IMEI).

References argo::iridium::Iridium::id_imei_, and util::thread::RecursiveLockable::mutex_.

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignParentInstrument(), argo::iridium::xfloat::XFloatDiveData::DecodeMailMessag
argo::iridium::xfloat::XFloatDiveData::GenerateListingString(), argo::iridium::xfloat::XFloatMailMessage::GenerateListingString(),
argo::iridium::Iridium::set_id_imei(), and set_metadata().

Here is the caller graph for this function:



10.86.3.51 id_instrument()

```
int argo::iridium::Iridium::id_instrument () const  [inline], [inherited]
```

Get the unique instrument identifier or instrument serial number.

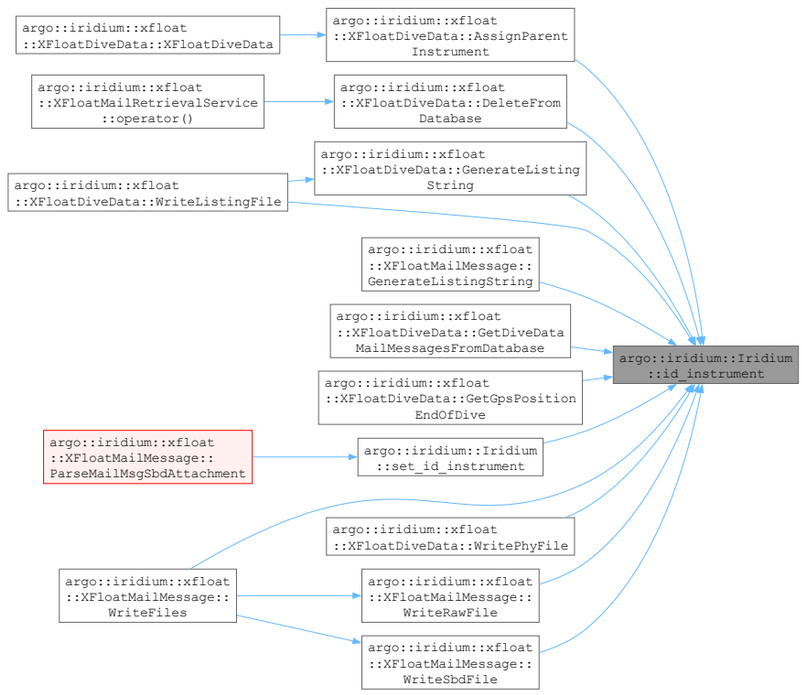
Returns

The unique instrument identifier or instrument serial number.

References argo::iridium::Iridium::id_instrument_, and util::thread::RecursiveLockable::mutex_.

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignParentInstrument(), argo::iridium::xfloat::XFloatDiveData::DeleteFromDatabas argo::iridium::xfloat::XFloatDiveData::GenerateListingString(), argo::iridium::xfloat::XFloatMailMessage::GenerateListingString(), argo::iridium::xfloat::XFloatDiveData::GetDiveDataMailMessagesFromDatabase(), argo::iridium::xfloat::XFloatDiveData::GetGpsPositionE argo::iridium::Iridium::set_id_instrument(), argo::iridium::xfloat::XFloatMailMessage::WriteFiles(), argo::iridium::xfloat::XFloatDiveData::Wr argo::iridium::xfloat::XFloatDiveData::WritePhyFile(), argo::iridium::xfloat::XFloatMailMessage::WriteRawFile(), and argo::iridium::xfloat::XFloatMailMessage::WriteSbdFile().

Here is the caller graph for this function:



10.86.3.52 id_wmo()

```
int argo::iridium::Iridium::id_wmo () const  [inline], [inherited]
```

Get the unique World Meteorological Organization (WMO) identifier.

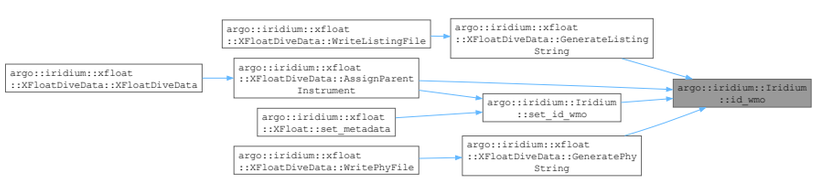
Returns

The unique World Meteorological Organization (WMO) identifier.

References argo::iridium::Iridium::id_wmo_, and util::thread::RecursiveLockable::mutex_.

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignParentInstrument(), argo::iridium::xfloat::XFloatDiveData::GenerateListingStrir argo::iridium::xfloat::XFloatDiveData::GeneratePhyString(), and argo::iridium::Iridium::set_id_wmo().

Here is the caller graph for this function:



10.86.3.53 instrument_ids_to_disable_phy_generation()

```
const vector< unsigned int > & argo::iridium::xfloat::XFloat::instrument_ids_to_disable_phy_↩
generation ()  [static]
```

Get the floats that the user has specified to disable PHY file generation.

Returns

A vector of floats that the user has specified to disable PHY file generation.

References instrument_ids_to_disable_phy_generation_.

10.86.3.54 instrument_ids_to_enable_phy_generation()

```
const vector< unsigned int > & argo::iridium::xfloat::XFloat::instrument_ids_to_enable_phy_↩
generation ()  [static]
```

Get the floats that the user has specified to enable PHY file generation.

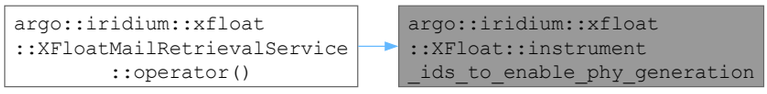
Returns

A vector of floats that the user has specified to enable PHY file generation.

References instrument_ids_to_enable_phy_generation_.

Referenced by argo::iridium::xfloat::XFloatMailRetrievalService::operator()().

Here is the caller graph for this function:



10.86.3.55 IsPositionValid()

```
template<typename T >
static bool util::position::PositionUtils::IsPositionValid (
            T lat,
            T lon)  [inline], [static], [inherited]
```

Check position validity.

Parameters

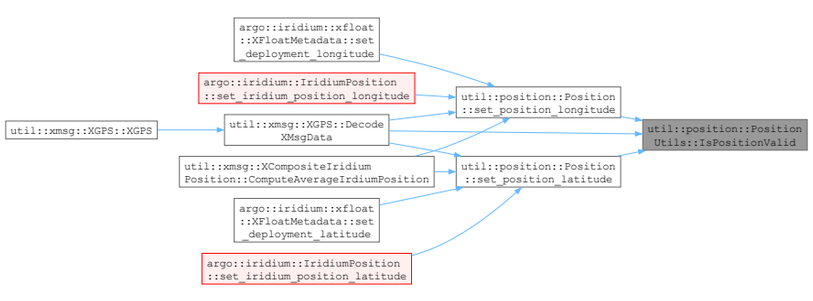
in	*lat*	Latitude of position.
in	*lon*	Longitude of position.

Returns

TRUE if (lat, lon) represents a valid geographical position, or FALSE otherwise.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), util::position::Position::set_position_latitude(), and util::position::Position::set_positio

Here is the caller graph for this function:



10.86.3.56 ListDirectories()

```
static vector< path > util::file::FileSystemUtils::ListDirectories (
            const path & dir)  [inline], [static], [inherited]
```

List all directories contained in the given directory.

Parameters

in	*dir*	The path to the directory of interest.

Exceptions

log::MessageException	if the directory does not exist, is not a directory, or an error occurs.

Returns

A vector of paths to all the directories contained in the given directory.

References CATCH_BLOCK, and EXC_THROW.

10.86.3.57 ListFiles()

```
static vector< path > util::file::FileSystemUtils::ListFiles (
            path dir,
            size_t n = 0)  [inline], [static], [inherited]
```

List all regular files contained in the given directory.

Parameters

in	*dir*	The path to the directory of interest.
in	*n*	If n>0, the maximum number of paths to return, else no effect.

Exceptions

log::MessageException	if the directory does not exist, is not a directory, or an error occurs.

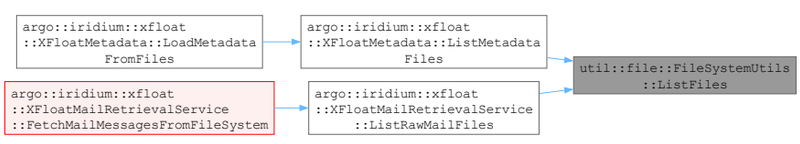
Returns

A vector of paths to all the regular files contained in the given directory.

References CATCH_BLOCK, and EXC_THROW.

Referenced by argo::iridium::xfloat::XFloatMetadata::ListMetadataFiles(), and argo::iridium::xfloat::XFloatMailRetrievalService::ListRawMa

Here is the caller graph for this function:



10.86.3.58 LoadLocalCacheFromDatabase()

```
void argo::iridium::xfloat::XFloat::LoadLocalCacheFromDatabase ()  [static]
```

Load all persistent instrument data from the database and use this data to populate the local cache.

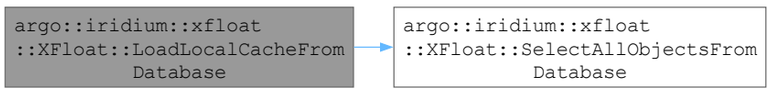
Exceptions

util::log::MessageException	if an error occurs while querying the database or populating the local cache.

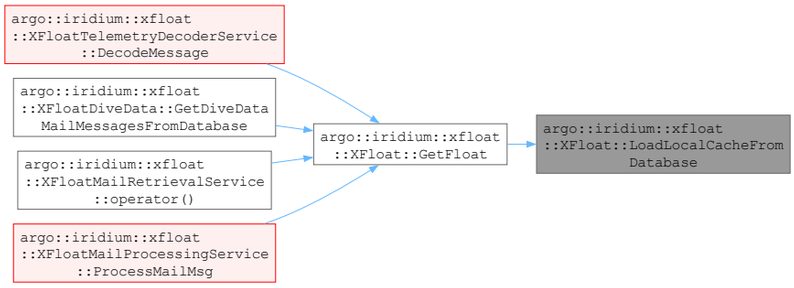
References id2float_map_mutex_, linf, lout, and SelectAllObjectsFromDatabase().

Referenced by GetFloat().

Here is the call graph for this function:



Here is the caller graph for this function:



10.86.3.59 LoadMetadataFromFiles()

```
void argo::iridium::xfloat::XFloat::LoadMetadataFromFiles ()  [static]
```

Load the instrument metadata.

This function loads the instruments' metadata from the metadata files on the local filesystem. It is called by the daemon thread during application initialization. A vector of metadata objects is created from the metadata files by calling XFloatMetadata::LoadMetadataFromFiles(). The function then iterates over the metadata files and assigns each to the corresponding float object before persisting the metadata object to the SQL database.

Exceptions

util::log::MessageException	if an error occurs while loading the instrument metadata from the files on the local file system.

Note

Metadata files that specify a DAC identifier of 9999 or -99 are used as place holders for instruments that have not yet been assigned a DAC identifier. Therefore, metadata files or objects specifying a DAC id less than 1, equal to 9999, or greater than 99998 will be discarded.

References linf, and lout.

10.86.3.60 LockObject()

```
bool util::thread::RecursiveLockable::LockObject (
            bool block)  [inline], [inherited]
```

Lock the object.

Parameters

in	*block*	If TRUE, then the calling thread will block until the lock becomes available.

Exceptions

util::log::MessageException	if an error occurs.

Returns

TRUE if the lock was successfully obtained, or FALSE if block is FALSE and another thread owns the lock on the mutex.

References CATCH_BLOCK, and util::thread::RecursiveLockable::mutex_.

10.86.3.61 metadata()

```
const shared_ptr< XFloatMetadata > & argo::iridium::xfloat::XFloat::metadata () const  [inline]
```

Get the metadata object for this instrument.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

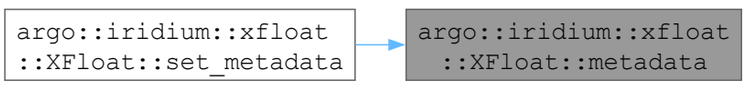
Returns

A shared pointer to the metadata object for this instrument.

References metadata_, and util::thread::RecursiveLockable::mutex_.

Referenced by set_metadata().

Here is the caller graph for this function:



10.86.3.62 MoveFileToDirectory()

```
static path util::file::FileSystemUtils::MoveFileToDirectory (
            const path & file,
            const path & dir_dst,
            bool create_dir = false)  [inline], [static], [inherited]
```

Copy a file into a directory.

Warning

Despite its name, this function copies the file and does not remove the source. If a file with the same name already exists in the destination directory, it is left unchanged. It is not currently called.

Parameters

in	*file*	The file to copy.
in	*dir_dst*	The destination directory.
in	*create_dir*	If TRUE, create the destination directory if it does not exist.

Exceptions

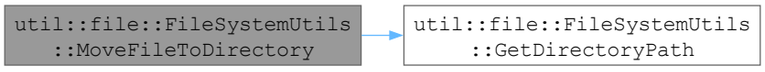
util::log::MessageException	if the file does not exist, is not a regular file, or cannot be copied.

Returns

The path of the file in the destination directory.

References CATCH_BLOCK, EXC_THROW, and util::file::FileSystemUtils::GetDirectoryPath().

Here is the call graph for this function:



10.86.3.63 offset_pressure()

```
uint16_t argo::iridium::xfloat::XFloat::offset_pressure () const  [inline]
```

Get the pressure offset value for this instrument, in dBar.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The pressure offset value for this instrument, in dBar.

References util::thread::RecursiveLockable::mutex_, and offset_pressure_.

10.86.3.64 offset_pressure_surface()

```
uint16_t argo::iridium::xfloat::XFloat::offset_pressure_surface () const  [inline]
```

Get the pressure offset value for surface data, in dBar.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The pressure offset value for this instrument, in dBar.

References util::thread::RecursiveLockable::mutex_, and offset_pressure_surface_.

10.86.3.65 offset_salinity()

```
uint16_t argo::iridium::xfloat::XFloat::offset_salinity () const  [inline]
```

Get the salinity offset value for this instrument, in PSU.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The salinity offset value for this instrument, in PSU.

References util::thread::RecursiveLockable::mutex_, and offset_salinity_.

10.86.3.66 offset_salinity_surface()

```
uint16_t argo::iridium::xfloat::XFloat::offset_salinity_surface () const  [inline]
```

Get the salinity offset value for surface data, in PSU.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The salinity offset value for this instrument, in PSU.

References util::thread::RecursiveLockable::mutex_, and offset_salinity_surface_.

10.86.3.67 offset_temperature()

```
uint16_t argo::iridium::xfloat::XFloat::offset_temperature () const  [inline]
```

Get the temperature offset value for this instrument, in degC.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The temperature offset value for this instrument, in degC.

References util::thread::RecursiveLockable::mutex_, and offset_temperature_.

10.86.3.68 offset_temperature_surface()

```
uint16_t argo::iridium::xfloat::XFloat::offset_temperature_surface () const  [inline]
```

Get the temperature offset value for surface data, in degC.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The temperature offset value for this instrument, in degC.

References util::thread::RecursiveLockable::mutex_, and offset_temperature_surface_.

10.86.3.69 phy_file_generation_is_enabled()

```
bool argo::iridium::xfloat::XFloat::phy_file_generation_is_enabled () const  [inline]
```

Is PHY file generation enabled?

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

TRUE if PHY file generation is enabled, or FALSE otherwise.

References util::thread::RecursiveLockable::mutex_, and phy_file_generation_is_enabled_.

10.86.3.70 ResetCachedDiveData()

```
void argo::iridium::xfloat::XFloat::ResetCachedDiveData (
            int dive)
```

Reset the dive data object for the given dive or cycle number.

This function releases the shared reference to the dive data object. It is called by the telemetry decoder thread once it determines that the dive data object is complete and no longer needed. Once all the shared references to the dive data object are released, then the object will be purged from memory.

Exceptions

None.	Never throws.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Parameters

in	*dive*	The dive or cycle number of the dive data object to reset.

References CATCH_NOTHROW.

10.86.3.71 ResetDiveData()

```
void argo::iridium::xfloat::XFloat::ResetDiveData ()
```

Reset the dive data objects for this float.

References CATCH_BLOCK.

10.86.3.72 SaveLocalCacheToDatabase()

```
void argo::iridium::xfloat::XFloat::SaveLocalCacheToDatabase ()  [static]
```

Save the local cache of X-Float objects to the database.

Exceptions

util::log::MessageException	if an error occurs.

References CATCH_NOTHROW, linf, and lout.

10.86.3.73 SaveObjectToDatabase()

```
void argo::iridium::xfloat::XFloat::SaveObjectToDatabase () const  [private], [virtual]
```

Save this float object to the database.

This function is used to persist the float object to the SQL database. If the float object has a non-NULL reference to a metadata object, then the metadata object will be persisted to the database as well.

Exceptions

util::log::MessageException	if an error occurs while saving this float object or its metadata object to the database.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Implements util::mysql::LocalPersistentEntity.

References CATCH_BLOCK, and ldbg.

10.86.3.74 SelectAllObjectsFromDatabase()

```
vector< shared_ptr< XFloat > > argo::iridium::xfloat::XFloat::SelectAllObjectsFromDatabase (
            const string & groupBy = "",
            const string & orderBy = "",
            const unsigned int & limit = 0,
            const unsigned int & offset = 0)  [static]
```

Select all XFloat objects from this SQL database table.

Parameters

in	*groupBy*	The SQL select statement group-by clause.
in	*orderBy*	The SQL select statement order-by clause.
in	*limit*	The SQL select statement result set row limit value.
in	*offset*	The SQL select statement result set row offset.

Exceptions

util::log::MessageException	if an error occurs while querying the database.

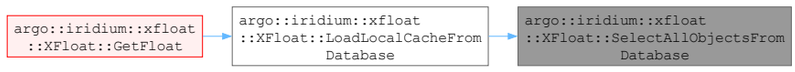
Returns

A vector of shared pointers to the object selected from the database.

References CATCH_BLOCK.

Referenced by LoadLocalCacheFromDatabase().

Here is the caller graph for this function:



10.86.3.75 SelectObjectFromDatabase()

```
shared_ptr< XFloat > argo::iridium::xfloat::XFloat::SelectObjectFromDatabase (
            const string & where = "",
            const string & groupBy = "",
            const string & orderBy = "",
            const unsigned int & offset = 0)  [static]
```

Get a single XFloat object from the database.

Parameters

in	*where*	The SQL select statement where-condition clause.
in	*groupBy*	The SQL select statement group-by clause.
in	*orderBy*	The SQL select statement order-by clause.
in	*offset*	The SQL select statement result set row offset.

Exceptions

util::log::MessageException	if an error occurs while querying the database.

Returns

A shared pointer to the object selected from the database.

References CATCH_BLOCK.

10.86.3.76 SelectObjectsFromDatabase()

```
vector< shared_ptr< XFloat > > argo::iridium::xfloat::XFloat::SelectObjectsFromDatabase (
            const string & where = "",
            const string & groupBy = "",
            const string & orderBy = "",
            const unsigned int & limit = 0,
            const unsigned int & offset = 0)  [static]
```

Query float objects from the SQL database.

This function is used to query the SQL database table corresponding to the float class and construct a vector of float objects, populated from the results of the SQL query.

Exceptions

util::log::MessageException	if an error occurs while querying the database. or constructing the float objects from the result of the query.

Parameters

in	*where*	The SQL select statement "where-condition" clause.
in	*groupBy*	The SQL select statement "group-by expression" clause.
in	*orderBy*	The SQL select statement "order-by expression" clause.
in	*limit*	The SQL select statement result set row limit value.
in	*offset*	The SQL select statement result set row offset value.

Returns

A vector of shared references to the float objects constructed from the results of the SQL query.

References CATCH_BLOCK.

10.86.3.77 SelectValueFromDatabase()

```
template<typename TT >
template uint8_t argo::iridium::xfloat::XFloat::SelectValueFromDatabase< uint8_t > (
            const string & what,
            const string & where = "",
            const string & groupBy = "",
            const string & orderBy = "",
            const unsigned int & offset = 0)  [static]
```

Select a value from the SQL database table that contains the persisted float objects.

This function may be used to select a singular value from the table that comprises the persisted float objects. For example, if an identifier is unknown, it is unnecessary to query the database to construct and entire float object to recover the identifier. Rather, this function may be used to recover the identifier without constructing a new object in the process.

Exceptions

util::log::MessageException	if the SQL database query fails or if an error occurs while attempting to interpret the result of the query.

Note

The result set of this query is always limited to a single result. That is, the result set row limit value is automatically set to one. If more than one row in the database table satisfy the "where condition" clause of the select statement, then only the value of the first result will be returned.

Parameters

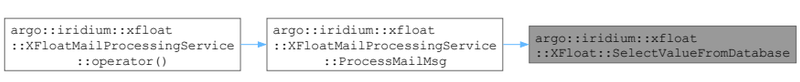
in	*what*	The SQL select statement "what" clause. This clause may only contain a singular result e.g. the name of the SQL database table column of interest.
in	*where*	The SQL select statement "where condition" clause.
in	*groupBy*	The SQL select statement "group-by expression" clause.
in	*orderBy*	The SQL select statement "order-by expression" clause.
in	*offset*	The SQL select statement result set row offset value.

Returns

The selected value from the SQL database table.

Referenced by argo::iridium::xfloat::XFloatMailProcessingService::ProcessMailMsg().

Here is the caller graph for this function:



10.86.3.78 set_argo_instrument_type()

```
void argo::iridium::xfloat::XFloat::set_argo_instrument_type (
            uint8_t argo_instrument_type)  [inline], [private]
```

Set the enumerated Argo instrument type identifier.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Parameters

in	*argo_instrument_type*	The enumerated Argo instrument type identifier to set.

References argo_instrument_type(), argo_instrument_type_, and util::thread::RecursiveLockable::mutex_.

Here is the call graph for this function:

10.86.3.79 set_fw_version()

```
void argo::iridium::xfloat::XFloat::set_fw_version (
            uint32_t fw_version)  [inline], [private]
```

Set the instrument firmware version.

Note

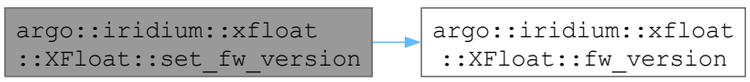
This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Parameters

in	*fw_version*	The firmware version

References fw_version(), fw_version_, and util::thread::RecursiveLockable::mutex_.

Here is the call graph for this function:



10.86.3.80 set_gain_pressure()

```
void argo::iridium::xfloat::XFloat::set_gain_pressure (
            uint16_t gain)  [inline], [private]
```

Set the instrument pressure gain value, in dBar/LSB.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Parameters

in	*gain*	The instrument pressure gain value to set, in dBar/LSB.

References gain_pressure_, and util::thread::RecursiveLockable::mutex_.

10.86.3.81 set_gain_pressure_surface()

```
void argo::iridium::xfloat::XFloat::set_gain_pressure_surface (
            uint16_t gain)  [inline], [private]
```

Set the instrument pressure gain value for surface data, in dBar/LSB.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Parameters

in	*gain*	The instrument pressure gain value to set, in dBar/LSB.

References gain_pressure_surface_, and util::thread::RecursiveLockable::mutex_.

10.86.3.82 set_gain_salinity()

```
void argo::iridium::xfloat::XFloat::set_gain_salinity (
            uint16_t gain)  [inline], [private]
```

Set the instrument salinity gain value, in PSU/LSB.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Parameters

in	*gain*	The instrument salinity gain value to set, in PSU/LSB.

References gain_salinity_, and util::thread::RecursiveLockable::mutex_.

10.86.3.83 set_gain_salinity_surface()

```
void argo::iridium::xfloat::XFloat::set_gain_salinity_surface (
            uint16_t gain)  [inline], [private]
```

Set the instrument salinity gain value for surface data, in PSU/LSB.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Parameters

in	*gain*	The instrument salinity gain value to set, in PSU/LSB.

References gain_salinity_surface_, and util::thread::RecursiveLockable::mutex_.

10.86.3.84 set_gain_temperature()

```
void argo::iridium::xfloat::XFloat::set_gain_temperature (
            uint16_t gain)  [inline], [private]
```

Set the instrument temperature gain value, in degC/LSB.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Parameters

in	*gain*	The instrument temperature gain value to set, in degC/LSB.

References gain_temperature_, and util::thread::RecursiveLockable::mutex_.

10.86.3.85 set_gain_temperature_surface()

```
void argo::iridium::xfloat::XFloat::set_gain_temperature_surface (
            uint16_t gain)  [inline], [private]
```

Set the instrument temperature gain value for surface data, in degC/LSB.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Parameters

in	*gain*	The instrument temperature gain value to set, in degC/LSB.

References gain_temperature_surface_, and util::thread::RecursiveLockable::mutex_.

10.86.3.86 set_id_dac()

```
void argo::iridium::Iridium::set_id_dac (
            int id_dac)  [inline], [protected], [inherited]
```

Set the unique Data Assembly Center (DAC) identifier.

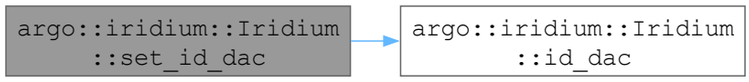
Parameters

in	*id_dac*	The unique Data Assembly Center (DAC) identifier.

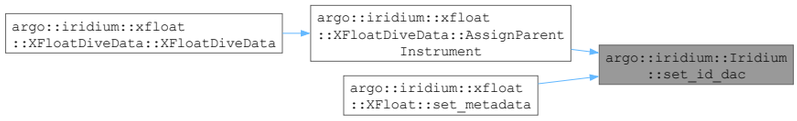
References argo::iridium::Iridium::id_dac(), argo::iridium::Iridium::id_dac_, and util::thread::RecursiveLockable::mutex_.

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignParentInstrument(), and set_metadata().

Here is the call graph for this function:



Here is the caller graph for this function:



10.86.3.87 set_id_imei()

```
void argo::iridium::Iridium::set_id_imei (
            long id_imei)  [inline], [protected], [inherited]
```

Set the unique International Mobile Equipment Identifier (IMEI).

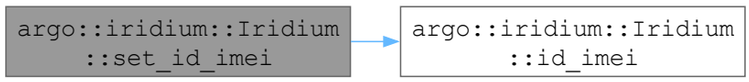
Parameters

in	*id_imei*	The unique International Mobile Equipment Identifier (IMEI).

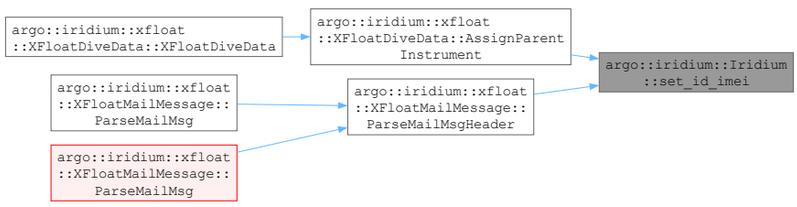
References argo::iridium::Iridium::id_imei(), argo::iridium::Iridium::id_imei_, and util::thread::RecursiveLockable::mutex_.

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignParentInstrument(), and argo::iridium::xfloat::XFloatMailMessage::ParseMailM

Here is the call graph for this function:



Here is the caller graph for this function:



10.86.3.88 set_id_instrument()

```
void argo::iridium::Iridium::set_id_instrument (
            int id_instrument)  [inline], [protected], [inherited]
```

Set the unique instrument identifier or instrument serial number.

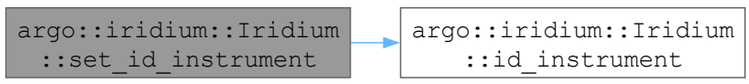
Parameters

in	*id_instrument*	The unique instrument identifier or instrument serial number

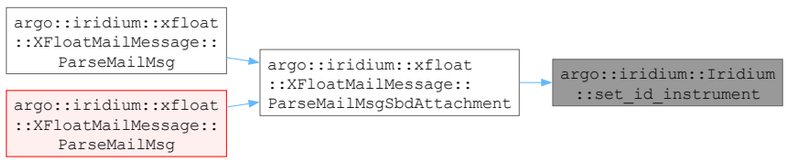
References argo::iridium::Iridium::id_instrument(), argo::iridium::Iridium::id_instrument_, and util::thread::RecursiveLockable::mutex_.

Referenced by argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgSbdAttachment().

Here is the call graph for this function:



Here is the caller graph for this function:



10.86.3.89 set_id_wmo()

```
void argo::iridium::Iridium::set_id_wmo (
            int id_wmo)  [inline], [protected], [inherited]
```

Set the unique World Meteorological Organization (WMO) identifier.

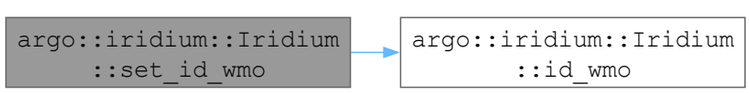
Parameters

in	*id_wmo*	The unique World Meteorological Organization (WMO) identifier.

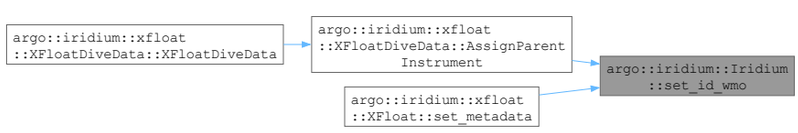
References argo::iridium::Iridium::id_wmo(), argo::iridium::Iridium::id_wmo_, and util::thread::RecursiveLockable::mutex_.

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignParentInstrument(), and set_metadata().

Here is the call graph for this function:



Here is the caller graph for this function:



10.86.3.90 set_metadata()

```
void argo::iridium::xfloat::XFloat::set_metadata (
            const shared_ptr< XFloatMetadata > & metadata)  [inline], [private]
```

Assign an instrument metadata object to this X-Float object.

Note

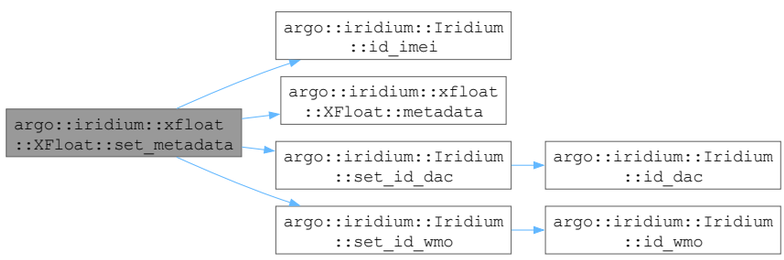
This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Parameters

in	*metadata*	A shared pointer to the instrument metadata object to assign to this X-Float object.

References argo::iridium::Iridium::id_imei(), metadata(), metadata_, util::thread::RecursiveLockable::mutex_, argo::iridium::Iridium::set_id_dac(), and argo::iridium::Iridium::set_id_wmo().

Here is the call graph for this function:



10.86.3.91 set_offset_pressure()

```
void argo::iridium::xfloat::XFloat::set_offset_pressure (
            uint16_t offs)  [inline], [private]
```

Set the instrument pressure offset value, in dBar.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Parameters

in	*offs*	The instrument pressure offset value to set, in dBar.

References util::thread::RecursiveLockable::mutex_, and offset_pressure_.

10.86.3.92 set_offset_pressure_surface()

```
void argo::iridium::xfloat::XFloat::set_offset_pressure_surface (
            uint16_t offs)  [inline], [private]
```

Set the instrument pressure offset value for surface data, in dBar.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Parameters

in	*offs*	The instrument pressure offset value to set, in dBar.

References util::thread::RecursiveLockable::mutex_, and offset_pressure_surface_.

10.86.3.93 set_offset_salinity()

```
void argo::iridium::xfloat::XFloat::set_offset_salinity (
            uint16_t offs)  [inline], [private]
```

Set the instrument salinity offset value, in PSU.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Parameters

in	*offs*	The instrument salinity offset value to set, in PSU.

References util::thread::RecursiveLockable::mutex_, and offset_salinity_.

10.86.3.94 set_offset_salinity_surface()

```
void argo::iridium::xfloat::XFloat::set_offset_salinity_surface (
            uint16_t offs)  [inline], [private]
```

Set the instrument salinity offset value for surface data, in PSU.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Parameters

in	*offs*	The instrument salinity offset value to set, in PSU.

References util::thread::RecursiveLockable::mutex_, and offset_salinity_surface_.

10.86.3.95 set_offset_temperature()

```
void argo::iridium::xfloat::XFloat::set_offset_temperature (
            uint16_t offs)  [inline], [private]
```

Set the instrument temperature offset value, in degC.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Parameters

in	*offs*	The instrument temperature offset value to set, in degC.

References util::thread::RecursiveLockable::mutex_, and offset_temperature_.

10.86.3.96 set_offset_temperature_surface()

```
void argo::iridium::xfloat::XFloat::set_offset_temperature_surface (
            uint16_t offs)  [inline], [private]
```

Set the instrument temperature offset value for surface data, in degC.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Parameters

in	*offs*	The instrument temperature offset value to set, in degC.

References util::thread::RecursiveLockable::mutex_, and offset_temperature_surface_.

10.86.3.97 set_phy_file_generation_is_enabled()

```
void argo::iridium::xfloat::XFloat::set_phy_file_generation_is_enabled (
            bool is_enabled)  [inline], [private]
```

Set the PHY-file-generation-is-enabled flag.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Parameters

in	*is_enabled*	The PHY-file-generation-is-enabled flag to set.

References util::thread::RecursiveLockable::mutex_, and phy_file_generation_is_enabled_.

10.86.3.98 set_start_of_mission_datetime()

```
void argo::iridium::xfloat::XFloat::set_start_of_mission_datetime (
            const ptime & datetime)  [inline], [private]
```

Set the date/time of the start-of-mission for this instrument.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Parameters

in	*datetime*	The date/time of the start-of-mission for this instrument to set.

References util::thread::RecursiveLockable::mutex_, and start_of_mission_datetime_.

10.86.3.99 set_xmsg_version_major()

```
void argo::iridium::xfloat::XFloat::set_xmsg_version_major (
            uint8_t xmsg_version_major)  [inline], [private]
```

Set the X-Message major version number.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Parameters

in	*xmsg_version_major*	The X-Message major version number to set.

References util::thread::RecursiveLockable::mutex_, xmsg_version_major(), and xmsg_version_major_.

Here is the call graph for this function:

10.86.3.100 set_xmsg_version_minor()

```
void argo::iridium::xfloat::XFloat::set_xmsg_version_minor (
            uint8_t xmsg_version_minor)  [inline], [private]
```

Set the X-Message minor version number.

Note

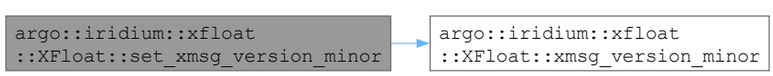
This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Parameters

in	*xmsg_version_minor*	The X-Message minor version number to set.

References util::thread::RecursiveLockable::mutex_, xmsg_version_minor(), and xmsg_version_minor_.

Here is the call graph for this function:



10.86.3.101 SetAllDefaultKmlSyles()

```
static void util::kml::KmlUtil::SetAllDefaultKmlSyles (
            const DocumentPtr & document)  [inline], [static], [inherited]
```

Add the default styles for every palette colour to a KML document.

Parameters

in	*document*	The KML document to add the styles to.

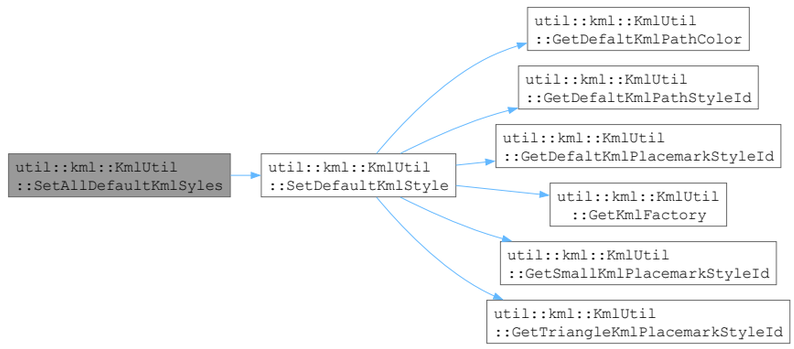
Exceptions

util::log::MessageException	if an error occurs.

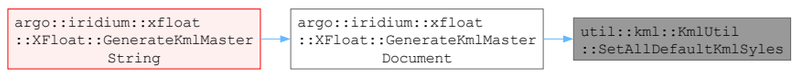
References CATCH_BLOCK, and util::kml::KmlUtil::SetDefaultKmlStyle().

Referenced by GenerateKmlMasterDocument().

Here is the call graph for this function:



Here is the caller graph for this function:



10.86.3.102 SetDefaultKmlStyle()

```
static void util::kml::KmlUtil::SetDefaultKmlStyle (
            const DocumentPtr & document,
            unsigned int style_idx)  [inline], [static], [inherited]
```

Add the four default styles for one palette colour to a KML document.

Adds a 5-pixel path (line) style, a large dot placemark style, a small dot placemark style and a triangle placemark style, all in the colour `style_idx` (modulo the palette size). The icons are loaded from `http://argo.whoi.edu/kml/img/`. Errors are logged, not thrown.

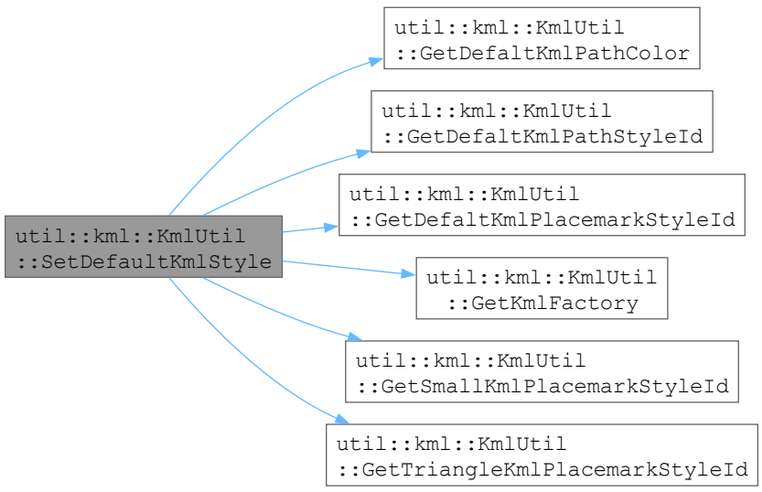
Parameters

in	*document*	The KML document to add the styles to.
in	*style_idx*	The palette index of the colour.

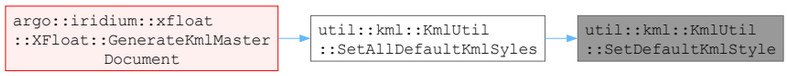
References CATCH_NOTHROW, util::kml::KmlUtil::GetDefaltKmlPathColor(), util::kml::KmlUtil::GetDefaltKmlPathStyleId(), util::kml::KmlUtil::GetDefaltKmlPlacemarkStyleId(), util::kml::KmlUtil::GetKmlFactory(), util::kml::KmlUtil::GetSmallKmlPlacemarkStyleId(), and util::kml::KmlUtil::GetTriangleKmlPlacemarkStyleId().

Referenced by util::kml::KmlUtil::SetAllDefaultKmlSyles().

Here is the call graph for this function:



Here is the caller graph for this function:



10.86.3.103 start_of_mission_datetime()

```
ptime argo::iridium::xfloat::XFloat::start_of_mission_datetime () const  [inline]
```

Get the date/time of the start-of-mission for this instrument.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

Returns

The date/time of the start-of-mission for this instrument.

References util::thread::RecursiveLockable::mutex_, and start_of_mission_datetime_.

10.86.3.104 StartupTasks()

```
void argo::iridium::xfloat::XFloat::StartupTasks ()  [static]
```

Perform XFloat startup tasks.

If the user has specified that PHY file generation should be enabled or disabled for one or more floats, then PHY file generation flag will be updated by this function.

References CATCH_BLOCK, and Iout.

10.86.3.105 UnlockObject()

```
void util::thread::RecursiveLockable::UnlockObject ()  [inline], [inherited]
```

Unlock the object.

Exceptions

util::log::MessageException	if an error occurs.

References CATCH_BLOCK, and util::thread::RecursiveLockable::mutex_.

10.86.3.106 WriteKmlFile()

```
bool argo::iridium::xfloat::XFloat::WriteKmlFile ()  [private]
```

Write the Google Earth KML file for this float.

Returns

TRUE if the file was written, or FALSE if KML file generation is disabled or the KML file is already up-to-date.

References CATCH_BLOCK, and linf.

10.86.3.107 WriteKmlMasterFile()

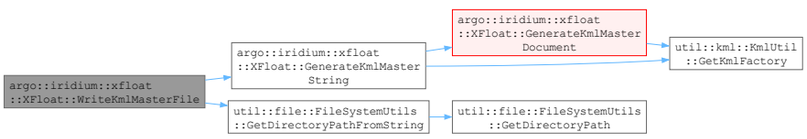
```
void argo::iridium::xfloat::XFloat::WriteKmlMasterFile ()  [static], [private]
```

Write the KML master file.

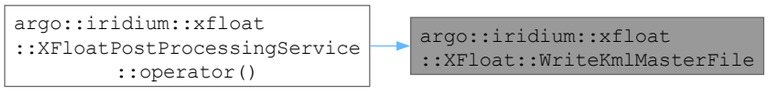
References CATCH_BLOCK, GenerateKmlMasterString(), util::file::FileSystemUtils::GetDirectoryPathFromString(), kml_directory_path_, kml_file_generation_is_enabled_, and linf.

Referenced by argo::iridium::xfloat::XFloatPostProcessingService::operator()().

Here is the call graph for this function:



Here is the caller graph for this function:



10.86.3.108 WriteStatusFile()

```
void argo::iridium::xfloat::XFloat::WriteStatusFile ()  [private]
```

Write the status file for this float.

References CATCH_BLOCK, and linf.

10.86.3.109 xmsg_version_major()

```
uint8_t argo::iridium::xfloat::XFloat::xmsg_version_major () const  [inline]
```

Get the X-Message major version identifier for this instrument.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

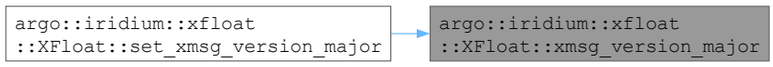
Returns

The X-Message major version identifier for this instrument.

References util::thread::RecursiveLockable::mutex_, and xmsg_version_major_.

Referenced by set_xmsg_version_major().

Here is the caller graph for this function:



10.86.3.110 xmsg_version_minor()

```
uint8_t argo::iridium::xfloat::XFloat::xmsg_version_minor () const  [inline]
```

Get the X-Message minor version identifier for this instrument.

Note

This object will be locked for the scope of the execution of this function. This function places a lock guard on the mutual exclusion (mutex) lock, thereby locking the float object and preventing access or modification of the object by any other thread. Any thread that does attempt to access or modify this object will block until the lock is released.

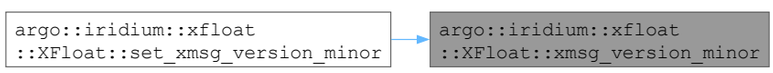
Returns

The X-Message minor version identifier for this instrument.

References util::thread::RecursiveLockable::mutex_, and xmsg_version_minor_.

Referenced by set_xmsg_version_minor().

Here is the caller graph for this function:



10.86.4 Friends And Related Symbol Documentation

10.86.4.1 Initialize

```
const util::config::LocalConfiguration & Initialize (
            int argc,
            char ** argv)  [friend]
```

Initialize the application.

All startup initialization of the application is performed by this function. First, the local configuration initialization is invoked, which parses the configuration file and/or the command-line arguments and also acquires a run-time lock. Then the global logger is initialized, setting the sources, sinks, formats, and filters of the global logger. Next, the signal handlers are registered, defining the behavior of the application if a catchable signal is encountered from the system. Finally, requisite data and metadata is loaded from files on the local file system and from the SQL database.

Parameters

in	*argc*	The number of command-line arguments.
in	*argv*	The command-line argument list.

Exceptions

None.	Never throws.

Returns

The local configuration object.

10.86.4.2 util::config::LocalConfiguration::GenerateCommandLineOptionsDescription

```
options_description util::config::LocalConfiguration::GenerateCommandLineOptionsDescription ()
[friend]
```

Friend function for populating some of the static members of this class from the command-line options.

10.86.4.3 util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription

```
options_description util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription
() [friend]
```

Friend function for populating some of the static members of this class from the configuration file options.

10.86.5 Member Data Documentation

10.86.5.1 address_

```
std::string util::mysql::LocalMysqlConfiguration::address_ [static], [private], [inherited]
```

The SQL server qualified name or IP address string.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.86.5.2 argo_instrument_type_

```
XArgoInstrumentType_t argo::iridium::xfloat::XFloat::argo_instrument_type_ [private]
```

The enumerated Argo instrument type identifier.

This field is used to identify the specific type of instrument i.e. Solo-II, Deep Argo, or ALAMO/Hurricane floats. The value of this field may affect the behavior of some aspects of the application. For example, the manner in which some Iridium Short Burst Data (SBD) messages are decoded is dependent on the specific type of instrument.

Referenced by argo_instrument_type(), and set_argo_instrument_type().

10.86.5.3 dive2data_map_

```
flat_map<int, shared_ptr<XFloatDiveData> > argo::iridium::xfloat::XFloat::dive2data_map_ [private]
```

The dive-number-to-dive-data-object map.

This object is a flat map that contains shared references to all of the incomplete dive data objects that belong to this instrument. The dive data objects are uniquely indexed by the dive or cycle number. The map acts as a dynamic, local cache of the incomplete instrument dive data.

Each time a thread requests the dive data object from the float object using the GetDiveData() function, the dive or cycle number is used to look up the dive data object in this map. If the corresponding dive data object does already exist in the map, it will be created and added to the map before the shared reference is returned.

10.86.5.4 EARTH_RADIUS_KM_EQUATORIAL

```
const double util::position::PositionUtils::EARTH_RADIUS_KM_EQUATORIAL = 6378.1  [static], [private],
[inherited]
```

The equatorial radius of the earth, in kilometers.

Referenced by util::position::PositionUtils::CalculateEllipsoidHeight().

10.86.5.5 EARTH_RADIUS_KM_POLAR

```
const double util::position::PositionUtils::EARTH_RADIUS_KM_POLAR = 6356.1  [static], [private],
[inherited]
```

The polar radius of the earth, in kilometers.

Referenced by util::position::PositionUtils::CalculateEllipsoidHeight().

10.86.5.6 fw_version_

```
uint32_t argo::iridium::xfloat::XFloat::fw_version_  [private]
```

The reported instrument firmware version.

Referenced by fw_version(), and set_fw_version().

10.86.5.7 gain_pressure_

```
uint16_t argo::iridium::xfloat::XFloat::gain_pressure_  [private]
```

The pressure sensitivity gain factor, in dBar/LSB.

This field is used for decoding the drift or profile data encoded in the the Iridium Short Burst Data (SBD) messages. It along with the corresponding offset, is used to convert the raw sensor counts into scientific units.

Referenced by gain_pressure(), and set_gain_pressure().

10.86.5.8 gain_pressure_surface_

```
uint16_t argo::iridium::xfloat::XFloat::gain_pressure_surface_  [private]
```

The pressure sensitivity gain factor for surface data, in dBar/LSB.

This field is used for decoding the surface data encoded in the the Iridium Short Burst Data (SBD) messages. It along with the corresponding offset, is used to convert the raw sensor counts into scientific units.

Referenced by gain_pressure_surface(), and set_gain_pressure_surface().

10.86.5.9 gain_salinity_

```
uint16_t argo::iridium::xfloat::XFloat::gain_salinity_  [private]
```

The salinity sensitivity scale factor, in PSU/LSB.

This field is used for decoding the drift or profile data encoded in the the Iridium Short Burst Data (SBD) messages. It along with the corresponding offset, is used to convert the raw sensor counts into scientific units.

Referenced by gain_salinity(), and set_gain_salinity().

10.86.5.10 gain_salinity_surface_

uint16_t argo::iridium::xfloat::XFloat::gain_salinity_surface_ [private]

The salinity sensitivity scale factor for surface data, in PSU/LSB.

This field is used for decoding the surface data encoded in the the Iridium Short Burst Data (SBD) messages. It along with the corresponding offset, is used to convert the raw sensor counts into scientific units.

Referenced by gain_salinity_surface(), and set_gain_salinity_surface().

10.86.5.11 gain_temperature_

uint16_t argo::iridium::xfloat::XFloat::gain_temperature_ [private]

The temperature sensitivity gain factor, in degC/LSB.

This field is used for decoding the drift or profile data encoded in the the Iridium Short Burst Data (SBD) messages. It along with the corresponding offset, is used to convert the raw sensor counts into scientific units.

Referenced by gain_temperature(), and set_gain_temperature().

10.86.5.12 gain_temperature_surface_

uint16_t argo::iridium::xfloat::XFloat::gain_temperature_surface_ [private]

The temperature sensitivity gain factor for surface data, in degC/LSB.

This field is used for decoding the surface data encoded in the the Iridium Short Burst Data (SBD) messages. It along with the corresponding offset, is used to convert the raw sensor counts into scientific units.

Referenced by gain_temperature_surface(), and set_gain_temperature_surface().

10.86.5.13 id2float_map_

flat_map< int, shared_ptr< XFloat > > argo::iridium::xfloat::XFloat::id2float_map_ [static], [private]

The instrument-identifier-to-instrument-object map.

This object is a flat map that contains shared references to all of the instances of this class. It acts as a static, local cache of the instrument data. The object maps the unique instrument identifier to the corresponding float object. During application initialization, this map is populated as a result of querying all the known instruments from the SQL database.

When a specific float object is requested by any of the threads, this map is searched for the instrument object first. If the float object does not already exist, then it is created and added to this map.

Upon application termination, all of the float objects in this map are persisted to the SQL database so that they may be again loaded into this map when the application is restarted.

Accesses and modifications to this map are synchronized using the recursive mutex id2float_map_mutex_. The mutex provides an exclusive lock so that only one thread at a time may either access or modify this object.

Referenced by GetAllCachedFloats(), and GetFloat().

10.86.5.14 id2float_map_mutex_

recursive_mutex argo::iridium::xfloat::XFloat::id2float_map_mutex_ [static], [private]

The mutex lock for synchronizing access to the instrument-identifier-to-instrument-object map.

This mutual exclusion lock is used to ensure that accesses and modifications to the instrument-identifier-to-instrument-object map, id2float_map_, are synchronized so that only one thread at a time may either access or modify this object. Because the mutex is recursive, it may be locked multiple times by the same thread without causing the thread to block indefinitely.

Referenced by GetAllCachedFloats(), GetFloat(), and LoadLocalCacheFromDatabase().

10.86.5.15 id_dac_

```
int argo::iridium::Iridium::id_dac_  [private], [inherited]
```

The unique Data Assembly Center (DAC) identifier.

Referenced by argo::iridium::Iridium::id_dac(), and argo::iridium::Iridium::set_id_dac().

10.86.5.16 id_imei_

```
long argo::iridium::Iridium::id_imei_  [private], [inherited]
```

The unique International Mobile Equipment Identifier (IMEI).

Referenced by argo::iridium::Iridium::id_imei(), and argo::iridium::Iridium::set_id_imei().

10.86.5.17 id_instrument_

```
int argo::iridium::Iridium::id_instrument_  [private], [inherited]
```

The unique instrument identifier or instrument serial number.

Referenced by argo::iridium::Iridium::id_instrument(), and argo::iridium::Iridium::set_id_instrument().

10.86.5.18 id_wmo_

```
int argo::iridium::Iridium::id_wmo_  [private], [inherited]
```

The unique World Meteorological Organization (WMO) identifier.

Referenced by argo::iridium::Iridium::id_wmo(), and argo::iridium::Iridium::set_id_wmo().

10.86.5.19 instrument_ids_to_disable_phy_generation_

```
vector< unsigned int > argo::iridium::xfloat::XFloat::instrument_ids_to_disable_phy_generation_
[static], [private]
```

A list of instrument identifiers for which to disable PHY file generation.

On startup, the list of instruments contained by this vector will be used to set the XFloatDiveData::phy_file_generation_is_enabled_ at the start of execution of the mail retrieval thread.

The value of this field is set as a result of parsing the command-line arguments and/or configuration file options during application initialization.

Referenced by instrument_ids_to_disable_phy_generation().

10.86.5.20 instrument_ids_to_enable_phy_generation_

```
vector< unsigned int > argo::iridium::xfloat::XFloat::instrument_ids_to_enable_phy_generation_↩
[static], [private]
```

A list of instrument identifiers for which to enable PHY file generation.

On startup, the list of instruments contained by this vector will be used to set the XFloatDiveData::phy_file_generation_is_enabled_ at the start of execution of the mail retrieval thread. Subsequently, the dive data for these instruments will be reprocessed in order generate PHY files for previous dives.

The value of this field is set as a result of parsing the command-line arguments and/or configuration file options during application initialization.

Referenced by instrument_ids_to_enable_phy_generation().

10.86.5.21 kml_default_path_colors_

```
const Color32 util::kml::KmlUtil::kml_default_path_colors_  [static], [private], [inherited]
```

Initial value:

```
= {
        Color32(0xB0, 0xFF, 0x00, 0x00),
        Color32(0xB0, 0xFF, 0x80, 0x00),
        Color32(0xB0, 0xFF, 0xFF, 0x00),
        Color32(0xB0, 0x80, 0xFF, 0x00),
        Color32(0xB0, 0x00, 0xFF, 0x00),
        Color32(0xB0, 0x00, 0xFF, 0x80),
        Color32(0xB0, 0x00, 0xFF, 0xFF),
        Color32(0xB0, 0x00, 0x80, 0xFF),
        Color32(0xB0, 0x00, 0x00, 0xFF),
        Color32(0xB0, 0x80, 0x00, 0xFF),
        Color32(0xB0, 0xFF, 0x00, 0xFF),
        Color32(0xB0, 0xFF, 0x00, 0x80),
        Color32(0xB0, 0x80, 0x80, 0x080),
 }
```

The colour palette for instrument paths and placemarks, indexed by style index.

Referenced by util::kml::KmlUtil::GetDefaltKmlPathColor().

10.86.5.22 kml_default_path_style_ids_

```
const string util::kml::KmlUtil::kml_default_path_style_ids_  [static], [private], [inherited]
```

Initial value:

```
= {
        "path1",
        "path2",
        "path3",
        "path4",
        "path5",
        "path6",
        "path7",
        "path8",
        "path9",
        "pathA",
        "pathB",
        "pathC",
        "pathD",}
```

The KML style IDs of the path (line) styles, one per palette colour.

Referenced by util::kml::KmlUtil::GetDefaltKmlPathStyleId().

10.86.5.23 kml_default_placemark_style_ids_

```
const string util::kml::KmlUtil::kml_default_placemark_style_ids_  [static], [private], [inherited]
```

Initial value:

```
= {
        "dot1_1",
        "dot1_2",
        "dot1_3",
        "dot1_4",
        "dot1_5",
        "dot1_6",
        "dot1_7",
        "dot1_8",
        "dot1_9",
        "dot1_A",
        "dot1_B",
        "dot1_C",
        "dot1_D", }
```

The KML style IDs of the large placemark styles, one per palette colour.

Referenced by util::kml::KmlUtil::GetDefaltKmlPlacemarkStyleId().

10.86.5.24 kml_directory_path_

```
path argo::iridium::xfloat::XFloat::kml_directory_path_  [static], [private]
```

The KML file directory.

Referenced by WriteKmlMasterFile().

10.86.5.25 kml_document_

```
kmldom::DocumentPtr argo::iridium::xfloat::XFloat::kml_document_  [private]
```

The GE KML document.

10.86.5.26 kml_document_updated_

```
bool argo::iridium::xfloat::XFloat::kml_document_updated_  [private]
```

The KML document is updated flag.

10.86.5.27 kml_factory_

```
thread_specific_ptr< KmlFactory > util::kml::KmlUtil::kml_factory_  [static], [private], [inherited]
```

The libkml factory used to create KML elements, held per thread.

Warning

This holds libkml's process-wide factory (`KmlFactory::GetFactory()`), and the default cleanup deletes it when the owning thread exits or CloseKmlFactory() is called. Only the post-processing thread uses KML, so this currently happens only at shutdown.

Referenced by util::kml::KmlUtil::CloseKmlFactory(), and util::kml::KmlUtil::GetKmlFactory().

10.86.5.28 kml_file_generation_is_enabled_

```
bool argo::iridium::xfloat::XFloat::kml_file_generation_is_enabled_  [static], [private]
```

The global KML-file-generation-is-enabled flag.

The value of this field determines whether or not any KML files will be written to the local filesystem. If TRUE, then KML files will be written each time a complete set of dive data is received from a deployed instrument. Otherwise, if FALSE, no KML files will be written to the local filesystem.

The value of this field is set as a result of parsing the command-line arguments and/or configuration file options during application initialization.

Referenced by WriteKmlMasterFile().

10.86.5.29 kml_file_path_

```
path argo::iridium::xfloat::XFloat::kml_file_path_  [private]
```

The KMLfile path.

10.86.5.30 kml_file_updated_

```
bool argo::iridium::xfloat::XFloat::kml_file_updated_  [private]
```

The KML file is updated flag.

10.86.5.31 kml_small_placemark_style_ids_

```
const string util::kml::KmlUtil::kml_small_placemark_style_ids_  [static], [private], [inherited]
```

Initial value:

```
= {
        "dot1s_1",
        "dot1s_2",
        "dot1s_3",
        "dot1s_4",
        "dot1s_5",
        "dot1s_6",
        "dot1s_7",
        "dot1s_8",
        "dot1s_9",
        "dot1s_A",
        "dot1s_B",
        "dot1s_C",
        "dot1s_D", }
```

The KML style IDs of the small placemark styles, one per palette colour.

Referenced by util::kml::KmlUtil::GetSmallKmlPlacemarkStyleId().

10.86.5.32 kml_triangle_placemark_style_ids_

```
const string util::kml::KmlUtil::kml_triangle_placemark_style_ids_  [static], [private], [inherited]
```

Initial value:

```
= {
        "tri1_1",
        "tri1_2",
        "tri1_3",
        "tri1_4",
        "tri1_5",
        "tri1_6",
        "tri1_7",
        "tri1_8",
        "tri1_9",
        "tri1_A",
        "tri1_B",
        "tri1_C",
        "tri1_D", }
```

The KML style IDs of the triangle placemark styles, one per palette colour.

Referenced by util::kml::KmlUtil::GetTriangleKmlPlacemarkStyleId().

10.86.5.33 latest_gps_position_datetime_

```
ptime argo::iridium::xfloat::XFloat::latest_gps_position_datetime_  [private]
```

The latest GPS position date/time.

10.86.5.34 latest_gps_position_dive_number_

```
int argo::iridium::xfloat::XFloat::latest_gps_position_dive_number_  [private]
```

The latest GPS position dive number.

10.86.5.35 latest_gps_position_latitude_

```
double argo::iridium::xfloat::XFloat::latest_gps_position_latitude_  [private]
```

The latest GPS position latitude.

10.86.5.36 latest_gps_position_longitude_

```
double argo::iridium::xfloat::XFloat::latest_gps_position_longitude_  [private]
```

The latest GPS position longitude.

10.86.5.37 latest_iridium_position_cep_radius_

```
int argo::iridium::xfloat::XFloat::latest_iridium_position_cep_radius_ [private]
```

The latest iridium position circular error probable radius.

10.86.5.38 latest_iridium_position_datetime_

```
ptime argo::iridium::xfloat::XFloat::latest_iridium_position_datetime_ [private]
```

The latest iridium position date/time.

10.86.5.39 latest_iridium_position_dive_number_

```
int argo::iridium::xfloat::XFloat::latest_iridium_position_dive_number_ [private]
```

The latest iridium position dive number.

10.86.5.40 latest_iridium_position_latitude_

```
double argo::iridium::xfloat::XFloat::latest_iridium_position_latitude_ [private]
```

The latest iridium position latitude.

10.86.5.41 latest_iridium_position_longitude_

```
double argo::iridium::xfloat::XFloat::latest_iridium_position_longitude_ [private]
```

The latest iridium position longitude.

10.86.5.42 latest_iridium_sbd_momsn_

```
int argo::iridium::xfloat::XFloat::latest_iridium_sbd_momsn_ [private]
```

The latest iridium position Mobile-Originated Message Number (MOMSN).

10.86.5.43 latest_iridium_sbd_mtmsn_

```
int argo::iridium::xfloat::XFloat::latest_iridium_sbd_mtmsn_ [private]
```

The latest iridium position Mobile-Terminated Message Number (MTMSN).

10.86.5.44 latest_iridium_startup_position_datetime_

```
ptime argo::iridium::xfloat::XFloat::latest_iridium_startup_position_datetime_ [private]
```

The latest startup iridium position date/time.

10.86.5.45 latest_iridium_startup_sbd_momsn_

```
int argo::iridium::xfloat::XFloat::latest_iridium_startup_sbd_momsn_ [private]
```

The latest startup iridium position Mobile-Originated Message Number (MOMSN).

10.86.5.46 metadata_

```
shared_ptr<XFloatMetadata> argo::iridium::xfloat::XFloat::metadata_  [private]
```

The instrument metadata.

This is a shared reference to this instrument metadata object corresponding to this instrument. During application initialization the metadata object is constructed from the metadata file on the local filesystem. If there is no metadata file corresponding to to this instrument, then the reference will be NULL.

This object contains the metadata for this instrument. It is object is used, in part, to determine whether the instrument has been deployed, and consequently, whether or not a PHY file should be generated once a the complete data for a dive is received. If the instrument object does not have a corresponding metadata object, then PHY files will NOT be generated for this instrument

Referenced by metadata(), and set_metadata().

10.86.5.47 mutex_

```
recursive_mutex util::thread::RecursiveLockable::mutex_  [mutable], [protected], [inherited]
```

Recursive mutual exclusion lock for synchronizing operations.

Referenced by util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AggregateS
argo_instrument_type(), argo::iridium::xfloat::XFloatDiveData::argo_instrument_type(), argo::iridium::xfloat::XFloatDiveData::AssignArgoD
argo::iridium::xfloat::XFloatDiveData::AssignAuxTemperatureBinned(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SAL
argo::iridium::xfloat::XFloatDiveData::AssignDriftPressure(), argo::iridium::xfloat::XFloatDiveData::AssignDriftSalinity(),
argo::iridium::xfloat::XFloatDiveData::AssignDriftTemperatre(), argo::iridium::xfloat::XFloatDiveData::AssignEngineeringAbort(),
argo::iridium::xfloat::XFloatDiveData::AssignEngineeringAscent(), argo::iridium::xfloat::XFloatDiveData::AssignEngineeringBitFail(),
argo::iridium::xfloat::XFloatDiveData::AssignEngineeringBitPass(), argo::iridium::xfloat::XFloatDiveData::AssignEngineeringDescent(),
argo::iridium::xfloat::XFloatDiveData::AssignEngineeringDiagnostic(), argo::iridium::xfloat::XFloatDiveData::AssignEngineeringDrift(),
argo::iridium::xfloat::XFloatDiveData::AssignEngineeringIce(), argo::iridium::xfloat::XFloatDiveData::AssignEngineeringStandard(),
argo::iridium::xfloat::XFloatDiveData::AssignEngineeringSurface(), argo::iridium::xfloat::XFloatDiveData::AssignFallTimeSeries(),
argo::iridium::xfloat::XFloatDiveData::AssignGps(), argo::iridium::xfloat::XFloatDiveData::AssignMailMessage(),
argo::iridium::xfloat::XFloatDiveData::AssignMissionCmdEchoback(), argo::iridium::xfloat::XFloatDiveData::AssignParentInstrument(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignPressureMessage(
argo::iridium::xfloat::XFloatDiveData::AssignProfilePressureBinned(), argo::iridium::xfloat::XFloatDiveData::AssignProfilePressureUnbinn
argo::iridium::xfloat::XFloatDiveData::AssignProfileSalinityBinned(), argo::iridium::xfloat::XFloatDiveData::AssignProfileSalinityUnbinned()
argo::iridium::xfloat::XFloatDiveData::AssignProfileTemperatureBinned(), argo::iridium::xfloat::XFloatDiveData::AssignProfileTemperatureU
argo::iridium::xfloat::XFloatDiveData::AssignPumpTimeSeries(), argo::iridium::xfloat::XFloatDiveData::AssignRiseTimeSeries(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignSalinityMessage(),
argo::iridium::xfloat::XFloatDiveData::AssignSurfaceTimeSeries(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_C
argo::iridium::xfloat::XFloatDiveData::AssignValveData(), argo::iridium::xfloat::XFloatDiveData::CheckCompleteness(),
argo::iridium::xfloat::XFloatMetadata::comment(), argo::iridium::xfloat::XFloatDiveData::ComputePressureOffsetFromSurfaceTimeSeries(
argo::iridium::xfloat::XFloatMetadata::deployer(), argo::iridium::xfloat::XFloatMetadata::deployer_address(), argo::iridium::xfloat::XFloatMe
argo::iridium::xfloat::XFloatMetadata::deployment_datetime(), argo::iridium::xfloat::XFloatMetadata::deployment_datetime_status(),
argo::iridium::xfloat::XFloatMetadata::deployment_latitude(), argo::iridium::xfloat::XFloatMetadata::deployment_longitude(),
argo::iridium::xfloat::XFloatMetadata::deployment_platform(), argo::iridium::xfloat::XFloatMetadata::deployment_position_status(),
argo::iridium::xfloat::XFloatMetadata::deployment_type(), fw_version(), gain_pressure(), gain_pressure_surface(),
gain_salinity(), gain_salinity_surface(), gain_temperature(), gain_temperature_surface(), argo::iridium::xfloat::XFloatDiveData::GenerateC
argo::iridium::xfloat::XFloatDiveData::GenerateCompositeIridiumPosition(), argo::iridium::xfloat::XFloatDiveData::GenerateCompositePun
argo::iridium::xfloat::XFloatDiveData::GenerateCompositeRiseTimeSeriesData(), argo::iridium::xfloat::XFloatDiveData::GenerateListingSt
util::xmsg::XCompositeDrift::GenerateListingString(), util::xmsg::XCompositeHiResProfile::GenerateListingString(),
util::xmsg::XCompositeProfile::GenerateListingString(), argo::iridium::xfloat::XFloatDiveData::GenerateOrderedMomsnList(),
argo::iridium::xfloat::XFloatDiveData::GeneratePhyString(), util::xmsg::XCompositeDrift::GeneratePhyString(), util::xmsg::XCompositeHiR
util::xmsg::XCompositeProfile::GeneratePhyString(), argo::iridium::xfloat::XFloatDiveData::GetDiveDataMailMessagesFromDatabase(),
argo::iridium::xfloat::XFloatDiveData::GetGainsAndOffsets(), argo::iridium::xfloat::XFloatDiveData::GetGpsPositionEndOfDive(),
argo::iridium::xfloat::XFloatDiveData::GetSurfaceGainsAndOffsets(), argo::iridium::xfloat::XFloatDiveData::HasBinnedProfileData(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::HasContiguousMessageIr
argo::iridium::xfloat::XFloatDiveData::HasContiguousMomsns(), argo::iridium::xfloat::XFloatDiveData::HasContiguousPids(),
argo::iridium::xfloat::XFloatDiveData::HasDiveData(), argo::iridium::xfloat::XFloatDiveData::HasDriftData(), argo::iridium::xfloat::XFloatDiv
argo::iridium::xfloat::XFloatDiveData::HasFallRiseData(), argo::iridium::xfloat::XFloatDiveData::HasPositionData(),
argo::iridium::xfloat::XFloatDiveData::HasUnbinnedProfileData(), argo::iridium::xfloat::XFloatDiveData::HasValidGpsPosition(),
argo::iridium::Iridium::id_dac(), argo::iridium::xfloat::XFloatDiveData::id_dive(), argo::iridium::xfloat::XFloatMailMessage::id_dive(),
argo::iridium::Iridium::id_imei(), argo::iridium::Iridium::id_instrument(), argo::iridium::xfloat::XFloatMailMessage::id_msg(),
argo::iridium::xfloat::XFloatMailMessage::id_msg_uid(), argo::iridium::xfloat::XFloatMailMessage::id_sbd_momsn(),
argo::iridium::xfloat::XFloatMailMessage::id_sbd_mtmsn(), argo::iridium::xfloat::XFloatMailMessage::id_sbd_packet(),
argo::iridium::Iridium::id_wmo(), argo::iridium::xfloat::XFloatDiveData::intermediate_listing_file_path(), util::xmsg::AbstractXSensorDataCo
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::IsDataComplete(),
argo::iridium::xfloat::XFloatMetadata::local_path(), util::thread::RecursiveLockable::LockObject(), argo::iridium::xfloat::XFloatMetadata::ma

metadata(), argo::iridium::xfloat::XFloatMailMessage::msg_body_is_valid(), argo::iridium::xfloat::XFloatMailMessage::msg_body_text(), argo::iridium::xfloat::XFloatMailMessage::msg_header_datetime(), argo::iridium::xfloat::XFloatMailMessage::msg_header_from(), argo::iridium::xfloat::XFloatMailMessage::msg_header_is_valid(), argo::iridium::xfloat::XFloatMailMessage::msg_header_raw_text(), argo::iridium::xfloat::XFloatMailMessage::msg_header_subject(), argo::iridium::xfloat::XFloatMailMessage::msg_is_valid(), argo::iridium::xfloat::XFloatMailMessage::msg_raw_text(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, T util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::num_saln_samples(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::num_temp_samples(), offset_pressure(), offset_pressure_surface(), offset_salinity(), offset_salinity_surface(), offset_temperature(), offset_temperature_surface(), argo::iridium::xfloat::XFloatMailMessage::ParseMailMsg(), argo::iridium::xfloat::XFloatMailMessage::ParseM argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgBody(), argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgHeader(), argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgSbdAttachment(), phy_file_generation_is_enabled(), argo::iridium::xfloat::XFloatD argo::iridium::xfloat::XFloatMetadata::principal_investigators(), argo::iridium::xfloat::XFloatMetadata::principal_investigators_address(), argo::iridium::xfloat::XFloatMetadata::principal_investigators_originating_country(), argo::iridium::xfloat::XFloatMailMessage::raw_file_loc util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::Reset(), argo::iridium::xfloat::XFloatDiveData::ResetData(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_ util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SamplePresAsFloat(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleSaln(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleSalnAsFloat(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleTemp(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleTempAsFloat(), argo::iridium::xfloat::XFloatDiveData::SaveObjectToDatabase(), argo::iridium::xfloat::XFloatMailMessage::SaveObjectToDatabase(), argo::iridium::xfloat::XFloatMetadata::SaveObjectToDatabase(), argo::iridium::xfloat::XFloatMailMessage::sbd_data(), argo::iridium::xfloat::XFloatMailMessage::sbd_data_is_valid(), argo::iridium::xfloat::XFloatMailMessage::sbd_file_local_path(), argo::iridium::xfloat::XFloatMailMessage::sbd_file_name(), argo::iridium::xfloat::XFloatMailMessage::sbd_file_size_bytes(), argo::iridium::xfloat::XFloatMailMessage::sbd_session_status_code(), argo::iridium::xfloat::XFloatMailMessage::sbd_session_status_strir argo::iridium::xfloat::XFloatDiveData::SensorGainsAndOffsetsAreKnown(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, set_argo_instrument_type(), argo::iridium::xfloat::XFloatDiveData::set_argo_instrument_type(), argo::iridium::xfloat::XFloatMetadata::set_ argo::iridium::xfloat::XFloatMetadata::set_deployer(), argo::iridium::xfloat::XFloatMetadata::set_deployer_address(), argo::iridium::xfloat::XFloatMetadata::set_deployment_cruise_id(), argo::iridium::xfloat::XFloatMetadata::set_deployment_datetime(), argo::iridium::xfloat::XFloatMetadata::set_deployment_datetime_status(), argo::iridium::xfloat::XFloatMetadata::set_deployment_latitude() argo::iridium::xfloat::XFloatMetadata::set_deployment_longitude(), argo::iridium::xfloat::XFloatMetadata::set_deployment_platform(), argo::iridium::xfloat::XFloatMetadata::set_deployment_position_status(), argo::iridium::xfloat::XFloatMetadata::set_deployment_type(), set_fw_version(), set_gain_pressure(), set_gain_pressure_surface(), set_gain_salinity(), set_gain_salinity_surface(), set_gain_temperature(), set_gain_temperature_surface(), argo::iridium::Iridium::set_id_dac(), argo::iridium::xfloat::XFloatDiveData::set_id argo::iridium::xfloat::XFloatMailMessage::set_id_dive(), argo::iridium::Iridium::set_id_imei(), argo::iridium::Iridium::set_id_instrument(), argo::iridium::xfloat::XFloatMailMessage::set_id_msg(), argo::iridium::xfloat::XFloatMailMessage::set_id_msg_uid(), argo::iridium::xfloat::XFloatMailMessage::set_id_sbd_momsn(), argo::iridium::xfloat::XFloatMailMessage::set_id_sbd_mtmsn(), argo::iridium::xfloat::XFloatMailMessage::set_id_sbd_packet(), argo::iridium::Iridium::set_id_wmo(), argo::iridium::xfloat::XFloatDiveData: argo::iridium::xfloat::XFloatMetadata::set_local_path(), argo::iridium::xfloat::XFloatMetadata::set_manufacturer(), set_metadata(), argo::iridium::xfloat::XFloatMailMessage::set_msg_body_is_valid(), argo::iridium::xfloat::XFloatMailMessage::set_msg_b argo::iridium::xfloat::XFloatMailMessage::set_msg_header_datetime(), argo::iridium::xfloat::XFloatMailMessage::set_msg_header_from() argo::iridium::xfloat::XFloatMailMessage::set_msg_header_is_valid(), argo::iridium::xfloat::XFloatMailMessage::set_msg_header_raw_tex argo::iridium::xfloat::XFloatMailMessage::set_msg_header_subject(), argo::iridium::xfloat::XFloatMailMessage::set_msg_is_valid(), argo::iridium::xfloat::XFloatMailMessage::set_msg_raw_text(), set_offset_pressure(), set_offset_pressure_surface(), set_offset_salinity(), set_offset_salinity_surface(), set_offset_temperature(), set_offset_temperature_surface(), set_phy_file_generation_is_enabled(), argo::iridium::xfloat::XFloatDiveData::set_phy_file_path(), argo::iridium::xfloat::XFloatMetadata::se argo::iridium::xfloat::XFloatMetadata::set_principal_investigators_address(), argo::iridium::xfloat::XFloatMetadata::set_principal_investiga argo::iridium::xfloat::XFloatMailMessage::set_raw_file_local_path(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_data(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_data_is_valid(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_file_local_path(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_file_name(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_file_size_bytes(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_session_status_code(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_session_sta set_start_of_mission_datetime(), argo::iridium::xfloat::XFloatMetadata::set_wmo_instrument_type(), argo::iridium::xfloat::XFloatMetadata set_xmsg_version_major(), set_xmsg_version_minor(), start_of_mission_datetime(), util::thread::RecursiveLockable::UnlockObject(), argo::iridium::xfloat::XFloatDiveData::UpdateDerivedSurfaceDataFields(), argo::iridium::xfloat::XFloatMetadata::wmo_instrument_type(), argo::iridium::xfloat::XFloatMetadata::wmo_recorder_type(), argo::iridium::xfloat::XFloatDiveData::WriteListingFile(), argo::iridium::xfloat::XFloatDiveData::WritePhyFile(), argo::iridium::xfloat::XFloatMailMessage::WriteSbdFile(), xmsg_version_major(), and xmsg_version_minor().

10.86.5.48 netcdf_file_generation_is_enabled_

```
bool argo::iridium::xfloat::XFloat::netcdf_file_generation_is_enabled_  [private]
```

The NetCDF-file-generation-is-enabled flag.

10.86.5.49 offset_pressure_

```
uint16_t argo::iridium::xfloat::XFloat::offset_pressure_  [private]
```

The pressure offset, in dBar.

This field is used for decoding the drift or profile data encoded in the the Iridium Short Burst Data (SBD) messages. It along with the corresponding gain, is used to convert the raw sensor counts into scientific units.

Referenced by offset_pressure(), and set_offset_pressure().

10.86.5.50 offset_pressure_surface_

```
uint16_t argo::iridium::xfloat::XFloat::offset_pressure_surface_  [private]
```

The pressure offset for surface data, in dBar.

This field is used for decoding the surface data encoded in the the Iridium Short Burst Data (SBD) messages. It along with the corresponding gain, is used to convert the raw sensor counts into scientific units.

Referenced by offset_pressure_surface(), and set_offset_pressure_surface().

10.86.5.51 offset_salinity_

```
uint16_t argo::iridium::xfloat::XFloat::offset_salinity_  [private]
```

The salinity offset, in PSU.

This field is used for decoding the drift or profile data encoded in the the Iridium Short Burst Data (SBD) messages. It along with the corresponding gain, is used to convert the raw sensor counts into scientific units.

Referenced by offset_salinity(), and set_offset_salinity().

10.86.5.52 offset_salinity_surface_

```
uint16_t argo::iridium::xfloat::XFloat::offset_salinity_surface_  [private]
```

The salinity offset for surface data, in PSU.

This field is used for decoding the surface data encoded in the the Iridium Short Burst Data (SBD) messages. It along with the corresponding gain, is used to convert the raw sensor counts into scientific units.

Referenced by offset_salinity_surface(), and set_offset_salinity_surface().

10.86.5.53 offset_temperature_

```
uint16_t argo::iridium::xfloat::XFloat::offset_temperature_  [private]
```

The temperature offset, in degC.

This field is used for decoding the drift or profile data encoded in the the Iridium Short Burst Data (SBD) messages. It along with the corresponding gain, is used to convert the raw sensor counts into scientific units.

Referenced by offset_temperature(), and set_offset_temperature().

10.86.5.54 offset_temperature_surface_

```
uint16_t argo::iridium::xfloat::XFloat::offset_temperature_surface_  [private]
```

The temperature offset for surface data, in degC.

This field is used for decoding the surface data encoded in the the Iridium Short Burst Data (SBD) messages. It along with the corresponding gain, is used to convert the raw sensor counts into scientific units.

Referenced by offset_temperature_surface(), and set_offset_temperature_surface().

10.86.5.55 password_

```
std::string util::mysql::LocalMysqlConfiguration::password_  [static], [private], [inherited]
```

The SQL server account password string.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.86.5.56 phy_file_generation_is_enabled_

```
bool argo::iridium::xfloat::XFloat::phy_file_generation_is_enabled_ [private]
```

The PHY-file-generation-enabled flag.

This field determines, in part, whether or not a PHY file will be generated once the complete data for a single dive is received from the instrument. If FALSE, then PHY files will NOT be generated for this instrument. Otherwise, if TRUE, PHY files for this instrument will be generated only if the other requisite conditions are met.

Referenced by phy_file_generation_is_enabled(), and set_phy_file_generation_is_enabled().

10.86.5.57 port_

```
unsigned int util::mysql::LocalMysqlConfiguration::port_ [static], [private], [inherited]
```

The SQL server port number.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.86.5.58 protocol_

```
std::string util::mysql::LocalMysqlConfiguration::protocol_ [static], [private], [inherited]
```

The protocol to use in the SQL server connection e.g.

"tcp".

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.86.5.59 schema_

```
std::string util::mysql::LocalMysqlConfiguration::schema_ [static], [private], [inherited]
```

The SQL server schema name.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.86.5.60 sql_connection_

```
thread_specific_ptr<Connection> util::mysql::MysqlUtils::sql_connection_ [static], [private],
[inherited]
```

Tthe thread-specific pointer to MySQL connection object.

Thread-specific MySQL connection, released by `MysqlUtils::CleanupSqlConnection()` when the thread exits.

Referenced by util::mysql::MysqlUtils::CloseSqlConnection(), and util::mysql::MysqlUtils::GetSqlConnection().

10.86.5.61 sql_driver_

```
thread_specific_ptr<MySQL_Driver> util::mysql::MysqlUtils::sql_driver_ [static], [private],
[inherited]
```

The thread-specific pointer to MySQL driver object.

Thread-specific reference to the MySQL driver.

Its cleanup function, `MysqlUtils::CleanupSqlDriver()`, does not delete the shared driver instance.

Referenced by util::mysql::MysqlUtils::GetSqlDriver().

10.86.5.62 sql_table_description_

```
const util::mysql::SqlTableDescription< XFloat > argo::iridium::xfloat::XFloat::sql_table_↩
description_ [static], [private]
```

The SQL table description object.

This SQL table description provides the details needed to map this object to a relational database table. It utilizes the accessor methods of the class and the details of the SQL table column to (1) create an SQL INSERT statement to save the object into the database, (2) populate one or more objects' fields as a result of an SQL SELECT query, or (3) query the SQL database to access one or more of the values that are stored the the SQL table corresponding to this object.

10.86.5.63 start_of_mission_datetime_

```
ptime argo::iridium::xfloat::XFloat::start_of_mission_datetime_ [private]
```

The date/time of startup of the instrument.

This field represents the most recent date/time that the instrument was started up. This field is used, in part, to validate incoming message from the instrument. Messages that have either and Iridium position or email header date/time that is subsequent to the value of this field will be discarded.

Generally, the value of this field is assigned from the date/time reported by the GPS message that is reported as a result of the instrument self-test when the instrument is started. However, if the instrument is unable to obtain a valid GPS fix, then the value of this field needs to be set by alternative means.

Todo Implement an alternative method to assign the value of this field if the date/time reported by the start-of-mission GPS position is invalid or unavailable.

Referenced by set_start_of_mission_datetime(), and start_of_mission_datetime().

10.86.5.64 status_directory_path_

```
path argo::iridium::xfloat::XFloat::status_directory_path_ [static], [private]
```

The global status file directory path.

All status files will be written to this directory.

10.86.5.65 status_file_generation_is_enabled_

```
bool argo::iridium::xfloat::XFloat::status_file_generation_is_enabled_ [static], [private]
```

The global status-file-generation-is-enabled flag.

If TRUE, then status files will be written.

10.86.5.66 status_file_path_

```
path argo::iridium::xfloat::XFloat::status_file_path_ [private]
```

The status file path.

10.86.5.67 status_file_updated_

```
bool argo::iridium::xfloat::XFloat::status_file_updated_ [private]
```

The status-file-is-updated flag.

10.86.5.68 user_

```
std::string util::mysql::LocalMysqlConfiguration::user_  [static], [private], [inherited]
```

The SQL server account username string.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.86.5.69 xmsg_version_major_

```
uint8_t argo::iridium::xfloat::XFloat::xmsg_version_major_  [private]
```

The X-Message format major version number.

This field indicates the X-Message version used by this instrument's firmware. It affects the manner in which the X-↩ Messages are decoded that reflect the iterative version of the X-Message format.

Referenced by set_xmsg_version_major(), and xmsg_version_major().

10.86.5.70 xmsg_version_minor_

```
uint8_t argo::iridium::xfloat::XFloat::xmsg_version_minor_  [private]
```

The X-Message format minor version number.

This field indicates the X-Message version used by this instrument's firmware. It affects the manner in which the X-↩ Messages are decoded that reflect the iterative version of the X-Message format.

Referenced by set_xmsg_version_minor(), and xmsg_version_minor().

The documentation for this class was generated from the following files:

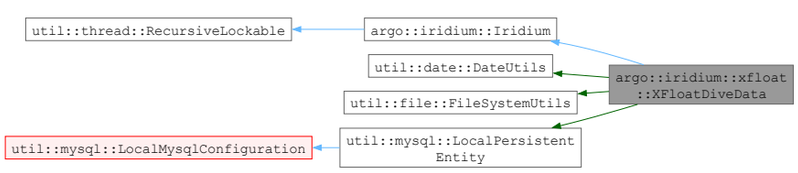
- data/XFloat.h
- data/XFloat.cpp

10.87 argo::iridium::xfloat::XFloatDiveData Class Reference

The instrument or float dive data class.

```
#include <XFloatDiveData.h>
```

Inheritance diagram for argo::iridium::xfloat::XFloatDiveData:



Public Member Functions

- XFloatDiveData ()

  The default constructor.
- XFloatDiveData (const shared_ptr< XFloat > &parent, int dive)

  Parameterized constructor.
- ~XFloatDiveData ()

  The destructor.
- XArgoInstrumentType_t argo_instrument_type () const

  Get the enumerated Argo instrument type identifier.
- bool CheckCompleteness ()

  Is this dive data object complete?
- float ComputePressureOffsetFromSurfaceTimeSeries (unsigned int max_num_dives)

  Compute this dive's surface pressure offset from the surface time-series data.
- int DecodeMailMessageData (shared_ptr< XFloatMailMessage > &msg)

  Decode the SBD data of a mail message into this dive data object.
- void DecodeXMsgData (const vector< uint8_t > &data, shared_ptr< vector< uint8_t > > xmsg_id_↩ types=shared_ptr< vector< uint8_t > >()) ATTR_NO_EXCEPTIONS

  Decode an X-Message from the binary message data.
- string GenerateListingString () const

  Generate a listing entry string representation of this dive data object.
- string GeneratePhyString () const

  Generate a PHY entry string representation of this dive data object.
- string **GeneratePressureCorrectionListingString** () const
- vector< shared_ptr< XFloatMailMessage > > GetDiveDataMailMessagesFromDatabase ()

  Get this dive's stored mail messages from the SQL database.
- void GetGainsAndOffsets (uint16_t &pg, uint16_t &po, uint16_t &tg, uint16_t &to, uint16_t &sg, uint16_t &so) const

  Get the pressure, temperature, and salinity sensor gains and offsets.
- shared_ptr< x::XGPS > GetGpsPositionEndOfDive () const

  Get the end-of-dive GPS message.
- shared_ptr< x::XGPS > **GetGpsPositionStartOfDive** () const
- shared_ptr< x::XGPS > **GetGpsProfilePosition** () const
- void GetSurfaceGainsAndOffsets (uint16_t &pg, uint16_t &po, uint16_t &tg, uint16_t &to, uint16_t &sg, uint16_t &so) const

  Get the pressure, temperature, and salinity sensor gains and offsets for surface data.
- bool **GpsPositionIsValidProfilePosition** (const shared_ptr< x::XGPS > x) const
- bool HasBinnedProfileData () const

  Does this dive data object have any binned profile data?
- bool **HasBinnedProfileSalinityData** () const
- bool HasCompleteBinnedProfileData () const

  Does this dive have complete binned profile data?
- bool HasCompleteProfileData () const

  Does this dive have complete binned or unbinned profile data?
- bool HasCompleteUnbinnedProfileData () const

  Does this dive have complete unbinned profile data?
- bool HasContiguousMomsns () const

  Do the messages that comprise this dive data object have contiguously increasing MOMSN numbers?
- bool HasContiguousPids () const

  Do the messages that comprise this dive data object have contiguously increasing packet index identifiers (PIDs) starting with zero?
- bool HasDiveData () const

  Does this dive data object have any dive data?
- bool HasDriftData () const

  Does this dive data object have any drift data?
- bool HasEngineeringData () const

  Does this dive data object have any engineering data?
- bool HasFallRiseData () const

  Does this dive data object have any rise or fall data?
- bool HasPositionData () const

  Does this dive data object have any position data?
- bool **HasProfileSalinityData** () const
- bool HasUnbinnedProfileData () const

  Does this dive data object have any unbinned profile data?

- bool **HasUnbinnedProfileSalinityData** () const
- bool HasValidGpsPosition () const

  Does this dive data object have a valid GPS position for this dive's profile data?
- bool HasValidIridiumPosition () const

  Does this dive data object have a valid Iridium position?
- bool HasValidPosition () const

  Does this dive data object have a valid position for this dive's profile data?
- bool **HasWaveData** () const
- int id_dac () const

  Get the unique Data Assembly Center (DAC) identifier.
- int id_dive () const

  Get the dive or cycle number of this dive data object.
- long id_imei () const

  Get the unique International Mobile Equipment Identifier (IMEI).
- int id_instrument () const

  Get the unique instrument identifier or instrument serial number.
- int id_wmo () const

  Get the unique World Meteorological Organization (WMO) identifier.
- const path & intermediate_listing_file_path () const

  Get the path to the intermediate listing file on the local filesystem.
- const ptime & **iridium_position_datetime_first** () const
- const ptime & **iridium_position_datetime_last** () const
- bool **is_complete** () const
- bool **is_listing_file_updated** () const
- bool **is_netcdf_file_updated** () const
- bool **is_phy_file_updated** () const
- bool **is_pres_offs_valid** () const
- bool **is_surface_pres_valid** () const
- bool LockObject (bool block)

  Lock the object.
- const path & **netcdf_file_path** () const
- uint16_t **num_sbd_msgs** () const
- shared_ptr< XFloat > **parent_instrument** () const
- const path & phy_file_path () const

  Get the path to the PHY listing file on the local filesystem.
- float **pres_offs** () const
- int **pres_offs_num_dives** () const
- int **pres_offs_num_samples** () const
- void ResetData ()
- void SaveObjectToDatabase () const

  Save this dive data object to the SQL database.
- bool SensorGainsAndOffsetsAreKnown () const

  Are the sensor gains and offsets known for this dive data object.
- float **surface_pres_max** () const
- float **surface_pres_mean** () const
- float **surface_pres_min** () const
- int **surface_pres_num_samples** () const
- float **surface_pres_variance** () const
- void UnlockObject ()

  Unlock the object.
- void WriteListingFile ()

  Write the listing file for this dive.
- void WritePhyFile ()

  Write the PHY file for this dive.

Static Public Member Functions

- static void DeleteFromDatabase (int id_instrument)

  Delete all dive data for an instrument from the SQL database.
- static void DeleteObjectsFromDatabase (const string &where)

  Delete dive data rows from the SQL database table.
- static string GenerateReadableHex (const vector< uint8_t > &data)

  Generate a human-readable hex string.
- static shared_ptr< XFloatDiveData > GetDiveDataObjectFromDatabase (const shared_ptr< XFloat > &x, int dive)

  Get a dive data object for a given instrument and dive number from the database, if it exists.
- static shared_ptr< XFloatDiveData > SelectObjectFromDatabase (const string &where="", const string &groupBy="", const string &orderBy="", const unsigned int &offset=0)

  Select a single dive data object from the SQL database table.
- static vector< shared_ptr< XFloatDiveData > > SelectObjectsFromDatabase (const string &where="", const string &groupBy="", const string &orderBy="", const unsigned int &limit=0, const unsigned int &offset=0)

  Select dive data objects from the SQL database table.
- template<typename TT >
  static TT SelectValueFromDatabase (const string &what, const string &where="", const string &groupBy="", const string &orderBy="", const unsigned int &offset=0)

  Select a single value from the dive data SQL database table.

Protected Member Functions

- void set_id_dac (int id_dac)

  Set the unique Data Assembly Center (DAC) identifier.
- void set_id_imei (long id_imei)

  Set the unique International Mobile Equipment Identifier (IMEI).
- void set_id_instrument (int id_instrument)

  Set the unique instrument identifier or instrument serial number.
- void set_id_wmo (int id_wmo)

  Set the unique World Meteorological Organization (WMO) identifier.

Static Protected Member Functions

- static void CloseSqlConnection ()

  Close the thread-specific MySQL connection if it is currently open.
- static shared_ptr< sql::ResultSet > **executeDeleteQuery** (const string &from, const string &where="")
- static shared_ptr< ResultSet > ExecuteQuery (Connection *con, const string &query)

  Execute an SQL query and return a shared pointer to the ResultSet object.
- static shared_ptr< sql::ResultSet > executeQuery (const string &query)

  Execute an SQL query and return a shared pointer to the ResultSet object.
- static shared_ptr< sql::ResultSet > executeSelectQuery (const string &what, const string &from, const string &where="", const string &groupBy="", const string &orderBy="", unsigned int limit=0, unsigned int offset=0)

  Execute an SQL select query and return a shared pointer to the ResultSet object.
- static int ExecuteUpdate (Connection *con, const string &update)

  Execute an SQL update and return the status code.
- static int executeUpdate (const string &update)

  Execute an SQL update and return the status code.
- static string GenerateCurrentUtcTimeIso8601String ()

  Generate an ISO8601 format date/time string representation of the current time, in UTC.
- static string GenerateIso8601String (const ptime &p, bool extended=true)

  Generate an ISO8601 format date/time string representation of a POSIX time object.
- static double GenerateJulianDay (const ptime &dt, const ptime &ref_dt)

  Convert a date/time to fractional days since a reference date/time.
- static shared_ptr< PreparedStatement > GeneratePreparedStatement (Connection *con, const string &statement)

  Generate a prepared SQL statement object for the given SQL statement.
- static shared_ptr< sql::PreparedStatement > generatePreparedStatement (const string &statement)

  Generate a prepared SQL statement object for the given SQL statement.
- static string GenerateSqlDeleteStr (const string &from, const string &where="")

  Generate an SQL DELETE statement.

- static string GenerateSqlSelectStr (const string &what, const string &from, const string &where="", const string &groupBy="", const string &orderBy="", unsigned int limit=0, unsigned int offset=0)

  Generate an SQL select statement.
- static ptime GetCurrentUtcTime ()

  Generate a POSIX time object representing the current date and time (rounded to the nearest second).
- static const path & GetDirectoryPath (const path &dir, bool create=false)

  Get a directory path.
- static path GetDirectoryPathFromString (const string &dir, bool create=false)

  Generate a boost::filesystem::path object from the directory path string.
- static thread_specific_ptr< Connection > & GetSqlConnection (string url, string user, string password, string schema)

  Get a thread-specific instance of a MySQL connection object.
- static sql::Connection * getSqlConnection ()

  Generate a thread-specific instance of a MySQL connection object.
- static thread_specific_ptr< MySQL_Driver > & GetSqlDriver ()

  Get a thread-specific instance of a MySQL driver object.
- static vector< path > ListDirectories (const path &dir)

  List all directories contained in the given directory.
- static vector< path > ListFiles (path dir, size_t n=0)

  List all regular files contained in the given directory.
- static path MoveFileToDirectory (const path &file, const path &dir_dst, bool create_dir=false)

  Copy a file into a directory.

Protected Attributes

- recursive_mutex mutex_

  Recursive mutual exclusion lock for synchronizing operations.

Private Member Functions

- void AssignArgoData (const shared_ptr< x::XArgo > &x)

  Assign an Argo data message to this dive data object.
- void AssignAuxTemperatureBinned (const shared_ptr< x::XProfileTemperature > &x)

  Assign a binned auxiliary-channel profile temperature message to this dive data object.
- void AssignDriftPressure (const shared_ptr< x::XDriftPressure > &x)

  Assign a drift pressure message to this dive data object.
- void AssignDriftSalinity (const shared_ptr< x::XDriftSalinity > &x)

  Assign a drift salinity message to this dive data object.
- void AssignDriftTemperatre (const shared_ptr< x::XDriftTemperature > &x)

  Assign a drift temperature message to this dive data object.
- void AssignEngineeringAbort (const shared_ptr< x::XEngAbort > &x)

  Assign an engineering abort/beacon message to this dive data object.
- void AssignEngineeringAscent (const shared_ptr< x::XEngAscent > &x)

  Assign an engineering ascent message to this dive data object.
- void AssignEngineeringBitFail (const shared_ptr< x::XEngBITFail > &x)

  Assign an engineering BIT failed message to this dive data object.
- void AssignEngineeringBitPass (const shared_ptr< x::XEngBITPass > &x)

  Assign an engineering BIT passed message to this dive data object.
- void AssignEngineeringDescent (const shared_ptr< x::XEngDescent > &x)

  Assign an engineering descent message to this dive data object.
- void AssignEngineeringDiagnostic (const shared_ptr< x::XEngDiagnostic > &x)

  Assign an engineering diagnostic dive message to this dive data object.
- void AssignEngineeringDrift (const shared_ptr< x::XEngDrift > &x)

  Assign an engineering drift/park message to this dive data object.
- void AssignEngineeringIce (const shared_ptr< x::XEngIce > &x)

  Assign an engineering (ice) message to this dive data object.
- void AssignEngineeringStandard (const shared_ptr< x::XEngStandard > &x)

  Assign an engineering standard dive message to this dive data object.
- void AssignEngineeringSurface (const shared_ptr< x::XEngSurface > &x)

  Assign an engineering surface message to this dive data object.
- void AssignFallTimeSeries (const shared_ptr< x::XFall > &x)

  Assign a fall time-series message to this dive data object.

- void AssignGps (shared_ptr< x::XGPS > &x)

  Assign a GPS message to this dive data object.
- int AssignMailMessage (const shared_ptr< XFloatMailMessage > &msg)

  Assign a mail message to this dive data object.
- void AssignMissionCmdEchoback (const shared_ptr< x::XMissionCmdEcho > &x)

  Assign a ground-station command echo-back message to this dive data object.
- void AssignParentInstrument (const shared_ptr< XFloat > &flt)

  Set the shared pointer to the parent float object that owns this dive data.
- void AssignProfilePressureBinned (const shared_ptr< x::XProfilePressure > &x)

  Assign a bin-averaged profile pressure message to this dive data object.
- void AssignProfilePressureUnbinned (const shared_ptr< x::XHiResPressure > &x)

  Assign an unbinned profile pressure message to this dive data object.
- void AssignProfileSalinityBinned (const shared_ptr< x::XProfileSalinity > &x)

  Assign a bin-averaged profile salinity message to this dive data object.
- void AssignProfileSalinityUnbinned (const shared_ptr< x::XHiResSalinity > &x)

  Assign an unbinned profile salinity message to this dive data object.
- void AssignProfileTemperatureBinned (const shared_ptr< x::XProfileTemperature > &x)

  Assign a bin-averaged profile temperature message to this dive data object.
- void AssignProfileTemperatureUnbinned (const shared_ptr< x::XHiResTemperature > &x)

  Assign an unbinned profile temperature message to this dive data object.
- void AssignPumpTimeSeries (const shared_ptr< x::XPump > &x)

  Assign a pump time-series message to this dive data object.
- void AssignRiseTimeSeries (const shared_ptr< x::XRise > &x)

  Assign a rise time-series message to this dive data object.
- void AssignSurfaceTimeSeries (const shared_ptr< x::XSurfaceData > &x)

  Assign a surface pressure time-series message to this dive data object.
- void AssignValveData (const shared_ptr< x::XValve > &x)

  Assign a valve data message to this dive data object.
- shared_ptr< x::XCompositeFall > GenerateCompositeFallTimeSeriesData () const

  Generate a composite fall time-series data for this dive.
- shared_ptr< x::XCompositeIridiumPosition > GenerateCompositeIridiumPosition () const

  Generate a composite Iridium position for this dive.
- shared_ptr< x::XCompositePump > GenerateCompositePumpTimeSeriesData () const

  Get the composite pump time-series data for this dive.
- shared_ptr< x::XCompositeRise > GenerateCompositeRiseTimeSeriesData () const

  Get the composite rise time-series data for this dive.
- shared_ptr< x::XCompositeValve > **GenerateCompositeValveData** () const
- vector< int > GenerateOrderedMomsnList () const

  Generate a list of ordered Iridium message MOMSNs for this dive.
- vector< int > GenerateOrderedPidList () const

  Generate a list of ordered Iridium message PIDs for this dive.
- void set_argo_instrument_type (XArgoInstrumentType_t type)

  Set the enumerated Argo instrument type identifier.
- void **set_complete** (bool complete)
- void set_id_dive (int id_dive)

  Set the dive or cycle number of this dive data object.
- void set_intermediate_listing_file_path (const path &file_path)

  Set the path to the intermediate listing file.
- void **set_iridium_position_datetime_first** (const ptime &iridium_position_datetime_first)
- void **set_iridium_position_datetime_last** (const ptime &iridium_position_datetime_last)
- void **set_listing_file_updated** (bool listing_file_updated)
- void **set_netcdf_file_path** (const path &netcdf_file_path)
- void **set_netcdf_file_updated** (bool netcdf_file_updated=true)
- void **set_num_sbd_msgs** (uint16_t num_sbd_msgs)
- void set_phy_file_path (const path &file_path)

  Set the path to the PHY listing file.
- void **set_phy_file_updated** (bool phy_file_updated)
- void **set_pres_offs** (float pres_offs)
- void **set_pres_offs_num_dives** (int pres_offs_num_dives)
- void **set_pres_offs_num_samples** (int pres_offs_num_samples)
- void **set_pres_offs_valid** (bool pres_offs_valid)
- void **set_surface_pres_max** (float surface_pres_max)
- void **set_surface_pres_mean** (float surface_pres_mean)
- void **set_surface_pres_min** (float surface_pres_min)

- void **set_surface_pres_num_samples** (int surface_pres_num_samples)
- void **set_surface_pres_valid** (bool surface_pres_valid)
- void **set_surface_pres_variance** (float surface_pres_variance)
- void UpdateDerivedSurfaceDataFields ()

  Update the derived surface pressure fields from the surface time-series data.

Static Private Member Functions

- static void CleanupSqlConnection (Connection *con)

  Cleanup an SQL connection.
- static void CleanupSqlDriver (MySQL_Driver *driver)

  Function for memory cleanup of MySQL driver on destruct.

Private Attributes

- XArgoInstrumentType_t argo_instrument_type_

  The enumerated Argo instrument type identifier.
- bool complete_

  The dive-data-is-complete flag.
- int id_dac_

  The unique Data Assembly Center (DAC) identifier.
- int id_dive_

  The dive or cycle number of this dive data object.
- long id_imei_

  The unique International Mobile Equipment Identifier (IMEI).
- int id_instrument_

  The unique instrument identifier or instrument serial number.
- int id_wmo_

  The unique World Meteorological Organization (WMO) identifier.
- ptime iridium_position_datetime_first_

  The date/time of transmission of the first Iridium message sent by the instrument for this dive.
- ptime iridium_position_datetime_last_

  The date/time of transmission of the last Iridium message sent by the instrument for this dive.
- path listing_file_path_

  The path to the intermediate listing file for this dive data on the local filesystem.
- bool listing_file_updated_

  The listing-file-is-updated flag.
- static_vector< shared_ptr< XFloatMailMessage >, XMSG_MAX_MAIL > mail_messages_

  The mail messages for this dive data object.
- path netcdf_file_path_

  The NetCDF dimensions.
- bool netcdf_file_updated_

  The NetCDF-file-is-updated flag.
- uint16_t num_sbd_msgs_

  The number of SBD messages that have been received for this dive cycle.
- shared_ptr< XFloat > parent_instrument_

  The parent float or instrument object of this dive data object.
- path phy_file_path_

  The path to the PHY file for this dive data on the local filesystem.
- bool phy_file_updated_

  The phy-file-is-updated flag.
- float pres_offs_

  The computed surface pressure offset for this dive.
- int pres_offs_num_dives_

  The number of dives used to compute the surface pressure offset.
- int pres_offs_num_samples_

  The number of samples used to compute the surface pressure offset.
- bool pres_offs_valid_

  The surface-pressure offset-is-valid flag.
- float surface_pres_max_

  The maximum reported surface pressure.
- float surface_pres_mean_

The mean surface pressure.

- float surface_pres_min_

  The minimum reported surface pressure.

- int surface_pres_num_samples_

  The number of surface pressure samples reported.

- bool surface_pres_valid_

  The surface-pressure-is-valid flag.

- float surface_pres_variance_

  The surface pressure variance.

- shared_ptr< x::XAcousticPosition > xmsg_acoustic_positions_

  A vector of pointers to the acoustic position messages.

- shared_ptr< x::XArgo > xmsg_argo_data_

  A pointer to the Argo data message.

- x::XCompositeDrift xmsg_composite_drift_data_

  The composite drift data object.

- x::XCompositeProfile xmsg_composite_profile_binned_

  The composite binned profile data object.

- x::XCompositeHiResProfile xmsg_composite_profile_unbinned_

  The composite unbinned profile data object.

- shared_ptr< x::XEngAbort > xmsg_engineering_data_abort_

  A pointer to the engineering abort message.

- shared_ptr< x::XEngAscent > xmsg_engineering_data_ascent_

  A pointer the the engineering ascent message.

- shared_ptr< x::XEngBITFail > xmsg_engineering_data_bit_fail_

  A pointer to the engineering BIT failed message.

- shared_ptr< x::XEngBITPass > xmsg_engineering_data_bit_passed_

  A pointer to the engineering BIT passed message.

- shared_ptr< x::XEngDescent > xmsg_engineering_data_descent_

  A pointer to the engineering descent message.

- shared_ptr< x::XEngDiagnostic > xmsg_engineering_data_diagnostic_dive_

  A pointer to the engineering diagnostic or startup message.

- shared_ptr< x::XEngDrift > xmsg_engineering_data_drift_

  A pointer to the engineering drift message.

- shared_ptr< x::XEngIce > xmsg_engineering_data_ice_

  A pointer to the ice-detection engineering message.

- shared_ptr< x::XEngStandard > xmsg_engineering_data_standard_dive_

  A pointer to the standard dive-cycle engineering message.

- shared_ptr< x::XEngSurface > xmsg_engineering_data_surface_

  A pointer to the engineering surface message.

- static_vector< shared_ptr< x::XFall >, XMSG_MAX_FALL > xmsg_fall_time_series_

  A static vector of pointers to the fall time-series messages.

- static_vector< shared_ptr< x::XGPS >, XMSG_MAX_GPS > xmsg_gps_positions_

  A static vector of pointers to the GPS position message(s).

- x::XMfaMissionCfg xmsg_mission_cfg_

  The MFA mission configruation object.

- vector< shared_ptr< x::XMissionCmdEcho > > xmsg_mission_cmd_echoback_

  A vector of pointer to mission configuration command upload echo-back messages.

- static_vector< shared_ptr< x::XPump >, XMSG_MAX_PUMP > xmsg_pump_time_series_

  A static vector of pointers to the pump time-series messages.

- static_vector< shared_ptr< x::XRise >, XMSG_MAX_RISE > xmsg_rise_time_series_

  A static vector of pointers to the fall time-series messages.

- x::XCompositeSurfaceData xmsg_surface_time_series_

  The surface pressure time-series object.

- static_vector< shared_ptr< x::XValve >, XMSG_MAX_VALVE > xmsg_valve_data_

  Valve operation messages.

- shared_ptr< x::XWave > xmsg_wave_field_

  A static vector of pointers to the surface wave field data messages.

Static Private Attributes

- static string address_
  The SQL server qualified name or IP address string.
- static path intermediate_listing_directory_path_
  The intermediate listing directory path on the local filesystem.
- static bool intermediate_listing_file_generation_is_enabled_
  The global intermediate-listing-file-generation-is-enabled flag.
- static path netcdf_directory_path_
  The NetCDF directory path on the local filesystem.
- static bool netcdf_file_generation_is_enabled_
  The global NetCDF-file-generation-is-enabled flag.
- static const ptime netcdf_reference_datetime_
  The NetCDF referecne date/time.
- static string password_
  The SQL server account password string.
- static path phy_directory_path_
  The PHY directory path on the local filesystem.
- static bool phy_file_generation_is_enabled_
  The global PHY-file-generation-is-enabled flag.
- static unsigned int port_
  The SQL server port number.
- static bool pressure_offset_correction_enabled_list_file_
  The global surface-pressure-offset-correction-is-enabled-in-intermediate-listing-files flag.
- static bool pressure_offset_correction_enabled_netcdf_file_
  The global surface-pressure-offset-correction-is-enabled-in-NetCDF-files flag.
- static bool pressure_offset_correction_enabled_phy_file_
  The global surface-pressure-offset-correction-is-enabled-in-PHY-files flag.
- static string protocol_
  The protocol to use in the SQL server connection e.g.
- static string schema_
  The SQL server schema name.
- static thread_specific_ptr< Connection > sql_connection_
  Tthe thread-specific pointer to MySQL connection object.
- static thread_specific_ptr< MySQL_Driver > sql_driver_
  The thread-specific pointer to MySQL driver object.
- static const util::mysql::SqlTableDescription< XFloatDiveData > sql_table_description_
  The SQL table description object.
- static string user_
  The SQL server account username string.

Friends

- shared_ptr< XFloatDiveData > std::make_shared ()
  Friend function for creating an X-Float object using the private constructor and a shared pointer to that object.
- options_description util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription ()
  Friend function for populating some of the static data members of this class from the configuration file.
- class **XFloat**
- class **XFloatTelemetryDecoderService**

10.87.1 Detailed Description

The instrument or float dive data class.

This class contains all of the mail messages and corresponding decoded telemetry data for an instrument for a single dive. Each time a mail message is received by the telemetry decoder thread, XFloatTelemetryDecoderService, the corresponding dive data object is retrieved and the mail message is assigned to it. Then, the binary data from the attached Iridium Short Burst Data (SBD) message is decoded by calling DecodeXMsgData(). As a result of decoding the binary message data, an object corresponding to the message type is created and its field are populated from the decoded data in the binary message. The newly created object will be of a type that inherits from util::xmsg::XMessage. A shared reference to the object is assigned to the dive data object.

This class contains the functionality for writing the formatted intermediate and PHY listing files to the local filesystem. Each time a new message is received, the entire intermediate listing file will be written to the local filesystem, overwriting the previous listing file, if it exists. The PHY file is written only when the dive data object is determined to contain the complete data for dive, and only if the instrument has been deployed i.e. it has a metadata file with valid DAC and WMO identifiers.

Note

This object is lockable. It contains a recursive mutual exclusion (mutex) lock in order to synchronize access to and modification of objects of this type by various worker threads. Once the object is locked, any other thread that attempts to access or modify the object will block until the initial lock is released. Since the mutex is recursive, the object may be locked multiple times by the same thread without causing the thread to block indefinitely.

This class is mapped to an SQL database table by the sql_table_description_ object. This allows some or all of the field of this class to be persisted to the SQL database. Likewise, an object of this type may be constructed from the results of an SQL query to the corresponding table.

Todo Devise a scheme to ensure that the dive data objects do not remain in memory indefinitely if one or more message for the dive or cycle are not received and the dive data object is not completed.

10.87.2 Constructor & Destructor Documentation

10.87.2.1 XFloatDiveData() [1/2]

```
argo::iridium::xfloat::XFloatDiveData::XFloatDiveData ()
```

The default constructor.

Exceptions

None.	Never throws.

10.87.2.2 XFloatDiveData() [2/2]

```
argo::iridium::xfloat::XFloatDiveData::XFloatDiveData (
            const shared_ptr< XFloat > & parent,
            int dive)
```

Parameterized constructor.

Parameters

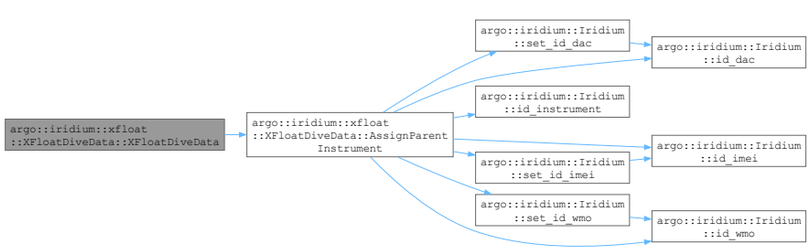
in	*parent*	A pointer to the float object that owns this dive data.
in	*dive*	The cycle or dive number of this dive.

Exceptions

util::log::MessageException	if an error occurs during construction, or if xFloat is NULL.

References AssignParentInstrument(), CATCH_BLOCK, and EXC_THROW.

Here is the call graph for this function:



10.87.2.3 ~XFloatDiveData()

```
argo::iridium::xfloat::XFloatDiveData::~XFloatDiveData ()
```

The destructor.

10.87.3 Member Function Documentation

10.87.3.1 argo_instrument_type()

```
XArgoInstrumentType_t argo::iridium::xfloat::XFloatDiveData::argo_instrument_type () const  [inline]
```

Get the enumerated Argo instrument type identifier.

Returns

The enumerated Argo instrument type identifier.

References argo_instrument_type_, and util::thread::RecursiveLockable::mutex_.

10.87.3.2 AssignArgoData()

```
void argo::iridium::xfloat::XFloatDiveData::AssignArgoData (
            const shared_ptr< x::XArgo > & x)  [private]
```

Assign an Argo data message to this dive data object.

Parameters

in	*x*	A pointer to the Argo data message to assign.

References util::thread::RecursiveLockable::mutex_, parent_instrument_, and xmsg_argo_data_.

10.87.3.3 AssignAuxTemperatureBinned()

```
void argo::iridium::xfloat::XFloatDiveData::AssignAuxTemperatureBinned (
            const shared_ptr< x::XProfileTemperature > & x)  [inline], [private]
```

Assign a binned auxiliary-channel profile temperature message to this dive data object.

Parameters

in	*x*	A pointer to the auxiliary-channel profile temperature message.

Exceptions

util::log::MessageException	if a message with the same index has already been assigned.

References util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignAuxTem
util::thread::RecursiveLockable::mutex_, and xmsg_composite_profile_binned_.

Here is the call graph for this function:

10.87.3.4 AssignDriftPressure()

```
void argo::iridium::xfloat::XFloatDiveData::AssignDriftPressure (
            const shared_ptr< x::XDriftPressure > & x)  [inline], [private]
```

Assign a drift pressure message to this dive data object.

Note

This vector is sorted upon insertion by order of non-decreasing message indices.

Parameters

in	*x*	A pointer to the drift pressure message to assign.

References util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignPressu
util::thread::RecursiveLockable::mutex_, and xmsg_composite_drift_data_.

Here is the call graph for this function:

10.87.3.5 AssignDriftSalinity()

```
void argo::iridium::xfloat::XFloatDiveData::AssignDriftSalinity (
            const shared_ptr< x::XDriftSalinity > & x)  [inline], [private]
```

Assign a drift salinity message to this dive data object.

Note

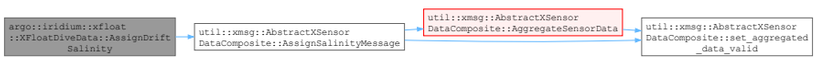
This vector is sorted upon insertion by order of non-decreasing message indices.

Parameters

in	*x*	A pointer to the drift salinity message to assign.

References util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignSalinity
util::thread::RecursiveLockable::mutex_, and xmsg_composite_drift_data_.

Here is the call graph for this function:



10.87.3.6 AssignDriftTemperatre()

```
void argo::iridium::xfloat::XFloatDiveData::AssignDriftTemperatre (
            const shared_ptr< x::XDriftTemperature > & x)  [inline], [private]
```

Assign a drift temperature message to this dive data object.

Note

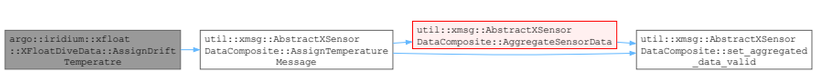
This vector is sorted upon insertion by order of non-decreasing message indices.

Parameters

in	*x*	A pointer to the drift temperature message to assign.

References util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignTemper
util::thread::RecursiveLockable::mutex_, and xmsg_composite_drift_data_.

Here is the call graph for this function:



10.87.3.7 AssignEngineeringAbort()

```
void argo::iridium::xfloat::XFloatDiveData::AssignEngineeringAbort (
            const shared_ptr< x::XEngAbort > & x)  [inline], [private]
```

Assign an engineering abort/beacon message to this dive data object.

Parameters

in	*x*	A pointer to the engineering abort/beacon message to assign.

References util::thread::RecursiveLockable::mutex_, and xmsg_engineering_data_abort_.

10.87.3.8 AssignEngineeringAscent()

```
void argo::iridium::xfloat::XFloatDiveData::AssignEngineeringAscent (
            const shared_ptr< x::XEngAscent > & x)  [inline], [private]
```

Assign an engineering ascent message to this dive data object.

Note

This message type is used by ALAMO floats only.

Parameters

in	*x*	A pointer to the XXengineering ascentX message to assign.

References util::thread::RecursiveLockable::mutex_, and xmsg_engineering_data_ascent_.

10.87.3.9 AssignEngineeringBitFail()

```
void argo::iridium::xfloat::XFloatDiveData::AssignEngineeringBitFail (
            const shared_ptr< x::XEngBITFail > & x)  [inline], [private]
```

Assign an engineering BIT failed message to this dive data object.

Parameters

in	*x*	A pointer to the engineering BIT failed message to assign.

References util::thread::RecursiveLockable::mutex_, and xmsg_engineering_data_bit_fail_.

10.87.3.10 AssignEngineeringBitPass()

```
void argo::iridium::xfloat::XFloatDiveData::AssignEngineeringBitPass (
            const shared_ptr< x::XEngBITPass > & x)  [inline], [private]
```

Assign an engineering BIT passed message to this dive data object.

Parameters

in	*x*	A pointer to the engineering BIT passed message to assign.

References util::thread::RecursiveLockable::mutex_, and xmsg_engineering_data_bit_passed_.

10.87.3.11 AssignEngineeringDescent()

```
void argo::iridium::xfloat::XFloatDiveData::AssignEngineeringDescent (
            const shared_ptr< x::XEngDescent > & x)  [inline], [private]
```

Assign an engineering descent message to this dive data object.

Note

This message type is used by ALAMO floats only.

Parameters

in	*x*	A pointer to the engineering descent message to assign.

References util::thread::RecursiveLockable::mutex_, and xmsg_engineering_data_descent_.

10.87.3.12 AssignEngineeringDiagnostic()

```
void argo::iridium::xfloat::XFloatDiveData::AssignEngineeringDiagnostic (
            const shared_ptr< x::XEngDiagnostic > & x)  [inline], [private]
```

Assign an engineering diagnostic dive message to this dive data object.

Parameters

in	*x*	A pointer to the engineering diagnostic dive message to assign.

References util::thread::RecursiveLockable::mutex_, and xmsg_engineering_data_diagnostic_dive_.

10.87.3.13 AssignEngineeringDrift()

```
void argo::iridium::xfloat::XFloatDiveData::AssignEngineeringDrift (
            const shared_ptr< x::XEngDrift > & x)  [inline], [private]
```

Assign an engineering drift/park message to this dive data object.

Note

This message type is used by ALAMO floats only.

Parameters

in	*x*	A pointer to the engineering drift/park message to assign.

References util::thread::RecursiveLockable::mutex_, and xmsg_engineering_data_drift_.

10.87.3.14 AssignEngineeringIce()

```
void argo::iridium::xfloat::XFloatDiveData::AssignEngineeringIce (
            const shared_ptr< x::XEngIce > & x)  [inline], [private]
```

Assign an engineering (ice) message to this dive data object.

Replaces any previously assigned engineering (ice) message.

Parameters

in	*x*	A pointer to the engineering (ice) message to assign.

References util::thread::RecursiveLockable::mutex_, and xmsg_engineering_data_ice_.

10.87.3.15 AssignEngineeringStandard()

```
void argo::iridium::xfloat::XFloatDiveData::AssignEngineeringStandard (
            const shared_ptr< x::XEngStandard > & x)  [inline], [private]
```

Assign an engineering standard dive message to this dive data object.

Parameters

in	*x*	A pointer to the engineering standard dive message to assign.

References util::thread::RecursiveLockable::mutex_, and xmsg_engineering_data_standard_dive_.

10.87.3.16 AssignEngineeringSurface()

```
void argo::iridium::xfloat::XFloatDiveData::AssignEngineeringSurface (
            const shared_ptr< x::XEngSurface > & x)  [inline], [private]
```

Assign an engineering surface message to this dive data object.

Note

This message type is used by ALAMO floats only.

Parameters

in	*x*	A pointer to the engineering surface message to assign.

References util::thread::RecursiveLockable::mutex_, and xmsg_engineering_data_surface_.

10.87.3.17 AssignFallTimeSeries()

```
void argo::iridium::xfloat::XFloatDiveData::AssignFallTimeSeries (
            const shared_ptr< x::XFall > & x)  [inline], [private]
```

Assign a fall time-series message to this dive data object.

Note

This vector is sorted upon insertion by order of non-decreasing message indices.

Parameters

in	*x*	A pointer to the fall time-series message to assign.

References EXC_THROW, util::thread::RecursiveLockable::mutex_, and xmsg_fall_time_series_.

10.87.3.18 AssignGps()

```
void argo::iridium::xfloat::XFloatDiveData::AssignGps (
            shared_ptr< x::XGPS > & x)  [private]
```

Assign a GPS message to this dive data object.

Note

If the GPS message is the startup/BIT GPS message, then the position date/time will be used to set the parent start-of-mission time.

Parameters

in	*x*	The GPS message to set.

References CATCH_BLOCK, CATCH_NOTHROW, EXC_THROW, util::date::DateUtils::GetCurrentUtcTime(), util::thread::RecursiveLockable::mutex_, XFLT_POS_INVALID_LAT, XFLT_POS_INVALID_LON, and xmsg_gps_positions_.

Here is the call graph for this function:

10.87.3.19 AssignMailMessage()

```
int argo::iridium::xfloat::XFloatDiveData::AssignMailMessage (
            const shared_ptr< XFloatMailMessage > & x)  [private]
```

Assign a mail message to this dive data object.

Exceptions

util::log::MessageException	if !x or an error occurs.

Note

This vector is sorted upon insertion by order of non-decreasing message indices.

Parameters

in	*x*	The mail message to assign.

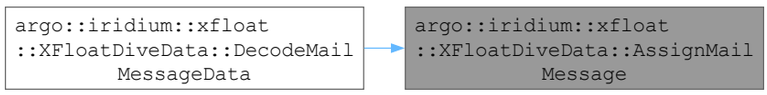
Returns

0 on successful assignment of the mail message to this dive data object, 1 if a duplicate mail message UID is encountered, or 2 if the mail message was assigned but is a retransmission of an already assigned message (identical SBD data), so its SBD data should not be decoded again.

References CATCH_BLOCK, EXC_THROW, ldbg, mail_messages_, util::thread::RecursiveLockable::mutex_, and num_sbd_msgs_.

Referenced by DecodeMailMessageData().

Here is the caller graph for this function:



10.87.3.20 AssignMissionCmdEchoback()

```
void argo::iridium::xfloat::XFloatDiveData::AssignMissionCmdEchoback (
            const shared_ptr< x::XMissionCmdEcho > & x)  [inline], [private]
```

Assign a ground-station command echo-back message to this dive data object.

Parameters

in	*x*	A pointer to the ground-station command echo-back message to assign.

References util::thread::RecursiveLockable::mutex_, and xmsg_mission_cmd_echoback_.

10.87.3.21 AssignParentInstrument()

```
void argo::iridium::xfloat::XFloatDiveData::AssignParentInstrument (
            const shared_ptr< XFloat > & parent)  [private]
```

Set the shared pointer to the parent float object that owns this dive data.

Note

This dive data object will be locked for the scope of this function.

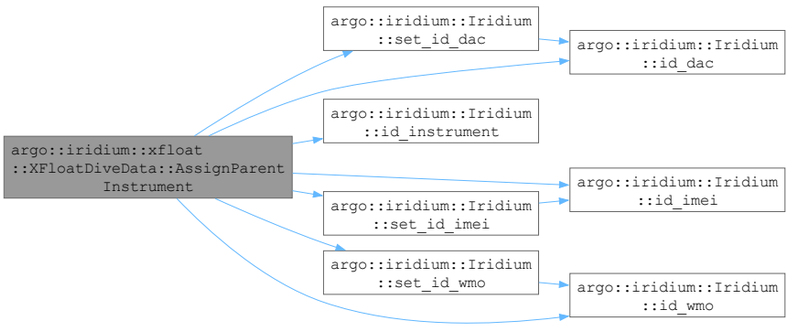
Parameters

in	*parent*	The shared pointer to the parent float object to set.

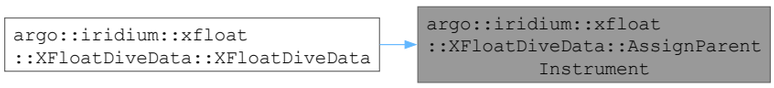
References CATCH_BLOCK, EXC_THROW, argo::iridium::Iridium::id_dac(), argo::iridium::Iridium::id_imei(), argo::iridium::Iridium::id_instrument(), argo::iridium::Iridium::id_wmo(), util::thread::RecursiveLockable::mutex_, parent_instrument_, argo::iridium::Iridium::set_id_dac(), argo::iridium::Iridium::set_id_imei(), argo::iridium::Iridium::set_id_wmo(), and XFLT_ID_UNKNOWN.

Referenced by XFloatDiveData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.87.3.22 AssignProfilePressureBinned()

```
void argo::iridium::xfloat::XFloatDiveData::AssignProfilePressureBinned (
            const shared_ptr< x::XProfilePressure > & x)  [inline], [private]
```

Assign a bin-averaged profile pressure message to this dive data object.

Note

This vector is sorted upon insertion by order of non-decreasing message indices.

Parameters

in	*x*	A pointer to the bin-averaged profile pressure message to assign.

References util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignPressu util::thread::RecursiveLockable::mutex_, and xmsg_composite_profile_binned_.

Here is the call graph for this function:

10.87.3.23 AssignProfilePressureUnbinned()

```
void argo::iridium::xfloat::XFloatDiveData::AssignProfilePressureUnbinned (
            const shared_ptr< x::XHiResPressure > & x)  [inline], [private]
```

Assign an unbinned profile pressure message to this dive data object.

Note

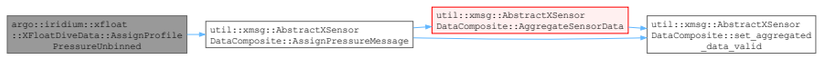
This vector is sorted upon insertion by order of non-decreasing message indices.

Parameters

in	*x*	A pointer to the unbinned profile pressure message to assign.

References util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignPressu
util::thread::RecursiveLockable::mutex_, and xmsg_composite_profile_unbinned_.

Here is the call graph for this function:



10.87.3.24 AssignProfileSalinityBinned()

```
void argo::iridium::xfloat::XFloatDiveData::AssignProfileSalinityBinned (
            const shared_ptr< x::XProfileSalinity > & x)  [inline], [private]
```

Assign a bin-averaged profile salinity message to this dive data object.

Note

This vector is sorted upon insertion by order of non-decreasing message indices.

Parameters

in	*x*	A pointer to the bin-averaged profile salinity message to assign.

References util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignSalinity
util::thread::RecursiveLockable::mutex_, and xmsg_composite_profile_binned_.

Here is the call graph for this function:

10.87.3.25 AssignProfileSalinityUnbinned()

```
void argo::iridium::xfloat::XFloatDiveData::AssignProfileSalinityUnbinned (
            const shared_ptr< x::XHiResSalinity > & x)  [inline], [private]
```

Assign an unbinned profile salinity message to this dive data object.

Note

This vector is sorted upon insertion by order of non-decreasing message indices.

Parameters

in	*x*	A pointer to the unbinned profile salinity message to assign.

References util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignSalinity
util::thread::RecursiveLockable::mutex_, and xmsg_composite_profile_unbinned_.

Here is the call graph for this function:

10.87.3.26 AssignProfileTemperatureBinned()

```
void argo::iridium::xfloat::XFloatDiveData::AssignProfileTemperatureBinned (
            const shared_ptr< x::XProfileTemperature > & x)  [inline], [private]
```

Assign a bin-averaged profile temperature message to this dive data object.

Note

This vector is sorted upon insertion by order of non-decreasing message indices.

Parameters

in	*x*	A pointer to the bin-averaged profile temperature message to assign.

References util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignTemper
util::thread::RecursiveLockable::mutex_, and xmsg_composite_profile_binned_.

Here is the call graph for this function:

10.87.3.27 AssignProfileTemperatureUnbinned()

```
void argo::iridium::xfloat::XFloatDiveData::AssignProfileTemperatureUnbinned (
            const shared_ptr< x::XHiResTemperature > & x)  [inline], [private]
```

Assign an unbinned profile temperature message to this dive data object.

Note

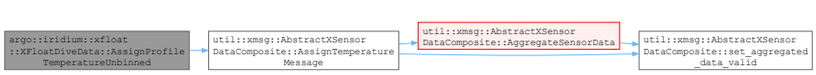
This vector is sorted upon insertion by order of non-decreasing message indices.

Parameters

in	*x*	A pointer to the unbinned profile temperature message to assign.

References util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignTemper
util::thread::RecursiveLockable::mutex_, and xmsg_composite_profile_unbinned_.

Here is the call graph for this function:



10.87.3.28 AssignPumpTimeSeries()

```
void argo::iridium::xfloat::XFloatDiveData::AssignPumpTimeSeries (
            const shared_ptr< x::XPump > & x)  [inline], [private]
```

Assign a pump time-series message to this dive data object.

Note

This vector is sorted upon insertion by order of non-decreasing message indices.

Parameters

in	*x*	A pointer to the pump time-series message to assign.

References EXC_THROW, util::thread::RecursiveLockable::mutex_, and xmsg_pump_time_series_.

10.87.3.29 AssignRiseTimeSeries()

```
void argo::iridium::xfloat::XFloatDiveData::AssignRiseTimeSeries (
            const shared_ptr< x::XRise > & x)  [inline], [private]
```

Assign a rise time-series message to this dive data object.

Note

This vector is sorted upon insertion by order of non-decreasing message indices.

Parameters

in	*x*	A pointer to the rise time-series message to assign.

References EXC_THROW, util::thread::RecursiveLockable::mutex_, and xmsg_rise_time_series_.

10.87.3.30 AssignSurfaceTimeSeries()

```
void argo::iridium::xfloat::XFloatDiveData::AssignSurfaceTimeSeries (
            const shared_ptr< x::XSurfaceData > & x)  [inline], [private]
```

Assign a surface pressure time-series message to this dive data object.

Note

This message type is used by ALAMO floats only.

This vector is sorted upon insertion by order of non-decreasing message indices.

Parameters

in	*x*	A pointer to the surface pressure time-series message to assign.

References util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::AssignMessage(), CATCH_BLOCK, util::thread::RecursiveLockable::mutex_, and xmsg_surface_time_series_.

Here is the call graph for this function:

10.87.3.31 AssignValveData()

```
void argo::iridium::xfloat::XFloatDiveData::AssignValveData (
            const shared_ptr< x::XValve > & x)  [inline], [private]
```

Assign a valve data message to this dive data object.

Note

The valve data messages are kept sorted by message index.

Parameters

in	*x*	A pointer to the valve data message to assign.

Exceptions

util::log::MessageException	if a message with the same index has already been assigned.

References EXC_THROW, util::thread::RecursiveLockable::mutex_, and xmsg_valve_data_.

10.87.3.32 CheckCompleteness()

```
bool argo::iridium::xfloat::XFloatDiveData::CheckCompleteness ()
```

Is this dive data object complete?

This function determines if the dive data is complete and sets the complete_ flag accordingly. The completeness of the dive data is determined by CheckCompleteness() which is invoked after each new mail message is decoded and assigned to this dive data object.

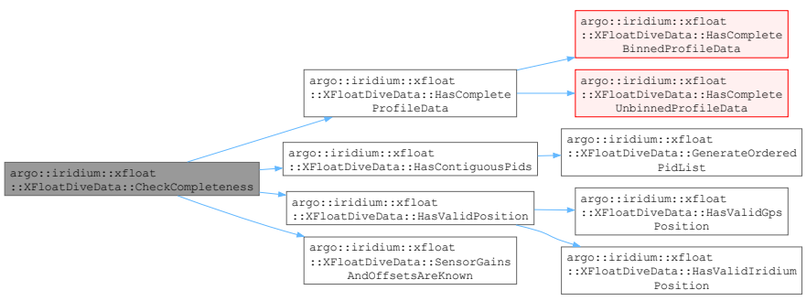
The dive data is determined to be complete if the dive data object meets all of the following criteria: it contains a complete set of profile data, the sensor gains and offsets for this dive are known (i.e. the Argo data message has been received), it contains a set of mail messages with contiguous packet identifiers (PIDs) starting at PID=0, and it contains a valid position for the profile. These criteria is determined by the functions HasCompleteProfileData(), SensorGainsAndOffsetsAreKnown(), HasContiguousPids(), HasValidPosition(), respectively.

Returns

TRUE if all data has been received for this dive, or false otherwise.

References CATCH_BLOCK, HasCompleteProfileData(), HasContiguousPids(), HasValidPosition(), util::thread::RecursiveLockable::mutex
and SensorGainsAndOffsetsAreKnown().

Here is the call graph for this function:



10.87.3.33 CleanupSqlConnection()

```
static void util::mysql::MysqlUtils::CleanupSqlConnection (
            Connection * con)  [inline], [static], [private], [inherited]
```

Cleanup an SQL connection.

This function for memory cleanup of the SQL connection on destruction. The thread-specific SQL connection object will be deleted if it exists.

Parameters

in	*con*	The SQL connection to cleanup.

Exceptions

log::MessageException	if an error occurs while cleaning up the SQL connection object.

References CATCH_BLOCK.

10.87.3.34 CleanupSqlDriver()

```
static void util::mysql::MysqlUtils::CleanupSqlDriver (
            MySQL_Driver * driver)  [inline], [static], [private], [inherited]
```

Function for memory cleanup of MySQL driver on destruct.

Signals the SQL driver to end operation, if it exists.

Parameters

in	*driver*	Driver to cleanup.

Exceptions

log::MessageException	if an error occurs while cleaning up the SQL driver object.

References CATCH_BLOCK.

10.87.3.35 CloseSqlConnection()

```
static void util::mysql::MysqlUtils::CloseSqlConnection ()  [inline], [static], [inherited]
```

Close the thread-specific MySQL connection if it is currently open.

If the thread local MySQL connection is not currently open, then this function will have no effect.

Note

The MySQL connection should be closed when idle to prevent the connection from timing out and throwing exceptions.

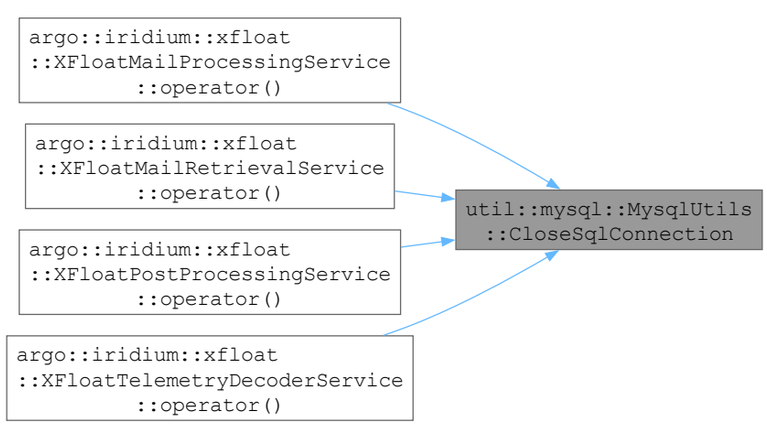
Exceptions

log::MessageException	if an error occurs while attempting to close the MySQL connection.

References CATCH_BLOCK, and util::mysql::MysqlUtils::sql_connection_.

Referenced by argo::iridium::xfloat::XFloatMailProcessingService::operator()(), argo::iridium::xfloat::XFloatMailRetrievalService::operator(
argo::iridium::xfloat::XFloatPostProcessingService::operator()(), and argo::iridium::xfloat::XFloatTelemetryDecoderService::operator()().

Here is the caller graph for this function:



10.87.3.36 ComputePressureOffsetFromSurfaceTimeSeries()

```
float argo::iridium::xfloat::XFloatDiveData::ComputePressureOffsetFromSurfaceTimeSeries (
            unsigned int max_num_dives)
```

Compute this dive's surface pressure offset from the surface time-series data.

The offset is the mean of the surface pressure means (see UpdateDerivedSurfaceDataFields()) of this dive and up to `max_num_dives` preceding dives, counting only dives whose surface pressure is valid. Each dive counts equally, regardless of its number of samples. The result is stored in the pressure offset fields (`pres_offs`, `pres_offs_num_dives`, `pres_offs_num_samples`, `pres_offs_valid`), which the listing and PHY output use for the pressure offset correction.

The offset is only computed for instruments whose conductivity sensor needs the correction (WMO instrument types 866, 867 and 875). For SBE sensors (868, 873, 876) and unknown types the offset is left invalid; an unexpected type is logged as a warning. It is also left invalid if no dive has valid surface pressure data.

Note

Preceding dives are loaded with XFloat::GetDiveData(), which may read them from the database.

Parameters

in	*max_num_dives*	The maximum number of preceding dives to include (0 = none, and the offset is not computed).

Exceptions

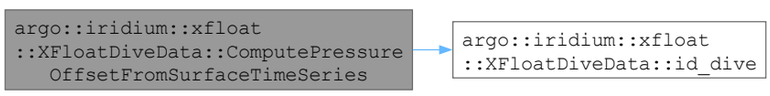
util::log::MessageException	if an error occurs.

Returns

Always 0; read the result from the pressure offset fields.

References CATCH_BLOCK, id_dive(), ldbg, lwrn, util::thread::RecursiveLockable::mutex_, and XFLT_ID_UNKNOWN.

Here is the call graph for this function:



10.87.3.37 DecodeMailMessageData()

```
int argo::iridium::xfloat::XFloatDiveData::DecodeMailMessageData (
            shared_ptr< XFloatMailMessage > & msg)
```

Decode the SBD data of a mail message into this dive data object.

Messages transmitted more than 15 minutes before the instrument's start-of-mission or deployment date/time are ignored. Otherwise, the mail message is assigned to this dive (see AssignMailMessage()), its SBD data is decoded into X-messages (see DecodeXMsgData()), the X-message type identifiers are stored in the mail message, and the parent instrument's IMEI and latest Iridium position are updated.

A message with a duplicate message ID is not assigned or decoded. A retransmission (a message whose SBD data is identical to one already assigned to this dive) is assigned, but its SBD data is not decoded again.

Parameters

in	*msg*	The mail message to decode.

Exceptions

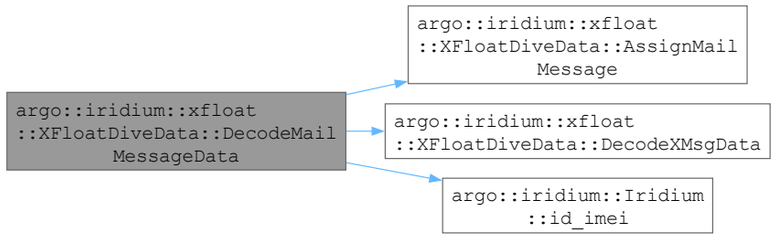
util::log::MessageException	if `msg` is NULL or an error occurs while decoding.

Returns

1 if the message is ignored because it was transmitted before the start-of-mission or deployment date/time, or 0 otherwise (including duplicates and retransmissions).

References AssignMailMessage(), CATCH_BLOCK, DecodeXMsgData(), EXC_THROW, argo::iridium::Iridium::id_imei(), and lwrn.

Here is the call graph for this function:



10.87.3.38 DecodeXMsgData()

```
void argo::iridium::xfloat::XFloatDiveData::DecodeXMsgData (
            const vector< uint8_t > & data,
            shared_ptr< vector< uint8_t > > xmsg_id_types = shared_ptr<vector<uint8_t> >())
```

Decode an X-Message from the binary message data.

Exceptions

None.	Never throws.

Parameters

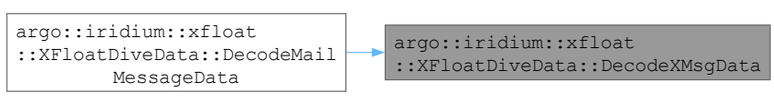
`in`	*data*	The binary message data to decode.
`out`	*xmsg_id_types*	A list of the x-message type identifiers contained in the `data` message.

Todo Verify the X-Message checksum value during the decoding of the X-Message data.

References CATCH_BLOCK, CATCH_NOTHROW, EXC_THROW, ldbg, linf, lwrn, and u16.

Referenced by DecodeMailMessageData().

Here is the caller graph for this function:



10.87.3.39 DeleteFromDatabase()

```
static void argo::iridium::xfloat::XFloatDiveData::DeleteFromDatabase (
            int id_instrument)  [inline], [static]
```

Delete all dive data for an instrument from the SQL database.

Deletes every GPS message row (see util::xmsg::XGPS::DeleteFromDatabase(int)) and every dive data row for the instrument. This is used when an instrument's messages are reprocessed, so that the dives can be rebuilt from the stored mail messages. Stored mail messages are not deleted.

Warning

This permanently removes all of the instrument's dive and GPS rows.

Parameters

in	*id_instrument*	The unique instrument identifier (serial number).

Exceptions

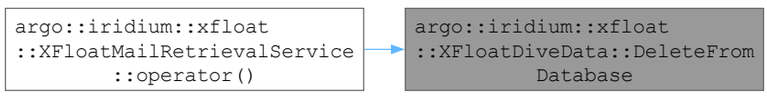
util::log::MessageException	if a delete query fails.

References CATCH_BLOCK, argo::iridium::Iridium::id_instrument(), and sql_table_description_.

Referenced by argo::iridium::xfloat::XFloatMailRetrievalService::operator()().

Here is the call graph for this function:

Here is the caller graph for this function:



10.87.3.40 DeleteObjectsFromDatabase()

```
static void argo::iridium::xfloat::XFloatDiveData::DeleteObjectsFromDatabase (
            const string & where)  [inline], [static]
```

Delete dive data rows from the SQL database table.

Parameters

in	*where*	The SQL "where-condition" clause selecting the rows to delete.

Exceptions

util::log::MessageException	if the delete query fails.

References CATCH_BLOCK, and sql_table_description_.

10.87.3.41 ExecuteQuery()

```
static shared_ptr< ResultSet > util::mysql::MysqlUtils::ExecuteQuery (
            Connection * con,
            const string & query)  [inline], [static], [inherited]
```

Execute an SQL query and return a shared pointer to the ResultSet object.

Parameters

in	*con*	A pointer to the SQL connection object. This must not be NULL.
in	*query*	The SQL query to execute. This must be a well-formatted SQL query string.

Exceptions

log::MessageException	if 'con' is null, 'query' is a malfored SQL string, or any other error occurs.

Returns

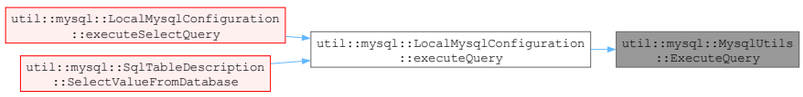
A shared pointer to the query ResultSet object.

References CATCH_BLOCK, and util::mysql::MysqlUtils::GeneratePreparedStatement().

Referenced by util::mysql::LocalMysqlConfiguration::executeQuery().

Here is the call graph for this function:

Here is the caller graph for this function:



10.87.3.42 executeQuery()

```
shared_ptr< sql::ResultSet > util::mysql::LocalMysqlConfiguration::executeQuery (
            const string & query)  [static], [inherited]
```

Execute an SQL query and return a shared pointer to the ResultSet object.

Parameters

in	*query*	The SQL query to execute. This must be a well-formatted SQL query string.

Exceptions

log::MessageException	'query' is a malfored SQL string or any other error occurs while executing the query.

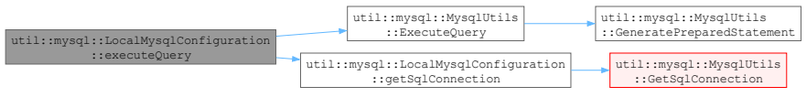
Returns

A shared pointer to the query ResultSet object.

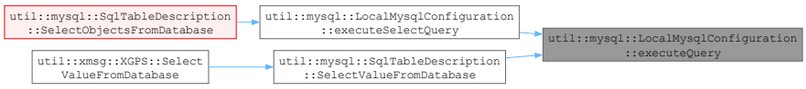
References util::mysql::MysqlUtils::ExecuteQuery(), and util::mysql::LocalMysqlConfiguration::getSqlConnection().

Referenced by util::mysql::LocalMysqlConfiguration::executeSelectQuery(), and util::mysql::SqlTableDescription< T >::SelectValueFromD

Here is the call graph for this function:



Here is the caller graph for this function:



10.87.3.43 executeSelectQuery()

```
shared_ptr< sql::ResultSet > util::mysql::LocalMysqlConfiguration::executeSelectQuery (
            const string & what,
            const string & from,
            const string & where = "",
            const string & groupBy = "",
            const string & orderBy = "",
            unsigned int limit = 0,
            unsigned int offset = 0)  [static], [inherited]
```

Execute an SQL select query and return a shared pointer to the ResultSet object.

Parameters

in	*what*	The SQL select statement select-expression clause.
in	*from*	The SQL select statement table–references clause.
in	*where*	The SQL select statement where-condition clause.
in	*groupBy*	The SQL select statement group-by clause.
in	*orderBy*	The SQL select statement order-by clause.
in	*limit*	The SQL select statement result set row limit value.
in	*offset*	The SQL select statement result set row offset.

Exceptions

log::MessageException	if an error occurs while executing the query.

Returns

A shared pointer to the ResultSet object.

References CATCH_BLOCK, util::mysql::LocalMysqlConfiguration::executeQuery(), and util::mysql::MysqlUtils::GenerateSqlSelectStr().

Referenced by util::mysql::SqlTableDescription< T >::SelectObjectsFromDatabase().

Here is the call graph for this function:

Here is the caller graph for this function:

10.87.3.44 ExecuteUpdate()

```
static int util::mysql::MysqlUtils::ExecuteUpdate (
            Connection * con,
            const string & update)  [inline], [static], [inherited]
```

Execute an SQL update and return the status code.

Parameters

in	*con*	A pointer to the SQL connection object. This must not be NULL.
in	*update*	The SQL update to execute. This must be a well-formatted SQL update string.

Exceptions

log::MessageException	if 'con' is null, 'update' is a malformed SQL string, or any other error occurs.

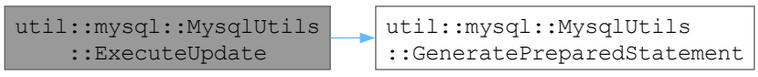
Returns

The update status code.

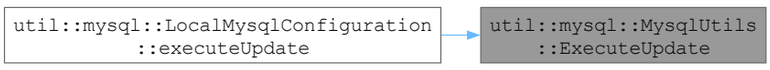
References CATCH_BLOCK, and util::mysql::MysqlUtils::GeneratePreparedStatement().

Referenced by util::mysql::LocalMysqlConfiguration::executeUpdate().

Here is the call graph for this function:



Here is the caller graph for this function:



10.87.3.45 executeUpdate()

```
int util::mysql::LocalMysqlConfiguration::executeUpdate (
            const string & update)  [static], [inherited]
```

Execute an SQL update and return the status code.

Parameters

in	*update*	The SQL update to execute. This must be a well-formatted SQL update string.

Exceptions

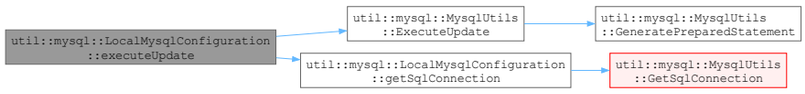
log::MessageException	if 'update' is a malformed SQL string, or any other error occurs while executing the update.

Returns

The update status code.

References util::mysql::MysqlUtils::ExecuteUpdate(), and util::mysql::LocalMysqlConfiguration::getSqlConnection().

Here is the call graph for this function:



10.87.3.46 GenerateCompositeFallTimeSeriesData()

```
shared_ptr< x::XCompositeFall > argo::iridium::xfloat::XFloatDiveData::GenerateCompositeFall↩
TimeSeriesData () const  [private]
```

Generate a composite fall time-series data for this dive.

Exceptions

util::log::MessageException	if an error occurs while generating the composite data.

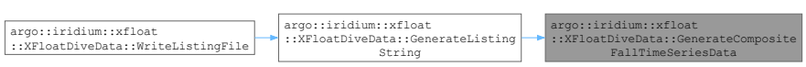
Returns

The The composite fall time-series data for this dive.

References CATCH_BLOCK, ldbg, util::thread::RecursiveLockable::mutex_, std::make_shared, xmsg_fall_time_series_, and XMSG_MAX_FALL.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.87.3.47 GenerateCompositeIridiumPosition()

```
shared_ptr< x::XCompositeIridiumPosition > argo::iridium::xfloat::XFloatDiveData::Generate↩
CompositeIridiumPosition () const  [private]
```

Generate a composite Iridium position for this dive.

Exceptions

util::log::MessageException	if an error occurs while generating the composite data.

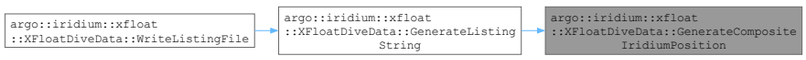
Returns

The composite Iridium position for this dive.

References CATCH_BLOCK, ldbg, mail_messages_, util::thread::RecursiveLockable::mutex_, std::make_shared, and XMSG_MAX_MAIL.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.87.3.48 GenerateCompositePumpTimeSeriesData()

```
shared_ptr< x::XCompositePump > argo::iridium::xfloat::XFloatDiveData::GenerateCompositePump↩
TimeSeriesData () const  [private]
```

Get the composite pump time-series data for this dive.

Exceptions

util::log::MessageException	if an error occurs while generating the composite data.

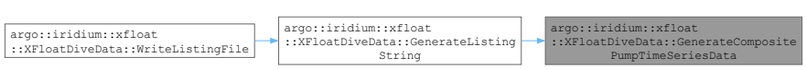
Returns

The composite pump time-series data for this dive.

References CATCH_BLOCK, ldbg, util::thread::RecursiveLockable::mutex_, std::make_shared, XMSG_MAX_PUMP, and xmsg_pump_time_series_.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.87.3.49 GenerateCompositeRiseTimeSeriesData()

```
shared_ptr< x::XCompositeRise > argo::iridium::xfloat::XFloatDiveData::GenerateCompositeRise↩
TimeSeriesData () const  [private]
```

Get the composite rise time-series data for this dive.

Exceptions

util::log::MessageException	if an error occurs while generating the composite data.

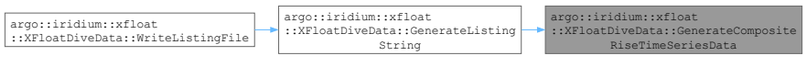
Returns

The composite rise time-series data for this dive.

References CATCH_BLOCK, ldbg, util::thread::RecursiveLockable::mutex_, std::make_shared, XMSG_MAX_RISE, and xmsg_rise_time_series_.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.87.3.50 GenerateCurrentUtcTimeIso8601String()

```
static string util::date::DateUtils::GenerateCurrentUtcTimeIso8601String ()  [inline], [static],
[inherited]
```

Generate an ISO8601 format date/time string representation of the current time, in UTC.

Note

All date/time objects use the UTC time-zone.

Returns

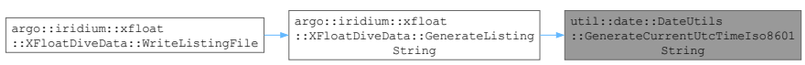
An ISO8601 format date/time string representation of the current time.

References util::date::DateUtils::GenerateIso8601String(), and util::date::DateUtils::GetCurrentUtcTime().

Referenced by GenerateListingString().

Here is the call graph for this function:

Here is the caller graph for this function:



10.87.3.51 GenerateIso8601String()

```
static string util::date::DateUtils::GenerateIso8601String (
            const ptime & p,
            bool extended = true)  [inline], [static], [inherited]
```

Generate an ISO8601 format date/time string representation of a POSIX time object.

Note

All date/time objects use the UTC time-zone.

Parameters

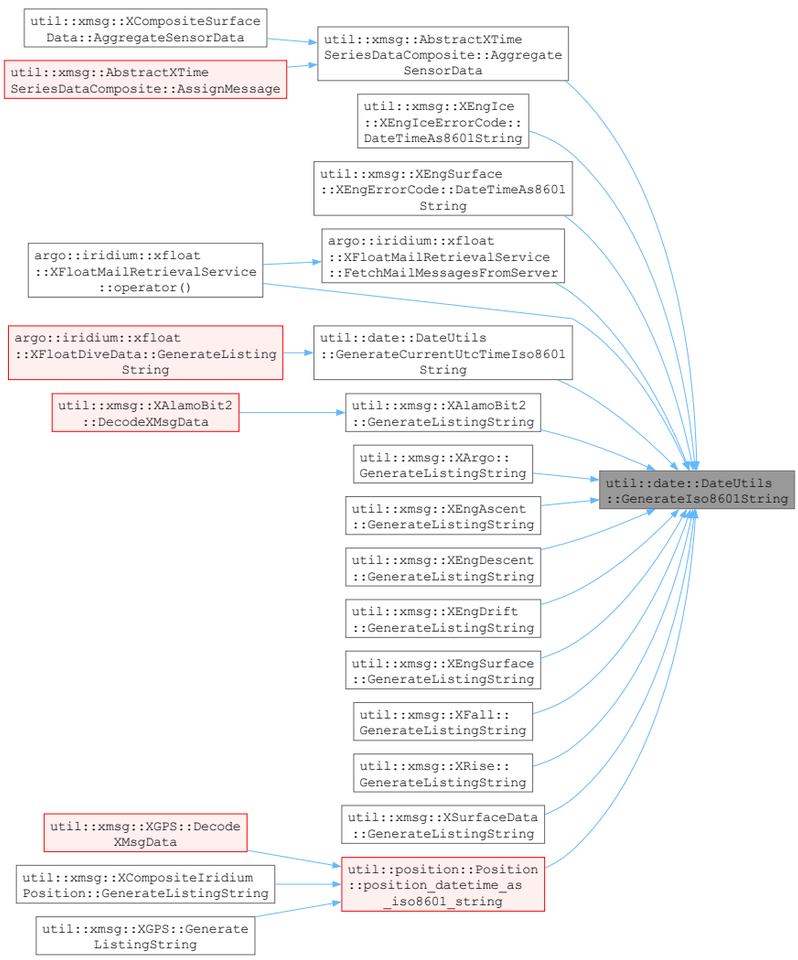
in	*p*	POSIX time from which to generate string.
in	*extended*	If TRUE, use the ISO 8601 extended format (YYYY-MM-DDThh:mm:ssZ); otherwise, use the basic format (YYYYMMDDThhmmss).

Returns

An ISO8601 format date/time string representation of the POSIX time object. If p is equal to boost::posix_↩ time::not_a_date_time, then the string "0000-00-00T00:00:00Z" (extended) or "00000000000000" (basic) will be returned.

Referenced by util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::AggregateSensorData(), util::xmsg::XEngIce::XEngIce util::xmsg::XEngSurface::XEngErrorCode::DateTimeAs8601String(), argo::iridium::xfloat::XFloatMailRetrievalService::FetchMailMessages util::date::DateUtils::GenerateCurrentUtcTimeIso8601String(), util::xmsg::XAlamoBit2::GenerateListingString(), util::xmsg::XArgo::Genera util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListing util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(), argo::iridium::xfloat::XFloatMailRetrievalService::operator()(), and util::position::Position::position_datetime_as_iso8601_string().

Here is the caller graph for this function:



10.87.3.52 GenerateJulianDay()

```
static double util::date::DateUtils::GenerateJulianDay (
            const ptime & dt,
            const ptime & ref_dt)  [inline], [static], [inherited]
```

Convert a date/time to fractional days since a reference date/time.

Parameters

in	*dt*	The date/time to convert.
in	*ref_dt*	The reference date/time (day 0).

Exceptions

util::log::MessageException	if either date/time is not valid.

Returns

The number of days (with millisecond resolution) from `ref_dt` to `dt`.

References CATCH_BLOCK, and EXC_THROW.

10.87.3.53 GenerateListingString()

```
string argo::iridium::xfloat::XFloatDiveData::GenerateListingString () const
```

Generate a listing entry string representation of this dive data object.

Exceptions

util::log::MessageException	if an error occurs while generating the string representation.

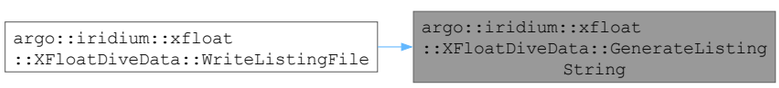
Returns

A listing entry string representation of this dive data object.

References CATCH_BLOCK, CATCH_NOTHROW, EXC_THROW, GenerateCompositeFallTimeSeriesData(), GenerateCompositeIridiumPosition(), GenerateCompositePumpTimeSeriesData(), GenerateCompositeRiseTimeSeriesData(), util::date::DateUtils::GenerateCurrentUtcTimeIso8601String(), argo::iridium::xfloat::XFloatMailMessage::GenerateListingComment(), util::xmsg::XCompositeDrift::GenerateListingString(), util::xmsg::XCompositeHiResProfile::GenerateListingString(), util::xmsg::XCompositeProfile::GenerateListingString(), util::xmsg::XCompositeSurfaceData::GenerateListingString(), util::xmsg::XMfaMissionCfg::GenerateListingString(), GetGainsAndOffsets(), GetSurfaceGainsAndOffsets(), HasBinnedProfileData(), HasCompleteProfileData(), HasContiguousMomsns(), HasContiguousPids(), HasDiveData(), HasDriftData(), HasEngineeringData(), HasPositionData(), HasUnbinnedProfileData(), HasValidPosition(), argo::iridium::Iridium::id_dac(), id_dive(), argo::iridium::Iridium::id_imei(), argo::iridium::Iridium::id_instrument(), argo::iridium::Iridium::id_wmo(), intermediate_listing_file_path(), util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::IsEmpty(), mail_messages_, util::thread::RecursiveLockable::mutex_, parent_instrument_, pressure_offset_correction_enabled_list_file_, PROG_VERS, PROG_XVERS, SensorGainsAndOffsetsAreKnown(), xmsg_acoustic_positions_, xmsg_argo_data_, xmsg_composite_drift_data_, xmsg_composite_profile_binned_, xmsg_composite_profile_unbinned_, xmsg_engineering_data_abort_, xmsg_engineering_data_ascen xmsg_engineering_data_bit_fail_, xmsg_engineering_data_bit_passed_, xmsg_engineering_data_descent_, xmsg_engineering_data_dia xmsg_engineering_data_drift_, xmsg_engineering_data_ice_, xmsg_engineering_data_standard_dive_, xmsg_engineering_data_surface xmsg_fall_time_series_, xmsg_gps_positions_, XMSG_HEADER, XMSG_HEADER2, xmsg_mission_cfg_, xmsg_mission_cmd_echobac xmsg_pump_time_series_, xmsg_rise_time_series_, XMSG_SECTION, xmsg_surface_time_series_, xmsg_valve_data_, and xmsg_wave_field_.

Referenced by WriteListingFile().

Here is the caller graph for this function:



10.87.3.54 GenerateOrderedMomsnList()

```
vector< int > argo::iridium::xfloat::XFloatDiveData::GenerateOrderedMomsnList () const  [private]
```

Generate a list of ordered Iridium message MOMSNs for this dive.

Exceptions

util::log::MessageException	if an error occurs.

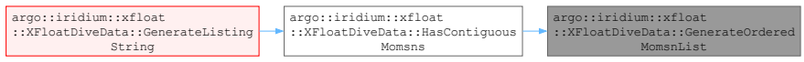
Returns

A vector of MOMSNs, sorted in non-decreasing order.

References CATCH_BLOCK, mail_messages_, and util::thread::RecursiveLockable::mutex_.

Referenced by HasContiguousMomsns().

Here is the caller graph for this function:



10.87.3.55 GenerateOrderedPidList()

```
vector< int > argo::iridium::xfloat::XFloatDiveData::GenerateOrderedPidList () const  [private]
```

Generate a list of ordered Iridium message PIDs for this dive.

Exceptions

util::log::MessageException	if an error occurs.

Returns

A vector of PIDs, sorted in non-decreasing order.

References CATCH_BLOCK, and mail_messages_.

Referenced by HasContiguousPids().

Here is the caller graph for this function:

10.87.3.56 GeneratePhyString()

```
string argo::iridium::xfloat::XFloatDiveData::GeneratePhyString () const
```

Generate a PHY entry string representation of this dive data object.

Exceptions

util::log::MessageException	if an error occurs while generating the string representation.

Returns

A phy entry string representation of this dive data object.

Todo Implement the functionality for generate a PHY string representation of a dive data object for a Solo-II float.

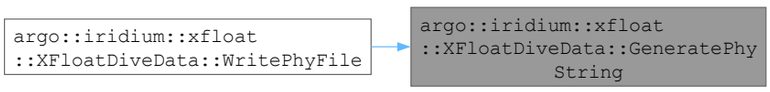
Todo Implement the functionality for generate a PHY string representation of a dive data object for a Deep Argo float.

References CATCH_BLOCK, EXC_THROW, util::xmsg::XCompositeHiResProfile::GeneratePhyString(), util::xmsg::XCompositeProfile::G
GetGainsAndOffsets(), HasCompleteBinnedProfileData(), HasCompleteUnbinnedProfileData(), argo::iridium::Iridium::id_dac(),
id_dive(), argo::iridium::Iridium::id_wmo(), mail_messages_, util::thread::RecursiveLockable::mutex_, parent_instrument_,
pressure_offset_correction_enabled_phy_file_, XARGO_INSRUMENT_TYPE_ALAMO, XARGO_INSRUMENT_TYPE_DEEP_ARGO,
XARGO_INSRUMENT_TYPE_SOLO2, XARGO_INSRUMENT_TYPE_UNKNOWN, xmsg_composite_profile_binned_,
xmsg_composite_profile_unbinned_, and XMSG_PHY.

Referenced by WritePhyFile().

Here is the call graph for this function:

Here is the caller graph for this function:



10.87.3.57 GeneratePreparedStatement()

```
static shared_ptr< PreparedStatement > util::mysql::MysqlUtils::GeneratePreparedStatement (
            Connection * con,
            const string & statement)  [inline], [static], [inherited]
```

Generate a prepared SQL statement object for the given SQL statement.

Parameters

in	*con*	A pointer to the SQL connection object. This must not be NULL.
in	*statement*	The SQL statement to prepare. This must be a well-formed SQL string.

Exceptions

log::MessageException	if 'con' is NULL, 'statement' is not a valid SQL string, or if an error occurs.

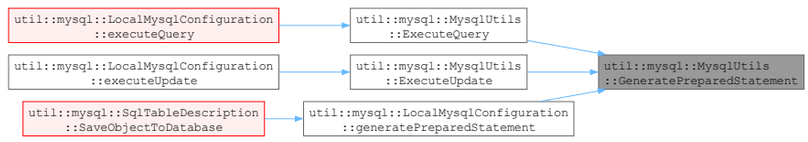
Returns

A prepared SQL statement object generated from the given SQL statement.

References CATCH_BLOCK, and EXC_THROW.

Referenced by util::mysql::MysqlUtils::ExecuteQuery(), util::mysql::MysqlUtils::ExecuteUpdate(), and util::mysql::LocalMysqlConfiguration:

Here is the caller graph for this function:



10.87.3.58 generatePreparedStatement()

```
std::shared_ptr< sql::PreparedStatement > util::mysql::LocalMysqlConfiguration::generatePrepared↩
Statement (
            const string & statement)  [static], [inherited]
```

Generate a prepared SQL statement object for the given SQL statement.

Parameters

in	*statement*	The SQL statement to prepare. This must be a well-formed SQL string.

Exceptions

log::MessageException	if 'statement' is not a valid SQL string or if an error occurs while generating the prepared statement object..

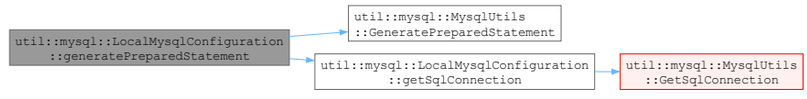
Returns

A prepared SQL statement object generated from the given SQL statement.

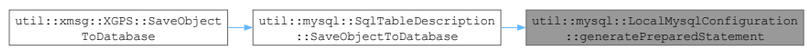
References util::mysql::MysqlUtils::GeneratePreparedStatement(), and util::mysql::LocalMysqlConfiguration::getSqlConnection().

Referenced by util::mysql::SqlTableDescription< T >::SaveObjectToDatabase().

Here is the call graph for this function:



Here is the caller graph for this function:



10.87.3.59 GenerateReadableHex()

```
string argo::iridium::xfloat::XFloatDiveData::GenerateReadableHex (
            const vector< uint8_t > & data)  [static]
```

Generate a human-readable hex string.

Parameters

in	*data*	The message data.

Returns

A human readable hex representation of the data.

References CATCH_BLOCK.

10.87.3.60 GenerateSqlDeleteStr()

```
static string util::mysql::MysqlUtils::GenerateSqlDeleteStr (
            const string & from,
            const string & where = "")  [inline], [static], [inherited]
```

Generate an SQL DELETE statement.

Warning

Without a `where` clause, the statement deletes every row in the table.

Parameters

in	*from*	The name of the table to delete from.
in	*where*	The SQL "where-condition" clause (empty = all rows).

Returns

The SQL statement.

10.87.3.61 GenerateSqlSelectStr()

```
static string util::mysql::MysqlUtils::GenerateSqlSelectStr (
            const string & what,
            const string & from,
            const string & where = "",
            const string & groupBy = "",
            const string & orderBy = "",
            unsigned int limit = 0,
            unsigned int offset = 0)  [inline], [static], [inherited]
```

Generate an SQL select statement.

Parameters

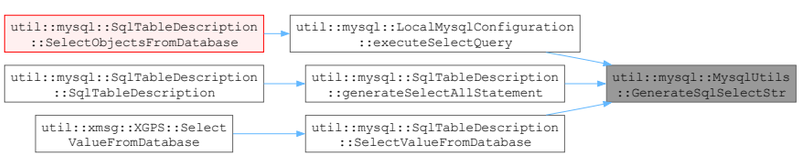
in	*what*	The SQL select statement select-expression clause.
in	*from*	The SQL select statement table-references clause.
in	*where*	The SQL select statement where-condition clause.
in	*groupBy*	The SQL select statement group-by clause.
in	*orderBy*	The SQL select statement order-by clause.
in	*limit*	The SQL select statement result set row limit value.
in	*offset*	The SQL select statement result set row offset.

Returns

The SQL select statement.

Referenced by util::mysql::LocalMysqlConfiguration::executeSelectQuery(), util::mysql::SqlTableDescription< T >::generateSelectAllState and util::mysql::SqlTableDescription< T >::SelectValueFromDatabase().

Here is the caller graph for this function:



10.87.3.62 GetCurrentUtcTime()

```
static ptime util::date::DateUtils::GetCurrentUtcTime ()  [inline], [static], [inherited]
```

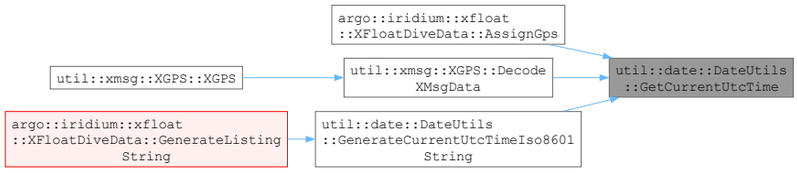
Generate a POSIX time object representing the current date and time (rounded to the nearest second).

Returns

A POSIX time object representing the current date and time (rounded to the nearest second).

Referenced by AssignGps(), util::xmsg::XGPS::DecodeXMsgData(), and util::date::DateUtils::GenerateCurrentUtcTimeIso8601String().

Here is the caller graph for this function:



10.87.3.63 GetDirectoryPath()

```
static const path & util::file::FileSystemUtils::GetDirectoryPath (
            const path & dir,
            bool create = false)  [inline], [static], [inherited]
```

Get a directory path.

Exceptions

MessageException	if create == FALSE and the directory specified by path does not exist.

Parameters

in	*dir*	Directory path.
in	*create*	If TRUE, the directory specified by path will be created if it does not already exists. If FALSE, an exception will be thrown if the directory does not exist.

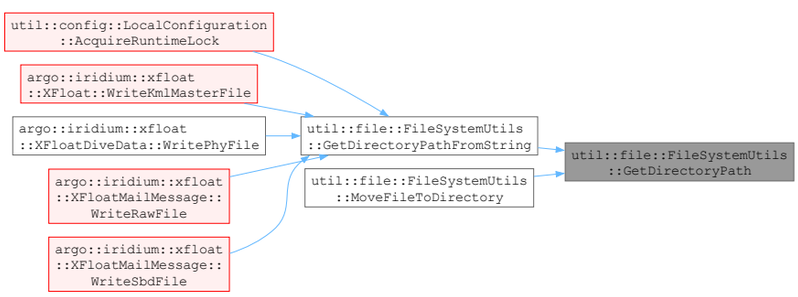
Returns

dir

References CATCH_BLOCK, EXC_THROW, and ldbg.

Referenced by util::file::FileSystemUtils::GetDirectoryPathFromString(), and util::file::FileSystemUtils::MoveFileToDirectory().

Here is the caller graph for this function:



10.87.3.64 GetDirectoryPathFromString()

```
static path util::file::FileSystemUtils::GetDirectoryPathFromString (
            const string & dir,
            bool create = false)  [inline], [static], [inherited]
```

Generate a boost::filesystem::path object from the directory path string.

Exceptions

MessageException	if create == FALSE and the directory specified by path does not exist.

Parameters

in	*dir*	Directory path string.
in	*create*	If TRUE, the directory specified by path will be created if it does not already exists. If FALSE, an exception will be thrown if the directory does not exist.

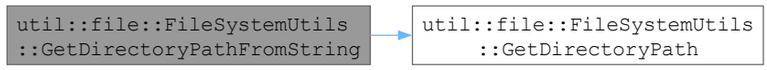
Returns

A boost::filesystem::path object from the directory path string.

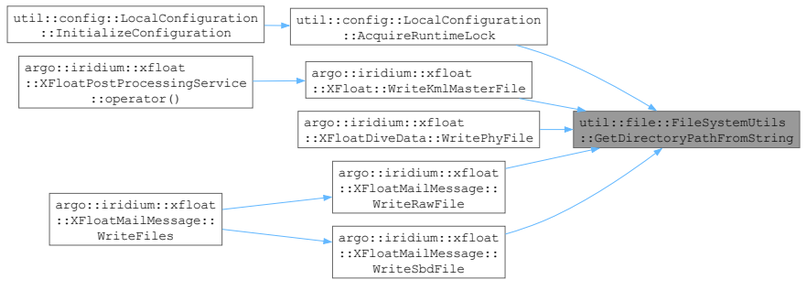
References CATCH_BLOCK, and util::file::FileSystemUtils::GetDirectoryPath().

Referenced by util::config::LocalConfiguration::AcquireRuntimeLock(), argo::iridium::xfloat::XFloat::WriteKmlMasterFile(), WritePhyFile(), argo::iridium::xfloat::XFloatMailMessage::WriteRawFile(), and argo::iridium::xfloat::XFloatMailMessage::WriteSbdFile().

Here is the call graph for this function:



Here is the caller graph for this function:



10.87.3.65 GetDiveDataMailMessagesFromDatabase()

```
vector< shared_ptr< XFloatMailMessage > > argo::iridium::xfloat::XFloatDiveData::GetDiveData↩
MailMessagesFromDatabase ()
```

Get this dive's stored mail messages from the SQL database.

Selects the valid mail messages (`msg_is_valid` = 1) for this instrument and dive. Messages transmitted more than 15 minutes before the instrument's start-of-mission or deployment date/time are excluded. Used to rebuild a dive object loaded from the database.

Exceptions

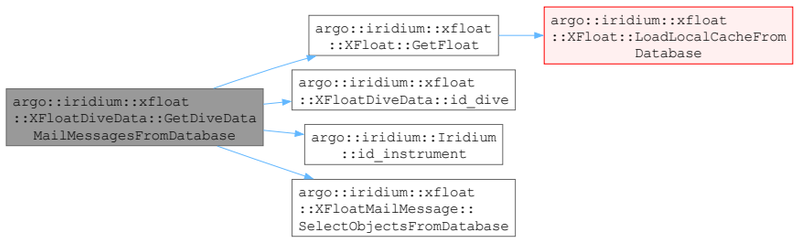
util::log::MessageException	if the instrument or dive identifier is unknown, or the query fails.

Returns

The matching mail messages.

References CATCH_BLOCK, EXC_THROW, argo::iridium::xfloat::XFloat::GetFloat(), id_dive(), argo::iridium::Iridium::id_instrument(), util::thread::RecursiveLockable::mutex_, argo::iridium::xfloat::XFloatMailMessage::SelectObjectsFromDatabase(), XFLT_DIVE_UNKNOWN, and XFLT_ID_UNKNOWN.

Here is the call graph for this function:



10.87.3.66 GetDiveDataObjectFromDatabase()

```
shared_ptr< XFloatDiveData > argo::iridium::xfloat::XFloatDiveData::GetDiveDataObjectFromDatabase
(
            const shared_ptr< XFloat > & xfloat,
            int dive)  [static]
```

Get a dive data object for a given instrument and dive number from the database, if it exists.

Parameters

in	*xfloat*	A shared pointer to the instrument object. Must not be NULL.
in	*dive*	The dive number.

Exceptions

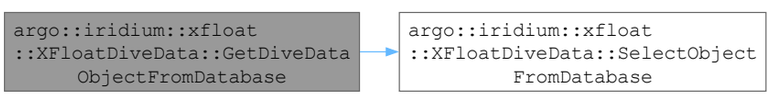
util::log::MessageException	if an error occurs while querying the database or xfloat is NULL.

Returns

A shared pointer to the dive data object or NULL if a corresponding entry does not exist in the database.

References CATCH_BLOCK, EXC_THROW, SelectObjectFromDatabase(), and XFLT_ID_UNKNOWN.

Here is the call graph for this function:



10.87.3.67 GetGainsAndOffsets()

```
void argo::iridium::xfloat::XFloatDiveData::GetGainsAndOffsets (
            uint16_t & pg,
            uint16_t & po,
            uint16_t & tg,
            uint16_t & to,
            uint16_t & sg,
            uint16_t & so) const
```

Get the pressure, temperature, and salinity sensor gains and offsets.

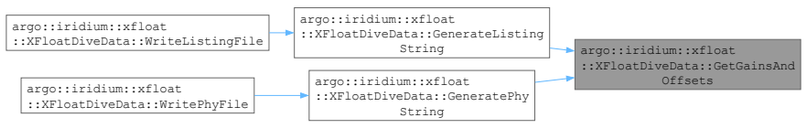
Parameters

out	*pg*	The pressure sensitivity gain factor, in dBar/LSB.
out	*po*	The pressure offset, in dBar.
out	*tg*	The temperature sensitivity gain factor, in degC/LSB.
out	*to*	The temperature offset, in degC.
out	*sg*	The salinity sensitivity scale factor, in PSU/LSB.
out	*so*	The salinity offset, in PSU.

References util::thread::RecursiveLockable::mutex_, XFLT_GAIN_INVALID, XFLT_OFFS_INVALID, and xmsg_argo_data_.

Referenced by GenerateListingString(), and GeneratePhyString().

Here is the caller graph for this function:



10.87.3.68 GetGpsPositionEndOfDive()

```
shared_ptr< x::XGPS > argo::iridium::xfloat::XFloatDiveData::GetGpsPositionEndOfDive () const
```

Get the end-of-dive GPS message.

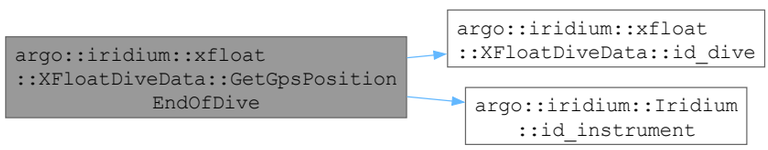
This function returns a pointer to the GPS message that represents the position of the profile.

Returns

A pointer to the end-of-dive GPS message, or to a newly constructed empty GPS message if one does not exist.

References id_dive(), argo::iridium::Iridium::id_instrument(), util::thread::RecursiveLockable::mutex_, std::make_shared, and xmsg_gps_positions_.

Here is the call graph for this function:



10.87.3.69 GetSqlConnection()

```
static thread_specific_ptr< Connection > & util::mysql::MysqlUtils::GetSqlConnection (
            string url,
            string user,
            string password,
            string schema)  [inline], [static], [inherited]
```

Get a thread-specific instance of a MySQL connection object.

A new connection will be created, if necessary.

Parameters

in	*url*	The MySQL host server qualified address.
in	*user*	The MySQL account username.
in	*password*	The MySQL account password.
in	*schema*	The MySQL account schema name.

Exceptions

log::MessageException	if an error occurs while creating a new thread-specific instance of the SQL connection object

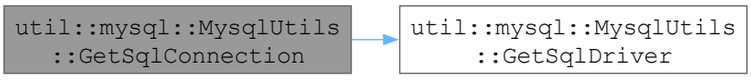
Returns

A thread-specific instance of a MySQL connection object.

References CATCH_BLOCK, util::mysql::MysqlUtils::GetSqlDriver(), and util::mysql::MysqlUtils::sql_connection_.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

Here is the call graph for this function:



Here is the caller graph for this function:

10.87.3.70 getSqlConnection()

`sql::Connection * util::mysql::LocalMysqlConfiguration::getSqlConnection () [static], [protected], [inherited]`

Generate a thread-specific instance of a MySQL connection object.

A new connection will be created, if necessary.

Exceptions

log::MessageException	if an error occurs while generating the SQL connection object.

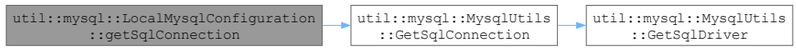
Returns

A thread-specific instance of a MySQL connection object.

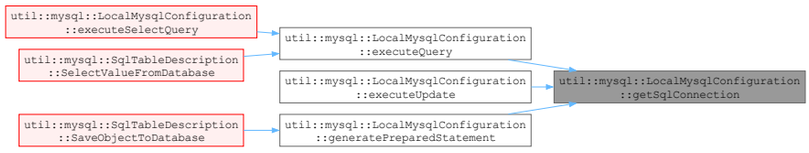
References util::mysql::LocalMysqlConfiguration::address_, CATCH_BLOCK, util::mysql::MysqlUtils::GetSqlConnection(), util::mysql::LocalMysqlConfiguration::password_, util::mysql::LocalMysqlConfiguration::port_, util::mysql::LocalMysqlConfiguration::protoc util::mysql::LocalMysqlConfiguration::schema_, and util::mysql::LocalMysqlConfiguration::user_.

Referenced by util::mysql::LocalMysqlConfiguration::executeQuery(), util::mysql::LocalMysqlConfiguration::executeUpdate(), and util::mysql::LocalMysqlConfiguration::generatePreparedStatement().

Here is the call graph for this function:



Here is the caller graph for this function:



10.87.3.71 GetSqlDriver()

```
static thread_specific_ptr< MySQL_Driver > & util::mysql::MysqlUtils::GetSqlDriver ()  [inline],
[static], [inherited]
```

Get a thread-specific instance of a MySQL driver object.

A new driver will created, if necessary.

Exceptions

log::MessageException	if an error occurs while creating a new thread-specific instance of the SQL driver object

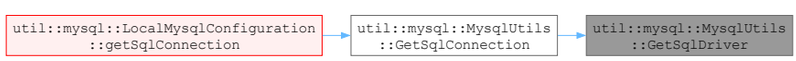
Returns

A thread-specific instance of a MySQL driver object.

References CATCH_BLOCK, and util::mysql::MysqlUtils::sql_driver_.

Referenced by util::mysql::MysqlUtils::GetSqlConnection().

Here is the caller graph for this function:



10.87.3.72 GetSurfaceGainsAndOffsets()

```
void argo::iridium::xfloat::XFloatDiveData::GetSurfaceGainsAndOffsets (
            uint16_t & pg,
            uint16_t & po,
            uint16_t & tg,
            uint16_t & to,
            uint16_t & sg,
            uint16_t & so) const
```

Get the pressure, temperature, and salinity sensor gains and offsets for surface data.

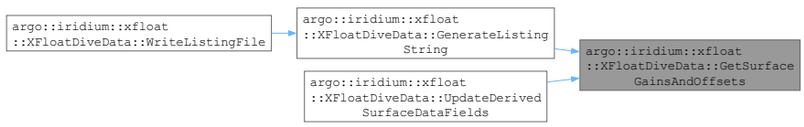
Parameters

out	*pg*	The pressure sensitivity gain factor, in dBar/LSB.
out	*po*	The pressure offset, in dBar.
out	*tg*	The temperature sensitivity gain factor, in degC/LSB.
out	*to*	The temperature offset, in degC.
out	*sg*	The salinity sensitivity scale factor, in PSU/LSB.
out	*so*	The salinity offset, in PSU.

References util::thread::RecursiveLockable::mutex_, XFLT_GAIN_INVALID, XFLT_OFFS_INVALID, and xmsg_argo_data_.

Referenced by GenerateListingString(), and UpdateDerivedSurfaceDataFields().

Here is the caller graph for this function:



10.87.3.73 HasBinnedProfileData()

```
bool argo::iridium::xfloat::XFloatDiveData::HasBinnedProfileData () const
```

Does this dive data object have any binned profile data?

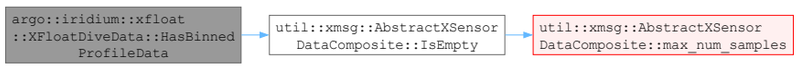
Returns

TRUE if this dive data object has any binned profile data, or FALSE otherwise.

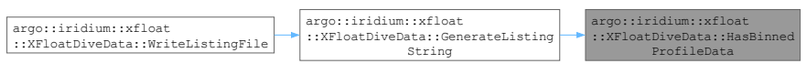
References util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::IsEmpty(), util::thread::RecursiveLockable::mutex_, and xmsg_composite_profile_binned_.

Referenced by GenerateListingString().

Here is the call graph for this function:



Here is the caller graph for this function:



10.87.3.74 HasCompleteBinnedProfileData()

```
bool argo::iridium::xfloat::XFloatDiveData::HasCompleteBinnedProfileData () const
```

Does this dive have complete binned profile data?

Note

The conditions that need to be met in order for the profile data for this dive to be considered complete are:

- $N_{pres} > 0$, and
- $N_{pres} = N_{temp}$, and
- $[(N_{saln=0})$ or $(N_{saln} = N_{pres})]$, and
- pres., temp., and saln. message each have contiguously incrementing message indices.

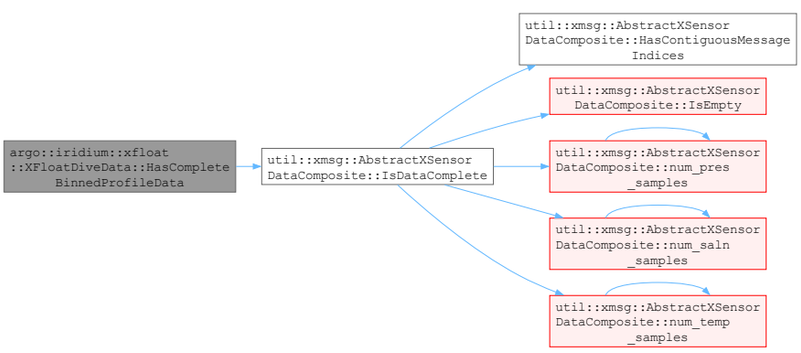
Returns

TRUE if this dive has complete profile data, or FALSE otherwise.

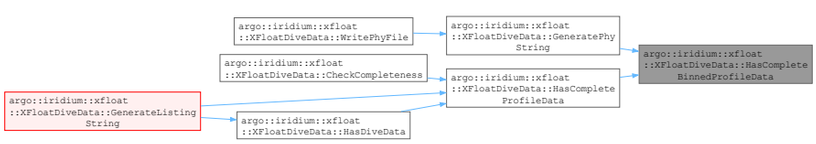
References util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::IsDataComple and xmsg_composite_profile_binned_.

Referenced by GeneratePhyString(), and HasCompleteProfileData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.87.3.75 HasCompleteProfileData()

```
bool argo::iridium::xfloat::XFloatDiveData::HasCompleteProfileData () const
```

Does this dive have complete binned or unbinned profile data?

Note

The conditions that need to be met in order for the profile data for this dive to be considered complete are:

- $N_{pres} > 0$, and
- $N_{pres} = N_{temp}$, and
- $[(N_{saln=0})$ or $(N_{saln} = N_{pres})]$, and
- pres., temp., and saln. message each have contiguously incrementing message indices.

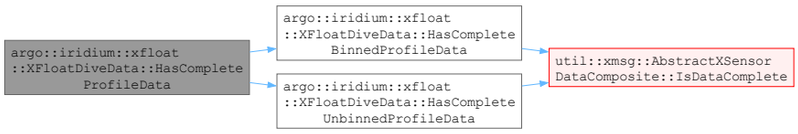
Returns

TRUE if this dive has complete profile data, or FALSE otherwise.

References HasCompleteBinnedProfileData(), and HasCompleteUnbinnedProfileData().

Referenced by CheckCompleteness(), GenerateListingString(), and HasDiveData().

Here is the call graph for this function:



Here is the caller graph for this function:

10.87.3.76 HasCompleteUnbinnedProfileData()

```
bool argo::iridium::xfloat::XFloatDiveData::HasCompleteUnbinnedProfileData () const
```

Does this dive have complete unbinned profile data?

Note

The conditions that need to be met in order for the profile data for this dive to be considered complete are:

- $N_{pres} > 0$, and
- $N_{pres} = N_{temp}$, and
- $[(N_{saln=0})$ or $(N_{saln} = N_{pres})]$, and
- pres., temp., and saln. message each have contiguously incrementing message indices.

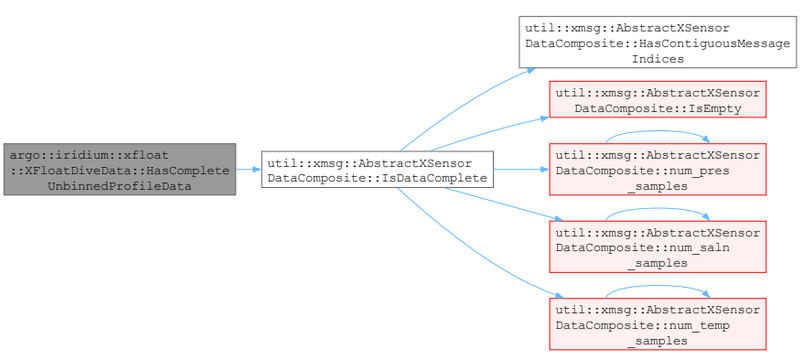
Returns

TRUE if this dive has complete profile data, or FALSE otherwise.

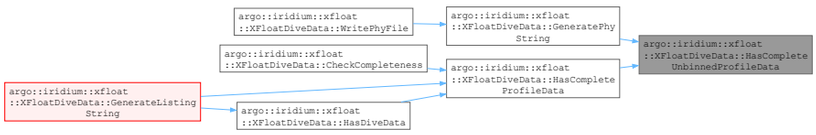
References util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::IsDataComple
and xmsg_composite_profile_unbinned_.

Referenced by GeneratePhyString(), and HasCompleteProfileData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.87.3.77 HasContiguousMomsns()

```
bool argo::iridium::xfloat::XFloatDiveData::HasContiguousMomsns () const
```

Do the messages that comprise this dive data object have contiguously increasing MOMSN numbers?

Note

This method will still return TRUE if there are Iridium message with duplicate MOMSNs, as long as no numbers are missing from the sequence.

Exceptions

util::log::MessageException	if an error occurs.

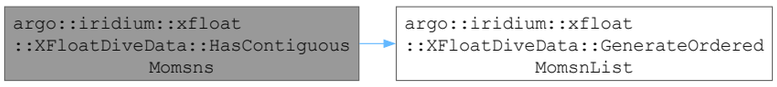
Returns

TRUE if the mail message have contiguously incrementing MOMSNs, or FALSE otherwise.

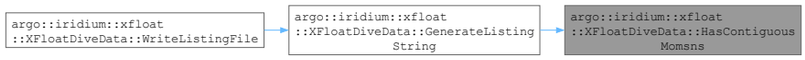
References CATCH_BLOCK, GenerateOrderedMomsnList(), and util::thread::RecursiveLockable::mutex_.

Referenced by GenerateListingString().

Here is the call graph for this function:



Here is the caller graph for this function:



10.87.3.78 HasContiguousPids()

```
bool argo::iridium::xfloat::XFloatDiveData::HasContiguousPids () const
```

Do the messages that comprise this dive data object have contiguously increasing packet index identifiers (PIDs) starting with zero?

Note

This method will still return TRUE if there are Iridium message with duplicate PIDs, as long as no numbers are missing from the sequence.

Exceptions

util::log::MessageException	if an error occurs.

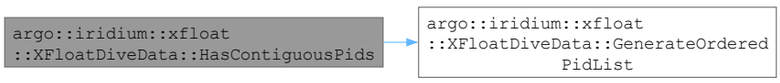
Returns

TRUE if the mail message have contiguously increasing packet index identifiers (PIDs), or FALSE otherwise.

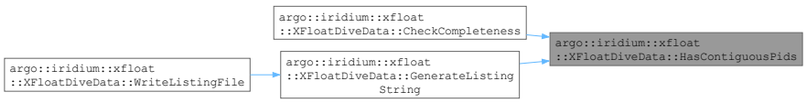
References CATCH_BLOCK, GenerateOrderedPidList(), and util::thread::RecursiveLockable::mutex_.

Referenced by CheckCompleteness(), and GenerateListingString().

Here is the call graph for this function:



Here is the caller graph for this function:



10.87.3.79 HasDiveData()

```
bool argo::iridium::xfloat::XFloatDiveData::HasDiveData () const
```

Does this dive data object have any dive data?

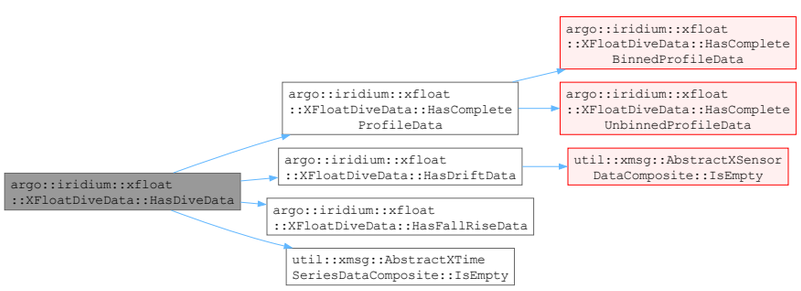
Returns

TRUE if this dive data object has any dive data, or FALSE otherwise.

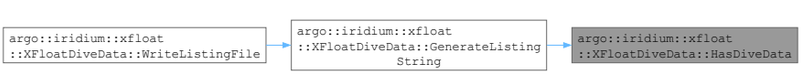
References HasCompleteProfileData(), HasDriftData(), HasFallRiseData(), util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLAS
util::thread::RecursiveLockable::mutex_, and xmsg_surface_time_series_.

Referenced by GenerateListingString().

Here is the call graph for this function:



Here is the caller graph for this function:



10.87.3.80 HasDriftData()

```
bool argo::iridium::xfloat::XFloatDiveData::HasDriftData () const
```

Does this dive data object have any drift data?

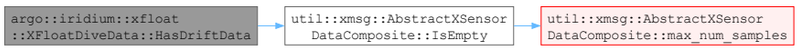
Returns

TRUE if this dive data object has any profile data, or FALSE otherwise.

References util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::IsEmpty(), util::thread::RecursiveLockable::mutex_, and xmsg_composite_drift_data_.

Referenced by GenerateListingString(), and HasDiveData().

Here is the call graph for this function:



Here is the caller graph for this function:

10.87.3.81 HasEngineeringData()

```
bool argo::iridium::xfloat::XFloatDiveData::HasEngineeringData () const
```

Does this dive data object have any engineering data?

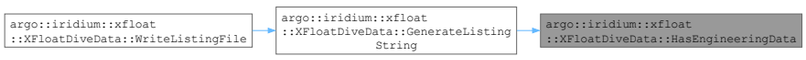
Returns

TRUE if this dive data object has any engineering data, or FALSE otherwise.

References util::thread::RecursiveLockable::mutex_, xmsg_engineering_data_abort_, xmsg_engineering_data_ascent_, xmsg_engineering_data_bit_fail_, xmsg_engineering_data_bit_passed_, xmsg_engineering_data_descent_, xmsg_engineering_data_dia xmsg_engineering_data_drift_, xmsg_engineering_data_standard_dive_, and xmsg_engineering_data_surface_.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.87.3.82 HasFallRiseData()

```
bool argo::iridium::xfloat::XFloatDiveData::HasFallRiseData () const
```

Does this dive data object have any rise or fall data?

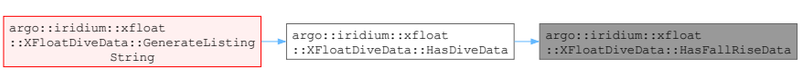
Returns

TRUE if this dive data object has any rise or fall data, or FALSE otherwise.

References util::thread::RecursiveLockable::mutex_, xmsg_fall_time_series_, and xmsg_rise_time_series_.

Referenced by HasDiveData().

Here is the caller graph for this function:



10.87.3.83 HasPositionData()

```
bool argo::iridium::xfloat::XFloatDiveData::HasPositionData () const
```

Does this dive data object have any position data?

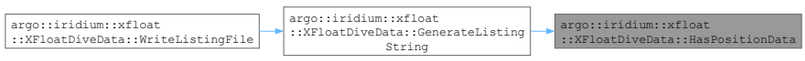
Returns

TRUE if this dive data object has any position data, or FALSE otherwise.

References mail_messages_, util::thread::RecursiveLockable::mutex_, and xmsg_gps_positions_.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.87.3.84 HasUnbinnedProfileData()

```
bool argo::iridium::xfloat::XFloatDiveData::HasUnbinnedProfileData () const
```

Does this dive data object have any unbinned profile data?

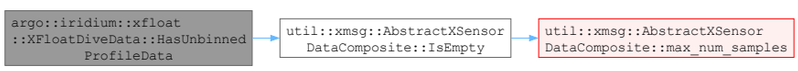
Returns

TRUE if this dive data object has any unbinned profile data, or FALSE otherwise.

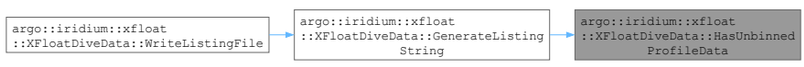
References util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::IsEmpty(), util::thread::RecursiveLockable::mutex_, and xmsg_composite_profile_unbinned_.

Referenced by GenerateListingString().

Here is the call graph for this function:



Here is the caller graph for this function:



10.87.3.85 HasValidGpsPosition()

```
bool argo::iridium::xfloat::XFloatDiveData::HasValidGpsPosition () const
```

Does this dive data object have a valid GPS position for this dive's profile data?

This function checks if the dive data object has a valid GPS position that accurately represents the position and date/time of the profile data for this dive.

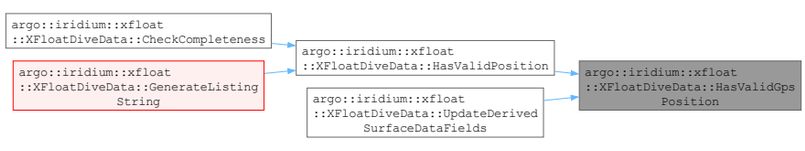
Returns

TRUE if this dive data object has a valid GPS position, or FALSE otherwise.

References CATCH_BLOCK, and util::thread::RecursiveLockable::mutex_.

Referenced by HasValidPosition(), and UpdateDerivedSurfaceDataFields().

Here is the caller graph for this function:



10.87.3.86 HasValidIridiumPosition()

```
bool argo::iridium::xfloat::XFloatDiveData::HasValidIridiumPosition () const
```

Does this dive data object have a valid Iridium position?

This function checks if the dive data object has a valid Iridium position that accurately represents the position and date/time of the profile data for this dive cycle.

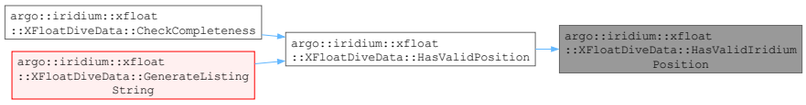
Returns

TRUE if this dive data object has a valid composite Iridium position, or FALSE otherwise.

Todo Implement the functionality to determine if a dive data object has a valid Iridium position that accurately represents the position and date/time of the profile data for this dive cycle.

Referenced by HasValidPosition().

Here is the caller graph for this function:



10.87.3.87 HasValidPosition()

```
bool argo::iridium::xfloat::XFloatDiveData::HasValidPosition () const
```

Does this dive data object have a valid position for this dive's profile data?

Check if we have either a valid GPS position, or a valid composite Iridium position to represent the profile location.

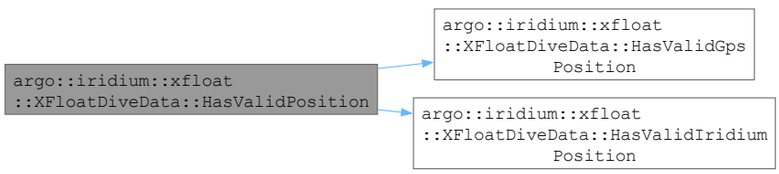
Returns

TRUE if this dive data object has a valid position for this dive's profile data, or FALSE otherwise.

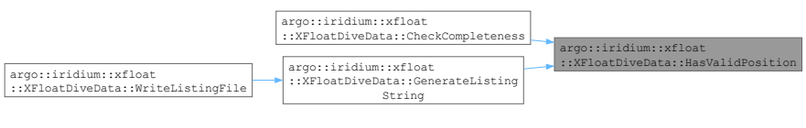
References HasValidGpsPosition(), and HasValidIridiumPosition().

Referenced by CheckCompleteness(), and GenerateListingString().

Here is the call graph for this function:



Here is the caller graph for this function:



10.87.3.88 id_dac()

```
int argo::iridium::Iridium::id_dac () const  [inline], [inherited]
```

Get the unique Data Assembly Center (DAC) identifier.

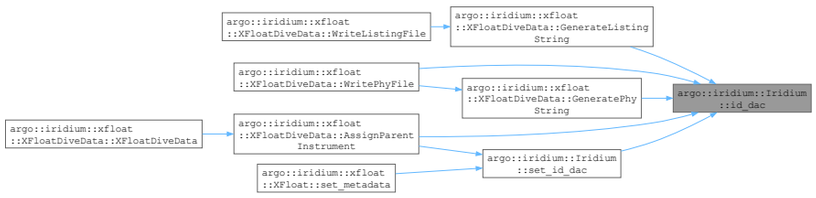
Returns

The unique Data Assembly Center (DAC) identifier.

References argo::iridium::Iridium::id_dac_, and util::thread::RecursiveLockable::mutex_.

Referenced by AssignParentInstrument(), GenerateListingString(), GeneratePhyString(), argo::iridium::Iridium::set_id_dac(), and WritePhyFile().

Here is the caller graph for this function:



10.87.3.89 id_dive()

```
int argo::iridium::xfloat::XFloatDiveData::id_dive () const  [inline]
```

Get the dive or cycle number of this dive data object.

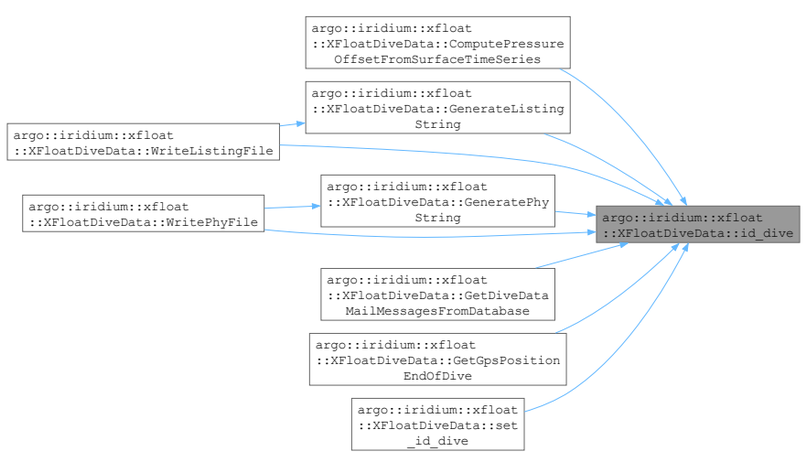
Returns

The dive or cycle number of this dive data object.

References id_dive_, and util::thread::RecursiveLockable::mutex_.

Referenced by ComputePressureOffsetFromSurfaceTimeSeries(), GenerateListingString(), GeneratePhyString(), GetDiveDataMailMessagesFromDatabase(), GetGpsPositionEndOfDive(), set_id_dive(), WriteListingFile(), and WritePhyFile().

Here is the caller graph for this function:



10.87.3.90 id_imei()

```
long argo::iridium::Iridium::id_imei () const  [inline], [inherited]
```

Get the unique International Mobile Equipment Identifier (IMEI).

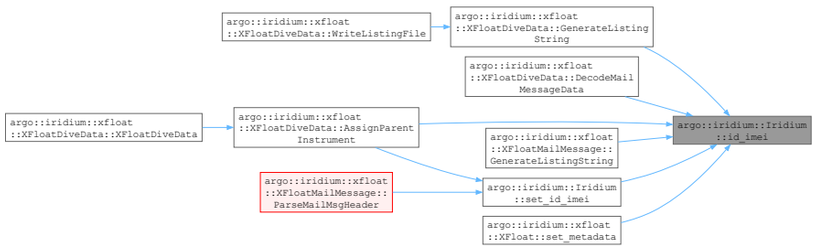
Returns

The unique International Mobile Equipment Identifier (IMEI).

References argo::iridium::Iridium::id_imei_, and util::thread::RecursiveLockable::mutex_.

Referenced by AssignParentInstrument(), DecodeMailMessageData(), GenerateListingString(), argo::iridium::xfloat::XFloatMailMessage::
argo::iridium::Iridium::set_id_imei(), and argo::iridium::xfloat::XFloat::set_metadata().

Here is the caller graph for this function:



10.87.3.91 id_instrument()

```
int argo::iridium::Iridium::id_instrument () const  [inline], [inherited]
```

Get the unique instrument identifier or instrument serial number.

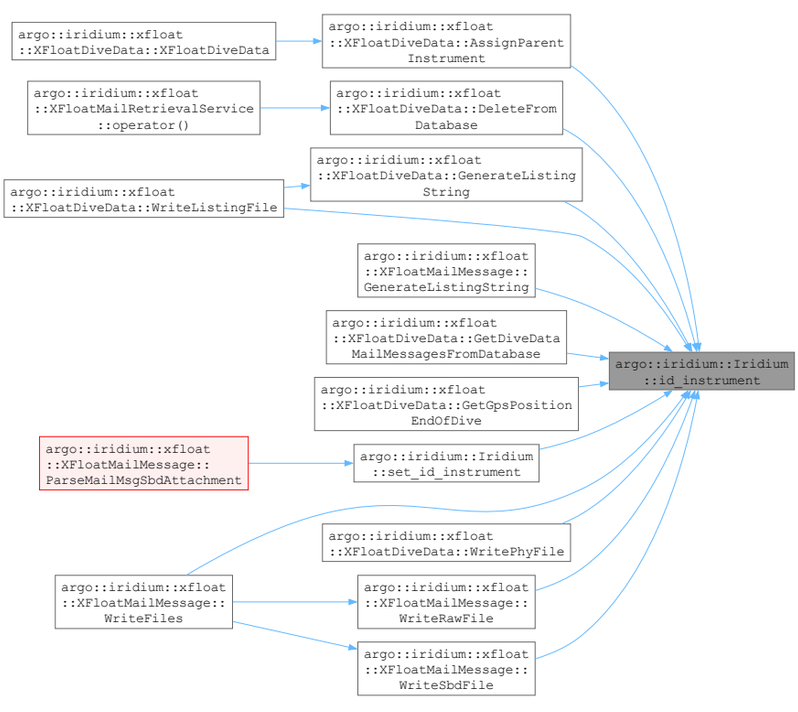
Returns

The unique instrument identifier or instrument serial number.

References argo::iridium::Iridium::id_instrument_, and util::thread::RecursiveLockable::mutex_.

Referenced by AssignParentInstrument(), DeleteFromDatabase(), GenerateListingString(), argo::iridium::xfloat::XFloatMailMessage::Gene
GetDiveDataMailMessagesFromDatabase(), GetGpsPositionEndOfDive(), argo::iridium::Iridium::set_id_instrument(),
argo::iridium::xfloat::XFloatMailMessage::WriteFiles(), WriteListingFile(), WritePhyFile(), argo::iridium::xfloat::XFloatMailMessage::WriteR
and argo::iridium::xfloat::XFloatMailMessage::WriteSbdFile().

Here is the caller graph for this function:



10.87.3.92 id_wmo()

```
int argo::iridium::Iridium::id_wmo () const  [inline], [inherited]
```

Get the unique World Meteorological Organization (WMO) identifier.

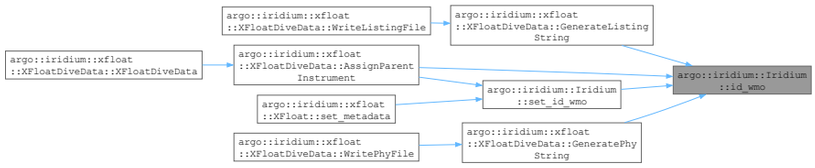
Returns

The unique World Meteorological Organization (WMO) identifier.

References argo::iridium::Iridium::id_wmo_, and util::thread::RecursiveLockable::mutex_.

Referenced by AssignParentInstrument(), GenerateListingString(), GeneratePhyString(), and argo::iridium::Iridium::set_id_wmo().

Here is the caller graph for this function:



10.87.3.93 intermediate_listing_file_path()

```
const path & argo::iridium::xfloat::XFloatDiveData::intermediate_listing_file_path () const  [inline]
```

Get the path to the intermediate listing file on the local filesystem.

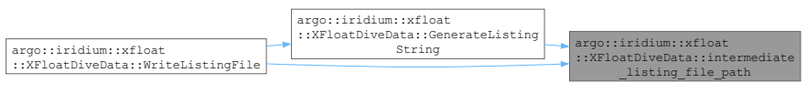
Returns

The path to the intermediate listing file on the local filesystem.

References listing_file_path_, and util::thread::RecursiveLockable::mutex_.

Referenced by GenerateListingString(), and WriteListingFile().

Here is the caller graph for this function:



10.87.3.94 ListDirectories()

```
static vector< path > util::file::FileSystemUtils::ListDirectories (
            const path & dir)  [inline], [static], [inherited]
```

List all directories contained in the given directory.

Parameters

in	*dir*	The path to the directory of interest.

Exceptions

log::MessageException	if the directory does not exist, is not a directory, or an error occurs.

Returns

A vector of paths to all the directories contained in the given directory.

References CATCH_BLOCK, and EXC_THROW.

10.87.3.95 ListFiles()

```
static vector< path > util::file::FileSystemUtils::ListFiles (
            path dir,
            size_t n = 0)  [inline], [static], [inherited]
```

List all regular files contained in the given directory.

Parameters

in	*dir*	The path to the directory of interest.
in	*n*	If n>0, the maximum number of paths to return, else no effect.

Exceptions

log::MessageException	if the directory does not exist, is not a directory, or an error occurs.

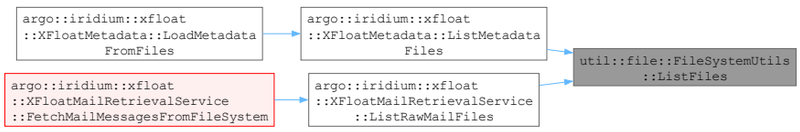
Returns

A vector of paths to all the regular files contained in the given directory.

References CATCH_BLOCK, and EXC_THROW.

Referenced by argo::iridium::xfloat::XFloatMetadata::ListMetadataFiles(), and argo::iridium::xfloat::XFloatMailRetrievalService::ListRawMa

Here is the caller graph for this function:



10.87.3.96 LockObject()

```
bool util::thread::RecursiveLockable::LockObject (
            bool block)  [inline], [inherited]
```

Lock the object.

Parameters

in	*block*	If TRUE, then the calling thread will block until the lock becomes available.

Exceptions

util::log::MessageException	if an error occurs.

Returns

TRUE if the lock was successfully obtained, or FALSE if block is FALSE and another thread owns the lock on the mutex.

References CATCH_BLOCK, and util::thread::RecursiveLockable::mutex_.

10.87.3.97 MoveFileToDirectory()

```
static path util::file::FileSystemUtils::MoveFileToDirectory (
            const path & file,
            const path & dir_dst,
            bool create_dir = false)  [inline], [static], [inherited]
```

Copy a file into a directory.

Warning

Despite its name, this function copies the file and does not remove the source. If a file with the same name already exists in the destination directory, it is left unchanged. It is not currently called.

Parameters

in	*file*	The file to copy.
in	*dir_dst*	The destination directory.
in	*create_dir*	If TRUE, create the destination directory if it does not exist.

Exceptions

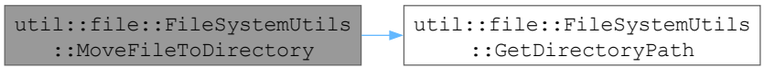
util::log::MessageException	if the file does not exist, is not a regular file, or cannot be copied.

Returns

The path of the file in the destination directory.

References CATCH_BLOCK, EXC_THROW, and util::file::FileSystemUtils::GetDirectoryPath().

Here is the call graph for this function:



10.87.3.98 phy_file_path()

```
const path & argo::iridium::xfloat::XFloatDiveData::phy_file_path () const  [inline]
```

Get the path to the PHY listing file on the local filesystem.

Returns

The path to the PHY listing file on the local filesystem.

References util::thread::RecursiveLockable::mutex_, and phy_file_path_.

Referenced by WritePhyFile().

Here is the caller graph for this function:

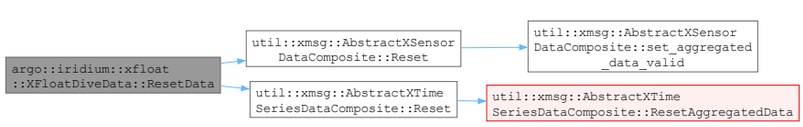
10.87.3.99 ResetData()

```
void argo::iridium::xfloat::XFloatDiveData::ResetData ()
```

Todo Implement NC variable generation for XFloatDiveData

References CATCH_BLOCK, mail_messages_, util::thread::RecursiveLockable::mutex_, util::xmsg::AbstractXSensorDataComposite< PF
util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::Reset(), xmsg_acoustic_positions_, xmsg_argo_data_,
xmsg_composite_drift_data_, xmsg_composite_profile_binned_, xmsg_composite_profile_unbinned_, xmsg_engineering_data_abort_,
xmsg_engineering_data_ascent_, xmsg_engineering_data_bit_fail_, xmsg_engineering_data_bit_passed_, xmsg_engineering_data_des
xmsg_engineering_data_diagnostic_dive_, xmsg_engineering_data_drift_, xmsg_engineering_data_ice_, xmsg_engineering_data_stand
xmsg_engineering_data_surface_, xmsg_fall_time_series_, xmsg_gps_positions_, xmsg_mission_cmd_echoback_,
xmsg_pump_time_series_, xmsg_rise_time_series_, xmsg_surface_time_series_, and xmsg_valve_data_.

Here is the call graph for this function:



10.87.3.100 SaveObjectToDatabase()

```
void argo::iridium::xfloat::XFloatDiveData::SaveObjectToDatabase () const  [virtual]
```

Save this dive data object to the SQL database.

The parent instrument object is saved first, so that its row exists before the dive row that references it.

The parent instrument object is saved first, so that its row exists before the dive row that references it. Otherwise, the dive data for a float that is not yet in the database would fail the foreign key constraint.

Exceptions

util::log::MessageException	if an error occurs while saving the dive data or GPS message objects.

Implements util::mysql::LocalPersistentEntity.

References CATCH_BLOCK, CATCH_NOTHROW, ldbg, util::thread::RecursiveLockable::mutex_, sql_table_description_, and xmsg_gps_positions_.

10.87.3.101 SelectObjectFromDatabase()

```
static shared_ptr< XFloatDiveData > argo::iridium::xfloat::XFloatDiveData::SelectObjectFrom↩
Database (
            const string & where = "",
            const string & groupBy = "",
            const string & orderBy = "",
            const unsigned int & offset = 0)  [inline], [static]
```

Select a single dive data object from the SQL database table.

Parameters

in	*where*	The SQL select statement "where-condition" clause.
in	*groupBy*	The SQL select statement "group-by expression" clause.
in	*orderBy*	The SQL select statement "order-by expression" clause.
in	*offset*	The SQL select statement result set row offset value.

Exceptions

util::log::MessageException	if the query fails.

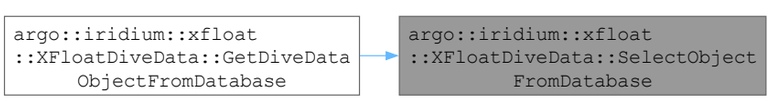
Returns

The first dive data object matching the query.

References CATCH_BLOCK, sql_table_description_, and std::make_shared.

Referenced by GetDiveDataObjectFromDatabase().

Here is the caller graph for this function:



10.87.3.102 SelectObjectsFromDatabase()

```
static vector< shared_ptr< XFloatDiveData > > argo::iridium::xfloat::XFloatDiveData::Select↩
ObjectsFromDatabase (
            const string & where = "",
            const string & groupBy = "",
            const string & orderBy = "",
            const unsigned int & limit = 0,
            const unsigned int & offset = 0)  [inline], [static]
```

Select dive data objects from the SQL database table.

Parameters

in	*where*	The SQL select statement "where-condition" clause.
in	*groupBy*	The SQL select statement "group-by expression" clause.
in	*orderBy*	The SQL select statement "order-by expression" clause.
in	*limit*	The SQL select statement result set row limit value (0 = no limit).
in	*offset*	The SQL select statement result set row offset value.

Exceptions

util::log::MessageException	if the query fails.

Returns

The dive data objects matching the query.

References CATCH_BLOCK, and sql_table_description_.

10.87.3.103 SelectValueFromDatabase()

```
template<typename TT >
static TT argo::iridium::xfloat::XFloatDiveData::SelectValueFromDatabase (
            const string & what,
            const string & where = "",
            const string & groupBy = "",
            const string & orderBy = "",
            const unsigned int & offset = 0)  [inline], [static]
```

Select a single value from the dive data SQL database table.

Template Parameters

TT	The type of the value to select.

Parameters

in	*what*	The SQL select statement "what" clause (a single column or expression).
in	*where*	The SQL select statement "where-condition" clause.
in	*groupBy*	The SQL select statement "group-by expression" clause.
in	*orderBy*	The SQL select statement "order-by expression" clause.
in	*offset*	The SQL select statement result set row offset value.

Exceptions

util::log::MessageException	if the query fails.

Returns

The value from the first matching row.

References sql_table_description_.

10.87.3.104 SensorGainsAndOffsetsAreKnown()

```
bool argo::iridium::xfloat::XFloatDiveData::SensorGainsAndOffsetsAreKnown () const
```

Are the sensor gains and offsets known for this dive data object.

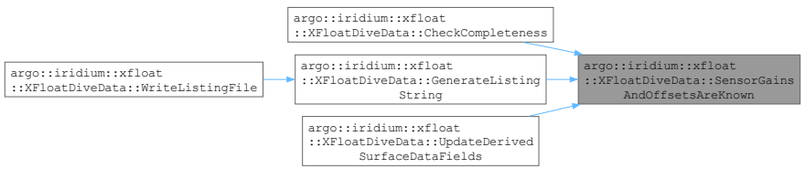
Returns

TRUE if the gains and offsets are known, or FALSE otherwise.

References util::thread::RecursiveLockable::mutex_, and xmsg_argo_data_.

Referenced by CheckCompleteness(), GenerateListingString(), and UpdateDerivedSurfaceDataFields().

Here is the caller graph for this function:



10.87.3.105 set_argo_instrument_type()

```
void argo::iridium::xfloat::XFloatDiveData::set_argo_instrument_type (
            XArgoInstrumentType_t type)  [inline], [private]
```

Set the enumerated Argo instrument type identifier.

Parameters

in	*type*	The enumerated Argo instrument type identifier.

References argo_instrument_type_, and util::thread::RecursiveLockable::mutex_.

10.87.3.106 set_id_dac()

```
void argo::iridium::Iridium::set_id_dac (
            int id_dac)  [inline], [protected], [inherited]
```

Set the unique Data Assembly Center (DAC) identifier.

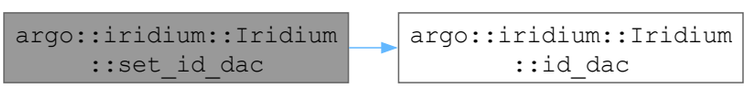
Parameters

in	*id_dac*	The unique Data Assembly Center (DAC) identifier.

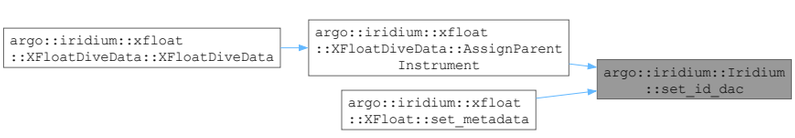
References argo::iridium::Iridium::id_dac(), argo::iridium::Iridium::id_dac_, and util::thread::RecursiveLockable::mutex_.

Referenced by AssignParentInstrument(), and argo::iridium::xfloat::XFloat::set_metadata().

Here is the call graph for this function:



Here is the caller graph for this function:



10.87.3.107 set_id_dive()

```
void argo::iridium::xfloat::XFloatDiveData::set_id_dive (
            int id_dive)  [inline], [private]
```

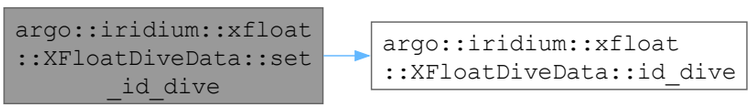
Set the dive or cycle number of this dive data object.

Parameters

in	*id_dive*	The dive or cycle number.

References id_dive(), id_dive_, and util::thread::RecursiveLockable::mutex_.

Here is the call graph for this function:



10.87.3.108 set_id_imei()

```
void argo::iridium::Iridium::set_id_imei (
            long id_imei)  [inline], [protected], [inherited]
```

Set the unique International Mobile Equipment Identifier (IMEI).

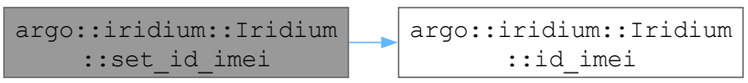
Parameters

in	*id_imei*	The unique International Mobile Equipment Identifier (IMEI).

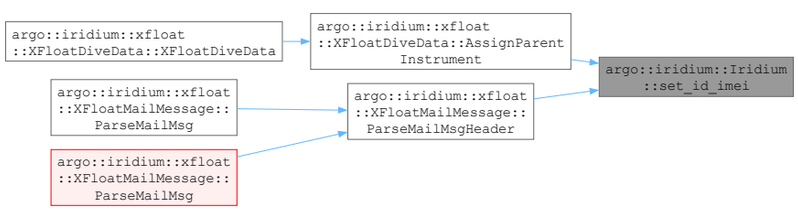
References argo::iridium::Iridium::id_imei(), argo::iridium::Iridium::id_imei_, and util::thread::RecursiveLockable::mutex_.

Referenced by AssignParentInstrument(), and argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgHeader().

Here is the call graph for this function:



Here is the caller graph for this function:



10.87.3.109 set_id_instrument()

```
void argo::iridium::Iridium::set_id_instrument (
            int id_instrument)  [inline], [protected], [inherited]
```

Set the unique instrument identifier or instrument serial number.

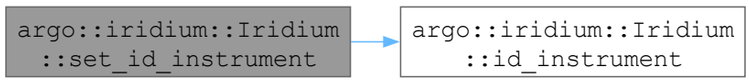
Parameters

in	*id_instrument*	The unique instrument identifier or instrument serial number

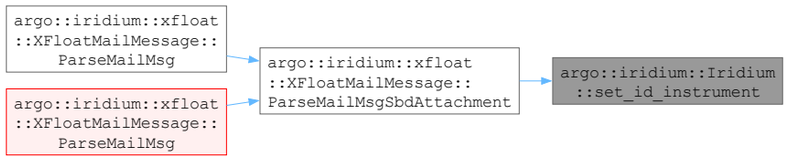
References argo::iridium::Iridium::id_instrument(), argo::iridium::Iridium::id_instrument_, and util::thread::RecursiveLockable::mutex_.

Referenced by argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgSbdAttachment().

Here is the call graph for this function:



Here is the caller graph for this function:



10.87.3.110 set_id_wmo()

```
void argo::iridium::Iridium::set_id_wmo (
            int id_wmo)  [inline], [protected], [inherited]
```

Set the unique World Meteorological Organization (WMO) identifier.

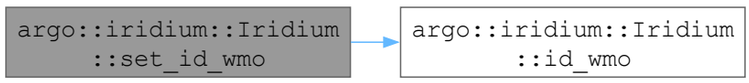
Parameters

in	*id_wmo*	The unique World Meteorological Organization (WMO) identifier.

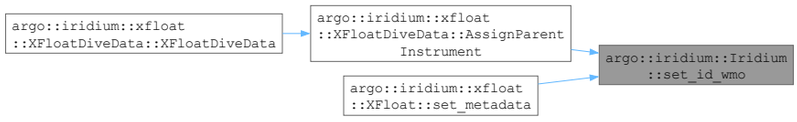
References argo::iridium::Iridium::id_wmo(), argo::iridium::Iridium::id_wmo_, and util::thread::RecursiveLockable::mutex_.

Referenced by AssignParentInstrument(), and argo::iridium::xfloat::XFloat::set_metadata().

Here is the call graph for this function:



Here is the caller graph for this function:



10.87.3.111 set_intermediate_listing_file_path()

```
void argo::iridium::xfloat::XFloatDiveData::set_intermediate_listing_file_path (
            const path & file_path)  [inline], [private]
```

Set the path to the intermediate listing file.

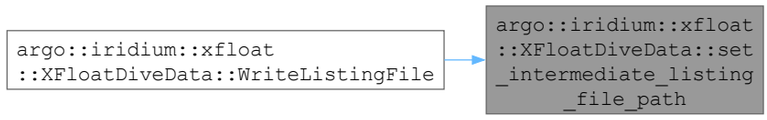
Parameters

in	*file_path*	The path to the intermediate listing file.

References listing_file_path_, and util::thread::RecursiveLockable::mutex_.

Referenced by WriteListingFile().

Here is the caller graph for this function:



10.87.3.112 set_phy_file_path()

```
void argo::iridium::xfloat::XFloatDiveData::set_phy_file_path (
            const path & file_path)  [inline], [private]
```

Set the path to the PHY listing file.

Parameters

in	*file_path*	The path to the PHY listing file.

References util::thread::RecursiveLockable::mutex_, and phy_file_path_.

Referenced by WritePhyFile().

Here is the caller graph for this function:

10.87.3.113 UnlockObject()

```
void util::thread::RecursiveLockable::UnlockObject ()  [inline], [inherited]
```

Unlock the object.

Exceptions

util::log::MessageException	if an error occurs.

References CATCH_BLOCK, and util::thread::RecursiveLockable::mutex_.

10.87.3.114 UpdateDerivedSurfaceDataFields()

```
void argo::iridium::xfloat::XFloatDiveData::UpdateDerivedSurfaceDataFields ()  [inline], [private]
```

Update the derived surface pressure fields from the surface time-series data.

Computes the surface pressure maximum, mean, minimum, number of samples and variance from the surface time series (see xmsg_surface_time_series_), using the surface sensor gains and offsets. These fields feed the surface pressure offset correction (see ComputePressureOffsetFromSurfaceTimeSeries()).

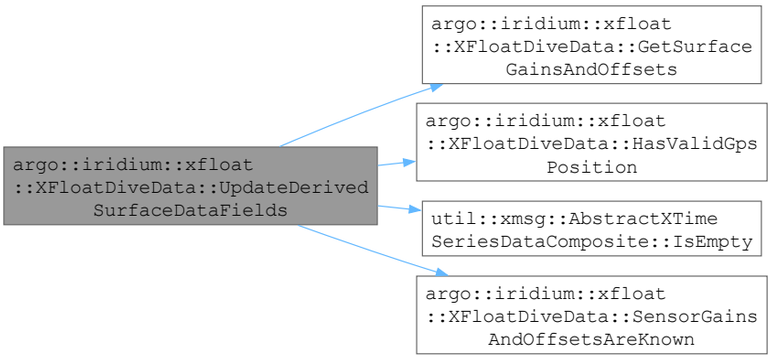
The fields are set to XMSG_SENS_BAD_PRES and marked invalid if the sensor gains and offsets are unknown, there is no surface data, or the mean surface pressure is outside ±5 dbar. Otherwise they are set, and marked valid only if the surface time series is complete and the dive has a valid GPS position.

Exceptions

util::log::MessageException	if an error occurs.

References CATCH_BLOCK, GetSurfaceGainsAndOffsets(), HasValidGpsPosition(), util::xmsg::AbstractXTimeSeriesDataComposite< M ldbg, util::thread::RecursiveLockable::mutex_, SensorGainsAndOffsetsAreKnown(), XMSG_SENS_BAD_PRES, and xmsg_surface_time_series_.

Here is the call graph for this function:



10.87.3.115 WriteListingFile()

```
void argo::iridium::xfloat::XFloatDiveData::WriteListingFile ()
```

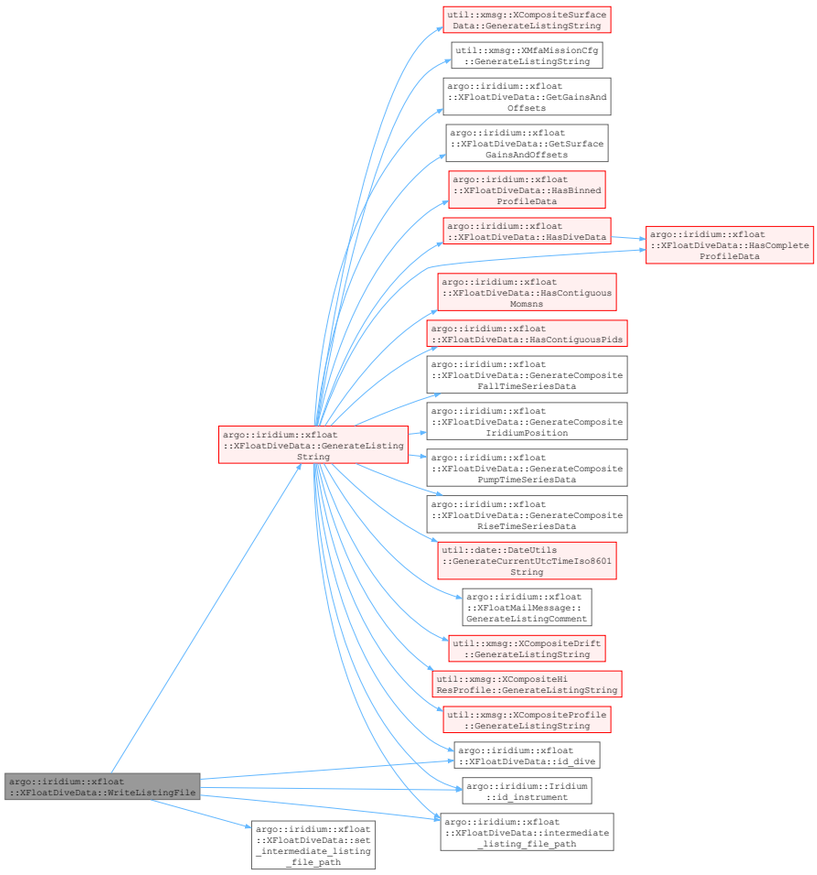
Write the listing file for this dive.

Exceptions

util::log::MessageException	if an error occurs while writing the listing file.

References CATCH_BLOCK, GenerateListingString(), id_dive(), argo::iridium::Iridium::id_instrument(), intermediate_listing_file_generatio intermediate_listing_file_path(), linf, util::thread::RecursiveLockable::mutex_, and set_intermediate_listing_file_path().

Here is the call graph for this function:



10.87.3.116 WritePhyFile()

```
void argo::iridium::xfloat::XFloatDiveData::WritePhyFile ()
```

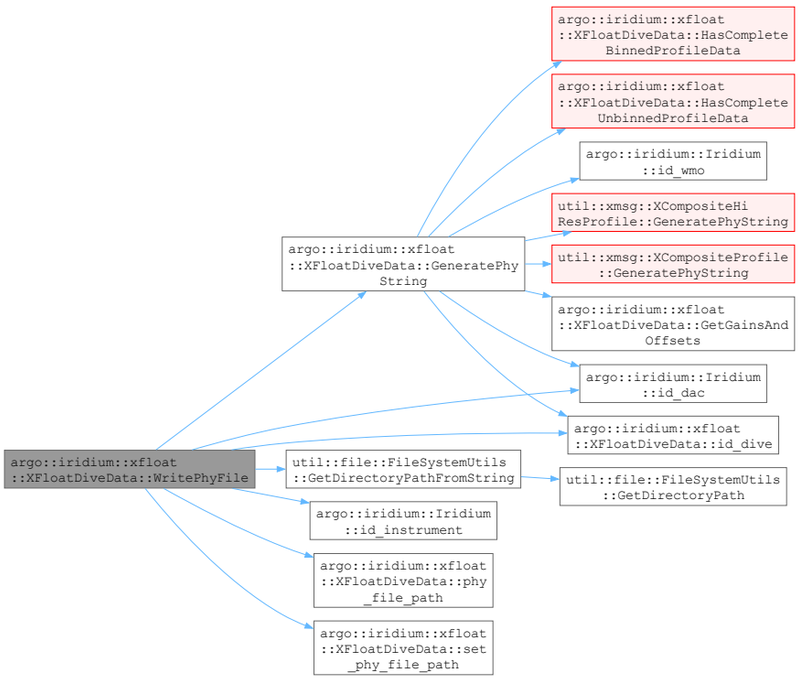
Write the PHY file for this dive.

Exceptions

util::log::MessageException	if an error occurs while writing the PHY file.

References CATCH_BLOCK, GeneratePhyString(), util::file::FileSystemUtils::GetDirectoryPathFromString(), argo::iridium::Iridium::id_dac(id_dive(), argo::iridium::Iridium::id_instrument(), linf, util::thread::RecursiveLockable::mutex_, parent_instrument_, phy_directory_path_, phy_file_generation_is_enabled_, phy_file_path(), and set_phy_file_path().

Here is the call graph for this function:



10.87.4 Friends And Related Symbol Documentation

10.87.4.1 std::make_shared

```
shared_ptr< XFloatDiveData > std::make_shared ()  [friend]
```

Friend function for creating an X-Float object using the private constructor and a shared pointer to that object.

Referenced by GenerateCompositeFallTimeSeriesData(), GenerateCompositeIridiumPosition(), GenerateCompositePumpTimeSeriesDat
GenerateCompositeRiseTimeSeriesData(), GetGpsPositionEndOfDive(), and SelectObjectFromDatabase().

10.87.4.2 util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription

```
options_description util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription
()  [friend]
```

Friend function for populating some of the static data members of this class from the configuration file.

10.87.5 Member Data Documentation

10.87.5.1 address_

```
std::string util::mysql::LocalMysqlConfiguration::address_  [static], [private], [inherited]
```

The SQL server qualified name or IP address string.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.87.5.2 argo_instrument_type_

```
XArgoInstrumentType_t argo::iridium::xfloat::XFloatDiveData::argo_instrument_type_  [private]
```

The enumerated Argo instrument type identifier.

This field is used to identify the specific type of instrument i.e. Solo-II, Deep Argo, or ALAMO/Hurricane floats. The value of this field may affect the behavior of some aspects of the application. For example, the manner in which some Iridium Short Burst Data (SBD) messages are decoded is dependent on the specific type of instrument.

Referenced by argo_instrument_type(), and set_argo_instrument_type().

10.87.5.3 complete_

```
bool argo::iridium::xfloat::XFloatDiveData::complete_  [private]
```

The dive-data-is-complete flag.

The value of this field indicates whether the dive data object is complete. The completeness of the dive data is determined by CheckCompleteness() which is invoked after each new mail message is decoded and assigned to this dive data object.

The dive data is determined to be complete if the dive data object meets all of the following criteria: it contains a complete set of profile data, the sensor gains and offsets for this dive are known (i.e. the Argo data message has been received), it contains a set of mail messages with contiguous packet identifiers (PIDs) starting at PID=0, and it contains a valid position for the profile. These criteria is determined by the functions HasCompleteProfileData(), SensorGainsAndOffsetsAreKnown(), HasContiguousPids(), HasValidPosition(), respectively.

10.87.5.4 id_dac_

```
int argo::iridium::Iridium::id_dac_  [private], [inherited]
```

The unique Data Assembly Center (DAC) identifier.

Referenced by argo::iridium::Iridium::id_dac(), and argo::iridium::Iridium::set_id_dac().

10.87.5.5 id_dive_

```
int argo::iridium::xfloat::XFloatDiveData::id_dive_  [private]
```

The dive or cycle number of this dive data object.

This field is used to index the dive data object for a given instrument. Together, the unique instrument identifier and the dive or cycle number provide a unique index of the dive data object.

Referenced by id_dive(), and set_id_dive().

10.87.5.6 id_imei_

```
long argo::iridium::Iridium::id_imei_  [private], [inherited]
```

The unique International Mobile Equipment Identifier (IMEI).

Referenced by argo::iridium::Iridium::id_imei(), and argo::iridium::Iridium::set_id_imei().

10.87.5.7 id_instrument_

```
int argo::iridium::Iridium::id_instrument_  [private], [inherited]
```

The unique instrument identifier or instrument serial number.

Referenced by argo::iridium::Iridium::id_instrument(), and argo::iridium::Iridium::set_id_instrument().

10.87.5.8 id_wmo_

```
int argo::iridium::Iridium::id_wmo_  [private], [inherited]
```

The unique World Meteorological Organization (WMO) identifier.

Referenced by argo::iridium::Iridium::id_wmo(), and argo::iridium::Iridium::set_id_wmo().

10.87.5.9 intermediate_listing_directory_path_

path argo::iridium::xfloat::XFloatDiveData::intermediate_listing_directory_path_ [static], [private]

The intermediate listing directory path on the local filesystem.

The value of this field determines the base directory to which intermediate listing files will be written. Intermediate listing files will be written to subdirectories of this path, which are organized by the unique instrument identifiers. Intermediate listing files will be written each time a new message is received from an instrument.

The value of this field is set as a result of parsing the command-line arguments and/or configuration file options during application initialization.

10.87.5.10 intermediate_listing_file_generation_is_enabled_

bool argo::iridium::xfloat::XFloatDiveData::intermediate_listing_file_generation_is_enabled_↩ [static], [private]

The global intermediate-listing-file-generation-is-enabled flag.

The value of this field determines whether or not any intermediate listing files will be written to the local filesystem. If TRUE, the intermediate listing files will be written as new messages are received by the application. Otherwise, if FALSE, no intermediate listing files will be written to the local filesystem.

The value of this field is set as a result of parsing the command-line arguments and/or configuration file options during application initialization.

Referenced by WriteListingFile().

10.87.5.11 iridium_position_datetime_first_

ptime argo::iridium::xfloat::XFloatDiveData::iridium_position_datetime_first_ [private]

The date/time of transmission of the first Iridium message sent by the instrument for this dive.

This field is updated as a result of assigning a mail message to this dive data object by calling the function AssignMailMessage(const shared_ptr<XFloatMailMessage>& x).

10.87.5.12 iridium_position_datetime_last_

ptime argo::iridium::xfloat::XFloatDiveData::iridium_position_datetime_last_ [private]

The date/time of transmission of the last Iridium message sent by the instrument for this dive.

This field is updated as a result of assigning a mail message to this dive data object by calling the function AssignMailMessage(const shared_ptr<XFloatMailMessage>& x).

10.87.5.13 listing_file_path_

path argo::iridium::xfloat::XFloatDiveData::listing_file_path_ [private]

The path to the intermediate listing file for this dive data on the local filesystem.

This field is set as a result of writing the intermediate listing file i.e. by calling WriteListingFile(). This path is relative to the listing directory, which is specified by the static data member intermediate_listing_directory_path_.

Referenced by intermediate_listing_file_path(), and set_intermediate_listing_file_path().

10.87.5.14 listing_file_updated_

bool argo::iridium::xfloat::XFloatDiveData::listing_file_updated_ [private]

The listing-file-is-updated flag.

10.87.5.15 mail_messages_

static_vector<shared_ptr<XFloatMailMessage>, XMSG_MAX_MAIL> argo::iridium::xfloat::XFloatDive↩
Data::mail_messages_ [private]

The mail messages for this dive data object.

This vector contains a set of shared reference to the mail messages that contain the telemetry data for this dive. Each time a mail message is received by the telemetry decoder thread, it is assigned to the corresponding dive data object before the thread goes on to decode the message.

This set of mail message may be used to reprocess the telemetry data for a dive, if necessary. For example, if a new Argo message arrives with different sensor gain and offset values than the previous dive, then the sensor data messages may be reprocessed using the new sensor gain and offset values.

Referenced by AssignMailMessage(), GenerateCompositeIridiumPosition(), GenerateListingString(), GenerateOrderedMomsnList(), GenerateOrderedPidList(), GeneratePhyString(), HasPositionData(), and ResetData().

10.87.5.16 mutex_

recursive_mutex util::thread::RecursiveLockable::mutex_ [mutable], [protected], [inherited]

Recursive mutual exclusion lock for synchronizing operations.

Referenced by util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AggregateS
argo::iridium::xfloat::XFloat::argo_instrument_type(), argo_instrument_type(), AssignArgoData(), AssignAuxTemperatureBinned(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignAuxTemperatureMe
AssignDriftPressure(), AssignDriftSalinity(), AssignDriftTemperatre(), AssignEngineeringAbort(), AssignEngineeringAscent(),
AssignEngineeringBitFail(), AssignEngineeringBitPass(), AssignEngineeringDescent(), AssignEngineeringDiagnostic(),
AssignEngineeringDrift(), AssignEngineeringIce(), AssignEngineeringStandard(), AssignEngineeringSurface(),
AssignFallTimeSeries(), AssignGps(), AssignMailMessage(), AssignMissionCmdEchoback(), AssignParentInstrument(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignPressureMessage(
AssignProfilePressureBinned(), AssignProfilePressureUnbinned(), AssignProfileSalinityBinned(), AssignProfileSalinityUnbinned(),
AssignProfileTemperatureBinned(), AssignProfileTemperatureUnbinned(), AssignPumpTimeSeries(), AssignRiseTimeSeries(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignSalinityMessage(),
AssignSurfaceTimeSeries(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >
AssignValveData(), CheckCompleteness(), argo::iridium::xfloat::XFloatMetadata::comment(), ComputePressureOffsetFromSurfaceTimeS
argo::iridium::xfloat::XFloatMetadata::deployer(), argo::iridium::xfloat::XFloatMetadata::deployer_address(), argo::iridium::xfloat::XFloatMe
argo::iridium::xfloat::XFloatMetadata::deployment_datetime(), argo::iridium::xfloat::XFloatMetadata::deployment_datetime_status(),
argo::iridium::xfloat::XFloatMetadata::deployment_latitude(), argo::iridium::xfloat::XFloatMetadata::deployment_longitude(),
argo::iridium::xfloat::XFloatMetadata::deployment_platform(), argo::iridium::xfloat::XFloatMetadata::deployment_position_status(),
argo::iridium::xfloat::XFloatMetadata::deployment_type(), argo::iridium::xfloat::XFloat::fw_version(), argo::iridium::xfloat::XFloat::gain_pres
argo::iridium::xfloat::XFloat::gain_pressure_surface(), argo::iridium::xfloat::XFloat::gain_salinity(), argo::iridium::xfloat::XFloat::gain_salinit
argo::iridium::xfloat::XFloat::gain_temperature(), argo::iridium::xfloat::XFloat::gain_temperature_surface(), GenerateCompositeFallTimeSe
GenerateCompositeIridiumPosition(), GenerateCompositePumpTimeSeriesData(), GenerateCompositeRiseTimeSeriesData(),
GenerateListingString(), util::xmsg::XCompositeDrift::GenerateListingString(), util::xmsg::XCompositeHiResProfile::GenerateListingString
util::xmsg::XCompositeProfile::GenerateListingString(), GenerateOrderedMomsnList(), GeneratePhyString(), util::xmsg::XCompositeDrift:
util::xmsg::XCompositeHiResProfile::GeneratePhyString(), util::xmsg::XCompositeProfile::GeneratePhyString(),
GetDiveDataMailMessagesFromDatabase(), GetGainsAndOffsets(), GetGpsPositionEndOfDive(), GetSurfaceGainsAndOffsets(),
HasBinnedProfileData(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::H
HasContiguousMomsns(), HasContiguousPids(), HasDiveData(), HasDriftData(), HasEngineeringData(), HasFallRiseData(),
HasPositionData(), HasUnbinnedProfileData(), HasValidGpsPosition(), argo::iridium::Iridium::id_dac(), id_dive(),
argo::iridium::xfloat::XFloatMailMessage::id_dive(), argo::iridium::Iridium::id_imei(), argo::iridium::Iridium::id_instrument(),
argo::iridium::xfloat::XFloatMailMessage::id_msg(), argo::iridium::xfloat::XFloatMailMessage::id_msg_uid(), argo::iridium::xfloat::XFloatMa
argo::iridium::xfloat::XFloatMailMessage::id_sbd_mtmsn(), argo::iridium::xfloat::XFloatMailMessage::id_sbd_packet(),
argo::iridium::Iridium::id_wmo(), intermediate_listing_file_path(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CL
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::IsDataComplete(),
argo::iridium::xfloat::XFloatMetadata::local_path(), util::thread::RecursiveLockable::LockObject(), argo::iridium::xfloat::XFloatMetadata::ma
argo::iridium::xfloat::XFloat::metadata(), argo::iridium::xfloat::XFloatMailMessage::msg_body_is_valid(), argo::iridium::xfloat::XFloatMailM
argo::iridium::xfloat::XFloatMailMessage::msg_header_datetime(), argo::iridium::xfloat::XFloatMailMessage::msg_header_from(),
argo::iridium::xfloat::XFloatMailMessage::msg_header_is_valid(), argo::iridium::xfloat::XFloatMailMessage::msg_header_raw_text(),
argo::iridium::xfloat::XFloatMailMessage::msg_header_subject(), argo::iridium::xfloat::XFloatMailMessage::msg_is_valid(),
argo::iridium::xfloat::XFloatMailMessage::msg_raw_text(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, T
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::num_saln_samples(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::num_temp_samples(),
argo::iridium::xfloat::XFloat::offset_pressure(), argo::iridium::xfloat::XFloat::offset_pressure_surface(), argo::iridium::xfloat::XFloat::offset_
argo::iridium::xfloat::XFloat::offset_salinity_surface(), argo::iridium::xfloat::XFloat::offset_temperature(), argo::iridium::xfloat::XFloat::offset
argo::iridium::xfloat::XFloatMailMessage::ParseMailMsg(), argo::iridium::xfloat::XFloatMailMessage::ParseMailMsg(),
argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgBody(), argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgHeader(),
argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgSbdAttachment(), argo::iridium::xfloat::XFloat::phy_file_generation_is_enabled(),

phy_file_path(), argo::iridium::xfloat::XFloatMetadata::principal_investigators(), argo::iridium::xfloat::XFloatMetadata::principal_investigato
argo::iridium::xfloat::XFloatMetadata::principal_investigators_originating_country(), argo::iridium::xfloat::XFloatMailMessage::raw_file_loc
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::Reset(),
ResetData(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SamplePres
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SamplePresAsFloat(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleSaln(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleSalnAsFloat(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleTemp(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleTempAsFloat(),
SaveObjectToDatabase(), argo::iridium::xfloat::XFloatMailMessage::SaveObjectToDatabase(), argo::iridium::xfloat::XFloatMetadata::Save
argo::iridium::xfloat::XFloatMailMessage::sbd_data(), argo::iridium::xfloat::XFloatMailMessage::sbd_data_is_valid(),
argo::iridium::xfloat::XFloatMailMessage::sbd_file_local_path(), argo::iridium::xfloat::XFloatMailMessage::sbd_file_name(),
argo::iridium::xfloat::XFloatMailMessage::sbd_file_size_bytes(), argo::iridium::xfloat::XFloatMailMessage::sbd_session_status_code(),
argo::iridium::xfloat::XFloatMailMessage::sbd_session_status_string(), SensorGainsAndOffsetsAreKnown(), util::xmsg::AbstractXSensorD
argo::iridium::xfloat::XFloat::set_argo_instrument_type(), set_argo_instrument_type(), argo::iridium::xfloat::XFloatMetadata::set_commen
argo::iridium::xfloat::XFloatMetadata::set_deployer(), argo::iridium::xfloat::XFloatMetadata::set_deployer_address(),
argo::iridium::xfloat::XFloatMetadata::set_deployment_cruise_id(), argo::iridium::xfloat::XFloatMetadata::set_deployment_datetime(),
argo::iridium::xfloat::XFloatMetadata::set_deployment_datetime_status(), argo::iridium::xfloat::XFloatMetadata::set_deployment_latitude(
argo::iridium::xfloat::XFloatMetadata::set_deployment_longitude(), argo::iridium::xfloat::XFloatMetadata::set_deployment_platform(),
argo::iridium::xfloat::XFloatMetadata::set_deployment_position_status(), argo::iridium::xfloat::XFloatMetadata::set_deployment_type(),
argo::iridium::xfloat::XFloat::set_fw_version(), argo::iridium::xfloat::XFloat::set_gain_pressure(), argo::iridium::xfloat::XFloat::set_gain_pre
argo::iridium::xfloat::XFloat::set_gain_salinity(), argo::iridium::xfloat::XFloat::set_gain_salinity_surface(), argo::iridium::xfloat::XFloat::set_
argo::iridium::xfloat::XFloat::set_gain_temperature_surface(), argo::iridium::Iridium::set_id_dac(), set_id_dive(),
argo::iridium::xfloat::XFloatMailMessage::set_id_dive(), argo::iridium::Iridium::set_id_imei(), argo::iridium::Iridium::set_id_instrument(),
argo::iridium::xfloat::XFloatMailMessage::set_id_msg(), argo::iridium::xfloat::XFloatMailMessage::set_id_msg_uid(),
argo::iridium::xfloat::XFloatMailMessage::set_id_sbd_momsn(), argo::iridium::xfloat::XFloatMailMessage::set_id_sbd_mtmsn(),
argo::iridium::xfloat::XFloatMailMessage::set_id_sbd_packet(), argo::iridium::Iridium::set_id_wmo(), set_intermediate_listing_file_path(),
argo::iridium::xfloat::XFloatMetadata::set_local_path(), argo::iridium::xfloat::XFloatMetadata::set_manufacturer(),
argo::iridium::xfloat::XFloat::set_metadata(), argo::iridium::xfloat::XFloatMailMessage::set_msg_body_is_valid(),
argo::iridium::xfloat::XFloatMailMessage::set_msg_body_text(), argo::iridium::xfloat::XFloatMailMessage::set_msg_header_datetime(),
argo::iridium::xfloat::XFloatMailMessage::set_msg_header_from(), argo::iridium::xfloat::XFloatMailMessage::set_msg_header_is_valid(),
argo::iridium::xfloat::XFloatMailMessage::set_msg_header_raw_text(), argo::iridium::xfloat::XFloatMailMessage::set_msg_header_subjec
argo::iridium::xfloat::XFloatMailMessage::set_msg_is_valid(), argo::iridium::xfloat::XFloatMailMessage::set_msg_raw_text(),
argo::iridium::xfloat::XFloat::set_offset_pressure(), argo::iridium::xfloat::XFloat::set_offset_pressure_surface(), argo::iridium::xfloat::XFloa
argo::iridium::xfloat::XFloat::set_offset_salinity_surface(), argo::iridium::xfloat::XFloat::set_offset_temperature(),
argo::iridium::xfloat::XFloat::set_offset_temperature_surface(), argo::iridium::xfloat::XFloat::set_phy_file_generation_is_enabled(),
set_phy_file_path(), argo::iridium::xfloat::XFloatMetadata::set_principal_investigators(), argo::iridium::xfloat::XFloatMetadata::set_principa
argo::iridium::xfloat::XFloatMetadata::set_principal_investigators_originating_country(), argo::iridium::xfloat::XFloatMailMessage::set_raw
argo::iridium::xfloat::XFloatMailMessage::set_sbd_data(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_data_is_valid(),
argo::iridium::xfloat::XFloatMailMessage::set_sbd_file_local_path(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_file_name(),
argo::iridium::xfloat::XFloatMailMessage::set_sbd_file_size_bytes(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_session_status_c
argo::iridium::xfloat::XFloatMailMessage::set_sbd_session_status_string(), argo::iridium::xfloat::XFloat::set_start_of_mission_datetime(),
argo::iridium::xfloat::XFloatMetadata::set_wmo_instrument_type(), argo::iridium::xfloat::XFloatMetadata::set_wmo_recorder_type(),
argo::iridium::xfloat::XFloat::set_xmsg_version_major(), argo::iridium::xfloat::XFloat::set_xmsg_version_minor(),
argo::iridium::xfloat::XFloat::start_of_mission_datetime(), util::thread::RecursiveLockable::UnlockObject(), UpdateDerivedSurfaceDataFiel
argo::iridium::xfloat::XFloatMetadata::wmo_instrument_type(), argo::iridium::xfloat::XFloatMetadata::wmo_recorder_type(),
WriteListingFile(), WritePhyFile(), argo::iridium::xfloat::XFloatMailMessage::WriteSbdFile(), argo::iridium::xfloat::XFloat::xmsg_version_ma
and argo::iridium::xfloat::XFloat::xmsg_version_minor().

10.87.5.17 netcdf_directory_path_

`path argo::iridium::xfloat::XFloatDiveData::netcdf_directory_path_ [static], [private]`

The NetCDF directory path on the local filesystem.

The value of this field determines the base directory to which NetCDF files will be written. NetCDF files will be written to subdirectories of this path, which are organized by the unique instrument identifiers. NetCDF files will be written each time a new message is received from an instrument that completes all the necessary data for that dive and NetCDF file generation is enabled for that instrument.

The value of this field is set as a result of parsing the command-line arguments and/or configuration file options during application initialization.

10.87.5.18 netcdf_file_generation_is_enabled_

`bool argo::iridium::xfloat::XFloatDiveData::netcdf_file_generation_is_enabled_ [static], [private]`

The global NetCDF-file-generation-is-enabled flag.

The value of this field determines whether or not any NetCDF files will be written to the local filesystem. If TRUE, then NetCDF files will be written each time a complete set of dive data is received from a deployed instrument and NetCDF file generation is enabled for that instrument. Otherwise, if FALSE, no NetCDF files will be written to the local filesystem.

The value of this field is set as a result of parsing the command-line arguments and/or configuration file options during application initialization.

10.87.5.19 netcdf_file_path_

```
path argo::iridium::xfloat::XFloatDiveData::netcdf_file_path_  [private]
```

The NetCDF dimensions.

The NetCDF file path.

10.87.5.20 netcdf_file_updated_

```
bool argo::iridium::xfloat::XFloatDiveData::netcdf_file_updated_  [private]
```

The NetCDF-file-is-updated flag.

10.87.5.21 netcdf_reference_datetime_

```
const ptime argo::iridium::xfloat::XFloatDiveData::netcdf_reference_datetime_  [static], [private]
```

The NetCDF referecne date/time.

This field is used as the reference date/time for generation of NetCDF files where dates are represented as days since 00:00Z on this date and the fractional remainder represents the time-of-day. The value of this field represents the date/time 1950-01-01T00:00:00Z.

10.87.5.22 num_sbd_msgs_

```
uint16_t argo::iridium::xfloat::XFloatDiveData::num_sbd_msgs_  [private]
```

The number of SBD messages that have been received for this dive cycle.

Referenced by AssignMailMessage().

10.87.5.23 parent_instrument_

```
shared_ptr<XFloat> argo::iridium::xfloat::XFloatDiveData::parent_instrument_  [private]
```

The parent float or instrument object of this dive data object.

This field provides a shared back-reference to the instrument object that owns this dive data object. This field specifies the any-to-one or many-to-one relationship between dive data object and instrument objects.

Every dive data object must have a parent instrument object. In other words, value of this field should never be NULL.

Referenced by AssignArgoData(), AssignParentInstrument(), GenerateListingString(), GeneratePhyString(), and WritePhyFile().

10.87.5.24 password_

```
std::string util::mysql::LocalMysqlConfiguration::password_  [static], [private], [inherited]
```

The SQL server account password string.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.87.5.25 phy_directory_path_

```
path argo::iridium::xfloat::XFloatDiveData::phy_directory_path_  [static], [private]
```

The PHY directory path on the local filesystem.

The value of this field determines the base directory to which PHY files will be written. PHY files will be written to subdirectories of this path, which are organized by the unique instrument identifiers. PHY files will be written each time a new message is received from an instrument that completes all the necessary data for that dive and PHY file generation is enabled for that instrument.

The value of this field is set as a result of parsing the command-line arguments and/or configuration file options during application initialization.

Referenced by WritePhyFile().

10.87.5.26 phy_file_generation_is_enabled_

```
bool argo::iridium::xfloat::XFloatDiveData::phy_file_generation_is_enabled_  [static], [private]
```

The global PHY-file-generation-is-enabled flag.

The value of this field determines whether or not any PHY files will be written to the local filesystem. If TRUE, then PHY files will be written each time a complete set of dive data is received from a deployed instrument and PHY file generation is enabled for that instrument. Otherwise, if FALSE, no PHY files will be written to the local filesystem.

The value of this field is set as a result of parsing the command-line arguments and/or configuration file options during application initialization.

Referenced by WritePhyFile().

10.87.5.27 phy_file_path_

```
path argo::iridium::xfloat::XFloatDiveData::phy_file_path_  [private]
```

The path to the PHY file for this dive data on the local filesystem.

This field is set as a result of writing the PHY listing file i.e. by calling WritePhyFile(). This path is relative to the PHY listing directory, which is specified by the static data member, intermediate_listing_directory_path_.

Referenced by phy_file_path(), and set_phy_file_path().

10.87.5.28 phy_file_updated_

```
bool argo::iridium::xfloat::XFloatDiveData::phy_file_updated_  [private]
```

The phy-file-is-updated flag.

10.87.5.29 port_

```
unsigned int util::mysql::LocalMysqlConfiguration::port_  [static], [private], [inherited]
```

The SQL server port number.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.87.5.30 pres_offs_

```
float argo::iridium::xfloat::XFloatDiveData::pres_offs_  [private]
```

The computed surface pressure offset for this dive.

10.87.5.31 pres_offs_num_dives_

```
int argo::iridium::xfloat::XFloatDiveData::pres_offs_num_dives_  [private]
```

The number of dives used to compute the surface pressure offset.

10.87.5.32 pres_offs_num_samples_

```
int argo::iridium::xfloat::XFloatDiveData::pres_offs_num_samples_  [private]
```

The number of samples used to compute the surface pressure offset.

10.87.5.33 pres_offs_valid_

```
bool argo::iridium::xfloat::XFloatDiveData::pres_offs_valid_  [private]
```

The surface-pressure offset-is-valid flag.

10.87.5.34 pressure_offset_correction_enabled_list_file_

```
bool argo::iridium::xfloat::XFloatDiveData::pressure_offset_correction_enabled_list_file_  [static],
[private]
```

The global surface-pressure-offset-correction-is-enabled-in-intermediate-listing-files flag.

The value of this flag determines whether the surface pressure offset correction is applied to all of the pressure values in the intermediate listing files. If TRUE, the surface pressure offset correction will be applied to the intermediate listing files. Otherwise, the pressure offset correction will not be applied.

The value of this field is set as a result of parsing the command-line arguments and/or configuration file options during application initialization.

Referenced by GenerateListingString().

10.87.5.35 pressure_offset_correction_enabled_netcdf_file_

```
bool argo::iridium::xfloat::XFloatDiveData::pressure_offset_correction_enabled_netcdf_file_↩
[static], [private]
```

The global surface-pressure-offset-correction-is-enabled-in-NetCDF-files flag.

The value of this flag determines whether the surface pressure offset correction is applied to all of the pressure values in the NetCDF files. If TRUE, the surface pressure offset correction will be applied to the NetCDF files. Otherwise, the pressure offset correction will not be applied.

The value of this field is set as a result of parsing the command-line arguments and/or configuration file options during application initialization.

10.87.5.36 pressure_offset_correction_enabled_phy_file_

```
bool argo::iridium::xfloat::XFloatDiveData::pressure_offset_correction_enabled_phy_file_  [static],
[private]
```

The global surface-pressure-offset-correction-is-enabled-in-PHY-files flag.

The value of this flag determines whether the surface pressure offset correction is applied to all of the pressure values in the PHY files. If TRUE, the surface pressure offset correction will be applied to the PHY files. Otherwise, the pressure offset correction will not be applied.

The value of this field is set as a result of parsing the command-line arguments and/or configuration file options during application initialization.

Referenced by GeneratePhyString().

10.87.5.37 protocol_

std::string util::mysql::LocalMysqlConfiguration::protocol_ [static], [private], [inherited]

The protocol to use in the SQL server connection e.g.

"tcp".

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.87.5.38 schema_

std::string util::mysql::LocalMysqlConfiguration::schema_ [static], [private], [inherited]

The SQL server schema name.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.87.5.39 sql_connection_

thread_specific_ptr<Connection> util::mysql::MysqlUtils::sql_connection_ [static], [private], [inherited]

Tthe thread-specific pointer to MySQL connection object.

Thread-specific MySQL connection, released by `MysqlUtils::CleanupSqlConnection()` when the thread exits.

Referenced by util::mysql::MysqlUtils::CloseSqlConnection(), and util::mysql::MysqlUtils::GetSqlConnection().

10.87.5.40 sql_driver_

thread_specific_ptr<MySQL_Driver> util::mysql::MysqlUtils::sql_driver_ [static], [private], [inherited]

The thread-specific pointer to MySQL driver object.

Thread-specific reference to the MySQL driver.

Its cleanup function, `MysqlUtils::CleanupSqlDriver()`, does not delete the shared driver instance.

Referenced by util::mysql::MysqlUtils::GetSqlDriver().

10.87.5.41 sql_table_description_

const util::mysql::SqlTableDescription< XFloatDiveData > argo::iridium::xfloat::XFloatDiveData↩ ::sql_table_description_ [static], [private]

The SQL table description object.

This SQL table description provides the details needed to map this object to a relational database table. It utilizes the accessor methods of the class and the details of the SQL table column to (1) create an SQL INSERT statement to save the object into the database, (2) populate one or more objects' fields as a result of an SQL SELECT query, or (3) query the SQL database to access one or more of the values that are stored the the SQL table corresponding to this object.

Referenced by DeleteFromDatabase(), DeleteObjectsFromDatabase(), SaveObjectToDatabase(), SelectObjectFromDatabase(), SelectObjectsFromDatabase(), and SelectValueFromDatabase().

10.87.5.42 surface_pres_max_

float argo::iridium::xfloat::XFloatDiveData::surface_pres_max_ [private]

The maximum reported surface pressure.

10.87.5.43 surface_pres_mean_

```
float argo::iridium::xfloat::XFloatDiveData::surface_pres_mean_  [private]
```

The mean surface pressure.

10.87.5.44 surface_pres_min_

```
float argo::iridium::xfloat::XFloatDiveData::surface_pres_min_  [private]
```

The minimum reported surface pressure.

10.87.5.45 surface_pres_num_samples_

```
int argo::iridium::xfloat::XFloatDiveData::surface_pres_num_samples_  [private]
```

The number of surface pressure samples reported.

10.87.5.46 surface_pres_valid_

```
bool argo::iridium::xfloat::XFloatDiveData::surface_pres_valid_  [private]
```

The surface-pressure-is-valid flag.

10.87.5.47 surface_pres_variance_

```
float argo::iridium::xfloat::XFloatDiveData::surface_pres_variance_  [private]
```

The surface pressure variance.

10.87.5.48 user_

```
std::string util::mysql::LocalMysqlConfiguration::user_  [static], [private], [inherited]
```

The SQL server account username string.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.87.5.49 xmsg_acoustic_positions_

```
shared_ptr<x::XAcousticPosition> argo::iridium::xfloat::XFloatDiveData::xmsg_acoustic_positions↩
_  [private]
```

A vector of pointers to the acoustic position messages.

Note

This message type is used by MIZ floats only.

This vector is sorted upon insertion by order of non-decreasing message indices.

Referenced by GenerateListingString(), and ResetData().

10.87.5.50 xmsg_argo_data_

```
shared_ptr<x::XArgo> argo::iridium::xfloat::XFloatDiveData::xmsg_argo_data_  [private]
```

A pointer to the Argo data message.

Referenced by AssignArgoData(), GenerateListingString(), GetGainsAndOffsets(), GetSurfaceGainsAndOffsets(), ResetData(), and SensorGainsAndOffsetsAreKnown().

10.87.5.51 xmsg_composite_drift_data_

x::XCompositeDrift argo::iridium::xfloat::XFloatDiveData::xmsg_composite_drift_data_ [private]

The composite drift data object.

This object contains all of the drift data for this dive/cycle.

Referenced by AssignDriftPressure(), AssignDriftSalinity(), AssignDriftTemperatre(), GenerateListingString(), HasDriftData(), and ResetData().

10.87.5.52 xmsg_composite_profile_binned_

x::XCompositeProfile argo::iridium::xfloat::XFloatDiveData::xmsg_composite_profile_binned_ [private]

The composite binned profile data object.

This object contains all of the binned profile data for this dive/cycle.

Referenced by AssignAuxTemperatureBinned(), AssignProfilePressureBinned(), AssignProfileSalinityBinned(), AssignProfileTemperatureBinned(), GenerateListingString(), GeneratePhyString(), HasBinnedProfileData(), HasCompleteBinnedProfileDa and ResetData().

10.87.5.53 xmsg_composite_profile_unbinned_

x::XCompositeHiResProfile argo::iridium::xfloat::XFloatDiveData::xmsg_composite_profile_unbinned↩
_ [private]

The composite unbinned profile data object.

This object contains all of the unbinned profile data for this dive/cycle.

Referenced by AssignProfilePressureUnbinned(), AssignProfileSalinityUnbinned(), AssignProfileTemperatureUnbinned(), GenerateListingString(), GeneratePhyString(), HasCompleteUnbinnedProfileData(), HasUnbinnedProfileData(), and ResetData().

10.87.5.54 xmsg_engineering_data_abort_

shared_ptr<x::XEngAbort> argo::iridium::xfloat::XFloatDiveData::xmsg_engineering_data_abort_↩
[private]

A pointer to the engineering abort message.

Referenced by AssignEngineeringAbort(), GenerateListingString(), HasEngineeringData(), and ResetData().

10.87.5.55 xmsg_engineering_data_ascent_

shared_ptr<x::XEngAscent> argo::iridium::xfloat::XFloatDiveData::xmsg_engineering_data_ascent_↩
[private]

A pointer the the engineering ascent message.

Note

This message type is used by ALAMO floats only.

Referenced by AssignEngineeringAscent(), GenerateListingString(), HasEngineeringData(), and ResetData().

10.87.5.56 xmsg_engineering_data_bit_fail_

```
shared_ptr<x::XEngBITFail> argo::iridium::xfloat::XFloatDiveData::xmsg_engineering_data_bit_↩
fail_ [private]
```

A pointer to the engineering BIT failed message.

Referenced by AssignEngineeringBitFail(), GenerateListingString(), HasEngineeringData(), and ResetData().

10.87.5.57 xmsg_engineering_data_bit_passed_

```
shared_ptr<x::XEngBITPass> argo::iridium::xfloat::XFloatDiveData::xmsg_engineering_data_bit_↩
passed_ [private]
```

A pointer to the engineering BIT passed message.

Referenced by AssignEngineeringBitPass(), GenerateListingString(), HasEngineeringData(), and ResetData().

10.87.5.58 xmsg_engineering_data_descent_

```
shared_ptr<x::XEngDescent> argo::iridium::xfloat::XFloatDiveData::xmsg_engineering_data_descent↩
_ [private]
```

A pointer to the engineering descent message.

Note

This message type is used by ALAMO floats only.

Referenced by AssignEngineeringDescent(), GenerateListingString(), HasEngineeringData(), and ResetData().

10.87.5.59 xmsg_engineering_data_diagnostic_dive_

```
shared_ptr<x::XEngDiagnostic> argo::iridium::xfloat::XFloatDiveData::xmsg_engineering_data_↩
diagnostic_dive_ [private]
```

A pointer to the engineering diagnostic or startup message.

Referenced by AssignEngineeringDiagnostic(), GenerateListingString(), HasEngineeringData(), and ResetData().

10.87.5.60 xmsg_engineering_data_drift_

```
shared_ptr<x::XEngDrift> argo::iridium::xfloat::XFloatDiveData::xmsg_engineering_data_drift_↩
[private]
```

A pointer to the engineering drift message.

Note

This message type is used by ALAMO floats only.

Referenced by AssignEngineeringDrift(), GenerateListingString(), HasEngineeringData(), and ResetData().

10.87.5.61 xmsg_engineering_data_ice_

```
shared_ptr<x::XEngIce> argo::iridium::xfloat::XFloatDiveData::xmsg_engineering_data_ice_ [private]
```

A pointer to the ice-detection engineering message.

Referenced by AssignEngineeringIce(), GenerateListingString(), and ResetData().

10.87.5.62 xmsg_engineering_data_standard_dive_

shared_ptr<x::XEngStandard> argo::iridium::xfloat::XFloatDiveData::xmsg_engineering_data_standard_dive_ [private]

A pointer to the standard dive-cycle engineering message.

Referenced by AssignEngineeringStandard(), GenerateListingString(), HasEngineeringData(), and ResetData().

10.87.5.63 xmsg_engineering_data_surface_

shared_ptr<x::XEngSurface> argo::iridium::xfloat::XFloatDiveData::xmsg_engineering_data_surface_ [private]

A pointer to the engineering surface message.

Note

This message type is used by ALAMO floats only.

Referenced by AssignEngineeringSurface(), GenerateListingString(), HasEngineeringData(), and ResetData().

10.87.5.64 xmsg_fall_time_series_

static_vector<shared_ptr<x::XFall>, XMSG_MAX_FALL> argo::iridium::xfloat::XFloatDiveData::xmsg_fall_time_series_ [private]

A static vector of pointers to the fall time-series messages.

Note

This vector is sorted upon insertion by order of non-decreasing message indices.

Referenced by AssignFallTimeSeries(), GenerateCompositeFallTimeSeriesData(), GenerateListingString(), HasFallRiseData(), and ResetData().

10.87.5.65 xmsg_gps_positions_

static_vector<shared_ptr<x::XGPS>, XMSG_MAX_GPS> argo::iridium::xfloat::XFloatDiveData::xmsg_gps_positions_ [private]

A static vector of pointers to the GPS position message(s).

Note

This vector is sorted upon insertion by order of non-decreasing message indices.

Referenced by AssignGps(), GenerateListingString(), GetGpsPositionEndOfDive(), HasPositionData(), ResetData(), and SaveObjectToDatabase().

10.87.5.66 xmsg_mission_cfg_

x::XMfaMissionCfg argo::iridium::xfloat::XFloatDiveData::xmsg_mission_cfg_ [private]

The MFA mission configruation object.

Referenced by GenerateListingString().

10.87.5.67 xmsg_mission_cmd_echoback_

```
vector<shared_ptr<x::XMissionCmdEcho> > argo::iridium::xfloat::XFloatDiveData::xmsg_mission_↩
cmd_echoback_ [private]
```

A vector of pointer to mission configuration command upload echo-back messages.

Note

These messages are unordered.

Referenced by AssignMissionCmdEchoback(), GenerateListingString(), and ResetData().

10.87.5.68 xmsg_pump_time_series_

```
static_vector<shared_ptr<x::XPump>, XMSG_MAX_PUMP> argo::iridium::xfloat::XFloatDiveData::xmsg↩
_pump_time_series_ [private]
```

A static vector of pointers to the pump time-series messages.

Note

This vector is sorted upon insertion by order of non-decreasing message indices.

Referenced by AssignPumpTimeSeries(), GenerateCompositePumpTimeSeriesData(), GenerateListingString(), and ResetData().

10.87.5.69 xmsg_rise_time_series_

```
static_vector<shared_ptr<x::XRise>, XMSG_MAX_RISE> argo::iridium::xfloat::XFloatDiveData::xmsg↩
_rise_time_series_ [private]
```

A static vector of pointers to the fall time-series messages.

Note

This vector is sorted upon insertion by order of non-decreasing message indices.

Referenced by AssignRiseTimeSeries(), GenerateCompositeRiseTimeSeriesData(), GenerateListingString(), HasFallRiseData(), and ResetData().

10.87.5.70 xmsg_surface_time_series_

```
x::XCompositeSurfaceData argo::iridium::xfloat::XFloatDiveData::xmsg_surface_time_series_ [private]
```

The surface pressure time-series object.

Referenced by AssignSurfaceTimeSeries(), GenerateListingString(), HasDiveData(), ResetData(), and UpdateDerivedSurfaceDataFields()

10.87.5.71 xmsg_valve_data_

```
static_vector<shared_ptr<x::XValve>, XMSG_MAX_VALVE> argo::iridium::xfloat::XFloatDiveData↩
::xmsg_valve_data_ [private]
```

Valve operation messages.

Referenced by AssignValveData(), GenerateListingString(), and ResetData().

10.87.5.72 xmsg_wave_field_

```
shared_ptr<x::XWave> argo::iridium::xfloat::XFloatDiveData::xmsg_wave_field_  [private]
```

A static vector of pointers to the surface wave field data messages.

Note

This vector is sorted upon insertion by order of non-decreasing message indices.

Referenced by GenerateListingString().

The documentation for this class was generated from the following files:

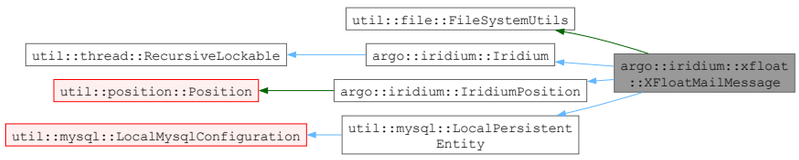
- data/XFloatDiveData.h
- data/XFloat.cpp
- data/XFloatDiveData.cpp
- data/XFloatMailMessage.cpp

10.88 argo::iridium::xfloat::XFloatMailMessage Class Reference

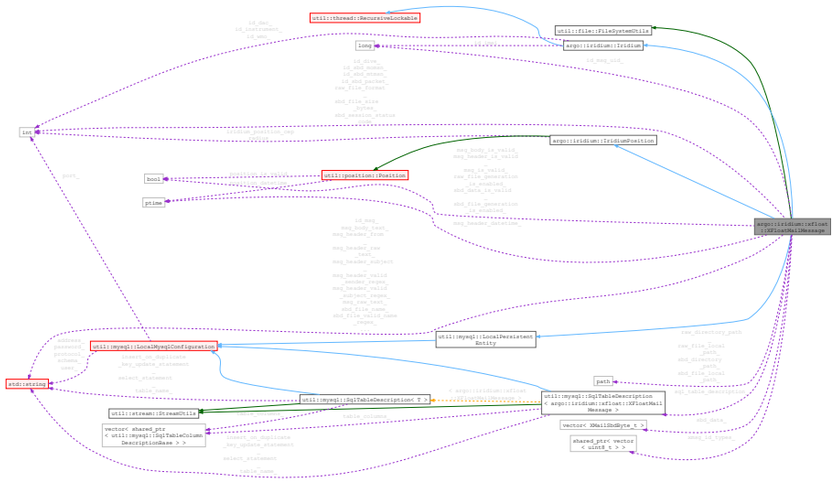
The mail message class.

```
#include <XFloatMailMessage.h>
```

Inheritance diagram for argo::iridium::xfloat::XFloatMailMessage:



Collaboration diagram for argo::iridium::xfloat::XFloatMailMessage:



Public Member Functions

- XFloatMailMessage ()

  The default constructor.
- XFloatMailMessage (const vmime::shared_ptr< vmime::net::message > &msg)

  Construct from a VMime mail message.
- ~XFloatMailMessage ()

  The destructor.
- const string GenerateListingString () const

  Generate the listing file entry string for this mail message.
- const string **GenerateSbdHexListingString** () const
- int id_dac () const

  Get the unique Data Assembly Center (DAC) identifier.
- int id_dive () const

  Get the cycle or dive number of this mail message.
- long id_imei () const

  Get the unique International Mobile Equipment Identifier (IMEI).
- int id_instrument () const

  Get the unique instrument identifier or instrument serial number.
- string id_msg () const

  Get the mail message unique identifier string.
- long id_msg_uid () const

  Get the mail message unique identifier.
- int id_sbd_momsn () const

  Get the Iridium Short Burst Data (SBD) Service Mobile Originated Message Sequence Number (MOMSN) identifier.
- int id_sbd_mtmsn () const

  Get the Iridium Short Burst Data (SBD) Service Mobile Terminated Message Sequence Number (MTMSN) identifier.
- int id_sbd_packet () const

  Get the Iridium Short Burst Data (SBD) message packet index.
- int id_wmo () const

  Get the unique World Meteorological Organization (WMO) identifier.
- string iridium_position_as_iso6709_string () const

  Get a formatted string representing the Iridium position composed using the ISO 6709 standard.
- int iridium_position_cep_radius () const

  Get the Iridium position Circular Error Probable (CEP) radius, in kilometers.
- ptime iridium_position_datetime () const

  Get the Iridium position date/time.
- string iridium_position_datetime_as_iso8601_string () const

  Get a formatted string representing the Iridium position date/time composed using the ISO 8601 standard.
- bool iridium_position_is_valid () const

  Get the Iridium position is valid flag.
- double iridium_position_latitude () const

  Get the Iridium position latitude.
- double iridium_position_longitude () const

  Get the Iridium position longitude.
- bool LockObject (bool block)

  Lock the object.
- bool msg_body_is_valid () const

  Is the body of the mail message valid?
- string msg_body_text () const

  Get the mail message body text.
- ptime msg_header_datetime () const

  Get the mail message header date/time.
- string msg_header_from () const

  Get the mail message header FROM field.
- bool msg_header_is_valid () const

  Is the header of the mail message valid?
- string msg_header_raw_text () const

  Get the message header raw text.
- string msg_header_subject () const

  Get the mail message header SUBJECT field.
- bool msg_is_valid () const

  Is the mail message valid?

- string msg_raw_text () const

  Get the full mail message raw text.
- path raw_file_local_path () const

  Get the path to the raw mail message file on the local filesystem.
- void SaveObjectToDatabase () const

  Save this mail message object to the SQL database.
- vector< uint8_t > sbd_data () const

  Get the Iridium Short Burst Data (SBD) binary data.
- bool sbd_data_is_valid () const

  Is the Iridium Short Burst Data (SBD) message valid?
- path sbd_file_local_path () const

  Get the path to the Iridium Short Burst Data (SBD) binary file on the local filesystem.
- string sbd_file_name () const

  Get the Iridium Short Burst Data (SBD) email attachment filename.
- int sbd_file_size_bytes () const

  Get the size of the Iridium Short Burst Data (SBD) message, in bytes.
- int sbd_session_status_code () const

  Get the Iridium Short Burst Data (SBD) session status code.
- string sbd_session_status_string () const

  Get the Iridium Short Burst Data (SBD) session status string.
- void UnlockObject ()

  Unlock the object.
- const shared_ptr< vector< uint8_t > > & **xmsg_id_types** () const

Static Public Member Functions

- static void CloseSqlConnection ()

  Close the thread-specific MySQL connection if it is currently open.
- static shared_ptr< sql::ResultSet > **executeDeleteQuery** (const string &from, const string &where="")
- static shared_ptr< ResultSet > ExecuteQuery (Connection *con, const string &query)

  Execute an SQL query and return a shared pointer to the ResultSet object.
- static shared_ptr< sql::ResultSet > executeQuery (const string &query)

  Execute an SQL query and return a shared pointer to the ResultSet object.
- static shared_ptr< sql::ResultSet > executeSelectQuery (const string &what, const string &from, const string &where="", const string &groupBy="", const string &orderBy="", unsigned int limit=0, unsigned int offset=0)

  Execute an SQL select query and return a shared pointer to the ResultSet object.
- static int ExecuteUpdate (Connection *con, const string &update)

  Execute an SQL update and return the status code.
- static int executeUpdate (const string &update)

  Execute an SQL update and return the status code.
- static string GenerateListingComment ()

  Get the mail message listing file comment string.
- static shared_ptr< PreparedStatement > GeneratePreparedStatement (Connection *con, const string &statement)

  Generate a prepared SQL statement object for the given SQL statement.
- static shared_ptr< sql::PreparedStatement > generatePreparedStatement (const string &statement)

  Generate a prepared SQL statement object for the given SQL statement.
- static string GenerateSqlDeleteStr (const string &from, const string &where="")

  Generate an SQL DELETE statement.
- static string GenerateSqlSelectStr (const string &what, const string &from, const string &where="", const string &groupBy="", const string &orderBy="", unsigned int limit=0, unsigned int offset=0)

  Generate an SQL select statement.
- static thread_specific_ptr< Connection > & GetSqlConnection (string url, string user, string password, string schema)

  Get a thread-specific instance of a MySQL connection object.
- static thread_specific_ptr< MySQL_Driver > & GetSqlDriver ()

  Get a thread-specific instance of a MySQL driver object.

Protected Member Functions

- ptime position_datetime () const

  Get the date/time of the position.
- string position_datetime_as_iso8601_string () const

  Get a formatted string representing the position date/time composed using the ISO 8601 standard.
- bool position_is_valid () const

  Is the position valid?
- double position_latitude () const

  Get the latitude of the position, in decimal degrees.
- double position_longitude () const

  Get the longitude of the position, in decimal degrees.
- string postition_as_iso6709_string () const

  Get a formatted string representing the position composed using the ISO 6709 standard.
- void set_gps_position_datetime (unsigned int gpsWeek, unsigned int gpsDay, unsigned int gpsHour=0, unsigned int gpsMinute=0, unsigned int gpsSecond=0)

  Set the date and time of this position from the given GPS week, day, and time.
- void set_id_dac (int id_dac)

  Set the unique Data Assembly Center (DAC) identifier.
- void set_id_imei (long id_imei)

  Set the unique International Mobile Equipment Identifier (IMEI).
- void set_id_instrument (int id_instrument)

  Set the unique instrument identifier or instrument serial number.
- void set_id_wmo (int id_wmo)

  Set the unique World Meteorological Organization (WMO) identifier.
- void set_iridium_position_cep_radius (int iridium_position_cep_radius)

  Set the Iridium position Circular Error Probable (CEP) radius, in kilometers.
- void set_iridium_position_datetime (const ptime &iridium_position_datetime)

  Set the Iridium position date/time.
- void set_iridium_position_is_valid (bool iridium_position_is_valid)

  Set the Iridium position is valid flag.
- void set_iridium_position_latitude (double iridium_position_latitude)

  Set the Iridium position latitude, in decimal degrees.
- void set_iridium_position_longitude (double iridium_position_longitude)

  Set the Iridium position longitude, in decimal degrees.
- void set_position_datetime (const ptime &datetime)

  Set the date/time of the position.
- void set_position_is_valid (bool is_valid=true)

  Set the position is valid flag.
- void set_position_latitude (double lat)

  Set the latitude of the position, in decimal degrees.
- void set_position_longitude (double lon)

  Set the longitude of the position, in decimal degrees.

Static Protected Member Functions

- static float **bitcastUint32ToFloat** (uint32_t u)
- template<typename T >
  static double CalculateEllipsoidHeight (T lat)

  Get the approximate surface altitude or height of earth for a given latitude, in kilometers.
- template<typename T >
  static double CalculateGeographicDistance (T lat1, T lon1, T lat0, T lon0)

  Get the geographical distance between two position, in kilometers, from the positions' locations, in decimal degrees.
- template<typename T >
  static double degToRad (T deg)

  Convert angular units from from degrees to decimal radians.
- static string GenerateCurrentUtcTimeIso8601String ()

  Generate an ISO8601 format date/time string representation of the current time, in UTC.
- template<typename T >
  static std::string GenerateIso6709String (T lat, T lon)

  Generate an IS06709 string representation of the position given by (`lat`, `lon`).
- static string GenerateIso8601String (const ptime &p, bool extended=true)

  Generate an ISO8601 format date/time string representation of a POSIX time object.

- static double GenerateJulianDay (const ptime &dt, const ptime &ref_dt)

  Convert a date/time to fractional days since a reference date/time.
- static ptime GetCurrentUtcTime ()

  Generate a POSIX time object representing the current date and time (rounded to the nearest second).
- static const path & GetDirectoryPath (const path &dir, bool create=false)

  Get a directory path.
- static path GetDirectoryPathFromString (const string &dir, bool create=false)

  Generate a boost::filesystem::path object from the directory path string.
- static sql::Connection * getSqlConnection ()

  Generate a thread-specific instance of a MySQL connection object.
- template<typename T >
  static bool IsPositionValid (T lat, T lon)

  Check position validity.
- static vector< path > ListDirectories (const path &dir)

  List all directories contained in the given directory.
- static vector< path > ListFiles (path dir, size_t n=0)

  List all regular files contained in the given directory.
- static path MoveFileToDirectory (const path &file, const path &dir_dst, bool create_dir=false)

  Copy a file into a directory.

Protected Attributes

- recursive_mutex mutex_

  Recursive mutual exclusion lock for synchronizing operations.

Private Member Functions

- void ParseMailMsg (const vmime::shared_ptr< vmime::message > &msg)

  Parse a mail message.
- void ParseMailMsg (const vmime::shared_ptr< vmime::net::message > &msg)

  Parse a mail message.
- void ParseMailMsgBody (const vmime::shared_ptr< vmime::message > &msg)

  Parse the mail message body.
- void ParseMailMsgHeader (const vmime::shared_ptr< const vmime::header > &head)

  Parse the mail message header.
- void ParseMailMsgSbdAttachment (const vmime::shared_ptr< vmime::message > &msg)

  Parse the mail message SBD attachment.
- void set_id_dive (int id_dive)

  Set the dive or cycle number of this mail message.
- void set_id_msg (const string &id_msg)

  Set the mail message unique identifier string.
- void set_id_msg_uid (long id_msg_uid)

  Set the mail message unique identifier.
- void set_id_sbd_momsn (int momsn)

  Set the Iridium Short Burst Data (SBD) Service Mobile Originated Message Sequence Number (MOMSN).
- void set_id_sbd_mtmsn (int mtmsn)

  Set the Iridium Short Burst Data (SBD) Service Mobile Terminated Message Sequence Number (MTMSN).
- void set_id_sbd_packet (int id_sbd_packet)

  Set the Iridium Short Burst Data (SBD) message packet index.
- void set_msg_body_is_valid (bool is_valid)

  Set the message-body-is-valid flag.
- void set_msg_body_text (const string &msg_body_text)

  Set the mail message body text.
- void set_msg_header_datetime (const ptime &datetime)

  Set the mail message header date/time.
- void set_msg_header_from (const string &from)

  Set the mail message header FROM field value.
- void set_msg_header_is_valid (bool is_valid)

  Set the mail-message-header-is-valid flag.
- void set_msg_header_raw_text (const string &text)

  Set the mail message header raw text.
- void set_msg_header_subject (const string &subject)

Set the mail message header SUBJECT field value.

- void set_msg_is_valid (bool is_valid)

  Set the mail-message-is-valid flag.
- void set_msg_raw_text (const string &text)

  Set the full mail message raw text.
- void set_raw_file_local_path (const path &path)

  Set the path to the raw mail message file on the local file system.
- void set_sbd_data (const vector< uint8_t > &sbd_data)

  Set the Iridium Short Burst Data (SBD) for this mail message.
- void set_sbd_data_is_valid (bool is_valid)

  Set the SBD-message-is-valid flag.
- void set_sbd_file_local_path (const path &path)

  Set the path to the Iridium Short Burst Data (SBD) message file on the local file system.
- void set_sbd_file_name (const string &name)

  Set the name of the Iridium Short Burst Data (SBD) file.
- void set_sbd_file_size_bytes (int size)

  Set the size of the Iridium Short Burst Data (SBD) message, in bytes.
- void set_sbd_session_status_code (int code)

  Set the Iridium Short Burst Data (SBD) session status code.
- void set_sbd_session_status_string (const string &str)

  Set the Iridium Short Burst Data (SBD) session status string.
- void **set_xmsg_id_types** (const shared_ptr< vector< uint8_t > > &xmsg_id_types)
- void WriteFiles ()

  Write the mail message files.
- void WriteRawFile ()

  Write the RAW mail message file.
- void WriteSbdFile ()

  Write the binary Iridium Short Burst Data (SBD) file.

Static Private Member Functions

- static void CleanupSqlConnection (Connection *con)

  Cleanup an SQL connection.
- static void CleanupSqlDriver (MySQL_Driver *driver)

  Function for memory cleanup of MySQL driver on destruct.
- static vector< shared_ptr< XFloatMailMessage > > SelectAllObjectsFromDatabase (const string &groupBy="", const string &orderBy="", const unsigned int &limit=0, const unsigned int &offset=0)

  Select all mail message objects from the SQL database table.
- static shared_ptr< XFloatMailMessage > SelectObjectFromDatabase (const string &where="", const string &groupBy="", const string &orderBy="", const unsigned int &offset=0)

  Select a single mail message object from the SQL database table.
- static vector< shared_ptr< XFloatMailMessage > > SelectObjectsFromDatabase (const string &where="", const string &groupBy="", const string &orderBy="", const unsigned int &limit=0, const unsigned int &offset=0)

  Query mail message objects from the SQL database.
- template<typename TT >
  static TT SelectValueFromDatabase (const string &what, const string &where="", const string &groupBy="", const string &orderBy="", const unsigned int &offset=0)

  Select a value from the SQL database table that contains the persisted mail message objects.

Private Attributes

- int id_dac_

  The unique Data Assembly Center (DAC) identifier.
- int id_dive_

  The dive or cycle number of this mail message object.
- long id_imei_

  The unique International Mobile Equipment Identifier (IMEI).
- int id_instrument_

  The unique instrument identifier or instrument serial number.
- string id_msg_

  The mail message unique identifier string.
- long id_msg_uid_

The mail message unique identifier (UID).

- int id_sbd_momsn_

  The Iridium Short Burst Data (SBD) Service Mobile Originated Message Sequence Number (MOMSN) identifier.
- int id_sbd_mtmsn_

  The Iridium Short Burst Data (SBD) Service Mobile Terminated Message Sequence Number (MTMSN) identifier.
- int id_sbd_packet_

  The Iridium Short Burst Data (SBD) message packet index.
- int id_wmo_

  The unique World Meteorological Organization (WMO) identifier.
- int iridium_position_cep_radius_

  Iridium position Circular Error Probable (CEP) radius, in kilometers.
- bool msg_body_is_valid_

  The mail message body is valid flag.
- string msg_body_text_

  The mail message body text.
- ptime msg_header_datetime_

  The mail message header date/time.
- string msg_header_from_

  The mail message header FROM field.
- bool msg_header_is_valid_

  The mail message header is valid flag.
- string msg_header_raw_text_

  The mail message header raw text.
- string msg_header_subject_

  The mail message header SUBJECT field.
- bool msg_is_valid_

  The mail message is valid flag.
- string msg_raw_text_

  The full mail message raw text.
- ptime position_datetime_

  The date and time of the position.
- bool position_is_valid_

  The valid position indicator.
- double position_latitude_

  The latitude of the position, in decimal degrees.
- double position_longitude_

  The longitude of the position, in decimal degrees.
- path raw_file_local_path_

  The path to the raw mail message file on the local filesystem.
- XMailSbdData_t sbd_data_

  The Iridium Short Burst Data (SBD) binary data.
- bool sbd_data_is_valid_

  The Iridium Short Burst Data (SBD) binary data is valid flag.
- path sbd_file_local_path_

  The path to the Iridium Short Burst Data (SBD) binary file on the local filesystem.
- string sbd_file_name_

  The Iridium Short Burst Data (SBD) email attachment filename.
- int sbd_file_size_bytes_

  The size of the Iridium Short Burst Data (SBD) message, in bytes.
- int sbd_session_status_code_

  The Iridium Short Burst Data (SBD) session status code.
- string sbd_session_status_string_

  The Iridium Short Burst Data (SBD) session status string.
- shared_ptr< vector< uint8_t > > **xmsg_id_types_**

Static Private Attributes

- static string address_

  The SQL server qualified name or IP address string.
- static const double EARTH_RADIUS_KM_EQUATORIAL = 6378.1

  The equatorial radius of the earth, in kilometers.
- static const double EARTH_RADIUS_KM_POLAR = 6356.1

  The polar radius of the earth, in kilometers.
- static string msg_header_valid_sender_regex_

  The mail message header valid sender regular expression.
- static string msg_header_valid_subject_regex_

  The mail message header valid subject regular expression.
- static string password_

  The SQL server account password string.
- static unsigned int port_

  The SQL server port number.
- static string protocol_

  The protocol to use in the SQL server connection e.g.
- static path raw_directory_path_

  The RAW mail message file directory.
- static int raw_file_format_

  The raw mail message file format.
- static bool raw_file_generation_is_enabled_

  The global RAW-mail-message-file-generation-is-enabled flag.
- static path sbd_directory_path_

  The SBD file directory.
- static bool sbd_file_generation_is_enabled_

  The global binary-SBD-file-generation-is-enabled flag.
- static string sbd_file_valid_name_regex_

  The valid SBD file email attachment filename regular expression.
- static string schema_

  The SQL server schema name.
- static thread_specific_ptr< Connection > sql_connection_

  Tthe thread-specific pointer to MySQL connection object.
- static thread_specific_ptr< MySQL_Driver > sql_driver_

  The thread-specific pointer to MySQL driver object.
- static const util::mysql::SqlTableDescription< XFloatMailMessage > sql_table_description_

  The SQL table description object.
- static string user_

  The SQL server account username string.

Friends

- ostream & operator<< (ostream &os, const XFloatMailMessage &mail)

  The stream insertion operator overload function for XFloatMailMessage.
- istream & operator>> (istream &is, XFloatMailMessage &mail)

  The stream extraction operator overload function for XFloatMailMessage.
- options_description util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription ()

  Friend function for populating some of the static members of this class from the configuration file options.
- class **XFloat**
- class **XFloatDiveData**
- class **XFloatMailProcessingService**
- class **XFloatMailRetrievalService**

10.88.1 Detailed Description

The mail message class.

This class represents a single mail message received from the Iridium system, including fields that are parsed from the various parts of the mail message and the attached Iridium Short Burst Data (SBD) file. It is used to parse, validate, and write the mail message files to the local filesystem.

Note

This class is mapped to an SQL database table by the sql_table_description_ object. This allows some or all of the field of this class to be persisted to the SQL database. Likewise, an object of this type may be constructed from the results of an SQL query to the corresponding table.

10.88.2 Constructor & Destructor Documentation

10.88.2.1 XFloatMailMessage() [1/2]

```
argo::iridium::xfloat::XFloatMailMessage::XFloatMailMessage ()  [inline]
```

The default constructor.

References sbd_data_.

10.88.2.2 XFloatMailMessage() [2/2]

```
argo::iridium::xfloat::XFloatMailMessage::XFloatMailMessage (
            const vmime::shared_ptr< vmime::net::message > & msg)  [inline]
```

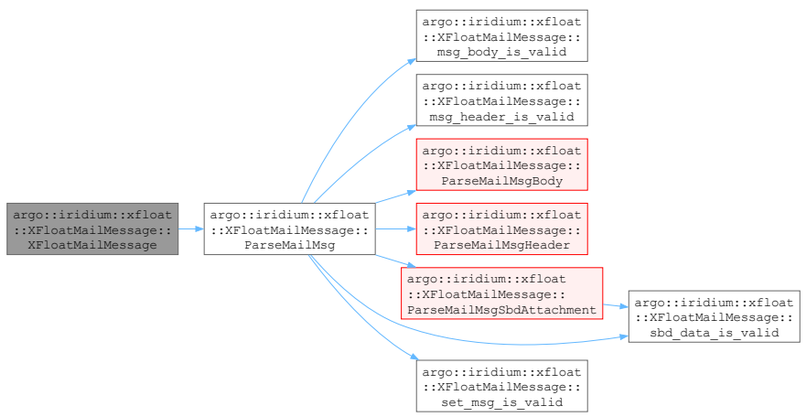
Construct from a VMime mail message.

Parameters

in	*msg*	The mail message.

References CATCH_BLOCK, ParseMailMsg(), and sbd_data_.

Here is the call graph for this function:



10.88.2.3 ~XFloatMailMessage()

```
argo::iridium::xfloat::XFloatMailMessage::~XFloatMailMessage ()  [inline]
```

The destructor.

10.88.3 Member Function Documentation

10.88.3.1 CalculateEllipsoidHeight()

```
template<typename T >
static double util::position::PositionUtils::CalculateEllipsoidHeight (
            T lat)  [inline], [static], [inherited]
```

Get the approximate surface altitude or height of earth for a given latitude, in kilometers.

Warning

This method provides approximate surface height estimates only! This approximation is calculated from a reference ellipsoid – a mathematically-defined surface that approximates the geoid, the truer figure of the Earth.

Parameters

in	*lat*	The geographic latitude of interest, in decimal degrees.

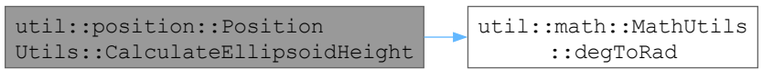
Returns

The approximate surface altitude or height of earth for the given latitude, in kilometers.

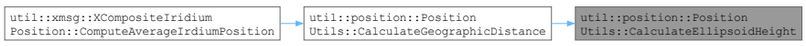
References util::math::MathUtils::degToRad(), util::position::PositionUtils::EARTH_RADIUS_KM_EQUATORIAL, and util::position::PositionUtils::EARTH_RADIUS_KM_POLAR.

Referenced by util::position::PositionUtils::CalculateGeographicDistance().

Here is the call graph for this function:



Here is the caller graph for this function:



10.88.3.2 CalculateGeographicDistance()

```
template<typename T >
static double util::position::PositionUtils::CalculateGeographicDistance (
            T lat1,
            T lon1,
            T lat0,
            T lon0)  [inline], [static], [inherited]
```

Get the geographical distance between two position, in kilometers, from the positions' locations, in decimal degrees.

Warning

This method provides approximate geographic estimates only! This approximation is calculated from a reference ellipsoid – a mathematically-defined surface that approximates the geoid, the truer figure of the Earth.

Note

This method uses the 'haversine' formula to calculate the great-circle distance between two points – that is, the shortest distance over the earth's surface – giving an 'as-the-crow-flies' distance between the points (ignoring any hills they fly over, of course!).

Parameters

in	*lat1*	The latitude of the geographic final position, in decimal degrees North.
in	*lon1*	The longitude of the geographic final position, in decimal degrees East.
in	*lat0*	The latitude of the geographic initial position, in decimal degrees North.
in	*lon0*	The longitude of the geographic initial position, in decimal degrees East.

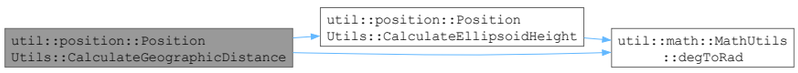
Returns

The geographical distance between the two given positions, in kilometers.

References util::position::PositionUtils::CalculateEllipsoidHeight(), and util::math::MathUtils::degToRad().

Referenced by util::xmsg::XCompositeIridiumPosition::ComputeAverageIrdiumPosition().

Here is the call graph for this function:



Here is the caller graph for this function:

10.88.3.3 CleanupSqlConnection()

```
static void util::mysql::MysqlUtils::CleanupSqlConnection (
            Connection * con)  [inline], [static], [private], [inherited]
```

Cleanup an SQL connection.

This function for memory cleanup of the SQL connection on destruction. The thread-specific SQL connection object will be deleted if it exists.

Parameters

in	*con*	The SQL connection to cleanup.

Exceptions

log::MessageException	if an error occurs while cleaning up the SQL connection object.

References CATCH_BLOCK.

10.88.3.4 CleanupSqlDriver()

```
static void util::mysql::MysqlUtils::CleanupSqlDriver (
            MySQL_Driver * driver)  [inline], [static], [private], [inherited]
```

Function for memory cleanup of MySQL driver on destruct.

Signals the SQL driver to end operation, if it exists.

Parameters

in	*driver*	Driver to cleanup.

Exceptions

log::MessageException	if an error occurs while cleaning up the SQL driver object.

References CATCH_BLOCK.

10.88.3.5 CloseSqlConnection()

```
static void util::mysql::MysqlUtils::CloseSqlConnection ()  [inline], [static], [inherited]
```

Close the thread-specific MySQL connection if it is currently open.

If the thread local MySQL connection is not currently open, then this function will have no effect.

Note

The MySQL connection should be closed when idle to prevent the connection from timing out and throwing exceptions.

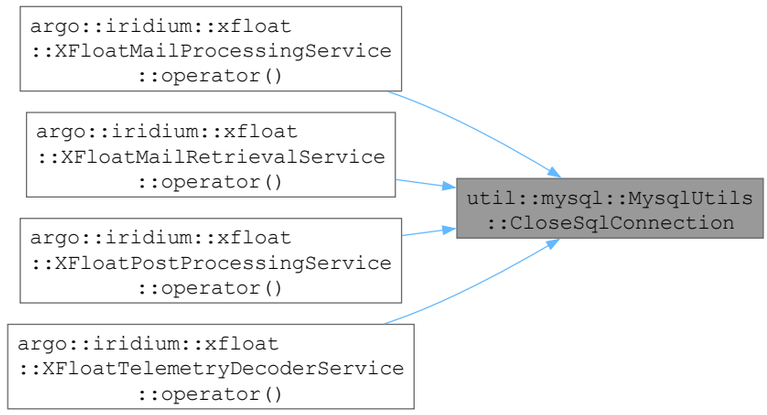
Exceptions

log::MessageException	if an error occurs while attempting to close the MySQL connection.

References CATCH_BLOCK, and util::mysql::MysqlUtils::sql_connection_.

Referenced by argo::iridium::xfloat::XFloatMailProcessingService::operator()(), argo::iridium::xfloat::XFloatMailRetrievalService::operator(
argo::iridium::xfloat::XFloatPostProcessingService::operator()(), and argo::iridium::xfloat::XFloatTelemetryDecoderService::operator()().

Here is the caller graph for this function:



10.88.3.6 degToRad()

```
template<typename T >
static double util::math::MathUtils::degToRad (
            T deg)  [inline], [static], [inherited]
```

Convert angular units from from degrees to decimal radians.

Parameters

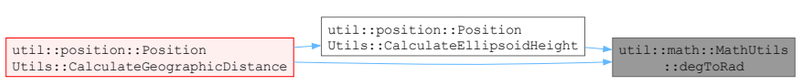
in	*deg*	Angle in integer or decimal degrees to convert to radians.

Returns

Angle in decimal radians.

Referenced by util::position::PositionUtils::CalculateEllipsoidHeight(), and util::position::PositionUtils::CalculateGeographicDistance().

Here is the caller graph for this function:



10.88.3.7 ExecuteQuery()

```
static shared_ptr< ResultSet > util::mysql::MysqlUtils::ExecuteQuery (
            Connection * con,
            const string & query)  [inline], [static], [inherited]
```

Execute an SQL query and return a shared pointer to the ResultSet object.

Parameters

in	*con*	A pointer to the SQL connection object. This must not be NULL.
in	*query*	The SQL query to execute. This must be a well-formatted SQL query string.

Exceptions

log::MessageException	if 'con' is null, 'query' is a malfored SQL string, or any other error occurs.

Returns

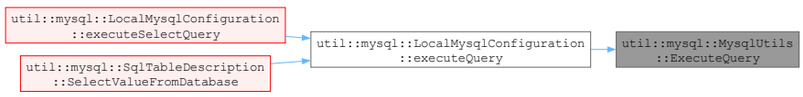
A shared pointer to the query ResultSet object.

References CATCH_BLOCK, and util::mysql::MysqlUtils::GeneratePreparedStatement().

Referenced by util::mysql::LocalMysqlConfiguration::executeQuery().

Here is the call graph for this function:

Here is the caller graph for this function:



10.88.3.8 executeQuery()

```
shared_ptr< sql::ResultSet > util::mysql::LocalMysqlConfiguration::executeQuery (
            const string & query)  [static], [inherited]
```

Execute an SQL query and return a shared pointer to the ResultSet object.

Parameters

in	*query*	The SQL query to execute. This must be a well-formatted SQL query string.

Exceptions

log::MessageException	'query' is a malfored SQL string or any other error occurs while executing the query.

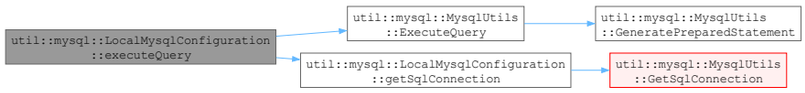
Returns

A shared pointer to the query ResultSet object.

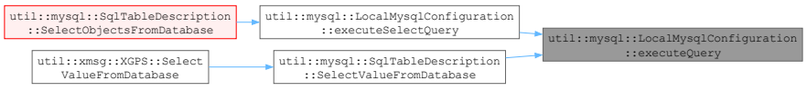
References util::mysql::MysqlUtils::ExecuteQuery(), and util::mysql::LocalMysqlConfiguration::getSqlConnection().

Referenced by util::mysql::LocalMysqlConfiguration::executeSelectQuery(), and util::mysql::SqlTableDescription< T >::SelectValueFromD

Here is the call graph for this function:



Here is the caller graph for this function:



10.88.3.9 executeSelectQuery()

```
shared_ptr< sql::ResultSet > util::mysql::LocalMysqlConfiguration::executeSelectQuery (
            const string & what,
            const string & from,
            const string & where = "",
            const string & groupBy = "",
            const string & orderBy = "",
            unsigned int limit = 0,
            unsigned int offset = 0)  [static], [inherited]
```

Execute an SQL select query and return a shared pointer to the ResultSet object.

Parameters

in	*what*	The SQL select statement select-expression clause.
in	*from*	The SQL select statement table–references clause.
in	*where*	The SQL select statement where-condition clause.
in	*groupBy*	The SQL select statement group-by clause.
in	*orderBy*	The SQL select statement order-by clause.
in	*limit*	The SQL select statement result set row limit value.
in	*offset*	The SQL select statement result set row offset.

Exceptions

log::MessageException	if an error occurs while executing the query.

Returns

A shared pointer to the ResultSet object.

References CATCH_BLOCK, util::mysql::LocalMysqlConfiguration::executeQuery(), and util::mysql::MysqlUtils::GenerateSqlSelectStr().

Referenced by util::mysql::SqlTableDescription< T >::SelectObjectsFromDatabase().

Here is the call graph for this function:

Here is the caller graph for this function:

10.88.3.10 ExecuteUpdate()

```
static int util::mysql::MysqlUtils::ExecuteUpdate (
            Connection * con,
            const string & update)  [inline], [static], [inherited]
```

Execute an SQL update and return the status code.

Parameters

in	*con*	A pointer to the SQL connection object. This must not be NULL.
in	*update*	The SQL update to execute. This must be a well-formatted SQL update string.

Exceptions

log::MessageException	if 'con' is null, 'update' is a malformed SQL string, or any other error occurs.

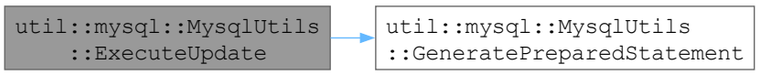
Returns

The update status code.

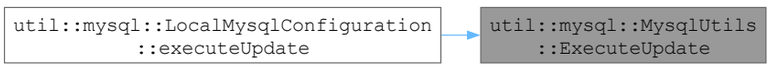
References CATCH_BLOCK, and util::mysql::MysqlUtils::GeneratePreparedStatement().

Referenced by util::mysql::LocalMysqlConfiguration::executeUpdate().

Here is the call graph for this function:



Here is the caller graph for this function:



10.88.3.11 executeUpdate()

```
int util::mysql::LocalMysqlConfiguration::executeUpdate (
            const string & update)  [static], [inherited]
```

Execute an SQL update and return the status code.

Parameters

in	*update*	The SQL update to execute. This must be a well-formatted SQL update string.

Exceptions

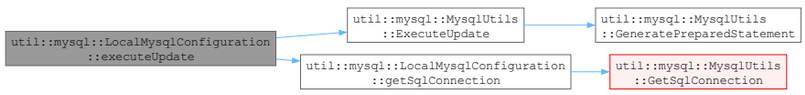
log::MessageException	if 'update' is a malformed SQL string, or any other error occurs while executing the update.

Returns

The update status code.

References util::mysql::MysqlUtils::ExecuteUpdate(), and util::mysql::LocalMysqlConfiguration::getSqlConnection().

Here is the call graph for this function:



10.88.3.12 GenerateCurrentUtcTimeIso8601String()

```
static string util::date::DateUtils::GenerateCurrentUtcTimeIso8601String ()  [inline], [static],
[inherited]
```

Generate an ISO8601 format date/time string representation of the current time, in UTC.

Note

All date/time objects use the UTC time-zone.

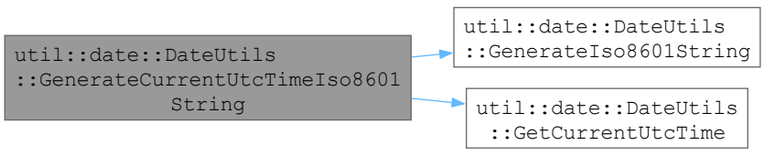
Returns

An ISO8601 format date/time string representation of the current time.

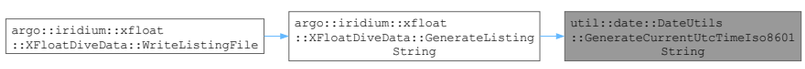
References util::date::DateUtils::GenerateIso8601String(), and util::date::DateUtils::GetCurrentUtcTime().

Referenced by argo::iridium::xfloat::XFloatDiveData::GenerateListingString().

Here is the call graph for this function:



Here is the caller graph for this function:



10.88.3.13 GenerateIso6709String()

```
template<typename T >
static std::string util::position::PositionUtils::GenerateIso6709String (
            T lat,
            T lon)  [inline], [static], [inherited]
```

Generate an IS06709 string representation of the position given by (`lat`, `lon`).

Parameters

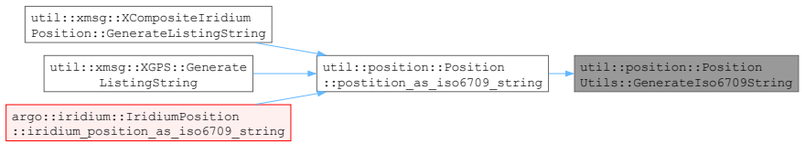
in	*lat*	The latitude of the position, in degrees North.
in	*lon*	The longitude of the position, in degrees East.

Returns

An IS06709 string representation of the position given by (`lat`, `lon`).

Referenced by util::position::Position::postition_as_iso6709_string().

Here is the caller graph for this function:



10.88.3.14 GenerateIso8601String()

```
static string util::date::DateUtils::GenerateIso8601String (
            const ptime & p,
            bool extended = true)  [inline], [static], [inherited]
```

Generate an ISO8601 format date/time string representation of a POSIX time object.

Note

All date/time objects use the UTC time-zone.

Parameters

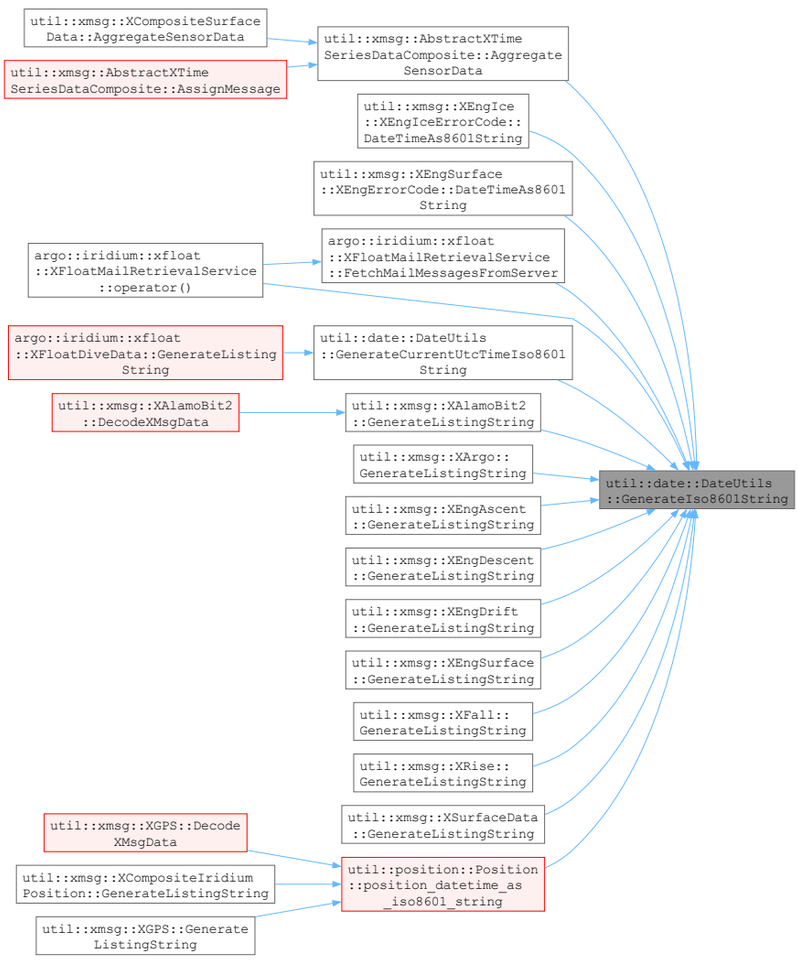
in	*p*	POSIX time from which to generate string.
in	*extended*	If TRUE, use the ISO 8601 extended format (YYYY-MM-DDThh:mm:ssZ); otherwise, use the basic format (YYYYMMDDThhmmss).

Returns

An ISO8601 format date/time string representation of the POSIX time object. If p is equal to boost::posix_↩ time::not_a_date_time, then the string "0000-00-00T00:00:00Z" (extended) or "00000000000000" (basic) will be returned.

Referenced by util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::AggregateSensorData(), util::xmsg::XEngIce::XEngIce
util::xmsg::XEngSurface::XEngErrorCode::DateTimeAs8601String(), argo::iridium::xfloat::XFloatMailRetrievalService::FetchMailMessages
util::date::DateUtils::GenerateCurrentUtcTimeIso8601String(), util::xmsg::XAlamoBit2::GenerateListingString(), util::xmsg::XArgo::Genera
util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListing
util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XRise::GenerateListingString(),
util::xmsg::XSurfaceData::GenerateListingString(), argo::iridium::xfloat::XFloatMailRetrievalService::operator()(), and
util::position::Position::position_datetime_as_iso8601_string().

Here is the caller graph for this function:



10.88.3.15 GenerateJulianDay()

```
static double util::date::DateUtils::GenerateJulianDay (
            const ptime & dt,
            const ptime & ref_dt)  [inline], [static], [inherited]
```

Convert a date/time to fractional days since a reference date/time.

Parameters

in	*dt*	The date/time to convert.
in	*ref↩ _dt*	The reference date/time (day 0).

Exceptions

util::log::MessageException	if either date/time is not valid.

Returns

The number of days (with millisecond resolution) from `ref_dt` to `dt`.

References CATCH_BLOCK, and EXC_THROW.

10.88.3.16 GenerateListingComment()

```
static string argo::iridium::xfloat::XFloatMailMessage::GenerateListingComment ()  [inline], [static]
```

Get the mail message listing file comment string.

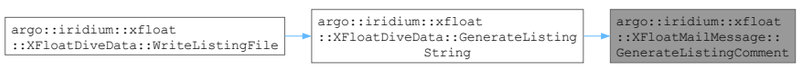
Returns

The mail message listing file comment string.

References CATCH_BLOCK.

Referenced by argo::iridium::xfloat::XFloatDiveData::GenerateListingString().

Here is the caller graph for this function:



10.88.3.17 GenerateListingString()

```
const string argo::iridium::xfloat::XFloatMailMessage::GenerateListingString () const  [inline]
```

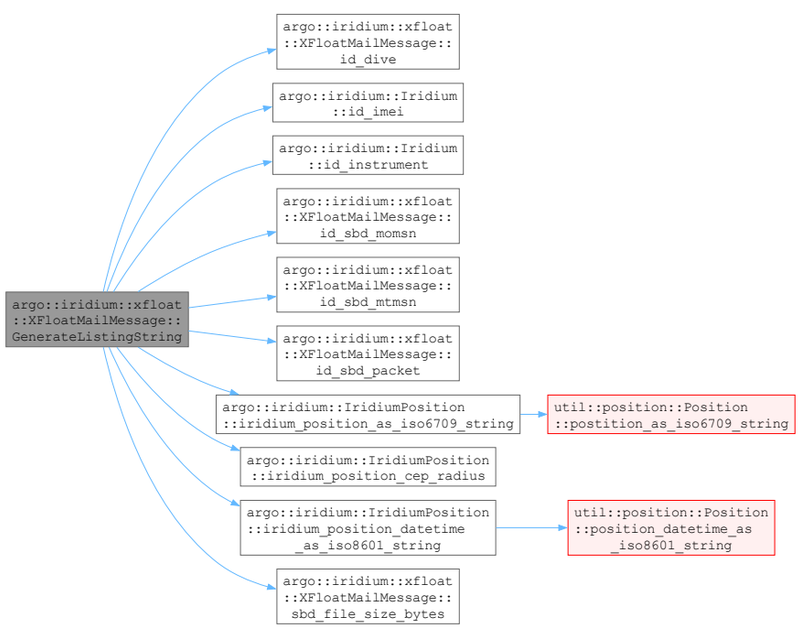
Generate the listing file entry string for this mail message.

Returns

The listing file entry string for this mail message.

References CATCH_BLOCK, id_dive(), argo::iridium::Iridium::id_imei(), argo::iridium::Iridium::id_instrument(), id_sbd_momsn(), id_sbd_mtmsn(), id_sbd_packet(), argo::iridium::IridiumPosition::iridium_position_as_iso6709_string(), argo::iridium::IridiumPosition::iridium_position_cep_radius(), argo::iridium::IridiumPosition::iridium_position_datetime_as_iso8601_string() and sbd_file_size_bytes().

Here is the call graph for this function:



10.88.3.18 GeneratePreparedStatement()

```
static shared_ptr< PreparedStatement > util::mysql::MysqlUtils::GeneratePreparedStatement (
            Connection * con,
            const string & statement)  [inline], [static], [inherited]
```

Generate a prepared SQL statement object for the given SQL statement.

Parameters

in	*con*	A pointer to the SQL connection object. This must not be NULL.
in	*statement*	The SQL statement to prepare. This must be a well-formed SQL string.

Exceptions

log::MessageException	if 'con' is NULL, 'statement' is not a valid SQL string, or if an error occurs.

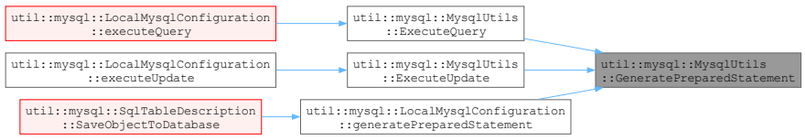
Returns

A prepared SQL statement object generated from the given SQL statement.

References CATCH_BLOCK, and EXC_THROW.

Referenced by util::mysql::MysqlUtils::ExecuteQuery(), util::mysql::MysqlUtils::ExecuteUpdate(), and util::mysql::LocalMysqlConfiguration:

Here is the caller graph for this function:



10.88.3.19 generatePreparedStatement()

```
std::shared_ptr< sql::PreparedStatement > util::mysql::LocalMysqlConfiguration::generatePrepared↩
Statement (
            const string & statement)  [static], [inherited]
```

Generate a prepared SQL statement object for the given SQL statement.

Parameters

in	*statement*	The SQL statement to prepare. This must be a well-formed SQL string.

Exceptions

log::MessageException	if 'statement' is not a valid SQL string or if an error occurs while generating the prepared statement object..

Returns

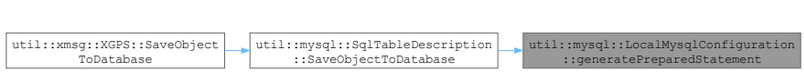
A prepared SQL statement object generated from the given SQL statement.

References util::mysql::MysqlUtils::GeneratePreparedStatement(), and util::mysql::LocalMysqlConfiguration::getSqlConnection().

Referenced by util::mysql::SqlTableDescription< T >::SaveObjectToDatabase().

Here is the call graph for this function:

Here is the caller graph for this function:



10.88.3.20 GenerateSqlDeleteStr()

```
static string util::mysql::MysqlUtils::GenerateSqlDeleteStr (
            const string & from,
            const string & where = "")  [inline], [static], [inherited]
```

Generate an SQL DELETE statement.

Warning

Without a `where` clause, the statement deletes every row in the table.

Parameters

in	*from*	The name of the table to delete from.
in	*where*	The SQL "where-condition" clause (empty = all rows).

Returns

The SQL statement.

10.88.3.21 GenerateSqlSelectStr()

```
static string util::mysql::MysqlUtils::GenerateSqlSelectStr (
            const string & what,
            const string & from,
            const string & where = "",
            const string & groupBy = "",
            const string & orderBy = "",
            unsigned int limit = 0,
            unsigned int offset = 0)  [inline], [static], [inherited]
```

Generate an SQL select statement.

Parameters

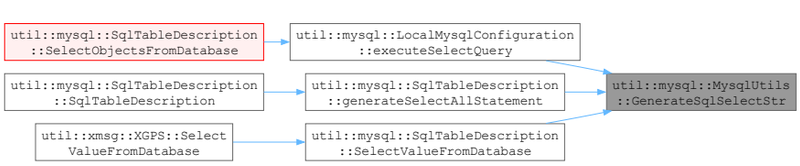
in	*what*	The SQL select statement select-expression clause.
in	*from*	The SQL select statement table-references clause.
in	*where*	The SQL select statement where-condition clause.
in	*groupBy*	The SQL select statement group-by clause.
in	*orderBy*	The SQL select statement order-by clause.
in	*limit*	The SQL select statement result set row limit value.
in	*offset*	The SQL select statement result set row offset.

Returns

The SQL select statement.

Referenced by util::mysql::LocalMysqlConfiguration::executeSelectQuery(), util::mysql::SqlTableDescription< T >::generateSelectAllState
and util::mysql::SqlTableDescription< T >::SelectValueFromDatabase().

Here is the caller graph for this function:



10.88.3.22 GetCurrentUtcTime()

```
static ptime util::date::DateUtils::GetCurrentUtcTime ()  [inline], [static], [inherited]
```

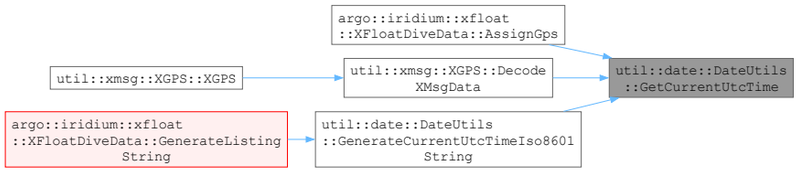
Generate a POSIX time object representing the current date and time (rounded to the nearest second).

Returns

A POSIX time object representing the current date and time (rounded to the nearest second).

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignGps(), util::xmsg::XGPS::DecodeXMsgData(), and util::date::DateUtils::GenerateCurrentUtcTimeIso8601String().

Here is the caller graph for this function:



10.88.3.23 GetDirectoryPath()

```
static const path & util::file::FileSystemUtils::GetDirectoryPath (
            const path & dir,
            bool create = false)  [inline], [static], [inherited]
```

Get a directory path.

Exceptions

MessageException	if create == FALSE and the directory specified by path does not exist.

Parameters

in	*dir*	Directory path.
in	*create*	If TRUE, the directory specified by path will be created if it does not already exists. If FALSE, an exception will be thrown if the directory does not exist.

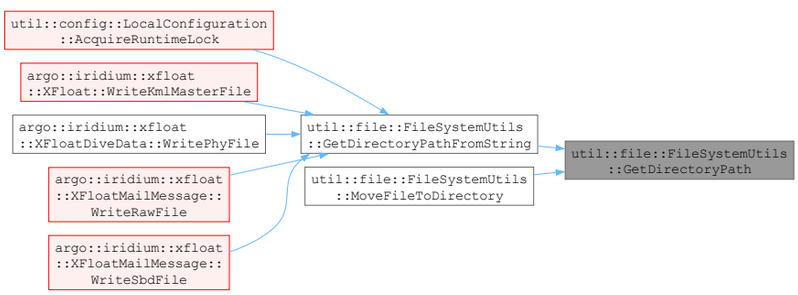
Returns

dir

References CATCH_BLOCK, EXC_THROW, and ldbg.

Referenced by util::file::FileSystemUtils::GetDirectoryPathFromString(), and util::file::FileSystemUtils::MoveFileToDirectory().

Here is the caller graph for this function:



10.88.3.24 GetDirectoryPathFromString()

```
static path util::file::FileSystemUtils::GetDirectoryPathFromString (
            const string & dir,
            bool create = false)  [inline], [static], [inherited]
```

Generate a boost::filesystem::path object from the directory path string.

Exceptions

MessageException	if create == FALSE and the directory specified by path does not exist.

Parameters

in	*dir*	Directory path string.
in	*create*	If TRUE, the directory specified by path will be created if it does not already exists. If FALSE, an exception will be thrown if the directory does not exist.

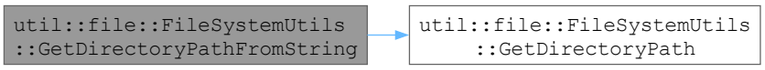
Returns

A boost::filesystem::path object from the directory path string.

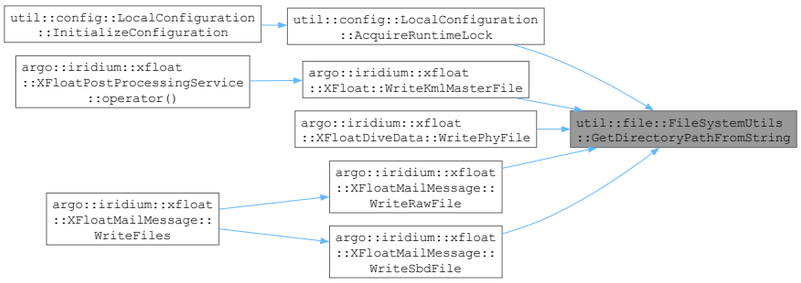
References CATCH_BLOCK, and util::file::FileSystemUtils::GetDirectoryPath().

Referenced by util::config::LocalConfiguration::AcquireRuntimeLock(), argo::iridium::xfloat::XFloat::WriteKmlMasterFile(), argo::iridium::xfloat::XFloatDiveData::WritePhyFile(), WriteRawFile(), and WriteSbdFile().

Here is the call graph for this function:



Here is the caller graph for this function:



10.88.3.25 GetSqlConnection()

```
static thread_specific_ptr< Connection > & util::mysql::MysqlUtils::GetSqlConnection (
            string url,
            string user,
            string password,
            string schema)  [inline], [static], [inherited]
```

Get a thread-specific instance of a MySQL connection object.

A new connection will be created, if necessary.

Parameters

in	*url*	The MySQL host server qualified address.
in	*user*	The MySQL account username.
in	*password*	The MySQL account password.
in	*schema*	The MySQL account schema name.

Exceptions

log::MessageException	if an error occurs while creating a new thread-specific instance of the SQL connection object

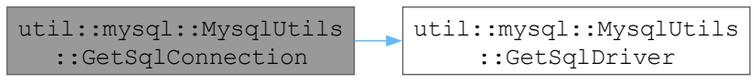
Returns

A thread-specific instance of a MySQL connection object.

References CATCH_BLOCK, util::mysql::MysqlUtils::GetSqlDriver(), and util::mysql::MysqlUtils::sql_connection_.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

Here is the call graph for this function:



Here is the caller graph for this function:

10.88.3.26 getSqlConnection()

`sql::Connection * util::mysql::LocalMysqlConfiguration::getSqlConnection () [static], [protected], [inherited]`

Generate a thread-specific instance of a MySQL connection object.

A new connection will be created, if necessary.

Exceptions

log::MessageException	if an error occurs while generating the SQL connection object.

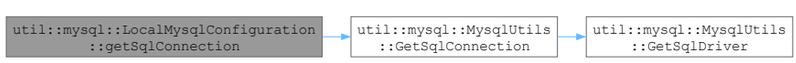
Returns

A thread-specific instance of a MySQL connection object.

References util::mysql::LocalMysqlConfiguration::address_, CATCH_BLOCK, util::mysql::MysqlUtils::GetSqlConnection(), util::mysql::LocalMysqlConfiguration::password_, util::mysql::LocalMysqlConfiguration::port_, util::mysql::LocalMysqlConfiguration::protoc util::mysql::LocalMysqlConfiguration::schema_, and util::mysql::LocalMysqlConfiguration::user_.

Referenced by util::mysql::LocalMysqlConfiguration::executeQuery(), util::mysql::LocalMysqlConfiguration::executeUpdate(), and util::mysql::LocalMysqlConfiguration::generatePreparedStatement().

Here is the call graph for this function:



Here is the caller graph for this function:

10.88.3.27 GetSqlDriver()

```
static thread_specific_ptr< MySQL_Driver > & util::mysql::MysqlUtils::GetSqlDriver ()  [inline],
[static], [inherited]
```

Get a thread-specific instance of a MySQL driver object.

A new driver will created, if necessary.

Exceptions

log::MessageException	if an error occurs while creating a new thread-specific instance of the SQL driver object

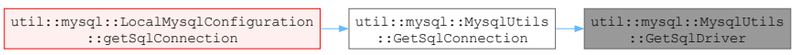
Returns

A thread-specific instance of a MySQL driver object.

References CATCH_BLOCK, and util::mysql::MysqlUtils::sql_driver_.

Referenced by util::mysql::MysqlUtils::GetSqlConnection().

Here is the caller graph for this function:



10.88.3.28 id_dac()

```
int argo::iridium::Iridium::id_dac () const  [inline], [inherited]
```

Get the unique Data Assembly Center (DAC) identifier.

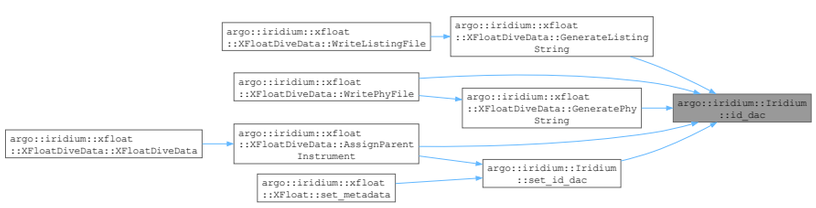
Returns

The unique Data Assembly Center (DAC) identifier.

References argo::iridium::Iridium::id_dac_, and util::thread::RecursiveLockable::mutex_.

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignParentInstrument(), argo::iridium::xfloat::XFloatDiveData::GenerateListingStrin
argo::iridium::xfloat::XFloatDiveData::GeneratePhyString(), argo::iridium::Iridium::set_id_dac(), and argo::iridium::xfloat::XFloatDiveData::

Here is the caller graph for this function:



10.88.3.29 id_dive()

```
int argo::iridium::xfloat::XFloatMailMessage::id_dive () const  [inline]
```

Get the cycle or dive number of this mail message.

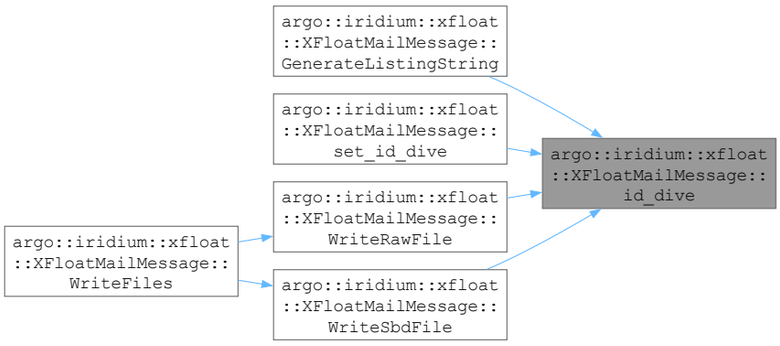
Returns

The cycle or dive number of this mail message.

References id_dive_, and util::thread::RecursiveLockable::mutex_.

Referenced by GenerateListingString(), set_id_dive(), WriteRawFile(), and WriteSbdFile().

Here is the caller graph for this function:



10.88.3.30 id_imei()

```
long argo::iridium::Iridium::id_imei () const  [inline], [inherited]
```

Get the unique International Mobile Equipment Identifier (IMEI).

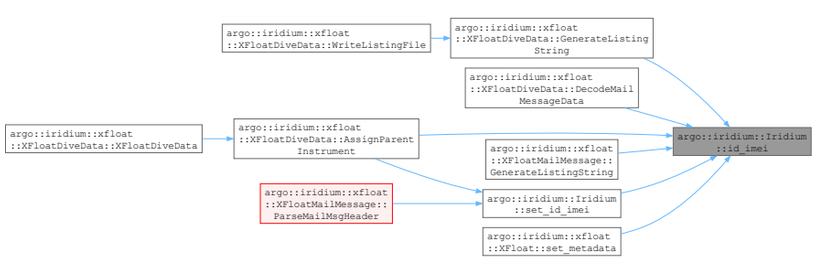
Returns

The unique International Mobile Equipment Identifier (IMEI).

References argo::iridium::Iridium::id_imei_, and util::thread::RecursiveLockable::mutex_.

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignParentInstrument(), argo::iridium::xfloat::XFloatDiveData::DecodeMailMessag
argo::iridium::xfloat::XFloatDiveData::GenerateListingString(), GenerateListingString(), argo::iridium::Iridium::set_id_imei(),
and argo::iridium::xfloat::XFloat::set_metadata().

Here is the caller graph for this function:



10.88.3.31 id_instrument()

```
int argo::iridium::Iridium::id_instrument () const  [inline], [inherited]
```

Get the unique instrument identifier or instrument serial number.

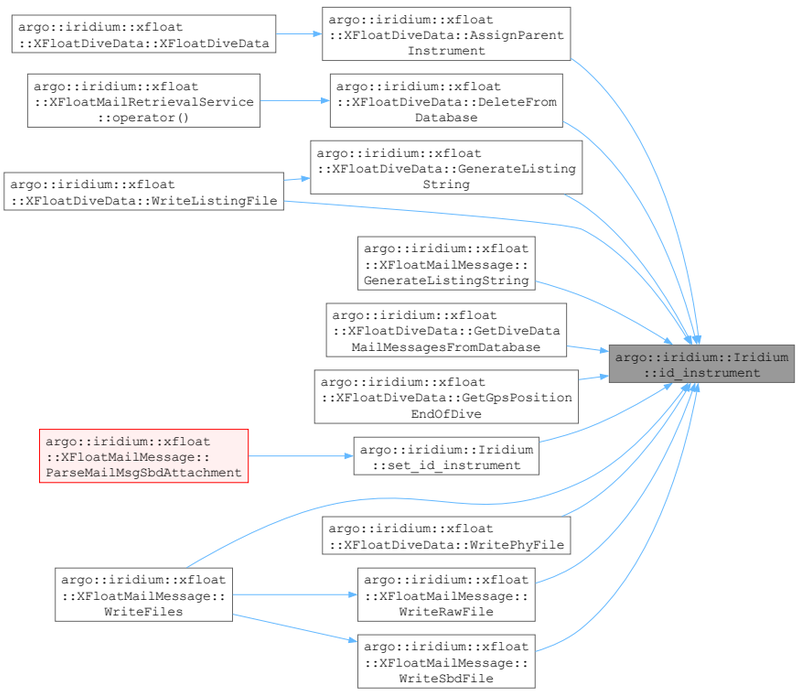
Returns

The unique instrument identifier or instrument serial number.

References argo::iridium::Iridium::id_instrument_, and util::thread::RecursiveLockable::mutex_.

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignParentInstrument(), argo::iridium::xfloat::XFloatDiveData::DeleteFromDatabas
argo::iridium::xfloat::XFloatDiveData::GenerateListingString(), GenerateListingString(), argo::iridium::xfloat::XFloatDiveData::GetDiveData
argo::iridium::xfloat::XFloatDiveData::GetGpsPositionEndOfDive(), argo::iridium::Iridium::set_id_instrument(), WriteFiles(),
argo::iridium::xfloat::XFloatDiveData::WriteListingFile(), argo::iridium::xfloat::XFloatDiveData::WritePhyFile(), WriteRawFile(),
and WriteSbdFile().

Here is the caller graph for this function:



10.88.3.32 id_msg()

```
string argo::iridium::xfloat::XFloatMailMessage::id_msg () const  [inline]
```

Get the mail message unique identifier string.

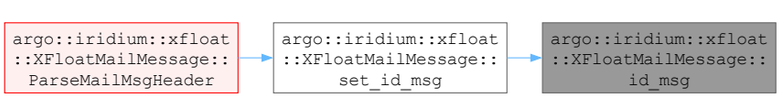
Returns

The mail message unique identifier string.

References id_msg_, and util::thread::RecursiveLockable::mutex_.

Referenced by set_id_msg().

Here is the caller graph for this function:



10.88.3.33 id_msg_uid()

```
long argo::iridium::xfloat::XFloatMailMessage::id_msg_uid () const  [inline]
```

Get the mail message unique identifier.

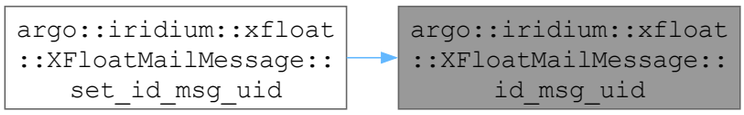
Returns

The mail message unique identifier.

References id_msg_uid_, and util::thread::RecursiveLockable::mutex_.

Referenced by set_id_msg_uid().

Here is the caller graph for this function:



10.88.3.34 id_sbd_momsn()

```
int argo::iridium::xfloat::XFloatMailMessage::id_sbd_momsn () const  [inline]
```

Get the Iridium Short Burst Data (SBD) Service Mobile Originated Message Sequence Number (MOMSN) identifier.

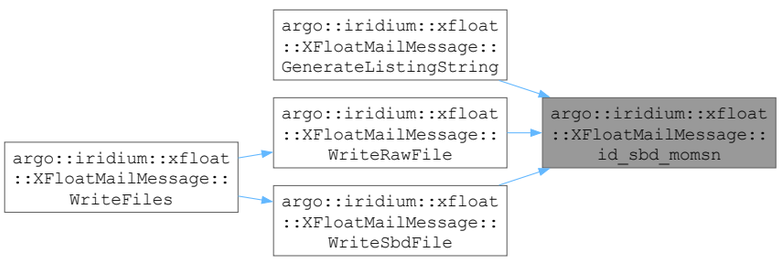
Returns

The Iridium Short Burst Data (SBD) Service Mobile Originated Message Sequence Number (MOMSN) identifier.

References id_sbd_momsn_, and util::thread::RecursiveLockable::mutex_.

Referenced by GenerateListingString(), WriteRawFile(), and WriteSbdFile().

Here is the caller graph for this function:



10.88.3.35 id_sbd_mtmsn()

```
int argo::iridium::xfloat::XFloatMailMessage::id_sbd_mtmsn () const  [inline]
```

Get the Iridium Short Burst Data (SBD) Service Mobile Terminated Message Sequence Number (MTMSN) identifier.

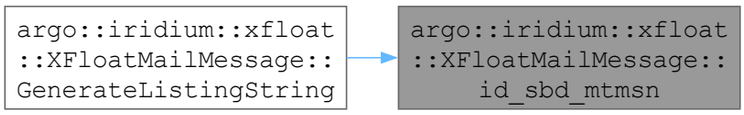
Returns

The Iridium Short Burst Data (SBD) Service Mobile Terminated Message Sequence Number (MTMSN) identifier.

References id_sbd_mtmsn_, and util::thread::RecursiveLockable::mutex_.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.88.3.36 id_sbd_packet()

```
int argo::iridium::xfloat::XFloatMailMessage::id_sbd_packet () const  [inline]
```

Get the Iridium Short Burst Data (SBD) message packet index.

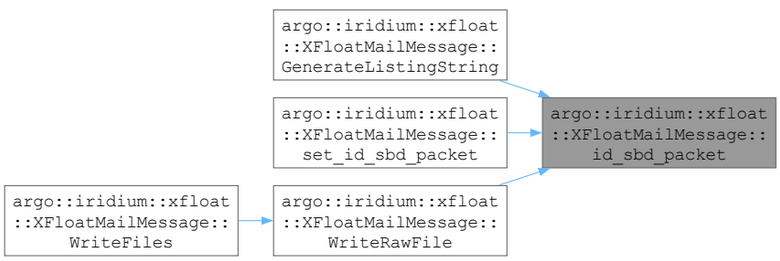
Returns

The Iridium Short Burst Data (SBD) message packet index.

References id_sbd_packet_, and util::thread::RecursiveLockable::mutex_.

Referenced by GenerateListingString(), set_id_sbd_packet(), and WriteRawFile().

Here is the caller graph for this function:



10.88.3.37 id_wmo()

```
int argo::iridium::Iridium::id_wmo () const  [inline], [inherited]
```

Get the unique World Meteorological Organization (WMO) identifier.

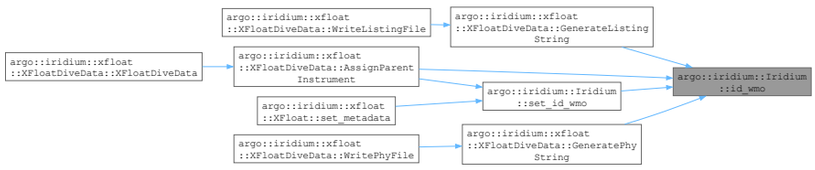
Returns

The unique World Meteorological Organization (WMO) identifier.

References argo::iridium::Iridium::id_wmo_, and util::thread::RecursiveLockable::mutex_.

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignParentInstrument(), argo::iridium::xfloat::XFloatDiveData::GenerateListingStrir
argo::iridium::xfloat::XFloatDiveData::GeneratePhyString(), and argo::iridium::Iridium::set_id_wmo().

Here is the caller graph for this function:



10.88.3.38 iridium_position_as_iso6709_string()

```
string argo::iridium::IridiumPosition::iridium_position_as_iso6709_string () const  [inline],
[inherited]
```

Get a formatted string representing the Iridium position composed using the ISO 6709 standard.

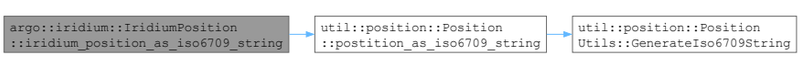
Returns

The formatted string representation of the Iridium position.

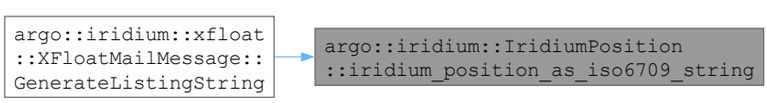
References util::position::Position::postition_as_iso6709_string().

Referenced by GenerateListingString().

Here is the call graph for this function:



Here is the caller graph for this function:



10.88.3.39 iridium_position_cep_radius()

```
int argo::iridium::IridiumPosition::iridium_position_cep_radius () const  [inline], [inherited]
```

Get the Iridium position Circular Error Probable (CEP) radius, in kilometers.

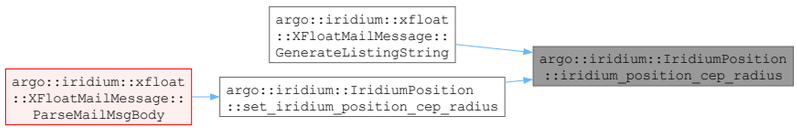
Returns

The Iridium position Circular Error Probable (CEP) radius, in kilometers

References argo::iridium::IridiumPosition::iridium_position_cep_radius_.

Referenced by GenerateListingString(), and argo::iridium::IridiumPosition::set_iridium_position_cep_radius().

Here is the caller graph for this function:



10.88.3.40 iridium_position_datetime()

```
ptime argo::iridium::IridiumPosition::iridium_position_datetime () const  [inline], [inherited]
```

Get the Iridium position date/time.

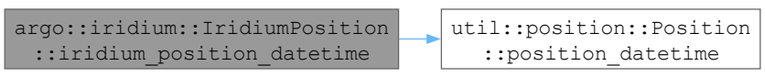
Returns

The Iridium position date/time.

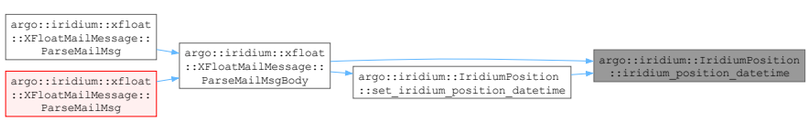
References util::position::Position::position_datetime().

Referenced by ParseMailMsgBody(), and argo::iridium::IridiumPosition::set_iridium_position_datetime().

Here is the call graph for this function:



Here is the caller graph for this function:



10.88.3.41 iridium_position_datetime_as_iso8601_string()

```
string argo::iridium::IridiumPosition::iridium_position_datetime_as_iso8601_string () const  [inline], [inherited]
```

Get a formatted string representing the Iridium position date/time composed using the ISO 8601 standard.

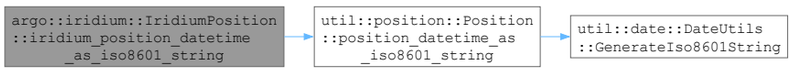
Returns

The formatted string representation of the Iridium position date/time.

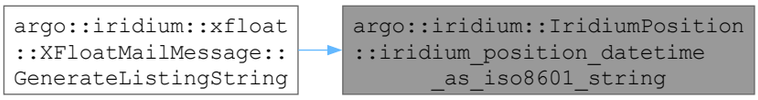
References util::position::Position::position_datetime_as_iso8601_string().

Referenced by GenerateListingString().

Here is the call graph for this function:



Here is the caller graph for this function:



10.88.3.42 iridium_position_is_valid()

```
bool argo::iridium::IridiumPosition::iridium_position_is_valid () const  [inline], [inherited]
```

Get the Iridium position is valid flag.

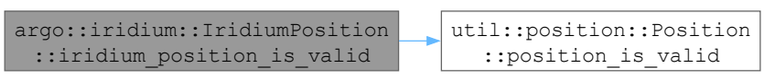
Returns

TRUE if the Iridium position is valid, or FALSE otherwise.

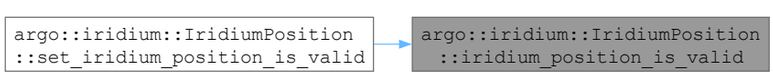
References util::position::Position::position_is_valid().

Referenced by argo::iridium::IridiumPosition::set_iridium_position_is_valid().

Here is the call graph for this function:



Here is the caller graph for this function:



10.88.3.43 iridium_position_latitude()

```
double argo::iridium::IridiumPosition::iridium_position_latitude () const  [inline], [inherited]
```

Get the Iridium position latitude.

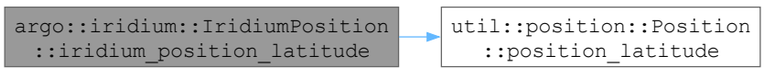
Returns

The Iridium position latitude.

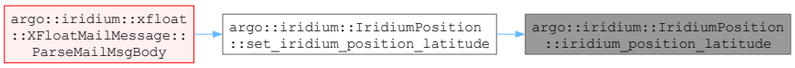
References util::position::Position::position_latitude().

Referenced by argo::iridium::IridiumPosition::set_iridium_position_latitude().

Here is the call graph for this function:



Here is the caller graph for this function:



10.88.3.44 iridium_position_longitude()

```
double argo::iridium::IridiumPosition::iridium_position_longitude () const  [inline], [inherited]
```

Get the Iridium position longitude.

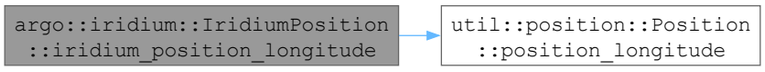
Returns

The Iridium position longitude.

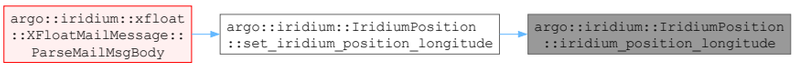
References util::position::Position::position_longitude().

Referenced by argo::iridium::IridiumPosition::set_iridium_position_longitude().

Here is the call graph for this function:



Here is the caller graph for this function:



10.88.3.45 IsPositionValid()

```
template<typename T >
static bool util::position::PositionUtils::IsPositionValid (
            T lat,
            T lon)  [inline], [static], [inherited]
```

Check position validity.

Parameters

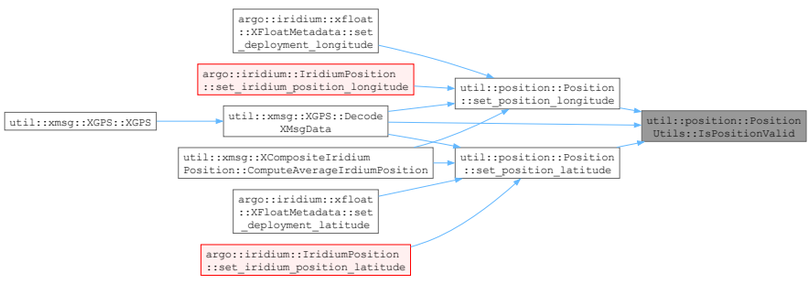
in	*lat*	Latitude of position.
in	*lon*	Longitude of position.

Returns

TRUE if (lat, lon) represents a valid geographical position, or FALSE otherwise.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), util::position::Position::set_position_latitude(), and util::position::Position::set_positio

Here is the caller graph for this function:



10.88.3.46 ListDirectories()

```
static vector< path > util::file::FileSystemUtils::ListDirectories (
            const path & dir)  [inline], [static], [inherited]
```

List all directories contained in the given directory.

Parameters

in	*dir*	The path to the directory of interest.

Exceptions

log::MessageException	if the directory does not exist, is not a directory, or an error occurs.

Returns

A vector of paths to all the directories contained in the given directory.

References CATCH_BLOCK, and EXC_THROW.

10.88.3.47 ListFiles()

```
static vector< path > util::file::FileSystemUtils::ListFiles (
            path dir,
            size_t n = 0)  [inline], [static], [inherited]
```

List all regular files contained in the given directory.

Parameters

in	*dir*	The path to the directory of interest.
in	*n*	If n>0, the maximum number of paths to return, else no effect.

Exceptions

log::MessageException	if the directory does not exist, is not a directory, or an error occurs.

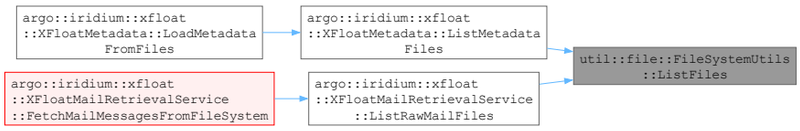
Returns

A vector of paths to all the regular files contained in the given directory.

References CATCH_BLOCK, and EXC_THROW.

Referenced by argo::iridium::xfloat::XFloatMetadata::ListMetadataFiles(), and argo::iridium::xfloat::XFloatMailRetrievalService::ListRawMa

Here is the caller graph for this function:



10.88.3.48 LockObject()

```
bool util::thread::RecursiveLockable::LockObject (
            bool block)  [inline], [inherited]
```

Lock the object.

Parameters

in	*block*	If TRUE, then the calling thread will block until the lock becomes available.

Exceptions

util::log::MessageException	if an error occurs.

Returns

TRUE if the lock was successfully obtained, or FALSE if block is FALSE and another thread owns the lock on the mutex.

References CATCH_BLOCK, and util::thread::RecursiveLockable::mutex_.

10.88.3.49 MoveFileToDirectory()

```
static path util::file::FileSystemUtils::MoveFileToDirectory (
            const path & file,
            const path & dir_dst,
            bool create_dir = false)  [inline], [static], [inherited]
```

Copy a file into a directory.

Warning

Despite its name, this function copies the file and does not remove the source. If a file with the same name already exists in the destination directory, it is left unchanged. It is not currently called.

Parameters

in	*file*	The file to copy.
in	*dir_dst*	The destination directory.
in	*create_dir*	If TRUE, create the destination directory if it does not exist.

Exceptions

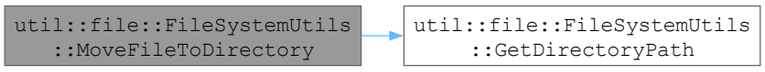
util::log::MessageException	if the file does not exist, is not a regular file, or cannot be copied.

Returns

The path of the file in the destination directory.

References CATCH_BLOCK, EXC_THROW, and util::file::FileSystemUtils::GetDirectoryPath().

Here is the call graph for this function:



10.88.3.50 msg_body_is_valid()

```
bool argo::iridium::xfloat::XFloatMailMessage::msg_body_is_valid () const  [inline]
```

Is the body of the mail message valid?

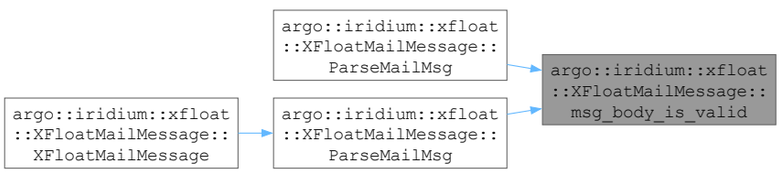
Returns

TRUE if the body of the mail message is valid, or FALSE otherwise.

References msg_body_is_valid_, and util::thread::RecursiveLockable::mutex_.

Referenced by ParseMailMsg(), and ParseMailMsg().

Here is the caller graph for this function:



10.88.3.51 msg_body_text()

```
string argo::iridium::xfloat::XFloatMailMessage::msg_body_text () const  [inline]
```

Get the mail message body text.

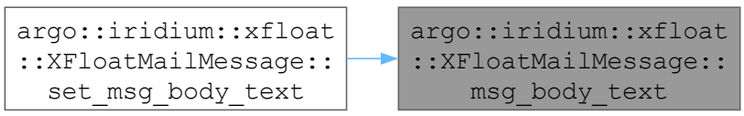
Returns

The mail message body text.

References msg_body_text_, and util::thread::RecursiveLockable::mutex_.

Referenced by set_msg_body_text().

Here is the caller graph for this function:



10.88.3.52 msg_header_datetime()

```
ptime argo::iridium::xfloat::XFloatMailMessage::msg_header_datetime () const  [inline]
```

Get the mail message header date/time.

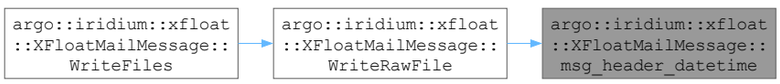
Returns

The mail message header date/time.

References msg_header_datetime_, and util::thread::RecursiveLockable::mutex_.

Referenced by WriteRawFile().

Here is the caller graph for this function:



10.88.3.53 msg_header_from()

```
string argo::iridium::xfloat::XFloatMailMessage::msg_header_from () const  [inline]
```

Get the mail message header FROM field.

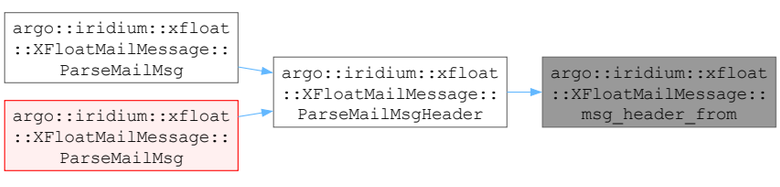
Returns

The mail message header FROM field.

References msg_header_from_, and util::thread::RecursiveLockable::mutex_.

Referenced by ParseMailMsgHeader().

Here is the caller graph for this function:



10.88.3.54 msg_header_is_valid()

```
bool argo::iridium::xfloat::XFloatMailMessage::msg_header_is_valid () const  [inline]
```

Is the header of the mail message valid?

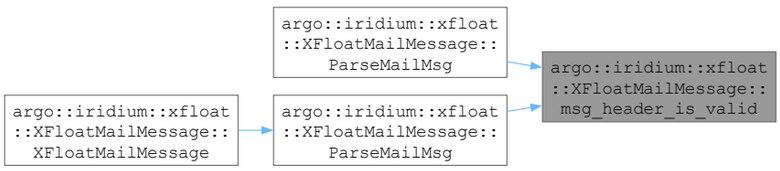
Returns

TRUE if the header of the mail message is valid, or FALSE otherwise.

References msg_header_is_valid_, and util::thread::RecursiveLockable::mutex_.

Referenced by ParseMailMsg(), and ParseMailMsg().

Here is the caller graph for this function:



10.88.3.55 msg_header_raw_text()

```
string argo::iridium::xfloat::XFloatMailMessage::msg_header_raw_text () const  [inline]
```

Get the message header raw text.

Returns

The message header raw text.

References msg_header_raw_text_, and util::thread::RecursiveLockable::mutex_.

10.88.3.56 msg_header_subject()

```
string argo::iridium::xfloat::XFloatMailMessage::msg_header_subject () const  [inline]
```

Get the mail message header SUBJECT field.

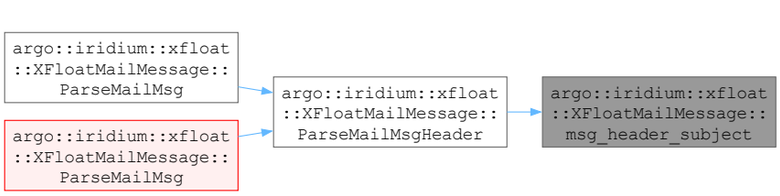
Returns

The mail message header SUBJECT field.

References msg_header_subject_, and util::thread::RecursiveLockable::mutex_.

Referenced by ParseMailMsgHeader().

Here is the caller graph for this function:



10.88.3.57 msg_is_valid()

```
bool argo::iridium::xfloat::XFloatMailMessage::msg_is_valid () const  [inline]
```

Is the mail message valid?

Returns

TRUE if the mail message is valid, or FALSE otherwise.

References msg_is_valid_, and util::thread::RecursiveLockable::mutex_.

10.88.3.58 msg_raw_text()

```
string argo::iridium::xfloat::XFloatMailMessage::msg_raw_text () const  [inline]
```

Get the full mail message raw text.

Returns

The full mail message raw text.

References msg_raw_text_, and util::thread::RecursiveLockable::mutex_.

10.88.3.59 ParseMailMsg() [1/2]

```
void argo::iridium::xfloat::XFloatMailMessage::ParseMailMsg (
            const vmime::shared_ptr< vmime::message > & msg)  [inline], [private]
```

Parse a mail message.

This function is used to parse and validate a mail message. As a result of parsing the mail message, the fields of this class are populated.

Exceptions

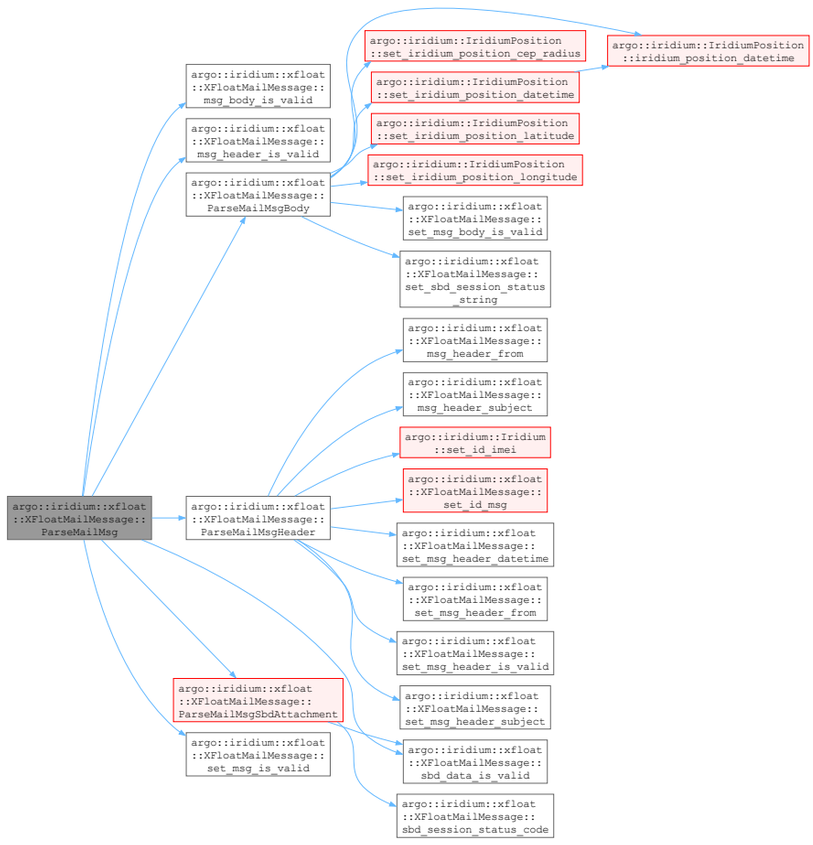
log::util::MessageException	if `msg` is NULL or an error occurs while parsing the various parts of the mail message.

Parameters

`in`	*msg*	The mail message to parse.

References CATCH_BLOCK, EXC_THROW, msg_body_is_valid(), msg_header_is_valid(), msg_raw_text_, util::thread::RecursiveLockable::mutex_, ParseMailMsgBody(), ParseMailMsgHeader(), ParseMailMsgSbdAttachment(), sbd_data_is_valid(), and set_msg_is_valid().

Here is the call graph for this function:



10.88.3.60 ParseMailMsg() [2/2]

```
void argo::iridium::xfloat::XFloatMailMessage::ParseMailMsg (
            const vmime::shared_ptr< vmime::net::message > & msg)  [inline], [private]
```

Parse a mail message.

This function is used to parse and validate a mail message. As a result of parsing the mail message, the fields of this class are populated.

Exceptions

log::util::MessageException	if `msg` is NULL or an error occurs while parsing the various parts of the mail message.

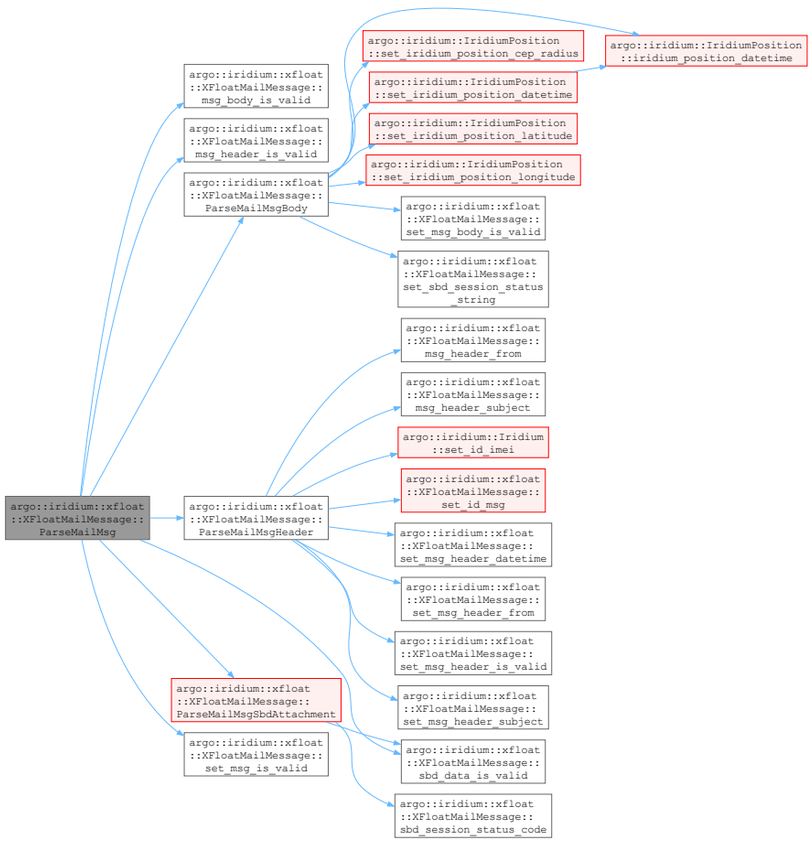
Parameters

`in`	*msg*	The mail message to parse.

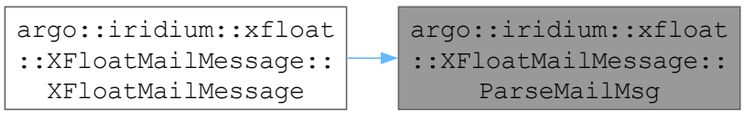
References CATCH_BLOCK, EXC_THROW, id_msg_uid_, msg_body_is_valid(), msg_header_is_valid(), msg_raw_text_, util::thread::RecursiveLockable::mutex_, ParseMailMsgBody(), ParseMailMsgHeader(), ParseMailMsgSbdAttachment(), sbd_data_is_valid(), and set_msg_is_valid().

Referenced by XFloatMailMessage().

Here is the call graph for this function:



Here is the caller graph for this function:



10.88.3.61 ParseMailMsgBody()

```
void argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgBody (
            const vmime::shared_ptr< vmime::message > & msg)  [inline], [private]
```

Parse the mail message body.

This function is used to parse and validate the mail message body. The body of a valid mail message includes information about the Iridium session which are parsed in order to populate the corresponding fields of this class.

Exceptions

log::util::MessageException	if `msg` is NULL or an error occurs while parsing the body of the mail message.

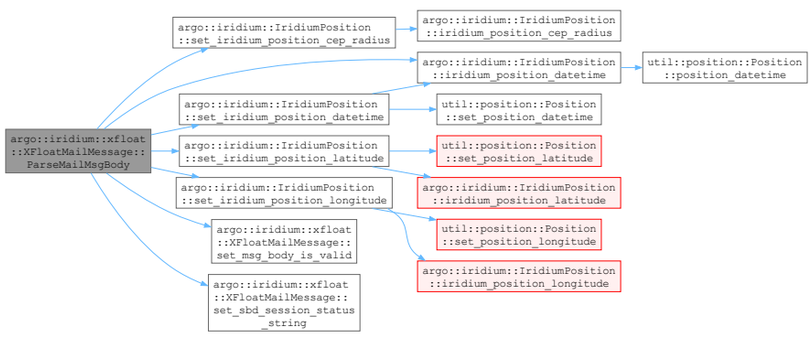
Parameters

`in`	*msg*	The message to parse

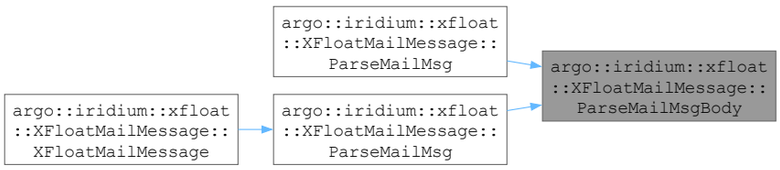
References CATCH_BLOCK, EXC_THROW, id_sbd_momsn_, id_sbd_mtmsn_, argo::iridium::IridiumPosition::iridium_position_datetime(), msg_body_text_, util::thread::RecursiveLockable::mutex_, sbd_file_size_bytes_, sbd_session_status_code_, argo::iridium::IridiumPosition::set_iridium_position_cep_radius(), argo::iridium::IridiumPosition::set_iridium_position_datetime(), argo::iridium::IridiumPosition::set_iridium_position_latitude(), argo::iridium::IridiumPosition::set_iridium_position_longitude(), set_msg_body_is_valid(), set_sbd_session_status_string(), and XMSG_ID_UNKNOWN.

Referenced by ParseMailMsg(), and ParseMailMsg().

Here is the call graph for this function:



Here is the caller graph for this function:



10.88.3.62 ParseMailMsgHeader()

```
void argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgHeader (
            const vmime::shared_ptr< const vmime::header > & head)  [inline], [private]
```

Parse the mail message header.

This function is used to parse and validate the mail message header. As a result of parsing the mail message, the field of this class relating to the mail message header are populated.

Exceptions

log::util::MessageException	if `head` is NULL or an error occurs while parsing the header of the mail message.

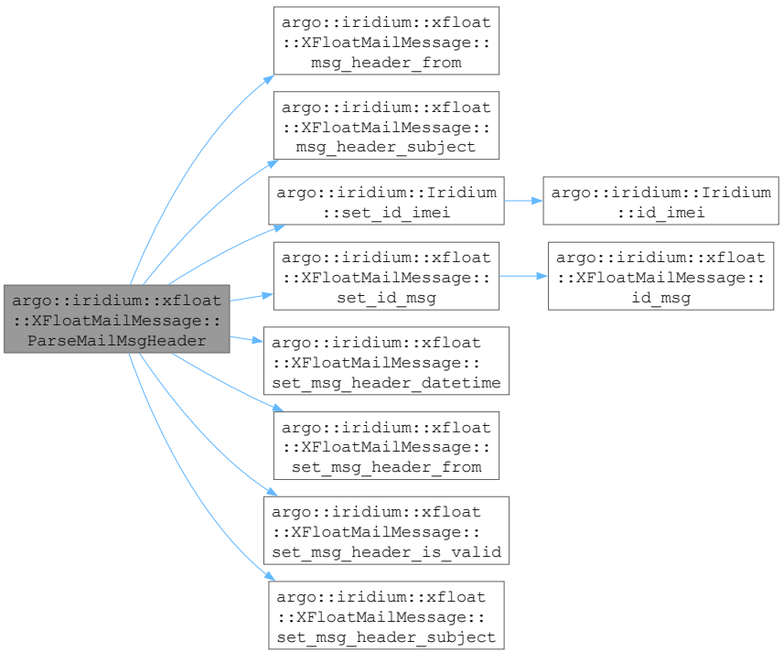
Parameters

`in`	*head*	The mail message header to parse.

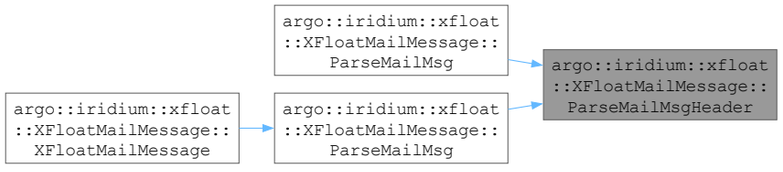
References CATCH_BLOCK, EXC_THROW, msg_header_from(), msg_header_raw_text_, msg_header_subject(), msg_header_valid_sender_regex_, msg_header_valid_subject_regex_, util::thread::RecursiveLockable::mutex_, argo::iridium::Iridium::set_id_imei(), set_id_msg(), set_msg_header_datetime(), set_msg_header_from(), set_msg_header_is_valid(), and set_msg_header_subject().

Referenced by ParseMailMsg(), and ParseMailMsg().

Here is the call graph for this function:



Here is the caller graph for this function:



10.88.3.63 ParseMailMsgSbdAttachment()

```
void argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgSbdAttachment (
            const vmime::shared_ptr< vmime::message > & msg)  [inline], [private]
```

Parse the mail message SBD attachment.

This function is used to parse and validate the attachment of the mail message. A valid mail message contains a binary SBD file attachment that represents the data transmitted by the instrument.

Exceptions

log::util::MessageException	if `msg` is NULL or an error occurs while parsing the attachment of the mail message.

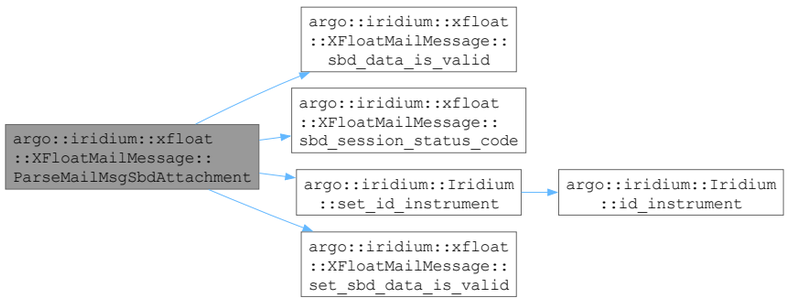
Parameters

`in`	*msg*	The mail message to parse.

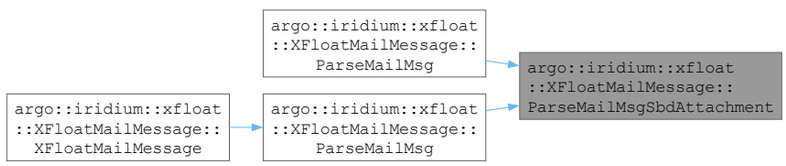
References CATCH_BLOCK, EXC_THROW, i16, id_dive_, id_sbd_packet_, util::thread::RecursiveLockable::mutex_, sbd_data_, sbd_data_is_valid(), sbd_file_name_, sbd_file_valid_name_regex_, sbd_session_status_code(), argo::iridium::Iridium::set_id_instrument(), set_sbd_data_is_valid(), u08, u16, and XMSG_SBD_MIN_BYTES.

Referenced by ParseMailMsg(), and ParseMailMsg().

Here is the call graph for this function:



Here is the caller graph for this function:



10.88.3.64 position_datetime()

```
ptime util::position::Position::position_datetime () const  [inline], [inherited]
```

Get the date/time of the position.

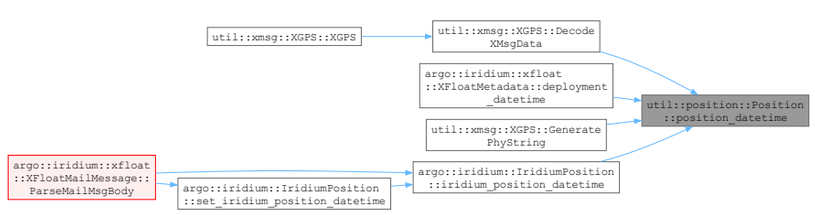
Returns

The date/time of the position.

References util::position::Position::position_datetime_.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), argo::iridium::xfloat::XFloatMetadata::deployment_datetime(), util::xmsg::XGPS::GeneratePhyString(), and argo::iridium::IridiumPosition::iridium_position_datetime().

Here is the caller graph for this function:



10.88.3.65 position_datetime_as_iso8601_string()

```
string util::position::Position::position_datetime_as_iso8601_string () const  [inline], [inherited]
```

Get a formatted string representing the position date/time composed using the ISO 8601 standard.

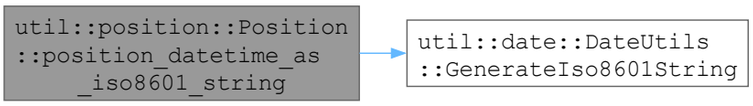
Returns

The formatted string representation of the position date/time.

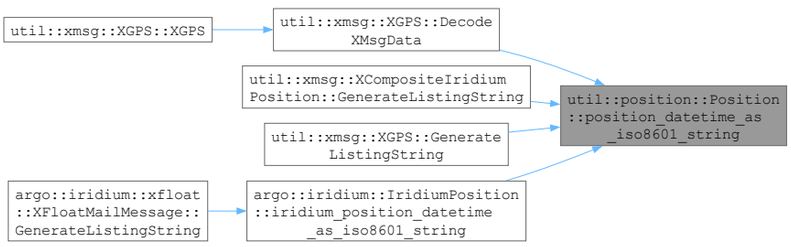
References util::date::DateUtils::GenerateIso8601String(), and util::position::Position::position_datetime_.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XCompositeIridiumPosition::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), and argo::iridium::IridiumPosition::iridium_position_datetime_as_iso8601_string().

Here is the call graph for this function:



Here is the caller graph for this function:



10.88.3.66 position_is_valid()

```
bool util::position::Position::position_is_valid () const  [inline], [inherited]
```

Is the position valid?

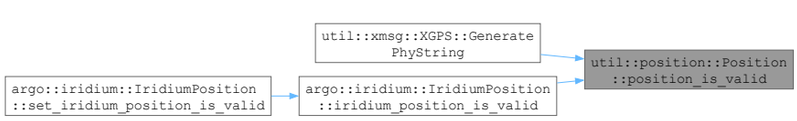
Returns

TRUE if the position is valid, or false otherwise.

References util::position::Position::position_is_valid_.

Referenced by util::xmsg::XGPS::GeneratePhyString(), and argo::iridium::IridiumPosition::iridium_position_is_valid().

Here is the caller graph for this function:



10.88.3.67 position_latitude()

```
double util::position::Position::position_latitude () const  [inline], [inherited]
```

Get the latitude of the position, in decimal degrees.

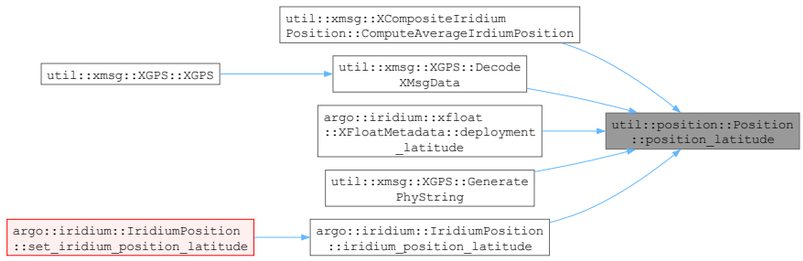
Returns

The latitude of the position, in decimal degrees.

References util::position::Position::position_latitude_.

Referenced by util::xmsg::XCompositeIridiumPosition::ComputeAverageIrdiumPosition(), util::xmsg::XGPS::DecodeXMsgData(), argo::iridium::xfloat::XFloatMetadata::deployment_latitude(), util::xmsg::XGPS::GeneratePhyString(), and argo::iridium::IridiumPosition::iri

Here is the caller graph for this function:



10.88.3.68 position_longitude()

```
double util::position::Position::position_longitude () const  [inline], [inherited]
```

Get the longitude of the position, in decimal degrees.

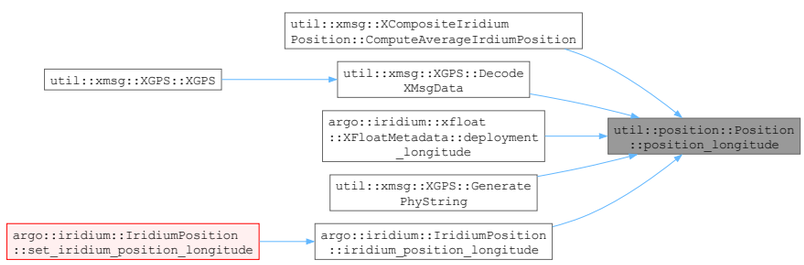
Returns

The longitude of the position, in decimal degrees.

References util::position::Position::position_longitude_.

Referenced by util::xmsg::XCompositeIridiumPosition::ComputeAverageIrdiumPosition(), util::xmsg::XGPS::DecodeXMsgData(), argo::iridium::xfloat::XFloatMetadata::deployment_longitude(), util::xmsg::XGPS::GeneratePhyString(), and argo::iridium::IridiumPosition::

Here is the caller graph for this function:



10.88.3.69 postition_as_iso6709_string()

```
string util::position::Position::postition_as_iso6709_string () const  [inline], [inherited]
```

Get a formatted string representing the position composed using the ISO 6709 standard.

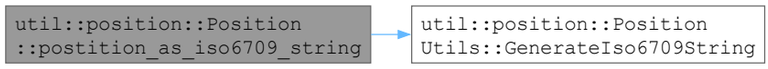
Returns

The formatted string representation of the position.

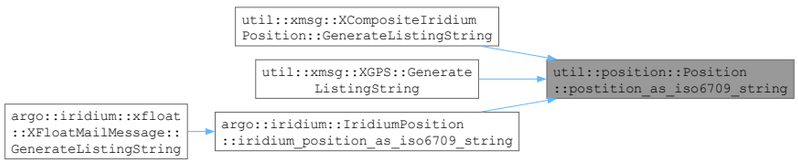
References util::position::PositionUtils::GenerateIso6709String(), util::position::Position::position_latitude_, and util::position::Position::position_longitude_.

Referenced by util::xmsg::XCompositeIridiumPosition::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), and argo::iridium::IridiumPosition::iridium_position_as_iso6709_string().

Here is the call graph for this function:



Here is the caller graph for this function:



10.88.3.70 raw_file_local_path()

```
path argo::iridium::xfloat::XFloatMailMessage::raw_file_local_path () const  [inline]
```

Get the path to the raw mail message file on the local filesystem.

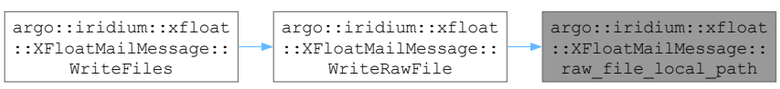
Returns

The path to the raw mail message file on the local filesystem.

References util::thread::RecursiveLockable::mutex_, and raw_file_local_path_.

Referenced by WriteRawFile().

Here is the caller graph for this function:



10.88.3.71 SaveObjectToDatabase()

```
void argo::iridium::xfloat::XFloatMailMessage::SaveObjectToDatabase () const  [inline], [virtual]
```

Save this mail message object to the SQL database.

Exceptions

util::log::MessageException	if an error occurs.

Implements util::mysql::LocalPersistentEntity.

References CATCH_BLOCK, util::thread::RecursiveLockable::mutex_, and sql_table_description_.

10.88.3.72 sbd_data()

```
vector< uint8_t > argo::iridium::xfloat::XFloatMailMessage::sbd_data () const  [inline]
```

Get the Iridium Short Burst Data (SBD) binary data.

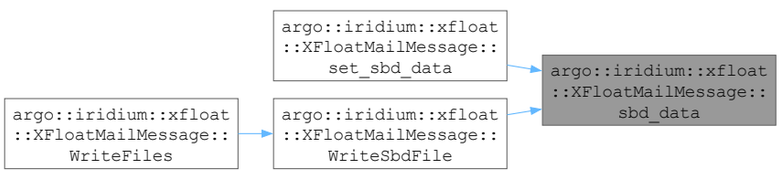
Returns

The Iridium Short Burst Data (SBD) binary data.

References util::thread::RecursiveLockable::mutex_, and sbd_data_.

Referenced by set_sbd_data(), and WriteSbdFile().

Here is the caller graph for this function:



10.88.3.73 sbd_data_is_valid()

```
bool argo::iridium::xfloat::XFloatMailMessage::sbd_data_is_valid () const  [inline]
```

Is the Iridium Short Burst Data (SBD) message valid?

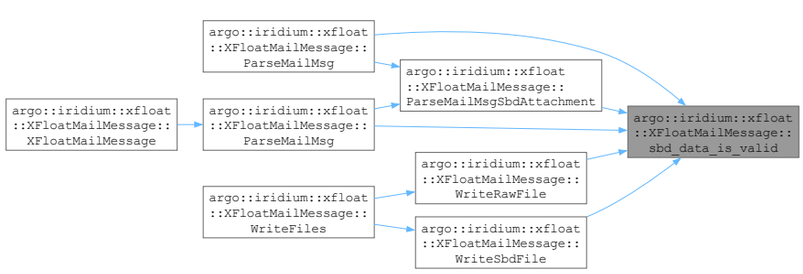
Returns

TRUE if the Iridium Short Burst Data (SBD) message is valid, or FALSE otherwise.

References util::thread::RecursiveLockable::mutex_, and sbd_data_is_valid_.

Referenced by ParseMailMsg(), ParseMailMsg(), ParseMailMsgSbdAttachment(), WriteRawFile(), and WriteSbdFile().

Here is the caller graph for this function:



10.88.3.74 sbd_file_local_path()

```
path argo::iridium::xfloat::XFloatMailMessage::sbd_file_local_path () const  [inline]
```

Get the path to the Iridium Short Burst Data (SBD) binary file on the local filesystem.

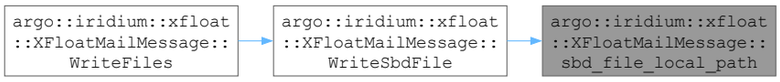
Returns

The path to the Iridium Short Burst Data (SBD) binary file on the local filesystem.

References util::thread::RecursiveLockable::mutex_, and sbd_file_local_path_.

Referenced by WriteSbdFile().

Here is the caller graph for this function:



10.88.3.75 sbd_file_name()

```
string argo::iridium::xfloat::XFloatMailMessage::sbd_file_name () const  [inline]
```

Get the Iridium Short Burst Data (SBD) email attachment filename.

Returns

The Iridium Short Burst Data (SBD) email attachment filename.

References util::thread::RecursiveLockable::mutex_, and sbd_file_name_.

10.88.3.76 sbd_file_size_bytes()

```
int argo::iridium::xfloat::XFloatMailMessage::sbd_file_size_bytes () const  [inline]
```

Get the size of the Iridium Short Burst Data (SBD) message, in bytes.

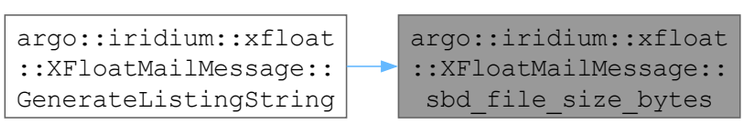
Returns

The size of the Iridium Short Burst Data (SBD) message, in bytes.

References util::thread::RecursiveLockable::mutex_, and sbd_file_size_bytes_.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.88.3.77 sbd_session_status_code()

```
int argo::iridium::xfloat::XFloatMailMessage::sbd_session_status_code () const  [inline]
```

Get the Iridium Short Burst Data (SBD) session status code.

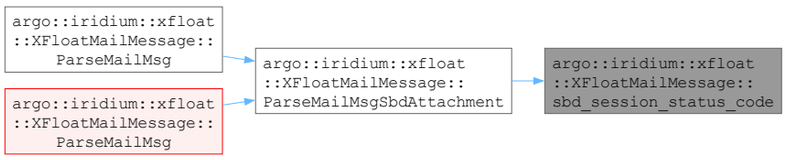
Returns

The Iridium Short Burst Data (SBD) session status code.

References util::thread::RecursiveLockable::mutex_, and sbd_session_status_code_.

Referenced by ParseMailMsgSbdAttachment().

Here is the caller graph for this function:



10.88.3.78 sbd_session_status_string()

```
string argo::iridium::xfloat::XFloatMailMessage::sbd_session_status_string () const  [inline]
```

Get the Iridium Short Burst Data (SBD) session status string.

Returns

The Iridium Short Burst Data (SBD) session status string.

References util::thread::RecursiveLockable::mutex_, and sbd_session_status_string_.

10.88.3.79 SelectAllObjectsFromDatabase()

```
static vector< shared_ptr< XFloatMailMessage > > argo::iridium::xfloat::XFloatMailMessage::↩
SelectAllObjectsFromDatabase (
            const string & groupBy = "",
            const string & orderBy = "",
            const unsigned int & limit = 0,
            const unsigned int & offset = 0)  [inline], [static], [private]
```

Select all mail message objects from the SQL database table.

Parameters

in	*groupBy*	The SQL select statement "group-by expression" clause.
in	*orderBy*	The SQL select statement "order-by expression" clause.
in	*limit*	The SQL select statement result set row limit value (0 = no limit).
in	*offset*	The SQL select statement result set row offset value.

Exceptions

util::log::MessageException	if the query fails.

Returns

The mail message objects.

References CATCH_BLOCK, and sql_table_description_.

10.88.3.80 SelectObjectFromDatabase()

```
static shared_ptr< XFloatMailMessage > argo::iridium::xfloat::XFloatMailMessage::SelectObject↩
FromDatabase (
            const string & where = "",
            const string & groupBy = "",
            const string & orderBy = "",
            const unsigned int & offset = 0)  [inline], [static], [private]
```

Select a single mail message object from the SQL database table.

Parameters

in	*where*	The SQL select statement "where-condition" clause.
in	*groupBy*	The SQL select statement "group-by expression" clause.
in	*orderBy*	The SQL select statement "order-by expression" clause.
in	*offset*	The SQL select statement result set row offset value.

Exceptions

util::log::MessageException	if the query fails.

Returns

The first mail message object matching the query.

References CATCH_BLOCK, and sql_table_description_.

10.88.3.81 SelectObjectsFromDatabase()

```
static vector< shared_ptr< XFloatMailMessage > > argo::iridium::xfloat::XFloatMailMessage::↩
SelectObjectsFromDatabase (
            const string & where = "",
            const string & groupBy = "",
            const string & orderBy = "",
            const unsigned int & limit = 0,
            const unsigned int & offset = 0)  [inline], [static], [private]
```

Query mail message objects from the SQL database.

This function is used to query the SQL database table corresponding to the mail message class and construct a vector of mail message objects, populated from the results of the SQL query.

Exceptions

util::log::MessageException	if an error occurs while querying the database or constructing the float objects from the result of the query.

Parameters

in	*where*	The SQL select statement "where-condition" clause.
in	*groupBy*	The SQL select statement "group-by expression" clause.
in	*orderBy*	The SQL select statement "order-by expression" clause.
in	*limit*	The SQL select statement result set row limit value.
in	*offset*	The SQL select statement result set row offset value.

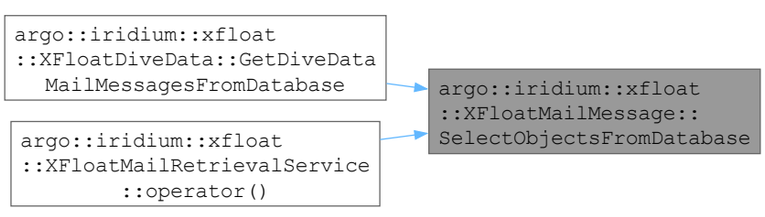
Returns

A vector of shared references to the float objects constructed from the results of the SQL query.

References CATCH_BLOCK, and sql_table_description_.

Referenced by argo::iridium::xfloat::XFloatDiveData::GetDiveDataMailMessagesFromDatabase(), and argo::iridium::xfloat::XFloatMailRetr

Here is the caller graph for this function:



10.88.3.82 SelectValueFromDatabase()

```
template<typename TT >
static TT argo::iridium::xfloat::XFloatMailMessage::SelectValueFromDatabase (
            const string & what,
            const string & where = "",
            const string & groupBy = "",
            const string & orderBy = "",
            const unsigned int & offset = 0)  [inline], [static], [private]
```

Select a value from the SQL database table that contains the persisted mail message objects.

This function may be used to select a singular value from the table that comprises the persisted mail message objects.

Exceptions

util::log::MessageException	if the SQL database query fails or if an error occurs while attempting to interpret the result of the query.

Note

The result set of this query is always limited to a single result. That is, the result set row limit value is automatically set to one. If more than one row in the database table satisfy the "where condition" clause of the select statement, then only the value of the first result will be returned.

Parameters

in	*what*	The SQL select statement "what" clause. This clause may only contain a singular result e.g. the name of the SQL database table column of interest.
in	*where*	The SQL select statement "where condition" clause.
in	*groupBy*	The SQL select statement "group-by expression" clause.
in	*orderBy*	The SQL select statement "order-by expression" clause.
in	*offset*	The SQL select statement result set row offset value.

Returns

The selected value from the SQL database table.

References sql_table_description_.

10.88.3.83 set_gps_position_datetime()

```
void util::position::Position::set_gps_position_datetime (
            unsigned int gpsWeek,
            unsigned int gpsDay,
            unsigned int gpsHour = 0,
            unsigned int gpsMinute = 0,
            unsigned int gpsSecond = 0)  [inline], [protected], [inherited]
```

Set the date and time of this position from the given GPS week, day, and time.

Parameters

in	*gpsWeek*	The GPS week, in weeks since midnight of January 6, 1980.
in	*gpsDay*	The GPS day of week (0 - 6).
in	*gpsHour*	The UTC hour of day (0 - 23).
in	*gpsMinute*	The UTC minute of hour (0 - 59)..
in	*gpsSecond*	The UTC second of minute (0 - 59).

References util::position::Position::set_position_datetime().

Referenced by util::xmsg::XGPS::DecodeXMsgData().

Here is the call graph for this function:

Here is the caller graph for this function:

10.88.3.84 set_id_dac()

```
void argo::iridium::Iridium::set_id_dac (
            int id_dac)  [inline], [protected], [inherited]
```

Set the unique Data Assembly Center (DAC) identifier.

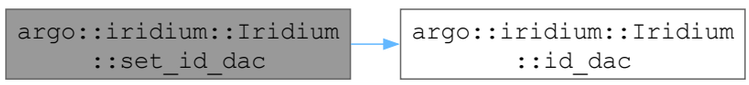
Parameters

in	*id_dac*	The unique Data Assembly Center (DAC) identifier.

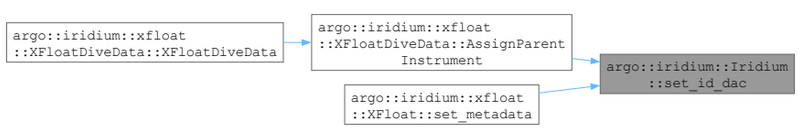
References argo::iridium::Iridium::id_dac(), argo::iridium::Iridium::id_dac_, and util::thread::RecursiveLockable::mutex_.

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignParentInstrument(), and argo::iridium::xfloat::XFloat::set_metadata().

Here is the call graph for this function:



Here is the caller graph for this function:



10.88.3.85 set_id_dive()

```
void argo::iridium::xfloat::XFloatMailMessage::set_id_dive (
            int id_dive)  [inline], [private]
```

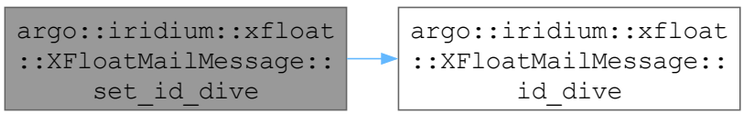
Set the dive or cycle number of this mail message.

Parameters

in	*id_dive*	The dive or cycle number of this mail message.

References id_dive(), id_dive_, and util::thread::RecursiveLockable::mutex_.

Here is the call graph for this function:



10.88.3.86 set_id_imei()

```
void argo::iridium::Iridium::set_id_imei (
            long id_imei)  [inline], [protected], [inherited]
```

Set the unique International Mobile Equipment Identifier (IMEI).

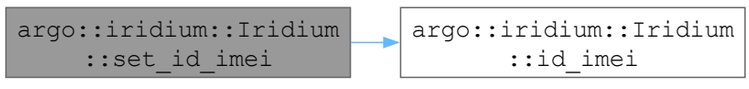
Parameters

in	*id_imei*	The unique International Mobile Equipment Identifier (IMEI).

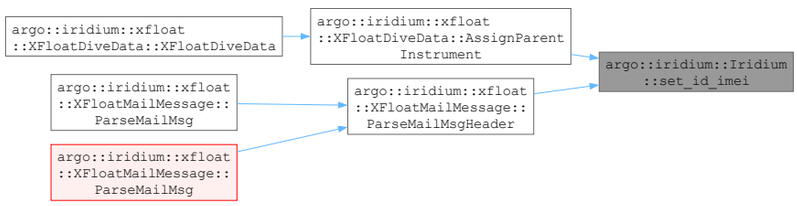
References argo::iridium::Iridium::id_imei(), argo::iridium::Iridium::id_imei_, and util::thread::RecursiveLockable::mutex_.

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignParentInstrument(), and ParseMailMsgHeader().

Here is the call graph for this function:



Here is the caller graph for this function:



10.88.3.87 set_id_instrument()

```
void argo::iridium::Iridium::set_id_instrument (
            int id_instrument)  [inline], [protected], [inherited]
```

Set the unique instrument identifier or instrument serial number.

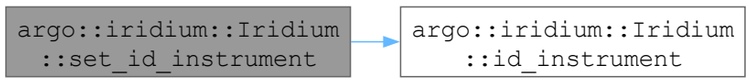
Parameters

in	*id_instrument*	The unique instrument identifier or instrument serial number

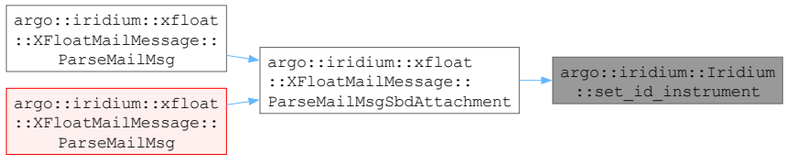
References argo::iridium::Iridium::id_instrument(), argo::iridium::Iridium::id_instrument_, and util::thread::RecursiveLockable::mutex_.

Referenced by ParseMailMsgSbdAttachment().

Here is the call graph for this function:



Here is the caller graph for this function:



10.88.3.88 set_id_msg()

```
void argo::iridium::xfloat::XFloatMailMessage::set_id_msg (
            const string & id_msg)  [inline], [private]
```

Set the mail message unique identifier string.

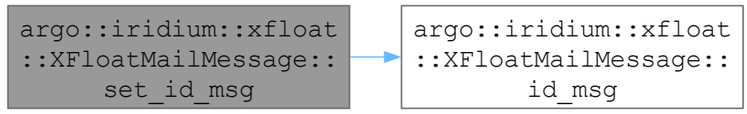
Parameters

in	*id_msg*	The mail message unique identifier string.

References id_msg(), id_msg_, and util::thread::RecursiveLockable::mutex_.

Referenced by ParseMailMsgHeader().

Here is the call graph for this function:



Here is the caller graph for this function:

10.88.3.89 set_id_msg_uid()

```
void argo::iridium::xfloat::XFloatMailMessage::set_id_msg_uid (
            long id_msg_uid)  [inline], [private]
```

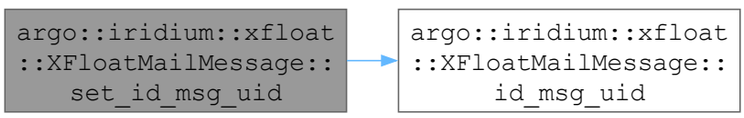
Set the mail message unique identifier.

Parameters

in	*id_msg_uid*	The mail message unique identifier.

References id_msg_uid(), id_msg_uid_, and util::thread::RecursiveLockable::mutex_.

Here is the call graph for this function:



10.88.3.90 set_id_sbd_momsn()

```
void argo::iridium::xfloat::XFloatMailMessage::set_id_sbd_momsn (
            int momsn)  [inline], [private]
```

Set the Iridium Short Burst Data (SBD) Service Mobile Originated Message Sequence Number (MOMSN).

Parameters

in	*momsn*	The Iridium Short Burst Data (SBD) Service Mobile Originated Message Sequence Number (MOMSN).

References id_sbd_momsn_, and util::thread::RecursiveLockable::mutex_.

10.88.3.91 set_id_sbd_mtmsn()

```
void argo::iridium::xfloat::XFloatMailMessage::set_id_sbd_mtmsn (
            int mtmsn)  [inline], [private]
```

Set the Iridium Short Burst Data (SBD) Service Mobile Terminated Message Sequence Number (MTMSN).

Parameters

in	*mtmsn*	The Iridium Short Burst Data (SBD) Service Mobile Terminated Message Sequence Number (MTMSN).

References id_sbd_mtmsn_, and util::thread::RecursiveLockable::mutex_.

10.88.3.92 set_id_sbd_packet()

```
void argo::iridium::xfloat::XFloatMailMessage::set_id_sbd_packet (
            int id_sbd_packet)  [inline], [private]
```

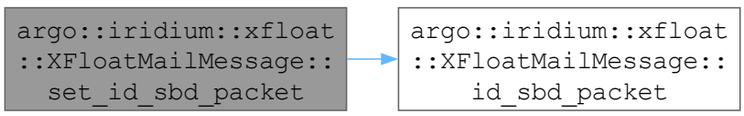
Set the Iridium Short Burst Data (SBD) message packet index.

Parameters

in	*id_sbd_packet*	Iridium Short Burst Data (SBD) message packet index.

References id_sbd_packet(), id_sbd_packet_, and util::thread::RecursiveLockable::mutex_.

Here is the call graph for this function:



10.88.3.93 set_id_wmo()

```
void argo::iridium::Iridium::set_id_wmo (
            int id_wmo)  [inline], [protected], [inherited]
```

Set the unique World Meteorological Organization (WMO) identifier.

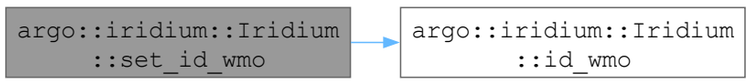
Parameters

in	*id_wmo*	The unique World Meteorological Organization (WMO) identifier.

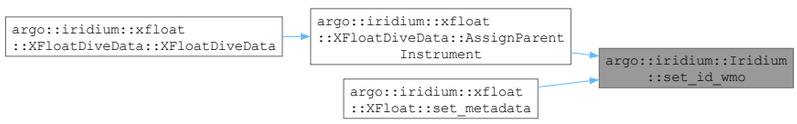
References argo::iridium::Iridium::id_wmo(), argo::iridium::Iridium::id_wmo_, and util::thread::RecursiveLockable::mutex_.

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignParentInstrument(), and argo::iridium::xfloat::XFloat::set_metadata().

Here is the call graph for this function:



Here is the caller graph for this function:



10.88.3.94 set_iridium_position_cep_radius()

```
void argo::iridium::IridiumPosition::set_iridium_position_cep_radius (
            int iridium_position_cep_radius)  [inline], [protected], [inherited]
```

Set the Iridium position Circular Error Probable (CEP) radius, in kilometers.

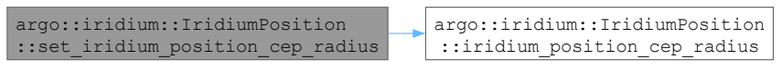
Parameters

in	*iridium_position_cep_radius*	The Iridium position Circular Error Probable (CEP) radius to set, in kilometers.

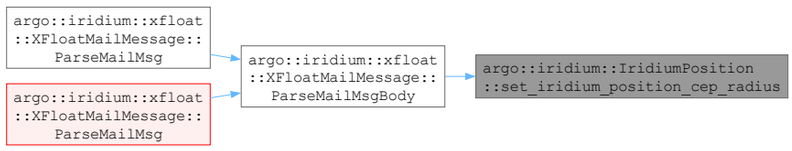
References argo::iridium::IridiumPosition::iridium_position_cep_radius(), and argo::iridium::IridiumPosition::iridium_position_cep_radius_.

Referenced by ParseMailMsgBody().

Here is the call graph for this function:



Here is the caller graph for this function:



10.88.3.95 set_iridium_position_datetime()

```
void argo::iridium::IridiumPosition::set_iridium_position_datetime (
            const ptime & iridium_position_datetime)  [inline], [protected], [inherited]
```

Set the Iridium position date/time.

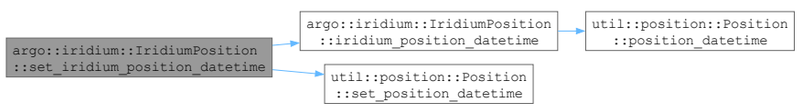
Parameters

in	*iridium_position_datetime*	The Iridium position date/time to set.

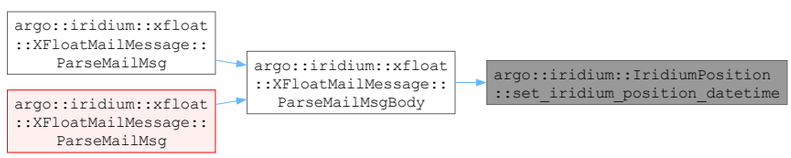
References argo::iridium::IridiumPosition::iridium_position_datetime(), and util::position::Position::set_position_datetime().

Referenced by ParseMailMsgBody().

Here is the call graph for this function:



Here is the caller graph for this function:



10.88.3.96 set_iridium_position_is_valid()

```
void argo::iridium::IridiumPosition::set_iridium_position_is_valid (
            bool iridium_position_is_valid)  [inline], [protected], [inherited]
```

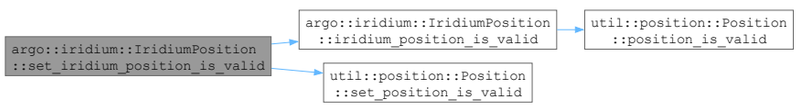
Set the Iridium position is valid flag.

Parameters

in	*iridium_position_is_valid*	The Iridium position is valid flag to set.

References argo::iridium::IridiumPosition::iridium_position_is_valid(), and util::position::Position::set_position_is_valid().

Here is the call graph for this function:



10.88.3.97 set_iridium_position_latitude()

```
void argo::iridium::IridiumPosition::set_iridium_position_latitude (
            double iridium_position_latitude)  [inline], [protected], [inherited]
```

Set the Iridium position latitude, in decimal degrees.

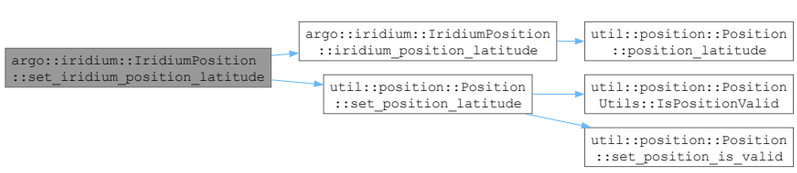
Parameters

in	*iridium_position_latitude*	The Iridium position latitude to set, in decimal degrees.

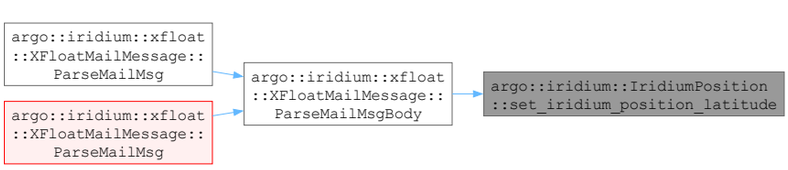
References argo::iridium::IridiumPosition::iridium_position_latitude(), and util::position::Position::set_position_latitude().

Referenced by ParseMailMsgBody().

Here is the call graph for this function:



Here is the caller graph for this function:



10.88.3.98 set_iridium_position_longitude()

```
void argo::iridium::IridiumPosition::set_iridium_position_longitude (
            double iridium_position_longitude)  [inline], [protected], [inherited]
```

Set the Iridium position longitude, in decimal degrees.

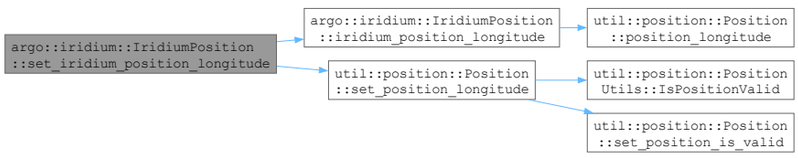
Parameters

in	*iridium_position_longitude*	The Iridium position longitude to set, in decimal degrees.

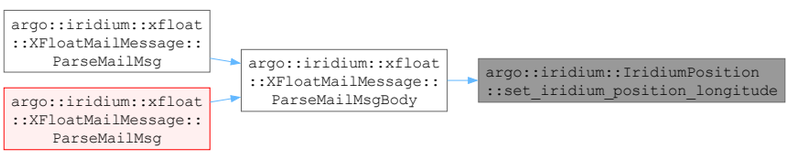
References argo::iridium::IridiumPosition::iridium_position_longitude(), and util::position::Position::set_position_longitude().

Referenced by ParseMailMsgBody().

Here is the call graph for this function:



Here is the caller graph for this function:



10.88.3.99 set_msg_body_is_valid()

```
void argo::iridium::xfloat::XFloatMailMessage::set_msg_body_is_valid (
            bool is_valid)  [inline], [private]
```

Set the message-body-is-valid flag.

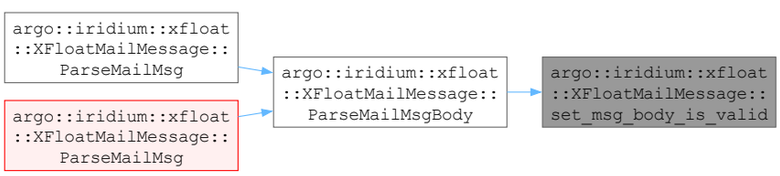
Parameters

in	*is_valid*	TRUE if the body of the Iridium message is valid, or FALSE otherwise.

References msg_body_is_valid_, and util::thread::RecursiveLockable::mutex_.

Referenced by ParseMailMsgBody().

Here is the caller graph for this function:



10.88.3.100 set_msg_body_text()

```
void argo::iridium::xfloat::XFloatMailMessage::set_msg_body_text (
            const string & msg_body_text)  [inline], [private]
```

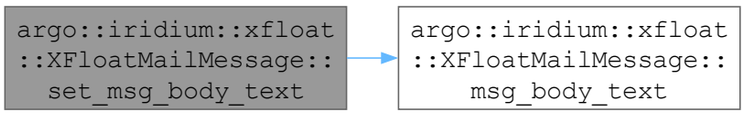
Set the mail message body text.

Parameters

in	*msg_body_text*	The mail message body text.

References msg_body_text(), msg_body_text_, and util::thread::RecursiveLockable::mutex_.

Here is the call graph for this function:



10.88.3.101 set_msg_header_datetime()

```
void argo::iridium::xfloat::XFloatMailMessage::set_msg_header_datetime (
            const ptime & datetime)  [inline], [private]
```

Set the mail message header date/time.

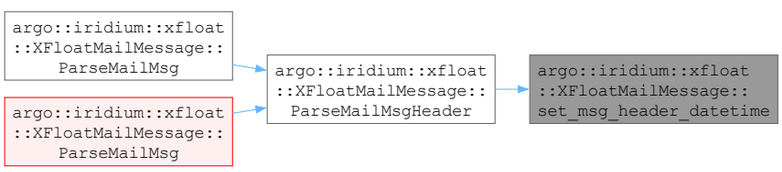
Parameters

in	*datetime*	The mail message header date/time.

References msg_header_datetime_, and util::thread::RecursiveLockable::mutex_.

Referenced by ParseMailMsgHeader().

Here is the caller graph for this function:



10.88.3.102 set_msg_header_from()

```
void argo::iridium::xfloat::XFloatMailMessage::set_msg_header_from (
            const string & from)  [inline], [private]
```

Set the mail message header FROM field value.

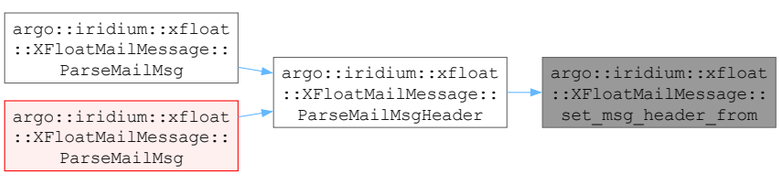
Parameters

in	*from*	The mail message header FROM field value.

References msg_header_from_, and util::thread::RecursiveLockable::mutex_.

Referenced by ParseMailMsgHeader().

Here is the caller graph for this function:



10.88.3.103 set_msg_header_is_valid()

```
void argo::iridium::xfloat::XFloatMailMessage::set_msg_header_is_valid (
            bool is_valid)  [inline], [private]
```

Set the mail-message-header-is-valid flag.

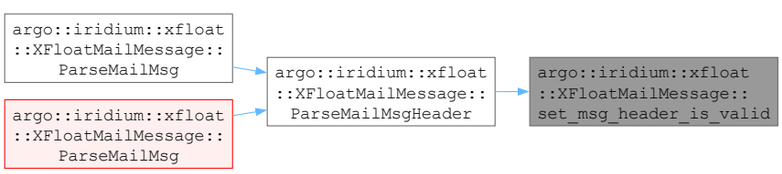
Parameters

in	*is_valid*	TRUE if the mail message header is valid, or FALSE otherwise.

References msg_header_is_valid_, and util::thread::RecursiveLockable::mutex_.

Referenced by ParseMailMsgHeader().

Here is the caller graph for this function:



10.88.3.104 set_msg_header_raw_text()

```
void argo::iridium::xfloat::XFloatMailMessage::set_msg_header_raw_text (
            const string & text)  [inline], [private]
```

Set the mail message header raw text.

Parameters

in	*text*	The mail message header raw text.

References msg_header_raw_text_, and util::thread::RecursiveLockable::mutex_.

10.88.3.105 set_msg_header_subject()

```
void argo::iridium::xfloat::XFloatMailMessage::set_msg_header_subject (
            const string & subject)  [inline], [private]
```

Set the mail message header SUBJECT field value.

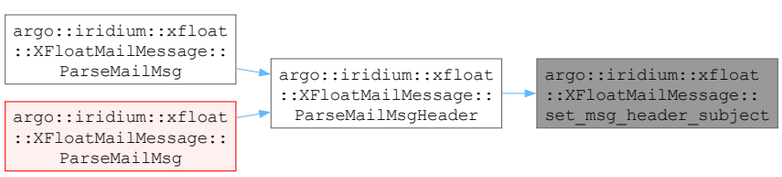
Parameters

in	*subject*	The mail message header SUBJECT field value.

References msg_header_subject_, and util::thread::RecursiveLockable::mutex_.

Referenced by ParseMailMsgHeader().

Here is the caller graph for this function:



10.88.3.106 set_msg_is_valid()

```
void argo::iridium::xfloat::XFloatMailMessage::set_msg_is_valid (
            bool is_valid)  [inline], [private]
```

Set the mail-message-is-valid flag.

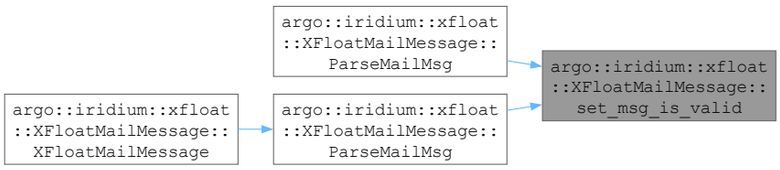
Parameters

in	*is_valid*	TRUE if the mail message is valid, or FALSE otherwise.

References msg_is_valid_, and util::thread::RecursiveLockable::mutex_.

Referenced by ParseMailMsg(), and ParseMailMsg().

Here is the caller graph for this function:



10.88.3.107 set_msg_raw_text()

```
void argo::iridium::xfloat::XFloatMailMessage::set_msg_raw_text (
            const string & text)  [inline], [private]
```

Set the full mail message raw text.

Parameters

in	*text*	The full mail message raw text.

References msg_raw_text_, and util::thread::RecursiveLockable::mutex_.

10.88.3.108 set_position_datetime()

```
void util::position::Position::set_position_datetime (
            const ptime & datetime)  [inline], [protected], [inherited]
```

Set the date/time of the position.

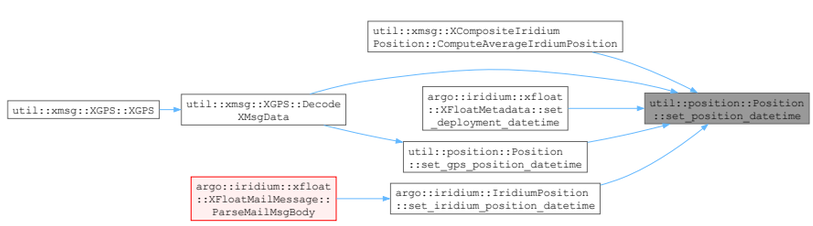
Parameters

in	*datetime*	The date/time of the position to set.

References util::position::Position::position_datetime_.

Referenced by util::xmsg::XCompositeIridiumPosition::ComputeAverageIrdiumPosition(), util::xmsg::XGPS::DecodeXMsgData(), argo::iridium::xfloat::XFloatMetadata::set_deployment_datetime(), util::position::Position::set_gps_position_datetime(), and argo::iridium::IridiumPosition::set_iridium_position_datetime().

Here is the caller graph for this function:



10.88.3.109 set_position_is_valid()

```
void util::position::Position::set_position_is_valid (
            bool is_valid = true)  [inline], [protected], [inherited]
```

Set the position is valid flag.

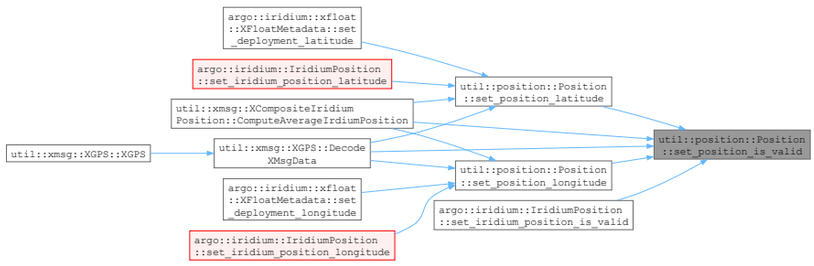
Parameters

in	*is_valid*	TRUE if the position is valid, or FALSE otherwise.

References util::position::Position::position_is_valid_.

Referenced by util::xmsg::XCompositeIridiumPosition::ComputeAverageIrdiumPosition(), util::xmsg::XGPS::DecodeXMsgData(), argo::iridium::IridiumPosition::set_iridium_position_is_valid(), util::position::Position::set_position_latitude(), and util::position::Position::set_position_longitude().

Here is the caller graph for this function:



10.88.3.110 set_position_latitude()

```
void util::position::Position::set_position_latitude (
            double lat)  [inline], [protected], [inherited]
```

Set the latitude of the position, in decimal degrees.

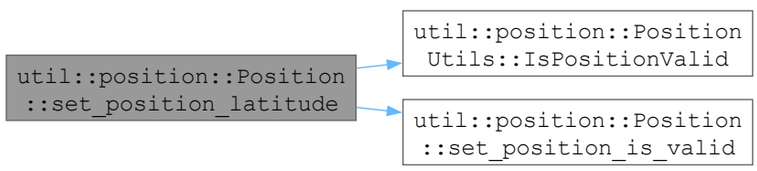
Parameters

in	*lat*	The latitude of the position, in decimal degrees.

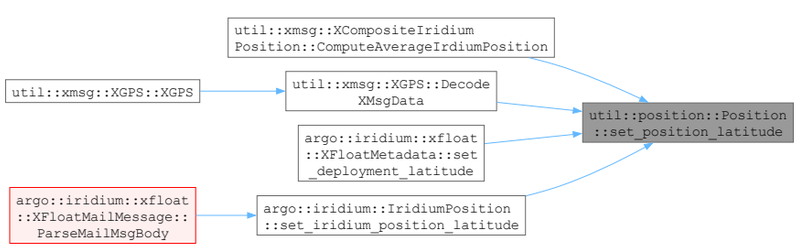
References util::position::PositionUtils::IsPositionValid(), util::position::Position::position_latitude_, util::position::Position::position_longitud and util::position::Position::set_position_is_valid().

Referenced by util::xmsg::XCompositeIridiumPosition::ComputeAverageIrdiumPosition(), util::xmsg::XGPS::DecodeXMsgData(), argo::iridium::xfloat::XFloatMetadata::set_deployment_latitude(), and argo::iridium::IridiumPosition::set_iridium_position_latitude().

Here is the call graph for this function:



Here is the caller graph for this function:



10.88.3.111 set_position_longitude()

```
void util::position::Position::set_position_longitude (
            double lon)  [inline], [protected], [inherited]
```

Set the longitude of the position, in decimal degrees.

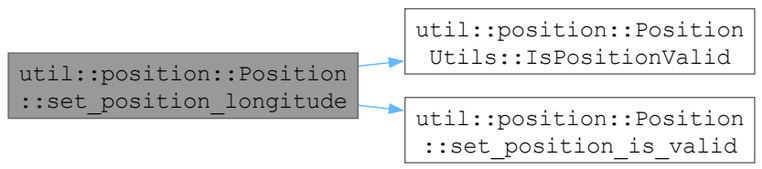
Parameters

in	*lon*	The longitude of the position, in decimal degrees.

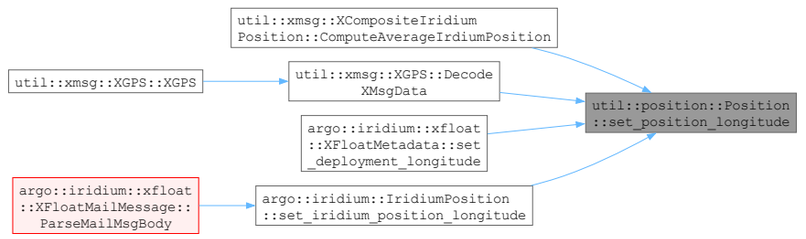
References util::position::PositionUtils::IsPositionValid(), util::position::Position::position_latitude_, util::position::Position::position_longitud and util::position::Position::set_position_is_valid().

Referenced by util::xmsg::XCompositeIridiumPosition::ComputeAverageIrdiumPosition(), util::xmsg::XGPS::DecodeXMsgData(), argo::iridium::xfloat::XFloatMetadata::set_deployment_longitude(), and argo::iridium::IridiumPosition::set_iridium_position_longitude().

Here is the call graph for this function:



Here is the caller graph for this function:



10.88.3.112 set_raw_file_local_path()

```
void argo::iridium::xfloat::XFloatMailMessage::set_raw_file_local_path (
            const path & path)  [inline], [private]
```

Set the path to the raw mail message file on the local file system.

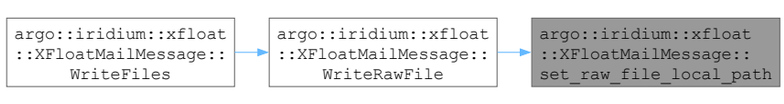
Parameters

in	*path*	The path to the raw mail message file on the local file system.

References util::thread::RecursiveLockable::mutex_, and raw_file_local_path_.

Referenced by WriteRawFile().

Here is the caller graph for this function:



10.88.3.113 set_sbd_data()

```
void argo::iridium::xfloat::XFloatMailMessage::set_sbd_data (
            const vector< uint8_t > & sbd_data)  [inline], [private]
```

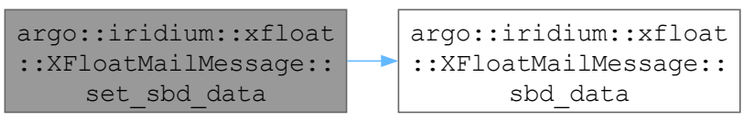
Set the Iridium Short Burst Data (SBD) for this mail message.

Parameters

in	*sbd_data*	The Iridium Short Burst Data (SBD) for this mail message.

References util::thread::RecursiveLockable::mutex_, sbd_data(), and sbd_data_.

Here is the call graph for this function:



10.88.3.114 set_sbd_data_is_valid()

```
void argo::iridium::xfloat::XFloatMailMessage::set_sbd_data_is_valid (
            bool is_valid)  [inline], [private]
```

Set the SBD-message-is-valid flag.

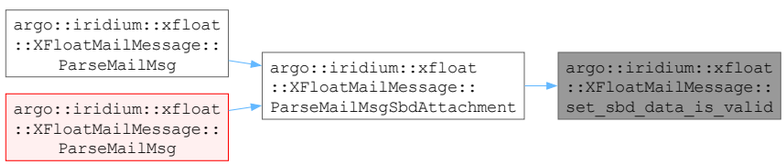
Parameters

in	*is_valid*	TRUE if the Iridium Short Burst Data (SBD) message is valid, or FALSE otherwise.

References util::thread::RecursiveLockable::mutex_, and sbd_data_is_valid_.

Referenced by ParseMailMsgSbdAttachment().

Here is the caller graph for this function:



10.88.3.115 set_sbd_file_local_path()

```
void argo::iridium::xfloat::XFloatMailMessage::set_sbd_file_local_path (
            const path & path)  [inline], [private]
```

Set the path to the Iridium Short Burst Data (SBD) message file on the local file system.

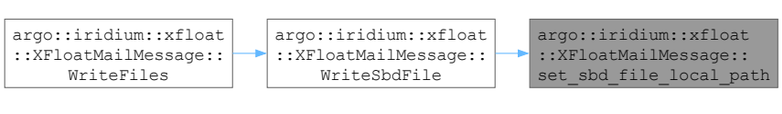
Parameters

in	*path*	The path to the Iridium Short Burst Data (SBD) message file on the local file system.

References util::thread::RecursiveLockable::mutex_, and sbd_file_local_path_.

Referenced by WriteSbdFile().

Here is the caller graph for this function:



10.88.3.116 set_sbd_file_name()

```
void argo::iridium::xfloat::XFloatMailMessage::set_sbd_file_name (
            const string & name)  [inline], [private]
```

Set the name of the Iridium Short Burst Data (SBD) file.

Parameters

in	*name*	The name of the Iridium Short Burst Data (SBD) file.

References util::thread::RecursiveLockable::mutex_, and sbd_file_name_.

10.88.3.117 set_sbd_file_size_bytes()

```
void argo::iridium::xfloat::XFloatMailMessage::set_sbd_file_size_bytes (
            int size)  [inline], [private]
```

Set the size of the Iridium Short Burst Data (SBD) message, in bytes.

Parameters

in	*size*	The size of the Iridium Short Burst Data (SBD) message, in bytes.

References util::thread::RecursiveLockable::mutex_, and sbd_file_size_bytes_.

10.88.3.118 set_sbd_session_status_code()

```
void argo::iridium::xfloat::XFloatMailMessage::set_sbd_session_status_code (
            int code)  [inline], [private]
```

Set the Iridium Short Burst Data (SBD) session status code.

Parameters

in	*code*	The Iridium Short Burst Data (SBD) session status code.

References util::thread::RecursiveLockable::mutex_, and sbd_session_status_code_.

10.88.3.119 set_sbd_session_status_string()

```
void argo::iridium::xfloat::XFloatMailMessage::set_sbd_session_status_string (
            const string & str)  [inline], [private]
```

Set the Iridium Short Burst Data (SBD) session status string.

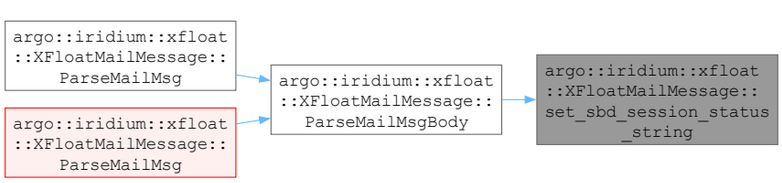
Parameters

in	*str*	The Iridium Short Burst Data (SBD) session status string.

References util::thread::RecursiveLockable::mutex_, and sbd_session_status_string_.

Referenced by ParseMailMsgBody().

Here is the caller graph for this function:



10.88.3.120 UnlockObject()

```
void util::thread::RecursiveLockable::UnlockObject ()  [inline], [inherited]
```

Unlock the object.

Exceptions

util::log::MessageException	if an error occurs.

References CATCH_BLOCK, and util::thread::RecursiveLockable::mutex_.

10.88.3.121 WriteFiles()

```
void argo::iridium::xfloat::XFloatMailMessage::WriteFiles ()  [inline], [private]
```

Write the mail message files.

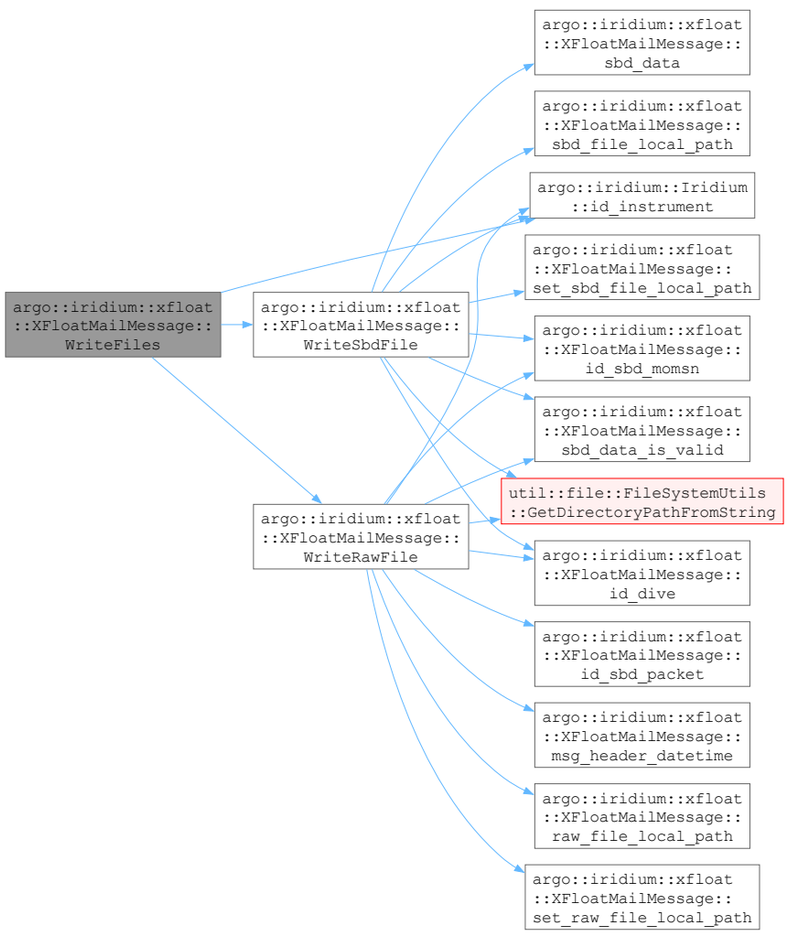
This function is used to write the RAW and SBD files to the local filesystem. First, the function checks if the unique instrument identifier is known. If the unique instrument identifier is unknown, the function will throw an exception. Otherwise, the function will continue on to write the RAW mail message file and the binary SBD message file to the local filesystem, if their respective criteria are met.

Exceptions

util::log::MessageException	if the unique instrument identifier is unknown, or if an error occurs while writing the mail files to the local filesystem,

References CATCH_BLOCK, EXC_THROW, argo::iridium::Iridium::id_instrument(), WriteRawFile(), WriteSbdFile(), and XFLT_ID_UNKNOWN.

Here is the call graph for this function:



10.88.3.122 WriteRawFile()

```
void argo::iridium::xfloat::XFloatMailMessage::WriteRawFile ()  [private]
```

Write the RAW mail message file.

This function is used to write the raw mail message file to the local filesystem. If the global RAW-file-generation-is-enabled flag, raw_file_generation_is_enabled_, is FALSE, then this function will have no effect. Otherwise, the function will generate the path to the RAW mail file on the local filesystem and write the file to that path. If the RAW mail message file already exists, then it is left unchanged and is not rewritten.

The path to the RAW mail message file will be constructed in the format [RAW-DIR]/[WWWW]/raw[WWWW]-[MMMMM]-[DDDD]-[PPP].txt, where

- [RAW-DIR] is the RAW mail directory path specified, by raw_directory_path_,
- [WWWW] is the four-digit unique instrument serial number,
- [MMMMM] is the five-digit MOMSN number,
- [DDDD] is the four-digit dive or cycle number, and
- [PPP] is the three-digit SBD packet number.

If the mail message does not contain valid SBD data, then the path is [RAW-DIR]/[WWWW]/raw[WWWW]-[MMMMM]-incomplete-[DATETIME].txt, where [DATETIME] is the mail header date/time in ISO 8601 basic format.

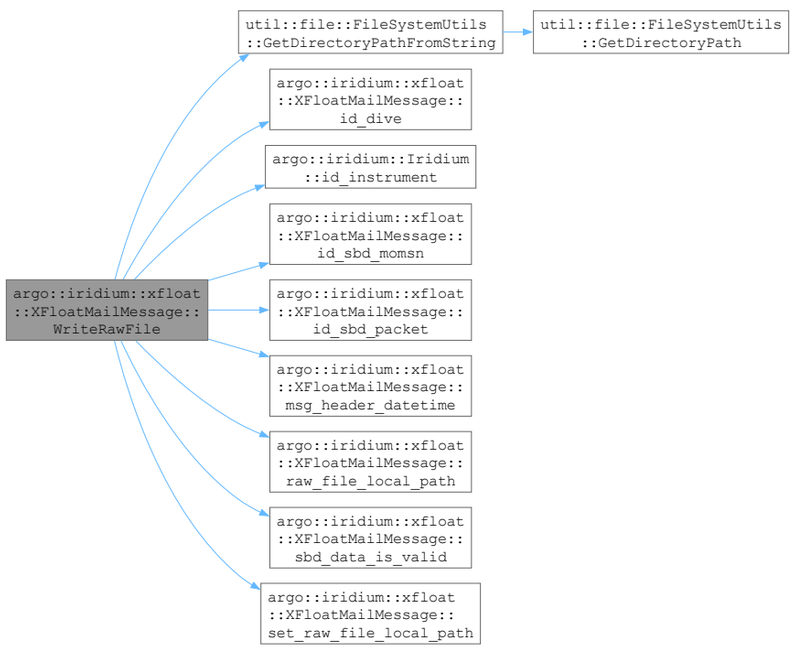
Exceptions

log::util::MessageException	if an error occurs while writing the file.

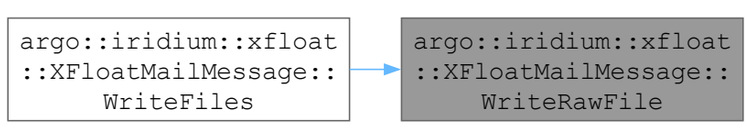
References CATCH_BLOCK, util::file::FileSystemUtils::GetDirectoryPathFromString(), id_dive(), argo::iridium::Iridium::id_instrument(), id_sbd_momsn(), id_sbd_packet(), linf, msg_header_datetime(), raw_directory_path_, raw_file_generation_is_enabled_, raw_file_local_path(), sbd_data_is_valid(), and set_raw_file_local_path().

Referenced by WriteFiles().

Here is the call graph for this function:



Here is the caller graph for this function:



10.88.3.123 WriteSbdFile()

```
void argo::iridium::xfloat::XFloatMailMessage::WriteSbdFile ()  [inline], [private]
```

Write the binary Iridium Short Burst Data (SBD) file.

This function is used to write the binary SBD file to the local filesystem. If either the global SBD-file-generation-is-enabled flag, sbd_file_generation_is_enabled_, or the SBD-data-is-valid field, sbd_data_is_valid_, is FALSE, then this function will have no effect.

Otherwise, if the SBD data is valid and global SBD-file-generation-is-enabled flag is TRUE, then the SBD file path will be generated. If the SBD file already exists at the constructed path, then it will not be overwritten. If the file does not already exist, it will be written.

The path to the binary SBD message file will be constructed in the format [SBD-DIR]/[WWWW]/[WWWW]-[MMMMMM]-[DDDD].sbd, where

- [SBD-DIR] is the SBD mail directory path specified, by sbd_directory_path_,
- [WWWW] is the four-digit unique instrument serial number,
- [MMMMMM] is the six-digit MOMSN number, and
- [DDDD] is the four-digit dive or cycle number.

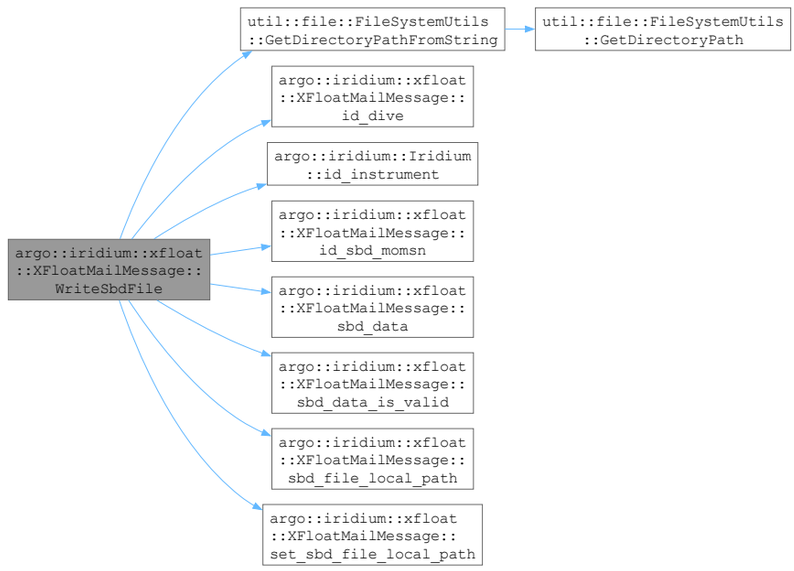
Exceptions

util::log::MessageException	if an error occurs while writing the binary SBD file to the local filesystem.

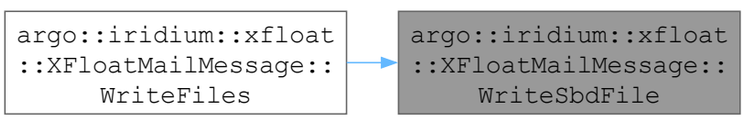
References CATCH_BLOCK, util::file::FileSystemUtils::GetDirectoryPathFromString(), id_dive(), argo::iridium::Iridium::id_instrument(), id_sbd_momsn(), linf, util::thread::RecursiveLockable::mutex_, sbd_data(), sbd_data_is_valid(), sbd_directory_path_, sbd_file_generation_is_enabled_, sbd_file_local_path(), and set_sbd_file_local_path().

Referenced by WriteFiles().

Here is the call graph for this function:



Here is the caller graph for this function:



10.88.4 Friends And Related Symbol Documentation

10.88.4.1 operator<<

```
ostream & operator<< (
            ostream & os,
            const XFloatMailMessage & mail)  [friend]
```

The stream insertion operator overload function for XFloatMailMessage.

Generates and inserts a RAW mail string representation of the mail message.

Exceptions

util::log::MessageException	of an error occurs.

Parameters

in	*os*	The ostream object.
in	*mail*	The mail message object.

Returns

The ostream object, os.

10.88.4.2 operator>>

```
istream & operator>> (
            istream & is,
            XFloatMailMessage & mail)  [friend]
```

The stream extraction operator overload function for XFloatMailMessage.

Parses mail message from istream.

Exceptions

util::log::MessageException	of an error occurs.

Parameters

in	*is*	The istream object.
in	*mail*	The mail message object.

Returns

The istream object, is.

10.88.4.3 util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription

```
options_description util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription
() [friend]
```

Friend function for populating some of the static members of this class from the configuration file options.

10.88.5 Member Data Documentation

10.88.5.1 address_

```
std::string util::mysql::LocalMysqlConfiguration::address_  [static], [private], [inherited]
```

The SQL server qualified name or IP address string.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.88.5.2 EARTH_RADIUS_KM_EQUATORIAL

```
const double util::position::PositionUtils::EARTH_RADIUS_KM_EQUATORIAL = 6378.1  [static], [private],
[inherited]
```

The equatorial radius of the earth, in kilometers.

Referenced by util::position::PositionUtils::CalculateEllipsoidHeight().

10.88.5.3 EARTH_RADIUS_KM_POLAR

```
const double util::position::PositionUtils::EARTH_RADIUS_KM_POLAR = 6356.1  [static], [private],
[inherited]
```

The polar radius of the earth, in kilometers.

Referenced by util::position::PositionUtils::CalculateEllipsoidHeight().

10.88.5.4 id_dac_

```
int argo::iridium::Iridium::id_dac_  [private], [inherited]
```

The unique Data Assembly Center (DAC) identifier.

Referenced by argo::iridium::Iridium::id_dac(), and argo::iridium::Iridium::set_id_dac().

10.88.5.5 id_dive_

```
int argo::iridium::xfloat::XFloatMailMessage::id_dive_  [private]
```

The dive or cycle number of this mail message object.

This field is provides a non-unique index of the dive or cycle number of the mail message object for a given instrument. It is used to determine to which dive this mail message belongs. Together, the dive or cycle number and the Iridium Short Burst Data (SBD) packet index, id_sbd_packet_, provide a unique index of the mail message. However, these fields are not used as the primary index of the mail message objects.

The value of this field is set as a result of parsing the mail message attachment. The value is read from the header of the attached SBD file. If the mail message has no SBD attachment, or the attachment is invalid, then the value of this field will be set to XFLT_DIVE_UNKNOWN, which specifies an unknown dive or cycle number.

Referenced by id_dive(), ParseMailMsgSbdAttachment(), and set_id_dive().

10.88.5.6 id_imei_

```
long argo::iridium::Iridium::id_imei_  [private], [inherited]
```

The unique International Mobile Equipment Identifier (IMEI).

Referenced by argo::iridium::Iridium::id_imei(), and argo::iridium::Iridium::set_id_imei().

10.88.5.7 id_instrument_

```
int argo::iridium::Iridium::id_instrument_  [private], [inherited]
```

The unique instrument identifier or instrument serial number.

Referenced by argo::iridium::Iridium::id_instrument(), and argo::iridium::Iridium::set_id_instrument().

10.88.5.8 id_msg_

```
string argo::iridium::xfloat::XFloatMailMessage::id_msg_  [private]
```

The mail message unique identifier string.

This character sequence is a unique identifier for the mail message that is assigned by the mail server. However, this value is not used a primary identifier because the mail server all provides a unique integer identifier.

The value of this field is set as a result of parsing the mail message header. All mail messages will be assigned a unique identifier string, regardless of their validity.

Referenced by id_msg(), and set_id_msg().

10.88.5.9 id_msg_uid_

```
long argo::iridium::xfloat::XFloatMailMessage::id_msg_uid_  [private]
```

The mail message unique identifier (UID).

This unique integer identifier is assigned by the remote mail server. It is used as the primary unique identifier for a mail message.

The mail messages' UIDs also specify the order in which the mail messages were received by the mail server. Each message that the mail server receives is assigned a monotonically increasing UID value. Thus this field is also used to determine which mail messages have already been fetch from the mail server. When the mail retrieval thread, XFloatMailRetrievalService, queries the maximum UID of mail messages that have been persisted to the SQL database. The result of the query plus one determines the minimum UID of the mail messages to fetch from the mail server.

The value of this field is set as a result of parsing the mail message header. All mail messages will be assigned a unique integer identifier, regardless of their validity.

Referenced by id_msg_uid(), ParseMailMsg(), and set_id_msg_uid().

10.88.5.10 id_sbd_momsn_

```
int argo::iridium::xfloat::XFloatMailMessage::id_sbd_momsn_  [private]
```

The Iridium Short Burst Data (SBD) Service Mobile Originated Message Sequence Number (MOMSN) identifier.

This identifier, provided by the Iridium system, uniquely identifies the sequence number of messages successfully sent from the instrument to the ground station. For each new message that is transmitted from the instrument, this number is incremented. It is provided in the body of the email from the Iridium system along with information about the position, etc.

The value of this field is set as a result of parsing the mail message body text. If the mail message body is invalid, then the value of this field will be set to XFLT_ID_UNKNOWN.

Warning

> This identifier should NOT be used to unique identify mail messages for a given instrument. If the instrument experiences a incomplete or failed Iridium transmission that results in an email from the Iridium system, then MOMSN will NOT be incremented for that message. Thus, there may exist multiple mail messages with the same MOMSN identifier.

Referenced by id_sbd_momsn(), ParseMailMsgBody(), and set_id_sbd_momsn().

10.88.5.11 id_sbd_mtmsn_

```
int argo::iridium::xfloat::XFloatMailMessage::id_sbd_mtmsn_  [private]
```

The Iridium Short Burst Data (SBD) Service Mobile Terminated Message Sequence Number (MTMSN) identifier.

This identifier, provided by the Iridium system, uniquely identifies the sequence number of messages successfully sent from the ground station to the instrument. For each new message that is transmitted to the instrument from the ground station, this number is incremented. It is provided in the body of the email from the Iridium system along with information about the position, etc.

The value of this field is set as a result of parsing the mail message body text. If the mail message body is invalid, then the value of this field will be set to XFLT_ID_UNKNOWN.

Referenced by id_sbd_mtmsn(), ParseMailMsgBody(), and set_id_sbd_mtmsn().

10.88.5.12 id_sbd_packet_

```
int argo::iridium::xfloat::XFloatMailMessage::id_sbd_packet_  [private]
```

The Iridium Short Burst Data (SBD) message packet index.

This identifier uniquely indexes each SBD message transmitted for a single dive. It is used both to reconstruct the sequence in which the message were transmitted and to determine whether any of the message in the sequence are missing. A complete set of mail messages for a given dive will have a sequence of monotonically increasing SBD packet identifiers starting with a value of zero.

The value of this field is set as a result of parsing the mail message attachment. The value is read from the header of the attached SBD file. If the mail message has no SBD attachment, or the attachment is invalid, then the value of this field will be set to XFLT_DIVE_UNKNOWN, which specifies an unknown SBD message packet index.

Referenced by id_sbd_packet(), ParseMailMsgSbdAttachment(), and set_id_sbd_packet().

10.88.5.13 id_wmo_

```
int argo::iridium::Iridium::id_wmo_  [private], [inherited]
```

The unique World Meteorological Organization (WMO) identifier.

Referenced by argo::iridium::Iridium::id_wmo(), and argo::iridium::Iridium::set_id_wmo().

10.88.5.14 iridium_position_cep_radius_

```
int argo::iridium::IridiumPosition::iridium_position_cep_radius_  [private], [inherited]
```

Iridium position Circular Error Probable (CEP) radius, in kilometers.

This field defines the estimate of an error of the Iridium position. It is provided by the Iridium system, along with the Iridium position, in the body of the email containing the Iridium SBD message.

Since multiple Iridium messages are transmitted for each dive cycle, a better estimate of the instrument position is provided by a composite position, constructed from two or more individual positions. To compute this composite position, each position is weighted proportionally to the inverse of the CEP radius.

Referenced by argo::iridium::IridiumPosition::iridium_position_cep_radius(), and argo::iridium::IridiumPosition::set_iridium_position_cep_r

10.88.5.15 msg_body_is_valid_

```
bool argo::iridium::xfloat::XFloatMailMessage::msg_body_is_valid_  [private]
```

The mail message body is valid flag.

This field specifies the validity of the mail message body. It is set as a result of parsing the mail message body text, which contains information about the Iridium session including the Iridium position, date/time, status, and message identifiers.

Referenced by msg_body_is_valid(), and set_msg_body_is_valid().

10.88.5.16 msg_body_text_

```
string argo::iridium::xfloat::XFloatMailMessage::msg_body_text_  [private]
```

The mail message body text.

This field contains the full text of the body of the mail message. The body of a valid mail message includes information about the Iridium session that is parsed and stored in the corresponding field of this object. These field include

- the MOMSN identifier,
- the MTMSN identifier,
- the time of the Iridium session,
- the Iridium session status code and status string, and
- the Iridium position and Circular Error Probable (CEP) radius.

Referenced by msg_body_text(), ParseMailMsgBody(), and set_msg_body_text().

10.88.5.17 msg_header_datetime_

```
ptime argo::iridium::xfloat::XFloatMailMessage::msg_header_datetime_  [private]
```

The mail message header date/time.

This field represents the date/time-of-flight of the mail message from the Iridium system. The value of this field, is set as a result of parsing the mail message header.

Note

This date/time, along with the Iridium session time may be used to calculate the latency of the Iridium system.

Referenced by msg_header_datetime(), and set_msg_header_datetime().

10.88.5.18 msg_header_from_

```
string argo::iridium::xfloat::XFloatMailMessage::msg_header_from_  [private]
```

The mail message header FROM field.

This field represents the sender of the mail message. This field is used to validate the mail message by ensuring the only messages from desired senders are consumed by the application. The value of this field is set as a result of parsing the mail message header.

Referenced by msg_header_from(), and set_msg_header_from().

10.88.5.19 msg_header_is_valid_

```
bool argo::iridium::xfloat::XFloatMailMessage::msg_header_is_valid_  [private]
```

The mail message header is valid flag.

This field indicates the validity of the mail message header. It is set as a result of parsing the mail message header. The criteria for a valid mail message header is that

- the SUBJECT field of the mail message header matches the valid sender regular expression, msg_header_valid_sender_regex_, and
- the FROM field of the mail message header matcher the valid subject regular expression, msg_header_valid_subject_regex_.

Referenced by msg_header_is_valid(), and set_msg_header_is_valid().

10.88.5.20 msg_header_raw_text_

```
string argo::iridium::xfloat::XFloatMailMessage::msg_header_raw_text_  [private]
```

The mail message header raw text.

This field contains the "raw" text of the mail message header. This text represents the header of the mail message in its native form, which includes additional information about routing, etc. that is not currently parsed from the mail message header.

Referenced by msg_header_raw_text(), ParseMailMsgHeader(), and set_msg_header_raw_text().

10.88.5.21 msg_header_subject_

```
string argo::iridium::xfloat::XFloatMailMessage::msg_header_subject_  [private]
```

The mail message header SUBJECT field.

This field contains the text of the subject line of the mail message. It is used to validate the mail message by ensuring that only messages with the proper subject line are consumed by the application. In addition, a valid mail message contains the IMEI number in the subject line. The subject line is parsed to determine the IMEI of the instrument that transmitted the message.

Referenced by msg_header_subject(), and set_msg_header_subject().

10.88.5.22 msg_header_valid_sender_regex_

```
string argo::iridium::xfloat::XFloatMailMessage::msg_header_valid_sender_regex_  [static], [private]
```

The mail message header valid sender regular expression.

This regular expression is used the validate the FROM field of the email message header. If the mail message header FROM field does not match this regular expression, the the mail message will be considered invalid and will be discarded.

The value of this field is set as a result of parsing the command-line arguments and/or configuration file options during application initialization. If the value is unspecified, then it defaults to a value of XMAIL_DFLT_MSG_HEADER_VALID_SNDR_REGEX.

Referenced by ParseMailMsgHeader().

10.88.5.23 msg_header_valid_subject_regex_

```
string argo::iridium::xfloat::XFloatMailMessage::msg_header_valid_subject_regex_  [static], [private]
```

The mail message header valid subject regular expression.

This regular expression is used to validate the SUBJECT field of the email message header. If the mail message header SUBJECT field does not match this regular expression, the the mail message will be considered invalid and will be discarded.

The value of this field is set as a result of parsing the command-line arguments and/or configuration file options during application initialization. If the value is unspecified, then it defaults to a value of XMAIL_DFLT_MSG_HEADER_VALID_SUBJ_REGEX.

Referenced by ParseMailMsgHeader().

10.88.5.24 msg_is_valid_

```
bool argo::iridium::xfloat::XFloatMailMessage::msg_is_valid_  [private]
```

The mail message is valid flag.

This flag indicates the validity of the mail message. It is set as a result of parsing the mail message. The criteria for a valid mail message is that

- the mail message header is valid (see msg_header_is_valid_),
- the mail message body is valid (see msg_body_is_valid_), and
- the mail message SBD file attachment is valid (see sbd_data_is_valid_).

Referenced by msg_is_valid(), and set_msg_is_valid().

10.88.5.25 msg_raw_text_

```
string argo::iridium::xfloat::XFloatMailMessage::msg_raw_text_  [private]
```

The full mail message raw text.

This field contains the full mail message text in its raw form. This represents the native ASCII format of the mail message.

Referenced by msg_raw_text(), ParseMailMsg(), ParseMailMsg(), and set_msg_raw_text().

10.88.5.26 mutex_

```
recursive_mutex util::thread::RecursiveLockable::mutex_  [mutable], [protected], [inherited]
```

Recursive mutual exclusion lock for synchronizing operations.

Referenced by util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AggregateS
argo::iridium::xfloat::XFloat::argo_instrument_type(), argo::iridium::xfloat::XFloatDiveData::argo_instrument_type(),
argo::iridium::xfloat::XFloatDiveData::AssignArgoData(), argo::iridium::xfloat::XFloatDiveData::AssignAuxTemperatureBinned(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignAuxTemperatureMe
argo::iridium::xfloat::XFloatDiveData::AssignDriftPressure(), argo::iridium::xfloat::XFloatDiveData::AssignDriftSalinity(),
argo::iridium::xfloat::XFloatDiveData::AssignDriftTemperatre(), argo::iridium::xfloat::XFloatDiveData::AssignEngineeringAbort(),
argo::iridium::xfloat::XFloatDiveData::AssignEngineeringAscent(), argo::iridium::xfloat::XFloatDiveData::AssignEngineeringBitFail(),
argo::iridium::xfloat::XFloatDiveData::AssignEngineeringBitPass(), argo::iridium::xfloat::XFloatDiveData::AssignEngineeringDescent(),
argo::iridium::xfloat::XFloatDiveData::AssignEngineeringDiagnostic(), argo::iridium::xfloat::XFloatDiveData::AssignEngineeringDrift(),
argo::iridium::xfloat::XFloatDiveData::AssignEngineeringIce(), argo::iridium::xfloat::XFloatDiveData::AssignEngineeringStandard(),
argo::iridium::xfloat::XFloatDiveData::AssignEngineeringSurface(), argo::iridium::xfloat::XFloatDiveData::AssignFallTimeSeries(),
argo::iridium::xfloat::XFloatDiveData::AssignGps(), argo::iridium::xfloat::XFloatDiveData::AssignMailMessage(),
argo::iridium::xfloat::XFloatDiveData::AssignMissionCmdEchoback(), argo::iridium::xfloat::XFloatDiveData::AssignParentInstrument(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignPressureMessage(
argo::iridium::xfloat::XFloatDiveData::AssignProfilePressureBinned(), argo::iridium::xfloat::XFloatDiveData::AssignProfilePressureUnbinn
argo::iridium::xfloat::XFloatDiveData::AssignProfileSalinityBinned(), argo::iridium::xfloat::XFloatDiveData::AssignProfileSalinityUnbinned()
argo::iridium::xfloat::XFloatDiveData::AssignProfileTemperatureBinned(), argo::iridium::xfloat::XFloatDiveData::AssignProfileTemperatureU
argo::iridium::xfloat::XFloatDiveData::AssignPumpTimeSeries(), argo::iridium::xfloat::XFloatDiveData::AssignRiseTimeSeries(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignSalinityMessage(),
argo::iridium::xfloat::XFloatDiveData::AssignSurfaceTimeSeries(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_C
argo::iridium::xfloat::XFloatDiveData::AssignValveData(), argo::iridium::xfloat::XFloatDiveData::CheckCompleteness(),
argo::iridium::xfloat::XFloatMetadata::comment(), argo::iridium::xfloat::XFloatDiveData::ComputePressureOffsetFromSurfaceTimeSeries(
argo::iridium::xfloat::XFloatMetadata::deployer(), argo::iridium::xfloat::XFloatMetadata::deployer_address(), argo::iridium::xfloat::XFloatMe
argo::iridium::xfloat::XFloatMetadata::deployment_datetime(), argo::iridium::xfloat::XFloatMetadata::deployment_datetime_status(),
argo::iridium::xfloat::XFloatMetadata::deployment_latitude(), argo::iridium::xfloat::XFloatMetadata::deployment_longitude(),
argo::iridium::xfloat::XFloatMetadata::deployment_platform(), argo::iridium::xfloat::XFloatMetadata::deployment_position_status(),
argo::iridium::xfloat::XFloatMetadata::deployment_type(), argo::iridium::xfloat::XFloat::fw_version(), argo::iridium::xfloat::XFloat::gain_pres
argo::iridium::xfloat::XFloat::gain_pressure_surface(), argo::iridium::xfloat::XFloat::gain_salinity(), argo::iridium::xfloat::XFloat::gain_salinit
argo::iridium::xfloat::XFloat::gain_temperature(), argo::iridium::xfloat::XFloat::gain_temperature_surface(), argo::iridium::xfloat::XFloatDive
argo::iridium::xfloat::XFloatDiveData::GenerateCompositeIridiumPosition(), argo::iridium::xfloat::XFloatDiveData::GenerateCompositePun
argo::iridium::xfloat::XFloatDiveData::GenerateCompositeRiseTimeSeriesData(), argo::iridium::xfloat::XFloatDiveData::GenerateListingSt
util::xmsg::XCompositeDrift::GenerateListingString(), util::xmsg::XCompositeHiResProfile::GenerateListingString(),
util::xmsg::XCompositeProfile::GenerateListingString(), argo::iridium::xfloat::XFloatDiveData::GenerateOrderedMomsnList(),
argo::iridium::xfloat::XFloatDiveData::GeneratePhyString(), util::xmsg::XCompositeDrift::GeneratePhyString(), util::xmsg::XCompositeHiR
util::xmsg::XCompositeProfile::GeneratePhyString(), argo::iridium::xfloat::XFloatDiveData::GetDiveDataMailMessagesFromDatabase(),
argo::iridium::xfloat::XFloatDiveData::GetGainsAndOffsets(), argo::iridium::xfloat::XFloatDiveData::GetGpsPositionEndOfDive(),
argo::iridium::xfloat::XFloatDiveData::GetSurfaceGainsAndOffsets(), argo::iridium::xfloat::XFloatDiveData::HasBinnedProfileData(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::HasContiguousMessageI
argo::iridium::xfloat::XFloatDiveData::HasContiguousMomsns(), argo::iridium::xfloat::XFloatDiveData::HasContiguousPids(),
argo::iridium::xfloat::XFloatDiveData::HasDiveData(), argo::iridium::xfloat::XFloatDiveData::HasDriftData(), argo::iridium::xfloat::XFloatDiv
argo::iridium::xfloat::XFloatDiveData::HasFallRiseData(), argo::iridium::xfloat::XFloatDiveData::HasPositionData(),
argo::iridium::xfloat::XFloatDiveData::HasUnbinnedProfileData(), argo::iridium::xfloat::XFloatDiveData::HasValidGpsPosition(),
argo::iridium::Iridium::id_dac(), argo::iridium::xfloat::XFloatDiveData::id_dive(), id_dive(), argo::iridium::Iridium::id_imei(),
argo::iridium::Iridium::id_instrument(), id_msg(), id_msg_uid(), id_sbd_momsn(), id_sbd_mtmsn(), id_sbd_packet(),
argo::iridium::Iridium::id_wmo(), argo::iridium::xfloat::XFloatDiveData::intermediate_listing_file_path(), util::xmsg::AbstractXSensorDataCo
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::IsDataComplete(),
argo::iridium::xfloat::XFloatMetadata::local_path(), util::thread::RecursiveLockable::LockObject(), argo::iridium::xfloat::XFloatMetadata::ma
argo::iridium::xfloat::XFloat::metadata(), msg_body_is_valid(), msg_body_text(), msg_header_datetime(), msg_header_from(),
msg_header_is_valid(), msg_header_raw_text(), msg_header_subject(), msg_is_valid(), msg_raw_text(), util::xmsg::AbstractXSensorData
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::num_saln_samples(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::num_temp_samples(),
argo::iridium::xfloat::XFloat::offset_pressure(), argo::iridium::xfloat::XFloat::offset_pressure_surface(), argo::iridium::xfloat::XFloat::offset_
argo::iridium::xfloat::XFloat::offset_salinity_surface(), argo::iridium::xfloat::XFloat::offset_temperature(), argo::iridium::xfloat::XFloat::offset
ParseMailMsg(), ParseMailMsg(), ParseMailMsgBody(), ParseMailMsgHeader(), ParseMailMsgSbdAttachment(),

argo::iridium::xfloat::XFloat::phy_file_generation_is_enabled(), argo::iridium::xfloat::XFloatDiveData::phy_file_path(), argo::iridium::xfloat::XFloatMetadata::principal_investigators(), argo::iridium::xfloat::XFloatMetadata::principal_investigators_address(), argo::iridium::xfloat::XFloatMetadata::principal_investigators_originating_country(), raw_file_local_path(), util::xmsg::AbstractXSensorData argo::iridium::xfloat::XFloatDiveData::ResetData(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_C util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SamplePresAsFloat(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleSaln(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleSalnAsFloat(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleTemp(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleTempAsFloat(), argo::iridium::xfloat::XFloatDiveData::SaveObjectToDatabase(), SaveObjectToDatabase(), argo::iridium::xfloat::XFloatMetadata::SaveObj sbd_data(), sbd_data_is_valid(), sbd_file_local_path(), sbd_file_name(), sbd_file_size_bytes(), sbd_session_status_code(), sbd_session_status_string(), argo::iridium::xfloat::XFloatDiveData::SensorGainsAndOffsetsAreKnown(), util::xmsg::AbstractXSensorData argo::iridium::xfloat::XFloat::set_argo_instrument_type(), argo::iridium::xfloat::XFloatDiveData::set_argo_instrument_type(), argo::iridium::xfloat::XFloatMetadata::set_comment(), argo::iridium::xfloat::XFloatMetadata::set_deployer(), argo::iridium::xfloat::XFloatMe argo::iridium::xfloat::XFloatMetadata::set_deployment_cruise_id(), argo::iridium::xfloat::XFloatMetadata::set_deployment_datetime(), argo::iridium::xfloat::XFloatMetadata::set_deployment_datetime_status(), argo::iridium::xfloat::XFloatMetadata::set_deployment_latitude() argo::iridium::xfloat::XFloatMetadata::set_deployment_longitude(), argo::iridium::xfloat::XFloatMetadata::set_deployment_platform(), argo::iridium::xfloat::XFloatMetadata::set_deployment_position_status(), argo::iridium::xfloat::XFloatMetadata::set_deployment_type(), argo::iridium::xfloat::XFloat::set_fw_version(), argo::iridium::xfloat::XFloat::set_gain_pressure(), argo::iridium::xfloat::XFloat::set_gain_pre argo::iridium::xfloat::XFloat::set_gain_salinity(), argo::iridium::xfloat::XFloat::set_gain_salinity_surface(), argo::iridium::xfloat::XFloat::set_g argo::iridium::xfloat::XFloat::set_gain_temperature_surface(), argo::iridium::Iridium::set_id_dac(), argo::iridium::xfloat::XFloatDiveData::se set_id_dive(), argo::iridium::Iridium::set_id_imei(), argo::iridium::Iridium::set_id_instrument(), set_id_msg(), set_id_msg_uid(), set_id_sbd_momsn(), set_id_sbd_mtmsn(), set_id_sbd_packet(), argo::iridium::Iridium::set_id_wmo(), argo::iridium::xfloat::XFloatDiveDat argo::iridium::xfloat::XFloatMetadata::set_local_path(), argo::iridium::xfloat::XFloatMetadata::set_manufacturer(), argo::iridium::xfloat::XFloat::set_metadata(), set_msg_body_is_valid(), set_msg_body_text(), set_msg_header_datetime(), set_msg_header_from(), set_msg_header_is_valid(), set_msg_header_raw_text(), set_msg_header_subject(), set_msg_is_valid(), set_msg_raw_text(), argo::iridium::xfloat::XFloat::set_offset_pressure(), argo::iridium::xfloat::XFloat::set_offset_press argo::iridium::xfloat::XFloat::set_offset_salinity(), argo::iridium::xfloat::XFloat::set_offset_salinity_surface(), argo::iridium::xfloat::XFloat::se argo::iridium::xfloat::XFloat::set_offset_temperature_surface(), argo::iridium::xfloat::XFloat::set_phy_file_generation_is_enabled(), argo::iridium::xfloat::XFloatDiveData::set_phy_file_path(), argo::iridium::xfloat::XFloatMetadata::set_principal_investigators(), argo::iridium::xfloat::XFloatMetadata::set_principal_investigators_address(), argo::iridium::xfloat::XFloatMetadata::set_principal_investiga set_raw_file_local_path(), set_sbd_data(), set_sbd_data_is_valid(), set_sbd_file_local_path(), set_sbd_file_name(), set_sbd_file_size_bytes(), set_sbd_session_status_code(), set_sbd_session_status_string(), argo::iridium::xfloat::XFloat::set_start_of_mi argo::iridium::xfloat::XFloatMetadata::set_wmo_instrument_type(), argo::iridium::xfloat::XFloatMetadata::set_wmo_recorder_type(), argo::iridium::xfloat::XFloat::set_xmsg_version_major(), argo::iridium::xfloat::XFloat::set_xmsg_version_minor(), argo::iridium::xfloat::XFloat::start_of_mission_datetime(), util::thread::RecursiveLockable::UnlockObject(), argo::iridium::xfloat::XFloatDive argo::iridium::xfloat::XFloatMetadata::wmo_instrument_type(), argo::iridium::xfloat::XFloatMetadata::wmo_recorder_type(), argo::iridium::xfloat::XFloatDiveData::WriteListingFile(), argo::iridium::xfloat::XFloatDiveData::WritePhyFile(), WriteSbdFile(), argo::iridium::xfloat::XFloat::xmsg_version_major(), and argo::iridium::xfloat::XFloat::xmsg_version_minor().

10.88.5.27 password_

`std::string util::mysql::LocalMysqlConfiguration::password_ [static], [private], [inherited]`

The SQL server account password string.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.88.5.28 port_

`unsigned int util::mysql::LocalMysqlConfiguration::port_ [static], [private], [inherited]`

The SQL server port number.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.88.5.29 position_datetime_

`ptime util::position::Position::position_datetime_ [private], [inherited]`

The date and time of the position.

Referenced by util::position::Position::position_datetime(), util::position::Position::position_datetime_as_iso8601_string(), and util::position::Position::set_position_datetime().

10.88.5.30 position_is_valid_

```
bool util::position::Position::position_is_valid_  [private], [inherited]
```

The valid position indicator.

Referenced by util::position::Position::position_is_valid(), and util::position::Position::set_position_is_valid().

10.88.5.31 position_latitude_

```
double util::position::Position::position_latitude_  [private], [inherited]
```

The latitude of the position, in decimal degrees.

Referenced by util::position::Position::position_latitude(), util::position::Position::postition_as_iso6709_string(), util::position::Position::set_position_latitude(), and util::position::Position::set_position_longitude().

10.88.5.32 position_longitude_

```
double util::position::Position::position_longitude_  [private], [inherited]
```

The longitude of the position, in decimal degrees.

Referenced by util::position::Position::position_longitude(), util::position::Position::postition_as_iso6709_string(), util::position::Position::set_position_latitude(), and util::position::Position::set_position_longitude().

10.88.5.33 protocol_

```
std::string util::mysql::LocalMysqlConfiguration::protocol_  [static], [private], [inherited]
```

The protocol to use in the SQL server connection e.g.

"tcp".

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.88.5.34 raw_directory_path_

```
path argo::iridium::xfloat::XFloatMailMessage::raw_directory_path_  [static], [private]
```

The RAW mail message file directory.

This field determines the directory to which RAW mail message files are written. If enabled, RAW mail message files will be written to subdirectories of this directory that are organized by the unique instrument identifiers.

The value of this field is set as a result of parsing the command-line arguments and/or configuration file options during application initialization.

Referenced by WriteRawFile().

10.88.5.35 raw_file_format_

```
int argo::iridium::xfloat::XFloatMailMessage::raw_file_format_  [static], [private]
```

The raw mail message file format.

The value of this field determines the format of the raw mail message that is written to the local filesystem. The support formats are enumerated by XFloatMailRawFileFormat_t.

The value of this field is set as a result of parsing the command-line arguments and/or configuration file options during application initialization. If the value is unspecified, then it defaults to a value of XMAIL_DFLT_RAW_FILE_FORMAT.

10.88.5.36 raw_file_generation_is_enabled_

bool argo::iridium::xfloat::XFloatMailMessage::raw_file_generation_is_enabled_ [static], [private]

The global RAW-mail-message-file-generation-is-enabled flag.

This flag determines whether the ASCII RAW mail message files are written to the local filesystem. If TRUE, then the RAW mail message files will be written to subdirectories of the raw mail message directory which is specified by raw_directory_path_. Otherwise, if FALSE, raw mail message will not be written to the local filesystem.

The value of this field is set as a result of parsing the command-line arguments and/or configuration file options during application initialization. If the value is unspecified, then it defaults to a value of XMAIL_DFLT_SBD_FILE_VALID_NAME_REGEX.

Referenced by WriteRawFile().

10.88.5.37 raw_file_local_path_

path argo::iridium::xfloat::XFloatMailMessage::raw_file_local_path_ [private]

The path to the raw mail message file on the local filesystem.

This path is set as a result of writing the raw mail message file to the local filesystem. This path is relative to the raw file directory, which is specified by raw_directory_path_.

Referenced by raw_file_local_path(), and set_raw_file_local_path().

10.88.5.38 sbd_data_

XMailSbdData_t argo::iridium::xfloat::XFloatMailMessage::sbd_data_ [private]

The Iridium Short Burst Data (SBD) binary data.

This field contains the binary data that is read from the SBD file attachment. The content of this data is exactly that which was transmitted by the instrument.

Referenced by XFloatMailMessage(), XFloatMailMessage(), ParseMailMsgSbdAttachment(), sbd_data(), and set_sbd_data().

10.88.5.39 sbd_data_is_valid_

bool argo::iridium::xfloat::XFloatMailMessage::sbd_data_is_valid_ [private]

The Iridium Short Burst Data (SBD) binary data is valid flag.

This field indicates whether the attached SBD message is valid. The criteria for a valid SBD message is that

- the mail message has a file attachment,
- the filename of the attached file matches the SBD file email attachment filename regular expression, sbd_file_valid_name_regex_,
- the Iridium session status code, sbd_session_status_code_, has a value of zero, and
- the SBD message size, in bytes, is greater than or equal to XMSG_SBD_MIN_BYTES.

Referenced by sbd_data_is_valid(), and set_sbd_data_is_valid().

10.88.5.40 sbd_directory_path_

path argo::iridium::xfloat::XFloatMailMessage::sbd_directory_path_ [static], [private]

The SBD file directory.

This field determines the directory to which the binary Iridium Short Burst Data (SBD) files are written. If enabled, SBD files will be written to subdirectories of this directory that are organized by the unique instrument identifiers.

The value of this field is set as a result of parsing the command-line arguments and/or configuration file options during application initialization.

Referenced by WriteSbdFile().

10.88.5.41 sbd_file_generation_is_enabled_

```
bool argo::iridium::xfloat::XFloatMailMessage::sbd_file_generation_is_enabled_  [static], [private]
```

The global binary-SBD-file-generation-is-enabled flag.

This flag determines whether binary Iridium Short Burst Data (SBD) files are written to the local file system. If TRUE, then SBD files will be written to subdirectories of the SBD directory which is specified by sbd_directory_path_. Otherwise, if FALSE, SBD files will not be written to the local filesystem.

The value of this field is set as a result of parsing the command-line arguments and/or configuration file options during application initialization. If the value is unspecified, then it defaults to a value of FALSE.

Referenced by WriteSbdFile().

10.88.5.42 sbd_file_local_path_

```
path argo::iridium::xfloat::XFloatMailMessage::sbd_file_local_path_  [private]
```

The path to the Iridium Short Burst Data (SBD) binary file on the local filesystem.

This path is set as a result of writing the binary SBD message file to the local filesystem. This path is relative to the SBD file directory, which is specified by sbd_directory_path_.

Referenced by sbd_file_local_path(), and set_sbd_file_local_path().

10.88.5.43 sbd_file_name_

```
string argo::iridium::xfloat::XFloatMailMessage::sbd_file_name_  [private]
```

The Iridium Short Burst Data (SBD) email attachment filename.

This field represents the name of the file attached to the mail message. This field is used, in part, to determined the validity of the SBD message attachment.

Referenced by ParseMailMsgSbdAttachment(), sbd_file_name(), and set_sbd_file_name().

10.88.5.44 sbd_file_size_bytes_

```
int argo::iridium::xfloat::XFloatMailMessage::sbd_file_size_bytes_  [private]
```

The size of the Iridium Short Burst Data (SBD) message, in bytes.

This field is parsed from the body of the mail message. It should match the size of the SBD data, sbd_data_, which is read from the mail message attachment.

Referenced by ParseMailMsgBody(), sbd_file_size_bytes(), and set_sbd_file_size_bytes().

10.88.5.45 sbd_file_valid_name_regex_

```
string argo::iridium::xfloat::XFloatMailMessage::sbd_file_valid_name_regex_  [static], [private]
```

The valid SBD file email attachment filename regular expression.

This regular expression is used to validate the filename of the email message attachment. If the filename of the file attachment to the mail message matches this regular expression, then the mail message will be considered valid. Otherwise, if the filename does not match this regular expression, then the mail message will be considered invalid and will be discarded.

The value of this field is set as a result of parsing the command-line arguments and/or configuration file options during application initialization. If the value is unspecified, then it defaults to a value of XMAIL_DFLT_SBD_FILE_VALID_NAME_REGEX.

Referenced by ParseMailMsgSbdAttachment().

10.88.5.46 sbd_session_status_code_

```
int argo::iridium::xfloat::XFloatMailMessage::sbd_session_status_code_  [private]
```

The Iridium Short Burst Data (SBD) session status code.

This field is parsed from the body of the mail message. It indicates that status of the Iridium session. A successful Iridium session reports a status code of 0. An unsuccessful Iridium session reports a non-zero status code, where the exact value of the status code indicates a specific error or type of error in the Iridium session.

Referenced by ParseMailMsgBody(), sbd_session_status_code(), and set_sbd_session_status_code().

10.88.5.47 sbd_session_status_string_

```
string argo::iridium::xfloat::XFloatMailMessage::sbd_session_status_string_  [private]
```

The Iridium Short Burst Data (SBD) session status string.

This field is parsed from the body of the mail message. It contains a brief description of the status of the Iridium session that is indicated by the status code, sbd_session_status_code_.

Referenced by sbd_session_status_string(), and set_sbd_session_status_string().

10.88.5.48 schema_

```
std::string util::mysql::LocalMysqlConfiguration::schema_  [static], [private], [inherited]
```

The SQL server schema name.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.88.5.49 sql_connection_

```
thread_specific_ptr<Connection> util::mysql::MysqlUtils::sql_connection_  [static], [private],
[inherited]
```

Tthe thread-specific pointer to MySQL connection object.

Thread-specific MySQL connection, released by `MysqlUtils::CleanupSqlConnection()` when the thread exits.

Referenced by util::mysql::MysqlUtils::CloseSqlConnection(), and util::mysql::MysqlUtils::GetSqlConnection().

10.88.5.50 sql_driver_

```
thread_specific_ptr<MySQL_Driver> util::mysql::MysqlUtils::sql_driver_  [static], [private],
[inherited]
```

The thread-specific pointer to MySQL driver object.

Thread-specific reference to the MySQL driver.

Its cleanup function, `MysqlUtils::CleanupSqlDriver()`, does not delete the shared driver instance.

Referenced by util::mysql::MysqlUtils::GetSqlDriver().

10.88.5.51 sql_table_description_

```
const util::mysql::SqlTableDescription< XFloatMailMessage > argo::iridium::xfloat::XFloatMail↩
Message::sql_table_description_  [static], [private]
```

The SQL table description object.

This SQL table description provides the details needed to map this object to a relational database table. It utilizes the accessor methods of the class and the details of the SQL table column to (1) create an SQL INSERT statement to save the object into the database, (2) populate one or more objects' fields as a result of an SQL SELECT query, or (3) query the SQL database to access one or more of the values that are stored the the SQL table corresponding to this object.

Referenced by SaveObjectToDatabase(), SelectAllObjectsFromDatabase(), SelectObjectFromDatabase(), SelectObjectsFromDatabase(), and SelectValueFromDatabase().

10.88.5.52 user_

```
std::string util::mysql::LocalMysqlConfiguration::user_  [static], [private], [inherited]
```

The SQL server account username string.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

The documentation for this class was generated from the following files:

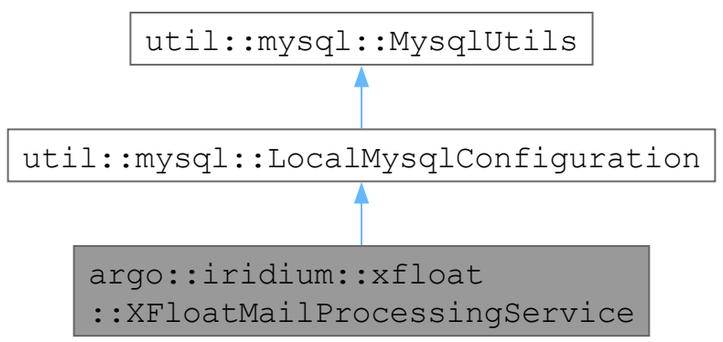
- data/XFloatMailMessage.h
- data/XFloatMailMessage.cpp

10.89 argo::iridium::xfloat::XFloatMailProcessingService Class Reference

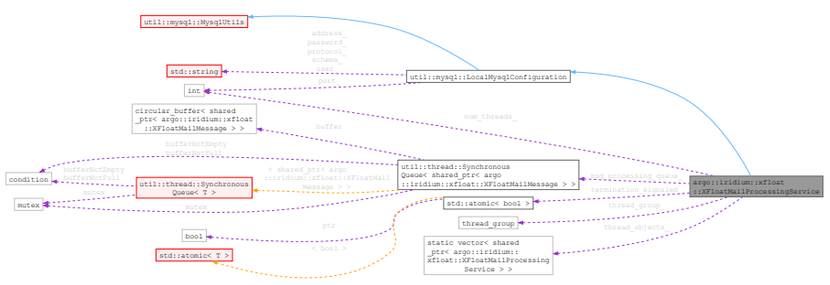
The mail processing thread class.

```
#include <stage2_MailProcessingThread.h>
```

Inheritance diagram for argo::iridium::xfloat::XFloatMailProcessingService:



Collaboration diagram for argo::iridium::xfloat::XFloatMailProcessingService:



Public Member Functions

- XFloatMailProcessingService ()

  The constructor.
- ~XFloatMailProcessingService ()

  The destructor.
- void operator() () ATTR_NO_EXCEPTIONS

  Functor operator overload function: the main entry point for the mail processing thread(s).

Static Public Member Functions

- static void CloseSqlConnection ()

  Close the thread-specific MySQL connection if it is currently open.
- static shared_ptr< sql::ResultSet > **executeDeleteQuery** (const string &from, const string &where="")
- static shared_ptr< ResultSet > ExecuteQuery (Connection *con, const string &query)

  Execute an SQL query and return a shared pointer to the ResultSet object.
- static shared_ptr< sql::ResultSet > executeQuery (const string &query)

  Execute an SQL query and return a shared pointer to the ResultSet object.
- static shared_ptr< sql::ResultSet > executeSelectQuery (const string &what, const string &from, const string &where="", const string &groupBy="", const string &orderBy="", unsigned int limit=0, unsigned int offset=0)

  Execute an SQL select query and return a shared pointer to the ResultSet object.
- static int ExecuteUpdate (Connection *con, const string &update)

  Execute an SQL update and return the status code.
- static int executeUpdate (const string &update)

  Execute an SQL update and return the status code.
- static shared_ptr< PreparedStatement > GeneratePreparedStatement (Connection *con, const string &statement)

  Generate a prepared SQL statement object for the given SQL statement.
- static shared_ptr< sql::PreparedStatement > generatePreparedStatement (const string &statement)

  Generate a prepared SQL statement object for the given SQL statement.
- static string GenerateSqlDeleteStr (const string &from, const string &where="")

  Generate an SQL DELETE statement.
- static string GenerateSqlSelectStr (const string &what, const string &from, const string &where="", const string &groupBy="", const string &orderBy="", unsigned int limit=0, unsigned int offset=0)

  Generate an SQL select statement.
- static thread_specific_ptr< Connection > & GetSqlConnection (string url, string user, string password, string schema)

  Get a thread-specific instance of a MySQL connection object.
- static thread_specific_ptr< MySQL_Driver > & GetSqlDriver ()

  Get a thread-specific instance of a MySQL driver object.
- static bool MsgProcessingQueuePush (const shared_ptr< XFloatMailMessage > &msg, bool block=true)

  Push a mail message into the mail processing queue.
- static void MsgProcessingQueuePushAll (vector< shared_ptr< XFloatMailMessage > >::const_iterator first, vector< shared_ptr< XFloatMailMessage > >::const_iterator last)

  Push multiple mail messages into the mail processing queue.
- static unsigned int num_threads ()

  Get the number of mail message processing threads.
- static void StartWorkerThreads () ATTR_NO_EXCEPTIONS

  Instantiate the mail processing worker thread(s).
- static void TerminateWorkerThreads () ATTR_NO_EXCEPTIONS

  Terminate the mail processing worker thread(s).

Static Protected Member Functions

- static sql::Connection * getSqlConnection ()

  Generate a thread-specific instance of a MySQL connection object.

Private Member Functions

- void ProcessMailMsg (const shared_ptr< XFloatMailMessage > &msg) const

  Process a mail message.

Static Private Member Functions

- static void CleanupSqlConnection (Connection ∗con)

  Cleanup an SQL connection.
- static void CleanupSqlDriver (MySQL_Driver ∗driver)

  Function for memory cleanup of MySQL driver on destruct.
- static bool MsgProcessingQueuePop (shared_ptr< XFloatMailMessage > &msg, bool block=false)

  Pop a mail message out of the mail processing queue.
- static void set_termination_signaled (bool termination_signaled=true)

  Set the termination signaled flag.

Static Private Attributes

- static string address_

  The SQL server qualified name or IP address string.
- static util::thread::SynchronousQueue< shared_ptr< XFloatMailMessage > > msg_processing_queue_

  The incoming mail message processing queue.
- static unsigned int num_threads_

  The number of mail processing threads.
- static string password_

  The SQL server account password string.
- static unsigned int port_

  The SQL server port number.
- static string protocol_

  The protocol to use in the SQL server connection e.g.
- static string schema_

  The SQL server schema name.
- static thread_specific_ptr< Connection > sql_connection_

  Tthe thread-specific pointer to MySQL connection object.
- static thread_specific_ptr< MySQL_Driver > sql_driver_

  The thread-specific pointer to MySQL driver object.
- static std::atomic< bool > termination_signaled_

  The termination signaled flag.
- static thread_group thread_group_

  The mail processing thread group.
- static vector< shared_ptr< XFloatMailProcessingService > > thread_objects_

  The mail processing thread object(s).
- static string user_

  The SQL server account username string.

Friends

- po::options_description util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription ()

  Friend function for populating some of the static members of this class from the configuration file options.

10.89.1 Detailed Description

The mail processing thread class.

This class defines the operation of the mail processing thread(s), which are the second stage in the processing pipeline. The mail processing thread(s) consumes messages that were fetched from the mail server in the previous stage of the processing pipeline, XFloatMailRetrievalService. The messages are validated, saved to the SQL database, queued for consumption in the next stage of the processing pipeline, the telemetry decoder thread(s), XFloatTelemetryDecoderService, and finally their file representations are written to the local file system.

Zero or more instance of this thread may be instantiated after application initialization, depending on the configuration options. Each thread enters a primary loop in which messages are fetched from a common queue and processed. If the incoming queue is empty, then the thread will sleep for intervals of XFLT_THREAD_SLEEP_MS milliseconds.

The primary loop will continue until the daemon thread signals termination. When termination is signaled, the remaining messages in the incoming queue will be processed, then all instances of this thread will die. This ensures that processing pipeline will be thoroughly flushed before application termination, and that no messages will be partially or incompletely processed.

10.89.2 Constructor & Destructor Documentation

10.89.2.1 XFloatMailProcessingService()

```
argo::iridium::xfloat::XFloatMailProcessingService::XFloatMailProcessingService ()
```

The constructor.

10.89.2.2 ~XFloatMailProcessingService()

```
argo::iridium::xfloat::XFloatMailProcessingService::~XFloatMailProcessingService ()
```

The destructor.

10.89.3 Member Function Documentation

10.89.3.1 CleanupSqlConnection()

```
static void util::mysql::MysqlUtils::CleanupSqlConnection (
            Connection * con)  [inline], [static], [private], [inherited]
```

Cleanup an SQL connection.

This function for memory cleanup of the SQL connection on destruction. The thread-specific SQL connection object will be deleted if it exists.

Parameters

in	*con*	The SQL connection to cleanup.

Exceptions

log::MessageException	if an error occurs while cleaning up the SQL connection object.

References CATCH_BLOCK.

10.89.3.2 CleanupSqlDriver()

```
static void util::mysql::MysqlUtils::CleanupSqlDriver (
            MySQL_Driver * driver)  [inline], [static], [private], [inherited]
```

Function for memory cleanup of MySQL driver on destruct.

Signals the SQL driver to end operation, if it exists.

Parameters

in	*driver*	Driver to cleanup.

Exceptions

log::MessageException	if an error occurs while cleaning up the SQL driver object.

References CATCH_BLOCK.

10.89.3.3 CloseSqlConnection()

```
static void util::mysql::MysqlUtils::CloseSqlConnection ()  [inline], [static], [inherited]
```

Close the thread-specific MySQL connection if it is currently open.

If the thread local MySQL connection is not currently open, then this function will have no effect.

Note

The MySQL connection should be closed when idle to prevent the connection from timing out and throwing exceptions.

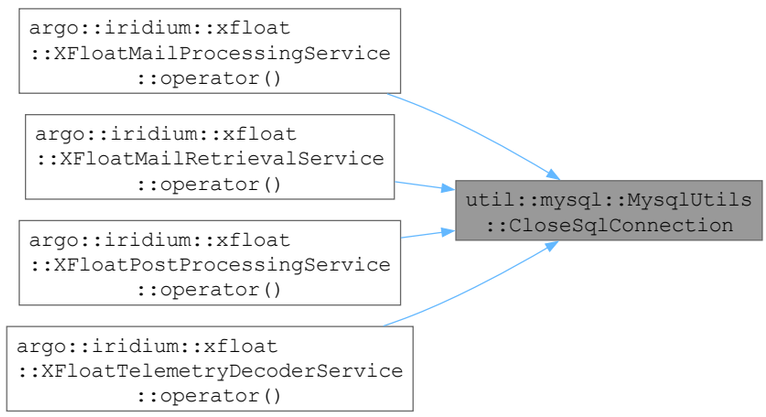
Exceptions

log::MessageException	if an error occurs while attempting to close the MySQL connection.

References CATCH_BLOCK, and util::mysql::MysqlUtils::sql_connection_.

Referenced by operator()(), argo::iridium::xfloat::XFloatMailRetrievalService::operator()(), argo::iridium::xfloat::XFloatPostProcessingServi and argo::iridium::xfloat::XFloatTelemetryDecoderService::operator()().

Here is the caller graph for this function:



10.89.3.4 ExecuteQuery()

```
static shared_ptr< ResultSet > util::mysql::MysqlUtils::ExecuteQuery (
            Connection * con,
            const string & query)  [inline], [static], [inherited]
```

Execute an SQL query and return a shared pointer to the ResultSet object.

Parameters

in	*con*	A pointer to the SQL connection object. This must not be NULL.
in	*query*	The SQL query to execute. This must be a well-formatted SQL query string.

Exceptions

log::MessageException	if 'con' is null, 'query' is a malfored SQL string, or any other error occurs.

Returns

A shared pointer to the query ResultSet object.

References CATCH_BLOCK, and util::mysql::MysqlUtils::GeneratePreparedStatement().

Referenced by util::mysql::LocalMysqlConfiguration::executeQuery().

Here is the call graph for this function:

Here is the caller graph for this function:

10.89.3.5 executeQuery()

```
shared_ptr< sql::ResultSet > util::mysql::LocalMysqlConfiguration::executeQuery (
            const string & query)  [static], [inherited]
```

Execute an SQL query and return a shared pointer to the ResultSet object.

Parameters

in	*query*	The SQL query to execute. This must be a well-formatted SQL query string.

Exceptions

log::MessageException	'query' is a malfored SQL string or any other error occurs while executing the query.

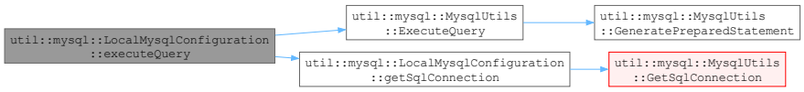
Returns

A shared pointer to the query ResultSet object.

References util::mysql::MysqlUtils::ExecuteQuery(), and util::mysql::LocalMysqlConfiguration::getSqlConnection().

Referenced by util::mysql::LocalMysqlConfiguration::executeSelectQuery(), and util::mysql::SqlTableDescription< T >::SelectValueFromD

Here is the call graph for this function:



Here is the caller graph for this function:

10.89.3.6 executeSelectQuery()

```
shared_ptr< sql::ResultSet > util::mysql::LocalMysqlConfiguration::executeSelectQuery (
            const string & what,
            const string & from,
            const string & where = "",
            const string & groupBy = "",
            const string & orderBy = "",
            unsigned int limit = 0,
            unsigned int offset = 0)  [static], [inherited]
```

Execute an SQL select query and return a shared pointer to the ResultSet object.

Parameters

in	*what*	The SQL select statement select-expression clause.
in	*from*	The SQL select statement table–references clause.
in	*where*	The SQL select statement where-condition clause.
in	*groupBy*	The SQL select statement group-by clause.
in	*orderBy*	The SQL select statement order-by clause.
in	*limit*	The SQL select statement result set row limit value.
in	*offset*	The SQL select statement result set row offset.

Exceptions

log::MessageException	if an error occurs while executing the query.

Returns

A shared pointer to the ResultSet object.

References CATCH_BLOCK, util::mysql::LocalMysqlConfiguration::executeQuery(), and util::mysql::MysqlUtils::GenerateSqlSelectStr().

Referenced by util::mysql::SqlTableDescription< T >::SelectObjectsFromDatabase().

Here is the call graph for this function:

Here is the caller graph for this function:

10.89.3.7 ExecuteUpdate()

```
static int util::mysql::MysqlUtils::ExecuteUpdate (
            Connection * con,
            const string & update)  [inline], [static], [inherited]
```

Execute an SQL update and return the status code.

Parameters

in	*con*	A pointer to the SQL connection object. This must not be NULL.
in	*update*	The SQL update to execute. This must be a well-formatted SQL update string.

Exceptions

log::MessageException	if 'con' is null, 'update' is a malformed SQL string, or any other error occurs.

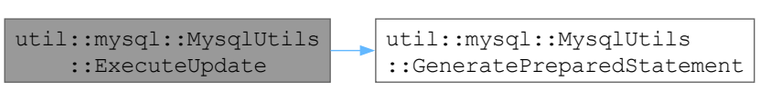
Returns

The update status code.

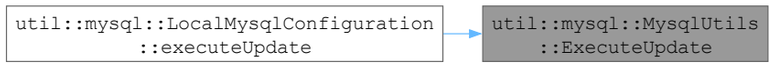
References CATCH_BLOCK, and util::mysql::MysqlUtils::GeneratePreparedStatement().

Referenced by util::mysql::LocalMysqlConfiguration::executeUpdate().

Here is the call graph for this function:



Here is the caller graph for this function:



10.89.3.8 executeUpdate()

```
int util::mysql::LocalMysqlConfiguration::executeUpdate (
            const string & update)  [static], [inherited]
```

Execute an SQL update and return the status code.

Parameters

in	*update*	The SQL update to execute. This must be a well-formatted SQL update string.

Exceptions

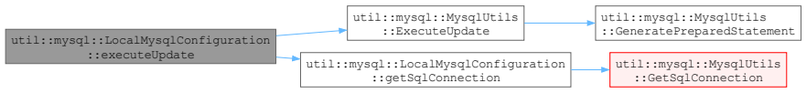
log::MessageException	if 'update' is a malformed SQL string, or any other error occurs while executing the update.

Returns

The update status code.

References util::mysql::MysqlUtils::ExecuteUpdate(), and util::mysql::LocalMysqlConfiguration::getSqlConnection().

Here is the call graph for this function:



10.89.3.9 GeneratePreparedStatement()

```
static shared_ptr< PreparedStatement > util::mysql::MysqlUtils::GeneratePreparedStatement (
            Connection * con,
            const string & statement)  [inline], [static], [inherited]
```

Generate a prepared SQL statement object for the given SQL statement.

Parameters

in	*con*	A pointer to the SQL connection object. This must not be NULL.
in	*statement*	The SQL statement to prepare. This must be a well-formed SQL string.

Exceptions

log::MessageException	if 'con' is NULL, 'statement' is not a valid SQL string, or if an error occurs.

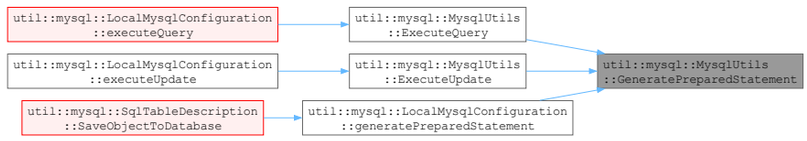
Returns

A prepared SQL statement object generated from the given SQL statement.

References CATCH_BLOCK, and EXC_THROW.

Referenced by util::mysql::MysqlUtils::ExecuteQuery(), util::mysql::MysqlUtils::ExecuteUpdate(), and util::mysql::LocalMysqlConfiguration:

Here is the caller graph for this function:



10.89.3.10 generatePreparedStatement()

```
std::shared_ptr< sql::PreparedStatement > util::mysql::LocalMysqlConfiguration::generatePrepared↩
Statement (
            const string & statement)  [static], [inherited]
```

Generate a prepared SQL statement object for the given SQL statement.

Parameters

in	*statement*	The SQL statement to prepare. This must be a well-formed SQL string.

Exceptions

log::MessageException	if 'statement' is not a valid SQL string or if an error occurs while generating the prepared statement object..

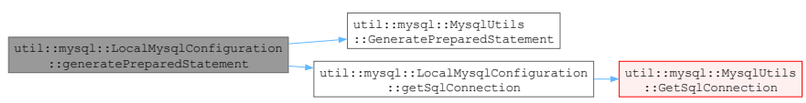
Returns

A prepared SQL statement object generated from the given SQL statement.

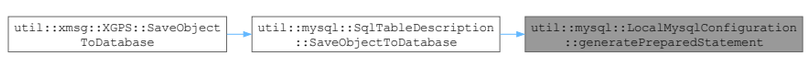
References util::mysql::MysqlUtils::GeneratePreparedStatement(), and util::mysql::LocalMysqlConfiguration::getSqlConnection().

Referenced by util::mysql::SqlTableDescription< T >::SaveObjectToDatabase().

Here is the call graph for this function:



Here is the caller graph for this function:



10.89.3.11 GenerateSqlDeleteStr()

```
static string util::mysql::MysqlUtils::GenerateSqlDeleteStr (
            const string & from,
            const string & where = "")  [inline], [static], [inherited]
```

Generate an SQL DELETE statement.

Warning

Without a `where` clause, the statement deletes every row in the table.

Parameters

in	*from*	The name of the table to delete from.
in	*where*	The SQL "where-condition" clause (empty = all rows).

Returns

The SQL statement.

10.89.3.12 GenerateSqlSelectStr()

```
static string util::mysql::MysqlUtils::GenerateSqlSelectStr (
            const string & what,
            const string & from,
            const string & where = "",
            const string & groupBy = "",
            const string & orderBy = "",
            unsigned int limit = 0,
            unsigned int offset = 0)  [inline], [static], [inherited]
```

Generate an SQL select statement.

Parameters

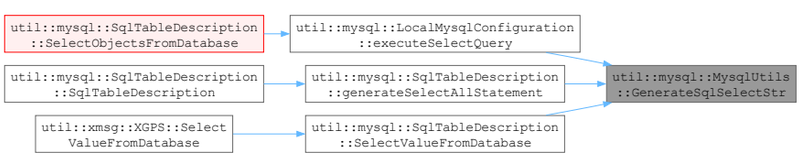
in	*what*	The SQL select statement select-expression clause.
in	*from*	The SQL select statement table-references clause.
in	*where*	The SQL select statement where-condition clause.
in	*groupBy*	The SQL select statement group-by clause.
in	*orderBy*	The SQL select statement order-by clause.
in	*limit*	The SQL select statement result set row limit value.
in	*offset*	The SQL select statement result set row offset.

Returns

The SQL select statement.

Referenced by util::mysql::LocalMysqlConfiguration::executeSelectQuery(), util::mysql::SqlTableDescription< T >::generateSelectAllState
and util::mysql::SqlTableDescription< T >::SelectValueFromDatabase().

Here is the caller graph for this function:



10.89.3.13 GetSqlConnection()

```
static thread_specific_ptr< Connection > & util::mysql::MysqlUtils::GetSqlConnection (
            string url,
            string user,
            string password,
            string schema)  [inline], [static], [inherited]
```

Get a thread-specific instance of a MySQL connection object.

A new connection will be created, if necessary.

Parameters

in	*url*	The MySQL host server qualified address.
in	*user*	The MySQL account username.
in	*password*	The MySQL account password.
in	*schema*	The MySQL account schema name.

Exceptions

log::MessageException	if an error occurs while creating a new thread-specific instance of the SQL connection object

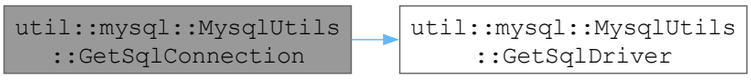
Returns

A thread-specific instance of a MySQL connection object.

References CATCH_BLOCK, util::mysql::MysqlUtils::GetSqlDriver(), and util::mysql::MysqlUtils::sql_connection_.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

Here is the call graph for this function:



Here is the caller graph for this function:

10.89.3.14 getSqlConnection()

`sql::Connection * util::mysql::LocalMysqlConfiguration::getSqlConnection () [static], [protected], [inherited]`

Generate a thread-specific instance of a MySQL connection object.

A new connection will be created, if necessary.

Exceptions

log::MessageException	if an error occurs while generating the SQL connection object.

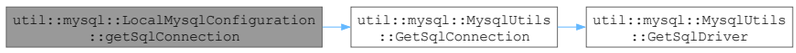
Returns

A thread-specific instance of a MySQL connection object.

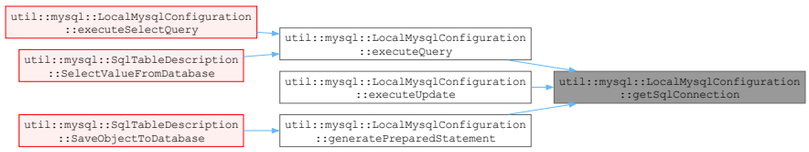
References util::mysql::LocalMysqlConfiguration::address_, CATCH_BLOCK, util::mysql::MysqlUtils::GetSqlConnection(), util::mysql::LocalMysqlConfiguration::password_, util::mysql::LocalMysqlConfiguration::port_, util::mysql::LocalMysqlConfiguration::protoc util::mysql::LocalMysqlConfiguration::schema_, and util::mysql::LocalMysqlConfiguration::user_.

Referenced by util::mysql::LocalMysqlConfiguration::executeQuery(), util::mysql::LocalMysqlConfiguration::executeUpdate(), and util::mysql::LocalMysqlConfiguration::generatePreparedStatement().

Here is the call graph for this function:



Here is the caller graph for this function:



10.89.3.15 GetSqlDriver()

```
static thread_specific_ptr< MySQL_Driver > & util::mysql::MysqlUtils::GetSqlDriver ()  [inline],
[static], [inherited]
```

Get a thread-specific instance of a MySQL driver object.

A new driver will created, if necessary.

Exceptions

log::MessageException	if an error occurs while creating a new thread-specific instance of the SQL driver object

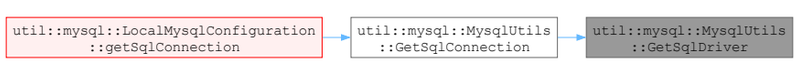
Returns

A thread-specific instance of a MySQL driver object.

References CATCH_BLOCK, and util::mysql::MysqlUtils::sql_driver_.

Referenced by util::mysql::MysqlUtils::GetSqlConnection().

Here is the caller graph for this function:



10.89.3.16 MsgProcessingQueuePop()

```
bool argo::iridium::xfloat::XFloatMailProcessingService::MsgProcessingQueuePop (
            shared_ptr< XFloatMailMessage > & msg,
            bool block = false)  [static], [private]
```

Pop a mail message out of the mail processing queue.

This function is used by the mail processing thread(s) to receive mail messages produced by the previous stage in the processing pipeline, the mail retrieval thread, XFloatMailRetrievalService. The mail message processing queue is synchronous, so the underlying pop function is atomic.

Parameters

out	*msg*	The destination of the mail message to pop out of the queue.
in	*block*	If TRUE, this function will block until the queue is not empty. If FALSE, this function will return FALSE if the queue is empty (and no element will be popped out of the queue).

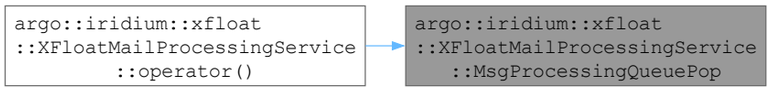
Returns

TRUE if the element was successfully popped out of the queue, or FALSE otherwise.

References msg_processing_queue_.

Referenced by operator()().

Here is the caller graph for this function:



10.89.3.17 MsgProcessingQueuePush()

```
bool argo::iridium::xfloat::XFloatMailProcessingService::MsgProcessingQueuePush (
            const shared_ptr< XFloatMailMessage > & msg,
            bool block = true)  [static]
```

Push a mail message into the mail processing queue.

This function is used by the previous stage in the processing pipeline, the mail retrieval thread, XFloatMailRetrievalService, to pass a mail message to the mail processing thread(s). The mail message processing queue is synchronous, so the underlying push function is atomic.

Parameters

in	*msg*	The mail message to push into the mail processing queue.
in	*block*	If TRUE, this function will block until the queue is not full. If FALSE, this function will return FALSE if the queue is full (and the element will not be pushed into the queue).

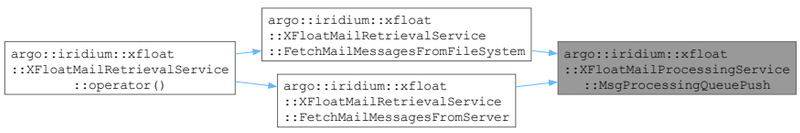
Returns

TRUE if the element was successfully pushed into the queue, or FALSE otherwise.

References msg_processing_queue_.

Referenced by argo::iridium::xfloat::XFloatMailRetrievalService::FetchMailMessagesFromFileSystem(), and argo::iridium::xfloat::XFloatMa

Here is the caller graph for this function:



10.89.3.18 MsgProcessingQueuePushAll()

```
void argo::iridium::xfloat::XFloatMailProcessingService::MsgProcessingQueuePushAll (
            vector< shared_ptr< XFloatMailMessage > >::const_iterator first,
            vector< shared_ptr< XFloatMailMessage > >::const_iterator last)  [static]
```

Push multiple mail messages into the mail processing queue.

This function is used by the previous stage in the processing pipeline, the mail retrieval thread, XFloatMailRetrievalService, to pass a group of mail messages to the mail processing thread. The mail message processing queue is synchronous, so the underlying push function is atomic.

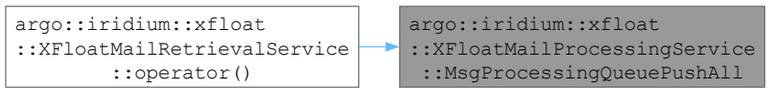
Parameters

in	*first,last*	Input iterators to the initial and final positions in a range. The range used is [first,last), which includes all the elements between first and last, including the element pointed by first but not the element pointed by last.

References msg_processing_queue_.

Referenced by argo::iridium::xfloat::XFloatMailRetrievalService::operator()().

Here is the caller graph for this function:



10.89.3.19 num_threads()

```
unsigned int argo::iridium::xfloat::XFloatMailProcessingService::num_threads ()  [static]
```

Get the number of mail message processing threads.

Returns

The number of mail message processing threads.

References num_threads_.

10.89.3.20 operator()()

```
void argo::iridium::xfloat::XFloatMailProcessingService::operator() ()
```

Functor operator overload function: the main entry point for the mail processing thread(s).

This function defines the behavior of the mail processing thread(s), which are responsible for processing the messages fetched from the mail server by the mail retrieval thread, XFloatMailRetrievalService. After initialization, zero or more instances of this thread are instantiated. The static configuration options for this thread are set as a result of parsing the command-line arguments and/or configuration file.

First, the scope thread attribute "ThreadName" is set to "mail-process". This attribute is used by the global logger object for filtering, etc. Then the thread enters its primary loop, in which it pops mail messages from the incoming queue one at a time and processes the messages before passing them to the next stage in the processing pipeline, XFloatTelemetryDecoderService. If the incoming message queue is empty, then the connection to the SQL database is closed, and the thread sleeps for XFLT_THREAD_SLEEP_MS milliseconds.

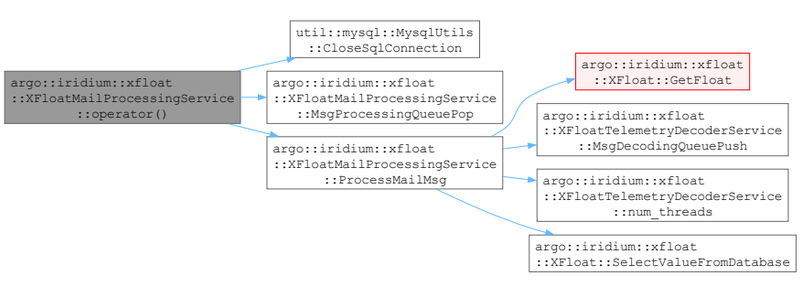
Once termination is signaled by the daemon process i.e. termination_signaled_ is TRUE, the thread will exit its primary loop and die.

Exceptions

None.	Never throws.

References CATCH_NOTHROW, util::mysql::MysqlUtils::CloseSqlConnection(), ldbg, MsgProcessingQueuePop(), ProcessMailMsg(), termination_signaled_, and XFLT_THREAD_SLEEP_MS.

Here is the call graph for this function:



10.89.3.21 ProcessMailMsg()

```
void argo::iridium::xfloat::XFloatMailProcessingService::ProcessMailMsg (
            const shared_ptr< XFloatMailMessage > & msg) const  [private]
```

Process a mail message.

This function is responsible for processing and validating a mail message before queuing the message for consumption by the next thread in the processing pipeline, XFloatTelemetryDecoderService.

First, the function checks the message reference `msg`. If it is NULL, then the function throws an exception. Otherwise, it continues on to set the global logger attributes:

- DiveID,
- MsgUID,
- PcktID,
- MomsnID,
- WhoiID,
- DacID, and
- WmoID.

If the values of the WhoiID, DacID, or WmoID attributes are unknown, the thread will attempt to query these identifiers from the SQL database. These attributes are used by the global logger object for filtering, etc.

Next, the message processing begins. If the message header is invalid, then the message is discarded. Otherwise, if the header is valid, the message is save to the SQL database. If the SBD data is valid, then the message is queued for consumption by the next thread in the processing pipeline, XFloatTelemetryDecoderService.

Finally, the raw mail file and binary SBD data file are written to the local filesystem. The SQL database entry for the mail messages is then updated with the paths to the files on the local filesystem.

Parameters

in	*msg*	A shared reference to the mail message to process.

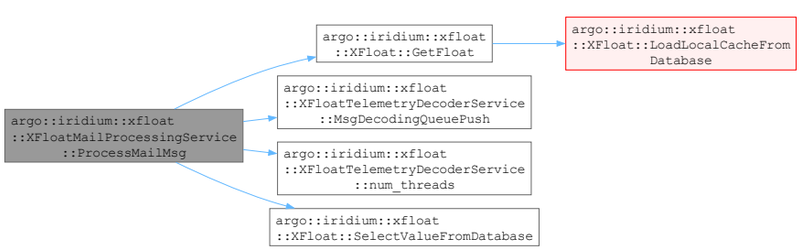
Exceptions

util::log::MessageException	if an error occurs while processing the mail message or the shared pointer to the mail message is NULL.

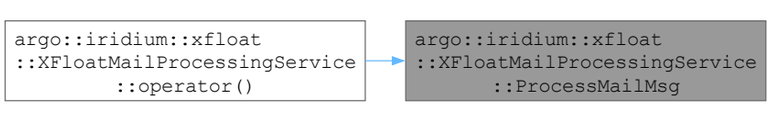
References CATCH_BLOCK, EXC_THROW, argo::iridium::xfloat::XFloat::GetFloat(), ldbg, linf, lwrn, argo::iridium::xfloat::XFloatTelemetryI argo::iridium::xfloat::XFloatTelemetryDecoderService::num_threads(), argo::iridium::xfloat::XFloat::SelectValueFromDatabase(), and XFLT_ID_UNKNOWN.

Referenced by operator()().

Here is the call graph for this function:



Here is the caller graph for this function:



10.89.3.22 set_termination_signaled()

```
void argo::iridium::xfloat::XFloatMailProcessingService::set_termination_signaled (
            bool termination_signaled = true)  [static], [private]
```

Set the termination signaled flag.

This function is called by TerminateWorkerThreads() in order to cause all currently running instances of the mail processing thread to exit their primary loop and die.

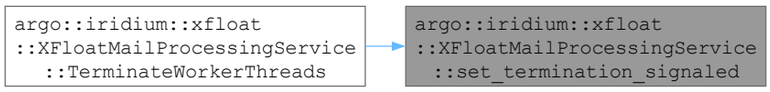
Parameters

in	*termination_signaled*	The termination signaled value to set.

References termination_signaled_.

Referenced by TerminateWorkerThreads().

Here is the caller graph for this function:



10.89.3.23 StartWorkerThreads()

```
void argo::iridium::xfloat::XFloatMailProcessingService::StartWorkerThreads ()  [static]
```

Instantiate the mail processing worker thread(s).

This function is used by the daemon thread to instantiate zero or more mail processing threads after initialization. The number of threads that are instantiated by this function is determined by the value of num_threads_, which is set as a result of parsing the command-line argument and/or configuration file during initialization.

Each thread is instantiated by creating an instance of this class, which is pushed into the thread_objects_ vector, and invoking the functor of each object. Each thread that is created is added to the thread_group_ object.

Exceptions

None.	Never throws.

References ldbg, num_threads_, thread_group_, and thread_objects_.

10.89.3.24 TerminateWorkerThreads()

```
void argo::iridium::xfloat::XFloatMailProcessingService::TerminateWorkerThreads ()  [static]
```

Terminate the mail processing worker thread(s).

This function is used by the daemon thread to kill the mail message processing worker thread(s) upon application termination. The function signals termination, and then attempts to join the thread group.

Warning

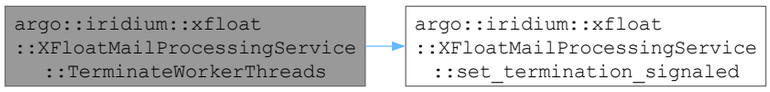
> If any of the mail message processing threads fails to terminate properly, then the `join_all()` function could block indefinitely. Precautions have been take to ensure that this does not happen, but it may still be possible.

Exceptions

None.	Never throws.

References ldbg, num_threads_, set_termination_signaled(), and thread_group_.

Here is the call graph for this function:



10.89.4 Friends And Related Symbol Documentation

10.89.4.1 util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription

```
po::options_description util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription
() [friend]
```

Friend function for populating some of the static members of this class from the configuration file options.

10.89.5 Member Data Documentation

10.89.5.1 address_

```
std::string util::mysql::LocalMysqlConfiguration::address_ [static], [private], [inherited]
```

The SQL server qualified name or IP address string.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.89.5.2 msg_processing_queue_

```
util::thread::SynchronousQueue< shared_ptr< XFloatMailMessage > > argo::iridium::xfloat::↩
XFloatMailProcessingService::msg_processing_queue_ [static], [private]
```

The incoming mail message processing queue.

This synchronous FIFO queue is used to pass messages produced by the previous stage of the processing pipeline, the mail retrieval thread, XFloatMailRetrievalService, to the mail processing thread(s). All the operations performed by on the queue are atomic. Depending on the number of mail processing threads, this queue may function as a one-to-one queue or one-to-many queue.

Referenced by MsgProcessingQueuePop(), MsgProcessingQueuePush(), and MsgProcessingQueuePushAll().

10.89.5.3 num_threads_

```
unsigned int argo::iridium::xfloat::XFloatMailProcessingService::num_threads_ [static], [private]
```

The number of mail processing threads.

The value of this field determines the number of mail processing threads that are instantiated after initialization. Its value is set as a result of parsing the configuration file and/or command-line arguments during initialization. If the value is unspecified, then it defaults to a value of CFG_DEFAULT_NPROC.

Referenced by num_threads(), StartWorkerThreads(), and TerminateWorkerThreads().

10.89.5.4 password_

```
std::string util::mysql::LocalMysqlConfiguration::password_  [static], [private], [inherited]
```

The SQL server account password string.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.89.5.5 port_

```
unsigned int util::mysql::LocalMysqlConfiguration::port_  [static], [private], [inherited]
```

The SQL server port number.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.89.5.6 protocol_

```
std::string util::mysql::LocalMysqlConfiguration::protocol_  [static], [private], [inherited]
```

The protocol to use in the SQL server connection e.g.

"tcp".

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.89.5.7 schema_

```
std::string util::mysql::LocalMysqlConfiguration::schema_  [static], [private], [inherited]
```

The SQL server schema name.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.89.5.8 sql_connection_

```
thread_specific_ptr<Connection> util::mysql::MysqlUtils::sql_connection_  [static], [private],
[inherited]
```

Tthe thread-specific pointer to MySQL connection object.

Thread-specific MySQL connection, released by `MysqlUtils::CleanupSqlConnection()` when the thread exits.

Referenced by util::mysql::MysqlUtils::CloseSqlConnection(), and util::mysql::MysqlUtils::GetSqlConnection().

10.89.5.9 sql_driver_

```
thread_specific_ptr<MySQL_Driver> util::mysql::MysqlUtils::sql_driver_  [static], [private],
[inherited]
```

The thread-specific pointer to MySQL driver object.

Thread-specific reference to the MySQL driver.

Its cleanup function, `MysqlUtils::CleanupSqlDriver()`, does not delete the shared driver instance.

Referenced by util::mysql::MysqlUtils::GetSqlDriver().

10.89.5.10 termination_signaled_

std::atomic< bool > argo::iridium::xfloat::XFloatMailProcessingService::termination_signaled_↩
[static], [private]

The termination signaled flag.

This flag is used to signal termination of the mail processing thread(s). During normal operation, the value of this field will be FALSE. Once termination is signaled, it will become TRUE causing all of the instances of the mail processing thread to process the remaining messages in the incoming queue, exiting their primary loop, and then die.

Referenced by operator()(), and set_termination_signaled().

10.89.5.11 thread_group_

thread_group argo::iridium::xfloat::XFloatMailProcessingService::thread_group_ [static], [private]

The mail processing thread group.

This object represents the group of mail processing threads that are instantiated after initialization. Each new thread that is instantiated is added to this group. Upon termination, the thread group is joined.

Referenced by StartWorkerThreads(), and TerminateWorkerThreads().

10.89.5.12 thread_objects_

vector< shared_ptr< XFloatMailProcessingService > > argo::iridium::xfloat::XFloatMailProcessing↩
Service::thread_objects_ [static], [private]

The mail processing thread object(s).

Each instance of the mail processing thread is created by invoking the functor of an instance of this runnable class. Each instance of this object that is created during instantiation of a new thread is stored in this vector.

Referenced by StartWorkerThreads().

10.89.5.13 user_

std::string util::mysql::LocalMysqlConfiguration::user_ [static], [private], [inherited]

The SQL server account username string.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

The documentation for this class was generated from the following files:

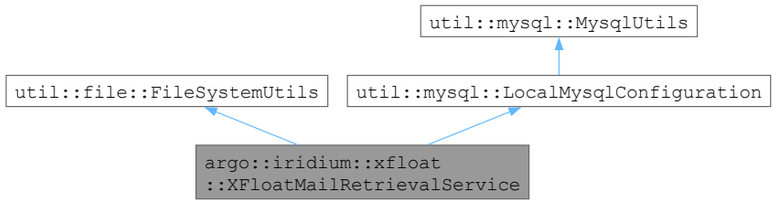
- stage2_MailProcessingThread.h
- stage2_MailProcessingThread.cpp

10.90 argo::iridium::xfloat::XFloatMailRetrievalService Class Reference

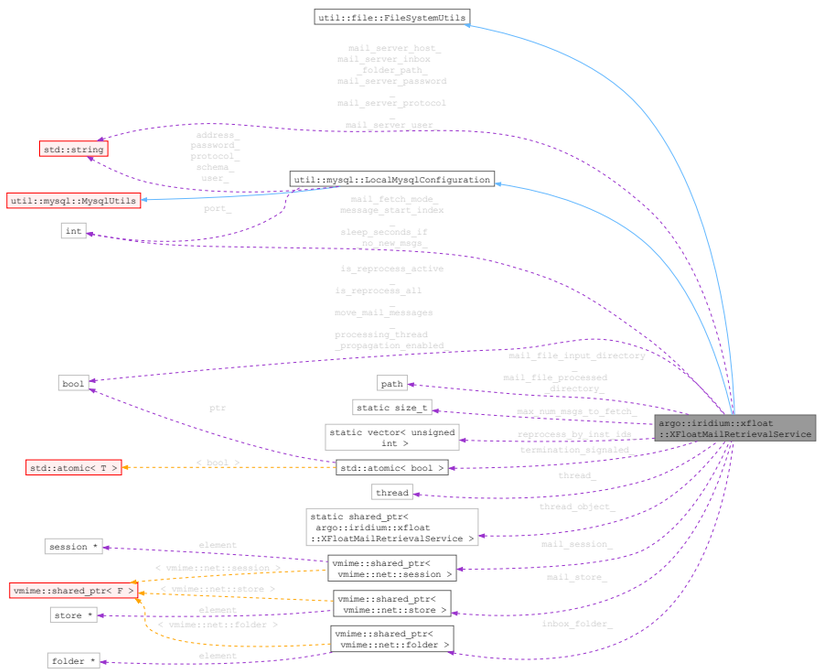
The mail retrieval thread class.

`#include <stage1_MailRetrievalThread.h>`

Inheritance diagram for argo::iridium::xfloat::XFloatMailRetrievalService:



Collaboration diagram for argo::iridium::xfloat::XFloatMailRetrievalService:



Public Member Functions

- XFloatMailRetrievalService (bool prop=(XFloatMailProcessingService::num_threads() > 0))

  The constructor.
- ~XFloatMailRetrievalService ()

  The destructor.
- void operator() () ATTR_NO_EXCEPTIONS

  Functor operator overload function: the main entry point for the mail retrieval thread.

Static Public Member Functions

- static void CloseSqlConnection ()

  Close the thread-specific MySQL connection if it is currently open.
- static shared_ptr< sql::ResultSet > **executeDeleteQuery** (const string &from, const string &where="")
- static shared_ptr< ResultSet > ExecuteQuery (Connection *con, const string &query)

  Execute an SQL query and return a shared pointer to the ResultSet object.
- static shared_ptr< sql::ResultSet > executeQuery (const string &query)

  Execute an SQL query and return a shared pointer to the ResultSet object.

- static shared_ptr< sql::ResultSet > executeSelectQuery (const string &what, const string &from, const string &where="", const string &groupBy="", const string &orderBy="", unsigned int limit=0, unsigned int offset=0)

  Execute an SQL select query and return a shared pointer to the ResultSet object.
- static int ExecuteUpdate (Connection *con, const string &update)

  Execute an SQL update and return the status code.
- static int executeUpdate (const string &update)

  Execute an SQL update and return the status code.
- static shared_ptr< PreparedStatement > GeneratePreparedStatement (Connection *con, const string &statement)

  Generate a prepared SQL statement object for the given SQL statement.
- static shared_ptr< sql::PreparedStatement > generatePreparedStatement (const string &statement)

  Generate a prepared SQL statement object for the given SQL statement.
- static string GenerateSqlDeleteStr (const string &from, const string &where="")

  Generate an SQL DELETE statement.
- static string GenerateSqlSelectStr (const string &what, const string &from, const string &where="", const string &groupBy="", const string &orderBy="", unsigned int limit=0, unsigned int offset=0)

  Generate an SQL select statement.
- static const path & GetDirectoryPath (const path &dir, bool create=false)

  Get a directory path.
- static path GetDirectoryPathFromString (const string &dir, bool create=false)

  Generate a boost::filesystem::path object from the directory path string.
- static thread_specific_ptr< Connection > & GetSqlConnection (string url, string user, string password, string schema)

  Get a thread-specific instance of a MySQL connection object.
- static thread_specific_ptr< MySQL_Driver > & GetSqlDriver ()

  Get a thread-specific instance of a MySQL driver object.
- static vector< path > ListDirectories (const path &dir)

  List all directories contained in the given directory.
- static vector< path > ListFiles (path dir, size_t n=0)

  List all regular files contained in the given directory.
- static path MoveFileToDirectory (const path &file, const path &dir_dst, bool create_dir=false)

  Copy a file into a directory.
- static void StartWorkerThread () ATTR_NO_EXCEPTIONS

  Instantiate the mail retrieval worker thread.
- static void TerminateWorkerThread () ATTR_NO_EXCEPTIONS

  Terminate the mail retrieval worker thread.

Static Protected Member Functions

- static sql::Connection * getSqlConnection ()

  Generate a thread-specific instance of a MySQL connection object.

Private Member Functions

- vmime::shared_ptr< vmime::net::store > ConnectToMailServer ()

  Connect to the mail server.
- void DisconnectFromMailServer () ATTR_NO_EXCEPTIONS

  Disconnect from the mail server.
- size_t FetchMailMessagesFromFileSystem ()

  Fetch a batch of messages from the file system.
- size_t FetchMailMessagesFromServer ()

  Fetch a batch of mail messages from the mail server.
- string GenerateMailServerUrlString () const

  Generate the mail server URL string.
- const vmime::shared_ptr< vmime::net::session > & mail_session ()

  Get the mail session object.
- const vmime::shared_ptr< vmime::net::folder > & OpenMailInboxFolder ()

  Open the inbox folder on the remote mail server.

Static Private Member Functions

- static void CleanupSqlConnection (Connection *con)

  Cleanup an SQL connection.
- static void CleanupSqlDriver (MySQL_Driver *driver)

  Function for memory cleanup of MySQL driver on destruct.
- static vector< path > ListRawMailFiles ()

  List the paths raw mail files on the local filesystem.
- static void set_termination_signaled (bool termination_signaled=true)

  Set the termination signaled flag.

Private Attributes

- vmime::shared_ptr< vmime::net::folder > inbox_folder_

  The VMime inbox folder object reference.
- vmime::shared_ptr< vmime::net::session > mail_session_

  The VMime session object reference.
- vmime::shared_ptr< vmime::net::store > mail_store_

  The VMime mail store object reference.
- bool processing_thread_propagation_enabled_

  Is propagation of mail messages to the mail processing thread enabled?

Static Private Attributes

- static string address_

  The SQL server qualified name or IP address string.
- static bool **is_reprocess_active_**
- static bool is_reprocess_all_

  The reprocess-all-mail-messages boolean flag.
- static int mail_fetch_mode_

  The mail fetch mode flag.
- static path **mail_file_input_directory_**
- static path **mail_file_processed_directory_**
- static string mail_server_host_

  The mail server host qualified name or IP address string.
- static string mail_server_inbox_folder_path_

  The mail server remote incoming mail message folder path string.
- static string mail_server_password_

  The mail server account password string.
- static string mail_server_protocol_

  The mail server protocol name string.
- static string mail_server_user_

  The mail server account username string.
- static size_t max_num_msgs_to_fetch_

  The maximum number of messages to simultaneously fetch from the mail server.
- static unsigned int message_start_index_

  The starting index of the sequence of messages to fetch.
- static bool move_mail_messages_

  Move mail messages flag.
- static string password_

  The SQL server account password string.
- static unsigned int port_

  The SQL server port number.
- static string protocol_

  The protocol to use in the SQL server connection e.g.
- static vector< unsigned int > reprocess_by_inst_ids_

  A vector containing the unique instrument identifier of messages to reprocess from the SQL database.
- static string schema_

  The SQL server schema name.
- static unsigned int sleep_seconds_if_no_new_msgs_

  The time to sleep if no new mail messages are fetched, in seconds.
- static thread_specific_ptr< Connection > sql_connection_

Tthe thread-specific pointer to MySQL connection object.

- static thread_specific_ptr< MySQL_Driver > sql_driver_

  The thread-specific pointer to MySQL driver object.
- static std::atomic< bool > termination_signaled_

  The termination signaled flag.
- static thread thread_

  The instance of the mail retrieval thread.
- static shared_ptr< XFloatMailRetrievalService > thread_object_

  The mail retrieval thread object.
- static string user_

  The SQL server account username string.

Friends

- options_description util::config::LocalConfiguration::GenerateCommandLineOptionsDescription ()

  Friend function for populating static members of this class from the command-line options.
- options_description util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription ()

  Friend function for populating some of the static members of this class from the configuration file options.

10.90.1 Detailed Description

The mail retrieval thread class.

This class defines the operation of the mail retrieval thread, which is responsible for fetching email messages for the mail server and/or SQL database, and queuing them for consumption by the next stage in the processing pipeline, XFloatMailProcessingService. The mail retrieval thread is the first stage in the processing pipeline.

A single instance of this thread will be instantiated after application initialization. The function of the thread is defined by functor operator overload function. The thread enters a primary loop in which mail messages are queried from the database and mail server. If no messages are returned, the thread disconnects from the mail server and database, and sleeps for a period of time before resuming the loop. The loop continues until termination is signaled by the daemon thread, at which point the thread will disconnect from the mail server and die.

All the credentials for connecting to the mail server are stored as static members of this class. These values are set as a result of parsing the command-line and/or configuration options during application initialization.

10.90.2 Constructor & Destructor Documentation

10.90.2.1 XFloatMailRetrievalService()

```
argo::iridium::xfloat::XFloatMailRetrievalService::XFloatMailRetrievalService (
            bool prop = (XFloatMailProcessingService::num_threads() > 0))
```

The constructor.

Parameters

in	*prop*	TRUE if propagation of mail messages to the mail processing thread is enabled, or FALSE otherwise.

10.90.2.2 ~XFloatMailRetrievalService()

```
argo::iridium::xfloat::XFloatMailRetrievalService::~XFloatMailRetrievalService ()
```

The destructor.

10.90.3 Member Function Documentation

10.90.3.1 CleanupSqlConnection()

```
static void util::mysql::MysqlUtils::CleanupSqlConnection (
            Connection * con)  [inline], [static], [private], [inherited]
```

Cleanup an SQL connection.

This function for memory cleanup of the SQL connection on destruction. The thread-specific SQL connection object will be deleted if it exists.

Parameters

in	*con*	The SQL connection to cleanup.

Exceptions

log::MessageException	if an error occurs while cleaning up the SQL connection object.

References CATCH_BLOCK.

10.90.3.2 CleanupSqlDriver()

```
static void util::mysql::MysqlUtils::CleanupSqlDriver (
            MySQL_Driver * driver)  [inline], [static], [private], [inherited]
```

Function for memory cleanup of MySQL driver on destruct.

Signals the SQL driver to end operation, if it exists.

Parameters

in	*driver*	Driver to cleanup.

Exceptions

log::MessageException	if an error occurs while cleaning up the SQL driver object.

References CATCH_BLOCK.

10.90.3.3 CloseSqlConnection()

```
static void util::mysql::MysqlUtils::CloseSqlConnection ()  [inline], [static], [inherited]
```

Close the thread-specific MySQL connection if it is currently open.

If the thread local MySQL connection is not currently open, then this function will have no effect.

Note

The MySQL connection should be closed when idle to prevent the connection from timing out and throwing exceptions.

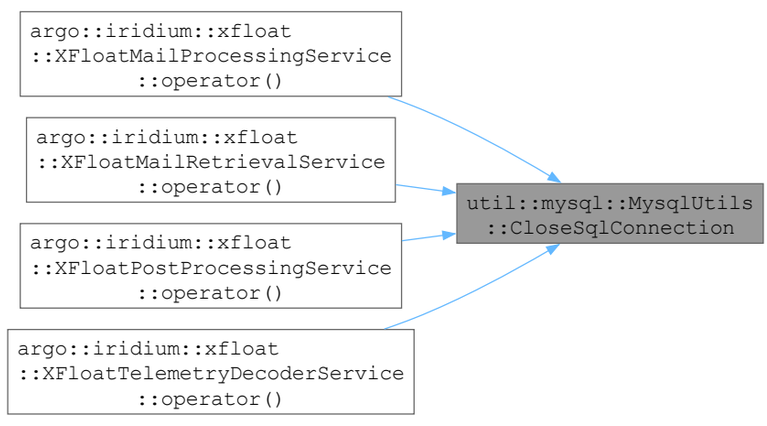
Exceptions

log::MessageException	if an error occurs while attempting to close the MySQL connection.

References CATCH_BLOCK, and util::mysql::MysqlUtils::sql_connection_.

Referenced by argo::iridium::xfloat::XFloatMailProcessingService::operator()(), operator()(), argo::iridium::xfloat::XFloatPostProcessingSe and argo::iridium::xfloat::XFloatTelemetryDecoderService::operator()().

Here is the caller graph for this function:



10.90.3.4 ConnectToMailServer()

```
vmime::shared_ptr< vmime::net::store > argo::iridium::xfloat::XFloatMailRetrievalService::↩
ConnectToMailServer () [private]
```

Connect to the mail server.

If the thread is not connected to the mail server i.e. mail_store_ is NULL, then this function creates the mail_store_ object from the mail server URL string and attempts to connect to the mail server. If the thread is already connected to the mail server, then this function will have no effect.

Exceptions

log::MessageException	if the function fails to connect to the mail server.

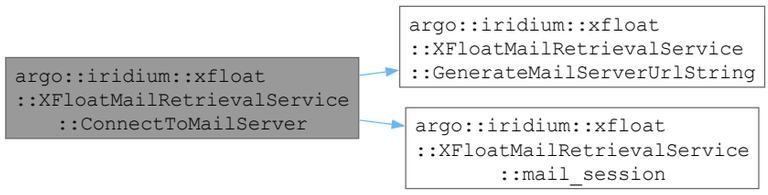
Returns

A shared reference to mail_store_.

References CATCH_BLOCK, GenerateMailServerUrlString(), ldbg, mail_server_host_, mail_session(), and mail_store_.

Referenced by OpenMailInboxFolder().

Here is the call graph for this function:



Here is the caller graph for this function:

10.90.3.5 DisconnectFromMailServer()

```
void argo::iridium::xfloat::XFloatMailRetrievalService::DisconnectFromMailServer () [private]
```

Disconnect from the mail server.

This function disconnects from the mail server and releases the reference to the VMime objects. If the thread is already disconnected from the mail server, then this function will have no effect. Otherwise, the inbox_folder_ object is closed and released. Then, the mail_store_ object is disconnected and released. Finally, the mail_session_ object is released.

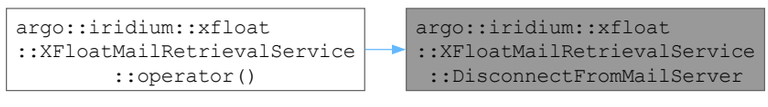
Exceptions

log::MessageException	if an error occurs while attempting to disconnect from the mail server.

References CATCH_BLOCK, CATCH_NOTHROW, inbox_folder_, ldbg, mail_session_, and mail_store_.

Referenced by operator()().

Here is the caller graph for this function:



10.90.3.6 ExecuteQuery()

```
static shared_ptr< ResultSet > util::mysql::MysqlUtils::ExecuteQuery (
            Connection * con,
            const string & query)  [inline], [static], [inherited]
```

Execute an SQL query and return a shared pointer to the ResultSet object.

Parameters

in	*con*	A pointer to the SQL connection object. This must not be NULL.
in	*query*	The SQL query to execute. This must be a well-formatted SQL query string.

Exceptions

log::MessageException	if 'con' is null, 'query' is a malfored SQL string, or any other error occurs.

Returns

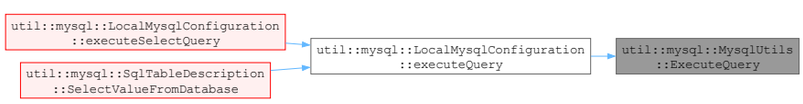
A shared pointer to the query ResultSet object.

References CATCH_BLOCK, and util::mysql::MysqlUtils::GeneratePreparedStatement().

Referenced by util::mysql::LocalMysqlConfiguration::executeQuery().

Here is the call graph for this function:

Here is the caller graph for this function:



10.90.3.7 executeQuery()

```
shared_ptr< sql::ResultSet > util::mysql::LocalMysqlConfiguration::executeQuery (
            const string & query)  [static], [inherited]
```

Execute an SQL query and return a shared pointer to the ResultSet object.

Parameters

in	*query*	The SQL query to execute. This must be a well-formatted SQL query string.

Exceptions

log::MessageException	'query' is a malfored SQL string or any other error occurs while executing the query.

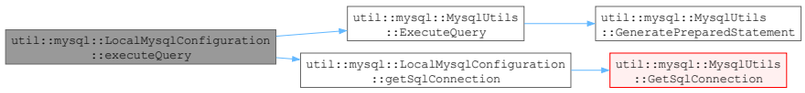
Returns

A shared pointer to the query ResultSet object.

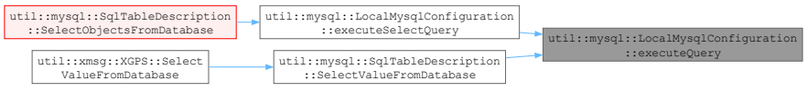
References util::mysql::MysqlUtils::ExecuteQuery(), and util::mysql::LocalMysqlConfiguration::getSqlConnection().

Referenced by util::mysql::LocalMysqlConfiguration::executeSelectQuery(), and util::mysql::SqlTableDescription< T >::SelectValueFromD

Here is the call graph for this function:



Here is the caller graph for this function:



10.90.3.8 executeSelectQuery()

```
shared_ptr< sql::ResultSet > util::mysql::LocalMysqlConfiguration::executeSelectQuery (
            const string & what,
            const string & from,
            const string & where = "",
            const string & groupBy = "",
            const string & orderBy = "",
            unsigned int limit = 0,
            unsigned int offset = 0)  [static], [inherited]
```

Execute an SQL select query and return a shared pointer to the ResultSet object.

Parameters

in	*what*	The SQL select statement select-expression clause.
in	*from*	The SQL select statement table–references clause.
in	*where*	The SQL select statement where-condition clause.
in	*groupBy*	The SQL select statement group-by clause.
in	*orderBy*	The SQL select statement order-by clause.
in	*limit*	The SQL select statement result set row limit value.
in	*offset*	The SQL select statement result set row offset.

Exceptions

log::MessageException	if an error occurs while executing the query.

Returns

A shared pointer to the ResultSet object.

References CATCH_BLOCK, util::mysql::LocalMysqlConfiguration::executeQuery(), and util::mysql::MysqlUtils::GenerateSqlSelectStr().

Referenced by util::mysql::SqlTableDescription< T >::SelectObjectsFromDatabase().

Here is the call graph for this function:

Here is the caller graph for this function:

10.90.3.9 ExecuteUpdate()

```
static int util::mysql::MysqlUtils::ExecuteUpdate (
            Connection * con,
            const string & update)  [inline], [static], [inherited]
```

Execute an SQL update and return the status code.

Parameters

in	*con*	A pointer to the SQL connection object. This must not be NULL.
in	*update*	The SQL update to execute. This must be a well-formatted SQL update string.

Exceptions

log::MessageException	if 'con' is null, 'update' is a malformed SQL string, or any other error occurs.

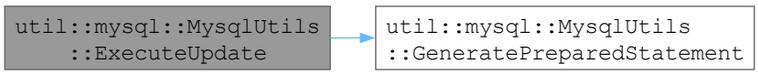
Returns

The update status code.

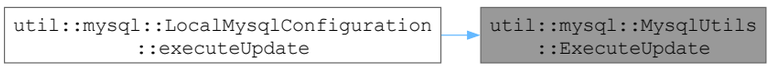
References CATCH_BLOCK, and util::mysql::MysqlUtils::GeneratePreparedStatement().

Referenced by util::mysql::LocalMysqlConfiguration::executeUpdate().

Here is the call graph for this function:



Here is the caller graph for this function:



10.90.3.10 executeUpdate()

```
int util::mysql::LocalMysqlConfiguration::executeUpdate (
            const string & update)  [static], [inherited]
```

Execute an SQL update and return the status code.

Parameters

in	*update*	The SQL update to execute. This must be a well-formatted SQL update string.

Exceptions

log::MessageException	if 'update' is a malformed SQL string, or any other error occurs while executing the update.

Returns

The update status code.

References util::mysql::MysqlUtils::ExecuteUpdate(), and util::mysql::LocalMysqlConfiguration::getSqlConnection().

Here is the call graph for this function:

10.90.3.11 FetchMailMessagesFromFileSystem()

```
size_t argo::iridium::xfloat::XFloatMailRetrievalService::FetchMailMessagesFromFileSystem ()  [private]
```

Fetch a batch of messages from the file system.

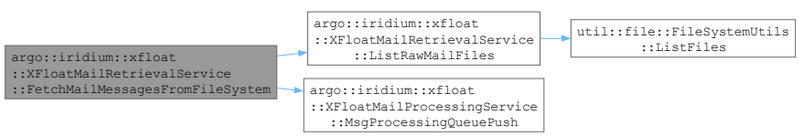
Returns

The number of messages fetched.

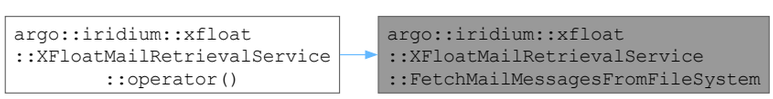
References CATCH_BLOCK, CATCH_NOTHROW, EXC_THROW, ldbg, ListRawMailFiles(), lwrn, max_num_msgs_to_fetch_, move_mail_messages_, and argo::iridium::xfloat::XFloatMailProcessingService::MsgProcessingQueuePush().

Referenced by operator()().

Here is the call graph for this function:



Here is the caller graph for this function:



10.90.3.12 FetchMailMessagesFromServer()

```
size_t argo::iridium::xfloat::XFloatMailRetrievalService::FetchMailMessagesFromServer ()  [private]
```

Fetch a batch of mail messages from the mail server.

Connects to the mail server and fetches a batch of mail messages. Fetched messages are pushed into the mail processing queue.

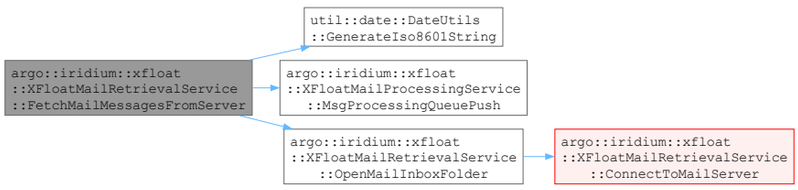
Returns

The number of messages fetched.

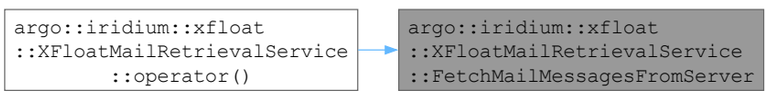
References CATCH_BLOCK, CATCH_NOTHROW, EXC_THROW, util::date::DateUtils::GenerateIso8601String(), ldbg, lout, max_num_msgs_to_fetch_, message_start_index_, move_mail_messages_, argo::iridium::xfloat::XFloatMailProcessingService::Msg and OpenMailInboxFolder().

Referenced by operator()().

Here is the call graph for this function:



Here is the caller graph for this function:



10.90.3.13 GenerateMailServerUrlString()

```
string argo::iridium::xfloat::XFloatMailRetrievalService::GenerateMailServerUrlString () const
[private]
```

Generate the mail server URL string.

The mail server URL string is generated from the static member fields that are set as a result of parsing the configuration file and/or command-line arguments during application initialization.

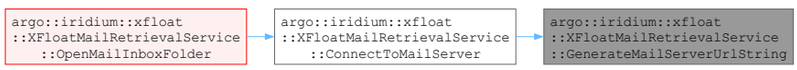
Returns

The generated mail server URL string.

References mail_server_host_, mail_server_password_, mail_server_protocol_, and mail_server_user_.

Referenced by ConnectToMailServer().

Here is the caller graph for this function:



10.90.3.14 GeneratePreparedStatement()

```
static shared_ptr< PreparedStatement > util::mysql::MysqlUtils::GeneratePreparedStatement (
            Connection * con,
            const string & statement)  [inline], [static], [inherited]
```

Generate a prepared SQL statement object for the given SQL statement.

Parameters

in	*con*	A pointer to the SQL connection object. This must not be NULL.
in	*statement*	The SQL statement to prepare. This must be a well-formed SQL string.

Exceptions

log::MessageException	if 'con' is NULL, 'statement' is not a valid SQL string, or if an error occurs.

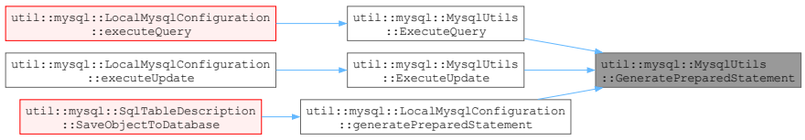
Returns

A prepared SQL statement object generated from the given SQL statement.

References CATCH_BLOCK, and EXC_THROW.

Referenced by util::mysql::MysqlUtils::ExecuteQuery(), util::mysql::MysqlUtils::ExecuteUpdate(), and util::mysql::LocalMysqlConfiguration:

Here is the caller graph for this function:



10.90.3.15 generatePreparedStatement()

```
std::shared_ptr< sql::PreparedStatement > util::mysql::LocalMysqlConfiguration::generatePrepared↩
Statement (
            const string & statement)  [static], [inherited]
```

Generate a prepared SQL statement object for the given SQL statement.

Parameters

in	*statement*	The SQL statement to prepare. This must be a well-formed SQL string.

Exceptions

log::MessageException	if 'statement' is not a valid SQL string or if an error occurs while generating the prepared statement object..

Returns

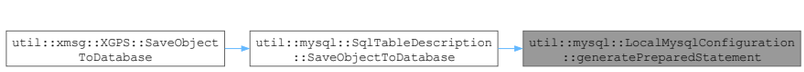
A prepared SQL statement object generated from the given SQL statement.

References util::mysql::MysqlUtils::GeneratePreparedStatement(), and util::mysql::LocalMysqlConfiguration::getSqlConnection().

Referenced by util::mysql::SqlTableDescription< T >::SaveObjectToDatabase().

Here is the call graph for this function:

Here is the caller graph for this function:



10.90.3.16 GenerateSqlDeleteStr()

```
static string util::mysql::MysqlUtils::GenerateSqlDeleteStr (
            const string & from,
            const string & where = "")  [inline], [static], [inherited]
```

Generate an SQL DELETE statement.

Warning

Without a `where` clause, the statement deletes every row in the table.

Parameters

in	*from*	The name of the table to delete from.
in	*where*	The SQL "where-condition" clause (empty = all rows).

Returns

The SQL statement.

10.90.3.17 GenerateSqlSelectStr()

```
static string util::mysql::MysqlUtils::GenerateSqlSelectStr (
            const string & what,
            const string & from,
            const string & where = "",
            const string & groupBy = "",
            const string & orderBy = "",
            unsigned int limit = 0,
            unsigned int offset = 0)  [inline], [static], [inherited]
```

Generate an SQL select statement.

Parameters

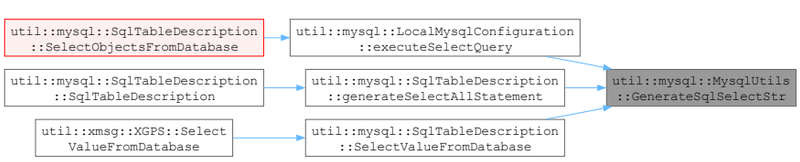
in	*what*	The SQL select statement select-expression clause.
in	*from*	The SQL select statement table-references clause.
in	*where*	The SQL select statement where-condition clause.
in	*groupBy*	The SQL select statement group-by clause.
in	*orderBy*	The SQL select statement order-by clause.
in	*limit*	The SQL select statement result set row limit value.
in	*offset*	The SQL select statement result set row offset.

Returns

The SQL select statement.

Referenced by util::mysql::LocalMysqlConfiguration::executeSelectQuery(), util::mysql::SqlTableDescription< T >::generateSelectAllState
and util::mysql::SqlTableDescription< T >::SelectValueFromDatabase().

Here is the caller graph for this function:



10.90.3.18 GetDirectoryPath()

```
static const path & util::file::FileSystemUtils::GetDirectoryPath (
            const path & dir,
            bool create = false)  [inline], [static], [inherited]
```

Get a directory path.

Exceptions

MessageException	if create == FALSE and the directory specified by path does not exist.

Parameters

in	*dir*	Directory path.
in	*create*	If TRUE, the directory specified by path will be created if it does not already exists. If FALSE, an exception will be thrown if the directory does not exist.

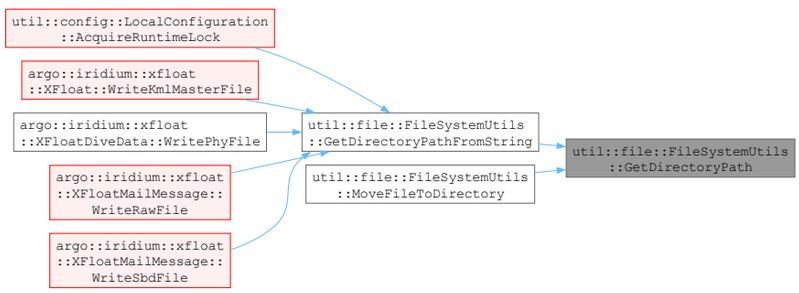
Returns

dir

References CATCH_BLOCK, EXC_THROW, and ldbg.

Referenced by util::file::FileSystemUtils::GetDirectoryPathFromString(), and util::file::FileSystemUtils::MoveFileToDirectory().

Here is the caller graph for this function:



10.90.3.19 GetDirectoryPathFromString()

```
static path util::file::FileSystemUtils::GetDirectoryPathFromString (
            const string & dir,
            bool create = false)  [inline], [static], [inherited]
```

Generate a boost::filesystem::path object from the directory path string.

Exceptions

MessageException	if create == FALSE and the directory specified by path does not exist.

Parameters

in	*dir*	Directory path string.
in	*create*	If TRUE, the directory specified by path will be created if it does not already exists. If FALSE, an exception will be thrown if the directory does not exist.

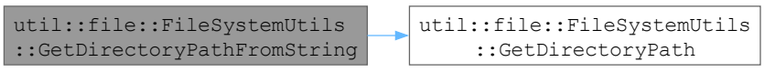
Returns

A boost::filesystem::path object from the directory path string.

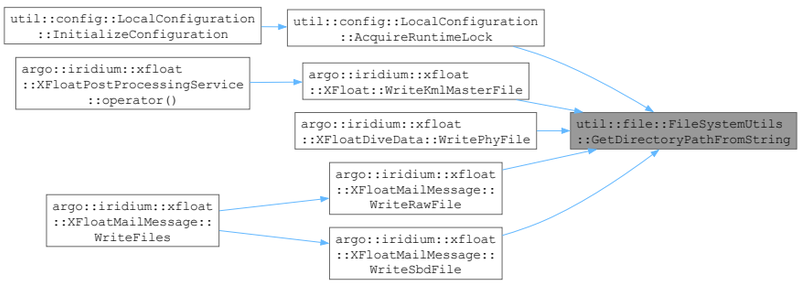
References CATCH_BLOCK, and util::file::FileSystemUtils::GetDirectoryPath().

Referenced by util::config::LocalConfiguration::AcquireRuntimeLock(), argo::iridium::xfloat::XFloat::WriteKmlMasterFile(), argo::iridium::xfloat::XFloatDiveData::WritePhyFile(), argo::iridium::xfloat::XFloatMailMessage::WriteRawFile(), and argo::iridium::xfloat::XFloatMailMessage::WriteSbdFile().

Here is the call graph for this function:



Here is the caller graph for this function:



10.90.3.20 GetSqlConnection()

```
static thread_specific_ptr< Connection > & util::mysql::MysqlUtils::GetSqlConnection (
            string url,
            string user,
            string password,
            string schema)  [inline], [static], [inherited]
```

Get a thread-specific instance of a MySQL connection object.

A new connection will be created, if necessary.

Parameters

in	*url*	The MySQL host server qualified address.
in	*user*	The MySQL account username.
in	*password*	The MySQL account password.
in	*schema*	The MySQL account schema name.

Exceptions

log::MessageException	if an error occurs while creating a new thread-specific instance of the SQL connection object

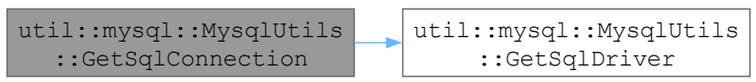
Returns

A thread-specific instance of a MySQL connection object.

References CATCH_BLOCK, util::mysql::MysqlUtils::GetSqlDriver(), and util::mysql::MysqlUtils::sql_connection_.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

Here is the call graph for this function:



Here is the caller graph for this function:

10.90.3.21 getSqlConnection()

`sql::Connection * util::mysql::LocalMysqlConfiguration::getSqlConnection () [static], [protected], [inherited]`

Generate a thread-specific instance of a MySQL connection object.

A new connection will be created, if necessary.

Exceptions

log::MessageException	if an error occurs while generating the SQL connection object.

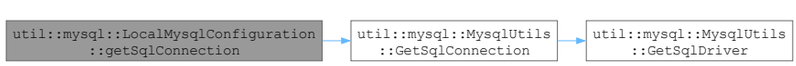
Returns

A thread-specific instance of a MySQL connection object.

References util::mysql::LocalMysqlConfiguration::address_, CATCH_BLOCK, util::mysql::MysqlUtils::GetSqlConnection(), util::mysql::LocalMysqlConfiguration::password_, util::mysql::LocalMysqlConfiguration::port_, util::mysql::LocalMysqlConfiguration::protoc util::mysql::LocalMysqlConfiguration::schema_, and util::mysql::LocalMysqlConfiguration::user_.

Referenced by util::mysql::LocalMysqlConfiguration::executeQuery(), util::mysql::LocalMysqlConfiguration::executeUpdate(), and util::mysql::LocalMysqlConfiguration::generatePreparedStatement().

Here is the call graph for this function:



Here is the caller graph for this function:

10.90.3.22 GetSqlDriver()

```
static thread_specific_ptr< MySQL_Driver > & util::mysql::MysqlUtils::GetSqlDriver ()  [inline],
[static], [inherited]
```

Get a thread-specific instance of a MySQL driver object.

A new driver will created, if necessary.

Exceptions

log::MessageException	if an error occurs while creating a new thread-specific instance of the SQL driver object

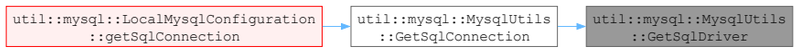
Returns

A thread-specific instance of a MySQL driver object.

References CATCH_BLOCK, and util::mysql::MysqlUtils::sql_driver_.

Referenced by util::mysql::MysqlUtils::GetSqlConnection().

Here is the caller graph for this function:



10.90.3.23 ListDirectories()

```
static vector< path > util::file::FileSystemUtils::ListDirectories (
            const path & dir)  [inline], [static], [inherited]
```

List all directories contained in the given directory.

Parameters

in	*dir*	The path to the directory of interest.

Exceptions

log::MessageException	if the directory does not exist, is not a directory, or an error occurs.

Returns

A vector of paths to all the directories contained in the given directory.

References CATCH_BLOCK, and EXC_THROW.

10.90.3.24 ListFiles()

```
static vector< path > util::file::FileSystemUtils::ListFiles (
            path dir,
            size_t n = 0)  [inline], [static], [inherited]
```

List all regular files contained in the given directory.

Parameters

in	*dir*	The path to the directory of interest.
in	*n*	If n>0, the maximum number of paths to return, else no effect.

Exceptions

log::MessageException	if the directory does not exist, is not a directory, or an error occurs.

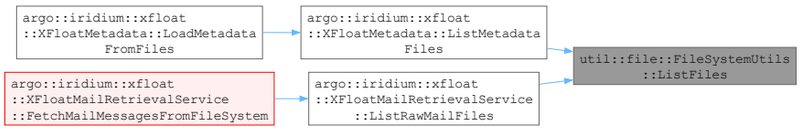
Returns

A vector of paths to all the regular files contained in the given directory.

References CATCH_BLOCK, and EXC_THROW.

Referenced by argo::iridium::xfloat::XFloatMetadata::ListMetadataFiles(), and ListRawMailFiles().

Here is the caller graph for this function:



10.90.3.25 ListRawMailFiles()

`vector< path > argo::iridium::xfloat::XFloatMailRetrievalService::ListRawMailFiles () [static], [private]`

List the paths raw mail files on the local filesystem.

Exceptions

util::log::MessageException	if an error occurs while listing the raw mail paths.

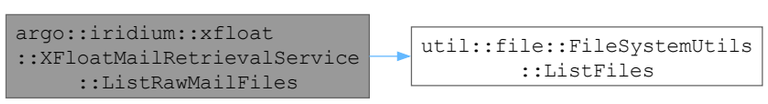
Returns

A vector containing the paths to the raw mail files on the local filesystem.

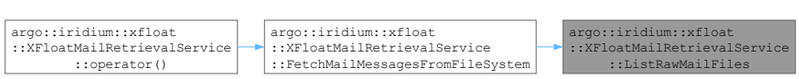
References CATCH_BLOCK, util::file::FileSystemUtils::ListFiles(), and max_num_msgs_to_fetch_.

Referenced by FetchMailMessagesFromFileSystem().

Here is the call graph for this function:



Here is the caller graph for this function:



10.90.3.26 mail_session()

```
const vmime::shared_ptr< vmime::net::session > & argo::iridium::xfloat::XFloatMailRetrieval↩
Service::mail_session ()  [private]
```

Get the mail session object.

If the mail session does not exist, a new one will be created.

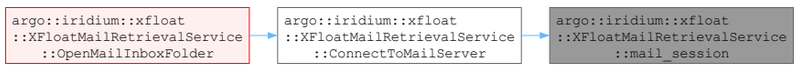
Returns

A shared reference to the VMime session object, mail_session_.

References mail_session_.

Referenced by ConnectToMailServer().

Here is the caller graph for this function:



10.90.3.27 MoveFileToDirectory()

```
static path util::file::FileSystemUtils::MoveFileToDirectory (
            const path & file,
            const path & dir_dst,
            bool create_dir = false)  [inline], [static], [inherited]
```

Copy a file into a directory.

Warning

Despite its name, this function copies the file and does not remove the source. If a file with the same name already exists in the destination directory, it is left unchanged. It is not currently called.

Parameters

in	*file*	The file to copy.
in	*dir_dst*	The destination directory.
in	*create_dir*	If TRUE, create the destination directory if it does not exist.

Exceptions

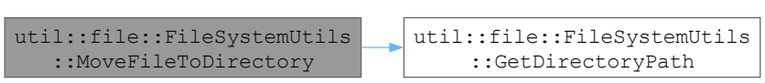
util::log::MessageException	if the file does not exist, is not a regular file, or cannot be copied.

Returns

The path of the file in the destination directory.

References CATCH_BLOCK, EXC_THROW, and util::file::FileSystemUtils::GetDirectoryPath().

Here is the call graph for this function:



10.90.3.28 OpenMailInboxFolder()

```
const vmime::shared_ptr< vmime::net::folder > & argo::iridium::xfloat::XFloatMailRetrieval↩
Service::OpenMailInboxFolder ()  [private]
```

Open the inbox folder on the remote mail server.

If the inbox folder is not already open, then this function will attempt to open it. The path to the inbox folder is specified by the static member string mail_server_inbox_folder_path_, which is set as a result of parsing the configuration file and/or command-line arguments during application initialization. If the string mail_server_inbox_folder_path_ is empty, then the default mail folder will be requested from the mail server.

Exceptions

util::log::MessageException	of an error occurs while attempting to open the inbox folder.

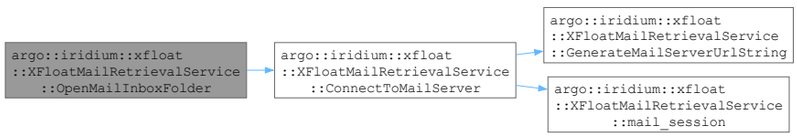
Returns

A shared reference to the VMime folder object, inbox_folder_.

References CATCH_BLOCK, ConnectToMailServer(), dbg1, inbox_folder_, and mail_server_inbox_folder_path_.

Referenced by FetchMailMessagesFromServer().

Here is the call graph for this function:



Here is the caller graph for this function:

10.90.3.29 operator()()

```
void argo::iridium::xfloat::XFloatMailRetrievalService::operator() ()
```

Functor operator overload function: the main entry point for the mail retrieval thread.

This function defines the behavior of the mail retrieval thread, which is responsible for retrieving messages from the mail server. After initialization, a single instance of this thread will be instantiated. The static configuration options for this thread are set as a result of parsing the command-line arguments and/or configuration file.

First, the scope thread attribute "ThreadName" is set to "mail-fetch". This attribute is used by the global logger object for filtering, etc. Next, the instruments whose messages are to be reprocessed are determined: all cached instruments if is_reprocess_all_ is TRUE; otherwise, those listed in reprocess_by_inst_ids_ and those with PHY generation enabled, plus, if `is_reprocess_active_` is TRUE, those that reported in the last 90 days.

For each instrument to reprocess, its existing dive data is deleted from the SQL database, its stored mail messages are fetched from the SQL database and passed on to the mail processing thread. Then, the thread will sleep for a short duration before proceeding to allow time for the daemon thread to signal termination if the application is configured to terminate after reprocessing.

Then the thread enters its primary loop, which depends on mail_fetch_mode_. In `MAIL_FETCH_SERVER` mode, FetchMailMessagesFromServer() fetches the next batch of messages from the mail server, by message index starting at message_start_index_. In `MAIL_FETCH_FILESYSTEM` mode, FetchMailMessagesFromFileSystem() reads RAW mail files from `mail_file_input_directory_`. The fetched messages are queued to be processed by the next thread in the pipeline, XFloatMailProcessingService. If no new messages are fetched, then the connections to the mail server and SQL database are closed and the thread sleeps for sleep_seconds_if_no_new_msgs_ seconds.

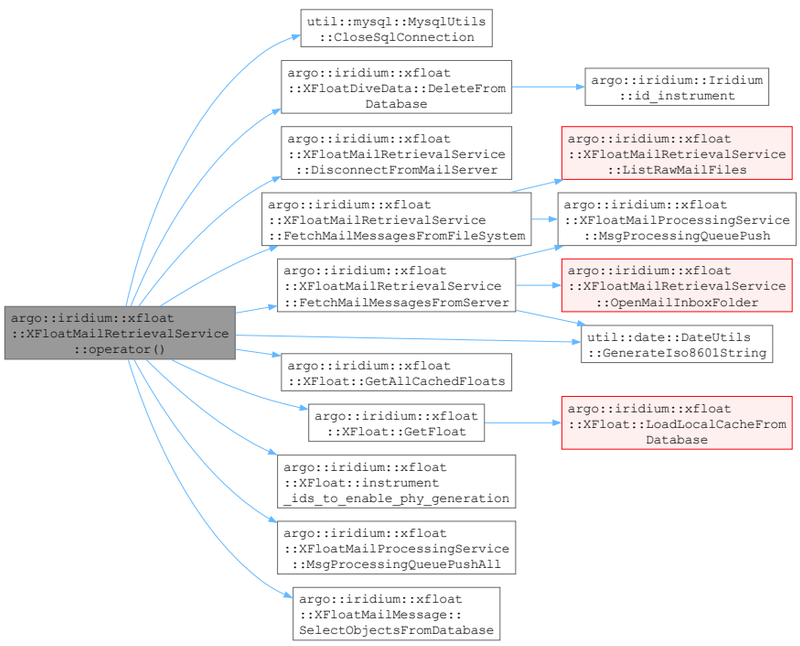
The primary loop terminates once termination is signaled (i.e. termination_signaled_ is TRUE) and the thread will disconnect from the mail server and then die. If termination is signaled before the thread begins its first iteration, then exactly zero iteration of the loop will be executed.

Exceptions

None.	Never throws.

References CATCH_NOTHROW, util::mysql::MysqlUtils::CloseSqlConnection(), argo::iridium::xfloat::XFloatDiveData::DeleteFromDatabas DisconnectFromMailServer(), FetchMailMessagesFromFileSystem(), FetchMailMessagesFromServer(), util::date::DateUtils::GenerateIso8 argo::iridium::xfloat::XFloat::GetAllCachedFloats(), argo::iridium::xfloat::XFloat::GetFloat(), argo::iridium::xfloat::XFloat::instrument_ids_to is_reprocess_all_, ldbg, linf, lout, mail_fetch_mode_, argo::iridium::xfloat::XFloatMailProcessingService::MsgProcessingQueuePushAll(), reprocess_by_inst_ids_, argo::iridium::xfloat::XFloatMailMessage::SelectObjectsFromDatabase(), sleep_seconds_if_no_new_msgs_, termination_signaled_, and XFLT_THREAD_SLEEP_MS.

Here is the call graph for this function:



10.90.3.30 set_termination_signaled()

```
void argo::iridium::xfloat::XFloatMailRetrievalService::set_termination_signaled (
            bool termination_signaled = true)  [static], [private]
```

Set the termination signaled flag.

This function is called by TerminateWorkerThreads() in order to cause all currently running instances of the mail retrieval thread to exit their primary loop and die.

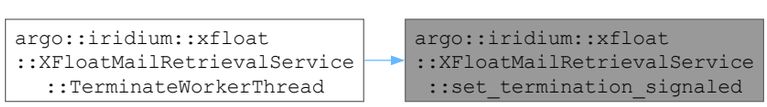
Parameters

in	*termination_signaled*	The termination signaled value to set.

References termination_signaled_.

Referenced by TerminateWorkerThread().

Here is the caller graph for this function:



10.90.3.31 StartWorkerThread()

```
void argo::iridium::xfloat::XFloatMailRetrievalService::StartWorkerThread ()  [static]
```

Instantiate the mail retrieval worker thread.

This function is invoked by the daemon thread after initialization to instantiate the mail retrieval thread. It creates a new instance of XFloatMailRetrievalService, and instantiates a new thread that invokes the functor operator of the object.

Exceptions

None.	Never throws.

References CATCH_BLOCK, ldbg, thread_, and thread_object_.

10.90.3.32 TerminateWorkerThread()

```
void argo::iridium::xfloat::XFloatMailRetrievalService::TerminateWorkerThread ()  [static]
```

Terminate the mail retrieval worker thread.

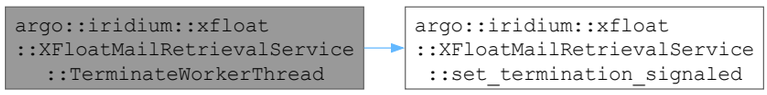
This function is used by the daemon thread to kill the worker thread upon application termination. The function signals termination, and then attempts to join the child thread. If the child thread does not join, then the function will timeout after XMTPS_THREAD_JOIN_SECONDS seconds, a warning will be issued, and the child thread will be abandoned.

Exceptions

None.	Never throws.

References CATCH_BLOCK, ldbg, lwrn, set_termination_signaled(), thread_, and XMTPS_THREAD_JOIN_SECONDS.

Here is the call graph for this function:



10.90.4 Friends And Related Symbol Documentation

10.90.4.1 util::config::LocalConfiguration::GenerateCommandLineOptionsDescription

```
options_description util::config::LocalConfiguration::GenerateCommandLineOptionsDescription ()
[friend]
```

Friend function for populating static members of this class from the command-line options.

10.90.4.2 util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription

```
options_description util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription
()  [friend]
```

Friend function for populating some of the static members of this class from the configuration file options.

10.90.5 Member Data Documentation

10.90.5.1 address_

```
std::string util::mysql::LocalMysqlConfiguration::address_  [static], [private], [inherited]
```

The SQL server qualified name or IP address string.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.90.5.2 inbox_folder_

vmime::shared_ptr<vmime::net::folder> argo::iridium::xfloat::XFloatMailRetrievalService::inbox_folder_ [private]

The VMime inbox folder object reference.

This object represents the folder on the mail server that contains incoming messages. Typically, this folder is "INBOX".

Referenced by DisconnectFromMailServer(), and OpenMailInboxFolder().

10.90.5.3 is_reprocess_all_

bool argo::iridium::xfloat::XFloatMailRetrievalService::is_reprocess_all_ [static], [private]

The reprocess-all-mail-messages boolean flag.

The value of this field is set as a result of parsing the command-line options. If unspecified in the command-line arguments, this field defaults to a the value CFG_DEFAULT_REPROCESS_ALL.

The value of this boolean flag determines which messages are retrieved from the mail server upon startup of the application. If TRUE, then mail messages will be fetched from mail server starting with UID 0. Otherwise, if FALSE, the starting UID of mail messages to be fetched from the server will be determined by querying the maximum UID of the mail messages stored in the SQL database. Mail message will be fetched from the mail server, starting with the UID queried from the database plus one.

Referenced by operator()().

10.90.5.4 mail_fetch_mode_

int argo::iridium::xfloat::XFloatMailRetrievalService::mail_fetch_mode_ [static], [private]

The mail fetch mode flag.

- 0: Fetch mail from mail server.
- 1: Fetch mail from file system.

Referenced by operator()().

10.90.5.5 mail_server_host_

string argo::iridium::xfloat::XFloatMailRetrievalService::mail_server_host_ [static], [private]

The mail server host qualified name or IP address string.

The value of this field is set as a result of parsing the command-line options. If unspecified, it defaults to "maila.whoi.edu".

Referenced by ConnectToMailServer(), and GenerateMailServerUrlString().

10.90.5.6 mail_server_inbox_folder_path_

string argo::iridium::xfloat::XFloatMailRetrievalService::mail_server_inbox_folder_path_ [static], [private]

The mail server remote incoming mail message folder path string.

The value of this field is set as a result of parsing the command-line options. If unspecified, it defaults to "INBOX".

Referenced by OpenMailInboxFolder().

10.90.5.7 mail_server_password_

string argo::iridium::xfloat::XFloatMailRetrievalService::mail_server_password_ [static], [private]

The mail server account password string.

The value of this field is set as a result of parsing the command-line options.

Referenced by GenerateMailServerUrlString().

10.90.5.8 mail_server_protocol_

string argo::iridium::xfloat::XFloatMailRetrievalService::mail_server_protocol_ [static], [private]

The mail server protocol name string.

The value of this field is set as a result of parsing the command-line options. If unspecified, it defaults to "imap".

Referenced by GenerateMailServerUrlString().

10.90.5.9 mail_server_user_

string argo::iridium::xfloat::XFloatMailRetrievalService::mail_server_user_ [static], [private]

The mail server account username string.

The value of this field is set as a result of parsing the command-line options. If unspecified, it defaults to "imap".

Referenced by GenerateMailServerUrlString().

10.90.5.10 mail_session_

vmime::shared_ptr<vmime::net::session> argo::iridium::xfloat::XFloatMailRetrievalService::mail_↩session_ [private]

The VMime session object reference.

This object is used to connect to the mail server, and create the mail_store_ object. It is created when the thread connects to the mail server and released when the thread disconnects from the mail server.

Referenced by DisconnectFromMailServer(), and mail_session().

10.90.5.11 mail_store_

vmime::shared_ptr<vmime::net::store> argo::iridium::xfloat::XFloatMailRetrievalService::mail_↩store_ [private]

The VMime mail store object reference.

This object encapsulates the protocols that provide access to the mail drop. It is created when the thread connects to the mail server and released when the thread disconnects from the mail server.

Referenced by ConnectToMailServer(), and DisconnectFromMailServer().

10.90.5.12 max_num_msgs_to_fetch_

size_t argo::iridium::xfloat::XFloatMailRetrievalService::max_num_msgs_to_fetch_ [static], [private]

The maximum number of messages to simultaneously fetch from the mail server.

The value of this field is set as a result of parsing the command-line options. If unspecified, it defaults to CFG_DEFAULT_MAX_FETCH.

Referenced by FetchMailMessagesFromFileSystem(), FetchMailMessagesFromServer(), and ListRawMailFiles().

10.90.5.13 message_start_index_

```
unsigned int argo::iridium::xfloat::XFloatMailRetrievalService::message_start_index_ [static],
[private]
```

The starting index of the sequence of messages to fetch.

If messages are moved out of the INBOX folder after being processed, then the messages sequence start number will always be one. Otherwise, this field will contain the starting index of the first message not fetched from the mail server.

Referenced by FetchMailMessagesFromServer().

10.90.5.14 move_mail_messages_

```
bool argo::iridium::xfloat::XFloatMailRetrievalService::move_mail_messages_ [static], [private]
```

Move mail messages flag.

If set, then the mail messages will be moved out of the inbox folder after being processed.

Referenced by FetchMailMessagesFromFileSystem(), and FetchMailMessagesFromServer().

10.90.5.15 password_

```
std::string util::mysql::LocalMysqlConfiguration::password_ [static], [private], [inherited]
```

The SQL server account password string.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.90.5.16 port_

```
unsigned int util::mysql::LocalMysqlConfiguration::port_ [static], [private], [inherited]
```

The SQL server port number.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.90.5.17 processing_thread_propagation_enabled_

```
bool argo::iridium::xfloat::XFloatMailRetrievalService::processing_thread_propagation_enabled_↩
[private]
```

Is propagation of mail messages to the mail processing thread enabled?

This field determines whether fetched messages are queued for consumption by the next thread in the processing pipeline, XFloatMailProcessingService. If no instances of that thread are running, then this field will be FALSE and message will be discarded rather than queued. Otherwise, this field will be TRUE and messages will be queued for consumption by XFloatMailProcessingService.

10.90.5.18 protocol_

```
std::string util::mysql::LocalMysqlConfiguration::protocol_ [static], [private], [inherited]
```

The protocol to use in the SQL server connection e.g.

"tcp".

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.90.5.19 reprocess_by_inst_ids_

```
vector< unsigned int > argo::iridium::xfloat::XFloatMailRetrievalService::reprocess_by_inst_ids↩
_ [static], [private]
```

A vector containing the unique instrument identifier of messages to reprocess from the SQL database.

If the user specifies one or more instruments to reprocess by specifying their unique instrument identifiers, then those identifiers will be contained in this vector. Otherwise, this vector will be empty.

After the thread is instantiated, it will check the size of this vector. If the vector size is non-zero, then the thread will query the SQL database for messages with instrument identifiers matching those contained in this vector. Those messages will then be passed on to the mail processing thread.

Regardless of the size of this vector, the mail retrieval thread will then continue on to enter its primary loop until termination is signaled.

Referenced by operator()().

10.90.5.20 schema_

```
std::string util::mysql::LocalMysqlConfiguration::schema_ [static], [private], [inherited]
```

The SQL server schema name.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.90.5.21 sleep_seconds_if_no_new_msgs_

```
unsigned int argo::iridium::xfloat::XFloatMailRetrievalService::sleep_seconds_if_no_new_msgs_↩
[static], [private]
```

The time to sleep if no new mail messages are fetched, in seconds.

If no new mail messages are fetched, the mail retrieval thread will sleep for this period of time, in increments of XFLT_THREAD_SLEEP_MS.

The value of this field is set as a result of parsing the command-line options. If unspecified, it defaults to CFG_DEFAULT_MAIL_CHK_DLY.

Referenced by operator()().

10.90.5.22 sql_connection_

```
thread_specific_ptr<Connection> util::mysql::MysqlUtils::sql_connection_ [static], [private],
[inherited]
```

Tthe thread-specific pointer to MySQL connection object.

Thread-specific MySQL connection, released by `MysqlUtils::CleanupSqlConnection()` when the thread exits.

Referenced by util::mysql::MysqlUtils::CloseSqlConnection(), and util::mysql::MysqlUtils::GetSqlConnection().

10.90.5.23 sql_driver_

```
thread_specific_ptr<MySQL_Driver> util::mysql::MysqlUtils::sql_driver_ [static], [private],
[inherited]
```

The thread-specific pointer to MySQL driver object.

Thread-specific reference to the MySQL driver.

Its cleanup function, `MysqlUtils::CleanupSqlDriver()`, does not delete the shared driver instance.

Referenced by util::mysql::MysqlUtils::GetSqlDriver().

10.90.5.24 termination_signaled_

```
std::atomic< bool > argo::iridium::xfloat::XFloatMailRetrievalService::termination_signaled_
[static], [private]
```

The termination signaled flag.

This flag is used to signal termination of the mail retrieval thread. During normal operation, the value of this field will be FALSE. Once termination is signaled, it will become TRUE causing all of the single instance of the mail retrieval to to exit its primary loop and die.

Referenced by operator()(), and set_termination_signaled().

10.90.5.25 thread_

```
thread argo::iridium::xfloat::XFloatMailRetrievalService::thread_ [static], [private]
```

The instance of the mail retrieval thread.

The thread operation is defined by the functor operator of the runnable object, thread_object_, which is invoked when this thread is instantiated.

Referenced by StartWorkerThread(), and TerminateWorkerThread().

10.90.5.26 thread_object_

```
shared_ptr< XFloatMailRetrievalService > argo::iridium::xfloat::XFloatMailRetrievalService
::thread_object_ [static], [private]
```

The mail retrieval thread object.

This is a reference to a runnable object whose functor operator is invoked during instantiation of the mail retrieval thread, thread_.

Referenced by StartWorkerThread().

10.90.5.27 user_

```
std::string util::mysql::LocalMysqlConfiguration::user_ [static], [private], [inherited]
```

The SQL server account username string.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

The documentation for this class was generated from the following files:

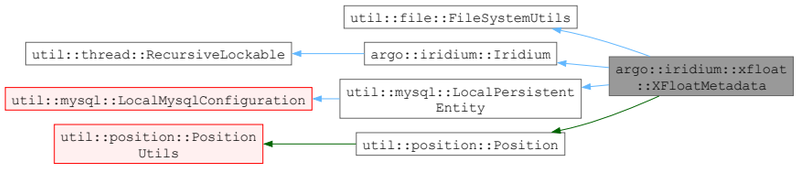
- stage1_MailRetrievalThread.h
- stage1_MailRetrievalThread.cpp

10.91 argo::iridium::xfloat::XFloatMetadata Class Reference

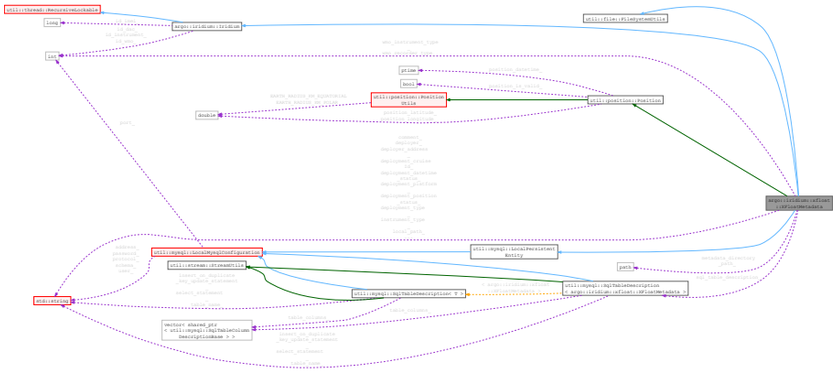
The instrument metadata class.

`#include <XFloatMetadata.h>`

Inheritance diagram for argo::iridium::xfloat::XFloatMetadata:



Collaboration diagram for argo::iridium::xfloat::XFloatMetadata:



Public Member Functions

- XFloatMetadata ()

  The default constructor.
- XFloatMetadata (const path &path)

  The file constructor.
- ~XFloatMetadata ()

  The destructor.
- string comment () const

  Get the instrument comment(s).
- string deployer () const

  Get the name of the instrument deployer(s).
- string deployer_address () const

  Get the address of the instrument deployer(s).
- string deployment_cruise_id () const

  Get the unique identifier of the deployment or launch cruise.
- ptime deployment_datetime () const

  Get the date/time of the deployment.
- string deployment_datetime_status () const

  Get the status of the accompanying date/time of the instrument deployment or launch.
- double deployment_latitude () const

  Get the latitude of the deployment position, in decimal degrees North.
- double deployment_longitude () const

  Get the longitude of the deployment position, in decimal degrees East.
- string deployment_platform () const

  Get the deployment or launch platform.
- string **deployment_position_as_iso6709_string** () const
- string deployment_position_status () const

  Get the status of the accompanying date/time of the instrument deployment or launch.
- string deployment_type () const

Get the deployment or launch type.

- int id_dac () const

  Get the unique Data Assembly Center (DAC) identifier.
- long id_imei () const

  Get the unique International Mobile Equipment Identifier (IMEI).
- int id_instrument () const

  Get the unique instrument identifier or instrument serial number.
- int id_wmo () const

  Get the unique World Meteorological Organization (WMO) identifier.
- const string & **instrument_type** () const
- bool **is_deployment_position_valid** () const
- string local_path () const

  Get the path to the metadata file on the local filesystem.
- bool LockObject (bool block)

  Lock the object.
- string manufacturer () const

  Get the instrument manufacturer.
- string principal_investigators () const

  Get the names of the principal investigator(s).
- string principal_investigators_address () const

  Get the address of the principal investigator(s).
- string principal_investigators_originating_country () const

  Get the originating country of the instrument.
- void SaveObjectToDatabase () const

  Save this object to the database.
- void UnlockObject ()

  Unlock the object.
- int wmo_instrument_type () const

  Get the WMO instrument type identifier.
- int wmo_recorder_type () const

  Get the WMO recorder type identifier.

Static Public Member Functions

- static void CloseSqlConnection ()

  Close the thread-specific MySQL connection if it is currently open.
- static shared_ptr< sql::ResultSet > **executeDeleteQuery** (const string &from, const string &where="")
- static shared_ptr< ResultSet > ExecuteQuery (Connection *con, const string &query)

  Execute an SQL query and return a shared pointer to the ResultSet object.
- static shared_ptr< sql::ResultSet > executeQuery (const string &query)

  Execute an SQL query and return a shared pointer to the ResultSet object.
- static shared_ptr< sql::ResultSet > executeSelectQuery (const string &what, const string &from, const string &where="", const string &groupBy="", const string &orderBy="", unsigned int limit=0, unsigned int offset=0)

  Execute an SQL select query and return a shared pointer to the ResultSet object.
- static int ExecuteUpdate (Connection *con, const string &update)

  Execute an SQL update and return the status code.
- static int executeUpdate (const string &update)

  Execute an SQL update and return the status code.
- static shared_ptr< PreparedStatement > GeneratePreparedStatement (Connection *con, const string &statement)

  Generate a prepared SQL statement object for the given SQL statement.
- static shared_ptr< sql::PreparedStatement > generatePreparedStatement (const string &statement)

  Generate a prepared SQL statement object for the given SQL statement.
- static string GenerateSqlDeleteStr (const string &from, const string &where="")

  Generate an SQL DELETE statement.
- static string GenerateSqlSelectStr (const string &what, const string &from, const string &where="", const string &groupBy="", const string &orderBy="", unsigned int limit=0, unsigned int offset=0)

  Generate an SQL select statement.
- static const path & GetDirectoryPath (const path &dir, bool create=false)

  Get a directory path.
- static path GetDirectoryPathFromString (const string &dir, bool create=false)

  Generate a boost::filesystem::path object from the directory path string.

- static thread_specific_ptr< Connection > & GetSqlConnection (string url, string user, string password, string schema)

  Get a thread-specific instance of a MySQL connection object.
- static thread_specific_ptr< MySQL_Driver > & GetSqlDriver ()

  Get a thread-specific instance of a MySQL driver object.
- static vector< path > ListDirectories (const path &dir)

  List all directories contained in the given directory.
- static vector< path > ListFiles (path dir, size_t n=0)

  List all regular files contained in the given directory.
- static path MoveFileToDirectory (const path &file, const path &dir_dst, bool create_dir=false)

  Copy a file into a directory.

Protected Member Functions

- void LoadFile (const path &path)

  Read instrument metadata from a file on the local filesystem.
- ptime position_datetime () const

  Get the date/time of the position.
- string position_datetime_as_iso8601_string () const

  Get a formatted string representing the position date/time composed using the ISO 8601 standard.
- bool position_is_valid () const

  Is the position valid?
- double position_latitude () const

  Get the latitude of the position, in decimal degrees.
- double position_longitude () const

  Get the longitude of the position, in decimal degrees.
- string postition_as_iso6709_string () const

  Get a formatted string representing the position composed using the ISO 6709 standard.
- void set_comment (const string &comment)

  Set the instrument comment(s).
- void set_deployer (const string &deployer)

  Set the name of the instrument deployer(s).
- void set_deployer_address (const string &address)

  Set the address of the instrument deployer(s).
- void set_deployment_cruise_id (const string &id)

  Set the unique identifier of the deployment or launch cruise.
- void set_deployment_datetime (const ptime &dt)

  Set the date/time of the deployment.
- void set_deployment_datetime_status (const string &stat)

  Set the status of the accompanying date/time of the instrument deployment or launch.
- void set_deployment_latitude (double lat)

  Set the deployment position latitude, in decimal degrees North.
- void set_deployment_longitude (double lon)

  Set the deployment position longitude, in decimal degrees East.
- void set_deployment_platform (const string &platform)

  Set the deployment or launch platform.
- void set_deployment_position_status (const string &status)

  Set the status of the accompanying date/time of the instrument deployment or launch.
- void **set_deployment_position_valid** (bool valid)
- void set_deployment_type (const string &type)

  Set the deployment or launch type.
- void set_gps_position_datetime (unsigned int gpsWeek, unsigned int gpsDay, unsigned int gpsHour=0, unsigned int gpsMinute=0, unsigned int gpsSecond=0)

  Set the date and time of this position from the given GPS week, day, and time.
- void set_id_dac (int id_dac)

  Set the unique Data Assembly Center (DAC) identifier.
- void set_id_imei (long id_imei)

  Set the unique International Mobile Equipment Identifier (IMEI).
- void set_id_instrument (int id_instrument)

  Set the unique instrument identifier or instrument serial number.
- void set_id_wmo (int id_wmo)

  Set the unique World Meteorological Organization (WMO) identifier.

- void **set_instrument_type** (const string &instrument_type)
- void set_local_path (const string &local_path)

  Set the path to the metadata file on the local filesystem.
- void set_manufacturer (const string &man)

  Set the instrument manufacturer.
- void set_position_datetime (const ptime &datetime)

  Set the date/time of the position.
- void set_position_is_valid (bool is_valid=true)

  Set the position is valid flag.
- void set_position_latitude (double lat)

  Set the latitude of the position, in decimal degrees.
- void set_position_longitude (double lon)

  Set the longitude of the position, in decimal degrees.
- void set_principal_investigators (const string &pis)

  Set the names of the principal investigator(s).
- void set_principal_investigators_address (const string &addr)

  Set the address of the principal investigator(s).
- void set_principal_investigators_originating_country (const string &country)

  Set the originating country of the instrument.
- void set_wmo_instrument_type (int type)

  Set the WMO instrument type identifier.
- void set_wmo_recorder_type (int type)

  Set the WMO recorder type identifier.

Static Protected Member Functions

- static float **bitcastUint32ToFloat** (uint32_t u)
- template<typename T >
  static double CalculateEllipsoidHeight (T lat)

  Get the approximate surface altitude or height of earth for a given latitude, in kilometers.
- template<typename T >
  static double CalculateGeographicDistance (T lat1, T lon1, T lat0, T lon0)

  Get the geographical distance between two position, in kilometers, from the positions' locations, in decimal degrees.
- template<typename T >
  static double degToRad (T deg)

  Convert angular units from from degrees to decimal radians.
- static string GenerateCurrentUtcTimeIso8601String ()

  Generate an ISO8601 format date/time string representation of the current time, in UTC.
- template<typename T >
  static std::string GenerateIso6709String (T lat, T lon)

  Generate an IS06709 string representation of the position given by (`lat`, `lon`).
- static string GenerateIso8601String (const ptime &p, bool extended=true)

  Generate an ISO8601 format date/time string representation of a POSIX time object.
- static double GenerateJulianDay (const ptime &dt, const ptime &ref_dt)

  Convert a date/time to fractional days since a reference date/time.
- static ptime GetCurrentUtcTime ()

  Generate a POSIX time object representing the current date and time (rounded to the nearest second).
- static sql::Connection * getSqlConnection ()

  Generate a thread-specific instance of a MySQL connection object.
- template<typename T >
  static bool IsPositionValid (T lat, T lon)

  Check position validity.
- static vector< path > ListMetadataFiles ()

  List the paths to all metadata files on the local filesystem.
- static vector< shared_ptr< XFloatMetadata > > LoadMetadataFromFiles ()

  Load all metadata from the files on the local filesystem.

Protected Attributes

- recursive_mutex mutex_

  Recursive mutual exclusion lock for synchronizing operations.

Static Private Member Functions

- static void CleanupSqlConnection (Connection *con)

  Cleanup an SQL connection.
- static void CleanupSqlDriver (MySQL_Driver *driver)

  Function for memory cleanup of MySQL driver on destruct.

Private Attributes

- string comment_

  The instrument comment(s).
- string deployer_

  The name of the instrument deployer(s).
- string deployer_address_

  The address of the instrument deployer(s).
- string deployment_cruiseId_

  The unique identifier of the deployment or launch cruise.
- string deployment_datetime_status_

  The status of the accompanying date/time of the instrument deployment or launch.
- string deployment_platform_

  The deployment or launch platform.
- string deployment_position_status_

  The status of the accompanying date/time of the instrument deployment or launch.
- string deployment_type_

  The deployment or launch type.
- int id_dac_

  The unique Data Assembly Center (DAC) identifier.
- long id_imei_

  The unique International Mobile Equipment Identifier (IMEI).
- int id_instrument_

  The unique instrument identifier or instrument serial number.
- int id_wmo_

  The unique World Meteorological Organization (WMO) identifier.
- string **instrument_type_**
- string local_path_

  The path to the metadata file on the local filesystem.
- string manufacturer_

  The instrument manufacturer.
- ptime position_datetime_

  The date and time of the position.
- bool position_is_valid_

  The valid position indicator.
- double position_latitude_

  The latitude of the position, in decimal degrees.
- double position_longitude_

  The longitude of the position, in decimal degrees.
- string principal_investigators_

  The names of the principal investigator(s).
- string principal_investigators_address_

  The address of the principal investigator(s).
- string principal_investigators_originating_country_

  The originating country of the instrument.
- int wmo_instrument_type_

  The WMO instrument type identifier.
- int wmo_recorder_type_

  The WMO recorder type identifier.

Static Private Attributes

- static string address_
  The SQL server qualified name or IP address string.
- static const double EARTH_RADIUS_KM_EQUATORIAL = 6378.1
  The equatorial radius of the earth, in kilometers.
- static const double EARTH_RADIUS_KM_POLAR = 6356.1
  The polar radius of the earth, in kilometers.
- static path metadata_directory_path_
  The path to metadata directory on the local filesystem.
- static string password_
  The SQL server account password string.
- static unsigned int port_
  The SQL server port number.
- static string protocol_
  The protocol to use in the SQL server connection e.g.
- static string schema_
  The SQL server schema name.
- static thread_specific_ptr< Connection > sql_connection_
  Tthe thread-specific pointer to MySQL connection object.
- static thread_specific_ptr< MySQL_Driver > sql_driver_
  The thread-specific pointer to MySQL driver object.
- static const util::mysql::SqlTableDescription< XFloatMetadata > sql_table_description_
  The SQL table description object.
- static string user_
  The SQL server account username string.

Friends

- istream & operator>> (istream &in, XFloatMetadata &metadata)
  Stream extraction operator overload function for the XFloatMetadata class.
- options_description util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription ()
  Friend function for setting the value of some of the static configuration options that are members of this class.
- class **XFloat**

10.91.1 Detailed Description

The instrument metadata class.

This class contains the metadata for a single instrument. The metadata object includes information about the instrument including details about deployment, principal investigators, and the instrument type. During application initialization, the metadata files are read from the local filesystem, and used to construct one metadata object per metadata file. Additionally, the metadata is persisted to the SQL database.

Todo Devise a method to actively check for (1) changes in the metadata files and (2) newly created metadata files. Load the metadata from the new or modified files and assign them to the corresponding float object. If necessary, retroactively process the mail messages for these instruments. Persist updates to the SQL database as necessary.

10.91.2 Constructor & Destructor Documentation

10.91.2.1 XFloatMetadata() [1/2]

```
argo::iridium::xfloat::XFloatMetadata::XFloatMetadata ()  [inline]
```

The default constructor.

10.91.2.2 XFloatMetadata() [2/2]

```
argo::iridium::xfloat::XFloatMetadata::XFloatMetadata (
            const path & path)  [inline]
```

The file constructor.

Reads instrument metadata from a file on the local filesystem.

Parameters

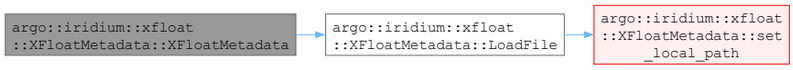
in	*path*	The path to the metadata file on the local filesystem.

Exceptions

util::log::MessageException	if the file does not exist or extraction of metadata from the file fails.

References CATCH_BLOCK, and LoadFile().

Here is the call graph for this function:



10.91.2.3 ~XFloatMetadata()

```
argo::iridium::xfloat::XFloatMetadata::~XFloatMetadata ()  [inline]
```

The destructor.

10.91.3 Member Function Documentation

10.91.3.1 CalculateEllipsoidHeight()

```
template<typename T >
static double util::position::PositionUtils::CalculateEllipsoidHeight (
            T lat)  [inline], [static], [inherited]
```

Get the approximate surface altitude or height of earth for a given latitude, in kilometers.

Warning

This method provides approximate surface height estimates only! This approximation is calculated from a reference ellipsoid – a mathematically-defined surface that approximates the geoid, the truer figure of the Earth.

Parameters

in	*lat*	The geographic latitude of interest, in decimal degrees.

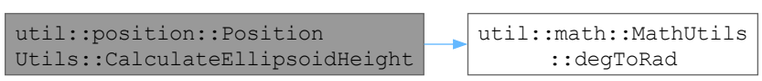
Returns

The approximate surface altitude or height of earth for the given latitude, in kilometers.

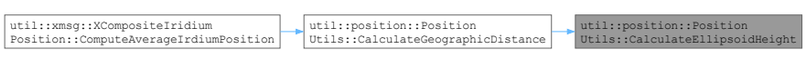
References util::math::MathUtils::degToRad(), util::position::PositionUtils::EARTH_RADIUS_KM_EQUATORIAL, and util::position::PositionUtils::EARTH_RADIUS_KM_POLAR.

Referenced by util::position::PositionUtils::CalculateGeographicDistance().

Here is the call graph for this function:



Here is the caller graph for this function:



10.91.3.2 CalculateGeographicDistance()

```
template<typename T >
static double util::position::PositionUtils::CalculateGeographicDistance (
            T lat1,
            T lon1,
            T lat0,
            T lon0)  [inline], [static], [inherited]
```

Get the geographical distance between two position, in kilometers, from the positions' locations, in decimal degrees.

Warning

This method provides approximate geographic estimates only! This approximation is calculated from a reference ellipsoid – a mathematically-defined surface that approximates the geoid, the truer figure of the Earth.

Note

This method uses the 'haversine' formula to calculate the great-circle distance between two points – that is, the shortest distance over the earth's surface – giving an 'as-the-crow-flies' distance between the points (ignoring any hills they fly over, of course!).

Parameters

in	*lat1*	The latitude of the geographic final position, in decimal degrees North.
in	*lon1*	The longitude of the geographic final position, in decimal degrees East.
in	*lat0*	The latitude of the geographic initial position, in decimal degrees North.
in	*lon0*	The longitude of the geographic initial position, in decimal degrees East.

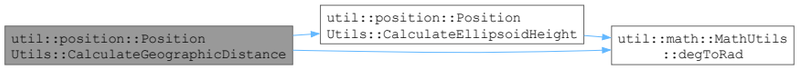
Returns

The geographical distance between the two given positions, in kilometers.

References util::position::PositionUtils::CalculateEllipsoidHeight(), and util::math::MathUtils::degToRad().

Referenced by util::xmsg::XCompositeIridiumPosition::ComputeAverageIrdiumPosition().

Here is the call graph for this function:



Here is the caller graph for this function:

10.91.3.3 CleanupSqlConnection()

```
static void util::mysql::MysqlUtils::CleanupSqlConnection (
            Connection * con)  [inline], [static], [private], [inherited]
```

Cleanup an SQL connection.

This function for memory cleanup of the SQL connection on destruction. The thread-specific SQL connection object will be deleted if it exists.

Parameters

in	*con*	The SQL connection to cleanup.

Exceptions

log::MessageException	if an error occurs while cleaning up the SQL connection object.

References CATCH_BLOCK.

10.91.3.4 CleanupSqlDriver()

```
static void util::mysql::MysqlUtils::CleanupSqlDriver (
            MySQL_Driver * driver)  [inline], [static], [private], [inherited]
```

Function for memory cleanup of MySQL driver on destruct.

Signals the SQL driver to end operation, if it exists.

Parameters

in	*driver*	Driver to cleanup.

Exceptions

log::MessageException	if an error occurs while cleaning up the SQL driver object.

References CATCH_BLOCK.

10.91.3.5 CloseSqlConnection()

```
static void util::mysql::MysqlUtils::CloseSqlConnection ()  [inline], [static], [inherited]
```

Close the thread-specific MySQL connection if it is currently open.

If the thread local MySQL connection is not currently open, then this function will have no effect.

Note

The MySQL connection should be closed when idle to prevent the connection from timing out and throwing exceptions.

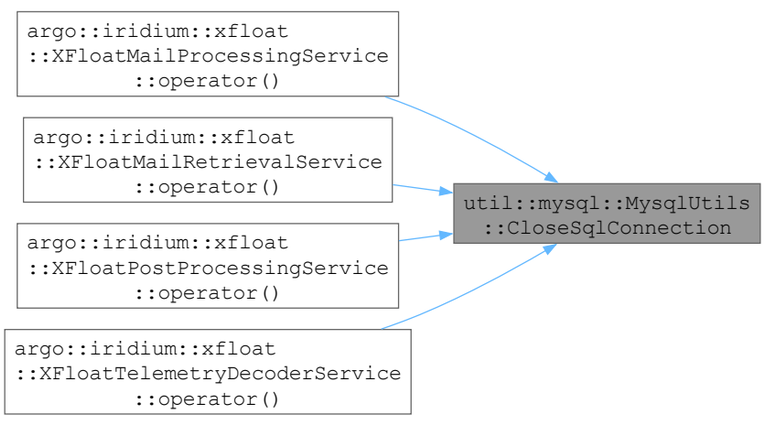
Exceptions

log::MessageException	if an error occurs while attempting to close the MySQL connection.

References CATCH_BLOCK, and util::mysql::MysqlUtils::sql_connection_.

Referenced by argo::iridium::xfloat::XFloatMailProcessingService::operator()(), argo::iridium::xfloat::XFloatMailRetrievalService::operator(
argo::iridium::xfloat::XFloatPostProcessingService::operator()(), and argo::iridium::xfloat::XFloatTelemetryDecoderService::operator()().

Here is the caller graph for this function:



10.91.3.6 comment()

```
string argo::iridium::xfloat::XFloatMetadata::comment () const  [inline]
```

Get the instrument comment(s).

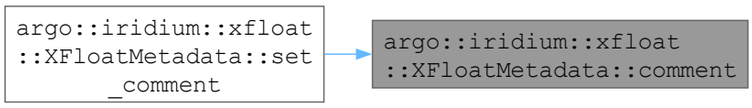
Returns

The instrument comment(s).

References comment_, and util::thread::RecursiveLockable::mutex_.

Referenced by set_comment().

Here is the caller graph for this function:



10.91.3.7 degToRad()

```
template<typename T >
static double util::math::MathUtils::degToRad (
            T deg)  [inline], [static], [inherited]
```

Convert angular units from from degrees to decimal radians.

Parameters

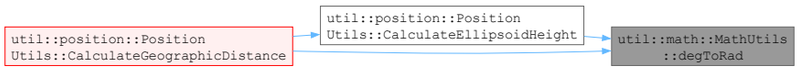
in	*deg*	Angle in integer or decimal degrees to convert to radians.

Returns

Angle in decimal radians.

Referenced by util::position::PositionUtils::CalculateEllipsoidHeight(), and util::position::PositionUtils::CalculateGeographicDistance().

Here is the caller graph for this function:



10.91.3.8 deployer()

```
string argo::iridium::xfloat::XFloatMetadata::deployer () const  [inline]
```

Get the name of the instrument deployer(s).

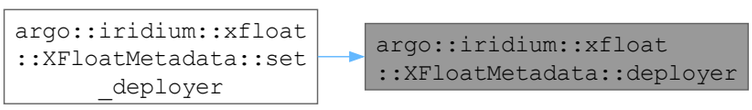
Returns

The name of the instrument deployer(s).

References deployer_, and util::thread::RecursiveLockable::mutex_.

Referenced by set_deployer().

Here is the caller graph for this function:



10.91.3.9 deployer_address()

```
string argo::iridium::xfloat::XFloatMetadata::deployer_address () const  [inline]
```

Get the address of the instrument deployer(s).

Returns

The address of the instrument deployer(s).

References deployer_address_, and util::thread::RecursiveLockable::mutex_.

10.91.3.10 deployment_cruise_id()

```
string argo::iridium::xfloat::XFloatMetadata::deployment_cruise_id () const  [inline]
```

Get the unique identifier of the deployment or launch cruise.

Returns

The unique identifier of the deployment or launch cruise.

References deployment_cruiseId_, and util::thread::RecursiveLockable::mutex_.

10.91.3.11 deployment_datetime()

```
ptime argo::iridium::xfloat::XFloatMetadata::deployment_datetime () const  [inline]
```

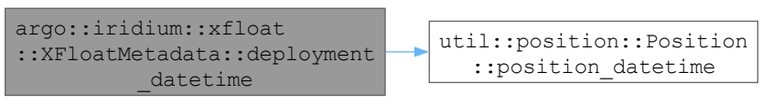
Get the date/time of the deployment.

Returns

The date/time of the deployment.

References util::thread::RecursiveLockable::mutex_, and util::position::Position::position_datetime().

Here is the call graph for this function:



10.91.3.12 deployment_datetime_status()

```
string argo::iridium::xfloat::XFloatMetadata::deployment_datetime_status () const  [inline]
```

Get the status of the accompanying date/time of the instrument deployment or launch.

Returns

The status of the accompanying date/time of the instrument deployment or launch.

References deployment_datetime_status_, and util::thread::RecursiveLockable::mutex_.

10.91.3.13 deployment_latitude()

```
double argo::iridium::xfloat::XFloatMetadata::deployment_latitude () const  [inline]
```

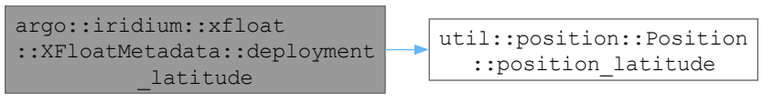
Get the latitude of the deployment position, in decimal degrees North.

Returns

The latitude of the deployment position, in decimal degrees North.

References util::thread::RecursiveLockable::mutex_, and util::position::Position::position_latitude().

Here is the call graph for this function:



10.91.3.14 deployment_longitude()

```
double argo::iridium::xfloat::XFloatMetadata::deployment_longitude () const  [inline]
```

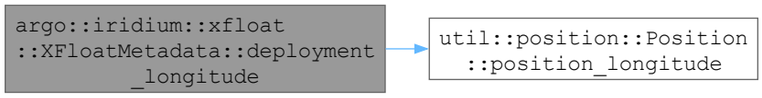
Get the longitude of the deployment position, in decimal degrees East.

Returns

The longitude of the deployment position, in decimal degrees East.

References util::thread::RecursiveLockable::mutex_, and util::position::Position::position_longitude().

Here is the call graph for this function:



10.91.3.15 deployment_platform()

```
string argo::iridium::xfloat::XFloatMetadata::deployment_platform () const  [inline]
```

Get the deployment or launch platform.

Returns

The deployment or launch platform.

References deployment_platform_, and util::thread::RecursiveLockable::mutex_.

10.91.3.16 deployment_position_status()

```
string argo::iridium::xfloat::XFloatMetadata::deployment_position_status () const  [inline]
```

Get the status of the accompanying date/time of the instrument deployment or launch.

Returns

The status of the accompanying date/time of the instrument deployment or launch.

References deployment_position_status_, and util::thread::RecursiveLockable::mutex_.

10.91.3.17 deployment_type()

```
string argo::iridium::xfloat::XFloatMetadata::deployment_type () const  [inline]
```

Get the deployment or launch type.

Returns

The deployment or launch type.

References deployment_type_, and util::thread::RecursiveLockable::mutex_.

10.91.3.18 ExecuteQuery()

```
static shared_ptr< ResultSet > util::mysql::MysqlUtils::ExecuteQuery (
            Connection * con,
            const string & query)  [inline], [static], [inherited]
```

Execute an SQL query and return a shared pointer to the ResultSet object.

Parameters

in	*con*	A pointer to the SQL connection object. This must not be NULL.
in	*query*	The SQL query to execute. This must be a well-formatted SQL query string.

Exceptions

log::MessageException	if 'con' is null, 'query' is a malfored SQL string, or any other error occurs.

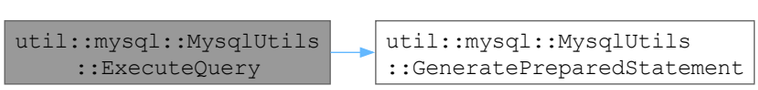
Returns

A shared pointer to the query ResultSet object.

References CATCH_BLOCK, and util::mysql::MysqlUtils::GeneratePreparedStatement().

Referenced by util::mysql::LocalMysqlConfiguration::executeQuery().

Here is the call graph for this function:



Here is the caller graph for this function:

10.91.3.19 executeQuery()

```
shared_ptr< sql::ResultSet > util::mysql::LocalMysqlConfiguration::executeQuery (
            const string & query)  [static], [inherited]
```

Execute an SQL query and return a shared pointer to the ResultSet object.

Parameters

in	*query*	The SQL query to execute. This must be a well-formatted SQL query string.

Exceptions

log::MessageException	'query' is a malfored SQL string or any other error occurs while executing the query.

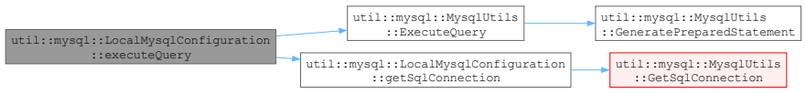
Returns

A shared pointer to the query ResultSet object.

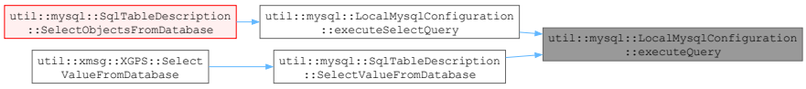
References util::mysql::MysqlUtils::ExecuteQuery(), and util::mysql::LocalMysqlConfiguration::getSqlConnection().

Referenced by util::mysql::LocalMysqlConfiguration::executeSelectQuery(), and util::mysql::SqlTableDescription< T >::SelectValueFromD

Here is the call graph for this function:



Here is the caller graph for this function:



10.91.3.20 executeSelectQuery()

```
shared_ptr< sql::ResultSet > util::mysql::LocalMysqlConfiguration::executeSelectQuery (
            const string & what,
            const string & from,
            const string & where = "",
            const string & groupBy = "",
            const string & orderBy = "",
            unsigned int limit = 0,
            unsigned int offset = 0)  [static], [inherited]
```

Execute an SQL select query and return a shared pointer to the ResultSet object.

Parameters

in	*what*	The SQL select statement select-expression clause.
in	*from*	The SQL select statement table–references clause.
in	*where*	The SQL select statement where-condition clause.
in	*groupBy*	The SQL select statement group-by clause.
in	*orderBy*	The SQL select statement order-by clause.
in	*limit*	The SQL select statement result set row limit value.
in	*offset*	The SQL select statement result set row offset.

Exceptions

log::MessageException	if an error occurs while executing the query.

Returns

A shared pointer to the ResultSet object.

References CATCH_BLOCK, util::mysql::LocalMysqlConfiguration::executeQuery(), and util::mysql::MysqlUtils::GenerateSqlSelectStr().

Referenced by util::mysql::SqlTableDescription< T >::SelectObjectsFromDatabase().

Here is the call graph for this function:

Here is the caller graph for this function:

10.91.3.21 ExecuteUpdate()

```
static int util::mysql::MysqlUtils::ExecuteUpdate (
            Connection * con,
            const string & update)  [inline], [static], [inherited]
```

Execute an SQL update and return the status code.

Parameters

in	*con*	A pointer to the SQL connection object. This must not be NULL.
in	*update*	The SQL update to execute. This must be a well-formatted SQL update string.

Exceptions

log::MessageException	if 'con' is null, 'update' is a malformed SQL string, or any other error occurs.

Returns

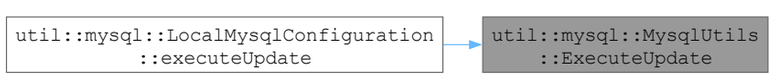
The update status code.

References CATCH_BLOCK, and util::mysql::MysqlUtils::GeneratePreparedStatement().

Referenced by util::mysql::LocalMysqlConfiguration::executeUpdate().

Here is the call graph for this function:

Here is the caller graph for this function:



10.91.3.22 executeUpdate()

```
int util::mysql::LocalMysqlConfiguration::executeUpdate (
            const string & update)  [static], [inherited]
```

Execute an SQL update and return the status code.

Parameters

in	*update*	The SQL update to execute. This must be a well-formatted SQL update string.

Exceptions

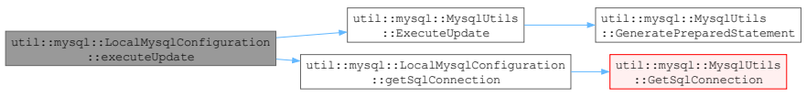
log::MessageException	if 'update' is a malformed SQL string, or any other error occurs while executing the update.

Returns

The update status code.

References util::mysql::MysqlUtils::ExecuteUpdate(), and util::mysql::LocalMysqlConfiguration::getSqlConnection().

Here is the call graph for this function:



10.91.3.23 GenerateCurrentUtcTimeIso8601String()

```
static string util::date::DateUtils::GenerateCurrentUtcTimeIso8601String ()  [inline], [static],
[inherited]
```

Generate an ISO8601 format date/time string representation of the current time, in UTC.

Note

All date/time objects use the UTC time-zone.

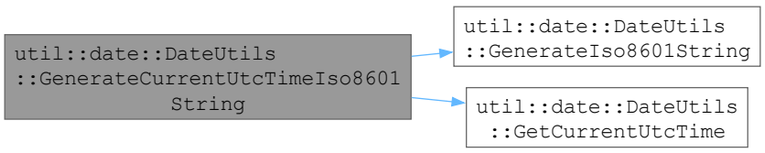
Returns

An ISO8601 format date/time string representation of the current time.

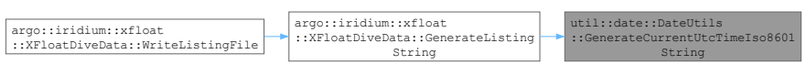
References util::date::DateUtils::GenerateIso8601String(), and util::date::DateUtils::GetCurrentUtcTime().

Referenced by argo::iridium::xfloat::XFloatDiveData::GenerateListingString().

Here is the call graph for this function:



Here is the caller graph for this function:



10.91.3.24 GenerateIso6709String()

```
template<typename T >
static std::string util::position::PositionUtils::GenerateIso6709String (
            T lat,
            T lon)  [inline], [static], [inherited]
```

Generate an IS06709 string representation of the position given by (`lat`, `lon`).

Parameters

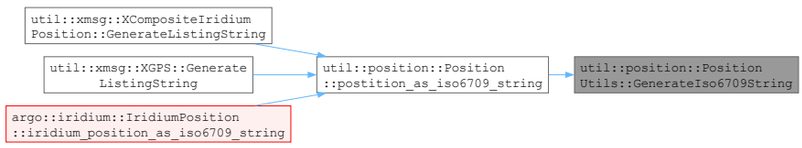
in	*lat*	The latitude of the position, in degrees North.
in	*lon*	The longitude of the position, in degrees East.

Returns

An IS06709 string representation of the position given by (`lat`, `lon`).

Referenced by util::position::Position::postition_as_iso6709_string().

Here is the caller graph for this function:



10.91.3.25 GenerateIso8601String()

```
static string util::date::DateUtils::GenerateIso8601String (
            const ptime & p,
            bool extended = true)  [inline], [static], [inherited]
```

Generate an ISO8601 format date/time string representation of a POSIX time object.

Note

All date/time objects use the UTC time-zone.

Parameters

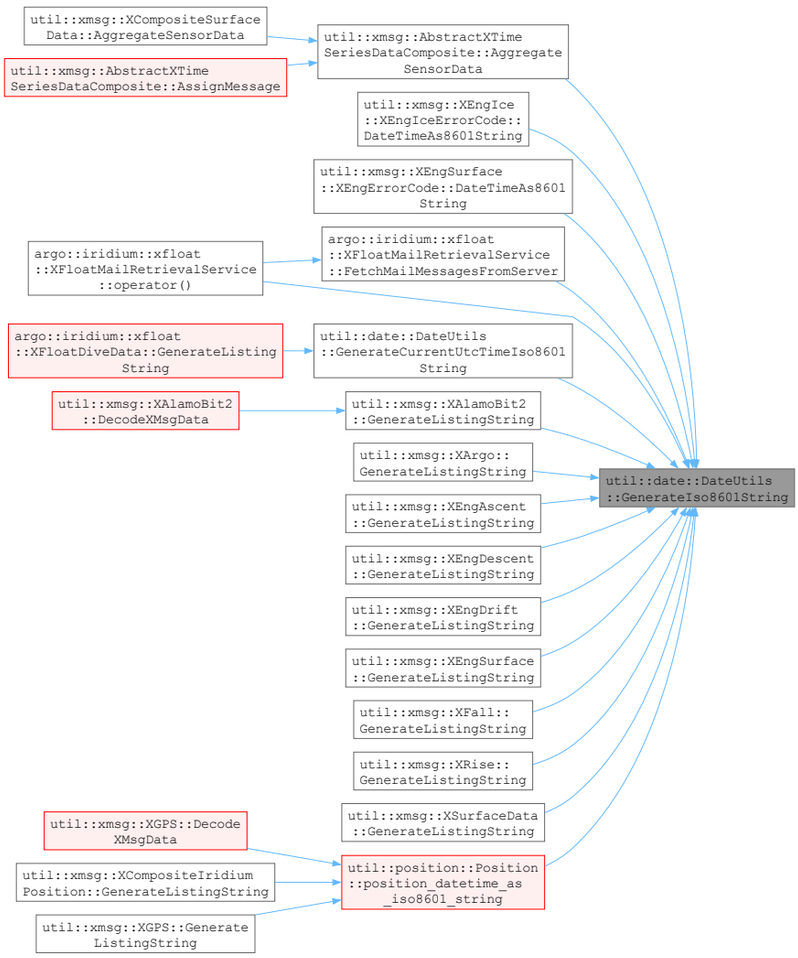
in	*p*	POSIX time from which to generate string.
in	*extended*	If TRUE, use the ISO 8601 extended format (YYYY-MM-DDThh:mm:ssZ); otherwise, use the basic format (YYYYMMDDThhmmss).

Returns

An ISO8601 format date/time string representation of the POSIX time object. If p is equal to boost::posix_↩ time::not_a_date_time, then the string "0000-00-00T00:00:00Z" (extended) or "00000000000000" (basic) will be returned.

Referenced by util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::AggregateSensorData(), util::xmsg::XEngIce::XEngIce
util::xmsg::XEngSurface::XEngErrorCode::DateTimeAs8601String(), argo::iridium::xfloat::XFloatMailRetrievalService::FetchMailMessages
util::date::DateUtils::GenerateCurrentUtcTimeIso8601String(), util::xmsg::XAlamoBit2::GenerateListingString(), util::xmsg::XArgo::Genera
util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListing
util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XRise::GenerateListingString(),
util::xmsg::XSurfaceData::GenerateListingString(), argo::iridium::xfloat::XFloatMailRetrievalService::operator()(), and
util::position::Position::position_datetime_as_iso8601_string().

Here is the caller graph for this function:



10.91.3.26 GenerateJulianDay()

```
static double util::date::DateUtils::GenerateJulianDay (
            const ptime & dt,
            const ptime & ref_dt)  [inline], [static], [inherited]
```

Convert a date/time to fractional days since a reference date/time.

Parameters

in	*dt*	The date/time to convert.
in	*ref↩ _dt*	The reference date/time (day 0).

Exceptions

util::log::MessageException	if either date/time is not valid.

Returns

The number of days (with millisecond resolution) from `ref_dt` to `dt`.

References CATCH_BLOCK, and EXC_THROW.

10.91.3.27 GeneratePreparedStatement()

```
static shared_ptr< PreparedStatement > util::mysql::MysqlUtils::GeneratePreparedStatement (
            Connection * con,
            const string & statement)  [inline], [static], [inherited]
```

Generate a prepared SQL statement object for the given SQL statement.

Parameters

in	*con*	A pointer to the SQL connection object. This must not be NULL.
in	*statement*	The SQL statement to prepare. This must be a well-formed SQL string.

Exceptions

log::MessageException	if 'con' is NULL, 'statement' is not a valid SQL string, or if an error occurs.

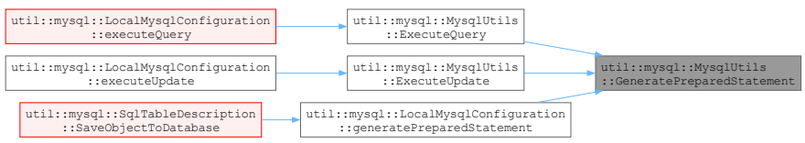
Returns

A prepared SQL statement object generated from the given SQL statement.

References CATCH_BLOCK, and EXC_THROW.

Referenced by util::mysql::MysqlUtils::ExecuteQuery(), util::mysql::MysqlUtils::ExecuteUpdate(), and util::mysql::LocalMysqlConfiguration:

Here is the caller graph for this function:



10.91.3.28 generatePreparedStatement()

```
std::shared_ptr< sql::PreparedStatement > util::mysql::LocalMysqlConfiguration::generatePrepared↩
Statement (
            const string & statement)  [static], [inherited]
```

Generate a prepared SQL statement object for the given SQL statement.

Parameters

in	*statement*	The SQL statement to prepare. This must be a well-formed SQL string.

Exceptions

log::MessageException	if 'statement' is not a valid SQL string or if an error occurs while generating the prepared statement object..

Returns

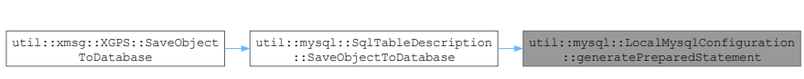
A prepared SQL statement object generated from the given SQL statement.

References util::mysql::MysqlUtils::GeneratePreparedStatement(), and util::mysql::LocalMysqlConfiguration::getSqlConnection().

Referenced by util::mysql::SqlTableDescription< T >::SaveObjectToDatabase().

Here is the call graph for this function:

Here is the caller graph for this function:



10.91.3.29 GenerateSqlDeleteStr()

```
static string util::mysql::MysqlUtils::GenerateSqlDeleteStr (
            const string & from,
            const string & where = "")  [inline], [static], [inherited]
```

Generate an SQL DELETE statement.

Warning

Without a `where` clause, the statement deletes every row in the table.

Parameters

in	*from*	The name of the table to delete from.
in	*where*	The SQL "where-condition" clause (empty = all rows).

Returns

The SQL statement.

10.91.3.30 GenerateSqlSelectStr()

```
static string util::mysql::MysqlUtils::GenerateSqlSelectStr (
            const string & what,
            const string & from,
            const string & where = "",
            const string & groupBy = "",
            const string & orderBy = "",
            unsigned int limit = 0,
            unsigned int offset = 0)  [inline], [static], [inherited]
```

Generate an SQL select statement.

Parameters

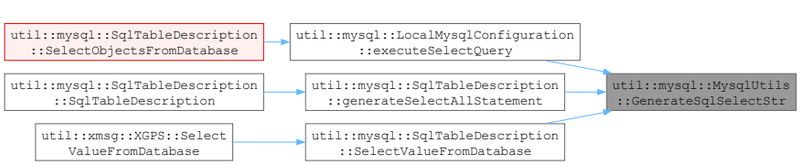
in	*what*	The SQL select statement select-expression clause.
in	*from*	The SQL select statement table-references clause.
in	*where*	The SQL select statement where-condition clause.
in	*groupBy*	The SQL select statement group-by clause.
in	*orderBy*	The SQL select statement order-by clause.
in	*limit*	The SQL select statement result set row limit value.
in	*offset*	The SQL select statement result set row offset.

Returns

The SQL select statement.

Referenced by util::mysql::LocalMysqlConfiguration::executeSelectQuery(), util::mysql::SqlTableDescription< T >::generateSelectAllState
and util::mysql::SqlTableDescription< T >::SelectValueFromDatabase().

Here is the caller graph for this function:



10.91.3.31 GetCurrentUtcTime()

```
static ptime util::date::DateUtils::GetCurrentUtcTime ()  [inline], [static], [inherited]
```

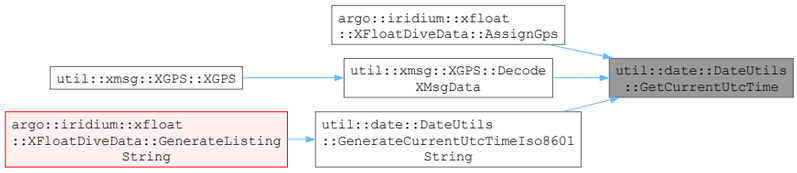
Generate a POSIX time object representing the current date and time (rounded to the nearest second).

Returns

A POSIX time object representing the current date and time (rounded to the nearest second).

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignGps(), util::xmsg::XGPS::DecodeXMsgData(), and util::date::DateUtils::GenerateCurrentUtcTimeIso8601String().

Here is the caller graph for this function:



10.91.3.32 GetDirectoryPath()

```
static const path & util::file::FileSystemUtils::GetDirectoryPath (
            const path & dir,
            bool create = false)  [inline], [static], [inherited]
```

Get a directory path.

Exceptions

MessageException	if create == FALSE and the directory specified by path does not exist.

Parameters

in	*dir*	Directory path.
in	*create*	If TRUE, the directory specified by path will be created if it does not already exists. If FALSE, an exception will be thrown if the directory does not exist.

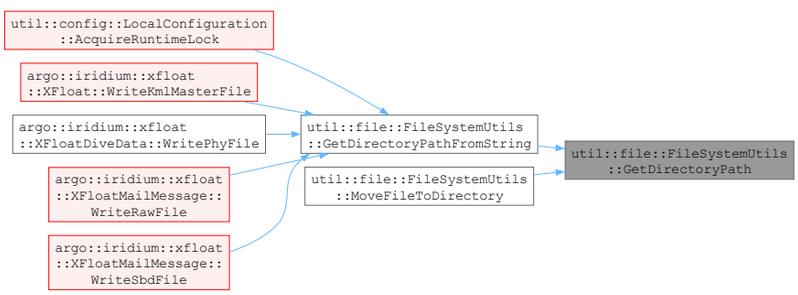
Returns

dir

References CATCH_BLOCK, EXC_THROW, and ldbg.

Referenced by util::file::FileSystemUtils::GetDirectoryPathFromString(), and util::file::FileSystemUtils::MoveFileToDirectory().

Here is the caller graph for this function:



10.91.3.33 GetDirectoryPathFromString()

```
static path util::file::FileSystemUtils::GetDirectoryPathFromString (
            const string & dir,
            bool create = false)  [inline], [static], [inherited]
```

Generate a boost::filesystem::path object from the directory path string.

Exceptions

MessageException	if create == FALSE and the directory specified by path does not exist.

Parameters

in	*dir*	Directory path string.
in	*create*	If TRUE, the directory specified by path will be created if it does not already exists. If FALSE, an exception will be thrown if the directory does not exist.

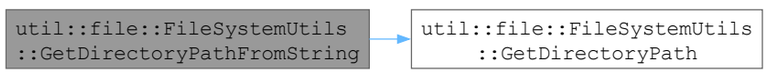
Returns

A boost::filesystem::path object from the directory path string.

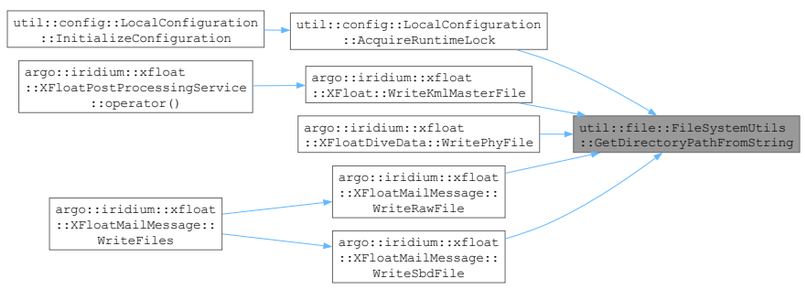
References CATCH_BLOCK, and util::file::FileSystemUtils::GetDirectoryPath().

Referenced by util::config::LocalConfiguration::AcquireRuntimeLock(), argo::iridium::xfloat::XFloat::WriteKmlMasterFile(), argo::iridium::xfloat::XFloatDiveData::WritePhyFile(), argo::iridium::xfloat::XFloatMailMessage::WriteRawFile(), and argo::iridium::xfloat::XFloatMailMessage::WriteSbdFile().

Here is the call graph for this function:



Here is the caller graph for this function:



10.91.3.34 GetSqlConnection()

```
static thread_specific_ptr< Connection > & util::mysql::MysqlUtils::GetSqlConnection (
            string url,
            string user,
            string password,
            string schema)  [inline], [static], [inherited]
```

Get a thread-specific instance of a MySQL connection object.

A new connection will be created, if necessary.

Parameters

in	*url*	The MySQL host server qualified address.
in	*user*	The MySQL account username.
in	*password*	The MySQL account password.
in	*schema*	The MySQL account schema name.

Exceptions

log::MessageException	if an error occurs while creating a new thread-specific instance of the SQL connection object

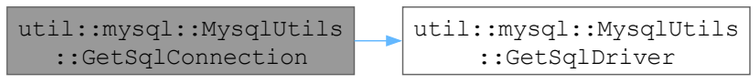
Returns

A thread-specific instance of a MySQL connection object.

References CATCH_BLOCK, util::mysql::MysqlUtils::GetSqlDriver(), and util::mysql::MysqlUtils::sql_connection_.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

Here is the call graph for this function:



Here is the caller graph for this function:

10.91.3.35 getSqlConnection()

`sql::Connection * util::mysql::LocalMysqlConfiguration::getSqlConnection () [static], [protected], [inherited]`

Generate a thread-specific instance of a MySQL connection object.

A new connection will be created, if necessary.

Exceptions

log::MessageException	if an error occurs while generating the SQL connection object.

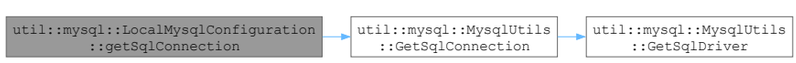
Returns

A thread-specific instance of a MySQL connection object.

References util::mysql::LocalMysqlConfiguration::address_, CATCH_BLOCK, util::mysql::MysqlUtils::GetSqlConnection(), util::mysql::LocalMysqlConfiguration::password_, util::mysql::LocalMysqlConfiguration::port_, util::mysql::LocalMysqlConfiguration::protoc util::mysql::LocalMysqlConfiguration::schema_, and util::mysql::LocalMysqlConfiguration::user_.

Referenced by util::mysql::LocalMysqlConfiguration::executeQuery(), util::mysql::LocalMysqlConfiguration::executeUpdate(), and util::mysql::LocalMysqlConfiguration::generatePreparedStatement().

Here is the call graph for this function:



Here is the caller graph for this function:

10.91.3.36 GetSqlDriver()

```
static thread_specific_ptr< MySQL_Driver > & util::mysql::MysqlUtils::GetSqlDriver ()  [inline],
[static], [inherited]
```

Get a thread-specific instance of a MySQL driver object.

A new driver will created, if necessary.

Exceptions

log::MessageException	if an error occurs while creating a new thread-specific instance of the SQL driver object

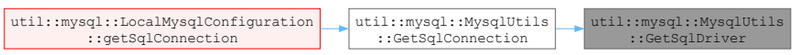
Returns

A thread-specific instance of a MySQL driver object.

References CATCH_BLOCK, and util::mysql::MysqlUtils::sql_driver_.

Referenced by util::mysql::MysqlUtils::GetSqlConnection().

Here is the caller graph for this function:



10.91.3.37 id_dac()

```
int argo::iridium::Iridium::id_dac () const  [inline], [inherited]
```

Get the unique Data Assembly Center (DAC) identifier.

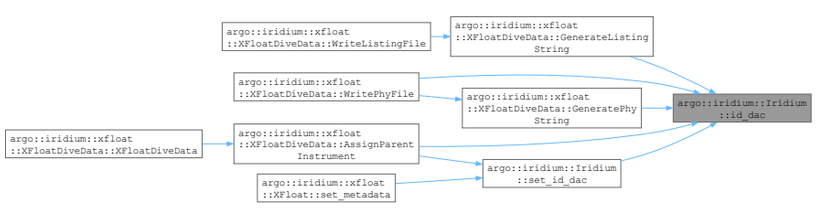
Returns

The unique Data Assembly Center (DAC) identifier.

References argo::iridium::Iridium::id_dac_, and util::thread::RecursiveLockable::mutex_.

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignParentInstrument(), argo::iridium::xfloat::XFloatDiveData::GenerateListingStrin
argo::iridium::xfloat::XFloatDiveData::GeneratePhyString(), argo::iridium::Iridium::set_id_dac(), and argo::iridium::xfloat::XFloatDiveData::

Here is the caller graph for this function:



10.91.3.38 id_imei()

```
long argo::iridium::Iridium::id_imei () const  [inline], [inherited]
```

Get the unique International Mobile Equipment Identifier (IMEI).

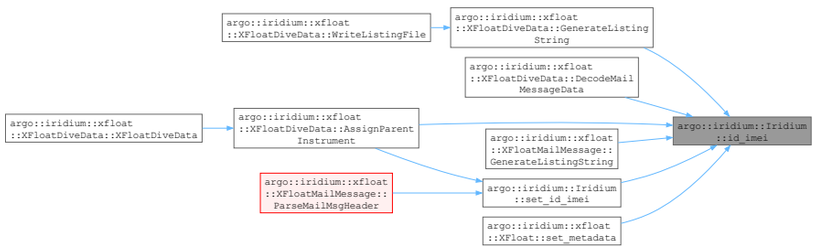
Returns

The unique International Mobile Equipment Identifier (IMEI).

References argo::iridium::Iridium::id_imei_, and util::thread::RecursiveLockable::mutex_.

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignParentInstrument(), argo::iridium::xfloat::XFloatDiveData::DecodeMailMessag
argo::iridium::xfloat::XFloatDiveData::GenerateListingString(), argo::iridium::xfloat::XFloatMailMessage::GenerateListingString(),
argo::iridium::Iridium::set_id_imei(), and argo::iridium::xfloat::XFloat::set_metadata().

Here is the caller graph for this function:



10.91.3.39 id_instrument()

```
int argo::iridium::Iridium::id_instrument () const  [inline], [inherited]
```

Get the unique instrument identifier or instrument serial number.

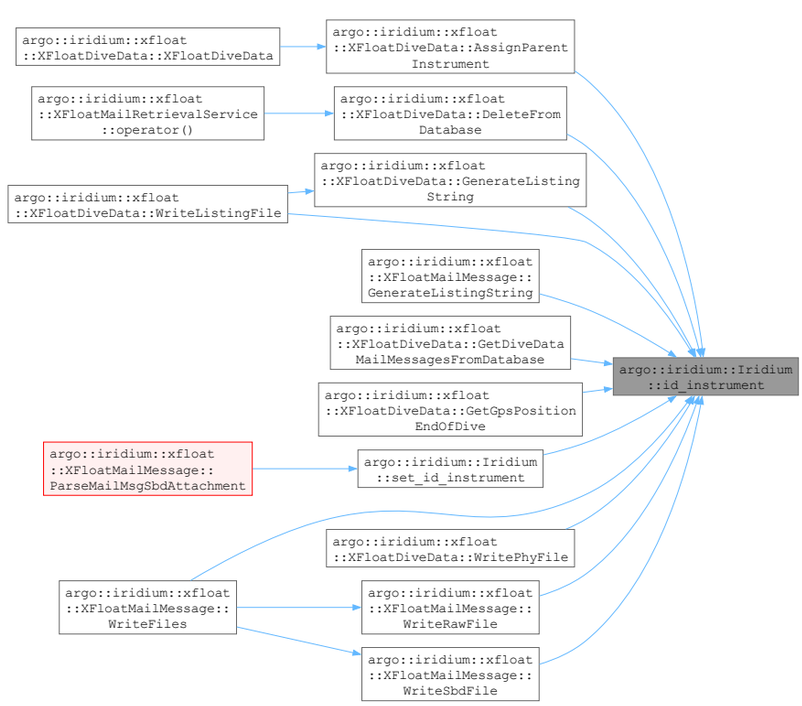
Returns

The unique instrument identifier or instrument serial number.

References argo::iridium::Iridium::id_instrument_, and util::thread::RecursiveLockable::mutex_.

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignParentInstrument(), argo::iridium::xfloat::XFloatDiveData::DeleteFromDatabas
argo::iridium::xfloat::XFloatDiveData::GenerateListingString(), argo::iridium::xfloat::XFloatMailMessage::GenerateListingString(),
argo::iridium::xfloat::XFloatDiveData::GetDiveDataMailMessagesFromDatabase(), argo::iridium::xfloat::XFloatDiveData::GetGpsPositionE
argo::iridium::Iridium::set_id_instrument(), argo::iridium::xfloat::XFloatMailMessage::WriteFiles(), argo::iridium::xfloat::XFloatDiveData::Wr
argo::iridium::xfloat::XFloatDiveData::WritePhyFile(), argo::iridium::xfloat::XFloatMailMessage::WriteRawFile(), and
argo::iridium::xfloat::XFloatMailMessage::WriteSbdFile().

Here is the caller graph for this function:



10.91.3.40 id_wmo()

```
int argo::iridium::Iridium::id_wmo () const  [inline], [inherited]
```

Get the unique World Meteorological Organization (WMO) identifier.

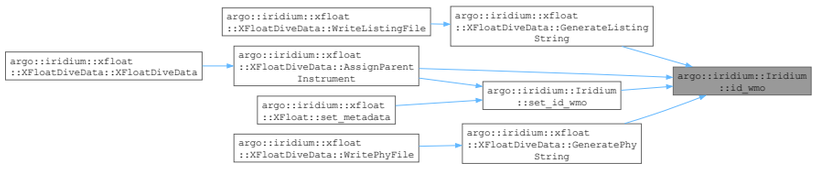
Returns

The unique World Meteorological Organization (WMO) identifier.

References argo::iridium::Iridium::id_wmo_, and util::thread::RecursiveLockable::mutex_.

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignParentInstrument(), argo::iridium::xfloat::XFloatDiveData::GenerateListingStrir
argo::iridium::xfloat::XFloatDiveData::GeneratePhyString(), and argo::iridium::Iridium::set_id_wmo().

Here is the caller graph for this function:



10.91.3.41 IsPositionValid()

```
template<typename T >
static bool util::position::PositionUtils::IsPositionValid (
            T lat,
            T lon)  [inline], [static], [inherited]
```

Check position validity.

Parameters

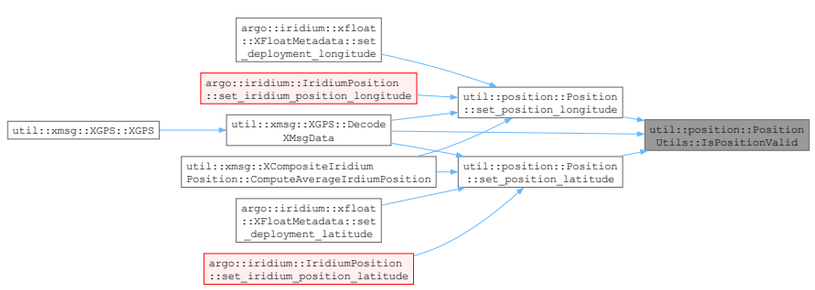
in	*lat*	Latitude of position.
in	*lon*	Longitude of position.

Returns

TRUE if (lat, lon) represents a valid geographical position, or FALSE otherwise.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), util::position::Position::set_position_latitude(), and util::position::Position::set_positi

Here is the caller graph for this function:



10.91.3.42 ListDirectories()

```
static vector< path > util::file::FileSystemUtils::ListDirectories (
            const path & dir)  [inline], [static], [inherited]
```

List all directories contained in the given directory.

Parameters

in	*dir*	The path to the directory of interest.

Exceptions

log::MessageException	if the directory does not exist, is not a directory, or an error occurs.

Returns

A vector of paths to all the directories contained in the given directory.

References CATCH_BLOCK, and EXC_THROW.

10.91.3.43 ListFiles()

```
static vector< path > util::file::FileSystemUtils::ListFiles (
            path dir,
            size_t n = 0)  [inline], [static], [inherited]
```

List all regular files contained in the given directory.

Parameters

in	*dir*	The path to the directory of interest.
in	*n*	If n>0, the maximum number of paths to return, else no effect.

Exceptions

log::MessageException	if the directory does not exist, is not a directory, or an error occurs.

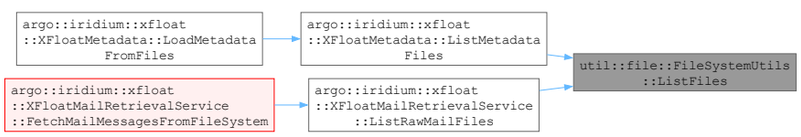
Returns

A vector of paths to all the regular files contained in the given directory.

References CATCH_BLOCK, and EXC_THROW.

Referenced by ListMetadataFiles(), and argo::iridium::xfloat::XFloatMailRetrievalService::ListRawMailFiles().

Here is the caller graph for this function:



10.91.3.44 ListMetadataFiles()

```
static vector< path > argo::iridium::xfloat::XFloatMetadata::ListMetadataFiles ()  [inline],
[static], [protected]
```

List the paths to all metadata files on the local filesystem.

Exceptions

util::log::MessageException	if an error occurs while listing the metadata paths.

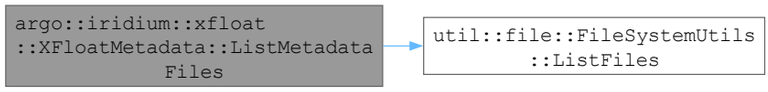
Returns

A vector containing the paths to the metadata files on the local filesystem.

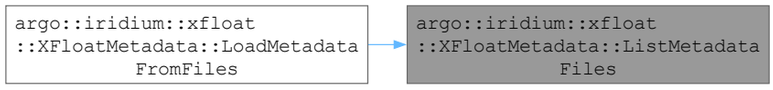
References CATCH_BLOCK, EXC_THROW, util::file::FileSystemUtils::ListFiles(), and metadata_directory_path_.

Referenced by LoadMetadataFromFiles().

Here is the call graph for this function:



Here is the caller graph for this function:



10.91.3.45 LoadFile()

```
void argo::iridium::xfloat::XFloatMetadata::LoadFile (
            const path & path)  [inline], [protected]
```

Read instrument metadata from a file on the local filesystem.

Parameters

in	*path*	The path to the metadata file on the local filesystem.

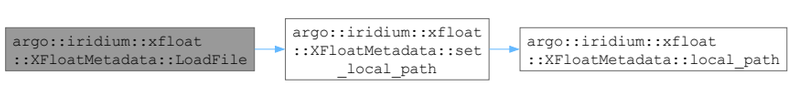
Exceptions

util::log::MessageException	if the file does not exist or extraction of metadata from the file fails.

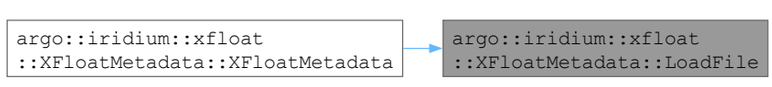
References CATCH_NOTHROW, EXC_THROW, ldbg, and set_local_path().

Referenced by XFloatMetadata().

Here is the call graph for this function:



Here is the caller graph for this function:



10.91.3.46 LoadMetadataFromFiles()

```
static vector< shared_ptr< XFloatMetadata > > argo::iridium::xfloat::XFloatMetadata::Load↩
MetadataFromFiles ()  [inline], [static], [protected]
```

Load all metadata from the files on the local filesystem.

Exceptions

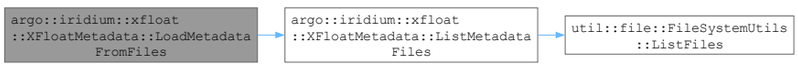
util::log::MessageException	if an error occurs while loading the metadata from the files on the local filesystem.

Returns

A vector of shared pointers to the metadata objects constructed from the metadata files on the local filesystem.

References CATCH_BLOCK, and ListMetadataFiles().

Here is the call graph for this function:



10.91.3.47 local_path()

```
string argo::iridium::xfloat::XFloatMetadata::local_path () const  [inline]
```

Get the path to the metadata file on the local filesystem.

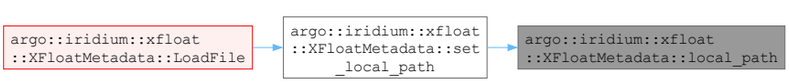
Returns

The path to the metadata file on the local filesystem.

References local_path_, and util::thread::RecursiveLockable::mutex_.

Referenced by set_local_path().

Here is the caller graph for this function:



10.91.3.48 LockObject()

```
bool util::thread::RecursiveLockable::LockObject (
            bool block)  [inline], [inherited]
```

Lock the object.

Parameters

in	*block*	If TRUE, then the calling thread will block until the lock becomes available.

Exceptions

util::log::MessageException	if an error occurs.

Returns

TRUE if the lock was successfully obtained, or FALSE if block is FALSE and another thread owns the lock on the mutex.

References CATCH_BLOCK, and util::thread::RecursiveLockable::mutex_.

10.91.3.49 manufacturer()

```
string argo::iridium::xfloat::XFloatMetadata::manufacturer () const  [inline]
```

Get the instrument manufacturer.

Returns

The instrument manufacturer.

References manufacturer_, and util::thread::RecursiveLockable::mutex_.

10.91.3.50 MoveFileToDirectory()

```
static path util::file::FileSystemUtils::MoveFileToDirectory (
            const path & file,
            const path & dir_dst,
            bool create_dir = false)  [inline], [static], [inherited]
```

Copy a file into a directory.

Warning

Despite its name, this function copies the file and does not remove the source. If a file with the same name already exists in the destination directory, it is left unchanged. It is not currently called.

Parameters

in	*file*	The file to copy.
in	*dir_dst*	The destination directory.
in	*create_dir*	If TRUE, create the destination directory if it does not exist.

Exceptions

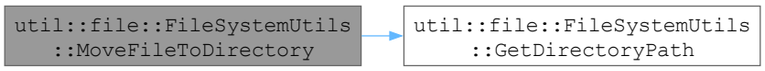
util::log::MessageException	if the file does not exist, is not a regular file, or cannot be copied.

Returns

The path of the file in the destination directory.

References CATCH_BLOCK, EXC_THROW, and util::file::FileSystemUtils::GetDirectoryPath().

Here is the call graph for this function:



10.91.3.51 position_datetime()

```
ptime util::position::Position::position_datetime () const  [inline], [inherited]
```

Get the date/time of the position.

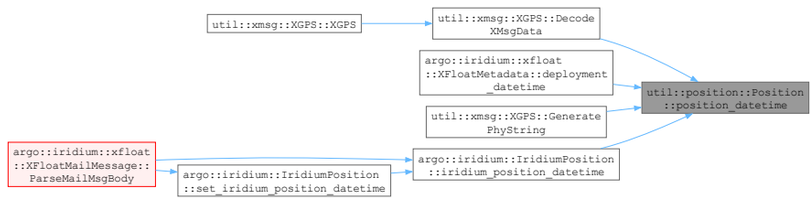
Returns

The date/time of the position.

References util::position::Position::position_datetime_.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), deployment_datetime(), util::xmsg::XGPS::GeneratePhyString(), and argo::iridium::IridiumPosition::iridium_position_datetime().

Here is the caller graph for this function:



10.91.3.52 position_datetime_as_iso8601_string()

```
string util::position::Position::position_datetime_as_iso8601_string () const  [inline], [inherited]
```

Get a formatted string representing the position date/time composed using the ISO 8601 standard.

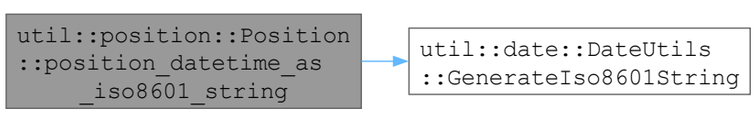
Returns

The formatted string representation of the position date/time.

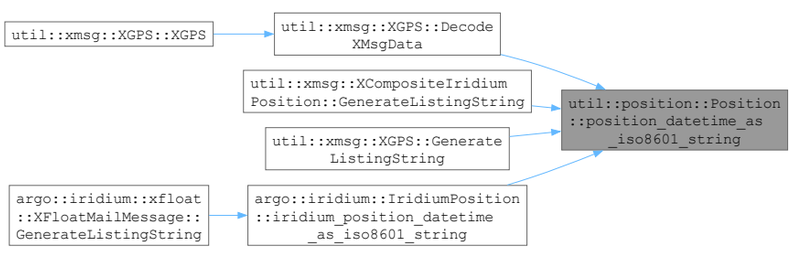
References util::date::DateUtils::GenerateIso8601String(), and util::position::Position::position_datetime_.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XCompositeIridiumPosition::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), and argo::iridium::IridiumPosition::iridium_position_datetime_as_iso8601_string().

Here is the call graph for this function:



Here is the caller graph for this function:



10.91.3.53 position_is_valid()

```
bool util::position::Position::position_is_valid () const  [inline], [inherited]
```

Is the position valid?

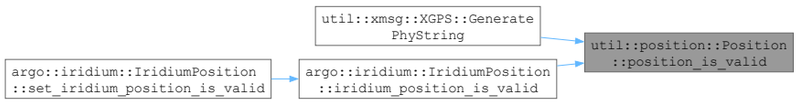
Returns

TRUE if the position is valid, or false otherwise.

References util::position::Position::position_is_valid_.

Referenced by util::xmsg::XGPS::GeneratePhyString(), and argo::iridium::IridiumPosition::iridium_position_is_valid().

Here is the caller graph for this function:



10.91.3.54 position_latitude()

```
double util::position::Position::position_latitude () const  [inline], [inherited]
```

Get the latitude of the position, in decimal degrees.

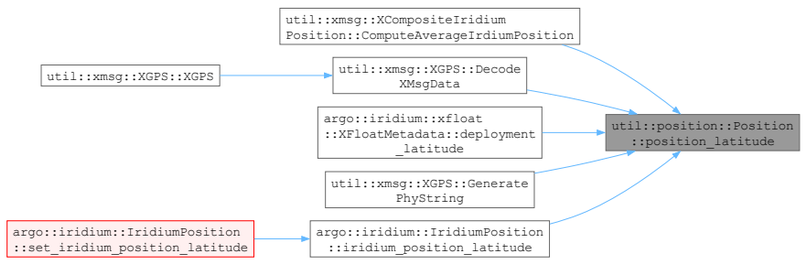
Returns

The latitude of the position, in decimal degrees.

References util::position::Position::position_latitude_.

Referenced by util::xmsg::XCompositeIridiumPosition::ComputeAverageIrdiumPosition(), util::xmsg::XGPS::DecodeXMsgData(), deployment_latitude(), util::xmsg::XGPS::GeneratePhyString(), and argo::iridium::IridiumPosition::iridium_position_latitude().

Here is the caller graph for this function:



10.91.3.55 position_longitude()

```
double util::position::Position::position_longitude () const  [inline], [inherited]
```

Get the longitude of the position, in decimal degrees.

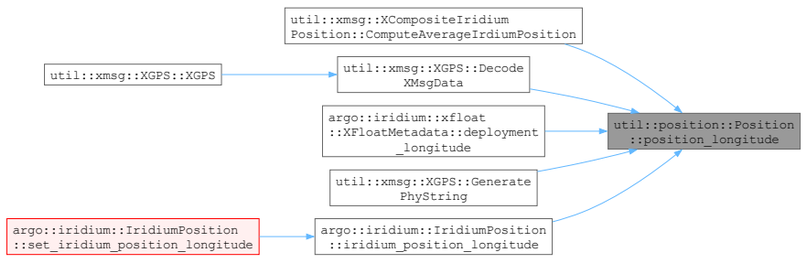
Returns

The longitude of the position, in decimal degrees.

References util::position::Position::position_longitude_.

Referenced by util::xmsg::XCompositeIridiumPosition::ComputeAverageIrdiumPosition(), util::xmsg::XGPS::DecodeXMsgData(), deployment_longitude(), util::xmsg::XGPS::GeneratePhyString(), and argo::iridium::IridiumPosition::iridium_position_longitude().

Here is the caller graph for this function:



10.91.3.56 postition_as_iso6709_string()

```
string util::position::Position::postition_as_iso6709_string () const  [inline], [inherited]
```

Get a formatted string representing the position composed using the ISO 6709 standard.

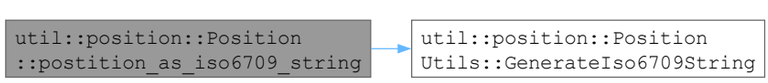
Returns

The formatted string representation of the position.

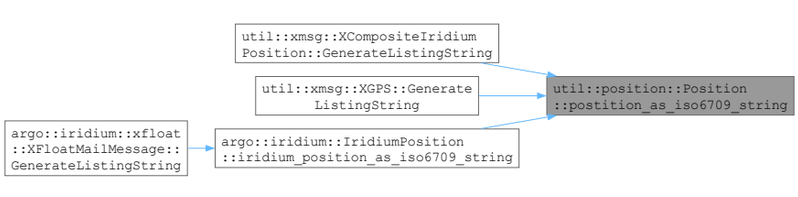
References util::position::PositionUtils::GenerateIso6709String(), util::position::Position::position_latitude_, and util::position::Position::position_longitude_.

Referenced by util::xmsg::XCompositeIridiumPosition::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), and argo::iridium::IridiumPosition::iridium_position_as_iso6709_string().

Here is the call graph for this function:



Here is the caller graph for this function:



10.91.3.57 principal_investigators()

```
string argo::iridium::xfloat::XFloatMetadata::principal_investigators () const  [inline]
```

Get the names of the principal investigator(s).

Returns

The names of the principal investigator(s).

References util::thread::RecursiveLockable::mutex_, and principal_investigators_.

10.91.3.58 principal_investigators_address()

```
string argo::iridium::xfloat::XFloatMetadata::principal_investigators_address () const  [inline]
```

Get the address of the principal investigator(s).

Returns

The address of the principal investigator(s).

References util::thread::RecursiveLockable::mutex_, and principal_investigators_address_.

10.91.3.59 principal_investigators_originating_country()

```
string argo::iridium::xfloat::XFloatMetadata::principal_investigators_originating_country () const
[inline]
```

Get the originating country of the instrument.

Returns

The originating country of the instrument.

References util::thread::RecursiveLockable::mutex_, and principal_investigators_originating_country_.

10.91.3.60 SaveObjectToDatabase()

```
void argo::iridium::xfloat::XFloatMetadata::SaveObjectToDatabase () const  [inline], [virtual]
```

Save this object to the database.

Exceptions

util::log::MessageException	if an error occurs while saving this object to the database.

Implements util::mysql::LocalPersistentEntity.

References CATCH_BLOCK, ldbg, util::thread::RecursiveLockable::mutex_, and sql_table_description_.

10.91.3.61 set_comment()

```
void argo::iridium::xfloat::XFloatMetadata::set_comment (
            const string & comment)  [inline], [protected]
```

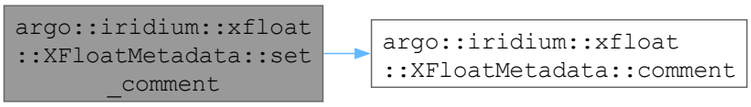
Set the instrument comment(s).

Parameters

in	*comment*	The instrument comment(s).

References comment(), comment_, and util::thread::RecursiveLockable::mutex_.

Here is the call graph for this function:



10.91.3.62 set_deployer()

```
void argo::iridium::xfloat::XFloatMetadata::set_deployer (
            const string & deployer)  [inline], [protected]
```

Set the name of the instrument deployer(s).

Parameters

in	*deployer*	The name of the instrument deployer(s).

References deployer(), deployer_, and util::thread::RecursiveLockable::mutex_.

Here is the call graph for this function:

10.91.3.63 set_deployer_address()

```
void argo::iridium::xfloat::XFloatMetadata::set_deployer_address (
            const string & address)  [inline], [protected]
```

Set the address of the instrument deployer(s).

Parameters

in	*address*	The address of the instrument deployer(s).

References deployer_address_, and util::thread::RecursiveLockable::mutex_.

10.91.3.64 set_deployment_cruise_id()

```
void argo::iridium::xfloat::XFloatMetadata::set_deployment_cruise_id (
            const string & id)  [inline], [protected]
```

Set the unique identifier of the deployment or launch cruise.

Parameters

in	*id*	The unique identifier of the deployment or launch cruise.

References deployment_cruiseId_, and util::thread::RecursiveLockable::mutex_.

10.91.3.65 set_deployment_datetime()

```
void argo::iridium::xfloat::XFloatMetadata::set_deployment_datetime (
            const ptime & dt)  [inline], [protected]
```

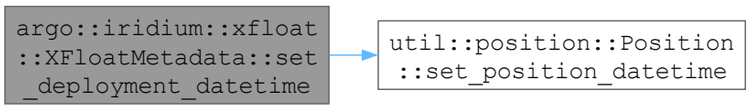
Set the date/time of the deployment.

Parameters

in	*dt*	The date/time of the deployment.

References util::thread::RecursiveLockable::mutex_, and util::position::Position::set_position_datetime().

Here is the call graph for this function:



10.91.3.66 set_deployment_datetime_status()

```
void argo::iridium::xfloat::XFloatMetadata::set_deployment_datetime_status (
            const string & stat)  [inline], [protected]
```

Set the status of the accompanying date/time of the instrument deployment or launch.

Parameters

in	*stat*	The status of the accompanying date/time of the instrument deployment or launch.

References deployment_datetime_status_, and util::thread::RecursiveLockable::mutex_.

10.91.3.67 set_deployment_latitude()

```
void argo::iridium::xfloat::XFloatMetadata::set_deployment_latitude (
            double lat)  [inline], [protected]
```

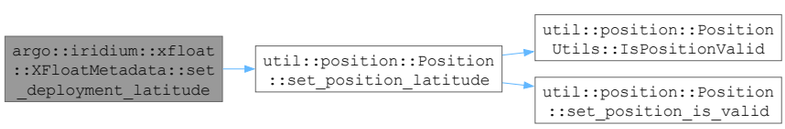
Set the deployment position latitude, in decimal degrees North.

Parameters

in	*lat*	The deployment position latitude, in decimal degrees North.

References util::thread::RecursiveLockable::mutex_, and util::position::Position::set_position_latitude().

Here is the call graph for this function:



10.91.3.68 set_deployment_longitude()

```
void argo::iridium::xfloat::XFloatMetadata::set_deployment_longitude (
            double lon)  [inline], [protected]
```

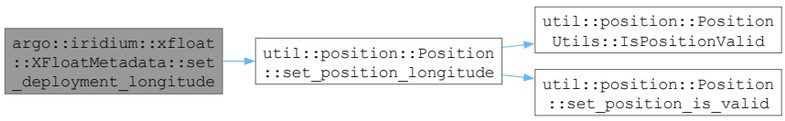
Set the deployment position longitude, in decimal degrees East.

Parameters

in	*lon*	The deployment position longitude, in decimal degrees East.

References util::thread::RecursiveLockable::mutex_, and util::position::Position::set_position_longitude().

Here is the call graph for this function:



10.91.3.69 set_deployment_platform()

```
void argo::iridium::xfloat::XFloatMetadata::set_deployment_platform (
            const string & platform)  [inline], [protected]
```

Set the deployment or launch platform.

Parameters

in	*platform*	The deployment or launch platform.

References deployment_platform_, and util::thread::RecursiveLockable::mutex_.

10.91.3.70 set_deployment_position_status()

```
void argo::iridium::xfloat::XFloatMetadata::set_deployment_position_status (
            const string & status)  [inline], [protected]
```

Set the status of the accompanying date/time of the instrument deployment or launch.

Parameters

in	*status*	The status of the accompanying date/time of the instrument deployment or launch.

References deployment_position_status_, and util::thread::RecursiveLockable::mutex_.

10.91.3.71 set_deployment_type()

```
void argo::iridium::xfloat::XFloatMetadata::set_deployment_type (
            const string & type)  [inline], [protected]
```

Set the deployment or launch type.

Parameters

in	*type*	The deployment or launch type.

References deployment_type_, and util::thread::RecursiveLockable::mutex_.

10.91.3.72 set_gps_position_datetime()

```
void util::position::Position::set_gps_position_datetime (
            unsigned int gpsWeek,
            unsigned int gpsDay,
            unsigned int gpsHour = 0,
            unsigned int gpsMinute = 0,
            unsigned int gpsSecond = 0)  [inline], [protected], [inherited]
```

Set the date and time of this position from the given GPS week, day, and time.

Parameters

in	*gpsWeek*	The GPS week, in weeks since midnight of January 6, 1980.
in	*gpsDay*	The GPS day of week (0 - 6).
in	*gpsHour*	The UTC hour of day (0 - 23).
in	*gpsMinute*	The UTC minute of hour (0 - 59)..
in	*gpsSecond*	The UTC second of minute (0 - 59).

References util::position::Position::set_position_datetime().

Referenced by util::xmsg::XGPS::DecodeXMsgData().

Here is the call graph for this function:

Here is the caller graph for this function:

10.91.3.73 set_id_dac()

```
void argo::iridium::Iridium::set_id_dac (
            int id_dac)  [inline], [protected], [inherited]
```

Set the unique Data Assembly Center (DAC) identifier.

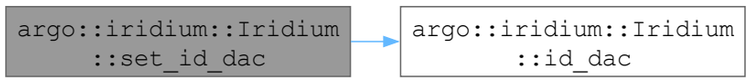
Parameters

in	*id_dac*	The unique Data Assembly Center (DAC) identifier.

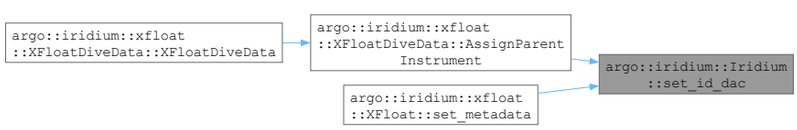
References argo::iridium::Iridium::id_dac(), argo::iridium::Iridium::id_dac_, and util::thread::RecursiveLockable::mutex_.

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignParentInstrument(), and argo::iridium::xfloat::XFloat::set_metadata().

Here is the call graph for this function:



Here is the caller graph for this function:



10.91.3.74 set_id_imei()

```
void argo::iridium::Iridium::set_id_imei (
            long id_imei)  [inline], [protected], [inherited]
```

Set the unique International Mobile Equipment Identifier (IMEI).

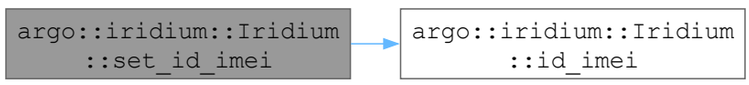
Parameters

in	*id_imei*	The unique International Mobile Equipment Identifier (IMEI).

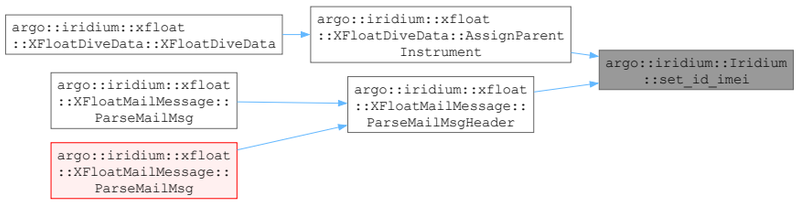
References argo::iridium::Iridium::id_imei(), argo::iridium::Iridium::id_imei_, and util::thread::RecursiveLockable::mutex_.

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignParentInstrument(), and argo::iridium::xfloat::XFloatMailMessage::ParseMailM

Here is the call graph for this function:



Here is the caller graph for this function:



10.91.3.75 set_id_instrument()

```
void argo::iridium::Iridium::set_id_instrument (
            int id_instrument)  [inline], [protected], [inherited]
```

Set the unique instrument identifier or instrument serial number.

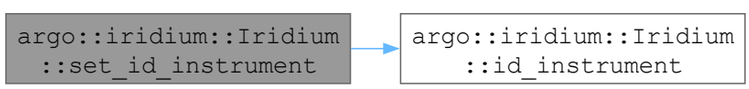
Parameters

in	*id_instrument*	The unique instrument identifier or instrument serial number

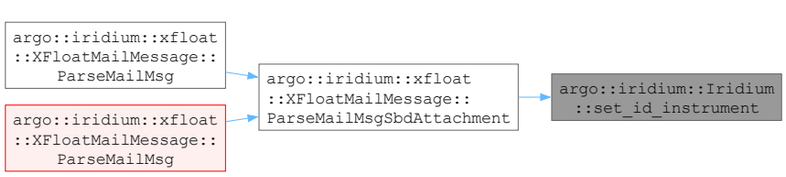
References argo::iridium::Iridium::id_instrument(), argo::iridium::Iridium::id_instrument_, and util::thread::RecursiveLockable::mutex_.

Referenced by argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgSbdAttachment().

Here is the call graph for this function:



Here is the caller graph for this function:



10.91.3.76 set_id_wmo()

```
void argo::iridium::Iridium::set_id_wmo (
            int id_wmo)  [inline], [protected], [inherited]
```

Set the unique World Meteorological Organization (WMO) identifier.

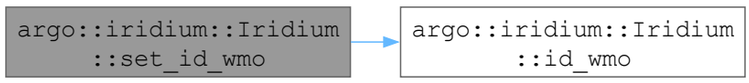
Parameters

in	*id_wmo*	The unique World Meteorological Organization (WMO) identifier.

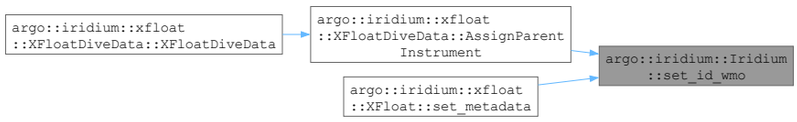
References argo::iridium::Iridium::id_wmo(), argo::iridium::Iridium::id_wmo_, and util::thread::RecursiveLockable::mutex_.

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignParentInstrument(), and argo::iridium::xfloat::XFloat::set_metadata().

Here is the call graph for this function:



Here is the caller graph for this function:



10.91.3.77 set_local_path()

```
void argo::iridium::xfloat::XFloatMetadata::set_local_path (
            const string & local_path)  [inline], [protected]
```

Set the path to the metadata file on the local filesystem.

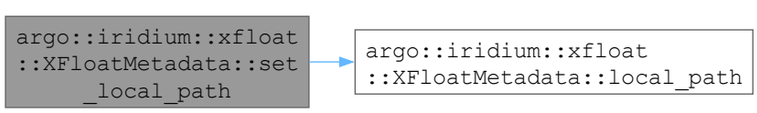
Parameters

in	*local_path*	The path to the metadata file on the local filesystem.

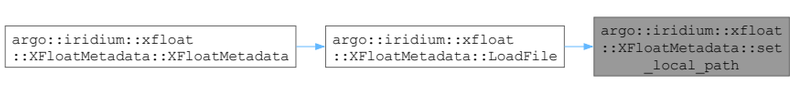
References local_path(), local_path_, and util::thread::RecursiveLockable::mutex_.

Referenced by LoadFile().

Here is the call graph for this function:



Here is the caller graph for this function:



10.91.3.78 set_manufacturer()

```
void argo::iridium::xfloat::XFloatMetadata::set_manufacturer (
            const string & man)  [inline], [protected]
```

Set the instrument manufacturer.

Parameters

in	*man*	The instrument manufacturer.

References manufacturer_, and util::thread::RecursiveLockable::mutex_.

10.91.3.79 set_position_datetime()

```
void util::position::Position::set_position_datetime (
            const ptime & datetime)  [inline], [protected], [inherited]
```

Set the date/time of the position.

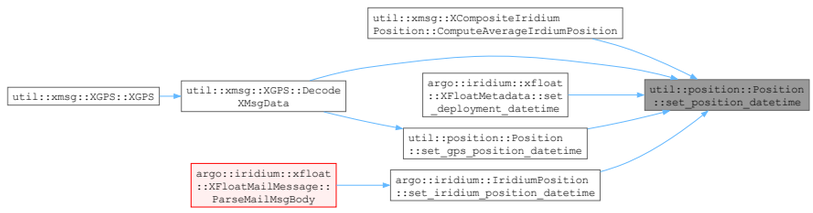
Parameters

in	*datetime*	The date/time of the position to set.

References util::position::Position::position_datetime_.

Referenced by util::xmsg::XCompositeIridiumPosition::ComputeAverageIrdiumPosition(), util::xmsg::XGPS::DecodeXMsgData(), set_deployment_datetime(), util::position::Position::set_gps_position_datetime(), and argo::iridium::IridiumPosition::set_iridium_position_d

Here is the caller graph for this function:



10.91.3.80 set_position_is_valid()

```
void util::position::Position::set_position_is_valid (
            bool is_valid = true)  [inline], [protected], [inherited]
```

Set the position is valid flag.

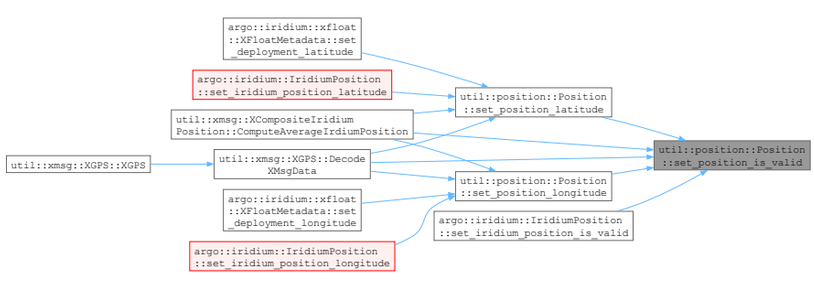
Parameters

in	*is_valid*	TRUE if the position is valid, or FALSE otherwise.

References util::position::Position::position_is_valid_.

Referenced by util::xmsg::XCompositeIridiumPosition::ComputeAverageIrdiumPosition(), util::xmsg::XGPS::DecodeXMsgData(), argo::iridium::IridiumPosition::set_iridium_position_is_valid(), util::position::Position::set_position_latitude(), and util::position::Position::set_position_longitude().

Here is the caller graph for this function:



10.91.3.81 set_position_latitude()

```
void util::position::Position::set_position_latitude (
            double lat)  [inline], [protected], [inherited]
```

Set the latitude of the position, in decimal degrees.

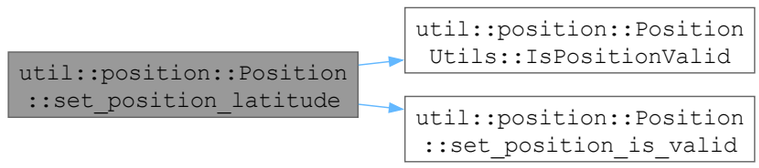
Parameters

in	*lat*	The latitude of the position, in decimal degrees.

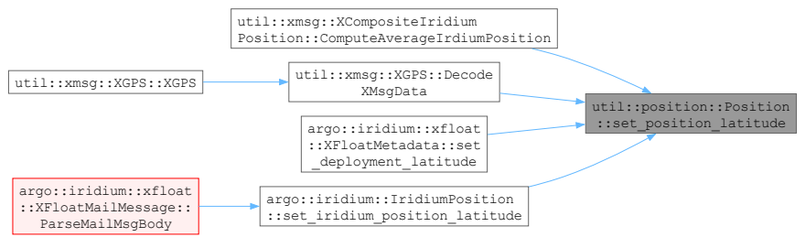
References util::position::PositionUtils::IsPositionValid(), util::position::Position::position_latitude_, util::position::Position::position_longitud and util::position::Position::set_position_is_valid().

Referenced by util::xmsg::XCompositeIridiumPosition::ComputeAverageIrdiumPosition(), util::xmsg::XGPS::DecodeXMsgData(), set_deployment_latitude(), and argo::iridium::IridiumPosition::set_iridium_position_latitude().

Here is the call graph for this function:



Here is the caller graph for this function:



10.91.3.82 set_position_longitude()

```
void util::position::Position::set_position_longitude (
            double lon)  [inline], [protected], [inherited]
```

Set the longitude of the position, in decimal degrees.

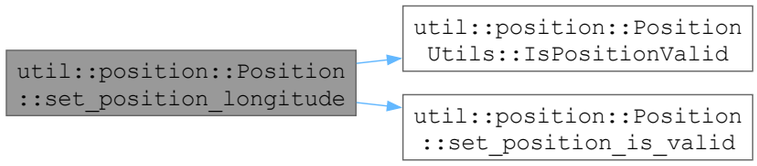
Parameters

in	*lon*	The longitude of the position, in decimal degrees.

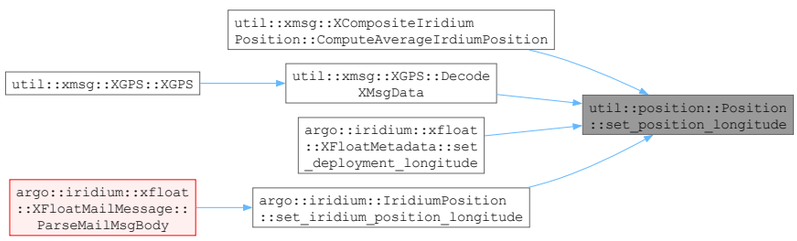
References util::position::PositionUtils::IsPositionValid(), util::position::Position::position_latitude_, util::position::Position::position_longitud and util::position::Position::set_position_is_valid().

Referenced by util::xmsg::XCompositeIridiumPosition::ComputeAverageIrdiumPosition(), util::xmsg::XGPS::DecodeXMsgData(), set_deployment_longitude(), and argo::iridium::IridiumPosition::set_iridium_position_longitude().

Here is the call graph for this function:



Here is the caller graph for this function:



10.91.3.83 set_principal_investigators()

```
void argo::iridium::xfloat::XFloatMetadata::set_principal_investigators (
            const string & pis)  [inline], [protected]
```

Set the names of the principal investigator(s).

Parameters

in	*pis*	The names of the principal investigator(s).

References util::thread::RecursiveLockable::mutex_, and principal_investigators_.

10.91.3.84 set_principal_investigators_address()

```
void argo::iridium::xfloat::XFloatMetadata::set_principal_investigators_address (
            const string & addr)  [inline], [protected]
```

Set the address of the principal investigator(s).

Parameters

in	*addr*	The address of the principal investigator(s).

References util::thread::RecursiveLockable::mutex_, and principal_investigators_address_.

10.91.3.85 set_principal_investigators_originating_country()

```
void argo::iridium::xfloat::XFloatMetadata::set_principal_investigators_originating_country (
            const string & country)  [inline], [protected]
```

Set the originating country of the instrument.

Parameters

in	*country*	The originating country of the instrument.

References util::thread::RecursiveLockable::mutex_, and principal_investigators_originating_country_.

10.91.3.86 set_wmo_instrument_type()

```
void argo::iridium::xfloat::XFloatMetadata::set_wmo_instrument_type (
            int type)  [inline], [protected]
```

Set the WMO instrument type identifier.

Parameters

in	*type*	The WMO instrument type identifier.

References util::thread::RecursiveLockable::mutex_, and wmo_instrument_type_.

10.91.3.87 set_wmo_recorder_type()

```
void argo::iridium::xfloat::XFloatMetadata::set_wmo_recorder_type (
            int type)  [inline], [protected]
```

Set the WMO recorder type identifier.

Parameters

in	*type*	The WMO recorder type identifier.

References util::thread::RecursiveLockable::mutex_, and wmo_recorder_type_.

10.91.3.88 UnlockObject()

```
void util::thread::RecursiveLockable::UnlockObject ()  [inline], [inherited]
```

Unlock the object.

Exceptions

util::log::MessageException	if an error occurs.

References CATCH_BLOCK, and util::thread::RecursiveLockable::mutex_.

10.91.3.89 wmo_instrument_type()

```
int argo::iridium::xfloat::XFloatMetadata::wmo_instrument_type () const  [inline]
```

Get the WMO instrument type identifier.

Returns

The WMO instrument type identifier.

References util::thread::RecursiveLockable::mutex_, and wmo_instrument_type_.

10.91.3.90 wmo_recorder_type()

```
int argo::iridium::xfloat::XFloatMetadata::wmo_recorder_type () const  [inline]
```

Get the WMO recorder type identifier.

Returns

The WMO recorder type identifier.

References util::thread::RecursiveLockable::mutex_, and wmo_recorder_type_.

10.91.4 Friends And Related Symbol Documentation

10.91.4.1 operator>>

```
istream & operator>> (
            istream & in,
            XFloatMetadata & metadata)  [friend]
```

Stream extraction operator overload function for the XFloatMetadata class.

Parameters

in	*in*	The input stream.
in	*metadata*	The metadata object.

Exceptions

util::log::MessageException	if the stream extraction fails.

Returns

The reference to the input stream, `istream`.

Todo Implement the functionality for parsing additional metadata field from stream

10.91.4.2 util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription

```
options_description util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription
() [friend]
```

Friend function for setting the value of some of the static configuration options that are members of this class.

10.91.5 Member Data Documentation

10.91.5.1 address_

```
std::string util::mysql::LocalMysqlConfiguration::address_  [static], [private], [inherited]
```

The SQL server qualified name or IP address string.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.91.5.2 comment_

```
string argo::iridium::xfloat::XFloatMetadata::comment_  [private]
```

The instrument comment(s).

Referenced by comment(), and set_comment().

10.91.5.3 deployer_

```
string argo::iridium::xfloat::XFloatMetadata::deployer_  [private]
```

The name of the instrument deployer(s).

Referenced by deployer(), and set_deployer().

10.91.5.4 deployer_address_

```
string argo::iridium::xfloat::XFloatMetadata::deployer_address_  [private]
```

The address of the instrument deployer(s).

Referenced by deployer_address(), and set_deployer_address().

10.91.5.5 deployment_cruiseId_

```
string argo::iridium::xfloat::XFloatMetadata::deployment_cruiseId_  [private]
```

The unique identifier of the deployment or launch cruise.

Referenced by deployment_cruise_id(), and set_deployment_cruise_id().

10.91.5.6 deployment_datetime_status_

```
string argo::iridium::xfloat::XFloatMetadata::deployment_datetime_status_  [private]
```

The status of the accompanying date/time of the instrument deployment or launch.

Referenced by deployment_datetime_status(), and set_deployment_datetime_status().

10.91.5.7 deployment_platform_

```
string argo::iridium::xfloat::XFloatMetadata::deployment_platform_  [private]
```

The deployment or launch platform.

Referenced by deployment_platform(), and set_deployment_platform().

10.91.5.8 deployment_position_status_

```
string argo::iridium::xfloat::XFloatMetadata::deployment_position_status_  [private]
```

The status of the accompanying date/time of the instrument deployment or launch.

Referenced by deployment_position_status(), and set_deployment_position_status().

10.91.5.9 deployment_type_

```
string argo::iridium::xfloat::XFloatMetadata::deployment_type_  [private]
```

The deployment or launch type.

Referenced by deployment_type(), and set_deployment_type().

10.91.5.10 EARTH_RADIUS_KM_EQUATORIAL

```
const double util::position::PositionUtils::EARTH_RADIUS_KM_EQUATORIAL = 6378.1  [static], [private],
[inherited]
```

The equatorial radius of the earth, in kilometers.

Referenced by util::position::PositionUtils::CalculateEllipsoidHeight().

10.91.5.11 EARTH_RADIUS_KM_POLAR

```
const double util::position::PositionUtils::EARTH_RADIUS_KM_POLAR = 6356.1  [static], [private],
[inherited]
```

The polar radius of the earth, in kilometers.

Referenced by util::position::PositionUtils::CalculateEllipsoidHeight().

10.91.5.12 id_dac_

```
int argo::iridium::Iridium::id_dac_  [private], [inherited]
```

The unique Data Assembly Center (DAC) identifier.

Referenced by argo::iridium::Iridium::id_dac(), and argo::iridium::Iridium::set_id_dac().

10.91.5.13 id_imei_

```
long argo::iridium::Iridium::id_imei_  [private], [inherited]
```

The unique International Mobile Equipment Identifier (IMEI).

Referenced by argo::iridium::Iridium::id_imei(), and argo::iridium::Iridium::set_id_imei().

10.91.5.14 id_instrument_

```
int argo::iridium::Iridium::id_instrument_  [private], [inherited]
```

The unique instrument identifier or instrument serial number.

Referenced by argo::iridium::Iridium::id_instrument(), and argo::iridium::Iridium::set_id_instrument().

10.91.5.15 id_wmo_

```
int argo::iridium::Iridium::id_wmo_  [private], [inherited]
```

The unique World Meteorological Organization (WMO) identifier.

Referenced by argo::iridium::Iridium::id_wmo(), and argo::iridium::Iridium::set_id_wmo().

10.91.5.16 local_path_

```
string argo::iridium::xfloat::XFloatMetadata::local_path_  [private]
```

The path to the metadata file on the local filesystem.

Referenced by local_path(), and set_local_path().

10.91.5.17 manufacturer_

```
string argo::iridium::xfloat::XFloatMetadata::manufacturer_  [private]
```

The instrument manufacturer.

Referenced by manufacturer(), and set_manufacturer().

10.91.5.18 metadata_directory_path_

```
path argo::iridium::xfloat::XFloatMetadata::metadata_directory_path_  [static], [private]
```

The path to metadata directory on the local filesystem.

This path indicates the directory containing the instrument metadata files. The value of this field is set during application initialization as a result of parsing the command-line arguments and configuration file arguments.

After the run-time configuration settings are parsed, but before the completion of application initialization, the metadata files are read from this directory and used to generate one instance of this class for each metadata file read. The resulting object are assigned to the corresponding instrument objects (XFloat), and are save to the SQL database.

Referenced by ListMetadataFiles().

10.91.5.19 mutex_

```
recursive_mutex util::thread::RecursiveLockable::mutex_  [mutable], [protected], [inherited]
```

Recursive mutual exclusion lock for synchronizing operations.

Referenced by util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AggregateS
argo::iridium::xfloat::XFloat::argo_instrument_type(), argo::iridium::xfloat::XFloatDiveData::argo_instrument_type(),
argo::iridium::xfloat::XFloatDiveData::AssignArgoData(), argo::iridium::xfloat::XFloatDiveData::AssignAuxTemperatureBinned(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignAuxTemperatureMe
argo::iridium::xfloat::XFloatDiveData::AssignDriftPressure(), argo::iridium::xfloat::XFloatDiveData::AssignDriftSalinity(),
argo::iridium::xfloat::XFloatDiveData::AssignDriftTemperatre(), argo::iridium::xfloat::XFloatDiveData::AssignEngineeringAbort(),
argo::iridium::xfloat::XFloatDiveData::AssignEngineeringAscent(), argo::iridium::xfloat::XFloatDiveData::AssignEngineeringBitFail(),
argo::iridium::xfloat::XFloatDiveData::AssignEngineeringBitPass(), argo::iridium::xfloat::XFloatDiveData::AssignEngineeringDescent(),
argo::iridium::xfloat::XFloatDiveData::AssignEngineeringDiagnostic(), argo::iridium::xfloat::XFloatDiveData::AssignEngineeringDrift(),
argo::iridium::xfloat::XFloatDiveData::AssignEngineeringIce(), argo::iridium::xfloat::XFloatDiveData::AssignEngineeringStandard(),
argo::iridium::xfloat::XFloatDiveData::AssignEngineeringSurface(), argo::iridium::xfloat::XFloatDiveData::AssignFallTimeSeries(),
argo::iridium::xfloat::XFloatDiveData::AssignGps(), argo::iridium::xfloat::XFloatDiveData::AssignMailMessage(),
argo::iridium::xfloat::XFloatDiveData::AssignMissionCmdEchoback(), argo::iridium::xfloat::XFloatDiveData::AssignParentInstrument(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignPressureMessage(
argo::iridium::xfloat::XFloatDiveData::AssignProfilePressureBinned(), argo::iridium::xfloat::XFloatDiveData::AssignProfilePressureUnbinn
argo::iridium::xfloat::XFloatDiveData::AssignProfileSalinityBinned(), argo::iridium::xfloat::XFloatDiveData::AssignProfileSalinityUnbinned()
argo::iridium::xfloat::XFloatDiveData::AssignProfileTemperatureBinned(), argo::iridium::xfloat::XFloatDiveData::AssignProfileTemperatureU
argo::iridium::xfloat::XFloatDiveData::AssignPumpTimeSeries(), argo::iridium::xfloat::XFloatDiveData::AssignRiseTimeSeries(),
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignSalinityMessage(),
argo::iridium::xfloat::XFloatDiveData::AssignSurfaceTimeSeries(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_C
argo::iridium::xfloat::XFloatDiveData::AssignValveData(), argo::iridium::xfloat::XFloatDiveData::CheckCompleteness(),
comment(), argo::iridium::xfloat::XFloatDiveData::ComputePressureOffsetFromSurfaceTimeSeries(), deployer(),
deployer_address(), deployment_cruise_id(), deployment_datetime(), deployment_datetime_status(), deployment_latitude(),
deployment_longitude(), deployment_platform(), deployment_position_status(), deployment_type(), argo::iridium::xfloat::XFloat::fw_versio
argo::iridium::xfloat::XFloat::gain_pressure(), argo::iridium::xfloat::XFloat::gain_pressure_surface(), argo::iridium::xfloat::XFloat::gain_sali
argo::iridium::xfloat::XFloat::gain_salinity_surface(), argo::iridium::xfloat::XFloat::gain_temperature(), argo::iridium::xfloat::XFloat::gain_te
argo::iridium::xfloat::XFloatDiveData::GenerateCompositeFallTimeSeriesData(), argo::iridium::xfloat::XFloatDiveData::GenerateComposit
argo::iridium::xfloat::XFloatDiveData::GenerateCompositePumpTimeSeriesData(), argo::iridium::xfloat::XFloatDiveData::GenerateCompo
argo::iridium::xfloat::XFloatDiveData::GenerateListingString(), util::xmsg::XCompositeDrift::GenerateListingString(),
util::xmsg::XCompositeHiResProfile::GenerateListingString(), util::xmsg::XCompositeProfile::GenerateListingString(),
argo::iridium::xfloat::XFloatDiveData::GenerateOrderedMomsnList(), argo::iridium::xfloat::XFloatDiveData::GeneratePhyString(),
util::xmsg::XCompositeDrift::GeneratePhyString(), util::xmsg::XCompositeHiResProfile::GeneratePhyString(), util::xmsg::XCompositeProf
argo::iridium::xfloat::XFloatDiveData::GetDiveDataMailMessagesFromDatabase(), argo::iridium::xfloat::XFloatDiveData::GetGainsAndOff
argo::iridium::xfloat::XFloatDiveData::GetGpsPositionEndOfDive(), argo::iridium::xfloat::XFloatDiveData::GetSurfaceGainsAndOffsets(),
argo::iridium::xfloat::XFloatDiveData::HasBinnedProfileData(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLA
argo::iridium::xfloat::XFloatDiveData::HasContiguousMomsns(), argo::iridium::xfloat::XFloatDiveData::HasContiguousPids(),
argo::iridium::xfloat::XFloatDiveData::HasDiveData(), argo::iridium::xfloat::XFloatDiveData::HasDriftData(), argo::iridium::xfloat::XFloatDiv
argo::iridium::xfloat::XFloatDiveData::HasFallRiseData(), argo::iridium::xfloat::XFloatDiveData::HasPositionData(),
argo::iridium::xfloat::XFloatDiveData::HasUnbinnedProfileData(), argo::iridium::xfloat::XFloatDiveData::HasValidGpsPosition(),

argo::iridium::Iridium::id_dac(), argo::iridium::xfloat::XFloatDiveData::id_dive(), argo::iridium::xfloat::XFloatMailMessage::id_dive(), argo::iridium::Iridium::id_imei(), argo::iridium::Iridium::id_instrument(), argo::iridium::xfloat::XFloatMailMessage::id_msg(), argo::iridium::xfloat::XFloatMailMessage::id_msg_uid(), argo::iridium::xfloat::XFloatMailMessage::id_sbd_momsn(), argo::iridium::xfloat::XFloatMailMessage::id_sbd_mtmsn(), argo::iridium::xfloat::XFloatMailMessage::id_sbd_packet(), argo::iridium::Iridium::id_wmo(), argo::iridium::xfloat::XFloatDiveData::intermediate_listing_file_path(), util::xmsg::AbstractXSensorDataCo util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::IsDataComplete(), local_path(), util::thread::RecursiveLockable::LockObject(), manufacturer(), argo::iridium::xfloat::XFloat::metadata(), argo::iridium::xfloat::XFloatMailMessage::msg_body_is_valid(), argo::iridium::xfloat::XFloatMailMessage::msg_body_text(), argo::iridium::xfloat::XFloatMailMessage::msg_header_datetime(), argo::iridium::xfloat::XFloatMailMessage::msg_header_from(), argo::iridium::xfloat::XFloatMailMessage::msg_header_is_valid(), argo::iridium::xfloat::XFloatMailMessage::msg_header_raw_text(), argo::iridium::xfloat::XFloatMailMessage::msg_header_subject(), argo::iridium::xfloat::XFloatMailMessage::msg_is_valid(), argo::iridium::xfloat::XFloatMailMessage::msg_raw_text(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, T util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::num_saln_samples(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::num_temp_samples(), argo::iridium::xfloat::XFloat::offset_pressure(), argo::iridium::xfloat::XFloat::offset_pressure_surface(), argo::iridium::xfloat::XFloat::offset_ argo::iridium::xfloat::XFloat::offset_salinity_surface(), argo::iridium::xfloat::XFloat::offset_temperature(), argo::iridium::xfloat::XFloat::offset argo::iridium::xfloat::XFloatMailMessage::ParseMailMsg(), argo::iridium::xfloat::XFloatMailMessage::ParseMailMsg(), argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgBody(), argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgHeader(), argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgSbdAttachment(), argo::iridium::xfloat::XFloat::phy_file_generation_is_enabled(), argo::iridium::xfloat::XFloatDiveData::phy_file_path(), principal_investigators(), principal_investigators_address(), principal_investigators_originating_country(), argo::iridium::xfloat::XFloatMailMessage::raw_file_local_path(), util::xmsg::AbstractXSensor argo::iridium::xfloat::XFloatDiveData::ResetData(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_C util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SamplePresAsFloat(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleSaln(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleSalnAsFloat(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleTemp(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleTempAsFloat(), argo::iridium::xfloat::XFloatDiveData::SaveObjectToDatabase(), argo::iridium::xfloat::XFloatMailMessage::SaveObjectToDatabase(), SaveObjectToDatabase(), argo::iridium::xfloat::XFloatMailMessage::sbd_data(), argo::iridium::xfloat::XFloatMailMessage::sbd_data_is_va argo::iridium::xfloat::XFloatMailMessage::sbd_file_local_path(), argo::iridium::xfloat::XFloatMailMessage::sbd_file_name(), argo::iridium::xfloat::XFloatMailMessage::sbd_file_size_bytes(), argo::iridium::xfloat::XFloatMailMessage::sbd_session_status_code(), argo::iridium::xfloat::XFloatMailMessage::sbd_session_status_string(), argo::iridium::xfloat::XFloatDiveData::SensorGainsAndOffsetsAreI util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::set_aggregated_data_vali argo::iridium::xfloat::XFloat::set_argo_instrument_type(), argo::iridium::xfloat::XFloatDiveData::set_argo_instrument_type(), set_comment(), set_deployer(), set_deployer_address(), set_deployment_cruise_id(), set_deployment_datetime(), set_deployment_datetime_status(), set_deployment_latitude(), set_deployment_longitude(), set_deployment_platform(), set_deployment_position_status(), set_deployment_type(), argo::iridium::xfloat::XFloat::set_fw_version(), argo::iridium::xfloat::XFloat::set_ argo::iridium::xfloat::XFloat::set_gain_pressure_surface(), argo::iridium::xfloat::XFloat::set_gain_salinity(), argo::iridium::xfloat::XFloat::set argo::iridium::xfloat::XFloat::set_gain_temperature(), argo::iridium::xfloat::XFloat::set_gain_temperature_surface(), argo::iridium::Iridium::set_id_dac(), argo::iridium::xfloat::XFloatDiveData::set_id_dive(), argo::iridium::xfloat::XFloatMailMessage::set_id_d argo::iridium::Iridium::set_id_imei(), argo::iridium::Iridium::set_id_instrument(), argo::iridium::xfloat::XFloatMailMessage::set_id_msg(), argo::iridium::xfloat::XFloatMailMessage::set_id_msg_uid(), argo::iridium::xfloat::XFloatMailMessage::set_id_sbd_momsn(), argo::iridium::xfloat::XFloatMailMessage::set_id_sbd_mtmsn(), argo::iridium::xfloat::XFloatMailMessage::set_id_sbd_packet(), argo::iridium::Iridium::set_id_wmo(), argo::iridium::xfloat::XFloatDiveData::set_intermediate_listing_file_path(), set_local_path(), set_manufacturer(), argo::iridium::xfloat::XFloat::set_metadata(), argo::iridium::xfloat::XFloatMailMessage::set_msg_body_is_valid(), argo::iridium::xfloat::XFloatMailMessage::set_msg_body_text(), argo::iridium::xfloat::XFloatMailMessage::set_msg_header_datetime(), argo::iridium::xfloat::XFloatMailMessage::set_msg_header_from(), argo::iridium::xfloat::XFloatMailMessage::set_msg_header_is_valid(), argo::iridium::xfloat::XFloatMailMessage::set_msg_header_raw_text(), argo::iridium::xfloat::XFloatMailMessage::set_msg_header_subjec argo::iridium::xfloat::XFloatMailMessage::set_msg_is_valid(), argo::iridium::xfloat::XFloatMailMessage::set_msg_raw_text(), argo::iridium::xfloat::XFloat::set_offset_pressure(), argo::iridium::xfloat::XFloat::set_offset_pressure_surface(), argo::iridium::xfloat::XFloat argo::iridium::xfloat::XFloat::set_offset_salinity_surface(), argo::iridium::xfloat::XFloat::set_offset_temperature(), argo::iridium::xfloat::XFloat::set_offset_temperature_surface(), argo::iridium::xfloat::XFloat::set_phy_file_generation_is_enabled(), argo::iridium::xfloat::XFloatDiveData::set_phy_file_path(), set_principal_investigators(), set_principal_investigators_address(), set_principal_investigators_originating_country(), argo::iridium::xfloat::XFloatMailMessage::set_raw_file_local_path(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_data(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_data_is_valid(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_file_local_path(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_file_name(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_file_size_bytes(), argo::iridium::xfloat::XFloatMailMessage::set_sbd_session_status_c argo::iridium::xfloat::XFloatMailMessage::set_sbd_session_status_string(), argo::iridium::xfloat::XFloat::set_start_of_mission_datetime(), set_wmo_instrument_type(), set_wmo_recorder_type(), argo::iridium::xfloat::XFloat::set_xmsg_version_major(), argo::iridium::xfloat::XFloat::set_xmsg_version_minor(), argo::iridium::xfloat::XFloat::start_of_mission_datetime(), util::thread::RecursiveLockable::UnlockObject(), argo::iridium::xfloat::XFloatDiveData::UpdateDerivedSurfaceDataFields(), wmo_instrument_type(), wmo_recorder_type(), argo::iridium::xfloat::XFloatDiveData::WriteListingFile(), argo::iridium::xfloat::XFloatDiveDa argo::iridium::xfloat::XFloatMailMessage::WriteSbdFile(), argo::iridium::xfloat::XFloat::xmsg_version_major(), and argo::iridium::xfloat::XFloat::xmsg_version_minor().

10.91.5.20 password_

```
std::string util::mysql::LocalMysqlConfiguration::password_  [static], [private], [inherited]
```

The SQL server account password string.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.91.5.21 port_

```
unsigned int util::mysql::LocalMysqlConfiguration::port_  [static], [private], [inherited]
```

The SQL server port number.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.91.5.22 position_datetime_

```
ptime util::position::Position::position_datetime_  [private], [inherited]
```

The date and time of the position.

Referenced by util::position::Position::position_datetime(), util::position::Position::position_datetime_as_iso8601_string(), and util::position::Position::set_position_datetime().

10.91.5.23 position_is_valid_

```
bool util::position::Position::position_is_valid_  [private], [inherited]
```

The valid position indicator.

Referenced by util::position::Position::position_is_valid(), and util::position::Position::set_position_is_valid().

10.91.5.24 position_latitude_

```
double util::position::Position::position_latitude_  [private], [inherited]
```

The latitude of the position, in decimal degrees.

Referenced by util::position::Position::position_latitude(), util::position::Position::postition_as_iso6709_string(), util::position::Position::set_position_latitude(), and util::position::Position::set_position_longitude().

10.91.5.25 position_longitude_

```
double util::position::Position::position_longitude_  [private], [inherited]
```

The longitude of the position, in decimal degrees.

Referenced by util::position::Position::position_longitude(), util::position::Position::postition_as_iso6709_string(), util::position::Position::set_position_latitude(), and util::position::Position::set_position_longitude().

10.91.5.26 principal_investigators_

```
string argo::iridium::xfloat::XFloatMetadata::principal_investigators_  [private]
```

The names of the principal investigator(s).

Referenced by principal_investigators(), and set_principal_investigators().

10.91.5.27 principal_investigators_address_

```
string argo::iridium::xfloat::XFloatMetadata::principal_investigators_address_  [private]
```

The address of the principal investigator(s).

Referenced by principal_investigators_address(), and set_principal_investigators_address().

10.91.5.28 principal_investigators_originating_country_

```
string argo::iridium::xfloat::XFloatMetadata::principal_investigators_originating_country_  [private]
```

The originating country of the instrument.

Referenced by principal_investigators_originating_country(), and set_principal_investigators_originating_country().

10.91.5.29 protocol_

```
std::string util::mysql::LocalMysqlConfiguration::protocol_  [static], [private], [inherited]
```

The protocol to use in the SQL server connection e.g.

"tcp".

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.91.5.30 schema_

```
std::string util::mysql::LocalMysqlConfiguration::schema_  [static], [private], [inherited]
```

The SQL server schema name.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.91.5.31 sql_connection_

```
thread_specific_ptr<Connection> util::mysql::MysqlUtils::sql_connection_  [static], [private],
[inherited]
```

Tthe thread-specific pointer to MySQL connection object.

Thread-specific MySQL connection, released by `MysqlUtils::CleanupSqlConnection()` when the thread exits.

Referenced by util::mysql::MysqlUtils::CloseSqlConnection(), and util::mysql::MysqlUtils::GetSqlConnection().

10.91.5.32 sql_driver_

```
thread_specific_ptr<MySQL_Driver> util::mysql::MysqlUtils::sql_driver_  [static], [private],
[inherited]
```

The thread-specific pointer to MySQL driver object.

Thread-specific reference to the MySQL driver.

Its cleanup function, `MysqlUtils::CleanupSqlDriver()`, does not delete the shared driver instance.

Referenced by util::mysql::MysqlUtils::GetSqlDriver().

10.91.5.33 sql_table_description_

```
const util::mysql::SqlTableDescription< XFloatMetadata > argo::iridium::xfloat::XFloatMetadata↩
::sql_table_description_  [static], [private]
```

The SQL table description object.

This SQL table description provides the details needed to map this object to a relational database table. It utilizes the accessor methods of the class and the details of the SQL table column to (1) create an SQL INSERT statement to save the object into the database, (2) populate one or more objects' fields as a result of an SQL SELECT query, or (3) query the SQL database to access one or more of the values that are stored the the SQL table corresponding to this object.

Referenced by SaveObjectToDatabase().

10.91.5.34 user_

`std::string util::mysql::LocalMysqlConfiguration::user_ [static], [private], [inherited]`

The SQL server account username string.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.91.5.35 wmo_instrument_type_

`int argo::iridium::xfloat::XFloatMetadata::wmo_instrument_type_ [private]`

The WMO instrument type identifier.

Note

This value corresponds to the WMO reference table 1770.

Referenced by set_wmo_instrument_type(), and wmo_instrument_type().

10.91.5.36 wmo_recorder_type_

`int argo::iridium::xfloat::XFloatMetadata::wmo_recorder_type_ [private]`

The WMO recorder type identifier.

Note

This value corresponds to the WMO reference table 4770.

Referenced by set_wmo_recorder_type(), and wmo_recorder_type().

The documentation for this class was generated from the following files:

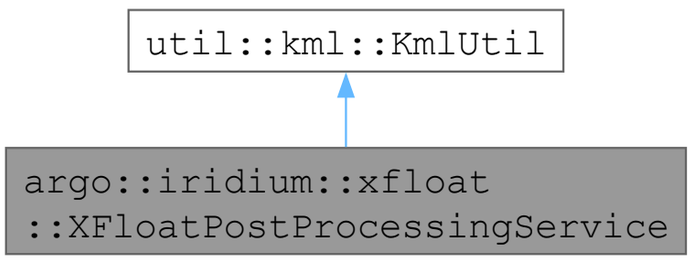
- data/XFloatMetadata.h
- data/XFloatMetadata.cpp

10.92 argo::iridium::xfloat::XFloatPostProcessingService Class Reference

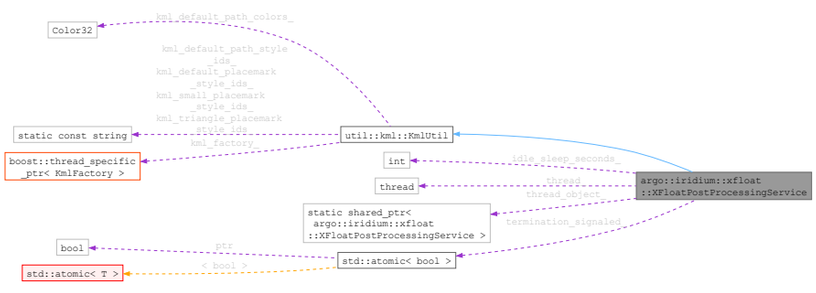
The post-processing analysis thread class.

`#include <stage4b_PostProcessingThread.h>`

Inheritance diagram for argo::iridium::xfloat::XFloatPostProcessingService:



Collaboration diagram for argo::iridium::xfloat::XFloatPostProcessingService:



Public Member Functions

- XFloatPostProcessingService ()

  The constructor.
- ~XFloatPostProcessingService ()

  The destructor.
- void operator() () ATTR_NO_EXCEPTIONS

  Functor operator overload function: the main entry point for the post-processing analysis thread.

Static Public Member Functions

- static void CloseKmlFactory ()

  Release this thread's KML factory pointer.
- static const Color32 & GetDefaltKmlPathColor (int id)

  Get the palette colour for an index (an instrument ID, for example), modulo the palette size.
- static const string & GetDefaltKmlPathStyleId (int id)

  Get the path (line) style ID for an index, modulo the palette size.
- static const string & GetDefaltKmlPlacemarkStyleId (int id)

  Get the large placemark style ID for an index, modulo the palette size.
- static thread_specific_ptr< KmlFactory > & GetKmlFactory ()

  Get the libkml factory, initializing it for this thread if necessary.
- static const string & GetSmallKmlPlacemarkStyleId (int id)

  Get the small placemark style ID for an index, modulo the palette size.
- static const string & GetTriangleKmlPlacemarkStyleId (int id)

  Get the triangle placemark style ID for an index, modulo the palette size.
- static void SetAllDefaultKmlSyles (const DocumentPtr &document)

  Add the default styles for every palette colour to a KML document.
- static void SetDefaultKmlStyle (const DocumentPtr &document, unsigned int style_idx)

  Add the four default styles for one palette colour to a KML document.
- static void StartWorkerThread () ATTR_NO_EXCEPTIONS

  Instantiate the post-processing analysis worker thread.
- static void TerminateWorkerThread () ATTR_NO_EXCEPTIONS

  Terminate the mail post-processing analysis thread.

Static Private Member Functions

- static void **set_termination_signaled** (bool termination_signaled=true)

Static Private Attributes

- static unsigned int idle_sleep_seconds_

  Duration to sleep when not performing post-processing, in seconds.
- static const Color32 kml_default_path_colors_ [KML_NUM_DFLT_PATH_COLORS]

  The colour palette for instrument paths and placemarks, indexed by style index.
- static const string kml_default_path_style_ids_ [KML_NUM_DFLT_PATH_COLORS]

  The KML style IDs of the path (line) styles, one per palette colour.
- static const string kml_default_placemark_style_ids_ [KML_NUM_DFLT_PATH_COLORS]

  The KML style IDs of the large placemark styles, one per palette colour.
- static thread_specific_ptr< KmlFactory > kml_factory_

  The libkml factory used to create KML elements, held per thread.
- static const string kml_small_placemark_style_ids_ [KML_NUM_DFLT_PATH_COLORS]

  The KML style IDs of the small placemark styles, one per palette colour.
- static const string kml_triangle_placemark_style_ids_ [KML_NUM_DFLT_PATH_COLORS]

  The KML style IDs of the triangle placemark styles, one per palette colour.
- static std::atomic< bool > termination_signaled_

  The termination signaled flag.
- static thread thread_

  The post-processing analysis thread group.
- static shared_ptr< XFloatPostProcessingService > thread_object_

  The post-processing analysis thread objects.

Friends

- options_description util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription ()

  Friend function for populating some of the static members of this class from the configuration file options.

10.92.1 Detailed Description

The post-processing analysis thread class.

This class is defines the operation of the post-processing analysis thread(s), which are the fourth stage of the processing pipeline. The post-processing analysis thread(s) are responsible for generating plots, Matlab, and NetCDF files as well as performing some analysis of the decoded telemetry fields in order to derive performance metrics of the instrument.

10.92.2 Constructor & Destructor Documentation

10.92.2.1 XFloatPostProcessingService()

```
argo::iridium::xfloat::XFloatPostProcessingService::XFloatPostProcessingService ()  [inline]
```

The constructor.

10.92.2.2 ~XFloatPostProcessingService()

```
argo::iridium::xfloat::XFloatPostProcessingService::~XFloatPostProcessingService ()  [inline]
```

The destructor.

10.92.3 Member Function Documentation

10.92.3.1 CloseKmlFactory()

```
static void util::kml::KmlUtil::CloseKmlFactory ()  [inline], [static], [inherited]
```

Release this thread's KML factory pointer.

Warning

This deletes libkml's shared factory (see kml_factory_). It is not currently called.

Exceptions

util::log::MessageException	if an error occurs.

References CATCH_BLOCK, and util::kml::KmlUtil::kml_factory_.

10.92.3.2 GetDefaltKmlPathColor()

```
static const Color32 & util::kml::KmlUtil::GetDefaltKmlPathColor (
            int id)  [inline], [static], [inherited]
```

Get the palette colour for an index (an instrument ID, for example), modulo the palette size.

Parameters

in	*id*	The index.

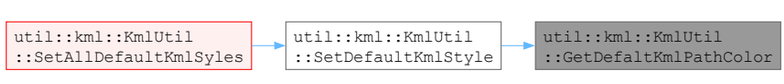
Returns

The colour.

References util::kml::KmlUtil::kml_default_path_colors_.

Referenced by util::kml::KmlUtil::SetDefaultKmlStyle().

Here is the caller graph for this function:



10.92.3.3 GetDefaltKmlPathStyleId()

```
static const string & util::kml::KmlUtil::GetDefaltKmlPathStyleId (
            int id)  [inline], [static], [inherited]
```

Get the path (line) style ID for an index, modulo the palette size.

Parameters

in	*id*	The index.

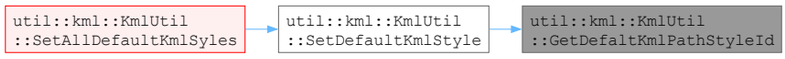
Returns

The KML style ID.

References util::kml::KmlUtil::kml_default_path_style_ids_.

Referenced by util::kml::KmlUtil::SetDefaultKmlStyle().

Here is the caller graph for this function:



10.92.3.4 GetDefaltKmlPlacemarkStyleId()

```
static const string & util::kml::KmlUtil::GetDefaltKmlPlacemarkStyleId (
            int id)  [inline], [static], [inherited]
```

Get the large placemark style ID for an index, modulo the palette size.

Parameters

in	*id*	The index.

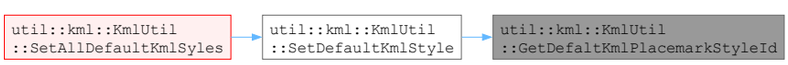
Returns

The KML style ID.

References util::kml::KmlUtil::kml_default_placemark_style_ids_.

Referenced by util::kml::KmlUtil::SetDefaultKmlStyle().

Here is the caller graph for this function:



10.92.3.5 GetKmlFactory()

```
static thread_specific_ptr< KmlFactory > & util::kml::KmlUtil::GetKmlFactory ()  [inline], [static],
[inherited]
```

Get the libkml factory, initializing it for this thread if necessary.

Exceptions

util::log::MessageException	if the factory cannot be obtained.

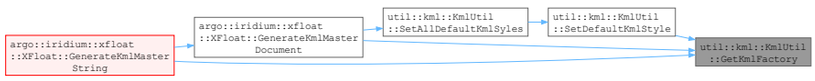
Returns

The thread's KML factory pointer (see kml_factory_).

References CATCH_BLOCK, and util::kml::KmlUtil::kml_factory_.

Referenced by argo::iridium::xfloat::XFloat::GenerateKmlMasterDocument(), argo::iridium::xfloat::XFloat::GenerateKmlMasterString(), and util::kml::KmlUtil::SetDefaultKmlStyle().

Here is the caller graph for this function:



10.92.3.6 GetSmallKmlPlacemarkStyleId()

```
static const string & util::kml::KmlUtil::GetSmallKmlPlacemarkStyleId (
            int id)  [inline], [static], [inherited]
```

Get the small placemark style ID for an index, modulo the palette size.

Parameters

in	*id*	The index.

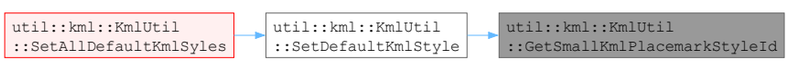
Returns

The KML style ID.

References util::kml::KmlUtil::kml_small_placemark_style_ids_.

Referenced by util::kml::KmlUtil::SetDefaultKmlStyle().

Here is the caller graph for this function:



10.92.3.7 GetTriangleKmlPlacemarkStyleId()

```
static const string & util::kml::KmlUtil::GetTriangleKmlPlacemarkStyleId (
            int id)  [inline], [static], [inherited]
```

Get the triangle placemark style ID for an index, modulo the palette size.

Parameters

in	*id*	The index.

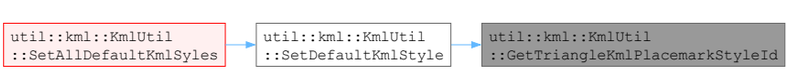
Returns

The KML style ID.

References util::kml::KmlUtil::kml_triangle_placemark_style_ids_.

Referenced by util::kml::KmlUtil::SetDefaultKmlStyle().

Here is the caller graph for this function:



10.92.3.8 operator()()

```
void argo::iridium::xfloat::XFloatPostProcessingService::operator() ()  [inline]
```

Functor operator overload function: the main entry point for the post-processing analysis thread.

This function defines the behavior of the post-processing analysis thread which is responsible for generating tertiary files (e.g. Google Earth KML and status files).

During execution the primary loop consists of the following. First, sleep for idle_sleep_seconds_. Then, iterate over all of the cached instrument objects and update files as needed. This loop continues until termination is signaled by the daemon/service thread.

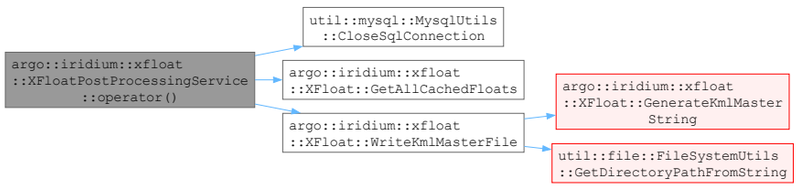
Once termination is signaled by the daemon process i.e. termination_signaled_ is TRUE, the thread will exit its primary loop and die.

Exceptions

None.	Never throws.

References CATCH_NOTHROW, util::mysql::MysqlUtils::CloseSqlConnection(), argo::iridium::xfloat::XFloat::GetAllCachedFloats(), idle_sleep_seconds_, ldbg, termination_signaled_, argo::iridium::xfloat::XFloat::WriteKmlMasterFile(), and XFLT_THREAD_SLEEP_MS.

Here is the call graph for this function:



10.92.3.9 SetAllDefaultKmlSyles()

```
static void util::kml::KmlUtil::SetAllDefaultKmlSyles (
             const DocumentPtr & document)  [inline], [static], [inherited]
```

Add the default styles for every palette colour to a KML document.

Parameters

in	*document*	The KML document to add the styles to.

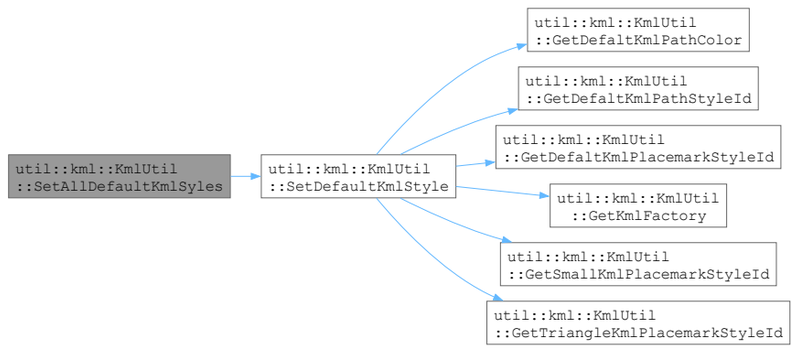
Exceptions

util::log::MessageException	if an error occurs.

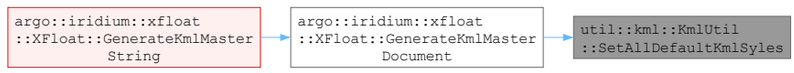
References CATCH_BLOCK, and util::kml::KmlUtil::SetDefaultKmlStyle().

Referenced by argo::iridium::xfloat::XFloat::GenerateKmlMasterDocument().

Here is the call graph for this function:



Here is the caller graph for this function:



10.92.3.10 SetDefaultKmlStyle()

```
static void util::kml::KmlUtil::SetDefaultKmlStyle (
            const DocumentPtr & document,
            unsigned int style_idx)  [inline], [static], [inherited]
```

Add the four default styles for one palette colour to a KML document.

Adds a 5-pixel path (line) style, a large dot placemark style, a small dot placemark style and a triangle placemark style, all in the colour `style_idx` (modulo the palette size). The icons are loaded from `http://argo.whoi.edu/kml/img/`. Errors are logged, not thrown.

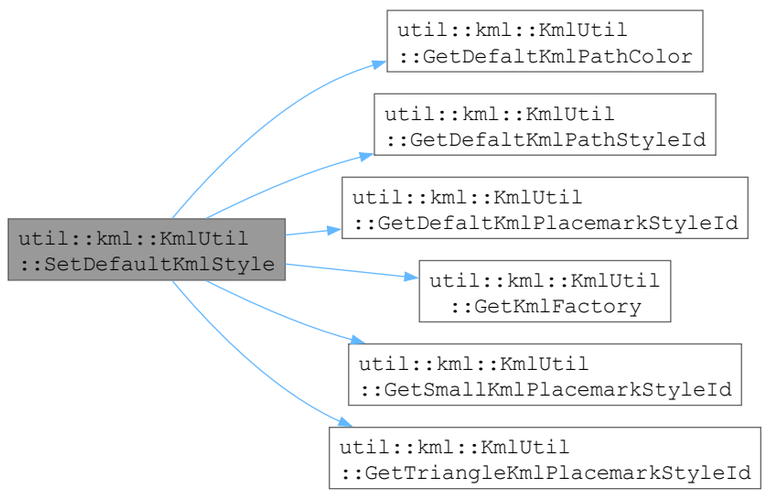
Parameters

in	*document*	The KML document to add the styles to.
in	*style_idx*	The palette index of the colour.

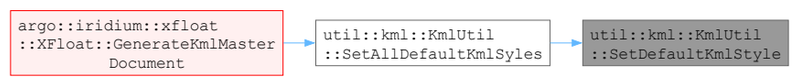
References CATCH_NOTHROW, util::kml::KmlUtil::GetDefaltKmlPathColor(), util::kml::KmlUtil::GetDefaltKmlPathStyleId(), util::kml::KmlUtil::GetDefaltKmlPlacemarkStyleId(), util::kml::KmlUtil::GetKmlFactory(), util::kml::KmlUtil::GetSmallKmlPlacemarkStyleId(), and util::kml::KmlUtil::GetTriangleKmlPlacemarkStyleId().

Referenced by util::kml::KmlUtil::SetAllDefaultKmlSyles().

Here is the call graph for this function:



Here is the caller graph for this function:



10.92.3.11 StartWorkerThread()

```
static void argo::iridium::xfloat::XFloatPostProcessingService::StartWorkerThread ()  [inline],
[static]
```

Instantiate the post-processing analysis worker thread.

This function is invoked by the daemon thread after initialization to instantiate the post-processing analysis thread. It creates a new instance of XFloatPostProcessingService, and instantiates a new thread that invokes the functor operator of the object.

Exceptions

None.	Never throws.

References CATCH_BLOCK, ldbg, thread_, and thread_object_.

10.92.3.12 TerminateWorkerThread()

```
static void argo::iridium::xfloat::XFloatPostProcessingService::TerminateWorkerThread ()  [inline],
[static]
```

Terminate the mail post-processing analysis thread.

This function is used by the daemon thread to kill the worker thread upon application termination. The function signals termination, and then attempts to join the child thread. If the child thread does not join, then the function will timeout after XMTPS_THREAD_JOIN_SECONDS seconds, a warning will be issued, and the child thread will be abandoned.

Exceptions

None.	Never throws.

References CATCH_BLOCK, ldbg, lwrn, thread_, and XMTPS_THREAD_JOIN_SECONDS.

10.92.4 Friends And Related Symbol Documentation

10.92.4.1 util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription

```
options_description util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription
()  [friend]
```

Friend function for populating some of the static members of this class from the configuration file options.

10.92.5 Member Data Documentation

10.92.5.1 idle_sleep_seconds_

```
unsigned int argo::iridium::xfloat::XFloatPostProcessingService::idle_sleep_seconds_  [static],
[private]
```

Duration to sleep when not performing post-processing, in seconds.

The thread sleeps for this duration between periodically iterating over all cached instrument data objects and performing post-processing as needed.

Referenced by operator()().

10.92.5.2 kml_default_path_colors_

```
const Color32 util::kml::KmlUtil::kml_default_path_colors_  [static], [private], [inherited]
```

Initial value:

```
= {
        Color32(0xB0, 0xFF, 0x00, 0x00),
        Color32(0xB0, 0xFF, 0x80, 0x00),
        Color32(0xB0, 0xFF, 0xFF, 0x00),
        Color32(0xB0, 0x80, 0xFF, 0x00),
        Color32(0xB0, 0x00, 0xFF, 0x00),
        Color32(0xB0, 0x00, 0xFF, 0x80),
        Color32(0xB0, 0x00, 0xFF, 0xFF),
        Color32(0xB0, 0x00, 0x80, 0xFF),
        Color32(0xB0, 0x00, 0x00, 0xFF),
        Color32(0xB0, 0x80, 0x00, 0xFF),
        Color32(0xB0, 0xFF, 0x00, 0xFF),
        Color32(0xB0, 0xFF, 0x00, 0x80),
        Color32(0xB0, 0x80, 0x80, 0x080),
 }
```

The colour palette for instrument paths and placemarks, indexed by style index.

Referenced by util::kml::KmlUtil::GetDefaltKmlPathColor().

10.92.5.3 kml_default_path_style_ids_

```
const string util::kml::KmlUtil::kml_default_path_style_ids_  [static], [private], [inherited]
```

Initial value:

```
= {
        "path1",
        "path2",
        "path3",
        "path4",
        "path5",
        "path6",
        "path7",
        "path8",
        "path9",
        "pathA",
        "pathB",
        "pathC",
        "pathD",}
```

The KML style IDs of the path (line) styles, one per palette colour.

Referenced by util::kml::KmlUtil::GetDefaltKmlPathStyleId().

10.92.5.4 kml_default_placemark_style_ids_

```
const string util::kml::KmlUtil::kml_default_placemark_style_ids_  [static], [private], [inherited]
```

Initial value:

```
= {
        "dot1_1",
        "dot1_2",
        "dot1_3",
        "dot1_4",
        "dot1_5",
        "dot1_6",
        "dot1_7",
        "dot1_8",
        "dot1_9",
        "dot1_A",
        "dot1_B",
        "dot1_C",
        "dot1_D", }
```

The KML style IDs of the large placemark styles, one per palette colour.

Referenced by util::kml::KmlUtil::GetDefaltKmlPlacemarkStyleId().

10.92.5.5 kml_factory_

```
thread_specific_ptr< KmlFactory > util::kml::KmlUtil::kml_factory_  [static], [private], [inherited]
```

The libkml factory used to create KML elements, held per thread.

Warning

> This holds libkml's process-wide factory (`KmlFactory::GetFactory()`), and the default cleanup deletes it when the owning thread exits or CloseKmlFactory() is called. Only the post-processing thread uses KML, so this currently happens only at shutdown.

Referenced by util::kml::KmlUtil::CloseKmlFactory(), and util::kml::KmlUtil::GetKmlFactory().

10.92.5.6 kml_small_placemark_style_ids_

```
const string util::kml::KmlUtil::kml_small_placemark_style_ids_  [static], [private], [inherited]
```

Initial value:

```
= {
        "dot1s_1",
        "dot1s_2",
        "dot1s_3",
        "dot1s_4",
        "dot1s_5",
        "dot1s_6",
        "dot1s_7",
        "dot1s_8",
        "dot1s_9",
        "dot1s_A",
        "dot1s_B",
        "dot1s_C",
        "dot1s_D", }
```

The KML style IDs of the small placemark styles, one per palette colour.

Referenced by util::kml::KmlUtil::GetSmallKmlPlacemarkStyleId().

10.92.5.7 kml_triangle_placemark_style_ids_

```
const string util::kml::KmlUtil::kml_triangle_placemark_style_ids_  [static], [private], [inherited]
```

Initial value:

```
= {
        "tri1_1",
        "tri1_2",
        "tri1_3",
        "tri1_4",
        "tri1_5",
        "tri1_6",
        "tri1_7",
        "tri1_8",
        "tri1_9",
        "tri1_A",
        "tri1_B",
        "tri1_C",
        "tri1_D", }
```

The KML style IDs of the triangle placemark styles, one per palette colour.

Referenced by util::kml::KmlUtil::GetTriangleKmlPlacemarkStyleId().

10.92.5.8 termination_signaled_

```
std::atomic< bool > argo::iridium::xfloat::XFloatPostProcessingService::termination_signaled_↩
[static], [private]
```

The termination signaled flag.

This flag is used to signal termination of the post-processing analysis thread. During normal operation, the value of this field will be FALSE. Once termination is signaled, it will become TRUE causing all of the single instance of the mail retrieval to to exit its primary loop and die.

Referenced by operator()().

10.92.5.9 thread_

```
thread argo::iridium::xfloat::XFloatPostProcessingService::thread_  [static], [private]
```

The post-processing analysis thread group.

This object represents the group of post-processing analysis threads that are instantiated after initialization. Each new thread that is instantiated is added to this group. Upon termination, the thread group is joined.

Referenced by StartWorkerThread(), and TerminateWorkerThread().

10.92.5.10 thread_object_

shared_ptr< XFloatPostProcessingService > argo::iridium::xfloat::XFloatPostProcessingService↩
::thread_object_ [static], [private]

The post-processing analysis thread objects.

Each instance of the post-processing analysis threads is created by invoking the functor of an instance of this runnable class. Each instance of this object that is created during instantiation of a new thread is stored in this vector.

Referenced by StartWorkerThread().

The documentation for this class was generated from the following files:

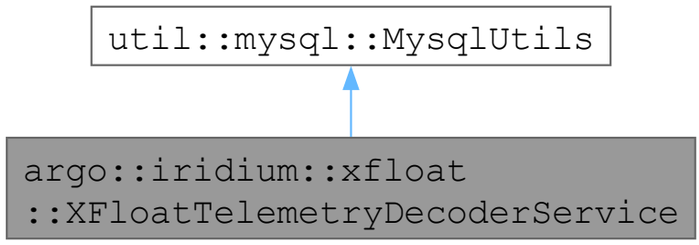
- stage4b_PostProcessingThread.h
- stage4b_PostProcessingThread.cpp

10.93 argo::iridium::xfloat::XFloatTelemetryDecoderService Class Reference

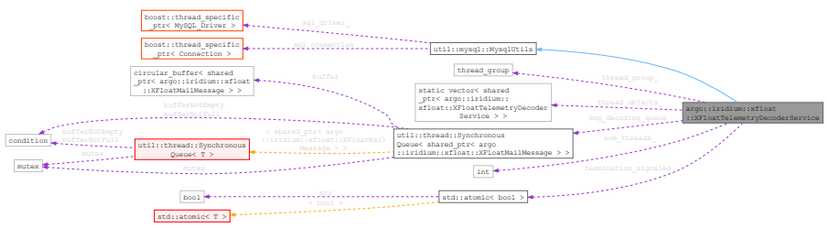
The telemetry decoder thread class.

#include <stage3_TelemetryDecoderThread.h>

Inheritance diagram for argo::iridium::xfloat::XFloatTelemetryDecoderService:



Collaboration diagram for argo::iridium::xfloat::XFloatTelemetryDecoderService:



Public Member Functions

- XFloatTelemetryDecoderService ()

  The constructor.
- ~XFloatTelemetryDecoderService ()

  The destructor.
- void DecodeMessage (shared_ptr< XFloatMailMessage > msg) const

  Decode the telemetry data from a processed mail message.
- void operator() () ATTR_NO_EXCEPTIONS

  Functor operator overload function: the main entry point for the telemetry decoder thread(s).
- void ProcessDiveData (shared_ptr< XFloatDiveData > &dive_data) const

  Process a dive data object.

Static Public Member Functions

- static void CloseSqlConnection ()

  Close the thread-specific MySQL connection if it is currently open.
- static shared_ptr< ResultSet > ExecuteQuery (Connection *con, const string &query)

  Execute an SQL query and return a shared pointer to the ResultSet object.
- static int ExecuteUpdate (Connection *con, const string &update)

  Execute an SQL update and return the status code.
- static shared_ptr< PreparedStatement > GeneratePreparedStatement (Connection *con, const string &statement)

  Generate a prepared SQL statement object for the given SQL statement.
- static string GenerateSqlDeleteStr (const string &from, const string &where="")

  Generate an SQL DELETE statement.
- static string GenerateSqlSelectStr (const string &what, const string &from, const string &where="", const string &groupBy="", const string &orderBy="", unsigned int limit=0, unsigned int offset=0)

  Generate an SQL select statement.
- static thread_specific_ptr< Connection > & GetSqlConnection (string url, string user, string password, string schema)

  Get a thread-specific instance of a MySQL connection object.
- static thread_specific_ptr< MySQL_Driver > & GetSqlDriver ()

  Get a thread-specific instance of a MySQL driver object.
- static bool MsgDecodingQueuePush (const shared_ptr< XFloatMailMessage > &msg, bool block=true)

  Push a mail message into the telemetry decoder queue.
- static unsigned int num_threads ()

  Get the number of telemetry decoder threads.
- static void StartWorkerThreads () ATTR_NO_EXCEPTIONS

  Instantiate the telemetry decoder worker thread(s).
- static void TerminateWorkerThreads () ATTR_NO_EXCEPTIONS

  Terminate the telemetry decoder worker thread(s).

Static Private Member Functions

- static void CleanupSqlConnection (Connection *con)

  Cleanup an SQL connection.
- static void CleanupSqlDriver (MySQL_Driver *driver)

  Function for memory cleanup of MySQL driver on destruct.
- static bool MsgProcessingQueuePop (shared_ptr< XFloatMailMessage > &msg, bool block=false)

  Pop a mail message out of the telemetry decoder queue.
- static void set_termination_signaled (bool termination_signaled=true)

  Set the termination signaled flag.

Static Private Attributes

- static util::thread::SynchronousQueue< shared_ptr< XFloatMailMessage > > msg_decoding_queue_

  The telemetry decode queue.
- static unsigned int num_threads_

  The number of telemetry decoder threads.
- static thread_specific_ptr< Connection > sql_connection_

  Tthe thread-specific pointer to MySQL connection object.
- static thread_specific_ptr< MySQL_Driver > sql_driver_

  The thread-specific pointer to MySQL driver object.
- static std::atomic< bool > termination_signaled_

  The termination signaled flag.
- static thread_group thread_group_

  The telemetry decoder thread group.
- static vector< shared_ptr< XFloatTelemetryDecoderService > > thread_objects_

  The telemetry decoder thread objects.

Friends

- options_description util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription ()

  Friend function for populating some of the static members of this class from the configuration file options.

10.93.1 Detailed Description

The telemetry decoder thread class.

This class defines the operation of the telemetry decoder thread(s), which are the third stage in the processing pipeline. The telemetry decoder thread(s) consumes processed mail messages that are produced by the previous stage of the processing pipeline, the mail processing thread, XFloatMailProcessingService.

For each message received from the mail processing thread, the corresponding instrument object, XFloat, is retrieved. Likewise, the dive data object, XFloatDiveData, corresponding to the dive or cycle number of the mail message is retrieved from the instrument object. The mail message is assigned to the dive data object. Finally, the SBD message data attached to the mail message is decoded into one of the util::xmsg::XMessage which is determined by the X-↩ Message type identifier contained in the binary message data. The resulting util::xmsg::XMessage object is assigned to the XFloatDiveData.

Zero or more instance of this thread may be instantiated after application initialization, depending on the configuration options. Each thread enters a primary loop in which messages are fetched from a common queue and processed. If the incoming queue is empty, then the thread will sleep for intervals of XFLT_THREAD_SLEEP_MS milliseconds.

The primary loop will continue until the daemon thread signals termination. When termination is signaled, the remaining messages in the incoming queue will be processed, then all instances of this thread will die. This ensures that processing pipeline will be thoroughly flushed before application termination, and that no messages will be partially or incompletely processed.

10.93.2 Constructor & Destructor Documentation

10.93.2.1 XFloatTelemetryDecoderService()

```
argo::iridium::xfloat::XFloatTelemetryDecoderService::XFloatTelemetryDecoderService ()  [inline]
```

The constructor.

10.93.2.2 ~XFloatTelemetryDecoderService()

```
argo::iridium::xfloat::XFloatTelemetryDecoderService::~XFloatTelemetryDecoderService ()  [inline]
```

The destructor.

10.93.3 Member Function Documentation

10.93.3.1 CleanupSqlConnection()

```
static void util::mysql::MysqlUtils::CleanupSqlConnection (
            Connection * con)  [inline], [static], [private], [inherited]
```

Cleanup an SQL connection.

This function for memory cleanup of the SQL connection on destruction. The thread-specific SQL connection object will be deleted if it exists.

Parameters

in	*con*	The SQL connection to cleanup.

Exceptions

log::MessageException	if an error occurs while cleaning up the SQL connection object.

References CATCH_BLOCK.

10.93.3.2 CleanupSqlDriver()

```
static void util::mysql::MysqlUtils::CleanupSqlDriver (
            MySQL_Driver * driver)  [inline], [static], [private], [inherited]
```

Function for memory cleanup of MySQL driver on destruct.

Signals the SQL driver to end operation, if it exists.

Parameters

in	*driver*	Driver to cleanup.

Exceptions

log::MessageException	if an error occurs while cleaning up the SQL driver object.

References CATCH_BLOCK.

10.93.3.3 CloseSqlConnection()

```
static void util::mysql::MysqlUtils::CloseSqlConnection ()  [inline], [static], [inherited]
```

Close the thread-specific MySQL connection if it is currently open.

If the thread local MySQL connection is not currently open, then this function will have no effect.

Note

The MySQL connection should be closed when idle to prevent the connection from timing out and throwing exceptions.

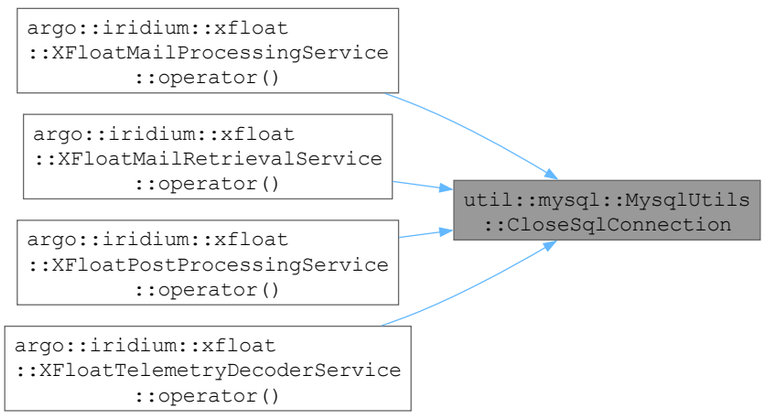
Exceptions

log::MessageException	if an error occurs while attempting to close the MySQL connection.

References CATCH_BLOCK, and util::mysql::MysqlUtils::sql_connection_.

Referenced by argo::iridium::xfloat::XFloatMailProcessingService::operator()(), argo::iridium::xfloat::XFloatMailRetrievalService::operator(
argo::iridium::xfloat::XFloatPostProcessingService::operator()(), and operator()().

Here is the caller graph for this function:



10.93.3.4 DecodeMessage()

```
void argo::iridium::xfloat::XFloatTelemetryDecoderService::DecodeMessage (
            shared_ptr< XFloatMailMessage > msg) const  [inline]
```

Decode the telemetry data from a processed mail message.

This function is used to decode the binary Iridium Short Burst Data (SBD) message that is supplied as an attachment to the email message. Mail messages that are received from the previous stage in the processing pipeline, the mail processing thread(s), XFloatMailProcessingService, are decoded using this function.

First, the function checks the message reference `msg`. If it is NULL, then the function throws an exception. Otherwise, it continues on to set the global logger attributes:

- DiveID,
- MsgUID,
- PcktID,
- MomsnID,
- WhoiID,
- DacID, and
- WmoID.

These attributes are used by the global logger object for filtering, etc. The values of the attributes are assumed to be known since, if any were unknown, then the mail processing threads would have queried them from the SQL database.

Next, the float object, `xfloat`, that corresponds to the instrument that transmitted this message is retrieved. If the float object does not already exist, it will be created. Also, the metadata object for this instrument is retrieved, which contains information about the start-of-mission and deployment date/times. The time of flight of the message is examined to ensure that the message is valid. Messages that were transmitted after the start-of-mission or deployment date/times are discarded.

Then, the dive data object, `xdive`, which contains all the data received for a single instrument and single dive/cycle is retrieved from the float object. If the dive data object for this dive/cycle does not already exist, it will be created. The mail message, `msg`, is then assigned to the dive data object.

The SBD message is passed to the ProcessDiveData() function to be decoded.

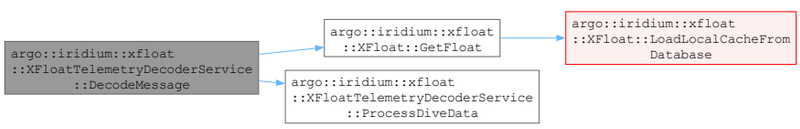
Parameters

in	*msg*	A shared reference to the mail message to decode.

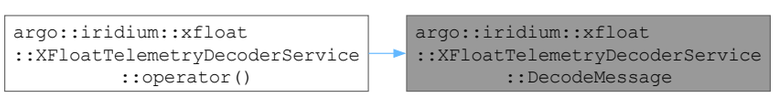
References CATCH_BLOCK, EXC_THROW, argo::iridium::xfloat::XFloat::GetFloat(), ldbg, ProcessDiveData(), XFLT_DIVE_UNKNOWN, util::xmsg::XMSG_TYPE_ID_ARGO_DATA, util::xmsg::XMSG_TYPE_ID_GPS_ENDOFDIVE, util::xmsg::XMSG_TYPE_ID_GPS_STARTOFDIVE, util::xmsg::XMSG_TYPE_ID_PRES_SURF0, util::xmsg::XMSG_TYPE_ID_PRES_SU util::xmsg::XMSG_TYPE_ID_PRES_SURF2, util::xmsg::XMSG_TYPE_ID_PRES_SURF3, util::xmsg::XMSG_TYPE_ID_PRES_SURF4, util::xmsg::XMSG_TYPE_ID_PRES_SURF5, util::xmsg::XMSG_TYPE_ID_PRES_SURF6, util::xmsg::XMSG_TYPE_ID_PRES_SURF7, util::xmsg::XMSG_TYPE_ID_PRES_SURF8, util::xmsg::XMSG_TYPE_ID_PRES_SURF9, util::xmsg::XMSG_TYPE_ID_PRES_SURFA, util::xmsg::XMSG_TYPE_ID_PRES_SURFB, util::xmsg::XMSG_TYPE_ID_PRES_SURFC, util::xmsg::XMSG_TYPE_ID_PRES_SURFD, util::xmsg::XMSG_TYPE_ID_PRES_SURFE, and util::xmsg::XMSG_TYPE_ID_PRES_SURFF.

Referenced by operator()().

Here is the call graph for this function:



Here is the caller graph for this function:



10.93.3.5 ExecuteQuery()

```
static shared_ptr< ResultSet > util::mysql::MysqlUtils::ExecuteQuery (
            Connection * con,
            const string & query)  [inline], [static], [inherited]
```

Execute an SQL query and return a shared pointer to the ResultSet object.

Parameters

in	*con*	A pointer to the SQL connection object. This must not be NULL.
in	*query*	The SQL query to execute. This must be a well-formatted SQL query string.

Exceptions

log::MessageException	if 'con' is null, 'query' is a malfored SQL string, or any other error occurs.

Returns

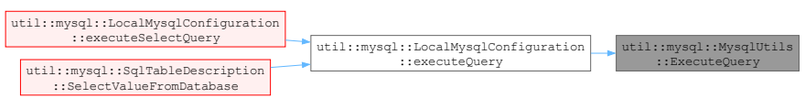
A shared pointer to the query ResultSet object.

References CATCH_BLOCK, and util::mysql::MysqlUtils::GeneratePreparedStatement().

Referenced by util::mysql::LocalMysqlConfiguration::executeQuery().

Here is the call graph for this function:

Here is the caller graph for this function:



10.93.3.6 ExecuteUpdate()

```
static int util::mysql::MysqlUtils::ExecuteUpdate (
            Connection * con,
            const string & update)  [inline], [static], [inherited]
```

Execute an SQL update and return the status code.

Parameters

in	*con*	A pointer to the SQL connection object. This must not be NULL.
in	*update*	The SQL update to execute. This must be a well-formatted SQL update string.

Exceptions

log::MessageException	if 'con' is null, 'update' is a malformed SQL string, or any other error occurs.

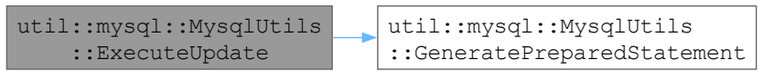
Returns

The update status code.

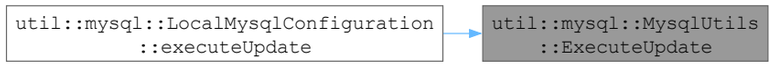
References CATCH_BLOCK, and util::mysql::MysqlUtils::GeneratePreparedStatement().

Referenced by util::mysql::LocalMysqlConfiguration::executeUpdate().

Here is the call graph for this function:



Here is the caller graph for this function:



10.93.3.7 GeneratePreparedStatement()

```
static shared_ptr< PreparedStatement > util::mysql::MysqlUtils::GeneratePreparedStatement (
            Connection * con,
            const string & statement)  [inline], [static], [inherited]
```

Generate a prepared SQL statement object for the given SQL statement.

Parameters

in	*con*	A pointer to the SQL connection object. This must not be NULL.
in	*statement*	The SQL statement to prepare. This must be a well-formed SQL string.

Exceptions

log::MessageException	if 'con' is NULL, 'statement' is not a valid SQL string, or if an error occurs.

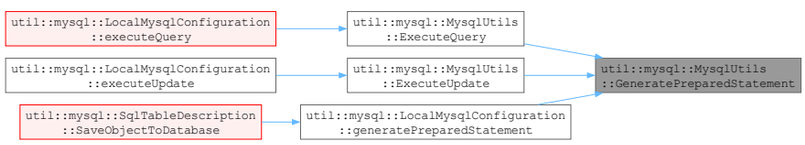
Returns

A prepared SQL statement object generated from the given SQL statement.

References CATCH_BLOCK, and EXC_THROW.

Referenced by util::mysql::MysqlUtils::ExecuteQuery(), util::mysql::MysqlUtils::ExecuteUpdate(), and util::mysql::LocalMysqlConfiguration:

Here is the caller graph for this function:



10.93.3.8 GenerateSqlDeleteStr()

```
static string util::mysql::MysqlUtils::GenerateSqlDeleteStr (
            const string & from,
            const string & where = "")  [inline], [static], [inherited]
```

Generate an SQL DELETE statement.

Warning

Without a `where` clause, the statement deletes every row in the table.

Parameters

in	*from*	The name of the table to delete from.
in	*where*	The SQL "where-condition" clause (empty = all rows).

Returns

The SQL statement.

10.93.3.9 GenerateSqlSelectStr()

```
static string util::mysql::MysqlUtils::GenerateSqlSelectStr (
            const string & what,
            const string & from,
            const string & where = "",
            const string & groupBy = "",
            const string & orderBy = "",
            unsigned int limit = 0,
            unsigned int offset = 0)  [inline], [static], [inherited]
```

Generate an SQL select statement.

Parameters

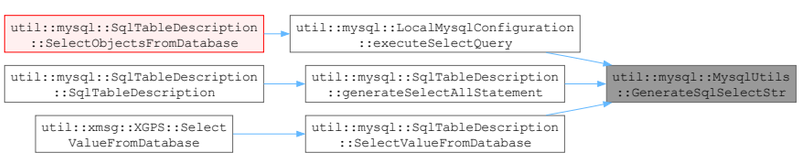
in	*what*	The SQL select statement select-expression clause.
in	*from*	The SQL select statement table-references clause.
in	*where*	The SQL select statement where-condition clause.
in	*groupBy*	The SQL select statement group-by clause.
in	*orderBy*	The SQL select statement order-by clause.
in	*limit*	The SQL select statement result set row limit value.
in	*offset*	The SQL select statement result set row offset.

Returns

The SQL select statement.

Referenced by util::mysql::LocalMysqlConfiguration::executeSelectQuery(), util::mysql::SqlTableDescription< T >::generateSelectAllState
and util::mysql::SqlTableDescription< T >::SelectValueFromDatabase().

Here is the caller graph for this function:



10.93.3.10 GetSqlConnection()

```
static thread_specific_ptr< Connection > & util::mysql::MysqlUtils::GetSqlConnection (
            string url,
            string user,
            string password,
            string schema)  [inline], [static], [inherited]
```

Get a thread-specific instance of a MySQL connection object.

A new connection will be created, if necessary.

Parameters

in	*url*	The MySQL host server qualified address.
in	*user*	The MySQL account username.
in	*password*	The MySQL account password.
in	*schema*	The MySQL account schema name.

Exceptions

log::MessageException	if an error occurs while creating a new thread-specific instance of the SQL connection object

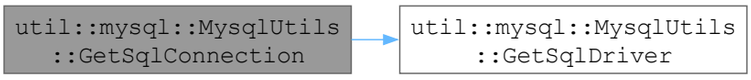
Returns

A thread-specific instance of a MySQL connection object.

References CATCH_BLOCK, util::mysql::MysqlUtils::GetSqlDriver(), and util::mysql::MysqlUtils::sql_connection_.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

Here is the call graph for this function:



Here is the caller graph for this function:

10.93.3.11 GetSqlDriver()

```
static thread_specific_ptr< MySQL_Driver > & util::mysql::MysqlUtils::GetSqlDriver ()  [inline],
[static], [inherited]
```

Get a thread-specific instance of a MySQL driver object.

A new driver will created, if necessary.

Exceptions

log::MessageException	if an error occurs while creating a new thread-specific instance of the SQL driver object

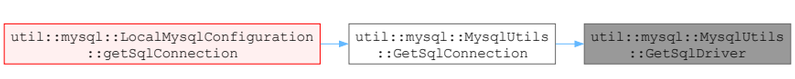
Returns

A thread-specific instance of a MySQL driver object.

References CATCH_BLOCK, and util::mysql::MysqlUtils::sql_driver_.

Referenced by util::mysql::MysqlUtils::GetSqlConnection().

Here is the caller graph for this function:



10.93.3.12 MsgDecodingQueuePush()

```
static bool argo::iridium::xfloat::XFloatTelemetryDecoderService::MsgDecodingQueuePush (
            const shared_ptr< XFloatMailMessage > & msg,
            bool block = true)  [inline], [static]
```

Push a mail message into the telemetry decoder queue.

This function is used by the previous stage in the processing pipeline, the mail processing thread, XFloatMailProcessingService, to pass a mail message to the telemetry decoder thread(s). The mail message processing queue is synchronous, so the underlying push function is atomic.

Parameters

in	*msg*	The mail message to push into the queue.
in	*block*	If TRUE, this function will block until the queue is not full. If FALSE, this function will return FALSE if the queue is full (and the element will not be pushed into the queue).

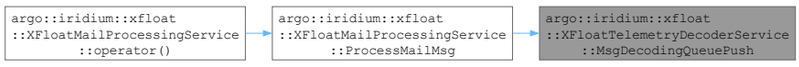
Returns

TRUE if the element was successfully pushed into the queue, or FALSE otherwise.

References msg_decoding_queue_.

Referenced by argo::iridium::xfloat::XFloatMailProcessingService::ProcessMailMsg().

Here is the caller graph for this function:



10.93.3.13 MsgProcessingQueuePop()

```
static bool argo::iridium::xfloat::XFloatTelemetryDecoderService::MsgProcessingQueuePop (
            shared_ptr< XFloatMailMessage > & msg,
            bool block = false)  [inline], [static], [private]
```

Pop a mail message out of the telemetry decoder queue.

This function is used by the telemetry decoder thread(s) to receive mail messages produced by the previous stage in the processing pipeline, the mail processing thread, XFloatMailProcessingService. The queue is synchronous, so the underlying pop function is atomic.

Parameters

out	*msg*	The destination of the mail message to pop out of the queue.
in	*block*	If TRUE, this function will block until the queue is not empty. If FALSE, this function will return FALSE if the queue is empty (and no element will be popped out of the queue).

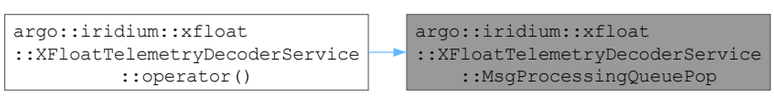
Returns

TRUE if the element was successfully popped out of the queue, or FALSE otherwise.

References msg_decoding_queue_.

Referenced by operator()().

Here is the caller graph for this function:



10.93.3.14 num_threads()

```
static unsigned int argo::iridium::xfloat::XFloatTelemetryDecoderService::num_threads () [inline],
[static]
```

Get the number of telemetry decoder threads.

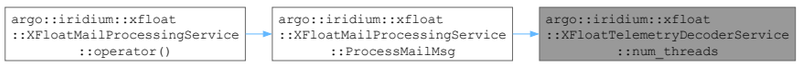
Returns

The number of telemetry decoder threads.

References num_threads_.

Referenced by argo::iridium::xfloat::XFloatMailProcessingService::ProcessMailMsg().

Here is the caller graph for this function:



10.93.3.15 operator()()

```
void argo::iridium::xfloat::XFloatTelemetryDecoderService::operator() () [inline]
```

Functor operator overload function: the main entry point for the telemetry decoder thread(s).

This function defines the behavior of the telemetry decoder thread(s), which are responsible for decoding the binary data from the processed mail messages that are produced by the mail processing thread, XFloatMailProcessingService. After initialization, zero or more instances of this thread are instantiated. The static configuration options for this thread are set as a result of parsing the command-line arguments and/or configuration file.

First, the scope thread attribute "ThreadName" is set to "sbd-decode". This attribute is used by the global logger object for filtering, etc. Then the thread enters its primary loop, in which it pops mail messages from the incoming queue one at a time and decodes the messages before passing them to the next stage in the processing pipeline, the post-processing analysis thread(s), XFloatPostProcessingService. If the incoming message queue is empty, then the connection o the SQL database is closed, and the thread sleeps for XFLT_THREAD_SLEEP_MS milliseconds.

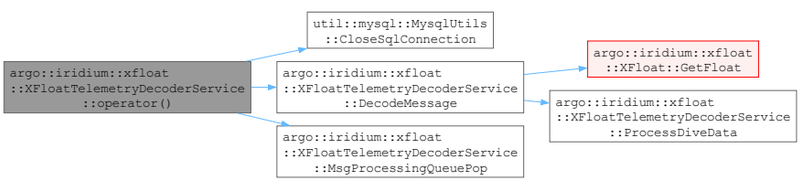
Once termination is signaled by the daemon process i.e. termination_signaled_ is TRUE, the thread will exit its primary loop and die.

Exceptions

None.	Never throws.

References CATCH_NOTHROW, util::mysql::MysqlUtils::CloseSqlConnection(), DecodeMessage(), ldbg, MsgProcessingQueuePop(), termination_signaled_, and XFLT_THREAD_SLEEP_MS.

Here is the call graph for this function:



10.93.3.16 ProcessDiveData()

```
void argo::iridium::xfloat::XFloatTelemetryDecoderService::ProcessDiveData (
            shared_ptr< XFloatDiveData > & dive_data) const [inline]
```

Process a dive data object.

Parameters

in	*dive_data*	The dive data object to process.

This function is invoked each time a new message for this dive is decoded. The function first checks if the dive data object is complete and proceeds accordingly.

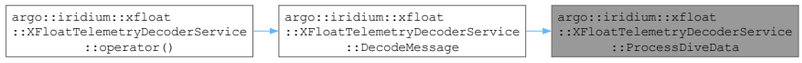
If the dive data object is complete, then PHY and listing files are written, the object is saved to the database, the KML-document-is-updated flag is set to false, the dive data object is pushed into the NetCDF file writing queue, and the parent instrument object is pushed into the float processing queue.

If the dive data object is incomplete, then the listing file is written, the dive data object is saved to the database, and the parent instrument is pushed into the float processing queue.

References CATCH_BLOCK, EXC_THROW, ldbg, and lout.

Referenced by DecodeMessage().

Here is the caller graph for this function:



10.93.3.17 set_termination_signaled()

```
static void argo::iridium::xfloat::XFloatTelemetryDecoderService::set_termination_signaled (
            bool termination_signaled = true)  [inline], [static], [private]
```

Set the termination signaled flag.

This function is called by TerminateWorkerThreads() in order to cause all currently running instances of the telemetry decoder thread to exit their primary loop and die.

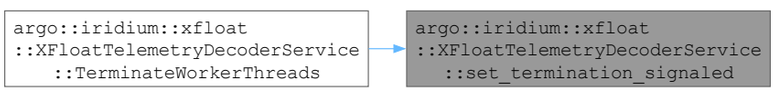
Parameters

in	*termination_signaled*	The termination signaled value to set.

References termination_signaled_.

Referenced by TerminateWorkerThreads().

Here is the caller graph for this function:



10.93.3.18 StartWorkerThreads()

```
static void argo::iridium::xfloat::XFloatTelemetryDecoderService::StartWorkerThreads ()  [inline],
[static]
```

Instantiate the telemetry decoder worker thread(s).

This function is used by the daemon thread to instantiate zero or more telemetry decoder threads after initialization. The number of threads that are instantiated by this function is determined by the value of num_threads_, which is set as a result of parsing the command-line argument and/or configuration file during initialization.

Each thread is instantiated by creating an instance of this class, which is pushed into the thread_objects_ vector, and invoking the functor of each object. Each thread that is created is added to the thread_group_ object.

Exceptions

None.	Never throws.

References CATCH_BLOCK, ldbg, num_threads_, thread_group_, and thread_objects_.

10.93.3.19 TerminateWorkerThreads()

```
static void argo::iridium::xfloat::XFloatTelemetryDecoderService::TerminateWorkerThreads () [inline],
[static]
```

Terminate the telemetry decoder worker thread(s).

This function is used by the daemon thread to kill the telemetry decoder worker thread(s) upon application termination. The function signals termination, and then attempts to join the thread group.

Warning

> If any of telemetry decoder threads fails to terminate properly, then the `join_all()` function could block indefinitely. Precautions have been take to ensure that this does not happen, but it may still be possible.

Exceptions

None.	Never throws.

References CATCH_BLOCK, ldbg, num_threads_, set_termination_signaled(), and thread_group_.

Here is the call graph for this function:

10.93.4 Friends And Related Symbol Documentation

10.93.4.1 util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription

```
options_description util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription
() [friend]
```

Friend function for populating some of the static members of this class from the configuration file options.

10.93.5 Member Data Documentation

10.93.5.1 msg_decoding_queue_

```
util::thread::SynchronousQueue< shared_ptr< XFloatMailMessage > > argo::iridium::xfloat::↩
XFloatTelemetryDecoderService::msg_decoding_queue_ [static], [private]
```

The telemetry decode queue.

This synchronous FIFO queue is used to pass messages produced by the previous stage of the processing pipeline, the mail processing thread(s), XFloatMailRetrievalService, to the telemetry decoder thread(s). All the operations performed by on the queue are atomic. Depending on the number of telemetry decoder threads, this queue may function as a one-to-one queue or one-to-many queue.

Referenced by MsgDecodingQueuePush(), and MsgProcessingQueuePop().

10.93.5.2 num_threads_

```
unsigned int argo::iridium::xfloat::XFloatTelemetryDecoderService::num_threads_ [static], [private]
```

The number of telemetry decoder threads.

The value of this field determines the number of telemetry decoder threads that are instantiated after initialization. Its value is set as a result of parsing the configuration file and/or command-line arguments during initialization. If the value is unspecified, then it defaults to a value of CFG_DEFAULT_NDECO.

Referenced by num_threads(), StartWorkerThreads(), and TerminateWorkerThreads().

10.93.5.3 sql_connection_

```
thread_specific_ptr<Connection> util::mysql::MysqlUtils::sql_connection_ [static], [private],
[inherited]
```

Tthe thread-specific pointer to MySQL connection object.

Thread-specific MySQL connection, released by `MysqlUtils::CleanupSqlConnection()` when the thread exits.

Referenced by util::mysql::MysqlUtils::CloseSqlConnection(), and util::mysql::MysqlUtils::GetSqlConnection().

10.93.5.4 sql_driver_

```
thread_specific_ptr<MySQL_Driver> util::mysql::MysqlUtils::sql_driver_ [static], [private],
[inherited]
```

The thread-specific pointer to MySQL driver object.

Thread-specific reference to the MySQL driver.

Its cleanup function, `MysqlUtils::CleanupSqlDriver()`, does not delete the shared driver instance.

Referenced by util::mysql::MysqlUtils::GetSqlDriver().

10.93.5.5 termination_signaled_

```
std::atomic< bool > argo::iridium::xfloat::XFloatTelemetryDecoderService::termination_signaled_
[static], [private]
```

The termination signaled flag.

This flag is used to signal termination of the telemetry decoder thread(s). During normal operation, the value of this field will be FALSE. Once termination is signaled, it will become TRUE causing all of the instances of the telemetry decoder thread to process the remaining messages in the incoming queue, exiting their primary loop, and then die.

Referenced by operator()(), and set_termination_signaled().

10.93.5.6 thread_group_

```
thread_group argo::iridium::xfloat::XFloatTelemetryDecoderService::thread_group_ [static], [private]
```

The telemetry decoder thread group.

This object represents the group of telemetry decoder threads that are instantiated after initialization. Each new thread that is instantiated is added to this group. Upon termination, the thread group is joined.

Referenced by StartWorkerThreads(), and TerminateWorkerThreads().

10.93.5.7 thread_objects_

vector< shared_ptr< XFloatTelemetryDecoderService > > argo::iridium::xfloat::XFloatTelemetry↩
DecoderService::thread_objects_ [static], [private]

The telemetry decoder thread objects.

Each instance of the telemetry decoder threads is created by invoking the functor of an instance of this runnable class. Each instance of this object that is created during instantiation of a new thread is stored in this vector.

Referenced by StartWorkerThreads().

The documentation for this class was generated from the following files:

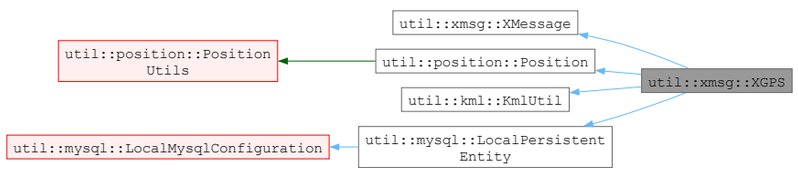
- stage3_TelemetryDecoderThread.h
- stage3_TelemetryDecoderThread.cpp

10.94 util::xmsg::XGPS Class Reference

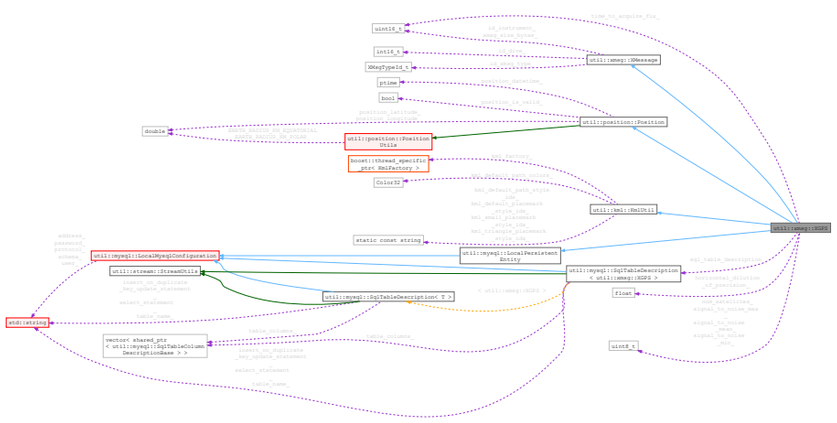
Class representing a GPS type X-Message.

#include <XGPS.h>

Inheritance diagram for util::xmsg::XGPS:



Collaboration diagram for util::xmsg::XGPS:



Public Member Functions

- XGPS (int16_t dive, uint16_t whoi, const vector< uint8_t > &data)

  The parameterized, binary data constructor.
- XGPS (int16_t dive, uint16_t whoi, XMsgTypeId_t type) ATTR_NO_EXCEPTIONS

  The parameterized, blank constructor.
- ~XGPS ()

  The destructor.
- const uint8_t * CheckXMsgData (const vector< uint8_t > &data, int id, int minBytes, bool parseSize=true)

  Check the X-message for the correct ID, minimum size, and message terminator.
- void DecodeXMsgData (const vector< uint8_t > &data)

  Decode the GPS X-message from binary data.

- const string GenerateListingString () const

  Return a string representation of this GPS message.
- const string GeneratePhyString (size_t xmits) const

  Generate a phy file entry string representation of this GPS position message object.
- float **horizontal_dilution_of_precision** () const
- int16_t id_dive () const

  Get the dive or cycle number of this X-Message.
- uint16_t id_instrument () const

  Get the unique instrument identifier of this X-Message.
- uint8_t **id_type** () const
- XMsgTypeId_t id_xmsg_type () const

  Get the enumerated X-Message type identifier.
- uint8_t **num_satellites** () const
- ptime position_datetime () const

  Get the date/time of the position.
- string position_datetime_as_iso8601_string () const

  Get a formatted string representing the position date/time composed using the ISO 8601 standard.
- bool position_is_valid () const

  Is the position valid?
- double position_latitude () const

  Get the latitude of the position, in decimal degrees.
- double position_longitude () const

  Get the longitude of the position, in decimal degrees.
- string postition_as_iso6709_string () const

  Get a formatted string representing the position composed using the ISO 6709 standard.
- void SaveObjectToDatabase () const

  Save this GPS message object to the SQL database.
- uint8_t **signal_to_noise_max** () const
- uint8_t **signal_to_noise_mean** () const
- uint8_t **signal_to_noise_min** () const
- uint16_t **time_to_acquire_fix** () const
- uint16_t xmsg_size_bytes () const

  Get the size of the X-Message, in bytes.

Static Public Member Functions

- static void CloseKmlFactory ()

  Release this thread's KML factory pointer.
- static void CloseSqlConnection ()

  Close the thread-specific MySQL connection if it is currently open.
- static void DeleteObjectsFromDatabase (const string &where)

  Delete GPS message rows from the SQL database table.
- static shared_ptr< sql::ResultSet > **executeDeleteQuery** (const string &from, const string &where="")
- static shared_ptr< ResultSet > ExecuteQuery (Connection *con, const string &query)

  Execute an SQL query and return a shared pointer to the ResultSet object.
- static shared_ptr< sql::ResultSet > executeQuery (const string &query)

  Execute an SQL query and return a shared pointer to the ResultSet object.
- static shared_ptr< sql::ResultSet > executeSelectQuery (const string &what, const string &from, const string &where="", const string &groupBy="", const string &orderBy="", unsigned int limit=0, unsigned int offset=0)

  Execute an SQL select query and return a shared pointer to the ResultSet object.
- static int ExecuteUpdate (Connection *con, const string &update)

  Execute an SQL update and return the status code.
- static int executeUpdate (const string &update)

  Execute an SQL update and return the status code.
- static string GenerateListingComment ()

  Get the listing entry comment string.
- static shared_ptr< PreparedStatement > GeneratePreparedStatement (Connection *con, const string &statement)

  Generate a prepared SQL statement object for the given SQL statement.
- static shared_ptr< sql::PreparedStatement > generatePreparedStatement (const string &statement)

  Generate a prepared SQL statement object for the given SQL statement.
- static string GenerateSqlDeleteStr (const string &from, const string &where="")

  Generate an SQL DELETE statement.

- static string GenerateSqlSelectStr (const string &what, const string &from, const string &where="", const string &groupBy="", const string &orderBy="", unsigned int limit=0, unsigned int offset=0)

  Generate an SQL select statement.
- static const Color32 & GetDefaltKmlPathColor (int id)

  Get the palette colour for an index (an instrument ID, for example), modulo the palette size.
- static const string & GetDefaltKmlPathStyleId (int id)

  Get the path (line) style ID for an index, modulo the palette size.
- static const string & GetDefaltKmlPlacemarkStyleId (int id)

  Get the large placemark style ID for an index, modulo the palette size.
- static thread_specific_ptr< KmlFactory > & GetKmlFactory ()

  Get the libkml factory, initializing it for this thread if necessary.
- static const string & GetSmallKmlPlacemarkStyleId (int id)

  Get the small placemark style ID for an index, modulo the palette size.
- static thread_specific_ptr< Connection > & GetSqlConnection (string url, string user, string password, string schema)

  Get a thread-specific instance of a MySQL connection object.
- static thread_specific_ptr< MySQL_Driver > & GetSqlDriver ()

  Get a thread-specific instance of a MySQL driver object.
- static const string & GetTriangleKmlPlacemarkStyleId (int id)

  Get the triangle placemark style ID for an index, modulo the palette size.
- static vector< shared_ptr< XGPS > > SelectAllObjectsFromDatabase (const string &groupBy="", const string &orderBy="", const unsigned int &limit=0, const unsigned int &offset=0)

  Select all GPS message objects from the SQL database table.
- static shared_ptr< XGPS > SelectObjectFromDatabase (const string &where="", const string &groupBy="", const string &orderBy="", const unsigned int &offset=0)

  Select a single GPS message object from the SQL database table.
- static vector< shared_ptr< XGPS > > SelectObjectsFromDatabase (const string &where="", const string &groupBy="", const string &orderBy="", const unsigned int &limit=0, const unsigned int &offset=0)

  Select GPS message objects from the SQL database table.
- template<typename TT >
  static TT SelectValueFromDatabase (const string &what, const string &where="", const string &groupBy="", const string &orderBy="", const unsigned int &offset=0)

  Select a single value from the GPS message SQL database table.
- static void SetAllDefaultKmlSyles (const DocumentPtr &document)

  Add the default styles for every palette colour to a KML document.
- static void SetDefaultKmlStyle (const DocumentPtr &document, unsigned int style_idx)

  Add the four default styles for one palette colour to a KML document.

Protected Member Functions

- void set_gps_position_datetime (unsigned int gpsWeek, unsigned int gpsDay, unsigned int gpsHour=0, unsigned int gpsMinute=0, unsigned int gpsSecond=0)

  Set the date and time of this position from the given GPS week, day, and time.
- void set_id_dive (int16_t id_dive)

  Set the dive or cycle number of this message.
- void set_id_instrument (uint16_t id_instrument)

  Set the unique instrument identifier.
- void set_id_xmsg_type (XMsgTypeId_t id_xmsg_type)

  Set the X-Message type identifier.
- void set_position_datetime (const ptime &datetime)

  Set the date/time of the position.
- void set_position_is_valid (bool is_valid=true)

  Set the position is valid flag.
- void set_position_latitude (double lat)

  Set the latitude of the position, in decimal degrees.
- void set_position_longitude (double lon)

  Set the longitude of the position, in decimal degrees.
- void set_xmsg_size_bytes (uint16_t xmsg_size_bytes)

  Set the size of the X-Message, in bytes.

Static Protected Member Functions

- static float **bitcastUint32ToFloat** (uint32_t u)
- template<typename T >
  static double CalculateEllipsoidHeight (T lat)

  Get the approximate surface altitude or height of earth for a given latitude, in kilometers.
- template<typename T >
  static double CalculateGeographicDistance (T lat1, T lon1, T lat0, T lon0)

  Get the geographical distance between two position, in kilometers, from the positions' locations, in decimal degrees.
- template<typename T >
  static double degToRad (T deg)

  Convert angular units from from degrees to decimal radians.
- static string GenerateCurrentUtcTimeIso8601String ()

  Generate an ISO8601 format date/time string representation of the current time, in UTC.
- template<typename T >
  static std::string GenerateIso6709String (T lat, T lon)

  Generate an IS06709 string representation of the position given by `(lat, lon)`.
- static string GenerateIso8601String (const ptime &p, bool extended=true)

  Generate an ISO8601 format date/time string representation of a POSIX time object.
- static double GenerateJulianDay (const ptime &dt, const ptime &ref_dt)

  Convert a date/time to fractional days since a reference date/time.
- static ptime GetCurrentUtcTime ()

  Generate a POSIX time object representing the current date and time (rounded to the nearest second).
- static sql::Connection * getSqlConnection ()

  Generate a thread-specific instance of a MySQL connection object.
- template<typename T >
  static bool IsPositionValid (T lat, T lon)

  Check position validity.

Private Member Functions

- void **set_horizontal_dilution_of_precision** (float horizontal_dilution_of_precision)
- void **set_id_type** (uint8_t type)
- void **set_num_satellites** (uint8_t num_satellites)
- void **set_signal_to_noise_max** (uint8_t signal_to_noise_max)
- void **set_signal_to_noise_mean** (uint8_t signal_to_noise_mean)
- void **set_signal_to_noise_min** (uint8_t signal_to_noise_min)
- void **set_time_to_acquire_fix** (uint16_t time_to_acquire_fix)

Static Private Member Functions

- static void CleanupSqlConnection (Connection *con)

  Cleanup an SQL connection.
- static void CleanupSqlDriver (MySQL_Driver *driver)

  Function for memory cleanup of MySQL driver on destruct.
- static void DeleteFromDatabase (int id_inst)

  Delete all GPS message rows for an instrument from the SQL database.
- static void DeleteFromDatabase (int id_inst, int id_dive)

  Delete the GPS message rows for one dive of an instrument from the SQL database.

Private Attributes

- float horizontal_dilution_of_precision_

  The horizontal Dilution of Precision (HDOP) of the GPS fix.
- int16_t id_dive_

  The dive or cycle number of this message.
- uint16_t id_instrument_

  The unique WHOI identifier of this instrument.
- XMsgTypeId_t id_xmsg_type_

  The X-message type identifier.
- uint8_t num_satellites_

  The number of satellites used in GPS fix.
- ptime position_datetime_

The date and time of the position.

- bool position_is_valid_

  The valid position indicator.
- double position_latitude_

  The latitude of the position, in decimal degrees.
- double position_longitude_

  The longitude of the position, in decimal degrees.
- uint8_t signal_to_noise_max_

  The maximum Signal-to-Noise Ratio (SNR) for this GPS fix.
- uint8_t signal_to_noise_mean_

  The mean Signal-to-Noise Ratio (SNR) for this GPS fix.
- uint8_t signal_to_noise_min_

  The minimum Signal-to-Noise Ratio (SNR) for this GPS fix.
- uint16_t time_to_acquire_fix_

  The time to acquire this GPS fix, in seconds.
- uint16_t xmsg_size_bytes_

  The size of the X message, in bytes.

Static Private Attributes

- static string address_

  The SQL server qualified name or IP address string.
- static const double EARTH_RADIUS_KM_EQUATORIAL = 6378.1

  The equatorial radius of the earth, in kilometers.
- static const double EARTH_RADIUS_KM_POLAR = 6356.1

  The polar radius of the earth, in kilometers.
- static const Color32 kml_default_path_colors_ [KML_NUM_DFLT_PATH_COLORS]

  The colour palette for instrument paths and placemarks, indexed by style index.
- static const string kml_default_path_style_ids_ [KML_NUM_DFLT_PATH_COLORS]

  The KML style IDs of the path (line) styles, one per palette colour.
- static const string kml_default_placemark_style_ids_ [KML_NUM_DFLT_PATH_COLORS]

  The KML style IDs of the large placemark styles, one per palette colour.
- static thread_specific_ptr< KmlFactory > kml_factory_

  The libkml factory used to create KML elements, held per thread.
- static const string kml_small_placemark_style_ids_ [KML_NUM_DFLT_PATH_COLORS]

  The KML style IDs of the small placemark styles, one per palette colour.
- static const string kml_triangle_placemark_style_ids_ [KML_NUM_DFLT_PATH_COLORS]

  The KML style IDs of the triangle placemark styles, one per palette colour.
- static string password_

  The SQL server account password string.
- static unsigned int port_

  The SQL server port number.
- static string protocol_

  The protocol to use in the SQL server connection e.g.
- static string schema_

  The SQL server schema name.
- static thread_specific_ptr< Connection > sql_connection_

  Tthe thread-specific pointer to MySQL connection object.
- static thread_specific_ptr< MySQL_Driver > sql_driver_

  The thread-specific pointer to MySQL driver object.
- static const util::mysql::SqlTableDescription< XGPS > **sql_table_description_**
- static string user_

  The SQL server account username string.

Friends

- class **argo::iridium::xfloat::XFloatDiveData**
- shared_ptr< XGPS > std::make_shared ()

  Friend function for creating an X-Float object using the private constructor and a shared pointer to that object.

10.94.1 Detailed Description

Class representing a GPS type X-Message.

10.94.2 Constructor & Destructor Documentation

10.94.2.1 XGPS() [1/2]

```
util::xmsg::XGPS::XGPS (
            int16_t dive,
            uint16_t whoi,
            XMsgTypeId_t type)  [inline]
```

The parameterized, blank constructor.

Parameters

in	*dive*	The dive or cycle number of this message.
in	*whoi*	The unique WHOI identifier.
in	*type*	The X-Message type identifier.

Exceptions

None.	Never throws.

10.94.2.2 XGPS() [2/2]

```
util::xmsg::XGPS::XGPS (
            int16_t dive,
            uint16_t whoi,
            const vector< uint8_t > & data)  [inline]
```

The parameterized, binary data constructor.

Parameters

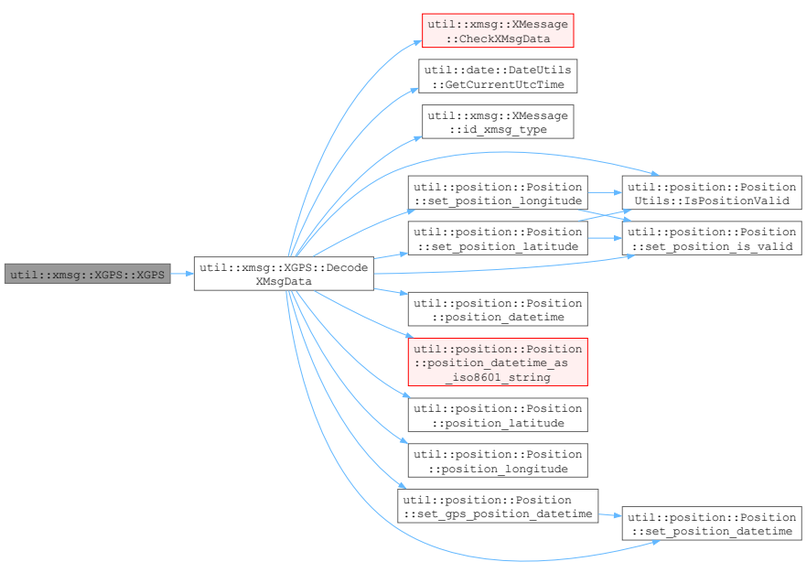
in	*dive*	The dive or cycle number of this message.
in	*whoi*	The unique WHOI identifier.
in	*data*	The binary X-message data to decode.

Exceptions

util::log::MessageException	if an decode error occurs.

References DecodeXMsgData().

Here is the call graph for this function:



10.94.2.3 ~XGPS()

```
util::xmsg::XGPS::~XGPS ()  [inline]
```

The destructor.

10.94.3 Member Function Documentation

10.94.3.1 CalculateEllipsoidHeight()

```
template<typename T >
static double util::position::PositionUtils::CalculateEllipsoidHeight (
            T lat)  [inline], [static], [inherited]
```

Get the approximate surface altitude or height of earth for a given latitude, in kilometers.

Warning

This method provides approximate surface height estimates only! This approximation is calculated from a reference ellipsoid – a mathematically-defined surface that approximates the geoid, the truer figure of the Earth.

Parameters

in	*lat*	The geographic latitude of interest, in decimal degrees.

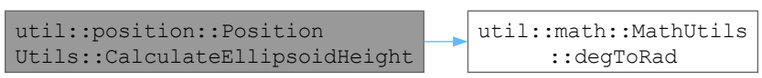
Returns

The approximate surface altitude or height of earth for the given latitude, in kilometers.

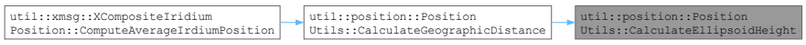
References util::math::MathUtils::degToRad(), util::position::PositionUtils::EARTH_RADIUS_KM_EQUATORIAL, and util::position::PositionUtils::EARTH_RADIUS_KM_POLAR.

Referenced by util::position::PositionUtils::CalculateGeographicDistance().

Here is the call graph for this function:



Here is the caller graph for this function:



10.94.3.2 CalculateGeographicDistance()

```
template<typename T >
static double util::position::PositionUtils::CalculateGeographicDistance (
            T lat1,
            T lon1,
            T lat0,
            T lon0)  [inline], [static], [inherited]
```

Get the geographical distance between two position, in kilometers, from the positions' locations, in decimal degrees.

Warning

This method provides approximate geographic estimates only! This approximation is calculated from a reference ellipsoid – a mathematically-defined surface that approximates the geoid, the truer figure of the Earth.

Note

This method uses the 'haversine' formula to calculate the great-circle distance between two points – that is, the shortest distance over the earth's surface – giving an 'as-the-crow-flies' distance between the points (ignoring any hills they fly over, of course!).

Parameters

in	*lat1*	The latitude of the geographic final position, in decimal degrees North.
in	*lon1*	The longitude of the geographic final position, in decimal degrees East.
in	*lat0*	The latitude of the geographic initial position, in decimal degrees North.
in	*lon0*	The longitude of the geographic initial position, in decimal degrees East.

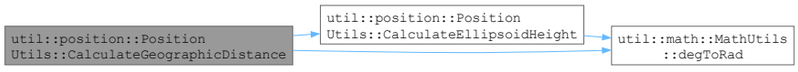
Returns

The geographical distance between the two given positions, in kilometers.

References util::position::PositionUtils::CalculateEllipsoidHeight(), and util::math::MathUtils::degToRad().

Referenced by util::xmsg::XCompositeIridiumPosition::ComputeAverageIrdiumPosition().

Here is the call graph for this function:



Here is the caller graph for this function:

10.94.3.3 CheckXMsgData()

```
const uint8_t * util::xmsg::XMessage::CheckXMsgData (
            const vector< uint8_t > & data,
            int id,
            int minBytes,
            bool parseSize = true)  [inline], [inherited]
```

Check the X-message for the correct ID, minimum size, and message terminator.

Parameters

in	*data*	The X-message binary data to check.
in	*id*	The expected ID of the X-message. If this field is less that 0, then the ID check will be skipped.
in	*minBytes*	The minimum number of bytes expected in a valid message.
in	*parseSize*	If TRUE, then the message size field will be parsed from the X-Message. Otherwise, the size field will not be parsed.

Exceptions

util::log::MessageException	if • the message does not contain the expected ID, • the message does not contain the minimum number of expected bytes, • the message does not contain a terminator (';') in the correct location, or, • any other error occurs.

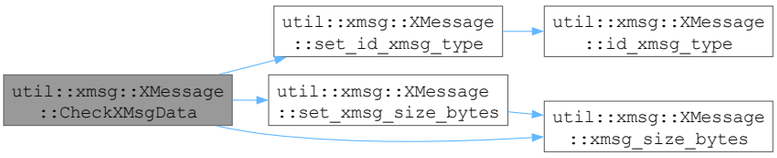
Returns

A pointer to the underlying data buffer of data.

References CATCH_BLOCK, EXC_THROW, lwrn, util::xmsg::XMessage::set_id_xmsg_type(), util::xmsg::XMessage::set_xmsg_size_byte
u16, and util::xmsg::XMessage::xmsg_size_bytes().

Referenced by util::xmsg::XAcousticPosition::DecodeXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XDriftPressure::Dec
util::xmsg::XDriftSalinity::DecodeXMsgData(), util::xmsg::XDriftTemperature::DecodeXMsgData(), util::xmsg::XEngAbort::DecodeXMsgDa
util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngBIT::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(),
util::xmsg::XEngDiagnostic::DecodeXMsgData(), util::xmsg::XEngDrift::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(),
util::xmsg::XEngStandard::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), DecodeXMsgData(),
util::xmsg::XHiResPressure::DecodeXMsgData(), util::xmsg::XHiResSalinity::DecodeXMsgData(), util::xmsg::XHiResTemperature::Decod
util::xmsg::XMissionCmdEcho::DecodeXMsgData(), util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XProfilePressure::
util::xmsg::XProfileSalinity::DecodeXMsgData(), util::xmsg::XProfileTemperature::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgD
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the call graph for this function:



10.94.3.4 CleanupSqlConnection()

```
static void util::mysql::MysqlUtils::CleanupSqlConnection (
            Connection * con)  [inline], [static], [private], [inherited]
```

Cleanup an SQL connection.

This function for memory cleanup of the SQL connection on destruction. The thread-specific SQL connection object will be deleted if it exists.

Parameters

in	*con*	The SQL connection to cleanup.

Exceptions

log::MessageException	if an error occurs while cleaning up the SQL connection object.

References CATCH_BLOCK.

10.94.3.5 CleanupSqlDriver()

```
static void util::mysql::MysqlUtils::CleanupSqlDriver (
            MySQL_Driver * driver)  [inline], [static], [private], [inherited]
```

Function for memory cleanup of MySQL driver on destruct.

Signals the SQL driver to end operation, if it exists.

Parameters

in	*driver*	Driver to cleanup.

Exceptions

log::MessageException	if an error occurs while cleaning up the SQL driver object.

References CATCH_BLOCK.

10.94.3.6 CloseKmlFactory()

```
static void util::kml::KmlUtil::CloseKmlFactory ()  [inline], [static], [inherited]
```

Release this thread's KML factory pointer.

Warning

This deletes libkml's shared factory (see kml_factory_). It is not currently called.

Exceptions

util::log::MessageException	if an error occurs.

References CATCH_BLOCK, and util::kml::KmlUtil::kml_factory_.

10.94.3.7 CloseSqlConnection()

```
static void util::mysql::MysqlUtils::CloseSqlConnection ()  [inline], [static], [inherited]
```

Close the thread-specific MySQL connection if it is currently open.

If the thread local MySQL connection is not currently open, then this function will have no effect.

Note

The MySQL connection should be closed when idle to prevent the connection from timing out and throwing exceptions.

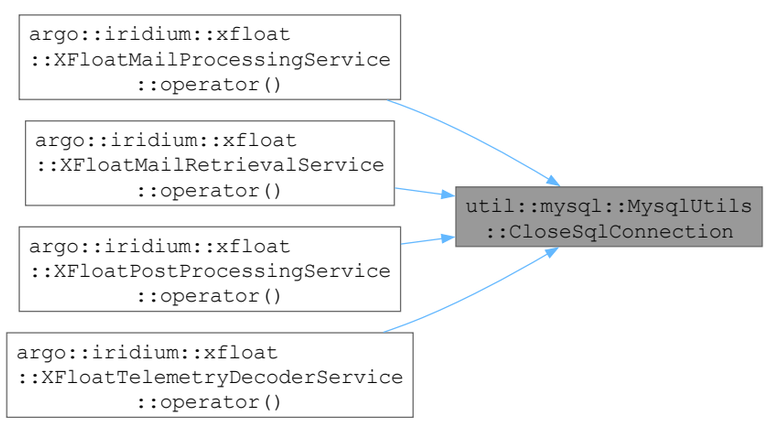
Exceptions

log::MessageException	if an error occurs while attempting to close the MySQL connection.

References CATCH_BLOCK, and util::mysql::MysqlUtils::sql_connection_.

Referenced by argo::iridium::xfloat::XFloatMailProcessingService::operator()(), argo::iridium::xfloat::XFloatMailRetrievalService::operator(
argo::iridium::xfloat::XFloatPostProcessingService::operator()(), and argo::iridium::xfloat::XFloatTelemetryDecoderService::operator()().

Here is the caller graph for this function:



10.94.3.8 DecodeXMsgData()

```
void util::xmsg::XGPS::DecodeXMsgData (
            const vector< uint8_t > & data)  [inline], [virtual]
```

Decode the GPS X-message from binary data.

Parameters

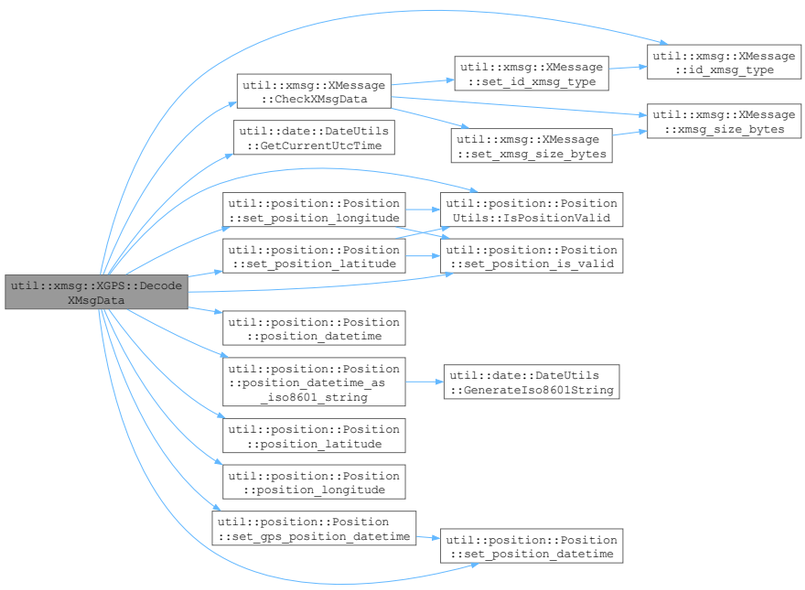
in	*data*	The binary GPS X-message data to decode.

Implements util::xmsg::XMessage.

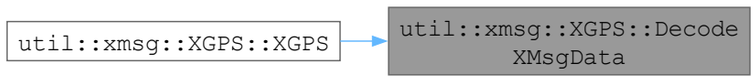
References CATCH_BLOCK, util::xmsg::XMessage::CheckXMsgData(), EXC_THROW, util::date::DateUtils::GetCurrentUtcTime(), horizontal_dilution_of_precision_, i08, i32, util::xmsg::XMessage::id_xmsg_type(), util::position::PositionUtils::IsPositionValid(), ldbg, num_satellites_, util::position::Position::position_datetime(), util::position::Position::position_datetime_as_iso8601_string(), util::position::Position::position_latitude(), util::position::Position::position_longitude(), util::position::Position::set_gps_position_datetime(), util::position::Position::set_position_datetime(), util::position::Position::set_position_is_valid(), util::position::Position::set_position_latitude(util::position::Position::set_position_longitude(), signal_to_noise_max_, signal_to_noise_mean_, signal_to_noise_min_, time_to_acquire_fix_, u08, u16, UTIL_POS_INVALID_LAT, and UTIL_POS_INVALID_LON.

Referenced by XGPS().

Here is the call graph for this function:



Here is the caller graph for this function:



10.94.3.9 degToRad()

```
template<typename T >
static double util::math::MathUtils::degToRad (
            T deg)  [inline], [static], [inherited]
```

Convert angular units from from degrees to decimal radians.

Parameters

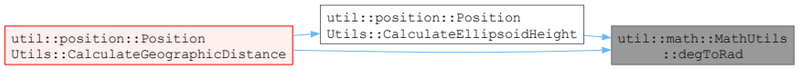
in	*deg*	Angle in integer or decimal degrees to convert to radians.

Returns

Angle in decimal radians.

Referenced by util::position::PositionUtils::CalculateEllipsoidHeight(), and util::position::PositionUtils::CalculateGeographicDistance().

Here is the caller graph for this function:



10.94.3.10 DeleteFromDatabase() [1/2]

```
static void util::xmsg::XGPS::DeleteFromDatabase (
            int id_inst)  [inline], [static], [private]
```

Delete all GPS message rows for an instrument from the SQL database.

Parameters

in	*id_inst*	The unique instrument identifier.

Exceptions

util::log::MessageException	if the delete query fails.

References CATCH_BLOCK.

10.94.3.11 DeleteFromDatabase() [2/2]

```
static void util::xmsg::XGPS::DeleteFromDatabase (
            int id_inst,
            int id_dive)  [inline], [static], [private]
```

Delete the GPS message rows for one dive of an instrument from the SQL database.

Warning

The generated condition uses == , which MySQL does not accept, so this query would fail. It is not currently called.

Parameters

in	*id_inst*	The unique instrument identifier.
in	*id_dive*	The dive or cycle number.

Exceptions

util::log::MessageException	if the delete query fails.

References CATCH_BLOCK, and util::xmsg::XMessage::id_dive().

Here is the call graph for this function:

10.94.3.12 DeleteObjectsFromDatabase()

```
static void util::xmsg::XGPS::DeleteObjectsFromDatabase (
            const string & where)  [inline], [static]
```

Delete GPS message rows from the SQL database table.

Parameters

in	*where*	The SQL "where-condition" clause selecting the rows to delete.

Exceptions

util::log::MessageException	if the delete query fails.

References CATCH_BLOCK.

10.94.3.13 ExecuteQuery()

```
static shared_ptr< ResultSet > util::mysql::MysqlUtils::ExecuteQuery (
            Connection * con,
            const string & query)  [inline], [static], [inherited]
```

Execute an SQL query and return a shared pointer to the ResultSet object.

Parameters

in	*con*	A pointer to the SQL connection object. This must not be NULL.
in	*query*	The SQL query to execute. This must be a well-formatted SQL query string.

Exceptions

log::MessageException	if 'con' is null, 'query' is a malfored SQL string, or any other error occurs.

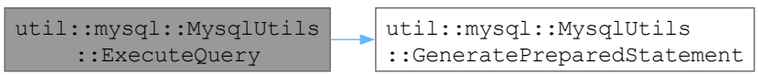
Returns

A shared pointer to the query ResultSet object.

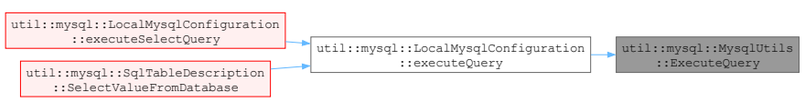
References CATCH_BLOCK, and util::mysql::MysqlUtils::GeneratePreparedStatement().

Referenced by util::mysql::LocalMysqlConfiguration::executeQuery().

Here is the call graph for this function:



Here is the caller graph for this function:



10.94.3.14 executeQuery()

```
shared_ptr< sql::ResultSet > util::mysql::LocalMysqlConfiguration::executeQuery (
            const string & query)  [static], [inherited]
```

Execute an SQL query and return a shared pointer to the ResultSet object.

Parameters

in	*query*	The SQL query to execute. This must be a well-formatted SQL query string.

Exceptions

log::MessageException	'query' is a malfored SQL string or any other error occurs while executing the query.

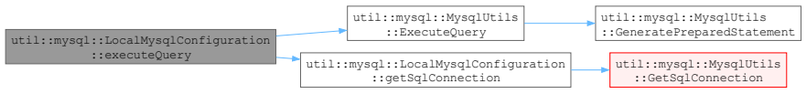
Returns

A shared pointer to the query ResultSet object.

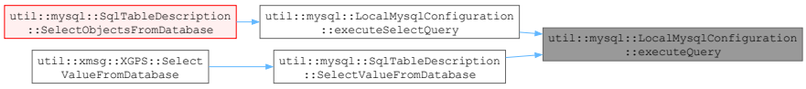
References util::mysql::MysqlUtils::ExecuteQuery(), and util::mysql::LocalMysqlConfiguration::getSqlConnection().

Referenced by util::mysql::LocalMysqlConfiguration::executeSelectQuery(), and util::mysql::SqlTableDescription< T >::SelectValueFromD

Here is the call graph for this function:



Here is the caller graph for this function:



10.94.3.15 executeSelectQuery()

```
shared_ptr< sql::ResultSet > util::mysql::LocalMysqlConfiguration::executeSelectQuery (
            const string & what,
            const string & from,
            const string & where = "",
            const string & groupBy = "",
            const string & orderBy = "",
            unsigned int limit = 0,
            unsigned int offset = 0)  [static], [inherited]
```

Execute an SQL select query and return a shared pointer to the ResultSet object.

Parameters

in	*what*	The SQL select statement select-expression clause.
in	*from*	The SQL select statement table–references clause.
in	*where*	The SQL select statement where-condition clause.
in	*groupBy*	The SQL select statement group-by clause.
in	*orderBy*	The SQL select statement order-by clause.
in	*limit*	The SQL select statement result set row limit value.
in	*offset*	The SQL select statement result set row offset.

Exceptions

log::MessageException	if an error occurs while executing the query.

Returns

A shared pointer to the ResultSet object.

References CATCH_BLOCK, util::mysql::LocalMysqlConfiguration::executeQuery(), and util::mysql::MysqlUtils::GenerateSqlSelectStr().

Referenced by util::mysql::SqlTableDescription< T >::SelectObjectsFromDatabase().

Here is the call graph for this function:

Here is the caller graph for this function:

10.94.3.16 ExecuteUpdate()

```
static int util::mysql::MysqlUtils::ExecuteUpdate (
            Connection * con,
            const string & update)  [inline], [static], [inherited]
```

Execute an SQL update and return the status code.

Parameters

in	*con*	A pointer to the SQL connection object. This must not be NULL.
in	*update*	The SQL update to execute. This must be a well-formatted SQL update string.

Exceptions

log::MessageException	if 'con' is null, 'update' is a malformed SQL string, or any other error occurs.

Returns

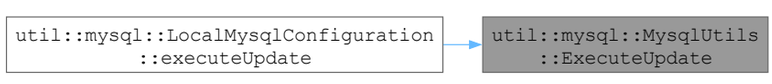
The update status code.

References CATCH_BLOCK, and util::mysql::MysqlUtils::GeneratePreparedStatement().

Referenced by util::mysql::LocalMysqlConfiguration::executeUpdate().

Here is the call graph for this function:

Here is the caller graph for this function:



10.94.3.17 executeUpdate()

```
int util::mysql::LocalMysqlConfiguration::executeUpdate (
            const string & update)  [static], [inherited]
```

Execute an SQL update and return the status code.

Parameters

in	*update*	The SQL update to execute. This must be a well-formatted SQL update string.

Exceptions

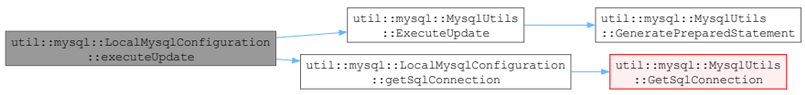
log::MessageException	if 'update' is a malformed SQL string, or any other error occurs while executing the update.

Returns

The update status code.

References util::mysql::MysqlUtils::ExecuteUpdate(), and util::mysql::LocalMysqlConfiguration::getSqlConnection().

Here is the call graph for this function:



10.94.3.18 GenerateCurrentUtcTimeIso8601String()

```
static string util::date::DateUtils::GenerateCurrentUtcTimeIso8601String ()  [inline], [static],
[inherited]
```

Generate an ISO8601 format date/time string representation of the current time, in UTC.

Note

All date/time objects use the UTC time-zone.

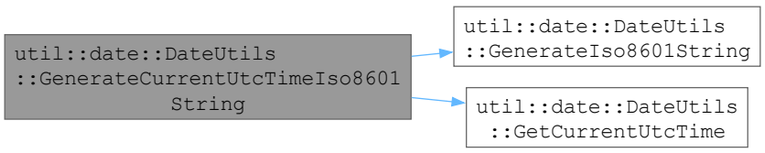
Returns

An ISO8601 format date/time string representation of the current time.

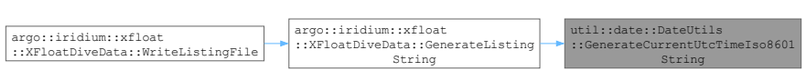
References util::date::DateUtils::GenerateIso8601String(), and util::date::DateUtils::GetCurrentUtcTime().

Referenced by argo::iridium::xfloat::XFloatDiveData::GenerateListingString().

Here is the call graph for this function:



Here is the caller graph for this function:



10.94.3.19 GenerateIso6709String()

```
template<typename T >
static std::string util::position::PositionUtils::GenerateIso6709String (
            T lat,
            T lon)  [inline], [static], [inherited]
```

Generate an IS06709 string representation of the position given by (`lat`, `lon`).

Parameters

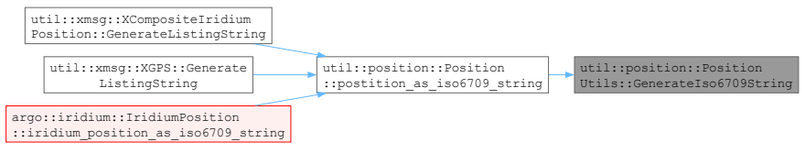
in	*lat*	The latitude of the position, in degrees North.
in	*lon*	The longitude of the position, in degrees East.

Returns

An IS06709 string representation of the position given by (`lat`, `lon`).

Referenced by util::position::Position::postition_as_iso6709_string().

Here is the caller graph for this function:



10.94.3.20 GenerateIso8601String()

```
static string util::date::DateUtils::GenerateIso8601String (
            const ptime & p,
            bool extended = true)  [inline], [static], [inherited]
```

Generate an ISO8601 format date/time string representation of a POSIX time object.

Note

All date/time objects use the UTC time-zone.

Parameters

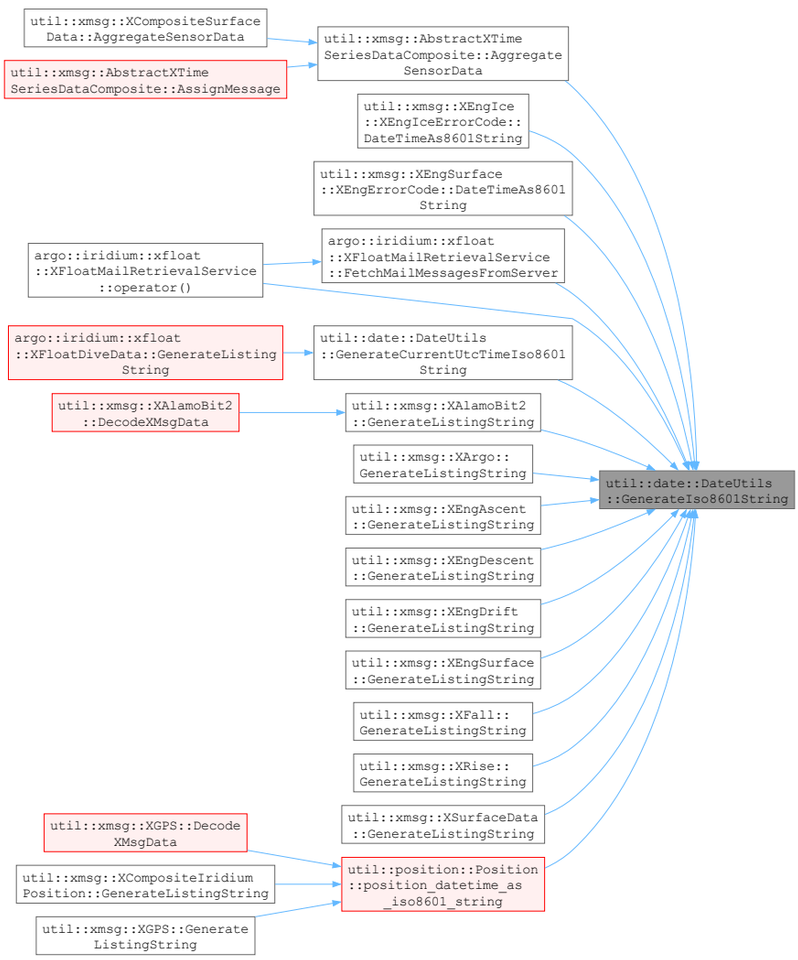
in	*p*	POSIX time from which to generate string.
in	*extended*	If TRUE, use the ISO 8601 extended format (YYYY-MM-DDThh:mm:ssZ); otherwise, use the basic format (YYYYMMDDThhmmss).

Returns

An ISO8601 format date/time string representation of the POSIX time object. If p is equal to boost::posix_↩ time::not_a_date_time, then the string "0000-00-00T00:00:00Z" (extended) or "00000000000000" (basic) will be returned.

Referenced by util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::AggregateSensorData(), util::xmsg::XEngIce::XEngIce
util::xmsg::XEngSurface::XEngErrorCode::DateTimeAs8601String(), argo::iridium::xfloat::XFloatMailRetrievalService::FetchMailMessages
util::date::DateUtils::GenerateCurrentUtcTimeIso8601String(), util::xmsg::XAlamoBit2::GenerateListingString(), util::xmsg::XArgo::Genera
util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListing
util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XRise::GenerateListingString(),
util::xmsg::XSurfaceData::GenerateListingString(), argo::iridium::xfloat::XFloatMailRetrievalService::operator()(), and
util::position::Position::position_datetime_as_iso8601_string().

Here is the caller graph for this function:



10.94.3.21 GenerateJulianDay()

```
static double util::date::DateUtils::GenerateJulianDay (
            const ptime & dt,
            const ptime & ref_dt)  [inline], [static], [inherited]
```

Convert a date/time to fractional days since a reference date/time.

Parameters

in	*dt*	The date/time to convert.
in	*ref↩ _dt*	The reference date/time (day 0).

Exceptions

util::log::MessageException	if either date/time is not valid.

Returns

The number of days (with millisecond resolution) from `ref_dt` to `dt`.

References CATCH_BLOCK, and EXC_THROW.

10.94.3.22 GenerateListingComment()

```
static string util::xmsg::XGPS::GenerateListingComment ()  [inline], [static]
```

Get the listing entry comment string.

Returns

The listing entry comment string.

References CATCH_BLOCK.

10.94.3.23 GenerateListingString()

```
const string util::xmsg::XGPS::GenerateListingString () const  [inline]
```

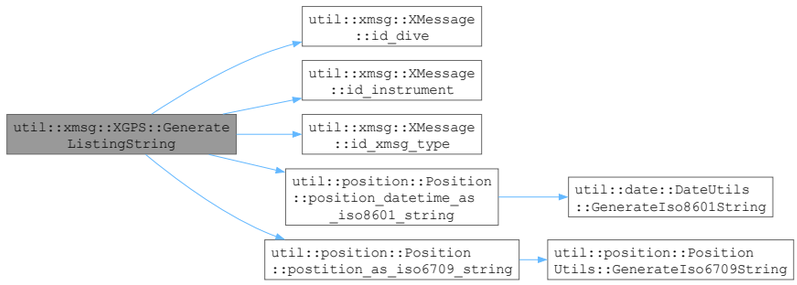
Return a string representation of this GPS message.

Returns

A string representation of this GPS message.

References CATCH_BLOCK, util::xmsg::XMessage::id_dive(), util::xmsg::XMessage::id_instrument(), util::xmsg::XMessage::id_xmsg_typ
util::position::Position::position_datetime_as_iso8601_string(), and util::position::Position::postition_as_iso6709_string().

Here is the call graph for this function:



10.94.3.24 GeneratePhyString()

```
const string util::xmsg::XGPS::GeneratePhyString (
            size_t xmits) const  [inline]
```

Generate a phy file entry string representation of this GPS position message object.

Parameters

in	*xmits*	The number of transmissions for this dive data.

Exceptions

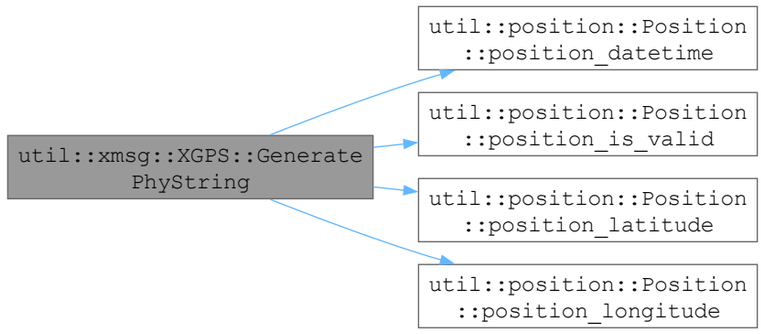
log::MessageException	if an error occurs

Returns

A phy file entry string representation of this GPS position message object.

References CATCH_BLOCK, num_satellites_, util::position::Position::position_datetime(), util::position::Position::position_is_valid(), util::position::Position::position_latitude(), and util::position::Position::position_longitude().

Here is the call graph for this function:



10.94.3.25 GeneratePreparedStatement()

```
static shared_ptr< PreparedStatement > util::mysql::MysqlUtils::GeneratePreparedStatement (
            Connection * con,
            const string & statement)  [inline], [static], [inherited]
```

Generate a prepared SQL statement object for the given SQL statement.

Parameters

in	*con*	A pointer to the SQL connection object. This must not be NULL.
in	*statement*	The SQL statement to prepare. This must be a well-formed SQL string.

Exceptions

log::MessageException	if 'con' is NULL, 'statement' is not a valid SQL string, or if an error occurs.

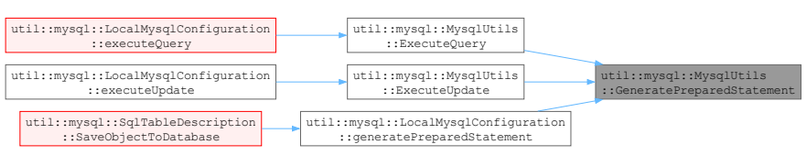
Returns

A prepared SQL statement object generated from the given SQL statement.

References CATCH_BLOCK, and EXC_THROW.

Referenced by util::mysql::MysqlUtils::ExecuteQuery(), util::mysql::MysqlUtils::ExecuteUpdate(), and util::mysql::LocalMysqlConfiguration:

Here is the caller graph for this function:



10.94.3.26 generatePreparedStatement()

```
std::shared_ptr< sql::PreparedStatement > util::mysql::LocalMysqlConfiguration::generatePrepared↩
Statement (
            const string & statement)  [static], [inherited]
```

Generate a prepared SQL statement object for the given SQL statement.

Parameters

in	*statement*	The SQL statement to prepare. This must be a well-formed SQL string.

Exceptions

log::MessageException	if 'statement' is not a valid SQL string or if an error occurs while generating the prepared statement object..

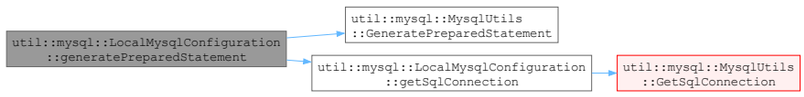
Returns

A prepared SQL statement object generated from the given SQL statement.

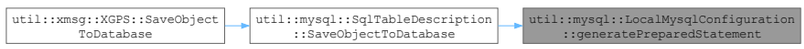
References util::mysql::MysqlUtils::GeneratePreparedStatement(), and util::mysql::LocalMysqlConfiguration::getSqlConnection().

Referenced by util::mysql::SqlTableDescription< T >::SaveObjectToDatabase().

Here is the call graph for this function:



Here is the caller graph for this function:



10.94.3.27 GenerateSqlDeleteStr()

```
static string util::mysql::MysqlUtils::GenerateSqlDeleteStr (
            const string & from,
            const string & where = "")  [inline], [static], [inherited]
```

Generate an SQL DELETE statement.

Warning

Without a `where` clause, the statement deletes every row in the table.

Parameters

in	*from*	The name of the table to delete from.
in	*where*	The SQL "where-condition" clause (empty = all rows).

Returns

The SQL statement.

10.94.3.28 GenerateSqlSelectStr()

```
static string util::mysql::MysqlUtils::GenerateSqlSelectStr (
            const string & what,
            const string & from,
            const string & where = "",
            const string & groupBy = "",
            const string & orderBy = "",
            unsigned int limit = 0,
            unsigned int offset = 0)  [inline], [static], [inherited]
```

Generate an SQL select statement.

Parameters

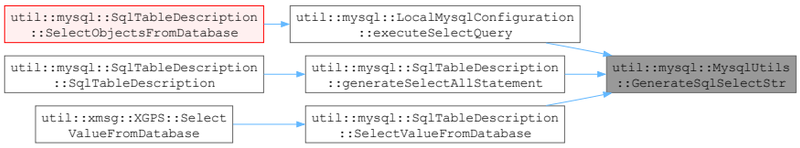
in	*what*	The SQL select statement select-expression clause.
in	*from*	The SQL select statement table-references clause.
in	*where*	The SQL select statement where-condition clause.
in	*groupBy*	The SQL select statement group-by clause.
in	*orderBy*	The SQL select statement order-by clause.
in	*limit*	The SQL select statement result set row limit value.
in	*offset*	The SQL select statement result set row offset.

Returns

The SQL select statement.

Referenced by util::mysql::LocalMysqlConfiguration::executeSelectQuery(), util::mysql::SqlTableDescription< T >::generateSelectAllState and util::mysql::SqlTableDescription< T >::SelectValueFromDatabase().

Here is the caller graph for this function:



10.94.3.29 GetCurrentUtcTime()

```
static ptime util::date::DateUtils::GetCurrentUtcTime ()  [inline], [static], [inherited]
```

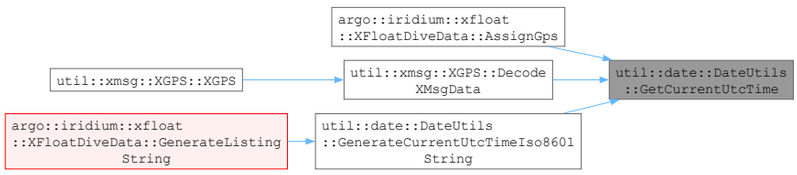
Generate a POSIX time object representing the current date and time (rounded to the nearest second).

Returns

A POSIX time object representing the current date and time (rounded to the nearest second).

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignGps(), DecodeXMsgData(), and util::date::DateUtils::GenerateCurrentUtcTime

Here is the caller graph for this function:



10.94.3.30 GetDefaltKmlPathColor()

```
static const Color32 & util::kml::KmlUtil::GetDefaltKmlPathColor (
            int id)  [inline], [static], [inherited]
```

Get the palette colour for an index (an instrument ID, for example), modulo the palette size.

Parameters

in	*id*	The index.

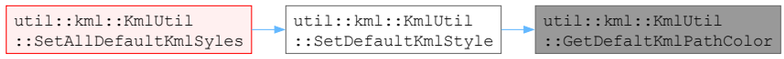
Returns

The colour.

References util::kml::KmlUtil::kml_default_path_colors_.

Referenced by util::kml::KmlUtil::SetDefaultKmlStyle().

Here is the caller graph for this function:



10.94.3.31 GetDefaltKmlPathStyleId()

```
static const string & util::kml::KmlUtil::GetDefaltKmlPathStyleId (
            int id)  [inline], [static], [inherited]
```

Get the path (line) style ID for an index, modulo the palette size.

Parameters

in	*id*	The index.

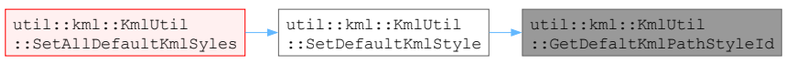
Returns

The KML style ID.

References util::kml::KmlUtil::kml_default_path_style_ids_.

Referenced by util::kml::KmlUtil::SetDefaultKmlStyle().

Here is the caller graph for this function:



10.94.3.32 GetDefaltKmlPlacemarkStyleId()

```
static const string & util::kml::KmlUtil::GetDefaltKmlPlacemarkStyleId (
            int id)  [inline], [static], [inherited]
```

Get the large placemark style ID for an index, modulo the palette size.

Parameters

in	*id*	The index.

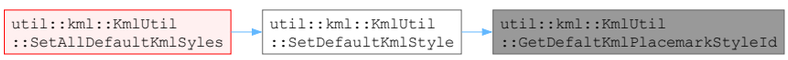
Returns

The KML style ID.

References util::kml::KmlUtil::kml_default_placemark_style_ids_.

Referenced by util::kml::KmlUtil::SetDefaultKmlStyle().

Here is the caller graph for this function:



10.94.3.33 GetKmlFactory()

```
static thread_specific_ptr< KmlFactory > & util::kml::KmlUtil::GetKmlFactory ()  [inline], [static],
[inherited]
```

Get the libkml factory, initializing it for this thread if necessary.

Exceptions

util::log::MessageException	if the factory cannot be obtained.

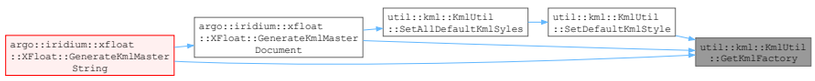
Returns

The thread's KML factory pointer (see kml_factory_).

References CATCH_BLOCK, and util::kml::KmlUtil::kml_factory_.

Referenced by argo::iridium::xfloat::XFloat::GenerateKmlMasterDocument(), argo::iridium::xfloat::XFloat::GenerateKmlMasterString(), and util::kml::KmlUtil::SetDefaultKmlStyle().

Here is the caller graph for this function:



10.94.3.34 GetSmallKmlPlacemarkStyleId()

```
static const string & util::kml::KmlUtil::GetSmallKmlPlacemarkStyleId (
            int id)  [inline], [static], [inherited]
```

Get the small placemark style ID for an index, modulo the palette size.

Parameters

in	*id*	The index.

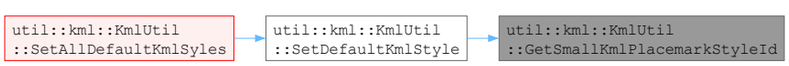
Returns

The KML style ID.

References util::kml::KmlUtil::kml_small_placemark_style_ids_.

Referenced by util::kml::KmlUtil::SetDefaultKmlStyle().

Here is the caller graph for this function:



10.94.3.35 GetSqlConnection()

```
static thread_specific_ptr< Connection > & util::mysql::MysqlUtils::GetSqlConnection (
            string url,
            string user,
            string password,
            string schema)  [inline], [static], [inherited]
```

Get a thread-specific instance of a MySQL connection object.

A new connection will be created, if necessary.

Parameters

in	*url*	The MySQL host server qualified address.
in	*user*	The MySQL account username.
in	*password*	The MySQL account password.
in	*schema*	The MySQL account schema name.

Exceptions

log::MessageException	if an error occurs while creating a new thread-specific instance of the SQL connection object

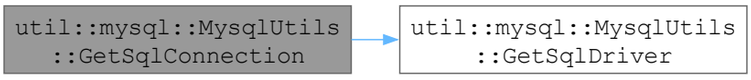
Returns

A thread-specific instance of a MySQL connection object.

References CATCH_BLOCK, util::mysql::MysqlUtils::GetSqlDriver(), and util::mysql::MysqlUtils::sql_connection_.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

Here is the call graph for this function:



Here is the caller graph for this function:

10.94.3.36 getSqlConnection()

`sql::Connection * util::mysql::LocalMysqlConfiguration::getSqlConnection () [static], [protected], [inherited]`

Generate a thread-specific instance of a MySQL connection object.

A new connection will be created, if necessary.

Exceptions

log::MessageException	if an error occurs while generating the SQL connection object.

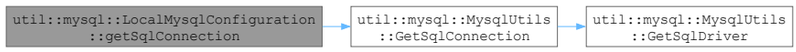
Returns

A thread-specific instance of a MySQL connection object.

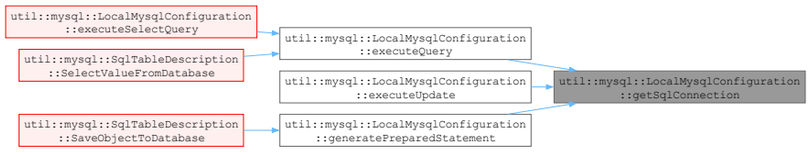
References util::mysql::LocalMysqlConfiguration::address_, CATCH_BLOCK, util::mysql::MysqlUtils::GetSqlConnection(), util::mysql::LocalMysqlConfiguration::password_, util::mysql::LocalMysqlConfiguration::port_, util::mysql::LocalMysqlConfiguration::protoc util::mysql::LocalMysqlConfiguration::schema_, and util::mysql::LocalMysqlConfiguration::user_.

Referenced by util::mysql::LocalMysqlConfiguration::executeQuery(), util::mysql::LocalMysqlConfiguration::executeUpdate(), and util::mysql::LocalMysqlConfiguration::generatePreparedStatement().

Here is the call graph for this function:



Here is the caller graph for this function:



10.94.3.37 GetSqlDriver()

```
static thread_specific_ptr< MySQL_Driver > & util::mysql::MysqlUtils::GetSqlDriver ()  [inline],
[static], [inherited]
```

Get a thread-specific instance of a MySQL driver object.

A new driver will created, if necessary.

Exceptions

log::MessageException	if an error occurs while creating a new thread-specific instance of the SQL driver object

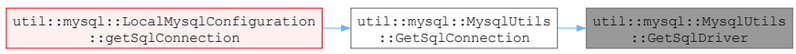
Returns

A thread-specific instance of a MySQL driver object.

References CATCH_BLOCK, and util::mysql::MysqlUtils::sql_driver_.

Referenced by util::mysql::MysqlUtils::GetSqlConnection().

Here is the caller graph for this function:



10.94.3.38 GetTriangleKmlPlacemarkStyleId()

```
static const string & util::kml::KmlUtil::GetTriangleKmlPlacemarkStyleId (
            int id)  [inline], [static], [inherited]
```

Get the triangle placemark style ID for an index, modulo the palette size.

Parameters

in	*id*	The index.

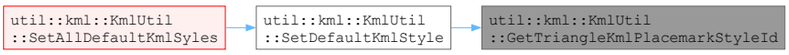
Returns

The KML style ID.

References util::kml::KmlUtil::kml_triangle_placemark_style_ids_.

Referenced by util::kml::KmlUtil::SetDefaultKmlStyle().

Here is the caller graph for this function:



10.94.3.39 id_dive()

```
int16_t util::xmsg::XMessage::id_dive () const  [inline], [inherited]
```

Get the dive or cycle number of this X-Message.

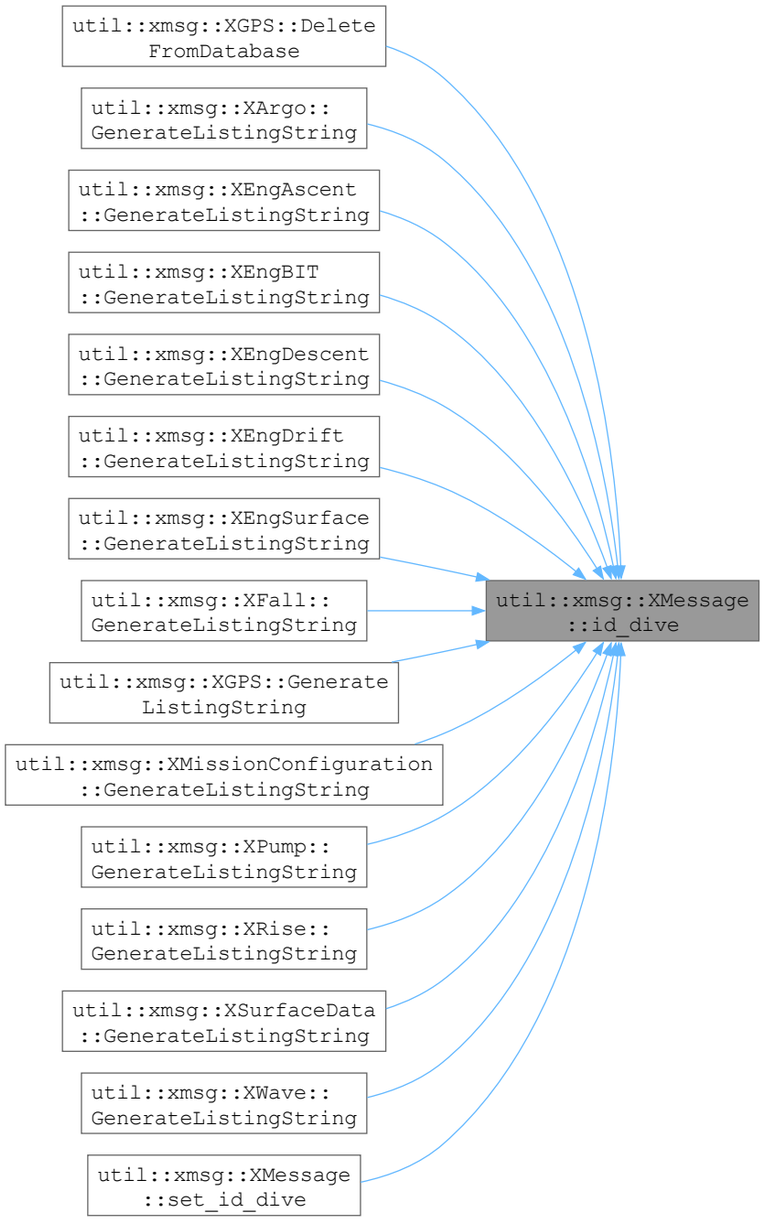
Returns

The dive or cycle number of this X-Message.

References util::xmsg::XMessage::id_dive_.

Referenced by DeleteFromDatabase(), util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngBIT::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingStr util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing util::xmsg::XSurfaceData::GenerateListingString(), util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_dive().

Here is the caller graph for this function:



10.94.3.40 id_instrument()

```
uint16_t util::xmsg::XMessage::id_instrument () const  [inline], [inherited]
```

Get the unique instrument identifier of this X-Message.

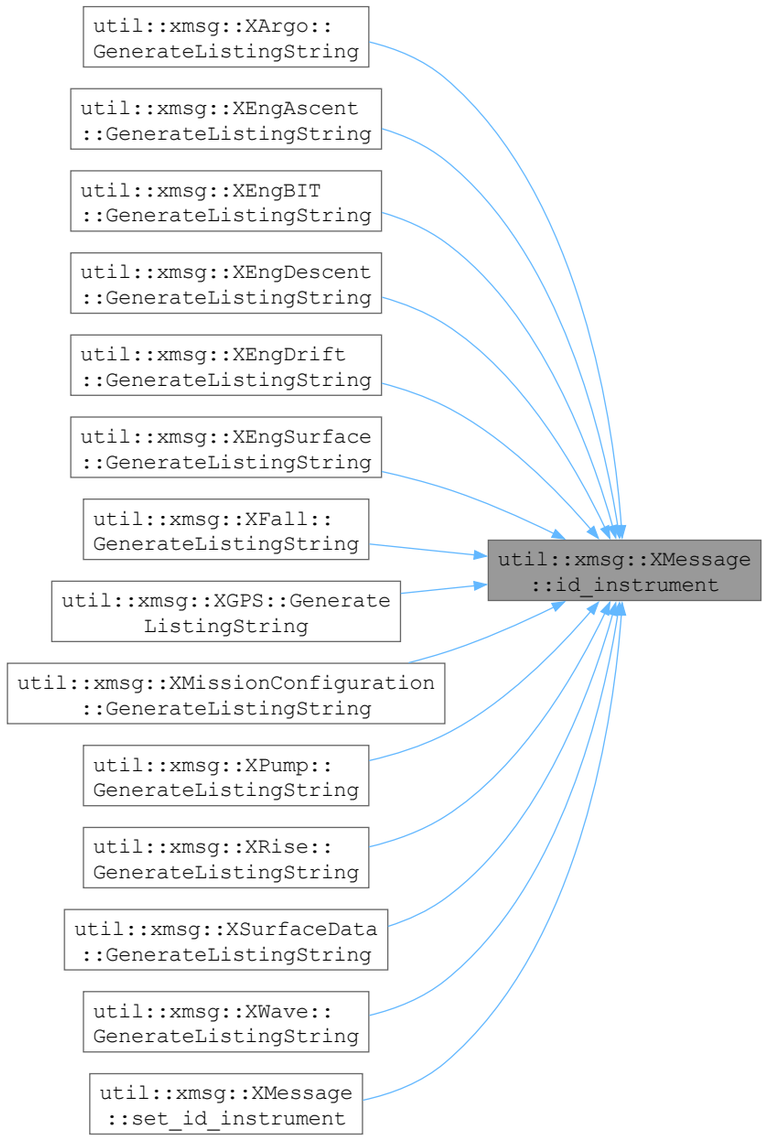
Returns

The unique instrument identifier of this X-Message.

References util::xmsg::XMessage::id_instrument_.

Referenced by util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngBIT::Generate
util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), util::xmsg::XEngSurface::GenerateListin
util::xmsg::XFall::GenerateListingString(), GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingString(),
util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(),
util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_instrument().

Here is the caller graph for this function:



10.94.3.41 id_xmsg_type()

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type () const  [inline], [inherited]
```

Get the enumerated X-Message type identifier.

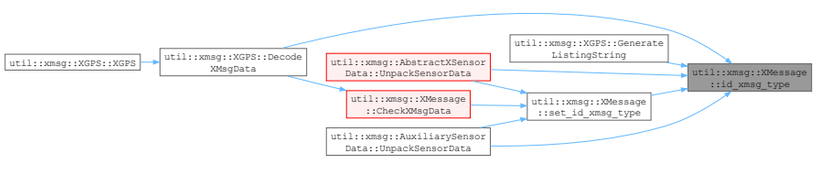
Returns

The enumerated X-Message type identifier.

References util::xmsg::XMessage::id_xmsg_type_.

Referenced by DecodeXMsgData(), GenerateListingString(), util::xmsg::XMessage::set_id_xmsg_type(), util::xmsg::AbstractXSensorData and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the caller graph for this function:



10.94.3.42 IsPositionValid()

```
template<typename T >
static bool util::position::PositionUtils::IsPositionValid (
            T lat,
            T lon)  [inline], [static], [inherited]
```

Check position validity.

Parameters

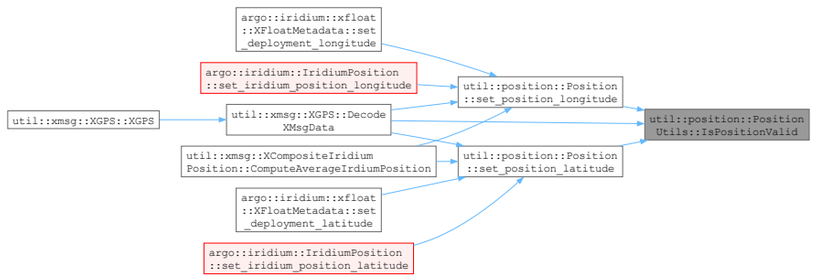
in	*lat*	Latitude of position.
in	*lon*	Longitude of position.

Returns

TRUE if (lat, lon) represents a valid geographical position, or FALSE otherwise.

Referenced by DecodeXMsgData(), util::position::Position::set_position_latitude(), and util::position::Position::set_position_longitude().

Here is the caller graph for this function:



10.94.3.43 position_datetime()

```
ptime util::position::Position::position_datetime () const  [inline], [inherited]
```

Get the date/time of the position.

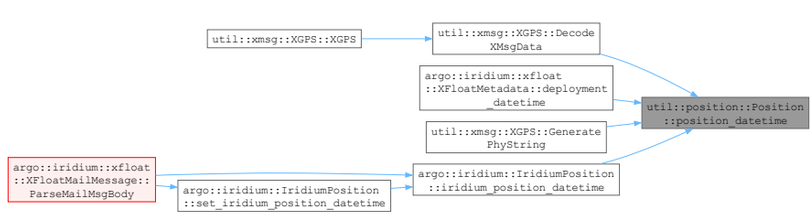
Returns

The date/time of the position.

References util::position::Position::position_datetime_.

Referenced by DecodeXMsgData(), argo::iridium::xfloat::XFloatMetadata::deployment_datetime(), GeneratePhyString(), and argo::iridium::IridiumPosition::iridium_position_datetime().

Here is the caller graph for this function:



10.94.3.44 position_datetime_as_iso8601_string()

```
string util::position::Position::position_datetime_as_iso8601_string () const  [inline], [inherited]
```

Get a formatted string representing the position date/time composed using the ISO 8601 standard.

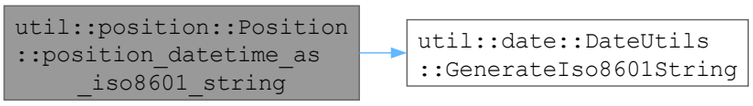
Returns

The formatted string representation of the position date/time.

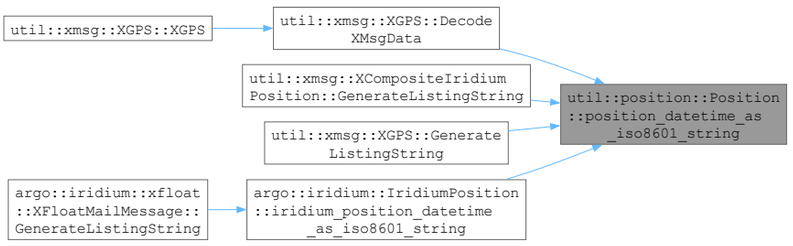
References util::date::DateUtils::GenerateIso8601String(), and util::position::Position::position_datetime_.

Referenced by DecodeXMsgData(), util::xmsg::XCompositeIridiumPosition::GenerateListingString(), GenerateListingString(), and argo::iridium::IridiumPosition::iridium_position_datetime_as_iso8601_string().

Here is the call graph for this function:



Here is the caller graph for this function:



10.94.3.45 position_is_valid()

```
bool util::position::Position::position_is_valid () const  [inline], [inherited]
```

Is the position valid?

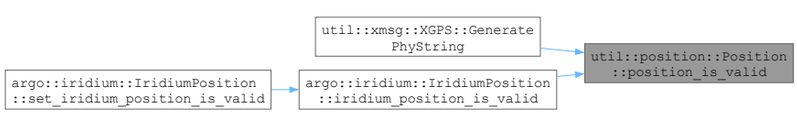
Returns

TRUE if the position is valid, or false otherwise.

References util::position::Position::position_is_valid_.

Referenced by GeneratePhyString(), and argo::iridium::IridiumPosition::iridium_position_is_valid().

Here is the caller graph for this function:



10.94.3.46 position_latitude()

```
double util::position::Position::position_latitude () const  [inline], [inherited]
```

Get the latitude of the position, in decimal degrees.

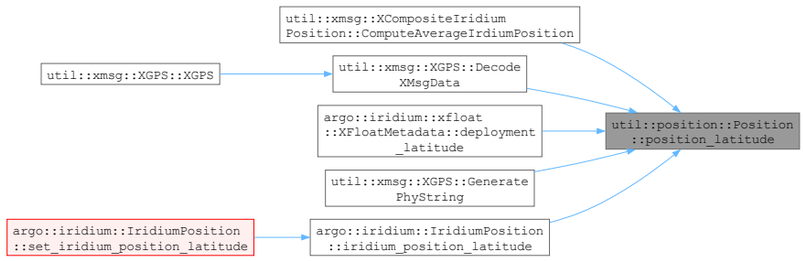
Returns

The latitude of the position, in decimal degrees.

References util::position::Position::position_latitude_.

Referenced by util::xmsg::XCompositeIridiumPosition::ComputeAverageIrdiumPosition(), DecodeXMsgData(), argo::iridium::xfloat::XFloatMetadata::deployment_latitude(), GeneratePhyString(), and argo::iridium::IridiumPosition::iridium_position_latit

Here is the caller graph for this function:



10.94.3.47 position_longitude()

```
double util::position::Position::position_longitude () const  [inline], [inherited]
```

Get the longitude of the position, in decimal degrees.

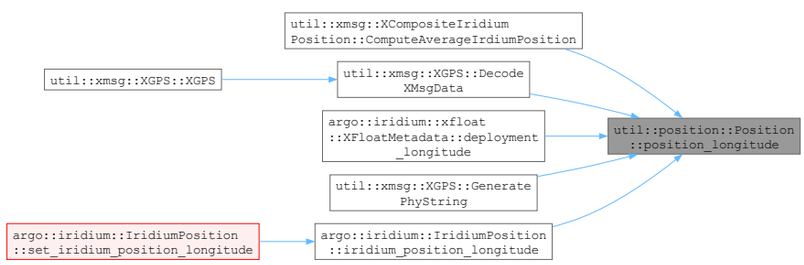
Returns

The longitude of the position, in decimal degrees.

References util::position::Position::position_longitude_.

Referenced by util::xmsg::XCompositeIridiumPosition::ComputeAverageIrdiumPosition(), DecodeXMsgData(), argo::iridium::xfloat::XFloatMetadata::deployment_longitude(), GeneratePhyString(), and argo::iridium::IridiumPosition::iridium_position_lo

Here is the caller graph for this function:



10.94.3.48 postition_as_iso6709_string()

```
string util::position::Position::postition_as_iso6709_string () const  [inline], [inherited]
```

Get a formatted string representing the position composed using the ISO 6709 standard.

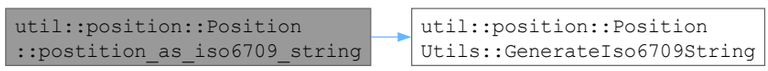
Returns

The formatted string representation of the position.

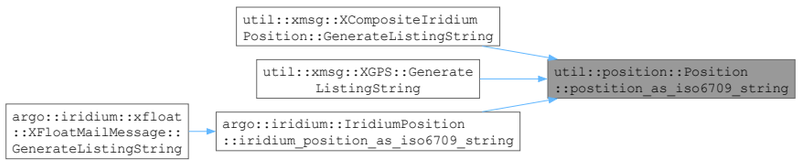
References util::position::PositionUtils::GenerateIso6709String(), util::position::Position::position_latitude_, and util::position::Position::position_longitude_.

Referenced by util::xmsg::XCompositeIridiumPosition::GenerateListingString(), GenerateListingString(), and argo::iridium::IridiumPosition:

Here is the call graph for this function:



Here is the caller graph for this function:



10.94.3.49 SaveObjectToDatabase()

```
void util::xmsg::XGPS::SaveObjectToDatabase () const  [inline], [virtual]
```

Save this GPS message object to the SQL database.

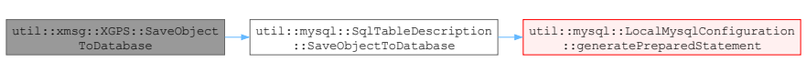
Exceptions

util::log::MessageException	if an error occurs.

Implements util::mysql::LocalPersistentEntity.

References CATCH_BLOCK, and util::mysql::SqlTableDescription< T >::SaveObjectToDatabase().

Here is the call graph for this function:



10.94.3.50 SelectAllObjectsFromDatabase()

```
static vector< shared_ptr< XGPS > > util::xmsg::XGPS::SelectAllObjectsFromDatabase (
            const string & groupBy = "",
            const string & orderBy = "",
            const unsigned int & limit = 0,
            const unsigned int & offset = 0)  [inline], [static]
```

Select all GPS message objects from the SQL database table.

Parameters

in	*groupBy*	The SQL select statement "group-by expression" clause.
in	*orderBy*	The SQL select statement "order-by expression" clause.
in	*limit*	The SQL select statement result set row limit value (0 = no limit).
in	*offset*	The SQL select statement result set row offset value.

Exceptions

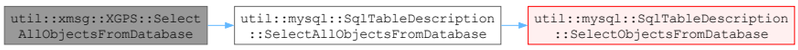
util::log::MessageException	if the query fails.

Returns

The GPS message objects.

References CATCH_BLOCK, and util::mysql::SqlTableDescription< T >::SelectAllObjectsFromDatabase().

Here is the call graph for this function:



10.94.3.51 SelectObjectFromDatabase()

```
static shared_ptr< XGPS > util::xmsg::XGPS::SelectObjectFromDatabase (
            const string & where = "",
            const string & groupBy = "",
            const string & orderBy = "",
            const unsigned int & offset = 0)  [inline], [static]
```

Select a single GPS message object from the SQL database table.

Parameters

in	*where*	The SQL select statement "where-condition" clause.
in	*groupBy*	The SQL select statement "group-by expression" clause.
in	*orderBy*	The SQL select statement "order-by expression" clause.
in	*offset*	The SQL select statement result set row offset value.

Exceptions

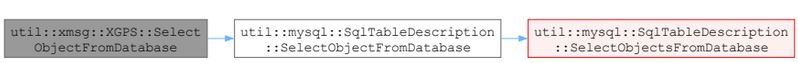
util::log::MessageException	if the query fails.

Returns

The first GPS message object matching the query.

References CATCH_BLOCK, util::mysql::SqlTableDescription< T >::SelectObjectFromDatabase(), and std::make_shared.

Here is the call graph for this function:



10.94.3.52 SelectObjectsFromDatabase()

```
static vector< shared_ptr< XGPS > > util::xmsg::XGPS::SelectObjectsFromDatabase (
            const string & where = "",
            const string & groupBy = "",
            const string & orderBy = "",
            const unsigned int & limit = 0,
            const unsigned int & offset = 0)  [inline], [static]
```

Select GPS message objects from the SQL database table.

Exceptions

util::log::MessageException	if an error occurs while querying the database or constructing the GPS message objects from the result of the query.

Parameters

in	*where*	The SQL select statement "where-condition" clause.
in	*groupBy*	The SQL select statement "group-by expression" clause.
in	*orderBy*	The SQL select statement "order-by expression" clause.
in	*limit*	The SQL select statement result set row limit value.
in	*offset*	The SQL select statement result set row offset value.

Returns

A vector of shared references to the GPS message objects constructed from the results of the SQL query.

References CATCH_BLOCK, and util::mysql::SqlTableDescription< T >::SelectObjectsFromDatabase().

Here is the call graph for this function:

10.94.3.53 SelectValueFromDatabase()

```
template<typename TT >
static TT util::xmsg::XGPS::SelectValueFromDatabase (
            const string & what,
            const string & where = "",
            const string & groupBy = "",
            const string & orderBy = "",
            const unsigned int & offset = 0)  [inline], [static]
```

Select a single value from the GPS message SQL database table.

Exceptions

util::log::MessageException	if the SQL database query fails or if an error occurs while attempting to interpret the result of the query.

Note

The result set of this query is always limited to a single result. That is, the result set row limit value is automatically set to one. If more than one row in the database table satisfy the "where condition" clause of the select statement, then only the value of the first result will be returned.

Parameters

in	*what*	The SQL select statement "what" clause. This clause may only contain a singular result e.g. the name of the SQL database table column of interest.
in	*where*	The SQL select statement "where condition" clause.
in	*groupBy*	The SQL select statement "group-by expression" clause.
in	*orderBy*	The SQL select statement "order-by expression" clause.
in	*offset*	The SQL select statement result set row offset value.

Returns

The selected value from the SQL database table.

References util::mysql::SqlTableDescription< T >::SelectValueFromDatabase().

Here is the call graph for this function:

10.94.3.54 set_gps_position_datetime()

```
void util::position::Position::set_gps_position_datetime (
            unsigned int gpsWeek,
            unsigned int gpsDay,
            unsigned int gpsHour = 0,
            unsigned int gpsMinute = 0,
            unsigned int gpsSecond = 0)  [inline], [protected], [inherited]
```

Set the date and time of this position from the given GPS week, day, and time.

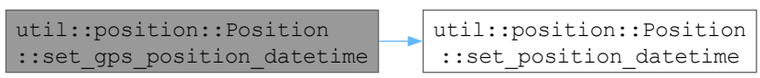
Parameters

in	*gpsWeek*	The GPS week, in weeks since midnight of January 6, 1980.
in	*gpsDay*	The GPS day of week (0 - 6).
in	*gpsHour*	The UTC hour of day (0 - 23).
in	*gpsMinute*	The UTC minute of hour (0 - 59)..
in	*gpsSecond*	The UTC second of minute (0 - 59).

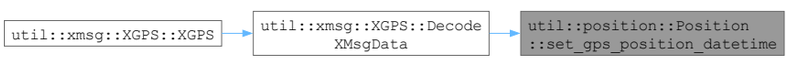
References util::position::Position::set_position_datetime().

Referenced by DecodeXMsgData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.94.3.55 set_id_dive()

```
void util::xmsg::XMessage::set_id_dive (
            int16_t id_dive)  [inline], [protected], [inherited]
```

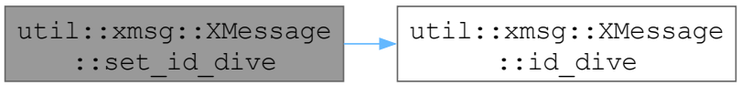
Set the dive or cycle number of this message.

Parameters

in	*id_dive*	The dive or cycle number of this message to set.

References util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::id_dive_.

Here is the call graph for this function:



10.94.3.56 set_id_instrument()

```
void util::xmsg::XMessage::set_id_instrument (
            uint16_t id_instrument)  [inline], [protected], [inherited]
```

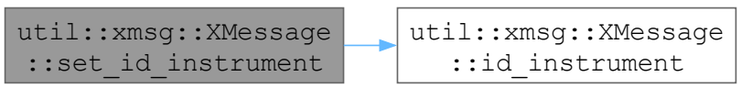
Set the unique instrument identifier.

Parameters

in	*id_instrument*	The unique instrument identifier to set.

References util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::id_instrument_.

Here is the call graph for this function:



10.94.3.57 set_id_xmsg_type()

```
void util::xmsg::XMessage::set_id_xmsg_type (
            XMsgTypeId_t id_xmsg_type)  [inline], [protected], [inherited]
```

Set the X-Message type identifier.

Parameters

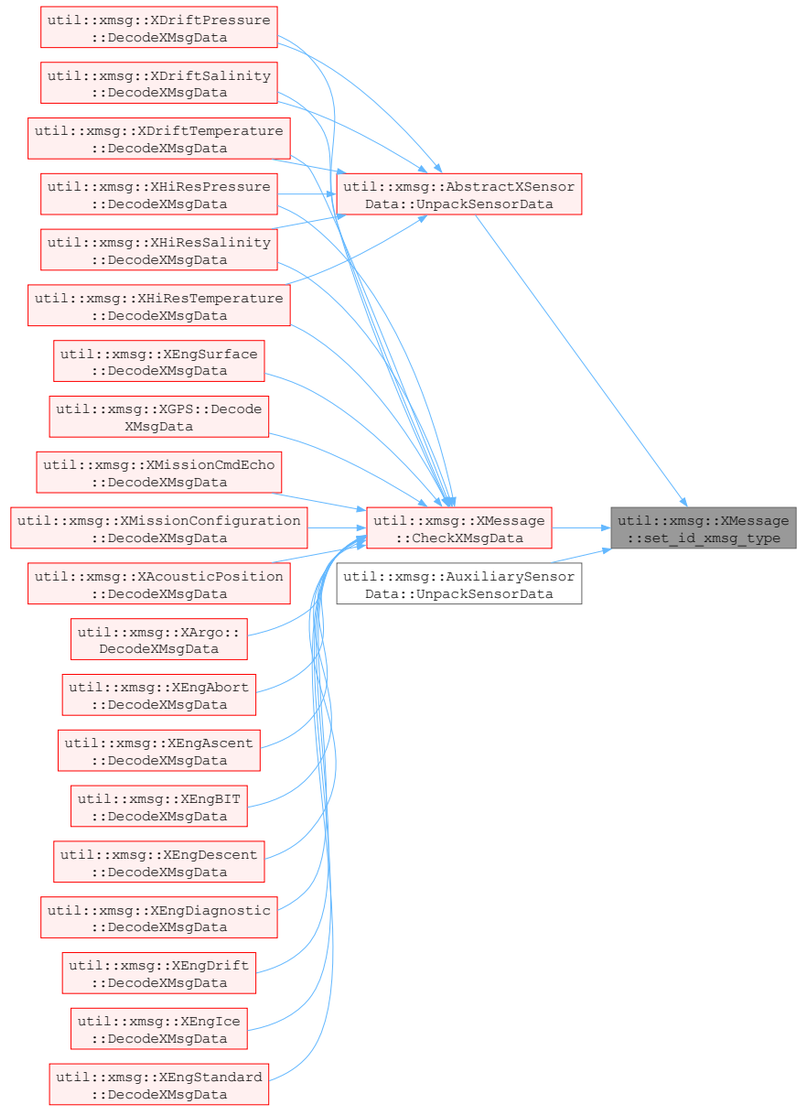
in	*id_xmsg_type*	The X-Message type identifier to set.

References util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:

Here is the caller graph for this function:



10.94.3.58 set_position_datetime()

```
void util::position::Position::set_position_datetime (
            const ptime & datetime)  [inline], [protected], [inherited]
```

Set the date/time of the position.

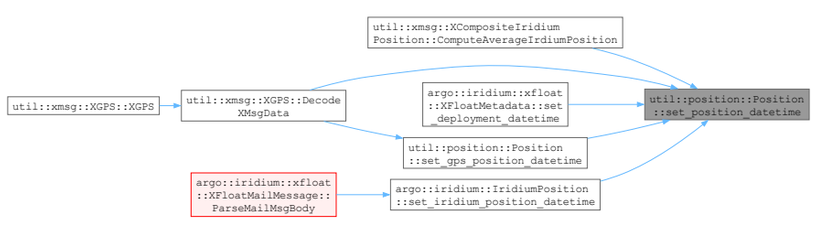
Parameters

in	*datetime*	The date/time of the position to set.

References util::position::Position::position_datetime_.

Referenced by util::xmsg::XCompositeIridiumPosition::ComputeAverageIrdiumPosition(), DecodeXMsgData(), argo::iridium::xfloat::XFloatMetadata::set_deployment_datetime(), util::position::Position::set_gps_position_datetime(), and argo::iridium::IridiumPosition::set_iridium_position_datetime().

Here is the caller graph for this function:



10.94.3.59 set_position_is_valid()

```
void util::position::Position::set_position_is_valid (
            bool is_valid = true)  [inline], [protected], [inherited]
```

Set the position is valid flag.

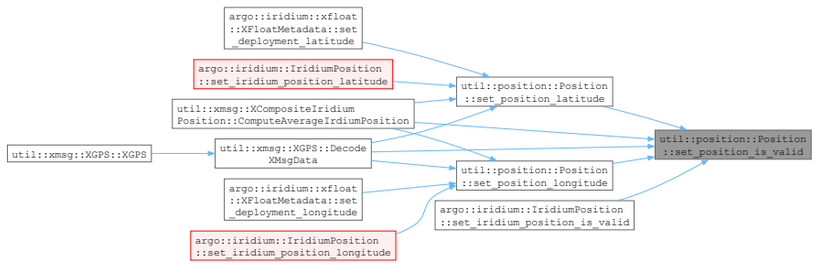
Parameters

in	*is_valid*	TRUE if the position is valid, or FALSE otherwise.

References util::position::Position::position_is_valid_.

Referenced by util::xmsg::XCompositeIridiumPosition::ComputeAverageIrdiumPosition(), DecodeXMsgData(), argo::iridium::IridiumPosition::set_iridium_position_is_valid(), util::position::Position::set_position_latitude(), and util::position::Position::set_position_longitude().

Here is the caller graph for this function:



10.94.3.60 set_position_latitude()

```
void util::position::Position::set_position_latitude (
            double lat)  [inline], [protected], [inherited]
```

Set the latitude of the position, in decimal degrees.

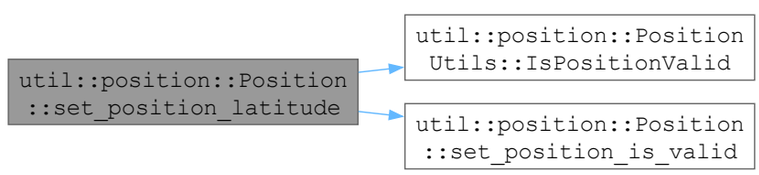
Parameters

in	*lat*	The latitude of the position, in decimal degrees.

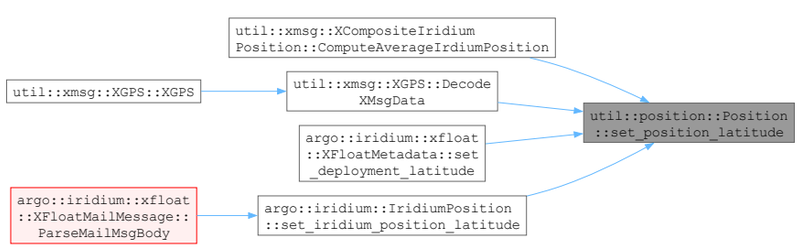
References util::position::PositionUtils::IsPositionValid(), util::position::Position::position_latitude_, util::position::Position::position_longitud and util::position::Position::set_position_is_valid().

Referenced by util::xmsg::XCompositeIridiumPosition::ComputeAverageIrdiumPosition(), DecodeXMsgData(), argo::iridium::xfloat::XFloatMetadata::set_deployment_latitude(), and argo::iridium::IridiumPosition::set_iridium_position_latitude().

Here is the call graph for this function:



Here is the caller graph for this function:



10.94.3.61 set_position_longitude()

```
void util::position::Position::set_position_longitude (
            double lon)  [inline], [protected], [inherited]
```

Set the longitude of the position, in decimal degrees.

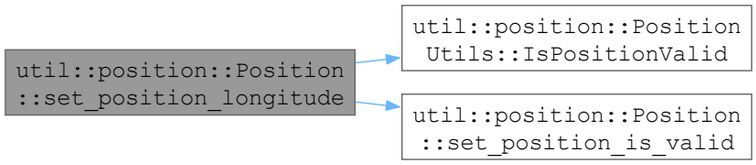
Parameters

in	*lon*	The longitude of the position, in decimal degrees.

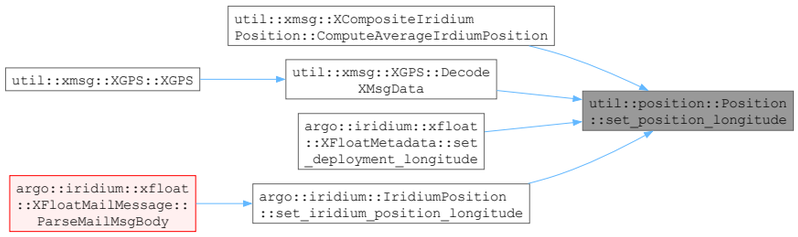
References util::position::PositionUtils::IsPositionValid(), util::position::Position::position_latitude_, util::position::Position::position_longitud and util::position::Position::set_position_is_valid().

Referenced by util::xmsg::XCompositeIridiumPosition::ComputeAverageIrdiumPosition(), DecodeXMsgData(), argo::iridium::xfloat::XFloatMetadata::set_deployment_longitude(), and argo::iridium::IridiumPosition::set_iridium_position_longitude().

Here is the call graph for this function:



Here is the caller graph for this function:



10.94.3.62 set_xmsg_size_bytes()

```
void util::xmsg::XMessage::set_xmsg_size_bytes (
            uint16_t xmsg_size_bytes)  [inline], [protected], [inherited]
```

Set the size of the X-Message, in bytes.

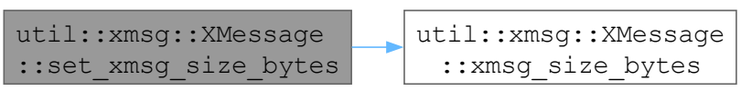
Parameters

in	*xmsg_size_bytes*	The size of the X-Message to set, in bytes.

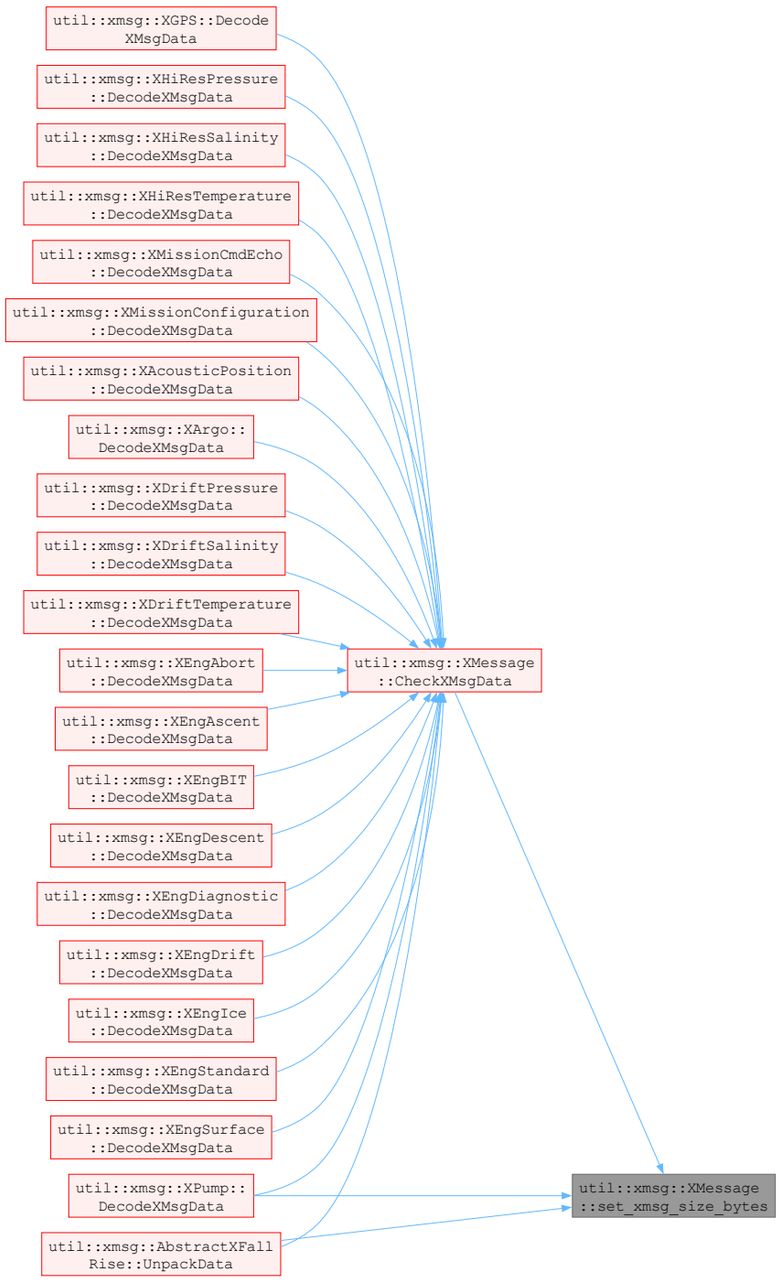
References util::xmsg::XMessage::xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XPump::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::Unpa

Here is the call graph for this function:



Here is the caller graph for this function:



10.94.3.63 SetAllDefaultKmlSyles()

```
static void util::kml::KmlUtil::SetAllDefaultKmlSyles (
            const DocumentPtr & document)  [inline], [static], [inherited]
```

Add the default styles for every palette colour to a KML document.

Parameters

in	*document*	The KML document to add the styles to.

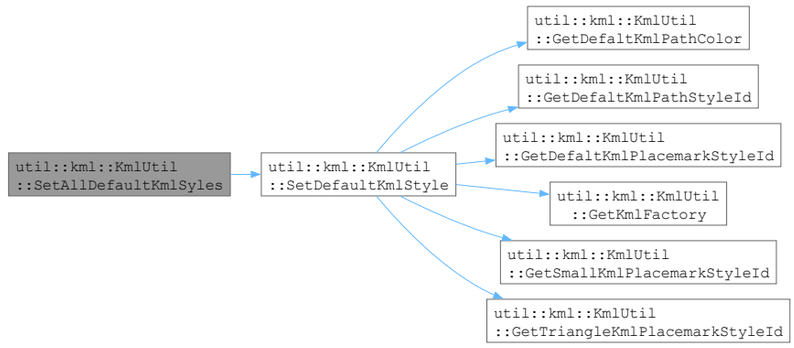
Exceptions

util::log::MessageException	if an error occurs.

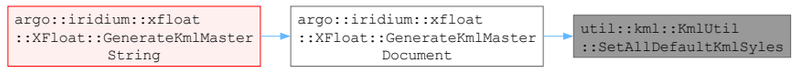
References CATCH_BLOCK, and util::kml::KmlUtil::SetDefaultKmlStyle().

Referenced by argo::iridium::xfloat::XFloat::GenerateKmlMasterDocument().

Here is the call graph for this function:



Here is the caller graph for this function:



10.94.3.64 SetDefaultKmlStyle()

```
static void util::kml::KmlUtil::SetDefaultKmlStyle (
            const DocumentPtr & document,
            unsigned int style_idx)  [inline], [static], [inherited]
```

Add the four default styles for one palette colour to a KML document.

Adds a 5-pixel path (line) style, a large dot placemark style, a small dot placemark style and a triangle placemark style, all in the colour `style_idx` (modulo the palette size). The icons are loaded from http://argo.whoi.↩edu/kml/img/. Errors are logged, not thrown.

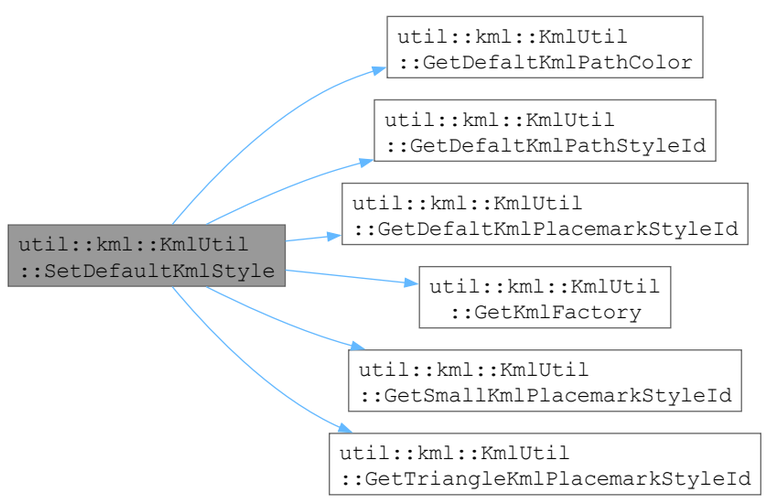
Parameters

in	*document*	The KML document to add the styles to.
in	*style_idx*	The palette index of the colour.

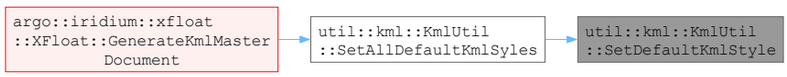
References CATCH_NOTHROW, util::kml::KmlUtil::GetDefaltKmlPathColor(), util::kml::KmlUtil::GetDefaltKmlPathStyleId(), util::kml::KmlUtil::GetDefaltKmlPlacemarkStyleId(), util::kml::KmlUtil::GetKmlFactory(), util::kml::KmlUtil::GetSmallKmlPlacemarkStyleId(), and util::kml::KmlUtil::GetTriangleKmlPlacemarkStyleId().

Referenced by util::kml::KmlUtil::SetAllDefaultKmlSyles().

Here is the call graph for this function:



Here is the caller graph for this function:



10.94.3.65 xmsg_size_bytes()

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes () const  [inline], [inherited]
```

Get the size of the X-Message, in bytes.

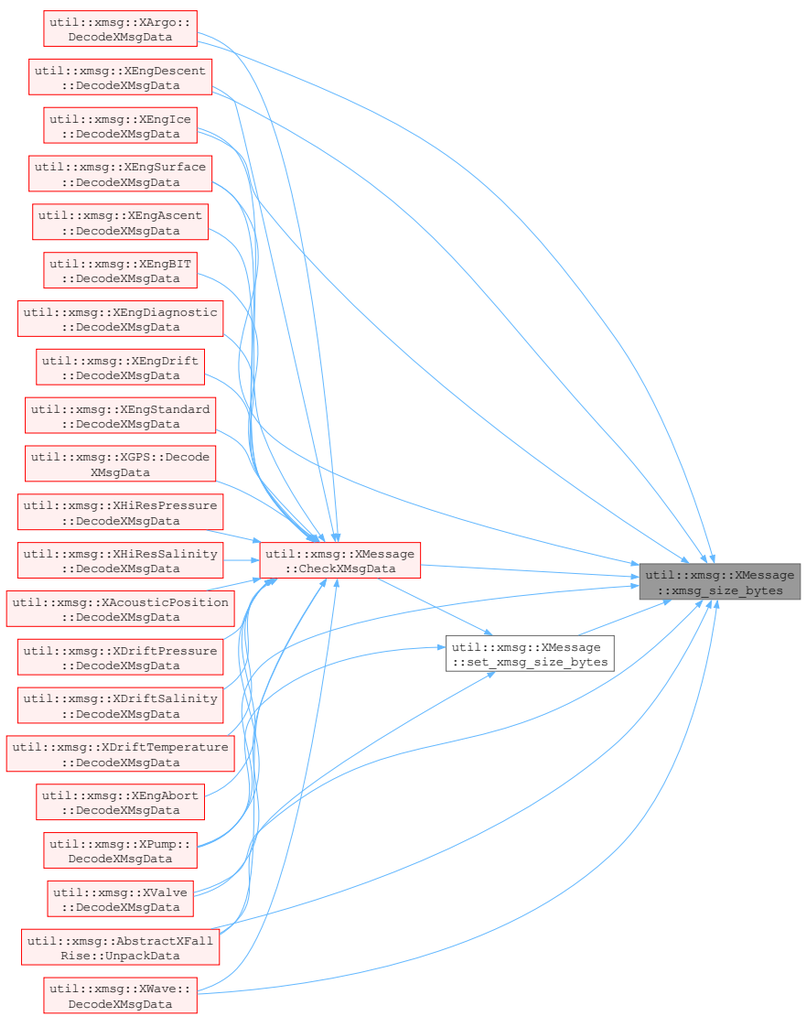
Returns

The size of the X-Message, in bytes.

References util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgI
util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(),
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), util::xmsg::XMessage::set_xmsg_size_bytes(),
and util::xmsg::AbstractXFallRise::UnpackData().

Here is the caller graph for this function:



10.94.4 Friends And Related Symbol Documentation

10.94.4.1 std::make_shared

```
shared_ptr< XGPS > std::make_shared ()  [friend]
```

Friend function for creating an X-Float object using the private constructor and a shared pointer to that object.

Referenced by SelectObjectFromDatabase().

10.94.5 Member Data Documentation

10.94.5.1 address_

```
std::string util::mysql::LocalMysqlConfiguration::address_  [static], [private], [inherited]
```

The SQL server qualified name or IP address string.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.94.5.2 EARTH_RADIUS_KM_EQUATORIAL

```
const double util::position::PositionUtils::EARTH_RADIUS_KM_EQUATORIAL = 6378.1  [static], [private],
[inherited]
```

The equatorial radius of the earth, in kilometers.

Referenced by util::position::PositionUtils::CalculateEllipsoidHeight().

10.94.5.3 EARTH_RADIUS_KM_POLAR

```
const double util::position::PositionUtils::EARTH_RADIUS_KM_POLAR = 6356.1  [static], [private],
[inherited]
```

The polar radius of the earth, in kilometers.

Referenced by util::position::PositionUtils::CalculateEllipsoidHeight().

10.94.5.4 horizontal_dilution_of_precision_

```
float util::xmsg::XGPS::horizontal_dilution_of_precision_  [private]
```

The horizontal Dilution of Precision (HDOP) of the GPS fix.

Referenced by DecodeXMsgData().

10.94.5.5 id_dive_

```
int16_t util::xmsg::XMessage::id_dive_  [private], [inherited]
```

The dive or cycle number of this message.

Referenced by util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::set_id_dive().

10.94.5.6 id_instrument_

```
uint16_t util::xmsg::XMessage::id_instrument_  [private], [inherited]
```

The unique WHOI identifier of this instrument.

Referenced by util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::set_id_instrument().

10.94.5.7 id_xmsg_type_

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type_  [private], [inherited]
```

The X-message type identifier.

Referenced by util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::set_id_xmsg_type().

10.94.5.8 kml_default_path_colors_

```
const Color32 util::kml::KmlUtil::kml_default_path_colors_  [static], [private], [inherited]
```

Initial value:

```
= {
        Color32(0xB0, 0xFF, 0x00, 0x00),
        Color32(0xB0, 0xFF, 0x80, 0x00),
        Color32(0xB0, 0xFF, 0xFF, 0x00),
        Color32(0xB0, 0x80, 0xFF, 0x00),
        Color32(0xB0, 0x00, 0xFF, 0x00),
        Color32(0xB0, 0x00, 0xFF, 0x80),
        Color32(0xB0, 0x00, 0xFF, 0xFF),
        Color32(0xB0, 0x00, 0x80, 0xFF),
        Color32(0xB0, 0x00, 0x00, 0xFF),
        Color32(0xB0, 0x80, 0x00, 0xFF),
        Color32(0xB0, 0xFF, 0x00, 0xFF),
        Color32(0xB0, 0xFF, 0x00, 0x80),
        Color32(0xB0, 0x80, 0x80, 0x080),
 }
```

The colour palette for instrument paths and placemarks, indexed by style index.

Referenced by util::kml::KmlUtil::GetDefaltKmlPathColor().

10.94.5.9 kml_default_path_style_ids_

```
const string util::kml::KmlUtil::kml_default_path_style_ids_  [static], [private], [inherited]
```

Initial value:

```
= {
        "path1",
        "path2",
        "path3",
        "path4",
        "path5",
        "path6",
        "path7",
        "path8",
        "path9",
        "pathA",
        "pathB",
        "pathC",
        "pathD",}
```

The KML style IDs of the path (line) styles, one per palette colour.

Referenced by util::kml::KmlUtil::GetDefaltKmlPathStyleId().

10.94.5.10 kml_default_placemark_style_ids_

```
const string util::kml::KmlUtil::kml_default_placemark_style_ids_  [static], [private], [inherited]
```

Initial value:

```
= {
        "dot1_1",
        "dot1_2",
        "dot1_3",
        "dot1_4",
        "dot1_5",
        "dot1_6",
        "dot1_7",
        "dot1_8",
        "dot1_9",
        "dot1_A",
        "dot1_B",
        "dot1_C",
        "dot1_D", }
```

The KML style IDs of the large placemark styles, one per palette colour.

Referenced by util::kml::KmlUtil::GetDefaltKmlPlacemarkStyleId().

10.94.5.11 kml_factory_

```
thread_specific_ptr< KmlFactory > util::kml::KmlUtil::kml_factory_  [static], [private], [inherited]
```

The libkml factory used to create KML elements, held per thread.

Warning

This holds libkml's process-wide factory (`KmlFactory::GetFactory()`), and the default cleanup deletes it when the owning thread exits or CloseKmlFactory() is called. Only the post-processing thread uses KML, so this currently happens only at shutdown.

Referenced by util::kml::KmlUtil::CloseKmlFactory(), and util::kml::KmlUtil::GetKmlFactory().

10.94.5.12 kml_small_placemark_style_ids_

```
const string util::kml::KmlUtil::kml_small_placemark_style_ids_  [static], [private], [inherited]
```

Initial value:

```
= {
        "dot1s_1",
        "dot1s_2",
        "dot1s_3",
        "dot1s_4",
        "dot1s_5",
        "dot1s_6",
        "dot1s_7",
        "dot1s_8",
        "dot1s_9",
        "dot1s_A",
        "dot1s_B",
        "dot1s_C",
        "dot1s_D", }
```

The KML style IDs of the small placemark styles, one per palette colour.

Referenced by util::kml::KmlUtil::GetSmallKmlPlacemarkStyleId().

10.94.5.13 kml_triangle_placemark_style_ids_

```
const string util::kml::KmlUtil::kml_triangle_placemark_style_ids_  [static], [private], [inherited]
```

Initial value:

```
= {
        "tri1_1",
        "tri1_2",
        "tri1_3",
        "tri1_4",
        "tri1_5",
        "tri1_6",
        "tri1_7",
        "tri1_8",
        "tri1_9",
        "tri1_A",
        "tri1_B",
        "tri1_C",
        "tri1_D", }
```

The KML style IDs of the triangle placemark styles, one per palette colour.

Referenced by util::kml::KmlUtil::GetTriangleKmlPlacemarkStyleId().

10.94.5.14 num_satellites_

```
uint8_t util::xmsg::XGPS::num_satellites_  [private]
```

The number of satellites used in GPS fix.

Referenced by DecodeXMsgData(), and GeneratePhyString().

10.94.5.15 password_

`std::string util::mysql::LocalMysqlConfiguration::password_ [static], [private], [inherited]`

The SQL server account password string.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.94.5.16 port_

`unsigned int util::mysql::LocalMysqlConfiguration::port_ [static], [private], [inherited]`

The SQL server port number.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.94.5.17 position_datetime_

`ptime util::position::Position::position_datetime_ [private], [inherited]`

The date and time of the position.

Referenced by util::position::Position::position_datetime(), util::position::Position::position_datetime_as_iso8601_string(), and util::position::Position::set_position_datetime().

10.94.5.18 position_is_valid_

`bool util::position::Position::position_is_valid_ [private], [inherited]`

The valid position indicator.

Referenced by util::position::Position::position_is_valid(), and util::position::Position::set_position_is_valid().

10.94.5.19 position_latitude_

`double util::position::Position::position_latitude_ [private], [inherited]`

The latitude of the position, in decimal degrees.

Referenced by util::position::Position::position_latitude(), util::position::Position::postition_as_iso6709_string(), util::position::Position::set_position_latitude(), and util::position::Position::set_position_longitude().

10.94.5.20 position_longitude_

`double util::position::Position::position_longitude_ [private], [inherited]`

The longitude of the position, in decimal degrees.

Referenced by util::position::Position::position_longitude(), util::position::Position::postition_as_iso6709_string(), util::position::Position::set_position_latitude(), and util::position::Position::set_position_longitude().

10.94.5.21 protocol_

`std::string util::mysql::LocalMysqlConfiguration::protocol_ [static], [private], [inherited]`

The protocol to use in the SQL server connection e.g.

"tcp".

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.94.5.22 schema_

```
std::string util::mysql::LocalMysqlConfiguration::schema_  [static], [private], [inherited]
```

The SQL server schema name.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.94.5.23 signal_to_noise_max_

```
uint8_t util::xmsg::XGPS::signal_to_noise_max_  [private]
```

The maximum Signal-to-Noise Ratio (SNR) for this GPS fix.

Referenced by DecodeXMsgData().

10.94.5.24 signal_to_noise_mean_

```
uint8_t util::xmsg::XGPS::signal_to_noise_mean_  [private]
```

The mean Signal-to-Noise Ratio (SNR) for this GPS fix.

Referenced by DecodeXMsgData().

10.94.5.25 signal_to_noise_min_

```
uint8_t util::xmsg::XGPS::signal_to_noise_min_  [private]
```

The minimum Signal-to-Noise Ratio (SNR) for this GPS fix.

Referenced by DecodeXMsgData().

10.94.5.26 sql_connection_

```
thread_specific_ptr<Connection> util::mysql::MysqlUtils::sql_connection_  [static], [private],
[inherited]
```

Tthe thread-specific pointer to MySQL connection object.

Thread-specific MySQL connection, released by `MysqlUtils::CleanupSqlConnection()` when the thread exits.

Referenced by util::mysql::MysqlUtils::CloseSqlConnection(), and util::mysql::MysqlUtils::GetSqlConnection().

10.94.5.27 sql_driver_

```
thread_specific_ptr<MySQL_Driver> util::mysql::MysqlUtils::sql_driver_  [static], [private],
[inherited]
```

The thread-specific pointer to MySQL driver object.

Thread-specific reference to the MySQL driver.

Its cleanup function, `MysqlUtils::CleanupSqlDriver()`, does not delete the shared driver instance.

Referenced by util::mysql::MysqlUtils::GetSqlDriver().

10.94.5.28 time_to_acquire_fix_

```
uint16_t util::xmsg::XGPS::time_to_acquire_fix_  [private]
```

The time to acquire this GPS fix, in seconds.

Referenced by DecodeXMsgData().

10.94.5.29 user_

```
std::string util::mysql::LocalMysqlConfiguration::user_  [static], [private], [inherited]
```

The SQL server account username string.

Referenced by util::mysql::LocalMysqlConfiguration::getSqlConnection().

10.94.5.30 xmsg_size_bytes_

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes_  [private], [inherited]
```

The size of the X message, in bytes.

Referenced by util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes().

The documentation for this class was generated from the following files:

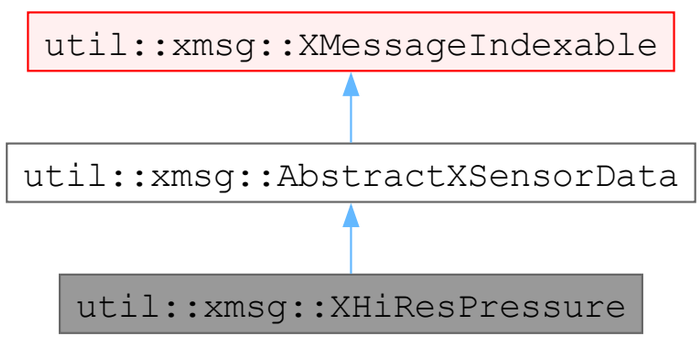
- data/xmsg/XGPS.h
- data/xmsg/XGPS.cpp

10.95 util::xmsg::XHiResPressure Class Reference

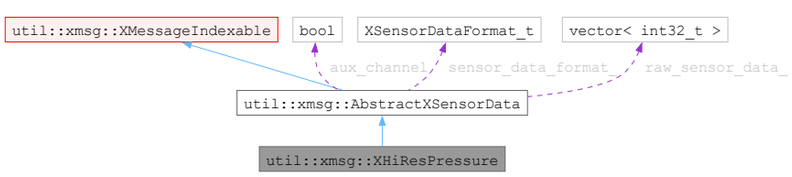
Class representing high-resolution/raw/unbinned profile pressure X-Message.

```
#include <XHiResPressure.h>
```

Inheritance diagram for util::xmsg::XHiResPressure:



Collaboration diagram for util::xmsg::XHiResPressure:



Public Member Functions

- XHiResPressure (int16_t dive, uint16_t whoi, const vector< uint8_t > &data)

  The parameterized constructor.
- ~XHiResPressure ()

  The destructor.
- const uint8_t * CheckXMsgData (const vector< uint8_t > &data, int id, int minBytes, bool parseSize=true)

  Check the X-message for the correct ID, minimum size, and message terminator.
- void DecodeXMsgData (const vector< uint8_t > &data)

  Decode the high-resolution/raw/unbinned profile pressure X-Message from the binary message data.
- int16_t id_dive () const

  Get the dive or cycle number of this X-Message.
- uint16_t id_instrument () const

  Get the unique instrument identifier of this X-Message.
- XMsgTypeId_t id_xmsg_type () const

  Get the enumerated X-Message type identifier.
- bool **is_aux_channel** () const
- size_t num_samples () const

  Get the number of samples in this message.
- const vector< int32_t > & raw_sensor_data () const

  Get the raw sensor data vector.
- int32_t Sample (size_t i) const

  Get the raw integer value of the i^{th} sample.
- template<typename T = double>
  T SampleValue (size_t i, int16_t gain, int16_t offs) const

  Get the i^{th} sensor value in scientific units.
- template<typename T = double>
  T SampleValueAsPressure (size_t i, int16_t gain, int16_t offs) const

  Get the i^{th} pressure sensor value, in dBar.
- template<typename T = double>
  T SampleValueAsSalinity (size_t i, int16_t gain, int16_t offs) const

  Get the i^{th} salinity sensor value, in PSU.
- template<typename T = double>
  T SampleValueAsTemperature (size_t i, int16_t gain, int16_t offs) const

  Get the i^{th} temperature sensor value, in degC.
- XSensorDataFormat_t sensor_data_format () const

  Get the enumerated sensor data format value.
- uint8_t xmsg_index () const

  Get the index of this X-Message.
- uint16_t xmsg_size_bytes () const

  Get the size of the X-Message, in bytes.

Protected Member Functions

- void set_id_dive (int16_t id_dive)

  Set the dive or cycle number of this message.
- void set_id_instrument (uint16_t id_instrument)

  Set the unique instrument identifier.
- void set_id_xmsg_type (XMsgTypeId_t id_xmsg_type)

  Set the X-Message type identifier.
- void set_raw_sensor_data (const vector< int32_t > &raw_sensor_data)

  Set raw integer counts of the measured sensor data.
- void set_sensor_data_format (XSensorDataFormat_t sensor_data_format)

  Set the sensor data format.
- void set_xmsg_index (uint8_t xmsg_index)

  Set the index of this X-Message.
- void set_xmsg_size_bytes (uint16_t xmsg_size_bytes)

  Set the size of the X-Message, in bytes.
- virtual vector< int32_t > UnpackSensorData (const vector< uint8_t > &data)

  Unpack the sensor data from the binary message.

Protected Attributes

- bool **aux_channel**
- vector< int32_t > raw_sensor_data_

  The raw integer counts of the measured sensor data.
- XSensorDataFormat_t sensor_data_format_

  The enumerated sensor data packing format.

Private Attributes

- int16_t id_dive_

  The dive or cycle number of this message.
- uint16_t id_instrument_

  The unique WHOI identifier of this instrument.
- XMsgTypeId_t id_xmsg_type_

  The X-message type identifier.
- uint8_t xmsg_index_

  The index of this X-Message.
- uint16_t xmsg_size_bytes_

  The size of the X message, in bytes.

10.95.1 Detailed Description

Class representing high-resolution/raw/unbinned profile pressure X-Message.

10.95.2 Constructor & Destructor Documentation

10.95.2.1 XHiResPressure()

```
util::xmsg::XHiResPressure::XHiResPressure (
            int16_t dive,
            uint16_t whoi,
            const vector< uint8_t > & data)  [inline]
```

The parameterized constructor.

Parameters

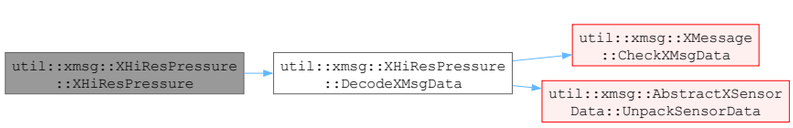
in	*dive*	The dive or cycle number of this message.
in	*whoi*	The unique WHOI identifier.
in	*data*	The binary X-message data to decode.

Exceptions

util::log::MessageException	if an decode error occurs.

References DecodeXMsgData().

Here is the call graph for this function:



10.95.2.2 ~XHiResPressure()

```
util::xmsg::XHiResPressure::~XHiResPressure ()  [inline]
```

The destructor.

10.95.3 Member Function Documentation

10.95.3.1 CheckXMsgData()

```
const uint8_t * util::xmsg::XMessage::CheckXMsgData (
            const vector< uint8_t > & data,
            int id,
            int minBytes,
            bool parseSize = true)  [inline], [inherited]
```

Check the X-message for the correct ID, minimum size, and message terminator.

Parameters

in	*data*	The X-message binary data to check.
in	*id*	The expected ID of the X-message. If this field is less that 0, then the ID check will be skipped.
in	*minBytes*	The minimum number of bytes expected in a valid message.
in	*parseSize*	If TRUE, then the message size field will be parsed from the X-Message. Otherwise, the size field will not be parsed.

Exceptions

util::log::MessageException	if • the message does not contain the expected ID, • the message does not contain the minimum number of expected bytes, • the message does not contain a terminator (';') in the correct location, or, • any other error occurs.

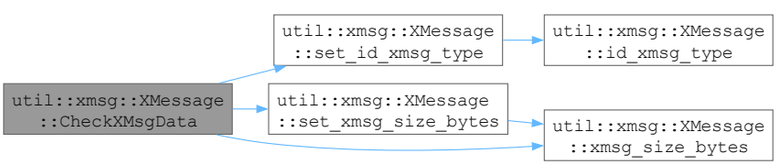
Returns

A pointer to the underlying data buffer of data.

References CATCH_BLOCK, EXC_THROW, lwrn, util::xmsg::XMessage::set_id_xmsg_type(), util::xmsg::XMessage::set_xmsg_size_byte
u16, and util::xmsg::XMessage::xmsg_size_bytes().

Referenced by util::xmsg::XAcousticPosition::DecodeXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XDriftPressure::Dec
util::xmsg::XDriftSalinity::DecodeXMsgData(), util::xmsg::XDriftTemperature::DecodeXMsgData(), util::xmsg::XEngAbort::DecodeXMsgDa
util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngBIT::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(),
util::xmsg::XEngDiagnostic::DecodeXMsgData(), util::xmsg::XEngDrift::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(),
util::xmsg::XEngStandard::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XGPS::DecodeXMsgData(),
DecodeXMsgData(), util::xmsg::XHiResSalinity::DecodeXMsgData(), util::xmsg::XHiResTemperature::DecodeXMsgData(),
util::xmsg::XMissionCmdEcho::DecodeXMsgData(), util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XProfilePressure:
util::xmsg::XProfileSalinity::DecodeXMsgData(), util::xmsg::XProfileTemperature::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgD
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the call graph for this function:



10.95.3.2 DecodeXMsgData()

```
void util::xmsg::XHiResPressure::DecodeXMsgData (
            const vector< uint8_t > & data)  [inline], [virtual]
```

Decode the high-resolution/raw/unbinned profile pressure X-Message from the binary message data.

Parameters

in	*data*	The binary message data to decode.

Exceptions

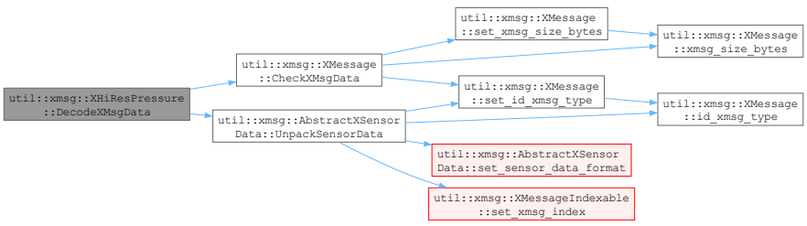
util::log::MessageException	if the message is invalid or an error occurs during decoding.

Implements util::xmsg::AbstractXSensorData.

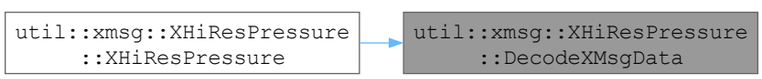
References CATCH_BLOCK, util::xmsg::XMessage::CheckXMsgData(), EXC_THROW, util::xmsg::AbstractXSensorData::UnpackSensorD
and util::xmsg::XMSG_TYPE_ID_PRES_HI_RES0.

Referenced by XHiResPressure().

Here is the call graph for this function:



Here is the caller graph for this function:



10.95.3.3 id_dive()

```
int16_t util::xmsg::XMessage::id_dive () const  [inline], [inherited]
```

Get the dive or cycle number of this X-Message.

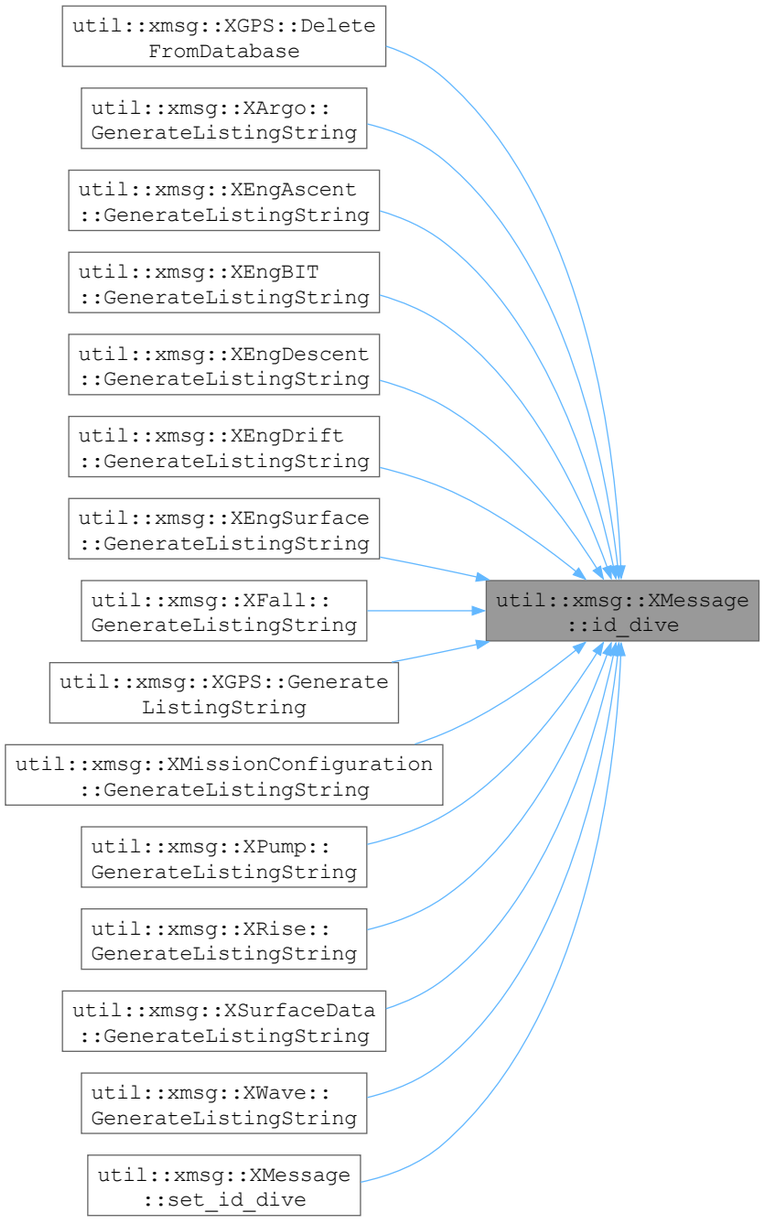
Returns

The dive or cycle number of this X-Message.

References util::xmsg::XMessage::id_dive_.

Referenced by util::xmsg::XGPS::DeleteFromDatabase(), util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateLis
util::xmsg::XEngBIT::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingStr
util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(),
util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing
util::xmsg::XSurfaceData::GenerateListingString(), util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_dive().

Here is the caller graph for this function:



10.95.3.4 id_instrument()

```
uint16_t util::xmsg::XMessage::id_instrument () const  [inline], [inherited]
```

Get the unique instrument identifier of this X-Message.

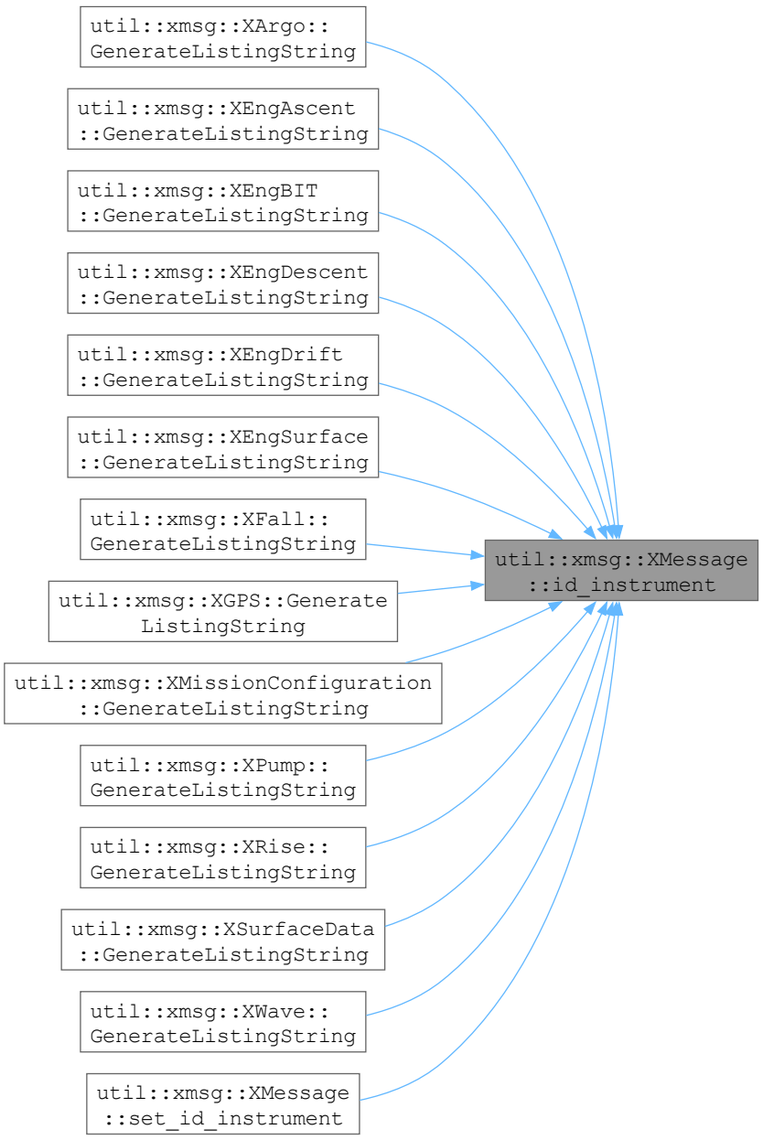
Returns

The unique instrument identifier of this X-Message.

References util::xmsg::XMessage::id_instrument_.

Referenced by util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngBIT::Generate
util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), util::xmsg::XEngSurface::GenerateListin
util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingS
util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(),
util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_instrument().

Here is the caller graph for this function:



10.95.3.5 id_xmsg_type()

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type () const  [inline], [inherited]
```

Get the enumerated X-Message type identifier.

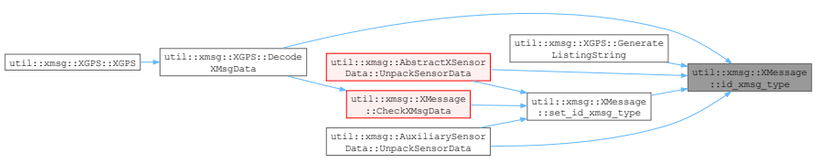
Returns

The enumerated X-Message type identifier.

References util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMessage::set_id_xmsg_typ
util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the caller graph for this function:



10.95.3.6 num_samples()

```
size_t util::xmsg::AbstractXSensorData::num_samples () const  [inline], [inherited]
```

Get the number of samples in this message.

Returns

The number of samples in this message.

References util::xmsg::AbstractXSensorData::raw_sensor_data_.

10.95.3.7 raw_sensor_data()

```
const vector< int32_t > & util::xmsg::AbstractXSensorData::raw_sensor_data () const  [inline],
[inherited]
```

Get the raw sensor data vector.

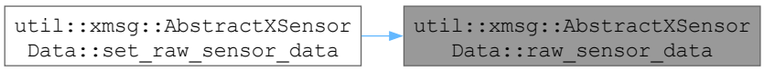
Returns

The raw sensor data vector.

References util::xmsg::AbstractXSensorData::raw_sensor_data_.

Referenced by util::xmsg::AbstractXSensorData::set_raw_sensor_data().

Here is the caller graph for this function:



10.95.3.8 Sample()

```
int32_t util::xmsg::AbstractXSensorData::Sample (
            size_t i) const  [inline], [inherited]
```

Get the raw integer value of the i^{th} sample.

Parameters

in	*i*	The index of the sensor value.

Returns

The raw integer value of the i^{th} ith sample.

References util::xmsg::AbstractXSensorData::raw_sensor_data_.

10.95.3.9 SampleValue()

```
template<typename T = double>
T util::xmsg::AbstractXSensorData::SampleValue (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline], [inherited]
```

Get the i^{th} sensor value in scientific units.

Parameters

in	*i*	The index of the sensor value.
in	*gain*	The sensor sensitivity scale factor.
in	*offs*	The sensor offset.

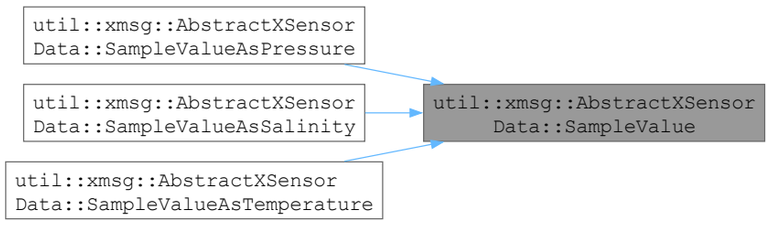
Returns

The i^{th} sensor value in scientific units.

References CATCH_BLOCK, and util::xmsg::AbstractXSensorData::raw_sensor_data_.

Referenced by util::xmsg::AbstractXSensorData::SampleValueAsPressure(), util::xmsg::AbstractXSensorData::SampleValueAsSalinity(), and util::xmsg::AbstractXSensorData::SampleValueAsTemperature().

Here is the caller graph for this function:



10.95.3.10 SampleValueAsPressure()

```
template<typename T = double>
T util::xmsg::AbstractXSensorData::SampleValueAsPressure (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline], [inherited]
```

Get the i^{th} pressure sensor value, in dBar.

Parameters

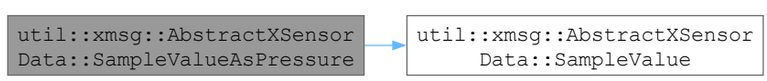
in	*i*	The index of the sensor value.
in	*gain*	The pressure sensor sensitivity scale factor, in LSB/dBar.
in	*offs*	The pressure sensor offset, in dBar.

Returns

The i^{th} pressure sensor value, in dBar.

References util::xmsg::AbstractXSensorData::SampleValue(), XMSG_SENS_BAD_PRES, and XMSG_SENS_MAX_PRES.

Here is the call graph for this function:



10.95.3.11 SampleValueAsSalinity()

```
template<typename T = double>
T util::xmsg::AbstractXSensorData::SampleValueAsSalinity (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline], [inherited]
```

Get the i^{th} salinity sensor value, in PSU.

Parameters

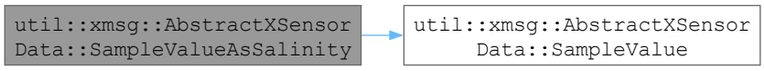
in	*i*	The index of the sensor value.
in	*gain*	The salinity sensor sensitivity scale factor, in LSB/PSU.
in	*offs*	The salinity sensor offset, in PSU.

Returns

The i^{th} salinity sensor value, in PSU.

References util::xmsg::AbstractXSensorData::SampleValue(), XMSG_SENS_BAD_SALN, and XMSG_SENS_MAX_SALN.

Here is the call graph for this function:



10.95.3.12 SampleValueAsTemperature()

```
template<typename T = double>
T util::xmsg::AbstractXSensorData::SampleValueAsTemperature (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline], [inherited]
```

Get the i^{th} temperature sensor value, in degC.

Parameters

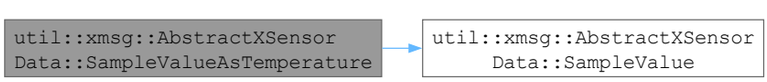
in	*i*	The index of the sensor value.
in	*gain*	The temperature sensor sensitivity scale factor, in LSB/degC.
in	*offs*	The temperature sensor offset, in degC.

Returns

The i^{th} temperature sensor value, in degC.

References util::xmsg::AbstractXSensorData::SampleValue(), XMSG_SENS_BAD_TEMP, and XMSG_SENS_MAX_TEMP.

Here is the call graph for this function:



10.95.3.13 sensor_data_format()

```
XSensorDataFormat_t util::xmsg::AbstractXSensorData::sensor_data_format () const  [inline], [inherited]
```

Get the enumerated sensor data format value.

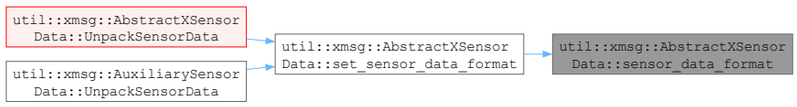
Returns

The enumerated sensor data format value.

References util::xmsg::AbstractXSensorData::sensor_data_format_.

Referenced by util::xmsg::AbstractXSensorData::set_sensor_data_format().

Here is the caller graph for this function:



10.95.3.14 set_id_dive()

```
void util::xmsg::XMessage::set_id_dive (
            int16_t id_dive)  [inline], [protected], [inherited]
```

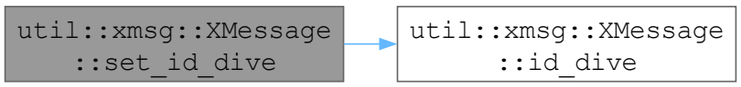
Set the dive or cycle number of this message.

Parameters

in	*id_dive*	The dive or cycle number of this message to set.

References util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::id_dive_.

Here is the call graph for this function:



10.95.3.15 set_id_instrument()

```
void util::xmsg::XMessage::set_id_instrument (
            uint16_t id_instrument)  [inline], [protected], [inherited]
```

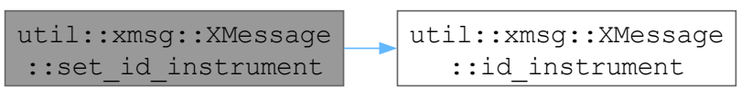
Set the unique instrument identifier.

Parameters

in	*id_instrument*	The unique instrument identifier to set.

References util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::id_instrument_.

Here is the call graph for this function:



10.95.3.16 set_id_xmsg_type()

```
void util::xmsg::XMessage::set_id_xmsg_type (
            XMsgTypeId_t id_xmsg_type)  [inline], [protected], [inherited]
```

Set the X-Message type identifier.

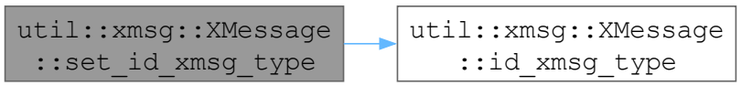
Parameters

in	*id_xmsg_type*	The X-Message type identifier to set.

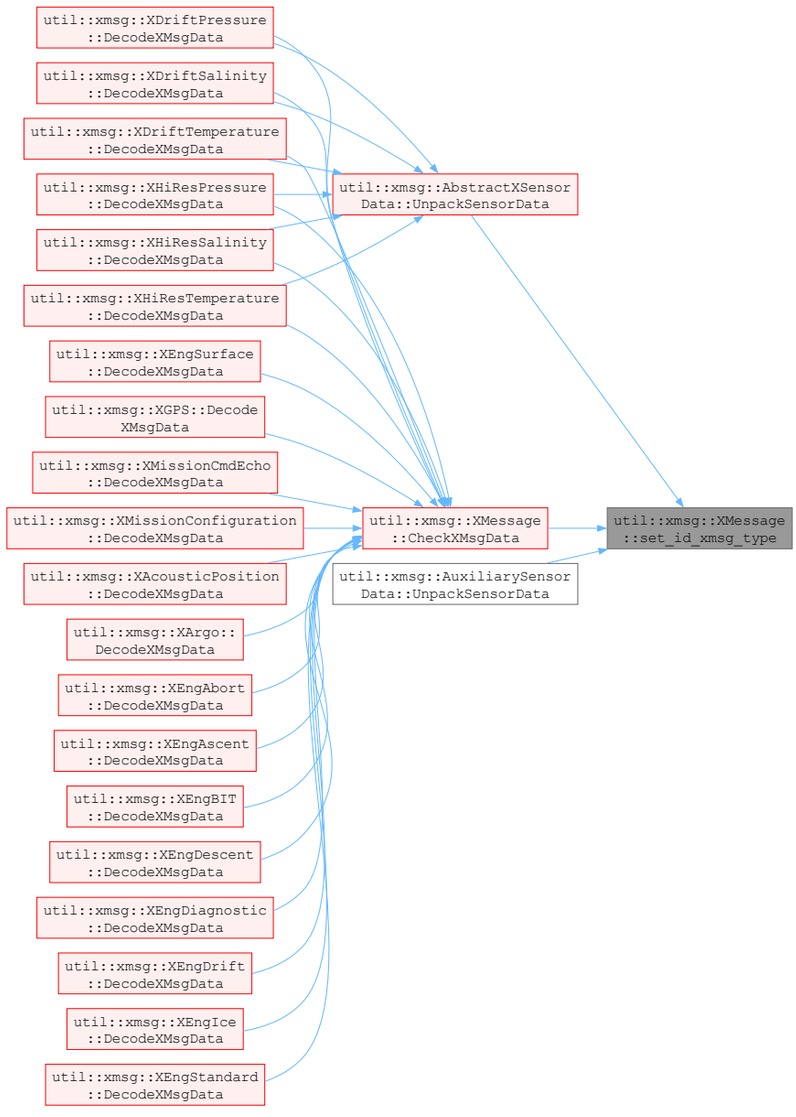
References util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.95.3.17 set_raw_sensor_data()

```
void util::xmsg::AbstractXSensorData::set_raw_sensor_data (
            const vector< int32_t > & raw_sensor_data)  [inline], [protected], [inherited]
```

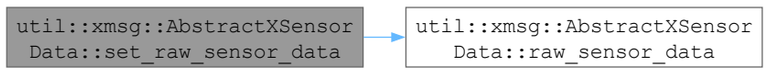
Set raw integer counts of the measured sensor data.

Parameters

in	*raw_sensor_data*	The data to set.

References util::xmsg::AbstractXSensorData::raw_sensor_data(), and util::xmsg::AbstractXSensorData::raw_sensor_data_.

Here is the call graph for this function:



10.95.3.18 set_sensor_data_format()

```
void util::xmsg::AbstractXSensorData::set_sensor_data_format (
            XSensorDataFormat_t sensor_data_format)  [inline], [protected], [inherited]
```

Set the sensor data format.

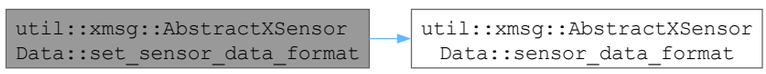
Parameters

in	*sensor_data_format*	The sensor data format to set.

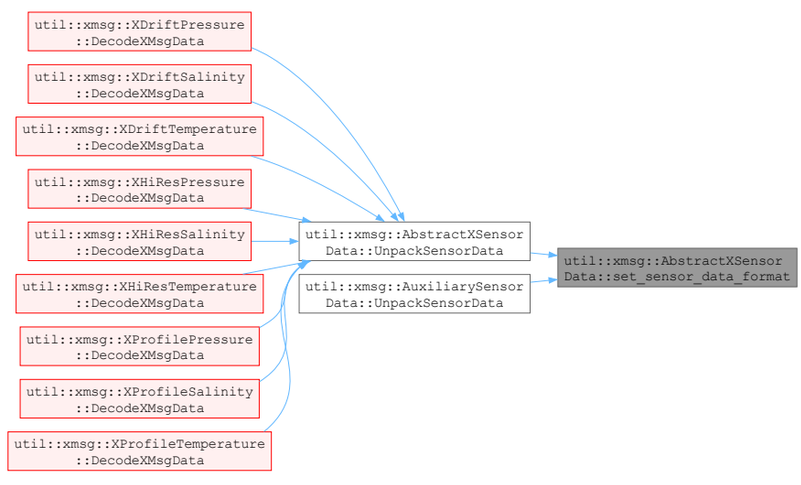
References util::xmsg::AbstractXSensorData::sensor_data_format(), and util::xmsg::AbstractXSensorData::sensor_data_format_.

Referenced by util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.95.3.19 set_xmsg_index()

```
void util::xmsg::XMessageIndexable::set_xmsg_index (
            uint8_t xmsg_index)  [inline], [protected], [inherited]
```

Set the index of this X-Message.

Parameters

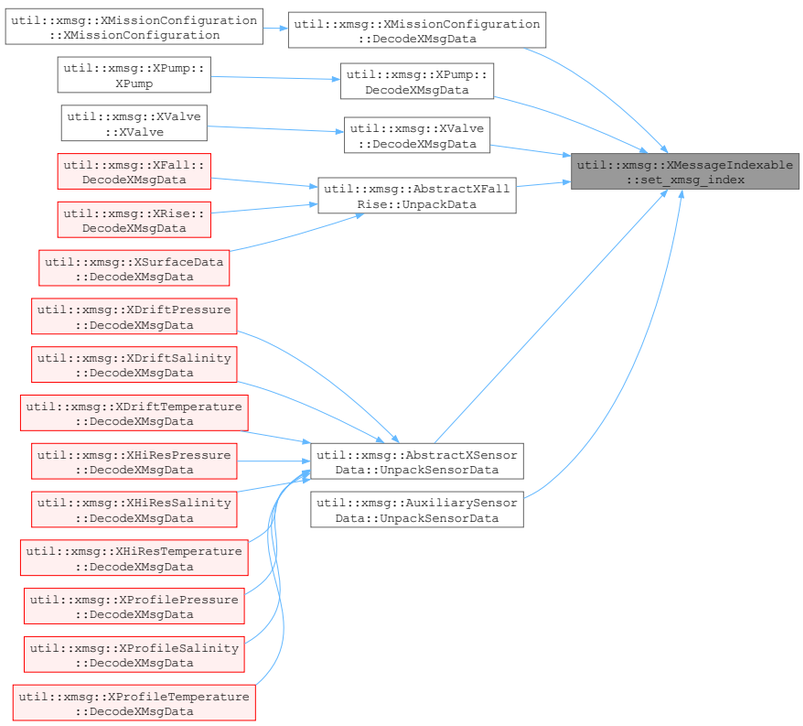
in	*xmsg_index*	The index of this X-Message.

References util::xmsg::XMessageIndexable::xmsg_index(), and util::xmsg::XMessageIndexable::xmsg_index_.

Referenced by util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(), util::xmsg::XValve::DecodeXMsgData(), util::xmsg::AbstractXFallRise::UnpackData(), util::xmsg::AbstractXSensorData::UnpackSensorDa and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:

Here is the caller graph for this function:



10.95.3.20 set_xmsg_size_bytes()

```
void util::xmsg::XMessage::set_xmsg_size_bytes (
            uint16_t xmsg_size_bytes)  [inline], [protected], [inherited]
```

Set the size of the X-Message, in bytes.

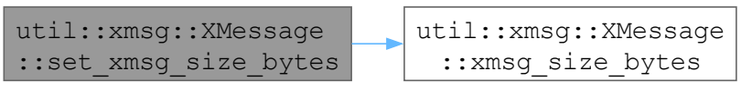
Parameters

in	*xmsg_size_bytes*	The size of the X-Message to set, in bytes.

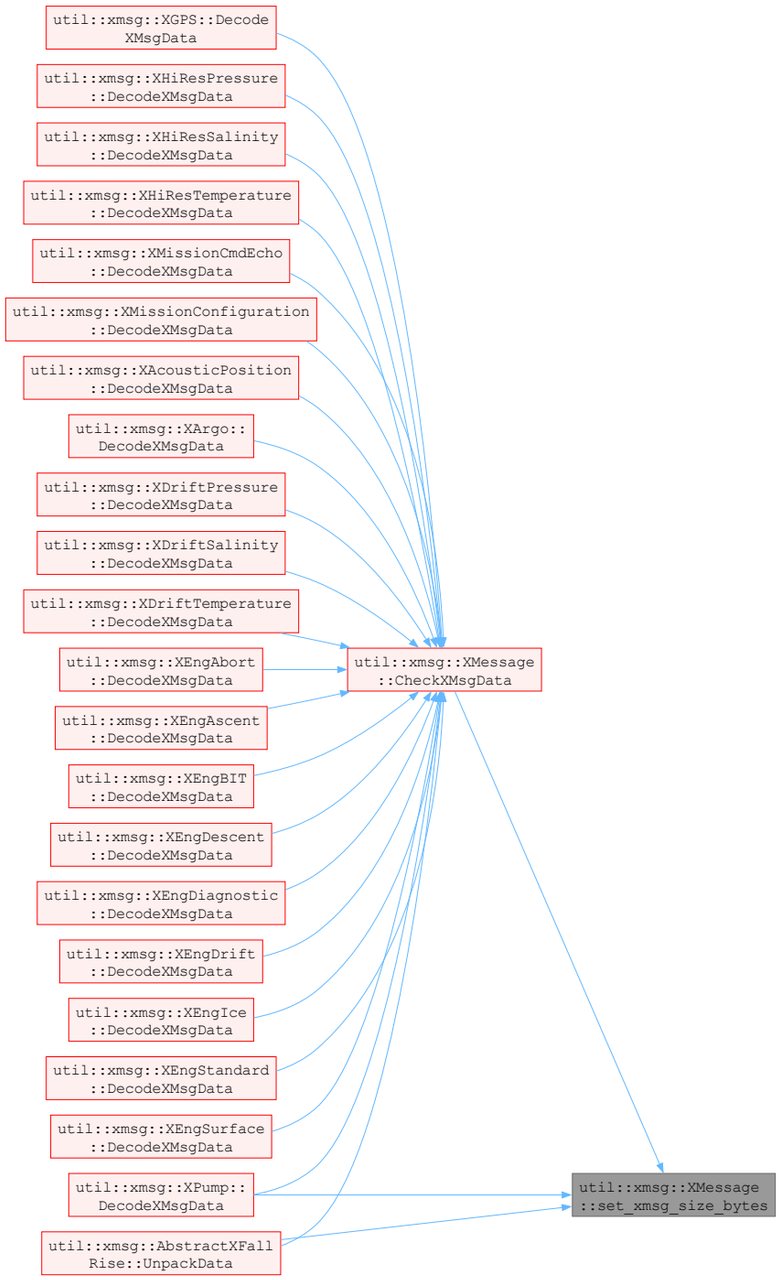
References util::xmsg::XMessage::xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XPump::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::Unpa

Here is the call graph for this function:



Here is the caller graph for this function:



10.95.3.21 UnpackSensorData()

```
virtual vector< int32_t > util::xmsg::AbstractXSensorData::UnpackSensorData (
            const vector< uint8_t > & data)  [inline], [protected], [virtual], [inherited]
```

Unpack the sensor data from the binary message.

This field unpacks the encoded sensor data from the message and stores the decoded sensor values in raw_sensor_data_. First the function checks the size of the message, and the data packing format. The message data is decoded according to the packing format type. Both first-difference and curvature encoding are supported. The encoding method is determined from the value of the high nibble of byte `data`[1].

Once this function completes successfully, then the data may be recovered in scientific units from the raw sensor values by applying the appropriate sensor gain and offset values.

Parameters

in	*data*	The binary message to unpack, in a vector of unsigned bytes.

Warning

The function assumes the message data is valid. Message validity should be ensured before this function is called.

Exceptions

util::log::MessageException	if • an error occurs while decoding the message, • the message is of insufficient length, or • an invalid packing format is encountered.

Returns

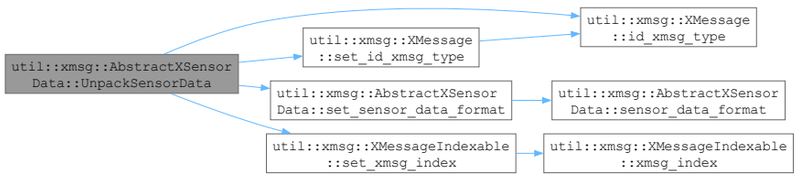
The sensor data raw counts, in a vector of signed integers.

Reimplemented in util::xmsg::AuxiliarySensorData.

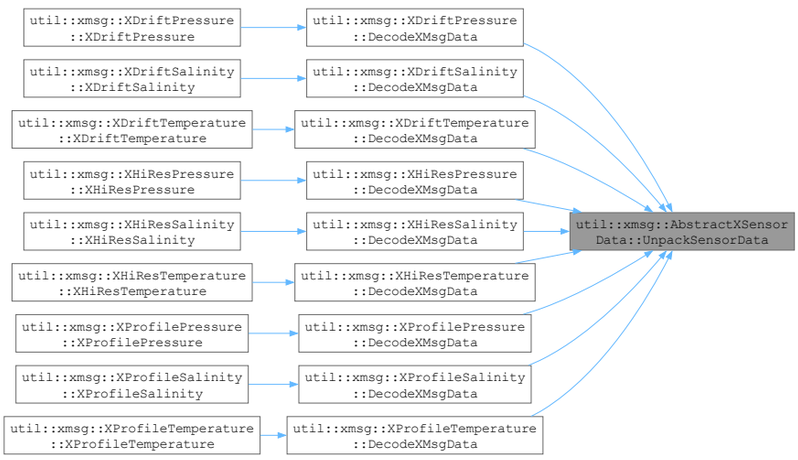
References CATCH_BLOCK, CATCH_NOTHROW, EXC_THROW, i24, util::xmsg::XMessage::id_xmsg_type(), ldbg, lwrn, util::xmsg::AbstractXSensorData::raw_sensor_data_, util::xmsg::XMessage::set_id_xmsg_type(), util::xmsg::AbstractXSensorData::s util::xmsg::XMessageIndexable::set_xmsg_index(), u16, u24, XMSG_SENS_MAX_SUB, util::xmsg::XMSG_TYPE_ID_PRES0, util::xmsg::XMSG_TYPE_ID_PRES_DRIFT8, util::xmsg::XMSG_TYPE_ID_PRES_HI_RES0, util::xmsg::XMSG_TYPE_ID_SALN0, util::xmsg::XMSG_TYPE_ID_SALN_DRIFT8, util::xmsg::XMSG_TYPE_ID_SALN_HI_RES0, util::xmsg::XMSG_TYPE_ID_TEMP0, util::xmsg::XMSG_TYPE_ID_TEMP_DRIFT8, util::xmsg::XMSG_TYPE_ID_TEMP_HI_RES0, util::xmsg::XSENS_FMT_CURVATURE, and util::xmsg::XSENS_FMT_DIFFERENCE.

Referenced by util::xmsg::XDriftPressure::DecodeXMsgData(), util::xmsg::XDriftSalinity::DecodeXMsgData(), util::xmsg::XDriftTemperatur DecodeXMsgData(), util::xmsg::XHiResSalinity::DecodeXMsgData(), util::xmsg::XHiResTemperature::DecodeXMsgData(), util::xmsg::XProfilePressure::DecodeXMsgData(), util::xmsg::XProfileSalinity::DecodeXMsgData(), and util::xmsg::XProfileTemperature::D

Here is the call graph for this function:



Here is the caller graph for this function:



10.95.3.22 xmsg_index()

```
uint8_t util::xmsg::XMessageIndexable::xmsg_index () const  [inline], [inherited]
```

Get the index of this X-Message.

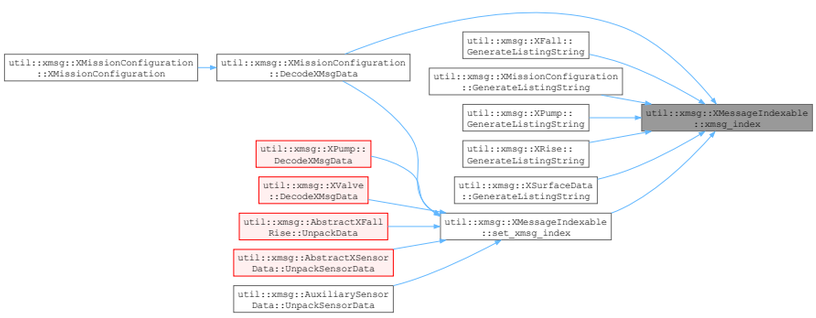
Returns

The index of this X-Message.

References util::xmsg::XMessageIndexable::xmsg_index_.

Referenced by util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing util::xmsg::XSurfaceData::GenerateListingString(), and util::xmsg::XMessageIndexable::set_xmsg_index().

Here is the caller graph for this function:



10.95.3.23 xmsg_size_bytes()

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes () const  [inline], [inherited]
```

Get the size of the X-Message, in bytes.

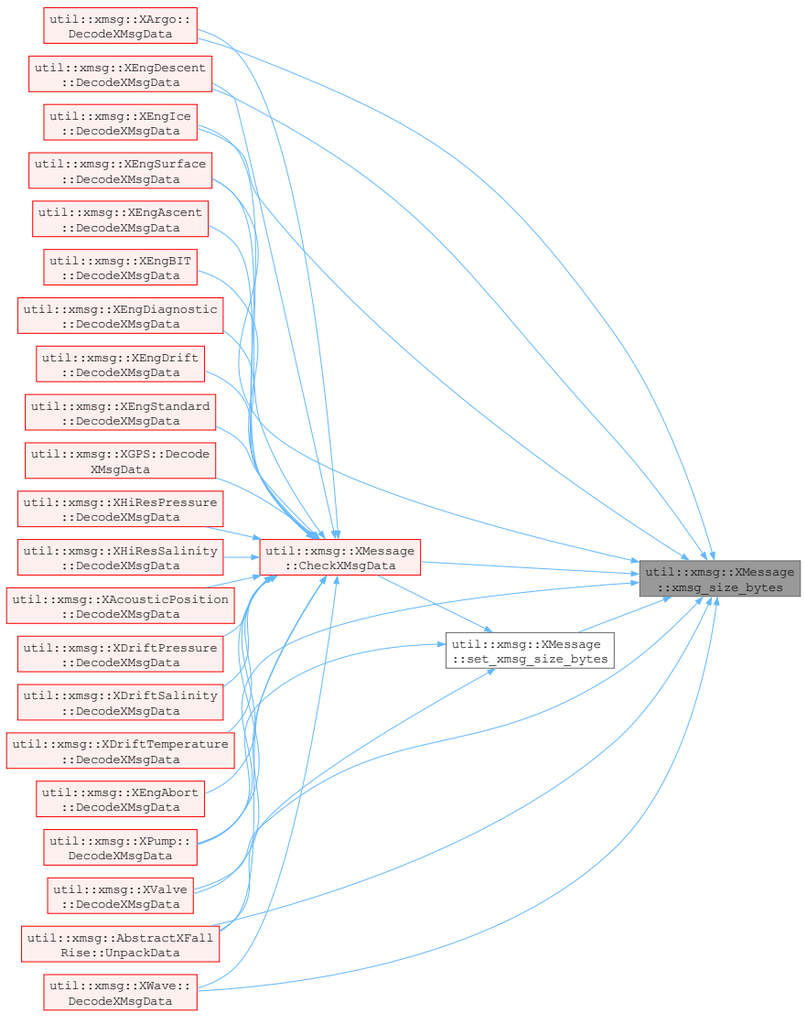
Returns

The size of the X-Message, in bytes.

References util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgI
util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(),
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), util::xmsg::XMessage::set_xmsg_size_bytes(),
and util::xmsg::AbstractXFallRise::UnpackData().

Here is the caller graph for this function:



10.95.4 Member Data Documentation

10.95.4.1 id_dive_

```
int16_t util::xmsg::XMessage::id_dive_  [private], [inherited]
```

The dive or cycle number of this message.

Referenced by util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::set_id_dive().

10.95.4.2 id_instrument_

```
uint16_t util::xmsg::XMessage::id_instrument_  [private], [inherited]
```

The unique WHOI identifier of this instrument.

Referenced by util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::set_id_instrument().

10.95.4.3 id_xmsg_type_

XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type_ [private], [inherited]

The X-message type identifier.

Referenced by util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::set_id_xmsg_type().

10.95.4.4 raw_sensor_data_

vector<int32_t> util::xmsg::AbstractXSensorData::raw_sensor_data_ [protected], [inherited]

The raw integer counts of the measured sensor data.

This vector of integers represents the "raw" sample values of the message. That is, by applying the appropriate gain and offset corrections, the sensor values may be recovered in scientific units. This data may represent pressure, temperature, salinity, or any other data that is recorded during the drift or profile phases of the dive cycle.

Referenced by util::xmsg::AbstractXSensorData::num_samples(), util::xmsg::AbstractXSensorData::raw_sensor_data(), util::xmsg::AbstractXSensorData::Sample(), util::xmsg::AbstractXSensorData::SampleValue(), util::xmsg::AbstractXSensorData::set_raw_ util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

10.95.4.5 sensor_data_format_

XSensorDataFormat_t util::xmsg::AbstractXSensorData::sensor_data_format_ [protected], [inherited]

The enumerated sensor data packing format.

This field indicates the format of encoded sensor data. The manner in which the sensor data is decoded is determined by the format of this field.

Referenced by util::xmsg::AbstractXSensorData::sensor_data_format(), and util::xmsg::AbstractXSensorData::set_sensor_data_format().

10.95.4.6 xmsg_index_

uint8_t util::xmsg::XMessageIndexable::xmsg_index_ [private], [inherited]

The index of this X-Message.

Referenced by util::xmsg::XMessageIndexable::set_xmsg_index(), and util::xmsg::XMessageIndexable::xmsg_index().

10.95.4.7 xmsg_size_bytes_

uint16_t util::xmsg::XMessage::xmsg_size_bytes_ [private], [inherited]

The size of the X message, in bytes.

Referenced by util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes().

The documentation for this class was generated from the following file:

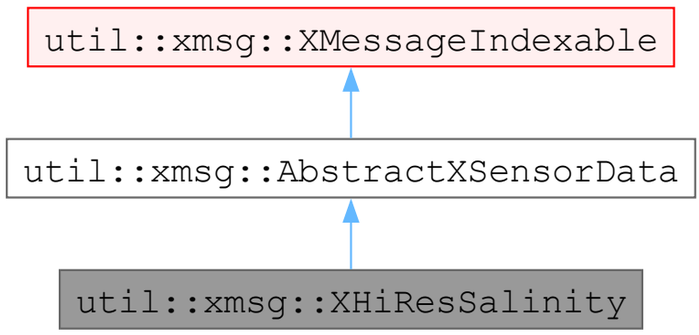
- data/xmsg/XHiResPressure.h

10.96 util::xmsg::XHiResSalinity Class Reference

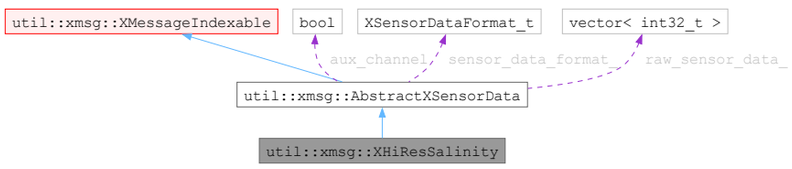
Class representing high-resolution/raw/unbinned profile salinity X-Message.

`#include <XHiResSalinity.h>`

Inheritance diagram for util::xmsg::XHiResSalinity:



Collaboration diagram for util::xmsg::XHiResSalinity:



Public Member Functions

- XHiResSalinity (int16_t dive, uint16_t whoi, const vector< uint8_t > &data)
  Parameterized constructor.
- ~XHiResSalinity ()
  Destructor.
- const uint8_t * CheckXMsgData (const vector< uint8_t > &data, int id, int minBytes, bool parseSize=true)
  Check the X-message for the correct ID, minimum size, and message terminator.
- void DecodeXMsgData (const vector< uint8_t > &data)
  Decode the high-resolution/raw/unbinned profile salinity X-Message from the binary message data.
- int16_t id_dive () const
  Get the dive or cycle number of this X-Message.
- uint16_t id_instrument () const
  Get the unique instrument identifier of this X-Message.
- XMsgTypeId_t id_xmsg_type () const
  Get the enumerated X-Message type identifier.
- bool **is_aux_channel** () const
- size_t num_samples () const
  Get the number of samples in this message.
- const vector< int32_t > & raw_sensor_data () const
  Get the raw sensor data vector.
- int32_t Sample (size_t i) const
  Get the raw integer value of the i^{th} sample.
- template<typename T = double>
  T SampleValue (size_t i, int16_t gain, int16_t offs) const
  Get the i^{th} sensor value in scientific units.
- template<typename T = double>
  T SampleValueAsPressure (size_t i, int16_t gain, int16_t offs) const

Get the i^{th} pressure sensor value, in dBar.

- template<typename T = double>
  T SampleValueAsSalinity (size_t i, int16_t gain, int16_t offs) const

  Get the i^{th} salinity sensor value, in PSU.
- template<typename T = double>
  T SampleValueAsTemperature (size_t i, int16_t gain, int16_t offs) const

  Get the i^{th} temperature sensor value, in degC.
- XSensorDataFormat_t sensor_data_format () const

  Get the enumerated sensor data format value.
- uint8_t xmsg_index () const

  Get the index of this X-Message.
- uint16_t xmsg_size_bytes () const

  Get the size of the X-Message, in bytes.

Protected Member Functions

- void set_id_dive (int16_t id_dive)

  Set the dive or cycle number of this message.
- void set_id_instrument (uint16_t id_instrument)

  Set the unique instrument identifier.
- void set_id_xmsg_type (XMsgTypeId_t id_xmsg_type)

  Set the X-Message type identifier.
- void set_raw_sensor_data (const vector< int32_t > &raw_sensor_data)

  Set raw integer counts of the measured sensor data.
- void set_sensor_data_format (XSensorDataFormat_t sensor_data_format)

  Set the sensor data format.
- void set_xmsg_index (uint8_t xmsg_index)

  Set the index of this X-Message.
- void set_xmsg_size_bytes (uint16_t xmsg_size_bytes)

  Set the size of the X-Message, in bytes.
- virtual vector< int32_t > UnpackSensorData (const vector< uint8_t > &data)

  Unpack the sensor data from the binary message.

Protected Attributes

- bool **aux_channel**
- vector< int32_t > raw_sensor_data_

  The raw integer counts of the measured sensor data.
- XSensorDataFormat_t sensor_data_format_

  The enumerated sensor data packing format.

Private Attributes

- int16_t id_dive_

  The dive or cycle number of this message.
- uint16_t id_instrument_

  The unique WHOI identifier of this instrument.
- XMsgTypeId_t id_xmsg_type_

  The X-message type identifier.
- uint8_t xmsg_index_

  The index of this X-Message.
- uint16_t xmsg_size_bytes_

  The size of the X message, in bytes.

10.96.1 Detailed Description

Class representing high-resolution/raw/unbinned profile salinity X-Message.

10.96.2 Constructor & Destructor Documentation

10.96.2.1 XHiResSalinity()

```
util::xmsg::XHiResSalinity::XHiResSalinity (
            int16_t dive,
            uint16_t whoi,
            const vector< uint8_t > & data)  [inline]
```

Parameterized constructor.

Parameters

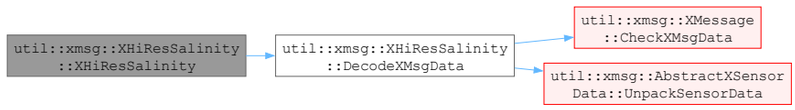
in	*dive*	The dive or cycle number of this message.
in	*whoi*	The unique WHOI identifier.
in	*data*	The binary X-message data to decode.

Exceptions

util::log::MessageException	if an decode error occurs.

References DecodeXMsgData().

Here is the call graph for this function:



10.96.2.2 ~XHiResSalinity()

```
util::xmsg::XHiResSalinity::~XHiResSalinity ()  [inline]
```

Destructor.

10.96.3 Member Function Documentation

10.96.3.1 CheckXMsgData()

```
const uint8_t * util::xmsg::XMessage::CheckXMsgData (
            const vector< uint8_t > & data,
            int id,
            int minBytes,
            bool parseSize = true)  [inline], [inherited]
```

Check the X-message for the correct ID, minimum size, and message terminator.

Parameters

in	*data*	The X-message binary data to check.
in	*id*	The expected ID of the X-message. If this field is less that 0, then the ID check will be skipped.
in	*minBytes*	The minimum number of bytes expected in a valid message.
in	*parseSize*	If TRUE, then the message size field will be parsed from the X-Message. Otherwise, the size field will not be parsed.

Exceptions

util::log::MessageException	if • the message does not contain the expected ID, • the message does not contain the minimum number of expected bytes, • the message does not contain a terminator (';') in the correct location, or, • any other error occurs.

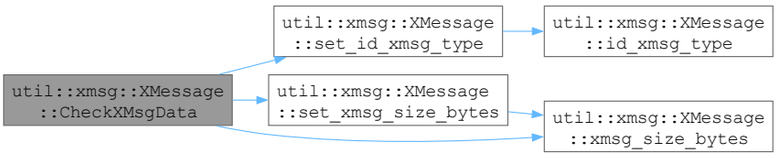
Returns

A pointer to the underlying data buffer of data.

References CATCH_BLOCK, EXC_THROW, lwrn, util::xmsg::XMessage::set_id_xmsg_type(), util::xmsg::XMessage::set_xmsg_size_byte
u16, and util::xmsg::XMessage::xmsg_size_bytes().

Referenced by util::xmsg::XAcousticPosition::DecodeXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XDriftPressure::Dec
util::xmsg::XDriftSalinity::DecodeXMsgData(), util::xmsg::XDriftTemperature::DecodeXMsgData(), util::xmsg::XEngAbort::DecodeXMsgDa
util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngBIT::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(),
util::xmsg::XEngDiagnostic::DecodeXMsgData(), util::xmsg::XEngDrift::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(),
util::xmsg::XEngStandard::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XGPS::DecodeXMsgData(),
util::xmsg::XHiResPressure::DecodeXMsgData(), DecodeXMsgData(), util::xmsg::XHiResTemperature::DecodeXMsgData(),
util::xmsg::XMissionCmdEcho::DecodeXMsgData(), util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XProfilePressure:
util::xmsg::XProfileSalinity::DecodeXMsgData(), util::xmsg::XProfileTemperature::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgD
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the call graph for this function:



10.96.3.2 DecodeXMsgData()

```
void util::xmsg::XHiResSalinity::DecodeXMsgData (
            const vector< uint8_t > & data)  [inline], [virtual]
```

Decode the high-resolution/raw/unbinned profile salinity X-Message from the binary message data.

Parameters

in	*data*	The binary message data to decode.

Exceptions

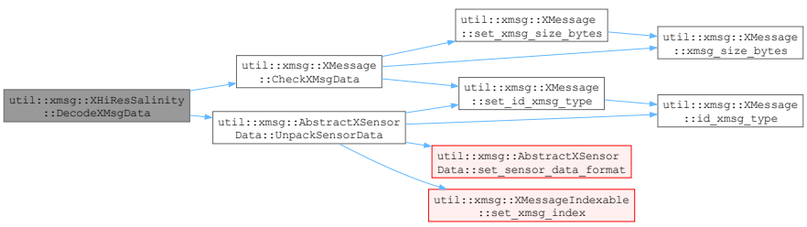
util::log::MessageException	if the message is invalid or an error occurs during decoding.

Implements util::xmsg::AbstractXSensorData.

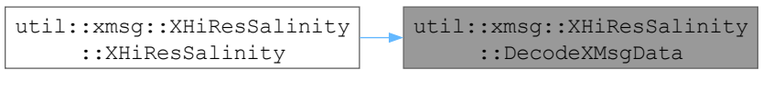
References CATCH_BLOCK, util::xmsg::XMessage::CheckXMsgData(), EXC_THROW, util::xmsg::AbstractXSensorData::UnpackSensorD
and util::xmsg::XMSG_TYPE_ID_SALN_HI_RES0.

Referenced by XHiResSalinity().

Here is the call graph for this function:



Here is the caller graph for this function:



10.96.3.3 id_dive()

```
int16_t util::xmsg::XMessage::id_dive () const  [inline], [inherited]
```

Get the dive or cycle number of this X-Message.

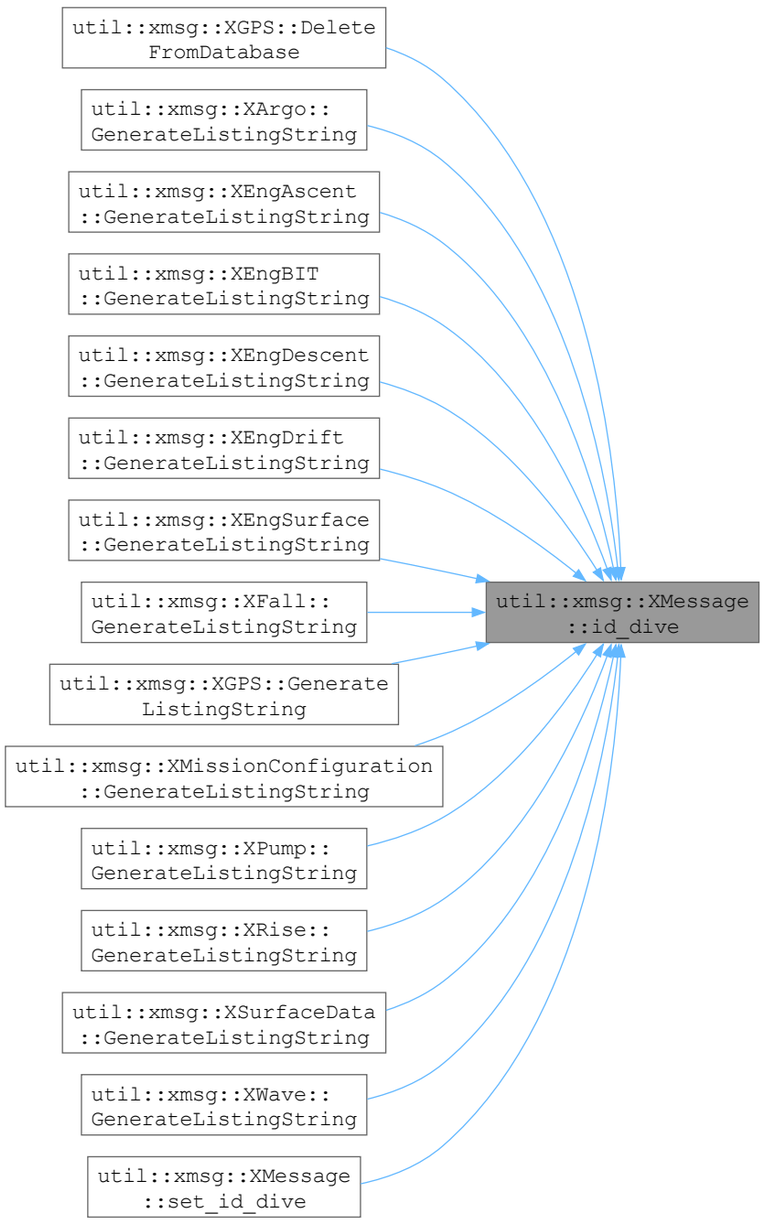
Returns

The dive or cycle number of this X-Message.

References util::xmsg::XMessage::id_dive_.

Referenced by util::xmsg::XGPS::DeleteFromDatabase(), util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateLis
util::xmsg::XEngBIT::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingStr
util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(),
util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing
util::xmsg::XSurfaceData::GenerateListingString(), util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_dive().

Here is the caller graph for this function:



10.96.3.4 id_instrument()

```
uint16_t util::xmsg::XMessage::id_instrument () const  [inline], [inherited]
```

Get the unique instrument identifier of this X-Message.

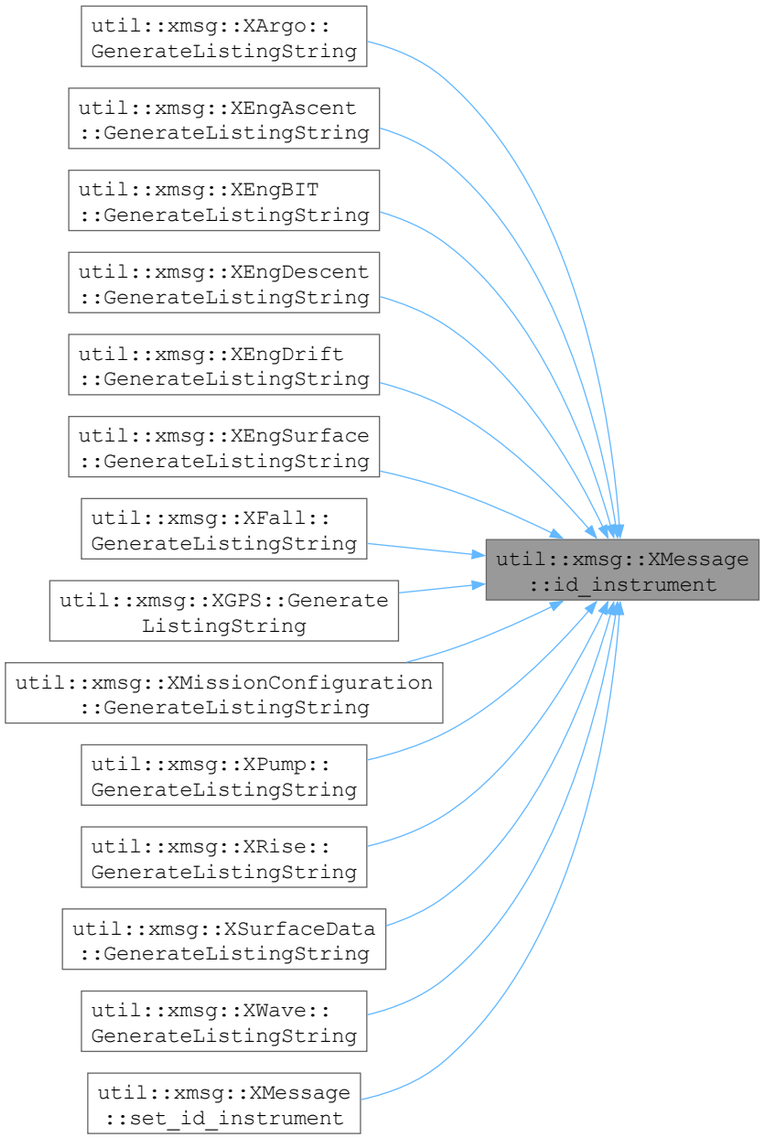
Returns

The unique instrument identifier of this X-Message.

References util::xmsg::XMessage::id_instrument_.

Referenced by util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngBIT::Generate
util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), util::xmsg::XEngSurface::GenerateListin
util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingS
util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(),
util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_instrument().

Here is the caller graph for this function:



10.96.3.5 id_xmsg_type()

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type () const  [inline], [inherited]
```

Get the enumerated X-Message type identifier.

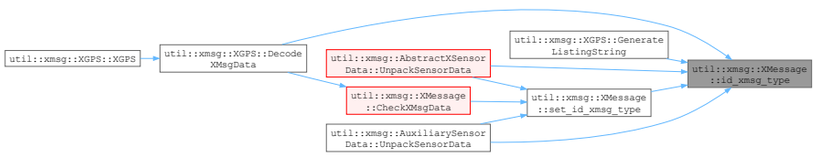
Returns

The enumerated X-Message type identifier.

References util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMessage::set_id_xmsg_typ
util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the caller graph for this function:



10.96.3.6 num_samples()

```
size_t util::xmsg::AbstractXSensorData::num_samples () const  [inline], [inherited]
```

Get the number of samples in this message.

Returns

The number of samples in this message.

References util::xmsg::AbstractXSensorData::raw_sensor_data_.

10.96.3.7 raw_sensor_data()

```
const vector< int32_t > & util::xmsg::AbstractXSensorData::raw_sensor_data () const  [inline],
[inherited]
```

Get the raw sensor data vector.

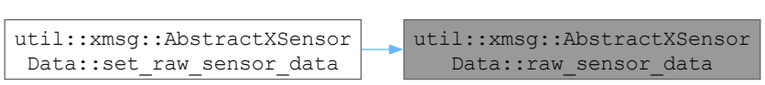
Returns

The raw sensor data vector.

References util::xmsg::AbstractXSensorData::raw_sensor_data_.

Referenced by util::xmsg::AbstractXSensorData::set_raw_sensor_data().

Here is the caller graph for this function:



10.96.3.8 Sample()

```
int32_t util::xmsg::AbstractXSensorData::Sample (
            size_t i) const  [inline], [inherited]
```

Get the raw integer value of the i^{th} sample.

Parameters

in	*i*	The index of the sensor value.

Returns

The raw integer value of the i^{th} ith sample.

References util::xmsg::AbstractXSensorData::raw_sensor_data_.

10.96.3.9 SampleValue()

```
template<typename T = double>
T util::xmsg::AbstractXSensorData::SampleValue (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline], [inherited]
```

Get the i^{th} sensor value in scientific units.

Parameters

in	*i*	The index of the sensor value.
in	*gain*	The sensor sensitivity scale factor.
in	*offs*	The sensor offset.

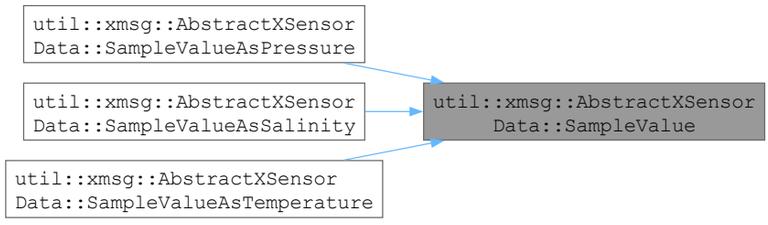
Returns

The i^{th} sensor value in scientific units.

References CATCH_BLOCK, and util::xmsg::AbstractXSensorData::raw_sensor_data_.

Referenced by util::xmsg::AbstractXSensorData::SampleValueAsPressure(), util::xmsg::AbstractXSensorData::SampleValueAsSalinity(), and util::xmsg::AbstractXSensorData::SampleValueAsTemperature().

Here is the caller graph for this function:



10.96.3.10 SampleValueAsPressure()

```
template<typename T = double>
T util::xmsg::AbstractXSensorData::SampleValueAsPressure (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline], [inherited]
```

Get the i^{th} pressure sensor value, in dBar.

Parameters

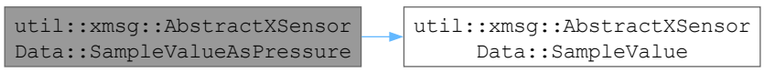
in	*i*	The index of the sensor value.
in	*gain*	The pressure sensor sensitivity scale factor, in LSB/dBar.
in	*offs*	The pressure sensor offset, in dBar.

Returns

The i^{th} pressure sensor value, in dBar.

References util::xmsg::AbstractXSensorData::SampleValue(), XMSG_SENS_BAD_PRES, and XMSG_SENS_MAX_PRES.

Here is the call graph for this function:



10.96.3.11 SampleValueAsSalinity()

```
template<typename T = double>
T util::xmsg::AbstractXSensorData::SampleValueAsSalinity (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline], [inherited]
```

Get the i^{th} salinity sensor value, in PSU.

Parameters

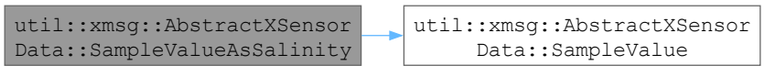
in	*i*	The index of the sensor value.
in	*gain*	The salinity sensor sensitivity scale factor, in LSB/PSU.
in	*offs*	The salinity sensor offset, in PSU.

Returns

The i^{th} salinity sensor value, in PSU.

References util::xmsg::AbstractXSensorData::SampleValue(), XMSG_SENS_BAD_SALN, and XMSG_SENS_MAX_SALN.

Here is the call graph for this function:



10.96.3.12 SampleValueAsTemperature()

```
template<typename T = double>
T util::xmsg::AbstractXSensorData::SampleValueAsTemperature (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline], [inherited]
```

Get the i^{th} temperature sensor value, in degC.

Parameters

in	*i*	The index of the sensor value.
in	*gain*	The temperature sensor sensitivity scale factor, in LSB/degC.
in	*offs*	The temperature sensor offset, in degC.

Returns

The i^{th} temperature sensor value, in degC.

References util::xmsg::AbstractXSensorData::SampleValue(), XMSG_SENS_BAD_TEMP, and XMSG_SENS_MAX_TEMP.

Here is the call graph for this function:

10.96.3.13 sensor_data_format()

```
XSensorDataFormat_t util::xmsg::AbstractXSensorData::sensor_data_format () const  [inline], [inherited]
```

Get the enumerated sensor data format value.

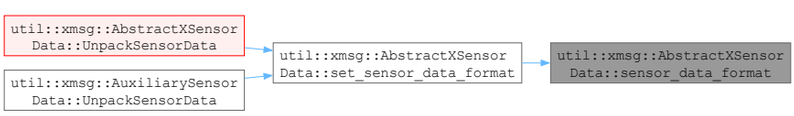
Returns

The enumerated sensor data format value.

References util::xmsg::AbstractXSensorData::sensor_data_format_.

Referenced by util::xmsg::AbstractXSensorData::set_sensor_data_format().

Here is the caller graph for this function:



10.96.3.14 set_id_dive()

```
void util::xmsg::XMessage::set_id_dive (
            int16_t id_dive)  [inline], [protected], [inherited]
```

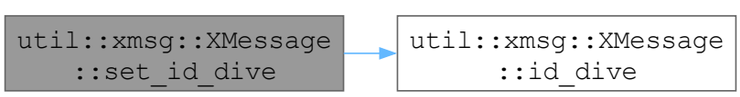
Set the dive or cycle number of this message.

Parameters

in	*id_dive*	The dive or cycle number of this message to set.

References util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::id_dive_.

Here is the call graph for this function:



10.96.3.15 set_id_instrument()

```
void util::xmsg::XMessage::set_id_instrument (
            uint16_t id_instrument)  [inline], [protected], [inherited]
```

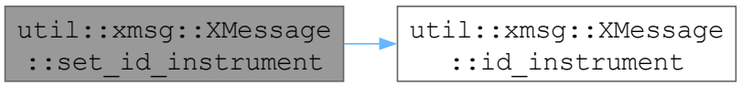
Set the unique instrument identifier.

Parameters

in	*id_instrument*	The unique instrument identifier to set.

References util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::id_instrument_.

Here is the call graph for this function:



10.96.3.16 set_id_xmsg_type()

```
void util::xmsg::XMessage::set_id_xmsg_type (
            XMsgTypeId_t id_xmsg_type)  [inline], [protected], [inherited]
```

Set the X-Message type identifier.

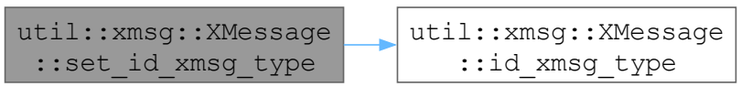
Parameters

in	*id_xmsg_type*	The X-Message type identifier to set.

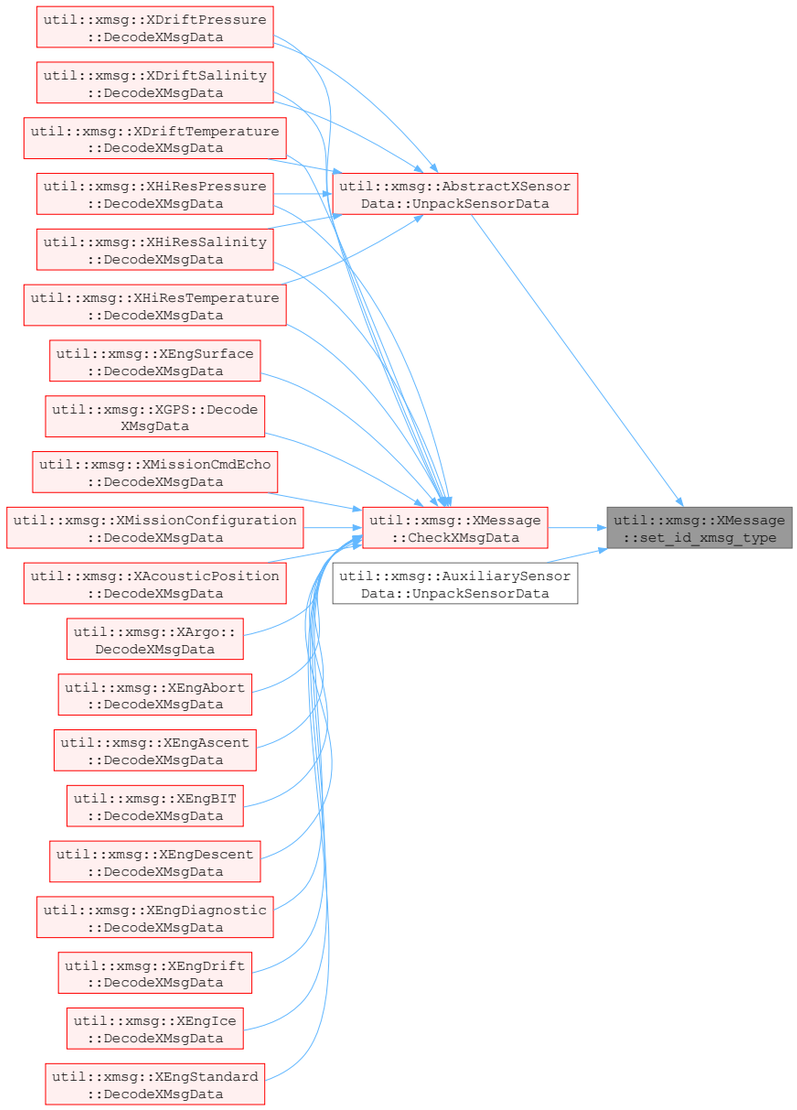
References util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.96.3.17 set_raw_sensor_data()

```
void util::xmsg::AbstractXSensorData::set_raw_sensor_data (
            const vector< int32_t > & raw_sensor_data)  [inline], [protected], [inherited]
```

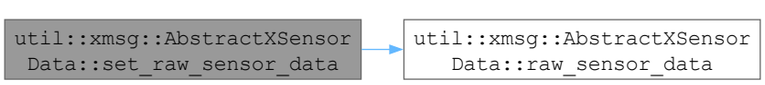
Set raw integer counts of the measured sensor data.

Parameters

in	*raw_sensor_data*	The data to set.

References util::xmsg::AbstractXSensorData::raw_sensor_data(), and util::xmsg::AbstractXSensorData::raw_sensor_data_.

Here is the call graph for this function:



10.96.3.18 set_sensor_data_format()

```
void util::xmsg::AbstractXSensorData::set_sensor_data_format (
            XSensorDataFormat_t sensor_data_format)  [inline], [protected], [inherited]
```

Set the sensor data format.

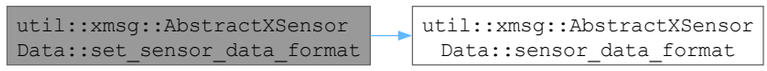
Parameters

in	*sensor_data_format*	The sensor data format to set.

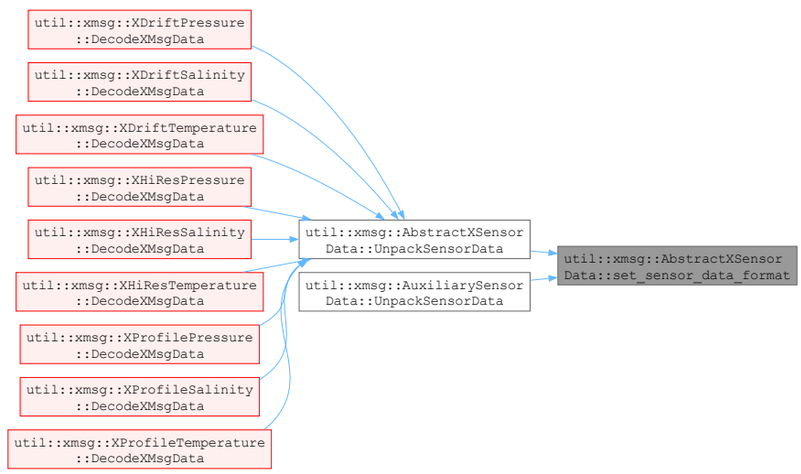
References util::xmsg::AbstractXSensorData::sensor_data_format(), and util::xmsg::AbstractXSensorData::sensor_data_format_.

Referenced by util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.96.3.19 set_xmsg_index()

```
void util::xmsg::XMessageIndexable::set_xmsg_index (
            uint8_t xmsg_index)  [inline], [protected], [inherited]
```

Set the index of this X-Message.

Parameters

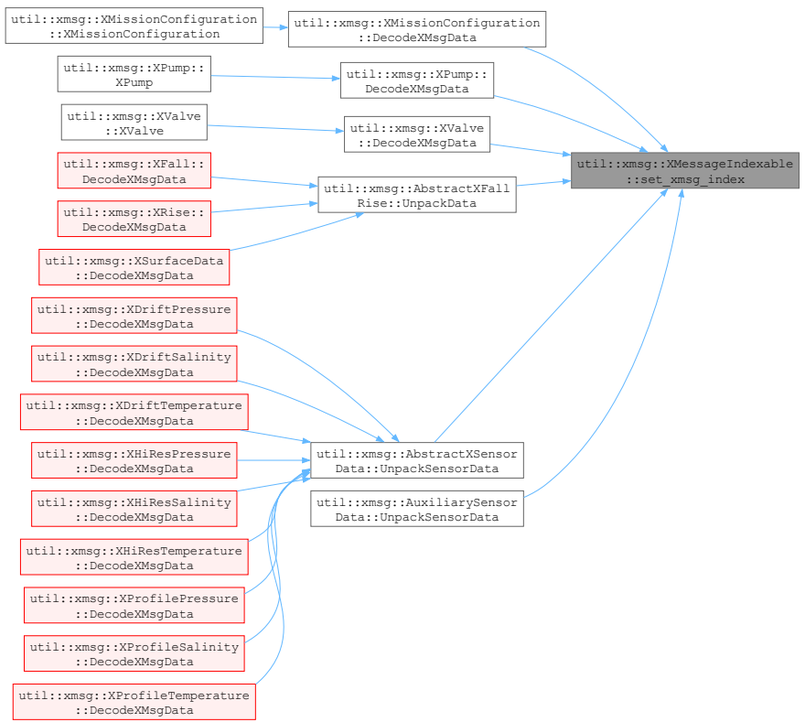
in	*xmsg_index*	The index of this X-Message.

References util::xmsg::XMessageIndexable::xmsg_index(), and util::xmsg::XMessageIndexable::xmsg_index_.

Referenced by util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(), util::xmsg::XValve::DecodeXMsgData(), util::xmsg::AbstractXFallRise::UnpackData(), util::xmsg::AbstractXSensorData::UnpackSensorDa and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:

Here is the caller graph for this function:



10.96.3.20 set_xmsg_size_bytes()

```
void util::xmsg::XMessage::set_xmsg_size_bytes (
            uint16_t xmsg_size_bytes)  [inline], [protected], [inherited]
```

Set the size of the X-Message, in bytes.

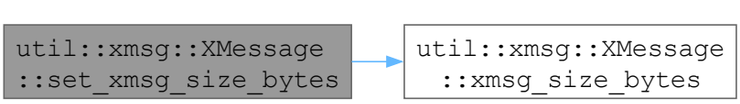
Parameters

in	*xmsg_size_bytes*	The size of the X-Message to set, in bytes.

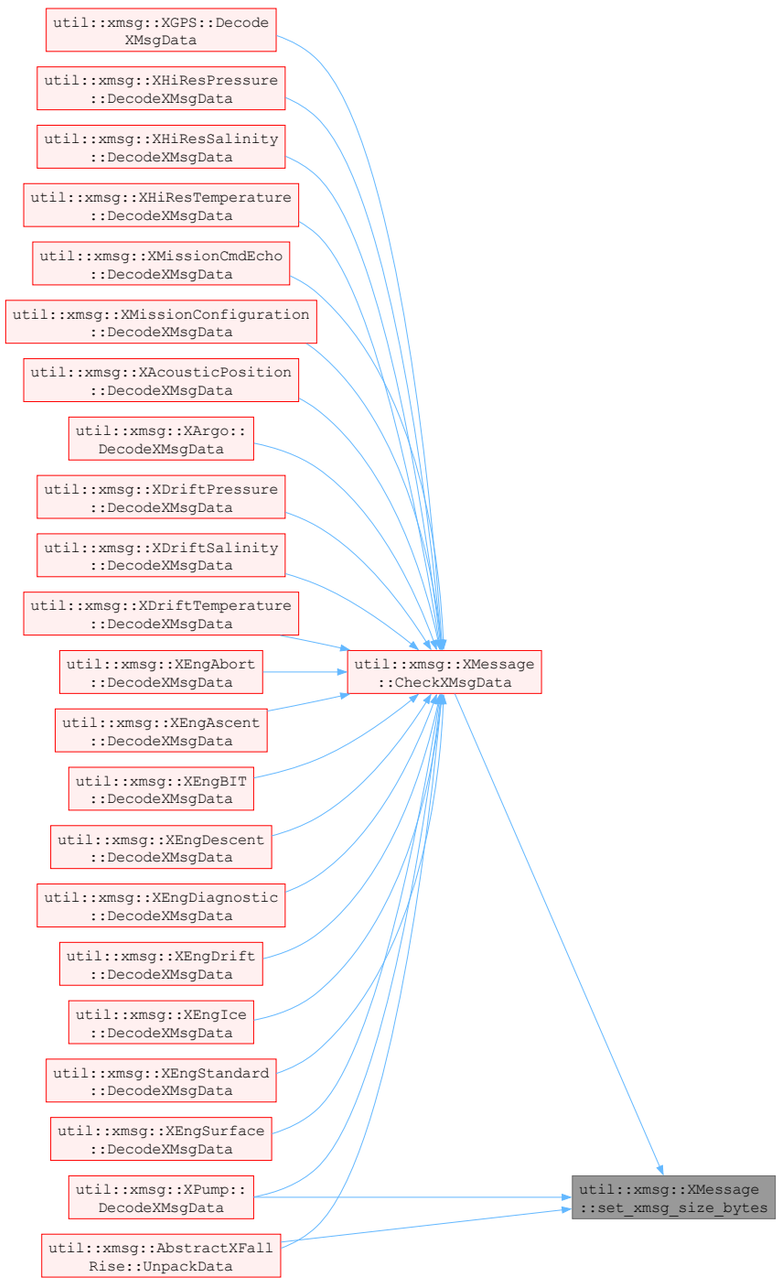
References util::xmsg::XMessage::xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XPump::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::Unpa

Here is the call graph for this function:



Here is the caller graph for this function:



10.96.3.21 UnpackSensorData()

```
virtual vector< int32_t > util::xmsg::AbstractXSensorData::UnpackSensorData (
            const vector< uint8_t > & data)  [inline], [protected], [virtual], [inherited]
```

Unpack the sensor data from the binary message.

This field unpacks the encoded sensor data from the message and stores the decoded sensor values in raw_sensor_data_. First the function checks the size of the message, and the data packing format. The message data is decoded according to the packing format type. Both first-difference and curvature encoding are supported. The encoding method is determined from the value of the high nibble of byte `data`[1].

Once this function completes successfully, then the data may be recovered in scientific units from the raw sensor values by applying the appropriate sensor gain and offset values.

Parameters

in	*data*	The binary message to unpack, in a vector of unsigned bytes.

Warning

The function assumes the message data is valid. Message validity should be ensured before this function is called.

Exceptions

util::log::MessageException	if • an error occurs while decoding the message, • the message is of insufficient length, or • an invalid packing format is encountered.

Returns

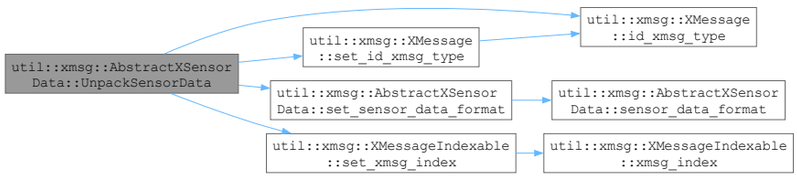
The sensor data raw counts, in a vector of signed integers.

Reimplemented in util::xmsg::AuxiliarySensorData.

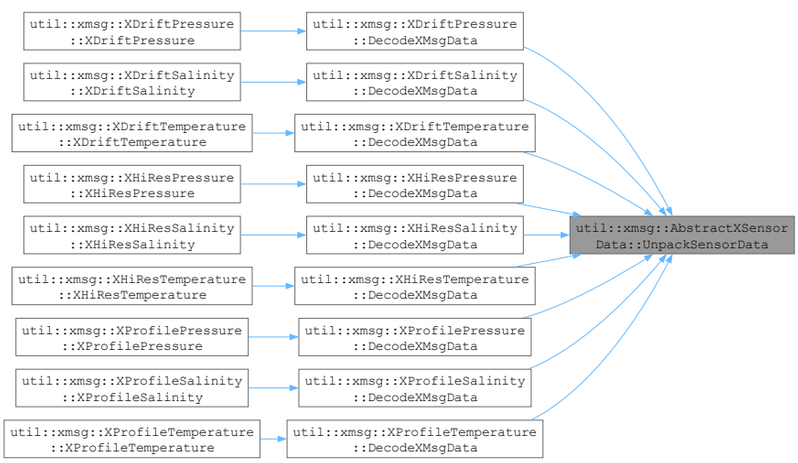
References CATCH_BLOCK, CATCH_NOTHROW, EXC_THROW, i24, util::xmsg::XMessage::id_xmsg_type(), ldbg, lwrn, util::xmsg::AbstractXSensorData::raw_sensor_data_, util::xmsg::XMessage::set_id_xmsg_type(), util::xmsg::AbstractXSensorData::s util::xmsg::XMessageIndexable::set_xmsg_index(), u16, u24, XMSG_SENS_MAX_SUB, util::xmsg::XMSG_TYPE_ID_PRES0, util::xmsg::XMSG_TYPE_ID_PRES_DRIFT8, util::xmsg::XMSG_TYPE_ID_PRES_HI_RES0, util::xmsg::XMSG_TYPE_ID_SALN0, util::xmsg::XMSG_TYPE_ID_SALN_DRIFT8, util::xmsg::XMSG_TYPE_ID_SALN_HI_RES0, util::xmsg::XMSG_TYPE_ID_TEMP0, util::xmsg::XMSG_TYPE_ID_TEMP_DRIFT8, util::xmsg::XMSG_TYPE_ID_TEMP_HI_RES0, util::xmsg::XSENS_FMT_CURVATURE, and util::xmsg::XSENS_FMT_DIFFERENCE.

Referenced by util::xmsg::XDriftPressure::DecodeXMsgData(), util::xmsg::XDriftSalinity::DecodeXMsgData(), util::xmsg::XDriftTemperatur util::xmsg::XHiResPressure::DecodeXMsgData(), DecodeXMsgData(), util::xmsg::XHiResTemperature::DecodeXMsgData(), util::xmsg::XProfilePressure::DecodeXMsgData(), util::xmsg::XProfileSalinity::DecodeXMsgData(), and util::xmsg::XProfileTemperature::D

Here is the call graph for this function:



Here is the caller graph for this function:



10.96.3.22 xmsg_index()

```
uint8_t util::xmsg::XMessageIndexable::xmsg_index () const  [inline], [inherited]
```

Get the index of this X-Message.

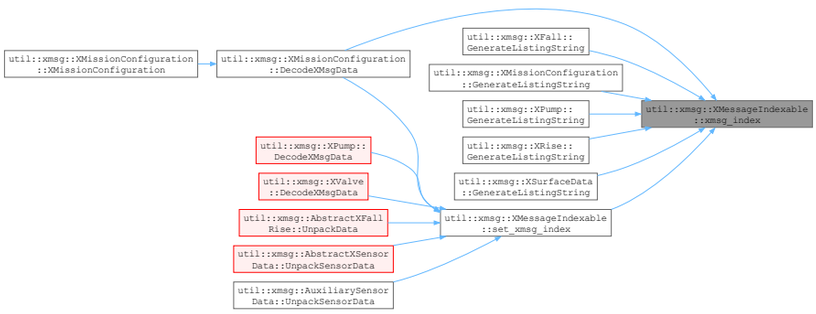
Returns

The index of this X-Message.

References util::xmsg::XMessageIndexable::xmsg_index_.

Referenced by util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing util::xmsg::XSurfaceData::GenerateListingString(), and util::xmsg::XMessageIndexable::set_xmsg_index().

Here is the caller graph for this function:



10.96.3.23 xmsg_size_bytes()

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes () const  [inline], [inherited]
```

Get the size of the X-Message, in bytes.

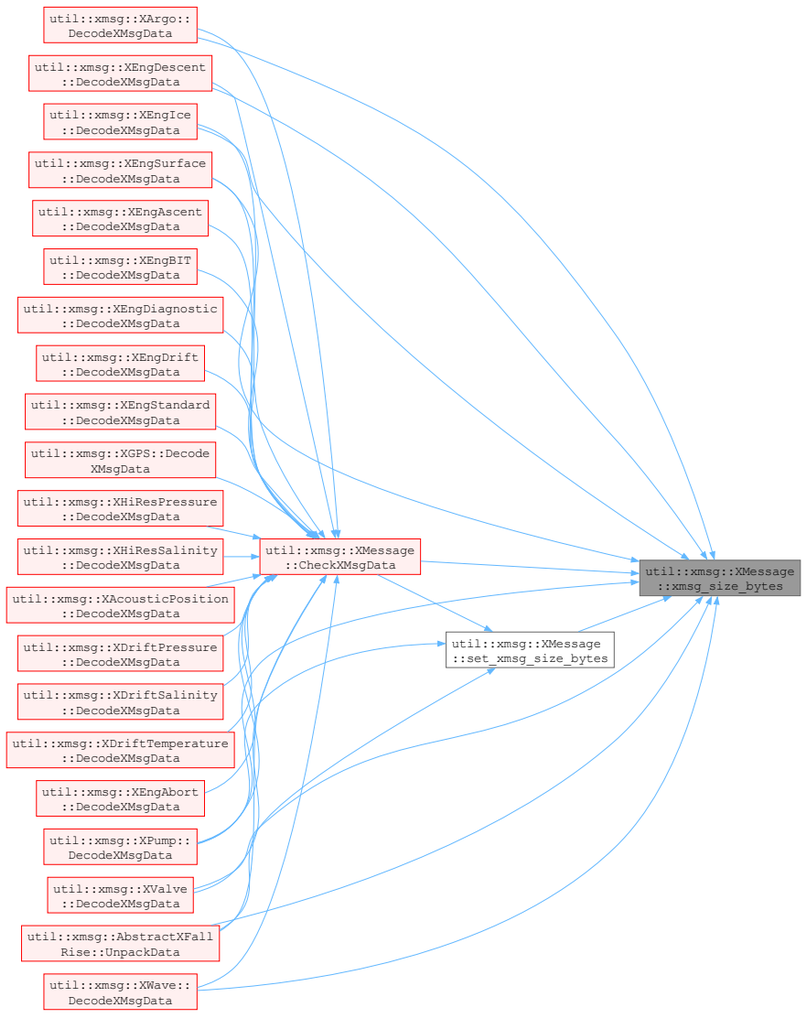
Returns

The size of the X-Message, in bytes.

References util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgI util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(), util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the caller graph for this function:



10.96.4 Member Data Documentation

10.96.4.1 id_dive_

```
int16_t util::xmsg::XMessage::id_dive_  [private], [inherited]
```

The dive or cycle number of this message.

Referenced by util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::set_id_dive().

10.96.4.2 id_instrument_

```
uint16_t util::xmsg::XMessage::id_instrument_  [private], [inherited]
```

The unique WHOI identifier of this instrument.

Referenced by util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::set_id_instrument().

10.96.4.3 id_xmsg_type_

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type_  [private], [inherited]
```

The X-message type identifier.

Referenced by util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::set_id_xmsg_type().

10.96.4.4 raw_sensor_data_

```
vector<int32_t> util::xmsg::AbstractXSensorData::raw_sensor_data_  [protected], [inherited]
```

The raw integer counts of the measured sensor data.

This vector of integers represents the "raw" sample values of the message. That is, by applying the appropriate gain and offset corrections, the sensor values may be recovered in scientific units. This data may represent pressure, temperature, salinity, or any other data that is recorded during the drift or profile phases of the dive cycle.

Referenced by util::xmsg::AbstractXSensorData::num_samples(), util::xmsg::AbstractXSensorData::raw_sensor_data(), util::xmsg::AbstractXSensorData::Sample(), util::xmsg::AbstractXSensorData::SampleValue(), util::xmsg::AbstractXSensorData::set_raw_ util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

10.96.4.5 sensor_data_format_

```
XSensorDataFormat_t util::xmsg::AbstractXSensorData::sensor_data_format_  [protected], [inherited]
```

The enumerated sensor data packing format.

This field indicates the format of encoded sensor data. The manner in which the sensor data is decoded is determined by the format of this field.

Referenced by util::xmsg::AbstractXSensorData::sensor_data_format(), and util::xmsg::AbstractXSensorData::set_sensor_data_format().

10.96.4.6 xmsg_index_

```
uint8_t util::xmsg::XMessageIndexable::xmsg_index_  [private], [inherited]
```

The index of this X-Message.

Referenced by util::xmsg::XMessageIndexable::set_xmsg_index(), and util::xmsg::XMessageIndexable::xmsg_index().

10.96.4.7 xmsg_size_bytes_

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes_  [private], [inherited]
```

The size of the X message, in bytes.

Referenced by util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes().

The documentation for this class was generated from the following file:

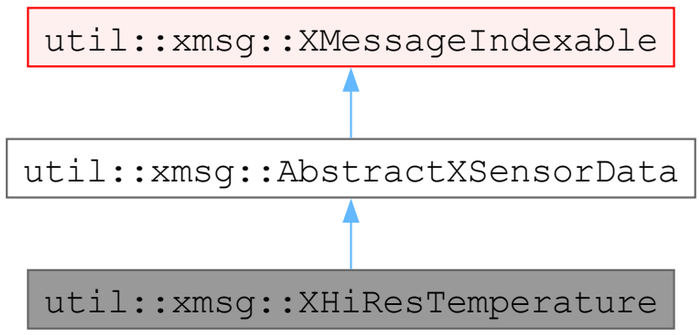
- data/xmsg/XHiResSalinity.h

10.97 util::xmsg::XHiResTemperature Class Reference

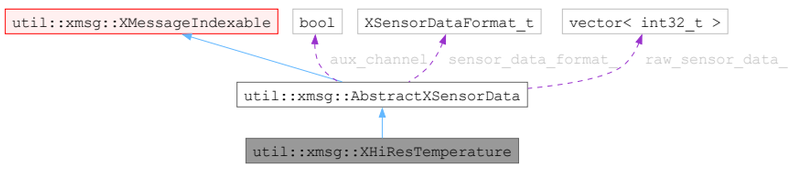
Class representing high-resolution/raw/unbinned profile temperature X-Message.

`#include <XHiResTemperature.h>`

Inheritance diagram for util::xmsg::XHiResTemperature:



Collaboration diagram for util::xmsg::XHiResTemperature:



Public Member Functions

- XHiResTemperature (int16_t dive, uint16_t whoi, const vector< uint8_t > &data)
  Parameterized constructor.
- ~XHiResTemperature ()
  Destructor.
- const uint8_t * CheckXMsgData (const vector< uint8_t > &data, int id, int minBytes, bool parseSize=true)
  Check the X-message for the correct ID, minimum size, and message terminator.
- void DecodeXMsgData (const vector< uint8_t > &data)
  Decode the high-resolution/raw/unbinned profile temperature X-Message from the binary message data.
- int16_t id_dive () const
  Get the dive or cycle number of this X-Message.
- uint16_t id_instrument () const
  Get the unique instrument identifier of this X-Message.
- XMsgTypeId_t id_xmsg_type () const
  Get the enumerated X-Message type identifier.
- bool **is_aux_channel** () const
- size_t num_samples () const
  Get the number of samples in this message.
- const vector< int32_t > & raw_sensor_data () const
  Get the raw sensor data vector.
- int32_t Sample (size_t i) const
  Get the raw integer value of the i^{th} sample.
- template<typename T = double>
  T SampleValue (size_t i, int16_t gain, int16_t offs) const
  Get the i^{th} sensor value in scientific units.
- template<typename T = double>
  T SampleValueAsPressure (size_t i, int16_t gain, int16_t offs) const

Get the i^{th} pressure sensor value, in dBar.

- template<typename T = double>
  T SampleValueAsSalinity (size_t i, int16_t gain, int16_t offs) const

  Get the i^{th} salinity sensor value, in PSU.
- template<typename T = double>
  T SampleValueAsTemperature (size_t i, int16_t gain, int16_t offs) const

  Get the i^{th} temperature sensor value, in degC.
- XSensorDataFormat_t sensor_data_format () const

  Get the enumerated sensor data format value.
- uint8_t xmsg_index () const

  Get the index of this X-Message.
- uint16_t xmsg_size_bytes () const

  Get the size of the X-Message, in bytes.

Protected Member Functions

- void set_id_dive (int16_t id_dive)

  Set the dive or cycle number of this message.
- void set_id_instrument (uint16_t id_instrument)

  Set the unique instrument identifier.
- void set_id_xmsg_type (XMsgTypeId_t id_xmsg_type)

  Set the X-Message type identifier.
- void set_raw_sensor_data (const vector< int32_t > &raw_sensor_data)

  Set raw integer counts of the measured sensor data.
- void set_sensor_data_format (XSensorDataFormat_t sensor_data_format)

  Set the sensor data format.
- void set_xmsg_index (uint8_t xmsg_index)

  Set the index of this X-Message.
- void set_xmsg_size_bytes (uint16_t xmsg_size_bytes)

  Set the size of the X-Message, in bytes.
- virtual vector< int32_t > UnpackSensorData (const vector< uint8_t > &data)

  Unpack the sensor data from the binary message.

Protected Attributes

- bool **aux_channel**
- vector< int32_t > raw_sensor_data_

  The raw integer counts of the measured sensor data.
- XSensorDataFormat_t sensor_data_format_

  The enumerated sensor data packing format.

Private Attributes

- int16_t id_dive_

  The dive or cycle number of this message.
- uint16_t id_instrument_

  The unique WHOI identifier of this instrument.
- XMsgTypeId_t id_xmsg_type_

  The X-message type identifier.
- uint8_t xmsg_index_

  The index of this X-Message.
- uint16_t xmsg_size_bytes_

  The size of the X message, in bytes.

10.97.1 Detailed Description

Class representing high-resolution/raw/unbinned profile temperature X-Message.

10.97.2 Constructor & Destructor Documentation

10.97.2.1 XHiResTemperature()

```
util::xmsg::XHiResTemperature::XHiResTemperature (
            int16_t dive,
            uint16_t whoi,
            const vector< uint8_t > & data)  [inline]
```

Parameterized constructor.

Parameters

in	*dive*	The dive or cycle number of this message.
in	*whoi*	The unique WHOI identifier.
in	*data*	The binary X-message data to decode.

Exceptions

util::log::MessageException	if an decode error occurs.

References DecodeXMsgData().

Here is the call graph for this function:

10.97.2.2 ~XHiResTemperature()

```
util::xmsg::XHiResTemperature::~XHiResTemperature ()  [inline]
```

Destructor.

10.97.3 Member Function Documentation

10.97.3.1 CheckXMsgData()

```
const uint8_t * util::xmsg::XMessage::CheckXMsgData (
            const vector< uint8_t > & data,
            int id,
            int minBytes,
            bool parseSize = true)  [inline], [inherited]
```

Check the X-message for the correct ID, minimum size, and message terminator.

Parameters

in	*data*	The X-message binary data to check.
in	*id*	The expected ID of the X-message. If this field is less that 0, then the ID check will be skipped.
in	*minBytes*	The minimum number of bytes expected in a valid message.
in	*parseSize*	If TRUE, then the message size field will be parsed from the X-Message. Otherwise, the size field will not be parsed.

Exceptions

util::log::MessageException	if • the message does not contain the expected ID, • the message does not contain the minimum number of expected bytes, • the message does not contain a terminator (';') in the correct location, or, • any other error occurs.

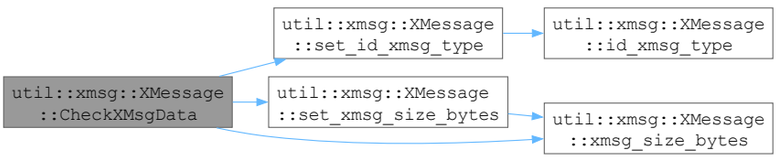
Returns

A pointer to the underlying data buffer of data.

References CATCH_BLOCK, EXC_THROW, lwrn, util::xmsg::XMessage::set_id_xmsg_type(), util::xmsg::XMessage::set_xmsg_size_byte
u16, and util::xmsg::XMessage::xmsg_size_bytes().

Referenced by util::xmsg::XAcousticPosition::DecodeXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XDriftPressure::Dec
util::xmsg::XDriftSalinity::DecodeXMsgData(), util::xmsg::XDriftTemperature::DecodeXMsgData(), util::xmsg::XEngAbort::DecodeXMsgDa
util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngBIT::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(),
util::xmsg::XEngDiagnostic::DecodeXMsgData(), util::xmsg::XEngDrift::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(),
util::xmsg::XEngStandard::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XGPS::DecodeXMsgData(),
util::xmsg::XHiResPressure::DecodeXMsgData(), util::xmsg::XHiResSalinity::DecodeXMsgData(), DecodeXMsgData(),
util::xmsg::XMissionCmdEcho::DecodeXMsgData(), util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XProfilePressure:
util::xmsg::XProfileSalinity::DecodeXMsgData(), util::xmsg::XProfileTemperature::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgD
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the call graph for this function:



10.97.3.2 DecodeXMsgData()

```
void util::xmsg::XHiResTemperature::DecodeXMsgData (
            const vector< uint8_t > & data)  [inline], [virtual]
```

Decode the high-resolution/raw/unbinned profile temperature X-Message from the binary message data.

Parameters

in	*data*	The binary message data to decode.

Exceptions

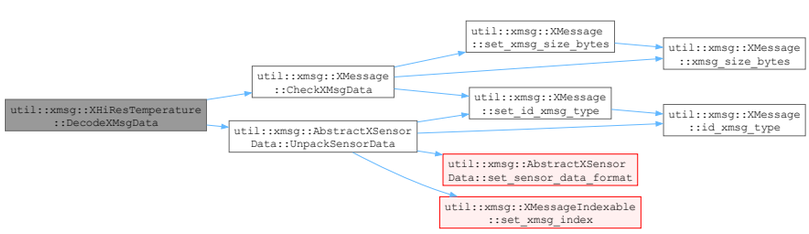
util::log::MessageException	if the message is invalid or an error occurs during decoding.

Implements util::xmsg::AbstractXSensorData.

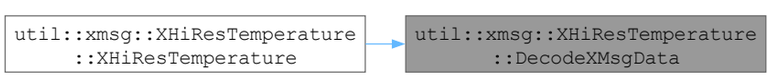
References CATCH_BLOCK, util::xmsg::XMessage::CheckXMsgData(), EXC_THROW, util::xmsg::AbstractXSensorData::UnpackSensorD
and util::xmsg::XMSG_TYPE_ID_TEMP_HI_RES0.

Referenced by XHiResTemperature().

Here is the call graph for this function:



Here is the caller graph for this function:



10.97.3.3 id_dive()

```
int16_t util::xmsg::XMessage::id_dive () const  [inline], [inherited]
```

Get the dive or cycle number of this X-Message.

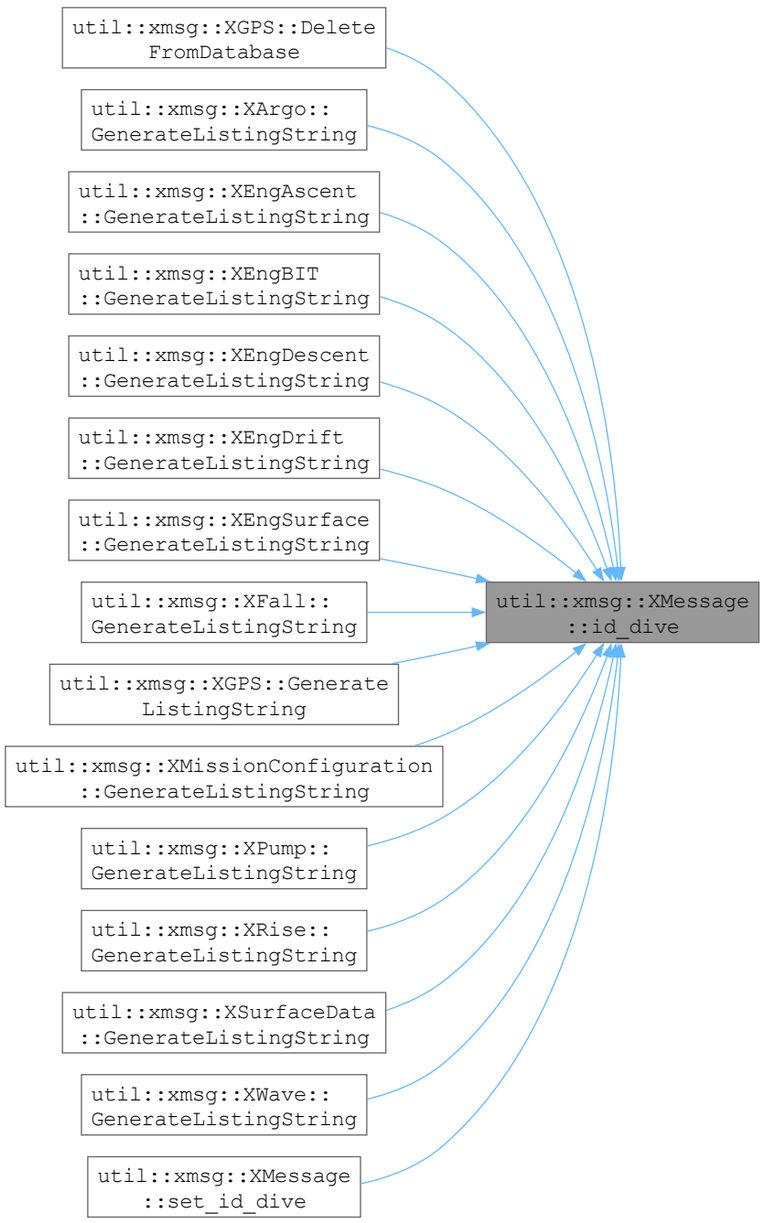
Returns

The dive or cycle number of this X-Message.

References util::xmsg::XMessage::id_dive_.

Referenced by util::xmsg::XGPS::DeleteFromDatabase(), util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateLis
util::xmsg::XEngBIT::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingStr
util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(),
util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing
util::xmsg::XSurfaceData::GenerateListingString(), util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_dive().

Here is the caller graph for this function:



10.97.3.4 id_instrument()

```
uint16_t util::xmsg::XMessage::id_instrument () const  [inline], [inherited]
```

Get the unique instrument identifier of this X-Message.

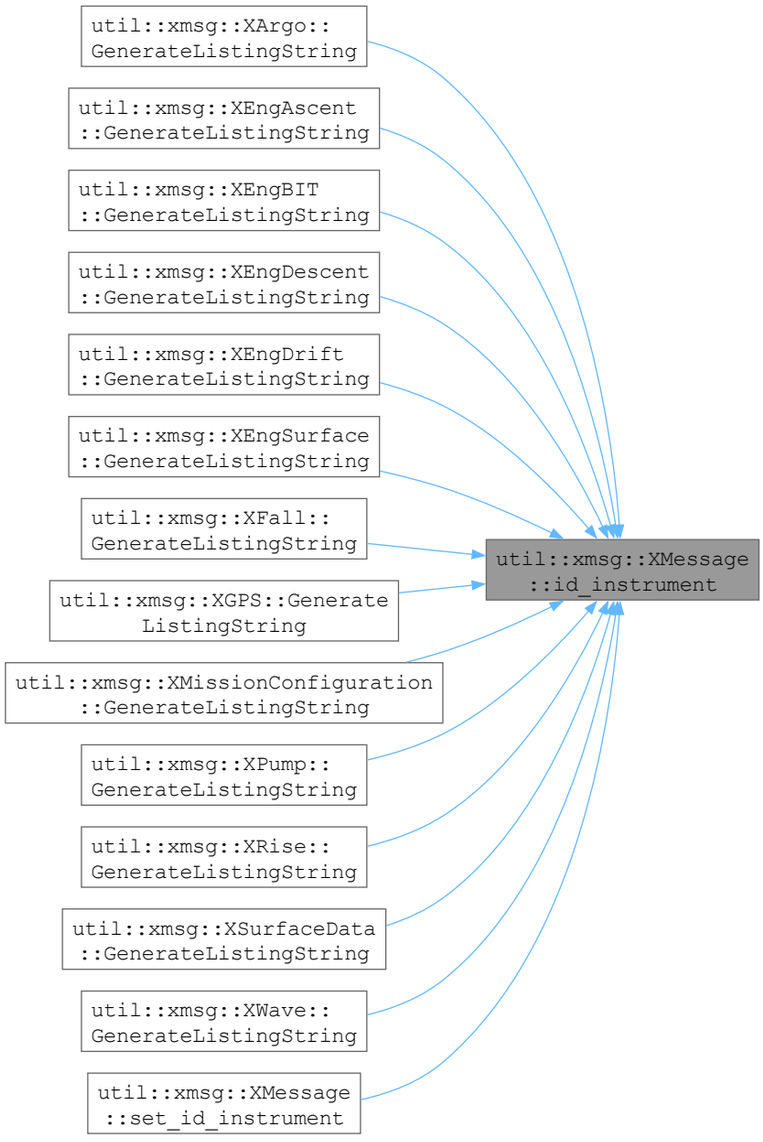
Returns

The unique instrument identifier of this X-Message.

References util::xmsg::XMessage::id_instrument_.

Referenced by util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngBIT::Generate
util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), util::xmsg::XEngSurface::GenerateListin
util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingS
util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(),
util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_instrument().

Here is the caller graph for this function:



10.97.3.5 id_xmsg_type()

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type () const  [inline], [inherited]
```

Get the enumerated X-Message type identifier.

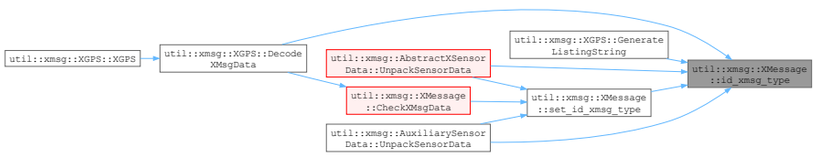
Returns

The enumerated X-Message type identifier.

References util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMessage::set_id_xmsg_typ
util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the caller graph for this function:



10.97.3.6 num_samples()

```
size_t util::xmsg::AbstractXSensorData::num_samples () const  [inline], [inherited]
```

Get the number of samples in this message.

Returns

The number of samples in this message.

References util::xmsg::AbstractXSensorData::raw_sensor_data_.

10.97.3.7 raw_sensor_data()

```
const vector< int32_t > & util::xmsg::AbstractXSensorData::raw_sensor_data () const  [inline],
[inherited]
```

Get the raw sensor data vector.

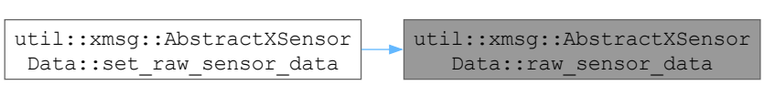
Returns

The raw sensor data vector.

References util::xmsg::AbstractXSensorData::raw_sensor_data_.

Referenced by util::xmsg::AbstractXSensorData::set_raw_sensor_data().

Here is the caller graph for this function:



10.97.3.8 Sample()

```
int32_t util::xmsg::AbstractXSensorData::Sample (
            size_t i) const  [inline], [inherited]
```

Get the raw integer value of the i^{th} sample.

Parameters

in	*i*	The index of the sensor value.

Returns

The raw integer value of the i^{th} ith sample.

References util::xmsg::AbstractXSensorData::raw_sensor_data_.

10.97.3.9 SampleValue()

```
template<typename T = double>
T util::xmsg::AbstractXSensorData::SampleValue (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline], [inherited]
```

Get the i^{th} sensor value in scientific units.

Parameters

in	*i*	The index of the sensor value.
in	*gain*	The sensor sensitivity scale factor.
in	*offs*	The sensor offset.

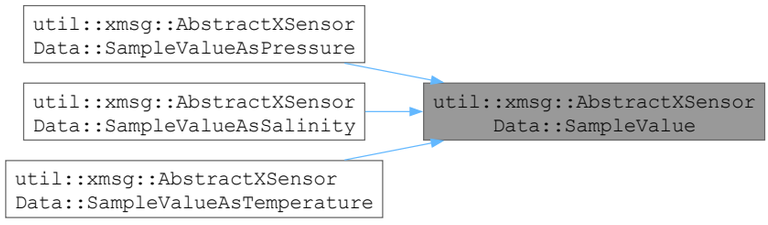
Returns

The i^{th} sensor value in scientific units.

References CATCH_BLOCK, and util::xmsg::AbstractXSensorData::raw_sensor_data_.

Referenced by util::xmsg::AbstractXSensorData::SampleValueAsPressure(), util::xmsg::AbstractXSensorData::SampleValueAsSalinity(), and util::xmsg::AbstractXSensorData::SampleValueAsTemperature().

Here is the caller graph for this function:



10.97.3.10 SampleValueAsPressure()

```
template<typename T = double>
T util::xmsg::AbstractXSensorData::SampleValueAsPressure (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline], [inherited]
```

Get the i^{th} pressure sensor value, in dBar.

Parameters

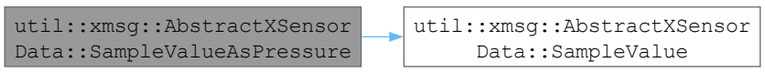
in	*i*	The index of the sensor value.
in	*gain*	The pressure sensor sensitivity scale factor, in LSB/dBar.
in	*offs*	The pressure sensor offset, in dBar.

Returns

The i^{th} pressure sensor value, in dBar.

References util::xmsg::AbstractXSensorData::SampleValue(), XMSG_SENS_BAD_PRES, and XMSG_SENS_MAX_PRES.

Here is the call graph for this function:



10.97.3.11 SampleValueAsSalinity()

```
template<typename T = double>
T util::xmsg::AbstractXSensorData::SampleValueAsSalinity (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline], [inherited]
```

Get the i^{th} salinity sensor value, in PSU.

Parameters

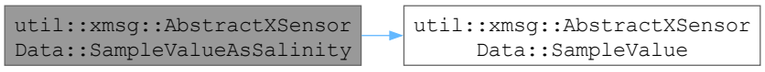
in	*i*	The index of the sensor value.
in	*gain*	The salinity sensor sensitivity scale factor, in LSB/PSU.
in	*offs*	The salinity sensor offset, in PSU.

Returns

The i^{th} salinity sensor value, in PSU.

References util::xmsg::AbstractXSensorData::SampleValue(), XMSG_SENS_BAD_SALN, and XMSG_SENS_MAX_SALN.

Here is the call graph for this function:



10.97.3.12 SampleValueAsTemperature()

```
template<typename T = double>
T util::xmsg::AbstractXSensorData::SampleValueAsTemperature (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline], [inherited]
```

Get the i^{th} temperature sensor value, in degC.

Parameters

in	*i*	The index of the sensor value.
in	*gain*	The temperature sensor sensitivity scale factor, in LSB/degC.
in	*offs*	The temperature sensor offset, in degC.

Returns

The i^{th} temperature sensor value, in degC.

References util::xmsg::AbstractXSensorData::SampleValue(), XMSG_SENS_BAD_TEMP, and XMSG_SENS_MAX_TEMP.

Here is the call graph for this function:

10.97.3.13 sensor_data_format()

```
XSensorDataFormat_t util::xmsg::AbstractXSensorData::sensor_data_format () const  [inline], [inherited]
```

Get the enumerated sensor data format value.

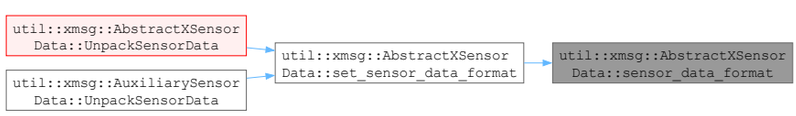
Returns

The enumerated sensor data format value.

References util::xmsg::AbstractXSensorData::sensor_data_format_.

Referenced by util::xmsg::AbstractXSensorData::set_sensor_data_format().

Here is the caller graph for this function:



10.97.3.14 set_id_dive()

```
void util::xmsg::XMessage::set_id_dive (
            int16_t id_dive)  [inline], [protected], [inherited]
```

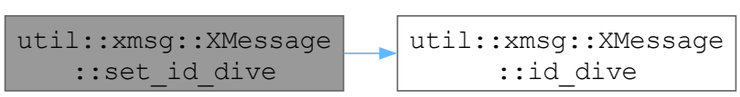
Set the dive or cycle number of this message.

Parameters

in	*id_dive*	The dive or cycle number of this message to set.

References util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::id_dive_.

Here is the call graph for this function:



10.97.3.15 set_id_instrument()

```
void util::xmsg::XMessage::set_id_instrument (
            uint16_t id_instrument)  [inline], [protected], [inherited]
```

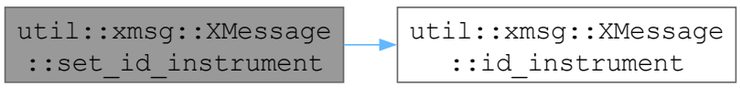
Set the unique instrument identifier.

Parameters

in	*id_instrument*	The unique instrument identifier to set.

References util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::id_instrument_.

Here is the call graph for this function:



10.97.3.16 set_id_xmsg_type()

```
void util::xmsg::XMessage::set_id_xmsg_type (
            XMsgTypeId_t id_xmsg_type)  [inline], [protected], [inherited]
```

Set the X-Message type identifier.

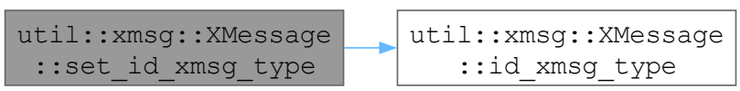
Parameters

in	*id_xmsg_type*	The X-Message type identifier to set.

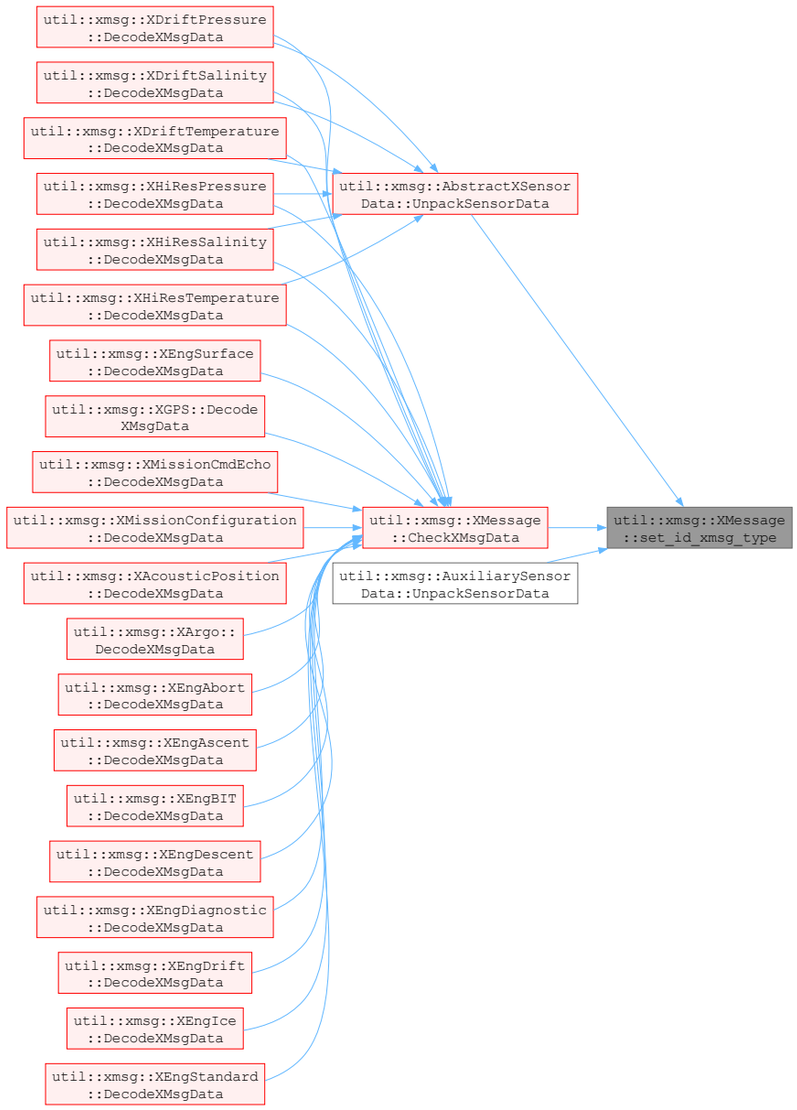
References util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.97.3.17 set_raw_sensor_data()

```
void util::xmsg::AbstractXSensorData::set_raw_sensor_data (
            const vector< int32_t > & raw_sensor_data)  [inline], [protected], [inherited]
```

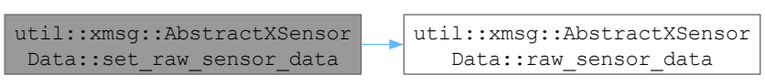
Set raw integer counts of the measured sensor data.

Parameters

in	*raw_sensor_data*	The data to set.

References util::xmsg::AbstractXSensorData::raw_sensor_data(), and util::xmsg::AbstractXSensorData::raw_sensor_data_.

Here is the call graph for this function:



10.97.3.18 set_sensor_data_format()

```
void util::xmsg::AbstractXSensorData::set_sensor_data_format (
            XSensorDataFormat_t sensor_data_format)  [inline], [protected], [inherited]
```

Set the sensor data format.

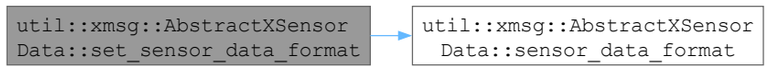
Parameters

in	*sensor_data_format*	The sensor data format to set.

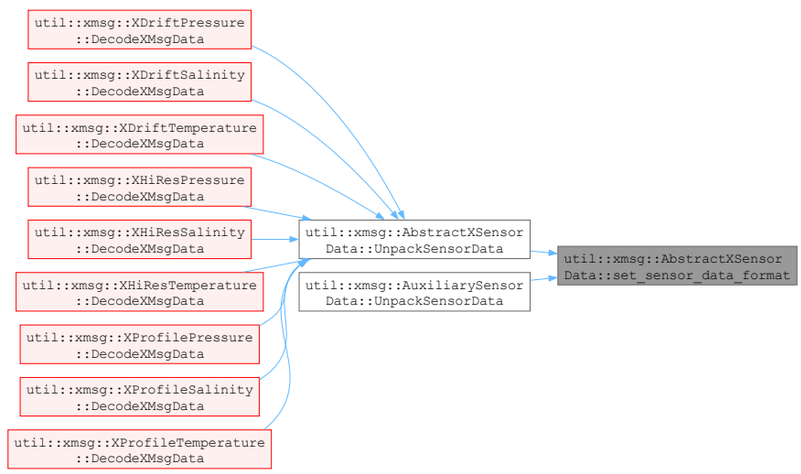
References util::xmsg::AbstractXSensorData::sensor_data_format(), and util::xmsg::AbstractXSensorData::sensor_data_format_.

Referenced by util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.97.3.19 set_xmsg_index()

```
void util::xmsg::XMessageIndexable::set_xmsg_index (
            uint8_t xmsg_index)  [inline], [protected], [inherited]
```

Set the index of this X-Message.

Parameters

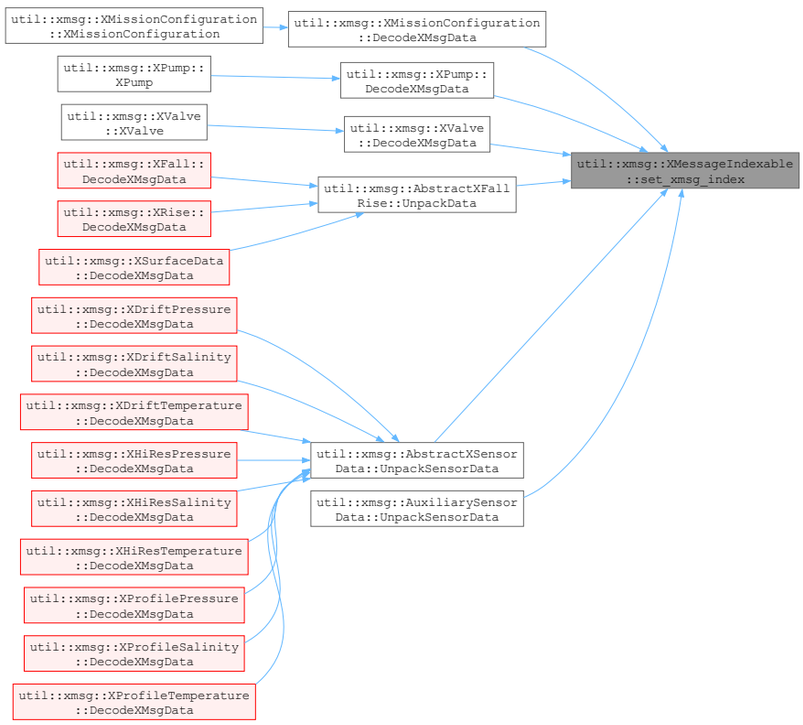
in	*xmsg_index*	The index of this X-Message.

References util::xmsg::XMessageIndexable::xmsg_index(), and util::xmsg::XMessageIndexable::xmsg_index_.

Referenced by util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(), util::xmsg::XValve::DecodeXMsgData(), util::xmsg::AbstractXFallRise::UnpackData(), util::xmsg::AbstractXSensorData::UnpackSensorDa and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:

Here is the caller graph for this function:



10.97.3.20 set_xmsg_size_bytes()

```
void util::xmsg::XMessage::set_xmsg_size_bytes (
            uint16_t xmsg_size_bytes)  [inline], [protected], [inherited]
```

Set the size of the X-Message, in bytes.

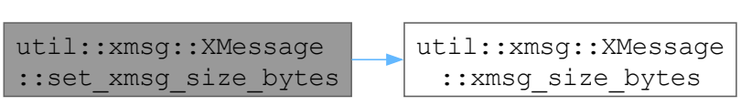
Parameters

in	*xmsg_size_bytes*	The size of the X-Message to set, in bytes.

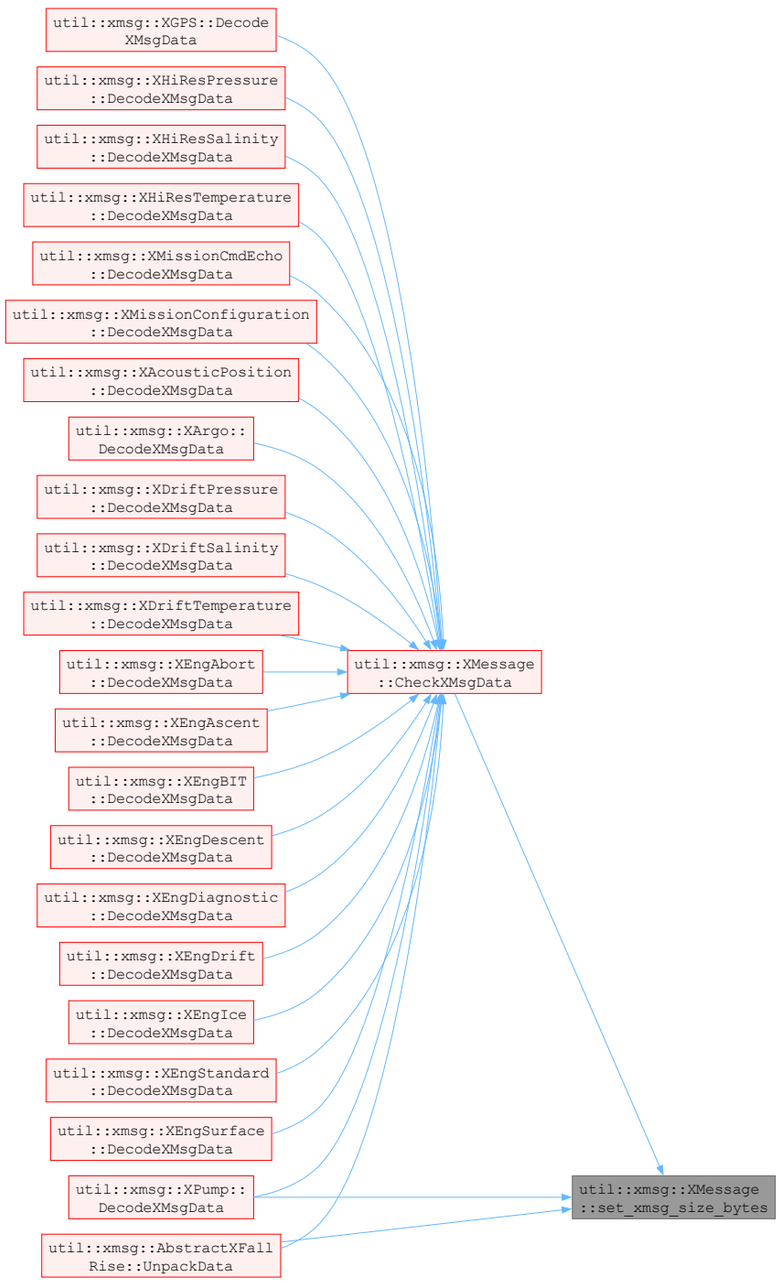
References util::xmsg::XMessage::xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XPump::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::Unpa

Here is the call graph for this function:



Here is the caller graph for this function:



10.97.3.21 UnpackSensorData()

```
virtual vector< int32_t > util::xmsg::AbstractXSensorData::UnpackSensorData (
            const vector< uint8_t > & data)  [inline], [protected], [virtual], [inherited]
```

Unpack the sensor data from the binary message.

This field unpacks the encoded sensor data from the message and stores the decoded sensor values in raw_sensor_data_. First the function checks the size of the message, and the data packing format. The message data is decoded according to the packing format type. Both first-difference and curvature encoding are supported. The encoding method is determined from the value of the high nibble of byte `data`[1].

Once this function completes successfully, then the data may be recovered in scientific units from the raw sensor values by applying the appropriate sensor gain and offset values.

Parameters

`in`	*data*	The binary message to unpack, in a vector of unsigned bytes.

Warning

The function assumes the message data is valid. Message validity should be ensured before this function is called.

Exceptions

util::log::MessageException	if • an error occurs while decoding the message, • the message is of insufficient length, or • an invalid packing format is encountered.

Returns

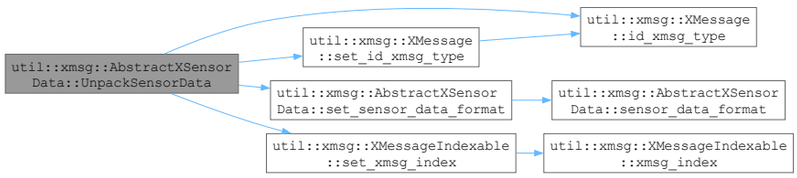
The sensor data raw counts, in a vector of signed integers.

Reimplemented in util::xmsg::AuxiliarySensorData.

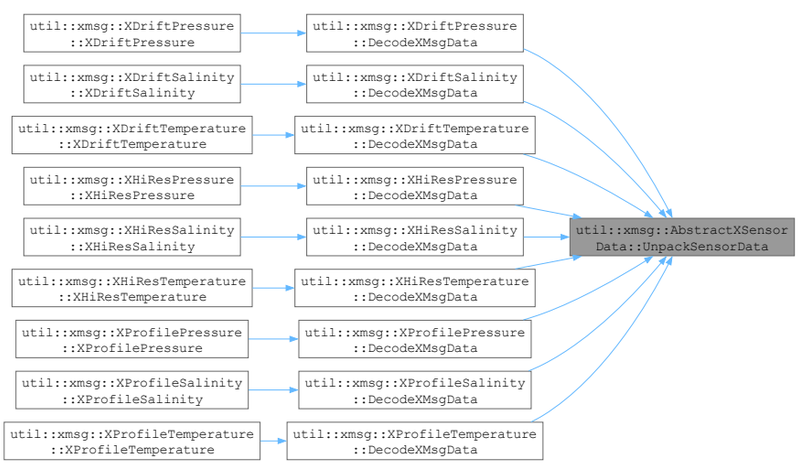
References CATCH_BLOCK, CATCH_NOTHROW, EXC_THROW, i24, util::xmsg::XMessage::id_xmsg_type(), ldbg, lwrn, util::xmsg::AbstractXSensorData::raw_sensor_data_, util::xmsg::XMessage::set_id_xmsg_type(), util::xmsg::AbstractXSensorData::s util::xmsg::XMessageIndexable::set_xmsg_index(), u16, u24, XMSG_SENS_MAX_SUB, util::xmsg::XMSG_TYPE_ID_PRES0, util::xmsg::XMSG_TYPE_ID_PRES_DRIFT8, util::xmsg::XMSG_TYPE_ID_PRES_HI_RES0, util::xmsg::XMSG_TYPE_ID_SALN0, util::xmsg::XMSG_TYPE_ID_SALN_DRIFT8, util::xmsg::XMSG_TYPE_ID_SALN_HI_RES0, util::xmsg::XMSG_TYPE_ID_TEMP0, util::xmsg::XMSG_TYPE_ID_TEMP_DRIFT8, util::xmsg::XMSG_TYPE_ID_TEMP_HI_RES0, util::xmsg::XSENS_FMT_CURVATURE, and util::xmsg::XSENS_FMT_DIFFERENCE.

Referenced by util::xmsg::XDriftPressure::DecodeXMsgData(), util::xmsg::XDriftSalinity::DecodeXMsgData(), util::xmsg::XDriftTemperatur util::xmsg::XHiResPressure::DecodeXMsgData(), util::xmsg::XHiResSalinity::DecodeXMsgData(), DecodeXMsgData(), util::xmsg::XProfilePressure::DecodeXMsgData(), util::xmsg::XProfileSalinity::DecodeXMsgData(), and util::xmsg::XProfileTemperature::D

Here is the call graph for this function:



Here is the caller graph for this function:



10.97.3.22 xmsg_index()

```
uint8_t util::xmsg::XMessageIndexable::xmsg_index () const  [inline], [inherited]
```

Get the index of this X-Message.

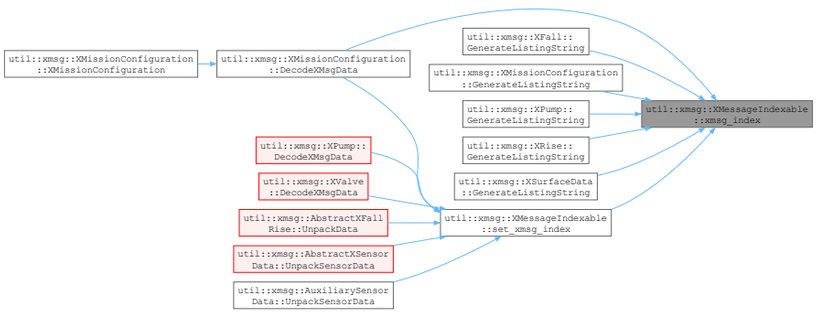
Returns

The index of this X-Message.

References util::xmsg::XMessageIndexable::xmsg_index_.

Referenced by util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing util::xmsg::XSurfaceData::GenerateListingString(), and util::xmsg::XMessageIndexable::set_xmsg_index().

Here is the caller graph for this function:



10.97.3.23 xmsg_size_bytes()

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes () const  [inline], [inherited]
```

Get the size of the X-Message, in bytes.

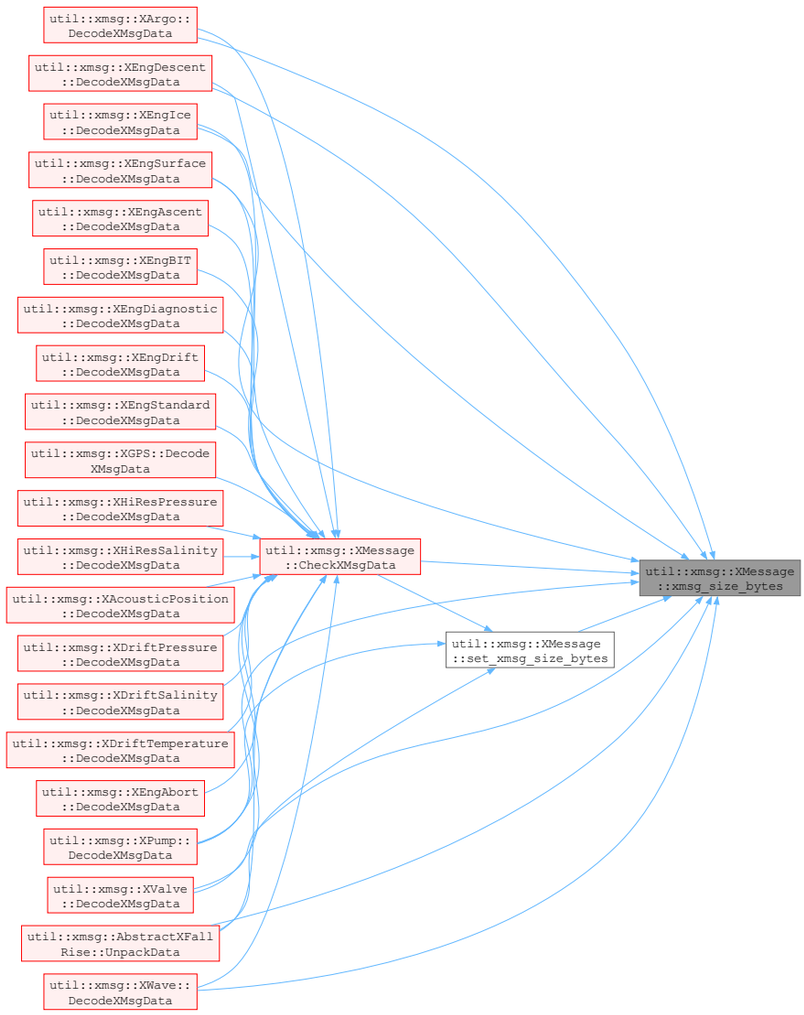
Returns

The size of the X-Message, in bytes.

References util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgI util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(), util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the caller graph for this function:



10.97.4 Member Data Documentation

10.97.4.1 id_dive_

```
int16_t util::xmsg::XMessage::id_dive_  [private], [inherited]
```

The dive or cycle number of this message.

Referenced by util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::set_id_dive().

10.97.4.2 id_instrument_

```
uint16_t util::xmsg::XMessage::id_instrument_  [private], [inherited]
```

The unique WHOI identifier of this instrument.

Referenced by util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::set_id_instrument().

10.97.4.3 id_xmsg_type_

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type_  [private], [inherited]
```

The X-message type identifier.

Referenced by util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::set_id_xmsg_type().

10.97.4.4 raw_sensor_data_

```
vector<int32_t> util::xmsg::AbstractXSensorData::raw_sensor_data_  [protected], [inherited]
```

The raw integer counts of the measured sensor data.

This vector of integers represents the "raw" sample values of the message. That is, by applying the appropriate gain and offset corrections, the sensor values may be recovered in scientific units. This data may represent pressure, temperature, salinity, or any other data that is recorded during the drift or profile phases of the dive cycle.

Referenced by util::xmsg::AbstractXSensorData::num_samples(), util::xmsg::AbstractXSensorData::raw_sensor_data(), util::xmsg::AbstractXSensorData::Sample(), util::xmsg::AbstractXSensorData::SampleValue(), util::xmsg::AbstractXSensorData::set_raw_ util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

10.97.4.5 sensor_data_format_

```
XSensorDataFormat_t util::xmsg::AbstractXSensorData::sensor_data_format_  [protected], [inherited]
```

The enumerated sensor data packing format.

This field indicates the format of encoded sensor data. The manner in which the sensor data is decoded is determined by the format of this field.

Referenced by util::xmsg::AbstractXSensorData::sensor_data_format(), and util::xmsg::AbstractXSensorData::set_sensor_data_format().

10.97.4.6 xmsg_index_

```
uint8_t util::xmsg::XMessageIndexable::xmsg_index_  [private], [inherited]
```

The index of this X-Message.

Referenced by util::xmsg::XMessageIndexable::set_xmsg_index(), and util::xmsg::XMessageIndexable::xmsg_index().

10.97.4.7 xmsg_size_bytes_

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes_  [private], [inherited]
```

The size of the X message, in bytes.

Referenced by util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes().

The documentation for this class was generated from the following file:

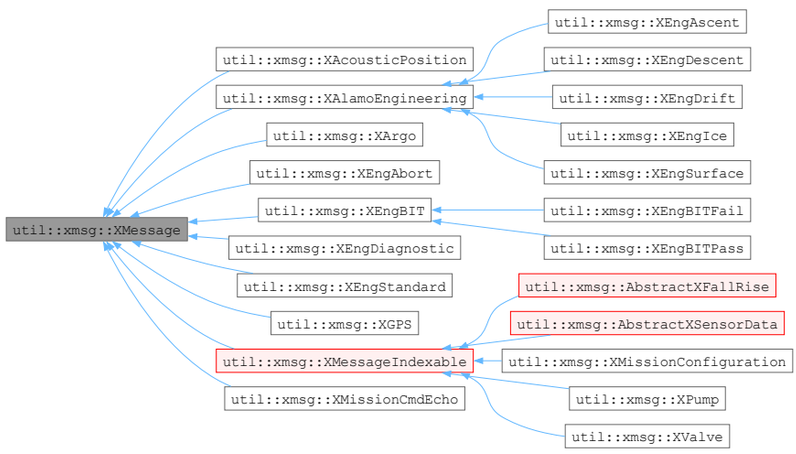
- data/xmsg/XHiResTemperature.h

10.98 util::xmsg::XMessage Class Reference

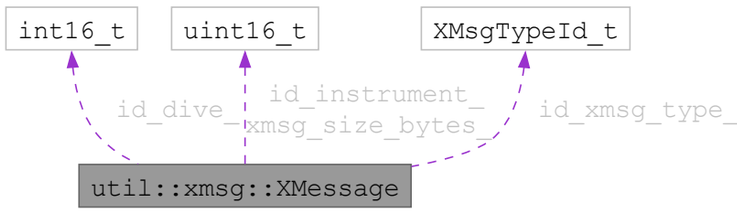
Class representing a message following the X format specification.

`#include <XMessage.h>`

Inheritance diagram for util::xmsg::XMessage:



Collaboration diagram for util::xmsg::XMessage:



Public Member Functions

- XMessage (int16_t dive, uint16_t whoi, const vector< uint8_t > &data) ATTR_NO_EXCEPTIONS

  The Parameterized constructor.
- virtual ~XMessage ()

  The destructor.
- const uint8_t * CheckXMsgData (const vector< uint8_t > &data, int id, int minBytes, bool parseSize=true)

  Check the X-message for the correct ID, minimum size, and message terminator.
- virtual void DecodeXMsgData (const vector< uint8_t > &data)=0

  Decode the X-Message data.
- int16_t id_dive () const

  Get the dive or cycle number of this X-Message.
- uint16_t id_instrument () const

  Get the unique instrument identifier of this X-Message.
- XMsgTypeId_t id_xmsg_type () const

  Get the enumerated X-Message type identifier.
- uint16_t xmsg_size_bytes () const

  Get the size of the X-Message, in bytes.

Protected Member Functions

- XMessage (int16_t dive=XMSG_DIVE_UNKNOWN, uint16_t whoi=XMSG_ID_UNKNOWN, XMsgTypeId_t type=XMSG_TYPE_ID_UNKNOWN) ATTR_NO_EXCEPTIONS

  The default constructor.
- void set_id_dive (int16_t id_dive)

  Set the dive or cycle number of this message.
- void set_id_instrument (uint16_t id_instrument)

  Set the unique instrument identifier.
- void set_id_xmsg_type (XMsgTypeId_t id_xmsg_type)

  Set the X-Message type identifier.
- void set_xmsg_size_bytes (uint16_t xmsg_size_bytes)

  Set the size of the X-Message, in bytes.

Private Attributes

- int16_t id_dive_

  The dive or cycle number of this message.
- uint16_t id_instrument_

  The unique WHOI identifier of this instrument.
- XMsgTypeId_t id_xmsg_type_

  The X-message type identifier.
- uint16_t xmsg_size_bytes_

  The size of the X message, in bytes.

10.98.1 Detailed Description

Class representing a message following the X format specification.

10.98.2 Constructor & Destructor Documentation

10.98.2.1 XMessage() [1/2]

```
util::xmsg::XMessage::XMessage (
            int16_t dive = XMSG_DIVE_UNKNOWN,
            uint16_t whoi = XMSG_ID_UNKNOWN,
            XMsgTypeId_t type = XMSG_TYPE_ID_UNKNOWN)  [inline], [protected]
```

The default constructor.

Parameters

in	*dive*	The dive number of this X-Message object.
in	*whoi*	The unique instrument identifier.
in	*type*	The X-Message type identifier.

Exceptions

None.	Never throws.

10.98.2.2 XMessage() [2/2]

```
util::xmsg::XMessage::XMessage (
            int16_t dive,
            uint16_t whoi,
            const vector< uint8_t > & data)  [inline]
```

The Parameterized constructor.

Parameters

in	*dive*	The dive or cycle number of this message.
in	*whoi*	The unique WHOI identifier.
in	*data*	The binary X-message data to decode.

Exceptions

None.	Never throws.

10.98.2.3 ~XMessage()

```
virtual util::xmsg::XMessage::~XMessage ()  [inline], [virtual]
```

The destructor.

10.98.3 Member Function Documentation

10.98.3.1 CheckXMsgData()

```
const uint8_t * util::xmsg::XMessage::CheckXMsgData (
            const vector< uint8_t > & data,
            int id,
            int minBytes,
            bool parseSize = true)  [inline]
```

Check the X-message for the correct ID, minimum size, and message terminator.

Parameters

in	*data*	The X-message binary data to check.
in	*id*	The expected ID of the X-message. If this field is less that 0, then the ID check will be skipped.
in	*minBytes*	The minimum number of bytes expected in a valid message.
in	*parseSize*	If TRUE, then the message size field will be parsed from the X-Message. Otherwise, the size field will not be parsed.

Exceptions

util::log::MessageException	if • the message does not contain the expected ID, • the message does not contain the minimum number of expected bytes, • the message does not contain a terminator (';') in the correct location, or, • any other error occurs.

Returns

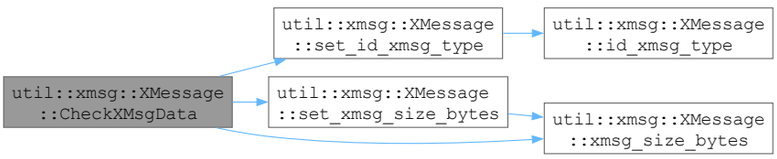
A pointer to the underlying data buffer of data.

References CATCH_BLOCK, EXC_THROW, lwrn, set_id_xmsg_type(), set_xmsg_size_bytes(), u16, and xmsg_size_bytes().

Referenced by util::xmsg::XAcousticPosition::DecodeXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XDriftPressure::Deco
util::xmsg::XDriftSalinity::DecodeXMsgData(), util::xmsg::XDriftTemperature::DecodeXMsgData(), util::xmsg::XEngAbort::DecodeXMsgDa
util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngBIT::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(),
util::xmsg::XEngDiagnostic::DecodeXMsgData(), util::xmsg::XEngDrift::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(),
util::xmsg::XEngStandard::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XGPS::DecodeXMsgData(),
util::xmsg::XHiResPressure::DecodeXMsgData(), util::xmsg::XHiResSalinity::DecodeXMsgData(), util::xmsg::XHiResTemperature::Decod
util::xmsg::XMissionCmdEcho::DecodeXMsgData(), util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XProfilePressure:

util::xmsg::XProfileSalinity::DecodeXMsgData(), util::xmsg::XProfileTemperature::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgD
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the call graph for this function:



10.98.3.2 DecodeXMsgData()

```
virtual void util::xmsg::XMessage::DecodeXMsgData (
            const vector< uint8_t > & data)  [pure virtual]
```

Decode the X-Message data.

Parameters

in	*data*	The encoded X message to decode.

Implemented in util::xmsg::AbstractXFallRise, util::xmsg::AbstractXSensorData, util::xmsg::XAcousticPosition, util::xmsg::XArgo, util::xmsg::XDriftPressure, util::xmsg::XDriftSalinity, util::xmsg::XDriftTemperature, util::xmsg::XEngAbort, util::xmsg::XEngAscent, util::xmsg::XEngBIT, util::xmsg::XEngDescent, util::xmsg::XEngDiagnostic, util::xmsg::XEngDrift, util::xmsg::XEngIce, util::xmsg::XEngSurface, util::xmsg::XFall, util::xmsg::XGPS, util::xmsg::XHiResPressure, util::xmsg::XHiResSalinity, util::xmsg::XHiResTemperature, util::xmsg::XMessageIndexable, util::xmsg::XMissionCmdEcho, util::xmsg::XMissionConfiguration, util::xmsg::XProfilePressure, util::xmsg::XProfileSalinity, util::xmsg::XProfileTemperature, util::xmsg::XPump, util::xmsg::XRise, util::xmsg::XSurfaceData, util::xmsg::XValve, and util::xmsg::XWave.

10.98.3.3 id_dive()

```
int16_t util::xmsg::XMessage::id_dive () const  [inline]
```

Get the dive or cycle number of this X-Message.

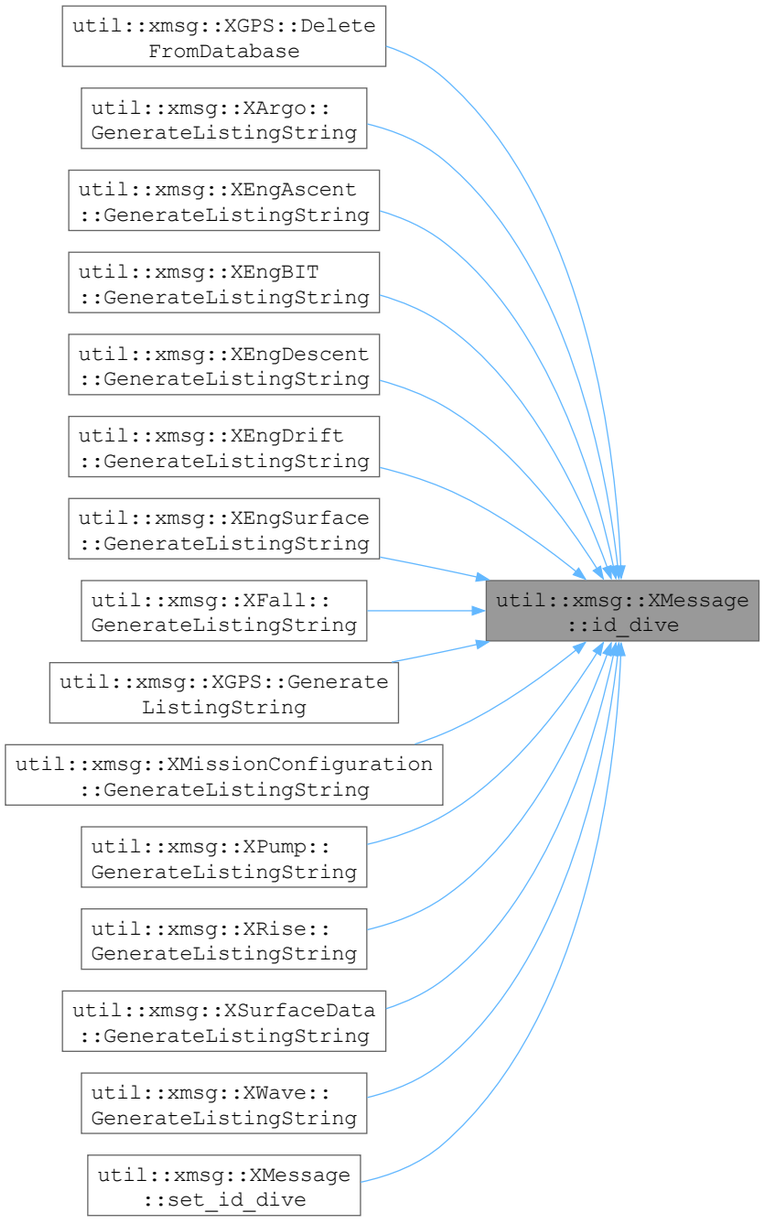
Returns

The dive or cycle number of this X-Message.

References id_dive_.

Referenced by util::xmsg::XGPS::DeleteFromDatabase(), util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateLis
util::xmsg::XEngBIT::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingStr
util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(),
util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing
util::xmsg::XSurfaceData::GenerateListingString(), util::xmsg::XWave::GenerateListingString(), and set_id_dive().

Here is the caller graph for this function:



10.98.3.4 id_instrument()

```
uint16_t util::xmsg::XMessage::id_instrument () const  [inline]
```

Get the unique instrument identifier of this X-Message.

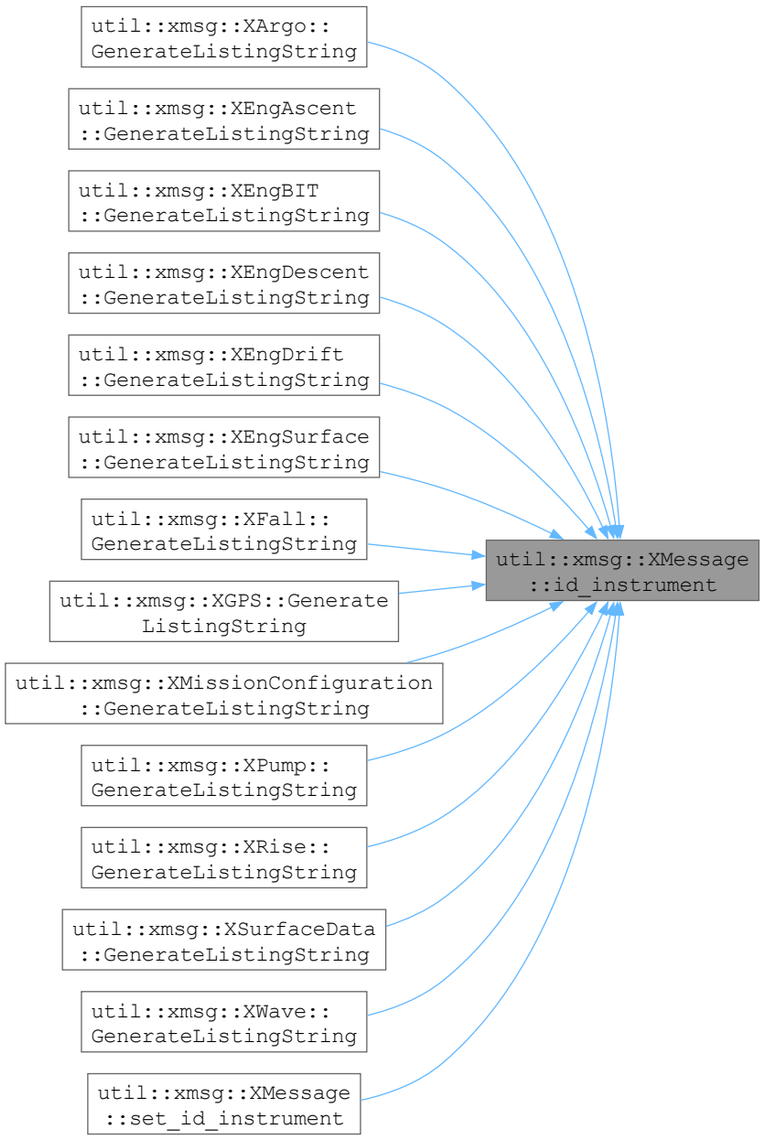
Returns

The unique instrument identifier of this X-Message.

References id_instrument_.

Referenced by util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngBIT::Generate
util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), util::xmsg::XEngSurface::GenerateListin
util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingS
util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(),
util::xmsg::XWave::GenerateListingString(), and set_id_instrument().

Here is the caller graph for this function:



10.98.3.5 id_xmsg_type()

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type () const  [inline]
```

Get the enumerated X-Message type identifier.

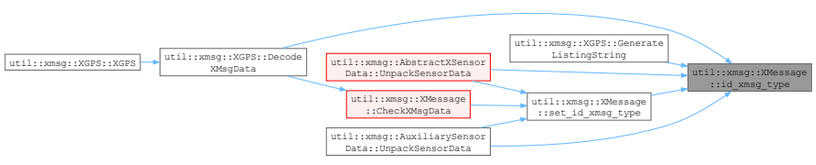
Returns

The enumerated X-Message type identifier.

References id_xmsg_type_.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XGPS::GenerateListingString(), set_id_xmsg_type(), util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the caller graph for this function:



10.98.3.6 set_id_dive()

```
void util::xmsg::XMessage::set_id_dive (
            int16_t id_dive)  [inline], [protected]
```

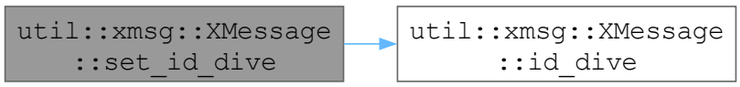
Set the dive or cycle number of this message.

Parameters

in	*id_dive*	The dive or cycle number of this message to set.

References id_dive(), and id_dive_.

Here is the call graph for this function:



10.98.3.7 set_id_instrument()

```
void util::xmsg::XMessage::set_id_instrument (
            uint16_t id_instrument)  [inline], [protected]
```

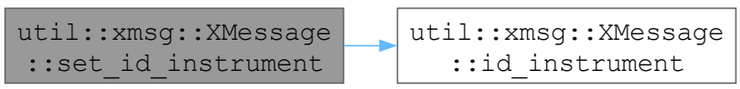
Set the unique instrument identifier.

Parameters

in	*id_instrument*	The unique instrument identifier to set.

References id_instrument(), and id_instrument_.

Here is the call graph for this function:



10.98.3.8 set_id_xmsg_type()

```
void util::xmsg::XMessage::set_id_xmsg_type (
            XMsgTypeId_t id_xmsg_type)  [inline], [protected]
```

Set the X-Message type identifier.

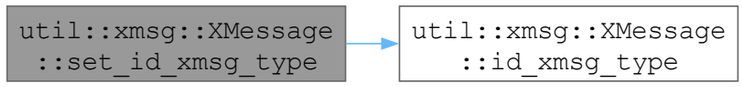
Parameters

in	*id_xmsg_type*	The X-Message type identifier to set.

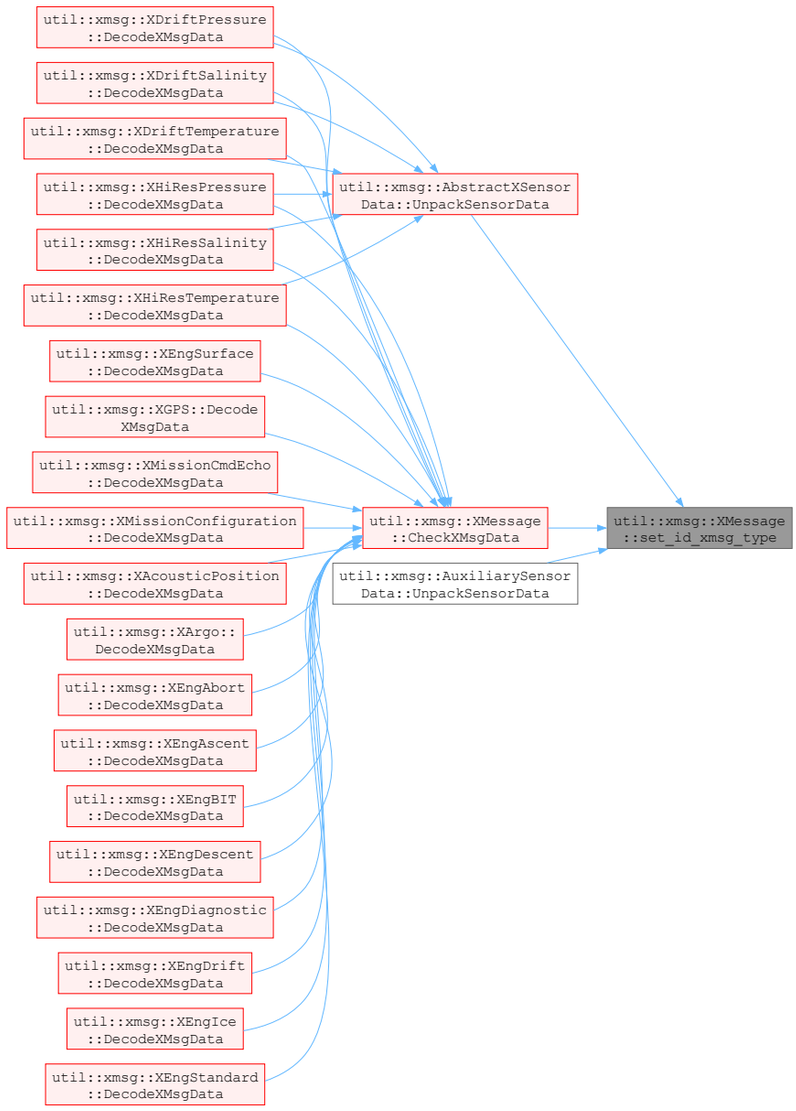
References id_xmsg_type(), and id_xmsg_type_.

Referenced by CheckXMsgData(), util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackS

Here is the call graph for this function:



Here is the caller graph for this function:



10.98.3.9 set_xmsg_size_bytes()

```
void util::xmsg::XMessage::set_xmsg_size_bytes (
            uint16_t xmsg_size_bytes)  [inline], [protected]
```

Set the size of the X-Message, in bytes.

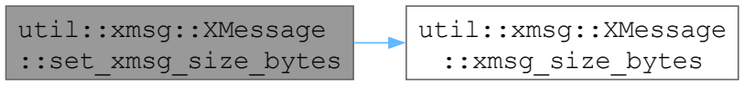
Parameters

in	*xmsg_size_bytes*	The size of the X-Message to set, in bytes.

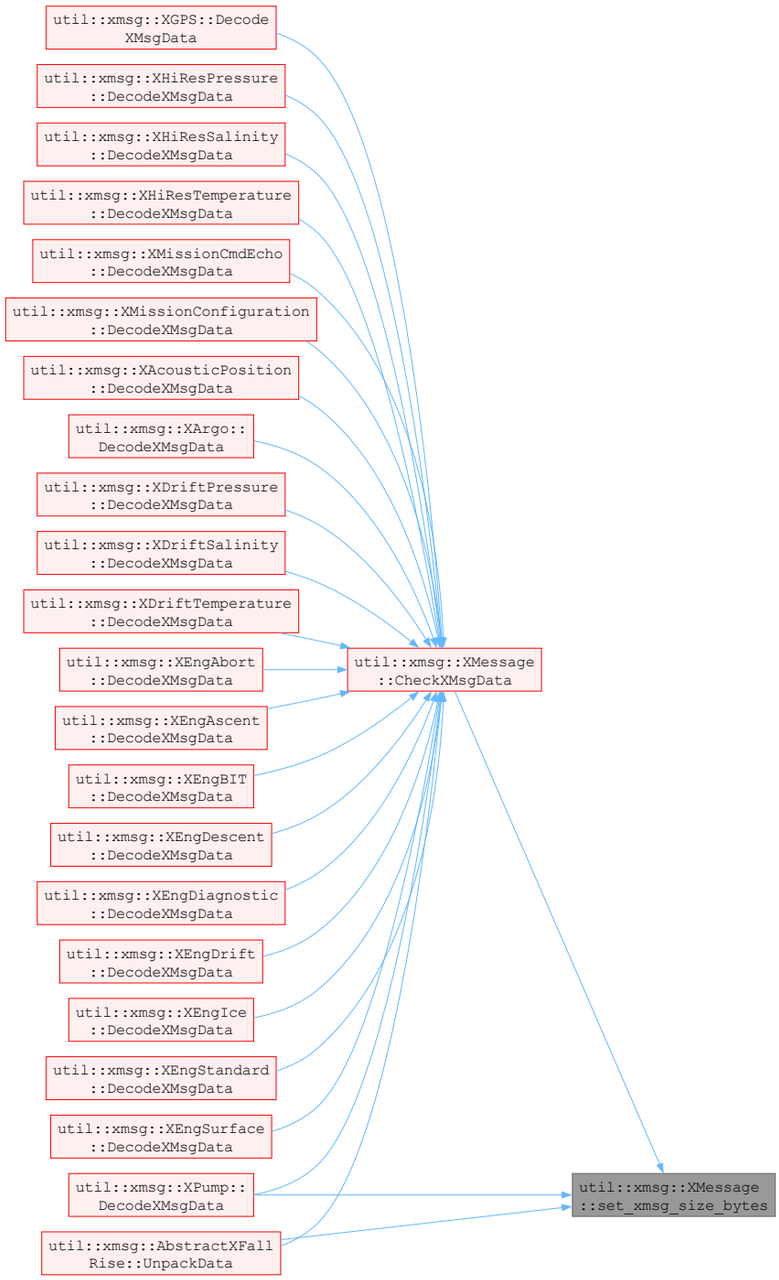
References xmsg_size_bytes(), and xmsg_size_bytes_.

Referenced by CheckXMsgData(), util::xmsg::XPump::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.98.3.10 xmsg_size_bytes()

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes () const  [inline]
```

Get the size of the X-Message, in bytes.

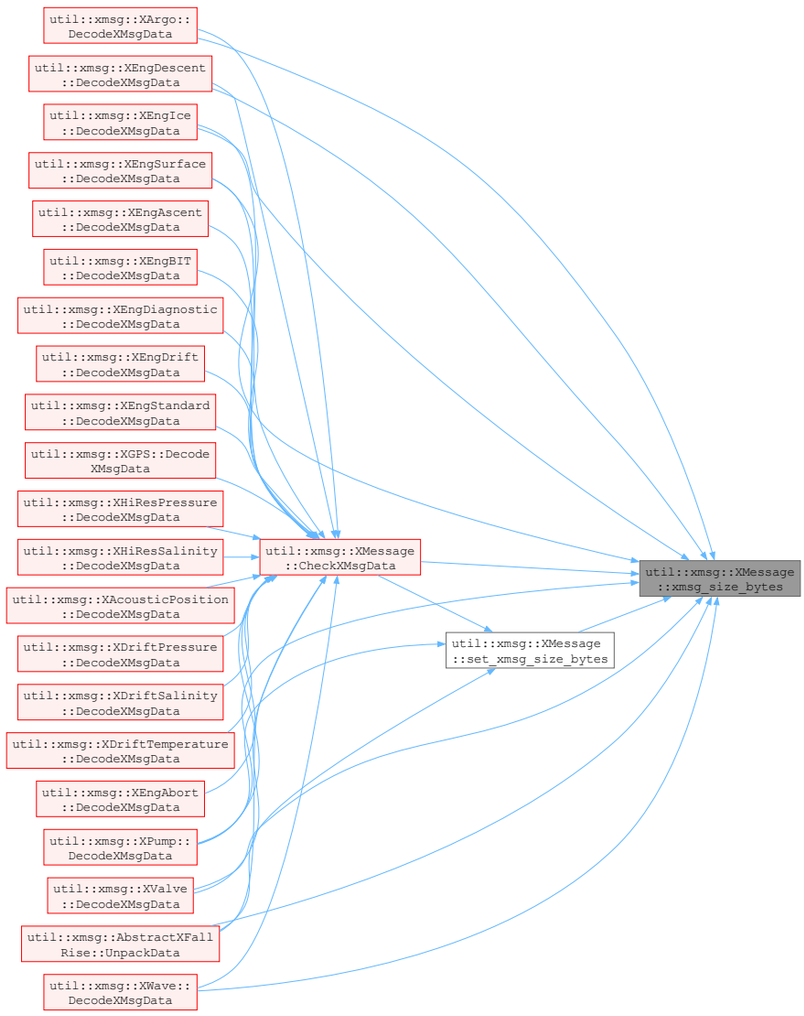
Returns

The size of the X-Message, in bytes.

References xmsg_size_bytes_.

Referenced by CheckXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(), util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), set_xmsg_size_bytes(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the caller graph for this function:



10.98.4 Member Data Documentation

10.98.4.1 id_dive_

```
int16_t util::xmsg::XMessage::id_dive_  [private]
```

The dive or cycle number of this message.

Referenced by id_dive(), and set_id_dive().

10.98.4.2 id_instrument_

```
uint16_t util::xmsg::XMessage::id_instrument_  [private]
```

The unique WHOI identifier of this instrument.

Referenced by id_instrument(), and set_id_instrument().

10.98.4.3 id_xmsg_type_

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type_  [private]
```

The X-message type identifier.

Referenced by id_xmsg_type(), and set_id_xmsg_type().

10.98.4.4 xmsg_size_bytes_

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes_  [private]
```

The size of the X message, in bytes.

Referenced by set_xmsg_size_bytes(), and xmsg_size_bytes().

The documentation for this class was generated from the following file:

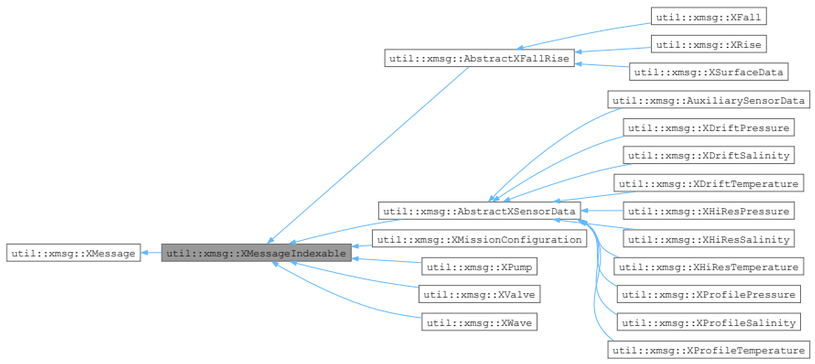
- data/xmsg/XMessage.h

10.99 util::xmsg::XMessageIndexable Class Reference

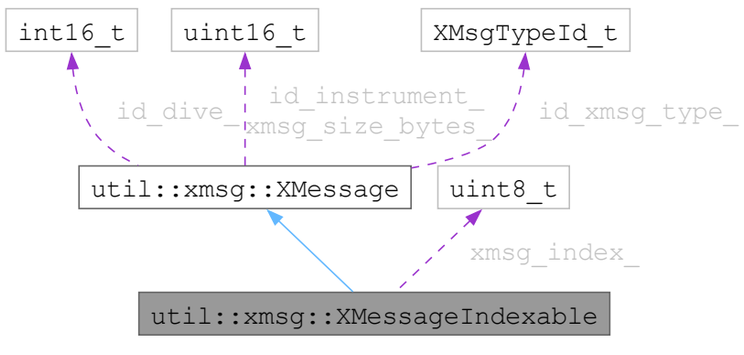
Class representing an indexable X-Message.

`#include <XMessageIndexable.h>`

Inheritance diagram for util::xmsg::XMessageIndexable:



Collaboration diagram for util::xmsg::XMessageIndexable:



Public Member Functions

- XMessageIndexable (int16_t dive, uint16_t whoi, const vector< uint8_t > &data) ATTR_NO_EXCEPTIONS
  The parameterized constructor.
- ~XMessageIndexable ()
  The destructor.
- const uint8_t * CheckXMsgData (const vector< uint8_t > &data, int id, int minBytes, bool parseSize=true)
  Check the X-message for the correct ID, minimum size, and message terminator.
- virtual void DecodeXMsgData (const vector< uint8_t > &data)=0
  Decode the X-Message data.
- int16_t id_dive () const
  Get the dive or cycle number of this X-Message.
- uint16_t id_instrument () const
  Get the unique instrument identifier of this X-Message.
- XMsgTypeId_t id_xmsg_type () const
  Get the enumerated X-Message type identifier.
- uint8_t xmsg_index () const
  Get the index of this X-Message.
- uint16_t xmsg_size_bytes () const
  Get the size of the X-Message, in bytes.

Protected Member Functions

- XMessageIndexable () ATTR_NO_EXCEPTIONS
  The default constructor.
- void set_id_dive (int16_t id_dive)
  Set the dive or cycle number of this message.
- void set_id_instrument (uint16_t id_instrument)
  Set the unique instrument identifier.
- void set_id_xmsg_type (XMsgTypeId_t id_xmsg_type)
  Set the X-Message type identifier.
- void set_xmsg_index (uint8_t xmsg_index)
  Set the index of this X-Message.
- void set_xmsg_size_bytes (uint16_t xmsg_size_bytes)
  Set the size of the X-Message, in bytes.

Private Attributes

- int16_t id_dive_
  The dive or cycle number of this message.
- uint16_t id_instrument_
  The unique WHOI identifier of this instrument.
- XMsgTypeId_t id_xmsg_type_
  The X-message type identifier.
- uint8_t xmsg_index_
  The index of this X-Message.
- uint16_t xmsg_size_bytes_
  The size of the X message, in bytes.

10.99.1 Detailed Description

Class representing an indexable X-Message.

10.99.2 Constructor & Destructor Documentation

10.99.2.1 XMessageIndexable() [1/2]

```
util::xmsg::XMessageIndexable::XMessageIndexable ()  [inline], [protected]
```

The default constructor.

Exceptions

None.	Never throws.

10.99.2.2 XMessageIndexable() [2/2]

```
util::xmsg::XMessageIndexable::XMessageIndexable (
            int16_t dive,
            uint16_t whoi,
            const vector< uint8_t > & data)  [inline]
```

The parameterized constructor.

Parameters

in	*dive*	The dive or cycle number of this message.
in	*whoi*	The unique WHOI identifier.
in	*data*	The binary X-message data to decode.

Exceptions

None.	Never throws.

10.99.2.3 ~XMessageIndexable()

```
util::xmsg::XMessageIndexable::~XMessageIndexable ()  [inline]
```

The destructor.

10.99.3 Member Function Documentation

10.99.3.1 CheckXMsgData()

```
const uint8_t * util::xmsg::XMessage::CheckXMsgData (
            const vector< uint8_t > & data,
            int id,
            int minBytes,
            bool parseSize = true)  [inline], [inherited]
```

Check the X-message for the correct ID, minimum size, and message terminator.

Parameters

in	*data*	The X-message binary data to check.
in	*id*	The expected ID of the X-message. If this field is less that 0, then the ID check will be skipped.
in	*minBytes*	The minimum number of bytes expected in a valid message.
in	*parseSize*	If TRUE, then the message size field will be parsed from the X-Message. Otherwise, the size field will not be parsed.

Exceptions

util::log::MessageException	if • the message does not contain the expected ID, • the message does not contain the minimum number of expected bytes, • the message does not contain a terminator (';') in the correct location, or, • any other error occurs.

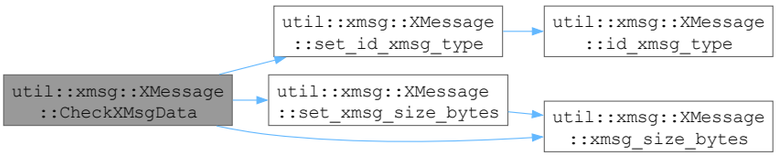
Returns

A pointer to the underlying data buffer of data.

References CATCH_BLOCK, EXC_THROW, lwrn, util::xmsg::XMessage::set_id_xmsg_type(), util::xmsg::XMessage::set_xmsg_size_byte
u16, and util::xmsg::XMessage::xmsg_size_bytes().

Referenced by util::xmsg::XAcousticPosition::DecodeXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XDriftPressure::Dec
util::xmsg::XDriftSalinity::DecodeXMsgData(), util::xmsg::XDriftTemperature::DecodeXMsgData(), util::xmsg::XEngAbort::DecodeXMsgDa
util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngBIT::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(),
util::xmsg::XEngDiagnostic::DecodeXMsgData(), util::xmsg::XEngDrift::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(),
util::xmsg::XEngStandard::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XGPS::DecodeXMsgData(),
util::xmsg::XHiResPressure::DecodeXMsgData(), util::xmsg::XHiResSalinity::DecodeXMsgData(), util::xmsg::XHiResTemperature::Decod
util::xmsg::XMissionCmdEcho::DecodeXMsgData(), util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XProfilePressure::
util::xmsg::XProfileSalinity::DecodeXMsgData(), util::xmsg::XProfileTemperature::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgD
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the call graph for this function:



10.99.3.2 DecodeXMsgData()

```
virtual void util::xmsg::XMessageIndexable::DecodeXMsgData (
            const vector< uint8_t > & data)  [pure virtual]
```

Decode the X-Message data.

Parameters

in	*data*	The encoded X message to decode.

Implements util::xmsg::XMessage.

Implemented in util::xmsg::AbstractXFallRise, util::xmsg::AbstractXSensorData, util::xmsg::XDriftPressure, util::xmsg::XDriftSalinity,
util::xmsg::XDriftTemperature, util::xmsg::XFall, util::xmsg::XHiResPressure, util::xmsg::XHiResSalinity, util::xmsg::XHiResTemperature,
util::xmsg::XMissionConfiguration, util::xmsg::XProfilePressure, util::xmsg::XProfileSalinity, util::xmsg::XProfileTemperature,
util::xmsg::XPump, util::xmsg::XRise, util::xmsg::XSurfaceData, util::xmsg::XValve, and util::xmsg::XWave.

10.99.3.3 id_dive()

```
int16_t util::xmsg::XMessage::id_dive () const  [inline], [inherited]
```

Get the dive or cycle number of this X-Message.

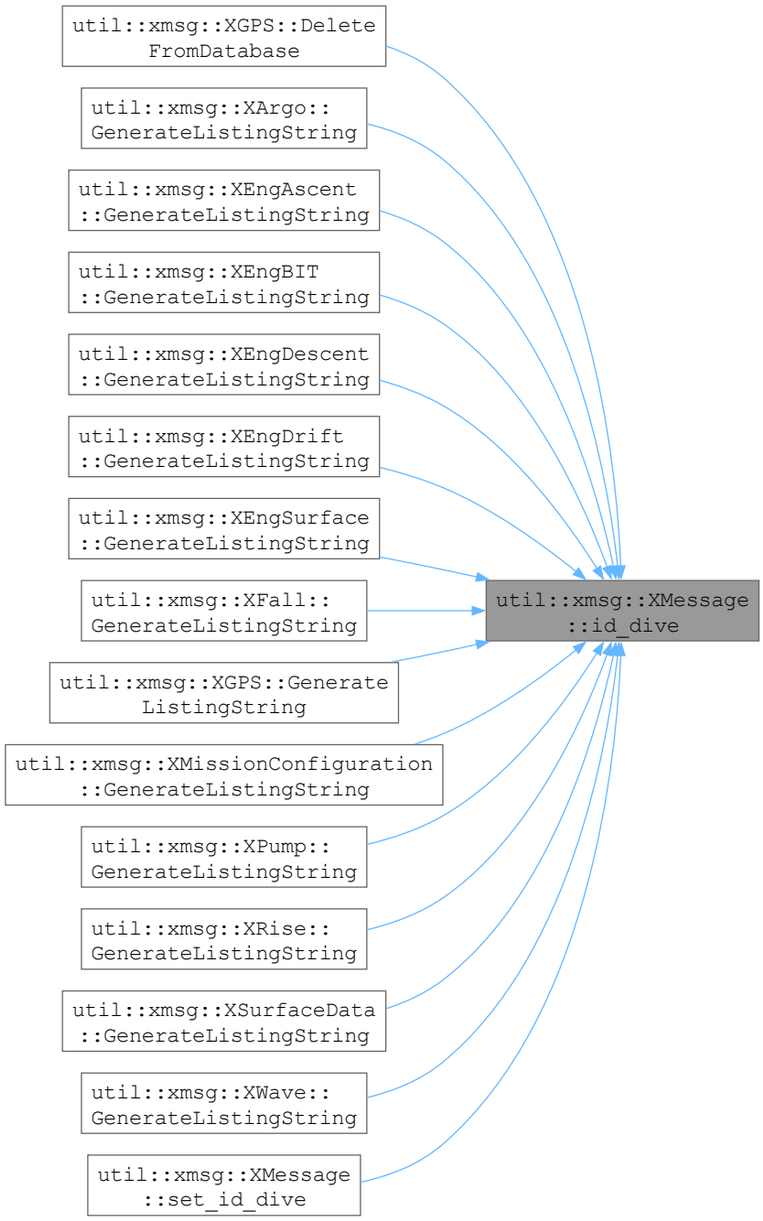
Returns

The dive or cycle number of this X-Message.

References util::xmsg::XMessage::id_dive_.

Referenced by util::xmsg::XGPS::DeleteFromDatabase(), util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateLis
util::xmsg::XEngBIT::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingStr
util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(),
util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing
util::xmsg::XSurfaceData::GenerateListingString(), util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_dive().

Here is the caller graph for this function:



10.99.3.4 id_instrument()

```
uint16_t util::xmsg::XMessage::id_instrument () const  [inline], [inherited]
```

Get the unique instrument identifier of this X-Message.

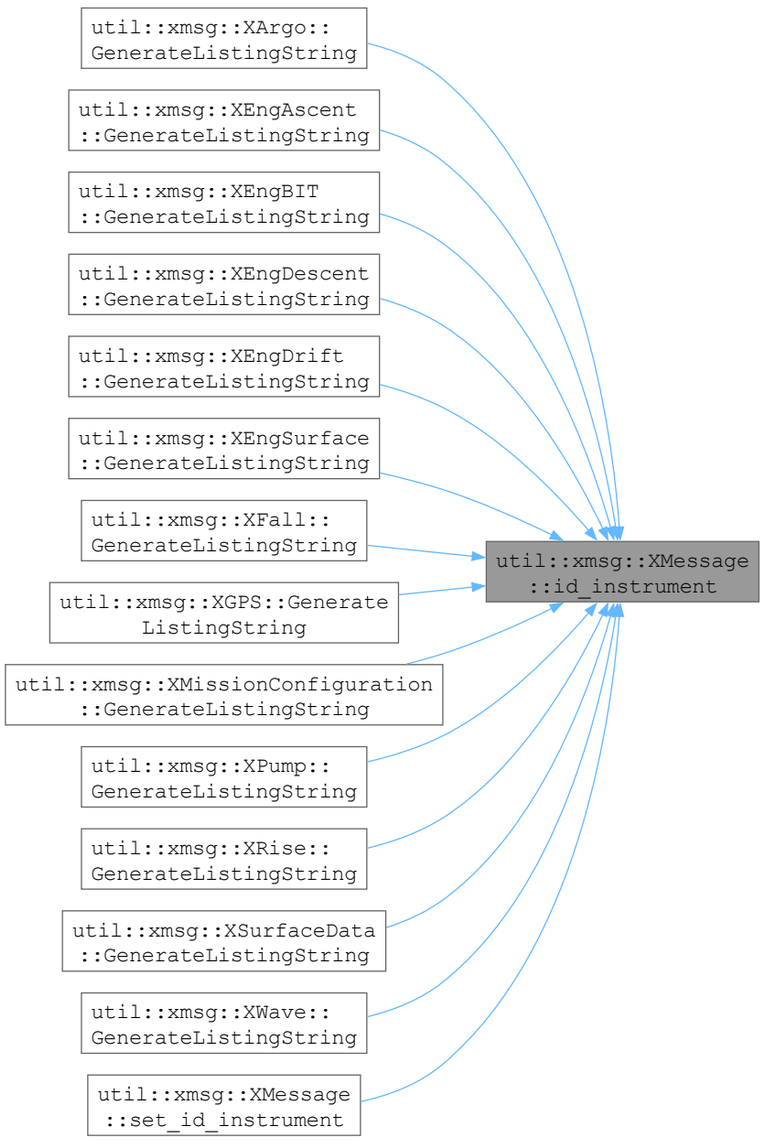
Returns

The unique instrument identifier of this X-Message.

References util::xmsg::XMessage::id_instrument_.

Referenced by util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngBIT::Generate
util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), util::xmsg::XEngSurface::GenerateListin
util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingS
util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(),
util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_instrument().

Here is the caller graph for this function:



10.99.3.5 id_xmsg_type()

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type () const  [inline], [inherited]
```

Get the enumerated X-Message type identifier.

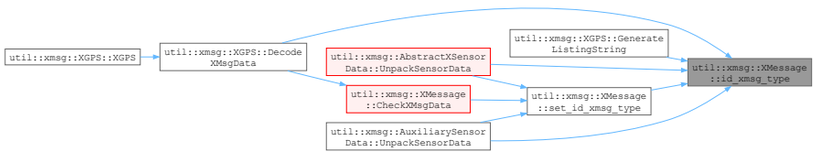
Returns

The enumerated X-Message type identifier.

References util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMessage::set_id_xmsg_typ
util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the caller graph for this function:



10.99.3.6 set_id_dive()

```
void util::xmsg::XMessage::set_id_dive (
            int16_t id_dive)  [inline], [protected], [inherited]
```

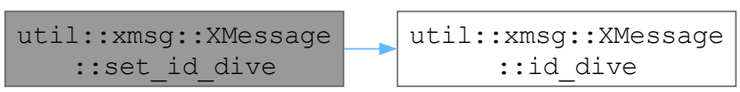
Set the dive or cycle number of this message.

Parameters

in	*id_dive*	The dive or cycle number of this message to set.

References util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::id_dive_.

Here is the call graph for this function:



10.99.3.7 set_id_instrument()

```
void util::xmsg::XMessage::set_id_instrument (
            uint16_t id_instrument)  [inline], [protected], [inherited]
```

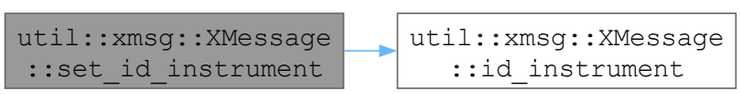
Set the unique instrument identifier.

Parameters

in	*id_instrument*	The unique instrument identifier to set.

References util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::id_instrument_.

Here is the call graph for this function:



10.99.3.8 set_id_xmsg_type()

```
void util::xmsg::XMessage::set_id_xmsg_type (
            XMsgTypeId_t id_xmsg_type)  [inline], [protected], [inherited]
```

Set the X-Message type identifier.

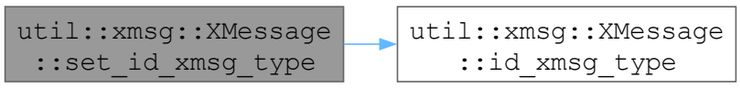
Parameters

in	*id_xmsg_type*	The X-Message type identifier to set.

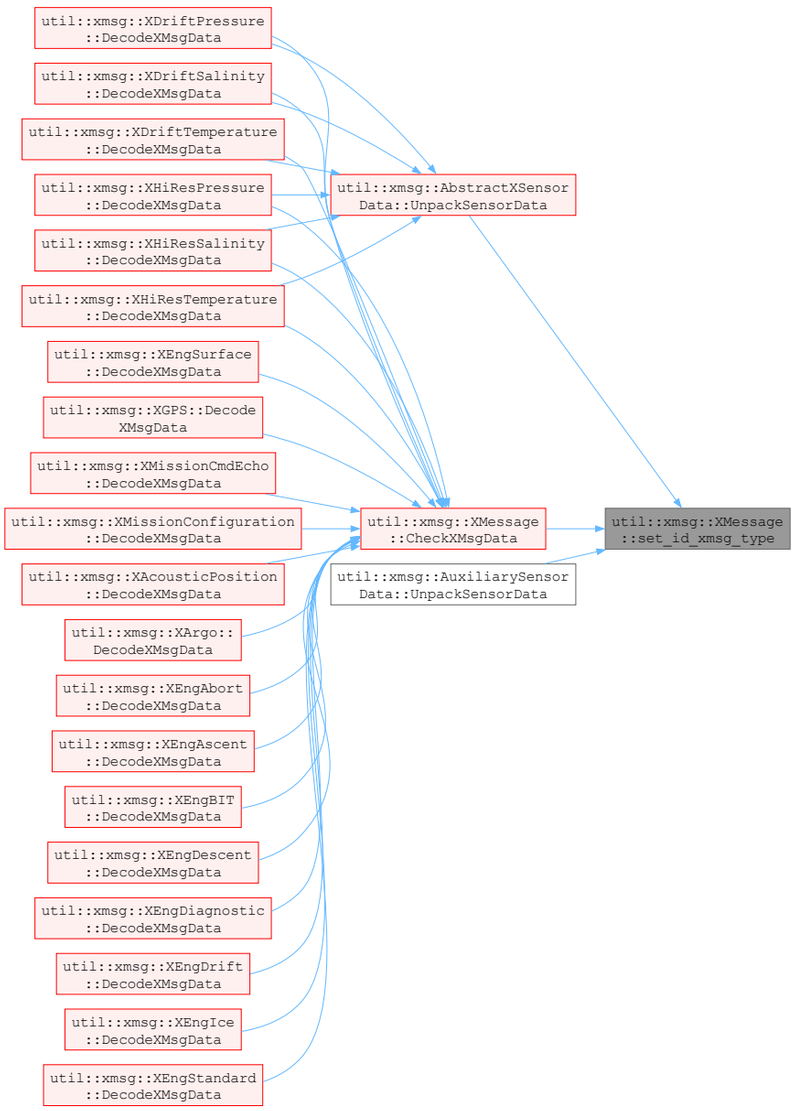
References util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.99.3.9 set_xmsg_index()

```
void util::xmsg::XMessageIndexable::set_xmsg_index (
            uint8_t xmsg_index)  [inline], [protected]
```

Set the index of this X-Message.

Parameters

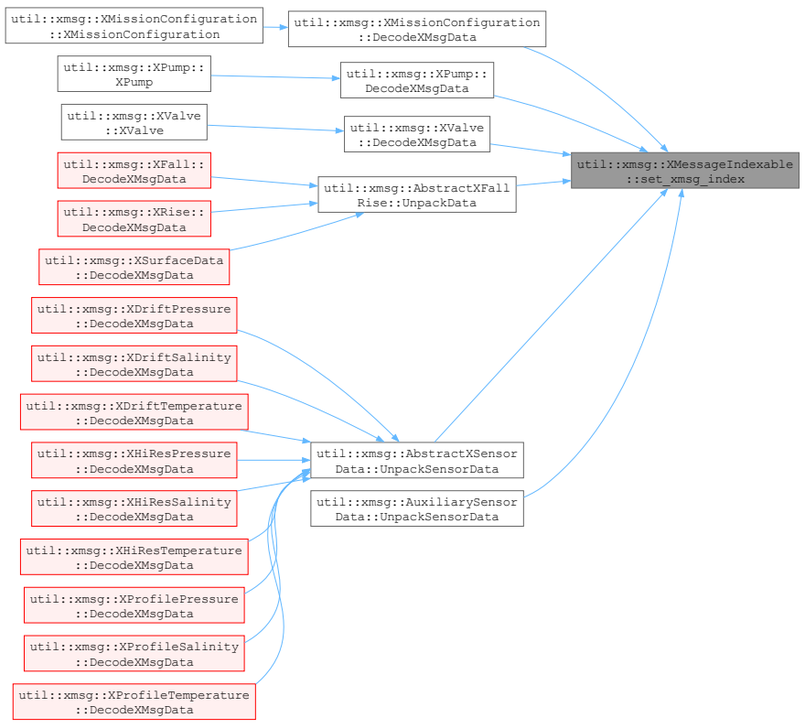
in	*xmsg_index*	The index of this X-Message.

References xmsg_index(), and xmsg_index_.

Referenced by util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(), util::xmsg::XValve::DecodeXMsgData(), util::xmsg::AbstractXFallRise::UnpackData(), util::xmsg::AbstractXSensorData::UnpackSensorDa and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:

Here is the caller graph for this function:



10.99.3.10 set_xmsg_size_bytes()

```
void util::xmsg::XMessage::set_xmsg_size_bytes (
            uint16_t xmsg_size_bytes)  [inline], [protected], [inherited]
```

Set the size of the X-Message, in bytes.

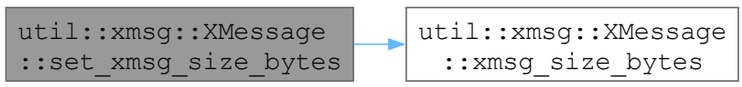
Parameters

in	*xmsg_size_bytes*	The size of the X-Message to set, in bytes.

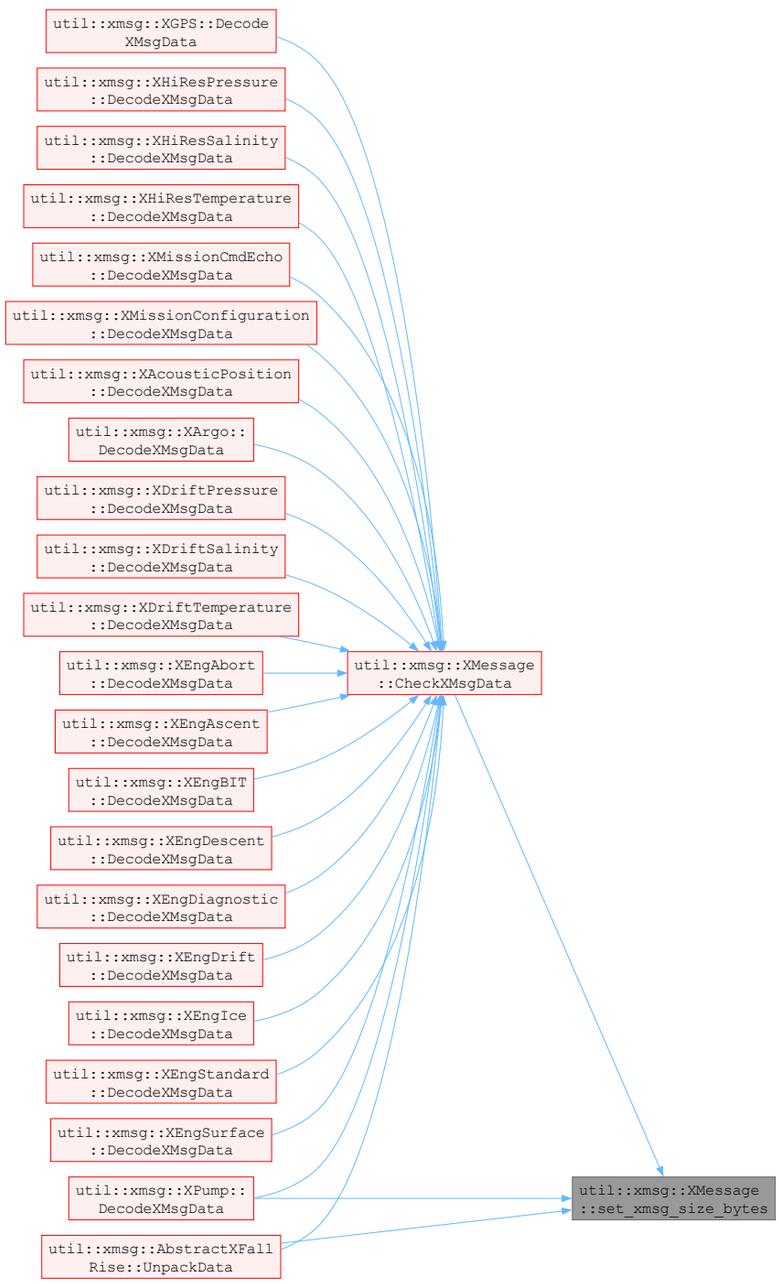
References util::xmsg::XMessage::xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XPump::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::Unpa

Here is the call graph for this function:



Here is the caller graph for this function:



10.99.3.11 xmsg_index()

```
uint8_t util::xmsg::XMessageIndexable::xmsg_index () const  [inline]
```

Get the index of this X-Message.

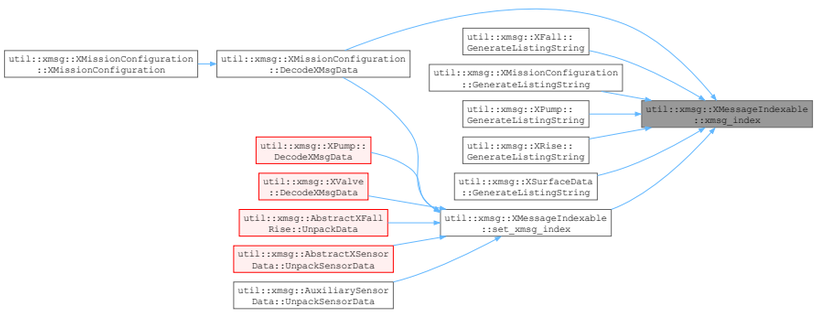
Returns

The index of this X-Message.

References xmsg_index_.

Referenced by util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing util::xmsg::XSurfaceData::GenerateListingString(), and set_xmsg_index().

Here is the caller graph for this function:



10.99.3.12 xmsg_size_bytes()

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes () const  [inline], [inherited]
```

Get the size of the X-Message, in bytes.

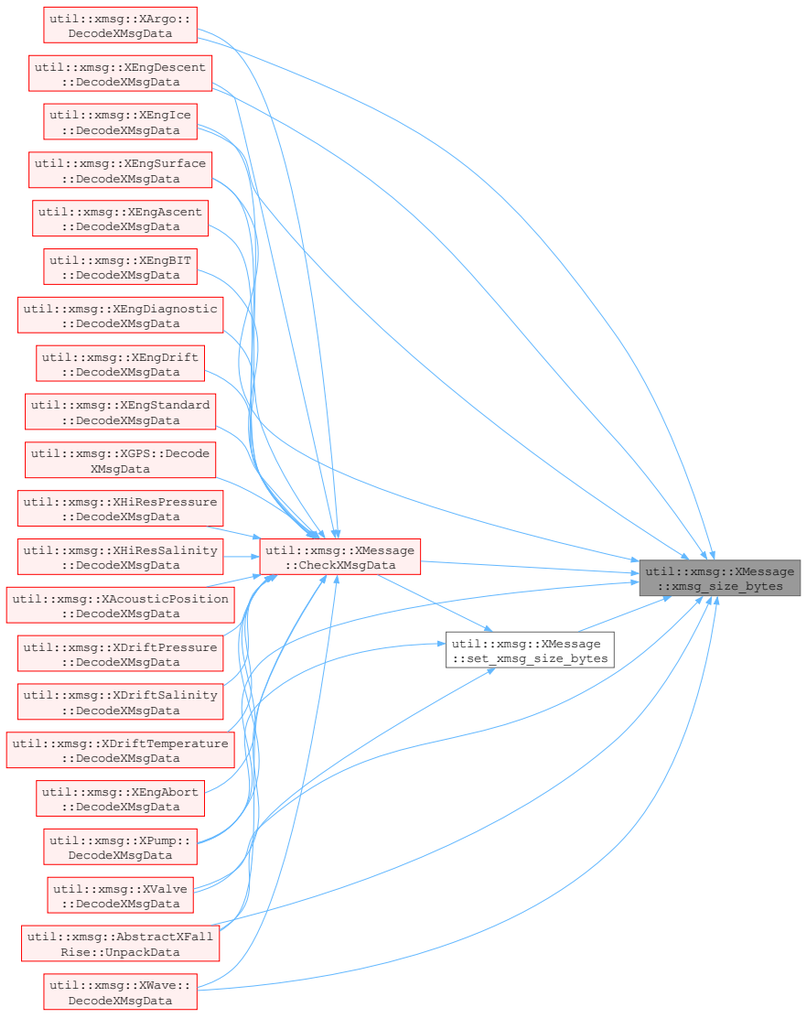
Returns

The size of the X-Message, in bytes.

References util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgI util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(), util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the caller graph for this function:



10.99.4 Member Data Documentation

10.99.4.1 id_dive_

```
int16_t util::xmsg::XMessage::id_dive_  [private], [inherited]
```

The dive or cycle number of this message.

Referenced by util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::set_id_dive().

10.99.4.2 id_instrument_

```
uint16_t util::xmsg::XMessage::id_instrument_  [private], [inherited]
```

The unique WHOI identifier of this instrument.

Referenced by util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::set_id_instrument().

10.99.4.3 id_xmsg_type_

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type_  [private], [inherited]
```

The X-message type identifier.

Referenced by util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::set_id_xmsg_type().

10.99.4.4 xmsg_index_

`uint8_t util::xmsg::XMessageIndexable::xmsg_index_ [private]`

The index of this X-Message.

Referenced by set_xmsg_index(), and xmsg_index().

10.99.4.5 xmsg_size_bytes_

`uint16_t util::xmsg::XMessage::xmsg_size_bytes_ [private], [inherited]`

The size of the X message, in bytes.

Referenced by util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes().

The documentation for this class was generated from the following file:

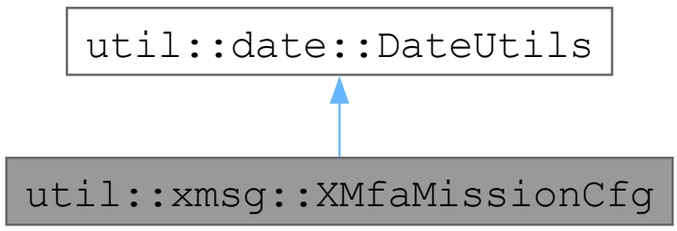
- data/xmsg/XMessageIndexable.h

10.100 util::xmsg::XMfaMissionCfg Class Reference

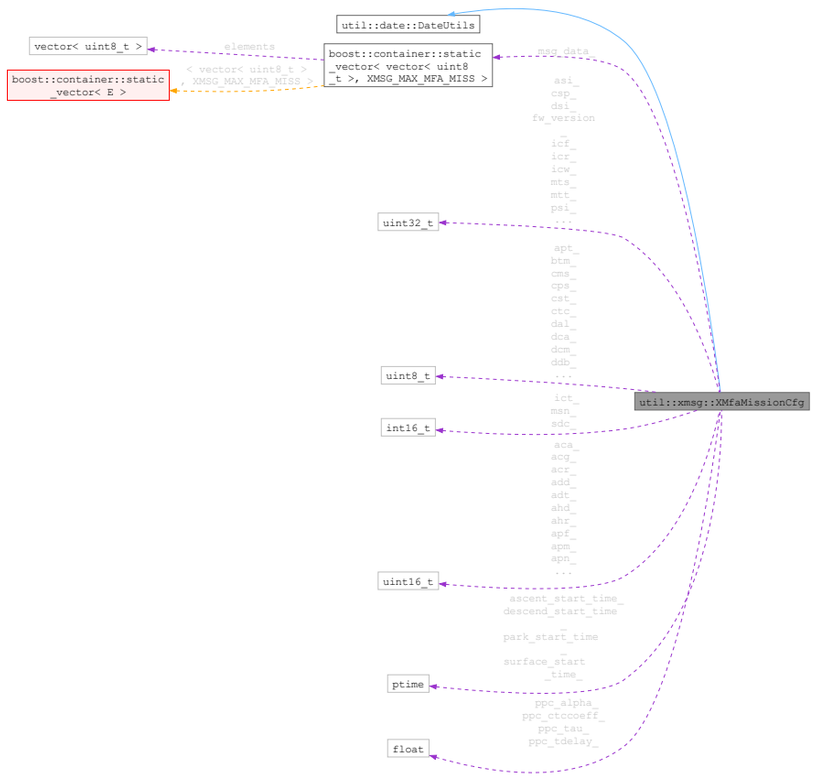
The ALAMO mission configuration (mission parameters) message class.

`#include <XMfaMissionCfg.h>`

Inheritance diagram for util::xmsg::XMfaMissionCfg:



Collaboration diagram for util::xmsg::XMfaMissionCfg:



Public Member Functions

- XMfaMissionCfg ()
  The default constructor.
- ~XMfaMissionCfg ()
  The destructor.
- unsigned int **aca** () const
- unsigned int **acg** () const
- unsigned int **acr** () const
- unsigned int **add** () const
- unsigned int **adt** () const
- unsigned int **ahd** () const
- unsigned int **ahr** () const
- float **ahr_mps** () const
- unsigned int **apf** () const
- unsigned int **apm** () const
- unsigned int **apn** () const
- unsigned int **aps** () const
- unsigned int **apt** () const
- unsigned int **arc** () const
- float **arc_sec** () const
- unsigned int **arr** () const
- float **arr_mps** () const
- const ptime & **ascent_start_time** () const
- unsigned int **asd** () const
- unsigned int **asi** () const
- float **asi_sec** () const
- unsigned int **asn** () const
- void AssignMessage (const vector< uint8_t > &data)
  Assign a mission configuration message.
- unsigned int **btm** () const
- unsigned int **cpb1** () const
- unsigned int **cpb2** () const
- unsigned int **cpb3** () const
- unsigned int **cpn1** () const
- float **cpn1_dbar** () const
- unsigned int **cpn2** () const
- float **cpn2_dbar** () const
- unsigned int **cpn3** () const
- float **cpn3_dbar** () const
- unsigned int **cpp1** () const
- float **cpp1_sec** () const
- unsigned int **cpp2** () const
- float **cpp2_sec** () const
- unsigned int **cpp3** () const
- float **cpp3_sec** () const
- unsigned int **cps** () const
- unsigned int **crd** () const
- unsigned int **crs** () const
- unsigned int **csp** () const
- float **csp_sec** () const
- unsigned int **cst** () const
- unsigned int **ctc** () const
- unsigned int **ctf** () const
- unsigned int **ctl** () const
- unsigned int **ctu** () const
- unsigned int **dal** () const
- unsigned int **dca** () const
- unsigned int **dcd** () const
- unsigned int **dcf** () const
- unsigned int **dcm** () const
- unsigned int **dcp** () const
- unsigned int **dcr** () const
- unsigned int **dct** () const
- unsigned int **dct0** () const
- unsigned int **dct1** () const
- unsigned int **dct2** () const

- unsigned int **dcx** () const
- unsigned int **ddb** () const
- unsigned int **ddc** () const
- unsigned int **ddd** () const
- unsigned int **ddf** () const
- unsigned int **ddp** () const
- unsigned int **dds** () const
- unsigned int **ddt** () const
- void DecodeXMsgData (uint32_t fw_version, uint8_t msg_idx, const vector< uint8_t > &data)

  Decode the mission configuration message data.
- const ptime & **descend_start_time** () const
- unsigned int **dft** () const
- unsigned int **dft0** () const
- unsigned int **dft1** () const
- unsigned int **dft2** () const
- unsigned int **dns** () const
- unsigned int **dpd** () const
- unsigned int **dpt** () const
- unsigned int **dra** () const
- float **dra_mps** () const
- unsigned int **drc** () const
- unsigned int **drt** () const
- unsigned int **dsd** () const
- unsigned int **dsd0** () const
- unsigned int **dsd1** () const
- unsigned int **dsd2** () const
- unsigned int **dsi** () const
- float **dsi_sec** () const
- unsigned int **dst** () const
- unsigned int **dst0** () const
- unsigned int **dst1** () const
- unsigned int **dst2** () const
- unsigned int **dsw** () const
- unsigned int **dtd** () const
- unsigned int **dtd0** () const
- unsigned int **dtd1** () const
- unsigned int **dtd2** () const
- unsigned int **duc** () const
- unsigned int **dut** () const
- unsigned int **dvd** () const
- unsigned int **dvt** () const
- unsigned int **dvw** () const
- unsigned int **fos** () const
- unsigned int **fta** () const
- unsigned int **ftm** () const
- unsigned int **fto** () const
- unsigned int **fw_version** () const
- const string GenerateListingString (int whoi, int dive, int idx) const

  Generate the mission configuration message listing string.
- unsigned int **iat** () const
- unsigned int **ica0** () const
- unsigned int **ica1** () const
- unsigned int **icb0** () const
- unsigned int **icb1** () const
- unsigned int **icd0** () const
- unsigned int **icd1** () const
- unsigned int **icf** () const
- unsigned int **icg0** () const
- unsigned int **icg1** () const
- unsigned int **icm** () const
- unsigned int **icn** () const
- unsigned int **ico0** () const
- unsigned int **ico1** () const
- unsigned int **icr0** () const
- unsigned int **icr1** () const
- unsigned int **ics** () const
- int **ict** () const

- float **ict_degc** () const
- unsigned int **icw** () const
- unsigned int **icx** () const
- unsigned int **ilt0** () const
- unsigned int **ilt1** () const
- unsigned int **imm** () const
- unsigned int **imt0** () const
- unsigned int **imt1** () const
- unsigned int **isa** () const
- unsigned int **isd** () const
- unsigned int **ism** () const
- unsigned int **iso** () const
- unsigned int **isp** () const
- unsigned int **iss0** () const
- unsigned int **iss1** () const
- unsigned int **ist** () const
- float **ist_psu** () const
- unsigned int **isv** () const
- unsigned int **isw** () const
- unsigned int **iue0** () const
- unsigned int **iue1** () const
- unsigned int **mam** () const
- unsigned int **map** () const
- unsigned int **mat** () const
- unsigned int **mat1** () const
- unsigned int **mat2** () const
- unsigned int **mds** () const
- unsigned int **mnm0** () const
- unsigned int **mnm1** () const
- unsigned int **mnm2** () const
- unsigned int **mns0** () const
- unsigned int **mns1** () const
- unsigned int **mns2** () const
- unsigned int **mpc** () const
- unsigned int **mps** () const
- unsigned int **msg_version** () const
- int **msn** () const
- unsigned int **mtd** () const
- unsigned int **mts** () const
- unsigned int **mtt** () const
- float **mtt_sec** () const
- const ptime & **park_start_time** () const
- unsigned int **pbh** () const
- unsigned int **pbl** () const
- unsigned int **pc0** () const
- unsigned int **pc1** () const
- unsigned int **pdt** () const
- unsigned int **pfc** () const
- unsigned int **pfd** () const
- unsigned int **pfd0** () const
- unsigned int **pfd1** () const
- unsigned int **pfd2** () const
- unsigned int **pfm** () const
- unsigned int **pfm0** () const
- unsigned int **pfm1** () const
- unsigned int **pfm2** () const
- unsigned int **pfr** () const
- unsigned int **pft** () const
- unsigned int **pft0** () const
- unsigned int **pft1** () const
- unsigned int **pft2** () const
- unsigned int **pfv** () const
- unsigned int **pfv0** () const
- unsigned int **pfv1** () const
- unsigned int **pfv2** () const
- unsigned int **pnb** () const
- float **pnb_v** () const
- unsigned int **pnc** () const

- float **pnc_psi** () const
- unsigned int **pns** () const
- unsigned int **pp0** () const
- unsigned int **pp1** () const
- float **ppc_alpha** () const
- float **ppc_ctccoeff** () const
- float **ppc_tau** () const
- float **ppc_tdelay** () const
- uint8_t **ppe** () const
- unsigned int **psa** () const
- unsigned int **psd** () const
- unsigned int **psi** () const
- float **psi_sec** () const
- unsigned int **psm** () const
- unsigned int **pss** () const
- unsigned int **pst** () const
- unsigned int **psv** () const
- unsigned int **psw** () const
- unsigned int **ptd** () const
- unsigned int **pth** () const
- unsigned int **pth0** () const
- unsigned int **pth1** () const
- unsigned int **pth2** () const
- unsigned int **pto** () const
- unsigned int **pto0** () const
- unsigned int **pto1** () const
- unsigned int **pto2** () const
- unsigned int **pv0** () const
- unsigned int **pv5** () const
- unsigned int **pvh** () const
- unsigned int **pvl** () const
- unsigned int **pxc** () const
- float **pxc_psi** () const
- unsigned int **sam** () const
- unsigned int **san** () const
- float **san_psi** () const
- unsigned int **sao** () const
- unsigned int **sap** () const
- unsigned int **sax** () const
- float **sax_psi** () const
- unsigned int **sbm** () const
- unsigned int **sbt** () const
- int **sdc** () const
- unsigned int **sds** () const
- unsigned int **sdt** () const
- unsigned int **sensor_type** () const
- unsigned int **set** () const
- unsigned int **sfn** () const
- unsigned int **sgt** () const
- unsigned int **spt** () const
- unsigned int **ssa** () const
- unsigned int **ssi** () const
- float **ssi_sec** () const
- unsigned int **ssp** () const
- const ptime & **surface_start_time** () const
- unsigned int **wmt** () const
- unsigned int **xas** () const

Static Public Member Functions

- static string GenerateCurrentUtcTimeIso8601String ()

  Generate an ISO8601 format date/time string representation of the current time, in UTC.
- static string GenerateIso8601String (const ptime &p, bool extended=true)

  Generate an ISO8601 format date/time string representation of a POSIX time object.
- static double GenerateJulianDay (const ptime &dt, const ptime &ref_dt)

  Convert a date/time to fractional days since a reference date/time.
- static ptime GetCurrentUtcTime ()

  Generate a POSIX time object representing the current date and time (rounded to the nearest second).

Private Member Functions

- void **set_aca** (uint16_t aca)
- void **set_acg** (uint16_t acg)
- void **set_acr** (uint16_t acr)
- void **set_add** (uint16_t add)
- void **set_adt** (uint16_t adt)
- void **set_ahd** (uint16_t ahd)
- void **set_ahr** (uint16_t ahr)
- void **set_apf** (uint16_t apf)
- void **set_apm** (uint16_t apm)
- void **set_apn** (uint16_t apn)
- void **set_aps** (uint16_t aps)
- void **set_apt** (uint8_t apt)
- void **set_arc** (uint16_t arc)
- void **set_arr** (uint16_t arr)
- void **set_ascent_start_time** (const ptime &ascent_start_time)
- void **set_asd** (uint16_t asd)
- void **set_asi** (uint32_t asi)
- void **set_asn** (uint16_t asn)
- void **set_btm** (uint8_t btm)
- void **set_cpb1** (uint16_t cpb1)
- void **set_cpb2** (uint16_t cpb2)
- void **set_cpb3** (uint16_t cpb3)
- void **set_cpn1** (uint16_t cpn1)
- void **set_cpn2** (uint16_t cpn2)
- void **set_cpn3** (uint16_t cpn3)
- void **set_cpp1** (uint16_t cpp1)
- void **set_cpp2** (uint16_t cpp2)
- void **set_cpp3** (uint16_t cpp3)
- void **set_cps** (uint8_t cps)
- void **set_crd** (uint16_t crd)
- void **set_crs** (uint16_t crs)
- void **set_csp** (uint32_t csp)
- void **set_cst** (uint8_t cst)
- void **set_ctc** (uint8_t ctc)
- void **set_ctf** (uint16_t ctf)
- void **set_ctl** (uint16_t ctl)
- void **set_ctu** (uint16_t ctu)
- void **set_dal** (uint8_t dal)
- void **set_dca** (uint8_t dca)
- void **set_dcd** (uint16_t dcd)
- void **set_dcf** (uint16_t dcf)
- void **set_dcm** (uint8_t dcm)
- void **set_dcp** (uint16_t dcp)
- void **set_dcr** (uint16_t dcr)
- void **set_dct** (uint16_t dct)
- void **set_dct0** (uint16_t dct)
- void **set_dct1** (uint16_t dct)
- void **set_dct2** (uint16_t dct)
- void **set_dcx** (uint16_t dcx)
- void **set_ddb** (uint8_t ddb)
- void **set_ddc** (uint16_t ddc)
- void **set_ddd** (uint16_t ddd)
- void **set_ddf** (uint16_t ddf)
- void **set_ddp** (uint16_t ddp)
- void **set_dds** (uint16_t dds)
- void **set_ddt** (uint16_t ddt)
- void **set_descend_start_time** (const ptime &descend_start_time)
- void **set_dft** (uint16_t dft)
- void **set_dft0** (uint16_t dft)
- void **set_dft1** (uint16_t dft)
- void **set_dft2** (uint16_t dft)
- void **set_dns** (uint16_t dns)
- void **set_dpd** (uint16_t dpd)
- void **set_dpt** (uint16_t dpt)
- void **set_dra** (uint8_t dra)
- void **set_drc** (uint16_t drc)

- void **set_drt** (uint16_t drt)
- void **set_dsd** (uint16_t dsd)
- void **set_dsd0** (uint16_t dsd)
- void **set_dsd1** (uint16_t dsd)
- void **set_dsd2** (uint16_t dsd)
- void **set_dsi** (uint32_t dsi)
- void **set_dst** (uint16_t dst)
- void **set_dst0** (uint16_t dst)
- void **set_dst1** (uint16_t dst)
- void **set_dst2** (uint16_t dst)
- void **set_dsw** (uint16_t dsw)
- void **set_dtd** (uint16_t dtd)
- void **set_dtd0** (uint16_t dtd)
- void **set_dtd1** (uint16_t dtd)
- void **set_dtd2** (uint16_t dtd)
- void **set_duc** (uint16_t duc)
- void **set_dut** (uint16_t dut)
- void **set_dvd** (uint16_t dvd)
- void **set_dvt** (uint16_t dvt)
- void **set_dvw** (uint16_t dvw)
- void **set_fos** (uint16_t fos)
- void **set_fta** (uint8_t fta)
- void **set_ftm** (uint8_t ftm)
- void **set_fto** (uint16_t fto)
- void **set_fw_version** (uint32_t fw_version)
- void **set_iat** (uint16_t iat)
- void **set_ica0** (uint16_t ica0)
- void **set_ica1** (uint16_t ica1)
- void **set_icb0** (uint16_t icb0)
- void **set_icb1** (uint16_t icb1)
- void **set_icd0** (uint16_t icd0)
- void **set_icd1** (uint16_t icd1)
- void **set_icf** (uint32_t icf)
- void **set_icg0** (uint16_t icg0)
- void **set_icg1** (uint16_t icg1)
- void **set_icm** (uint8_t icm)
- void **set_icn** (uint16_t icn)
- void **set_ico0** (uint16_t ico0)
- void **set_ico1** (uint16_t ico1)
- void **set_icr0** (uint16_t icr0)
- void **set_icr1** (uint16_t icr1)
- void **set_ics** (uint16_t ics)
- void **set_ict** (int16_t ict)
- void **set_icw** (uint32_t icw)
- void **set_icx** (uint16_t icx)
- void **set_ilt0** (uint16_t ilt0)
- void **set_ilt1** (uint16_t ilt1)
- void **set_imm** (uint16_t imm)
- void **set_imt0** (uint16_t imt0)
- void **set_imt1** (uint16_t imt1)
- void **set_isa** (uint8_t isa)
- void **set_isd** (uint16_t isd)
- void **set_ism** (uint8_t ism)
- void **set_iso** (uint16_t iso)
- void **set_isp** (uint8_t isp)
- void **set_iss0** (uint16_t iss0)
- void **set_iss1** (uint16_t iss1)
- void **set_ist** (uint16_t ist)
- void **set_isv** (uint16_t isv)
- void **set_isw** (uint16_t isw)
- void **set_iue0** (uint16_t iue0)
- void **set_iue1** (uint16_t iue1)
- void **set_mam** (uint8_t mam)
- void **set_map** (uint16_t map)
- void **set_mat** (uint16_t mat)
- void **set_mat1** (uint16_t mat)
- void **set_mat2** (uint16_t mat)
- void **set_mds** (uint16_t mds)

- void **set_mnm0** (uint16_t mnm)
- void **set_mnm1** (uint16_t mnm)
- void **set_mnm2** (uint16_t mnm)
- void **set_mns0** (uint8_t mns)
- void **set_mns1** (uint8_t mns)
- void **set_mns2** (uint8_t mns)
- void **set_mpc** (uint16_t mpc)
- void **set_mps** (uint8_t mps)
- void **set_msg_version** (uint8_t msg_version)
- void **set_msn** (int16_t msn)
- void **set_mtd** (uint16_t mtd)
- void **set_mts** (uint32_t mts)
- void **set_mtt** (uint32_t mtt)
- void **set_park_start_time** (const ptime &park_start_time)
- void **set_pbh** (uint16_t pbh)
- void **set_pbl** (uint16_t pbl)
- void **set_pc0** (uint16_t pc0)
- void **set_pc1** (uint16_t pc1)
- void **set_pdt** (uint16_t pdt)
- void **set_pfc** (uint16_t pfc)
- void **set_pfd** (uint16_t pfd)
- void **set_pfd0** (uint16_t pfd)
- void **set_pfd1** (uint16_t pfd)
- void **set_pfd2** (uint16_t pfd)
- void **set_pfm** (uint8_t pfm)
- void **set_pfm0** (uint8_t pfm)
- void **set_pfm1** (uint8_t pfm)
- void **set_pfm2** (uint8_t pfm)
- void **set_pfr** (uint16_t pfr)
- void **set_pft** (uint16_t pft)
- void **set_pft0** (uint16_t pft)
- void **set_pft1** (uint16_t pft)
- void **set_pft2** (uint16_t pft)
- void **set_pfv** (uint16_t pfv)
- void **set_pfv0** (uint16_t pfv)
- void **set_pfv1** (uint16_t pfv)
- void **set_pfv2** (uint16_t pfv)
- void **set_pnb** (uint16_t pnb)
- void **set_pnc** (uint16_t pnc)
- void **set_pns** (uint16_t pns)
- void **set_pp0** (uint16_t pp0)
- void **set_pp1** (uint16_t pp1)
- void **set_ppc_alpha** (float ppc_alpha)
- void **set_ppc_ctccoeff** (float ppc_ctccoeff)
- void **set_ppc_tau** (float ppc_tau)
- void **set_ppc_tdelay** (float ppc_tdelay)
- void **set_ppe** (uint8_t ppe)
- void **set_psa** (uint8_t psa)
- void **set_psd** (uint16_t psd)
- void **set_psi** (uint32_t psi)
- void **set_psm** (uint8_t psm)
- void **set_pss** (uint8_t pss)
- void **set_pst** (uint16_t pst)
- void **set_psv** (uint16_t psv)
- void **set_psw** (uint16_t psw)
- void **set_ptd** (uint32_t ptd)
- void **set_pth** (uint32_t pth)
- void **set_pth0** (uint32_t pth)
- void **set_pth1** (uint32_t pth)
- void **set_pth2** (uint32_t pth)
- void **set_pto** (uint32_t pto)
- void **set_pto0** (uint32_t pto)
- void **set_pto1** (uint32_t pto)
- void **set_pto2** (uint32_t pto)
- void **set_pv0** (uint16_t pv0)
- void **set_pv5** (uint16_t pv5)
- void **set_pvh** (uint16_t pvh)
- void **set_pvl** (uint16_t pvl)

- void **set_pxc** (uint16_t pxc)
- void **set_sam** (uint8_t sam)
- void **set_san** (uint16_t san)
- void **set_sao** (uint16_t sao)
- void **set_sap** (uint16_t sap)
- void **set_sax** (uint16_t sax)
- void **set_sbm** (uint8_t sbm)
- void **set_sbt** (uint16_t sbt)
- void **set_sdc** (int16_t sdc)
- void **set_sds** (uint16_t sds)
- void **set_sdt** (uint32_t sdt)
- void **set_sensor_type** (uint8_t sensor_type)
- void **set_set** (uint16_t set)
- void **set_sfn** (uint16_t sfn)
- void **set_sgt** (uint16_t sgt)
- void **set_spt** (uint16_t spt)
- void **set_ssa** (uint16_t ssa)
- void **set_ssi** (uint32_t ssi)
- void **set_ssp** (uint16_t ssp)
- void **set_surface_start_time** (const ptime &surface_start_time)
- void **set_wmt** (uint16_t wmt)
- void **set_xas** (uint8_t xas)

Private Attributes

- uint16_t aca_

  The motion processing unit (MPU) accelerometer sensitivity scale factor identifier.
- uint16_t acg_

  The motion processing unit (MPU) gyroscope sensitivity scale factor identifier.
- uint16_t acr_

  The motion processing unit (MPU) sample rate, in Hz.
- uint16_t add_

  The ascent state data storage pressure interval, in dBar.
- uint16_t adt_

  The ascent state data storage interval timeout, in seconds.
- uint16_t **ahd_**
- uint16_t **ahr_**
- uint16_t apf_

  The ascent state pump off duration/interval/timeout, in seconds.
- uint16_t **apm_**
- uint16_t apn_

  The ascent state pump on duration/interval/timeout, in seconds.
- uint16_t **aps_**
- uint8_t apt_

  The auto-transmit mission configuration message flag.
- uint16_t **arc_**
- uint16_t arr_

  The ascent state rise rate, in centimeters per second.
- ptime ascent_start_time_

  The start time of the ascent state of the dive cycle.
- uint16_t asd_

  The ascent state surface threshold pressure, in dBar.
- uint32_t asi_

  The ascent state sampling interval/timeout, in milliseconds.
- uint16_t asn_

  The ascent number.
- uint8_t btm_

  The bluetooth mode.
- uint8_t cms_

  The CTD profiling mode.
- uint16_t cpb_ [3]

  The CTD sampling period pressure boundaries, in dBar.
- uint16_t cpn_ [3]

  The CTD sampling period bin sizes, in 0.1 dBar.

- uint16_t cpp_ [3]

  The CTD sampling period interval, in milliseconds.
- uint8_t cps_

  The CTD profiling mode.
- uint16_t crd_

  The CTD raw sampling threshold, in dBar.
- uint16_t crs_

  The maximum number of raw samples to collect above the raw sampling threshold.
- uint32_t csp_

  The CTD continuous sampling period, in milliseconds.
- uint8_t cst_

  The CTD sampling type.
- uint8_t ctc_

  The auxiliary temperature channel mode.
- uint16_t ctf_

  The profile cutoff pressure.
- uint16_t ctl_

  The pressure sensor offset reset lower pressure limit.
- uint16_t ctu_

  The pressure sensor offset reset lower pressure limit.
- uint8_t **dal_**
- uint8_t **dca_**
- uint16_t **dcd_**
- uint16_t **dcf_**
- uint8_t **dcm_**
- uint16_t **dcp_**
- uint16_t **dcr_**
- uint16_t **dct_** [3]
- uint16_t **dcx_**
- uint8_t ddb_

  Diagnostic dive bypass mode.
- uint16_t **ddc_**
- uint16_t ddd_

  The descent state data storage pressure interval, in dBar.
- uint16_t **ddf_**
- uint16_t **ddp_**
- uint16_t **dds_**
- uint16_t ddt_

  The descent state data storage interval, in seconds.
- ptime descend_start_time_

  The start time of the descend state of the dive cycle.
- uint16_t dft_ [3]

  The descent state fall timeout, in seconds.
- uint16_t dns_

  The descent number.
- uint16_t **dpd_**
- uint16_t **dpt_**
- uint8_t **dra_**
- uint16_t **drc_**
- uint16_t **drt_**
- uint16_t dsd_ [3]

  The descent state start pressure, in dBar.
- uint32_t dsi_

  The descent state sampling interval, in milliseconds.
- uint16_t dst_ [3]

  The descent state start timeout, in seconds.
- uint16_t **dsw_**
- uint16_t dtd_ [3]

  The target descent/drift/park pressure, in dBar.
- uint16_t **duc_**
- uint16_t **dut_**
- uint16_t **dvd_**
- uint16_t **dvt_**
- uint16_t **dvw_**

- uint16_t **fos_**
- uint8_t **fta_**
- uint8_t **ftm_**
- uint16_t **fto_**
- uint32_t fw_version_

  The software version identifier.
- uint16_t iat_

  Ice-detection – .
- uint16_t ica_ [2]

  Ice-detection – surface attempts.
- uint16_t icb_ [2]

  Ice-detection – cycles between surface attempts.
- uint16_t icd_ [2]

  Ice-detection – hold pressure, in dBar.
- uint32_t icf_

  Ice-detection – ice-free month mask.
- uint8_t icg_ [2]

  Ice-detection – seek mode.
- uint8_t icm_

  Ice-detection mode.
- uint16_t icn_

  Ice-detection – temperature threshold test upper pressure boundary, in dBar.
- uint16_t ico_ [2]

  Ice-detection – ice-detection hold-pressure duration, in minutes.
- uint32_t icr_ [2]

  Ice-detection – .
- uint16_t ics_

  Ice-detection – temperature threshold test sample interval, in seconds.
- int16_t ict_

  Ice-detection – temperature threshold test temperature, in 1/100 degC.
- uint32_t icw_

  Ice-detection – iced-over month mask.
- uint16_t icx_

  Ice-detection – temperature threshold test lower pressure boundary, in dBar.
- uint16_t ilt_ [2]

  Ice-detection – duration since last successful comms urgency criteria, in days.
- uint16_t imm_

  Ice-detection – month mask.
- uint16_t imt_ [2]

  Ice-detection – percentage used storage urgency criteria.
- uint8_t isa_

  Ice-detection – .
- uint16_t isd_

  Ice-detection – .
- uint8_t ism_

  Ice-detection – surfacing achieved criteria mode.
- uint16_t iso_

  Ice-detection – duration between multiple surfacing attempts, in seconds.
- uint8_t isp_

  Ice-detection – multiple surfacing profile mode.
- uint8_t iss_ [2]

  Ice-detection – multiple surface attempts mode.
- uint16_t ist_

  Ice-detection – surface achieved salinity threshold, in 1/10 PSU.
- uint16_t isv_

  Ice-detection – .
- uint16_t isw_

  Ice-detection – .
- uint8_t iue_ [2]

  Ice-detection – urgency mode exit criteria.
- uint8_t mam_

  The motion processing unit (MPU) data acquisition mode mask.
- uint16_t **map_**

- uint16_t mat_ [2]

  The motion processing unit (MPU) sampling duration, in seconds.
- uint16_t mds_

  The maximum descent/fall/profile pressure, in dBar.
- uint16_t mnm_ [3]

  Number of cycles for mission phase.
- uint8_t mns_ [3]

  Next mission phase.
- uint16_t mpc_

  Cycle count for current mission phase.
- uint8_t mps_

  Current mission phase.
- static_vector< vector< uint8_t >, XMSG_MAX_MFA_MISS > msg_data_

  The mission configuration messages' raw data.
- uint8_t msg_version_

  The message version identifier.
- int16_t msn_

  The mission, dive, or cycle number for this mission configuration.
- uint16_t mtd_

  The initial trigger depth, in dBar.
- uint32_t **mts_**
- uint32_t mtt_

  The initial trigger timeout/interval, in milliseconds.
- ptime park_start_time_

  The start time of the park/drift state of the dive cycle.
- uint16_t **pbh_**
- uint16_t **pbl_**
- uint16_t **pc0_**
- uint16_t **pc1_**
- uint16_t pdt_

  The park/drift state data storage interval/timeout, in seconds.
- uint16_t **pfc_**
- uint16_t pfd_ [3]

  The park/drift state fall depth (i.e.
- uint8_t pfm_ [3]

  The park/drift state fall mode.
- uint16_t **pfr_**
- uint16_t **pft_** [3]
- uint16_t **pfv_** [3]
- uint16_t **pnb_**
- uint16_t **pnc_**
- uint16_t pns_

  The park/drift number.
- uint16_t **pp0_**
- uint16_t **pp1_**
- float **ppc_alpha_**
- float **ppc_ctccoeff_**
- float **ppc_tau_**
- float **ppc_tdelay_**
- uint8_t **ppe_**
- uint8_t psa_

  The park/drift state sampling mode.
- uint16_t **psd_**
- uint32_t psi_

  The park/drift state sample interval/timeout, in milliseconds.
- uint8_t **psm_**
- uint8_t **pss_**
- uint16_t **pst_**
- uint16_t **psv_**
- uint16_t **psw_**
- uint32_t **ptd_**
- uint32_t pth_ [3]

  The bit-encoded park/drift state hour timeout trigger(s).
- uint32_t pto_ [3]

The park/drift state timeout, in seconds.

- uint16_t **pv0_**
- uint16_t **pv5_**
- uint16_t **pvh_**
- uint16_t **pvl_**
- uint16_t **pxc_**
- uint8_t **sam_**
- uint16_t **san_**
- uint16_t **sao_**
- uint16_t **sap_**
- uint16_t **sax_**
- uint8_t sbm_

  The surface state beacon mode flag.
- uint16_t sbt_

  The surface state beacon interval timeout, in seconds.
- int16_t sdc_

  The surface state sensor depth compensation offset, in dBar.
- uint16_t sds_

  The surface state sampling storage duration/interval/timeout, in seconds.
- uint32_t **sdt_**
- uint8_t sensor_type_

  The sensor type identifier.
- uint16_t set_

  The surface state Iridium session timeout, in minutes.
- uint16_t sfn_

  The surface number.
- uint16_t sgt_

  The surface state GPS timeout, in minutes.
- uint16_t **spt_**
- uint16_t **ssa_**
- uint32_t ssi_

  The surface state sampling interval timeout, in milliseconds.
- uint16_t **ssp_**
- ptime surface_start_time_

  The start time of the surface state of the dive cycle.
- uint16_t wmt_

  WIMS timeout during surface state, in seconds.
- uint8_t xas_

  The profile data encode method.

10.100.1 Detailed Description

The ALAMO mission configuration (mission parameters) message class.

An instance of this class assembles the mission configuration X-messages transmitted by an instrument and decodes the mission parameters according to the instrument's firmware version.

10.100.2 Constructor & Destructor Documentation

10.100.2.1 XMfaMissionCfg()

```
util::xmsg::XMfaMissionCfg::XMfaMissionCfg ()
```

The default constructor.

References msg_data_, and XMSG_MAX_MFA_MISS.

10.100.2.2 ~XMfaMissionCfg()

```
util::xmsg::XMfaMissionCfg::~XMfaMissionCfg ()
```

The destructor.

10.100.3 Member Function Documentation

10.100.3.1 AssignMessage()

```
void util::xmsg::XMfaMissionCfg::AssignMessage (
            const vector< uint8_t > & data)
```

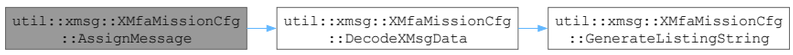
Assign a mission configuration message.

Parameters

in	*data*	The message.

References CATCH_BLOCK, DecodeXMsgData(), EXC_THROW, msg_data_, u32, and XMSG_MAX_MFA_MISS.

Here is the call graph for this function:



10.100.3.2 DecodeXMsgData()

```
void util::xmsg::XMfaMissionCfg::DecodeXMsgData (
            uint32_t fw_ver,
            uint8_t msg_idx,
            const vector< uint8_t > & data)
```

Decode the mission configuration message data.

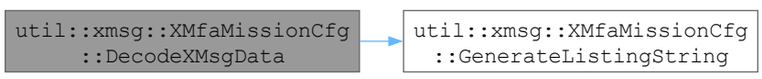
Parameters

in	*fw_ver*	The firmware version.
in	*msg_idx*	The message index.
in	*data*	The message data.

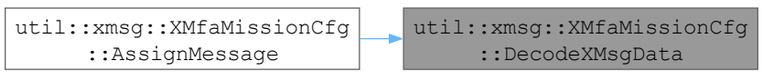
References CATCH_BLOCK, cpb_, cpn_, cpp_, EXC_THROW, GenerateListingString(), i16, ldbg, lwrn, s2000, u08, u16, u32, and XMSG_BAD_U16.

Referenced by AssignMessage().

Here is the call graph for this function:



Here is the caller graph for this function:



10.100.3.3 GenerateCurrentUtcTimeIso8601String()

```
static string util::date::DateUtils::GenerateCurrentUtcTimeIso8601String ()  [inline], [static],
[inherited]
```

Generate an ISO8601 format date/time string representation of the current time, in UTC.

Note

All date/time objects use the UTC time-zone.

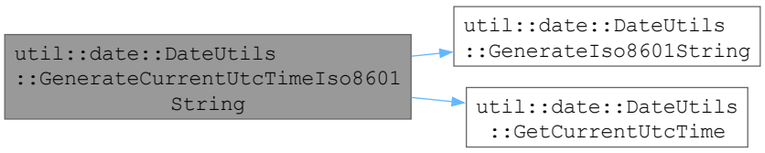
Returns

An ISO8601 format date/time string representation of the current time.

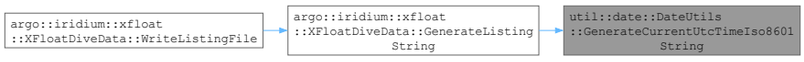
References util::date::DateUtils::GenerateIso8601String(), and util::date::DateUtils::GetCurrentUtcTime().

Referenced by argo::iridium::xfloat::XFloatDiveData::GenerateListingString().

Here is the call graph for this function:



Here is the caller graph for this function:



10.100.3.4 GenerateIso8601String()

```
static string util::date::DateUtils::GenerateIso8601String (
            const ptime & p,
            bool extended = true)  [inline], [static], [inherited]
```

Generate an ISO8601 format date/time string representation of a POSIX time object.

Note

All date/time objects use the UTC time-zone.

Parameters

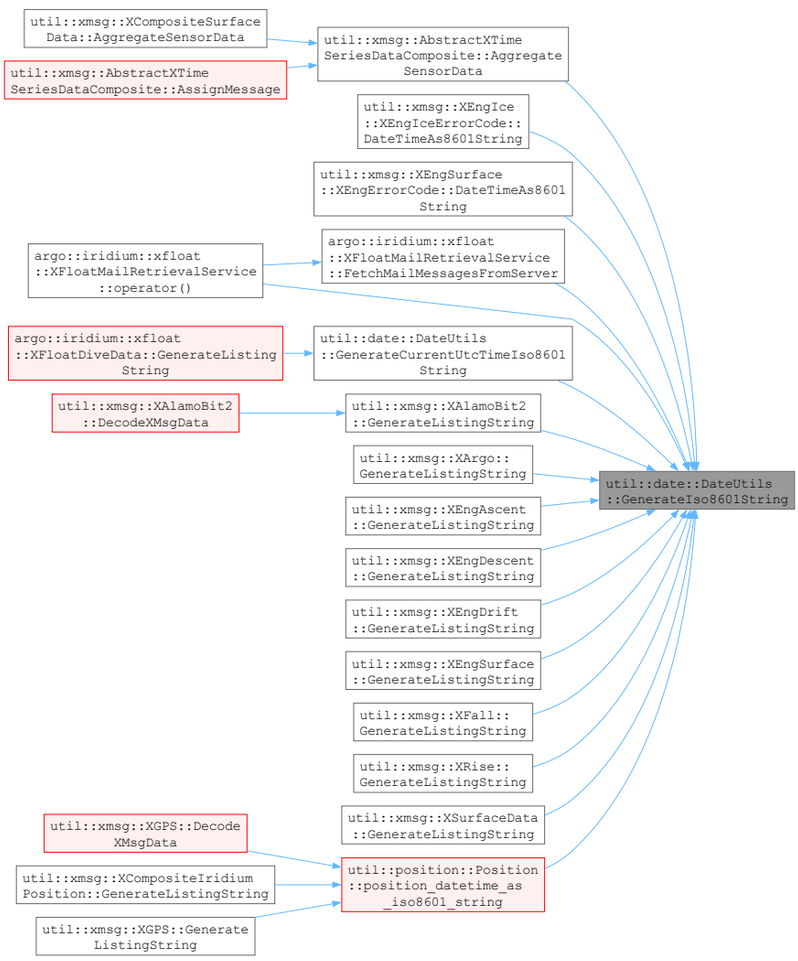
in	*p*	POSIX time from which to generate string.
in	*extended*	If TRUE, use the ISO 8601 extended format (YYYY-MM-DDThh:mm:ssZ); otherwise, use the basic format (YYYYMMDDThhmmss).

Returns

An ISO8601 format date/time string representation of the POSIX time object. If p is equal to boost::posix_↩ time::not_a_date_time, then the string "0000-00-00T00:00:00Z" (extended) or "00000000000000" (basic) will be returned.

Referenced by util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::AggregateSensorData(), util::xmsg::XEngIce::XEngIce
util::xmsg::XEngSurface::XEngErrorCode::DateTimeAs8601String(), argo::iridium::xfloat::XFloatMailRetrievalService::FetchMailMessages
util::date::DateUtils::GenerateCurrentUtcTimeIso8601String(), util::xmsg::XAlamoBit2::GenerateListingString(), util::xmsg::XArgo::Genera
util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListing
util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XRise::GenerateListingString(),
util::xmsg::XSurfaceData::GenerateListingString(), argo::iridium::xfloat::XFloatMailRetrievalService::operator()(), and
util::position::Position::position_datetime_as_iso8601_string().

Here is the caller graph for this function:



10.100.3.5 GenerateJulianDay()

```
static double util::date::DateUtils::GenerateJulianDay (
            const ptime & dt,
            const ptime & ref_dt)  [inline], [static], [inherited]
```

Convert a date/time to fractional days since a reference date/time.

Parameters

in	*dt*	The date/time to convert.
in	*ref↩ _dt*	The reference date/time (day 0).

Exceptions

util::log::MessageException	if either date/time is not valid.

Returns

The number of days (with millisecond resolution) from `ref_dt` to `dt`.

References CATCH_BLOCK, and EXC_THROW.

10.100.3.6 GenerateListingString()

```
const string util::xmsg::XMfaMissionCfg::GenerateListingString (
            int whoi,
            int dive,
            int idx) const
```

Generate the mission configuration message listing string.

Parameters

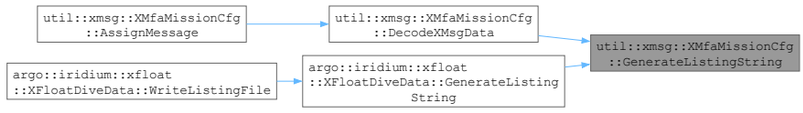
in	*whoi*	The instrument identifier.
in	*dive*	The dive cycle number
in	*idx*	

Returns

References CATCH_BLOCK, EXC_THROW, and lwrn.

Referenced by DecodeXMsgData(), and argo::iridium::xfloat::XFloatDiveData::GenerateListingString().

Here is the caller graph for this function:



10.100.3.7 GetCurrentUtcTime()

```
static ptime util::date::DateUtils::GetCurrentUtcTime ()  [inline], [static], [inherited]
```

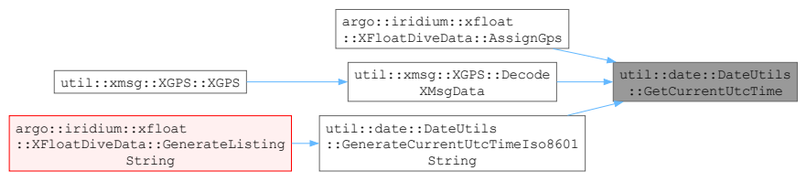
Generate a POSIX time object representing the current date and time (rounded to the nearest second).

Returns

A POSIX time object representing the current date and time (rounded to the nearest second).

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignGps(), util::xmsg::XGPS::DecodeXMsgData(), and util::date::DateUtils::GenerateCurrentUtcTimeIso8601String().

Here is the caller graph for this function:



10.100.4 Member Data Documentation

10.100.4.1 aca_

```
uint16_t util::xmsg::XMfaMissionCfg::aca_  [private]
```

The motion processing unit (MPU) accelerometer sensitivity scale factor identifier.

10.100.4.2 acg_

```
uint16_t util::xmsg::XMfaMissionCfg::acg_  [private]
```

The motion processing unit (MPU) gyroscope sensitivity scale factor identifier.

10.100.4.3 acr_

```
uint16_t util::xmsg::XMfaMissionCfg::acr_  [private]
```

The motion processing unit (MPU) sample rate, in Hz.

10.100.4.4 add_

```
uint16_t util::xmsg::XMfaMissionCfg::add_  [private]
```

The ascent state data storage pressure interval, in dBar.

10.100.4.5 adt_

```
uint16_t util::xmsg::XMfaMissionCfg::adt_  [private]
```

The ascent state data storage interval timeout, in seconds.

10.100.4.6 apf_

```
uint16_t util::xmsg::XMfaMissionCfg::apf_  [private]
```

The ascent state pump off duration/interval/timeout, in seconds.

10.100.4.7 apn_

```
uint16_t util::xmsg::XMfaMissionCfg::apn_  [private]
```

The ascent state pump on duration/interval/timeout, in seconds.

10.100.4.8 apt_

```
uint8_t util::xmsg::XMfaMissionCfg::apt_  [private]
```

The auto-transmit mission configuration message flag.

10.100.4.9 arr_

```
uint16_t util::xmsg::XMfaMissionCfg::arr_  [private]
```

The ascent state rise rate, in centimeters per second.

10.100.4.10 ascent_start_time_

```
ptime util::xmsg::XMfaMissionCfg::ascent_start_time_  [private]
```

The start time of the ascent state of the dive cycle.

10.100.4.11 asd_

```
uint16_t util::xmsg::XMfaMissionCfg::asd_  [private]
```

The ascent state surface threshold pressure, in dBar.

10.100.4.12 asi_

```
uint32_t util::xmsg::XMfaMissionCfg::asi_  [private]
```

The ascent state sampling interval/timeout, in milliseconds.

10.100.4.13 asn_

```
uint16_t util::xmsg::XMfaMissionCfg::asn_  [private]
```

The ascent number.

10.100.4.14 btm_

```
uint8_t util::xmsg::XMfaMissionCfg::btm_  [private]
```

The bluetooth mode.

10.100.4.15 cms_

```
uint8_t util::xmsg::XMfaMissionCfg::cms_  [private]
```

The CTD profiling mode.

10.100.4.16 cpb_

```
uint16_t util::xmsg::XMfaMissionCfg::cpb_[3]  [private]
```

The CTD sampling period pressure boundaries, in dBar.

Referenced by DecodeXMsgData().

10.100.4.17 cpn_

```
uint16_t util::xmsg::XMfaMissionCfg::cpn_[3]  [private]
```

The CTD sampling period bin sizes, in 0.1 dBar.

Referenced by DecodeXMsgData().

10.100.4.18 cpp_

```
uint16_t util::xmsg::XMfaMissionCfg::cpp_[3]  [private]
```

The CTD sampling period interval, in milliseconds.

Referenced by DecodeXMsgData().

10.100.4.19 cps_

```
uint8_t util::xmsg::XMfaMissionCfg::cps_  [private]
```

The CTD profiling mode.

10.100.4.20 crd_

```
uint16_t util::xmsg::XMfaMissionCfg::crd_  [private]
```

The CTD raw sampling threshold, in dBar.

10.100.4.21 crs_

```
uint16_t util::xmsg::XMfaMissionCfg::crs_  [private]
```

The maximum number of raw samples to collect above the raw sampling threshold.

10.100.4.22 csp_

```
uint32_t util::xmsg::XMfaMissionCfg::csp_  [private]
```

The CTD continuous sampling period, in milliseconds.

10.100.4.23 cst_

```
uint8_t util::xmsg::XMfaMissionCfg::cst_  [private]
```

The CTD sampling type.

10.100.4.24 ctc_

```
uint8_t util::xmsg::XMfaMissionCfg::ctc_  [private]
```

The auxiliary temperature channel mode.

Note

Applies to RBR CTDs only.

10.100.4.25 ctf_

```
uint16_t util::xmsg::XMfaMissionCfg::ctf_  [private]
```

The profile cutoff pressure.

Todo Determine gain for profile cutoff pressure field.

Note

This field was added in firmware version 0x02006900.

10.100.4.26 ctl_

```
uint16_t util::xmsg::XMfaMissionCfg::ctl_  [private]
```

The pressure sensor offset reset lower pressure limit.

Todo Determine gain for pressure sensor offset reset lower pressure limit field.

Note

This field was added in firmware version 0x02006900.

10.100.4.27 ctu_

```
uint16_t util::xmsg::XMfaMissionCfg::ctu_  [private]
```

The pressure sensor offset reset lower pressure limit.

Todo Determine gain for pressure sensor offset reset lower pressure limit field.

Note

This field was added in firmware version 0x02006900.

10.100.4.28 ddb_

```
uint8_t util::xmsg::XMfaMissionCfg::ddb_  [private]
```

Diagnostic dive bypass mode.

10.100.4.29 ddd_

```
uint16_t util::xmsg::XMfaMissionCfg::ddd_  [private]
```

The descent state data storage pressure interval, in dBar.

10.100.4.30 ddt_

```
uint16_t util::xmsg::XMfaMissionCfg::ddt_  [private]
```

The descent state data storage interval, in seconds.

10.100.4.31 descend_start_time_

```
ptime util::xmsg::XMfaMissionCfg::descend_start_time_  [private]
```

The start time of the descend state of the dive cycle.

10.100.4.32 dft_

```
uint16_t util::xmsg::XMfaMissionCfg::dft_[3]  [private]
```

The descent state fall timeout, in seconds.

10.100.4.33 dns_

```
uint16_t util::xmsg::XMfaMissionCfg::dns_  [private]
```

The descent number.

10.100.4.34 dsd_

```
uint16_t util::xmsg::XMfaMissionCfg::dsd_[3]  [private]
```

The descent state start pressure, in dBar.

10.100.4.35 dsi_

```
uint32_t util::xmsg::XMfaMissionCfg::dsi_  [private]
```

The descent state sampling interval, in milliseconds.

10.100.4.36 dst_

```
uint16_t util::xmsg::XMfaMissionCfg::dst_[3]  [private]
```

The descent state start timeout, in seconds.

10.100.4.37 dtd_

```
uint16_t util::xmsg::XMfaMissionCfg::dtd_[3]  [private]
```

The target descent/drift/park pressure, in dBar.

10.100.4.38 fw_version_

```
uint32_t util::xmsg::XMfaMissionCfg::fw_version_  [private]
```

The software version identifier.

10.100.4.39 iat_

```
uint16_t util::xmsg::XMfaMissionCfg::iat_  [private]
```

Ice-detection – .

10.100.4.40 ica_

```
uint16_t util::xmsg::XMfaMissionCfg::ica_[2]  [private]
```

Ice-detection – surface attempts.

10.100.4.41 icb_

```
uint16_t util::xmsg::XMfaMissionCfg::icb_[2]  [private]
```

Ice-detection – cycles between surface attempts.

10.100.4.42 icd_

```
uint16_t util::xmsg::XMfaMissionCfg::icd_[2]  [private]
```

Ice-detection – hold pressure, in dBar.

10.100.4.43 icf_

```
uint32_t util::xmsg::XMfaMissionCfg::icf_  [private]
```

Ice-detection – ice-free month mask.

10.100.4.44 icg_

```
uint8_t util::xmsg::XMfaMissionCfg::icg_[2]  [private]
```

Ice-detection – seek mode.

10.100.4.45 icm_

```
uint8_t util::xmsg::XMfaMissionCfg::icm_  [private]
```

Ice-detection mode.

10.100.4.46 icn_

```
uint16_t util::xmsg::XMfaMissionCfg::icn_  [private]
```

Ice-detection – temperature threshold test upper pressure boundary, in dBar.

10.100.4.47 ico_

```
uint16_t util::xmsg::XMfaMissionCfg::ico_[2]  [private]
```

Ice-detection – ice-detection hold-pressure duration, in minutes.

10.100.4.48 icr_

```
uint32_t util::xmsg::XMfaMissionCfg::icr_[2]  [private]
```

Ice-detection – .

10.100.4.49 ics_

```
uint16_t util::xmsg::XMfaMissionCfg::ics_  [private]
```

Ice-detection – temperature threshold test sample interval, in seconds.

10.100.4.50 ict_

```
int16_t util::xmsg::XMfaMissionCfg::ict_  [private]
```

Ice-detection – temperature threshold test temperature, in 1/100 degC.

10.100.4.51 icw_

```
uint32_t util::xmsg::XMfaMissionCfg::icw_  [private]
```

Ice-detection – iced-over month mask.

10.100.4.52 icx_

```
uint16_t util::xmsg::XMfaMissionCfg::icx_  [private]
```

Ice-detection – temperature threshold test lower pressure boundary, in dBar.

10.100.4.53 ilt_

```
uint16_t util::xmsg::XMfaMissionCfg::ilt_[2]  [private]
```

Ice-detection – duration since last successful comms urgency criteria, in days.

10.100.4.54 imm_

```
uint16_t util::xmsg::XMfaMissionCfg::imm_  [private]
```

Ice-detection – month mask.

10.100.4.55 imt_

```
uint16_t util::xmsg::XMfaMissionCfg::imt_[2]  [private]
```

Ice-detection – percentage used storage urgency criteria.

10.100.4.56 isa_

```
uint8_t util::xmsg::XMfaMissionCfg::isa_  [private]
```

Ice-detection – .

10.100.4.57 isd_

```
uint16_t util::xmsg::XMfaMissionCfg::isd_  [private]
```

Ice-detection – .

10.100.4.58 ism_

```
uint8_t util::xmsg::XMfaMissionCfg::ism_  [private]
```

Ice-detection – surfacing achieved criteria mode.

10.100.4.59 iso_

```
uint16_t util::xmsg::XMfaMissionCfg::iso_  [private]
```

Ice-detection – duration between multiple surfacing attempts, in seconds.

10.100.4.60 isp_

```
uint8_t util::xmsg::XMfaMissionCfg::isp_  [private]
```

Ice-detection – multiple surfacing profile mode.

10.100.4.61 iss_

```
uint8_t util::xmsg::XMfaMissionCfg::iss_[2]  [private]
```

Ice-detection – multiple surface attempts mode.

10.100.4.62 ist_

```
uint16_t util::xmsg::XMfaMissionCfg::ist_  [private]
```

Ice-detection – surface achieved salinity threshold, in 1/10 PSU.

10.100.4.63 isv_

```
uint16_t util::xmsg::XMfaMissionCfg::isv_  [private]
```

Ice-detection – .

10.100.4.64 isw_

```
uint16_t util::xmsg::XMfaMissionCfg::isw_  [private]
```

Ice-detection – .

10.100.4.65 iue_

```
uint8_t util::xmsg::XMfaMissionCfg::iue_[2]  [private]
```

Ice-detection – urgency mode exit criteria.

10.100.4.66 mam_

```
uint8_t util::xmsg::XMfaMissionCfg::mam_  [private]
```

The motion processing unit (MPU) data acquisition mode mask.

10.100.4.67 mat_

```
uint16_t util::xmsg::XMfaMissionCfg::mat_[2]  [private]
```

The motion processing unit (MPU) sampling duration, in seconds.

10.100.4.68 mds_

```
uint16_t util::xmsg::XMfaMissionCfg::mds_  [private]
```

The maximum descent/fall/profile pressure, in dBar.

10.100.4.69 mnm_

```
uint16_t util::xmsg::XMfaMissionCfg::mnm_[3]  [private]
```

Number of cycles for mission phase.

10.100.4.70 mns_

```
uint8_t util::xmsg::XMfaMissionCfg::mns_[3]  [private]
```

Next mission phase.

10.100.4.71 mpc_

```
uint16_t util::xmsg::XMfaMissionCfg::mpc_  [private]
```

Cycle count for current mission phase.

e

10.100.4.72 mps_

```
uint8_t util::xmsg::XMfaMissionCfg::mps_  [private]
```

Current mission phase.

10.100.4.73 msg_data_

```
static_vector<vector<uint8_t>, XMSG_MAX_MFA_MISS> util::xmsg::XMfaMissionCfg::msg_data_  [private]
```

The mission configuration messages' raw data.

Referenced by XMfaMissionCfg(), and AssignMessage().

10.100.4.74 msg_version_

```
uint8_t util::xmsg::XMfaMissionCfg::msg_version_  [private]
```

The message version identifier.

10.100.4.75 msn_

```
int16_t util::xmsg::XMfaMissionCfg::msn_  [private]
```

The mission, dive, or cycle number for this mission configuration.

10.100.4.76 mtd_

```
uint16_t util::xmsg::XMfaMissionCfg::mtd_  [private]
```

The initial trigger depth, in dBar.

10.100.4.77 mtt_

```
uint32_t util::xmsg::XMfaMissionCfg::mtt_  [private]
```

The initial trigger timeout/interval, in milliseconds.

10.100.4.78 park_start_time_

```
ptime util::xmsg::XMfaMissionCfg::park_start_time_  [private]
```

The start time of the park/drift state of the dive cycle.

10.100.4.79 pdt_

```
uint16_t util::xmsg::XMfaMissionCfg::pdt_  [private]
```

The park/drift state data storage interval/timeout, in seconds.

10.100.4.80 pfd_

```
uint16_t util::xmsg::XMfaMissionCfg::pfd_[3]  [private]
```

The park/drift state fall depth (i.e.

profile depth), in dBar.

10.100.4.81 pfm_

```
uint8_t util::xmsg::XMfaMissionCfg::pfm_[3]  [private]
```

The park/drift state fall mode.

10.100.4.82 pns_

```
uint16_t util::xmsg::XMfaMissionCfg::pns_  [private]
```

The park/drift number.

10.100.4.83 psa_

```
uint8_t util::xmsg::XMfaMissionCfg::psa_  [private]
```

The park/drift state sampling mode.

10.100.4.84 psi_

```
uint32_t util::xmsg::XMfaMissionCfg::psi_  [private]
```

The park/drift state sample interval/timeout, in milliseconds.

10.100.4.85 pth_

```
uint32_t util::xmsg::XMfaMissionCfg::pth_[3]  [private]
```

The bit-encoded park/drift state hour timeout trigger(s).

10.100.4.86 pto_

```
uint32_t util::xmsg::XMfaMissionCfg::pto_[3]  [private]
```

The park/drift state timeout, in seconds.

10.100.4.87 sbm_

```
uint8_t util::xmsg::XMfaMissionCfg::sbm_  [private]
```

The surface state beacon mode flag.

10.100.4.88 sbt_

```
uint16_t util::xmsg::XMfaMissionCfg::sbt_  [private]
```

The surface state beacon interval timeout, in seconds.

10.100.4.89 sdc_

```
int16_t util::xmsg::XMfaMissionCfg::sdc_  [private]
```

The surface state sensor depth compensation offset, in dBar.

10.100.4.90 sds_

```
uint16_t util::xmsg::XMfaMissionCfg::sds_  [private]
```

The surface state sampling storage duration/interval/timeout, in seconds.

10.100.4.91 sensor_type_

```
uint8_t util::xmsg::XMfaMissionCfg::sensor_type_  [private]
```

The sensor type identifier.

10.100.4.92 set_

```
uint16_t util::xmsg::XMfaMissionCfg::set_  [private]
```

The surface state Iridium session timeout, in minutes.

10.100.4.93 sfn_

```
uint16_t util::xmsg::XMfaMissionCfg::sfn_  [private]
```

The surface number.

10.100.4.94 sgt_

```
uint16_t util::xmsg::XMfaMissionCfg::sgt_  [private]
```

The surface state GPS timeout, in minutes.

10.100.4.95 ssi_

```
uint32_t util::xmsg::XMfaMissionCfg::ssi_  [private]
```

The surface state sampling interval timeout, in milliseconds.

10.100.4.96 surface_start_time_

```
ptime util::xmsg::XMfaMissionCfg::surface_start_time_  [private]
```

The start time of the surface state of the dive cycle.

10.100.4.97 wmt_

```
uint16_t util::xmsg::XMfaMissionCfg::wmt_  [private]
```

WIMS timeout during surface state, in seconds.

10.100.4.98 xas_

```
uint8_t util::xmsg::XMfaMissionCfg::xas_  [private]
```

The profile data encode method.

The documentation for this class was generated from the following files:

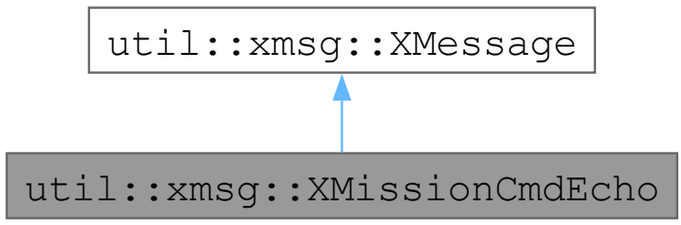
- data/xmsg/XMfaMissionCfg.h
- data/xmsg/XMfaMissionCfg.cpp

10.101 util::xmsg::XMissionCmdEcho Class Reference

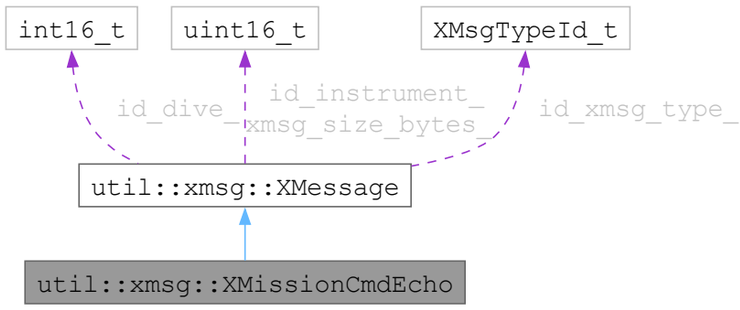
Class representing a ground station command echo-back X-Message.

```
#include <XMissionCmdEcho.h>
```

Inheritance diagram for util::xmsg::XMissionCmdEcho:



Collaboration diagram for util::xmsg::XMissionCmdEcho:



Public Member Functions

- XMissionCmdEcho (int16_t dive, uint16_t whoi, const vector< uint8_t > &data)

  The parameterized constructor.
- ~XMissionCmdEcho ()

  The destructor.
- const uint8_t * CheckXMsgData (const vector< uint8_t > &data, int id, int minBytes, bool parseSize=true)

  Check the X-message for the correct ID, minimum size, and message terminator.
- void DecodeXMsgData (const vector< uint8_t > &data)

  Decode the ground station command echo-back X-Message from the binary message data.
- const string GenerateListingString () const

  Generate a listing string representation of the ground station command echo-back message.

- int16_t id_dive () const

  Get the dive or cycle number of this X-Message.
- uint16_t id_instrument () const

  Get the unique instrument identifier of this X-Message.
- XMsgTypeId_t id_xmsg_type () const

  Get the enumerated X-Message type identifier.
- uint16_t xmsg_size_bytes () const

  Get the size of the X-Message, in bytes.

Protected Member Functions

- void set_id_dive (int16_t id_dive)

  Set the dive or cycle number of this message.
- void set_id_instrument (uint16_t id_instrument)

  Set the unique instrument identifier.
- void set_id_xmsg_type (XMsgTypeId_t id_xmsg_type)

  Set the X-Message type identifier.
- void set_xmsg_size_bytes (uint16_t xmsg_size_bytes)

  Set the size of the X-Message, in bytes.

Private Attributes

- int16_t id_dive_

  The dive or cycle number of this message.
- uint16_t id_instrument_

  The unique WHOI identifier of this instrument.
- XMsgTypeId_t id_xmsg_type_

  The X-message type identifier.
- uint16_t xmsg_size_bytes_

  The size of the X message, in bytes.

10.101.1 Detailed Description

Class representing a ground station command echo-back X-Message.

10.101.2 Constructor & Destructor Documentation

10.101.2.1 XMissionCmdEcho()

```
util::xmsg::XMissionCmdEcho::XMissionCmdEcho (
            int16_t dive,
            uint16_t whoi,
            const vector< uint8_t > & data)  [inline]
```

The parameterized constructor.

Parameters

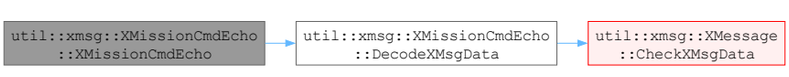
in	*dive*	The dive or cycle number of this message.
in	*whoi*	The unique WHOI identifier.
in	*data*	The binary X-message data to decode.

Exceptions

util::log::MessageException	if an decode error occurs.

References DecodeXMsgData().

Here is the call graph for this function:



10.101.2.2 ~XMissionCmdEcho()

```
util::xmsg::XMissionCmdEcho::~XMissionCmdEcho ()  [inline]
```

The destructor.

10.101.3 Member Function Documentation

10.101.3.1 CheckXMsgData()

```
const uint8_t * util::xmsg::XMessage::CheckXMsgData (
            const vector< uint8_t > & data,
            int id,
            int minBytes,
            bool parseSize = true)  [inline], [inherited]
```

Check the X-message for the correct ID, minimum size, and message terminator.

Parameters

in	*data*	The X-message binary data to check.
in	*id*	The expected ID of the X-message. If this field is less that 0, then the ID check will be skipped.
in	*minBytes*	The minimum number of bytes expected in a valid message.
in	*parseSize*	If TRUE, then the message size field will be parsed from the X-Message. Otherwise, the size field will not be parsed.

Exceptions

util::log::MessageException	if • the message does not contain the expected ID, • the message does not contain the minimum number of expected bytes, • the message does not contain a terminator (';') in the correct location, or, • any other error occurs.

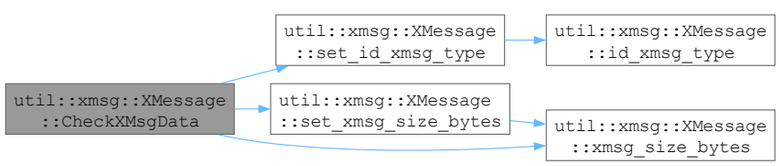
Returns

A pointer to the underlying data buffer of data.

References CATCH_BLOCK, EXC_THROW, lwrn, util::xmsg::XMessage::set_id_xmsg_type(), util::xmsg::XMessage::set_xmsg_size_byte
u16, and util::xmsg::XMessage::xmsg_size_bytes().

Referenced by util::xmsg::XAcousticPosition::DecodeXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XDriftPressure::Dec
util::xmsg::XDriftSalinity::DecodeXMsgData(), util::xmsg::XDriftTemperature::DecodeXMsgData(), util::xmsg::XEngAbort::DecodeXMsgDa
util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngBIT::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(),
util::xmsg::XEngDiagnostic::DecodeXMsgData(), util::xmsg::XEngDrift::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(),
util::xmsg::XEngStandard::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XGPS::DecodeXMsgData(),
util::xmsg::XHiResPressure::DecodeXMsgData(), util::xmsg::XHiResSalinity::DecodeXMsgData(), util::xmsg::XHiResTemperature::Decod
DecodeXMsgData(), util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XProfilePressure::DecodeXMsgData(),
util::xmsg::XProfileSalinity::DecodeXMsgData(), util::xmsg::XProfileTemperature::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgD
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the call graph for this function:



10.101.3.2 DecodeXMsgData()

```
void util::xmsg::XMissionCmdEcho::DecodeXMsgData (
            const vector< uint8_t > & data)  [inline], [virtual]
```

Decode the ground station command echo-back X-Message from the binary message data.

Parameters

in	*data*	The binary message data to decode.

Exceptions

util::log::MessageException	if the message is invalid or an error occurs during decoding.

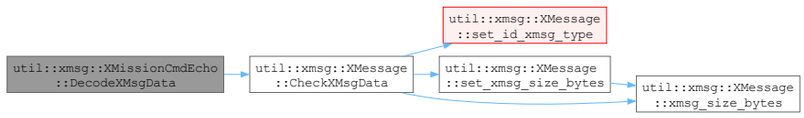
Todo Implement the functionality to decode the ground station command echo-back X-Message from the binary message data.

Implements util::xmsg::XMessage.

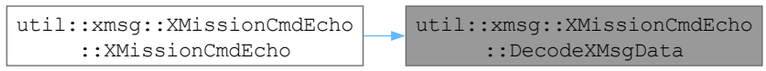
References CATCH_BLOCK, util::xmsg::XMessage::CheckXMsgData(), and util::xmsg::XMSG_TYPE_ID_MISS_CMD_ECHO.

Referenced by XMissionCmdEcho().

Here is the call graph for this function:



Here is the caller graph for this function:



10.101.3.3 GenerateListingString()

```
const string util::xmsg::XMissionCmdEcho::GenerateListingString () const  [inline]
```

Generate a listing string representation of the ground station command echo-back message.

Returns

A listing string representation of the ground station command echo-back message.

Todo Implement the functionality for generating a listing string for the ground station upload command echo-back message.

References CATCH_BLOCK.

10.101.3.4 id_dive()

```
int16_t util::xmsg::XMessage::id_dive () const  [inline], [inherited]
```

Get the dive or cycle number of this X-Message.

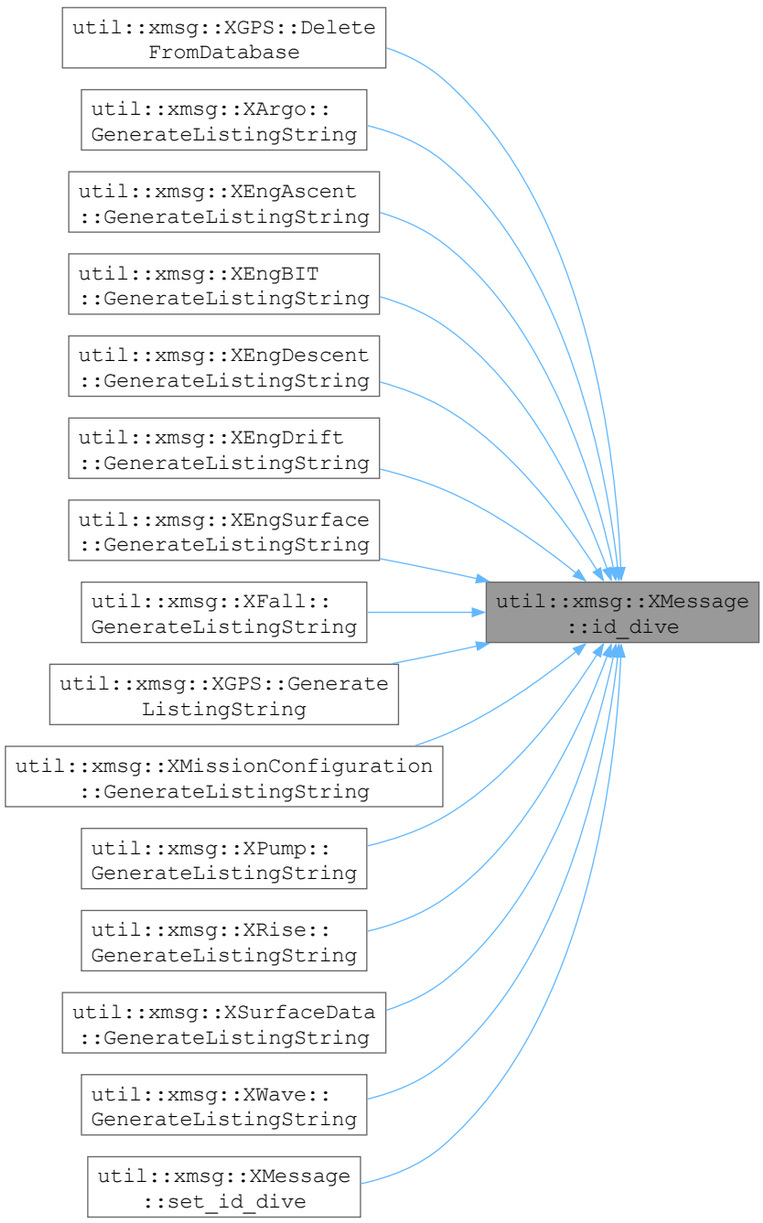
Returns

The dive or cycle number of this X-Message.

References util::xmsg::XMessage::id_dive_.

Referenced by util::xmsg::XGPS::DeleteFromDatabase(), util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateLis
util::xmsg::XEngBIT::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingStr
util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(),
util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing
util::xmsg::XSurfaceData::GenerateListingString(), util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_dive().

Here is the caller graph for this function:



10.101.3.5 id_instrument()

```
uint16_t util::xmsg::XMessage::id_instrument () const  [inline], [inherited]
```

Get the unique instrument identifier of this X-Message.

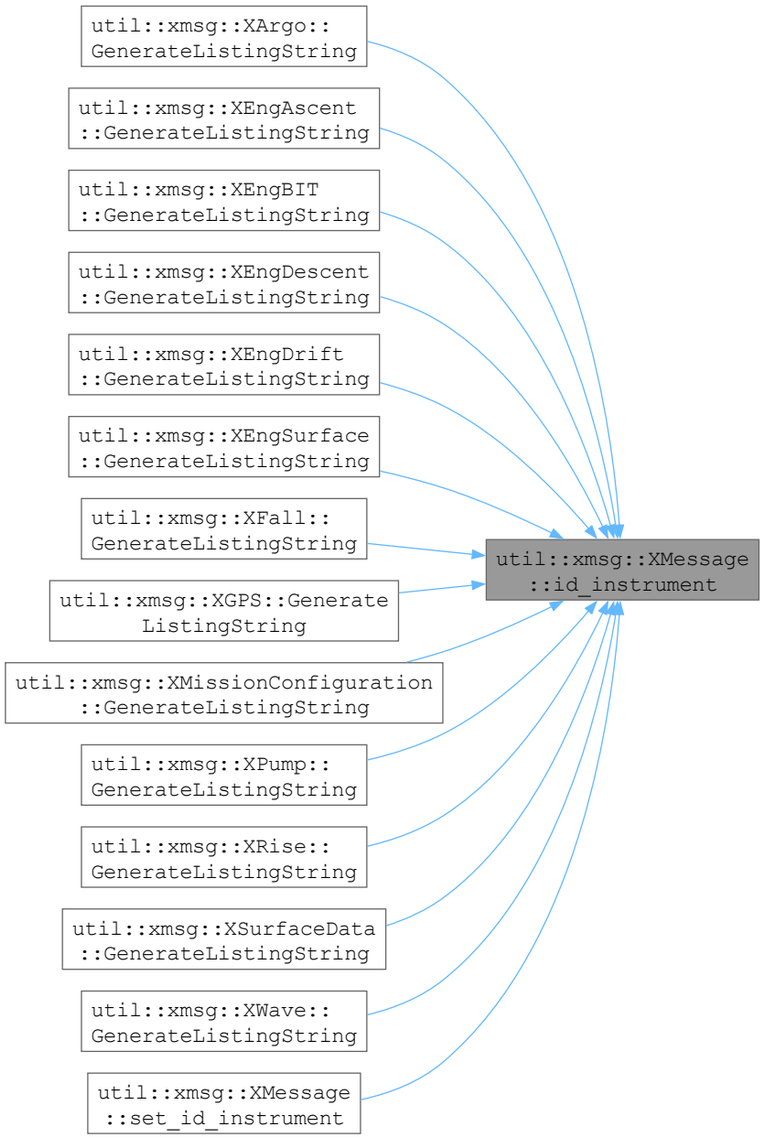
Returns

The unique instrument identifier of this X-Message.

References util::xmsg::XMessage::id_instrument_.

Referenced by util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngBIT::Generate
util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), util::xmsg::XEngSurface::GenerateListin
util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingS
util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(),
util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_instrument().

Here is the caller graph for this function:



10.101.3.6 id_xmsg_type()

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type () const  [inline], [inherited]
```

Get the enumerated X-Message type identifier.

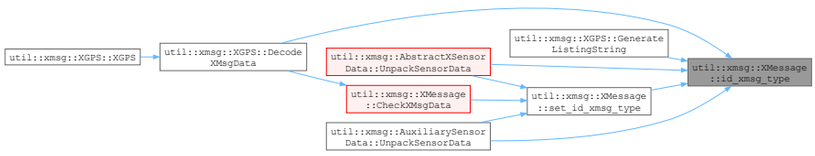
Returns

The enumerated X-Message type identifier.

References util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMessage::set_id_xmsg_typ
util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the caller graph for this function:



10.101.3.7 set_id_dive()

```
void util::xmsg::XMessage::set_id_dive (
            int16_t id_dive)  [inline], [protected], [inherited]
```

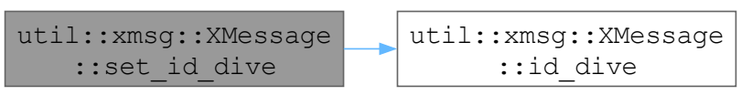
Set the dive or cycle number of this message.

Parameters

in	*id_dive*	The dive or cycle number of this message to set.

References util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::id_dive_.

Here is the call graph for this function:



10.101.3.8 set_id_instrument()

```
void util::xmsg::XMessage::set_id_instrument (
            uint16_t id_instrument)  [inline], [protected], [inherited]
```

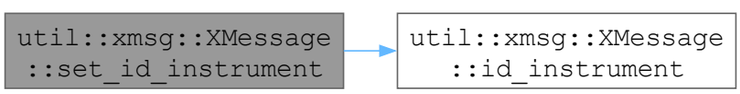
Set the unique instrument identifier.

Parameters

in	*id_instrument*	The unique instrument identifier to set.

References util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::id_instrument_.

Here is the call graph for this function:



10.101.3.9 set_id_xmsg_type()

```
void util::xmsg::XMessage::set_id_xmsg_type (
            XMsgTypeId_t id_xmsg_type)  [inline], [protected], [inherited]
```

Set the X-Message type identifier.

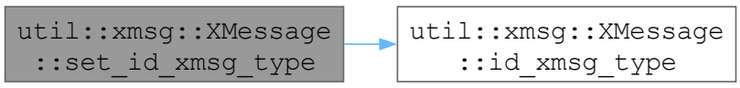
Parameters

in	*id_xmsg_type*	The X-Message type identifier to set.

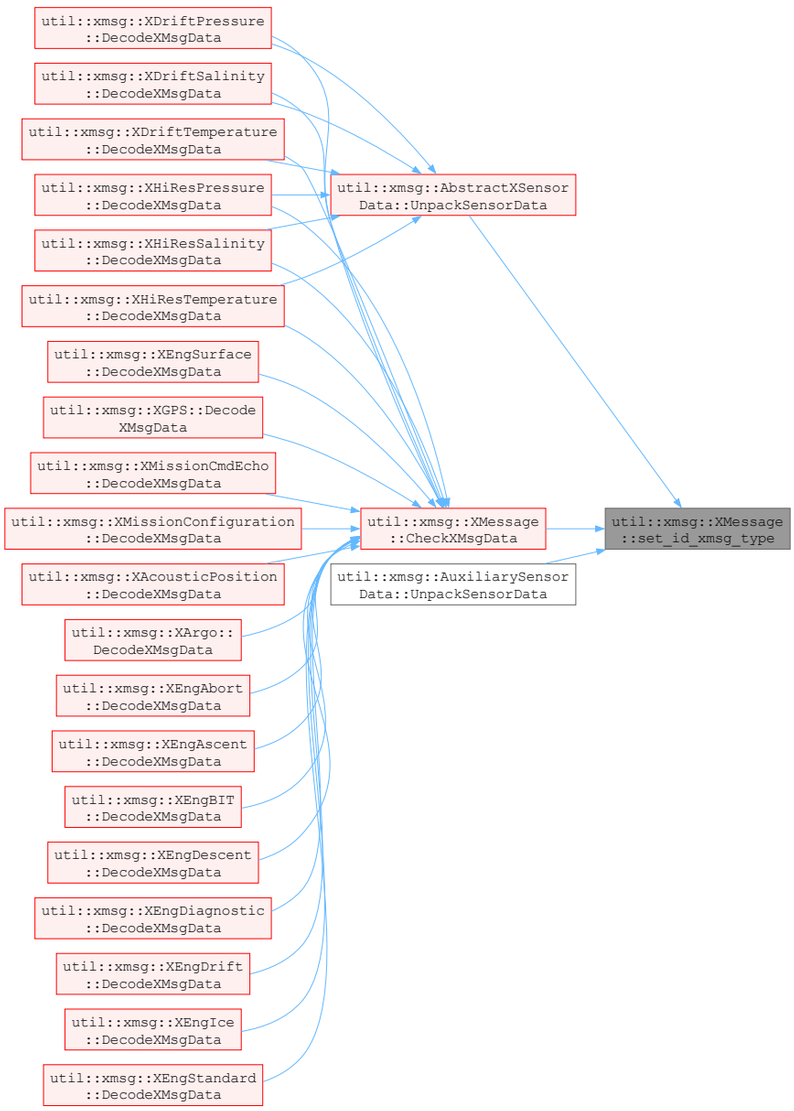
References util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.101.3.10 set_xmsg_size_bytes()

```
void util::xmsg::XMessage::set_xmsg_size_bytes (
            uint16_t xmsg_size_bytes)  [inline], [protected], [inherited]
```

Set the size of the X-Message, in bytes.

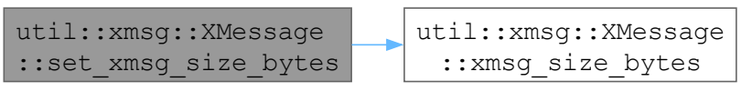
Parameters

in	*xmsg_size_bytes*	The size of the X-Message to set, in bytes.

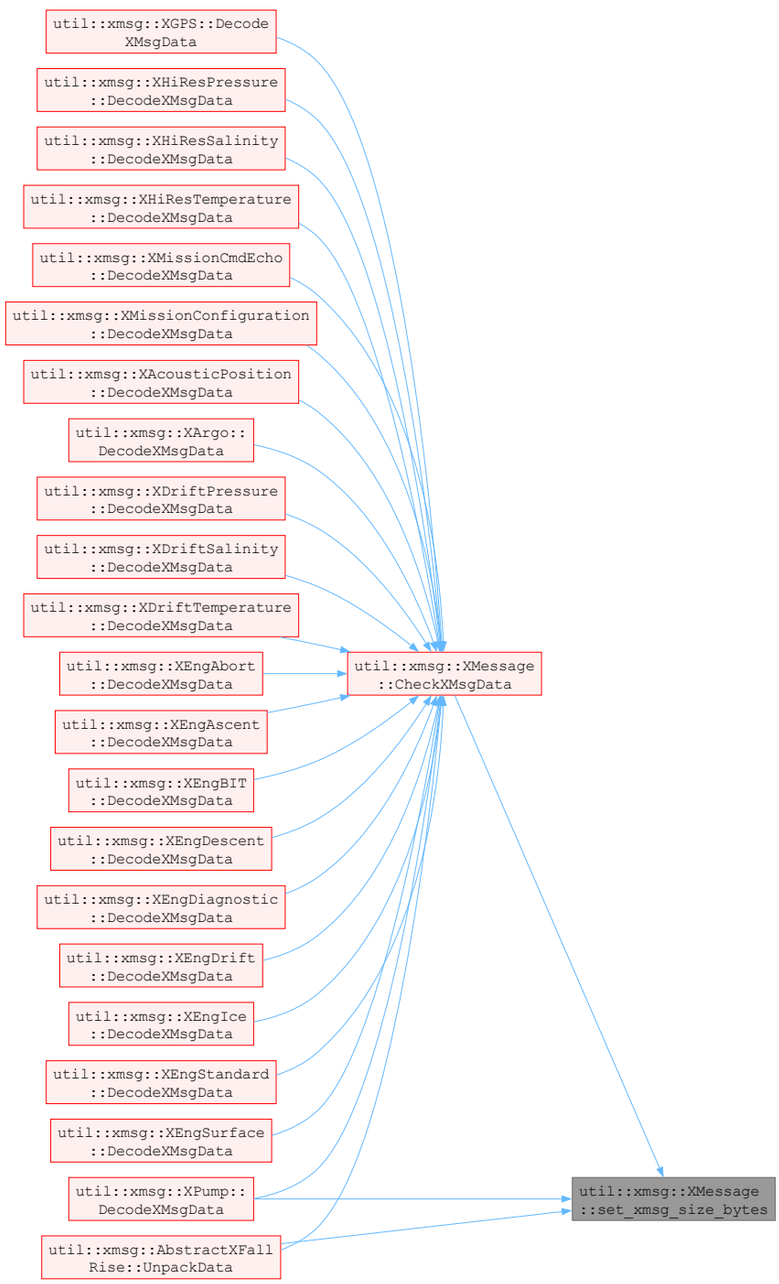
References util::xmsg::XMessage::xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XPump::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::Unpa

Here is the call graph for this function:



Here is the caller graph for this function:



10.101.3.11 xmsg_size_bytes()

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes () const  [inline], [inherited]
```

Get the size of the X-Message, in bytes.

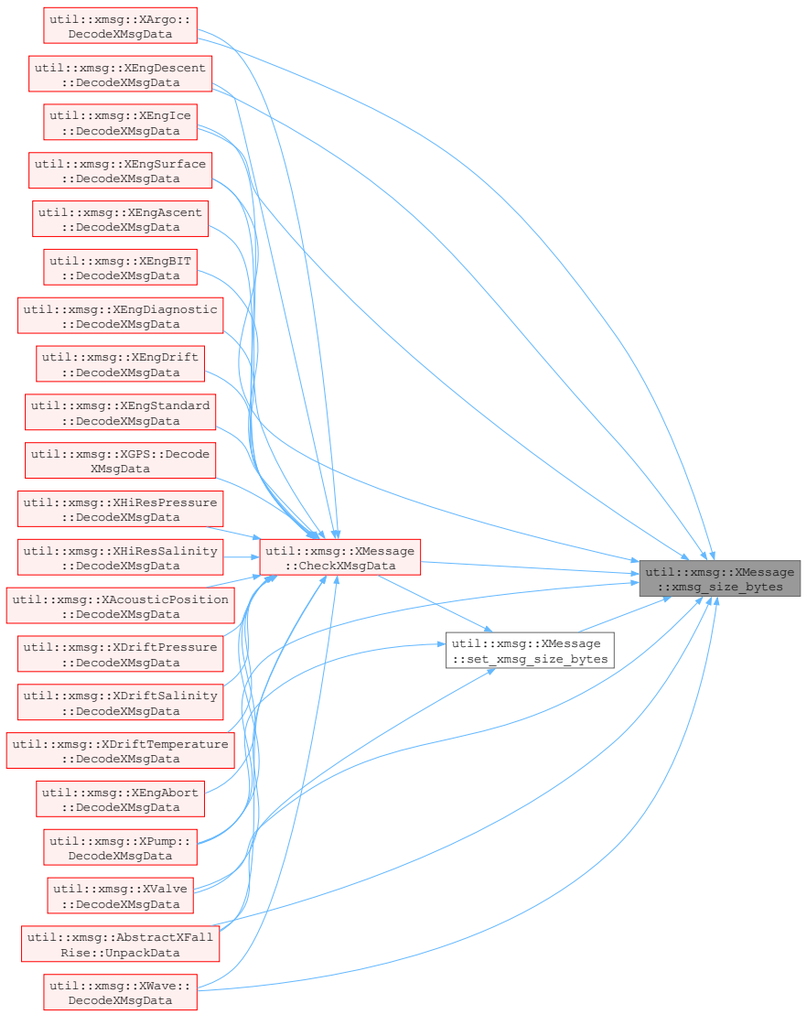
Returns

The size of the X-Message, in bytes.

References util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgI
util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(),
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), util::xmsg::XMessage::set_xmsg_size_bytes(),
and util::xmsg::AbstractXFallRise::UnpackData().

Here is the caller graph for this function:



10.101.4 Member Data Documentation

10.101.4.1 id_dive_

```
int16_t util::xmsg::XMessage::id_dive_  [private], [inherited]
```

The dive or cycle number of this message.

Referenced by util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::set_id_dive().

10.101.4.2 id_instrument_

```
uint16_t util::xmsg::XMessage::id_instrument_  [private], [inherited]
```

The unique WHOI identifier of this instrument.

Referenced by util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::set_id_instrument().

10.101.4.3 id_xmsg_type_

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type_  [private], [inherited]
```

The X-message type identifier.

Referenced by util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::set_id_xmsg_type().

10.101.4.4 xmsg_size_bytes_

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes_  [private], [inherited]
```

The size of the X message, in bytes.

Referenced by util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes().

The documentation for this class was generated from the following file:

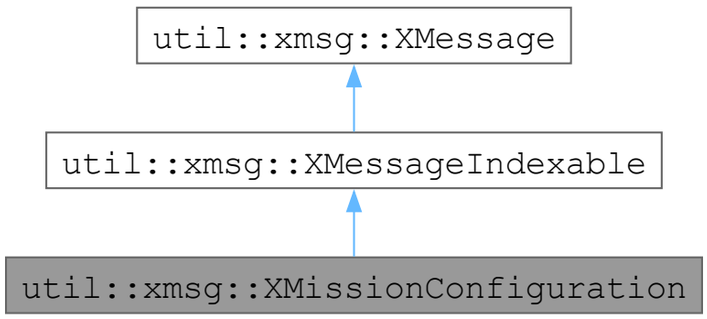
- data/xmsg/XMissionCmdEcho.h

10.102 util::xmsg::XMissionConfiguration Class Reference

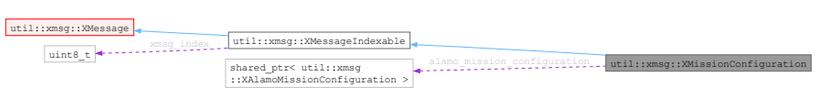
Class representing a mission configuration X-Message.

```
#include <XMissionConfiguration.h>
```

Inheritance diagram for util::xmsg::XMissionConfiguration:



Collaboration diagram for util::xmsg::XMissionConfiguration:



Public Member Functions

- XMissionConfiguration (int16_t dive, uint16_t whoi, const vector< uint8_t > &data)
  The parameterized constructor.
- ~XMissionConfiguration ()
  The destructor.
- const uint8_t * CheckXMsgData (const vector< uint8_t > &data, int id, int minBytes, bool parseSize=true)
  Check the X-message for the correct ID, minimum size, and message terminator.
- void DecodeXMsgData (const vector< uint8_t > &data)
  Decode the mission configuration message from the binary data.
- const string GenerateListingString () const
  Get the listing entry string for this mission configuration X-Message.
- int16_t id_dive () const
  Get the dive or cycle number of this X-Message.
- uint16_t id_instrument () const
  Get the unique instrument identifier of this X-Message.
- XMsgTypeId_t id_xmsg_type () const
  Get the enumerated X-Message type identifier.
- uint8_t xmsg_index () const
  Get the index of this X-Message.
- uint16_t xmsg_size_bytes () const
  Get the size of the X-Message, in bytes.

Protected Member Functions

- void set_id_dive (int16_t id_dive)
  Set the dive or cycle number of this message.
- void set_id_instrument (uint16_t id_instrument)
  Set the unique instrument identifier.
- void set_id_xmsg_type (XMsgTypeId_t id_xmsg_type)
  Set the X-Message type identifier.
- void set_xmsg_index (uint8_t xmsg_index)
  Set the index of this X-Message.
- void set_xmsg_size_bytes (uint16_t xmsg_size_bytes)
  Set the size of the X-Message, in bytes.

Private Attributes

- shared_ptr< XAlamoMissionConfiguration > alamo_mission_configuration_
  A shared pointer to the ALAMO mission configuration object.
- int16_t id_dive_
  The dive or cycle number of this message.
- uint16_t id_instrument_
  The unique WHOI identifier of this instrument.
- XMsgTypeId_t id_xmsg_type_
  The X-message type identifier.
- uint8_t xmsg_index_
  The index of this X-Message.
- uint16_t xmsg_size_bytes_
  The size of the X message, in bytes.

10.102.1 Detailed Description

Class representing a mission configuration X-Message.

Note

This object supports both ALAMO and Solo-II mission configurations.

10.102.2 Constructor & Destructor Documentation

10.102.2.1 XMissionConfiguration()

```
util::xmsg::XMissionConfiguration::XMissionConfiguration (
            int16_t dive,
            uint16_t whoi,
            const vector< uint8_t > & data)  [inline]
```

The parameterized constructor.

Parameters

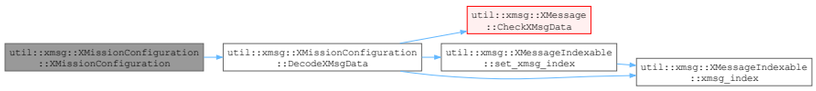
in	*dive*	The dive or cycle number of this message.
in	*whoi*	The unique WHOI identifier.
in	*data*	The binary mission configuration X-message data to decode.

Exceptions

util::log::MessageException	if an decode error occurs.

References DecodeXMsgData().

Here is the call graph for this function:



10.102.2.2 ~XMissionConfiguration()

```
util::xmsg::XMissionConfiguration::~XMissionConfiguration ()  [inline]
```

The destructor.

10.102.3 Member Function Documentation

10.102.3.1 CheckXMsgData()

```
const uint8_t * util::xmsg::XMessage::CheckXMsgData (
            const vector< uint8_t > & data,
            int id,
            int minBytes,
            bool parseSize = true)  [inline], [inherited]
```

Check the X-message for the correct ID, minimum size, and message terminator.

Parameters

in	*data*	The X-message binary data to check.
in	*id*	The expected ID of the X-message. If this field is less that 0, then the ID check will be skipped.
in	*minBytes*	The minimum number of bytes expected in a valid message.
in	*parseSize*	If TRUE, then the message size field will be parsed from the X-Message. Otherwise, the size field will not be parsed.

Exceptions

util::log::MessageException	if • the message does not contain the expected ID, • the message does not contain the minimum number of expected bytes, • the message does not contain a terminator (';') in the correct location, or, • any other error occurs.

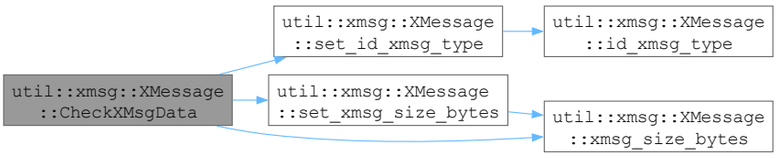
Returns

A pointer to the underlying data buffer of data.

References CATCH_BLOCK, EXC_THROW, lwrn, util::xmsg::XMessage::set_id_xmsg_type(), util::xmsg::XMessage::set_xmsg_size_byte
u16, and util::xmsg::XMessage::xmsg_size_bytes().

Referenced by util::xmsg::XAcousticPosition::DecodeXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XDriftPressure::Dec
util::xmsg::XDriftSalinity::DecodeXMsgData(), util::xmsg::XDriftTemperature::DecodeXMsgData(), util::xmsg::XEngAbort::DecodeXMsgDa
util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngBIT::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(),
util::xmsg::XEngDiagnostic::DecodeXMsgData(), util::xmsg::XEngDrift::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(),
util::xmsg::XEngStandard::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XGPS::DecodeXMsgData(),
util::xmsg::XHiResPressure::DecodeXMsgData(), util::xmsg::XHiResSalinity::DecodeXMsgData(), util::xmsg::XHiResTemperature::Decod
util::xmsg::XMissionCmdEcho::DecodeXMsgData(), DecodeXMsgData(), util::xmsg::XProfilePressure::DecodeXMsgData(),
util::xmsg::XProfileSalinity::DecodeXMsgData(), util::xmsg::XProfileTemperature::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgD
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the call graph for this function:



10.102.3.2 DecodeXMsgData()

```
void util::xmsg::XMissionConfiguration::DecodeXMsgData (
            const vector< uint8_t > & data)  [inline], [virtual]
```

Decode the mission configuration message from the binary data.

Parameters

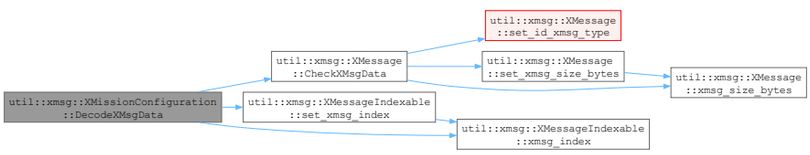
in	*data*	The binary message to decode.

Implements util::xmsg::XMessageIndexable.

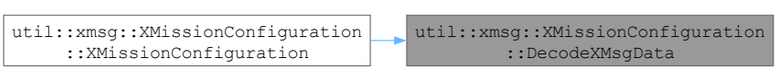
References alamo_mission_configuration_, CATCH_BLOCK, util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XMessageIndexable::s
and util::xmsg::XMessageIndexable::xmsg_index().

Referenced by XMissionConfiguration().

Here is the call graph for this function:



Here is the caller graph for this function:



10.102.3.3 GenerateListingString()

```
const string util::xmsg::XMissionConfiguration::GenerateListingString () const  [inline]
```

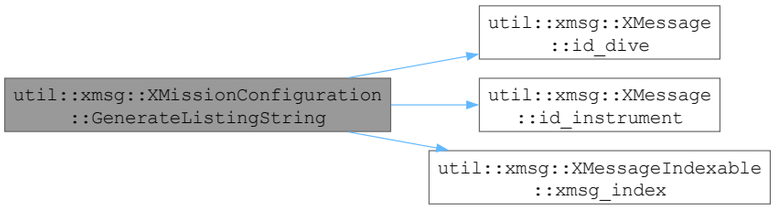
Get the listing entry string for this mission configuration X-Message.

Returns

The listing entry string for this mission configuration X-Message.

References alamo_mission_configuration_, CATCH_BLOCK, util::xmsg::XMessage::id_dive(), util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessageIndexable::xmsg_index().

Here is the call graph for this function:



10.102.3.4 id_dive()

```
int16_t util::xmsg::XMessage::id_dive () const  [inline], [inherited]
```

Get the dive or cycle number of this X-Message.

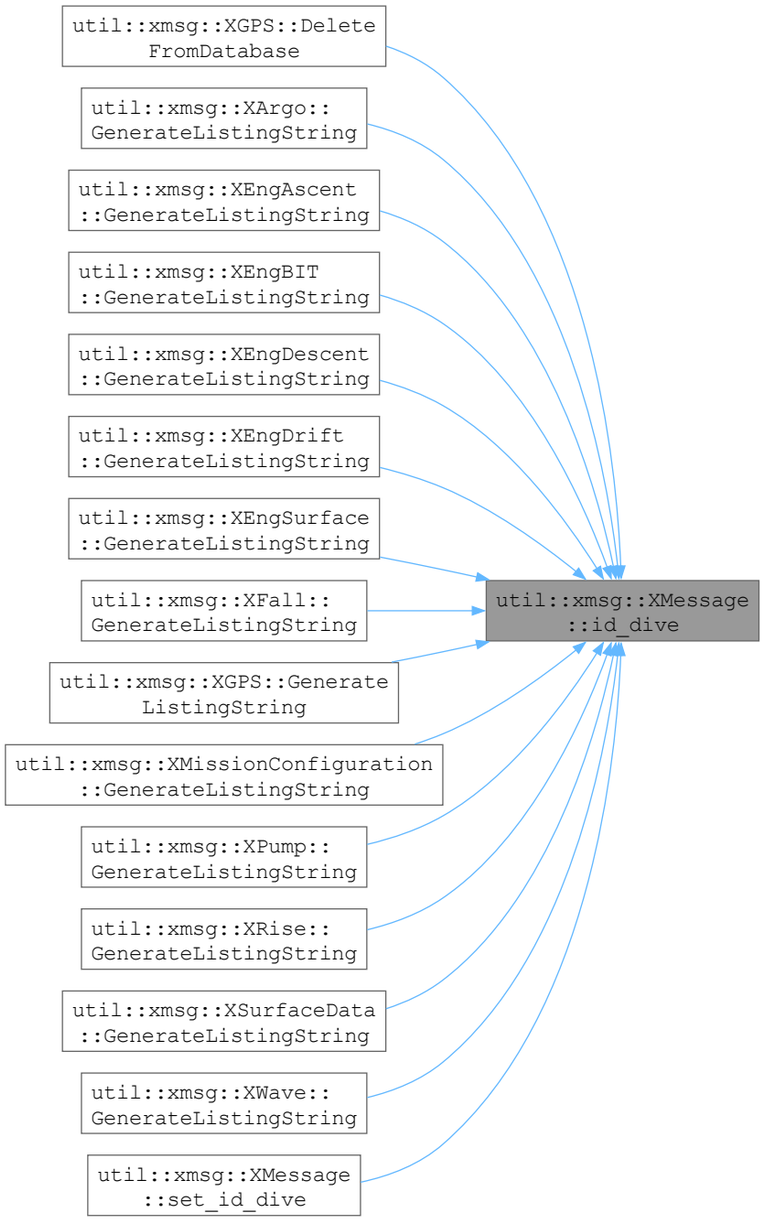
Returns

The dive or cycle number of this X-Message.

References util::xmsg::XMessage::id_dive_.

Referenced by util::xmsg::XGPS::DeleteFromDatabase(), util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateLis util::xmsg::XEngBIT::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingStr util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(), util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_dive().

Here is the caller graph for this function:



10.102.3.5 id_instrument()

```
uint16_t util::xmsg::XMessage::id_instrument () const  [inline], [inherited]
```

Get the unique instrument identifier of this X-Message.

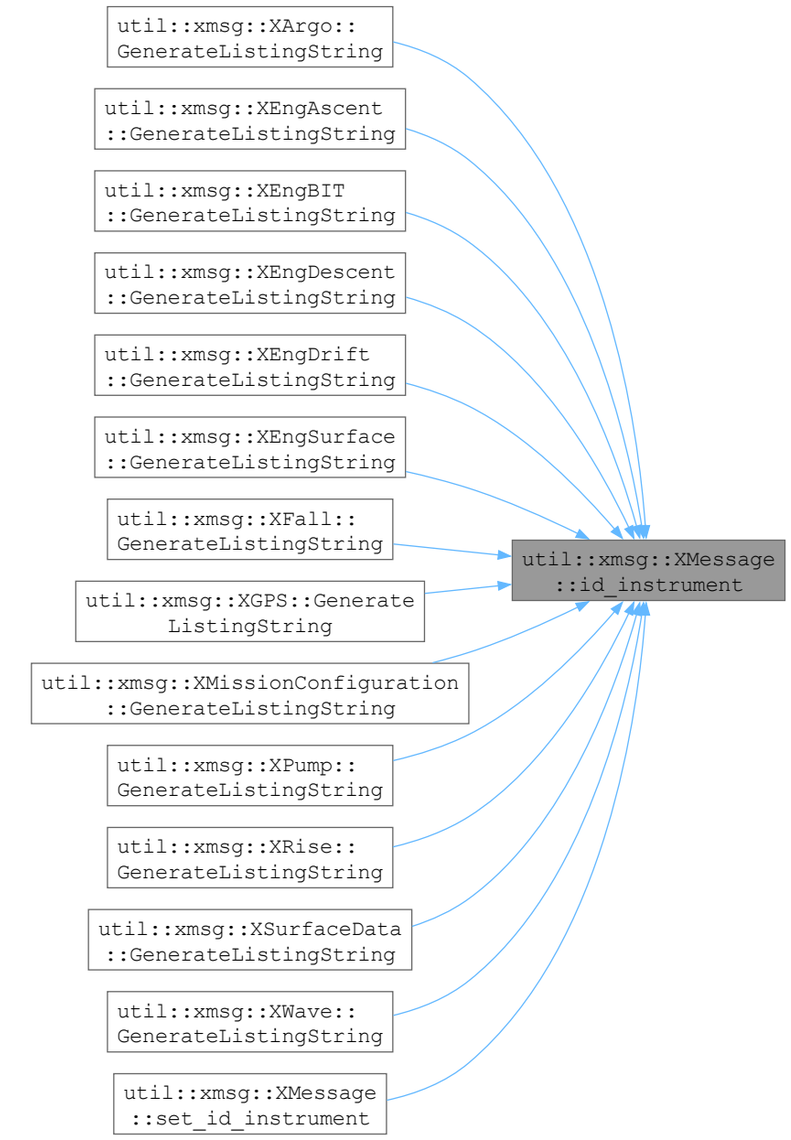
Returns

The unique instrument identifier of this X-Message.

References util::xmsg::XMessage::id_instrument_.

Referenced by util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngBIT::Generate
util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), util::xmsg::XEngSurface::GenerateListin
util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), GenerateListingString(), util::xmsg::XPump::Generat
util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(), util::xmsg::XWave::GenerateListingString(),
and util::xmsg::XMessage::set_id_instrument().

Here is the caller graph for this function:



10.102.3.6 id_xmsg_type()

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type () const  [inline], [inherited]
```

Get the enumerated X-Message type identifier.

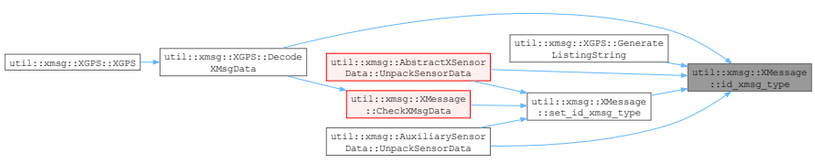
Returns

The enumerated X-Message type identifier.

References util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMessage::set_id_xmsg_typ
util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the caller graph for this function:



10.102.3.7 set_id_dive()

```
void util::xmsg::XMessage::set_id_dive (
            int16_t id_dive)  [inline], [protected], [inherited]
```

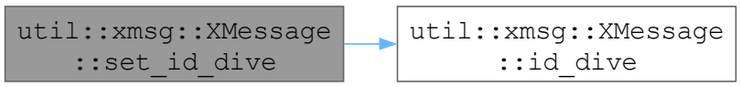
Set the dive or cycle number of this message.

Parameters

in	*id_dive*	The dive or cycle number of this message to set.

References util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::id_dive_.

Here is the call graph for this function:



10.102.3.8 set_id_instrument()

```
void util::xmsg::XMessage::set_id_instrument (
            uint16_t id_instrument)  [inline], [protected], [inherited]
```

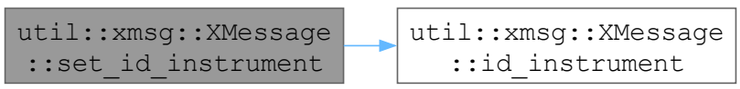
Set the unique instrument identifier.

Parameters

in	*id_instrument*	The unique instrument identifier to set.

References util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::id_instrument_.

Here is the call graph for this function:



10.102.3.9 set_id_xmsg_type()

```
void util::xmsg::XMessage::set_id_xmsg_type (
            XMsgTypeId_t id_xmsg_type)  [inline], [protected], [inherited]
```

Set the X-Message type identifier.

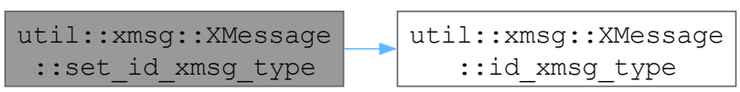
Parameters

in	*id_xmsg_type*	The X-Message type identifier to set.

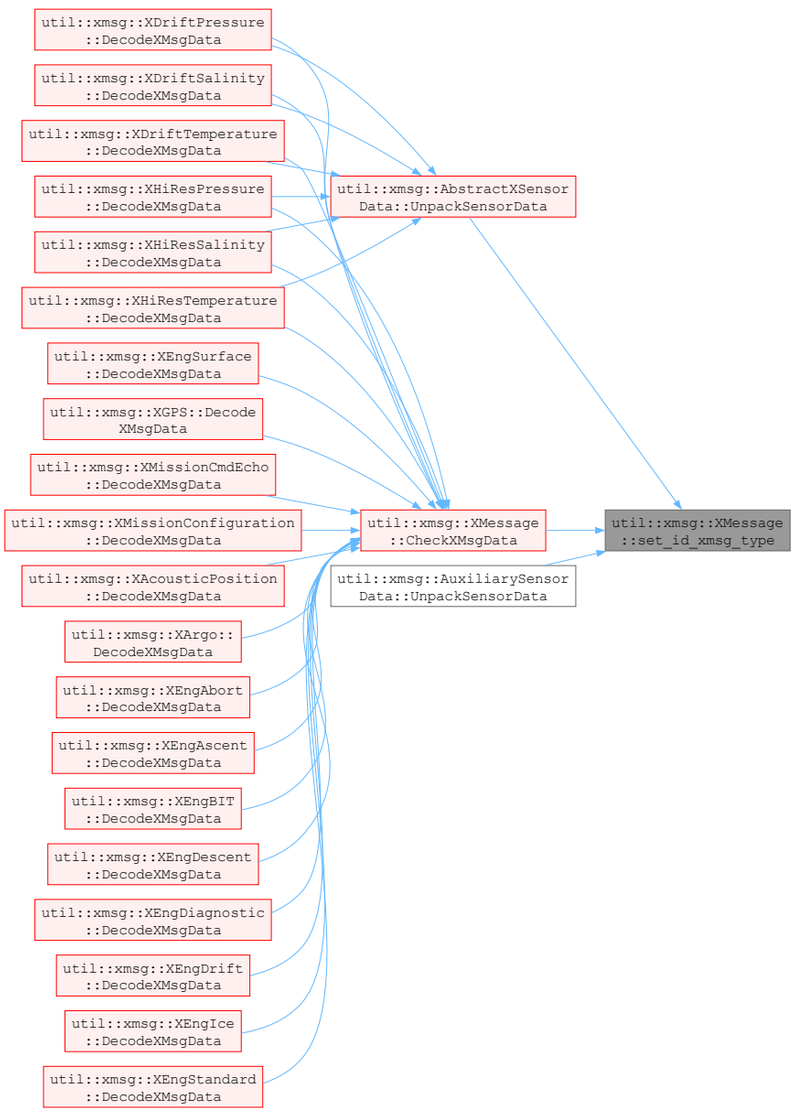
References util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.102.3.10 set_xmsg_index()

```
void util::xmsg::XMessageIndexable::set_xmsg_index (
            uint8_t xmsg_index)  [inline], [protected], [inherited]
```

Set the index of this X-Message.

Parameters

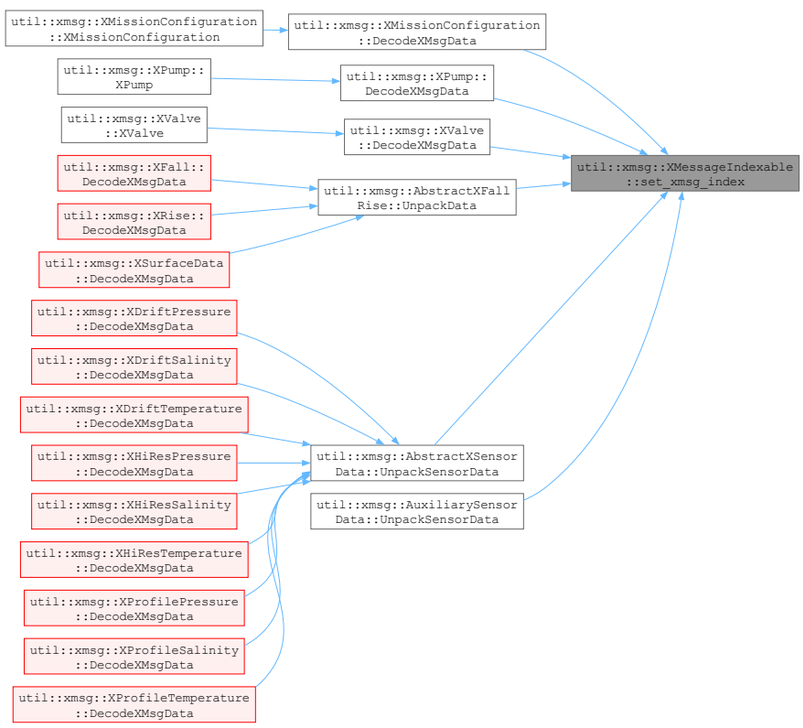
in	*xmsg_index*	The index of this X-Message.

References util::xmsg::XMessageIndexable::xmsg_index(), and util::xmsg::XMessageIndexable::xmsg_index_.

Referenced by DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(), util::xmsg::XValve::DecodeXMsgData(), util::xmsg::AbstractXFallRise::UnpackData(), util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData

Here is the call graph for this function:

Here is the caller graph for this function:



10.102.3.11 set_xmsg_size_bytes()

```
void util::xmsg::XMessage::set_xmsg_size_bytes (
            uint16_t xmsg_size_bytes)  [inline], [protected], [inherited]
```

Set the size of the X-Message, in bytes.

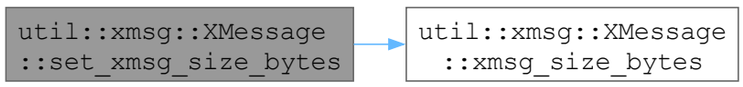
Parameters

in	*xmsg_size_bytes*	The size of the X-Message to set, in bytes.

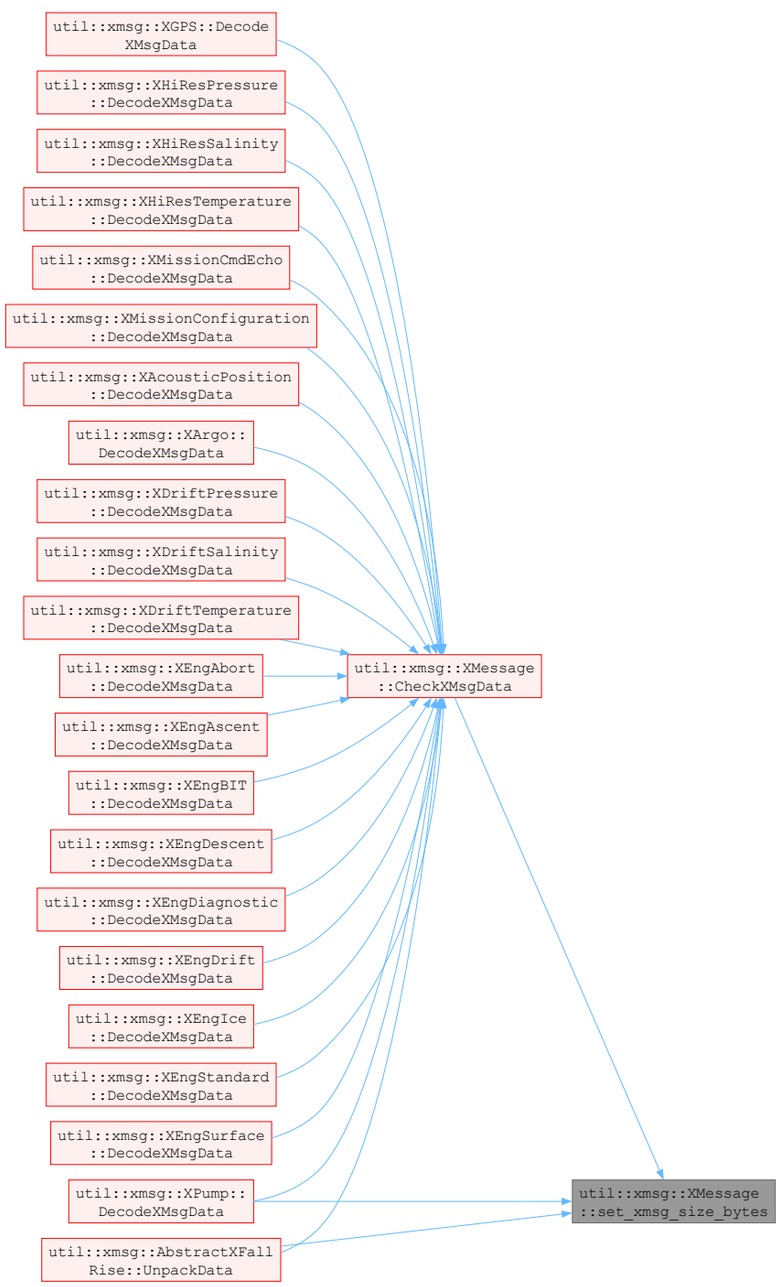
References util::xmsg::XMessage::xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XPump::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::Unpa

Here is the call graph for this function:



Here is the caller graph for this function:



10.102.3.12 xmsg_index()

```
uint8_t util::xmsg::XMessageIndexable::xmsg_index () const  [inline], [inherited]
```

Get the index of this X-Message.

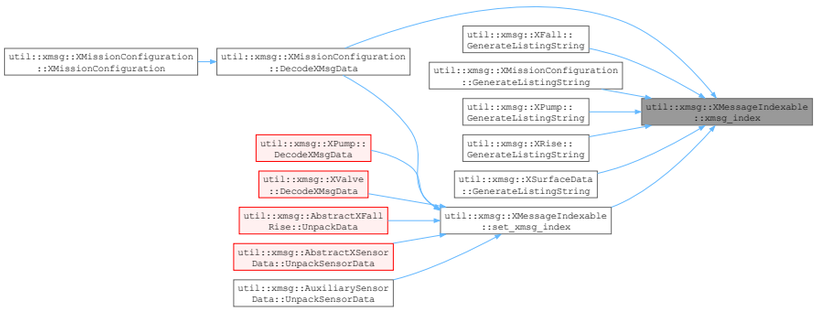
Returns

The index of this X-Message.

References util::xmsg::XMessageIndexable::xmsg_index_.

Referenced by DecodeXMsgData(), util::xmsg::XFall::GenerateListingString(), GenerateListingString(), util::xmsg::XPump::GenerateListing
util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(), and util::xmsg::XMessageIndexable::set_xm

Here is the caller graph for this function:



10.102.3.13 xmsg_size_bytes()

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes () const  [inline], [inherited]
```

Get the size of the X-Message, in bytes.

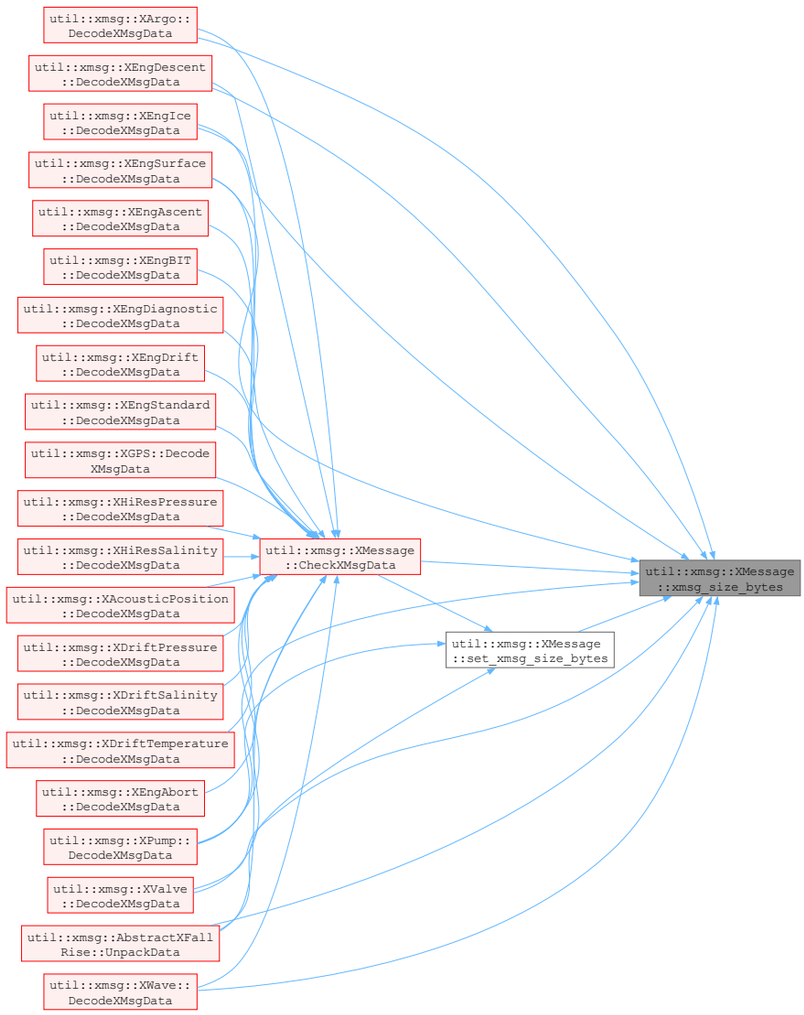
Returns

The size of the X-Message, in bytes.

References util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgI
util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(),
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), util::xmsg::XMessage::set_xmsg_size_bytes(),
and util::xmsg::AbstractXFallRise::UnpackData().

Here is the caller graph for this function:



10.102.4 Member Data Documentation

10.102.4.1 alamo_mission_configuration_

```
shared_ptr<XAlamoMissionConfiguration> util::xmsg::XMissionConfiguration::alamo_mission_configuration↩
_  [private]
```

A shared pointer to the ALAMO mission configuration object.

Note

This pointer is only used for ALAMO-type mission configuration messages. If this is not an ALAMO-type message, then this pointer will be null.

Referenced by DecodeXMsgData(), and GenerateListingString().

10.102.4.2 id_dive_

```
int16_t util::xmsg::XMessage::id_dive_  [private], [inherited]
```

The dive or cycle number of this message.

Referenced by util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::set_id_dive().

10.102.4.3 id_instrument_

```
uint16_t util::xmsg::XMessage::id_instrument_  [private], [inherited]
```

The unique WHOI identifier of this instrument.

Referenced by util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::set_id_instrument().

10.102.4.4 id_xmsg_type_

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type_  [private], [inherited]
```

The X-message type identifier.

Referenced by util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::set_id_xmsg_type().

10.102.4.5 xmsg_index_

```
uint8_t util::xmsg::XMessageIndexable::xmsg_index_  [private], [inherited]
```

The index of this X-Message.

Referenced by util::xmsg::XMessageIndexable::set_xmsg_index(), and util::xmsg::XMessageIndexable::xmsg_index().

10.102.4.6 xmsg_size_bytes_

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes_  [private], [inherited]
```

The size of the X message, in bytes.

Referenced by util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes().

The documentation for this class was generated from the following file:

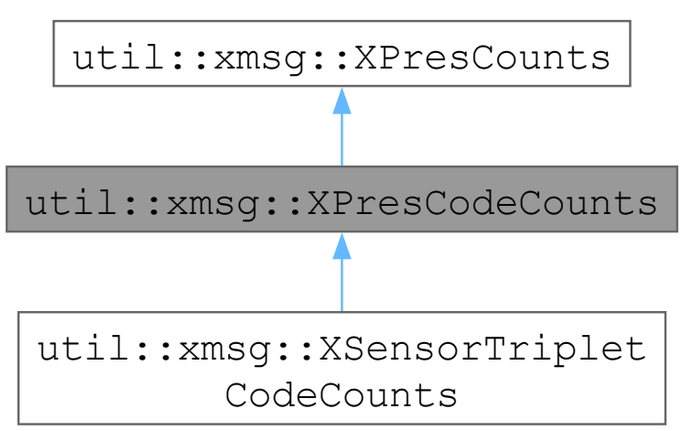
- data/xmsg/XMissionConfiguration.h

10.103 util::xmsg::XPresCodeCounts Class Reference

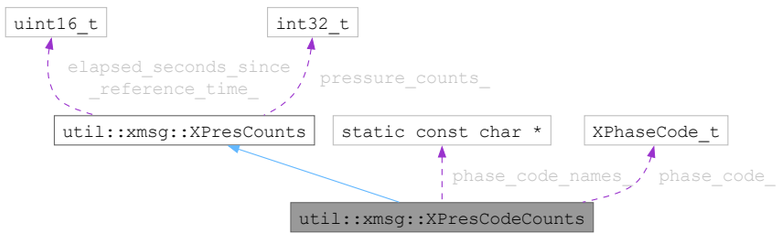
Class representing a raw pressure measurement and phase code for a rise or fall data X-Message.

```
#include <XPresCodeCounts.h>
```

Inheritance diagram for util::xmsg::XPresCodeCounts:



Collaboration diagram for util::xmsg::XPresCodeCounts:



Public Member Functions

- XPresCodeCounts (uint16_t sec, XPhaseCode_t code, int32_t counts)
  The constructor.
- ~XPresCodeCounts ()
  The destructor.
- uint16_t elapsed_seconds_since_reference_time () const
  Get the elapsed time since the reference time, in seconds.
- const string GenerateListingString (uint16_t pg, uint16_t po, float pc) const
  Generate a listing entry string from this pressure measurement object.
- XPhaseCode_t phase_code () const
  Get the phase code.
- const char * phase_code_name () const
  Get the phase code name C-sting.
- int32_t **pressure_counts** () const
- template<typename T = float>
  T PressureValue (uint16_t pg, uint16_t po, float pres_corr) const
  Get the measured pressure value, in dBar.

Static Public Member Functions

- static const char * GetPhaseCodeName (XPhaseCode_t code)
  Get the phase code name C-string for a given phase code value.

Private Attributes

- uint16_t elapsed_seconds_since_reference_time_
  The elapsed time since the reference time, in seconds.
- XPhaseCode_t phase_code_
  The phase code of this pressure measurement.
- int32_t pressure_counts_
  The pressure measurement, in raw sensor counts.

Static Private Attributes

- static const char * phase_code_names_ [ ]
  The phase code names.

10.103.1 Detailed Description

Class representing a raw pressure measurement and phase code for a rise or fall data X-Message.

10.103.2 Constructor & Destructor Documentation

10.103.2.1 XPresCodeCounts()

```
util::xmsg::XPresCodeCounts::XPresCodeCounts (
            uint16_t sec,
            XPhaseCode_t code,
            int32_t counts)  [inline]
```

The constructor.

Parameters

in	*sec*	The elapsed time since the reference time, in seconds.
in	*code*	The phase code of this pressure measurement.
in	*counts*	The pressure measurement, in raw sensor counts.

10.103.2.2 ~XPresCodeCounts()

```
util::xmsg::XPresCodeCounts::~XPresCodeCounts ()  [inline]
```

The destructor.

10.103.3 Member Function Documentation

10.103.3.1 elapsed_seconds_since_reference_time()

```
uint16_t util::xmsg::XPresCounts::elapsed_seconds_since_reference_time () const  [inline], [inherited]
```

Get the elapsed time since the reference time, in seconds.

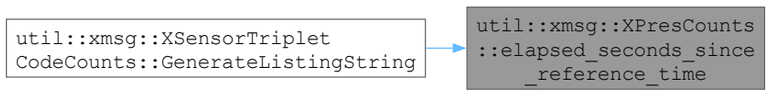
Returns

The elapsed time since the reference time, in seconds.

References util::xmsg::XPresCounts::elapsed_seconds_since_reference_time_.

Referenced by util::xmsg::XSensorTripletCodeCounts::GenerateListingString().

Here is the caller graph for this function:



10.103.3.2 GenerateListingString()

```
const string util::xmsg::XPresCodeCounts::GenerateListingString (
            uint16_t pg,
            uint16_t po,
            float pc) const  [inline]
```

Generate a listing entry string from this pressure measurement object.

Parameters

in	*pg*	The pressure sensor sensitivity scale factor, in LSB/dBar.
in	*po*	The pressure sensor offset, in dBar.
in	*pc*	The pressure correction, in dBar, to be subtracted computed pressure value.

Exceptions

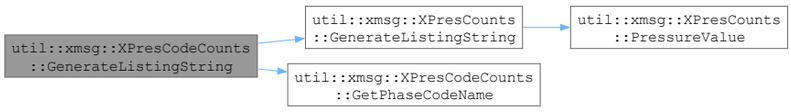
log::MessageException	if an error occurs while generating the listing string.

Returns

A listing entry string representation of this pressure measurement object.

References CATCH_BLOCK, util::xmsg::XPresCounts::GenerateListingString(), GetPhaseCodeName(), and phase_code_.

Here is the call graph for this function:



10.103.3.3 GetPhaseCodeName()

```
static const char * util::xmsg::XPresCodeCounts::GetPhaseCodeName (
            XPhaseCode_t code)  [inline], [static]
```

Get the phase code name C-string for a given phase code value.

Parameters

in	*code*	The phase code.

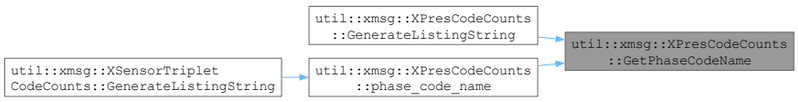
Returns

The phase code name C-string for the given phase code value.

References CATCH_BLOCK, EXC_THROW, and phase_code_names_.

Referenced by GenerateListingString(), and phase_code_name().

Here is the caller graph for this function:



10.103.3.4 phase_code()

```
XPhaseCode_t util::xmsg::XPresCodeCounts::phase_code () const  [inline]
```

Get the phase code.

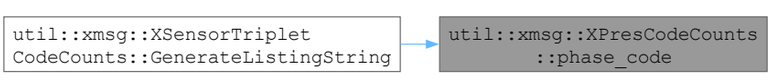
Returns

The phase code.

References phase_code_.

Referenced by util::xmsg::XSensorTripletCodeCounts::GenerateListingString().

Here is the caller graph for this function:



10.103.3.5 phase_code_name()

```
const char * util::xmsg::XPresCodeCounts::phase_code_name () const  [inline]
```

Get the phase code name C-sting.

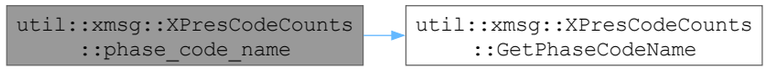
Returns

The phase code name C-sting.

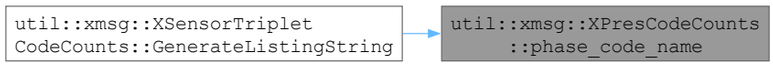
References GetPhaseCodeName(), and phase_code_.

Referenced by util::xmsg::XSensorTripletCodeCounts::GenerateListingString().

Here is the call graph for this function:



Here is the caller graph for this function:



10.103.3.6 PressureValue()

```
template<typename T = float>
T util::xmsg::XPresCounts::PressureValue (
            uint16_t pg,
            uint16_t po,
            float pres_corr) const  [inline], [inherited]
```

Get the measured pressure value, in dBar.

Parameters

in	*pg*	The pressure sensor sensitivity scale factor, in LSB/dBar.
in	*po*	The pressure sensor offset, in dBar.
in	*pres_corr*	The pressure correction, in dBar, to be subtracted computed pressure value.

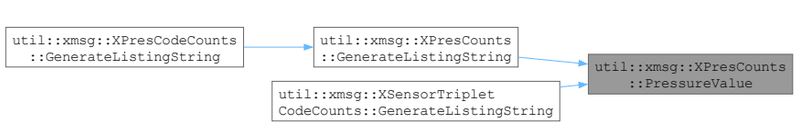
Returns

The measured pressure value, in dBar.

References util::xmsg::XPresCounts::pressure_counts_, XMSG_SENS_BAD_PRES, and XMSG_SENS_MAX_PRES.

Referenced by util::xmsg::XPresCounts::GenerateListingString(), and util::xmsg::XSensorTripletCodeCounts::GenerateListingString().

Here is the caller graph for this function:



10.103.4 Member Data Documentation

10.103.4.1 elapsed_seconds_since_reference_time_

```
uint16_t util::xmsg::XPresCounts::elapsed_seconds_since_reference_time_  [private], [inherited]
```

The elapsed time since the reference time, in seconds.

Referenced by util::xmsg::XPresCounts::elapsed_seconds_since_reference_time(), and util::xmsg::XPresCounts::GenerateListingString()

10.103.4.2 phase_code_

```
XPhaseCode_t util::xmsg::XPresCodeCounts::phase_code_  [private]
```

The phase code of this pressure measurement.

Referenced by GenerateListingString(), phase_code(), and phase_code_name().

10.103.4.3 phase_code_names_

```
const char * util::xmsg::XPresCodeCounts::phase_code_names_  [static], [private]
```

Initial value:

```
= {
        "unknown",
        "start of descent",
        "first descent pump",
        "seek to drift pres",
        "start of drift",
        "seek during drift",
        "start of descent to profile",
        "start of ascent",
        "end of ascent",
        "ice turn-around",
        "descending",
        "drift/park",
        "descending to profile",
        "ascending",
        "surface",
        "invalid phase code"
        }
```

The phase code names.

Referenced by GetPhaseCodeName().

10.103.4.4 pressure_counts_

```
int32_t util::xmsg::XPresCounts::pressure_counts_  [private], [inherited]
```

The pressure measurement, in raw sensor counts.

Referenced by util::xmsg::XPresCounts::PressureValue().

The documentation for this class was generated from the following files:

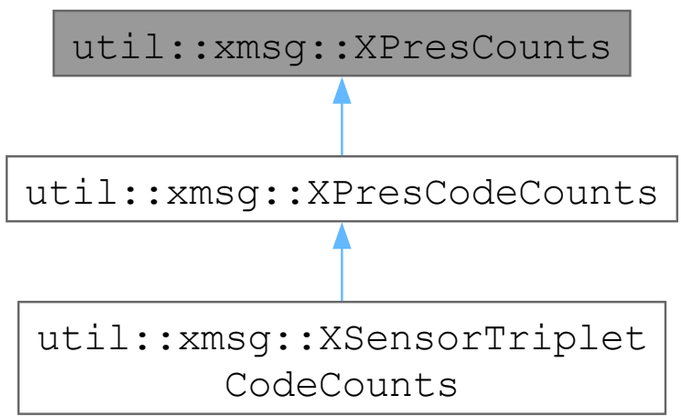
- data/xmsg/common/XPresCodeCounts.h
- data/xmsg/common/XPresCodeCounts.cpp

10.104 util::xmsg::XPresCounts Class Reference

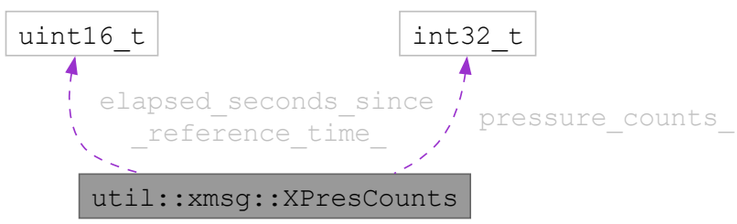
Class representing a raw pressure measurement for a rise or fall data X-Message.

`#include <XPresCounts.h>`

Inheritance diagram for util::xmsg::XPresCounts:



Collaboration diagram for util::xmsg::XPresCounts:



Public Member Functions

- XPresCounts (uint16_t sec, int32_t counts)
  The parameterized constructor.
- ~XPresCounts ()
  The destructor.
- uint16_t elapsed_seconds_since_reference_time () const
  Get the elapsed time since the reference time, in seconds.
- const string GenerateListingString (uint16_t pg, uint16_t po, float pc) const
  Generate a listing entry string from this pressure measurement object.
- int32_t **pressure_counts** () const
- template<typename T = float>
  T PressureValue (uint16_t pg, uint16_t po, float pres_corr) const
  Get the measured pressure value, in dBar.

Private Attributes

- uint16_t elapsed_seconds_since_reference_time_
  The elapsed time since the reference time, in seconds.
- int32_t pressure_counts_
  The pressure measurement, in raw sensor counts.

Friends

- class **XDiveData**

10.104.1 Detailed Description

Class representing a raw pressure measurement for a rise or fall data X-Message.

10.104.2 Constructor & Destructor Documentation

10.104.2.1 XPresCounts()

```
util::xmsg::XPresCounts::XPresCounts (
            uint16_t sec,
            int32_t counts)  [inline]
```

The parameterized constructor.

Parameters

in	*sec*	The elapsed time since the reference time, in seconds.
in	*counts*	The pressure measurement, in raw counts.

10.104.2.2 ~XPresCounts()

```
util::xmsg::XPresCounts::~XPresCounts ()  [inline]
```

The destructor.

10.104.3 Member Function Documentation

10.104.3.1 elapsed_seconds_since_reference_time()

```
uint16_t util::xmsg::XPresCounts::elapsed_seconds_since_reference_time () const  [inline]
```

Get the elapsed time since the reference time, in seconds.

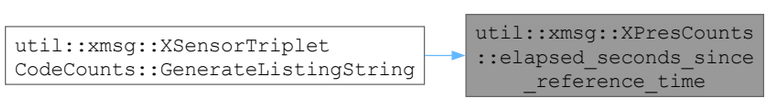
Returns

The elapsed time since the reference time, in seconds.

References elapsed_seconds_since_reference_time_.

Referenced by util::xmsg::XSensorTripletCodeCounts::GenerateListingString().

Here is the caller graph for this function:



10.104.3.2 GenerateListingString()

```
const string util::xmsg::XPresCounts::GenerateListingString (
            uint16_t pg,
            uint16_t po,
            float pc) const  [inline]
```

Generate a listing entry string from this pressure measurement object.

Parameters

in	*pg*	The pressure sensor sensitivity scale factor, in LSB/dBar.
in	*po*	The pressure sensor offset, in dBar.
in	*pc*	The pressure correction, in dBar, to be subtracted computed pressure value.

Exceptions

log::MessageException	if an error occurs while generating the listing string.

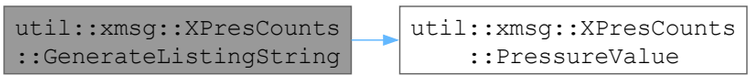
Returns

A listing entry string representation of this pressure measurement object.

References CATCH_BLOCK, elapsed_seconds_since_reference_time_, and PressureValue().

Referenced by util::xmsg::XPresCodeCounts::GenerateListingString().

Here is the call graph for this function:



Here is the caller graph for this function:

10.104.3.3 PressureValue()

```
template<typename T = float>
T util::xmsg::XPresCounts::PressureValue (
            uint16_t pg,
            uint16_t po,
            float pres_corr) const  [inline]
```

Get the measured pressure value, in dBar.

Parameters

in	*pg*	The pressure sensor sensitivity scale factor, in LSB/dBar.
in	*po*	The pressure sensor offset, in dBar.
in	*pres_corr*	The pressure correction, in dBar, to be subtracted computed pressure value.

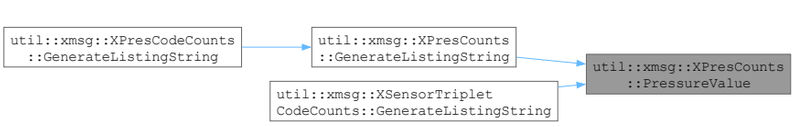
Returns

The measured pressure value, in dBar.

References pressure_counts_, XMSG_SENS_BAD_PRES, and XMSG_SENS_MAX_PRES.

Referenced by GenerateListingString(), and util::xmsg::XSensorTripletCodeCounts::GenerateListingString().

Here is the caller graph for this function:



10.104.4 Member Data Documentation

10.104.4.1 elapsed_seconds_since_reference_time_

```
uint16_t util::xmsg::XPresCounts::elapsed_seconds_since_reference_time_  [private]
```

The elapsed time since the reference time, in seconds.

Referenced by elapsed_seconds_since_reference_time(), and GenerateListingString().

10.104.4.2 pressure_counts_

```
int32_t util::xmsg::XPresCounts::pressure_counts_  [private]
```

The pressure measurement, in raw sensor counts.

Referenced by PressureValue().

The documentation for this class was generated from the following file:

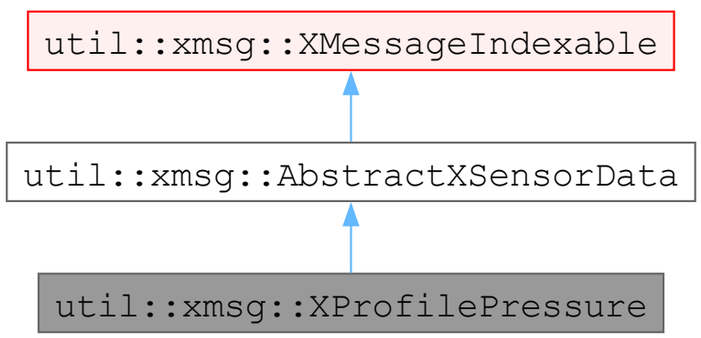
- data/xmsg/common/XPresCounts.h

10.105 util::xmsg::XProfilePressure Class Reference

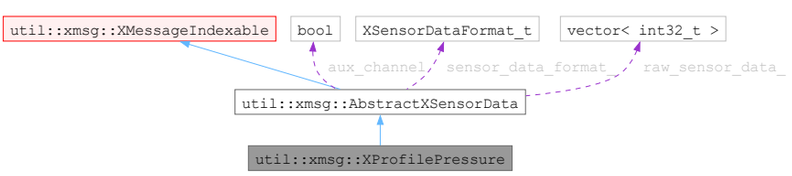
Class representing a profile pressure data X-Message.

```
#include <XProfilePressure.h>
```

Inheritance diagram for util::xmsg::XProfilePressure:



Collaboration diagram for util::xmsg::XProfilePressure:



Public Member Functions

- XProfilePressure (int16_t dive, uint16_t whoi, const vector< uint8_t > &data)

  Parameterized constructor.
- ~XProfilePressure ()

  Destructor.
- const uint8_t * CheckXMsgData (const vector< uint8_t > &data, int id, int minBytes, bool parseSize=true)

  Check the X-message for the correct ID, minimum size, and message terminator.
- void DecodeXMsgData (const vector< uint8_t > &data)

  Decode the bin-averaged profile pressure X-Message from the binary message data.
- int16_t id_dive () const

  Get the dive or cycle number of this X-Message.
- uint16_t id_instrument () const

  Get the unique instrument identifier of this X-Message.
- XMsgTypeId_t id_xmsg_type () const

  Get the enumerated X-Message type identifier.
- bool **is_aux_channel** () const
- size_t num_samples () const

  Get the number of samples in this message.
- const vector< int32_t > & raw_sensor_data () const

  Get the raw sensor data vector.
- int32_t Sample (size_t i) const

  Get the raw integer value of the i^{th} sample.
- template<typename T = double>
  T SampleValue (size_t i, int16_t gain, int16_t offs) const

  Get the i^{th} sensor value in scientific units.
- template<typename T = double>
  T SampleValueAsPressure (size_t i, int16_t gain, int16_t offs) const

  Get the i^{th} pressure sensor value, in dBar.
- template<typename T = double>
  T SampleValueAsSalinity (size_t i, int16_t gain, int16_t offs) const

  Get the i^{th} salinity sensor value, in PSU.
- template<typename T = double>
  T SampleValueAsTemperature (size_t i, int16_t gain, int16_t offs) const

  Get the i^{th} temperature sensor value, in degC.
- XSensorDataFormat_t sensor_data_format () const

  Get the enumerated sensor data format value.
- uint8_t xmsg_index () const

  Get the index of this X-Message.
- uint16_t xmsg_size_bytes () const

  Get the size of the X-Message, in bytes.

Protected Member Functions

- void set_id_dive (int16_t id_dive)

  Set the dive or cycle number of this message.
- void set_id_instrument (uint16_t id_instrument)

  Set the unique instrument identifier.
- void set_id_xmsg_type (XMsgTypeId_t id_xmsg_type)

  Set the X-Message type identifier.
- void set_raw_sensor_data (const vector< int32_t > &raw_sensor_data)

  Set raw integer counts of the measured sensor data.
- void set_sensor_data_format (XSensorDataFormat_t sensor_data_format)

  Set the sensor data format.
- void set_xmsg_index (uint8_t xmsg_index)

  Set the index of this X-Message.
- void set_xmsg_size_bytes (uint16_t xmsg_size_bytes)

  Set the size of the X-Message, in bytes.
- virtual vector< int32_t > UnpackSensorData (const vector< uint8_t > &data)

  Unpack the sensor data from the binary message.

Protected Attributes

- bool **aux_channel**
- vector< int32_t > raw_sensor_data_

  The raw integer counts of the measured sensor data.
- XSensorDataFormat_t sensor_data_format_

  The enumerated sensor data packing format.

Private Attributes

- int16_t id_dive_

  The dive or cycle number of this message.
- uint16_t id_instrument_

  The unique WHOI identifier of this instrument.
- XMsgTypeId_t id_xmsg_type_

  The X-message type identifier.
- uint8_t xmsg_index_

  The index of this X-Message.
- uint16_t xmsg_size_bytes_

  The size of the X message, in bytes.

10.105.1 Detailed Description

Class representing a profile pressure data X-Message.

10.105.2 Constructor & Destructor Documentation

10.105.2.1 XProfilePressure()

```
util::xmsg::XProfilePressure::XProfilePressure (
            int16_t dive,
            uint16_t whoi,
            const vector< uint8_t > & data)  [inline]
```

Parameterized constructor.

Parameters

in	*dive*	The dive or cycle number of this message.
in	*whoi*	The unique WHOI identifier.
in	*data*	The binary profile pressure data X-message data to decode.

Exceptions

util::log::MessageException	if an decode error occurs.

References DecodeXMsgData().

Here is the call graph for this function:

10.105.2.2 ~XProfilePressure()

```
util::xmsg::XProfilePressure::~XProfilePressure ()  [inline]
```

Destructor.

10.105.3 Member Function Documentation

10.105.3.1 CheckXMsgData()

```
const uint8_t * util::xmsg::XMessage::CheckXMsgData (
            const vector< uint8_t > & data,
            int id,
            int minBytes,
            bool parseSize = true)  [inline], [inherited]
```

Check the X-message for the correct ID, minimum size, and message terminator.

Parameters

in	*data*	The X-message binary data to check.
in	*id*	The expected ID of the X-message. If this field is less that 0, then the ID check will be skipped.
in	*minBytes*	The minimum number of bytes expected in a valid message.
in	*parseSize*	If TRUE, then the message size field will be parsed from the X-Message. Otherwise, the size field will not be parsed.

Exceptions

util::log::MessageException	if • the message does not contain the expected ID, • the message does not contain the minimum number of expected bytes, • the message does not contain a terminator (';') in the correct location, or, • any other error occurs.

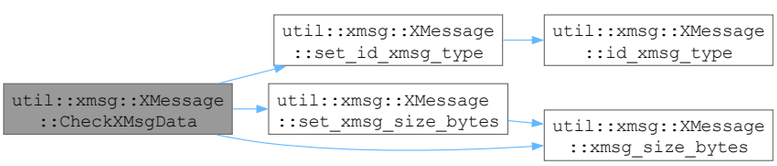
Returns

A pointer to the underlying data buffer of data.

References CATCH_BLOCK, EXC_THROW, lwrn, util::xmsg::XMessage::set_id_xmsg_type(), util::xmsg::XMessage::set_xmsg_size_byte
u16, and util::xmsg::XMessage::xmsg_size_bytes().

Referenced by util::xmsg::XAcousticPosition::DecodeXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XDriftPressure::Dec
util::xmsg::XDriftSalinity::DecodeXMsgData(), util::xmsg::XDriftTemperature::DecodeXMsgData(), util::xmsg::XEngAbort::DecodeXMsgDa
util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngBIT::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(),
util::xmsg::XEngDiagnostic::DecodeXMsgData(), util::xmsg::XEngDrift::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(),
util::xmsg::XEngStandard::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XGPS::DecodeXMsgData(),
util::xmsg::XHiResPressure::DecodeXMsgData(), util::xmsg::XHiResSalinity::DecodeXMsgData(), util::xmsg::XHiResTemperature::Decod
util::xmsg::XMissionCmdEcho::DecodeXMsgData(), util::xmsg::XMissionConfiguration::DecodeXMsgData(), DecodeXMsgData(),
util::xmsg::XProfileSalinity::DecodeXMsgData(), util::xmsg::XProfileTemperature::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgD
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the call graph for this function:



10.105.3.2 DecodeXMsgData()

```
void util::xmsg::XProfilePressure::DecodeXMsgData (
            const vector< uint8_t > & data)  [inline], [virtual]
```

Decode the bin-averaged profile pressure X-Message from the binary message data.

Parameters

in	*data*	The binary message data to decode.

Exceptions

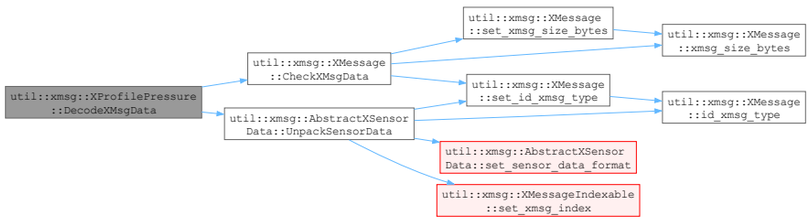
util::log::MessageException	if the message is invalid or an error occurs during decoding.

Implements util::xmsg::AbstractXSensorData.

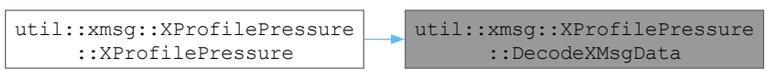
References CATCH_BLOCK, util::xmsg::XMessage::CheckXMsgData(), EXC_THROW, util::xmsg::AbstractXSensorData::UnpackSensorD and util::xmsg::XMSG_TYPE_ID_PRES0.

Referenced by XProfilePressure().

Here is the call graph for this function:



Here is the caller graph for this function:



10.105.3.3 id_dive()

```
int16_t util::xmsg::XMessage::id_dive () const  [inline], [inherited]
```

Get the dive or cycle number of this X-Message.

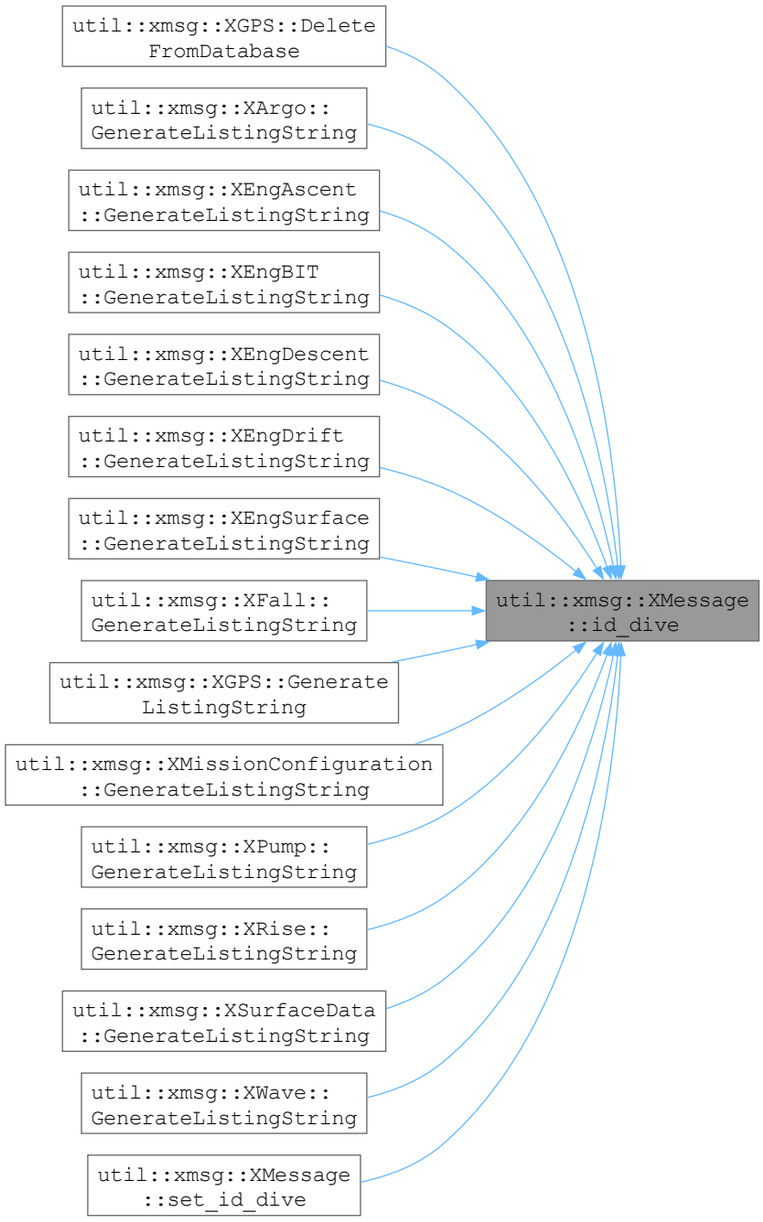
Returns

The dive or cycle number of this X-Message.

References util::xmsg::XMessage::id_dive_.

Referenced by util::xmsg::XGPS::DeleteFromDatabase(), util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateLis util::xmsg::XEngBIT::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingStr util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing util::xmsg::XSurfaceData::GenerateListingString(), util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_dive().

Here is the caller graph for this function:



10.105.3.4 id_instrument()

```
uint16_t util::xmsg::XMessage::id_instrument () const  [inline], [inherited]
```

Get the unique instrument identifier of this X-Message.

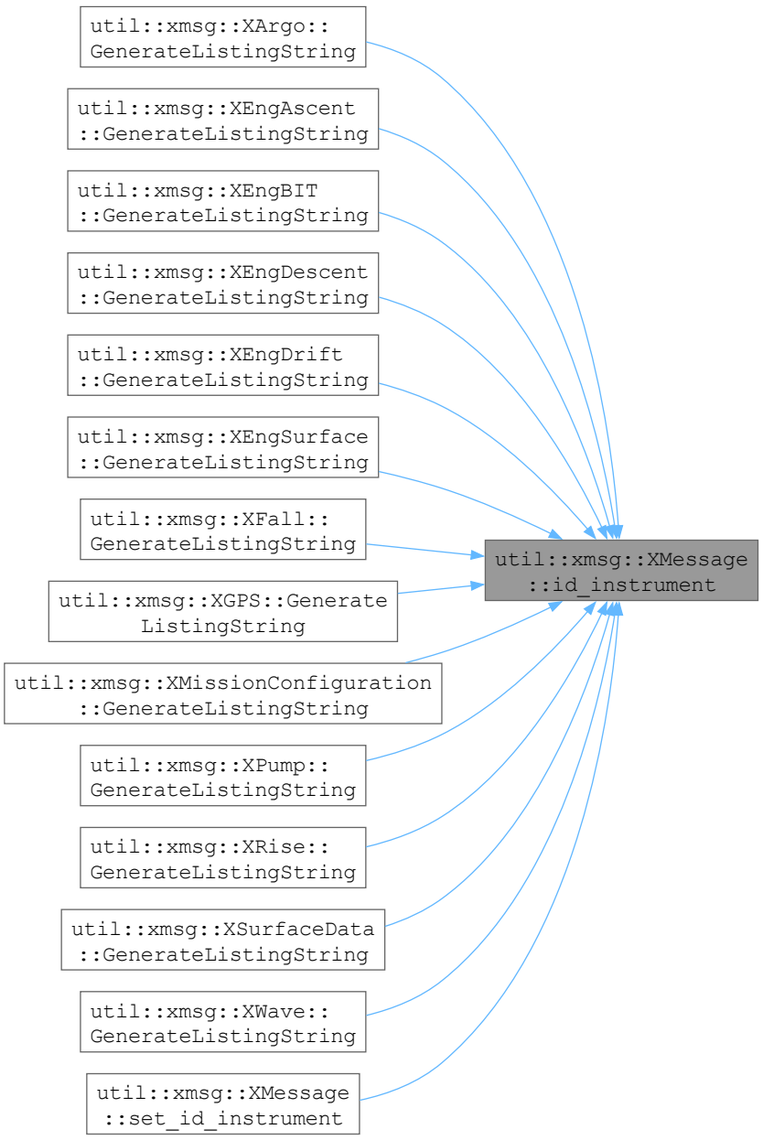
Returns

The unique instrument identifier of this X-Message.

References util::xmsg::XMessage::id_instrument_.

Referenced by util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngBIT::Generate
util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), util::xmsg::XEngSurface::GenerateListin
util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingS
util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(),
util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_instrument().

Here is the caller graph for this function:



10.105.3.5 id_xmsg_type()

`XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type () const [inline], [inherited]`

Get the enumerated X-Message type identifier.

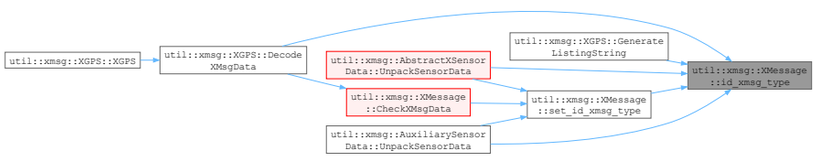
Returns

The enumerated X-Message type identifier.

References util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMessage::set_id_xmsg_typ
util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the caller graph for this function:



10.105.3.6 num_samples()

```
size_t util::xmsg::AbstractXSensorData::num_samples () const  [inline], [inherited]
```

Get the number of samples in this message.

Returns

The number of samples in this message.

References util::xmsg::AbstractXSensorData::raw_sensor_data_.

10.105.3.7 raw_sensor_data()

```
const vector< int32_t > & util::xmsg::AbstractXSensorData::raw_sensor_data () const  [inline],
[inherited]
```

Get the raw sensor data vector.

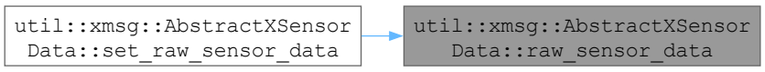
Returns

The raw sensor data vector.

References util::xmsg::AbstractXSensorData::raw_sensor_data_.

Referenced by util::xmsg::AbstractXSensorData::set_raw_sensor_data().

Here is the caller graph for this function:



10.105.3.8 Sample()

```
int32_t util::xmsg::AbstractXSensorData::Sample (
            size_t i) const  [inline], [inherited]
```

Get the raw integer value of the i^{th} sample.

Parameters

in	*i*	The index of the sensor value.

Returns

The raw integer value of the i^{th} ith sample.

References util::xmsg::AbstractXSensorData::raw_sensor_data_.

10.105.3.9 SampleValue()

```
template<typename T = double>
T util::xmsg::AbstractXSensorData::SampleValue (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline], [inherited]
```

Get the i^{th} sensor value in scientific units.

Parameters

in	*i*	The index of the sensor value.
in	*gain*	The sensor sensitivity scale factor.
in	*offs*	The sensor offset.

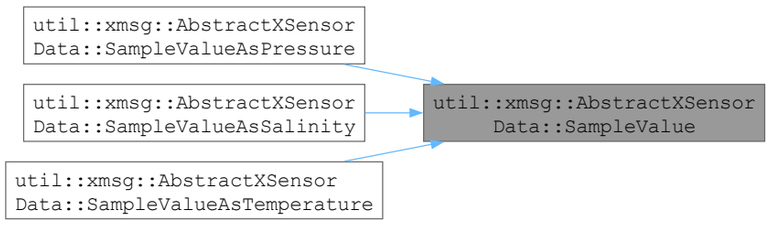
Returns

The i^{th} sensor value in scientific units.

References CATCH_BLOCK, and util::xmsg::AbstractXSensorData::raw_sensor_data_.

Referenced by util::xmsg::AbstractXSensorData::SampleValueAsPressure(), util::xmsg::AbstractXSensorData::SampleValueAsSalinity(), and util::xmsg::AbstractXSensorData::SampleValueAsTemperature().

Here is the caller graph for this function:



10.105.3.10 SampleValueAsPressure()

```
template<typename T = double>
T util::xmsg::AbstractXSensorData::SampleValueAsPressure (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline], [inherited]
```

Get the i^{th} pressure sensor value, in dBar.

Parameters

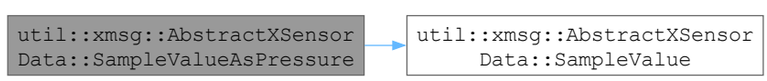
in	*i*	The index of the sensor value.
in	*gain*	The pressure sensor sensitivity scale factor, in LSB/dBar.
in	*offs*	The pressure sensor offset, in dBar.

Returns

The i^{th} pressure sensor value, in dBar.

References util::xmsg::AbstractXSensorData::SampleValue(), XMSG_SENS_BAD_PRES, and XMSG_SENS_MAX_PRES.

Here is the call graph for this function:



10.105.3.11 SampleValueAsSalinity()

```
template<typename T = double>
T util::xmsg::AbstractXSensorData::SampleValueAsSalinity (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline], [inherited]
```

Get the i^{th} salinity sensor value, in PSU.

Parameters

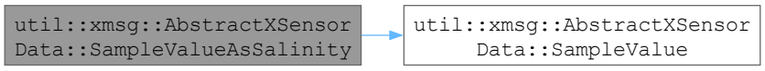
in	*i*	The index of the sensor value.
in	*gain*	The salinity sensor sensitivity scale factor, in LSB/PSU.
in	*offs*	The salinity sensor offset, in PSU.

Returns

The i^{th} salinity sensor value, in PSU.

References util::xmsg::AbstractXSensorData::SampleValue(), XMSG_SENS_BAD_SALN, and XMSG_SENS_MAX_SALN.

Here is the call graph for this function:



10.105.3.12 SampleValueAsTemperature()

```
template<typename T = double>
T util::xmsg::AbstractXSensorData::SampleValueAsTemperature (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline], [inherited]
```

Get the i^{th} temperature sensor value, in degC.

Parameters

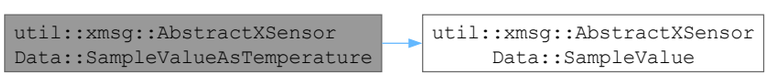
in	*i*	The index of the sensor value.
in	*gain*	The temperature sensor sensitivity scale factor, in LSB/degC.
in	*offs*	The temperature sensor offset, in degC.

Returns

The i^{th} temperature sensor value, in degC.

References util::xmsg::AbstractXSensorData::SampleValue(), XMSG_SENS_BAD_TEMP, and XMSG_SENS_MAX_TEMP.

Here is the call graph for this function:



10.105.3.13 sensor_data_format()

```
XSensorDataFormat_t util::xmsg::AbstractXSensorData::sensor_data_format () const  [inline], [inherited]
```

Get the enumerated sensor data format value.

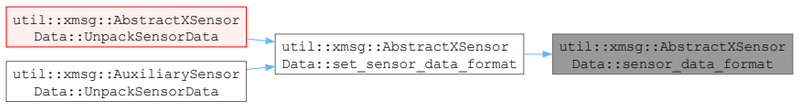
Returns

The enumerated sensor data format value.

References util::xmsg::AbstractXSensorData::sensor_data_format_.

Referenced by util::xmsg::AbstractXSensorData::set_sensor_data_format().

Here is the caller graph for this function:



10.105.3.14 set_id_dive()

```
void util::xmsg::XMessage::set_id_dive (
            int16_t id_dive)  [inline], [protected], [inherited]
```

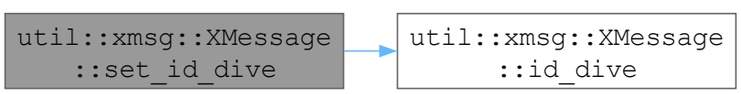
Set the dive or cycle number of this message.

Parameters

in	*id_dive*	The dive or cycle number of this message to set.

References util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::id_dive_.

Here is the call graph for this function:



10.105.3.15 set_id_instrument()

```
void util::xmsg::XMessage::set_id_instrument (
            uint16_t id_instrument)  [inline], [protected], [inherited]
```

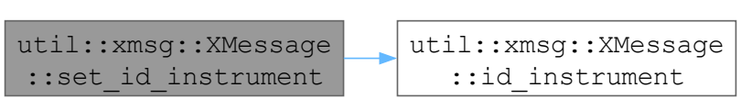
Set the unique instrument identifier.

Parameters

in	*id_instrument*	The unique instrument identifier to set.

References util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::id_instrument_.

Here is the call graph for this function:



10.105.3.16 set_id_xmsg_type()

```
void util::xmsg::XMessage::set_id_xmsg_type (
            XMsgTypeId_t id_xmsg_type)  [inline], [protected], [inherited]
```

Set the X-Message type identifier.

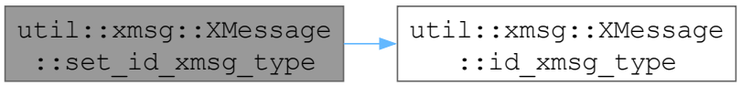
Parameters

in	*id_xmsg_type*	The X-Message type identifier to set.

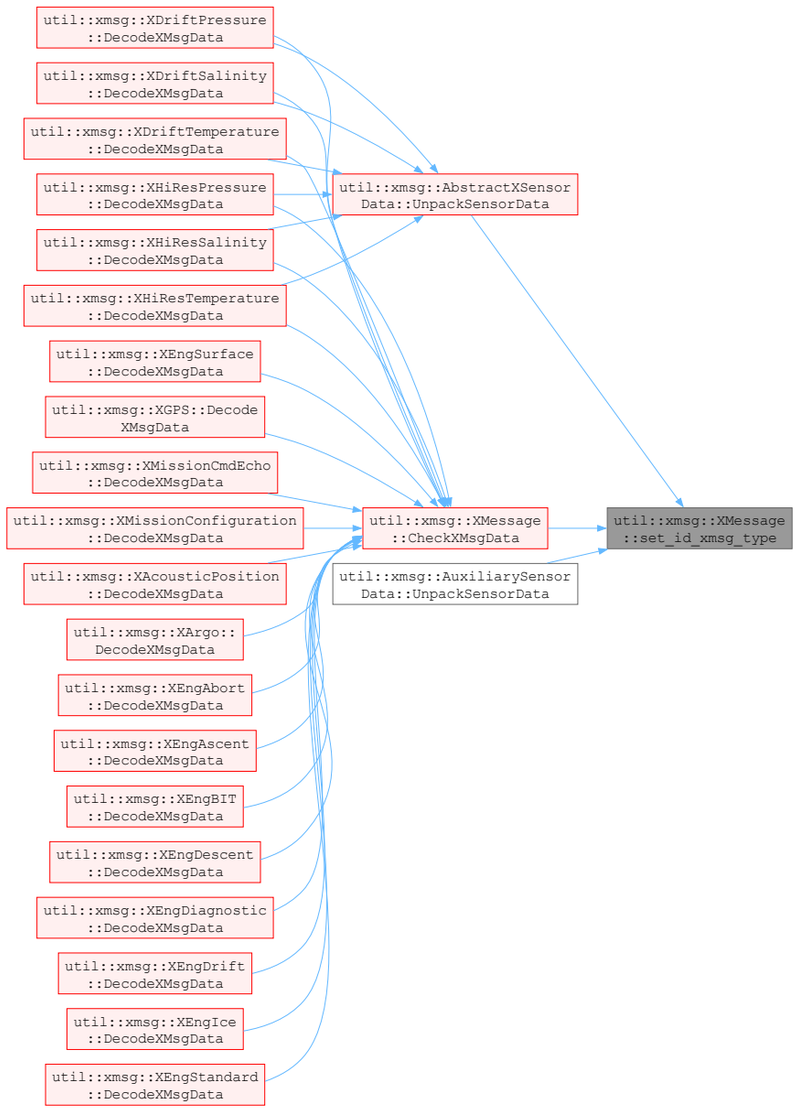
References util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.105.3.17 set_raw_sensor_data()

```
void util::xmsg::AbstractXSensorData::set_raw_sensor_data (
            const vector< int32_t > & raw_sensor_data)  [inline], [protected], [inherited]
```

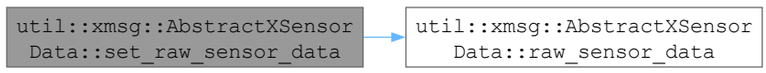
Set raw integer counts of the measured sensor data.

Parameters

in	*raw_sensor_data*	The data to set.

References util::xmsg::AbstractXSensorData::raw_sensor_data(), and util::xmsg::AbstractXSensorData::raw_sensor_data_.

Here is the call graph for this function:



10.105.3.18 set_sensor_data_format()

```
void util::xmsg::AbstractXSensorData::set_sensor_data_format (
            XSensorDataFormat_t sensor_data_format)  [inline], [protected], [inherited]
```

Set the sensor data format.

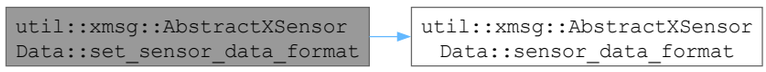
Parameters

in	*sensor_data_format*	The sensor data format to set.

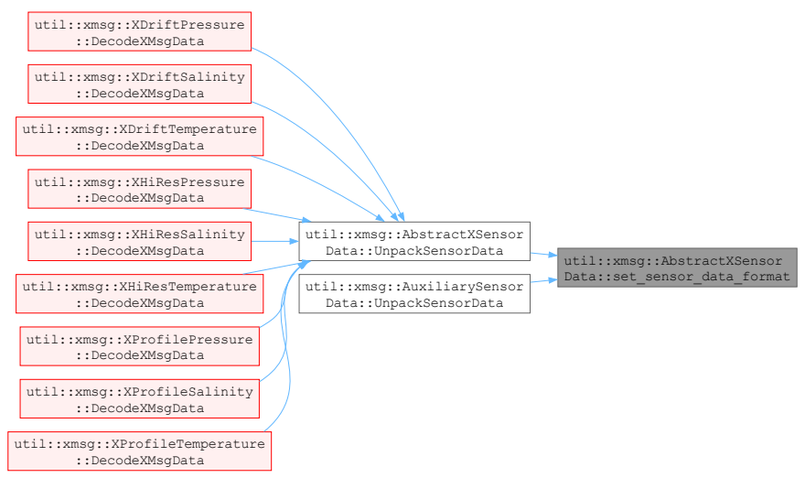
References util::xmsg::AbstractXSensorData::sensor_data_format(), and util::xmsg::AbstractXSensorData::sensor_data_format_.

Referenced by util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.105.3.19 set_xmsg_index()

```
void util::xmsg::XMessageIndexable::set_xmsg_index (
            uint8_t xmsg_index)  [inline], [protected], [inherited]
```

Set the index of this X-Message.

Parameters

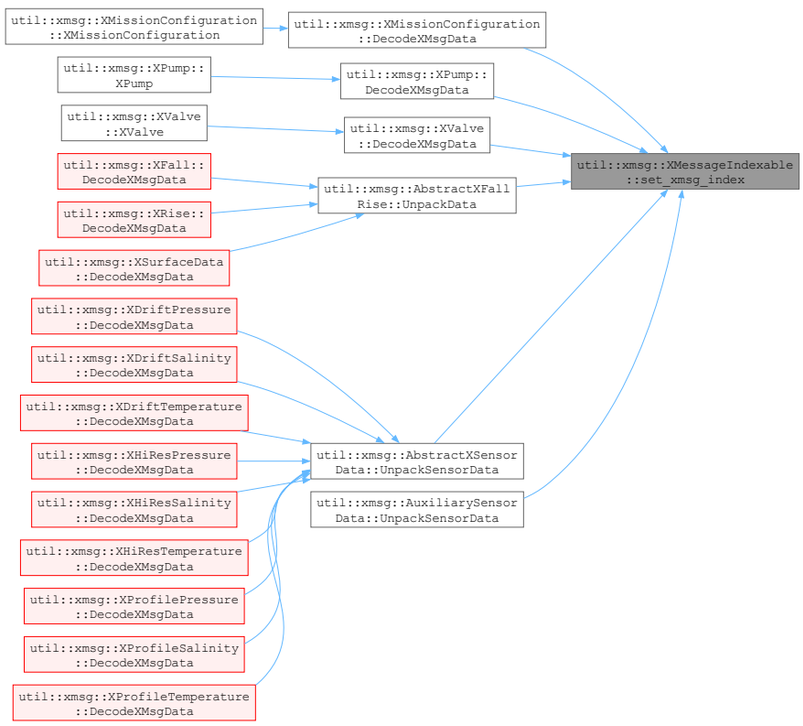
in	*xmsg_index*	The index of this X-Message.

References util::xmsg::XMessageIndexable::xmsg_index(), and util::xmsg::XMessageIndexable::xmsg_index_.

Referenced by util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(), util::xmsg::XValve::DecodeXMsgData(), util::xmsg::AbstractXFallRise::UnpackData(), util::xmsg::AbstractXSensorData::UnpackSensorDa and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:

Here is the caller graph for this function:



10.105.3.20 set_xmsg_size_bytes()

```
void util::xmsg::XMessage::set_xmsg_size_bytes (
            uint16_t xmsg_size_bytes)  [inline], [protected], [inherited]
```

Set the size of the X-Message, in bytes.

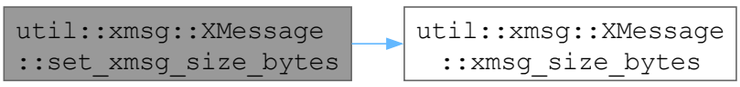
Parameters

in	*xmsg_size_bytes*	The size of the X-Message to set, in bytes.

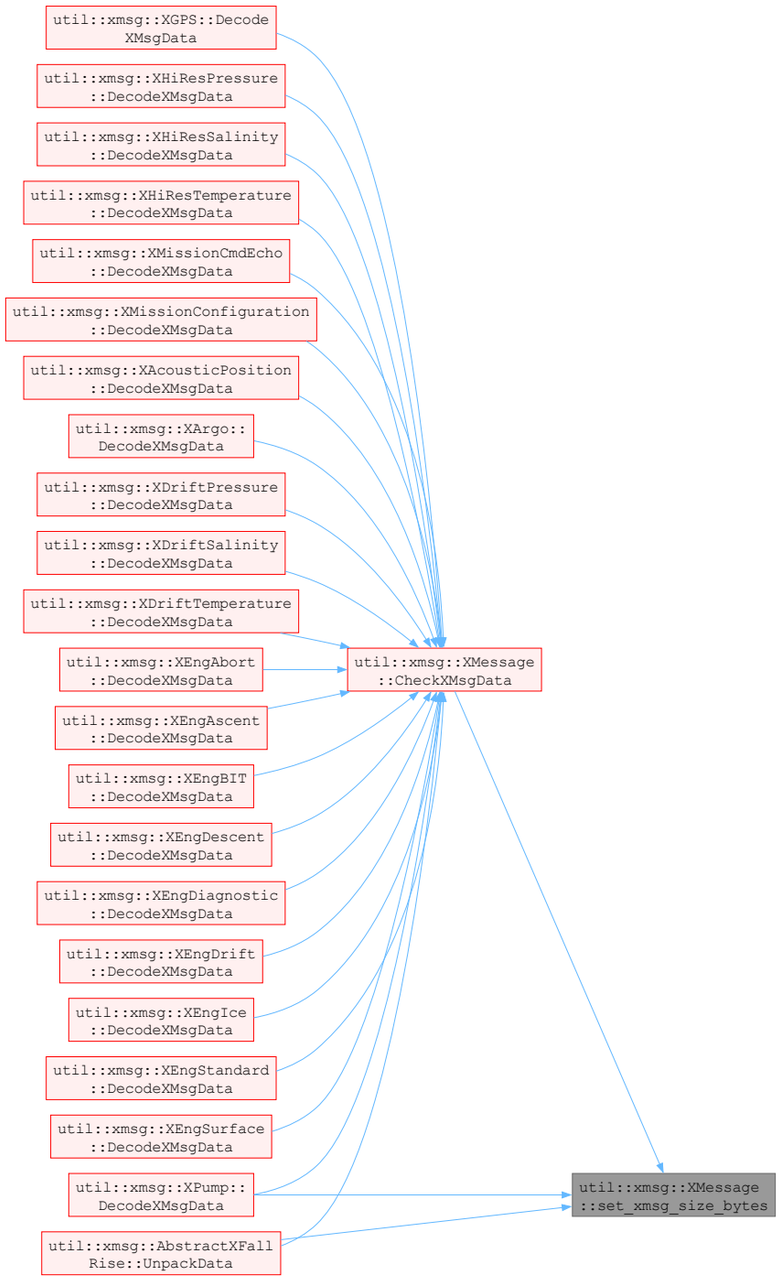
References util::xmsg::XMessage::xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XPump::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::Unpa

Here is the call graph for this function:



Here is the caller graph for this function:



10.105.3.21 UnpackSensorData()

```
virtual vector< int32_t > util::xmsg::AbstractXSensorData::UnpackSensorData (
            const vector< uint8_t > & data)  [inline], [protected], [virtual], [inherited]
```

Unpack the sensor data from the binary message.

This field unpacks the encoded sensor data from the message and stores the decoded sensor values in raw_sensor_data_. First the function checks the size of the message, and the data packing format. The message data is decoded according to the packing format type. Both first-difference and curvature encoding are supported. The encoding method is determined from the value of the high nibble of byte `data`[1].

Once this function completes successfully, then the data may be recovered in scientific units from the raw sensor values by applying the appropriate sensor gain and offset values.

Parameters

in	*data*	The binary message to unpack, in a vector of unsigned bytes.

Warning

The function assumes the message data is valid. Message validity should be ensured before this function is called.

Exceptions

util::log::MessageException	if • an error occurs while decoding the message, • the message is of insufficient length, or • an invalid packing format is encountered.

Returns

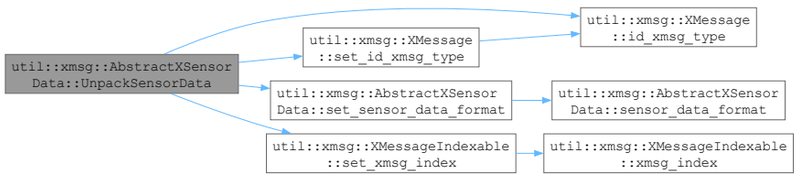
The sensor data raw counts, in a vector of signed integers.

Reimplemented in util::xmsg::AuxiliarySensorData.

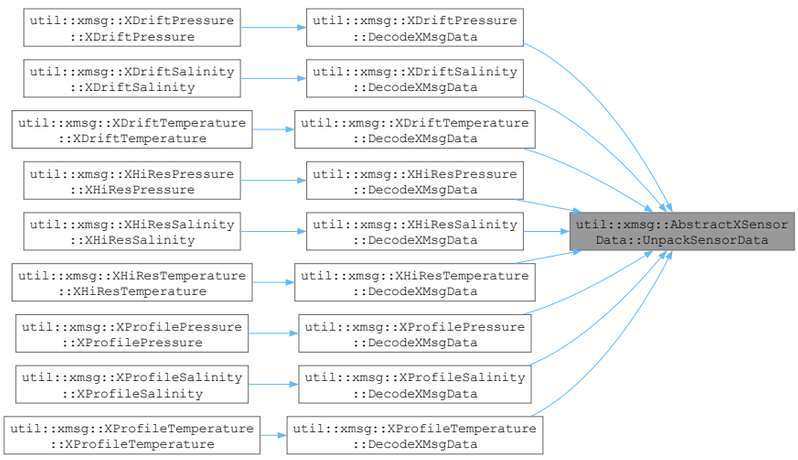
References CATCH_BLOCK, CATCH_NOTHROW, EXC_THROW, i24, util::xmsg::XMessage::id_xmsg_type(), ldbg, lwrn, util::xmsg::AbstractXSensorData::raw_sensor_data_, util::xmsg::XMessage::set_id_xmsg_type(), util::xmsg::AbstractXSensorData::s util::xmsg::XMessageIndexable::set_xmsg_index(), u16, u24, XMSG_SENS_MAX_SUB, util::xmsg::XMSG_TYPE_ID_PRES0, util::xmsg::XMSG_TYPE_ID_PRES_DRIFT8, util::xmsg::XMSG_TYPE_ID_PRES_HI_RES0, util::xmsg::XMSG_TYPE_ID_SALN0, util::xmsg::XMSG_TYPE_ID_SALN_DRIFT8, util::xmsg::XMSG_TYPE_ID_SALN_HI_RES0, util::xmsg::XMSG_TYPE_ID_TEMP0, util::xmsg::XMSG_TYPE_ID_TEMP_DRIFT8, util::xmsg::XMSG_TYPE_ID_TEMP_HI_RES0, util::xmsg::XSENS_FMT_CURVATURE, and util::xmsg::XSENS_FMT_DIFFERENCE.

Referenced by util::xmsg::XDriftPressure::DecodeXMsgData(), util::xmsg::XDriftSalinity::DecodeXMsgData(), util::xmsg::XDriftTemperatur util::xmsg::XHiResPressure::DecodeXMsgData(), util::xmsg::XHiResSalinity::DecodeXMsgData(), util::xmsg::XHiResTemperature::Decod DecodeXMsgData(), util::xmsg::XProfileSalinity::DecodeXMsgData(), and util::xmsg::XProfileTemperature::DecodeXMsgData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.105.3.22 xmsg_index()

```
uint8_t util::xmsg::XMessageIndexable::xmsg_index () const  [inline], [inherited]
```

Get the index of this X-Message.

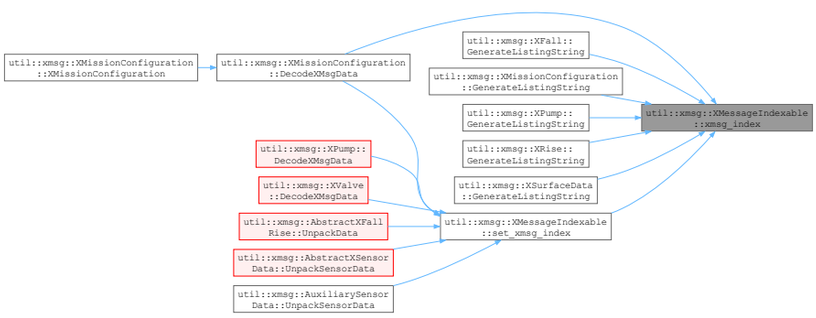
Returns

The index of this X-Message.

References util::xmsg::XMessageIndexable::xmsg_index_.

Referenced by util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing util::xmsg::XSurfaceData::GenerateListingString(), and util::xmsg::XMessageIndexable::set_xmsg_index().

Here is the caller graph for this function:



10.105.3.23 xmsg_size_bytes()

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes () const  [inline], [inherited]
```

Get the size of the X-Message, in bytes.

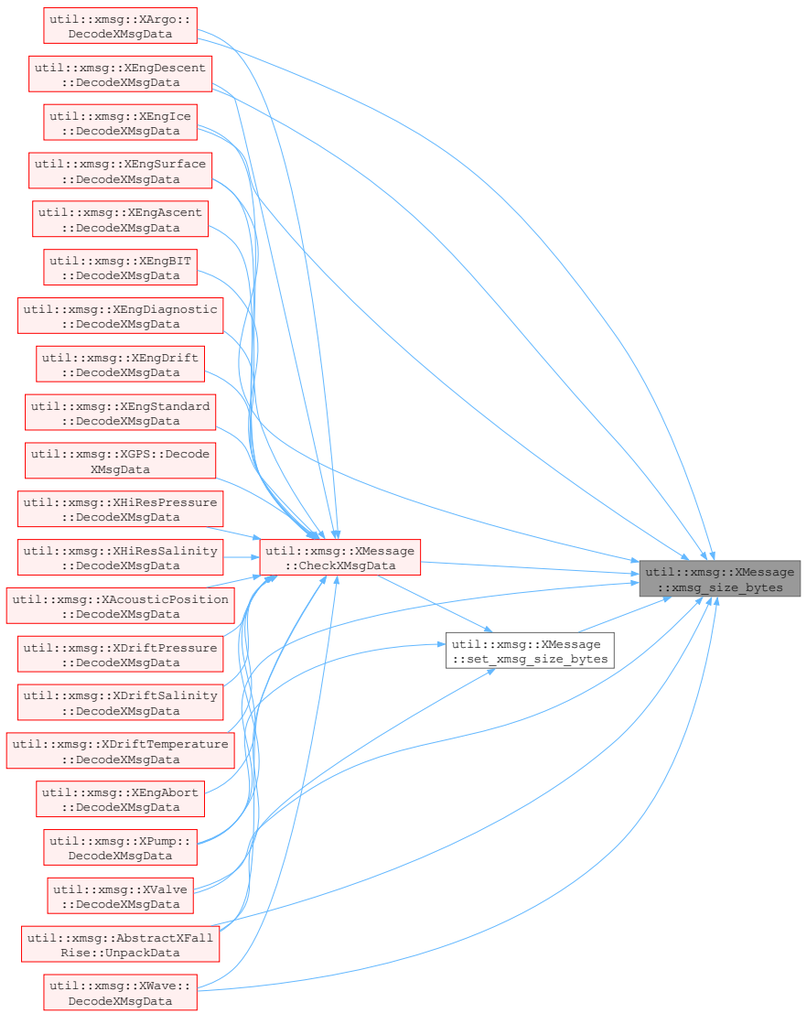
Returns

The size of the X-Message, in bytes.

References util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgI
util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(),
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), util::xmsg::XMessage::set_xmsg_size_bytes(),
and util::xmsg::AbstractXFallRise::UnpackData().

Here is the caller graph for this function:



10.105.4 Member Data Documentation

10.105.4.1 id_dive_

```
int16_t util::xmsg::XMessage::id_dive_  [private], [inherited]
```

The dive or cycle number of this message.

Referenced by util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::set_id_dive().

10.105.4.2 id_instrument_

```
uint16_t util::xmsg::XMessage::id_instrument_  [private], [inherited]
```

The unique WHOI identifier of this instrument.

Referenced by util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::set_id_instrument().

10.105.4.3 id_xmsg_type_

XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type_ [private], [inherited]

The X-message type identifier.

Referenced by util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::set_id_xmsg_type().

10.105.4.4 raw_sensor_data_

vector<int32_t> util::xmsg::AbstractXSensorData::raw_sensor_data_ [protected], [inherited]

The raw integer counts of the measured sensor data.

This vector of integers represents the "raw" sample values of the message. That is, by applying the appropriate gain and offset corrections, the sensor values may be recovered in scientific units. This data may represent pressure, temperature, salinity, or any other data that is recorded during the drift or profile phases of the dive cycle.

Referenced by util::xmsg::AbstractXSensorData::num_samples(), util::xmsg::AbstractXSensorData::raw_sensor_data(), util::xmsg::AbstractXSensorData::Sample(), util::xmsg::AbstractXSensorData::SampleValue(), util::xmsg::AbstractXSensorData::set_raw_ util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

10.105.4.5 sensor_data_format_

XSensorDataFormat_t util::xmsg::AbstractXSensorData::sensor_data_format_ [protected], [inherited]

The enumerated sensor data packing format.

This field indicates the format of encoded sensor data. The manner in which the sensor data is decoded is determined by the format of this field.

Referenced by util::xmsg::AbstractXSensorData::sensor_data_format(), and util::xmsg::AbstractXSensorData::set_sensor_data_format().

10.105.4.6 xmsg_index_

uint8_t util::xmsg::XMessageIndexable::xmsg_index_ [private], [inherited]

The index of this X-Message.

Referenced by util::xmsg::XMessageIndexable::set_xmsg_index(), and util::xmsg::XMessageIndexable::xmsg_index().

10.105.4.7 xmsg_size_bytes_

uint16_t util::xmsg::XMessage::xmsg_size_bytes_ [private], [inherited]

The size of the X message, in bytes.

Referenced by util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes().

The documentation for this class was generated from the following file:

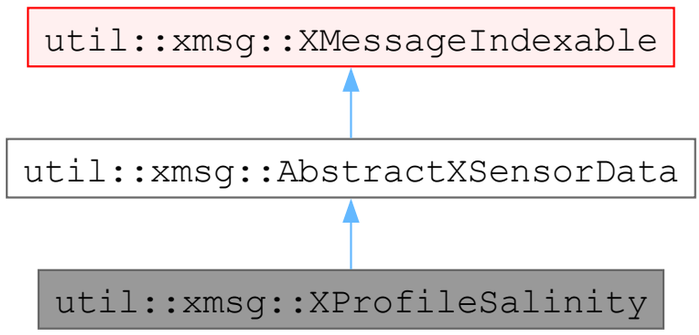
- data/xmsg/XProfilePressure.h

10.106 util::xmsg::XProfileSalinity Class Reference

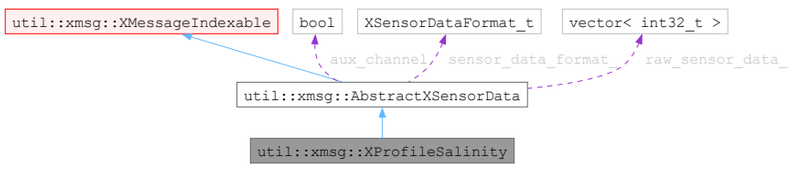
Class representing a profile salinity X-Message.

`#include <XProfileSalinity.h>`

Inheritance diagram for util::xmsg::XProfileSalinity:



Collaboration diagram for util::xmsg::XProfileSalinity:



Public Member Functions

- XProfileSalinity (int16_t dive, uint16_t whoi, const vector< uint8_t > &data)
  The parameterized constructor.
- ~XProfileSalinity ()
  The destructor.
- const uint8_t * CheckXMsgData (const vector< uint8_t > &data, int id, int minBytes, bool parseSize=true)
  Check the X-message for the correct ID, minimum size, and message terminator.
- void DecodeXMsgData (const vector< uint8_t > &data)
  Decode the bin-averaged profile salinity X-Message from the binary message data.
- int16_t id_dive () const
  Get the dive or cycle number of this X-Message.
- uint16_t id_instrument () const
  Get the unique instrument identifier of this X-Message.
- XMsgTypeId_t id_xmsg_type () const
  Get the enumerated X-Message type identifier.
- bool **is_aux_channel** () const
- size_t num_samples () const
  Get the number of samples in this message.
- const vector< int32_t > & raw_sensor_data () const
  Get the raw sensor data vector.
- int32_t Sample (size_t i) const
  Get the raw integer value of the i^{th} sample.
- template<typename T = double>
  T SampleValue (size_t i, int16_t gain, int16_t offs) const
  Get the i^{th} sensor value in scientific units.
- template<typename T = double>
  T SampleValueAsPressure (size_t i, int16_t gain, int16_t offs) const

Get the i^{th} pressure sensor value, in dBar.

- template<typename T = double>
  T SampleValueAsSalinity (size_t i, int16_t gain, int16_t offs) const

  Get the i^{th} salinity sensor value, in PSU.
- template<typename T = double>
  T SampleValueAsTemperature (size_t i, int16_t gain, int16_t offs) const

  Get the i^{th} temperature sensor value, in degC.
- XSensorDataFormat_t sensor_data_format () const

  Get the enumerated sensor data format value.
- uint8_t xmsg_index () const

  Get the index of this X-Message.
- uint16_t xmsg_size_bytes () const

  Get the size of the X-Message, in bytes.

Protected Member Functions

- void set_id_dive (int16_t id_dive)

  Set the dive or cycle number of this message.
- void set_id_instrument (uint16_t id_instrument)

  Set the unique instrument identifier.
- void set_id_xmsg_type (XMsgTypeId_t id_xmsg_type)

  Set the X-Message type identifier.
- void set_raw_sensor_data (const vector< int32_t > &raw_sensor_data)

  Set raw integer counts of the measured sensor data.
- void set_sensor_data_format (XSensorDataFormat_t sensor_data_format)

  Set the sensor data format.
- void set_xmsg_index (uint8_t xmsg_index)

  Set the index of this X-Message.
- void set_xmsg_size_bytes (uint16_t xmsg_size_bytes)

  Set the size of the X-Message, in bytes.
- virtual vector< int32_t > UnpackSensorData (const vector< uint8_t > &data)

  Unpack the sensor data from the binary message.

Protected Attributes

- bool **aux_channel**
- vector< int32_t > raw_sensor_data_

  The raw integer counts of the measured sensor data.
- XSensorDataFormat_t sensor_data_format_

  The enumerated sensor data packing format.

Private Attributes

- int16_t id_dive_

  The dive or cycle number of this message.
- uint16_t id_instrument_

  The unique WHOI identifier of this instrument.
- XMsgTypeId_t id_xmsg_type_

  The X-message type identifier.
- uint8_t xmsg_index_

  The index of this X-Message.
- uint16_t xmsg_size_bytes_

  The size of the X message, in bytes.

10.106.1 Detailed Description

Class representing a profile salinity X-Message.

10.106.2 Constructor & Destructor Documentation

10.106.2.1 XProfileSalinity()

```
util::xmsg::XProfileSalinity::XProfileSalinity (
            int16_t dive,
            uint16_t whoi,
            const vector< uint8_t > & data)  [inline]
```

The parameterized constructor.

Parameters

in	*dive*	The dive or cycle number of this message.
in	*whoi*	The unique WHOI identifier.
in	*data*	The binary profile salinity X-message data to decode.

Exceptions

util::log::MessageException	if an decode error occurs.

References DecodeXMsgData().

Here is the call graph for this function:

10.106.2.2 ~XProfileSalinity()

```
util::xmsg::XProfileSalinity::~XProfileSalinity ()  [inline]
```

The destructor.

10.106.3 Member Function Documentation

10.106.3.1 CheckXMsgData()

```
const uint8_t * util::xmsg::XMessage::CheckXMsgData (
            const vector< uint8_t > & data,
            int id,
            int minBytes,
            bool parseSize = true)  [inline], [inherited]
```

Check the X-message for the correct ID, minimum size, and message terminator.

Parameters

in	*data*	The X-message binary data to check.
in	*id*	The expected ID of the X-message. If this field is less that 0, then the ID check will be skipped.
in	*minBytes*	The minimum number of bytes expected in a valid message.
in	*parseSize*	If TRUE, then the message size field will be parsed from the X-Message. Otherwise, the size field will not be parsed.

Exceptions

util::log::MessageException	if • the message does not contain the expected ID, • the message does not contain the minimum number of expected bytes, • the message does not contain a terminator (';') in the correct location, or, • any other error occurs.

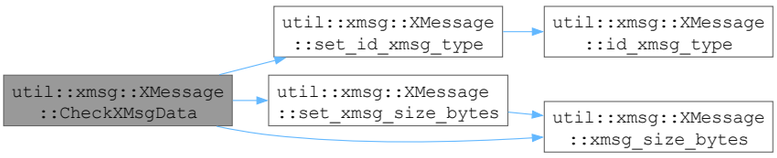
Returns

A pointer to the underlying data buffer of data.

References CATCH_BLOCK, EXC_THROW, lwrn, util::xmsg::XMessage::set_id_xmsg_type(), util::xmsg::XMessage::set_xmsg_size_byte
u16, and util::xmsg::XMessage::xmsg_size_bytes().

Referenced by util::xmsg::XAcousticPosition::DecodeXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XDriftPressure::Dec
util::xmsg::XDriftSalinity::DecodeXMsgData(), util::xmsg::XDriftTemperature::DecodeXMsgData(), util::xmsg::XEngAbort::DecodeXMsgDa
util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngBIT::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(),
util::xmsg::XEngDiagnostic::DecodeXMsgData(), util::xmsg::XEngDrift::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(),
util::xmsg::XEngStandard::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XGPS::DecodeXMsgData(),
util::xmsg::XHiResPressure::DecodeXMsgData(), util::xmsg::XHiResSalinity::DecodeXMsgData(), util::xmsg::XHiResTemperature::Decod
util::xmsg::XMissionCmdEcho::DecodeXMsgData(), util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XProfilePressure:
DecodeXMsgData(), util::xmsg::XProfileTemperature::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(),
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the call graph for this function:



10.106.3.2 DecodeXMsgData()

```
void util::xmsg::XProfileSalinity::DecodeXMsgData (
            const vector< uint8_t > & data)  [inline], [virtual]
```

Decode the bin-averaged profile salinity X-Message from the binary message data.

Parameters

in	*data*	The binary message data to decode.

Exceptions

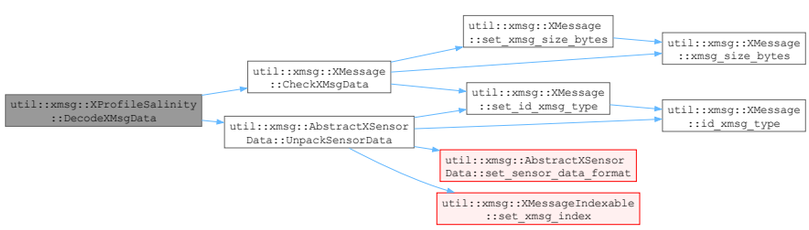
util::log::MessageException	if the message is invalid or an error occurs during decoding.

Implements util::xmsg::AbstractXSensorData.

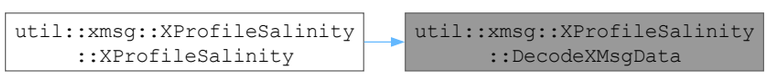
References CATCH_BLOCK, util::xmsg::XMessage::CheckXMsgData(), EXC_THROW, util::xmsg::AbstractXSensorData::UnpackSensorD
and util::xmsg::XMSG_TYPE_ID_SALN0.

Referenced by XProfileSalinity().

Here is the call graph for this function:



Here is the caller graph for this function:



10.106.3.3 id_dive()

```
int16_t util::xmsg::XMessage::id_dive () const  [inline], [inherited]
```

Get the dive or cycle number of this X-Message.

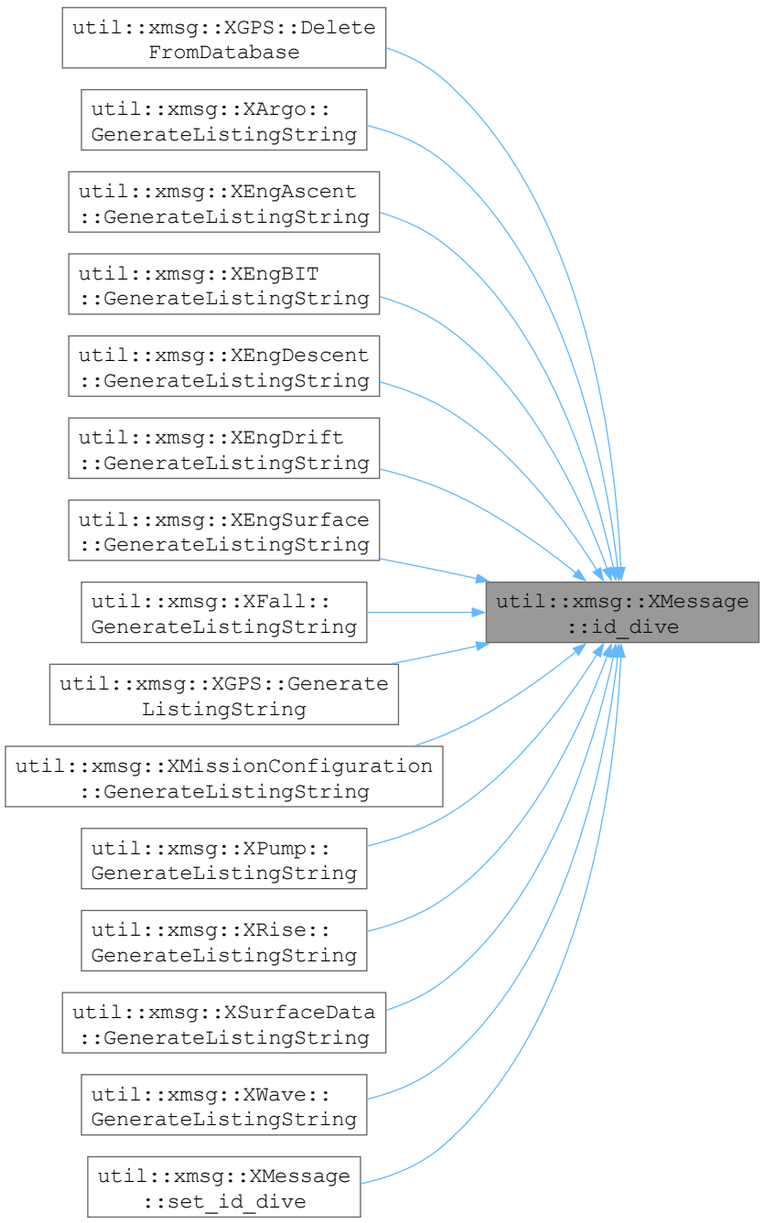
Returns

The dive or cycle number of this X-Message.

References util::xmsg::XMessage::id_dive_.

Referenced by util::xmsg::XGPS::DeleteFromDatabase(), util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateLis
util::xmsg::XEngBIT::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingStr
util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(),
util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing
util::xmsg::XSurfaceData::GenerateListingString(), util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_dive().

Here is the caller graph for this function:



10.106.3.4 id_instrument()

```
uint16_t util::xmsg::XMessage::id_instrument () const  [inline], [inherited]
```

Get the unique instrument identifier of this X-Message.

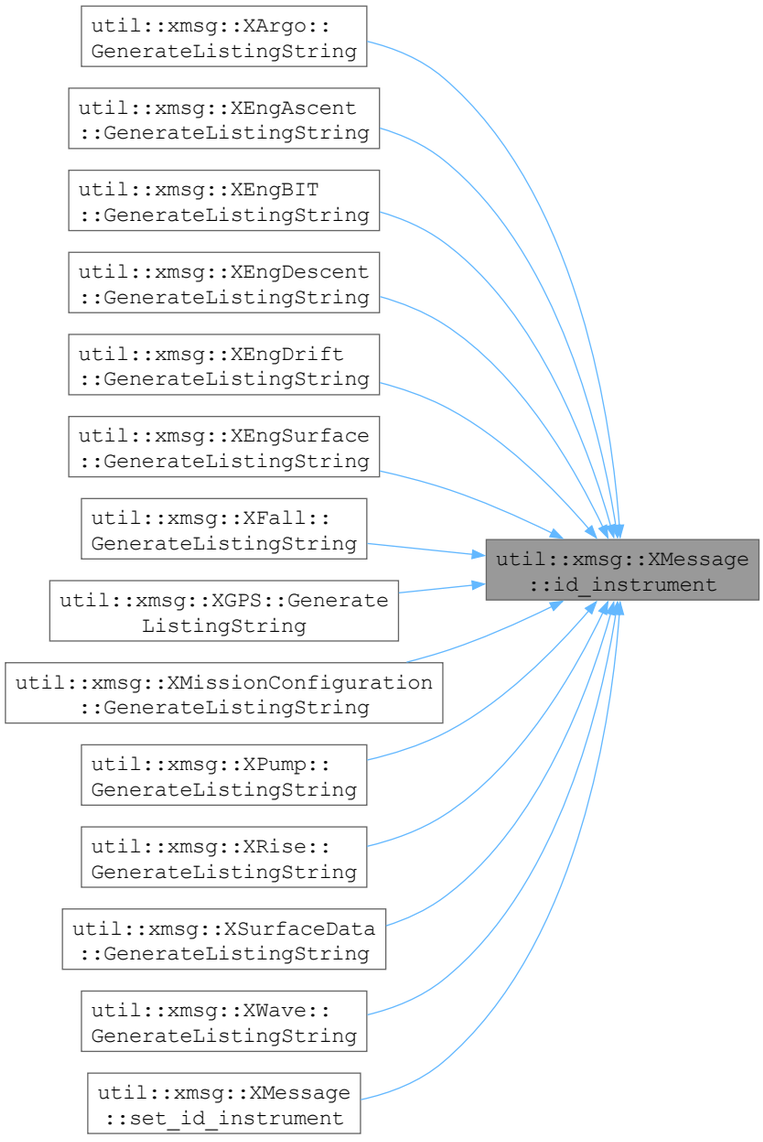
Returns

The unique instrument identifier of this X-Message.

References util::xmsg::XMessage::id_instrument_.

Referenced by util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngBIT::Generate
util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), util::xmsg::XEngSurface::GenerateListin
util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingS
util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(),
util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_instrument().

Here is the caller graph for this function:



10.106.3.5 id_xmsg_type()

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type () const  [inline], [inherited]
```

Get the enumerated X-Message type identifier.

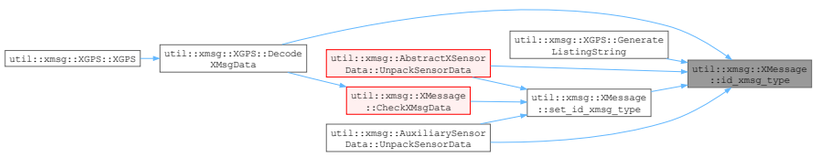
Returns

The enumerated X-Message type identifier.

References util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMessage::set_id_xmsg_typ
util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the caller graph for this function:



10.106.3.6 num_samples()

```
size_t util::xmsg::AbstractXSensorData::num_samples () const  [inline], [inherited]
```

Get the number of samples in this message.

Returns

The number of samples in this message.

References util::xmsg::AbstractXSensorData::raw_sensor_data_.

10.106.3.7 raw_sensor_data()

```
const vector< int32_t > & util::xmsg::AbstractXSensorData::raw_sensor_data () const  [inline],
[inherited]
```

Get the raw sensor data vector.

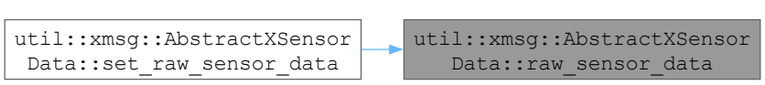
Returns

The raw sensor data vector.

References util::xmsg::AbstractXSensorData::raw_sensor_data_.

Referenced by util::xmsg::AbstractXSensorData::set_raw_sensor_data().

Here is the caller graph for this function:



10.106.3.8 Sample()

```
int32_t util::xmsg::AbstractXSensorData::Sample (
            size_t i) const  [inline], [inherited]
```

Get the raw integer value of the i^{th} sample.

Parameters

in	*i*	The index of the sensor value.

Returns

The raw integer value of the i^{th} ith sample.

References util::xmsg::AbstractXSensorData::raw_sensor_data_.

10.106.3.9 SampleValue()

```
template<typename T = double>
T util::xmsg::AbstractXSensorData::SampleValue (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline], [inherited]
```

Get the i^{th} sensor value in scientific units.

Parameters

in	*i*	The index of the sensor value.
in	*gain*	The sensor sensitivity scale factor.
in	*offs*	The sensor offset.

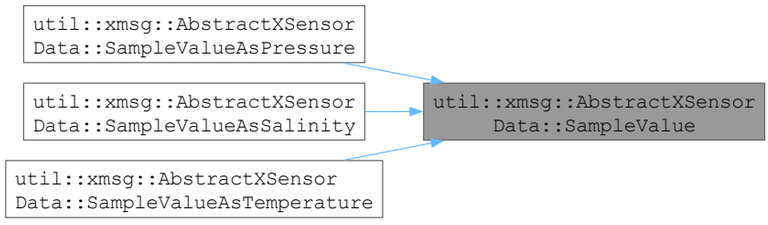
Returns

The i^{th} sensor value in scientific units.

References CATCH_BLOCK, and util::xmsg::AbstractXSensorData::raw_sensor_data_.

Referenced by util::xmsg::AbstractXSensorData::SampleValueAsPressure(), util::xmsg::AbstractXSensorData::SampleValueAsSalinity(), and util::xmsg::AbstractXSensorData::SampleValueAsTemperature().

Here is the caller graph for this function:



10.106.3.10 SampleValueAsPressure()

```
template<typename T = double>
T util::xmsg::AbstractXSensorData::SampleValueAsPressure (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline], [inherited]
```

Get the i^{th} pressure sensor value, in dBar.

Parameters

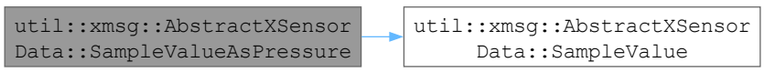
in	*i*	The index of the sensor value.
in	*gain*	The pressure sensor sensitivity scale factor, in LSB/dBar.
in	*offs*	The pressure sensor offset, in dBar.

Returns

The i^{th} pressure sensor value, in dBar.

References util::xmsg::AbstractXSensorData::SampleValue(), XMSG_SENS_BAD_PRES, and XMSG_SENS_MAX_PRES.

Here is the call graph for this function:



10.106.3.11 SampleValueAsSalinity()

```
template<typename T = double>
T util::xmsg::AbstractXSensorData::SampleValueAsSalinity (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline], [inherited]
```

Get the i^{th} salinity sensor value, in PSU.

Parameters

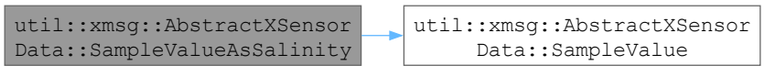
in	*i*	The index of the sensor value.
in	*gain*	The salinity sensor sensitivity scale factor, in LSB/PSU.
in	*offs*	The salinity sensor offset, in PSU.

Returns

The i^{th} salinity sensor value, in PSU.

References util::xmsg::AbstractXSensorData::SampleValue(), XMSG_SENS_BAD_SALN, and XMSG_SENS_MAX_SALN.

Here is the call graph for this function:



10.106.3.12 SampleValueAsTemperature()

```
template<typename T = double>
T util::xmsg::AbstractXSensorData::SampleValueAsTemperature (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline], [inherited]
```

Get the i^{th} temperature sensor value, in degC.

Parameters

in	*i*	The index of the sensor value.
in	*gain*	The temperature sensor sensitivity scale factor, in LSB/degC.
in	*offs*	The temperature sensor offset, in degC.

Returns

The i^{th} temperature sensor value, in degC.

References util::xmsg::AbstractXSensorData::SampleValue(), XMSG_SENS_BAD_TEMP, and XMSG_SENS_MAX_TEMP.

Here is the call graph for this function:

10.106.3.13 sensor_data_format()

```
XSensorDataFormat_t util::xmsg::AbstractXSensorData::sensor_data_format () const  [inline], [inherited]
```

Get the enumerated sensor data format value.

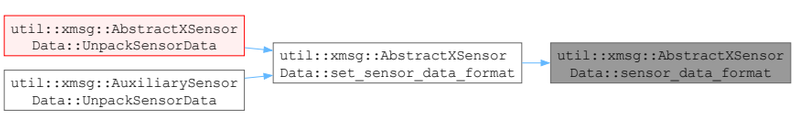
Returns

The enumerated sensor data format value.

References util::xmsg::AbstractXSensorData::sensor_data_format_.

Referenced by util::xmsg::AbstractXSensorData::set_sensor_data_format().

Here is the caller graph for this function:



10.106.3.14 set_id_dive()

```
void util::xmsg::XMessage::set_id_dive (
            int16_t id_dive)  [inline], [protected], [inherited]
```

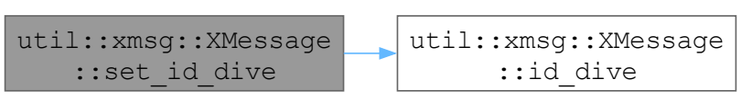
Set the dive or cycle number of this message.

Parameters

in	*id_dive*	The dive or cycle number of this message to set.

References util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::id_dive_.

Here is the call graph for this function:



10.106.3.15 set_id_instrument()

```
void util::xmsg::XMessage::set_id_instrument (
            uint16_t id_instrument)  [inline], [protected], [inherited]
```

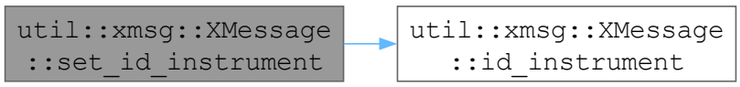
Set the unique instrument identifier.

Parameters

in	*id_instrument*	The unique instrument identifier to set.

References util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::id_instrument_.

Here is the call graph for this function:



10.106.3.16 set_id_xmsg_type()

```
void util::xmsg::XMessage::set_id_xmsg_type (
            XMsgTypeId_t id_xmsg_type)  [inline], [protected], [inherited]
```

Set the X-Message type identifier.

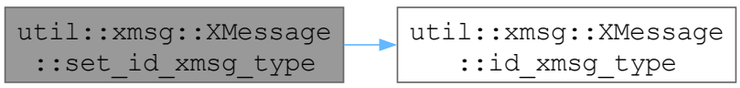
Parameters

in	*id_xmsg_type*	The X-Message type identifier to set.

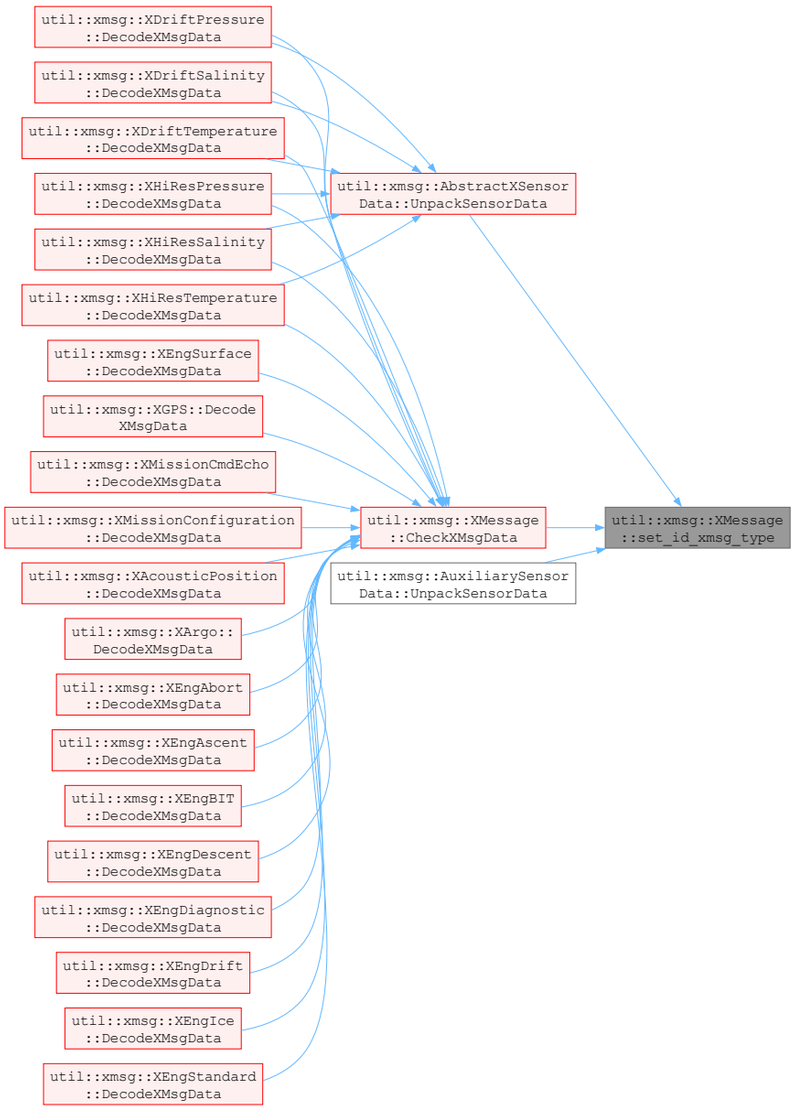
References util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.106.3.17 set_raw_sensor_data()

```
void util::xmsg::AbstractXSensorData::set_raw_sensor_data (
            const vector< int32_t > & raw_sensor_data)  [inline], [protected], [inherited]
```

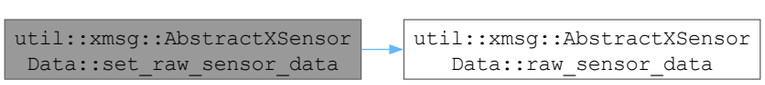
Set raw integer counts of the measured sensor data.

Parameters

in	*raw_sensor_data*	The data to set.

References util::xmsg::AbstractXSensorData::raw_sensor_data(), and util::xmsg::AbstractXSensorData::raw_sensor_data_.

Here is the call graph for this function:



10.106.3.18 set_sensor_data_format()

```
void util::xmsg::AbstractXSensorData::set_sensor_data_format (
            XSensorDataFormat_t sensor_data_format)  [inline], [protected], [inherited]
```

Set the sensor data format.

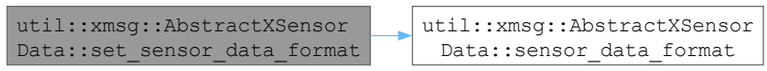
Parameters

in	*sensor_data_format*	The sensor data format to set.

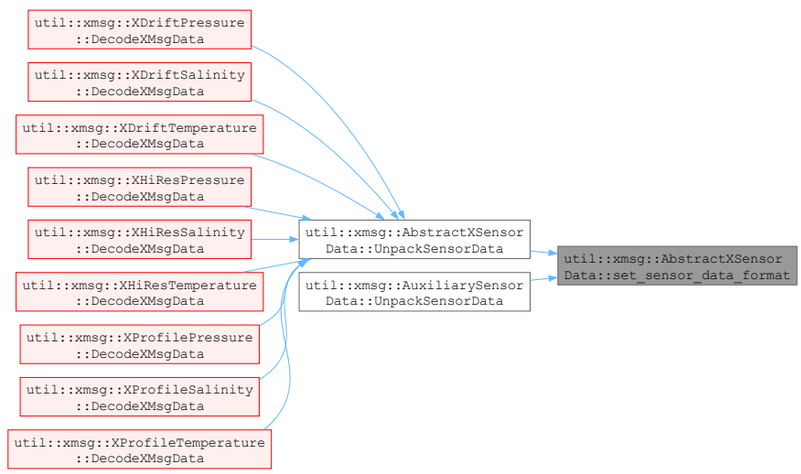
References util::xmsg::AbstractXSensorData::sensor_data_format(), and util::xmsg::AbstractXSensorData::sensor_data_format_.

Referenced by util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.106.3.19 set_xmsg_index()

```
void util::xmsg::XMessageIndexable::set_xmsg_index (
            uint8_t xmsg_index)  [inline], [protected], [inherited]
```

Set the index of this X-Message.

Parameters

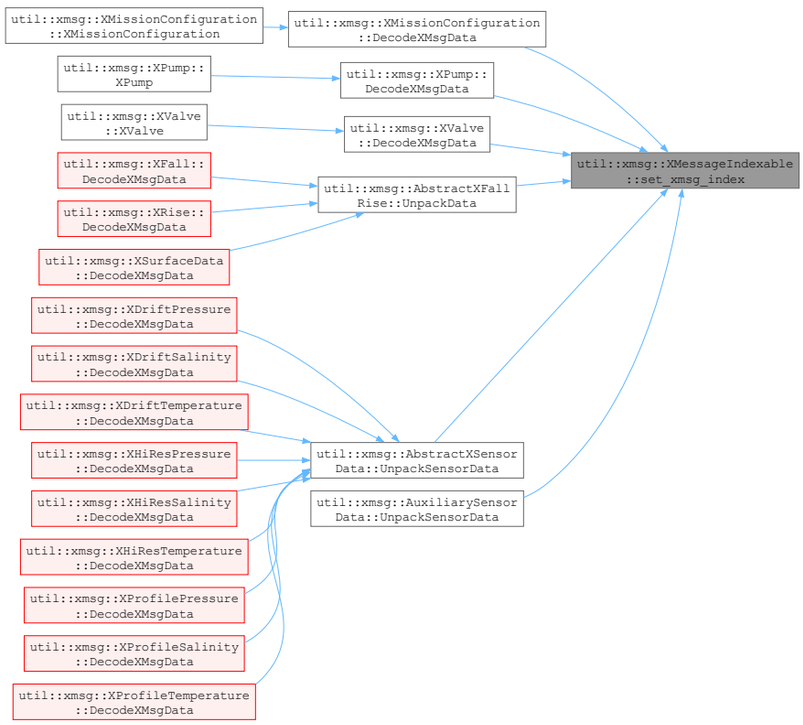
in	*xmsg_index*	The index of this X-Message.

References util::xmsg::XMessageIndexable::xmsg_index(), and util::xmsg::XMessageIndexable::xmsg_index_.

Referenced by util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(), util::xmsg::XValve::DecodeXMsgData(), util::xmsg::AbstractXFallRise::UnpackData(), util::xmsg::AbstractXSensorData::UnpackSensorDa and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:

Here is the caller graph for this function:



10.106.3.20 set_xmsg_size_bytes()

```
void util::xmsg::XMessage::set_xmsg_size_bytes (
            uint16_t xmsg_size_bytes)  [inline], [protected], [inherited]
```

Set the size of the X-Message, in bytes.

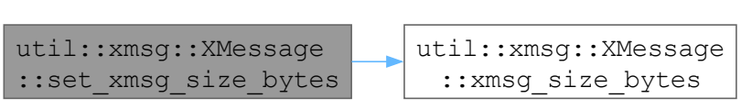
Parameters

in	*xmsg_size_bytes*	The size of the X-Message to set, in bytes.

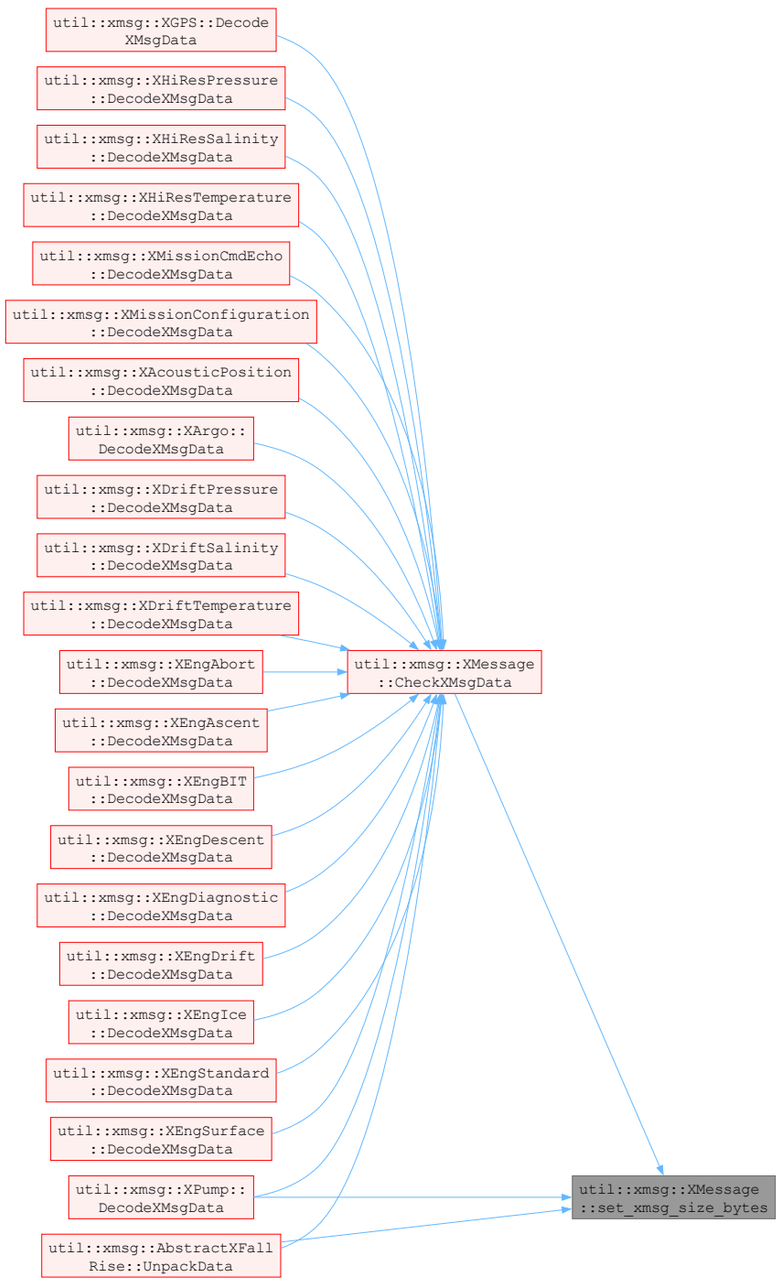
References util::xmsg::XMessage::xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XPump::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::Unpa

Here is the call graph for this function:



Here is the caller graph for this function:



10.106.3.21 UnpackSensorData()

```
virtual vector< int32_t > util::xmsg::AbstractXSensorData::UnpackSensorData (
            const vector< uint8_t > & data)  [inline], [protected], [virtual], [inherited]
```

Unpack the sensor data from the binary message.

This field unpacks the encoded sensor data from the message and stores the decoded sensor values in raw_sensor_data_. First the function checks the size of the message, and the data packing format. The message data is decoded according to the packing format type. Both first-difference and curvature encoding are supported. The encoding method is determined from the value of the high nibble of byte `data`[1].

Once this function completes successfully, then the data may be recovered in scientific units from the raw sensor values by applying the appropriate sensor gain and offset values.

Parameters

in	*data*	The binary message to unpack, in a vector of unsigned bytes.

Warning

The function assumes the message data is valid. Message validity should be ensured before this function is called.

Exceptions

util::log::MessageException	if • an error occurs while decoding the message, • the message is of insufficient length, or • an invalid packing format is encountered.

Returns

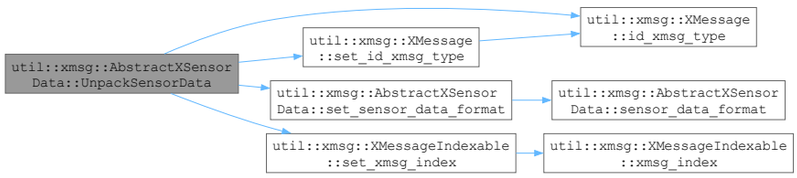
The sensor data raw counts, in a vector of signed integers.

Reimplemented in util::xmsg::AuxiliarySensorData.

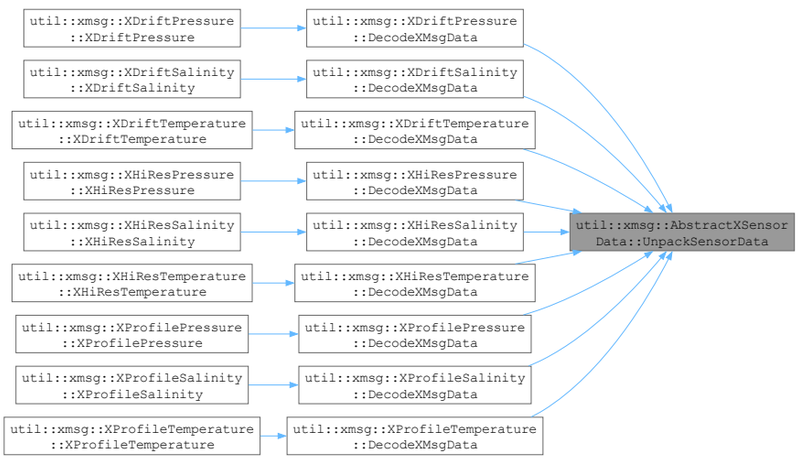
References CATCH_BLOCK, CATCH_NOTHROW, EXC_THROW, i24, util::xmsg::XMessage::id_xmsg_type(), ldbg, lwrn, util::xmsg::AbstractXSensorData::raw_sensor_data_, util::xmsg::XMessage::set_id_xmsg_type(), util::xmsg::AbstractXSensorData::s util::xmsg::XMessageIndexable::set_xmsg_index(), u16, u24, XMSG_SENS_MAX_SUB, util::xmsg::XMSG_TYPE_ID_PRES0, util::xmsg::XMSG_TYPE_ID_PRES_DRIFT8, util::xmsg::XMSG_TYPE_ID_PRES_HI_RES0, util::xmsg::XMSG_TYPE_ID_SALN0, util::xmsg::XMSG_TYPE_ID_SALN_DRIFT8, util::xmsg::XMSG_TYPE_ID_SALN_HI_RES0, util::xmsg::XMSG_TYPE_ID_TEMP0, util::xmsg::XMSG_TYPE_ID_TEMP_DRIFT8, util::xmsg::XMSG_TYPE_ID_TEMP_HI_RES0, util::xmsg::XSENS_FMT_CURVATURE, and util::xmsg::XSENS_FMT_DIFFERENCE.

Referenced by util::xmsg::XDriftPressure::DecodeXMsgData(), util::xmsg::XDriftSalinity::DecodeXMsgData(), util::xmsg::XDriftTemperatur util::xmsg::XHiResPressure::DecodeXMsgData(), util::xmsg::XHiResSalinity::DecodeXMsgData(), util::xmsg::XHiResTemperature::Decod util::xmsg::XProfilePressure::DecodeXMsgData(), DecodeXMsgData(), and util::xmsg::XProfileTemperature::DecodeXMsgData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.106.3.22 xmsg_index()

```
uint8_t util::xmsg::XMessageIndexable::xmsg_index () const  [inline], [inherited]
```

Get the index of this X-Message.

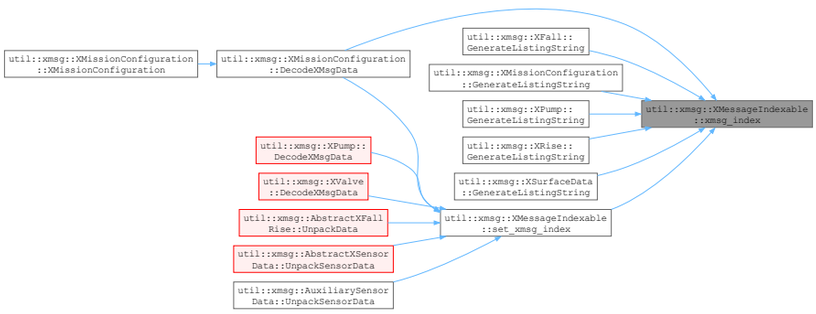
Returns

The index of this X-Message.

References util::xmsg::XMessageIndexable::xmsg_index_.

Referenced by util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing util::xmsg::XSurfaceData::GenerateListingString(), and util::xmsg::XMessageIndexable::set_xmsg_index().

Here is the caller graph for this function:



10.106.3.23 xmsg_size_bytes()

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes () const  [inline], [inherited]
```

Get the size of the X-Message, in bytes.

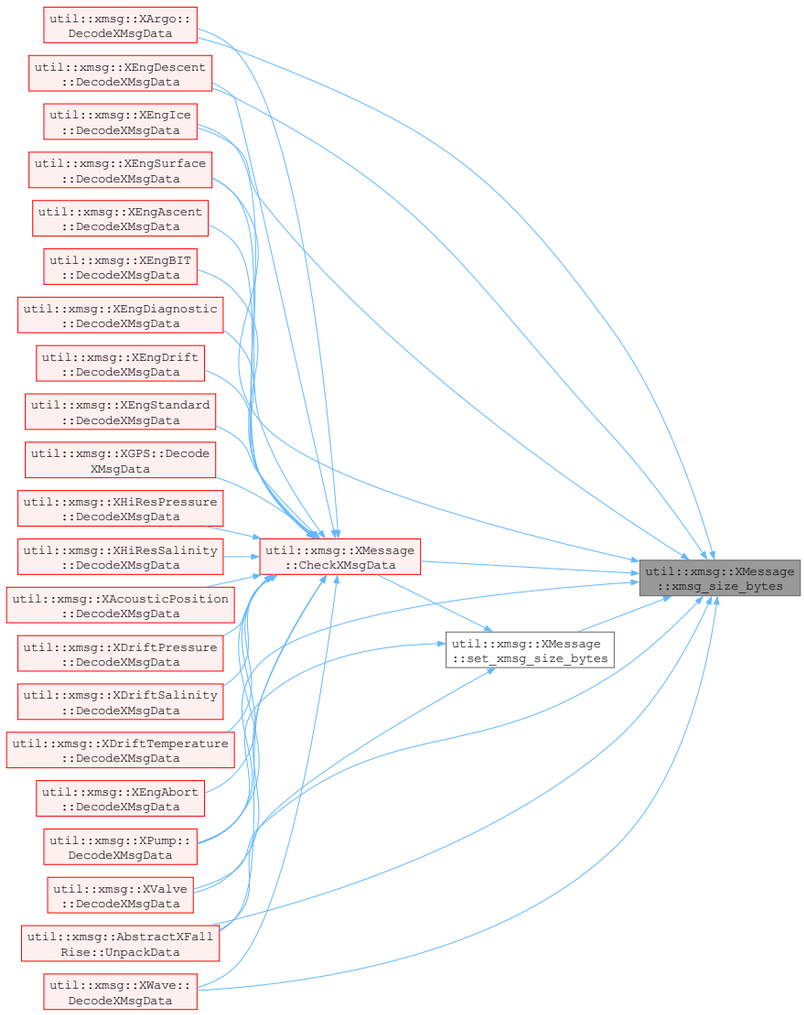
Returns

The size of the X-Message, in bytes.

References util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgI util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(), util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the caller graph for this function:



10.106.4 Member Data Documentation

10.106.4.1 id_dive_

```
int16_t util::xmsg::XMessage::id_dive_  [private], [inherited]
```

The dive or cycle number of this message.

Referenced by util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::set_id_dive().

10.106.4.2 id_instrument_

```
uint16_t util::xmsg::XMessage::id_instrument_  [private], [inherited]
```

The unique WHOI identifier of this instrument.

Referenced by util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::set_id_instrument().

10.106.4.3 id_xmsg_type_

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type_  [private], [inherited]
```

The X-message type identifier.

Referenced by util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::set_id_xmsg_type().

10.106.4.4 raw_sensor_data_

```
vector<int32_t> util::xmsg::AbstractXSensorData::raw_sensor_data_  [protected], [inherited]
```

The raw integer counts of the measured sensor data.

This vector of integers represents the "raw" sample values of the message. That is, by applying the appropriate gain and offset corrections, the sensor values may be recovered in scientific units. This data may represent pressure, temperature, salinity, or any other data that is recorded during the drift or profile phases of the dive cycle.

Referenced by util::xmsg::AbstractXSensorData::num_samples(), util::xmsg::AbstractXSensorData::raw_sensor_data(), util::xmsg::AbstractXSensorData::Sample(), util::xmsg::AbstractXSensorData::SampleValue(), util::xmsg::AbstractXSensorData::set_raw_ util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

10.106.4.5 sensor_data_format_

```
XSensorDataFormat_t util::xmsg::AbstractXSensorData::sensor_data_format_  [protected], [inherited]
```

The enumerated sensor data packing format.

This field indicates the format of encoded sensor data. The manner in which the sensor data is decoded is determined by the format of this field.

Referenced by util::xmsg::AbstractXSensorData::sensor_data_format(), and util::xmsg::AbstractXSensorData::set_sensor_data_format().

10.106.4.6 xmsg_index_

```
uint8_t util::xmsg::XMessageIndexable::xmsg_index_  [private], [inherited]
```

The index of this X-Message.

Referenced by util::xmsg::XMessageIndexable::set_xmsg_index(), and util::xmsg::XMessageIndexable::xmsg_index().

10.106.4.7 xmsg_size_bytes_

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes_  [private], [inherited]
```

The size of the X message, in bytes.

Referenced by util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes().

The documentation for this class was generated from the following file:

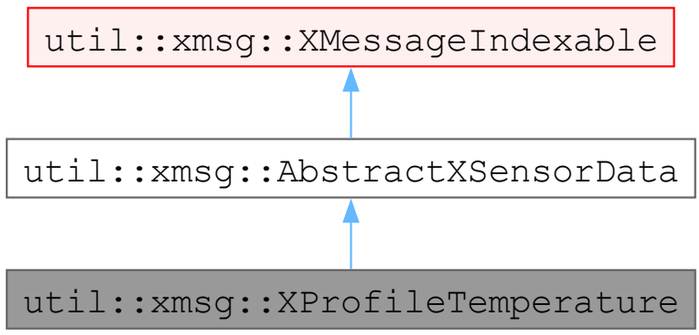
- data/xmsg/XProfileSalinity.h

10.107 util::xmsg::XProfileTemperature Class Reference

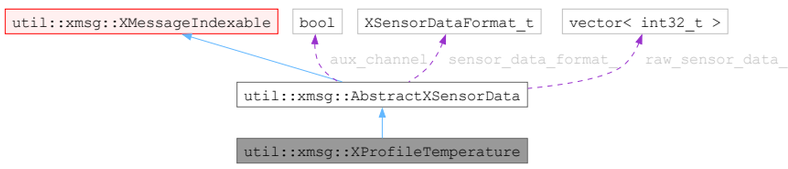
Class representing a profile temperature X-Message.

`#include <XProfileTemperature.h>`

Inheritance diagram for util::xmsg::XProfileTemperature:



Collaboration diagram for util::xmsg::XProfileTemperature:



Public Member Functions

- XProfileTemperature (int16_t dive, uint16_t whoi, const vector< uint8_t > &data)
  Parameterized constructor.
- ~XProfileTemperature ()
  Destructor.
- const uint8_t * CheckXMsgData (const vector< uint8_t > &data, int id, int minBytes, bool parseSize=true)
  Check the X-message for the correct ID, minimum size, and message terminator.
- void DecodeXMsgData (const vector< uint8_t > &data)
  Decode the bin-averaged profile temperature X-Message from the binary message data.
- int16_t id_dive () const
  Get the dive or cycle number of this X-Message.
- uint16_t id_instrument () const
  Get the unique instrument identifier of this X-Message.
- XMsgTypeId_t id_xmsg_type () const
  Get the enumerated X-Message type identifier.
- bool **is_aux_channel** () const
- size_t num_samples () const
  Get the number of samples in this message.
- const vector< int32_t > & raw_sensor_data () const
  Get the raw sensor data vector.
- int32_t Sample (size_t i) const
  Get the raw integer value of the i^{th} sample.
- template<typename T = double>
  T SampleValue (size_t i, int16_t gain, int16_t offs) const
  Get the i^{th} sensor value in scientific units.
- template<typename T = double>
  T SampleValueAsPressure (size_t i, int16_t gain, int16_t offs) const

Get the i^{th} pressure sensor value, in dBar.

- template<typename T = double>
  T SampleValueAsSalinity (size_t i, int16_t gain, int16_t offs) const

  Get the i^{th} salinity sensor value, in PSU.
- template<typename T = double>
  T SampleValueAsTemperature (size_t i, int16_t gain, int16_t offs) const

  Get the i^{th} temperature sensor value, in degC.
- XSensorDataFormat_t sensor_data_format () const

  Get the enumerated sensor data format value.
- uint8_t xmsg_index () const

  Get the index of this X-Message.
- uint16_t xmsg_size_bytes () const

  Get the size of the X-Message, in bytes.

Protected Member Functions

- void set_id_dive (int16_t id_dive)

  Set the dive or cycle number of this message.
- void set_id_instrument (uint16_t id_instrument)

  Set the unique instrument identifier.
- void set_id_xmsg_type (XMsgTypeId_t id_xmsg_type)

  Set the X-Message type identifier.
- void set_raw_sensor_data (const vector< int32_t > &raw_sensor_data)

  Set raw integer counts of the measured sensor data.
- void set_sensor_data_format (XSensorDataFormat_t sensor_data_format)

  Set the sensor data format.
- void set_xmsg_index (uint8_t xmsg_index)

  Set the index of this X-Message.
- void set_xmsg_size_bytes (uint16_t xmsg_size_bytes)

  Set the size of the X-Message, in bytes.
- virtual vector< int32_t > UnpackSensorData (const vector< uint8_t > &data)

  Unpack the sensor data from the binary message.

Protected Attributes

- bool **aux_channel**
- vector< int32_t > raw_sensor_data_

  The raw integer counts of the measured sensor data.
- XSensorDataFormat_t sensor_data_format_

  The enumerated sensor data packing format.

Private Attributes

- int16_t id_dive_

  The dive or cycle number of this message.
- uint16_t id_instrument_

  The unique WHOI identifier of this instrument.
- XMsgTypeId_t id_xmsg_type_

  The X-message type identifier.
- uint8_t xmsg_index_

  The index of this X-Message.
- uint16_t xmsg_size_bytes_

  The size of the X message, in bytes.

10.107.1 Detailed Description

Class representing a profile temperature X-Message.

10.107.2 Constructor & Destructor Documentation

10.107.2.1 XProfileTemperature()

```
util::xmsg::XProfileTemperature::XProfileTemperature (
            int16_t dive,
            uint16_t whoi,
            const vector< uint8_t > & data)  [inline]
```

Parameterized constructor.

Parameters

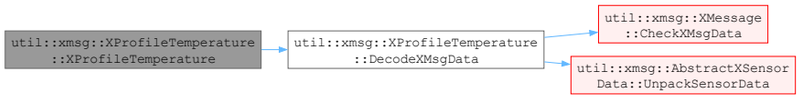
in	*dive*	The dive or cycle number of this message.
in	*whoi*	The unique WHOI identifier.
in	*data*	The binary profile temperature X-message data to decode.

Exceptions

util::log::MessageException	if an decode error occurs.

References DecodeXMsgData().

Here is the call graph for this function:



10.107.2.2 ~XProfileTemperature()

```
util::xmsg::XProfileTemperature::~XProfileTemperature ()  [inline]
```

Destructor.

10.107.3 Member Function Documentation

10.107.3.1 CheckXMsgData()

```
const uint8_t * util::xmsg::XMessage::CheckXMsgData (
            const vector< uint8_t > & data,
            int id,
            int minBytes,
            bool parseSize = true)  [inline], [inherited]
```

Check the X-message for the correct ID, minimum size, and message terminator.

Parameters

in	*data*	The X-message binary data to check.
in	*id*	The expected ID of the X-message. If this field is less that 0, then the ID check will be skipped.
in	*minBytes*	The minimum number of bytes expected in a valid message.
in	*parseSize*	If TRUE, then the message size field will be parsed from the X-Message. Otherwise, the size field will not be parsed.

Exceptions

util::log::MessageException	if • the message does not contain the expected ID, • the message does not contain the minimum number of expected bytes, • the message does not contain a terminator (';') in the correct location, or, • any other error occurs.

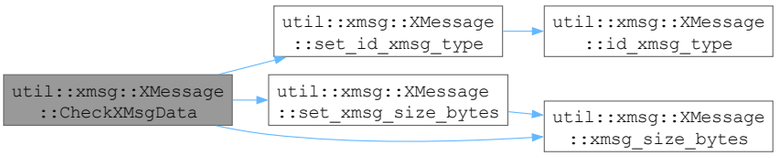
Returns

A pointer to the underlying data buffer of data.

References CATCH_BLOCK, EXC_THROW, lwrn, util::xmsg::XMessage::set_id_xmsg_type(), util::xmsg::XMessage::set_xmsg_size_byte
u16, and util::xmsg::XMessage::xmsg_size_bytes().

Referenced by util::xmsg::XAcousticPosition::DecodeXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XDriftPressure::Dec
util::xmsg::XDriftSalinity::DecodeXMsgData(), util::xmsg::XDriftTemperature::DecodeXMsgData(), util::xmsg::XEngAbort::DecodeXMsgDa
util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngBIT::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(),
util::xmsg::XEngDiagnostic::DecodeXMsgData(), util::xmsg::XEngDrift::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(),
util::xmsg::XEngStandard::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XGPS::DecodeXMsgData(),
util::xmsg::XHiResPressure::DecodeXMsgData(), util::xmsg::XHiResSalinity::DecodeXMsgData(), util::xmsg::XHiResTemperature::Decod
util::xmsg::XMissionCmdEcho::DecodeXMsgData(), util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XProfilePressure:
util::xmsg::XProfileSalinity::DecodeXMsgData(), DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(),
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the call graph for this function:



10.107.3.2 DecodeXMsgData()

```
void util::xmsg::XProfileTemperature::DecodeXMsgData (
            const vector< uint8_t > & data)  [inline], [virtual]
```

Decode the bin-averaged profile temperature X-Message from the binary message data.

Parameters

in	*data*	The binary message data to decode.

Exceptions

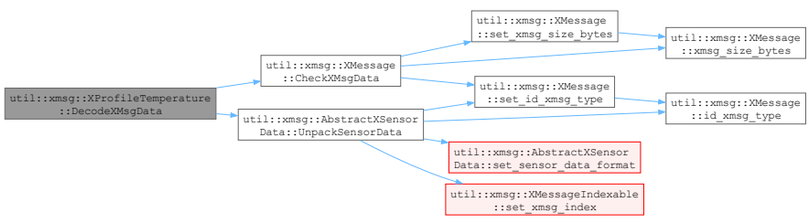
util::log::MessageException	if the message is invalid or an error occurs during decoding.

Implements util::xmsg::AbstractXSensorData.

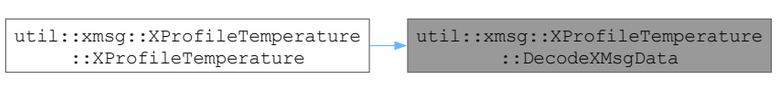
References CATCH_BLOCK, util::xmsg::XMessage::CheckXMsgData(), EXC_THROW, util::xmsg::AbstractXSensorData::UnpackSensorD
and util::xmsg::XMSG_TYPE_ID_TEMP0.

Referenced by XProfileTemperature().

Here is the call graph for this function:



Here is the caller graph for this function:



10.107.3.3 id_dive()

```
int16_t util::xmsg::XMessage::id_dive () const  [inline], [inherited]
```

Get the dive or cycle number of this X-Message.

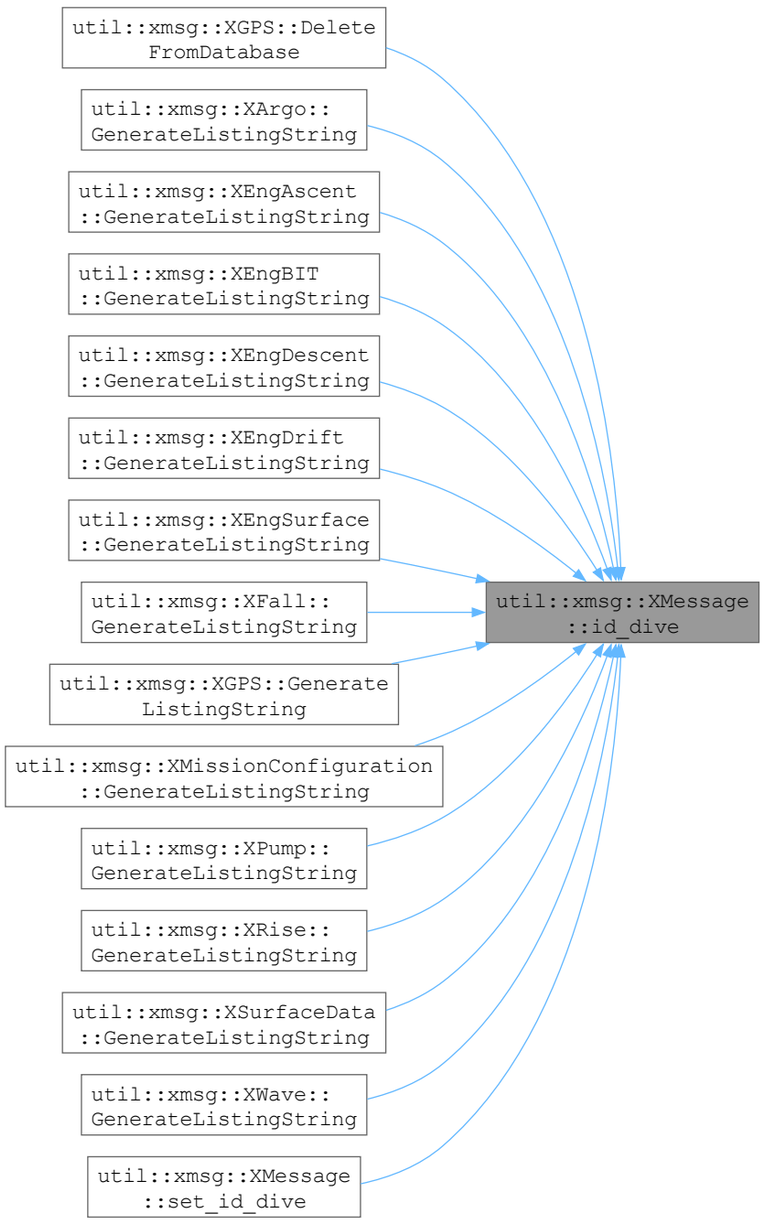
Returns

The dive or cycle number of this X-Message.

References util::xmsg::XMessage::id_dive_.

Referenced by util::xmsg::XGPS::DeleteFromDatabase(), util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateLis
util::xmsg::XEngBIT::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingStr
util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(),
util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing
util::xmsg::XSurfaceData::GenerateListingString(), util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_dive().

Here is the caller graph for this function:



10.107.3.4 id_instrument()

```
uint16_t util::xmsg::XMessage::id_instrument () const  [inline], [inherited]
```

Get the unique instrument identifier of this X-Message.

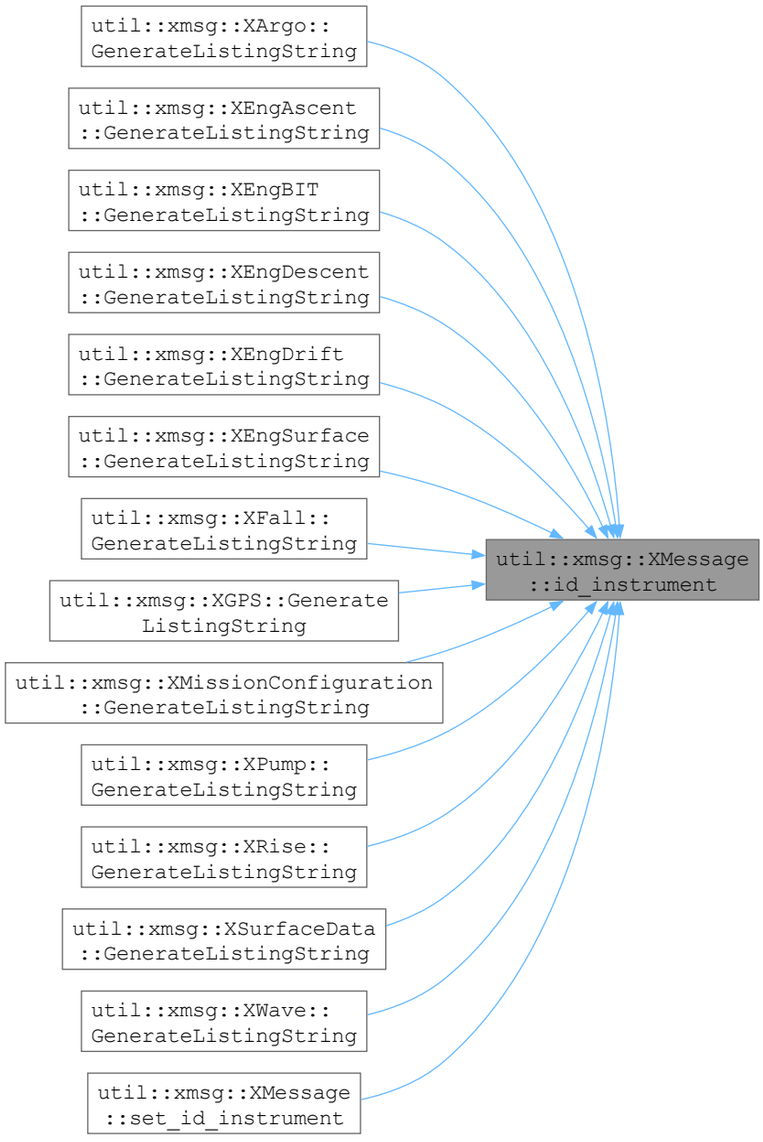
Returns

The unique instrument identifier of this X-Message.

References util::xmsg::XMessage::id_instrument_.

Referenced by util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngBIT::Generate
util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), util::xmsg::XEngSurface::GenerateListin
util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingS
util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(),
util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_instrument().

Here is the caller graph for this function:



10.107.3.5 id_xmsg_type()

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type () const  [inline], [inherited]
```

Get the enumerated X-Message type identifier.

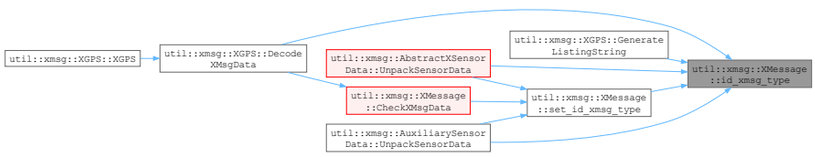
Returns

The enumerated X-Message type identifier.

References util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMessage::set_id_xmsg_typ
util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the caller graph for this function:



10.107.3.6 num_samples()

```
size_t util::xmsg::AbstractXSensorData::num_samples () const  [inline], [inherited]
```

Get the number of samples in this message.

Returns

The number of samples in this message.

References util::xmsg::AbstractXSensorData::raw_sensor_data_.

10.107.3.7 raw_sensor_data()

```
const vector< int32_t > & util::xmsg::AbstractXSensorData::raw_sensor_data () const  [inline],
[inherited]
```

Get the raw sensor data vector.

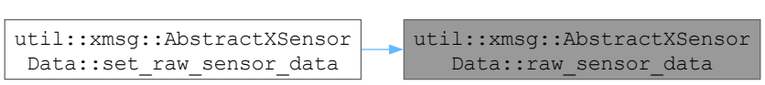
Returns

The raw sensor data vector.

References util::xmsg::AbstractXSensorData::raw_sensor_data_.

Referenced by util::xmsg::AbstractXSensorData::set_raw_sensor_data().

Here is the caller graph for this function:



10.107.3.8 Sample()

```
int32_t util::xmsg::AbstractXSensorData::Sample (
            size_t i) const  [inline], [inherited]
```

Get the raw integer value of the i^{th} sample.

Parameters

in	*i*	The index of the sensor value.

Returns

The raw integer value of the i^{th} ith sample.

References util::xmsg::AbstractXSensorData::raw_sensor_data_.

10.107.3.9 SampleValue()

```
template<typename T = double>
T util::xmsg::AbstractXSensorData::SampleValue (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline], [inherited]
```

Get the i^{th} sensor value in scientific units.

Parameters

in	*i*	The index of the sensor value.
in	*gain*	The sensor sensitivity scale factor.
in	*offs*	The sensor offset.

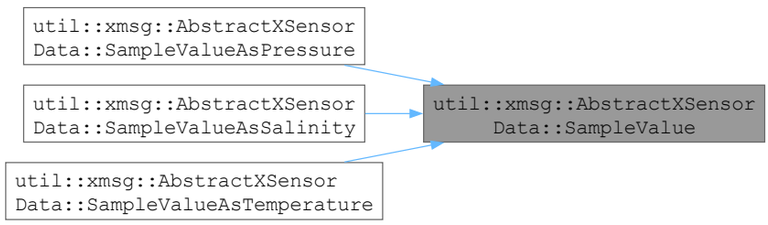
Returns

The i^{th} sensor value in scientific units.

References CATCH_BLOCK, and util::xmsg::AbstractXSensorData::raw_sensor_data_.

Referenced by util::xmsg::AbstractXSensorData::SampleValueAsPressure(), util::xmsg::AbstractXSensorData::SampleValueAsSalinity(), and util::xmsg::AbstractXSensorData::SampleValueAsTemperature().

Here is the caller graph for this function:



10.107.3.10 SampleValueAsPressure()

```
template<typename T = double>
T util::xmsg::AbstractXSensorData::SampleValueAsPressure (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline], [inherited]
```

Get the i^{th} pressure sensor value, in dBar.

Parameters

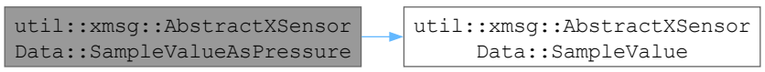
in	*i*	The index of the sensor value.
in	*gain*	The pressure sensor sensitivity scale factor, in LSB/dBar.
in	*offs*	The pressure sensor offset, in dBar.

Returns

The i^{th} pressure sensor value, in dBar.

References util::xmsg::AbstractXSensorData::SampleValue(), XMSG_SENS_BAD_PRES, and XMSG_SENS_MAX_PRES.

Here is the call graph for this function:



10.107.3.11 SampleValueAsSalinity()

```
template<typename T = double>
T util::xmsg::AbstractXSensorData::SampleValueAsSalinity (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline], [inherited]
```

Get the i^{th} salinity sensor value, in PSU.

Parameters

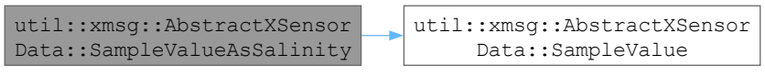
in	*i*	The index of the sensor value.
in	*gain*	The salinity sensor sensitivity scale factor, in LSB/PSU.
in	*offs*	The salinity sensor offset, in PSU.

Returns

The i^{th} salinity sensor value, in PSU.

References util::xmsg::AbstractXSensorData::SampleValue(), XMSG_SENS_BAD_SALN, and XMSG_SENS_MAX_SALN.

Here is the call graph for this function:



10.107.3.12 SampleValueAsTemperature()

```
template<typename T = double>
T util::xmsg::AbstractXSensorData::SampleValueAsTemperature (
            size_t i,
            int16_t gain,
            int16_t offs) const  [inline], [inherited]
```

Get the i^{th} temperature sensor value, in degC.

Parameters

in	*i*	The index of the sensor value.
in	*gain*	The temperature sensor sensitivity scale factor, in LSB/degC.
in	*offs*	The temperature sensor offset, in degC.

Returns

The i^{th} temperature sensor value, in degC.

References util::xmsg::AbstractXSensorData::SampleValue(), XMSG_SENS_BAD_TEMP, and XMSG_SENS_MAX_TEMP.

Here is the call graph for this function:

10.107.3.13 sensor_data_format()

```
XSensorDataFormat_t util::xmsg::AbstractXSensorData::sensor_data_format () const  [inline], [inherited]
```

Get the enumerated sensor data format value.

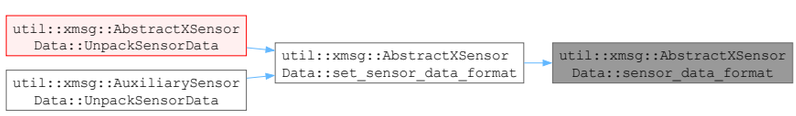
Returns

The enumerated sensor data format value.

References util::xmsg::AbstractXSensorData::sensor_data_format_.

Referenced by util::xmsg::AbstractXSensorData::set_sensor_data_format().

Here is the caller graph for this function:



10.107.3.14 set_id_dive()

```
void util::xmsg::XMessage::set_id_dive (
            int16_t id_dive)  [inline], [protected], [inherited]
```

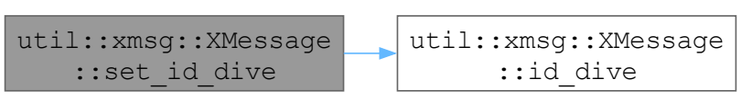
Set the dive or cycle number of this message.

Parameters

in	*id_dive*	The dive or cycle number of this message to set.

References util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::id_dive_.

Here is the call graph for this function:



10.107.3.15 set_id_instrument()

```
void util::xmsg::XMessage::set_id_instrument (
            uint16_t id_instrument)  [inline], [protected], [inherited]
```

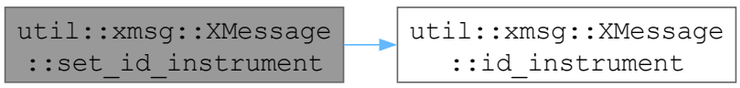
Set the unique instrument identifier.

Parameters

in	*id_instrument*	The unique instrument identifier to set.

References util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::id_instrument_.

Here is the call graph for this function:



10.107.3.16 set_id_xmsg_type()

```
void util::xmsg::XMessage::set_id_xmsg_type (
            XMsgTypeId_t id_xmsg_type)  [inline], [protected], [inherited]
```

Set the X-Message type identifier.

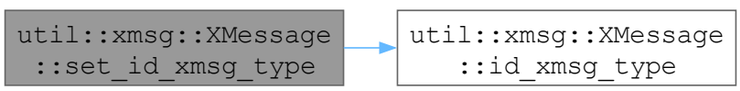
Parameters

in	*id_xmsg_type*	The X-Message type identifier to set.

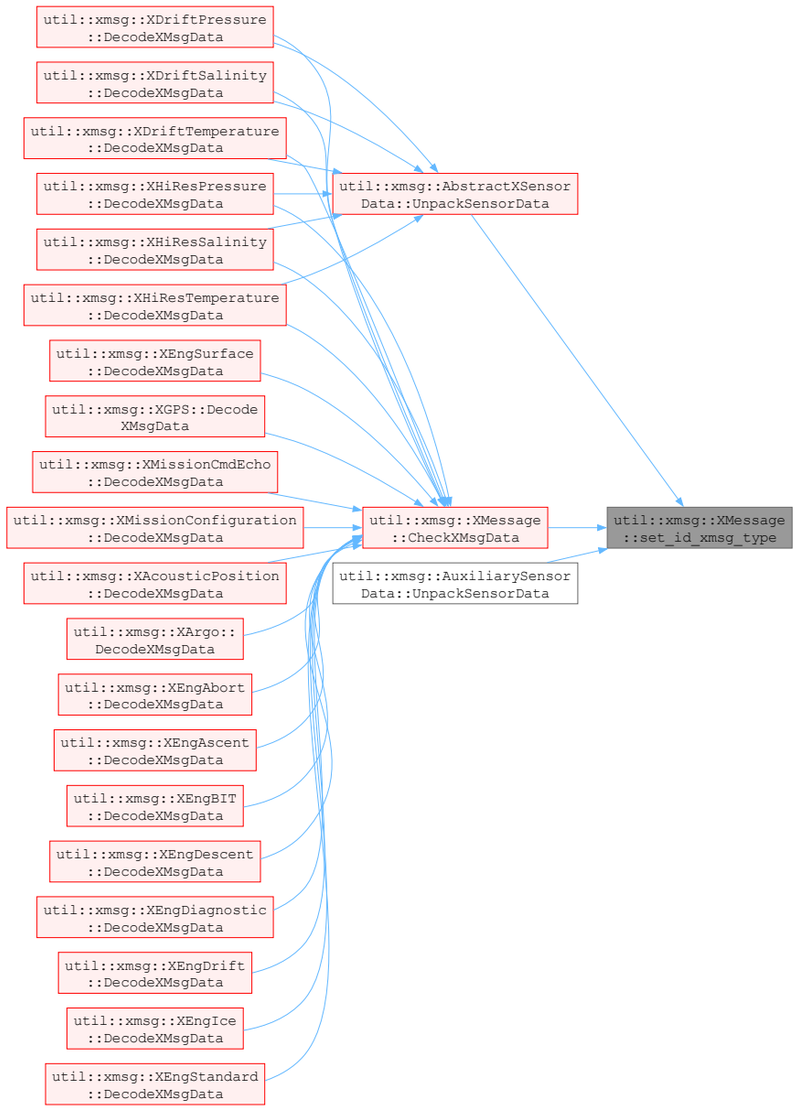
References util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.107.3.17 set_raw_sensor_data()

```
void util::xmsg::AbstractXSensorData::set_raw_sensor_data (
            const vector< int32_t > & raw_sensor_data)  [inline], [protected], [inherited]
```

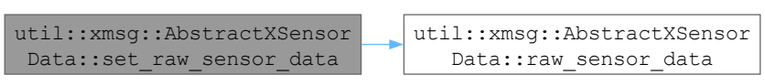
Set raw integer counts of the measured sensor data.

Parameters

in	*raw_sensor_data*	The data to set.

References util::xmsg::AbstractXSensorData::raw_sensor_data(), and util::xmsg::AbstractXSensorData::raw_sensor_data_.

Here is the call graph for this function:



10.107.3.18 set_sensor_data_format()

```
void util::xmsg::AbstractXSensorData::set_sensor_data_format (
            XSensorDataFormat_t sensor_data_format)  [inline], [protected], [inherited]
```

Set the sensor data format.

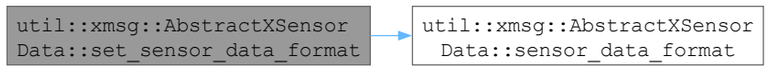
Parameters

in	*sensor_data_format*	The sensor data format to set.

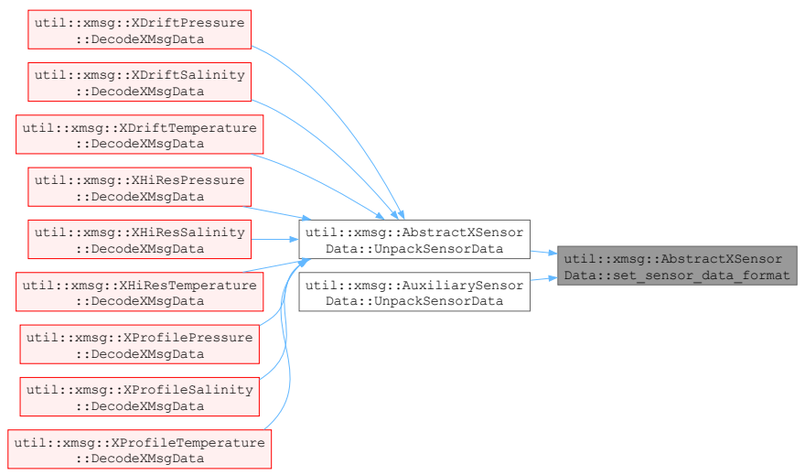
References util::xmsg::AbstractXSensorData::sensor_data_format(), and util::xmsg::AbstractXSensorData::sensor_data_format_.

Referenced by util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.107.3.19 set_xmsg_index()

```
void util::xmsg::XMessageIndexable::set_xmsg_index (
            uint8_t xmsg_index)  [inline], [protected], [inherited]
```

Set the index of this X-Message.

Parameters

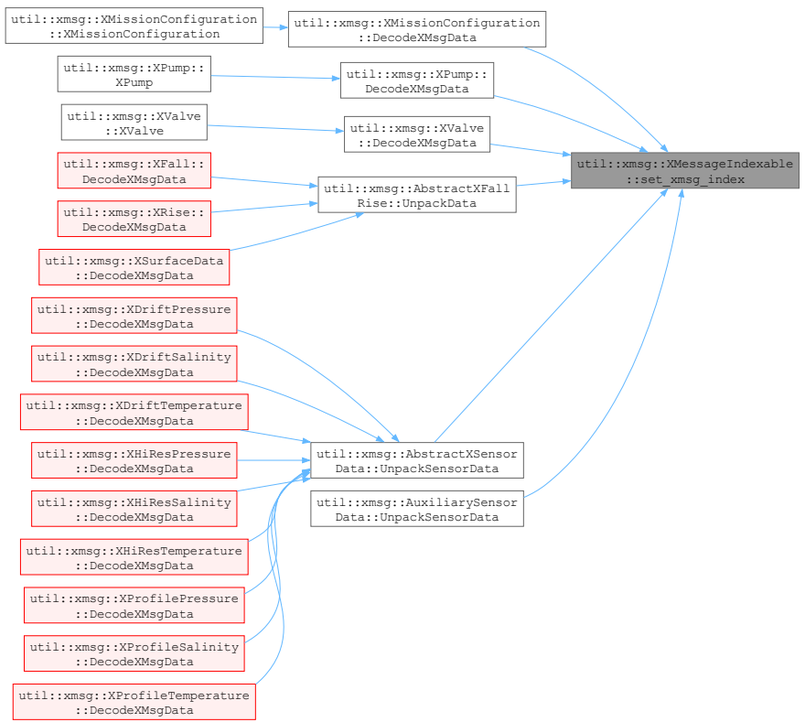
in	*xmsg_index*	The index of this X-Message.

References util::xmsg::XMessageIndexable::xmsg_index(), and util::xmsg::XMessageIndexable::xmsg_index_.

Referenced by util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(), util::xmsg::XValve::DecodeXMsgData(), util::xmsg::AbstractXFallRise::UnpackData(), util::xmsg::AbstractXSensorData::UnpackSensorDa and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:

Here is the caller graph for this function:



10.107.3.20 set_xmsg_size_bytes()

```
void util::xmsg::XMessage::set_xmsg_size_bytes (
            uint16_t xmsg_size_bytes)  [inline], [protected], [inherited]
```

Set the size of the X-Message, in bytes.

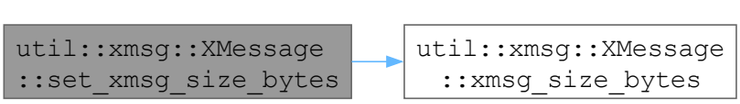
Parameters

in	*xmsg_size_bytes*	The size of the X-Message to set, in bytes.

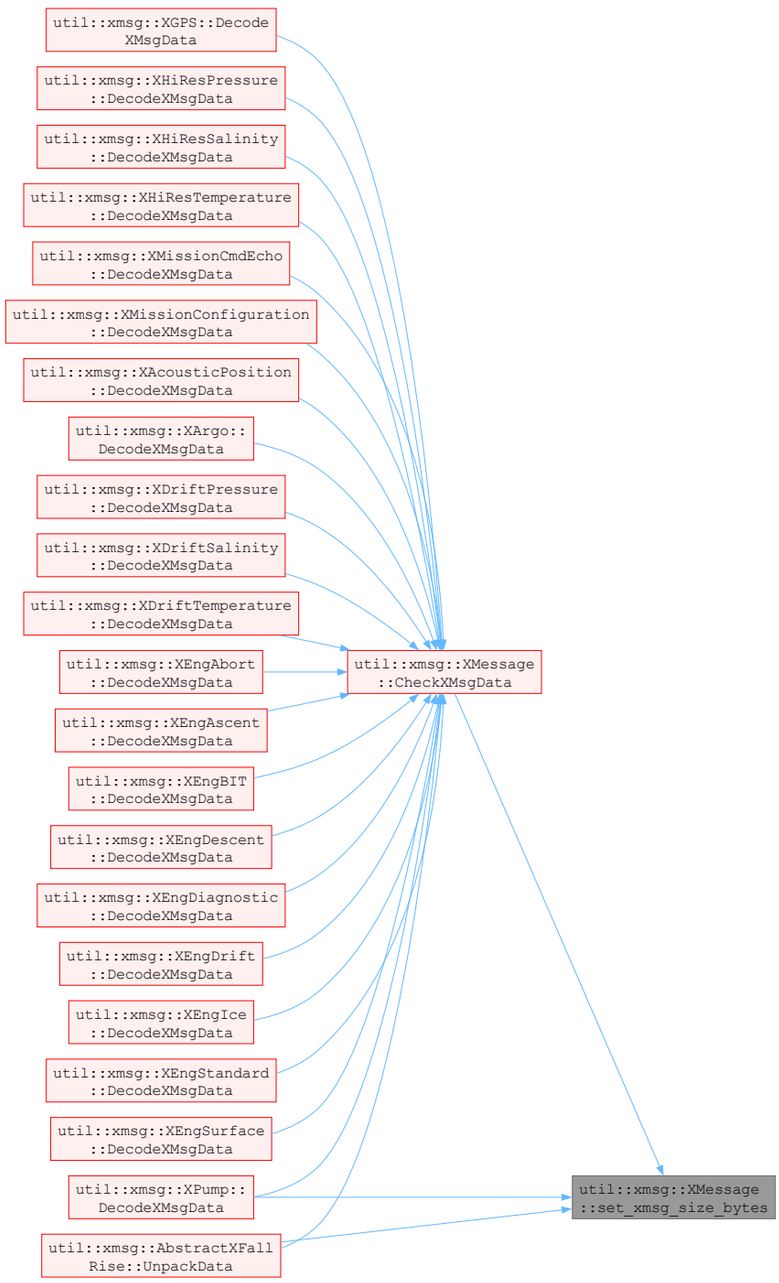
References util::xmsg::XMessage::xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XPump::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::Unpa

Here is the call graph for this function:



Here is the caller graph for this function:



10.107.3.21 UnpackSensorData()

```
virtual vector< int32_t > util::xmsg::AbstractXSensorData::UnpackSensorData (
            const vector< uint8_t > & data)  [inline], [protected], [virtual], [inherited]
```

Unpack the sensor data from the binary message.

This field unpacks the encoded sensor data from the message and stores the decoded sensor values in raw_sensor_data_. First the function checks the size of the message, and the data packing format. The message data is decoded according to the packing format type. Both first-difference and curvature encoding are supported. The encoding method is determined from the value of the high nibble of byte `data`[1].

Once this function completes successfully, then the data may be recovered in scientific units from the raw sensor values by applying the appropriate sensor gain and offset values.

Parameters

`in`	*data*	The binary message to unpack, in a vector of unsigned bytes.

Warning

The function assumes the message data is valid. Message validity should be ensured before this function is called.

Exceptions

util::log::MessageException	if • an error occurs while decoding the message, • the message is of insufficient length, or • an invalid packing format is encountered.

Returns

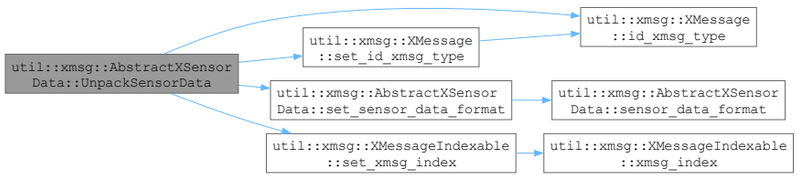
The sensor data raw counts, in a vector of signed integers.

Reimplemented in util::xmsg::AuxiliarySensorData.

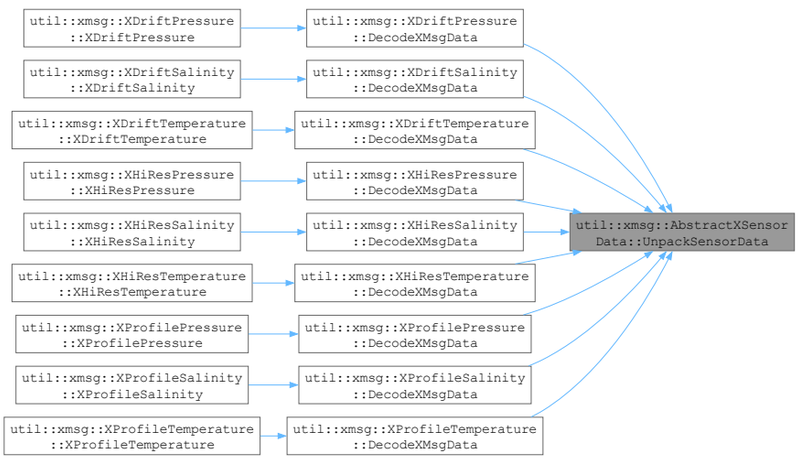
References CATCH_BLOCK, CATCH_NOTHROW, EXC_THROW, i24, util::xmsg::XMessage::id_xmsg_type(), ldbg, lwrn, util::xmsg::AbstractXSensorData::raw_sensor_data_, util::xmsg::XMessage::set_id_xmsg_type(), util::xmsg::AbstractXSensorData::s util::xmsg::XMessageIndexable::set_xmsg_index(), u16, u24, XMSG_SENS_MAX_SUB, util::xmsg::XMSG_TYPE_ID_PRES0, util::xmsg::XMSG_TYPE_ID_PRES_DRIFT8, util::xmsg::XMSG_TYPE_ID_PRES_HI_RES0, util::xmsg::XMSG_TYPE_ID_SALN0, util::xmsg::XMSG_TYPE_ID_SALN_DRIFT8, util::xmsg::XMSG_TYPE_ID_SALN_HI_RES0, util::xmsg::XMSG_TYPE_ID_TEMP0, util::xmsg::XMSG_TYPE_ID_TEMP_DRIFT8, util::xmsg::XMSG_TYPE_ID_TEMP_HI_RES0, util::xmsg::XSENS_FMT_CURVATURE, and util::xmsg::XSENS_FMT_DIFFERENCE.

Referenced by util::xmsg::XDriftPressure::DecodeXMsgData(), util::xmsg::XDriftSalinity::DecodeXMsgData(), util::xmsg::XDriftTemperatur util::xmsg::XHiResPressure::DecodeXMsgData(), util::xmsg::XHiResSalinity::DecodeXMsgData(), util::xmsg::XHiResTemperature::Decod util::xmsg::XProfilePressure::DecodeXMsgData(), util::xmsg::XProfileSalinity::DecodeXMsgData(), and DecodeXMsgData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.107.3.22 xmsg_index()

```
uint8_t util::xmsg::XMessageIndexable::xmsg_index () const  [inline], [inherited]
```

Get the index of this X-Message.

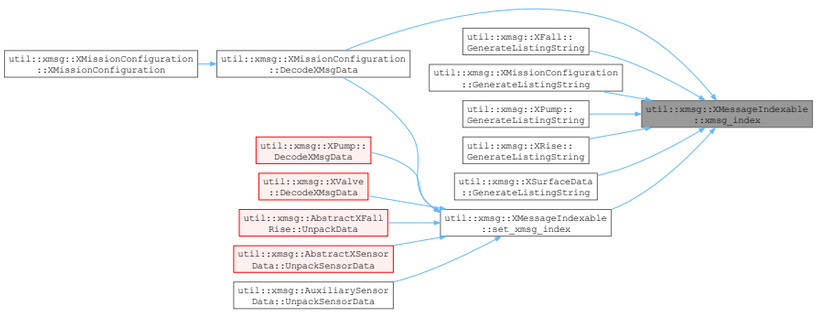
Returns

The index of this X-Message.

References util::xmsg::XMessageIndexable::xmsg_index_.

Referenced by util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing util::xmsg::XSurfaceData::GenerateListingString(), and util::xmsg::XMessageIndexable::set_xmsg_index().

Here is the caller graph for this function:



10.107.3.23 xmsg_size_bytes()

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes () const  [inline], [inherited]
```

Get the size of the X-Message, in bytes.

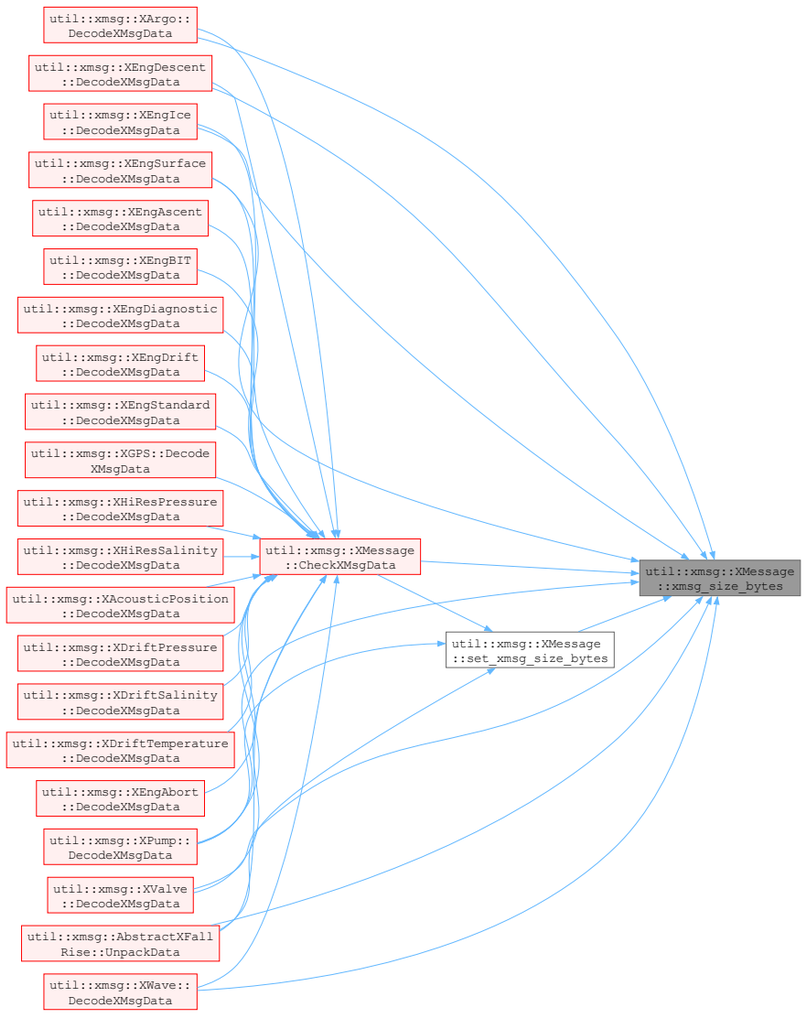
Returns

The size of the X-Message, in bytes.

References util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgI util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(), util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the caller graph for this function:



10.107.4 Member Data Documentation

10.107.4.1 id_dive_

```
int16_t util::xmsg::XMessage::id_dive_  [private], [inherited]
```

The dive or cycle number of this message.

Referenced by util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::set_id_dive().

10.107.4.2 id_instrument_

```
uint16_t util::xmsg::XMessage::id_instrument_  [private], [inherited]
```

The unique WHOI identifier of this instrument.

Referenced by util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::set_id_instrument().

10.107.4.3 id_xmsg_type_

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type_  [private], [inherited]
```

The X-message type identifier.

Referenced by util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::set_id_xmsg_type().

10.107.4.4 raw_sensor_data_

```
vector<int32_t> util::xmsg::AbstractXSensorData::raw_sensor_data_  [protected], [inherited]
```

The raw integer counts of the measured sensor data.

This vector of integers represents the "raw" sample values of the message. That is, by applying the appropriate gain and offset corrections, the sensor values may be recovered in scientific units. This data may represent pressure, temperature, salinity, or any other data that is recorded during the drift or profile phases of the dive cycle.

Referenced by util::xmsg::AbstractXSensorData::num_samples(), util::xmsg::AbstractXSensorData::raw_sensor_data(), util::xmsg::AbstractXSensorData::Sample(), util::xmsg::AbstractXSensorData::SampleValue(), util::xmsg::AbstractXSensorData::set_raw_ util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

10.107.4.5 sensor_data_format_

```
XSensorDataFormat_t util::xmsg::AbstractXSensorData::sensor_data_format_  [protected], [inherited]
```

The enumerated sensor data packing format.

This field indicates the format of encoded sensor data. The manner in which the sensor data is decoded is determined by the format of this field.

Referenced by util::xmsg::AbstractXSensorData::sensor_data_format(), and util::xmsg::AbstractXSensorData::set_sensor_data_format().

10.107.4.6 xmsg_index_

```
uint8_t util::xmsg::XMessageIndexable::xmsg_index_  [private], [inherited]
```

The index of this X-Message.

Referenced by util::xmsg::XMessageIndexable::set_xmsg_index(), and util::xmsg::XMessageIndexable::xmsg_index().

10.107.4.7 xmsg_size_bytes_

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes_  [private], [inherited]
```

The size of the X message, in bytes.

Referenced by util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes().

The documentation for this class was generated from the following file:

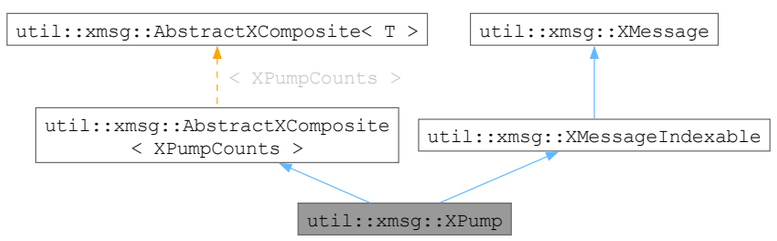
- data/xmsg/XProfileTemperature.h

10.108 util::xmsg::XPump Class Reference

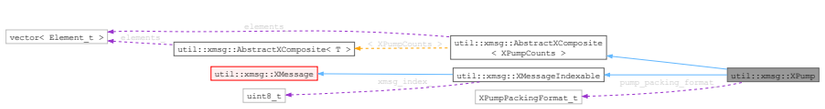
Class representing a pump data X-Message.

`#include <XPump.h>`

Inheritance diagram for util::xmsg::XPump:



Collaboration diagram for util::xmsg::XPump:



Public Member Functions

- XPump (int16_t dive, uint16_t whoi, const vector< uint8_t > &data)

  The parameterized constructor.
- ~XPump ()

  The destructor.
- void AppendAllElements (const static_vector< Element_t, I > &elements)

  Append a vector of elements to the vector of individual X-Messages elements that compose this composite object.
- virtual void AssignElement (const Element_t &element)

  Assign an element to this composite object.
- const uint8_t * CheckXMsgData (const vector< uint8_t > &data, int id, int minBytes, bool parseSize=true)

  Check the X-message for the correct ID, minimum size, and message terminator.
- void DecodeXMsgData (const vector< uint8_t > &data)

  Decode the pump data message from the binary data to decode.
- const vector< Element_t > & elements () const

  Get the vector of individual X-Messages elements that compose this composite object.
- const string GenerateListingString (uint16_t pg, uint16_t po, float pc) const

  Generate the listing file entry string for this pump data message.
- int16_t id_dive () const

  Get the dive or cycle number of this X-Message.
- uint16_t id_instrument () const

  Get the unique instrument identifier of this X-Message.
- XMsgTypeId_t id_xmsg_type () const

  Get the enumerated X-Message type identifier.
- virtual bool **IsEmpty** () const
- virtual size_t num_elements () const

  Get the number of individual elements that compose this composite object.
- XPumpPackingFormat_t pump_packing_format () const

  Get the enumerated pump packing format identifier.
- uint8_t xmsg_index () const

  Get the index of this X-Message.
- uint16_t xmsg_size_bytes () const

  Get the size of the X-Message, in bytes.

Static Public Member Functions

- static const string GenerateListingComment ()

  The the listing file comment string.

Protected Member Functions

- void AppendAllElements (const vector< Element_t > &elements)

  Append a vector of elements to the vector of individual X-Messages elements that compose this composite object.
- void AppendElement (const Element_t &element)

  Append one element to the vector of individual X-Messages elements that compose this composite object.
- void set_id_dive (int16_t id_dive)

  Set the dive or cycle number of this message.
- void set_id_instrument (uint16_t id_instrument)

  Set the unique instrument identifier.
- void set_id_xmsg_type (XMsgTypeId_t id_xmsg_type)

  Set the X-Message type identifier.
- void set_xmsg_index (uint8_t xmsg_index)

  Set the index of this X-Message.
- void set_xmsg_size_bytes (uint16_t xmsg_size_bytes)

  Set the size of the X-Message, in bytes.

Private Types

- typedef XPumpCounts Element_t

  Type definition for the element aggregated by this object.
- typedef ElementVec_t::const_iterator **ElementConstIter_t**
- typedef ElementVec_t::iterator **ElementIter_t**
- typedef vector< Element_t > **ElementVec_t**

Private Attributes

- ElementVec_t elements_

  The vector of individual X-Messages elements that compose this composite object.
- int16_t id_dive_

  The dive or cycle number of this message.
- uint16_t id_instrument_

  The unique WHOI identifier of this instrument.
- XMsgTypeId_t id_xmsg_type_

  The X-message type identifier.
- XPumpPackingFormat_t pump_packing_format_

  The packing format of this pump data message.
- uint8_t xmsg_index_

  The index of this X-Message.
- uint16_t xmsg_size_bytes_

  The size of the X message, in bytes.

10.108.1 Detailed Description

Class representing a pump data X-Message.

10.108.2 Member Typedef Documentation

10.108.2.1 Element_t

```
typedef XPumpCounts util::xmsg::AbstractXComposite< XPumpCounts >::Element_t  [private], [inherited]
```

Type definition for the element aggregated by this object.

10.108.3 Constructor & Destructor Documentation

10.108.3.1 XPump()

```
util::xmsg::XPump::XPump (
            int16_t dive,
            uint16_t whoi,
            const vector< uint8_t > & data)  [inline]
```

The parameterized constructor.

Parameters

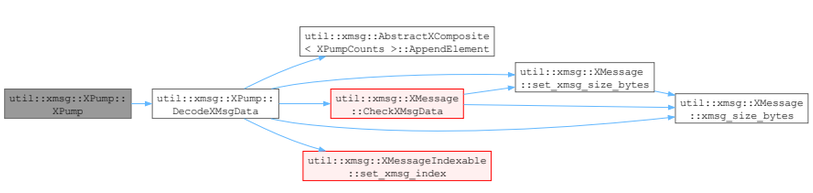
in	*dive*	The dive or cycle number of this message.
in	*whoi*	The unique WHOI identifier.
in	*data*	The binary X-message data to decode.

Exceptions

util::log::MessageException	if an decode error occurs.

References DecodeXMsgData().

Here is the call graph for this function:



10.108.3.2 ~XPump()

```
util::xmsg::XPump::~XPump ()  [inline]
```

The destructor.

10.108.4 Member Function Documentation

10.108.4.1 AppendAllElements() [1/2]

```
void util::xmsg::AbstractXComposite< XPumpCounts >::AppendAllElements (
            const static_vector< Element_t, I > & elements)  [inline], [inherited]
```

Append a vector of elements to the vector of individual X-Messages elements that compose this composite object.

Parameters

in	*elements*	The vector of elements to append.

10.108.4.2 AppendAllElements() [2/2]

```
void util::xmsg::AbstractXComposite< XPumpCounts >::AppendAllElements (
            const vector< Element_t > & elements)  [inline], [protected], [inherited]
```

Append a vector of elements to the vector of individual X-Messages elements that compose this composite object.

Parameters

in	*elements*	The vector of elements to append.

10.108.4.3 AppendElement()

```
void util::xmsg::AbstractXComposite< XPumpCounts >::AppendElement (
            const Element_t & element)  [inline], [protected], [inherited]
```

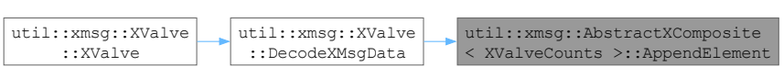
Append one element to the vector of individual X-Messages elements that compose this composite object.

Parameters

in	*element*	The element to append.

Referenced by DecodeXMsgData().

Here is the caller graph for this function:



10.108.4.4 AssignElement()

```
virtual void util::xmsg::AbstractXComposite< XPumpCounts >::AssignElement (
            const Element_t & element)  [inline], [virtual], [inherited]
```

Assign an element to this composite object.

The default implementation appends the element; derived classes may override it to insert elements in order or reject duplicates.

Parameters

in	*element*	The element to assign.

10.108.4.5 CheckXMsgData()

```
const uint8_t * util::xmsg::XMessage::CheckXMsgData (
            const vector< uint8_t > & data,
            int id,
            int minBytes,
            bool parseSize = true)  [inline], [inherited]
```

Check the X-message for the correct ID, minimum size, and message terminator.

Parameters

in	*data*	The X-message binary data to check.
in	*id*	The expected ID of the X-message. If this field is less that 0, then the ID check will be skipped.
in	*minBytes*	The minimum number of bytes expected in a valid message.
in	*parseSize*	If TRUE, then the message size field will be parsed from the X-Message. Otherwise, the size field will not be parsed.

Exceptions

util::log::MessageException	if • the message does not contain the expected ID, • the message does not contain the minimum number of expected bytes, • the message does not contain a terminator (';') in the correct location, or, • any other error occurs.

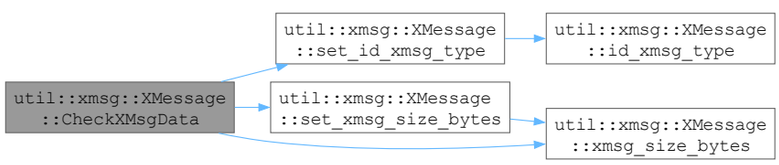
Returns

A pointer to the underlying data buffer of data.

References CATCH_BLOCK, EXC_THROW, lwrn, util::xmsg::XMessage::set_id_xmsg_type(), util::xmsg::XMessage::set_xmsg_size_byte
u16, and util::xmsg::XMessage::xmsg_size_bytes().

Referenced by util::xmsg::XAcousticPosition::DecodeXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XDriftPressure::Dec
util::xmsg::XDriftSalinity::DecodeXMsgData(), util::xmsg::XDriftTemperature::DecodeXMsgData(), util::xmsg::XEngAbort::DecodeXMsgDa
util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngBIT::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(),
util::xmsg::XEngDiagnostic::DecodeXMsgData(), util::xmsg::XEngDrift::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(),
util::xmsg::XEngStandard::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XGPS::DecodeXMsgData(),
util::xmsg::XHiResPressure::DecodeXMsgData(), util::xmsg::XHiResSalinity::DecodeXMsgData(), util::xmsg::XHiResTemperature::Decod
util::xmsg::XMissionCmdEcho::DecodeXMsgData(), util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XProfilePressure:
util::xmsg::XProfileSalinity::DecodeXMsgData(), util::xmsg::XProfileTemperature::DecodeXMsgData(), DecodeXMsgData(),
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the call graph for this function:



10.108.4.6 DecodeXMsgData()

```
void util::xmsg::XPump::DecodeXMsgData (
            const vector< uint8_t > & data)  [inline], [virtual]
```

Decode the pump data message from the binary data to decode.

Parameters

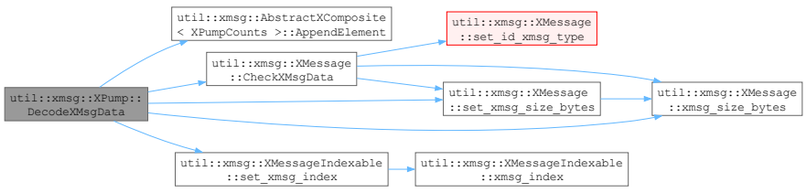
in	*data*	The binary data to decode.

Implements util::xmsg::XMessageIndexable.

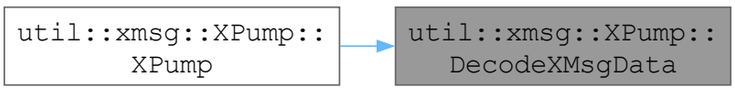
References util::xmsg::AbstractXComposite< XPumpCounts >::AppendElement(), CATCH_BLOCK, util::xmsg::XMessage::CheckXMsgD
lwrn, pump_packing_format_, util::xmsg::XMessageIndexable::set_xmsg_index(), util::xmsg::XMessage::set_xmsg_size_bytes(),
XMSG_BAD_U16, util::xmsg::XMessage::xmsg_size_bytes(), util::xmsg::XPHASE_CODE_UNKNOWN, util::xmsg::XPUMP_PACK_LEGA
util::xmsg::XPUMP_PACK_STANDARD, and util::xmsg::XPUMP_PACK_STANDARD2.

Referenced by XPump().

Here is the call graph for this function:



Here is the caller graph for this function:



10.108.4.7 elements()

```
const vector< Element_t > & util::xmsg::AbstractXComposite< XPumpCounts >::elements () const
[inline], [inherited]
```

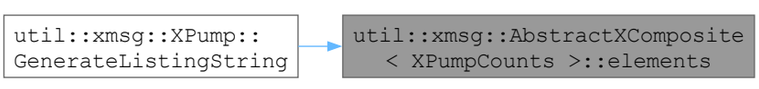
Get the vector of individual X-Messages elements that compose this composite object.

Returns

The vector of individual X-Messages elements that compose this composite object.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.108.4.8 GenerateListingComment()

```
static const string util::xmsg::XPump::GenerateListingComment ()  [inline], [static]
```

The the listing file comment string.

Returns

The listing file comment string.

References CATCH_BLOCK.

Referenced by util::xmsg::XCompositePump::GenerateListingString().

Here is the caller graph for this function:

10.108.4.9 GenerateListingString()

```
const string util::xmsg::XPump::GenerateListingString (
            uint16_t pg,
            uint16_t po,
            float pc) const  [inline]
```

Generate the listing file entry string for this pump data message.

Parameters

in	*pg*	The pressure sensor sensitivity scale factor, in LSB/dBar.
in	*po*	The pressure sensor offset, in dBar.
in	*pc*	The pressure correction, in dBar, to be subtracted computed pressure value.

Exceptions

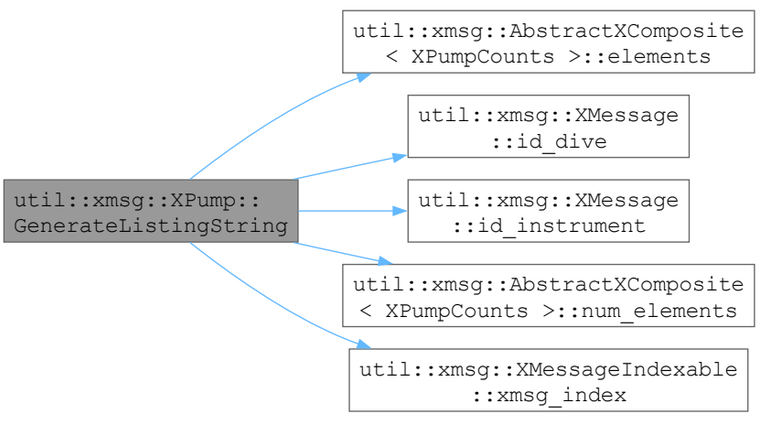
log::MessageException	if an error occurs while generating the listing string.

Returns

The listing file entry string for this pump data message.

References CATCH_BLOCK, util::xmsg::AbstractXComposite< XPumpCounts >::elements(), util::xmsg::XMessage::id_dive(), util::xmsg::XMessage::id_instrument(), util::xmsg::AbstractXComposite< XPumpCounts >::num_elements(), and util::xmsg::XMessageIndexable::xmsg_index().

Here is the call graph for this function:



10.108.4.10 id_dive()

```
int16_t util::xmsg::XMessage::id_dive () const  [inline], [inherited]
```

Get the dive or cycle number of this X-Message.

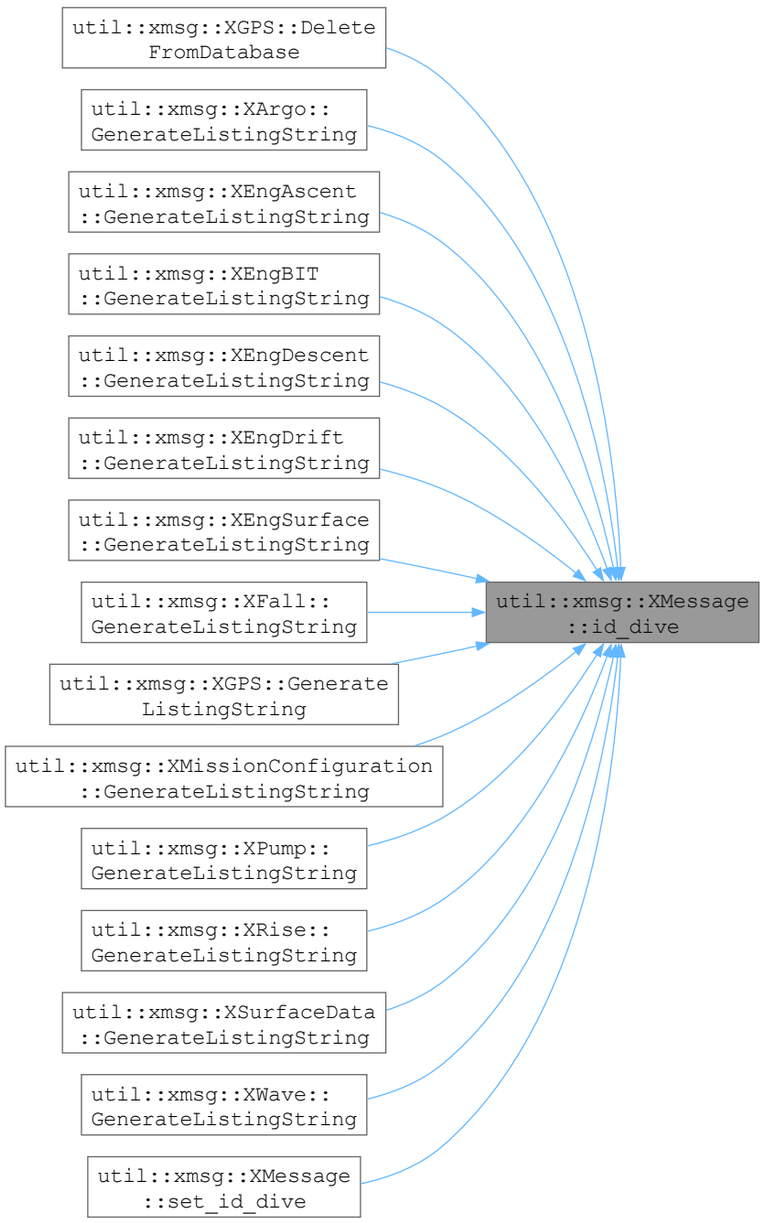
Returns

The dive or cycle number of this X-Message.

References util::xmsg::XMessage::id_dive_.

Referenced by util::xmsg::XGPS::DeleteFromDatabase(), util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateLis
util::xmsg::XEngBIT::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingStr
util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(),
util::xmsg::XMissionConfiguration::GenerateListingString(), GenerateListingString(), util::xmsg::XRise::GenerateListingString(),
util::xmsg::XSurfaceData::GenerateListingString(), util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_dive().

Here is the caller graph for this function:



10.108.4.11 id_instrument()

```
uint16_t util::xmsg::XMessage::id_instrument () const  [inline], [inherited]
```

Get the unique instrument identifier of this X-Message.

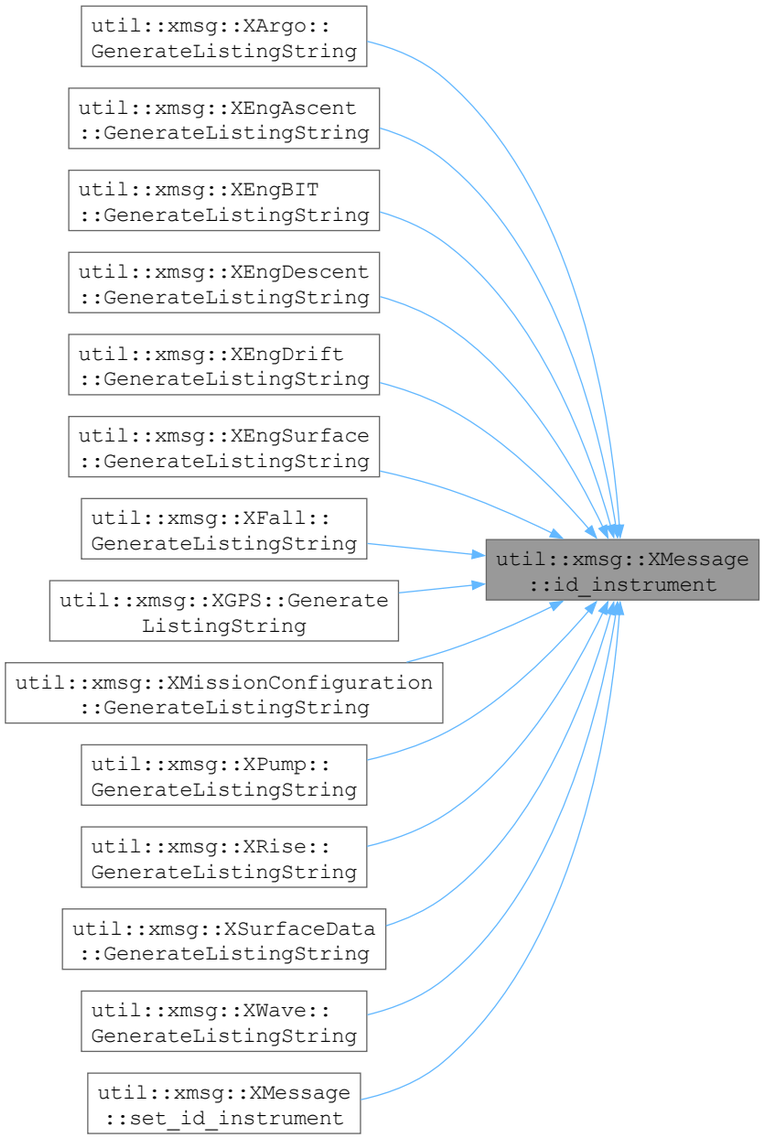
Returns

The unique instrument identifier of this X-Message.

References util::xmsg::XMessage::id_instrument_.

Referenced by util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngBIT::Generate
util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), util::xmsg::XEngSurface::GenerateListin
util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingS
GenerateListingString(), util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(),
util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_instrument().

Here is the caller graph for this function:



10.108.4.12 id_xmsg_type()

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type () const  [inline], [inherited]
```

Get the enumerated X-Message type identifier.

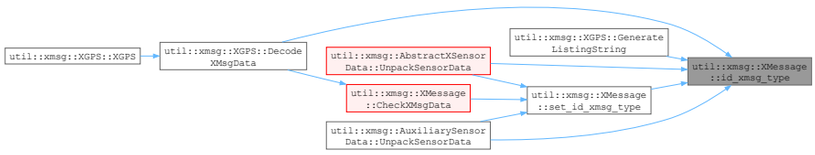
Returns

The enumerated X-Message type identifier.

References util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMessage::set_id_xmsg_typ
util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the caller graph for this function:



10.108.4.13 num_elements()

```
virtual size_t util::xmsg::AbstractXComposite< XPumpCounts >::num_elements () const  [inline],
[virtual], [inherited]
```

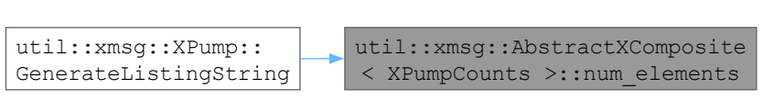
Get the number of individual elements that compose this composite object.

Returns

The number of individual elements that compose this composite object.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.108.4.14 pump_packing_format()

```
XPumpPackingFormat_t util::xmsg::XPump::pump_packing_format () const  [inline]
```

Get the enumerated pump packing format identifier.

Returns

The enumerated pump packing format identifier.

References pump_packing_format_.

10.108.4.15 set_id_dive()

```
void util::xmsg::XMessage::set_id_dive (
            int16_t id_dive)  [inline], [protected], [inherited]
```

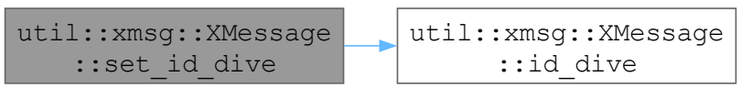
Set the dive or cycle number of this message.

Parameters

in	*id_dive*	The dive or cycle number of this message to set.

References util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::id_dive_.

Here is the call graph for this function:



10.108.4.16 set_id_instrument()

```
void util::xmsg::XMessage::set_id_instrument (
            uint16_t id_instrument)  [inline], [protected], [inherited]
```

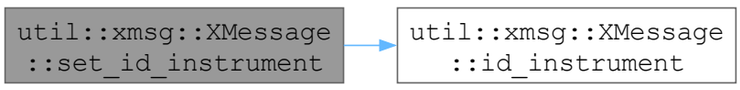
Set the unique instrument identifier.

Parameters

in	*id_instrument*	The unique instrument identifier to set.

References util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::id_instrument_.

Here is the call graph for this function:



10.108.4.17 set_id_xmsg_type()

```
void util::xmsg::XMessage::set_id_xmsg_type (
            XMsgTypeId_t id_xmsg_type)  [inline], [protected], [inherited]
```

Set the X-Message type identifier.

Parameters

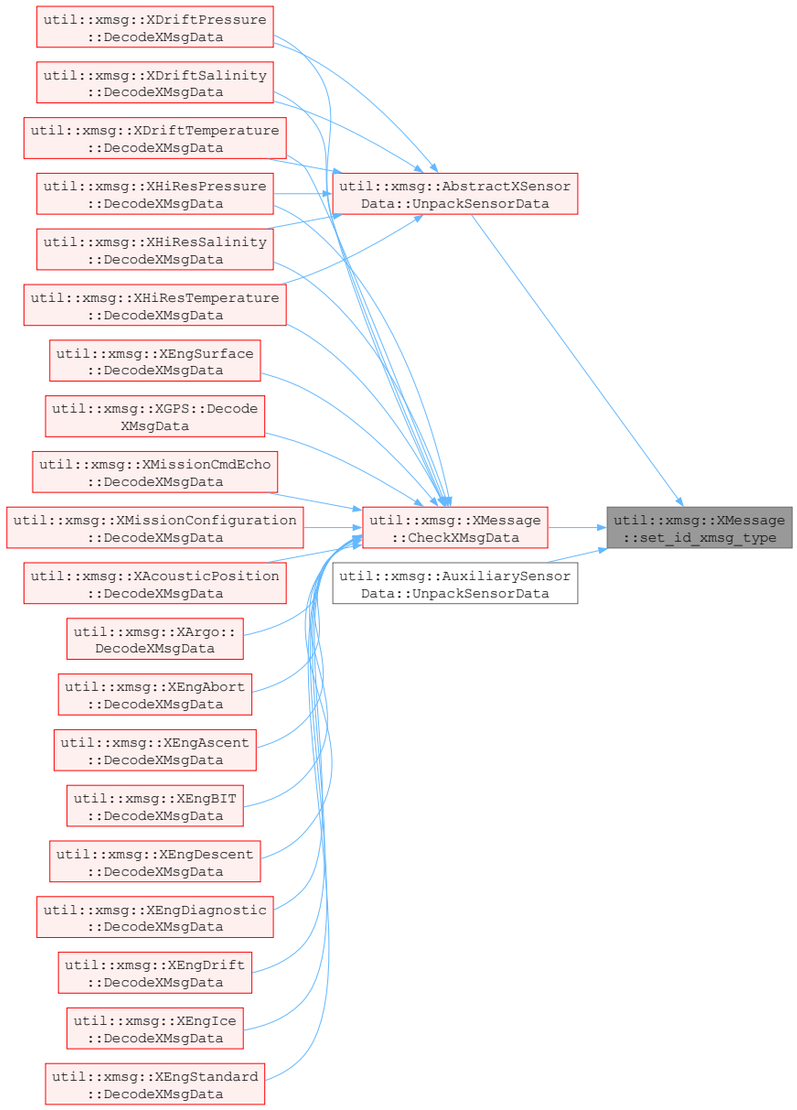
in	*id_xmsg_type*	The X-Message type identifier to set.

References util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:

Here is the caller graph for this function:



10.108.4.18 set_xmsg_index()

```
void util::xmsg::XMessageIndexable::set_xmsg_index (
            uint8_t xmsg_index)  [inline], [protected], [inherited]
```

Set the index of this X-Message.

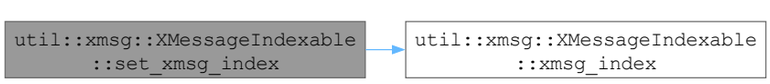
Parameters

in	*xmsg_index*	The index of this X-Message.

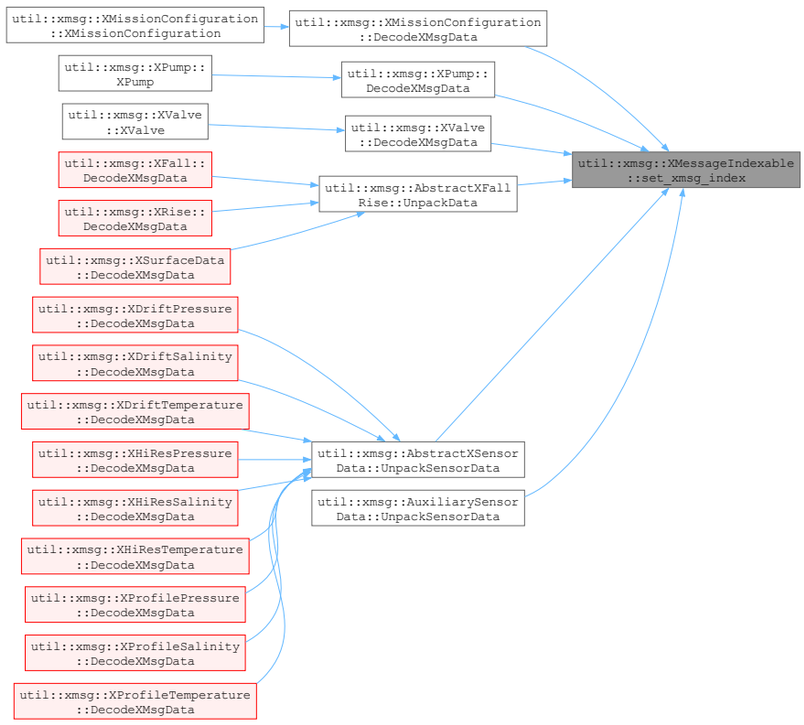
References util::xmsg::XMessageIndexable::xmsg_index(), and util::xmsg::XMessageIndexable::xmsg_index_.

Referenced by util::xmsg::XMissionConfiguration::DecodeXMsgData(), DecodeXMsgData(), util::xmsg::XValve::DecodeXMsgData(), util::xmsg::AbstractXFallRise::UnpackData(), util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData

Here is the call graph for this function:



Here is the caller graph for this function:



10.108.4.19 set_xmsg_size_bytes()

```
void util::xmsg::XMessage::set_xmsg_size_bytes (
            uint16_t xmsg_size_bytes)  [inline], [protected], [inherited]
```

Set the size of the X-Message, in bytes.

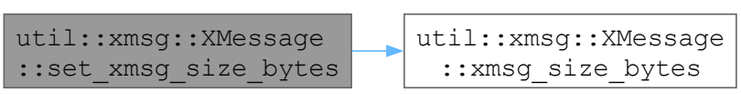
Parameters

in	*xmsg_size_bytes*	The size of the X-Message to set, in bytes.

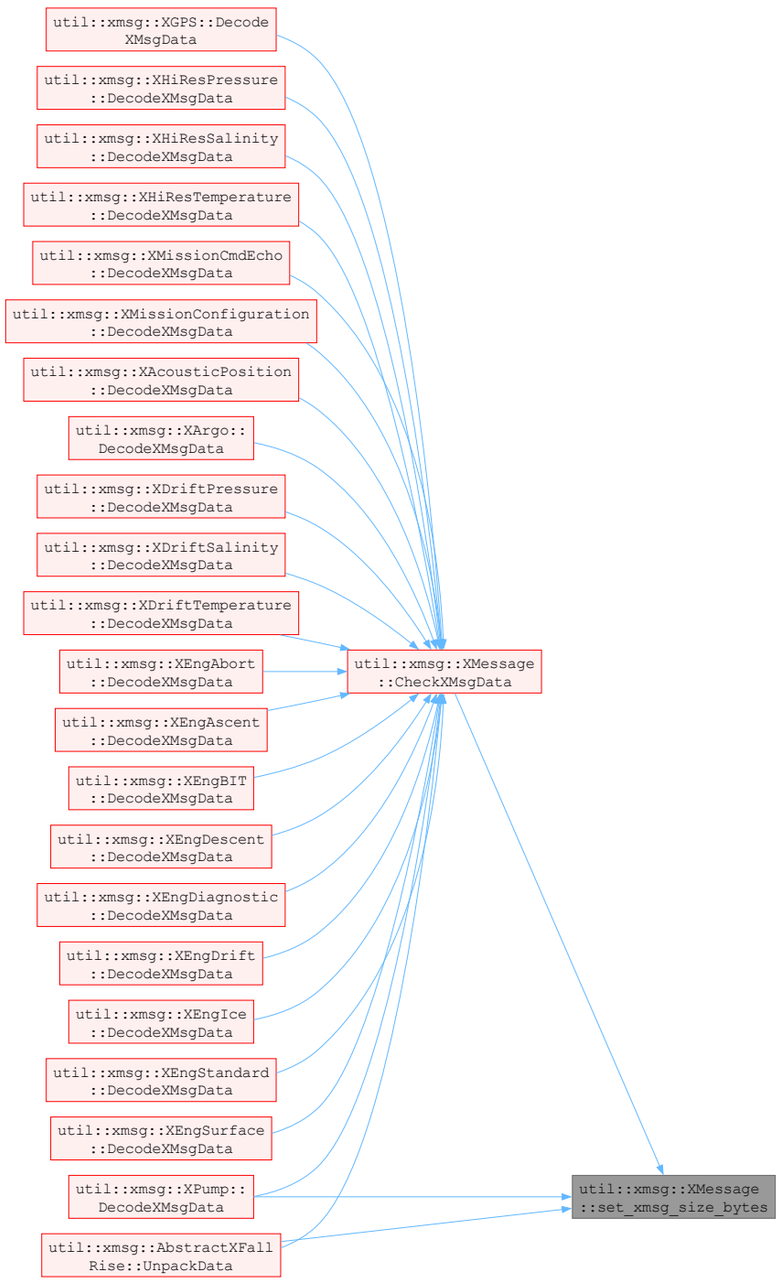
References util::xmsg::XMessage::xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), DecodeXMsgData(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.108.4.20 xmsg_index()

```
uint8_t util::xmsg::XMessageIndexable::xmsg_index () const  [inline], [inherited]
```

Get the index of this X-Message.

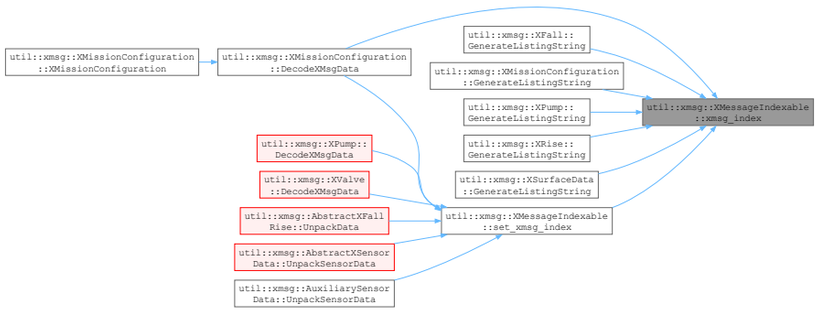
Returns

The index of this X-Message.

References util::xmsg::XMessageIndexable::xmsg_index_.

Referenced by util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingString(), GenerateListingString(), util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(), and util::xmsg::XMessageIndexable::set_xmsg_index().

Here is the caller graph for this function:



10.108.4.21 xmsg_size_bytes()

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes () const  [inline], [inherited]
```

Get the size of the X-Message, in bytes.

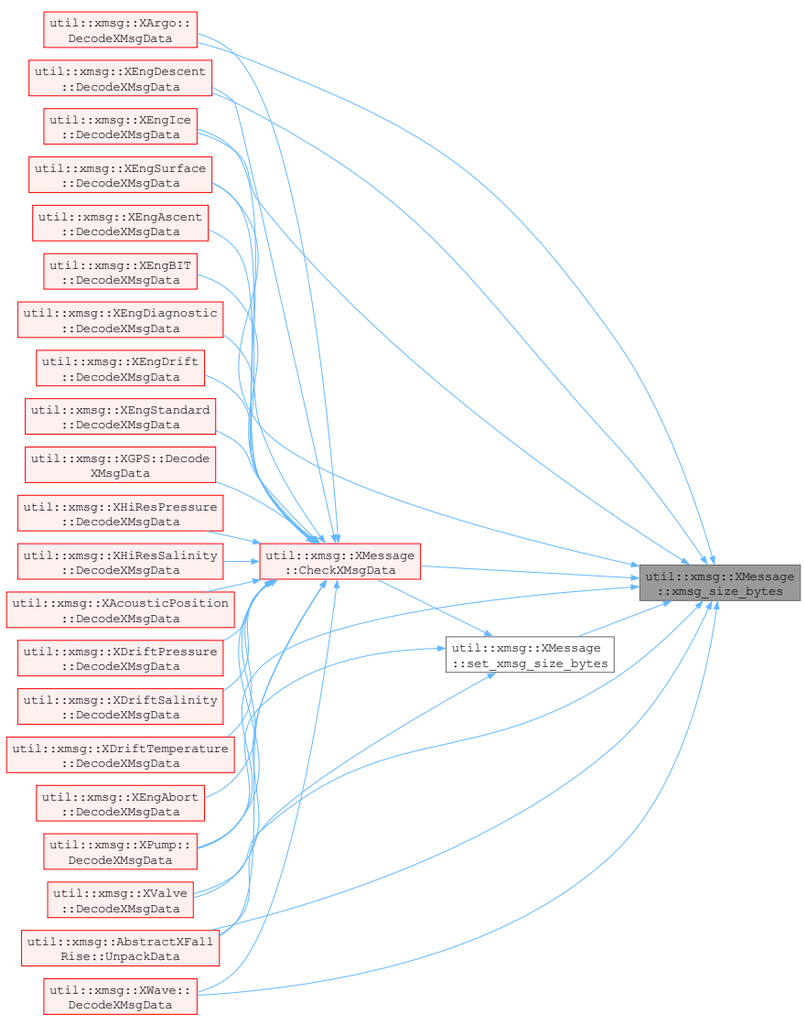
Returns

The size of the X-Message, in bytes.

References util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgI util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), DecodeXMsgData(), util::xmsg::XValve::DecodeX util::xmsg::XWave::DecodeXMsgData(), util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the caller graph for this function:



10.108.5 Member Data Documentation

10.108.5.1 elements_

```
ElementVec_t util::xmsg::AbstractXComposite< XPumpCounts >::elements_  [private], [inherited]
```

The vector of individual X-Messages elements that compose this composite object.

10.108.5.2 id_dive_

```
int16_t util::xmsg::XMessage::id_dive_  [private], [inherited]
```

The dive or cycle number of this message.

Referenced by util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::set_id_dive().

10.108.5.3 id_instrument_

```
uint16_t util::xmsg::XMessage::id_instrument_  [private], [inherited]
```

The unique WHOI identifier of this instrument.

Referenced by util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::set_id_instrument().

10.108.5.4 id_xmsg_type_

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type_  [private], [inherited]
```

The X-message type identifier.

Referenced by util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::set_id_xmsg_type().

10.108.5.5 pump_packing_format_

```
XPumpPackingFormat_t util::xmsg::XPump::pump_packing_format_  [private]
```

The packing format of this pump data message.

Referenced by DecodeXMsgData(), and pump_packing_format().

10.108.5.6 xmsg_index_

```
uint8_t util::xmsg::XMessageIndexable::xmsg_index_  [private], [inherited]
```

The index of this X-Message.

Referenced by util::xmsg::XMessageIndexable::set_xmsg_index(), and util::xmsg::XMessageIndexable::xmsg_index().

10.108.5.7 xmsg_size_bytes_

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes_  [private], [inherited]
```

The size of the X message, in bytes.

Referenced by util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes().

The documentation for this class was generated from the following file:

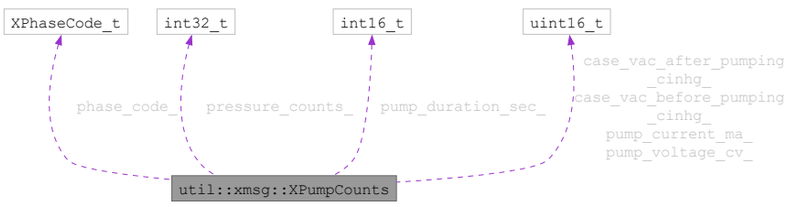
- data/xmsg/XPump.h

10.109 util::xmsg::XPumpCounts Class Reference

Class representing a single pump data measurement for a pump data time-series X-Message.

`#include <XPumpCounts.h>`

Collaboration diagram for util::xmsg::XPumpCounts:



Public Member Functions

- XPumpCounts (XPhaseCode_t code, int32_t count, int16_t sec, uint16_t v, uint16_t i, uint16_t vac0, uint16_t vac1) ATTR_NO_EXCEPTIONS

  Parameterized constructor.
- ~XPumpCounts ()

  Destructor.
- uint16_t case_vac_after_pumping_cinhg () const

  Get the measured vacuum after pumping, in 0.01inHg.
- template<typename T = float>
  T case_vac_after_pumping_inhg () const

  Get the measured vacuum after pumping, in inHg.
- uint16_t case_vac_before_pumping_cinhg () const

  Get the measured vacuum before pumping, in 0.01inHg.
- template<typename T = float>
  T case_vac_before_pumping_inhg () const

  Get the measured vacuum before pumping, in inHg.
- const string GenerateListingString (uint16_t pg, uint16_t po, float pres_corr) const

  Generate a listing entry string representation of this pump counts object.

Protected Member Functions

- template<typename T = double>
  T PressureValue (uint16_t pg, uint16_t po, float pres_corr) const

  Get the measured pressure value, in dBar.
- template<typename T = double>
  T pump_current_a () const

  Get the measured pump current, in amperes.
- template<typename T = double>
  T pump_voltage_v () const

  Get the measured pump voltage, in volts.

Private Attributes

- uint16_t case_vac_after_pumping_cinhg_

  The measured vacuum after pumping, in 0.01inHg.
- uint16_t case_vac_before_pumping_cinhg_

  The measured vacuum before pumping, in 0.01inHg.
- XPhaseCode_t phase_code_

  The phase code of this pump data measurement.
- int32_t pressure_counts_

  The raw pressure counts of the measured pressure during pumping.
- uint16_t pump_current_ma_

  The measured current of the pump supply during pumping, in milliamps.
- int16_t pump_duration_sec_

  The duration that the pump ran, in seconds.
- uint16_t pump_voltage_cv_

  The measured voltage of the pump supply during pumping, in 0.01 volts.

10.109.1 Detailed Description

Class representing a single pump data measurement for a pump data time-series X-Message.

10.109.2 Constructor & Destructor Documentation

10.109.2.1 XPumpCounts()

```
util::xmsg::XPumpCounts::XPumpCounts (
            XPhaseCode_t code,
            int32_t count,
            int16_t sec,
            uint16_t v,
            uint16_t i,
            uint16_t vac0,
            uint16_t vac1)  [inline]
```

Parameterized constructor.

Parameters

in	*code*	The phase code of this pump data measurement.
in	*count*	The raw pressure counts of the measured pressure during pumping.
in	*sec*	The duration that the pump ran, in seconds.
in	*v*	The measured voltage of the pump supply during pumping, in 0.01 volts.
in	*i*	The measured current of the pump supply during pumping, in milliamps.
in	*vac0*	The measured vacuum counts before pumping.
in	*vac1*	The measured vacuum counts after pumping.

10.109.2.2 ~XPumpCounts()

```
util::xmsg::XPumpCounts::~XPumpCounts ()  [inline]
```

Destructor.

10.109.3 Member Function Documentation

10.109.3.1 case_vac_after_pumping_cinhg()

```
uint16_t util::xmsg::XPumpCounts::case_vac_after_pumping_cinhg () const  [inline]
```

Get the measured vacuum after pumping, in 0.01inHg.

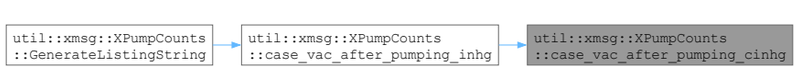
Returns

The measured vacuum after pumping, in 0.01inHg.

References case_vac_after_pumping_cinhg_.

Referenced by case_vac_after_pumping_inhg().

Here is the caller graph for this function:



10.109.3.2 case_vac_after_pumping_inhg()

```
template<typename T = float>
T util::xmsg::XPumpCounts::case_vac_after_pumping_inhg () const  [inline]
```

Get the measured vacuum after pumping, in inHg.

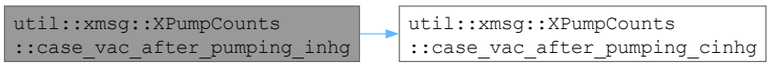
Returns

The measured vacuum after pumping, in inHg.

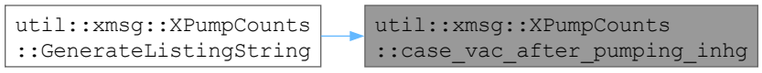
References case_vac_after_pumping_cinhg().

Referenced by GenerateListingString().

Here is the call graph for this function:



Here is the caller graph for this function:



10.109.3.3 case_vac_before_pumping_cinhg()

```
uint16_t util::xmsg::XPumpCounts::case_vac_before_pumping_cinhg () const  [inline]
```

Get the measured vacuum before pumping, in 0.01inHg.

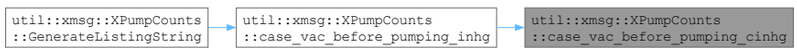
Returns

The measured vacuum before pumping, in 0.01inHg.

References case_vac_before_pumping_cinhg_.

Referenced by case_vac_before_pumping_inhg().

Here is the caller graph for this function:



10.109.3.4 case_vac_before_pumping_inhg()

```
template<typename T = float>
T util::xmsg::XPumpCounts::case_vac_before_pumping_inhg () const  [inline]
```

Get the measured vacuum before pumping, in inHg.

Returns

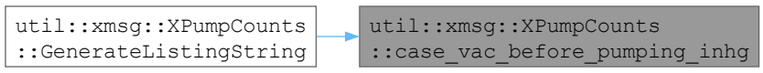
The measured vacuum before pumping, in inHg.

References case_vac_before_pumping_cinhg().

Referenced by GenerateListingString().

Here is the call graph for this function:

Here is the caller graph for this function:



10.109.3.5 GenerateListingString()

```
const string util::xmsg::XPumpCounts::GenerateListingString (
            uint16_t pg,
            uint16_t po,
            float pres_corr) const  [inline]
```

Generate a listing entry string representation of this pump counts object.

Parameters

in	*pg*	The pressure sensor sensitivity scale factor, in LSB/dBar.
in	*po*	The pressure sensor offset, in dBar.
in	*pres_corr*	The pressure correction, in dBar, to be subtracted computed pressure value.

Exceptions

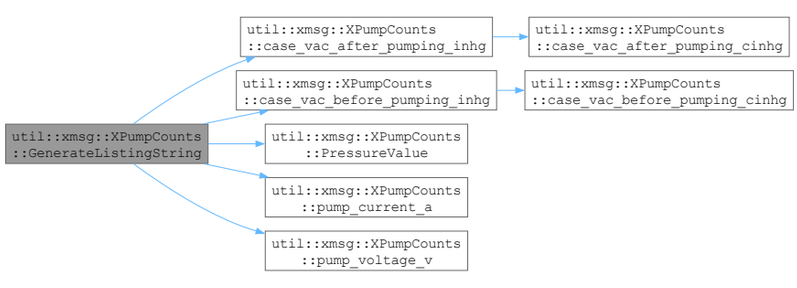
log::MessageException	if an error occurs while generating the listing string.

Returns

A listing entry string representation of this pump counts object.

References case_vac_after_pumping_inhg(), case_vac_before_pumping_inhg(), CATCH_BLOCK, PressureValue(), pump_current_a(), pump_duration_sec_, and pump_voltage_v().

Here is the call graph for this function:



10.109.3.6 PressureValue()

```
template<typename T = double>
T util::xmsg::XPumpCounts::PressureValue (
            uint16_t pg,
            uint16_t po,
            float pres_corr) const  [inline], [protected]
```

Get the measured pressure value, in dBar.

Parameters

in	*pg*	The pressure sensor sensitivity scale factor, in LSB/dBar.
in	*po*	The pressure sensor offset, in dBar.
in	*pres_corr*	The pressure correction, in dBar, to be subtracted computed pressure value.

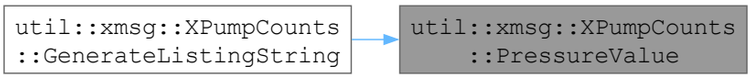
Returns

The measured pressure value, in dBar.

References pressure_counts_, XMSG_SENS_BAD_PRES, and XMSG_SENS_MAX_PRES.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.109.3.7 pump_current_a()

```
template<typename T = double>
T util::xmsg::XPumpCounts::pump_current_a () const  [inline], [protected]
```

Get the measured pump current, in amperes.

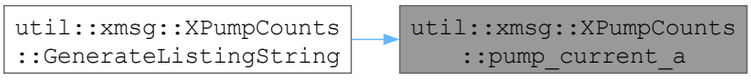
Returns

The measured pump current, in amperes.

References pump_current_ma_.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.109.3.8 pump_voltage_v()

```
template<typename T = double>
T util::xmsg::XPumpCounts::pump_voltage_v () const  [inline], [protected]
```

Get the measured pump voltage, in volts.

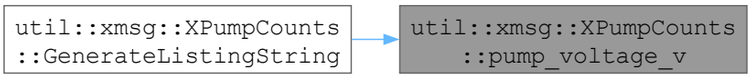
Returns

The measured pump voltage, in volts.

References pump_voltage_cv_.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.109.4 Member Data Documentation

10.109.4.1 case_vac_after_pumping_cinhg_

```
uint16_t util::xmsg::XPumpCounts::case_vac_after_pumping_cinhg_  [private]
```

The measured vacuum after pumping, in 0.01inHg.

Referenced by case_vac_after_pumping_cinhg().

10.109.4.2 case_vac_before_pumping_cinhg_

```
uint16_t util::xmsg::XPumpCounts::case_vac_before_pumping_cinhg_  [private]
```

The measured vacuum before pumping, in 0.01inHg.

Referenced by case_vac_before_pumping_cinhg().

10.109.4.3 phase_code_

```
XPhaseCode_t util::xmsg::XPumpCounts::phase_code_  [private]
```

The phase code of this pump data measurement.

10.109.4.4 pressure_counts_

```
int32_t util::xmsg::XPumpCounts::pressure_counts_  [private]
```

The raw pressure counts of the measured pressure during pumping.

Referenced by PressureValue().

10.109.4.5 pump_current_ma_

```
uint16_t util::xmsg::XPumpCounts::pump_current_ma_  [private]
```

The measured current of the pump supply during pumping, in milliamps.

Referenced by pump_current_a().

10.109.4.6 pump_duration_sec_

```
int16_t util::xmsg::XPumpCounts::pump_duration_sec_  [private]
```

The duration that the pump ran, in seconds.

Referenced by GenerateListingString().

10.109.4.7 pump_voltage_cv_

```
uint16_t util::xmsg::XPumpCounts::pump_voltage_cv_  [private]
```

The measured voltage of the pump supply during pumping, in 0.01 volts.

Referenced by pump_voltage_v().

The documentation for this class was generated from the following file:

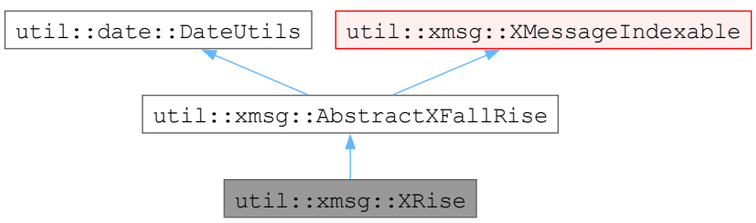
- data/xmsg/common/XPumpCounts.h

10.110 util::xmsg::XRise Class Reference

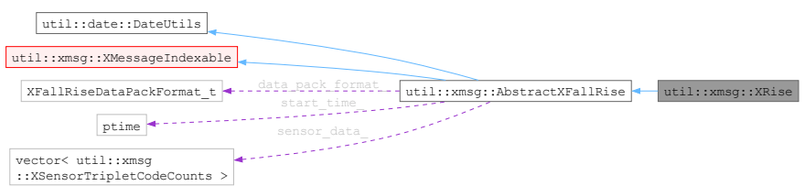
Class representing a rise data X-Message.

```
#include <XRise.h>
```

Inheritance diagram for util::xmsg::XRise:



Collaboration diagram for util::xmsg::XRise:



Public Member Functions

- XRise (int16_t dive, uint16_t whoi, const vector< uint8_t > &data)

  The parameterized constructor.
- ~XRise ()

  The destructor.
- const uint8_t * CheckXMsgData (const vector< uint8_t > &data, int id, int minBytes, bool parseSize=true)

  Check the X-message for the correct ID, minimum size, and message terminator.
- XFallRiseDataPackFormat_t data_pack_format () const

  Get the enumerated data packing format type.
- void DecodeXMsgData (const vector< uint8_t > &data)

  Decode the rise data X-Message from the binary message data.
- const string GenerateListingString (uint16_t pg, uint16_t po, uint16_t sg, uint16_t so, uint16_t tg, uint16_t to, float pc) const

  Get the listing entry string for this rise data X-Message.
- int16_t id_dive () const

  Get the dive or cycle number of this X-Message.
- uint16_t id_instrument () const

  Get the unique instrument identifier of this X-Message.
- XMsgTypeId_t id_xmsg_type () const

  Get the enumerated X-Message type identifier.
- size_t num_samples () const

  Get the number of samples in this message.
- const vector< XSensorTripletCodeCounts > & sensor_data () const

  Get the vector of sensor data triplets.
- const ptime & start_time () const

  Get the start time of this message.
- uint8_t xmsg_index () const

  Get the index of this X-Message.
- uint16_t xmsg_size_bytes () const

  Get the size of the X-Message, in bytes.

Static Public Member Functions

- static string GenerateCurrentUtcTimeIso8601String ()

  Generate an ISO8601 format date/time string representation of the current time, in UTC.
- static string GenerateIso8601String (const ptime &p, bool extended=true)

  Generate an ISO8601 format date/time string representation of a POSIX time object.
- static double GenerateJulianDay (const ptime &dt, const ptime &ref_dt)

  Convert a date/time to fractional days since a reference date/time.
- static const string GenerateListingComment ()

  Get the listing entry comment string.
- static ptime GetCurrentUtcTime ()

  Generate a POSIX time object representing the current date and time (rounded to the nearest second).

Protected Member Functions

- void set_data_pack_format (XFallRiseDataPackFormat_t data_pack_format)

  Set the enumerated data packing format value for this message.
- void set_id_dive (int16_t id_dive)

  Set the dive or cycle number of this message.
- void set_id_instrument (uint16_t id_instrument)

  Set the unique instrument identifier.
- void set_id_xmsg_type (XMsgTypeId_t id_xmsg_type)

  Set the X-Message type identifier.
- void set_start_time (const ptime &start_time)

  Set the start time of this message.
- void set_xmsg_index (uint8_t xmsg_index)

  Set the index of this X-Message.
- void set_xmsg_size_bytes (uint16_t xmsg_size_bytes)

  Set the size of the X-Message, in bytes.
- vector< XSensorTripletCodeCounts > UnpackData (const vector< uint8_t > &data)

  Unpack the fall/rise data from the binary message.

Private Attributes

- XFallRiseDataPackFormat_t data_pack_format_
  The packing format type identifier.
- int16_t id_dive_
  The dive or cycle number of this message.
- uint16_t id_instrument_
  The unique WHOI identifier of this instrument.
- XMsgTypeId_t id_xmsg_type_
  The X-message type identifier.
- vector< XSensorTripletCodeCounts > sensor_data_
  The raw counts of measured fall, rise, or surface time-series data.
- ptime start_time_
  The start time reference.
- uint8_t xmsg_index_
  The index of this X-Message.
- uint16_t xmsg_size_bytes_
  The size of the X message, in bytes.

10.110.1 Detailed Description

Class representing a rise data X-Message.

10.110.2 Constructor & Destructor Documentation

10.110.2.1 XRise()

```
util::xmsg::XRise::XRise (
            int16_t dive,
            uint16_t whoi,
            const vector< uint8_t > & data)  [inline]
```

The parameterized constructor.

Parameters

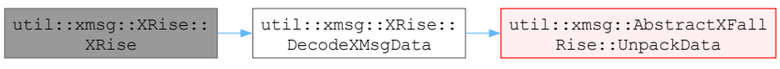
in	*dive*	The dive or cycle number of this message.
in	*whoi*	The unique WHOI identifier.
in	*data*	The binary X-message data to decode.

Exceptions

util::log::MessageException	if an decode error occurs.

References DecodeXMsgData().

Here is the call graph for this function:



10.110.2.2 ~XRise()

```
util::xmsg::XRise::~XRise ()  [inline]
```

The destructor.

10.110.3 Member Function Documentation

10.110.3.1 CheckXMsgData()

```
const uint8_t * util::xmsg::XMessage::CheckXMsgData (
            const vector< uint8_t > & data,
            int id,
            int minBytes,
            bool parseSize = true)  [inline], [inherited]
```

Check the X-message for the correct ID, minimum size, and message terminator.

Parameters

in	*data*	The X-message binary data to check.
in	*id*	The expected ID of the X-message. If this field is less that 0, then the ID check will be skipped.
in	*minBytes*	The minimum number of bytes expected in a valid message.
in	*parseSize*	If TRUE, then the message size field will be parsed from the X-Message. Otherwise, the size field will not be parsed.

Exceptions

util::log::MessageException	if • the message does not contain the expected ID, • the message does not contain the minimum number of expected bytes, • the message does not contain a terminator (';') in the correct location, or, • any other error occurs.

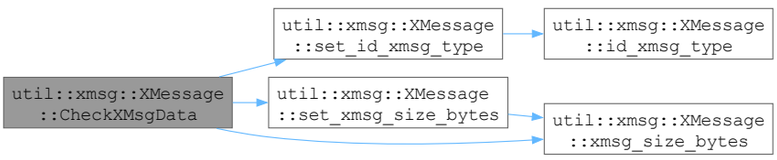
Returns

A pointer to the underlying data buffer of data.

References CATCH_BLOCK, EXC_THROW, lwrn, util::xmsg::XMessage::set_id_xmsg_type(), util::xmsg::XMessage::set_xmsg_size_byte
u16, and util::xmsg::XMessage::xmsg_size_bytes().

Referenced by util::xmsg::XAcousticPosition::DecodeXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XDriftPressure::Dec
util::xmsg::XDriftSalinity::DecodeXMsgData(), util::xmsg::XDriftTemperature::DecodeXMsgData(), util::xmsg::XEngAbort::DecodeXMsgDa
util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngBIT::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(),
util::xmsg::XEngDiagnostic::DecodeXMsgData(), util::xmsg::XEngDrift::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(),
util::xmsg::XEngStandard::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XGPS::DecodeXMsgData(),
util::xmsg::XHiResPressure::DecodeXMsgData(), util::xmsg::XHiResSalinity::DecodeXMsgData(), util::xmsg::XHiResTemperature::Decod
util::xmsg::XMissionCmdEcho::DecodeXMsgData(), util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XProfilePressure:
util::xmsg::XProfileSalinity::DecodeXMsgData(), util::xmsg::XProfileTemperature::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgD
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the call graph for this function:



10.110.3.2 data_pack_format()

```
XFallRiseDataPackFormat_t util::xmsg::AbstractXFallRise::data_pack_format () const  [inline],
[inherited]
```

Get the enumerated data packing format type.

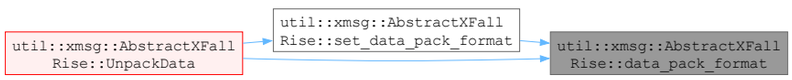
Returns

The enumerated data packing format type.

References util::xmsg::AbstractXFallRise::data_pack_format_.

Referenced by util::xmsg::AbstractXFallRise::set_data_pack_format(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the caller graph for this function:



10.110.3.3 DecodeXMsgData()

```
void util::xmsg::XRise::DecodeXMsgData (
            const vector< uint8_t > & data)  [inline], [virtual]
```

Decode the rise data X-Message from the binary message data.

Parameters

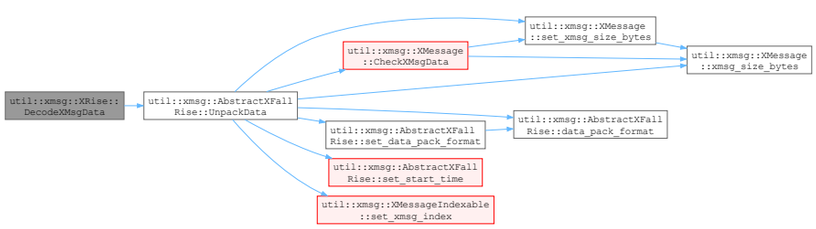
in	*data*	The binary message data to decode.

Implements util::xmsg::AbstractXFallRise.

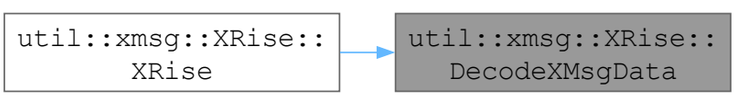
References CATCH_BLOCK, EXC_THROW, util::xmsg::AbstractXFallRise::UnpackData(), and util::xmsg::XMSG_TYPE_ID_RISE0.

Referenced by XRise().

Here is the call graph for this function:



Here is the caller graph for this function:



10.110.3.4 GenerateCurrentUtcTimeIso8601String()

```
static string util::date::DateUtils::GenerateCurrentUtcTimeIso8601String ()  [inline], [static],
[inherited]
```

Generate an ISO8601 format date/time string representation of the current time, in UTC.

Note

All date/time objects use the UTC time-zone.

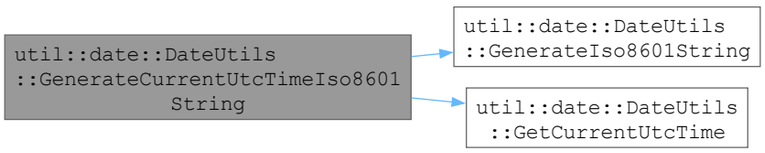
Returns

An ISO8601 format date/time string representation of the current time.

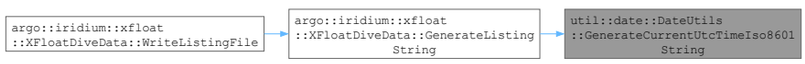
References util::date::DateUtils::GenerateIso8601String(), and util::date::DateUtils::GetCurrentUtcTime().

Referenced by argo::iridium::xfloat::XFloatDiveData::GenerateListingString().

Here is the call graph for this function:



Here is the caller graph for this function:



10.110.3.5 GenerateIso8601String()

```
static string util::date::DateUtils::GenerateIso8601String (
            const ptime & p,
            bool extended = true)  [inline], [static], [inherited]
```

Generate an ISO8601 format date/time string representation of a POSIX time object.

Note

All date/time objects use the UTC time-zone.

Parameters

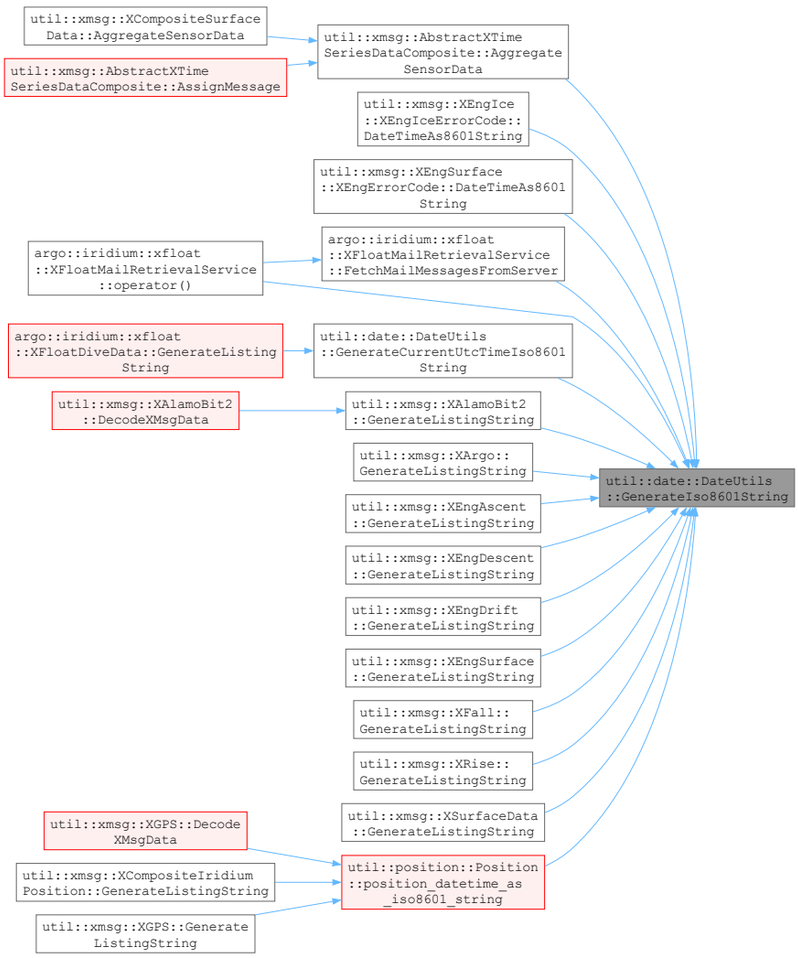
in	*p*	POSIX time from which to generate string.
in	*extended*	If TRUE, use the ISO 8601 extended format (YYYY-MM-DDThh:mm:ssZ); otherwise, use the basic format (YYYYMMDDThhmmss).

Returns

An ISO8601 format date/time string representation of the POSIX time object. If p is equal to boost::posix_↩ time::not_a_date_time, then the string "0000-00-00T00:00:00Z" (extended) or "00000000000000" (basic) will be returned.

Referenced by util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::AggregateSensorData(), util::xmsg::XEngIce::XEngIce
util::xmsg::XEngSurface::XEngErrorCode::DateTimeAs8601String(), argo::iridium::xfloat::XFloatMailRetrievalService::FetchMailMessages
util::date::DateUtils::GenerateCurrentUtcTimeIso8601String(), util::xmsg::XAlamoBit2::GenerateListingString(), util::xmsg::XArgo::Genera
util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListing
util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), GenerateListingString(),
util::xmsg::XSurfaceData::GenerateListingString(), argo::iridium::xfloat::XFloatMailRetrievalService::operator()(), and
util::position::Position::position_datetime_as_iso8601_string().

Here is the caller graph for this function:



10.110.3.6 GenerateJulianDay()

```
static double util::date::DateUtils::GenerateJulianDay (
            const ptime & dt,
            const ptime & ref_dt)  [inline], [static], [inherited]
```

Convert a date/time to fractional days since a reference date/time.

Parameters

in	*dt*	The date/time to convert.
in	*ref↩ _dt*	The reference date/time (day 0).

Exceptions

util::log::MessageException	if either date/time is not valid.

Returns

The number of days (with millisecond resolution) from `ref_dt` to `dt`.

References CATCH_BLOCK, and EXC_THROW.

10.110.3.7 GenerateListingComment()

```
static const string util::xmsg::AbstractXFallRise::GenerateListingComment ()  [inline], [static],
[inherited]
```

Get the listing entry comment string.

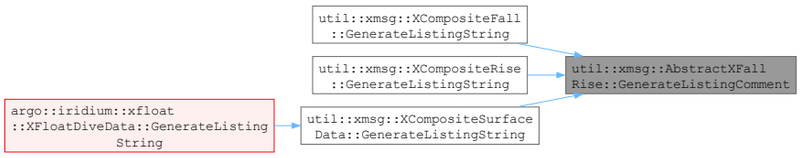
Returns

The listing entry comment string.

References CATCH_BLOCK.

Referenced by util::xmsg::XCompositeFall::GenerateListingString(), util::xmsg::XCompositeRise::GenerateListingString(), and util::xmsg::XCompositeSurfaceData::GenerateListingString().

Here is the caller graph for this function:



10.110.3.8 GenerateListingString()

```
const string util::xmsg::XRise::GenerateListingString (
            uint16_t pg,
            uint16_t po,
            uint16_t sg,
            uint16_t so,
            uint16_t tg,
            uint16_t to,
            float pc) const  [inline]
```

Get the listing entry string for this rise data X-Message.

Parameters

in	*pg*	The pressure sensor sensitivity scale factor, in LSB/dBar.
in	*po*	The pressure sensor offset, in dBar.
in	*sg*	The salinity sensor sensitivity scale factor, in LSB/PSU.
in	*so*	The salinity sensor offset, in PSU.
in	*tg*	The temperature sensor sensitivity scale factor, in LSB/degC.
in	*to*	The temperature sensor offset, in degC.
in	*pc*	The pressure correction, in dBar, to be subtracted computed pressure value.

Exceptions

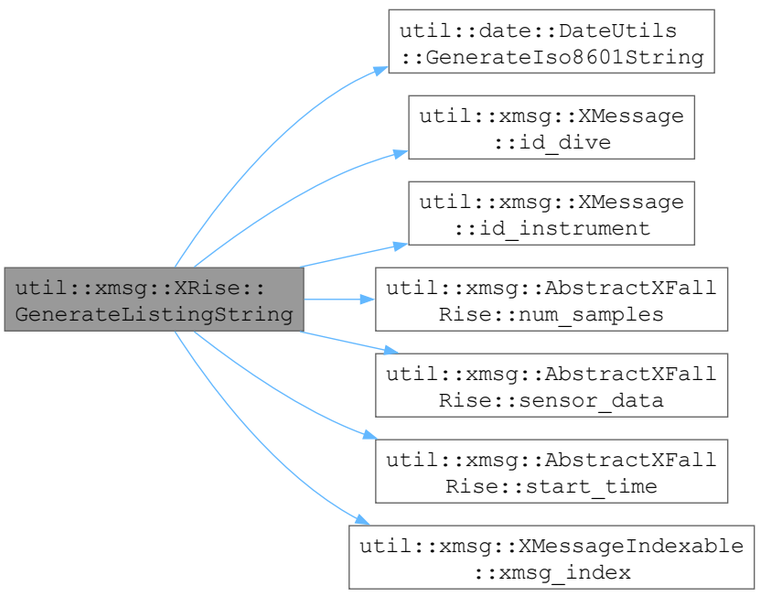
log::MessageException	if an error occurs while generating the listing string.

Returns

The listing entry string for this rise data X-Message.

References CATCH_BLOCK, util::date::DateUtils::GenerateIso8601String(), util::xmsg::XMessage::id_dive(), util::xmsg::XMessage::id_ins
util::xmsg::AbstractXFallRise::num_samples(), util::xmsg::AbstractXFallRise::sensor_data(), util::xmsg::AbstractXFallRise::start_time(),
and util::xmsg::XMessageIndexable::xmsg_index().

Here is the call graph for this function:



10.110.3.9 GetCurrentUtcTime()

```
static ptime util::date::DateUtils::GetCurrentUtcTime ()  [inline], [static], [inherited]
```

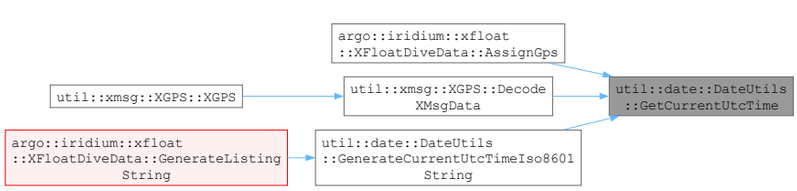
Generate a POSIX time object representing the current date and time (rounded to the nearest second).

Returns

A POSIX time object representing the current date and time (rounded to the nearest second).

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignGps(), util::xmsg::XGPS::DecodeXMsgData(), and util::date::DateUtils::GenerateCurrentUtcTimeIso8601String().

Here is the caller graph for this function:



10.110.3.10 id_dive()

```
int16_t util::xmsg::XMessage::id_dive () const  [inline], [inherited]
```

Get the dive or cycle number of this X-Message.

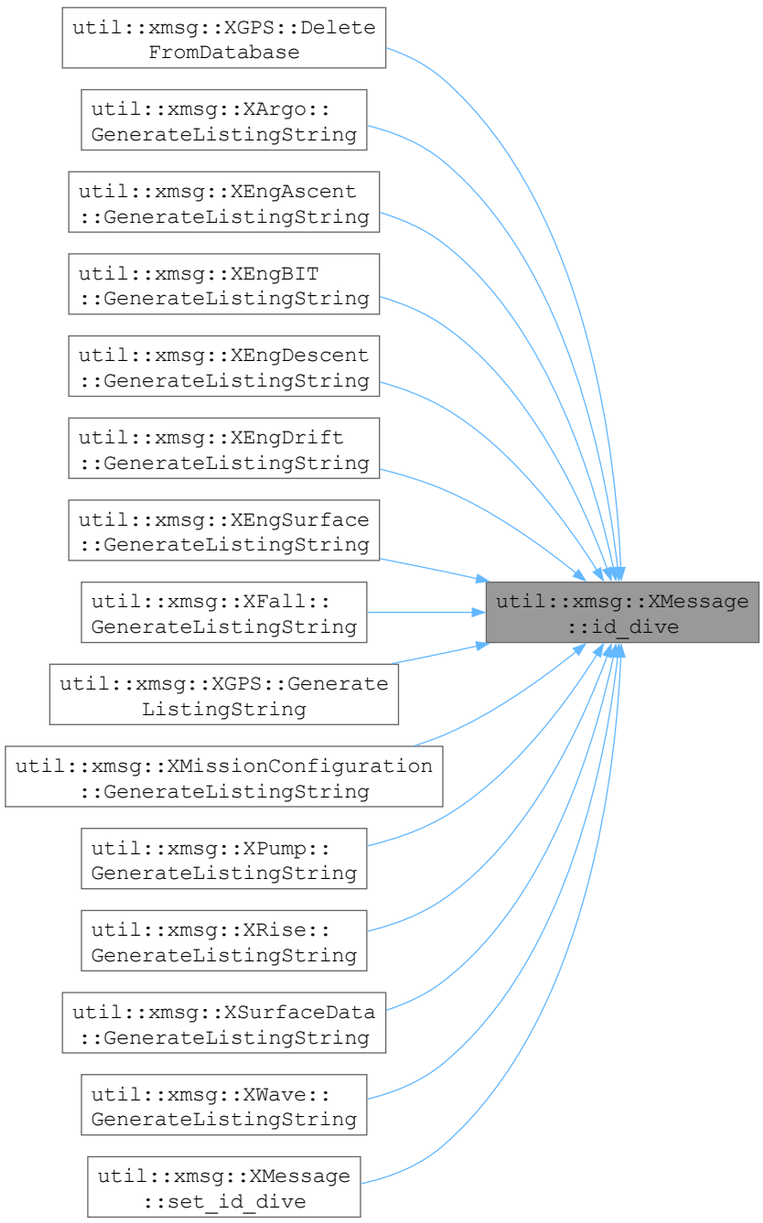
Returns

The dive or cycle number of this X-Message.

References util::xmsg::XMessage::id_dive_.

Referenced by util::xmsg::XGPS::DeleteFromDatabase(), util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateLis
util::xmsg::XEngBIT::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingStr
util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(),
util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), GenerateListingString(),
util::xmsg::XSurfaceData::GenerateListingString(), util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_dive().

Here is the caller graph for this function:



10.110.3.11 id_instrument()

```
uint16_t util::xmsg::XMessage::id_instrument () const  [inline], [inherited]
```

Get the unique instrument identifier of this X-Message.

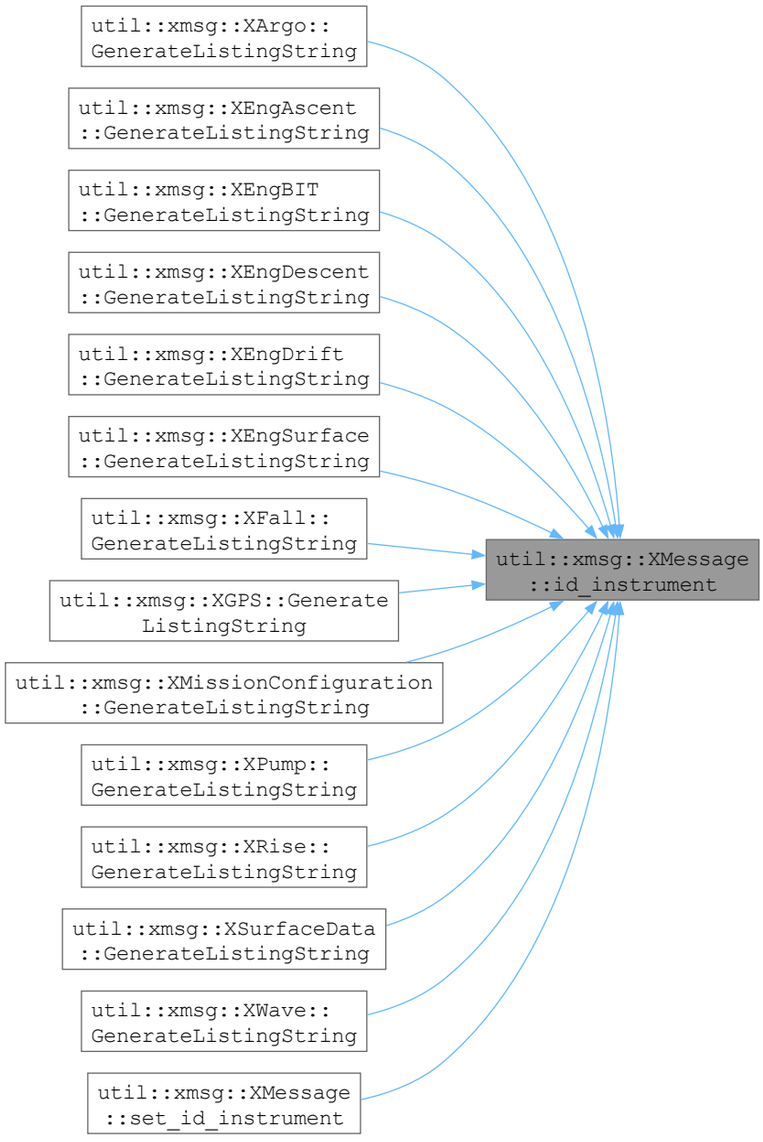
Returns

The unique instrument identifier of this X-Message.

References util::xmsg::XMessage::id_instrument_.

Referenced by util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngBIT::Generate
util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), util::xmsg::XEngSurface::GenerateListin
util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingS
util::xmsg::XPump::GenerateListingString(), GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(),
util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_instrument().

Here is the caller graph for this function:



10.110.3.12 id_xmsg_type()

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type () const  [inline], [inherited]
```

Get the enumerated X-Message type identifier.

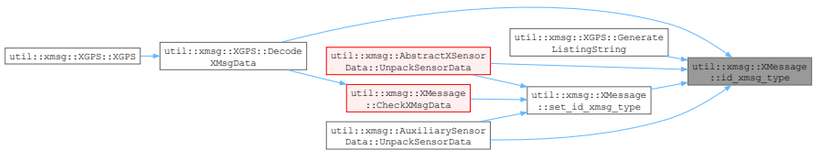
Returns

The enumerated X-Message type identifier.

References util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMessage::set_id_xmsg_typ
util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the caller graph for this function:



10.110.3.13 num_samples()

```
size_t util::xmsg::AbstractXFallRise::num_samples () const  [inline], [inherited]
```

Get the number of samples in this message.

Returns

The number of samples in this message.

References util::xmsg::AbstractXFallRise::sensor_data_.

Referenced by util::xmsg::XFall::GenerateListingString(), GenerateListingString(), and util::xmsg::XSurfaceData::GenerateListingString().

Here is the caller graph for this function:

10.110.3.14 sensor_data()

```
const vector< XSensorTripletCodeCounts > & util::xmsg::AbstractXFallRise::sensor_data () const
[inline], [inherited]
```

Get the vector of sensor data triplets.

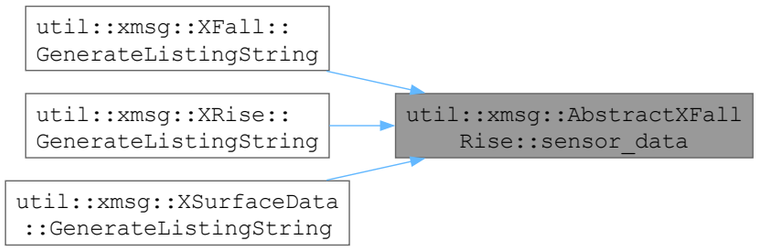
Returns

The vector of sensor data triplets.

References util::xmsg::AbstractXFallRise::sensor_data_.

Referenced by util::xmsg::XFall::GenerateListingString(), GenerateListingString(), and util::xmsg::XSurfaceData::GenerateListingString().

Here is the caller graph for this function:



10.110.3.15 set_data_pack_format()

```
void util::xmsg::AbstractXFallRise::set_data_pack_format (
            XFallRiseDataPackFormat_t data_pack_format)  [inline], [protected], [inherited]
```

Set the enumerated data packing format value for this message.

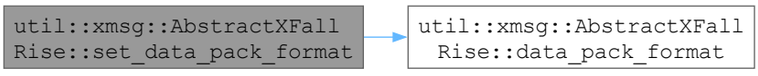
Parameters

in	*data_pack_format*	The enumerated data packing format value for this message to set.

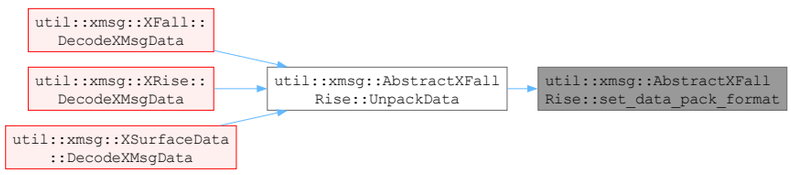
References util::xmsg::AbstractXFallRise::data_pack_format(), and util::xmsg::AbstractXFallRise::data_pack_format_.

Referenced by util::xmsg::AbstractXFallRise::UnpackData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.110.3.16 set_id_dive()

```
void util::xmsg::XMessage::set_id_dive (
            int16_t id_dive)  [inline], [protected], [inherited]
```

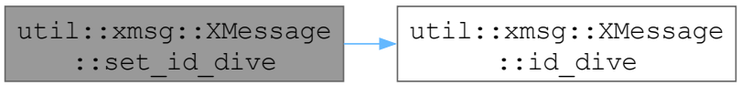
Set the dive or cycle number of this message.

Parameters

in	*id_dive*	The dive or cycle number of this message to set.

References util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::id_dive_.

Here is the call graph for this function:



10.110.3.17 set_id_instrument()

```
void util::xmsg::XMessage::set_id_instrument (
            uint16_t id_instrument)  [inline], [protected], [inherited]
```

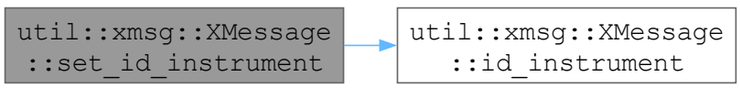
Set the unique instrument identifier.

Parameters

in	*id_instrument*	The unique instrument identifier to set.

References util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::id_instrument_.

Here is the call graph for this function:



10.110.3.18 set_id_xmsg_type()

```
void util::xmsg::XMessage::set_id_xmsg_type (
            XMsgTypeId_t id_xmsg_type)  [inline], [protected], [inherited]
```

Set the X-Message type identifier.

Parameters

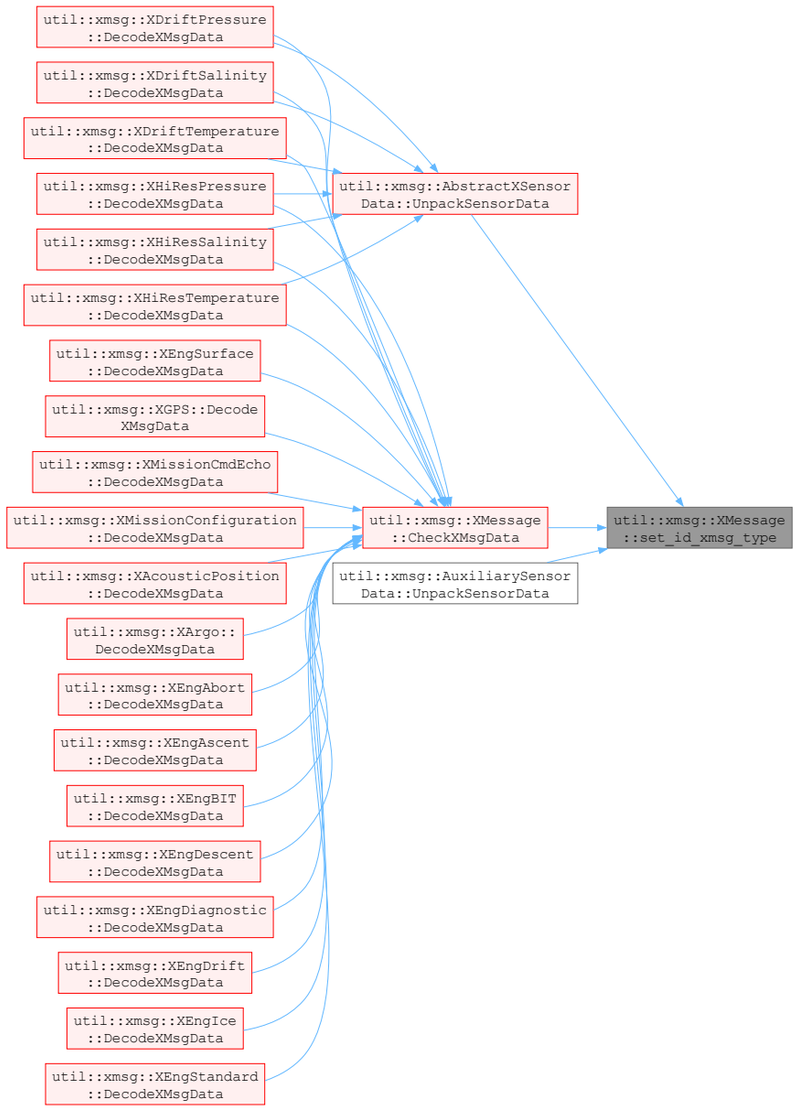
in	*id_xmsg_type*	The X-Message type identifier to set.

References util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:

Here is the caller graph for this function:



10.110.3.19 set_start_time()

```
void util::xmsg::AbstractXFallRise::set_start_time (
            const ptime & start_time)  [inline], [protected], [inherited]
```

Set the start time of this message.

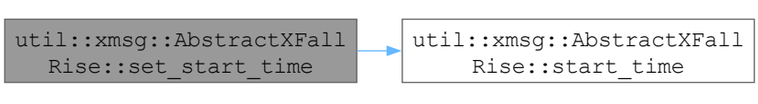
Parameters

in	*start_time*	The start time to set.

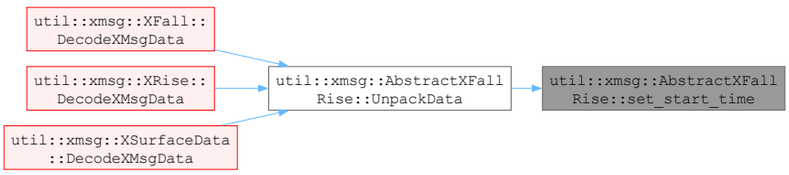
References util::xmsg::AbstractXFallRise::start_time(), and util::xmsg::AbstractXFallRise::start_time_.

Referenced by util::xmsg::AbstractXFallRise::UnpackData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.110.3.20 set_xmsg_index()

```
void util::xmsg::XMessageIndexable::set_xmsg_index (
            uint8_t xmsg_index)  [inline], [protected], [inherited]
```

Set the index of this X-Message.

Parameters

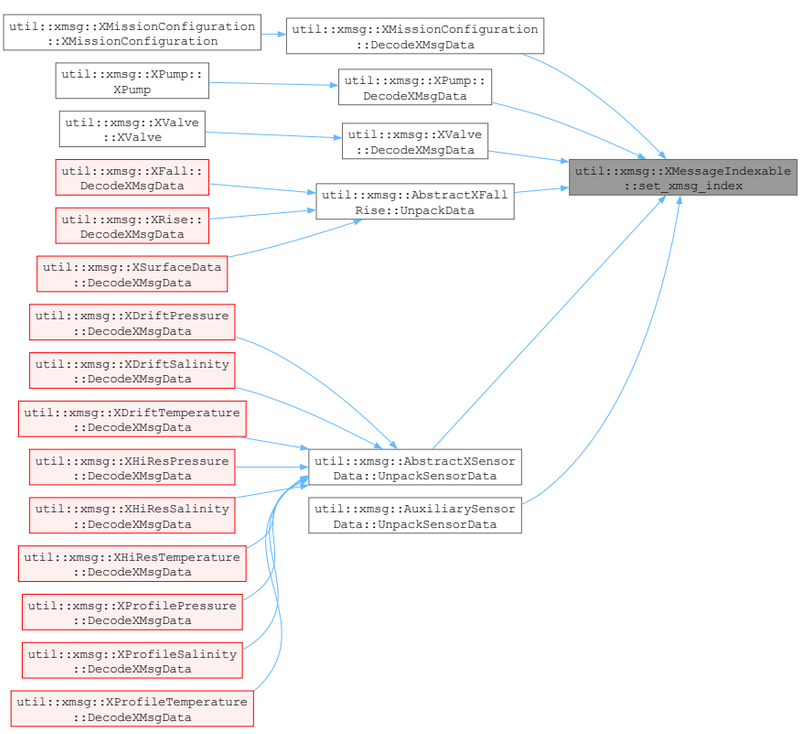
in	*xmsg_index*	The index of this X-Message.

References util::xmsg::XMessageIndexable::xmsg_index(), and util::xmsg::XMessageIndexable::xmsg_index_.

Referenced by util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(), util::xmsg::XValve::DecodeXMsgData(), util::xmsg::AbstractXFallRise::UnpackData(), util::xmsg::AbstractXSensorData::UnpackSensorDa and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:

Here is the caller graph for this function:



10.110.3.21 set_xmsg_size_bytes()

```
void util::xmsg::XMessage::set_xmsg_size_bytes (
            uint16_t xmsg_size_bytes)  [inline], [protected], [inherited]
```

Set the size of the X-Message, in bytes.

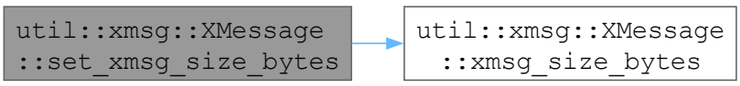
Parameters

in	*xmsg_size_bytes*	The size of the X-Message to set, in bytes.

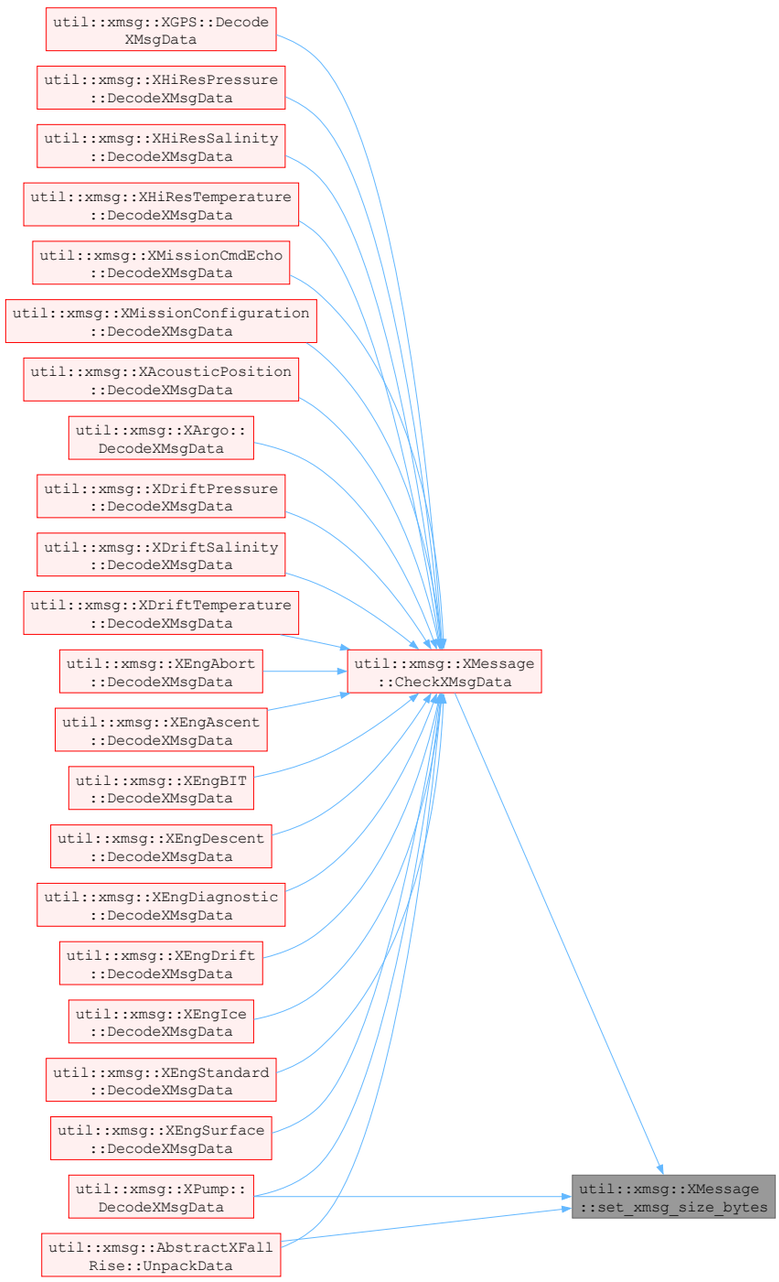
References util::xmsg::XMessage::xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XPump::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::Unpa

Here is the call graph for this function:



Here is the caller graph for this function:



10.110.3.22 start_time()

```
const ptime & util::xmsg::AbstractXFallRise::start_time () const  [inline], [inherited]
```

Get the start time of this message.

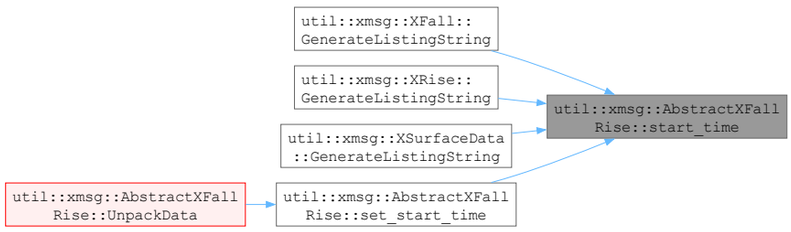
Returns

The start time of this message.

References util::xmsg::AbstractXFallRise::start_time_.

Referenced by util::xmsg::XFall::GenerateListingString(), GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(), and util::xmsg::AbstractXFallRise::set_start_time().

Here is the caller graph for this function:



10.110.3.23 UnpackData()

```
vector< XSensorTripletCodeCounts > util::xmsg::AbstractXFallRise::UnpackData (
            const vector< uint8_t > & data)  [inline], [protected], [inherited]
```

Unpack the fall/rise data from the binary message.

Parameters

in	*data*	The binary message to unpack.

Exceptions

util::log::MessageException	if • the message is of insufficient length, • an invalid packing format is encountered, or • the message terminator is in an invalid location.

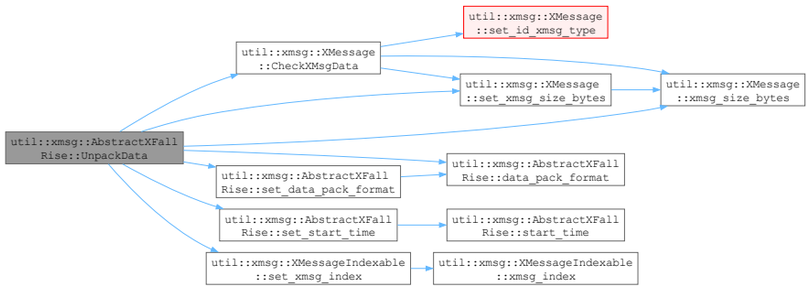
Returns

The raw counts of measured fall or rise time-series data.

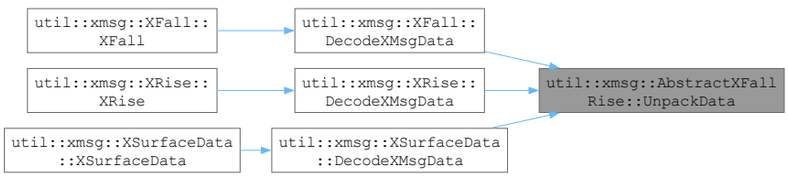
References CATCH_BLOCK, util::xmsg::XMessage::CheckXMsgData(), util::xmsg::AbstractXFallRise::data_pack_format(), EXC_THROW, lwrn, s2000, util::xmsg::AbstractXFallRise::sensor_data_, util::xmsg::AbstractXFallRise::set_data_pack_format(), util::xmsg::AbstractXFallRise::set_start_time(), util::xmsg::XMessageIndexable::set_xmsg_index(), util::xmsg::XMessage::set_xmsg_size_ XMSG_SENS_BAD, util::xmsg::XMessage::xmsg_size_bytes(), util::xmsg::XPHASE_CODE_UNKNOWN, util::xmsg::XRISEFALL_FMT_ util::xmsg::XRISEFALL_FMT_STNDRD, util::xmsg::XRISEFALL_FMT_STNDRD_WITH_TEMP_AND_SALN, and util::xmsg::XRISEFALL_FMT_UNKNOWN.

Referenced by util::xmsg::XFall::DecodeXMsgData(), DecodeXMsgData(), and util::xmsg::XSurfaceData::DecodeXMsgData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.110.3.24 xmsg_index()

```
uint8_t util::xmsg::XMessageIndexable::xmsg_index () const  [inline], [inherited]
```

Get the index of this X-Message.

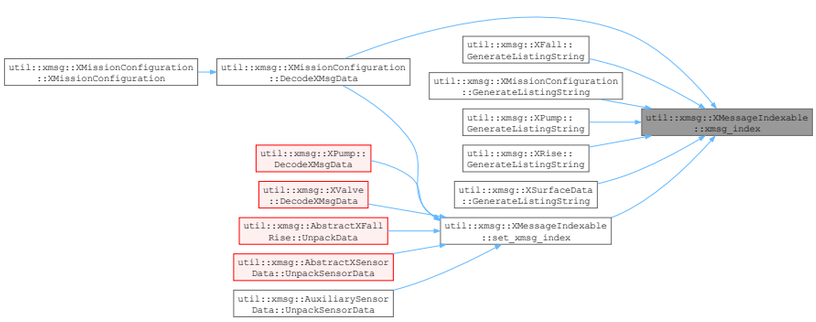
Returns

The index of this X-Message.

References util::xmsg::XMessageIndexable::xmsg_index_.

Referenced by util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(), and util::xmsg::XMessageIndexable::set_xmsg_index().

Here is the caller graph for this function:



10.110.3.25 xmsg_size_bytes()

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes () const  [inline], [inherited]
```

Get the size of the X-Message, in bytes.

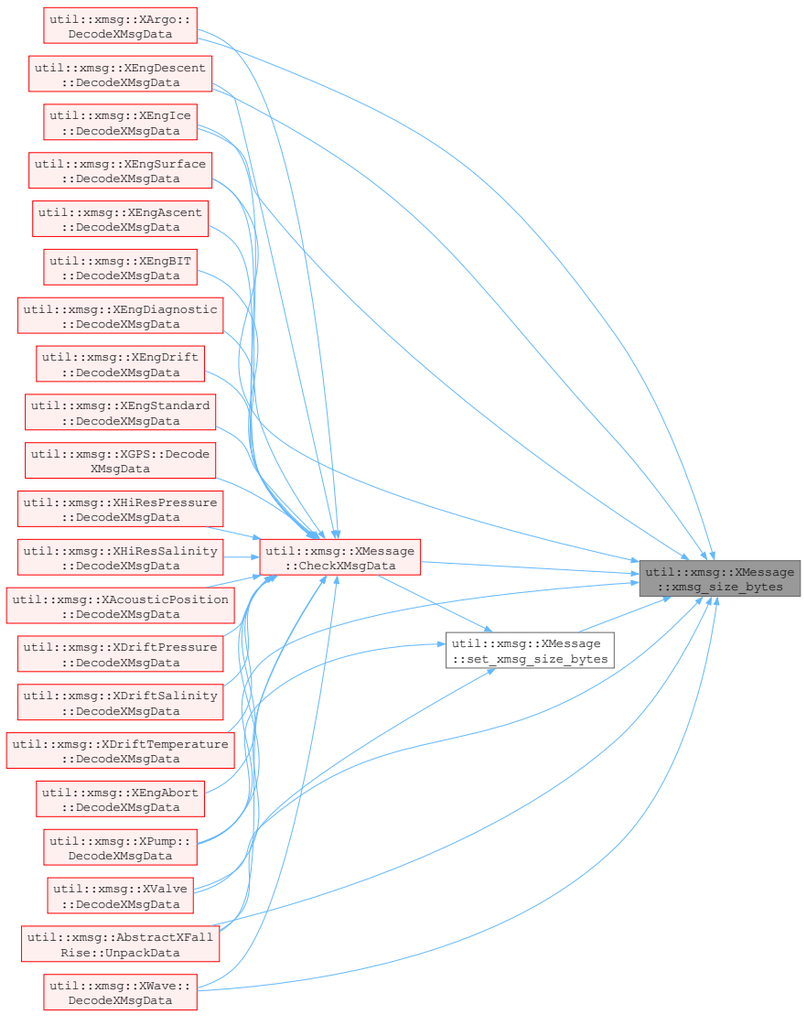
Returns

The size of the X-Message, in bytes.

References util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgI
util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(),
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), util::xmsg::XMessage::set_xmsg_size_bytes(),
and util::xmsg::AbstractXFallRise::UnpackData().

Here is the caller graph for this function:



10.110.4 Member Data Documentation

10.110.4.1 data_pack_format_

```
XFallRiseDataPackFormat_t util::xmsg::AbstractXFallRise::data_pack_format_  [private], [inherited]
```

The packing format type identifier.

Referenced by util::xmsg::AbstractXFallRise::data_pack_format(), and util::xmsg::AbstractXFallRise::set_data_pack_format().

10.110.4.2 id_dive_

`int16_t util::xmsg::XMessage::id_dive_ [private], [inherited]`

The dive or cycle number of this message.

Referenced by util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::set_id_dive().

10.110.4.3 id_instrument_

`uint16_t util::xmsg::XMessage::id_instrument_ [private], [inherited]`

The unique WHOI identifier of this instrument.

Referenced by util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::set_id_instrument().

10.110.4.4 id_xmsg_type_

`XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type_ [private], [inherited]`

The X-message type identifier.

Referenced by util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::set_id_xmsg_type().

10.110.4.5 sensor_data_

`vector<XSensorTripletCodeCounts> util::xmsg::AbstractXFallRise::sensor_data_ [private], [inherited]`

The raw counts of measured fall, rise, or surface time-series data.

Referenced by util::xmsg::AbstractXFallRise::num_samples(), util::xmsg::AbstractXFallRise::sensor_data(), and util::xmsg::AbstractXFallRise::UnpackData().

10.110.4.6 start_time_

`ptime util::xmsg::AbstractXFallRise::start_time_ [private], [inherited]`

The start time reference.

Referenced by util::xmsg::AbstractXFallRise::set_start_time(), and util::xmsg::AbstractXFallRise::start_time().

10.110.4.7 xmsg_index_

`uint8_t util::xmsg::XMessageIndexable::xmsg_index_ [private], [inherited]`

The index of this X-Message.

Referenced by util::xmsg::XMessageIndexable::set_xmsg_index(), and util::xmsg::XMessageIndexable::xmsg_index().

10.110.4.8 xmsg_size_bytes_

`uint16_t util::xmsg::XMessage::xmsg_size_bytes_ [private], [inherited]`

The size of the X message, in bytes.

Referenced by util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes().

The documentation for this class was generated from the following file:

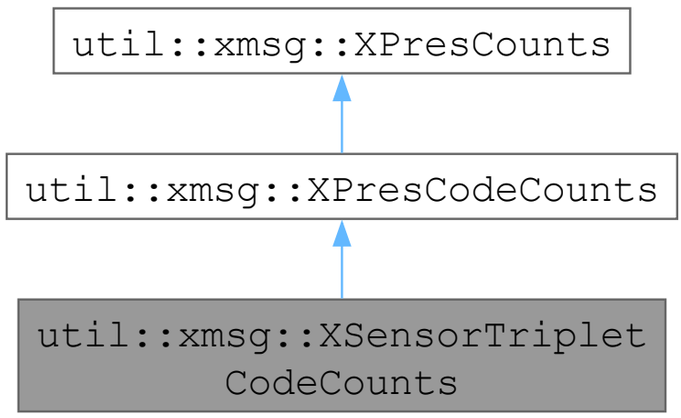
- data/xmsg/XRise.h

10.111 util::xmsg::XSensorTripletCodeCounts Class Reference

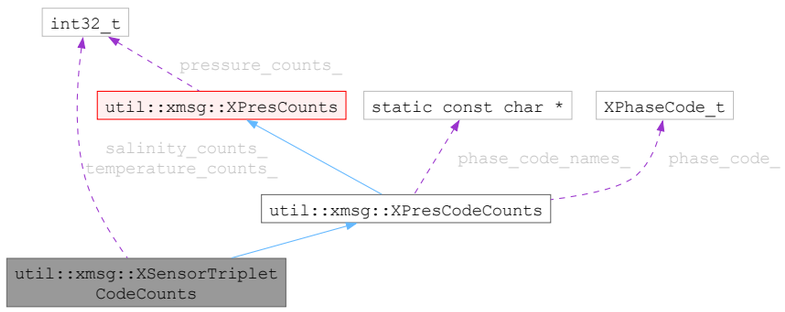
The sensor triplet and phase code class.

`#include <XSensorTripletCodeCounts.h>`

Inheritance diagram for util::xmsg::XSensorTripletCodeCounts:



Collaboration diagram for util::xmsg::XSensorTripletCodeCounts:



Public Member Functions

- XSensorTripletCodeCounts (uint16_t sec, XPhaseCode_t code, int32_t pres_counts, int32_t temp_↩ counts=XMSG_SENS_BAD, int32_t saln_counts=XMSG_SENS_BAD)

  The constructor.
- ~XSensorTripletCodeCounts ()

  The destructor.
- uint16_t elapsed_seconds_since_reference_time () const

  Get the elapsed time since the reference time, in seconds.
- const string GenerateListingString (uint16_t pg, uint16_t po, float pc) const

  Generate a listing entry string from this pressure measurement object.
- const string GenerateListingString (uint16_t pg, uint16_t po, uint16_t sg, uint16_t so, uint16_t tg, uint16_t to, float pc) const

  Generate a listing entry string from this sensor triplet object.
- XPhaseCode_t phase_code () const

  Get the phase code.
- const char * phase_code_name () const

  Get the phase code name C-sting.
- int32_t **pressure_counts** () const

- template<typename T = float>
  T PressureValue (uint16_t pg, uint16_t po, float pres_corr) const
  Get the measured pressure value, in dBar.
- int32_t **salinity_counts** () const
- template<typename T = double>
  T SalinityValue (uint16_t sg, uint16_t so) const
  Get the measured salinity value, in PSU.
- int32_t **temperature_counts** () const
- template<typename T = double>
  T TemperatureValue (uint16_t tg, uint16_t to) const
  Get the measured temperature value, in degC.

Static Public Member Functions

- static const char * GetPhaseCodeName (XPhaseCode_t code)
  Get the phase code name C-string for a given phase code value.

Private Attributes

- uint16_t elapsed_seconds_since_reference_time_
  The elapsed time since the reference time, in seconds.
- XPhaseCode_t phase_code_
  The phase code of this pressure measurement.
- int32_t pressure_counts_
  The pressure measurement, in raw sensor counts.
- int32_t salinity_counts_
  The salinity measurement, in raw sensor counts.
- int32_t temperature_counts_
  The temperature measurement, in raw sensor counts.

Static Private Attributes

- static const char * phase_code_names_ [ ]
  The phase code names.

10.111.1 Detailed Description

The sensor triplet and phase code class.

This class represents a pressure, temperature, and salinity measurement triplet and a phase code. The phase code is used to indicate the context of the measurement (see XPhaseCode_t). This class is used to store a single data point for the rise, fall, or surface time-series data.

10.111.2 Constructor & Destructor Documentation

10.111.2.1 XSensorTripletCodeCounts()

```
util::xmsg::XSensorTripletCodeCounts::XSensorTripletCodeCounts (
            uint16_t sec,
            XPhaseCode_t code,
            int32_t pres_counts,
            int32_t temp_counts = XMSG_SENS_BAD,
            int32_t saln_counts = XMSG_SENS_BAD)  [inline]
```

The constructor.

Parameters

in	*sec*	The elapsed time since the reference time, in seconds.
in	*code*	The phase code of this pressure measurement.
in	*pres_counts*	The pressure measurement, in raw sensor counts.
in	*temp_counts*	The temperature measurement, in raw sensor counts.
in	*saln_counts*	The salinity measurement, in raw sensor counts.

10.111.2.2 ~XSensorTripletCodeCounts()

```
util::xmsg::XSensorTripletCodeCounts::~XSensorTripletCodeCounts ()  [inline]
```

The destructor.

10.111.3 Member Function Documentation

10.111.3.1 elapsed_seconds_since_reference_time()

```
uint16_t util::xmsg::XPresCounts::elapsed_seconds_since_reference_time () const  [inline], [inherited]
```

Get the elapsed time since the reference time, in seconds.

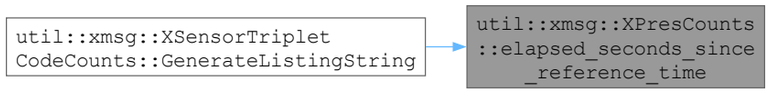
Returns

The elapsed time since the reference time, in seconds.

References util::xmsg::XPresCounts::elapsed_seconds_since_reference_time_.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.111.3.2 GenerateListingString() [1/2]

```
const string util::xmsg::XPresCodeCounts::GenerateListingString (
            uint16_t pg,
            uint16_t po,
            float pc) const  [inline], [inherited]
```

Generate a listing entry string from this pressure measurement object.

Parameters

in	*pg*	The pressure sensor sensitivity scale factor, in LSB/dBar.
in	*po*	The pressure sensor offset, in dBar.
in	*pc*	The pressure correction, in dBar, to be subtracted computed pressure value.

Exceptions

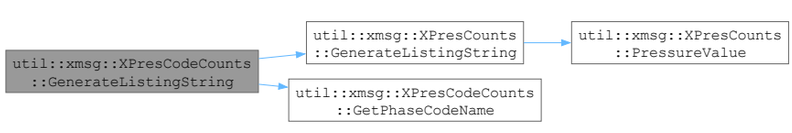
log::MessageException	if an error occurs while generating the listing string.

Returns

A listing entry string representation of this pressure measurement object.

References CATCH_BLOCK, util::xmsg::XPresCounts::GenerateListingString(), util::xmsg::XPresCodeCounts::GetPhaseCodeName(), and util::xmsg::XPresCodeCounts::phase_code_.

Here is the call graph for this function:



10.111.3.3 GenerateListingString() [2/2]

```
const string util::xmsg::XSensorTripletCodeCounts::GenerateListingString (
            uint16_t pg,
            uint16_t po,
            uint16_t sg,
            uint16_t so,
            uint16_t tg,
            uint16_t to,
            float pc) const  [inline]
```

Generate a listing entry string from this sensor triplet object.

Parameters

in	*pg*	The pressure sensor sensitivity scale factor, in LSB/dBar.
in	*po*	The pressure sensor offset, in dBar.
in	*sg*	The salinity sensor sensitivity scale factor, in LSB/PSU.
in	*so*	The salinity sensor offset, in PSU.
in	*tg*	The temperature sensor sensitivity scale factor, in LSB/degC.
in	*to*	The temperature sensor offset, in degC.
in	*pc*	The pressure correction, in dBar, to be subtracted computed pressure value.

Exceptions

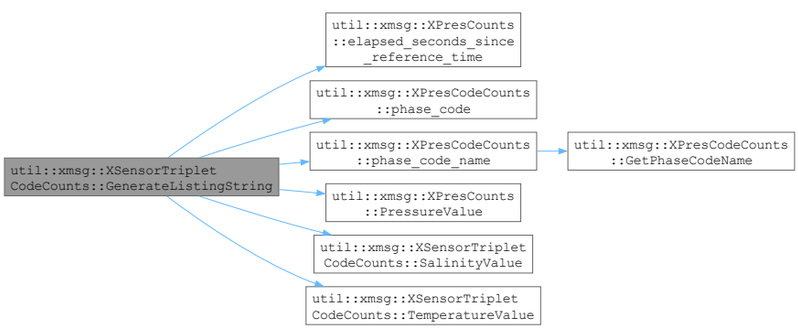
log::MessageException	if an error occurs while generating the listing string.

Returns

A listing entry string representation of this sensor triplet object.

References CATCH_BLOCK, util::xmsg::XPresCounts::elapsed_seconds_since_reference_time(), util::xmsg::XPresCodeCounts::phase_c
util::xmsg::XPresCodeCounts::phase_code_name(), util::xmsg::XPresCounts::PressureValue(), SalinityValue(), and
TemperatureValue().

Here is the call graph for this function:



10.111.3.4 GetPhaseCodeName()

```
static const char * util::xmsg::XPresCodeCounts::GetPhaseCodeName (
            XPhaseCode_t code)  [inline], [static], [inherited]
```

Get the phase code name C-string for a given phase code value.

Parameters

in	*code*	The phase code.

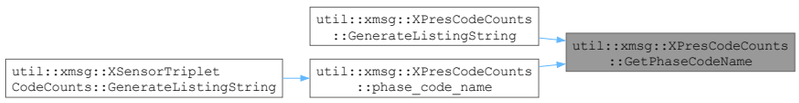
Returns

The phase code name C-string for the given phase code value.

References CATCH_BLOCK, EXC_THROW, and util::xmsg::XPresCodeCounts::phase_code_names_.

Referenced by util::xmsg::XPresCodeCounts::GenerateListingString(), and util::xmsg::XPresCodeCounts::phase_code_name().

Here is the caller graph for this function:



10.111.3.5 phase_code()

```
XPhaseCode_t util::xmsg::XPresCodeCounts::phase_code () const  [inline], [inherited]
```

Get the phase code.

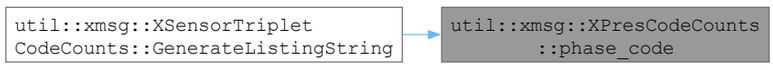
Returns

The phase code.

References util::xmsg::XPresCodeCounts::phase_code_.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.111.3.6 phase_code_name()

```
const char * util::xmsg::XPresCodeCounts::phase_code_name () const  [inline], [inherited]
```

Get the phase code name C-sting.

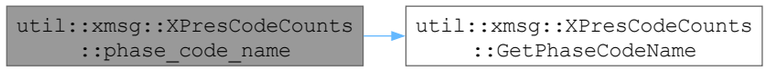
Returns

The phase code name C-sting.

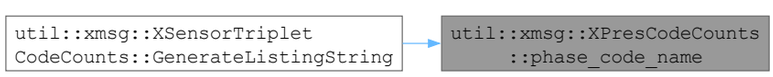
References util::xmsg::XPresCodeCounts::GetPhaseCodeName(), and util::xmsg::XPresCodeCounts::phase_code_.

Referenced by GenerateListingString().

Here is the call graph for this function:



Here is the caller graph for this function:



10.111.3.7 PressureValue()

```
template<typename T = float>
T util::xmsg::XPresCounts::PressureValue (
            uint16_t pg,
            uint16_t po,
            float pres_corr) const  [inline], [inherited]
```

Get the measured pressure value, in dBar.

Parameters

in	*pg*	The pressure sensor sensitivity scale factor, in LSB/dBar.
in	*po*	The pressure sensor offset, in dBar.
in	*pres_corr*	The pressure correction, in dBar, to be subtracted computed pressure value.

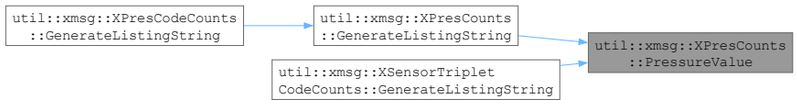
Returns

The measured pressure value, in dBar.

References util::xmsg::XPresCounts::pressure_counts_, XMSG_SENS_BAD_PRES, and XMSG_SENS_MAX_PRES.

Referenced by util::xmsg::XPresCounts::GenerateListingString(), and GenerateListingString().

Here is the caller graph for this function:



10.111.3.8 SalinityValue()

```
template<typename T = double>
T util::xmsg::XSensorTripletCodeCounts::SalinityValue (
            uint16_t sg,
            uint16_t so) const  [inline]
```

Get the measured salinity value, in PSU.

Parameters

in	*sg*	The salinity sensor sensitivity scale factor, in LSB/PSU.
in	*so*	The salinity sensor offset, in PSU.

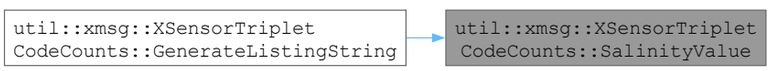
Returns

The measured salinity value, in PSU.

References salinity_counts_, XMSG_SENS_BAD_SALN, and XMSG_SENS_MAX_SALN.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.111.3.9 TemperatureValue()

```
template<typename T = double>
T util::xmsg::XSensorTripletCodeCounts::TemperatureValue (
            uint16_t tg,
            uint16_t to) const  [inline]
```

Get the measured temperature value, in degC.

Parameters

in	*tg*	The temperature sensor sensitivity scale factor, in LSB/degC.
in	*to*	The temperature sensor offset, in degC.

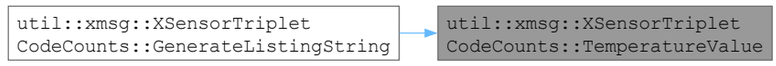
Returns

The measured temperature value, in degC.

References temperature_counts_, XMSG_SENS_BAD_TEMP, and XMSG_SENS_MAX_TEMP.

Referenced by GenerateListingString().

Here is the caller graph for this function:



10.111.4 Member Data Documentation

10.111.4.1 elapsed_seconds_since_reference_time_

```
uint16_t util::xmsg::XPresCounts::elapsed_seconds_since_reference_time_  [private], [inherited]
```

The elapsed time since the reference time, in seconds.

Referenced by util::xmsg::XPresCounts::elapsed_seconds_since_reference_time(), and util::xmsg::XPresCounts::GenerateListingString()

10.111.4.2 phase_code_

```
XPhaseCode_t util::xmsg::XPresCodeCounts::phase_code_  [private], [inherited]
```

The phase code of this pressure measurement.

Referenced by util::xmsg::XPresCodeCounts::GenerateListingString(), util::xmsg::XPresCodeCounts::phase_code(), and util::xmsg::XPresCodeCounts::phase_code_name().

10.111.4.3 phase_code_names_

```
const char * util::xmsg::XPresCodeCounts::phase_code_names_  [static], [private], [inherited]
```

Initial value:

```
= {
        "unknown",
        "start of descent",
        "first descent pump",
        "seek to drift pres",
        "start of drift",
        "seek during drift",
        "start of descent to profile",
        "start of ascent",
        "end of ascent",
        "ice turn-around",
        "descending",
        "drift/park",
        "descending to profile",
        "ascending",
        "surface",
        "invalid phase code"
        }
```

The phase code names.

Referenced by util::xmsg::XPresCodeCounts::GetPhaseCodeName().

10.111.4.4 pressure_counts_

```
int32_t util::xmsg::XPresCounts::pressure_counts_  [private], [inherited]
```

The pressure measurement, in raw sensor counts.

Referenced by util::xmsg::XPresCounts::PressureValue().

10.111.4.5 salinity_counts_

```
int32_t util::xmsg::XSensorTripletCodeCounts::salinity_counts_  [private]
```

The salinity measurement, in raw sensor counts.

Referenced by SalinityValue().

10.111.4.6 temperature_counts_

```
int32_t util::xmsg::XSensorTripletCodeCounts::temperature_counts_  [private]
```

The temperature measurement, in raw sensor counts.

Referenced by TemperatureValue().

The documentation for this class was generated from the following file:

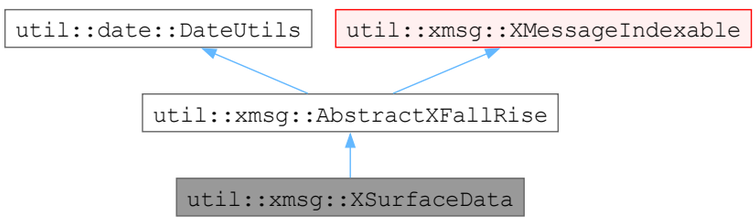
- data/xmsg/common/XSensorTripletCodeCounts.h

10.112 util::xmsg::XSurfaceData Class Reference

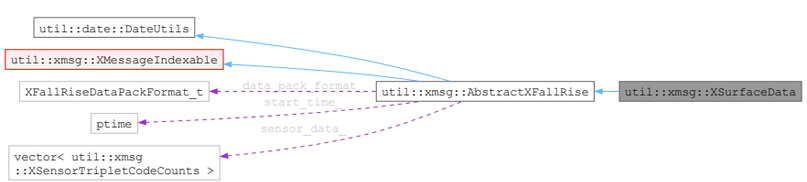
The surface time-series data X-Message class.

```
#include <XSurfaceData.h>
```

Inheritance diagram for util::xmsg::XSurfaceData:



Collaboration diagram for util::xmsg::XSurfaceData:



Public Member Functions

- XSurfaceData (int16_t dive, uint16_t whoi, const vector< uint8_t > &data)

  Parameterized constructor.
- ~XSurfaceData ()

  Destructor.
- const uint8_t * CheckXMsgData (const vector< uint8_t > &data, int id, int minBytes, bool parseSize=true)

  Check the X-message for the correct ID, minimum size, and message terminator.
- XFallRiseDataPackFormat_t data_pack_format () const

  Get the enumerated data packing format type.
- void DecodeXMsgData (const vector< uint8_t > &data)

  Decode the surface pressure X-message.
- const string GenerateListingString (uint16_t pg, uint16_t po, uint16_t sg, uint16_t so, uint16_t tg, uint16_t to, float pc) const

  Generate the listing entry string for this surface time-series data X-Message.
- int16_t id_dive () const

  Get the dive or cycle number of this X-Message.
- uint16_t id_instrument () const

  Get the unique instrument identifier of this X-Message.
- XMsgTypeId_t id_xmsg_type () const

  Get the enumerated X-Message type identifier.
- size_t num_samples () const

  Get the number of samples in this message.
- const vector< XSensorTripletCodeCounts > & sensor_data () const

  Get the vector of sensor data triplets.
- const ptime & start_time () const

  Get the start time of this message.
- uint8_t xmsg_index () const

  Get the index of this X-Message.
- uint16_t xmsg_size_bytes () const

  Get the size of the X-Message, in bytes.

Static Public Member Functions

- static string GenerateCurrentUtcTimeIso8601String ()

  Generate an ISO8601 format date/time string representation of the current time, in UTC.
- static string GenerateIso8601String (const ptime &p, bool extended=true)

  Generate an ISO8601 format date/time string representation of a POSIX time object.
- static double GenerateJulianDay (const ptime &dt, const ptime &ref_dt)

  Convert a date/time to fractional days since a reference date/time.
- static const string GenerateListingComment ()

  Get the listing entry comment string.
- static ptime GetCurrentUtcTime ()

  Generate a POSIX time object representing the current date and time (rounded to the nearest second).

Protected Member Functions

- void set_data_pack_format (XFallRiseDataPackFormat_t data_pack_format)

  Set the enumerated data packing format value for this message.
- void set_id_dive (int16_t id_dive)

  Set the dive or cycle number of this message.
- void set_id_instrument (uint16_t id_instrument)

  Set the unique instrument identifier.
- void set_id_xmsg_type (XMsgTypeId_t id_xmsg_type)

  Set the X-Message type identifier.
- void set_start_time (const ptime &start_time)

  Set the start time of this message.
- void set_xmsg_index (uint8_t xmsg_index)

  Set the index of this X-Message.
- void set_xmsg_size_bytes (uint16_t xmsg_size_bytes)

  Set the size of the X-Message, in bytes.
- vector< XSensorTripletCodeCounts > UnpackData (const vector< uint8_t > &data)

  Unpack the fall/rise data from the binary message.

Private Attributes

- XFallRiseDataPackFormat_t data_pack_format_

  The packing format type identifier.
- int16_t id_dive_

  The dive or cycle number of this message.
- uint16_t id_instrument_

  The unique WHOI identifier of this instrument.
- XMsgTypeId_t id_xmsg_type_

  The X-message type identifier.
- vector< XSensorTripletCodeCounts > sensor_data_

  The raw counts of measured fall, rise, or surface time-series data.
- ptime start_time_

  The start time reference.
- uint8_t xmsg_index_

  The index of this X-Message.
- uint16_t xmsg_size_bytes_

  The size of the X message, in bytes.

10.112.1 Detailed Description

The surface time-series data X-Message class.

10.112.2 Constructor & Destructor Documentation

10.112.2.1 XSurfaceData()

```
util::xmsg::XSurfaceData::XSurfaceData (
            int16_t dive,
            uint16_t whoi,
            const vector< uint8_t > & data)  [inline]
```

Parameterized constructor.

Parameters

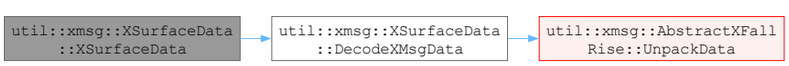
in	*dive*	The dive or cycle number of this message.
in	*whoi*	The unique WHOI identifier.
in	*data*	Binary surface pressure X-message data to decode.

Exceptions

util::log::MessageException	if an decode error occurs.

References DecodeXMsgData().

Here is the call graph for this function:



10.112.2.2 ~XSurfaceData()

```
util::xmsg::XSurfaceData::~XSurfaceData ()  [inline]
```

Destructor.

10.112.3 Member Function Documentation

10.112.3.1 CheckXMsgData()

```
const uint8_t * util::xmsg::XMessage::CheckXMsgData (
            const vector< uint8_t > & data,
            int id,
            int minBytes,
            bool parseSize = true)  [inline], [inherited]
```

Check the X-message for the correct ID, minimum size, and message terminator.

Parameters

in	*data*	The X-message binary data to check.
in	*id*	The expected ID of the X-message. If this field is less that 0, then the ID check will be skipped.
in	*minBytes*	The minimum number of bytes expected in a valid message.
in	*parseSize*	If TRUE, then the message size field will be parsed from the X-Message. Otherwise, the size field will not be parsed.

Exceptions

util::log::MessageException	if • the message does not contain the expected ID, • the message does not contain the minimum number of expected bytes, • the message does not contain a terminator (';') in the correct location, or, • any other error occurs.

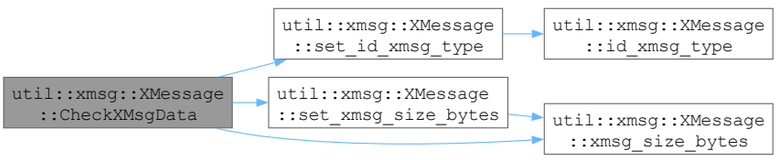
Returns

A pointer to the underlying data buffer of data.

References CATCH_BLOCK, EXC_THROW, lwrn, util::xmsg::XMessage::set_id_xmsg_type(), util::xmsg::XMessage::set_xmsg_size_byte
u16, and util::xmsg::XMessage::xmsg_size_bytes().

Referenced by util::xmsg::XAcousticPosition::DecodeXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XDriftPressure::Dec
util::xmsg::XDriftSalinity::DecodeXMsgData(), util::xmsg::XDriftTemperature::DecodeXMsgData(), util::xmsg::XEngAbort::DecodeXMsgDa
util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngBIT::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(),
util::xmsg::XEngDiagnostic::DecodeXMsgData(), util::xmsg::XEngDrift::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(),
util::xmsg::XEngStandard::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XGPS::DecodeXMsgData(),
util::xmsg::XHiResPressure::DecodeXMsgData(), util::xmsg::XHiResSalinity::DecodeXMsgData(), util::xmsg::XHiResTemperature::Decod
util::xmsg::XMissionCmdEcho::DecodeXMsgData(), util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XProfilePressure:
util::xmsg::XProfileSalinity::DecodeXMsgData(), util::xmsg::XProfileTemperature::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgD
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the call graph for this function:



10.112.3.2 data_pack_format()

```
XFallRiseDataPackFormat_t util::xmsg::AbstractXFallRise::data_pack_format () const  [inline],
[inherited]
```

Get the enumerated data packing format type.

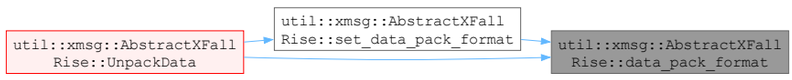
Returns

The enumerated data packing format type.

References util::xmsg::AbstractXFallRise::data_pack_format_.

Referenced by util::xmsg::AbstractXFallRise::set_data_pack_format(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the caller graph for this function:



10.112.3.3 DecodeXMsgData()

```
void util::xmsg::XSurfaceData::DecodeXMsgData (
            const vector< uint8_t > & data)  [inline], [virtual]
```

Decode the surface pressure X-message.

Parameters

in	*data*	Binary surface pressure X-message data to decode.

Exceptions

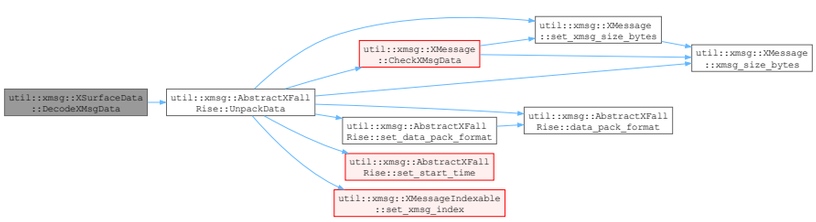
util::log::MessageException	if an decode error occurs.

Implements util::xmsg::AbstractXFallRise.

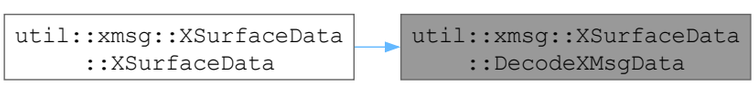
References CATCH_BLOCK, EXC_THROW, util::xmsg::AbstractXFallRise::UnpackData(), and util::xmsg::XMSG_TYPE_ID_PRES_SURF

Referenced by XSurfaceData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.112.3.4 GenerateCurrentUtcTimeIso8601String()

```
static string util::date::DateUtils::GenerateCurrentUtcTimeIso8601String ()  [inline], [static],
[inherited]
```

Generate an ISO8601 format date/time string representation of the current time, in UTC.

Note

All date/time objects use the UTC time-zone.

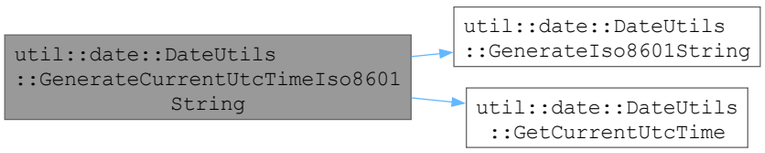
Returns

An ISO8601 format date/time string representation of the current time.

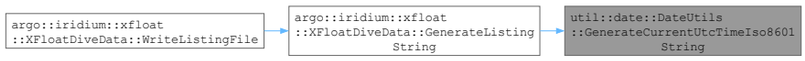
References util::date::DateUtils::GenerateIso8601String(), and util::date::DateUtils::GetCurrentUtcTime().

Referenced by argo::iridium::xfloat::XFloatDiveData::GenerateListingString().

Here is the call graph for this function:



Here is the caller graph for this function:



10.112.3.5 GenerateIso8601String()

```
static string util::date::DateUtils::GenerateIso8601String (
            const ptime & p,
            bool extended = true)  [inline], [static], [inherited]
```

Generate an ISO8601 format date/time string representation of a POSIX time object.

Note

All date/time objects use the UTC time-zone.

Parameters

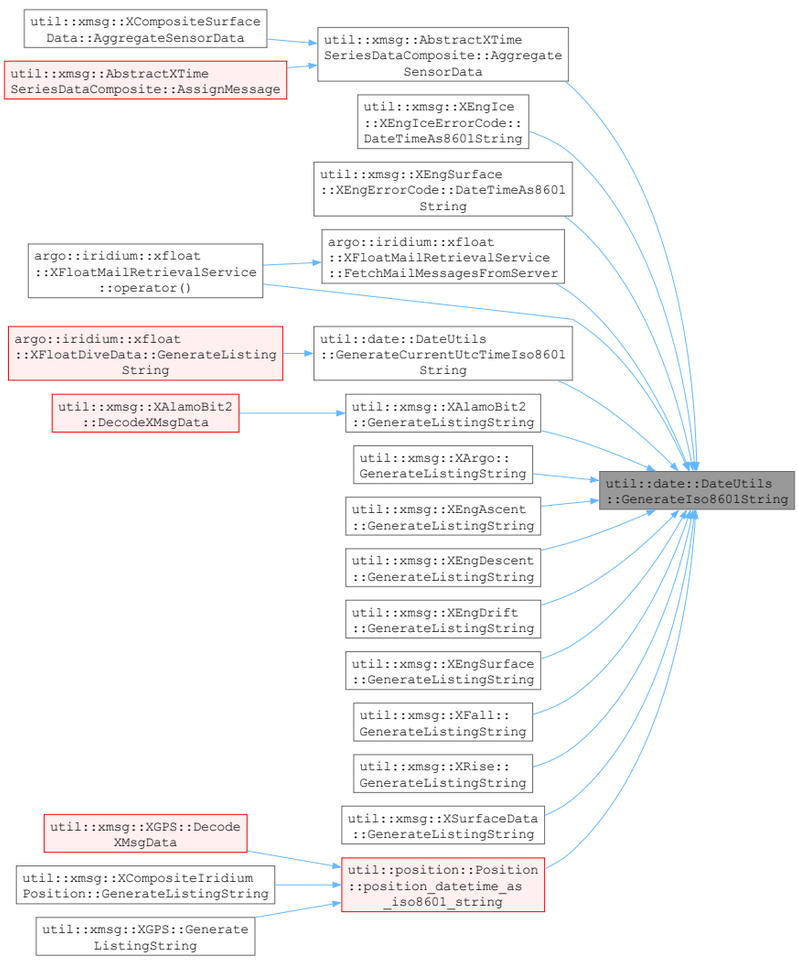
in	*p*	POSIX time from which to generate string.
in	*extended*	If TRUE, use the ISO 8601 extended format (YYYY-MM-DDThh:mm:ssZ); otherwise, use the basic format (YYYYMMDDThhmmss).

Returns

An ISO8601 format date/time string representation of the POSIX time object. If p is equal to boost::posix_↩ time::not_a_date_time, then the string "0000-00-00T00:00:00Z" (extended) or "00000000000000" (basic) will be returned.

Referenced by util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::AggregateSensorData(), util::xmsg::XEngIce::XEngIce
util::xmsg::XEngSurface::XEngErrorCode::DateTimeAs8601String(), argo::iridium::xfloat::XFloatMailRetrievalService::FetchMailMessages
util::date::DateUtils::GenerateCurrentUtcTimeIso8601String(), util::xmsg::XAlamoBit2::GenerateListingString(), util::xmsg::XArgo::Genera
util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListing
util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XRise::GenerateListingString(),
GenerateListingString(), argo::iridium::xfloat::XFloatMailRetrievalService::operator()(), and util::position::Position::position_datetime_as_is

Here is the caller graph for this function:



10.112.3.6 GenerateJulianDay()

```
static double util::date::DateUtils::GenerateJulianDay (
            const ptime & dt,
            const ptime & ref_dt)  [inline], [static], [inherited]
```

Convert a date/time to fractional days since a reference date/time.

Parameters

in	*dt*	The date/time to convert.
in	*ref*↩ *_dt*	The reference date/time (day 0).

Exceptions

util::log::MessageException	if either date/time is not valid.

Returns

The number of days (with millisecond resolution) from `ref_dt` to `dt`.

References CATCH_BLOCK, and EXC_THROW.

10.112.3.7 GenerateListingComment()

```
static const string util::xmsg::AbstractXFallRise::GenerateListingComment ()  [inline], [static],
[inherited]
```

Get the listing entry comment string.

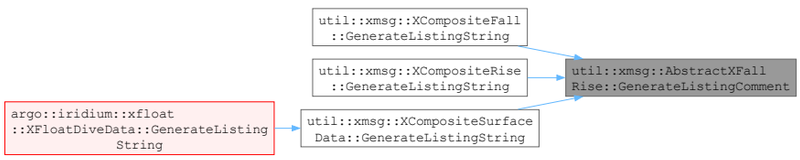
Returns

The listing entry comment string.

References CATCH_BLOCK.

Referenced by util::xmsg::XCompositeFall::GenerateListingString(), util::xmsg::XCompositeRise::GenerateListingString(), and util::xmsg::XCompositeSurfaceData::GenerateListingString().

Here is the caller graph for this function:



10.112.3.8 GenerateListingString()

```
const string util::xmsg::XSurfaceData::GenerateListingString (
            uint16_t pg,
            uint16_t po,
            uint16_t sg,
            uint16_t so,
            uint16_t tg,
            uint16_t to,
            float pc) const  [inline]
```

Generate the listing entry string for this surface time-series data X-Message.

Parameters

in	*pg*	The pressure sensor sensitivity scale factor, in LSB/dBar.
in	*po*	The pressure sensor offset, in dBar.
in	*sg*	The salinity sensor sensitivity scale factor, in LSB/PSU.
in	*so*	The salinity sensor offset, in PSU.
in	*tg*	The temperature sensor sensitivity scale factor, in LSB/degC.
in	*to*	The temperature sensor offset, in degC.
in	*pc*	The pressure correction, in dBar, to be subtracted computed pressure value.

Exceptions

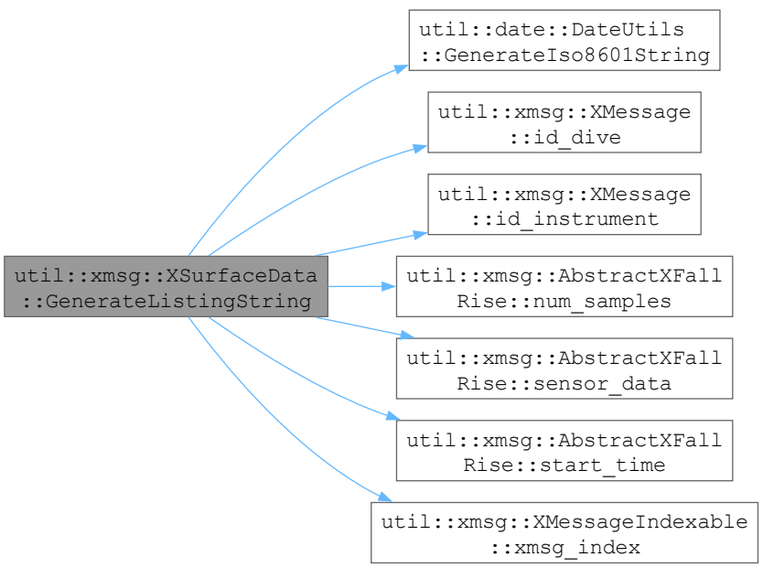
log::MessageException	if an error occurs while generating the listing string.

Returns

The listing entry string for this surface time-series data X-Message.

References CATCH_BLOCK, util::date::DateUtils::GenerateIso8601String(), util::xmsg::XMessage::id_dive(), util::xmsg::XMessage::id_ins
util::xmsg::AbstractXFallRise::num_samples(), util::xmsg::AbstractXFallRise::sensor_data(), util::xmsg::AbstractXFallRise::start_time(),
and util::xmsg::XMessageIndexable::xmsg_index().

Here is the call graph for this function:



10.112.3.9 GetCurrentUtcTime()

```
static ptime util::date::DateUtils::GetCurrentUtcTime ()  [inline], [static], [inherited]
```

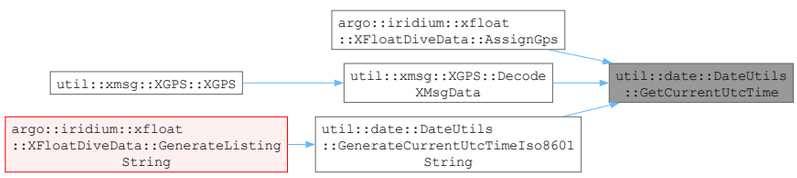
Generate a POSIX time object representing the current date and time (rounded to the nearest second).

Returns

A POSIX time object representing the current date and time (rounded to the nearest second).

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignGps(), util::xmsg::XGPS::DecodeXMsgData(), and util::date::DateUtils::GenerateCurrentUtcTimeIso8601String().

Here is the caller graph for this function:



10.112.3.10 id_dive()

```
int16_t util::xmsg::XMessage::id_dive () const  [inline], [inherited]
```

Get the dive or cycle number of this X-Message.

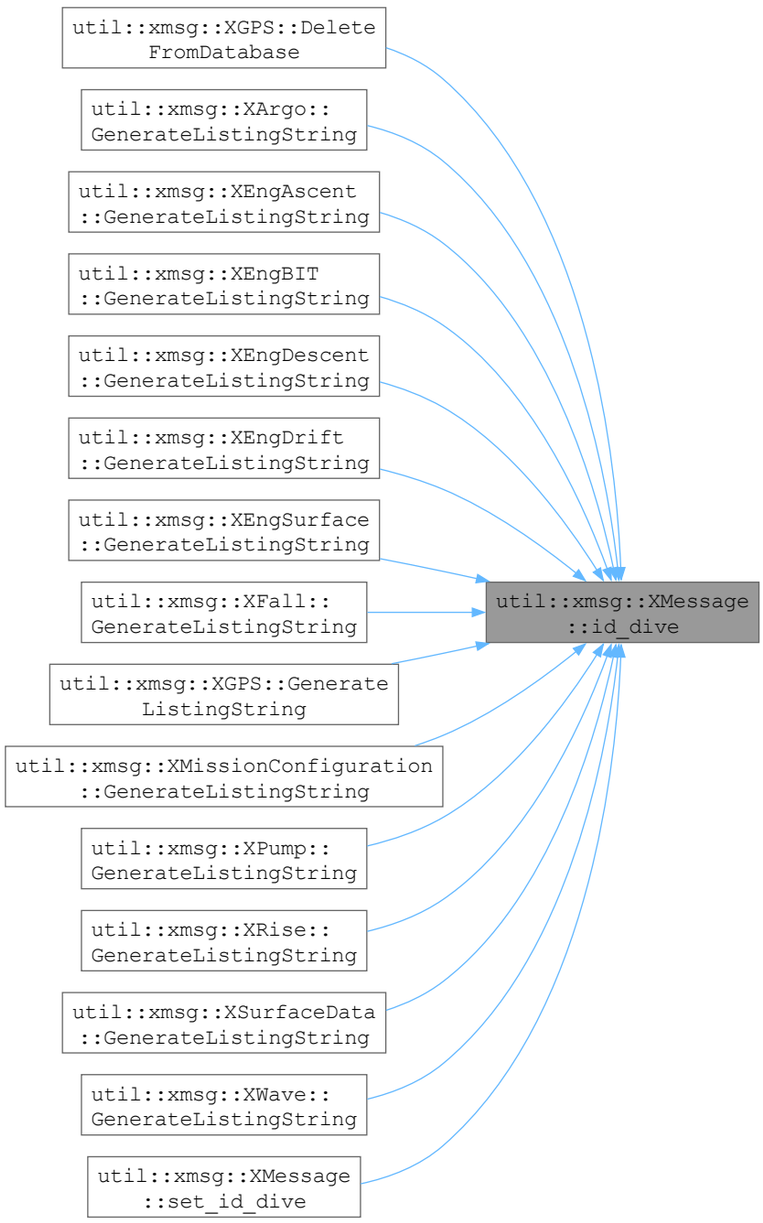
Returns

The dive or cycle number of this X-Message.

References util::xmsg::XMessage::id_dive_.

Referenced by util::xmsg::XGPS::DeleteFromDatabase(), util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateLis
util::xmsg::XEngBIT::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingStr
util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(),
util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing
GenerateListingString(), util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_dive().

Here is the caller graph for this function:



10.112.3.11 id_instrument()

```
uint16_t util::xmsg::XMessage::id_instrument () const  [inline], [inherited]
```

Get the unique instrument identifier of this X-Message.

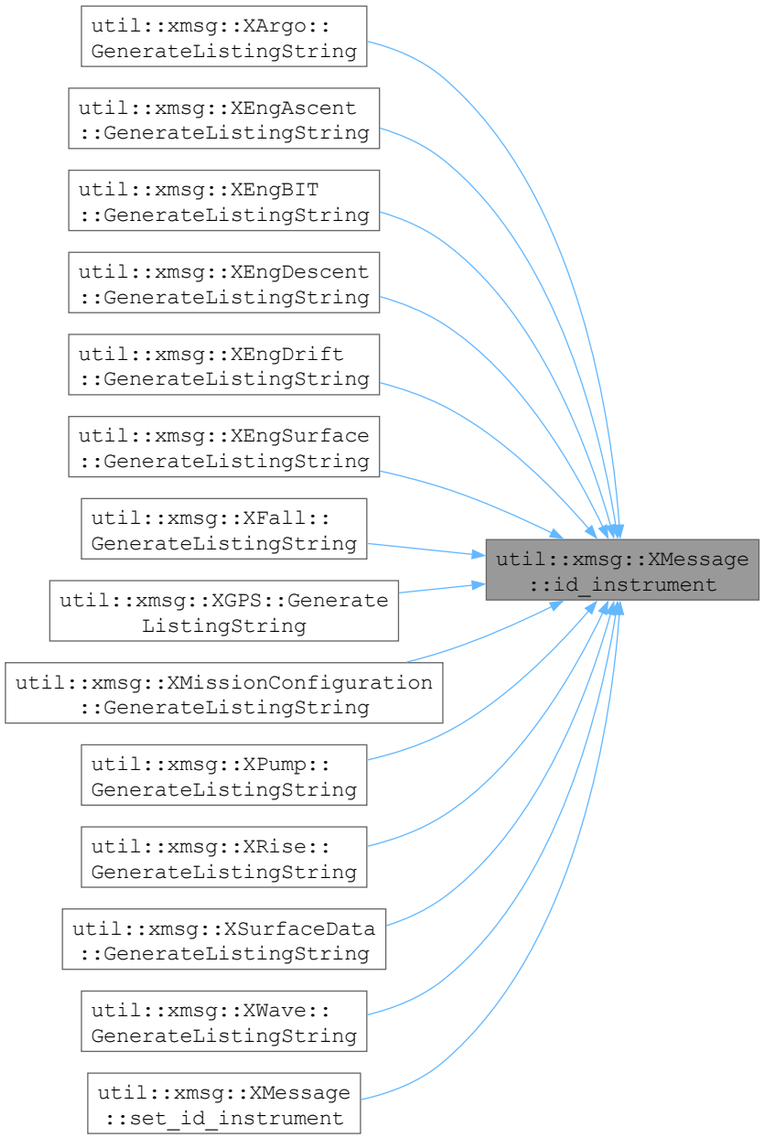
Returns

The unique instrument identifier of this X-Message.

References util::xmsg::XMessage::id_instrument_.

Referenced by util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngBIT::Generate
util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), util::xmsg::XEngSurface::GenerateListin
util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingS
util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), GenerateListingString(),
util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_instrument().

Here is the caller graph for this function:



10.112.3.12 id_xmsg_type()

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type () const  [inline], [inherited]
```

Get the enumerated X-Message type identifier.

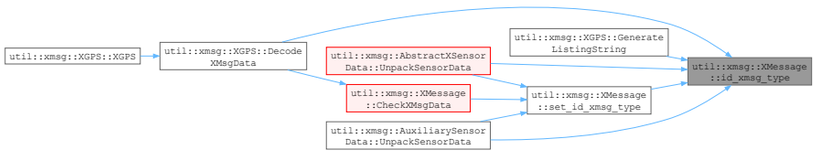
Returns

The enumerated X-Message type identifier.

References util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMessage::set_id_xmsg_typ
util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the caller graph for this function:



10.112.3.13 num_samples()

```
size_t util::xmsg::AbstractXFallRise::num_samples () const  [inline], [inherited]
```

Get the number of samples in this message.

Returns

The number of samples in this message.

References util::xmsg::AbstractXFallRise::sensor_data_.

Referenced by util::xmsg::XFall::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), and GenerateListingString().

Here is the caller graph for this function:

10.112.3.14 sensor_data()

```
const vector< XSensorTripletCodeCounts > & util::xmsg::AbstractXFallRise::sensor_data () const
[inline], [inherited]
```

Get the vector of sensor data triplets.

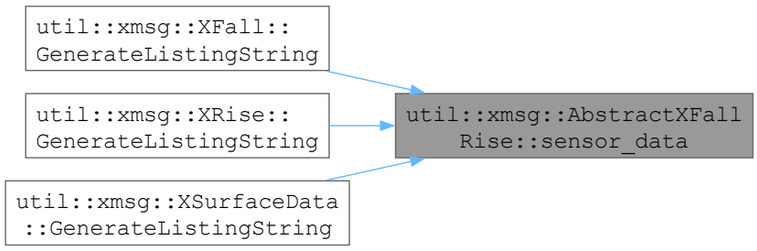
Returns

The vector of sensor data triplets.

References util::xmsg::AbstractXFallRise::sensor_data_.

Referenced by util::xmsg::XFall::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), and GenerateListingString().

Here is the caller graph for this function:



10.112.3.15 set_data_pack_format()

```
void util::xmsg::AbstractXFallRise::set_data_pack_format (
            XFallRiseDataPackFormat_t data_pack_format)  [inline], [protected], [inherited]
```

Set the enumerated data packing format value for this message.

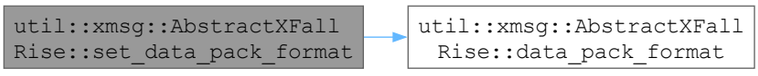
Parameters

in	*data_pack_format*	The enumerated data packing format value for this message to set.

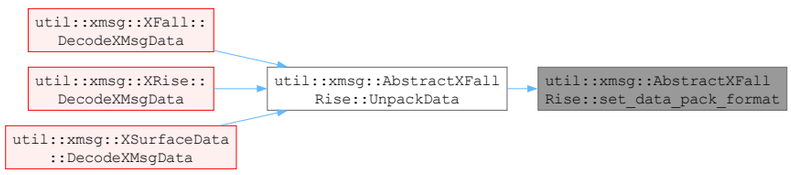
References util::xmsg::AbstractXFallRise::data_pack_format(), and util::xmsg::AbstractXFallRise::data_pack_format_.

Referenced by util::xmsg::AbstractXFallRise::UnpackData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.112.3.16 set_id_dive()

```
void util::xmsg::XMessage::set_id_dive (
            int16_t id_dive)  [inline], [protected], [inherited]
```

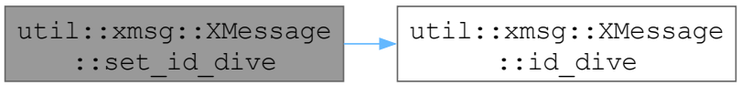
Set the dive or cycle number of this message.

Parameters

in	*id_dive*	The dive or cycle number of this message to set.

References util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::id_dive_.

Here is the call graph for this function:



10.112.3.17 set_id_instrument()

```
void util::xmsg::XMessage::set_id_instrument (
            uint16_t id_instrument)  [inline], [protected], [inherited]
```

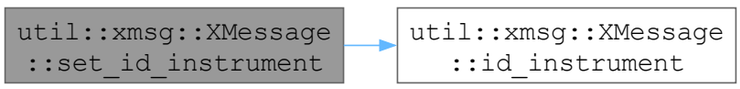
Set the unique instrument identifier.

Parameters

in	*id_instrument*	The unique instrument identifier to set.

References util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::id_instrument_.

Here is the call graph for this function:



10.112.3.18 set_id_xmsg_type()

```
void util::xmsg::XMessage::set_id_xmsg_type (
            XMsgTypeId_t id_xmsg_type)  [inline], [protected], [inherited]
```

Set the X-Message type identifier.

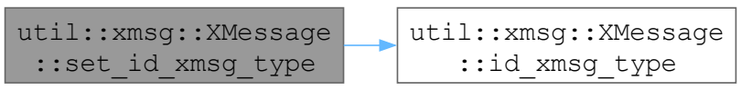
Parameters

in	*id_xmsg_type*	The X-Message type identifier to set.

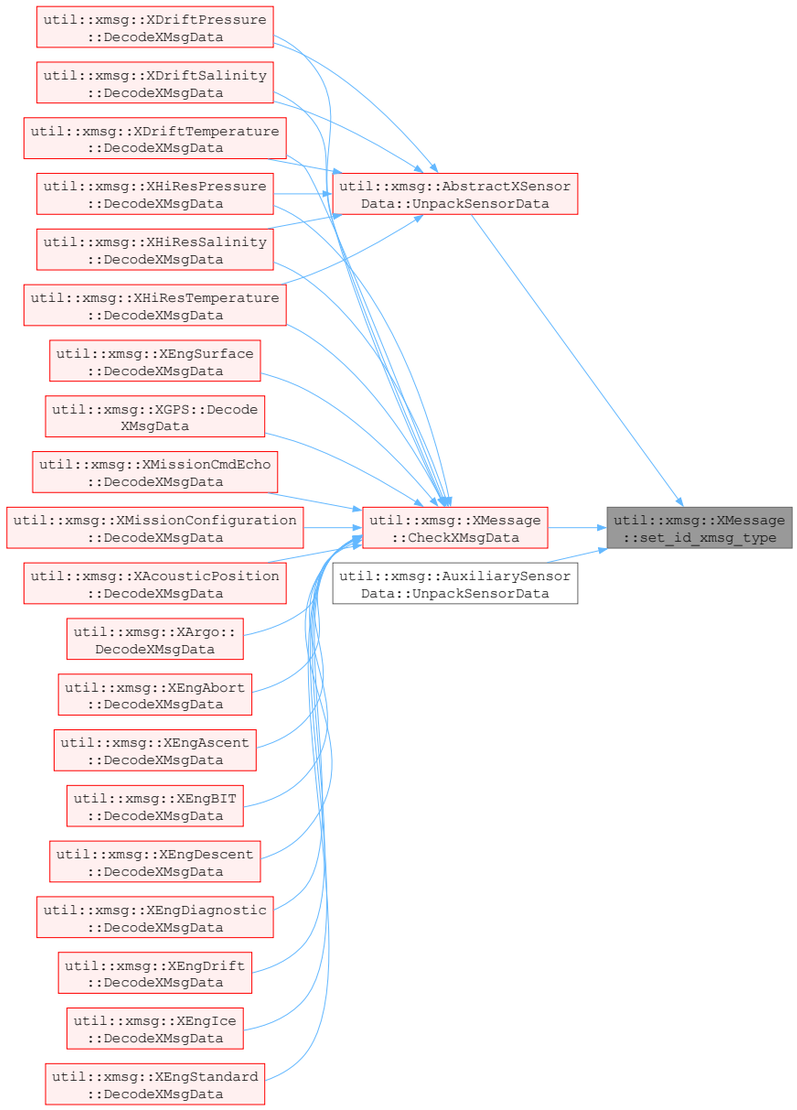
References util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.112.3.19 set_start_time()

```
void util::xmsg::AbstractXFallRise::set_start_time (
            const ptime & start_time)  [inline], [protected], [inherited]
```

Set the start time of this message.

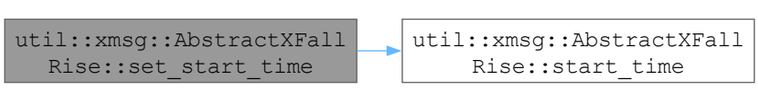
Parameters

in	*start_time*	The start time to set.

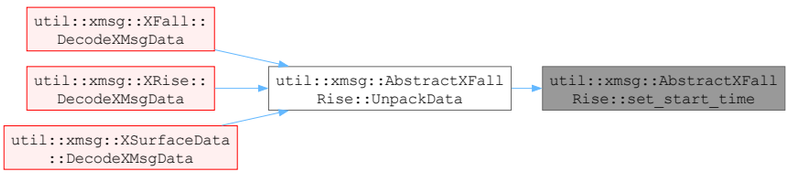
References util::xmsg::AbstractXFallRise::start_time(), and util::xmsg::AbstractXFallRise::start_time_.

Referenced by util::xmsg::AbstractXFallRise::UnpackData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.112.3.20 set_xmsg_index()

```
void util::xmsg::XMessageIndexable::set_xmsg_index (
            uint8_t xmsg_index)  [inline], [protected], [inherited]
```

Set the index of this X-Message.

Parameters

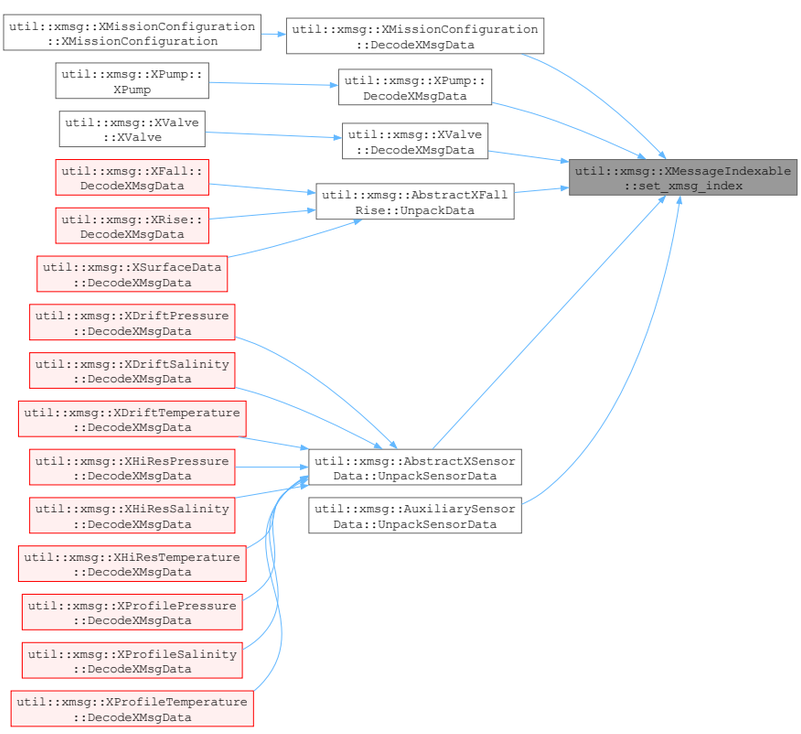
in	*xmsg_index*	The index of this X-Message.

References util::xmsg::XMessageIndexable::xmsg_index(), and util::xmsg::XMessageIndexable::xmsg_index_.

Referenced by util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(), util::xmsg::XValve::DecodeXMsgData(), util::xmsg::AbstractXFallRise::UnpackData(), util::xmsg::AbstractXSensorData::UnpackSensorDa and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:

Here is the caller graph for this function:



10.112.3.21 set_xmsg_size_bytes()

```
void util::xmsg::XMessage::set_xmsg_size_bytes (
            uint16_t xmsg_size_bytes)  [inline], [protected], [inherited]
```

Set the size of the X-Message, in bytes.

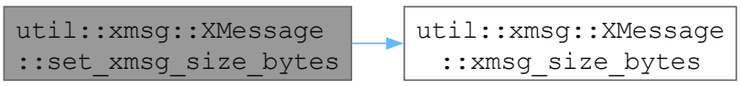
Parameters

in	*xmsg_size_bytes*	The size of the X-Message to set, in bytes.

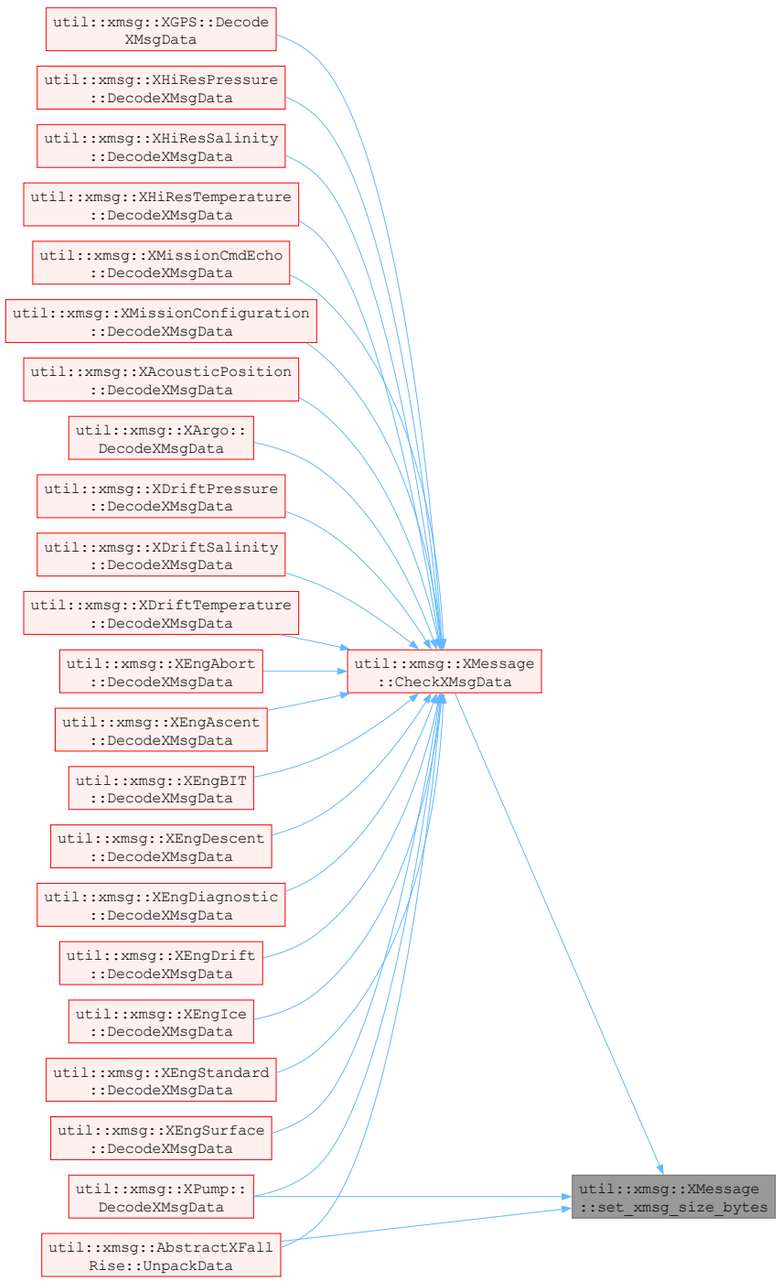
References util::xmsg::XMessage::xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XPump::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::Unpa

Here is the call graph for this function:



Here is the caller graph for this function:



10.112.3.22 start_time()

```
const ptime & util::xmsg::AbstractXFallRise::start_time () const  [inline], [inherited]
```

Get the start time of this message.

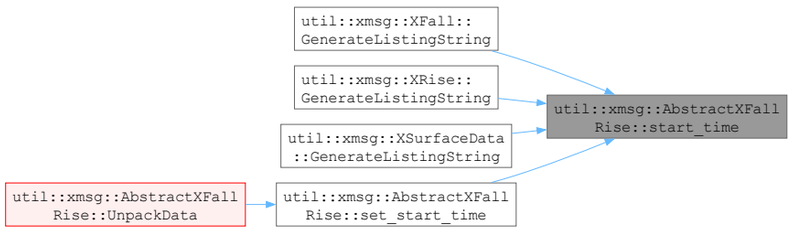
Returns

The start time of this message.

References util::xmsg::AbstractXFallRise::start_time_.

Referenced by util::xmsg::XFall::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), GenerateListingString(), and util::xmsg::AbstractXFallRise::set_start_time().

Here is the caller graph for this function:



10.112.3.23 UnpackData()

```
vector< XSensorTripletCodeCounts > util::xmsg::AbstractXFallRise::UnpackData (
            const vector< uint8_t > & data)  [inline], [protected], [inherited]
```

Unpack the fall/rise data from the binary message.

Parameters

in	*data*	The binary message to unpack.

Exceptions

util::log::MessageException	if • the message is of insufficient length, • an invalid packing format is encountered, or • the message terminator is in an invalid location.

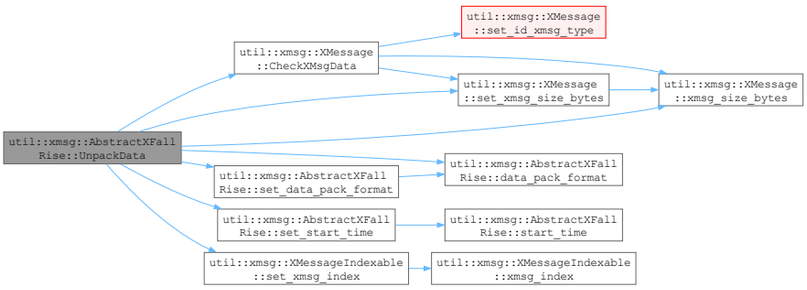
Returns

The raw counts of measured fall or rise time-series data.

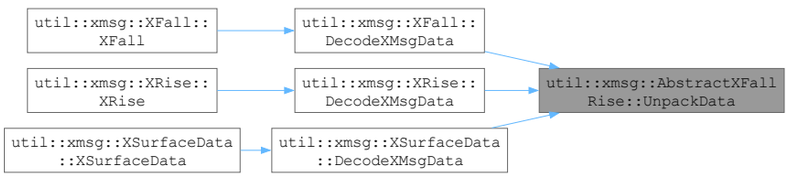
References CATCH_BLOCK, util::xmsg::XMessage::CheckXMsgData(), util::xmsg::AbstractXFallRise::data_pack_format(), EXC_THROW, lwrn, s2000, util::xmsg::AbstractXFallRise::sensor_data_, util::xmsg::AbstractXFallRise::set_data_pack_format(), util::xmsg::AbstractXFallRise::set_start_time(), util::xmsg::XMessageIndexable::set_xmsg_index(), util::xmsg::XMessage::set_xmsg_size_ XMSG_SENS_BAD, util::xmsg::XMessage::xmsg_size_bytes(), util::xmsg::XPHASE_CODE_UNKNOWN, util::xmsg::XRISEFALL_FMT_ util::xmsg::XRISEFALL_FMT_STNDRD, util::xmsg::XRISEFALL_FMT_STNDRD_WITH_TEMP_AND_SALN, and util::xmsg::XRISEFALL_FMT_UNKNOWN.

Referenced by util::xmsg::XFall::DecodeXMsgData(), util::xmsg::XRise::DecodeXMsgData(), and DecodeXMsgData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.112.3.24 xmsg_index()

```
uint8_t util::xmsg::XMessageIndexable::xmsg_index () const  [inline], [inherited]
```

Get the index of this X-Message.

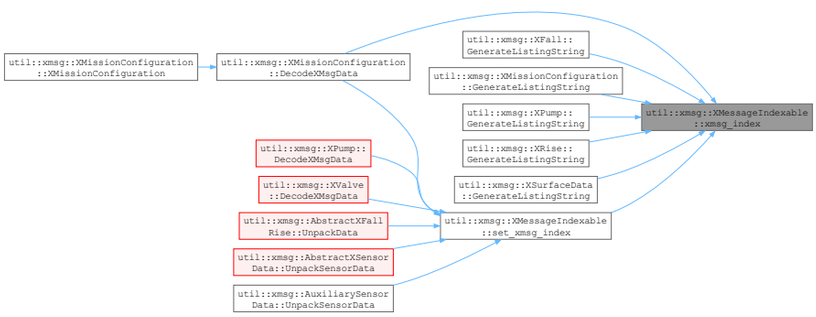
Returns

The index of this X-Message.

References util::xmsg::XMessageIndexable::xmsg_index_.

Referenced by util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing GenerateListingString(), and util::xmsg::XMessageIndexable::set_xmsg_index().

Here is the caller graph for this function:



10.112.3.25 xmsg_size_bytes()

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes () const  [inline], [inherited]
```

Get the size of the X-Message, in bytes.

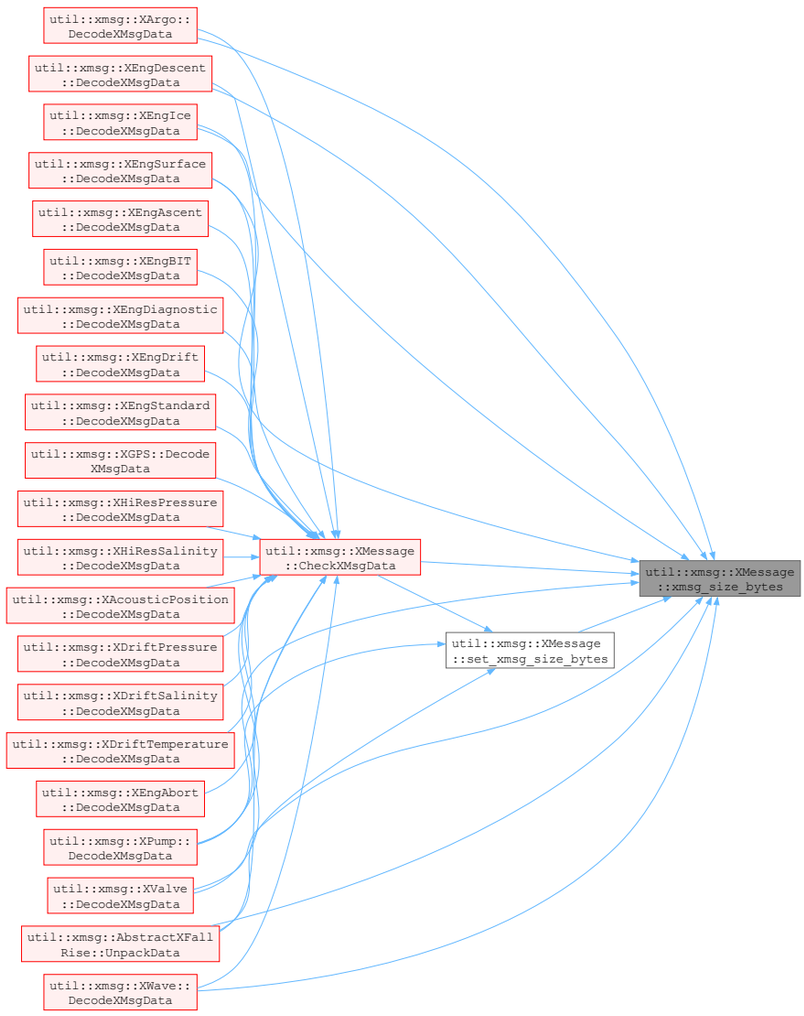
Returns

The size of the X-Message, in bytes.

References util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgI
util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(),
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), util::xmsg::XMessage::set_xmsg_size_bytes(),
and util::xmsg::AbstractXFallRise::UnpackData().

Here is the caller graph for this function:



10.112.4 Member Data Documentation

10.112.4.1 data_pack_format_

```
XFallRiseDataPackFormat_t util::xmsg::AbstractXFallRise::data_pack_format_  [private], [inherited]
```

The packing format type identifier.

Referenced by util::xmsg::AbstractXFallRise::data_pack_format(), and util::xmsg::AbstractXFallRise::set_data_pack_format().

10.112.4.2 id_dive_

```
int16_t util::xmsg::XMessage::id_dive_  [private], [inherited]
```

The dive or cycle number of this message.

Referenced by util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::set_id_dive().

10.112.4.3 id_instrument_

```
uint16_t util::xmsg::XMessage::id_instrument_  [private], [inherited]
```

The unique WHOI identifier of this instrument.

Referenced by util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::set_id_instrument().

10.112.4.4 id_xmsg_type_

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type_  [private], [inherited]
```

The X-message type identifier.

Referenced by util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::set_id_xmsg_type().

10.112.4.5 sensor_data_

```
vector<XSensorTripletCodeCounts> util::xmsg::AbstractXFallRise::sensor_data_  [private], [inherited]
```

The raw counts of measured fall, rise, or surface time-series data.

Referenced by util::xmsg::AbstractXFallRise::num_samples(), util::xmsg::AbstractXFallRise::sensor_data(), and util::xmsg::AbstractXFallRise::UnpackData().

10.112.4.6 start_time_

```
ptime util::xmsg::AbstractXFallRise::start_time_  [private], [inherited]
```

The start time reference.

Referenced by util::xmsg::AbstractXFallRise::set_start_time(), and util::xmsg::AbstractXFallRise::start_time().

10.112.4.7 xmsg_index_

```
uint8_t util::xmsg::XMessageIndexable::xmsg_index_  [private], [inherited]
```

The index of this X-Message.

Referenced by util::xmsg::XMessageIndexable::set_xmsg_index(), and util::xmsg::XMessageIndexable::xmsg_index().

10.112.4.8 xmsg_size_bytes_

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes_  [private], [inherited]
```

The size of the X message, in bytes.

Referenced by util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes().

The documentation for this class was generated from the following file:

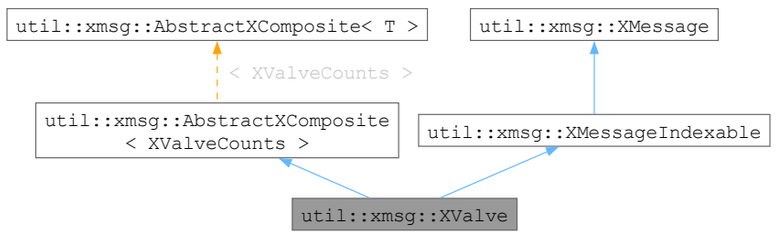
- data/xmsg/XSurfaceData.h

10.113 util::xmsg::XValve Class Reference

The valve data message class.

`#include <XValve.h>`

Inheritance diagram for util::xmsg::XValve:



Collaboration diagram for util::xmsg::XValve:

Public Member Functions

- XValve (int16_t dive, uint16_t whoi, const vector< uint8_t > &data)

  Parameterized constructor.
- void AppendAllElements (const static_vector< Element_t, I > &elements)

  Append a vector of elements to the vector of individual X-Messages elements that compose this composite object.
- virtual void AssignElement (const Element_t &element)

  Assign an element to this composite object.
- const uint8_t * CheckXMsgData (const vector< uint8_t > &data, int id, int minBytes, bool parseSize=true)

  Check the X-message for the correct ID, minimum size, and message terminator.
- void DecodeXMsgData (const vector< uint8_t > &data)

  Decode a valve data X-message (type 0xEB).
- const vector< Element_t > & elements () const

  Get the vector of individual X-Messages elements that compose this composite object.
- const string **GenerateListingString** (uint16_t pGain, uint16_t pOff, float pres_correction) const
- int16_t id_dive () const

  Get the dive or cycle number of this X-Message.
- uint16_t id_instrument () const

  Get the unique instrument identifier of this X-Message.
- XMsgTypeId_t id_xmsg_type () const

  Get the enumerated X-Message type identifier.
- virtual bool **IsEmpty** () const
- virtual size_t num_elements () const

  Get the number of individual elements that compose this composite object.
- uint8_t xmsg_index () const

  Get the index of this X-Message.
- uint16_t xmsg_size_bytes () const

  Get the size of the X-Message, in bytes.

Static Public Member Functions

- static const string GenerateListingComment ()

  The the listing file comment string.

Protected Member Functions

- void AppendAllElements (const vector< Element_t > &elements)

  Append a vector of elements to the vector of individual X-Messages elements that compose this composite object.
- void AppendElement (const Element_t &element)

  Append one element to the vector of individual X-Messages elements that compose this composite object.
- void set_id_dive (int16_t id_dive)

  Set the dive or cycle number of this message.
- void set_id_instrument (uint16_t id_instrument)

  Set the unique instrument identifier.
- void set_id_xmsg_type (XMsgTypeId_t id_xmsg_type)

  Set the X-Message type identifier.
- void set_xmsg_index (uint8_t xmsg_index)

  Set the index of this X-Message.
- void set_xmsg_size_bytes (uint16_t xmsg_size_bytes)

  Set the size of the X-Message, in bytes.

Private Types

- typedef XValveCounts Element_t

  Type definition for the element aggregated by this object.
- typedef ElementVec_t::const_iterator **ElementConstIter_t**
- typedef ElementVec_t::iterator **ElementIter_t**
- typedef vector< Element_t > **ElementVec_t**

Private Attributes

- ElementVec_t elements_

  The vector of individual X-Messages elements that compose this composite object.
- int16_t id_dive_

  The dive or cycle number of this message.
- uint16_t id_instrument_

  The unique WHOI identifier of this instrument.
- XMsgTypeId_t id_xmsg_type_

  The X-message type identifier.
- uint8_t xmsg_index_

  The index of this X-Message.
- uint16_t xmsg_size_bytes_

  The size of the X message, in bytes.

10.113.1 Detailed Description

The valve data message class.

An instance of this class represents a decoded valve data X-message, composed of the valve operation samples (see XValveCounts) reported for a dive.

10.113.2 Member Typedef Documentation

10.113.2.1 Element_t

```
typedef XValveCounts util::xmsg::AbstractXComposite< XValveCounts >::Element_t  [private], [inherited]
```

Type definition for the element aggregated by this object.

10.113.3 Constructor & Destructor Documentation

10.113.3.1 XValve()

```
util::xmsg::XValve::XValve (
            int16_t dive,
            uint16_t whoi,
            const vector< uint8_t > & data)  [inline]
```

Parameterized constructor.

Parameters

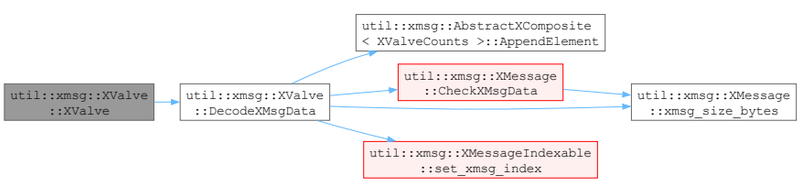
in	*dive*	The dive or cycle number of this message.
in	*whoi*	The unique WHOI instrument identifier.
in	*data*	The binary X-message data to decode.

Exceptions

util::log::MessageException	if the message cannot be decoded.

References DecodeXMsgData().

Here is the call graph for this function:



10.113.4 Member Function Documentation

10.113.4.1 AppendAllElements() [1/2]

```
void util::xmsg::AbstractXComposite< XValveCounts >::AppendAllElements (
            const static_vector< Element_t, I > & elements)  [inline], [inherited]
```

Append a vector of elements to the vector of individual X-Messages elements that compose this composite object.

Parameters

in	*elements*	The vector of elements to append.

10.113.4.2 AppendAllElements() [2/2]

```
void util::xmsg::AbstractXComposite< XValveCounts >::AppendAllElements (
            const vector< Element_t > & elements)  [inline], [protected], [inherited]
```

Append a vector of elements to the vector of individual X-Messages elements that compose this composite object.

Parameters

in	*elements*	The vector of elements to append.

10.113.4.3 AppendElement()

```
void util::xmsg::AbstractXComposite< XValveCounts >::AppendElement (
            const Element_t & element)  [inline], [protected], [inherited]
```

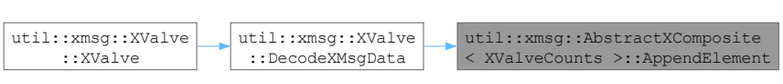
Append one element to the vector of individual X-Messages elements that compose this composite object.

Parameters

in	*element*	The element to append.

Referenced by DecodeXMsgData().

Here is the caller graph for this function:



10.113.4.4 AssignElement()

```
virtual void util::xmsg::AbstractXComposite< XValveCounts >::AssignElement (
            const Element_t & element)  [inline], [virtual], [inherited]
```

Assign an element to this composite object.

The default implementation appends the element; derived classes may override it to insert elements in order or reject duplicates.

Parameters

in	*element*	The element to assign.

10.113.4.5 CheckXMsgData()

```
const uint8_t * util::xmsg::XMessage::CheckXMsgData (
            const vector< uint8_t > & data,
            int id,
            int minBytes,
            bool parseSize = true)  [inline], [inherited]
```

Check the X-message for the correct ID, minimum size, and message terminator.

Parameters

in	*data*	The X-message binary data to check.
in	*id*	The expected ID of the X-message. If this field is less that 0, then the ID check will be skipped.
in	*minBytes*	The minimum number of bytes expected in a valid message.
in	*parseSize*	If TRUE, then the message size field will be parsed from the X-Message. Otherwise, the size field will not be parsed.

Exceptions

util::log::MessageException	if • the message does not contain the expected ID, • the message does not contain the minimum number of expected bytes, • the message does not contain a terminator (';') in the correct location, or, • any other error occurs.

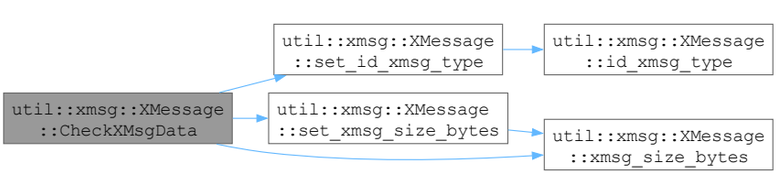
Returns

A pointer to the underlying data buffer of data.

References CATCH_BLOCK, EXC_THROW, lwrn, util::xmsg::XMessage::set_id_xmsg_type(), util::xmsg::XMessage::set_xmsg_size_byte
u16, and util::xmsg::XMessage::xmsg_size_bytes().

Referenced by util::xmsg::XAcousticPosition::DecodeXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XDriftPressure::Dec
util::xmsg::XDriftSalinity::DecodeXMsgData(), util::xmsg::XDriftTemperature::DecodeXMsgData(), util::xmsg::XEngAbort::DecodeXMsgDa
util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngBIT::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(),
util::xmsg::XEngDiagnostic::DecodeXMsgData(), util::xmsg::XEngDrift::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(),
util::xmsg::XEngStandard::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XGPS::DecodeXMsgData(),
util::xmsg::XHiResPressure::DecodeXMsgData(), util::xmsg::XHiResSalinity::DecodeXMsgData(), util::xmsg::XHiResTemperature::Decod
util::xmsg::XMissionCmdEcho::DecodeXMsgData(), util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XProfilePressure:
util::xmsg::XProfileSalinity::DecodeXMsgData(), util::xmsg::XProfileTemperature::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgD
DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the call graph for this function:



10.113.4.6 DecodeXMsgData()

```
void util::xmsg::XValve::DecodeXMsgData (
            const vector< uint8_t > & data)  [inline], [virtual]
```

Decode a valve data X-message (type 0xEB).

The low nibble of byte 3 is the message index. From byte 4, each valve operation is a 7-byte sample: a phase code (high nibble of the first byte), a 20-bit pressure count (low nibble plus the next two bytes), the valve duration in seconds (2 bytes) and the case vacuum after the operation (2 bytes). The case vacuum before the operation is not reported and is set to XMSG_BAD_U16. Each sample is stored as an XValveCounts element.

Parameters

in	*data*	The binary X-message data to decode.

Exceptions

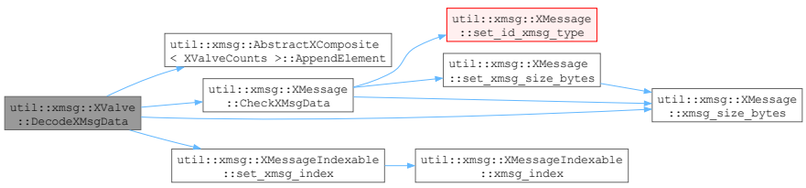
util::log::MessageException	if the message is shorter than its size field or the terminator is missing.

Implements util::xmsg::XMessageIndexable.

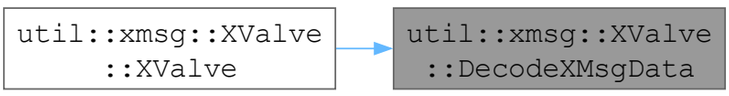
References util::xmsg::AbstractXComposite< XValveCounts >::AppendElement(), CATCH_BLOCK, util::xmsg::XMessage::CheckXMsgD
lwrn, util::xmsg::XMessageIndexable::set_xmsg_index(), XMSG_BAD_U16, and util::xmsg::XMessage::xmsg_size_bytes().

Referenced by XValve().

Here is the call graph for this function:



Here is the caller graph for this function:



10.113.4.7 elements()

```
const vector< Element_t > & util::xmsg::AbstractXComposite< XValveCounts >::elements () const
[inline], [inherited]
```

Get the vector of individual X-Messages elements that compose this composite object.

Returns

The vector of individual X-Messages elements that compose this composite object.

10.113.4.8 GenerateListingComment()

```
static const string util::xmsg::XValve::GenerateListingComment ()  [inline], [static]
```

The the listing file comment string.

Returns

The listing file comment string.

References CATCH_BLOCK.

10.113.4.9 id_dive()

```
int16_t util::xmsg::XMessage::id_dive () const  [inline], [inherited]
```

Get the dive or cycle number of this X-Message.

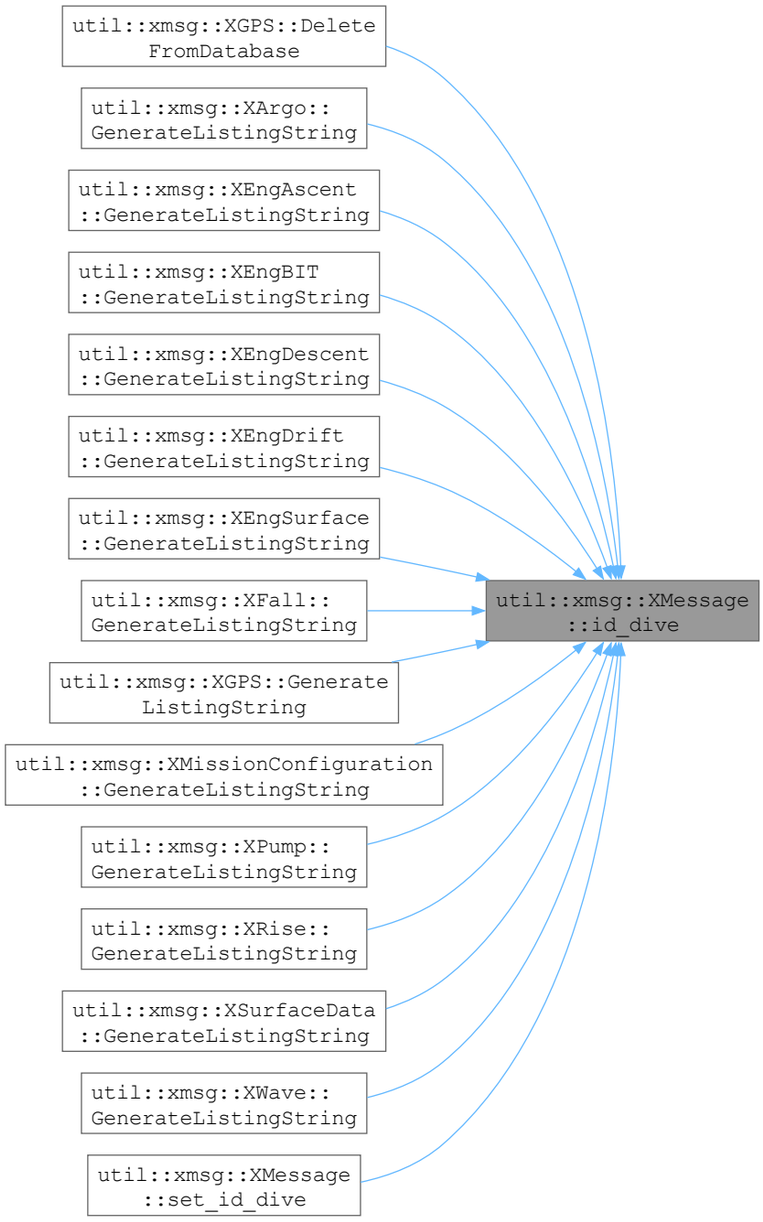
Returns

The dive or cycle number of this X-Message.

References util::xmsg::XMessage::id_dive_.

Referenced by util::xmsg::XGPS::DeleteFromDatabase(), util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateLis
util::xmsg::XEngBIT::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingStr
util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(),
util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing
util::xmsg::XSurfaceData::GenerateListingString(), util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_dive().

Here is the caller graph for this function:



10.113.4.10 id_instrument()

```
uint16_t util::xmsg::XMessage::id_instrument () const  [inline], [inherited]
```

Get the unique instrument identifier of this X-Message.

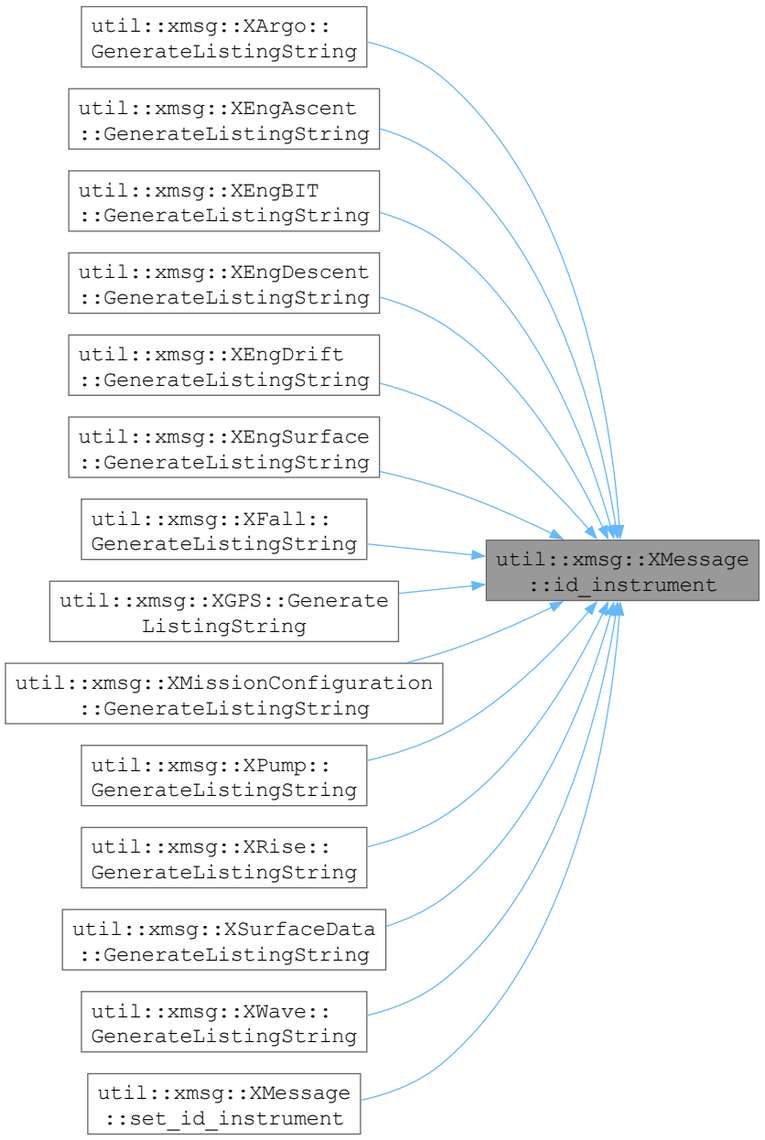
Returns

The unique instrument identifier of this X-Message.

References util::xmsg::XMessage::id_instrument_.

Referenced by util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngBIT::Generate
util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), util::xmsg::XEngSurface::GenerateListin
util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingS
util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(),
util::xmsg::XWave::GenerateListingString(), and util::xmsg::XMessage::set_id_instrument().

Here is the caller graph for this function:



10.113.4.11 id_xmsg_type()

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type () const  [inline], [inherited]
```

Get the enumerated X-Message type identifier.

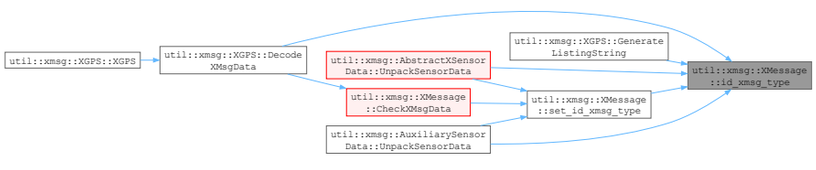
Returns

The enumerated X-Message type identifier.

References util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMessage::set_id_xmsg_typ
util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the caller graph for this function:



10.113.4.12 num_elements()

```
virtual size_t util::xmsg::AbstractXComposite< XValveCounts >::num_elements () const  [inline],
[virtual], [inherited]
```

Get the number of individual elements that compose this composite object.

Returns

The number of individual elements that compose this composite object.

10.113.4.13 set_id_dive()

```
void util::xmsg::XMessage::set_id_dive (
            int16_t id_dive)  [inline], [protected], [inherited]
```

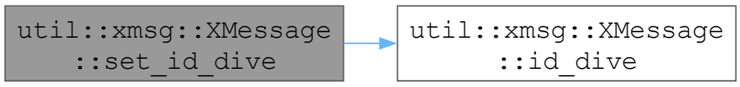
Set the dive or cycle number of this message.

Parameters

in	*id_dive*	The dive or cycle number of this message to set.

References util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::id_dive_.

Here is the call graph for this function:



10.113.4.14 set_id_instrument()

```
void util::xmsg::XMessage::set_id_instrument (
            uint16_t id_instrument)  [inline], [protected], [inherited]
```

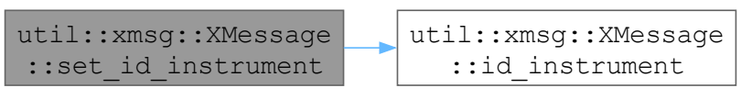
Set the unique instrument identifier.

Parameters

in	*id_instrument*	The unique instrument identifier to set.

References util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::id_instrument_.

Here is the call graph for this function:



10.113.4.15 set_id_xmsg_type()

```
void util::xmsg::XMessage::set_id_xmsg_type (
            XMsgTypeId_t id_xmsg_type)  [inline], [protected], [inherited]
```

Set the X-Message type identifier.

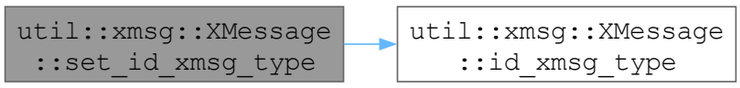
Parameters

in	*id_xmsg_type*	The X-Message type identifier to set.

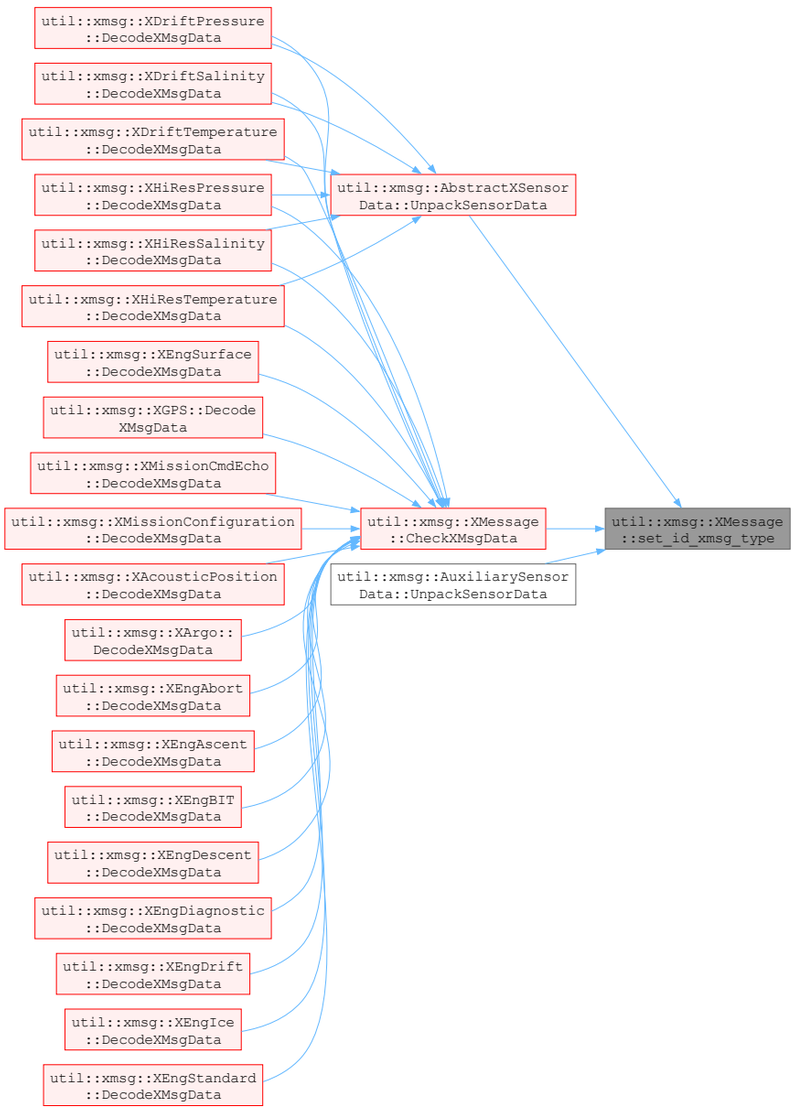
References util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.113.4.16 set_xmsg_index()

```
void util::xmsg::XMessageIndexable::set_xmsg_index (
            uint8_t xmsg_index)  [inline], [protected], [inherited]
```

Set the index of this X-Message.

Parameters

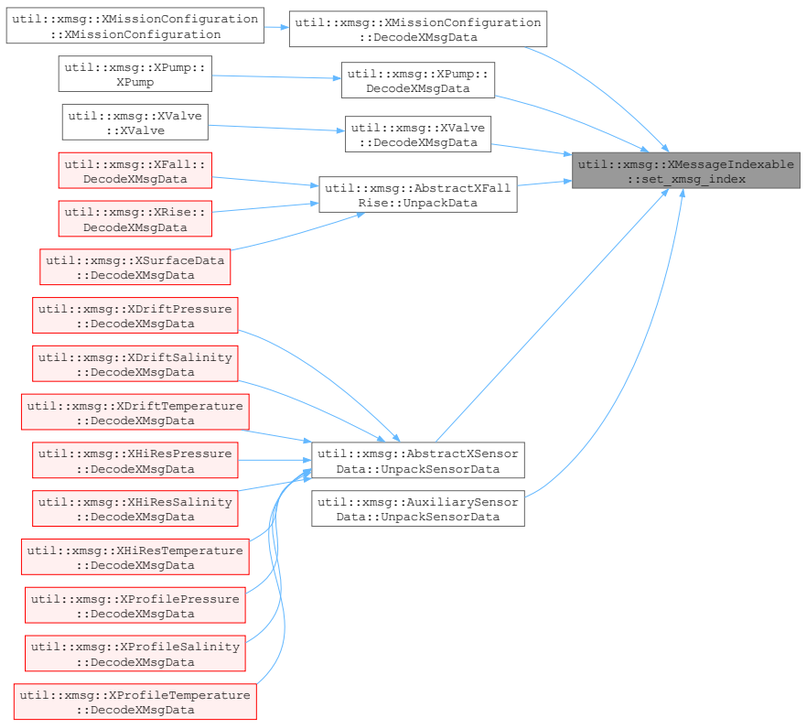
in	*xmsg_index*	The index of this X-Message.

References util::xmsg::XMessageIndexable::xmsg_index(), and util::xmsg::XMessageIndexable::xmsg_index_.

Referenced by util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(), DecodeXMsgData(), util::xmsg::AbstractXFallRise::UnpackData(), util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:

Here is the caller graph for this function:



10.113.4.17 set_xmsg_size_bytes()

```
void util::xmsg::XMessage::set_xmsg_size_bytes (
            uint16_t xmsg_size_bytes)  [inline], [protected], [inherited]
```

Set the size of the X-Message, in bytes.

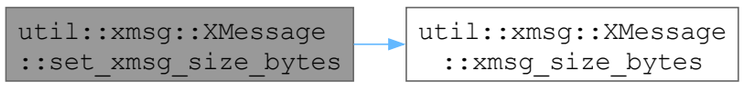
Parameters

in	*xmsg_size_bytes*	The size of the X-Message to set, in bytes.

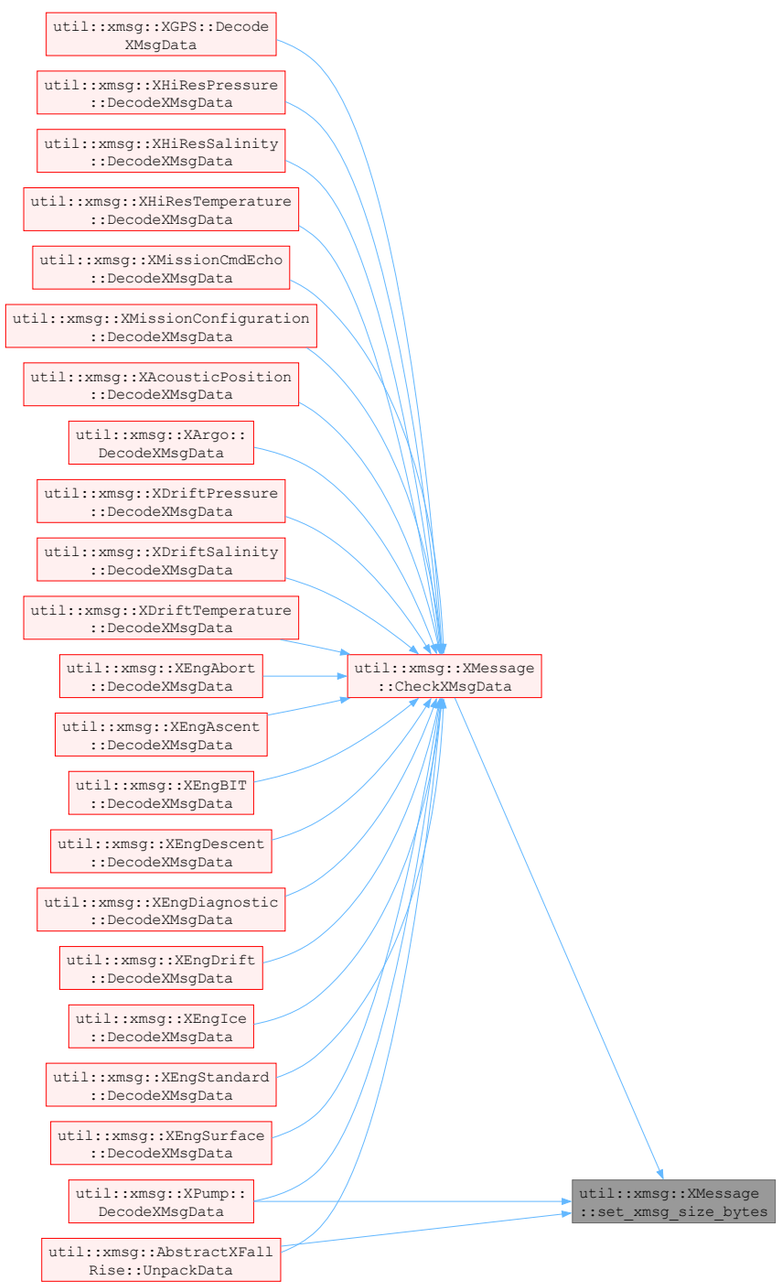
References util::xmsg::XMessage::xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XPump::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::Unpa

Here is the call graph for this function:



Here is the caller graph for this function:



10.113.4.18 xmsg_index()

```
uint8_t util::xmsg::XMessageIndexable::xmsg_index () const  [inline], [inherited]
```

Get the index of this X-Message.

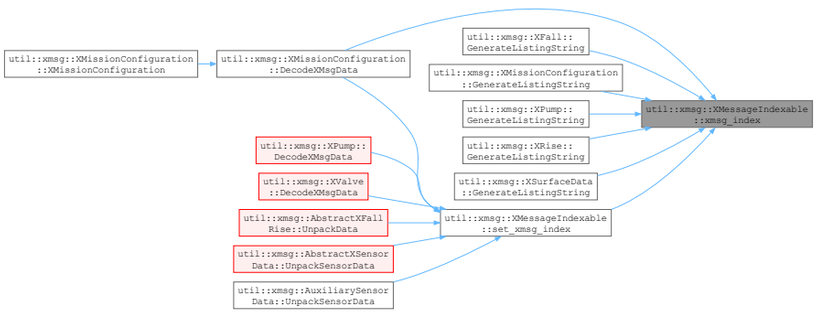
Returns

The index of this X-Message.

References util::xmsg::XMessageIndexable::xmsg_index_.

Referenced by util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing util::xmsg::XSurfaceData::GenerateListingString(), and util::xmsg::XMessageIndexable::set_xmsg_index().

Here is the caller graph for this function:



10.113.4.19 xmsg_size_bytes()

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes () const  [inline], [inherited]
```

Get the size of the X-Message, in bytes.

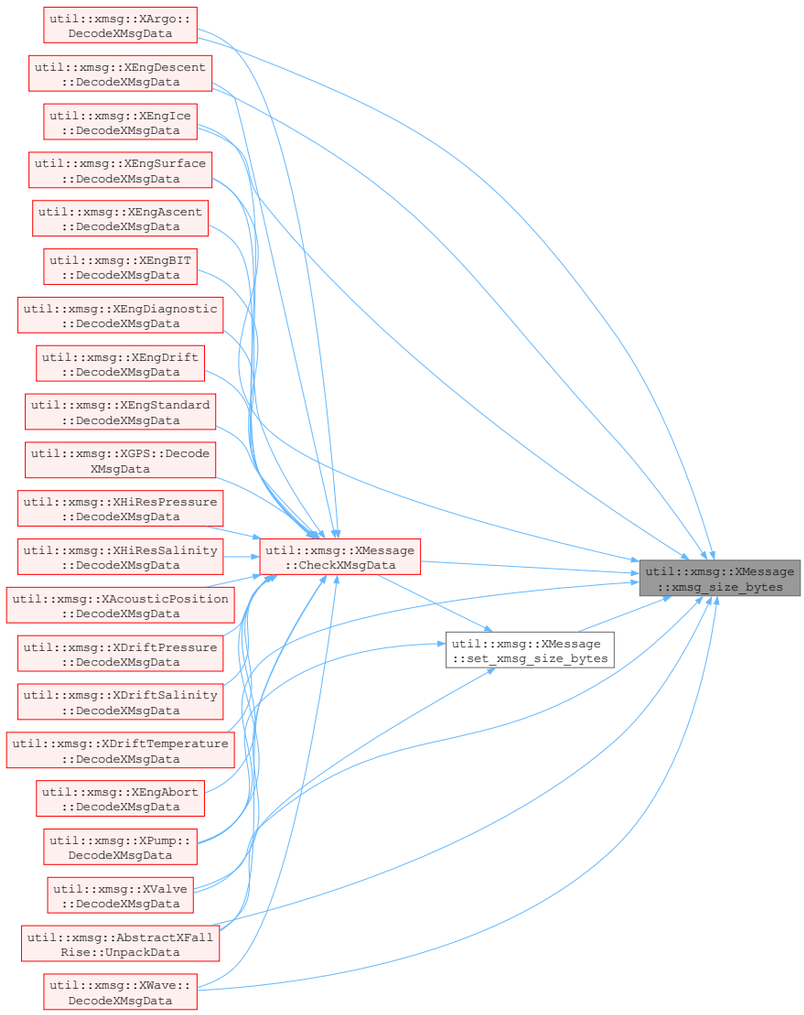
Returns

The size of the X-Message, in bytes.

References util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgI util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(), DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the caller graph for this function:



10.113.5 Member Data Documentation

10.113.5.1 elements_

```
ElementVec_t util::xmsg::AbstractXComposite< XValveCounts >::elements_  [private], [inherited]
```

The vector of individual X-Messages elements that compose this composite object.

10.113.5.2 id_dive_

```
int16_t util::xmsg::XMessage::id_dive_  [private], [inherited]
```

The dive or cycle number of this message.

Referenced by util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::set_id_dive().

10.113.5.3 id_instrument_

```
uint16_t util::xmsg::XMessage::id_instrument_  [private], [inherited]
```

The unique WHOI identifier of this instrument.

Referenced by util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::set_id_instrument().

10.113.5.4 id_xmsg_type_

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type_  [private], [inherited]
```

The X-message type identifier.

Referenced by util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::set_id_xmsg_type().

10.113.5.5 xmsg_index_

```
uint8_t util::xmsg::XMessageIndexable::xmsg_index_  [private], [inherited]
```

The index of this X-Message.

Referenced by util::xmsg::XMessageIndexable::set_xmsg_index(), and util::xmsg::XMessageIndexable::xmsg_index().

10.113.5.6 xmsg_size_bytes_

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes_  [private], [inherited]
```

The size of the X message, in bytes.

Referenced by util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes().

The documentation for this class was generated from the following file:

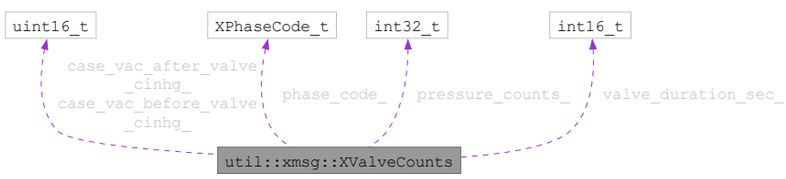
- data/xmsg/XValve.h

10.114 util::xmsg::XValveCounts Class Reference

The valve counts class.

```
#include <XValveCounts.h>
```

Collaboration diagram for util::xmsg::XValveCounts:



Public Member Functions

- XValveCounts (XPhaseCode_t code, int32_t count, int16_t sec, uint16_t vac0, uint16_t vac1) ATTR_NO_EXCEPTIONS
  The parameterized constructor.
- ~XValveCounts ()
  The destructor.
- template<typename T = float>
  T case_vac_after_pumping_inhg () const
  Get the measured vacuum after pumping, in inHg.
- uint16_t **case_vac_after_valve_cinhg** () const
- template<typename T = float>
  T case_vac_before_pumping_inhg () const
  Get the measured vacuum before pumping, in inHg.
- uint16_t **case_vac_before_valve_cinhg** () const
- const string **GenerateListingString** (uint16_t pg, uint16_t po, float pres_correction) const
- XPhaseCode_t **phase_code** () const
- int32_t **pressure_counts** () const
- int16_t **valve_duration_sec** () const

Protected Member Functions

- template<typename T = double>
  T PressureValue (uint16_t pg, uint16_t po, float pc) const

  Get the measured pressure value, in dBar.
- void **set_case_vac_after_valve_cinhg** (uint16_t case_vac_after_valve_cinhg)
- void **set_case_vac_before_valve_cinhg** (uint16_t case_vac_before_valve_cinhg)
- void **set_phase_code** (XPhaseCode_t phase_code)
- void **set_pressure_counts** (int32_t pressure_counts)
- void **set_valve_duration_sec** (int16_t valve_duration_sec)

Private Attributes

- uint16_t case_vac_after_valve_cinhg_

  The measured vacuum after opening the valve, in 0.01inHg.
- uint16_t case_vac_before_valve_cinhg_

  The measured vacuum before opening the valve, in 0.01inHg.
- XPhaseCode_t phase_code_

  The phase code of this valve data measurement.
- int32_t pressure_counts_

  The measured pressure, before opening the valve, in raw sensor counts.
- int16_t valve_duration_sec_

  The duration that the valve was open, in seconds.

Friends

- class **XDiveData**

10.114.1 Detailed Description

The valve counts class.

An instance of this class represents a single valve operation decoded from a valve data message (see XValve): the pressure, and the case vacuum before and after the valve operation.

10.114.2 Constructor & Destructor Documentation

10.114.2.1 XValveCounts()

```
util::xmsg::XValveCounts::XValveCounts (
            XPhaseCode_t code,
            int32_t count,
            int16_t sec,
            uint16_t vac0,
            uint16_t vac1)  [inline]
```

The parameterized constructor.

Parameters

in	*code*	The phase code.
in	*count*	The measured pressure, before opening the valve, in raw sensor counts.
in	*sec*	The duration that the valve was open, in seconds.
in	*vac0*	The measured vacuum after opening the valve, in 0.01inHg.
in	*vac1*	The measured vacuum before opening the valve, in 0.01inHg.

10.114.2.2 ~XValveCounts()

```
util::xmsg::XValveCounts::~XValveCounts ()  [inline]
```

The destructor.

10.114.3 Member Function Documentation

10.114.3.1 case_vac_after_pumping_inhg()

```
template<typename T = float>
T util::xmsg::XValveCounts::case_vac_after_pumping_inhg () const  [inline]
```

Get the measured vacuum after pumping, in inHg.

Returns

The measured vacuum after pumping, in inHg.

10.114.3.2 case_vac_before_pumping_inhg()

```
template<typename T = float>
T util::xmsg::XValveCounts::case_vac_before_pumping_inhg () const  [inline]
```

Get the measured vacuum before pumping, in inHg.

Returns

The measured vacuum before pumping, in inHg.

10.114.3.3 PressureValue()

```
template<typename T = double>
T util::xmsg::XValveCounts::PressureValue (
            uint16_t pg,
            uint16_t po,
            float pc) const  [inline], [protected]
```

Get the measured pressure value, in dBar.

Parameters

in	*pg*	The pressure sensor sensitivity scale factor, in LSB/dBar.
in	*po*	The pressure sensor offset, in dBar.
in	*pc*	The pressure correction, in dBar, to be subtracted computed pressure value.

Returns

The measured pressure value, in dBar.

References pressure_counts_, XMSG_SENS_BAD_PRES, and XMSG_SENS_MAX_PRES.

10.114.4 Member Data Documentation

10.114.4.1 case_vac_after_valve_cinhg_

```
uint16_t util::xmsg::XValveCounts::case_vac_after_valve_cinhg_  [private]
```

The measured vacuum after opening the valve, in 0.01inHg.

10.114.4.2 case_vac_before_valve_cinhg_

```
uint16_t util::xmsg::XValveCounts::case_vac_before_valve_cinhg_  [private]
```

The measured vacuum before opening the valve, in 0.01inHg.

10.114.4.3 phase_code_

```
XPhaseCode_t util::xmsg::XValveCounts::phase_code_  [private]
```

The phase code of this valve data measurement.

10.114.4.4 pressure_counts_

```
int32_t util::xmsg::XValveCounts::pressure_counts_  [private]
```

The measured pressure, before opening the valve, in raw sensor counts.

Referenced by PressureValue().

10.114.4.5 valve_duration_sec_

```
int16_t util::xmsg::XValveCounts::valve_duration_sec_  [private]
```

The duration that the valve was open, in seconds.

The documentation for this class was generated from the following file:

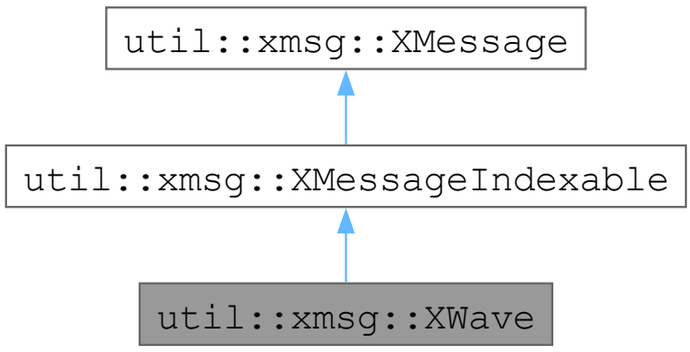
- data/xmsg/common/XValveCounts.h

10.115 util::xmsg::XWave Class Reference

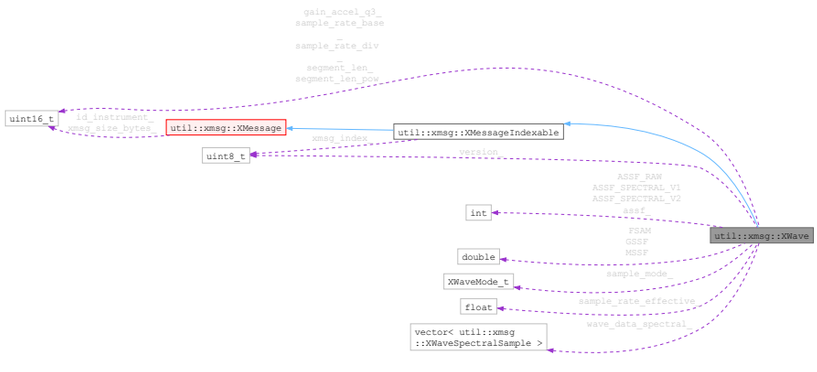
Class representing measured ocean surface wave data X message.

```
#include <XWave.h>
```

Inheritance diagram for util::xmsg::XWave:



Collaboration diagram for util::xmsg::XWave:



Public Member Functions

- XWave (int16_t dive, uint16_t whoi, const vector< uint8_t > &data)

  The parameterized constructor.
- ~XWave ()

  The destructor.
- const uint8_t * CheckXMsgData (const vector< uint8_t > &data, int id, int minBytes, bool parseSize=true)

  Check the X-message for the correct ID, minimum size, and message terminator.
- void DecodeXMsgData (const vector< uint8_t > &data)

  Decode the wave data message.
- unsigned int **effective_ssf** () const
- void FixFuckup9000 ()

  Fix MRV's 9000th data encoding fuckup.
- float **gain_accel** () const
- const string GenerateListingString () const

  Generate a listing entry string from this ocean surface wave-field data message.
- int16_t id_dive () const

  Get the dive or cycle number of this X-Message.
- uint16_t id_instrument () const

  Get the unique instrument identifier of this X-Message.
- XMsgTypeId_t id_xmsg_type () const

  Get the enumerated X-Message type identifier.
- uint16_t **segment_len** () const
- uint16_t **segment_len_pow** () const
- void **set_segment_len** (uint16_t segment_len)
- void **set_segment_len_pow** (uint16_t segment_len_pow)
- uint8_t xmsg_index () const

  Get the index of this X-Message.
- uint16_t xmsg_size_bytes () const

  Get the size of the X-Message, in bytes.

Protected Member Functions

- void set_id_dive (int16_t id_dive)

  Set the dive or cycle number of this message.
- void set_id_instrument (uint16_t id_instrument)

  Set the unique instrument identifier.
- void set_id_xmsg_type (XMsgTypeId_t id_xmsg_type)

  Set the X-Message type identifier.
- void set_xmsg_index (uint8_t xmsg_index)

  Set the index of this X-Message.
- void set_xmsg_size_bytes (uint16_t xmsg_size_bytes)

  Set the size of the X-Message, in bytes.

Private Attributes

- unsigned int assf_

  The accelerometer sensitivity scale factor, in LSB/g.
- uint16_t gain_accel_q3_

  Accelerometer software gain.
- int16_t id_dive_

  The dive or cycle number of this message.
- uint16_t id_instrument_

  The unique WHOI identifier of this instrument.
- XMsgTypeId_t id_xmsg_type_

  The X-message type identifier.
- XWaveMode_t sample_mode_

  The ocean surface wave field sampling mode.
- uint16_t sample_rate_base_

  The base sample rate, in Hz.
- uint16_t sample_rate_div_

  The sample rate divider, in Hz/Hz.

- float sample_rate_effective_

  The sample rate of the ocean surface wave measurements, in Hz.
- uint16_t segment_len_

  The power-of-two segment length for spectral data.
- uint16_t segment_len_pow_

  The power-of-two segment length for spectral data.
- uint8_t **version_**
- vector< XWaveSpectralSample > wave_data_spectral_

  The spectral data samples.
- uint8_t xmsg_index_

  The index of this X-Message.
- uint16_t xmsg_size_bytes_

  The size of the X message, in bytes.

Static Private Attributes

- static const unsigned int ASSF_RAW [ ] = { 16384, 8192, 4096, 2048 }

  Available accelerometer sensitivity scale factors for raw wave data.
- static const unsigned int ASSF_SPECTRAL_V1 [ ] = { 8192, 2048, 512, 128 }

  Available accelerometer sensitivity scale factors for spectral wave data – message version 1.
- static const unsigned int ASSF_SPECTRAL_V2 [ ] = { 8192, 2048, 512, 128 }

  Available accelerometer sensitivity scale factors for spectral wave data – message version 2.
- static const double FSAM [ ] = { 1 }

  Available ample rate frequencies.
- static const double GSSF [ ] = { 131.072, 65.536, 32.768, 16.384 }

  Available gyrscope sensitivity scale factors.
- static const double MSSF [ ] = { (30.0 / 9.0) }

  Available magenetometer sensitivity scale factors.

10.115.1 Detailed Description

Class representing measured ocean surface wave data X message.

10.115.2 Constructor & Destructor Documentation

10.115.2.1 XWave()

```
util::xmsg::XWave::XWave (
            int16_t dive,
            uint16_t whoi,
            const vector< uint8_t > & data)  [inline]
```

The parameterized constructor.

Parameters

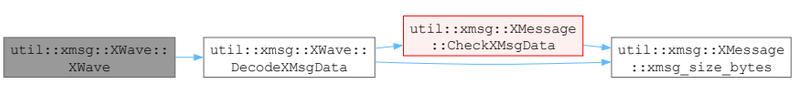
in	*dive*	The dive or cycle number of this message.
in	*whoi*	The unique WHOI identifier.
in	*data*	Binary surface pressure X-message data to decode.

Exceptions

util::log::MessageException	if an decode error occurs.

References DecodeXMsgData().

Here is the call graph for this function:



10.115.2.2 ~XWave()

```
util::xmsg::XWave::~XWave ()  [inline]
```

The destructor.

10.115.3 Member Function Documentation

10.115.3.1 CheckXMsgData()

```
const uint8_t * util::xmsg::XMessage::CheckXMsgData (
            const vector< uint8_t > & data,
            int id,
            int minBytes,
            bool parseSize = true)  [inline], [inherited]
```

Check the X-message for the correct ID, minimum size, and message terminator.

Parameters

in	*data*	The X-message binary data to check.
in	*id*	The expected ID of the X-message. If this field is less that 0, then the ID check will be skipped.
in	*minBytes*	The minimum number of bytes expected in a valid message.
in	*parseSize*	If TRUE, then the message size field will be parsed from the X-Message. Otherwise, the size field will not be parsed.

Exceptions

util::log::MessageException	if • the message does not contain the expected ID, • the message does not contain the minimum number of expected bytes, • the message does not contain a terminator (';') in the correct location, or, • any other error occurs.

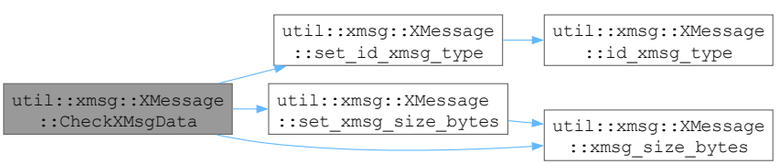
Returns

A pointer to the underlying data buffer of data.

References CATCH_BLOCK, EXC_THROW, lwrn, util::xmsg::XMessage::set_id_xmsg_type(), util::xmsg::XMessage::set_xmsg_size_byte
u16, and util::xmsg::XMessage::xmsg_size_bytes().

Referenced by util::xmsg::XAcousticPosition::DecodeXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XDriftPressure::Dec
util::xmsg::XDriftSalinity::DecodeXMsgData(), util::xmsg::XDriftTemperature::DecodeXMsgData(), util::xmsg::XEngAbort::DecodeXMsgDa
util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngBIT::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(),
util::xmsg::XEngDiagnostic::DecodeXMsgData(), util::xmsg::XEngDrift::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(),
util::xmsg::XEngStandard::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XGPS::DecodeXMsgData(),
util::xmsg::XHiResPressure::DecodeXMsgData(), util::xmsg::XHiResSalinity::DecodeXMsgData(), util::xmsg::XHiResTemperature::Decod
util::xmsg::XMissionCmdEcho::DecodeXMsgData(), util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XProfilePressure:
util::xmsg::XProfileSalinity::DecodeXMsgData(), util::xmsg::XProfileTemperature::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgD
util::xmsg::XValve::DecodeXMsgData(), DecodeXMsgData(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the call graph for this function:



10.115.3.2 DecodeXMsgData()

```
void util::xmsg::XWave::DecodeXMsgData (
            const vector< uint8_t > & data)  [virtual]
```

Decode the wave data message.

Parameters

in	*data*	The byte array to decode.

Exceptions

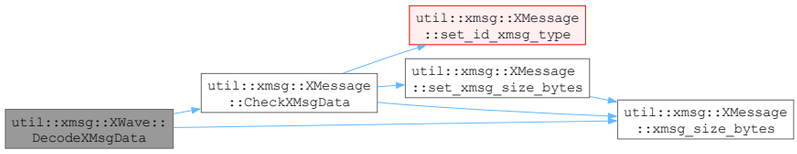
log::MessageException	on error.

Implements util::xmsg::XMessageIndexable.

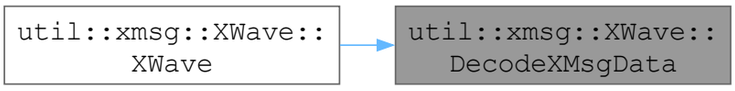
References assf_, ASSF_SPECTRAL_V1, ASSF_SPECTRAL_V2, CATCH_BLOCK, util::xmsg::XMessage::CheckXMsgData(), dbg1, EXC_THROW, gain_accel_q3_, i16, ldbg, lerr, lwrn, sample_mode_, sample_rate_base_, sample_rate_div_, sample_rate_effective_, u08, u16, wave_data_spectral_, util::xmsg::XMessage::xmsg_size_bytes(), util::xmsg::XWAVE_MODE_RAW, util::xmsg::XWAVE_MODE_SPECTRAL, and util::xmsg::XWAVE_MODE_UNKNOWN.

Referenced by XWave().

Here is the call graph for this function:



Here is the caller graph for this function:



10.115.3.3 FixFuckup9000()

```
void util::xmsg::XWave::FixFuckup9000 ()  [inline]
```

Fix MRV's 9000th data encoding fuckup.

For select firmware versions, only every other spectral value is returned. The seconds half of the spectra is all zeros. Here we fix this by doubling the effective sample rate

References sample_rate_base_, sample_rate_div_, sample_rate_effective_, and wave_data_spectral_.

10.115.3.4 GenerateListingString()

```
const string util::xmsg::XWave::GenerateListingString () const
```

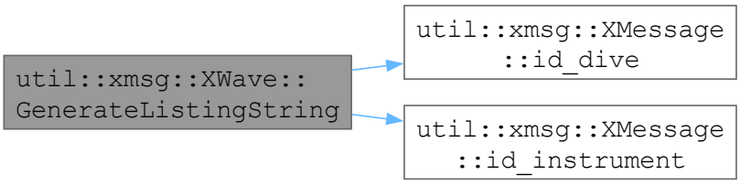
Generate a listing entry string from this ocean surface wave-field data message.

Returns

A listing entry string from this ocean surface wave-field data message.

References assf_, CATCH_BLOCK, EXC_THROW, util::xmsg::XMessage::id_dive(), util::xmsg::XMessage::id_instrument(), sample_mode_, sample_rate_base_, sample_rate_div_, sample_rate_effective_, wave_data_spectral_, util::xmsg::XWAVE_MODE_RAW and util::xmsg::XWAVE_MODE_SPECTRAL.

Here is the call graph for this function:



10.115.3.5 id_dive()

```
int16_t util::xmsg::XMessage::id_dive () const  [inline], [inherited]
```

Get the dive or cycle number of this X-Message.

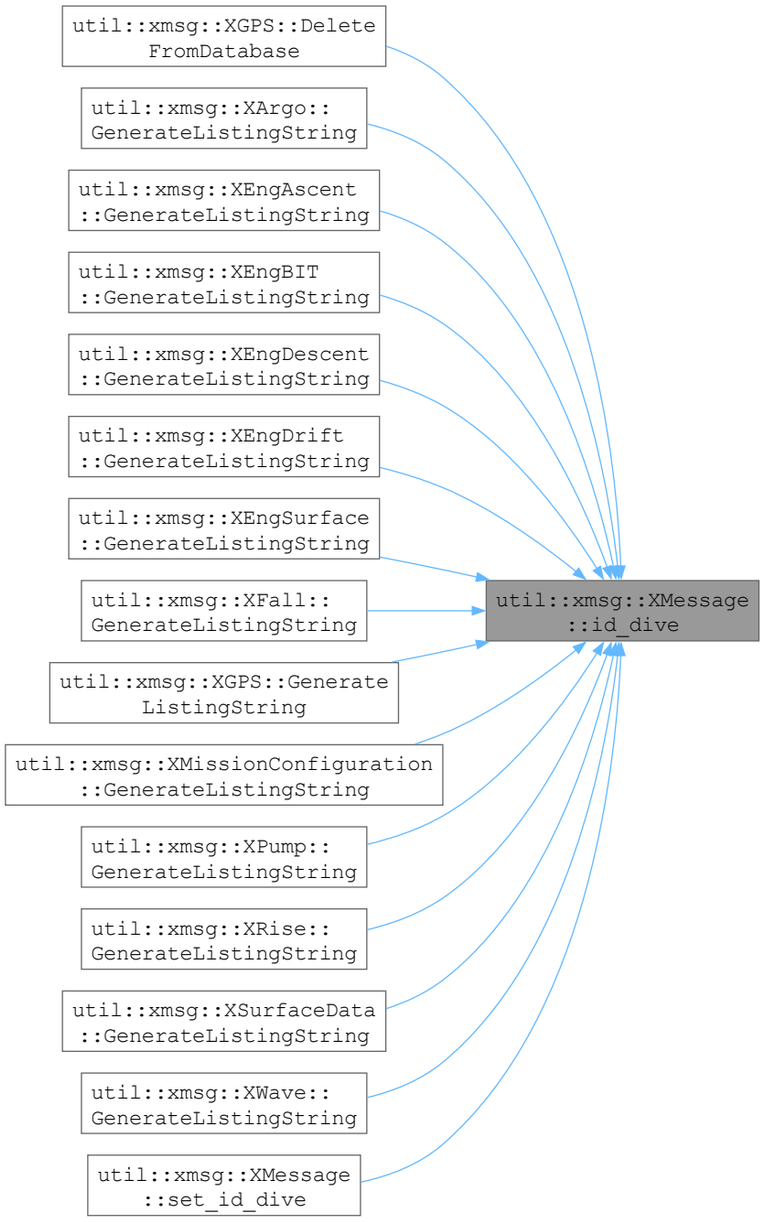
Returns

The dive or cycle number of this X-Message.

References util::xmsg::XMessage::id_dive_.

Referenced by util::xmsg::XGPS::DeleteFromDatabase(), util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateLis
util::xmsg::XEngBIT::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingStr
util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(),
util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing
util::xmsg::XSurfaceData::GenerateListingString(), GenerateListingString(), and util::xmsg::XMessage::set_id_dive().

Here is the caller graph for this function:



10.115.3.6 id_instrument()

```
uint16_t util::xmsg::XMessage::id_instrument () const  [inline], [inherited]
```

Get the unique instrument identifier of this X-Message.

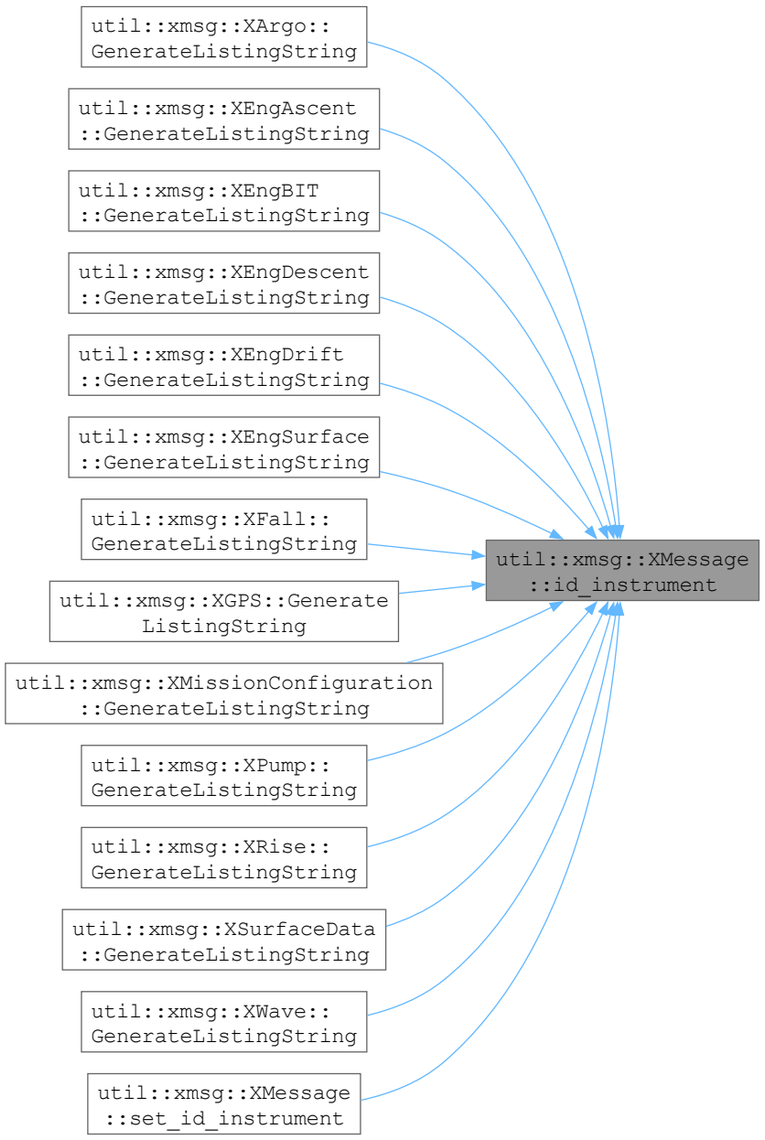
Returns

The unique instrument identifier of this X-Message.

References util::xmsg::XMessage::id_instrument_.

Referenced by util::xmsg::XArgo::GenerateListingString(), util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngBIT::Generate
util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), util::xmsg::XEngSurface::GenerateListin
util::xmsg::XFall::GenerateListingString(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingS
util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListingString(), util::xmsg::XSurfaceData::GenerateListingString(),
GenerateListingString(), and util::xmsg::XMessage::set_id_instrument().

Here is the caller graph for this function:



10.115.3.7 id_xmsg_type()

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type () const  [inline], [inherited]
```

Get the enumerated X-Message type identifier.

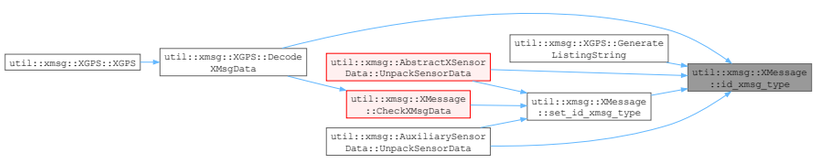
Returns

The enumerated X-Message type identifier.

References util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMessage::set_id_xmsg_typ
util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the caller graph for this function:



10.115.3.8 set_id_dive()

```
void util::xmsg::XMessage::set_id_dive (
            int16_t id_dive)  [inline], [protected], [inherited]
```

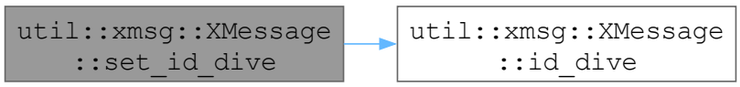
Set the dive or cycle number of this message.

Parameters

in	*id_dive*	The dive or cycle number of this message to set.

References util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::id_dive_.

Here is the call graph for this function:



10.115.3.9 set_id_instrument()

```
void util::xmsg::XMessage::set_id_instrument (
            uint16_t id_instrument)  [inline], [protected], [inherited]
```

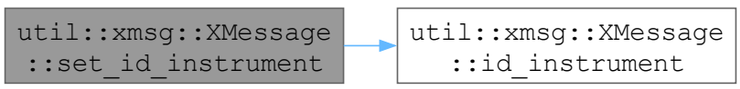
Set the unique instrument identifier.

Parameters

in	*id_instrument*	The unique instrument identifier to set.

References util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::id_instrument_.

Here is the call graph for this function:



10.115.3.10 set_id_xmsg_type()

```
void util::xmsg::XMessage::set_id_xmsg_type (
            XMsgTypeId_t id_xmsg_type)  [inline], [protected], [inherited]
```

Set the X-Message type identifier.

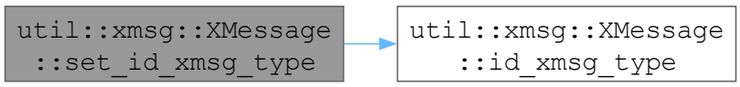
Parameters

in	*id_xmsg_type*	The X-Message type identifier to set.

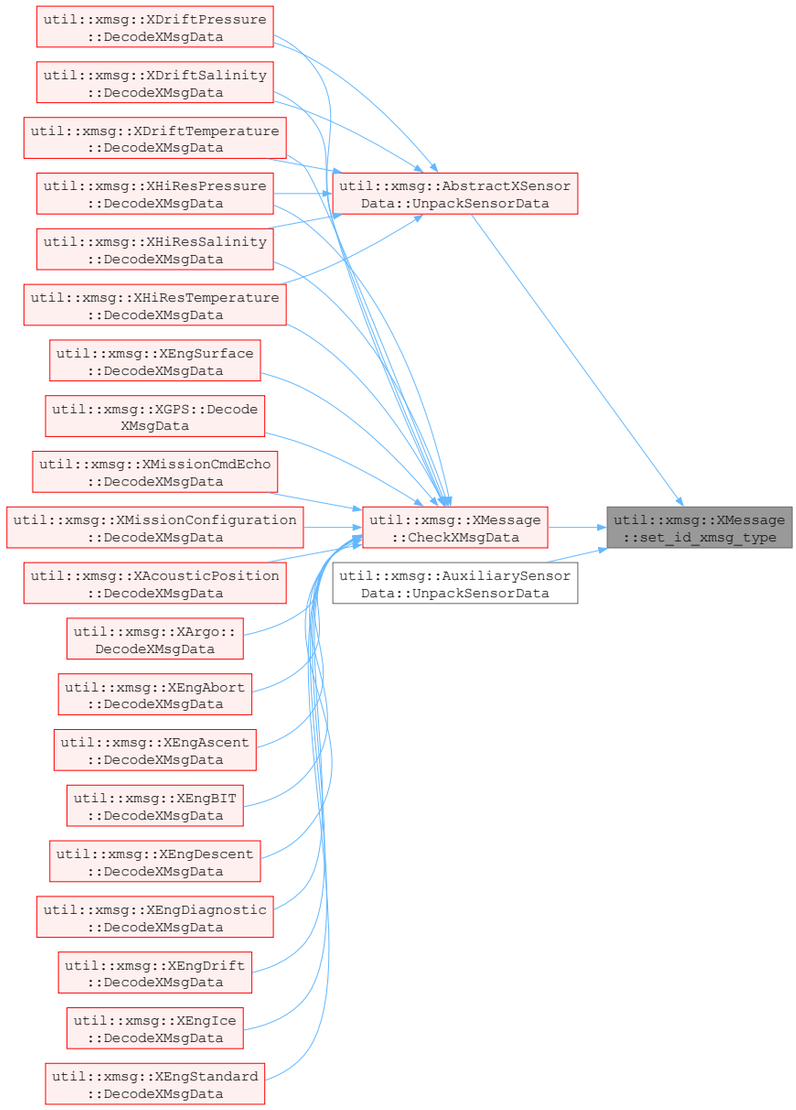
References util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::id_xmsg_type_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:



Here is the caller graph for this function:



10.115.3.11 set_xmsg_index()

```
void util::xmsg::XMessageIndexable::set_xmsg_index (
            uint8_t xmsg_index)  [inline], [protected], [inherited]
```

Set the index of this X-Message.

Parameters

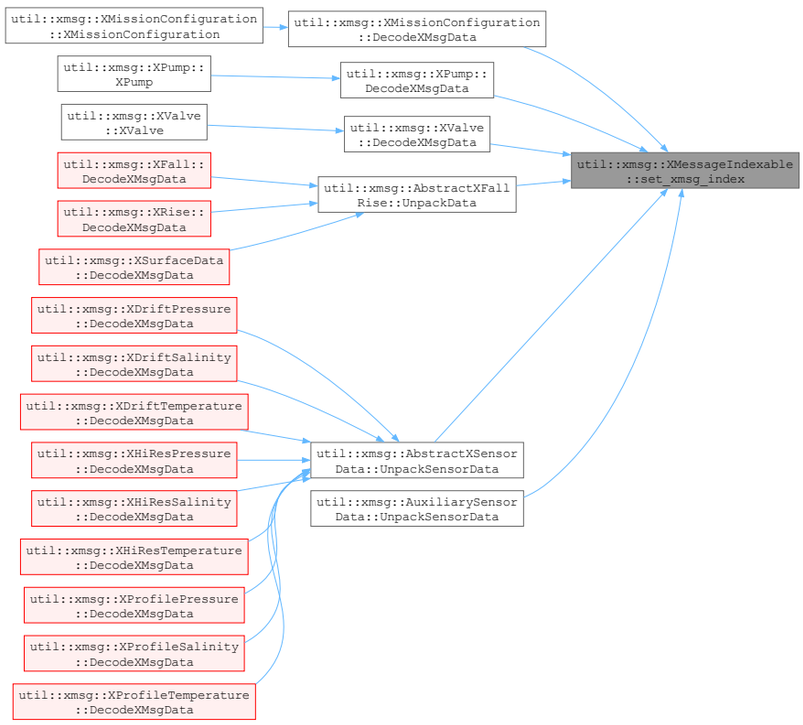
in	*xmsg_index*	The index of this X-Message.

References util::xmsg::XMessageIndexable::xmsg_index(), and util::xmsg::XMessageIndexable::xmsg_index_.

Referenced by util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(), util::xmsg::XValve::DecodeXMsgData(), util::xmsg::AbstractXFallRise::UnpackData(), util::xmsg::AbstractXSensorData::UnpackSensorDa and util::xmsg::AuxiliarySensorData::UnpackSensorData().

Here is the call graph for this function:

Here is the caller graph for this function:



10.115.3.12 set_xmsg_size_bytes()

```
void util::xmsg::XMessage::set_xmsg_size_bytes (
            uint16_t xmsg_size_bytes)  [inline], [protected], [inherited]
```

Set the size of the X-Message, in bytes.

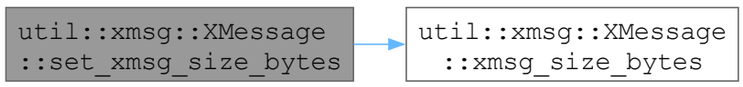
Parameters

in	*xmsg_size_bytes*	The size of the X-Message to set, in bytes.

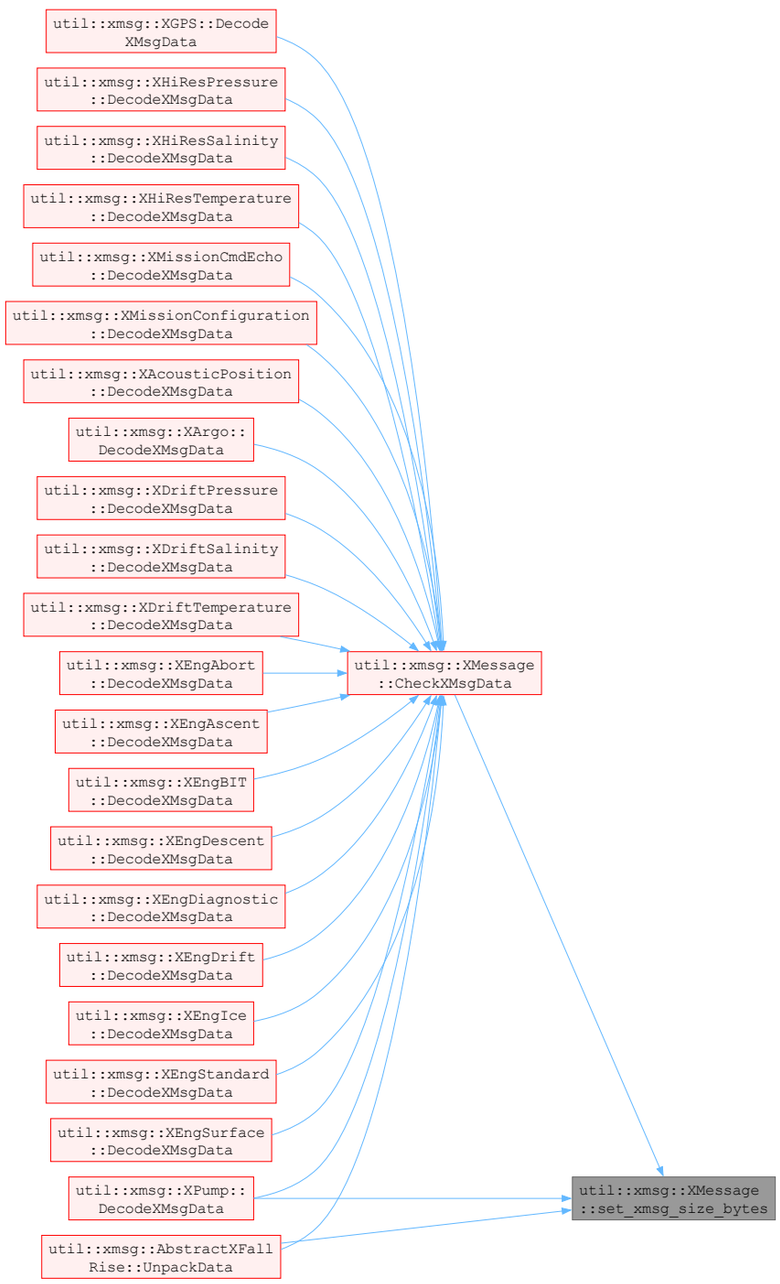
References util::xmsg::XMessage::xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XPump::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::Unpa

Here is the call graph for this function:



Here is the caller graph for this function:



10.115.3.13 xmsg_index()

```
uint8_t util::xmsg::XMessageIndexable::xmsg_index () const  [inline], [inherited]
```

Get the index of this X-Message.

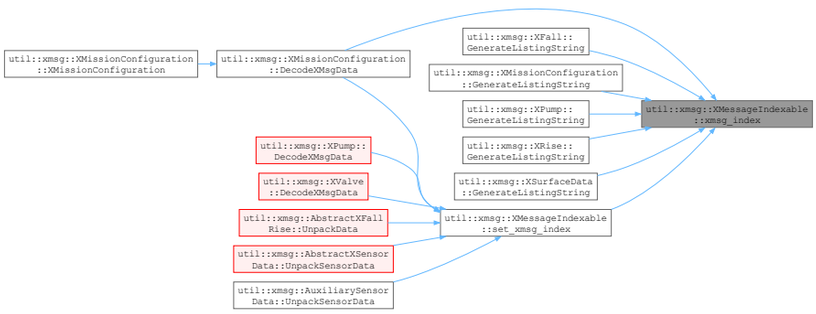
Returns

The index of this X-Message.

References util::xmsg::XMessageIndexable::xmsg_index_.

Referenced by util::xmsg::XMissionConfiguration::DecodeXMsgData(), util::xmsg::XFall::GenerateListingString(), util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XRise::GenerateListing util::xmsg::XSurfaceData::GenerateListingString(), and util::xmsg::XMessageIndexable::set_xmsg_index().

Here is the caller graph for this function:



10.115.3.14 xmsg_size_bytes()

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes () const  [inline], [inherited]
```

Get the size of the X-Message, in bytes.

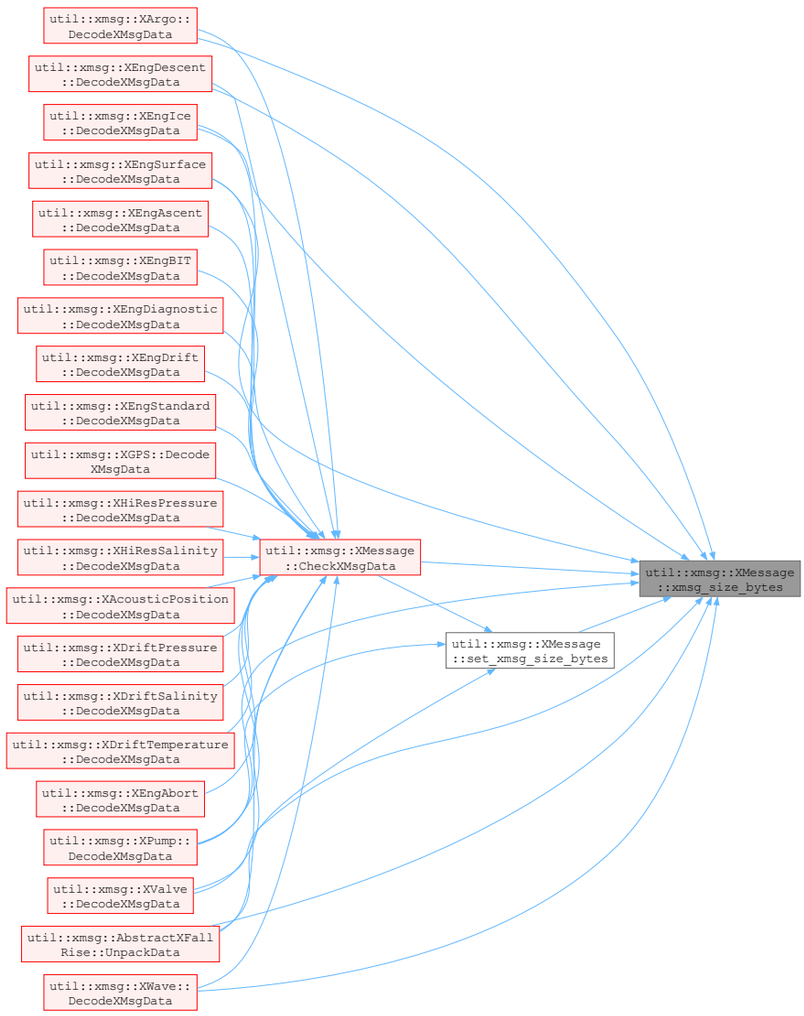
Returns

The size of the X-Message, in bytes.

References util::xmsg::XMessage::xmsg_size_bytes_.

Referenced by util::xmsg::XMessage::CheckXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgI util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(), util::xmsg::XValve::DecodeXMsgData(), DecodeXMsgData(), util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::AbstractXFallRise::UnpackData().

Here is the caller graph for this function:



10.115.4 Member Data Documentation

10.115.4.1 assf_

```
unsigned int util::xmsg::XWave::assf_  [private]
```

The accelerometer sensitivity scale factor, in LSB/g.

Note

Dividing accelerometer counts by this number produces this measured acceleration, in g.

Referenced by DecodeXMsgData(), and GenerateListingString().

10.115.4.2 ASSF_RAW

```
const unsigned int util::xmsg::XWave::ASSF_RAW = { 16384, 8192, 4096, 2048 }  [static], [private]
```

Available accelerometer sensitivity scale factors for raw wave data.

10.115.4.3 ASSF_SPECTRAL_V1

```
const unsigned int util::xmsg::XWave::ASSF_SPECTRAL_V1 = { 8192, 2048, 512, 128 }  [static], [private]
```

Available accelerometer sensitivity scale factors for spectral wave data – message version 1.

Referenced by DecodeXMsgData().

10.115.4.4 ASSF_SPECTRAL_V2

```
const unsigned int util::xmsg::XWave::ASSF_SPECTRAL_V2 = { 8192, 2048, 512, 128 }  [static], [private]
```

Available accelerometer sensitivity scale factors for spectral wave data – message version 2.

Referenced by DecodeXMsgData().

10.115.4.5 FSAM

```
const double util::xmsg::XWave::FSAM = { 1 }  [static], [private]
```

Available ample rate frequencies.

10.115.4.6 gain_accel_q3_

```
uint16_t util::xmsg::XWave::gain_accel_q3_  [private]
```

Accelerometer software gain.

Referenced by DecodeXMsgData().

10.115.4.7 GSSF

```
const double util::xmsg::XWave::GSSF = { 131.072, 65.536, 32.768, 16.384 }  [static], [private]
```

Available gyrscope sensitivity scale factors.

10.115.4.8 id_dive_

```
int16_t util::xmsg::XMessage::id_dive_  [private], [inherited]
```

The dive or cycle number of this message.

Referenced by util::xmsg::XMessage::id_dive(), and util::xmsg::XMessage::set_id_dive().

10.115.4.9 id_instrument_

```
uint16_t util::xmsg::XMessage::id_instrument_  [private], [inherited]
```

The unique WHOI identifier of this instrument.

Referenced by util::xmsg::XMessage::id_instrument(), and util::xmsg::XMessage::set_id_instrument().

10.115.4.10 id_xmsg_type_

```
XMsgTypeId_t util::xmsg::XMessage::id_xmsg_type_  [private], [inherited]
```

The X-message type identifier.

Referenced by util::xmsg::XMessage::id_xmsg_type(), and util::xmsg::XMessage::set_id_xmsg_type().

10.115.4.11 MSSF

```
const double util::xmsg::XWave::MSSF = { (30.0 / 9.0) }  [static], [private]
```

Available magenetometer sensitivity scale factors.

10.115.4.12 sample_mode_

```
XWaveMode_t util::xmsg::XWave::sample_mode_  [private]
```

The ocean surface wave field sampling mode.

This field indicates whether this message contains raw wave motion measurements or spectrally encoded data.

Referenced by DecodeXMsgData(), and GenerateListingString().

10.115.4.13 sample_rate_base_

```
uint16_t util::xmsg::XWave::sample_rate_base_  [private]
```

The base sample rate, in Hz.

Referenced by DecodeXMsgData(), FixFuckup9000(), and GenerateListingString().

10.115.4.14 sample_rate_div_

```
uint16_t util::xmsg::XWave::sample_rate_div_  [private]
```

The sample rate divider, in Hz/Hz.

Referenced by DecodeXMsgData(), FixFuckup9000(), and GenerateListingString().

10.115.4.15 sample_rate_effective_

```
float util::xmsg::XWave::sample_rate_effective_  [private]
```

The sample rate of the ocean surface wave measurements, in Hz.

Referenced by DecodeXMsgData(), FixFuckup9000(), and GenerateListingString().

10.115.4.16 segment_len_

```
uint16_t util::xmsg::XWave::segment_len_  [private]
```

The power-of-two segment length for spectral data.

10.115.4.17 segment_len_pow_

```
uint16_t util::xmsg::XWave::segment_len_pow_  [private]
```

The power-of-two segment length for spectral data.

10.115.4.18 wave_data_spectral_

```
vector<XWaveSpectralSample> util::xmsg::XWave::wave_data_spectral_  [private]
```

The spectral data samples.

Referenced by DecodeXMsgData(), FixFuckup9000(), and GenerateListingString().

10.115.4.19 xmsg_index_

```
uint8_t util::xmsg::XMessageIndexable::xmsg_index_  [private], [inherited]
```

The index of this X-Message.

Referenced by util::xmsg::XMessageIndexable::set_xmsg_index(), and util::xmsg::XMessageIndexable::xmsg_index().

10.115.4.20 xmsg_size_bytes_

```
uint16_t util::xmsg::XMessage::xmsg_size_bytes_  [private], [inherited]
```

The size of the X message, in bytes.

Referenced by util::xmsg::XMessage::set_xmsg_size_bytes(), and util::xmsg::XMessage::xmsg_size_bytes().

The documentation for this class was generated from the following files:

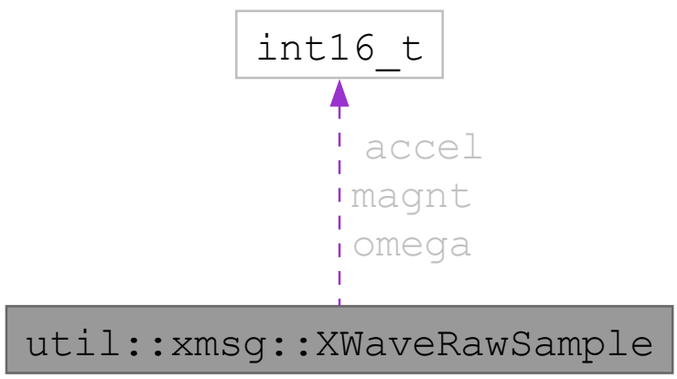
- data/xmsg/XWave.h
- data/xmsg/XWave.cpp

10.116 util::xmsg::XWaveRawSample Class Reference

Class representing a single raw motion measurement in the surface time series wave data X-message.

```
#include <XWaveRawSample.h>
```

Collaboration diagram for util::xmsg::XWaveRawSample:



Public Member Functions

- XWaveRawSample () ATTR_NO_EXCEPTIONS
  Default constructor.
- ~XWaveRawSample ()
  Destructor.
- const string GenerateListingString (double assf, double gssf, double mssf) const
  Generate a listing entry string representation of this raw motion measurement.

Private Attributes

- int16_t accel [3]
  Measured acceleration in the starboard (x'), bow (y'), and upward (z') directions.
- int16_t magnt [3]
  Measured magnetism in the starboard (x'), bow (y'), and upward (z') directions.
- int16_t omega [3]
  Measured rotational velocity in the gamma (starboard-up), beta (bow-up), and alpha (increasing azimuth) directions.

Friends

- class **XWave**

10.116.1 Detailed Description

Class representing a single raw motion measurement in the surface time series wave data X-message.

Note

This class is used for raw data mode only.

10.116.2 Constructor & Destructor Documentation

10.116.2.1 XWaveRawSample()

```
util::xmsg::XWaveRawSample::XWaveRawSample ()  [inline]
```

Default constructor.

10.116.2.2 ~XWaveRawSample()

```
util::xmsg::XWaveRawSample::~XWaveRawSample ()  [inline]
```

Destructor.

10.116.3 Member Function Documentation

10.116.3.1 GenerateListingString()

```
const string util::xmsg::XWaveRawSample::GenerateListingString (
            double assf,
            double gssf,
            double mssf) const  [inline]
```

Generate a listing entry string representation of this raw motion measurement.

Parameters

in	*assf*	The accelerometer sensitivity scale factor, in LSB/g.
in	*gssf*	The gyroscope sensitivity scale factor, in LSB/(deg/s).
in	*mssf*	The magnetometer sensitivity scale factor, in LSB/uT.

Returns

A listing entry string representation of this raw motion measurement.

References accel, CATCH_BLOCK, magnt, and omega.

10.116.4 Member Data Documentation

10.116.4.1 accel

```
int16_t util::xmsg::XWaveRawSample::accel[3]  [private]
```

Measured acceleration in the starboard (x'), bow (y'), and upward (z') directions.

Referenced by GenerateListingString().

10.116.4.2 magnt

```
int16_t util::xmsg::XWaveRawSample::magnt[3]  [private]
```

Measured magnetism in the starboard (x'), bow (y'), and upward (z') directions.

Referenced by GenerateListingString().

10.116.4.3 omega

```
int16_t util::xmsg::XWaveRawSample::omega[3]  [private]
```

Measured rotational velocity in the gamma (starboard-up), beta (bow-up), and alpha (increasing azimuth) directions.

Referenced by GenerateListingString().

The documentation for this class was generated from the following file:

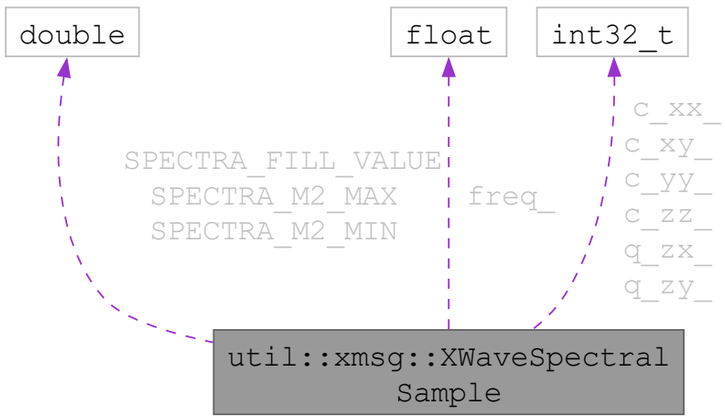
- data/xmsg/common/XWaveRawSample.h

10.117 util::xmsg::XWaveSpectralSample Class Reference

The wave spectral sample class.

```
#include <XWaveSpectralSample.h>
```

Collaboration diagram for util::xmsg::XWaveSpectralSample:



Protected Member Functions

- template<typename T = double>
  T **c_xx_g2hz** (T scale) const
- template<typename T = double>
  T **c_xx_m2hz** (T scale, T freq) const
- template<typename T = double>
  T **c_xy_g2hz** (T scale) const
- template<typename T = double>
  T **c_xy_m2hz** (T scale, T freq) const
- template<typename T = double>
  T **c_yy_g2hz** (T scale) const
- template<typename T = double>
  T **c_yy_m2hz** (T scale, T freq) const
- template<typename T = double>
  T **c_zz_g2hz** (T scale) const
- template<typename T = double>
  T **c_zz_m2hz** (T scale, T freq) const
- template<typename T = double>
  T **q_zx_g2hz** (T scale) const
- template<typename T = double>
  T **q_zx_m2hz** (T scale, T freq) const
- template<typename T = double>
  T **q_zy_g2hz** (T scale) const
- template<typename T = double>
  T **q_zy_m2hz** (T scale, T freq) const

Private Attributes

- int32_t **c_xx_**
- int32_t **c_xy_**
- int32_t **c_yy_**
- int32_t **c_zz_**
- float **freq_**
- int32_t **q_zx_**
- int32_t **q_zy_**

Static Private Attributes

- static const double **SPECTRA_FILL_VALUE** = -999.99999
- static const double **SPECTRA_M2_MAX** = 999.9999
- static const double **SPECTRA_M2_MIN** = -999.9999

Friends

- class **XWave**

10.117.1 Detailed Description

The wave spectral sample class.

An instance of this class represents the acceleration spectra and co-/quadrature spectra at a single frequency, decoded from a wave X-message (see XWave).

The documentation for this class was generated from the following files:

- data/xmsg/common/XWaveSpectralSample.h
- data/xmsg/common/XWaveSpectralSample.cpp

Chapter 11

File Documentation

11.1 AdditionalDocumentation.h File Reference

Additional Doxygen documentation.

Classes

- class boost::container::flat_map< C, D >

  Dummy template class used for configuring Doxygen to interpret boost::container::flat_map correctly.
- class boost::shared_ptr< A >

  Dummy template class used for configuring Doxygen to interpret boost::shared_ptr objects as plain pointers.
- class vmime::shared_ptr< F >

  Dummy template class used for configuring Doxygen to interpret vmime::shared_ptr objects as plain pointers.
- class boost::container::static_vector< E >

  Dummy template class used for configuring Doxygen to interpret boost::container::static_vector correctly.
- class boost::thread_specific_ptr< B >

  Dummy template class used for configuring Doxygen to interpret boost::thread_specific_ptr objects as plain pointers.

Namespaces

- namespace argo

  The Argo instrument namespace.
- namespace argo::iridium

  The Argo Iridium instrument namespace.
- namespace argo::iridium::xfloat

  The X-Float instrument namespace.
- namespace boost

  The Boost C++ library namespace.
- namespace boost::container

  Boost container library namespace.
- namespace util

  The utility namespace.
- namespace util::config

  The configuration utility namespace.
- namespace util::date

  Date utility namespace.
- namespace util::file

  The filesystem utility namespace.
- namespace util::ftp

  The ftp utility namespace.
- namespace util::log

  The logging utility namespace.
- namespace util::mail

  The mail utility namespace.
- namespace util::math

  The math utility namespace.

- namespace util::mysql

  The MySQL utility namespace.
- namespace util::position

  The position utility namespace.
- namespace util::sig

  The system signals utility namespace.
- namespace util::stream

  The stream utility namespace.
- namespace util::string

  The string utility namespace.
- namespace util::sys

  The system utility namespace.
- namespace util::thread

  The thread utility namespace.
- namespace util::types

  The types utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.
- namespace vmime

  VMime library namespace.

11.1.1 Detailed Description

Additional Doxygen documentation.

Version

0.6

Date

Apr 4, 2014 10:46:35 AM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

Note

This document is used for generating Doxygen documentation of namespaces and should not be included in the compiled project.

This file defines Doxygen annotations for documenting elements that are otherwise undocumented.

11.2 AdditionalDocumentation.h

Go to the documentation of this file.

```
00001 /** Additional Doxygen documentation.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Apr 4, 2014 10:46:35 AM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  *
00022  * @note        This document is used for generating Doxygen documentation of namespaces and should
00023  *              not be included in the compiled project.
00024  *
00025  * This file defines Doxygen annotations for documenting elements that are otherwise undocumented.
00026  *
00027  *
00028  */
00029
00030 #ifndef NAMESPACEDEFS_H_
00031 #define NAMESPACEDEFS_H_
00032
00033 #error  "This file is used for documentation purposes only. It should not be included from the project."
00034
00035 /** The Argo instrument namespace.
00036  *
00037  */
00038 namespace argo {
00039
00040 /** The Argo Iridium instrument namespace.
00041  *
00042  */
00043 namespace iridium {
00044
00045 /** The X-Float instrument namespace.
00046  *
00047  *  @note           This namespace contains all object that represents individual X-Floats
00048  *                  (including ALAMOs and Solo-IIs), their dives and mail messages, and all
00049  *                  of the constituents that make up these objects.
00050  */
00051 namespace xfloat {
00052
00053 }
00054 }
00055 }
00056
00057 /** The Boost C++ library namespace.
00058  *
00059  */
00060 namespace boost {
00061
00062 /** Dummy template class used for configuring Doxygen to interpret boost::shared_ptr objects as
00063  *  plain pointers.
00064  */
00065 template<class A> class shared_ptr {
00066
00067     /** The pointer to the element.
00068      *
00069      */
00070     A * element;
00071 };
00072
00073 /** Dummy template class used for configuring Doxygen to interpret boost::thread_specific_ptr
00074  *  objects as plain pointers.
00075  */
00076 template<class B> class thread_specific_ptr {
00077
00078     /** The pointer to the element.
00079      *
00080      */
00081     B * element;
00082 };
00083
00084 /** Boost container library namespace.
00085  *
00086  */
00087 namespace container {
00088
00089 /** Dummy template class used for configuring Doxygen to interpret
00090  *  boost::container::flat_map correctly.
00091
00092  *
00093  */
00094 template<class C, class D> class flat_map {
00095
00096     /** The index.*/
00097     C index[];
00098
00099     /** The elements.
00100      *
00101      */
00102     D elements[];
00103 };
```

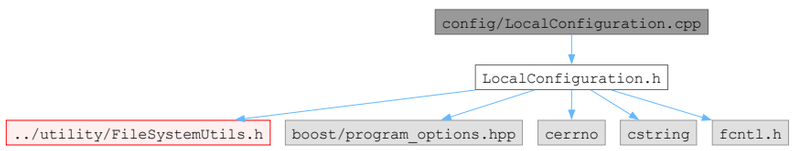
```
00104
00105 /** Dummy template class used for configuring Doxygen to interpret
00106  *  boost::container::static_vector correctly.
00107
00108  *
00109  */
00110 template<class E> class static_vector {
00111
00112     /** The elements.
00113      *
00114      */
00115     E elements[];
00116 };
00117 }
00118 }
00119
00120 /** The utility namespace.
00121  *
00122  */
00123 namespace util {
00124
00125 /** The configuration utility namespace.
00126  *
00127
00128  *
00129  */
00130 namespace config {
00131
00132 }
00133 /** Date utility namespace.
00134  *
00135  */
00136
00137 namespace date {
00138
00139 }
00140
00141 /** The filesystem utility namespace.
00142  *
00143  */
00144
00145 namespace file {
00146
00147 }
00148
00149 /** The ftp utility namespace.
00150  *
00151  *  @deprecated         FTP client integration features have been scrapped. This namespace is
00152  *                      no longer utilized.
00153  */
00154 namespace ftp {
00155
00156 }
00157
00158 /** The logging utility namespace.
00159  *
00160  */
00161
00162 namespace log {
00163
00164 }
00165
00166 /** The mail utility namespace.
00167  *
00168  */
00169 namespace mail {
00170
00171 }
00172
00173 /** The math utility namespace.
00174  *
00175  */
00176 namespace math {
00177
00178 }
00179
00180 /** The MySQL  utility namespace.
00181  *
00182  */
00183 namespace mysql {
00184
00185 }
00186
00187 /** The position utility namespace.
00188  *
00189  */
00190 namespace position {
00191
00192 }
00193
00194 /** The system signals utility namespace.
00195  *
00196  */
00197 namespace sig {
00198
00199 }
00200
00201 /** The string utility namespace.
00202  *
00203  */
00204 namespace string {
00205
00206 }
00207
00208 /** The stream utility namespace
00209  *
00210  */
00211 namespace stream {
```

```
00212
00213 }
00214
00215 /** The system utility namespace
00216  *
00217  */
00218 namespace sys {
00219
00220 }
00221
00222 /** The thread utility namespace.
00223  *
00224  */
00225 namespace thread {
00226
00227 }
00228
00229 /** The types utility namespace.
00230  *
00231  */
00232 namespace types {
00233
00234 }
00235
00236 /** The X-Message utility namespace.
00237  *
00238  */
00239 namespace xmsg {
00240
00241 }
00242 }
00243
00244 /** VMime library namespace.
00245  *
00246  */
00247 namespace vmime {
00248
00249 /** Dummy template class used for configuring Doxygen to interpret vmime::shared_ptr objects as
00250  *  plain pointers.
00251
00252  *
00253  */
00254 template<class F> class shared_ptr {
00255
00256     /** The pointer to the element.
00257      *
00258      */
00259     F * element;
00260 };
00261 }
00262
00263 #endif /* NAMESPACEDEFS_H_
00264      *
00265      */
```

11.3 config/LocalConfiguration.cpp File Reference

The util::config::LocalConfiguration class source file.

`#include "LocalConfiguration.h"`
Include dependency graph for LocalConfiguration.cpp:



Namespaces

- namespace util

  The utility namespace.
- namespace util::config

  The configuration utility namespace.

11.3.1 Detailed Description

The util::config::LocalConfiguration class source file.

Version

0.6

Date

May 12, 2014 4:32:16 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

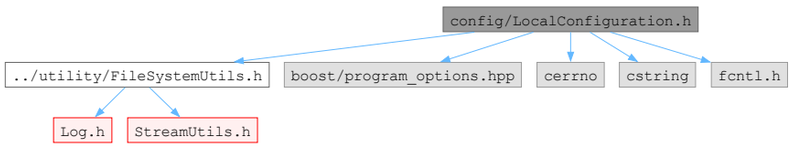
This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.4 config/LocalConfiguration.h File Reference

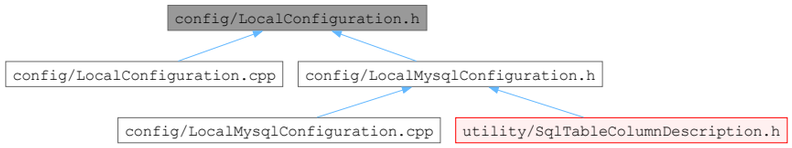
The util::config::LocalConfiguration class header file.

```
#include "../utility/FileSystemUtils.h"
#include <boost/program_options.hpp>
#include <cerrno>
#include <cstring>
#include <fcntl.h>
```

Include dependency graph for LocalConfiguration.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::config::LocalConfiguration

  Class representing all program run-time configuration options.

Namespaces

- namespace util

  The utility namespace.
- namespace util::config

  The configuration utility namespace.

Macros

- #define CFG_DEFAULT_ISDAEMON (false)

  The default value for daemon/service mode operation, if unspecified in the configuration settings.
- #define CFG_DEFAULT_MAIL_CHK_DLY (60)

  The default configuration file path to use, if not specified in the command line arguments.
- #define CFG_DEFAULT_MAX_FETCH (127)

  The default number of mail messages to fetch concurrently, if unspecified in the configuration settings.
- #define CFG_DEFAULT_MIN_UID (0)

  The placeholder value for an unspecified starting mail message UID.
- #define CFG_DEFAULT_NDECO (1)

  The default number of telemetry decoder threads to instantiate, if unspecified in the configuration settings.
- #define CFG_DEFAULT_NPROC (1)

  The default number of mail message processing threads to instantiate, if unspecified in the configuration settings.
- #define CFG_DEFAULT_PATH "../cfg/options.cfg"

  The default configuration file path to use, if not specified in the command line arguments.
- #define CFG_DEFAULT_PATH_PID "/var/lock/" PROG_NAME "/lock.pid"

  The default path to the PID file.
- #define **CFG_DEFAULT_REPROCESS_ACTIVE** (true)
- #define CFG_DEFAULT_REPROCESS_ALL (false)

  The default value for the reprocess-all boolean switch, if unspecified in the configuration settings.
- #define CFG_FMT_DESC_LEN (85)

  The configuration option description minimum line length.
- #define CFG_FMT_LINE_LEN (100)

  The configuration option line length for printing to standard output.
- #define PROG_DESC "X-Message Mail-Telemetry Processing Service"

  Brief program description.
- #define PROG_ID "=== " PROG_NAME " v" PROG_VERS " - " PROG_DESC " ==="

  Program identifier string.
- #define PROG_MAJOR_VERS 0

  Program major version.
- #define PROG_MINOR_VERS 6

  Program version.
- #define PROG_NAME "xmtps"

  Program name.
- #define PROG_VERS

  Program version.
- #define PROG_XVERS 2.2

  Supported X-Message version.

11.4.1 Detailed Description

The util::config::LocalConfiguration class header file.

Version

0.6

Date

May 12, 2014 4:32:16 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.4.2 Macro Definition Documentation

11.4.2.1 CFG_DEFAULT_ISDAEMON

```
#define CFG_DEFAULT_ISDAEMON (false)
```

The default value for daemon/service mode operation, if unspecified in the configuration settings.

Referenced by util::config::LocalConfiguration::GenerateCommandLineOptionsDescription().

11.4.2.2 CFG_DEFAULT_MAIL_CHK_DLY

```
#define CFG_DEFAULT_MAIL_CHK_DLY (60)
```

The default configuration file path to use, if not specified in the command line arguments.

Referenced by util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription().

11.4.2.3 CFG_DEFAULT_MAX_FETCH

```
#define CFG_DEFAULT_MAX_FETCH (127)
```

The default number of mail messages to fetch concurrently, if unspecified in the configuration settings.

Referenced by util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription().

11.4.2.4 CFG_DEFAULT_MIN_UID

```
#define CFG_DEFAULT_MIN_UID (0)
```

The placeholder value for an unspecified starting mail message UID.

11.4.2.5 CFG_DEFAULT_NDECO

```
#define CFG_DEFAULT_NDECO (1)
```

The default number of telemetry decoder threads to instantiate, if unspecified in the configuration settings.

Referenced by util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription().

11.4.2.6 CFG_DEFAULT_NPROC

```
#define CFG_DEFAULT_NPROC (1)
```

The default number of mail message processing threads to instantiate, if unspecified in the configuration settings.

Referenced by util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription().

11.4.2.7 CFG_DEFAULT_PATH

```
#define CFG_DEFAULT_PATH "../cfg/options.cfg"
```

The default configuration file path to use, if not specified in the command line arguments.

Referenced by util::config::LocalConfiguration::GenerateCommandLineOptionsDescription().

11.4.2.8 CFG_DEFAULT_PATH_PID

```
#define CFG_DEFAULT_PATH_PID "/var/lock/" PROG_NAME "/lock.pid"
```

The default path to the PID file.

Referenced by util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription().

11.4.2.9 CFG_DEFAULT_REPROCESS_ALL

```
#define CFG_DEFAULT_REPROCESS_ALL (false)
```

The default value for the reprocess-all boolean switch, if unspecified in the configuration settings.

Referenced by util::config::LocalConfiguration::GenerateCommandLineOptionsDescription().

11.4.2.10 CFG_FMT_DESC_LEN

```
#define CFG_FMT_DESC_LEN (85)
```

The configuration option description minimum line length.

Referenced by util::config::LocalConfiguration::GenerateCommandLineOptionsDescription(), and util::config::LocalConfiguration::Generat

11.4.2.11 CFG_FMT_LINE_LEN

```
#define CFG_FMT_LINE_LEN (100)
```

The configuration option line length for printing to standard output.

Referenced by util::config::LocalConfiguration::GenerateCommandLineOptionsDescription(), and util::config::LocalConfiguration::Generat

11.4.2.12 PROG_DESC

```
#define PROG_DESC "X-Message Mail-Telemetry Processing Service"
```

Brief program description.

11.4.2.13 PROG_ID

```
#define PROG_ID "=== " PROG_NAME " v" PROG_VERS " - " PROG_DESC " ==="
```

Program identifier string.

11.4.2.14 PROG_MAJOR_VERS

```
#define PROG_MAJOR_VERS 0
```

Program major version.

11.4.2.15 PROG_MINOR_VERS

```
#define PROG_MINOR_VERS 6
```

Program version.

11.4.2.16 PROG_NAME

```
#define PROG_NAME "xmtps"
```

Program name.

11.4.2.17 PROG_VERS

```
#define PROG_VERS
```

Value:

```
PP_STR(PROG_MAJOR_VERS) "." PP_STR(PROG_MINOR_VERS) "." \
PP_STR(BUILD_NUMBER) "b"
```

Program version.

Referenced by argo::iridium::xfloat::XFloatDiveData::GenerateListingString().

11.4.2.18 PROG_XVERS

```
#define PROG_XVERS 2.2
```

Supported X-Message version.

Referenced by argo::iridium::xfloat::XFloatDiveData::GenerateListingString().

11.5 LocalConfiguration.h

Go to the documentation of this file.

```
00001 /** The @ref util::config::LocalConfiguration class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       May 12, 2014 4:32:16 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef LOCALCONFIGURATION_H_
00024 #define LOCALCONFIGURATION_H_
00025
00026 /* Includes */
00027
00028 // local
00029 #include "../utility/FileSystemUtils.h"
00030
00031 // boost
00032 #include <boost/program_options.hpp>
00033 namespace po = boost::program_options;
00034
00035 // system libraries
00036 #include <cerrno>
00037 #include <cstring>
00038 #include <fcntl.h>
00039
00040 /* Definitions */
00041
00042 #if !defined(BUILD_NUMBER)
00043
00044 #error "BUILD_NUMBER is not defined!"
00045
00046 #endif // !defined(BUILD_NUMBER)
00047
00048 #if !defined(BUILD_DATETIME)
00049
00050 #error "BUILD_DATETIME is not defined!"
00051
00052 #endif // !defined(BUILD_DATETIME)
00053
00054 /** Program name. */
00055 #define PROG_NAME                       "xmtps"
00056
```

```
00057 /** Supported X-Message version. */
00058 #define PROG_XVERS                      2.2
00059
00060 /** Program major version. */
00061 #define PROG_MAJOR_VERS                 0
00062
00063 /** Program version. */
00064 #define PROG_MINOR_VERS                 6
00065
00066 #if defined(BUILD_DEBUG) && BUILD_DEBUG
00067
00068 /** Program version. */
00069 #   define PROG_VERS                   PP_STR(PROG_MAJOR_VERS) "." PP_STR(PROG_MINOR_VERS) "." \
00070                                         PP_STR(BUILD_NUMBER) "a"
00071
00072 #else
00073
00074 /** Program version. */
00075 #   define PROG_VERS                   PP_STR(PROG_MAJOR_VERS) "." PP_STR(PROG_MINOR_VERS) "." \
00076                                         PP_STR(BUILD_NUMBER) "b"
00077
00078 #endif // defined(BUILD_DEBUG) && BUILD_DEBUG
00079
00080 /** Brief program description. */
00081 #define PROG_DESC                       "X-Message Mail-Telemetry Processing Service"
00082
00083 /** Program identifier string. */
00084 #define PROG_ID                         "=== " PROG_NAME " v" PROG_VERS " - " PROG_DESC " ==="
00085
00086 /** The default value for daemon/service mode operation, if unspecified in the configuration
00087  *  settings.
00088  */
00089 #define CFG_DEFAULT_ISDAEMON            (false)
00090
00091 /** The default configuration file path to use, if not specified in the command line arguments. */
00092 #define CFG_DEFAULT_PATH                "../cfg/options.cfg"
00093
00094 /** The default path to the PID file. */
00095 #define CFG_DEFAULT_PATH_PID            "/var/lock/" PROG_NAME "/lock.pid"
00096
00097 /** The default configuration file path to use, if not specified in the command line arguments. */
00098 #define CFG_DEFAULT_MAIL_CHK_DLY        (60)
00099
00100 /** The default number of mail messages to fetch concurrently, if unspecified in the configuration
00101  *  settings.
00102  */
00103 #define CFG_DEFAULT_MAX_FETCH           (127)
00104
00105 /** The placeholder value for an unspecified starting mail message UID */
00106 #define CFG_DEFAULT_MIN_UID             (0)
00107
00108 /** The default number of mail message processing threads to instantiate, if unspecified in the
00109  *  configuration settings.
00110  */
00111 #define CFG_DEFAULT_NPROC               (1)
00112
00113 /** The default number of telemetry decoder threads to instantiate, if unspecified in the
00114  *  configuration settings.
00115  */
00116 #define CFG_DEFAULT_NDECO               (1)
00117
00118 /** The default value for the reprocess-all boolean switch, if unspecified in the configuration
00119  *  settings.
00120  */
00121 #define CFG_DEFAULT_REPROCESS_ALL       (false)
00122
00123 #define CFG_DEFAULT_REPROCESS_ACTIVE    (true)
00124
00125 /** The configuration option line length for printing to standard output. */
00126 #define CFG_FMT_LINE_LEN                (100)
00127
00128 /** The configuration option description minimum line length. */
00129 #define CFG_FMT_DESC_LEN                (85)
00130
00131 namespace util {
00132 namespace config {
00133
00134 using boost::filesystem::exists;
00135 using boost::filesystem::ifstream;
00136 using boost::filesystem::ofstream;
00137 using boost::filesystem::path;
00138 using boost::filesystem::remove;
00139 using boost::program_options::bool_switch;
00140 using boost::program_options::notify;
00141 using boost::program_options::options_description;
00142 using boost::program_options::store;
00143 using boost::program_options::variables_map;
00144 using boost::program_options::value;
00145 using std::cout;
00146 using std::endl;
00147 using std::ostringstream;
00148 using std::string;
00149 using std::vector;
00150
00151 /* Class Definitions */
00152
00153 /** Class representing all program run-time configuration options.
00154  *
00155  *  This class defines all run-time configuration options, descriptions, and default values. The
00156  *  boost::program_options library is used parse run-time configuration options from a
00157  *  configuration file and/or Command-Line Interface (CLI) arguments.
00158  *
00159  */
00160 class LocalConfiguration:
00161         public file::FileSystemUtils {
00162
00163 private:
00164
```

```
00165     /* Data Members */
00166
00167     /** Statically-allocated global run-time configuration options object.
00168      *
00169      *  All program run-time configuration settings are stored here.
00170      *
00171      */
00172     static LocalConfiguration configuration_;
00173
00174     /** Configuration file path string. */
00175     static path configuration_file_path_;
00176
00177     /** Indicates whether the program is to run as a daemon.
00178      *
00179      *
00180      */
00181     static bool is_daemon_;
00182
00183     /** The path to the PID file on the local filesystem.
00184      *
00185      *  This PID file is used to ensure that only a single instance of the program is running
00186      *  at any time. The file contains a single ASCII integer corresponding to the PID of the
00187      *  running instance. It is created during program initialization, and is removed upon
00188      *  termination. The file is checked periodically during execution to ensure that it still
00189      *  exists and that the PID is correct.
00190      */
00191     static path pid_file_path_;
00192
00193     /** The splash string.
00194      *
00195      *  This c-string is printed to standard output upon program startup,
00196      *
00197      */
00198     static const char* SPLASH;
00199
00200     /** The listing splash string.
00201      *
00202      *  The file header for created ASCII files such as RAW mail messages, and intermediate listing
00203      *  files.
00204      */
00205     static const char* SPLASH_LIS;
00206
00207 private:
00208
00209     /* Member Functions */
00210
00211     /** Parse the run-time configuration options from the command-line arguments and/or the
00212      *  configuration file.
00213      *
00214      *  Populates the configuration options object.
00215      *
00216      *  @param[in]      argc        The number of command-line arguments.
00217      *  @param[in]      argv        The command-line argument list.
00218      *
00219      *  @throw          log::MessageException if an error occurs while parsing the command-line
00220      *                  arguments or the configuration file.
00221      */
00222     void ParseProgramOptions(int argc, char** argv) ATTR_NO_EXCEPTIONS {
00223         using util::log::Log;
00224
00225         try {
00226             cout « "Loading configuration..." « endl;
00227
00228             // Generate available options.
00229             options_description cliOptDesc = GenerateCommandLineOptionsDescription();
00230             options_description cfgOptDesc = GenerateConfigurationFileOptionsDescription();
00231             options_description allOptDesc("Available Options");
00232             allOptDesc.add(cliOptDesc);
00233             allOptDesc.add(cfgOptDesc);
00234
00235             // Parse command-line options.
00236             variables_map vm;
00237             store(parse_command_line(argc, argv, allOptDesc), vm);
00238             notify(vm);
00239
00240             /* Process configuration options. */
00241
00242             // Help text.
00243             if(vm.count("help")) {
00244                 cout « allOptDesc « endl;
00245                 exit(EXIT_SUCCESS);
00246             }
00247
00248             // Version string.
00249             if(vm.count("version")) {
00250                 exit(EXIT_SUCCESS);
00251             }
00252
00253             // Set logging level.
00254             if(vm.count("debug"))
00255                 Log::set_log_reporting_level(util::log::LOG_DBG3);
00256             else if(vm.count("verbose"))
00257                 Log::set_log_reporting_level(util::log::LOG_INF);
00258             else if(vm.count("quiet"))
00259                 Log::set_log_reporting_level(util::log::LOG_WRN);
00260             else
00261                 Log::set_log_reporting_level(DEFAULT_LOG_LEVEL);
00262
00263             // Open configuration file.
00264             if(!exists(configuration_file_path_))
00265                 EXC_THROW("Configuration file not found!");
00266             ifstream ifs_cfg(configuration_file_path_);
00267             if(!ifs_cfg)
00268                 EXC_THROW(
00269                         "Failed to read configuration file: " + configuration_file_path_.string());
00270
00271             // Parse configuration file options.
00272             store(parse_config_file(ifs_cfg, cfgOptDesc, false), vm);
```

```
00273             notify(vm);
00274             ifs_cfg.close();
00275 
00276             cout « "Done." « endl;
00277         }CATCH_BLOCK("Failed to configure program options!")
00278     }
00279 
00280 public:
00281 
00282     /* Member Functions */
00283 
00284     /** Configure the application run-time settings and acquire a run-time lock.
00285      *
00286      *  Parse the configuration options, then acquire a run-time lock. Verify the run-time lock before
00287      *  returning.
00288      *
00289      *  @param[in]      argc        The number of command-line arguments.
00290      *  @param[in]      argv        The command-line argument list.
00291      *
00292      *  @throw          log::MessageException if an error occurs while parsing the run-time
00293      *                  configuration options or acquiring the run-time lock.
00294      *
00295      *  @return         A const reference to the configuration_ object.
00296      */
00297     static const LocalConfiguration& InitializeConfiguration(int argc, char** argv) ATTR_NO_EXCEPTIONS {
00298         using util::log::Log;
00299 
00300         try {
00301             // First, print the program version string splash text.
00302             cout « endl « LocalConfiguration::SPLASH « endl;
00303 
00304             // Configure run-time options.
00305             configuration_.ParseProgramOptions(argc, argv);
00306 
00307             // Acquire run-time lock.
00308             configuration_.AcquireRuntimeLock();
00309 
00310         }CATCH_BLOCK("Failed to configure run-time options!")
00311 
00312         return configuration_;
00313     }
00314 
00315 
00316 public:
00317 
00318     /* Member Functions */
00319 
00320     /** The default constructor.
00321      *
00322      */
00323     LocalConfiguration() {
00324         /* Do nothing. */
00325     }
00326 
00327     /** The destructor.
00328      *
00329      */
00330     ~LocalConfiguration() {
00331         /* Do nothing. */
00332     }
00333 
00334     /** Acquire a run-time lock.
00335      *
00336      *  A PID file is created upon successful acquisition of a run-time lock. This PID file is used
00337      *  to ensure that only a single instance of the program is running at any time. The file contains
00338      *  a single ASCII integer corresponding to the PID of the running instance. It is created during
00339      *  program initialization, and is removed upon termination. The file is checked periodically
00340      *  during execution to ensure that it still exists and that the PID is correct.
00341      *
00342      *  @throw          util::log::MessageException if acquisition of the run-time lock fails e.g.
00343      *                  another instance of the program has already acquired a run-time lock.
00344      *
00345      *  @note           To release the run-time lock, call the releaseRuntimeLock() function.
00346      *
00347      *  @return         The PID of the process acquiring the run-time lock.
00348      */
00349     static __pid_t AcquireRuntimeLock() ATTR_NO_EXCEPTIONS {
00350         __pid_t pid = 0;
00351 
00352         try {
00353             cout « "Acquiring run-time lock...";
00354 
00355             // Create lock file directory, if necessary.
00356             GetDirectoryPathFromString(pid_file_path_.parent_path().string(), true);
00357 
00358             /* Create the lock file exclusively, so two instances starting at the same time cannot
00359              * both acquire the lock. */
00360             int fd = ::open(pid_file_path_.c_str(), O_WRONLY | O_CREAT | O_EXCL, 0644);
00361             if(fd < 0) {
00362 
00363                 if(errno != EEXIST)
00364                     EXC_THROW("Failed to create PID file: " + pid_file_path_.string() + ": "
00365                               + strerror(errno));
00366 
00367                 // Get PID of previous lock.
00368                 ifstream ifs(pid_file_path_);
00369                 ifs » pid;
00370                 ifs.close();
00371 
00372                 // Compose and throw exception.
00373                 ostringstream oss;
00374                 oss « "PID file already exists: " « pid_file_path_ « endl;
00375                 oss « "\tPID: " « pid;
00376                 EXC_THROW(oss.str());
00377             }
00378 
00379             // Write PID to lock file.
00380             ostringstream oss_pid;
```

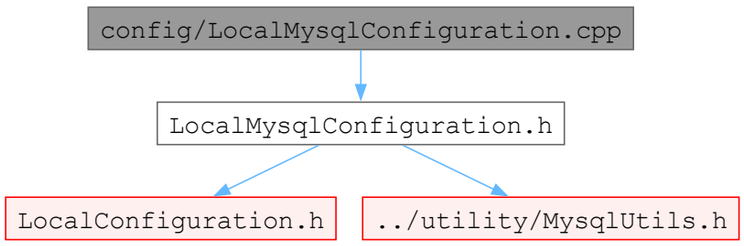
```
00381             oss_pid « (pid = getpid());
00382             const string pid_str = oss_pid.str();
00383             ssize_t num_written = ::write(fd, pid_str.data(), pid_str.size());
00384             ::close(fd);
00385             if(num_written != (ssize_t) pid_str.size()) {
00386                 ::unlink(pid_file_path_.c_str());
00387                 EXC_THROW("Failed to write PID file: " + pid_file_path_.string() + "!");
00388             }
00389
00390             // Check the lock.
00391             CheckRuntimeLock();
00392
00393             cout « " Done. Run-time lock successfully acquired. PID is " « pid « "." « endl;
00394
00395         }CATCH_BLOCK("FATAL: Failed to acquire run-time lock!");
00396         return pid;
00397     }
00398
00399     /** Check the run-time lock.
00400      *
00401      *  This method checks the run-time lock file to ensure that this instance of the program retains
00402      *  the run-time lock acquired using the acquireRuntimeLock() function. This means that the lock
00403      *  file exists, is accessible, and contains the PID of this instance of the program.
00404      *
00405      *
00406      *  @throw          log::MessageException if the run-time lock check fails e.g. the run-time lock
00407      *                  file does not exist, is not accessible, or contains the incorrect PID.
00408      */
00409     static void CheckRuntimeLock() {
00410
00411         __pid_t pid = 0;
00412         try {
00413             // Check PID file.
00414             if(exists(pid_file_path_)) {
00415                 // Check if lock file PID matches current PID.
00416                 ifstream ifs(pid_file_path_);
00417                 ifs » pid;
00418                 if(pid != getpid()) {
00419                     // Compose and throw exception.
00420                     ostringstream oss;
00421                     oss « "PID mismatch (expected " « getpid() « ", got " « pid « ")!";
00422                     EXC_THROW(oss.str());
00423                 }
00424             } else {
00425                 // Compose and throw exception.
00426                 EXC_THROW("PID file does not exist: " + pid_file_path_.string() + "!");
00427             }
00428
00429         }CATCH_BLOCK("Run-time lock check failed!");
00430     }
00431
00432     static options_description GenerateConfigurationFileOptionsDescription() ATTR_NO_EXCEPTIONS;
00433
00434     static options_description GenerateCommandLineOptionsDescription() ATTR_NO_EXCEPTIONS;
00435
00436     /** Release the run-time lock.
00437      *
00438      *  This method releases the run-time lock acquired by the AcquireRuntimeLock() function. It
00439      *  deletes the lock file from the local filesystem
00440      *
00441      */
00442     static void ReleaseRuntimeLock() {
00443         try {
00444             lout() « "Releasing run-time lock...";
00445
00446             // Delete run-time lock file, but only if it belongs to this process.
00447             if(exists(pid_file_path_)) {
00448                 __pid_t pid = 0;
00449                 ifstream ifs(pid_file_path_);
00450                 ifs » pid;
00451                 ifs.close();
00452                 if(pid != getpid()) {
00453                     lwrn() « "Not removing PID file owned by another process (PID " « pid
00454                            « "): " « pid_file_path_;
00455                     return;
00456                 }
00457
00458                 ldbg() « "Removing PID file: " « pid_file_path_ « " ...";
00459                 remove(pid_file_path_);
00460                 if(exists(pid_file_path_))
00461                     EXC_THROW("Failed to remove PID file: " + pid_file_path_.string() + "!");
00462             }
00463
00464             linf() « "Done. Run-time lock successfully removed.";
00465         }CATCH_BLOCK("Failed to release run-time lock!")
00466     }
00467
00468     /** Get the local configuration run-time settings object.
00469      *
00470      *  @return         The local configuration run-time settings object.
00471      */
00472     static const LocalConfiguration& configuration() {
00473         return configuration_;
00474     }
00475
00476     /** Is the application configured to run as a daemon process?
00477      *
00478      *  @return         TRUE if the application is configured to run as a daemon process, or FALSE
00479      *                  otherwise.
00480      */
00481     static bool is_daemon() {
00482         return is_daemon_;
00483     }
00484
00485     /** Get the splash listing text string.
00486      *
00487      *  @return         The splash listing text string.
00488      */
```

```
00489     static const char* splash_lis() {
00490         return SPLASH_LIS;
00491     }
00492
00493 };
00494
00495 } /* namespace config */
00496 } /* namespace util */
00497
00498 #endif /* LOCALCONFIGURATION_H_ */
```

11.6 config/LocalMysqlConfiguration.cpp File Reference

The util::mysql::LocalMysqlConfiguration class source file.

`#include "LocalMysqlConfiguration.h"`
Include dependency graph for LocalMysqlConfiguration.cpp:



Namespaces

- namespace util

  The utility namespace.
- namespace util::mysql

  The MySQL utility namespace.

11.6.1 Detailed Description

The util::mysql::LocalMysqlConfiguration class source file.

Version

0.6

Date

Aug 17, 2014 4:09:43 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

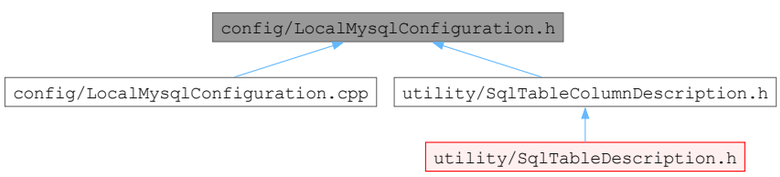
11.7 config/LocalMysqlConfiguration.h File Reference

The util::mysql::LocalMysqlConfiguration class header file.

```
#include "LocalConfiguration.h"
#include "../utility/MysqlUtils.h"
```

Include dependency graph for LocalMysqlConfiguration.h:

This graph shows which files directly or indirectly include this file:



Classes

- class util::mysql::LocalMysqlConfiguration

  Class providing useful functions for accessing and manipulating the MySQL database, given the local run-time configuration settings.

Namespaces

- namespace util

  The utility namespace.
- namespace util::mysql

  The MySQL utility namespace.

Macros

- #define CFG_MYSQL_DFLT_ADDRESS "127.0.0.1"

  The default MySQL server address.
- #define CFG_MYSQL_DFLT_PORT 3306

  The default MySQL server port number.
- #define CFG_MYSQL_DFLT_PROTOCOL "tcp"

  The default protocol for connecting to the MySQL server.
- #define CFG_MYSQL_DFLT_SCHEMA "float_db"

  The default MySQL database schema.

11.7.1 Detailed Description

The util::mysql::LocalMysqlConfiguration class header file.

Version

0.6

Date

Aug 17, 2014 4:09:43 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.7.2 Macro Definition Documentation

11.7.2.1 CFG_MYSQL_DFLT_ADDRESS

```
#define CFG_MYSQL_DFLT_ADDRESS "127.0.0.1"
```

The default MySQL server address.

Referenced by util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription().

11.7.2.2 CFG_MYSQL_DFLT_PORT

```
#define CFG_MYSQL_DFLT_PORT 3306
```

The default MySQL server port number.

Referenced by util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription().

11.7.2.3 CFG_MYSQL_DFLT_PROTOCOL

```
#define CFG_MYSQL_DFLT_PROTOCOL "tcp"
```

The default protocol for connecting to the MySQL server.

Referenced by util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription().

11.7.2.4 CFG_MYSQL_DFLT_SCHEMA

```
#define CFG_MYSQL_DFLT_SCHEMA "float_db"
```

The default MySQL database schema.

Referenced by util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription().

11.8 LocalMysqlConfiguration.h

Go to the documentation of this file.

```
00001 /** The @ref util::mysql::LocalMysqlConfiguration class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Aug 17, 2014 4:09:43 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef LOCALMYSQLCONFIGURATION_H_
00024 #define LOCALMYSQLCONFIGURATION_H_
00025
00026 /* Includes */
00027
00028 #include "LocalConfiguration.h"
00029 #include "../utility/MysqlUtils.h"
00030
00031 /* Definitions */
00032
00033 /** The default MySQL server address. */
00034 #define CFG_MYSQL_DFLT_ADDRESS      "127.0.0.1"
00035
00036 /** The default MySQL server port number. */
00037 #define CFG_MYSQL_DFLT_PORT         3306
00038
00039 /** The default protocol for connecting to the MySQL server. */
00040 #define CFG_MYSQL_DFLT_PROTOCOL     "tcp"
```

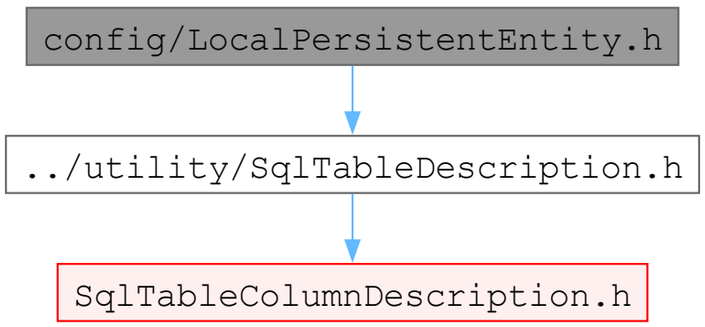
```
00041
00042 /** The default MySQL database schema. */
00043 #define CFG_MYSQL_DFLT_SCHEMA       "float_db"
00044
00045 namespace util {
00046 namespace mysql {
00047
00048 /* Class Definitions */
00049
00050 /** Class providing useful functions for accessing and manipulating the MySQL database, given the
00051  *  local run-time configuration settings.
00052  *
00053  */
00054 class LocalMysqlConfiguration:
00055         public util::mysql::MysqlUtils {
00056
00057     /* Friends */
00058
00059     friend class config::LocalConfiguration;
00060
00061 private:
00062
00063     /* Data Members */
00064
00065     /** The SQL server qualified name or IP address string. */
00066     static string address_;
00067
00068     /** The SQL server account username string. */
00069     static string user_;
00070
00071     /** The SQL server account password string. */
00072     static string password_;
00073
00074     /** The SQL server port number. */
00075     static unsigned int port_;
00076
00077     /** The protocol to use in the SQL server connection e.g. "tcp". */
00078     static string protocol_;
00079
00080     /** The SQL server schema name. */
00081     static string schema_;
00082
00083 protected:
00084
00085     /* Member Functions */
00086
00087     static sql::Connection* getSqlConnection();
00088
00089 public:
00090
00091     /* Member Functions */
00092
00093     LocalMysqlConfiguration();
00094
00095     virtual ~LocalMysqlConfiguration();
00096
00097     static shared_ptr<sql::ResultSet> executeQuery(const string& query);
00098
00099     static int executeUpdate(const string& update);
00100
00101     static shared_ptr<sql::ResultSet> executeSelectQuery(const string& what,
00102                                                          const string& from,
00103                                                          const string& where = "",
00104                                                          const string& groupBy = "",
00105                                                          const string& orderBy = "",
00106                                                          unsigned int limit = 0,
00107                                                          unsigned int offset = 0);
00108
00109     static shared_ptr<sql::ResultSet> executeDeleteQuery(
00110                                                          const string& from,
00111                                                          const string& where = "");
00112
00113     static shared_ptr<sql::PreparedStatement> generatePreparedStatement(
00114             const string& statement);
00115 };
00116
00117 } /* namespace mysql */
00118 } /* namespace util */
00119 #endif /* LOCALMYSQLCONFIGURATION_H_ */
```

11.9 config/LocalPersistentEntity.h File Reference

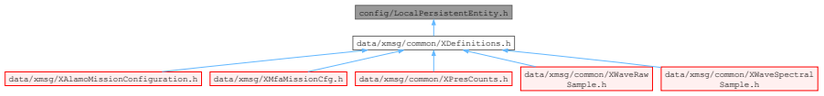
The util::mysql::LocalPersistentEntity class header file.

```
#include "../utility/SqlTableDescription.h"
```

Include dependency graph for LocalPersistentEntity.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::mysql::LocalPersistentEntity

  An abstract class representing a persistent database entity in the database defined by the config::LocalConfiguration.

Namespaces

- namespace util

  The utility namespace.
- namespace util::mysql

  The MySQL utility namespace.

11.9.1 Detailed Description

The util::mysql::LocalPersistentEntity class header file.

Version

0.6

Date

May 14, 2014 1:14:55 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.10 LocalPersistentEntity.h

Go to the documentation of this file.

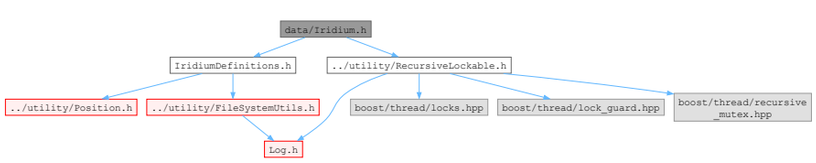
```
00001 /** The @ref util::mysql::LocalPersistentEntity class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       May 14, 2014 1:14:55 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef LOCALPERSISTENTENTITY_H_
00024 #define LOCALPERSISTENTENTITY_H_
00025
00026 /* Includes */
00027
00028 // local
00029 #include "../utility/SqlTableDescription.h"
00030
00031 namespace util {
00032 namespace mysql {
00033
00034 /* Class Definition */
00035
00036 /** An abstract class representing a persistent database entity in the database defined by the
00037  *  @ref config::LocalConfiguration.
00038  *
00039  */
00040 class LocalPersistentEntity: public LocalMysqlConfiguration{
00041
00042 public:
00043
00044     /* Public Member Functions: */
00045
00046     /** Default constructor.
00047      *
00048      */
00049     LocalPersistentEntity() {
00050         /* Do nothing. */
00051     }
00052
00053     /** The destructor.
00054      *
00055      */
00056     virtual ~LocalPersistentEntity() {
00057         /* Do nothing. */
00058     }
00059
00060     /** Persist this database entity.
00061      *
00062      */
00063     virtual void SaveObjectToDatabase() const = 0;
00064
00065 };
00066
00067 } /* namespace mysql */
00068 } /* namespace util */
00069 #endif /* LOCALPERSISTENTENTITY_H_ */
```

11.11 data/Iridium.h File Reference

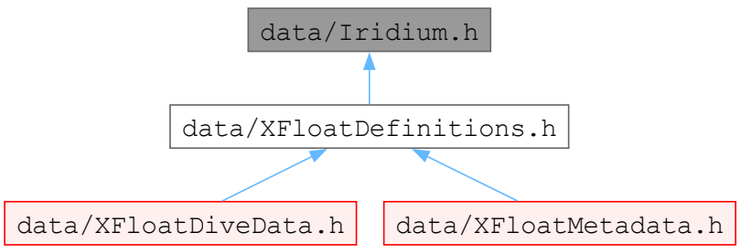
The argo::iridium::Iridium class header file.

```
#include "IridiumDefinitions.h"
#include "../utility/RecursiveLockable.h"
```

Include dependency graph for Iridium.h:



This graph shows which files directly or indirectly include this file:



Classes

- class argo::iridium::Iridium

  The Iridium instrument class.

Namespaces

- namespace argo

  The Argo instrument namespace.
- namespace argo::iridium

  The Argo Iridium instrument namespace.

11.11.1 Detailed Description

The argo::iridium::Iridium class header file.

This is the header file for the argo::iridium::Iridium class. This class represents an individual instrument and its associated unique identifiers. This class is not used explicitly. Rather, all other classes that are associated with an instrument's unique identifiers inherit from this class.

This file contains the class definition and the complete functionality of the class.

Version

0.6

Date

Apr 3, 2014 2:45:21 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.12 Iridium.h

Go to the documentation of this file.

```
00001 /** The @ref argo::iridium::Iridium class header file.
00002  *
00003  *  This is the header file for the @ref argo::iridium::Iridium class. This class represents an
00004  *  individual instrument and its associated unique identifiers. This class is not used explicitly.
00005  *  Rather, all other classes that are associated with an instrument's unique identifiers inherit
00006  *  from this class.
00007  *
00008  *  This file contains the class definition and the complete functionality of the class.
00009  *
00010  *  @file
00011  *
00012  *  @version    0.6
00013  *
00014  *  @date       Apr 3, 2014 2:45:21 PM
00015  *
00016  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00017  *              Engineer I,
00018  *              Department of Physical Oceanography,
00019  *              Woods Hole Oceanographic Institution
00020  *
00021  *  @copyright  2014 Woods Hole Oceanographic Institution.
00022  *              Released under the GNU General Public License.
00023  *
00024  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00025  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00026  *  General Public License for more details. You should have received a copy of the GNU General
00027  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00028  */
00029
00030 #ifndef ARGOIRIDIUM_H_
00031 #define ARGOIRIDIUM_H_
00032
00033 /* Includes: */
00034
00035 // local
00036 #include "IridiumDefinitions.h"
00037 #include "../utility/RecursiveLockable.h"
00038
00039 /* Definitions */
00040
00041 namespace argo {
00042 namespace iridium {
00043
00044 /** The Iridium instrument class.
00045  *
00046  *  This class represents an individual instrument and its associated unique identifiers. This
00047  *  class is not used explicitly. Rather, all other classes that are associated with an
00048  *  instrument's unique identifiers inherit from this class.
00049  *
00050  *  @note   This object is lockable. It contains a recursive mutual exclusion (mutex) lock in
00051  *          order to synchronize access to and modification of objects of this type by various
00052  *          worker threads. Once the object is locked, any other thread that attempts to access
00053  *          or modify the object will block until the initial lock is released. Since the mutex
00054  *          is recursive, the object may be locked multiple times by the same thread without
00055  *          causing the thread to block indefinitely.
00056  */
00057 class Iridium:
00058         public util::thread::RecursiveLockable {
00059
00060 private:
00061
00062     /* Dynamic Data Members: */
00063
00064     /** The unique Data Assembly Center (DAC) identifier. */
00065     int id_dac_;
00066
00067     /** The unique International Mobile Equipment Identifier (IMEI). */
00068     long id_imei_;
00069
00070     /** The unique instrument identifier or instrument serial number. */
00071     int id_instrument_;
00072
00073     /** The unique World Meteorological Organization (WMO) identifier. */
00074     int id_wmo_;
00075
00076 protected:
00077
00078     /* Protected Member Functions: */
00079
00080     /** The default constructor.
00081      *
00082      */
00083     Iridium() :
00084             id_dac_(IRI_ID_UNKNOWN),
00085             id_imei_(IRI_ID_UNKNOWN),
00086             id_instrument_(IRI_ID_UNKNOWN),
00087             id_wmo_(IRI_ID_UNKNOWN) {
00088         /* Do nothing. */
00089     }
00090
00091     /** Set the unique Data Assembly Center (DAC) identifier.
00092      *
00093      *  @param[in]  id_dac          The unique Data Assembly Center (DAC) identifier.
00094      */
00095     void set_id_dac(int id_dac) {
00096
00097         // Get an exclusive lock.
00098         lock_guard<recursive_mutex> guard(mutex_);
00099
00100         id_dac_ = id_dac;
00101     }
00102
00103     /** Set the unique International Mobile Equipment Identifier (IMEI).
```

```
00104      *
00105      *  @param[in]  id_imei         The unique International Mobile Equipment Identifier (IMEI).
00106      */
00107     void set_id_imei(long id_imei) {
00108
00109         // Get an exclusive lock.
00110         lock_guard<recursive_mutex> guard(mutex_);
00111
00112         id_imei_ = id_imei;
00113     }
00114
00115     /** Set the unique instrument identifier or instrument serial number.
00116      *
00117      *  @param[in] id_instrument    The unique instrument identifier or instrument serial number
00118      */
00119     void set_id_instrument(int id_instrument) {
00120         // Get an exclusive lock.
00121         lock_guard<recursive_mutex> guard(mutex_);
00122
00123         id_instrument_ = id_instrument;
00124     }
00125
00126     /** Set the unique World Meteorological Organization (WMO) identifier.
00127      *
00128      *  @param[in] id_wmo           The unique World Meteorological Organization (WMO) identifier.
00129      */
00130     void set_id_wmo(int id_wmo) {
00131
00132         // Get an exclusive lock.
00133         lock_guard<recursive_mutex> guard(mutex_);
00134
00135         id_wmo_ = id_wmo;
00136     }
00137
00138 public:
00139
00140     /* Public Member Functions: */
00141
00142     /** The parameterized constructor.
00143      *
00144      *  @param[in]      imei        The unique International Mobile Equipment Identifier (IMEI).
00145      *  @param[in]      whoi        The unique WHOI identifier.
00146      *  @param[in]      wmo         The unique World Meteorological Organization (WMO) identifier.
00147      *  @param[in]      dac         The unique Data Assembly Center (DAC) identifier.
00148      */
00149     Iridium(int whoi, long imei = IRI_ID_UNKNOWN, int wmo = IRI_ID_UNKNOWN, int dac =
00150     IRI_ID_UNKNOWN) :
00151             id_dac_(dac),
00152             id_imei_(imei),
00153             id_instrument_(whoi),
00154             id_wmo_(wmo) {
00155         /* Do nothing. */
00156     }
00157
00158     /** The destructor.
00159      *
00160      */
00161     virtual ~Iridium() {
00162         /* Do nothing. */
00163     }
00164
00165     /** Get the unique Data Assembly Center (DAC) identifier.
00166      *
00167      *  @return     The unique Data Assembly Center (DAC) identifier.
00168      */
00169     int id_dac() const {
00170
00171         // Get an exclusive lock.
00172         lock_guard<recursive_mutex> guard(mutex_);
00173
00174         return id_dac_;
00175     }
00176
00177     /** Get the unique International Mobile Equipment Identifier (IMEI).
00178      *
00179      *  @return     The unique International Mobile Equipment Identifier (IMEI).
00180      */
00181     long id_imei() const {
00182
00183         // Get an exclusive lock.
00184         lock_guard<recursive_mutex> guard(mutex_);
00185
00186         return id_imei_;
00187     }
00188
00189     /** Get the unique instrument identifier or instrument serial number.
00190      *
00191      *  @return     The unique instrument identifier or instrument serial number.
00192      */
00193     int id_instrument() const {
00194
00195         // Get an exclusive lock.
00196         lock_guard<recursive_mutex> guard(mutex_);
00197
00198         return id_instrument_;
00199     }
00200
00201     /** Get the unique World Meteorological Organization (WMO) identifier.
00202      *
00203      *  @return     The unique World Meteorological Organization (WMO) identifier.
00204      */
00205     int id_wmo() const {
00206
00207         // Get an exclusive lock.
00208         lock_guard<recursive_mutex> guard(mutex_);
00209
00210         return id_wmo_;
00211     }
```

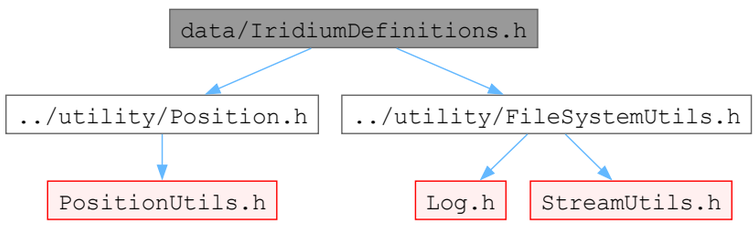
```
00212
00213 };
00214
00215 } /* namespace iridium */
00216 } /* namespace argo */
00217
00218 #endif /* ARGOIRIDIUM_H_ */
```

11.13 data/IridiumDefinitions.h File Reference

The Iridium Definitions header file.

```
#include "../utility/Position.h"
#include "../utility/FileSystemUtils.h"
```

Include dependency graph for IridiumDefinitions.h:



This graph shows which files directly or indirectly include this file:

Namespaces

- namespace argo

  The Argo instrument namespace.
- namespace argo::iridium

  The Argo Iridium instrument namespace.

Macros

- #define IRI_DIVE_UNKNOWN -999

  The default/bad/invalid dive value.
- #define IRI_GAIN_INVALID 9999

  Default/fill value for invalid Iridium message gain value.
- #define IRI_ID_UNKNOWN -1

  The default/bad/invalid unique identifier value.
- #define IRI_OFFS_INVALID 9999

  Default/fill value for invalid Iridium message offset value.
- #define IRI_POS_INVALID_CEP 999999

  Default/fill value for invalid Iridium Circular Error Probable (CEP) radius.
- #define IRI_POS_INVALID_LAT UTIL_POS_INVALID_LAT

  Default/fill value for invalid Iridium position latitude.
- #define IRI_POS_INVALID_LON UTIL_POS_INVALID_LAT

  Default/fill value for invalid Iridium position longitude.

11.13.1 Detailed Description

The Iridium Definitions header file.

This file contains definitions that are used by the argo::iridium::Iridium class and the classes that inherit from it.

Version

0.6

Date

Jun 9, 2014 2:17:29 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.13.2 Macro Definition Documentation

11.13.2.1 IRI_DIVE_UNKNOWN

```
#define IRI_DIVE_UNKNOWN -999
```

The default/bad/invalid dive value.

11.13.2.2 IRI_GAIN_INVALID

```
#define IRI_GAIN_INVALID 9999
```

Default/fill value for invalid Iridium message gain value.

11.13.2.3 IRI_ID_UNKNOWN

```
#define IRI_ID_UNKNOWN -1
```

The default/bad/invalid unique identifier value.

11.13.2.4 IRI_OFFS_INVALID

```
#define IRI_OFFS_INVALID 9999
```

Default/fill value for invalid Iridium message offset value.

11.13.2.5 IRI_POS_INVALID_CEP

```
#define IRI_POS_INVALID_CEP 999999
```

Default/fill value for invalid Iridium Circular Error Probable (CEP) radius.

11.13.2.6 IRI_POS_INVALID_LAT

```
#define IRI_POS_INVALID_LAT UTIL_POS_INVALID_LAT
```

Default/fill value for invalid Iridium position latitude.

11.13.2.7 IRI_POS_INVALID_LON

```
#define IRI_POS_INVALID_LON UTIL_POS_INVALID_LAT
```

Default/fill value for invalid Iridium position longitude.

11.14 IridiumDefinitions.h

Go to the documentation of this file.

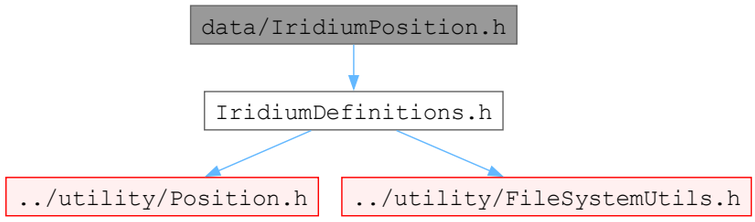
```
00001 /** The Iridium Definitions header file.
00002  *
00003  *  This file contains definitions that are used by the @ref argo::iridium::Iridium class and the
00004  *  classes that inherit from it.
00005  *
00006  *  @file
00007  *
00008  *  @version    0.6
00009  *
00010  *  @date       Jun 9, 2014 2:17:29 PM
00011  *
00012  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00013  *              Engineer I,
00014  *              Department of Physical Oceanography,
00015  *              Woods Hole Oceanographic Institution
00016  *
00017  *  @copyright  2014 Woods Hole Oceanographic Institution.
00018  *              Released under the GNU General Public License.
00019  *
00020  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00021  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00022  *  General Public License for more details. You should have received a copy of the GNU General
00023  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00024  */
00025
00026 #ifndef IRIDIUMDEFINITIONS_H_
00027 #define IRIDIUMDEFINITIONS_H_
00028
00029 /* Includes: */
00030
00031 // local
00032 #include "../utility/Position.h"
00033 #include "../utility/FileSystemUtils.h"
00034
00035 /* Definitions: */
00036
00037 /** The default/bad/invalid dive value. */
00038 #define IRI_DIVE_UNKNOWN           -999
00039
00040 /** Default/fill value for invalid Iridium message gain value. */
00041 #define IRI_GAIN_INVALID            9999
00042
00043 /** The default/bad/invalid unique identifier value. */
00044 #define IRI_ID_UNKNOWN               -1
00045
00046 /** Default/fill value for invalid Iridium message offset value. */
00047 #define IRI_OFFS_INVALID            9999
00048
00049 /** Default/fill value for invalid Iridium Circular Error Probable (CEP) radius. */
00050 #define IRI_POS_INVALID_CEP         999999
00051
00052 /** Default/fill value for invalid Iridium position latitude. */
00053 #define IRI_POS_INVALID_LAT         UTIL_POS_INVALID_LAT
00054
00055 /** Default/fill value for invalid Iridium position longitude. */
00056 #define IRI_POS_INVALID_LON         UTIL_POS_INVALID_LAT
00057
00058 namespace argo {
00059 namespace iridium {
00060 using boost::algorithm::trim;
00061 using boost::filesystem::ifstream;
00062 using boost::filesystem::ofstream;
00063 using boost::filesystem::path;
00064 using boost::format;
00065 using boost::gregorian::date;
00066 using boost::lexical_cast;
00067 using std::lock_guard;
00068 using std::make_shared;
00069 using boost::posix_time::ptime;
00070 using boost::posix_time::hours;
00071 using boost::posix_time::millisec;
00072 using boost::posix_time::minutes;
00073 using boost::posix_time::not_a_date_time;
00074 using boost::posix_time::seconds;
00075 using boost::posix_time::time_duration;
00076 using std::shared_ptr;
00077 using std::recursive_mutex;
00078 using std::endl;
00079 using std::for_each;
```

```
00080 using std::istream;
00081 using std::istringstream;
00082 using std::max;
00083 using std::ostream;
00084 using std::ostringstream;
00085 using std::smatch;
00086 using std::string;
00087 using std::stringstream;
00088 using std::regex;
00089 using std::regex_match;
00090 using std::regex_replace;
00091 using std::regex_search;
00092 using std::vector;
00093 }
00094 }
00095
00096 #endif /* IRIDIUMDEFINITIONS_H_ */
```

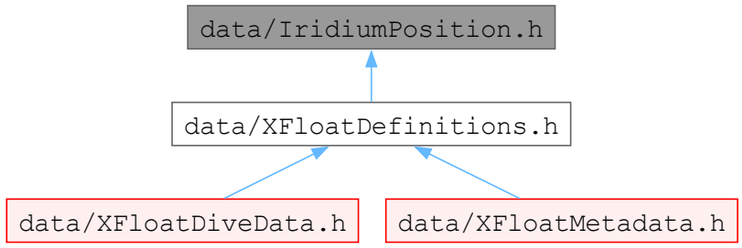
11.15 data/IridiumPosition.h File Reference

The argo::iridium::IridiumPosition class header file.

`#include "IridiumDefinitions.h"`
Include dependency graph for IridiumPosition.h:



This graph shows which files directly or indirectly include this file:



Classes

- class argo::iridium::IridiumPosition

  The Iridium position class.

Namespaces

- namespace argo

  The Argo instrument namespace.
- namespace argo::iridium

  The Argo Iridium instrument namespace.

11.15.1 Detailed Description

The argo::iridium::IridiumPosition class header file.

This is the header file for the argo::iridium::IridiumPosition class. This class represents a position that was acquired through the Iridium system. It represents an estimate of the position of the instrument and is provided in the body of an email message received from the Iridium system along the an Iridium Short Burst Data (SBD) message attachment.

This file contains the class definition and the complete functionality of the class.

Version

0.6

Date

May 12, 2014 4:00:20 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.16 IridiumPosition.h

Go to the documentation of this file.

```
00001 /** The @ref argo::iridium::IridiumPosition class header file.
00002  *
00003  *  This is the header file for the @ref argo::iridium::IridiumPosition class. This class
00004  *  represents a position that was acquired through the Iridium system. It represents an
00005  *  estimate of the position of the instrument and is provided in the body of an email message
00006  *  received from the Iridium system along the an Iridium Short Burst Data (SBD) message
00007  *  attachment.
00008  *
00009  *  This file contains the class definition and the complete functionality of the class.
00010  *
00011  *  @file
00012  *
00013  *  @version    0.6
00014  *
00015  *  @date       May 12, 2014 4:00:20 PM
00016  *
00017  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00018  *              Engineer I,
00019  *              Department of Physical Oceanography,
00020  *              Woods Hole Oceanographic Institution
00021  *
00022  *  @copyright  2014 Woods Hole Oceanographic Institution.
00023  *              Released under the GNU General Public License.
00024  *
00025  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00026  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00027  *  General Public License for more details. You should have received a copy of the GNU General
00028  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00029  */
00030
00031 #ifndef IRIDIUMPOSITION_H_
00032 #define IRIDIUMPOSITION_H_
00033
00034 /* Includes: */
00035
00036 // local
00037 #include "IridiumDefinitions.h"
00038
00039 namespace argo {
00040 namespace iridium {
00041
00042 using boost::posix_time::ptime;
00043
00044 /* Class Definition: */
00045
00046 /** The Iridium position class.
00047  *
```

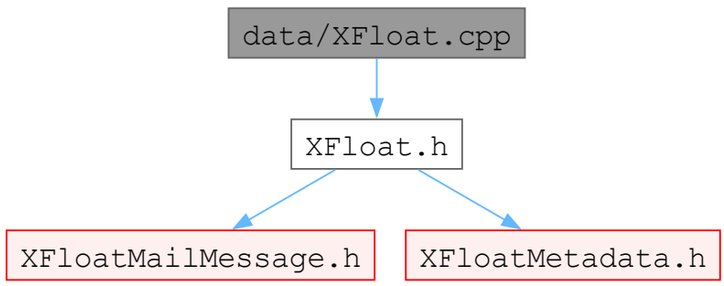
```
00048  *  This class represents a position that was acquired through the Iridium system. It represents
00049  *  an estimate of the position of the instrument and is provided in the body of an email message
00050  *  received from the Iridium system along the an Iridium Short Burst Data (SBD) message
00051  *  attachment.
00052  *
00053  */
00054 class IridiumPosition: protected util::position::Position {
00055
00056 private:
00057
00058     /* Dynamic Data Members: */
00059
00060     /** Iridium position Circular Error Probable (CEP) radius, in kilometers.
00061      *
00062      *  This field defines the estimate of an error of the Iridium position. It is provided by
00063      *  the Iridium system, along with the Iridium position, in the body of the email containing
00064      *  the Iridium SBD message.
00065      *
00066      *  Since multiple Iridium messages are transmitted for each dive cycle, a better estimate of
00067      *  the instrument position is provided by a composite position, constructed from two or more
00068      *  individual positions. To compute this composite position, each position is weighted
00069      *  proportionally to the inverse of the CEP radius.
00070      */
00071     int iridium_position_cep_radius_;
00072
00073 protected:
00074
00075     /* Protected Member Functions: */
00076
00077     /** Set the Iridium position Circular Error Probable (CEP) radius, in kilometers.
00078      *
00079      *  @param[in]  iridium_position_cep_radius The Iridium position Circular Error Probable (CEP)
00080      *                                          radius to set, in kilometers.
00081      */
00082     void set_iridium_position_cep_radius(int iridium_position_cep_radius) {
00083         iridium_position_cep_radius_ = iridium_position_cep_radius;
00084     }
00085
00086     /** Set the Iridium position date/time.
00087      *
00088      *  @param[in]  iridium_position_datetime   The Iridium position date/time to set.
00089      */
00090     void set_iridium_position_datetime(const ptime& iridium_position_datetime) {
00091         set_position_datetime(iridium_position_datetime);
00092     }
00093
00094     /** Set the Iridium position is valid flag.
00095      *
00096      *  @param[in]  iridium_position_is_valid   The Iridium position is valid flag to set.
00097      */
00098     void set_iridium_position_is_valid(bool iridium_position_is_valid) {
00099         set_position_is_valid(iridium_position_is_valid);
00100     }
00101
00102     /** Set the Iridium position latitude, in decimal degrees.
00103      *
00104      *  @param[in]  iridium_position_latitude   The Iridium position latitude to set, in decimal
00105      *                                          degrees.
00106      */
00107     void set_iridium_position_latitude(double iridium_position_latitude) {
00108         set_position_latitude(iridium_position_latitude);
00109     }
00110
00111     /** Set the Iridium position longitude, in decimal degrees.
00112      *
00113      *  @param[in]  iridium_position_longitude  The Iridium position longitude to set, in decimal
00114      *                                          degrees.
00115      */
00116     void set_iridium_position_longitude(double iridium_position_longitude) {
00117         set_position_longitude(iridium_position_longitude);
00118     }
00119
00120 public:
00121
00122     /* Public Member Functions: */
00123
00124     /** The default constructor.
00125      *
00126      */
00127     IridiumPosition() :
00128             Position(), iridium_position_cep_radius_(IRI_POS_INVALID_CEP) {
00129         /* Do nothing. */
00130     }
00131
00132     /** The destructor.
00133      *
00134      */
00135     ~IridiumPosition() {
00136         /* Do nothing. */
00137     }
00138
00139     /** Get the Iridium position Circular Error Probable (CEP) radius, in kilometers
00140      *
00141      *  @return     The Iridium position Circular Error Probable (CEP) radius, in kilometers
00142      */
00143     int iridium_position_cep_radius() const {
00144         return iridium_position_cep_radius_;
00145     }
00146
00147     /** Get a formatted string representing the Iridium position composed using the ISO 6709
00148      *  standard.
00149      *
00150      *  @return     The formatted string representation of the Iridium position.
00151      */
00152     string iridium_position_as_iso6709_string() const {
00153         return postition_as_iso6709_string();
00154     }
00155
```

```
00156     /** Get a formatted string representing the Iridium position date/time composed using the ISO
00157      *  8601 standard.
00158      *
00159      *  @return     The formatted string representation of the Iridium position date/time.
00160      */
00161     string iridium_position_datetime_as_iso8601_string() const {
00162         return position_datetime_as_iso8601_string();
00163     }
00164
00165     /** Get the Iridium position date/time.
00166      *
00167      *  @return     The Iridium position date/time.
00168      */
00169     ptime iridium_position_datetime() const {
00170         return position_datetime();
00171     }
00172
00173     /** Get the Iridium position is valid flag.
00174      *
00175      *  @return     TRUE if the Iridium position is valid, or FALSE otherwise.
00176      */
00177     bool iridium_position_is_valid() const {
00178         return position_is_valid();
00179     }
00180
00181     /** Get the Iridium position latitude.
00182      *
00183      *  @return     The Iridium position latitude.
00184      */
00185     double iridium_position_latitude() const {
00186         return position_latitude();
00187     }
00188
00189     /** Get the Iridium position longitude.
00190      *
00191      *  @return     The Iridium position longitude.
00192      */
00193     double iridium_position_longitude() const {
00194         return position_longitude();
00195     }
00196 };
00197
00198 } /* namespace iridium */
00199 } /* namespace argo */
00200
00201 #endif /* IRIDIUMPOSITION_H_ */
```

11.17 data/XFloat.cpp File Reference

The argo::iridium::xfloat::XFloat class source file.

`#include "XFloat.h"`
Include dependency graph for XFloat.cpp:



Namespaces

- namespace argo

  The Argo instrument namespace.
- namespace argo::iridium

  The Argo Iridium instrument namespace.
- namespace argo::iridium::xfloat

  The X-Float instrument namespace.

11.17.1 Detailed Description

The argo::iridium::xfloat::XFloat class source file.

This is the source file for the argo::iridium::xfloat::XFloat class. This class represents a single instrument or float. Each instance of this contains information about an instrument including unique identifiers, sensor gains and offsets, metadata, and the instrument type. It also contains a dive-or-cycle-number-to-dive-data-object map that contains all of the incomplete dive data objects that belong to the instrument, uniquely indexed by dive or cycle number. As the application receives the telemetry data from the instrument, the data is decoded and assigned to the corresponding dive data object. Once it is determined that the data is complete for a single dive, the intermediate and PHY listing files are written to the local filesystem, the dive data object is persisted to the SQL database, and then the shared reference to the dive data object is released, thereby purging the dive data from the program's memory.

At run-time, at most one float object will exist for each instrument. Each float object is accessed and/or manipulated by the worker threads through a shared reference. A shared reference to all float objects is contained in the instrument-identifier-to-instrument-object map which is indexed by the unique instrument identifier. During application initialization, the map is populated as a result of querying all the known instruments from the SQL database. Each time a float object is created, it is added to the map. During application termination, the daemon thread iterates over the contents of the map and persists each of the float objects to the SQL database.

This file contains the static data members and some of the member functions of the argo::iridium::xfloat::XFloat class. Several of the member functions of the argo::iridium::xfloat::XFloatDiveData class are also defined in this file.

Version

0.6

Date

May 12, 2014 4:10:33 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

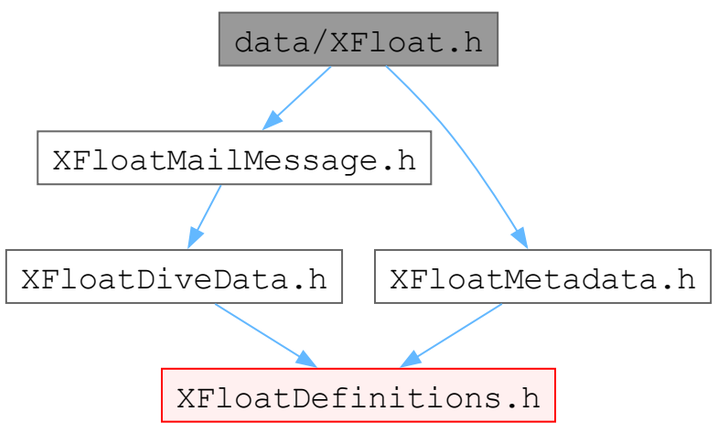
This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.18 data/XFloat.h File Reference

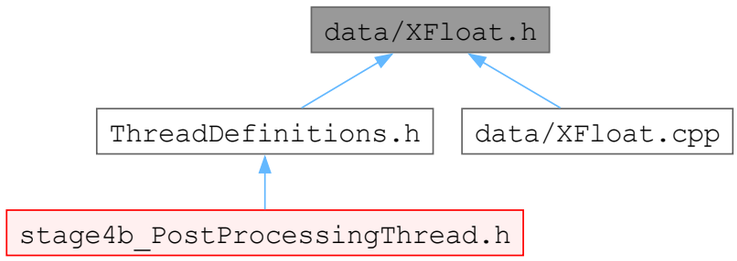
The argo::iridium::xfloat::XFloat class header file.

```
#include "XFloatMailMessage.h"
#include "XFloatMetadata.h"
```

Include dependency graph for XFloat.h:



This graph shows which files directly or indirectly include this file:



Classes

- class argo::iridium::xfloat::XFloat

  The instrument or float class.

Namespaces

- namespace argo

  The Argo instrument namespace.
- namespace argo::iridium

  The Argo Iridium instrument namespace.
- namespace argo::iridium::xfloat

  The X-Float instrument namespace.

11.18.1 Detailed Description

The argo::iridium::xfloat::XFloat class header file.

This is the header file for the argo::iridium::xfloat::XFloat class. This class represents a single instrument or float. Each instance of this contains information about an instrument including unique identifiers, sensor gains and offsets, metadata, and the instrument type. It also contains a dive-or-cycle-number-to-dive-data-object map that contains all of the incomplete dive data objects that belong to the instrument, uniquely indexed by dive or cycle number. As the application receives the telemetry data from the instrument, the data is decoded and assigned to the corresponding dive data object. Once it is determined that the data is complete for a single dive, the intermediate and PHY listing files are written to the local filesystem, the dive data object is persisted to the SQL database, and then the shared reference to the dive data object is released, thereby purging the dive data from the program's memory.

At run-time, at most one float object will exist for each instrument. Each float object is accessed and/or manipulated by the worker threads through a shared reference. A shared reference to all float objects is contained in the instrument-identifier-to-instrument-object map which is indexed by the unique instrument identifier. During application initialization, the map is populated as a result of querying all the known instruments from the SQL database. Each time a float object is created, it is added to the map. During application termination, the daemon thread iterates over the contents of the map and persists each of the float objects to the SQL database.

This file contains the argo::iridium::xfloat::XFloat class definition and the majority of the functionality of the class.

Version

0.6

Date

May 12, 2014 4:10:48 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.19 XFloat.h

Go to the documentation of this file.

```
00001 /** The @ref argo::iridium::xfloat::XFloat class header file.
00002  *
00003  *  This is the header file for the @ref argo::iridium::xfloat::XFloat class. This class
00004  *  represents a single instrument or float. Each instance of this contains information about an
00005  *  instrument including unique identifiers, sensor gains and offsets, metadata, and the
00006  *  instrument type. It also contains a dive-or-cycle-number-to-dive-data-object map that
00007  *  contains all of the incomplete dive data objects that belong to the instrument, uniquely
00008  *  indexed by dive or cycle number. As the application receives the telemetry data from the
00009  *  instrument, the data is decoded and assigned to the corresponding dive data object. Once it
00010  *  is determined that the data is complete for a single dive, the intermediate and PHY listing
00011  *  files are written to the local filesystem, the dive data object is persisted to the SQL
00012  *  database, and then the shared reference to the dive data object is released, thereby purging
00013  *  the dive data from the program's memory.
00014  *
00015  *  At run-time, at most one float object will exist for each instrument. Each float object is
00016  *  accessed and/or manipulated by the worker threads through a shared reference. A shared
00017  *  reference to all float objects is contained in the instrument-identifier-to-instrument-object
00018  *  map which is indexed by the unique instrument identifier. During application initialization,
00019  *  the map is populated as a result of querying all the known instruments from the SQL database.
00020  *  Each time a float object is created, it is added to the map. During application termination,
00021  *  the daemon thread iterates over the contents of the map and persists each of the float objects
00022  *  to the SQL database.
00023  *
00024  *  This file contains the @ref argo::iridium::xfloat::XFloat class definition and the majority
00025  *  of the functionality of the class.
00026  *
00027  *  @file
00028  *
00029  *  @version    0.6
00030  *
00031  *  @date       May 12, 2014 4:10:48 PM
00032  *
00033  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00034  *              Engineer I,
00035  *              Department of Physical Oceanography,
00036  *              Woods Hole Oceanographic Institution
00037  *
00038  *  @copyright  2014 Woods Hole Oceanographic Institution.
00039  *              Released under the GNU General Public License.
00040  *
00041  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00042  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00043  *  General Public License for more details. You should have received a copy of the GNU General
00044  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00045  */
00046
00047 #ifndef XFLOAT_H_
00048 #define XFLOAT_H_
00049
00050 /* Includes: */
00051
00052 #include "XFloatMailMessage.h"          // local
00053 #include "XFloatMetadata.h"
00054
00055 namespace argo {
00056 namespace iridium {
00057 namespace xfloat {
00058
00059 /* Class Definition: */
00060
00061 class XFloatTelemetryDecoderService;    // forward declaration
00062
00063 /** The instrument or float class.
00064  *
00065  *  This class represents a single instrument or float. Each instance of this contains information
00066  *  about an instrument including unique identifiers, sensor gains and offsets, metadata, and the
00067  *  instrument type. It also contains a dive-or-cycle-number-to-dive-data-object, @ref
00068  *  dive2data_map_, map that contains all of the incomplete dive data objects that belong to the
00069  *  instrument, uniquely indexed by dive or cycle number. As the application receives the
00070  *  telemetry data from the instrument, the data is decoded and assigned to the corresponding dive
00071  *  data object. Once it is determined that the data is complete for a single dive, the
00072  *  intermediate and PHY listing files are written to the local filesystem, the dive data object
00073  *  is persisted to the SQL database, and then the shared reference to the dive data object is
00074  *  released, thereby purging the dive data from the program's memory.
00075  *
00076  *  At run-time, at most one float object will exist for each instrument. Each float object is
00077  *  accessed and/or manipulated by the worker threads through a shared reference. A shared
00078  *  reference to all float objects is contained in the instrument-identifier-to-instrument-object
00079  *  map, @ref id2float_map_, which is indexed by the unique instrument identifier. During
00080  *  application initialization, the map is populated as a result of querying all the known
00081  *  instruments from the SQL database. Each time a float object is created, it is added to the
00082  *  map. During application termination, the daemon thread iterates over the contents of the map
00083  *  and persists each of the float objects to the SQL database.
00084  *
00085  *  @note   This object is lockable. It contains a recursive mutual exclusion (mutex) lock in
00086  *          order to synchronize access to and modification of objects of this type by various
00087  *          worker threads. Once the object is locked, any other thread that attempts to access
00088  *          or modify the object will block until the initial lock is released. Since the mutex
00089  *          is recursive, the object may be locked multiple times by the same thread without
00090  *          causing the thread to block indefinitely.
00091  *
00092  *  @note   This class is mapped to an SQL database table by the @ref sql_table_description_
00093  *          object. This allows some or all of the field of this class to be persisted to the SQL
00094  *          database. Likewise, an object of this type may be constructed from the results of an
00095  *          SQL query to the corresponding table.
00096  *
00097  *  @todo   Devise a method to actively check for (1) changes in the metadata files and (2) newly
00098  *          created metadata files. Load the metadata from the new or modified files and assign
00099  *          them to the corresponding float object. If necessary, retroactively process the mail
00100 *          messages for these instruments. Persist updates to the SQL database as necessary.
00101  */
00102 class XFloat:
00103     protected util::file::FileSystemUtils,
```

```
00104     public Iridium,
00105     public util::kml::KmlUtil,
00106     public util::mysql::LocalPersistentEntity,
00107     protected util::position::PositionUtils {
00108
00109     /* Friends Classes: */
00110
00111     friend class XFloatDiveData;
00112
00113     friend class XFloatMailProcessingService;
00114
00115     friend class XFloatTelemetryDecoderService;
00116
00117     friend class XFloatPostProcessingService;
00118
00119     /* Friend Functions: */
00120
00121     /** Friend function for populating some of the static members of this class from the
00122      *  command-line options.
00123      */
00124     friend options_description util::config::LocalConfiguration::GenerateCommandLineOptionsDescription();
00125
00126     /** Friend function for populating some of the static members of this class from the
00127      *  configuration file options.
00128      */
00129     friend options_description
      util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription();
00130
00131     // Friend function for performing startup tasks during initialization (documented in Main.cpp).
00132     friend const util::config::LocalConfiguration& Initialize(int argc, char** argv);
00133
00134
00135   private:
00136
00137     /* Private Static Data Members: */
00138
00139     /** The instrument-identifier-to-instrument-object map.
00140      *
00141      *  This object is a flat map that contains shared references to all of the instances of this
00142      *  class. It acts as a static, local cache of the instrument data. The object maps the
00143      *  unique instrument identifier to the corresponding float object. During application
00144      *  initialization, this map is populated as a result of querying all the known instruments
00145      *  from the SQL database.
00146      *
00147      *  When a specific float object is requested by any of the threads, this map is searched for
00148      *  the instrument object first. If the float object does not already exist, then it is
00149      *  created and added to this map.
00150      *
00151      *  Upon application termination, all of the float objects in this map are persisted to the
00152      *  SQL database so that they may be again loaded into this map when the application is
00153      *  restarted.
00154      *
00155      *  Accesses and modifications to this map are synchronized using the recursive mutex @ref
00156      *  id2float_map_mutex_. The mutex provides an exclusive lock so that only one thread at a
00157      *  time may either access or modify this object.
00158      */
00159     static flat_map<int, std::shared_ptr<XFloat> > id2float_map_;
00160
00161     /** The mutex lock for synchronizing access to the instrument-identifier-to-instrument-object
00162      *  map.
00163      *
00164      *  This mutual exclusion lock is used to ensure that accesses and modifications to the
00165      *  instrument-identifier-to-instrument-object map, @ref id2float_map_, are synchronized so
00166      *  that only one thread at a time may either access or modify this object. Because the mutex
00167      *  is recursive, it may be locked multiple times by the same thread without causing the
00168      *  thread to block indefinitely.
00169      */
00170     static recursive_mutex id2float_map_mutex_;
00171
00172     /** The SQL table description object.
00173      *
00174      *  This SQL table description provides the details needed to map this object to a relational
00175      *  database table. It utilizes the accessor methods of the class and the details of the SQL
00176      *  table column to (1) create an SQL INSERT statement to save the object into the database,
00177      *  (2) populate one or more objects' fields as a result of an SQL SELECT query, or (3) query
00178      *  the SQL database to access one or more of the values that are stored the the SQL table
00179      *  corresponding to this object.
00180      */
00181     static const util::mysql::SqlTableDescription<XFloat> sql_table_description_;
00182
00183   private:
00184
00185     /* Private Static Data Members - GE KML Files: */
00186
00187     /** The global KML-file-generation-is-enabled flag.
00188      *
00189      *  The value of this field determines whether or not any KML files will be written to the
00190      *  local filesystem. If TRUE, then KML files will be written each time a complete set of dive
00191      *  data is received from a deployed instrument. Otherwise, if FALSE, no KML files will be
00192      *  written to the local filesystem.
00193      *
00194      *  The value of this field is set as a result of parsing the command-line arguments and/or
00195      *  configuration file options during application initialization.
00196      */
00197     static bool kml_file_generation_is_enabled_;
00198
00199     /** The KML file directory.
00200      *
00201      */
00202     static path kml_directory_path_;
00203
00204   private:
00205
00206     /* Private Static Data Members - Startup Tasks: */
00207
00208     /** A list of instrument identifiers for which to enable PHY file generation.
00209      *
00210      *  On startup, the list of instruments contained by this vector will be used to set the
```

```
00211      *  \ref XFloatDiveData::phy_file_generation_is_enabled_ at the start of execution of the mail
00212      *  retrieval thread. Subsequently, the dive data for these instruments will be reprocessed in
00213      *  order generate PHY files for previous dives.
00214      *
00215      *  The value of this field is set as a result of parsing the command-line arguments and/or
00216      *  configuration file options during application initialization.
00217      */
00218     static vector<unsigned int> instrument_ids_to_enable_phy_generation_;
00219
00220     /** A list of instrument identifiers for which to disable PHY file generation.
00221      *
00222      *  On startup, the list of instruments contained by this vector will be used to set the
00223      *  \ref XFloatDiveData::phy_file_generation_is_enabled_ at the start of execution of the mail
00224      *  retrieval thread.
00225      *
00226      *  The value of this field is set as a result of parsing the command-line arguments and/or
00227      *  configuration file options during application initialization.
00228      */
00229     static vector<unsigned int> instrument_ids_to_disable_phy_generation_;
00230
00231   private:
00232
00233     /* Private Static Data Members - Status Files: */
00234
00235     /** The global status-file-generation-is-enabled flag.
00236      *
00237      *  If TRUE, then status files will be written.
00238      */
00239     static bool status_file_generation_is_enabled_;
00240
00241     /** The global status file directory path.
00242      *
00243      *  All status files will be written to this directory.
00244      */
00245     static path status_directory_path_;
00246
00247   private:
00248
00249     /* Private Data Members: */
00250
00251     /** The enumerated Argo instrument type identifier.
00252      *
00253      *  This field is used to identify the specific type of instrument i.e. Solo-II, Deep Argo,
00254      *  or ALAMO/Hurricane floats. The value of this field may affect the behavior of some
00255      *  aspects of the application. For example, the manner in which some Iridium Short Burst Data
00256      *  (SBD) messages are decoded is dependent on the specific type of instrument.
00257      */
00258     XArgoInstrumentType_t argo_instrument_type_;
00259
00260     /** The dive-number-to-dive-data-object map.
00261      *
00262      *  This object is a flat map that contains shared references to all of the incomplete dive
00263      *  data objects that belong to this instrument. The dive data objects are uniquely indexed
00264      *  by the dive or cycle number. The map acts as a dynamic, local cache of the incomplete
00265      *  instrument dive data.
00266      *
00267      *  Each time a thread requests the dive data object from the float object using the @ref
00268      *  GetDiveData() function, the dive or cycle number is used to look up the dive data object
00269      *  in this map. If the corresponding dive data object does already exist in the map, it will
00270      *  be created and added to the map before the shared reference is returned.
00271      */
00272     flat_map<int, shared_ptr<XFloatDiveData> > dive2data_map_;
00273
00274     /** The reported instrument firmware version.
00275      *
00276      */
00277     uint32_t fw_version_;
00278
00279     /** The instrument metadata.
00280      *
00281      *  This is a shared reference to this instrument metadata object corresponding to this
00282      *  instrument. During application initialization the metadata object is constructed from
00283      *  the metadata file on the local filesystem. If there is no metadata file corresponding to
00284      *  to this instrument, then the reference will be NULL.
00285      *
00286      *  This object contains the metadata for this instrument. It is object is used, in part, to
00287      *  determine whether the instrument has been deployed, and consequently, whether or not a
00288      *  PHY file should be generated once a the complete data for a dive is received. If the
00289      *  instrument object does not have a corresponding metadata object, then PHY files will NOT
00290      *  be generated for this instrument
00291      *
00292      */
00293     shared_ptr<XFloatMetadata> metadata_;
00294
00295     /** The date/time of startup of the instrument.
00296      *
00297      *  This field represents the most recent date/time that the instrument was started up. This
00298      *  field is used, in part, to validate incoming message from the instrument. Messages that
00299      *  have either and Iridium position or email header date/time that is subsequent to the value
00300      *  of this field will be discarded.
00301      *
00302      *  Generally, the value of this field is assigned from the date/time reported by the GPS
00303      *  message that is reported as a result of the instrument self-test when the instrument is
00304      *  started. However, if the instrument is unable to obtain a valid GPS fix, then the value
00305      *  of this field needs to be set by alternative means.
00306      *
00307      *  @todo       Implement an alternative method to assign the value of this field if the
00308      *              date/time reported by the start-of-mission GPS position is invalid or
00309      *              unavailable.
00310      */
00311     ptime start_of_mission_datetime_;
00312
00313     /** The X-Message format major version number.
00314      *
00315      *  This field indicates the X-Message version used by this instrument's firmware. It affects
00316      *  the manner in which the X-Messages are decoded that reflect the iterative version of the
00317      *  X-Message format.
00318      */
```

```
00319     uint8_t xmsg_version_major_;
00320
00321     /** The X-Message format minor version number.
00322      *
00323      *  This field indicates the X-Message version used by this instrument's firmware. It affects
00324      *  the manner in which the X-Messages are decoded that reflect the iterative version of the
00325      *  X-Message format.
00326      */
00327     uint8_t xmsg_version_minor_;
00328
00329   private:
00330
00331     /* Private Data Members - GE KML Files: */
00332
00333     /** The GE KML document.
00334      *
00335      */
00336     kmldom::DocumentPtr kml_document_;
00337
00338     /** The KML document is updated flag.
00339      *
00340      */
00341     bool kml_document_updated_;
00342
00343     /** The KMLfile path.
00344      *
00345      */
00346     path kml_file_path_;
00347
00348     /** The KML file is updated flag
00349      *
00350      */
00351     bool kml_file_updated_;
00352
00353   private:
00354
00355     /* Private Data Members - GPS: */
00356
00357     /** The latest GPS position date/time.
00358      *
00359      */
00360     ptime latest_gps_position_datetime_;
00361
00362     /** The latest GPS position dive number.
00363      *
00364      */
00365     int latest_gps_position_dive_number_;
00366
00367     /** The latest GPS position latitude.
00368      *
00369      */
00370     double latest_gps_position_latitude_;
00371
00372     /** The latest GPS position longitude.
00373      *
00374      */
00375     double latest_gps_position_longitude_;
00376
00377   private:
00378
00379     /* Private Data Members - Iridium: */
00380
00381     /** The latest iridium position circular error probable radius.
00382      *
00383      */
00384     int latest_iridium_position_cep_radius_;
00385
00386     /** The latest iridium position date/time.
00387      *
00388      */
00389     ptime latest_iridium_position_datetime_;
00390
00391     /** The latest iridium position dive number.
00392      *
00393      */
00394     int latest_iridium_position_dive_number_;
00395
00396     /** The latest iridium position latitude.
00397      *
00398      */
00399     double latest_iridium_position_latitude_;
00400
00401     /** The latest iridium position longitude.
00402      *
00403      */
00404     double latest_iridium_position_longitude_;
00405
00406     /** The latest iridium position Mobile-Originated Message Number (MOMSN).
00407      *
00408      */
00409     int latest_iridium_sbd_momsn_;
00410
00411     /** The latest iridium position Mobile-Terminated Message Number (MTMSN).
00412      *
00413      */
00414     int latest_iridium_sbd_mtmsn_;
00415
00416     /** The latest startup iridium position date/time.
00417      *
00418      */
00419     ptime latest_iridium_startup_position_datetime_;
00420
00421     /** The latest startup iridium position Mobile-Originated Message Number (MOMSN).
00422      *
00423      */
00424     int latest_iridium_startup_sbd_momsn_;
00425
00426   private:
```

```
00427
00428     /* Private Data Members - NetCDF Files: */
00429
00430     /** The NetCDF-file-generation-is-enabled flag.
00431      *
00432      */
00433     bool netcdf_file_generation_is_enabled_;
00434
00435   private:
00436
00437     /* Private Data Members - PHY Files: */
00438
00439     /** The PHY-file-generation-enabled flag.
00440      *
00441      *  This field determines, in part, whether or not a PHY file will be generated once the
00442      *  complete data for a single dive is received from the instrument. If FALSE, then PHY files
00443      *  will NOT be generated for this instrument. Otherwise, if TRUE, PHY files for this
00444      *  instrument will be generated only if the other requisite conditions are met.
00445      */
00446     bool phy_file_generation_is_enabled_;
00447
00448   private:
00449
00450     /* Private Data Members - Status Files: */
00451
00452     /** The status file path.
00453      *
00454      */
00455     path status_file_path_;
00456
00457     /** The status-file-is-updated flag.
00458      *
00459      */
00460     bool status_file_updated_;
00461
00462   private:
00463
00464     /* Private Data Members - Sensor Gains and Offsets: */
00465
00466     /** The pressure sensitivity gain factor, in dBar/LSB.
00467      *
00468      *  This field is used for decoding the drift or profile data encoded in the the Iridium Short
00469      *  Burst Data (SBD) messages. It along with the corresponding offset, is used to convert the
00470      *  raw sensor counts into scientific units.
00471      */
00472     uint16_t gain_pressure_;
00473
00474     /** The pressure sensitivity gain factor for surface data, in dBar/LSB.
00475      *
00476      *  This field is used for decoding the surface data encoded in the the Iridium Short Burst
00477      *  Data (SBD) messages. It along with the corresponding offset, is used to convert the raw
00478      *  sensor counts into scientific units.
00479      */
00480     uint16_t gain_pressure_surface_;
00481
00482     /** The salinity sensitivity scale factor, in PSU/LSB.
00483      *
00484      *  This field is used for decoding the drift or profile data encoded in the the Iridium Short
00485      *  Burst Data (SBD) messages. It along with the corresponding offset, is used to convert the
00486      *  raw sensor counts into scientific units.
00487      */
00488     uint16_t gain_salinity_;
00489
00490     /** The salinity sensitivity scale factor for surface data, in PSU/LSB.
00491      *
00492      *  This field is used for decoding the surface data encoded in the the Iridium Short Burst
00493      *  Data (SBD) messages. It along with the corresponding offset, is used to convert the raw
00494      *  sensor counts into scientific units.
00495      */
00496     uint16_t gain_salinity_surface_;
00497
00498     /** The temperature sensitivity gain factor, in degC/LSB.
00499      *
00500      *  This field is used for decoding the drift or profile data encoded in the the Iridium Short
00501      *  Burst Data (SBD) messages. It along with the corresponding offset, is used to convert the
00502      *  raw sensor counts into scientific units.
00503      */
00504     uint16_t gain_temperature_;
00505
00506     /** The temperature sensitivity gain factor for surface data, in degC/LSB.
00507      *
00508      *  This field is used for decoding the surface data encoded in the the Iridium Short Burst
00509      *  Data (SBD) messages. It along with the corresponding offset, is used to convert the raw
00510      *  sensor counts into scientific units.
00511      */
00512     uint16_t gain_temperature_surface_;
00513
00514     /** The pressure offset, in dBar.
00515      *
00516      *  This field is used for decoding the drift or profile data encoded in the the Iridium Short
00517      *  Burst Data (SBD) messages. It along with the corresponding gain, is used to convert the
00518      *  raw sensor counts into scientific units.
00519      */
00520     uint16_t offset_pressure_;
00521
00522     /** The pressure offset for surface data, in dBar.
00523      *
00524      *  This field is used for decoding the surface data encoded in the the Iridium Short Burst
00525      *  Data (SBD) messages. It along with the corresponding gain, is used to convert the raw
00526      *  sensor counts into scientific units.
00527      */
00528     uint16_t offset_pressure_surface_;
00529
00530     /** The salinity offset, in PSU.
00531      *
00532      *  This field is used for decoding the drift or profile data encoded in the the Iridium Short
00533      *  Burst Data (SBD) messages. It along with the corresponding gain, is used to convert the
00534      *  raw sensor counts into scientific units.
```

```
00535      */
00536     uint16_t offset_salinity_;
00537
00538     /** The salinity offset for surface data, in PSU.
00539      *
00540      *  This field is used for decoding the surface data encoded in the the Iridium Short Burst
00541      *  Data (SBD) messages. It along with the corresponding gain, is used to convert the raw
00542      *  sensor counts into scientific units.
00543      */
00544     uint16_t offset_salinity_surface_;
00545
00546     /** The temperature offset, in degC.
00547      *
00548      *  This field is used for decoding the drift or profile data encoded in the the Iridium Short
00549      *  Burst Data (SBD) messages. It along with the corresponding gain, is used to convert the
00550      *  raw sensor counts into scientific units.
00551      */
00552     uint16_t offset_temperature_;
00553
00554     /** The temperature offset for surface data, in degC.
00555      *
00556      *  This field is used for decoding the surface data encoded in the the Iridium Short Burst
00557      *  Data (SBD) messages. It along with the corresponding gain, is used to convert the raw
00558      *  sensor counts into scientific units.
00559      */
00560     uint16_t offset_temperature_surface_;
00561
00562   public:
00563
00564     /* Public Static Member Functions */
00565
00566     static vector<shared_ptr<XFloat> > GetAllCachedFloats();
00567
00568     static shared_ptr<XFloat> GetFloat(int instrument_id);
00569
00570     static void LoadLocalCacheFromDatabase();
00571
00572     static void LoadMetadataFromFiles();
00573
00574     static void SaveLocalCacheToDatabase();
00575
00576     static shared_ptr<XFloat> SelectObjectFromDatabase(
00577         const string& where = "",
00578         const string& groupBy = "",
00579         const string& orderBy = "",
00580         const unsigned int& offset = 0);
00581
00582     static vector<shared_ptr<XFloat> > SelectObjectsFromDatabase(
00583         const string& where = "",
00584         const string& groupBy = "",
00585         const string& orderBy = "",
00586         const unsigned int& limit = 0,
00587         const unsigned int& offset = 0);
00588
00589     static vector<shared_ptr<XFloat> > SelectAllObjectsFromDatabase(
00590         const string& groupBy = "",
00591         const string& orderBy = "",
00592         const unsigned int& limit = 0,
00593         const unsigned int& offset = 0);
00594
00595     template<typename TT> static TT SelectValueFromDatabase(
00596         const string& what,
00597         const string& where = "",
00598         const string& groupBy = "",
00599         const string& orderBy = "",
00600         const unsigned int& offset = 0);
00601
00602     static void StartupTasks();
00603
00604   private:
00605
00606     /* Private Static Member Functions: */
00607
00608     static kmldom::DocumentPtr GenerateKmlMasterDocument();
00609
00610     static string GenerateKmlMasterString();
00611
00612     static void WriteKmlMasterFile();
00613
00614   public:
00615
00616     /* Public Member Functions */
00617
00618     XFloat(
00619         int whoi,
00620         long long imei = XFLT_ID_UNKNOWN,
00621         int wmo = XFLT_ID_UNKNOWN,
00622         int dac = XFLT_ID_UNKNOWN);
00623
00624     XFloat();
00625
00626     ~XFloat();
00627
00628     vector<shared_ptr<XFloatDiveData> > GetAllDiveData();
00629
00630     shared_ptr<XFloatDiveData> GetDiveData(int dive, bool load_from_local_cache_only = false);
00631
00632     vector<shared_ptr<x::XGPS> > GetGpsPositions() const;
00633
00634     path GetListingDirectoryPath() const;
00635
00636     void ResetCachedDiveData(int dive) ATTR_NO_EXCEPTIONS;
00637
00638     void ResetDiveData();
00639
00640   private:
00641
00642     /* Private Member Functions: */
```

```
00643
00644     kmldom::DocumentPtr GenerateKmlDocument(
00645       bool set_default_kml_style = true,
00646       bool visibility = true) const;
00647
00648     string GenerateKmlString();
00649
00650     string GenerateStatusString() const;
00651
00652     void SaveObjectToDatabase() const;
00653
00654     bool WriteKmlFile();
00655
00656     void WriteStatusFile();
00657
00658   public:
00659
00660     /* Public Static Accessor Methods: */
00661
00662     static const vector<unsigned int>& instrument_ids_to_disable_phy_generation();
00663
00664     static const vector<unsigned int>& instrument_ids_to_enable_phy_generation();
00665
00666   public:
00667
00668     /* Public Accessor Methods: */
00669
00670     /** Get the enumerated Argo instrument type identifier.
00671      *
00672      *  @note       This object will be locked for the scope of the execution of this function.
00673      *              This function places a lock guard on the mutual exclusion (mutex) lock,
00674      *              thereby locking the float object and preventing access or modification of the
00675      *              object by any other thread. Any thread that does attempt to access or modify
00676      *              this object will block until the lock is released.
00677      *
00678      *  @return     The enumerated Argo instrument type identifier.
00679      */
00680     uint8_t argo_instrument_type() const {
00681
00682       // Get an exclusive lock.
00683       lock_guard<recursive_mutex> guard(mutex_);
00684
00685       return argo_instrument_type_;
00686     }
00687
00688     /** Get the firmware version.
00689      *
00690      *  @note       This object will be locked for the scope of the execution of this function.
00691      *              This function places a lock guard on the mutual exclusion (mutex) lock,
00692      *              thereby locking the float object and preventing access or modification of the
00693      *              object by any other thread. Any thread that does attempt to access or modify
00694      *              this object will block until the lock is released.
00695      *
00696      *  @return   The firmware version.
00697      */
00698     uint32_t fw_version() const {
00699
00700       // Get an exclusive lock.
00701       lock_guard<recursive_mutex> guard(mutex_);
00702
00703       return fw_version_;
00704     }
00705
00706     /** Get the pressure gain value, in dBar/LSB.
00707      *
00708      *  @note       This object will be locked for the scope of the execution of this function.
00709      *              This function places a lock guard on the mutual exclusion (mutex) lock,
00710      *              thereby locking the float object and preventing access or modification of the
00711      *              object by any other thread. Any thread that does attempt to access or modify
00712      *              this object will block until the lock is released.
00713      *
00714      *  @return     The pressure gain value, in dBar/LSB.
00715      */
00716     uint16_t gain_pressure() const ATTR_DEPRECATED {
00717       // Get an exclusive lock.
00718       lock_guard<recursive_mutex> guard(mutex_);
00719
00720       return gain_pressure_;
00721     }
00722
00723     /** Get the pressure gain value for surface data, in dBar/LSB.
00724      *
00725      *  @note       This object will be locked for the scope of the execution of this function.
00726      *              This function places a lock guard on the mutual exclusion (mutex) lock,
00727      *              thereby locking the float object and preventing access or modification of the
00728      *              object by any other thread. Any thread that does attempt to access or modify
00729      *              this object will block until the lock is released.
00730      *
00731      *  @return     The pressure gain value, in dBar/LSB.
00732      */
00733     uint16_t gain_pressure_surface() const ATTR_DEPRECATED {
00734       // Get an exclusive lock.
00735       lock_guard<recursive_mutex> guard(mutex_);
00736
00737       return gain_pressure_surface_;
00738     }
00739
00740     /** Get the salinity gain value, in PSU/LSB.
00741      *
00742      *  @note       This object will be locked for the scope of the execution of this function.
00743      *              This function places a lock guard on the mutual exclusion (mutex) lock,
00744      *              thereby locking the float object and preventing access or modification of the
00745      *              object by any other thread. Any thread that does attempt to access or modify
00746      *              this object will block until the lock is released.
00747      *
00748      *  @return     The salinity gain value, in PSU/LSB.
00749      */
00750     uint16_t gain_salinity() const ATTR_DEPRECATED {
```

```
00751       // Get an exclusive lock.
00752       lock_guard<recursive_mutex> guard(mutex_);
00753
00754       return gain_salinity_;
00755     }
00756
00757     /** Get the salinity gain value for surface data, in PSU/LSB.
00758      *
00759      *  @note       This object will be locked for the scope of the execution of this function.
00760      *              This function places a lock guard on the mutual exclusion (mutex) lock,
00761      *              thereby locking the float object and preventing access or modification of the
00762      *              object by any other thread. Any thread that does attempt to access or modify
00763      *              this object will block until the lock is released.
00764      *
00765      *  @return     The salinity gain value, in PSU/LSB.
00766      */
00767     uint16_t gain_salinity_surface() const ATTR_DEPRECATED {
00768       // Get an exclusive lock.
00769       lock_guard<recursive_mutex> guard(mutex_);
00770
00771       return gain_salinity_surface_;
00772     }
00773
00774     /** Get the temperature gain value, in degC/LSB.
00775      *
00776      *  @note       This object will be locked for the scope of the execution of this function.
00777      *              This function places a lock guard on the mutual exclusion (mutex) lock,
00778      *              thereby locking the float object and preventing access or modification of the
00779      *              object by any other thread. Any thread that does attempt to access or modify
00780      *              this object will block until the lock is released.
00781      *
00782      *  @return     The temperature gain value, in degC/LSB.
00783      */
00784     uint16_t gain_temperature() const ATTR_DEPRECATED {
00785       // Get an exclusive lock.
00786       lock_guard<recursive_mutex> guard(mutex_);
00787
00788       return gain_temperature_;
00789     }
00790
00791     /** Get the temperature gain value for surface data, in degC/LSB.
00792      *
00793      *  @note       This object will be locked for the scope of the execution of this function.
00794      *              This function places a lock guard on the mutual exclusion (mutex) lock,
00795      *              thereby locking the float object and preventing access or modification of the
00796      *              object by any other thread. Any thread that does attempt to access or modify
00797      *              this object will block until the lock is released.
00798      *
00799      *  @return     The temperature gain value, in degC/LSB.
00800      */
00801     uint16_t gain_temperature_surface() const ATTR_DEPRECATED {
00802       // Get an exclusive lock.
00803       lock_guard<recursive_mutex> guard(mutex_);
00804
00805       return gain_temperature_surface_;
00806     }
00807
00808     bool is_kml_document_updated() const {
00809
00810       // Get an exclusive lock.
00811       lock_guard<recursive_mutex> guard(mutex_);
00812
00813       return kml_document_updated_;
00814     }
00815
00816     bool is_kml_file_updated() const {
00817
00818       // Get an exclusive lock.
00819       lock_guard<recursive_mutex> guard(mutex_);
00820
00821       return kml_file_updated_;
00822
00823     }
00824
00825     bool is_status_file_updated() const {
00826
00827       // Get an exclusive lock.
00828       lock_guard<recursive_mutex> guard(mutex_);
00829
00830       return status_file_updated_;
00831
00832     }
00833
00834     const kmldom::DocumentPtr& kml_document() const {
00835
00836       // Get an exclusive lock.
00837       lock_guard<recursive_mutex> guard(mutex_);
00838
00839       return kml_document_;
00840     }
00841
00842     const path& kml_file_path() const {
00843
00844       // Get an exclusive lock.
00845       lock_guard<recursive_mutex> guard(mutex_);
00846
00847       return kml_file_path_;
00848     }
00849
00850     int latest_gps_position_dive_number() const {
00851
00852       // Get an exclusive lock.
00853       lock_guard<recursive_mutex> guard(mutex_);
00854
00855       return latest_gps_position_dive_number_;
00856     }
00857
00858     ptime latest_gps_position_datetime() const {
```

```
00859
00860       // Get an exclusive lock.
00861       lock_guard<recursive_mutex> guard(mutex_);
00862
00863       return latest_gps_position_datetime_;
00864     }
00865
00866     double latest_gps_position_latitude() const {
00867
00868       // Get an exclusive lock.
00869       lock_guard<recursive_mutex> guard(mutex_);
00870
00871       return latest_gps_position_latitude_;
00872     }
00873
00874     double latest_gps_position_longitude() const {
00875
00876       // Get an exclusive lock.
00877       lock_guard<recursive_mutex> guard(mutex_);
00878
00879       return latest_gps_position_longitude_;
00880     }
00881
00882     string latest_gps_position_as_iso6709_string() const {
00883       return GenerateIso6709String(latest_gps_position_latitude(), latest_gps_position_longitude());
00884     }
00885
00886     int latest_iridium_position_cep_radius() const {
00887
00888       // Get an exclusive lock.
00889       lock_guard<recursive_mutex> guard(mutex_);
00890
00891       return latest_iridium_position_cep_radius_;
00892     }
00893
00894     ptime latest_iridium_position_datetime() const {
00895
00896       // Get an exclusive lock.
00897       lock_guard<recursive_mutex> guard(mutex_);
00898
00899       return latest_iridium_position_datetime_;
00900     }
00901
00902     int latest_iridium_position_dive_number() const {
00903
00904       // Get an exclusive lock.
00905       lock_guard<recursive_mutex> guard(mutex_);
00906
00907       return latest_iridium_position_dive_number_;
00908     }
00909
00910     double latest_iridium_position_latitude() const {
00911
00912       // Get an exclusive lock.
00913       lock_guard<recursive_mutex> guard(mutex_);
00914
00915       return latest_iridium_position_latitude_;
00916     }
00917
00918     double latest_iridium_position_longitude() const {
00919
00920       // Get an exclusive lock.
00921       lock_guard<recursive_mutex> guard(mutex_);
00922
00923       return latest_iridium_position_longitude_;
00924     }
00925
00926     string latest_iridium_position_as_iso6709_string() const {
00927       return GenerateIso6709String(
00928           latest_iridium_position_latitude(),
00929           latest_iridium_position_longitude());
00930     }
00931
00932     const ptime& latest_iridium_startup_position_datetime() const {
00933
00934       // Get an exclusive lock.
00935       lock_guard<recursive_mutex> guard(mutex_);
00936
00937       return latest_iridium_startup_position_datetime_;
00938     }
00939
00940     int latest_iridium_startup_sbd_momsn() const {
00941       return latest_iridium_startup_sbd_momsn_;
00942     }
00943
00944     int latest_sbd_momsn() const {
00945
00946       // Get an exclusive lock.
00947       lock_guard<recursive_mutex> guard(mutex_);
00948
00949       return latest_iridium_sbd_momsn_;
00950     }
00951
00952     int latest_sbd_mtmsn() const {
00953
00954       // Get an exclusive lock.
00955       lock_guard<recursive_mutex> guard(mutex_);
00956
00957       return latest_iridium_sbd_mtmsn_;
00958     }
00959
00960     /** Get the metadata object for this instrument.
00961      *
00962      *  @note       This object will be locked for the scope of the execution of this function.
00963      *              This function places a lock guard on the mutual exclusion (mutex) lock,
00964      *              thereby locking the float object and preventing access or modification of the
00965      *              object by any other thread. Any thread that does attempt to access or modify
00966      *              this object will block until the lock is released.
```

```
00967      *
00968      *  @return     A shared pointer to the metadata object for this instrument.
00969      */
00970     const shared_ptr<XFloatMetadata>& metadata() const {
00971
00972       // Get an exclusive lock.
00973       lock_guard<recursive_mutex> guard(mutex_);
00974
00975       return metadata_;
00976     }
00977
00978     bool netcdf_file_generation_is_enabled() const {
00979       return netcdf_file_generation_is_enabled_;
00980     }
00981
00982     /** Get the pressure offset value for this instrument, in dBar.
00983      *
00984      *  @note       This object will be locked for the scope of the execution of this function.
00985      *              This function places a lock guard on the mutual exclusion (mutex) lock,
00986      *              thereby locking the float object and preventing access or modification of the
00987      *              object by any other thread. Any thread that does attempt to access or modify
00988      *              this object will block until the lock is released.
00989      *
00990      *  @return     The pressure offset value for this instrument, in dBar.
00991      */
00992     uint16_t offset_pressure() const ATTR_DEPRECATED {
00993
00994       // Get an exclusive lock.
00995       lock_guard<recursive_mutex> guard(mutex_);
00996
00997       return offset_pressure_;
00998     }
00999
01000     /** Get the pressure offset value for surface data, in dBar.
01001      *
01002      *  @note       This object will be locked for the scope of the execution of this function.
01003      *              This function places a lock guard on the mutual exclusion (mutex) lock,
01004      *              thereby locking the float object and preventing access or modification of the
01005      *              object by any other thread. Any thread that does attempt to access or modify
01006      *              this object will block until the lock is released.
01007      *
01008      *  @return     The pressure offset value for this instrument, in dBar.
01009      */
01010     uint16_t offset_pressure_surface() const ATTR_DEPRECATED {
01011
01012       // Get an exclusive lock.
01013       lock_guard<recursive_mutex> guard(mutex_);
01014
01015       return offset_pressure_surface_;
01016     }
01017
01018     /** Get the salinity offset value for this instrument, in PSU.
01019      *
01020      *  @note       This object will be locked for the scope of the execution of this function.
01021      *              This function places a lock guard on the mutual exclusion (mutex) lock,
01022      *              thereby locking the float object and preventing access or modification of the
01023      *              object by any other thread. Any thread that does attempt to access or modify
01024      *              this object will block until the lock is released.
01025      *
01026      *  @return     The salinity offset value for this instrument, in PSU.
01027      */
01028     uint16_t offset_salinity() const ATTR_DEPRECATED {
01029       // Get an exclusive lock.
01030       lock_guard<recursive_mutex> guard(mutex_);
01031
01032       return offset_salinity_;
01033     }
01034
01035     /** Get the salinity offset value for surface data, in PSU.
01036      *
01037      *  @note       This object will be locked for the scope of the execution of this function.
01038      *              This function places a lock guard on the mutual exclusion (mutex) lock,
01039      *              thereby locking the float object and preventing access or modification of the
01040      *              object by any other thread. Any thread that does attempt to access or modify
01041      *              this object will block until the lock is released.
01042      *
01043      *  @return     The salinity offset value for this instrument, in PSU.
01044      */
01045     uint16_t offset_salinity_surface() const ATTR_DEPRECATED {
01046
01047       // Get an exclusive lock.
01048       lock_guard<recursive_mutex> guard(mutex_);
01049
01050       return offset_salinity_surface_;
01051     }
01052
01053     /** Get the temperature offset value for this instrument, in degC.
01054      *
01055      *  @note       This object will be locked for the scope of the execution of this function.
01056      *              This function places a lock guard on the mutual exclusion (mutex) lock,
01057      *              thereby locking the float object and preventing access or modification of the
01058      *              object by any other thread. Any thread that does attempt to access or modify
01059      *              this object will block until the lock is released.
01060      *
01061      *  @return     The temperature offset value for this instrument, in degC.
01062      */
01063     uint16_t offset_temperature() const ATTR_DEPRECATED {
01064
01065       // Get an exclusive lock.
01066       lock_guard<recursive_mutex> guard(mutex_);
01067
01068       return offset_temperature_;
01069     }
01070
01071     /** Get the temperature offset value for surface data, in degC.
01072      *
01073      *  @note       This object will be locked for the scope of the execution of this function.
01074      *              This function places a lock guard on the mutual exclusion (mutex) lock,
```

```
01075      *              thereby locking the float object and preventing access or modification of the
01076      *              object by any other thread. Any thread that does attempt to access or modify
01077      *              this object will block until the lock is released.
01078      *
01079      *  @return     The temperature offset value for this instrument, in degC.
01080      */
01081     uint16_t offset_temperature_surface() const ATTR_DEPRECATED {
01082
01083       // Get an exclusive lock.
01084       lock_guard<recursive_mutex> guard(mutex_);
01085
01086       return offset_temperature_surface_;
01087     }
01088
01089     /** Is PHY file generation enabled?
01090      *
01091      *  @note       This object will be locked for the scope of the execution of this function.
01092      *              This function places a lock guard on the mutual exclusion (mutex) lock,
01093      *              thereby locking the float object and preventing access or modification of the
01094      *              object by any other thread. Any thread that does attempt to access or modify
01095      *              this object will block until the lock is released.
01096      *
01097      *  @return     TRUE if PHY file generation is enabled, or FALSE otherwise.
01098      */
01099     bool phy_file_generation_is_enabled() const {
01100
01101       // Get an exclusive lock.
01102       lock_guard<recursive_mutex> guard(mutex_);
01103
01104       return phy_file_generation_is_enabled_;
01105     }
01106
01107     /** Get the date/time of the start-of-mission for this instrument.
01108      *
01109      *  @note       This object will be locked for the scope of the execution of this function.
01110      *              This function places a lock guard on the mutual exclusion (mutex) lock,
01111      *              thereby locking the float object and preventing access or modification of the
01112      *              object by any other thread. Any thread that does attempt to access or modify
01113      *              this object will block until the lock is released.
01114      *
01115      *  @return     The date/time of the start-of-mission for this instrument.
01116      */
01117     ptime start_of_mission_datetime() const {
01118
01119       // Get an exclusive lock.
01120       lock_guard<recursive_mutex> guard(mutex_);
01121
01122       return start_of_mission_datetime_;
01123     }
01124
01125     const path& status_file_path() const {
01126
01127       // Get an exclusive lock.
01128       lock_guard<recursive_mutex> guard(mutex_);
01129
01130       return status_file_path_;
01131     }
01132
01133     /** Get the X-Message major version identifier for this instrument.
01134      *
01135      *  @note       This object will be locked for the scope of the execution of this function.
01136      *              This function places a lock guard on the mutual exclusion (mutex) lock,
01137      *              thereby locking the float object and preventing access or modification of the
01138      *              object by any other thread. Any thread that does attempt to access or modify
01139      *              this object will block until the lock is released.
01140      *
01141      *  @return     The X-Message major version identifier for this instrument.
01142      */
01143     uint8_t xmsg_version_major() const {
01144
01145       // Get an exclusive lock.
01146       lock_guard<recursive_mutex> guard(mutex_);
01147
01148       return xmsg_version_major_;
01149     }
01150
01151     /** Get the X-Message minor version identifier for this instrument.
01152      *
01153      *  @note       This object will be locked for the scope of the execution of this function.
01154      *              This function places a lock guard on the mutual exclusion (mutex) lock,
01155      *              thereby locking the float object and preventing access or modification of the
01156      *              object by any other thread. Any thread that does attempt to access or modify
01157      *              this object will block until the lock is released.
01158      *
01159      *  @return     The X-Message minor version identifier for this instrument.
01160      */
01161     uint8_t xmsg_version_minor() const {
01162
01163       // Get an exclusive lock.
01164       lock_guard<recursive_mutex> guard(mutex_);
01165
01166       return xmsg_version_minor_;
01167     }
01168
01169   private:
01170
01171     /* Private Mutator Methods: */
01172
01173     /** Set the enumerated Argo instrument type identifier.
01174      *
01175      *  @note       This object will be locked for the scope of the execution of this function.
01176      *              This function places a lock guard on the mutual exclusion (mutex) lock,
01177      *              thereby locking the float object and preventing access or modification of the
01178      *              object by any other thread. Any thread that does attempt to access or modify
01179      *              this object will block until the lock is released.
01180      *
01181      *  @param[in]  argo_instrument_type    The enumerated Argo instrument type identifier to set.
01182      */
```

```
01183     void set_argo_instrument_type(uint8_t argo_instrument_type) {
01184
01185       // Get an exclusive lock.
01186       lock_guard<recursive_mutex> guard(mutex_);
01187
01188       argo_instrument_type_ = static_cast<XArgoInstrumentType_t>(argo_instrument_type);
01189     }
01190
01191     /** Set the instrument firmware version.
01192      *
01193      *  @note       This object will be locked for the scope of the execution of this function.
01194      *              This function places a lock guard on the mutual exclusion (mutex) lock,
01195      *              thereby locking the float object and preventing access or modification of the
01196      *              object by any other thread. Any thread that does attempt to access or modify
01197      *              this object will block until the lock is released.
01198      *
01199      *  @param[in]  fw_version  The firmware version
01200      */
01201     void set_fw_version(uint32_t fw_version) {
01202
01203       // Get an exclusive lock.
01204       lock_guard<recursive_mutex> guard(mutex_);
01205
01206       fw_version_ = fw_version;
01207     }
01208
01209     /** Set the instrument pressure gain value, in dBar/LSB.
01210      *
01211      *  @note       This object will be locked for the scope of the execution of this function.
01212      *              This function places a lock guard on the mutual exclusion (mutex) lock,
01213      *              thereby locking the float object and preventing access or modification of the
01214      *              object by any other thread. Any thread that does attempt to access or modify
01215      *              this object will block until the lock is released.
01216      *
01217      *  @param[in]  gain    The instrument pressure gain value to set, in dBar/LSB.
01218      */
01219     void set_gain_pressure(uint16_t gain) {
01220
01221       // Get an exclusive lock.
01222       lock_guard<recursive_mutex> guard(mutex_);
01223
01224       gain_pressure_ = gain;
01225     }
01226
01227     /** Set the instrument pressure gain value for surface data, in dBar/LSB.
01228      *
01229      *  @note       This object will be locked for the scope of the execution of this function.
01230      *              This function places a lock guard on the mutual exclusion (mutex) lock,
01231      *              thereby locking the float object and preventing access or modification of the
01232      *              object by any other thread. Any thread that does attempt to access or modify
01233      *              this object will block until the lock is released.
01234      *
01235      *  @param[in]  gain    The instrument pressure gain value to set, in dBar/LSB.
01236      */
01237     void set_gain_pressure_surface(uint16_t gain) {
01238
01239       // Get an exclusive lock.
01240       lock_guard<recursive_mutex> guard(mutex_);
01241
01242       gain_pressure_surface_ = gain;
01243     }
01244
01245     /** Set the instrument salinity gain value, in PSU/LSB.
01246      *
01247      *  @note       This object will be locked for the scope of the execution of this function.
01248      *              This function places a lock guard on the mutual exclusion (mutex) lock,
01249      *              thereby locking the float object and preventing access or modification of the
01250      *              object by any other thread. Any thread that does attempt to access or modify
01251      *              this object will block until the lock is released.
01252      *
01253      *  @param[in]  gain    The instrument salinity gain value to set, in PSU/LSB.
01254      */
01255     void set_gain_salinity(uint16_t gain) {
01256
01257       // Get an exclusive lock.
01258       lock_guard<recursive_mutex> guard(mutex_);
01259
01260       gain_salinity_ = gain;
01261     }
01262
01263     /** Set the instrument salinity gain value for surface data, in PSU/LSB.
01264      *
01265      *  @note       This object will be locked for the scope of the execution of this function.
01266      *              This function places a lock guard on the mutual exclusion (mutex) lock,
01267      *              thereby locking the float object and preventing access or modification of the
01268      *              object by any other thread. Any thread that does attempt to access or modify
01269      *              this object will block until the lock is released.
01270      *
01271      *  @param[in]  gain    The instrument salinity gain value to set, in PSU/LSB.
01272      */
01273     void set_gain_salinity_surface(uint16_t gain) {
01274
01275       // Get an exclusive lock.
01276       lock_guard<recursive_mutex> guard(mutex_);
01277
01278       gain_salinity_surface_ = gain;
01279     }
01280
01281     /** Set the instrument temperature gain value, in degC/LSB.
01282      *
01283      *  @note       This object will be locked for the scope of the execution of this function.
01284      *              This function places a lock guard on the mutual exclusion (mutex) lock,
01285      *              thereby locking the float object and preventing access or modification of the
01286      *              object by any other thread. Any thread that does attempt to access or modify
01287      *              this object will block until the lock is released.
01288      *
01289      *  @param[in]  gain    The instrument temperature gain value to set, in degC/LSB.
01290      */
```

```
01291     void set_gain_temperature(uint16_t gain) {
01292
01293       // Get an exclusive lock.
01294       lock_guard<recursive_mutex> guard(mutex_);
01295
01296       gain_temperature_ = gain;
01297     }
01298
01299     /** Set the instrument temperature gain value for surface data, in degC/LSB.
01300      *
01301      *  @note       This object will be locked for the scope of the execution of this function.
01302      *              This function places a lock guard on the mutual exclusion (mutex) lock,
01303      *              thereby locking the float object and preventing access or modification of the
01304      *              object by any other thread. Any thread that does attempt to access or modify
01305      *              this object will block until the lock is released.
01306      *
01307      *  @param[in]  gain    The instrument temperature gain value to set, in degC/LSB.
01308      */
01309     void set_gain_temperature_surface(uint16_t gain) {
01310
01311       // Get an exclusive lock.
01312       lock_guard<recursive_mutex> guard(mutex_);
01313
01314       gain_temperature_surface_ = gain;
01315     }
01316
01317     void set_kml_document(const kmldom::DocumentPtr& kml_document) {
01318
01319       // Get an exclusive lock.
01320       lock_guard<recursive_mutex> guard(mutex_);
01321
01322       kml_document_ = kml_document;
01323     }
01324
01325     void set_kml_file_path(const path& kml_file_path) {
01326
01327       // Get an exclusive lock.
01328       lock_guard<recursive_mutex> guard(mutex_);
01329
01330       kml_file_path_ = kml_file_path;
01331     }
01332
01333     void set_kml_file_updated(bool kml_file_updated = true) {
01334
01335       // Get an exclusive lock.
01336       lock_guard<recursive_mutex> guard(mutex_);
01337
01338       kml_file_updated_ = kml_file_updated;
01339     }
01340
01341     void set_kml_document_updated(bool kml_document_updated = true) {
01342
01343       // Get an exclusive lock.
01344       lock_guard<recursive_mutex> guard(mutex_);
01345
01346       kml_document_updated_ = kml_document_updated;
01347     }
01348
01349     void set_latest_gps_position_dive_number(int latest_gps_position_dive_number) {
01350
01351       // Get an exclusive lock.
01352       lock_guard<recursive_mutex> guard(mutex_);
01353
01354       latest_gps_position_dive_number_ = latest_gps_position_dive_number;
01355     }
01356
01357     void set_latest_gps_position_datetime(const ptime& latest_gps_position_datetime) {
01358
01359       // Get an exclusive lock.
01360       lock_guard<recursive_mutex> guard(mutex_);
01361
01362       latest_gps_position_datetime_ = latest_gps_position_datetime;
01363     }
01364
01365     void set_latest_gps_position_latitude(double latest_gps_position_latitude) {
01366
01367       // Get an exclusive lock.
01368       lock_guard<recursive_mutex> guard(mutex_);
01369
01370       latest_gps_position_latitude_ = latest_gps_position_latitude;
01371     }
01372
01373     void set_latest_gps_position_longitude(double latest_gps_position_longitude) {
01374
01375       // Get an exclusive lock.
01376       lock_guard<recursive_mutex> guard(mutex_);
01377
01378       latest_gps_position_longitude_ = latest_gps_position_longitude;
01379     }
01380
01381     void set_latest_iridium_position_cep_radius(int latest_iridium_position_cep_radius) {
01382
01383       // Get an exclusive lock.
01384       lock_guard<recursive_mutex> guard(mutex_);
01385
01386       latest_iridium_position_cep_radius_ = latest_iridium_position_cep_radius;
01387     }
01388
01389     void set_latest_iridium_position_datetime(const ptime& latest_iridium_position_datetime) {
01390
01391       // Get an exclusive lock.
01392       lock_guard<recursive_mutex> guard(mutex_);
01393
01394       latest_iridium_position_datetime_ = latest_iridium_position_datetime;
01395     }
01396
01397     void set_latest_iridium_position_dive_number(int latest_iridium_position_dive_number) {
01398
```

```
01399       // Get an exclusive lock.
01400       lock_guard<recursive_mutex> guard(mutex_);
01401
01402       latest_iridium_position_dive_number_ = latest_iridium_position_dive_number;
01403     }
01404
01405     void set_latest_iridium_position_latitude(double latest_iridium_position_latitude) {
01406
01407       // Get an exclusive lock.
01408       lock_guard<recursive_mutex> guard(mutex_);
01409
01410       latest_iridium_position_latitude_ = latest_iridium_position_latitude;
01411     }
01412
01413     void set_latest_iridium_position_longitude(double latest_iridium_position_longitude) {
01414
01415       // Get an exclusive lock.
01416       lock_guard<recursive_mutex> guard(mutex_);
01417
01418       latest_iridium_position_longitude_ = latest_iridium_position_longitude;
01419     }
01420
01421     void set_latest_iridium_startup_position_datetime(
01422         const ptime& latest_iridium_startup_position_datetime) {
01423
01424       // Get an exclusive lock.
01425       lock_guard<recursive_mutex> guard(mutex_);
01426
01427       latest_iridium_startup_position_datetime_ = latest_iridium_startup_position_datetime;
01428     }
01429
01430     void set_latest_iridium_startup_sbd_momsn(int latest_iridium_startup_sbd_momsn) {
01431       latest_iridium_startup_sbd_momsn_ = latest_iridium_startup_sbd_momsn;
01432     }
01433
01434     void set_latest_sbd_momsn(int latest_sbd_momsn) {
01435
01436       // Get an exclusive lock.
01437       lock_guard<recursive_mutex> guard(mutex_);
01438
01439       latest_iridium_sbd_momsn_ = latest_sbd_momsn;
01440     }
01441
01442     void set_latest_sbd_mtmsn(int latest_sbd_mtmsn) {
01443
01444       // Get an exclusive lock.
01445       lock_guard<recursive_mutex> guard(mutex_);
01446
01447       latest_iridium_sbd_mtmsn_ = latest_sbd_mtmsn;
01448     }
01449
01450     /** Assign an instrument metadata object to this X-Float object.
01451      *
01452      *  @note       This object will be locked for the scope of the execution of this function.
01453      *              This function places a lock guard on the mutual exclusion (mutex) lock,
01454      *              thereby locking the float object and preventing access or modification of the
01455      *              object by any other thread. Any thread that does attempt to access or modify
01456      *              this object will block until the lock is released.
01457      *
01458      *  @param[in]  metadata                A shared pointer to the instrument metadata object to
01459      *                                      assign to this X-Float object.
01460      */
01461     void set_metadata(const shared_ptr<XFloatMetadata>& metadata) {
01462
01463       // Get an exclusive lock.
01464       lock_guard<recursive_mutex> guard(mutex_);
01465
01466       metadata_ = metadata;
01467
01468       if(!metadata)
01469         return;
01470
01471       // Update the metadata IMEI.
01472       if(metadata->id_imei() != id_imei())
01473         metadata->set_id_imei(id_imei());
01474
01475       // Update float ID fields
01476       set_id_dac(metadata->id_dac());
01477       set_id_wmo(metadata->id_wmo());
01478     }
01479
01480     void set_netcdf_file_generation_is_enabled(bool netcdf_file_generation_is_enabled) {
01481       netcdf_file_generation_is_enabled_ = netcdf_file_generation_is_enabled;
01482     }
01483
01484     /** Set the instrument pressure offset value, in dBar.
01485      *
01486      *  @note       This object will be locked for the scope of the execution of this function.
01487      *              This function places a lock guard on the mutual exclusion (mutex) lock,
01488      *              thereby locking the float object and preventing access or modification of the
01489      *              object by any other thread. Any thread that does attempt to access or modify
01490      *              this object will block until the lock is released.
01491      *
01492      *  @param[in]  offs    The instrument pressure offset value to set, in dBar.
01493      */
01494     void set_offset_pressure(uint16_t offs) {
01495
01496       // Get an exclusive lock.
01497       lock_guard<recursive_mutex> guard(mutex_);
01498
01499       offset_pressure_ = offs;
01500     }
01501
01502     /** Set the instrument pressure offset value for surface data, in dBar.
01503      *
01504      *  @note       This object will be locked for the scope of the execution of this function.
01505      *              This function places a lock guard on the mutual exclusion (mutex) lock,
01506      *              thereby locking the float object and preventing access or modification of the
```

```
01507      *              object by any other thread. Any thread that does attempt to access or modify
01508      *              this object will block until the lock is released.
01509      *
01510      *  @param[in]  offs    The instrument pressure offset value to set, in dBar.
01511      */
01512     void set_offset_pressure_surface(uint16_t offs) {
01513
01514       // Get an exclusive lock.
01515       lock_guard<recursive_mutex> guard(mutex_);
01516
01517       offset_pressure_surface_ = offs;
01518     }
01519
01520     /** Set the instrument salinity offset value, in PSU.
01521      *
01522      *  @note       This object will be locked for the scope of the execution of this function.
01523      *              This function places a lock guard on the mutual exclusion (mutex) lock,
01524      *              thereby locking the float object and preventing access or modification of the
01525      *              object by any other thread. Any thread that does attempt to access or modify
01526      *              this object will block until the lock is released.
01527      *
01528      *  @param[in]  offs    The instrument salinity offset value to set, in PSU.
01529      */
01530     void set_offset_salinity(uint16_t offs) {
01531
01532       // Get an exclusive lock.
01533       lock_guard<recursive_mutex> guard(mutex_);
01534
01535       offset_salinity_ = offs;
01536     }
01537
01538     /** Set the instrument salinity offset value for surface data, in PSU.
01539      *
01540      *  @note       This object will be locked for the scope of the execution of this function.
01541      *              This function places a lock guard on the mutual exclusion (mutex) lock,
01542      *              thereby locking the float object and preventing access or modification of the
01543      *              object by any other thread. Any thread that does attempt to access or modify
01544      *              this object will block until the lock is released.
01545      *
01546      *  @param[in]  offs    The instrument salinity offset value to set, in PSU.
01547      */
01548     void set_offset_salinity_surface(uint16_t offs) {
01549
01550       // Get an exclusive lock.
01551       lock_guard<recursive_mutex> guard(mutex_);
01552
01553       offset_salinity_surface_ = offs;
01554     }
01555
01556     /** Set the instrument temperature offset value, in degC.
01557      *
01558      *  @note       This object will be locked for the scope of the execution of this function.
01559      *              This function places a lock guard on the mutual exclusion (mutex) lock,
01560      *              thereby locking the float object and preventing access or modification of the
01561      *              object by any other thread. Any thread that does attempt to access or modify
01562      *              this object will block until the lock is released.
01563      *
01564      *  @param[in]  offs    The instrument temperature offset value to set, in degC.
01565      */
01566     void set_offset_temperature(uint16_t offs) {
01567
01568       // Get an exclusive lock.
01569       lock_guard<recursive_mutex> guard(mutex_);
01570
01571       offset_temperature_ = offs;
01572     }
01573
01574     /** Set the instrument temperature offset value for surface data, in degC.
01575      *
01576      *  @note       This object will be locked for the scope of the execution of this function.
01577      *              This function places a lock guard on the mutual exclusion (mutex) lock,
01578      *              thereby locking the float object and preventing access or modification of the
01579      *              object by any other thread. Any thread that does attempt to access or modify
01580      *              this object will block until the lock is released.
01581      *
01582      *  @param[in]  offs    The instrument temperature offset value to set, in degC.
01583      */
01584     void set_offset_temperature_surface(uint16_t offs) {
01585
01586       // Get an exclusive lock.
01587       lock_guard<recursive_mutex> guard(mutex_);
01588
01589       offset_temperature_surface_ = offs;
01590     }
01591
01592     /** Set the PHY-file-generation-is-enabled flag.
01593      *
01594      *  @note       This object will be locked for the scope of the execution of this function.
01595      *              This function places a lock guard on the mutual exclusion (mutex) lock,
01596      *              thereby locking the float object and preventing access or modification of the
01597      *              object by any other thread. Any thread that does attempt to access or modify
01598      *              this object will block until the lock is released.
01599      *
01600      *  @param[in]  is_enabled              The PHY-file-generation-is-enabled flag to set.
01601      */
01602     void set_phy_file_generation_is_enabled(bool is_enabled) {
01603
01604       // Get an exclusive lock.
01605       lock_guard<recursive_mutex> guard(mutex_);
01606
01607       phy_file_generation_is_enabled_ = is_enabled;
01608     }
01609
01610     /** Set the date/time of the start-of-mission for this instrument.
01611      *
01612      *  @note       This object will be locked for the scope of the execution of this function.
01613      *              This function places a lock guard on the mutual exclusion (mutex) lock,
01614      *              thereby locking the float object and preventing access or modification of the
```

```
01615       *              object by any other thread. Any thread that does attempt to access or modify
01616       *              this object will block until the lock is released.
01617       *
01618       *  @param[in]  datetime               The date/time of the start-of-mission for this
01619       *                                     instrument to set.
01620       */
01621      void set_start_of_mission_datetime(const ptime& datetime) {
01622
01623        // Get an exclusive lock.
01624        lock_guard<recursive_mutex> guard(mutex_);
01625
01626        start_of_mission_datetime_ = datetime;
01627      }
01628
01629      void set_status_file_path(const path& path) {
01630
01631        // Get an exclusive lock.
01632        lock_guard<recursive_mutex> guard(mutex_);
01633
01634        status_file_path_ = path;
01635      }
01636
01637      void set_status_file_updated(bool status_file_updated = true) {
01638
01639        // Get an exclusive lock.
01640        lock_guard<recursive_mutex> guard(mutex_);
01641
01642        status_file_updated_ = status_file_updated;
01643      }
01644
01645      /** Set the X-Message major version number.
01646       *
01647       *  @note       This object will be locked for the scope of the execution of this function.
01648       *              This function places a lock guard on the mutual exclusion (mutex) lock,
01649       *              thereby locking the float object and preventing access or modification of the
01650       *              object by any other thread. Any thread that does attempt to access or modify
01651       *              this object will block until the lock is released.
01652       *
01653       *  @param[in]  xmsg_version_major      The X-Message major version number to set.
01654       */
01655      void set_xmsg_version_major(uint8_t xmsg_version_major) {
01656
01657        // Get an exclusive lock.
01658        lock_guard<recursive_mutex> guard(mutex_);
01659
01660        xmsg_version_major_ = xmsg_version_major;
01661      }
01662
01663      /** Set the X-Message minor version number.
01664       *
01665       *  @note       This object will be locked for the scope of the execution of this function.
01666       *              This function places a lock guard on the mutual exclusion (mutex) lock,
01667       *              thereby locking the float object and preventing access or modification of the
01668       *              object by any other thread. Any thread that does attempt to access or modify
01669       *              this object will block until the lock is released.
01670       *
01671       *  @param[in]  xmsg_version_minor      The X-Message minor version number to set.
01672       */
01673      void set_xmsg_version_minor(uint8_t xmsg_version_minor) {
01674
01675        // Get an exclusive lock.
01676        lock_guard<recursive_mutex> guard(mutex_);
01677
01678        xmsg_version_minor_ = xmsg_version_minor;
01679      }
01680
01681
01682
01683 };
01684
01685 } /* namespace xfloat */
01686 } /* namespace iridium */
01687 } /* namespace argo */
01688
01689 #endif /* XFLOAT_H_ */
```

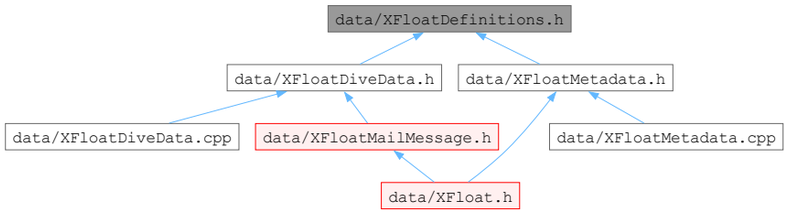
11.20 data/XFloatDefinitions.h File Reference

The XFloat Definitions header file.

```
#include "Iridium.h"
#include "IridiumPosition.h"
#include "xmsg/XAcousticPosition.h"
#include "xmsg/XArgo.h"
#include "xmsg/XCompositeAuxTemp.h"
#include "xmsg/XCompositeDrift.h"
#include "xmsg/XCompositeFall.h"
#include "xmsg/XCompositeHiResProfile.h"
#include "xmsg/XCompositeIridiumPosition.h"
#include "xmsg/XCompositeProfile.h"
#include "xmsg/XCompositePump.h"
#include "xmsg/XCompositeRise.h"
#include "xmsg/XCompositeSurfaceData.h"
#include "xmsg/XCompositeValve.h"
```

```
#include "xmsg/XEngAbort.h"
#include "xmsg/XEngAscent.h"
#include "xmsg/XEngBITFail.h"
#include "xmsg/XEngBITPass.h"
#include "xmsg/XEngDescent.h"
#include "xmsg/XEngDiagnostic.h"
#include "xmsg/XEngDrift.h"
#include "xmsg/XEngIce.h"
#include "xmsg/XEngStandard.h"
#include "xmsg/XEngSurface.h"
#include "xmsg/XGPS.h"
#include "xmsg/XMfaMissionCfg.h"
#include "xmsg/XMissionCmdEcho.h"
#include "xmsg/XWave.h"
#include <vmime/vmime.hpp>
#include <vmime/platforms/posix/posixHandler.hpp>
```

This graph shows which files directly or indirectly include this file:



Namespaces

- namespace argo

  The Argo instrument namespace.
- namespace argo::iridium

  The Argo Iridium instrument namespace.
- namespace argo::iridium::xfloat

  The X-Float instrument namespace.

Macros

- #define XFLT_DAEMON_FLUSH_LOG_SEC (30)

  The interval at which the daemon tread will flush the global logger, in seconds.
- #define **XFLT_DAEMON_THREAD_SPINUP_DELAY_SEC** (10)
- #define XFLT_DIVE_UNKNOWN XMSG_DIVE_UNKNOWN

  The default/bad/invalid dive value.
- #define XFLT_GAIN_INVALID XMSG_GAIN_INVALID

  Default/fill value for invalid Iridium message gain value.
- #define XFLT_ID_UNKNOWN XMSG_ID_UNKNOWN

  The default/bad/invalid unique identifier value.
- #define XFLT_OFFS_INVALID XMSG_OFFS_INVALID

  Default/fill value for invalid Iridium message offset value.
- #define XFLT_PID_INVALID (-1)

  Default/fill value for invalid Iridium message offset value.
- #define XFLT_POS_INVALID_LAT IRI_POS_INVALID_LAT

  Default/fill value for invalid position latitude.
- #define XFLT_POS_INVALID_LON IRI_POS_INVALID_LAT

  Default/fill value for invalid position longitude.
- #define XFLT_QUEUE_SIZE_MAIL_PROCESSING (1024∗8)

  Default mail message processing queue capacity.
- #define **XFLT_QUEUE_SIZE_NC_FILE_WRITER** (1024∗64)
- #define **XFLT_QUEUE_SIZE_PPROCESSING** (1024∗64)
- #define XFLT_QUEUE_SIZE_TELEMETRY (1024∗8)

  Default telemetry decode queue capacity.
- #define XFLT_SBD_SESSION_STATUS_CODE_UNKNOWN (-1)

  The default/fill value for the Iridium session status code.
- #define XFLT_SBD_SIZE_BYTES_UNKNOWN (-1)

  The default/fill value for SBD data size, in bytes.
- #define XFLT_THREAD_SLEEP_MS (100)

  XFloat service threads' idle sleep time, in milliseconds.

11.20.1 Detailed Description

The XFloat Definitions header file.

This file contains definitions that are used by the argo::iridium::xfloat::XFloat class and the classes that are associated with it.

Version

0.6

Date

Jun 9, 2014 4:12:18 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.20.2 Macro Definition Documentation

11.20.2.1 XFLT_DAEMON_FLUSH_LOG_SEC

```
#define XFLT_DAEMON_FLUSH_LOG_SEC (30)
```

The interval at which the daemon tread will flush the global logger, in seconds.

Referenced by main().

11.20.2.2 XFLT_DIVE_UNKNOWN

```
#define XFLT_DIVE_UNKNOWN XMSG_DIVE_UNKNOWN
```

The default/bad/invalid dive value.

Referenced by argo::iridium::xfloat::XFloatTelemetryDecoderService::DecodeMessage(), and argo::iridium::xfloat::XFloatDiveData::GetDi

11.20.2.3 XFLT_GAIN_INVALID

```
#define XFLT_GAIN_INVALID XMSG_GAIN_INVALID
```

Default/fill value for invalid Iridium message gain value.

Referenced by argo::iridium::xfloat::XFloatDiveData::GetGainsAndOffsets(), and argo::iridium::xfloat::XFloatDiveData::GetSurfaceGainsA

11.20.2.4 XFLT_ID_UNKNOWN

```
#define XFLT_ID_UNKNOWN XMSG_ID_UNKNOWN
```

The default/bad/invalid unique identifier value.

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignParentInstrument(), argo::iridium::xfloat::XFloatDiveData::ComputePressureO argo::iridium::xfloat::XFloatDiveData::GetDiveDataMailMessagesFromDatabase(), argo::iridium::xfloat::XFloatDiveData::GetDiveDataObj argo::iridium::xfloat::XFloatMailProcessingService::ProcessMailMsg(), and argo::iridium::xfloat::XFloatMailMessage::WriteFiles().

11.20.2.5 XFLT_OFFS_INVALID

```
#define XFLT_OFFS_INVALID XMSG_OFFS_INVALID
```

Default/fill value for invalid Iridium message offset value.

Referenced by argo::iridium::xfloat::XFloatDiveData::GetGainsAndOffsets(), and argo::iridium::xfloat::XFloatDiveData::GetSurfaceGainsA

11.20.2.6 XFLT_PID_INVALID

```
#define XFLT_PID_INVALID (-1)
```

Default/fill value for invalid Iridium message offset value.

11.20.2.7 XFLT_POS_INVALID_LAT

```
#define XFLT_POS_INVALID_LAT IRI_POS_INVALID_LAT
```

Default/fill value for invalid position latitude.

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignGps().

11.20.2.8 XFLT_POS_INVALID_LON

```
#define XFLT_POS_INVALID_LON IRI_POS_INVALID_LAT
```

Default/fill value for invalid position longitude.

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignGps().

11.20.2.9 XFLT_QUEUE_SIZE_MAIL_PROCESSING

```
#define XFLT_QUEUE_SIZE_MAIL_PROCESSING (1024*8)
```

Default mail message processing queue capacity.

11.20.2.10 XFLT_QUEUE_SIZE_TELEMETRY

```
#define XFLT_QUEUE_SIZE_TELEMETRY (1024*8)
```

Default telemetry decode queue capacity.

11.20.2.11 XFLT_SBD_SESSION_STATUS_CODE_UNKNOWN

```
#define XFLT_SBD_SESSION_STATUS_CODE_UNKNOWN (-1)
```

The default/fill value for the Iridium session status code.

11.20.2.12 XFLT_SBD_SIZE_BYTES_UNKNOWN

```
#define XFLT_SBD_SIZE_BYTES_UNKNOWN (-1)
```

The default/fill value for SBD data size, in bytes.

11.20.2.13 XFLT_THREAD_SLEEP_MS

```
#define XFLT_THREAD_SLEEP_MS (100)
```

XFloat service threads' idle sleep time, in milliseconds.

Referenced by main(), argo::iridium::xfloat::XFloatMailProcessingService::operator()(), argo::iridium::xfloat::XFloatMailRetrievalService::o
argo::iridium::xfloat::XFloatPostProcessingService::operator()(), and argo::iridium::xfloat::XFloatTelemetryDecoderService::operator()().

11.21 XFloatDefinitions.h

Go to the documentation of this file.

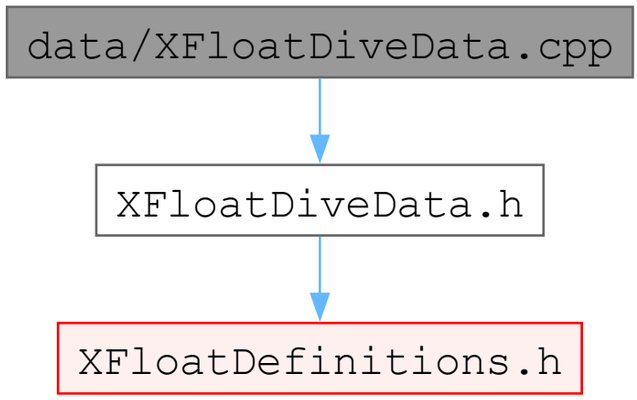
```
00001 /** The XFloat Definitions header file.
00002  *
00003  *  This file contains definitions that are used by the @ref argo::iridium::xfloat::XFloat class
00004  *  and the classes that are associated with it.
00005  *
00006  *  @file
00007  *
00008  *  @version    0.6
00009  *
00010  *  @date       Jun 9, 2014 4:12:18 PM
00011  *
00012  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00013  *              Engineer I,
00014  *              Department of Physical Oceanography,
00015  *              Woods Hole Oceanographic Institution
00016  *
00017  *  @copyright  2014 Woods Hole Oceanographic Institution.
00018  *              Released under the GNU General Public License.
00019  *
00020  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00021  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00022  *  General Public License for more details. You should have received a copy of the GNU General
00023  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00024  */
00025
00026 #ifndef XFLOATDEFINITIONS_H_
00027 #define XFLOATDEFINITIONS_H_
00028
00029 /* Includes: */
00030
00031 #include "Iridium.h"                            // local
00032 #include "IridiumPosition.h"
00033 #include "xmsg/XAcousticPosition.h"
00034 #include "xmsg/XArgo.h"
00035 #include "xmsg/XCompositeAuxTemp.h"
00036 #include "xmsg/XCompositeDrift.h"
00037 #include "xmsg/XCompositeFall.h"
00038 #include "xmsg/XCompositeHiResProfile.h"
00039 #include "xmsg/XCompositeIridiumPosition.h"
00040 #include "xmsg/XCompositeProfile.h"
00041 #include "xmsg/XCompositePump.h"
00042 #include "xmsg/XCompositeRise.h"
00043 #include "xmsg/XCompositeSurfaceData.h"
00044 #include "xmsg/XCompositeValve.h"
00045 #include "xmsg/XEngAbort.h"
00046 #include "xmsg/XEngAscent.h"
00047 #include "xmsg/XEngBITFail.h"
00048 #include "xmsg/XEngBITPass.h"
00049 #include "xmsg/XEngDescent.h"
00050 #include "xmsg/XEngDescent.h"
00051 #include "xmsg/XEngDiagnostic.h"
00052 #include "xmsg/XEngDiagnostic.h"
00053 #include "xmsg/XEngDrift.h"
00054 #include "xmsg/XEngIce.h"
00055 #include "xmsg/XEngStandard.h"
00056 #include "xmsg/XEngSurface.h"
00057 #include "xmsg/XGPS.h"
00058 #include "xmsg/XMfaMissionCfg.h"
00059 #include "xmsg/XMissionCmdEcho.h"
00060 #include "xmsg/XWave.h"
00061 //#include "../utility/NetcdfUtils.h"
00062 #include <vmime/vmime.hpp>                      // vmime
00063 #include <vmime/platforms/posix/posixHandler.hpp>
00064
00065 /* Definitions: */
00066
00067 /** The default/bad/invalid dive value. */
00068 #define XFLT_DIVE_UNKNOWN                       XMSG_DIVE_UNKNOWN
00069
00070 /** Default/fill value for invalid Iridium message gain value. */
00071 #define XFLT_GAIN_INVALID                       XMSG_GAIN_INVALID
00072
00073 /** The default/bad/invalid unique identifier value. */
00074 #define XFLT_ID_UNKNOWN                         XMSG_ID_UNKNOWN
00075
00076 /** Default/fill value for invalid Iridium message offset value. */
00077 #define XFLT_OFFS_INVALID                       XMSG_OFFS_INVALID
00078
00079 /** Default/fill value for invalid Iridium message offset value. */
00080 #define XFLT_PID_INVALID                        (-1)
00081
00082 /** The default/fill value for SBD data size, in bytes.
00083  *
00084  */
00085 #define XFLT_SBD_SIZE_BYTES_UNKNOWN             (-1)
00086
```

```
00087 /** The default/fill value for the Iridium session status code.
00088  *
00089  */
00090 #define XFLT_SBD_SESSION_STATUS_CODE_UNKNOWN    (-1)
00091
00092 /** Default/fill value for invalid position latitude. */
00093 #define XFLT_POS_INVALID_LAT                    IRI_POS_INVALID_LAT
00094
00095 /** Default/fill value for invalid position longitude. */
00096 #define XFLT_POS_INVALID_LON                    IRI_POS_INVALID_LAT
00097
00098 /** Default mail message processing queue capacity. */
00099 #define XFLT_QUEUE_SIZE_MAIL_PROCESSING         (1024*8)
00100
00101 /** Default telemetry decode queue capacity. */
00102 #define XFLT_QUEUE_SIZE_TELEMETRY               (1024*8)
00103
00104 #define XFLT_QUEUE_SIZE_PPROCESSING             (1024*64)
00105
00106 #define XFLT_QUEUE_SIZE_NC_FILE_WRITER          (1024*64)
00107
00108 /** XFloat service threads' idle sleep time, in milliseconds. */
00109 #define XFLT_THREAD_SLEEP_MS                    (100)
00110
00111 /** The interval at which the daemon tread will flush the global logger, in seconds.
00112  *
00113  */
00114 #define XFLT_DAEMON_FLUSH_LOG_SEC               (30)
00115
00116 #define XFLT_DAEMON_THREAD_SPINUP_DELAY_SEC     (10)
00117
00118 /*  Namespace Definitions: */
00119 namespace argo {
00120 namespace iridium {
00121 namespace xfloat {
00122
00123 using boost::container::flat_map;
00124 using boost::container::static_vector;
00125 using boost::program_options::options_description;
00126 using boost::this_thread::sleep;
00127 using boost::thread_group;
00128 using boost::thread;
00129 using boost::cmatch;
00130 using boost::smatch;
00131 using boost::regex;
00132 using boost::regex_match;
00133 using boost::regex_search;
00134
00135 //using netCDF::NcAtt;
00136 //using netCDF::NcDim;
00137 //using netCDF::NcFile;
00138 //using netCDF::NcGroup;
00139 //using netCDF::NcGroupAtt;
00140 //using netCDF::NcType;
00141 //using netCDF::NcVar;
00142
00143 using std::dec;
00144 using std::get;
00145 using std::hex;
00146 using std::istreambuf_iterator;
00147 using std::make_pair;
00148 using std::map;
00149 using std::pair;
00150 using std::setfill;
00151 using std::setw;
00152 using std::tuple;
00153 using std::uppercase;
00154
00155 using vmime::net::fetchAttributes;
00156
00157 } /* namespace xfloat */
00158 } /* namespace iridium */
00159 } /* namespace argo */
00160
00161 #endif /* XFLOATDEFINITIONS_H_ */
```

11.22 data/XFloatDiveData.cpp File Reference

The argo::iridium::xfloat::XFloatDiveData class source file.

```
#include "XFloatDiveData.h"
```
Include dependency graph for XFloatDiveData.cpp:



Namespaces

- namespace argo

  The Argo instrument namespace.
- namespace argo::iridium

  The Argo Iridium instrument namespace.
- namespace argo::iridium::xfloat

  The X-Float instrument namespace.

11.22.1 Detailed Description

The argo::iridium::xfloat::XFloatDiveData class source file.

This is the source file for the argo::iridium::xfloat::XFloatDiveData class. This class contains all of the mail messages and corresponding decoded telemetry data for an instrument for a single dive. Each time a mail message is received by the telemetry decoder thread, argo::iridium::xfloat::XFloatTelemetryDecoderService, the corresponding dive data object is retrieved and the mail message is assigned to it. Then, the binary data from the attached Iridium Short Burst Data (SBD) message is decoded. As a result of decoding the binary message data, an object corresponding to the message type is created and its field are populated from the decoded data in the binary message. The newly created object will be of a type that inherits from util::xmsg::XMessage. A shared reference to the object is assigned to the dive data object.

This class contains the functionality for writing the formatted intermediate and PHY listing files to the local filesystem. Each time a new message is received, the entire intermediate listing file will be written to the local filesystem, overwriting the previous listing file, if it exists. The PHY file is written only when the dive data object is determined to contain the complete data for dive, and only if the instrument has been deployed i.e. it has a metadata file with valid DAC and WMO identifiers.

This file contains the static data members of the class.

Version

0.5

Date

Jul 3, 2014 8:56:05 AM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

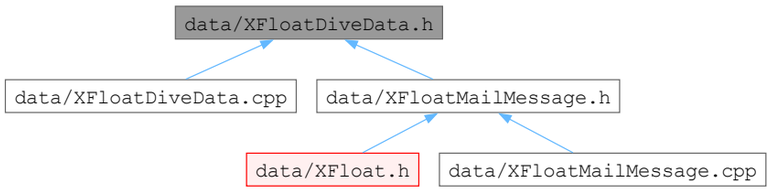
11.23 data/XFloatDiveData.h File Reference

The argo::iridium::xfloat::XFloatDiveData class header file.

```
#include "XFloatDefinitions.h"
```

Include dependency graph for XFloatDiveData.h:

This graph shows which files directly or indirectly include this file:



Classes

- class argo::iridium::xfloat::XFloatDiveData

  The instrument or float dive data class.

Namespaces

- namespace argo

  The Argo instrument namespace.
- namespace argo::iridium

  The Argo Iridium instrument namespace.
- namespace argo::iridium::xfloat

  The X-Float instrument namespace.

11.23.1 Detailed Description

The argo::iridium::xfloat::XFloatDiveData class header file.

This is the header file for the argo::iridium::xfloat::XFloatDiveData class. This class contains all of the mail messages and corresponding decoded telemetry data for an instrument for a single dive. Each time a mail message is received by the telemetry decoder thread, argo::iridium::xfloat::XFloatTelemetryDecoderService, the corresponding dive data object is retrieved and the mail message is assigned to it. Then, the binary data from the attached Iridium Short Burst Data (SBD) message is decoded. As a result of decoding the binary message data, an object corresponding to the message type is created and its field are populated from the decoded data in the binary message. The newly created object will be of a type that inherits from util::xmsg::XMessage. A shared reference to the object is assigned to the dive data object.

This class contains the functionality for writing the formatted intermediate and PHY listing files to the local filesystem. Each time a new message is received, the entire intermediate listing file will be written to the local filesystem, overwriting the previous listing file, if it exists. The PHY file is written only when the dive data object is determined to contain the complete data for dive, and only if the instrument has been deployed i.e. it has a metadata file with valid DAC and WMO identifiers.

This file contains the class definition and the majority of the class functionality.

Version

0.6

Date

Jul 3, 2014 8:56:05 AM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.24 XFloatDiveData.h

Go to the documentation of this file.

```
00001 /** The @ref argo::iridium::xfloat::XFloatDiveData class header file.
00002  *
00003  *  This is the header file for the @ref argo::iridium::xfloat::XFloatDiveData class. This class
00004  *  contains all of the mail messages and corresponding decoded telemetry data for an instrument
00005  *  for a single dive. Each time a mail message is received by the telemetry decoder thread, @ref
00006  *  argo::iridium::xfloat::XFloatTelemetryDecoderService, the corresponding dive data object is
00007  *  retrieved and the mail message is assigned to it. Then, the binary data from the attached
00008  *  Iridium Short Burst Data (SBD) message is decoded. As a result of decoding the binary message
00009  *  data, an object corresponding to the message type is created and its field  are populated from
00010  *  the decoded data in the binary message. The newly created object will be  of a type that
00011  *  inherits from @ref util::xmsg::XMessage. A shared reference to the object is assigned to the
00012  *  dive data object.
00013  *
00014  *  This class contains the functionality for writing the formatted intermediate and PHY listing
00015  *  files to the local filesystem. Each time a new message is received, the entire intermediate
00016  *  listing file will be written to the local filesystem, overwriting the previous listing file,
00017  *  if it exists. The PHY file is written only when the dive data object is determined to contain
00018  *  the complete data for dive, and only if the instrument has been deployed i.e. it has a
00019  *  metadata file with valid DAC and WMO identifiers.
00020  *
00021  *  This file contains the class definition and the majority of the class functionality.
00022  *
00023  *  @file
00024  *
00025  *  @version    0.6
00026  *
00027  *  @date       Jul 3, 2014 8:56:05 AM
00028  *
00029  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00030  *              Engineer I,
00031  *              Department of Physical Oceanography,
00032  *              Woods Hole Oceanographic Institution
00033  *
00034  *  @copyright  2014 Woods Hole Oceanographic Institution.
00035  *              Released under the GNU General Public License.
00036  *
00037  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00038  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00039  *  General Public License for more details. You should have received a copy of the GNU General
00040  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00041  */
00042
00043 #ifndef XFLOATDIVEDATA_H_
00044 #define XFLOATDIVEDATA_H_
00045
00046 /* Includes: */
00047
00048 #include "XFloatDefinitions.h"  // local
00049
00050 namespace argo {
00051 namespace iridium {
00052 namespace xfloat {
00053
00054 /* Class Definitions: */
00055
00056 // forward declarations
00057 class XFloat;
00058 class XFloatMailMessage;
00059
00060 /** The instrument or float dive data class.
00061  *
00062  *  This class contains all of the mail messages and corresponding decoded telemetry data for an
00063  *  instrument for a single dive. Each time a mail message is received by the telemetry decoder
00064  *  thread, @ref XFloatTelemetryDecoderService, the corresponding dive data object is retrieved
00065  *  and the mail message is assigned to it. Then, the binary data from the attached Iridium Short
00066  *  Burst Data (SBD) message is decoded by calling @ref DecodeXMsgData(). As a result of decoding
00067  *  the binary message data, an object corresponding to the message type is created and its field
00068  *  are populated from the decoded data in the binary message. The newly created object will be
00069  *  of a type that inherits from @ref util::xmsg::XMessage. A shared reference to the object is
00070  *  assigned to the dive data object.
00071  *
```

```
00072  *  This class contains the functionality for writing the formatted intermediate and PHY listing
00073  *  files to the local filesystem. Each time a new message is received, the entire intermediate
00074  *  listing file will be written to the local filesystem, overwriting the previous listing file,
00075  *  if it exists. The PHY file is written only when the dive data object is determined to contain
00076  *  the complete data for dive, and only if the instrument has been deployed i.e. it has a
00077  *  metadata file with valid DAC and WMO identifiers.
00078  *
00079  *  @note   This object is lockable. It contains a recursive mutual exclusion (mutex) lock in
00080  *          order to synchronize access to and modification of objects of this type by various
00081  *          worker threads. Once the object is locked, any other thread that attempts to access
00082  *          or modify the object will block until the initial lock is released. Since the mutex
00083  *          is recursive, the object may be locked multiple times by the same thread without
00084  *          causing the thread to block indefinitely.
00085  *
00086  *  @note   This class is mapped to an SQL database table by the @ref sql_table_description_
00087  *          object. This allows some or all of the field of this class to be persisted to the SQL
00088  *          database. Likewise, an object of this type may be constructed from the results of an
00089  *          SQL query to the corresponding table.
00090  *
00091  *  @todo   Devise a scheme to ensure that the dive data objects do not remain in memory
00092  *          indefinitely if one or more message for the dive or cycle are not received and the dive
00093  *          data object is not completed.
00094  */
00095 class XFloatDiveData:
00096     public Iridium,
00097     protected util::date::DateUtils,
00098     protected util::file::FileSystemUtils,
00099     protected util::mysql::LocalPersistentEntity/*,
00100     public util::netcdf::NetcdfUtils*/ {
00101
00102     /* Friend Classes: */
00103
00104     friend class XFloat;
00105
00106     friend class XFloatTelemetryDecoderService;
00107
00108     /* Friend Functions: */
00109
00110     /** Friend function for populating some of the static data members of this class from the
00111      *  configuration file.
00112      */
00113     friend options_description
       util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription();
00114
00115     /** Friend function for creating an X-Float object using the private constructor and a shared
00116      *  pointer to that object.
00117      */
00118     friend shared_ptr<XFloatDiveData> std::make_shared<XFloatDiveData>();
00119
00120   private:
00121
00122     /* Static Const Data Members: */
00123
00124     /** The NetCDF referecne date/time.
00125      *
00126      *  This field is used as the reference date/time for generation of NetCDF files where dates
00127      *  are represented as days since 00:00Z on this date and the fractional remainder represents
00128      *  the time-of-day. The value of this field represents the date/time 1950-01-01T00:00:00Z.
00129      */
00130     static const ptime netcdf_reference_datetime_;
00131
00132     /** The SQL table description object.
00133      *
00134      *  This SQL table description provides the details needed to map this object to a relational
00135      *  database table. It utilizes the accessor methods of the class and the details of the SQL
00136      *  table column to (1) create an SQL INSERT statement to save the object into the database,
00137      *  (2) populate one or more objects' fields as a result of an SQL SELECT query, or (3) query
00138      *  the SQL database to access one or more of the values that are stored the the SQL table
00139      *  corresponding to this object.
00140      */
00141     static const util::mysql::SqlTableDescription<XFloatDiveData> sql_table_description_;
00142
00143   private:
00144
00145     /* Static Data Members: */
00146
00147     /** The global intermediate-listing-file-generation-is-enabled flag.
00148      *
00149      *  The value of this field determines whether or not any intermediate listing files will be
00150      *  written to the local filesystem. If TRUE, the intermediate listing files will be written
00151      *  as new messages are received by the application. Otherwise, if FALSE, no intermediate
00152      *  listing files will be written to the local filesystem.
00153      *
00154      *  The value of this field is set as a result of parsing the command-line arguments and/or
00155      *  configuration file options during application initialization.
00156      */
00157     static bool intermediate_listing_file_generation_is_enabled_;
00158
00159     /** The intermediate listing directory path on the local filesystem.
00160      *
00161      *  The value of this field determines the base directory to which intermediate listing files
00162      *  will be written. Intermediate listing files will be written to subdirectories of this path,
00163      *  which are organized by the unique instrument identifiers. Intermediate listing files will
00164      *  be written each time a new message is received from an instrument.
00165      *
00166      *  The value of this field is set as a result of parsing the command-line arguments and/or
00167      *  configuration file options during application initialization.
00168      */
00169     static path intermediate_listing_directory_path_;
00170
00171     /** The global NetCDF-file-generation-is-enabled flag.
00172      *
00173      *  The value of this field determines whether or not any NetCDF files will be written to the
00174      *  local filesystem. If TRUE, then NetCDF files will be written each time a complete set of
00175      *  dive data is received from a deployed instrument and NetCDF file generation is enabled for
00176      *  that instrument. Otherwise, if FALSE, no NetCDF files will be written to the local
00177      *  filesystem.
00178      *
```

```
00179      *  The value of this field is set as a result of parsing the command-line arguments and/or
00180      *  configuration file options during application initialization.
00181      */
00182     static bool netcdf_file_generation_is_enabled_;
00183
00184     /** The NetCDF directory path on the local filesystem.
00185      *
00186      *  The value of this field determines the base directory to which NetCDF files will be
00187      *  written. NetCDF files will be written to subdirectories of this path, which are organized
00188      *  by the unique instrument identifiers. NetCDF files will be written each time a new message
00189      *  is received from an instrument that completes all the necessary data for that dive and
00190      *  NetCDF file generation is enabled for that instrument.
00191      *
00192      *  The value of this field is set as a result of parsing the command-line arguments and/or
00193      *  configuration file options during application initialization.
00194      */
00195     static path netcdf_directory_path_;
00196
00197     /** The global PHY-file-generation-is-enabled flag.
00198      *
00199      *  The value of this field determines whether or not any PHY files will be written to the
00200      *  local filesystem. If TRUE, then PHY files will be written each time a complete set of dive
00201      *  data is received from a deployed instrument and PHY file generation is enabled for that
00202      *  instrument. Otherwise, if FALSE, no PHY files will be written to the local filesystem.
00203      *
00204      *  The value of this field is set as a result of parsing the command-line arguments and/or
00205      *  configuration file options during application initialization.
00206      */
00207     static bool phy_file_generation_is_enabled_;
00208
00209     /** The PHY directory path on the local filesystem.
00210      *
00211      *  The value of this field determines the base directory to which PHY files will be written.
00212      *  PHY files will be written to subdirectories of this path, which are organized by the
00213      *  unique instrument identifiers. PHY files will be written each time a new message is
00214      *  received from an instrument that completes all the necessary data for that dive and PHY
00215      *  file generation is enabled for that instrument.
00216      *
00217      *  The value of this field is set as a result of parsing the command-line arguments and/or
00218      *  configuration file options during application initialization.
00219      */
00220     static path phy_directory_path_;
00221
00222     /** The global surface-pressure-offset-correction-is-enabled-in-intermediate-listing-files flag.
00223      *
00224      *  The value of this flag determines whether the surface pressure offset correction is applied
00225      *  to all of the pressure values in the intermediate listing files. If TRUE, the surface
00226      *  pressure offset correction will be applied to the intermediate listing files. Otherwise,
00227      *  the pressure offset correction will not be applied.
00228      *
00229      *  The value of this field is set as a result of parsing the command-line arguments and/or
00230      *  configuration file options during application initialization.
00231      */
00232     static bool pressure_offset_correction_enabled_list_file_;
00233
00234     /** The global surface-pressure-offset-correction-is-enabled-in-NetCDF-files flag.
00235      *
00236      *  The value of this flag determines whether the surface pressure offset correction is applied
00237      *  to all of the pressure values in the NetCDF files. If TRUE, the surface pressure offset
00238      *  correction will be applied to the NetCDF files. Otherwise, the pressure offset correction
00239      *  will not be applied.
00240      *
00241      *  The value of this field is set as a result of parsing the command-line arguments and/or
00242      *  configuration file options during application initialization.
00243      */
00244     static bool pressure_offset_correction_enabled_netcdf_file_;
00245
00246     /** The global surface-pressure-offset-correction-is-enabled-in-PHY-files flag.
00247      *
00248      *  The value of this flag determines whether the surface pressure offset correction is applied
00249      *  to all of the pressure values in the PHY files. If TRUE, the surface pressure offset
00250      *  correction will be applied to the PHY files. Otherwise, the pressure offset correction
00251      *  will not be applied.
00252      *
00253      *  The value of this field is set as a result of parsing the command-line arguments and/or
00254      *  configuration file options during application initialization.
00255      */
00256     static bool pressure_offset_correction_enabled_phy_file_;
00257
00258   private:
00259
00260     /* Data Members: */
00261
00262     /** The enumerated Argo instrument type identifier.
00263      *
00264      *  This field is used to identify the specific type of instrument i.e. Solo-II, Deep Argo,
00265      *  or ALAMO/Hurricane floats. The value of this field may affect the behavior of some
00266      *  aspects of the application. For example, the manner in which some Iridium Short Burst Data
00267      *  (SBD) messages are decoded is dependent on the specific type of instrument.
00268      */
00269     XArgoInstrumentType_t argo_instrument_type_;
00270
00271     /** The dive-data-is-complete flag.
00272      *
00273      *  The value of this field indicates whether the dive data object is complete. The
00274      *  completeness of the dive data is determined by \ref CheckCompleteness() which is invoked
00275      *  after each new mail message is decoded and assigned to this dive data object.
00276      *
00277      *  The dive data is determined to be complete if the dive data object meets all of the
00278      *  following criteria: it contains a complete set of profile data, the sensor gains and
00279      *  offsets for this dive are known (i.e. the Argo data message has been received), it
00280      *  contains a set of mail messages with contiguous packet identifiers (PIDs) starting at
00281      *  PID=0, and it contains a valid position for the profile. These criteria is determined by
00282      *  the functions \ref HasCompleteProfileData(), \ref SensorGainsAndOffsetsAreKnown(),
00283      *  \ref HasContiguousPids(), \ref HasValidPosition(), respectively.
00284      */
00285     bool complete_;
00286
```

```
00287     /** The dive or cycle number of this dive data object.
00288      *
00289      *  This field is used to index the dive data object for a given instrument. Together, the
00290      *  unique instrument identifier and the dive or cycle number provide a unique index of the
00291      *  dive data object.
00292      */
00293     int id_dive_;
00294
00295     /** The date/time of transmission of the first Iridium message sent by the instrument for this
00296      *  dive.
00297      *
00298      *  This field is updated as a result of assigning a mail message to this dive data object by
00299      *  calling the function \ref AssignMailMessage(const shared_ptr<XFloatMailMessage>& x).
00300      */
00301     ptime iridium_position_datetime_first_;
00302
00303     /** The date/time of transmission of the last Iridium message sent by the instrument for this
00304      *  dive.
00305      *
00306      *  This field is updated as a result of assigning a mail message to this dive data object by
00307      *  calling the function \ref AssignMailMessage(const shared_ptr<XFloatMailMessage>& x).
00308      */
00309     ptime iridium_position_datetime_last_;
00310
00311     /** The mail messages for this dive data object.
00312      *
00313      *  This vector contains a set of shared reference to the mail messages that contain the
00314      *  telemetry data for this dive. Each time a mail message is received by the telemetry
00315      *  decoder thread, it is assigned to the corresponding dive data object before the thread
00316      *  goes on to decode the message.
00317      *
00318      *  This set of mail message may be used to reprocess the telemetry data for a dive, if
00319      *  necessary. For example, if a new Argo message arrives with different sensor gain and
00320      *  offset values than the previous dive, then the sensor data messages may be reprocessed
00321      *  using the new sensor gain and offset values.
00322      */
00323     static_vector<shared_ptr<XFloatMailMessage>, XMSG_MAX_MAIL> mail_messages_;
00324
00325     /** The number of SBD messages that have been received for this dive cycle.
00326      *
00327      */
00328     uint16_t num_sbd_msgs_;
00329
00330     /** The parent float or instrument object of this dive data object.
00331      *
00332      *  This field provides a shared back-reference to the instrument object that owns this dive
00333      *  data object. This field specifies the any-to-one or many-to-one relationship between dive
00334      *  data object and instrument objects.
00335      *
00336      *  Every dive data object must have a parent instrument object. In other words, value of this
00337      *  field should never be NULL.
00338      */
00339     shared_ptr<XFloat> parent_instrument_;
00340
00341     /* Data Members - NetCDF: */
00342
00343     /** The NetCDF dimensions.
00344      *
00345      */
00346 //    map<string, NcDim> nc_dims_;
00347
00348     /** The NetCDF file path.
00349      *
00350      */
00351     path netcdf_file_path_;
00352
00353     /** The NetCDF-file-is-updated flag.
00354      *
00355      */
00356     bool netcdf_file_updated_;
00357
00358     /* Data Members - Derived Fields - Pressure Offset: */
00359
00360     /** The computed surface pressure offset for this dive.
00361      *
00362      */
00363     float pres_offs_;
00364
00365     /** The number of dives used to compute the surface pressure offset.
00366      *
00367      */
00368     int pres_offs_num_dives_;
00369
00370     /** The number of samples used to compute the surface pressure offset.
00371      *
00372      */
00373     int pres_offs_num_samples_;
00374
00375     /** The surface-pressure offset-is-valid flag.
00376      *
00377      */
00378     bool pres_offs_valid_;
00379
00380     /* Data Members - Derived Fields - Surface Pressure: */
00381
00382     /** The maximum reported surface pressure.
00383      *
00384      */
00385     float surface_pres_max_;
00386
00387     /** The mean surface pressure.
00388      *
00389      */
00390     float surface_pres_mean_;
00391
00392     /** The minimum reported surface pressure.
00393      *
00394      */
```

```
00395     float surface_pres_min_;
00396
00397     /** The number of surface pressure samples reported.
00398      *
00399      */
00400     int surface_pres_num_samples_;
00401
00402     /** The surface-pressure-is-valid flag.
00403      *
00404      */
00405     bool surface_pres_valid_;
00406
00407     /** The surface pressure variance.
00408      *
00409      */
00410     float surface_pres_variance_;
00411
00412     /* Data Members - Files: */
00413
00414     /** The path to the intermediate listing file for this dive data on the local filesystem.
00415      *
00416      *  This field is set as a result of writing the intermediate listing file i.e. by calling
00417      *  @ref WriteListingFile(). This path is relative to the listing directory, which is
00418      *  specified by the static data member @ref intermediate_listing_directory_path_.
00419      */
00420     path listing_file_path_;
00421
00422     /** The listing-file-is-updated flag.
00423      *
00424      */
00425     bool listing_file_updated_;
00426
00427     /** The path to the PHY file for this dive data on the local filesystem.
00428      *
00429      *  This field is set as a result of writing the PHY listing file i.e. by calling @ref
00430      *  WritePhyFile(). This path is relative to the PHY listing directory, which is specified by
00431      *  the static data member, @ref intermediate_listing_directory_path_.
00432      */
00433     path phy_file_path_;
00434
00435     /** The phy-file-is-updated flag.
00436      *
00437      */
00438     bool phy_file_updated_;
00439
00440     /* Dynamic Data Members - X-Message Data: */
00441
00442     /** A vector of pointers to the acoustic position messages.
00443      *
00444      *  @note           This message type is used by MIZ floats only.
00445      *  @note           This vector is sorted upon insertion by order of non-decreasing message
00446      *                  indices.
00447      */
00448     shared_ptr<x::XAcousticPosition> xmsg_acoustic_positions_;
00449
00450     /** A pointer to the Argo data message. */
00451     shared_ptr<x::XArgo> xmsg_argo_data_;
00452
00453     /** The composite binned profile data object.
00454      *
00455      *  This object contains all of the binned profile data for this dive/cycle.
00456      */
00457     x::XCompositeProfile xmsg_composite_profile_binned_;
00458
00459     /** The composite unbinned profile data object.
00460      *
00461      *  This object contains all of the unbinned profile data for this dive/cycle.
00462      */
00463     x::XCompositeHiResProfile xmsg_composite_profile_unbinned_;
00464
00465     /** The composite drift data object.
00466      *
00467      *  This object contains all of the drift data for this dive/cycle.
00468      *
00469      */
00470     x::XCompositeDrift xmsg_composite_drift_data_;
00471
00472     /** A pointer to the engineering abort message. */
00473     shared_ptr<x::XEngAbort> xmsg_engineering_data_abort_;
00474
00475     /** A pointer the the engineering ascent message.
00476      *
00477      *  @note           This message type is used by ALAMO floats only.
00478      */
00479     shared_ptr<x::XEngAscent> xmsg_engineering_data_ascent_;
00480
00481     /** A pointer to the engineering BIT failed message.
00482      *
00483      */
00484     shared_ptr<x::XEngBITFail> xmsg_engineering_data_bit_fail_;
00485
00486     /** A pointer to the engineering BIT passed message.
00487      *
00488      */
00489     shared_ptr<x::XEngBITPass> xmsg_engineering_data_bit_passed_;
00490
00491     /** A pointer to the engineering descent message.
00492      *
00493      *  @note           This message type is used by ALAMO floats only.
00494      */
00495     shared_ptr<x::XEngDescent> xmsg_engineering_data_descent_;
00496
00497     /** A pointer to the engineering diagnostic or startup message.
00498      *
00499      */
00500     shared_ptr<x::XEngDiagnostic> xmsg_engineering_data_diagnostic_dive_;
00501
00502     /** A pointer to the engineering drift message.
```

```
00503      *
00504      *  @note           This message type is used by ALAMO floats only.
00505      */
00506     shared_ptr<x::XEngDrift> xmsg_engineering_data_drift_;
00507
00508     /** A pointer to the ice-detection engineering message.
00509      *
00510      */
00511     shared_ptr<x::XEngIce> xmsg_engineering_data_ice_;
00512
00513     /** A pointer to the standard dive-cycle engineering message.
00514      *
00515      */
00516     shared_ptr<x::XEngStandard> xmsg_engineering_data_standard_dive_;
00517
00518     /** A pointer to the engineering surface message.
00519      *
00520      *  @note           This message type is used by ALAMO floats only.
00521      */
00522     shared_ptr<x::XEngSurface> xmsg_engineering_data_surface_;
00523
00524     /** A static vector of pointers to the fall time-series messages.
00525      *
00526      *  @note           This vector is sorted upon insertion by order of non-decreasing message
00527      *                  indices.
00528      */
00529     static_vector<shared_ptr<x::XFall>, XMSG_MAX_FALL> xmsg_fall_time_series_;
00530
00531     /** A static vector of pointers to the GPS position message(s).
00532      *
00533      *  @note           This vector is sorted upon insertion by order of non-decreasing message
00534      *                  indices.
00535      */
00536     static_vector<shared_ptr<x::XGPS>, XMSG_MAX_GPS> xmsg_gps_positions_;
00537
00538     /** A vector of pointer to mission configuration command upload echo-back messages.
00539      *
00540      *  @note           These messages are unordered.
00541      */
00542     vector<shared_ptr<x::XMissionCmdEcho> > xmsg_mission_cmd_echoback_;
00543
00544     /** The MFA mission configruation object.
00545      *
00546      */
00547     x::XMfaMissionCfg xmsg_mission_cfg_;
00548
00549     /** A static vector of pointers to the pump time-series messages.
00550      *
00551      *  @note           This vector is sorted upon insertion by order of non-decreasing message
00552      *                  indices.
00553      */
00554     static_vector<shared_ptr<x::XPump>, XMSG_MAX_PUMP> xmsg_pump_time_series_;
00555
00556     /** A static vector of pointers to the fall time-series messages.
00557      *
00558      *  @note           This vector is sorted upon insertion by order of non-decreasing message
00559      *                  indices.
00560      */
00561     static_vector<shared_ptr<x::XRise>, XMSG_MAX_RISE> xmsg_rise_time_series_;
00562
00563     /** The surface pressure time-series object.
00564      *
00565      */
00566     x::XCompositeSurfaceData xmsg_surface_time_series_;
00567
00568     /** Valve operation messages.
00569      *
00570      */
00571     static_vector<shared_ptr<x::XValve>, XMSG_MAX_VALVE> xmsg_valve_data_;
00572
00573     /** A static vector of pointers to the surface wave field data messages.
00574      *
00575      *  @note           This vector is sorted upon insertion by order of non-decreasing message
00576      *                  indices.
00577      */
00578     shared_ptr<x::XWave> xmsg_wave_field_;
00579
00580   public:
00581
00582     /* Public Static Member Functions*/
00583
00584     /** Generate a human-readable hex string.
00585      *
00586      *  @param[in]  data  The message data.
00587      *
00588      *  @return    A human readable hex representation of the data.
00589      */
00590     static string GenerateReadableHex(const vector<uint8_t>& data);
00591
00592   public:
00593
00594     /* Public Member Functions: */
00595
00596     XFloatDiveData();
00597
00598     XFloatDiveData(const shared_ptr<XFloat>& parent, int dive);
00599
00600     ~XFloatDiveData();
00601
00602     bool CheckCompleteness();
00603
00604     int DecodeMailMessageData(shared_ptr<XFloatMailMessage>& msg);
00605
00606     void DecodeXMsgData(
00607         const vector<uint8_t>& data,
00608         shared_ptr<vector<uint8_t> > xmsg_id_types = shared_ptr<vector<uint8_t> >())
00609             ATTR_NO_EXCEPTIONS;
00610
```

```
00611     void GetGainsAndOffsets(
00612         uint16_t& pg,
00613         uint16_t& po,
00614         uint16_t& tg,
00615         uint16_t& to,
00616         uint16_t& sg,
00617         uint16_t& so) const;
00618 
00619     void GetSurfaceGainsAndOffsets(
00620         uint16_t& pg,
00621         uint16_t& po,
00622         uint16_t& tg,
00623         uint16_t& to,
00624         uint16_t& sg,
00625         uint16_t& so) const;
00626 
00627     float ComputePressureOffsetFromSurfaceTimeSeries(unsigned int max_num_dives);
00628 
00629     string GenerateListingString() const;
00630 
00631     string GeneratePhyString() const;
00632 
00633     string GeneratePressureCorrectionListingString() const;
00634 
00635     shared_ptr<x::XGPS> GetGpsPositionEndOfDive() const;
00636 
00637     shared_ptr<x::XGPS> GetGpsPositionStartOfDive() const;
00638 
00639     shared_ptr<x::XGPS> GetGpsProfilePosition() const;
00640 
00641     bool GpsPositionIsValidProfilePosition(const shared_ptr<x::XGPS> x) const;
00642 
00643     bool HasDiveData() const;
00644 
00645     bool HasDriftData() const;
00646 
00647     bool HasEngineeringData() const;
00648 
00649     bool HasFallRiseData() const;
00650 
00651     bool HasCompleteBinnedProfileData() const;
00652 
00653     bool HasCompleteProfileData() const;
00654 
00655     bool HasCompleteUnbinnedProfileData() const;
00656 
00657     bool HasContiguousMomsns() const;
00658 
00659     bool HasContiguousPids() const;
00660 
00661     bool HasPositionData() const;
00662 
00663     bool HasBinnedProfileData() const;
00664 
00665     bool HasBinnedProfileSalinityData() const;
00666 
00667     bool HasUnbinnedProfileData() const;
00668 
00669     bool HasUnbinnedProfileSalinityData() const;
00670 
00671     bool HasProfileSalinityData() const;
00672 
00673     bool HasValidGpsPosition() const;
00674 
00675     bool HasValidIridiumPosition() const;
00676 
00677     bool HasValidPosition() const;
00678 
00679     bool HasWaveData() const;
00680 
00681     void ResetData();
00682 
00683     bool SensorGainsAndOffsetsAreKnown() const;
00684 
00685     void WriteListingFile();
00686 
00687 //    void WriteNetCDFFile();
00688 
00689     void WritePhyFile();
00690 
00691   private:
00692 
00693     /* Private Member Functions: */
00694 
00695 
00696     shared_ptr<x::XCompositeFall> GenerateCompositeFallTimeSeriesData() const;
00697 
00698     shared_ptr<x::XCompositeIridiumPosition> GenerateCompositeIridiumPosition() const;
00699 
00700     shared_ptr<x::XCompositePump> GenerateCompositePumpTimeSeriesData() const;
00701 
00702     shared_ptr<x::XCompositeRise> GenerateCompositeRiseTimeSeriesData() const;
00703 
00704     shared_ptr<x::XCompositeValve> GenerateCompositeValveData() const;
00705 
00706     vector<int> GenerateOrderedPidList() const;
00707 
00708     vector<int> GenerateOrderedMomsnList() const;
00709 
00710     /** Update the derived surface pressure fields from the surface time-series data.
00711      *
00712      *  Computes the surface pressure maximum, mean, minimum, number of samples and variance from
00713      *  the surface time series (see @ref xmsg_surface_time_series_), using the surface sensor
00714      *  gains and offsets. These fields feed the surface pressure offset correction (see
00715      *  @ref ComputePressureOffsetFromSurfaceTimeSeries()).
00716      *
00717      *  The fields are set to @ref XMSG_SENS_BAD_PRES and marked invalid if the sensor gains and
00718      *  offsets are unknown, there is no surface data, or the mean surface pressure is outside
```

```
00719      *  ±5 dbar. Otherwise they are set, and marked valid only if the surface time series is
00720      *  complete and the dive has a valid GPS position.
00721      *
00722      *  @throw      util::log::MessageException if an error occurs.
00723      */
00724     void UpdateDerivedSurfaceDataFields() {
00725
00726       // Sensor gains & offsets for surface data.
00727       uint16_t spg, spo, ssg, sso, stg, sto;
00728
00729       try {
00730
00731         // Get an exclusive lock.
00732         lock_guard<recursive_mutex> guard(mutex_);
00733
00734         ldbg() « "Updating derived surface data fields...";
00735
00736         if(!SensorGainsAndOffsetsAreKnown()) {
00737
00738           set_surface_pres_max(XMSG_SENS_BAD_PRES);
00739           set_surface_pres_mean(XMSG_SENS_BAD_PRES);
00740           set_surface_pres_min(XMSG_SENS_BAD_PRES);
00741           set_surface_pres_valid(false);
00742           set_surface_pres_variance(XMSG_SENS_BAD_PRES);
00743
00744           return;
00745
00746         }
00747
00748         // Get surface gains & offsets.
00749         GetSurfaceGainsAndOffsets(spg, spo, stg, sto, ssg, sso);
00750
00751         // Handle missing data.
00752         if(xmsg_surface_time_series_.IsEmpty()) {
00753
00754           set_surface_pres_max(XMSG_SENS_BAD_PRES);
00755           set_surface_pres_mean(XMSG_SENS_BAD_PRES);
00756           set_surface_pres_min(XMSG_SENS_BAD_PRES);
00757           set_surface_pres_valid(false);
00758           set_surface_pres_variance(XMSG_SENS_BAD_PRES);
00759
00760           return;
00761
00762         }
00763
00764         if(xmsg_surface_time_series_.PresMeanAsFloat(spg, spo) > 5.
00765             || xmsg_surface_time_series_.PresMeanAsFloat(spg, spo) < -5.) {
00766           set_surface_pres_max(XMSG_SENS_BAD_PRES);
00767           set_surface_pres_mean(XMSG_SENS_BAD_PRES);
00768           set_surface_pres_min(XMSG_SENS_BAD_PRES);
00769           set_surface_pres_valid(false);
00770           set_surface_pres_variance(XMSG_SENS_BAD_PRES);
00771
00772           return;
00773
00774         }
00775
00776         // Compute derived surface data fields.
00777         set_surface_pres_max(xmsg_surface_time_series_.PresMaxAsFloat(spg, spo));
00778         set_surface_pres_mean(xmsg_surface_time_series_.PresMeanAsFloat(spg, spo));
00779         set_surface_pres_min(xmsg_surface_time_series_.PresMinAsFloat(spg, spo));
00780         set_surface_pres_num_samples(xmsg_surface_time_series_.pres_num_samples());
00781         set_surface_pres_variance(xmsg_surface_time_series_.PresVarAsFloat(spg));
00782
00783         // Determine surface pressure validity
00784         if(!xmsg_surface_time_series_.IsDataComplete()) {
00785
00786           ldbg() « "Surface data incomplete.";
00787
00788           set_surface_pres_valid(false);
00789
00790           return;
00791
00792         }
00793
00794         if(!HasValidGpsPosition()) {
00795
00796           ldbg() « "Surface data invalid. No GPS position.";
00797
00798           set_surface_pres_valid(false);
00799
00800           return;
00801
00802         }
00803
00804         ldbg() « "Surface data valid.";
00805
00806         set_surface_pres_valid(true);
00807
00808         // TODO Check status of ice-detection.
00809
00810       }CATCH_BLOCK("Failed to update derived surface data fields!")
00811     }
00812
00813   private:
00814
00815     /* Private NetCDF Member Functions: */
00816
00817 //    map<string, NcDim> NC_GenerateDimensions(NcGroup& nc_group);
00818 //
00819 //    void NC_GenerateGlobalAttrs(NcGroup& nc_group) const;
00820 //
00821 //    void NC_GenerateVariables(NcGroup& nc_group);
00822
00823   public:
00824
00825     /* Public SQL Database Member Functions: */
00826
```

```
00827     /** Delete all dive data for an instrument from the SQL database.
00828      *
00829      *  Deletes every GPS message row (see @ref util::xmsg::XGPS::DeleteFromDatabase(int)) and every dive
      data
00830      *  row for the instrument. This is used when an instrument's messages are reprocessed, so that
00831      *  the dives can be rebuilt from the stored mail messages. Stored mail messages are not
00832      *  deleted.
00833      *
00834      *  @warning    This permanently removes all of the instrument's dive and GPS rows.
00835      *
00836      *  @param[in]  id_instrument   The unique instrument identifier (serial number).
00837      *
00838      *  @throw      util::log::MessageException if a delete query fails.
00839      */
00840     static void DeleteFromDatabase(int id_instrument) {
00841
00842       ostringstream os;
00843
00844       try {
00845
00846         // Delete GPS objects.
00847         x::XGPS::DeleteFromDatabase(id_instrument);
00848
00849         // Delete dive data objects;
00850         os « "id_instrument = " « id_instrument;
00851         sql_table_description_.DeleteObjectsFromDatabase(os.str());
00852
00853       }CATCH_BLOCK("Failed to delete dive data object(s) from database!")
00854
00855     }
00856
00857     /** Delete dive data rows from the SQL database table.
00858      *
00859      *  @param[in]  where       The SQL "where-condition" clause selecting the rows to delete.
00860      *
00861      *  @throw      util::log::MessageException if the delete query fails.
00862      */
00863     static void DeleteObjectsFromDatabase(const string& where) {
00864
00865       try {
00866
00867         sql_table_description_.DeleteObjectsFromDatabase(where);
00868
00869       }CATCH_BLOCK("Failed to delete GPS objects from database!")
00870
00871     }
00872
00873     /** Get this dive's stored mail messages from the SQL database.
00874      *
00875      *  Selects the valid mail messages (@c msg_is_valid = 1) for this instrument and dive.
00876      *  Messages transmitted more than 15 minutes before the instrument's start-of-mission or
00877      *  deployment date/time are excluded. Used to rebuild a dive object loaded from the database.
00878      *
00879      *  @throw      util::log::MessageException if the instrument or dive identifier is unknown, or
00880      *              the query fails.
00881      *
00882      *  @return     The matching mail messages.
00883      */
00884     vector<shared_ptr<XFloatMailMessage» GetDiveDataMailMessagesFromDatabase();
00885
00886     static shared_ptr<XFloatDiveData> GetDiveDataObjectFromDatabase(
00887         const shared_ptr<XFloat>& x,
00888         int dive);
00889
00890     /** Save this dive data object to the SQL database.
00891      *
00892      *  The parent instrument object is saved first, so that its row exists before the dive row
00893      *  that references it.
00894      */
00895     void SaveObjectToDatabase() const;
00896
00897     /** Select a single dive data object from the SQL database table.
00898      *
00899      *  @param[in]  where       The SQL select statement "where-condition" clause.
00900      *  @param[in]  groupBy     The SQL select statement "group-by expression" clause.
00901      *  @param[in]  orderBy     The SQL select statement "order-by expression" clause.
00902      *  @param[in]  offset      The SQL select statement result set row offset value.
00903      *
00904      *  @throw      util::log::MessageException if the query fails.
00905      *
00906      *  @return     The first dive data object matching the query.
00907      */
00908     static shared_ptr<XFloatDiveData> SelectObjectFromDatabase(
00909         const string& where = "",
00910         const string& groupBy = "",
00911         const string& orderBy = "",
00912         const unsigned int& offset = 0) {
00913
00914       try {
00915
00916         return sql_table_description_.SelectObjectFromDatabase(where, groupBy, orderBy, offset);
00917
00918       }CATCH_BLOCK("Failed to query XFloat objects from the database!")
00919
00920       return make_shared<XFloatDiveData>();
00921     }
00922
00923     /** Select dive data objects from the SQL database table.
00924      *
00925      *  @param[in]  where       The SQL select statement "where-condition" clause.
00926      *  @param[in]  groupBy     The SQL select statement "group-by expression" clause.
00927      *  @param[in]  orderBy     The SQL select statement "order-by expression" clause.
00928      *  @param[in]  limit       The SQL select statement result set row limit value (0 = no limit).
00929      *  @param[in]  offset      The SQL select statement result set row offset value.
00930      *
00931      *  @throw      util::log::MessageException if the query fails.
00932      *
00933      *  @return     The dive data objects matching the query.
```

```
00934      */
00935     static vector<shared_ptr<XFloatDiveData> > SelectObjectsFromDatabase(
00936         const string& where = "",
00937         const string& groupBy = "",
00938         const string& orderBy = "",
00939         const unsigned int& limit = 0,
00940         const unsigned int& offset = 0) {
00941
00942       try {
00943
00944         return sql_table_description_.SelectObjectsFromDatabase(
00945             where,
00946             groupBy,
00947             orderBy,
00948             limit,
00949             offset);
00950
00951       }CATCH_BLOCK("Failed to query XFloat objects from the database!")
00952
00953       return vector<shared_ptr<XFloatDiveData> >();
00954     }
00955
00956     /** Select a single value from the dive data SQL database table.
00957      *
00958      *  @tparam     TT          The type of the value to select.
00959      *
00960      *  @param[in]  what        The SQL select statement "what" clause (a single column or
00961      *                          expression).
00962      *  @param[in]  where       The SQL select statement "where-condition" clause.
00963      *  @param[in]  groupBy     The SQL select statement "group-by expression" clause.
00964      *  @param[in]  orderBy     The SQL select statement "order-by expression" clause.
00965      *  @param[in]  offset      The SQL select statement result set row offset value.
00966      *
00967      *  @throw      util::log::MessageException if the query fails.
00968      *
00969      *  @return     The value from the first matching row.
00970      */
00971     template<typename TT> static TT SelectValueFromDatabase(
00972         const string& what,
00973         const string& where = "",
00974         const string& groupBy = "",
00975         const string& orderBy = "",
00976         const unsigned int& offset = 0) {
00977
00978       return sql_table_description_.SelectValueFromDatabase<TT>(
00979           what,
00980           where,
00981           groupBy,
00982           orderBy,
00983           offset);
00984     }
00985
00986   public:
00987
00988     /* Public Accessor Methods: */
00989
00990     /** Get the enumerated Argo instrument type identifier.
00991      *
00992      *  @return         The enumerated Argo instrument type identifier.
00993      */
00994     XArgoInstrumentType_t argo_instrument_type() const {
00995
00996       // Get an exclusive lock.
00997       lock_guard<recursive_mutex> guard(mutex_);
00998
00999       return argo_instrument_type_;
01000     }
01001
01002     /** Get the dive or cycle number of this dive data object.
01003      *
01004      *  @return         The dive or cycle number of this dive data object.
01005      */
01006     int id_dive() const {
01007
01008       // Get an exclusive lock.
01009       lock_guard<recursive_mutex> guard(mutex_);
01010
01011       return id_dive_;
01012     }
01013
01014     /** Get the path to the intermediate listing file on the local filesystem.
01015      *
01016      *  @return         The path to the intermediate listing file on the local filesystem.
01017      */
01018     const path& intermediate_listing_file_path() const {
01019
01020       // Get an exclusive lock.
01021       lock_guard<recursive_mutex> guard(mutex_);
01022
01023       return listing_file_path_;
01024     }
01025
01026     const ptime& iridium_position_datetime_first() const {
01027
01028       // Get an exclusive lock.
01029       lock_guard<recursive_mutex> guard(mutex_);
01030
01031       return iridium_position_datetime_first_;
01032     }
01033
01034     const ptime& iridium_position_datetime_last() const {
01035
01036       // Get an exclusive lock.
01037       lock_guard<recursive_mutex> guard(mutex_);
01038
01039       return iridium_position_datetime_last_;
01040     }
01041
```

```
01042     bool is_complete() const {
01043
01044       // Get an exclusive lock.
01045       lock_guard<recursive_mutex> guard(mutex_);
01046
01047       return complete_;
01048     }
01049
01050     bool is_listing_file_updated() const {
01051
01052       // Get an exclusive lock.
01053       lock_guard<recursive_mutex> guard(mutex_);
01054
01055       return listing_file_updated_;
01056     }
01057
01058     bool is_netcdf_file_updated() const {
01059
01060       // Get an exclusive lock.
01061       lock_guard<recursive_mutex> guard(mutex_);
01062
01063       return netcdf_file_updated_;
01064     }
01065
01066     bool is_phy_file_updated() const {
01067
01068       // Get an exclusive lock.
01069       lock_guard<recursive_mutex> guard(mutex_);
01070
01071       return phy_file_updated_;
01072     }
01073
01074     bool is_pres_offs_valid() const {
01075
01076       // Get an exclusive lock.
01077       lock_guard<recursive_mutex> guard(mutex_);
01078
01079       return pres_offs_valid_;
01080     }
01081
01082     bool is_surface_pres_valid() const {
01083
01084       // Get an exclusive lock.
01085       lock_guard<recursive_mutex> guard(mutex_);
01086
01087       return surface_pres_valid_;
01088     }
01089
01090     uint16_t num_sbd_msgs() const {
01091
01092       // Get an exclusive lock.
01093       lock_guard<recursive_mutex> guard(mutex_);
01094
01095       return num_sbd_msgs_;
01096     }
01097
01098     const path& netcdf_file_path() const {
01099
01100       // Get an exclusive lock.
01101       lock_guard<recursive_mutex> guard(mutex_);
01102
01103       return netcdf_file_path_;
01104     }
01105
01106     shared_ptr<XFloat> parent_instrument() const {
01107
01108       // Get an exclusive lock.
01109       lock_guard<recursive_mutex> guard(mutex_);
01110
01111       return parent_instrument_;
01112     }
01113
01114     /** Get the path to the PHY listing file on the local filesystem.
01115      *
01116      *  @return         The path to the PHY listing file on the local filesystem.
01117      */
01118     const path& phy_file_path() const {
01119
01120       // Get an exclusive lock.
01121       lock_guard<recursive_mutex> guard(mutex_);
01122
01123       return phy_file_path_;
01124     }
01125
01126     float surface_pres_max() const {
01127
01128       // Get an exclusive lock.
01129       lock_guard<recursive_mutex> guard(mutex_);
01130
01131       return surface_pres_max_;
01132     }
01133
01134     float surface_pres_mean() const {
01135
01136       // Get an exclusive lock.
01137       lock_guard<recursive_mutex> guard(mutex_);
01138
01139       return surface_pres_mean_;
01140     }
01141
01142     float surface_pres_min() const {
01143
01144       // Get an exclusive lock.
01145       lock_guard<recursive_mutex> guard(mutex_);
01146
01147       return surface_pres_min_;
01148     }
01149
```

```
01150     float surface_pres_variance() const {
01151
01152       // Get an exclusive lock.
01153       lock_guard<recursive_mutex> guard(mutex_);
01154
01155       return surface_pres_variance_;
01156     }
01157
01158     float pres_offs() const {
01159
01160       // Get an exclusive lock.
01161       lock_guard<recursive_mutex> guard(mutex_);
01162
01163       return pres_offs_;
01164     }
01165
01166     int pres_offs_num_dives() const {
01167
01168       // Get an exclusive lock.
01169       lock_guard<recursive_mutex> guard(mutex_);
01170
01171       return pres_offs_num_dives_;
01172     }
01173
01174     int pres_offs_num_samples() const {
01175
01176       // Get an exclusive lock.
01177       lock_guard<recursive_mutex> guard(mutex_);
01178
01179       return pres_offs_num_samples_;
01180     }
01181
01182     int surface_pres_num_samples() const {
01183
01184       // Get an exclusive lock.
01185       lock_guard<recursive_mutex> guard(mutex_);
01186
01187       return surface_pres_num_samples_;
01188     }
01189
01190   private:
01191
01192     /* Private Mutator Functions: */
01193
01194
01195
01196     void AssignArgoData(const shared_ptr<x::XArgo>& x);
01197
01198     /** Assign a drift pressure message to this dive data object.
01199      *
01200      *  @note           This vector is sorted upon insertion by order of non-decreasing message indices.
01201      *
01202      *  @param[in]      x       A pointer to the drift pressure message to assign.
01203      */
01204     void AssignDriftPressure(const shared_ptr<x::XDriftPressure>& x) {
01205
01206       // Get an exclusive lock.
01207       lock_guard<recursive_mutex> guard(mutex_);
01208
01209       xmsg_composite_drift_data_.AssignPressureMessage(x);
01210
01211       // Update file status.
01212       set_listing_file_updated(false);
01213
01214     }
01215
01216     /** Assign a drift salinity message to this dive data object.
01217      *
01218      *  @note           This vector is sorted upon insertion by order of non-decreasing message indices.
01219      *
01220      *  @param[in]      x       A pointer to the drift salinity message to assign.
01221      */
01222     void AssignDriftSalinity(const shared_ptr<x::XDriftSalinity>& x) {
01223
01224       // Get an exclusive lock.
01225       lock_guard<recursive_mutex> guard(mutex_);
01226
01227       xmsg_composite_drift_data_.AssignSalinityMessage(x);
01228
01229       // Update file status.
01230       set_listing_file_updated(false);
01231
01232     }
01233
01234     /** Assign a drift temperature message to this dive data object.
01235      *
01236      *  @note           This vector is sorted upon insertion by order of non-decreasing message indices.
01237      *
01238      *  @param[in]      x       A pointer to the drift temperature message to assign.
01239      */
01240     void AssignDriftTemperatre(const shared_ptr<x::XDriftTemperature>& x) {
01241
01242       // Get an exclusive lock.
01243       lock_guard<recursive_mutex> guard(mutex_);
01244
01245       xmsg_composite_drift_data_.AssignTemperatureMessage(x);
01246
01247       // Update file status.
01248       set_listing_file_updated(false);
01249
01250     }
01251
01252     /** Assign an engineering abort/beacon message to this dive data object.
01253      *
01254      *  @param[in]      x       A pointer to the engineering abort/beacon message to assign.
01255      */
01256     void AssignEngineeringAbort(const shared_ptr<x::XEngAbort>& x) {
01257
```

```
01258       // Get an exclusive lock.
01259       lock_guard<recursive_mutex> guard(mutex_);
01260
01261       xmsg_engineering_data_abort_ = x;
01262
01263       // Update file status.
01264       set_listing_file_updated(false);
01265
01266     }
01267
01268     /** Assign an engineering ascent message to this dive data object.
01269      *
01270      *  @note           This message type is used by ALAMO floats only.
01271      *
01272      *  @param[in]      x       A pointer to the XXengineering ascentX message to assign.
01273      */
01274     void AssignEngineeringAscent(const shared_ptr<x::XEngAscent>& x) {
01275
01276       // Get an exclusive lock.
01277       lock_guard<recursive_mutex> guard(mutex_);
01278
01279       xmsg_engineering_data_ascent_ = x;
01280
01281       // Update file status.
01282       set_listing_file_updated(false);
01283
01284     }
01285
01286     /** Assign an engineering BIT failed message to this dive data object.
01287      *
01288      *  @param[in]      x       A pointer to the engineering BIT failed message to assign.
01289      */
01290     void AssignEngineeringBitFail(const shared_ptr<x::XEngBITFail>& x) {
01291
01292       // Get an exclusive lock.
01293       lock_guard<recursive_mutex> guard(mutex_);
01294
01295       xmsg_engineering_data_bit_fail_ = x;
01296
01297       // Update file status.
01298       set_listing_file_updated(false);
01299
01300     }
01301
01302     /** Assign an engineering BIT passed message to this dive data object.
01303      *
01304      *  @param[in]      x       A pointer to the engineering BIT passed message to assign.
01305      */
01306     void AssignEngineeringBitPass(const shared_ptr<x::XEngBITPass>& x) {
01307
01308       // Get an exclusive lock.
01309       lock_guard<recursive_mutex> guard(mutex_);
01310
01311       xmsg_engineering_data_bit_passed_ = x;
01312
01313       // Update file status.
01314       set_listing_file_updated(false);
01315
01316     }
01317
01318     /** Assign an engineering descent message to this dive data object.
01319      *
01320      *  @note           This message type is used by ALAMO floats only.
01321      *
01322      *  @param[in]      x       A pointer to the engineering descent message to assign.
01323      */
01324     void AssignEngineeringDescent(const shared_ptr<x::XEngDescent>& x) {
01325
01326       // Get an exclusive lock.
01327       lock_guard<recursive_mutex> guard(mutex_);
01328
01329       xmsg_engineering_data_descent_ = x;
01330
01331       // Update file status.
01332       set_listing_file_updated(false);
01333
01334     }
01335
01336     /** Assign an engineering diagnostic dive message to this dive data object.
01337      *
01338      *  @param[in]      x       A pointer to the engineering diagnostic dive message to assign.
01339      */
01340     void AssignEngineeringDiagnostic(const shared_ptr<x::XEngDiagnostic>& x) {
01341
01342       // Get an exclusive lock.
01343       lock_guard<recursive_mutex> guard(mutex_);
01344
01345       xmsg_engineering_data_diagnostic_dive_ = x;
01346
01347       // Update file status.
01348       set_listing_file_updated(false);
01349
01350     }
01351
01352     /** Assign an engineering drift/park message to this dive data object.
01353      *
01354      *  @note           This message type is used by ALAMO floats only.
01355      *
01356      *  @param[in]      x       A pointer to the engineering drift/park message to assign.
01357      */
01358     void AssignEngineeringDrift(const shared_ptr<x::XEngDrift>& x) {
01359
01360       // Get an exclusive lock.
01361       lock_guard<recursive_mutex> guard(mutex_);
01362
01363       xmsg_engineering_data_drift_ = x;
01364
01365       // Update file status.
```

```
01366       set_listing_file_updated(false);
01367
01368     }
01369
01370     /** Assign an engineering (ice) message to this dive data object.
01371      *
01372      *  Replaces any previously assigned engineering (ice) message.
01373      *
01374      *  @param[in]      x       A pointer to the engineering (ice) message to assign.
01375      */
01376     void AssignEngineeringIce(const shared_ptr<x::XEngIce>& x) {
01377
01378       // Get an exclusive lock.
01379       lock_guard<recursive_mutex> guard(mutex_);
01380
01381       xmsg_engineering_data_ice_ = x;
01382
01383       // Update file status.
01384       set_listing_file_updated(false);
01385
01386     }
01387
01388     /** Assign an engineering standard dive message to this dive data object.
01389      *
01390      *  @param[in]      x       A pointer to the engineering standard dive message to assign.
01391      */
01392     void AssignEngineeringStandard(const shared_ptr<x::XEngStandard>& x) {
01393
01394       // Get an exclusive lock.
01395       lock_guard<recursive_mutex> guard(mutex_);
01396
01397       xmsg_engineering_data_standard_dive_ = x;
01398
01399       // Update file status.
01400       set_listing_file_updated(false);
01401
01402     }
01403
01404     /** Assign an engineering surface message to this dive data object.
01405      *
01406      *  @note           This message type is used by ALAMO floats only.
01407      *
01408      *  @param[in]      x       A pointer to the engineering surface message to assign.
01409      */
01410     void AssignEngineeringSurface(const shared_ptr<x::XEngSurface>& x) {
01411
01412       // Get an exclusive lock.
01413       lock_guard<recursive_mutex> guard(mutex_);
01414
01415       xmsg_engineering_data_surface_ = x;
01416
01417       // Update file status.
01418       set_listing_file_updated(false);
01419
01420     }
01421
01422     /** Assign a fall time-series message to this dive data object.
01423      *
01424      *  @note       This vector is sorted upon insertion by order of non-decreasing message indices.
01425      *
01426      *  @param[in]  x       A pointer to the fall time-series message to assign.
01427      */
01428     void AssignFallTimeSeries(const shared_ptr<x::XFall>& x) {
01429
01430       // Get an exclusive lock.
01431       lock_guard<recursive_mutex> guard(mutex_);
01432
01433       // Iterate to emplacement position.
01434       auto it = xmsg_fall_time_series_.begin();
01435       for(; it < xmsg_fall_time_series_.end(); ++it)
01436
01437         // Check for duplicate messages.
01438         if(x->xmsg_index() == (*it)->xmsg_index())
01439
01440           // Duplicate message encountered.
01441           EXC_THROW("Duplicate X-Message index encountered!");
01442
01443         else if(x->xmsg_index() < (*it)->xmsg_index())
01444
01445           // Emplacement position found.
01446           break;
01447
01448       // Emplace element at position.
01449       xmsg_fall_time_series_.emplace(it, x);
01450
01451       // Update file status.
01452       set_listing_file_updated(false);
01453
01454     }
01455
01456     void AssignGps(shared_ptr<x::XGPS>& x);
01457
01458     int AssignMailMessage(const shared_ptr<XFloatMailMessage>& msg);
01459
01460     /** Assign a ground-station command echo-back message to this dive data object.
01461      *
01462      *  @param[in]      x       A pointer to the ground-station command echo-back message to assign.
01463      */
01464     void AssignMissionCmdEchoback(const shared_ptr<x::XMissionCmdEcho>& x) {
01465
01466       // Get an exclusive lock.
01467       lock_guard<recursive_mutex> guard(mutex_);
01468
01469       xmsg_mission_cmd_echoback_.push_back(x);
01470
01471       // Update file status.
01472       set_listing_file_updated(false);
01473
```

```
01474     }
01475
01476     void AssignParentInstrument(const shared_ptr<XFloat>& flt);
01477
01478     /** Assign a bin-averaged profile pressure message to this dive data object.
01479      *
01480      *  @note           This vector is sorted upon insertion by order of non-decreasing message
01481      *                  indices.
01482      *
01483      *  @param[in]      x       A pointer to the bin-averaged profile pressure message to assign.
01484      */
01485     void AssignProfilePressureBinned(const shared_ptr<x::XProfilePressure>& x) {
01486
01487       // Get an exclusive lock.
01488       lock_guard<recursive_mutex> guard(mutex_);
01489
01490       xmsg_composite_profile_binned_.AssignPressureMessage(x);
01491
01492       // Update file status.
01493       set_listing_file_updated(false);
01494
01495     }
01496
01497     /** Assign an unbinned profile pressure message to this dive data object.
01498      *
01499      *  @note           This vector is sorted upon insertion by order of non-decreasing message
01500      *                  indices.
01501      *
01502      *  @param[in]      x       A pointer to the unbinned profile pressure message to assign.
01503      */
01504     void AssignProfilePressureUnbinned(const shared_ptr<x::XHiResPressure>& x) {
01505
01506       // Get an exclusive lock.
01507       lock_guard<recursive_mutex> guard(mutex_);
01508
01509       xmsg_composite_profile_unbinned_.AssignPressureMessage(x);
01510
01511       // Update file status.
01512       set_listing_file_updated(false);
01513
01514     }
01515
01516     /** Assign a bin-averaged profile salinity message to this dive data object.
01517      *
01518      *  @note           This vector is sorted upon insertion by order of non-decreasing message
01519      *                  indices.
01520      *
01521      *  @param[in]      x       A pointer to the bin-averaged profile salinity message to assign.
01522      */
01523     void AssignProfileSalinityBinned(const shared_ptr<x::XProfileSalinity>& x) {
01524
01525       // Get an exclusive lock.
01526       lock_guard<recursive_mutex> guard(mutex_);
01527
01528       xmsg_composite_profile_binned_.AssignSalinityMessage(x);
01529
01530       // Update file status.
01531       set_listing_file_updated(false);
01532
01533     }
01534
01535     /** Assign an unbinned profile salinity message to this dive data object.
01536      *
01537      *  @note           This vector is sorted upon insertion by order of non-decreasing message
01538      *                  indices.
01539      *
01540      *  @param[in]      x       A pointer to the unbinned profile salinity message to assign.
01541      */
01542     void AssignProfileSalinityUnbinned(const shared_ptr<x::XHiResSalinity>& x) {
01543
01544       // Get an exclusive lock.
01545       lock_guard<recursive_mutex> guard(mutex_);
01546
01547       xmsg_composite_profile_unbinned_.AssignSalinityMessage(x);
01548
01549       // Update file status.
01550       set_listing_file_updated(false);
01551
01552     }
01553
01554     /** Assign a bin-averaged profile temperature message to this dive data object.
01555      *
01556      *  @note           This vector is sorted upon insertion by order of non-decreasing message
01557      *                  indices.
01558      *
01559      *  @param[in]      x       A pointer to the bin-averaged profile temperature message to assign.
01560      */
01561     void AssignProfileTemperatureBinned(const shared_ptr<x::XProfileTemperature>& x) {
01562
01563       // Get an exclusive lock.
01564       lock_guard<recursive_mutex> guard(mutex_);
01565
01566       xmsg_composite_profile_binned_.AssignTemperatureMessage(x);
01567
01568       // Update file status.
01569       set_listing_file_updated(false);
01570
01571     }
01572
01573     /** Assign a binned auxiliary-channel profile temperature message to this dive data object.
01574      *
01575      *  @param[in]      x       A pointer to the auxiliary-channel profile temperature message.
01576      *
01577      *  @throw          util::log::MessageException if a message with the same index has already
01578      *                  been assigned.
01579      */
01580     void AssignAuxTemperatureBinned(const shared_ptr<x::XProfileTemperature>& x) {
01581
```

```
01582       // Get an exclusive lock.
01583       lock_guard<recursive_mutex> guard(mutex_);
01584
01585       xmsg_composite_profile_binned_.AssignAuxTemperatureMessage(x);
01586
01587       // Update file status.
01588       set_listing_file_updated(false);
01589
01590     }
01591
01592     /** Assign an unbinned profile temperature message to this dive data object.
01593      *
01594      *
01595      *  @note           This vector is sorted upon insertion by order of non-decreasing message
01596      *                  indices.
01597      *
01598      *  @param[in]      x       A pointer to the unbinned profile temperature message to assign.
01599      */
01600     void AssignProfileTemperatureUnbinned(const shared_ptr<x::XHiResTemperature>& x) {
01601
01602       // Get an exclusive lock.
01603       lock_guard<recursive_mutex> guard(mutex_);
01604
01605       xmsg_composite_profile_unbinned_.AssignTemperatureMessage(x);
01606
01607       // Update file status.
01608       set_listing_file_updated(false);
01609
01610     }
01611
01612     /** Assign a pump time-series message to this dive data object.
01613      *
01614      *  @note           This vector is sorted upon insertion by order of non-decreasing message indices.
01615      *
01616      *  @param[in]      x       A pointer to the pump time-series message to assign.
01617      */
01618     void AssignPumpTimeSeries(const shared_ptr<x::XPump>& x) {
01619
01620       // Get an exclusive lock.
01621       lock_guard<recursive_mutex> guard(mutex_);
01622
01623       // Iterate to emplacement position.
01624       auto it = xmsg_pump_time_series_.begin();
01625       for(; it < xmsg_pump_time_series_.end(); ++it)
01626
01627         // Check for duplicate messages.
01628         if(x->xmsg_index() == (*it)->xmsg_index())
01629
01630           // Duplicate message encountered.
01631           EXC_THROW("Duplicate X-Message index encountered!");
01632
01633         else if(x->xmsg_index() < (*it)->xmsg_index())
01634
01635           // Emplacement position found.
01636           break;
01637
01638       // Emplace element at position.
01639       xmsg_pump_time_series_.emplace(it, x);
01640
01641       // Update file status.
01642       set_listing_file_updated(false);
01643
01644     }
01645
01646     /** Assign a rise time-series message to this dive data object.
01647      *
01648      *  @note           This vector is sorted upon insertion by order of non-decreasing message indices.
01649      *
01650      *  @param[in]      x       A pointer to the rise time-series message to assign.
01651      */
01652     void AssignRiseTimeSeries(const shared_ptr<x::XRise>& x) {
01653
01654       // Get an exclusive lock.
01655       lock_guard<recursive_mutex> guard(mutex_);
01656
01657       // Iterate to emplacement position.
01658       auto it = xmsg_rise_time_series_.begin();
01659       for(; it < xmsg_rise_time_series_.end(); ++it)
01660
01661         // Check for duplicate messages.
01662         if(x->xmsg_index() == (*it)->xmsg_index())
01663
01664           // Duplicate message encountered.
01665           EXC_THROW("Duplicate X-Message index encountered!");
01666
01667         else if(x->xmsg_index() < (*it)->xmsg_index())
01668
01669           // Emplacement position found.
01670           break;
01671
01672       // Emplace element at position.
01673       xmsg_rise_time_series_.emplace(it, x);
01674
01675       // Update file status.
01676       set_listing_file_updated(false);
01677
01678     }
01679
01680     /** Assign a surface pressure time-series message to this dive data object.
01681      *
01682      *  @note           This message type is used by ALAMO floats only.
01683      *  @note           This vector is sorted upon insertion by order of non-decreasing message indices.
01684      *
01685      *  @param[in]      x       A pointer to the surface pressure time-series message to assign.
01686      */
01687     void AssignSurfaceTimeSeries(const shared_ptr<x::XSurfaceData>& x) {
01688
01689       try {
```

```
01690
01691         // Get an exclusive lock.
01692         lock_guard<recursive_mutex> guard(mutex_);
01693
01694         // Assign the message.
01695         xmsg_surface_time_series_.AssignMessage(x);
01696
01697         // Update file status.
01698         set_listing_file_updated(false);
01699
01700       }CATCH_BLOCK("Failed to assign surface time-series data message!")
01701
01702     }
01703
01704     /** Assign a valve data message to this dive data object.
01705      *
01706      *  @note           The valve data messages are kept sorted by message index.
01707      *
01708      *  @param[in]      x       A pointer to the valve data message to assign.
01709      *
01710      *  @throw          util::log::MessageException if a message with the same index has already
01711      *                  been assigned.
01712      */
01713     void AssignValveData(const shared_ptr<x::XValve>& x) {
01714
01715       // Get an exclusive lock.
01716       lock_guard<recursive_mutex> guard(mutex_);
01717
01718       // Iterate to emplacement position.
01719       auto it = xmsg_valve_data_.begin();
01720       for(; it < xmsg_valve_data_.end(); ++it)
01721
01722         // Check for duplicate messages.
01723         if(x->xmsg_index() == (*it)->xmsg_index())
01724
01725           // Duplicate message encountered.
01726           EXC_THROW("Duplicate X-Message index encountered!");
01727
01728         else if(x->xmsg_index() < (*it)->xmsg_index())
01729
01730           // Emplacement position found.
01731           break;
01732
01733       // Emplace element at position.
01734       xmsg_valve_data_.emplace(it, x);
01735
01736       // Update file status.
01737       set_listing_file_updated(false);
01738
01739     }
01740
01741     /** Set the enumerated Argo instrument type identifier.
01742      *
01743      *  @param[in]  type        The enumerated Argo instrument type identifier.
01744      */
01745     void set_argo_instrument_type(XArgoInstrumentType_t type) {
01746
01747       // Get an exclusive lock.
01748       lock_guard<recursive_mutex> guard(mutex_);
01749
01750       argo_instrument_type_ = type;
01751     }
01752
01753     void set_complete(bool complete) {
01754
01755       // Get an exclusive lock.
01756       lock_guard<recursive_mutex> guard(mutex_);
01757
01758       complete_ = complete;
01759     }
01760
01761     /** Set the dive or cycle number of this dive data object.
01762      *
01763      *  @param[in]  id_dive     The dive or cycle number.
01764      */
01765     void set_id_dive(int id_dive) {
01766
01767       // Get an exclusive lock.
01768       lock_guard<recursive_mutex> guard(mutex_);
01769
01770       id_dive_ = id_dive;
01771     }
01772
01773     /** Set the path to the intermediate listing file.
01774      *
01775      *  @param[in]  file_path   The path to the intermediate listing file.
01776      */
01777     void set_intermediate_listing_file_path(const path& file_path) {
01778
01779       // Get an exclusive lock.
01780       lock_guard<recursive_mutex> guard(mutex_);
01781
01782       listing_file_path_ = file_path;
01783     }
01784
01785     void set_iridium_position_datetime_first(const ptime& iridium_position_datetime_first) {
01786
01787       // Get an exclusive lock.
01788       lock_guard<recursive_mutex> guard(mutex_);
01789
01790       iridium_position_datetime_first_ = iridium_position_datetime_first;
01791     }
01792
01793     void set_iridium_position_datetime_last(const ptime& iridium_position_datetime_last) {
01794
01795       // Get an exclusive lock.
01796       lock_guard<recursive_mutex> guard(mutex_);
01797
```

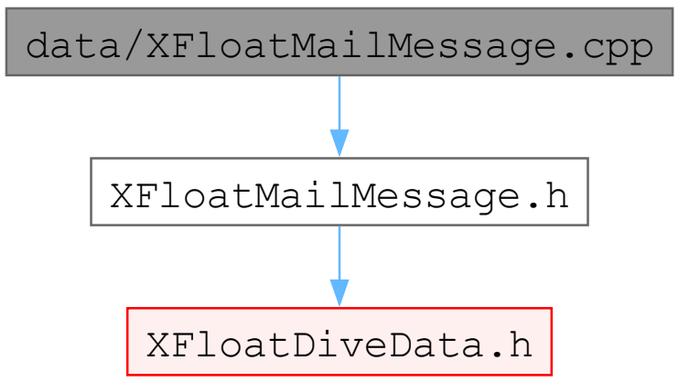
```
01798       iridium_position_datetime_last_ = iridium_position_datetime_last;
01799     }
01800
01801     void set_listing_file_updated(bool listing_file_updated) {
01802
01803       // Get an exclusive lock.
01804       lock_guard<recursive_mutex> guard(mutex_);
01805
01806       listing_file_updated_ = listing_file_updated;
01807     }
01808
01809     void set_netcdf_file_path(const path& netcdf_file_path) {
01810
01811       // Get an exclusive lock.
01812       lock_guard<recursive_mutex> guard(mutex_);
01813
01814       netcdf_file_path_ = netcdf_file_path;
01815     }
01816
01817     void set_netcdf_file_updated(bool netcdf_file_updated = true) {
01818
01819       // Get an exclusive lock.
01820       lock_guard<recursive_mutex> guard(mutex_);
01821
01822       netcdf_file_updated_ = netcdf_file_updated;
01823     }
01824
01825     void set_num_sbd_msgs(uint16_t num_sbd_msgs) {
01826
01827       // Get an exclusive lock.
01828       lock_guard<recursive_mutex> guard(mutex_);
01829
01830       num_sbd_msgs_ = num_sbd_msgs;
01831     }
01832
01833     /** Set the path to the PHY listing file.
01834      *
01835      *  @param[in]  file_path   The path to the PHY listing file.
01836      */
01837     void set_phy_file_path(const path& file_path) {
01838
01839       // Get an exclusive lock.
01840       lock_guard<recursive_mutex> guard(mutex_);
01841
01842       phy_file_path_ = file_path;
01843     }
01844
01845     void set_phy_file_updated(bool phy_file_updated) {
01846
01847       // Get an exclusive lock.
01848       lock_guard<recursive_mutex> guard(mutex_);
01849
01850       phy_file_updated_ = phy_file_updated;
01851     }
01852
01853     void set_surface_pres_max(float surface_pres_max) {
01854
01855       // Get an exclusive lock.
01856       lock_guard<recursive_mutex> guard(mutex_);
01857
01858       surface_pres_max_ = surface_pres_max;
01859     }
01860
01861     void set_surface_pres_mean(float surface_pres_mean) {
01862
01863       // Get an exclusive lock.
01864       lock_guard<recursive_mutex> guard(mutex_);
01865
01866       surface_pres_mean_ = surface_pres_mean;
01867     }
01868
01869     void set_surface_pres_min(float surface_pres_min) {
01870
01871       // Get an exclusive lock.
01872       lock_guard<recursive_mutex> guard(mutex_);
01873
01874       surface_pres_min_ = surface_pres_min;
01875     }
01876
01877     void set_surface_pres_valid(bool surface_pres_valid) {
01878
01879       // Get an exclusive lock.
01880       lock_guard<recursive_mutex> guard(mutex_);
01881
01882       surface_pres_valid_ = surface_pres_valid;
01883     }
01884
01885     void set_surface_pres_variance(float surface_pres_variance) {
01886
01887       // Get an exclusive lock.
01888       lock_guard<recursive_mutex> guard(mutex_);
01889
01890       surface_pres_variance_ = surface_pres_variance;
01891     }
01892
01893     void set_pres_offs(float pres_offs) {
01894
01895       // Get an exclusive lock.
01896       lock_guard<recursive_mutex> guard(mutex_);
01897
01898       pres_offs_ = pres_offs;
01899     }
01900
01901     void set_pres_offs_num_dives(int pres_offs_num_dives) {
01902
01903       // Get an exclusive lock.
01904       lock_guard<recursive_mutex> guard(mutex_);
01905
```

```
01906       pres_offs_num_dives_ = pres_offs_num_dives;
01907     }
01908
01909     void set_pres_offs_num_samples(int pres_offs_num_samples) {
01910
01911       // Get an exclusive lock.
01912       lock_guard<recursive_mutex> guard(mutex_);
01913
01914       pres_offs_num_samples_ = pres_offs_num_samples;
01915     }
01916
01917     void set_pres_offs_valid(bool pres_offs_valid) {
01918
01919       // Get an exclusive lock.
01920       lock_guard<recursive_mutex> guard(mutex_);
01921
01922       pres_offs_valid_ = pres_offs_valid;
01923     }
01924
01925     void set_surface_pres_num_samples(int surface_pres_num_samples) {
01926       surface_pres_num_samples_ = surface_pres_num_samples;
01927     }
01928
01929
01930 };
01931
01932 } /* namespace xfloat */
01933 } /* namespace iridium */
01934 } /* namespace argo */
01935
01936 #endif /* XFLOATDIVEDATA_H_ */
```

11.25 data/XFloatMailMessage.cpp File Reference

The argo::iridium::xfloat::XFloatMailMessage class source file.

`#include "XFloatMailMessage.h"`
Include dependency graph for XFloatMailMessage.cpp:



Namespaces

- namespace argo

  The Argo instrument namespace.
- namespace argo::iridium

  The Argo Iridium instrument namespace.
- namespace argo::iridium::xfloat

  The X-Float instrument namespace.
- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

Functions

- ostream & argo::iridium::xfloat::operator<< (ostream &os, const XFloatMailMessage &mail)

  The stream insertion operator overload function for XFloatMailMessage.
- ostream & argo::iridium::xfloat::operator<< (ostream &os, const XMailSbdData_t &data)

  The stream insertion operator for the SBD data type.
- istream & argo::iridium::xfloat::operator>> (istream &is, XFloatMailMessage &mail)

  The stream extraction operator overload function for XFloatMailMessage.

11.25.1 Detailed Description

The argo::iridium::xfloat::XFloatMailMessage class source file.

This is the header file for the argo::iridium::xfloat::XFloatMailMessage class. This class represents a single mail message received from the Iridium system, including fields that are parsed from the various parts of the mail message and the attached Iridium Short Burst Data (SBD) file. It is used to parse, validate, and write the mail message files to the local filesystem.

This file contains the static data members of the class and some of the class's functionality.

Version

0.6

Date

May 12, 2014 4:31:19 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

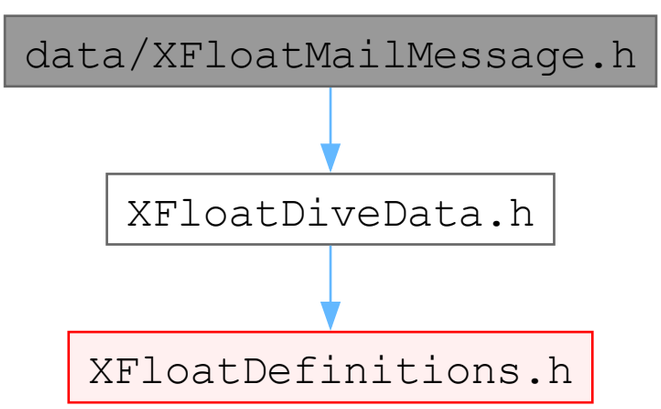
This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.26 data/XFloatMailMessage.h File Reference

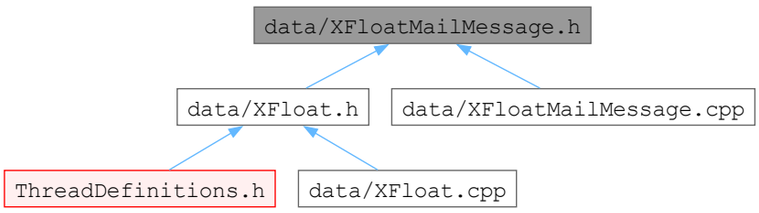
The argo::iridium::xfloat::XFloatMailMessage class header file.

```
#include "XFloatDiveData.h"
```

Include dependency graph for XFloatMailMessage.h:



This graph shows which files directly or indirectly include this file:



Classes

- class argo::iridium::xfloat::XFloatMailMessage

  The mail message class.

Namespaces

- namespace argo

  The Argo instrument namespace.
- namespace argo::iridium

  The Argo Iridium instrument namespace.
- namespace argo::iridium::xfloat

  The X-Float instrument namespace.

Macros

- #define XMAIL_DFLT_MSG_HEADER_VALID_SNDR_REGEX "(sbdservice@sbd.iridium.com|service@sbd.↩ pac.disa.mil)"

  The default mail message header valid sender regular expression value.
- #define XMAIL_DFLT_MSG_HEADER_VALID_SUBJ_REGEX "SBD Msg From Unit: (\\d+)"

  The default mail message header valid subject regular expression value.
- #define XMAIL_DFLT_RAW_FILE_FORMAT XMAIL_RAW_FORMAT_EXTENDED

  The default mail message raw file format.
- #define XMAIL_DFLT_RAW_FILE_GENERATRION_ENABLE true

  The default mail message RAW-mail-file-generation-is-enabled flag value.
- #define XMAIL_DFLT_SBD_FILE_GENERATRION_ENABLE true

  The default mail message binary-SBD-file-generation-is-enabled flag value.
- #define XMAIL_DFLT_SBD_FILE_VALID_NAME_REGEX "(\\d+)_(\\d{6})\\.sbd"

  The default mail message binary SBD file attachment file name valid regular expression.

Typedefs

- typedef uint8_t argo::iridium::xfloat::XMailSbdByte_t

  The SBD byte type.
- typedef vector< XMailSbdByte_t > argo::iridium::xfloat::XMailSbdData_t

  The SBD data type.

Enumerations

- enum argo::iridium::xfloat::XFloatMailRawFileFormat_t {
  argo::iridium::xfloat::XMAIL_RAW_FORMAT_STANDARD = 0 ,
  argo::iridium::xfloat::XMAIL_RAW_FORMAT_EXTENDED = 1 }

  The mail message raw file format type identifier enumeration.

Functions

- ostream & argo::iridium::xfloat::operator<< (ostream &os, const XFloatMailMessage &mail)

  The stream insertion operator overload function for XFloatMailMessage.
- ostream & argo::iridium::xfloat::operator<< (ostream &os, const XMailSbdData_t &data)

  The stream insertion operator for the SBD data type.
- istream & argo::iridium::xfloat::operator>> (istream &is, XFloatMailMessage &mail)

  The stream extraction operator overload function for XFloatMailMessage.

11.26.1 Detailed Description

The argo::iridium::xfloat::XFloatMailMessage class header file.

This is the header file for the argo::iridium::xfloat::XFloatMailMessage class. This class represents a single mail message received from the Iridium system, including fields that are parsed from the various parts of the mail message and the attached Iridium Short Burst Data (SBD) file. It is used to parse, validate, and write the mail message files to the local filesystem.

This file contains the class definition and the majority of the functionality of the class.

Version

0.6

Date

May 12, 2014 4:31:19 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.26.2 Macro Definition Documentation

11.26.2.1 XMAIL_DFLT_MSG_HEADER_VALID_SNDR_REGEX

```
#define XMAIL_DFLT_MSG_HEADER_VALID_SNDR_REGEX  "(sbdservice@sbd.iridium.com|service@sbd.pac.↩
disa.mil)"
```

The default mail message header valid sender regular expression value.

The mail message header valid sender regular expression value, argo::iridium::xfloat::XFloatMailMessage::msg_header_valid_sender_reg is set as a result of parsing the command-line arguments and/or configuration file options during application initialization. If the option is not explicitly specified, then it will default to this value.

Referenced by util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription().

11.26.2.2 XMAIL_DFLT_MSG_HEADER_VALID_SUBJ_REGEX

```
#define XMAIL_DFLT_MSG_HEADER_VALID_SUBJ_REGEX "SBD Msg From Unit:  (\\d+)"
```

The default mail message header valid subject regular expression value.

The mail message header valid subject regular expression value, argo::iridium::xfloat::XFloatMailMessage::msg_header_valid_subject_reg
is set as a result of parsing the command-line arguments and/or configuration file options during application initialization.
If the option is not explicitly specified, then it will default to this value.

Referenced by util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription().

11.26.2.3 XMAIL_DFLT_RAW_FILE_FORMAT

```
#define XMAIL_DFLT_RAW_FILE_FORMAT XMAIL_RAW_FORMAT_EXTENDED
```

The default mail message raw file format.

The mail message raw file format, argo::iridium::xfloat::XFloatMailMessage::raw_file_format_, is set as a result of parsing the command-line arguments and/or configuration file options during application initialization. If the option is not explicitly specified, then it will default to this value.

11.26.2.4 XMAIL_DFLT_RAW_FILE_GENERATRION_ENABLE

```
#define XMAIL_DFLT_RAW_FILE_GENERATRION_ENABLE true
```

The default mail message RAW-mail-file-generation-is-enabled flag value.

The mail message RAW-mail-file-generation-is-enabled flag value, argo::iridium::xfloat::XFloatMailMessage::raw_file_generation_is_enabl
is set as a result of parsing the command-line arguments and/or configuration file options during application initialization.
If the option is not explicitly specified, then it will default to this value.

11.26.2.5 XMAIL_DFLT_SBD_FILE_GENERATRION_ENABLE

```
#define XMAIL_DFLT_SBD_FILE_GENERATRION_ENABLE true
```

The default mail message binary-SBD-file-generation-is-enabled flag value.

The mail message binary-SBD-file-generation-is-enabled flag value, argo::iridium::xfloat::XFloatMailMessage::sbd_file_generation_is_ena
is set as a result of parsing the command-line arguments and/or configuration file options during application initialization.
If the option is not explicitly specified, then it will default to this value.

11.26.2.6 XMAIL_DFLT_SBD_FILE_VALID_NAME_REGEX

```
#define XMAIL_DFLT_SBD_FILE_VALID_NAME_REGEX "(\\d+)_(\\d{6})\\.sbd"
```

The default mail message binary SBD file attachment file name valid regular expression.

The mail message binary SBD file attachment file name valid regular expression, argo::iridium::xfloat::XFloatMailMessage::sbd_file_valid_r
is set as a result of parsing the command-line arguments and/or configuration file options during application initialization.
If the option is not explicitly specified, then it will default to this value.

Referenced by util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription().

11.27 XFloatMailMessage.h

Go to the documentation of this file.

```
00001 /** The @ref argo::iridium::xfloat::XFloatMailMessage class header file.
00002  *
00003  *  This is the header file for the @ref argo::iridium::xfloat::XFloatMailMessage class. This
00004  *  class represents a single mail message received from the Iridium system, including fields
00005  *  that are parsed from the various parts of the mail message and the attached Iridium Short
00006  *  Burst Data (SBD) file. It is used to parse, validate, and write the mail message files to the
00007  *  local filesystem.
00008  *
00009  *  This file contains the class definition and the majority of the functionality of the class.
00010  *
00011  *  @file
00012  *
00013  *  @version    0.6
00014  *
00015  *  @date       May 12, 2014 4:31:19 PM
00016  *
00017  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00018  *              Engineer I,
00019  *              Department of Physical Oceanography,
00020  *              Woods Hole Oceanographic Institution
00021  *
00022  *  @copyright  2014 Woods Hole Oceanographic Institution.
00023  *              Released under the GNU General Public License.
00024  *
00025  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00026  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00027  *  General Public License for more details. You should have received a copy of the GNU General
00028  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00029  */
00030
00031 #ifndef XFLOATMAILMESSAGE_H_
00032 #define XFLOATMAILMESSAGE_H_
00033
00034 /* Includes: */
00035
00036 // local
00037 #include "XFloatDiveData.h"
00038
00039 /* Definitions: */
00040
00041 /** The default mail message header valid sender regular expression value.
00042  *
00043  *  The mail message header valid sender regular expression value, @ref
00044  *  argo::iridium::xfloat::XFloatMailMessage::msg_header_valid_sender_regex_, is set as a result
00045  *  of parsing the command-line arguments and/or configuration file options during application
00046  *  initialization. If the option is not explicitly specified, then it will default to this value.
00047  */
00048 #define XMAIL_DFLT_MSG_HEADER_VALID_SNDR_REGEX \
00049     "(sbdservice@sbd.iridium.com|service@sbd.pac.disa.mil)"
00050
00051 /** The default mail message header valid subject regular expression value.
00052  *
00053  *  The mail message header valid subject regular expression value, @ref
00054  *  argo::iridium::xfloat::XFloatMailMessage::msg_header_valid_subject_regex_, is set as a result
00055  *  of parsing the command-line arguments and/or configuration file options during application
00056  *  initialization. If the option is not explicitly specified, then it will default to this value.
00057  */
00058 #define XMAIL_DFLT_MSG_HEADER_VALID_SUBJ_REGEX  "SBD Msg From Unit: (\\d+)"
00059
00060 /** The default mail message binary SBD file attachment file name valid regular expression.
00061  *
00062  *  The mail message binary SBD file attachment file name valid regular expression, @ref
00063  *  argo::iridium::xfloat::XFloatMailMessage::sbd_file_valid_name_regex_, is set as a result
00064  *  of parsing the command-line arguments and/or configuration file options during application
00065  *  initialization. If the option is not explicitly specified, then it will default to this value.
00066  */
00067 #define XMAIL_DFLT_SBD_FILE_VALID_NAME_REGEX    "(\\d+)_(\\d{6})\\.sbd"
00068
00069 /** The default mail message raw file format.
00070  *
00071  *  The mail message raw file format, @ref
00072  *  argo::iridium::xfloat::XFloatMailMessage::raw_file_format_, is set as a result of parsing the
00073  *  command-line arguments and/or configuration file options during application initialization. If
00074  *  the option is not explicitly specified, then it will default to this value.
00075  */
00076 #define XMAIL_DFLT_RAW_FILE_FORMAT              XMAIL_RAW_FORMAT_EXTENDED
00077
00078 /** The default mail message binary-SBD-file-generation-is-enabled flag value.
00079  *
00080  *  The mail message binary-SBD-file-generation-is-enabled flag value, @ref
00081  *  argo::iridium::xfloat::XFloatMailMessage::sbd_file_generation_is_enabled_, is set as a result
00082  *  of parsing the command-line arguments and/or configuration file options during application
00083  *  initialization. If the option is not explicitly specified, then it will default to this value.
00084  *
00085  */
00086 #define XMAIL_DFLT_SBD_FILE_GENERATRION_ENABLE  true
00087
00088 /** The default mail message RAW-mail-file-generation-is-enabled flag value.
00089  *
00090  *  The mail message RAW-mail-file-generation-is-enabled flag value, @ref
00091  *  argo::iridium::xfloat::XFloatMailMessage::raw_file_generation_is_enabled_, is set as a result
00092  *  of parsing the command-line arguments and/or configuration file options during application
00093  *  initialization. If the option is not explicitly specified, then it will default to this value.
00094  */
00095 #define XMAIL_DFLT_RAW_FILE_GENERATRION_ENABLE  true
00096
00097 namespace argo {
00098 namespace iridium {
00099 namespace xfloat {
00100
00101 /* Type Definitions: */
00102
00103 /** The SBD byte type.
```

```
00104  *
00105  *  This type definition determines the type that is used to represent a single byte the Iridium
00106  *  Short Burst Data (SBD) message. Generally, this field should be a single signed or unsigned
00107  *  byte type.
00108  */
00109 typedef uint8_t XMailSbdByte_t;
00110
00111 /** The SBD data type.
00112  *
00113  *  This type definition determines the type that is used to represent the Iridium Short Burst
00114  *  Data (SBD) message. It is defined to be a vector of bytes of type @ref XMailSbdByte_t.
00115  */
00116 typedef vector<XMailSbdByte_t> XMailSbdData_t;
00117
00118 /** The mail message raw file format type identifier enumeration.
00119  *
00120  *  This enumeration lists the supported formats for the raw mail message file. The format of RAW
00121  *  mail message files that are written to the local files system will be specified by one of the
00122  *  values of this enumeration.
00123  */
00124 typedef enum XFloatMailRawFileFormat {
00125
00126   /** The standard or legacy format for raw mail message files.
00127    *
00128    *  This format represents the standard raw mail message file format in which the attachment
00129    *  data is encoded in base-64. This format is currently used by the Solo-II decoding software.
00130    */
00131   XMAIL_RAW_FORMAT_STANDARD = 0,
00132
00133   /** The extended format for raw mail message files.
00134    *
00135    *  This format is an extended version of the raw file format in which the attachment data
00136    *  is encoded in hex, rather than base-64.
00137    */
00138   XMAIL_RAW_FORMAT_EXTENDED = 1
00139
00140 } XFloatMailRawFileFormat_t;
00141
00142 /* Class Definition: */
00143
00144 /** The mail message class.
00145  *
00146  *  This class represents a single mail message received from the Iridium system, including
00147  *  fields that are parsed from the various parts of the mail message and the attached Iridium
00148  *  Short Burst Data (SBD) file. It is used to parse, validate, and write the mail message files
00149  *  to the local filesystem.
00150  *
00151  *
00152  *  @note   This class is mapped to an SQL database table by the @ref sql_table_description_
00153  *          object. This allows some or all of the field of this class to be persisted to the SQL
00154  *          database. Likewise, an object of this type may be constructed from the results of an
00155  *          SQL query to the corresponding table.
00156  */
00157 class XFloatMailMessage:
00158    protected util::file::FileSystemUtils,
00159    public Iridium,
00160    public IridiumPosition,
00161    public util::mysql::LocalPersistentEntity {
00162
00163    /* Friends Classes: */
00164
00165    friend class XFloat;
00166
00167    friend class XFloatDiveData;
00168
00169    friend class XFloatMailProcessingService;
00170
00171    friend class XFloatMailRetrievalService;
00172
00173    /* Friend Functions: */
00174
00175    friend ostream& operator«(ostream& os, const XFloatMailMessage& mail);
00176
00177    friend istream& operator»(istream& is, XFloatMailMessage& mail) ;
00178
00179    /** Friend function for populating some of the static members of this class from the
00180     *  configuration file options.
00181     */
00182    friend options_description
     util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription();
00183
00184  private:
00185
00186    /* Private Static Data Members: */
00187
00188    /** The mail message header valid sender regular expression.
00189     *
00190     *  This regular expression is used the validate the FROM field of the email message header.
00191     *  If the mail message header FROM field does not match this regular expression, the the mail
00192     *  message will be considered invalid and will be discarded.
00193     *
00194     *  The value of this field is set as a result of parsing the command-line arguments and/or
00195     *  configuration file options during application initialization. If the value is unspecified,
00196     *  then it defaults to a value of @ref XMAIL_DFLT_MSG_HEADER_VALID_SNDR_REGEX.
00197     */
00198    static string msg_header_valid_sender_regex_;
00199
00200    /** The mail message header valid subject regular expression.
00201     *
00202     *  This regular expression is used to validate the SUBJECT field of the email message header.
00203     *  If the mail message header SUBJECT field does not match this regular expression, the the
00204     *  mail message will be considered invalid and will be discarded.
00205     *
00206     *  The value of this field is set as a result of parsing the command-line arguments and/or
00207     *  configuration file options during application initialization. If the value is unspecified,
00208     *  then it defaults to a value of @ref XMAIL_DFLT_MSG_HEADER_VALID_SUBJ_REGEX.
00209     */
00210    static string msg_header_valid_subject_regex_;
```

```
00211
00212     /** The RAW mail message file directory.
00213      *
00214      *  This field determines the directory to which RAW mail message files are written. If
00215      *  enabled, RAW mail message files will be written to subdirectories of this directory that
00216      *  are organized by the unique instrument identifiers.
00217      *
00218      *  The value of this field is set as a result of parsing the command-line arguments and/or
00219      *  configuration file options during application initialization.
00220      */
00221     static path raw_directory_path_;
00222
00223     /** The raw mail message file format.
00224      *
00225      *  The value of this field determines the format of the raw mail message that is written to
00226      *  the local filesystem. The support formats are enumerated by @ref XFloatMailRawFileFormat_t.
00227      *
00228      *  The value of this field is set as a result of parsing the command-line arguments and/or
00229      *  configuration file options during application initialization. If the value is unspecified,
00230      *  then it defaults to a value of @ref XMAIL_DFLT_RAW_FILE_FORMAT.
00231      */
00232     static int raw_file_format_;
00233
00234     /** The global RAW-mail-message-file-generation-is-enabled flag.
00235      *
00236      *  This flag determines whether the ASCII RAW mail message files are written to the local
00237      *  filesystem. If TRUE, then the RAW mail message files will be written to subdirectories of
00238      *  the raw mail message directory which is specified by @ref raw_directory_path_. Otherwise,
00239      *  if FALSE, raw mail message will not be written to the local filesystem.
00240      *
00241      *  The value of this field is set as a result of parsing the command-line arguments and/or
00242      *  configuration file options during application initialization. If the value is unspecified,
00243      *  then it defaults to a value of @ref XMAIL_DFLT_SBD_FILE_VALID_NAME_REGEX.
00244      */
00245     static bool raw_file_generation_is_enabled_;
00246
00247     /** The global binary-SBD-file-generation-is-enabled flag.
00248      *
00249      *  This flag determines whether binary Iridium Short Burst Data (SBD) files are written to
00250      *  the local file system. If TRUE, then SBD files will be written to subdirectories of the
00251      *  SBD directory which is specified by @ref sbd_directory_path_. Otherwise, if FALSE, SBD
00252      *  files will not be written to the local filesystem.
00253      *
00254      *  The value of this field is set as a result of parsing the command-line arguments and/or
00255      *  configuration file options during application initialization. If the value is unspecified,
00256      *  then it defaults to a value of FALSE.
00257      */
00258     static bool sbd_file_generation_is_enabled_;
00259
00260     /** The valid SBD file email attachment filename regular expression.
00261      *
00262      *  This regular expression is used to validate the filename of the email message attachment.
00263      *  If the filename of the file attachment to the mail message matches this regular expression,
00264      *  then the mail message will be considered valid. Otherwise, if the filename does not match
00265      *  this regular expression, then the mail message will be considered invalid and will be
00266      *  discarded.
00267      *
00268      *  The value of this field is set as a result of parsing the command-line arguments and/or
00269      *  configuration file options during application initialization. If the value is unspecified,
00270      *  then it defaults to a value of @ref XMAIL_DFLT_SBD_FILE_VALID_NAME_REGEX.
00271      */
00272     static string sbd_file_valid_name_regex_;
00273
00274     /** The SBD file directory.
00275      *
00276      *  This field determines the directory to which the binary Iridium Short Burst Data (SBD)
00277      *  files are written. If enabled, SBD files will be written to subdirectories of this
00278      *  directory that are organized by the unique instrument identifiers.
00279      *
00280      *  The value of this field is set as a result of parsing the command-line arguments and/or
00281      *  configuration file options during application initialization.
00282      */
00283     static path sbd_directory_path_;
00284
00285     /** The SQL table description object.
00286      *
00287      *  This SQL table description provides the details needed to map this object to a relational
00288      *  database table. It utilizes the accessor methods of the class and the details of the SQL
00289      *  table column to (1) create an SQL INSERT statement to save the object into the database,
00290      *  (2) populate one or more objects' fields as a result of an SQL SELECT query, or (3) query
00291      *  the SQL database to access one or more of the values that are stored the the SQL table
00292      *  corresponding to this object.
00293      */
00294     static const util::mysql::SqlTableDescription<XFloatMailMessage> sql_table_description_;
00295
00296   private:
00297
00298     /* Private Data Members: */
00299
00300     /** The dive or cycle number of this mail message object.
00301      *
00302      *  This field is provides a non-unique index of the dive or cycle number of the mail message
00303      *  object for a given instrument. It is used to determine to which dive this mail message
00304      *  belongs. Together, the dive or cycle number and the Iridium Short Burst Data (SBD) packet
00305      *  index, @ref id_sbd_packet_, provide a unique index of the mail message. However, these
00306      *  fields are not used as the primary index of the mail message objects.
00307      *
00308      *  The value of this field is set as a result of parsing the mail message attachment. The
00309      *  value is read from the header of the attached SBD file. If the mail message has no SBD
00310      *  attachment, or the attachment is invalid, then the value of this field will be set to
00311      *  @ref XFLT_DIVE_UNKNOWN, which specifies an unknown dive or cycle number.
00312      */
00313     int id_dive_;
00314
00315     /** The mail message unique identifier string.
00316      *
00317      *  This character sequence is a unique identifier for the mail message that is assigned by
00318      *  the mail server. However, this value is not used a primary identifier because the mail
```

```
00319      *  server all provides a unique integer identifier.
00320      *
00321      *  The value of this field is set as a result of parsing the mail message header. All mail
00322      *  messages will be assigned a unique identifier string, regardless of their validity.
00323      */
00324     string id_msg_;
00325
00326     /** The mail message unique identifier (UID).
00327      *
00328      *  This unique integer identifier is assigned by the remote mail server. It is used as the
00329      *  primary unique identifier for a mail message.
00330      *
00331      *  The mail messages' UIDs also specify the order in which the mail messages were received by
00332      *  the mail server. Each message that the mail server receives is assigned a monotonically
00333      *  increasing UID value. Thus this field is also used to determine which mail messages have
00334      *  already been fetch from the mail server. When the mail retrieval thread, @ref
00335      *  XFloatMailRetrievalService, queries the maximum UID of mail messages that have been
00336      *  persisted to the SQL database. The result of the query plus one determines the minimum UID
00337      *  of the mail messages to fetch from the mail server.
00338      *
00339      *
00340      *  The value of this field is set as a result of parsing the mail message header. All mail
00341      *  messages will be assigned a unique integer identifier, regardless of their validity.
00342      */
00343     long id_msg_uid_;
00344
00345     /** The Iridium Short Burst Data (SBD) Service Mobile Originated Message Sequence Number
00346      *  (MOMSN) identifier.
00347      *
00348      *  This identifier, provided by the Iridium system, uniquely identifies the sequence number
00349      *  of messages successfully sent from the instrument to the ground station. For each new
00350      *  message that is transmitted from the instrument, this number is incremented. It is
00351      *  provided in the body of the email from the Iridium system along with information about
00352      *  the position, etc.
00353      *
00354      *  The value of this field is set as a result of parsing the mail message body text. If the
00355      *  mail message body is invalid, then the value of this field will be set to @ref
00356      *  XFLT_ID_UNKNOWN.
00357      *
00358      *  @warning    This identifier should NOT be used to unique identify mail messages for a given
00359      *              instrument. If the instrument experiences a incomplete or failed Iridium
00360      *              transmission that results in an email from the Iridium system, then MOMSN will
00361      *              NOT be incremented for that message. Thus, there may exist multiple mail
00362      *              messages with the same MOMSN identifier.
00363      */
00364     int id_sbd_momsn_;
00365
00366     /** The Iridium Short Burst Data (SBD) Service Mobile Terminated Message Sequence Number
00367      *  (MTMSN) identifier.
00368      *
00369      *  This identifier, provided by the Iridium system, uniquely identifies the sequence number
00370      *  of messages successfully sent from the ground station to the instrument. For each new
00371      *  message that is transmitted to the instrument from the ground station, this number is
00372      *  incremented. It is provided in the body of the email from the Iridium system along with
00373      *  information about the position, etc.
00374      *
00375      *  The value of this field is set as a result of parsing the mail message body text. If the
00376      *  mail message body is invalid, then the value of this field will be set to @ref
00377      *  XFLT_ID_UNKNOWN.
00378      */
00379     int id_sbd_mtmsn_;
00380
00381     /** The Iridium Short Burst Data (SBD) message packet index.
00382      *
00383      *  This identifier uniquely indexes each SBD message transmitted for a single dive. It is
00384      *  used both to reconstruct the sequence in which the message were transmitted and to
00385      *  determine whether any of the message in the sequence are missing. A complete set of mail
00386      *  messages for a given dive will have a sequence of monotonically increasing SBD packet
00387      *  identifiers starting with a value of zero.
00388      *
00389      *  The value of this field is set as a result of parsing the mail message attachment. The
00390      *  value is read from the header of the attached SBD file. If the mail message has no SBD
00391      *  attachment, or the attachment is invalid, then the value of this field will be set to
00392      *  @ref XFLT_DIVE_UNKNOWN, which specifies an unknown SBD message packet index.
00393      */
00394     int id_sbd_packet_;
00395
00396     /** The mail message body is valid flag.
00397      *
00398      *  This field specifies the validity of the mail message body. It is set as a result of
00399      *  parsing the mail message body text, which contains information about the Iridium session
00400      *  including the Iridium position, date/time, status, and message identifiers.
00401      */
00402     bool msg_body_is_valid_;
00403
00404     /** The mail message body text.
00405      *
00406      *  This field contains the full text of the body of the mail message. The body of a valid
00407      *  mail message includes information about the Iridium session that is parsed and stored in
00408      *  the corresponding field of this object. These field include
00409      *      - the MOMSN identifier,
00410      *      - the MTMSN identifier,
00411      *      - the time of the Iridium session,
00412      *      - the Iridium session status code and status string, and
00413      *      - the Iridium position and Circular Error Probable (CEP) radius.
00414      */
00415     string msg_body_text_;
00416
00417     /** The mail message header date/time.
00418      *
00419      *  This field represents the date/time-of-flight of the mail message from the Iridium system.
00420      *  The value of this field, is set as a result of parsing the mail message header.
00421      *
00422      *  @note       This date/time, along with the Iridium session time may be used to calculate
00423      *              the latency of the Iridium system.
00424      */
00425     ptime msg_header_datetime_;
00426
```

```
00427     /** The mail message header FROM field.
00428      *
00429      *  This field represents the sender of the mail message. This field is used to validate the
00430      *  mail message by ensuring the only messages from desired senders are consumed by the
00431      *  application. The value of this field is set as a result of parsing the mail message header.
00432      */
00433     string msg_header_from_;
00434
00435     /** The mail message header is valid flag.
00436      *
00437      *  This field indicates the validity of the mail message header. It is set as a result of
00438      *  parsing the mail message header. The criteria for a valid mail message header is that
00439      *      - the SUBJECT field of the mail message header matches the valid sender regular
00440      *        expression, @ref msg_header_valid_sender_regex_, and
00441      *      - the FROM field of the mail message header matcher the valid subject regular
00442      *        expression, @ref msg_header_valid_subject_regex_.
00443      */
00444     bool msg_header_is_valid_;
00445
00446     /** The mail message header raw text.
00447      *
00448      *  This field contains the "raw" text of the mail message header. This text represents the
00449      *  header of the mail message in its native form, which includes additional information about
00450      *  routing, etc. that is not currently parsed from the mail message header.
00451      */
00452     string msg_header_raw_text_;
00453
00454     /** The mail message header SUBJECT field.
00455      *
00456      *  This field contains the text of the subject line of the mail message. It is used to
00457      *  validate the mail message by ensuring that only messages with the proper subject line are
00458      *  consumed by the application. In addition, a valid mail message contains the IMEI number
00459      *  in the subject line. The subject line is parsed to determine the IMEI of the instrument
00460      *  that transmitted the message.
00461      */
00462     string msg_header_subject_;
00463
00464     /** The mail message is valid flag.
00465      *
00466      *  This flag indicates the validity of the mail message. It is set as a result of parsing the
00467      *  mail message. The criteria for a valid mail message is that
00468      *      - the mail message header is valid (see @ref msg_header_is_valid_),
00469      *      - the mail message body is valid (see @ref msg_body_is_valid_), and
00470      *      - the mail message SBD file attachment is valid (see @ref sbd_data_is_valid_).
00471      */
00472     bool msg_is_valid_;
00473
00474     /** The full mail message raw text.
00475      *
00476      *  This field contains the full mail message text in its raw form. This represents the native
00477      *  ASCII format of the mail message.
00478      */
00479     string msg_raw_text_;
00480
00481     /** The path to the raw mail message file on the local filesystem.
00482      *
00483      *  This path is set as a result of writing the raw mail message file to the local filesystem.
00484      *  This path is relative to the raw file directory, which is specified by @ref
00485      *  raw_directory_path_.
00486      *
00487      */
00488     path raw_file_local_path_;
00489
00490     /** The Iridium Short Burst Data (SBD) binary data.
00491      *
00492      *  This field contains the binary data that is read from the SBD file attachment. The content
00493      *  of this data is exactly that which was transmitted by the instrument.
00494      */
00495     XMailSbdData_t sbd_data_;
00496
00497     /** The Iridium Short Burst Data (SBD) binary data is valid flag.
00498      *
00499      *  This field indicates whether the attached SBD message is valid. The criteria for a valid
00500      *  SBD message is that
00501      *      - the mail message has a file attachment,
00502      *      - the filename of the attached file matches the SBD file email attachment filename
00503      *        regular expression, @ref sbd_file_valid_name_regex_,
00504      *      - the Iridium session status code, @ref sbd_session_status_code_, has a value of
00505      *        zero, and
00506      *      - the SBD message size, in bytes, is greater than or equal to @ref XMSG_SBD_MIN_BYTES.
00507      */
00508     bool sbd_data_is_valid_;
00509
00510     /** The path to the Iridium Short Burst Data (SBD) binary file on the local filesystem.
00511      *
00512      *  This path is set as a result of writing the binary SBD message file to the local
00513      *  filesystem. This path is relative to the SBD file directory, which is specified by @ref
00514      *  sbd_directory_path_.
00515      */
00516     path sbd_file_local_path_;
00517
00518     /** The Iridium Short Burst Data (SBD) email attachment filename.
00519      *
00520      *  This field represents the name of the file attached to the mail message. This field is
00521      *  used, in part, to determined the validity of the SBD message attachment.
00522      */
00523     string sbd_file_name_;
00524
00525     /** The size of the Iridium Short Burst Data (SBD) message, in bytes.
00526      *
00527      *  This field is parsed from the body of the mail message. It should match the size of the
00528      *  SBD data, @ref sbd_data_, which is read from the mail message attachment.
00529      */
00530     int sbd_file_size_bytes_;
00531
00532     /** The Iridium Short Burst Data (SBD) session status code.
00533      *
00534      *  This field is parsed from the body of the mail message. It indicates that status of the
```

```
00535      *  Iridium session. A successful Iridium session reports a status code of 0. An unsuccessful
00536      *  Iridium session reports a non-zero status code, where the exact value of the status code
00537      *  indicates a specific error or type of error in the Iridium session.
00538      */
00539     int sbd_session_status_code_;
00540
00541     /** The Iridium Short Burst Data (SBD) session status string.
00542      *
00543      *  This field is parsed from the body of the mail message. It contains a brief description
00544      *  of the status of the Iridium session that is indicated by the status code, @ref
00545      *  sbd_session_status_code_.
00546      */
00547     string sbd_session_status_string_;
00548
00549     shared_ptr<vector<uint8_t> > xmsg_id_types_;
00550
00551
00552   private:
00553
00554     /* Private Member Functions: */
00555
00556     /** Parse a mail message.
00557      *
00558      *  This function is used to parse and validate a mail message. As a result of parsing the
00559      *  mail message, the fields of this class are populated.
00560      *
00561      *  @exception      log::util::MessageException if @c msg is NULL or an error occurs while
00562      *                  parsing the various parts of the mail message.
00563      *
00564      *  @param[in]      msg     The mail message to parse.
00565      */
00566     void ParseMailMsg(const vmime::shared_ptr<vmime::net::message>& msg) {
00567
00568       try {
00569
00570         // Get an exclusive lock.
00571         lock_guard<recursive_mutex> guard(mutex_);
00572
00573         if(!msg)
00574           EXC_THROW("Message pointer is NULL!");
00575
00576         // Get the message RAW text.
00577         vmime::utility::outputStreamStringAdapter msg_raw_text_ostream(msg_raw_text_);
00578         msg->extract(msg_raw_text_ostream);
00579
00580         // Parse the mail message.
00581         istringstream(msg->getUID()) » id_msg_uid_;
00582         vmime::shared_ptr<vmime::message> parsed_msg(msg->getParsedMessage());
00583         ParseMailMsgHeader(msg->getHeader());
00584         ParseMailMsgBody(parsed_msg);
00585         ParseMailMsgSbdAttachment(parsed_msg);
00586
00587         // Set the validity of the mail message.
00588         set_msg_is_valid(msg_header_is_valid() && msg_body_is_valid() && sbd_data_is_valid());
00589
00590       }CATCH_BLOCK("Failed to parse mail message!")
00591     }
00592
00593     /** Parse a mail message.
00594      *
00595      *  This function is used to parse and validate a mail message. As a result of parsing the
00596      *  mail message, the fields of this class are populated.
00597      *
00598      *  @exception      log::util::MessageException if @c msg is NULL or an error occurs while
00599      *                  parsing the various parts of the mail message.
00600      *
00601      *  @param[in]      msg     The mail message to parse.
00602      */
00603     void ParseMailMsg(const vmime::shared_ptr<vmime::message>& msg) {
00604
00605       try {
00606
00607         // Get an exclusive lock.
00608         lock_guard<recursive_mutex> guard(mutex_);
00609
00610         if(!msg)
00611           EXC_THROW("Message pointer is NULL!");
00612
00613         // Get the message RAW text.
00614         vmime::utility::outputStreamStringAdapter msg_raw_text_ostream(msg_raw_text_);
00615         msg->generate(msg_raw_text_ostream);
00616
00617         // Parse the mail message.
00618         //istringstream(msg->getUID()) » id_msg_uid_;
00619         ParseMailMsgHeader(msg->getHeader());
00620         ParseMailMsgBody(msg);
00621         ParseMailMsgSbdAttachment(msg);
00622
00623         // Set the validity of the mail message.
00624         set_msg_is_valid(msg_header_is_valid() && msg_body_is_valid() && sbd_data_is_valid());
00625
00626       }CATCH_BLOCK("Failed to parse mail message!")
00627     }
00628
00629     /** Parse the mail message header.
00630      *
00631      *  This function is used to parse and validate the mail message header. As a result of
00632      *  parsing the mail message, the field of this class relating to the mail message header
00633      *  are populated.
00634      *
00635      *  @exception      log::util::MessageException if @c head is NULL or an error occurs while
00636      *                  parsing the header of the mail message.
00637      *
00638      *  @param[in]      head        The mail message header to parse.
00639      */
00640     void ParseMailMsgHeader(const vmime::shared_ptr<const vmime::header>& head) {
00641       static const vmime::charset cs = vmime::charset(vmime::charsets::UTF_8);
00642
```

```
00643       smatch sm;
00644
00645       try {
00646
00647         // Get an exclusive lock.
00648         lock_guard<recursive_mutex> guard(mutex_);
00649
00650         if(!head)
00651           EXC_THROW("Message header pointer is NULL!");
00652
00653         // Get the complete message header text.
00654         vmime::utility::outputStreamStringAdapter msg_header_text_stream(msg_header_raw_text_);
00655         head->generate(msg_header_text_stream);
00656
00657         // Get "FROM" field.
00658         set_msg_header_from(head->From()->getValue<vmime::mailbox>()->getEmail().toString());
00659
00660         // Get "DATE" field, converted to UTC (the zone is the offset from GMT, in minutes).
00661         vmime::shared_ptr<const vmime::datetime> vdt(head->Date()->getValue<vmime::datetime>());
00662         set_msg_header_datetime(
00663             ptime(
00664                 date(vdt->getYear(), vdt->getMonth(), vdt->getDay()),
00665                 hours(vdt->getHour()) + minutes(vdt->getMinute() - vdt->getZone())
00666                 + seconds(vdt->getSecond())));
00667
00668         try {
00669           // Get "SUBJECT" field.
00670           set_msg_header_subject(
00671               head->Subject()->getValue<vmime::text>()->getConvertedText(cs));
00672           if(regex_search(msg_header_subject(), sm, regex(msg_header_valid_subject_regex_))) {
00673             long imei;
00674             istringstream(sm[1]) » imei;
00675             set_id_imei(imei);
00676           }
00677         }CATCH_BLOCK("Failed to parse SUBJECT field!")
00678
00679         // Get "MESSAGE-ID" field.
00680         set_id_msg(head->MessageId()->getValue<vmime::messageId>()->getId());
00681
00682         try {
00683           // Validate FROM and SUBJECT fields.
00684           if(!regex_match(
00685               msg_header_from(),
00686               regex(msg_header_valid_sender_regex_, regex::extended)))
00687             set_msg_header_is_valid(false);
00688           else if(!regex_match(
00689               msg_header_subject(),
00690               regex(msg_header_valid_subject_regex_, regex::extended)))
00691             set_msg_header_is_valid(false);
00692           else
00693             set_msg_header_is_valid(true);
00694
00695         }CATCH_BLOCK("Failed to validate FROM and SUBJECT fields!")
00696
00697       }CATCH_BLOCK("Failed to parse mail message header!")
00698     }
00699
00700     /** Parse the mail message body.
00701      *
00702      *  This function is used to parse and validate the mail message body. The body of a valid
00703      *  mail message includes information about the Iridium session which are parsed in order to
00704      *  populate the corresponding fields of this class.
00705      *
00706      *  @exception      log::util::MessageException if @c msg is NULL or an error occurs while
00707      *                  parsing the body of the mail message.
00708      *
00709      *  @param[in]      msg         The message to parse
00710      */
00711     void ParseMailMsgBody(const vmime::shared_ptr<vmime::message>& msg) {
00712       smatch sm;
00713
00714       try {
00715
00716         // Get an exclusive lock.
00717         lock_guard<recursive_mutex> guard(mutex_);
00718
00719         if(!msg)
00720           EXC_THROW("Message pointer is NULL!");
00721
00722         // Get body text.
00723         vmime::messageParser mp(msg);
00724         vmime::utility::outputStreamStringAdapter body_text_stream(msg_body_text_);
00725         vmime::shared_ptr<const vmime::body> body(msg->getBody());
00726         for(size_t i = 0; i < mp.getTextPartCount(); ++i)
00727           vmime::shared_ptr<const vmime::textPart>(mp.getTextPartAt(i))->getText()->extract(
00728               body_text_stream);
00729         regex_replace(msg_body_text_, regex("\\n[^a-zA-Z0-9]*\\n"), string("\n"));
00730         trim(msg_body_text_);
00731
00732         // Parse MOMSN.
00733         if(regex_search(msg_body_text_, sm, regex("MOMSN: (\\d+)")))
00734           istringstream(sm[1]) » id_sbd_momsn_;
00735
00736         // Parse MTMSN.
00737         if(regex_search(msg_body_text_, sm, regex("MTMSN: (\\d+)")))
00738           istringstream(sm[1]) » id_sbd_mtmsn_;
00739
00740         // Parse Time of Session.
00741         if(regex_search(
00742             msg_body_text_,
00743             sm,
00744             regex("Time of Session \\(UTC\\): ([a-zA-Z]{3} ([a-zA-Z]{3})[ ]+(\\d{1,2}) "
00745                 "(\\d{2}:\\d{2}:\\d{2}) (\\d{4})")")) {
00746           ptime time;
00747           stringstream ss;
00748           ss « sm[5] « "-" « sm[2] « "-" « setw(2) « setfill('0') « sm[3] « " " « sm[4];
00749           ss » time;
00750           set_iridium_position_datetime(time);
```

```
00751         }
00752
00753         // Parse session status.
00754         if(regex_search(msg_body_text_, sm, regex("Session Status: (\\d{2}) - ([^\n]+)"))) {
00755           istringstream(sm[1]) » sbd_session_status_code_;
00756           set_sbd_session_status_string(sm[2]);
00757         }
00758
00759         // Parse SBD message size (bytes);
00760         if(regex_search(msg_body_text_, sm, regex("Message Size \\(bytes\\): (\\d+)")))
00761           istringstream(sm[1]) » sbd_file_size_bytes_;
00762
00763         // Parse Iridium location.
00764         if(regex_search(
00765             msg_body_text_,
00766             sm,
00767             regex("Unit Location: Lat = (-?\\d+\\.\\d+) Long = (-?\\d+\\.\\d+)"))) {
00768           double lat, lon;
00769           istringstream(sm[1]) » lat;
00770           istringstream(sm[2]) » lon;
00771           set_iridium_position_latitude(lat);
00772           set_iridium_position_longitude(lon);
00773         }
00774         if(regex_search(msg_body_text_, sm, regex("CEPradius = (\\d+)"))) {
00775           int cep;
00776           istringstream(sm[1]) » cep;
00777           set_iridium_position_cep_radius(cep);
00778         }
00779
00780         // Validate message body.
00781         set_msg_body_is_valid(
00782             id_sbd_momsn_ != XMSG_ID_UNKNOWN && id_sbd_mtmsn_ != XMSG_ID_UNKNOWN
00783             && !iridium_position_datetime().is_not_a_date_time());
00784       }CATCH_BLOCK("Failed to parse mail message body!")
00785     }
00786
00787     /** Parse the mail message SBD attachment.
00788      *
00789      *  This function is used to parse and validate the attachment of the mail message. A valid
00790      *  mail message contains a binary SBD file attachment that represents the data transmitted
00791      *  by the instrument.
00792      *
00793      *  @exception      log::util::MessageException if @c msg is NULL or an error occurs while
00794      *                  parsing the attachment of the mail message.
00795      *
00796      *  @param[in]      msg         The mail message to parse.
00797      */
00798     void ParseMailMsgSbdAttachment(const vmime::shared_ptr<vmime::message>& msg) {
00799
00800       try {
00801
00802         // Get an exclusive lock.
00803         lock_guard<recursive_mutex> guard(mutex_);
00804
00805         if(!msg)
00806           EXC_THROW("Message pointer is NULL!");
00807
00808         // Parse the attachment(s).
00809         vmime::messageParser mp(msg);
00810         for(size_t i = 0; i < mp.getAttachmentCount(); ++i) {
00811           vmime::shared_ptr<const vmime::attachment> attach = mp.getAttachmentAt(i);
00812
00813           // Check attachment type.
00814           vmime::mediaType type = attach->getType();
00815           if(type.getType() != vmime::mediaTypes::APPLICATION)
00816             continue;
00817
00818           // Check attachment name.
00819           if(!regex_search(
00820               sbd_file_name_ = attach->getName().getBuffer(),
00821               regex(sbd_file_valid_name_regex_)))
00822             continue;
00823
00824           // Get bytes.
00825           vmime::utility::outputStreamByteArrayAdapter ostream_adapter(sbd_data_);
00826           attach->getData()->extract(ostream_adapter);
00827
00828           // Validate.
00829           set_sbd_data_is_valid(sbd_data_.size() >= XMSG_SBD_MIN_BYTES
00830               && (sbd_session_status_code() == 0 || sbd_session_status_code() == 2));
00831           if(sbd_data_is_valid()) {
00832
00833             // Parse dive and instrument identifier.
00834             set_id_instrument(u16(3, sbd_data_.data()));
00835             id_dive_ = i16(5, sbd_data_.data());
00836             id_sbd_packet_ = u08(7, sbd_data_.data());
00837           }
00838           return;
00839         }
00840
00841       }CATCH_BLOCK("Failed to parse mail message attachment!")
00842     }
00843
00844     /** Select all mail message objects from the SQL database table.
00845      *
00846      *  @param[in]  groupBy     The SQL select statement "group-by expression" clause.
00847      *  @param[in]  orderBy     The SQL select statement "order-by expression" clause.
00848      *  @param[in]  limit       The SQL select statement result set row limit value (0 = no limit).
00849      *  @param[in]  offset      The SQL select statement result set row offset value.
00850      *
00851      *  @throw      util::log::MessageException if the query fails.
00852      *
00853      *  @return     The mail message objects.
00854      */
00855     static vector<shared_ptr<XFloatMailMessage> > SelectAllObjectsFromDatabase(
00856         const string& groupBy = "",
00857         const string& orderBy = "",
00858         const unsigned int& limit = 0,
```

```
00859         const unsigned int& offset = 0) {
00860
00861       try {
00862
00863         return sql_table_description_.SelectAllObjectsFromDatabase(
00864             groupBy,
00865             orderBy,
00866             limit,
00867             offset);
00868
00869       }CATCH_BLOCK("Failed to query instrument objects from the database!")
00870
00871       return vector<shared_ptr<XFloatMailMessage> >();
00872     }
00873
00874     /** Select a single mail message object from the SQL database table.
00875      *
00876      *  @param[in]  where       The SQL select statement "where-condition" clause.
00877      *  @param[in]  groupBy     The SQL select statement "group-by expression" clause.
00878      *  @param[in]  orderBy     The SQL select statement "order-by expression" clause.
00879      *  @param[in]  offset      The SQL select statement result set row offset value.
00880      *
00881      *  @throw      util::log::MessageException if the query fails.
00882      *
00883      *  @return     The first mail message object matching the query.
00884      */
00885     static shared_ptr<XFloatMailMessage> SelectObjectFromDatabase(
00886         const string& where = "",
00887         const string& groupBy = "",
00888         const string& orderBy = "",
00889         const unsigned int& offset = 0) {
00890
00891       try {
00892
00893         return sql_table_description_.SelectObjectFromDatabase(where, groupBy, orderBy, offset);
00894
00895       }CATCH_BLOCK("Failed to query XFloat objects from the database!")
00896
00897       return make_shared<XFloatMailMessage>();
00898     }
00899
00900     /** Query mail message objects from the SQL database.
00901      *
00902      *  This function is used to query the SQL database table corresponding to the mail message
00903      *  class and construct a vector of mail message objects, populated from the results of the
00904      *  SQL query.
00905      *
00906      *  @exception  util::log::MessageException if an error occurs while querying the database
00907      *              or constructing the float objects from the result of the query.
00908      *
00909      *  @param[in]  where       The SQL select statement "where-condition" clause.
00910      *  @param[in]  groupBy     The SQL select statement "group-by expression" clause.
00911      *  @param[in]  orderBy     The SQL select statement "order-by expression" clause.
00912      *  @param[in]  limit       The SQL select statement result set row limit value.
00913      *  @param[in]  offset      The SQL select statement result set row offset value.
00914      *
00915      *  @return     A vector of shared references to the float objects constructed from the
00916      *              results of the SQL query.
00917      */
00918     static vector<shared_ptr<XFloatMailMessage> > SelectObjectsFromDatabase(
00919         const string& where = "",
00920         const string& groupBy = "",
00921         const string& orderBy = "",
00922         const unsigned int& limit = 0,
00923         const unsigned int& offset = 0) {
00924
00925       try {
00926
00927         return sql_table_description_.SelectObjectsFromDatabase(
00928             where,
00929             groupBy,
00930             orderBy,
00931             limit,
00932             offset);
00933
00934       }CATCH_BLOCK("Failed to query all X-Float objects from the database!")
00935
00936       return vector<shared_ptr<XFloatMailMessage> >();
00937     }
00938
00939     /** Select a value from the SQL database table that contains the persisted mail message
00940      *  objects.
00941      *
00942      *  This function may be used to select a singular value from the table that comprises the
00943      *  persisted mail message objects.
00944      *
00945      *  @exception  util::log::MessageException if the SQL database query fails or if an error
00946      *              occurs while attempting to interpret the result of the query.
00947      *
00948      *  @note       The result set of this query is always limited to a single result. That is,
00949      *              the result set row limit value is automatically set to one. If more than one
00950      *              row in the database table satisfy the "where condition" clause of the select
00951      *              statement, then only the value of the first result will be returned.
00952      *
00953      *  @param[in]  what        The SQL select statement "what" clause. This clause may only
00954      *                          contain a singular result e.g. the name of the SQL database table
00955      *                          column of interest.
00956      *  @param[in]  where       The SQL select statement "where condition" clause.
00957      *  @param[in]  groupBy     The SQL select statement "group-by expression" clause.
00958      *  @param[in]  orderBy     The SQL select statement "order-by expression" clause.
00959      *  @param[in]  offset      The SQL select statement result set row offset value.
00960      *
00961      *  @return     The selected value from the SQL database table.
00962      */
00963     template<typename TT> static TT SelectValueFromDatabase(
00964         const string& what,
00965         const string& where = "",
00966         const string& groupBy = "",
```

```
00967         const string& orderBy = "",
00968         const unsigned int& offset = 0) {
00969
00970       return sql_table_description_.SelectValueFromDatabase<TT>(
00971           what,
00972           where,
00973           groupBy,
00974           orderBy,
00975           offset);
00976
00977     }
00978
00979     /** Set the dive or cycle number of this mail message.
00980      *
00981      *  @param[in]  id_dive     The dive or cycle number of this mail message.
00982      */
00983     void set_id_dive(int id_dive) {
00984       // Get an exclusive lock.
00985       lock_guard<recursive_mutex> guard(mutex_);
00986
00987       id_dive_ = id_dive;
00988     }
00989
00990     /** Set the mail message unique identifier string.
00991      *
00992      *  @param[in]  id_msg      The mail message unique identifier string.
00993      */
00994     void set_id_msg(const string& id_msg) {
00995       // Get an exclusive lock.
00996       lock_guard<recursive_mutex> guard(mutex_);
00997
00998       id_msg_ = id_msg;
00999     }
01000
01001     /** Set the mail message unique identifier
01002      *
01003      *  @param[in]  id_msg_uid  The mail message unique identifier.
01004      */
01005     void set_id_msg_uid(long id_msg_uid) {
01006       // Get an exclusive lock.
01007       lock_guard<recursive_mutex> guard(mutex_);
01008
01009       id_msg_uid_ = id_msg_uid;
01010     }
01011
01012     /** Set the Iridium Short Burst Data (SBD) Service Mobile Originated Message Sequence Number
01013      *  (MOMSN).
01014      *
01015      *  @param[in]  momsn       The Iridium Short Burst Data (SBD) Service Mobile Originated
01016      *                          Message Sequence Number (MOMSN).
01017      */
01018     void set_id_sbd_momsn(int momsn) {
01019       // Get an exclusive lock.
01020       lock_guard<recursive_mutex> guard(mutex_);
01021
01022       id_sbd_momsn_ = momsn;
01023     }
01024
01025     /** Set the Iridium Short Burst Data (SBD) Service Mobile Terminated Message Sequence Number
01026      *  (MTMSN).
01027      *
01028      *  @param[in]  mtmsn       The Iridium Short Burst Data (SBD) Service Mobile Terminated
01029      *                          Message Sequence Number (MTMSN).
01030      */
01031     void set_id_sbd_mtmsn(int mtmsn) {
01032       // Get an exclusive lock.
01033       lock_guard<recursive_mutex> guard(mutex_);
01034
01035       id_sbd_mtmsn_ = mtmsn;
01036     }
01037
01038     /** Set the Iridium Short Burst Data (SBD) message packet index.
01039      *
01040      *  @param[in]  id_sbd_packet   Iridium Short Burst Data (SBD) message packet index.
01041      */
01042     void set_id_sbd_packet(int id_sbd_packet) {
01043       // Get an exclusive lock.
01044       lock_guard<recursive_mutex> guard(mutex_);
01045
01046       id_sbd_packet_ = id_sbd_packet;
01047     }
01048
01049     /** Set the message-body-is-valid flag.
01050      *
01051      *  @param[in]  is_valid    TRUE if the body of the Iridium message is valid, or FALSE
01052      *                          otherwise.
01053      */
01054     void set_msg_body_is_valid(bool is_valid) {
01055       // Get an exclusive lock.
01056       lock_guard<recursive_mutex> guard(mutex_);
01057
01058       msg_body_is_valid_ = is_valid;
01059     }
01060
01061     /** Set the mail message body text.
01062      *
01063      *  @param[in]  msg_body_text   The mail message body text.
01064      */
01065     void set_msg_body_text(const string& msg_body_text) {
01066       // Get an exclusive lock.
01067       lock_guard<recursive_mutex> guard(mutex_);
01068
01069       msg_body_text_ = msg_body_text;
01070     }
01071
01072     /** Set the mail message header date/time.
01073      *
01074      *  @param[in]  datetime        The mail message header date/time.
```

```
01075       */
01076      void set_msg_header_datetime(const ptime& datetime) {
01077        // Get an exclusive lock.
01078        lock_guard<recursive_mutex> guard(mutex_);
01079
01080        msg_header_datetime_ = datetime;
01081      }
01082
01083      /** Set the mail message header FROM field value.
01084       *
01085       *  @param[in]  from        The mail message header FROM field value.
01086       */
01087      void set_msg_header_from(const string& from) {
01088        // Get an exclusive lock.
01089        lock_guard<recursive_mutex> guard(mutex_);
01090
01091        msg_header_from_ = from;
01092      }
01093
01094      /** Set the mail-message-header-is-valid flag.
01095       *
01096       *  @param[in]  is_valid    TRUE if the mail message header is valid, or FALSE otherwise.
01097       */
01098      void set_msg_header_is_valid(bool is_valid) {
01099        // Get an exclusive lock.
01100        lock_guard<recursive_mutex> guard(mutex_);
01101
01102        msg_header_is_valid_ = is_valid;
01103      }
01104
01105      /** Set the mail message header SUBJECT field value.
01106       *
01107       *  @param[in]  subject     The mail message header SUBJECT field value.
01108       */
01109      void set_msg_header_subject(const string& subject) {
01110        // Get an exclusive lock.
01111        lock_guard<recursive_mutex> guard(mutex_);
01112
01113        msg_header_subject_ = subject;
01114      }
01115
01116      /** Set the mail message header raw text.
01117       *
01118       *  @param[in]  text        The mail message header raw text.
01119       */
01120      void set_msg_header_raw_text(const string& text) {
01121        // Get an exclusive lock.
01122        lock_guard<recursive_mutex> guard(mutex_);
01123
01124        msg_header_raw_text_ = text;
01125      }
01126
01127      /** Set the mail-message-is-valid flag.
01128       *
01129       *  @param[in]  is_valid    TRUE if the mail message is valid, or FALSE otherwise.
01130       */
01131      void set_msg_is_valid(bool is_valid) {
01132        // Get an exclusive lock.
01133        lock_guard<recursive_mutex> guard(mutex_);
01134
01135        msg_is_valid_ = is_valid;
01136      }
01137
01138      /** Set the full mail message raw text.
01139       *
01140       *  @param[in]  text        The full mail message raw text.
01141       */
01142      void set_msg_raw_text(const string& text) {
01143
01144        // Get an exclusive lock.
01145        lock_guard<recursive_mutex> guard(mutex_);
01146
01147        msg_raw_text_ = text;
01148      }
01149
01150      /** Set the path to the raw mail message file on the local file system.
01151       *
01152       *  @param[in]  path        The path to the raw mail message file on the local file system.
01153       */
01154      void set_raw_file_local_path(const path& path) {
01155        // Get an exclusive lock.
01156        lock_guard<recursive_mutex> guard(mutex_);
01157
01158        raw_file_local_path_ = path;
01159      }
01160
01161      /** Set the Iridium Short Burst Data (SBD) for this mail message.
01162       *
01163       *  @param[in]  sbd_data    The Iridium Short Burst Data (SBD) for this mail message.
01164       */
01165      void set_sbd_data(const vector<uint8_t>& sbd_data) {
01166        // Get an exclusive lock.
01167        lock_guard<recursive_mutex> guard(mutex_);
01168
01169        sbd_data_ = sbd_data;
01170      }
01171
01172      /** Set the SBD-message-is-valid flag.
01173       *
01174       *  @param[in]  is_valid    TRUE if the Iridium Short Burst Data (SBD) message is valid, or
01175       *                          FALSE otherwise.
01176       */
01177      void set_sbd_data_is_valid(bool is_valid) {
01178        // Get an exclusive lock.
01179        lock_guard<recursive_mutex> guard(mutex_);
01180
01181        sbd_data_is_valid_ = is_valid;
01182      }
```

```
01183
01184     /** Set the path to the Iridium Short Burst Data (SBD) message file on the local file system.
01185      *
01186      *  @param[in]  path        The path to the Iridium Short Burst Data (SBD) message file on the
01187      *                          local file system.
01188      */
01189     void set_sbd_file_local_path(const path& path) {
01190       // Get an exclusive lock.
01191       lock_guard<recursive_mutex> guard(mutex_);
01192
01193       sbd_file_local_path_ = path;
01194     }
01195
01196     /** Set the name of the Iridium Short Burst Data (SBD) file.
01197      *
01198      *  @param[in]  name        The name of the Iridium Short Burst Data (SBD) file.
01199      */
01200     void set_sbd_file_name(const string& name) {
01201       // Get an exclusive lock.
01202       lock_guard<recursive_mutex> guard(mutex_);
01203
01204       sbd_file_name_ = name;
01205     }
01206
01207     /** Set the size of the Iridium Short Burst Data (SBD) message, in bytes.
01208      *
01209      *  @param[in]  size        The size of the Iridium Short Burst Data (SBD) message, in bytes.
01210      */
01211     void set_sbd_file_size_bytes(int size) {
01212       // Get an exclusive lock.
01213       lock_guard<recursive_mutex> guard(mutex_);
01214
01215       sbd_file_size_bytes_ = size;
01216     }
01217
01218     /** Set the Iridium Short Burst Data (SBD) session status code.
01219      *
01220      *  @param[in]  code       The Iridium Short Burst Data (SBD) session status code.
01221      */
01222     void set_sbd_session_status_code(int code) {
01223       // Get an exclusive lock.
01224       lock_guard<recursive_mutex> guard(mutex_);
01225
01226       sbd_session_status_code_ = code;
01227     }
01228
01229     /** Set the Iridium Short Burst Data (SBD) session status string.
01230      *
01231      *  @param[in]  str         The Iridium Short Burst Data (SBD) session status string.
01232      */
01233     void set_sbd_session_status_string(const string& str) {
01234       // Get an exclusive lock.
01235       lock_guard<recursive_mutex> guard(mutex_);
01236
01237       sbd_session_status_string_ = str;
01238     }
01239
01240     void set_xmsg_id_types(const shared_ptr<vector<uint8_t> >& xmsg_id_types) {
01241
01242       // Get an exclusive lock.
01243       lock_guard<recursive_mutex> guard(mutex_);
01244
01245       xmsg_id_types_ = xmsg_id_types;
01246     }
01247
01248     /** Write the mail message files.
01249      *
01250      *  This function is used to write the RAW and SBD files to the local filesystem. First, the
01251      *  function checks if the unique instrument identifier is known. If the unique instrument
01252      *  identifier is unknown, the function will throw an exception. Otherwise, the function will
01253      *  continue on to write the RAW mail message file and the binary SBD message file to the
01254      *  local filesystem, if their respective criteria are met.
01255      *
01256      *  @exception  util::log::MessageException if the unique instrument identifier is unknown, or
01257      *              if an error occurs while writing the mail files to the local filesystem,
01258      */
01259     void WriteFiles() {
01260
01261       try {
01262
01263
01264         // Check the WHOI ID.
01265         if(id_instrument() == XFLT_ID_UNKNOWN)
01266           EXC_THROW("The WHOI identifier is unknown!");
01267
01268         // Write mail files.
01269         WriteRawFile();
01270         WriteSbdFile();
01271
01272       }CATCH_BLOCK("Failed to write mail message files!")
01273     }
01274
01275     void WriteRawFile();
01276
01277     /** Write the binary Iridium Short Burst Data (SBD) file.
01278      *
01279      *  This function is used to write the binary SBD file to the local filesystem. If either the
01280      *  global SBD-file-generation-is-enabled flag, @ref sbd_file_generation_is_enabled_, or the
01281      *  SBD-data-is-valid field, @ref sbd_data_is_valid_, is FALSE, then this function will have
01282      *  no effect.
01283      *
01284      *  Otherwise, if the SBD data is valid and global SBD-file-generation-is-enabled flag is TRUE,
01285      *  then the SBD file path will be generated. If the SBD file already exists at the
01286      *  constructed path, then it will not be overwritten. If the file does not already exist, it
01287      *  will be written.
01288      *
01289      *  The path to the binary SBD message file will be constructed in the format
01290      *  [SBD-DIR]/[WWWW]/[WWWW]-[MMMMMM]-[DDDD].sbd, where
```

```
01291      *      - [SBD-DIR] is the SBD mail directory path specified, by @ref sbd_directory_path_,
01292      *      - [WWWW] is the four-digit unique instrument serial number,
01293      *      - [MMMMMM] is the six-digit MOMSN number, and
01294      *      - [DDDD] is the four-digit dive or cycle number.
01295      *
01296      *  @exception  util::log::MessageException if an error occurs while writing the binary SBD
01297      *              file to the local filesystem.
01298      */
01299     void WriteSbdFile() {
01300
01301       try {
01302
01303         // Get an exclusive lock.
01304         lock_guard<recursive_mutex> guard(mutex_);
01305
01306         if(!sbd_file_generation_is_enabled_ || !sbd_data_is_valid())
01307           return;
01308
01309         {   // Get mail directory path.
01310           ostringstream os;
01311           os « format("%s/%04d") % sbd_directory_path_.string() % id_instrument();
01312           GetDirectoryPathFromString(os.str(), true);
01313         }
01314
01315         {   // Set SBD local path.
01316           ostringstream os;
01317           os « format("%04d/%04d-%06d-%04d.sbd")   //
01318                     % id_instrument() % id_instrument() % id_sbd_momsn() % id_dive();
01319           set_sbd_file_local_path(os.str());
01320         }
01321
01322         // Write SBD file.
01323         path sbd_file_path(sbd_directory_path_.string() + "/" + sbd_file_local_path().string());
01324         if(!bfs::exists(sbd_file_path)) {
01325           linf() « "Writing SBD file: " « sbd_file_path « "...";
01326           ofstream sbd_file_stream(sbd_file_path);
01327           sbd_file_stream.write((char*) sbd_data().data(), sbd_data().size());
01328           sbd_file_stream.flush();
01329           sbd_file_stream.close();
01330         }
01331       }CATCH_BLOCK("Failed to write SBD file!")
01332     }
01333
01334   public:
01335
01336     /* Public Member Functions: */
01337
01338     /** The default constructor.
01339      *
01340      */
01341     XFloatMailMessage() :
01342       Iridium(),
01343       IridiumPosition(),
01344       id_dive_(XFLT_DIVE_UNKNOWN),
01345       id_msg_(),
01346       id_msg_uid_(XFLT_ID_UNKNOWN),
01347       id_sbd_momsn_(XFLT_ID_UNKNOWN),
01348       id_sbd_mtmsn_(XFLT_ID_UNKNOWN),
01349       id_sbd_packet_(XFLT_ID_UNKNOWN),
01350       msg_body_is_valid_(false),
01351       msg_body_text_(),
01352       msg_header_datetime_(not_a_date_time),
01353       msg_header_from_(),
01354       msg_header_is_valid_(false),
01355       msg_header_subject_(),
01356       msg_is_valid_(false),
01357       raw_file_local_path_(),
01358       sbd_data_(),
01359       sbd_data_is_valid_(false),
01360       sbd_file_local_path_(),
01361       sbd_file_name_(),
01362       sbd_file_size_bytes_(XFLT_SBD_SIZE_BYTES_UNKNOWN),
01363       sbd_session_status_code_(XFLT_SBD_SESSION_STATUS_CODE_UNKNOWN),
01364       sbd_session_status_string_() {
01365
01366       sbd_data_.reserve(340);
01367
01368     }
01369
01370     /** Construct from a VMime mail message.
01371      *
01372      *  @param[in]      msg         The mail message.
01373      */
01374     XFloatMailMessage(const vmime::shared_ptr<vmime::net::message>& msg) :
01375       Iridium(),
01376       IridiumPosition(),
01377       id_dive_(XFLT_DIVE_UNKNOWN),
01378       id_msg_(),
01379       id_msg_uid_(XFLT_ID_UNKNOWN),
01380       id_sbd_momsn_(XFLT_ID_UNKNOWN),
01381       id_sbd_mtmsn_(XFLT_ID_UNKNOWN),
01382       id_sbd_packet_(XFLT_ID_UNKNOWN),
01383       msg_body_is_valid_(false),
01384       msg_body_text_(),
01385       msg_header_datetime_(not_a_date_time),
01386       msg_header_from_(),
01387       msg_header_is_valid_(false),
01388       msg_header_subject_(),
01389       msg_is_valid_(false),
01390       raw_file_local_path_(),
01391       sbd_data_(),
01392       sbd_data_is_valid_(false),
01393       sbd_file_local_path_(),
01394       sbd_file_name_(),
01395       sbd_file_size_bytes_(XFLT_SBD_SIZE_BYTES_UNKNOWN),
01396       sbd_session_status_code_(XFLT_SBD_SESSION_STATUS_CODE_UNKNOWN),
01397       sbd_session_status_string_() {
01398
```

```
01399       sbd_data_.reserve(340);
01400
01401       try {
01402
01403         ParseMailMsg(msg);
01404
01405       }CATCH_BLOCK("Failed to construct mail message object!")
01406     }
01407
01408     /** The destructor.
01409      *
01410      */
01411     ~XFloatMailMessage() {
01412       /* Do nothing. */
01413     }
01414
01415     /** Get the mail message listing file comment string.
01416      *
01417      *  @return         The mail message listing file comment string.
01418      */
01419     static string GenerateListingComment() {
01420
01421       using namespace std;
01422       using boost::format;
01423
01424       ostringstream os;
01425
01426       try {
01427
01428         //            static const format fmt1("#   %5s %4s %4s %15s %5d %5d %3d %20s %19s %4d");
01429         //            static const format fmt2("#    %5s %4s %4s %15s %5d %5d %3d %20s %19s %4d");
01430         //
01431         //            os « format(fmt1) % "SN" % "dive" % "size" % "IMEI" % "MOMSN" % "MTMSN" % "PID"
01432         //                       % "date/time" % "position" % "CEP"
01433         //               « endl;
01434         //            os « format(fmt2) % "" % "" % "(b)" % "" % "" % "" % "" % "" % "" % "" % "(km)";
01435         os « "#      SN dive size            IMEI MOMSN MTMSN PID            date/time            position
    CEP"
01436            « endl
01437            « "#               (b)
    (km)";
01438
01439       }CATCH_BLOCK("Failed to generate mail message listing comment!")
01440
01441       return os.str();
01442     }
01443
01444     /** Generate the listing file entry string for this mail message.
01445      *
01446      *  @return         The listing file entry string for this mail message.
01447      */
01448     const string GenerateListingString() const {
01449       ostringstream os;
01450
01451       try {
01452
01453         const format fmt("SBD %5d %4d %4d %015d %5d %5d %3d %20s %19s %4d");
01454
01455         os « format(fmt) % id_instrument() % id_dive() % sbd_file_size_bytes() % (id_imei() % 10000000)
01456                          % id_sbd_momsn() % id_sbd_mtmsn() % id_sbd_packet()
01457                          % iridium_position_datetime_as_iso8601_string()
01458                          % iridium_position_as_iso6709_string() % iridium_position_cep_radius();
01459
01460       }CATCH_BLOCK("Failed to generate listing string from mail message object!")
01461
01462       return os.str();
01463     }
01464
01465     const string GenerateSbdHexListingString() const {
01466
01467       static const format fmt_char(" %2c");
01468       static const format fmt_hex(" %02X");
01469       static const format fmt_int(" %02d");
01470       static const format fmt_line("shx %5d %4d %5d %3d %3d 0x%04X (%04u) | 0x");
01471
01472       size_t byte_count = 0;
01473       ostringstream os;
01474       size_t idx = 0;
01475
01476       try {
01477
01478         /* Print header lines:*/
01479
01480         {
01481
01482           os « "#      SN dive   size MOMSN MTMSN PID"
01483              « endl
01484              « "#                 (b)"
01485              « endl;
01486
01487           //          static const format fmt1("#   %4s %4s %6s %5d %5d %3d");
01488           //          static const format fmt2("#    %4s %4s%7s %5d %5d %3d");
01489           //          os « format(fmt1) % "SN" % "dive" % "size"    % "MOMSN" % "MTMSN" % "PID" « endl;
01490           //          os « format(fmt2) % ""   % ""     % "(bytes)" % ""      % ""      % ""    « endl;
01491         }
01492
01493         { static const format fmt("SHX %5d %4d %6d %5d %5d %3d");
01494         os « format(fmt)
01495                   % id_instrument() % id_dive()      % sbd_file_size_bytes()
01496                   % id_sbd_momsn()  % id_sbd_mtmsn() % id_sbd_packet()        « endl;
01497         }
01498
01499         os «
    "#-------------------------------------------------------------------------------------------------"
01500            « endl
01501            « "#      SN dive MOMSN idx PID               | 0x  0  1  2  3  4  5  6  7  8  9  A  B  C
    D  E  F"
01502            « endl
```

```
01503             « "#                                                |    00 01 02 03  04 05 06 07  08 09 10 11  12
      13 14 15"
01504             « endl
01505             « "# - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
      - - - - "
01506             « endl;
01507
01508         // Print the bytes in a readable format.
01509         vector<uint8_t> data(sbd_data());
01510         for(vector<uint8_t>::const_iterator data_iterator = data.begin();  data_iterator < data.end();) {
01511
01512           ostringstream readable_line_char;
01513           ostringstream readable_line_hex;
01514
01515           // Iterate over bytes in groups of 16.
01516           for(size_t i = 0; i < 16 && data_iterator < data.end(); i++) {
01517             if(i == 0) {
01518               readable_line_hex « format(fmt_line)
01519                       % id_instrument() % id_dive() % id_sbd_momsn() % idx++ % id_sbd_packet()
01520                       % byte_count % byte_count;
01521               readable_line_char « "#                                                |   ";
01522             } else if(byte_count % 4 == 0) {
01523               readable_line_hex « " ";
01524               readable_line_char « " ";
01525             }
01526
01527             readable_line_hex « format(fmt_hex) % (int) *data_iterator;
01528
01529             if(isprint(*data_iterator)) readable_line_char « format(fmt_char) % (char) *data_iterator;
01530             else                        readable_line_char « format(fmt_char) % ".";
01531
01532             data_iterator++;
01533             byte_count++;
01534           }
01535
01536           os « readable_line_hex.str() « endl « readable_line_char.str() « endl;
01537         }
01538         os «
      "#-----------------------------------------------------------------------------------------------" «
      endl;
01539
01540       }CATCH_BLOCK("Failed to generate SBD hex listing string from mail message object!")
01541
01542       return os.str();
01543     }
01544
01545     /** Get the cycle or dive number of this mail message.
01546      *
01547      *  @return         The cycle or dive number of this mail message.
01548      */
01549     int id_dive() const {
01550
01551       // Get an exclusive lock.
01552       lock_guard<recursive_mutex> guard(mutex_);
01553
01554       return id_dive_;
01555     }
01556
01557     /** Get the mail message unique identifier string.
01558      *
01559      *  @return         The mail message unique identifier string.
01560      */
01561     string id_msg() const {
01562
01563       // Get an exclusive lock.
01564       lock_guard<recursive_mutex> guard(mutex_);
01565
01566       return id_msg_;
01567     }
01568
01569     /** Get the mail message unique identifier.
01570      *
01571      *  @return         The mail message unique identifier.
01572      */
01573     long id_msg_uid() const {
01574
01575       // Get an exclusive lock.
01576       lock_guard<recursive_mutex> guard(mutex_);
01577
01578       return id_msg_uid_;
01579     }
01580
01581     /** Get the Iridium Short Burst Data (SBD) Service Mobile Originated Message Sequence Number
01582      *  (MOMSN) identifier.
01583      *
01584      *  @return         The  Iridium Short Burst Data (SBD) Service Mobile Originated Message
01585      *                  Sequence Number (MOMSN) identifier.
01586      */
01587     int id_sbd_momsn() const {
01588
01589       // Get an exclusive lock.
01590       lock_guard<recursive_mutex> guard(mutex_);
01591
01592       return id_sbd_momsn_;
01593     }
01594
01595     /** Get the Iridium Short Burst Data (SBD) Service Mobile Terminated Message Sequence Number
01596      *  (MTMSN) identifier.
01597      *
01598      *  @return         The Iridium Short Burst Data (SBD) Service Mobile Terminated Message
01599      *                  Sequence Number (MTMSN) identifier.
01600      */
01601     int id_sbd_mtmsn() const {
01602
01603       // Get an exclusive lock.
01604       lock_guard<recursive_mutex> guard(mutex_);
01605
01606       return id_sbd_mtmsn_;
```

```
01607     }
01608
01609     /** Get the Iridium Short Burst Data (SBD) message packet index.
01610      *
01611      *  @return         The Iridium Short Burst Data (SBD) message packet index.
01612      */
01613     int id_sbd_packet() const {
01614
01615       // Get an exclusive lock.
01616       lock_guard<recursive_mutex> guard(mutex_);
01617
01618       return id_sbd_packet_;
01619     }
01620
01621     /** Is the body of the mail message valid?
01622      *
01623      *  @return         TRUE if the body of the mail message is valid, or FALSE otherwise.
01624      */
01625     bool msg_body_is_valid() const {
01626
01627       // Get an exclusive lock.
01628       lock_guard<recursive_mutex> guard(mutex_);
01629
01630       return msg_body_is_valid_;
01631     }
01632
01633     /** Get the mail message body text.
01634      *
01635      *  @return         The mail message body text.
01636      */
01637     string msg_body_text() const {
01638
01639       // Get an exclusive lock.
01640       lock_guard<recursive_mutex> guard(mutex_);
01641
01642       return msg_body_text_;
01643     }
01644
01645     /** Get the mail message header date/time.
01646      *
01647      *  @return         The mail message header date/time.
01648      */
01649     ptime msg_header_datetime() const {
01650
01651       // Get an exclusive lock.
01652       lock_guard<recursive_mutex> guard(mutex_);
01653
01654       return msg_header_datetime_;
01655     }
01656
01657     /** Get the mail message header FROM field.
01658      *
01659      *  @return         The mail message header FROM field.
01660      */
01661     string msg_header_from() const {
01662       // Get an exclusive lock.
01663       lock_guard<recursive_mutex> guard(mutex_);
01664
01665       return msg_header_from_;
01666     }
01667
01668     /** Is the header of the mail message valid?
01669      *
01670      *  @return         TRUE if the header of the mail message is valid, or FALSE otherwise.
01671      */
01672     bool msg_header_is_valid() const {
01673
01674       // Get an exclusive lock.
01675       lock_guard<recursive_mutex> guard(mutex_);
01676
01677       return msg_header_is_valid_;
01678     }
01679
01680     /** Get the message header raw text.
01681      *
01682      *  @return         The message header raw text.
01683      */
01684     string msg_header_raw_text() const {
01685
01686       // Get an exclusive lock.
01687       lock_guard<recursive_mutex> guard(mutex_);
01688
01689       return msg_header_raw_text_;
01690     }
01691
01692     /** Get the mail message header SUBJECT field.
01693      *
01694      *  @return         The mail message header SUBJECT field.
01695      */
01696     string msg_header_subject() const {
01697
01698       // Get an exclusive lock.
01699       lock_guard<recursive_mutex> guard(mutex_);
01700
01701       return msg_header_subject_;
01702     }
01703
01704     /** Is the mail message valid?
01705      *
01706      *  @return         TRUE if the mail message is valid, or FALSE otherwise.
01707      */
01708     bool msg_is_valid() const {
01709
01710       // Get an exclusive lock.
01711       lock_guard<recursive_mutex> guard(mutex_);
01712
01713       return msg_is_valid_;
01714     }
```

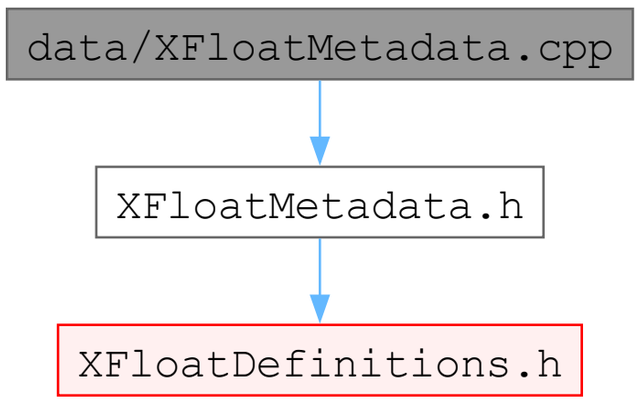
```
01715
01716     /** Get the full mail message raw text.
01717      *
01718      *  @return         The full mail message raw text.
01719      */
01720     string msg_raw_text() const {
01721
01722       // Get an exclusive lock.
01723       lock_guard<recursive_mutex> guard(mutex_);
01724
01725       return msg_raw_text_;
01726     }
01727
01728     /** Get the path to the raw mail message file on the local filesystem.
01729      *
01730      *  @return         The path to the raw mail message file on the local filesystem.
01731      */
01732     path raw_file_local_path() const {
01733
01734       // Get an exclusive lock.
01735       lock_guard<recursive_mutex> guard(mutex_);
01736
01737       return raw_file_local_path_;
01738     }
01739
01740     /** Save this mail message object to the SQL database.
01741      *
01742      *  @throw          util::log::MessageException if an error occurs.
01743      *
01744      */
01745     void SaveObjectToDatabase() const {
01746
01747       try {
01748
01749         // Get an exclusive lock.
01750         lock_guard<recursive_mutex> guard(mutex_);
01751
01752         sql_table_description_.SaveObjectToDatabase(*this);
01753
01754       }CATCH_BLOCK("Failed to save mail message to database!")
01755     }
01756
01757     /** Get the Iridium Short Burst Data (SBD) binary data.
01758      *
01759      *  @return         The Iridium Short Burst Data (SBD) binary data.
01760      */
01761     vector<uint8_t> sbd_data() const {
01762
01763       // Get an exclusive lock.
01764       lock_guard<recursive_mutex> guard(mutex_);
01765
01766       return sbd_data_;
01767     }
01768
01769     /** Is the Iridium Short Burst Data (SBD) message valid?
01770      *
01771      *  @return         TRUE if the Iridium Short Burst Data (SBD) message is valid, or FALSE
01772      *                  otherwise.
01773      */
01774     bool sbd_data_is_valid() const {
01775
01776       // Get an exclusive lock.
01777       lock_guard<recursive_mutex> guard(mutex_);
01778
01779       return sbd_data_is_valid_;
01780     }
01781
01782     /** Get the path to the Iridium Short Burst Data (SBD) binary file on the local filesystem.
01783      *
01784      *  @return         The path to the Iridium Short Burst Data (SBD) binary file on the local
01785      *                  filesystem.
01786      */
01787     path sbd_file_local_path() const {
01788
01789       // Get an exclusive lock.
01790       lock_guard<recursive_mutex> guard(mutex_);
01791
01792       return sbd_file_local_path_;
01793     }
01794
01795     /** Get the Iridium Short Burst Data (SBD) email attachment filename.
01796      *
01797      *  @return         The Iridium Short Burst Data (SBD) email attachment filename.
01798      */
01799     string sbd_file_name() const {
01800
01801       // Get an exclusive lock.
01802       lock_guard<recursive_mutex> guard(mutex_);
01803
01804       return sbd_file_name_;
01805     }
01806
01807     /** Get the size of the Iridium Short Burst Data (SBD) message, in bytes.
01808      *
01809      *  @return         The size of the Iridium Short Burst Data (SBD) message, in bytes.
01810      */
01811     int sbd_file_size_bytes() const {
01812
01813       // Get an exclusive lock.
01814       lock_guard<recursive_mutex> guard(mutex_);
01815
01816       return sbd_file_size_bytes_;
01817     }
01818
01819     /** Get the Iridium Short Burst Data (SBD) session status code.
01820      *
01821      *  @return         The Iridium Short Burst Data (SBD) session status code.
01822      */
```

```
01823     int sbd_session_status_code() const {
01824
01825       // Get an exclusive lock.
01826       lock_guard<recursive_mutex> guard(mutex_);
01827
01828       return sbd_session_status_code_;
01829     }
01830
01831     /** Get the Iridium Short Burst Data (SBD) session status string.
01832      *
01833      *  @return         The Iridium Short Burst Data (SBD) session status string.
01834      */
01835     string sbd_session_status_string() const {
01836
01837       // Get an exclusive lock.
01838       lock_guard<recursive_mutex> guard(mutex_);
01839
01840       return sbd_session_status_string_;
01841     }
01842
01843     const shared_ptr<vector<uint8_t> >& xmsg_id_types() const {
01844
01845       // Get an exclusive lock.
01846       lock_guard<recursive_mutex> guard(mutex_);
01847
01848       return xmsg_id_types_;
01849     }
01850
01851 };
01852
01853 /* Function Prototypes: */
01854
01855 ostream& operator<<(ostream& os, const XMailSbdData_t& data);
01856
01857 ostream& operator<<(ostream& os, const XFloatMailMessage& mail);
01858
01859 istream& operator>>(istream& is, XFloatMailMessage& mail);
01860
01861 } /* namespace xfloat */
01862 } /* namespace iridium */
01863 } /* namespace argo */
01864
01865 #endif /* XFLOATMAILMESSAGE_H_ */
```

11.28 data/XFloatMetadata.cpp File Reference

The argo::iridium::xfloat::XFloatMetadata class source file.

`#include "XFloatMetadata.h"`
Include dependency graph for XFloatMetadata.cpp:



Namespaces

- namespace argo

  The Argo instrument namespace.
- namespace argo::iridium

  The Argo Iridium instrument namespace.
- namespace argo::iridium::xfloat

  The X-Float instrument namespace.

Functions

- istream & argo::iridium::xfloat::operator>> (istream &in, XFloatMetadata &metadata)

  Stream extraction operator overload function for the XFloatMetadata class.

11.28.1 Detailed Description

The argo::iridium::xfloat::XFloatMetadata class source file.

This is the source file for the argo::iridium::xfloat::XFloatMetadata class. This class contains the metadata for a single instrument. The metadata object includes information about the instrument including details about deployment, principal investigators, and the instrument type. During application initialization, the metadata files are read from the local filesystem, and used to construct one metadata object per metadata file. Additionally, the metadata is persisted to the SQL database.

This file contains the class definition and the majority of the functionality of the class.

Version

0.6

Date

Aug 18, 2014 4:25:34 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

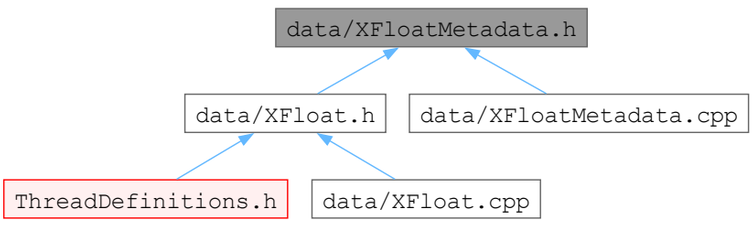
11.29 data/XFloatMetadata.h File Reference

The argo::iridium::xfloat::XFloatMetadata class header file.

```
#include "XFloatDefinitions.h"
```

Include dependency graph for XFloatMetadata.h:

This graph shows which files directly or indirectly include this file:



Classes

- class argo::iridium::xfloat::XFloatMetadata

  The instrument metadata class.

Namespaces

- namespace argo

  The Argo instrument namespace.
- namespace argo::iridium

  The Argo Iridium instrument namespace.
- namespace argo::iridium::xfloat

  The X-Float instrument namespace.

Functions

- istream & argo::iridium::xfloat::operator>> (istream &in, XFloatMetadata &metadata)

  Stream extraction operator overload function for the XFloatMetadata class.

11.29.1 Detailed Description

The argo::iridium::xfloat::XFloatMetadata class header file.

This is the header file for the argo::iridium::xfloat::XFloatMetadata class. This class contains the metadata for a single instrument. The metadata object includes information about the instrument including details about deployment, principal investigators, and the instrument type. During application initialization, the metadata files are read from the local filesystem, and used to construct one metadata object per metadata file. Additionally, the metadata is persisted to the SQL database.

This file contains the class definition and the majority of the functionality of the class.

Version

0.6

Date

Aug 18, 2014 4:25:34 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.30 XFloatMetadata.h

Go to the documentation of this file.

```
00001 /** The @ref argo::iridium::xfloat::XFloatMetadata class header file.
00002  *
00003  *  This is the header file for the @ref argo::iridium::xfloat::XFloatMetadata class. This class
00004  *  contains the metadata for a single instrument. The metadata object includes information about
00005  *  the instrument including details about deployment, principal investigators, and the instrument
00006  *  type.  During application initialization, the metadata files are read from the local
00007  *  filesystem, and used to construct one metadata object per metadata file. Additionally,
00008  *  the metadata is persisted to the SQL database.
00009  *
00010  *  This file contains the class definition and the majority of the functionality of the class.
00011  *
00012  *  @file
00013  *
00014  *  @version    0.6
00015  *
00016  *  @date       Aug 18, 2014 4:25:34 PM
00017  *
00018  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00019  *              Engineer I,
00020  *              Department of Physical Oceanography,
00021  *              Woods Hole Oceanographic Institution
00022  *
00023  *  @copyright  2014 Woods Hole Oceanographic Institution.
00024  *              Released under the GNU General Public License.
00025  *
00026  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00027  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00028  *  General Public License for more details. You should have received a copy of the GNU General
00029  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00030  */
00031
00032 #ifndef XFLOATMETADATA_H_
00033 #define XFLOATMETADATA_H_
00034
00035 /* Includes: */
00036
00037 #include "XFloatDefinitions.h"
00038
00039 namespace argo {
00040 namespace iridium {
00041 namespace xfloat {
00042
00043 /* Class Definition: */
00044
00045 /** The instrument metadata class.
00046  *
00047  *  This class contains the metadata for a single instrument. The metadata object includes
00048  *  information about the instrument including details about deployment, principal investigators,
00049  *  and the instrument type.  During application initialization, the metadata files are read from
00050  *  the local filesystem, and used to construct one metadata object per metadata file.
00051  *  Additionally, the metadata is persisted to the SQL database.
00052  *
00053  *  @todo   Devise a method to actively check for (1) changes in the metadata files and (2) newly
00054  *          created metadata files. Load the metadata from the new or modified files and assign
00055  *          them to the corresponding float object. If necessary, retroactively process the mail
00056  *          messages for these instruments. Persist updates to the SQL database as necessary.
00057  */
00058 class XFloatMetadata:
00059         public util::file::FileSystemUtils,
00060         public argo::iridium::Iridium,
00061         public util::mysql::LocalPersistentEntity,
00062         protected util::position::Position {
00063
00064     /* Friends Classes: */
00065
00066     friend class XFloat;
00067
00068     /* Friend Functions: */
00069
00070     /** Friend function for setting the value of some of the static configuration options that are
00071      *  members of this class.
00072      */
00073     friend options_description
      util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription();
00074
00075     friend istream& operator»(istream& in, XFloatMetadata& metadata);
00076
00077 private:
00078
00079     /* Static Data Members: */
00080
00081     /** The path to metadata directory on the local filesystem.
00082      *
00083      *  This path indicates the directory containing the instrument metadata files. The value of
00084      *  this field is set during application initialization as a result of parsing the
00085      *  command-line arguments and configuration file arguments.
00086      *
00087      *  After the run-time configuration settings are parsed, but before the completion of
00088      *  application initialization, the metadata files are read from this directory and used to
00089      *  generate one instance of this class for each metadata file read. The resulting object are
00090      *  assigned to the corresponding instrument objects (@ref XFloat), and are save to the SQL
00091      *  database.
00092      *
00093      */
00094     static path metadata_directory_path_;
00095
00096     /** The SQL table description object.
00097      *
00098      *  This SQL table description provides the details needed to map this object to a relational
00099      *  database table. It utilizes the accessor methods of the class and the details of the SQL
00100     *  table column to (1) create an SQL INSERT statement to save the object into the database,
00101     *  (2) populate one or more objects' fields as a result of an SQL SELECT query, or (3) query
00102     *  the SQL database to access one or more of the values that are stored the the SQL table
```

```
00103      *  corresponding to this object.
00104      */
00105     static const util::mysql::SqlTableDescription<XFloatMetadata> sql_table_description_;
00106
00107 private:
00108
00109     /* Dynamic Data Members: */
00110
00111     /** The instrument comment(s).
00112      *
00113      */
00114     string comment_;
00115
00116     /* Dynamic Data Members - Deployment/Launch: */
00117
00118     /** The name of the instrument deployer(s).
00119      *
00120      */
00121     string deployer_;
00122
00123     /** The address of the instrument deployer(s).
00124      *
00125      */
00126     string deployer_address_;
00127
00128     /** The unique identifier of the deployment or launch cruise.
00129      *
00130      */
00131     string deployment_cruiseId_;
00132
00133     /** The status of the accompanying date/time of the instrument deployment or launch.
00134      *
00135      */
00136     string deployment_datetime_status_;
00137
00138     /** The deployment or launch platform.
00139      *
00140      */
00141     string deployment_platform_;
00142
00143     /** The status of the accompanying date/time of the instrument deployment or launch.
00144      *
00145      */
00146     string deployment_position_status_;
00147
00148     /** The deployment or launch type.
00149      *
00150      */
00151     string deployment_type_;
00152
00153     /* Dynamic Data Members - Other: */
00154
00155     /** The instrument manufacturer.
00156      *
00157      */
00158     string manufacturer_;
00159
00160     /** The path to the metadata file on the local filesystem.
00161      *
00162      */
00163     string local_path_;
00164
00165     string instrument_type_;
00166
00167     /* Dynamic Data Members - Principal Investigators: */
00168
00169     /** The names of the principal investigator(s).
00170      *
00171      */
00172     string principal_investigators_;
00173
00174     /** The address of the principal investigator(s).
00175      *
00176      */
00177     string principal_investigators_address_;
00178
00179     /** The originating country of the instrument.
00180      *
00181      */
00182     string principal_investigators_originating_country_;
00183
00184     /* Dynamic Data Members - WMO Identifiers: */
00185
00186     /** The WMO instrument type identifier.
00187      *
00188      *  @note           This value corresponds to the WMO reference table 1770.
00189      *
00190      */
00191     int wmo_instrument_type_;
00192
00193     /** The WMO recorder type identifier.
00194      *
00195      *  @note           This value corresponds to the WMO reference table 4770.
00196      *
00197      */
00198     int wmo_recorder_type_;
00199
00200 protected:
00201
00202     /* Protected Member Functions */
00203
00204     /** List the paths to all metadata files on the local filesystem.
00205      *
00206      *  @throw          util::log::MessageException if an error occurs while listing the metadata
00207      *                  paths.
00208      *
00209      *  @return         A vector containing the paths to the metadata files on the local filesystem.
00210      */
```

```
00211     static vector<path> ListMetadataFiles() {
00212
00213         vector<path> paths;
00214
00215         try {
00216             // List all files in the metadata directory.
00217             if(!is_directory(metadata_directory_path_))
00218                 EXC_THROW("Invalid metadata path: " + metadata_directory_path_.string());
00219
00220             vector<path> files(ListFiles(metadata_directory_path_));
00221
00222             // Copy paths to files matching the regex.
00223             copy_if(files.begin(), files.end(), back_inserter(paths), [](const path & p) {
00224
00225                 return regex_match(p.string(),
00226                         regex(".*/[0-9]{4,5}_[0-9]{4,6}\\.meta", regex::extended));
00227             });
00228
00229         }CATCH_BLOCK("Failed to get paths to metadata files!")
00230
00231         return paths;
00232     }
00233
00234     /** Read instrument metadata from a file on the local filesystem.
00235      *
00236      *  @param[in]      path        The path to the metadata file on the local filesystem.
00237      *
00238      *  @throw          util::log::MessageException if the file does not exist or extraction of
00239      *                  metadata from the file fails.
00240      */
00241     void LoadFile(const path& path) {
00242
00243         try {
00244
00245             // Check if file exists.
00246             if(!exists(path))
00247                 EXC_THROW("File does not exist!");
00248
00249             ldbg() « "Loading instrument metadata from file: " « path.string() « endl;
00250
00251             // Set local path.
00252             set_local_path(path.string());
00253
00254             // Read metadata from stream.
00255             ifstream ifs(path);
00256             ifs » *this;
00257             ifs.close();
00258
00259         }CATCH_NOTHROW("Failed to extract metadata from file: " + path.string() + "!")
00260     }
00261
00262     /** Load all metadata from the files on the local filesystem.
00263      *
00264      *  @throw          util::log::MessageException if an error occurs while loading the metadata from
00265      *                  the files on the local filesystem.
00266      *
00267      *  @return         A vector of shared pointers to the metadata objects constructed from the
00268      *                  metadata files on the local filesystem.
00269      */
00270     static vector<shared_ptr<XFloatMetadata> > LoadMetadataFromFiles() {
00271
00272         vector<shared_ptr<XFloatMetadata> > md;
00273
00274         try {
00275
00276             vector<path> paths(ListMetadataFiles());
00277             for_each(paths.begin(), paths.end(), [&](const path& path) {
00278
00279                 md.push_back(make_shared<XFloatMetadata>(path));
00280             });
00281
00282         }CATCH_BLOCK("Failed to load metadata!")
00283
00284         return md;
00285     }
00286
00287     /** Set the instrument comment(s).
00288      *
00289      *  @param[in]  comment     The instrument comment(s).
00290      */
00291     void set_comment(const string& comment) {
00292
00293         // Get an exclusive lock.
00294         lock_guard<recursive_mutex> guard(mutex_);
00295
00296         comment_ = comment;
00297     }
00298
00299     /** Set the name of the instrument deployer(s).
00300      *
00301      *  @param[in]  deployer    The name of the instrument deployer(s).
00302      */
00303     void set_deployer(const string& deployer) {
00304
00305         // Get an exclusive lock.
00306         lock_guard<recursive_mutex> guard(mutex_);
00307
00308         deployer_ = deployer;
00309     }
00310
00311     /** Set the address of the instrument deployer(s).
00312      *
00313      *  @param[in]  address    The address of the instrument deployer(s).
00314      */
00315     void set_deployer_address(const string& address) {
00316
00317         // Get an exclusive lock.
00318         lock_guard<recursive_mutex> guard(mutex_);
```

```
00319
00320         deployer_address_ = address;
00321     }
00322
00323     /** Set the unique identifier of the deployment or launch cruise.
00324      *
00325      *  @param[in]  id          The unique identifier of the deployment or launch cruise.
00326      */
00327     void set_deployment_cruise_id(const string& id) {
00328
00329         // Get an exclusive lock.
00330         lock_guard<recursive_mutex> guard(mutex_);
00331
00332         deployment_cruiseId_ = id;
00333     }
00334
00335     /** Set the date/time of the deployment.
00336      *
00337      *  @param[in]  dt          The date/time of the deployment.
00338      */
00339     void set_deployment_datetime(const ptime& dt) {
00340
00341         // Get an exclusive lock.
00342         lock_guard<recursive_mutex> guard(mutex_);
00343
00344         set_position_datetime(dt);
00345     }
00346
00347     /** Set the status of the accompanying date/time of the instrument deployment or launch.
00348      *
00349      *  @param[in]  stat        The status of the accompanying date/time of the instrument
00350      *                          deployment or launch.
00351      */
00352     void set_deployment_datetime_status(const string& stat) {
00353
00354         // Get an exclusive lock.
00355         lock_guard<recursive_mutex> guard(mutex_);
00356
00357         deployment_datetime_status_ = stat;
00358     }
00359
00360     /** Set the deployment position latitude, in decimal degrees North.
00361      *
00362      *  @param[in]  lat         The deployment position latitude, in decimal degrees North.
00363      */
00364     void set_deployment_latitude(double lat) {
00365
00366         // Get an exclusive lock.
00367         lock_guard<recursive_mutex> guard(mutex_);
00368
00369         set_position_latitude(lat);
00370     }
00371
00372     /** Set the deployment position longitude, in decimal degrees East.
00373      *
00374      *  @param[in]  lon         The deployment position longitude, in decimal degrees East.
00375      */
00376     void set_deployment_longitude(double lon) {
00377
00378         // Get an exclusive lock.
00379         lock_guard<recursive_mutex> guard(mutex_);
00380
00381         set_position_longitude(lon);
00382     }
00383
00384     /** Set the deployment or launch platform.
00385      *
00386      *  @param[in]  platform    The deployment or launch platform.
00387      */
00388     void set_deployment_platform(const string& platform) {
00389
00390         // Get an exclusive lock.
00391         lock_guard<recursive_mutex> guard(mutex_);
00392
00393         deployment_platform_ = platform;
00394     }
00395
00396     /** Set the status of the accompanying date/time of the instrument deployment or launch.
00397      *
00398      *  @param[in]  status      The status of the accompanying date/time of the instrument
00399      *                          deployment or launch.
00400      */
00401     void set_deployment_position_status(const string& status) {
00402
00403         // Get an exclusive lock.
00404         lock_guard<recursive_mutex> guard(mutex_);
00405
00406         deployment_position_status_ = status;
00407     }
00408
00409     void set_deployment_position_valid(bool valid) {
00410
00411         // Get an exclusive lock.
00412         lock_guard<recursive_mutex> guard(mutex_);
00413
00414         set_position_is_valid(valid);
00415     }
00416
00417     /** Set the deployment or launch type.
00418      *
00419      *  @param[in]  type        The deployment or launch type.
00420      */
00421     void set_deployment_type(const string& type) {
00422
00423         // Get an exclusive lock.
00424         lock_guard<recursive_mutex> guard(mutex_);
00425
00426         deployment_type_ = type;
```

```
00427     }
00428
00429
00430     void set_instrument_type(const string& instrument_type) {
00431       instrument_type_ = instrument_type;
00432     }
00433
00434     /** Set the path to the metadata file on the local filesystem.
00435      *
00436      *  @param[in]  local_path  The path to the metadata file on the local filesystem.
00437      */
00438     void set_local_path(const string& local_path) {
00439
00440         // Get an exclusive lock.
00441         lock_guard<recursive_mutex> guard(mutex_);
00442
00443         local_path_ = local_path;
00444     }
00445
00446     /** Set the instrument manufacturer.
00447      *
00448      *  @param[in]  man         The instrument manufacturer.
00449      */
00450     void set_manufacturer(const string& man) {
00451
00452         // Get an exclusive lock.
00453         lock_guard<recursive_mutex> guard(mutex_);
00454
00455         manufacturer_ = man;
00456     }
00457
00458     /** Set the names of the principal investigator(s).
00459      *
00460      *  @param[in]  pis         The names of the principal investigator(s).
00461      */
00462     void set_principal_investigators(const string& pis) {
00463
00464         // Get an exclusive lock.
00465         lock_guard<recursive_mutex> guard(mutex_);
00466
00467         principal_investigators_ = pis;
00468     }
00469
00470     /** Set the address of the principal investigator(s).
00471      *
00472      *  @param[in]  addr        The address of the principal investigator(s).
00473      */
00474     void set_principal_investigators_address(const string& addr) {
00475
00476         // Get an exclusive lock.
00477         lock_guard<recursive_mutex> guard(mutex_);
00478
00479         principal_investigators_address_ = addr;
00480     }
00481
00482     /** Set the originating country of the instrument.
00483      *
00484      *  @param[in]  country     The originating country of the instrument.
00485      */
00486     void set_principal_investigators_originating_country(const string& country) {
00487
00488         // Get an exclusive lock.
00489         lock_guard<recursive_mutex> guard(mutex_);
00490
00491         principal_investigators_originating_country_ = country;
00492     }
00493
00494     /** Set the WMO instrument type identifier.
00495      *
00496      *  @param[in]  type        The WMO instrument type identifier.
00497      */
00498     void set_wmo_instrument_type(int type) {
00499
00500         // Get an exclusive lock.
00501         lock_guard<recursive_mutex> guard(mutex_);
00502
00503         wmo_instrument_type_ = type;
00504     }
00505
00506     /** Set the WMO recorder type identifier.
00507      *
00508      *  @param[in]  type        The WMO recorder type identifier.
00509      */
00510     void set_wmo_recorder_type(int type) {
00511
00512         // Get an exclusive lock.
00513         lock_guard<recursive_mutex> guard(mutex_);
00514
00515         wmo_recorder_type_ = type;
00516     }
00517
00518 public:
00519
00520     /* Public Member Functions: */
00521
00522     /** The default constructor.
00523      *
00524      */
00525     XFloatMetadata() :
00526                     Iridium(),
00527                     deployer_("unknown"),
00528                     deployer_address_("unknown"),
00529                     deployment_cruiseId_("unknown"),
00530                     deployment_datetime_status_("unknown"),
00531                     deployment_platform_("unknown"),
00532                     deployment_position_status_("unknown"),
00533                     deployment_type_("unknown"),
00534                     manufacturer_("unknown"),
```

```
00535                   principal_investigators_("Steven Jayne, Breck Owens, P.E. Robbins"),
00536                   principal_investigators_address_("WHOI, Woods Hole, MA"),
00537                   principal_investigators_originating_country_("USA"),
00538                   wmo_instrument_type_(XFLT_ID_UNKNOWN),
00539                   wmo_recorder_type_(XFLT_ID_UNKNOWN) {
00540
00541       /* Do nothing. */
00542   }
00543
00544   /** The file constructor.
00545    *
00546    *  Reads instrument metadata from a file on the local filesystem.
00547    *
00548    *  @param[in]      path        The path to the metadata file on the local filesystem.
00549    *
00550    *  @throw          util::log::MessageException if the file does not exist or extraction of
00551    *                  metadata from the file fails.
00552    */
00553   XFloatMetadata(const path& path) :
00554                   Iridium(),
00555                   deployer_("unknown"),
00556                   deployer_address_("unknown"),
00557                   deployment_cruiseId_("unknown"),
00558                   deployment_datetime_status_("unknown"),
00559                   deployment_platform_("unknown"),
00560                   deployment_position_status_("unknown"),
00561                   deployment_type_("unknown"),
00562                   manufacturer_("unknown"),
00563                   principal_investigators_("Steven Jayne, Breck Owens, P.E. Robbins"),
00564                   principal_investigators_address_("WHOI, Woods Hole, MA"),
00565                   principal_investigators_originating_country_("USA"),
00566                   wmo_instrument_type_(XFLT_ID_UNKNOWN),
00567                   wmo_recorder_type_(XFLT_ID_UNKNOWN) {
00568       try {
00569
00570           // Extract data from the file at path.
00571           LoadFile(path);
00572
00573       }CATCH_BLOCK("Failed to construct Metadata object!")
00574   }
00575
00576   /** The destructor.
00577    *
00578    */
00579   ~XFloatMetadata() {
00580
00581       /* Do nothing. */
00582   }
00583
00584   /** Get the instrument comment(s).
00585    *
00586    *  @return         The instrument comment(s).
00587    */
00588   string comment() const {
00589
00590       // Get an exclusive lock.
00591       lock_guard<recursive_mutex> guard(mutex_);
00592
00593       return comment_;
00594   }
00595
00596   /** Get the name of the instrument deployer(s).
00597    *
00598    *  @return         The name of the instrument deployer(s).
00599    */
00600   string deployer() const {
00601
00602       // Get an exclusive lock.
00603       lock_guard<recursive_mutex> guard(mutex_);
00604
00605       return deployer_;
00606   }
00607
00608   /** Get the address of the instrument deployer(s).
00609    *
00610    *  @return         The address of the instrument deployer(s).
00611    */
00612   string deployer_address() const {
00613
00614       // Get an exclusive lock.
00615       lock_guard<recursive_mutex> guard(mutex_);
00616
00617       return deployer_address_;
00618   }
00619
00620   /** Get the unique identifier of the deployment or launch cruise.
00621    *
00622    *  @return         The unique identifier of the deployment or launch cruise.
00623    */
00624   string deployment_cruise_id() const {
00625
00626       // Get an exclusive lock.
00627       lock_guard<recursive_mutex> guard(mutex_);
00628
00629       return deployment_cruiseId_;
00630   }
00631
00632   /** Get the date/time of the deployment.
00633    *
00634    *  @return         The date/time of the deployment.
00635    */
00636   ptime deployment_datetime() const {
00637
00638       // Get an exclusive lock.
00639       lock_guard<recursive_mutex> guard(mutex_);
00640
00641       return position_datetime();
00642   }
```

```
00643
00644     /** Get the latitude of the deployment position, in decimal degrees North.
00645      *
00646      *  @return         The latitude of the deployment position, in decimal degrees North.
00647      */
00648     double deployment_latitude() const {
00649
00650         // Get an exclusive lock.
00651         lock_guard<recursive_mutex> guard(mutex_);
00652
00653         return position_latitude();
00654     }
00655
00656     /** Get the longitude of the deployment position, in decimal degrees East.
00657      *
00658      *  @return         The longitude of the deployment position, in decimal degrees East.
00659      */
00660     double deployment_longitude() const {
00661
00662         // Get an exclusive lock.
00663         lock_guard<recursive_mutex> guard(mutex_);
00664
00665         return position_longitude();
00666     }
00667
00668     /** Get the status of the accompanying date/time of the instrument deployment or launch.
00669      *
00670      *  @return         The status of the accompanying date/time of the instrument deployment or
00671      *                  launch.
00672      */
00673     string deployment_datetime_status() const {
00674
00675         // Get an exclusive lock.
00676         lock_guard<recursive_mutex> guard(mutex_);
00677
00678         return deployment_datetime_status_;
00679     }
00680
00681     /** Get the deployment or launch platform.
00682      *
00683      *  @return         The deployment or launch platform.
00684      */
00685     string deployment_platform() const {
00686
00687         // Get an exclusive lock.
00688         lock_guard<recursive_mutex> guard(mutex_);
00689
00690         return deployment_platform_;
00691     }
00692
00693     /** Get the status of the accompanying date/time of the instrument deployment or launch.
00694      *
00695      *  @return         The status of the accompanying date/time of the instrument deployment or
00696      *                  launch.
00697      */
00698     string deployment_position_status() const {
00699
00700         // Get an exclusive lock.
00701         lock_guard<recursive_mutex> guard(mutex_);
00702
00703         return deployment_position_status_;
00704     }
00705
00706     string deployment_position_as_iso6709_string() const {
00707
00708         // Get an exclusive lock.
00709         lock_guard<recursive_mutex> guard(mutex_);
00710
00711         return postition_as_iso6709_string();
00712     }
00713
00714     /** Get the deployment or launch type.
00715      *
00716      *  @return         The deployment or launch type.
00717      */
00718     string deployment_type() const {
00719
00720         // Get an exclusive lock.
00721         lock_guard<recursive_mutex> guard(mutex_);
00722
00723         return deployment_type_;
00724     }
00725
00726     bool is_deployment_position_valid() const {
00727
00728         // Get an exclusive lock.
00729         lock_guard<recursive_mutex> guard(mutex_);
00730
00731         return position_is_valid();
00732     }
00733
00734     /** Get the path to the metadata file on the local filesystem.
00735      *
00736      *  @return         The path to the metadata file on the local filesystem.
00737      */
00738     string local_path() const {
00739
00740         // Get an exclusive lock.
00741         lock_guard<recursive_mutex> guard(mutex_);
00742
00743         return local_path_;
00744     }
00745
00746     /** Get the instrument manufacturer.
00747      *
00748      *  @return         The instrument manufacturer.
00749      */
00750     string manufacturer() const {
```

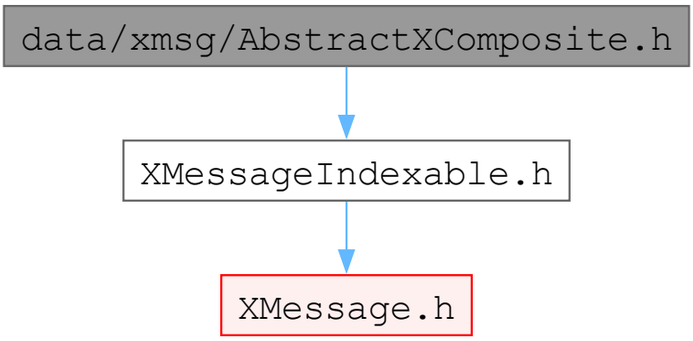
```
00751
00752         // Get an exclusive lock.
00753         lock_guard<recursive_mutex> guard(mutex_);
00754
00755         return manufacturer_;
00756     }
00757
00758     /** Get the names of the principal investigator(s).
00759      *
00760      *  @return         The names of the principal investigator(s).
00761      */
00762     string principal_investigators() const {
00763
00764         // Get an exclusive lock.
00765         lock_guard<recursive_mutex> guard(mutex_);
00766
00767         return principal_investigators_;
00768     }
00769
00770     /** Get the address of the principal investigator(s).
00771      *
00772      *  @return         The address of the principal investigator(s).
00773      */
00774     string principal_investigators_address() const {
00775
00776         // Get an exclusive lock.
00777         lock_guard<recursive_mutex> guard(mutex_);
00778
00779         return principal_investigators_address_;
00780     }
00781
00782     /** Get the originating country of the instrument.
00783      *
00784      *  @return         The originating country of the instrument.
00785      */
00786     string principal_investigators_originating_country() const {
00787
00788         // Get an exclusive lock.
00789         lock_guard<recursive_mutex> guard(mutex_);
00790
00791         return principal_investigators_originating_country_;
00792     }
00793
00794     /** Save this object to the database.
00795      *
00796      *  @throw      util::log::MessageException if an error occurs while saving this object to the
00797      *              database.
00798      *
00799      */
00800     void SaveObjectToDatabase() const {
00801
00802         try {
00803
00804             // Get an exclusive lock.
00805             lock_guard<recursive_mutex> guard(mutex_);
00806
00807             ldbg() « "Saving metadata object to database...";
00808
00809             sql_table_description_.SaveObjectToDatabase(*this);
00810
00811         }CATCH_BLOCK("Failed to save metadata to database!")
00812     }
00813
00814     /** Get the WMO instrument type identifier.
00815      *
00816      *  @return         The WMO instrument type identifier.
00817      */
00818     int wmo_instrument_type() const {
00819
00820         // Get an exclusive lock.
00821         lock_guard<recursive_mutex> guard(mutex_);
00822
00823         return wmo_instrument_type_;
00824     }
00825
00826     /** Get the WMO recorder type identifier.
00827      *
00828      *  @return         The WMO recorder type identifier.
00829      */
00830     int wmo_recorder_type() const {
00831
00832         // Get an exclusive lock.
00833         lock_guard<recursive_mutex> guard(mutex_);
00834
00835         return wmo_recorder_type_;
00836     }
00837
00838     const string& instrument_type() const {
00839       return instrument_type_;
00840     }
00841
00842 };
00843
00844 /* Function Prototypes: */
00845
00846 istream& operator»(istream& in, XFloatMetadata& metadata);
00847
00848 } /* namespace xfloat */
00849 } /* namespace iridium */
00850 } /* namespace argo */
00851 #endif /* XFLOATMETADATA_H_ */
```

11.31 data/xmsg/AbstractXComposite.h File Reference

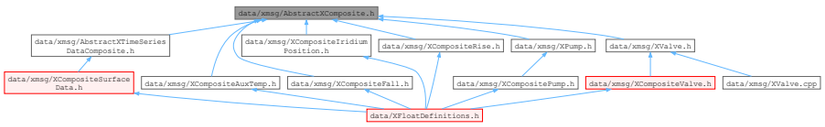
The util::xmsg::AbstractXComposite class source file.

```
#include "XMessageIndexable.h"
```

Include dependency graph for AbstractXComposite.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::AbstractXComposite< T >

  An abstract class representing XFloat dive data that consists of multiple X-Messages.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.31.1 Detailed Description

The util::xmsg::AbstractXComposite class source file.

Version

0.6

Date

Jul 22, 2014 12:39:08 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.32 AbstractXComposite.h

Go to the documentation of this file.

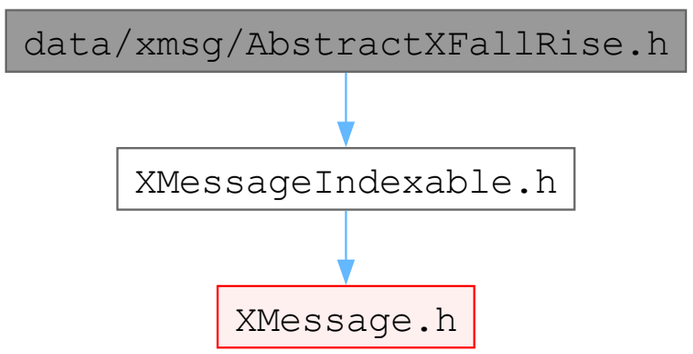
```
00001 /** The @ref util::xmsg::AbstractXComposite class source file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jul 22, 2014 12:39:08 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022 #ifndef AbstractXComposite_H_
00023 #define AbstractXComposite_H_
00024
00025 /* Includes */
00026
00027 // local
00028 #include "XMessageIndexable.h"
00029
00030 /* Definitions */
00031
00032 namespace util {
00033 namespace xmsg {
00034
00035 /** An abstract class representing XFloat dive data that consists of multiple X-Messages.
00036  *
00037  */
00038 template<typename T> class AbstractXComposite {
00039
00040     /* Type Definitions: */
00041
00042     /** Type definition for the element aggregated by this object.
00043      *
00044      */
00045     typedef T Element_t;
00046
00047     typedef vector<Element_t> ElementVec_t;
00048
00049     typedef typename ElementVec_t::iterator ElementIter_t;
00050
00051     typedef typename ElementVec_t::const_iterator ElementConstIter_t;
00052
00053   protected:
00054
00055   private:
00056
00057     /** The vector of individual X-Messages elements that compose this composite object. */
00058     ElementVec_t elements_;
00059
00060   protected:
00061
00062     /** Append one element to the vector of individual X-Messages elements that compose this
00063      *  composite object.
00064      *
00065      *  @param[in]      element         The element to append.
00066      */
00067     void AppendElement(const Element_t& element) {
00068
00069       elements_.push_back(element);
00070
00071     }
00072
00073     /** Append a vector of elements to the vector of individual X-Messages elements that compose
00074      *  this composite object.
00075      *
00076      *  @param[in]      elements        The vector of elements to append.
00077      */
00078     void AppendAllElements(const vector<Element_t>& elements) {
00079
00080       elements_.insert(elements_.end(), elements.begin(), elements.end());
00081
00082     }
00083
00084
00085
00086   public:
00087
00088     /** The default constructor.
00089      *
00090      */
00091     AbstractXComposite() :
00092             elements_() {
00093
00094       /* Do nothing. */
00095
00096     }
00097
00098     /** The destructor.
00099      *
00100      */
00101     virtual ~AbstractXComposite() {
00102
00103       /* Do nothing. */
```

```
00104
00105     }
00106
00107     /** Assign an element to this composite object.
00108      *
00109      *  The default implementation appends the element; derived classes may override it to insert
00110      *  elements in order or reject duplicates.
00111      *
00112      *  @param[in]  element     The element to assign.
00113      */
00114     virtual void AssignElement(const Element_t& element) {
00115
00116       // Default behavior: append element.
00117       AppendElement(element);
00118
00119     }
00120
00121     /** Append a vector of elements to the vector of individual X-Messages elements that compose
00122      *  this composite object.
00123      *
00124      *  @param[in]      elements        The vector of elements to append.
00125      */
00126     template<int I> void AppendAllElements(const static_vector<Element_t, I>& elements) {
00127       elements_.insert(elements_.end(), elements.begin(), elements.end());
00128     }
00129
00130     /** Get the vector of individual X-Messages elements that compose this composite object.
00131      *
00132      *  @return         The vector of individual X-Messages elements that compose this composite
00133      *                  object.
00134      */
00135     const vector<Element_t>& elements() const {
00136
00137       return elements_;
00138
00139     }
00140
00141     virtual bool IsEmpty() const {
00142
00143       return num_elements() == 0;
00144
00145     }
00146
00147     /** Get the number of individual elements that compose this composite object.
00148      *
00149      *  @return         The number of individual elements that compose this composite object.
00150      */
00151     virtual size_t num_elements() const {
00152
00153       return elements_.size();
00154
00155     }
00156
00157 };
00158
00159 } /* namespace xmsg */
00160 } /* namespace util */
00161 #endif /* AbstractXComposite_H_ */
```

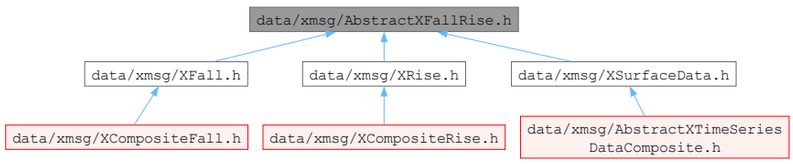
11.33 data/xmsg/AbstractXFallRise.h File Reference

The util::xmsg::AbstractXFallRise class header file.

`#include "XMessageIndexable.h"`
Include dependency graph for AbstractXFallRise.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::AbstractXFallRise

  An abstract class representing a fall, rise, or surface pressure time-series data X-message.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

Enumerations

- enum util::xmsg::XFallRiseDataPackFormat_t {
  util::xmsg::XRISEFALL_FMT_LEGACY = 0 ,
  util::xmsg::XRISEFALL_FMT_STNDRD = 1 ,
  util::xmsg::XRISEFALL_FMT_STNDRD_WITH_TEMP_AND_SALN = 2 ,
  util::xmsg::XRISEFALL_FMT_UNKNOWN = 0xFF }

  Packing format type enumeration for rise and fall data X-Messages.

11.33.1 Detailed Description

The util::xmsg::AbstractXFallRise class header file.

Version

0.6

Date

Jul 2, 2014 10:38:13 AM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.34 AbstractXFallRise.h

Go to the documentation of this file.

```
00001 /** The @ref util::xmsg::AbstractXFallRise class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jul 2, 2014 10:38:13 AM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef ABSTRACTXFALLRISE_H_
00024 #define ABSTRACTXFALLRISE_H_
00025
00026 /* Includes */
00027
00028 // local
00029 #include "XMessageIndexable.h"
00030
00031 /* Definitions */
00032
00033 namespace util {
00034 namespace xmsg {
00035
00036 /* Type Definitions: */
00037
00038 /** Packing format type enumeration for rise and fall data X-Messages.
00039  *
00040  */
00041 typedef enum XFallRiseDataPackFormat {
00042
00043     /** The legacy 4-byte packing format.
00044      *
00045      *  @note       This format is only used by instrument previous to X-Message format version
00046      *              2.0.
00047      *
00048      */
00049     XRISEFALL_FMT_LEGACY = 0,
00050
00051     /** The standard 5-byte packing format.
00052      *
00053      *  @note       This packing format was add in X-Message version 2.0.
00054      */
00055     XRISEFALL_FMT_STNDRD = 1,
00056
00057     /** The standard 9-byte packing format with temperature and salinity.
00058      *
00059      *  @note       This packing format was add in X-Message version 2.2.
00060      */
00061     XRISEFALL_FMT_STNDRD_WITH_TEMP_AND_SALN = 2,
00062
00063     /** Unknown, unset, or unspecified packing format.
00064      *
00065      */
00066     XRISEFALL_FMT_UNKNOWN = 0xFF
00067
00068 } XFallRiseDataPackFormat_t;
00069
00070 /* Class Definition: */
00071
00072 /** An abstract class representing a fall, rise, or surface pressure time-series data X-message.
00073  *
00074  */
00075 class AbstractXFallRise:
00076         public util::date::DateUtils,
00077         public XMessageIndexable {
00078
00079 private:
00080
00081     /** The packing format type identifier.
00082      *
00083      */
00084     XFallRiseDataPackFormat_t data_pack_format_;
00085
00086     /** The start time reference.
00087      *
00088      */
00089     ptime start_time_;
00090
00091     /** The raw counts of measured fall, rise, or surface time-series data.
00092      *
00093      */
00094     vector<XSensorTripletCodeCounts> sensor_data_;
00095
00096 protected:
00097
00098     /** Set the enumerated data packing format value for this message.
00099      *
00100      *  @param[in]  data_pack_format    The enumerated data packing format value for this message
00101      *                                  to set.
00102      */
00103     void set_data_pack_format(XFallRiseDataPackFormat_t data_pack_format) {
```

```
00104
00105         data_pack_format_ = data_pack_format;
00106
00107     }
00108
00109     /** Set the start time of this message.
00110      *
00111      *  @param[in]  start_time          The start time to set.
00112      */
00113     void set_start_time(const ptime& start_time) {
00114
00115         start_time_ = start_time;
00116
00117     }
00118
00119     /** Unpack the fall/rise data from the binary message.
00120      *
00121      *  @param[in]      data            The binary message to unpack.
00122      *
00123      *  @throw          util::log::MessageException if
00124      *                      - the message is of insufficient length,
00125      *                      - an invalid packing format is encountered, or
00126      *                      - the message terminator is in an invalid location.
00127      *
00128      *  @return         The raw counts of measured fall or rise time-series data.
00129      */
00130     vector<XSensorTripletCodeCounts> UnpackData(const vector<uint8_t>& data) {
00131
00132         try {
00133
00134             const uint8_t* d = CheckXMsgData(data, -1, 7, false);
00135
00136             // Clear previous sensor data, if any.
00137             sensor_data_.clear();
00138
00139             {   // Get the message packing format.
00140                 int fmt;
00141                 switch((fmt = (d[1] & 0xF0) » 4)) {
00142                 case XRISEFALL_FMT_LEGACY:
00143                 case XRISEFALL_FMT_STNDRD:
00144                 case XRISEFALL_FMT_STNDRD_WITH_TEMP_AND_SALN:
00145
00146                     set_data_pack_format(static_cast<XFallRiseDataPackFormat_t>(fmt));
00147                     break;
00148
00149                 default:
00150
00151                     EXC_THROW(
00152                             "Unsupported rise/fall/surface data packing format: " +
      lexical_cast<string>(fmt) +"!");
00153                     break;
00154                 }
00155             }
00156
00157             // Parse the message header.
00158             set_start_time(s2000(d, 3));                        // Get start time.
00159             set_xmsg_index(0x0F & d[0]);                        // Get the message index.
00160             set_xmsg_size_bytes(((d[1] & 0x0F) « 8) | d[2]);   // Get the message size.
00161
00162             // Check message size.
00163             if(xmsg_size_bytes() > data.size()) {
00164
00165                 EXC_THROW(
00166                         "Invalid XMessage size: message specifies at least " +
      lexical_cast<string>(xmsg_size_bytes()) + " bytes, but only " + lexical_cast<string>(xmsg_size_bytes()) +"
      bytes were received.");
00167
00168             } else if(data.size() > xmsg_size_bytes()) {
00169
00170                 lwrn() « "Extra bytes in XMessage: message specifies "
00171                          « xmsg_size_bytes()
00172                          « "bytes, but "
00173                          « data.size()
00174                          « " bytes were received.";
00175
00176             }
00177
00178             // Check for terminator.
00179             if(d[xmsg_size_bytes() - 1] != 0x3B) {
00180                 ostringstream os;
00181                 os « "Invalid XMessage: expected terminator ';' (0x3B) at byte "
00182                    « (xmsg_size_bytes() - 1)
00183                    « ", but received 0x"
00184                    « hex
00185                    « d[xmsg_size_bytes() - 1];
00186                 BOOST_THROW_EXCEPTION(log::MessageException(os));
00187             }
00188
00189             {   // Unpack the sensor data.
00190                 uint16_t sec;
00191                 int32_t pres_counts, temp_counts = XMSG_SENS_BAD, saln_counts = XMSG_SENS_BAD;
00192                 XPhaseCode_t phase_code = XPHASE_CODE_UNKNOWN;
00193
00194                 switch(data_pack_format()) {
00195                 case XRISEFALL_FMT_LEGACY:
00196
00197                     for(vector<uint8_t>::const_iterator it = data.begin() + 7; it < data.end() - 4;
00198                             ) {
00199                         sec = ((*it++) « 8) | (*it++);
00200                         pres_counts = ((*it++) « 8) | (*it++);
00201                         sensor_data_.push_back(
00202                                 XSensorTripletCodeCounts(
00203                                         sec,
00204                                         phase_code,
00205                                         pres_counts,
00206                                         temp_counts,
00207                                         saln_counts));
00208                     }
```

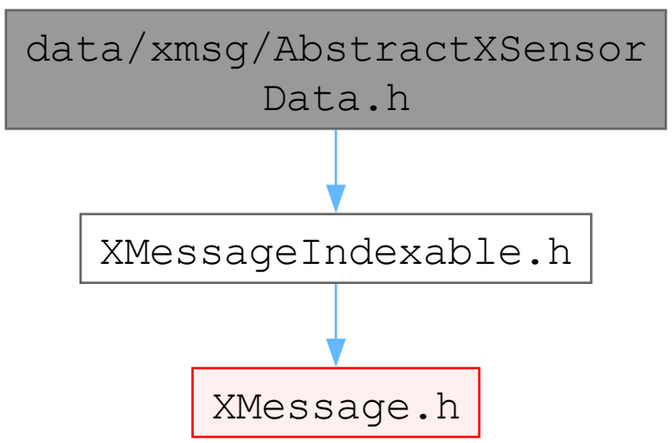
```
00209                     break;
00210
00211                 case XRISEFALL_FMT_STNDRD:
00212
00213                     for(vector<uint8_t>::const_iterator it = data.begin() + 7; it < data.end() - 5;
00214                             ) {
00215                         sec = ((*it++) << 8) | (*it++);
00216                         phase_code = (XPhaseCode_t) (*it >> 4);
00217                         pres_counts = ((*it++ & 0x0F) << 16) | ((*it++) << 8) | (*it++);
00218                         sensor_data_.push_back(
00219                                 XSensorTripletCodeCounts(
00220                                         sec,
00221                                         phase_code,
00222                                         pres_counts,
00223                                         temp_counts,
00224                                         saln_counts));
00225                     }
00226                     break;
00227
00228                 case XRISEFALL_FMT_STNDRD_WITH_TEMP_AND_SALN:
00229
00230                     for(vector<uint8_t>::const_iterator it = data.begin() + 7; it < data.end() - 9;
00231                             ) {
00232                         sec = ((*it++) << 8) | (*it++);
00233                         phase_code = (XPhaseCode_t) (*it >> 4);
00234                         pres_counts = ((*it++ & 0x0F) << 16) | ((*it++) << 8) | (*it++);
00235                         temp_counts = ((*it++) << 8) | (*it++);
00236                         saln_counts = ((*it++) << 8) | (*it++);
00237                         sensor_data_.push_back(
00238                                 XSensorTripletCodeCounts(
00239                                         sec,
00240                                         phase_code,
00241                                         pres_counts,
00242                                         temp_counts,
00243                                         saln_counts));
00244                     }
00245                     break;
00246
00247                 case XRISEFALL_FMT_UNKNOWN:
00248
00249                     EXC_THROW("Unknown rise/fall/surface data packing format!");
00250                     break;
00251                 }
00252
00253             }
00254
00255         }CATCH_BLOCK("Failed to unpack X-Message sensor data.")
00256
00257         return sensor_data_;
00258
00259     }
00260
00261 public:
00262
00263     /** The parameterized constructor.
00264      *
00265      *  @param[in]      dive        The dive or cycle number of this message.
00266      *  @param[in]      whoi        The unique WHOI identifier.
00267      *  @param[in]      data        Binary X-message data to decode.
00268      *
00269      *  @exception  None. Never throws.
00270      */
00271     AbstractXFallRise(int16_t dive, uint16_t whoi, const vector<uint8_t>& data) ATTR_NO_EXCEPTIONS :
00272                     XMessageIndexable(dive, whoi, data),
00273                     data_pack_format_(XRISEFALL_FMT_UNKNOWN),
00274                     start_time_(not_a_date_time) {
00275
00276         /* Do nothing. */
00277
00278     }
00279
00280     /** The destructor.
00281      *
00282      */
00283     virtual ~AbstractXFallRise() {
00284
00285         /* Do nothing. */
00286
00287     }
00288
00289     /** Get the enumerated data packing format type.
00290      *
00291      *  @return         The enumerated data packing format type.
00292      */
00293     XFallRiseDataPackFormat_t data_pack_format() const {
00294
00295         return data_pack_format_;
00296
00297     }
00298
00299     /** Decode the X message.
00300      *
00301      *  @param[in]      data        The encoded X message to decode.
00302      *
00303      *  @throw          util::log::MessageException if
00304      *                      - an invalid X-Message type identifier is encountered.
00305      *                      - the message is of insufficient length,
00306      *                      - an invalid packing format is encountered, or
00307      *                      - the message terminator is in an invalid location.
00308      */
00309     virtual void DecodeXMsgData(const vector<uint8_t>& data) = 0;
00310
00311     /** Get the listing entry comment string.
00312      *
00313      *  @return         The listing entry comment string.
00314      */
00315     static const string GenerateListingComment() {
00316
```

```
00317         ostringstream os;
00318 
00319         try {
00320 
00321 //             const format fmt("#   %5s %4s %3s %4s %20s %9s %7s %7s %9s   %s\n"
00322 //                     "#   %5s %4s %3s %4s %21s%10s%8s %8s%9s   %s");
00323 //             os << format(fmt)
00324 //             % "SN" % "dive" % "idx" % "nSam" % "time" % "pres" % "temp" % "saln" % "phaseCode" %
    "phaseCodeName"
00325 //             % "" % "" % "" % "" % "(start-time/+sec)" % "(dBar)" % "(degC)" % "(PSU)"  % "" % "";
00326 
00327           os << "#      SN dive idx nSam                 time      pres    temp    saln phaseCode
    phaseCodeName"
00328              << endl
00329              << "#                                    (UTC/+sec)    (dBar)  (degC)  (PSU)";
00330 
00331         }CATCH_BLOCK("Failed to generate fall/rise data listing comment!")
00332 
00333         return os.str();
00334     }
00335 
00336     /** Get the number of samples in this message.
00337      *
00338      * @return          The number of samples in this message.
00339      */
00340     size_t num_samples() const {
00341 
00342         return sensor_data_.size();
00343 
00344     }
00345 
00346     /** Get the vector of sensor data triplets.
00347      *
00348      *  @return     The vector of sensor data triplets.
00349      */
00350     const vector<XSensorTripletCodeCounts>& sensor_data() const {
00351 
00352         return sensor_data_;
00353 
00354     }
00355 
00356     /** Get the start time of this message.
00357      *
00358      *  @return         The start time of this message.
00359      */
00360     const ptime& start_time() const {
00361 
00362         return start_time_;
00363 
00364     }
00365 
00366 };
00367 
00368 } /* namespace xmsg */
00369 } /* namespace util */
00370 #endif /* ABSTRACTXFALLRISE_H_ */
```

11.35 data/xmsg/AbstractXSensorData.h File Reference

The util::xmsg::AbstractXSensorData class header file.

`#include "XMessageIndexable.h"`
Include dependency graph for AbstractXSensorData.h:



This graph shows which files directly or indirectly include this file:

Classes

- class util::xmsg::AbstractXSensorData

  The abstract sensor data class.

Namespaces

- namespace argo

  The Argo instrument namespace.
- namespace argo::iridium

  The Argo Iridium instrument namespace.
- namespace argo::iridium::xfloat

  The X-Float instrument namespace.
- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

Enumerations

- enum util::xmsg::XSensorDataFormat_t {
  util::xmsg::XSENS_FMT_DIFFERENCE = 0 ,
  util::xmsg::XSENS_FMT_CURVATURE = 1 ,
  XSENS_FMT_CURVATURE_AUX_TEMP_CHANNEL = 3 ,
  util::xmsg::XSENS_FMT_UNKNOWN = 0xFF }

  The X-Message sensor data packing format enumeration.

11.35.1 Detailed Description

The util::xmsg::AbstractXSensorData class header file.

This is the header file for the util::xmsg::AbstractXSensorData class. The abstract sensor data class provides the functionality to decode sensor data messages using either first-difference or curvature encoding. It is used to decode and represent the sensor data returned from the instrument that is contained within a single message.

The classes that inherit from this class represent profile or drift data taken from a single sensor e.g. pressure, temperature, or salinity. The sensor data is stored in a vector of raw integer counts, util::xmsg::AbstractXSensorData::raw_sensor_data_. The data from the sensor is recovered in scientific units by applying the appropriate offset and gain corrections, as specified in the Argo data message.

Version

0.6

Date

Jul 6, 2014 5:19:53 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.36 AbstractXSensorData.h

Go to the documentation of this file.

```
00001 /** The @ref util::xmsg::AbstractXSensorData class header file.
00002  *
00003  *  This is the header file for the @ref util::xmsg::AbstractXSensorData class. The abstract
00004  *  sensor data class provides the functionality to decode sensor data messages using either
00005  *  first-difference or curvature encoding. It is used to decode and represent the sensor data
00006  *  returned from the instrument that is contained within a single message.
00007  *
00008  *  The classes that inherit from this class represent profile or drift data taken from a single
00009  *  sensor e.g. pressure, temperature, or salinity. The sensor data is stored in a vector of raw
00010  *  integer counts, @ref util::xmsg::AbstractXSensorData::raw_sensor_data_. The data from the
00011  *  sensor is recovered in scientific units by applying the appropriate offset and gain
00012  *  corrections, as specified in the Argo data message.
00013  *
00014  *  @file
00015  *
00016  *  @version    0.6
00017  *
00018  *  @date       Jul 6, 2014 5:19:53 PM
00019  *
00020  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00021  *              Engineer I,
00022  *              Department of Physical Oceanography,
00023  *              Woods Hole Oceanographic Institution
00024  *
00025  *  @copyright  2014 Woods Hole Oceanographic Institution.
00026  *              Released under the GNU General Public License.
00027  *
00028  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00029  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00030  *  General Public License for more details. You should have received a copy of the GNU General
00031  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00032  */
00033
00034 #ifndef ABSTRACTXSENSORDATA_H_
00035 #define ABSTRACTXSENSORDATA_H_
00036
00037 /* Includes: */
00038
00039 // local
00040 #include "XMessageIndexable.h"
00041
00042 namespace argo {
00043   namespace iridium {
00044     namespace xfloat {
00045
00046 // forward declaration
00047       class XFloatDiveData;
00048
00049     } /* namespace xfloat */
00050   } /* namespace iridium */
00051 } /* namespace argo */
00052
00053 namespace util {
00054   namespace xmsg {
00055
00056     /* Type Definitions: */
00057
00058     /** The X-Message sensor data packing format enumeration.
00059      *
00060      *  This type enumerates the different packing format types that may be used by the class
00061      *  @ref AbstractXSensorData and the classes that inherit from it. The manner in which the sensor
00062      *  data is decoded is determined by the data packing formats enumerated here.
00063      */
00064     typedef enum XSensorDataFormat {
00065
00066       /** The X-Message sensor data packing format 0: Legacy first-difference packing format.
00067        *
00068        */
00069       XSENS_FMT_DIFFERENCE = 0,
00070
00071       /** The X-Message sensor data packing format 1: Standard curvature packing format.
00072        *
00073        *  @note           The packing format was added in X version 2.0.
00074        *
00075        */
00076       XSENS_FMT_CURVATURE = 1,
00077
00078       XSENS_FMT_CURVATURE_AUX_TEMP_CHANNEL = 3,
00079
00080       /** The X-Message sensor data packing format 255: Unknown packing format.
00081        *
00082        */
00083       XSENS_FMT_UNKNOWN = 0xFF
00084
00085     } XSensorDataFormat_t;
00086
00087     /* Class Definition: */
00088
00089     /** The abstract sensor data class.
00090      *
00091      *  The abstract sensor data class provides the functionality to decode sensor data messages using
00092      *  either first-difference or curvature encoding. It is used to decode and represent the sensor
00093      *  data returned from the instrument that is contained within a single message.
00094      *
00095      *  The classes that inherit from this class represent profile or drift data taken from a single
00096      *  sensor e.g. pressure, temperature, or salinity. The sensor data is stored in a vector of raw
00097      *  integer counts, @ref raw_sensor_data_. The data from the sensor is recovered in scientific
00098      *  units by applying the appropriate offset and gain corrections, as specified in the Argo data
00099      *  message.
00100      *
00101      */
00102     class AbstractXSensorData:
00103         public XMessageIndexable {
```

```
00104
00105         /* Friend Classes: */
00106
00107 #   if defined(XMSG_ENABLE_MULTI_PROF) && XMSG_ENABLE_MULTI_PROF
00108         friend class argo::iridium::xfloat::XFloatDiveData;
00109 #   endif // defined(XMSG_ENABLE_MULTI_PROF) && XMSG_ENABLE_MULTI_PROF
00110
00111       protected:
00112
00113         /* Protected Data Members: */
00114
00115         bool aux_channel;
00116
00117         /** The enumerated sensor data packing format.
00118          *
00119          *  This field indicates the format of encoded sensor data. The manner in which the sensor
00120          *  data is decoded is determined by the format of this field.
00121          *
00122          */
00123         XSensorDataFormat_t sensor_data_format_;
00124
00125         /** The raw integer counts of the measured sensor data.
00126          *
00127          *  This vector of integers represents the "raw" sample values of the message. That is, by
00128          *  applying the appropriate gain and offset corrections, the sensor values may be recovered
00129          *  in scientific units. This data may represent pressure, temperature, salinity, or any other
00130          *  data that is recorded during the drift or profile phases of the dive cycle.
00131          */
00132         vector<int32_t> raw_sensor_data_;
00133
00134       public:
00135
00136         /* Public Member Functions: */
00137
00138         /** Parameterized constructor.
00139          *
00140          *  @param[in]      dive        The dive or cycle number of this message.
00141          *  @param[in]      whoi        The unique WHOI identifier.
00142          *  @param[in]      data        Binary X-message data to decode.
00143          *
00144          *  @throw          util::log::MessageException if
00145          *                      - an invalid X-Message type identifier is encountered.
00146          *                      - the message is of insufficient length,
00147          *                      - an invalid packing format is encountered, or
00148          *                      - the message terminator is in an invalid location.
00149          */
00150         AbstractXSensorData(int16_t dive, uint16_t whoi, const vector<uint8_t>& data) :
00151                 XMessageIndexable(dive, whoi, data),
00152                 aux_channel(false),
00153                 sensor_data_format_(XSENS_FMT_UNKNOWN) {
00154           /* Do nothing. */
00155         }
00156
00157         /** Destructor.
00158          *
00159          */
00160         virtual ~AbstractXSensorData() {
00161           /* Do nothing. */
00162         }
00163
00164         /** Decode the X message.
00165          *
00166          *  @param[in]      data        The encoded X message to decode.
00167          */
00168         virtual void DecodeXMsgData(const vector<uint8_t>& data) = 0;
00169
00170         /** Get the number of samples in this message.
00171          *
00172          *  @return         The number of samples in this message.
00173          */
00174         size_t num_samples() const {
00175           return raw_sensor_data_.size();
00176         }
00177
00178         /** Get the raw sensor data vector.
00179          *
00180          *  @return         The raw sensor data vector.
00181          */
00182         const vector<int32_t>& raw_sensor_data() const {
00183           return raw_sensor_data_;
00184         }
00185
00186         /** Get the raw integer value of the @f$i^{th}@f$ sample.
00187          *
00188          *  @param[in]      i       The index of the sensor value.
00189          *
00190          *  @return         The raw integer value of the @f$i^{th}@f$ ith sample.
00191          */
00192         int32_t Sample(size_t i) const {
00193           return raw_sensor_data_.at(i);
00194         }
00195
00196         /** Get the @f$i^{th}@f$ sensor value in scientific units.
00197          *
00198          *  @param[in]      i       The index of the sensor value.
00199          *  @param[in]      gain    The sensor sensitivity scale factor.
00200          *  @param[in]      offs    The sensor offset.
00201          *
00202          *  @return         The @f$i^{th}@f$ sensor value in scientific units.
00203          */
00204         template<typename T = double> T SampleValue(size_t i, int16_t gain, int16_t offs) const {
00205           T val;
00206
00207           try {
00208             val = gain ? (T) raw_sensor_data_.at(i) / (T) gain - (T) offs : -100;
00209           }CATCH_BLOCK("Failed to get sensor data value from raw counts sample!")
00210
00211           return val;
```

```
00212         }
00213
00214         /** Get the @f$i^{th}@f$ pressure sensor value, in dBar.
00215          *
00216          *  @param[in]      i       The index of the sensor value.
00217          *  @param[in]      gain    The pressure sensor sensitivity scale factor, in LSB/dBar.
00218          *  @param[in]      offs    The pressure sensor offset, in dBar.
00219          *
00220          * @return          The @f$i^{th}@f$ pressure sensor value, in dBar.
00221          */
00222         template<typename T = double> T SampleValueAsPressure(
00223             size_t i,
00224             int16_t gain,
00225             int16_t offs) const {
00226           T val = SampleValue<T>(i, gain, offs);
00227
00228           if(val < XMSG_SENS_MIN_PRES || val > XMSG_SENS_MAX_PRES)
00229             return XMSG_SENS_BAD_PRES;
00230
00231           return val;
00232         }
00233
00234         /** Get the @f$i^{th}@f$ salinity sensor value, in PSU.
00235          *
00236          *  @param[in]      i       The index of the sensor value.
00237          *  @param[in]      gain    The salinity sensor sensitivity scale factor, in LSB/PSU.
00238          *  @param[in]      offs    The salinity sensor offset, in PSU.
00239          *
00240          * @return          The @f$i^{th}@f$ salinity sensor value, in PSU.
00241          */
00242         template<typename T = double> T SampleValueAsSalinity(
00243             size_t i,
00244             int16_t gain,
00245             int16_t offs) const {
00246           T val = SampleValue<T>(i, gain, offs);
00247
00248           if(val < XMSG_SENS_MIN_SALN || val > XMSG_SENS_MAX_SALN)
00249             return XMSG_SENS_BAD_SALN;
00250
00251           return val;
00252         }
00253
00254         /** Get the @f$i^{th}@f$ temperature sensor value, in degC.
00255          *
00256          *  @param[in]      i       The index of the sensor value.
00257          *  @param[in]      gain    The temperature sensor sensitivity scale factor, in LSB/degC.
00258          *  @param[in]      offs    The temperature sensor offset, in degC.
00259          *
00260          * @return          The @f$i^{th}@f$ temperature sensor value, in degC.
00261          */
00262         template<typename T = double> T SampleValueAsTemperature(
00263             size_t i,
00264             int16_t gain,
00265             int16_t offs) const {
00266           T val = SampleValue<T>(i, gain, offs);
00267
00268           if(val < XMSG_SENS_MIN_TEMP || val > XMSG_SENS_MAX_TEMP)
00269             return XMSG_SENS_BAD_TEMP;
00270
00271           return val;
00272         }
00273
00274         /** Get the enumerated sensor data format value.
00275          *
00276          *  @return         The enumerated sensor data format value.
00277          */
00278         XSensorDataFormat_t sensor_data_format() const {
00279           return sensor_data_format_;
00280         }
00281
00282         bool is_aux_channel() const {
00283           return aux_channel;
00284         }
00285
00286       protected:
00287
00288         /* Protected Member Functions: */
00289
00290         /** Default constructor.
00291          *
00292          *  Constructs an empty sensor data message with an unknown sensor data format.
00293          */
00294         AbstractXSensorData() :
00295                 XMessageIndexable(),
00296                 aux_channel(false),
00297                 sensor_data_format_(XSENS_FMT_UNKNOWN) {
00298           /* Do nothing. */
00299         }
00300
00301         /** Set raw integer counts of the measured sensor data.
00302          *
00303          * @param[in]   raw_sensor_data     The data to set.
00304          */
00305         void set_raw_sensor_data(const vector<int32_t>& raw_sensor_data) {
00306           raw_sensor_data_ = raw_sensor_data;
00307         }
00308
00309         /** Set the sensor data format
00310          *
00311          *  @param[in]  sensor_data_format  The sensor data format to set.
00312          */
00313         void set_sensor_data_format(XSensorDataFormat_t sensor_data_format) {
00314           sensor_data_format_ = sensor_data_format;
00315         }
00316
00317         /** Unpack the sensor data from the binary message.
00318          *
00319          *  This field unpacks the encoded sensor data from the message and stores the decoded sensor
```

```
00320          *  values in @ref raw_sensor_data_. First the function checks the size of the message, and
00321          *  the data packing format. The message data is decoded according to the packing format type.
00322          *  Both first-difference and curvature encoding are supported. The encoding method is
00323          *  determined from the value of the high nibble of byte @c data[1].
00324          *
00325          *  Once this function completes successfully, then the data may be recovered in scientific
00326          *  units from the raw sensor values by applying the appropriate sensor gain and offset values.
00327          *
00328          *  @param[in]      data        The binary message to unpack, in a vector of unsigned bytes.
00329          *
00330          *  @warning        The function assumes the message data is valid. Message validity should
00331          *                  be ensured before this function is called.
00332          *
00333          *  @throw          util::log::MessageException if
00334          *                      - an error occurs while decoding the message,
00335          *                      - the message is of insufficient length, or
00336          *                      - an invalid packing format is encountered.
00337          *
00338          *  @return         The sensor data raw counts, in a vector of signed integers.
00339          */
00340         virtual vector<int32_t> UnpackSensorData(const vector<uint8_t>& data) {
00341           try {
00342             // Check message length
00343             if(data.size() < 5) {
00344               ostringstream os;
00345               os « "Message too short! Expected at least 5 bytes, but only received "
00346                  « data.size()
00347                  « " bytes!";
00348             }
00349
00350             // Set the message type & index.
00351             set_id_xmsg_type(static_cast<XMsgTypeId_t>(data.at(0)));
00352             set_xmsg_index(0x0F & data.at(0));
00353
00354             // Clear previous, if any.
00355             raw_sensor_data_.clear();
00356
00357             /* Do Version-Specific Decoding: */
00358
00359             switch((data.at(1) & 0xF0) » 4) {
00360
00361               case XSENS_FMT_DIFFERENCE:
00362
00363                 /* Handle Fist-Difference Decoding: */
00364
00365                 try {
00366
00367                   ldbg() « "Unpacking first-difference-encoded sensor data..." « endl;
00368
00369                   uint32_t scale;
00370                   int32_t x0;
00371
00372                   // This message uses the first-difference packing method.
00373                   set_sensor_data_format(XSENS_FMT_DIFFERENCE);
00374
00375                   // Iterate over the sensor data.
00376                   for(vector<uint8_t>::const_iterator it = data.begin() + 3; it < data.end() - 5;) {
00377
00378                     // Parse sub-block header.
00379                     scale = *it++;
00380                     raw_sensor_data_.push_back(x0 = (*it++ « 8) | *it++);
00381                     size_t isub = XMSG_SENS_MAX_SUB;
00382
00383                     // Iterate over sub-block data.
00384                     while(it < data.end() - 1 && --isub) {
00385                       int32_t dx = *it++;
00386                       if(dx > 0x7F)
00387                         dx -= 0x0100;
00388                       raw_sensor_data_.push_back(x0 += dx * scale);
00389                     }
00390                   }
00391
00392                 }CATCH_BLOCK("Failed to unpack first-difference data!")
00393
00394                 break;
00395
00396               case XSENS_FMT_CURVATURE_AUX_TEMP_CHANNEL:
00397
00398                 aux_channel = true;
00399
00400                 /* No break. */
00401
00402               case XSENS_FMT_CURVATURE:
00403
00404                 /* Handle Curvature Decoding: */
00405
00406                 try {
00407
00408                   ldbg() « "Unpacking curvature-encoded sensor data..." « endl;
00409
00410                   /*  The data first differences.
00411                    *
00412                    *  The first difference are used to reconstruct the raw sensor data, and
00413                    *  are themselves constructed from the data second differences.
00414                    *
00415                    */
00416                   vector<int32_t> data_1st_differences;
00417
00418                   /*  The data second differences.
00419                    *
00420                    *  The encoded data represents the second differences of the raw sensor data.
00421                    *  The second differences are used to reconstruct the first differences,
00422                    *  which, in turn, are used to reconstruct the raw sensor data.
00423                    *
00424                    */
00425                   vector<int32_t> data_2nd_differences;
00426
00427                   /*  The message data iterator.
```

```
00428                  *
00429                  *  This field is used for iterating over the values of the sub-blocks of the
00430                  *  message. One nibble is read at a time, so this iterator is iterated for
00431                  *  every other nibble that is read.
00432                  */
00433                 vector<uint8_t>::const_iterator data_iterator;
00434
00435                 /*  The 32 3-bit packing factors.
00436                  *
00437                  *  The packing factor represents the dynamic range for each sub-block. The
00438                  *  packing factor represents the number of nibbles that is needed to store
00439                  *  the value.
00440                  */
00441                 uint8_t data_packing_factors[32];
00442                 memset(data_packing_factors, 0, sizeof(data_packing_factors));
00443
00444                 /*  The number of remaining values in the message.
00445                  *
00446                  *  This field is used as a control variable to determine when data unpacking
00447                  *  is complete.
00448                  *
00449                  */
00450                 size_t num_remaining_values = 0;
00451
00452                 /*  The total number of values in this message.
00453                  *
00454                  *  This field is parsed from the message header. It is used as a control
00455                  *  variable do determine when message unpacking is complete.
00456                  *
00457                  */
00458                 size_t num_values = 0;
00459
00460                 /*  The index of the sub block.
00461                  *
00462                  *  This field is used to track the index of the sub-block during iteration
00463                  *  of the message data. It is used as an index to the data_packing_factors[]
00464                  *  array.
00465                  */
00466                 size_t sub_block_index = 0;
00467
00468                 /*  The sub-block-nibble-is-high flag.
00469                  *
00470                  *  This field is used to keep track of whether we are reading the high or low
00471                  *  nibble of the data.
00472                  */
00473                 bool sub_block_nibble_is_high = true;
00474
00475                 /* The number of values in a sub-block.
00476                  *
00477                  * This field is used for iteration over the values in a sub-block.
00478                  *
00479                  */
00480                 size_t sub_block_num_values = 0;
00481
00482                 // This message uses the second-difference (curvature) packing method.
00483                 set_sensor_data_format(XSENS_FMT_CURVATURE);
00484
00485                 /* Parse the message header: */
00486
00487                 // Get the first sensor value.
00488                 const int32_t first_sensor_value = i24(data, 6);
00489
00490                 // Read the first difference value.
00491                 data_1st_differences.push_back(i24(data, 9));
00492
00493                 // Determine if the first sensor value should be included in the data.
00494                 switch(id_xmsg_type()) {
00495                   case XMSG_TYPE_ID_PRES0:            // 0x10
00496                   case XMSG_TYPE_ID_TEMP0:            // 0x20
00497                   case XMSG_TYPE_ID_SALN0:            // 0x30
00498                   case XMSG_TYPE_ID_PRES_HI_RES0:     // 0x90
00499                   case XMSG_TYPE_ID_TEMP_HI_RES0:     // 0xA0
00500                   case XMSG_TYPE_ID_SALN_HI_RES0:     // 0xB0
00501                   case XMSG_TYPE_ID_PRES_DRIFT8:      // 0x98
00502                   case XMSG_TYPE_ID_TEMP_DRIFT8:      // 0xA8
00503                   case XMSG_TYPE_ID_SALN_DRIFT8:      // 0xB8
00504                     raw_sensor_data_.push_back(first_sensor_value);
00505
00506                     break;
00507
00508                   default:
00509
00510                     /* Do nothing. */
00511
00512                     break;
00513                 }
00514
00515                 // Get the number of values.
00516                 switch(num_values = u16(data, 4)) {
00517                   case 0:
00518
00519                     /* Handle messages containing zero values: */
00520
00521                     // Throw exception.
00522                     EXC_THROW("Message contains 0 sensor values!");
00523                     break;
00524
00525                   case 2: {
00526
00527                     /* Handle messages containing two values: */
00528
00529                     // Compute the second sensor value from the first difference.
00530                     raw_sensor_data_.push_back(first_sensor_value + data_1st_differences.at(0));
00531                   }
00532                     /* No break. */
00533
00534                   case 1:
00535
```

```
00536                       /* Handle messages containing one value: */
00537
00538                       // We have all the data for this message, so return.
00539                       return raw_sensor_data_;
00540
00541                     default:
00542
00543                       /* Handle messages containing three or more values: */
00544
00545                       raw_sensor_data_.reserve(num_values);
00546                       data_1st_differences.reserve(num_values);
00547                       data_2nd_differences.reserve(num_values);
00548                       break;
00549                   }
00550
00551                   /* Get the 32 3-bit packing factors from the 12-byte field: */
00552
00553                   // Iterate over the four 3-byte blocks.
00554                   for(size_t i = 0; i < 4; ++i)
00555
00556                     // Iterate over the eight values in each 3-byte block.
00557                     for(size_t j = 0; j < 8; ++j)
00558
00559                       // Parse the packing factor.
00560                       data_packing_factors[i * 8 + j] = (u24(data, 12 + i*3) » (3 * (7 - j)))
00561                           & 0x07;
00562
00563 #                   if defined(XMSG_DEBUG_SENSOR_CURVATURE) && XMSG_DEBUG_SENSOR_CURVATURE
00564                   {
00565                     ostringstream os1, os2;
00566
00567                     os1 « "Decoding curvature data packing factors..."
00568                     « endl
00569                     « " Curvature Data Packing Factors:"
00570                     « endl
00571                     « "    ";
00572
00573                     static const format fmt1(" p[%2d]"), fmt2(" %5d");
00574
00575                     for(size_t i = 0; i < 32; ++i) {
00576                       os1 « format(fmt1) % i;
00577                       os2 « format(fmt2) % (unsigned int) data_packing_factors[i];
00578                     }
00579
00580                     ldbg() « os1.str() « endl « "    " « os2.str() « endl;
00581                   }
00582 #                   endif
00583
00584                   /* Unpack the message data: */
00585
00586 #                   if defined(XMSG_DEBUG_SENSOR_CURVATURE) && XMSG_DEBUG_SENSOR_CURVATURE
00587                   ostringstream xmsg_debug_os_;
00588                   xmsg_debug_os_ « "Decoding curvature data..." « endl;
00589 #                   endif
00590
00591                   // Iterate over the sub-blocks.
00592                   data_iterator = data.begin() + 24;
00593                   num_remaining_values = num_values - 2;
00594                   while(num_remaining_values > 0 && data_iterator < data.end() - 1) {
00595
00596                     try {
00597
00598                       // Get the data packing factor for this sub-block.
00599                       uint8_t data_packing_factor = data_packing_factors[sub_block_index];
00600
00601                       // Skip sub-blocks with data packing factor = 0.
00602                       if(!data_packing_factor) {
00603                         sub_block_index++;
00604                         continue;
00605                       }
00606
00607                       // Get the number of values in this sub-block.
00608                       sub_block_num_values = min(16, (int) num_remaining_values);
00609
00610 #                         if defined(XMSG_DEBUG_SENSOR_CURVATURE) && XMSG_DEBUG_SENSOR_CURVATURE
00611                       {
00612                         static const format fmt1(
00613                             " sub-block %2d (p: %u) %4d values remaining");
00614                         static const format fmt2(" %8d");
00615                         xmsg_debug_os_ « format(fmt1) % sub_block_index
00616                         % (unsigned int) data_packing_factors[sub_block_index]
00617                         % num_remaining_values
00618                         « endl
00619                         « "      ";
00620                         for(size_t i = 0; i < 16; ++i)
00621                         xmsg_debug_os_ « format(fmt2) % i;
00622                         xmsg_debug_os_ « endl « "    0x";
00623                       }
00624 #                         endif
00625
00626                       // Iterate over the values in this sub-block.
00627                       for(size_t sub_block_value_idx = 0;
00628                           sub_block_value_idx < sub_block_num_values
00629                             && data_iterator < data.end() - 1; ++sub_block_value_idx) {
00630                         try {
00631
00632 #                           if defined(XMSG_DEBUG_SENSOR_CURVATURE) && XMSG_DEBUG_SENSOR_CURVATURE
00633                         ostringstream xmsg_debug_nibbles_os_;
00634 #                           endif
00635
00636                         // Iterate over the nibbles of the value.
00637                         int32_t second_difference = 0;
00638
00639                         for(uint8_t value_nibble_index = 0;
00640                             value_nibble_index < data_packing_factor && data_iterator < data.end();
00641                             ++value_nibble_index) {
00642
00643                           // Read the nibble. If this is a low nibble, increment the iterator.
```

```
00644                             uint8_t byte = (*data_iterator);
00645                             uint8_t nibble = 0;
00646                             if(sub_block_nibble_is_high) {
00647                               nibble = byte » 4;
00648                             } else {
00649                               nibble = byte & 0x0F;
00650                               data_iterator++;
00651                             }
00652                             second_difference = (second_difference « 4) | nibble;
00653 
00654                             // Update the nibble-is-high value.
00655                             sub_block_nibble_is_high = !sub_block_nibble_is_high;
00656 
00657 #                                 if defined(XMSG_DEBUG_SENSOR_CURVATURE) &&
    XMSG_DEBUG_SENSOR_CURVATURE
00658                             {
00659                               static const format fmt("%X");
00660                               xmsg_debug_nibbles_os_ « format(fmt)
00661                               % (unsigned int) nibble;
00662                             }
00663 #                                 endif
00664 
00665                           }
00666 
00667                           // Correct for signed second differences.
00668                           if(second_difference >= (1 « (4 * data_packing_factor - 1)))
00669                             second_difference -= (1 « (4 * data_packing_factor));
00670 
00671 #                                 if defined(XMSG_DEBUG_SENSOR_CURVATURE) && XMSG_DEBUG_SENSOR_CURVATURE
00672                           {
00673                             static const format fmt(" %8s");
00674                             xmsg_debug_os_ « format(fmt)
00675                             % xmsg_debug_nibbles_os_.str();
00676                           }
00677 #                                 endif
00678 
00679                           // Append the computed second difference.
00680                           data_2nd_differences.push_back(second_difference);
00681 
00682                           // Update the number of remaining values.
00683                           num_remaining_values--;
00684 
00685                         }CATCH_BLOCK(
00686                             string("Failed to decoded the sensor value in sub-block: ")
00687                                 + lexical_cast<string>(sub_block_value_idx) + "!")
00688                       }
00689 
00690 #                           if defined(XMSG_DEBUG_SENSOR_CURVATURE) && XMSG_DEBUG_SENSOR_CURVATURE
00691                       {
00692                         xmsg_debug_os_ « endl;
00693                       }
00694 #                           endif
00695 
00696                       // Update the sub-block index.
00697                       sub_block_index++;
00698 
00699                     }CATCH_BLOCK(
00700                         string("Failed to decode sub-block: ")
00701                             + lexical_cast<string>(sub_block_index) + "!")
00702 
00703                   }
00704 
00705                   /* Compute the sensor data values: */
00706 
00707                   // Compute the first differences.
00708                   try {
00709                     int32_t first_difference = data_1st_differences.at(0);
00710                     for_each(
00711                         data_2nd_differences.begin(),
00712                         data_2nd_differences.end(),
00713                         [&](const int32_t second_difference) {
00714                           data_1st_differences.push_back((first_difference = first_difference +
    second_difference));
00715                         });
00716 
00717                   }CATCH_BLOCK("Failed to compute the sensor data first differences!")
00718 
00719                   try {
00720                     // Get the first sensor value.
00721                     int32_t sensor_value = first_sensor_value;
00722 
00723                     // Compute the sensor values.
00724                     for_each(
00725                         data_1st_differences.begin(),
00726                         data_1st_differences.end(),
00727                         [&](const int32_t first_difference) {
00728                           raw_sensor_data_.push_back((sensor_value = sensor_value + first_difference));
00729                         });
00730                   }CATCH_BLOCK("Failed to compute the sensor data values!")
00731 
00732                   /* --- Start of XMSG_DEBUG_SENSOR_CURVATURE Block
    ------------------------------------------------ */
00733 #                   if defined(XMSG_DEBUG_SENSOR_CURVATURE) && XMSG_DEBUG_SENSOR_CURVATURE
00734                   try {
00735 
00736                     static const format fmt1("%20s %20s %20s %20s");
00737                     static const format fmt2("%20d %20d %20d %20d");
00738                     xmsg_debug_os_ « format(fmt1) % "idx" % "2nd" % "1st" % "value" « endl;
00739                     for(size_t i = 0; i < data_2nd_differences.size(); i++) {
00740                       xmsg_debug_os_ « format(fmt2) % i % data_2nd_differences.at(i)
00741                       % data_1st_differences.at(i)
00742                       % raw_sensor_data_.at(i)
00743                       « endl;
00744                     }
00745                     ldbg() « xmsg_debug_os_.str();
00746                   }CATCH_NOTHROW("Throw in XMSG_DEBUG_SENSOR_CURVATURE block!")
00747 #                   endif
00748                   /* --- End of XMSG_DEBUG_SENSOR_CURVATURE Block
```

```
    ------------------------------------------------ */
00749
00750                   // Throw exception if we do not have all the variables.
00751                   if(num_remaining_values) {
00752                     ostringstream os;
00753                     os << "Reached end of message data before reading all values! "
00754                        << num_remaining_values
00755                        << " of "
00756                        << num_values
00757                        << " values remaining.";
00758                     lwrn() << os.str();
00759 //                    EXC_THROW(os.str());
00760                   }
00761
00762                 }CATCH_BLOCK("Failed to unpack curvature data!")
00763
00764                 break;
00765
00766               default:
00767
00768                 /* Handle Unsupported Packing Formats: */
00769
00770                 // Throw exception
00771                 EXC_THROW("Unsupported packing format!");
00772                 break;
00773             }
00774
00775           }CATCH_BLOCK("Failed to unpack X-Message sensor data.")
00776
00777           return raw_sensor_data_;
00778         }
00779
00780       private:
00781
00782         /* Private Member Functions: */
00783
00784     };
00785
00786   } /* namespace xmsg */
00787 } /* namespace util */
00788 #endif /* ABSTRACTXSENSORDATA_H_ */
```

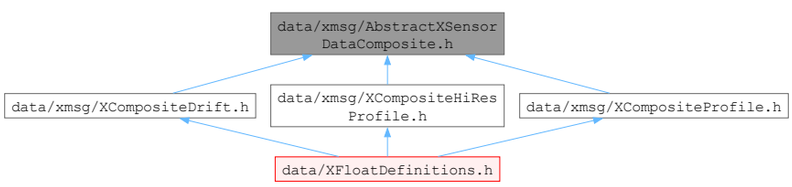
11.37 data/xmsg/AbstractXSensorDataComposite.h File Reference

The util::xmsg::AbstractXSensorDataComposite class header file.

```
#include "XDriftPressure.h"
#include "XDriftSalinity.h"
#include "XDriftTemperature.h"
#include "XHiResPressure.h"
#include "XHiResSalinity.h"
#include "XHiResTemperature.h"
#include "XProfilePressure.h"
#include "XProfileSalinity.h"
#include "XProfileTemperature.h"
```

Include dependency graph for AbstractXSensorDataComposite.h:

This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >

  The AbstractXSensorDataComposite class.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.37.1 Detailed Description

The util::xmsg::AbstractXSensorDataComposite class header file.

Version

0.6

Date

Jul 10, 2015 3:10:15 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.38 AbstractXSensorDataComposite.h

Go to the documentation of this file.

```
00001 /** The @ref util::xmsg::AbstractXSensorDataComposite class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jul 10, 2015 3:10:15 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef ABSTRACTXSENSORDATACOMPOSITE_H_
00024 #define ABSTRACTXSENSORDATACOMPOSITE_H_
00025
00026 /* Includes: */
00027
00028 // local
00029 #include "XDriftPressure.h"
00030 #include "XDriftSalinity.h"
00031 #include "XDriftTemperature.h"
00032 #include "XHiResPressure.h"
00033 #include "XHiResSalinity.h"
00034 #include "XHiResTemperature.h"
00035 #include "XProfilePressure.h"
00036 #include "XProfileSalinity.h"
00037 #include "XProfileTemperature.h"
00038
00039 /* Definitions: */
00040
00041 namespace util {
00042 namespace xmsg {
```

```
00043
00044 /* Types: */
00045
00046 /** The AbstractXSensorDataComposite class.
00047  *
00048  *  This abstract, template class represents the composite sensor data for a single dive cycle. It
00049  *  provided the functionality for aggregating sensor data messages into a single object.
00050  *
00051  */
00052 template<class PRES_CLASS, class SALN_CLASS, class TEMP_CLASS, unsigned int MAX_MSGS>
00053 class AbstractXSensorDataComposite:
00054         public thread::RecursiveLockable {
00055
00056     /* Type Definitions: */
00057
00058     /** Type definition for a shared pointer to the pressure data message.
00059      *
00060      */
00061     typedef shared_ptr<PRES_CLASS> PresMsgPtr_t;
00062
00063     /** Type definition for a shared pointer to the salinity data message.
00064      *
00065      */
00066     typedef shared_ptr<SALN_CLASS> SalnMsgPtr_t;
00067
00068     /** Type definition for a shared pointer to the temperature data message.
00069      *
00070      */
00071     typedef shared_ptr<TEMP_CLASS> TempMsgPtr_t;
00072
00073     /** Type definition for a static vector (of size MAX_MSGS) of shared pointers to the pressure
00074      *  data message.
00075      *
00076      */
00077     typedef static_vector<PresMsgPtr_t, MAX_MSGS> PresMsgVector_t;
00078
00079     /** Type definition for a static vector (of size MAX_MSGS) of shared pointers to the salinity
00080      *  data message.
00081      *
00082      */
00083     typedef static_vector<SalnMsgPtr_t, MAX_MSGS> SalnMsgVector_t;
00084
00085     /** Type definition for a static vector (of size MAX_MSGS) of shared pointers to the
00086      *  temperature data message.
00087      *
00088      */
00089     typedef static_vector<TempMsgPtr_t, MAX_MSGS> TempMsgVector_t;
00090
00091 private:
00092
00093     /* Data Members: */
00094
00095     /** The aggregated data is valid flag.
00096      *
00097      *  This flag indicates the status of the aggregated sensor data. It is used to ensure the
00098      *  composite sensor data is aggregated only once following each assignment of sensor data to
00099      *  the composite data message.
00100      *
00101      *  This field is set to FALSE during initialization, and is set to FALSE each time a sensor
00102      *  data message is assigned to the composite data message. Each time @ref AggregateSensorData()
00103      *  is called, the data will be aggregated, and this field will be set to TRUE.
00104      *
00105      */
00106     bool aggregated_data_valid_;
00107
00108     /** A vector of the aggregated pressure data samples, in raw counts.
00109      *
00110      *  Each time the data is aggregated, the sensor data samples from the sensor data messages
00111      *  assigned to this composite objects are stored in this vector.
00112      *
00113      */
00114     vector<int32_t> aggregated_pres_data_;
00115
00116     /** A vector of the aggregated salinity data samples, in raw counts.
00117      *
00118      *  Each time the data is aggregated, the sensor data samples from the sensor data messages
00119      *  assigned to this composite objects are stored in this vector.
00120      *
00121      */
00122     vector<int32_t> aggregated_saln_data_;
00123
00124     /** A vector of the aggregated temperature data samples, in raw counts.
00125      *
00126      *  Each time the data is aggregated, the sensor data samples from the sensor data messages
00127      *  assigned to this composite object are stored in this vector.
00128      *
00129      */
00130     vector<int32_t> aggregated_temp_data_;
00131
00132     vector<int32_t> aggregated_aux_temp_data_;
00133
00134     /** A static vector (of size MAX_MSGS) containing pointers the pressure sensor data messages
00135      *  assigned to this composite object.
00136      *
00137      *  Each time a pressure sensor data message is assigned to the this object, it will be
00138      *  emplaced in this vector in order of increasing message indices.
00139      *
00140      */
00141     PresMsgVector_t pres_messages_;
00142
00143     /** A static vector (of size MAX_MSGS) containing pointers the salinity sensor data messages
00144      *  assigned to this composite object.
00145      *
00146      *  Each time a salinity sensor data message is assigned to the this object, it will be
00147      *  emplaced in this vector in order of increasing message indices.
00148      *
00149      */
00150     SalnMsgVector_t saln_messages_;
```

```
00151
00152     /** A static vector (of size MAX_MSGS) containing pointers the temperature sensor data messages
00153      *  assigned to this composite object.
00154      *
00155      *  Each time a temperature sensor data message is assigned to the this object, it will be
00156      *  emplaced in this vector in order of increasing message indices.
00157      *
00158      */
00159     TempMsgVector_t temp_messages_;
00160
00161     TempMsgVector_t aux_temp_messages_;
00162
00163 protected:
00164
00165     /* Protected Member Functions: */
00166
00167     /** Aggregate the sensor data for this composite sensor data object.
00168      *
00169      *  This function iterates over each of the sensor data messages assigned to this composite
00170      *  data object, and copies of the sensor data values into the corresponding sensor data
00171      *  vector.
00172      *
00173      *  If @ref aggregated_data_valid_ is TRUE, then this function will have no effect. Otherwise,
00174      *  each of the aggregegated sensor data vectors will be cleared and the appropriate number of
00175      *  samples will be reserved.
00176      *
00177      *  Then, the sensor data message will be iterated over in order of increasing message indices,
00178      *  and the sensor data values will be copied into the corresponding sensor data vector in the
00179      *  appropriate order.
00180      *
00181      *  Finally, @ref aggregated_data_valid_ will be set to FALSE to ensure that subsequent calls
00182      *  to this function will have no effect, until another sensor data message object is assigned
00183      *  to this composite object and @ref aggregated_data_valid_ is set to TRUE.
00184      *
00185      *  @exception  util::log::MessageException if an error occurs while aggregating the sensor
00186      *              data.
00187      *
00188      *  @note       This object will be locked for the scope of the execution of this function.
00189      *              This function places a lock guard on the mutual exclusion (mutex) lock,
00190      *              thereby locking the float object and preventing access or modification of the
00191      *              object by any other thread. Any thread that does attempt to access or modify
00192      *              this object will block until the lock is released.
00193      *
00194      */
00195     void AggregateSensorData() ATTR_NO_EXCEPTIONS {
00196
00197         try {
00198
00199             // Get an exclusive lock.
00200             lock_guard<recursive_mutex> guard(mutex_);
00201
00202             // Check if sensor data is already aggregated.
00203             if(is_aggregated_data_valid())
00204                 return;
00205
00206             // Clear existing sensor data, if any.
00207             aggregated_pres_data_.clear();
00208             aggregated_saln_data_.clear();
00209             aggregated_temp_data_.clear();
00210             aggregated_aux_temp_data_.clear();
00211
00212             // Reallocate storage if necessary.
00213             aggregated_pres_data_.reserve(num_pres_samples());
00214             aggregated_saln_data_.reserve(num_saln_samples());
00215             aggregated_temp_data_.reserve(num_temp_samples());
00216             aggregated_aux_temp_data_.reserve(num_aux_temp_samples());
00217
00218             // Aggregate the data.
00219             for_each(
00220                     pres_messages_.begin(),
00221                     pres_messages_.end(),
00222                     [&](const shared_ptr<PRES_CLASS>& msg) {
00223                         if(msg)
00224                         aggregated_pres_data_.insert(
00225                                 aggregated_pres_data_.end(),
00226                                 msg->raw_sensor_data().begin(),
00227                                 msg->raw_sensor_data().end());
00228                     });
00229             for_each(
00230                     saln_messages_.begin(),
00231                     saln_messages_.end(),
00232                     [&](const shared_ptr<SALN_CLASS>& msg) {
00233                         if(msg)
00234                         aggregated_saln_data_.insert(
00235                                 aggregated_saln_data_.end(),
00236                                 msg->raw_sensor_data().begin(),
00237                                 msg->raw_sensor_data().end());
00238                     });
00239             for_each(
00240                     temp_messages_.begin(),
00241                     temp_messages_.end(),
00242                     [&](const shared_ptr<TEMP_CLASS>& msg) {
00243                         if(msg)
00244                         aggregated_temp_data_.insert(
00245                                 aggregated_temp_data_.end(),
00246                                 msg->raw_sensor_data().begin(),
00247                                 msg->raw_sensor_data().end());
00248                     });
00249             for_each(
00250                     aux_temp_messages_.begin(),
00251                     aux_temp_messages_.end(),
00252                     [&](const shared_ptr<TEMP_CLASS>& msg) {
00253                         if(msg)
00254                         aggregated_aux_temp_data_.insert(
00255                             aggregated_aux_temp_data_.end(),
00256                                 msg->raw_sensor_data().begin(),
00257                                 msg->raw_sensor_data().end());
00258                     });
```

```
00259             set_aggregated_data_valid(true);
00260
00261         }CATCH_BLOCK("Failed to aggregate profile data!")
00262
00263     }
00264
00265     /** Set the value of the @ref aggregated_data_valid_ field.
00266      *
00267      *  @param[in]  valid   TRUE if the aggregated data is valid, or FALSE otherwise.
00268      *
00269      *  @note       This object will be locked for the scope of the execution of this function.
00270      *              This function places a lock guard on the mutual exclusion (mutex) lock,
00271      *              thereby locking the float object and preventing access or modification of the
00272      *              object by any other thread. Any thread that does attempt to access or modify
00273      *              this object will block until the lock is released.
00274      */
00275     void set_aggregated_data_valid(bool valid) {
00276         // Get an exclusive lock.
00277         lock_guard<recursive_mutex> guard(mutex_);
00278
00279         aggregated_data_valid_ = valid;
00280     }
00281
00282 public:
00283
00284     /* Public Data Members: */
00285
00286     /** The constructor.
00287      *
00288      *  The field @ref aggregated_data_valid_ initialized to FALSE, and the aggregated sensor
00289      *  data vectors reserve @ref XMSG_INIT_SAM_N_RESERVE samples.
00290      *
00291      */
00292     AbstractXSensorDataComposite() :
00293                     aggregated_data_valid_(false) {
00294
00295         // Pre-allocate memory for aggregated data.
00296         aggregated_pres_data_.reserve(XMSG_INIT_SAM_N_RESERVE);
00297         aggregated_saln_data_.reserve(XMSG_INIT_SAM_N_RESERVE);
00298         aggregated_temp_data_.reserve(XMSG_INIT_SAM_N_RESERVE);
00299
00300     }
00301
00302     /** The destructor.
00303      *
00304      */
00305     virtual ~AbstractXSensorDataComposite() {
00306         /* Do nothing. */
00307     }
00308
00309     /** Assign a pressure data message to this composite data object.
00310      *
00311      *  The pressure data message will be emplaced into the sensor data message vector in order of
00312      *  increasing message indices. If a message with the same index is as @p x is already contained
00313      *  in the vector, then an exception will be thrown.
00314      *
00315      *  @param[in]  x   A shared pointer to the pressure data message.
00316      *
00317      *  @exception  util::log::MessageException if @p x is a null pointer, a duplicate message index
00318      *              is encountered, or any other error occurs.
00319      *
00320      *  @note       This object will be locked for the scope of the execution of this function.
00321      *              This function places a lock guard on the mutual exclusion (mutex) lock,
00322      *              thereby locking the float object and preventing access or modification of the
00323      *              object by any other thread. Any thread that does attempt to access or modify
00324      *              this object will block until the lock is released.
00325      */
00326     void AssignPressureMessage(const PresMsgPtr_t& x) {
00327         typename PresMsgVector_t::iterator it;
00328         try {
00329
00330             ldbg() « "Assigning pressure data message to composite sensor data object." « endl;
00331
00332             // Get an exclusive lock.
00333             lock_guard<recursive_mutex> guard(mutex_);
00334
00335             // Check the pointer to the sample data message.
00336             if(!x)
00337                 EXC_THROW("Pointer to message object is NULL!");
00338
00339             // Iterate to emplacement position.
00340             for(it = pres_messages_.begin(); it < pres_messages_.end(); it++) {
00341                 if(x->xmsg_index() == (*it)->xmsg_index()) {
00342                     EXC_THROW("Duplicate X-Message index encountered!");
00343                 } else if(x->xmsg_index() < (*it)->xmsg_index())
00344                     break;
00345             }
00346
00347             // Emplace element at position.
00348             set_aggregated_data_valid(false);
00349             pres_messages_.emplace(it, x);
00350
00351             // Aggregate the sensor data.
00352             AggregateSensorData();
00353
00354         }CATCH_BLOCK("Failed to assign pressure message!")
00355
00356     }
00357
00358     /** Assign a salinity data message to this composite data object.
00359      *
00360      *  The salinity data message will be emplaced into the sensor data message vector in order of
00361      *  increasing message indices. If a message with the same index is as @p x is already contained
00362      *  in the vector, then an exception will be thrown.
00363      *
00364      *  @param[in]  x   A shared pointer to the salinity data message.
00365      *
00366      *  @exception  util::log::MessageException if @p x is a null pointer, a duplicate message index
```

```
00367      *              is encountered, or any other error occurs.
00368      *
00369      *  @note       This object will be locked for the scope of the execution of this function.
00370      *              This function places a lock guard on the mutual exclusion (mutex) lock,
00371      *              thereby locking the float object and preventing access or modification of the
00372      *              object by any other thread. Any thread that does attempt to access or modify
00373      *              this object will block until the lock is released.
00374      */
00375     void AssignSalinityMessage(const SalnMsgPtr_t& x) {
00376         typename SalnMsgVector_t::iterator it;
00377
00378         try {
00379             // Get an exclusive lock.
00380             lock_guard<recursive_mutex> guard(mutex_);
00381
00382             ldbg() « "Assigning salinity data message to composite sensor data object." « endl;
00383
00384             // Check the pointer to the sample data message.
00385             if(!x)
00386                 EXC_THROW("Pointer to message object is NULL!");
00387
00388             // Iterate to emplacement position.
00389             for(it = saln_messages_.begin(); it < saln_messages_.end(); it++) {
00390                 if(x->xmsg_index() == (*it)->xmsg_index()) {
00391                     EXC_THROW("Duplicate XMessage index encountered!");
00392                 } else if(x->xmsg_index() < (*it)->xmsg_index())
00393                     break;
00394             }
00395
00396             // Emplace element at position.
00397             set_aggregated_data_valid(false);
00398             saln_messages_.emplace(it, x);
00399
00400             // Aggregate the sensor data.
00401             AggregateSensorData();
00402
00403         }CATCH_BLOCK("Failed to assign salinity message!")
00404
00405     }
00406
00407     /** Assign a temperature data message to this composite data object.
00408      *
00409      *  The temperature data message will be emplaced into the sensor data message vector in order
00410      *  of  increasing message indices. If a message with the same index is as @p x is already
00411      *  contained in the vector, then an exception will be thrown.
00412      *
00413      *  @param[in]  x   A shared pointer to the temperature data message.
00414      *
00415      *  @exception  util::log::MessageException if @p x is a null pointer, a duplicate message index
00416      *              is encountered, or any other error occurs.
00417      *
00418      *  @note       This object will be locked for the scope of the execution of this function.
00419      *              This function places a lock guard on the mutual exclusion (mutex) lock,
00420      *              thereby locking the float object and preventing access or modification of the
00421      *              object by any other thread. Any thread that does attempt to access or modify
00422      *              this object will block until the lock is released.
00423      */
00424     void AssignTemperatureMessage(const TempMsgPtr_t& x) {
00425         typename TempMsgVector_t::iterator it;
00426
00427         try {
00428
00429             // Get an exclusive lock.
00430             lock_guard<recursive_mutex> guard(mutex_);
00431
00432             ldbg() « "Assigning temperature data message to composite sensor data object."
00433                    « endl;
00434
00435             // Check the pointer to the sample data message.
00436             if(!x)
00437                 EXC_THROW("Pointer to message object is NULL!");
00438
00439             // Iterate to emplacement position.
00440             for(it = temp_messages_.begin(); it < temp_messages_.end(); it++) {
00441                 if(x->xmsg_index() == (*it)->xmsg_index()) {
00442                     EXC_THROW("Duplicate XMessage index encountered!");
00443                 } else if(x->xmsg_index() < (*it)->xmsg_index())
00444                     break;
00445             }
00446
00447             // Emplace element at position.
00448             set_aggregated_data_valid(false);
00449             temp_messages_.emplace(it, x);
00450
00451             // Aggregate the sensor data.
00452             AggregateSensorData();
00453
00454         }CATCH_BLOCK("Failed to assign temperature message!")
00455
00456     }
00457
00458     /** Assign an auxiliary-channel temperature message and re-aggregate the sensor data.
00459      *
00460      *  The message is inserted in order of message index, and the aggregated sensor data is
00461      *  rebuilt.
00462      *
00463      *  @param[in]  x       The auxiliary-channel temperature message to assign.
00464      *
00465      *  @throw      util::log::MessageException if @p x is NULL or an auxiliary temperature message
00466      *              with the same index has already been assigned.
00467      */
00468     void AssignAuxTemperatureMessage(const TempMsgPtr_t& x) {
00469         typename TempMsgVector_t::iterator it;
00470
00471         try {
00472
00473             // Get an exclusive lock.
00474             lock_guard<recursive_mutex> guard(mutex_);
```

```
00475
00476             ldbg() « "Assigning aux. temperature data message to composite sensor data object."
00477                    « endl;
00478
00479             // Check the pointer to the sample data message.
00480             if(!x)
00481                 EXC_THROW("Pointer to message object is NULL!");
00482
00483             // Iterate to emplacement position.
00484             for(it = aux_temp_messages_.begin(); it < aux_temp_messages_.end(); it++) {
00485                 if(x->xmsg_index() == (*it)->xmsg_index()) {
00486                     EXC_THROW("Duplicate XMessage index encountered!");
00487                 } else if(x->xmsg_index() < (*it)->xmsg_index())
00488                     break;
00489             }
00490
00491             // Emplace element at position.
00492             set_aggregated_data_valid(false);
00493             aux_temp_messages_.emplace(it, x);
00494
00495             // Aggregate the sensor data.
00496             AggregateSensorData();
00497
00498         }CATCH_BLOCK("Failed to assign aux. temperature message!")
00499
00500     }
00501
00502     template<typename T = double> void CopyPressureDataToArray(
00503             T dest[],
00504             size_t max_len,
00505             int16_t gain,
00506             int16_t offs,
00507             float pres_correction,
00508             T fill_value = XMSG_SENS_BAD_PRES) const {
00509
00510         try {
00511
00512             // Get an exclusive lock.
00513             lock_guard<recursive_mutex> guard(mutex_);
00514
00515             size_t nsam = num_pres_samples();
00516             double sam;
00517
00518             for(size_t i = 0; i < nsam && i < max_len; ++i)
00519
00520                 if((sam = SamplePresAsFloat(i, gain, offs, pres_correction)) == XMSG_SENS_BAD_PRES)
00521                     dest[i] = fill_value;
00522                 else
00523                     dest[i] = sam;
00524
00525         }CATCH_BLOCK("Failed to copy pressure data to array!")
00526
00527     }
00528
00529     template<typename T = double> void CopySalinityDataToArray(
00530             T dest[],
00531             size_t max_len,
00532             int16_t gain,
00533             int16_t offs,
00534             T fill_value = XMSG_SENS_BAD_SALN) const {
00535
00536         try {
00537
00538             // Get an exclusive lock.
00539             lock_guard<recursive_mutex> guard(mutex_);
00540
00541             size_t nsam = num_saln_samples();
00542             double sam;
00543
00544             for(size_t i = 0; i < nsam && i < max_len; ++i)
00545
00546                 if((sam = SampleSalnAsFloat(i, gain, offs)) == XMSG_SENS_BAD_SALN)
00547                     dest[i] = fill_value;
00548                 else
00549                     dest[i] = sam;
00550
00551         }CATCH_BLOCK("Failed to copy salinity data to array!")
00552
00553     }
00554
00555     template<typename T = double> void CopyTemperatureDataToArray(
00556             T dest[],
00557             size_t max_len,
00558             int16_t gain,
00559             int16_t offs,
00560             T fill_value = XMSG_SENS_BAD_TEMP) const {
00561
00562         try {
00563
00564             // Get an exclusive lock.
00565             lock_guard<recursive_mutex> guard(mutex_);
00566
00567             size_t nsam = num_temp_samples();
00568             double sam;
00569
00570             for(size_t i = 0; i < nsam && i < max_len; ++i)
00571
00572                 if((sam = SampleTempAsFloat(i, gain, offs)) == XMSG_SENS_BAD_TEMP)
00573                     dest[i] = fill_value;
00574                 else
00575                     dest[i] = sam;
00576
00577         }CATCH_BLOCK("Failed to copy temperature data to array!")
00578
00579     }
00580
00581     /** Generate a listing string representation of the this composite sensor data object.
00582      *
```

```
00583      *  @param[in]  id_inst The unique instrument identifier for this composite sensor data object.
00584      *  @param[in]  id_dive The dive or cycle number for this composite data object.
00585      *  @param[in]  pg      The pressure sensor sensitivity scale factor, in LSB/dBar.
00586      *  @param[in]  po      The pressure sensor offset, in dBar.
00587      *  @param[in]  sg      The salinity sensor sensitivity scale factor, in LSB/PSU.
00588      *  @param[in]  so      The salinity sensor offset, in PSU.
00589      *  @param[in]  tg      The temperature sensor sensitivity scale factor, in LSB/degC.
00590      *  @param[in]  to      The temperature sensor offset, in degC.
00591      *  @param[in]  pc      The pressure correction, in dBar, to be subtracted computed pressure
00592      *                      value.
00593      *
00594      *  @throw      log::MessageException if an error occurs while generating the listing string.
00595      *
00596      *  @note       This object will be locked for the scope of the execution of this function.
00597      *              This function places a lock guard on the mutual exclusion (mutex) lock,
00598      *              thereby locking the float object and preventing access or modification of the
00599      *              object by any other thread. Any thread that does attempt to access or modify
00600      *              this object will block until the lock is released.
00601      *
00602      *  @return     A listing string representation of this composite sensor data object.
00603      */
00604     virtual const string GenerateListingString(
00605             int id_inst,
00606             int id_dive,
00607             uint16_t pg,
00608             uint16_t po,
00609             uint16_t sg,
00610             uint16_t so,
00611             uint16_t tg,
00612             uint16_t to,
00613             float pc) const = 0;
00614
00615     /** Get the total number of pressure sensor samples contained in the pressure data messages
00616      *  assigned to this composite data object.
00617      *
00618      *  @exception  util::log::MessageException if an error occurs.
00619      *
00620      *  @note       This object will be locked for the scope of the execution of this function.
00621      *              This function places a lock guard on the mutual exclusion (mutex) lock,
00622      *              thereby locking the float object and preventing access or modification of the
00623      *              object by any other thread. Any thread that does attempt to access or modify
00624      *              this object will block until the lock is released.
00625      *
00626      *  @return     The total number of pressure sensor samples contained in the pressure data
00627      *              messages assigned to this composite data object.
00628      */
00629     size_t num_pres_samples() const {
00630         size_t num_pres_samples = 0;
00631
00632         try {
00633
00634             // Get an exclusive lock.
00635             lock_guard<recursive_mutex> guard(mutex_);
00636
00637             //  If the aggregated data is valid, then return the size of the aggregated data.
00638             if(is_aggregated_data_valid())
00639                 return aggregated_pres_data_.size();
00640
00641             // Iterate over pressure messages.
00642             for_each(
00643                     pres_messages_.begin(),
00644                     pres_messages_.end(),
00645                     [&](const shared_ptr<PRES_CLASS>& msg) {
00646                         if(msg)
00647                         num_pres_samples += msg->num_samples();
00648                     });
00649
00650         }CATCH_BLOCK("Failed to get the number of pressure samples!")
00651
00652         return num_pres_samples;
00653     }
00654
00655     /** Get the total number of salinity sensor samples contained in the salinity data messages
00656      *  assigned to this composite data object.
00657      *
00658      *  @exception  util::log::MessageException if an error occurs.
00659      *
00660      *  @note       This object will be locked for the scope of the execution of this function.
00661      *              This function places a lock guard on the mutual exclusion (mutex) lock,
00662      *              thereby locking the float object and preventing access or modification of the
00663      *              object by any other thread. Any thread that does attempt to access or modify
00664      *              this object will block until the lock is released.
00665      *
00666      *  @return     The total number of pressure sensor samples contained in the pressure data
00667      *              messages assigned to this composite data object.
00668      */
00669     size_t num_saln_samples() const {
00670         size_t num_saln_samples = 0;
00671
00672         try {
00673
00674             // Get an exclusive lock.
00675             lock_guard<recursive_mutex> guard(mutex_);
00676
00677             //  If the aggregated data is valid, then return the size of the aggregated data.
00678             if(is_aggregated_data_valid())
00679                 return aggregated_saln_data_.size();
00680
00681             // Iterate over salinity messages.
00682             for_each(
00683                     saln_messages_.begin(),
00684                     saln_messages_.end(),
00685                     [&](const shared_ptr<SALN_CLASS>& msg) {
00686                         if(msg)
00687                         num_saln_samples += msg->num_samples();
00688                     });
00689
00690         }CATCH_BLOCK("Failed to get the number of salinity samples!")
```

```
00691
00692         return num_saln_samples;
00693     }
00694
00695     /** Get the total number of temperature sensor samples contained in the temperature data
00696      *  messages assigned to this composite data object.
00697      *
00698      *  @exception  util::log::MessageException if an error occurs.
00699      *
00700      *  @note       This object will be locked for the scope of the execution of this function.
00701      *              This function places a lock guard on the mutual exclusion (mutex) lock,
00702      *              thereby locking the float object and preventing access or modification of the
00703      *              object by any other thread. Any thread that does attempt to access or modify
00704      *              this object will block until the lock is released.
00705      *
00706      *  @return     The total number of pressure sensor samples contained in the pressure data
00707      *              messages assigned to this composite data object.
00708      */
00709     size_t num_temp_samples() const {
00710         size_t num_temp_samples = 0;
00711
00712         try {
00713
00714             // Get an exclusive lock.
00715             lock_guard<recursive_mutex> guard(mutex_);
00716
00717             //  If the aggregated data is valid, then return the size of the aggregated data.
00718             if(is_aggregated_data_valid())
00719                 return aggregated_temp_data_.size();
00720
00721             // Iterate over temperature messages.
00722             for_each(
00723                     temp_messages_.begin(),
00724                     temp_messages_.end(),
00725                     [&](const shared_ptr<TEMP_CLASS>& msg) {
00726                         if(msg)
00727                         num_temp_samples += msg->num_samples();
00728                     });
00729
00730         }CATCH_BLOCK("Failed to get the number of temperature samples!")
00731
00732         return num_temp_samples;
00733     }
00734
00735     size_t num_aux_temp_samples() const {
00736         size_t num_temp_samples = 0;
00737
00738         try {
00739
00740             // Get an exclusive lock.
00741             lock_guard<recursive_mutex> guard(mutex_);
00742
00743             //  If the aggregated data is valid, then return the size of the aggregated data.
00744             if(is_aggregated_data_valid())
00745                 return aggregated_aux_temp_data_.size();
00746
00747             // Iterate over temperature messages.
00748             for_each(
00749                 aux_temp_messages_.begin(),
00750                 aux_temp_messages_.end(),
00751                     [&](const shared_ptr<TEMP_CLASS>& msg) {
00752                         if(msg)
00753                           num_temp_samples += msg->num_samples();
00754                     });
00755
00756         }CATCH_BLOCK("Failed to get the number of aux temperature samples!")
00757
00758         return num_temp_samples;
00759     }
00760
00761     /** Return the maximum of the number of pressure, temperature, or salinity sensor samples in
00762      *  the sensor data messages assigned to this composite sensor data object.
00763      *
00764      *
00765      *
00766      *  @return     The maximum of the number of pressure, temperature, or salinity sensor samples.
00767      */
00768     size_t max_num_samples() const {
00769         return max(max(num_pres_samples(), num_saln_samples()), num_temp_samples());
00770     }
00771
00772     /** Do the sensor data messages assigned to this composite data object have contiguous message
00773      *  indices?
00774      *
00775      *  This function is used, in part, to determine the sensor data is complete.
00776      *
00777      *  @note       This object will be locked for the scope of the execution of this function.
00778      *              This function places a lock guard on the mutual exclusion (mutex) lock,
00779      *              thereby locking the float object and preventing access or modification of the
00780      *              object by any other thread. Any thread that does attempt to access or modify
00781      *              this object will block until the lock is released.
00782      *
00783      *  @return     TRUE if the sensor data messages have contiguous message indices or if there
00784      *              is less than two of each sensor data message, or FALSE otherwise.
00785      */
00786     bool HasContiguousMessageIndices() const {
00787
00788         try {
00789
00790             // Get an exclusive lock.
00791             lock_guard<recursive_mutex> guard(mutex_);
00792
00793             ldbg() « "Checking if sensor data has contiguous message indices." « endl;
00794
00795             // Check the pressure messages' indices.
00796             if(pres_messages_.size() > 1) {
00797                 int idx = pres_messages_.at(0)->xmsg_index();
00798
```

```
00799                 typename PresMsgVector_t::const_iterator it;
00800                 for(it = pres_messages_.begin() + 1; it < pres_messages_.end(); ++it)
00801                     if((*it)->xmsg_index() != ++idx)
00802                         return false;
00803 
00804             }
00805 
00806             // Check the salinity messages' indices.
00807             if(saln_messages_.size() > 1) {
00808                 int idx = saln_messages_.at(0)->xmsg_index();
00809 
00810                 typename SalnMsgVector_t::const_iterator it;
00811                 for(it = saln_messages_.begin() + 1; it < saln_messages_.end(); ++it)
00812                     if((*it)->xmsg_index() != ++idx)
00813                         return false;
00814 
00815             }
00816 
00817             // Check the pressure messages' indices.
00818             if(temp_messages_.size() > 1) {
00819                 int idx = temp_messages_.at(0)->xmsg_index();
00820 
00821                 typename TempMsgVector_t::const_iterator it;
00822                 for(it = temp_messages_.begin() + 1; it < temp_messages_.end(); ++it)
00823                     if((*it)->xmsg_index() != ++idx)
00824                         return false;
00825 
00826             }
00827 
00828             if(aux_temp_messages_.size() > 1) {
00829                 int idx = aux_temp_messages_.at(0)->xmsg_index();
00830 
00831                 typename TempMsgVector_t::const_iterator it;
00832                 for(it = aux_temp_messages_.begin() + 1; it < aux_temp_messages_.end(); ++it)
00833                     if((*it)->xmsg_index() != ++idx)
00834                         return false;
00835 
00836             }
00837 
00838         }CATCH_BLOCK("Failed to determine if sensor data has contiguous message indices!")
00839 
00840         return true;
00841     }
00842 
00843     bool HasAuxTempData() const {
00844       return !aux_temp_messages_.empty();
00845     }
00846 
00847     /** Is the aggregated data valid?
00848      *
00849      *  @note       This object will be locked for the scope of the execution of this function.
00850      *              This function places a lock guard on the mutual exclusion (mutex) lock,
00851      *              thereby locking the float object and preventing access or modification of the
00852      *              object by any other thread. Any thread that does attempt to access or modify
00853      *              this object will block until the lock is released.
00854      *
00855      *  @return     TRUE if the aggregated sensor is valid, or FALSE otherwise.
00856      */
00857     bool is_aggregated_data_valid() const {
00858 
00859         // Get an exclusive lock.
00860         lock_guard<recursive_mutex> guard(mutex_);
00861 
00862         return aggregated_data_valid_;
00863     }
00864 
00865     /** Is the sensor data for this composite date message complete?
00866      *
00867      *  This function returns true if
00868      *  - this composite sensor data object is not empty,
00869      *  - the sensor data messages assigned to this composite data object have contiguous message
00870      *  indices,
00871      *  - this sensor data object contains the same number of temperature and pressure sensor
00872      *  values, and
00873      *  - the number of salinity data samples is either zero or equal to the number of pressure
00874      *  data samples.
00875      *
00876      *  @note       This object will be locked for the scope of the execution of this function.
00877      *              This function places a lock guard on the mutual exclusion (mutex) lock,
00878      *              thereby locking the float object and preventing access or modification of the
00879      *              object by any other thread. Any thread that does attempt to access or modify
00880      *              this object will block until the lock is released.
00881      *
00882      *  @return     TRUE if the sensor data is complete, or FALSE otherwise.
00883      */
00884     bool IsDataComplete() const {
00885 
00886         try {
00887 
00888             // Get an exclusive lock.
00889             lock_guard<recursive_mutex> guard(mutex_);
00890 
00891             if(IsEmpty())
00892                 return false;
00893 
00894             if(!HasContiguousMessageIndices())
00895                 return false;
00896 
00897             if(num_pres_samples() != num_temp_samples())
00898                 return false;
00899 
00900             if(num_aux_temp_samples() && (num_aux_temp_samples() != num_temp_samples()))
00901               return false;
00902 
00903             if(num_saln_samples() && (num_saln_samples() != num_pres_samples()))
00904                 return false;
00905 
00906         }CATCH_BLOCK("Failed to determine if sensor data is complete!")
```

```
00907
00908         return true;
00909
00910     }
00911
00912     /** Is this composite data object empty?
00913      *
00914      *  @return     TRUE if empty, or FALSE otherwise.
00915      */
00916     bool IsEmpty() const {
00917
00918         return max_num_samples() == 0;
00919
00920     }
00921
00922     /** Remove the assigned pressure, temperature and salinity messages and their aggregated data.
00923      *
00924      *  @warning    The auxiliary temperature messages and aggregated auxiliary temperature data are
00925      *              not cleared, so re-assigning them after a reset fails with a duplicate index.
00926      *
00927      *  @throw      util::log::MessageException if an error occurs.
00928      */
00929     void Reset() {
00930
00931         try {
00932
00933             // Get an exclusive lock.
00934             lock_guard<recursive_mutex> guard(mutex_);
00935
00936             set_aggregated_data_valid(false);
00937             aggregated_pres_data_.clear();
00938             aggregated_saln_data_.clear();
00939             aggregated_temp_data_.clear();
00940             pres_messages_.clear();
00941             saln_messages_.clear();
00942             temp_messages_.clear();
00943
00944         }CATCH_BLOCK("Failed to reset sensor data object!")
00945
00946     }
00947
00948     /** Get the @f$i^{th}@f$ pressure sensor value, in raw counts
00949      *
00950      *  @param[in]  i       The index of the pressure sensor sample to return. If the index exceeds
00951      *                      the number of values in the aggregated data vector, then
00952      *                      @ref XMSG_SENS_BAD will be returned.
00953      *
00954      *  @note       This object will be locked for the scope of the execution of this function.
00955      *              This function places a lock guard on the mutual exclusion (mutex) lock,
00956      *              thereby locking the float object and preventing access or modification of the
00957      *              object by any other thread. Any thread that does attempt to access or modify
00958      *              this object will block until the lock is released.
00959      *
00960      * @return     The @f$i^{th}@f$ pressure sensor value, in raw counts, or @ref XMSG_SENS_BAD if
00961      *              the index @p i exceeds the number of pressure sensor samples in the aggregated
00962      *              data vector.
00963      */
00964     int32_t SamplePres(size_t i) const {
00965
00966         // Get an exclusive lock.
00967         lock_guard<recursive_mutex> guard(mutex_);
00968
00969         return i < num_pres_samples() ? aggregated_pres_data_.at(i) : XMSG_SENS_BAD;
00970     }
00971
00972     /** Get the @f$i^{th}@f$ pressure sensor value, in dBar.
00973      *
00974      *  @param[in]  i       The index of the sensor value. If the index exceeds the number of
00975      *                      values in the aggregated data vector, then @ref XMSG_SENS_BAD_PRES will
00976      *                      be returned.
00977      *  @param[in]  pg      The pressure sensor sensitivity scale factor, in LSB/dBar.
00978      *  @param[in]  po      The pressure sensor offset, in dBar.
00979      *  @param[in]  pc      The pressure correction, in dBar, to be subtracted computed pressure
00980      *                      value.
00981      *
00982      *  @note       This object will be locked for the scope of the execution of this function.
00983      *              This function places a lock guard on the mutual exclusion (mutex) lock,
00984      *              thereby locking the float object and preventing access or modification of the
00985      *              object by any other thread. Any thread that does attempt to access or modify
00986      *              this object will block until the lock is released.
00987      *
00988      *  @return     The @f$i^{th}@f$ pressure sensor value, in dBar. If the index exceeds the
00989      *              number of values in the aggregated data vector, then @ref XMSG_SENS_BAD_PRES
00990      *              will be returned.
00991      */
00992     template<typename T = double> T SamplePresAsFloat(
00993             size_t i,
00994             int16_t pg,
00995             int16_t po,
00996             float pc) const {
00997
00998         T val = XMSG_SENS_BAD_PRES;
00999
01000         try {
01001
01002             // Get an exclusive lock.
01003             lock_guard<recursive_mutex> guard(mutex_);
01004
01005             if(i < num_pres_samples() && pg) {
01006
01007                 val = (T) aggregated_pres_data_.at(i) / (T) pg - (T) po - pc;
01008
01009                 if(val < XMSG_SENS_MIN_PRES || val > XMSG_SENS_MAX_PRES)
01010                     val = XMSG_SENS_BAD_PRES;
01011
01012             }
01013
01014         }CATCH_BLOCK("Failed to get sensor data value from raw sample counts!")
```

```
01015
01016         return val;
01017     }
01018
01019     /** Get the @f$i^{th}@f$ salinity sensor value, in raw counts
01020      *
01021      *  @param[in]  i       The index of the salinity sensor sample to return. If the index exceeds
01022      *                      the number of values in the aggregated data vector, then
01023      *                      @ref XMSG_SENS_BAD will be returned.
01024      *
01025      *  @note       This object will be locked for the scope of the execution of this function.
01026      *              This function places a lock guard on the mutual exclusion (mutex) lock,
01027      *              thereby locking the float object and preventing access or modification of the
01028      *              object by any other thread. Any thread that does attempt to access or modify
01029      *              this object will block until the lock is released.
01030      *
01031      * @return     The @f$i^{th}@f$ salinity sensor value, in raw counts, or @ref XMSG_SENS_BAD if
01032      *              the index @p i exceeds the number of pressure sensor samples in the aggregated
01033      *              data vector.
01034      */
01035     int32_t SampleSaln(size_t i) const {
01036         // Get an exclusive lock.
01037         lock_guard<recursive_mutex> guard(mutex_);
01038
01039         return i < num_saln_samples() ? aggregated_saln_data_.at(i) : XMSG_SENS_BAD;
01040     }
01041
01042     /** Get the @f$i^{th}@f$ salinity sensor value, in PSU.
01043      *
01044      *  @param[in]  i       The index of the sensor value. If the index exceeds the number of
01045      *                      values in the aggregated data vector, then @ref XMSG_SENS_BAD_SALN will
01046      *                      be returned.
01047      *  @param[in]  gain    The salinity sensor sensitivity scale factor, in LSB/PSU.
01048      *  @param[in]  offs    The salinity sensor offset, in PSU.
01049      *
01050      *  @note       This object will be locked for the scope of the execution of this function.
01051      *              This function places a lock guard on the mutual exclusion (mutex) lock,
01052      *              thereby locking the float object and preventing access or modification of the
01053      *              object by any other thread. Any thread that does attempt to access or modify
01054      *              this object will block until the lock is released.
01055      *
01056      *  @return     The @f$i^{th}@f$ salinity sensor value, in PSU. If the index exceeds the
01057      *              number of values in the aggregated data vector, then @ref XMSG_SENS_BAD_SALN
01058      *              will be returned.
01059      */
01060     template<typename T = double> T SampleSalnAsFloat(size_t i, int16_t gain, int16_t offs) const {
01061
01062         T val = XMSG_SENS_BAD_SALN;
01063
01064         try {
01065
01066             // Get an exclusive lock.
01067             lock_guard<recursive_mutex> guard(mutex_);
01068
01069             if(i < num_saln_samples() && gain) {
01070
01071                 val = (T) aggregated_saln_data_.at(i) / (T) gain - (T) offs;
01072
01073                 if(val < XMSG_SENS_MIN_SALN || val > XMSG_SENS_MAX_SALN)
01074                     val = XMSG_SENS_BAD_SALN;
01075             }
01076
01077         }CATCH_BLOCK("Failed to get sensor data value from raw sample counts!")
01078
01079         return val;
01080     }
01081
01082     /** Get the @f$i^{th}@f$ temperature sensor value, in raw counts
01083      *
01084      *  @param[in]  i       The index of the temperature sensor sample to return. If the index exceeds
01085      *                      the number of values in the aggregated data vector, then
01086      *                      @ref XMSG_SENS_BAD will be returned.
01087      *
01088      *  @note       This object will be locked for the scope of the execution of this function.
01089      *              This function places a lock guard on the mutual exclusion (mutex) lock,
01090      *              thereby locking the float object and preventing access or modification of the
01091      *              object by any other thread. Any thread that does attempt to access or modify
01092      *              this object will block until the lock is released.
01093      *
01094      * @return     The @f$i^{th}@f$ temperature sensor value, in raw counts, or @ref XMSG_SENS_BAD
01095      *              if the index @p i exceeds the number of pressure sensor samples in the
01096      *              aggregated data vector.
01097      */
01098     int32_t SampleTemp(size_t i) const {
01099
01100         // Get an exclusive lock.
01101         lock_guard<recursive_mutex> guard(mutex_);
01102
01103         return i < num_temp_samples() ? aggregated_temp_data_.at(i) : XMSG_SENS_BAD;
01104     }
01105
01106     /** Get the @f$i^{th}@f$ temperature sensor value, in degC.
01107      *
01108      *  @param[in]  i       The index of the sensor value. If the index exceeds the number of
01109      *                      values in the aggregated data vector, then @ref XMSG_SENS_BAD_SALN will
01110      *                      be returned.
01111      *  @param[in]  gain    The temperature sensor sensitivity scale factor, in LSB/degC.
01112      *  @param[in]  offs    The temperature sensor offset, in degC.
01113      *
01114      *  @note       This object will be locked for the scope of the execution of this function.
01115      *              This function places a lock guard on the mutual exclusion (mutex) lock,
01116      *              thereby locking the float object and preventing access or modification of the
01117      *              object by any other thread. Any thread that does attempt to access or modify
01118      *              this object will block until the lock is released.
01119      *
01120      *  @return     The @f$i^{th}@f$ temperature sensor value, in degC. If the index exceeds the
01121      *              number of values in the aggregated data vector, then @ref XMSG_SENS_BAD_TEMP
01122      *              will be returned.
```

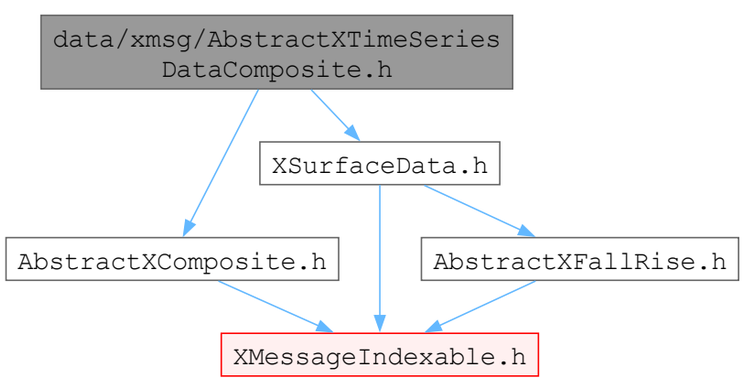
```
01123      */
01124     template<typename T = double> T SampleTempAsFloat(size_t i, int16_t gain, int16_t offs) const {
01125
01126         T val = XMSG_SENS_BAD_TEMP;
01127
01128         try {
01129             // Get an exclusive lock.
01130             lock_guard<recursive_mutex> guard(mutex_);
01131
01132             if(i < num_temp_samples() && gain) {
01133                 val = (T) aggregated_temp_data_.at(i) / (T) gain - (T) offs;
01134
01135                 if(val < XMSG_SENS_MIN_TEMP || val > XMSG_SENS_MAX_TEMP)
01136                     val = XMSG_SENS_BAD_TEMP;
01137
01138             }
01139         }CATCH_BLOCK("Failed to get sensor data value from raw sample counts!")
01140
01141         return val;
01142     }
01143
01144     template<typename T = double> T SampleAuxTempAsFloat(size_t i, int16_t gain, int16_t offs) const {
01145
01146         T val = XMSG_SENS_BAD_TEMP;
01147
01148         try {
01149             // Get an exclusive lock.
01150             lock_guard<recursive_mutex> guard(mutex_);
01151
01152             if(i < num_aux_temp_samples() && gain) {
01153                 val = (T) aggregated_aux_temp_data_.at(i) / (T) gain - (T) offs;
01154
01155                 if(val < XMSG_SENS_MIN_TEMP || val > XMSG_SENS_MAX_TEMP)
01156                     val = XMSG_SENS_BAD_TEMP;
01157
01158             }
01159         }CATCH_BLOCK("Failed to get sensor data value from raw sample counts!")
01160
01161         return val;
01162     }
01163
01164 };
01165
01166 /* Functions: */
01167
01168 } /* namespace xmsg */
01169 } /* namespace util */
01170
01171 #endif /* ABSTRACTXSENSORDATACOMPOSITE_H_ */
```

11.39 data/xmsg/AbstractXTimeSeriesDataComposite.h File Reference

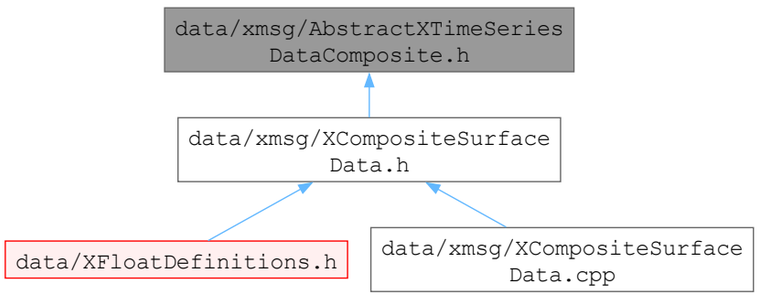
The util::xmsg::AbstractXTimeSeriesDataComposite class header file.

```
#include "AbstractXComposite.h"
#include "XSurfaceData.h"
```

Include dependency graph for AbstractXTimeSeriesDataComposite.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >

  The abstract time-series data composite class template.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.39.1 Detailed Description

The util::xmsg::AbstractXTimeSeriesDataComposite class header file.

Version

0.6

Date

Jun 17, 2016 3:03:57 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.40 AbstractXTimeSeriesDataComposite.h

Go to the documentation of this file.

```
00001 /** The @ref util::xmsg::AbstractXTimeSeriesDataComposite class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jun 17, 2016 3:03:57 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef DATA_XMSG_ABSTRACTXTIMESERIESDATACOMPOSITE_H_
00024 #define DATA_XMSG_ABSTRACTXTIMESERIESDATACOMPOSITE_H_
00025
00026 /* Includes: */
00027
00028 // local
00029 #include "AbstractXComposite.h"
00030 #include "XSurfaceData.h"
00031
00032 /* Definitions: */
00033
00034
00035
00036 namespace util {
00037 namespace xmsg {
00038
00039 /* Types: */
00040
00041 /** The abstract time-series data composite class template.
00042  *
00043  *  This template aggregates the indexed time-series X-messages of type @p MSG_CLASS received for
00044  *  a single dive into one time series. It is used for the surface time-series data (see
00045  *  @ref XCompositeSurfaceData).
00046  */
00047 template<class MSG_CLASS> class AbstractXTimeSeriesDataComposite/*:
00048         public util::thread::RecursiveLockable */{
00049
00050 public:
00051
00052     /* Type Definitions: */
00053
00054     typedef vector<uint32_t> ElapsedTimeVector_t;
00055
00056     typedef shared_ptr<MSG_CLASS> MsgPtr_t;
00057
00058     typedef vector<MsgPtr_t> MsgVector_t;
00059
00060     typedef typename MsgVector_t::iterator MsgVectorIter_t;
00061
00062     typedef typename MsgVector_t::const_iterator MsgVectorIterConst_t;
00063
00064     typedef vector<uint8_t> PhaseCodeVector_t;
00065
00066     typedef int32_t Sample_t;
00067
00068
00069     typedef int64_t DerivedSample_t;
00070
00071     typedef vector<Sample_t> SampleVector_t;
00072
00073 public:
00074
00075     /* Public Static Data Members: */
00076
00077     static const int32_t DERIVED_FIELDS_STATIC_GAIN_EXP;
00078
00079 private:
00080
00081     /* Private Data Members: */
00082
00083     /** The aggregated data is valid flag.
00084      *
00085      *  This flag indicates the status of the aggregated sensor data. It is used to ensure the
00086      *  composite sensor data is aggregated only once following each assignment of sensor data to
00087      *  the composite data message.
00088      *
00089      *  This field is set to FALSE during initialization, and is set to FALSE each time a sensor
00090      *  data message is assigned to the composite data message. Each time @c AggregateSensorData()
00091      *  is called, the data will be aggregated, and this field will be set to TRUE.
00092      *
00093      */
00094     bool aggregated_data_valid_;
00095
00096
00097     DerivedSample_t pres_max_;
00098
00099     DerivedSample_t pres_mean_;
00100
00101     DerivedSample_t pres_min_;
00102
00103 //protected:
```

```
00104 //
00105 //    /* Protected Data Members: */
00106 //
00107 //
00108     ElapsedTimeVector_t aggregated_elapsed_time_;
00109 //
00110     PhaseCodeVector_t aggregated_phase_codes_;
00111 //
00112     SampleVector_t aggregated_pres_data_;
00113 //
00114     SampleVector_t aggregated_saln_data_;
00115 //
00116     SampleVector_t aggregated_temp_data_;
00117 //
00118     MsgVector_t messages_;
00119 //
00120 //
00121 public:
00122 //
00123     /* Public Static Member Functions: */
00124 //
00125     static int32_t derived_fields_static_gain_exp() {
00126 //
00127         return DERIVED_FIELDS_STATIC_GAIN_EXP;
00128 //
00129     }
00130 //
00131 public:
00132 //
00133     /* Public Member Functions: */
00134 //
00135     AbstractXTimeSeriesDataComposite() :
00136                     aggregated_data_valid_(false),
00137                     pres_max_(XMSG_SENS_BAD),
00138                     pres_mean_(XMSG_SENS_BAD),
00139                     pres_min_(XMSG_SENS_BAD) {
00140 //
00141         // Pre-allocate memory for aggregated data.
00142         aggregated_pres_data_.reserve(XMSG_INIT_SAM_N_RESERVE);
00143         aggregated_saln_data_.reserve(XMSG_INIT_SAM_N_RESERVE);
00144         aggregated_temp_data_.reserve(XMSG_INIT_SAM_N_RESERVE);
00145 //
00146     }
00147 //
00148     virtual ~AbstractXTimeSeriesDataComposite() {
00149 //
00150         /* Do nothing. */
00151 //
00152     }
00153 //
00154     /** Assign a time-series message to this composite and re-aggregate the time series.
00155      *
00156      *  The message is inserted in order of message index, and @c AggregateSensorData() is called
00157      *  to rebuild the combined time series.
00158      *
00159      *  @param[in]  x       The time-series message to assign.
00160      *
00161      *  @throw      util::log::MessageException if @p x is NULL or a message with the same index
00162      *              has already been assigned.
00163      */
00164     virtual void AssignMessage(const MsgPtr_t& x) {
00165 //
00166         MsgVectorIter_t it;
00167 //
00168         try {
00169 //
00170             ldbg() « "Assigning data message to composite time-series data object." « endl;
00171 //
00172             // Get an exclusive lock.
00173             //lock_guard<recursive_mutex> guard(mutex_);
00174 //
00175             // Check the pointer to the sample data message.
00176             if(!x)
00177                 EXC_THROW("Pointer to message object is NULL!");
00178 //
00179             // Iterate to emplacement position.
00180             for(it = messages_.begin(); it < messages_.end(); it++)
00181 //
00182                 // Compare message indices.
00183                 if(x->xmsg_index() == (*it)->xmsg_index()) {
00184 //
00185                     EXC_THROW("Duplicate X-Message index encountered!");
00186 //
00187                 } else if(x->xmsg_index() < (*it)->xmsg_index())
00188 //
00189                     break;
00190 //
00191             set_aggregated_data_valid(false);
00192 //
00193             // Emplace element at position.
00194             messages_.emplace(it, x);
00195 //
00196             // Aggregate the sensor data.
00197             AggregateSensorData();
00198 //
00199         }CATCH_BLOCK("Failed to assign time-series message to composite object!")
00200 //
00201     }
00202 //
00203     virtual const string GenerateListingString(
00204             uint16_t& pg,
00205             uint16_t& po,
00206             uint16_t& sg,
00207             uint16_t& so,
00208             uint16_t& tg,
00209             uint16_t& to,
00210             float pres_correction) const {
00211 //
```

```
00212         MsgVectorIterConst_t it;
00213         ostringstream os;
00214 
00215         try {
00216 
00217             os « AbstractXFallRise::GenerateListingComment();
00218 
00219             for_each(messages_.begin(), messages_.end(), [&](const MsgPtr_t& msg) {
00220 
00221                 if(msg) os « endl « msg->GenerateListingString(pg, po, sg, so, tg, to, pres_correction);
00222 
00223             });
00224 
00225         }CATCH_BLOCK("Failed to generate listing string for composite time-series data object!")
00226 
00227         return os.str();
00228     }
00229 
00230     virtual bool HasContiguousMessageIndices() const {
00231 
00232         size_t idx = 0;
00233 
00234         try {
00235 
00236             // Get an exclusive lock.
00237             //lock_guard<recursive_mutex> guard(mutex_);
00238 
00239             ldbg() « "Checking if -series data has contiguous message indices." « endl;
00240 
00241             // Check the pressure messages' indices.
00242             if(messages_.size() > 1) {
00243 
00244                 // Iterate over the messages.
00245                 for(MsgVectorIterConst_t it = messages_.begin(); it < messages_.end(); ++it)
00246 
00247                     // Compare the actual vs. expected messages indices
00248                     if((*it)->xmsg_index() != idx++)
00249 
00250                         return false;
00251 
00252             }
00253 
00254         }CATCH_BLOCK("Failed to determine if time-series data has contiguous message indices!")
00255 
00256         return true;
00257     }
00258 
00259     /** Is the aggregated data valid?
00260      *
00261      *  @note       This object will be locked for the scope of the execution of this function.
00262      *              This function places a lock guard on the mutual exclusion (mutex) lock,
00263      *              thereby locking the float object and preventing access or modification of the
00264      *              object by any other thread. Any thread that does attempt to access or modify
00265      *              this object will block until the lock is released.
00266      *
00267      *  @return     TRUE if the aggregated sensor is valid, or FALSE otherwise.
00268      */
00269     bool is_aggregated_data_valid() const {
00270 
00271         // Get an exclusive lock.
00272         //lock_guard<recursive_mutex> guard(mutex_);
00273 
00274         return aggregated_data_valid_;
00275     }
00276 
00277     bool IsDataComplete() const {
00278 
00279         try {
00280 
00281             // Get an exclusive lock.
00282             //lock_guard<recursive_mutex> guard(mutex_);
00283 
00284             return !IsEmpty();
00285 
00286         }CATCH_BLOCK("Failed to determine if time-series data is complete!")
00287 
00288         return true;
00289 
00290     }
00291 
00292     /** Is this composite data object empty?
00293      *
00294      *  @return     TRUE if empty, or FALSE otherwise.
00295      */
00296     bool IsEmpty() const {
00297 
00298         return SampleCount() == 0;
00299 
00300     }
00301 
00302     size_t NumPresSamples() const {
00303 
00304         size_t num_samples = 0;
00305 
00306         try {
00307 
00308             // Get an exclusive lock.
00309             //lock_guard<recursive_mutex> guard(mutex_);
00310 
00311             // If the aggregated data is valid, then return the size of the aggregated data.
00312             if(is_aggregated_data_valid())
00313                 return aggregated_pres_data_.size();
00314 
00315             // Check if there are any messages.
00316             if(!messages_.size())
00317                 return 0;
00318 
00319             // Determine message encoding type.
```

```
00320             switch(messages_.at(0)->data_pack_format()) {
00321             case XRISEFALL_FMT_LEGACY:
00322             case XRISEFALL_FMT_STNDRD:
00323             case XRISEFALL_FMT_STNDRD_WITH_TEMP_AND_SALN:
00324 
00325                 // Iterate over the messages.
00326                 for_each(messages_.begin(), messages_.end(), [&](const MsgPtr_t msg) {
00327                     num_samples += msg->num_samples();
00328                 });
00329 
00330                 break;
00331 
00332             case XRISEFALL_FMT_UNKNOWN:
00333             default:
00334 
00335                 EXC_THROW("Unsupported message encoding type!");
00336 
00337             }
00338 
00339         }CATCH_BLOCK("Failed to compute pressure sample count!")
00340 
00341         return num_samples;
00342 
00343     }
00344 
00345     size_t NumTempSamples() const {
00346 
00347         size_t num_samples = 0;
00348 
00349         try {
00350 
00351             // Get an exclusive lock.
00352             //lock_guard<recursive_mutex> guard(mutex_);
00353 
00354             //  If the aggregated data is valid, then return the size of the aggregated data.
00355             if(is_aggregated_data_valid())
00356                 return aggregated_temp_data_.size();
00357 
00358             // Check if there are any messages.
00359             if(!messages_.size())
00360                 return 0;
00361 
00362             // Determine message encoding type.
00363             switch(messages_.at(0)->data_pack_format()) {
00364 
00365             case XRISEFALL_FMT_LEGACY:
00366             case XRISEFALL_FMT_STNDRD:
00367 
00368                 /* Do nothing. */
00369 
00370                 break;
00371 
00372             case XRISEFALL_FMT_STNDRD_WITH_TEMP_AND_SALN:
00373 
00374                 // Iterate over the messages.
00375                 for_each(messages_.begin(), messages_.end(), [&](const MsgPtr_t msg) {
00376                     num_samples += msg->num_samples();
00377                 });
00378 
00379                 break;
00380 
00381             case XRISEFALL_FMT_UNKNOWN:
00382             default:
00383 
00384                 EXC_THROW("Unsupported message encoding type!");
00385 
00386             }
00387 
00388         }CATCH_BLOCK("Failed to compute temperature sample count!")
00389 
00390         return num_samples;
00391 
00392     }
00393 
00394     size_t NumSalnSamples() const {
00395 
00396         size_t num_samples = 0;
00397 
00398         try {
00399 
00400             // Get an exclusive lock.
00401             //lock_guard<recursive_mutex> guard(mutex_);
00402 
00403             //  If the aggregated data is valid, then return the size of the aggregated data.
00404             if(is_aggregated_data_valid())
00405                 return aggregated_saln_data_.size();
00406 
00407             // Check if there are any messages.
00408             if(!messages_.size())
00409                 return 0;
00410 
00411             // Determine message encoding type.
00412             switch(messages_.at(0)->data_pack_format()) {
00413 
00414             case XRISEFALL_FMT_LEGACY:
00415             case XRISEFALL_FMT_STNDRD:
00416 
00417                 /* Do nothing. */
00418 
00419                 break;
00420 
00421             case XRISEFALL_FMT_STNDRD_WITH_TEMP_AND_SALN:
00422 
00423                 // Iterate over the messages.
00424                 for_each(messages_.begin(), messages_.end(), [&](const MsgPtr_t msg) {
00425                     num_samples += msg->num_samples();
00426                 });
00427 
```

```
00428                 break;
00429
00430             case XRISEFALL_FMT_UNKNOWN:
00431             default:
00432
00433                 EXC_THROW("Unsupported message encoding type!");
00434
00435             }
00436
00437         }CATCH_BLOCK("Failed to compute salinity sample count!")
00438
00439         return num_samples;
00440
00441     }
00442
00443     DerivedSample_t pres_max() const {
00444
00445         // Get an exclusive lock.
00446         //lock_guard<recursive_mutex> guard(mutex_);
00447
00448         return pres_max_;
00449     }
00450
00451     DerivedSample_t pres_mean() const {
00452
00453         // Get an exclusive lock.
00454         //lock_guard<recursive_mutex> guard(mutex_);
00455
00456         return pres_mean_;
00457     }
00458
00459     DerivedSample_t pres_min() const {
00460
00461         // Get an exclusive lock.
00462         //lock_guard<recursive_mutex> guard(mutex_);
00463
00464         return pres_min_;
00465     }
00466
00467     /** Clear the aggregated time series and its derived pressure statistics.
00468      *
00469      *  The assigned messages are kept; the next call to @c AggregateSensorData() rebuilds the
00470      *  aggregated data from them.
00471      *
00472      *  @throw      util::log::MessageException if an error occurs.
00473      */
00474     void ResetAggregatedData() {
00475
00476         try {
00477
00478             // Get an exclusive lock.
00479             //lock_guard<recursive_mutex> guard(mutex_);
00480
00481             // Clear aggregated data
00482             set_aggregated_data_valid(false);
00483             aggregated_elapsed_time_.clear();
00484             aggregated_phase_codes_.clear();
00485             aggregated_pres_data_.clear();
00486             aggregated_saln_data_.clear();
00487             aggregated_temp_data_.clear();
00488             set_pres_max(XMSG_SENS_BAD);
00489             set_pres_mean(XMSG_SENS_BAD);
00490             set_pres_min(XMSG_SENS_BAD);
00491
00492         }CATCH_BLOCK("Failed to reset aggregated data!")
00493
00494     }
00495
00496     /** Remove all assigned messages and clear the aggregated data.
00497      *
00498      *  @throw      util::log::MessageException if an error occurs.
00499      */
00500     void Reset() {
00501
00502         try {
00503
00504             // Get an exclusive lock.
00505             //lock_guard<recursive_mutex> guard(mutex_);
00506
00507             // Reset data.
00508             ResetAggregatedData();
00509             messages_.clear();
00510
00511         }CATCH_BLOCK("Failed to reset time-series data object!")
00512
00513     }
00514
00515     virtual size_t SampleCount() const {
00516
00517         size_t num_samples = 0;
00518
00519         try {
00520
00521             // Get an exclusive lock.
00522             //lock_guard<recursive_mutex> guard(mutex_);
00523
00524             // Compute the number of samples.
00525             if(messages_.size())
00526                 num_samples = std::max(
00527                         NumPresSamples(),
00528                         std::max(NumTempSamples(), NumSalnSamples()));
00529
00530         }CATCH_BLOCK("Failed to get the number of samples in the composite time-series message!")
00531
00532         return num_samples;
00533     }
00534
00535     template<typename T = float> T PresMaxAsFloat(uint16_t gain, uint16_t offs,
```

```
00536             float pres_correction = 0 ) const {
00537
00538         T val = XMSG_SENS_BAD_PRES;
00539
00540         // Get an exclusive lock.
00541         //lock_guard<recursive_mutex> guard(mutex_);
00542
00543         if(gain) {
00544
00545             val = ((T) pres_max() / (T) gain / (T)(1 « derived_fields_static_gain_exp()))
00546                     - (T) offs - pres_correction;
00547
00548             if(val < XMSG_SENS_MIN_PRES || val > XMSG_SENS_MAX_PRES)
00549                 val = XMSG_SENS_BAD_PRES;
00550
00551         }
00552
00553         return val;
00554
00555     }
00556
00557     template<typename T = float> T PresMeanAsFloat(uint16_t gain, uint16_t offs,
00558             float pres_correction = 0) const {
00559
00560         T val = XMSG_SENS_BAD_PRES;
00561
00562         // Get an exclusive lock.
00563         //lock_guard<recursive_mutex> guard(mutex_);
00564
00565         if(gain) {
00566
00567             val = ((T) pres_mean() / (T) gain / (T)(1 « derived_fields_static_gain_exp()))
00568                     - (T) offs - pres_correction;
00569
00570             if(val < XMSG_SENS_MIN_PRES || val > XMSG_SENS_MAX_PRES)
00571                 val = XMSG_SENS_BAD_PRES;
00572
00573         }
00574
00575         return val;
00576
00577     }
00578
00579     template<typename T = float> T PresMinAsFloat(uint16_t gain, uint16_t offs,
00580             float pres_correction = 0) const {
00581
00582         T val = XMSG_SENS_BAD_PRES;
00583
00584         // Get an exclusive lock.
00585         //lock_guard<recursive_mutex> guard(mutex_);
00586
00587         if(gain) {
00588
00589             val = ((T) pres_min() / (T) gain / (T)(1 « derived_fields_static_gain_exp()))
00590                     - (T) offs - pres_correction;
00591
00592             if(val < XMSG_SENS_MIN_PRES || val > XMSG_SENS_MAX_PRES)
00593                 val = XMSG_SENS_BAD_PRES;
00594
00595         }
00596
00597         return val;
00598
00599     }
00600
00601     /** Get the @f$i^{th}@f$ pressure sensor value, in raw counts
00602      *
00603      *  @param[in]  i       The index of the pressure sensor sample to return. If the index exceeds
00604      *                      the number of values in the aggregated data vector, then
00605      *                      @ref XMSG_SENS_BAD will be returned.
00606      *
00607      *  @note       This object will be locked for the scope of the execution of this function.
00608      *              This function places a lock guard on the mutual exclusion (mutex) lock,
00609      *              thereby locking the float object and preventing access or modification of the
00610      *              object by any other thread. Any thread that does attempt to access or modify
00611      *              this object will block until the lock is released.
00612      *
00613      * @return     The @f$i^{th}@f$ pressure sensor value, in raw counts, or @ref XMSG_SENS_BAD if
00614      *              the index @p i exceeds the number of pressure sensor samples in the aggregated
00615      *              data vector.
00616      */
00617     int32_t SamplePres(size_t i) const {
00618
00619         // Get an exclusive lock.
00620         //lock_guard<recursive_mutex> guard(mutex_);
00621
00622         return i < NumPresSamples() ? aggregated_pres_data_.at(i) : XMSG_SENS_BAD;
00623
00624     }
00625
00626     /** Get the @f$i^{th}@f$ pressure sensor value, in dBar.
00627      *
00628      *  @param[in]  i       The index of the sensor value. If the index exceeds the number of
00629      *                      values in the aggregated data vector, then @ref XMSG_SENS_BAD_PRES will
00630      *                      be returned.
00631      *  @param[in]  pg      The pressure sensor sensitivity scale factor, in LSB/dBar.
00632      *  @param[in]  po      The pressure sensor offset, in dBar.
00633      *  @param[in]  pc      The pressure correction, in dBar, to be subtracted computed pressure
00634      *                      value.
00635      *
00636      *  @note       This object will be locked for the scope of the execution of this function.
00637      *              This function places a lock guard on the mutual exclusion (mutex) lock,
00638      *              thereby locking the float object and preventing access or modification of the
00639      *              object by any other thread. Any thread that does attempt to access or modify
00640      *              this object will block until the lock is released.
00641      *
00642      *  @return     The @f$i^{th}@f$ pressure sensor value, in dBar. If the index exceeds the
00643      *              number of values in the aggregated data vector, then @ref XMSG_SENS_BAD_PRES
```

```
00644      *              will be returned.
00645      */
00646     template<typename T = float> T SamplePresAsFloat(
00647             size_t i,
00648             int16_t pg,
00649             int16_t po,
00650             float pc) const {
00651 
00652         T val = XMSG_SENS_BAD_PRES;
00653 
00654         try {
00655 
00656             // Get an exclusive lock.
00657             //lock_guard<recursive_mutex> guard(mutex_);
00658 
00659             if(i < NumPresSamples() && pg) {
00660 
00661                 val = (T) aggregated_pres_data_.at(i) / (T) pg - (T) po - pc;
00662 
00663                 if(val < XMSG_SENS_MIN_PRES || val > XMSG_SENS_MAX_PRES)
00664                     val = XMSG_SENS_BAD_PRES;
00665 
00666             }
00667 
00668         }CATCH_BLOCK("Failed to get sensor data value from raw sample counts!")
00669 
00670         return val;
00671 
00672     }
00673 
00674     /** Get the @f$i^{th}@f$ salinity sensor value, in raw counts
00675      *
00676      *  @param[in]  i       The index of the salinity sensor sample to return. If the index exceeds
00677      *                      the number of values in the aggregated data vector, then
00678      *                      @ref XMSG_SENS_BAD will be returned.
00679      *
00680      *  @note       This object will be locked for the scope of the execution of this function.
00681      *              This function places a lock guard on the mutual exclusion (mutex) lock,
00682      *              thereby locking the float object and preventing access or modification of the
00683      *              object by any other thread. Any thread that does attempt to access or modify
00684      *              this object will block until the lock is released.
00685      *
00686      * @return     The @f$i^{th}@f$ salinity sensor value, in raw counts, or @ref XMSG_SENS_BAD if
00687      *              the index @p i exceeds the number of pressure sensor samples in the aggregated
00688      *              data vector.
00689      */
00690     int32_t SampleSaln(size_t i) const {
00691 
00692         // Get an exclusive lock.
00693         //lock_guard<recursive_mutex> guard(mutex_);
00694 
00695         return i < NumSalnSamples() ? aggregated_saln_data_.at(i) : XMSG_SENS_BAD;
00696 
00697     }
00698 
00699     /** Get the @f$i^{th}@f$ salinity sensor value, in PSU.
00700      *
00701      *  @param[in]  i       The index of the sensor value. If the index exceeds the number of
00702      *                      values in the aggregated data vector, then @ref XMSG_SENS_BAD_SALN will
00703      *                      be returned.
00704      *  @param[in]  gain    The salinity sensor sensitivity scale factor, in LSB/PSU.
00705      *  @param[in]  offs    The salinity sensor offset, in PSU.
00706      *
00707      *  @note       This object will be locked for the scope of the execution of this function.
00708      *              This function places a lock guard on the mutual exclusion (mutex) lock,
00709      *              thereby locking the float object and preventing access or modification of the
00710      *              object by any other thread. Any thread that does attempt to access or modify
00711      *              this object will block until the lock is released.
00712      *
00713      *  @return     The @f$i^{th}@f$ salinity sensor value, in PSU. If the index exceeds the
00714      *              number of values in the aggregated data vector, then @ref XMSG_SENS_BAD_SALN
00715      *              will be returned.
00716      */
00717     template<typename T = float> T SampleSalnAsFloat(size_t i, int16_t gain, int16_t offs) const {
00718 
00719         T val = XMSG_SENS_BAD_SALN;
00720 
00721         try {
00722 
00723             // Get an exclusive lock.
00724             //lock_guard<recursive_mutex> guard(mutex_);
00725 
00726             if(i < NumSalnSamples() && gain) {
00727 
00728                 val = (T) aggregated_saln_data_.at(i) / (T) gain - (T) offs;
00729 
00730                 if(val < XMSG_SENS_MIN_SALN || val > XMSG_SENS_MAX_SALN)
00731                     val = XMSG_SENS_BAD_SALN;
00732             }
00733 
00734         }CATCH_BLOCK("Failed to get sensor data value from raw sample counts!")
00735 
00736         return val;
00737     }
00738 
00739     /** Get the @f$i^{th}@f$ temperature sensor value, in raw counts
00740      *
00741      *  @param[in]  i       The index of the temperature sensor sample to return. If the index exceeds
00742      *                      the number of values in the aggregated data vector, then
00743      *                      @ref XMSG_SENS_BAD will be returned.
00744      *
00745      *  @note       This object will be locked for the scope of the execution of this function.
00746      *              This function places a lock guard on the mutual exclusion (mutex) lock,
00747      *              thereby locking the float object and preventing access or modification of the
00748      *              object by any other thread. Any thread that does attempt to access or modify
00749      *              this object will block until the lock is released.
00750      *
00751      * @return     The @f$i^{th}@f$ temperature sensor value, in raw counts, or @ref XMSG_SENS_BAD
```

```
00752      *              if the index @p i exceeds the number of pressure sensor samples in the
00753      *              aggregated data vector.
00754      */
00755     int32_t SampleTemp(size_t i) const {
00756
00757         // Get an exclusive lock.
00758         //lock_guard<recursive_mutex> guard(mutex_);
00759
00760         return i < NumTempSamples() ? aggregated_temp_data_.at(i) : XMSG_SENS_BAD;
00761
00762     }
00763
00764     /** Get the @f$i^{th}@f$ temperature sensor value, in degC.
00765      *
00766      *  @param[in]  i       The index of the sensor value. If the index exceeds the number of
00767      *                      values in the aggregated data vector, then @ref XMSG_SENS_BAD_SALN will
00768      *                      be returned.
00769      *  @param[in]  gain    The temperature sensor sensitivity scale factor, in LSB/degC.
00770      *  @param[in]  offs    The temperature sensor offset, in degC.
00771      *
00772      *  @note       This object will be locked for the scope of the execution of this function.
00773      *              This function places a lock guard on the mutual exclusion (mutex) lock,
00774      *              thereby locking the float object and preventing access or modification of the
00775      *              object by any other thread. Any thread that does attempt to access or modify
00776      *              this object will block until the lock is released.
00777      *
00778      *  @return     The @f$i^{th}@f$ temperature sensor value, in degC. If the index exceeds the
00779      *              number of values in the aggregated data vector, then @ref XMSG_SENS_BAD_TEMP
00780      *              will be returned.
00781      */
00782     template<typename T = float> T SampleTempAsFloat(size_t i, int16_t gain, int16_t offs) const {
00783
00784         T val = XMSG_SENS_BAD_TEMP;
00785
00786         try {
00787
00788             // Get an exclusive lock.
00789             //lock_guard<recursive_mutex> guard(mutex_);
00790
00791             if(i < NumTempSamples() && gain) {
00792
00793                 val = (T) aggregated_temp_data_.at(i) / (T) gain - (T) offs;
00794
00795                 if(val < XMSG_SENS_MIN_TEMP || val > XMSG_SENS_MAX_TEMP)
00796                     val = XMSG_SENS_BAD_TEMP;
00797
00798             }
00799
00800         }CATCH_BLOCK("Failed to get sensor data value from raw sample counts!")
00801
00802         return val;
00803     }
00804
00805 protected:
00806
00807     /* Protected Member Functions: */
00808
00809     ElapsedTimeVector_t& aggregated_elapsed_time() {
00810         return aggregated_elapsed_time_;
00811     }
00812
00813     PhaseCodeVector_t& aggregated_phase_codes() {
00814         return aggregated_phase_codes_;
00815     }
00816
00817     SampleVector_t& aggregated_pres_data() {
00818         return aggregated_pres_data_;
00819     }
00820
00821     SampleVector_t& aggregated_saln_data() {
00822         return aggregated_saln_data_;
00823     }
00824
00825     SampleVector_t& aggregated_temp_data() {
00826       return aggregated_temp_data_;
00827     }
00828
00829     /** Combine the assigned time-series messages into one aggregated time series.
00830      *
00831      *  Does nothing if the aggregated data is already valid. Otherwise the aggregated data is
00832      *  cleared and rebuilt from the messages, in message-index order: the pressure counts and
00833      *  phase codes, plus temperature and salinity counts for messages in the
00834      *  @c XRISEFALL_FMT_STNDRD_WITH_TEMP_AND_SALN format. Salinity data is dropped if it contains
00835      *  no valid samples (temperature-only instruments).
00836      *
00837      *  The pressure maximum, minimum and mean are computed from the valid samples (ignoring
00838      *  @c XMSG_SENS_BAD and 0xFFFF), after applying the static gain
00839      *  (@c derived_fields_static_gain_exp). The aggregated data is marked valid only if there is at
00840      *  least one valid pressure sample.
00841      *
00842      *  @throw      util::log::MessageException if no messages are assigned or the message format
00843      *              is unsupported.
00844      */
00845     virtual void AggregateSensorData() {
00846
00847         MsgPtr_t msg;
00848         MsgVectorIter_t msg_iter;
00849         ptime ref_time;
00850         int64_t pres_sum = 0;
00851         Sample_t pres = 0;
00852         size_t num_valid_pres_samples = 0;
00853         vector<XSensorTripletCodeCounts>::const_iterator sample_it;
00854
00855         try {
00856
00857             // Get an exclusive lock.
00858             //lock_guard<recursive_mutex> guard(mutex_);
00859
```

```
00860             ldbg() « "Aggregating time-series sensor data...";
00861
00862             // Check if sensor data is already aggregated.
00863             if(is_aggregated_data_valid())
00864                 return;
00865
00866             // Clear existing sensor data, if any.
00867             ResetAggregatedData();
00868
00869             if(!messages_.size()) {
00870                 EXC_THROW("No messages found!");
00871             }
00872
00873             // Determine message encoding type.
00874             switch(messages_.at(0)->data_pack_format()) {
00875
00876             case XRISEFALL_FMT_LEGACY:
00877             case XRISEFALL_FMT_STNDRD:
00878
00879                 // Reserve storage for sample data.
00880                 aggregated_pres_data().reserve(NumPresSamples());
00881
00882                 // Iterate over messages.
00883                 for(msg_iter = messages_.begin(); msg_iter < messages_.end(); ++msg_iter) {
00884
00885                     if(!(msg = *msg_iter)){
00886                         EXC_THROW("Null pointer to message!");
00887                     }
00888
00889                     ref_time = msg->start_time();
00890
00891                     // Iterate over samples.
00892                     for(sample_it = msg->sensor_data().begin();
00893                             sample_it < msg->sensor_data().end(); ++sample_it) {
00894
00895                                 // Append pressure value to aggregated data.
00896                                 aggregated_pres_data().push_back(pres = sample_it->pressure_counts());
00897
00898                                 // Append phase code value to aggregated data.
00899                                 aggregated_phase_codes().push_back(sample_it->phase_code());
00900
00901                                 // TODO Append elapsed time value to aggregated data.
00902
00903                                 /* Compute derived fields: */
00904
00905                                 // Ignore invalid values.
00906                                 switch(pres) {
00907                                 case XMSG_SENS_BAD:
00908                                 case UINT16_MAX:
00909                                     continue;
00910
00911                                 default:
00912                                     break;
00913                                 }
00914
00915                                 // Apply static gain.
00916                                 pres «= derived_fields_static_gain_exp();
00917
00918                                 // Max.
00919                                 if(pres_max() < pres || pres_max() == XMSG_SENS_BAD)
00920                                     set_pres_max(pres);
00921
00922                                 // Sum.
00923                                 pres_sum += pres;
00924
00925                                 // Min.
00926                                 if(pres_min() > pres || pres_min() == XMSG_SENS_BAD)
00927                                     set_pres_min(pres);
00928
00929                                 // Count the number of valid samples.
00930                                 num_valid_pres_samples++;
00931
00932                             }
00933
00934                 }
00935
00936                 break;
00937
00938             case XRISEFALL_FMT_STNDRD_WITH_TEMP_AND_SALN:
00939
00940                 // Reserve storage for sample data.
00941 #               if defined( XMSG_DEBUG_AGGREGATE_TIMESERIES_DATA) &&  XMSG_DEBUG_AGGREGATE_TIMESERIES_DATA
00942                 ldbg() « "Reserving " « NumPresSamples() « " samples.";
00943 #               endif // defined( XMSG_DEBUG_AGGREGATE_TIMESERIES_DATA) &&
      XMSG_DEBUG_AGGREGATE_TIMESERIES_DATA
00944                 aggregated_pres_data().reserve(NumPresSamples());
00945                 aggregated_temp_data().reserve(NumTempSamples());
00946                 aggregated_saln_data().reserve(NumSalnSamples());
00947
00948                 // Iterate over messages.
00949                 for(msg_iter = messages_.begin(); msg_iter < messages_.end(); ++msg_iter) {
00950
00951                     if(!(msg = *msg_iter)){
00952                         EXC_THROW("Null pointer to message!");
00953                     }
00954
00955 #                   if defined( XMSG_DEBUG_AGGREGATE_TIMESERIES_DATA) &&
      XMSG_DEBUG_AGGREGATE_TIMESERIES_DATA
00956                     {
00957                         ldbg() « "Processing surface message " « (int) msg->xmsg_index() « "...";
00958                     }
00959 #                   endif // defined( XMSG_DEBUG_AGGREGATE_TIMESERIES_DATA) &&
      XMSG_DEBUG_AGGREGATE_TIMESERIES_DATA
00960
00961                     ref_time = msg->start_time();
00962
00963 #                   if defined( XMSG_DEBUG_AGGREGATE_TIMESERIES_DATA) &&
      XMSG_DEBUG_AGGREGATE_TIMESERIES_DATA
```

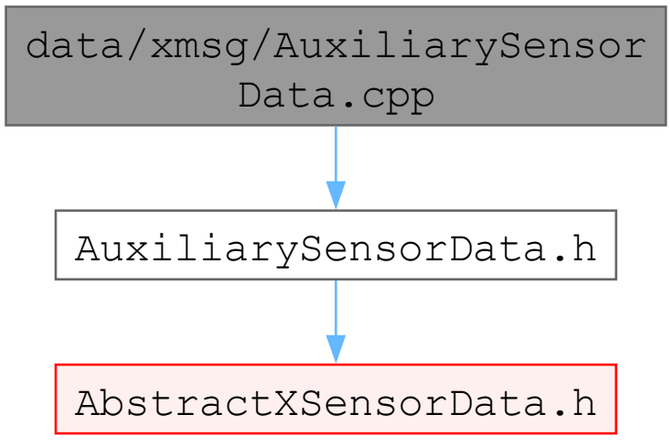
```
00964                     {
00965                         ldbg() « "ref time:  "
00966                                << util::date::DateUtils::GenerateIso8601String(ref_time);
00967                     }
00968 #                   endif // defined( XMSG_DEBUG_AGGREGATE_TIMESERIES_DATA) &&
      XMSG_DEBUG_AGGREGATE_TIMESERIES_DATA
00969
00970                     // Iterate over samples.
00971                     for(sample_it = msg->sensor_data().begin();
00972                             sample_it < msg->sensor_data().end(); ++sample_it) {
00973
00974 #                       if defined(XMSG_DEBUG_AGGREGATE_TIMESERIES_DATA) &&
      XMSG_DEBUG_AGGREGATE_TIMESERIES_DATA
00975                         {
00976                             const format fmt(
00977                                     "Processing surface data sample p = %5d t = %5d s = %5d ...");
00978                             ldbg()
00979                                     « format(fmt) % sample_it->pressure_counts()
00980                                             % sample_it->temperature_counts()
00981                                             % sample_it->salinity_counts();
00982                         }
00983 #                       endif // defined( XMSG_DEBUG_AGGREGATE_TIMESERIES_DATA) &&
      XMSG_DEBUG_AGGREGATE_TIMESERIES_DATA
00984
00985                         // Append values to aggregated data.
00986 #                       if defined(XMSG_DEBUG_AGGREGATE_TIMESERIES_DATA) &&
      XMSG_DEBUG_AGGREGATE_TIMESERIES_DATA
00987                         {
00988                             ldbg() « "Appending samples to aggregated data.";
00989                         }
00990 #                       endif
00991                         aggregated_pres_data().push_back(pres = sample_it->pressure_counts());
00992                         aggregated_temp_data().push_back(sample_it->temperature_counts());
00993                         aggregated_saln_data().push_back(sample_it->salinity_counts());
00994                         aggregated_phase_codes().push_back(sample_it->phase_code());
00995
00996                         // TODO Append elapsed time value to aggregated data.
00997
00998                         /* Compute derived fields: */
00999
01000                         // Ignore invalid values.
01001                         switch(pres) {
01002                         case XMSG_SENS_BAD:
01003                         case UINT16_MAX:
01004                             continue;
01005
01006                         default:
01007                             break;
01008                         }
01009
01010 #                       if defined(XMSG_DEBUG_AGGREGATE_TIMESERIES_DATA) &&
      XMSG_DEBUG_AGGREGATE_TIMESERIES_DATA
01011                         {
01012                             ldbg() « "Applying static gain "
01013                                      « (int) (1 « derived_fields_static_gain_exp());
01014                         }
01015 #                       endif
01016
01017                         // Apply static gain.
01018                         pres «= derived_fields_static_gain_exp();
01019
01020                         // Max.
01021                         if(pres_max() < pres || pres_max() == XMSG_SENS_BAD)
01022                             set_pres_max(pres);
01023
01024                         // Sum.
01025                         pres_sum += pres;
01026
01027                         // Min.
01028                         if(pres_min() > pres || pres_min() == XMSG_SENS_BAD)
01029                             set_pres_min(pres);
01030
01031                         // Count the number of valid samples.
01032                         num_valid_pres_samples++;
01033
01034                     }
01035
01036                 }
01037
01038                 {   // TODO Remove null salinity data (temp. only floats).
01039                     bool contains_valid_saln_data = false;
01040                     for(SampleVector_t::const_iterator it = aggregated_saln_data().begin();
01041                             it < aggregated_saln_data_.end(); ++it) {
01042
01043                         if(*it != XMSG_SENS_BAD) {
01044
01045                             // Found valid salinity data.
01046                             contains_valid_saln_data = true;
01047                                 break;
01048
01049                         }
01050
01051                     }
01052
01053                     if(!contains_valid_saln_data)
01054                         aggregated_saln_data_.clear();
01055
01056                 }
01057
01058                 break;
01059
01060             case XRISEFALL_FMT_UNKNOWN:
01061             default:
01062
01063                 EXC_THROW("Unsupported message encoding type!");
01064                 break;
01065             }
01066
```

```
01067             if(!num_valid_pres_samples) {
01068
01069                 lwrn() « "No valid pressure samples found in time-series data.";
01070
01071                 // Update aggregated data status.
01072                 set_aggregated_data_valid(false);
01073
01074                 return;
01075             }
01076
01077             // Compute mean pressure.
01078             set_pres_mean(pres_sum / num_valid_pres_samples);
01079
01080             // Update aggregated data status.
01081             set_aggregated_data_valid(true);
01082
01083         }CATCH_BLOCK("Failed to aggregate time-series data!")
01084
01085     }
01086
01087
01088
01089     const MsgVector_t& messages() const {
01090         return messages_;
01091     }
01092
01093     /** Set the value of the @ref aggregated_data_valid_ field.
01094      *
01095      *  @param[in]  valid   TRUE if the aggregated data is valid, or FALSE otherwise.
01096      *
01097      *  @note       This object will be locked for the scope of the execution of this function.
01098      *              This function places a lock guard on the mutual exclusion (mutex) lock,
01099      *              thereby locking the float object and preventing access or modification of the
01100      *              object by any other thread. Any thread that does attempt to access or modify
01101      *              this object will block until the lock is released.
01102      */
01103     void set_aggregated_data_valid(bool valid) {
01104
01105         // Get an exclusive lock.
01106         //lock_guard<recursive_mutex> guard(mutex_);
01107
01108         aggregated_data_valid_ = valid;
01109     }
01110
01111     void set_pres_max(DerivedSample_t pres_max) {
01112
01113         // Get an exclusive lock.
01114         //lock_guard<recursive_mutex> guard(mutex_);
01115
01116         pres_max_ = pres_max;
01117     }
01118
01119     void set_pres_mean(DerivedSample_t pres_mean) {
01120
01121         // Get an exclusive lock.
01122         //lock_guard<recursive_mutex> guard(mutex_);
01123
01124         pres_mean_ = pres_mean;
01125     }
01126
01127     void set_pres_min(DerivedSample_t pres_min) {
01128
01129         // Get an exclusive lock.
01130         //lock_guard<recursive_mutex> guard(mutex_);
01131
01132         pres_min_ = pres_min;
01133     }
01134
01135 };
01136
01137 /* Functions: */
01138
01139 } /* namespace xmsg */
01140 } /* namespace util */
01141
01142 #endif /* DATA_XMSG_ABSTRACTXTIMESERIESDATACOMPOSITE_H_ */
```

11.41 data/xmsg/AuxiliarySensorData.cpp File Reference

The util::xmsg::AuxiliarySensorData class source file.

```
#include "AuxiliarySensorData.h"
```
Include dependency graph for AuxiliarySensorData.cpp:



Namespaces

- namespace argo

  The Argo instrument namespace.
- namespace argo::iridium

  The Argo Iridium instrument namespace.
- namespace argo::iridium::xfloat

  The X-Float instrument namespace.

11.41.1 Detailed Description

The util::xmsg::AuxiliarySensorData class source file.

Version

0.6

Date

May 24, 2017 9:49:24 AM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

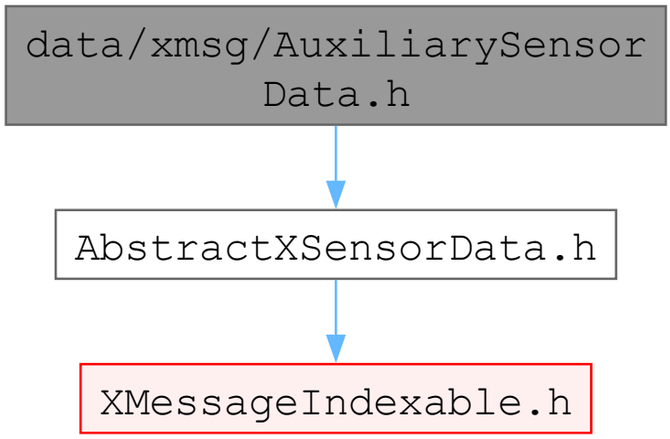
This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.42 data/xmsg/AuxiliarySensorData.h File Reference

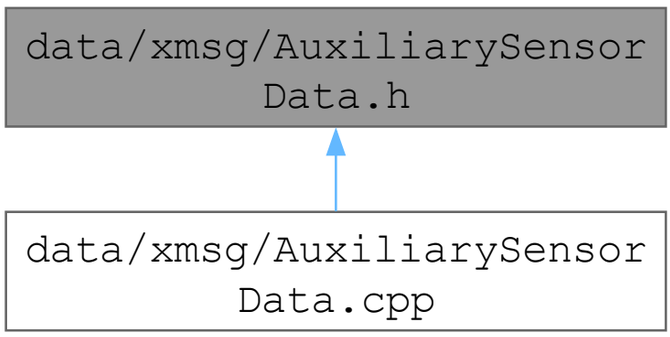
The util::xmsg::AuxiliarySensorData class header file.

```
#include "AbstractXSensorData.h"
```

Include dependency graph for AuxiliarySensorData.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::AuxiliarySensorData

  The auxiliary sensor data class.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.42.1 Detailed Description

The util::xmsg::AuxiliarySensorData class header file.

Version

0.6

Date

May 24, 2017 9:49:24 AM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.43 AuxiliarySensorData.h

Go to the documentation of this file.

```
00001 /** The @ref util::xmsg::AuxiliarySensorData class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       May 24, 2017 9:49:24 AM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef DATA_XMSG_AUXILIARYSENSORDATA_H_
00024 #define DATA_XMSG_AUXILIARYSENSORDATA_H_
00025
00026 /* Includes: */
00027
00028 #include "AbstractXSensorData.h"
00029
00030 /* Definitions: */
00031
00032 namespace util {
00033   namespace xmsg {
00034
00035     /* Types: */
00036
00037     /** The auxiliary sensor data class.
00038      *
00039      *  An instance of this class represents sensor data from an instrument's auxiliary sensor
00040      *  channel, decoded in the same way as the primary sensor data (see @ref AbstractXSensorData).
00041      */
00042     class AuxiliarySensorData:
00043         public util::xmsg::AbstractXSensorData {
00044
00045       private:
00046
00047         /* Private Data Members: */
00048
00049       public:
00050
00051         /* Public Member Functions: */
00052
00053         /** Default constructor.
00054          *
00055          */
00056         AuxiliarySensorData() {
```

```
00057
00058           /* Do nothing. */
00059
00060         }
00061
00062         /** Destructor.
00063          *
00064          */
00065         virtual ~AuxiliarySensorData() {
00066
00067           /* Do nothing. */
00068         }
00069
00070       protected:
00071
00072         /* Protected Member Functions: */
00073
00074         /** Unpack the sensor data from the binary message.
00075          *
00076          *  This field unpacks the encoded sensor data from the message and stores the decoded sensor
00077          *  values in @ref raw_sensor_data_. First the function checks the size of the message, and
00078          *  the data packing format. The message data is decoded according to the packing format type.
00079          *  Both first-difference and curvature encoding are supported. The encoding method is
00080          *  determined from the value of the high nibble of byte @c data[1].
00081          *
00082          *  Once this function completes successfully, then the data may be recovered in scientific
00083          *  units from the raw sensor values by applying the appropriate sensor gain and offset values.
00084          *
00085          *  @param[in]      data        The binary message to unpack, in a vector of unsigned bytes.
00086          *
00087          *  @warning        The function assumes the message data is valid. Message validity should
00088          *                  be ensured before this function is called.
00089          *
00090          *  @throw          util::log::MessageException if
00091          *                      - an error occurs while decoding the message,
00092          *                      - the message is of insufficient length, or
00093          *                      - an invalid packing format is encountered.
00094          *
00095          *  @return         The sensor data raw counts, in a vector of signed integers.
00096          */
00097         vector<int32_t> UnpackSensorData(const vector<uint8_t>& data) {
00098           try {
00099             // Check message length
00100             if(data.size() < 5) {
00101               ostringstream os;
00102               os « "Message too short! Expected at least 5 bytes, but only received "
00103                  « data.size()
00104                  « " bytes!";
00105             }
00106
00107             // Set the message type & index.
00108             set_id_xmsg_type(static_cast<XMsgTypeId_t>(data.at(0)));
00109             set_xmsg_index(0x0F & data.at(0));
00110
00111             // Clear previous, if any.
00112             raw_sensor_data_.clear();
00113
00114             /* Do Version-Specific Decoding: */
00115
00116             switch((data.at(1) & 0xF0) » 4) {
00117
00118               case XSENS_FMT_DIFFERENCE:
00119
00120                 /* Handle Fist-Difference Decoding: */
00121
00122                 try {
00123
00124                   ldbg() « "Unpacking first-difference-encoded sensor data..." « endl;
00125
00126                   uint32_t scale;
00127                   int32_t x0;
00128
00129                   // This message uses the first-difference packing method.
00130                   set_sensor_data_format(XSENS_FMT_DIFFERENCE);
00131
00132                   // Iterate over the sensor data.
00133                   for(vector<uint8_t>::const_iterator it = data.begin() + 3; it < data.end() - 5;) {
00134
00135                     // Parse sub-block header.
00136                     scale = *it++;
00137                     raw_sensor_data_.push_back(x0 = (*it++ « 8) | *it++);
00138                     size_t isub = XMSG_SENS_MAX_SUB;
00139
00140                     // Iterate over sub-block data.
00141                     while(it < data.end() - 1 && --isub) {
00142                       int32_t dx = *it++;
00143                       if(dx > 0x7F)
00144                         dx -= 0x0100;
00145                       raw_sensor_data_.push_back(x0 += dx * scale);
00146                     }
00147                   }
00148
00149                 }CATCH_BLOCK("Failed to unpack first-difference data!")
00150
00151                 break;
00152
00153               case XSENS_FMT_CURVATURE:
00154
00155                 /* Handle Curvature Decoding: */
00156
00157                 try {
00158
00159                   ldbg() « "Unpacking curvature-encoded sensor data..." « endl;
00160
00161                   /*  The data first differences.
00162                    *
00163                    *  The first difference are used to reconstruct the raw sensor data, and
00164                    *  are themselves constructed from the data second differences.
```

```
00165                  *
00166                  */
00167                 vector<int32_t> data_1st_differences;
00168
00169                 /*  The data second differences.
00170                  *
00171                  *  The encoded data represents the second differences of the raw sensor data.
00172                  *  The second differences are used to reconstruct the first differences,
00173                  *  which, in turn, are used to reconstruct the raw sensor data.
00174                  *
00175                  */
00176                 vector<int32_t> data_2nd_differences;
00177
00178                 /*  The message data iterator.
00179                  *
00180                  *  This field is used for iterating over the values of the sub-blocks of the
00181                  *  message. One nibble is read at a time, so this iterator is iterated for
00182                  *  every other nibble that is read.
00183                  */
00184                 vector<uint8_t>::const_iterator data_iterator;
00185
00186                 /*  The 32 3-bit packing factors.
00187                  *
00188                  *  The packing factor represents the dynamic range for each sub-block. The
00189                  *  packing factor represents the number of nibbles that is needed to store
00190                  *  the value.
00191                  */
00192                 uint8_t data_packing_factors[32];
00193                 memset(data_packing_factors, 0, sizeof(data_packing_factors));
00194
00195                 /*  The number of remaining values in the message.
00196                  *
00197                  *  This field is used as a control variable to determine when data unpacking
00198                  *  is complete.
00199                  *
00200                  */
00201                 size_t num_remaining_values = 0;
00202
00203                 /*  The total number of values in this message.
00204                  *
00205                  *  This field is parsed from the message header. It is used as a control
00206                  *  variable do determine when message unpacking is complete.
00207                  *
00208                  */
00209                 size_t num_values = 0;
00210
00211                 /*  The index of the sub block.
00212                  *
00213                  *  This field is used to track the index of the sub-block during iteration
00214                  *  of the message data. It is used as an index to the data_packing_factors[]
00215                  *  array.
00216                  */
00217                 size_t sub_block_index = 0;
00218
00219                 /*  The sub-block-nibble-is-high flag.
00220                  *
00221                  *  This field is used to keep track of whether we are reading the high or low
00222                  *  nibble of the data.
00223                  */
00224                 bool sub_block_nibble_is_high = true;
00225
00226                 /* The number of values in a sub-block.
00227                  *
00228                  * This field is used for iteration over the values in a sub-block.
00229                  *
00230                  */
00231                 size_t sub_block_num_values = 0;
00232
00233                 // This message uses the second-difference (curvature) packing method.
00234                 set_sensor_data_format(XSENS_FMT_CURVATURE);
00235
00236                 /* Parse the message header: */
00237
00238                 // Get the first sensor value.
00239                 const int32_t first_sensor_value = i24(data, 6);
00240
00241                 // Read the first difference value.
00242                 data_1st_differences.push_back(i24(data, 9));
00243
00244                 // Determine if the first sensor value should be included in the data.
00245                 switch(id_xmsg_type()) {
00246                   case XMSG_TYPE_ID_PRES0:            // 0x10
00247                   case XMSG_TYPE_ID_TEMP0:            // 0x20
00248                   case XMSG_TYPE_ID_SALN0:            // 0x30
00249                   case XMSG_TYPE_ID_PRES_HI_RES0:     // 0x90
00250                   case XMSG_TYPE_ID_TEMP_HI_RES0:     // 0xA0
00251                   case XMSG_TYPE_ID_SALN_HI_RES0:     // 0xB0
00252                   case XMSG_TYPE_ID_PRES_DRIFT8:      // 0x98
00253                   case XMSG_TYPE_ID_TEMP_DRIFT8:      // 0xA8
00254                   case XMSG_TYPE_ID_SALN_DRIFT8:      // 0xB8
00255                     raw_sensor_data_.push_back(first_sensor_value);
00256
00257                     break;
00258
00259                   default:
00260
00261                     /* Do nothing. */
00262
00263                     break;
00264                 }
00265
00266                 // Get the number of values.
00267                 switch(num_values = u16(data, 4)) {
00268                   case 0:
00269
00270                     /* Handle messages containing zero values: */
00271
00272                     // Throw exception.
```

```
00273                     EXC_THROW("Message contains 0 sensor values!");
00274                     break;
00275
00276                   case 2: {
00277
00278                     /* Handle messages containing two values: */
00279
00280                     // Compute the second sensor value from the first difference.
00281                     raw_sensor_data_.push_back(
00282                         first_sensor_value + data_1st_differences.at(0));
00283                   }
00284                     /* No break. */
00285
00286                   case 1:
00287
00288                     /* Handle messages containing one value: */
00289
00290                     // We have all the data for this message, so return.
00291                     return raw_sensor_data_;
00292
00293                   default:
00294
00295                     /* Handle messages containing three or more values: */
00296
00297                     raw_sensor_data_.reserve(num_values);
00298                     data_1st_differences.reserve(num_values);
00299                     data_2nd_differences.reserve(num_values);
00300                     break;
00301                 }
00302
00303                 /* Get the 32 3-bit packing factors from the 12-byte field: */
00304
00305                 // Iterate over the four 3-byte blocks.
00306                 for(size_t i = 0; i < 4; ++i)
00307
00308                   // Iterate over the eight values in each 3-byte block.
00309                   for(size_t j = 0; j < 8; ++j)
00310
00311                     // Parse the packing factor.
00312                     data_packing_factors[i * 8 + j] = (u24(data, 12 + i*3) >> (3 * (7 - j)))
00313                         & 0x07;
00314
00315 #                 if defined(XMSG_DEBUG_SENSOR_CURVATURE) && XMSG_DEBUG_SENSOR_CURVATURE
00316                   {
00317                     ostringstream os1, os2;
00318
00319                     os1 << "Decoding curvature data packing factors..."
00320                     << endl
00321                     << " Curvature Data Packing Factors:"
00322                     << endl
00323                     << "    ";
00324
00325                     static const format fmt1(" p[%2d]"), fmt2(" %5d");
00326
00327                     for(size_t i = 0; i < 32; ++i) {
00328                       os1 << format(fmt1) % i;
00329                       os2 << format(fmt2) % (unsigned int) data_packing_factors[i];
00330                     }
00331
00332                     ldbg() << os1.str() << endl << "    " << os2.str() << endl;
00333                   }
00334 #                 endif
00335
00336                   /* Unpack the message data: */
00337
00338 #                 if defined(XMSG_DEBUG_SENSOR_CURVATURE) && XMSG_DEBUG_SENSOR_CURVATURE
00339                   ostringstream xmsg_debug_os_;
00340                   xmsg_debug_os_ << "Decoding curvature data..." << endl;
00341 #                 endif
00342
00343                   // Iterate over the sub-blocks.
00344                   data_iterator = data.begin() + 24;
00345                   num_remaining_values = num_values - 2;
00346                   while(num_remaining_values > 0 && data_iterator < data.end() - 1) {
00347
00348                     try {
00349
00350                       // Get the data packing factor for this sub-block.
00351                       uint8_t data_packing_factor = data_packing_factors[sub_block_index];
00352
00353                       // Skip sub-blocks with data packing factor = 0.
00354                       if(!data_packing_factor) {
00355                         sub_block_index++;
00356                         continue;
00357                       }
00358
00359                       // Get the number of values in this sub-block.
00360                       sub_block_num_values = min(16, (int) num_remaining_values);
00361
00362 #                     if defined(XMSG_DEBUG_SENSOR_CURVATURE) && XMSG_DEBUG_SENSOR_CURVATURE
00363                       {
00364                         static const format fmt1(
00365                             " sub-block %2d (p: %u) %4d values remaining");
00366                         static const format fmt2(" %8d");
00367                         xmsg_debug_os_ << format(fmt1) % sub_block_index
00368                         % (unsigned int) data_packing_factors[sub_block_index]
00369                         % num_remaining_values
00370                         << endl
00371                         << "     ";
00372                         for(size_t i = 0; i < 16; ++i)
00373                         xmsg_debug_os_ << format(fmt2) % i;
00374                         xmsg_debug_os_ << endl << "    0x";
00375                       }
00376 #                     endif
00377
00378                       // Iterate over the values in this sub-block.
00379                       for(size_t sub_block_value_idx = 0;
00380                           sub_block_value_idx < sub_block_num_values
```

```
00381                             && data_iterator < data.end() - 1; ++sub_block_value_idx) {
00382                         try {
00383
00384 #                         if defined(XMSG_DEBUG_SENSOR_CURVATURE) && XMSG_DEBUG_SENSOR_CURVATURE
00385                           ostringstream xmsg_debug_nibbles_os_;
00386 #                         endif
00387
00388                           // Iterate over the nibbles of the value.
00389                           int32_t second_difference = 0;
00390
00391                           for(uint8_t value_nibble_index = 0;
00392                               value_nibble_index < data_packing_factor && data_iterator < data.end();
00393                               ++value_nibble_index) {
00394
00395                             // Read the nibble. If this is a low nibble, increment the iterator.
00396                             uint8_t byte = (*data_iterator);
00397                             uint8_t nibble = 0;
00398                             if(sub_block_nibble_is_high) {
00399                               nibble = byte » 4;
00400                             } else {
00401                               nibble = byte & 0x0F;
00402                               data_iterator++;
00403                             }
00404                             second_difference = (second_difference « 4) | nibble;
00405
00406                             // Update the nibble-is-high value.
00407                             sub_block_nibble_is_high = !sub_block_nibble_is_high;
00408
00409 #                                       if defined(XMSG_DEBUG_SENSOR_CURVATURE) &&
    XMSG_DEBUG_SENSOR_CURVATURE
00410                             {
00411                               static const format fmt("%X");
00412                               xmsg_debug_nibbles_os_ « format(fmt)
00413                               % (unsigned int) nibble;
00414                             }
00415 #                                       endif
00416
00417                           }
00418
00419                           // Correct for signed second differences.
00420                           if(second_difference >= (1 « (4 * data_packing_factor - 1)))
00421                             second_difference -= (1 « (4 * data_packing_factor));
00422
00423 #                                 if defined(XMSG_DEBUG_SENSOR_CURVATURE) && XMSG_DEBUG_SENSOR_CURVATURE
00424                           {
00425                             static const format fmt(" %8s");
00426                             xmsg_debug_os_ « format(fmt)
00427                             % xmsg_debug_nibbles_os_.str();
00428                           }
00429 #                                 endif
00430
00431                           // Append the computed second difference.
00432                           data_2nd_differences.push_back(second_difference);
00433
00434                           // Update the number of remaining values.
00435                           num_remaining_values--;
00436
00437                         }CATCH_BLOCK(
00438                             string("Failed to decoded the sensor value in sub-block: ")
00439                                 + lexical_cast<string>(sub_block_value_idx) + "!")
00440                       }
00441
00442 #                     if defined(XMSG_DEBUG_SENSOR_CURVATURE) && XMSG_DEBUG_SENSOR_CURVATURE
00443                       xmsg_debug_os_ « endl;
00444 #                     endif
00445
00446                       // Update the sub-block index.
00447                       sub_block_index++;
00448
00449                     }CATCH_BLOCK(
00450                         string("Failed to decode sub-block: ")
00451                             + lexical_cast<string>(sub_block_index) + "!")
00452
00453                   }
00454
00455                   /* Compute the sensor data values: */
00456
00457                   // Compute the first differences.
00458                   try {
00459                     int32_t first_difference = data_1st_differences.at(0);
00460                     for_each(
00461                         data_2nd_differences.begin(),
00462                         data_2nd_differences.end(),
00463                         [&](const int32_t second_difference) {
00464                           data_1st_differences.push_back((first_difference = first_difference +
    second_difference));
00465                         });
00466
00467                   }CATCH_BLOCK("Failed to compute the sensor data first differences!")
00468
00469                   try {
00470                     // Get the first sensor value.
00471                     int32_t sensor_value = first_sensor_value;
00472
00473                     // Compute the sensor values.
00474                     for_each(
00475                         data_1st_differences.begin(),
00476                         data_1st_differences.end(),
00477                         [&](const int32_t first_difference) {
00478                           raw_sensor_data_.push_back((sensor_value = sensor_value + first_difference));
00479                         });
00480                   }CATCH_BLOCK("Failed to compute the sensor data values!")
00481
00482 /* --- Start of XMSG_DEBUG_SENSOR_CURVATURE Block ------------------------------------------ */
00483 #                 if defined(XMSG_DEBUG_SENSOR_CURVATURE) && XMSG_DEBUG_SENSOR_CURVATURE
00484                   try {
00485
00486                     static const format fmt1("%20s %20s %20s %20s");
```

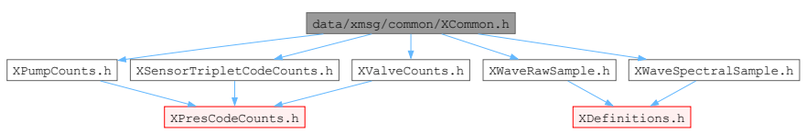
```
00487                     static const format fmt2("%20d %20d %20d %20d");
00488                     xmsg_debug_os_ « format(fmt1) % "idx" % "2nd" % "1st" % "value" « endl;
00489                     for(size_t i = 0; i < data_2nd_differences.size(); i++) {
00490                       xmsg_debug_os_ « format(fmt2) % i % data_2nd_differences.at(i)
00491                       % data_1st_differences.at(i)
00492                       % raw_sensor_data_.at(i)
00493                       « endl;
00494                     }
00495                     ldbg() « xmsg_debug_os_.str();
00496                   }CATCH_NOTHROW("Throw in XMSG_DEBUG_SENSOR_CURVATURE block!")
00497 #                 endif
00498 /* --- End of XMSG_DEBUG_SENSOR_CURVATURE Block ------------------------------------------------ */
00499
00500                   // Throw exception if we do not have all the variables.
00501                   if(num_remaining_values) {
00502                     ostringstream os;
00503                     os « "Reached end of message data before reading all values! "
00504                        « num_remaining_values
00505                        « " of "
00506                        « num_values
00507                        « " values remaining.";
00508                     EXC_THROW(os.str());
00509                   }
00510
00511                 }CATCH_BLOCK("Failed to unpack curvature data!")
00512
00513                 break;
00514
00515               default:
00516
00517                 /* Handle Unsupported Packing Formats: */
00518
00519                 // Throw exception
00520                 EXC_THROW("Unsupported packing format!");
00521                 break;
00522             }
00523
00524           }CATCH_BLOCK("Failed to unpack X-Message sensor data.")
00525
00526           return raw_sensor_data_;
00527         }
00528     };
00529
00530   /* Functions: */
00531
00532   } /* namespace xmsg */
00533 } /* namespace util */
00534 #endif /* DATA_XMSG_AUXILIARYSENSORDATA_H_ */
```

11.44 data/xmsg/common/XCommon.h File Reference

The XCommon source file.

```
#include "XPumpCounts.h"
#include "XSensorTripletCodeCounts.h"
#include "XValveCounts.h"
#include "XWaveRawSample.h"
#include "XWaveSpectralSample.h"
```

Include dependency graph for XCommon.h:



This graph shows which files directly or indirectly include this file:

11.44.1 Detailed Description

The XCommon source file.

Version

0.6

Date

Jul 8, 2014 10:43:47 AM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.45 XCommon.h

Go to the documentation of this file.

```
00001 /** The XCommon source file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jul 8, 2014 10:43:47 AM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef XCOMMON_H_
00024 #define XCOMMON_H_
00025
00026 /* Includes */
00027
00028 // local
00029 #include "XPumpCounts.h"
00030 #include "XSensorTripletCodeCounts.h"
00031 #include "XValveCounts.h"
00032 #include "XWaveRawSample.h"
00033 #include "XWaveSpectralSample.h"
00034 namespace x = util::xmsg;
00035
00036 #endif /* XCOMMON_H_ */
```

11.46 data/xmsg/common/XDefinitions.h File Reference

The XDefinitions header file.

```
#include "../../IridiumDefinitions.h"
#include "../../../config/LocalPersistentEntity.h"
#include "../../../utility/ContainerUtils.h"
#include "../../../utility/KmlUtil.h"
#include <boost/date_time/posix_time/ptime.hpp>
#include <boost/format/format_class.hpp>
#include <boost/format/format_fwd.hpp>
#include <boost/format/format_implementation.hpp>
#include <boost/format/free_funcs.hpp>
#include <cstdint>
#include <iomanip>
#include <iostream>
#include <sstream>
#include <string>
#include <vector>
```

```
#include <bit>
```

Include dependency graph for XDefinitions.h:

This graph shows which files directly or indirectly include this file:

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

Macros

- #define **f32**(x, n) util::math::MathUtils::bitcastUint32ToFloat(u32(x,n))
- #define i08(x, n) ((int8_t)x[n])

  Get signed one byte [n:n] value from array x.
- #define i16(x, n) ((int16_t)u16(x,n))

  Get signed two byte [n:n+2] value from array x.
- #define i24(x, n) (u24(x,n) >= (1<<23) ? ((int32_t)u24(x,n)) - (1 << 24) : u24(x,n))

  Get signed three byte [n:n+2] value from array x.
- #define i32(x, n) ((int32_t)u32(x,n))

  Get signed four byte [n:n+3] value from array x.
- #define s1970(x, n)

  Get posix time from given seconds since 1970-01-01T00:00:00Z.
- #define s2000(x, n)

  Get posix time from given seconds since 2000-01-01T00:00:00Z.
- #define u08(x, n) ((uint8_t)x[n])

  Get unsigned one byte [n:n] value from array x.
- #define u16(x, n) (((uint16_t)u08(x,(n)) << 8) | (uint16_t)u08(x,(n)+1))

  Get unsigned two byte [n:n+1] value from array x.
- #define u24(x, n) (((uint32_t)u08(x,(n)) << 16) | (uint32_t)(u08(x,(n)+1) << 8) | (uint32_t)u08(x,(n)+2))

  Get unsigned three byte [n:n+2] value from array x.
- #define u32(x, n) (((uint32_t)u16(x,(n)) << 16) | (uint32_t)u16(x,(n)+2))

  Get unsigned four byte [n:n+3] value from array x.
- #define XMSG_BAD_DBL DBL_MIN

  The default/bad/invalid/fill/unknown value for fields of type double.
- #define XMSG_BAD_FLT FLT_MIN

  The default/bad/invalid/fill/unknown value for fields of type float.
- #define XMSG_BAD_I08 INT8_MIN

  The default/bad/invalid/fill/unknown value for fields of type int8_t.
- #define XMSG_BAD_I16 INT16_MIN

  The default/bad/invalid/fill/unknown value for fields of type int16_t.
- #define XMSG_BAD_I32 INT32_MIN

  The default/bad/invalid/fill/unknown value for fields of type int32_t.
- #define XMSG_BAD_U08 UINT8_MAX

  The default/bad/invalid/fill/unknown value for fields of type uint8_t.
- #define XMSG_BAD_U16 UINT16_MAX

  The default/bad/invalid/fill/unknown value for fields of type uint16_t.
- #define XMSG_BAD_U32 UINT32_MAX

  The default/bad/invalid/fill/unknown value for fields of type uint32_t.
- #define XMSG_DEBUG_AGGREGATE_TIMESERIES_DATA (false)

The debug-aggregation-of-time-series-data flag.

- #define XMSG_DEBUG_ALAMO_MISS_GFG (false)

  The debug-alamo-mission-configuration flag.
- #define XMSG_DEBUG_BIST (true)

  The debug-bist flag.
- #define XMSG_DEBUG_ENG_SURF (false)

  The debug-surface-engineering flag.
- #define XMSG_DEBUG_MFA_MISS_GFG (true)

  The debug-MFA-mission-configuration flag.
- #define XMSG_DEBUG_SENSOR_CURVATURE (false)

  The debug-curvature-encoded-sensor-data flag.
- #define XMSG_DEBUG_SURF_PRES_AGGR_DATA (false)

  The debug-surface-data flag.
- #define XMSG_DIVE_UNKNOWN IRI_DIVE_UNKNOWN

  The default/bad/invalid dive value.
- #define XMSG_ENABLE_MULTI_PROF (false)

  Preprocessor switch enable multiple profile data message with the same unique message identifier to be included in the same dive data object.
- #define XMSG_ENG_BAD_VER XMSG_BAD_U08

  The default/bad/invalid ALAMO engineering message version.
- #define XMSG_GAIN_INVALID IRI_GAIN_INVALID

  Default/fill value for invalid Iridium message gain value.
- #define XMSG_HEADER(x)

  Macro to generate a section header for ASCII listing files.
- #define XMSG_HEADER2(x)

  Macro to generate a section sub-header for ASCII listing files.
- #define XMSG_ID_UNKNOWN IRI_ID_UNKNOWN

  The default/bad/invalid unique identifier value.
- #define XMSG_INIT_SAM_N_RESERVE (1024)

  The number of samples to reserve upon construction of the composite sensor data objects.
- #define XMSG_MAX_DRFT (8)

  The maximum number of drift data messages that may be received for a single sensor (e.g.
- #define XMSG_MAX_FALL (16)

  The maximum number of fall data messages that may be received for a single dive.
- #define XMSG_MAX_GPS (1024)

  The maximum number of GPS position messages that may be received for a single dive.
- #define XMSG_MAX_MAIL (256)

  The maximum number of email/SBD data messages that may be received for a single dive.
- #define XMSG_MAX_MFA_MISS (2)

  The maximum number of mission configuration messages that may be received for a single dive.
- #define XMSG_MAX_MISS (16)

  The maximum number of mission configuration messages that may be received for a single dive.
- #define XMSG_MAX_PROF (16)

  The maximum number of binned profile messages that may be received for a single sensor (e.g.
- #define XMSG_MAX_PROF_HIRES (8)

  The maximum number of unbinned or row profile messages that may be received for a single sensor (e.g.
- #define XMSG_MAX_PUMP (16)

  The maximum number of pump data messages that may be received for a single dive.
- #define XMSG_MAX_RISE (16)

  The maximum number of rise data messages that may be received for a single dive.
- #define XMSG_MAX_SURF_PRES (16)

  The maximum number of surface pressure data messages that may be received for a single dive.
- #define XMSG_MAX_VALVE (16)

  The maximum number of valve data messages that may be received for a single dive.
- #define XMSG_MAX_WAVE (256)

  The maximum number of ocean surface wave data messages that may be received for a single dive.
- #define XMSG_OFFS_INVALID IRI_OFFS_INVALID

  Default/fill value for invalid Iridium message offset value.
- #define XMSG_PHY(os, name, value) os << boost::format("%-39s %s") % name % value << "\n"

  Macro for stupid PHY file format lines.
- #define XMSG_SBD_MIN_BYTES (8)

  The minimum allowable number of bytes in an SBD message.
- #define XMSG_SECTION(x)

Macro to generate a section divider for ASCII listing files.

- #define XMSG_SECTION_BREAK()

  Macro to generate a section break for ASCII listing files.
- #define XMSG_SENS_BAD (-99999)

  The default/bad/invalid sensor counts fill value.
- #define XMSG_SENS_BAD_PRES (-9.999)

  The default/bad/invalid pressure sensor fill value.
- #define XMSG_SENS_BAD_SALN (-9.999)

  The default/bad/invalid salinity sensor fill value.
- #define XMSG_SENS_BAD_TEMP (-9.999)

  The default/bad/invalid temperature sensor fill value.
- #define XMSG_SENS_MAX_PRES (2610.000)

  The maximum valid pressure sensor value.
- #define XMSG_SENS_MAX_SALN (41.000)

  The maximum valid salinity sensor value.
- #define XMSG_SENS_MAX_SAMPLES (1024∗16)

  The maximum number of samples for a single instrument, sensor, and dive cycle.
- #define XMSG_SENS_MAX_SUB (25)

  The maximum number of sensor data samples in a profile data message sub-block.
- #define XMSG_SENS_MAX_TEMP (40.000)

  The maximum valid temperature sensor value.
- #define XMSG_SENS_MIN_PRES (-5.000)

  The minimum valid pressure sensor value.
- #define XMSG_SENS_MIN_SALN (2.000)

  The minimum valid salinity sensor value.
- #define XMSG_SENS_MIN_TEMP (-5.0)

  The minimum valid temperature sensor value.
- #define XMSG_VERSION_UNKNOWN (255)

  The unknown X-Message version value.

11.46.1 Detailed Description

The XDefinitions header file.

This file includes common preprocessor definitions and macros that are used by the X-Message classes.

Version

0.6

Date

Aug 14, 2014 11:37:15 AM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.46.2 Macro Definition Documentation

11.46.2.1 i08

```
#define i08(
              x,
              n) (( int8_t)x[n])
```

Get signed one byte [n:n] value from array x.

Referenced by util::xmsg::XGPS::DecodeXMsgData().

11.46.2.2 i16

```
#define i16(
              x,
              n) ((int16_t)u16(x,n))
```

Get signed two byte [n:n+2] value from array x.

Referenced by util::xmsg::XAlamoMissionConfiguration::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XMfaMissionCfg::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), and argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgSbdAttachment().

11.46.2.3 i24

```
#define i24(
              x,
              n) (u24(x,n) >= (1<<23) ?  ((int32_t)u24(x,n)) - (1 << 24) :  u24(x,n))
```

Get signed three byte [n:n+2] value from array x.

Referenced by util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

11.46.2.4 i32

```
#define i32(
              x,
              n) ((int32_t)u32(x,n))
```

Get signed four byte [n:n+3] value from array x.

Referenced by util::xmsg::XGPS::DecodeXMsgData().

11.46.2.5 s1970

```
#define s1970(
              x,
              n)
```

Value:

```
(u32(x,n) ? (boost::posix_time::ptime(\
boost::gregorian::date(1970,boost::date_time::Jan,1),\
boost::posix_time::seconds((long int)u32(x,n))))\
: (boost::posix_time::ptime(boost::posix_time::not_a_date_time)))
```

Get posix time from given seconds since 1970-01-01T00:00:00Z.

Referenced by util::xmsg::XAlamoBit2::DecodeXMsgData().

11.46.2.6 s2000

```
#define s2000(
              x,
              n)
```

Value:

```
(u32(x,n) ? (boost::posix_time::ptime(\
boost::gregorian::date(2000,boost::date_time::Jan,1),\
boost::posix_time::seconds((long int)u32(x,n))))\
: (boost::posix_time::ptime(boost::posix_time::not_a_date_time)))
```

Get posix time from given seconds since 2000-01-01T00:00:00Z.

Referenced by util::xmsg::XAlamoMissionConfiguration::DecodeXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(), util::xmsg::XEngDrift::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XMfaMissionCfg::DecodeXMsgData(), and util::xmsg::AbstractXFallRise::UnpackData().

11.46.2.7 u08

```
#define u08(
              x,
              n) ((uint8_t)x[n])
```

Get unsigned one byte [n:n] value from array x.

Referenced by util::xmsg::XAcousticPosition::DecodeXMsgData(), util::xmsg::XAlamoBit2::DecodeXMsgData(), util::xmsg::XAlamoBit::DecodeXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(), util::xmsg::XEngDrift::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XMfaMissionCfg::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), and argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgSbdAttachment().

11.46.2.8 u16

```
#define u16(
              x,
              n) (((uint16_t)u08(x,(n)) << 8) | (uint16_t)u08(x,(n)+1))
```

Get unsigned two byte [n:n+1] value from array x.

Referenced by util::xmsg::XMessage::CheckXMsgData(), argo::iridium::xfloat::XFloatDiveData::DecodeXMsgData(), util::xmsg::XAcousticCacst::DecodeXMsgData(), util::xmsg::XAlamoBit2::DecodeXMsgData(), util::xmsg::XAlamoBit::DecodeXMsgData(), util::xmsg::XAlamoMissionConfiguration::DecodeXMsgData(), util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XEngAscent::DecodeXM util::xmsg::XEngDescent::DecodeXMsgData(), util::xmsg::XEngDrift::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XMfaMissionCfg::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgSbdAttachment(), util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

11.46.2.9 u24

```
#define u24(
              x,
              n) (((uint32_t)u08(x,(n)) << 16) | (uint32_t)(u08(x,(n)+1) << 8) | (uint32_t)u08(x,(n)+2))
```

Get unsigned three byte [n:n+2] value from array x.

Referenced by util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

11.46.2.10 u32

```
#define u32(
              x,
              n) (((uint32_t)u16(x,(n)) << 16) | (uint32_t)u16(x,(n)+2))
```

Get unsigned four byte [n:n+3] value from array x.

Referenced by util::xmsg::XMfaMissionCfg::AssignMessage(), util::xmsg::XAcousticCacst::DecodeXMsgData(), util::xmsg::XAcousticWindow::DecodeXMsgData(), util::xmsg::XAlamoBit2::DecodeXMsgData(), util::xmsg::XAlamoBit::DecodeXMsgData util::xmsg::XAlamoMissionConfiguration::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngSurface::Decode and util::xmsg::XMfaMissionCfg::DecodeXMsgData().

11.46.2.11 XMSG_BAD_DBL

```
#define XMSG_BAD_DBL DBL_MIN
```

The default/bad/invalid/fill/unknown value for fields of type double.

11.46.2.12 XMSG_BAD_FLT

```
#define XMSG_BAD_FLT FLT_MIN
```

The default/bad/invalid/fill/unknown value for fields of type float.

11.46.2.13 XMSG_BAD_I08

```
#define XMSG_BAD_I08 INT8_MIN
```

The default/bad/invalid/fill/unknown value for fields of type int8_t.

11.46.2.14 XMSG_BAD_I16

```
#define XMSG_BAD_I16 INT16_MIN
```

The default/bad/invalid/fill/unknown value for fields of type int16_t.

11.46.2.15 XMSG_BAD_I32

```
#define XMSG_BAD_I32 INT32_MIN
```

The default/bad/invalid/fill/unknown value for fields of type int32_t.

11.46.2.16 XMSG_BAD_U08

```
#define XMSG_BAD_U08 UINT8_MAX
```

The default/bad/invalid/fill/unknown value for fields of type uint8_t.

11.46.2.17 XMSG_BAD_U16

```
#define XMSG_BAD_U16 UINT16_MAX
```

The default/bad/invalid/fill/unknown value for fields of type uint16_t.

Referenced by util::xmsg::XAlamoMissionConfiguration::DecodeXMsgData(), util::xmsg::XMfaMissionCfg::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(), and util::xmsg::XValve::DecodeXMsgData().

11.46.2.18 XMSG_BAD_U32

```
#define XMSG_BAD_U32 UINT32_MAX
```

The default/bad/invalid/fill/unknown value for fields of type uint32_t.

11.46.2.19 XMSG_DEBUG_AGGREGATE_TIMESERIES_DATA

```
#define XMSG_DEBUG_AGGREGATE_TIMESERIES_DATA (false)
```

The debug-aggregation-of-time-series-data flag.

If TRUE, then additional debug messages will be reported when decoding aggregate time-series data messages.

11.46.2.20 XMSG_DEBUG_ALAMO_MISS_GFG

```
#define XMSG_DEBUG_ALAMO_MISS_GFG (false)
```

The debug-alamo-mission-configuration flag.

If TRUE, then additional debug messages will be reported when decoding ALAMO mission configuration messages.

11.46.2.21 XMSG_DEBUG_BIST

```
#define XMSG_DEBUG_BIST (true)
```

The debug-bist flag.

If TRUE, then additional debug messages will be reported when decoding MFA mission configuration messages.

11.46.2.22 XMSG_DEBUG_ENG_SURF

```
#define XMSG_DEBUG_ENG_SURF (false)
```

The debug-surface-engineering flag.

11.46.2.23 XMSG_DEBUG_MFA_MISS_GFG

```
#define XMSG_DEBUG_MFA_MISS_GFG (true)
```

The debug-MFA-mission-configuration flag.

If TRUE, then additional debug messages will be reported when decoding MFA mission configuration messages.

11.46.2.24 XMSG_DEBUG_SENSOR_CURVATURE

```
#define XMSG_DEBUG_SENSOR_CURVATURE (false)
```

The debug-curvature-encoded-sensor-data flag.

If TRUE, then additional debug messages will be reported when decoding curvature-encoded sensor data.

11.46.2.25 XMSG_DEBUG_SURF_PRES_AGGR_DATA

```
#define XMSG_DEBUG_SURF_PRES_AGGR_DATA (false)
```

The debug-surface-data flag.

If TRUE, then additional debug messages will be reported when decoding surface data.

11.46.2.26 XMSG_DIVE_UNKNOWN

```
#define XMSG_DIVE_UNKNOWN IRI_DIVE_UNKNOWN
```

The default/bad/invalid dive value.

11.46.2.27 XMSG_ENABLE_MULTI_PROF

```
#define XMSG_ENABLE_MULTI_PROF (false)
```

Preprocessor switch enable multiple profile data message with the same unique message identifier to be included in the same dive data object.

Note

This switch is a temporary fix to include the full profile data in the intermediate listing for ALAMO #9038 which erroneously returns profile data messages with non-unique identifiers.

Warning

This switch should be disabled for the release build configuration.

If TRUE, then multiple profile data message with the same unique message identifier will be allowed for a single dive. Otherwise, messages with duplicate unique identifiers will overwrite or replace previously received messages with the same identifier.

11.46.2.28 XMSG_ENG_BAD_VER

```
#define XMSG_ENG_BAD_VER XMSG_BAD_U08
```

The default/bad/invalid ALAMO engineering message version.

11.46.2.29 XMSG_GAIN_INVALID

```
#define XMSG_GAIN_INVALID IRI_GAIN_INVALID
```

Default/fill value for invalid Iridium message gain value.

11.46.2.30 XMSG_HEADER

```
#define XMSG_HEADER(
                x)
```

Value:

```
do {\
    static const char* _line =
      "#=============================================================================================================";\
    os « _line « std::endl « "# " « x « std::endl « _line « std::endl;\
} while(0)
```

Macro to generate a section header for ASCII listing files.

Referenced by argo::iridium::xfloat::XFloatDiveData::GenerateListingString().

11.46.2.31 XMSG_HEADER2

```
#define XMSG_HEADER2(
                x)
```

Value:

```
do {\
    static const char* _line = "#============================================================"\
        "================================\n";\
    static const char* _sec = "# " x "\n";\
    os « _line  « _sec  « _line ;\
} while(0)
```

Macro to generate a section sub-header for ASCII listing files.

Referenced by argo::iridium::xfloat::XFloatDiveData::GenerateListingString().

11.46.2.32 XMSG_ID_UNKNOWN

```
#define XMSG_ID_UNKNOWN IRI_ID_UNKNOWN
```

The default/bad/invalid unique identifier value.

Referenced by argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgBody().

11.46.2.33 XMSG_INIT_SAM_N_RESERVE

```
#define XMSG_INIT_SAM_N_RESERVE (1024)
```

The number of samples to reserve upon construction of the composite sensor data objects.

Referenced by util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AbstractXS

11.46.2.34 XMSG_MAX_DRFT

```
#define XMSG_MAX_DRFT (8)
```

The maximum number of drift data messages that may be received for a single sensor (e.g.

pressure, temperature, or salinity) and for a single dive.

11.46.2.35 XMSG_MAX_FALL

```
#define XMSG_MAX_FALL (16)
```

The maximum number of fall data messages that may be received for a single dive.

Referenced by argo::iridium::xfloat::XFloatDiveData::GenerateCompositeFallTimeSeriesData().

11.46.2.36 XMSG_MAX_GPS

```
#define XMSG_MAX_GPS (1024)
```

The maximum number of GPS position messages that may be received for a single dive.

11.46.2.37 XMSG_MAX_MAIL

```
#define XMSG_MAX_MAIL (256)
```

The maximum number of email/SBD data messages that may be received for a single dive.

Referenced by argo::iridium::xfloat::XFloatDiveData::GenerateCompositeIridiumPosition().

11.46.2.38 XMSG_MAX_MFA_MISS

```
#define XMSG_MAX_MFA_MISS ( 2 )
```

The maximum number of mission configuration messages that may be received for a single dive.

Referenced by util::xmsg::XMfaMissionCfg::XMfaMissionCfg(), and util::xmsg::XMfaMissionCfg::AssignMessage().

11.46.2.39 XMSG_MAX_MISS

```
#define XMSG_MAX_MISS (16)
```

The maximum number of mission configuration messages that may be received for a single dive.

11.46.2.40 XMSG_MAX_PROF

```
#define XMSG_MAX_PROF (16)
```

The maximum number of binned profile messages that may be received for a single sensor (e.g.

pressure, temperature, or salinity) and for a single dive.

11.46.2.41 XMSG_MAX_PROF_HIRES

```
#define XMSG_MAX_PROF_HIRES (8)
```

The maximum number of unbinned or row profile messages that may be received for a single sensor (e.g.

pressure, temperature, or salinity) and for a single dive.

11.46.2.42 XMSG_MAX_PUMP

```
#define XMSG_MAX_PUMP (16)
```

The maximum number of pump data messages that may be received for a single dive.

Referenced by argo::iridium::xfloat::XFloatDiveData::GenerateCompositePumpTimeSeriesData().

11.46.2.43 XMSG_MAX_RISE

```
#define XMSG_MAX_RISE (16)
```

The maximum number of rise data messages that may be received for a single dive.

Referenced by argo::iridium::xfloat::XFloatDiveData::GenerateCompositeRiseTimeSeriesData().

11.46.2.44 XMSG_MAX_SURF_PRES

```
#define XMSG_MAX_SURF_PRES (16)
```

The maximum number of surface pressure data messages that may be received for a single dive.

11.46.2.45 XMSG_MAX_VALVE

```
#define XMSG_MAX_VALVE (16)
```

The maximum number of valve data messages that may be received for a single dive.

11.46.2.46 XMSG_MAX_WAVE

```
#define XMSG_MAX_WAVE (256)
```

The maximum number of ocean surface wave data messages that may be received for a single dive.

11.46.2.47 XMSG_OFFS_INVALID

```
#define XMSG_OFFS_INVALID IRI_OFFS_INVALID
```

Default/fill value for invalid Iridium message offset value.

11.46.2.48 XMSG_PHY

```
#define XMSG_PHY(
              os,
              name,
              value) os << boost::format("%-39s %s") % name % value << "\n"
```

Macro for stupid PHY file format lines.

Referenced by argo::iridium::xfloat::XFloatDiveData::GeneratePhyString().

11.46.2.49 XMSG_SBD_MIN_BYTES

```
#define XMSG_SBD_MIN_BYTES (8)
```

The minimum allowable number of bytes in an SBD message.

Any SBD message that has less than this number of bytes will be considered invalid and ignored.

Referenced by argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgSbdAttachment().

11.46.2.50 XMSG_SECTION

```
#define XMSG_SECTION(
              x)
```

Value:

```
do {\
    static const char* _line = "#----------------------------------------------------------------"\
        "---------------------------------\n";\
    static const char* _sec = "# " x "\n";\
    os « _line « _sec  « _line ;\
} while(0)
```

Macro to generate a section divider for ASCII listing files.

Referenced by argo::iridium::xfloat::XFloatDiveData::GenerateListingString(), and util::xmsg::XCompositeSurfaceData::GenerateListingStr

11.46.2.51 XMSG_SECTION_BREAK

```
#define XMSG_SECTION_BREAK()
```

Value:

```
do {\
    static const char* _line = "#----------------------------------------------------------------"\
        "---------------------------------\n";\
    os « _line;\
} while(0)
```

Macro to generate a section break for ASCII listing files.

Referenced by util::xmsg::XCompositeFall::GenerateListingString(), util::xmsg::XCompositePump::GenerateListingString(), and util::xmsg::XCompositeRise::GenerateListingString().

11.46.2.52 XMSG_SENS_BAD

```
#define XMSG_SENS_BAD (-99999)
```

The default/bad/invalid sensor counts fill value.

Referenced by util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::AggregateSensorData(), util::xmsg::XCompositeSurfa util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::ResetAggregatedData(), util::xmsg::AbstractXSensorDataComposite util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::SamplePres(), util::xmsg::AbstractXSensorDataComposite< PRES_ util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::SampleSaln(), util::xmsg::AbstractXSensorDataComposite< PRES_ util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::SampleTemp(), and util::xmsg::AbstractXFallRise::UnpackData().

11.46.2.53 XMSG_SENS_BAD_PRES

```
#define XMSG_SENS_BAD_PRES (-9.999)
```

The default/bad/invalid pressure sensor fill value.

Referenced by util::xmsg::XPresCounts::PressureValue(), util::xmsg::XPumpCounts::PressureValue(), util::xmsg::XValveCounts::Pressure
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SamplePresAsFloat(),
util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::SamplePresAsFloat(), util::xmsg::AbstractXSensorData::SampleValu
and argo::iridium::xfloat::XFloatDiveData::UpdateDerivedSurfaceDataFields().

11.46.2.54 XMSG_SENS_BAD_SALN

```
#define XMSG_SENS_BAD_SALN (-9.999)
```

The default/bad/invalid salinity sensor fill value.

Referenced by util::xmsg::XSensorTripletCodeCounts::SalinityValue(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN
util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::SampleSalnAsFloat(), and util::xmsg::AbstractXSensorData::SampleV

11.46.2.55 XMSG_SENS_BAD_TEMP

```
#define XMSG_SENS_BAD_TEMP (-9.999)
```

The default/bad/invalid temperature sensor fill value.

Referenced by util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleTem
util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::SampleTempAsFloat(), util::xmsg::AbstractXSensorData::SampleValu
and util::xmsg::XSensorTripletCodeCounts::TemperatureValue().

11.46.2.56 XMSG_SENS_MAX_PRES

```
#define XMSG_SENS_MAX_PRES (2610.000)
```

The maximum valid pressure sensor value.

Referenced by util::xmsg::XPresCounts::PressureValue(), util::xmsg::XPumpCounts::PressureValue(), util::xmsg::XValveCounts::Pressure
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SamplePresAsFloat(),
util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::SamplePresAsFloat(), and util::xmsg::AbstractXSensorData::SampleV

11.46.2.57 XMSG_SENS_MAX_SALN

```
#define XMSG_SENS_MAX_SALN (41.000)
```

The maximum valid salinity sensor value.

Referenced by util::xmsg::XSensorTripletCodeCounts::SalinityValue(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN
util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::SampleSalnAsFloat(), and util::xmsg::AbstractXSensorData::SampleV

11.46.2.58 XMSG_SENS_MAX_SAMPLES

```
#define XMSG_SENS_MAX_SAMPLES (1024*16)
```

The maximum number of samples for a single instrument, sensor, and dive cycle.

11.46.2.59 XMSG_SENS_MAX_SUB

```
#define XMSG_SENS_MAX_SUB (25)
```

The maximum number of sensor data samples in a profile data message sub-block.

Referenced by util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::UnpackSensorData().

11.46.2.60 XMSG_SENS_MAX_TEMP

```
#define XMSG_SENS_MAX_TEMP (40.000)
```

The maximum valid temperature sensor value.

Referenced by util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::SampleTem
util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::SampleTempAsFloat(), util::xmsg::AbstractXSensorData::SampleValu
and util::xmsg::XSensorTripletCodeCounts::TemperatureValue().

11.46.2.61 XMSG_SENS_MIN_PRES

```
#define XMSG_SENS_MIN_PRES (-5.000)
```

The minimum valid pressure sensor value.

11.46.2.62 XMSG_SENS_MIN_SALN

```
#define XMSG_SENS_MIN_SALN (2.000)
```

The minimum valid salinity sensor value.

11.46.2.63 XMSG_SENS_MIN_TEMP

```
#define XMSG_SENS_MIN_TEMP (-5.0)
```

The minimum valid temperature sensor value.

11.46.2.64 XMSG_VERSION_UNKNOWN

```
#define XMSG_VERSION_UNKNOWN (255)
```

The unknown X-Message version value.

11.47 XDefinitions.h

Go to the documentation of this file.

```
00001 /** The XDefinitions header file.
00002  *
00003  *  This file includes common preprocessor definitions and macros that are used by the X-Message
00004  *  classes.
00005  *
00006  *  @file
00007  *
00008  *  @version    0.6
00009  *
00010  *  @date       Aug 14, 2014 11:37:15 AM
00011  *
00012  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00013  *              Engineer I,
00014  *              Department of Physical Oceanography,
00015  *              Woods Hole Oceanographic Institution
00016  *
00017  *  @copyright  2014 Woods Hole Oceanographic Institution.
00018  *              Released under the GNU General Public License.
00019  *
00020  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00021  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00022  *  General Public License for more details. You should have received a copy of the GNU General
00023  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00024  */
00025
00026 #ifndef XINCLUDES_H_
00027 #define XINCLUDES_H_
00028
00029 /* Includes: */
00030
00031 #include "../../IridiumDefinitions.h"              // local libs
00032 #include "../../../config/LocalPersistentEntity.h"
00033 #include "../../../utility/ContainerUtils.h"
00034 #include "../../../utility/KmlUtil.h"
00035
00036 #include <boost/date_time/posix_time/ptime.hpp>     // boost libs
00037 #include <boost/format/format_class.hpp>
00038 #include <boost/format/format_fwd.hpp>
```

```
00039 #include <boost/format/format_implementation.hpp>
00040 #include <boost/format/free_funcs.hpp>
00041
00042 #include <cstdint>                                  // std libs
00043 #include <iomanip>
00044 #include <iostream>
00045 #include <sstream>
00046 #include <string>
00047 #include <vector>
00048 #include <bit>
00049
00050 /* Definitions */
00051
00052 /** The default/bad/invalid/fill/unknown value for fields of type float. */
00053 #define XMSG_BAD_FLT                            FLT_MIN
00054
00055 /** The default/bad/invalid/fill/unknown value for fields of type double. */
00056 #define XMSG_BAD_DBL                            DBL_MIN
00057
00058 /** The default/bad/invalid/fill/unknown value for fields of type int8_t. */
00059 #define XMSG_BAD_I08                            INT8_MIN
00060
00061 /** The default/bad/invalid/fill/unknown value for fields of type int16_t. */
00062 #define XMSG_BAD_I16                            INT16_MIN
00063
00064 /** The default/bad/invalid/fill/unknown value for fields of type int32_t. */
00065 #define XMSG_BAD_I32                            INT32_MIN
00066
00067 /** The default/bad/invalid/fill/unknown value for fields of type uint8_t. */
00068 #define XMSG_BAD_U08                            UINT8_MAX
00069
00070 /** The default/bad/invalid/fill/unknown value for fields of type uint16_t. */
00071 #define XMSG_BAD_U16                            UINT16_MAX
00072
00073 /** The default/bad/invalid/fill/unknown value for fields of type uint32_t. */
00074 #define XMSG_BAD_U32                            UINT32_MAX
00075
00076 /** The debug-alamo-mission-configuration flag.
00077  *
00078  *  If TRUE, then additional debug messages will be reported when decoding ALAMO mission
00079  *  configuration messages.
00080  */
00081 #define XMSG_DEBUG_ALAMO_MISS_GFG               (false)
00082
00083 /** The debug-bist flag.
00084  *
00085  *  If TRUE, then additional debug messages will be reported when decoding MFA mission
00086  *  configuration messages.
00087  */
00088 #define XMSG_DEBUG_BIST                         (true)
00089
00090 /** The debug-MFA-mission-configuration flag.
00091  *
00092  *  If TRUE, then additional debug messages will be reported when decoding MFA mission
00093  *  configuration messages.
00094  */
00095 #define XMSG_DEBUG_MFA_MISS_GFG                 (true)
00096
00097 /** The debug-aggregation-of-time-series-data flag.
00098  *
00099  *  If TRUE, then additional debug messages will be reported when decoding aggregate time-series
00100  *  data messages.
00101  */
00102 #define XMSG_DEBUG_AGGREGATE_TIMESERIES_DATA    (false)
00103
00104 /** The debug-surface-engineering flag.
00105  *
00106  */
00107 #define XMSG_DEBUG_ENG_SURF                     (false)
00108
00109 /** The debug-curvature-encoded-sensor-data flag.
00110  *
00111  *  If TRUE, then additional debug messages will be reported when decoding curvature-encoded sensor
00112  *  data.
00113  */
00114 #define XMSG_DEBUG_SENSOR_CURVATURE             (false)
00115
00116 /** The debug-surface-data flag.
00117  *
00118  *  If TRUE, then additional debug messages will be reported when decoding surface data.
00119  */
00120 #define XMSG_DEBUG_SURF_PRES_AGGR_DATA          (false)
00121
00122 /** The default/bad/invalid dive value. */
00123 #define XMSG_DIVE_UNKNOWN                       IRI_DIVE_UNKNOWN
00124
00125 /** Preprocessor switch enable multiple profile data message with the same unique message
00126  *  identifier to be included in the same dive data object.
00127  *
00128  *  @note           This switch is a temporary fix to include the full profile data in the
00129  *                  intermediate listing for ALAMO #9038 which erroneously returns profile data
00130  *                  messages with non-unique identifiers.
00131  *
00132  *  @warning        This switch should be disabled for the release build configuration.
00133  *
00134  *  If TRUE, then multiple profile data message with the same unique message identifier will be
00135  *  allowed for a single dive. Otherwise, messages with duplicate unique identifiers will
00136  *  overwrite or replace previously received messages with the same identifier.
00137  */
00138 #define XMSG_ENABLE_MULTI_PROF                  (false)
00139
00140 /** Default/fill value for invalid Iridium message gain value. */
00141 #define XMSG_GAIN_INVALID                       IRI_GAIN_INVALID
00142
00143 /** The default/bad/invalid unique identifier value. */
00144 #define XMSG_ID_UNKNOWN                         IRI_ID_UNKNOWN
00145
00146 /** The number of samples to reserve upon construction of the composite sensor data objects.
```

```
00147  *
00148  */
00149 #define XMSG_INIT_SAM_N_RESERVE                 (1024)
00150
00151 /** Default/fill value for invalid Iridium message offset value. */
00152 #define XMSG_OFFS_INVALID                       IRI_OFFS_INVALID
00153
00154 /** The default/bad/invalid ALAMO engineering message version. */
00155 #define XMSG_ENG_BAD_VER                        XMSG_BAD_U08
00156
00157 /** The minimum allowable number of bytes in an SBD message.
00158  *
00159  *  Any SBD message that has less than this number of bytes will be considered invalid and ignored.
00160  *
00161  */
00162 #define XMSG_SBD_MIN_BYTES                      (8)
00163
00164 /** The default/bad/invalid sensor counts fill value. */
00165 #define XMSG_SENS_BAD                           (-99999)
00166
00167 /** The default/bad/invalid pressure sensor fill value. */
00168 #define XMSG_SENS_BAD_PRES                      (-9.999)
00169
00170 /** The default/bad/invalid salinity sensor fill value. */
00171 #define XMSG_SENS_BAD_SALN                      (-9.999)
00172
00173 /** The default/bad/invalid temperature sensor fill value. */
00174 #define XMSG_SENS_BAD_TEMP                      (-9.999)
00175
00176 /** The minimum valid pressure sensor value. */
00177 #define XMSG_SENS_MIN_PRES                      (-5.000)
00178
00179 /** The minimum valid salinity sensor value. */
00180 #define XMSG_SENS_MIN_SALN                      (2.000)
00181
00182 /** The minimum valid temperature sensor value. */
00183 #define XMSG_SENS_MIN_TEMP                      (-5.0)
00184
00185 /** The maximum valid pressure sensor value. */
00186 #define XMSG_SENS_MAX_PRES                      (2610.000)
00187
00188 /** The maximum valid salinity sensor value. */
00189 #define XMSG_SENS_MAX_SALN                      (41.000)
00190
00191 /** The maximum number of samples for a single instrument, sensor, and dive cycle.
00192  *
00193  */
00194 #define XMSG_SENS_MAX_SAMPLES                   (1024*16)
00195
00196 /** The maximum number of sensor data samples in a profile data message sub-block. */
00197 #define XMSG_SENS_MAX_SUB                       (25)
00198
00199 /** The maximum valid temperature sensor value. */
00200 #define XMSG_SENS_MAX_TEMP                      (40.000)
00201
00202 /** The maximum number of drift data messages that may be received for a single sensor (e.g.
00203  *  pressure, temperature, or salinity) and for a single dive.
00204  */
00205 #define XMSG_MAX_DRFT                           (8)
00206
00207 /** The maximum number of fall data messages that may be received for a single dive. */
00208 #define XMSG_MAX_FALL                           (16)
00209
00210 /** The maximum number of email/SBD data messages that may be received for a single dive. */
00211 #define XMSG_MAX_MAIL                           (256)
00212
00213 /** The maximum number of mission configuration messages that may be received for a single dive. */
00214 #define XMSG_MAX_MISS                           (16)
00215
00216 /** The maximum number of mission configuration messages that may be received for a single dive. */
00217 #define XMSG_MAX_MFA_MISS                       ( 2 )
00218
00219 #if defined(XMSG_ENABLE_MULTI_PROF) && XMSG_ENABLE_MULTI_PROF
00220
00221 /** The maximum number of GPS position messages that may be received for a single dive. */
00222 #   define XMSG_MAX_GPS                         (1024)
00223
00224 /** The maximum number of binned profile messages that may be received for a single sensor (e.g.
00225  *  pressure, temperature, or salinity) and for a single dive.
00226  */
00227 #   define XMSG_MAX_PROF                        (256)
00228
00229 /** The maximum number of unbinned or row profile messages that may be received for a single sensor
00230  *  (e.g. pressure, temperature, or salinity) and for a single dive.
00231  */
00232 #   define XMSG_MAX_PROF_HIRES                  (256)
00233
00234 #else
00235
00236 /** The maximum number of GPS position messages that may be received for a single dive. */
00237 #   define XMSG_MAX_GPS                         (1024)
00238
00239 /** The maximum number of binned profile messages that may be received for a single sensor (e.g.
00240  *  pressure, temperature, or salinity) and for a single dive.
00241  */
00242 #   define XMSG_MAX_PROF                        (16)
00243
00244 /** The maximum number of unbinned or row profile messages that may be received for a single sensor
00245  *  (e.g. pressure, temperature, or salinity) and for a single dive.
00246  */
00247 #   define XMSG_MAX_PROF_HIRES                  (8)
00248
00249 #endif // defined(XMSG_ENABLE_MULTI_PROF) && XMSG_ENABLE_MULTI_PROF
00250
00251 /** The maximum number of pump data messages that may be received for a single dive. */
00252 #define XMSG_MAX_PUMP                           (16)
00253
00254 /** The maximum number of rise data messages that may be received for a single dive. */
```

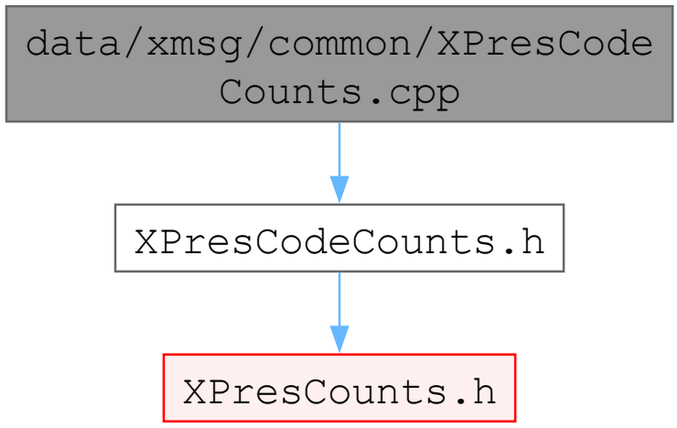
```
00255 #define XMSG_MAX_RISE                           (16)
00256
00257 /** The maximum number of surface pressure data messages that may be received for a single dive. */
00258 #define XMSG_MAX_SURF_PRES                      (16)
00259
00260 /** The maximum number of valve data messages that may be received for a single dive. */
00261 #define XMSG_MAX_VALVE                          (16)
00262
00263 /** The maximum number of ocean surface wave data messages that may be received for a single dive.
00264  *
00265  */
00266 #define XMSG_MAX_WAVE                           (256)
00267
00268 /** The unknown X-Message version value.
00269  *
00270  */
00271 #define XMSG_VERSION_UNKNOWN                    (255)
00272
00273 /* Macro Definitions: */
00274
00275 /** Macro for stupid PHY file format lines. */
00276 #define XMSG_PHY(os, name, value)   os « boost::format("%-39s %s") % name % value « "\n"
00277
00278 /** Macro to generate a section divider for ASCII listing files. */
00279 #define XMSG_SECTION(x)\
00280 do {\
00281     static const char* _line = "#-----------------------------------------------------------------\"\
00282         "---------------------------------\n";\
00283     static const char* _sec = "# " x "\n";\
00284     os « _line « _sec  « _line ;\
00285 } while(0)
00286
00287 /** Macro to generate a section break for ASCII listing files. */
00288 #define XMSG_SECTION_BREAK()\
00289 do {\
00290     static const char* _line = "#-----------------------------------------------------------------\"\
00291         "---------------------------------\n";\
00292     os « _line;\
00293 } while(0)
00294
00295 /** Macro to generate a section sub-header for ASCII listing files. */
00296 #define XMSG_HEADER2(x)\
00297 do {\
00298     static const char* _line = "#=================================================================\"\
00299         "=================================\n";\
00300     static const char* _sec = "# " x "\n";\
00301     os « _line  « _sec  « _line ;\
00302 } while(0)
00303
00304 /** Macro to generate a section header for ASCII listing files. */
00305 #define XMSG_HEADER(x)\
00306 do {\
00307     static const char* _line =
    "#=============================================================================================================";\
00308     os « _line « std::endl « "# " « x « std::endl « _line « std::endl;\
00309 } while(0)
00310
00311 /** Get unsigned one byte [n:n] value from array x. */
00312 #define u08(x,n)    ((uint8_t)x[n])
00313
00314 /** Get unsigned two byte [n:n+1] value from array x. */
00315 #define u16(x,n)    (((uint16_t)u08(x,(n)) «  8) | (uint16_t)u08(x,(n)+1))
00316
00317 /** Get unsigned three byte [n:n+2] value from array x. */
00318 #define u24(x,n)    (((uint32_t)u08(x,(n)) « 16) | (uint32_t)(u08(x,(n)+1) « 8) | (uint32_t)u08(x,(n)+2))
00319
00320 /** Get unsigned four byte [n:n+3] value from array x. */
00321 #define u32(x,n)    (((uint32_t)u16(x,(n)) « 16) | (uint32_t)u16(x,(n)+2))
00322
00323 /** Get signed one byte [n:n] value from array x. */
00324 #define i08(x,n)    (( int8_t)x[n])
00325
00326 /** Get signed two byte [n:n+2] value from array x. */
00327 #define i16(x,n)    ((int16_t)u16(x,n))
00328
00329 /** Get signed three byte [n:n+2] value from array x. */
00330 #define i24(x,n)    (u24(x,n) >= (1«23) ? ((int32_t)u24(x,n)) - (1 « 24) : u24(x,n))
00331
00332 /** Get signed four byte [n:n+3] value from array x. */
00333 #define i32(x,n)    ((int32_t)u32(x,n))
00334
00335 /** Get posix time from given seconds since 2000-01-01T00:00:00Z. */
00336 #define s2000(x,n)  (u32(x,n) ? (boost::posix_time::ptime(\
00337                         boost::gregorian::date(2000,boost::date_time::Jan,1),\
00338                         boost::posix_time::seconds((long int)u32(x,n))))\
00339                         : (boost::posix_time::ptime(boost::posix_time::not_a_date_time)))
00340
00341 /** Get posix time from given seconds since 1970-01-01T00:00:00Z. */
00342 #define s1970(x,n)  (u32(x,n) ? (boost::posix_time::ptime(\
00343                         boost::gregorian::date(1970,boost::date_time::Jan,1),\
00344                         boost::posix_time::seconds((long int)u32(x,n))))\
00345                         : (boost::posix_time::ptime(boost::posix_time::not_a_date_time)))
00346
00347 //#define f32(x,n)    (*((float *)((void *)(&x[n]))))
00348
00349 #define f32(x,n)    util::math::MathUtils::bitcastUint32ToFloat(u32(x,n))
00350
00351 namespace util {
00352 namespace xmsg {
00353
00354 /* Namespace Declarations: */
00355
00356 using boost::algorithm::trim;
00357 using boost::container::flat_map;
00358 using boost::container::static_vector;
00359 using boost::filesystem::exists;
00360 using boost::filesystem::file_size;
00361 using boost::filesystem::ifstream;
```

```
00362 using boost::filesystem::is_empty;
00363 using boost::filesystem::is_regular_file;
00364 using boost::filesystem::ofstream;
00365 using boost::filesystem::path;
00366 using boost::format;
00367 using boost::gregorian::date;
00368 using boost::lexical_cast;
00369 using std::lock_guard;
00370 using std::make_shared;
00371 using boost::posix_time::hours;
00372 using boost::posix_time::minutes;
00373 using boost::posix_time::not_a_date_time;
00374 using boost::posix_time::ptime;
00375 using boost::posix_time::seconds;
00376 using boost::posix_time::time_duration;
00377 using std::recursive_mutex;
00378 using std::shared_ptr;
00379 using std::cmatch;
00380 using std::dec;
00381 using std::endl;
00382 using std::for_each;
00383 using std::get;
00384 using std::hex;
00385 using std::istream;
00386 using std::istringstream;
00387 using std::make_tuple;
00388 using std::max;
00389 using std::min;
00390 using std::noshowbase;
00391 using std::ostream;
00392 using std::ostringstream;
00393 using std::setfill;
00394 using std::setw;
00395 using std::smatch;
00396 using std::string;
00397 using std::stringstream;
00398 using std::tuple;
00399 using std::regex;
00400 using std::regex_match;
00401 using std::regex_replace;
00402 using std::regex_search;
00403 using std::uppercase;
00404 using std::vector;
00405
00406 } /* namespace util */
00407 } /* namespace xmsg */
00408
00409 #endif /* XINCLUDES_H_ */
```

11.48 data/xmsg/common/XPresCodeCounts.cpp File Reference

The util::xmsg::XPresCodeCounts class source file.

`#include "XPresCodeCounts.h"`
Include dependency graph for XPresCodeCounts.cpp:



Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.48.1 Detailed Description

The util::xmsg::XPresCodeCounts class source file.

Version

0.6

Date

Oct 16, 2014 1:52:44 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

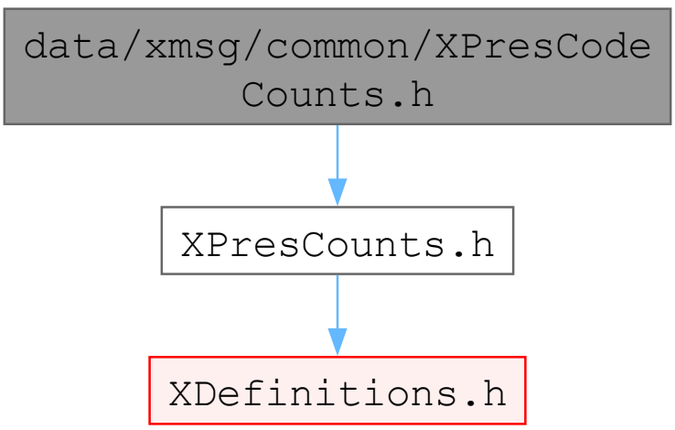
This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.49 data/xmsg/common/XPresCodeCounts.h File Reference

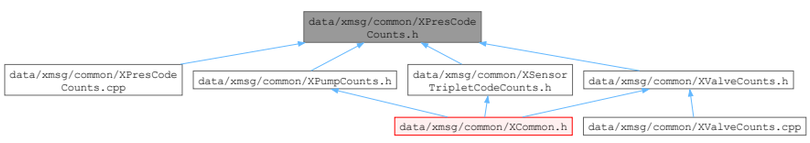
The util::xmsg::XPresCodeCounts class header file.

```
#include "XPresCounts.h"
```

Include dependency graph for XPresCodeCounts.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XPresCodeCounts

  Class representing a raw pressure measurement and phase code for a rise or fall data X-Message.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

Enumerations

- enum util::xmsg::XPhaseCode_t {
  util::xmsg::XPHASE_CODE_UNKNOWN = 0 ,
  util::xmsg::XPHASE_CODE_START_OF_SINK = 1 ,
  util::xmsg::XPHASE_CODE_BUOYANCY_AT_100DBAR = 2 ,
  util::xmsg::XPHASE_CODE_SEEK = 3 ,
  util::xmsg::XPHASE_CODE_BEGINNING_OF_DRIFT = 4 ,
  util::xmsg::XPHASE_CODE_SEEK_DURING_DRIFT = 5 ,
  util::xmsg::XPHASE_CODE_BEGINING_OF_FALL_TO_PROFILE = 6 ,
  util::xmsg::XPHASE_CODE_START_OF_RISE = 7 ,
  util::xmsg::XPHASE_CODE_END_OF_RISE = 8 ,
  util::xmsg::XPHASE_CODE_ICE_TURNAROUND = 9 ,
  util::xmsg::XPHASE_CODE_SINKING = 10 ,
  util::xmsg::XPHASE_CODE_DRIFTING = 11 ,
  util::xmsg::XPHASE_CODE_FALLING_TO_PROFILE = 12 ,
  util::xmsg::XPHASE_CODE_RISING = 13 ,
  util::xmsg::XPHASE_CODE_SURFACE = 14 ,
  util::xmsg::XPHASE_CODE_INVALID = 15 }

  The X-message phase codes enumeration.

11.49.1 Detailed Description

The util::xmsg::XPresCodeCounts class header file.

Version

0.6

Date

Jul 8, 2014 10:47:35 AM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.50 XPresCodeCounts.h

Go to the documentation of this file.

```
00001 /** The @ref util::xmsg::XPresCodeCounts class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jul 8, 2014 10:47:35 AM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef XPRESCOUNTCODES_H_
00024 #define XPRESCOUNTCODES_H_
00025
00026 /* Includes: */
00027
00028 // local
00029 #include "XPresCounts.h"
00030
00031 /* Definitions */
00032
00033 namespace util {
00034 namespace xmsg {
00035
00036 /* Type Definitions: */
00037
00038 /** The X-message phase codes enumeration.
00039  *
00040  *  The phase code is used to indicate the context of a measurement i.e. what phase of the dive
00041  *  cycle the measurement was taken.
00042  *
00043  */
00044 typedef enum XPhaseCode {
00045
00046     /** Phase code 0: unknown mission phase.
00047      *
00048      *  This code is only used for instruments with an X-Message format previous to version 2.0.
00049      *
00050      */
00051     XPHASE_CODE_UNKNOWN = 0,
00052
00053     /** Phase code 1: start of the descent-from-surface phase of the dive cycle.
00054      *
00055      */
00056     XPHASE_CODE_START_OF_SINK = 1,
00057
00058     /** Phase code 2: first pump (Tlastx) at 100 dBar.
00059      *
00060      */
00061     XPHASE_CODE_BUOYANCY_AT_100DBAR = 2,
00062
00063     /** Phase code 3: seek during the descent-to-drift/park-pressure.
00064      *
00065      */
00066     XPHASE_CODE_SEEK = 3,
00067
00068     /** Phase code 4: the beginning of the drift phase of the dive cycle.
00069      *
00070      */
00071     XPHASE_CODE_BEGINNING_OF_DRIFT = 4,
00072
00073     /** Phase code 5: seek to drift/park pressure during the drift phase of the dive cycle.
00074      *
00075      */
00076     XPHASE_CODE_SEEK_DURING_DRIFT = 5,
00077
00078     /** Phase code 6: beginning of the descent from drift/park depth to profile depth.
00079      *
00080      */
00081     XPHASE_CODE_BEGINING_OF_FALL_TO_PROFILE = 6,
00082
00083     /** Phase code 7: start of the ascent/profile/rise phase of the dive cycle.
00084      *
00085      */
00086     XPHASE_CODE_START_OF_RISE = 7,
00087
00088     /** Phase code 8: end of the ascent/profile/rise phase of the dive cycle.
00089      *
00090      */
00091     XPHASE_CODE_END_OF_RISE = 8,
00092
00093     /** Phase code 9: turn-around due to the ice detection algorithm.
00094      *
00095      */
00096     XPHASE_CODE_ICE_TURNAROUND = 9,
00097
00098     /** Phase code 10: the descent-from-surface phase of the dive cycle.
00099      *
00100      */
00101     XPHASE_CODE_SINKING = 10,
00102
00103     /** Phase code 11: the drift phase of the dive cycle.
```

```
00104      *
00105      */
00106     XPHASE_CODE_DRIFTING = 11,
00107
00108     /** Phase code 12: the descent from drift/park depth to profile depth.
00109      *
00110      */
00111     XPHASE_CODE_FALLING_TO_PROFILE = 12,
00112
00113     /** Phase code 13:  the ascent/profile/rise phase of the dive cycle.
00114      *
00115      */
00116     XPHASE_CODE_RISING = 13,
00117
00118     /** Phase code 14: the surface phase of the dive cycle.
00119      *
00120      */
00121     XPHASE_CODE_SURFACE = 14,
00122
00123     /** Phase code 15: invalid.
00124      *
00125      *  This code is not used.
00126      *
00127      */
00128     XPHASE_CODE_INVALID = 15
00129 } XPhaseCode_t;
00130
00131 /* Class Definition: */
00132
00133 /** Class representing a raw pressure measurement and phase code for a rise or fall data X-Message.
00134  *
00135  */
00136 class XPresCodeCounts:
00137         public XPresCounts {
00138
00139 private:
00140
00141     /* Data Members: */
00142
00143     /** The phase code names.
00144      *
00145      */
00146     static const char* phase_code_names_[];
00147
00148     /** The phase code of this pressure measurement. */
00149     XPhaseCode_t phase_code_;
00150
00151 public:
00152
00153     /* Member Functions: */
00154
00155     /** The constructor.
00156      *
00157      *  @param[in]      sec         The elapsed time since the reference time, in seconds.
00158      *  @param[in]      code        The phase code of this pressure measurement.
00159      *  @param[in]      counts      The pressure measurement, in raw sensor counts.
00160      */
00161     XPresCodeCounts(uint16_t sec, XPhaseCode_t code, int32_t counts) :
00162                     XPresCounts(sec, counts),
00163                     phase_code_(code) {
00164         /* Do nothing. */
00165     }
00166
00167     /** The destructor.
00168      *
00169      */
00170     ~XPresCodeCounts() {
00171         /* Do nothing. */
00172     }
00173
00174     /** Generate a listing entry string from this pressure measurement object.
00175      *
00176      *  @param[in]  pg      The pressure sensor sensitivity scale factor, in LSB/dBar.
00177      *  @param[in]  po      The pressure sensor offset, in dBar.
00178      *  @param[in]  pc      The pressure correction, in dBar, to be subtracted computed pressure
00179      *                      value.
00180      *
00181      *  @throw      log::MessageException if an error occurs while generating the listing string.
00182      *
00183      *  @return     A listing entry string representation of this pressure measurement object.
00184      */
00185     const string GenerateListingString(uint16_t pg, uint16_t po, float pc) const {
00186         ostringstream os;
00187
00188         try {
00189             const format fmt(" %9d # %s");
00190             os « XPresCounts::GenerateListingString(pg, po, pc)
00191                « format(fmt) % (int) phase_code_ % GetPhaseCodeName(phase_code_);
00192         }CATCH_BLOCK("Failed to generate listing string for pressure-code-counts object!")
00193
00194         return os.str();
00195     }
00196
00197     /** Get the phase code name C-string for a given phase code value.
00198      *
00199      *  @param[in]  code    The phase code.
00200      *
00201      *  @return     The phase code name C-string for the given phase code value.
00202      */
00203     static const char* GetPhaseCodeName(XPhaseCode_t code) {
00204
00205         try {
00206
00207             if(code > 14)
00208                 EXC_THROW("Unsupported phase code: " + lexical_cast<string>((int)code) + "!");
00209
00210             return phase_code_names_[code];
00211
```

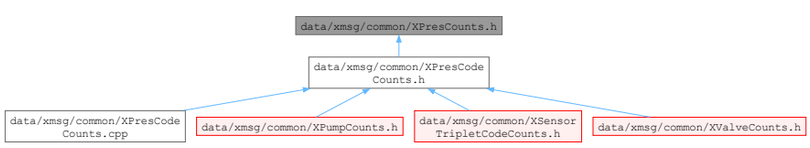
```
00212         }CATCH_BLOCK("Failed to get phase code name!")
00213
00214         return NULL;
00215     }
00216
00217     /** Get the phase code.
00218      *
00219      *  @return     The phase code.
00220      */
00221     XPhaseCode_t phase_code() const {
00222         return phase_code_;
00223     }
00224
00225     /** Get the phase code name C-sting.
00226      *
00227      *  @return     The phase code name C-sting.
00228      */
00229     const char* phase_code_name() const {
00230         return GetPhaseCodeName(phase_code_);
00231     }
00232 };
00233
00234 } /* namespace xmsg */
00235 } /* namespace util */
00236 #endif /* XPRESCOUNTCODES_H_ */
```

11.51 data/xmsg/common/XPresCounts.h File Reference

The util::xmsg::XPresCounts class header file.

`#include "XDefinitions.h"`
Include dependency graph for XPresCounts.h:

This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XPresCounts

  Class representing a raw pressure measurement for a rise or fall data X-Message.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.51.1 Detailed Description

The util::xmsg::XPresCounts class header file.

Version

0.6

Date

Jul 8, 2014 10:42:06 AM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.52 XPresCounts.h

Go to the documentation of this file.

```
00001 /** The @ref util::xmsg::XPresCounts class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jul 8, 2014 10:42:06 AM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef XPRESCOUNTS_H_
00024 #define XPRESCOUNTS_H_
00025
00026 /* Includes: */
00027
00028 // local
00029 #include "XDefinitions.h"
00030
00031 namespace util {
00032 namespace xmsg {
00033
00034 /** Class representing a raw pressure measurement for a rise or fall data X-Message.
00035  *
00036  */
00037 class XPresCounts {
00038
00039     /* Friends Classes: */
00040
00041     friend class XDiveData;
00042
00043 private:
00044
00045     /* Data Members: */
00046
00047     /** The elapsed time since the reference time, in seconds.
00048      *
00049      */
00050     uint16_t elapsed_seconds_since_reference_time_;
00051
00052     /** The pressure measurement, in raw sensor counts.
00053      *
00054      */
00055     int32_t pressure_counts_;
00056
00057 public:
00058
00059     /* Member Functions: */
00060
00061     /** The parameterized constructor.
00062      *
00063      *  @param[in]      sec         The elapsed time since the reference time, in seconds.
00064      *  @param[in]      counts      The pressure measurement, in raw counts.
00065      */
00066     XPresCounts(uint16_t sec, int32_t counts) :
00067                     elapsed_seconds_since_reference_time_(sec),
00068                     pressure_counts_(counts) {
00069         /* Do nothing. */
00070     }
00071
00072     /** The destructor
00073      *
00074      */
00075     ~XPresCounts() {
00076         /* Do nothing. */
00077     }
00078
```

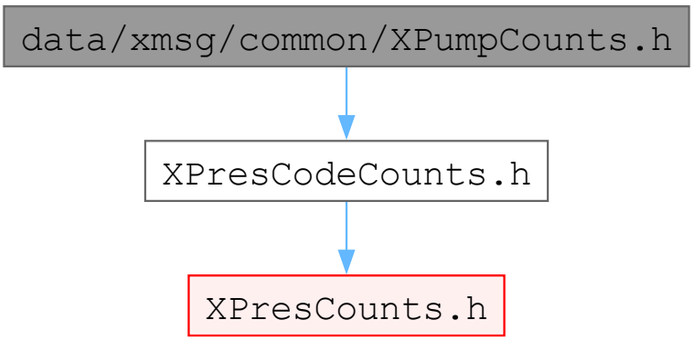
```
00079     /** Get the elapsed time since the reference time, in seconds.
00080      *
00081      *  @return     The elapsed time since the reference time, in seconds.
00082      */
00083     uint16_t elapsed_seconds_since_reference_time() const {
00084         return elapsed_seconds_since_reference_time_;
00085     }
00086
00087     /** Generate a listing entry string from this pressure measurement object.
00088      *
00089      *  @param[in]  pg      The pressure sensor sensitivity scale factor, in LSB/dBar.
00090      *  @param[in]  po      The pressure sensor offset, in dBar.
00091      *  @param[in]  pc      The pressure correction, in dBar, to be subtracted computed pressure
00092      *                      value.
00093      *
00094      *  @throw      log::MessageException if an error occurs while generating the listing string.
00095      *
00096      *  @return     A listing entry string representation of this pressure measurement object.
00097      */
00098     const string GenerateListingString(uint16_t pg, uint16_t po, float pc) const {
00099         ostringstream os;
00100         try {
00101             const format fmt("%+20d %8.3f");
00102             os « format(fmt) % (int) elapsed_seconds_since_reference_time_
00103                     % PressureValue(pg, po, pc);
00104         }CATCH_BLOCK("Failed to generate listing string for pressure-counts object!")
00105         return os.str();
00106     }
00107
00108     int32_t pressure_counts() const {
00109         return pressure_counts_;
00110     }
00111
00112     /** Get the measured pressure value, in dBar.
00113      *
00114      *  @param[in]  pg          The pressure sensor sensitivity scale factor, in LSB/dBar.
00115      *  @param[in]  po          The pressure sensor offset, in dBar.
00116      *  @param[in]  pres_corr   The pressure correction, in dBar, to be subtracted computed
00117      *                          pressure value.
00118      *
00119      *  @return     The measured pressure value, in dBar.
00120      */
00121     template<typename T = float> T PressureValue(uint16_t pg, uint16_t po, float pres_corr) const {
00122         T val = pg ? (T) pressure_counts_ / (T) pg - (T) po - pres_corr: (T) XMSG_SENS_BAD_PRES;
00123
00124         return (val < XMSG_SENS_MIN_PRES || val > XMSG_SENS_MAX_PRES) ? (T) XMSG_SENS_BAD_PRES : val;
00125     }
00126
00127
00128 };
00129
00130 } /* namespace xmsg */
00131 } /* namespace util */
00132 #endif /* XPRESCOUNTS_H_ */
```

11.53 data/xmsg/common/XPumpCounts.h File Reference

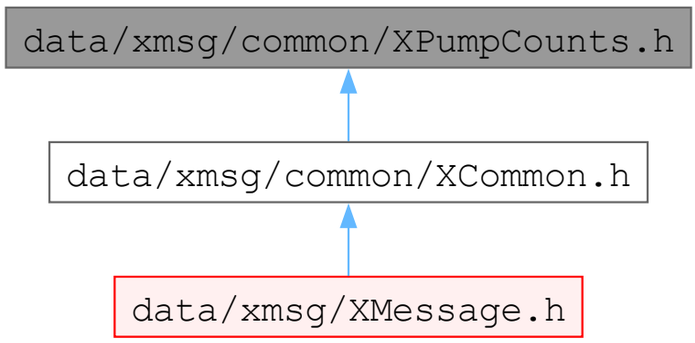
The util::xmsg::XPumpCounts class source file.

```
#include "XPresCodeCounts.h"
```

Include dependency graph for XPumpCounts.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XPumpCounts

  Class representing a single pump data measurement for a pump data time-series X-Message.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.53.1 Detailed Description

The util::xmsg::XPumpCounts class source file.

Version

0.6

Date

Jul 9, 2014 3:00:00 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.54 XPumpCounts.h

Go to the documentation of this file.

```
00001 /** The @ref util::xmsg::XPumpCounts class source file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jul 9, 2014 3:00:00 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef XPUMPCOUNTS_H_
00024 #define XPUMPCOUNTS_H_
00025
00026 /* Includes */
00027
00028 #include "XPresCodeCounts.h"
00029
00030 namespace util {
00031 namespace xmsg {
00032
00033 /** Class representing a single pump data measurement for a pump data time-series X-Message.
00034  *
00035  */
00036 class XPumpCounts {
00037
00038 private:
00039
00040     /* Data Members */
00041
00042     /** The phase code of this pump data measurement.
00043      *
00044      */
00045     XPhaseCode_t phase_code_;
00046
00047     /** The raw pressure counts of the measured pressure during pumping.
00048      *
00049      */
00050     int32_t pressure_counts_;
00051
00052     /** The duration that the pump ran, in seconds.
00053      *
00054      */
00055     int16_t pump_duration_sec_;
00056
00057     /** The measured voltage of the pump supply during pumping, in 0.01 volts.
00058      *
00059      */
00060     uint16_t pump_voltage_cv_;
00061
00062     /** The measured current of the pump supply during pumping, in milliamps.
00063      *
00064      */
00065     uint16_t pump_current_ma_;
00066
00067     /** The measured vacuum before pumping, in 0.01inHg.
00068      *
00069      */
00070     uint16_t case_vac_before_pumping_cinhg_;
00071
00072     /** The measured vacuum after pumping, in 0.01inHg.
00073      *
00074      */
00075     uint16_t case_vac_after_pumping_cinhg_;
00076
00077 protected:
00078
00079     /* Member Functions */
00080
00081     /** Get the measured pressure value, in dBar.
00082      *
00083      *  @param[in]  pg          The pressure sensor sensitivity scale factor, in LSB/dBar.
00084      *  @param[in]  po          The pressure sensor offset, in dBar.
00085      *  @param[in]  pres_corr   The pressure correction, in dBar, to be subtracted computed
00086      *                          pressure value.
00087      *
00088      *  @return     The measured pressure value, in dBar.
00089      */
00090     template<typename T = double> T PressureValue(
00091             uint16_t pg,
00092             uint16_t po,
00093             float pres_corr) const {
00094         T val = pg ? (T) pressure_counts_ / (T) pg - (T) po - pres_corr : XMSG_SENS_BAD_PRES;
00095
00096         return (val < XMSG_SENS_MIN_PRES || val > XMSG_SENS_MAX_PRES) ? XMSG_SENS_BAD_PRES : val;
00097     }
00098
00099     /** Get the measured pump current, in amperes.
00100      *
00101      *  @return     The measured pump current, in amperes.
00102      */
00103     template<typename T = double> T pump_current_a() const {
```

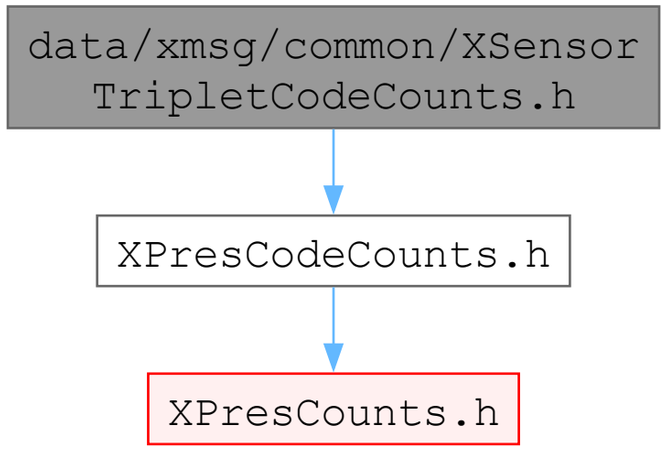
```
00104         return (T) pump_current_ma_ / 1000.;
00105     }
00106
00107     /** Get the measured pump voltage, in volts.
00108      *
00109      *  @return         The measured pump voltage, in volts.
00110      */
00111     template<typename T = double> T pump_voltage_v() const {
00112         return (T) pump_voltage_cv_ / 100.;
00113     }
00114
00115 public:
00116
00117     /* Member Functions */
00118
00119     /** Parameterized constructor.
00120      *
00121      *  @param[in]      code        The phase code of this pump data measurement.
00122      *  @param[in]      count       The raw pressure counts of the measured pressure during pumping.
00123      *  @param[in]      sec         The duration that the pump ran, in seconds.
00124      *  @param[in]      v           The measured voltage of the pump supply during pumping, in 0.01
00125      *                              volts.
00126      *  @param[in]      i           The measured current of the pump supply during pumping, in
00127      *                              milliamps.
00128      *  @param[in]      vac0        The measured vacuum counts before pumping.
00129      *  @param[in]      vac1        The measured vacuum counts after pumping.
00130      */
00131     XPumpCounts(
00132             XPhaseCode_t code,
00133             int32_t count,
00134             int16_t sec,
00135             uint16_t v,
00136             uint16_t i,
00137             uint16_t vac0,
00138             uint16_t vac1) ATTR_NO_EXCEPTIONS:
00139     phase_code_(code),
00140     pressure_counts_(count),
00141     pump_duration_sec_(sec),
00142     pump_voltage_cv_(v),
00143     pump_current_ma_(i),
00144     case_vac_before_pumping_cinhg_(vac0),
00145     case_vac_after_pumping_cinhg_(vac1) {
00146         /* Do nothing. */
00147     }
00148
00149     /** Destructor.
00150      *
00151      */
00152     ~XPumpCounts() {
00153         /* Do nothing. */
00154     }
00155
00156     /** Generate a listing entry string representation of this pump counts object.
00157      *
00158      *  @param[in]  pg          The pressure sensor sensitivity scale factor, in LSB/dBar.
00159      *  @param[in]  po          The pressure sensor offset, in dBar.
00160      *  @param[in]  pres_corr   The pressure correction, in dBar, to be subtracted computed
00161      *                          pressure value.
00162      *
00163      *  @throw      log::MessageException if an error occurs while generating the listing string.
00164      *
00165      *  @return     A listing entry string representation of this pump counts object.
00166      */
00167     const string GenerateListingString(uint16_t pg, uint16_t po, float pres_corr) const {
00168         ostringstream os;
00169
00170         try {
00171             static const format fmt("%10.2f %10d %10.2f %10.3f %10.2f %10.2f");
00172             os << format(fmt) % PressureValue(pg, po, pres_corr) % pump_duration_sec_ % pump_voltage_v() %
    pump_current_a()
00173             % case_vac_before_pumping_inhg() % case_vac_after_pumping_inhg();
00174         }CATCH_BLOCK("Failed to generate listing string for pump counts object!")
00175
00176         return os.str();
00177     }
00178
00179     /** Get the measured vacuum after pumping, in 0.01inHg.
00180      *
00181      *  @return     The measured vacuum after pumping, in 0.01inHg.
00182      */
00183     uint16_t case_vac_after_pumping_cinhg() const
00184     {
00185         return case_vac_after_pumping_cinhg_;
00186     }
00187
00188     /** Get the measured vacuum before pumping, in 0.01inHg.
00189      *
00190      *  @return     The measured vacuum before pumping, in 0.01inHg.
00191      */
00192     uint16_t case_vac_before_pumping_cinhg() const
00193     {
00194         return case_vac_before_pumping_cinhg_;
00195     }
00196
00197     /** Get the measured vacuum after pumping, in inHg.
00198      *
00199      *  @return     The measured vacuum after pumping, in inHg.
00200      */
00201     template<typename T = float> T case_vac_after_pumping_inhg() const {
00202         return (T) case_vac_after_pumping_cinhg() / 100.;
00203     }
00204
00205     /** Get the measured vacuum before pumping, in inHg.
00206      *
00207      *  @return     The measured vacuum before pumping, in inHg.
00208      */
00209     template<typename T = float> T case_vac_before_pumping_inhg() const {
00210         return (T) case_vac_before_pumping_cinhg() / 100.;
```

```
00211     }
00212
00213 };
00214
00215 }
00216 /* namespace xmsg */
00217 } /* namespace util */
00218
00219 #endif /* XPUMPCOUNTS_H_ */
```

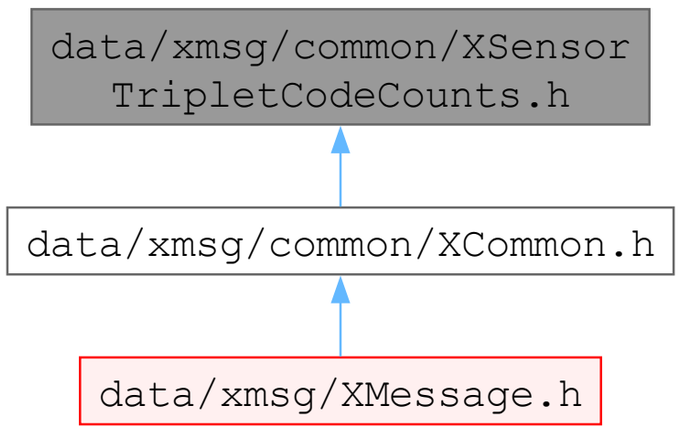
11.55 data/xmsg/common/XSensorTripletCodeCounts.h File Reference

The util::xmsg::XSensorTripletCodeCounts class header file.

`#include "XPresCodeCounts.h"`
Include dependency graph for XSensorTripletCodeCounts.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XSensorTripletCodeCounts

  The sensor triplet and phase code class.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.55.1 Detailed Description

The util::xmsg::XSensorTripletCodeCounts class header file.

This is the header file for the sensor triplet and phase code class, util::xmsg::XSensorTripletCodeCounts. This class represents a pressure, temperature, and salinity measurement triplet and a phase code. The phase code is used to indicate the context of the measurement (see util::xmsg::XPhaseCode_t). This class is used to store a single data point for the rise, fall, or surface time-series data.

Version

0.6

Date

Mar 18, 2015 11:16:33 AM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.56 XSensorTripletCodeCounts.h

Go to the documentation of this file.

```
00001 /** The @ref util::xmsg::XSensorTripletCodeCounts class header file.
00002  *
00003  *  This is the header file for the sensor triplet and phase code class, @ref
00004  *  util::xmsg::XSensorTripletCodeCounts. This class represents a pressure, temperature, and
00005  *  salinity measurement triplet and a phase code. The phase code is used to indicate the context
00006  *  of the measurement (see @ref util::xmsg::XPhaseCode_t). This class is used to store a single
00007  *  data point for the rise, fall, or surface time-series data.
00008  *
00009  *  @file
00010  *
00011  *  @version    0.6
00012  *
00013  *  @date       Mar 18, 2015 11:16:33 AM
00014  *
00015  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00016  *              Engineer I,
00017  *              Department of Physical Oceanography,
00018  *              Woods Hole Oceanographic Institution
00019  *
00020  *  @copyright  2014 Woods Hole Oceanographic Institution.
00021  *              Released under the GNU General Public License.
00022  *
00023  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00024  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00025  *  General Public License for more details. You should have received a copy of the GNU General
00026  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00027  */
00028
00029 #ifndef XSENSORTRIPLETCOUNTS_H_
00030 #define XSENSORTRIPLETCOUNTS_H_
00031
00032 /* Includes: */
00033
00034 // local
00035 #include "XPresCodeCounts.h"
```

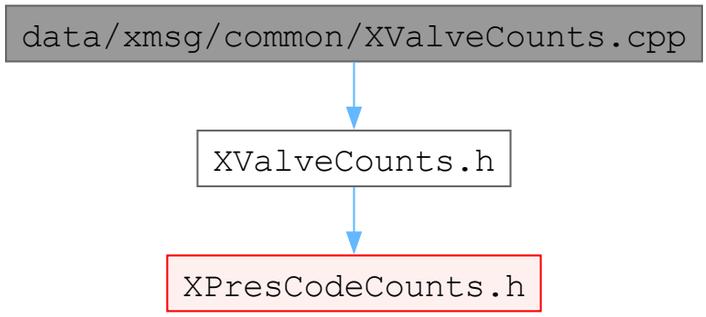
```
00036
00037 namespace util {
00038 namespace xmsg {
00039
00040 /* Class Definition: */
00041
00042 /** The sensor triplet and phase code class.
00043  *
00044  *  This class represents a pressure, temperature, and salinity measurement triplet and a phase
00045  *  code. The phase code is used to indicate the context of the measurement (see @ref
00046  *  XPhaseCode_t). This class is used to store a single data point for the rise, fall, or surface
00047  *  time-series data.
00048  *
00049  */
00050 class XSensorTripletCodeCounts:
00051         public XPresCodeCounts {
00052
00053 private:
00054
00055     /** The salinity measurement, in raw sensor counts.
00056      *
00057      */
00058     int32_t salinity_counts_;
00059
00060     /** The temperature measurement, in raw sensor counts.
00061      *
00062      */
00063     int32_t temperature_counts_;
00064
00065 public:
00066
00067     /** The constructor.
00068      *
00069      *  @param[in]  sec         The elapsed time since the reference time, in seconds.
00070      *  @param[in]  code        The phase code of this pressure measurement.
00071      *  @param[in]  pres_counts The pressure measurement, in raw sensor counts.
00072      *  @param[in]  temp_counts The temperature measurement, in raw sensor counts.
00073      *  @param[in]  saln_counts The salinity measurement, in raw sensor counts.
00074      */
00075     XSensorTripletCodeCounts(
00076             uint16_t sec,
00077             XPhaseCode_t code,
00078             int32_t pres_counts,
00079             int32_t temp_counts = XMSG_SENS_BAD,
00080             int32_t saln_counts = XMSG_SENS_BAD) :
00081                     XPresCodeCounts(sec, code, pres_counts),
00082                     salinity_counts_(saln_counts),
00083                     temperature_counts_(temp_counts) {
00084         /* Do nothing. */
00085     }
00086
00087
00088
00089     /** The destructor.
00090      *
00091      */
00092     ~XSensorTripletCodeCounts() {
00093         /* Do nothing. */
00094     }
00095
00096     /** Generate a listing entry string from this sensor triplet object.
00097      *
00098      *  @param[in]  pg      The pressure sensor sensitivity scale factor, in LSB/dBar.
00099      *  @param[in]  po      The pressure sensor offset, in dBar.
00100      *  @param[in]  sg      The salinity sensor sensitivity scale factor, in LSB/PSU.
00101      *  @param[in]  so      The salinity sensor offset, in PSU.
00102      *  @param[in]  tg      The temperature sensor sensitivity scale factor, in LSB/degC.
00103      *  @param[in]  to      The temperature sensor offset, in degC.
00104      *  @param[in]  pc      The pressure correction, in dBar, to be subtracted computed pressure
00105      *                      value.
00106      *
00107      *  @throw      log::MessageException if an error occurs while generating the listing string.
00108      *
00109      *  @return     A listing entry string representation of this sensor triplet object.
00110      */
00111     const string GenerateListingString(
00112             uint16_t pg,
00113             uint16_t po,
00114             uint16_t sg,
00115             uint16_t so,
00116             uint16_t tg,
00117             uint16_t to,
00118             float pc) const {
00119         ostringstream os;
00120
00121         try {
00122             const format fmt("%+20d %9.3f %7.3f %7.3f %7.3f %9d # %s");
00123             os
00124                     << format(fmt) % (int) elapsed_seconds_since_reference_time()
00125                             % PressureValue(pg, po, pc) % TemperatureValue(tg, to)
00126                             % SalinityValue(sg, so) % (int) phase_code()
00127                             % phase_code_name();
00128         }CATCH_BLOCK("Failed to generate listing string for sensor triplet object!")
00129
00130         return os.str();
00131     }
00132
00133     int32_t salinity_counts() const {
00134         return salinity_counts_;
00135     }
00136
00137     int32_t temperature_counts() const {
00138         return temperature_counts_;
00139     }
00140
00141     /** Get the measured salinity value, in PSU.
00142      *
00143      *  @param[in]  sg          The salinity sensor sensitivity scale factor, in LSB/PSU.
```

```
00144      *  @param[in]  so          The salinity sensor offset, in PSU.
00145      *
00146      *  @return     The measured salinity value, in PSU.
00147      */
00148     template<typename T = double> T SalinityValue(uint16_t sg, uint16_t so) const {
00149         T val = sg ? (T) salinity_counts_ / (T) sg - (T) so : XMSG_SENS_BAD_SALN;
00150         return (val < XMSG_SENS_MIN_SALN || val > XMSG_SENS_MAX_SALN) ? XMSG_SENS_BAD_SALN : val;
00151     }
00152
00153     /** Get the measured temperature value, in degC.
00154      *
00155      *  @param[in]  tg          The temperature sensor sensitivity scale factor, in LSB/degC.
00156      *  @param[in]  to          The temperature sensor offset, in degC.
00157      *
00158      *  @return     The measured temperature value, in degC.
00159      */
00160     template<typename T = double> T TemperatureValue(uint16_t tg, uint16_t to) const {
00161         T val = tg ? (T) temperature_counts_ / (T) tg - (T) to : XMSG_SENS_BAD_TEMP;
00162         return (val < XMSG_SENS_MIN_TEMP || val > XMSG_SENS_MAX_TEMP) ? XMSG_SENS_BAD_TEMP : val;
00163     }
00164 };
00165
00166 } /* namespace xmsg */
00167 } /* namespace util */
00168
00169 #endif /* XSENSORTRIPLETCOUNTS_H_ */
```

11.57 data/xmsg/common/XValveCounts.cpp File Reference

The util::xmsg::XValveCounts class source file.

`#include "XValveCounts.h"`
Include dependency graph for XValveCounts.cpp:



Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.57.1 Detailed Description

The util::xmsg::XValveCounts class source file.

Version

0.6

Date

Mar 22, 2016 9:20:55 AM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

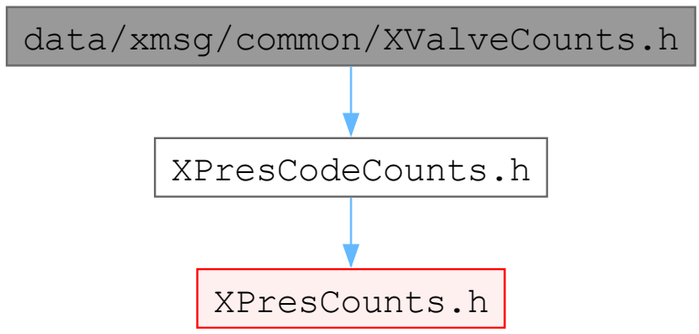
This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.58 data/xmsg/common/XValveCounts.h File Reference

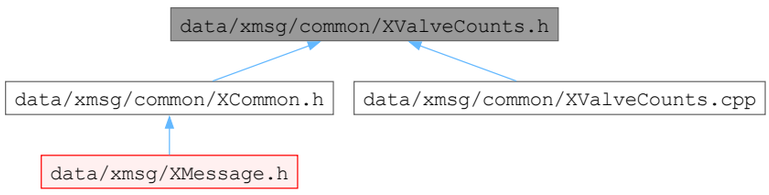
The util::xmsg::XValveCounts class header file.

```
#include "XPresCodeCounts.h"
```

Include dependency graph for XValveCounts.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XValveCounts

  The valve counts class.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.58.1 Detailed Description

The util::xmsg::XValveCounts class header file.

Version

0.6

Date

Mar 22, 2016 9:20:55 AM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.59 XValveCounts.h

Go to the documentation of this file.

```
00001 /** The @ref util::xmsg::XValveCounts class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Mar 22, 2016 9:20:55 AM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef XVALVECOUNTS_H_
00024 #define XVALVECOUNTS_H_
00025
00026 /* Includes: */
00027
00028 #include "XPresCodeCounts.h"
00029
00030 /* Definitions: */
00031
00032 namespace util {
00033 namespace xmsg {
00034
00035 /* Types: */
00036
00037 /** The valve counts class.
00038  *
00039  *  An instance of this class represents a single valve operation decoded from a valve data
00040  *  message (see @ref XValve): the pressure, and the case vacuum before and after the valve
00041  *  operation.
00042  */
00043 class XValveCounts {
00044
00045 private:
00046
00047     /* Friends Classes: */
00048
00049     friend class XDiveData;
00050
00051 private:
00052
00053     /* Private Data Members: */
00054
00055     /** The measured vacuum after opening the valve, in 0.01inHg.
00056      *
```

```
00057      */
00058     uint16_t case_vac_after_valve_cinhg_;
00059
00060     /** The measured vacuum before opening the valve, in 0.01inHg.
00061      *
00062      */
00063     uint16_t case_vac_before_valve_cinhg_;
00064
00065     /** The phase code of this valve data measurement.
00066      *
00067      */
00068     XPhaseCode_t phase_code_;
00069
00070     /** The measured pressure, before opening the valve, in raw sensor counts.
00071      *
00072      */
00073     int32_t pressure_counts_;
00074
00075     /** The duration that the valve was open, in seconds.
00076      *
00077      */
00078     int16_t valve_duration_sec_;
00079
00080 protected:
00081
00082     /* Protected Member Functions: */
00083
00084     /** Get the measured pressure value, in dBar.
00085      *
00086      *  @param[in]  pg      The pressure sensor sensitivity scale factor, in LSB/dBar.
00087      *  @param[in]  po      The pressure sensor offset, in dBar.
00088      *  @param[in]  pc      The pressure correction, in dBar, to be subtracted computed pressure
00089      *                      value.
00090      *
00091      *  @return     The measured pressure value, in dBar.
00092      */
00093     template<typename T = double> T PressureValue(uint16_t pg, uint16_t po, float pc) const {
00094
00095         T val = pg ? (T) pressure_counts_ / (T) pg - (T) po - pc: XMSG_SENS_BAD_PRES;
00096
00097         return (val < XMSG_SENS_MIN_PRES || val > XMSG_SENS_MAX_PRES) ? XMSG_SENS_BAD_PRES : val;
00098     }
00099
00100 public:
00101
00102     /* Public Member Functions: */
00103
00104     /** The parameterized constructor.
00105      *
00106      *  @param[in]  code    The phase code.
00107      *  @param[in]  count   The measured pressure, before opening the valve, in raw sensor counts.
00108      *  @param[in]  sec     The duration that the valve was open, in seconds.
00109      *  @param[in]  vac0    The measured vacuum after opening the valve, in 0.01inHg.
00110      *  @param[in]  vac1    The measured vacuum before opening the valve, in 0.01inHg.
00111      */
00112     XValveCounts(XPhaseCode_t code, int32_t count, int16_t sec, uint16_t vac0, uint16_t vac1)
00113     ATTR_NO_EXCEPTIONS :
00114                     case_vac_after_valve_cinhg_(vac1),
00115                     case_vac_before_valve_cinhg_(vac0),
00116                     phase_code_(code),
00117                     pressure_counts_(count),
00118                     valve_duration_sec_(sec) {
00119
00120         /* Do nothing. */
00121
00122     }
00123
00124     /** The destructor.
00125      *
00126      */
00127     ~XValveCounts() {
00128
00129         /* Do nothing. */
00130
00131     }
00132
00133     uint16_t case_vac_after_valve_cinhg() const {
00134         return case_vac_after_valve_cinhg_;
00135     }
00136
00137     /** Get the measured vacuum after pumping, in inHg.
00138      *
00139      *  @return     The measured vacuum after pumping, in inHg.
00140      */
00141     template<typename T = float> T case_vac_after_pumping_inhg() const {
00142         return (T) case_vac_after_valve_cinhg() / (T) 100.;
00143     }
00144
00145     uint16_t case_vac_before_valve_cinhg() const {
00146         return case_vac_before_valve_cinhg_;
00147     }
00148
00149     /** Get the measured vacuum before pumping, in inHg.
00150      *
00151      *  @return     The measured vacuum before pumping, in inHg.
00152      */
00153     template<typename T = float> T case_vac_before_pumping_inhg() const {
00154         return (T) case_vac_before_valve_cinhg() / (T) 100.;
00155     }
00156
00157     const string GenerateListingString(uint16_t pg, uint16_t po, float pres_correction) const {
00158
00159         ostringstream os;
00160
00161         try {
00162
00163             static const format fmt("%10.2f "
00164                     "%10d "
```

```
00165                     "%10.2f %10.2f");
00166             os « format(fmt) % PressureValue(pg, po, pres_correction) //
00167                     % valve_duration_sec() //
00168                     % case_vac_before_pumping_inhg() % case_vac_after_pumping_inhg();
00169 
00170         }CATCH_BLOCK("Failed to generate listing string for valve counts object!")
00171 
00172         return os.str();
00173     }
00174 
00175     XPhaseCode_t phase_code() const {
00176         return phase_code_;
00177     }
00178 
00179     int32_t pressure_counts() const {
00180         return pressure_counts_;
00181     }
00182 
00183     int16_t valve_duration_sec() const {
00184         return valve_duration_sec_;
00185     }
00186 
00187 protected:
00188 
00189     /* Protected Member Functions: */
00190 
00191     void set_case_vac_after_valve_cinhg(uint16_t case_vac_after_valve_cinhg) {
00192         case_vac_after_valve_cinhg_ = case_vac_after_valve_cinhg;
00193     }
00194 
00195     void set_case_vac_before_valve_cinhg(uint16_t case_vac_before_valve_cinhg) {
00196         case_vac_before_valve_cinhg_ = case_vac_before_valve_cinhg;
00197     }
00198 
00199     void set_phase_code(XPhaseCode_t phase_code) {
00200         phase_code_ = phase_code;
00201     }
00202 
00203     void set_pressure_counts(int32_t pressure_counts) {
00204         pressure_counts_ = pressure_counts;
00205     }
00206 
00207     void set_valve_duration_sec(int16_t valve_duration_sec) {
00208         valve_duration_sec_ = valve_duration_sec;
00209     }
00210 
00211 };
00212 
00213 } /* namespace xmsg */
00214 } /* namespace util */
00215 
00216 #endif /* XVALVECOUNTS_H_ */
```

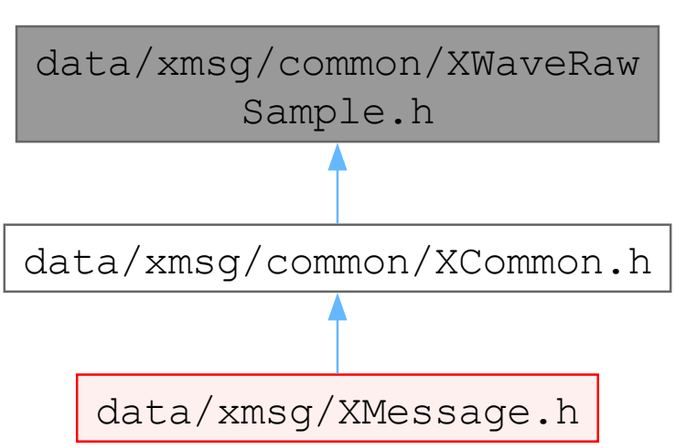
11.60 data/xmsg/common/XWaveRawSample.h File Reference

The util::xmsg::XWaveRawSample class header file.

`#include "XDefinitions.h"`

Include dependency graph for XWaveRawSample.h:

This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XWaveRawSample

  Class representing a single raw motion measurement in the surface time series wave data X-message.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.60.1 Detailed Description

The util::xmsg::XWaveRawSample class header file.

Version

0.6

Date

Jul 27, 2014 4:15:28 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.61 XWaveRawSample.h

Go to the documentation of this file.

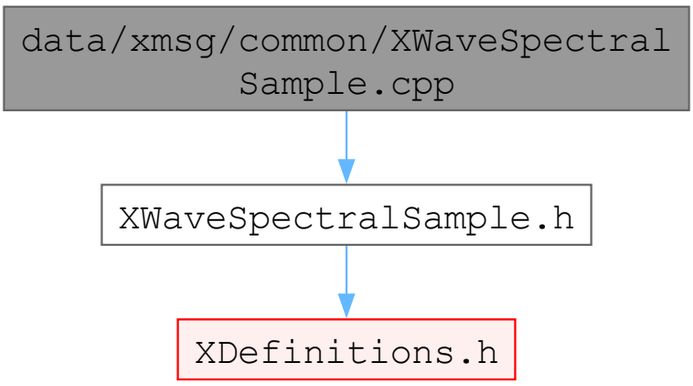
```
00001 /** The @ref util::xmsg::XWaveRawSample class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jul 27, 2014 4:15:28 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef XWAVERAWSAMPLE_H_
00024 #define XWAVERAWSAMPLE_H_
00025
00026 /* Includes */
00027
00028 // local
00029 #include "XDefinitions.h"
00030
00031
00032
00033 namespace util {
```

```
00034 namespace xmsg {
00035
00036 /** Class representing a single raw motion measurement in the surface time series wave data
00037  *  X-message.
00038  *
00039  *  @note           This class is used for raw data mode only.
00040  *
00041  */
00042 class XWaveRawSample {
00043
00044     /* Friends */
00045
00046     friend class XWave;
00047
00048 private:
00049
00050     /** Measured acceleration in the starboard (x'), bow (y'), and upward (z') directions. */
00051     int16_t accel[3];
00052
00053     /** Measured magnetism in the starboard (x'), bow (y'), and upward (z') directions.  */
00054     int16_t magnt[3];
00055
00056     /** Measured rotational velocity in the gamma (starboard-up), beta (bow-up), and alpha
00057      *  (increasing azimuth) directions.
00058      */
00059     int16_t omega[3];
00060
00061 public:
00062
00063     /** Default constructor.
00064      *
00065      */
00066     XWaveRawSample() ATTR_NO_EXCEPTIONS {
00067         /* Do nothing. */
00068     }
00069
00070     /** Destructor.
00071      *
00072      */
00073     ~XWaveRawSample() {
00074         /* Do nothing. */
00075     }
00076
00077     /** Generate a listing entry string representation of this raw motion measurement.
00078      *
00079      *  @param[in]      assf            The accelerometer sensitivity scale factor, in LSB/g.
00080      *  @param[in]      gssf            The gyroscope sensitivity scale factor, in LSB/(deg/s).
00081      *  @param[in]      mssf            The magnetometer sensitivity scale factor, in LSB/uT.
00082      *
00083      *  @return         A listing entry string representation of this raw motion measurement.
00084      */
00085     const string GenerateListingString(double assf, double gssf, double mssf) const {
00086         ostringstream os;
00087
00088         try {
00089             static const format fmt("%12.6f %12.6f %12.6f %9.1f %9.1f %9.1f %12.6f %12.6f %12.6f");
00090             os << format(fmt) % (accel[0] / assf) % (accel[1] / assf) % (accel[2] / assf)
00091                     % (magnt[0] / mssf) % (magnt[1] / mssf) % (magnt[2] / mssf) % (omega[0] / gssf)
00092                     % (omega[1] / gssf) % (omega[2] / gssf);
00093         }CATCH_BLOCK("Failed to generate listing string from XWaveRawSample object!")
00094
00095         return os.str();
00096     }
00097 };
00098
00099 } /* namespace xmsg */
00100 } /* namespace util */
00101 #endif /* XWAVERAWSAMPLE_H_ */
```

11.62 data/xmsg/common/XWaveSpectralSample.cpp File Reference

The util::xmsg::XWaveSpectralSample class source file.

```
#include "XWaveSpectralSample.h"
```
Include dependency graph for XWaveSpectralSample.cpp:



Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.62.1 Detailed Description

The util::xmsg::XWaveSpectralSample class source file.

Version

0.6

Date

Jul 20, 2020 11:29:19 AM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

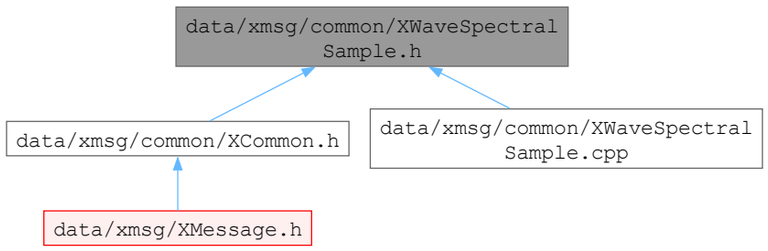
11.63 data/xmsg/common/XWaveSpectralSample.h File Reference

The util::xmsg::XWaveSpectralSample class header file.

```
#include "XDefinitions.h"
```

Include dependency graph for XWaveSpectralSample.h:

This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XWaveSpectralSample
  The wave spectral sample class.

Namespaces

- namespace util
  The utility namespace.
- namespace util::xmsg
  The X-Message utility namespace.

11.63.1 Detailed Description

The util::xmsg::XWaveSpectralSample class header file.

Version

0.6

Date

Jun 19, 2019 1:18:26 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.64 XWaveSpectralSample.h

Go to the documentation of this file.

```
00001 /** The @ref util::xmsg::XWaveSpectralSample class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jun 19, 2019 1:18:26 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef DATA_XMSG_COMMON_XWAVESPECTRALSAMPLE_H_
00024 #define DATA_XMSG_COMMON_XWAVESPECTRALSAMPLE_H_
00025
00026 /* Includes */
00027
00028 // local
00029 #include "XDefinitions.h"
00030
00031 /* Definitions: */
00032
00033 namespace util {
00034 namespace xmsg {
00035
00036 /* Types: */
00037
00038 /** The wave spectral sample class.
00039  *
00040  *  An instance of this class represents the acceleration spectra and co-/quadrature spectra at a
00041  *  single frequency, decoded from a wave X-message (see @ref XWave).
00042  */
00043 class XWaveSpectralSample {
00044
00045     /* Friends: */
00046
00047     friend class XWave;
00048
00049   private:
00050
00051     static const double SPECTRA_FILL_VALUE;
00052
00053     static const double SPECTRA_M2_MAX;
00054
00055     static const double SPECTRA_M2_MIN;
00056
00057   private:
00058
00059     float freq_;
00060     int32_t c_xx_;
00061     int32_t c_xy_;
00062     int32_t c_yy_;
00063     int32_t c_zz_;
00064     int32_t q_zx_;
00065     int32_t q_zy_;
00066
00067   public:
00068
00069     XWaveSpectralSample() ATTR_NO_EXCEPTIONS :
00070     freq_(-99),
00071     c_xx_(INT32_MIN),
00072     c_xy_(INT32_MIN),
00073     c_yy_(INT32_MIN),
00074     c_zz_(INT32_MIN),
00075     q_zx_(INT32_MIN),
00076     q_zy_(INT32_MIN) {
00077       /* Do nothing. */
00078     }
00079
00080     ~XWaveSpectralSample() {
00081       /* Do nothing. */
00082     }
00083
00084   protected:
00085
00086     template<typename T = double> T c_xx_g2hz(T scale) const {
00087       if(c_xx_ == INT32_MIN) return XWaveSpectralSample::SPECTRA_FILL_VALUE;
00088 //      if(!scale)             return XWaveSpectralSample::SPECTRA_FILL_VALUE;
00089       return ((T) c_xx_ / (T) scale);
00090     }
00091
00092     template<typename T = double> T c_xy_g2hz(T scale) const {
00093       if(c_xy_ == INT32_MIN) return XWaveSpectralSample::SPECTRA_FILL_VALUE;
00094 //      if(!scale)             return XWaveSpectralSample::SPECTRA_FILL_VALUE;
00095       return ((T) c_xy_ / (T) scale);
00096     }
00097
00098     template<typename T = double> T c_yy_g2hz(T scale) const {
00099       if(c_yy_ == INT32_MIN) return XWaveSpectralSample::SPECTRA_FILL_VALUE;
00100 //      if(!scale)             return XWaveSpectralSample::SPECTRA_FILL_VALUE;
00101       return ((T) c_yy_ / (T) scale);
00102     }
00103
```

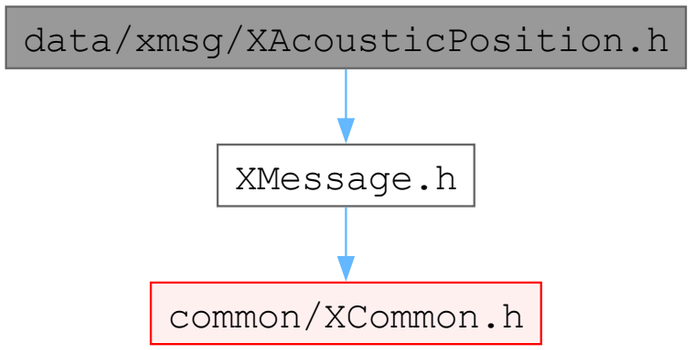
```
00104     template<typename T = double> T c_zz_g2hz(T scale) const {
00105       if(c_zz_ == INT32_MIN) return XWaveSpectralSample::SPECTRA_FILL_VALUE;
00106 //      if(!scale)             return XWaveSpectralSample::SPECTRA_FILL_VALUE;
00107       return ((T) c_zz_ / (T) scale);
00108     }
00109
00110     template<typename T = double> T q_zx_g2hz(T scale) const {
00111       if(q_zx_ == INT32_MIN) return XWaveSpectralSample::SPECTRA_FILL_VALUE;
00112 //      if(!scale)             return XWaveSpectralSample::SPECTRA_FILL_VALUE;
00113       return ((T) q_zx_ / (T) scale);
00114     }
00115
00116     template<typename T = double> T q_zy_g2hz(T scale) const {
00117       if(q_zy_ == INT32_MIN) return XWaveSpectralSample::SPECTRA_FILL_VALUE;
00118 //      if(!scale)             return XWaveSpectralSample::SPECTRA_FILL_VALUE;
00119       return ((T) q_zy_ / (T) scale);
00120     }
00121
00122     template<typename T = double> T c_xx_m2hz(T scale, T freq) const {
00123
00124       const T g_squared = (T) 9.80665 * 9.80665;
00125       T two_pi_f;
00126       T tmp;
00127
00128       if(c_xx_ == INT32_MIN) return SPECTRA_FILL_VALUE;
00129 //      if(scale == 0)         return SPECTRA_FILL_VALUE;
00130 //      if(!freq)              return XWaveSpectralSample::SPECTRA_FILL_VALUE;
00131
00132       two_pi_f = (T) 2 * 3.14159265359 * freq;
00133       tmp = g_squared * c_xx_g2hz(scale) / two_pi_f / two_pi_f / two_pi_f / two_pi_f;
00134
00135       tmp = (tmp > SPECTRA_M2_MAX ? SPECTRA_M2_MAX : tmp);
00136
00137       return tmp;
00138     }
00139
00140     template<typename T = double> T c_xy_m2hz(T scale, T freq) const {
00141
00142       const T g_squared = (T) 9.80665 * 9.80665;
00143       T two_pi_f;
00144       T tmp;
00145
00146       if(c_xy_ == INT32_MIN) return XWaveSpectralSample::SPECTRA_FILL_VALUE;
00147 //      if(!scale)             return XWaveSpectralSample::SPECTRA_FILL_VALUE;
00148 //      if(!freq)              return XWaveSpectralSample::SPECTRA_FILL_VALUE;
00149
00150       two_pi_f = (T) 2 * 3.14159265359 * freq;
00151       tmp = g_squared * c_xy_g2hz(scale) / two_pi_f / two_pi_f / two_pi_f / two_pi_f;
00152
00153       tmp = (tmp > SPECTRA_M2_MAX ? SPECTRA_M2_MAX : tmp);
00154       tmp = (tmp < SPECTRA_M2_MIN ? SPECTRA_M2_MIN : tmp);
00155
00156       return tmp;
00157     }
00158
00159     template<typename T = double> T c_yy_m2hz(T scale,T freq) const {
00160
00161       const T g_squared = (T) 9.80665 * 9.80665;
00162       T two_pi_f;
00163       T tmp;
00164
00165       if(c_yy_ == INT32_MIN) return XWaveSpectralSample::SPECTRA_FILL_VALUE;
00166 //      if(!scale)             return XWaveSpectralSample::SPECTRA_FILL_VALUE;
00167 //      if(!freq)              return XWaveSpectralSample::SPECTRA_FILL_VALUE;
00168
00169       two_pi_f = (T) 2 * 3.14159265359 * freq;
00170       tmp = g_squared * c_yy_g2hz(scale) / two_pi_f / two_pi_f / two_pi_f / two_pi_f;
00171
00172       tmp = (tmp > SPECTRA_M2_MAX ? SPECTRA_M2_MAX : tmp);
00173
00174       return tmp;
00175     }
00176
00177     template<typename T = double> T c_zz_m2hz(T scale,T freq) const {
00178
00179       const T g_squared = (T) 9.80665 * 9.80665;
00180       T two_pi_f;
00181       T tmp;
00182
00183       if(c_zz_ == INT32_MIN) return XWaveSpectralSample::SPECTRA_FILL_VALUE;
00184 //      if(!scale)             return XWaveSpectralSample::SPECTRA_FILL_VALUE;
00185 //      if(!freq)              return XWaveSpectralSample::SPECTRA_FILL_VALUE;
00186
00187       two_pi_f = (T) 2 * 3.14159265359 * freq;
00188       tmp = g_squared * c_zz_g2hz(scale) / two_pi_f / two_pi_f / two_pi_f / two_pi_f;
00189
00190       tmp = (tmp > SPECTRA_M2_MAX ? SPECTRA_M2_MAX : tmp);
00191
00192       return tmp;
00193     }
00194
00195     template<typename T = double> T q_zx_m2hz(T scale,T freq) const {
00196
00197       const T g_squared = (T) 9.80665 * 9.80665;
00198       T two_pi_f;
00199       T tmp;
00200
00201       if(q_zx_ == INT32_MIN) return XWaveSpectralSample::SPECTRA_FILL_VALUE;
00202 //      if(!scale)             return XWaveSpectralSample::SPECTRA_FILL_VALUE;
00203 //      if(!freq)              return XWaveSpectralSample::SPECTRA_FILL_VALUE;
00204
00205       two_pi_f = (T) 2 * 3.14159265359 * freq;
00206       tmp = g_squared * q_zx_g2hz(scale) / two_pi_f / two_pi_f / two_pi_f / two_pi_f;
00207
00208       tmp = (tmp > SPECTRA_M2_MAX ? SPECTRA_M2_MAX : tmp);
00209       tmp = (tmp < SPECTRA_M2_MIN ? SPECTRA_M2_MIN : tmp);
00210
00211       return tmp;
```

```
00212     }
00213
00214     template<typename T = double> T q_zy_m2hz(T scale,T freq) const {
00215
00216       const T g_squared = (T) 9.80665 * 9.80665;
00217       T two_pi_f;
00218       T tmp;
00219
00220       if(q_zy_ == INT32_MIN) return XWaveSpectralSample::SPECTRA_FILL_VALUE;
00221 //      if(!scale)             return XWaveSpectralSample::SPECTRA_FILL_VALUE;
00222 //      if(!freq)              return XWaveSpectralSample::SPECTRA_FILL_VALUE;
00223
00224       two_pi_f = (T) 2 * 3.14159265359 * freq;
00225       tmp = g_squared * q_zy_g2hz(scale) / two_pi_f / two_pi_f / two_pi_f / two_pi_f;
00226
00227       tmp = (tmp > SPECTRA_M2_MAX ? SPECTRA_M2_MAX : tmp);
00228       tmp = (tmp < SPECTRA_M2_MIN ? SPECTRA_M2_MIN : tmp);
00229
00230       return tmp;
00231     }
00232 };
00233
00234 /* Functions: */
00235
00236 } /* namespace xmsg */
00237 } /* namespace util */
00238
00239 #endif /* DATA_XMSG_COMMON_XWAVESPECTRALSAMPLE_H_ */
```

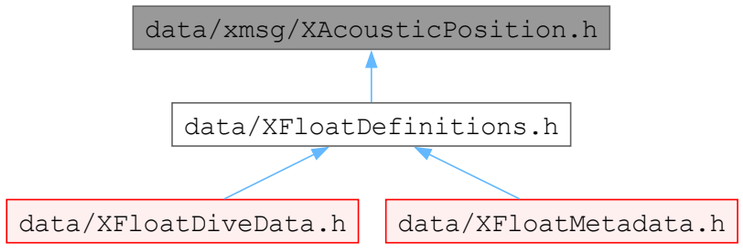
11.65 data/xmsg/XAcousticPosition.h File Reference

The util::xmsg::XAcousticPosition class header file.

`#include "XMessage.h"`
Include dependency graph for XAcousticPosition.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XAcousticCacst
  An acoustic reception (CACST) record.
- class util::xmsg::XAcousticPosition
  Class representing an acoustic position X-Message.
- class util::xmsg::XAcousticWindow
  An acoustic listening window.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.65.1 Detailed Description

The util::xmsg::XAcousticPosition class header file.

Version

0.6

Date

Jul 1, 2014 1:56:04 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.66 XAcousticPosition.h

Go to the documentation of this file.

```
00001 /** The @ref util::xmsg::XAcousticPosition class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jul 1, 2014 1:56:04 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef XACOUSTICPOSITION_H_
00024 #define XACOUSTICPOSITION_H_
00025
00026 /* Includes */
00027
00028 #include "XMessage.h"
00029
00030 namespace bpt = boost::posix_time;
00031
00032 /* Definitions */
00033
00034 namespace util {
00035 namespace xmsg {
00036
00037 using bpt::ptime;
00038
00039
00040
00041 /* Class Definitions */
00042
```

```
00043 /** An acoustic reception (CACST) record.
00044  *
00045  *  An instance of this class represents a single acoustic signal received by the instrument,
00046  *  decoded from an acoustic position message (see @ref XAcousticPosition): its counter, time of
00047  *  arrival (seconds and microseconds), source identifier, signal power, PSK error code, input
00048  *  and output signal-to-noise ratios, and the index of the listening window it was received in.
00049  */
00050 class XAcousticCacst {
00051
00052     friend class XAcousticPosition;
00053
00054
00055   private:
00056
00057     uint32_t counter_;
00058
00059     /** Time of arrival, in UNIX time
00060      *
00061      */
00062     uint32_t toa_sec_;
00063
00064     /** Microsecond of arrival time.
00065      *
00066      */
00067     uint32_t toa_usec_;
00068
00069     /** Matched filter peak value, in dB.
00070      *
00071      */
00072     uint32_t mfd_power_;
00073
00074     /** Acoustic message source ID.
00075      *
00076      */
00077     int16_t src_;
00078
00079     /** PSK error code enumeration.
00080      *
00081      */
00082     uint16_t psk_error_code_;
00083
00084     /** Input SNR, in 1/10 dB.
00085      *
00086      */
00087     int32_t snr_in_;
00088
00089     /** SNR at output of equalizer, in 1/10 dB.
00090      *
00091      */
00092     int32_t snr_out_;
00093
00094     int window_idx_;
00095
00096   public:
00097
00098     /** Default constructor.
00099      *
00100      */
00101     XAcousticCacst() :
00102             counter_(UINT32_MAX),
00103             toa_sec_(UINT32_MAX),
00104             toa_usec_(UINT32_MAX),
00105             mfd_power_(0),
00106             src_(INT16_MIN),
00107             psk_error_code_(0),
00108             snr_in_(0),
00109             snr_out_(0),
00110             window_idx_(-1) {
00111       /* Do nothing. */
00112     }
00113
00114     /** Construct from message data.
00115      *
00116      *  @param[in]      data        The binary message data to decode.
00117      */
00118     XAcousticCacst(const vector<uint8_t>& data) : XAcousticCacst()  {
00119       DecodeXMsgData(data);
00120     }
00121
00122     /** Destructor.
00123      *
00124      */
00125     ~XAcousticCacst() {
00126       /* Do nothing. */
00127     }
00128
00129     /** Decode the acoustic position CACST X-Message from the binary message data.
00130      *
00131      *  @param[in]      data        The binary message data to decode.
00132      */
00133     void DecodeXMsgData(const vector<uint8_t>& data) {
00134
00135         try {
00136
00137           // Check minimum message size.
00138           static const int EXPECTED_SIZE = 33;
00139           if((int) data.size() != EXPECTED_SIZE) {
00140             ostringstream msg;
00141             msg « "Invalid XMessage: expected " « EXPECTED_SIZE « " bytes, but only received "
00142                 « data.size() « " bytes!";
00143             BOOST_THROW_EXCEPTION(log::MessageException(msg));
00144           }
00145
00146           const uint8_t* d = data.data();
00147
00148           set_counter(        u32(d,  4));
00149           set_toa_sec(        u32(d,  8));
00150           set_toa_usec(       u32(d, 12));
```

```
00151           set_mfd_power(      u32(d, 16));
00152           set_src(            u16(d, 20));
00153           set_psk_error_code( u16(d, 22));
00154           set_snr_in(         u32(d, 24));
00155           set_snr_out(        u32(d, 28));
00156 
00157         }CATCH_BLOCK("Failed to decode acoustic position CACST X-Message!")
00158     }
00159 
00160   public:
00161 
00162     /* Helper functions: */
00163 
00164     ptime toa_ptime() const {
00165       return bpt::from_time_t(toa_sec());
00166     }
00167 
00168 
00169     string toa_iso8601_string() const {
00170 
00171       static const string not_a_datetime_string_extended = "0000-00-00T00:00:00.000000Z";
00172 
00173       ostringstream os;
00174 
00175       ptime p = toa_ptime();
00176 
00177       if(p.is_not_a_date_time() || toa_sec() == UINT32_MAX || toa_usec() > 999999) {
00178         os « not_a_datetime_string_extended;
00179       } else {
00180         os « bpt::to_iso_extended_string(p) « ".";
00181         os « format("%06dZ") % toa_usec();
00182       }
00183 
00184       return os.str();
00185     }
00186 
00187     float snr_in_as_float() {
00188       return (float) snr_in() / 10.;
00189     }
00190 
00191     float snr_out_as_float() {
00192       return (float) snr_out() / 10.;
00193     }
00194 
00195 
00196   public:
00197 
00198     /* Mutators and Accessors: */
00199 
00200     uint32_t counter() const {
00201       return counter_;
00202     }
00203 
00204     void set_counter(uint32_t counter) {
00205       counter_ = counter;
00206     }
00207 
00208     uint32_t mfd_power() const {
00209       return mfd_power_;
00210     }
00211 
00212     void set_mfd_power(uint32_t mfd_power) {
00213       mfd_power_ = mfd_power;
00214     }
00215 
00216     uint16_t psk_error_code() const {
00217       return psk_error_code_;
00218     }
00219 
00220     void set_psk_error_code(uint16_t psk_error_code) {
00221       psk_error_code_ = psk_error_code;
00222     }
00223 
00224 
00225     uint32_t toa_sec() const {
00226       return toa_sec_;
00227     }
00228 
00229     void set_toa_sec(uint32_t toa_sec) {
00230       toa_sec_ = toa_sec;
00231     }
00232 
00233     uint32_t toa_usec() const {
00234       return toa_usec_;
00235     }
00236 
00237     void set_toa_usec(uint32_t toa_usec) {
00238       toa_usec_ = toa_usec;
00239     }
00240 
00241     int32_t snr_in() const {
00242       return snr_in_;
00243     }
00244 
00245     void set_snr_in(int32_t snr_in) {
00246       snr_in_ = snr_in;
00247     }
00248 
00249     int32_t snr_out() const {
00250       return snr_out_;
00251     }
00252 
00253     void set_snr_out(int32_t snr_out) {
00254       snr_out_ = snr_out;
00255     }
00256 
00257     int16_t src() const {
00258       return src_;
```

```
00259     }
00260
00261     void set_src(int16_t src) {
00262       src_ = src;
00263     }
00264
00265     int window_idx() const {
00266       return window_idx_;
00267     }
00268
00269     void set_window_idx(int idx) {
00270       window_idx_ = idx;
00271     }
00272 };
00273
00274 /** An acoustic listening window.
00275  *
00276  *  An instance of this class represents a period during which the instrument listened for
00277  *  acoustic signals, decoded from an acoustic position message (see @ref XAcousticPosition): its
00278  *  start and end times, the pressure, temperature and salinity at the start and end, its index,
00279  *  and the number of receptions (see @ref XAcousticCacst) that fall within it.
00280  */
00281 class XAcousticWindow {
00282
00283     friend class XAcousticPosition;
00284
00285   private:
00286
00287     /** Start and end of window timestamps, in UNIX time.
00288      *
00289      */
00290     uint32_t ts_end_, ts_start_;
00291
00292     /** CTD triplet at end of window.
00293      *
00294      */
00295     uint32_t p_end_, t_end_, s_end_;
00296
00297     /** CTD triplet at start of window.
00298      *
00299      */
00300     uint32_t p_start_, t_start_, s_start_;
00301
00302     int idx_;
00303
00304     int rx_count_;
00305
00306   public:
00307
00308     /** Default constructor.
00309      *
00310      */
00311     XAcousticWindow() :
00312             ts_start_(UINT32_MAX),
00313             ts_end_(0),
00314             p_end_(UINT32_MAX),
00315             t_end_(UINT32_MAX),
00316             s_end_(UINT32_MAX),
00317             p_start_(UINT32_MAX),
00318             t_start_(UINT32_MAX),
00319             s_start_(UINT32_MAX),
00320             idx_(-1),
00321             rx_count_(0) {
00322       /* Do nothing. */
00323     }
00324
00325     /** Construct from message data.
00326      *
00327      *  @param[in]      data        The binary message data to decode.
00328      */
00329     XAcousticWindow(const vector<uint8_t>& data) : XAcousticWindow() {
00330       DecodeXMsgData(data);
00331     }
00332
00333
00334     /** Destructor
00335      *
00336      */
00337     ~XAcousticWindow() {
00338       /* Do nothing. */
00339     }
00340
00341
00342     /** Decode the acoustic position CACST X-Message from the binary message data.
00343      *
00344      *  @param[in]      data        The binary message data to decode.
00345      */
00346     void DecodeXMsgData(const vector<uint8_t>& data) {
00347
00348         try {
00349
00350           // Check minimum message size.
00351           static const int EXPECTED_SIZE = 37;
00352           if((int) data.size() != EXPECTED_SIZE) {
00353             ostringstream msg;
00354             msg « "Invalid XMessage: expected " « EXPECTED_SIZE « " bytes, but only received "
00355                 « data.size() « " bytes!";
00356             BOOST_THROW_EXCEPTION(log::MessageException(msg));
00357           }
00358
00359           const uint8_t* d = data.data();
00360
00361           set_ts_start(     u32(d,  4));
00362           set_p_start(      u32(d,  8));
00363           set_t_start(      u32(d, 12));
00364           set_s_start(      u32(d, 16));
00365           set_ts_end(       u32(d, 20));
00366           set_p_end(        u32(d, 24));
```

```
00367           set_t_end(        u32(d, 28));
00368           set_s_end(        u32(d, 32));
00369
00370         }CATCH_BLOCK("Failed to decode acoustic window X-Message!")
00371     }
00372
00373   public:
00374
00375     /* Helper functions: */
00376
00377     ptime ts_end_ptime() const {
00378       return bpt::from_time_t(ts_end());
00379     }
00380
00381     string ts_end_iso8601_string() const {
00382
00383       static const string not_a_datetime_string = "0000-00-00T00:00:00Z";
00384
00385       ostringstream os;
00386
00387       ptime p = ts_end_ptime();
00388
00389       if(p.is_not_a_date_time()) {
00390         os « not_a_datetime_string;
00391       } else {
00392         os « bpt::to_iso_extended_string(p) « "Z";
00393       }
00394
00395       return os.str();
00396     }
00397
00398     ptime ts_start_ptime() const {
00399       return bpt::from_time_t(ts_start());
00400     }
00401
00402     string ts_start_iso8601_string() const {
00403
00404       static const string not_a_datetime_string = "0000-00-00T00:00:00Z";
00405
00406       ostringstream os;
00407
00408       ptime p = ts_start_ptime();
00409
00410       if(p.is_not_a_date_time()) {
00411         os « not_a_datetime_string;
00412       } else {
00413         os « bpt::to_iso_extended_string(p) « "Z";
00414       }
00415
00416       return os.str();
00417     }
00418
00419   public:
00420
00421     /* Mutators and Accessors: */
00422
00423     uint32_t p_end() const {
00424       return p_end_;
00425     }
00426
00427     void set_p_end(uint32_t p_end) {
00428       p_end_ = p_end;
00429     }
00430
00431     uint32_t p_start() const {
00432       return p_start_;
00433     }
00434
00435     void set_p_start(uint32_t p_start) {
00436       p_start_ = p_start;
00437     }
00438
00439     uint32_t s_end() const {
00440       return s_end_;
00441     }
00442
00443     void set_s_end(uint32_t s_end) {
00444       s_end_ = s_end;
00445     }
00446
00447     uint32_t s_start() const {
00448       return s_start_;
00449     }
00450
00451     void set_s_start(uint32_t s_start) {
00452       s_start_ = s_start;
00453     }
00454
00455     uint32_t t_end() const {
00456       return t_end_;
00457     }
00458
00459     void set_t_end(uint32_t t_end) {
00460       t_end_ = t_end;
00461     }
00462
00463     uint32_t t_start() const {
00464       return t_start_;
00465     }
00466
00467     void set_t_start(uint32_t t_start) {
00468       t_start_ = t_start;
00469     }
00470
00471     uint32_t ts_end() const {
00472       return ts_end_;
00473     }
00474
```

```
00475     void set_ts_end(uint32_t ts_end) {
00476       ts_end_ = ts_end;
00477     }
00478
00479     uint32_t ts_start() const {
00480       return ts_start_;
00481     }
00482
00483     void set_ts_start(uint32_t ts_start) {
00484       ts_start_ = ts_start;
00485     }
00486
00487     int idx() const {
00488       return idx_;
00489     }
00490
00491     void set_idx(int idx) {
00492       idx_ = idx;
00493     }
00494
00495     int rx_count() const {
00496       return rx_count_;
00497     }
00498
00499     void set_rx_count(int rx_count) {
00500       rx_count_ = rx_count;
00501     }
00502 };
00503
00504 /** Class representing an acoustic position X-Message.
00505  *
00506  *  @note       This message type was added in X-Message version 2.0.
00507  *  @note       This message type is used by marginal ice-zone (MIZ) floats only.
00508  *
00509  */
00510 class XAcousticPosition:
00511         public util::xmsg::XMessage {
00512
00513   private:
00514
00515     vector<shared_ptr<XAcousticCacst» acoustic_cacst_;
00516
00517     vector<shared_ptr<XAcousticWindow» acoustic_window_;
00518
00519   public:
00520
00521     /* Public member functions: */
00522
00523     /** Parameterized constructor.
00524      *
00525      *  @param[in]      dive        The dive or cycle number of this message.
00526      *  @param[in]      whoi        The unique WHOI identifier.
00527      *  @param[in]      data        The binary X-message data to decode.
00528      *
00529      *  @throw          util::log::MessageException if an decode error occurs.
00530      */
00531     XAcousticPosition(int16_t dive, uint16_t whoi, const vector<uint8_t>& data) :
00532                       XMessage(dive, whoi, data) {
00533         DecodeXMsgData(data);    // Decode message.
00534     }
00535
00536     /** Destructor.
00537      *
00538      */
00539     ~XAcousticPosition() {
00540         /* Do nothing. */
00541     }
00542
00543     /** Decode the acoustic position X-Message from the binary message data.
00544      *
00545      *  @param[in]      data        The binary message data to decode.
00546      */
00547     void DecodeXMsgData(const vector<uint8_t>& data) {
00548
00549         try {
00550
00551             // Check message.
00552             const uint8_t* d = CheckXMsgData(data, XMSG_TYPE_ID_MIZ_ACOUSTIC, -1);
00553
00554             /* Decode message based on subtype: */
00555
00556             uint8_t subtype_id = u08(d, 3);
00557
00558             switch (subtype_id) {
00559
00560               case 3: // window
00561
00562                 acoustic_window_.push_back(make_shared<XAcousticWindow>(data));
00563
00564                 break;
00565
00566               case 5: // CACST
00567
00568                 acoustic_cacst_.push_back(make_shared<XAcousticCacst>(data));
00569
00570                 break;
00571
00572               default:
00573                 break;
00574             }
00575
00576             /** @todo   Implement the functionality for decoding the acoustic position message.
00577              *
00578              */
00579
00580         }CATCH_BLOCK("Failed to decode acoustic position X-Message!")
00581     }
00582
```

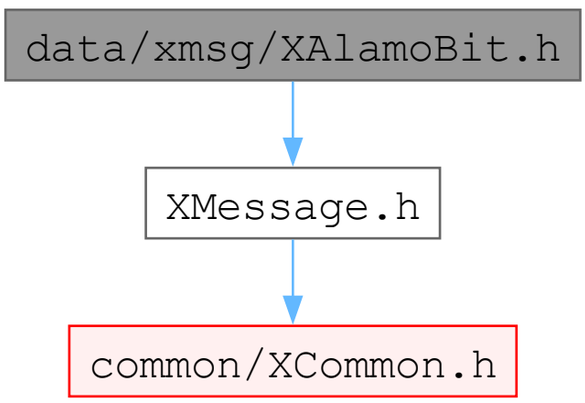
```
00583     /** Generate a listing string representation of the acoustic position message.
00584      *
00585      *  @return     A listing string representation of the acoustic position message.
00586      */
00587     const string GenerateListingString() const {
00588         ostringstream os;
00589 
00590         try {
00591 
00592 
00593 
00594           /** @todo   Implement the functionality for generating a listing string for the
00595            *          acoustic position message.
00596            *
00597            */
00598           os « "# --- Acoustic Listening Windows ------------------------------------------------"
00599               "--------------------"
00600              « endl;
00601 
00602           const format fmt3_h(
00603               "#   %5s %4s %4s %4s"
00604               " %20s %20s"
00605               );
00606           os « format(fmt3_h)
00607               % "SN" % "dive" % "midx" % "widx"
00608               % "start-date/time" % "end-date/time"
00609               « endl;
00610           os « format(fmt3_h)
00611               % "" % "" % "" % ""
00612               % "(UTC)" % "(UTC)"
00613               « endl;
00614 
00615           const format fmt3(
00616               "APS %5d %4d %4d %4d"
00617               " %20s %20s "
00618               );
00619 
00620           for_each(
00621               acoustic_window_.begin(),
00622               acoustic_window_.end(),
00623               [&](const shared_ptr<XAcousticWindow> window) {
00624 
00625             os « format(fmt3)
00626                 % id_instrument() % id_dive() % 3 % window->idx()
00627                 % window->ts_start_iso8601_string() % window->ts_end_iso8601_string()
00628                 « endl;
00629           });
00630 
00631 
00632 
00633           os « "# --- Acoustic Communication Cycle Receive Statistics ---------------------------"
00634               "--------------------"
00635              « endl;
00636 
00637           const format fmt5_h(
00638               "#   %5s %4s %4s %4s"
00639               " %27s %6s %6s"
00640               " %6s %6s %6s");
00641 
00642           os « format(fmt5_h)
00643               % "SN" % "dive" % "midx" % "widx"
00644               % "date/time" % "src" % "MFDPwr"
00645               % "PSKErr" % "SNRIn" % "SNROut"
00646               « endl;
00647           os « format(fmt5_h)
00648               % "" % "" % "" % ""
00649               % "(UTC)" % "" % "(dB)"
00650               % "" % "(dB)" % "(dB)"
00651 
00652               « endl;
00653 
00654           const format fmt5(
00655               "APS %5d %4d %4d %4d"
00656               " %27s %6d %6d"
00657               " %6d %6.1f %6.1f");
00658 
00659           for_each(
00660               acoustic_cacst_.begin(),
00661               acoustic_cacst_.end(),
00662               [&](const shared_ptr<XAcousticCacst> cacst){
00663 
00664             os « format(fmt5)
00665                 % id_instrument() % id_dive() % 5 % cacst->window_idx()
00666                 % cacst->toa_iso8601_string() % (int)cacst->src() % (int)cacst->mfd_power()
00667                 % cacst->psk_error_code() % cacst->snr_in_as_float() % cacst->snr_in_as_float()
00668                 « endl;
00669           });
00670 
00671         } CATCH_BLOCK("Failed to generate listing string for acoustic position message!")
00672 
00673         return os.str();
00674     }
00675 
00676   public:
00677 
00678     /** Number the acoustic listening windows and assign each ping to its window.
00679      *
00680      *  Windows are numbered in order, starting at 0. Each ping (CACST record) whose time of
00681      *  arrival falls within a window's start and end times is given that window's index, and the
00682      *  window's receive count is incremented. A ping outside every window keeps its window index
00683      *  unchanged.
00684      *
00685      *  @throw      util::log::MessageException if an error occurs.
00686      */
00687     void AssignWindowIndices(void) {
00688 
00689       try {
00690 
```

```
00691         // Assign indices to windows.
00692
00693         int idx = 0;
00694         for_each(
00695             acoustic_window_.begin(),
00696             acoustic_window_.end(),
00697             [&](shared_ptr<XAcousticWindow> window) {
00698           window->set_idx(idx++);
00699         });
00700
00701         // Assign pings to windows.
00702
00703         for(auto it_cacst = acoustic_cacst_.begin();
00704             it_cacst != acoustic_cacst_.end();
00705             it_cacst++) {
00706           shared_ptr<XAcousticCacst> cacst = *it_cacst;
00707
00708           // Find which window this ping belongs to.
00709
00710           for(auto it_w = acoustic_window_.begin(); it_w != acoustic_window_.end(); it_w++) {
00711             shared_ptr<XAcousticWindow> window = *it_w;
00712
00713             uint32_t ts = cacst->toa_sec();
00714
00715             if((ts >= window->ts_start()) && (ts <= window->ts_end())) {
00716
00717               cacst->set_window_idx(window->idx());
00718               window->set_rx_count(window->rx_count()+1);
00719               break;
00720
00721             }
00722           }
00723         }
00724       } CATCH_BLOCK("Failed to assign acoustic window indices.")
00725     }
00726 };
00727
00728 } /* namespace xmsg */
00729 } /* namespace util */
00730 #endif /* XACOUSTICPOSITION_H_ */
```

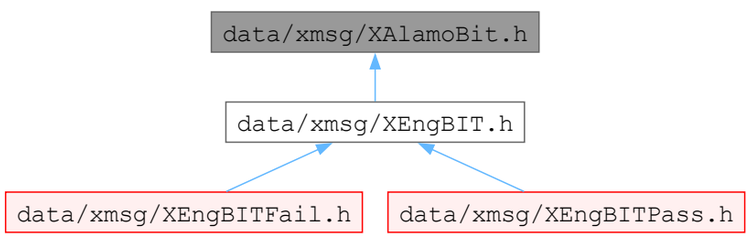
11.67 data/xmsg/XAlamoBit.h File Reference

The util::xmsg::XAlamoBit class header file.

`#include "XMessage.h"`
Include dependency graph for XAlamoBit.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XAlamoBit

  Class representing the BIT test of an ALAMO float.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.67.1 Detailed Description

The util::xmsg::XAlamoBit class header file.

Version

0.6

Date

Aug 14, 2014 11:24:25 AM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.68 XAlamoBit.h

Go to the documentation of this file.

```
00001 /** The @ref util::xmsg::XAlamoBit class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Aug 14, 2014 11:24:25 AM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef XALAMOBIT_H_
00024 #define XALAMOBIT_H_
00025
00026 /* Includes */
00027
00028 #include "XMessage.h"
00029
00030 /* Definitions */
00031
00032 namespace util {
00033 namespace xmsg {
```

```
00034
00035 /* Class Definitions */
00036
00037 /** Class representing the BIT test of an ALAMO float.
00038  *
00039  */
00040 class XAlamoBit {
00041
00042 private:
00043
00044     /* Data Members */
00045
00046     /** The Built-in-Test (BIT) version identifier.
00047      *
00048      */
00049     uint8_t bit_version_;
00050
00051     /** The software version identifier.
00052      *
00053      */
00054     uint32_t float_software_version_;
00055
00056     /** The Built-in-Test (BIT) status code.
00057      *
00058      */
00059     uint32_t bit_status_code_;
00060
00061     uint16_t bit_duration_;
00062
00063     uint16_t bit_pressure_;
00064
00065     uint16_t bit_temperature_;
00066
00067     uint16_t bit_salinity_;
00068
00069     bool bit_passed_;
00070
00071     uint32_t ctd_status_;
00072
00073     uint16_t vac_at_start_of_bit_;
00074
00075     uint16_t vac_at_end_of_bit_;
00076
00077     uint16_t battery_voltage_at_start_of_bit_;
00078
00079     uint16_t battery_voltage_at_end_of_bit_;
00080
00081     uint16_t air_valve_open_position_;
00082
00083     uint16_t air_valve_open_cal_position_;
00084
00085     uint16_t air_valve_closed_position_;
00086
00087     uint16_t air_valve_closed_cal_position_;
00088
00089     uint16_t air_valve_middle_position_;
00090
00091     uint16_t air_valve_middle_cal_position_;
00092
00093     uint16_t air_pump_test_duration_;
00094
00095     uint8_t air_pump_cycle_count_;
00096
00097     uint16_t air_pressure_max_;
00098
00099     uint16_t air_pump_max_load_current_;
00100
00101     uint16_t air_pump_max_load_current_12v_;
00102
00103     uint16_t air_pump_max_load_battery_voltage_;
00104
00105     uint16_t oil_valve_open_position_;
00106
00107     uint16_t oil_valve_open_cal_position_;
00108
00109     uint16_t oil_valve_closed_position_;
00110
00111     uint16_t oil_valve_closed_cal_position_;
00112
00113     uint16_t oil_pump_test_duration_;
00114
00115     uint16_t oil_pump_max_load_current_;
00116
00117     uint16_t oil_pump_max_load_current_12v_;
00118
00119     uint16_t oil_pump_max_load_battery_voltage_;
00120
00121     uint16_t pres_counts_after_offset_reset_;
00122
00123     uint16_t pres_counts_before_offset_reset_;
00124
00125     /** The measured pressure offset, in 1/1000 dBar.
00126      *
00127      */
00128     uint16_t pres_offset_;
00129
00130 protected:
00131
00132     /* Member Functions */
00133
00134     /** Set the Built-in-Test (BIT) status code.
00135      *
00136      *  @param[in]  bit_status_code     The Built-in-Test (BIT) status code to set.
00137      */
00138     void set_bit_status_code(uint32_t bit_status_code) {
00139         bit_status_code_ = bit_status_code;
00140     }
00141
```

```
00142     /** Set the Built-in-Test (BIT) version.
00143      *
00144      *  @param[in]  bit_version         The Built-in-Test (BIT) version to set.
00145      */
00146     void set_bit_version(uint8_t bit_version) {
00147         bit_version_ = bit_version;
00148     }
00149
00150     /** Set the instrument software version.
00151      *
00152      *  @param[in]  float_software_version  The instrument software version to set.
00153      */
00154     void set_float_software_version(uint32_t float_software_version) {
00155         float_software_version_ = float_software_version;
00156     }
00157
00158     void set_air_pressure_max(uint16_t air_pressure_max) {
00159         air_pressure_max_ = air_pressure_max;
00160     }
00161
00162     void set_air_pump_cycle_count(uint8_t air_pump_cycle_count) {
00163         air_pump_cycle_count_ = air_pump_cycle_count;
00164     }
00165
00166     void set_air_pump_max_load_battery_voltage(uint16_t air_pump_max_load_battery_voltage) {
00167         air_pump_max_load_battery_voltage_ = air_pump_max_load_battery_voltage;
00168     }
00169
00170     void set_air_pump_max_load_current(uint16_t air_pump_max_load_current) {
00171         air_pump_max_load_current_ = air_pump_max_load_current;
00172     }
00173
00174     void set_air_pump_max_load_current_12v(uint16_t air_pump_max_load_current_12v) {
00175         air_pump_max_load_current_12v_ = air_pump_max_load_current_12v;
00176     }
00177
00178     void set_air_pump_test_duration(uint16_t air_pump_test_duration) {
00179         air_pump_test_duration_ = air_pump_test_duration;
00180     }
00181
00182     void set_air_valve_closed_cal_position(uint16_t air_valve_closed_cal_position) {
00183         air_valve_closed_cal_position_ = air_valve_closed_cal_position;
00184     }
00185
00186     void set_air_valve_closed_position(uint16_t air_valve_closed_position) {
00187         air_valve_closed_position_ = air_valve_closed_position;
00188     }
00189
00190     void set_air_valve_middle_cal_position(uint16_t air_valve_middle_cal_position) {
00191         air_valve_middle_cal_position_ = air_valve_middle_cal_position;
00192     }
00193
00194     void set_air_valve_middle_position(uint16_t air_valve_middle_position) {
00195         air_valve_middle_position_ = air_valve_middle_position;
00196     }
00197
00198     void set_air_valve_open_cal_position(uint16_t air_valve_open_cal_position) {
00199         air_valve_open_cal_position_ = air_valve_open_cal_position;
00200     }
00201
00202     void set_air_valve_open_position(uint16_t air_valve_open_position) {
00203         air_valve_open_position_ = air_valve_open_position;
00204     }
00205
00206     void set_battery_voltage_at_end_of_bit(uint16_t battery_voltage_at_end_of_bit) {
00207         battery_voltage_at_end_of_bit_ = battery_voltage_at_end_of_bit;
00208     }
00209
00210     void set_battery_voltage_at_start_of_bit(uint16_t battery_voltage_at_start_of_bit) {
00211         battery_voltage_at_start_of_bit_ = battery_voltage_at_start_of_bit;
00212     }
00213
00214     void set_bit_duration(uint16_t bit_duration) {
00215         bit_duration_ = bit_duration;
00216     }
00217
00218     void set_bit_passed(bool bit_passed) {
00219         bit_passed_ = bit_passed;
00220     }
00221
00222     void set_bit_pressure(uint16_t bit_pressure) {
00223         bit_pressure_ = bit_pressure;
00224     }
00225
00226     void set_bit_salinity(uint16_t bit_salinity) {
00227         bit_salinity_ = bit_salinity;
00228     }
00229
00230     void set_bit_temperature(uint16_t bit_temperature) {
00231         bit_temperature_ = bit_temperature;
00232     }
00233
00234     void set_ctd_status(uint32_t ctd_status) {
00235         ctd_status_ = ctd_status;
00236     }
00237
00238     void set_oil_pump_max_load_current(uint16_t oil_pump_max_load_current) {
00239         oil_pump_max_load_current_ = oil_pump_max_load_current;
00240     }
00241
00242     void set_oil_pump_max_load_current_12v(uint16_t oil_pump_max_load_current_12v) {
00243         oil_pump_max_load_current_12v_ = oil_pump_max_load_current_12v;
00244     }
00245
00246     void set_oil_pump_max_load_battery_voltage(uint16_t oil_pump_max_load_battery_voltage) {
00247         oil_pump_max_load_battery_voltage_ = oil_pump_max_load_battery_voltage;
00248     }
00249
```

```
00250     void set_oil_pump_test_duration(uint16_t oil_pump_test_duration) {
00251         oil_pump_test_duration_ = oil_pump_test_duration;
00252     }
00253
00254     void set_oil_valve_closed_cal_position(uint16_t oil_valve_closed_cal_position) {
00255         oil_valve_closed_cal_position_ = oil_valve_closed_cal_position;
00256     }
00257
00258     void set_oil_valve_closed_position(uint16_t oil_valve_closed_position) {
00259         oil_valve_closed_position_ = oil_valve_closed_position;
00260     }
00261
00262     void set_oil_valve_open_cal_position(uint16_t oil_valve_open_cal_position) {
00263         oil_valve_open_cal_position_ = oil_valve_open_cal_position;
00264     }
00265
00266     void set_oil_valve_open_position(uint16_t oil_valve_open_position) {
00267         oil_valve_open_position_ = oil_valve_open_position;
00268     }
00269
00270     void set_vac_at_end_of_bit(uint16_t vac_at_end_of_bit) {
00271         vac_at_end_of_bit_ = vac_at_end_of_bit;
00272     }
00273
00274     void set_vac_at_start_of_bit(uint16_t vac_at_start_of_bit) {
00275         vac_at_start_of_bit_ = vac_at_start_of_bit;
00276     }
00277
00278     void set_pres_counts_after_offset_reset(uint16_t pres_counts_after_offset_reset) {
00279         pres_counts_after_offset_reset_ = pres_counts_after_offset_reset;
00280     }
00281
00282     void set_pres_counts_before_offset_reset(uint16_t pres_counts_before_offset_reset) {
00283         pres_counts_before_offset_reset_ = pres_counts_before_offset_reset;
00284     }
00285
00286     void set_pres_offset(uint16_t pres_offset) {
00287         pres_offset_ = pres_offset;
00288     }
00289
00290 public:
00291
00292     /* Member Functions */
00293
00294     /** The parameterized constructor.
00295      *
00296      *  @param[in]      data        The binary data to decode.
00297      *
00298      *  @throw          log::MessageException if an error occurs while decoding.
00299      */
00300     XAlamoBit(const vector<uint8_t>& data) :
00301                     bit_version_(0xFF),
00302                     float_software_version_(0xFFFFFF),
00303                     bit_status_code_(0xFFFFFF),
00304                     bit_duration_(0xFFFF),
00305                     bit_pressure_(0xFFFF),
00306                     bit_temperature_(0xFFFF),
00307                     bit_salinity_(0xFFFF),
00308                     ctd_status_(0xFFFFFFFF),
00309                     vac_at_start_of_bit_(0xFFFF),
00310                     vac_at_end_of_bit_(0xFFFF),
00311                     battery_voltage_at_start_of_bit_(0xFFFF),
00312                     battery_voltage_at_end_of_bit_(0xFFFF),
00313                     air_valve_open_position_(0xFFFF),
00314                     air_valve_open_cal_position_(0xFFFF),
00315                     air_valve_closed_position_(0xFFFF),
00316                     air_valve_closed_cal_position_(0xFFFF),
00317                     air_valve_middle_position_(0xFFFF),
00318                     air_valve_middle_cal_position_(0xFFFF),
00319                     air_pump_test_duration_(0xFFFF),
00320                     air_pump_cycle_count_(0),
00321                     air_pressure_max_(0xFFFF),
00322                     air_pump_max_load_current_(0xFFFF),
00323                     air_pump_max_load_current_12v_(0xFFFF),
00324                     air_pump_max_load_battery_voltage_(0xFFFF),
00325                     oil_valve_open_position_(0xFFFF),
00326                     oil_valve_open_cal_position_(0xFFFF),
00327                     oil_valve_closed_position_(0xFFFF),
00328                     oil_valve_closed_cal_position_(0xFFFF),
00329                     oil_pump_test_duration_(0xFFFF),
00330                     oil_pump_max_load_current_(0xFFFF),
00331                     oil_pump_max_load_current_12v_(0xFFFF),
00332                     oil_pump_max_load_battery_voltage_(0xFFFF) {
00333
00334         // Decode the message.
00335         DecodeXMsgData(data);
00336
00337     }
00338
00339     /** The destructor. */
00340     ~XAlamoBit() {
00341         /* Do nothing. */
00342     }
00343
00344     /** Decode the ALAMO BIT test engineering message from the binary message data.
00345      *
00346      *  @param[in]      data        The binary data to decode.
00347      *
00348      *  @throw          log::MessageException if an error occurs while decoding.
00349      */
00350     void DecodeXMsgData(const vector<uint8_t>& data) {
00351         try {
00352             const uint8_t* d = data.data();
00353
00354             // Check message size.
00355             if(data.size() < 13) {
00356                 ostringstream os;
00357                 os « "Invalid X-Message: Expected at least 13 bytes, but only received"
```

```
00358                    « data.size()
00359                    « " bytes!";
00360                EXC_THROW(os.str());
00361            }
00362
00363            // Check if BIT passed or failed.
00364            if(d[0] == 0xE5)
00365                set_bit_passed(true);
00366            else if(d[0] == 0xE6)
00367                set_bit_passed(false);
00368            else {
00369                ostringstream os;
00370                os « "Invalid X-Message: Expected X-Message format identifiers 0xE5 or 0xE6, but received
     0x"
00371                   « std::hex
00372                   « d[0]
00373                   « "!";
00374                EXC_THROW(os.str());
00375            }
00376
00377            // Get BIT version.
00378            set_bit_version(u08(d, 3));
00379
00380            switch(bit_version_) {
00381
00382            case 6: // ALAMO BIT version 6.
00383
00384                // Check the message size.
00385                if(data.size() != 13) {
00386                    ostringstream os;
00387                    os « "Invalid X-Message: Expected at 13 bytes, but received "
00388                       « data.size()
00389                       « " bytes!";
00390                    EXC_THROW(os.str());
00391                }
00392
00393                set_float_software_version(u32(d, 4));
00394                set_bit_status_code(u32(d, 8));
00395
00396                break;
00397
00398            case 7: // ALAMO BIT version 7.
00399
00400                // Check the message size.
00401                if(data.size() != 72) {
00402                    ostringstream os;
00403                    os « "Invalid X-Message: Expected 72 bytes, but received "
00404                       « data.size()
00405                       « " bytes!";
00406                    EXC_THROW(os.str());
00407                }
00408
00409                set_float_software_version(u32(d, 4));
00410                set_bit_status_code(u32(d, 8));
00411                set_bit_duration(u16(d, 12));
00412                set_bit_pressure(u16(d, 14));
00413                set_bit_temperature(u16(d, 16));
00414                set_bit_salinity(u16(d, 18));
00415                set_ctd_status(u32(d, 20));
00416                set_vac_at_start_of_bit(u16(d, 24));
00417                set_vac_at_end_of_bit(u16(d, 26));
00418                set_battery_voltage_at_start_of_bit(u16(d, 28));
00419                set_battery_voltage_at_end_of_bit(u16(d, 30));
00420                set_air_valve_open_position(u16(d, 32));
00421                set_air_valve_open_cal_position(u16(d, 34));
00422                set_air_valve_closed_position(u16(d, 36));
00423                set_air_valve_closed_cal_position(u16(d, 38));
00424                set_air_valve_middle_position(u16(d, 40));
00425                set_air_valve_middle_cal_position(u16(d, 42));
00426                set_air_pump_test_duration(u16(d, 44));
00427                set_air_pump_cycle_count(u08(d, 46));
00428                set_air_pressure_max(u16(d, 47));
00429                set_air_pump_max_load_current(u16(d, 49));
00430                set_air_pump_max_load_current_12v(u16(d, 51));
00431                set_air_pump_max_load_battery_voltage(u16(d, 53));
00432                set_oil_valve_open_position(u16(d, 55));
00433                set_oil_valve_open_cal_position(u16(d, 57));
00434                set_oil_valve_closed_position(u16(d, 59));
00435                set_oil_valve_closed_cal_position(u16(d, 61));
00436                set_oil_pump_test_duration(u16(d, 63));
00437                set_oil_pump_max_load_current(u16(d, 65));
00438                set_oil_pump_max_load_current_12v(u16(d, 67));
00439                set_oil_pump_max_load_battery_voltage(u16(d, 69));
00440
00441                break;
00442
00443            case 8: // ALAMO BIT version 8.
00444
00445              // Check the message size.
00446              if(data.size() != 76) {
00447                  ostringstream os;
00448                  os « "Invalid X-Message: Expected 76 bytes, but received "
00449                     « data.size()
00450                     « " bytes!";
00451                  EXC_THROW(os.str());
00452              }
00453
00454              set_float_software_version(u32(d, 4));
00455              set_bit_status_code(u32(d, 8));
00456              set_bit_duration(u16(d, 12));
00457              set_pres_counts_before_offset_reset(u16(d, 14));
00458              set_pres_offset(u16(d, 16));
00459              set_pres_counts_after_offset_reset(u16(d, 18));
00460              set_bit_temperature(u16(d, 22));
00461              set_bit_salinity(u16(d, 22));
00462              set_ctd_status(u32(d, 24));
00463              set_vac_at_start_of_bit(u16(d, 28));
00464              set_vac_at_end_of_bit(u16(d, 30));
```

```
00465               set_battery_voltage_at_start_of_bit(u16(d, 32));
00466               set_battery_voltage_at_end_of_bit(u16(d, 34));
00467               set_air_valve_open_position(u16(d, 36));
00468               set_air_valve_open_cal_position(u16(d, 48));
00469               set_air_valve_closed_position(u16(d, 40));
00470               set_air_valve_closed_cal_position(u16(d, 42));
00471               set_air_valve_middle_position(u16(d, 44));
00472               set_air_valve_middle_cal_position(u16(d, 46));
00473               set_air_pump_test_duration(u16(d, 48));
00474               set_air_pump_cycle_count(u08(d, 50));
00475               set_air_pressure_max(u16(d, 51));
00476               set_air_pump_max_load_current(u16(d, 53));
00477               set_air_pump_max_load_current_12v(u16(d, 55));
00478               set_air_pump_max_load_battery_voltage(u16(d, 57));
00479               set_oil_valve_open_position(u16(d, 59));
00480               set_oil_valve_open_cal_position(u16(d, 61));
00481               set_oil_valve_closed_position(u16(d, 63));
00482               set_oil_valve_closed_cal_position(u16(d, 65));
00483               set_oil_pump_test_duration(u16(d, 67));
00484               set_oil_pump_max_load_current(u16(d, 69));
00485               set_oil_pump_max_load_current_12v(u16(d, 71));
00486               set_oil_pump_max_load_battery_voltage(u16(d, 73));
00487
00488               break;
00489
00490           default: { // Unsupported ALAMO BIT version.
00491
00492                ostringstream os;
00493                os « "Unsupported ALAMO BIT version: " « (int) bit_version_ « "!";
00494                EXC_THROW(os.str());
00495             }
00496
00497               break;
00498           }
00499       }CATCH_BLOCK("Failed to decode ALAMO Engineering (BIT) data message!")
00500   }
00501
00502   /** Generate a listing entry string representation of the the ALAMO BIT test engineering
00503    *  message.
00504    *
00505    *  @param[in]      whoi        The instrument serial number.
00506    *  @param[in]      dive        The dive number of this ALAMO BIT test engineering message.
00507    *  @param[in]      pg          The pressure sensor sensitivity scale factor, in LSB/dBar.
00508    *  @param[in]      po          The pressure sensor offset, in dBar.
00509    *  @param[in]      sg          The salinity sensor sensitivity scale factor, in LSB/PSU.
00510    *  @param[in]      so          The salinity sensor offset, in PSU.
00511    *  @param[in]      tg          The temperature sensor sensitivity scale factor, in LSB/degC.
00512    *  @param[in]      to          The temperatUre sensor offset, in degC.
00513    *
00514    *  @return         A listing entry string representation of the the ALAMO BIT test engineering
00515    *                  message.
00516    */
00517   const string GenerateListingString(
00518           int whoi,
00519           int dive,
00520           uint16_t pg,
00521           uint16_t po,
00522           uint16_t sg,
00523           uint16_t so,
00524           uint16_t tg,
00525           uint16_t to) const {
00526       ostringstream os;
00527
00528       static const format fmt1("#   %5s %4s %4s "
00529               "%15s "
00530               "%15s "
00531               "%15s");
00532       static const format fmt2("E0%c %5d %4d %4d "
00533               "%15d "
00534               "     0x%08X "
00535               "     0x%08X");
00536
00537       switch(bit_version_) {
00538
00539       case 6: { // ALAMO BIT version 6.
00540
00541           // Minor index 0.
00542           os « format(fmt1) % "SN" % "dive" % "midx" % "BITVer" % "SWVer" % "BITResults" « endl;
00543           os « format(fmt2) % (is_bit_passed() ? '5' : '6') % whoi % dive % 0
00544                   % (int) bit_version() % float_software_version() % bit_status_code();
00545
00546       }
00547
00548           break;
00549
00550       case 7: { // ALAMO BIT version 7.
00551
00552           // Minor index 0.
00553           os « format(fmt1) % "SN" % "dive" % "midx" % "BITVer" % "SWVer" % "BITResults" « endl;
00554           os « format(fmt2) % (is_bit_passed() ? '5' : '6') % whoi % dive % 0
00555                   % (int) bit_version() % float_software_version() % bit_status_code()
00556              « endl;
00557
00558           // Minor index 1.
00559           static const format fmt3("#   %5s %4s %4s %7s   %8s %8s   %7s %8s   %7s %8s");
00560           static const format fmt4("#   %5s %4s %4s %7s   %8s %9s   %7s %8s   %7s %8s");
00561           static const format fmt5("E0%c %5d %4d %4d %7d   %8.3f %8d   %7.3f %8d   %7.3f %8d");
00562           os « "#
      ----------------------------------------------------------------------------------------------"
00563              « endl;
00564           os « format(fmt3) % "SN" % "dive" % "midx" % "BITTime" % "pres" % "pres" % "temp"
00565                   % "temp" % "saln" % "saln"
00566              « endl;
00567           os « format(fmt4) % "" % "" % "" % "(sec)" % "(dBar)" % "(counts)" % "(degC)"
00568                   % "(counts)" % "(PSU)" % "(counts)"
00569              « endl;
00570           os « format(fmt5) % (is_bit_passed() ? '5' : '6') % whoi % dive % 1 % bit_duration()
00571                   % GetDecodedBitPressure(pg, po) % bit_pressure()
```

```
00572                       % GetDecodedBitTemperature(tg, to) % bit_temperature()
00573                       % GetDecodedBitSalinity(sg, so) % bit_salinity()
00574                  « endl;
00575
00576              // Minor index 2.
00577              static const format fmt6("#   %5s %4s %4s %10s %8s %8s %8s %8s");
00578              static const format fmt7("#   %5s %4s %4s %10s %8s %8s %8s %8s");
00579              static const format fmt8("E0%c %5d %4d %4d 0x%08X %8.2f %8.2f %8.2f %8.2f");
00580              os « "#
    ------------------------------------------------------------------------------------------------------------"
00581                 « endl;
00582              os « format(fmt6) % "SN" % "dive" % "midx" % "CTDStat" % "VacStart" % "VacEnd"
00583                       % "BatStart" % "BatEnd"
00584                 « endl;
00585              os « format(fmt7) % "" % "" % "" % "" % "(inHg)" % "(inHg)" % "(V)" % "(V)" « endl;
00586              os « format(fmt8) % (is_bit_passed() ? '5' : '6') % whoi % dive
00587                       % 2 //
00588                       % ctd_status() % GetDecodedVacAtStartOfBit() % GetDecodedVacAtEndOfBit()
00589                       % GetDecodedBatteryVoltageAtStartOfBit()
00590                       % GetDecodedBatteryVoltageAtEndOfBit()
00591                  « endl;
00592
00593              // Minor index 3.
00594              static const format fmt9("#   %5s %4s %4s %13s %13s %11s %11s %14s %13s");
00595              static const format fmt10("#   %5s %4s %4s %13s %13s %11s %11s %14s %13s");
00596              static const format fmt11("E0%c %5d %4d %4d %13.3f %13.3f %11.3f %11.3f %14.3f %13.3f");
00597              os « "# --- Air Valve (AV) Tests
    ------------------------------------------------------------------------"
00598                 « endl;
00599              os « format(fmt9) % "SN" % "dive" % "midx" % "AVOpenPos" % "AVOpenPosCal" % "AVMidPos"
00600                       % "AVMidPosCal" % "AVClosePos" % "AVClosePosCal"
00601                 « endl;
00602              os « format(fmt10) % "" % "" % "" % "(V)" % "(V)" % "(V)" % "(V)" % "(V)" % "(V)"
00603                 « endl;
00604              os « format(fmt11) % (is_bit_passed() ? '5' : '6') % whoi % dive % 3
00605                       % ((float) air_valve_open_position_ / 1000)
00606                       % ((float) air_valve_open_cal_position_ / 1000)
00607                       % ((float) air_valve_middle_position_ / 1000)
00608                       % ((float) air_valve_middle_cal_position_ / 1000)
00609                       % ((float) air_valve_closed_position_ / 1000)
00610                       % ((float) air_valve_closed_cal_position_ / 1000)
00611                  « endl;
00612
00613              // Minor index 4.
00614              static const format fmt12("#   %5s %4s %4s %13s %13s %11s %11s %14s %13s");
00615              static const format fmt13("#   %5s %4s %4s %13s %13s %11s %11s %14s %13s");
00616              static const format fmt14("E0%c %5d %4d %4d %13d %13d %11.3f %11.3f %14.3f %13.3f");
00617              os « "# --- Air Pump (AP) Tests
    ------------------------------------------------------------------------"
00618                 « endl;
00619              os « format(fmt12) % "SN" % "dive" % "midx" % "APTestTime" % "APCycleCount"
00620                       % "APMxAPres" % "APMxLoadCur" % "APMxLoadCur12V" % "APMxLoadBatV"
00621                 « endl;
00622              os « format(fmt13) % "" % "" % "" % "(sec)" % "" % "(PSI)" % "(A)" % "(A)" % "(V)"
00623                 « endl;
00624              os « format(fmt14) % (is_bit_passed() ? '5' : '6') % whoi % dive % 4
00625                       % air_pump_test_duration() % (int) air_pump_cycle_count()
00626                       % ((float) air_pressure_max() / 1000)
00627                       % ((float) air_pump_max_load_current() / 1000)
00628                       % ((float) air_pump_max_load_current_12v() / 1000)
00629                       % ((float) air_pump_max_load_battery_voltage() / 1000)
00630                  « endl;
00631
00632              // Minor index 5.
00633              static const format fmt15("#   %5s %4s %4s %13s %13s %14s %14s");
00634              static const format fmt16("#   %5s %4s %4s %13s %13s %14s %14s");
00635              static const format fmt17("E0%c %5d %4d %4d %13.3f %13.3f %14.3f %14.3f");
00636              os « "# --- Oil Valve (OV) Tests
    ------------------------------------------------------------------------"
00637                 « endl;
00638              os « format(fmt15) % "SN" % "dive" % "midx" % "OVOpenPos" % "OVOpenPosCal"
00639                       % "OVClosePos" % "OVClosePosCal"
00640                 « endl;
00641              os « format(fmt16) % "" % "" % "" % "(V)" % "(V)" % "(V)" % "(V)" « endl;
00642              os « format(fmt17) % (is_bit_passed() ? '5' : '6') % whoi % dive % 5
00643                       % ((float) oil_valve_open_position() / 1000)
00644                       % ((float) oil_valve_open_cal_position() / 1000)
00645                       % ((float) oil_valve_closed_position() / 1000)
00646                       % ((float) oil_valve_closed_cal_position() / 1000)
00647                  « endl;
00648
00649              // Minor index 6.
00650              static const format fmt18("#   %5s %4s %4s %13s %13s %14s %14s");
00651              static const format fmt19("#   %5s %4s %4s %13s %13s %14s %14s");
00652              static const format fmt20("E0%c %5d %4d %4d %13d %13.3f %14.3f %14.3f");
00653              os « "# --- Oil Pump (OP) Tests
    ------------------------------------------------------------------------"
00654                 « endl;
00655              os « format(fmt18) % "SN" % "dive" % "midx" % "OPTestTime" % "OPMxLoadCur"
00656                       % "OPMxLoadCur12V" % "OPMxLoadBatV"
00657                 « endl;
00658              os « format(fmt19) % "" % "" % "" % "(sec)" % "(A)" % "(A)" % "(V)" « endl;
00659              os « format(fmt20) % (is_bit_passed() ? '5' : '6') % whoi % dive % 6
00660                     % oil_pump_test_duration() % ((float) oil_pump_max_load_current() / 1000)
00661                     % ((float) oil_pump_max_load_current_12v() / 1000)
00662                     % ((float) oil_pump_max_load_battery_voltage() / 1000);
00663
00664          }
00665
00666              break;
00667
00668          case 8: { // ALAMO BIT version 8.
00669
00670              // Minor index 0.
00671              os « format(fmt1) % "SN" % "dive" % "midx" % "BITVer" % "SWVer" % "BITResults" « endl;
00672              os « format(fmt2) % (is_bit_passed() ? '5' : '6') % whoi % dive % 0
00673                       % (int) bit_version() % float_software_version() % bit_status_code()
00674                  « endl;
```

```
00675
00676             // Minor index 1.
00677             static const format fmt21("#   %5s %4s %4s " //
00678                     "%15s"
00679                     "%15s "
00680                     );
00681             static const format fmt22("#   %5s %4s %4s "
00682                     "%15s"
00683                     "%15s "
00684                     );
00685             static const format fmt23("E0%c %5d %4d %4d "
00686                     "%15d"
00687                     "     0x%08X "
00688                     );
00689             os « "#
    ------------------------------------------------------------------------------------------------------------"
00690                « endl;
00691             os « format(fmt21) % "SN" % "dive" % "midx" //
00692                     % "BITTime" //
00693                     % "CTDStat" //
00694                « endl;
00695             os « format(fmt22) % "" % "" % "" //
00696                     % "(sec)" //
00697                     % "" //
00698                « endl;
00699             os « format(fmt23) % (is_bit_passed() ? '5' : '6') % whoi % dive
00700                       % 1 // midx
00701                       % bit_duration() //
00702                       % ctd_status() //
00703                « endl;
00704
00705             // Minor index 2.
00706             static const format fmt24("#   %5s %4s %4s " //
00707                     "%15s %15s "
00708                     "%15s %15s "
00709                     "%15s"
00710                     );
00711             static const format fmt25("#   %5s %4s %4s "
00712                     "%15s %15s "
00713                     "%15s %15s "
00714                     "%15s"
00715                     );
00716             static const format fmt26("E0%c %5d %4d %4d "
00717                     "%15.3f %15d "
00718                     "%15.3f %15d "
00719                     "%15.3d"
00720                     );
00721             os « "#
    ------------------------------------------------------------------------------------------------------------"
00722                « endl;
00723             os « format(fmt24) % "SN" % "dive" % "midx" //
00724                     % "presPostReset" % "presPostReset" //
00725                     % "presPreReset" % "presPreReset" //
00726                     % "presOffset" //
00727                « endl;
00728             os « format(fmt25) % "" % "" % "" //
00729                     % "(dBar)" % "(counts)" //
00730                     % "(dBar)" % "(counts)" //
00731                     % "(dBar)"
00732                « endl;
00733             os « format(fmt26) % (is_bit_passed() ? '5' : '6') % whoi % dive
00734                       % 2 // midx
00735                       % GetDecodedBitPressureAfterOffset(pg, po) % pres_counts_after_offset_reset() //
00736                       % GetDecodedBitPressureBeforeOffset(pg, po) % pres_counts_before_offset_reset() //
00737                       % GetDecodedBitPressureOffset() //
00738                « endl;
00739
00740             // Minor index 3.
00741             static const format fmt3("#   %5s %4s %4s " //
00742                     "%15s %15s "
00743                     "%15s %15s "
00744                     );
00745             static const format fmt4("#   %5s %4s %4s "
00746                     "%16s %15s "
00747                     "%15s %15s "
00748                     );
00749             static const format fmt5("E0%c %5d %4d %4d "
00750                     "%15.3f %15d "
00751                     "%15.3f %15d "
00752                     );
00753             os « "#
    ------------------------------------------------------------------------------------------------------------"
00754                « endl;
00755             os « format(fmt3) % "SN" % "dive" % "midx" //
00756                     % "temp"  % "temp"  //
00757                     % "saln" % "saln" //
00758                « endl;
00759             os « format(fmt4) % "" % "" % ""  //
00760                     % "(degC)" % "(counts)"  //
00761                     % "(PSU)" % "(counts)" //
00762                « endl;
00763             os « format(fmt5) % (is_bit_passed() ? '5' : '6') % whoi % dive
00764                       % 3 // midx
00765                       % GetDecodedBitTemperature(tg, to) % bit_temperature() //
00766                       % GetDecodedBitSalinity(sg, so) % bit_salinity() //
00767                « endl;
00768
00769             // Minor index 4.
00770             static const format fmt6("#   %5s %4s %4s "
00771                     "%15s %15s "
00772                     "%15s %15s");
00773             static const format fmt7("#   %5s %4s %4s "
00774                     "%15s %15s "
00775                     "%15s %15s");
00776             static const format fmt8("E0%c %5d %4d %4d "
00777                     "%15.2f %15.2f "
00778                     "%15.2f %15.2f");
00779             os « "#
```

```
    ------------------------------------------------------------------------------------------------"
00780                << endl;
00781            os << format(fmt6) % "SN" % "dive" % "midx" //
00782                    % "VacStart" % "VacEnd" //
00783                       % "BatStart" % "BatEnd" //
00784                << endl;
00785            os << format(fmt7) % "" % "" % ""
00786                    % "(inHg)" % "(inHg)" //pres
00787                    % "(V)" % "(V)"  //
00788                    << endl;
00789            os << format(fmt8) % (is_bit_passed() ? '5' : '6') % whoi % dive
00790                       % 4 // midx
00791                       % GetDecodedVacAtStartOfBit() % GetDecodedVacAtEndOfBit() //
00792                       % GetDecodedBatteryVoltageAtStartOfBit()
00793                       % GetDecodedBatteryVoltageAtEndOfBit() //
00794                << endl;
00795
00796            // Minor index 5.
00797            static const format fmt9("#   %5s %4s %4s %13s %13s %11s %11s %14s %13s");
00798            static const format fmt10("#   %5s %4s %4s %13s %13s %11s %11s %14s %13s");
00799            static const format fmt11("E0%c %5d %4d %4d %13.3f %13.3f %11.3f %11.3f %14.3f %13.3f");
00800            os << "# --- Air Valve (AV) Tests
    ------------------------------------------------------------------------"
00801                << endl;
00802            os << format(fmt9) % "SN" % "dive" % "midx" % "AVOpenPos" % "AVOpenPosCal" % "AVMidPos"
00803                       % "AVMidPosCal" % "AVClosePos" % "AVClosePosCal"
00804                << endl;
00805            os << format(fmt10) % "" % "" % "" % "(V)" % "(V)" % "(V)" % "(V)" % "(V)" % "(V)"
00806                << endl;
00807            os << format(fmt11) % (is_bit_passed() ? '5' : '6') % whoi % dive
00808                       % 5 // midx
00809                       % ((float) air_valve_open_position_ / 1000)
00810                       % ((float) air_valve_open_cal_position_ / 1000)
00811                       % ((float) air_valve_middle_position_ / 1000)
00812                       % ((float) air_valve_middle_cal_position_ / 1000)
00813                       % ((float) air_valve_closed_position_ / 1000)
00814                       % ((float) air_valve_closed_cal_position_ / 1000)
00815                << endl;
00816
00817            // Minor index 6.
00818            static const format fmt12("#   %5s %4s %4s %13s %13s %11s %11s %14s %13s");
00819            static const format fmt13("#   %5s %4s %4s %13s %13s %11s %11s %14s %13s");
00820            static const format fmt14("E0%c %5d %4d %4d %13d %13d %11.3f %11.3f %14.3f %13.3f");
00821            os << "# --- Air Pump (AP) Tests
    ------------------------------------------------------------------------"
00822                << endl;
00823            os << format(fmt12) % "SN" % "dive" % "midx" % "APTestTime" % "APCycleCount"
00824                       % "APMxAPres" % "APMxLoadCur" % "APMxLoadCur12V" % "APMxLoadBatV"
00825                << endl;
00826            os << format(fmt13) % "" % "" % "" % "(sec)" % "" % "(PSI)" % "(A)" % "(A)" % "(V)"
00827                << endl;
00828            os << format(fmt14) % (is_bit_passed() ? '5' : '6') % whoi % dive
00829                       % 6 // midx
00830                       % air_pump_test_duration() % (int) air_pump_cycle_count()
00831                       % ((float) air_pressure_max() / 1000)
00832                       % ((float) air_pump_max_load_current() / 1000)
00833                       % ((float) air_pump_max_load_current_12v() / 1000)
00834                       % ((float) air_pump_max_load_battery_voltage() / 1000)
00835                << endl;
00836
00837            // Minor index 7.
00838            static const format fmt15("#   %5s %4s %4s %15s %15s %15s %15s");
00839            static const format fmt16("#   %5s %4s %4s %15s %15s %15s %15s");
00840            static const format fmt17("E0%c %5d %4d %4d %15.3f %15.3f %15.3f %15.3f");
00841            os << "# --- Oil Valve (OV) Tests
    ------------------------------------------------------------------------"
00842                << endl;
00843            os << format(fmt15) % "SN" % "dive" % "midx" % "OVOpenPos" % "OVOpenPosCal"
00844                       % "OVClosePos" % "OVClosePosCal"
00845                << endl;
00846            os << format(fmt16) % "" % "" % "" % "(V)" % "(V)" % "(V)" % "(V)" << endl;
00847            os << format(fmt17) % (is_bit_passed() ? '5' : '6') % whoi % dive
00848                       % 7 // midx
00849                       % ((float) oil_valve_open_position() / 1000)
00850                       % ((float) oil_valve_open_cal_position() / 1000)
00851                       % ((float) oil_valve_closed_position() / 1000)
00852                       % ((float) oil_valve_closed_cal_position() / 1000)
00853                << endl;
00854
00855            // Minor index 8.
00856            static const format fmt18("#   %5s %4s %4s %15s %15s %15s %15s");
00857            static const format fmt19("#   %5s %4s %4s %15s %15s %15s %15s");
00858            static const format fmt20("E0%c %5d %4d %4d %15d %15.3f %15.3f %15.3f");
00859            os << "# --- Oil Pump (OP) Tests
    ------------------------------------------------------------------------"
00860                << endl;
00861            os << format(fmt18) % "SN" % "dive" % "midx" % "OPTestTime" % "OPMxLoadCur"
00862                       % "OPMxLoadCur12V" % "OPMxLoadBatV"
00863                << endl;
00864            os << format(fmt19) % "" % "" % "" % "(sec)" % "(A)" % "(A)" % "(V)" << endl;
00865            os << format(fmt20) % (is_bit_passed() ? '5' : '6') % whoi % dive
00866                    % 8 // midx
00867                    % oil_pump_test_duration() % ((float) oil_pump_max_load_current() / 1000)
00868                    % ((float) oil_pump_max_load_current_12v() / 1000)
00869                    % ((float) oil_pump_max_load_battery_voltage() / 1000);
00870
00871        }
00872
00873            break;
00874
00875        default: { // Unsupported Engineering (BIT) version.
00876            ostringstream os;
00877            os << "Unsupported ALAMO Engineering (BIT) message version: 0x"
00878               << hex
00879               << (int) bit_version()
00880               << "!";
00881            EXC_THROW(os.str());
00882        }
```

```
00883             break;
00884         }
00885
00886         return os.str();
00887     }
00888
00889     /** Get the Built-in-Test (BIT) status code.
00890      *
00891      *  @return         The Built-in-Test (BIT) status code.
00892      */
00893     uint32_t bit_status_code() const {
00894         return bit_status_code_;
00895     }
00896
00897     /** Get the Built-in-Test (BIT) version.
00898      *
00899      *  @return         The Built-in-Test (BIT) version.
00900      */
00901     uint8_t bit_version() const {
00902         return bit_version_;
00903     }
00904
00905     /** Get the instrument software version.
00906      *
00907      *  @return         The instrument software version.
00908      */
00909     uint32_t float_software_version() const {
00910         return float_software_version_;
00911     }
00912
00913     uint16_t air_pressure_max() const {
00914         return air_pressure_max_;
00915     }
00916
00917     uint8_t air_pump_cycle_count() const {
00918         return air_pump_cycle_count_;
00919     }
00920
00921     uint16_t air_pump_max_load_battery_voltage() const {
00922         return air_pump_max_load_battery_voltage_;
00923     }
00924
00925     uint16_t air_pump_max_load_current() const {
00926         return air_pump_max_load_current_;
00927     }
00928
00929     uint16_t air_pump_max_load_current_12v() const {
00930         return air_pump_max_load_current_12v_;
00931     }
00932
00933     uint16_t air_pump_test_duration() const {
00934         return air_pump_test_duration_;
00935     }
00936
00937     uint16_t air_valve_closed_cal_position() const {
00938         return air_valve_closed_cal_position_;
00939     }
00940
00941     uint16_t air_valve_closed_position() const {
00942         return air_valve_closed_position_;
00943     }
00944
00945     uint16_t air_valve_middle_cal_position() const {
00946         return air_valve_middle_cal_position_;
00947     }
00948
00949     uint16_t air_valve_middle_position() const {
00950         return air_valve_middle_position_;
00951     }
00952
00953     uint16_t air_valve_open_cal_position() const {
00954         return air_valve_open_cal_position_;
00955     }
00956
00957     uint16_t air_valve_open_position() const {
00958         return air_valve_open_position_;
00959     }
00960
00961     uint16_t battery_voltage_at_end_of_bit() const {
00962         return battery_voltage_at_end_of_bit_;
00963     }
00964
00965     uint16_t battery_voltage_at_start_of_bit() const {
00966         return battery_voltage_at_start_of_bit_;
00967     }
00968
00969     uint16_t bit_duration() const {
00970         return bit_duration_;
00971     }
00972
00973     uint16_t bit_pressure() const {
00974         return bit_pressure_;
00975     }
00976
00977     uint16_t bit_salinity() const {
00978         return bit_salinity_;
00979     }
00980
00981     uint16_t bit_temperature() const {
00982         return bit_temperature_;
00983     }
00984
00985     uint32_t ctd_status() const {
00986         return ctd_status_;
00987     }
00988
00989     template<typename T = double> T GetDecodedBitPressure(int16_t gain, int16_t offs) const {
00990         T val;
```

```
00991
00992         val = gain ? (T) bit_pressure_ / (T) gain - (T) offs : XMSG_SENS_BAD_PRES;
00993
00994         if(val < XMSG_SENS_MIN_PRES || val > XMSG_SENS_MAX_PRES)
00995             return XMSG_SENS_BAD_PRES;
00996
00997         return val;
00998     }
00999
01000     template<typename T = double> T GetDecodedBitPressureAfterOffset(int16_t gain, int16_t offs) const {
01001         T val;
01002
01003         val = gain ? (T) pres_counts_after_offset_reset() / (T) gain - (T) offs : XMSG_SENS_BAD_PRES;
01004
01005         if(val < XMSG_SENS_MIN_PRES || val > XMSG_SENS_MAX_PRES)
01006             return XMSG_SENS_BAD_PRES;
01007
01008         return val;
01009     }
01010
01011     template<typename T = double> T GetDecodedBitPressureBeforeOffset(int16_t gain, int16_t offs) const {
01012         T val;
01013
01014         val = gain ? (T) pres_counts_before_offset_reset() / (T) gain - (T) offs : XMSG_SENS_BAD_PRES;
01015
01016         if(val < XMSG_SENS_MIN_PRES || val > XMSG_SENS_MAX_PRES)
01017             return XMSG_SENS_BAD_PRES;
01018
01019         return val;
01020     }
01021
01022     template<typename T = double> T GetDecodedBitPressureOffset() const {
01023         T val;
01024
01025         val =  (T) pres_offset() / (T) 1000;
01026
01027         if(val < XMSG_SENS_MIN_PRES || val > XMSG_SENS_MAX_PRES)
01028             return XMSG_SENS_BAD_PRES;
01029
01030         return val;
01031     }
01032
01033     template<typename T = double> T GetDecodedBitSalinity(int16_t gain, int16_t offs) const {
01034         T val;
01035
01036         val = gain ? (T) bit_salinity_ / (T) gain - (T) offs : XMSG_SENS_BAD_SALN;
01037
01038         if(val < XMSG_SENS_MIN_SALN || val > XMSG_SENS_MAX_SALN)
01039             return XMSG_SENS_BAD_SALN;
01040
01041         return val;
01042     }
01043
01044     template<typename T = double> T GetDecodedBitTemperature(int16_t gain, int16_t offs) const {
01045         T val;
01046
01047         val = gain ? (T) bit_temperature_ / (T) gain - (T) offs : XMSG_SENS_BAD_TEMP;
01048
01049         if(val < XMSG_SENS_MIN_TEMP || val > XMSG_SENS_MAX_TEMP)
01050             return XMSG_SENS_BAD_TEMP;
01051
01052         return val;
01053     }
01054
01055     template<typename T = double> T GetDecodedVacAtEndOfBit() const {
01056         return (T) vac_at_end_of_bit() / (T) 100;
01057     }
01058     template<typename T = double> T GetDecodedVacAtStartOfBit() const {
01059         return (T) vac_at_start_of_bit() / (T) 100;
01060     }
01061
01062     template<typename T = double> T GetDecodedBatteryVoltageAtEndOfBit() const {
01063         return (T) battery_voltage_at_end_of_bit() / (T) 100;
01064     }
01065
01066     template<typename T = double> T GetDecodedBatteryVoltageAtStartOfBit() const {
01067         return (T) battery_voltage_at_start_of_bit() / (T) 100;
01068     }
01069
01070     bool is_bit_passed() const {
01071         return bit_passed_;
01072     }
01073
01074     uint16_t oil_pump_max_load_current() const {
01075         return oil_pump_max_load_current_;
01076     }
01077
01078     uint16_t oil_pump_max_load_current_12v() const {
01079         return oil_pump_max_load_current_12v_;
01080     }
01081
01082     uint16_t oil_pump_max_load_battery_voltage() const {
01083         return oil_pump_max_load_battery_voltage_;
01084     }
01085
01086     uint16_t oil_pump_test_duration() const {
01087         return oil_pump_test_duration_;
01088     }
01089
01090     uint16_t oil_valve_closed_cal_position() const {
01091         return oil_valve_closed_cal_position_;
01092     }
01093
01094     uint16_t oil_valve_closed_position() const {
01095         return oil_valve_closed_position_;
01096     }
01097
01098     uint16_t oil_valve_open_cal_position() const {
```

```
01099         return oil_valve_open_cal_position_;
01100     }
01101
01102     uint16_t oil_valve_open_position() const {
01103         return oil_valve_open_position_;
01104     }
01105
01106     uint16_t vac_at_end_of_bit() const {
01107         return vac_at_end_of_bit_;
01108     }
01109
01110     uint16_t vac_at_start_of_bit() const {
01111         return vac_at_start_of_bit_;
01112     }
01113
01114     uint16_t pres_counts_after_offset_reset() const {
01115         return pres_counts_after_offset_reset_;
01116     }
01117
01118     uint16_t pres_counts_before_offset_reset() const {
01119         return pres_counts_before_offset_reset_;
01120     }
01121
01122     uint16_t pres_offset() const {
01123         return pres_offset_;
01124     }
01125
01126 };
01127
01128 } /* namespace xmsg */
01129 } /* namespace util */
01130 #endif /* XALAMOBIT_H_ */
```

11.69 XAlamoBit2.h

```
00001 /*
00002  * XAlamoBit2.h
00003  *
00004  *  Created on: Apr 20, 2023
00005  *      Author: aekholm
00006  */
00007
00008 #ifndef DATA_XMSG_XALAMOBIT2_H_
00009 #define DATA_XMSG_XALAMOBIT2_H_
00010
00011 /* Includes */
00012
00013 #include "XMessage.h"
00014
00015 namespace util {
00016 namespace xmsg {
00017
00018 /* Type Definitions */
00019
00020
00021 typedef enum XAlamoBIT2Type {
00022     UNKNOWN = -1,
00023     NO_SKY_VIEW = 0,
00024     SKY_VIEW= 1,
00025 } XAlamoBIT2Type_t;
00026
00027 /* Class Definitions */
00028
00029 /** The ALAMO Built-in-Test (BIT) message, version 2.
00030  *
00031  *  An instance of this class represents a decoded BIT message from ALAMO instruments with
00032  *  firmware version 10.x or later (BIT formats 9, 10 and 11; see @ref XEngBIT). Earlier
00033  *  instruments use @ref XAlamoBit.
00034  */
00035 class XAlamoBit2:
00036     public util::date::DateUtils {
00037
00038   private:
00039
00040     /* BIT Header Information */
00041
00042     /** The software version identifier.
00043      *
00044      */
00045     uint32_t float_software_version_;
00046
00047
00048     /** The Built-in-Test (BIT) version identifier.
00049      *
00050      */
00051     uint8_t bit_version_;
00052
00053
00054     /* BIT Time Fields */
00055
00056     /** BIT Start time.
00057      *
00058      */
00059     ptime dt_start_;
00060
00061     /** BIT start time, in seconds since 1970-01-01T00:00:00Z.
00062      *
00063      */
00064     uint32_t dt_start_epoch_;
00065
00066     /** BIT End time.
00067      *
00068      */
00069     ptime dt_end_;
```

```
00070
00071     /** BIT End time, in seconds since 1970-01-01T00:00:00Z.
00072      *
00073      */
00074     uint32_t dt_end_epoch_;
00075
00076     /** BIT duration in seconds.
00077      *
00078      */
00079     int32_t duration_sec_;
00080
00081     /* BIT Results */
00082
00083     /** The BIT passed flag.
00084      *
00085      */
00086     bool bit_passed_;
00087
00088     /** The Built-in-Test (BIT) status code.
00089      *
00090      */
00091     uint32_t bit_status_code_;
00092
00093     /* BIT Results - Enumerations and Bitmasks */
00094
00095     /** BIT test type enum.
00096      *
00097      */
00098     uint8_t type_;
00099
00100     /** Sensor type enum.
00101      *
00102      */
00103     uint8_t sensor_type_;
00104
00105     /** Air pump errors bitmask.
00106      *
00107      */
00108     uint32_t air_pump_errors_;
00109
00110     /** Oil pump errors bitmask.
00111      *
00112      */
00113     uint32_t oil_pump_errors_;
00114
00115     /** Air valve errors bitmask.
00116      *
00117      */
00118     uint32_t air_valve_errors_;
00119
00120
00121     /** Oil valve errors bitmask.
00122      *
00123      */
00124     uint32_t oil_valve_errors_;
00125
00126     /** GPS errors bitmask.
00127      *
00128      */
00129     uint32_t gps_errors_;
00130
00131     uint32_t ctd_errors_;
00132
00133     uint32_t iridum_errors_;
00134
00135     uint32_t case_pres_errors_;
00136
00137     uint32_t bat_errors_;
00138
00139     uint32_t traveler_errors_;
00140
00141
00142     /* BIT Results - CTD Fields */
00143
00144     float pres_before_cal_;
00145
00146     float pres_offset_;
00147
00148     float pres_after_cal_;
00149
00150     float temp_;
00151
00152     float saln_;
00153
00154     uint32_t ctd_status_;
00155
00156
00157     /* BIT Results - Case Vac */
00158
00159     /** Case vac at start and end of BIT, in inHg.
00160      *
00161      */
00162     float case_vac_start_, case_vac_end_;
00163
00164     /* BIT Results - Battery */
00165
00166     /** Battery voltages at start and end of BIT, in volts.
00167      *
00168      */
00169     float bat_volt_start_, bat_volt_end_;
00170
00171     /* BIT Results - Air valve */
00172
00173     /** Air valve open, closed, and middle calibration positions, in volts.
00174      *
00175      */
00176     float air_valve_cal_open_, air_valve_cal_closed_, air_valve_cal_mid_;
00177
```

```
00178     /** Air valve open, closed, and middle positions, in volts.
00179      *
00180      */
00181     float air_valve_open_, air_valve_closed_, air_valve_mid_;
00182
00183     /* BIT Results - Air pump */
00184
00185     /** Air pump test duration, in seconds.
00186      *
00187      */
00188     uint16_t air_pump_test_duration_;
00189
00190     /** Air pump cycle count.
00191      *
00192      */
00193     uint8_t air_pump_cycle_count_;
00194
00195     /** Max. voltage during air pump test, in volts.
00196      *
00197      */
00198     float air_pump_volt_max_;
00199
00200     /** Max. 12V rail current during air pump test, in amps.
00201      *
00202      */
00203     float air_pump_v12_curr_max_;
00204
00205     /** Max. and mean current during air pump test, in amps.
00206      *
00207      */
00208     float air_pump_curr_max_, air_pump_curr_avg_;
00209
00210     float air_pump_pres_max_;
00211
00212
00213     /* BIT Results - Oil valve */
00214
00215     /** Oil valve open and clased calibration positions, in volts.
00216      *
00217      */
00218     float oil_valve_cal_open_, oil_valve_cal_closed_;
00219
00220     /** Oil valve open and closed positions, in volts.
00221      *
00222      */
00223     float oil_valve_open_, oil_valve_closed_;
00224
00225     /* BIT Results - Oil pump */
00226
00227     /** Oil pump test duration, in seconds.
00228      *
00229      */
00230     uint16_t oil_pump_test_duration_;
00231
00232     /** Max. and mean oil pump current during test, in amps.
00233      *
00234      */
00235     float oil_pump_curr_max_, oil_pump_curr_avg_;
00236
00237     /** Max 12V rail current during oil pump test, in amps.
00238      *
00239      */
00240     float oil_pump_v12_curr_max_;
00241
00242     /** Max battery voltage during oil pump test, in volts.
00243      *
00244      */
00245     float oil_pump_volt_max_;
00246
00247   protected:
00248
00249     /* Mutator Methods */
00250
00251     /** Set the BIT passed flag.
00252      *
00253      *  @param[in]  bit_passed          BIT passed flag to set.
00254      */
00255     void set_bit_passed(bool bit_passed) {
00256       bit_passed_ = bit_passed;
00257     }
00258
00259     /** Set the Built-in-Test (BIT) status code.
00260      *
00261      *  @param[in]  bit_status_code     The Built-in-Test (BIT) status code to set.
00262      */
00263     void set_bit_status_code(uint32_t bit_status_code) {
00264       bit_status_code_ = bit_status_code;
00265     }
00266
00267     /** Set the Built-in-Test (BIT) version.
00268      *
00269      *  @param[in]  bit_version         The Built-in-Test (BIT) version to set.
00270      */
00271     void set_bit_version(uint8_t bit_version) {
00272       bit_version_ = bit_version;
00273     }
00274
00275     /** Set the instrument software version.
00276      *
00277      *  @param[in]  float_software_version  The instrument software version to set.
00278      */
00279     void set_float_software_version(uint32_t float_software_version) {
00280       float_software_version_ = float_software_version;
00281     }
00282
00283     void set_duration_sec(int32_t duration_sec) {
00284       duration_sec_ = duration_sec;
00285     }
```

```
00286
00287     void set_dt_start(const ptime &dt_start) {
00288       dt_start_ = dt_start;
00289     }
00290
00291     void set_dt_end(const ptime &dt_end) {
00292       dt_end_ = dt_end;
00293     }
00294
00295     void set_air_pump_errors(uint32_t air_pump_errors) {
00296       air_pump_errors_ = air_pump_errors;
00297     }
00298
00299     void set_air_valve_errors(uint32_t air_valve_errors) {
00300       air_valve_errors_ = air_valve_errors;
00301     }
00302
00303     void set_bat_errors(uint32_t bat_errors) {
00304       bat_errors_ = bat_errors;
00305     }
00306
00307     void set_case_pres_errors(uint32_t case_pres_errors) {
00308       case_pres_errors_ = case_pres_errors;
00309     }
00310
00311     void set_ctd_errors(uint32_t ctd_errors) {
00312       ctd_errors_ = ctd_errors;
00313     }
00314
00315     void set_sensor_type(uint8_t sensor_type) {
00316       sensor_type_ = sensor_type;
00317     }
00318
00319     void set_gps_errors(uint32_t gps_errors) {
00320       gps_errors_ = gps_errors;
00321     }
00322
00323     void set_iridum_errors(uint32_t iridum_errors) {
00324       iridum_errors_ = iridum_errors;
00325     }
00326
00327     void set_oil_pump_errors(uint32_t oil_pump_errors) {
00328       oil_pump_errors_ = oil_pump_errors;
00329     }
00330
00331     void set_oil_valve_errors(uint32_t oil_valve_errors) {
00332       oil_valve_errors_ = oil_valve_errors;
00333     }
00334
00335     void set_type(uint8_t type) {
00336       type_ = type;
00337     }
00338
00339     void set_dt_end_epoch(uint32_t dt_end_epoch) {
00340       dt_end_epoch_ = dt_end_epoch;
00341     }
00342
00343     void set_dt_start_epoch(uint32_t dt_start_epoch) {
00344       dt_start_epoch_ = dt_start_epoch;
00345     }
00346
00347     void set_ctd_status(uint32_t ctd_status) {
00348       ctd_status_ = ctd_status;
00349     }
00350
00351     void set_pres_after_cal(float pres_after_cal) {
00352       pres_after_cal_ = pres_after_cal;
00353     }
00354
00355     void set_pres_before_cal(float pres_before_cal) {
00356       pres_before_cal_ = pres_before_cal;
00357     }
00358
00359     void set_pres_offset(float pres_offset) {
00360       pres_offset_ = pres_offset;
00361     }
00362
00363     void set_saln(float saln) {
00364       saln_ = saln;
00365     }
00366
00367     void set_temp(float temp) {
00368       temp_ = temp;
00369     }
00370
00371     void set_bat_volt_end(float bat_volt_end) {
00372       bat_volt_end_ = bat_volt_end;
00373     }
00374
00375     void set_bat_volt_start(float bat_volt_start) {
00376       bat_volt_start_ = bat_volt_start;
00377     }
00378
00379     void set_case_vac_end(float case_vac_end) {
00380       case_vac_end_ = case_vac_end;
00381     }
00382
00383     void set_case_vac_start(float case_vac_start) {
00384       case_vac_start_ = case_vac_start;
00385     }
00386
00387     void set_air_pump_cycle_count(uint8_t air_pump_cycle_count) {
00388       air_pump_cycle_count_ = air_pump_cycle_count;
00389     }
00390
00391     void set_air_pump_v12_curr_max(float air_pump_v12_curr_max) {
00392       air_pump_v12_curr_max_ = air_pump_v12_curr_max;
00393     }
```

```
00394
00395     void set_air_pump_volt_max(float air_pump_volt_max) {
00396       air_pump_volt_max_ = air_pump_volt_max;
00397     }
00398
00399     void set_air_pump_test_duration(uint16_t air_pump_test_duration) {
00400       air_pump_test_duration_ = air_pump_test_duration;
00401     }
00402
00403     void set_air_valve_cal_closed(float air_valve_cal_closed) {
00404       air_valve_cal_closed_ = air_valve_cal_closed;
00405     }
00406
00407     void set_air_valve_cal_mid(float air_valve_cal_mid) {
00408       air_valve_cal_mid_ = air_valve_cal_mid;
00409     }
00410
00411     void set_air_valve_cal_open(float air_valve_cal_open) {
00412       air_valve_cal_open_ = air_valve_cal_open;
00413     }
00414
00415     void set_air_valve_closed(float air_valve_closed) {
00416       air_valve_closed_ = air_valve_closed;
00417     }
00418
00419     void set_air_valve_mid(float air_valve_mid) {
00420       air_valve_mid_ = air_valve_mid;
00421     }
00422
00423     void set_air_valve_open(float air_valve_open) {
00424       air_valve_open_ = air_valve_open;
00425     }
00426
00427     void set_oil_pump_curr_avg(float oil_pump_curr_avg) {
00428       oil_pump_curr_avg_ = oil_pump_curr_avg;
00429     }
00430
00431     void set_oil_pump_curr_max(float oil_pump_curr_max) {
00432       oil_pump_curr_max_ = oil_pump_curr_max;
00433     }
00434
00435     void set_oil_pump_volt_max(float oil_pump_volt_max) {
00436       oil_pump_volt_max_ = oil_pump_volt_max;
00437     }
00438
00439     void set_oil_pump_test_duration(uint16_t oil_pump_test_duration) {
00440       oil_pump_test_duration_ = oil_pump_test_duration;
00441     }
00442
00443     void set_oil_pump_v12_curr_max(float oil_pump_v12_curr_max) {
00444       oil_pump_v12_curr_max_ = oil_pump_v12_curr_max;
00445     }
00446
00447     void set_oil_valve_cal_closed(float oil_valve_cal_closed) {
00448       oil_valve_cal_closed_ = oil_valve_cal_closed;
00449     }
00450
00451     void set_oil_valve_cal_open(float oil_valve_cal_open) {
00452       oil_valve_cal_open_ = oil_valve_cal_open;
00453     }
00454
00455     void set_oil_valve_closed(float oil_valve_closed) {
00456       oil_valve_closed_ = oil_valve_closed;
00457     }
00458
00459     void set_oil_valve_open(float oil_valve_open) {
00460       oil_valve_open_ = oil_valve_open;
00461     }
00462
00463
00464     void set_air_pump_pres_max(float air_pump_pres_max) {
00465       air_pump_pres_max_ = air_pump_pres_max;
00466     }
00467
00468     void set_air_pump_curr_max(float air_pump_curr_max) {
00469       air_pump_curr_max_ = air_pump_curr_max;
00470     }
00471
00472     void set_air_pump_curr_avg(float air_pump_curr_avg) {
00473       air_pump_curr_avg_ = air_pump_curr_avg;
00474     }
00475
00476     void set_traveler_errors(uint32_t travelerErrors) {
00477       traveler_errors_ = travelerErrors;
00478     }
00479
00480   public:
00481
00482     /* Accessor Methods */
00483
00484     /** Get the Built-in-Test (BIT) version.
00485      *
00486      *  @return         The Built-in-Test (BIT) version.
00487      */
00488     uint8_t bit_version() const {
00489         return bit_version_;
00490     }
00491
00492     /** Get the instrument software version.
00493      *
00494      *  @return         The instrument software version.
00495      */
00496     uint32_t float_software_version() const {
00497         return float_software_version_;
00498     }
00499
00500     bool bit_passed() const {
00501       return bit_passed_;
```

```
00502     }
00503
00504     uint32_t bit_status_code() const {
00505       return bit_status_code_;
00506     }
00507
00508     const ptime& dt_end() const {
00509       return dt_end_;
00510     }
00511
00512     const ptime& dt_start() const {
00513       return dt_start_;
00514     }
00515
00516     int32_t duration_sec() const {
00517       return duration_sec_;
00518     }
00519
00520     uint32_t air_pump_errors() const {
00521       return air_pump_errors_;
00522     }
00523
00524     uint32_t air_valve_errors() const {
00525       return air_valve_errors_;
00526     }
00527
00528     uint32_t bat_errors() const {
00529       return bat_errors_;
00530     }
00531
00532     uint32_t case_pres_errors() const {
00533       return case_pres_errors_;
00534     }
00535
00536     uint32_t ctd_errors() const {
00537       return ctd_errors_;
00538     }
00539
00540     uint8_t sensor_type() const {
00541       return sensor_type_;
00542     }
00543
00544     uint32_t gps_errors() const {
00545       return gps_errors_;
00546     }
00547
00548     uint32_t iridum_errors() const {
00549       return iridum_errors_;
00550     }
00551
00552     uint32_t oil_pump_errors() const {
00553       return oil_pump_errors_;
00554     }
00555
00556     uint32_t oil_valve_errors() const {
00557       return oil_valve_errors_;
00558     }
00559
00560     uint8_t type() const {
00561       return type_;
00562     }
00563
00564     uint32_t dt_end_epoch() const {
00565       return dt_end_epoch_;
00566     }
00567
00568     uint32_t dt_start_epoch() const {
00569       return dt_start_epoch_;
00570     }
00571
00572     uint32_t ctd_status() const {
00573       return ctd_status_;
00574     }
00575
00576     float pres_after_cal() const {
00577       return pres_after_cal_;
00578     }
00579
00580     float pres_before_cal() const {
00581       return pres_before_cal_;
00582     }
00583
00584     float pres_offset() const {
00585       return pres_offset_;
00586     }
00587
00588     float saln() const {
00589       return saln_;
00590     }
00591
00592     float temp() const {
00593       return temp_;
00594     }
00595
00596     float bat_volt_end() const {
00597       return bat_volt_end_;
00598     }
00599
00600     float bat_volt_start() const {
00601       return bat_volt_start_;
00602     }
00603
00604     float case_vac_end() const {
00605       return case_vac_end_;
00606     }
00607
00608     float case_vac_start() const {
00609       return case_vac_start_;
```

```
00610     }
00611
00612     uint8_t air_pump_cycle_count() const {
00613       return air_pump_cycle_count_;
00614     }
00615
00616     float air_pump_v12_curr_max() const {
00617       return air_pump_v12_curr_max_;
00618     }
00619
00620     float air_pump_volt_max() const {
00621       return air_pump_volt_max_;
00622     }
00623
00624     uint16_t air_pump_test_duration() const {
00625       return air_pump_test_duration_;
00626     }
00627
00628     float air_valve_cal_closed() const {
00629       return air_valve_cal_closed_;
00630     }
00631
00632     float air_valve_cal_mid() const {
00633       return air_valve_cal_mid_;
00634     }
00635
00636     float air_valve_cal_open() const {
00637       return air_valve_cal_open_;
00638     }
00639
00640     float air_valve_closed() const {
00641       return air_valve_closed_;
00642     }
00643
00644     float air_valve_mid() const {
00645       return air_valve_mid_;
00646     }
00647
00648     float air_valve_open() const {
00649       return air_valve_open_;
00650     }
00651
00652     float oil_pump_curr_avg() const {
00653       return oil_pump_curr_avg_;
00654     }
00655
00656     float oil_pump_curr_max() const {
00657       return oil_pump_curr_max_;
00658     }
00659
00660     float oil_pump_volt_max() const {
00661       return oil_pump_volt_max_;
00662     }
00663
00664     uint16_t oil_pump_test_duration() const {
00665       return oil_pump_test_duration_;
00666     }
00667
00668     float oil_pump_v12_curr_max() const {
00669       return oil_pump_v12_curr_max_;
00670     }
00671
00672     float oil_valve_cal_closed() const {
00673       return oil_valve_cal_closed_;
00674     }
00675
00676     float oil_valve_cal_open() const {
00677       return oil_valve_cal_open_;
00678     }
00679
00680     float oil_valve_closed() const {
00681       return oil_valve_closed_;
00682     }
00683
00684     float oil_valve_open() const {
00685       return oil_valve_open_;
00686     }
00687
00688     float air_pump_pres_max() const {
00689       return air_pump_pres_max_;
00690     }
00691
00692
00693     float air_pump_curr_max() const {
00694       return air_pump_curr_max_;
00695     }
00696
00697     float air_pump_curr_avg() const {
00698       return air_pump_curr_avg_;
00699     }
00700
00701     uint32_t traveler_errors() const {
00702       return traveler_errors_;
00703     }
00704
00705   public:
00706
00707     /* Member Functions */
00708
00709     /** The parameterized constructor.
00710      *
00711      *  @param[in]      data        The binary data to decode.
00712      *
00713      *  @throw          log::MessageException if an error occurs while decoding.
00714      */
00715     XAlamoBit2(const vector<uint8_t> &data) {
00716       DecodeXMsgData(data); // Decode the message.
00717     }
```

```
00718
00719     /** The destructor. */
00720     ~XAlamoBit2() { /* Do nothing. */ }
00721
00722     /** Decode the ALAMO BIT test engineering message from the binary message data.
00723      *
00724      *  @param[in]      data        The binary data to decode.
00725      *
00726      *  @throw          log::MessageException if an error occurs while decoding.
00727      */
00728     void DecodeXMsgData(const vector<uint8_t> &data) {
00729
00730       try {
00731
00732         const uint8_t *d = data.data();
00733
00734         if(data.size() < 13) { // Check message size.
00735           ostringstream os;
00736           os « "Invalid X-Message: Expected at least 13 bytes, but only received"
00737              « data.size()
00738              « " bytes!";
00739           EXC_THROW(os.str());
00740         }
00741
00742         if(d[0] == 0xE5) { // Check if BIT passed or failed.
00743           set_bit_passed(true);
00744         } else if(d[0] == 0xE6) {
00745           set_bit_passed(false);
00746         } else {
00747           ostringstream os;
00748           os « "Invalid X-Message: Expected X-Message format identifiers 0xE5 or 0xE6, but "
00749              « "received 0x"
00750              « std::hex
00751              « d[0]
00752              « "!";
00753           EXC_THROW(os.str());
00754         }
00755
00756         set_bit_version(u08(d, 3)); // Get BIT version.
00757
00758
00759         /* Parse the BIT message: */
00760
00761         switch(bit_version_) {
00762
00763           case 9: // ALAMO BIT version 9. Starting firmware version 10.5.0
00764           case 10: // ALAMO BIT version 10. Starting firmware version 10.6.0
00765
00766             if(data.size() != 256) { // Check the message size.
00767               ostringstream os;
00768               os « "Invalid X-Message: Expected 260 bytes, but received "
00769                  « data.size()
00770                  « " bytes!";
00771 #if defined(XMSG_DEBUG_BIST) && XMSG_DEBUG_BIST
00772               lwrn() « os.str();
00773 #else
00774               EXC_THROW(os.str());
00775 #endif // defined(XMSG_DEBUG_BIST) && XMSG_DEBUG_BIST
00776             }
00777
00778             /* header */
00779
00780             set_float_software_version(   u32(d,   4));
00781             set_type(                     u08(d,   8));
00782             set_sensor_type(              u08(d,   9));
00783
00784             /* status codes */
00785
00786             set_bit_status_code(          u32(d,  10));
00787             set_air_pump_errors(          u32(d,  14));
00788             set_oil_pump_errors(          u32(d,  18));
00789             set_air_valve_errors(         u32(d,  22));
00790             set_oil_valve_errors(         u32(d,  26));
00791             set_gps_errors(               u32(d,  30));
00792             set_ctd_errors(               u32(d,  34));
00793             set_iridum_errors(            u32(d,  38));
00794             set_case_pres_errors(         u32(d,  42));
00795             set_bat_errors(               u32(d,  46));
00796
00797             /* time stamps */
00798
00799             set_dt_start_epoch(           u32(d,  50));
00800             set_dt_start(               s1970(d,  50));
00801             set_dt_end_epoch(             u32(d,  54));
00802             set_dt_end(                 s1970(d,  54));
00803             set_duration_sec(dt_end_epoch() - dt_start_epoch());
00804
00805             /* ctd */
00806
00807             set_pres_before_cal(          f32(d,  58));
00808             set_pres_offset(              f32(d,  62));
00809             set_pres_after_cal(           f32(d,  66));
00810             set_temp(                     f32(d,  70));
00811             set_saln(                     f32(d,  74));
00812             set_ctd_status(               u32(d,  78));
00813
00814             /* case vac */
00815
00816             set_case_vac_start(           f32(d,  82));
00817             set_case_vac_end(             f32(d,  86));
00818
00819             /* bat voltage */
00820
00821             set_bat_volt_start(           f32(d,  90));
00822             set_bat_volt_end(             f32(d,  94));
00823
00824             /* air valve */
00825
```

```
00826             set_air_valve_open(           f32(d,  98));
00827             set_air_valve_cal_open(       f32(d, 102));
00828             set_air_valve_closed(         f32(d, 106));
00829             set_air_valve_cal_closed(     f32(d, 110));
00830             set_air_valve_mid(            f32(d, 114));
00831             set_air_valve_cal_mid(        f32(d, 118));
00832
00833             /* air pump */
00834
00835             set_air_pump_test_duration(   u16(d, 122));
00836             set_air_pump_cycle_count(     u08(d, 124));
00837             set_air_pump_pres_max(        f32(d, 125));
00838             set_air_pump_curr_max(        f32(d, 129));
00839             set_air_pump_curr_avg(        f32(d, 133));
00840             set_air_pump_v12_curr_max(    f32(d, 137));
00841             set_air_pump_volt_max(        f32(d, 141));
00842
00843             /* oil valve */
00844
00845             set_oil_valve_open(           f32(d, 145));
00846             set_oil_valve_cal_open(       f32(d, 149));
00847             set_oil_valve_closed(         f32(d, 153));
00848             set_oil_valve_cal_closed(     f32(d, 157));
00849
00850             /* oil pump */
00851
00852             set_oil_pump_test_duration(   u16(d, 161));
00853             set_oil_pump_curr_max(        f32(d, 163));
00854             set_oil_pump_curr_avg(        f32(d, 167));
00855             set_oil_pump_v12_curr_max(    f32(d, 171));
00856             set_oil_pump_volt_max(        f32(d, 175));
00857
00858             break;
00859
00860           case 11: // ALAMO BIT version 11.
00861
00862             if(data.size() != 256 && data.size() != 264) { // Check the message size.
00863               ostringstream os;
00864               os « "Invalid X-Message: Expected 256 or 264 bytes, but received "
00865                  « data.size()
00866                  « " bytes!";
00867 #if defined(XMSG_DEBUG_BIST) && XMSG_DEBUG_BIST
00868               lwrn() « os.str();
00869 #else
00870               EXC_THROW(os.str());
00871 #endif // defined(XMSG_DEBUG_BIST) && XMSG_DEBUG_BIST
00872             }
00873
00874
00875             /* header */
00876
00877             set_float_software_version(   u32(d,   4));
00878             set_type(                     u08(d,   8));
00879             set_sensor_type(              u08(d,   9));
00880
00881
00882             /* status codes */
00883
00884             set_bit_status_code(          u32(d,  10));
00885             set_air_pump_errors(          u32(d,  14));
00886             set_oil_pump_errors(          u32(d,  18));
00887             set_air_valve_errors(         u32(d,  22));
00888             set_oil_valve_errors(         u32(d,  26));
00889             set_gps_errors(               u32(d,  30));
00890             set_ctd_errors(               u32(d,  34));
00891             set_iridum_errors(            u32(d,  38));
00892             set_case_pres_errors(         u32(d,  42));
00893             set_bat_errors(               u32(d,  46));
00894             set_traveler_errors(          u32(d,  50)); // added 10.8
00895
00896             /* time stamps */
00897
00898             set_dt_start_epoch(           u32(d,  54));
00899             set_dt_start(               s1970(d,  54));
00900             set_dt_end_epoch(             u32(d,  58));
00901             set_dt_end(                 s1970(d,  58));
00902             set_duration_sec(dt_end_epoch() - dt_start_epoch());
00903
00904             /* ctd */
00905
00906             set_pres_before_cal(          f32(d,  62));
00907             set_pres_offset(              f32(d,  66));
00908             set_pres_after_cal(           f32(d,  70));
00909             set_temp(                     f32(d,  74));
00910             set_saln(                     f32(d,  78));
00911             set_ctd_status(               u32(d,  82));
00912
00913             /* case vac */
00914
00915             set_case_vac_start(           f32(d,  86));
00916             set_case_vac_end(             f32(d,  90));
00917
00918             /* bat voltage */
00919
00920             set_bat_volt_start(           f32(d,  94));
00921             set_bat_volt_end(             f32(d,  98));
00922
00923             /* air valve */
00924
00925             set_air_valve_open(           f32(d, 102));
00926             set_air_valve_cal_open(       f32(d, 106));
00927             set_air_valve_closed(         f32(d, 110));
00928             set_air_valve_cal_closed(     f32(d, 114));
00929             set_air_valve_mid(            f32(d, 118));
00930             set_air_valve_cal_mid(        f32(d, 122));
00931
00932             /* air pump */
00933
```

```
00934             set_air_pump_test_duration(   u16(d, 126));
00935             set_air_pump_cycle_count(     u08(d, 128));
00936             set_air_pump_pres_max(        f32(d, 129));
00937             set_air_pump_curr_max(        f32(d, 133));
00938             set_air_pump_curr_avg(        f32(d, 137));
00939             set_air_pump_v12_curr_max(    f32(d, 141));
00940             set_air_pump_volt_max(        f32(d, 145));
00941
00942             /* oil valve */
00943
00944             set_oil_valve_open(           f32(d, 149));
00945             set_oil_valve_cal_open(       f32(d, 153));
00946             set_oil_valve_closed(         f32(d, 157));
00947             set_oil_valve_cal_closed(     f32(d, 161));
00948
00949             /* oil pump */
00950
00951             set_oil_pump_test_duration(   u16(d, 165));
00952             set_oil_pump_curr_max(        f32(d, 167));
00953             set_oil_pump_curr_avg(        f32(d, 171));
00954             set_oil_pump_v12_curr_max(    f32(d, 175));
00955             set_oil_pump_volt_max(        f32(d, 179));
00956
00957             break;
00958
00959           default: { // Unsupported ALAMO BIT version.
00960
00961             ostringstream os;
00962             os « "Unsupported ALAMO BIT version: " « (int) bit_version_ « "!";
00963             EXC_THROW(os.str());
00964           }
00965
00966             break;
00967
00968         } // switch(bit_version_)
00969
00970 #       if defined(XMSG_DEBUG_BIST) && XMSG_DEBUG_BIST
00971
00972         ldbg() « "BIT message listing string: "
00973                « endl « GenerateListingString(0, 0);
00974
00975 #       endif // defined(XMSG_DEBUG_BIST) && XMSG_DEBUG_BIST
00976
00977       } CATCH_BLOCK("Failed to decode ALAMO Engineering (BIT v2) data message!")
00978
00979     }
00980
00981     /** Generate a listing entry string representation of the the ALAMO BIT test engineering
00982      *  message.
00983      *
00984      *  @param[in]      whoi        The instrument serial number.
00985      *  @param[in]      dive        The dive number of this ALAMO BIT test engineering message.
00986      *
00987      *  @return         A listing entry string representation of the the ALAMO BIT test engineering
00988      *                  message.
00989      */
00990     const string GenerateListingString(
00991         int whoi,
00992         int dive) const {
00993
00994       ostringstream os;
00995       switch(bit_version()) {
00996
00997         case 9: // ALAMO BIT version 9. Starting firmware version 10.5.0
00998         case 10: // ALAMO BIT version 10. Starting firmware version 10.6.1
00999         case 11: // ALAMO BIT version 11.
01000
01001         {
01002
01003           int midx = 0;
01004           string sensor_type_str;
01005
01006           switch (sensor_type()) {
01007             case 0: sensor_type_str = "RBR"; break;
01008             case 1: sensor_type_str = "SBE"; break;
01009             default: sensor_type_str = "unknown"; break;
01010           }
01011
01012           /* BIT results  */
01013
01014           {
01015             static const format fmt1(
01016                 "#   %5s %4s %4s %6s "
01017                 "%13s %13s %13s   %14s");
01018             static const format fmt3(
01019                 "E0%c %5d %4d %4d %6d "
01020                 "   0x%08X    0x%08X %13d # %s");
01021             os « format(fmt1)
01022                 % "SN" % "dive" % "midx" % "BITVer"
01023                 % "FWVer" % "BITResults" %  "sensorType" % "sensorTypeName"« endl;
01024             os « "#" « endl;
01025             os « format(fmt3)
01026                 % (bit_passed() ? '5' : '6') % whoi % dive % midx++ % (int) bit_version()
01027                 % float_software_version() % bit_status_code() % (int)sensor_type()
01028                 % sensor_type_str
01029                « endl;
01030           }
01031
01032           /* Timestamps */
01033
01034           os « "# --- Date/Time ----------------------------------------------------------------------"
01035               "--------------------"
01036              « endl;
01037
01038           {
01039             static const format fmt1(
01040                 "#   %5s %4s %4s %6s "
01041                 "%27s %27s %13s");
```

```
01042             static const format fmt2(
01043                 "#   %5s %4s %4s %6s "
01044                 "%27s %27s %13s");
01045             static const format fmt3(
01046                 "E0%c %5d %4d %4d %6d "
01047                 "%27s %27s %13d");
01048             os « format(fmt1)
01049                 % "SN" % "dive" % "midx" % "BITVer"
01050                 % "startTime" % "endTime" % "duration"
01051                « endl;
01052             os « format(fmt2)
01053                 % "" % "" % "" % ""
01054                 % "" % "" % "(sec)"
01055                « endl;
01056             os « format(fmt3)
01057                 % (bit_passed() ? '5' : '6') % whoi % dive % midx++ % (int) bit_version()
01058                 % GenerateIso8601String(dt_start()) % GenerateIso8601String(dt_end())
01059                 % duration_sec()
01060                « endl;
01061           }
01062
01063           /* CTD */
01064
01065           os « "# --- CTD ----------------------------------------------------------------------"
01066               "-------------------"
01067              « endl;
01068
01069           {
01070             static const format fmt1(
01071                 "#   %5s %4s %4s %6s "
01072                 "%13s %13s %13s %13s %13s");
01073             static const format fmt2(
01074                 "#   %5s %4s %4s %6s "
01075                 "%13s %13s %13s %13s %13s");
01076             static const format fmt3(
01077                 "E0%c %5d %4d %4d %6d "
01078                 "   0x%08X    0x%08X %13.3f %13.3f %13.3f");
01079             os « format(fmt1)
01080                 % "SN" % "dive" % "midx" % "BITVer"
01081                 % "errors" % "status" % "presPreCal" % "presOffset" % "presPostCal"
01082                « endl;
01083             os « format(fmt2)
01084                 % "" % "" % "" % ""
01085                 % "" % "" % "(dBar)" % "(dBar)" % "(dBar)"
01086                « endl;
01087             os « format(fmt3)
01088                 % (bit_passed() ? '5' : '6') % whoi % dive % midx++ % (int) bit_version()
01089                 % ctd_errors() %ctd_status()  % pres_before_cal() % pres_offset()
01090                 % pres_after_cal()
01091                « endl;
01092           }
01093
01094           os « "# ------------------------------------------------------------------------------"
01095               "-------------------"
01096              « endl;
01097
01098           {
01099             static const format fmt1(
01100                 "#   %5s %4s %4s %6s "
01101                 "%13s %13s");
01102             static const format fmt2(
01103                 "#   %5s %4s %4s %6s "
01104                 "%13s %13s");
01105             static const format fmt3(
01106                 "E0%c %5d %4d %4d %6d "
01107                 "%13.3f %13.3f");
01108             os « format(fmt1)
01109                 % "SN" % "dive" % "midx" % "BITVer"
01110                 % "temp" % "saln"
01111                « endl;
01112             os « format(fmt2)
01113                 % "" % "" % "" % ""
01114                 % "(degC)" % "(PSU)"
01115                « endl;
01116             os « format(fmt3)
01117                 % (bit_passed() ? '5' : '6') % whoi % dive % midx++ % (int) bit_version()
01118                 % temp() % saln()
01119                 « endl;
01120           }
01121
01122           /* Case vac */
01123
01124           os « "# --- Case Vacuum -----------------------------------------------------------"
01125               "-------------------"
01126              « endl;
01127
01128           {
01129             static const format fmt1(
01130                 "#   %5s %4s %4s %6s "
01131                 "%13s %13s %13s");
01132             static const format fmt2(
01133                 "#   %5s %4s %4s %6s "
01134                 "%13s %13s %13s");
01135             static const format fmt3(
01136                 "E0%c %5d %4d %4d %6d "
01137                 "   0x%08X %13.3f %13.3f");
01138             os « format(fmt1)
01139                 % "SN" % "dive" % "midx" % "BITVer"
01140                 % "errors" % "caseVacStart" % "caseVacEnd"
01141                « endl;
01142             os « format(fmt2)
01143                 % "" % "" % "" % ""
01144                 % "" % "(inHg)" % "(inHg)"
01145                « endl;
01146             os « format(fmt3)
01147                 % (bit_passed() ? '5' : '6') % whoi % dive % midx++ % (int) bit_version()
01148                 % case_pres_errors()  % case_vac_start() % case_vac_end()
01149                « endl;
```

```
01150           }
01151
01152           /* Battery Voltage */
01153
01154           os « "# --- Battery Voltage ------------------------------------------------------------"
01155               "--------------------"
01156              « endl;
01157
01158           {
01159             static const format fmt1(
01160                 "#   %5s %4s %4s %6s "
01161                 "%13s %13s %13s");
01162             static const format fmt2(
01163                 "#   %5s %4s %4s %6s "
01164                 "%13s %13s %13s");
01165             static const format fmt3(
01166                 "E0%c %5d %4d %4d %6d "
01167                 "   0x%08X %13.3f %13.3f");
01168             os « format(fmt1)
01169                 % "SN" % "dive" % "midx" % "BITVer"
01170                 % "errors" % "batVoltStart" % "batVoltEnd"
01171                « endl;
01172             os « format(fmt2)
01173                 % "" % "" % "" % ""
01174                 % "" % "(V)" % "(V)"
01175                « endl;
01176             os « format(fmt3)
01177                 % (bit_passed() ? '5' : '6') % whoi % dive % midx++ % (int) bit_version()
01178                 % bat_errors()  % bat_volt_start() % bat_volt_end()
01179                « endl;
01180           }
01181
01182           /* Air Valve */
01183
01184           os « "# --- Air Valve Positions ------------------------------------------------------"
01185               "--------------------"
01186              « endl;
01187
01188           {
01189             static const format fmt1(
01190                 "#   %5s %4s %4s %6s "
01191                 "%13s %13s %13s");
01192             static const format fmt2(
01193                 "#   %5s %4s %4s %6s "
01194                 "%13s %13s %13s");
01195             static const format fmt3(
01196                 "E0%c %5d %4d %4d %6d "
01197                 "%13.3f %13.3f %13.3f");
01198             os « format(fmt1)
01199                 % "SN" % "dive" % "midx" % "BITVer"
01200                 % "open" % "mid" % "closed"
01201                « endl;
01202             os « format(fmt2)
01203                 % "" % "" % "" % ""
01204                 % "(V)" % "(V)" % "(V)"
01205                « endl;
01206             os « format(fmt3)
01207                 % (bit_passed() ? '5' : '6') % whoi % dive % midx++ % (int) bit_version()
01208                 % air_valve_open()  % air_valve_mid() % air_valve_closed()
01209                « endl;
01210           }
01211
01212           os « "# --- Air Valve Calibration Positions ----------------------------------------"
01213               "--------------------"
01214              « endl;
01215
01216           {
01217             static const format fmt1(
01218                 "#   %5s %4s %4s %6s "
01219                 "%13s %13s %13s");
01220             static const format fmt2(
01221                 "#   %5s %4s %4s %6s "
01222                 "%13s %13s %13s");
01223             static const format fmt3(
01224                 "E0%c %5d %4d %4d %6d "
01225                 "%13.3f %13.3f %13.3f");
01226             os « format(fmt1)
01227                 % "SN" % "dive" % "midx" % "BITVer"
01228                 % "open" % "mid" % "closed"
01229                « endl;
01230             os « format(fmt2)
01231                 % "" % "" % "" % ""
01232                 % "(V)" % "(V)" % "(V)"
01233                « endl;
01234             os « format(fmt3)
01235                 % (bit_passed() ? '5' : '6') % whoi % dive % midx++ % (int) bit_version()
01236                 % air_valve_cal_open()  % air_valve_cal_mid() % air_valve_cal_closed()
01237                « endl;
01238           }
01239
01240
01241           /* Air pump */
01242
01243           os « "# --- Air Pump ----------------------------------------------------------------"
01244               "--------------------"
01245              « endl;
01246
01247           {
01248             static const format fmt1(
01249                 "#   %5s %4s %4s %6s "
01250                 "%13s %13s %13s "
01251                 "%13s");
01252             static const format fmt2(
01253                 "#   %5s %4s %4s %6s "
01254                 "%13s %13s %13s "
01255                 "%13s");
01256             static const format fmt3(
01257                 "E0%c %5d %4d %4d %6d "
```

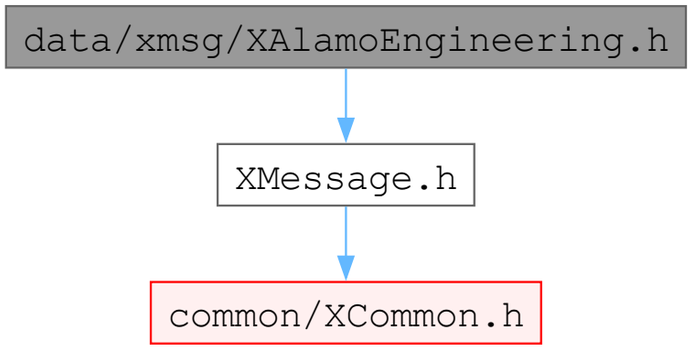
```
01258                 "   0x%08X %13d %13d "
01259                 "%13.3f");
01260             os « format(fmt1)
01261                 % "SN" % "dive" % "midx" % "BITVer"
01262                 % "errors" % "duration" % "cycles"
01263                 % "presMax"
01264                « endl;
01265             os « format(fmt2)
01266                 % "" % "" % "" % ""
01267                 % "" % "(sec)" % ""
01268                 % "(PSI)"
01269                « endl;
01270             os « format(fmt3)
01271                 % (bit_passed() ? '5' : '6') % whoi % dive % midx++ % (int) bit_version()
01272                 % air_pump_errors() % air_pump_test_duration() % (int)air_pump_cycle_count()
01273                 % air_pump_pres_max()
01274                « endl;
01275           }
01276
01277           os « "# ------------------------------------------------------------------------------"
01278              "-------------------"
01279             « endl;
01280
01281           {
01282             static const format fmt1(
01283                 "#   %5s %4s %4s %6s "
01284                 "%13s %13s %13s "
01285                 "%13s %13s");
01286             static const format fmt2(
01287                 "#   %5s %4s %4s %6s "
01288                 "%13s %13s %13s "
01289                 "%13s %13s");
01290             static const format fmt3(
01291                 "E0%c %5d %4d %4d %6d "
01292                 "%13.3f %13.3f %13.3f "
01293                 "%13.3f %13.3f");
01294             os « format(fmt1)
01295                 % "SN" % "dive" % "midx" % "BITVer"
01296                 % "currAvg" % "currMax" % "12VCurrMax"
01297                 % "batMax" % "powerMax"
01298                « endl;
01299             os « format(fmt2)
01300                 % "" % "" % "" % ""
01301                 % "(A)" % "(A)" % "(A)"
01302                 % "(V)" % "(W)"
01303                « endl;
01304             os « format(fmt3)
01305                 % (bit_passed() ? '5' : '6') % whoi % dive % midx++ % (int) bit_version()
01306                 % air_pump_curr_avg() % air_pump_curr_max() % air_pump_v12_curr_max()
01307                 % air_pump_volt_max() % (12*air_pump_v12_curr_max())
01308                « endl;
01309           }
01310
01311
01312           /* Oil Valve */
01313
01314           os « "# --- Oil Valve Positions ------------------------------------------------------"
01315              "-------------------"
01316             « endl;
01317
01318           {
01319             static const format fmt1(
01320                 "#   %5s %4s %4s %6s "
01321                 "%13s %13s");
01322             static const format fmt2(
01323                 "#   %5s %4s %4s %6s "
01324                 "%13s %13s");
01325             static const format fmt3(
01326                 "E0%c %5d %4d %4d %6d "
01327                 "%13.3f %13.3f");
01328             os « format(fmt1)
01329                 % "SN" % "dive" % "midx" % "BITVer"
01330                 % "open" % "closed"
01331                « endl;
01332             os « format(fmt2)
01333                 % "" % "" % "" % ""
01334                 % "(V)" % "(V)"
01335                « endl;
01336             os « format(fmt3)
01337                 % (bit_passed() ? '5' : '6') % whoi % dive % midx++ % (int) bit_version()
01338                 % oil_valve_open() % oil_valve_closed()
01339                « endl;
01340           }
01341
01342           os « "# --- Oil Valve Calibration Positions ---------------------------------------"
01343              "-------------------"
01344             « endl;
01345
01346           {
01347             static const format fmt1(
01348                 "#   %5s %4s %4s %6s "
01349                 "%13s %13s");
01350             static const format fmt2(
01351                 "#   %5s %4s %4s %6s "
01352                 "%13s %13s");
01353             static const format fmt3(
01354                 "E0%c %5d %4d %4d %6d "
01355                 "%13.3f %13.3f");
01356             os « format(fmt1)
01357                 % "SN" % "dive" % "midx" % "BITVer"
01358                 % "open" % "closed"
01359                « endl;
01360             os « format(fmt2)
01361                 % "" % "" % "" % ""
01362                 % "(V)" % "(V)"
01363                « endl;
01364             os « format(fmt3)
01365                 % (bit_passed() ? '5' : '6') % whoi % dive % midx++ % (int) bit_version()
```

```
01366                 % oil_valve_cal_open() % oil_valve_cal_closed()
01367                 « endl;
01368           }
01369
01370           /* Oil pump */
01371
01372           os « "# --- Oil Pump ----------------------------------------------------------------------"
01373               "--------------------"
01374              « endl;
01375
01376           {
01377             static const format fmt1(
01378                 "#   %5s %4s %4s %6s "
01379                 "%13s %13s ");
01380             static const format fmt2(
01381                 "#   %5s %4s %4s %6s "
01382                 "%13s %13s ");
01383             static const format fmt3(
01384                 "E0%c %5d %4d %4d %6d "
01385                 "   0x%08X %13d ");
01386             os « format(fmt1)
01387                 % "SN" % "dive" % "midx" % "BITVer"
01388                 % "errors" % "duration"
01389                « endl;
01390             os « format(fmt2)
01391                 % "" % "" % "" % ""
01392                 % "" % "(sec)"
01393                « endl;
01394             os « format(fmt3)
01395                 % (bit_passed() ? '5' : '6') % whoi % dive % midx++ % (int) bit_version()
01396                 % oil_pump_errors() % oil_pump_test_duration()
01397                « endl;
01398           }
01399
01400           os « "# ------------------------------------------------------------------------------"
01401               "--------------------"
01402              « endl;
01403
01404           {
01405             static const format fmt1(
01406                 "#   %5s %4s %4s %6s "
01407                 "%13s %13s %13s "
01408                 "%13s %13s");
01409             static const format fmt2(
01410                 "#   %5s %4s %4s %6s "
01411                 "%13s %13s %13s "
01412                 "%13s %13s");
01413             static const format fmt3(
01414                 "E0%c %5d %4d %4d %6d "
01415                 "%13.3f %13.3f %13.3f "
01416                 "%13.3f %13.3f");
01417             os « format(fmt1)
01418                 % "SN" % "dive" % "midx" % "BITVer"
01419                 % "currAvg" % "currMax" % "12VCurrMax"
01420                 % "batMax" % "powerMax"
01421                « endl;
01422             os « format(fmt2)
01423                 % "" % "" % "" % ""
01424                 % "(A)" % "(A)" % "(A)"
01425                 % "(V)" % "(W)"
01426                « endl;
01427             os « format(fmt3)
01428                 % (bit_passed() ? '5' : '6') % whoi % dive % midx++ % (int) bit_version()
01429                 % oil_pump_curr_avg()  % oil_pump_curr_max() % oil_pump_v12_curr_max()
01430                 % oil_pump_volt_max() % (12*oil_pump_v12_curr_max())
01431                « endl;
01432           }
01433
01434
01435       }
01436
01437
01438       break;
01439
01440
01441         default: { // Unsupported Engineering (BIT) version.
01442           ostringstream os;
01443           os « "Unsupported ALAMO Engineering (BIT v2) message version: 0x"
01444              « hex
01445              « (int) bit_version()
01446              « "!";
01447           EXC_THROW(os.str());
01448       }
01449
01450           break;
01451
01452       }
01453
01454
01455       return os.str();
01456
01457     }
01458
01459 };
01460
01461 } /* namespace xmsg */
01462 } /* namespace util */
01463
01464 #endif /* DATA_XMSG_XALAMOBIT2_H_ */
```

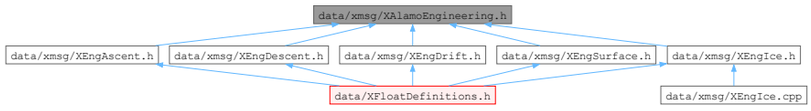
11.70 data/xmsg/XAlamoEngineering.h File Reference

The util::xmsg::XAlamoEngineering class header file.

`#include "XMessage.h"`
Include dependency graph for XAlamoEngineering.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XAlamoEngineering
  Abstract class representing a version controlled engineering message for ALAMO-type instruments.

Namespaces

- namespace util
  The utility namespace.
- namespace util::xmsg
  The X-Message utility namespace.

11.70.1 Detailed Description

The util::xmsg::XAlamoEngineering class header file.

Version

0.6

Date

Aug 5, 2014 1:32:39 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.71 XAlamoEngineering.h

Go to the documentation of this file.

```
00001 /** The @ref util::xmsg::XAlamoEngineering class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Aug 5, 2014 1:32:39 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef XALAMOENGINEERING_H_
00024 #define XALAMOENGINEERING_H_
00025
00026 /* Includes */
00027
00028 #include "XMessage.h"
00029
00030 /* Definitions */
00031
00032 namespace util {
00033 namespace xmsg {
00034
00035 /* Class Definitions */
00036
00037 /** Abstract class representing a version controlled engineering message for ALAMO-type instruments.
00038  *
00039  *  @note           The X-Message types extended from this class are used by ALAMO-type instruments
00040  *                  only.
00041  *  @note           The X-Message types extended from this class were introduced in the X-Message
00042  *                  format specification version 2.1.
00043  */
00044 class XAlamoEngineering:
00045         public XMessage,
00046         public util::date::DateUtils {
00047 private:
00048
00049     /* Data Members */
00050
00051     /** The engineering message version identifier. */
00052     uint8_t engineering_version_;
00053
00054     /** The engineering message state start time. */
00055     ptime start_time_;
00056
00057 protected:
00058
00059     /* Member Functions */
00060
00061     /** Set the engineering message version identifier.
00062      *
00063      *  @param[in]  engineering_version The engineering message version identifier.
00064      */
00065     void set_engineering_version(uint8_t engineering_version) {
00066         engineering_version_ = engineering_version;
00067     }
00068
00069     /** Set the engineering message state start time
00070      *
00071      *  @param[in]  start_time          The engineering message state start time.
00072      */
00073     void set_start_time(const ptime& start_time) {
00074         start_time_ = start_time;
00075     }
00076
00077 public:
00078
00079     /* Member Functions */
00080
00081     /** Parameterized constructor.
00082      *
00083      *  @param[in]      dive        The dive or cycle number of this message.
00084      *  @param[in]      whoi        The unique WHOI identifier.
00085      *  @param[in]      data        The binary X-message data to decode.
00086      */
00087     XAlamoEngineering(int16_t dive, uint16_t whoi, const vector<uint8_t>& data) :
00088                     XMessage(dive, whoi, data),
00089                     engineering_version_(XMSG_ENG_BAD_VER),
00090                     start_time_(not_a_date_time) {
00091         /* Do nothing. */
00092     }
00093
00094     /** Destructor. */
00095     virtual ~XAlamoEngineering() {
00096         /* Do nothing. */
00097     }
00098
00099     /** Get the engineering message state start time.
00100      *
00101      *  @return         The engineering message state start time.
00102      */
00103     const ptime& start_time() const {
```

```
00104         return start_time_;
00105     }
00106
00107     /** Get the engineering message version identifier.
00108      *
00109      *  @return         The engineering message version identifier.
00110      */
00111     uint8_t engineering_version() const {
00112         return engineering_version_;
00113     }
00114
00115     /** Get the engineering message verion identifier as a floating point number.
00116      *
00117      *  @return         The engineering message verion identifier as a floating point number.
00118      */
00119     float engineering_version_as_float() const {
00120         volatile float x = 0x0F & engineering_version_;
00121         while(x > 1)
00122             x /= 10;
00123         return x + ((0xF0 & engineering_version_) » 4);
00124     }
00125
00126 };
00127
00128 } /* namespace xmsg */
00129 } /* namespace util */
00130 #endif /* XALAMOENGINEERING_H_ */
```

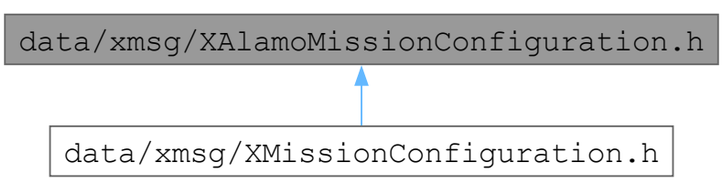
11.72 data/xmsg/XAlamoMissionConfiguration.h File Reference

The util::xmsg::XAlamoMissionConfiguration class header file.

```
#include "../../utility/DateUtils.h"
#include "../../utility/Log.h"
#include "common/XDefinitions.h"
```

Include dependency graph for XAlamoMissionConfiguration.h:

This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XAlamoMissionConfiguration

  Class representing an ALAMO Mission Configuration message.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.72.1 Detailed Description

The util::xmsg::XAlamoMissionConfiguration class header file.

Version

0.6

Date

Jul 15, 2014 4:02:57 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.73 XAlamoMissionConfiguration.h

Go to the documentation of this file.

```
00001 /** The @ref util::xmsg::XAlamoMissionConfiguration class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jul 15, 2014 4:02:57 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef XALAMOMISSIONCONFIGURATION_H_
00024 #define XALAMOMISSIONCONFIGURATION_H_
00025
00026 /* Includes */
00027
00028 #include "../../utility/DateUtils.h"
00029 #include "../../utility/Log.h"
00030 #include "common/XDefinitions.h"
00031
00032 namespace util {
00033 namespace xmsg {
00034
00035 /** Class representing an ALAMO Mission Configuration message.
00036  *
00037  */
00038 class XAlamoMissionConfiguration:
00039         public util::date::DateUtils {
00040
00041 private:
00042
00043     /* Data Members */
00044
00045     /** The software version identifier.  */
00046     uint32_t fw_version_;
00047
00048     /** The message version identifier.
00049      *
00050      */
00051     uint8_t msg_version_;
00052
00053     /** The sensor type identifier.
00054      *
00055      */
00056     uint8_t sensor_type_;
```

```
00057
00058     /* Data Members - General Configuration */
00059
00060     /** The mission, dive, or cycle number for this mission configuration. */
00061     int16_t msn_;
00062
00063     /** The target descent/drift/park pressure, in dBar. */
00064     uint16_t dtd_;
00065
00066     /** The maximum descent/fall/profile pressure, in dBar. */
00067     uint16_t mds_;
00068
00069     /** The auto-transmit mission configuration message flag.
00070      *
00071      */
00072     uint8_t apt_;
00073
00074     /** The bluetooth mode.
00075      *
00076      */
00077     uint8_t btm_;
00078
00079     /** The profile data encode method.
00080      *
00081      */
00082     uint8_t xas_;
00083
00084     /** The initial trigger depth, in dBar.
00085      *
00086      */
00087     uint16_t mtd_;
00088
00089     /** The initial trigger timeout/interval, in milliseconds.
00090      *
00091      */
00092     uint16_t mtt_;
00093
00094     /* Data Members - Ice Configuration */
00095
00096     uint8_t icm_;
00097
00098     uint16_t imm_;
00099
00100     uint8_t ism_;
00101
00102     uint16_t ica_[2];
00103
00104     uint16_t icb_[2];
00105
00106     uint16_t imt_[2];
00107
00108     uint16_t ilt_[2];
00109
00110     uint16_t icn_;
00111
00112     uint16_t icx_;
00113
00114     int16_t ict_;
00115
00116     uint16_t ics_;
00117
00118     uint8_t iss_[2];
00119
00120     uint16_t ist_;
00121
00122     uint16_t iso_;
00123
00124     uint8_t isp_;
00125
00126     uint8_t iue_[2];
00127
00128     uint16_t icd_[2];
00129
00130     uint32_t icr_[2];
00131
00132     uint8_t icg_[2];
00133
00134     uint8_t isa_;
00135
00136     uint16_t isd_;
00137
00138     uint16_t iat_;
00139
00140     uint16_t isv_;
00141
00142     uint16_t isw_;
00143
00144     uint16_t ico_[2];
00145
00146     uint32_t icf_;
00147
00148     uint32_t icw_;
00149
00150     /* Data Members - IMU Configuration */
00151
00152     /** The motion processing unit (MPU) data acquisition operational mode.
00153      *
00154      */
00155     uint8_t mam_;
00156
00157     /** The motion processing unit (MPU) sampling duration, in seconds.
00158      *
00159      */
00160     uint16_t mat_;
00161
00162     uint16_t aca_;
00163
00164     uint16_t acg_;
```

```
00165
00166     uint16_t acr_;
00167
00168     /* Data Members - BIT Configuration */
00169
00170     uint16_t pnb_;
00171
00172     uint16_t pnc_;
00173
00174     uint16_t pxc_;
00175
00176     /* Data Members - Calibration */
00177
00178     uint16_t pbl_;
00179
00180     uint16_t pbh_;
00181
00182     uint16_t pp0_;
00183
00184     uint16_t ppl_;
00185
00186     uint16_t pvl_;
00187
00188     uint16_t pvh_;
00189
00190     uint16_t pv0_;
00191
00192     uint16_t pv5_;
00193
00194     uint16_t pc0_;
00195
00196     uint16_t pcl_;
00197
00198     /* Data Members - Profile Configuration */
00199
00200     /** The CTD profiling mode.
00201      *
00202      */
00203     uint8_t cps_;
00204
00205     /** The CTD sampling type.
00206      *
00207      */
00208     uint8_t cst_;
00209
00210     /** The profile cutoff pressure.
00211      *
00212      *  @todo   Determine gain for profile cutoff pressure field.
00213      *
00214      *  @note   This field was added in firmware version 0x02006900.
00215      *
00216      */
00217     uint16_t ctf_;
00218
00219     /** The pressure sensor offset reset lower pressure limit.
00220      *
00221      *  @todo   Determine gain for pressure sensor offset reset lower pressure limit field.
00222      *
00223      *  @note   This field was added in firmware version 0x02006900.
00224      *
00225      */
00226     uint16_t ctl_;
00227
00228     /** The pressure sensor offset reset lower pressure limit.
00229      *
00230      *  @todo   Determine gain for pressure sensor offset reset lower pressure limit field.
00231      *
00232      *
00233      *  @note   This field was added in firmware version 0x02006900.
00234      *
00235      */
00236     uint16_t ctu_;
00237
00238     /** The CTD continuous sampling period, in milliseconds.
00239      *
00240      */
00241     uint32_t csp_;
00242
00243     /** The CTD sampling period pressure boundaries, in dBar.
00244      *
00245      */
00246     uint16_t cpb_[3];
00247
00248     /** The CTD sampling period bin sizes, in 0.1 dBar
00249      *
00250      */
00251     uint16_t cpn_[3];
00252
00253     /** The CTD sampling period interval, in milliseconds.
00254      *
00255      */
00256     uint16_t cpp_[3];
00257
00258     /* Data Members - Descent State Configuration */
00259
00260     /** The start time of the descend state of the dive cycle.
00261      *
00262      */
00263     ptime descend_start_time_;
00264
00265     /** The descent number.
00266      *
00267      */
00268     uint16_t dns_;
00269
00270     uint8_t dra_;
00271
00272     /** The descent state start pressure, in dBar.
```

```
00273      *
00274      */
00275     uint16_t dsd_;
00276
00277     /** The descent state start timeout, in seconds.
00278      *
00279      */
00280     uint16_t dst_;
00281
00282     uint16_t ddf_;
00283
00284     uint16_t ddc_;
00285
00286     /** The descent state sampling interval, in milliseconds.
00287      *
00288      */
00289     uint32_t dsi_;
00290
00291     /** The descent state data storage pressure interval, in dBar.
00292      *
00293      */
00294     uint16_t ddd_;
00295
00296     /** The descent state data storage interval, in seconds.
00297      *
00298      */
00299     uint16_t ddt_;
00300
00301     /** The descent state fall timeout, in seconds.
00302      *
00303      */
00304     uint16_t dft_;
00305
00306     uint8_t dcm_;
00307
00308     uint8_t dal_;
00309
00310     uint16_t dct_;
00311
00312
00313     uint8_t dca_;
00314
00315     uint16_t drc_;
00316
00317     uint16_t drt_;
00318
00319     uint16_t dcx_;
00320
00321     uint16_t dcp_;
00322
00323     uint16_t dcf_;
00324
00325     uint16_t dcd_;
00326
00327     uint16_t dcr_;
00328
00329     uint16_t dsw_;
00330
00331     uint16_t duc_;
00332
00333     uint16_t dpt_;
00334
00335     uint16_t dut_;
00336
00337     uint16_t dpd_;
00338
00339     uint16_t dvt_;
00340
00341     uint16_t dvd_;
00342
00343     uint16_t dvw_;
00344
00345
00346     uint16_t pfr_;
00347
00348     uint16_t pfc_;
00349
00350     uint8_t ftm_;
00351
00352     uint8_t fta_;
00353
00354     uint16_t fto_;
00355
00356     uint16_t fos_;
00357
00358     uint16_t ddp_;
00359
00360     uint16_t dds_;
00361
00362     /* Data Members - Drift/Park State Configuration */
00363
00364     /** The start time of the park/drift state of the dive cycle.
00365      *
00366      */
00367     ptime park_start_time_;
00368
00369     /** The park/drift number.
00370      *
00371      */
00372     uint16_t pns_;
00373
00374     /** The park/drift state timeout, in seconds.
00375      *
00376      */
00377     uint32_t pto_;
00378
00379     /** The bit-encoded park/drift state hour timeout trigger(s).
00380      *
```

```
00381      */
00382     uint32_t pth_;
00383
00384     /** The park/drift state sampling mode.
00385      *
00386      */
00387     uint8_t psa_;
00388
00389     /** The park/drift state sample interval/timeout, in milliseconds.
00390      *
00391      */
00392     uint32_t psi_;
00393
00394     /** The park/drift state fall mode.
00395      *
00396      */
00397     uint8_t pfm_;
00398
00399     uint16_t pft_;
00400
00401     /** The park/drift state fall depth (i.e. profile depth), in dBar.
00402      *
00403      */
00404     uint16_t pfd_;
00405
00406     uint16_t pfv_;
00407
00408     /** The park/drift state data storage interval/timeout, in seconds.
00409      *
00410      */
00411     uint16_t pdt_;
00412
00413     uint8_t psm_;
00414
00415     uint16_t psd_;
00416
00417     uint16_t pst_;
00418
00419     uint16_t psv_;
00420
00421     uint16_t psw_;
00422
00423     /* Data Members - Ascent State Configuration */
00424
00425     /** The start time of the ascent state of the dive cycle.
00426      *
00427      */
00428     ptime ascent_start_time_;
00429
00430     /** The ascent number.
00431      *
00432      */
00433     uint16_t asn_;
00434
00435     /** The ascent state rise rate, in centimeters per second.
00436      *
00437      */
00438     uint16_t arr_;
00439
00440     uint16_t apm_;
00441
00442     /** The ascent state pump on duration/interval/timeout, in seconds.
00443      *
00444      */
00445     uint16_t apn_;
00446
00447     /** The ascent state pump off duration/interval/timeout, in seconds.
00448      *
00449      */
00450     uint16_t apf_;
00451
00452     /** The ascent state sampling interval/timeout, in milliseconds.
00453      *
00454      */
00455     uint32_t asi_;
00456
00457     /** The ascent state surface threshold pressure, in dBar.
00458      *
00459      */
00460     uint16_t asd_;
00461
00462     /** The ascent state data storage pressure interval, in dBar.
00463      *
00464      */
00465     uint16_t add_;
00466
00467     /** The ascent state data storage interval timeout, in seconds.
00468      *
00469      */
00470     uint16_t adt_;
00471
00472     uint16_t arc_;
00473
00474     uint16_t ahr_;
00475
00476     uint16_t ahd_;
00477
00478     /* Data Members - Surface State Configuration */
00479
00480     /** The start time of the surface state of the dive cycle.
00481      *
00482      */
00483     ptime surface_start_time_;
00484
00485     /** The surface number.
00486      *
00487      */
00488     uint16_t sfn_;
```

```
00489
00490     /** The surface state sensor depth compensation offset, in dBar.
00491      *
00492      */
00493     int16_t sdc_;
00494
00495     /** The surface state GPS timeout, in minutes.
00496      *
00497      */
00498     uint16_t sgt_;
00499
00500     /** The surface state Iridium session timeout, in minutes.
00501      *
00502      */
00503     uint16_t set_;
00504
00505     /** The surface state beacon mode flag.
00506      *
00507      */
00508     uint8_t sbm_;
00509
00510     /** The surface state beacon interval timeout, in seconds.
00511      *
00512      */
00513     uint16_t sbt_;
00514
00515     uint32_t sdt_;
00516
00517     uint16_t ssa_;
00518
00519     /** The surface state sampling interval timeout, in milliseconds.
00520      *
00521      */
00522     uint32_t ssi_;
00523
00524     /** The surface state sampling storage duration/interval/timeout, in seconds.
00525      *
00526      */
00527     uint16_t sds_;
00528
00529     uint8_t sam_;
00530
00531     uint16_t sax_;
00532
00533     uint16_t san_;
00534
00535     uint16_t sao_;
00536
00537     uint16_t sap_;
00538
00539     uint16_t spt_;
00540
00541     uint16_t ssp_;
00542
00543 private:
00544
00545     /* Private Member Functions: */
00546
00547     /** Default constructor.
00548      *
00549      */
00550 #   pragma GCC diagnostic push
00551 #   pragma GCC diagnostic ignored "-Wreorder"
00552     XAlamoMissionConfiguration(void) :
00553             aca_(XMSG_BAD_U16),
00554             acg_(XMSG_BAD_U16),
00555             acr_(XMSG_BAD_U16),
00556             add_(XMSG_BAD_U16),
00557             adt_(XMSG_BAD_U16),
00558             ahd_(XMSG_BAD_U16),
00559             ahr_(XMSG_BAD_U16),
00560             apf_(XMSG_BAD_U16),
00561             apm_(XMSG_BAD_U16),
00562             apn_(XMSG_BAD_U16),
00563             apt_(XMSG_BAD_U08),
00564             arc_(XMSG_BAD_U16),
00565             arr_(XMSG_BAD_U16),
00566             asd_(XMSG_BAD_U16),
00567             asi_(XMSG_BAD_U32),
00568             asn_(XMSG_BAD_U16),
00569             btm_(XMSG_BAD_U08),
00570             cpb_{ XMSG_BAD_U16, XMSG_BAD_U16, XMSG_BAD_U16, },
00571             cpn_{ XMSG_BAD_U16, XMSG_BAD_U16, XMSG_BAD_U16, },
00572             cpp_{ XMSG_BAD_U16, XMSG_BAD_U16, XMSG_BAD_U16, },
00573             cps_(XMSG_BAD_U08),
00574             csp_(XMSG_BAD_U32),
00575             cst_(XMSG_BAD_U08),
00576             ctf_(XMSG_BAD_U16),
00577             ctl_(XMSG_BAD_U16),
00578             ctu_(XMSG_BAD_U16),
00579             dal_(XMSG_BAD_U08),
00580             dca_(XMSG_BAD_U08),
00581             dcd_(XMSG_BAD_U16),
00582             dcf_(XMSG_BAD_U16),
00583             dcm_(XMSG_BAD_U08),
00584             dcp_(XMSG_BAD_U16),
00585             dcr_(XMSG_BAD_U16),
00586             dct_(XMSG_BAD_U16),
00587             dcx_(XMSG_BAD_U16),
00588             ddc_(XMSG_BAD_U16),
00589             ddd_(XMSG_BAD_U16),
00590             ddf_(XMSG_BAD_U16),
00591             ddp_(XMSG_BAD_U16),
00592             dds_(XMSG_BAD_U16),
00593             ddt_(XMSG_BAD_U16),
00594             dft_(XMSG_BAD_U16),
00595             dns_(XMSG_BAD_U16),
00596             dpd_(XMSG_BAD_U16),
```

```
00597             dpt_(XMSG_BAD_U16),
00598             dra_(XMSG_BAD_U08),
00599             drc_(XMSG_BAD_U16),
00600             drt_(XMSG_BAD_U16),
00601             dsd_(XMSG_BAD_U16),
00602             dsi_(XMSG_BAD_U32),
00603             dst_(XMSG_BAD_U16),
00604             dsw_(XMSG_BAD_U16),
00605             dtd_(XMSG_BAD_U16),
00606             duc_(XMSG_BAD_U16),
00607             dut_(XMSG_BAD_U16),
00608             dvd_(XMSG_BAD_U16),
00609             dvt_(XMSG_BAD_U16),
00610             dvw_(XMSG_BAD_U16),
00611             fos_(XMSG_BAD_U16),
00612             fta_(XMSG_BAD_U08),
00613             ftm_(XMSG_BAD_U08),
00614             fto_(XMSG_BAD_U16),
00615             fw_version_(XMSG_BAD_U32),
00616             iat_(XMSG_BAD_U16),
00617             ica_{ XMSG_BAD_U16, XMSG_BAD_U16, },
00618             icb_{ XMSG_BAD_U16, XMSG_BAD_U16, },
00619             icd_{ XMSG_BAD_U16, XMSG_BAD_U16, },
00620             icf_(XMSG_BAD_U32),
00621             icg_{ XMSG_BAD_U08, XMSG_BAD_U08, },
00622             icm_(XMSG_BAD_U08),
00623             icn_(XMSG_BAD_U16),
00624             ico_{ XMSG_BAD_U16, XMSG_BAD_U16, },
00625             icr_{ XMSG_BAD_U32, XMSG_BAD_U32, },
00626             ics_(XMSG_BAD_U16),
00627             ict_(XMSG_BAD_I16),
00628             icw_(XMSG_BAD_U32),
00629             icx_(XMSG_BAD_U16),
00630             ilt_{ XMSG_BAD_U16, XMSG_BAD_U16, },
00631             imm_(XMSG_BAD_U16),
00632             imt_{ XMSG_BAD_U16, XMSG_BAD_U16, },
00633             isa_(XMSG_BAD_U08),
00634             isd_(XMSG_BAD_U16),
00635             ism_(XMSG_BAD_U08),
00636             iso_(XMSG_BAD_U16),
00637             isp_(XMSG_BAD_U08),
00638             iss_{ XMSG_BAD_U08, XMSG_BAD_U08, },
00639             ist_(XMSG_BAD_U16),
00640             isv_(XMSG_BAD_U16),
00641             isw_(XMSG_BAD_U16),
00642             iue_{ XMSG_BAD_U08, XMSG_BAD_U08, },
00643             mam_(XMSG_BAD_U08),
00644             mat_(XMSG_BAD_U16),
00645             mds_(XMSG_BAD_U16),
00646             msg_version_(XMSG_BAD_U08),
00647             msn_(XMSG_BAD_I16),
00648             mtd_(XMSG_BAD_U16),
00649             mtt_(XMSG_BAD_U16),
00650             pbh_(XMSG_BAD_U16),
00651             pbl_(XMSG_BAD_U16),
00652             pc0_(XMSG_BAD_U16),
00653             pc1_(XMSG_BAD_U16),
00654             pdt_(XMSG_BAD_U16),
00655             pfc_(XMSG_BAD_U16),
00656             pfd_(XMSG_BAD_U16),
00657             pfm_(XMSG_BAD_U08),
00658             pfr_(XMSG_BAD_U16),
00659             pft_(XMSG_BAD_U16),
00660             pfv_(XMSG_BAD_U16),
00661             pnb_(XMSG_BAD_U16),
00662             pnc_(XMSG_BAD_U16),
00663             pns_(XMSG_BAD_U16),
00664             pp0_(XMSG_BAD_U16),
00665             pp1_(XMSG_BAD_U16),
00666             psa_(XMSG_BAD_U08),
00667             psd_(XMSG_BAD_U16),
00668             psi_(XMSG_BAD_U32),
00669             psm_(XMSG_BAD_U08),
00670             pst_(XMSG_BAD_U16),
00671             psv_(XMSG_BAD_U16),
00672             psw_(XMSG_BAD_U16),
00673             pth_(XMSG_BAD_U32),
00674             pto_(XMSG_BAD_U32),
00675             pv0_(XMSG_BAD_U16),
00676             pv5_(XMSG_BAD_U16),
00677             pvh_(XMSG_BAD_U16),
00678             pvl_(XMSG_BAD_U16),
00679             pxc_(XMSG_BAD_U16),
00680             sam_(XMSG_BAD_U08),
00681             san_(XMSG_BAD_U16),
00682             sao_(XMSG_BAD_U16),
00683             sap_(XMSG_BAD_U16),
00684             sax_(XMSG_BAD_U16),
00685             sbm_(XMSG_BAD_U08),
00686             sbt_(XMSG_BAD_U16),
00687             sdc_(XMSG_BAD_I16),
00688             sds_(XMSG_BAD_U16),
00689             sdt_(XMSG_BAD_U32),
00690             sensor_type_(XMSG_BAD_U08),
00691             set_(XMSG_BAD_U16),
00692             sfn_(XMSG_BAD_U16),
00693             sgt_(XMSG_BAD_U16),
00694             spt_(XMSG_BAD_U16),
00695             ssa_(XMSG_BAD_U16),
00696             ssi_(XMSG_BAD_U32),
00697             ssp_(XMSG_BAD_U16),
00698             xas_(XMSG_BAD_U08) {
00699         /* Do nothing. */
00700     }
00701 #   pragma GCC diagnostic pop
00702
00703 public:
00704
```

```
00705     /* Public Member Functions: */
00706
00707     /** Parameterized Constructor.
00708      *
00709      *  @param[in]      data        The binary data to decode.
00710      *
00711      *  @throw          log::MessageException if the message is invalid or an error occurs while
00712      *                  decoding.
00713      */
00714 #   pragma GCC diagnostic push
00715 #   pragma GCC diagnostic ignored "-Wreorder"
00716     XAlamoMissionConfiguration(const vector<uint8_t>& data):
00717             aca_(XMSG_BAD_U16),
00718             acg_(XMSG_BAD_U16),
00719             acr_(XMSG_BAD_U16),
00720             add_(XMSG_BAD_U16),
00721             adt_(XMSG_BAD_U16),
00722             ahd_(XMSG_BAD_U16),
00723             ahr_(XMSG_BAD_U16),
00724             apf_(XMSG_BAD_U16),
00725             apm_(XMSG_BAD_U16),
00726             apn_(XMSG_BAD_U16),
00727             apt_(XMSG_BAD_U08),
00728             arc_(XMSG_BAD_U16),
00729             arr_(XMSG_BAD_U16),
00730             asd_(XMSG_BAD_U16),
00731             asi_(XMSG_BAD_U32),
00732             asn_(XMSG_BAD_U16),
00733             btm_(XMSG_BAD_U08),
00734             cpb_{ XMSG_BAD_U16, XMSG_BAD_U16, XMSG_BAD_U16, },
00735             cpn_{ XMSG_BAD_U16, XMSG_BAD_U16, XMSG_BAD_U16, },
00736             cpp_{ XMSG_BAD_U16, XMSG_BAD_U16, XMSG_BAD_U16, },
00737             cps_(XMSG_BAD_U08),
00738             csp_(XMSG_BAD_U32),
00739             cst_(XMSG_BAD_U08),
00740             ctf_(XMSG_BAD_U16),
00741             ctl_(XMSG_BAD_U16),
00742             ctu_(XMSG_BAD_U16),
00743             dal_(XMSG_BAD_U08),
00744             dca_(XMSG_BAD_U08),
00745             dcd_(XMSG_BAD_U16),
00746             dcf_(XMSG_BAD_U16),
00747             dcm_(XMSG_BAD_U08),
00748             dcp_(XMSG_BAD_U16),
00749             dcr_(XMSG_BAD_U16),
00750             dct_(XMSG_BAD_U16),
00751             dcx_(XMSG_BAD_U16),
00752             ddc_(XMSG_BAD_U16),
00753             ddd_(XMSG_BAD_U16),
00754             ddf_(XMSG_BAD_U16),
00755             ddp_(XMSG_BAD_U16),
00756             dds_(XMSG_BAD_U16),
00757             ddt_(XMSG_BAD_U16),
00758             dft_(XMSG_BAD_U16),
00759             dns_(XMSG_BAD_U16),
00760             dpd_(XMSG_BAD_U16),
00761             dpt_(XMSG_BAD_U16),
00762             dra_(XMSG_BAD_U08),
00763             drc_(XMSG_BAD_U16),
00764             drt_(XMSG_BAD_U16),
00765             dsd_(XMSG_BAD_U16),
00766             dsi_(XMSG_BAD_U32),
00767             dst_(XMSG_BAD_U16),
00768             dsw_(XMSG_BAD_U16),
00769             dtd_(XMSG_BAD_U16),
00770             duc_(XMSG_BAD_U16),
00771             dut_(XMSG_BAD_U16),
00772             dvd_(XMSG_BAD_U16),
00773             dvt_(XMSG_BAD_U16),
00774             dvw_(XMSG_BAD_U16),
00775             fos_(XMSG_BAD_U16),
00776             fta_(XMSG_BAD_U08),
00777             ftm_(XMSG_BAD_U08),
00778             fto_(XMSG_BAD_U16),
00779             fw_version_(XMSG_BAD_U32),
00780             iat_(XMSG_BAD_U16),
00781             ica_{ XMSG_BAD_U16, XMSG_BAD_U16, },
00782             icb_{ XMSG_BAD_U16, XMSG_BAD_U16, },
00783             icd_{ XMSG_BAD_U16, XMSG_BAD_U16, },
00784             icf_(XMSG_BAD_U32),
00785             icg_{ XMSG_BAD_U08, XMSG_BAD_U08, },
00786             icm_(XMSG_BAD_U08),
00787             icn_(XMSG_BAD_U16),
00788             ico_{ XMSG_BAD_U16, XMSG_BAD_U16, },
00789             icr_{ XMSG_BAD_U32, XMSG_BAD_U32, },
00790             ics_(XMSG_BAD_U16),
00791             ict_(XMSG_BAD_I16),
00792             icw_(XMSG_BAD_U32),
00793             icx_(XMSG_BAD_U16),
00794             ilt_{ XMSG_BAD_U16, XMSG_BAD_U16, },
00795             imm_(XMSG_BAD_U16),
00796             imt_{ XMSG_BAD_U16, XMSG_BAD_U16, },
00797             isa_(XMSG_BAD_U08),
00798             isd_(XMSG_BAD_U16),
00799             ism_(XMSG_BAD_U08),
00800             iso_(XMSG_BAD_U16),
00801             isp_(XMSG_BAD_U08),
00802             iss_{ XMSG_BAD_U08, XMSG_BAD_U08, },
00803             ist_(XMSG_BAD_U16),
00804             isv_(XMSG_BAD_U16),
00805             isw_(XMSG_BAD_U16),
00806             iue_{ XMSG_BAD_U08, XMSG_BAD_U08, },
00807             mam_(XMSG_BAD_U08),
00808             mat_(XMSG_BAD_U16),
00809             mds_(XMSG_BAD_U16),
00810             msg_version_(XMSG_BAD_U08),
00811             msn_(XMSG_BAD_I16),
00812             mtd_(XMSG_BAD_U16),
```

```
00813             mtt_(XMSG_BAD_U16),
00814             pbh_(XMSG_BAD_U16),
00815             pbl_(XMSG_BAD_U16),
00816             pc0_(XMSG_BAD_U16),
00817             pc1_(XMSG_BAD_U16),
00818             pdt_(XMSG_BAD_U16),
00819             pfc_(XMSG_BAD_U16),
00820             pfd_(XMSG_BAD_U16),
00821             pfm_(XMSG_BAD_U08),
00822             pfr_(XMSG_BAD_U16),
00823             pft_(XMSG_BAD_U16),
00824             pfv_(XMSG_BAD_U16),
00825             pnb_(XMSG_BAD_U16),
00826             pnc_(XMSG_BAD_U16),
00827             pns_(XMSG_BAD_U16),
00828             pp0_(XMSG_BAD_U16),
00829             pp1_(XMSG_BAD_U16),
00830             psa_(XMSG_BAD_U08),
00831             psd_(XMSG_BAD_U16),
00832             psi_(XMSG_BAD_U32),
00833             psm_(XMSG_BAD_U08),
00834             pst_(XMSG_BAD_U16),
00835             psv_(XMSG_BAD_U16),
00836             psw_(XMSG_BAD_U16),
00837             pth_(XMSG_BAD_U32),
00838             pto_(XMSG_BAD_U32),
00839             pv0_(XMSG_BAD_U16),
00840             pv5_(XMSG_BAD_U16),
00841             pvh_(XMSG_BAD_U16),
00842             pvl_(XMSG_BAD_U16),
00843             pxc_(XMSG_BAD_U16),
00844             sam_(XMSG_BAD_U08),
00845             san_(XMSG_BAD_U16),
00846             sao_(XMSG_BAD_U16),
00847             sap_(XMSG_BAD_U16),
00848             sax_(XMSG_BAD_U16),
00849             sbm_(XMSG_BAD_U08),
00850             sbt_(XMSG_BAD_U16),
00851             sdc_(XMSG_BAD_I16),
00852             sds_(XMSG_BAD_U16),
00853             sdt_(XMSG_BAD_U32),
00854             sensor_type_(XMSG_BAD_U08),
00855             set_(XMSG_BAD_U16),
00856             sfn_(XMSG_BAD_U16),
00857             sgt_(XMSG_BAD_U16),
00858             spt_(XMSG_BAD_U16),
00859             ssa_(XMSG_BAD_U16),
00860             ssi_(XMSG_BAD_U32),
00861             ssp_(XMSG_BAD_U16),
00862             xas_(XMSG_BAD_U08) {
00863         DecodeXMsgData(data); // Decode the message
00864     }
00865 #   pragma GCC diagnostic pop
00866
00867     /** Destructor.
00868      *
00869      */
00870     ~XAlamoMissionConfiguration() {
00871         /* Do nothing. */
00872     }
00873
00874     /** Decode the ALAMO mission configuration message from the binary message data.
00875      *
00876      *  @param[in]      data        The binary data to decode.
00877      *
00878      *  @throw          log::MessageException if the message is invalid or an error occurs while
00879      *                  decoding.
00880      */
00881     void DecodeXMsgData(const vector<uint8_t>& data) {
00882
00883         try {
00884             const uint8_t* d = data.data();
00885
00886             set_fw_version(u32(d, 4));
00887
00888             /* Mission Configuration */
00889
00890             switch(fw_version()) {
00891             case 0x02006200: // ALAMO Software version 0x02006200.
00892             case 0x02006300: // ALAMO Software version 0x02006300.
00893             case 0x02006700: // ALAMO Software version 0x02006700.
00894             case 0x02006800: // ALAMO Software version 0x02006800.
00895
00896             {
00897
00898                 if(data.size() != 323) {
00899                     ostringstream os;
00900                     os << "Invalid X-Message: Expected 323 bytes, but received "
00901                        << data.size()
00902                        << " bytes!";
00903                     EXC_THROW(os.str());
00904                 }
00905
00906                 set_msn(u16(d, 8));
00907                 set_dtd(u16(d, 10));
00908                 set_mds(u16(d, 12));
00909                 set_apt(u8(d, 14));
00910                 set_xas(u8(d, 15));
00911                 set_cps(u8(d, 16));
00912                 set_cst(u8(d, 17));
00913                 set_csp(u32(d, 18));
00914                 cpb_[0] = u16(d, 22);
00915                 cpb_[1] = u16(d, 24);
00916                 cpb_[2] = u16(d, 26);
00917                 cpn_[0] = u16(d, 28);
00918                 cpn_[1] = u16(d, 30);
00919                 cpn_[2] = u16(d, 32);
00920                 cpp_[0] = u16(d, 34);
```

```
00921                 cpp_[1] = u16(d, 36);
00922                 cpp_[2] = u16(d, 38);
00923                 set_mtd(u16(d, 40));
00924                 set_mtt(u32(d, 42));
00925                 set_mam(u8(d, 46));
00926                 set_mat(u16(d, 47));
00927                 set_aca(u16(d, 49));
00928                 set_acg(u16(d, 51));
00929                 set_acr(u16(d, 53));
00930                 set_pnb(u16(d, 55));
00931                 set_pnc(u16(d, 57));
00932                 set_pxc(u16(d, 59));
00933                 set_pbl(u16(d, 61));
00934                 set_pbh(u16(d, 63));
00935                 set_pp0(u16(d, 65));
00936                 set_pp1(u16(d, 67));
00937                 set_pvl(u16(d, 69));
00938                 set_pvh(u16(d, 71));
00939                 set_pv0(u16(d, 73));
00940                 set_pv5(u16(d, 75));
00941                 set_pc0(u16(d, 77));
00942                 set_pc1(u16(d, 79));
00943                 set_descend_start_time(s2000(d, 99));
00944                 set_dns(u16(d, 103));
00945                 set_dra(u16(d, 105));
00946                 set_dsd(u16(d, 107));
00947                 set_dst(u16(d, 109));
00948                 set_ddf(u16(d, 111));
00949                 set_ddc(u16(d, 113));
00950                 set_dsi(u32(d, 115));
00951                 set_ddd(u16(d, 119));
00952                 set_ddt(u16(d, 121));
00953                 set_dft(u16(d, 123));
00954                 set_dcm(u8(d, 125));
00955                 set_dal(u8(d, 126));
00956                 set_dct(u16(d, 127));
00957                 set_dca(u8(d, 129));
00958                 set_drc(u16(d, 130));
00959                 set_drt(u16(d, 132));
00960                 set_dcx(u16(d, 134));
00961                 set_dcp(u16(d, 136));
00962                 set_dcf(u16(d, 138));
00963                 set_dcd(u16(d, 140));
00964                 set_dcr(u16(d, 142));
00965                 set_dsw(u16(d, 144));
00966                 set_duc(u16(d, 146));
00967                 set_dpt(u16(d, 148));
00968                 set_dut(u16(d, 150));
00969                 set_dpd(u16(d, 152));
00970                 set_dvt(u16(d, 154));
00971                 set_pfr(u16(d, 156));
00972                 set_pfc(u16(d, 158));
00973                 set_ddp(u16(d, 160));
00974                 set_dds(u16(d, 162));
00975                 set_park_start_time(s2000(d, 179));
00976                 set_pns(u16(d, 183));
00977                 set_pto(u32(d, 185));
00978                 set_pth(u32(d, 189));
00979                 set_psa(u8(d, 193));
00980                 set_psi(u32(d, 194));
00981                 set_pfm(u8(d, 198));
00982                 set_pft(u16(d, 199));
00983                 set_pfd(u16(d, 201));
00984                 set_pfv(u16(d, 203));
00985                 set_pdt(u16(d, 205));
00986                 set_psm(u8(d, 207));
00987                 set_psd(u16(d, 208));
00988                 set_pst(u16(d, 210));
00989                 set_psv(u16(d, 212));
00990                 set_psw(u16(d, 214));
00991                 set_ascent_start_time(s2000(d, 227));
00992                 set_asn(u16(d, 231));
00993                 set_arr(u16(d, 233));
00994                 set_apm(u16(d, 235));
00995                 set_apn(u16(d, 237));
00996                 set_apf(u16(d, 239));
00997                 set_asi(u32(d, 241));
00998                 set_asd(u16(d, 245));
00999                 set_add(u16(d, 247));
01000                 set_adt(u16(d, 249));
01001                 set_arc(u16(d, 251));
01002                 set_ahr(u16(d, 253));
01003                 set_ahd(u16(d, 255));
01004                 set_surface_start_time(s2000(d, 271));
01005                 set_sfn(u16(d, 275));
01006                 set_sdc(i16(d, 277));
01007                 set_sgt(u16(d, 279));
01008                 set_set(u16(d, 281));
01009                 set_sbm(u8(d, 283));
01010                 set_sbt(u16(d, 284));
01011                 set_sdt(u32(d, 286));
01012                 set_ssa(u16(d, 290));
01013                 set_ssi(u32(d, 292));
01014                 set_sds(u16(d, 296));
01015                 set_sam(u8(d, 298));
01016                 set_sax(u16(d, 299));
01017                 set_san(u16(d, 301));
01018                 set_sao(u16(d, 303));
01019                 set_sap(u16(d, 305));
01020                 set_spt(u16(d, 307));
01021
01022             }
01023
01024                 break;
01025
01026             case 0x02006900: // ALAMO Software version 0x02006900.
01027             case 0x02036200: // ALAMO Software version 0x02036200.
01028             case 0x02046200: // ALAMO Software version 0x02046200.
```

```
01029             case 0x02056200: // ALAMO Software version 0x02056200.
01030
01031             {
01032
01033                 if(data.size() != 322) {
01034                     ostringstream os;
01035                     os « "Invalid X-Message: Expected 323 bytes, but received "
01036                        « data.size()
01037                        « " bytes!";
01038                     EXC_THROW(os.str());
01039                 }
01040
01041                 /* Config. - General: */
01042
01043                 set_msn(u16(d, 8));
01044                 set_dtd(u16(d, 10));
01045                 set_mds(u16(d, 12));
01046                 set_apt(u8(d, 14));
01047                 set_xas(u8(d, 15));
01048                 set_cps(u8(d, 16));
01049                 set_cst(u8(d, 17));
01050                 set_ctf(u16(d, 18));
01051                 set_ctl(u16(d, 20));
01052                 set_ctu(u16(d, 22));
01053
01054                 set_csp(u32(d, 18 + 6));
01055                 cpb_[0] = u16(d, 22 + 6);
01056                 cpb_[1] = u16(d, 24 + 6);
01057                 cpb_[2] = u16(d, 26 + 6);
01058                 cpn_[0] = u16(d, 28 + 6);
01059                 cpn_[1] = u16(d, 30 + 6);
01060                 cpn_[2] = u16(d, 32 + 6);
01061                 cpp_[0] = u16(d, 34 + 6);
01062                 cpp_[1] = u16(d, 36 + 6);
01063                 cpp_[2] = u16(d, 38 + 6);
01064                 set_mtd(u16(d, 40 + 6));
01065                 set_mtt(u32(d, 42 + 6));
01066                 set_mam(u8(d, 46 + 6));
01067                 set_mat(u16(d, 47 + 6));
01068                 set_aca(u16(d, 49 + 6));
01069                 set_acg(u16(d, 51 + 6));
01070                 set_acr(u16(d, 53 + 6));
01071                 set_pnb(u16(d, 55 + 6));
01072                 set_pnc(u16(d, 57 + 6));
01073                 set_pxc(u16(d, 59 + 6));
01074                 set_pbl(u16(d, 61 + 6));
01075                 set_pbh(u16(d, 63 + 6));
01076                 set_pp0(u16(d, 65 + 6));
01077                 set_pp1(u16(d, 67 + 6));
01078                 set_pvl(u16(d, 69 + 6));
01079                 set_pvh(u16(d, 71 + 6));
01080                 set_pv0(u16(d, 73 + 6));
01081                 set_pv5(u16(d, 75 + 6));
01082                 set_pc0(u16(d, 77 + 6));
01083                 set_pc1(u16(d, 79 + 6));
01084
01085                 set_icm(u8(d, 87));
01086                 set_imm(u16(d, 88));
01087                 set_ica0(u16(d, 90));
01088                 set_ica1(u16(d, 92));
01089                 set_icb0(u16(d, 94));
01090                 set_icb1(u16(d, 96));
01091                 set_imt0(u16(d, 98));
01092                 set_imt1(u16(d, 100));
01093                 set_ilt0(u16(d, 102));
01094                 set_ilt1(u16(d, 104));
01095                 set_icn(u16(d, 106));
01096                 set_icx(u16(d, 108));
01097                 set_ict(i16(d, 110));
01098                 set_ics(u16(d, 112));
01099                 set_iss0(u8(d, 114));
01100                 set_iss1(u8(d, 115));
01101                 set_ist(u16(d, 116));
01102                 set_iso(u16(d, 118));
01103                 set_isp(u8(d, 120));
01104                 set_iue0(u8(d, 121));
01105                 set_iue1(u8(d, 122));
01106                 set_icd0(u16(d, 123));
01107                 set_icd1(u16(d, 125));
01108                 set_icr0(u32(d, 127));
01109                 set_icr1(u32(d, 131));
01110                 set_icg0(u8(d, 135));
01111                 set_icg1(u8(d, 136));
01112                 set_ico0(u16(d, 137));
01113                 set_ico1(u16(d, 139));
01114                 set_icf(u8(d, 141));
01115                 set_icw(u8(d, 145));
01116
01117                 set_descend_start_time(s2000(d, 9 + 50));
01118                 set_dns(u16(d, 103 + 50));
01119                 set_dra(u16(d, 105 + 50));
01120                 set_dsd(u16(d, 107 + 50));
01121                 set_dst(u16(d, 109 + 50));
01122                 set_ddf(u16(d, 111 + 50));
01123                 set_ddc(u16(d, 113 + 50));
01124                 set_dsi(u32(d, 115 + 50));
01125                 set_ddd(u16(d, 119 + 50));
01126                 set_ddt(u16(d, 121));
01127                 set_dft(u16(d, 123 + 50));
01128                 set_dcm(u8(d, 125 + 50));
01129                 set_dal(u8(d, 126 + 50));
01130                 set_dct(u16(d, 127 + 50));
01131                 set_dca(u8(d, 129 + 50));
01132                 set_drc(u16(d, 130 + 50));
01133                 set_drt(u16(d, 132 + 50));
01134                 set_dcx(u16(d, 134 + 50));
01135                 set_dcp(u16(d, 136 + 50));
01136                 set_dcf(u16(d, 138 + 50));
```

```
01137                 set_dcd(u16(d, 140 + 50));
01138                 set_dcr(u16(d, 142 + 50));
01139                 set_dsw(u16(d, 144 + 50));
01140                 set_duc(u16(d, 146 + 50));
01141                 set_dpt(u16(d, 148 + 50));
01142                 set_dut(u16(d, 150 + 50));
01143                 set_dpd(u16(d, 152 + 50));
01144                 set_dvt(u16(d, 154 + 50));
01145                 set_pfr(u16(d, 156 + 50));
01146                 set_pfc(u16(d, 158 + 50));
01147                 set_ddp(u16(d, 160 + 50));
01148                 set_dds(u16(d, 162 + 50));
01149                 set_park_start_time(s2000(d, 179 + 35));
01150                 set_pns(u16(d, 183 + 35));
01151                 set_pto(u32(d, 185 + 35));
01152                 set_pth(u32(d, 189 + 35));
01153                 set_psa(u8(d, 193 + 35));
01154                 set_psi(u32(d, 194 + 35));
01155                 set_pfm(u8(d, 198 + 35));
01156                 set_pft(u16(d, 199 + 35));
01157                 set_pfd(u16(d, 201 + 35));
01158                 set_pfv(u16(d, 203 + 35));
01159                 set_pdt(u16(d, 205 + 35));
01160                 set_psm(u8(d, 207 + 35));
01161                 set_psd(u16(d, 208 + 35));
01162                 set_pst(u16(d, 210 + 35));
01163                 set_psv(u16(d, 212 + 35));
01164                 set_psw(u16(d, 214 + 35));
01165                 set_ascent_start_time(s2000(d, 227 + 24));
01166                 set_asn(u16(d, 231 + 24));
01167                 set_arr(u16(d, 233 + 24));
01168                 set_apm(u16(d, 235 + 24));
01169                 set_apn(u16(d, 237 + 24));
01170                 set_apf(u16(d, 239 + 24));
01171                 set_asi(u32(d, 241 + 24));
01172                 set_asd(u16(d, 245 + 24));
01173                 set_add(u16(d, 247 + 24));
01174                 set_adt(u16(d, 249 + 24));
01175                 set_arc(u16(d, 251 + 24));
01176                 set_ahr(u16(d, 253 + 24));
01177                 set_ahd(u16(d, 255 + 24));
01178                 set_surface_start_time(s2000(d, 271 + 10));
01179                 set_sfn(u16(d, 275 + 10));
01180                 set_sdc(i16(d, 277 + 10));
01181                 set_sgt(u16(d, 279 + 10));
01182                 set_set(u16(d, 281 + 10));
01183                 set_sbm(u8(d, 283 + 10));
01184                 set_sbt(u16(d, 284 + 10));
01185                 set_sdt(u32(d, 286 + 10));
01186                 set_ssa(u16(d, 290 + 10));
01187                 set_ssi(u32(d, 292 + 10));
01188                 set_sds(u16(d, 296 + 10));
01189                 set_sam(u8(d, 298 + 10));
01190                 set_sax(u16(d, 299 + 10));
01191                 set_san(u16(d, 301 + 10));
01192                 set_sao(u16(d, 303 + 10));
01193                 set_sap(u16(d, 305 + 10));
01194                 set_spt(u16(d, 307 + 10));
01195
01196                 /** @todo Implement ALAMO mission parameter: SSP.
01197                  *
01198                  */
01199 //              set_ssp(u16(d, 309 + 10));
01200             }
01201
01202                 break;
01203
01204             case 0x03006500: // ALAMO Software version 0x03006500.
01205
01206             {
01207
01208                 if(data.size() != 328) {
01209                     ostringstream os;
01210                     os « "Invalid X-Message: Expected 328 bytes, but received "
01211                        « data.size()
01212                        « " bytes!";
01213                     EXC_THROW(os.str());
01214                 }
01215
01216                 /* Config. - General: */
01217
01218                 set_msn(u16(d, 8));
01219                 set_dtd(u16(d, 10));
01220                 set_mds(u16(d, 12));
01221                 set_apt(u8(d, 14));
01222                 set_xas(u8(d, 15));
01223                 set_cps(u8(d, 16));
01224                 set_cst(u8(d, 17));
01225                 set_ctf(u16(d, 18));
01226                 set_ctl(u16(d, 20));
01227                 set_ctu(u16(d, 22));
01228
01229                 set_csp(u32(d, 18 + 6));
01230                 cpb_[0] = u16(d, 22 + 6);
01231                 cpb_[1] = u16(d, 24 + 6);
01232                 cpb_[2] = u16(d, 26 + 6);
01233                 cpn_[0] = u16(d, 28 + 6);
01234                 cpn_[1] = u16(d, 30 + 6);
01235                 cpn_[2] = u16(d, 32 + 6);
01236                 cpp_[0] = u16(d, 34 + 6);
01237                 cpp_[1] = u16(d, 36 + 6);
01238                 cpp_[2] = u16(d, 38 + 6);
01239                 set_mtd(u16(d, 40 + 6));
01240                 set_mtt(u32(d, 42 + 6));
01241                 set_mam(u8(d, 46 + 6));
01242                 set_mat(u16(d, 47 + 6));
01243                 set_aca(u16(d, 49 + 6));
01244                 set_acg(u16(d, 51 + 6));
```

```
01245                 set_acr(u16(d, 53 + 6));
01246                 set_pnb(u16(d, 55 + 6));
01247                 set_pnc(u16(d, 57 + 6));
01248                 set_pxc(u16(d, 59 + 6));
01249                 set_pbl(u16(d, 61 + 6));
01250                 set_pbh(u16(d, 63 + 6));
01251                 set_pp0(u16(d, 65 + 6));
01252                 set_ppl(u16(d, 67 + 6));
01253                 set_pvl(u16(d, 69 + 6));
01254                 set_pvh(u16(d, 71 + 6));
01255                 set_pv0(u16(d, 73 + 6));
01256                 set_pv5(u16(d, 75 + 6));
01257                 set_pc0(u16(d, 77 + 6));
01258                 set_pc1(u16(d, 79 + 6));
01259 
01260                 set_icm(u8(d, 87));
01261                 set_imm(u16(d, 88));
01262                 set_ica0(u16(d, 90));
01263                 set_ica1(u16(d, 92));
01264                 set_icb0(u16(d, 94));
01265                 set_icb1(u16(d, 96));
01266                 set_imt0(u16(d, 98));
01267                 set_imt1(u16(d, 100));
01268                 set_ilt0(u16(d, 102));
01269                 set_ilt1(u16(d, 104));
01270                 set_icn(u16(d, 106));
01271                 set_icx(u16(d, 108));
01272                 set_ict(i16(d, 110));
01273                 set_ics(u16(d, 112));
01274                 set_iss0(u8(d, 114));
01275                 set_iss1(u8(d, 115));
01276                 set_ist(u16(d, 116));
01277                 set_iso(u16(d, 118));
01278                 set_isp(u8(d, 120));
01279                 set_iue0(u8(d, 121));
01280                 set_iue1(u8(d, 122));
01281                 set_icd0(u16(d, 123));
01282                 set_icd1(u16(d, 125));
01283                 set_icr0(u32(d, 127));
01284                 set_icr1(u32(d, 131));
01285                 set_icg0(u8(d, 135));
01286                 set_icg1(u8(d, 136));
01287 
01288 
01289                 set_ico0(u16(d, 137 + 9));
01290                 set_ico1(u16(d, 139 + 9));
01291                 set_icf(u8(d, 141 + 9));
01292                 set_icw(u8(d, 145 + 9));
01293 
01294                 set_descend_start_time(s2000(d, 158));
01295                 set_dns(u16(d, 103 + 59));
01296                 set_dra(u16(d, 105 + 59));
01297                 set_dsd(u16(d, 107 + 59));
01298                 set_dst(u16(d, 109 + 59));
01299                 set_ddf(u16(d, 111 + 59));
01300                 set_ddc(u16(d, 113 + 59));
01301                 set_dsi(u32(d, 115 + 59));
01302                 set_ddd(u16(d, 119 + 59));
01303                 set_ddt(u16(d, 121));
01304                 set_dft(u16(d, 123 + 59));
01305                 set_dcm(u8(d, 125 + 59));
01306                 set_dal(u8(d, 126 + 59));
01307                 set_dct(u16(d, 127 + 59));
01308                 set_dca(u8(d, 129 + 59));
01309                 set_drc(u16(d, 130 + 59));
01310                 set_drt(u16(d, 132 + 59));
01311                 set_dcx(u16(d, 134 + 59));
01312                 set_dcp(u16(d, 136 + 59));
01313                 set_dcf(u16(d, 138 + 59));
01314                 set_dcd(u16(d, 140 + 59));
01315                 set_dcr(u16(d, 142 + 59));
01316                 set_dsw(u16(d, 144 + 59));
01317                 set_duc(u16(d, 146 + 59));
01318                 set_dpt(u16(d, 148 + 59));
01319                 set_dut(u16(d, 150 + 59));
01320                 set_dpd(u16(d, 152 + 59));
01321                 set_dvt(u16(d, 154 + 59));
01322                 set_pfr(u16(d, 156 + 59));
01323                 set_pfc(u16(d, 158 + 59));
01324                 set_ddp(u16(d, 160 + 59));
01325                 set_dds(u16(d, 162 + 59));
01326                 set_park_start_time(s2000(d, 179 + 44));
01327                 set_pns(u16(d, 183 + 44));
01328                 set_pto(u32(d, 185 + 44));
01329                 set_pth(u32(d, 189 + 44));
01330                 set_psa(u8(d, 193 + 44));
01331                 set_psi(u32(d, 194 + 44));
01332                 set_pfm(u8(d, 198 + 44));
01333                 set_pft(u16(d, 199 + 44));
01334                 set_pfd(u16(d, 201 + 44));
01335                 set_pfv(u16(d, 203 + 44));
01336                 set_pdt(u16(d, 205 + 44));
01337                 set_psm(u8(d, 207 + 44));
01338                 set_psd(u16(d, 208 + 44));
01339                 set_pst(u16(d, 210 + 44));
01340                 set_psv(u16(d, 212 + 44));
01341                 set_psw(u16(d, 214 + 44));
01342                 set_ascent_start_time(s2000(d, 227 + 33));
01343                 set_asn(u16(d, 231 + 33));
01344                 set_arr(u16(d, 233 + 33));
01345                 set_apm(u16(d, 235 + 33));
01346                 set_apn(u16(d, 237 + 33));
01347                 set_apf(u16(d, 239 + 33));
01348                 set_asi(u32(d, 241 + 33));
01349                 set_asd(u16(d, 245 + 33));
01350                 set_add(u16(d, 247 + 33));
01351                 set_adt(u16(d, 249 + 33));
01352                 set_arc(u16(d, 251 + 33));
```

```
01353                 set_ahr(u16(d, 253 + 33));
01354                 set_ahd(u16(d, 255 + 33));
01355                 set_surface_start_time(s2000(d, 271 + 19));
01356                 set_sfn(u16(d, 275 + 19));
01357                 set_sdc(i16(d, 277 + 19));
01358                 set_sgt(u16(d, 279 + 19));
01359                 set_set(u16(d, 281 + 19));
01360                 set_sbm(u8(d, 283 + 19));
01361                 set_sbt(u16(d, 284 + 19));
01362                 set_sdt(u32(d, 286 + 19));
01363                 set_ssa(u16(d, 290 + 19));
01364                 set_ssi(u32(d, 292 + 19));
01365                 set_sds(u16(d, 296 + 19));
01366                 set_sam(u8(d, 298 + 19));
01367                 set_sax(u16(d, 299 + 19));
01368                 set_san(u16(d, 301 + 19));
01369                 set_sao(u16(d, 303 + 19));
01370                 set_sap(u16(d, 305 + 19));
01371                 set_spt(XMSG_BAD_U16);
01372
01373                 /** @todo Implement ALAMO mission parameter: SSP.
01374                  *
01375                  */
01376 //                set_ssp(u16(d, 309 + 10));
01377             }
01378
01379                 break;
01380
01381             case 0x03016700: // ALAMO Software version 0x03016700 (Solo-II Hybrid #10083).
01382
01383                 if(data.size() != 328) {
01384                     ostringstream os;
01385                     os « "Invalid X-Message: Expected 328 bytes, but received "
01386                        « data.size()
01387                        « " bytes!";
01388                     EXC_THROW(os.str());
01389                 }
01390
01391                 /* Config. - General: */
01392
01393                 set_msn(u16(d, 8));
01394                 set_dtd(u16(d, 10));
01395                 set_mds(u16(d, 12));
01396                 set_apt(u8(d, 14));
01397                 set_xas(u8(d, 15));
01398                 set_cps(u8(d, 16));
01399                 set_cst(u8(d, 17));
01400                 set_ctf(u16(d, 18));
01401                 set_ctl(u16(d, 20));
01402                 set_ctu(u16(d, 22));
01403
01404                 set_csp(u32(d, 18 + 6));
01405                 cpb_[0] = u16(d, 22 + 6);
01406                 cpb_[1] = u16(d, 24 + 6);
01407                 cpb_[2] = u16(d, 26 + 6);
01408                 cpn_[0] = u16(d, 28 + 6);
01409                 cpn_[1] = u16(d, 30 + 6);
01410                 cpn_[2] = u16(d, 32 + 6);
01411                 cpp_[0] = u16(d, 34 + 6);
01412                 cpp_[1] = u16(d, 36 + 6);
01413                 cpp_[2] = u16(d, 38 + 6);
01414                 set_mtd(u16(d, 40 + 6));
01415                 set_mtt(u32(d, 42 + 6));
01416                 set_mam(u8(d, 46 + 6));
01417                 set_mat(u16(d, 47 + 6));
01418                 set_aca(u16(d, 49 + 6));
01419                 set_acg(u16(d, 51 + 6));
01420                 set_acr(u16(d, 53 + 6));
01421                 set_pnb(u16(d, 55 + 6));
01422                 set_pnc(u16(d, 57 + 6));
01423                 set_pxc(u16(d, 59 + 6));
01424                 set_pbl(u16(d, 61 + 6));
01425                 set_pbh(u16(d, 63 + 6));
01426                 set_pp0(u16(d, 65 + 6));
01427                 set_ppl(u16(d, 67 + 6));
01428                 set_pvl(u16(d, 69 + 6));
01429                 set_pvh(u16(d, 71 + 6));
01430                 set_pv0(u16(d, 73 + 6));
01431                 set_pv5(u16(d, 75 + 6));
01432                 set_pc0(u16(d, 77 + 6));
01433                 set_pc1(u16(d, 79 + 6));
01434
01435                 set_icm(u8(d, 87));
01436                 set_imm(u16(d, 88));
01437                 set_ica0(u16(d, 90));
01438                 set_ica1(u16(d, 92));
01439                 set_icb0(u16(d, 94));
01440                 set_icb1(u16(d, 96));
01441                 set_imt0(u16(d, 98));
01442                 set_imt1(u16(d, 100));
01443                 set_ilt0(u16(d, 102));
01444                 set_ilt1(u16(d, 104));
01445                 set_icn(u16(d, 106));
01446                 set_icx(u16(d, 108));
01447                 set_ict(i16(d, 110));
01448                 set_ics(u16(d, 112));
01449                 set_iss0(u8(d, 114));
01450                 set_iss1(u8(d, 115));
01451                 set_ist(u16(d, 116 + 1));
01452                 set_iso(u16(d, 118 + 1));
01453                 set_isp(u8(d, 120 + 1));
01454                 set_iue0(u8(d, 121 + 1));
01455                 set_iue1(u8(d, 122 + 1));
01456                 set_icd0(u16(d, 123 + 1));
01457                 set_icd1(u16(d, 125 + 1));
01458                 set_icr0(u32(d, 127 + 1));
01459                 set_icr1(u32(d, 131 + 1));
01460
```

```
01461                 set_icg0(u8(d, 135 + 1));
01462                 set_icg1(u8(d, 136 + 1));
01463                 set_ico0(u16(d, 137 + 10));
01464                 set_ico1(u16(d, 139 + 10));
01465                 set_icf(u8(d, 141 + 10));
01466                 set_icw(u8(d, 145 + 10));
01467 
01468                 set_descend_start_time(s2000(d, 159));
01469                 set_dns(u16(d, 103 + 60));
01470                 set_dra(u16(d, 105 + 60));
01471                 set_dsd(u16(d, 107 + 60));
01472                 set_dst(u16(d, 109 + 60));
01473                 set_ddf(u16(d, 111 + 60));
01474                 set_ddc(u16(d, 113 + 60));
01475                 set_dsi(u32(d, 115 + 60));
01476                 set_ddd(u16(d, 119 + 60));
01477                 set_ddt(u16(d, 121));
01478                 set_dft(u16(d, 123 + 60));
01479                 set_dcm(u8(d, 125 + 60));
01480                 set_dal(u8(d, 126 + 60));
01481                 set_dct(u16(d, 127 + 60));
01482                 set_dca(u8(d, 129 + 60));
01483                 set_drc(u16(d, 130 + 60));
01484                 set_drt(u16(d, 132 + 60));
01485                 set_dcx(u16(d, 134 + 60));
01486                 set_dcp(u16(d, 136 + 60));
01487                 set_dcf(u16(d, 138 + 60));
01488                 set_dcd(u16(d, 140 + 60));
01489                 set_dcr(u16(d, 142 + 60));
01490                 set_dsw(u16(d, 144 + 60));
01491                 set_duc(u16(d, 146 + 60));
01492                 set_dpt(u16(d, 148 + 60));
01493                 set_dut(u16(d, 150 + 60));
01494                 set_dpd(u16(d, 152 + 60));
01495                 set_dvt(u16(d, 154 + 60));
01496                 set_pfr(u16(d, 156 + 60));
01497                 set_pfc(u16(d, 158 + 60));
01498                 set_ddp(u16(d, 160 + 60));
01499                 set_dds(u16(d, 162 + 60));
01500                 set_park_start_time(s2000(d, 179 + 45));
01501                 set_pns(u16(d, 183 + 45));
01502                 set_pto(u32(d, 185 + 45));
01503                 set_pth(u32(d, 189 + 45));
01504                 set_psa(u8(d, 193 + 45));
01505                 set_psi(u32(d, 194 + 45));
01506                 set_pfm(u8(d, 198 + 45));
01507                 set_pft(u16(d, 199 + 45));
01508                 set_pfd(u16(d, 201 + 45));
01509                 set_pfv(u16(d, 203 + 45));
01510                 set_pdt(u16(d, 205 + 45));
01511                 set_psm(u8(d, 207 + 45));
01512                 set_psd(u16(d, 208 + 45));
01513                 set_pst(u16(d, 210 + 45));
01514                 set_psv(u16(d, 212 + 45));
01515                 set_psw(u16(d, 214 + 45));
01516                 set_ascent_start_time(s2000(d, 227 + 34));
01517                 set_asn(u16(d, 231 + 34));
01518                 set_arr(u16(d, 233 + 34));
01519                 set_apm(u16(d, 235 + 34));
01520                 set_apn(u16(d, 237 + 34));
01521                 set_apf(u16(d, 239 + 34));
01522                 set_asi(u32(d, 241 + 34));
01523                 set_asd(u16(d, 245 + 34));
01524                 set_add(u16(d, 247 + 34));
01525                 set_adt(u16(d, 249 + 34));
01526                 set_arc(u16(d, 251 + 34));
01527                 set_ahr(u16(d, 253 + 34));
01528                 set_ahd(u16(d, 255 + 34));
01529                 set_surface_start_time(s2000(d, 271 + 20));
01530                 set_sfn(u16(d, 275 + 20));
01531                 set_sdc(i16(d, 277 + 20));
01532                 set_sgt(u16(d, 279 + 20));
01533                 set_set(u16(d, 281 + 20));
01534                 set_sbm(u8(d, 283 + 20));
01535                 set_sbt(u16(d, 284 + 20));
01536                 set_sdt(u32(d, 286 + 20));
01537                 set_ssa(u16(d, 290 + 20));
01538                 set_ssi(u32(d, 292 + 20));
01539                 set_sds(u16(d, 296 + 20));
01540                 set_sam(u8(d, 298 + 20));
01541                 set_sax(u16(d, 299 + 20));
01542                 set_san(u16(d, 301 + 20));
01543                 set_sao(u16(d, 303 + 20));
01544                 set_sap(u16(d, 305 + 20));
01545 
01546                 // TODO: Implement additional parameters.
01547                 set_spt(XMSG_BAD_U16);
01548 
01549                 break;
01550 
01551             case 0x03026100: // ALAMO Software version 0x03026100 (2016 NASCar floats).
01552 
01553                 if(data.size() != 328) {
01554                     ostringstream os;
01555                     os « "Invalid X-Message: Expected 328 bytes, but received "
01556                        « data.size()
01557                        « " bytes!";
01558                     EXC_THROW(os.str());
01559                 }
01560 
01561                 /* Config. - General: */
01562 
01563                 set_msn(u16(d, 8));
01564                 set_dtd(u16(d, 10));
01565                 set_mds(u16(d, 12));
01566                 set_apt(u8(d, 14));
01567                 set_btm(u8(d, 15));
01568                 set_xas(u8(d, 15 + 1));
```

```
01569                set_cps(u8(d, 16 + 1));
01570                set_cst(u8(d, 17 + 1));
01571                set_ctf(u16(d, 18 + 1));
01572                set_ctl(u16(d, 20 + 1));
01573                set_ctu(u16(d, 22 + 1));
01574
01575                set_csp(u32(d, 18 + 6 + 1));
01576                cpb_[0] = u16(d, 22 + 7);
01577                cpb_[1] = u16(d, 24 + 7);
01578                cpb_[2] = u16(d, 26 + 7);
01579                cpn_[0] = u16(d, 28 + 7);
01580                cpn_[1] = u16(d, 30 + 7);
01581                cpn_[2] = u16(d, 32 + 7);
01582                cpp_[0] = u16(d, 34 + 7);
01583                cpp_[1] = u16(d, 36 + 7);
01584                cpp_[2] = u16(d, 38 + 7);
01585                set_mtd(u16(d, 40 + 6 + 1));
01586                set_mtt(u32(d, 42 + 6 + 1));
01587                set_mam(u8(d, 46 + 6 + 1));
01588                set_mat(u16(d, 47 + 6 + 1));
01589                set_aca(u16(d, 49 + 6 + 1));
01590                set_acg(u16(d, 51 + 6 + 1));
01591                set_acr(u16(d, 53 + 6 + 1));
01592                set_pnb(u16(d, 55 + 6 + 1));
01593                set_pnc(u16(d, 57 + 6 + 1));
01594                set_pxc(u16(d, 59 + 6 + 1));
01595                set_pbl(u16(d, 61 + 6 + 1));
01596                set_pbh(u16(d, 63 + 6 + 1));
01597                set_pp0(u16(d, 65 + 6 + 1));
01598                set_pp1(u16(d, 67 + 6 + 1));
01599                set_pv1(u16(d, 69 + 6 + 1));
01600                set_pvh(u16(d, 71 + 6 + 1));
01601                set_pv0(u16(d, 73 + 6 + 1));
01602                set_pv5(u16(d, 75 + 6 + 1));
01603                set_pc0(u16(d, 77 + 6 + 1));
01604                set_pc1(u16(d, 79 + 6 + 1));
01605
01606                set_icm(u8(d, 87 + 1));
01607                set_imm(u16(d, 88 + 1));
01608                set_ica0(u16(d, 90 + 1));
01609                set_ica1(u16(d, 92 + 1));
01610                set_icb0(u16(d, 94 + 1));
01611                set_icb1(u16(d, 96 + 1));
01612                set_imt0(u16(d, 98 + 1));
01613                set_imt1(u16(d, 100 + 1));
01614                set_ilt0(u16(d, 102 + 1));
01615                set_ilt1(u16(d, 104 + 1));
01616                set_icn(u16(d, 106 + 1));
01617                set_icx(u16(d, 108 + 1));
01618                set_ict(i16(d, 110 + 1));
01619                set_ics(u16(d, 112 + 1));
01620                set_iss0(u8(d, 114 + 1));
01621                set_iss1(u8(d, 115 + 1));
01622                set_ist(u16(d, 116 + 1 + 1));
01623                set_iso(u16(d, 118 + 1 + 1));
01624                set_isp(u8(d, 120 + 1 + 1));
01625                set_iue0(u8(d, 121 + 1 + 1));
01626                set_iue1(u8(d, 122 + 1 + 1));
01627                set_icd0(u16(d, 123 + 1 + 1));
01628                set_icd1(u16(d, 125 + 1 + 1));
01629                set_icr0(u32(d, 127 + 1 + 1));
01630                set_icr1(u32(d, 131 + 1 + 1));
01631
01632                set_icg0(u8(d, 135 + 1 + 1));
01633                set_icg1(u8(d, 136 + 1 + 1));
01634                set_ico0(u16(d, 137 + 10 + 1));
01635                set_ico1(u16(d, 139 + 10 + 1));
01636                set_icf(u8(d, 141 + 10 + 1));
01637                set_icw(u8(d, 145 + 10 + 1));
01638
01639                set_descend_start_time(s2000(d, 160));
01640                set_dns(u16(d, 103 + 61));
01641                set_dra(u16(d, 105 + 61));
01642                set_dsd(u16(d, 107 + 61));
01643                set_dst(u16(d, 109 + 61));
01644                set_ddf(u16(d, 111 + 61));
01645                set_ddc(u16(d, 113 + 61));
01646                set_dsi(u32(d, 115 + 61));
01647                set_ddd(u16(d, 119 + 61));
01648                set_ddt(u16(d, 121));
01649                set_dft(u16(d, 123 + 61));
01650                set_dcm(u8(d, 125 + 61));
01651                set_dal(u8(d, 126 + 61));
01652                set_dct(u16(d, 127 + 61));
01653                set_dca(u8(d, 129 + 61));
01654                set_drc(u16(d, 130 + 61));
01655                set_drt(u16(d, 132 + 61));
01656                set_dcx(u16(d, 134 + 61));
01657                set_dcp(u16(d, 136 + 61));
01658                set_dcf(u16(d, 138 + 61));
01659                set_dcd(u16(d, 140 + 61));
01660                set_dcr(u16(d, 142 + 61));
01661                set_dsw(u16(d, 144 + 61));
01662                set_duc(u16(d, 146 + 61));
01663                set_dpt(u16(d, 148 + 61));
01664                set_dut(u16(d, 150 + 61));
01665                set_dpd(u16(d, 152 + 61));
01666                set_dvt(u16(d, 154 + 61));
01667                set_pfr(u16(d, 156 + 61));
01668                set_pfc(u16(d, 158 + 61));
01669                set_ddp(u16(d, 160 + 61));
01670                set_dds(u16(d, 162 + 61));
01671                set_park_start_time(s2000(d, 179 + 46));
01672                set_pns(u16(d, 183 + 46));
01673                set_pto(u32(d, 185 + 46));
01674                set_pth(u32(d, 189 + 46));
01675                set_psa(u8(d, 193 + 46));
01676                set_psi(u32(d, 194 + 46));
```

```
01677                 set_pfm(u8(d, 198 + 46));
01678                 set_pft(u16(d, 199 + 46));
01679                 set_pfd(u16(d, 201 + 46));
01680                 set_pfv(u16(d, 203 + 46));
01681                 set_pdt(u16(d, 205 + 46));
01682                 set_psm(u8(d, 207 + 46));
01683                 set_psd(u16(d, 208 + 46));
01684                 set_pst(u16(d, 210 + 46));
01685                 set_psv(u16(d, 212 + 46));
01686                 set_psw(u16(d, 214 + 46));
01687                 set_ascent_start_time(s2000(d, 227 + 35));
01688                 set_asn(u16(d, 231 + 35));
01689                 set_arr(u16(d, 233 + 35));
01690                 set_apm(u16(d, 235 + 35));
01691                 set_apn(u16(d, 237 + 35));
01692                 set_apf(u16(d, 239 + 35));
01693                 set_asi(u32(d, 241 + 35));
01694                 set_asd(u16(d, 245 + 35));
01695                 set_add(u16(d, 247 + 35));
01696                 set_adt(u16(d, 249 + 35));
01697                 set_arc(u16(d, 251 + 35));
01698                 set_ahr(u16(d, 253 + 35));
01699                 set_ahd(u16(d, 255 + 35));
01700                 set_surface_start_time(s2000(d, 271 + 21));
01701                 set_sfn(u16(d, 275 + 21));
01702                 set_sdc(i16(d, 277 + 21));
01703                 set_sgt(u16(d, 279 + 21));
01704                 set_set(u16(d, 281 + 21));
01705                 set_sbm(u8(d, 283 + 21));
01706                 set_sbt(u16(d, 284 + 21));
01707                 set_sdt(u32(d, 286 + 21));
01708                 set_ssa(u16(d, 290 + 21));
01709                 set_ssi(u32(d, 292 + 21));
01710                 set_sds(u16(d, 296 + 21));
01711                 set_sam(u8(d, 298 + 21));
01712                 set_sax(u16(d, 299 + 21));
01713                 set_san(u16(d, 301 + 21));
01714                 set_sao(u16(d, 303 + 21));
01715                 set_sap(XMSG_BAD_U16);
01716
01717                 // TODO: Implement additional parameters.
01718                 set_spt(XMSG_BAD_U16);
01719
01720                 break;
01721
01722             case 0x04016100: // ALAMO Software version 0x04016100 (2016 NASCar floats).
01723
01724                 if(data.size() != 328) {
01725                     ostringstream os;
01726                     os « "Invalid X-Message: Expected 328 bytes, but received "
01727                        « data.size()
01728                        « " bytes!";
01729                     EXC_THROW(os.str());
01730                 }
01731
01732                 /* Config. - General: */
01733
01734                 set_msn(u16(d, 8));
01735                 set_dtd(u16(d, 10));
01736                 set_mds(u16(d, 12));
01737                 set_apt(u8(d, 14));
01738                 set_btm(u8(d, 15));
01739                 set_xas(u8(d, 15 + 1));
01740                 set_cps(u8(d, 16 + 1));
01741                 set_cst(u8(d, 17 + 1));
01742                 set_ctf(u16(d, 18 + 1));
01743                 set_ctl(u16(d, 20 + 1));
01744                 set_ctu(u16(d, 22 + 1));
01745
01746                 set_csp(u32(d, 18 + 6 + 1));
01747                 cpb_[0] = u16(d, 22 + 7);
01748                 cpb_[1] = u16(d, 24 + 7);
01749                 cpb_[2] = u16(d, 26 + 7);
01750                 cpn_[0] = u16(d, 28 + 7);
01751                 cpn_[1] = u16(d, 30 + 7);
01752                 cpn_[2] = u16(d, 32 + 7);
01753                 cpp_[0] = u16(d, 34 + 7);
01754                 cpp_[1] = u16(d, 36 + 7);
01755                 cpp_[2] = u16(d, 38 + 7);
01756                 set_mtd(u16(d, 40 + 6 + 1));
01757                 set_mtt(u32(d, 42 + 6 + 1));
01758                 set_mam(u8(d, 46 + 6 + 1));
01759                 set_mat(u16(d, 47 + 6 + 1));
01760                 set_aca(u16(d, 49 + 6 + 1));
01761                 set_acg(u16(d, 51 + 6 + 1));
01762                 set_acr(u16(d, 53 + 6 + 1));
01763                 set_pnb(u16(d, 55 + 6 + 1));
01764                 set_pnc(u16(d, 57 + 6 + 1));
01765                 set_pxc(u16(d, 59 + 6 + 1));
01766                 set_pbl(u16(d, 61 + 6 + 1));
01767                 set_pbh(u16(d, 63 + 6 + 1));
01768                 set_pp0(u16(d, 65 + 6 + 1));
01769                 set_pp1(u16(d, 67 + 6 + 1));
01770                 set_pvl(u16(d, 69 + 6 + 1));
01771                 set_pvh(u16(d, 71 + 6 + 1));
01772                 set_pv0(u16(d, 73 + 6 + 1));
01773                 set_pv5(u16(d, 75 + 6 + 1));
01774                 set_pc0(u16(d, 77 + 6 + 1));
01775                 set_pc1(u16(d, 79 + 6 + 1));
01776
01777                 set_icm(u8(d, 87 + 1));
01778                 set_imm(u16(d, 88 + 1));
01779                 set_ica0(u16(d, 90 + 1));
01780                 set_ica1(u16(d, 92 + 1));
01781                 set_icb0(u16(d, 94 + 1));
01782                 set_icb1(u16(d, 96 + 1));
01783                 set_imt0(u16(d, 98 + 1));
01784                 set_imt1(u16(d, 100 + 1));
```

```
01785                 set_ilt0(u16(d, 102 + 1));
01786                 set_ilt1(u16(d, 104 + 1));
01787                 set_icn(u16(d, 106 + 1));
01788                 set_icx(u16(d, 108 + 1));
01789                 set_ict(i16(d, 110 + 1));
01790                 set_ics(u16(d, 112 + 1));
01791                 set_iss0(u8(d, 114 + 1));
01792                 set_iss1(u8(d, 115 + 1));
01793                 set_ist(u16(d, 116 + 1 + 1));
01794                 set_iso(u16(d, 118 + 1 + 1));
01795                 set_isp(u8(d, 120 + 1 + 1));
01796                 set_iue0(u8(d, 121 + 1 + 1));
01797                 set_iue1(u8(d, 122 + 1 + 1));
01798                 set_icd0(u16(d, 123 + 1 + 1));
01799                 set_icd1(u16(d, 125 + 1 + 1));
01800                 set_icr0(u32(d, 127 + 1 + 1));
01801                 set_icr1(u32(d, 131 + 1 + 1));
01802
01803                 set_icg0(u8(d, 135 + 1 + 1));
01804                 set_icg1(u8(d, 136 + 1 + 1));
01805                 set_ico0(u16(d, 137 + 10 + 1));
01806                 set_ico1(u16(d, 139 + 10 + 1));
01807                 set_icf(u8(d, 141 + 10 + 1));
01808                 set_icw(u8(d, 145 + 10 + 1));
01809
01810                 set_descend_start_time(s2000(d, 160));
01811                 set_dns(u16(d, 103 + 61));
01812                 set_dra(u16(d, 105 + 61));
01813                 set_dsd(u16(d, 107 + 61));
01814                 set_dst(u16(d, 109 + 61));
01815                 set_ddf(u16(d, 111 + 61));
01816                 set_ddc(u16(d, 113 + 61));
01817                 set_dsi(u32(d, 115 + 61));
01818                 set_ddd(u16(d, 119 + 61));
01819                 set_ddt(u16(d, 121));
01820                 set_dft(u16(d, 123 + 61));
01821                 set_dcm(u8(d, 125 + 61));
01822                 set_dal(u8(d, 126 + 61));
01823                 set_dct(u16(d, 127 + 61));
01824                 set_dca(u8(d, 129 + 61));
01825                 set_drc(u16(d, 130 + 61));
01826                 set_drt(u16(d, 132 + 61));
01827                 set_dcx(u16(d, 134 + 61));
01828                 set_dcp(u16(d, 136 + 61));
01829                 set_dcf(u16(d, 138 + 61));
01830                 set_dcd(u16(d, 140 + 61));
01831                 set_dcr(u16(d, 142 + 61));
01832                 set_dsw(u16(d, 144 + 61));
01833                 set_duc(u16(d, 146 + 61));
01834                 set_dpt(u16(d, 148 + 61));
01835                 set_dut(u16(d, 150 + 61));
01836                 set_dpd(u16(d, 152 + 61));
01837                 set_dvt(u16(d, 154 + 61));
01838                 set_pfr(u16(d, 156 + 61));
01839                 set_pfc(u16(d, 158 + 61));
01840                 set_ddp(u16(d, 160 + 61));
01841                 set_dds(u16(d, 162 + 61));
01842                 set_park_start_time(s2000(d, 179 + 46));
01843                 set_pns(u16(d, 183 + 46));
01844                 set_pto(u32(d, 185 + 46));
01845                 set_pth(u32(d, 189 + 46));
01846                 set_psa(u8(d, 193 + 46));
01847                 set_psi(u32(d, 194 + 46));
01848                 set_pfm(u8(d, 198 + 46));
01849                 set_pft(u16(d, 199 + 46));
01850                 set_pfd(u16(d, 201 + 46));
01851                 set_pfv(u16(d, 203 + 46));
01852                 set_pdt(u16(d, 205 + 46));
01853                 set_psm(u8(d, 207 + 46));
01854                 set_psd(u16(d, 208 + 46));
01855                 set_pst(u16(d, 210 + 46));
01856                 set_psv(u16(d, 212 + 46));
01857                 set_psw(u16(d, 214 + 46));
01858                 set_ascent_start_time(s2000(d, 227 + 35));
01859                 set_asn(u16(d, 231 + 35));
01860                 set_arr(u16(d, 233 + 35));
01861                 set_apm(u16(d, 235 + 35));
01862                 set_apn(u16(d, 237 + 35));
01863                 set_apf(u16(d, 239 + 35));
01864                 set_asi(u32(d, 241 + 35));
01865                 set_asd(u16(d, 245 + 35));
01866                 set_add(u16(d, 247 + 35));
01867                 set_adt(u16(d, 249 + 35));
01868                 set_arc(u16(d, 251 + 35));
01869                 set_ahr(u16(d, 253 + 35));
01870                 set_ahd(u16(d, 255 + 35));
01871                 set_surface_start_time(s2000(d, 271 + 21));
01872                 set_sfn(u16(d, 275 + 21));
01873                 set_sdc(i16(d, 277 + 21));
01874                 set_sgt(u16(d, 279 + 21));
01875                 set_set(u16(d, 281 + 21));
01876                 set_sbm(u8(d, 283 + 21));
01877                 set_sbt(u16(d, 284 + 21));
01878                 set_sdt(u32(d, 286 + 21));
01879                 set_ssa(u16(d, 290 + 21));
01880                 set_ssi(u32(d, 292 + 21));
01881                 set_sds(u16(d, 296 + 21));
01882                 set_sam(u8(d, 298 + 21));
01883                 set_sax(u16(d, 299 + 21));
01884                 set_san(u16(d, 301 + 21));
01885                 set_sao(u16(d, 303 + 21));
01886                 set_sap(XMSG_BAD_U16);
01887
01888                 // TODO: Implement additional parameters.
01889                 set_spt(XMSG_BAD_U16);
01890
01891                 break;
01892
```

```
01893             case 0x04026200: // PAR floats
01894
01895
01896                 if(data.size() != 323) {
01897                     ostringstream os;
01898                     os « "Invalid X-Message: Expected 323 bytes, but received "
01899                        « data.size()
01900                        « " bytes!";
01901 #                   if defined(XMSG_DEBUG_ALAMO_MISS_GFG) && XMSG_DEBUG_ALAMO_MISS_GFG
01902                     {
01903                         lwrn() « os.str();
01904                     }
01905 #                   else
01906                     {
01907                         EXC_THROW(os.str());
01908                     }
01909 #                   endif // defined(XMSG_DEBUG_ALAMO_MISS_GFG) && XMSG_DEBUG_ALAMO_MISS_GFG
01910                 }
01911
01912                 /* Config. - General: */
01913
01914                 set_msg_version(u8(d, 8));
01915                 set_sensor_type(u8(d, 9));
01916                 set_msn(u16(d, 10));
01917                 set_dtd(u16(d, 12));
01918                 set_mds(u16(d, 14));
01919                 set_apt(u8(d, 16));
01920                 set_btm(u8(d, 17));
01921                 set_xas(u8(d, 18));
01922                 set_cps(u8(d, 19));
01923                 set_cst(u8(d, 20));
01924                 set_ctf(u16(d, 21));
01925                 set_ctl(u16(d, 23));
01926                 set_ctu(u16(d, 25));
01927
01928                 set_csp(u32(d, 27));
01929                 cpb_[0] = u16(d, 31);
01930                 cpb_[1] = u16(d, 33);
01931                 cpb_[2] = u16(d, 35);
01932                 cpn_[0] = u16(d, 37);
01933                 cpn_[1] = u16(d, 39);
01934                 cpn_[2] = u16(d, 41);
01935                 cpp_[0] = u16(d, 43);
01936                 cpp_[1] = u16(d, 45);
01937                 cpp_[2] = u16(d, 47);
01938                 set_mtd(u16(d, 49));
01939                 set_mtt(u32(d, 51));
01940                 set_mam(u8(d, 55));
01941                 set_mat(u16(d, 56));
01942                 set_aca(u16(d, 58));
01943                 set_acg(u16(d, 60));
01944                 set_acr(u16(d, 62));
01945                 set_pnb(u16(d, 64));
01946                 set_pnc(u16(d, 66));
01947                 set_pxc(u16(d, 68));
01948                 set_pbl(u16(d, 70));
01949                 set_pbh(u16(d, 72));
01950                 set_pp0(u16(d, 74));
01951                 set_pp1(u16(d, 76));
01952                 set_pvl(u16(d, 78));
01953                 set_pvh(u16(d, 80));
01954                 set_pv0(u16(d, 82));
01955                 set_pv5(u16(d, 84));
01956                 set_pc0(u16(d, 86));
01957                 set_pc1(u16(d, 88));
01958
01959                 set_icm(u8(d, 90));
01960                 set_imm(u16(d, 91));
01961                 set_ica0(u16(d, 93));
01962                 set_ica1(u16(d, 95));
01963                 set_icb0(u16(d, 97));
01964                 set_icb1(u16(d, 99));
01965                 set_imt0(u16(d, 101));
01966                 set_imt1(u16(d, 103));
01967                 set_ilt0(u16(d, 105));
01968                 set_ilt1(u16(d, 107));
01969                 set_icn(u16(d, 109));
01970                 set_icx(u16(d, 111));
01971                 set_ict(i16(d, 113));
01972                 set_ics(u16(d, 115));
01973                 set_iss0(u8(d, 117));
01974                 set_iss1(u8(d, 118));
01975
01976
01977                 set_ist(u16(d, 119+ 1));
01978                 set_iso(u16(d, 121+ 1));
01979                 set_isp(u8(d, 123+ 1));
01980                 set_iue0(u8(d, 124+ 1));
01981                 set_iue1(u8(d, 125+ 1));
01982                 set_icd0(u16(d, 126+ 1));
01983                 set_icd1(u16(d, 128+ 1));
01984                 set_icr0(u32(d, 130+ 1));
01985                 set_icr1(u32(d, 134+ 1));
01986                 set_icg0(u8(d, 138+ 1));
01987                 set_icg1(u8(d, 139+ 1));
01988                 set_isa(u8(d, 140+ 1));
01989                 set_isd(u16(d, 141+ 1));
01990                 set_iat(u16(d, 143+ 1));
01991                 set_isv(u16(d, 145+ 1));
01992                 set_isw(u16(d, 147+ 1));
01993                 set_ico0(u16(d, 149+ 1));
01994                 set_ico1(u16(d, 151+ 1));
01995                 set_icf(u32(d, 153+ 1));
01996                 set_icw(u32(d, 157+ 1));
01997
01998                 /* Descent state configuration: */
01999
02000                 set_descend_start_time(bpt::not_a_date_time);
```

```
02001                 set_dns(u16(d, 161+ 1));
02002                 set_dra(u16(d, 163+ 1));
02003                 set_dsd(u16(d, 165+ 1));
02004                 set_dst(u16(d, 167+ 1));
02005                 set_ddf(u16(d, 169+ 1));
02006                 set_ddc(u16(d, 171+ 1));
02007                 set_dsi(u32(d, 173+ 1));
02008                 set_ddd(u16(d, 177+ 1));
02009                 set_ddt(u16(d, 179+ 1));
02010                 set_dft(u16(d, 181+ 1));
02011                 set_dcm(u8(d, 183+ 1));
02012                 set_dal(u8(d, 184+ 1));
02013                 set_dct(u16(d, 185+ 1));
02014                 set_dca(u8(d, 187+ 1));
02015                 set_drc(u16(d, 188+ 1));
02016                 set_drt(u16(d, 190+ 1));
02017                 set_dcx(u16(d, 192+ 1));
02018                 set_dcp(u16(d, 194+ 1));
02019                 set_dcf(u16(d, 196+ 1));
02020                 set_dcd(u16(d, 198+ 1));
02021                 set_dcr(u16(d, 200+ 1));
02022                 set_dsw(u16(d, 202+ 1));
02023                 set_duc(u16(d, 204+ 1));
02024                 set_dpt(u16(d, 206+ 1));
02025                 set_dut(u16(d, 208+ 1));
02026                 set_dpd(u16(d, 210+ 1));
02027                 set_dvt(u16(d, 212+ 1));
02028
02029                 set_dvd(u16(d, 214+ 1));
02030                 set_dvw(u16(d, 216+ 1));
02031
02032
02033                 set_pfr(u16(d, 218+ 1));
02034                 set_pfc(u16(d, 220+ 1));
02035                 set_ddp(u16(d, 222+ 1));
02036                 set_dds(u16(d, 224+ 1));
02037                 set_park_start_time(bpt::not_a_date_time);
02038                 set_pns(u16(d, 226+ 1));
02039                 set_pto(u32(d, 228+ 1));
02040                 set_pth(u32(d, 232+ 1));
02041                 set_psa(u8(d, 236+ 1));
02042                 set_psi(u32(d, 237+ 1));
02043                 set_pfm(u8(d, 241+ 1));
02044                 set_pft(u16(d, 242+ 1));
02045                 set_pfd(u16(d, 244+ 1));
02046                 set_pfv(u16(d, 246+ 1));
02047                 set_pdt(u16(d, 248+ 1));
02048                 set_psm(u8(d, 250+ 1));
02049                 set_psd(u16(d, 251+ 1));
02050                 set_pst(u16(d, 253+ 1));
02051                 set_psv(u16(d, 255+ 1));
02052                 set_psw(u16(d, 257+ 1));
02053                 set_ascent_start_time(bpt::not_a_date_time);
02054                 set_asn(u16(d, 259+ 1));
02055                 set_arr(u16(d, 261+ 1));
02056                 set_apm(u16(d, 263+ 1));
02057                 set_apn(u16(d, 265+ 1));
02058                 set_apf(u16(d, 267+ 1));
02059                 set_asi(u32(d, 269+ 1));
02060                 set_asd(u16(d, 273+ 1));
02061                 set_add(u16(d, 275+ 1));
02062                 set_adt(u16(d, 277+ 1));
02063                 set_arc(u16(d, 279+ 1));
02064                 set_ahr(u16(d, 281+ 1));
02065                 set_ahd(u16(d, 283+ 1));
02066                 set_surface_start_time(bpt::not_a_date_time);
02067                 set_sfn(u16(d, 285+ 1));
02068                 set_sdc(i16(d, 287+ 1));
02069                 set_sgt(u16(d, 289+ 1));
02070                 set_set(u16(d, 291+ 1));
02071                 set_sbm(u8(d, 293+ 1));
02072                 set_sbt(u16(d, 294+ 1));
02073                 set_sdt(u32(d, 296+ 1));
02074                 set_ssa(u16(d, 300+ 1));
02075                 set_ssi(u32(d, 302+ 1));
02076                 set_sds(u16(d, 306+ 1));
02077                 set_sam(u8(d, 308+ 1));
02078                 set_sax(u16(d, 309+ 1));
02079                 set_san(u16(d, 311+ 1));
02080                 set_sao(u16(d, 313+ 1));
02081                 set_sap(u16(d, 315+ 1));
02082                 set_spt(u16(d, 317+ 1));
02083                 set_ssp(u16(d, 319+ 1));
02084
02085                 break;
02086
02087             case 0x04036100: // ALAMO Software version 0x04036100 (2017 S2h test float 10108).
02088
02089                 if(data.size() != 327) {
02090                     ostringstream os;
02091                     os « "Invalid X-Message: Expected 327 bytes, but received "
02092                        « data.size()
02093                        « " bytes!";
02094 #                   if defined(XMSG_DEBUG_ALAMO_MISS_GFG) && XMSG_DEBUG_ALAMO_MISS_GFG
02095                     lwrn() « os.str();
02096 #                   else
02097                     EXC_THROW(os.str());
02098 #                   endif // defined(XMSG_DEBUG_ALAMO_MISS_GFG) && XMSG_DEBUG_ALAMO_MISS_GFG
02099                 }
02100
02101                 /* Config. - General: */
02102
02103                 set_msg_version(u8(d, 8));
02104                 set_sensor_type(u8(d, 9));
02105                 set_msn(u16(d, 10));
02106                 set_dtd(u16(d, 12));
02107                 set_mds(u16(d, 14));
02108                 set_apt(u8(d, 16));
```

```
02109                 set_btm(u8(d, 17));
02110                 set_xas(u8(d, 18));
02111                 set_cps(u8(d, 19));
02112                 set_cst(u8(d, 20));
02113                 set_ctf(u16(d, 21));
02114                 set_ctl(u16(d, 23));
02115                 set_ctu(u16(d, 25));
02116 
02117                 set_csp(u32(d, 27));
02118                 cpb_[0] = u16(d, 31);
02119                 cpb_[1] = u16(d, 33);
02120                 cpb_[2] = u16(d, 35);
02121                 cpn_[0] = u16(d, 37);
02122                 cpn_[1] = u16(d, 39);
02123                 cpn_[2] = u16(d, 41);
02124                 cpp_[0] = u16(d, 43);
02125                 cpp_[1] = u16(d, 45);
02126                 cpp_[2] = u16(d, 47);
02127                 set_mtd(u16(d, 49 + 4));
02128                 set_mtt(u32(d, 51 + 4));
02129                 set_mam(u8(d, 55 + 4));
02130                 set_mat(u16(d, 56 + 4));
02131                 set_aca(u16(d, 58 + 4));
02132                 set_acg(u16(d, 60 + 4));
02133                 set_acr(u16(d, 62 + 4));
02134                 set_pnb(u16(d, 64 + 4));
02135                 set_pnc(u16(d, 66 + 4));
02136                 set_pxc(u16(d, 68 + 4));
02137                 set_pbl(u16(d, 70 + 4));
02138                 set_pbh(u16(d, 72 + 4));
02139                 set_pp0(u16(d, 74 + 4));
02140                 set_pp1(u16(d, 76 + 4));
02141                 set_pvl(u16(d, 78 + 4));
02142                 set_pvh(u16(d, 80 + 4));
02143                 set_pv0(u16(d, 82 + 4));
02144                 set_pv5(u16(d, 84 + 4));
02145                 set_pc0(u16(d, 86 + 4));
02146                 set_pc1(u16(d, 88 + 4));
02147 
02148                 set_icm(u8(d, 90 + 4));
02149                 set_imm(u16(d, 91 + 4));
02150                 set_ica0(u16(d, 93 + 4));
02151                 set_ica1(u16(d, 95 + 4));
02152                 set_icb0(u16(d, 97 + 4));
02153                 set_icb1(u16(d, 99 + 4));
02154                 set_imt0(u16(d, 101 + 4));
02155                 set_imt1(u16(d, 103 + 4));
02156                 set_ilt0(u16(d, 105 + 4));
02157                 set_ilt1(u16(d, 107 + 4));
02158                 set_icn(u16(d, 109 + 4));
02159                 set_icx(u16(d, 111 + 4));
02160                 set_ict(i16(d, 113 + 4));
02161                 set_ics(u16(d, 115 + 4));
02162                 set_iss0(u8(d, 117 + 4));
02163                 set_iss1(u8(d, 118 + 4));
02164                 set_ist(u16(d, 119 + 5));
02165                 set_iso(u16(d, 121 + 5));
02166                 set_isp(u8(d, 123 + 5));
02167                 set_iue0(u8(d, 124 + 5));
02168                 set_iue1(u8(d, 125 + 5));
02169                 set_icd0(u16(d, 126 + 5));
02170                 set_icd1(u16(d, 128 + 5));
02171                 set_icr0(u32(d, 130 + 5));
02172                 set_icr1(u32(d, 134 + 5));
02173                 set_icg0(u8(d, 138 + 5));
02174                 set_icg1(u8(d, 139 + 5));
02175                 set_isa(u8(d, 140 + 5));
02176                 set_isd(u16(d, 141 + 5));
02177                 set_iat(u16(d, 143 + 5));
02178                 set_isv(u16(d, 145 + 5));
02179                 set_isw(u16(d, 147 + 5));
02180                 set_ico0(u16(d, 149 + 5));
02181                 set_ico1(u16(d, 151 + 5));
02182                 set_icf(u32(d, 153 + 5));
02183                 set_icw(u32(d, 157 + 5));
02184 
02185                 /* Descent state configuration: */
02186 
02187                 set_descend_start_time(bpt::not_a_date_time);
02188                 set_dns(u16(d, 161 + 5));
02189                 set_dra(u16(d, 163 + 5));
02190                 set_dsd(u16(d, 165 + 5));
02191                 set_dst(u16(d, 167 + 5));
02192                 set_ddf(u16(d, 169 + 5));
02193                 set_ddc(u16(d, 171 + 5));
02194                 set_dsi(u32(d, 173 + 5));
02195                 set_ddd(u16(d, 177 + 5));
02196                 set_ddt(u16(d, 179 + 5));
02197                 set_dft(u16(d, 181 + 5));
02198                 set_dcm(u8(d, 183 + 5));
02199                 set_dal(u8(d, 184 + 5));
02200                 set_dct(u16(d, 185 + 5));
02201                 set_dca(u8(d, 187 + 5));
02202                 set_drc(u16(d, 188 + 5));
02203                 set_drt(u16(d, 190 + 5));
02204                 set_dcx(u16(d, 192 + 5));
02205                 set_dcp(u16(d, 194 + 5));
02206                 set_dcf(u16(d, 196 + 5));
02207                 set_dcd(u16(d, 198 + 5));
02208                 set_dcr(u16(d, 200 + 5));
02209                 set_dsw(u16(d, 202 + 5));
02210                 set_duc(u16(d, 204 + 5));
02211                 set_dpt(u16(d, 206 + 5));
02212                 set_dut(u16(d, 208 + 5));
02213                 set_dpd(u16(d, 210 + 5));
02214                 set_dvt(u16(d, 212 + 5));
02215 
02216                 set_dvd(u16(d, 214 + 5));
```

```
02217                 set_dvw(u16(d, 216 + 5));
02218
02219
02220                 set_pfr(u16(d, 218 + 5));
02221                 set_pfc(u16(d, 220 + 5));
02222                 set_ddp(u16(d, 222 + 5));
02223                 set_dds(u16(d, 224 + 5));
02224                 set_park_start_time(bpt::not_a_date_time);
02225                 set_pns(u16(d, 226 + 5));
02226                 set_pto(u32(d, 228 + 5));
02227                 set_pth(u32(d, 232 + 5));
02228                 set_psa(u8(d, 236 + 5));
02229                 set_psi(u32(d, 237 + 5));
02230                 set_pfm(u8(d, 241 + 5));
02231                 set_pft(u16(d, 242 + 5));
02232                 set_pfd(u16(d, 244 + 5));
02233                 set_pfv(u16(d, 246 + 5));
02234                 set_pdt(u16(d, 248 + 5));
02235                 set_psm(u8(d, 250 + 5));
02236                 set_psd(u16(d, 251 + 5));
02237                 set_pst(u16(d, 253 + 5));
02238                 set_psv(u16(d, 255 + 5));
02239                 set_psw(u16(d, 257 + 5));
02240                 set_ascent_start_time(bpt::not_a_date_time);
02241                 set_asn(u16(d, 259 + 5));
02242                 set_arr(u16(d, 261 + 5));
02243                 set_apm(u16(d, 263 + 5));
02244                 set_apn(u16(d, 265 + 5));
02245                 set_apf(u16(d, 267 + 5));
02246                 set_asi(u32(d, 269 + 5));
02247                 set_asd(u16(d, 273 + 5));
02248                 set_add(u16(d, 275 + 5));
02249                 set_adt(u16(d, 277 + 5));
02250                 set_arc(u16(d, 279 + 5));
02251                 set_ahr(u16(d, 281 + 5));
02252                 set_ahd(u16(d, 283 + 5));
02253                 set_surface_start_time(bpt::not_a_date_time);
02254                 set_sfn(u16(d, 285 + 5));
02255                 set_sdc(i16(d, 287 + 5));
02256                 set_sgt(u16(d, 289 + 5));
02257                 set_set(u16(d, 291 + 5));
02258                 set_sbm(u8(d, 293 + 5));
02259                 set_sbt(u16(d, 294 + 5));
02260                 set_sdt(u32(d, 296 + 5));
02261                 set_ssa(u16(d, 300 + 5));
02262                 set_ssi(u32(d, 302 + 5));
02263                 set_sds(u16(d, 306 + 5));
02264                 set_sam(u8(d, 308 + 5));
02265                 set_sax(u16(d, 309 + 5));
02266                 set_san(u16(d, 311 + 5));
02267                 set_sao(u16(d, 313 + 5));
02268                 set_sap(u16(d, 315 + 5));
02269                 set_spt(u16(d, 317 + 5));
02270                 set_ssp(u16(d, 319 + 5));
02271
02272                 break;
02273
02274             case 0x04046100: // ALAMO Software version 0x04046100 (2017 Hurricane Floats).
02275
02276                 if(data.size() != 328) {
02277                     ostringstream os;
02278                     os « "Invalid X-Message: Expected 328 bytes, but received "
02279                        « data.size()
02280                        « " bytes!";
02281 #                   if defined(XMSG_DEBUG_ALAMO_MISS_GFG) && XMSG_DEBUG_ALAMO_MISS_GFG
02282                     lwrn() « os.str();
02283 #                   else
02284                     EXC_THROW(os.str());
02285 #                   endif // defined(XMSG_DEBUG_ALAMO_MISS_GFG) && XMSG_DEBUG_ALAMO_MISS_GFG
02286                 }
02287
02288                 /* Config. - General: */
02289
02290                 set_msg_version(u8(d, 8));
02291                 set_sensor_type(u8(d, 9));
02292                 set_msn(u16(d, 10));
02293                 set_dtd(u16(d, 12));
02294                 set_mds(u16(d, 14));
02295                 set_apt(u8(d, 16));
02296                 set_btm(u8(d, 17));
02297                 set_xas(u8(d, 18));
02298                 set_cps(u8(d, 19));
02299                 set_cst(u8(d, 20));
02300                 set_ctf(u16(d, 21));
02301                 set_ctl(u16(d, 23));
02302                 set_ctu(u16(d, 25));
02303                 // set_ctc(u8(d, 27))
02304
02305                 set_csp(u32(d, 27 + 1));
02306                 set_cpb1(u16(d, 31 + 1));
02307                 set_cpb2(u16(d, 33 + 1));
02308                 set_cpb3(u16(d, 35 + 1));
02309                 set_cpn1(u16(d, 37 + 1));
02310                 set_cpn2(u16(d, 39 + 1));
02311                 set_cpn3(u16(d, 41 + 1));
02312                 set_cpp1(u16(d, 43 + 1));
02313                 set_cpp2(u16(d, 45 + 1));
02314                 set_cpp3(u16(d, 47 + 1));
02315                 // set_crd
02316                 // set_crs
02317
02318
02319                 set_mtd(u16(d, 49 + 5));  // good
02320                 set_mtt(u32(d, 51 + 5));
02321                 set_mam(u8(d, 55 + 5));
02322                 set_mat(u16(d, 56 + 5));
02323 //                set_mat2(u16(d, 56 + 5));
02324 //                set_map(u16(d, 56 + 5));
```

```
02325                 set_aca(u16(d, 58 + 5));
02326                 set_acg(u16(d, 60 + 5));
02327                 set_acr(u16(d, 62 + 5));
02328                 set_pnb(u16(d, 64 + 5));
02329                 set_pnc(u16(d, 66 + 5));
02330                 set_pxc(u16(d, 68 + 5));
02331                 set_pbl(u16(d, 70 + 5));
02332                 set_pbh(u16(d, 72 + 5));
02333                 set_pp0(u16(d, 74 + 5));
02334                 set_ppl(u16(d, 76 + 5));
02335                 set_pvl(u16(d, 78 + 5));
02336                 set_pvh(u16(d, 80 + 5));
02337                 set_pv0(u16(d, 82 + 5));
02338                 set_pv5(u16(d, 84 + 5));
02339                 set_pc0(u16(d, 86 + 5));
02340                 set_pc1(u16(d, 88 + 5));
02341
02342                 /* Ice Configuration */
02343
02344                 set_icm(u8(d, 90 + 5));
02345                 set_imm(u16(d, 91 + 5));
02346                 set_ica0(u16(d, 93 + 5));
02347                 set_ica1(u16(d, 95 + 5));
02348                 set_icb0(u16(d, 97 + 5));
02349                 set_icb1(u16(d, 99 + 5));
02350                 set_imt0(u16(d, 101 + 5));
02351                 set_imt1(u16(d, 103 + 5));
02352                 set_ilt0(u16(d, 105 + 5));
02353                 set_ilt1(u16(d, 107 + 5));
02354                 set_icn(u16(d, 109 + 5));
02355                 set_icx(u16(d, 111 + 5));
02356                 set_ict(i16(d, 113 + 5));
02357                 set_ics(u16(d, 115 + 5));
02358                 set_iss0(u8(d, 117 + 5));
02359                 set_iss1(u8(d, 118 + 5));
02360                 set_ist(u16(d, 119 + 6));
02361                 set_iso(u16(d, 121 + 6));
02362                 set_isp(u8(d, 123 + 6));
02363                 set_iue0(u8(d, 124 + 6));
02364                 set_iue1(u8(d, 125 + 6));
02365                 set_icd0(u16(d, 126 + 6));
02366                 set_icd1(u16(d, 128 + 6));
02367                 set_icr0(u32(d, 130 + 6));
02368                 set_icr1(u32(d, 134 + 6));
02369                 set_icg0(u8(d, 138 + 6));
02370                 set_icg1(u8(d, 139 + 6));
02371                 set_isa(u8(d, 140 + 6));
02372                 set_isd(u16(d, 141 + 6));
02373                 set_iat(u16(d, 143 + 6));
02374                 set_isv(u16(d, 145 + 6));
02375                 set_isw(u16(d, 147 + 6));
02376                 set_ico0(u16(d, 149 + 6));
02377                 set_ico1(u16(d, 151 + 6));
02378                 set_icf(u32(d, 153 + 6));
02379                 set_icw(u32(d, 157 + 6));
02380
02381                 /* Descent state configuration: */
02382
02383                 set_descend_start_time(bpt::not_a_date_time);
02384                 set_dns(u16(d, 161 + 6));
02385                 set_dra(u16(d, 163 + 6));
02386                 set_dsd(u16(d, 165 + 6));
02387                 set_dst(u16(d, 167 + 6));
02388                 set_ddf(u16(d, 169 + 6));
02389                 set_ddc(u16(d, 171 + 6));
02390                 set_dsi(u32(d, 173 + 6));
02391                 set_ddd(u16(d, 177 + 6));
02392                 set_ddt(u16(d, 179 + 6));
02393                 set_dft(u16(d, 181 + 6));
02394                 set_dcm(u8(d, 183 + 6));
02395                 set_dal(u8(d, 184 + 6));
02396                 set_dct(u16(d, 185 + 6));
02397                 set_dca(u8(d, 187 + 6));
02398                 set_drc(u16(d, 188 + 6));
02399                 set_drt(u16(d, 190 + 6));
02400                 set_dcx(u16(d, 192 + 6));
02401                 set_dcp(u16(d, 194 + 6));
02402                 set_dcf(u16(d, 196 + 6));
02403                 set_dcd(u16(d, 198 + 6));
02404                 set_dcr(u16(d, 200 + 6));
02405                 set_dsw(u16(d, 202 + 6));
02406                 set_duc(u16(d, 204 + 6));
02407                 set_dpt(u16(d, 206 + 6));
02408                 set_dut(u16(d, 208 + 6));
02409                 set_dpd(u16(d, 210 + 6));
02410                 set_dvt(u16(d, 212 + 6));
02411
02412                 set_dvd(u16(d, 214 + 6));
02413                 set_dvw(u16(d, 216 + 6));
02414
02415
02416                 set_pfr(u16(d, 218 + 6));
02417                 set_pfc(u16(d, 220 + 6));
02418                 set_ddp(u16(d, 222 + 6));
02419                 set_dds(u16(d, 224 + 6));
02420                 set_park_start_time(bpt::not_a_date_time);
02421                 set_pns(u16(d, 226 + 6));
02422                 set_pto(u32(d, 228 + 6));
02423                 set_pth(u32(d, 232 + 6));
02424                 // set_ptd(u32(d, 236 + 10))
02425                 set_psa(u8(d, 236 + 6));
02426                 set_psi(u32(d, 237 + 6));
02427                 set_pfm(u8(d, 241 + 6));
02428                 set_pft(u16(d, 242 + 6));
02429                 set_pfd(u16(d, 244 + 6));
02430                 set_pfv(u16(d, 246 + 6));
02431                 set_pdt(u16(d, 248 + 6));
02432                 set_psm(u8(d, 250 + 6));
```

```
02433                 set_psd(u16(d, 251 + 6));
02434                 set_pst(u16(d, 253 + 6));
02435                 set_psv(u16(d, 255 + 6));
02436                 set_psw(u16(d, 257 + 6));
02437                 set_ascent_start_time(bpt::not_a_date_time);
02438                 set_asn(u16(d, 259 + 6));
02439                 set_arr(u16(d, 261 + 6));
02440                 set_apm(u16(d, 263 + 6));
02441                 set_apn(u16(d, 265 + 6));
02442                 set_apf(u16(d, 267 + 6));
02443                 set_asi(u32(d, 269 + 6));
02444                 set_asd(u16(d, 273 + 6));
02445                 set_add(u16(d, 275 + 6));
02446                 set_adt(u16(d, 277 + 6));
02447                 set_arc(u16(d, 279 + 6));
02448                 set_ahr(u16(d, 281 + 6));
02449                 set_ahd(u16(d, 283 + 6));
02450                 set_surface_start_time(bpt::not_a_date_time);
02451                 set_sfn(u16(d, 285 + 6));
02452                 set_sdc(i16(d, 287 + 6));
02453                 set_sgt(u16(d, 289 + 6));
02454                 set_set(u16(d, 291 + 6));
02455                 set_sbm(u8(d, 293 + 6));
02456                 set_sbt(u16(d, 294 + 6));
02457                 set_sdt(u32(d, 296 + 6));
02458                 set_ssa(u16(d, 300 + 6));
02459                 set_ssi(u32(d, 302 + 6));
02460                 set_sds(u16(d, 306 + 6));
02461                 set_sam(u8(d, 308 + 6));
02462                 set_sax(u16(d, 309 + 6));
02463                 set_san(u16(d, 311 + 6));
02464                 set_sao(u16(d, 313 + 6));
02465                 set_sap(u16(d, 315 + 6));
02466                 set_spt(u16(d, 317 + 6));
02467                 set_ssp(u16(d, 319 + 6));
02468
02469                 break;
02470
02471             case 0x04056300: // ALAMO Software version 0x04056300 (2017 S2h test float 10108).
02472
02473                 if(data.size() != 328) {
02474                     ostringstream os;
02475                     os « "Invalid X-Message: Expected 328 bytes, but received "
02476                        « data.size()
02477                        « " bytes!";
02478 #                   if defined(XMSG_DEBUG_ALAMO_MISS_GFG) && XMSG_DEBUG_ALAMO_MISS_GFG
02479                     lwrn() « os.str();
02480 #                   else
02481                     EXC_THROW(os.str());
02482 #                   endif // defined(XMSG_DEBUG_ALAMO_MISS_GFG) && XMSG_DEBUG_ALAMO_MISS_GFG
02483                 }
02484
02485                 /* General Configuration: */
02486
02487                 set_msg_version(u8(d, 8));
02488                 set_sensor_type(u8(d, 9));
02489                 set_msn(u16(d, 10));
02490                 set_dtd(u16(d, 12));
02491                 set_mds(u16(d, 14));
02492                 set_apt(u8(d, 16));
02493                 set_btm(u8(d, 17));
02494                 set_xas(u8(d, 18));
02495                 set_cps(u8(d, 19));
02496                 set_cst(u8(d, 20));
02497                 set_ctf(u16(d, 21));
02498                 set_ctl(u16(d, 23));
02499                 set_ctu(u16(d, 25));
02500                 // set_ctc(u8(d, 27))
02501
02502                 set_csp(u32(d, 27 + 1));
02503                 set_cpb1(u16(d, 31 + 1));
02504                 set_cpb2(u16(d, 33 + 1));
02505                 set_cpb3(u16(d, 35 + 1));
02506                 set_cpn1(u16(d, 37 + 1));
02507                 set_cpn2(u16(d, 39 + 1));
02508                 set_cpn3(u16(d, 41 + 1));
02509                 set_cpp1(u16(d, 43 + 1));
02510                 set_cpp2(u16(d, 45 + 1));
02511                 set_cpp3(u16(d, 47 + 1));
02512                 // set_crd
02513                 // set_crs
02514
02515
02516                 set_mtd(u16(d, 49 + 5));
02517                 set_mtt(u32(d, 51 + 5));
02518                 set_mam(u8(d, 55 + 5));
02519 //                set_mat1(u16(d, 56 + 5));
02520 //                set_mat2(u16(d, 56 + 5));
02521 //                set_map(u16(d, 56 + 5));
02522                 set_aca(u16(d, 58 + 9));
02523                 set_acg(u16(d, 60 + 9));
02524                 set_acr(u16(d, 62 + 9));
02525                 set_pnb(u16(d, 64 + 9));
02526                 set_pnc(u16(d, 66 + 9));
02527                 set_pxc(u16(d, 68 + 9));
02528                 set_pbl(u16(d, 70 + 9));
02529                 set_pbh(u16(d, 72 + 9));
02530                 set_pp0(u16(d, 74 + 9));
02531                 set_pp1(u16(d, 76 + 9));
02532                 set_pvl(u16(d, 78 + 9));
02533                 set_pvh(u16(d, 80 + 9));
02534                 set_pv0(u16(d, 82 + 9));
02535                 set_pv5(u16(d, 84 + 9));
02536                 set_pc0(u16(d, 86 + 9));
02537                 set_pc1(u16(d, 88 + 9));
02538
02539                 /* Ice Configuration */
02540
```

```
02541                 set_icm(u8(d, 90 + 9));
02542                 set_imm(u16(d, 91 + 9));
02543                 set_ica0(u16(d, 93 + 9));
02544                 set_ica1(u16(d, 95 + 9));
02545                 set_icb0(u16(d, 97 + 9));
02546                 set_icb1(u16(d, 99 + 9));
02547                 set_imt0(u16(d, 101 + 9));
02548                 set_imt1(u16(d, 103 + 9));
02549                 set_ilt0(u16(d, 105 + 9));
02550                 set_ilt1(u16(d, 107 + 9));
02551                 set_icn(u16(d, 109 + 9));
02552                 set_icx(u16(d, 111 + 9));
02553                 set_ict(i16(d, 113 + 9));
02554                 set_ics(u16(d, 115 + 9));
02555                 set_iss0(u8(d, 117 + 9));
02556                 set_iss1(u8(d, 118 + 9));
02557                 set_ist(u16(d, 119 + 10));
02558                 set_iso(u16(d, 121 + 10));
02559                 set_isp(u8(d, 123 + 10));
02560                 set_iue0(u8(d, 124 + 10));
02561                 set_iue1(u8(d, 125 + 10));
02562                 set_icd0(u16(d, 126 + 10));
02563                 set_icd1(u16(d, 128 + 10));
02564                 set_icr0(u32(d, 130 + 10));
02565                 set_icr1(u32(d, 134 + 10));
02566                 set_icg0(u8(d, 138 + 10));
02567                 set_icg1(u8(d, 139 + 10));
02568                 set_isa(u8(d, 140 + 10));
02569                 set_isd(u16(d, 141 + 10));
02570                 set_iat(u16(d, 143 + 10));
02571                 set_isv(u16(d, 145 + 10));
02572                 set_isw(u16(d, 147 + 10));
02573                 set_ico0(u16(d, 149 + 10));
02574                 set_ico1(u16(d, 151 + 10));
02575                 set_icf(u32(d, 153 + 10));
02576                 set_icw(u32(d, 157 + 10));
02577 
02578                 /* Descent state configuration: */
02579 
02580                 set_descend_start_time(bpt::not_a_date_time);
02581                 set_dns(u16(d, 161 + 10));
02582                 set_dra(u16(d, 163 + 10));
02583                 set_dsd(u16(d, 165 + 10));
02584                 set_dst(u16(d, 167 + 10));
02585                 set_ddf(u16(d, 169 + 10));
02586                 set_ddc(u16(d, 171 + 10));
02587                 set_dsi(u32(d, 173 + 10));
02588                 set_ddd(u16(d, 177 + 10));
02589                 set_ddt(u16(d, 179 + 10));
02590                 set_dft(u16(d, 181 + 10));
02591                 set_dcm(u8(d, 183 + 10));
02592                 set_dal(u8(d, 184 + 10));
02593                 set_dct(u16(d, 185 + 10));
02594                 set_dca(u8(d, 187 + 10));
02595                 set_drc(u16(d, 188 + 10));
02596                 set_drt(u16(d, 190 + 10));
02597                 set_dcx(u16(d, 192 + 10));
02598                 set_dcp(u16(d, 194 + 10));
02599                 set_dcf(u16(d, 196 + 10));
02600                 set_dcd(u16(d, 198 + 10));
02601                 set_dcr(u16(d, 200 + 10));
02602                 set_dsw(u16(d, 202 + 10));
02603                 set_duc(u16(d, 204 + 10));
02604                 set_dpt(u16(d, 206 + 10));
02605                 set_dut(u16(d, 208 + 10));
02606                 set_dpd(u16(d, 210 + 10));
02607                 set_dvt(u16(d, 212 + 10));
02608 
02609                 set_dvd(u16(d, 214 + 10));
02610                 set_dvw(u16(d, 216 + 10));
02611 
02612 
02613                 set_pfr(u16(d, 218 + 10));
02614                 set_pfc(u16(d, 220 + 10));
02615                 set_ddp(u16(d, 222 + 10));
02616                 set_dds(u16(d, 224 + 10));
02617                 set_park_start_time(bpt::not_a_date_time);
02618                 set_pns(u16(d, 226 + 10));
02619                 set_pto(u32(d, 228 + 10));
02620                 set_pth(u32(d, 232 + 10));
02621                 // set_ptd(u32(d, 236 + 10))
02622                 set_psa(u8(d, 236 + 14));
02623                 set_psi(u32(d, 237 + 14));
02624                 set_pfm(u8(d, 241 + 14));
02625                 set_pft(u16(d, 242 + 14));
02626                 set_pfd(u16(d, 244 + 14));
02627                 set_pfv(u16(d, 246 + 14));
02628                 set_pdt(u16(d, 248 + 14));
02629                 set_psm(u8(d, 250 + 14));
02630                 set_psd(u16(d, 251 + 14));
02631                 set_pst(u16(d, 253 + 14));
02632                 set_psv(u16(d, 255 + 14));
02633                 set_psw(u16(d, 257 + 14));
02634                 set_ascent_start_time(bpt::not_a_date_time);
02635                 set_asn(u16(d, 259 + 14));
02636                 set_arr(u16(d, 261 + 14));
02637                 set_apm(u16(d, 263 + 14));
02638                 set_apn(u16(d, 265 + 14));
02639                 set_apf(u16(d, 267 + 14));
02640                 set_asi(u32(d, 269 + 14));
02641                 set_asd(u16(d, 273 + 14));
02642                 set_add(u16(d, 275 + 14));
02643                 set_adt(u16(d, 277 + 14));
02644                 set_arc(u16(d, 279 + 14));
02645                 set_ahr(u16(d, 281 + 14));
02646                 set_ahd(u16(d, 283 + 14));
02647                 set_surface_start_time(bpt::not_a_date_time);
02648                 set_sfn(u16(d, 285 + 14));
```

```
02649                 set_sdc(i16(d, 287 + 14));
02650                 set_sgt(u16(d, 289 + 14));
02651                 set_set(u16(d, 291 + 14));
02652                 set_sbm(u8(d, 293 + 14));
02653                 set_sbt(u16(d, 294 + 14));
02654                 set_sdt(u32(d, 296 + 14));
02655                 set_ssa(u16(d, 300 + 14));
02656                 set_ssi(u32(d, 302 + 14));
02657                 set_sds(u16(d, 306 + 14));
02658                 set_sam(u8(d, 308 + 14));
02659                 set_sax(u16(d, 309 + 14));
02660                 set_san(u16(d, 311 + 14));
02661                 set_sao(u16(d, 313 + 14));
02662 //                set_sap(u16(d, 315 + 14));
02663 //                set_spt(u16(d, 317 + 14));
02664 //                set_ssp(u16(d, 319 + 14));
02665
02666                 break;
02667
02668             case 0x04066100: // ALAMO Software version 0x04066100 (2017 S2h test float 10108).
02669
02670                 if(data.size() != 328) {
02671                     ostringstream os;
02672                     os « "Invalid X-Message: Expected 328 bytes, but received "
02673                        « data.size()
02674                        « " bytes!";
02675 #                   if defined(XMSG_DEBUG_ALAMO_MISS_GFG) && XMSG_DEBUG_ALAMO_MISS_GFG
02676                     lwrn() « os.str();
02677 #                   else
02678                     EXC_THROW(os.str());
02679 #                   endif // defined(XMSG_DEBUG_ALAMO_MISS_GFG) && XMSG_DEBUG_ALAMO_MISS_GFG
02680                 }
02681
02682                 /* General Configuration: */
02683
02684                 set_msg_version(u8(d, 8));
02685                 set_sensor_type(u8(d, 9));
02686                 set_msn(u16(d, 10));
02687                 set_dtd(u16(d, 12));
02688                 set_mds(u16(d, 14));
02689                 set_apt(u8(d, 16));
02690                 set_btm(u8(d, 17));
02691                 set_xas(u8(d, 18));
02692                 set_cps(u8(d, 19));
02693                 set_cst(u8(d, 20));
02694                 set_ctf(u16(d, 21));
02695                 set_ctl(u16(d, 23));
02696                 set_ctu(u16(d, 25));
02697                 // set_ctc(u8(d, 27))
02698
02699                 set_csp(u32(d, 27 + 1));
02700                 set_cpb1(u16(d, 31 + 1));
02701                 set_cpb2(u16(d, 33 + 1));
02702                 set_cpb3(u16(d, 35 + 1));
02703                 set_cpn1(u16(d, 37 + 1));
02704                 set_cpn2(u16(d, 39 + 1));
02705                 set_cpn3(u16(d, 41 + 1));
02706                 set_cpp1(u16(d, 43 + 1));
02707                 set_cpp2(u16(d, 45 + 1));
02708                 set_cpp3(u16(d, 47 + 1));
02709                 // set_crd
02710                 // set_crs
02711
02712
02713                 set_mtd(u16(d, 49 + 5));
02714                 set_mtt(u32(d, 51 + 5));
02715                 set_mam(u8(d, 55 + 5));
02716 //                set_mat1(u16(d, 56 + 5));
02717 //                set_mat2(u16(d, 56 + 5));
02718 //                set_map(u16(d, 56 + 5));
02719                 set_aca(u16(d, 58 + 9));
02720                 set_acg(u16(d, 60 + 9));
02721                 set_acr(u16(d, 62 + 9));
02722                 set_pnb(u16(d, 64 + 9));
02723                 set_pnc(u16(d, 66 + 9));
02724                 set_pxc(u16(d, 68 + 9));
02725                 set_pbl(u16(d, 70 + 9));
02726                 set_pbh(u16(d, 72 + 9));
02727                 set_pp0(u16(d, 74 + 9));
02728                 set_ppl(u16(d, 76 + 9));
02729                 set_pvl(u16(d, 78 + 9));
02730                 set_pvh(u16(d, 80 + 9));
02731                 set_pv0(u16(d, 82 + 9));
02732                 set_pv5(u16(d, 84 + 9));
02733                 set_pc0(u16(d, 86 + 9));
02734                 set_pc1(u16(d, 88 + 9));
02735
02736                 /* Ice Configuration */
02737
02738                 set_icm(u8(d, 90 + 9));
02739                 set_imm(u16(d, 91 + 9));
02740                 set_ica0(u16(d, 93 + 9));
02741                 set_ica1(u16(d, 95 + 9));
02742                 set_icb0(u16(d, 97 + 9));
02743                 set_icb1(u16(d, 99 + 9));
02744                 set_imt0(u16(d, 101 + 9));
02745                 set_imt1(u16(d, 103 + 9));
02746                 set_ilt0(u16(d, 105 + 9));
02747                 set_ilt1(u16(d, 107 + 9));
02748                 set_icn(u16(d, 109 + 9));
02749                 set_icx(u16(d, 111 + 9));
02750                 set_ict(i16(d, 113 + 9));
02751                 set_ics(u16(d, 115 + 9));
02752                 set_iss0(u8(d, 117 + 9));
02753                 set_iss1(u8(d, 118 + 9));
02754                 set_ist(u16(d, 119 + 10));
02755                 set_iso(u16(d, 121 + 10));
02756                 set_isp(u8(d, 123 + 10));
```

```
02757                 set_iue0(u8(d, 124 + 10));
02758                 set_iue1(u8(d, 125 + 10));
02759                 set_icd0(u16(d, 126 + 10));
02760                 set_icd1(u16(d, 128 + 10));
02761                 set_icr0(u32(d, 130 + 10));
02762                 set_icr1(u32(d, 134 + 10));
02763                 set_icg0(u8(d, 138 + 10));
02764                 set_icg1(u8(d, 139 + 10));
02765                 set_isa(u8(d, 140 + 10));
02766                 set_isd(u16(d, 141 + 10));
02767                 set_iat(u16(d, 143 + 10));
02768                 set_isv(u16(d, 145 + 10));
02769                 set_isw(u16(d, 147 + 10));
02770                 set_ico0(u16(d, 149 + 10));
02771                 set_ico1(u16(d, 151 + 10));
02772                 set_icf(u32(d, 153 + 10));
02773                 set_icw(u32(d, 157 + 10));
02774 
02775                 /* Descent state configuration: */
02776 
02777                 set_descend_start_time(bpt::not_a_date_time);
02778                 set_dns(u16(d, 161 + 10));
02779                 set_dra(u16(d, 163 + 10));
02780                 set_dsd(u16(d, 165 + 10));
02781                 set_dst(u16(d, 167 + 10));
02782                 set_ddf(u16(d, 169 + 10));
02783                 set_ddc(u16(d, 171 + 10));
02784                 set_dsi(u32(d, 173 + 10));
02785                 set_ddd(u16(d, 177 + 10));
02786                 set_ddt(u16(d, 179 + 10));
02787                 set_dft(u16(d, 181 + 10));
02788                 set_dcm(u8(d, 183 + 10));
02789                 set_dal(u8(d, 184 + 10));
02790                 set_dct(u16(d, 185 + 10));
02791                 set_dca(u8(d, 187 + 10));
02792                 set_drc(u16(d, 188 + 10));
02793                 set_drt(u16(d, 190 + 10));
02794                 set_dcx(u16(d, 192 + 10));
02795                 set_dcp(u16(d, 194 + 10));
02796                 set_dcf(u16(d, 196 + 10));
02797                 set_dcd(u16(d, 198 + 10));
02798                 set_dcr(u16(d, 200 + 10));
02799                 set_dsw(u16(d, 202 + 10));
02800                 set_duc(u16(d, 204 + 10));
02801                 set_dpt(u16(d, 206 + 10));
02802                 set_dut(u16(d, 208 + 10));
02803                 set_dpd(u16(d, 210 + 10));
02804                 set_dvt(u16(d, 212 + 10));
02805 
02806                 set_dvd(u16(d, 214 + 10));
02807                 set_dvw(u16(d, 216 + 10));
02808 
02809 
02810                 set_pfr(u16(d, 218 + 10));
02811                 set_pfc(u16(d, 220 + 10));
02812                 set_ddp(u16(d, 222 + 10));
02813                 set_dds(u16(d, 224 + 10));
02814                 set_park_start_time(bpt::not_a_date_time);
02815                 set_pns(u16(d, 226 + 10));
02816                 set_pto(u32(d, 228 + 10));
02817                 set_pth(u32(d, 232 + 10));
02818                 // set_ptd(u32(d, 236 + 10))
02819                 set_psa(u8(d, 236 + 14));
02820                 set_psi(u32(d, 237 + 14));
02821                 set_pfm(u8(d, 241 + 14));
02822                 set_pft(u16(d, 242 + 14));
02823                 set_pfd(u16(d, 244 + 14));
02824                 set_pfv(u16(d, 246 + 14));
02825                 set_pdt(u16(d, 248 + 14));
02826                 set_psm(u8(d, 250 + 14));
02827                 set_psd(u16(d, 251 + 14));
02828                 set_pst(u16(d, 253 + 14));
02829                 set_psv(u16(d, 255 + 14));
02830                 set_psw(u16(d, 257 + 14));
02831                 set_ascent_start_time(bpt::not_a_date_time);
02832                 set_asn(u16(d, 259 + 14));
02833                 set_arr(u16(d, 261 + 14));
02834                 set_apm(u16(d, 263 + 14));
02835                 set_apn(u16(d, 265 + 14));
02836                 set_apf(u16(d, 267 + 14));
02837                 set_asi(u32(d, 269 + 14));
02838                 set_asd(u16(d, 273 + 14));
02839                 set_add(u16(d, 275 + 14));
02840                 set_adt(u16(d, 277 + 14));
02841                 set_arc(u16(d, 279 + 14));
02842                 set_ahr(u16(d, 281 + 14));
02843                 set_ahd(u16(d, 283 + 14));
02844                 set_surface_start_time(bpt::not_a_date_time);
02845                 set_sfn(u16(d, 285 + 14));
02846                 set_sdc(i16(d, 287 + 14));
02847                 set_sgt(u16(d, 289 + 14));
02848                 set_set(u16(d, 291 + 14));
02849                 set_sbm(u8(d, 293 + 14));
02850                 set_sbt(u16(d, 294 + 14));
02851                 set_sdt(u32(d, 296 + 14));
02852                 set_ssa(u16(d, 300 + 14));
02853                 set_ssi(u32(d, 302 + 14));
02854                 set_sds(u16(d, 306 + 14));
02855                 set_sam(u8(d, 308 + 14));
02856                 set_sax(u16(d, 309 + 14));
02857                 set_san(u16(d, 311 + 14));
02858                 set_sao(u16(d, 313 + 14));
02859 //                set_sap(u16(d, 315 + 14));
02860 //                set_spt(u16(d, 317 + 14));
02861 //                set_ssp(u16(d, 319 + 14));
02862 
02863                 break;
02864 
```

```
02865             case 0x05006100: // ALAMO Software version 0x05006100 (2017 Alto test float 10109).
02866
02867               if(data.size() != 328) {
02868                   ostringstream os;
02869                   os « "Invalid X-Message: Expected 328 bytes, but received "
02870                      « data.size()
02871                      « " bytes!";
02872 #                 if defined(XMSG_DEBUG_ALAMO_MISS_GFG) && XMSG_DEBUG_ALAMO_MISS_GFG
02873                   lwrn() « os.str();
02874 #                 else
02875                   EXC_THROW(os.str());
02876 #                 endif // defined(XMSG_DEBUG_ALAMO_MISS_GFG) && XMSG_DEBUG_ALAMO_MISS_GFG
02877               }
02878
02879               /* General Configuration: */
02880
02881               set_msg_version(u8(d, 8));
02882               set_sensor_type(u8(d, 9));
02883               set_msn(u16(d, 10));
02884               set_dtd(u16(d, 12));
02885               set_mds(u16(d, 14));
02886               set_apt(u8(d, 16));
02887               set_btm(u8(d, 17));
02888               set_xas(u8(d, 18));
02889               set_cps(u8(d, 19));
02890               set_cst(u8(d, 20));
02891               set_ctf(u16(d, 21));
02892               set_ctl(u16(d, 23));
02893               set_ctu(u16(d, 25));
02894               // set_ctc(u8(d, 27))
02895
02896               set_csp(u32(d, 27 + 1));
02897               set_cpb1(u16(d, 31 + 1));
02898               set_cpb2(u16(d, 33 + 1));
02899               set_cpb3(u16(d, 35 + 1));
02900               set_cpn1(u16(d, 37 + 1));
02901               set_cpn2(u16(d, 39 + 1));
02902               set_cpn3(u16(d, 41 + 1));
02903               set_cpp1(u16(d, 43 + 1));
02904               set_cpp2(u16(d, 45 + 1));
02905               set_cpp3(u16(d, 47 + 1));
02906               // set_crd
02907               // set_crs
02908
02909
02910               set_mtd(u16(d, 49 + 5));
02911               set_mtt(u32(d, 51 + 5));
02912               set_mam(u8(d, 55 + 5));
02913 //                set_mat1(u16(d, 56 + 5));
02914 //                set_mat2(u16(d, 56 + 5));
02915 //                set_map(u16(d, 56 + 5));
02916               set_aca(u16(d, 58 + 9));
02917               set_acg(u16(d, 60 + 9));
02918               set_acr(u16(d, 62 + 9));
02919               set_pnb(u16(d, 64 + 9));
02920               set_pnc(u16(d, 66 + 9));
02921               set_pxc(u16(d, 68 + 9));
02922               set_pbl(u16(d, 70 + 9));
02923               set_pbh(u16(d, 72 + 9));
02924               set_pp0(u16(d, 74 + 9));
02925               set_pp1(u16(d, 76 + 9));
02926               set_pvl(u16(d, 78 + 9));
02927               set_pvh(u16(d, 80 + 9));
02928               set_pv0(u16(d, 82 + 9));
02929               set_pv5(u16(d, 84 + 9));
02930               set_pc0(u16(d, 86 + 9));
02931               set_pc1(u16(d, 88 + 9));
02932
02933               /* Ice Configuration */
02934
02935               set_icm(u8(d, 90 + 9));
02936               set_imm(u16(d, 91 + 9));
02937               set_ica0(u16(d, 93 + 9));
02938               set_ica1(u16(d, 95 + 9));
02939               set_icb0(u16(d, 97 + 9));
02940               set_icb1(u16(d, 99 + 9));
02941               set_imt0(u16(d, 101 + 9));
02942               set_imt1(u16(d, 103 + 9));
02943               set_ilt0(u16(d, 105 + 9));
02944               set_ilt1(u16(d, 107 + 9));
02945               set_icn(u16(d, 109 + 9));
02946               set_icx(u16(d, 111 + 9));
02947               set_ict(i16(d, 113 + 9));
02948               set_ics(u16(d, 115 + 9));
02949               set_iss0(u8(d, 117 + 9));
02950               set_iss1(u8(d, 118 + 9));
02951               set_ist(u16(d, 119 + 10));
02952               set_iso(u16(d, 121 + 10));
02953               set_isp(u8(d, 123 + 10));
02954               set_iue0(u8(d, 124 + 10));
02955               set_iue1(u8(d, 125 + 10));
02956               set_icd0(u16(d, 126 + 10));
02957               set_icd1(u16(d, 128 + 10));
02958               set_icr0(u32(d, 130 + 10));
02959               set_icr1(u32(d, 134 + 10));
02960               set_icg0(u8(d, 138 + 10));
02961               set_icg1(u8(d, 139 + 10));
02962               set_isa(u8(d, 140 + 10));
02963               set_isd(u16(d, 141 + 10));
02964               set_iat(u16(d, 143 + 10));
02965               set_isv(u16(d, 145 + 10));
02966               set_isw(u16(d, 147 + 10));
02967               set_ico0(u16(d, 149 + 10));
02968               set_ico1(u16(d, 151 + 10));
02969               set_icf(u32(d, 153 + 10));
02970               set_icw(u32(d, 157 + 10));
02971
02972               /* Descent state configuration: */
```

```
02973
02974               set_descend_start_time(bpt::not_a_date_time);
02975               set_dns(u16(d, 161 + 10));
02976               set_dra(u16(d, 163 + 10));
02977               set_dsd(u16(d, 165 + 10));
02978               set_dst(u16(d, 167 + 10));
02979               set_ddf(u16(d, 169 + 10));
02980               set_ddc(u16(d, 171 + 10));
02981               set_dsi(u32(d, 173 + 10));
02982               set_ddd(u16(d, 177 + 10));
02983               set_ddt(u16(d, 179 + 10));
02984               set_dft(u16(d, 181 + 10));
02985               set_dcm(u8(d, 183 + 10));
02986               set_dal(u8(d, 184 + 10));
02987               set_dct(u16(d, 185 + 10));
02988               set_dca(u8(d, 187 + 10));
02989               set_drc(u16(d, 188 + 10));
02990               set_drt(u16(d, 190 + 10));
02991               set_dcx(u16(d, 192 + 10));
02992               set_dcp(u16(d, 194 + 10));
02993               set_dcf(u16(d, 196 + 10));
02994               set_dcd(u16(d, 198 + 10));
02995               set_dcr(u16(d, 200 + 10));
02996               set_dsw(u16(d, 202 + 10));
02997               set_duc(u16(d, 204 + 10));
02998               set_dpt(u16(d, 206 + 10));
02999               set_dut(u16(d, 208 + 10));
03000               set_dpd(u16(d, 210 + 10));
03001               set_dvt(u16(d, 212 + 10));
03002
03003               set_dvd(u16(d, 214 + 10));
03004               set_dvw(u16(d, 216 + 10));
03005
03006
03007               set_pfr(u16(d, 218 + 10));
03008               set_pfc(u16(d, 220 + 10));
03009               set_ddp(u16(d, 222 + 10));
03010               set_dds(u16(d, 224 + 10));
03011               set_park_start_time(bpt::not_a_date_time);
03012               set_pns(u16(d, 226 + 10));
03013               set_pto(u32(d, 228 + 10));
03014               set_pth(u32(d, 232 + 10));
03015               // set_ptd(u32(d, 236 + 10))
03016               set_psa(u8(d, 236 + 14));
03017               set_psi(u32(d, 237 + 14));
03018               set_pfm(u8(d, 241 + 14));
03019               set_pft(u16(d, 242 + 14));
03020               set_pfd(u16(d, 244 + 14));
03021               set_pfv(u16(d, 246 + 14));
03022               set_pdt(u16(d, 248 + 14));
03023               set_psm(u8(d, 250 + 14));
03024               set_psd(u16(d, 251 + 14));
03025               set_pst(u16(d, 253 + 14));
03026               set_psv(u16(d, 255 + 14));
03027               set_psw(u16(d, 257 + 14));
03028               set_ascent_start_time(bpt::not_a_date_time);
03029               set_asn(u16(d, 259 + 14));
03030               set_arr(u16(d, 261 + 14));
03031               set_apm(u16(d, 263 + 14));
03032               set_apn(u16(d, 265 + 14));
03033               set_apf(u16(d, 267 + 14));
03034               set_asi(u32(d, 269 + 14));
03035               set_asd(u16(d, 273 + 14));
03036               set_add(u16(d, 275 + 14));
03037               set_adt(u16(d, 277 + 14));
03038               set_arc(u16(d, 279 + 14));
03039               set_ahr(u16(d, 281 + 14));
03040               set_ahd(u16(d, 283 + 14));
03041               set_surface_start_time(bpt::not_a_date_time);
03042               set_sfn(u16(d, 285 + 14));
03043               set_sdc(i16(d, 287 + 14));
03044               set_sgt(u16(d, 289 + 14));
03045               set_set(u16(d, 291 + 14));
03046               set_sbm(u8(d, 293 + 14));
03047               set_sbt(u16(d, 294 + 14));
03048               set_sdt(u32(d, 296 + 14));
03049               set_ssa(u16(d, 300 + 14));
03050               set_ssi(u32(d, 302 + 14));
03051               set_sds(u16(d, 306 + 14));
03052               set_sam(u8(d, 308 + 14));
03053               set_sax(u16(d, 309 + 14));
03054               set_san(u16(d, 311 + 14));
03055               set_sao(u16(d, 313 + 14));
03056
03057               break;
03058
03059             default: { // Unsupported ALAMO software version.
03060
03061                 ostringstream os;
03062                 os « "Unsupported ALAMO Mission Parameters message version: 0x"
03063                    « hex
03064                    « setw(8)
03065                    « setfill('0')
03066                    « fw_version()
03067                    « "!";
03068                 EXC_THROW(os.str());
03069             }
03070
03071                 break;
03072             }
03073
03074 #           if defined(XMSG_DEBUG_ALAMO_MISS_GFG) && XMSG_DEBUG_ALAMO_MISS_GFG
03075
03076             ldbg() « "Mission configuration listing string: "
03077                    « endl « GenerateListingString(0, 0, 0);
03078
03079 #           endif // defined(XMSG_DEBUG_ALAMO_MISS_GFG) && XMSG_DEBUG_ALAMO_MISS_GFG
03080
```

```
03081         }CATCH_BLOCK("Failed to decode ALAMO mission configuration message!")
03082
03083     }
03084
03085     /** Generate a listing entry string from this ALAMO mission configuration message object.
03086      *
03087      *  @param[in]      whoi        The unique WHOI instrument identifier or serial number.
03088      *  @param[in]      dive        The dive or cycle number of this ALAMO mission configuration
03089      *                              message object.
03090      *  @param[in]      idx         The X-Message index of this ALAMO mission configuration message
03091      *                              object.
03092      *
03093      *  @return         A listing entry string from this ALAMO mission configuration message object.
03094      */
03095     const string GenerateListingString(int whoi, int dive, int idx) const {
03096
03097         int midx = 0;
03098         ostringstream os;
03099
03100         try {
03101
03102             switch(fw_version()) {
03103             case 0x02006200: // ALAMO Software version 0x02006200.
03104             case 0x02006300: // ALAMO Software version 0x02006300.
03105             case 0x02006700: // ALAMO Software version 0x02006700.
03106             case 0x02006800: // ALAMO Software version 0x02006800.
03107             {
03108
03109                 // Header lines.
03110                 static const format fmt1("#   %5s %4s %4s %16s");
03111                 static const format fmt3("mis %05d %4d %4d       0x%08X");
03112                 os « format(fmt1) % "SN" % "dive" % "idx" % "SWVer" « endl;
03113                 os « format(fmt3) % whoi % dive % idx % fw_version() « endl;
03114
03115                 // Minor index 0.
03116                 static const format fmt4("#   %5s %4s %4s %4s %11s %11s %11s %11s %11s");
03117                 static const format fmt5("#    %5s %4s %4s %4s %11s %11s %11s %11s %11s");
03118                 static const format fmt6("mis %05d %4d %4d %4d %11d %11d %11d %11d %11d");
03119                 os « format(fmt4) % "SN" % "dive" % "idx" % "midx" % "MSN" % "DTD" % "MDS" % "APT"
03120                            % "XAS"
03121                    « endl;
03122                 os « format(fmt5) % "" % "" % "" % "" % "" % "(dBar)" % "(dBar)" % "" % "" « endl;
03123                 os « format(fmt6) % whoi % dive % idx % midx++ // midx
03124                            % msn() % dtd() % mds() % (int) apt() % (int) xas()
03125                    « endl;
03126
03127                 // Minor index 1.
03128                 static const format fmt7("#   %5s %4s %4s %4s %11s %11s");
03129                 static const format fmt8("#    %5s %4s %4s %4s %11s %11s");
03130                 static const format fmt9("mis %05d %4d %4d %4d %11d %11.3f");
03131                 os « "# --- Start-of-Mission Trigger ----------------------------------------------"
03132                    "------------------------"
03133                    « endl;
03134                 os « format(fmt7) % "SN" % "dive" % "idx" % "midx" % "MTD" % "MTT" « endl;
03135                 os « format(fmt8) % "" % "" % "" % "" % "(dBar)" % "(sec)" « endl;
03136                 os « format(fmt9) % whoi % dive % idx % midx++ // midx
03137                            % mtd() % mtt_sec()
03138                    « endl;
03139
03140                 // Minor index 2.
03141                 static const format fmt10("#   %5s %4s %4s %4s %11s %11s %11s %11s %11s");
03142                 static const format fmt11("#    %5s %4s %4s %4s %11s %11s %11s %11s %11s");
03143                 static const format fmt12("mis %05d %4d %4d %4d %11d %11d %11d %11d %11d");
03144                 os « "# --- Wave-Field Measurement -------------------------------------------------"
03145                    "------------------------"
03146                    « endl;
03147                 os « format(fmt10) % "SN" % "dive" % "idx" % "midx" % "MAM" % "MAT" % "ACA" % "ACG"
03148                            % "ACR"
03149                    « endl;
03150                 os « format(fmt11) % "" % "" % "" % "" % "" % "(sec)" % "(g/FSR)" % "(°/sec/FSR)"
03151                            % "(Hz)"
03152                    « endl;
03153                 os « format(fmt12) % whoi % dive % idx % midx++ // midx
03154                            % (int) mam() % mat() % aca() % acg() % acr()
03155                    « endl;
03156
03157                 // Minor index 3.
03158                 static const format fmt13("#   %5s %4s %4s %4s %11s %11s %11s");
03159                 static const format fmt14("#    %5s %4s %4s %4s %11s %11s %11s");
03160                 static const format fmt15("mis %05d %4d %4d %4d %11.2f %11.2f %11.2f");
03161                 os « "# --- Built-In Test (BIT) ------------------------------------------------"
03162                    "------------------------"
03163                    « endl;
03164                 os « format(fmt13) % "SN" % "dive" % "idx" % "midx" % "PNB" % "PNC" % "PXC"
03165                    « endl;
03166                 os « format(fmt14) % "" % "" % "" % "" % "(V)" % "(psi)" % "(psi)" « endl;
03167                 os « format(fmt15) % whoi % dive % idx % midx++ // midx
03168                            % pnb_v() % pnc_psi() % pxc_psi()
03169                    « endl;
03170
03171                 // Minor index 4.
03172                 static const format fmt16("#   %5s %4s %4s %4s %11s %11s %11s %11s %11s %11s");
03173                 static const format fmt17("#    %5s %4s %4s %4s %11s %11s %11s %11s %11s %11s");
03174                 static const format fmt18("mis %05d %4d %4d %4d %11d %11d %11d %11d %11d %11d");
03175                 os « "# --- Calibration -------------------------------------------------------------"
03176                    "------------------------"
03177                    « endl;
03178                 os « format(fmt16) % "SN" % "dive" % "idx" % "midx" % "PBL" % "PBH" % "PP0" % "PP1"
03179                            % "PVL" % "PVH"
03180                    « endl;
03181                 os « format(fmt17) % "" % "" % "" % "" % "" % "(LSB/12V)" % "(LSB/14V)" % "(LSB/0A)"
03182                            % "(LSB/1)A" % "(LSB/12V)" % "(LSB/14V)"
03183                    « endl;
03184                 os « format(fmt18) % whoi % dive % idx % midx++ // midx
03185                            % pbl() % pbh() % pp0() % ppl() % pvl() % pvh()
03186                    « endl;
03187
03188                 // Minor index 5.
```

```
03189                 static const format fmt19("#   %5s %4s %4s %4s %11s %11s %11s %11s");
03190                 static const format fmt20("#    %5s %4s %4s %4s %11s %11s %11s %11s");
03191                 static const format fmt21("mis %05d %4d %4d %4d %11d %11d %11d %11d");
03192                 os « format(fmt19) % "SN" % "dive" % "idx" % "midx" % "PV0" % "PV5" % "PC0" % "PC1"
03193                    « endl;
03194                 os « format(fmt20) % "" % "" % "" % "" % "(LSB/0inHg)" % "(LSB/5inHg)" % "(LSB/0A)"
03195                            % "(LSB/1A)"
03196                    « endl;
03197                 os « format(fmt21) % whoi % dive % idx % midx++ // midx
03198                            % pv0() % pv5() % pc0() % pc1()
03199                    « endl;
03200
03201                 // Minor index 6.
03202                 static const format fmt22("#   %5s %4s %4s %4s %11s %11s %11s %11s %11s %11s");
03203                 static const format fmt23("#    %5s %4s %4s %4s %11s %11s %11s %11s %11s %11s");
03204                 static const format fmt24("mis %05d %4d %4d %4d %11d %11d %11.3f %11d %11d %11d");
03205                 os « "# --- Profile Configuration ------------------------------------------------"
03206                    "-------------------------"
03207                    « endl;
03208                 os « format(fmt22) % "SN" % "dive" % "idx" % "midx" % "CPS" % "CST" % "CSP"
03209                            % "CPB1" % "CPB2" % "CPB3"
03210                    « endl;
03211                 os « format(fmt23) % "" % "" % "" % "" % "" % "" % "(sec)" % "(dBar)" % "(dBar)"
03212                            % "(dBar)"
03213                    « endl;
03214                 os « format(fmt24) % whoi % dive % idx % midx++ // midx
03215                            % (int) cps() % (int) cst() % csp_sec() % cpb1() % cpb2() % cpb3()
03216                    « endl;
03217
03218                 // Minor index 7.
03219                 static const format fmt25("#   %5s %4s %4s %4s %11s %11s %11s %11s %11s %11s");
03220                 static const format fmt26("#    %5s %4s %4s %4s %11s %11s %11s %11s %11s %11s");
03221                 static const format fmt27(
03222                        "mis %05d %4d %4d %4d %11.1f %11.1f %11.1f %11.3f %11.3f %11.3f");
03223                 os « format(fmt25) % "SN" % "dive" % "idx" % "midx" % "CPN1" % "CPN2" % "CPN3"
03224                            % "CPP1" % "CPP2" % "CPP3"
03225                    « endl;
03226                 os « format(fmt26) % "" % "" % "" % "" % "(dBar)" % "(dBar)" % "(dBar)" % "(sec)"
03227                            % "(sec)" % "(sec)"
03228                    « endl;
03229                 os « format(fmt27) % whoi % dive % idx
03230                            % midx++ // midx
03231                            % cpn1_dbar() % cpn2_dbar() % cpn3_dbar() % cpp1_sec() % cpp2_sec()
03232                            % cpp3_sec()
03233                    « endl;
03234
03235                 // Minor index 8.
03236                 static const format fmt28("#   %5s %4s %4s %4s %11s %11s %11s");
03237                 static const format fmt29("#    %5s %4s %4s %4s %11s %11s %11s");
03238                 static const format fmt30("mis %05d %4d %4d %4d %11d %11d %11d");
03239                 os « "# --- Diagnostic Dive ------------------------------------------------------"
03240                    "-------------------------"
03241                    « endl;
03242                 os « format(fmt28) % "SN" % "dive" % "idx" % "midx" % "DDP" % "DDS" % "DDF"
03243                    « endl;
03244                 os « format(fmt29) % "" % "" % "" % "" % "(dBar)" % "(sec)" % "(sec)" « endl;
03245                 os « format(fmt30) % whoi % dive % idx % midx++ // midx
03246                            % ddp() % dds() % ddf()
03247                    « endl;
03248
03249                 // Minor index 9.
03250                 static const format fmt31("#   %5s %4s %4s %4s %11s %11s %11s %11s");
03251                 static const format fmt32("#    %5s %4s %4s %4s %11s %11s %11s %11s");
03252                 static const format fmt33("mis %05d %4d %4d %4d %11d %11d %11d %11d");
03253                 os « "# --- Descent - General ---------------------------------------------------"
03254                    "-------------------------"
03255                    « endl;
03256                 os « format(fmt31) % "SN" % "dive" % "idx" % "midx" % "DNS" % "DTD" % "PFR" % "PFC"
03257                    « endl;
03258                 os « format(fmt32) % "" % "" % "" % "" % "" % "(dBar)" % "(mL/min)" % "(mL/dBar)"
03259                    « endl;
03260                 os « format(fmt33) % whoi % dive % idx % midx++ // midx
03261                            % dns() % dtd() % pfr() % pfc()
03262                    « endl;
03263
03264                 // Minor index 10.
03265                 static const format fmt34("#   %5s %4s %4s %4s %11s %11s");
03266                 static const format fmt35("#    %5s %4s %4s %4s %11s %11s");
03267                 static const format fmt36("mis %05d %4d %4d %4d %11d %11d");
03268                 os « "# --- Descent - Stage 1 (start) ------------------------------------------"
03269                    "-------------------------"
03270                    « endl;
03271                 os « format(fmt34) % "SN" % "dive" % "idx" % "midx" % "DSD" % "DST" « endl;
03272                 os « format(fmt35) % "" % "" % "" % "" % "(dBar)" % "(sec)" « endl;
03273                 os « format(fmt36) % whoi % dive % idx % midx++ // midx
03274                            % dsd() % dst()
03275                    « endl;
03276
03277                 // Minor index 11.
03278                 static const format fmt37("#   %5s %4s %4s %4s %11s %11s %11s %11s");
03279                 static const format fmt38("#    %5s %4s %4s %4s %11s %11s %11s %11s");
03280                 static const format fmt39("mis %05d %4d %4d %4d %11d %11d %11d %11.3f");
03281                 os « "# --- Descent - Stage 2 (fall) ------------------------------------------"
03282                    "-------------------------"
03283                    « endl;
03284                 os « format(fmt37) % "SN" % "dive" % "idx" % "midx" % "DFT" % "DDD" % "DDT" % "DSI"
03285                    « endl;
03286                 os « format(fmt38) % "" % "" % "" % "" % "(sec)" % "(dBar)" % "(sec)" % "(sec)"
03287                    « endl;
03288                 os « format(fmt39) % whoi % dive % idx % midx++ // midx
03289                            % dft() % ddd() % ddt() % dsi_sec()
03290                    « endl;
03291
03292                 // Minor index 12.
03293                 static const format fmt40("#   %5s %4s %4s %4s %11s %11s %11s %11s %11s %11s");
03294                 static const format fmt41("#    %5s %4s %4s %4s %11s %11s %11s %11s %11s %11s");
03295                 static const format fmt42("mis %05d %4d %4d %4d %11d %11d %11d %11d %11d %11d");
03296                 os « "# --- Descent - Stage 3 (calibration) ---------------------------------------"
```

```
03297                   "------------------------"
03298                   « endl;
03299                os « format(fmt40) % "SN" % "dive" % "idx" % "midx" % "DCT" % "DCM" % "DAL" % "DCA"
03300                           % "DSW" % "DUC"
03301                   « endl;
03302                os « format(fmt41) % "" % "" % "" % "" % "(sec)" % "" % "" % "" % "(sec)" % ""
03303                   « endl;
03304                os « format(fmt42) % whoi % dive % idx % midx++ // midx
03305                           % dct() % (int) dcm() % (int) dal() % (int) dca() % dsw() % duc()
03306                   « endl;
03307
03308                // Minor index 13.
03309                static const format fmt43("#   %5s %4s %4s %4s %11s %11s %11s %11s %11s %11s");
03310                static const format fmt44("#    %5s %4s %4s %4s %11s %11s %11s %11s %11s %11s");
03311                static const format fmt45("mis %05d %4d %4d %4d %11d %11d %11d %11d %11d %11d");
03312                os « format(fmt43) % "SN" % "dive" % "idx" % "midx" % "DPT" % "DUT" % "DVT" % "DCX"
03313                           % "DCP" % "DCF"
03314                   « endl;
03315                os « format(fmt44) % "" % "" % "" % "" % "(sec)" % "(sec)" % "(sec)" % "(sec)"
03316                           % "(sec)" % "(sec)"
03317                   « endl;
03318                os « format(fmt45) % whoi % dive % idx % midx++ // midx
03319                           % dpt() % dut() % dvt() % dcx() % dcp() % dcf()
03320                   « endl;
03321
03322                // Minor index 14.
03323                static const format fmt46("#   %5s %4s %4s %4s %11s %11s %11s %11s %11s %11s");
03324                static const format fmt47("#    %5s %4s %4s %4s %11s %11s %11s %11s %11s %11s");
03325                static const format fmt48("mis %05d %4d %4d %4d %11d %11d  0x%08X %11d %11.3f %11d");
03326                os « "# --- Park/Drift - General ------------------------------------------------"
03327                   "------------------------"
03328                   « endl;
03329                os « format(fmt46) % "SN" % "dive" % "idx" % "midx" % "PNS" % "PTO" % "PTH" % "PSA"
03330                           % "PSI" % "PFM"
03331                   « endl;
03332                os « format(fmt47) % "" % "" % "" % "" % "" % "(sec)" % "" % "" % "(sec)" % ""
03333                   « endl;
03334                os « format(fmt48) % whoi % dive % idx % midx++ // midx
03335                           % pns() % pto() % pth() % (int) psa() % psi_sec() % (int) pfm()
03336                   « endl;
03337
03338                // Minor index 15.
03339                static const format fmt49("#   %5s %4s %4s %4s %11s %11s");
03340                static const format fmt50("#    %5s %4s %4s %4s %11s %11s");
03341                static const format fmt51("mis %05d %4d %4d %4d %11d %11d");
03342                os « format(fmt49) % "SN" % "dive" % "idx" % "midx" % "PFD" % "PDT" « endl;
03343                os « format(fmt50) % "" % "" % "" % "" % "(dBar)" % "(sec)" « endl;
03344                os « format(fmt51) % whoi % dive % idx % 15 % pfd() % pdt() « endl;
03345
03346                // Minor index 16.
03347                static const format fmt52("#   %5s %4s %4s %4s %11s %11s %11s %11s %11s");
03348                static const format fmt53("#    %5s %4s %4s %4s %11s %11s %11s %11s %11s");
03349                static const format fmt54("mis %05d %4d %4d %4d %11s %11s %11s %11s %11s");
03350                os « "# --- Park/Drift - Seek ------------------------------------------------"
03351                   "------------------------"
03352                   « endl;
03353                os « format(fmt52) % "SN" % "dive" % "idx" % "midx" % "PSM" % "PST" % "PSD" % "PSV"
03354                           % "PSW"
03355                   « endl;
03356                os « format(fmt53) % "" % "" % "" % "" % "" % "(sec)" % "(dBar)" % "(sec)"
03357                           % "(sec)"
03358                   « endl;
03359                os « format(fmt54) % whoi % dive % idx % midx++ // midx
03360                           % (int) psm() % pst() % psd() % psv() % psw()
03361                   « endl;
03362
03363                // Minor index 17.
03364                static const format fmt55("#   %5s %4s %4s %4s %11s %11s %11s %11s %11s %11s");
03365                static const format fmt56("#    %5s %4s %4s %4s %11s %11s %11s %11s %11s %11s");
03366                static const format fmt57("mis %05d %4d %4d %4d %11d %11.2f %11d %11d %11d %11.3f");
03367                os « "# --- Ascent/Profile ---------------------------------------------------"
03368                   "------------------------"
03369                   « endl;
03370                os « format(fmt55) % "SN" % "dive" % "idx" % "midx" % "ASN" % "ARR" % "APM" % "APN"
03371                           % "APF" % "ASI"
03372                   « endl;
03373                os « format(fmt56) % "" % "" % "" % "" % "" % "(m/sec)" % "(sec)" % "(sec)"
03374                           % "(sec)" % "(sec)"
03375                   « endl;
03376                os « format(fmt57) % whoi % dive % idx % midx++ // midx
03377                           % asn() % arr_m_per_s() % apm() % apn() % apf() % asi_sec()
03378                   « endl;
03379
03380                // Minor index 18.
03381                static const format fmt58("#   %5s %4s %4s %4s %11s %11s %11s %11s %11s %11s");
03382                static const format fmt59("#    %5s %4s %4s %4s %11s %11s %11s %11s %11s %11s");
03383                static const format fmt60("mis %05d %4d %4d %4d %11d %11d %11d %11d %11.3f %11d");
03384                os « format(fmt58) % "SN" % "dive" % "idx" % "midx" % "ASD" % "ADD" % "ADT" % "ARC"
03385                           % "AHR" % "AHD"
03386                   « endl;
03387                os « format(fmt59) % "" % "" % "" % "" % "(dBar)" % "(dBar)" % "(sec)" % "(?)"
03388                           % "(m/sec)" % "(dBar)"
03389                   « endl;
03390                os « format(fmt60) % whoi % dive % idx % midx++ // midx
03391                           % asd() % add() % adt() % arc() % ahr_m_per_s() % ahd()
03392                   « endl;
03393
03394                // Minor index 19.
03395                static const format fmt61("#   %5s %4s %4s %4s %11s %11s %11s %11s %11s %11s");
03396                static const format fmt62("#    %5s %4s %4s %4s %11s %11s %11s %11s %11s %11s");
03397                static const format fmt63("mis %05d %4d %4d %4d %11d %11d %11d %11d %11d %11.3f");
03398                os « "# --- Surface -----------------------------------------------------------"
03399                   "------------------------"
03400                   « endl;
03401
03402                os « format(fmt61) % "SN" % "dive" % "idx" % "midx" % "SFN" % "SGT" % "SET" % "SDT"
03403                           % "SSA" % "SSI"
03404                   « endl;
```

```
03405                os « format(fmt62) % "" % "" % "" % "" % "" % "(min)" % "(min)" % "(?)" % ""
03406                           % "(sec)"
03407                   « endl;
03408                os « format(fmt63) % whoi % dive % idx % midx++ // midx
03409                           % sfn() % sgt() % set() % sdt() % ssa() % ssi_sec()
03410                   « endl;
03411
03412                // Minor index 20.
03413                static const format fmt64("#   %5s %4s %4s %4s %11s %11s");
03414                static const format fmt65("#    %5s %4s %4s %4s %11s %11s");
03415                static const format fmt66("mis %05d %4d %4d %4d %11d %11d");
03416
03417                os « format(fmt64) % "SN" % "dive" % "idx" % "midx" % "SDS" % "SPT" « endl;
03418                os « format(fmt65) % "" % "" % "" % "" % "(sec)" % "(sec)" « endl;
03419                os « format(fmt66) % whoi % dive % idx % midx++ // midx
03420                           % sds() % spt()
03421                   « endl;
03422
03423                // Minor index 21.
03424                static const format fmt67("#   %5s %4s %4s %4s %11s %11s %11s %11s %11s");
03425                static const format fmt68("#    %5s %4s %4s %4s %11s %11s %11s %11s %11s");
03426                static const format fmt69("mis %05d %4d %4d %4d %11d %11.3f %11.3f %11d %11d");
03427                os « "# --- Surface - Air Pump ------------------------------------------------"
03428                   "------------------------"
03429                   « endl;
03430
03431                os « format(fmt67) % "SN" % "dive" % "idx" % "midx" % "SAM" % "SAX" % "SAN" % "SAO"
03432                           % "SAP"
03433                   « endl;
03434                os « format(fmt68) % "" % "" % "" % "" % "" % "(psi)" % "(psi)" % "(sec)" % "(sec)"
03435                   « endl;
03436                os « format(fmt69) % whoi % dive % idx % midx++ // midx
03437                           % (int) sam() % sax_psi() % san_psi() % sao() % sap()
03438                   « endl;
03439
03440                // Minor index 22.
03441                static const format fmt70("#   %5s %4s %4s %4s %11s %11s");
03442                static const format fmt71("#    %5s %4s %4s %4s %11s %11s");
03443                static const format fmt72("mis %05d %4d %4d %4d %11d %11d");
03444                os « "# --- Surface - Abort/Beacon Mode ---------------------------------------"
03445                   "------------------------"
03446                   « endl;
03447                os « format(fmt70) % "SN" % "dive" % "idx" % "midx" % "SBM" % "SBT" « endl;
03448                os « format(fmt71) % "" % "" % "" % "" % "" % "(sec)" « endl;
03449                os « format(fmt72) % whoi % dive % idx % midx++ // midx
03450                           % (int) sbm() % sbt()
03451                   « endl;
03452            }
03453
03454                break;
03455
03456            case 0x02006900: // ALAMO Firmware version 0x02006900.
03457            case 0x02036200: // ALAMO Firmware version 0x02036200.
03458            case 0x02046200: // ALAMO Firmware version 0x02046200.
03459            case 0x02056200: // ALAMO Firmware version 0x02056200.
03460            case 0x03006500: // ALAMO Firmware version 0x03006500.
03461            case 0x03016700: // ALAMO Software version 0x03016700 (Solo-II Hybrid test float #10083).
03462            case 0x03026100: // ALAMO Software version 0x03026100 (2016 NASCar floats).
03463            case 0x04016100: // ALAMO Software version 0x04016100 (2016 NASCar floats).
03464
03465            {
03466
03467                {   // Header lines.
03468                    static const format fmt1("#   %5s %4s %4s %16s");
03469                    static const format fmt3("mis %05d %4d %4d       0x%08X");
03470                    os « format(fmt1) % "SN" % "dive" % "idx" % "SWVer" « endl;
03471                    os « format(fmt3) % whoi % dive % idx % fw_version() « endl;
03472                }
03473
03474                static const format fmt4("#   %5s %4s %4s %4s %11s %11s %11s %11s %11s");
03475                static const format fmt5("#    %5s %4s %4s %4s %11s %11s %11s %11s %11s");
03476                static const format fmt6("mis %05d %4d %4d %4d %11d %11d %11d %11d %11d");
03477                os « format(fmt4) % "SN" % "dive" % "idx" % "midx" % "MSN" % "DTD" % "MDS" % "APT"
03478                           % "XAS"
03479                   « endl;
03480                os « format(fmt5) % "" % "" % "" % "" % "" % "(dBar)" % "(dBar)" % "" % "" « endl;
03481                os « format(fmt6) % whoi % dive % idx % midx++ // midx
03482                           % msn() % dtd() % mds() % (int) apt() % (int) xas()
03483                   « endl;
03484
03485                static const format fmt7("#   %5s %4s %4s %4s %11s %11s");
03486                static const format fmt8("#    %5s %4s %4s %4s %11s %11s");
03487                static const format fmt9("mis %05d %4d %4d %4d %11d %11.3f");
03488                os « "# --- Start-of-Mission Trigger ---------------------------------------------"
03489                   "------------------------"
03490                   « endl;
03491                os « format(fmt7) % "SN" % "dive" % "idx" % "midx" % "MTD" % "MTT" « endl;
03492                os « format(fmt8) % "" % "" % "" % "" % "(dBar)" % "(sec)" « endl;
03493                os « format(fmt9) % whoi % dive % idx % midx++ // midx
03494                           % mtd() % mtt_sec()
03495                   « endl;
03496
03497                static const format fmt28("#   %5s %4s %4s %4s %11s %11s %11s");
03498                static const format fmt29("#    %5s %4s %4s %4s %11s %11s %11s");
03499                static const format fmt30("mis %05d %4d %4d %4d %11d %11d %11d");
03500                os « "# --- Diagnostic Dive ------------------------------------------------------"
03501                   "------------------------"
03502                   « endl;
03503                os « format(fmt28) % "SN" % "dive" % "idx" % "midx" % "DDP" % "DDS" % "DDF"
03504                   « endl;
03505                os « format(fmt29) % "" % "" % "" % "" % "" % "(dBar)" % "(sec)" % "(sec)" « endl;
03506                os « format(fmt30) % whoi % dive % idx % midx++ // midx
03507                           % ddp() % dds() % ddf()
03508                   « endl;
03509
03510                static const format fmt13("#   %5s %4s %4s %4s %11s %11s %11s");
03511                static const format fmt14("#    %5s %4s %4s %4s %11s %11s %11s");
03512                static const format fmt15("mis %05d %4d %4d %4d %11.2f %11.2f %11.2f");
```

```
03513                os « "# --- Built-In Test (BIT) ------------------------------------------------"
03514                   "-------------------------"
03515                   « endl;
03516                os « format(fmt13) % "SN" % "dive" % "idx" % "midx" % "PNB" % "PNC" % "PXC"
03517                   « endl;
03518                os « format(fmt14) % "" % "" % "" % "" % "(V)" % "(psi)" % "(psi)" « endl;
03519                os « format(fmt15) % whoi % dive % idx % midx++ // midx
03520                           % pnb_v() % pnc_psi() % pxc_psi()
03521                   « endl;
03522
03523                static const format fmt16("#   %5s %4s %4s %4s %11s %11s %11s %11s %11s %11s");
03524                static const format fmt17("#    %5s %4s %4s %4s %11s %11s %11s %11s %11s %11s");
03525                static const format fmt18("mis %05d %4d %4d %4d %11d %11d %11d %11d %11d %11d");
03526                os « "# --- Calibration -----------------------------------------------------------"
03527                   "-------------------------"
03528                   « endl;
03529                os « format(fmt16) % "SN" % "dive" % "idx" % "midx" % "PBL" % "PBH" % "PP0" % "PP1"
03530                           % "PVL" % "PVH"
03531                   « endl;
03532                os « format(fmt17) % "" % "" % "" % "" % "(LSB/12V)" % "(LSB/14V)" % "(LSB/0A)"
03533                           % "(LSB/1A)" % "(LSB/12V)" % "(LSB/14V)"
03534                   « endl;
03535                os « format(fmt18) % whoi % dive % idx % midx++ // midx
03536                           % pbl() % pbh() % pp0() % pp1() % pv1() % pvh()
03537                   « endl;
03538
03539                // Minor index 5.
03540                static const format fmt19("#   %5s %4s %4s %4s %11s %11s %11s %11s");
03541                static const format fmt20("#    %5s %4s %4s %4s %11s %11s %11s %11s");
03542                static const format fmt21("mis %05d %4d %4d %4d %11d %11d %11d %11d");
03543                os « format(fmt19) % "SN" % "dive" % "idx" % "midx" % "PV0" % "PV5" % "PC0" % "PC1"
03544                   « endl;
03545                os « format(fmt20) % "" % "" % "" % "" % "(LSB/0inHg)" % "(LSB/5inHg)" % "(LSB/0A)"
03546                           % "(LSB/1A)"
03547                   « endl;
03548                os « format(fmt21) % whoi % dive % idx % midx++ // midx
03549                           % pv0() % pv5() % pc0() % pc1()
03550                   « endl;
03551
03552                static const format fmt22("#   %5s %4s %4s %4s %11s %11s %11s %11s %11s %11s");
03553                static const format fmt23("#    %5s %4s %4s %4s %11s %11s %11s %11s %11s %11s");
03554                static const format fmt24("mis %05d %4d %4d %4d %11d %11d %11.3f %11d %11d %11d");
03555                os « "# --- Profile Configuration -------------------------------------------------"
03556                   "-------------------------"
03557                   « endl;
03558                os « format(fmt22) % "SN" % "dive" % "idx" % "midx" % "CPS" % "CST" % "CSP"
03559                           % "CPB1" % "CPB2" % "CPB3"
03560                   « endl;
03561                os « format(fmt23) % "" % "" % "" % "" % "" % "" % "(sec)" % "(dBar)" % "(dBar)"
03562                           % "(dBar)"
03563                   « endl;
03564                os « format(fmt24) % whoi % dive % idx % midx++ // midx
03565                           % (int) cps() % (int) cst() % csp_sec() % cpb1() % cpb2() % cpb3()
03566                   « endl;
03567
03568                static const format fmt25("#   %5s %4s %4s %4s %11s %11s %11s %11s %11s %11s");
03569                static const format fmt26("#    %5s %4s %4s %4s %11s %11s %11s %11s %11s %11s");
03570                static const format fmt27(
03571                        "mis %05d %4d %4d %4d %11.1f %11.1f %11.1f %11.3f %11.3f %11.3f");
03572                os « format(fmt25) % "SN" % "dive" % "idx" % "midx" % "CPN1" % "CPN2" % "CPN3"
03573                           % "CPP1" % "CPP2" % "CPP3"
03574                   « endl;
03575                os « format(fmt26) % "" % "" % "" % "" % "(dBar)" % "(dBar)" % "(dBar)" % "(sec)"
03576                           % "(sec)" % "(sec)"
03577                   « endl;
03578                os « format(fmt27) % whoi % dive % idx
03579                           % midx++ // midx
03580                           % cpn1_dbar() % cpn2_dbar() % cpn3_dbar() % cpp1_sec() % cpp2_sec()
03581                           % cpp3_sec()
03582                   « endl;
03583
03584                static const format fmt73("#   %5s %4s %4s %4s "
03585                        "%11s %11s %11s");
03586                static const format fmt74("#    %5s %4s %4s %4s "
03587                        "%11s %11s %11s");
03588                static const format fmt75("mis %05d %4d %4d %4d "
03589                        "%11d %11d %11d");
03590                os « format(fmt73) % "SN" % "dive" % "idx" % "midx" //
03591                           % "CTF" % "CTL" % "CTU"
03592                   « endl;
03593                os « format(fmt74) % "" % "" % "" % "" //
03594                           % "(dBar)" % "(dBar)" % "(dBar)"
03595                   « endl;
03596                os « format(fmt75) % whoi % dive % idx % midx++ // midx
03597                           % ctf() % ctl() % ctu()
03598                   « endl;
03599
03600                static const format fmt31("#   %5s %4s %4s %4s %11s %11s %11s %11s");
03601                static const format fmt32("#    %5s %4s %4s %4s %11s %11s %11s %11s");
03602                static const format fmt33("mis %05d %4d %4d %4d %11d %11d %11d %11d");
03603                os « "# --- Descent - General ------------------------------------------------------"
03604                   "-------------------------"
03605                   « endl;
03606                os « format(fmt31) % "SN" % "dive" % "idx" % "midx" % "DNS" % "DTD" % "PFR" % "PFC"
03607                   « endl;
03608                os « format(fmt32) % "" % "" % "" % "" % "" % "(dBar)" % "(mL/min)" % "(mL/dBar)"
03609                   « endl;
03610                os « format(fmt33) % whoi % dive % idx % midx++ // midx
03611                           % dns() % dtd() % pfr() % pfc()
03612                   « endl;
03613
03614                static const format fmt34("#   %5s %4s %4s %4s %11s %11s");
03615                static const format fmt35("#    %5s %4s %4s %4s %11s %11s");
03616                static const format fmt36("mis %05d %4d %4d %4d %11d %11d");
03617                os « "# --- Descent - Stage 1 (start) ----------------------------------------------"
03618                   "-------------------------"
03619                   « endl;
03620                os « format(fmt34) % "SN" % "dive" % "idx" % "midx" % "DSD" % "DST" « endl;
```

```
03621                os « format(fmt35) % "" % "" % "" % "" % "(dBar)" % "(sec)" « endl;
03622                os « format(fmt36) % whoi % dive % idx % midx++ // midx
03623                           % dsd() % dst()
03624                   « endl;
03625
03626                static const format fmt37("#   %5s %4s %4s %4s %11s %11s %11s %11s");
03627                static const format fmt38("#    %5s %4s %4s %4s %11s %11s %11s %11s");
03628                static const format fmt39("mis %05d %4d %4d %4d %11d %11d %11d %11.3f");
03629                os « "# --- Descent - Stage 2 (fall) ----------------------------------------------"
03630                   "------------------------"
03631                   « endl;
03632                os « format(fmt37) % "SN" % "dive" % "idx" % "midx" % "DFT" % "DDD" % "DDT" % "DSI"
03633                   « endl;
03634                os « format(fmt38) % "" % "" % "" % "" % "(sec)" % "(dBar)" % "(sec)" % "(sec)"
03635                   « endl;
03636                os « format(fmt39) % whoi % dive % idx % midx++ // midx
03637                           % dft() % ddd() % ddt() % dsi_sec()
03638                   « endl;
03639
03640                static const format fmt40("#   %5s %4s %4s %4s %11s %11s %11s %11s %11s %11s");
03641                static const format fmt41("#    %5s %4s %4s %4s %11s %11s %11s %11s %11s %11s");
03642                static const format fmt42("mis %05d %4d %4d %4d %11d %11d %11d %11d %11d %11d");
03643                os « "# --- Descent - Stage 3 (calibration) -------------------------------------"
03644                   "------------------------"
03645                   « endl;
03646                os « format(fmt40) % "SN" % "dive" % "idx" % "midx" % "DCT" % "DCM" % "DAL" % "DCA"
03647                           % "DSW" % "DUC"
03648                   « endl;
03649                os « format(fmt41) % "" % "" % "" % "" % "(sec)" % "" % "" % "" % "(sec)" % ""
03650                   « endl;
03651                os « format(fmt42) % whoi % dive % idx % midx++ // midx
03652                           % dct() % (int) dcm() % (int) dal() % (int) dca() % dsw() % duc()
03653                   « endl;
03654
03655                static const format fmt43("#   %5s %4s %4s %4s %11s %11s %11s %11s %11s %11s");
03656                static const format fmt44("#    %5s %4s %4s %4s %11s %11s %11s %11s %11s %11s");
03657                static const format fmt45("mis %05d %4d %4d %4d %11d %11d %11d %11d %11d %11d");
03658                os « format(fmt43) % "SN" % "dive" % "idx" % "midx" % "DPT" % "DUT" % "DVT" % "DCX"
03659                           % "DCP" % "DCF"
03660                   « endl;
03661                os « format(fmt44) % "" % "" % "" % "" % "(sec)" % "(sec)" % "(sec)" % "(sec)"
03662                           % "(sec)" % "(sec)"
03663                   « endl;
03664                os « format(fmt45) % whoi % dive % idx % midx++ // midx
03665                           % dpt() % dut() % dvt() % dcx() % dcp() % dcf()
03666                   « endl;
03667
03668                static const format fmt46("#   %5s %4s %4s %4s %11s %11s %11s %11s %11s %11s");
03669                static const format fmt47("#    %5s %4s %4s %4s %11s %11s %11s %11s %11s %11s");
03670                static const format fmt48("mis %05d %4d %4d %4d %11d %11d  0x%08X %11d %11.3f %11d");
03671                os « "# --- Park/Drift - General -------------------------------------------------"
03672                   "------------------------"
03673                   « endl;
03674                os « format(fmt46) % "SN" % "dive" % "idx" % "midx" % "PNS" % "PTO" % "PTH" % "PSA"
03675                           % "PSI" % "PFM"
03676                   « endl;
03677                os « format(fmt47) % "" % "" % "" % "" % "" % "(sec)" % "" % "" % "(sec)" % ""
03678                   « endl;
03679                os « format(fmt48) % whoi % dive % idx % midx++ // midx
03680                           % pns() % pto() % pth() % (int) psa() % psi_sec() % (int) pfm()
03681                   « endl;
03682
03683                static const format fmt49("#   %5s %4s %4s %4s %11s %11s");
03684                static const format fmt50("#    %5s %4s %4s %4s %11s %11s");
03685                static const format fmt51("mis %05d %4d %4d %4d %11d %11d");
03686                os « format(fmt49) % "SN" % "dive" % "idx" % "midx" % "PFD" % "PDT" « endl;
03687                os « format(fmt50) % "" % "" % "" % "" % "(dBar)" % "(sec)" « endl;
03688                os « format(fmt51) % whoi % dive % idx % midx++ // midx
03689                           % pfd() % pdt()
03690                   « endl;
03691
03692                static const format fmt52("#   %5s %4s %4s %4s %11s %11s %11s %11s %11s");
03693                static const format fmt53("#    %5s %4s %4s %4s %11s %11s %11s %11s %11s");
03694                static const format fmt54("mis %05d %4d %4d %4d %11s %11s %11s %11s %11s");
03695                os « "# --- Park/Drift - Seek ------------------------------------------------------"
03696                   "------------------------"
03697                   « endl « format(fmt52) % "SN" % "dive" % "idx" % "midx" % "PSM" % "PST" % "PSD"
03698                           % "PSV" % "PSW"
03699                   « endl « format(fmt53) % "" % "" % "" % "" % "" % "(sec)" % "(dBar)" % "(sec)"
03700                           % "(sec)"
03701                   « endl « format(fmt54) % whoi % dive % idx % midx++ // midx
03702                           % (int) psm() % pst() % psd() % psv() % psw()
03703                   « endl;
03704
03705                static const format fmt55("#   %5s %4s %4s %4s %11s %11s %11s %11s %11s %11s");
03706                static const format fmt56("#    %5s %4s %4s %4s %11s %11s %11s %11s %11s %11s");
03707                static const format fmt57("mis %05d %4d %4d %4d %11d %11.2f %11d %11d %11d %11.3f");
03708                os « "# --- Ascent/Profile ------------------------------------------------------"
03709                   "------------------------"
03710                   « endl « format(fmt55) % "SN" % "dive" % "idx" % "midx" % "ASN" % "ARR" % "APM"
03711                           % "APN" % "APF" % "ASI"
03712                   « endl « format(fmt56) % "" % "" % "" % "" % "" % "(m/sec)" % "(sec)" % "(sec)"
03713                           % "(sec)" % "(sec)"
03714                   « endl « format(fmt57) % whoi % dive % idx % midx++  // midx
03715                           % asn() % arr_m_per_s() % apm() % apn() % apf() % asi_sec()
03716                   « endl;
03717
03718                {
03719                    static const format fmt1("#   %5s %4s %4s %4s "
03720                            "%11s %11s "
03721                            "%11s %11s "
03722                            "%11s %11s");
03723                    static const format fmt2("#    %5s %4s %4s %4s "
03724                            "%11s %11s "
03725                            "%11s %11s "
03726                            "%11s %11s");
03727                    static const format fmt3("mis %05d %4d %4d %4d %11d "
03728                            "%11d %11d "
```

```
03729                           "%11d %11.3f "
03730                           "%11d");
03731                   os « format(fmt1) % "SN" % "dive" % "idx" % "midx"  //
03732                           % "ASD" % "ADD" //
03733                           % "ADT" % "ARC" //
03734                           % "AHR" % "AHD"
03735                      « endl « format(fmt2) % "" % "" % "" % "" //
03736                           % "(dBar)" % "(dBar)" //
03737                           % "(sec)" % "(?)" //
03738                           % "(m/sec)" % "(dBar)"
03739                      « endl « format(fmt3) % whoi % dive % idx % midx++ // midx
03740                           % asd() % add() //
03741                           % adt() % arc() //
03742                           % ahr_m_per_s() % ahd()
03743                      « endl;
03744               }
03745
03746               {
03747                   static const format fmt1("#   %5s %4s %4s %4s "
03748                           "%11s %11s "
03749                           "%11s %11s "
03750                           "%11s %11s");
03751                   static const format fmt2("#    %5s %4s %4s %4s "
03752                           "%11s %11s "
03753                           "%11s %11s "
03754                           "%12s %11s");
03755                   static const format fmt3("mis %05d %4d %4d %4d "
03756                           "%11d      0x%04X "
03757                           "%11d %11d "
03758                           "%11.2f %11d");
03759                   os « "# --- Ascent - Ice-Detection ----------------------------------------"
03760                      "---------------------------"
03761                      « endl « format(fmt1) % "SN" % "dive" % "idx" % "midx" //
03762                           % "ICM" % "IMM" //
03763                           % "ICN" % "ICX" //
03764                           % "ICT" % "ICS"
03765                      « endl « format(fmt2) % "" % "" % "" % "" //
03766                           % "" % "" //
03767                           % "(dBar)" % "(dBar)" //
03768                           % "(degC)" % "(sec)"
03769                      « endl « format(fmt3) % whoi % dive % idx % midx++ // midx
03770                           % (int) icm() % imm() //
03771                           % icn() % icx() //
03772                           % ict_degc() % ics()
03773                      « endl;
03774               }
03775
03776               {
03777                   static const format fmt1("#   %5s %4s %4s %4s "
03778                           "%11s %11s "
03779                           "%11s %11s "
03780                           "%11s %11s");
03781                   static const format fmt2("#    %5s %4s %4s %4s "
03782                           "%11s %11s "
03783                           "%11s %11s "
03784                           "%11s %11s");
03785                   static const format fmt3("mis %05d %4d %4d %4d "
03786                           "%11d %11d "
03787                           "%11.2f %11d "
03788                           " 0x%08X  0x%08X");
03789                   os « format(fmt1) % "SN" % "dive" % "idx" % "midx" //
03790                           % "ISP" % "ISM" //
03791                           % "IST" % "ISO" //
03792                           % "ICF" % "ICW"
03793                      « endl « format(fmt2) % "" % "" % "" % "" //
03794                           % "" % "" //
03795                           % "(PSU)" % "(sec)" //
03796                           % "" % "" //
03797                      « endl « format(fmt3) % whoi % dive % idx % midx++ // midx
03798                           % (int) isp() % (int) ism() //
03799                           % ist_psu() % iso() //
03800                           % icf() % icw()
03801                      « endl;
03802               }
03803
03804               {
03805                   static const format fmt1("#   %5s %4s %4s %4s "
03806                           "%11s %11s "
03807                           "%11s %11s "
03808                           "%11s %11s");
03809                   static const format fmt2("#    %5s %4s %4s %4s "
03810                           "%11s %11s "
03811                           "%11s %11s "
03812                           "%11s %11s");
03813                   static const format fmt3("mis %05d %4d %4d %4d "
03814                           "%11d %11d "
03815                           "%11d %11d "
03816                           "%11d %11d");
03817                   os « "# --- Ascent - Ice-Detection (Summer) ---------------------------------"
03818                      "---------------------------"
03819                      « endl « format(fmt1) % "SN" % "dive" % "idx" % "midx" //
03820                           % "ISS0" % "ICA0" //
03821                           % "ICR0" % "ICD0" //
03822                           % "ICO0" % "ICG0" //
03823                      « endl « format(fmt2) % "" % "" % "" % "" //
03824                           % "" % "" //
03825                           % "(min)" % "(dBar)" //
03826                           % "(sec)" % "" //
03827                      « endl « format(fmt3) % whoi % dive % idx % midx++ // midx
03828                           % (int) iss0() % ica0() //
03829                           % icr0() % icd0() //
03830                           % ico0() % (int) icg0()
03831                      « endl;
03832               }
03833
03834               {
03835                   static const format fmt1("#   %5s %4s %4s %4s "
03836                           "%11s %11s "
```

```
03837                           "%11s %11s");
03838                       static const format fmt2("#    %5s %4s %4s %4s "
03839                           "%11s %11s "
03840                           "%11s %11s");
03841                       static const format fmt3("mis %05d %4d %4d %4d "
03842                           "%11d %11d "
03843                           "%11d %11d");
03844                       os « format(fmt1) % "SN" % "dive" % "idx" % "midx" //
03845                           % "IUE0" % "ICB" //
03846                           % "IMT0" % "ILT0" //
03847                          « endl « format(fmt2) % "" % "" % "" % "" //
03848                           % "" % "" //
03849                           % "(%)" % "(sec)" //
03850                          « endl « format(fmt3) % whoi % dive % idx % midx++ // midx
03851                           % (int) iue0() % icb0() //
03852                           % imt0() % ilt0() //
03853                          « endl;
03854                   }
03855
03856                   {
03857                       static const format fmt1("#   %5s %4s %4s %4s "
03858                           "%11s %11s "
03859                           "%11s %11s "
03860                           "%11s %11s");
03861                       static const format fmt2("#    %5s %4s %4s %4s "
03862                           "%11s %11s "
03863                           "%11s %11s "
03864                           "%11s %11s");
03865                       static const format fmt3("mis %05d %4d %4d %4d "
03866                           "%11d %11d "
03867                           "%11d %11d "
03868                           "%11d %11d");
03869                       os « "# --- Ascent - Ice-Detection (Winter) ----------------------------------"
03870                          "----------------------------"
03871                          « endl « format(fmt1) % "SN" % "dive" % "idx" % "midx" //
03872                           % "ISS1" % "ICA1" //
03873                           % "ICR1" % "ICD1" //
03874                           % "ICO1" % "ICG1" //
03875                          « endl « format(fmt2) % "" % "" % "" % "" //
03876                           % "" % "" //
03877                           % "(min)" % "(dBar)" //
03878                           % "(sec)" % "" //
03879                          « endl « format(fmt3) % whoi % dive % idx % midx++ // midx
03880                           % (int) iss1() % ica1() //
03881                           % icr1() % icd1() //
03882                           % ico1() % (int) icg1()
03883                          « endl;
03884                   }
03885
03886                   {
03887                       static const format fmt1("#   %5s %4s %4s %4s "
03888                           "%11s %11s "
03889                           "%11s %11s");
03890                       static const format fmt2("#    %5s %4s %4s %4s "
03891                           "%11s %11s "
03892                           "%11s %11s");
03893                       static const format fmt3("mis %05d %4d %4d %4d "
03894                           "%11d %11d "
03895                           "%11d %11d");
03896                       os « format(fmt1) % "SN" % "dive" % "idx" % "midx" //
03897                           % "IUE1" % "ICB1" //
03898                           % "IMT1" % "ILT1" //
03899                          « endl « format(fmt2) % "" % "" % "" % "" //
03900                           % "" % "" //
03901                           % "(%)" % "(sec)" //
03902                          « endl « format(fmt3) % whoi % dive % idx % midx++ // midx
03903                           % (int) iue1() % icb1() //
03904                           % imt1() % ilt1() //
03905                          « endl;
03906                   }
03907
03908                   {
03909                       static const format fmt1("#   %5s %4s %4s %4s "
03910                           "%11s %11s "
03911                           "%11s %11s "
03912                           "%11s %11s");
03913                       static const format fmt2("#    %5s %4s %4s %4s "
03914                           "%11s %11s "
03915                           "%11s %11s "
03916                           "%11s %11s");
03917                       static const format fmt3("mis %05d %4d %4d %4d "
03918                           "%11d %11d "
03919                           "%11d %11d "
03920                           "%11d %11.3f");
03921                       os « "# --- Surface ------------------------------------------------------------"
03922                          "------------------------"
03923                          « endl
03924                          « format(fmt1) % "SN" % "dive" % "idx" % "midx" //
03925                           % "SFN" % "SGT" //
03926                           % "SET" % "SDT" //
03927                           % "SSA" % "SSI"
03928                          « endl
03929                          « format(fmt2) % "" % "" % "" % "" //
03930                           % "" % "(min)" //
03931                           % "(min)" % "(?)" //
03932                           % "" % "(sec)"
03933                          « endl
03934                          « format(fmt3) % whoi % dive % idx % midx++ // midx
03935                           % sfn() % sgt() //
03936                           % set() % sdt() //
03937                           % ssa() % ssi_sec()
03938                          « endl;
03939
03940                   }
03941
03942                   {
03943                       static const format fmt1("#   %5s %4s %4s %4s "
03944                           "%11s %11s");
```

```
03945                   static const format fmt2("#    %5s %4s %4s %4s "
03946                           "%11s %11s");
03947                   static const format fmt3("mis %05d %4d %4d %4d "
03948                           "%11d %11d");
03949                   os « format(fmt1) % "SN" % "dive" % "idx" % "midx" //
03950                           % "SDS" % "SPT"
03951                      « endl;
03952                   os « format(fmt2) % "" % "" % "" % "" //
03953                           % "(sec)" % "(sec)"
03954                      « endl;
03955                   os « format(fmt3) % whoi % dive % idx % midx++ // midx
03956                           % sds() % spt()
03957                      « endl;
03958               }
03959
03960               {
03961                   static const format fmt1("#   %5s %4s %4s %4s "
03962                           "%11s %11s %11s "
03963                           "%11s %11s");
03964                   static const format fmt2("#    %5s %4s %4s %4s "
03965                           "%11s %11s %11s "
03966                           "%11s %11s");
03967                   static const format fmt3("mis %05d %4d %4d %4d "
03968                           "%11d %11.3f %11.3f "
03969                           "%11d %11d");
03970                   os « "# --- Surface - Air Pump ----------------------------------------------"
03971                      "----------------------------"
03972                      « endl « format(fmt1) % "SN" % "dive" % "idx" % "midx" //
03973                           % "SAM" % "SAX" % "SAN" //
03974                           % "SAO" % "SAP" //
03975                      « endl « format(fmt2) % "" % "" % "" % "" //
03976                           % "" % "(psi)" % "(psi)" //
03977                           % "(sec)" % "(sec)"
03978                      « endl « format(fmt3) % whoi % dive % idx % midx++ // midx
03979                           % (int) sam() % sax_psi() % san_psi() //
03980                           % sao() % sap()
03981                      « endl;
03982               }
03983
03984               {
03985                   static const format fmt1("#   %5s %4s %4s %4s "
03986                           "%11s %11s "
03987                           "%11s %11s "
03988                           "%11s");
03989                   static const format fmt2("#    %5s %4s %4s %4s "
03990                           "%11s %11s "
03991                           "%11s %11s "
03992                           "%11s");
03993                   static const format fmt3("mis %05d %4d %4d %4d "
03994                           "%11d %11d "
03995                           "%11d %11d "
03996                           "%11d");
03997                   os « "# --- Surface - Wave-Field Measurement -------------------------------------"
03998                      "------------------------"
03999                      « endl
04000                      « format(fmt1) % "SN" % "dive" % "idx" % "midx" //
04001                           % "MAM" % "MAT" //
04002                           % "ACA" % "ACG" //
04003                           % "ACR"
04004                      « endl
04005                      « format(fmt2) % "" % "" % "" % "" //
04006                           % "" % "(sec)" //
04007                           % "(g/FSR)" % "(°/sec/FSR)" //
04008                           % "(Hz)"
04009                      « endl
04010                      « format(fmt3) % whoi % dive % idx % midx++ // midx
04011                           % (int) mam() % mat() //
04012                           % aca() % acg() //
04013                           % acr()
04014                      « endl;
04015               }
04016
04017               {
04018                   static const format fmt1("#   %5s %4s %4s %4s "
04019                           "%11s %11s");
04020                   static const format fmt2("#    %5s %4s %4s %4s "
04021                           "%11s %11s");
04022                   static const format fmt3("mis %05d %4d %4d %4d "
04023                           "%11d %11d");
04024                   os « "# --- Surface - Abort/Beacon Mode ----------------------------------------"
04025                      "------------------------"
04026                      « endl
04027                      « format(fmt1) % "SN" % "dive" % "idx" % "midx" //
04028                           % "SBM" % "SBT"
04029                      « endl
04030                      « format(fmt2) % "" % "" % "" % "" //
04031                           % "" % "(sec)"
04032                      « endl
04033                      « format(fmt3) % whoi % dive % idx % midx++ // midx
04034                           % (int) sbm() % sbt()
04035                      « endl;
04036               }
04037
04038           }
04039
04040               break;
04041
04042           case 0x04026200: // 2017 PAR floats
04043           case 0x04036100: // ALAMO Software version 0x04036100 (2017 S2h test float 10108).
04044           case 0x04046100: // ALAMO Software version 0x04046100 (2017 Hurricane Floats).
04045           case 0x04056300: // ALAMO Software version 0x04056300 (2017 S2h test float 10108).
04046           case 0x04066100: // ALAMO Software version 0x04066100 (2017 S2h test float 10108).
04047
04048           {   int stype;
04049               string stype_str;
04050               switch(stype = sensor_type()) {
04051               case 0:     stype_str = "RBR";                          break;
04052               case 1:     stype_str = "SBE";                          break;
```

```
04053                 default:    stype_str = "unknown"; stype = -99;         break; }
04054                 {   // Header line.
04055                     static const format fmt1("#   %5s %4s %4s %16s %11s %11s        %s");
04056                     static const format fmt3("MIS %05d %4d %4d             0x%02X "
      " 0x%08X "      "%11d "          "     # %s"          );
04057                     static const string str1(boost::str(format(fmt1)    % "SN"   % "dive"   % "idx"     %
      "CfgMsgVer"   % "FWVer"       % "sensor-type" % "sensor-type-name"   ));
04058                     static const string str2("#");
04059                     os « str1 « endl « str2 « endl « format(fmt3)  % whoi   % dive     % idx       %
      msg_version() % fw_version()  % sensor_type() % stype_str     « endl;
04060                 }
04061                 os « "# --- General
      ----------------------------------------------------------------------------------"            « endl;
04062                 {   static const format fmt1("#   %5s %4s %4s %4s "
      "%11s "         "%11s "          "%11s"          "%11s"          );
04063                     static const format fmt2("#    %5s %4s %4s %4s "
      "%11s "         "%11s "          "%11s"          "%11s"          );
04064                     static const format fmt3("mis %05d %4d %4d %4d "
      "%11d "         "%11d "          "%11d"          "%11d"          );
04065                     static const string str1(boost::str(format(fmt1)    % "SN"  % "dive"    % "idx"     %
      "midx"        % "MSN"          % "MDS"          % "APT"          % "XAS"         ));
04066                     static const string str2(boost::str(format(fmt2)    % ""    % ""        % ""        %
      ""            % ""            % "(dBar)"      % ""            % ""            ));
04067                     os « str1 « endl « str2 « endl « format(fmt3)  % whoi  % dive      % idx       %
      midx++        % msn()          % mds()          % apt()          %  xas()        « endl;
04068                 }
04069                 os « "# --- Start-of-Mission Trigger
      ------------------------------------------------------------------"            « endl;
04070                 {   static const format fmt1("#   %5s %4s %4s %4s "
      "%11s "         "%11s"          );
04071                     static const format fmt2("#    %5s %4s %4s %4s "
      "%11s "         "%11s"          );
04072                     static const format fmt3("mis %05d %4d %4d %4d "
      "%11d "         "%11.3f"        );
04073                     static const string str1(boost::str(format(fmt1)    % "SN"  % "dive"    % "idx"     %
      "midx"        % "MTD"          % "MTT"         ));
04074                     static const string str2(boost::str(format(fmt2)    % ""    % ""        % ""        %
      ""            % "(dBar)"      % "(sec)"       ));
04075                     os « str1 « endl « str2 « endl « format(fmt3)  % whoi  % dive      % idx       %
      midx++        % mtd()          % mtt_sec()     « endl;
04076                 }
04077                 os « "# --- Diagnostic Dive
      ----------------------------------------------------------------------------"            « endl;
04078                 {   static const format fmt1("#   %5s %4s %4s %4s "
      "%11s "         "%11s "          "%11s"          );
04079                     static const format fmt2("#    %5s %4s %4s %4s "
      "%11s "         "%11s "          "%11s"          );
04080                     static const format fmt3("mis %05d %4d %4d %4d "
      "%11d "         "%11d "          "%11d"          );
04081                     static const string str1(boost::str(format(fmt1)    % "SN"  % "dive"    % "idx"     %
      "midx"        % "DDP"          % "DDS"          % "DDF"          ));
04082                     static const string str2(boost::str(format(fmt2)    % ""    % ""        % ""        %
      ""            % "(dBar)"      % "(sec)"       % "(sec)"       ));
04083                     os « str1 « endl « str2 « endl « format(fmt3)  % whoi  % dive      % idx       %
      midx++        % ddp()          % dds()          % ddf()          « endl;
04084                 }
04085                 os « "# --- Built-In Test (BIT)
      ------------------------------------------------------------------"            « endl;
04086                 {   static const format fmt1("#   %5s %4s %4s %4s "
      "%11s "         "%11s "          "%11s"          );
04087                     static const format fmt2("#    %5s %4s %4s %4s "
      "%11s "         "%11s "          "%11s"          );
04088                     static const format fmt3("mis %05d %4d %4d %4d "
      "%11.2f "       "%11.2f "        "%11.2f"        );
04089                     static const string str1(boost::str(format(fmt1)    % "SN"  % "dive"    % "idx"     %
      "midx"        % "PNB"          % "PNC"          % "PXC"          ));
04090                     static const string str2(boost::str(format(fmt2)    % ""    % ""        % ""        %
      ""            % "(V)"          % "(psi)"       % "(psi)"       ));
04091                     os « str1 « endl « str2 « endl « format(fmt3)  % whoi  % dive      % idx       %
      midx++        % pnb_v()        % pnc_psi()     % pxc_psi()     « endl;
04092                 }
04093                 os « "# --- Calibration Coefficients
      ------------------------------------------------------------------"            « endl;
04094                 {   static const format fmt1("#   %5s %4s %4s %4s "
      "%11s "        "%11s "         "%11s "         "%11s "         "%11s "          "%11s"          );
04095                     static const format fmt2("#    %5s %4s %4s %4s "
      "%11s "        "%11s "         "%11s "         "%11s "         "%11s "          "%11s"          );
04096                     static const format fmt3("mis %05d %4d %4d %4d "
      "%11d "        "%11d "         "%11d "         "%11d "         "%11d "          "%11d"          );
04097                     static const string str1(boost::str(format(fmt1)    % "SN"  % "dive"    % "idx"     %
      "midx"        % "PBL"          % "PBH"          % "PP0"          % "PP1"          % "PVL"          % "PVH"
      ));
04098                     static const string str2(boost::str(format(fmt2)    % ""    % ""        % ""        %
      ""            % "(LSB/12V)"    % "(LSB/14V)"    % "(LSB/0A)"     % "(LSB/1A)"     % "(LSB/12V)"    %
      "(LSB/14V)"   ));
04099                     os « str1 « endl « str2 « endl « format(fmt3)  % whoi  % dive      % idx       %
      midx++        % pbl()          % pbh()          % pp0()          % pp1()          % pv1()          % pvh()
      « endl;
04100                 }
04101                 {   static const format fmt1("#   %5s %4s %4s %4s "
      "%11s "         "%11s "         "%11s "          "%11s"          );
04102                     static const format fmt2("#    %5s %4s %4s %4s "
      "%11s "         "%11s "         "%11s "          "%11s"          );
04103                     static const format fmt3("mis %05d %4d %4d %4d "
      "%11d "         "%11d "         "%11d "          "%11d"          );
04104                     static const string str1(boost::str(format(fmt1)    % "SN"  % "dive"    % "idx"     %
      "midx"        % "PV0"          % "PV5"          % "PC0"          % "PC1"         ));
04105                     static const string str2(boost::str(format(fmt2)    % ""    % ""        % ""        %
      ""            % "(LSB/0inHg)" % "(LSB/5inHg)" % "(LSB/0A)"     % "(LSB/1A)"    ));
04106                     os « str1 « endl « str2 « endl « format(fmt3)  % whoi  % dive      % idx       %
      midx++        % pv0()          % pv5()          % pc0()          % pc1()          « endl;
04107                 }
04108                 os « "# --- Profile
      ----------------------------------------------------------------------------------"            « endl;
04109                 {   static const format fmt1("#   %5s %4s %4s %4s "
      "%11s "         "%11s "         "%11s "         "%11s "         "%11s "          "%11s"          );
04110                     static const format fmt2("#    %5s %4s %4s %4s "
      "%11s "         "%11s "         "%11s "         "%11s "         "%11s "          "%11s"          );
```

```
04111                   static const format fmt3("mis %05d %4d %4d %4d "
      "%11d "         "%11d "         "%11.3f "       "%11d "         "%11d "         "%11d"          );
04112                   static const string str1(boost::str(format(fmt1)    % "SN"  % "dive"    % "idx"     %
      "midx"        % "CPS"         % "CST"         % "CSP"         % "CTF"         % "CTL"         % "CTU"
      ));
04113                   static const string str2(boost::str(format(fmt2)    % ""    % ""        % ""        %
      ""            % ""            % ""            % "(sec)"       % "(dBar)"      % "(dBar)"      % "(dBar)"
      ));
04114                   os « str1 « endl « str2 « endl « format(fmt3)  % whoi  % dive      % idx       %
      midx++        % (int) cps()   % (int) cst()   % csp_sec()     % ctf()         % ctl()         % ctu()
      « endl;
04115                 }
04116                 os « "# --- Profile - Binning Breakpoints
      ----------------------------------------------------------------"            « endl;
04117                 {   static const format fmt1("#   %5s %4s %4s %4s "
      "%11s "         "%11s"          );
04118                   static const format fmt2("#    %5s %4s %4s %4s "
      "%11s "         "%11s "         "%11s"          );
04119                   static const format fmt3("mis %05d %4d %4d %4d "
      "%11d "         "%11d "         "%11d"          );
04120                   static const string str1(boost::str(format(fmt1)    % "SN"  % "dive"    % "idx"     %
      "midx"        % "CPB1"        % "CPB2"        % "CPB3"        ));
04121                   static const string str2(boost::str(format(fmt2)    % ""    % ""        % ""        %
      ""            % "(dBar)"      % "(dBar)"      % "(dBar)"      ));
04122                   os « str1 « endl « str2 « endl « format(fmt3)  % whoi  % dive      % idx       %
      midx++        % cpb1()        % cpb2()        % cpb3()        « endl;
04123                 }
04124                 os « "# --- Profile - Binning Configuration
      ----------------------------------------------------------------"            « endl;
04125                 switch(sensor_type()) {
04126                 case 0: { // RBR
04127                   static const format fmt1("#   %5s %4s %4s %4s "
      "%11s "        "%11s "         "%11s "         "%11s "         "%11s "         "%11s"          );
04128                   static const format fmt2("#    %5s %4s %4s %4s "
      "%11s "        "%11s "         "%11s "         "%11s "         "%11s "         "%11s"          );
04129                   static const format fmt3("mis %05d %4d %4d %4d "
      "%11.1f "      "%11.1f "       "%11.1f "       "%11.3f "       "%11.3f "       "%11.3f"        );
04130                   static const string str1(boost::str(format(fmt1)    % "SN"  % "dive"    % "idx"     %
      "midx"        % "CPN1"        % "CPN2"        % "CPN3"        % "CPP1"        % "CPP2"        % "CPP3"
      ));
04131                   static const string str2(boost::str(format(fmt2)    % ""    % ""        % ""        %
      ""            % "(dBar)"      % "(dBar)"      % "(dBar)"      % "(sec)"       % "(sec)"       % "(sec)"
      ));
04132                   os « str1 « endl « str2 « endl « format(fmt3)  % whoi  % dive      % idx       %
      midx++        % cpn1_dbar()   % cpn2_dbar()   % cpn3_dbar()   % cpp1_sec()    % cpp2_sec()    % cpp3_sec()
      « endl;
04133                 }
04134                   break;
04135                 case 1:  { // SBE
04136                   static const format fmt1("#   %5s %4s %4s %4s "
      "%11s "         "%11s "         "%11s "         "%11s "         "%11s "         "%11s"          );
04137                   static const format fmt2("#    %5s %4s %4s %4s "
      "%11s "         "%11s "         "%11s "         "%11s "         "%11s "         "%11s"          );
04138                   static const format fmt3("mis %05d %4d %4d %4d "
      "%11d "         "%11d "         "%11d "         "%11d "         "%11d "         "%11d"          );
04139                   static const string str1(boost::str(format(fmt1)    % "SN"  % "dive"    % "idx"     %
      "midx"        % "CPN1"        % "CPN2"        % "CPN3"        % "CPP1"        % "CPP2"        % "CPP3"
      ));
04140                   static const string str2(boost::str(format(fmt2)    % ""    % ""        % ""        %
      ""            % "(n/a)"       % "(n/a)"       % "(n/a)"       % "(dBar)"      % "(dBar)"      % "(dBar)"
      ));
04141                   os « str1 « endl « str2 « endl « format(fmt3)  % whoi  % dive      % idx       %
      midx++        % cpn1()        % cpn2()        % cpn3()        % cpp1()        % cpp2()        % cpp3()
      « endl;
04142                 }
04143                   break;
04144                 default:
04145                   static const format fmt1("#   %5s %4s %4s %4s "
      "%11s "         "%11s "         "%11s "         "%11s "         "%11s "         "%11s"          );
04146                   static const format fmt2("#    %5s %4s %4s %4s "
      "%11s "         "%11s "         "%11s "         "%11s "         "%11s "         "%11s"          );
04147                   static const format fmt3("mis %05d %4d %4d %4d "
      "%11d "         "%11d "         "%11d "         "%11d "         "%11d "         "%11d"          );
04148                   static const string str1(boost::str(format(fmt1)    % "SN"  % "dive"    % "idx"     %
      "midx"        % "CPN1"        % "CPN2"        % "CPN3"        % "CPP1"        % "CPP2"        % "CPP3"
      ));
04149                   static const string str2(boost::str(format(fmt2)    % ""    % ""        % ""        %
      ""            % ""            % ""            % ""            % ""            % ""            % ""
      ));
04150                   os « str1 « endl « str2 « endl « format(fmt3)  % whoi  % dive      % idx       %
      midx++        % cpn1()        % cpn2()        % cpn3()        % cpp1()        % cpp2()        % cpp3()
      « endl;
04151                   break;
04152                 }
04153                 os « "# --- Pump Characteristics
      ----------------------------------------------------------------------"            « endl;
04154                 {   static const format fmt1("#   %5s %4s %4s %4s "
      "%11s "         "%11s"          );
04155                   static const format fmt2("#    %5s %4s %4s %4s "
      "%11s "         "%11s"          );
04156                   static const format fmt3("mis %05d %4d %4d %4d "
      "%11d "         "%11d"          );
04157                   static const string str1(boost::str(format(fmt1)    % "SN"  % "dive"    % "idx"     %
      "midx"        % "PFR"         % "PFC"         ));
04158                   static const string str2(boost::str(format(fmt2)    % ""    % ""        % ""        %
      ""            % "(mL/min)"    % "(mL/dBar)"   ));
04159                   os « str1 « endl « str2 « endl « format(fmt3)  % whoi  % dive      % idx       %
      midx++        % pfr()         % pfc()         « endl;
04160                 }
04161                 os « "# --- Descent - General
      ------------------------------------------------------------------------"            « endl;
04162                 {   static const format fmt1("#   %5s %4s %4s %4s "
      "%11s "         "%11s "         "%11s "         "%11s"          );
04163                   static const format fmt2("#    %5s %4s %4s %4s "
      "%11s "         "%11s "         "%11s "         "%11s"          );
04164                   static const format fmt3("mis %05d %4d %4d %4d "
      "%11d "         "%11d "         "%11d "         "%11.3f"        );
04165                   static const string str1(boost::str(format(fmt1)    % "SN"  % "dive"    % "idx"     %
```

```
    "midx"        % "DNS"         % "DTD"         % "DDD"         % "DDT"        % "DSI"          ));
04166                   static const string str2(boost::str(format(fmt2)    % ""    % ""        % ""        %
    ""            % ""            % "(dBar)"      % "(dBar)"      % "(sec)"      % "(sec)"        ));
04167                   os « str1 « endl « str2 « endl « format(fmt3)  % whoi  % dive      % idx       %
    midx++        % dns()          % dtd()         % ddd()         % ddt()        % dsi_sec()      « endl;
04168               }
04169               os « "# --- Descent - Stage 1 (Descent to First-Pump Pressure)
    ------------------------------------------"           « endl;
04170               {   static const format fmt1("#   %5s %4s %4s %4s "
    "%11s "         "%11s"          );
04171                   static const format fmt2("#    %5s %4s %4s %4s "
    "%11s "         "%11s"          );
04172                   static const format fmt3("mis %05d %4d %4d %4d "
    "%11d "         "%11d"          );
04173                   static const string str1(boost::str(format(fmt1)    % "SN"  % "dive"    % "idx"     %
    "midx"        % "DSD"         % "DST"         ));
04174                   static const string str2(boost::str(format(fmt2)    % ""    % ""        % ""        %
    ""            % "(dBar)"      % "(sec)"      ));
04175                   os « str1 « endl « str2 « endl « format(fmt3)  % whoi  % dive      % idx       %
    midx++        % dsd()         % dst()         « endl;
04176               }
04177               os « "# --- Descent - Stage 2 (First Descent Pump)
    ------------------------------------------------"           « endl;
04178               {   static const format fmt1("#   %5s %4s %4s %4s "
    "%11s "         );
04179                   static const format fmt2("#    %5s %4s %4s %4s "
    "%11s "         );
04180                   static const format fmt3("mis %05d %4d %4d %4d "
    "%11d "         );
04181                   static const string str1(boost::str(format(fmt1)    % "SN"  % "dive"    % "idx"     %
    "midx"        % "DFT"         ));
04182                   static const string str2(boost::str(format(fmt2)    % ""    % ""        % ""        %
    ""            % "(sec)"       ));
04183                   os « str1 « endl « str2 « endl « format(fmt3)  % whoi  % dive      % idx       %
    midx++        % dft()         « endl;
04184               }
04185               os « "# --- Descent - Stage 3 (Seek to Drift/Park Pressure)
    ----------------------------------------------"           « endl;
04186               {   static const format fmt1("#   %5s %4s %4s %4s "
    "%11s "         "%11s "         "%11s "         "%11s "         "%11s"          );
04187                   static const format fmt2("#    %5s %4s %4s %4s "
    "%11s "         "%11s "         "%11s "         "%11s "         "%11s"          );
04188                   static const format fmt3("mis %05d %4d %4d %4d "
    "%11d "         "%11d "         "%11d "         "%11d "         "%11d"          );
04189                   static const string str1(boost::str(format(fmt1)    % "SN"  % "dive"    % "idx"     %
    "midx"        % "DCT"         % "DCM"         % "DAL"         % "DCA"        % "DSW"         % "DUC"
    ));
04190                   static const string str2(boost::str(format(fmt2)    % ""    % ""        % ""        %
    ""            % "(sec)"       % ""            % ""            % ""           % "(sec)"       % ""
    ));
04191                   os « str1 « endl « str2 « endl « format(fmt3)  % whoi  % dive      % idx       %
    midx++        % dct()         % (int) dcm()   % (int) dal()   % (int) dca()   % dsw()         % duc()
    « endl;
04192               }
04193               {   static const format fmt1("#   %5s %4s %4s %4s "
    "%11s "         "%11s "         "%11s "         "%11s "         "%11s"          );
04194                   static const format fmt2("#    %5s %4s %4s %4s "
    "%11s "         "%11s "         "%11s "         "%11s "         "%11s"          );
04195                   static const format fmt3("mis %05d %4d %4d %4d "
    "%11d "         "%11d "         "%11d "         "%11d "         "%11d"          );
04196                   static const string str1(boost::str(format(fmt1)    % "SN"  % "dive"    % "idx"     %
    "midx"        % "DPT"         % "DUT"         % "DVT"         % "DCX"        % "DCP"         % "DCF"
    ));
04197                   static const string str2(boost::str(format(fmt2)    % ""    % ""        % ""        %
    ""            % "(sec)"       % "(sec)"       % "(sec)"       % "(sec)"      % "(sec)"       % "(sec)"
    ));
04198                   os « str1 « endl « str2 « endl « format(fmt3)  % whoi  % dive      % idx       %
    midx++        % dpt()         % dut()         % dvt()         % dcx()         % dcp()         % dcf()
    « endl;
04199               }
04200               os « "# --- Park/Drift - General
    ------------------------------------------------------------------"           « endl;
04201               {   static const format fmt1("#   %5s %4s %4s %4s "
    "%11s "         "%11s "         "%11s "         "%11s "         "%11s"          );
04202                   static const format fmt2("#    %5s %4s %4s %4s "
    "%11s "         "%11s "         "%11s "         "%11s "         "%11s"          );
04203                   static const format fmt3("mis %05d %4d %4d %4d "
    "%11d "         "%11d "         " 0x%08X "      "%11d "         "%11.3f "       "%11d"          );
04204                   static const string str1(boost::str(format(fmt1)    % "SN"  % "dive"    % "idx"     %
    "midx"        % "PNS"         % "PTO"         % "PTH"         % "PSA"        % "PSI"         % "PFM"
    ));
04205                   static const string str2(boost::str(format(fmt2)    % ""    % ""        % ""        %
    ""            % ""            % "(sec)"       % ""            % ""           % "(sec)"       % ""
    ));
04206                   os « str1 « endl « str2 « endl « format(fmt3)  % whoi  % dive      % idx       %
    midx++        % pns()         % pto()         % pth()         % (int) psa()   % psi_sec()     % (int)
    pfm()   « endl;
04207               }
04208               {   static const format fmt1("#   %5s %4s %4s %4s "
    "%11s "         "%11s"          );
04209                   static const format fmt2("#    %5s %4s %4s %4s "
    "%11s "         "%11s"          );
04210                   static const format fmt3("mis %05d %4d %4d %4d "
    "%11d "         "%11d"          );
04211                   static const string str1(boost::str(format(fmt1)    % "SN"  % "dive"    % "idx"     %
    "midx"        % "PFD"         % "PDT"         ));
04212                   static const string str2(boost::str(format(fmt2)    % ""    % ""        % ""        %
    ""            % "(dBar)"      % "(sec)"       ));
04213                   os « str1 « endl « str2 « endl « format(fmt3)  % whoi  % dive      % idx       %
    midx++        % pfd()         % pdt()         « endl;
04214               }
04215               os « "# --- Park/Drift - Seek
    ------------------------------------------------------------------------"           « endl;
04216               {
04217                   static const format fmt1("#   %5s %4s %4s %4s "
    "%11s "         "%11s "         "%11s "         "%11s "         "%11s"          );
04218                   static const format fmt2("#    %5s %4s %4s %4s "
    "%11s "         "%11s "         "%11s "         "%11s "         "%11s"          );
```

```
04219                     static const format fmt3("mis %05d %4d %4d %4d "
      "%11s "          "%11s "          "%11s "          "%11s "          "%11s"          );
04220                     static const string str1(boost::str(format(fmt1)    % "SN"  % "dive"    % "idx"     %
      "midx"         % "PSM"          % "PST"          % "PSD"          % "PSV"          % "PSW"          ));
04221                     static const string str2(boost::str(format(fmt2)    % ""    % ""        % ""        %
      ""             % ""             % "(sec)"       % "(dBar)"       % "(sec)"        % "(sec)"        ));
04222                     os « str1 « endl « str2 « endl « format(fmt3)  % whoi  % dive      % idx       %
      midx++         % (int) psm()   % pst()         % psd()          % psv()          % psw()          « endl;
04223                   }
04224                   os « "# --- Ascent/Profile
      ------------------------------------------------------------------------------"            « endl;
04225                   {
04226                     static const format fmt1("#   %5s %4s %4s %4s "
      "%11s "          "%11s "          "%11s "          "%11s "          "%11s "          "%11s"          );
04227                     static const format fmt2("#    %5s %4s %4s %4s "
      "%11s "          "%11s "          "%11s "          "%11s "          "%11s"          );
04228                     static const format fmt3("mis %05d %4d %4d %4d "
      "%11d "          "%11.2f "        "%11d "          "%11d "          "%11d "          "%11.3f"        );
04229                     static const string str1(boost::str(format(fmt1)    % "SN"  % "dive"    % "idx"     %
      "midx"         % "ASN"          % "ARR"          % "APM"          % "APN"          % "APF"          % "ASI"
      ));
04230                     static const string str2(boost::str(format(fmt2)    % ""    % ""        % ""        %
      ""             % ""             % "(m/sec)"     % "(sec)"        % "(sec)"        % "(sec)"        % "(sec)"
      ));
04231                     os « str1 « endl « str2 « endl « format(fmt3)  % whoi  % dive      % idx       %
      midx++         % asn()         % arr_m_per_s() % apm()          % apn()          % apf()          % asi_sec()
      « endl;
04232                   }
04233                   {
04234                     static const format fmt1("#   %5s %4s %4s %4s "
      "%11s "          "%11s "          "%11s "          "%11s "          "%11s "          "%11s"          );
04235                     static const format fmt2("#    %5s %4s %4s %4s "
      "%11s "          "%11s "          "%11s "          "%11s "          "%11s"          );
04236                     static const format fmt3("mis %05d %4d %4d %4d "
      "%11d "          "%11d "          "%11d "          "%11d "          "%11.3f "        "%11d"          );
04237                     static const string str1(boost::str(format(fmt1)    % "SN"  % "dive"    % "idx"     %
      "midx"         % "ASD"          % "ADD"          % "ADT"          % "ARC"          % "AHR"          % "AHD"
      ));
04238                     static const string str2(boost::str(format(fmt2)    % ""    % ""        % ""        %
      ""             % "(dBar)"       % "(dBar)"       % "(sec)"        % "(?)"          % "(m/sec)"      % "(dBar)"
      ));
04239                     os « str1 « endl « str2 « endl « format(fmt3)  % whoi  % dive      % idx       %
      midx++         % asd()         % add()          % adt()          % arc()          % ahr_m_per_s() % ahd()
      « endl;
04240                   }
04241                   os « "# --- Ascent - Ice-Detection
      ---------------------------------------------------------------------"            « endl;
04242                   {   static const format fmt1("#   %5s %4s %4s %4s "
      "%11s "          "%11s "          "%11s "          "%11s "          "%11s"          );
04243                     static const format fmt2("#    %5s %4s %4s %4s "
      "%11s "          "%11s "          "%11s "          "%12s "          "%11s"          );
04244                     static const format fmt3("mis %05d %4d %4d %4d "
      "%11d "          "     0x%04X "   "%11d "          "%11d "          "%11.2f "        "%11d"          );
04245                     static const string str1(boost::str(format(fmt1)    % "SN"  % "dive"    % "idx"     %
      "midx"         % "ICM"          % "IMM"          % "ICN"          % "ICX"          % "ICT"          % "ICS"
      ));
04246                     static const string str2(boost::str(format(fmt2)    % ""    % ""        % ""        %
      ""             % ""             % ""             % "(dBar)"       % "(dBar)"       % "(degC)"       % "(sec)"
      ));
04247                     os « str1 « endl « str2 « endl « format(fmt3)  % whoi  % dive      % idx       %
      midx++         % (int) icm()   % imm()          % icn()          % icx()          % ict_degc()     % ics()
      « endl;
04248                   }
04249                   {   static const format fmt1("#   %5s %4s %4s %4s "
      "%11s "          "%11s "          "%11s "          "%11s "          "%11s"          );
04250                     static const format fmt2("#    %5s %4s %4s %4s "
      "%11s "          "%11s "          "%11s "          "%11s "          "%11s"          );
04251                     static const format fmt3("mis %05d %4d %4d %4d "
      "%11d "          "%11.2f "        "%11d "          " 0x%08X "       " 0x%08X"        );
04252                     static const string str1(boost::str(format(fmt1)    % "SN"  % "dive"    % "idx"     %
      "midx"         % "ISP"          % "ISM"          % "IST"          % "ISO"          % "ICF"          % "ICW"
      ));
04253                     static const string str2(boost::str(format(fmt2)    % ""    % ""        % ""        %
      ""             % ""             % ""             % "(PSU)"        % "(sec)"        % ""             % ""
      ));
04254                     os « str1 « endl « str2 « endl « format(fmt3)  % whoi  % dive      % idx       %
      midx++         % (int) isp()   % (int) ism()   % ist_psu()      % iso()          % icf()          % icw()
      « endl;
04255                   }
04256                   os « "# --- Ascent - Ice-Detection (Summer)
      ------------------------------------------------------------"            « endl;
04257                   {   static const format fmt1("#   %5s %4s %4s %4s "
      "%11s "          "%11s "          "%11s "          "%11s "          "%11s "          "%11s"          );
04258                     static const format fmt2("#    %5s %4s %4s %4s "
      "%11s "          "%11s "          "%11s "          "%11s "          "%11s"          );
04259                     static const format fmt3("mis %05d %4d %4d %4d "
      "%11d "          "%11d "          "%11d "          "%11d "          "%11d "          "%11d"          );
04260                     static const string str1(boost::str(format(fmt1)    % "SN"  % "dive"    % "idx"     %
      "midx"         % "ISS0"         % "ICA0"         % "ICR0"         % "ICD0"         % "ICO0"         % "ICG0"
      ));
04261                     static const string str2(boost::str(format(fmt2)    % ""    % ""        % ""        %
      ""             % ""             % ""             % "(min)"        % "(dBar)"       % "(sec)"        % ""
      ));
04262                     os « str1 « endl « str2 « endl « format(fmt3)  % whoi  % dive      % idx       %
      midx++         % (int) iss0()  % ica0()         % icr0()         % icd0()         % ico0()         % (int)
      icg0()  « endl;
04263                   }
04264                   {   static const format fmt1("#   %5s %4s %4s %4s "
      "%11s "          "%11s "          "%11s "          "%11s"           );
04265                     static const format fmt2("#    %5s %4s %4s %4s "
      "%11s "          "%11s "          "%11s "          "%11s"           );
04266                     static const format fmt3("mis %05d %4d %4d %4d "
      "%11d "          "%11d "          "%11d "          "%11d"           );
04267                     static const string str1(boost::str(format(fmt1)    % "SN"  % "dive"    % "idx"     %
      "midx"         % "IUE0"         % "ICB"          % "IMT0"         % "ILT0"         ));
04268                     static const string str2(boost::str(format(fmt2)    % ""    % ""        % ""        %
      ""             % ""             % ""             % "(%)"          % "(sec)"        ));
04269                     os « str1 « endl « str2 « endl « format(fmt3)  % whoi  % dive      % idx       %
```

```
      midx++        % (int) iue0()  % icb0()        % imt0()        % ilt0()        « endl;
04270                 }
04271                 os « "# --- Ascent - Ice-Detection (Winter)
      ------------------------------------------------------------"            « endl;
04272                 {   static const format fmt1("#   %5s %4s %4s %4s "
      "%11s "         "%11s "         "%11s "         "%11s "         "%11s"          );
04273                     static const format fmt2("#    %5s %4s %4s %4s "
      "%11s "         "%11s "         "%11s "         "%11s "         "%11s"          );
04274                     static const format fmt3("mis %05d %4d %4d %4d "
      "%11d "         "%11d "         "%11d "         "%11d "         "%11d"          );
04275                     static const string str1(boost::str(format(fmt1)    % "SN"  % "dive"    % "idx"     %
      "midx"        % "ISS1"        % "ICA1"        % "ICR1"        % "ICD1"        % "ICO1"        % "ICG1"
      ));
04276                     static const string str2(boost::str(format(fmt2)    % ""    % ""        % ""        %
      ""            % ""            % ""            % "(min)"       % "(dBar)"      % "(sec)"       % ""
      ));
04277                     os « str1 « endl « str2 « endl « format(fmt3)  % whoi  % dive      % idx       %
      midx++        % (int) iss1()  % ica1()        % icr1()        % icd1()        % ico1()        % (int)
      icg1()  « endl;
04278                 }
04279                 {   static const format fmt1("#   %5s %4s %4s %4s "
      "%11s "         "%11s "         "%11s"          );
04280                     static const format fmt2("#    %5s %4s %4s %4s "
      "%11s "         "%11s "         "%11s"          );
04281                     static const format fmt3("mis %05d %4d %4d %4d "
      "%11d "         "%11d "         "%11d"          );
04282                     static const string str1(boost::str(format(fmt1)    % "SN"  % "dive"    % "idx"     %
      "midx"        % "IUE1"        % "ICB1"        % "IMT1"        % "ILT1"        ));
04283                     static const string str2(boost::str(format(fmt2)    % ""    % ""        % ""        %
      ""            % ""            % ""            % "(%)"         % "(sec)"       ));
04284                     os « str1 « endl « str2 « endl « format(fmt3)  % whoi  % dive      % idx       %
      midx++        % (int) iue1()  % icb1()        % imt1()        % ilt1()        « endl;
04285                 }
04286                 os « "# --- Surface
      ------------------------------------------------------------------------------------"            « endl;
04287                 {   static const format fmt1("#   %5s %4s %4s %4s "
      "%11s "         "%11s "         "%11s "         "%11s "         "%11s"          );
04288                     static const format fmt2("#    %5s %4s %4s %4s "
      "%11s "         "%11s "         "%11s "         "%11s "         "%11s"          );
04289                     static const format fmt3("mis %05d %4d %4d %4d "
      "%11d "         "%11d "         "%11d "         "%11d "         "%11.3f"        );
04290                     static const string str1(boost::str(format(fmt1)    % "SN"  % "dive"    % "idx"     %
      "midx"        % "SFN"         % "SGT"         % "SET"         % "SDT"         % "SSA"         % "SSI"
      ));
04291                     static const string str2(boost::str(format(fmt2)    % ""    % ""        % ""        %
      ""            % ""            % "(min)"       % "(min)"       % "(?)"         % ""            % "(sec)"
      ));
04292                     os « str1 « endl « str2 « endl « format(fmt3)  % whoi  % dive      % idx       %
      midx++        % sfn()         % sgt()         % set()         % sdt()         % ssa()         % ssi_sec()
      « endl;
04293                 }
04294                 {   static const format fmt1("#   %5s %4s %4s %4s "
      "%11s "         "%11s"          );
04295                     static const format fmt2("#    %5s %4s %4s %4s "
      "%11s "         "%11s"          );
04296                     static const format fmt3("mis %05d %4d %4d %4d "
      "%11d "         "%11d"          );
04297                     static const string str1(boost::str(format(fmt1)    % "SN"  % "dive"    % "idx"     %
      "midx"        % "SDS"         % "SPT"         ));
04298                     static const string str2(boost::str(format(fmt2)    % ""    % ""        % ""        %
      ""            % "(sec)"       % "(sec)"       ));
04299                     os « str1 « endl « str2 « endl « format(fmt3)  % whoi  % dive      % idx       %
      midx++        % sds()         % spt()         « endl;
04300                 }
04301                 os « "# --- Surface - Air Pump
      ------------------------------------------------------------------------"            « endl;
04302                 {   static const format fmt1("#   %5s %4s %4s %4s "
      "%11s "         "%11s "         "%11s "         "%11s"          );
04303                     static const format fmt2("#    %5s %4s %4s %4s "
      "%11s "         "%11s "         "%11s "         "%11s"          );
04304                     static const format fmt3("mis %05d %4d %4d %4d "
      "%11d "         "%11.3f "       "%11d "         "%11d"          );
04305                     static const string str1(boost::str(format(fmt1)    % "SN"  % "dive"    % "idx"     %
      "midx"        % "SAM"         % "SAX"         % "SAN"         % "SAO"         % "SAP"         ));
04306                     static const string str2(boost::str(format(fmt2)    % ""    % ""        % ""        %
      ""            % ""            % "(psi)"       % "(psi)"       % "(sec)"       % "(sec)"       ));
04307                     os « str1 « endl « str2 « endl « format(fmt3)  % whoi  % dive      % idx       %
      midx++        % (int) sam()   % sax_psi()     % san_psi()     % sao()         % sap()         « endl;
04308                 }
04309                 os « "# --- Surface - Wave-Field Measurement
      ------------------------------------------------------------"            « endl;
04310                 {   static const format fmt1("#   %5s %4s %4s %4s "
      "%11s "         "%11s "         "%11s "         "%11s"          );
04311                     static const format fmt2("#    %5s %4s %4s %4s "
      "%11s "         "%11s "         "%11s "         "%11s"          );
04312                     static const format fmt3("mis %05d %4d %4d %4d "
      "%11d "         "%11d "         "%11d "         "%11d"          );
04313                     static const string str1(boost::str(format(fmt1)    % "SN"  % "dive"    % "idx"     %
      "midx"        % "MAM"         % "MAT"         % "ACA"         % "ACG"         % "ACR"         ));
04314                     static const string str2(boost::str(format(fmt2)    % ""    % ""        % ""        %
      ""            % ""            % "(sec)"       % "(g/FSR)"     % "(°/sec/FSR)" % "(Hz)"        ));
04315                     os « str1 « endl « str2 « endl « format(fmt3)  % whoi  % dive      % idx       %
      midx++        % (int) mam()   % mat()         % aca()         % acg()         % acr()         « endl;
04316                 }
04317                 os « "# --- Surface - Abort/Beacon Mode
      ------------------------------------------------------------"            « endl;
04318                 {   static const format fmt1("#   %5s %4s %4s %4s "
      "%11s "         "%11s"          );
04319                     static const format fmt2("#    %5s %4s %4s %4s "
      "%11s "         "%11s"          );
04320                     static const format fmt3("mis %05d %4d %4d %4d "
      "%11d "         "%11d"          );
04321                     static const string str1(boost::str(format(fmt1)    % "SN"  % "dive"    % "idx"     %
      "midx"        % "SBM"         % "SBT"         ));
04322                     static const string str2(boost::str(format(fmt2)    % ""    % ""        % ""        %
      ""            % ""            % "(sec)"       ));
04323                     os « str1 « endl « str2 « endl « format(fmt3)  % whoi  % dive      % idx       %
      midx++        % (int) sbm()   % sbt()         « endl;
```

```
04324                 }
04325
04326             }
04327
04328                 break;
04329
04330             case 0x05006100: // ALAMO Software version 0x05006100 (2017 Alto test float 10109).
04331
04332               break;
04333
04334
04335             default: {          // Unsupported ALAMO software version.
04336                 ostringstream os;
04337                 os « "Unsupported ALAMO Mission Parameters message version: 0x"
04338                    « hex
04339                    « setw(8)
04340                    « setfill('0')
04341                    « fw_version()
04342                    « "!";
04343                 EXC_THROW(os.str());
04344             }
04345
04346                 break;
04347
04348             }
04349
04350         }CATCH_BLOCK("Failed to generate listing string for ALAMO mission parameters object!")
04351
04352         return os.str();
04353     }
04354
04355 public:
04356
04357     /* Public Accessor Methods: */
04358
04359     uint16_t aca() const {
04360         return aca_;
04361     }
04362
04363     uint16_t acg() const {
04364         return acg_;
04365     }
04366
04367     uint16_t acr() const {
04368         return acr_;
04369     }
04370
04371     uint16_t add() const {
04372         return add_;
04373     }
04374
04375     uint16_t adt() const {
04376         return adt_;
04377     }
04378
04379     uint16_t ahd() const {
04380         return ahd_;
04381     }
04382
04383     uint16_t ahr() const {
04384         return ahr_;
04385     }
04386
04387     uint16_t apf() const {
04388         return apf_;
04389     }
04390
04391     uint16_t apm() const {
04392         return apm_;
04393     }
04394
04395     uint16_t apn() const {
04396         return apn_;
04397     }
04398
04399     unsigned int apt() const {
04400         return apt_;
04401     }
04402
04403     uint16_t arc() const {
04404         return arc_;
04405     }
04406
04407     uint16_t arr() const {
04408         return arr_;
04409     }
04410
04411     const ptime& ascent_start_time() const {
04412         return ascent_start_time_;
04413     }
04414
04415     uint16_t asd() const {
04416         return asd_;
04417     }
04418
04419     uint32_t asi() const {
04420         return asi_;
04421     }
04422
04423     uint16_t asn() const {
04424         return asn_;
04425     }
04426
04427     uint16_t cpp1() const {
04428         return cpp_[0];
04429     }
04430
04431     uint16_t cpp2() const {
```

```
04432         return cpp_[1];
04433     }
04434
04435     uint16_t cpp3() const {
04436         return cpp_[2];
04437     }
04438
04439     uint8_t cps() const {
04440         return cps_;
04441     }
04442
04443     uint32_t csp() const {
04444         return csp_;
04445     }
04446
04447     uint8_t cst() const {
04448         return cst_;
04449     }
04450
04451     uint16_t ctf() const {
04452         return ctf_;
04453     }
04454
04455     uint16_t ctl() const {
04456         return ctl_;
04457     }
04458
04459     uint16_t ctu() const {
04460         return ctu_;
04461     }
04462
04463     uint8_t dal() const {
04464         return dal_;
04465     }
04466
04467     uint8_t dca() const {
04468         return dca_;
04469     }
04470
04471     uint16_t dcd() const {
04472         return dcd_;
04473     }
04474
04475     uint16_t dcf() const {
04476         return dcf_;
04477     }
04478
04479     uint8_t dcm() const {
04480         return dcm_;
04481     }
04482
04483     uint16_t dcp() const {
04484         return dcp_;
04485     }
04486
04487     uint16_t dcr() const {
04488         return dcr_;
04489     }
04490
04491     uint16_t dct() const {
04492         return dct_;
04493     }
04494
04495     uint16_t dcx() const {
04496         return dcx_;
04497     }
04498
04499     uint16_t ddc() const {
04500         return ddc_;
04501     }
04502
04503     uint16_t ddd() const {
04504         return ddd_;
04505     }
04506
04507     uint16_t ddf() const {
04508         return ddf_;
04509     }
04510
04511     uint16_t ddp() const {
04512         return ddp_;
04513     }
04514
04515     uint16_t dds() const {
04516         return dds_;
04517     }
04518
04519     uint16_t ddt() const {
04520         return ddt_;
04521     }
04522
04523     const ptime& descend_start_time() const {
04524         return descend_start_time_;
04525     }
04526
04527     uint16_t dft() const {
04528         return dft_;
04529     }
04530
04531     uint16_t dns() const {
04532         return dns_;
04533     }
04534
04535     uint16_t dpd() const {
04536         return dpd_;
04537     }
04538
04539     uint16_t dpt() const {
```

```
04540         return dpt_;
04541     }
04542
04543     uint8_t dra() const {
04544         return dra_;
04545     }
04546
04547     uint16_t drc() const {
04548         return drc_;
04549     }
04550
04551     uint16_t drt() const {
04552         return drt_;
04553     }
04554
04555     uint16_t dsd() const {
04556         return dsd_;
04557     }
04558
04559     uint32_t dsi() const {
04560         return dsi_;
04561     }
04562
04563     uint16_t dst() const {
04564         return dst_;
04565     }
04566
04567     uint16_t dsw() const {
04568         return dsw_;
04569     }
04570
04571     uint16_t dtd() const {
04572         return dtd_;
04573     }
04574
04575     uint16_t duc() const {
04576         return duc_;
04577     }
04578
04579     uint16_t dut() const {
04580         return dut_;
04581     }
04582
04583     uint16_t dvd() const {
04584         return dvd_;
04585     }
04586
04587     uint16_t dvw() const {
04588         return dvw_;
04589     }
04590
04591     uint16_t dvt() const {
04592         return dvt_;
04593     }
04594
04595     uint16_t fos() const {
04596         return fos_;
04597     }
04598
04599     uint8_t fta() const {
04600         return fta_;
04601     }
04602
04603     uint8_t ftm() const {
04604         return ftm_;
04605     }
04606
04607     uint16_t fto() const {
04608         return fto_;
04609     }
04610
04611     uint8_t mam() const {
04612         return mam_;
04613     }
04614
04615     uint16_t mat() const {
04616         return mat_;
04617     }
04618
04619     uint16_t mds() const {
04620         return mds_;
04621     }
04622
04623     unsigned int msg_version() const {
04624         return msg_version_;
04625     }
04626
04627     int16_t msn() const {
04628         return msn_;
04629     }
04630
04631     uint16_t mtd() const {
04632         return mtd_;
04633     }
04634
04635     uint16_t mtt() const {
04636         return mtt_;
04637     }
04638
04639     const ptime& park_start_time() const {
04640         return park_start_time_;
04641     }
04642
04643     uint16_t pbh() const {
04644         return pbh_;
04645     }
04646
04647     uint16_t pbl() const {
```

```
04648         return pbl_;
04649     }
04650
04651     uint16_t pc0() const {
04652         return pc0_;
04653     }
04654
04655     uint16_t pc1() const {
04656         return pc1_;
04657     }
04658
04659     uint16_t pdt() const {
04660         return pdt_;
04661     }
04662
04663     uint16_t pfc() const {
04664         return pfc_;
04665     }
04666
04667     uint16_t pfd() const {
04668         return pfd_;
04669     }
04670
04671     uint8_t pfm() const {
04672         return pfm_;
04673     }
04674
04675     uint16_t pfr() const {
04676         return pfr_;
04677     }
04678
04679     uint16_t pft() const {
04680         return pft_;
04681     }
04682
04683     uint16_t pfv() const {
04684         return pfv_;
04685     }
04686
04687     uint16_t pnb() const {
04688         return pnb_;
04689     }
04690
04691     uint16_t pnc() const {
04692         return pnc_;
04693     }
04694
04695     uint16_t pns() const {
04696         return pns_;
04697     }
04698
04699     uint16_t pp0() const {
04700         return pp0_;
04701     }
04702
04703     uint16_t pp1() const {
04704         return pp1_;
04705     }
04706
04707     uint8_t psa() const {
04708         return psa_;
04709     }
04710
04711     uint16_t psd() const {
04712         return psd_;
04713     }
04714
04715     uint32_t psi() const {
04716         return psi_;
04717     }
04718
04719     uint8_t psm() const {
04720         return psm_;
04721     }
04722
04723     uint16_t pst() const {
04724         return pst_;
04725     }
04726
04727     uint16_t psv() const {
04728         return psv_;
04729     }
04730
04731     uint16_t psw() const {
04732         return psw_;
04733     }
04734
04735     uint32_t pth() const {
04736         return pth_;
04737     }
04738
04739     uint32_t pto() const {
04740         return pto_;
04741     }
04742
04743     uint16_t pv0() const {
04744         return pv0_;
04745     }
04746
04747     uint16_t pv5() const {
04748         return pv5_;
04749     }
04750
04751     uint16_t pvh() const {
04752         return pvh_;
04753     }
04754
04755     uint16_t pvl() const {
```

```
04756         return pvl_;
04757     }
04758
04759     uint16_t pxc() const {
04760         return pxc_;
04761     }
04762
04763     uint8_t sam() const {
04764         return sam_;
04765     }
04766
04767     uint16_t san() const {
04768         return san_;
04769     }
04770
04771     uint16_t sao() const {
04772         return sao_;
04773     }
04774
04775     uint16_t sap() const {
04776         return sap_;
04777     }
04778
04779     uint16_t sax() const {
04780         return sax_;
04781     }
04782
04783     uint8_t sbm() const {
04784         return sbm_;
04785     }
04786
04787     uint16_t sbt() const {
04788         return sbt_;
04789     }
04790
04791     int16_t sdc() const {
04792         return sdc_;
04793     }
04794
04795     uint16_t sds() const {
04796         return sds_;
04797     }
04798
04799     uint32_t sdt() const {
04800         return sdt_;
04801     }
04802
04803     int sensor_type() const {
04804         return sensor_type_;
04805     }
04806
04807     uint16_t set() const {
04808         return set_;
04809     }
04810
04811     uint16_t sfn() const {
04812         return sfn_;
04813     }
04814
04815     uint16_t sgt() const {
04816         return sgt_;
04817     }
04818
04819     uint32_t fw_version() const {
04820         return fw_version_;
04821     }
04822
04823     uint16_t spt() const {
04824         return spt_;
04825     }
04826
04827     uint16_t ssa() const {
04828         return ssa_;
04829     }
04830
04831     uint32_t ssi() const {
04832         return ssi_;
04833     }
04834
04835     uint16_t ssp() const {
04836         return ssp_;
04837     }
04838
04839     const ptime& surface_start_time() const {
04840         return surface_start_time_;
04841     }
04842
04843     int xas() const {
04844         return xas_;
04845     }
04846
04847     template<typename T = float> T pnb_v() const {
04848         return (T) pnb() / 100.;
04849     }
04850
04851     template<typename T = float> T pnc_psi() const {
04852         return (T) pnc() / 100.;
04853     }
04854
04855     template<typename T = float> T pxc_psi() const {
04856         return (T) pxc() / 100.;
04857     }
04858
04859     template<typename T = float> T mtt_sec() const {
04860         return (T) mtt() / 1000.;
04861     }
04862
04863     template<typename T = float> T csp_sec() const {
```

```
04864         return (T) csp() / 1000.;
04865     }
04866
04867     uint16_t cpb1() const {
04868         return cpb_[0];
04869     }
04870
04871     uint16_t cpb2() const {
04872         return cpb_[1];
04873     }
04874
04875     uint16_t cpb3() const {
04876         return cpb_[2];
04877     }
04878
04879     uint16_t cpn1() const {
04880         return cpn_[0];
04881     }
04882
04883     uint16_t cpn2() const {
04884         return cpn_[1];
04885     }
04886
04887     uint16_t cpn3() const {
04888         return cpn_[2];
04889     }
04890
04891     uint32_t icf() const {
04892         return icf_;
04893     }
04894
04895     uint8_t icg0() const {
04896         return icg_[0];
04897     }
04898
04899     uint8_t icg1() const {
04900         return icg_[1];
04901     }
04902
04903     uint8_t icm() const {
04904         return icm_;
04905     }
04906
04907     uint16_t icn() const {
04908         return icn_;
04909     }
04910
04911     uint16_t ics() const {
04912         return ics_;
04913     }
04914
04915     int16_t ict() const {
04916         return ict_;
04917     }
04918
04919     uint32_t icw() const {
04920         return icw_;
04921     }
04922
04923     uint16_t icx() const {
04924         return icx_;
04925     }
04926
04927     uint16_t imm() const {
04928         return imm_;
04929     }
04930
04931     uint16_t iso() const {
04932         return iso_;
04933     }
04934
04935     uint8_t isp() const {
04936         return isp_;
04937     }
04938
04939     uint16_t ist() const {
04940         return ist_;
04941     }
04942
04943     uint16_t iat() const {
04944         return iat_;
04945     }
04946
04947     uint16_t isd() const {
04948         return isd_;
04949     }
04950
04951     uint16_t ica0() const {
04952         return ica_[0];
04953     }
04954
04955     uint16_t icb0() const {
04956         return icb_[0];
04957     }
04958
04959     uint16_t icd0() const {
04960         return icd_[0];
04961     }
04962
04963     uint16_t ilt0() const {
04964         return ilt_[0];
04965     }
04966
04967     uint16_t imt0() const {
04968         return imt_[0];
04969     }
04970
04971     uint8_t iss0() const {
```

```
04972         return iss_[0];
04973     }
04974
04975     uint8_t iue0() const {
04976         return iue_[0];
04977     }
04978
04979     uint16_t ico0() const {
04980         return ico_[0];
04981     }
04982
04983     uint32_t icr0() const {
04984         return icr_[0];
04985     }
04986
04987     uint16_t ica1() const {
04988         return ica_[1];
04989     }
04990
04991     uint16_t icb1() const {
04992         return icb_[1];
04993     }
04994
04995     uint16_t icd1() const {
04996         return icd_[1];
04997     }
04998
04999     uint16_t ilt1() const {
05000         return ilt_[1];
05001     }
05002
05003     uint16_t imt1() const {
05004         return imt_[1];
05005     }
05006
05007     uint8_t isa() const {
05008         return isa_;
05009     }
05010
05011     uint16_t isv() const {
05012         return isv_;
05013     }
05014
05015     uint16_t isw() const {
05016         return isw_;
05017     }
05018
05019     uint8_t iss1() const {
05020         return iss_[1];
05021     }
05022
05023     uint8_t iue1() const {
05024         return iue_[1];
05025     }
05026
05027     uint16_t ico1() const {
05028         return ico_[1];
05029     }
05030
05031     uint32_t icr1() const {
05032         return icr_[1];
05033     }
05034
05035     uint8_t ism() const {
05036         return ism_;
05037     }
05038
05039     uint8_t btm() const {
05040         return btm_;
05041     }
05042
05043     template<typename T = float> T cpn1_dbar() const {
05044         return (T) cpn1() / 10.;
05045     }
05046
05047     template<typename T = float> T cpn2_dbar() const {
05048         return (T) cpn2() / 10.;
05049     }
05050
05051     template<typename T = float> T cpn3_dbar() const {
05052         return (T) cpn3() / 10.;
05053     }
05054
05055     template<typename T = float> T cpp1_sec() const {
05056         return (T) cpp1() / 1000.;
05057     }
05058
05059     template<typename T = float> T cpp2_sec() const {
05060         return (T) cpp2() / 1000.;
05061     }
05062
05063     template<typename T = float> T cpp3_sec() const {
05064         return (T) cpp3() / 1000.;
05065     }
05066
05067     template<typename T = float> T dra_m_per_s() const {
05068         return (T) dra() / 100.;
05069     }
05070
05071     template<typename T = float> T dsi_sec() const {
05072         return (T) dsi() / 1000.;
05073     }
05074
05075     template<typename T = float> T psi_sec() const {
05076         return (T) psi() / 1000.;
05077     }
05078
05079     template<typename T = float> T arr_m_per_s() const {
```

```
05080         return (T) arr() / 100.;
05081     }
05082
05083     template<typename T = float> T asi_sec() const {
05084         return (T) asi() / 1000.;
05085     }
05086
05087     template<typename T = float> T ahr_m_per_s() const {
05088         return (T) ahr() / 1000.;
05089     }
05090
05091     template<typename T = float> T ssi_sec() const {
05092         return (T) ssi() / 1000.;
05093     }
05094
05095     template<typename T = float> T san_psi() const {
05096         return (T) san() / 1000.;
05097     }
05098
05099     template<typename T = float> T sax_psi() const {
05100         return (T) sax() / 1000.;
05101     }
05102
05103     template<typename T = float> T ict_degc() const {
05104         return (T) ict() / 100.;
05105     }
05106
05107     template<typename T = float> T ist_psu() const {
05108         return (T) ist() / 100.;
05109     }
05110
05111 protected:
05112
05113     /* Protected Mutator Methods: */
05114
05115     void set_aca(uint16_t aca) {
05116         aca_ = aca;
05117     }
05118
05119     void set_acg(uint16_t acg) {
05120         acg_ = acg;
05121     }
05122
05123     void set_acr(uint16_t acr) {
05124         acr_ = acr;
05125     }
05126
05127     void set_add(uint16_t add) {
05128         add_ = add;
05129     }
05130
05131     void set_adt(uint16_t adt) {
05132         adt_ = adt;
05133     }
05134
05135     void set_ahd(uint16_t ahd) {
05136         ahd_ = ahd;
05137     }
05138
05139     void set_ahr(uint16_t ahr) {
05140         ahr_ = ahr;
05141     }
05142
05143     void set_apf(uint16_t apf) {
05144         apf_ = apf;
05145     }
05146
05147     void set_apm(uint16_t apm) {
05148         apm_ = apm;
05149     }
05150
05151     void set_apn(uint16_t apn) {
05152         apn_ = apn;
05153     }
05154
05155     void set_apt(uint8_t apt) {
05156         apt_ = apt;
05157     }
05158
05159     void set_arc(uint16_t arc) {
05160         arc_ = arc;
05161     }
05162
05163     void set_arr(uint16_t arr) {
05164         arr_ = arr;
05165     }
05166
05167     void set_ascent_start_time(const ptime& ascent_start_time) {
05168         ascent_start_time_ = ascent_start_time;
05169     }
05170
05171     void set_asd(uint16_t asd) {
05172         asd_ = asd;
05173     }
05174
05175     void set_asi(uint32_t asi) {
05176         asi_ = asi;
05177     }
05178
05179     void set_asn(uint16_t asn) {
05180         asn_ = asn;
05181     }
05182
05183     void set_btm(uint8_t btm) {
05184         btm_ = btm;
05185     }
05186
05187     void set_cpb1(uint16_t cpb) {
```

```
05188         cpb_[0] = cpb;
05189     }
05190
05191     void set_cpb2(uint16_t cpb) {
05192         cpb_[1] = cpb;
05193     }
05194
05195     void set_cpb3(uint16_t cpb) {
05196         cpb_[2] = cpb;
05197     }
05198
05199
05200     void set_cpp1(uint16_t cpp) {
05201         cpp_[0] = cpp;
05202     }
05203
05204     void set_cpp2(uint16_t cpp) {
05205         cpp_[1] = cpp;
05206     }
05207
05208     void set_cpp3(uint16_t cpp) {
05209         cpp_[2] = cpp;
05210     }
05211
05212
05213     void set_cpn1(uint16_t cpn) {
05214         cpn_[0] = cpn;
05215     }
05216
05217     void set_cpn2(uint16_t cpn) {
05218         cpn_[1] = cpn;
05219     }
05220
05221     void set_cpn3(uint16_t cpn) {
05222         cpn_[2] = cpn;
05223     }
05224
05225
05226     void set_cps(uint8_t cps) {
05227         cps_ = cps;
05228     }
05229
05230     void set_csp(uint32_t csp) {
05231         csp_ = csp;
05232     }
05233
05234     void set_cst(uint8_t cst) {
05235         cst_ = cst;
05236     }
05237
05238     void set_ctf(uint16_t ctf) {
05239         ctf_ = ctf;
05240     }
05241
05242     void set_ctl(uint16_t ctl) {
05243         ctl_ = ctl;
05244     }
05245
05246     void set_ctu(uint16_t ctu) {
05247         ctu_ = ctu;
05248     }
05249
05250     void set_dal(uint8_t dal) {
05251         dal_ = dal;
05252     }
05253
05254     void set_dca(uint8_t dca) {
05255         dca_ = dca;
05256     }
05257
05258     void set_dcd(uint16_t dcd) {
05259         dcd_ = dcd;
05260     }
05261
05262     void set_dcf(uint16_t dcf) {
05263         dcf_ = dcf;
05264     }
05265
05266     void set_dcm(uint8_t dcm) {
05267         dcm_ = dcm;
05268     }
05269
05270     void set_dcp(uint16_t dcp) {
05271         dcp_ = dcp;
05272     }
05273
05274     void set_dcr(uint16_t dcr) {
05275         dcr_ = dcr;
05276     }
05277
05278     void set_dct(uint16_t dct) {
05279         dct_ = dct;
05280     }
05281
05282     void set_dcx(uint16_t dcx) {
05283         dcx_ = dcx;
05284     }
05285
05286     void set_ddc(uint16_t ddc) {
05287         ddc_ = ddc;
05288     }
05289
05290     void set_ddd(uint16_t ddd) {
05291         ddd_ = ddd;
05292     }
05293
05294     void set_ddf(uint16_t ddf) {
05295         ddf_ = ddf;
```

```
05296     }
05297
05298     void set_ddp(uint16_t ddp) {
05299         ddp_ = ddp;
05300     }
05301
05302     void set_dds(uint16_t dds) {
05303         dds_ = dds;
05304     }
05305
05306     void set_ddt(uint16_t ddt) {
05307         ddt_ = ddt;
05308     }
05309
05310     void set_descend_start_time(const ptime& descend_start_time) {
05311         descend_start_time_ = descend_start_time;
05312     }
05313
05314     void set_dft(uint16_t dft) {
05315         dft_ = dft;
05316     }
05317
05318     void set_dns(uint16_t dns) {
05319         dns_ = dns;
05320     }
05321
05322     void set_dpd(uint16_t dpd) {
05323         dpd_ = dpd;
05324     }
05325
05326     void set_dpt(uint16_t dpt) {
05327         dpt_ = dpt;
05328     }
05329
05330     void set_dra(uint8_t dra) {
05331         dra_ = dra;
05332     }
05333
05334     void set_drc(uint16_t drc) {
05335         drc_ = drc;
05336     }
05337
05338     void set_drt(uint16_t drt) {
05339         drt_ = drt;
05340     }
05341
05342     void set_dsd(uint16_t dsd) {
05343         dsd_ = dsd;
05344     }
05345
05346     void set_dsi(uint32_t dsi) {
05347         dsi_ = dsi;
05348     }
05349
05350     void set_dst(uint16_t dst) {
05351         dst_ = dst;
05352     }
05353
05354     void set_dsw(uint16_t dsw) {
05355         dsw_ = dsw;
05356     }
05357
05358     void set_dtd(uint16_t dtd) {
05359         dtd_ = dtd;
05360     }
05361
05362     void set_duc(uint16_t duc) {
05363         duc_ = duc;
05364     }
05365
05366     void set_dut(uint16_t dut) {
05367         dut_ = dut;
05368     }
05369
05370     void set_dvd(uint16_t dvd) {
05371         dvd_ = dvd;
05372     }
05373
05374     void set_dvt(uint16_t dvt) {
05375         dvt_ = dvt;
05376     }
05377
05378     void set_dvw(uint16_t dvw) {
05379             dvw_ = dvw;
05380         }
05381
05382     void set_fos(uint16_t fos) {
05383         fos_ = fos;
05384     }
05385
05386     void set_fta(uint8_t fta) {
05387         fta_ = fta;
05388     }
05389
05390     void set_ftm(uint8_t ftm) {
05391         ftm_ = ftm;
05392     }
05393
05394     void set_fto(uint16_t fto) {
05395         fto_ = fto;
05396     }
05397
05398     void set_iat(uint16_t iat) {
05399         iat_ = iat;
05400     }
05401
05402     void set_isd(uint16_t isd) {
05403         isd_ = isd;
```

```
05404     }
05405
05406     void set_mam(uint8_t mam) {
05407         mam_ = mam;
05408     }
05409
05410     void set_mat(uint16_t mat) {
05411         mat_ = mat;
05412     }
05413
05414     void set_mds(uint16_t mds) {
05415         mds_ = mds;
05416     }
05417
05418     void set_msg_version(uint8_t msg_version) {
05419         msg_version_ = msg_version;
05420     }
05421
05422     void set_msn(int16_t msn) {
05423         msn_ = msn;
05424     }
05425
05426     void set_mtd(uint16_t mtd) {
05427         mtd_ = mtd;
05428     }
05429
05430     void set_mtt(uint16_t mtt) {
05431         mtt_ = mtt;
05432     }
05433
05434     void set_park_start_time(const ptime& park_start_time) {
05435         park_start_time_ = park_start_time;
05436     }
05437
05438     void set_pbh(uint16_t pbh) {
05439         pbh_ = pbh;
05440     }
05441
05442     void set_pbl(uint16_t pbl) {
05443         pbl_ = pbl;
05444     }
05445
05446     void set_pc0(uint16_t pc0) {
05447         pc0_ = pc0;
05448     }
05449
05450     void set_pc1(uint16_t pc1) {
05451         pc1_ = pc1;
05452     }
05453
05454     void set_pdt(uint16_t pdt) {
05455         pdt_ = pdt;
05456     }
05457
05458     void set_pfc(uint16_t pfc) {
05459         pfc_ = pfc;
05460     }
05461
05462     void set_pfd(uint16_t pfd) {
05463         pfd_ = pfd;
05464     }
05465
05466     void set_pfm(uint8_t pfm) {
05467         pfm_ = pfm;
05468     }
05469
05470     void set_pfr(uint16_t pfr) {
05471         pfr_ = pfr;
05472     }
05473
05474     void set_pft(uint16_t pft) {
05475         pft_ = pft;
05476     }
05477
05478     void set_pfv(uint16_t pfv) {
05479         pfv_ = pfv;
05480     }
05481
05482     void set_pnb(uint16_t pnb) {
05483         pnb_ = pnb;
05484     }
05485
05486     void set_pnc(uint16_t pnc) {
05487         pnc_ = pnc;
05488     }
05489
05490     void set_pns(uint16_t pns) {
05491         pns_ = pns;
05492     }
05493
05494     void set_pp0(uint16_t pp0) {
05495         pp0_ = pp0;
05496     }
05497
05498     void set_pp1(uint16_t pp1) {
05499         pp1_ = pp1;
05500     }
05501
05502     void set_psa(uint8_t psa) {
05503         psa_ = psa;
05504     }
05505
05506     void set_psd(uint16_t psd) {
05507         psd_ = psd;
05508     }
05509
05510     void set_psi(uint32_t psi) {
05511         psi_ = psi;
```

```
05512     }
05513
05514     void set_psm(uint8_t psm) {
05515         psm_ = psm;
05516     }
05517
05518     void set_pst(uint16_t pst) {
05519         pst_ = pst;
05520     }
05521
05522     void set_psv(uint16_t psv) {
05523         psv_ = psv;
05524     }
05525
05526     void set_psw(uint16_t psw) {
05527         psw_ = psw;
05528     }
05529
05530     void set_pth(uint32_t pth) {
05531         pth_ = pth;
05532     }
05533
05534     void set_pto(uint32_t pto) {
05535         pto_ = pto;
05536     }
05537
05538     void set_pv0(uint16_t pv0) {
05539         pv0_ = pv0;
05540     }
05541
05542     void set_pv5(uint16_t pv5) {
05543         pv5_ = pv5;
05544     }
05545
05546     void set_pvh(uint16_t pvh) {
05547         pvh_ = pvh;
05548     }
05549
05550     void set_pvl(uint16_t pvl) {
05551         pvl_ = pvl;
05552     }
05553
05554     void set_pxc(uint16_t pxc) {
05555         pxc_ = pxc;
05556     }
05557
05558     void set_sam(uint8_t sam) {
05559         sam_ = sam;
05560     }
05561
05562     void set_san(uint16_t san) {
05563         san_ = san;
05564     }
05565
05566     void set_sao(uint16_t sao) {
05567         sao_ = sao;
05568     }
05569
05570     void set_sap(uint16_t sap) {
05571         sap_ = sap;
05572     }
05573
05574     void set_sax(uint16_t sax) {
05575         sax_ = sax;
05576     }
05577
05578     void set_sbm(uint8_t sbm) {
05579         sbm_ = sbm;
05580     }
05581
05582     void set_sbt(uint16_t sbt) {
05583         sbt_ = sbt;
05584     }
05585
05586     void set_sdc(int16_t sdc) {
05587         sdc_ = sdc;
05588     }
05589
05590     void set_sds(uint16_t sds) {
05591         sds_ = sds;
05592     }
05593
05594     void set_sdt(uint32_t sdt) {
05595         sdt_ = sdt;
05596     }
05597
05598     void set_sensor_type(uint8_t sensor_type) {
05599         sensor_type_ = sensor_type;
05600     }
05601
05602     void set_set(uint16_t set) {
05603         set_ = set;
05604     }
05605
05606     void set_sfn(uint16_t sfn) {
05607         sfn_ = sfn;
05608     }
05609
05610     void set_sgt(uint16_t sgt) {
05611         sgt_ = sgt;
05612     }
05613
05614     void set_fw_version(uint32_t fw_version) {
05615         fw_version_ = fw_version;
05616     }
05617
05618     void set_spt(uint16_t spt) {
05619         spt_ = spt;
```

```
05620     }
05621
05622     void set_ssa(uint16_t ssa) {
05623         ssa_ = ssa;
05624     }
05625
05626     void set_ssi(uint32_t ssi) {
05627         ssi_ = ssi;
05628     }
05629
05630     void set_ssp(uint16_t ssp) {
05631         ssp_ = ssp;
05632     }
05633
05634     void set_surface_start_time(const ptime& surface_start_time) {
05635         surface_start_time_ = surface_start_time;
05636     }
05637
05638     void set_xas(uint8_t xas) {
05639         xas_ = xas;
05640     }
05641
05642     void set_icf(uint32_t icf) {
05643         icf_ = icf;
05644     }
05645
05646     void set_icm(uint8_t icm) {
05647         icm_ = icm;
05648     }
05649
05650     void set_icn(uint16_t icn) {
05651         icn_ = icn;
05652     }
05653
05654     void set_ics(uint16_t ics) {
05655         ics_ = ics;
05656     }
05657
05658     void set_ict(int16_t ict) {
05659         ict_ = ict;
05660     }
05661
05662     void set_icw(uint32_t icw) {
05663         icw_ = icw;
05664     }
05665
05666     void set_icx(uint16_t icx) {
05667         icx_ = icx;
05668     }
05669
05670     void set_imm(uint16_t imm) {
05671         imm_ = imm;
05672     }
05673
05674     void set_iso(uint16_t iso) {
05675         iso_ = iso;
05676     }
05677
05678     void set_isp(uint8_t isp) {
05679         isp_ = isp;
05680     }
05681
05682     void set_ist(uint16_t ist) {
05683         ist_ = ist;
05684     }
05685
05686     void set_ica0(uint16_t ica) {
05687         ica_[0] = ica;
05688     }
05689
05690     void set_icb0(uint16_t icb) {
05691         icb_[0] = icb;
05692     }
05693
05694     void set_icd0(uint16_t icd) {
05695         icd_[0] = icd;
05696     }
05697
05698     void set_icg0(uint8_t icg) {
05699         icg_[0] = icg;
05700     }
05701
05702     void set_ico0(uint16_t ico) {
05703         ico_[0] = ico;
05704     }
05705
05706     void set_icr0(uint32_t icr) {
05707         icr_[0] = icr;
05708     }
05709
05710     void set_ilt0(uint16_t ilt) {
05711         ilt_[0] = ilt;
05712     }
05713
05714     void set_imt0(uint16_t imt) {
05715         imt_[0] = imt;
05716     }
05717
05718     void set_isa(uint8_t isa) {
05719         isa_ = isa;
05720     }
05721
05722     void set_isv(uint16_t isv) {
05723         isv_ = isv;
05724     }
05725
05726     void set_isw(uint16_t isw) {
05727         isw_ = isw;
```

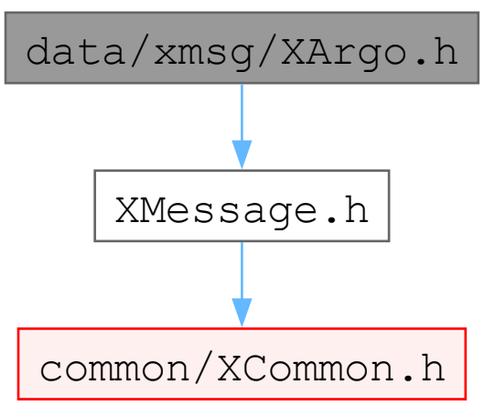
```
05728     }
05729
05730     void set_iss0(uint8_t iss) {
05731         iss_[0] = iss;
05732     }
05733
05734     void set_iue0(uint8_t iue) {
05735         iue_[0] = iue;
05736     }
05737
05738     void set_ica1(uint16_t ica) {
05739         ica_[1] = ica;
05740     }
05741
05742     void set_icb1(uint16_t icb) {
05743         icb_[1] = icb;
05744     }
05745
05746     void set_icd1(uint16_t icd) {
05747         icd_[1] = icd;
05748     }
05749
05750     void set_icg1(uint8_t icg) {
05751         icg_[1] = icg;
05752     }
05753
05754     void set_ico1(uint16_t ico) {
05755         ico_[1] = ico;
05756     }
05757
05758     void set_icr1(uint32_t icr) {
05759         icr_[1] = icr;
05760     }
05761
05762     void set_ilt1(uint16_t ilt) {
05763         ilt_[1] = ilt;
05764     }
05765
05766     void set_imt1(uint16_t imt) {
05767         imt_[1] = imt;
05768     }
05769
05770     void set_iss1(uint8_t iss) {
05771         iss_[1] = iss;
05772     }
05773
05774     void set_iue1(uint8_t iue) {
05775         iue_[1] = iue;
05776     }
05777
05778     void set_ism(uint8_t ism) {
05779         ism_ = ism;
05780     }
05781 };
05782
05783 } /* namespace xmsg */
05784 } /* namespace util */
05785 #endif /* XALAMOMISSIONCONFIGURATION_H_ */
```

11.74 data/xmsg/XArgo.h File Reference

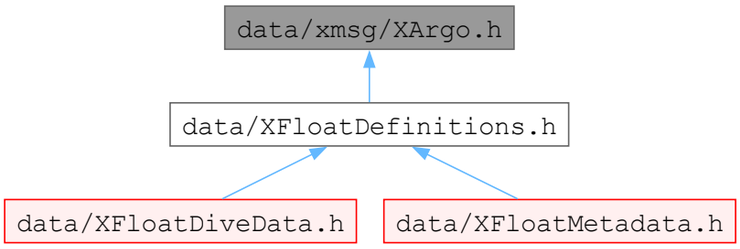
The util::xmsg::XArgo class header file.

```
#include "XMessage.h"
```

Include dependency graph for XArgo.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XArgo

  Class representing Argo X message.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

Enumerations

- enum XArgoInstrumentType_t {
  XARGO_INSRUMENT_TYPE_SOLO2 = 0 ,
  XARGO_INSRUMENT_TYPE_DEEP_ARGO = 1 ,
  XARGO_INSRUMENT_TYPE_ALAMO = 3 ,
  XARGO_INSRUMENT_TYPE_UNKNOWN = 99 }

  Enumeration of the Argo X-Message instrument type identifiers.

11.74.1 Detailed Description

The util::xmsg::XArgo class header file.

This is the header file for the Argo Data X-Message class, util::xmsg::XArgo. This X-Message object contains details about the dive cycle characteristics of the instrument as well as the sensor gain and offset values.

This file contains the class definition and all of its functionality.

Version

0.6

Date

Jun 12, 2014 2:29:34 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.74.2 Enumeration Type Documentation

11.74.2.1 XArgoInstrumentType_t

```
enum XArgoInstrumentType_t
```

Enumeration of the Argo X-Message instrument type identifiers.

Enumerator

XARGO_INSRUMENT_TYPE_SOLO2	Argo X-Message instrument type identifier 0: The Solo-II Argo float identifier.
XARGO_INSRUMENT_TYPE_DEEP_ARGO	Argo X-Message instrument type identifier 1: The Deep Argo float identifier.
XARGO_INSRUMENT_TYPE_ALAMO	Argo X-Message instrument type identifier 3: The Air-Launchable Autonomous Micro-Observer (ALAMO) hurricane float type identifier.
XARGO_INSRUMENT_TYPE_UNKNOWN	Argo X-Message instrument type identifier 99: The unknown, unset, unspecified, or invalid instrument type identifier.

11.75 XArgo.h

Go to the documentation of this file.

```
00001 /** The @ref util::xmsg::XArgo class header file.
00002  *
00003  *  This is the header file for the Argo Data X-Message class, @ref util::xmsg::XArgo. This
00004  *  X-Message object contains details about the dive cycle characteristics of the instrument as
00005  *  well as the sensor gain and offset values.
00006  *
00007  *  This file contains the class definition and all of its functionality.
00008  *
00009  *  @file
00010  *
00011  *  @version    0.6
00012  *
00013  *  @date       Jun 12, 2014 2:29:34 PM
00014  *
00015  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00016  *              Engineer I,
00017  *              Department of Physical Oceanography,
00018  *              Woods Hole Oceanographic Institution
00019  *
00020  *  @copyright  2014 Woods Hole Oceanographic Institution.
00021  *              Released under the GNU General Public License.
00022  *
00023  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00024  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00025  *  General Public License for more details. You should have received a copy of the GNU General
00026  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00027  */
00028
00029 #ifndef XARGO_H_
00030 #define XARGO_H_
00031
00032 /* Includes: */
00033
00034 // local
00035 #include "XMessage.h"
00036
00037 /* Type Definitions: */
00038
00039 /** Enumeration of the Argo X-Message instrument type identifiers.
00040  *
00041  */
00042 typedef enum XArgoInstrumentType {
00043
00044     /** Argo X-Message instrument type identifier 0: The Solo-II Argo float identifier.
00045      *
00046      */
00047     XARGO_INSRUMENT_TYPE_SOLO2 = 0,
00048
00049     /** Argo X-Message instrument type identifier 1: The Deep Argo float identifier.
00050      *
00051      */
00052     XARGO_INSRUMENT_TYPE_DEEP_ARGO = 1,
00053
00054     /** Argo X-Message instrument type identifier 3: The Air-Launchable Autonomous Micro-Observer
00055      *  (ALAMO) hurricane float type identifier.
00056      */
00057     XARGO_INSRUMENT_TYPE_ALAMO = 3,
00058
00059     /** Argo X-Message instrument type identifier 99: The unknown, unset, unspecified, or invalid
00060      *  instrument type identifier.
00061      */
00062     XARGO_INSRUMENT_TYPE_UNKNOWN = 99
00063
00064 } XArgoInstrumentType_t;
00065
00066 namespace util {
00067 namespace xmsg {
00068
00069 /* Class Definition: */
00070
00071 /** Class representing Argo X message.
00072  *
00073  *  This message contains information about the characteristics of the dive cycle.
00074  *
00075  *  @todo   Add support for the Solo-II-type Argo data message.
00076  *  @todo   Add support for the Deep-Argo-type Argo data message.
00077  */
00078 class XArgo:
00079         public util::date::DateUtils,
00080         public XMessage {
```

```
00081
00082 private:
00083
00084     /* Dynamic Data Members: */
00085
00086     /** The Argo instrument type identifier.
00087      *
00088      */
00089     XArgoInstrumentType_t argo_instrument_type_;
00090
00091     /** The X message format major version number.
00092      *
00093      */
00094     uint8_t x_version_major_;
00095
00096     /** The X Message format minor version number.
00097      *
00098      */
00099     uint8_t x_version_minor_;
00100
00101     /* Dynamic Data Members - State Start Times: */
00102
00103     /** The ascend state start time.
00104      *
00105      *  @note           This field is used for ALAMO-type floats only.
00106      *  @note           This field was added in X-Message format version 2.1.
00107      */
00108     ptime ascent_state_start_time_;
00109
00110     /** The descend state start time.
00111      *
00112      *  @note           This field is used for ALAMO-type floats only.
00113      *  @note           This field was added in X-Message format version 2.1.
00114      */
00115     ptime descent_state_start_time_;
00116
00117     /** The park state start time.
00118      *
00119      *  @note           This field is used for ALAMO-type floats only.
00120      *  @note           This field was added in X-Message format version 2.1.
00121      */
00122     ptime drift_state_start_time_;
00123
00124     /** The surface state start time.
00125      *
00126      *  @note           This field is used for ALAMO-type floats only.
00127      *  @note           This field was added in X-Message format version 2.1.
00128      */
00129     ptime surface_state_start_time_;
00130
00131     /* Dynamic Data Members - Dive Cycle Characteristics: */
00132
00133     /** The maximum time to fall from the surface to drift/park depth, in minutes.
00134      *
00135      */
00136     uint16_t max_fall_to_drift_duration_;
00137
00138     /** The maximum time to fall from the drift/park depth to profile depth, in seconds.
00139      *
00140      */
00141     uint16_t max_fall_to_profle_duration_;
00142
00143     /** The maximum rise/ascend/profile time, in minutes.
00144      *
00145      */
00146     uint16_t max_ascent_duration_;
00147
00148     /** The maximum duration at the surface, in seconds.
00149      *
00150      */
00151     uint16_t max_surface_duration_;
00152
00153     /** The mission phase number of this dive.
00154      *
00155      */
00156     uint8_t mission_phase_;
00157
00158     /** The number of seeks to drift depth.
00159      *
00160      */
00161     uint16_t num_seeks_to_drift_pressure_;
00162
00163     /** The interval duration between subsequent seeks to drift/park depth, in minutes.
00164      *
00165      */
00166     uint16_t seek_to_drift_pressure_interval_duration_;
00167
00168     /** The target ascent rate, in cm/s.
00169      *
00170      */
00171     uint8_t target_ascent_rate_;
00172
00173     /** The target duration of the drift/park phase of the dive cycle, in minutes.
00174      *
00175      */
00176     uint16_t target_drift_duration_;
00177
00178     /** The target parking or drift depth (DTD), in dBar.
00179      *
00180      */
00181     uint16_t target_drift_pressure_;
00182
00183     /** The target profile depth (PFD), in dBar.
00184      *
00185      */
00186     uint16_t target_profile_pressure_;
00187
00188     /* Dynamic Data Members - Sensor Gains & Offsets */
```

```
00189
00190     /** The pressure sensitivity gain factor, in LSB/dBar.
00191      *
00192      *  @note           This field was added in X-Message format version 1.2.
00193      */
00194     uint16_t gain_pressure_;
00195
00196     /** The pressure sensitivity gain factor for surface data, in LSB/dBar.
00197      *
00198      *  @note           This field was added in X-Message format version 2.2.
00199      */
00200     uint16_t gain_pressure_surface_;
00201
00202     /** The salinity sensitivity scale factor, in LSB/PSU.
00203      *
00204      *  @note           This field was added in X-Message format version 1.2.
00205      */
00206     uint16_t gain_salinity_;
00207
00208     /** The salinity sensitivity scale factor for surface data, in LSB/PSU.
00209      *
00210      *  @note           This field was added in X-Message format version 2.2.
00211      */
00212     uint16_t gain_salinity_surface_;
00213
00214     /** The temperature sensitivity scale factor, in LSB/degC.
00215      *
00216      *  @note           This field was added in X-Message format version 1.2.
00217      */
00218     uint16_t gain_temperature_;
00219
00220     /** The temperature sensitivity scale factor for surface data, in LSB/degC.
00221      *
00222      *  @note           This field was added in X-Message format version 2.2.
00223      */
00224     uint16_t gain_temperature_surface_;
00225
00226     /** The pressure offset, in dBar.
00227      *
00228      *  @note           This field was added in X-Message format version 1.2.
00229      */
00230     uint16_t offset_pressure_;
00231
00232     /** The pressure offset for surface data, in dBar.
00233      *
00234      *  @note           This field was added in X-Message format version 2.2.
00235      */
00236     uint16_t offset_pressure_surface_;
00237
00238     /** The salinity offset, in PSU.
00239      *
00240      *  @note           This field was added in X-Message format version 1.2.
00241      */
00242     uint16_t offset_salinity_;
00243
00244     /** The salinity offset for surface data, in PSU.
00245      *
00246      *  @note           This field was added in X-Message format version 2.2.
00247      */
00248     uint16_t offset_salinity_surface_;
00249
00250     /** The temperature offset, in degC.
00251      *
00252      *  @note           This field was added in X-Message format version 1.2.
00253      */
00254     uint16_t offset_temperature_;
00255
00256     /** The temperature offset for surface data, in degC.
00257      *
00258      *  @note           This field was added in X-Message format version 2.2.
00259      */
00260     uint16_t offset_temperature_surface_;
00261
00262 protected:
00263
00264     /* Protected Member Functions: */
00265
00266     /** Set the Argo instrument type identifier.
00267      *
00268      *  @param[in]  type    The Argo instrument type identifier.
00269      */
00270     void set_argo_instrument_type(XArgoInstrumentType_t type) {
00271         argo_instrument_type_ = type;
00272     }
00273
00274     /** Set the start time of the ascent state of the dive cycle.
00275      *
00276      *  @param[in]  time    The start time.
00277      */
00278     void set_ascent_state_start_time(const ptime& time) {
00279         ascent_state_start_time_ = time;
00280     }
00281
00282     /** Set the start time of the descent state of the dive cycle.
00283      *
00284      *  @param[in]  time    The start time.
00285      */
00286     void set_descent_state_start_time(const ptime& time) {
00287         descent_state_start_time_ = time;
00288     }
00289
00290     /** Set the start time of the drift state of the dive cycle.
00291      *
00292      *  @param[in]  time    The start time.
00293      */
00294     void set_drift_state_start_time(const ptime& time) {
00295         drift_state_start_time_ = time;
00296     }
```

```
00297
00298     /** Set the pressure sensor gain, in LSB/dBar.
00299      *
00300      *  @param[in]  gain    The sensor gain value.
00301      */
00302     void set_gain_pressure(uint16_t gain) {
00303         gain_pressure_ = gain;
00304     }
00305
00306     /** Set the pressure sensor gain for surface data, in LSB/dBar.
00307      *
00308      *  @param[in]  gain    The sensor gain value.
00309      */
00310     void set_gain_pressure_surface(uint16_t gain) {
00311         gain_pressure_surface_ = gain;
00312     }
00313
00314     /** Set the salinity sensor gain, in LSB/PSU.
00315      *
00316      *  @param[in]  gain    The sensor gain value.
00317      */
00318     void set_gain_salinity(uint16_t gain) {
00319         gain_salinity_ = gain;
00320     }
00321
00322     /** Set the salinity sensor gain for surface data, in PSU/PSU.
00323      *
00324      *  @param[in]  gain    The sensor gain value.
00325      */
00326     void set_gain_salinity_surface(uint16_t gain) {
00327         gain_salinity_surface_ = gain;
00328     }
00329
00330     /** Set the temperature sensor gain, in PSU/degC.
00331      *
00332      *  @param[in]  gain    The sensor gain value.
00333      */
00334     void set_gain_temperature(uint16_t gain) {
00335         gain_temperature_ = gain;
00336     }
00337
00338     /** Set the temperature sensor gain for surface data, in PSU/degC.
00339      *
00340      *  @param[in]  gain    The sensor gain value.
00341      */
00342     void set_gain_temperature_surface(uint16_t gain) {
00343         this->gain_temperature_surface_ = gain;
00344     }
00345
00346     /** Set the maximum duration for the ascent state of the dive cycle, in minutes.
00347      *
00348      *  @param[in]  duration    The maximum duration.
00349      */
00350     void set_max_ascent_duration(uint16_t duration) {
00351         max_ascent_duration_ = duration;
00352     }
00353
00354     /** Set the maximum duration for the descent-to-drift/park-pressure state of the dive cycle, in
00355      *  minutes.
00356      *
00357      *  @param[in]  duration    The maximum duration.
00358      */
00359     void set_max_fall_to_drift_duration(uint16_t duration) {
00360         max_fall_to_drift_duration_ = duration;
00361     }
00362
00363     /** Set the maximum duration for the descent-to-profile-pressure state of the dive cycle, in
00364      *  minutes.
00365      *
00366      *  @param[in]  duration    The maximum duration.
00367      */
00368     void set_max_fall_to_profle_duration(uint16_t duration) {
00369         max_fall_to_profle_duration_ = duration;
00370     }
00371
00372     /** Set the maximum duration for the surface state of the dive cycle, in seconds.
00373      *
00374      *  @param[in]  duration    The maximum duration.
00375      */
00376     void set_max_surface_duration(uint16_t duration) {
00377         max_surface_duration_ = duration;
00378     }
00379
00380     /** Set the mission phase number.
00381      *
00382      *  @param[in]  phase   The mission phase number.
00383      */
00384     void set_mission_phase(uint8_t phase) {
00385         mission_phase_ = phase;
00386     }
00387
00388     /** Set the maximum number of seeks to drift/park pressure.
00389      *
00390      *  @param[in]  seeks   The number of seeks.
00391      */
00392     void set_num_seeks_to_drift_pressure(uint16_t seeks) {
00393         num_seeks_to_drift_pressure_ = seeks;
00394     }
00395
00396     /** Set the pressure sensor offset, in dBar.
00397      *
00398      *  @param[in]  offs    The sensor offset.
00399      */
00400     void set_offset_pressure(uint16_t offs) {
00401         offset_pressure_ = offs;
00402     }
00403
00404     /** Set the pressure sensor offset for surface data, in dBar.
```

```
00405      *
00406      *  @param[in]  offs    The sensor offset.
00407      */
00408     void set_offset_pressure_surface(uint16_t offs) {
00409         offset_pressure_surface_ = offs;
00410     }
00411
00412     /** Set the salinity sensor offset, in PSU.
00413      *
00414      *  @param[in]  offs    The sensor offset.
00415      */
00416     void set_offset_salinity(uint16_t offs) {
00417         offset_salinity_ = offs;
00418     }
00419
00420     /** Set the salinity sensor offset for surface data, in PSU.
00421      *
00422      *  @param[in]  offs    The sensor offset.
00423      */
00424     void set_offset_salinity_surface(uint16_t offs) {
00425         offset_salinity_surface_ = offs;
00426     }
00427
00428     /** Set the temperature sensor offset, in degC.
00429      *
00430      *  @param[in]  offs    The sensor offset.
00431      */
00432     void set_offset_temperature(uint16_t offs) {
00433         offset_temperature_ = offs;
00434     }
00435
00436     /** Set the temperature sensor offset for surface data, in degC.
00437      *
00438      *  @param[in]  offs    The sensor offset.
00439      */
00440     void set_offset_temperature_surface(uint16_t offs) {
00441         offset_temperature_surface_ = offs;
00442     }
00443
00444     /** Set the seek-to-drift/park-pressure interval duration, in minutes.
00445      *
00446      *  @param[in]  duration    The interval duration.
00447      */
00448     void set_seek_to_drift_pressure_interval_duration(uint16_t duration) {
00449         seek_to_drift_pressure_interval_duration_ = duration;
00450     }
00451
00452     /** Set the start time of the surface state of the dive cycle.
00453      *
00454      *  @param[in]  time    The start time.
00455      */
00456     void set_surface_state_start_time(const ptime& time) {
00457         surface_state_start_time_ = time;
00458     }
00459
00460     /** Set the target ascent rate, in cm/s.
00461      *
00462      *  @param[in]  rate    The target rate.
00463      */
00464     void set_target_ascent_rate(uint8_t rate) {
00465         target_ascent_rate_ = rate;
00466     }
00467
00468     /** Set the target duration of the drift/park phase of the dive cycle, in minutes.
00469      *
00470      *  @param[in]  duration    The target duration.
00471      */
00472     void set_target_drift_duration(uint16_t duration) {
00473         target_drift_duration_ = duration;
00474     }
00475
00476     /** Set the target drift/park pressure, in dBar.
00477      *
00478      *  @param[in]  pres    The target pressure.
00479      */
00480     void set_target_drift_pressure(uint16_t pres) {
00481         target_drift_pressure_ = pres;
00482     }
00483
00484     /** Set the target profile pressure, in dBar.
00485      *
00486      *  @param[in]  pres    The target pressure.
00487      */
00488     void set_target_profile_pressure(uint16_t pres) {
00489         target_profile_pressure_ = pres;
00490     }
00491
00492     /** Set the X-Message major version identifier.
00493      *
00494      *  @param[in]  vers    The version.
00495      */
00496     void set_x_version_major(uint8_t vers) {
00497         x_version_major_ = vers;
00498     }
00499
00500     /** Set the X-Message minor version identifier.
00501      *
00502      *  @param[in]  vers    The version.
00503      */
00504     void set_x_version_minor(uint8_t vers) {
00505         x_version_minor_ = vers;
00506     }
00507
00508 public:
00509
00510     /* Public Member Functions: */
00511
00512     /** The constructor.
```

```
00513      *
00514      *  @param[in]  dive    The dive or cycle number of this message.
00515      *  @param[in]  whoi    The unique WHOI identifier.
00516      *  @param[in]  data    The binary Argo data X-message data to decode.
00517      *
00518      *  @exception  util::log::MessageException if an decode error occurs.
00519      */
00520     XArgo(int16_t dive, uint16_t whoi, const vector<uint8_t>& data) :
00521                     XMessage(dive, whoi, data),
00522                     argo_instrument_type_(XARGO_INSRUMENT_TYPE_UNKNOWN),
00523                     x_version_major_(XMSG_ENG_BAD_VER),
00524                     x_version_minor_(XMSG_ENG_BAD_VER),
00525                     // state start times
00526                     ascent_state_start_time_(bpt::not_a_date_time),
00527                     descent_state_start_time_(bpt::not_a_date_time),
00528                     drift_state_start_time_(bpt::not_a_date_time),
00529                     surface_state_start_time_(bpt::not_a_date_time),
00530                     // dive cycle characteristics
00531                     max_fall_to_drift_duration_(XMSG_BAD_U16),
00532                     max_fall_to_profle_duration_(XMSG_BAD_U16),
00533                     max_ascent_duration_(XMSG_BAD_U16),
00534                     max_surface_duration_(XMSG_BAD_U16),
00535                     mission_phase_(XMSG_BAD_U08),
00536                     num_seeks_to_drift_pressure_(XMSG_BAD_U08),
00537                     seek_to_drift_pressure_interval_duration_(XMSG_BAD_U16),
00538                     target_ascent_rate_(XMSG_BAD_U08),
00539                     target_drift_duration_(XMSG_BAD_U16),
00540                     target_drift_pressure_(XMSG_BAD_U16),
00541                     // sensor gains and offset
00542                     gain_pressure_(XMSG_GAIN_INVALID),
00543                     gain_pressure_surface_(XMSG_GAIN_INVALID),
00544                     gain_salinity_(XMSG_GAIN_INVALID),
00545                     gain_salinity_surface_(XMSG_GAIN_INVALID),
00546                     gain_temperature_(XMSG_GAIN_INVALID),
00547                     gain_temperature_surface_(XMSG_GAIN_INVALID),
00548                     offset_pressure_(XMSG_OFFS_INVALID),
00549                     offset_pressure_surface_(XMSG_OFFS_INVALID),
00550                     offset_salinity_(XMSG_OFFS_INVALID),
00551                     offset_salinity_surface_(XMSG_OFFS_INVALID),
00552                     offset_temperature_(XMSG_OFFS_INVALID),
00553                     offset_temperature_surface_(XMSG_OFFS_INVALID) {
00554         // Decode the Argo data message.
00555         DecodeXMsgData(data);
00556     }
00557
00558     /** The destructor.
00559      *
00560      */
00561     ~XArgo() {
00562         /* Do nothing. */
00563     }
00564
00565     /** Get the Argo instrument type identifier.
00566      *
00567      *  @return     The Argo instrument type identifier.
00568      */
00569     XArgoInstrumentType_t argo_instrument_type() const {
00570         return argo_instrument_type_;
00571     }
00572
00573     /** Get the start time of the ascent state of the dive cycle.
00574      *
00575      *  @return     The start time of the ascent state of the dive cycle.
00576      */
00577     const ptime& ascent_state_start_time() const {
00578         return ascent_state_start_time_;
00579     }
00580
00581     /** Get the start time of the descent state of the dive cycle.
00582      *
00583      *  @return     The start time of the descent state of the dive cycle.
00584      */
00585     const ptime& descent_state_start_time() const {
00586         return descent_state_start_time_;
00587     }
00588
00589     /** Decode the Argo X message.
00590      *
00591      *  @param[in]  data        The data to decode.
00592      *
00593      *  @exception  util::log::MessageException if an error occurs while decoding the message.
00594      */
00595     void DecodeXMsgData(const vector<uint8_t>& data) {
00596         try {
00597             // Check for valid X message ID, minimum size, & message terminator.
00598             const uint8_t* d = CheckXMsgData(data, (int) 0xF0, 23);
00599
00600             // Get X message version.
00601             set_x_version_major((d[3] & 0xF0) >> 4);
00602             set_x_version_minor((d[3] & 0x0F));
00603
00604             // Get depths.
00605             set_target_profile_pressure(u16(d, 4));
00606             set_target_drift_pressure(u16(d, 6));
00607
00608             /* Version-Specific Processing: */
00609
00610             /** @todo   Implement the functionality to decode the argo data message for Solo-II
00611              *          floats.
00612              *
00613              */
00614
00615             /** @todo   Implement the functionality to decode the argo data message for Deep Argo
00616              *          floats.
00617              *
00618              */
00619
00620             switch(x_version_major()) {
```

```
00621             case 2:
00622
00623                 /* X-Message Version 2.x */
00624
00625                 switch(x_version_minor()) {
00626                 case 1:
00627
00628                     /* X-Message Version 2.1 */
00629
00630                     if(xmsg_size_bytes() < 40) {
00631                         ostringstream os;
00632                         os « "Insufficient message length: expected 40 bytes, but received "
00633                            « (int) xmsg_size_bytes()
00634                            « " bytes!";
00635                         throw util::log::MessageException(os);
00636                     }
00637
00638                     if(d[24] == XARGO_INSRUMENT_TYPE_ALAMO) {
00639
00640                         /* ALAMO-Specific Processing: */
00641
00642                         set_argo_instrument_type(XARGO_INSRUMENT_TYPE_ALAMO);
00643
00644                         // Get times.
00645                         set_descent_state_start_time(s2000(d, 8));
00646                         set_drift_state_start_time(s2000(d, 12));
00647                         set_ascent_state_start_time(s2000(d, 16));
00648                         set_surface_state_start_time(s2000(d, 20));
00649
00650                         // Target ascent rate.
00651                         set_target_ascent_rate(u08(d, 25));
00652
00653                         // Gains & offsets.
00654                         set_gain_pressure(u16(d, 26));
00655                         set_offset_pressure(u16(d, 28));
00656                         set_gain_temperature(u16(d, 30));
00657                         set_offset_temperature(u16(d, 32));
00658                         set_gain_salinity(u16(d, 34));
00659                         set_offset_salinity(u16(d, 36));
00660
00661                         set_gain_pressure_surface(gain_pressure());
00662                         set_offset_pressure_surface(offset_pressure());
00663                         set_gain_temperature_surface(gain_temperature());
00664                         set_offset_temperature_surface(offset_temperature());
00665                         set_gain_salinity_surface(gain_salinity());
00666                         set_offset_salinity_surface(offset_salinity_surface());
00667                     }
00668
00669                     break;
00670
00671                 case 2:
00672
00673                     /* X-Message Version 2.2 */
00674
00675                     if(xmsg_size_bytes() < 66) {
00676                         ostringstream os;
00677                         os « "Insufficient message length: expected 66 bytes, but received "
00678                            « (int) xmsg_size_bytes()
00679                            « " bytes!";
00680                         throw util::log::MessageException(os);
00681                     }
00682
00683                     if(d[32] == XARGO_INSRUMENT_TYPE_ALAMO) {
00684
00685                         /* ALAMO-Specific Processing: */
00686
00687                         set_argo_instrument_type(XARGO_INSRUMENT_TYPE_ALAMO);
00688
00689                         // Get times.
00690                         set_descent_state_start_time(s2000(d, 16));
00691                         set_drift_state_start_time(s2000(d, 20));
00692                         set_ascent_state_start_time(s2000(d, 24));
00693                         set_surface_state_start_time(s2000(d, 28));
00694
00695                         // Get dive cycle characteristics.
00696                         set_max_ascent_duration(u16(d, 8));
00697                         set_max_fall_to_drift_duration(u16(d, 10));
00698                         set_max_fall_to_profle_duration(u16(d, 12));
00699                         set_target_drift_duration(u16(d, 14));
00700                         set_target_ascent_rate(u08(d, 33));
00701                         set_num_seeks_to_drift_pressure(u08(d, 34));
00702                         set_max_surface_duration(u16(d, 36));
00703                         set_seek_to_drift_pressure_interval_duration(u16(d, 38));
00704                         set_mission_phase(u08(d, 64));
00705
00706                         // Set sensor gains and offsets.
00707                         set_gain_pressure(u16(d, 40));
00708                         set_offset_pressure(u16(d, 42));
00709                         set_gain_temperature(u16(d, 44));
00710                         set_offset_temperature(u16(d, 46));
00711                         set_gain_salinity(u16(d, 48));
00712                         set_offset_salinity(u16(d, 50));
00713                         set_gain_pressure_surface(u16(d, 52));
00714                         set_offset_pressure_surface(u16(d, 54));
00715                         set_gain_temperature_surface(u16(d, 56));
00716                         set_offset_temperature_surface(u16(d, 58));
00717                         set_gain_salinity_surface(u16(d, 60));
00718                         set_offset_salinity_surface(u16(d, 62));
00719
00720                     }
00721
00722                     break;
00723
00724                 case 3:
00725
00726                     /* X-Message Version 2.3 */
00727
00728                     if(xmsg_size_bytes() < 66) {
```

```
00729                         ostringstream os;
00730                         os « "Insufficient message length: expected 66 bytes, but received "
00731                            « (int) xmsg_size_bytes()
00732                            « " bytes!";
00733                         throw util::log::MessageException(os);
00734                     }
00735 
00736                     if(d[32] == XARGO_INSRUMENT_TYPE_ALAMO) {
00737 
00738                         /* ALAMO-Specific Processing: */
00739 
00740                         set_argo_instrument_type(XARGO_INSRUMENT_TYPE_ALAMO);
00741 
00742                         // Get times.
00743                         set_descent_state_start_time(s2000(d, 16));
00744                         set_drift_state_start_time(s2000(d, 20));
00745                         set_ascent_state_start_time(s2000(d, 24));
00746                         set_surface_state_start_time(s2000(d, 28));
00747 
00748                         // Get dive cycle characteristics.
00749                         set_max_ascent_duration(u16(d, 8));
00750                         set_max_fall_to_drift_duration(u16(d, 10));
00751                         set_max_fall_to_profle_duration(u16(d, 12));
00752                         set_target_drift_duration(u16(d, 14));
00753                         set_target_ascent_rate(u08(d, 33));
00754                         set_num_seeks_to_drift_pressure(u08(d, 34));
00755                         set_max_surface_duration(u16(d, 36));
00756                         set_seek_to_drift_pressure_interval_duration(u16(d, 38));
00757                         set_mission_phase(u08(d, 64));
00758 
00759                         // Set sensor gains and offsets.
00760                         set_gain_pressure(u16(d, 40));
00761                         set_offset_pressure(u16(d, 42));
00762                         set_gain_temperature(u16(d, 44));
00763                         set_offset_temperature(u16(d, 46));
00764                         set_gain_salinity(u16(d, 48));
00765                         set_offset_salinity(u16(d, 50));
00766                         set_gain_pressure_surface(u16(d, 52));
00767                         set_offset_pressure_surface(u16(d, 54));
00768                         set_gain_temperature_surface(u16(d, 56));
00769                         set_offset_temperature_surface(u16(d, 58));
00770                         set_gain_salinity_surface(u16(d, 60));
00771                         set_offset_salinity_surface(u16(d, 62));
00772 
00773                     }
00774 
00775                     break;
00776 
00777                 case 4:
00778 
00779                     /* X-Message Version 2.4 */
00780 
00781                     if(xmsg_size_bytes() != 66) {
00782                         ostringstream os;
00783                         os « "Insufficient message length: expected 66 bytes, but received "
00784                            « (int) xmsg_size_bytes()
00785                            « " bytes!";
00786                         throw util::log::MessageException(os);
00787                     }
00788 
00789                     if(d[32] == XARGO_INSRUMENT_TYPE_ALAMO) {
00790 
00791                         /* ALAMO-Specific Processing: */
00792 
00793                         set_argo_instrument_type(XARGO_INSRUMENT_TYPE_ALAMO);
00794 
00795                         // Get times.
00796                         set_descent_state_start_time(s2000(d, 16));
00797                         set_drift_state_start_time(s2000(d, 20));
00798                         set_ascent_state_start_time(s2000(d, 24));
00799                         set_surface_state_start_time(s2000(d, 28));
00800 
00801                         // Get dive cycle characteristics.
00802                         set_max_ascent_duration(u16(d, 8));
00803                         set_max_fall_to_drift_duration(u16(d, 10));
00804                         set_max_fall_to_profle_duration(u16(d, 12));
00805                         set_target_drift_duration(u16(d, 14));
00806                         set_target_ascent_rate(u08(d, 33));
00807                         set_num_seeks_to_drift_pressure(u08(d, 34));
00808                         set_max_surface_duration(u16(d, 36));
00809                         set_seek_to_drift_pressure_interval_duration(u16(d, 38));
00810                         set_mission_phase(u08(d, 64));
00811 
00812                         // Set sensor gains and offsets.
00813                         set_gain_pressure(u16(d, 40));
00814                         set_offset_pressure(u16(d, 42));
00815                         set_gain_temperature(u16(d, 44));
00816                         set_offset_temperature(u16(d, 46));
00817                         set_gain_salinity(u16(d, 48));
00818                         set_offset_salinity(u16(d, 50));
00819                         set_gain_pressure_surface(u16(d, 52));
00820                         set_offset_pressure_surface(u16(d, 54));
00821                         set_gain_temperature_surface(u16(d, 56));
00822                         set_offset_temperature_surface(u16(d, 58));
00823                         set_gain_salinity_surface(u16(d, 60));
00824                         set_offset_salinity_surface(u16(d, 62));
00825 
00826                     }
00827 
00828                     break;
00829 
00830                 default:
00831                     throw util::log::MessageException("Encountered unsupported X-Message format!");
00832                     break;
00833                 }
00834 
00835                 break;
00836 
```

```
00837             default:
00838                 throw util::log::MessageException("Encountered unsupported X-Message format!");
00839                 break;
00840             }
00841 
00842         }CATCH_BLOCK("Failed to decode Argo Data X-message!")
00843     }
00844 
00845     /** Get the start time of the drift/park state of the dive cycle.
00846      *
00847      *  @return     The start time of the drift/park state of the dive cycle.
00848      */
00849     const ptime& drift_state_start_time() const {
00850         return drift_state_start_time_;
00851     }
00852 
00853     /** Get the pressure sensor gain value, in LSB/dBar.
00854      *
00855      *  @return     The pressure sensor gain value, in LSB/dBar.
00856      */
00857     uint16_t gain_pressure() const {
00858         return gain_pressure_;
00859     }
00860 
00861     /** Get the pressure sensor gain value for surface data, in LSB/dBar.
00862      *
00863      *  @return     The pressure sensor gain value, in LSB/dBar.
00864      */
00865     uint16_t gain_pressure_surface() const {
00866         return gain_pressure_surface_;
00867     }
00868 
00869     /** Get the salinity sensor gain value, in LSB/PSU.
00870      *
00871      *  @return     The salinity sensor gain value, in LSB/PSU.
00872      */
00873     uint16_t gain_salinity() const {
00874         return gain_salinity_;
00875     }
00876 
00877     /** Get the salinity sensor gain value for surface data, in LSB/PSU.
00878      *
00879      *  @return     The salinity sensor gain value, in LSB/PSU.
00880      */
00881     uint16_t gain_salinity_surface() const {
00882         return gain_salinity_surface_;
00883     }
00884 
00885     /** Get the temperature sensor gain value, in LSB/degC.
00886      *
00887      *  @return     The temperature sensor gain value, in LSB/degC.
00888      */
00889     uint16_t gain_temperature() const {
00890         return gain_temperature_;
00891     }
00892 
00893     /** Get the temperature sensor gain value for surface data, in LSB/degC.
00894      *
00895      *  @return     The temperature sensor gain value, in LSB/degC.
00896      */
00897     uint16_t gain_temperature_surface() const {
00898         return gain_temperature_surface_;
00899     }
00900 
00901     /** Generate a listing entry string representation of this Argo data message.
00902      *
00903      *  @return         A listing entry string representation of this Argo data message.
00904      */
00905     const string GenerateListingString() const {
00906         using namespace std;
00907         using boost::format;
00908         ostringstream os;
00909 
00910         try {
00911             switch(argo_instrument_type()) {
00912             case XARGO_INSRUMENT_TYPE_SOLO2:
00913 
00914                 /** @todo   Implement the functionality for generating a listing string for a
00915                  *          Solo-II-type Argo data message.
00916                  */
00917 
00918                 break;
00919 
00920             case XARGO_INSRUMENT_TYPE_DEEP_ARGO:
00921 
00922                 /** @todo   Implement the functionality for generating a listing string for a
00923                  *          Deep-Argo-type Argo data message.
00924                  */
00925 
00926                 break;
00927 
00928             case XARGO_INSRUMENT_TYPE_ALAMO: { // Generate listing string for ALAMO
00929 
00930                 { /* Argo Data Message Listing - Minor Index 0: */
00931                     const format fmt(
00932 //                                  "#   %5s %4s %4s %3s %4s %10s %9s %9s %9s %14s %14s\n"
00933 //                                  "#   %5s %4s %4s %3s %4s %10s %9s %9s %9s %14s %14s\n"
00934                                     "A   %5d %4d %4.1f %3d %4d %10d %9d %9d %9d %14d %14d");
00935                     os « "#      SN dive xVer typ midx  missPhase  profPres  parkPres   maxAscT
      maxDesc2ParkT  maxDesc2ProfT" « endl
00936                        « "#                                                    (dBar)    (dBar)     (min)
     (min)          (sec)" « endl
00937                        « format(fmt)
00938 //                                 % "SN" % "dive" % "xVer" % "typ" % "midx" % "missPhase"
00939 //                                 % "profPres" % "parkPres" % "maxAscT" % "maxDesc2ParkT"
00940 //                                 % "maxDesc2ProfT" % "" % "" % "" % "" % "" % "" % "(dBar)" % "(dBar)"
00941 //                                 % "(min)" % "(min)" % "(sec)"
00942                                    % id_instrument() % id_dive() % x_version_float()
```

```
00943                             % (int) argo_instrument_type() % 0 % (int) mission_phase()
00944                             % target_profile_pressure() % target_drift_pressure()
00945                             % max_ascent_duration() % max_fall_to_drift_duration()
00946                             % max_fall_to_profle_duration()
00947                         « endl « "#" « endl;
00948                 }
00949                 { /* Argo Data Message Listing - Minor Index 1: */
00950                     const format fmt(
00951 //                                "#   %5s %4s %4s %3s %4s %10s %7s %14s %11s %9s\n"
00952 //                                "#   %5s %4s %4s %3s %4s %10s %7s %14s %11s %9s\n"
00953                                 "A   %5d %4d %4.1f %3d %4d %10d %7d %14d %11d %9d");
00954                     os « "#      SN dive xVer typ midx   tarParkT   nSeek  seekIntervalT  tarAscRate
      maxSurfT" « endl
00955                         « "#                                                          (min)     (cm/sec)
      (sec)" « endl
00956                         « format(fmt)
00957 //                              % "SN" % "dive" % "xVer" % "typ" % "midx" % "tarParkT"
00958 //                              % "nSeek" % "seekIntervalT" % "tarAscRate" % "maxSurfT" % "" % "" % ""
00959 //                              % "" % "" % "" % "" % "(min)" % "(cm/sec)" % "(sec)"
00960                               % id_instrument() % id_dive() % x_version_float()
00961                               % (int) argo_instrument_type() % 1 % target_drift_duration()
00962                               % (int) num_seeks_to_drift_pressure()
00963                               % seek_to_drift_pressure_interval_duration()
00964                               % (int) target_ascent_rate() % max_surface_duration()
00965                         « endl « "#" « endl;
00966                 }
00967                 { /* Argo Data Message Listing - Minor Index 2: */
00968                     const format fmt(
00969 //                                "#   %5s %4s %4s %3s %4s %24s %24s\n"
00970                                 "A   %5d %4d %4.1f %3d %4d %24s %24s");
00971                     os « "#      SN dive xVer typ midx         start-of-descent           start-of-drift"
      « endl
00972                         « format(fmt)
00973 //                              % "SN" % "dive" % "xVer" % "typ" % "midx" % "start-of-descent"
00974 //                              % "start-of-drift"
00975                               % id_instrument() % id_dive() % x_version_float()
00976                               % (int) argo_instrument_type() % 2
00977                               % GenerateIso8601String(descent_state_start_time())
00978                               % GenerateIso8601String(drift_state_start_time())
00979                         « endl « "#" « endl;
00980                 }
00981                 { /* Argo Data Message Listing - Minor Index 3: */
00982                     const format fmt(
00983 //                                "#   %5s %4s %4s %3s %4s %24s %24s\n"
00984                                 "A   %5d %4d %4.1f %3d %4d %24s %24s");
00985                     os « "#      SN dive xVer typ midx          start-of-ascent         start-of-surface"
      « endl
00986                         « format(fmt)
00987 //                              % "SN" % "dive" % "xVer" % "typ" % "midx" % "start-of-ascent"
00988 //                              % "start-of-surface"
00989                               % id_instrument() % id_dive() % x_version_float()
00990                               % (int) argo_instrument_type() % 3
00991                               % GenerateIso8601String(ascent_state_start_time())
00992                               % GenerateIso8601String(surface_state_start_time())
00993                         « endl « "#" « endl;
00994                 }
00995                 { /* Argo Data Message Listing - Minor Index 4: */
00996                     const format fmt(
00997 //                                "#   %5s %4s %4s %3s %4s %11s %11s %11s %11s %11s %11s\n"
00998 //                                "#   %5s %4s %4s %3s %4s %11s %11s %12s %12s %11s %11s\n"
00999                                 "A   %5d %4d %4.1f %3d %4d %11s %11s %11s %11s %11s %10s");
01000                     os « "#      SN dive xVer typ midx       pGain        pOff       tGain        tOff
      sGain       sOff" « endl
01001                         « "#                                  (1/dBar)      (dBar)    (1/degC)      (degC)
      (1/PSU)      (PSU)" « endl
01002                         « format(fmt)
01003 //                              % "SN" % "dive" % "xVer" % "typ" % "midx" % "pGain" % "pOff"
01004 //                              % "tGain" % "tOff" % "sGain" % "sOff" % "" % "" % "" % "" % ""
01005 //                              % "(1/dBar)" % "(dBar)" % "(1/degC)" % "(degC)" % "(1/PSU)" % "(PSU)"
01006                               % id_instrument() % id_dive() % x_version_float()
01007                               % (int) argo_instrument_type() % 4 % gain_pressure() % offset_pressure()
01008                               % gain_temperature() % offset_temperature() % gain_salinity()
01009                               % offset_salinity()
01010                         « endl « "#" « endl;
01011                 }
01012                 { /* Argo Data Message Listing - Minor Index 5: */
01013                     const format fmt(
01014 //                                "#   %5s %4s %4s %3s %4s %11s %11s %11s %11s %11s %11s\n"
01015 //                                "#   %5s %4s %4s %3s %4s %11s %11s %12s %12s %11s %11s\n"
01016                                 "A   %5d %4d %4.1f %3d %4d %11d %11d %11d %11d %11d %10d");
01017                     os « "#      SN dive xVer typ midx   pGainSurf    pOffSurf   tGainSurf    tOffSurf
      sGainSurf   sOffSurf" « endl
01018                         « "#                                  (1/dBar)      (dBar)    (1/degC)      (degC)
      (1/PSU)      (PSU)" « endl
01019                         « format(fmt)
01020 //                              % "SN" % "dive" % "xVer" % "typ" % "midx" % "pGainSurf"
01021 //                              % "pOffSurf" % "tGainSurf" % "tOffSurf" % "sGainSurf" % "sOffSurf" % ""
01022 //                              % "" % "" % "" % "" % "(1/dBar)" % "(dBar)" % "(1/degC)" % "(degC)"
01023 //                              % "(1/PSU)" % "(PSU)"
01024                               % id_instrument() % id_dive() % x_version_float()
01025                               % (int) argo_instrument_type() % 5 % gain_pressure_surface()
01026                               % offset_pressure_surface() % gain_temperature_surface()
01027                               % offset_temperature_surface() % gain_salinity_surface()
01028                               % offset_salinity_surface();
01029                 }
01030             }
01031
01032                 break;
01033             case XARGO_INSRUMENT_TYPE_UNKNOWN:
01034             default:
01035                 os « "A (unknown float type)";
01036                 break;
01037             }
01038         }CATCH_BLOCK("Failed to generate Argo data listing string!")
01039
01040         return os.str();
01041     }
01042
```

```
01043     /** Get the maximum duration of the ascent state of the dive cycle, in minutes.
01044      *
01045      *  @return     The maximum duration of the ascent state of the dive cycle, in minutes.
01046      */
01047     uint16_t max_ascent_duration() const {
01048         return max_ascent_duration_;
01049     }
01050
01051     /** Get the maximum duration of the descent-to-drift/park-pressure phase of the dive cycle.
01052      *
01053      *  @return     The maximum duration of the descent-to-drift/park-pressure phase of the dive
01054      *              cycle.
01055      */
01056     uint16_t max_fall_to_drift_duration() const {
01057         return max_fall_to_drift_duration_;
01058     }
01059
01060     /** Get the maximum duration of the descent-to-profile-pressure phase of the dive cycle, in
01061      *  seconds.
01062      *
01063      *  @return     The maximum duration of the descent-to-profile-pressure phase of the dive
01064      *              cycle, in seconds.
01065      */
01066     uint16_t max_fall_to_profle_duration() const {
01067         return max_fall_to_profle_duration_;
01068     }
01069
01070     /** Get the maximum duration of the surface state of the dive cycle, in seconds.
01071      *
01072      *  @return     The maximum duration of the surface state of the dive cycle, in seconds.
01073      */
01074     uint16_t max_surface_duration() const {
01075         return max_surface_duration_;
01076     }
01077
01078     /** Get the mission phase.
01079      *
01080      *  @return     The mission phase.
01081      */
01082     uint8_t mission_phase() const {
01083         return mission_phase_;
01084     }
01085
01086     /** Get the maximum number of seeks to drift/park pressure.
01087      *
01088      *  @return     The maximum number of seeks to drift/park pressure.
01089      */
01090     uint16_t num_seeks_to_drift_pressure() const {
01091         return num_seeks_to_drift_pressure_;
01092     }
01093
01094     /** Get the pressure sensor offset, in dBar.
01095      *
01096      *  @return     The pressure sensor offset, in dBar.
01097      */
01098     uint16_t offset_pressure() const {
01099         return offset_pressure_;
01100     }
01101
01102     /** Get the pressure sensor offset for surface data, in dBar.
01103      *
01104      *  @return     The pressure sensor offset, in dBar.
01105      */
01106     uint16_t offset_pressure_surface() const {
01107         return offset_pressure_surface_;
01108     }
01109
01110     /** Get the salinity sensor offset, in PSU.
01111      *
01112      *  @return     The salinity sensor offset, in PSU.
01113      */
01114     uint16_t offset_salinity() const {
01115         return offset_salinity_;
01116     }
01117
01118     /** Get the salinity sensor offset for surface data, in PSU.
01119      *
01120      *  @return     The salinity sensor offset, in PSU.
01121      */
01122     uint16_t offset_salinity_surface() const {
01123         return offset_salinity_surface_;
01124     }
01125
01126     /** Get the temperature sensor offset, in degC.
01127      *
01128      *  @return     The temperature sensor offset, in degC.
01129      */
01130     uint16_t offset_temperature() const {
01131         return offset_temperature_;
01132     }
01133
01134     /** Get the temperature sensor offset for surface data, in degC.
01135      *
01136      *  @return     The temperature sensor offset, in degC.
01137      */
01138     uint16_t offset_temperature_surface() const {
01139         return offset_temperature_surface_;
01140     }
01141
01142     /** Get the seek-to-drift/park-pressure interval duration, in minutes.
01143      *
01144      *  @return     The seek-to-drift/park-pressure interval duration, in minutes.
01145      */
01146     uint16_t seek_to_drift_pressure_interval_duration() const {
01147         return seek_to_drift_pressure_interval_duration_;
01148     }
01149
01150     /** Get the start time of the surface state of the dive cycle.
```

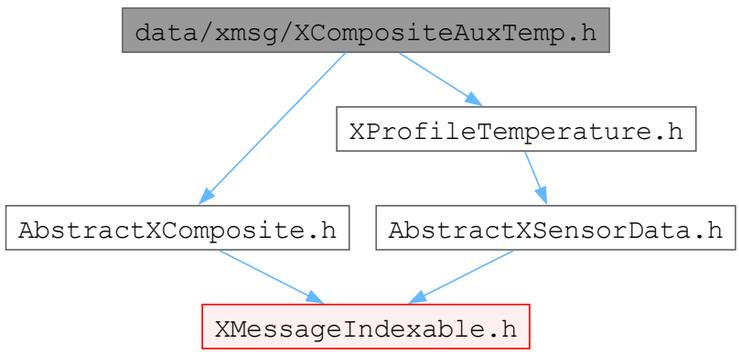
```
01151      *
01152      *  @return     The start time of the surface state of the dive cycle.
01153      */
01154     const ptime& surface_state_start_time() const {
01155         return surface_state_start_time_;
01156     }
01157
01158     /** Get the target ascent/profile rate, in cm/sec.
01159      *
01160      *  @return     The target ascent/profile rate, in cm/sec.
01161      */
01162     uint8_t target_ascent_rate() const {
01163         return target_ascent_rate_;
01164     }
01165
01166     /** Get the target druration of the drift/park phase of the dive cycle, in minutes.
01167      *
01168      *  @return     The target druration of the drift/park phase of the dive cycle, in minutes.
01169      */
01170     uint16_t target_drift_duration() const {
01171         return target_drift_duration_;
01172     }
01173
01174     /** Get the target drift/park pressure, in dBar.
01175      *
01176      *  @return     The target drift/park pressure, in dBar.
01177      */
01178     uint16_t target_drift_pressure() const {
01179         return target_drift_pressure_;
01180     }
01181
01182     /** Get the target profile pressure, in dBar.
01183      *
01184      *  @return     The target profile pressure, in dBar.
01185      */
01186     uint16_t target_profile_pressure() const {
01187         return target_profile_pressure_;
01188     }
01189
01190     /** Get the X-Message major version indicator.
01191      *
01192      *  @return     The X-Message major version indicator.
01193      */
01194     uint8_t x_version_major() const {
01195         return x_version_major_;
01196     }
01197
01198     /** Get the X-Message minor version indicator.
01199      *
01200      *  @return     The X-Message minor version indicator.
01201      */
01202     uint8_t x_version_minor() const {
01203         return x_version_minor_;
01204     }
01205
01206     /** Get the X-Message version as a floating-point value.
01207      *
01208      *  @return     The X-Message version as a floating-point value.
01209      */
01210     float x_version_float() const {
01211         float ver = x_version_minor();
01212
01213         do
01214             ver /= 10;
01215         while(ver > 1);
01216
01217         return (float) x_version_major() + ver;
01218     }
01219
01220 };
01221
01222 } /* namespace xmsg */
01223 } /* namespace util */
01224 #endif /* XARGO_H_ */
```

11.76 data/xmsg/XCompositeAuxTemp.h File Reference

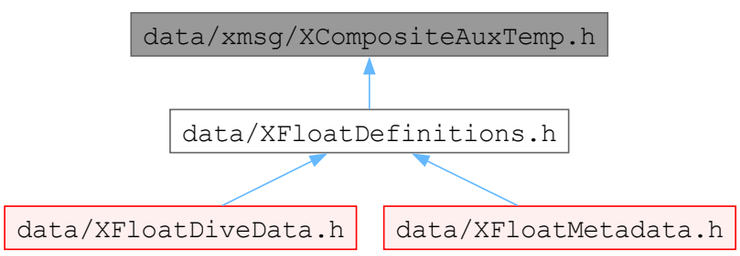
The util::xmsg::XCompositeAuxTemp class header file.

```
#include "AbstractXComposite.h"
#include "XProfileTemperature.h"
```

Include dependency graph for XCompositeAuxTemp.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XCompositeAuxTemp
  The composite auxiliary temperature class.

Namespaces

- namespace util
  The utility namespace.
- namespace util::xmsg
  The X-Message utility namespace.

11.76.1 Detailed Description

The util::xmsg::XCompositeAuxTemp class header file.

Version

0.6

Date

Jul 31, 2017 3:42:03 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.77 XCompositeAuxTemp.h

Go to the documentation of this file.

```
00001 /** The @ref util::xmsg::XCompositeAuxTemp class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jul 31, 2017 3:42:03 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef DATA_XMSG_XCOMPOSITEAUXTEMP_H_
00024 #define DATA_XMSG_XCOMPOSITEAUXTEMP_H_
00025
00026 /* Includes: */
00027
00028 #include "AbstractXComposite.h"
00029 #include "XProfileTemperature.h"
00030
00031 /* Definitions: */
00032
00033 namespace util {
00034 namespace xmsg {
00035
00036 /* Types: */
00037
00038 /** The composite auxiliary temperature class.
00039  *
00040  *  An instance of this class aggregates the auxiliary-channel profile temperature messages (see
00041  *  @ref XProfileTemperature) received for a single dive.
00042  */
00043 class XCompositeAuxTemp:
00044     public AbstractXComposite<shared_ptr<XProfileTemperature> > {
00045   public:
00046
00047     XCompositeAuxTemp() {
00048       /* Do nothing. */
00049     }
00050
00051     ~XCompositeAuxTemp() {
00052       /* Do nothing. */
00053     }
00054
00055     /** Generate the listing file section for the auxiliary temperature data.
00056      *
00057      *  Writes an @c AXT header line, then one @c axt line per sample of each auxiliary
00058      *  temperature message, with the raw counts and the temperature in degrees Celsius.
00059      *
00060      *  @param[in]  id_inst     The unique instrument identifier.
00061      *  @param[in]  id_dive     The dive or cycle number.
00062      *  @param[in]  tGain       The temperature gain used to convert counts.
00063      *  @param[in]  tOff        The temperature offset used to convert counts.
00064      *
00065      *  @return     The listing file section.
00066      */
00067     const string GenerateListingString(int id_inst, int id_dive, uint16_t tGain, uint16_t tOff) {
00068
00069       int idx = 0;
00070       ostringstream os;
00071
00072
00073       // Print the header line.
00074       static const format fmt1("#   %4s %4s");
00075       static const format fmt2("AXT %4d %4d");
00076       os « format(fmt1) % "SN" % "dive"  « endl;
00077       os « format(fmt2) % id_inst % id_dive « endl;
00078
00079
00080       static const format fmt3("#   %4s %4s %3s %11s %11s");
00081       static const format fmt4("#    %4s %4s %3s %11s %11s");
00082       static const format fmt5("axt %4d %4d %3s %11d %11.3f");
00083
00084       for(auto it = elements().begin(); it < elements().end(); ++it) {
00085
00086         const shared_ptr<XProfileTemperature> atemp = *it;
00087
00088         os « format(fmt3) % "SN" % "dive" % "idx"
00089             % "temp" % "temp" « endl;
00090         os « format(fmt4) % "" % "" % ""
00091             % "(degC)" % "(counts)" « endl;
00092
00093
00094         for(unsigned int i = 0; i < atemp->num_samples(); ++i) {
00095           os « format(fmt5) % id_inst % id_dive % idx % atemp->Sample(i)
00096                   % atemp->SampleValueAsTemperature(i, tGain, tOff) « endl;
00097         }
00098
00099         idx++;
00100
00101       }
00102
00103       return os.str();
```

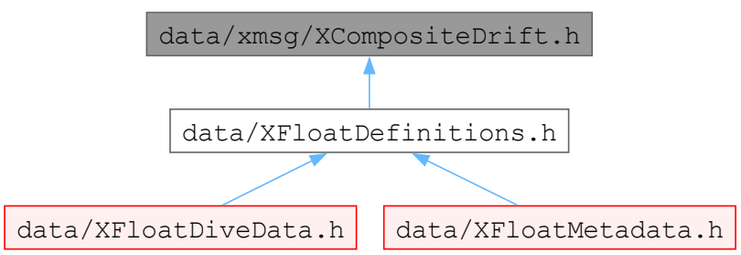
```
00104     }
00105
00106 };
00107
00108 /* Functions: */
00109
00110 } /* namespace xmsg */
00111 } /* namespace util */
00112
00113 #endif /* DATA_XMSG_XCOMPOSITEAUXTEMP_H_ */
```

11.78 data/xmsg/XCompositeDrift.h File Reference

The util::xmsg::XCompositeDrift class header file.

`#include "AbstractXSensorDataComposite.h"`
Include dependency graph for XCompositeDrift.h:

This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XCompositeDrift

  Class representing the aggregate drift data for a single dive, composed from one or more drift messages.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.78.1 Detailed Description

The util::xmsg::XCompositeDrift class header file.

Version

0.6

Date

Jul 19, 2014 12:41:37 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.79 XCompositeDrift.h

Go to the documentation of this file.

```
00001 /** The @ref util::xmsg::XCompositeDrift class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jul 19, 2014 12:41:37 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef XCOMPOSITEDRIFT_H_
00024 #define XCOMPOSITEDRIFT_H_
00025
00026 /* Includes */
00027
00028 // local
00029 #include "AbstractXSensorDataComposite.h"
00030
00031 namespace util {
00032 namespace xmsg {
00033
00034 /** Class representing the aggregate drift data for a single dive, composed from one or more drift
00035  *  messages.
00036  *
00037  */
00038 class XCompositeDrift:
00039         public AbstractXSensorDataComposite<XDriftPressure, XDriftSalinity, XDriftTemperature,
00040         XMSG_MAX_DRFT> {
00041 public:
00042
00043     /** The default constructor.
00044      *
00045      */
00046     XCompositeDrift() :
00047                     AbstractXSensorDataComposite<XDriftPressure, XDriftSalinity, XDriftTemperature,
00048                     XMSG_MAX_DRFT>() {
00049         /* Do nothing. */
00050     }
00051
00052     /** The destructor.
00053      *
00054      */
00055     ~XCompositeDrift() {
00056         /* Do nothing. */
00057     }
00058
00059     /** Generate a listing string representation of the this composite sensor data object.
00060      *
00061      *  @param[in]  id_inst The unique instrument identifier for this composite sensor data object.
00062      *  @param[in]  id_dive The dive or cycle number for this composite data object.
00063      *  @param[in]  pg      The pressure sensor sensitivity scale factor, in LSB/dBar.
00064      *  @param[in]  po      The pressure sensor offset, in dBar.
00065      *  @param[in]  sg      The salinity sensor sensitivity scale factor, in LSB/PSU.
00066      *  @param[in]  so      The salinity sensor offset, in PSU.
00067      *  @param[in]  tg      The temperature sensor sensitivity scale factor, in LSB/degC.
00068      *  @param[in]  to      The temperature sensor offset, in degC.
00069      *  @param[in]  pc      The pressure correction, in dBar, to be subtracted computed pressure
00070      *                      value.
00071      *
00072      *  @throw      log::MessageException if an error occurs while generating the listing string.
00073      *
00074      *  @note       This object will be locked for the scope of the execution of this function.
00075      *              This function places a lock guard on the mutual exclusion (mutex) lock,
00076      *              thereby locking the float object and preventing access or modification of the
00077      *              object by any other thread. Any thread that does attempt to access or modify
00078      *              this object will block until the lock is released.
```

```
00079      *
00080      *  @return     A listing string representation of this composite sensor data object.
00081      */
00082     const string GenerateListingString(
00083             int id_inst,
00084             int id_dive,
00085             uint16_t pg,
00086             uint16_t po,
00087             uint16_t sg,
00088             uint16_t so,
00089             uint16_t tg,
00090             uint16_t to,
00091             float pc) const {
00092         ostringstream os;
00093
00094         try {
00095
00096             // Get an exclusive lock.
00097             lock_guard<recursive_mutex> guard(mutex_);
00098
00099             // Print the header line.
00100 //            static const format fmt1("#   %5s %4s %11s");
00101             static const format fmt2("D   %5d %4d %11d");
00102             os « "#      SN dive  numSamples"
00103 //               « format(fmt1) % "SN" % "dive" % "numSamples"
00104                « endl
00105                « format(fmt2) % id_inst % id_dive % max_num_samples();
00106
00107             // Check if the data is complete.
00108             if(!IsDataComplete()) {
00109                 os « endl
00110                    « "#   (Incomplete Drift Data)";
00111                 return os.str();
00112             }
00113
00114             {
00115 //              static const format fmt1("#   %5s %4s %11s %11s %11s %11s");
00116 //              static const format fmt2("#    %5s %4s %11s %11s %12s %11s");
00117               static const format fmt3("d   %5d %4d %11d %11.3f %11.3f %11.3f");
00118
00119               // Iterate over the sensor data samples.
00120               for(size_t i = 0; i < max_num_samples(); i++) {
00121                 if(i % 50 == 0) {
00122                   os « endl
00123                      «
    "#-----------------------------------------------------------------------------------------------------"
00124                      « endl
00125 //                     « format(fmt1) % "SN" % "dive" % "sampleIdx" % "pres" % "temp" % "saln"
00126                      « "#      SN dive   sampleIdx        pres        temp        saln"
00127                      « endl
00128 //                     « format(fmt2) % "" % "" % "" % "(dBar)" % "(degC)" % "(PSU)";
00129                      « "#                                    (dBar)      (degC)      (PSU)";
00130                 }
00131
00132                 os « endl
00133                    « format(fmt3) % id_inst % id_dive % i % SamplePresAsFloat(i, pg, po, pc)
00134                       % SampleTempAsFloat(i, tg, to) % SampleSalnAsFloat(i, sg, so);
00135               }
00136             }
00137
00138         }CATCH_BLOCK("Failed to generate listing string for composite profile data (binned)!")
00139
00140         return os.str();
00141     }
00142
00143     /** Generate a PHY entry string from this composite drift data object.
00144      *
00145      *  @param[in]  pg      The pressure sensor sensitivity scale factor, in LSB/dBar.
00146      *  @param[in]  po      The pressure sensor offset, in dBar.
00147      *  @param[in]  sg      The salinity sensor sensitivity scale factor, in LSB/PSU.
00148      *  @param[in]  so      The salinity sensor offset, in PSU.
00149      *  @param[in]  tg      The temperature sensor sensitivity scale factor, in LSB/degC.
00150      *  @param[in]  to      The temperature sensor offset, in degC.
00151      *  @param[in]  pc      The pressure correction, in dBar, to be subtracted computed pressure
00152      *                      value.
00153      *
00154      *  @throw      log::MessageException if an error occurs while generating the PHY string.
00155      *
00156      *  @return     A PHY entry string representation of this composite drift dataobject.
00157      */
00158     const string GeneratePhyString(
00159             uint16_t pg,
00160             uint16_t po,
00161             uint16_t sg,
00162             uint16_t so,
00163             uint16_t tg,
00164             uint16_t to,
00165             float pc) const {
00166         ostringstream os;
00167
00168         try {
00169
00170             // Get an exclusive lock.
00171             lock_guard<recursive_mutex> guard(mutex_);
00172
00173             if(num_saln_samples()) {
00174
00175                 static const format fmt("%8.2f %7.3f %7.3f");
00176
00177                 // Iterate over the sensor data samples.
00178                 for(size_t i = 0; i < max_num_samples(); i++)
00179                     os « endl
00180                        « format(fmt) % SamplePresAsFloat(i, pg, po, pc)
00181                                % SampleTempAsFloat(i, tg, to) % SampleSalnAsFloat(i, sg, so);
00182
00183             } else {
00184
00185                 static const format fmt("%8.2f %7.3f");
```

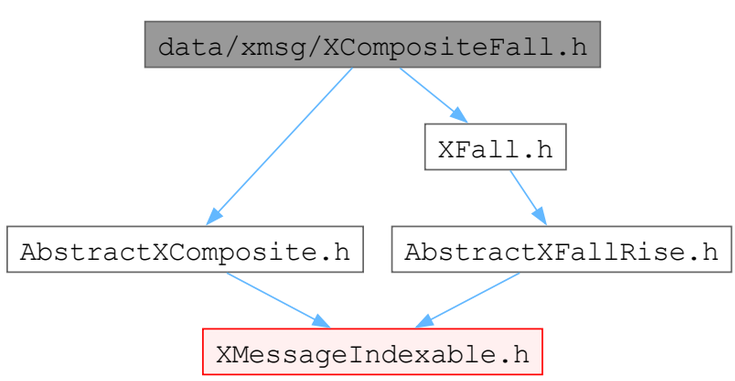
```
00186
00187                 // Iterate over the sensor data samples.
00188                 for(size_t i = 0; i < max_num_samples(); i++)
00189                     os « endl
00190                        « format(fmt) % SamplePresAsFloat(i, pg, po, pc)
00191                                % SampleTempAsFloat(i, tg, to);
00192             }
00193
00194         }CATCH_BLOCK("Failed to generate PHY string for composite drift data!")
00195
00196         return os.str();
00197     }
00198
00199 };
00200
00201 } /* namespace xmsg */
00202 } /* namespace util */
00203
00204 #endif /* XCOMPOSITEDRIFT_H_ */
```

11.80 data/xmsg/XCompositeFall.h File Reference

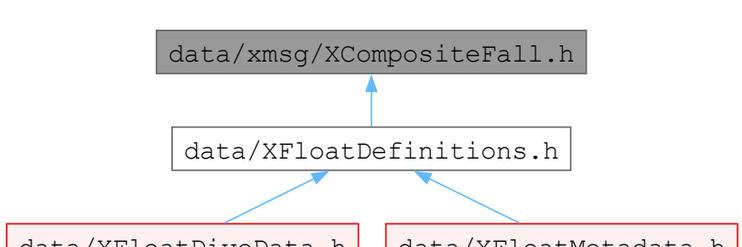
The util::xmsg::XCompositeFall class header file.

```
#include "AbstractXComposite.h"
#include "XFall.h"
```

Include dependency graph for XCompositeFall.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XCompositeFall

  Class representing the aggregate fall time-series data for a single dive, composed from one or more fall time-series messages.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.80.1 Detailed Description

The util::xmsg::XCompositeFall class header file.

Version

0.6

Date

Jul 24, 2014 2:10:58 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.81 XCompositeFall.h

Go to the documentation of this file.

```
00001 /** The @ref util::xmsg::XCompositeFall class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jul 24, 2014 2:10:58 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef XCOMPOSITEFALL_H_
00024 #define XCOMPOSITEFALL_H_
00025
00026 /* Includes: */
00027
00028 // local
00029 #include "AbstractXComposite.h"
00030 #include "XFall.h"
00031
00032 /* Definitions: */
00033
00034 namespace util {
00035 namespace xmsg {
00036
00037
00038 /* Class Definition: */
00039
00040 /** Class representing the aggregate fall time-series data for a single dive, composed from one or
00041  *  more fall time-series messages.
00042  */
00043 class XCompositeFall: public AbstractXComposite<shared_ptr<XFall> > {
00044 public:
00045
00046     /* Member Functions */
00047
00048     /** The default constructor.
00049      *
00050      */
00051     XCompositeFall() {
00052         /* Do nothing. */
00053
00054     }
00055
00056     /** The destructor.
```

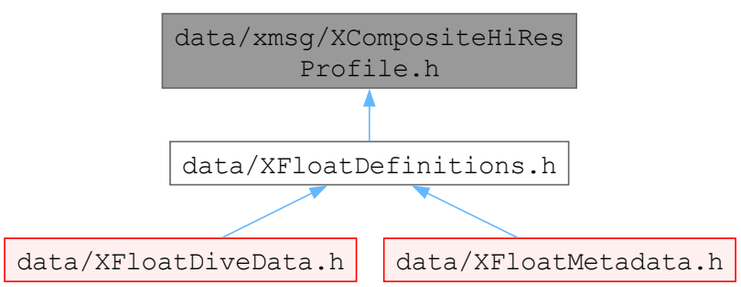
```
00057      *
00058      */
00059     ~XCompositeFall() {
00060         /* Do nothing. */
00061     }
00062
00063     /** Generate a listing entry string from this composite fall time-series data object.
00064      *
00065      *  @param[in]  pg      The pressure sensor sensitivity scale factor, in LSB/dBar.
00066      *  @param[in]  po      The pressure sensor offset, in dBar.
00067      *  @param[in]  sg      The salinity sensor sensitivity scale factor, in LSB/PSU.
00068      *  @param[in]  so      The salinity sensor offset, in PSU.
00069      *  @param[in]  tg      The temperature sensor sensitivity scale factor, in LSB/degC.
00070      *  @param[in]  to      The temperature sensor offset, in degC.
00071      *  @param[in]  pc      The pressure correction, in dBar, to be subtracted computed pressure
00072      *                      value.
00073      *
00074      *  @throw      log::MessageException if an error occurs while generating the listing string.
00075      *
00076      *  @return     A listing entry string representation of this composite fall time-series data
00077      *              object.
00078      */
00079     const string GenerateListingString(
00080             uint16_t pg,
00081             uint16_t po,
00082             uint16_t sg,
00083             uint16_t so,
00084             uint16_t tg,
00085             uint16_t to,
00086             float pc) const  {
00087         ostringstream os;
00088
00089         try {
00090             os « AbstractXFallRise::GenerateListingComment();
00091             for(auto it = elements().begin(); it < elements().end(); it++)
00092                 if(*it) {
00093                     if(it > elements().begin()) {
00094                         os « endl;
00095                         XMSG_SECTION_BREAK();
00096                         os « XFall::GenerateListingComment();
00097                     }
00098                     os « endl « (*it)->GenerateListingString(pg, po, sg, so, tg, to, pc);
00099                 }
00100
00101         }CATCH_BLOCK("Failed to generate listing string for composite fall data object!")
00102
00103         return os.str();
00104     }
00105 };
00106
00107 } /* namespace xmsg */
00108 } /* namespace util */
00109 #endif /* XCOMPOSITEFALL_H_ */
```

11.82 data/xmsg/XCompositeHiResProfile.h File Reference

The util::xmsg::XCompositeHiResProfile class header file.

`#include "AbstractXSensorDataComposite.h"`
Include dependency graph for XCompositeHiResProfile.h:

This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XCompositeHiResProfile

  Class representing the aggregate raw/high-resolution/unbinned profile data for a single dive, composed from one or more raw profile data messages.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.82.1 Detailed Description

The util::xmsg::XCompositeHiResProfile class header file.

Version

0.6

Date

Jul 31, 2014 2:25:18 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.83 XCompositeHiResProfile.h

Go to the documentation of this file.

```
00001 /** The @ref util::xmsg::XCompositeHiResProfile class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jul 31, 2014 2:25:18 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022 #ifndef XCOMPOSITEHIRESPROFILE_H_
00023 #define XCOMPOSITEHIRESPROFILE_H_
00024
00025 /* Includes */
00026
00027 // local
00028 #include "AbstractXSensorDataComposite.h"
00029
00030 /* Definitions */
00031
```

```
00032 namespace util {
00033 namespace xmsg {
00034
00035 /* Class Definitions */
00036
00037 /** Class representing the aggregate raw/high-resolution/unbinned profile data for a single dive,
00038  *  composed from one or more raw profile data messages.
00039  *
00040  */
00041 class XCompositeHiResProfile:
00042     public AbstractXSensorDataComposite<XHiResPressure, XHiResSalinity, XHiResTemperature,
00043     XMSG_MAX_PROF_HIRES> {
00044
00045   public:
00046
00047     /* Member Functions */
00048
00049     /** The default constructor.
00050      *
00051      */
00052     XCompositeHiResProfile() :
00053             AbstractXSensorDataComposite<XHiResPressure, XHiResSalinity, XHiResTemperature,
00054             XMSG_MAX_PROF_HIRES>() {
00055       /* Do nothing. */
00056
00057     }
00058
00059     /** The destructor.
00060      *
00061      */
00062     ~XCompositeHiResProfile() {
00063       /* Do nothing. */
00064     }
00065
00066     /** Generate a listing string representation of the this composite sensor data object.
00067      *
00068      *  @param[in]  id_inst The unique instrument identifier for this composite sensor data
00069      *                      object.
00070      *  @param[in]  id_dive The dive or cycle number for this composite data object.
00071      *  @param[in]  pg      The pressure sensor sensitivity scale factor, in LSB/dBar.
00072      *  @param[in]  po      The pressure sensor offset, in dBar.
00073      *  @param[in]  sg      The salinity sensor sensitivity scale factor, in LSB/PSU.
00074      *  @param[in]  so      The salinity sensor offset, in PSU.
00075      *  @param[in]  tg      The temperature sensor sensitivity scale factor, in LSB/degC.
00076      *  @param[in]  to      The temperature sensor offset, in degC.
00077      *  @param[in]  pc      The pressure correction, in dBar, to be subtracted computed pressure
00078      *                      value.
00079      *
00080      *  @throw      log::MessageException if an error occurs while generating the listing string.
00081      *
00082      *  @note       This object will be locked for the scope of the execution of this function.
00083      *              This function places a lock guard on the mutual exclusion (mutex) lock,
00084      *              thereby locking the float object and preventing access or modification of the
00085      *              object by any other thread. Any thread that does attempt to access or modify
00086      *              this object will block until the lock is released.
00087      *
00088      *  @return     A listing string representation of this composite sensor data object.
00089      */
00090     const string GenerateListingString(
00091         int id_inst,
00092         int id_dive,
00093         uint16_t pg,
00094         uint16_t po,
00095         uint16_t sg,
00096         uint16_t so,
00097         uint16_t tg,
00098         uint16_t to,
00099         float pc) const {
00100       ostringstream os;
00101
00102       try {
00103
00104         // Get an exclusive lock.
00105         lock_guard<recursive_mutex> guard(mutex_);
00106
00107         // Print the header line.
00108         static const format fmt1("#   %5s %4s %11s");
00109         static const format fmt2("PRU %5d %4d %11d");
00110         os « format(fmt1) % "SN" % "dive" % "numSamples" « endl;
00111         os « format(fmt2) % id_inst % id_dive % max_num_samples();
00112
00113         // Check if the data is complete.
00114         if(!IsDataComplete()) {
00115           os « endl « "#   (Incomplete Unbinned Profile Data)";
00116           return os.str();
00117         }
00118
00119         {
00120           static const format fmt1("#    %5s %4s %11s %11s %11s %11s");
00121           static const format fmt2("#    %5s %4s %11s %11s %12s %11s");
00122           static const format fmt3("pru %5d %4d %11d %11.3f %11.3f %11.3f");
00123
00124           // Iterate over the sensor data samples.
00125           for(size_t i = 0; i < max_num_samples(); i++) {
00126             if(i % 50 == 0) {
00127 //              os « endl
00128 //                 «
      "#-------------------------------------------------------------------------------------------------";
00129 //              os « endl « format(fmt1) % "SN" % "dive" % "sampleIdx" % "pres" % "temp" % "saln";
00130 //              os « endl « format(fmt2) % "" % "" % "" % "(dBar)" % "(degC)" % "(PSU)";
00131               os « endl
00132                  «
      "#-------------------------------------------------------------------------------------------------"
00133                  « endl
00134 //                 « format(fmt1) % "SN" % "dive" % "sampleIdx" % "pres" % "temp" % "saln"
00135                  « "#      SN dive   sampleIdx        pres        temp        saln"
00136                  « endl
00137 //                 « format(fmt2) % "" % "" % "" % "(dBar)" % "(degC)" % "(PSU)";
```

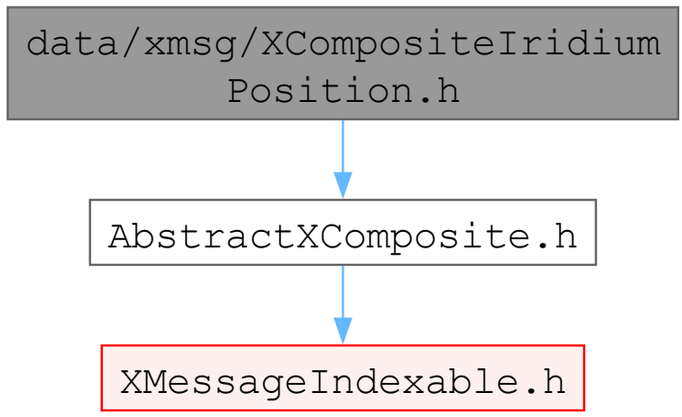
```
00138                 << "#                                (dBar)       (degC)       (PSU)";
00139             }
00140
00141             os << endl
00142                << format(fmt3) % id_inst % id_dive % i % SamplePresAsFloat(i, pg, po, pc)
00143                    % SampleTempAsFloat(i, tg, to) % SampleSalnAsFloat(i, sg, so);
00144           }
00145         }
00146
00147       }CATCH_BLOCK("Failed to generate listing string for composite drift data!")
00148
00149       return os.str();
00150     }
00151
00152     /** Generate a PHY entry string from this composite bin-averaged profile data object.
00153      *
00154      *  @param[in]  pg      The pressure sensor sensitivity scale factor, in LSB/dBar.
00155      *  @param[in]  po      The pressure sensor offset, in dBar.
00156      *  @param[in]  sg      The salinity sensor sensitivity scale factor, in LSB/PSU.
00157      *  @param[in]  so      The salinity sensor offset, in PSU.
00158      *  @param[in]  tg      The temperature sensor sensitivity scale factor, in LSB/degC.
00159      *  @param[in]  to      The temperature sensor offset, in degC.
00160      *  @param[in]  pc      The pressure correction, in dBar, to be subtracted computed pressure
00161      *                      value.
00162      *
00163      *  @throw      log::MessageException if an error occurs while generating the PHY string.
00164      *
00165      *  @return     A PHY entry string representation of this composite raw profile data object.
00166      */
00167     const string GeneratePhyString(
00168         uint16_t pg,
00169         uint16_t po,
00170         uint16_t sg,
00171         uint16_t so,
00172         uint16_t tg,
00173         uint16_t to,
00174         float pc) const {
00175       ostringstream os;
00176
00177       try {
00178
00179         // Get an exclusive lock.
00180         lock_guard<recursive_mutex> guard(mutex_);
00181
00182         if(num_saln_samples()) {
00183
00184           static const format fmt("%8.2f %7.3f %7.3f");
00185
00186           // Iterate over the sensor data samples.
00187           for(size_t i = 0; i < max_num_samples(); i++)
00188             os << endl
00189                << format(fmt) % SamplePresAsFloat(i, pg, po, pc) % SampleTempAsFloat(i, tg, to)
00190                    % SampleSalnAsFloat(i, sg, so);
00191
00192         } else {
00193
00194           static const format fmt("%8.2f %7.3f");
00195
00196           // Iterate over the sensor data samples.
00197           for(size_t i = 0; i < max_num_samples(); i++)
00198             os << endl
00199                << format(fmt) % SamplePresAsFloat(i, pg, po, pc) % SampleTempAsFloat(i, tg, to);
00200         }
00201
00202       }CATCH_BLOCK("Failed to generate PHY string for composite raw profile data!")
00203
00204       return os.str();
00205     }
00206
00207 };
00208
00209 } /* namespace xmsg */
00210 } /* namespace util */
00211 #endif /* XCOMPOSITEHIRESPROFILE_H_ */
```

11.84 data/xmsg/XCompositeIridiumPosition.h File Reference

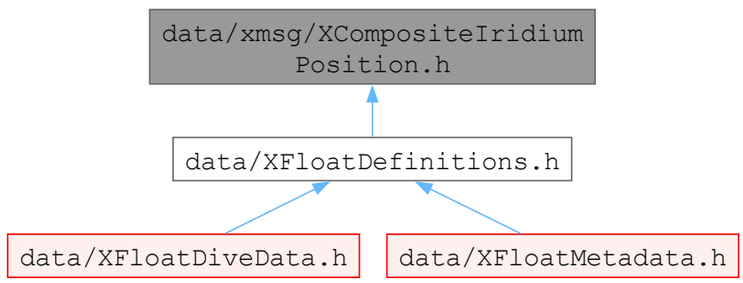
The util::xmsg::XCompositeIridiumPosition class header file.

```
#include "AbstractXComposite.h"
```

Include dependency graph for XCompositeIridiumPosition.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XCompositeIridiumPosition

  Class representing the aggregate Iridium position for a single dive, composed from one or more Iridium message positions.

Namespaces

- namespace argo

  The Argo instrument namespace.
- namespace argo::iridium

  The Argo Iridium instrument namespace.
- namespace argo::iridium::xfloat

  The X-Float instrument namespace.
- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

Macros

- #define XMSG_POS_INVALID_LAT UTIL_POS_INVALID_LAT

  Default/fill value for invalid position latitude.
- #define XMSG_POS_INVALID_LON UTIL_POS_INVALID_LON

  Default/fill value for invalid position longitude.
- #define XMSG_POS_INVALID_STDERR UTIL_POS_INVALID_STDERR

  Default/fill value for invalid position standard error.
- #define XMSG_POS_INVALID_VAR UTIL_POS_INVALID_VAR

  Default/fill value for invalid position variance.

11.84.1 Detailed Description

The util::xmsg::XCompositeIridiumPosition class header file.

Version

0.6

Date

Jul 25, 2014 1:47:14 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.84.2 Macro Definition Documentation

11.84.2.1 XMSG_POS_INVALID_LAT

```
#define XMSG_POS_INVALID_LAT UTIL_POS_INVALID_LAT
```

Default/fill value for invalid position latitude.

Referenced by util::xmsg::XCompositeIridiumPosition::ComputeAverageIrdiumPosition().

11.84.2.2 XMSG_POS_INVALID_LON

```
#define XMSG_POS_INVALID_LON UTIL_POS_INVALID_LON
```

Default/fill value for invalid position longitude.

Referenced by util::xmsg::XCompositeIridiumPosition::ComputeAverageIrdiumPosition().

11.84.2.3 XMSG_POS_INVALID_STDERR

```
#define XMSG_POS_INVALID_STDERR UTIL_POS_INVALID_STDERR
```

Default/fill value for invalid position standard error.

Referenced by util::xmsg::XCompositeIridiumPosition::ComputeAverageIrdiumPosition().

11.84.2.4 XMSG_POS_INVALID_VAR

```
#define XMSG_POS_INVALID_VAR UTIL_POS_INVALID_VAR
```

Default/fill value for invalid position variance.

Referenced by util::xmsg::XCompositeIridiumPosition::ComputeAverageIrdiumPosition().

11.85 XCompositeIridiumPosition.h

Go to the documentation of this file.

```
00001 /** The @ref util::xmsg::XCompositeIridiumPosition class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jul 25, 2014 1:47:14 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef XCOMPOSITEIRIDIUMPOSITION_H_
00024 #define XCOMPOSITEIRIDIUMPOSITION_H_
00025
00026 /* Includes */
00027
00028 // local
00029 #include "AbstractXComposite.h"
00030
00031 /* Definitions */
00032
00033 /** Default/fill value for invalid position latitude. */
00034 #define XMSG_POS_INVALID_LAT        UTIL_POS_INVALID_LAT
00035
00036 /** Default/fill value for invalid position longitude. */
00037 #define XMSG_POS_INVALID_LON        UTIL_POS_INVALID_LON
00038
00039 /** Default/fill value for invalid position standard error. */
00040 #define XMSG_POS_INVALID_STDERR     UTIL_POS_INVALID_STDERR
00041
00042 /** Default/fill value for invalid position variance. */
00043 #define XMSG_POS_INVALID_VAR        UTIL_POS_INVALID_VAR
00044
00045 namespace argo {
00046 namespace iridium {
00047 namespace xfloat {
00048 // forward declaration
00049 class XFloatMailMessage;
00050 } /* namespace argo */
00051 } /* namespace iridium */
00052 } /* namespace xfloat */
00053
00054 namespace util {
00055 namespace xmsg {
00056
00057 /* Class Definitions */
00058
00059 /** Class representing the aggregate Iridium position for a single dive, composed from one or more
00060  *  Iridium message positions.
00061  */
00062 class XCompositeIridiumPosition:
00063         public AbstractXComposite<shared_ptr<argo::iridium::xfloat::XFloatMailMessage> >,
00064         public util::position::Position {
00065 private:
00066
00067     /* Data Members */
00068
00069     /** The calculated standard error of this composite Iridium position, in km. */
00070     double position_standard_error_;
00071
00072     /** The calculated variance of this composite Iridium position, in km^2. */
00073     double position_variance_;
00074
00075     /** The Iridium session communication time, in seconds.
00076      *
00077      *  @note           This value is calculated from the difference in time between the first and
00078      *                  last Iridium messages that are included in this composite Iridium position.
00079      */
00080     int total_satellite_comm_time_seconds_;
00081
00082 protected:
00083
00084     /* Member Functions */
00085
00086     /** Set the calculated standard error of this composite Iridium position, in km.
00087      *
00088      *  @param[in]  position_standard_error The calculated standard error of this composite
00089      *                                      Iridium position, in km
00090      */
00091     void set_position_standard_error(double position_standard_error) {
00092         position_standard_error_ = position_standard_error;
00093     }
00094
00095     /** Set the calculated variance of this composite Iridium position, in km^2.
00096      *
00097      *  @param[in] position_variance        The calculated variance of this composite Iridium
00098      *                                      position, in km^2
00099      */
00100     void set_position_variance(double position_variance) {
00101         position_variance_ = position_variance;
00102     }
00103
```

```
00104     /** Set the Iridium session communication time, in seconds.
00105      *
00106      *  @param[in]  time                    The Iridium session communication time, in seconds.
00107      */
00108     void set_total_satellite_comm_time_seconds(int time) {
00109         total_satellite_comm_time_seconds_ = time;
00110     }
00111
00112 public:
00113     /* Member Functions */
00114
00115     /** The default constructor.
00116      *
00117      */
00118     XCompositeIridiumPosition() :
00119                     Position(),
00120                     position_standard_error_(XMSG_POS_INVALID_STDERR),
00121                     position_variance_(XMSG_POS_INVALID_VAR),
00122                     total_satellite_comm_time_seconds_(0) {
00123         /* Do nothing. */
00124     }
00125
00126     /** The destructor.
00127      *
00128      */
00129     ~XCompositeIridiumPosition() {
00130         /* Do nothing. */
00131     }
00132
00133     void ComputeAverageIrdiumPosition();
00134
00135     /** Generate the listing entry comment.
00136      *
00137      *  @return         The listing entry comment.
00138      */
00139     static const string GenerateListingComment() {
00140         ostringstream os;
00141
00142         const format fmt("#   %4s %4s %20s %21s %10s %10s %10s %10s\n"
00143                          "#    %4s %4s %20s %21s %10s %10s %10s %10s");
00144
00145         try {
00146 //             os « format(fmt) % "SN" % "dive" % "weighted-avg-time" % "weighted-avg-pos" //
00147 //                    % "nPos" % "variance" % "stdError" % "satTime" //
00148 //                    % "" % "" % "" % "" //
00149 //                    % "" % "(km^2)" % "(km)" % "(sec)";
00150
00151           os « "#      SN dive    weighted-avg-time    weighted-avg-pos       nPos    variance    stdError
      satTime"
00152              « endl
00153              « "#                                                                          (km^2)        (km)
      (sec)";
00154
00155         }CATCH_BLOCK("Failed to generate composite iridium position listing comment!")
00156
00157         return os.str();
00158     }
00159
00160     /** Generate a listing entry string from this composite Iridum position object.
00161      *
00162      *  @param[in]      whoi        The unique WHOI instrument identifier or serial number.
00163      *  @param[in]      dive        The dive or cycle number of this composite Iridum position object.
00164      *
00165      *  @return         A listing entry string from this composite Iridum position object.
00166      */
00167     const string GenerateListingString(int whoi, int dive) const {
00168         ostringstream os;
00169
00170         try {
00171             const format fmt("IRI %5d %4d %20s %19s %10d %10.2f %10.2f %10d");
00172             os « GenerateListingComment() « endl
00173                     « format(fmt) % whoi % dive % position_datetime_as_iso8601_string()
00174                             % postition_as_iso6709_string() % elements().size()
00175                             % position_variance_ % position_standard_error_ %
      total_satellite_comm_time_seconds_;
00176
00177         }CATCH_BLOCK("Failed to generate listing string from composite Iridium position!")
00178
00179         return os.str();
00180     }
00181
00182     /** Get the calculated variance of this composite Iridium position, in km^2.
00183      *
00184      *  @return         The calculated variance of this composite Iridium position, in km^2.
00185      */
00186     double position_standard_error() const {
00187         return position_standard_error_;
00188     }
00189
00190     /** Get the calculated variance of this composite Iridium position, in km^2.
00191      *
00192      *  @return         The calculated variance of this composite Iridium position, in km^2.
00193      */
00194     double position_variance() const {
00195         return position_variance_;
00196     }
00197
00198     /** Get the Iridium session communication time, in seconds.
00199      *
00200      *  @return         The Iridium session communication time, in seconds.
00201      */
00202     int total_satellite_comm_time_seconds() const {
00203         return total_satellite_comm_time_seconds_;
00204     }
00205
00206 };
00207
00208 } /* namespace xmsg */
```

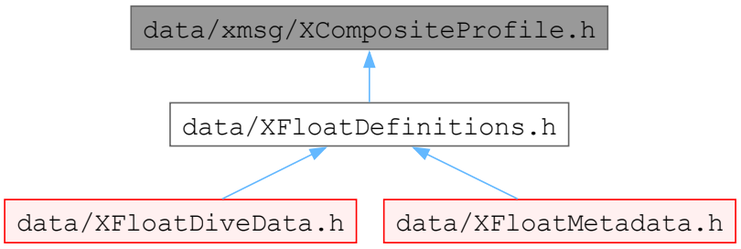
```
00209 } /* namespace util */
00210 #endif /* XCOMPOSITEIRIDIUMPOSITION_H_ */
```

11.86 data/xmsg/XCompositeProfile.h File Reference

The util::xmsg::XCompositeProfile class header file.

`#include "AbstractXSensorDataComposite.h"`
Include dependency graph for XCompositeProfile.h:

This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XCompositeProfile

  Class representing the aggregate bin-averaged profile data for a single dive, composed from one or more bin-averaged profile data messages.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.86.1 Detailed Description

The util::xmsg::XCompositeProfile class header file.

Version

0.6

Date

Jul 17, 2014 3:07:43 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.87 XCompositeProfile.h

Go to the documentation of this file.

```
00001 /** The @ref util::xmsg::XCompositeProfile class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jul 17, 2014 3:07:43 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef XCOMPOSITEPROFILE_H_
00024 #define XCOMPOSITEPROFILE_H_
00025
00026 /* Includes */
00027
00028 // local
00029 #include "AbstractXSensorDataComposite.h"
00030
00031 /* Definitions */
00032
00033 namespace util {
00034 namespace xmsg {
00035
00036 /* Class Definitions */
00037
00038 /** Class representing the aggregate bin-averaged profile data for a single dive, composed from one
00039  *  or more bin-averaged profile data messages.
00040  */
00041 class XCompositeProfile:
00042     public AbstractXSensorDataComposite<XProfilePressure, XProfileSalinity, XProfileTemperature,
00043     XMSG_MAX_PROF> {
00044
00045   public:
00046
00047     /* Public Member Functions: */
00048
00049     /** The default constructor.
00050      *
00051      */
00052     XCompositeProfile() :
00053             AbstractXSensorDataComposite<XProfilePressure, XProfileSalinity, XProfileTemperature,
00054             XMSG_MAX_PROF>() {
00055       /* Do nothing. */
00056     }
00057
00058     /** The destructor.
00059      *
00060      */
00061     ~XCompositeProfile() {
00062       /* Do nothing. */
00063     }
00064
00065     /** Generate a listing string representation of the this composite sensor data object.
00066      *
00067      *  @param[in]  id_inst The unique instrument identifier for this composite sensor data object.
00068      *  @param[in]  id_dive The dive or cycle number for this composite data object.
00069      *  @param[in]  pg      The pressure sensor sensitivity scale factor, in LSB/dBar.
00070      *  @param[in]  po      The pressure sensor offset, in dBar.
00071      *  @param[in]  sg      The salinity sensor sensitivity scale factor, in LSB/PSU.
00072      *  @param[in]  so      The salinity sensor offset, in PSU.
00073      *  @param[in]  tg      The temperature sensor sensitivity scale factor, in LSB/degC.
00074      *  @param[in]  to      The temperature sensor offset, in degC.
00075      *  @param[in]  pc      The pressure correction, in dBar, to be subtracted computed pressure
00076      *                      value.
00077      *
00078      *  @throw      log::MessageException if an error occurs while generating the listing string.
00079      *
00080      *  @exception  util::log::MessageException if @p x if an error occurs while generating a
00081      *              listing string representation of this composite sensor data object.
00082      *
00083      *  @note       This object will be locked for the scope of the execution of this function.
00084      *              This function places a lock guard on the mutual exclusion (mutex) lock,
00085      *              thereby locking the float object and preventing access or modification of the
00086      *              object by any other thread. Any thread that does attempt to access or modify
00087      *              this object will block until the lock is released.
00088      *
00089      *  @return     A listing string representation of this composite sensor data object.
00090      */
00091     const string GenerateListingString(
00092         int id_inst,
00093         int id_dive,
00094         uint16_t pg,
00095         uint16_t po,
00096         uint16_t sg,
00097         uint16_t so,
00098         uint16_t tg,
00099         uint16_t to,
00100         float pc) const {
00101       ostringstream os;
00102
00103       try {
```

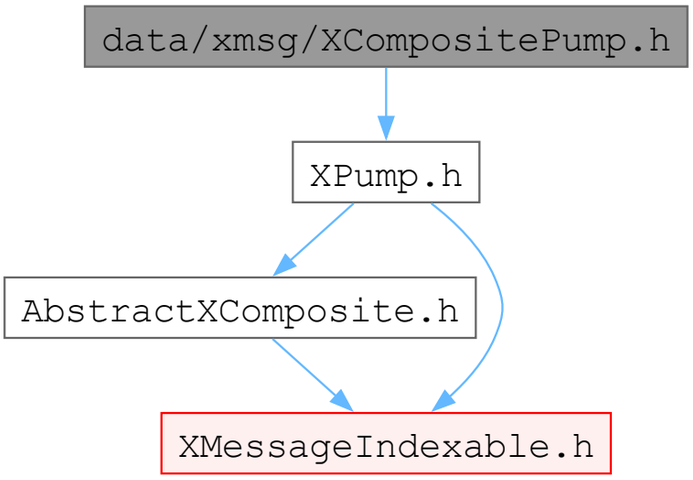
```
00104
00105         // Get an exclusive lock.
00106         lock_guard<recursive_mutex> guard(mutex_);
00107
00108         { // Print the header line.
00109 //          static const format fmt1("#   %5s %4s %11s");
00110           static const format fmt2("PRO %5d %4d %11d");
00111           os « "#      SN dive  numSamples"
00112 //             « format(fmt1) % "SN" % "dive" % "numSamples"
00113              « endl
00114              « format(fmt2) % id_inst % id_dive % max_num_samples();
00115         }
00116
00117         // Check if the data is complete.
00118         if(!IsDataComplete()) {
00119           os « endl « "#   (Incomplete Binned Profile Data)";
00120           return os.str();
00121         }
00122
00123         if(HasAuxTempData()) {
00124           static const format fmt1("#   %5s %4s %11s %11s %11s %11s %11s");
00125           static const format fmt2("#    %5s %4s %11s %11s %12s %12s %11s");
00126           static const format fmt3("pro %5d %4d %11d %11.3f %11.3f %11.3f %11.3f");
00127
00128           // Iterate over the sensor data samples.
00129           for(size_t i = 0; i < max_num_samples(); i++) {
00130             if(i % 50 == 0) {
00131               os « endl
00132                  «
    "#-------------------------------------------------------------------------------------------------------"
00133                  « endl
00134                  « "#      SN dive   sampleIdx        pres       temp1       temp2        saln"
00135                  « endl
00136                  « "#                                (dBar)      (degC)      (degC)       (PSU)"
00137                  ;
00138             }
00139
00140               os « endl
00141                  « format(fmt3) % id_inst % id_dive % i % SamplePresAsFloat(i, pg, po, pc)
00142                      % SampleTempAsFloat(i, tg, to) % SampleAuxTempAsFloat(i, tg, to)
00143                      % SampleSalnAsFloat(i, sg, so);
00144           }
00145
00146         } else {
00147
00148           static const format fmt3("pro %5d %4d %11d %11.3f %11.3f %11.3f");
00149
00150           // Iterate over the sensor data samples.
00151           for(size_t i = 0; i < max_num_samples(); i++) {
00152             if(i % 50 == 0) {
00153               os « endl
00154                  «
    "#-------------------------------------------------------------------------------------------------------"
00155                  « endl
00156                  « "#      SN dive   sampleIdx        pres        temp        saln"
00157                  « endl
00158                  « "#                                (dBar)      (degC)       (PSU)";
00159             }
00160
00161               os « endl
00162                  « format(fmt3) % id_inst % id_dive % i % SamplePresAsFloat(i, pg, po, pc)
00163                      % SampleTempAsFloat(i, tg, to) % SampleSalnAsFloat(i, sg, so);
00164           }
00165         }
00166
00167       }CATCH_BLOCK("Failed to generate listing string for composite profile data (binned)!")
00168
00169       return os.str();
00170     }
00171
00172     /** Generate a PHY entry string from this composite bin-averaged profile data object.
00173      *
00174      *  @param[in]  pg      The pressure sensor sensitivity scale factor, in LSB/dBar.
00175      *  @param[in]  po      The pressure sensor offset, in dBar.
00176      *  @param[in]  sg      The salinity sensor sensitivity scale factor, in LSB/PSU.
00177      *  @param[in]  so      The salinity sensor offset, in PSU.
00178      *  @param[in]  tg      The temperature sensor sensitivity scale factor, in LSB/degC.
00179      *  @param[in]  to      The temperature sensor offset, in degC.
00180      *  @param[in]  pc      The pressure correction, in dBar, to be subtracted computed pressure
00181      *                      value.
00182      *
00183      *  @throw      log::MessageException if an error occurs while generating the PHY string.
00184      *
00185      *  @return     A PHY entry string representation of this composite bin-averaged profile data
00186      *              object.
00187      */
00188     const string GeneratePhyString(
00189         uint16_t pg,
00190         uint16_t po,
00191         uint16_t sg,
00192         uint16_t so,
00193         uint16_t tg,
00194         uint16_t to,
00195         float pc) const {
00196       ostringstream os;
00197
00198       try {
00199
00200         // Get an exclusive lock.
00201         lock_guard<recursive_mutex> guard(mutex_);
00202
00203         if(num_saln_samples()) {
00204
00205           static const format fmt("%8.2f %7.3f %7.3f");
00206
00207
00208           // Ensure correct ordering
00209           if(num_saln_samples() > 1 && SamplePresAsFloat(0, pg, po, pc) > SamplePresAsFloat(1, pg, po,
```

```
      pc)) {
00210
00211             // Iterate over the sensor data samples.
00212             for(size_t i = 0; i < max_num_samples(); i++)
00213               if(SamplePresAsFloat(i, pg, po, 0) > 1)
00214                 os « endl « format(fmt) % SamplePresAsFloat(i, pg, po, pc) % SampleTempAsFloat(i, tg, to)
      % SampleSalnAsFloat(i, sg, so);
00215           } else {
00216
00217             size_t mx = max_num_samples() - 1;
00218
00219             // Iterate over the sensor data samples.
00220             for(size_t i = 0; i < max_num_samples(); i++)
00221               if(SamplePresAsFloat(i, pg, po, 0) > 1)
00222                 os « endl « format(fmt) % SamplePresAsFloat(mx - i, pg, po, pc) % SampleTempAsFloat(mx -
      i, tg, to) % SampleSalnAsFloat(mx - i, sg, so);
00223           }
00224
00225         } else {
00226
00227           static const format fmt("%8.2f %7.3f");
00228
00229           // Iterate over the sensor data samples.
00230           for(size_t i = 0; i < max_num_samples(); i++)
00231             os « endl  « format(fmt) % SamplePresAsFloat(i, pg, po, pc) % SampleTempAsFloat(i, tg, to);
00232         }
00233
00234       }CATCH_BLOCK("Failed to generate PHY string for composite binned profile data!")
00235
00236       return os.str();
00237     }
00238
00239 };
00240
00241 } /* namespace xmsg */
00242 } /* namespace util */
00243 #endif /* XCOMPOSITEPROFILE_H_ */
```

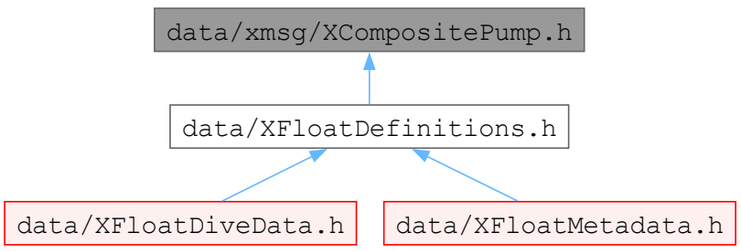
11.88 data/xmsg/XCompositePump.h File Reference

The util::xmsg::XCompositePump class header file.

`#include "XPump.h"`
Include dependency graph for XCompositePump.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XCompositePump

  Class representing the aggregate pump time-series data for a single dive, composed from one or more pump time-series messages.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.88.1 Detailed Description

The util::xmsg::XCompositePump class header file.

Version

0.6

Date

Jul 22, 2014 12:46:38 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.89 XCompositePump.h

Go to the documentation of this file.

```
00001 /** The @ref util::xmsg::XCompositePump class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jul 22, 2014 12:46:38 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023
00024 #ifndef XCOMPOSITEPUMP_H_
00025 #define XCOMPOSITEPUMP_H_
00026
00027 /* Includes */
00028
00029 // local
00030 #include "XPump.h"
00031
00032 /* Definitions */
00033
00034 namespace util {
00035 namespace xmsg {
00036
00037 /* Class Definitions */
00038
00039 /** Class representing the aggregate pump time-series data for a single dive, composed from one or
00040  *  more pump time-series messages.
00041  */
00042 class XCompositePump: public AbstractXComposite<shared_ptr<XPump> > {
00043
00044
00045 public:
00046
00047     /* Public Member Functions: */
00048
00049     /** The default constructor.
00050      *
00051      */
00052     XCompositePump() {
00053         /* Do nothing. */
00054     }
00055
00056     /** The destructor.
00057      *
00058      */
00059     ~XCompositePump() {
00060         /* Do nothing. */
00061     }
00062
00063     /** Generate a listing entry string from this composite pump data object.
00064      *
00065      *  @param[in]  pg      The pressure sensor sensitivity scale factor, in LSB/dBar.
00066      *  @param[in]  po      The pressure sensor offset, in dBar.
00067      *  @param[in]  pc      The pressure correction, in dBar, to be subtracted computed pressure
00068      *                      value.
00069      *
00070      *  @throw      log::MessageException if an error occurs while generating the listing string.
00071      *
00072      *  @return     A listing entry string representation of this composite pump data object.
00073      */
00074     const string GenerateListingString(uint16_t pg, uint16_t po, float pc) const {
00075
00076         ostringstream os;
00077
00078         try {
00079
00080             os << XPump::GenerateListingComment();
00081
00082             vector<shared_ptr<XPump> >::const_iterator it;
00083
00084             for(it = elements().begin(); it < elements().end(); it++)
00085
00086                 if(*it) {
00087
00088                     if(it > elements().begin()) {
00089
00090                         os << endl;
00091                         XMSG_SECTION_BREAK();
00092                         os << XPump::GenerateListingComment();
00093
00094                     }
00095
00096                     os << endl << (*it)->GenerateListingString(pg, po, pc);
00097
00098                 }
00099
00100         }CATCH_BLOCK("Failed to generate listing string for composite pump data object!")
00101
00102         return os.str();
00103     }
```

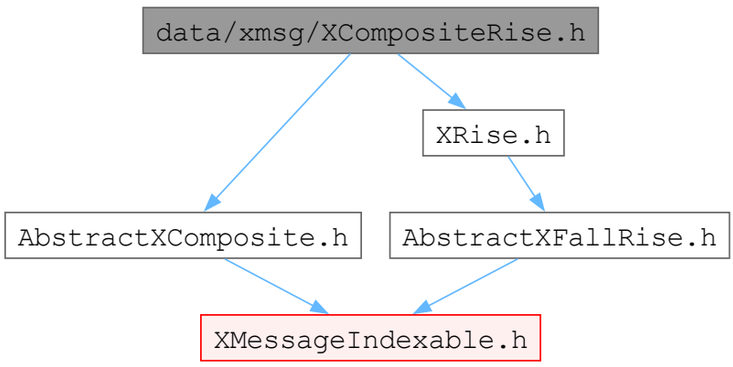
```
00104 };
00105
00106 } /* namespace xmsg */
00107 } /* namespace util */
00108 #endif /* XCOMPOSITEPUMP_H_ */
```

11.90 data/xmsg/XCompositeRise.h File Reference

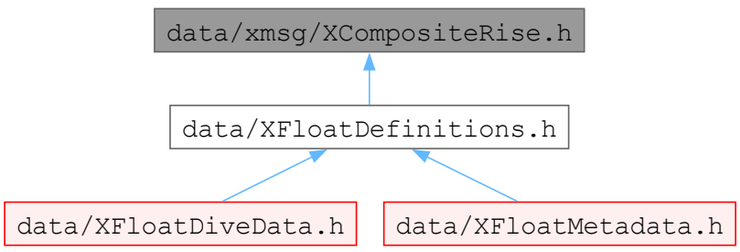
The util::xmsg::XCompositeRise class header file.

```
#include "AbstractXComposite.h"
#include "XRise.h"
```

Include dependency graph for XCompositeRise.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XCompositeRise

  Class representing the aggregate pump data for a single dive, composed from one or more pump data messages.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.90.1 Detailed Description

The util::xmsg::XCompositeRise class header file.

Version

0.6

Date

Jul 24, 2014 2:16:15 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.91 XCompositeRise.h

Go to the documentation of this file.

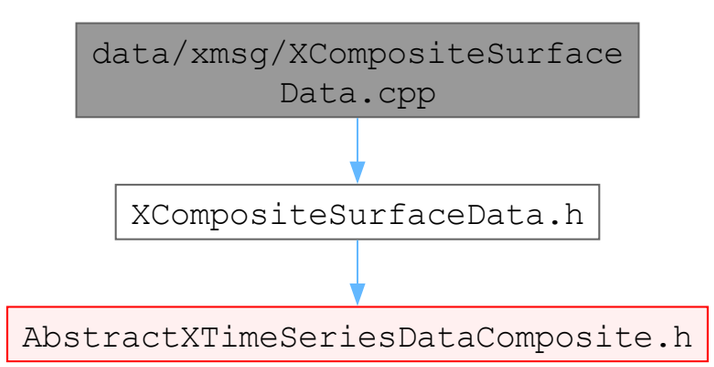
```
00001 /** The @ref util::xmsg::XCompositeRise class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jul 24, 2014 2:16:15 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef XCOMPOSITERISE_H_
00024 #define XCOMPOSITERISE_H_
00025
00026 /* Includes */
00027
00028 // local
00029 #include "AbstractXComposite.h"
00030 #include "XRise.h"
00031
00032 /* Definitions */
00033
00034 namespace util {
00035 namespace xmsg {
00036
00037 /* Type Definitions */
00038
00039 /* Class Definitions */
00040
00041 /** Class representing the aggregate pump data for a single dive, composed from one or more pump
00042  *  data messages.
00043  *
00044  */
00045 class XCompositeRise : public AbstractXComposite<shared_ptr<XRise> > {
00046
00047
00048 public:
00049
00050     /* Member Functions */
00051
00052     /** The default constructor.
00053      *
00054      */
00055     XCompositeRise() {
00056         /* Do nothing. */
```

```
00057     }
00058
00059     /** The destructor.
00060      *
00061      */
00062     ~XCompositeRise() {
00063         /* Do nothing. */
00064     }
00065
00066     /** Generate a listing entry string from this composite rise time-series data object.
00067      *
00068      *  @param[in]  pg      The pressure sensor sensitivity scale factor, in LSB/dBar.
00069      *  @param[in]  po      The pressure sensor offset, in dBar.
00070      *  @param[in]  sg      The salinity sensor sensitivity scale factor, in LSB/PSU.
00071      *  @param[in]  so      The salinity sensor offset, in PSU.
00072      *  @param[in]  tg      The temperature sensor sensitivity scale factor, in LSB/degC.
00073      *  @param[in]  to      The temperature sensor offset, in degC.
00074      *  @param[in]  pc      The pressure correction, in dBar, to be subtracted computed pressure
00075      *                      value.
00076      *
00077      *  @throw      log::MessageException if an error occurs while generating the listing string.
00078      *
00079      *  @return     A listing entry string representation of this composite rise time-series data
00080      *              object.
00081      */
00082     const string GenerateListingString(
00083             uint16_t pg,
00084             uint16_t po,
00085             uint16_t sg,
00086             uint16_t so,
00087             uint16_t tg,
00088             uint16_t to,
00089             float pc) const  {
00090         ostringstream os;
00091
00092         try {
00093
00094             os « AbstractXFallRise::GenerateListingComment();
00095             for(auto it = elements().begin(); it < elements().end(); it++)
00096                 if(*it) {
00097                     if(it > elements().begin()) {
00098                         os « endl;
00099                         XMSG_SECTION_BREAK();
00100                         os « XRise::GenerateListingComment();
00101                     }
00102                     os « endl « (*it)->GenerateListingString(pg, po, sg, so, tg, to, pc);
00103                 }
00104
00105         }CATCH_BLOCK("Failed to generate listing string for composite fall data object!")
00106
00107         return os.str();
00108     }
00109
00110 };
00111
00112 } /* namespace xmsg */
00113 } /* namespace util */
00114 #endif /* XCOMPOSITERISE_H_ */
```

11.92 data/xmsg/XCompositeSurfaceData.cpp File Reference

The util::xmsg::XCompositeSurfaceData class source file.

`#include "XCompositeSurfaceData.h"`
Include dependency graph for XCompositeSurfaceData.cpp:



Namespaces

- namespace util

The utility namespace.

- namespace util::xmsg

  The X-Message utility namespace.

11.92.1 Detailed Description

The util::xmsg::XCompositeSurfaceData class source file.

Version

0.6

Date

Jun 22, 2016 9:15:25 AM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

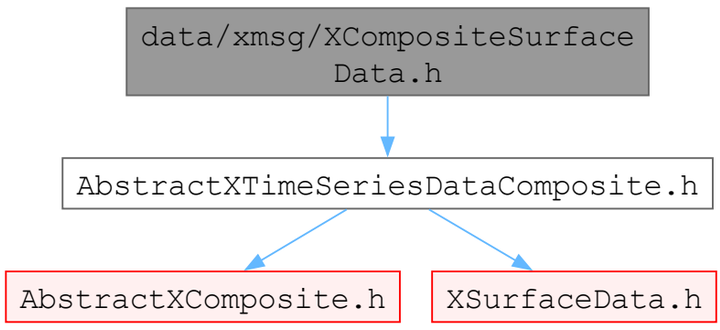
This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.93 data/xmsg/XCompositeSurfaceData.h File Reference

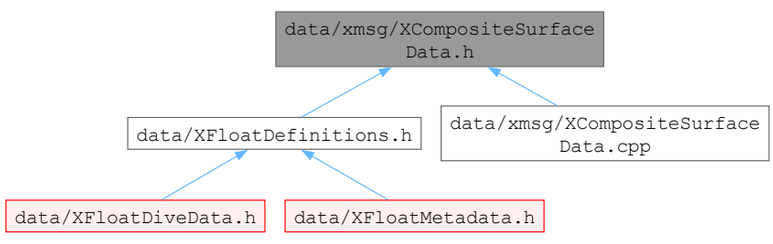
The util::xmsg::XCompositeSurfaceData class header file.

```
#include "AbstractXTimeSeriesDataComposite.h"
```

Include dependency graph for XCompositeSurfaceData.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XCompositeSurfaceData

  Class representing the aggregate surface pressure data for a single dive, composed from one or more surface pressure messages.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.93.1 Detailed Description

The util::xmsg::XCompositeSurfaceData class header file.

Version

0.6

Date

Jul 27, 2014 3:18:55 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.94 XCompositeSurfaceData.h

Go to the documentation of this file.

```
00001 /** The @ref util::xmsg::XCompositeSurfaceData class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jul 27, 2014 3:18:55 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef XCOMPOSITESURFACEPRESSURE_H_
00024 #define XCOMPOSITESURFACEPRESSURE_H_
00025
00026 /* Includes: */
00027
00028 // local
00029 #include "AbstractXTimeSeriesDataComposite.h"
00030
00031 /* Definitions */
```

```
00032
00033
00034
00035 namespace util {
00036 namespace xmsg {
00037
00038 /* Class Definition: */
00039
00040 /** Class representing the aggregate surface pressure data for a single dive, composed from one or
00041  *  more surface pressure messages.
00042  *
00043  */
00044 class XCompositeSurfaceData:
00045         public AbstractXTimeSeriesDataComposite<XSurfaceData> {
00046
00047 private:
00048
00049     /* Private Data Members: */
00050
00051     int pres_num_samples_;
00052
00053     DerivedSample_t pres_var_;
00054
00055 protected:
00056
00057     /* Protected Member Functions: */
00058
00059     void set_pres_var(DerivedSample_t pres_var) {
00060
00061         // Get an exclusive lock.
00062         //lock_guard<recursive_mutex> guard(mutex_);
00063
00064         pres_var_ = pres_var;
00065     }
00066
00067     void set_pres_num_samples(int pres_num_samples) {
00068
00069         // Get an exclusive lock.
00070         //lock_guard<recursive_mutex> guard(mutex_);
00071
00072         pres_num_samples_ = pres_num_samples;
00073     }
00074
00075 public:
00076
00077     /* Member Functions: */
00078
00079     /** Default constructor.
00080      *
00081      */
00082     XCompositeSurfaceData() :
00083                     AbstractXTimeSeriesDataComposite<XSurfaceData>(),
00084                     pres_num_samples_(-1),
00085                     pres_var_(XMSG_SENS_BAD_PRES) {
00086
00087         /* Do nothing. */
00088
00089     }
00090
00091     /** Destructor.
00092      *
00093      */
00094     ~XCompositeSurfaceData() {
00095
00096         /* Do nothing. */
00097
00098     }
00099
00100     /** Aggregate the surface time series and compute outlier-filtered pressure statistics.
00101      *
00102      *  First aggregates the messages as in
00103      *  @ref AbstractXTimeSeriesDataComposite::AggregateSensorData(), then refines the surface
00104      *  pressure statistics in three passes over the valid samples (after the static gain):
00105      *   -# compute the pressure variance about the mean;
00106      *   -# discard samples more than two standard deviations from the mean, and recompute the
00107      *      maximum, minimum and mean from the rest;
00108      *   -# recompute the variance from the remaining samples.
00109      *
00110      *  These statistics drive the surface pressure offset correction (see
00111      *  @c XFloatDiveData::UpdateDerivedSurfaceDataFields()).
00112      *
00113      *  @throw      util::log::MessageException if a pass finds no valid samples.
00114      */
00115     void AggregateSensorData() {
00116
00117         Sample_t pres;
00118         Sample_t pres_std_dev = 0;
00119         int64_t pres_sum = 0;
00120         int64_t pres_var_sum = 0;
00121         size_t valid_sample_count = 0;
00122
00123 #       if defined(XMSG_DEBUG_SURF_PRES_AGGR_DATA) && XMSG_DEBUG_SURF_PRES_AGGR_DATA
00124         ostringstream os;
00125 #       endif // defined(XMSG_DEBUG_SURF_PRES_AGGR_DATA) && XMSG_DEBUG_SURF_PRES_AGGR_DATA
00126
00127         try {
00128
00129             // Update aggregated data status.
00130             set_aggregated_data_valid(false);
00131
00132 #           if defined(XMSG_DEBUG_SURF_PRES_AGGR_DATA) && XMSG_DEBUG_SURF_PRES_AGGR_DATA
00133             {
00134                 os « "Aggregating sensor data..." « endl;
00135             }
00136 #           endif // defined(XMSG_DEBUG_SURF_PRES_AGGR_DATA) && XMSG_DEBUG_SURF_PRES_AGGR_DATA
00137
00138             AbstractXTimeSeriesDataComposite<XSurfaceData>::AggregateSensorData();
00139
```

```
00140             /* First Iteration: Compute surface pressure variance.
00141              *
00142              */
00143
00144 #           if defined(XMSG_DEBUG_SURF_PRES_AGGR_DATA) && XMSG_DEBUG_SURF_PRES_AGGR_DATA
00145             {
00146                 os « "First iteration:" « endl;
00147             }
00148 #           endif // defined(XMSG_DEBUG_SURF_PRES_AGGR_DATA) && XMSG_DEBUG_SURF_PRES_AGGR_DATA
00149
00150             for(auto it = aggregated_pres_data().begin(); it < aggregated_pres_data().end(); ++it) {
00151
00152                 pres = *it;
00153
00154                 // Discard null samples.
00155                 switch(pres) {
00156                 case XMSG_SENS_BAD:
00157                 case UINT16_MAX:
00158 #                   if defined(XMSG_DEBUG_SURF_PRES_AGGR_DATA) && XMSG_DEBUG_SURF_PRES_AGGR_DATA
00159                     {
00160                         lwrn() « "Questionable surface pressure value: " « pres;
00161                     }
00162 #                   endif // defined(XMSG_DEBUG_SURF_PRES_AGGR_DATA) && XMSG_DEBUG_SURF_PRES_AGGR_DATA
00163                     continue;
00164
00165                 default:
00166                     break;
00167                 }
00168
00169                 // Apply static gain.
00170                 pres «= derived_fields_static_gain_exp();
00171
00172                 valid_sample_count++;
00173                 pres_var_sum += std::pow(pres - pres_mean(), 2);
00174
00175 #               if defined(XMSG_DEBUG_SURF_PRES_AGGR_DATA) && XMSG_DEBUG_SURF_PRES_AGGR_DATA
00176                 {
00177                     static const format fmt("  count = %3d pres = %15d    pres_var_sum = %15d");
00178                     os « format(fmt) % valid_sample_count % pres % pres_var_sum « endl;
00179                 }
00180 #               endif // defined(XMSG_DEBUG_SURF_PRES_AGGR_DATA) && XMSG_DEBUG_SURF_PRES_AGGR_DATA
00181             }
00182
00183 #           if defined(XMSG_DEBUG_SURF_PRES_AGGR_DATA) && XMSG_DEBUG_SURF_PRES_AGGR_DATA
00184             {
00185                 ldbg() « os.str();
00186                 os.str("");
00187                 os.clear();
00188             }
00189 #           endif // defined(XMSG_DEBUG_SURF_PRES_AGGR_DATA) && XMSG_DEBUG_SURF_PRES_AGGR_DATA
00190
00191             if(!valid_sample_count) {
00192
00193                 EXC_THROW("Zero valid samples on first iteration!");
00194             }
00195
00196             // Compute variance and std. dev.
00197             set_pres_var(pres_var_sum / valid_sample_count);
00198             pres_std_dev = sqrt(pres_var());
00199
00200             /* Second Iteration: Recompute derived fields after discarding outliers.
00201              *
00202              */
00203
00204             valid_sample_count = 0;
00205             set_pres_max(XMSG_SENS_BAD);
00206             set_pres_min(XMSG_SENS_BAD);
00207
00208 #           if defined(XMSG_DEBUG_SURF_PRES_AGGR_DATA) && XMSG_DEBUG_SURF_PRES_AGGR_DATA
00209             {
00210                 os « "Second iteration:" « endl;
00211             }
00212 #           endif // defined(XMSG_DEBUG_SURF_PRES_AGGR_DATA) && XMSG_DEBUG_SURF_PRES_AGGR_DATA
00213
00214             for(auto it = aggregated_pres_data().begin(); it < aggregated_pres_data().end(); ++it) {
00215
00216                 pres = *it;
00217
00218                 // Discard null samples.
00219                 switch(pres) {
00220                 case XMSG_SENS_BAD:
00221                 case UINT16_MAX:
00222 #                   if defined(XMSG_DEBUG_SURF_PRES_AGGR_DATA) && XMSG_DEBUG_SURF_PRES_AGGR_DATA
00223                     {
00224                         lwrn() « "Questionable surface pressure value: " « pres;
00225                     }
00226 #                   endif // defined(XMSG_DEBUG_SURF_PRES_AGGR_DATA) && XMSG_DEBUG_SURF_PRES_AGGR_DATA
00227                     continue;
00228
00229                 default:
00230                     break;
00231                 }
00232
00233                 // Apply static gain.
00234                 pres «= derived_fields_static_gain_exp();
00235
00236                 if(std::abs(pres - pres_mean()) > 2 * pres_std_dev)
00237                     continue;
00238
00239                 valid_sample_count++;
00240
00241                 // Max.
00242                 if(pres_max() < pres || pres_max() == XMSG_SENS_BAD)
00243                     set_pres_max(pres);
00244
00245                 // Sum.
00246                 pres_sum += pres;
00247
```

```
00248                 // Min.
00249                 if(pres_min() > pres || pres_min() == XMSG_SENS_BAD)
00250                     set_pres_min(pres);
00251 
00252 #               if defined(XMSG_DEBUG_SURF_PRES_AGGR_DATA) && XMSG_DEBUG_SURF_PRES_AGGR_DATA
00253                 {
00254                     static const format fmt("  count = %3d pres = %15d    pres_sum = %15d");
00255                     os « format(fmt) % valid_sample_count % pres % pres_sum « endl;
00256                 }
00257 #               endif // defined(XMSG_DEBUG_SURF_PRES_AGGR_DATA) && XMSG_DEBUG_SURF_PRES_AGGR_DATA
00258 
00259             }
00260 
00261             // Update mean pressure.
00262             set_pres_mean(pres_sum/valid_sample_count);
00263 
00264 #           if defined(XMSG_DEBUG_SURF_PRES_AGGR_DATA) && XMSG_DEBUG_SURF_PRES_AGGR_DATA
00265             {
00266                 ldbg() « os.str();
00267                 os.str("");
00268                 os.clear();
00269             }
00270 #           endif // defined(XMSG_DEBUG_SURF_PRES_AGGR_DATA) && XMSG_DEBUG_SURF_PRES_AGGR_DATA
00271 
00272             if(!valid_sample_count) {
00273                 EXC_THROW("Zero valid samples on second iteration!");
00274             }
00275 
00276             /* Third Iteration: Recompute variance after discarding outliers.
00277              *
00278              */
00279 
00280             pres_var_sum = 0;
00281             valid_sample_count = 0;
00282 
00283 #           if defined(XMSG_DEBUG_SURF_PRES_AGGR_DATA) && XMSG_DEBUG_SURF_PRES_AGGR_DATA
00284             {
00285                 os « "Third iteration:" « endl;
00286             }
00287 #           endif // defined(XMSG_DEBUG_SURF_PRES_AGGR_DATA) && XMSG_DEBUG_SURF_PRES_AGGR_DATA
00288 
00289 
00290             for(auto it = aggregated_pres_data().begin(); it < aggregated_pres_data().end(); ++it) {
00291 
00292                 pres = *it;
00293 
00294                 // Discard null samples.
00295                 switch(pres) {
00296                 case XMSG_SENS_BAD:
00297                 case UINT16_MAX:
00298 #                   if defined(XMSG_DEBUG_SURF_PRES_AGGR_DATA) && XMSG_DEBUG_SURF_PRES_AGGR_DATA
00299                     {
00300                         lwrn() « "Questionable surface pressure value: " « pres;
00301                     }
00302 #                   endif // defined(XMSG_DEBUG_SURF_PRES_AGGR_DATA) && XMSG_DEBUG_SURF_PRES_AGGR_DATA
00303                     continue;
00304 
00305                 default:
00306                     break;
00307                 }
00308 
00309                 // Apply static gain.
00310                 pres «= derived_fields_static_gain_exp();
00311 
00312                 if(std::abs(pres - pres_mean()) > 2 * pres_std_dev)
00313                     continue;
00314 
00315                 valid_sample_count++;
00316                 pres_var_sum += std::pow(pres - pres_mean(), 2);
00317 
00318 
00319 #               if defined(XMSG_DEBUG_SURF_PRES_AGGR_DATA) && XMSG_DEBUG_SURF_PRES_AGGR_DATA
00320                 {
00321                     static const format fmt("  count = %3d pres = %15d    pres_var_sum = %15d");
00322                     os « format(fmt) % valid_sample_count % pres % pres_var_sum « endl;
00323                 }
00324 #               endif // defined(XMSG_DEBUG_SURF_PRES_AGGR_DATA) && XMSG_DEBUG_SURF_PRES_AGGR_DATA
00325 
00326             }
00327 
00328 #           if defined(XMSG_DEBUG_SURF_PRES_AGGR_DATA) && XMSG_DEBUG_SURF_PRES_AGGR_DATA
00329             {
00330                 ldbg() « os.str();
00331             }
00332 #           endif // defined(XMSG_DEBUG_SURF_PRES_AGGR_DATA) && XMSG_DEBUG_SURF_PRES_AGGR_DATA
00333 
00334             if(!valid_sample_count) {
00335 
00336                 EXC_THROW("Zero valid samples on third iteration!");
00337             }
00338 
00339             // Recompute mean and variance.
00340             set_pres_num_samples(valid_sample_count);
00341             set_pres_var(pres_var_sum / valid_sample_count);
00342 
00343             // Update aggregated data status.
00344             set_aggregated_data_valid(true);
00345 
00346 #           if defined(XMSG_DEBUG_SURF_PRES_AGGR_DATA) && XMSG_DEBUG_SURF_PRES_AGGR_DATA
00347             {
00348                 ldbg() « os.str();
00349                 os.str("");
00350                 os.clear();
00351             }
00352 #           endif // defined(XMSG_DEBUG_SURF_PRES_AGGR_DATA) && XMSG_DEBUG_SURF_PRES_AGGR_DATA
00353 
00354         }CATCH_BLOCK("Failed to aggregate surface data!")
00355 
```

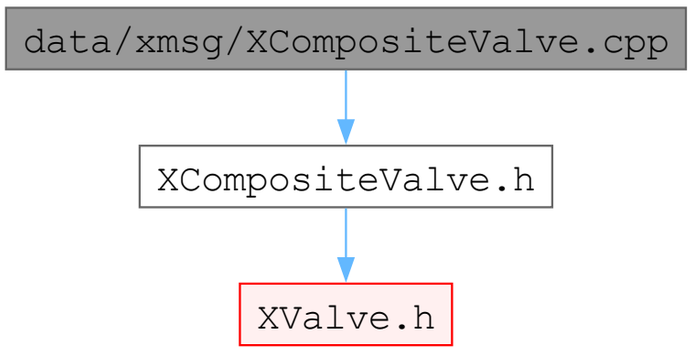
```
00356     }
00357
00358     /** Generate a listing entry string from this composite surface pressure time-series data
00359      *  object.
00360      *
00361      *  @param[in]  pg      The pressure sensor sensitivity scale factor, in LSB/dBar.
00362      *  @param[in]  po      The pressure sensor offset, in dBar.
00363      *  @param[in]  sg      The salinity sensor sensitivity scale factor, in LSB/PSU.
00364      *  @param[in]  so      The salinity sensor offset, in PSU.
00365      *  @param[in]  tg      The temperature sensor sensitivity scale factor, in LSB/degC.
00366      *  @param[in]  to      The temperature sensor offset, in degC.
00367      *  @param[in]  pc      The pressure correction, in dBar, to be subtracted computed pressure
00368      *                      value.
00369      *
00370      *  @throw      log::MessageException if an error occurs while generating the listing string.
00371      *
00372      *  @return     A listing entry string representation of this composite surface pressure
00373      *              time-series data object.
00374      */
00375     const string GenerateListingString(
00376             uint16_t pg,
00377             uint16_t po,
00378             uint16_t sg,
00379             uint16_t so,
00380             uint16_t tg,
00381             uint16_t to,
00382             float pc) const {
00383
00384         MsgVectorIterConst_t it;
00385         ostringstream os;
00386
00387         try {
00388
00389             // Get an exclusive lock.
00390 //            //lock_guard<recursive_mutex> guard(mutex_);
00391
00392             XMSG_SECTION("Surface Time-Series Data");
00393             os « AbstractXFallRise::GenerateListingComment() « endl;
00394
00395             for_each(messages().begin(), messages().end(), [&](const MsgPtr_t& msg) {
00396
00397                 if(msg) os « msg->GenerateListingString(pg, po, sg, so, tg, to, pc) « endl;
00398
00399             });
00400
00401             os « GenerateDerivedFieldsListingString(pg, po, pc);
00402
00403         }CATCH_BLOCK("Failed to generate listing string for composite surface data object!")
00404
00405         return os.str();
00406     }
00407
00408     const string GenerateDerivedFieldsListingString(uint16_t pg, uint16_t po,
00409             float pres_correction) const {
00410
00411         ostringstream os;
00412
00413         int midx = 0;
00414
00415         try {
00416
00417             // Get an exclusive lock.
00418 //            //lock_guard<recursive_mutex> guard(mutex_);
00419
00420             XMSG_SECTION("Surface Time-Series Data - Derived Fields");
00421
00422             {
00423                 const format fmt("#   %4s %4s %4s "
00424                         "%4s %12s "
00425                         "%15s %15s %15s "
00426                         "%15s");
00427                 os « format(fmt) % "SN" % "dive" % "midx" //
00428                         % "nSam" % "presValid" //
00429                         % "presMin" % "presMean" % "presMax" //
00430                         % "presVar"
00431                    « endl;
00432             }
00433
00434             {
00435                 const format fmt("#   %4s %4s %4s "
00436                         "%4s %12s "
00437                         "%15s %15s %15s "
00438                         "%15s");
00439                 os « format(fmt) % "" % "" % "" //
00440                         % "" % "" //
00441                         % "(dBar)" % "(dBar)" % "(dBar)" //
00442                         % "(dBar^2)"
00443                    « endl;
00444             }
00445
00446             {
00447                 const format fmt("SFD %4d %4d %4d "
00448                         "%4d %12d "
00449                         "%15.3f %15.3f %15.3f "
00450                         "%15.3f");
00451                 os « format(fmt) //
00452                 % (int) messages().at(0)->id_instrument() % (int) messages().at(0)->id_dive()
00453                         % midx++ //
00454                         % pres_num_samples() % (int) is_aggregated_data_valid() //
00455                         % PresMinAsFloat(pg, po,
00456                                 pres_correction) % PresMeanAsFloat(pg, po,
00457                                         pres_correction) % PresMaxAsFloat(pg, po,
00458                                                 pres_correction) //
00459                         % PresVarAsFloat(pg);
00460             }
00461
00462         }CATCH_BLOCK("Failed to generate listing string for derived surface data!")
00463
```

```
00464         return os.str();
00465     }
00466
00467     DerivedSample_t pres_var() const {
00468
00469         // Get an exclusive lock.
00470         //lock_guard<recursive_mutex> guard(mutex_);
00471
00472         return pres_var_;
00473     }
00474
00475      int pres_num_samples() const {
00476
00477         // Get an exclusive lock.
00478         //lock_guard<recursive_mutex> guard(mutex_);
00479
00480         return pres_num_samples_;
00481     }
00482
00483     template<typename T = float> T PresVarAsFloat(uint16_t gain) const {
00484
00485         T val = XMSG_SENS_BAD_PRES;
00486
00487         // Get an exclusive lock.
00488         //lock_guard<recursive_mutex> guard(mutex_);
00489
00490         if(gain) {
00491
00492             val = ((T) pres_var() / (T) gain
00493                     / (T) ((int64_t) 1 « (derived_fields_static_gain_exp() * 2)));
00494
00495 //             if(val < XMSG_SENS_MIN_PRES || val > XMSG_SENS_MAX_PRES)
00496 //                 val = XMSG_SENS_BAD_PRES;
00497
00498         }
00499
00500         return val;
00501
00502     }
00503
00504 };
00505
00506 } /* namespace xmsg */
00507 } /* namespace util */
00508
00509 #endif /* XCOMPOSITESURFACEPRESSURE_H_ */
```

11.95 data/xmsg/XCompositeValve.cpp File Reference

The util::xmsg::XCompositeValve class source file.

`#include "XCompositeValve.h"`
Include dependency graph for XCompositeValve.cpp:



Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.95.1 Detailed Description

The util::xmsg::XCompositeValve class source file.

Version

0.6

Date

Mar 22, 2016 10:08:10 AM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

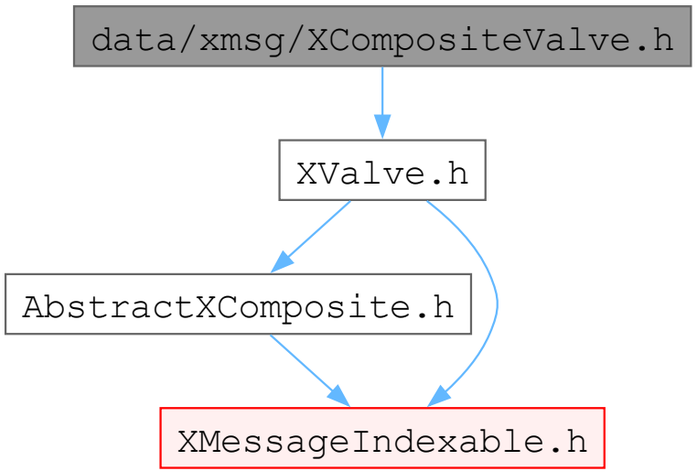
This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.96 data/xmsg/XCompositeValve.h File Reference

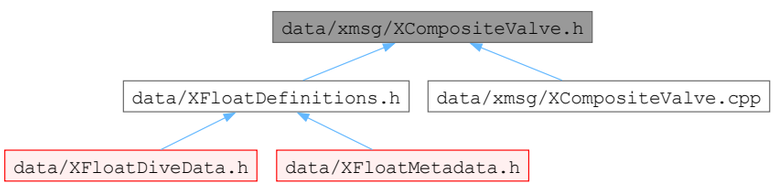
The util::xmsg::XCompositeValve class header file.

```
#include "XValve.h"
```

Include dependency graph for XCompositeValve.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XCompositeValve
  The composite valve data class.

Namespaces

- namespace util
  The utility namespace.
- namespace util::xmsg
  The X-Message utility namespace.

11.96.1 Detailed Description

The util::xmsg::XCompositeValve class header file.

Version

0.6

Date

Mar 22, 2016 10:08:10 AM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.97 XCompositeValve.h

Go to the documentation of this file.

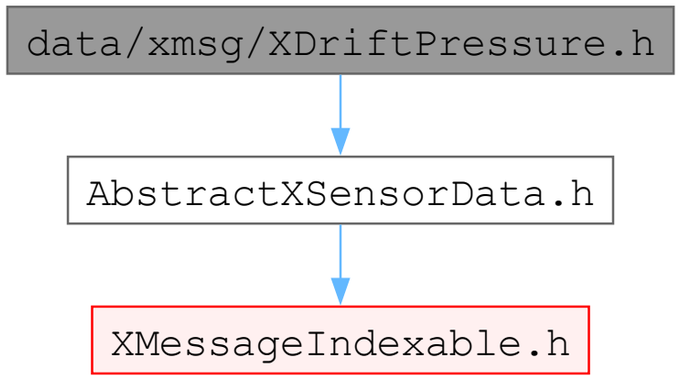
```
00001 /** The @ref util::xmsg::XCompositeValve class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Mar 22, 2016 10:08:10 AM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef XCOMPOSITEVALVE_H_
00024 #define XCOMPOSITEVALVE_H_
00025
00026 /* Includes: */
00027
00028 #include "XValve.h"
00029
00030 /* Definitions: */
00031
00032 namespace util {
00033 namespace xmsg {
```

```
00034
00035 /* Types: */
00036
00037 /** The composite valve data class.
00038  *
00039  *  An instance of this class aggregates the valve data messages (see @ref XValve) received for a
00040  *  single dive.
00041  */
00042 class XCompositeValve:
00043         public AbstractXComposite<shared_ptr<XValve> > {
00044
00045
00046 public:
00047
00048     /* Public Member Functions: */
00049
00050
00051     XCompositeValve() {
00052
00053         /* Do nothing. */
00054
00055     }
00056
00057      ~XCompositeValve() {
00058
00059          /* Do nothing. */
00060
00061      }
00062
00063      const string GenerateListingString(uint16_t pGain, uint16_t pOff, float pres_correction) const {
00064
00065          ostringstream os;
00066
00067          try {
00068
00069              os « XValve::GenerateListingComment();
00070
00071              vector<shared_ptr<XValve> >::const_iterator it;
00072              for(it = elements().begin(); it < elements().end(); it++)
00073
00074                  if(*it) {
00075
00076                      if(it > elements().begin()) {
00077
00078                          os « endl;
00079                          XMSG_SECTION_BREAK();
00080                          os « XValve::GenerateListingComment();
00081
00082                      }
00083
00084                      os « endl « (*it)->GenerateListingString(pGain, pOff, pres_correction);
00085
00086                  }
00087
00088          }CATCH_BLOCK("Failed to generate listing string for composite valve data object!")
00089
00090          return os.str();
00091
00092      }
00093
00094 };
00095
00096 /* Functions: */
00097
00098 } /* namespace xmsg */
00099 } /* namespace util */
00100
00101 #endif /* XCOMPOSITEVALVE_H_ */
```

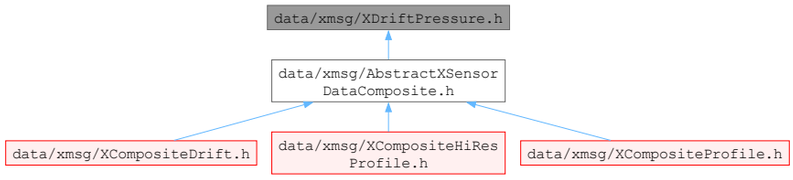
11.98 data/xmsg/XDriftPressure.h File Reference

The util::xmsg::XDriftPressure class source file.

```
#include "AbstractXSensorData.h"
```
Include dependency graph for XDriftPressure.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XDriftPressure

  Class representing a drift pressure time-series X-Message.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.98.1 Detailed Description

The util::xmsg::XDriftPressure class source file.

Version

0.6

Date

Jul 1, 2014 1:54:15 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.99 XDriftPressure.h

Go to the documentation of this file.

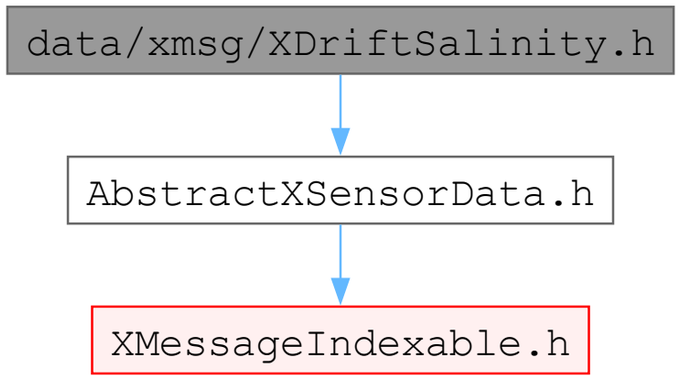
```
00001 /** The @ref util::xmsg::XDriftPressure class source file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jul 1, 2014 1:54:15 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef XDRIFTPRESSURE_H_
00024 #define XDRIFTPRESSURE_H_
00025
00026 /* Includes */
00027
00028 #include "AbstractXSensorData.h"
00029
00030 /* Definitions */
00031
00032 namespace util {
00033 namespace xmsg {
00034
00035 /* Class Definitions */
00036
00037 /** Class representing a drift pressure time-series X-Message.
00038  *
00039  */
00040 class XDriftPressure: public AbstractXSensorData {
00041
00042 public:
00043
00044     /* Member Functions */
00045
00046     /** Parameterized constructor.
00047      *
00048      *  @param[in]      dive        The dive or cycle number of this message.
00049      *  @param[in]      whoi        The unique WHOI identifier.
00050      *  @param[in]      data        The binary X-message data to decode.
00051      *
00052      *  @throw          util::log::MessageException if an decode error occurs.
00053      */
00054     XDriftPressure(int16_t dive, uint16_t whoi, const vector<uint8_t>& data) :
00055             AbstractXSensorData(dive, whoi, data) {
00056         DecodeXMsgData(data); // Decode message.
00057     }
00058
00059     /** Destructor.
00060      *
00061      */
00062     ~XDriftPressure() {
00063         /* Do nothing. */
00064     }
00065
00066     /** Decode the drift pressure X-Message from binary data.
00067      *
00068      *  @param[in]      data            The binary data to decode.
00069      */
00070     void DecodeXMsgData(const vector<uint8_t>& data) {
00071         try {
00072             const uint8_t* d = CheckXMsgData(data, (int) -1, (int) 5);
00073             if((d[0] & 0xF0) != (XMSG_TYPE_ID_PRES_DRIFT8 & 0xF0))
00074                 EXC_THROW("Invalid X-Message unique identifier!");
00075             UnpackSensorData(data);
00076         }CATCH_BLOCK("Failed to decode drift pressure X-Message!")
00077     }
00078 };
00079
00080 } /* namespace xmsg */
00081 } /* namespace util */
00082 #endif /* XDRIFTPRESSURE_H_ */
```

11.100 data/xmsg/XDriftSalinity.h File Reference

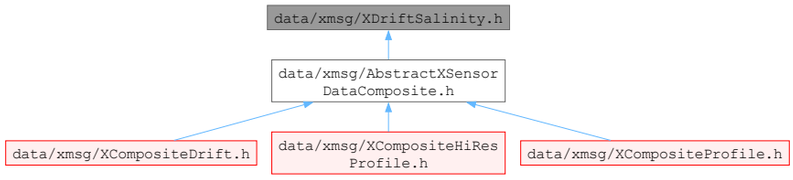
The util::xmsg::XDriftSalinity class source file.

```
#include "AbstractXSensorData.h"
```

Include dependency graph for XDriftSalinity.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XDriftSalinity

  Class representing a drift salinity time-series X-Message.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.100.1 Detailed Description

The util::xmsg::XDriftSalinity class source file.

Version

0.6

Date

Jul 1, 2014 1:55:25 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.101 XDriftSalinity.h

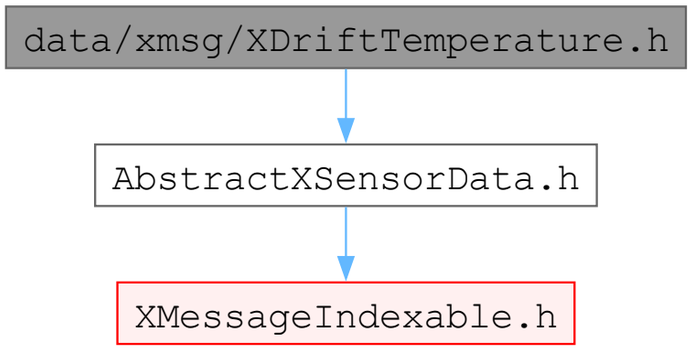
Go to the documentation of this file.

```
00001 /** The @ref util::xmsg::XDriftSalinity class source file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jul 1, 2014 1:55:25 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef XDRIFTSALINITY_H_
00024 #define XDRIFTSALINITY_H_
00025
00026 /* Includes */
00027
00028 #include "AbstractXSensorData.h"
00029
00030 /* Definitions */
00031
00032 namespace util {
00033 namespace xmsg {
00034
00035 /* Type Definitions */
00036
00037 /* Class Definitions */
00038
00039 /** Class representing a drift salinity time-series X-Message.
00040  *
00041  */
00042 class XDriftSalinity: public AbstractXSensorData {
00043 public:
00044
00045     /* Member Functions */
00046
00047     /** The parameterized constructor.
00048      *
00049      *  @param[in]      dive        The dive or cycle number of this message.
00050      *  @param[in]      whoi        The unique WHOI identifier.
00051      *  @param[in]      data        The binary X-message data to decode.
00052      *
00053      *  @throw          util::log::MessageException if an decode error occurs.
00054      */
00055     XDriftSalinity(int16_t dive, uint16_t whoi, const vector<uint8_t>& data) :
00056                     AbstractXSensorData(dive, whoi, data) {
00057         DecodeXMsgData(data); // Decode message.
00058     }
00059
00060     /** Destructor.
00061      *
00062      */
00063     ~XDriftSalinity() {
00064         /* Do nothing. */
00065     }
00066
00067     /** Decode the drift salinity X-Message from binary data.
00068      *
00069      *  @param[in]      data            The binary data to decode.
00070      */
00071     void DecodeXMsgData(const vector<uint8_t>& data) {
00072         try {
00073             const uint8_t* d = CheckXMsgData(data, (int) -1, (int) 5);
00074             if((d[0] & 0xF0) != (XMSG_TYPE_ID_SALN_DRIFT8 & 0xF0))
00075                 EXC_THROW("Invalid X-Message unique identifier!");
00076             UnpackSensorData(data);
00077         }CATCH_BLOCK("Failed to decode drift salinity X-Message!")
00078     }
00079
00080
00081 };
00082
00083 } /* namespace xmsg */
00084 } /* namespace util */
00085 #endif /* XDRIFTSALINITY_H_ */
```

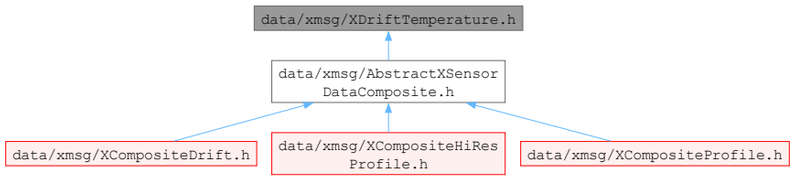
11.102 data/xmsg/XDriftTemperature.h File Reference

The util::xmsg::XDriftTemperature class source file.

```
#include "AbstractXSensorData.h"
```
Include dependency graph for XDriftTemperature.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XDriftTemperature

  Class reprenting a drift temperature X-Message.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.102.1 Detailed Description

The util::xmsg::XDriftTemperature class source file.

Version

0.6

Date

Jul 1, 2014 1:54:46 PM

Jun 12, 2014 9:23:09 AM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.103 XDriftTemperature.h

Go to the documentation of this file.

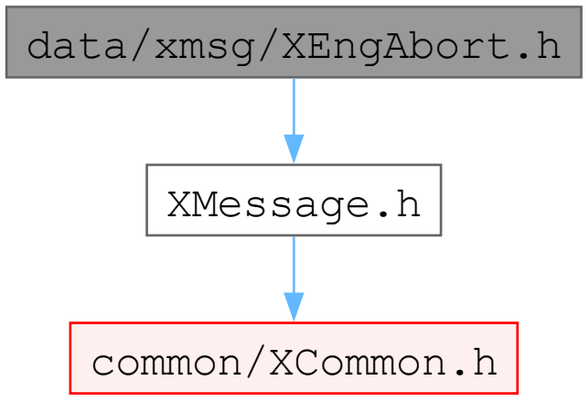
```
00001 /** The @ref util::xmsg::XDriftTemperature class source file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jul 1, 2014 1:54:46 PM
00008  *
00009  *  @date       Jun 12, 2014 9:23:09 AM
00010  *
00011  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00012  *              Engineer I,
00013  *              Department of Physical Oceanography,
00014  *              Woods Hole Oceanographic Institution
00015  *
00016  *  @copyright  2014 Woods Hole Oceanographic Institution.
00017  *              Released under the GNU General Public License.
00018  *
00019  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00020  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00021  *  General Public License for more details. You should have received a copy of the GNU General
00022  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00023  */
00024
00025 #ifndef XDRIFTTEMPERATURE_H_
00026 #define XDRIFTTEMPERATURE_H_
00027
00028 /* Includes */
00029
00030 #include "AbstractXSensorData.h"
00031
00032 /* Definitions */
00033
00034 namespace util {
00035 namespace xmsg {
00036
00037 /* Class Definitions */
00038
00039 /** Class reprenting a drift temperature X-Message.
00040  *
00041  */
00042 class XDriftTemperature:
00043         public AbstractXSensorData {
00044
00045 public:
00046
00047     /* Member Functions */
00048
00049     /** The parameterized constructor.
00050      *
00051      *  @param[in]      dive        The dive or cycle number of this message.
00052      *  @param[in]      whoi        The unique WHOI identifier.
00053      *  @param[in]      data        The binary X-message data to decode.
00054      *
00055      *  @throw          util::log::MessageException if an decode error occurs.
00056      */
00057     XDriftTemperature(int16_t dive, uint16_t whoi, const vector<uint8_t>& data) :
00058                     AbstractXSensorData(dive, whoi, data) {
00059         DecodeXMsgData(data); // Decode message.
00060     }
00061
00062     /** The destructor.
00063      *
00064      */
00065     ~XDriftTemperature() {
00066         /* Do nothing. */
00067     }
00068
00069     /** Decode the drift temperature X-Message from binary data.
00070      *
00071      *  @param[in]      data            The binary data to decode.
00072      */
00073     void DecodeXMsgData(const vector<uint8_t>& data) {
00074         try {
00075             const uint8_t* d = CheckXMsgData(data, (int) -1, (int) 5);
00076             if((d[0] & 0xF0) != (XMSG_TYPE_ID_TEMP_DRIFT8 & 0xF0))
00077                 EXC_THROW("Invalid X-Message unique identifier!");
00078             UnpackSensorData(data);
00079         }CATCH_BLOCK("Failed to decode drift temperature X-Message!")
00080     }
00081
00082 };
00083
00084 } /* namespace xmsg */
00085 } /* namespace util */
00086 #endif /* XDRIFTTEMPERATURE_H_ */
```

11.104 data/xmsg/XEngAbort.h File Reference

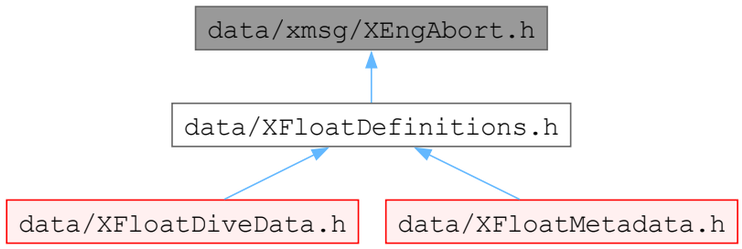
The util::xmsg::XEngAbort class header file.

```
#include "XMessage.h"
```

Include dependency graph for XEngAbort.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XEngAbort

  Class representing an abort/beacon mode engineering X-Message.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.104.1 Detailed Description

The util::xmsg::XEngAbort class header file.

Version

0.6

Date

Jul 1, 2014 2:01:30 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.105 XEngAbort.h

Go to the documentation of this file.

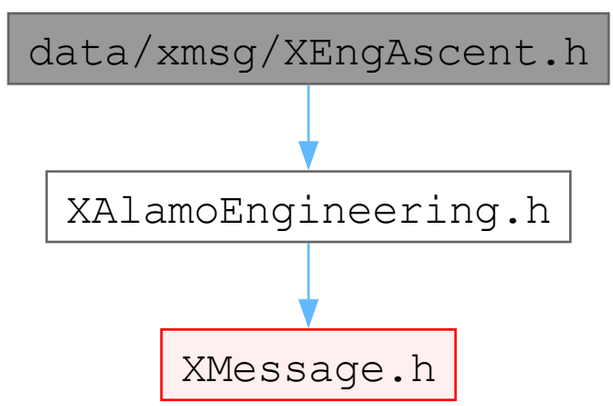
```
00001 /** The @ref util::xmsg::XEngAbort class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jul 1, 2014 2:01:30 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef XENGABORT_H_
00024 #define XENGABORT_H_
00025
00026 /* Includes */
00027
00028 #include "XMessage.h"
00029
00030 /* Definitions */
00031
00032 namespace util {
00033 namespace xmsg {
00034
00035 /* Class Definitions */
00036
00037 /** Class representing an abort/beacon mode engineering X-Message.
00038  *
00039  */
00040 class XEngAbort:
00041         public XMessage {
00042
00043 public:
00044
00045     /* Member Functions */
00046
00047     /** Parameterized constructor.
00048      *
00049      *  @param[in]      dive        The dive or cycle number of this message.
00050      *  @param[in]      whoi        The unique WHOI identifier.
00051      *  @param[in]      data        The binary X-message data to decode.
00052      *
00053      *  @throw          util::log::MessageException if an decode error occurs.
00054      */
00055     XEngAbort(int16_t dive, uint16_t whoi, const vector<uint8_t>& data) :
00056                     XMessage(dive, whoi, data) {
00057         DecodeXMsgData(data); // Decode message.
00058     }
00059
00060     /** Destructor.
00061      *
00062      */
00063     ~XEngAbort() {
00064         /* Do nothing. */
00065     }
00066
00067     /** Decode the abort/beacon mode engineering X-Message from the binary data.
00068      *
00069      *  @param[in]      data            The binary message to decode.
00070      */
00071     void DecodeXMsgData(const vector<uint8_t>& data) {
00072
00073         try {
00074
00075             // Check message.
00076             /* const uint8_t* d = */CheckXMsgData(data, XMSG_TYPE_ID_ENG_ABORT, -1);
00077
00078             /** @todo   Implement the functionality to decode the abort/beacon mode engineering
```

```
00079             *          X-Message from the binary data.
00080             */
00081
00082         }CATCH_BLOCK("Failed to decode abort/beacon mode engineering X-Message!")
00083     }
00084
00085     /** Generate a listing string representation of the abort/beacon mode engineering data message.
00086      *
00087      *  @return     A listing string representation of the abort/beacon mode engineering data
00088      *              message.
00089      */
00090     const string GenerateListingString() const {
00091         ostringstream os;
00092
00093         try {
00094
00095             /** @todo   Implement the functionality for generating a listing string for the Solo-II
00096              *          abort/beacon mode engineering data message.
00097              *
00098              */
00099
00100         }CATCH_BLOCK("Failed to !")
00101
00102         return os.str();
00103     }
00104
00105 };
00106
00107 } /* namespace xmsg */
00108 } /* namespace util */
00109 #endif /* XENGABORT_H_ */
```

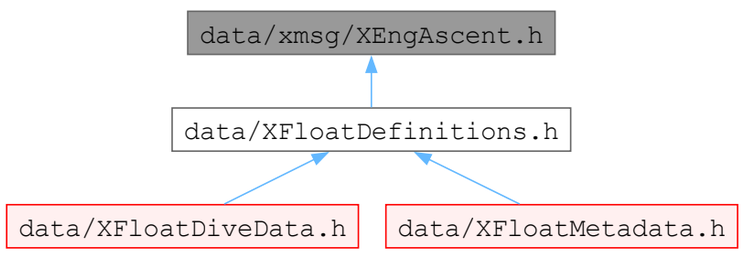
11.106 data/xmsg/XEngAscent.h File Reference

The util::xmsg::XEngAscent class header file.

`#include "XAlamoEngineering.h"`
Include dependency graph for XEngAscent.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XEngAscent

  Class representing an ascend state engineering X-Message.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.106.1 Detailed Description

The util::xmsg::XEngAscent class header file.

Version

0.6

Date

Jul 1, 2014 2:02:59 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.107 XEngAscent.h

Go to the documentation of this file.

```
00001 /** The @ref util::xmsg::XEngAscent class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jul 1, 2014 2:02:59 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023
00024 #ifndef XENGASCENT_H_
00025 #define XENGASCENT_H_
00026
00027 /* Includes */
00028
00029 // local
00030 #include "XAlamoEngineering.h"
00031
00032 namespace util {
00033 namespace xmsg {
```

```
00034
00035 /* Class Definitions */
00036
00037 /** Class representing an ascend state engineering X-Message.
00038  *
00039  *  @note           This X-Message is only used by ALAMO-type instruments.
00040  *  @note           This message type was added in X-Message version 2.1.
00041  */
00042 class XEngAscent:
00043         public XAlamoEngineering {
00044
00045 private:
00046
00047     /* Data Members */
00048
00049     /** The measured internal pres case vacuum, in 0.01 inHg, at the start of the
00050      *  ascent/profile phase of the dive cycle.
00051      *
00052      *  @note This field was added in version 0x11.
00053      *
00054      */
00055     uint16_t case_vac_at_start_cinhg_;
00056
00057     /** The measured internal pres case vacuum, in 0.01 inHg, at the end of the
00058      *  ascent/profile phase of the dive cycle.
00059      *
00060      *  @note This field was added in version 0x11.
00061      *
00062      */
00063     uint16_t case_vac_at_end_cinhg_;
00064
00065     uint16_t fall_start_pres_counts_;
00066
00067     ptime fall_start_time_;
00068
00069     /** The measured system current, in mA.
00070      *
00071      *  @note This field is only used for message version 0x10. It was removed in version 0x11.
00072      */
00073     uint16_t system_current_ma_;
00074
00075     /** The cumulative pump run time during the ascent phase of the dive cycle, in seconds.
00076      *
00077      *  @note This field is only used for message version 0x10. It was removed in version 0x11.
00078      */
00079     uint16_t pump_run_duration_;
00080
00081 public:
00082
00083     /* Member Functions */
00084
00085     /** Parameterized constructor.
00086      *
00087      *  @param[in]      dive        The dive or cycle number of this message.
00088      *  @param[in]      whoi        The unique WHOI identifier.
00089      *  @param[in]      data        The binary X-message data to decode.
00090      *
00091      *  @throw          util::log::MessageException if an decode error occurs.
00092      */
00093     XEngAscent(int16_t dive, uint16_t whoi, const vector<uint8_t>& data) :
00094                     XAlamoEngineering(dive, whoi, data) {
00095         DecodeXMsgData(data);    // Decode message.
00096     }
00097
00098     /** Destructor.
00099      *
00100      */
00101     ~XEngAscent() {
00102         /* Do nothing. */
00103     }
00104
00105     /** Decode the ascent state engineering X-Message from the binary data.
00106      *
00107      *  @param[in]      data            The binary message to decode.
00108      */
00109     void DecodeXMsgData(const vector<uint8_t>& data) {
00110         try {
00111             // Check message.
00112             const uint8_t* d = CheckXMsgData(data, 0xE9, 13);
00113
00114             // Decode message.
00115             set_engineering_version(d[3]);
00116             switch(engineering_version()) {
00117
00118             case 0x10: // Engineering (Ascent) version 0x10.
00119
00120                 if(data.size() != 13) {
00121                     ostringstream os;
00122                     os « "Invalid X-Message: Expected 13 bytes, but received "
00123                        « data.size()
00124                        « " bytes!";
00125                     EXC_THROW(os.str());
00126                 }
00127
00128                 set_start_time(s2000(d, 4));
00129                 set_system_current_ma(u16(d, 8));
00130                 set_pump_run_duration(u16(d, 10));
00131
00132                 break;
00133
00134             case 0x11: // Engineering (Ascent) version 0x10.
00135
00136                 if(data.size() != 13) {
00137                     ostringstream os;
00138                     os « "Invalid X-Message: Expected 13 bytes, but received "
00139                        « data.size()
00140                        « " bytes!";
00141                     EXC_THROW(os.str());
```

```
00142                 }
00143
00144                 set_start_time(s2000(d, 4));
00145                 set_case_vac_at_start_cinhg(u16(d, 8));
00146                 set_case_vac_at_end_cinhg(u16(d, 10));
00147
00148                 break;
00149
00150             case 0x12: // Engineering (Ascent) version 0x12.
00151
00152                 if(data.size() != 19) {
00153                     ostringstream os;
00154                     os « "Invalid X-Message: Expected 19 bytes, but received "
00155                        « data.size()
00156                        « " bytes!";
00157                     EXC_THROW(os.str());
00158                 }
00159
00160                 set_start_time(s2000(d, 4));
00161                 set_case_vac_at_start_cinhg(u16(d, 8));
00162                 set_case_vac_at_end_cinhg(u16(d, 10));
00163                 set_fall_start_time(s2000(d, 12));
00164                 set_fall_start_pres_counts(u16(d, 16));
00165
00166                 break;
00167
00168             default: { // Unsupported Engineering (Ascent) version.
00169
00170                 ostringstream os;
00171                 os « "Unsupported ALAMO Engineering (Ascent) message version: 0x"
00172                    « hex
00173                    « (int) engineering_version()
00174                    « "!";
00175                 EXC_THROW(os.str());
00176             }
00177
00178                 break;
00179             }
00180         }CATCH_BLOCK("Failed to ALAMO Engineering (Ascent) X-Message!")
00181     }
00182
00183     /** Get the listing entry string for this ascent engineering data X-Message
00184      *
00185      *  @param[in]  pg      The pressure sensor sensitivity scale factor, in LSB/dBar.
00186      *  @param[in]  po      The pressure sensor offset, in dBar.
00187      *  @param[in]  sg      The salinity sensor sensitivity scale factor, in LSB/PSU.
00188      *  @param[in]  so      The salinity sensor offset, in PSU.
00189      *  @param[in]  tg      The temperature sensor sensitivity scale factor, in LSB/degC.
00190      *  @param[in]  to      The temperature sensor offset, in degC.
00191      *
00192      *  @return         The listing entry string for this ascent engineering data X-Message
00193      */
00194     const string GenerateListingString(
00195         uint16_t pg,
00196         uint16_t po,
00197         uint16_t sg,
00198         uint16_t so,
00199         uint16_t tg,
00200         uint16_t to) const {
00201
00202       (void) sg;
00203       (void) so;
00204       (void) tg;
00205       (void) to;
00206
00207       ostringstream os;
00208
00209       switch(engineering_version()) {
00210
00211         case 0x10: { // Engineering (Ascent) version 0x10.
00212
00213           // Minor index 0.
00214           static const format fmt1("#   %5s %4s %4s %6s %20s %9s %9s");
00215           static const format fmt2("#   %5s %4s %4s %6s %20s %9s %9s");
00216           static const format fmt3("E09 %5d %4d %4d   0x%-02X %20s %9.3f %9d");
00217           os « format(fmt1) % "SN" % "dive" % "midx" % "engVer" % "start-of-ascent" % "sysI"
00218                 % "pumpT"
00219              « endl;
00220           os « format(fmt2) % "" % "" % "" % "" % "" % "(A)" % "(sec)" « endl;
00221           os « format(fmt3) % id_instrument() % id_dive() % 0 % (int) engineering_version()
00222               % GenerateIso8601String(start_time()) % system_current_a() % pump_run_duration();
00223         }
00224           break;
00225
00226         case 0x11: { // Engineering (Ascent) version 0x11.
00227
00228           // Minor index 0.
00229           static const format fmt1("#   %5s %4s %4s %6s %20s %9s %9s");
00230           static const format fmt2("#   %5s %4s %4s %6s %20s %9s %9s");
00231           static const format fmt3("E09 %5d %4d %4d   0x%-02X %20s %9.2f %9.2f");
00232           os « format(fmt1) % "SN" % "dive" % "midx" % "engVer" % "start-of-ascent" % "vac0"
00233                 % "vac1"
00234              « endl;
00235           os « format(fmt2) % "" % "" % "" % "" % "" % "(inHg)" % "(inHg)" « endl;
00236           os « format(fmt3) % id_instrument() % id_dive() % 0 % (int) engineering_version()
00237               % GenerateIso8601String(start_time()) % case_vac_at_start_inhg()
00238               % case_vac_at_end_inhg();
00239         }
00240           break;
00241
00242         case 0x12: { // TODO Engineering (Ascent) version 0x12.
00243
00244           { // Minor index 0.
00245             static const format fmt1("#   %5s %4s %4s %6s %20s %9s %9s");
00246             static const format fmt2("#   %5s %4s %4s %6s %20s %9s %9s");
00247             static const format fmt3("E09 %5d %4d %4d   0x%-02X %20s %9.2f %9.2f");
00248             os « format(fmt1) % "SN" % "dive" % "midx" % "engVer" % "start-of-ascent" % "vac0"
00249                   % "vac1"
```

```
00250                 « endl;
00251             os « format(fmt2) % "" % "" % "" % "" % "" % "(inHg)" % "(inHg)" « endl;
00252             os « format(fmt3) % id_instrument() % id_dive() % 0 % (int) engineering_version()
00253                 % GenerateIso8601String(start_time()) % case_vac_at_start_inhg()
00254                 % case_vac_at_end_inhg() « endl;
00255           }
00256           { // Minor index 1.
00257             static const format fmt1("#   %5s %4s %4s %20s %9s");
00258             static const format fmt2("#   %5s %4s %4s %20s %9.2f");
00259             static const format fmt3("E09 %5d %4d %4d %20s %9s");
00260             os « format(fmt1) % "SN" % "dive" % "midx" % "start-of-fall" % "fallP" « endl;
00261             os « format(fmt2) % "" % "" % "" % "" % "(dBar)" « endl;
00262             os « format(fmt3) % id_instrument() % id_dive() % 1
00263                 % GenerateIso8601String(fall_start_time()) % fall_start_pres_dbar(pg, po);
00264           }
00265
00266         }
00267           break;
00268
00269         default: { // Unsupported Engineering (Ascent) version.
00270           ostringstream os;
00271           os « "Unsupported ALAMO Engineering (Ascent) message version: 0x"
00272              « hex
00273              « (int) engineering_version()
00274              « "!";
00275           EXC_THROW(os.str());
00276         }
00277           break;
00278       }
00279
00280       return os.str();
00281     }
00282
00283 public:
00284
00285     /* Member Functions - Accessor Methods: */
00286
00287     uint16_t case_vac_at_end_cinhg() const {
00288         return case_vac_at_end_cinhg_;
00289     }
00290
00291     uint16_t case_vac_at_start_cinhg() const {
00292         return case_vac_at_start_cinhg_;
00293     }
00294
00295     template<typename T = float> T case_vac_at_start_inhg() const {
00296         return (T) case_vac_at_start_cinhg() / 100.;
00297     }
00298
00299     template<typename T = float> T case_vac_at_end_inhg() const {
00300         return (T) case_vac_at_end_cinhg() / 100.;
00301     }
00302
00303     uint16_t fall_start_pres_counts() const {
00304       return fall_start_pres_counts_;
00305     }
00306
00307     template<typename T = float> T fall_start_pres_dbar(int16_t gain, int16_t offs) const {
00308         T val;
00309
00310         val = gain ? (T) fall_start_pres_counts() / (T) gain - (T) offs : (T) XMSG_SENS_BAD_PRES;
00311
00312         if(val < XMSG_SENS_MIN_PRES || val > XMSG_SENS_MAX_PRES)
00313             return XMSG_SENS_BAD_PRES;
00314
00315         return val;
00316     }
00317
00318     const ptime& fall_start_time() const {
00319       return fall_start_time_;
00320     }
00321
00322     uint16_t pump_run_duration() const {
00323         return pump_run_duration_;
00324     }
00325
00326     uint16_t system_current_ma() const {
00327         return system_current_ma_;
00328     }
00329
00330     /** Get the system current, in amps.
00331      *
00332      *  @return         The system current, in amps.
00333      */
00334     template<typename T = float> T system_current_a() const {
00335         return (T) system_current_ma_ / 1000.;
00336     }
00337
00338 protected:
00339
00340     /* Member Functions - Mutator Methods: */
00341
00342     void set_case_vac_at_end_cinhg(uint16_t case_vac_at_end_cinhg) {
00343         case_vac_at_end_cinhg_ = case_vac_at_end_cinhg;
00344     }
00345
00346     void set_case_vac_at_start_cinhg(uint16_t case_vac_at_start_cinhg) {
00347         case_vac_at_start_cinhg_ = case_vac_at_start_cinhg;
00348     }
00349
00350     void set_fall_start_pres_counts(uint16_t fall_start_pres) {
00351       fall_start_pres_counts_ = fall_start_pres;
00352     }
00353
00354     void set_fall_start_time(const ptime& fall_start_time) {
00355       fall_start_time_ = fall_start_time;
00356     }
00357
```

```
00358     void set_pump_run_duration(uint16_t pump_run_duration) {
00359         pump_run_duration_ = pump_run_duration;
00360     }
00361
00362     void set_system_current_ma(uint16_t system_current_ma) {
00363         system_current_ma_ = system_current_ma;
00364     }
00365
00366 };
00367
00368 } /* namespace xmsg */
00369 } /* namespace util */
00370 #endif /* XENGASCENT_H_ */
```

11.108 XEngBIT.h

```
00001 /*
00002  * XEngBIT.h
00003  *
00004  *  Created on: Apr 20, 2023
00005  *      Author: aekholm
00006  */
00007
00008 #ifndef DATA_XMSG_XENGBIT_H_
00009 #define DATA_XMSG_XENGBIT_H_
00010
00011 /* Includes */
00012
00013 // local
00014 #include "XAlamoBit.h"
00015 #include "XAlamoBit2.h"
00016
00017 namespace util {
00018 namespace xmsg {
00019
00020 /** The ALAMO Built-in-Test (BIT) engineering message.
00021  *
00022  *  An instance of this class represents a BIT engineering X-message. Depending on the BIT
00023  *  format in byte 3, the message is decoded as an @ref XAlamoBit (formats 6-8, firmware before
00024  *  10.x) or an @ref XAlamoBit2 (formats 9-11, firmware 10.x and later).
00025  */
00026 class XEngBIT:
00027     public XMessage {
00028
00029 private:
00030
00031     /* Data Members */
00032
00033     /** A shared pointer to the ALALMO Built-in-Test (BIT) message.
00034      *
00035      *  @note           This member is only used for ALAMO-type instruments prior to firmware
00036      *                  version 10.x.
00037      */
00038     shared_ptr<XAlamoBit> alamo_bit_msg_;
00039
00040     /** A shared pointer to the ALALMO Built-in-Test (BIT) message.
00041      *
00042      *  @note           This member is only used for ALAMO-type instruments starting with firmware
00043      *                  version 10.x.
00044      */
00045     shared_ptr<XAlamoBit2> alamo_bit2_msg_;
00046
00047 protected:
00048
00049     /* Member Functions */
00050
00051     /** Set the ALALMO Built-in-Test (BIT) v1 message
00052      *
00053      *  @param[in]  alamo_bit_msg   A shared pointer to the ALALMO Built-in-Test (BIT) message to
00054      *                              set.
00055      */
00056     void set_alamo_bit_msg(const shared_ptr<XAlamoBit>& alamo_bit_msg) {
00057         alamo_bit_msg_ = alamo_bit_msg;
00058     }
00059
00060     /** Set the ALAMO Built-in-Test (BIT) v2 message.
00061      *
00062      *  @param[in]  alamo_bit2_msg  A shared pointer to the ALAMO Built-in-Test (BIT) v2 message to
00063      *                              set.
00064      */
00065     void set_alamo_bit2_msg(const shared_ptr<XAlamoBit2>& alamo_bit2_msg) {
00066         alamo_bit2_msg_ = alamo_bit2_msg;
00067     }
00068
00069 public:
00070
00071     /* Member Functions */
00072
00073     /** Parameterized constructor.
00074      *
00075      *  @param[in]      dive        The dive or cycle number of this message.
00076      *  @param[in]      whoi        The unique WHOI identifier.
00077      *  @param[in]      data        The binary X-message data to decode.
00078      *
00079      *  @throw          util::log::MessageException if an decode error occurs.
00080      */
00081     XEngBIT(int16_t dive, uint16_t whoi, const vector<uint8_t> data) :
00082             XMessage(dive, whoi, data) {
00083       DecodeXMsgData(data); // Decode message.
00084     }
00085
00086     /** Decode the BIT engineering X-Message from the binary data.
00087      *
00088      *  @param[in]      data            The binary message to decode.
```

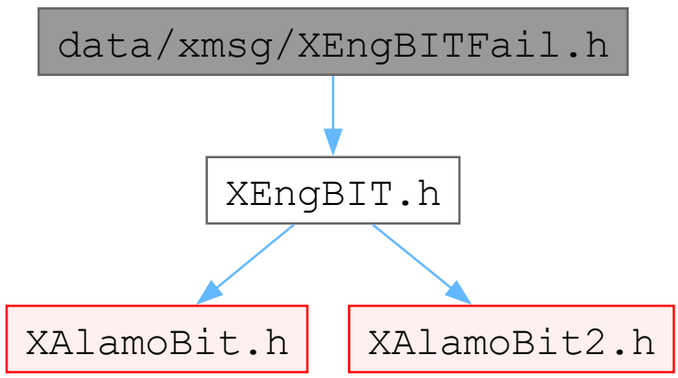
```
00089      *
00090      *  @throw          util::log::MessageException if an error occurs while decoding.
00091      */
00092     void DecodeXMsgData(const vector<uint8_t> &data) {
00093
00094       try {
00095
00096         CheckXMsgData(data, -1, 13);
00097         if(data.at(3) == 6 || data.at(3) == 7 || data.at(3) == 8) {
00098           alamo_bit_msg_ = std::make_shared<XAlamoBit>(data);
00099         } else if(data.at(3) == 9 || data.at(3) == 10 || data.at(3) == 11) {
00100           alamo_bit2_msg_ = std::make_shared<XAlamoBit2>(data);
00101         } else {
00102           ostringstream os;
00103           os « "Unsupported BIT message format: " « (int)data.at(3) « "!";
00104           EXC_THROW(os.str());
00105         }
00106
00107       } CATCH_BLOCK("Failed to decode BIT engineering X-Message!")
00108     }
00109
00110     /** Return a listing entry string representation of the BIT engineering X-Message.
00111      *
00112      *  @param[in]  pg          The pressure sensor sensitivity (gain) scale factor, in LSB/dBar.
00113      *  @param[in]  po          The pressure sensor offset, in dBar.
00114      *  @param[in]  sg          The salinity sensor sensitivity  (gain) scale factor, in LSB/PSU.
00115      *  @param[in]  so          The salinity sensor offset, in PSU.
00116      *  @param[in]  tg          The temperature sensor sensitivity  (gain) scale factor, in LSB/degC.
00117      *  @param[in]  to          The temperature sensor offset, in degC.
00118      *
00119      *  @throw      util::log::MessageException if an error occurs while generating the listing
00120      *              entry string representation.
00121      *
00122      *  @return     A listing entry string representation of the BIT engineering X-Message.
00123      *
00124      */
00125     const string GenerateListingString(
00126         uint16_t pg,
00127         uint16_t po,
00128         uint16_t sg,
00129         uint16_t so,
00130         uint16_t tg,
00131         uint16_t to) const {
00132       ostringstream os;
00133
00134       try {
00135
00136         if(alamo_bit_msg_) {
00137
00138           os « alamo_bit_msg_->GenerateListingString(
00139               id_instrument(),
00140               id_dive(),
00141               pg,
00142               po,
00143               sg,
00144               so,
00145               tg,
00146               to);
00147
00148         } else if(alamo_bit2_msg_) {
00149
00150           os « alamo_bit2_msg_->GenerateListingString(
00151               id_instrument(),
00152               id_dive());
00153
00154         }
00155
00156       }CATCH_BLOCK("Failed to generate listing entry string representation of BIT message!")
00157
00158       return os.str();
00159     }
00160
00161
00162
00163 };
00164
00165 } /* namespace xmsg */
00166 } /* namespace util */
00167
00168 #endif /* DATA_XMSG_XENGBIT_H_ */
```

11.109 data/xmsg/XEngBITFail.h File Reference

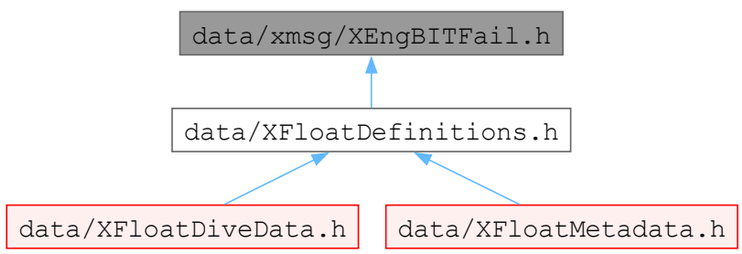
The util::xmsg::XEngBITFail class header file.

```
#include "XEngBIT.h"
```

Include dependency graph for XEngBITFail.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XEngBITFail

  Class representing an engineering startup message for BIT failed.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.109.1 Detailed Description

The util::xmsg::XEngBITFail class header file.

Version

0.6

Date

Jul 1, 2014 2:02:19 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.110 XEngBITFail.h

Go to the documentation of this file.

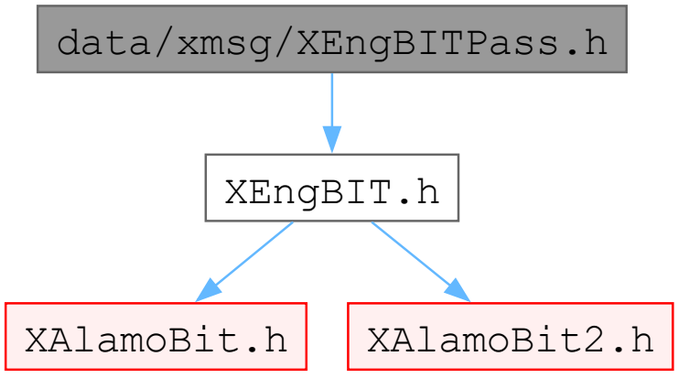
```
00001 /** The @ref util::xmsg::XEngBITFail class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jul 1, 2014 2:02:19 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023
00024 #ifndef XENGBITFAIL_H_
00025 #define XENGBITFAIL_H_
00026
00027 /* Includes */
00028
00029
00030 // local
00031 #include "XEngBIT.h"
00032
00033
00034 /* Definitions */
00035
00036 namespace util {
00037 namespace xmsg {
00038
00039 /* Class Definition */
00040
00041 /** Class representing an engineering startup message for BIT failed.
00042  *
00043  */
00044 class XEngBITFail:
00045         public XEngBIT {
00046
00047           public:
00048
00049     /* Member Functions */
00050
00051     /** Parameterized constructor.
00052      *
00053      *  @param[in]      dive        The dive or cycle number of this message.
00054      *  @param[in]      whoi        The unique WHOI identifier.
00055      *  @param[in]      data        The binary X-message data to decode.
00056      *
00057      *  @throw          util::log::MessageException if an decode error occurs.
00058      */
00059     XEngBITFail(int16_t dive, uint16_t whoi, const vector<uint8_t> data) :
00060             XEngBIT(dive, whoi, data) {
00061       /* Do nothing. */
00062     }
00063
00064     /** The destructor.
00065      *
00066      */
00067     ~XEngBITFail() { /* Do nothing. */ }
00068
00069 };
00070
00071 } /* namespace xmsg */
00072 } /* namespace util */
00073 #endif /* XENGBITFAIL_H_ */
```

11.111 data/xmsg/XEngBITPass.h File Reference

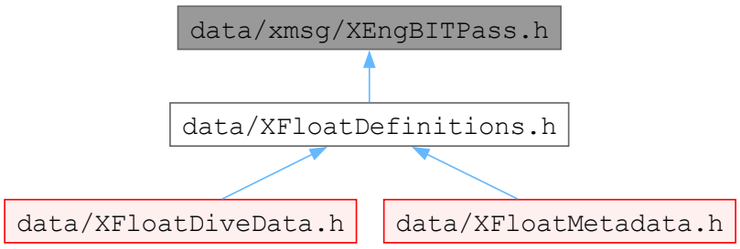
The util::xmsg::XEngBITPass class header file.

```
#include "XEngBIT.h"
```

Include dependency graph for XEngBITPass.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XEngBITPass

  Class representing an engineering startup message for BIT passed.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.111.1 Detailed Description

The util::xmsg::XEngBITPass class header file.

Version

0.6

Date

May 12, 2014 4:10:33 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.112 XEngBITPass.h

Go to the documentation of this file.

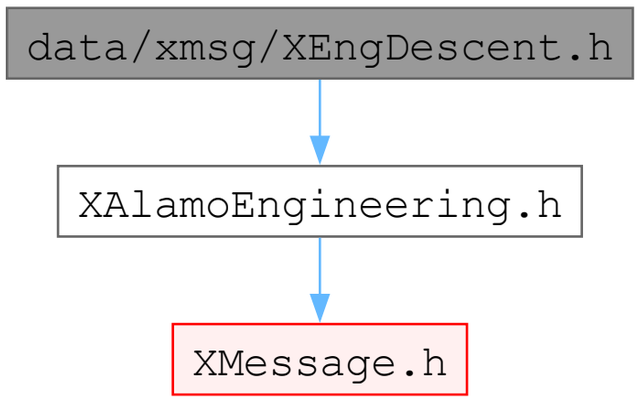
```
00001 /** The @ref util::xmsg::XEngBITPass class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       May 12, 2014 4:10:33 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef XENGBITPASS_H_
00024 #define XENGBITPASS_H_
00025
00026 /* Includes */
00027
00028 // local
00029 #include "XEngBIT.h"
00030
00031 namespace util {
00032 namespace xmsg {
00033
00034 /* Class Definition */
00035
00036 /** Class representing an engineering startup message for BIT passed.
00037  *
00038  *  @todo           Add support for Solo-II type bit passed engineering data message.
00039  */
00040 class XEngBITPass:
00041         public XEngBIT {
00042
00043 public:
00044
00045     /* Member Functions */
00046
00047     /** Parameterized constructor.
00048      *
00049      *  @param[in]      dive        The dive or cycle number of this message.
00050      *  @param[in]      whoi        The unique WHOI identifier.
00051      *  @param[in]      data        The binary X-message data to decode.
00052      *
00053      *  @throw          util::log::MessageException if an decode error occurs.
00054      */
00055     XEngBITPass(int16_t dive, uint16_t whoi, const vector<uint8_t> data) :
00056             XEngBIT(dive, whoi, data) {
00057       /* Do nothing. */
00058     }
00059
00060     /** Destructor.
00061      *
00062      */
00063     ~XEngBITPass() {
00064         /* Do nothing. */
00065     }
00066
00067
00068
00069
00070
00071 };
00072
00073 } /* namespace xmsg */
00074 } /* namespace util */
00075 #endif /* XENGBITPASS_H_ */
```

11.113 data/xmsg/XEngDescent.h File Reference

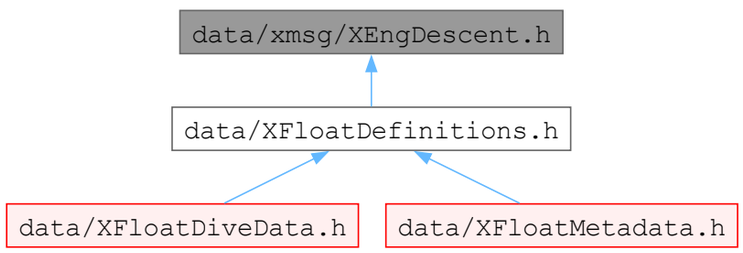
The util::xmsg::XEngDescent class header file.

```
#include "XAlamoEngineering.h"
```

Include dependency graph for XEngDescent.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XEngDescent

  Class representing an descend state engineering X-Message.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.113.1 Detailed Description

The util::xmsg::XEngDescent class header file.

Version

0.6

Date

Jul 1, 2014 2:00:18 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.114 XEngDescent.h

Go to the documentation of this file.

```
00001 /** The @ref util::xmsg::XEngDescent class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jul 1, 2014 2:00:18 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef XENGDESCENT_H_
00024 #define XENGDESCENT_H_
00025
00026 /* Includes */
00027
00028 #include "XAlamoEngineering.h"
00029
00030 namespace util {
00031 namespace xmsg {
00032
00033 /* Class Definition */
00034
00035 /** Class representing an descend state engineering X-Message.
00036  *
00037  *  @note           This X-Message is only used by ALAMO-type instruments.
00038  *  @note           This message type was added in X-Message version 2.1.
00039  */
00040 class XEngDescent:
00041         public XAlamoEngineering {
00042
00043 private:
00044
00045     /* Data Members */
00046
00047     /** The air valve middle position at surface, in .001 V.
00048      *
00049      *  @note This field was added in version 0x11.
00050      */
00051     uint16_t air_valve_middle_position_at_surface_mv_;
00052
00053     /** The measured internal pressure case vacuum, in 0.01 inHg, at the start of the
00054      *  descent-to-drift/park-pressure phase of the dive cycle.
00055      *
00056      *  @note This field was added in version 0x11.
00057      *
00058      */
00059     uint16_t case_vac_at_start_cinhg_;
00060
00061     /** The measured internal pressure case vacuum, in 0.01 inHg, at the end of the
00062      *  descent-to-drift/park-pressure phase of the dive cycle.
00063      *
00064      *  @note This field was added in version 0x11.
00065      *
00066      */
00067     uint16_t case_vac_at_end_cinhg_;
00068
00069     /** The measured internal pressure case vacuum, in 0.01 inHg.
00070      *
00071      *  @note This field is only used for message version 0x10. It was removed in version 0x11.
```

```
00072      *
00073      */
00074     uint16_t case_vac_cinhg_;
00075
00076     /** The the first-pump run time adjustment, in seconds.
00077      *
00078      */
00079     int16_t first_pump_duration_adjustment_s_;
00080
00081     /** The number of seek attempts.
00082      *
00083      */
00084     uint8_t num_seek_attempts_;
00085
00086     /** The oil valve open position at surface, in .001 V.
00087      *
00088      *  @note This field was added in version 0x11.
00089      */
00090     uint16_t oil_valve_open_position_at_surface_mv_;
00091
00092     /** The oil valve closed position during fall pump, in .001 V.
00093      *
00094      *  @todo Clarify the description of the "oil valve position during fall pump" field.
00095      *
00096      *  @note This field was added in version 0x11.
00097      *
00098      *
00099      */
00100     uint16_t oil_valve_closed_position_during_fall_pump_mv_;
00101
00102     /** The measured pump current, in milliamps.
00103      *
00104      *  @note This field is only used for message version 0x10. It was removed in version 0x11.
00105      *
00106      */
00107     uint16_t pump_current_ma_;
00108
00109     /** The cumulative pump run time for the descent state of the dive cycle, in seconds.
00110      *
00111      *  @note This field is only used for message version 0x10. It was removed in version 0x11.
00112      *
00113      */
00114     uint16_t pump_duration_sec_;
00115
00116     /** The measured system current, in milliamps.
00117      *
00118      *  @note This field is only used for message version 0x10. It was removed in version 0x11.
00119      */
00120     uint16_t system_current_ma_;
00121
00122     /** The measured system current, in millivolts.
00123      *
00124      *  @note This field is only used for message version 0x10. It was removed in version 0x11.
00125      */
00126     uint16_t system_voltage_mv_;
00127
00128 protected:
00129
00130     /* Member Functions */
00131
00132     void set_air_valve_middle_position_at_surface_mv(
00133             uint16_t air_valve_middle_position_at_surface_mv) {
00134         air_valve_middle_position_at_surface_mv_ = air_valve_middle_position_at_surface_mv;
00135     }
00136
00137     void set_case_vac_at_end_cinhg(uint16_t case_vac_at_end_cinhg) {
00138         case_vac_at_end_cinhg_ = case_vac_at_end_cinhg;
00139     }
00140
00141     void set_case_vac_at_start_cinhg(uint16_t case_vac_at_start_cinhg) {
00142         case_vac_at_start_cinhg_ = case_vac_at_start_cinhg;
00143     }
00144
00145     void set_case_vac_cinhg(uint16_t case_vac_cinhg) {
00146         case_vac_cinhg_ = case_vac_cinhg;
00147     }
00148
00149     void set_num_seek_attempts(uint8_t num_seek_attempts) {
00150         num_seek_attempts_ = num_seek_attempts;
00151     }
00152
00153     void set_oil_valve_closed_position_during_fall_pump_mv(
00154             uint16_t oil_valve_closed_position_during_fall_pump_mv) {
00155         oil_valve_closed_position_during_fall_pump_mv_ = oil_valve_closed_position_during_fall_pump_mv;
00156     }
00157
00158     void set_oil_valve_open_position_at_surface_mv(uint16_t oil_valve_open_position_at_surface_mv) {
00159         oil_valve_open_position_at_surface_mv_ = oil_valve_open_position_at_surface_mv;
00160     }
00161
00162     void set_pump_current_ma(uint16_t pump_current_ma) {
00163         pump_current_ma_ = pump_current_ma;
00164     }
00165
00166     void set_pump_duration_sec(uint16_t pump_duration_sec) {
00167         pump_duration_sec_ = pump_duration_sec;
00168     }
00169
00170     void set_first_pump_duration_adjustment_s(int16_t first_pump_duration_adjustment_s) {
00171         first_pump_duration_adjustment_s_ = first_pump_duration_adjustment_s;
00172     }
00173
00174     void set_system_current_ma(uint16_t system_current_ma) {
00175         system_current_ma_ = system_current_ma;
00176     }
00177
00178     void set_system_voltage_mv(uint16_t system_voltage_mv) {
00179         system_voltage_mv_ = system_voltage_mv;
```

```
00180     }
00181
00182 public:
00183
00184     /* Member Functions */
00185
00186     /** The parameterized constructor.
00187      *
00188      *  @param[in]      dive        The dive or cycle number of this message.
00189      *  @param[in]      whoi        The unique WHOI identifier.
00190      *  @param[in]      data        The binary X-message data to decode.
00191      *
00192      *  @throw          util::log::MessageException if an decode error occurs.
00193      */
00194     XEngDescent(int16_t dive, uint16_t whoi, const vector<uint8_t>& data) :
00195                     XAlamoEngineering(dive, whoi, data),
00196                     air_valve_middle_position_at_surface_mv_(XMSG_BAD_U16),
00197                     case_vac_at_start_cinhg_(XMSG_BAD_U16),
00198                     case_vac_at_end_cinhg_(XMSG_BAD_U16),
00199                     case_vac_cinhg_(XMSG_BAD_U16),
00200                     first_pump_duration_adjustment_s_(XMSG_BAD_I16),
00201                     num_seek_attempts_(XMSG_BAD_U08),
00202                     oil_valve_open_position_at_surface_mv_(XMSG_BAD_U16),
00203                     oil_valve_closed_position_during_fall_pump_mv_(XMSG_BAD_U16),
00204                     pump_current_ma_(XMSG_BAD_U16),
00205                     pump_duration_sec_(XMSG_BAD_U16),
00206                     system_current_ma_(XMSG_BAD_U16),
00207                     system_voltage_mv_(XMSG_BAD_U16) {
00208         DecodeXMsgData(data); // Decode message.
00209     }
00210
00211     /** The destructor.
00212      *
00213      */
00214     ~XEngDescent() {
00215         /* Do nothing. */
00216     }
00217
00218     /** Decode the descent state engineering X-Message from the binary data.
00219      *
00220      *  @param[in]      data            The binary message to decode.
00221      */
00222     void DecodeXMsgData(const vector<uint8_t>& data) {
00223         try {
00224             // Check message.
00225             const uint8_t* d = CheckXMsgData(data, 0xE7, 22);
00226
00227             set_engineering_version(d[3]);
00228             switch(engineering_version()) {
00229
00230             case 0x10: // Engineering (Descent) version 0x10.
00231
00232                 // Check the message size.
00233                 if(data.size() != 23 && data.size() != 27) {
00234                     ostringstream os;
00235                     os << "Invalid X-Message: Expected 23 or 27 bytes, but received "
00236                        << data.size()
00237                        << " bytes!";
00238                     EXC_THROW(os.str());
00239                 }
00240
00241                 set_start_time(s2000(d, 4));
00242                 set_pump_duration_sec(u16(d, 8));
00243                 set_pump_current_ma(u16(d, 10));
00244                 set_system_current_ma(u16(d, 12));
00245                 if(xmsg_size_bytes() == 27) {
00246                     set_system_voltage_mv(u16(d, 14));
00247                     set_case_vac_cinhg(u16(d, 16));
00248                     set_first_pump_duration_adjustment_s(i16(d, 18));
00249                 }
00250
00251                 break;
00252
00253             case 0x11: // Engineering (Descent) version 0x10.
00254
00255                 // Check the message size.
00256                 if(data.size() != 22) {
00257                     ostringstream os;
00258                     os << "Invalid X-Message: Expected 22 bytes, but received "
00259                        << data.size()
00260                        << " bytes!";
00261                     EXC_THROW(os.str());
00262                 }
00263
00264                 set_start_time(s2000(d, 4));
00265                 set_case_vac_at_start_cinhg(u16(d, 8));
00266                 set_case_vac_at_end_cinhg(u16(d, 10));
00267                 set_oil_valve_open_position_at_surface_mv(u16(d, 12));
00268                 set_oil_valve_closed_position_during_fall_pump_mv(u16(d, 14));
00269                 set_air_valve_middle_position_at_surface_mv(u16(d, 16));
00270                 set_num_seek_attempts(u08(d, 18));
00271                 set_first_pump_duration_adjustment_s(i16(d, 19));
00272
00273                 break;
00274
00275             default: { // Unsupported Engineering (Descent) version.
00276
00277                 ostringstream os;
00278                 os << "Unsupported ALAMO Engineering (Descent) message version: 0x"
00279                    << hex
00280                    << (int) engineering_version()
00281                    << "!";
00282                 EXC_THROW(os.str());
00283             }
00284
00285                 break;
00286             }
00287
```

```
00288         }CATCH_BLOCK("Failed to decode ALAMO Engineering (Descent) X-Message!")
00289     }
00290
00291     /** Get the listing entry string for this descent engineering data X-Message
00292      *
00293      *  @return         The listing entry string for this descent engineering data X-Message
00294      */
00295     const string GenerateListingString() const {
00296         ostringstream os;
00297
00298         try {
00299
00300             switch(engineering_version()) {
00301
00302             case 0x10: { // Engineering (Descent) version 0x10.
00303
00304                 static const format fmt1("#   %5s %4s %4s %6s %20s %5s %6s %6s %6s %6s %7s");
00305                 static const format fmt2("#   %5s %4s %4s %6s %20s %5s %6s %6s %6s %7s%7s");
00306                 static const format fmt3(
00307                         "E07 %05d %4d %4d   0x%-02X %20s %5d %6.3f %6.3f %6.3f %6.2f %+7d");
00308                 os « format(fmt1) % "SN" % "dive" % "midx" % "engVer" % "start-of-descent"
00309                           % "pumpT"
00310
00311                           % "pumpI" % "sysI" % "sysV" % "vac" % "pumpAdj"
00312                    « endl;
00313                 os « format(fmt2) % "" % "" % "" % "" % "" % "(sec)" % "(A)" % "(A)" % "(V)"
00314                           % "(inHg)" % "(+sec)"
00315                    « endl;
00316                 os « format(fmt3) % id_instrument() % id_dive() % 0 % (int) engineering_version()
00317                         % GenerateIso8601String(start_time()) % pump_duration_sec()
00318                         % pump_current_a() % system_current_a() % system_voltage_v()
00319                         % case_vac_inhg() % first_pump_duration_adjustment_s();
00320             }
00321                 break;
00322
00323             case 0x11: { // Engineering (Descent) version 0x11.
00324
00325                 // Minor index 0.
00326                 static const format fmt1("#   %5s %4s %4s %6s %20s %9s %9s %15s %15s");
00327                 static const format fmt2("#   %5s %4s %4s %6s %20s %9s %9s %15s %15s");
00328                 static const format fmt3("E07 %05d %4d %4d   0x%-02X %20s %9.2f %9.2f %15d %+15d");
00329                 os « format(fmt1) % "SN" % "dive" % "midx" % "engVer" % "start-of-descent" % "vac0"
00330                           % "vac1" % "seeks" % "seekPumpAdjT"
00331                    « endl;
00332                 os « format(fmt2) % "" % "" % "" % "" % "" % "(inHg)" % "(inHg)" % "" % "(+sec)"
00333                    « endl;
00334                 os « format(fmt3) % id_instrument() % id_dive() % 0 % (int) engineering_version()
00335                           % GenerateIso8601String(start_time()) % case_vac_at_start_inhg()
00336                           % case_vac_at_end_inhg() % (int) num_seek_attempts()
00337                           % first_pump_duration_adjustment_s()
00338                    « endl;
00339
00340                 // Minor index 1.
00341                 static const format fmt4("#   %5s %4s %4s %24s %24s %29s");
00342                 static const format fmt5("#   %5s %4s %4s %24s %24s %29s");
00343                 static const format fmt6("E07 %05d %4d %4d %24.3f %24.3f %29.3f");
00344                 os « "# --- Valve Position Measurements ------------------------------------------"
00345                           "-------------------------"
00346                    « endl;
00347                 os « format(fmt4) % "SN" % "dive" % "midx" % "OilValveOpenPosSurf"
00348                           % "OilValveClosePosFallPump" % "AirValveMidPosSurf"
00349                    « endl;
00350                 os « format(fmt5) % "" % "" % "" % "(V)" % "(V)" % "(V)" « endl;
00351                 os « format(fmt6) % id_instrument() % id_dive() % 1
00352                        % oil_valve_open_position_at_surface_v()
00353                        % oil_valve_closed_position_during_fall_pump_v()
00354                        % air_valve_middle_position_at_surface_v();
00355             }
00356                 break;
00357
00358             default: { // Unsupported Engineering (Descent) version.
00359                 ostringstream os;
00360                 os « "Unsupported ALAMO Engineering (Descent) message version: 0x"
00361                    « hex
00362                    « (int) engineering_version()
00363                    « "!";
00364                 EXC_THROW(os.str());
00365             }
00366                 break;
00367             }
00368
00369         }CATCH_BLOCK("Failed to generate listing string for ALAMO Engineering (Descent) message!")
00370
00371         return os.str();
00372     }
00373
00374     uint16_t air_valve_middle_position_at_surface_mv() const {
00375         return air_valve_middle_position_at_surface_mv_;
00376     }
00377
00378     uint16_t case_vac_at_end_cinhg() const {
00379         return case_vac_at_end_cinhg_;
00380     }
00381
00382     uint16_t case_vac_at_start_cinhg() const {
00383         return case_vac_at_start_cinhg_;
00384     }
00385
00386     uint16_t case_vac_cinhg() const {
00387         return case_vac_cinhg_;
00388     }
00389
00390     uint8_t num_seek_attempts() const {
00391         return num_seek_attempts_;
00392     }
00393
00394     uint16_t oil_valve_closed_position_during_fall_pump_mv() const {
00395         return oil_valve_closed_position_during_fall_pump_mv_;
```

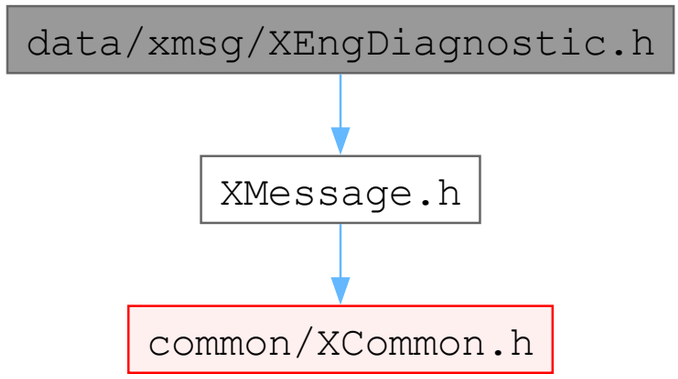
```
00396     }
00397
00398     uint16_t oil_valve_open_position_at_surface_mv() const {
00399         return oil_valve_open_position_at_surface_mv_;
00400     }
00401
00402     uint16_t pump_current_ma() const {
00403         return pump_current_ma_;
00404     }
00405
00406     uint16_t pump_duration_sec() const {
00407         return pump_duration_sec_;
00408     }
00409
00410     int16_t first_pump_duration_adjustment_s() const {
00411         return first_pump_duration_adjustment_s_;
00412     }
00413
00414     uint16_t system_current_ma() const {
00415         return system_current_ma_;
00416     }
00417
00418     uint16_t system_voltage_mv() const {
00419         return system_voltage_mv_;
00420     }
00421
00422     /** Get the measured pressure case vacuum, in inches of mercury.
00423      *
00424      *  @return         The measured pressure case vacuum, in inches of mercury.
00425      */
00426     template<typename T = float> T case_vac_inhg() const {
00427         return (T) case_vac_cinhg() / 100.;
00428     }
00429
00430     template<typename T = float> T case_vac_at_start_inhg() const {
00431         return (T) case_vac_at_start_cinhg() / 100.;
00432     }
00433
00434     template<typename T = float> T case_vac_at_end_inhg() const {
00435         return (T) case_vac_at_end_cinhg() / 100.;
00436     }
00437
00438     template<typename T = float> T oil_valve_open_position_at_surface_v() const {
00439         return (T) oil_valve_open_position_at_surface_mv() / 1000.;
00440     }
00441
00442     template<typename T = float> T oil_valve_closed_position_during_fall_pump_v() const {
00443         return (T) oil_valve_closed_position_during_fall_pump_mv() / 1000.;
00444     }
00445
00446     template<typename T = float> T air_valve_middle_position_at_surface_v() const {
00447         return (T) air_valve_middle_position_at_surface_mv() / 1000.;
00448     }
00449
00450     /** Get the measured pump current, in amperes.
00451      *
00452      *  @return         The measured pump current, in amperes.
00453      */
00454     template<typename T = float> T pump_current_a() const {
00455         return (T) pump_current_ma_ / 1000.;
00456     }
00457
00458     /** Get the measured system current, in amperes.
00459      *
00460      *  @return         The measured system current, in amperes.
00461      */
00462     template<typename T = float> T system_current_a() const {
00463         return (T) system_current_ma_ / 1000.;
00464     }
00465
00466     /** Get the measured system voltage, in volts.
00467      *
00468      *  @return         The measured system voltage, in volts.
00469      */
00470     template<typename T = float> T system_voltage_v() const {
00471         return (T) system_voltage_mv_ / 1000.;
00472     }
00473
00474 };
00475
00476 } /* namespace xmsg */
00477 } /* namespace util */
00478 #endif /* XENGDESCENT_H_ */
```

11.115 data/xmsg/XEngDiagnostic.h File Reference

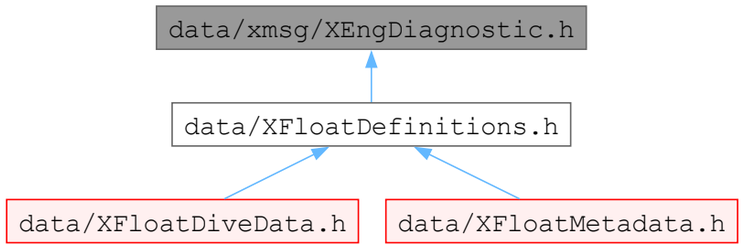
The util::xmsg::XEngDiagnostic class source file.

```
#include "XMessage.h"
```

Include dependency graph for XEngDiagnostic.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XEngDiagnostic

  Class representing a diagnostic dive engineering message.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.115.1 Detailed Description

The util::xmsg::XEngDiagnostic class source file.

Version

0.6

Date

Jul 1, 2014 1:56:51 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.116 XEngDiagnostic.h

Go to the documentation of this file.

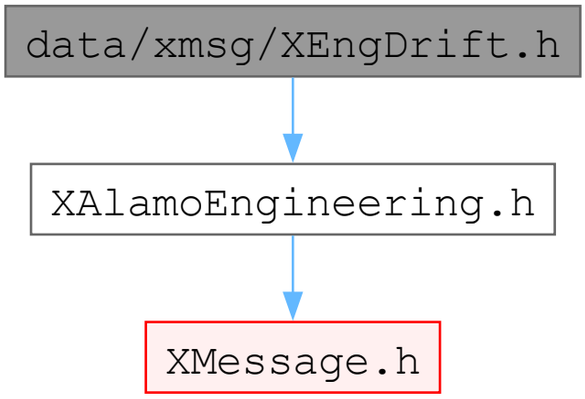
```
00001 /** The @ref util::xmsg::XEngDiagnostic class source file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jul 1, 2014 1:56:51 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef XENGDIAGNOSTIC_H_
00024 #define XENGDIAGNOSTIC_H_
00025
00026 /* Includes */
00027
00028 #include "XMessage.h"
00029
00030 /* Definitions */
00031
00032 namespace util {
00033 namespace xmsg {
00034
00035 /* Class Definitions */
00036
00037 /** Class representing a diagnostic dive engineering message.
00038  *
00039  */
00040 class XEngDiagnostic: public XMessage {
00041
00042 public:
00043
00044     /* Member Functions */
00045
00046     /** Parameterized constructor.
00047      *
00048      *  @param[in]      dive        The dive or cycle number of this message.
00049      *  @param[in]      whoi        The unique WHOI identifier.
00050      *  @param[in]      data        The binary X-message data to decode.
00051      *
00052      *  @throw          util::log::MessageException if an decode error occurs.
00053      */
00054     XEngDiagnostic(int16_t dive, uint16_t whoi, const vector<uint8_t>& data) :
00055             XMessage(dive, whoi, data) {
00056         DecodeXMsgData(data); // Decode message.
00057     }
00058
00059     /** Destructor
00060      *
00061      */
00062     ~XEngDiagnostic() {
00063         /* Do nothing. */
00064     }
00065
00066     /** Decode the diagnostic dive engineering X-Message from the binary data.
00067      *
00068      *  @param[in]      data            The binary message to decode.
00069      */
00070     void DecodeXMsgData(const vector<uint8_t>& data) {
00071
00072         try {
00073
00074             // Check message.
00075             /* const uint8_t* d = */ CheckXMsgData(data, XMSG_TYPE_ID_ENG_DIAGNOSTIC, -1);
00076
00077             /** @todo   Implement the functionality for decoding the Solo-II diagnostic engineering
00078              *          data message.
```

```
00079             *
00080             */
00081
00082         }CATCH_BLOCK("Failed to decode diagnostic dive engineering X-Message!")
00083     }
00084
00085     /** Generate a listing string representation of the diagnostic dive engineering data message.
00086      *
00087      *  @return     A listing string representation of the diagnostic dive engineering data message.
00088      */
00089     const string GenerateListingString() const {
00090         ostringstream os;
00091
00092         try {
00093
00094             /** @todo   Implement the functionality for generating a listing string for the Solo-II
00095              *          diagnostic engineering data message.
00096              *
00097              */
00098
00099         } CATCH_BLOCK("Failed to generate listing string for diagnostic engineering data message!")
00100
00101         return os.str();
00102     }
00103 };
00104
00105 } /* namespace xmsg */
00106 } /* namespace util */
00107 #endif /* XENGDIAGNOSTIC_H_ */
```

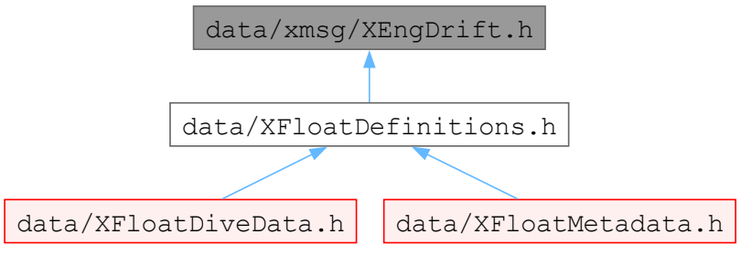
11.117 data/xmsg/XEngDrift.h File Reference

The util::xmsg::XEngDrift class header file.

`#include "XAlamoEngineering.h"`
Include dependency graph for XEngDrift.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XEngDrift

  Class representing an drift/park state engineering X-Message.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.117.1 Detailed Description

The util::xmsg::XEngDrift class header file.

Version

0.6

Date

Jul 1, 2014 2:00:39 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.118 XEngDrift.h

Go to the documentation of this file.

```
00001 /** The @ref util::xmsg::XEngDrift class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jul 1, 2014 2:00:39 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef XENGDRIFT_H_
00024 #define XENGDRIFT_H_
00025
00026 /* Includes */
00027
00028 #include "XAlamoEngineering.h"
00029
00030 namespace util {
00031 namespace xmsg {
00032
00033 /** Class representing an drift/park state engineering X-Message.
00034  *
00035  *  @note           This X-Message is only used by ALAMO-type instruments.
00036  *  @note           This message type was introduced in the X-Message format specification
00037  *                  version 2.1.
00038  */
00039 class XEngDrift:
00040         public XAlamoEngineering {
00041
00042 private:
```

```
00043
00044     /* Data Members */
00045
00046     /** The average pressure during the drift/park phase of the dive cycle, in raw pressure counts.
00047      */
00048     uint16_t average_drift_pressure_counts_;
00049
00050     uint16_t average_drift_salinity_counts_;
00051
00052     /** The average temperature during the drift/park phase of the dive cycle, in raw temperature
00053      *  counts.
00054      */
00055     uint16_t average_drift_temperature_counts_;
00056
00057     /** The measured internal pressure case vacuum, in 0.01 inHg, at the start of the
00058      *  drift/park phase of the dive cycle.
00059      *
00060      *  @note This field was added in version 0x11.
00061      *
00062      */
00063     uint16_t case_vac_at_start_cinhg_;
00064
00065     /** The measured internal pressure case vacuum, in 0.01 inHg, at the end of the
00066      *  drift/park phase of the dive cycle.
00067      *
00068      *  @note This field was added in version 0x11.
00069      *
00070      */
00071     uint16_t case_vac_at_end_cinhg_;
00072
00073     uint8_t num_seek_attempts_;
00074
00075     int16_t first_pump_duration_adjustment_s_;
00076
00077     /** The measured pressure at the start of the fall-to-profile phase of the dive cycle, in raw
00078      *  pressure counts.
00079      */
00080     uint16_t start_of_fall_to_profile_pressure_counts_;
00081
00082     /** The start time of the fall-to-profile phase of the dive cycle. */
00083     ptime start_of_fall_to_profile_pressure_time_;
00084
00085 protected:
00086
00087     /* Protected Member Functions */
00088
00089     void set_average_drift_pressure_counts(uint16_t average_drift_pressure_counts) {
00090         average_drift_pressure_counts_ = average_drift_pressure_counts;
00091     }
00092
00093     void set_average_drift_salinity_counts(uint16_t average_drift_salinity_counts) {
00094         average_drift_salinity_counts_ = average_drift_salinity_counts;
00095     }
00096
00097     void set_average_drift_temperature_counts(uint16_t average_drift_temperature_counts) {
00098         average_drift_temperature_counts_ = average_drift_temperature_counts;
00099     }
00100
00101     void set_case_vac_at_end_cinhg(uint16_t case_vac_at_end_cinhg) {
00102         case_vac_at_end_cinhg_ = case_vac_at_end_cinhg;
00103     }
00104
00105     void set_case_vac_at_start_cinhg(uint16_t case_vac_at_start_cinhg) {
00106         case_vac_at_start_cinhg_ = case_vac_at_start_cinhg;
00107     }
00108
00109     void set_first_pump_duration_adjustment_s(int16_t first_pump_duration_adjustment_s) {
00110         first_pump_duration_adjustment_s_ = first_pump_duration_adjustment_s;
00111     }
00112
00113     void set_num_seek_attempts(uint8_t num_seek_attempts) {
00114         num_seek_attempts_ = num_seek_attempts;
00115     }
00116
00117     void set_start_of_fall_to_profile_pressure_counts(
00118             uint16_t start_of_fall_to_profile_pressure_counts) {
00119         start_of_fall_to_profile_pressure_counts_ = start_of_fall_to_profile_pressure_counts;
00120     }
00121
00122     void set_start_of_fall_to_profile_pressure_time(
00123             const ptime& start_of_fall_to_profile_pressure_time) {
00124         start_of_fall_to_profile_pressure_time_ = start_of_fall_to_profile_pressure_time;
00125     }
00126
00127 public:
00128
00129     /* Public Member Functions */
00130
00131     /** Parameterized constructor.
00132      *
00133      *  @param[in]      dive        The dive or cycle number of this message.
00134      *  @param[in]      whoi        The unique WHOI identifier.
00135      *  @param[in]      data        The binary X-message data to decode.
00136      *
00137      *  @throw          util::log::MessageException if an decode error occurs.
00138      */
00139     XEngDrift(int16_t dive, uint16_t whoi, const vector<uint8_t>& data) :
00140                     XAlamoEngineering(dive, whoi, data),
00141                     average_drift_pressure_counts_(XMSG_BAD_U16),
00142                     average_drift_salinity_counts_(XMSG_BAD_U16),
00143                     average_drift_temperature_counts_(XMSG_BAD_U16),
00144                     case_vac_at_start_cinhg_(XMSG_BAD_U16),
00145                     case_vac_at_end_cinhg_(XMSG_BAD_U16),
00146                     num_seek_attempts_(XMSG_BAD_U08),
00147                     first_pump_duration_adjustment_s_(XMSG_BAD_I16),
00148                     start_of_fall_to_profile_pressure_counts_(XMSG_BAD_U16),
00149                     start_of_fall_to_profile_pressure_time_(not_a_date_time) {
00150         DecodeXMsgData(data); // Decode message.
```

```
00151     }
00152
00153     /** Destructor.
00154      *
00155      */
00156     ~XEngDrift() {
00157         /* Do nothing. */
00158     }
00159
00160     /** Decode the drift/park state engineering X-Message from the binary data.
00161      *
00162      *  @param[in]      data            The binary message to decode.
00163      */
00164     void DecodeXMsgData(const vector<uint8_t>& data) {
00165         try {
00166             // Check message.
00167             const uint8_t* d = CheckXMsgData(data, 0xE8, 19);
00168
00169             set_engineering_version(d[3]);
00170             switch(engineering_version()) {
00171
00172             case 0x10: { // Engineering (Drift) message version 0x10.
00173
00174               static const unsigned int EXPECTED_SIZE = 19;
00175
00176               if(data.size() != EXPECTED_SIZE) {
00177                 ostringstream os;
00178                 os « "Invalid X-Message: Expected"
00179                    « EXPECTED_SIZE
00180                    « "bytes, but received "
00181                    « data.size()
00182                    « " bytes!";
00183                 EXC_THROW(os.str());
00184               }
00185               set_start_time(s2000(d, 4));
00186               set_average_drift_pressure_counts(u16(d, 8));
00187               set_average_drift_temperature_counts(u16(d, 10));
00188               set_start_of_fall_to_profile_pressure_time(s2000(d, 12));
00189               set_start_of_fall_to_profile_pressure_counts(u16(d, 16));
00190
00191             }
00192
00193             break;
00194
00195             case 0x11: { // Engineering (Drift) message version 0x11.
00196
00197               static const unsigned int EXPECTED_SIZE = 28;
00198
00199               if(data.size() != EXPECTED_SIZE) {
00200                 ostringstream os;
00201                 os « "Invalid X-Message: Expected"
00202                    « EXPECTED_SIZE
00203                    « "bytes, but received "
00204                    « data.size()
00205                    « " bytes!";
00206                 EXC_THROW(os.str());
00207               }
00208
00209               set_start_time(s2000(d, 4));
00210               set_case_vac_at_start_cinhg(u16(d, 8));
00211               set_case_vac_at_end_cinhg(u16(d, 10));
00212               set_average_drift_pressure_counts(u16(d, 12));
00213               set_average_drift_temperature_counts(u16(d, 14));
00214               set_average_drift_salinity_counts(u16(d, 16));
00215               set_num_seek_attempts(u08(d, 18));
00216               set_first_pump_duration_adjustment_s(u16(d, 19));
00217               set_start_of_fall_to_profile_pressure_time(s2000(d, 21));
00218               set_start_of_fall_to_profile_pressure_counts(u16(d, 25));
00219             }
00220
00221             break;
00222
00223             case 0x12: { // Engineering (Drift) message version 0x12.
00224
00225               static const unsigned int EXPECTED_SIZE = 22;
00226
00227               if(data.size() != EXPECTED_SIZE) {
00228                   ostringstream os;
00229                   os « "Invalid X-Message: Expected"
00230                      « EXPECTED_SIZE
00231                      « "bytes, but received "
00232                      « data.size()
00233                      « " bytes!";
00234                   EXC_THROW(os.str());
00235               }
00236
00237               set_start_time(s2000(d, 4));
00238               set_case_vac_at_start_cinhg(u16(d, 8));
00239               set_case_vac_at_end_cinhg(u16(d, 10));
00240               set_average_drift_pressure_counts(u16(d, 12));
00241               set_average_drift_temperature_counts(u16(d, 14));
00242               set_average_drift_salinity_counts(u16(d, 16));
00243               set_num_seek_attempts(u08(d, 18));
00244               set_first_pump_duration_adjustment_s(u16(d, 19));
00245
00246             }
00247
00248               break;
00249
00250             default: { // Unsupported Engineering (Drift) message version.
00251
00252                 ostringstream os;
00253                 os « "Unsupported ALAMO Engineering (Drift) message version: 0x"
00254                    « hex
00255                    « (int) engineering_version()
00256                    « "!";
00257                 EXC_THROW(os.str());
00258             }
```

```
00259
00260                 break;
00261             }
00262             // Decode the message.
00263             set_engineering_version(d[3]);
00264             if(engineering_version() == 0x10) {
00265
00266             }
00267
00268         }
00269         CATCH_BLOCK("Failed to decode ALAMO Engineering (Drift) message!")
00270     }
00271
00272     /** Get the listing entry string for this drift/park engineering data X-Message.
00273      *
00274      *  @param[in]      pg          The pressure sensor sensitivity scale factor, in LSB/dBar.
00275      *  @param[in]      po          The pressure sensor offset, in dBar.
00276      *  @param[in]      sg          The salinity sensor sensitivity scale factor, in LSB/PSU.
00277      *  @param[in]      so          The salinity sensor offset, in PSU.
00278      *  @param[in]      tg          The temperature sensor sensitivity scale factor, in LSB/degC.
00279      *  @param[in]      to          The temperatyre sensor offset, in degC.
00280      *
00281      *  @return         The listing entry string for this drift/park engineering data X-Message.
00282      */
00283     const string GenerateListingString(
00284             uint16_t pg,
00285             uint16_t po,
00286             uint16_t sg,
00287             uint16_t so,
00288             uint16_t tg,
00289             uint16_t to) const {
00290         ostringstream os;
00291
00292         try {
00293
00294             switch(engineering_version()) {
00295
00296             case 0x10: { // Engineering (Drift) version 0x10.
00297
00298                 // Minor Index 0.
00299                 static const format fmt1("#   %5s %4s %4s %6s %20s %7s %7s %20s %7s");
00300                 static const format fmt2("#   %5s %4s %4s %6s %20s %7s %8s %20s %7s");
00301                 static const format fmt3("E08 %05d %4d %4d   0x%-02X %20s %7.2f %7.3f %20s %7.2f");
00302                 os « format(fmt1) % "SN" % "dive" % "midx" % "engVer" % "start-of-drift" % "meanP"
00303                           % "meanT" % "start-of-fall" % "fallP"
00304                    « endl;
00305                 os « format(fmt2) % "" % "" % "" % "" % "" % "(dBar)" % "(degC)" % "" % "(dBar)"
00306                    « endl;
00307                 os « format(fmt3) % id_instrument() % id_dive() % 0 % (int) engineering_version()
00308                         % GenerateIso8601String(start_time()) % average_drift_pressure_dbar(pg, po)
00309                         % average_drift_temperature_c(tg, to)
00310                         % GenerateIso8601String(start_of_fall_to_profile_pressure_time_)
00311                         % start_of_fall_to_profile_pressure_dbar(pg, po);
00312
00313             }
00314                 break;
00315
00316             case 0x11: { // Engineering (Drift) version 0x11.
00317
00318                 // Minor index 0.
00319                 static const format fmt1("#   %5s %4s %4s %6s %20s %9s %9s %15s %15s");
00320                 static const format fmt2("#   %5s %4s %4s %6s %20s %9s %9s %15s %15s");
00321                 static const format fmt3("E08 %05d %4d %4d   0x%-02X %20s %9.2f %9.2f %15d %+15d");
00322                 os « format(fmt1) % "SN" % "dive" % "midx" % "engVer" % "start-of-drift" % "vac0"
00323                           % "vac1" % "seeks" % "seekPumpAdjT"
00324                    « endl;
00325                 os « format(fmt2) % "" % "" % "" % "" % "" % "(inHg)" % "(inHg)" % "" % "(+sec)"
00326                    « endl;
00327                 os « format(fmt3) % id_instrument() % id_dive() % 0 % (int) engineering_version()
00328                           % GenerateIso8601String(start_time()) % case_vac_at_start_inhg()
00329                           % case_vac_at_end_inhg() % (int) num_seek_attempts()
00330                           % first_pump_duration_adjustment_s()
00331                    « endl;
00332
00333                 os « "# --- Drift Sensor Averages ----------------------------------------------"
00334                       "-------------------------"
00335                    « endl;
00336
00337                 // Minor Index 1.
00338                 static const format fmt4("#   %5s %4s %4s    %12s %12s %12s %12s %12s %12s");
00339                 static const format fmt5("#   %5s %4s %4s    %12s %13s %12s %12s %12s %12s");
00340                 static const format fmt6("E08 %05d %4d %4d    %12.3f %12d %12.3f %12d %12.3f %12d");
00341                 os « format(fmt4) % "SN" % "dive" % "midx" % "pres" % "pres" % "temp" % "temp"
00342                           % "saln" % "saln"
00343                    « endl;
00344                 os « format(fmt5) % "" % "" % "" % "(dBar)" % "(counts)" % "(degC)" % "(counts)"
00345                           % "(PSU)" % "(counts)"
00346                    « endl;
00347                 os « format(fmt6) % id_instrument() % id_dive() % 1
00348                         % average_drift_pressure_dbar(pg, po) % average_drift_pressure_counts()
00349                         % average_drift_temperature_c(tg, to) % average_drift_temperature_counts()
00350                         % average_drift_salinity_psu(sg, so) % average_drift_salinity_counts();
00351             }
00352                 break;
00353
00354             case 0x12: { // Engineering (Drift) version 0x12.
00355
00356               // Minor index 0.
00357               static const format fmt1("#   %5s %4s %4s %6s %20s %9s %9s %15s %15s");
00358               static const format fmt2("#   %5s %4s %4s %6s %20s %9s %9s %15s %15s");
00359               static const format fmt3("E08 %05d %4d %4d   0x%-02X %20s %9.2f %9.2f %15d %+15d");
00360               os « format(fmt1) % "SN" % "dive" % "midx" % "engVer" % "start-of-drift" % "vac0"
00361                         % "vac1" % "seeks" % "seekPumpAdjT"
00362                  « endl;
00363               os « format(fmt2) % "" % "" % "" % "" % "" % "(inHg)" % "(inHg)" % "" % "(+sec)"
00364                  « endl;
00365               os « format(fmt3) % id_instrument() % id_dive() % 0 % (int) engineering_version()
00366                         % GenerateIso8601String(start_time()) % case_vac_at_start_inhg()
```

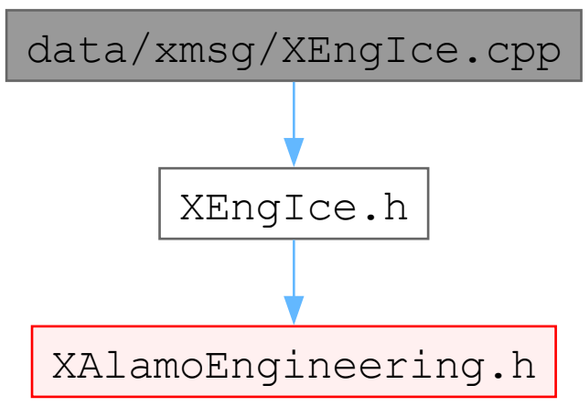
```
00367                         % case_vac_at_end_inhg() % (int) num_seek_attempts()
00368                         % first_pump_duration_adjustment_s()
00369                    « endl;
00370
00371                os « "# --- Drift Sensor Averages ----------------------------------------------"
00372                   "------------------------"
00373                   « endl;
00374
00375                // Minor Index 1.
00376                static const format fmt4("#   %5s %4s %4s    %12s %12s %12s %12s %12s %12s");
00377                static const format fmt5("#   %5s %4s %4s    %12s %13s %12s %12s %12s %12s");
00378                static const format fmt6("E08 %05d %4d %4d    %12.3f %12d %12.3f %12d %12.3f %12d");
00379                os « format(fmt4) % "SN" % "dive" % "midx" % "pres" % "pres" % "temp" % "temp"
00380                         % "saln" % "saln"
00381                   « endl;
00382                os « format(fmt5) % "" % "" % "" % "(dBar)" % "(counts)" % "(degC)" % "(counts)"
00383                         % "(PSU)" % "(counts)"
00384                   « endl;
00385                os « format(fmt6) % id_instrument() % id_dive() % 1
00386                       % average_drift_pressure_dbar(pg, po) % average_drift_pressure_counts()
00387                       % average_drift_temperature_c(tg, to) % average_drift_temperature_counts()
00388                       % average_drift_salinity_psu(sg, so) % average_drift_salinity_counts();
00389             }
00390               break;
00391
00392
00393             default: { // Unsupported Engineering (Drift) version.
00394                 ostringstream os;
00395                 os « "Unsupported ALAMO Engineering (Drift) message version: 0x"
00396                    « hex
00397                    « (int) engineering_version()
00398                    « "!";
00399                 EXC_THROW(os.str());
00400             }
00401                 break;
00402             }
00403
00404         }CATCH_BLOCK("Failed to generate listing string for ALAMO Engineering (Drift) message!")
00405
00406         return os.str();
00407     }
00408
00409     uint16_t average_drift_pressure_counts() const {
00410         return average_drift_pressure_counts_;
00411     }
00412
00413     uint16_t average_drift_salinity_counts() const {
00414         return average_drift_salinity_counts_;
00415     }
00416
00417     uint16_t average_drift_temperature_counts() const {
00418         return average_drift_temperature_counts_;
00419     }
00420
00421     uint16_t case_vac_at_end_cinhg() const {
00422         return case_vac_at_end_cinhg_;
00423     }
00424
00425     template<typename T = float> T case_vac_at_end_inhg() const {
00426         return (T) case_vac_at_end_cinhg() / 100.;
00427     }
00428
00429     template<typename T = float> T case_vac_at_start_inhg() const {
00430         return (T) case_vac_at_start_cinhg() / 100.;
00431     }
00432
00433     uint16_t case_vac_at_start_cinhg() const {
00434         return case_vac_at_start_cinhg_;
00435     }
00436
00437     int16_t first_pump_duration_adjustment_s() const {
00438         return first_pump_duration_adjustment_s_;
00439     }
00440
00441     uint8_t num_seek_attempts() const {
00442         return num_seek_attempts_;
00443     }
00444
00445     uint16_t start_of_fall_to_profile_pressure_counts() const {
00446         return start_of_fall_to_profile_pressure_counts_;
00447     }
00448
00449     const ptime& start_of_fall_to_profile_pressure_time() const {
00450         return start_of_fall_to_profile_pressure_time_;
00451     }
00452
00453     template<typename T = double> T start_of_fall_to_profile_pressure_dbar(
00454             int16_t gain,
00455             int16_t offs) const {
00456         T val;
00457
00458         val = gain ? (T) start_of_fall_to_profile_pressure_counts() / (T) gain - (T) offs :
00459         XMSG_SENS_BAD_PRES;
00460
00461         if(val < XMSG_SENS_MIN_PRES || val > XMSG_SENS_MAX_PRES)
00462             return XMSG_SENS_BAD_PRES;
00463
00464         return val;
00465     }
00466
00467     template<typename T = double> T average_drift_pressure_dbar(int16_t gain, int16_t offs) const {
00468         T val;
00469
00470         val = gain ? (T) average_drift_pressure_counts() / (T) gain - (T) offs : XMSG_SENS_BAD_PRES;
00471
00472         if(val < XMSG_SENS_MIN_PRES || val > XMSG_SENS_MAX_PRES)
00473             return XMSG_SENS_BAD_PRES;
00474
```

```
00475         return val;
00476     }
00477
00478     template<typename T = double> T average_drift_temperature_c(int16_t gain, int16_t offs) const {
00479         T val;
00480
00481         val = gain ? (T) average_drift_temperature_counts() / (T) gain - (T) offs :
00482         XMSG_SENS_BAD_TEMP;
00483
00484         if(val < XMSG_SENS_MIN_TEMP || val > XMSG_SENS_MAX_TEMP)
00485             return XMSG_SENS_BAD_TEMP;
00486
00487         return val;
00488     }
00489
00490     template<typename T = double> T average_drift_salinity_psu(int16_t gain, int16_t offs) const {
00491         T val;
00492
00493         val = gain ? (T) average_drift_salinity_counts() / (T) gain - (T) offs : XMSG_SENS_BAD_SALN;
00494
00495         if(val < XMSG_SENS_MIN_SALN || val > XMSG_SENS_MAX_SALN)
00496             return XMSG_SENS_BAD_SALN;
00497
00498         return val;
00499     }
00500
00501 };
00502
00503 } /* namespace xmsg */
00504 } /* namespace util */
00505 #endif /* XENGDRIFT_H_ */
```

11.119 data/xmsg/XEngIce.cpp File Reference

The util::xmsg::XEngIce class source file.

`#include "XEngIce.h"`
Include dependency graph for XEngIce.cpp:



Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.119.1 Detailed Description

The util::xmsg::XEngIce class source file.

Version

0.6

Date

Mar 22, 2016 4:39:49 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

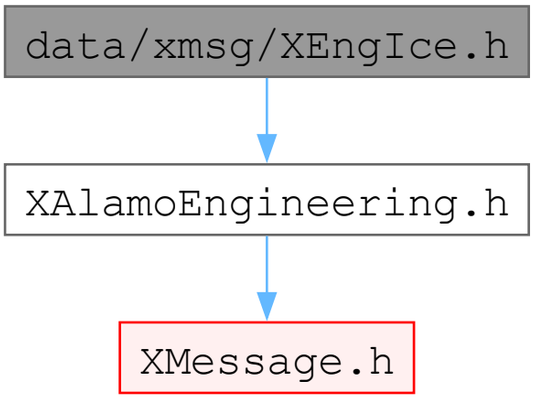
This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.120 data/xmsg/XEngIce.h File Reference

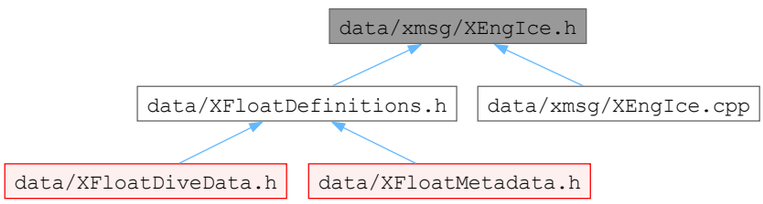
The util::xmsg::XEngIce class header file.

```
#include "XAlamoEngineering.h"
```

Include dependency graph for XEngIce.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XEngIce

  The ALAMO engineering (ice) message class.
- class util::xmsg::XEngIce::XEngIceErrorCode

  The engineering (ice) error code class.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.120.1 Detailed Description

The util::xmsg::XEngIce class header file.

Version

0.6

Date

Mar 22, 2016 4:39:49 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.121 XEngIce.h

Go to the documentation of this file.

```
00001 /** The @ref util::xmsg::XEngIce class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Mar 22, 2016 4:39:49 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef XENGICE_H_
00024 #define XENGICE_H_
00025
00026 /* Includes: */
00027
00028 #include "XAlamoEngineering.h"
00029
00030 /* Definitions: */
00031
00032 namespace util {
00033 namespace xmsg {
00034
00035 /* Types: */
00036
00037 /** The ALAMO engineering (ice) message class.
00038  *
00039  *  An instance of this class represents a decoded ALAMO engineering (ice detection) X-message,
00040  *  which reports the ice detection status, surfacing attempts, threshold test values and any
00041  *  error codes recorded by the instrument.
00042  */
```

```
00043 class XEngIce:
00044         public XAlamoEngineering {
00045
00046
00047     /** The engineering (ice) error code class.
00048      *
00049      *  An instance of this object represents a single error code reported in the ALAMO engineering
00050      *  (ice) data message. Each error consists of a timestamp and a 16-bit error code.
00051      */
00052     class XEngIceErrorCode:
00053             public util::date::DateUtils {
00054
00055     private:
00056
00057         /** The timestamp for the error code.
00058          *
00059          */
00060         ptime date_time_;
00061
00062         /** The error code.
00063          *
00064          */
00065         uint32_t error_code_;
00066
00067     protected:
00068
00069         /** Set the timestamp for the error code.
00070          *
00071          *  @param[in]  date_time   The timestamp for the error code.
00072          */
00073         void set_date_time(const ptime& date_time) {
00074             date_time_ = date_time;
00075         }
00076
00077         /** Set the error code.
00078          *
00079          *  @param[in]  error_code  The error code.
00080          */
00081         void set_error_code(uint32_t error_code) {
00082             error_code_ = error_code;
00083         }
00084
00085     public:
00086
00087         /** The parameterized constructor.
00088          *
00089          *  @param[in]  date_time   The timestamp of the error code.
00090          *  @param[in]  error_code  The error code.
00091          */
00092         XEngIceErrorCode(ptime date_time, uint32_t error_code) :
00093                         date_time_(date_time),
00094                         error_code_(error_code) {
00095             /* Do nothing. */
00096         }
00097
00098         /** The destructor.
00099          *
00100          */
00101         ~XEngIceErrorCode() {
00102             /* Do nothing. */
00103         }
00104
00105         /** Get the timestamp for the error code.
00106          *
00107          *  @return     The timestamp for the error code.
00108          */
00109         const ptime& date_time() const {
00110             return date_time_;
00111         }
00112
00113         /** Generate an ISO8601 format date/time string representation of the error date/time.
00114          *
00115          *  @note           All date/time objects use the UTC time-zone.
00116          *
00117          *  @return         An ISO8601 format date/time string representation of the error
00118          *                  date/time. If @ref date_time() returns
00119          *                  boost::posix_time::not_a_date_time, then the string
00120          *                  "0000-00-00T00:00:00Z" will be returned.
00121          */
00122         string DateTimeAs8601String() const {
00123             return GenerateIso8601String(date_time());
00124         }
00125
00126         /** Get the error code.
00127          *
00128          *  @return     The error code.
00129          */
00130         uint32_t error_code() const {
00131             return error_code_;
00132         }
00133
00134     };
00135
00136 private:
00137
00138     /* Private Data Members: */
00139
00140     uint16_t case_vac_at_end_cinhg_;
00141
00142     uint16_t case_vac_at_start_cinhg_;
00143
00144     uint8_t status_operation_mode_;
00145
00146     uint8_t status_urgency_;
00147
00148     uint8_t status_ice_detection_;
00149
00150     uint8_t num_surface_attempts_;
```

```
00151
00152     uint16_t num_stored_sbd_msgs_;
00153
00154     uint32_t time_since_last_transmission_secs_;
00155
00156     uint16_t pres_min_counts_;
00157
00158     uint16_t temp_threshold_test_;
00159
00160     uint16_t saln_surface_test_;
00161
00162     uint8_t num_errors_;
00163
00164     /** The error code values for this dive, if any.
00165      *
00166      *
00167      */
00168     vector<std::shared_ptr<XEngIceErrorCode> > error_codes_;
00169
00170 public:
00171
00172     /* Public Member Functions: */
00173
00174     /** The parameterized constructor.
00175      *
00176      *  @param[in]      dive        The dive or cycle number of this message.
00177      *  @param[in]      whoi        The unique WHOI identifier.
00178      *  @param[in]      data        The binary X-message data to decode.
00179      *
00180      *  @throw          util::log::MessageException if an decode error occurs.
00181      */
00182     XEngIce(int16_t dive, uint16_t whoi, const vector<uint8_t>& data) :
00183                     XAlamoEngineering(dive, whoi, data),
00184                     error_codes_(),
00185                     num_errors_(0) {
00186
00187         // Decode message.
00188         DecodeXMsgData(data);
00189
00190     }
00191
00192     /** The destructor.
00193      *
00194      */
00195     ~XEngIce() {
00196
00197         /* Do nothing. */
00198
00199     }
00200
00201     /** Decode an engineering (ice) X-message.
00202      *
00203      *  Byte 3 holds the message version. Versions 0x10 and 0x11 share the fields in bytes 4-19:
00204      *  operation mode, ice detection status, urgency, number of surfacing attempts, number of
00205      *  stored SBD messages, time since the last transmission, minimum pressure, temperature
00206      *  threshold test and surface salinity test. Version 0x11 adds error codes; its layout is
00207      *  described in the 0x11 case below.
00208      *
00209      *  @param[in]  data    The binary X-message data to decode.
00210      *
00211      *  @throw      util::log::MessageException if the message type, size or terminator is invalid,
00212      *              the version is unsupported, or (version 0x11) the error count does not match the
00213      *              number of error code entries.
00214      */
00215     void DecodeXMsgData(const vector<uint8_t>& data) {
00216
00217         try {
00218
00219             // Check message.
00220             const uint8_t* d = CheckXMsgData(data, XMSG_TYPE_ID_ENG_ICE, 21);
00221
00222             // Decode message.
00223             set_engineering_version(d[3]);
00224             switch(engineering_version()) {
00225
00226             case 0x10: // Engineering (Ice) message version 0x10.
00227
00228                 set_start_time(not_a_date_time);
00229                 set_status_operation_mode(u08(d, 4));
00230                 set_status_ice_detection(u08(d, 5));
00231                 set_status_urgency(u08(d, 6));
00232                 set_num_surface_attempts(u08(d, 7));
00233                 set_num_stored_sbd_msgs(u16(d, 8));
00234                 set_time_since_last_transmission_secs(u32(d, 10));
00235                 set_pres_min_counts(u16(d, 14));
00236                 set_temp_threshold_test(u16(d, 16));
00237                 set_saln_surface_test(u16(d, 18));
00238
00239                 break;
00240
00241             case 0x11: // TODO Engineering (Ice) message version 0x11.
00242
00243                 /* Message layout (from transmitted messages): bytes 4-19 as in version 0x10,
00244                  * bytes 20-21 an undocumented u16 field (observed values 0-30, apparently a
00245                  * counter; not decoded),
00246                  * byte 22 the number of error codes, then one 6-byte entry per error code from
00247                  * byte 23 (s2000 timestamp, u16 code), then the ';' terminator. A message with no
00248                  * error codes is 24 bytes. */
00249
00250                 // Check message size.
00251                 if(data.size() < 24) {
00252                     ostringstream os;
00253                     os << "Invalid X-Message: Expected at least 24 bytes, but received "
00254                        << data.size()
00255                        << " bytes!";
00256                     EXC_THROW(os.str());
00257                 }
00258
```

```
00259                 // Parse message.
00260                 set_start_time(not_a_date_time);
00261                 set_status_operation_mode(u08(d, 4));
00262                 set_status_ice_detection(u08(d, 5));
00263                 set_status_urgency(u08(d, 6));
00264                 set_num_surface_attempts(u08(d, 7));
00265                 set_num_stored_sbd_msgs(u16(d, 8));
00266                 set_time_since_last_transmission_secs(u32(d, 10));
00267                 set_pres_min_counts(u16(d, 14));
00268                 set_temp_threshold_test(u16(d, 16));
00269                 set_saln_surface_test(u16(d, 18));
00270                 set_num_errors(u08(d, 22));
00271 
00272                 // Parse the error codes (each entry must end before the terminator).
00273                 for(size_t i = 0; 6 * i + 29 < xmsg_size_bytes(); i++)
00274                     error_codes_.push_back(
00275                             std::make_shared<XEngIceErrorCode>(
00276                                     s2000(d, (6 * i) + 23),
00277                                     u16(d, (6 * i) + 27)));
00278 
00279                 // Check the number of error codes.
00280                 if(num_errors() != error_codes().size()) {
00281                     ostringstream os;
00282                     os << "Invalid X-Message: Expected "
00283                        << (int) num_errors()
00284                        << " error codes, but received "
00285                        << error_codes().size()
00286                        << " error codes!";
00287 #                   if defined(XMSG_DEBUG_ENG_SURF) && XMSG_DEBUG_ENG_SURF
00288                     {
00289                         lwrn() << os.str();
00290                     }
00291 #                   else
00292                     {
00293                         EXC_THROW(os.str());
00294                     }
00295 #                   endif
00296                 }
00297 
00298 
00299 
00300                 break;
00301 
00302             default: { // Unsupported Engineering (Ice) message version.
00303 
00304                 ostringstream os;
00305                 os << "Unsupported ALAMO Engineering (Ice) message version: 0x"
00306                    << hex
00307                    << (int) engineering_version()
00308                    << "!";
00309                 EXC_THROW(os.str());
00310             }
00311 
00312             }
00313 
00314         }CATCH_BLOCK("Failed to decode ALAMO Engineering (Ice) message!")
00315 
00316     }
00317 
00318     const string GenerateListingString(
00319             uint16_t pg,
00320             uint16_t po,
00321             uint16_t sg,
00322             uint16_t so,
00323             uint16_t tg,
00324             uint16_t to) const {
00325         ostringstream os;
00326 
00327         (void)tg;           // Unused parameters.
00328         (void)to;
00329 
00330         try {
00331 
00332             switch(engineering_version()) {
00333 
00334             case 0x10: { // Engineering (Ice) version 0x10.
00335 
00336                 // Minor Index 0.
00337                 static const format fmt1("#   %5s %4s %4s "
00338                         "%6s %8s");
00339                 static const format fmt2("#   %5s %4s %4s "
00340                         "%6s %8s");
00341                 static const format fmt3("E0C %05d %4d %4d   "
00342                         "0x%-02X %8d");
00343                 os << format(fmt1) % "SN" % "dive" % "midx" //
00344                         % "engVer" % "mode"  //
00345                    << endl;
00346                 os << format(fmt2) % "" % "" % "" //
00347                         % "" % "" //
00348                    << endl;
00349                 os << format(fmt3) % id_instrument() % id_dive() % 0 // midx
00350                         % (int) engineering_version() % (int) status_operation_mode()
00351                    << endl;
00352 
00353                 // Minor Index 1.
00354                 os << "# --- Status ----------------------------------------------------------------"
00355                    "-------------------------"
00356                    << endl;
00357                 static const format fmt4("#   %5s %4s %4s "
00358                         "%15s %15s %15s %15s %15s");
00359                 static const format fmt5("#   %5s %4s %4s "
00360                         "%15s %15s %15s %15s %15s");
00361                 static const format fmt6("E0C %05d %4d %4d "
00362                         "%15d %15d %15d %15d %15d");
00363                 os << format(fmt4) % "SN" % "dive" % "midx" //
00364                         % "ice-status" % "urgency" % "surfAttempts" % "numStoredSBD" % "lastXmit"
00365                    << endl;
00366                 os << format(fmt5) % "" % "" % ""  //
```

```
00367                         % "" % "" % "" % "" % "(sec)"  //
00368                    « endl;
00369                os « format(fmt6) % id_instrument() % id_dive()
00370                         % 1 // midx
00371                         % (int) status_ice_detection() % (int) status_urgency()
00372                         % (int) num_surface_attempts() % (int) num_stored_sbd_msgs()
00373                         % time_since_last_transmission_secs() //
00374
00375                    « endl;
00376
00377                // Minor Index 2.
00378                static const format fmt7("#   %5s %4s %4s "
00379                        "%15s %15s %15s");
00380                static const format fmt8("#   %5s %4s %4s "
00381                        "%15s %16s %15s");
00382                static const format fmt9("E0C %05d %4d %4d "
00383                        "%15.3f %15.3d %15.3d");
00384                os « format(fmt7) % "SN" % "dive" % "midx" //
00385                         % "minPres" % "testTemp" % "testSalinity"
00386                    « endl;
00387                os « format(fmt8) % "" % "" % ""  //
00388                         % "(dBar)" % "(degC)" % "(PSU)"  //
00389                    « endl;
00390                os « format(fmt9) % id_instrument() % id_dive()
00391                         % 2 // midx
00392                         % GetDecodedPresMin(pg, po) % temp_threshold_test()
00393                         % GetDecodedSalnSurfaceTest(sg, so);
00394            }
00395
00396                break;
00397
00398            case 0x11: { // TODO Engineering (Ice) message version 0x11.
00399
00400              // Minor Index 0.
00401              static const format fmt1("#   %5s %4s %4s "
00402                      "%6s %8s");
00403              static const format fmt2("#   %5s %4s %4s "
00404                      "%6s %8s");
00405              static const format fmt3("E0C %05d %4d %4d   "
00406                      "0x%-02X %8d");
00407              os « format(fmt1) % "SN" % "dive" % "midx" //
00408                         % "engVer" % "mode"  //
00409                 « endl;
00410              os « format(fmt2) % "" % "" % "" //
00411                         % "" % "" //
00412                 « endl;
00413              os « format(fmt3) % id_instrument() % id_dive() % 0 // midx
00414                         % (int) engineering_version() % (int) status_operation_mode()
00415                 « endl;
00416
00417              // Minor Index 1.
00418              os « "# --- Status ----------------------------------------------------------------"
00419                 "-------------------------"
00420                 « endl;
00421              static const format fmt4("#   %5s %4s %4s "
00422                      "%15s %15s %15s %15s %15s");
00423              static const format fmt5("#   %5s %4s %4s "
00424                      "%15s %15s %15s %15s %15s");
00425              static const format fmt6("E0C %05d %4d %4d "
00426                      "%15d %15d %15d %15d %15d");
00427              os « format(fmt4) % "SN" % "dive" % "midx" //
00428                         % "ice-status" % "urgency" % "surfAttempts" % "numStoredSBD" % "lastXmit"
00429                 « endl;
00430              os « format(fmt5) % "" % "" % ""  //
00431                         % "" % "" % "" % "" % "(sec)"  //
00432                 « endl;
00433              os « format(fmt6) % id_instrument() % id_dive()
00434                         % 1 // midx
00435                         % (int) status_ice_detection() % (int) status_urgency()
00436                         % (int) num_surface_attempts() % (int) num_stored_sbd_msgs()
00437                         % time_since_last_transmission_secs() //
00438
00439                 « endl;
00440
00441              // Minor Index 2.
00442              static const format fmt7("#   %5s %4s %4s "
00443                      "%15s %15s %15s");
00444              static const format fmt8("#   %5s %4s %4s "
00445                      "%15s %16s %15s");
00446              static const format fmt9("E0C %05d %4d %4d "
00447                      "%15.3f %15.3d %15.3d");
00448              os « format(fmt7) % "SN" % "dive" % "midx" //
00449                         % "minPres" % "testTemp" % "testSalinity"
00450                 « endl;
00451              os « format(fmt8) % "" % "" % ""  //
00452                         % "(dBar)" % "(degC)" % "(PSU)"  //
00453                 « endl;
00454              os « format(fmt9) % id_instrument() % id_dive()
00455                      % 2 // midx
00456                      % GetDecodedPresMin(pg, po) % temp_threshold_test()
00457                      % GetDecodedSalnSurfaceTest(sg, so) « endl;
00458
00459              // Minor Index 3.
00460              os « "# --- Errors ----------------------------------------------------------------"
00461                 "-------------------------"
00462                 « endl;
00463              static const format fmt13("#   %5s %4s %4s %15s");
00464              static const format fmt14("#   %5s %4s %4s %15s");
00465              static const format fmt15("E0C %05d %4d %4d %15d");
00466              os « format(fmt13) % "SN" % "dive" % "midx" % "nErr" « endl;
00467              os « format(fmt14) % "" % "" % "" % "" « endl;
00468              os « format(fmt15) % id_instrument() % id_dive() % 3// midx
00469                      % (int) num_errors();
00470
00471              // Error codes.
00472              if(!error_codes().size())
00473                  break;
00474              static const format fmt16("#   %5s %4s %4s %20s %15s");
```

```
00475               static const format fmt_err("e0c %05d %4d %4d %20s      0x%08X");
00476
00477               int midx;
00478
00479               if(num_errors()) {
00480                   os « endl « format(fmt16) % "SN" % "dive" % "idx" % "date-time" % "err-code";
00481                   midx = 0;
00482                   for(vector<shared_ptr<XEngIceErrorCode»::const_iterator it = error_codes_.begin();
00483                           it < error_codes_.end(); it++)
00484                       os « endl
00485                          « format(fmt_err) % id_instrument() % id_dive() % midx++
00486                                  % (*it)->DateTimeAs8601String() % (*it)->error_code();
00487               }
00488           }
00489
00490               break;
00491
00492             default: { // Unsupported Engineering (Surface) version.
00493                 ostringstream os;
00494                 os « "Unsupported ALAMO Engineering (Surface) message version: 0x"
00495                    « hex
00496                    « (int) engineering_version()
00497                    « "!";
00498                 EXC_THROW(os.str());
00499             }
00500                 break;
00501
00502             }
00503         }CATCH_BLOCK("Failed to generate listing string for ALAMO Engineering (Ice) message!")
00504
00505         return os.str();
00506     }
00507
00508     template<typename T = float> T GetDecodedCaseVacAtEnd() const {
00509         return (T) case_vac_at_end_cinhg() / (T) 100.;
00510     }
00511
00512     template<typename T = float> T GetDecodedCaseVacAtStart() const {
00513         return (T) case_vac_at_start_cinhg() / (T) 100.;
00514     }
00515
00516     template<typename T = double> T GetDecodedPresMin(int16_t gain, int16_t offs) const {
00517         T val;
00518
00519         val = (T) pres_min_counts() / (T) gain - (T) offs;
00520
00521         if(val < XMSG_SENS_MIN_PRES || val > XMSG_SENS_MAX_PRES)
00522             return XMSG_SENS_BAD_PRES;
00523
00524         return val;
00525     }
00526
00527     template<typename T = double> T GetDecodedTempThresholdTest(int16_t gain, int16_t offs) const {
00528         T val;
00529
00530         val = (T) temp_threshold_test() / (T) gain - (T) offs;
00531
00532         if(val < XMSG_SENS_MIN_TEMP || val > XMSG_SENS_MIN_TEMP)
00533             return XMSG_SENS_BAD_PRES;
00534
00535         return val;
00536     }
00537
00538     template<typename T = double> T GetDecodedSalnSurfaceTest(int16_t gain, int16_t offs) const {
00539         T val;
00540
00541         val = (T) saln_surface_test() / (T) gain - (T) offs;
00542
00543         if(val < XMSG_SENS_MIN_SALN || val > XMSG_SENS_MIN_SALN)
00544             return XMSG_SENS_MIN_SALN;
00545
00546         return val;
00547     }
00548
00549 public:
00550
00551     /* Public Accessor Methods: */
00552
00553     uint16_t case_vac_at_end_cinhg() const {
00554         return case_vac_at_end_cinhg_;
00555     }
00556
00557     uint16_t case_vac_at_start_cinhg() const {
00558         return case_vac_at_start_cinhg_;
00559     }
00560
00561     uint16_t num_stored_sbd_msgs() const {
00562         return num_stored_sbd_msgs_;
00563     }
00564
00565     uint8_t num_surface_attempts() const {
00566         return num_surface_attempts_;
00567     }
00568
00569     uint16_t pres_min_counts() const {
00570         return pres_min_counts_;
00571     }
00572
00573     uint16_t saln_surface_test() const {
00574         return saln_surface_test_;
00575     }
00576
00577     uint8_t status_ice_detection() const {
00578         return status_ice_detection_;
00579     }
00580
00581     uint8_t status_operation_mode() const {
00582         return status_operation_mode_;
```

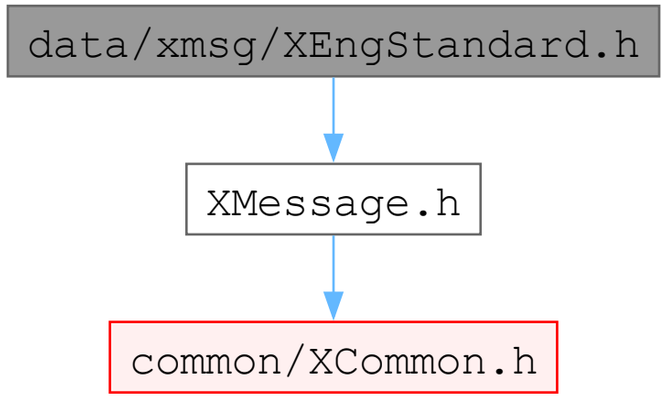
```
00583     }
00584
00585     uint8_t status_urgency() const {
00586         return status_urgency_;
00587     }
00588
00589     uint16_t temp_threshold_test() const {
00590         return temp_threshold_test_;
00591     }
00592
00593     uint32_t time_since_last_transmission_secs() const {
00594         return time_since_last_transmission_secs_;
00595     }
00596
00597
00598     const vector<shared_ptr<XEngIceErrorCode> >& error_codes() const {
00599       return error_codes_;
00600     }
00601
00602     uint8_t num_errors() const {
00603       return num_errors_;
00604     }
00605
00606
00607 protected:
00608
00609     /* Protected Mutator Methods: */
00610
00611     void set_case_vac_at_end_cinhg(uint16_t case_vac_at_end_cinhg) {
00612         case_vac_at_end_cinhg_ = case_vac_at_end_cinhg;
00613     }
00614
00615     void set_case_vac_at_start_cinhg(uint16_t case_vac_at_start_cinhg) {
00616         case_vac_at_start_cinhg_ = case_vac_at_start_cinhg;
00617     }
00618
00619     void set_num_stored_sbd_msgs(uint16_t num_stored_sbd_msgs) {
00620         num_stored_sbd_msgs_ = num_stored_sbd_msgs;
00621     }
00622
00623     void set_num_surface_attempts(uint8_t num_surface_attempts) {
00624         num_surface_attempts_ = num_surface_attempts;
00625     }
00626
00627     void set_pres_min_counts(uint16_t pres_min_counts) {
00628         pres_min_counts_ = pres_min_counts;
00629     }
00630
00631     void set_saln_surface_test(uint16_t saln_surface_test) {
00632         saln_surface_test_ = saln_surface_test;
00633     }
00634
00635     void set_status_ice_detection(uint8_t status_ice_detection) {
00636         status_ice_detection_ = status_ice_detection;
00637     }
00638
00639     void set_status_operation_mode(uint8_t status_operation_mode) {
00640         status_operation_mode_ = status_operation_mode;
00641     }
00642
00643     void set_status_urgency(uint8_t status_urgency) {
00644         status_urgency_ = status_urgency;
00645     }
00646
00647     void set_temp_threshold_test(uint16_t temp_threshold_test) {
00648         temp_threshold_test_ = temp_threshold_test;
00649     }
00650
00651     void set_time_since_last_transmission_secs(uint32_t time_since_last_transmission_secs) {
00652         time_since_last_transmission_secs_ = time_since_last_transmission_secs;
00653     }
00654
00655
00656     void set_num_errors(uint8_t num_errors) {
00657       num_errors_ = num_errors;
00658     }
00659
00660 };
00661
00662 /* Functions: */
00663
00664 } /* namespace xmsg */
00665 } /* namespace util */
00666
00667 #endif /* XENGICE_H_ */
```

11.122 data/xmsg/XEngStandard.h File Reference

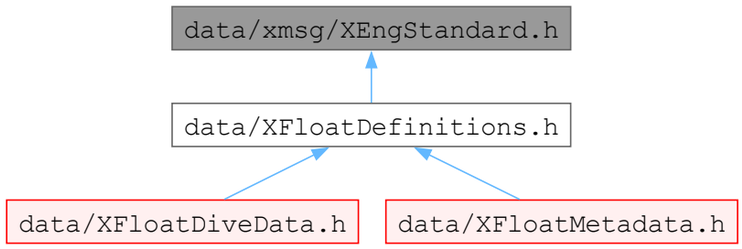
The util::xmsg::XEngStandard class source file.

```
#include "XMessage.h"
```

Include dependency graph for XEngStandard.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XEngStandard

  Class representing an engineering message for a standard dive cycle.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.122.1 Detailed Description

The util::xmsg::XEngStandard class source file.

Version

0.6

Date

Jul 1, 2014 1:59:47 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.123 XEngStandard.h

Go to the documentation of this file.

```
00001 /** The @ref util::xmsg::XEngStandard class source file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jul 1, 2014 1:59:47 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef XENGSTANDARD_H_
00024 #define XENGSTANDARD_H_
00025
00026 /* Includes */
00027
00028 #include "XMessage.h"
00029
00030 /* Definitions */
00031
00032 namespace util {
00033 namespace xmsg {
00034
00035 /* Type Definitions */
00036
00037 /* Class Definitions */
00038
00039 /** Class representing an engineering message for a standard dive cycle.
00040  *
00041  */
00042 class XEngStandard: public XMessage {
00043
00044 public:
00045
00046     /* Member Functions */
00047
00048     /** Parameterized constructor.
00049      *
00050      *  @param[in]      dive        The dive or cycle number of this message.
00051      *  @param[in]      whoi        The unique WHOI identifier.
00052      *  @param[in]      data        The binary X-message data to decode.
00053      *
00054      *  @throw          util::log::MessageException if an decode error occurs.
00055      */
00056     XEngStandard(uint16_t dive, uint16_t whoi, const std::vector<uint8_t>& data) :
00057             XMessage(dive, whoi, data) {
00058         DecodeXMsgData(data); // Decode message.
00059     }
00060
00061     /** Destructor.
00062      *
00063      */
00064     ~XEngStandard() {
00065         /* Do nothing. */
00066     }
00067
00068     /** Decode the standard dive engineering X-Message from the binary data.
00069      *
00070      *  @param[in]      data            The binary message to decode.
00071      */
00072     void DecodeXMsgData(const std::vector<uint8_t>& data) {
00073
00074         try {
00075
00076             // Check message.
00077             /* const uint8_t* d = */ CheckXMsgData(data, XMSG_TYPE_ID_ENG_DIVE, -1);
00078
```

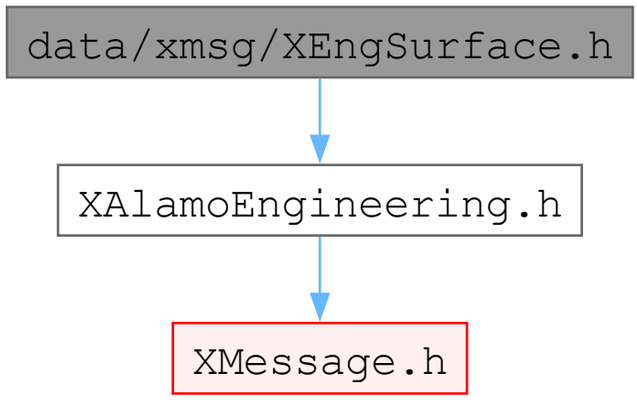
```
00079             /** @todo   Implement the functionality for decoding the Solo-II standard dive/cycle
00080              *          engineering data message.
00081              */
00082
00083         }CATCH_BLOCK("Failed to decode standard dive engineering X-Message!")
00084     }
00085
00086     /** Generate a listing string representation of the standard dive engineering data message.
00087      *
00088      *  @return     A listing string representation of the standard dive engineering data message.
00089      */
00090     const string GenerateListingString() const {
00091         ostringstream os;
00092
00093         try {
00094
00095             /** @todo   Implement the functionality for generating a listing string for the Solo-II
00096              *          standard dive engineering data message.
00097              *
00098              */
00099
00100         } CATCH_BLOCK("Failed to !")
00101
00102         return os.str();
00103     }
00104 };
00105
00106 } /* namespace xmsg */
00107 } /* namespace util */
00108 #endif /* XENGSTANDARD_H_ */
```

11.124 data/xmsg/XEngSurface.h File Reference

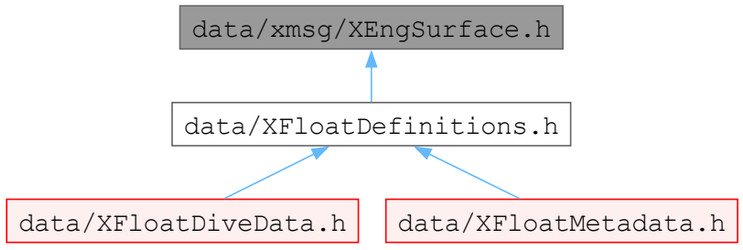
The util::xmsg::XEngSurface class header file.

```
#include "XAlamoEngineering.h"
```

Include dependency graph for XEngSurface.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XEngSurface::XEngErrorCode

  The engineering surface data error code class.
- class util::xmsg::XEngSurface

  Class representing an surface state engineering X-Message.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.124.1 Detailed Description

The util::xmsg::XEngSurface class header file.

Version

0.6

Date

Jul 1, 2014 2:03:16 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.125 XEngSurface.h

Go to the documentation of this file.

```
00001 /** The @ref util::xmsg::XEngSurface class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jul 1, 2014 2:03:16 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef XENGSURFACE_H_
00024 #define XENGSURFACE_H_
00025
00026 /* Includes */
00027
00028 #include "XAlamoEngineering.h"
00029
```

```
00030 /* Definitions */
00031
00032 namespace util {
00033 namespace xmsg {
00034
00035 /* Class Definitions */
00036
00037 /** Class representing an surface state engineering X-Message.
00038  *
00039  *  @note           This X-Message is only used by ALAMO-type instruments.
00040  *  @note           This message type was added in X-Message version 2.1.
00041  */
00042 class XEngSurface:
00043         public XAlamoEngineering {
00044
00045     /** The engineering surface data error code class.
00046      *
00047      *  An instance of this object represents a single error code reported in the ALAMO engineering
00048      *  (surface) data message. Each error consists of a timestamp and a
00049      *
00050      */
00051     class XEngErrorCode:
00052             public util::date::DateUtils {
00053
00054     private:
00055
00056         /** The timestamp for the error code.
00057          *
00058          */
00059         ptime date_time_;
00060
00061         /** The error code.
00062          *
00063          */
00064         uint32_t error_code_;
00065
00066     protected:
00067
00068         /** Set the timestamp for the error code.
00069          *
00070          *  @param[in]  date_time   The timestamp for the error code.
00071          */
00072         void set_date_time(const ptime& date_time) {
00073             date_time_ = date_time;
00074         }
00075
00076         /** Set the error code.
00077          *
00078          *  @param[in]  error_code  The error code.
00079          */
00080         void set_error_code(uint32_t error_code) {
00081             error_code_ = error_code;
00082         }
00083
00084     public:
00085
00086         /** The parameterized constructor.
00087          *
00088          *  @param[in]  date_time   The timestamp of the error code.
00089          *  @param[in]  error_code  The error code.
00090          */
00091         XEngErrorCode(ptime date_time, uint32_t error_code) :
00092                         date_time_(date_time),
00093                         error_code_(error_code) {
00094             /* Do nothing. */
00095         }
00096
00097         /** The destructor.
00098          *
00099          */
00100         ~XEngErrorCode() {
00101             /* Do nothing. */
00102         }
00103
00104         /** Get the timestamp for the error code.
00105          *
00106          *  @return     The timestamp for the error code.
00107          */
00108         const ptime& date_time() const {
00109             return date_time_;
00110         }
00111
00112         /** Generate an ISO8601 format date/time string representation of the error date/time.
00113          *
00114          *  @note           All date/time objects use the UTC time-zone.
00115          *
00116          *  @return         An ISO8601 format date/time string representation of the error
00117          *                  date/time. If @ref date_time() returns
00118          *                  boost::posix_time::not_a_date_time, then the string
00119          *                  "0000-00-00T00:00:00Z" will be returned.
00120          */
00121         string DateTimeAs8601String() const {
00122             return GenerateIso8601String(date_time());
00123         }
00124
00125         /** Get the error code.
00126          *
00127          *  @return     The error code.
00128          */
00129         uint32_t error_code() const {
00130             return error_code_;
00131         }
00132
00133
00134
00135
00136     };
00137
```

```
00138 private:
00139
00140     /* Private Data Members: */
00141
00142     /** The maximum measured air bladder pressure during the air pump, in .001 PSI.
00143      *
00144      *  @note This field was added in version 0x11.
00145      *
00146      */
00147     uint16_t air_bladder_max_pres_mpsi_;
00148
00149     /** The measured air pump current, in mA.
00150      *
00151      *  @note This field was added in version 0x11.
00152      */
00153     uint16_t air_pump_current_ma_;
00154
00155     /** The measured air pump run time, in seconds
00156      *
00157      *  @note This field was added in version 0x11.
00158      */
00159     uint16_t air_pump_runtime_s_;
00160
00161     /** The air valve closed calibration position, in mV.
00162      *
00163      *  @note This field was added in version 0x11.
00164      */
00165     uint16_t air_valve_closed_cal_position_mv_;
00166
00167     /** The air valve mid calibration position, in mV.
00168      *
00169      *  @note This field was added in version 0x11.
00170      */
00171     uint16_t air_valve_mid_cal_position_mv_;
00172
00173     /** The air valve mid position at the end of the surface phase of the dive cycle, in mV.
00174      *
00175      *  @note This field was added in version 0x11.
00176      */
00177     uint16_t air_valve_mid_position_at_end_mv_;
00178
00179     /** The air valve open calibration position, in mV.
00180      *
00181      *  @note This field was added in version 0x11.
00182      */
00183     uint16_t air_valve_open_cal_position_mv_;
00184
00185     /** The air valve open position at the start of the surface phase of the dive cycle, in mV.
00186      *
00187      *  @note This field was added in version 0x11.
00188      */
00189     uint16_t air_valve_open_position_at_start_mv_;
00190
00191     /** The measured system battery voltage, in mV.
00192      *
00193      *  @note This field is only used for message version 0x10. It was removed in version 0x11.
00194      *
00195      *
00196      */
00197     uint16_t battery_voltage_mv_;
00198
00199     /** The measured internal pressure case vacuum, in 0.01 inHg, at the end of the
00200      *  surface phase of the dive cycle.
00201      *
00202      *  @note This field was added in version 0x11.
00203      *
00204      */
00205     uint16_t case_vac_at_end_cinhg_;
00206
00207     /** The measured internal pressure case vacuum, in 0.01 inHg, at the start of the
00208      *  surface phase of the dive cycle.
00209      *
00210      *  @note This field was added in version 0x11.
00211      *
00212      */
00213     uint16_t case_vac_at_start_cinhg_;
00214
00215     /** The measured internal pressure case vacuum, in 0.01 inHg.
00216      *
00217      *  @note This field is only used for message version 0x10. It was removed in version 0x11.
00218      *
00219      *
00220      */
00221     uint16_t case_vac_cingh_;
00222
00223     /** The CTD status code.
00224      *
00225      *  @todo           Define status code values for the ALAMO surface engineering data message.
00226      */
00227     uint32_t ctd_status_code_;
00228
00229     /** The error code values for this dive, if any.
00230      *
00231      *
00232      */
00233     vector<shared_ptr<XEngErrorCode> > error_codes_;
00234
00235     /** The number of error codes reported for this dive cycle.
00236      *
00237      */
00238     uint8_t num_errors_;
00239
00240     /** The number of packets to be transmitted for the current dive.
00241      *
00242      *
00243      */
00244     uint16_t num_sbd_packets_for_dive_;
00245
```

```
00246     /** The number of attempts to transmit in the previous Iridium session.
00247      *
00248      *
00249      */
00250     uint16_t num_transmit_attempts_for_previous_dive_;
00251
00252     /** The oil valve closed calibration position, in mV.
00253      *
00254      */
00255     uint16_t oil_valve_closed_cal_position_mv_;
00256
00257     /** The oil valve closed position at the end of the surface phase of the dive cycle, in mV.
00258      *
00259      */
00260     uint16_t oil_valve_closed_position_at_end_mv_;
00261
00262     /** The oil valve open calibration position at the end of the surface phase of the dive cycle,
00263      *  in mV.
00264      *
00265      */
00266     uint16_t oil_valve_open_cal_position_mv_;
00267
00268     /** The oil valve open position during air pump, in mV.
00269      *
00270      */
00271     uint16_t oil_valve_open_position_during_air_assist_mv_;
00272
00273     /** The measured pressure sensor offset at the surface, in 1/1000 dBar.
00274      *
00275      */
00276     int16_t pressure_offset_reading_;
00277
00278     /** The measured system current, in mA.
00279      *
00280      *  @note This field is only used for message version 0x10. It was removed in version 0x11.
00281      *
00282      *
00283      */
00284     uint16_t system_current_ma_;
00285
00286     /* Suface data sampling: */
00287
00288     ptime surface_sampling_start_time_;
00289
00290     ptime surface_sampling_end_time_;
00291
00292     /* Private Data Members - Fields added in version 0x12: */
00293
00294     int16_t gps_rtc_time_delta_;
00295
00296     /* Private Data Members - Fields added in version 0x13: */
00297
00298     uint16_t prof_raw_sample_first_pres_;
00299
00300     uint16_t prof_raw_sample_first_temp_;
00301
00302     uint16_t prof_raw_sample_first_saln_;
00303
00304     uint16_t prof_raw_sample_last_pres_;
00305
00306     uint16_t prof_raw_sample_last_temp_;
00307
00308     uint16_t prof_raw_sample_last_saln_;
00309
00310     uint16_t prof_num_bins_;
00311
00312     uint16_t prof_num_bins_empty_;
00313
00314     uint16_t prof_num_raw_samples_;
00315
00316
00317 public:
00318
00319     /* Public Member Functions */
00320
00321     /** The parameterized constructor.
00322      *
00323      *  @param[in]      dive        The dive or cycle number of this message.
00324      *  @param[in]      whoi        The unique WHOI identifier.
00325      *  @param[in]      data        The binary X-message data to decode.
00326      *
00327      *  @throw          util::log::MessageException if an decode error occurs.
00328      */
00329 #   pragma GCC diagnostic push
00330 #   pragma GCC diagnostic ignored "-Wreorder"
00331     XEngSurface(int16_t dive, uint16_t whoi, const vector<uint8_t>& data) :
00332                     XAlamoEngineering(dive, whoi, data),
00333                     air_bladder_max_pres_mpsi_(XMSG_BAD_U16),
00334                     air_pump_current_ma_(XMSG_BAD_U16),
00335                     air_pump_runtime_s_(XMSG_BAD_U16),
00336                     air_valve_closed_cal_position_mv_(XMSG_BAD_U16),
00337                     air_valve_mid_cal_position_mv_(XMSG_BAD_U16),
00338                     air_valve_mid_position_at_end_mv_(XMSG_BAD_U16),
00339                     air_valve_open_cal_position_mv_(XMSG_BAD_U16),
00340                     air_valve_open_position_at_start_mv_(XMSG_BAD_U16),
00341                     battery_voltage_mv_(XMSG_BAD_U16),
00342                     case_vac_at_end_cinhg_(XMSG_BAD_U16),
00343                     case_vac_at_start_cinhg_(XMSG_BAD_U16),
00344                     case_vac_cingh_(XMSG_BAD_U16),
00345                     ctd_status_code_(XMSG_BAD_U32),
00346                     error_codes_(),
00347                     num_errors_(XMSG_BAD_U08),
00348                     num_sbd_packets_for_dive_(XMSG_BAD_U16),
00349                     num_transmit_attempts_for_previous_dive_(XMSG_BAD_U16),
00350                     oil_valve_closed_cal_position_mv_(XMSG_BAD_U16),
00351                     oil_valve_closed_position_at_end_mv_(XMSG_BAD_U16),
00352                     oil_valve_open_cal_position_mv_(XMSG_BAD_U16),
00353                     oil_valve_open_position_during_air_assist_mv_(XMSG_BAD_U16),
```

```
00354                   surface_sampling_end_time_(not_a_date_time),
00355                   surface_sampling_start_time_(not_a_date_time),
00356                   system_current_ma_(XMSG_BAD_U16) {
00357
00358         // Decode message.
00359         DecodeXMsgData(data);
00360     }
00361 #   pragma GCC diagnostic pop
00362
00363     /** The destructor.
00364      *
00365      */
00366     ~XEngSurface() {
00367
00368         /* Do nothing. */
00369
00370     }
00371
00372     /** Decode the surface state engineering X-Message from the binary data.
00373      *
00374      *  @param[in]      data            The binary message to decode.
00375      */
00376     void DecodeXMsgData(const vector<uint8_t>& data) {
00377
00378         try {
00379
00380             // Check message.
00381             const uint8_t* d = CheckXMsgData(data, XMSG_TYPE_ID_ENG_SURFACE, 46);
00382
00383             // Decode message.
00384             set_engineering_version(d[3]);
00385             switch(engineering_version()) {
00386
00387             case 0x10: { // Engineering (Surface) message version 0x10.
00388
00389               static const unsigned int MIN_SIZE = 48;
00390
00391               if(data.size() < MIN_SIZE) {
00392                   ostringstream os;
00393                   os << "Invalid X-Message: Expected at least " << MIN_SIZE << " bytes, but "
00394                      <<"received " << data.size() << " bytes!";
00395 #                 if defined(XMSG_DEBUG_ENG_SURF) && XMSG_DEBUG_ENG_SURF
00396                     lwrn() << os.str();
00397 #                 else
00398                     EXC_THROW(os.str());
00399 #                 endif
00400               }
00401
00402               set_start_time(s2000(d, 4));
00403               set_ctd_status_code(u32(d, 8));
00404               set_num_sbd_packets_for_dive(u16(d, 12));
00405               set_num_transmit_attempts_for_previous_dive(u16(d, 14));
00406               set_case_vac_cingh(u16(d, 16));
00407               set_battery_voltage_mv(u16(d, 18));
00408               set_system_current_ma(u16(d, 20));
00409               set_num_errors(u08(d, 46));
00410
00411               // Parse the error codes.
00412               for(size_t i = 0; 4 * i + 48 < xmsg_size_bytes(); i++)
00413                   error_codes_.push_back(
00414                           make_shared<XEngErrorCode>(not_a_date_time, u32(d, (4 * i) + 47)));
00415
00416             }
00417               break;
00418
00419             case 0x11: { // Engineering (Surface) message version 0x11.
00420
00421               static const unsigned int MIN_SIZE = 46;
00422
00423               if(data.size() < MIN_SIZE) {
00424                   ostringstream os;
00425                   os << "Invalid X-Message: Expected at least " << MIN_SIZE << " bytes, but "
00426                      <<"received " << data.size() << " bytes!";
00427 #                 if defined(XMSG_DEBUG_ENG_SURF) && XMSG_DEBUG_ENG_SURF
00428                     lwrn() << os.str();
00429 #                 else
00430                     EXC_THROW(os.str());
00431 #                 endif
00432               }
00433
00434               set_start_time(s2000(d, 4));
00435               set_case_vac_at_start_cinhg(u16(d, 8));
00436               set_case_vac_at_end_cinhg(u16(d, 10));
00437               set_ctd_status_code(u32(d, 12));
00438               set_num_sbd_packets_for_dive(u16(d, 16));
00439               set_num_transmit_attempts_for_previous_dive(u16(d, 18));
00440               set_air_pump_current_ma(u16(d, 20));
00441               set_air_pump_runtime_s(u16(d, 22));
00442               set_air_bladder_max_pres_mpsi(u16(d, 24));
00443               set_air_valve_open_position_at_start_mv(u16(d, 26));
00444               set_air_valve_mid_position_at_end_mv(u16(d, 28));
00445               set_oil_valve_open_position_during_air_assist_mv(u16(d, 30));
00446               set_oil_valve_closed_position_at_end_mv(u16(d, 32));
00447               set_air_valve_open_cal_position_mv(u16(d, 34));
00448               set_air_valve_closed_cal_position_mv(u16(d, 36));
00449               set_air_valve_mid_cal_position_mv(u16(d, 38));
00450               set_oil_valve_open_cal_position_mv(u16(d, 40));
00451               set_oil_valve_closed_cal_position_mv(u16(d, 42));
00452               set_num_errors(u08(d, 44));
00453
00454               // Parse the error codes.
00455               for(size_t i = 0; 6 * i + 50 < xmsg_size_bytes(); i++)
00456                   error_codes_.push_back(
00457                           make_shared<XEngErrorCode>(
00458                                   s2000(d, (6 * i) + 45),
00459                                   u16(d, (6 * i) + 49)));
00460
00461             }
```

```
00462               break;
00463
00464             case 0x12: { // Engineering (Surface) message version 0x12.
00465
00466               static const unsigned int MIN_SIZE = 48;
00467
00468               if(data.size() < MIN_SIZE) {
00469                   ostringstream os;
00470                   os « "Invalid X-Message: Expected at least " « MIN_SIZE « " bytes, but "
00471                      «"received " « data.size() « " bytes!";
00472 #                 if defined(XMSG_DEBUG_ENG_SURF) && XMSG_DEBUG_ENG_SURF
00473                     lwrn() « os.str();
00474 #                 else
00475                     EXC_THROW(os.str());
00476 #                 endif
00477               }
00478
00479               set_start_time(s2000(d, 4));
00480               set_case_vac_at_start_cinhg(u16(d, 8));
00481               set_case_vac_at_end_cinhg(u16(d, 10));
00482               set_ctd_status_code(u32(d, 12 + 2));
00483               set_num_sbd_packets_for_dive(u16(d, 16 + 2));
00484               set_num_transmit_attempts_for_previous_dive(u16(d, 18 + 2));
00485               set_air_pump_current_ma(u16(d, 20 + 2));
00486               set_air_pump_runtime_s(u16(d, 22 + 2));
00487               set_air_bladder_max_pres_mpsi(u16(d, 24 + 2));
00488               set_air_valve_open_position_at_start_mv(u16(d, 26 + 2));
00489               set_air_valve_mid_position_at_end_mv(u16(d, 28 + 2));
00490               set_oil_valve_open_position_during_air_assist_mv(u16(d, 30 + 2));
00491               set_oil_valve_closed_position_at_end_mv(u16(d, 32 + 2));
00492               set_air_valve_open_cal_position_mv(u16(d, 34 + 2));
00493               set_air_valve_closed_cal_position_mv(u16(d, 36 + 2));
00494               set_air_valve_mid_cal_position_mv(u16(d, 38 + 2));
00495               set_oil_valve_open_cal_position_mv(u16(d, 40 + 2));
00496               set_oil_valve_closed_cal_position_mv(u16(d, 42 + 2));
00497               set_pressure_offset_reading(i16(d, 44 + 2));
00498               set_num_errors(u08(d, 46 + 2));
00499
00500               // Parse the error codes.
00501               for(size_t i = 0; 6 * i + 52 + 2 < xmsg_size_bytes(); i++)
00502                   error_codes_.push_back(
00503                           make_shared<XEngErrorCode>(
00504                                   s2000(d, (6 * i) + 47 + 2),
00505                                   u16(d, (6 * i) + 51 + 2)));
00506             }
00507               break;
00508
00509             case 0x13: { // Engineering (Surface) message version 0x13.
00510
00511               static const unsigned int MIN_SIZE = 68;
00512
00513               if(data.size() < MIN_SIZE) {
00514                   ostringstream os;
00515                   os « "Invalid X-Message: Expected at least " « MIN_SIZE « " bytes, but "
00516                      «"received " « data.size() « " bytes!";
00517 #                 if defined(XMSG_DEBUG_ENG_SURF) && XMSG_DEBUG_ENG_SURF
00518                     lwrn() « os.str();
00519 #                 else
00520                     EXC_THROW(os.str());
00521 #                 endif
00522               }
00523
00524               set_start_time(                                    s2000(  d, 4));
00525               set_num_sbd_packets_for_dive(                      u16(    d, 8));
00526               set_num_transmit_attempts_for_previous_dive(       u16(    d, 10));
00527               set_case_vac_at_start_cinhg(                       u16(    d, 12));
00528               set_case_vac_at_end_cinhg(                         u16(    d, 14));
00529               set_gps_rtc_time_delta(                            i16(    d, 16));
00530               set_ctd_status_code(                               u32(    d, 18));
00531               set_prof_raw_sample_first_pres(                    u16(    d, 22));
00532               set_prof_raw_sample_first_temp(                    u16(    d, 24));
00533               set_prof_raw_sample_first_saln(                    u16(    d, 26));
00534               set_prof_raw_sample_last_pres(                     u16(    d, 28));
00535               set_prof_raw_sample_last_temp(                     u16(    d, 30));
00536               set_prof_raw_sample_last_saln(                     u16(    d, 32));
00537               set_prof_num_bins(                                 u16(    d, 34));
00538               set_prof_num_bins_empty(                           u16(    d, 36));
00539               set_prof_num_raw_samples(                          u16(    d, 38));
00540               set_air_pump_current_ma(                           u16(    d, 40));
00541               set_air_pump_runtime_s(                            u16(    d, 42));
00542               set_air_bladder_max_pres_mpsi(                     u16(    d, 44));
00543               set_air_valve_open_position_at_start_mv(           u16(    d, 46));
00544               set_air_valve_mid_position_at_end_mv(              u16(    d, 48));
00545               set_oil_valve_open_position_during_air_assist_mv(  u16(    d, 50));
00546               set_oil_valve_closed_position_at_end_mv(           u16(    d, 52));
00547               set_air_valve_open_cal_position_mv(                u16(    d, 54));
00548               set_air_valve_closed_cal_position_mv(              u16(    d, 56));
00549               set_air_valve_mid_cal_position_mv(                 u16(    d, 58));
00550               set_oil_valve_open_cal_position_mv(                u16(    d, 60));
00551               set_oil_valve_closed_cal_position_mv(              u16(    d, 62));
00552               set_pressure_offset_reading(                       i16(    d, 64));
00553               set_num_errors(                                    u08(    d, 66));
00554
00555               // Parse the error codes.
00556               for(size_t i = 1; 6 * i + 67 < xmsg_size_bytes(); i++)
00557                   error_codes_.push_back(
00558                           make_shared<XEngErrorCode>(            s2000(  d, (6 * i) + 61),
00559                                                                  u16(    d, (6 * i) + 65)));
00560
00561               break;
00562           }
00563
00564             case 0x14: { // Engineering (Surface) message version 0x14.
00565
00566               static const unsigned int MIN_SIZE = 76;
00567
00568               if(data.size() < MIN_SIZE) {
00569                   ostringstream os;
```

```
00570                   os « "Invalid X-Message: Expected at least " « MIN_SIZE « " bytes, but "
00571                      «"received " « data.size() « " bytes!";
00572 #                 if defined(XMSG_DEBUG_ENG_SURF) && XMSG_DEBUG_ENG_SURF
00573                     lwrn() « os.str();
00574 #                 else
00575                     EXC_THROW(os.str());
00576 #                 endif
00577               }
00578
00579               set_start_time(                                    s2000(  d, 4));
00580               set_surface_sampling_start_time(                   s2000(  d, 8));
00581               set_surface_sampling_end_time(                     s2000(  d, 12));
00582               set_num_sbd_packets_for_dive(                      u16(    d, 16));
00583               set_num_transmit_attempts_for_previous_dive(       u16(    d, 18));
00584               set_case_vac_at_start_cinhg(                       u16(    d, 20));
00585               set_case_vac_at_end_cinhg(                         u16(    d, 22));
00586               set_gps_rtc_time_delta(                            i16(    d, 24));
00587               set_ctd_status_code(                               u32(    d, 26));
00588               set_prof_raw_sample_first_pres(                    u16(    d, 30));
00589               set_prof_raw_sample_first_temp(                    u16(    d, 32));
00590               set_prof_raw_sample_first_saln(                    u16(    d, 34));
00591               set_prof_raw_sample_last_pres(                     u16(    d, 36));
00592               set_prof_raw_sample_last_temp(                     u16(    d, 38));
00593               set_prof_raw_sample_last_saln(                     u16(    d, 40));
00594               set_prof_num_bins(                                 u16(    d, 42));
00595               set_prof_num_bins_empty(                           u16(    d, 44));
00596               set_prof_num_raw_samples(                          u16(    d, 46));
00597               set_air_pump_current_ma(                           u16(    d, 48));
00598               set_air_pump_runtime_s(                            u16(    d, 50));
00599               set_air_bladder_max_pres_mpsi(                     u16(    d, 52));
00600               set_air_valve_open_position_at_start_mv(           u16(    d, 54));
00601               set_air_valve_mid_position_at_end_mv(              u16(    d, 56));
00602               set_oil_valve_open_position_during_air_assist_mv(  u16(    d, 58));
00603               set_oil_valve_closed_position_at_end_mv(           u16(    d, 60));
00604               set_air_valve_open_cal_position_mv(                u16(    d, 62));
00605               set_air_valve_closed_cal_position_mv(              u16(    d, 64));
00606               set_air_valve_mid_cal_position_mv(                 u16(    d, 66));
00607               set_oil_valve_open_cal_position_mv(                u16(    d, 68));
00608               set_oil_valve_closed_cal_position_mv(              u16(    d, 70));
00609               set_pressure_offset_reading(                       i16(    d, 72));
00610               set_num_errors(                                    u08(    d, 74));
00611
00612               // Parse the error codes.
00613               for(size_t i = 0; 6 * i + 80 < xmsg_size_bytes(); i++)
00614                   error_codes_.push_back(
00615                           make_shared<XEngErrorCode>(            s2000(  d, (6 * i) + 75),
00616                                                                  u16(    d, (6 * i) + 79)));
00617
00618               break;
00619             }
00620
00621             default: {
00622
00623               /* Handle unsupported Engineering (Surface) message version: */
00624
00625               ostringstream os;
00626               os « "Unsupported ALAMO Engineering (Surface) message version: 0x" « hex
00627                  « (int) engineering_version() « "!";
00628               EXC_THROW(os.str());
00629               }
00630
00631               break;
00632             }
00633
00634             // Check the number of error codes.
00635             if(num_errors() != error_codes().size()) {
00636                 ostringstream os;
00637                 os « "Invalid X-Message: Expected " « (int) num_errors() « " error codes, "
00638                    « "but received " « error_codes().size() « " error codes!";
00639 #               if defined(XMSG_DEBUG_ENG_SURF) && XMSG_DEBUG_ENG_SURF
00640                   lwrn() « os.str();
00641 #               else
00642                   EXC_THROW(os.str());
00643 #               endif
00644             }
00645
00646         }CATCH_BLOCK("Failed to decode ALAMO Engineering (Surface) message!")
00647     }
00648
00649     /** Generate the listing entry string for this surface engineering data X-Message.
00650      *
00651      *  @return         The listing entry string for this surface engineering data X-Message.
00652      */
00653     const string GenerateListingString(
00654             uint16_t pg,
00655             uint16_t po,
00656             uint16_t sg,
00657             uint16_t so,
00658             uint16_t tg,
00659             uint16_t to) const {
00660         ostringstream os;
00661
00662
00663
00664         try {
00665
00666             switch(engineering_version()) {
00667
00668             case 0x10: { // Engineering (Surface) version 0x10.
00669
00670                 // Minor index 0.
00671                 static const format fmt1("#   %5s %4s %4s %6s %20s %10s %5s %6s %6s %6s %5s %4s");
00672                 static const format fmt2("#   %5s %4s %4s %6s %20s %10s %5s %6s %7s %6s %5s %4s");
00673                 static const format fmt3(
00674                         "E0A %05d %4d %4d   0x%-02X %20s 0x%08X %5d %6d %6.2f %6.3f %5.3f %4d");
00675
00676                 os « format(fmt1) % "SN" % "dive" % "midx" % "engVer" % "start-of-surface"
00677                           % "CTD-status" % "nSBD" % "nAtmpt" % "vac" % "sysV" % "sysI" % "nErr"
```

```
00678                   « endl;
00679               os « format(fmt2) % "" % "" % "" % "" % "" % "" % "" % "" % "(inHg)" % "(V)"
00680                          % "(A)" % ""
00681                   « endl;
00682               os « format(fmt3) % id_instrument() % id_dive() % 0 % (int) engineering_version()
00683                      % GenerateIso8601String(start_time()) % ctd_status_code()
00684                      % num_sbd_packets_for_dive() % num_transmit_attempts_for_previous_dive()
00685                      % case_vacuum_inhg() % battery_voltage_v() % system_current_a()
00686                      % num_errors();
00687
00688               // Error codes.
00689               static const format fmt4("#   %5s %4s %4s %20s %15s");
00690               static const format fmt_err("e0A %05d %4d %4d %20s      0x%08X");
00691               if(num_errors()) {
00692                   os « endl « format(fmt4) % "SN" % "dive" % "idx" % "err-code";
00693                   int midx = 0;
00694                   for(vector<shared_ptr<XEngErrorCode»::const_iterator it = error_codes_.begin();
00695                           it < error_codes_.end(); it++)
00696                       os « endl
00697                          « format(fmt_err) % id_instrument() % id_dive() % midx++
00698                                  % (*it)->DateTimeAs8601String() % (*it)->error_code();
00699               }
00700           }
00701               break;
00702
00703           case 0x11: { // Engineering (Surface) version 0x11.
00704
00705               // Minor Index 0.
00706               static const format fmt1("#   %5s %4s %4s %6s %20s %6s %6s %6s %12s");
00707               static const format fmt2("#   %5s %4s %4s %6s %20s %6s %6s %6s %12s");
00708               static const format fmt3("E0A %05d %4d %4d   0x%-02X %20s %6.2f %6.2f %6d %12d");
00709               os « format(fmt1) % "SN" % "dive" % "midx" % "engVer" % "start-of-surface" % "vac0"
00710                          % "vac1" % "nSBD" % "nPrevAttempt"
00711                  « endl;
00712               os « format(fmt2) % "" % "" % "" % "" % "" % "(inHg)" % "(inHg)" % "" % "" « endl;
00713               os « format(fmt3) % id_instrument() % id_dive() % 0 % (int) engineering_version()
00714                          % GenerateIso8601String(start_time()) % case_vac_at_start_inhg()
00715                          % case_vac_at_end_inhg() % num_sbd_packets_for_dive()
00716                          % num_transmit_attempts_for_previous_dive()
00717                  « endl;
00718
00719               // Minor Index 1.
00720               os « "# --- Air Pump & Bladder ------------------------------------------------"
00721                  "-------------------------"
00722                  « endl;
00723               static const format fmt4("#   %5s %4s %4s %20s %20s %20s");
00724               static const format fmt5("#   %5s %4s %4s %20s %20s %20s");
00725               static const format fmt6("E0A %05d %4d %4d %20.3f %20d %20.3f");
00726               os « format(fmt4) % "SN" % "dive" % "midx" % "APCur" % "APRunT" % "AirBladMxP"
00727                  « endl;
00728               os « format(fmt5) % "" % "" % "" % "(A)" % "(sec)" % "(PSI)" « endl;
00729               os « format(fmt6) % id_instrument() % id_dive() % 1 % air_pump_current_a()
00730                          % air_pump_runtime_s() % air_bladder_max_pres_psi()
00731                  « endl;
00732
00733               // Minor Index 2.
00734               os « "# --- Air Valve Position ------------------------------------------------"
00735                  "-------------------------"
00736                  « endl;
00737               static const format fmt7("#   %5s %4s %4s %15s %15s %15s %15s %15s");
00738               static const format fmt8("#   %5s %4s %4s %15s %15s %15s %15s %15s");
00739               static const format fmt9("E0A %05d %4d %4d %15.3f %15.3f %15.3f %15.3f %15.3f");
00740               os « format(fmt7) % "SN" % "dive" % "midx" % "AVOpenStart" % "AVMidEnd"
00741                          % "ACVpenCal" % "AVMidCal" % "AVCloseCal"
00742                  « endl;
00743               os « format(fmt8) % "" % "" % "" % "(V)" % "(V)" % "(V)" % "(V)" % "(V)" « endl;
00744               os « format(fmt9) % id_instrument() % id_dive() % 2
00745                          % air_valve_open_position_at_start_v()
00746                          % air_valve_mid_position_at_end_v() % air_valve_open_cal_position_v()
00747                          % air_valve_mid_cal_position_v() % air_valve_closed_cal_position_v()
00748                  « endl;
00749
00750               // Minor Index 3.
00751               os « "# --- Oil Valve Position ------------------------------------------------"
00752                  "-------------------------"
00753                  « endl;
00754               static const format fmt10("#   %5s %4s %4s %15s %15s %15s %15s");
00755               static const format fmt11("#   %5s %4s %4s %15s %15s %15s %15s");
00756               static const format fmt12("E0A %05d %4d %4d %15.3f %15.3f %15.3f %15.3f");
00757               os « format(fmt10) % "SN" % "dive" % "midx" % "OVOpenAirPmp" % "OVCloseEnd"
00758                          % "OVOpenCal" % "OVClosCal"
00759                  « endl;
00760               os « format(fmt11) % "" % "" % "" % "(V)" % "(V)" % "(V)" % "(V)" « endl;
00761               os « format(fmt12) % id_instrument() % id_dive() % 3
00762                          % oil_valve_open_position_during_air_assist_v()
00763                          % oil_valve_closed_position_at_end_v() % oil_valve_open_cal_position_v()
00764                          % oil_valve_closed_cal_position_v()
00765                  « endl;
00766
00767               // Minor Index 4.
00768               os « "# --- Errors ------------------------------------------------------------"
00769                  "-------------------------"
00770                  « endl;
00771               static const format fmt13("#   %5s %4s %4s %15s");
00772               static const format fmt14("#   %5s %4s %4s %15s");
00773               static const format fmt15("e0A %05d %4d %4d %15d");
00774               os « format(fmt13) % "SN" % "dive" % "midx" % "nErr" « endl;
00775               os « format(fmt14) % "" % "" % "" % "" « endl;
00776               os « format(fmt15) % id_instrument() % id_dive() % 4 % (int) num_errors();
00777
00778               // Error codes.
00779               if(!error_codes().size())
00780                   break;
00781               static const format fmt16("#   %5s %4s %4s %20s %15s");
00782               static const format fmt_err("e0A %05d %4d %4d %20s      0x%08X");
00783
00784               if(num_errors()) {
00785                   os « endl « format(fmt16) % "SN" % "dive" % "idx" % "date-time" % "err-code";
```

```
00786                     int midx = 0;
00787                     for(vector<shared_ptr<XEngErrorCode»::const_iterator it = error_codes_.begin();
00788                             it < error_codes_.end(); it++)
00789                         os « endl
00790                            « format(fmt_err) % id_instrument() % id_dive() % midx++
00791                                    % (*it)->DateTimeAs8601String() % (*it)->error_code();
00792                 }
00793             }
00794                 break;
00795
00796             case 0x12: { // Engineering (Surface) version 0x12.
00797
00798                 // Minor Index 0.
00799                 static const format fmt1("#   %5s %4s %4s %6s %20s %9s %9s %15s %15s");
00800                 static const format fmt2("#   %5s %4s %4s %6s %20s %9s %9s %15s %15s");
00801                 static const format fmt3("E0A %05d %4d %4d   0x%-02X %20s %9.2f %9.2f %15d %15d");
00802                 os « format(fmt1) % "SN" % "dive" % "midx" % "engVer" % "start-of-surface" % "vac0"
00803                            % "vac1" % "nSBD" % "nPrevAttempt"
00804                    « endl;
00805                 os « format(fmt2) % "" % "" % "" % "" % "" % "(inHg)" % "(inHg)" % "" % "" « endl;
00806                 os « format(fmt3) % id_instrument() % id_dive() % 0 % (int) engineering_version()
00807                            % GenerateIso8601String(start_time()) % case_vac_at_start_inhg()
00808                            % case_vac_at_end_inhg() % num_sbd_packets_for_dive()
00809                            % num_transmit_attempts_for_previous_dive()
00810                    « endl;
00811
00812                 // Minor Index 1.
00813                 os « "# --- Air Pump & Bladder ------------------------------------------------"
00814                    "-------------------------"
00815                    « endl;
00816                 static const format fmt4("#   %5s %4s %4s %15s %15s %15s");
00817                 static const format fmt5("#   %5s %4s %4s %15s %15s %15s");
00818                 static const format fmt6("E0A %05d %4d %4d %15.3f %15d %15.3f");
00819                 os « format(fmt4) % "SN" % "dive" % "midx" % "APCur" % "APRunT" % "AirBladMxP"
00820                    « endl;
00821                 os « format(fmt5) % "" % "" % "" % "(A)" % "(sec)" % "(PSI)" « endl;
00822                 os « format(fmt6) % id_instrument() % id_dive() % 1 % air_pump_current_a()
00823                            % air_pump_runtime_s() % air_bladder_max_pres_psi()
00824                    « endl;
00825
00826                 // Minor Index 2.
00827                 os « "# --- Air Valve Position ------------------------------------------------"
00828                    "-------------------------"
00829                    « endl;
00830                 static const format fmt7("#   %5s %4s %4s %15s %15s %15s %15s %15s");
00831                 static const format fmt8("#   %5s %4s %4s %15s %15s %15s %15s %15s");
00832                 static const format fmt9("E0A %05d %4d %4d %15.3f %15.3f %15.3f %15.3f %15.3f");
00833                 os « format(fmt7) % "SN" % "dive" % "midx" % "AVOpenStart" % "AVMidEnd"
00834                            % "ACVpenCal" % "AVMidCal" % "AVCloseCal"
00835                    « endl;
00836                 os « format(fmt8) % "" % "" % "" % "(V)" % "(V)" % "(V)" % "(V)" % "(V)" « endl;
00837                 os « format(fmt9) % id_instrument() % id_dive() % 2
00838                            % air_valve_open_position_at_start_v()
00839                            % air_valve_mid_position_at_end_v() % air_valve_open_cal_position_v()
00840                            % air_valve_mid_cal_position_v() % air_valve_closed_cal_position_v()
00841                    « endl;
00842
00843                 // Minor Index 3.
00844                 os « "# --- Oil Valve Position -------------------------------------------------"
00845                    "-------------------------"
00846                    « endl;
00847                 static const format fmt10("#   %5s %4s %4s %15s %15s %15s %15s");
00848                 static const format fmt11("#   %5s %4s %4s %15s %15s %15s %15s");
00849                 static const format fmt12("E0A %05d %4d %4d %15.3f %15.3f %15.3f %15.3f");
00850                 os « format(fmt10) % "SN" % "dive" % "midx" % "OVOpenAirPmp" % "OVCloseEnd"
00851                            % "OVOpenCal" % "OVClosCal"
00852                    « endl;
00853                 os « format(fmt11) % "" % "" % "" % "(V)" % "(V)" % "(V)" % "(V)" « endl;
00854                 os « format(fmt12) % id_instrument() % id_dive() % 3
00855                            % oil_valve_open_position_during_air_assist_v()
00856                            % oil_valve_closed_position_at_end_v() % oil_valve_open_cal_position_v()
00857                            % oil_valve_closed_cal_position_v()
00858                    « endl;
00859
00860                 // Minor Index 4.
00861                 os « "# --- Surface Pressure ------------------------------------------------"
00862                    "-------------------------"
00863                    « endl;
00864                 static const format fmt17("#   %5s %4s %4s "
00865                         "%15s");
00866                 static const format fmt18("#   %5s %4s %4s "
00867                         "%15s");
00868                 static const format fmt19("E0A %05d %4d %4d "
00869                         "%15.3f");
00870                 os « format(fmt17) % "SN" % "dive" % "midx" //
00871                            % "presOffset" //
00872                    « endl;
00873                 os « format(fmt18) % "" % "" % "" //
00874                            % "(dBar)"
00875                    « endl;
00876                 os « format(fmt19) % id_instrument() % id_dive() % 4 // midx
00877                         % GetDecodedBitPressureOffset() //
00878                    « endl;
00879
00880                 // Minor Index 5.
00881                 os « "# --- Errors ----------------------------------------------------------------"
00882                    "-------------------------"
00883                    « endl;
00884                 static const format fmt13("#   %5s %4s %4s %15s");
00885                 static const format fmt14("#   %5s %4s %4s %15s");
00886                 static const format fmt15("e0A %05d %4d %4d %15d");
00887                 os « format(fmt13) % "SN" % "dive" % "midx" % "nErr" « endl;
00888                 os « format(fmt14) % "" % "" % "" % "" « endl;
00889                 os « format(fmt15) % id_instrument() % id_dive() % 5 // midx
00890                         % (int) num_errors();
00891
00892                 // Error codes.
00893                 if(!error_codes().size())
```

```
00894                   break;
00895               static const format fmt16("#   %5s %4s %4s %20s %15s");
00896               static const format fmt_err("e0A %05d %4d %4d %20s      0x%08X");
00897
00898               if(num_errors()) {
00899                   os « endl « format(fmt16) % "SN" % "dive" % "idx" % "date-time" % "err-code";
00900                   int midx = 0;
00901                   for(vector<shared_ptr<XEngErrorCode»::const_iterator it = error_codes_.begin();
00902                           it < error_codes_.end(); it++)
00903                       os « endl
00904                          « format(fmt_err) % id_instrument() % id_dive() % midx++
00905                                  % (*it)->DateTimeAs8601String() % (*it)->error_code();
00906               }
00907           }
00908               break;
00909
00910           case 0x13: { // Engineering (Surface) version 0x13.
00911
00912               unsigned int midx = 0;
00913
00914               {   static const format fmt1("#   %5s %4s %4s %6s %20s %9s %9s %15s %15s");
00915                   static const format fmt2("#   %5s %4s %4s %6s %20s %9s %9s %15s %15s");
00916                   static const format fmt3(
00917                           "E0A %05d %4d %4d   0x%-02X %20s %9.2f %9.2f %15d %15d");
00918                   os « format(fmt1) % "SN" % "dive" % "midx" % "engVer" % "start-of-surface"
00919                              % "vac0" % "vac1" % "nSBD" % "nPrevAttempt"
00920                      « endl;
00921                   os « format(fmt2) % "" % "" % "" % "" % "" % "(inHg)" % "(inHg)" % "" % ""
00922                      « endl;
00923                   os « format(fmt3) % id_instrument() % id_dive() % midx++
00924                              % (int) engineering_version() % GenerateIso8601String(start_time())
00925                              % case_vac_at_start_inhg() % case_vac_at_end_inhg()
00926                              % num_sbd_packets_for_dive()
00927                              % num_transmit_attempts_for_previous_dive()
00928                      « endl;
00929               }
00930
00931               os « "# --- Air Pump & Bladder ------------------------------------------------"
00932                  "-------------------------"
00933                  « endl;
00934               static const format fmt4("#   %5s %4s %4s %15s %15s %15s");
00935               static const format fmt5("#   %5s %4s %4s %15s %15s %15s");
00936               static const format fmt6("E0A %05d %4d %4d %15.3f %15d %15.3f");
00937               os « format(fmt4) % "SN" % "dive" % "midx" % "APCur" % "APRunT" % "AirBladMxP"
00938                  « endl;
00939               os « format(fmt5) % "" % "" % "" % "(A)" % "(sec)" % "(PSI)" « endl;
00940               os « format(fmt6) % id_instrument() % id_dive() % midx++ % air_pump_current_a()
00941                          % air_pump_runtime_s() % air_bladder_max_pres_psi()
00942                  « endl;
00943
00944               os « "# --- Air Valve Position ----------------------------------------------------"
00945                  "-------------------------"
00946                  « endl;
00947               static const format fmt7("#   %5s %4s %4s %15s %15s %15s %15s %15s");
00948               static const format fmt8("#   %5s %4s %4s %15s %15s %15s %15s %15s");
00949               static const format fmt9("E0A %05d %4d %4d %15.3f %15.3f %15.3f %15.3f %15.3f");
00950               os « format(fmt7) % "SN" % "dive" % "midx" % "AVOpenStart" % "AVMidEnd"
00951                          % "ACVpenCal" % "AVMidCal" % "AVCloseCal"
00952                  « endl;
00953               os « format(fmt8) % "" % "" % "" % "(V)" % "(V)" % "(V)" % "(V)" % "(V)" « endl;
00954               os « format(fmt9) % id_instrument() % id_dive() % midx++
00955                          % air_valve_open_position_at_start_v()
00956                          % air_valve_mid_position_at_end_v() % air_valve_open_cal_position_v()
00957                          % air_valve_mid_cal_position_v() % air_valve_closed_cal_position_v()
00958                  « endl;
00959
00960               os « "# --- Oil Valve Position ------------------------------------------------"
00961                  "-------------------------"
00962                  « endl;
00963               static const format fmt10("#   %5s %4s %4s %15s %15s %15s %15s");
00964               static const format fmt11("#   %5s %4s %4s %15s %15s %15s %15s");
00965               static const format fmt12("E0A %05d %4d %4d %15.3f %15.3f %15.3f %15.3f");
00966               os « format(fmt10) % "SN" % "dive" % "midx" % "OVOpenAirPmp" % "OVCloseEnd"
00967                          % "OVOpenCal" % "OVClosCal"
00968                  « endl;
00969               os « format(fmt11) % "" % "" % "" % "(V)" % "(V)" % "(V)" % "(V)" « endl;
00970               os « format(fmt12) % id_instrument() % id_dive() % midx++
00971                          % oil_valve_open_position_during_air_assist_v()
00972                          % oil_valve_closed_position_at_end_v() % oil_valve_open_cal_position_v()
00973                          % oil_valve_closed_cal_position_v()
00974                  « endl;
00975
00976               os « "# --- Surface Pressure ------------------------------------------------"
00977                  "-------------------------"
00978                  « endl;
00979               static const format fmt17("#   %5s %4s %4s "
00980                       "%15s");
00981               static const format fmt18("#   %5s %4s %4s "
00982                       "%15s");
00983               static const format fmt19("E0A %05d %4d %4d "
00984                       "%15.3f");
00985               os « format(fmt17) % "SN" % "dive" % "midx" //
00986                          % "presOffset" //
00987                  « endl;
00988               os « format(fmt18) % "" % "" % "" //
00989                          % "(dBar)"
00990                  « endl;
00991               os « format(fmt19) % id_instrument() % id_dive() % midx++ // midx
00992                      % GetDecodedBitPressureOffset() //
00993                  « endl;
00994
00995               {   os « "# --- Profile - First Sample ---------------------------------------"
00996                       "-----------------------------"
00997                      « endl;
00998                   static const format fmt1(
00999                       "#   %5s %4s %4s "
01000                       "   %12s %12s "
01001                       "%12s %12s "
```

```
01002                       "%12s %12s");
01003                   static const format fmt2(
01004                       "#   %5s %4s %4s "
01005                       "   %12s %12s "
01006                       "%13s %12s "
01007                       "%12s %12s");
01008                   static const format fmt3(
01009                       "E0A %05d %4d %4d "
01010                       "   %12.3f %12d "
01011                       "%12.3f %12d "
01012                       "%12.3f %12d");
01013                   os « format(fmt1)
01014                       % "SN"          % "dive"        % "midx"
01015                       % "firstPres"   % "firstPres"
01016                       % "firstTemp"   % "firstTemp"
01017                       % "firstSaln"   % "firstSaln"
01018                       « endl;
01019                   os « format(fmt2)
01020                       % ""            % ""            % ""
01021                       % "(dBar)"      % "(counts)"
01022                       % "(degC)"        % "(counts)"
01023                       % "(PSU)"       % "(counts)"
01024                       « endl;
01025                   os « format(fmt3)
01026                       % id_instrument() % id_dive() % midx++
01027                       % prof_raw_sample_first_pres_dbar(pg, po) % prof_raw_sample_first_pres_counts()
01028                       % prof_raw_sample_first_temp_degc(tg, to) % prof_raw_sample_first_temp_counts()
01029                       % prof_raw_sample_first_saln_psu(sg, so)  % prof_raw_sample_first_saln_counts()
01030                       « endl;
01031                   os « "# --- Profile - Last Sample ----------------------------------------"
01032                       "-----------------------------"
01033                       « endl;
01034                   os « format(fmt1)
01035                       % "SN"          % "dive"        % "midx"
01036                       % "lastPres"    % "lastPres"
01037                       % "lastTemp"    % "lastTemp"
01038                       % "lastSaln"    % "lastSaln"
01039                       « endl;
01040                   os « format(fmt2)
01041                       % ""            % ""            % ""
01042                       % "(dBar)"      % "(counts)"
01043                       % "(degC)"        % "(counts)"
01044                       % "(PSU)"       % "(counts)"
01045                       « endl;
01046                   os « format(fmt3)
01047                       % id_instrument() % id_dive() % midx++
01048                       % prof_raw_sample_last_pres_dbar(pg, po) % prof_raw_sample_last_pres_counts()
01049                       % prof_raw_sample_last_temp_degc(tg, to) % prof_raw_sample_last_temp_counts()
01050                       % prof_raw_sample_last_saln_psu(sg, so)  % prof_raw_sample_last_saln_counts()
01051                       « endl;
01052               }
01053
01054               {   os « "# --- Profile - Meta ------------------------------------------------"
01055                         "-----------------------------"
01056                       « endl;
01057                   static const format fmt1(
01058                       "#   %5s %4s %4s "
01059                       "%15s %15s %15s");
01060                   static const format fmt2(
01061                       "E0A %05d %4d %4d "
01062                       "%15d %15d %15d");
01063                   os « format(fmt1)
01064                        % "SN"                % "dive"               % "midx"
01065                        % "numBins"           % "numEmptyBins"       % "numRawSamples"
01066                        « endl;
01067                   os « format(fmt2)
01068                        % id_instrument()     % id_dive()            % midx++
01069                        % prof_num_bins()     % prof_num_bins_empty() % prof_num_raw_samples()
01070                        « endl;
01071               }
01072
01073               {   os « "# --- Real-Time Clock-----------------------------------------------"
01074                         "-----------------------------"
01075                       « endl;
01076                   static const format fmt1(
01077                       "#   %5s %4s %4s "
01078                       "%15s");
01079                   static const format fmt2(
01080                       "#   %5s %4s %4s "
01081                       "%15s");
01082                   static const format fmt3(
01083                       "E0A %05d %4d %4d "
01084                       "%15d");
01085                   os « format(fmt1)
01086                        % "SN"                % "dive"               % "midx"
01087                        % "gpsOffset"
01088                        « endl;
01089                   os « format(fmt2)
01090                        % ""                  % ""                  % ""
01091                        % "(sec)"
01092                        « endl;
01093                   os « format(fmt3)
01094                        % id_instrument()     % id_dive()            % midx++
01095                        % gps_rtc_time_delta()
01096                        « endl;
01097               }
01098
01099               // Minor Index 8.
01100               os « "# --- Errors ------------------------------------------------------------"
01101                  "-------------------------"
01102                  « endl;
01103               static const format fmt13("#   %5s %4s %4s %15s");
01104               static const format fmt14("#   %5s %4s %4s %15s");
01105               static const format fmt15("E0A %05d %4d %4d %15d");
01106               os « format(fmt13) % "SN" % "dive" % "midx" % "nErr" « endl;
01107               os « format(fmt14) % "" % "" % "" % "" « endl;
01108               os « format(fmt15) % id_instrument() % id_dive() % midx++ // midx
01109                       % (int) num_errors();
```

```
01110
01111                 // Error codes.
01112                 if(!error_codes().size())
01113                     break;
01114                 static const format fmt16("#   %5s %4s %4s %20s %15s");
01115                 static const format fmt_err("e0A %05d %4d %4d %20s      0x%08X");
01116
01117                 if(num_errors()) {
01118                     os « endl « format(fmt16) % "SN" % "dive" % "idx" % "date-time" % "err-code";
01119                     midx = 0;
01120                     for(vector<shared_ptr<XEngErrorCode»::const_iterator it = error_codes_.begin();
01121                             it < error_codes_.end(); it++)
01122                         os « endl
01123                            « format(fmt_err) % id_instrument() % id_dive() % midx++
01124                                    % (*it)->DateTimeAs8601String() % (*it)->error_code();
01125                 }
01126             }
01127                 break;
01128
01129             case 0x14: { // Engineering (Surface) version 0x14.
01130
01131                 unsigned int midx = 0;
01132
01133                 {   static const format fmt1("#   %5s %4s %4s %6s %20s %9s %9s %15s %15s");
01134                     static const format fmt2("#   %5s %4s %4s %6s %20s %9s %9s %15s %15s");
01135                     static const format fmt3(
01136                              "E0A %05d %4d %4d   0x%-02X %20s %9.2f %9.2f %15d %15d");
01137                     os « format(fmt1) % "SN" % "dive" % "midx" % "engVer" % "start-of-surface"
01138                                % "vac0" % "vac1" % "nSBD" % "nPrevAttempt"
01139                        « endl;
01140                     os « format(fmt2) % "" % "" % "" % "" % "" % "(inHg)" % "(inHg)" % "" % ""
01141                        « endl;
01142                     os « format(fmt3) % id_instrument() % id_dive() % midx++
01143                                % (int) engineering_version() % GenerateIso8601String(start_time())
01144                                % case_vac_at_start_inhg() % case_vac_at_end_inhg()
01145                                % num_sbd_packets_for_dive()
01146                                % num_transmit_attempts_for_previous_dive()
01147                        « endl;
01148                 }
01149
01150                 os « "# --- Air Pump & Bladder ------------------------------------------------"
01151                    "-------------------------"
01152                    « endl;
01153                 static const format fmt4("#   %5s %4s %4s %15s %15s %15s");
01154                 static const format fmt5("#   %5s %4s %4s %15s %15s %15s");
01155                 static const format fmt6("E0A %05d %4d %4d %15.3f %15d %15.3f");
01156                 os « format(fmt4) % "SN" % "dive" % "midx" % "APCur" % "APRunT" % "AirBladMxP"
01157                    « endl;
01158                 os « format(fmt5) % "" % "" % "" % "(A)" % "(sec)" % "(PSI)" « endl;
01159                 os « format(fmt6) % id_instrument() % id_dive() % midx++ % air_pump_current_a()
01160                           % air_pump_runtime_s() % air_bladder_max_pres_psi()
01161                    « endl;
01162
01163                 os « "# --- Air Valve Position ----------------------------------------------"
01164                    "-------------------------"
01165                    « endl;
01166                 static const format fmt7("#   %5s %4s %4s %15s %15s %15s %15s %15s");
01167                 static const format fmt8("#   %5s %4s %4s %15s %15s %15s %15s %15s");
01168                 static const format fmt9("E0A %05d %4d %4d %15.3f %15.3f %15.3f %15.3f %15.3f");
01169                 os « format(fmt7) % "SN" % "dive" % "midx" % "AVOpenStart" % "AVMidEnd"
01170                           % "ACVpenCal" % "AVMidCal" % "AVCloseCal"
01171                    « endl;
01172                 os « format(fmt8) % "" % "" % "" % "(V)" % "(V)" % "(V)" % "(V)" % "(V)" « endl;
01173                 os « format(fmt9) % id_instrument() % id_dive() % midx++
01174                           % air_valve_open_position_at_start_v()
01175                           % air_valve_mid_position_at_end_v() % air_valve_open_cal_position_v()
01176                           % air_valve_mid_cal_position_v() % air_valve_closed_cal_position_v()
01177                    « endl;
01178
01179                 os « "# --- Oil Valve Position ------------------------------------------------"
01180                    "-------------------------"
01181                    « endl;
01182                 static const format fmt10("#   %5s %4s %4s %15s %15s %15s %15s");
01183                 static const format fmt11("#   %5s %4s %4s %15s %15s %15s %15s");
01184                 static const format fmt12("E0A %05d %4d %4d %15.3f %15.3f %15.3f %15.3f");
01185                 os « format(fmt10) % "SN" % "dive" % "midx" % "OVOpenAirPmp" % "OVCloseEnd"
01186                           % "OVOpenCal" % "OVClosCal"
01187                    « endl;
01188                 os « format(fmt11) % "" % "" % "" % "(V)" % "(V)" % "(V)" % "(V)" « endl;
01189                 os « format(fmt12) % id_instrument() % id_dive() % midx++
01190                           % oil_valve_open_position_during_air_assist_v()
01191                           % oil_valve_closed_position_at_end_v() % oil_valve_open_cal_position_v()
01192                           % oil_valve_closed_cal_position_v()
01193                    « endl;
01194
01195                 os « "# --- Surface Pressure ------------------------------------------------"
01196                    "-------------------------"
01197                    « endl;
01198                 static const format fmt17("#   %5s %4s %4s "
01199                        "%15s");
01200                 static const format fmt18("#   %5s %4s %4s "
01201                        "%15s");
01202                 static const format fmt19("E0A %05d %4d %4d "
01203                        "%15.3f");
01204                 os « format(fmt17) % "SN" % "dive" % "midx" //
01205                           % "presOffset" //
01206                    « endl;
01207                 os « format(fmt18) % "" % "" % "" //
01208                           % "(dBar)"
01209                    « endl;
01210                 os « format(fmt19) % id_instrument() % id_dive() % midx++ // midx
01211                        % GetDecodedBitPressureOffset() //
01212                    « endl;
01213
01214                 {   os « "# --- Profile - First Sample ---------------------------------------"
01215                        "-----------------------------"
01216                       « endl;
01217                     static const format fmt1(
```

```
01218                       "#   %5s %4s %4s "
01219                       "   %12s %12s "
01220                       "%12s %12s "
01221                       "%12s %12s");
01222                   static const format fmt2(
01223                       "#   %5s %4s %4s "
01224                       "   %12s %12s "
01225                       "%13s %12s "
01226                       "%12s %12s");
01227                   static const format fmt3(
01228                       "E0A %05d %4d %4d "
01229                       "   %12.3f %12d "
01230                       "%12.3f %12d "
01231                       "%12.3f %12d");
01232                   os « format(fmt1)
01233                       % "SN"          % "dive"        % "midx"
01234                       % "firstPres"   % "firstPres"
01235                       % "firstTemp"   % "firstTemp"
01236                       % "firstSaln"   % "firstSaln"
01237                       « endl;
01238                   os « format(fmt2)
01239                       % ""            % ""            % ""
01240                       % "(dBar)"      % "(counts)"
01241                       % "(degC)"        % "(counts)"
01242                       % "(PSU)"       % "(counts)"
01243                       « endl;
01244                   os « format(fmt3)
01245                       % id_instrument() % id_dive() % midx++
01246                       % prof_raw_sample_first_pres_dbar(pg, po) % prof_raw_sample_first_pres_counts()
01247                       % prof_raw_sample_first_temp_degc(tg, to) % prof_raw_sample_first_temp_counts()
01248                       % prof_raw_sample_first_saln_psu(sg, so)  % prof_raw_sample_first_saln_counts()
01249                       « endl;
01250                   os « "# --- Profile - Last Sample ------------------------------------------"
01251                       "-----------------------------"
01252                       « endl;
01253                   os « format(fmt1)
01254                       % "SN"          % "dive"        % "midx"
01255                       % "lastPres"    % "lastPres"
01256                       % "lastTemp"    % "lastTemp"
01257                       % "lastSaln"    % "lastSaln"
01258                       « endl;
01259                   os « format(fmt2)
01260                       % ""            % ""            % ""
01261                       % "(dBar)"      % "(counts)"
01262                       % "(degC)"        % "(counts)"
01263                       % "(PSU)"       % "(counts)"
01264                       « endl;
01265                   os « format(fmt3)
01266                       % id_instrument() % id_dive() % midx++
01267                       % prof_raw_sample_last_pres_dbar(pg, po) % prof_raw_sample_last_pres_counts()
01268                       % prof_raw_sample_last_temp_degc(tg, to) % prof_raw_sample_last_temp_counts()
01269                       % prof_raw_sample_last_saln_psu(sg, so)  % prof_raw_sample_last_saln_counts()
01270                       « endl;
01271               }
01272
01273               {   os « "# --- Profile - Meta ------------------------------------------------"
01274                         "-----------------------------"
01275                       « endl;
01276                   static const format fmt1(
01277                       "#   %5s %4s %4s "
01278                       "%15s %15s %15s");
01279                   static const format fmt2(
01280                       "E0A %05d %4d %4d "
01281                       "%15d %15d %15d");
01282                   os « format(fmt1)
01283                        % "SN"              % "dive"               % "midx"
01284                        % "numBins"         % "numEmptyBins"       % "numRawSamples"
01285                        « endl;
01286                   os « format(fmt2)
01287                        % id_instrument()   % id_dive()           % midx++
01288                        % prof_num_bins()   % prof_num_bins_empty() % prof_num_raw_samples()
01289                        « endl;
01290               }
01291
01292               {   os « "# --- Real-Time Clock-------------------------------------------------"
01293                         "-----------------------------"
01294                       « endl;
01295                   static const format fmt1(
01296                       "#   %5s %4s %4s "
01297                       "%15s");
01298                   static const format fmt2(
01299                       "#   %5s %4s %4s "
01300                       "%15s");
01301                   static const format fmt3(
01302                       "E0A %05d %4d %4d "
01303                       "%15d");
01304                   os « format(fmt1)
01305                        % "SN"              % "dive"               % "midx"
01306                        % "gpsOffset"
01307                        « endl;
01308                   os « format(fmt2)
01309                        % ""                % ""                   % ""
01310                        % "(sec)"
01311                        « endl;
01312                   os « format(fmt3)
01313                        % id_instrument()   % id_dive()           % midx++
01314                        % gps_rtc_time_delta()
01315                        « endl;
01316               }
01317
01318               {   os « "# --- Surface Sampling ------------------------------------------------"
01319                         "-----------------------------"
01320                       « endl;
01321                   static const format fmt1(
01322                       "#   %5s %4s %4s "
01323                       "%20s %20s");
01324                   static const format fmt2(
01325                       "#   %5s %4s %4s "
```

```
01326                       "%20s %20s");
01327                   static const format fmt3(
01328                       "E0A %05d %4d %4d "
01329                       "%20s %20s");
01330                   os « format(fmt1)
01331                        % "SN"                % "dive"               % "midx"
01332                        % "start"             % "end"
01333                        « endl;
01334                   os « format(fmt2)
01335                        % ""                  % ""                  % ""
01336                        % ""                  % ""
01337                        « endl;
01338                   os « format(fmt3)
01339                        % id_instrument()     % id_dive()            % midx++
01340                        % GenerateIso8601String(surface_sampling_start_time())
01341                        %  GenerateIso8601String(surface_sampling_end_time())
01342                        « endl;
01343               }
01344
01345               // Minor Index 8.
01346               os « "# --- Errors ----------------------------------------------------------------"
01347                  "-------------------------"
01348                  « endl;
01349               static const format fmt13("#   %5s %4s %4s %15s");
01350               static const format fmt14("#   %5s %4s %4s %15s");
01351               static const format fmt15("E0A %05d %4d %4d %15d");
01352               os « format(fmt13) % "SN" % "dive" % "midx" % "nErr" « endl;
01353               os « format(fmt14) % "" % "" % "" % "" « endl;
01354               os « format(fmt15) % id_instrument() % id_dive() % midx++ // midx
01355                       % (int) num_errors();
01356
01357               // Error codes.
01358               if(!error_codes().size())
01359                   break;
01360               static const format fmt16("#   %5s %4s %4s %20s %15s");
01361               static const format fmt_err("e0A %05d %4d %4d %20s      0x%08X");
01362
01363               if(num_errors()) {
01364                   os « endl « format(fmt16) % "SN" % "dive" % "idx" % "date-time" % "err-code";
01365                   midx = 0;
01366                   for(vector<shared_ptr<XEngErrorCode»::const_iterator it = error_codes_.begin();
01367                           it < error_codes_.end(); it++)
01368                       os « endl
01369                          « format(fmt_err) % id_instrument() % id_dive() % midx++
01370                                  % (*it)->DateTimeAs8601String() % (*it)->error_code();
01371               }
01372           }
01373               break;
01374
01375           default: { // Unsupported Engineering (Surface) version.
01376               ostringstream os;
01377               os « "Unsupported ALAMO Engineering (Surface) message version: 0x"
01378                  « hex
01379                  « (int) engineering_version()
01380                  « "!";
01381               EXC_THROW(os.str());
01382           }
01383               break;
01384           }
01385
01386       }CATCH_BLOCK("Failed to generate listing string for ALAMO Engineering (Surface) message!")
01387
01388       return os.str();
01389   }
01390
01391 public:
01392
01393   /* Public Member Functions - Accessor Methods: */
01394
01395   uint16_t air_bladder_max_pres_mpsi() const {
01396       return air_bladder_max_pres_mpsi_;
01397   }
01398
01399   template<typename T = float> T air_bladder_max_pres_psi() const {
01400       return (T) air_bladder_max_pres_mpsi() / 1000.;
01401   }
01402
01403   template<typename T = float> T air_pump_current_a() const {
01404       return (T) air_pump_current_ma() / 1000.;
01405   }
01406
01407   uint16_t air_pump_current_ma() const {
01408       return air_pump_current_ma_;
01409   }
01410
01411   uint16_t air_pump_runtime_s() const {
01412       return air_pump_runtime_s_;
01413   }
01414
01415   uint16_t air_valve_closed_cal_position_mv() const {
01416       return air_valve_closed_cal_position_mv_;
01417   }
01418
01419   template<typename T = float> T air_valve_closed_cal_position_v() const {
01420       return (T) air_valve_closed_cal_position_mv() / 1000.;
01421   }
01422
01423   uint16_t air_valve_mid_cal_position_mv() const {
01424       return air_valve_mid_cal_position_mv_;
01425   }
01426
01427   template<typename T = float> T air_valve_mid_cal_position_v() const {
01428       return (T) air_valve_mid_cal_position_mv() / 1000.;
01429   }
01430
01431   uint16_t air_valve_mid_position_at_end_mv() const {
01432       return air_valve_mid_position_at_end_mv_;
01433   }
```

```
01434
01435     template<typename T = float> T air_valve_mid_position_at_end_v() const {
01436         return (T) air_valve_mid_position_at_end_mv() / 1000.;
01437     }
01438
01439     uint16_t air_valve_open_cal_position_mv() const {
01440         return air_valve_open_cal_position_mv_;
01441     }
01442
01443     template<typename T = float> T air_valve_open_cal_position_v() const {
01444         return (T) air_valve_open_cal_position_mv() / 1000.;
01445     }
01446
01447     uint16_t air_valve_open_position_at_start_mv() const {
01448         return air_valve_open_position_at_start_mv_;
01449     }
01450
01451     template<typename T = float> T air_valve_open_position_at_start_v() const {
01452         return (T) air_valve_open_position_at_start_mv() / 1000.;
01453     }
01454
01455     uint16_t battery_voltage_mv() const {
01456         return battery_voltage_mv_;
01457     }
01458
01459     /** Get the measured system battery voltage, in volts.
01460      *
01461      *  @return The measured system battery voltage, in volts.
01462      */
01463     template<typename T = float> T battery_voltage_v() const {
01464         return (T) battery_voltage_mv_ / 1000.;
01465     }
01466
01467     uint16_t case_vac_at_end_cinhg() const {
01468         return case_vac_at_end_cinhg_;
01469     }
01470
01471     template<typename T = float> T case_vac_at_end_inhg() const {
01472         return (T) case_vac_at_end_cinhg() / 100.;
01473     }
01474
01475     uint16_t case_vac_at_start_cinhg() const {
01476         return case_vac_at_start_cinhg_;
01477     }
01478
01479     template<typename T = float> T case_vac_at_start_inhg() const {
01480         return (T) case_vac_at_start_cinhg() / 100.;
01481     }
01482
01483     uint16_t case_vac_cingh() const {
01484         return case_vac_cingh_;
01485     }
01486
01487     /** Get the measured internal pressure case vacuum, in inches of Mercury.
01488      *
01489      *  @return     The measured internal pressure case vacuum, in inches of Mercury.
01490      */
01491     template<typename T = float> T case_vacuum_inhg() const {
01492         return (T) case_vac_cingh_ / 100.;
01493     }
01494
01495     uint32_t ctd_status_code() const {
01496         return ctd_status_code_;
01497     }
01498
01499
01500     template<typename T = double> T GetDecodedBitPressureOffset() const {
01501         T val;
01502
01503         val = (T) pressure_offset_reading() / (T) 1000;
01504
01505         if(val < XMSG_SENS_MIN_PRES || val > XMSG_SENS_MAX_PRES)
01506             return XMSG_SENS_BAD_PRES;
01507
01508         return val;
01509     }
01510
01511     int16_t gps_rtc_time_delta() const {
01512         return gps_rtc_time_delta_;
01513     }
01514
01515     uint8_t num_errors() const {
01516         return num_errors_;
01517     }
01518
01519     uint16_t num_sbd_packets_for_dive() const {
01520         return num_sbd_packets_for_dive_;
01521     }
01522
01523     uint16_t num_transmit_attempts_for_previous_dive() const {
01524         return num_transmit_attempts_for_previous_dive_;
01525     }
01526
01527     uint16_t oil_valve_closed_cal_position_mv() const {
01528         return oil_valve_closed_cal_position_mv_;
01529     }
01530
01531     template<typename T = float> T oil_valve_closed_cal_position_v() const {
01532         return (T) oil_valve_closed_cal_position_mv() / 1000.;
01533     }
01534
01535     uint16_t oil_valve_closed_position_at_end_mv() const {
01536         return oil_valve_closed_position_at_end_mv_;
01537     }
01538
01539     template<typename T = float> T oil_valve_closed_position_at_end_v() const {
01540         return (T) oil_valve_closed_position_at_end_mv() / 1000.;
01541     }
```

```
01542
01543     uint16_t oil_valve_open_cal_position_mv() const {
01544         return oil_valve_open_cal_position_mv_;
01545     }
01546
01547     template<typename T = float> T oil_valve_open_cal_position_v() const {
01548         return (T) oil_valve_open_cal_position_mv() / 1000.;
01549     }
01550
01551     uint16_t oil_valve_open_position_during_air_assist_mv() const {
01552         return oil_valve_open_position_during_air_assist_mv_;
01553     }
01554
01555     template<typename T = float> T oil_valve_open_position_during_air_assist_v() const {
01556         return (T) oil_valve_open_position_during_air_assist_mv() / 1000.;
01557     }
01558
01559     /** Get the measured system current, in amperes.
01560      *
01561      *  @return     The measured system current, in amperes.
01562      */
01563     template<typename T = float> T system_current_a() const {
01564         return (T) system_current_ma_ / 1000.;
01565     }
01566
01567     uint16_t system_current_ma() const {
01568         return system_current_ma_;
01569     }
01570
01571     const vector<shared_ptr<XEngErrorCode> >& error_codes() const {
01572         return error_codes_;
01573     }
01574
01575     int16_t pressure_offset_reading() const {
01576         return pressure_offset_reading_;
01577     }
01578
01579     uint16_t prof_num_bins() const {
01580         return prof_num_bins_;
01581     }
01582
01583     uint16_t prof_num_bins_empty() const {
01584         return prof_num_bins_empty_;
01585     }
01586
01587     uint16_t prof_raw_sample_first_pres_counts() const {
01588         return prof_raw_sample_first_pres_;
01589     }
01590
01591     template<typename T = float> T prof_raw_sample_first_pres_dbar(int16_t gain, int16_t offs) const {
01592         T val;
01593
01594         val = gain ? (T) prof_raw_sample_first_pres_counts() / (T) gain - (T) offs : (T)
      XMSG_SENS_BAD_PRES;
01595
01596         if(val < XMSG_SENS_MIN_PRES || val > XMSG_SENS_MAX_PRES)
01597             return XMSG_SENS_BAD_PRES;
01598
01599         return val;
01600     }
01601
01602
01603     uint16_t prof_raw_sample_first_saln_counts() const {
01604         return prof_raw_sample_first_saln_;
01605     }
01606
01607     template<typename T = float> T prof_raw_sample_first_saln_psu(int16_t gain, int16_t offs) const {
01608         T val;
01609
01610         val = gain ? (T) prof_raw_sample_first_saln_counts() / (T) gain - (T) offs : (T)
      XMSG_SENS_BAD_SALN;
01611
01612         if(val < XMSG_SENS_MIN_SALN || val > XMSG_SENS_MAX_SALN)
01613             return XMSG_SENS_BAD_SALN;
01614
01615         return val;
01616     }
01617
01618     uint16_t prof_raw_sample_first_temp_counts() const {
01619         return prof_raw_sample_first_temp_;
01620     }
01621
01622     template<typename T = float> T prof_raw_sample_first_temp_degc(int16_t gain, int16_t offs) const {
01623         T val;
01624
01625         val = gain ? (T) prof_raw_sample_first_temp_counts() / (T) gain - (T) offs : (T)
      XMSG_SENS_BAD_TEMP;
01626
01627         if(val < XMSG_SENS_MIN_TEMP || val > XMSG_SENS_MAX_TEMP)
01628             return XMSG_SENS_BAD_TEMP;
01629
01630         return val;
01631     }
01632
01633     uint16_t prof_raw_sample_last_pres_counts() const {
01634         return prof_raw_sample_last_pres_;
01635     }
01636
01637     template<typename T = float> T prof_raw_sample_last_pres_dbar(int16_t gain, int16_t offs) const {
01638         T val;
01639
01640         val = gain ? (T) prof_raw_sample_last_pres_counts() / (T) gain - (T) offs : (T)
      XMSG_SENS_BAD_PRES;
01641
01642         if(val < XMSG_SENS_MIN_PRES || val > XMSG_SENS_MAX_PRES)
01643             return XMSG_SENS_BAD_PRES;
01644
01645         return val;
```

```
01646     }
01647
01648     uint16_t prof_raw_sample_last_saln_counts() const {
01649         return prof_raw_sample_last_saln_;
01650     }
01651
01652     template<typename T = float> T prof_raw_sample_last_saln_psu(int16_t gain, int16_t offs) const {
01653         T val;
01654
01655         val = gain ? (T) prof_raw_sample_last_saln_counts() / (T) gain - (T) offs : (T)
      XMSG_SENS_BAD_SALN;
01656
01657         if(val < XMSG_SENS_MIN_SALN || val > XMSG_SENS_MAX_SALN)
01658             return XMSG_SENS_BAD_SALN;
01659
01660         return val;
01661     }
01662
01663
01664     uint16_t prof_raw_sample_last_temp_counts() const {
01665         return prof_raw_sample_last_temp_;
01666     }
01667
01668     template<typename T = float> T prof_raw_sample_last_temp_degc(int16_t gain, int16_t offs) const {
01669         T val;
01670
01671         val = gain ? (T) prof_raw_sample_last_temp_counts() / (T) gain - (T) offs : (T)
      XMSG_SENS_BAD_TEMP;
01672
01673         if(val < XMSG_SENS_MIN_TEMP || val > XMSG_SENS_MAX_TEMP)
01674             return XMSG_SENS_BAD_TEMP;
01675
01676         return val;
01677     }
01678
01679     uint16_t prof_num_raw_samples() const {
01680         return prof_num_raw_samples_;
01681     }
01682
01683     const ptime& surface_sampling_end_time() const {
01684       return surface_sampling_end_time_;
01685     }
01686
01687     const ptime& surface_sampling_start_time() const {
01688       return surface_sampling_start_time_;
01689     }
01690
01691 protected:
01692
01693     /* Protected Member Functions - Mutator Methods: */
01694
01695     void set_air_bladder_max_pres_mpsi(uint16_t air_bladder_max_pres_mpsi) {
01696         air_bladder_max_pres_mpsi_ = air_bladder_max_pres_mpsi;
01697     }
01698
01699     void set_air_pump_current_ma(uint16_t air_pump_current_ma) {
01700         air_pump_current_ma_ = air_pump_current_ma;
01701     }
01702
01703     void set_air_pump_runtime_s(uint16_t air_pump_runtime_s) {
01704         air_pump_runtime_s_ = air_pump_runtime_s;
01705     }
01706
01707     void set_air_valve_closed_cal_position_mv(uint16_t air_valve_closed_cal_position_mv) {
01708         air_valve_closed_cal_position_mv_ = air_valve_closed_cal_position_mv;
01709     }
01710
01711     void set_air_valve_mid_cal_position_mv(uint16_t air_valve_mid_cal_position_mv) {
01712         air_valve_mid_cal_position_mv_ = air_valve_mid_cal_position_mv;
01713     }
01714
01715     void set_air_valve_open_cal_position_mv(uint16_t air_valve_open_cal_position_mv) {
01716         air_valve_open_cal_position_mv_ = air_valve_open_cal_position_mv;
01717     }
01718
01719     void set_air_valve_mid_position_at_end_mv(uint16_t air_valve_mid_position_at_end_mv) {
01720         air_valve_mid_position_at_end_mv_ = air_valve_mid_position_at_end_mv;
01721     }
01722
01723     void set_air_valve_open_position_at_start_mv(uint16_t air_valve_open_position_at_start_mv) {
01724         air_valve_open_position_at_start_mv_ = air_valve_open_position_at_start_mv;
01725     }
01726
01727     void set_battery_voltage_mv(uint16_t battery_voltage_mv) {
01728         battery_voltage_mv_ = battery_voltage_mv;
01729     }
01730
01731     void set_case_vac_at_end_cinhg(uint16_t case_vac_at_end_cinhg) {
01732         case_vac_at_end_cinhg_ = case_vac_at_end_cinhg;
01733     }
01734
01735     void set_case_vac_at_start_cinhg(uint16_t case_vac_at_start_cinhg) {
01736         case_vac_at_start_cinhg_ = case_vac_at_start_cinhg;
01737     }
01738
01739     void set_case_vac_cingh(uint16_t case_vac_cingh) {
01740         case_vac_cingh_ = case_vac_cingh;
01741     }
01742
01743     void set_ctd_status_code(uint32_t ctd_status_code) {
01744         ctd_status_code_ = ctd_status_code;
01745     }
01746
01747     void set_error_codes(const vector<shared_ptr<XEngErrorCode> >& error_codes) {
01748         error_codes_ = error_codes;
01749     }
01750
01751     void set_num_errors(uint8_t num_errors) {
```

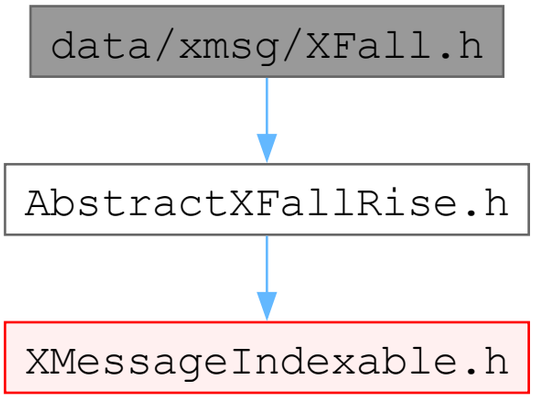
```
01752         num_errors_ = num_errors;
01753     }
01754
01755     void set_num_sbd_packets_for_dive(uint16_t num_sbd_packets_for_dive) {
01756         num_sbd_packets_for_dive_ = num_sbd_packets_for_dive;
01757     }
01758
01759     void set_num_transmit_attempts_for_previous_dive(
01760             uint16_t num_transmit_attempts_for_previous_dive) {
01761         num_transmit_attempts_for_previous_dive_ = num_transmit_attempts_for_previous_dive;
01762     }
01763
01764     void set_oil_valve_closed_cal_position_mv(uint16_t oil_valve_closed_cal_position_mv) {
01765         oil_valve_closed_cal_position_mv_ = oil_valve_closed_cal_position_mv;
01766     }
01767
01768     void set_oil_valve_closed_position_at_end_mv(uint16_t oil_valve_closed_position_at_end_mv) {
01769         oil_valve_closed_position_at_end_mv_ = oil_valve_closed_position_at_end_mv;
01770     }
01771
01772     void set_oil_valve_open_cal_position_mv(uint16_t oil_valve_open_cal_position_mv) {
01773         oil_valve_open_cal_position_mv_ = oil_valve_open_cal_position_mv;
01774     }
01775
01776     void set_oil_valve_open_position_during_air_assist_mv(
01777             uint16_t oil_valve_open_position_during_air_assist_mv) {
01778         oil_valve_open_position_during_air_assist_mv_ = oil_valve_open_position_during_air_assist_mv;
01779     }
01780
01781     void set_pressure_offset_reading(int16_t pressure_offset_reading) {
01782         pressure_offset_reading_ = pressure_offset_reading;
01783     }
01784
01785     void set_system_current_ma(uint16_t system_current_ma) {
01786         system_current_ma_ = system_current_ma;
01787     }
01788
01789     void set_prof_num_bins(uint16_t prof_num_bins) {
01790         prof_num_bins_ = prof_num_bins;
01791     }
01792
01793     void set_prof_num_bins_empty(uint16_t prof_num_bins_empty) {
01794         prof_num_bins_empty_ = prof_num_bins_empty;
01795     }
01796
01797     void set_prof_raw_sample_first_pres(uint16_t prof_raw_sample_first_pres) {
01798         prof_raw_sample_first_pres_ = prof_raw_sample_first_pres;
01799     }
01800
01801     void set_prof_raw_sample_first_saln(uint16_t prof_raw_sample_first_saln) {
01802         prof_raw_sample_first_saln_ = prof_raw_sample_first_saln;
01803     }
01804
01805     void set_prof_raw_sample_first_temp(uint16_t prof_raw_sample_first_temp) {
01806         prof_raw_sample_first_temp_ = prof_raw_sample_first_temp;
01807     }
01808
01809     void set_prof_raw_sample_last_pres(uint16_t prof_raw_sample_last_pres) {
01810         prof_raw_sample_last_pres_ = prof_raw_sample_last_pres;
01811     }
01812
01813     void set_prof_raw_sample_last_saln(uint16_t prof_raw_sample_last_saln) {
01814         prof_raw_sample_last_saln_ = prof_raw_sample_last_saln;
01815     }
01816
01817     void set_prof_raw_sample_last_temp(uint16_t prof_raw_sample_last_temp) {
01818         prof_raw_sample_last_temp_ = prof_raw_sample_last_temp;
01819     }
01820
01821     void set_prof_num_raw_samples(uint16_t prof_num_raw_samples) {
01822         prof_num_raw_samples_ = prof_num_raw_samples;
01823     }
01824
01825     void set_gps_rtc_time_delta(int16_t gps_rtc_time_delta) {
01826         gps_rtc_time_delta_ = gps_rtc_time_delta;
01827     }
01828
01829     void set_surface_sampling_end_time(const ptime& surface_sampling_end_time) {
01830       surface_sampling_end_time_ = surface_sampling_end_time;
01831     }
01832
01833     void set_surface_sampling_start_time(const ptime& surface_sampling_start_time) {
01834       surface_sampling_start_time_ = surface_sampling_start_time;
01835     }
01836 };
01837
01838 } /* namespace xmsg */
01839 } /* namespace util */
01840 #endif /* XENGSURFACE_H_ */
```

11.126 data/xmsg/XFall.h File Reference

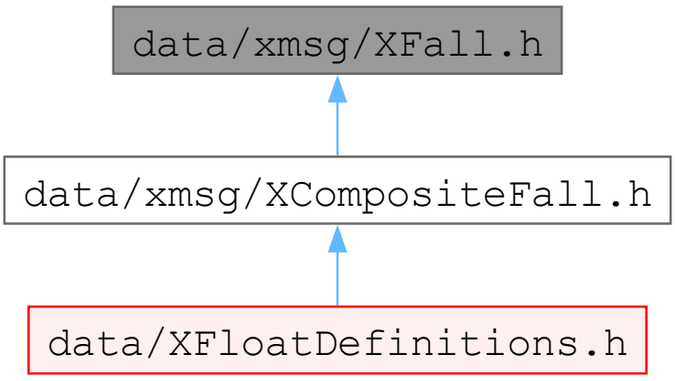
The util::xmsg::XFall class header file.

```
#include "AbstractXFallRise.h"
```

Include dependency graph for XFall.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XFall

  Class representing a fall time-series data X-Message.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.126.1 Detailed Description

The util::xmsg::XFall class header file.

Version

0.6

Date

Jul 1, 2014 11:18:07 AM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.127 XFall.h

Go to the documentation of this file.

```
00001 /** The @ref util::xmsg::XFall class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jul 1, 2014 11:18:07 AM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef XFALL_H_
00024 #define XFALL_H_
00025
00026 /* Includes */
00027
00028 #include "AbstractXFallRise.h"
00029
00030 namespace util {
00031   namespace xmsg {
00032
00033     /** Class representing a fall time-series data X-Message.
00034      *
00035      */
00036     class XFall:
00037         public AbstractXFallRise {
00038
00039       public:
00040
00041         /* Member Functions */
00042
00043         /** Parameterized constructor.
00044          *
00045          *  @param[in]  dive    The dive or cycle number of this message.
00046          *  @param[in]  whoi    The unique WHOI identifier.
00047          *  @param[in]  data    The binary fall rate X-message data to decode.
00048          *
00049          *  @exception  log::MessageException if an decode error occurs.
00050          */
00051         XFall(int16_t dive, uint16_t whoi, const vector<uint8_t>& data) :
00052                 AbstractXFallRise(dive, whoi, data) {
00053           DecodeXMsgData(data);     // Decode message data.
00054         }
00055
00056         /** Destructor.
00057          *
00058          */
00059         ~XFall() {
00060           /* Do nothing. */
00061         }
00062
00063         /** Decode the fall data X-Message from the binary message data.
00064          *
```

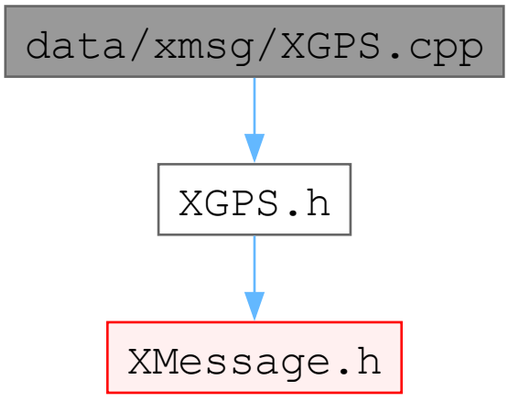
```
00065          *  @param[in]  data    The binary message data to decode.
00066          *
00067          *  @exception  log::MessageException if the message is invalid or an error occurs during
00068          *              decoding.
00069          */
00070         void DecodeXMsgData(const vector<uint8_t>& data) {
00071           try {
00072             if((data.data()[0] & 0xF0) != (XMSG_TYPE_ID_FALL0 & 0xF0)) {
00073               EXC_THROW("Invalid X-Message unique identifier!");
00074             }
00075             UnpackData(data);       // Unpack the message data.
00076           }CATCH_BLOCK("Failed to decode fall time-series X-Message!")
00077         }
00078
00079         /** Get the listing entry string for this fall data X-Message
00080          *
00081          *  @param[in]  pg      The pressure sensor sensitivity scale factor, in LSB/dBar.
00082          *  @param[in]  po      The pressure sensor offset, in dBar.
00083          *  @param[in]  sg      The salinity sensor sensitivity scale factor, in LSB/PSU.
00084          *  @param[in]  so      The salinity sensor offset, in PSU.
00085          *  @param[in]  tg      The temperature sensor sensitivity scale factor, in LSB/degC.
00086          *  @param[in]  to      The temperature sensor offset, in degC.
00087          *  @param[in]  pc      The pressure correction, in dBar, to be subtracted computed
00088          *                      pressure value.
00089          *
00090          *  @exception  log::MessageException if an error occurs while generating the listing string.
00091          *
00092          *  @return     The listing entry string for this fall data X-Message
00093          */
00094         const string GenerateListingString(
00095             uint16_t pg,
00096             uint16_t po,
00097             uint16_t sg,
00098             uint16_t so,
00099             uint16_t tg,
00100             uint16_t to,
00101             float pc) const {
00102           ostringstream os;
00103           size_t sam = 0;
00104
00105           try {
00106             {
00107               const format fmt("F   %5d %4d %3d %4d %20s");
00108               os « format(fmt) % id_instrument() % id_dive() % (int) xmsg_index() % num_samples()
00109                   % GenerateIso8601String(start_time());
00110             }
00111
00112             {
00113               const format fmt("\nf   %5d %4d %3d %4d ");
00114               for_each(
00115                   sensor_data().begin(),
00116                   sensor_data().end(),
00117                   [&](const XSensorTripletCodeCounts& sensor_data) {
00118
00119                     os « format(fmt) % id_instrument() % id_dive() % (int) xmsg_index() % sam++
00120                     « sensor_data.GenerateListingString(pg, po, sg, so, tg, to, pc);
00121
00122                   });
00123             }
00124           }CATCH_BLOCK("Failed to generate listing string for fall time-series data object!")
00125
00126           return os.str();
00127         }
00128     };
00129
00130   } /* namespace xmsg */
00131 } /* namespace util */
00132 #endif /* XFALL_H_ */
```

11.128 data/xmsg/XGPS.cpp File Reference

The util::xmsg::XGPS class source file.

```
#include "XGPS.h"
```

Include dependency graph for XGPS.cpp:



Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.128.1 Detailed Description

The util::xmsg::XGPS class source file.

Version

0.6

Date

Jun 9, 2014 3:57:31 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

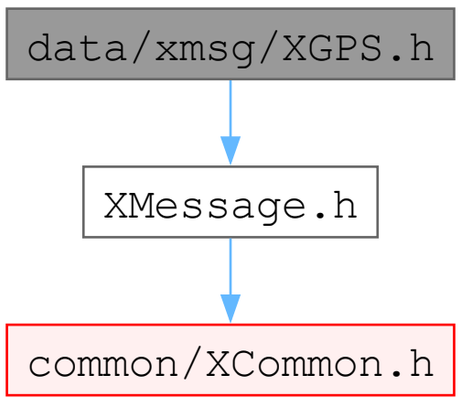
This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.129 data/xmsg/XGPS.h File Reference

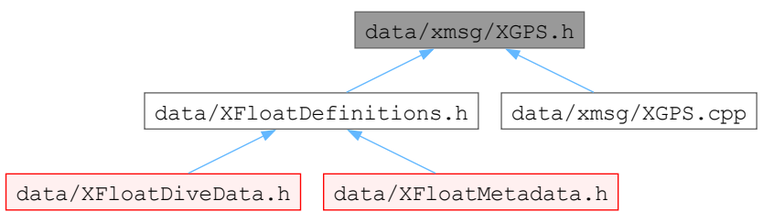
The util::xmsg::XGPS class header file.

```
#include "XMessage.h"
```

Include dependency graph for XGPS.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XGPS

  Class representing a GPS type X-Message.

Namespaces

- namespace argo

  The Argo instrument namespace.
- namespace argo::iridium

  The Argo Iridium instrument namespace.
- namespace argo::iridium::xfloat

  The X-Float instrument namespace.
- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.129.1 Detailed Description

The util::xmsg::XGPS class header file.

Version

0.6

Date

Jun 9, 2014 3:57:31 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.130 XGPS.h

Go to the documentation of this file.

```
00001 /** The @ref util::xmsg::XGPS class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jun 9, 2014 3:57:31 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef XGPS_H_
00024 #define XGPS_H_
00025
00026 /* Includes */
00027
00028 // local
00029 #include "XMessage.h"
00030
00031 namespace argo {
00032 namespace iridium {
00033 namespace xfloat {
00034
00035 // forward declarations
00036 class XFloatDiveData;
00037
00038 } /* namespace xfloat */
00039 } /* namespace iridium */
00040 } /* namespace argo */
00041
00042 namespace util {
00043 namespace xmsg {
00044
00045 using kmldom::CoordinatesPtr;
00046
00047 /** Class representing a GPS type X-Message.
00048  *
00049  */
00050 class XGPS:
00051         public XMessage,
00052         public position::Position,
00053         public util::kml::KmlUtil,
00054         public util::mysql::LocalPersistentEntity {
00055
00056     friend class argo::iridium::xfloat::XFloatDiveData;
```

```
00057
00058     /* Friend Functions: */
00059
00060     /** Friend function for creating an X-Float object using the private constructor and a shared
00061      *  pointer to that object.
00062      *
00063      */
00064     friend shared_ptr<XGPS> std::make_shared<XGPS>();
00065
00066 private:
00067
00068     /* Data Members: */
00069
00070     /** The horizontal Dilution of Precision (HDOP) of the GPS fix. */
00071     float horizontal_dilution_of_precision_;
00072
00073     /** The number of satellites used in GPS fix. */
00074     uint8_t num_satellites_;
00075
00076     /** The maximum Signal-to-Noise Ratio (SNR) for this GPS fix. */
00077     uint8_t signal_to_noise_max_;
00078
00079     /** The mean Signal-to-Noise Ratio (SNR) for this GPS fix. */
00080     uint8_t signal_to_noise_mean_;
00081
00082     /** The minimum Signal-to-Noise Ratio (SNR) for this GPS fix. */
00083     uint8_t signal_to_noise_min_;
00084
00085     static const util::mysql::SqlTableDescription<XGPS> sql_table_description_;
00086
00087     /** The time to acquire this GPS fix, in seconds. */
00088     uint16_t time_to_acquire_fix_;
00089
00090 private:
00091
00092     /* Private Member Functions: */
00093
00094     void set_horizontal_dilution_of_precision(float horizontal_dilution_of_precision) {
00095         horizontal_dilution_of_precision_ = horizontal_dilution_of_precision;
00096     }
00097
00098     void set_id_type(uint8_t type) {
00099         set_id_xmsg_type(static_cast<XMsgTypeId_t>(type));
00100     }
00101
00102     void set_num_satellites(uint8_t num_satellites) {
00103         num_satellites_ = num_satellites;
00104     }
00105
00106     void set_signal_to_noise_max(uint8_t signal_to_noise_max) {
00107         signal_to_noise_max_ = signal_to_noise_max;
00108     }
00109
00110     void set_signal_to_noise_mean(uint8_t signal_to_noise_mean) {
00111         signal_to_noise_mean_ = signal_to_noise_mean;
00112     }
00113
00114     void set_signal_to_noise_min(uint8_t signal_to_noise_min) {
00115         signal_to_noise_min_ = signal_to_noise_min;
00116     }
00117
00118     void set_time_to_acquire_fix(uint16_t time_to_acquire_fix) {
00119         time_to_acquire_fix_ = time_to_acquire_fix;
00120     }
00121
00122     /** Delete all GPS message rows for an instrument from the SQL database.
00123      *
00124      *  @param[in]  id_inst     The unique instrument identifier.
00125      *
00126      *  @throw      util::log::MessageException if the delete query fails.
00127      */
00128     static void DeleteFromDatabase(int id_inst) {
00129
00130         ostringstream os;
00131
00132         try {
00133
00134
00135
00136             os « "id_instrument = " « id_inst;
00137
00138             sql_table_description_.DeleteObjectsFromDatabase(
00139                                 os.str());
00140
00141         } CATCH_BLOCK("Failed to delete GPS objects from database!")
00142     }
00143
00144     /** Delete the GPS message rows for one dive of an instrument from the SQL database.
00145      *
00146      *  @warning    The generated condition uses @c == , which MySQL does not accept, so this query
00147      *              would fail. It is not currently called.
00148      *
00149      *  @param[in]  id_inst     The unique instrument identifier.
00150      *  @param[in]  id_dive     The dive or cycle number.
00151      *
00152      *  @throw      util::log::MessageException if the delete query fails.
00153      */
00154     static void DeleteFromDatabase(int id_inst, int id_dive) {
00155
00156         ostringstream os;
00157
00158         try {
00159
00160
00161
00162             os « "id_instrument = " « id_inst « " AND id_dive == " « id_dive;
00163
00164             sql_table_description_.DeleteObjectsFromDatabase(
```

```
00165                                os.str());
00166
00167         } CATCH_BLOCK("Failed to delete GPS objects from database!")
00168     }
00169
00170 public:
00171
00172     /* Public Member Functions: */
00173
00174     XGPS() :
00175                     XMessage(),
00176                     Position(),
00177                     horizontal_dilution_of_precision_(0),
00178                     num_satellites_(0),
00179                     signal_to_noise_max_(0),
00180                     signal_to_noise_mean_(0),
00181                     signal_to_noise_min_(0),
00182                     time_to_acquire_fix_(0) {
00183         /* Do nothing. */
00184     }
00185
00186     /** The parameterized, blank constructor.
00187      *
00188      *  @param[in]  dive        The dive or cycle number of this message.
00189      *  @param[in]  whoi        The unique WHOI identifier.
00190      *  @param[in]  type        The X-Message type identifier.
00191      *
00192      *
00193      *  @exception  None. Never throws.
00194      */
00195     XGPS(int16_t dive, uint16_t whoi, XMsgTypeId_t type) ATTR_NO_EXCEPTIONS :
00196                     XMessage(dive, whoi, type),
00197                     Position(),
00198                     horizontal_dilution_of_precision_(0),
00199                     num_satellites_(0),
00200                     signal_to_noise_max_(0),
00201                     signal_to_noise_mean_(0),
00202                     signal_to_noise_min_(0),
00203                     time_to_acquire_fix_(0) {
00204         /* Do nothing. */
00205     }
00206
00207     /** The parameterized, binary data constructor.
00208      *
00209      *  @param[in]      dive        The dive or cycle number of this message.
00210      *  @param[in]      whoi        The unique WHOI identifier.
00211      *  @param[in]      data        The binary X-message data to decode.
00212      *
00213      *  @throw          util::log::MessageException if an decode error occurs.
00214      */
00215     XGPS(int16_t dive, uint16_t whoi, const vector<uint8_t>& data) :
00216                     XMessage(dive, whoi, data),
00217                     Position(),
00218                     horizontal_dilution_of_precision_(0),
00219                     num_satellites_(0),
00220                     signal_to_noise_max_(0),
00221                     signal_to_noise_mean_(0),
00222                     signal_to_noise_min_(0),
00223                     time_to_acquire_fix_(0) {
00224         DecodeXMsgData(data);    // Decode message.
00225     }
00226
00227     /** The destructor.
00228      *
00229      */
00230     ~XGPS() {
00231         /* Do nothing. */
00232     }
00233
00234     /** Decode the GPS X-message from binary data.
00235      *
00236      *  @param[in]      data        The binary GPS X-message data to decode.
00237      */
00238     void DecodeXMsgData(const vector<uint8_t>& data) {
00239         //    int ifval;
00240         try {
00241             set_position_is_valid(false);
00242
00243             // Check for valid X message ID, size, & terminator.
00244             const uint8_t* d = CheckXMsgData(data, (int) -1, (int) 24);
00245             if(id_xmsg_type() > 5) {
00246                 ostringstream msg;
00247                 msg « "Invalid GPS XMessage ID: expected 0x00 through 0x05, received 0x"
00248                     « hex
00249                     « setw(2)
00250                     « setfill('0')
00251                     « id_xmsg_type()
00252                     « "!";
00253                 EXC_THROW(msg.str());
00254             }
00255
00256             // Check if the fix is valid.
00257             if(i08(d,3) == 0) {
00258
00259                 // GPS fix is not valid.
00260                 set_position_latitude(UTIL_POS_INVALID_LAT);
00261                 set_position_longitude(UTIL_POS_INVALID_LON);
00262                 set_position_is_valid(false);
00263
00264             } else {
00265
00266                 // GPS fix is valid -- parse GPS position.
00267                 int32_t llat = i32(d, 4), llon = i32(d, 8);
00268                 set_position_latitude(((double) llat) * 1e-7);
00269                 set_position_longitude(((double) llon) * 1e-7);
00270
00271                 if(!IsPositionValid(position_latitude(), position_longitude())) {
00272
```

```
00273                     set_position_latitude(UTIL_POS_INVALID_LAT);
00274                     set_position_longitude(UTIL_POS_INVALID_LON);
00275
00276                 } else {
00277
00278                     set_position_is_valid(true);
00279
00280                 }
00281             }
00282
00283             // Set position time from GPS date.
00284             set_gps_position_datetime(u16(d, 12), u08(d, 14), u08(d, 15), u08(d, 16));
00285             if(position_datetime() < bpt::from_iso_string(string("20000101T000000")) ||
      position_datetime() > GetCurrentUtcTime()) {
00286                 ldbg() « "GPS pos date/time: " « position_datetime_as_iso8601_string();
00287
00288                 set_position_datetime(bpt::not_a_date_time);
00289                 set_position_is_valid(false);
00290                 set_position_latitude(UTIL_POS_INVALID_LAT);
00291                 set_position_longitude(UTIL_POS_INVALID_LON);
00292             }
00293
00294             time_to_acquire_fix_ = u08(d,17) * 10;                                  // Time to fix
00295             num_satellites_ = u08(d,18) & 0x0F;                                     // No. satellites
00296             signal_to_noise_min_ = u08(d, 19);                                      // SNR
00297             signal_to_noise_mean_ = u08(d, 20);
00298             signal_to_noise_max_ = u08(d, 21);
00299
00300             horizontal_dilution_of_precision_ = (((float) u08(d, 22)) * 0.1);     // HDOP
00301
00302         }CATCH_BLOCK("Failed to decode GPS X message!")
00303     }
00304
00305     /** Delete GPS message rows from the SQL database table.
00306      *
00307      *  @param[in]  where       The SQL "where-condition" clause selecting the rows to delete.
00308      *
00309      *  @throw      util::log::MessageException if the delete query fails.
00310      */
00311     void static DeleteObjectsFromDatabase(const string& where)   {
00312
00313         try {
00314
00315             sql_table_description_.DeleteObjectsFromDatabase(where);
00316
00317         } CATCH_BLOCK("Failed to delete GPS objects from database!")
00318
00319     }
00320
00321     /** Get the listing entry comment string.
00322      *
00323      *  @return         The listing entry comment string.
00324      */
00325     static string GenerateListingComment() {
00326         ostringstream os;
00327
00328         try {
00329 //              const format fmt("#   %5s %4s %3s %20s %19s %5s %4s %5s %5s %5s %4s %4s\n"
00330 //                      "#   %5s %4s %3s %20s %21s %6s%4s %6s%6s%6s %4s %4s");
00331 //              os « format(fmt) % "SN" % "dive" % "idx" % "date/time" % "position" % "TTF" % "nSat"
00332 //                      % "SNR" % "SNR" % "SNR" % "HDOP" % "stat" % "" % "" % "" % "" % "" % "(sec)"
00333 //                      % "" % "(min)" % "(avg)" % "(max)" % "" % "";
00334           os « "#      SN dive idx            date/time            position    TTF nSat    SNR    SNR    SNR
      HDOP stat"
00335              « endl
00336              « "#                                                                (sec)       (min)  (avg)
      (max)";
00337
00338         }CATCH_BLOCK("Failed to generate GPS message listing comment!")
00339
00340         return os.str();
00341     }
00342
00343     /** Return a string representation of this GPS message.
00344      *
00345      *  @return         A string representation of this GPS message.
00346      */
00347     const string GenerateListingString() const {
00348         ostringstream os;
00349
00350         try {
00351             const format fmt("GPS %5d %4d %3s %20s %19s %5d %4d %5d %5d %5d %4.1f %4d");
00352             os « format(fmt) % id_instrument() % id_dive() % (int) id_xmsg_type()
00353                     % position_datetime_as_iso8601_string() % postition_as_iso6709_string()
00354                     % time_to_acquire_fix() % (int) num_satellites() % (int) signal_to_noise_min()
00355                     % (int) signal_to_noise_mean() % (int) signal_to_noise_max()
00356                     % horizontal_dilution_of_precision() % 0;
00357         }CATCH_BLOCK("Failed to generate listing string from GPS position object!")
00358
00359         return os.str();
00360     }
00361
00362     /** Generate a phy file entry string representation of this GPS position message object.
00363      *
00364      *  @param[in]      xmits       The number of transmissions for this dive data.
00365      *
00366      *  @throw          log::MessageException if an error occurs
00367      *
00368      *  @return         A phy file entry string representation of this GPS position message object.
00369      */
00370     const string GeneratePhyString(size_t xmits) const {
00371         ostringstream os;
00372
00373         try {
00374             bpt::ptime dt(position_datetime());
00375             os « "================================================================" « endl;
00376             os « "   LATITUDE  LONGITUDE YEAR/MO/DY HR:MN:SC XMITS SAT POSITION-QC" « endl;
00377             os « format(" %+10.5f %+10.5f ") % position_latitude() % position_longitude();
```

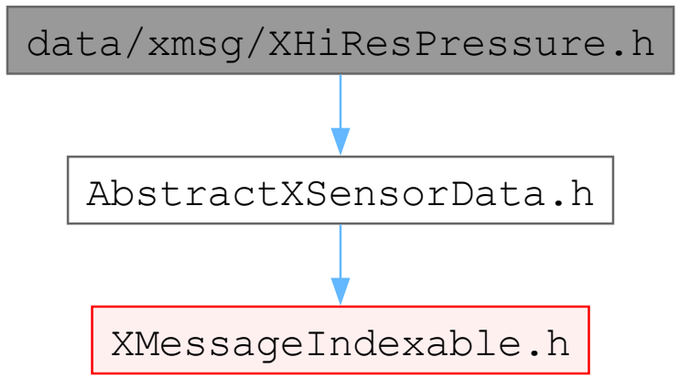
```
00378             if(dt.is_not_a_date_time()) {
00379                 os << "9999/99/99 99:99:99 ";
00380             } else {
00381                 os << format("%04d/%02d/%02d %02d:%02d:%02d ") //
00382                 % dt.date().year() % dt.date().month().as_number() % dt.date().day()
00383                         % dt.time_of_day().hours() % dt.time_of_day().minutes()
00384                         % dt.time_of_day().seconds();
00385             }
00386             os << format("%5d %3d %11d") //
00387             % xmits % (int) num_satellites_ % (position_is_valid() ? 0 : 9)
00388                << endl;
00389             os << "================================================================";
00390         }CATCH_BLOCK("Failed to generate PHY string representation of GPS object!")
00391
00392         return os.str();
00393     }
00394
00395     float horizontal_dilution_of_precision() const {
00396         return horizontal_dilution_of_precision_;
00397     }
00398
00399     uint8_t id_type() const {
00400         return id_xmsg_type();
00401     }
00402
00403     uint8_t num_satellites() const {
00404         return num_satellites_;
00405     }
00406
00407     /** Save this GPS message object to the SQL database.
00408      *
00409      *  @throw          util::log::MessageException if an error occurs.
00410      *
00411      */
00412     void SaveObjectToDatabase() const {
00413
00414         try {
00415
00416             sql_table_description_.SaveObjectToDatabase(*this);
00417
00418         }CATCH_BLOCK("Failed to save GPS object to database!")
00419     }
00420
00421     /** Select all GPS message objects from the SQL database table.
00422      *
00423      *  @param[in]  groupBy     The SQL select statement "group-by expression" clause.
00424      *  @param[in]  orderBy     The SQL select statement "order-by expression" clause.
00425      *  @param[in]  limit       The SQL select statement result set row limit value (0 = no limit).
00426      *  @param[in]  offset      The SQL select statement result set row offset value.
00427      *
00428      *  @throw      util::log::MessageException if the query fails.
00429      *
00430      *  @return     The GPS message objects.
00431      */
00432     static vector<shared_ptr<XGPS> > SelectAllObjectsFromDatabase(
00433             const string& groupBy = "",
00434             const string& orderBy = "",
00435             const unsigned int& limit = 0,
00436             const unsigned int& offset = 0) {
00437
00438         try {
00439
00440             return sql_table_description_.SelectAllObjectsFromDatabase(
00441                     groupBy,
00442                     orderBy,
00443                     limit,
00444                     offset);
00445
00446         }CATCH_BLOCK("Failed to query GPS message objects from the database!")
00447
00448         return vector<shared_ptr<XGPS> >();
00449     }
00450
00451     /** Select a single GPS message object from the SQL database table.
00452      *
00453      *  @param[in]  where       The SQL select statement "where-condition" clause.
00454      *  @param[in]  groupBy     The SQL select statement "group-by expression" clause.
00455      *  @param[in]  orderBy     The SQL select statement "order-by expression" clause.
00456      *  @param[in]  offset      The SQL select statement result set row offset value.
00457      *
00458      *  @throw      util::log::MessageException if the query fails.
00459      *
00460      *  @return     The first GPS message object matching the query.
00461      */
00462     static shared_ptr<XGPS> SelectObjectFromDatabase(
00463             const string& where = "",
00464             const string& groupBy = "",
00465             const string& orderBy = "",
00466             const unsigned int& offset = 0) {
00467
00468         try {
00469
00470             return sql_table_description_.SelectObjectFromDatabase(where, groupBy, orderBy, offset);
00471
00472         }CATCH_BLOCK("Failed to query GPS message objects from the database!")
00473
00474         return make_shared<XGPS>();
00475     }
00476
00477     /** Select GPS message objects from the SQL database table.
00478      *
00479      *  @exception  util::log::MessageException if an error occurs while querying the database
00480      *              or constructing the GPS message objects from the result of the query.
00481      *
00482      *  @param[in]  where       The SQL select statement "where-condition" clause.
00483      *  @param[in]  groupBy     The SQL select statement "group-by expression" clause.
00484      *  @param[in]  orderBy     The SQL select statement "order-by expression" clause.
00485      *  @param[in]  limit       The SQL select statement result set row limit value.
```

```
00486      *  @param[in]  offset      The SQL select statement result set row offset value.
00487      *
00488      *  @return     A vector of shared references to the GPS message objects constructed from the
00489      *              results of the SQL query.
00490      */
00491     static vector<shared_ptr<XGPS> > SelectObjectsFromDatabase(
00492             const string& where = "",
00493             const string& groupBy = "",
00494             const string& orderBy = "",
00495             const unsigned int& limit = 0,
00496             const unsigned int& offset = 0) {
00497
00498         try {
00499
00500             return sql_table_description_.SelectObjectsFromDatabase(
00501                     where,
00502                     groupBy,
00503                     orderBy,
00504                     limit,
00505                     offset);
00506
00507         }CATCH_BLOCK("Failed to query GPS message objects from the database!")
00508
00509         return vector<shared_ptr<XGPS> >();
00510     }
00511
00512     /** Select a single value from the GPS message SQL database table.
00513      *
00514      *  @exception  util::log::MessageException if the SQL database query fails or if an error
00515      *              occurs while attempting to interpret the result of the query.
00516      *
00517      *  @note       The result set of this query is always limited to a single result. That is,
00518      *              the result set row limit value is automatically set to one. If more than one
00519      *              row in the database table satisfy the "where condition" clause of the select
00520      *              statement, then only the value of the first result will be returned.
00521      *
00522      *  @param[in]  what        The SQL select statement "what" clause. This clause may only
00523      *                          contain a singular result e.g. the name of the SQL database table
00524      *                          column of interest.
00525      *  @param[in]  where       The SQL select statement "where condition" clause.
00526      *  @param[in]  groupBy     The SQL select statement "group-by expression" clause.
00527      *  @param[in]  orderBy     The SQL select statement "order-by expression" clause.
00528      *  @param[in]  offset      The SQL select statement result set row offset value.
00529      *
00530      *  @return     The selected value from the SQL database table.
00531      */
00532     template<typename TT> static TT SelectValueFromDatabase(
00533             const string& what,
00534             const string& where = "",
00535             const string& groupBy = "",
00536             const string& orderBy = "",
00537             const unsigned int& offset = 0) {
00538
00539         return sql_table_description_.SelectValueFromDatabase<TT>(
00540                 what,
00541                 where,
00542                 groupBy,
00543                 orderBy,
00544                 offset);
00545
00546     }
00547
00548     uint8_t signal_to_noise_max() const {
00549         return signal_to_noise_max_;
00550     }
00551
00552     uint8_t signal_to_noise_mean() const {
00553         return signal_to_noise_mean_;
00554     }
00555
00556     uint8_t signal_to_noise_min() const {
00557         return signal_to_noise_min_;
00558     }
00559
00560     uint16_t time_to_acquire_fix() const {
00561         return time_to_acquire_fix_;
00562     }
00563
00564 };
00565
00566 } /* namespace xmsg */
00567 } /* namespace util */
00568 #endif /* XGPS_H_ */
```

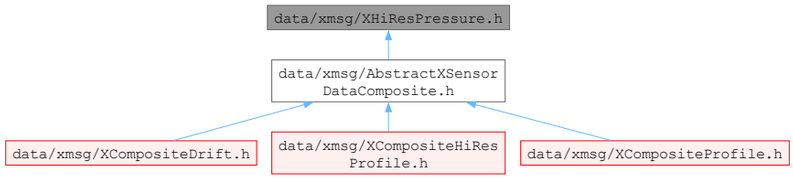
11.131 data/xmsg/XHiResPressure.h File Reference

The util::xmsg::XHiResPressure class header file.

```
#include "AbstractXSensorData.h"
```
Include dependency graph for XHiResPressure.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XHiResPressure

  Class representing high-resolution/raw/unbinned profile pressure X-Message.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.131.1 Detailed Description

The util::xmsg::XHiResPressure class header file.

Version

0.6

Date

Jul 1, 2014 1:58:33 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.132 XHiResPressure.h

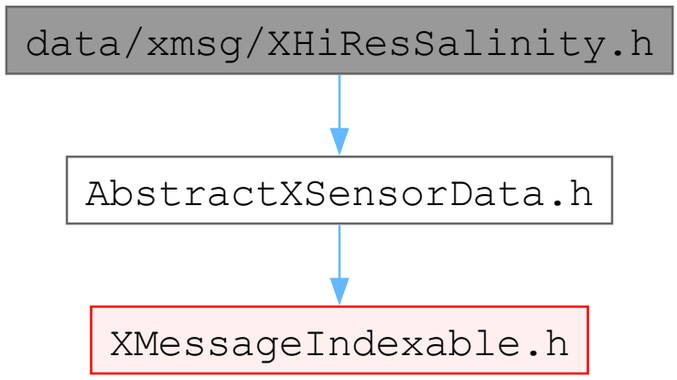
Go to the documentation of this file.

```
00001 /** The @ref util::xmsg::XHiResPressure class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jul 1, 2014 1:58:33 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef XHIRESPRESSURE_H_
00024 #define XHIRESPRESSURE_H_
00025
00026 /* Includes */
00027
00028 #include "AbstractXSensorData.h"
00029
00030 /* Definitions */
00031
00032 namespace util {
00033 namespace xmsg {
00034
00035 /** Class representing high-resolution/raw/unbinned profile pressure X-Message.
00036  *
00037  */
00038 class XHiResPressure: public AbstractXSensorData {
00039
00040 public:
00041
00042     /** The parameterized constructor.
00043      *
00044      *  @param[in]      dive        The dive or cycle number of this message.
00045      *  @param[in]      whoi        The unique WHOI identifier.
00046      *  @param[in]      data        The binary X-message data to decode.
00047      *
00048      *  @throw          util::log::MessageException if an decode error occurs.
00049      */
00050     XHiResPressure(int16_t dive, uint16_t whoi, const vector<uint8_t>& data) :
00051             AbstractXSensorData(dive, whoi, data) {
00052         DecodeXMsgData(data); // Decode message.
00053     }
00054
00055     /** The destructor.
00056      *
00057      */
00058     ~XHiResPressure() {
00059         /* Do nothing. */
00060     }
00061
00062     /** Decode the high-resolution/raw/unbinned profile pressure X-Message from the binary message data.
00063      *
00064      *  @param[in]      data        The binary message data to decode.
00065      *
00066      *  @throw          util::log::MessageException if the message is invalid or an error occurs during
00067      *                  decoding.
00068      */
00069     void DecodeXMsgData(const vector<uint8_t>& data) {
00070         try {
00071             const uint8_t* d = CheckXMsgData(data, (int) -1, (int) 5);
00072             if((d[0] & 0xF0) != (XMSG_TYPE_ID_PRES_HI_RES0 & 0xF0))
00073                 EXC_THROW("Invalid X-Message unique identifier!");
00074             UnpackSensorData(data);
00075         }CATCH_BLOCK("Failed to decode high resolution profile pressure X message!")
00076     }
00077 };
00078
00079 } /* namespace xmsg */
00080 } /* namespace util */
00081 #endif /* XHIRESPRESSURE_H_ */
```

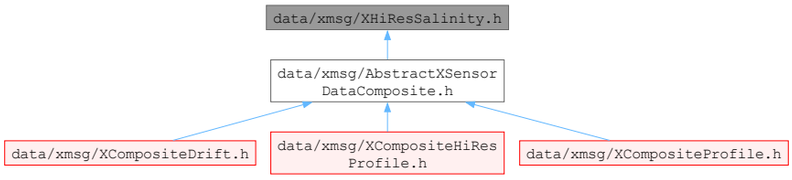
11.133 data/xmsg/XHiResSalinity.h File Reference

The util::xmsg::XHiResSalinity class header file.

```
#include "AbstractXSensorData.h"
```
Include dependency graph for XHiResSalinity.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XHiResSalinity

  Class representing high-resolution/raw/unbinned profile salinity X-Message.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.133.1 Detailed Description

The util::xmsg::XHiResSalinity class header file.

Version

0.6

Date

Jul 1, 2014 1:59:17 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.134 XHiResSalinity.h

Go to the documentation of this file.

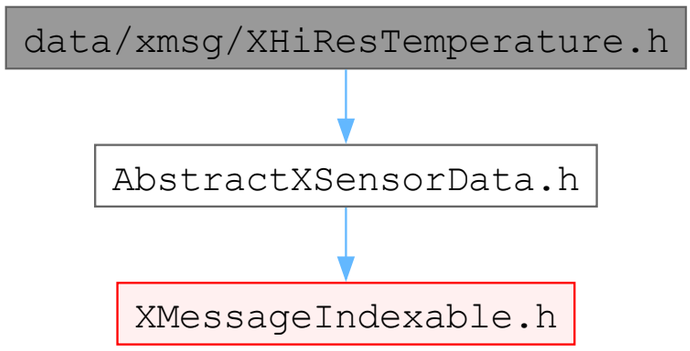
```
00001 /** The @ref util::xmsg::XHiResSalinity class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jul 1, 2014 1:59:17 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef XHIRESSALINITY_H_
00024 #define XHIRESSALINITY_H_
00025
00026 /* Includes */
00027
00028 #include "AbstractXSensorData.h"
00029
00030 /* Definitions */
00031
00032 namespace util {
00033 namespace xmsg {
00034
00035 /** Class representing high-resolution/raw/unbinned profile salinity X-Message.
00036  *
00037  */
00038 class XHiResSalinity:
00039         public AbstractXSensorData {
00040
00041 public:
00042
00043     /* Member Functions */
00044
00045     /** Parameterized constructor.
00046      *
00047      *  @param[in]      dive        The dive or cycle number of this message.
00048      *  @param[in]      whoi        The unique WHOI identifier.
00049      *  @param[in]      data        The binary X-message data to decode.
00050      *
00051      *  @throw          util::log::MessageException if an decode error occurs.
00052      */
00053     XHiResSalinity(int16_t dive, uint16_t whoi, const vector<uint8_t>& data) :
00054                     AbstractXSensorData(dive, whoi, data) {
00055         DecodeXMsgData(data); // Decode message.
00056     }
00057
00058     /** Destructor. */
00059     ~XHiResSalinity() {
00060         /* Do nothing. */
00061     }
00062
00063     /** Decode the high-resolution/raw/unbinned profile salinity X-Message from the binary message data.
00064      *
00065      *  @param[in]      data        The binary message data to decode.
00066      *
00067      *  @throw          util::log::MessageException if the message is invalid or an error occurs during
00068      *                  decoding.
00069      */
00070     void DecodeXMsgData(const vector<uint8_t>& data) {
00071         try {
00072             const uint8_t* d = CheckXMsgData(data, (int) -1, (int) 5);
00073             if((d[0] & 0xF0) != (XMSG_TYPE_ID_SALN_HI_RES0 & 0xF0))
00074                 EXC_THROW("Invalid X-Message unique identifier!");
00075             UnpackSensorData(data);
00076         }CATCH_BLOCK("Failed to decode high resolution profile salinity X message!")
00077     }
00078
00079 };
00080
00081 } /* namespace xmsg */
00082 } /* namespace util */
00083 #endif /* XHIRESSALINITY_H_ */
```

11.135 data/xmsg/XHiResTemperature.h File Reference

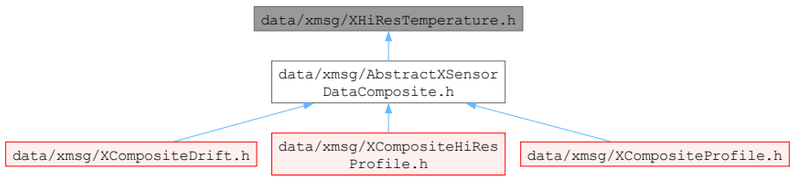
The util::xmsg::XHiResTemperature class header file.

```
#include "AbstractXSensorData.h"
```

Include dependency graph for XHiResTemperature.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XHiResTemperature

  Class representing high-resolution/raw/unbinned profile temperature X-Message.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.135.1 Detailed Description

The util::xmsg::XHiResTemperature class header file.

Version

0.6

Date

Jul 1, 2014 1:58:53 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.136 XHiResTemperature.h

Go to the documentation of this file.

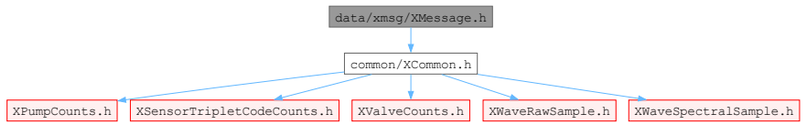
```
00001 /** The @ref util::xmsg::XHiResTemperature class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jul 1, 2014 1:58:53 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef XHIRESTEMPERATURE_H_
00024 #define XHIRESTEMPERATURE_H_
00025
00026 /* Includes */
00027
00028 #include "AbstractXSensorData.h"
00029
00030 namespace util {
00031 namespace xmsg {
00032
00033 /** Class representing  high-resolution/raw/unbinned profile temperature X-Message.
00034  *
00035  */
00036 class XHiResTemperature:
00037         public AbstractXSensorData {
00038
00039 public:
00040
00041     /* Member Functions */
00042
00043     /** Parameterized constructor.
00044      *
00045      *  @param[in]      dive        The dive or cycle number of this message.
00046      *  @param[in]      whoi        The unique WHOI identifier.
00047      *  @param[in]      data        The binary X-message data to decode.
00048      *
00049      *  @throw          util::log::MessageException if an decode error occurs.
00050      */
00051     XHiResTemperature(int16_t dive, uint16_t whoi, const vector<uint8_t>& data) :
00052                     AbstractXSensorData(dive, whoi, data) {
00053         DecodeXMsgData(data); // Decode message.
00054     }
00055
00056     /** Destructor. */
00057     ~XHiResTemperature() {
00058         /* Do nothing. */
00059     }
00060
00061     /** Decode the high-resolution/raw/unbinned profile temperature X-Message from the binary message
00062      *  data.
00063      *
00064      *  @param[in]      data        The binary message data to decode.
00065      *
00066      *  @throw          util::log::MessageException if the message is invalid or an error occurs during
00067      *                  decoding.
00068      */
00069     void DecodeXMsgData(const vector<uint8_t>& data) {
00070         try {
00071             const uint8_t* d = CheckXMsgData(data, (int) -1, (int) 5);
00072             if((d[0] & 0xF0) != (XMSG_TYPE_ID_TEMP_HI_RES0 & 0xF0))
00073                 EXC_THROW("Invalid X-Message unique identifier!");
00074             UnpackSensorData(data);
00075         }CATCH_BLOCK("Failed to decode high resolution profile temperature X message!")
00076     }
00077
00078 };
00079
00080 } /* namespace xmsg */
00081 } /* namespace util */
00082 #endif /* XHIRESTEMPERATURE_H_ */
```

11.137 data/xmsg/XMessage.h File Reference

The util::xmsg::XMessage class header file.

```
#include "common/XCommon.h"
```

Include dependency graph for XMessage.h:



This graph shows which files directly or indirectly include this file:

Classes

- class util::xmsg::XMessage

  Class representing a message following the X format specification.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

Enumerations

- enum util::xmsg::XMsgTypeId_t {
  util::xmsg::XMSG_TYPE_ID_GPS_DIAGNOSTIC = 0x00 ,
  util::xmsg::XMSG_TYPE_ID_GPS_STARTOFDIVE = 0x01 ,
  util::xmsg::XMSG_TYPE_ID_GPS_ENDOFDIVE = 0x02 ,
  util::xmsg::XMSG_TYPE_ID_GPS_ABORT = 0x03 ,
  util::xmsg::XMSG_TYPE_ID_GPS_BITEST = 0x05 ,
  util::xmsg::XMSG_TYPE_ID_PRES0 = 0x10 ,
  util::xmsg::XMSG_TYPE_ID_PRES1 = 0x11 ,
  util::xmsg::XMSG_TYPE_ID_PRES2 = 0x12 ,
  util::xmsg::XMSG_TYPE_ID_PRES3 = 0x13 ,
  util::xmsg::XMSG_TYPE_ID_PRES4 = 0x14 ,
  util::xmsg::XMSG_TYPE_ID_PRES5 = 0x15 ,
  util::xmsg::XMSG_TYPE_ID_PRES6 = 0x16 ,
  util::xmsg::XMSG_TYPE_ID_PRES7 = 0x17 ,
  util::xmsg::XMSG_TYPE_ID_PRES8 = 0x18 ,
  util::xmsg::XMSG_TYPE_ID_PRES9 = 0x19 ,
  util::xmsg::XMSG_TYPE_ID_PRESA = 0x1A ,
  util::xmsg::XMSG_TYPE_ID_PRESB = 0x1B ,
  util::xmsg::XMSG_TYPE_ID_PRESC = 0x1C ,
  util::xmsg::XMSG_TYPE_ID_PRESD = 0x1D ,
  util::xmsg::XMSG_TYPE_ID_PRESE = 0x1E ,
  util::xmsg::XMSG_TYPE_ID_PRESF = 0x1F ,
  util::xmsg::XMSG_TYPE_ID_TEMP0 = 0x20 ,
  util::xmsg::XMSG_TYPE_ID_TEMP1 = 0x21 ,
  util::xmsg::XMSG_TYPE_ID_TEMP2 = 0x22 ,
  util::xmsg::XMSG_TYPE_ID_TEMP3 = 0x23 ,
  util::xmsg::XMSG_TYPE_ID_TEMP4 = 0x24 ,
  util::xmsg::XMSG_TYPE_ID_TEMP5 = 0x25 ,
  util::xmsg::XMSG_TYPE_ID_TEMP6 = 0x26 ,
  util::xmsg::XMSG_TYPE_ID_TEMP7 = 0x27 ,
  util::xmsg::XMSG_TYPE_ID_TEMP8 = 0x28 ,
  util::xmsg::XMSG_TYPE_ID_TEMP9 = 0x29 ,
  util::xmsg::XMSG_TYPE_ID_TEMPA = 0x2A ,
  util::xmsg::XMSG_TYPE_ID_TEMPB = 0x2B ,
  util::xmsg::XMSG_TYPE_ID_TEMPC = 0x2C ,
  util::xmsg::XMSG_TYPE_ID_TEMPD = 0x2D ,

util::xmsg::XMSG_TYPE_ID_TEMPE = 0x2E ,
util::xmsg::XMSG_TYPE_ID_TEMPF = 0x2F ,
util::xmsg::XMSG_TYPE_ID_SALN0 = 0x30 ,
util::xmsg::XMSG_TYPE_ID_SALN1 = 0x31 ,
util::xmsg::XMSG_TYPE_ID_SALN2 = 0x32 ,
util::xmsg::XMSG_TYPE_ID_SALN3 = 0x33 ,
util::xmsg::XMSG_TYPE_ID_SALN4 = 0x34 ,
util::xmsg::XMSG_TYPE_ID_SALN5 = 0x35 ,
util::xmsg::XMSG_TYPE_ID_SALN6 = 0x36 ,
util::xmsg::XMSG_TYPE_ID_SALN7 = 0x37 ,
util::xmsg::XMSG_TYPE_ID_SALN8 = 0x38 ,
util::xmsg::XMSG_TYPE_ID_SALN9 = 0x39 ,
util::xmsg::XMSG_TYPE_ID_SALNA = 0x3A ,
util::xmsg::XMSG_TYPE_ID_SALNB = 0x3B ,
util::xmsg::XMSG_TYPE_ID_SALNC = 0x3C ,
util::xmsg::XMSG_TYPE_ID_SALND = 0x3D ,
util::xmsg::XMSG_TYPE_ID_SALNE = 0x3E ,
util::xmsg::XMSG_TYPE_ID_SALNF = 0x3F ,
util::xmsg::XMSG_TYPE_ID_FALL0 = 0x40 ,
util::xmsg::XMSG_TYPE_ID_FALL1 = 0x41 ,
util::xmsg::XMSG_TYPE_ID_FALL2 = 0x42 ,
util::xmsg::XMSG_TYPE_ID_FALL3 = 0x43 ,
util::xmsg::XMSG_TYPE_ID_FALL4 = 0x44 ,
util::xmsg::XMSG_TYPE_ID_FALL5 = 0x45 ,
util::xmsg::XMSG_TYPE_ID_FALL6 = 0x46 ,
util::xmsg::XMSG_TYPE_ID_FALL7 = 0x47 ,
util::xmsg::XMSG_TYPE_ID_FALL8 = 0x48 ,
util::xmsg::XMSG_TYPE_ID_FALL9 = 0x49 ,
util::xmsg::XMSG_TYPE_ID_FALLA = 0x4A ,
util::xmsg::XMSG_TYPE_ID_FALLB = 0x4B ,
util::xmsg::XMSG_TYPE_ID_FALLC = 0x4C ,
util::xmsg::XMSG_TYPE_ID_FALLD = 0x4D ,
util::xmsg::XMSG_TYPE_ID_FALLE = 0x4E ,
util::xmsg::XMSG_TYPE_ID_FALLF = 0x4F ,
util::xmsg::XMSG_TYPE_ID_RISE0 = 0x50 ,
util::xmsg::XMSG_TYPE_ID_RISE1 = 0x51 ,
util::xmsg::XMSG_TYPE_ID_RISE2 = 0x52 ,
util::xmsg::XMSG_TYPE_ID_RISE3 = 0x53 ,
util::xmsg::XMSG_TYPE_ID_RISE4 = 0x54 ,
util::xmsg::XMSG_TYPE_ID_RISE5 = 0x55 ,
util::xmsg::XMSG_TYPE_ID_RISE6 = 0x56 ,
util::xmsg::XMSG_TYPE_ID_RISE7 = 0x57 ,
util::xmsg::XMSG_TYPE_ID_RISE8 = 0x58 ,
util::xmsg::XMSG_TYPE_ID_RISE9 = 0x59 ,
util::xmsg::XMSG_TYPE_ID_RISEA = 0x5A ,
util::xmsg::XMSG_TYPE_ID_RISEB = 0x5B ,
util::xmsg::XMSG_TYPE_ID_RISEC = 0x5C ,
util::xmsg::XMSG_TYPE_ID_RISED = 0x5D ,
util::xmsg::XMSG_TYPE_ID_RISEE = 0x5E ,
util::xmsg::XMSG_TYPE_ID_RISEF = 0x5F ,
util::xmsg::XMSG_TYPE_ID_PUMP0 = 0x60 ,
util::xmsg::XMSG_TYPE_ID_PUMP1 = 0x61 ,
util::xmsg::XMSG_TYPE_ID_PUMP2 = 0x62 ,
util::xmsg::XMSG_TYPE_ID_PUMP3 = 0x63 ,
util::xmsg::XMSG_TYPE_ID_PUMP4 = 0x64 ,
util::xmsg::XMSG_TYPE_ID_PUMP5 = 0x65 ,
util::xmsg::XMSG_TYPE_ID_PUMP6 = 0x66 ,
util::xmsg::XMSG_TYPE_ID_PUMP7 = 0x67 ,
util::xmsg::XMSG_TYPE_ID_PUMP8 = 0x68 ,
util::xmsg::XMSG_TYPE_ID_PUMP9 = 0x69 ,
util::xmsg::XMSG_TYPE_ID_PUMPA = 0x6A ,
util::xmsg::XMSG_TYPE_ID_PUMPB = 0x6B ,
util::xmsg::XMSG_TYPE_ID_PUMPC = 0x6C ,
util::xmsg::XMSG_TYPE_ID_PUMPD = 0x6D ,
util::xmsg::XMSG_TYPE_ID_PUMPE = 0x6E ,
util::xmsg::XMSG_TYPE_ID_PUMPF = 0x6F ,
util::xmsg::XMSG_TYPE_ID_PRES_SURF0 = 0x80 ,
util::xmsg::XMSG_TYPE_ID_PRES_SURF1 = 0x81 ,
util::xmsg::XMSG_TYPE_ID_PRES_SURF2 = 0x82 ,
util::xmsg::XMSG_TYPE_ID_PRES_SURF3 = 0x83 ,
util::xmsg::XMSG_TYPE_ID_PRES_SURF4 = 0x84 ,
util::xmsg::XMSG_TYPE_ID_PRES_SURF5 = 0x85 ,

util::xmsg::XMSG_TYPE_ID_PRES_SURF6 = 0x86 ,
util::xmsg::XMSG_TYPE_ID_PRES_SURF7 = 0x87 ,
util::xmsg::XMSG_TYPE_ID_PRES_SURF8 = 0x88 ,
util::xmsg::XMSG_TYPE_ID_PRES_SURF9 = 0x89 ,
util::xmsg::XMSG_TYPE_ID_PRES_SURFA = 0x8A ,
util::xmsg::XMSG_TYPE_ID_PRES_SURFB = 0x8B ,
util::xmsg::XMSG_TYPE_ID_PRES_SURFC = 0x8C ,
util::xmsg::XMSG_TYPE_ID_PRES_SURFD = 0x8D ,
util::xmsg::XMSG_TYPE_ID_PRES_SURFE = 0x8E ,
util::xmsg::XMSG_TYPE_ID_PRES_SURFF = 0x8F ,
util::xmsg::XMSG_TYPE_ID_PRES_HI_RES0 = 0x90 ,
util::xmsg::XMSG_TYPE_ID_PRES_HI_RES1 = 0x91 ,
util::xmsg::XMSG_TYPE_ID_PRES_HI_RES2 = 0x92 ,
util::xmsg::XMSG_TYPE_ID_PRES_HI_RES3 = 0x93 ,
util::xmsg::XMSG_TYPE_ID_PRES_HI_RES4 = 0x94 ,
util::xmsg::XMSG_TYPE_ID_PRES_HI_RES5 = 0x95 ,
util::xmsg::XMSG_TYPE_ID_PRES_HI_RES6 = 0x96 ,
util::xmsg::XMSG_TYPE_ID_PRES_HI_RES7 = 0x97 ,
util::xmsg::XMSG_TYPE_ID_PRES_DRIFT8 = 0x98 ,
util::xmsg::XMSG_TYPE_ID_PRES_DRIFT9 = 0x99 ,
util::xmsg::XMSG_TYPE_ID_PRES_DRIFTA = 0x9A ,
util::xmsg::XMSG_TYPE_ID_PRES_DRIFTB = 0x9B ,
util::xmsg::XMSG_TYPE_ID_PRES_DRIFTC = 0x9C ,
util::xmsg::XMSG_TYPE_ID_PRES_DRIFTD = 0x9D ,
util::xmsg::XMSG_TYPE_ID_PRES_DRIFTE = 0x9E ,
util::xmsg::XMSG_TYPE_ID_PRES_DRIFTF = 0x9F ,
util::xmsg::XMSG_TYPE_ID_TEMP_HI_RES0 = 0xA0 ,
util::xmsg::XMSG_TYPE_ID_TEMP_HI_RES1 = 0xA1 ,
util::xmsg::XMSG_TYPE_ID_TEMP_HI_RES2 = 0xA2 ,
util::xmsg::XMSG_TYPE_ID_TEMP_HI_RES3 = 0xA3 ,
util::xmsg::XMSG_TYPE_ID_TEMP_HI_RES4 = 0xA4 ,
util::xmsg::XMSG_TYPE_ID_TEMP_HI_RES5 = 0xA5 ,
util::xmsg::XMSG_TYPE_ID_TEMP_HI_RES6 = 0xA6 ,
util::xmsg::XMSG_TYPE_ID_TEMP_HI_RES7 = 0xA7 ,
util::xmsg::XMSG_TYPE_ID_TEMP_DRIFT8 = 0xA8 ,
util::xmsg::XMSG_TYPE_ID_TEMP_DRIFT9 = 0xA9 ,
util::xmsg::XMSG_TYPE_ID_TEMP_DRIFTA = 0xAA ,
util::xmsg::XMSG_TYPE_ID_TEMP_DRIFTB = 0xAB ,
util::xmsg::XMSG_TYPE_ID_TEMP_DRIFTC = 0xAC ,
util::xmsg::XMSG_TYPE_ID_TEMP_DRIFTD = 0xAD ,
util::xmsg::XMSG_TYPE_ID_TEMP_DRIFTE = 0xAE ,
util::xmsg::XMSG_TYPE_ID_TEMP_DRIFTF = 0xAF ,
util::xmsg::XMSG_TYPE_ID_SALN_HI_RES0 = 0xB0 ,
util::xmsg::XMSG_TYPE_ID_SALN_HI_RES1 = 0xB1 ,
util::xmsg::XMSG_TYPE_ID_SALN_HI_RES2 = 0xB2 ,
util::xmsg::XMSG_TYPE_ID_SALN_HI_RES3 = 0xB3 ,
util::xmsg::XMSG_TYPE_ID_SALN_HI_RES4 = 0xB4 ,
util::xmsg::XMSG_TYPE_ID_SALN_HI_RES5 = 0xB5 ,
util::xmsg::XMSG_TYPE_ID_SALN_HI_RES6 = 0xB6 ,
util::xmsg::XMSG_TYPE_ID_SALN_HI_RES7 = 0xB7 ,
util::xmsg::XMSG_TYPE_ID_SALN_DRIFT8 = 0xB8 ,
util::xmsg::XMSG_TYPE_ID_SALN_DRIFT9 = 0xB9 ,
util::xmsg::XMSG_TYPE_ID_SALN_DRIFTA = 0xBA ,
util::xmsg::XMSG_TYPE_ID_SALN_DRIFTB = 0xBB ,
util::xmsg::XMSG_TYPE_ID_SALN_DRIFTC = 0xBC ,
util::xmsg::XMSG_TYPE_ID_SALN_DRIFTD = 0xBD ,
util::xmsg::XMSG_TYPE_ID_SALN_DRIFTE = 0xBE ,
util::xmsg::XMSG_TYPE_ID_SALN_DRIFTF = 0xBF ,
util::xmsg::XMSG_TYPE_ID_ALAMO_WAVE_C1 = 0xC1 ,
util::xmsg::XMSG_TYPE_ID_AUX_SENSOR_DATA = 0xC2 ,
util::xmsg::XMSG_TYPE_ID_ALAMO_WAVE_C3 = 0xC3 ,
util::xmsg::XMSG_TYPE_ID_MIZ_ACOUSTIC = 0xCA ,
util::xmsg::XMSG_TYPE_ID_MISS_CONF0 = 0xD0 ,
util::xmsg::XMSG_TYPE_ID_MISS_CONF1 = 0xD1 ,
util::xmsg::XMSG_TYPE_ID_MISS_CONF2 = 0xD2 ,
util::xmsg::XMSG_TYPE_ID_MISS_CONF3 = 0xD3 ,
util::xmsg::XMSG_TYPE_ID_MISS_CONF4 = 0xD4 ,
util::xmsg::XMSG_TYPE_ID_MISS_CONF5 = 0xD5 ,
util::xmsg::XMSG_TYPE_ID_MISS_CONF6 = 0xD6 ,
util::xmsg::XMSG_TYPE_ID_MISS_CONF7 = 0xD7 ,
util::xmsg::XMSG_TYPE_ID_MISS_CMD_ECHO = 0xDE ,
util::xmsg::XMSG_TYPE_ID_ENG_DIAGNOSTIC = 0xE0 ,

util::xmsg::XMSG_TYPE_ID_ENG_DIVE = 0xE2 ,
util::xmsg::XMSG_TYPE_ID_ENG_ABORT = 0xE3 ,
util::xmsg::XMSG_TYPE_ID_ENG_BIT_PASS = 0xE5 ,
util::xmsg::XMSG_TYPE_ID_ENG_BIT_FAIL = 0xE6 ,
XMSG_TYPE_ID_ENG_DESCENT = 0xE7 ,
XMSG_TYPE_ID_ENG_PARK = 0xE8 ,
XMSG_TYPE_ID_ENG_ASCENT = 0xE9 ,
XMSG_TYPE_ID_ENG_SURFACE = 0xEA ,
XMSG_TYPE_ID_ENG_VALVE = 0xEB ,
XMSG_TYPE_ID_ENG_ICE = 0xEC ,
util::xmsg::XMSG_TYPE_ID_ARGO_DATA = 0xF0 ,
util::xmsg::XMSG_TYPE_ID_TEST = 0xF1 ,
util::xmsg::XMSG_TYPE_ID_UNKNOWN = 0xFF }

Enumeration of the unique X message identifiers.

11.137.1 Detailed Description

The util::xmsg::XMessage class header file.

Version

0.6

Date

Jun 9, 2014 2:43:58 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.138 XMessage.h

Go to the documentation of this file.

```
00001 /** The @ref util::xmsg::XMessage class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jun 9, 2014 2:43:58 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022 #ifndef XMSG_H_
00023 #define XMSG_H_
00024
00025 /* Includes */
00026
00027 // local
00028 #include "common/XCommon.h"
00029
00030 /* Definitions */
00031
```

```
00032 namespace util {
00033   namespace xmsg {
00034
00035     /* Type Definitions */
00036
00037     /** Enumeration of the unique X message identifiers.
00038      *
00039      */
00040     typedef enum XMsgTypeId {
00041       XMSG_TYPE_ID_GPS_DIAGNOSTIC = 0x00,    //!< GPS0
00042       XMSG_TYPE_ID_GPS_STARTOFDIVE = 0x01,    //!< GPS1
00043       XMSG_TYPE_ID_GPS_ENDOFDIVE = 0x02,    //!< GPS2
00044       XMSG_TYPE_ID_GPS_ABORT = 0x03,    //!< GPS3
00045       XMSG_TYPE_ID_GPS_BITEST = 0x05,    //!< GPS5
00046 //      = 0x06,
00047 //      = 0x07,
00048 //      = 0x08,
00049 //      = 0x09,
00050 //      = 0x0A,
00051 //      = 0x0B,
00052 //      = 0x0C,
00053 //      = 0x0D,
00054 //      = 0x0E,
00055 //      = 0x0F,
00056       XMSG_TYPE_ID_PRES0 = 0x10,    //!< PRES0
00057       XMSG_TYPE_ID_PRES1 = 0x11,    //!< PRES1
00058       XMSG_TYPE_ID_PRES2 = 0x12,    //!< PRES2
00059       XMSG_TYPE_ID_PRES3 = 0x13,    //!< PRES3
00060       XMSG_TYPE_ID_PRES4 = 0x14,    //!< PRES4
00061       XMSG_TYPE_ID_PRES5 = 0x15,    //!< PRES5
00062       XMSG_TYPE_ID_PRES6 = 0x16,    //!< PRES6
00063       XMSG_TYPE_ID_PRES7 = 0x17,    //!< PRES7
00064       XMSG_TYPE_ID_PRES8 = 0x18,    //!< PRES8
00065       XMSG_TYPE_ID_PRES9 = 0x19,    //!< PRES9
00066       XMSG_TYPE_ID_PRESA = 0x1A,    //!< PRESA
00067       XMSG_TYPE_ID_PRESB = 0x1B,    //!< PRESB
00068       XMSG_TYPE_ID_PRESC = 0x1C,    //!< PRESC
00069       XMSG_TYPE_ID_PRESD = 0x1D,    //!< PRESD
00070       XMSG_TYPE_ID_PRESE = 0x1E,    //!< PRESE
00071       XMSG_TYPE_ID_PRESF = 0x1F,    //!< PRESF
00072       XMSG_TYPE_ID_TEMP0 = 0x20,    //!< TEMP0
00073       XMSG_TYPE_ID_TEMP1 = 0x21,    //!< TEMP1
00074       XMSG_TYPE_ID_TEMP2 = 0x22,    //!< TEMP2
00075       XMSG_TYPE_ID_TEMP3 = 0x23,    //!< TEMP3
00076       XMSG_TYPE_ID_TEMP4 = 0x24,    //!< TEMP4
00077       XMSG_TYPE_ID_TEMP5 = 0x25,    //!< TEMP5
00078       XMSG_TYPE_ID_TEMP6 = 0x26,    //!< TEMP6
00079       XMSG_TYPE_ID_TEMP7 = 0x27,    //!< TEMP7
00080       XMSG_TYPE_ID_TEMP8 = 0x28,    //!< TEMP8
00081       XMSG_TYPE_ID_TEMP9 = 0x29,    //!< TEMP9
00082       XMSG_TYPE_ID_TEMPA = 0x2A,    //!< TEMPA
00083       XMSG_TYPE_ID_TEMPB = 0x2B,    //!< TEMPB
00084       XMSG_TYPE_ID_TEMPC = 0x2C,    //!< TEMPC
00085       XMSG_TYPE_ID_TEMPD = 0x2D,    //!< TEMPD
00086       XMSG_TYPE_ID_TEMPE = 0x2E,    //!< TEMPE
00087       XMSG_TYPE_ID_TEMPF = 0x2F,    //!< TEMPF
00088       XMSG_TYPE_ID_SALN0 = 0x30,    //!< SALN0
00089       XMSG_TYPE_ID_SALN1 = 0x31,    //!< SALN1
00090       XMSG_TYPE_ID_SALN2 = 0x32,    //!< SALN2
00091       XMSG_TYPE_ID_SALN3 = 0x33,    //!< SALN3
00092       XMSG_TYPE_ID_SALN4 = 0x34,    //!< SALN4
00093       XMSG_TYPE_ID_SALN5 = 0x35,    //!< SALN5
00094       XMSG_TYPE_ID_SALN6 = 0x36,    //!< SALN6
00095       XMSG_TYPE_ID_SALN7 = 0x37,    //!< SALN7
00096       XMSG_TYPE_ID_SALN8 = 0x38,    //!< SALN8
00097       XMSG_TYPE_ID_SALN9 = 0x39,    //!< SALN9
00098       XMSG_TYPE_ID_SALNA = 0x3A,    //!< SALNA
00099       XMSG_TYPE_ID_SALNB = 0x3B,    //!< SALNB
00100       XMSG_TYPE_ID_SALNC = 0x3C,    //!< SALNC
00101       XMSG_TYPE_ID_SALND = 0x3D,    //!< SALND
00102       XMSG_TYPE_ID_SALNE = 0x3E,    //!< SALNE
00103       XMSG_TYPE_ID_SALNF = 0x3F,    //!< SALNF
00104       XMSG_TYPE_ID_FALL0 = 0x40,    //!< FALL0
00105       XMSG_TYPE_ID_FALL1 = 0x41,    //!< FALL1
00106       XMSG_TYPE_ID_FALL2 = 0x42,    //!< FALL2
00107       XMSG_TYPE_ID_FALL3 = 0x43,    //!< FALL3
00108       XMSG_TYPE_ID_FALL4 = 0x44,    //!< FALL4
00109       XMSG_TYPE_ID_FALL5 = 0x45,    //!< FALL5
00110       XMSG_TYPE_ID_FALL6 = 0x46,    //!< FALL6
00111       XMSG_TYPE_ID_FALL7 = 0x47,    //!< FALL7
00112       XMSG_TYPE_ID_FALL8 = 0x48,    //!< FALL8
00113       XMSG_TYPE_ID_FALL9 = 0x49,    //!< FALL9
00114       XMSG_TYPE_ID_FALLA = 0x4A,    //!< FALLA
00115       XMSG_TYPE_ID_FALLB = 0x4B,    //!< FALLB
00116       XMSG_TYPE_ID_FALLC = 0x4C,    //!< FALLC
00117       XMSG_TYPE_ID_FALLD = 0x4D,    //!< FALLD
00118       XMSG_TYPE_ID_FALLE = 0x4E,    //!< FALLE
00119       XMSG_TYPE_ID_FALLF = 0x4F,    //!< FALLF
00120       XMSG_TYPE_ID_RISE0 = 0x50,    //!< XMSG_TYPE_ID_RISE0
00121       XMSG_TYPE_ID_RISE1 = 0x51,    //!< XMSG_TYPE_ID_RISE1
00122       XMSG_TYPE_ID_RISE2 = 0x52,    //!< XMSG_TYPE_ID_RISE2
00123       XMSG_TYPE_ID_RISE3 = 0x53,    //!< XMSG_TYPE_ID_RISE3
00124       XMSG_TYPE_ID_RISE4 = 0x54,    //!< XMSG_TYPE_ID_RISE4
00125       XMSG_TYPE_ID_RISE5 = 0x55,    //!< XMSG_TYPE_ID_RISE5
00126       XMSG_TYPE_ID_RISE6 = 0x56,    //!< XMSG_TYPE_ID_RISE6
00127       XMSG_TYPE_ID_RISE7 = 0x57,    //!< XMSG_TYPE_ID_RISE7
00128       XMSG_TYPE_ID_RISE8 = 0x58,    //!< XMSG_TYPE_ID_RISE8
00129       XMSG_TYPE_ID_RISE9 = 0x59,    //!< XMSG_TYPE_ID_RISE9
00130       XMSG_TYPE_ID_RISEA = 0x5A,    //!< XMSG_TYPE_ID_RISEA
00131       XMSG_TYPE_ID_RISEB = 0x5B,    //!< XMSG_TYPE_ID_RISEB
00132       XMSG_TYPE_ID_RISEC = 0x5C,    //!< XMSG_TYPE_ID_RISEC
00133       XMSG_TYPE_ID_RISED = 0x5D,    //!< XMSG_TYPE_ID_RISED
00134       XMSG_TYPE_ID_RISEE = 0x5E,    //!< XMSG_TYPE_ID_RISEE
00135       XMSG_TYPE_ID_RISEF = 0x5F,    //!< XMSG_TYPE_ID_RISEF
00136       XMSG_TYPE_ID_PUMP0 = 0x60,    //!< XMSG_TYPE_ID_PUMP0
00137       XMSG_TYPE_ID_PUMP1 = 0x61,    //!< XMSG_TYPE_ID_PUMP1
00138       XMSG_TYPE_ID_PUMP2 = 0x62,    //!< XMSG_TYPE_ID_PUMP2
00139       XMSG_TYPE_ID_PUMP3 = 0x63,    //!< XMSG_TYPE_ID_PUMP3
```

```
00140       XMSG_TYPE_ID_PUMP4 = 0x64,    //!< XMSG_TYPE_ID_PUMP4
00141       XMSG_TYPE_ID_PUMP5 = 0x65,    //!< XMSG_TYPE_ID_PUMP5
00142       XMSG_TYPE_ID_PUMP6 = 0x66,    //!< XMSG_TYPE_ID_PUMP6
00143       XMSG_TYPE_ID_PUMP7 = 0x67,    //!< XMSG_TYPE_ID_PUMP7
00144       XMSG_TYPE_ID_PUMP8 = 0x68,    //!< XMSG_TYPE_ID_PUMP8
00145       XMSG_TYPE_ID_PUMP9 = 0x69,    //!< XMSG_TYPE_ID_PUMP9
00146       XMSG_TYPE_ID_PUMPA = 0x6A,    //!< XMSG_TYPE_ID_PUMPA
00147       XMSG_TYPE_ID_PUMPB = 0x6B,    //!< XMSG_TYPE_ID_PUMPB
00148       XMSG_TYPE_ID_PUMPC = 0x6C,    //!< XMSG_TYPE_ID_PUMPC
00149       XMSG_TYPE_ID_PUMPD = 0x6D,    //!< XMSG_TYPE_ID_PUMPD
00150       XMSG_TYPE_ID_PUMPE = 0x6E,    //!< XMSG_TYPE_ID_PUMPE
00151       XMSG_TYPE_ID_PUMPF = 0x6F,    //!< XMSG_TYPE_ID_PUMPF
00152       XMSG_TYPE_ID_PRES_SURF0 = 0x80,    //!< XMSG_TYPE_ID_PRES_SURF0
00153       XMSG_TYPE_ID_PRES_SURF1 = 0x81,    //!< XMSG_TYPE_ID_PRES_SURF1
00154       XMSG_TYPE_ID_PRES_SURF2 = 0x82,    //!< XMSG_TYPE_ID_PRES_SURF2
00155       XMSG_TYPE_ID_PRES_SURF3 = 0x83,    //!< XMSG_TYPE_ID_PRES_SURF3
00156       XMSG_TYPE_ID_PRES_SURF4 = 0x84,    //!< XMSG_TYPE_ID_PRES_SURF4
00157       XMSG_TYPE_ID_PRES_SURF5 = 0x85,    //!< XMSG_TYPE_ID_PRES_SURF5
00158       XMSG_TYPE_ID_PRES_SURF6 = 0x86,    //!< XMSG_TYPE_ID_PRES_SURF6
00159       XMSG_TYPE_ID_PRES_SURF7 = 0x87,    //!< XMSG_TYPE_ID_PRES_SURF7
00160       XMSG_TYPE_ID_PRES_SURF8 = 0x88,    //!< XMSG_TYPE_ID_PRES_SURF8
00161       XMSG_TYPE_ID_PRES_SURF9 = 0x89,    //!< XMSG_TYPE_ID_PRES_SURF9
00162       XMSG_TYPE_ID_PRES_SURFA = 0x8A,    //!< XMSG_TYPE_ID_PRES_SURFA
00163       XMSG_TYPE_ID_PRES_SURFB = 0x8B,    //!< XMSG_TYPE_ID_PRES_SURFB
00164       XMSG_TYPE_ID_PRES_SURFC = 0x8C,    //!< XMSG_TYPE_ID_PRES_SURFC
00165       XMSG_TYPE_ID_PRES_SURFD = 0x8D,    //!< XMSG_TYPE_ID_PRES_SURFD
00166       XMSG_TYPE_ID_PRES_SURFE = 0x8E,    //!< XMSG_TYPE_ID_PRES_SURFE
00167       XMSG_TYPE_ID_PRES_SURFF = 0x8F,    //!< XMSG_TYPE_ID_PRES_SURFF
00168       XMSG_TYPE_ID_PRES_HI_RES0 = 0x90,    //!< XMSG_TYPE_ID_PRES_HI_RES0
00169       XMSG_TYPE_ID_PRES_HI_RES1 = 0x91,    //!< XMSG_TYPE_ID_PRES_HI_RES1
00170       XMSG_TYPE_ID_PRES_HI_RES2 = 0x92,    //!< XMSG_TYPE_ID_PRES_HI_RES2
00171       XMSG_TYPE_ID_PRES_HI_RES3 = 0x93,    //!< XMSG_TYPE_ID_PRES_HI_RES3
00172       XMSG_TYPE_ID_PRES_HI_RES4 = 0x94,    //!< XMSG_TYPE_ID_PRES_HI_RES4
00173       XMSG_TYPE_ID_PRES_HI_RES5 = 0x95,    //!< XMSG_TYPE_ID_PRES_HI_RES5
00174       XMSG_TYPE_ID_PRES_HI_RES6 = 0x96,    //!< XMSG_TYPE_ID_PRES_HI_RES6
00175       XMSG_TYPE_ID_PRES_HI_RES7 = 0x97,    //!< XMSG_TYPE_ID_PRES_HI_RES7
00176       XMSG_TYPE_ID_PRES_DRIFT8 = 0x98,    //!< XMSG_TYPE_ID_PRES_DRIFT8
00177       XMSG_TYPE_ID_PRES_DRIFT9 = 0x99,    //!< XMSG_TYPE_ID_PRES_DRIFT9
00178       XMSG_TYPE_ID_PRES_DRIFTA = 0x9A,    //!< XMSG_TYPE_ID_PRES_DRIFTA
00179       XMSG_TYPE_ID_PRES_DRIFTB = 0x9B,    //!< XMSG_TYPE_ID_PRES_DRIFTB
00180       XMSG_TYPE_ID_PRES_DRIFTC = 0x9C,    //!< XMSG_TYPE_ID_PRES_DRIFTC
00181       XMSG_TYPE_ID_PRES_DRIFTD = 0x9D,    //!< XMSG_TYPE_ID_PRES_DRIFTD
00182       XMSG_TYPE_ID_PRES_DRIFTE = 0x9E,    //!< XMSG_TYPE_ID_PRES_DRIFTE
00183       XMSG_TYPE_ID_PRES_DRIFTF = 0x9F,    //!< XMSG_TYPE_ID_PRES_DRIFTF
00184       XMSG_TYPE_ID_TEMP_HI_RES0 = 0xA0,    //!< XMSG_TYPE_ID_TEMP_HI_RES0
00185       XMSG_TYPE_ID_TEMP_HI_RES1 = 0xA1,    //!< XMSG_TYPE_ID_TEMP_HI_RES1
00186       XMSG_TYPE_ID_TEMP_HI_RES2 = 0xA2,    //!< XMSG_TYPE_ID_TEMP_HI_RES2
00187       XMSG_TYPE_ID_TEMP_HI_RES3 = 0xA3,    //!< XMSG_TYPE_ID_TEMP_HI_RES3
00188       XMSG_TYPE_ID_TEMP_HI_RES4 = 0xA4,    //!< XMSG_TYPE_ID_TEMP_HI_RES4
00189       XMSG_TYPE_ID_TEMP_HI_RES5 = 0xA5,    //!< XMSG_TYPE_ID_TEMP_HI_RES5
00190       XMSG_TYPE_ID_TEMP_HI_RES6 = 0xA6,    //!< XMSG_TYPE_ID_TEMP_HI_RES6
00191       XMSG_TYPE_ID_TEMP_HI_RES7 = 0xA7,    //!< XMSG_TYPE_ID_TEMP_HI_RES7
00192       XMSG_TYPE_ID_TEMP_DRIFT8 = 0xA8,    //!< XMSG_TYPE_ID_TEMP_DRIFT8
00193       XMSG_TYPE_ID_TEMP_DRIFT9 = 0xA9,    //!< XMSG_TYPE_ID_TEMP_DRIFT9
00194       XMSG_TYPE_ID_TEMP_DRIFTA = 0xAA,    //!< XMSG_TYPE_ID_TEMP_DRIFTA
00195       XMSG_TYPE_ID_TEMP_DRIFTB = 0xAB,    //!< XMSG_TYPE_ID_TEMP_DRIFTB
00196       XMSG_TYPE_ID_TEMP_DRIFTC = 0xAC,    //!< XMSG_TYPE_ID_TEMP_DRIFTC
00197       XMSG_TYPE_ID_TEMP_DRIFTD = 0xAD,    //!< XMSG_TYPE_ID_TEMP_DRIFTD
00198       XMSG_TYPE_ID_TEMP_DRIFTE = 0xAE,    //!< XMSG_TYPE_ID_TEMP_DRIFTE
00199       XMSG_TYPE_ID_TEMP_DRIFTF = 0xAF,    //!< XMSG_TYPE_ID_TEMP_DRIFTF
00200       XMSG_TYPE_ID_SALN_HI_RES0 = 0xB0,    //!< XMSG_TYPE_ID_SALN_HI_RES0
00201       XMSG_TYPE_ID_SALN_HI_RES1 = 0xB1,    //!< XMSG_TYPE_ID_SALN_HI_RES1
00202       XMSG_TYPE_ID_SALN_HI_RES2 = 0xB2,    //!< XMSG_TYPE_ID_SALN_HI_RES2
00203       XMSG_TYPE_ID_SALN_HI_RES3 = 0xB3,    //!< XMSG_TYPE_ID_SALN_HI_RES3
00204       XMSG_TYPE_ID_SALN_HI_RES4 = 0xB4,    //!< XMSG_TYPE_ID_SALN_HI_RES4
00205       XMSG_TYPE_ID_SALN_HI_RES5 = 0xB5,    //!< XMSG_TYPE_ID_SALN_HI_RES5
00206       XMSG_TYPE_ID_SALN_HI_RES6 = 0xB6,    //!< XMSG_TYPE_ID_SALN_HI_RES6
00207       XMSG_TYPE_ID_SALN_HI_RES7 = 0xB7,    //!< XMSG_TYPE_ID_SALN_HI_RES7
00208       XMSG_TYPE_ID_SALN_DRIFT8 = 0xB8,    //!< XMSG_TYPE_ID_SALN_DRIFT8
00209       XMSG_TYPE_ID_SALN_DRIFT9 = 0xB9,    //!< XMSG_TYPE_ID_SALN_DRIFT9
00210       XMSG_TYPE_ID_SALN_DRIFTA = 0xBA,    //!< XMSG_TYPE_ID_SALN_DRIFTA
00211       XMSG_TYPE_ID_SALN_DRIFTB = 0xBB,    //!< XMSG_TYPE_ID_SALN_DRIFTB
00212       XMSG_TYPE_ID_SALN_DRIFTC = 0xBC,    //!< XMSG_TYPE_ID_SALN_DRIFTC
00213       XMSG_TYPE_ID_SALN_DRIFTD = 0xBD,    //!< XMSG_TYPE_ID_SALN_DRIFTD
00214       XMSG_TYPE_ID_SALN_DRIFTE = 0xBE,    //!< XMSG_TYPE_ID_SALN_DRIFTE
00215       XMSG_TYPE_ID_SALN_DRIFTF = 0xBF,    //!< XMSG_TYPE_ID_SALN_DRIFTF
00216 //        = 0xC0,    /
00217       XMSG_TYPE_ID_ALAMO_WAVE_C1 = 0xC1,    //!< ALAMO_WAVE
00218       XMSG_TYPE_ID_AUX_SENSOR_DATA = 0xC2,    //!< ALAMO_WAVE
00219       // = 0xC2
00220       XMSG_TYPE_ID_ALAMO_WAVE_C3 = 0xC3,    //!< ALAMO_WAVE
00221       // = 0xC4
00222       // = 0xC5
00223       // = 0xC6
00224       // = 0xC7
00225       // = 0xC8
00226       // = 0xC9
00227       XMSG_TYPE_ID_MIZ_ACOUSTIC = 0xCA, //!< MIZ_ACOUSTIC
00228       // = 0xCB
00229       // = 0xCC
00230       // = 0xCD
00231       // = 0xCE
00232       // = 0xCF
00233       XMSG_TYPE_ID_MISS_CONF0 = 0xD0,    //!< MISS_CONF0
00234       XMSG_TYPE_ID_MISS_CONF1 = 0xD1,    //!< MISS_CONF1
00235       XMSG_TYPE_ID_MISS_CONF2 = 0xD2,    //!< MISS_CONF2
00236       XMSG_TYPE_ID_MISS_CONF3 = 0xD3,    //!< MISS_CONF3
00237       XMSG_TYPE_ID_MISS_CONF4 = 0xD4,    //!< MISS_CONF4
00238       XMSG_TYPE_ID_MISS_CONF5 = 0xD5,    //!< MISS_CONF5
00239       XMSG_TYPE_ID_MISS_CONF6 = 0xD6,    //!< MISS_CONF6
00240       XMSG_TYPE_ID_MISS_CONF7 = 0xD7,    //!< MISS_CONF7
00241       // = 0xD8
00242       // = 0xD9
00243       // = 0xDA
00244       // = 0xDB
00245       // = 0xDC
00246       // = 0xDD
00247       // = 0xDE
```

```
00248       // = 0xDF
00249       XMSG_TYPE_ID_MISS_CMD_ECHO = 0xDE,    //!< MISS_CMD_ECHO
00250       // = 0xDF
00251       XMSG_TYPE_ID_ENG_DIAGNOSTIC = 0xE0,    //!< ENG_DIAGNOSTIC
00252 //    ENG_ = 0xE1,
00253       XMSG_TYPE_ID_ENG_DIVE = 0xE2,      //!< ENG_DIVE
00254       XMSG_TYPE_ID_ENG_ABORT = 0xE3,      //!< ENG_MISS_START
00255 //    ENG = 0xE4,
00256       XMSG_TYPE_ID_ENG_BIT_PASS = 0xE5,    //!< ENG_BIT_PASS
00257       XMSG_TYPE_ID_ENG_BIT_FAIL = 0xE6,    //!< ENG_BIT_FAIL
00258       XMSG_TYPE_ID_ENG_DESCENT = 0xE7,
00259       XMSG_TYPE_ID_ENG_PARK = 0xE8,
00260       XMSG_TYPE_ID_ENG_ASCENT = 0xE9,
00261       XMSG_TYPE_ID_ENG_SURFACE = 0xEA,
00262       XMSG_TYPE_ID_ENG_VALVE = 0xEB,
00263       XMSG_TYPE_ID_ENG_ICE = 0xEC,
00264 //    ENG = 0xED,
00265 //    ENG = 0xEE,
00266 //    ENG = 0xEF,
00267       XMSG_TYPE_ID_ARGO_DATA = 0xF0,     //!< ARGO_DATA
00268       XMSG_TYPE_ID_TEST = 0xF1,          //!< TEST
00269       XMSG_TYPE_ID_UNKNOWN = 0xFF    //!< UNKNOWN
00270     } XMsgTypeId_t;
00271
00272     /* Class Definition */
00273
00274     /** Class representing a message following the X format specification.
00275      *
00276      */
00277     class XMessage {
00278
00279       private:
00280
00281         /* Private Data Members: */
00282
00283         /** The dive or cycle number of this message. */
00284         int16_t id_dive_;
00285
00286         /** The unique WHOI identifier of this instrument. */
00287         uint16_t id_instrument_;
00288
00289         /** The X-message type identifier. */
00290         XMsgTypeId_t id_xmsg_type_;
00291
00292         /** The size of the X message, in bytes. */
00293         uint16_t xmsg_size_bytes_;
00294
00295       protected:
00296
00297         /* Protected Member Functions: */
00298
00299         /** The default constructor.
00300          *
00301          *  @param[in]  dive            The dive number of this X-Message object.
00302          *  @param[in]  whoi            The unique instrument identifier.
00303          *  @param[in]  type            The X-Message type identifier.
00304          *
00305          *  @exception  None. Never throws.
00306          */
00307         XMessage(
00308             int16_t dive = XMSG_DIVE_UNKNOWN,
00309             uint16_t whoi = XMSG_ID_UNKNOWN,
00310             XMsgTypeId_t type = XMSG_TYPE_ID_UNKNOWN) ATTR_NO_EXCEPTIONS :
00311                 id_dive_(dive),
00312                 id_instrument_(whoi),
00313                 id_xmsg_type_(type),
00314                 xmsg_size_bytes_(0) {
00315           /* Do nothing. */
00316         }
00317
00318         /** Set the dive or cycle number of this message.
00319          *
00320          *  @param[in]  id_dive         The dive or cycle number of this message to set.
00321          */
00322         void set_id_dive(int16_t id_dive) {
00323           id_dive_ = id_dive;
00324         }
00325
00326         /** Set the unique instrument identifier.
00327          *
00328          *  @param[in]  id_instrument   The unique instrument identifier to set.
00329          */
00330         void set_id_instrument(uint16_t id_instrument) {
00331           id_instrument_ = id_instrument;
00332         }
00333
00334         /** Set the X-Message type identifier.
00335          *
00336          *  @param[in]  id_xmsg_type    The X-Message type identifier to set.
00337          */
00338         void set_id_xmsg_type(XMsgTypeId_t id_xmsg_type) {
00339           id_xmsg_type_ = id_xmsg_type;
00340         }
00341
00342         /** Set the size of the X-Message, in bytes.
00343          *
00344          *  @param[in]  xmsg_size_bytes The size of the X-Message to set, in bytes.
00345          */
00346         void set_xmsg_size_bytes(uint16_t xmsg_size_bytes) {
00347           xmsg_size_bytes_ = xmsg_size_bytes;
00348         }
00349
00350       public:
00351
00352         /* Public Member Fucntions: */
00353
00354         /** The Parameterized constructor.
00355          *
```

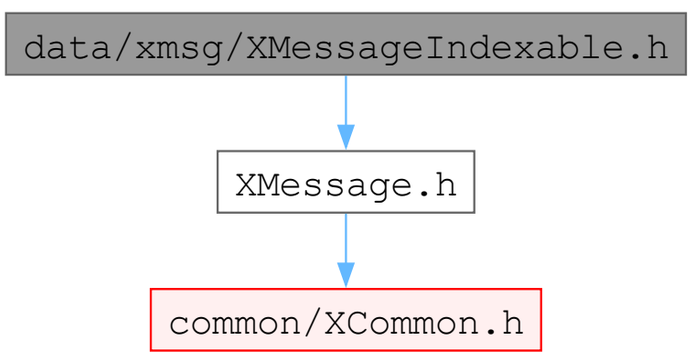
```
00356          *  @param[in]  dive        The dive or cycle number of this message.
00357          *  @param[in]  whoi        The unique WHOI identifier.
00358          *  @param[in]  data        The binary X-message data to decode.
00359          *
00360          *  @exception  None. Never throws.
00361          */
00362         XMessage(int16_t dive, uint16_t whoi, const vector<uint8_t>& data) ATTR_NO_EXCEPTIONS :
00363                 id_dive_(dive),
00364                 id_instrument_(whoi),
00365                 id_xmsg_type_((data.empty() ? XMSG_TYPE_ID_UNKNOWN : (XMsgTypeId_t) data.at(0))),
00366                 xmsg_size_bytes_(data.size()) {
00367           /* Do nothing. */
00368         }
00369
00370         /** The destructor.
00371          *
00372          */
00373         virtual ~XMessage() {
00374           /* Do nothing. */
00375         }
00376
00377         /** Check the X-message for the correct ID, minimum size, and message terminator.
00378          *
00379          *  @param[in]      data        The X-message binary data to check.
00380          *  @param[in]      id          The expected ID of the X-message. If this field is less that 0,
00381          *                              then the ID check will be skipped.
00382          *  @param[in]      minBytes    The minimum number of bytes expected in a valid message.
00383          *  @param[in]      parseSize   If TRUE, then the message size field will be parsed from the
00384          *                              X-Message. Otherwise, the size field will not be parsed.
00385          *
00386          *  @exception      util::log::MessageException if
00387          *                      - the message does not contain the expected ID,
00388          *                      - the message does not contain the minimum number of expected bytes,
00389          *                      - the message does not contain a terminator (';') in the correct location,
00390          *                        or,
00391          *                      - any other error occurs.
00392          *
00393          *  @return         A pointer to the underlying data buffer of data.
00394          */
00395         const uint8_t* CheckXMsgData(const vector<uint8_t>& data, int id, int minBytes,
00396         bool parseSize = true) {
00397           try {
00398             // Check minimum message size.
00399             if(data.empty()) {
00400               ostringstream msg;
00401               msg << "Invalid XMessage: Message is empty! Expected at least "
00402                   << minBytes
00403                   << " bytes, but received 0 bytes!";
00404               BOOST_THROW_EXCEPTION(log::MessageException(msg));
00405             }
00406
00407             // Check minimum message size.
00408             if((int) data.size() < minBytes) {
00409               ostringstream msg;
00410               msg << "Invalid XMessage: expected at least "
00411                   << minBytes
00412                   << " bytes, but only received "
00413                   << data.size()
00414                   << " bytes!";
00415               BOOST_THROW_EXCEPTION(log::MessageException(msg));
00416             }
00417
00418             // Get byte array.
00419             const uint8_t* d = data.data();
00420
00421             // Check X ID.
00422             if(id >= 0 && d[0] != id) {
00423               ostringstream msg;
00424               msg << "Invalid XMessage ID: expected 0x"
00425                   << setfill('0')
00426                   << setw(2)
00427                   << hex
00428                   << id
00429                   << ", received 0x"
00430                   << hex
00431                   << (int) d[0]
00432                   << "!";
00433               EXC_THROW(msg.str());
00434             }
00435             set_id_xmsg_type((XMsgTypeId_t) d[0]);
00436
00437             if(parseSize) {
00438               // Check message size.
00439               set_xmsg_size_bytes(0x0FFF & u16(d, 1));
00440               if(xmsg_size_bytes() > data.size()) {
00441                 ostringstream msg;
00442                 msg << "Invalid XMessage size: message specifies at least "
00443                     << xmsg_size_bytes()
00444                     << " bytes, but only "
00445                     << data.size()
00446                     << " bytes were received.";
00447                 EXC_THROW(msg.str());
00448               } else if(data.size() > xmsg_size_bytes()) {
00449                 lwrn() << "Extra bytes in XMessage: message specifies "
00450                        << xmsg_size_bytes()
00451                        << "bytes, but "
00452                        << data.size()
00453                        << " bytes were received.";
00454               }
00455
00456               // Check for terminator.
00457               if(d[xmsg_size_bytes() - 1] != 0x3B) {
00458                 ostringstream msg;
00459                 msg << "Invalid XMessage: expected terminator ';' (0x3B) at byte "
00460                     << (xmsg_size_bytes() - 1)
00461                     << ", but received 0x"
00462                     << hex
00463                     << d[xmsg_size_bytes() - 1];
```

```
00464                 EXC_THROW(msg.str());
00465               }
00466             }
00467
00468             return d;
00469
00470           }CATCH_BLOCK("X-message failed check!")
00471
00472           return NULL;
00473         }
00474
00475         /** Decode the X-Message data.
00476          *
00477          *  @param[in]      data        The encoded X message to decode.
00478          */
00479         virtual void DecodeXMsgData(const vector<uint8_t>& data) = 0;
00480
00481         /** Get the dive or cycle number of this X-Message.
00482          *
00483          *  @return         The dive or cycle number of this X-Message.
00484          */
00485         int16_t id_dive() const {
00486           return id_dive_;
00487         }
00488
00489         /** Get the unique instrument identifier of this X-Message.
00490          *
00491          *  @return         The unique instrument identifier of this X-Message.
00492          */
00493         uint16_t id_instrument() const {
00494           return id_instrument_;
00495         }
00496
00497         /** Get the enumerated X-Message type identifier.
00498          *
00499          *  @return         The enumerated X-Message type identifier.
00500          */
00501         XMsgTypeId_t id_xmsg_type() const {
00502           return id_xmsg_type_;
00503         }
00504
00505         /** Get the size of the X-Message, in bytes.
00506          *
00507          *  @return         The size of the X-Message, in bytes.
00508          */
00509         uint16_t xmsg_size_bytes() const {
00510           return xmsg_size_bytes_;
00511         }
00512
00513     };
00514
00515   } /* namespace xmsg */
00516 } /* namespace util */
00517
00518 #endif /* XMSG_H_ */
```

11.139 data/xmsg/XMessageIndexable.h File Reference

The util::xmsg::XMessageIndexable class source file.

`#include "XMessage.h"`
Include dependency graph for XMessageIndexable.h:



This graph shows which files directly or indirectly include this file:

Classes

- class util::xmsg::XMessageIndexable

  Class representing an indexable X-Message.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.139.1 Detailed Description

The util::xmsg::XMessageIndexable class source file.

Version

0.6

Date

Aug 2, 2014 5:31:10 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.140 XMessageIndexable.h

Go to the documentation of this file.

```
00001 /** The @ref util::xmsg::XMessageIndexable class source file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Aug 2, 2014 5:31:10 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef XMESSAGEINDEXABLE_H_
00024 #define XMESSAGEINDEXABLE_H_
00025
00026 /* Includes */
00027
00028 // local
00029 #include "XMessage.h"
00030
00031 namespace util {
00032 namespace xmsg {
00033
```

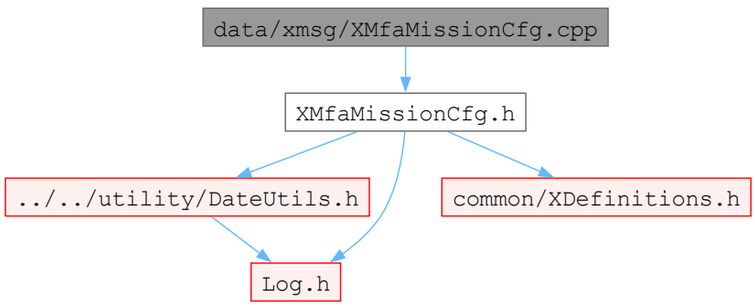
```
00034 /* Class Definitions */
00035
00036 /** Class representing an indexable X-Message. */
00037 class XMessageIndexable:
00038     public util::xmsg::XMessage {
00039
00040 private:
00041
00042   /* Data Members */
00043
00044   /** The index of this X-Message. */
00045   uint8_t xmsg_index_;
00046
00047 protected:
00048
00049   /* Member Functions */
00050
00051   /** The default constructor.
00052    *
00053    *  @exception  None. Never throws.
00054    */
00055   XMessageIndexable() ATTR_NO_EXCEPTIONS :
00056           XMessage(),
00057           xmsg_index_(0xFF) {
00058     /* Do nothing. */
00059   }
00060
00061   /** Set the index of this X-Message.
00062    *
00063    *  @param[in]  xmsg_index      The index of this X-Message.
00064    */
00065   void set_xmsg_index(uint8_t xmsg_index) {
00066     xmsg_index_ = xmsg_index;
00067   }
00068
00069 public:
00070
00071   /* Member Functions */
00072
00073   /** The parameterized constructor.
00074    *
00075    *  @param[in]  dive        The dive or cycle number of this message.
00076    *  @param[in]  whoi        The unique WHOI identifier.
00077    *  @param[in]  data        The binary X-message data to decode.
00078    *
00079    *  @exception  None. Never throws.
00080    */
00081   XMessageIndexable(int16_t dive, uint16_t whoi, const vector<uint8_t>& data) ATTR_NO_EXCEPTIONS :
00082           XMessage(dive, whoi, data),
00083           xmsg_index_(0xFF) {
00084     /* Do nothing. */
00085   }
00086
00087   /** The destructor.
00088    *
00089    */
00090   ~XMessageIndexable() {
00091     /* Do nothing. */
00092   }
00093
00094   /** Decode the X-Message data.
00095    *
00096    *  @param[in]      data        The encoded X message to decode.
00097    */
00098   virtual void DecodeXMsgData(const vector<uint8_t>& data) = 0;
00099
00100   /** Get the index of this X-Message.
00101    *
00102    *  @return         The index of this X-Message.
00103    */
00104   uint8_t xmsg_index() const {
00105     return xmsg_index_;
00106   }
00107
00108 };
00109
00110 } /* namespace xmsg */
00111 } /* namespace util */
00112 #endif /* XMESSAGEINDEXABLE_H_ */
```

11.141 data/xmsg/XMfaMissionCfg.cpp File Reference

The util::xmsg::XMfaMissionCfg class source file.

```
#include "XMfaMissionCfg.h"
```

Include dependency graph for XMfaMissionCfg.cpp:



Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.141.1 Detailed Description

The util::xmsg::XMfaMissionCfg class source file.

Version

0.6

Date

Oct 9, 2017 2:34:36 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

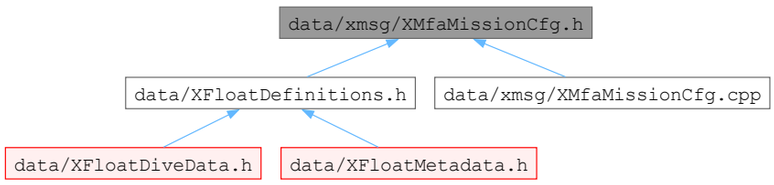
11.142 data/xmsg/XMfaMissionCfg.h File Reference

The util::xmsg::XMfaMissionCfg class header file.

```
#include "../../utility/DateUtils.h"
#include "../../utility/Log.h"
#include "common/XDefinitions.h"
```

Include dependency graph for XMfaMissionCfg.h:

This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XMfaMissionCfg

  The ALAMO mission configuration (mission parameters) message class.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.142.1 Detailed Description

The util::xmsg::XMfaMissionCfg class header file.

Version

0.6

Date

Oct 9, 2017 2:34:36 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.143 XMfaMissionCfg.h

Go to the documentation of this file.

```
00001 /** The @ref util::xmsg::XMfaMissionCfg class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Oct 9, 2017 2:34:36 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef DATA_XMSG_XMFAMISSIONCFG_H_
00024 #define DATA_XMSG_XMFAMISSIONCFG_H_
00025
00026 /* Includes: */
00027
00028 #include "../../utility/DateUtils.h"    // local libs
00029 #include "../../utility/Log.h"
00030 #include "common/XDefinitions.h"
00031
00032 namespace util {
00033 namespace xmsg {
00034
00035 /* Types: */
00036
00037 /** The ALAMO mission configuration (mission parameters) message class.
00038  *
00039  *  An instance of this class assembles the mission configuration X-messages transmitted by an
00040  *  instrument and decodes the mission parameters according to the instrument's firmware version.
00041  */
00042 class XMfaMissionCfg:
00043     public util::date::DateUtils {
00044
00045   private:  /* Private Data Members: */
00046
00047     /** The mission configuration messages' raw data.
00048      *
00049      */
00050     static_vector<vector<uint8_t>, XMSG_MAX_MFA_MISS> msg_data_;
00051
00052     /** The software version identifier.
00053      *
00054      */
00055     uint32_t fw_version_;
00056
00057     /** The message version identifier.
00058      *
00059      */
00060     uint8_t msg_version_;
00061
00062     /** The sensor type identifier.
00063      *
00064      */
00065     uint8_t sensor_type_;
00066
00067     /* Data Members - General Configuration: */
00068
00069     /** The mission, dive, or cycle number for this mission configuration.
00070      *
00071      */
00072     int16_t msn_;
00073
00074     /** The target descent/drift/park pressure, in dBar.
00075      *
00076      */
00077     uint16_t dtd_[3];
00078
00079     /** The maximum descent/fall/profile pressure, in dBar.
00080      *
00081      */
00082     uint16_t mds_;
00083
00084     /** The auto-transmit mission configuration message flag.
00085      *
00086      */
00087     uint8_t apt_;
00088
00089     /** The bluetooth mode.
00090      *
00091      */
00092     uint8_t btm_;
00093
00094     /** The profile data encode method.
00095      *
00096      */
00097     uint8_t xas_;
00098
00099     /** The initial trigger depth, in dBar.
00100      *
00101      */
00102     uint16_t mtd_;
00103
```

```
00104     /** The initial trigger timeout/interval, in milliseconds.
00105      *
00106      */
00107     uint32_t mtt_;
00108
00109     uint32_t mts_;
00110
00111     /* Data Members - Mission Phase */
00112
00113     /** Current mission phase.
00114      *
00115      */
00116     uint8_t mps_;
00117
00118     /** Cycle count for current mission phase.
00119      *e
00120      */
00121     uint16_t mpc_;
00122
00123     /** Number of cycles for mission phase
00124      *
00125      */
00126     uint16_t mnm_[3];
00127
00128     /** Next mission phase.
00129      *
00130      */
00131     uint8_t mns_[3];
00132
00133     /* Data Members - Ice Configuration */
00134
00135     /** Ice-detection mode.
00136      *
00137      */
00138     uint8_t icm_;
00139
00140     /** Ice-detection -- month mask.
00141      *
00142      */
00143     uint16_t imm_;
00144
00145     /** Ice-detection -- surfacing achieved criteria mode.
00146      *
00147      */
00148     uint8_t ism_;
00149
00150     /** Ice-detection -- surface attempts.
00151      *
00152      */
00153     uint16_t ica_[2];
00154
00155     /** Ice-detection -- cycles between surface attempts.
00156      *
00157      */
00158     uint16_t icb_[2];
00159
00160     /** Ice-detection -- percentage used storage urgency criteria.
00161      *
00162      */
00163     uint16_t imt_[2];
00164
00165     /** Ice-detection -- duration since last successful comms urgency criteria, in days.
00166      *
00167      */
00168     uint16_t ilt_[2];
00169
00170     /** Ice-detection -- temperature threshold test upper pressure boundary, in dBar.
00171      *
00172      */
00173     uint16_t icn_;
00174
00175     /** Ice-detection -- temperature threshold test lower pressure boundary, in dBar.
00176      *
00177      */
00178     uint16_t icx_;
00179
00180     /** Ice-detection -- temperature threshold test temperature, in 1/100 degC.
00181      *
00182      */
00183     int16_t ict_;
00184
00185     /** Ice-detection -- temperature threshold test sample interval, in seconds.
00186      *
00187      */
00188     uint16_t ics_;
00189
00190     /** Ice-detection -- multiple surface attempts mode.
00191      *
00192      */
00193     uint8_t iss_[2];
00194
00195     /** Ice-detection -- surface achieved salinity threshold, in 1/10 PSU.
00196      *
00197      */
00198     uint16_t ist_;
00199
00200     /** Ice-detection -- duration between multiple surfacing attempts, in seconds.
00201      *
00202      */
00203     uint16_t iso_;
00204
00205     /** Ice-detection -- multiple surfacing profile mode.
00206      *
00207      */
00208     uint8_t isp_;
00209
00210     /** Ice-detection -- urgency mode exit criteria.
00211      *
```

```
00212      */
00213     uint8_t iue_[2];
00214
00215     /** Ice-detection -- hold pressure, in dBar.
00216      *
00217      */
00218     uint16_t icd_[2];
00219
00220     /** Ice-detection -- .
00221      *
00222      */
00223     uint32_t icr_[2];
00224
00225     /** Ice-detection -- seek mode.
00226      *
00227      */
00228     uint8_t icg_[2];
00229
00230     /** Ice-detection -- .
00231      *
00232      */
00233     uint8_t isa_;
00234
00235     /** Ice-detection -- .
00236      *
00237      */
00238     uint16_t isd_;
00239
00240     /** Ice-detection -- .
00241      *
00242      */
00243     uint16_t iat_;
00244
00245     /** Ice-detection -- .
00246      *
00247      */
00248     uint16_t isv_;
00249
00250     /** Ice-detection -- .
00251      *
00252      */
00253     uint16_t isw_;
00254
00255     /** Ice-detection -- ice-detection hold-pressure duration, in minutes.
00256      *
00257      */
00258     uint16_t ico_[2];
00259
00260     /** Ice-detection -- ice-free month mask.
00261      *
00262      */
00263     uint32_t icf_;
00264
00265     /** Ice-detection -- iced-over month mask.
00266      *
00267      */
00268     uint32_t icw_;
00269
00270     /* Data Members - IMU Configuration */
00271
00272     /** The motion processing unit (MPU) data acquisition mode mask.
00273      *
00274      */
00275     uint8_t mam_;
00276
00277     /** The motion processing unit (MPU) sampling duration, in seconds.
00278      *
00279      */
00280     uint16_t mat_[2];
00281
00282     uint16_t map_;
00283
00284     /** The motion processing unit (MPU) accelerometer sensitivity scale factor identifier.
00285      *
00286      */
00287     uint16_t aca_;
00288
00289     /** The motion processing unit (MPU) gyroscope sensitivity scale factor identifier.
00290      *
00291      */
00292     uint16_t acg_;
00293
00294     /** The motion processing unit (MPU) sample rate, in Hz.
00295      *
00296      */
00297     uint16_t acr_;
00298
00299     /* Data Members - BIT Configuration */
00300
00301     uint16_t pnb_;
00302
00303     uint16_t pnc_;
00304
00305     uint16_t pxc_;
00306
00307     /* Data Members - Calibration */
00308
00309     uint16_t pbl_;
00310
00311     uint16_t pbh_;
00312
00313     uint16_t pp0_;
00314
00315     uint16_t pp1_;
00316
00317     uint16_t pvl_;
00318
00319     uint16_t pvh_;
```

```
00320
00321     uint16_t pv0_;
00322
00323     uint16_t pv5_;
00324
00325     uint16_t pc0_;
00326
00327     uint16_t pc1_;
00328
00329     /* Data Members - Profile Configuration */
00330
00331     /** The CTD profiling mode.
00332      *
00333      */
00334     uint8_t cps_;
00335
00336     /** The CTD sampling type.
00337      *
00338      */
00339     uint8_t cst_;
00340
00341     /** The auxiliary temperature channel mode.
00342      *
00343      *  @note     Applies to RBR CTDs only.
00344      */
00345     uint8_t ctc_;
00346
00347     /** The profile cutoff pressure.
00348      *
00349      *  @todo   Determine gain for profile cutoff pressure field.
00350      *
00351      *  @note   This field was added in firmware version 0x02006900.
00352      */
00353     uint16_t ctf_;
00354
00355     /** The pressure sensor offset reset lower pressure limit.
00356      *
00357      *  @todo   Determine gain for pressure sensor offset reset lower pressure limit field.
00358      *
00359      *  @note   This field was added in firmware version 0x02006900.
00360      *
00361      */
00362     uint16_t ctl_;
00363
00364     /** The pressure sensor offset reset lower pressure limit.
00365      *
00366      *  @todo   Determine gain for pressure sensor offset reset lower pressure limit field.
00367      *
00368      *
00369      *  @note   This field was added in firmware version 0x02006900.
00370      *
00371      */
00372     uint16_t ctu_;
00373
00374     /** The CTD continuous sampling period, in milliseconds.
00375      *
00376      */
00377     uint32_t csp_;
00378
00379     /** The CTD profiling mode.
00380      *
00381      */
00382     uint8_t cms_;
00383
00384     /** The CTD sampling period pressure boundaries, in dBar.
00385      *
00386      */
00387     uint16_t cpb_[3];
00388
00389     /** The CTD sampling period bin sizes, in 0.1 dBar
00390      *
00391      */
00392     uint16_t cpn_[3];
00393
00394     /** The CTD sampling period interval, in milliseconds.
00395      *
00396      */
00397     uint16_t cpp_[3];
00398
00399     /** The CTD raw sampling threshold, in dBar.
00400      *
00401      */
00402     uint16_t crd_;
00403
00404     /** The maximum number of raw samples to collect above the raw sampling threshold.
00405      *
00406      */
00407     uint16_t crs_;
00408
00409
00410     /* Data Members - Descent State Configuration */
00411
00412     /** The start time of the descend state of the dive cycle.
00413      *
00414      */
00415     ptime descend_start_time_;
00416
00417     /** The descent number.
00418      *
00419      */
00420     uint16_t dns_;
00421
00422     uint8_t dra_;
00423
00424     /** The descent state start pressure, in dBar.
00425      *
00426      */
00427     uint16_t dsd_[3];
```

```
00428
00429     /** The descent state start timeout, in seconds.
00430      *
00431      */
00432     uint16_t dst_[3];
00433
00434     uint16_t ddf_;
00435
00436     /** Diagnostic dive bypass mode.
00437      *
00438      */
00439     uint8_t ddb_;
00440
00441     uint16_t ddc_;
00442
00443     /** The descent state sampling interval, in milliseconds.
00444      *
00445      */
00446     uint32_t dsi_;
00447
00448     /** The descent state data storage pressure interval, in dBar.
00449      *
00450      */
00451     uint16_t ddd_;
00452
00453     /** The descent state data storage interval, in seconds.
00454      *
00455      */
00456     uint16_t ddt_;
00457
00458     /** The descent state fall timeout, in seconds.
00459      *
00460      */
00461     uint16_t dft_[3];
00462
00463     uint8_t dcm_;
00464
00465     uint8_t dal_;
00466
00467     uint16_t dct_[3];
00468
00469     uint8_t dca_;
00470
00471     uint16_t drc_;
00472
00473     uint16_t drt_;
00474
00475     uint16_t dcx_;
00476
00477     uint16_t dcp_;
00478
00479     uint16_t dcf_;
00480
00481     uint16_t dcd_;
00482
00483     uint16_t dcr_;
00484
00485     uint16_t dsw_;
00486
00487     uint16_t duc_;
00488
00489     uint16_t dpt_;
00490
00491     uint16_t dut_;
00492
00493     uint16_t dpd_;
00494
00495     uint16_t dvt_;
00496
00497     uint16_t dvd_;
00498
00499     uint16_t dvw_;
00500
00501     uint16_t pfr_;
00502
00503     uint16_t pfc_;
00504
00505     uint8_t ftm_;
00506
00507     uint8_t fta_;
00508
00509     uint16_t fto_;
00510
00511     uint16_t fos_;
00512
00513     uint16_t ddp_;
00514
00515     uint16_t dds_;
00516
00517     /* Data Members - Drift/Park State Configuration */
00518
00519     /** The start time of the park/drift state of the dive cycle.
00520      *
00521      */
00522     ptime park_start_time_;
00523
00524     /** The park/drift number.
00525      *
00526      */
00527     uint16_t pns_;
00528
00529     uint32_t ptd_;
00530
00531     /** The park/drift state timeout, in seconds.
00532      *
00533      */
00534     uint32_t pto_[3];
00535
```

```
00536     /** The bit-encoded park/drift state hour timeout trigger(s).
00537      *
00538      */
00539     uint32_t pth_[3];
00540
00541     /** The park/drift state sampling mode.
00542      *
00543      */
00544     uint8_t psa_;
00545
00546     /** The park/drift state sample interval/timeout, in milliseconds.
00547      *
00548      */
00549     uint32_t psi_;
00550
00551     /** The park/drift state fall mode.
00552      *
00553      */
00554     uint8_t pfm_[3];
00555
00556     uint16_t pft_[3];
00557
00558     /** The park/drift state fall depth (i.e. profile depth), in dBar.
00559      *
00560      */
00561     uint16_t pfd_[3];
00562
00563     uint16_t pfv_[3];
00564
00565     /** The park/drift state data storage interval/timeout, in seconds.
00566      *
00567      */
00568     uint16_t pdt_;
00569
00570     uint8_t psm_;
00571
00572     uint16_t psd_;
00573
00574     uint8_t pss_;
00575
00576     uint16_t pst_;
00577
00578     uint16_t psv_;
00579
00580     uint16_t psw_;
00581
00582     /* Data Members - Ascent State Configuration */
00583
00584     /** The start time of the ascent state of the dive cycle.
00585      *
00586      */
00587     ptime ascent_start_time_;
00588
00589     /** The ascent number.
00590      *
00591      */
00592     uint16_t asn_;
00593
00594     /** The ascent state rise rate, in centimeters per second.
00595      *
00596      */
00597     uint16_t arr_;
00598
00599     uint16_t apm_;
00600
00601     uint16_t aps_;
00602
00603     /** The ascent state pump on duration/interval/timeout, in seconds.
00604      *
00605      */
00606     uint16_t apn_;
00607
00608     /** The ascent state pump off duration/interval/timeout, in seconds.
00609      *
00610      */
00611     uint16_t apf_;
00612
00613     /** The ascent state sampling interval/timeout, in milliseconds.
00614      *
00615      */
00616     uint32_t asi_;
00617
00618     /** The ascent state surface threshold pressure, in dBar.
00619      *
00620      */
00621     uint16_t asd_;
00622
00623     /** The ascent state data storage pressure interval, in dBar.
00624      *
00625      */
00626     uint16_t add_;
00627
00628     /** The ascent state data storage interval timeout, in seconds.
00629      *
00630      */
00631     uint16_t adt_;
00632
00633     uint16_t arc_;
00634
00635     uint16_t ahr_;
00636
00637     uint16_t ahd_;
00638
00639     /* Data Members - Surface State Configuration */
00640
00641     /** The start time of the surface state of the dive cycle.
00642      *
00643      */
```

```
00644     ptime surface_start_time_;
00645
00646     /** The surface number.
00647      *
00648      */
00649     uint16_t sfn_;
00650
00651     /** The surface state sensor depth compensation offset, in dBar.
00652      *
00653      */
00654     int16_t sdc_;
00655
00656     /** The surface state GPS timeout, in minutes.
00657      *
00658      */
00659     uint16_t sgt_;
00660
00661     /** The surface state Iridium session timeout, in minutes.
00662      *
00663      */
00664     uint16_t set_;
00665
00666     /** The surface state beacon mode flag.
00667      *
00668      */
00669     uint8_t sbm_;
00670
00671     /** The surface state beacon interval timeout, in seconds.
00672      *
00673      */
00674     uint16_t sbt_;
00675
00676     uint32_t sdt_;
00677
00678     uint16_t ssa_;
00679
00680     /** The surface state sampling interval timeout, in milliseconds.
00681      *
00682      */
00683     uint32_t ssi_;
00684
00685     /** The surface state sampling storage duration/interval/timeout, in seconds.
00686      *
00687      */
00688     uint16_t sds_;
00689
00690     uint8_t sam_;
00691
00692     uint16_t sax_;
00693
00694     uint16_t san_;
00695
00696     uint16_t sao_;
00697
00698     uint16_t sap_;
00699
00700     uint16_t spt_;
00701
00702     uint16_t ssp_;
00703
00704     /** WIMS timeout during surface state, in seconds.
00705      *
00706      */
00707     uint16_t wmt_;
00708
00709
00710     /* Data Members - RBR Post Processing */
00711
00712     float ppc_tau_;
00713
00714     float ppc_alpha_;
00715
00716     float ppc_tdelay_;
00717
00718     float ppc_ctccoeff_;
00719
00720     uint8_t ppe_;
00721
00722
00723
00724   public:   /* Public Member Functions: */
00725
00726     XMfaMissionCfg();
00727     ~XMfaMissionCfg();
00728     void AssignMessage(const vector<uint8_t>& data);
00729     void DecodeXMsgData(uint32_t fw_version, uint8_t msg_idx, const vector<uint8_t>& data);
00730     const string GenerateListingString(int whoi, int dive, int idx) const;
00731
00732   public:   /* Public Accessor Methods: */
00733
00734     const ptime& ascent_start_time() const { return ascent_start_time_; }
00735     const ptime& descend_start_time() const { return descend_start_time_; }
00736     const ptime& park_start_time() const { return park_start_time_; }
00737     const ptime& surface_start_time() const { return surface_start_time_; }
00738     float ahr_mps() const { return (float) ahr() / 1000.; }
00739     float arc_sec() const { return (float) arc() / 1000.; }
00740     float arr_mps() const { return (float) arr() / 100.; }
00741     float asi_sec() const { return (float) asi() / 1000.; }
00742     float cpn1_dbar() const { return (float) cpn1() / 10.; }
00743     float cpn2_dbar() const { return (float) cpn2() / 10.; }
00744     float cpn3_dbar() const { return (float) cpn3() / 10.; }
00745     float cpp1_sec() const { return (float) cpp1() / 1000.; }
00746     float cpp2_sec() const { return (float) cpp2() / 1000.; }
00747     float cpp3_sec() const { return (float) cpp3() / 1000.; }
00748     float csp_sec() const { return (float) csp() / 1000.; }
00749     float dra_mps() const { return (float) dra() / 100.; }
00750     float dsi_sec() const { return (float) dsi() / 1000.; }
00751     float ict_degc() const { return (float) ict() / 100; }
```

```
00752     float ist_psu() const { return (float) ist() / 100; }
00753     float mtt_sec() const { return (float) mtt() / 1000; }
00754     float pnb_v() const { return (float) pnb() / 100; }
00755     float pnc_psi() const { return (float) pnc() / 100; }
00756     float psi_sec() const { return (float) psi() / 1000; }
00757     float pxc_psi() const { return (float) pxc() / 100; }
00758     float san_psi() const { return (float) san() / 1000; }
00759     float sax_psi() const { return (float) sax() / 1000; }
00760     float ssi_sec() const { return (float) ssi() / 1000; }
00761     int ict() const { return ict_; }
00762     int msn() const { return msn_; }
00763     int sdc() const { return sdc_; }
00764     unsigned int aca() const { return aca_; }
00765     unsigned int acg() const { return acg_; }
00766     unsigned int acr() const { return acr_; }
00767     unsigned int add() const { return add_; }
00768     unsigned int adt() const { return adt_; }
00769     unsigned int ahd() const { return ahd_; }
00770     unsigned int ahr() const { return ahr_; }
00771     unsigned int apf() const { return apf_; }
00772     unsigned int apm() const { return apm_; }
00773     unsigned int apn() const { return apn_; }
00774     unsigned int apt() const { return apt_; }
00775     unsigned int arc() const { return arc_; }
00776     unsigned int arr() const { return arr_; }
00777     unsigned int asd() const { return asd_; }
00778     unsigned int asi() const { return asi_; }
00779     unsigned int asn() const { return asn_; }
00780     unsigned int btm() const { return btm_; }
00781     unsigned int cpb1() const { return cpb_[0]; }
00782     unsigned int cpb2() const { return cpb_[1]; }
00783     unsigned int cpb3() const { return cpb_[2]; }
00784     unsigned int cpn1() const { return cpn_[0]; }
00785     unsigned int cpn2() const { return cpn_[1]; }
00786     unsigned int cpn3() const { return cpn_[2]; }
00787     unsigned int cpp1() const { return cpp_[0]; }
00788     unsigned int cpp2() const { return cpp_[1]; }
00789     unsigned int cpp3() const { return cpp_[2]; }
00790     unsigned int cps() const { return cps_; }
00791     unsigned int crd() const { return crd_; }
00792     unsigned int crs() const { return crs_; }
00793     unsigned int csp() const { return csp_; }
00794     unsigned int cst() const { return cst_; }
00795     unsigned int ctc() const { return ctc_; }
00796     unsigned int ctf() const { return ctf_; }
00797     unsigned int ctl() const { return ctl_; }
00798     unsigned int ctu() const { return ctu_; }
00799     unsigned int dal() const { return dal_; }
00800     unsigned int dca() const { return dca_; }
00801     unsigned int dcd() const { return dcd_; }
00802     unsigned int dcf() const { return dcf_; }
00803     unsigned int dcm() const { return dcm_; }
00804     unsigned int dcp() const { return dcp_; }
00805     unsigned int dcr() const { return dcr_; }
00806     unsigned int dct() const { return dct_[0]; }
00807     unsigned int dct0() const { return dct_[0]; }
00808     unsigned int dct1() const { return dct_[1]; }
00809     unsigned int dct2() const { return dct_[2]; }
00810     unsigned int dcx() const { return dcx_; }
00811     unsigned int ddc() const { return ddc_; }
00812     unsigned int ddd() const { return ddd_; }
00813     unsigned int ddf() const { return ddf_; }
00814     unsigned int ddp() const { return ddp_; }
00815     unsigned int dds() const { return dds_; }
00816     unsigned int ddt() const { return ddt_; }
00817     unsigned int dft() const { return dft_[0]; }
00818     unsigned int dft0() const { return dft_[0]; }
00819     unsigned int dft1() const { return dft_[1]; }
00820     unsigned int dft2() const { return dft_[2]; }
00821     unsigned int dns() const { return dns_; }
00822     unsigned int dpd() const { return dpd_; }
00823     unsigned int dpt() const { return dpt_; }
00824     unsigned int dra() const { return dra_; }
00825     unsigned int drc() const { return drc_; }
00826     unsigned int drt() const { return drt_; }
00827     unsigned int dsd() const { return dsd_[0]; }
00828     unsigned int dsd0() const { return dsd_[0]; }
00829     unsigned int dsd1() const { return dsd_[1]; }
00830     unsigned int dsd2() const { return dsd_[2]; }
00831     unsigned int dsi() const { return dsi_; }
00832     unsigned int dst() const { return dst_[0]; }
00833     unsigned int dst0() const { return dst_[0]; }
00834     unsigned int dst1() const { return dst_[1]; }
00835     unsigned int dst2() const { return dst_[2]; }
00836     unsigned int dsw() const { return dsw_; }
00837     unsigned int dtd() const { return dtd_[0]; }
00838     unsigned int dtd0() const { return dtd_[0]; }
00839     unsigned int dtd1() const { return dtd_[1]; }
00840     unsigned int dtd2() const { return dtd_[2]; }
00841     unsigned int duc() const { return duc_; }
00842     unsigned int dut() const { return dut_; }
00843     unsigned int dvd() const { return dvd_; }
00844     unsigned int dvt() const { return dvt_; }
00845     unsigned int dvw() const { return dvw_; }
00846     unsigned int fos() const { return fos_; }
00847     unsigned int fta() const { return fta_; }
00848     unsigned int ftm() const { return ftm_; }
00849     unsigned int fto() const { return fto_; }
00850     unsigned int fw_version() const { return fw_version_; }
00851     unsigned int iat() const { return iat_; }
00852     unsigned int ica0() const { return ica_[0]; }
00853     unsigned int ica1() const { return ica_[1]; }
00854     unsigned int icb0() const { return icb_[0]; }
00855     unsigned int icb1() const { return icb_[1]; }
00856     unsigned int icd0() const { return icd_[0]; }
00857     unsigned int icd1() const { return icd_[1]; }
00858     unsigned int icf() const { return icf_; }
00859     unsigned int icg0() const { return icg_[0]; }
```

```
00860     unsigned int icg1() const { return icg_[1]; }
00861     unsigned int icm() const { return icm_; }
00862     unsigned int icn() const { return icn_; }
00863     unsigned int ico0() const { return ico_[0]; }
00864     unsigned int ico1() const { return ico_[1]; }
00865     unsigned int icr0() const { return icr_[0]; }
00866     unsigned int icr1() const { return icr_[1]; }
00867     unsigned int ics() const { return ics_; }
00868     unsigned int icw() const { return icw_; }
00869     unsigned int icx() const { return icx_; }
00870     unsigned int ilt0() const { return ilt_[0]; }
00871     unsigned int ilt1() const { return ilt_[1]; }
00872     unsigned int imm() const { return imm_; }
00873     unsigned int imt0() const { return imt_[0]; }
00874     unsigned int imt1() const { return imt_[1]; }
00875     unsigned int isa() const { return isa_; }
00876     unsigned int isd() const { return isd_; }
00877     unsigned int ism() const { return ism_; }
00878     unsigned int iso() const { return iso_; }
00879     unsigned int isp() const { return isp_; }
00880     unsigned int iss0() const { return iss_[0]; }
00881     unsigned int iss1() const { return iss_[1]; }
00882     unsigned int ist() const { return ist_; }
00883     unsigned int isv() const { return isv_; }
00884     unsigned int isw() const { return isw_; }
00885     unsigned int iue0() const { return iue_[0]; }
00886     unsigned int iue1() const { return iue_[1]; }
00887     unsigned int mam() const { return mam_; }
00888     unsigned int map() const { return map_; }
00889     unsigned int mat() const { return mat_[1]; }
00890     unsigned int mat1() const { return mat_[0]; }
00891     unsigned int mat2() const { return mat_[1]; }
00892     unsigned int mds() const { return mds_; }
00893     unsigned int mnm0() const { return mnm_[0]; }
00894     unsigned int mnm1() const { return mnm_[1]; }
00895     unsigned int mnm2() const { return mnm_[2]; }
00896     unsigned int mns0() const { return mns_[0]; }
00897     unsigned int mns1() const { return mns_[1]; }
00898     unsigned int mns2() const { return mns_[2]; }
00899     unsigned int mpc() const { return mpc_; }
00900     unsigned int mps() const { return mps_; }
00901     unsigned int msg_version() const { return msg_version_; }
00902     unsigned int mtd() const { return mtd_; }
00903     unsigned int mts() const { return mts_; }
00904     unsigned int mtt() const { return mtt_; }
00905     unsigned int pbh() const { return pbh_; }
00906     unsigned int pbl() const { return pbl_; }
00907     unsigned int pc0() const { return pc0_; }
00908     unsigned int pc1() const { return pc1_; }
00909     unsigned int pdt() const { return pdt_; }
00910     unsigned int pfc() const { return pfc_; }
00911     unsigned int pfd() const { return pfd_[0]; }
00912     unsigned int pfd0() const { return pfd_[0]; }
00913     unsigned int pfd1() const { return pfd_[1]; }
00914     unsigned int pfd2() const { return pfd_[2]; }
00915     unsigned int pfm() const { return pfm_[0]; }
00916     unsigned int pfm0() const { return pfm_[0]; }
00917     unsigned int pfm1() const { return pfm_[1]; }
00918     unsigned int pfm2() const { return pfm_[2]; }
00919     unsigned int pfr() const { return pfr_; }
00920     unsigned int pft() const { return pft_[0]; }
00921     unsigned int pft0() const { return pft_[0]; }
00922     unsigned int pft1() const { return pft_[1]; }
00923     unsigned int pft2() const { return pft_[2]; }
00924     unsigned int pfv() const { return pfv_[0]; }
00925     unsigned int pfv0() const { return pfv_[0]; }
00926     unsigned int pfv1() const { return pfv_[1]; }
00927     unsigned int pfv2() const { return pfv_[2]; }
00928     unsigned int pnb() const { return pnb_; }
00929     unsigned int pnc() const { return pnc_; }
00930     unsigned int pns() const { return pns_; }
00931     unsigned int pp0() const { return pp0_; }
00932     unsigned int pp1() const { return pp1_; }
00933     unsigned int psa() const { return psa_; }
00934     unsigned int psd() const { return psd_; }
00935     unsigned int psi() const { return psi_; }
00936     unsigned int psm() const { return psm_; }
00937     unsigned int pss() const { return pss_; }
00938     unsigned int pst() const { return pst_; }
00939     unsigned int psv() const { return psv_; }
00940     unsigned int psw() const { return psw_; }
00941     unsigned int ptd() const { return ptd_; }
00942     unsigned int pth() const { return pth_[0]; }
00943     unsigned int pth0() const { return pth_[0]; }
00944     unsigned int pth1() const { return pth_[1]; }
00945     unsigned int pth2() const { return pth_[2]; }
00946     unsigned int pto() const { return pto_[0]; }
00947     unsigned int pto0() const { return pto_[0]; }
00948     unsigned int pto1() const { return pto_[1]; }
00949     unsigned int pto2() const { return pto_[2]; }
00950     unsigned int pv0() const { return pv0_; }
00951     unsigned int pv5() const { return pv5_; }
00952     unsigned int pvh() const { return pvh_; }
00953     unsigned int pvl() const { return pvl_; }
00954     unsigned int pxc() const { return pxc_; }
00955     unsigned int sam() const { return sam_; }
00956     unsigned int san() const { return san_; }
00957     unsigned int sao() const { return sao_; }
00958     unsigned int sap() const { return sap_; }
00959     unsigned int sax() const { return sax_; }
00960     unsigned int sbm() const { return sbm_; }
00961     unsigned int sbt() const { return sbt_; }
00962     unsigned int sds() const { return sds_; }
00963     unsigned int sdt() const { return sdt_; }
00964     unsigned int sensor_type() const { return sensor_type_; }
00965     unsigned int set() const { return set_; }
00966     unsigned int sfn() const { return sfn_; }
00967     unsigned int sgt() const { return sgt_; }
```

```
00968     unsigned int spt() const { return spt_; }
00969     unsigned int ssa() const { return ssa_; }
00970     unsigned int ssi() const { return ssi_; }
00971     unsigned int ssp() const { return ssp_; }
00972     unsigned int xas() const { return xas_; }
00973     unsigned int ddb() const { return ddb_; }
00974     unsigned int aps() const { return aps_; }
00975     unsigned int wmt() const { return wmt_; }
00976
00977     float ppc_alpha() const {
00978       return ppc_alpha_;
00979     }
00980
00981     float ppc_ctccoeff() const {
00982       return ppc_ctccoeff_;
00983     }
00984
00985     float ppc_tau() const {
00986       return ppc_tau_;
00987     }
00988
00989     float ppc_tdelay() const {
00990       return ppc_tdelay_;
00991     }
00992
00993     uint8_t ppe() const {
00994       return ppe_;
00995     }
00996
00997
00998   private:    /* Private Mutator Methods: */
00999
01000     void set_aca(uint16_t aca) { aca_ = aca; }
01001     void set_acg(uint16_t acg) { acg_ = acg; }
01002     void set_acr(uint16_t acr) { acr_ = acr; }
01003     void set_add(uint16_t add) { add_ = add; }
01004     void set_adt(uint16_t adt) { adt_ = adt; }
01005     void set_ahd(uint16_t ahd) { ahd_ = ahd; }
01006     void set_ahr(uint16_t ahr) { ahr_ = ahr; }
01007     void set_apf(uint16_t apf) { apf_ = apf; }
01008     void set_apm(uint16_t apm) { apm_ = apm; }
01009     void set_apn(uint16_t apn) { apn_ = apn; }
01010     void set_apt(uint8_t apt) { apt_ = apt; }
01011     void set_arc(uint16_t arc) { arc_ = arc; }
01012     void set_arr(uint16_t arr) { arr_ = arr; }
01013     void set_ascent_start_time(const ptime& ascent_start_time) { ascent_start_time_ = ascent_start_time; }
01014     void set_asd(uint16_t asd) { asd_ = asd; }
01015     void set_asi(uint32_t asi) { asi_ = asi; }
01016     void set_asn(uint16_t asn) { asn_ = asn; }
01017     void set_btm(uint8_t btm) { btm_ = btm; }
01018     void set_cpb1(uint16_t cpb1) { cpb_[0] = cpb1; }
01019     void set_cpb2(uint16_t cpb2) { cpb_[1] = cpb2; }
01020     void set_cpb3(uint16_t cpb3) { cpb_[2] = cpb3; }
01021     void set_cpn1(uint16_t cpn1) { cpn_[0] = cpn1; }
01022     void set_cpn2(uint16_t cpn2) { cpn_[1] = cpn2; }
01023     void set_cpn3(uint16_t cpn3) { cpn_[2] = cpn3; }
01024     void set_cpp1(uint16_t cpp1) { cpp_[0] = cpp1; }
01025     void set_cpp2(uint16_t cpp2) { cpp_[1] = cpp2; }
01026     void set_cpp3(uint16_t cpp3) { cpp_[2] = cpp3; }
01027     void set_cps(uint8_t cps) { cps_ = cps; }
01028     void set_crd(uint16_t crd) { crd_ = crd; }
01029     void set_crs(uint16_t crs) { crs_ = crs; }
01030     void set_csp(uint32_t csp) { csp_ = csp; }
01031     void set_cst(uint8_t cst) { cst_ = cst; }
01032     void set_ctc(uint8_t ctc) { ctc_ = ctc; }
01033     void set_ctf(uint16_t ctf) { ctf_ = ctf; }
01034     void set_ctl(uint16_t ctl) { ctl_ = ctl; }
01035     void set_ctu(uint16_t ctu) { ctu_ = ctu; }
01036     void set_dal(uint8_t dal) { dal_ = dal; }
01037     void set_dca(uint8_t dca) { dca_ = dca; }
01038     void set_dcd(uint16_t dcd) { dcd_ = dcd; }
01039     void set_dcf(uint16_t dcf) { dcf_ = dcf; }
01040     void set_dcm(uint8_t dcm) { dcm_ = dcm; }
01041     void set_dcp(uint16_t dcp) { dcp_ = dcp; }
01042     void set_dcr(uint16_t dcr) { dcr_ = dcr; }
01043     void set_dct(uint16_t dct) { dct_[0] = dct; }
01044     void set_dct0(uint16_t dct) { dct_[0] = dct; }
01045     void set_dct1(uint16_t dct) { dct_[1] = dct; }
01046     void set_dct2(uint16_t dct) { dct_[2] = dct; }
01047     void set_dcx(uint16_t dcx) { dcx_ = dcx; }
01048     void set_ddc(uint16_t ddc) { ddc_ = ddc; }
01049     void set_ddd(uint16_t ddd) { ddd_ = ddd; }
01050     void set_ddf(uint16_t ddf) { ddf_ = ddf; }
01051     void set_ddp(uint16_t ddp) { ddp_ = ddp; }
01052     void set_dds(uint16_t dds) { dds_ = dds; }
01053     void set_ddt(uint16_t ddt) { ddt_ = ddt; }
01054     void set_descend_start_time(const ptime& descend_start_time) { descend_start_time_ =
      descend_start_time; }
01055     void set_dft(uint16_t dft) { dft_[0] = dft; }
01056     void set_dft0(uint16_t dft) { dft_[0] = dft; }
01057     void set_dft1(uint16_t dft) { dft_[1] = dft; }
01058     void set_dft2(uint16_t dft) { dft_[2] = dft; }
01059     void set_dns(uint16_t dns) { dns_ = dns; }
01060     void set_dpd(uint16_t dpd) { dpd_ = dpd; }
01061     void set_dpt(uint16_t dpt) { dpt_ = dpt; }
01062     void set_dra(uint8_t dra) { dra_ = dra; }
01063     void set_drc(uint16_t drc) { drc_ = drc; }
01064     void set_drt(uint16_t drt) { drt_ = drt; }
01065     void set_dsd(uint16_t dsd) { dsd_[0] = dsd; }
01066     void set_dsd0(uint16_t dsd) { dsd_[0] = dsd; }
01067     void set_dsd1(uint16_t dsd) { dsd_[1] = dsd; }
01068     void set_dsd2(uint16_t dsd) { dsd_[2] = dsd; }
01069     void set_dsi(uint32_t dsi) { dsi_ = dsi; }
01070     void set_dst(uint16_t dst) { dst_[0] = dst; }
01071     void set_dst0(uint16_t dst) { dst_[0] = dst; }
01072     void set_dst1(uint16_t dst) { dst_[1] = dst; }
01073     void set_dst2(uint16_t dst) { dst_[2] = dst; }
01074     void set_dsw(uint16_t dsw) { dsw_ = dsw; }
```

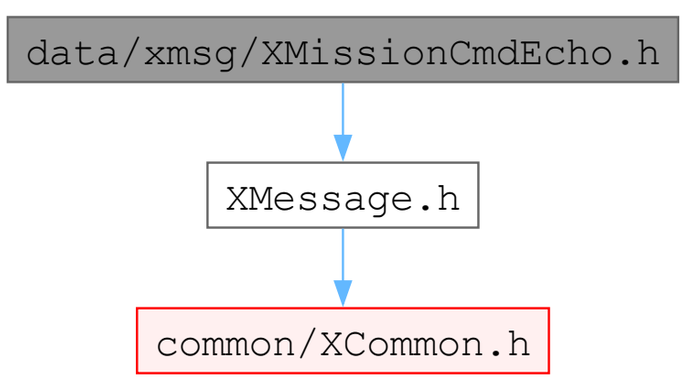
```
01075     void set_dtd(uint16_t dtd) { dtd_[0] = dtd; }
01076     void set_dtd0(uint16_t dtd) { dtd_[0] = dtd; }
01077     void set_dtd1(uint16_t dtd) { dtd_[1] = dtd; }
01078     void set_dtd2(uint16_t dtd) { dtd_[2] = dtd; }
01079     void set_duc(uint16_t duc) { duc_ = duc; }
01080     void set_dut(uint16_t dut) { dut_ = dut; }
01081     void set_dvd(uint16_t dvd) { dvd_ = dvd; }
01082     void set_dvt(uint16_t dvt) { dvt_ = dvt; }
01083     void set_dvw(uint16_t dvw) { dvw_ = dvw; }
01084     void set_fos(uint16_t fos) { fos_ = fos; }
01085     void set_fta(uint8_t fta) { fta_ = fta; }
01086     void set_ftm(uint8_t ftm) { ftm_ = ftm; }
01087     void set_fto(uint16_t fto) { fto_ = fto; }
01088     void set_fw_version(uint32_t fw_version) { fw_version_ = fw_version; }
01089     void set_iat(uint16_t iat) { iat_ = iat; }
01090     void set_ica0(uint16_t ica0) { ica_[0] = ica0; }
01091     void set_ica1(uint16_t ica1) { ica_[1] = ica1; }
01092     void set_icb0(uint16_t icb0) { icb_[0] = icb0; }
01093     void set_icb1(uint16_t icb1) { icb_[1] = icb1; }
01094     void set_icd0(uint16_t icd0) { icd_[0] = icd0; }
01095     void set_icd1(uint16_t icd1) { icd_[1] = icd1; }
01096     void set_icf(uint32_t icf) { icf_ = icf; }
01097     void set_icg0(uint16_t icg0) { icg_[0] = icg0; }
01098     void set_icg1(uint16_t icg1) { icg_[1] = icg1; }
01099     void set_icm(uint8_t icm) { icm_ = icm; }
01100     void set_icn(uint16_t icn) { icn_ = icn; }
01101     void set_ico0(uint16_t ico0) { ico_[0] = ico0; }
01102     void set_ico1(uint16_t ico1) { ico_[1] = ico1; }
01103     void set_icr0(uint16_t icr0) { icr_[0] = icr0; }
01104     void set_icr1(uint16_t icr1) { icr_[1] = icr1; }
01105     void set_ics(uint16_t ics) { ics_ = ics; }
01106     void set_ict(int16_t ict) { ict_ = ict; }
01107     void set_icw(uint32_t icw) { icw_ = icw; }
01108     void set_icx(uint16_t icx) { icx_ = icx; }
01109     void set_ilt0(uint16_t ilt0) { ilt_[0] = ilt0; }
01110     void set_ilt1(uint16_t ilt1) { ilt_[1] = ilt1; }
01111     void set_imm(uint16_t imm) { imm_ = imm; }
01112     void set_imt0(uint16_t imt0) { imt_[0] = imt0; }
01113     void set_imt1(uint16_t imt1) { imt_[1] = imt1; }
01114     void set_isa(uint8_t isa) { isa_ = isa; }
01115     void set_isd(uint16_t isd) { isd_ = isd; }
01116     void set_ism(uint8_t ism) { ism_ = ism; }
01117     void set_iso(uint16_t iso) { iso_ = iso; }
01118     void set_isp(uint8_t isp) { isp_ = isp; }
01119     void set_iss0(uint16_t iss0) { iss_[0] = iss0; }
01120     void set_iss1(uint16_t iss1) { iss_[1] = iss1; }
01121     void set_ist(uint16_t ist) { ist_ = ist; }
01122     void set_isv(uint16_t isv) { isv_ = isv; }
01123     void set_isw(uint16_t isw) { isw_ = isw; }
01124     void set_iue0(uint16_t iue0) { iue_[0] = iue0; }
01125     void set_iue1(uint16_t iue1) { iue_[1] = iue1; }
01126     void set_mam(uint8_t mam) { mam_ = mam; }
01127     void set_map(uint16_t map) { map_ = map; }
01128     void set_mat(uint16_t mat) { mat_[0] = mat; }
01129     void set_mat1(uint16_t mat) { mat_[0] = mat; }
01130     void set_mat2(uint16_t mat) { mat_[1] = mat; }
01131     void set_mds(uint16_t mds) { mds_ = mds; }
01132     void set_mnm0(uint16_t mnm) { mnm_[0] = mnm; }
01133     void set_mnm1(uint16_t mnm) { mnm_[1] = mnm; }
01134     void set_mnm2(uint16_t mnm) { mnm_[2] = mnm; }
01135     void set_mns0(uint8_t mns) { mns_[0] = mns; }
01136     void set_mns1(uint8_t mns) { mns_[1] = mns; }
01137     void set_mns2(uint8_t mns) { mns_[2] = mns; }
01138     void set_mpc(uint16_t mpc) { mpc_ = mpc; }
01139     void set_mps(uint8_t mps) { mps_ = mps; }
01140     void set_msg_version(uint8_t msg_version) { msg_version_ = msg_version; }
01141     void set_msn(int16_t msn) { msn_ = msn; }
01142     void set_mtd(uint16_t mtd) { mtd_ = mtd; }
01143     void set_mts(uint32_t mts) { mts_ = mts; }
01144     void set_mtt(uint32_t mtt) { mtt_ = mtt; }
01145     void set_park_start_time(const ptime& park_start_time) { park_start_time_ = park_start_time; }
01146     void set_pbh(uint16_t pbh) { pbh_ = pbh; }
01147     void set_pbl(uint16_t pbl) { pbl_ = pbl; }
01148     void set_pc0(uint16_t pc0) { pc0_ = pc0; }
01149     void set_pc1(uint16_t pc1) { pc1_ = pc1; }
01150     void set_pdt(uint16_t pdt) { pdt_ = pdt; }
01151     void set_pfc(uint16_t pfc) { pfc_ = pfc; }
01152     void set_pfd(uint16_t pfd) { pfd_[0] = pfd; }
01153     void set_pfd0(uint16_t pfd) { pfd_[0] = pfd; }
01154     void set_pfd1(uint16_t pfd) { pfd_[1] = pfd; }
01155     void set_pfd2(uint16_t pfd) { pfd_[2] = pfd; }
01156     void set_pfm(uint8_t pfm) { pfm_[0] = pfm; }
01157     void set_pfm0(uint8_t pfm) { pfm_[0] = pfm; }
01158     void set_pfm1(uint8_t pfm) { pfm_[1] = pfm; }
01159     void set_pfm2(uint8_t pfm) { pfm_[2] = pfm; }
01160     void set_pfr(uint16_t pfr) { pfr_ = pfr; }
01161     void set_pft(uint16_t pft) { pft_[0] = pft; }
01162     void set_pft0(uint16_t pft) { pft_[0] = pft; }
01163     void set_pft1(uint16_t pft) { pft_[1] = pft; }
01164     void set_pft2(uint16_t pft) { pft_[2] = pft; }
01165     void set_pfv(uint16_t pfv) { pfv_[0] = pfv; }
01166     void set_pfv0(uint16_t pfv) { pfv_[0] = pfv; }
01167     void set_pfv1(uint16_t pfv) { pfv_[1] = pfv; }
01168     void set_pfv2(uint16_t pfv) { pfv_[2] = pfv; }
01169     void set_pnb(uint16_t pnb) { pnb_ = pnb; }
01170     void set_pnc(uint16_t pnc) { pnc_ = pnc; }
01171     void set_pns(uint16_t pns) { pns_ = pns; }
01172     void set_pp0(uint16_t pp0) { pp0_ = pp0; }
01173     void set_pp1(uint16_t pp1) { pp1_ = pp1; }
01174     void set_psa(uint8_t psa) { psa_ = psa; }
01175     void set_psd(uint16_t psd) { psd_ = psd; }
01176     void set_psi(uint32_t psi) { psi_ = psi; }
01177     void set_psm(uint8_t psm) { psm_ = psm; }
01178     void set_pss(uint8_t pss) { pss_ = pss; }
01179     void set_pst(uint16_t pst) { pst_ = pst; }
01180     void set_psv(uint16_t psv) { psv_ = psv; }
01181     void set_psw(uint16_t psw) { psw_ = psw; }
01182     void set_ptd(uint32_t ptd) { ptd_= ptd; }
```

```
01183     void set_pth(uint32_t pth) { pth_[0] = pth; }
01184     void set_pth0(uint32_t pth) { pth_[0] = pth; }
01185     void set_pth1(uint32_t pth) { pth_[1] = pth; }
01186     void set_pth2(uint32_t pth) { pth_[2] = pth; }
01187     void set_pto(uint32_t pto) { pto_[0] = pto; }
01188     void set_pto0(uint32_t pto) { pto_[0] = pto; }
01189     void set_pto1(uint32_t pto) { pto_[1] = pto; }
01190     void set_pto2(uint32_t pto) { pto_[2] = pto; }
01191     void set_pv0(uint16_t pv0) { pv0_ = pv0; }
01192     void set_pv5(uint16_t pv5) { pv5_ = pv5; }
01193     void set_pvh(uint16_t pvh) { pvh_ = pvh; }
01194     void set_pvl(uint16_t pvl) { pvl_ = pvl; }
01195     void set_pxc(uint16_t pxc) { pxc_ = pxc; }
01196     void set_sam(uint8_t sam) { sam_ = sam; }
01197     void set_san(uint16_t san) { san_ = san; }
01198     void set_sao(uint16_t sao) { sao_ = sao; }
01199     void set_sap(uint16_t sap) { sap_ = sap; }
01200     void set_sax(uint16_t sax) { sax_ = sax; }
01201     void set_sbm(uint8_t sbm) { sbm_ = sbm; }
01202     void set_sbt(uint16_t sbt) { sbt_ = sbt; }
01203     void set_sdc(int16_t sdc) { sdc_ = sdc; }
01204     void set_sds(uint16_t sds) { sds_ = sds; }
01205     void set_sdt(uint32_t sdt) { sdt_ = sdt; }
01206     void set_sensor_type(uint8_t sensor_type) { sensor_type_ = sensor_type; }
01207     void set_set(uint16_t set) { set_ = set; }
01208     void set_sfn(uint16_t sfn) { sfn_ = sfn; }
01209     void set_sgt(uint16_t sgt) { sgt_ = sgt; }
01210     void set_spt(uint16_t spt) { spt_ = spt; }
01211     void set_ssa(uint16_t ssa) { ssa_ = ssa; }
01212     void set_ssi(uint32_t ssi) { ssi_ = ssi; }
01213     void set_ssp(uint16_t ssp) { ssp_ = ssp; }
01214     void set_surface_start_time(const ptime& surface_start_time) { surface_start_time_ =
      surface_start_time; }
01215     void set_ddb(uint8_t ddb) { ddb_ = ddb; }
01216     void set_xas(uint8_t xas) { xas_ = xas; }
01217     void set_aps(uint16_t aps) { aps_ = aps; }
01218     void set_wmt(uint16_t wmt) { wmt_ = wmt; }
01219
01220     void set_ppc_alpha(float ppc_alpha) {
01221       ppc_alpha_ = ppc_alpha;
01222     }
01223
01224     void set_ppc_ctccoeff(float ppc_ctccoeff) {
01225       ppc_ctccoeff_ = ppc_ctccoeff;
01226     }
01227
01228     void set_ppc_tau(float ppc_tau) {
01229       ppc_tau_ = ppc_tau;
01230     }
01231
01232     void set_ppc_tdelay(float ppc_tdelay) {
01233       ppc_tdelay_ = ppc_tdelay;
01234     }
01235
01236     void set_ppe(uint8_t ppe) {
01237       ppe_ = ppe;
01238     }
01239
01240
01241 };
01242
01243 } /* namespace xmsg */
01244 } /* namespace util */
01245
01246 #endif /* DATA_XMSG_XMFAMISSIONCFG_H_ */
```

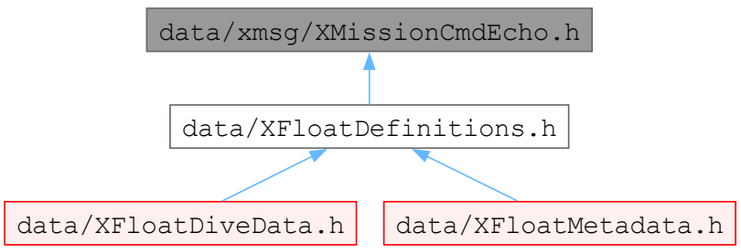
11.144 data/xmsg/XMissionCmdEcho.h File Reference

The util::xmsg::XMissionCmdEcho class source file.

`#include "XMessage.h"`
Include dependency graph for XMissionCmdEcho.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XMissionCmdEcho

  Class representing a ground station command echo-back X-Message.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.144.1 Detailed Description

The util::xmsg::XMissionCmdEcho class source file.

Version

0.6

Date

Jul 1, 2014 2:13:38 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.145 XMissionCmdEcho.h

Go to the documentation of this file.

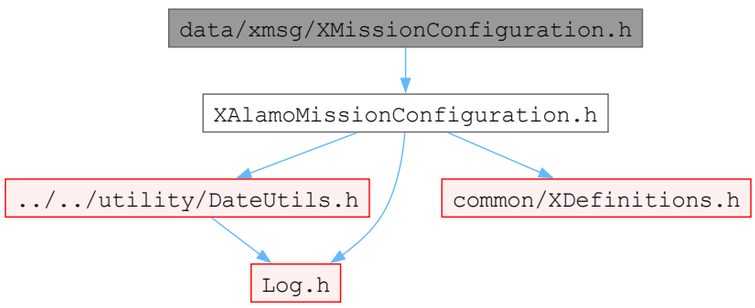
```
00001 /** The @ref util::xmsg::XMissionCmdEcho class source file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jul 1, 2014 2:13:38 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef XMISSIONCMDECHO_H_
00024 #define XMISSIONCMDECHO_H_
00025
00026 /* Includes */
00027
00028 #include "XMessage.h"
00029
00030 /* Definitions */
00031
00032 namespace util {
00033 namespace xmsg {
00034
00035 /* Class Definitions */
00036
00037 /** Class representing a ground station command echo-back X-Message.
00038  *
00039  */
00040 class XMissionCmdEcho:
00041         public XMessage {
00042
00043 public:
00044
00045     /* Member Functions */
00046
00047     /** The parameterized constructor.
00048      *
00049      *  @param[in]      dive        The dive or cycle number of this message.
00050      *  @param[in]      whoi        The unique WHOI identifier.
00051      *  @param[in]      data        The binary X-message data to decode.
00052      *
00053      *  @throw          util::log::MessageException if an decode error occurs.
00054      */
00055     XMissionCmdEcho(int16_t dive, uint16_t whoi, const vector<uint8_t>& data) :
00056                     XMessage(dive, whoi, data) {
00057         DecodeXMsgData(data); // Decode message.
00058     }
00059
00060     /** The destructor. */
00061     ~XMissionCmdEcho() {
00062         /* Do nothing. */
00063     }
00064
00065     /** Decode the ground station command echo-back X-Message from the binary message data.
00066      *
00067      *  @param[in]      data        The binary message data to decode.
00068      *
00069      *  @throw          util::log::MessageException if the message is invalid or an error occurs during
00070      *                  decoding.
00071      */
00072     void DecodeXMsgData(const vector<uint8_t>& data) {
00073
00074         try {
00075
00076             // Check message.
00077             /* const uint8_t* d = */CheckXMsgData(data, XMSG_TYPE_ID_MISS_CMD_ECHO, -1);
00078
00079             /** @todo   Implement the functionality to decode the ground station command echo-back
00080              *          X-Message from the binary message data.
00081              *
00082              */
00083
00084         }CATCH_BLOCK("Failed to decode ground station command echo-back message!")
00085     }
00086
00087     /** Generate a listing string representation of the ground station command echo-back message.
00088      *
00089      *  @return     A listing string representation of the ground station command echo-back message.
00090      */
00091     const string GenerateListingString() const {
00092         ostringstream os;
00093
00094         try {
00095
00096             /** @todo   Implement the functionality for generating a listing string for the ground
00097              *          station upload command echo-back message.
00098              *
00099              */
00100
00101         }CATCH_BLOCK("Failed to generate listing string ground station command echo-back message!")
00102
00103         return os.str();
```

```
00104     }
00105 };
00106
00107 } /* namespace xmsg */
00108 } /* namespace util */
00109 #endif /* XMISSIONCMDECHO_H_ */
```

11.146 data/xmsg/XMissionConfiguration.h File Reference

The util::xmsg::XMissionConfiguration class header file.

`#include "XAlamoMissionConfiguration.h"`
Include dependency graph for XMissionConfiguration.h:



Classes

- class util::xmsg::XMissionConfiguration
  Class representing a mission configuration X-Message.

Namespaces

- namespace util
  The utility namespace.
- namespace util::xmsg
  The X-Message utility namespace.

11.146.1 Detailed Description

The util::xmsg::XMissionConfiguration class header file.

Version

0.6

Date

Jul 1, 2014 11:19:37 AM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.147 XMissionConfiguration.h

Go to the documentation of this file.

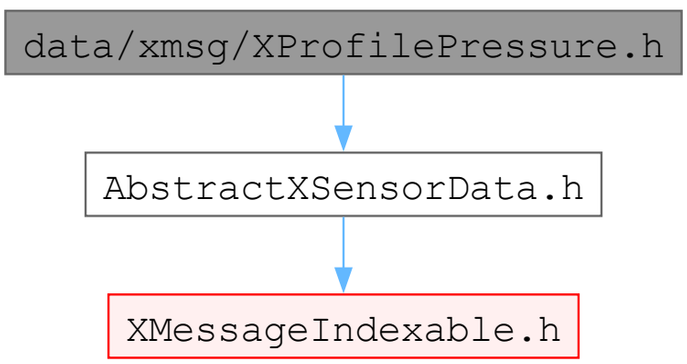
```
00001 /** The @ref util::xmsg::XMissionConfiguration class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jul 1, 2014 11:19:37 AM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef XMISSIONCONFIGURATION_H_
00024 #define XMISSIONCONFIGURATION_H_
00025
00026 /* Includes */
00027
00028 // local
00029 #include "XAlamoMissionConfiguration.h"
00030
00031 /* Definitions */
00032
00033 namespace util {
00034 namespace xmsg {
00035
00036 /** Class representing a mission configuration X-Message.
00037  *
00038  *  @note           This object supports both ALAMO and Solo-II mission configurations.
00039  */
00040 class XMissionConfiguration:
00041         public XMessageIndexable {
00042
00043 private:
00044
00045     /* Data Members */
00046
00047     /** A shared pointer to the ALAMO mission configuration object.
00048      *
00049      *  @note       This pointer is only used for ALAMO-type mission configuration messages. If
00050      *              this is not an ALAMO-type message, then this pointer will be null.
00051      */
00052     shared_ptr<XAlamoMissionConfiguration> alamo_mission_configuration_;
00053
00054 public:
00055
00056     /* Member Functions */
00057
00058     /** The parameterized constructor.
00059      *
00060      *  @param[in]      dive        The dive or cycle number of this message.
00061      *  @param[in]      whoi        The unique WHOI identifier.
00062      *  @param[in]      data        The binary mission configuration X-message data to decode.
00063      *
00064      *  @throw          util::log::MessageException if an decode error occurs.
00065      */
00066     XMissionConfiguration(int16_t dive, uint16_t whoi, const vector<uint8_t>& data) :
00067                     XMessageIndexable(dive, whoi, data) {
00068         DecodeXMsgData(data);    // Decode message.
00069     }
00070
00071     /** The destructor.
00072      *
00073      */
00074     ~XMissionConfiguration() {
00075         /* Do nothing. */
00076     }
00077
00078     /** Decode the mission configuration message from the binary data.
00079      *
00080      *  @param[in]      data            The binary message to decode.
00081      */
00082     void DecodeXMsgData(const vector<uint8_t>& data) {
00083
00084         try {
00085
00086             const uint8_t* d = CheckXMsgData(data, (int) -1, (int) 5);
00087
00088             // Check ID.
00089             if((d[0] & 0xF0) != 0xD0) {
00090
00091                 using namespace std;
00092                 ostringstream os;
00093                 os « "Invalid X-Message identifier: Expected 0xD[0:7], but received "
00094                    « hex
00095                    « setw(2)
00096                    « setfill('0')
00097                    « (int) d[0]
00098                    « "!";
00099                 BOOST_THROW_EXCEPTION(log::MessageException(os));
00100
00101             }
00102
00103             // Get message index.
```

```
00104             set_xmsg_index(d[0] & 0x0F);
00105
00106             // Check encoding type.
00107             if(d[3] == 0 && xmsg_index() == 0) {
00108
00109                 // This is binary encoded (ALAMO) message.
00110                 alamo_mission_configuration_ = std::make_shared<XAlamoMissionConfiguration>(data);
00111
00112             } else {
00113
00114                 // This is an ASCII encoded (Solo-II) message.
00115
00116             }
00117
00118         }CATCH_BLOCK("Failed to decode mission configuration X message!")
00119     }
00120
00121     /** Get the listing entry string for this mission configuration X-Message.
00122      *
00123      *  @return         The listing entry string for this mission configuration X-Message.
00124      */
00125     const string GenerateListingString() const {
00126         ostringstream os;
00127
00128         try {
00129
00130             if(alamo_mission_configuration_)
00131                 os « alamo_mission_configuration_->GenerateListingString(
00132                         id_instrument(),
00133                         id_dive(),
00134                         xmsg_index());
00135
00136         }CATCH_BLOCK("Failed to generate listing string for mission configuration object!")
00137
00138         return os.str();
00139     }
00140
00141 };
00142
00143 } /* namespace xmsg */
00144 } /* namespace util */
00145
00146 #endif /* XMISSIONCONFIGURATION_H_ */
```

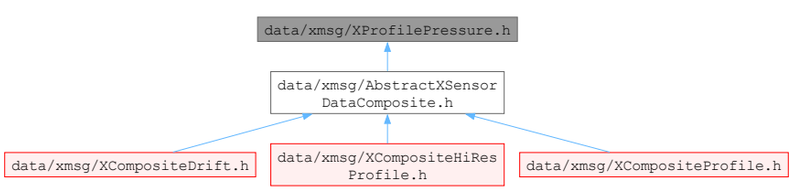
11.148 data/xmsg/XProfilePressure.h File Reference

The util::xmsg::XProfilePressure class header file.

`#include "AbstractXSensorData.h"`
Include dependency graph for XProfilePressure.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XProfilePressure

  Class representing a profile pressure data X-Message.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.148.1 Detailed Description

The util::xmsg::XProfilePressure class header file.

Version

0.6

Date

Jul 1, 2014 9:45:35 AM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.149 XProfilePressure.h

Go to the documentation of this file.

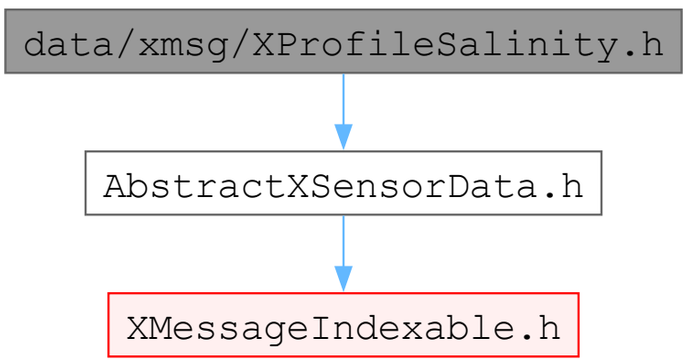
```
00001 /** The @ref util::xmsg::XProfilePressure class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jul 1, 2014 9:45:35 AM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef XPROFILEPRESSURE_H_
00024 #define XPROFILEPRESSURE_H_
00025
00026 /* Includes */
00027
00028 // local
00029 #include "AbstractXSensorData.h"
00030
00031 /* Definitions */
00032
00033 namespace util {
```

```
00034 namespace xmsg {
00035
00036 /* Class Definitions */
00037
00038 /** Class representing a profile pressure data X-Message.
00039  *
00040  */
00041 class XProfilePressure:
00042         public AbstractXSensorData {
00043
00044 public:
00045
00046     /* Member Functions */
00047
00048     /** Parameterized constructor.
00049      *
00050      *  @param[in]      dive        The dive or cycle number of this message.
00051      *  @param[in]      whoi        The unique WHOI identifier.
00052      *  @param[in]      data        The binary profile pressure data X-message data to decode.
00053      *
00054      *  @throw          util::log::MessageException if an decode error occurs.
00055      */
00056     XProfilePressure(
00057             int16_t dive,
00058             uint16_t whoi,
00059             const vector<uint8_t>& data) :
00060                     AbstractXSensorData(dive, whoi, data) {
00061         DecodeXMsgData(data);    // Decode GPS message.
00062     }
00063
00064     /** Destructor.
00065      *
00066      */
00067     ~XProfilePressure() {
00068         /* Do nothing. */
00069     }
00070
00071     /** Decode the bin-averaged profile pressure X-Message from the binary message data.
00072      *
00073      *  @param[in]      data        The binary message data to decode.
00074      *
00075      *  @throw          util::log::MessageException if the message is invalid or an error occurs during
00076      *                  decoding.
00077      */
00078     void DecodeXMsgData(const vector<uint8_t>& data) {
00079         try {
00080             const uint8_t* d = CheckXMsgData(data, (int) -1, (int) 5);
00081             if((d[0] & 0xF0) != (XMSG_TYPE_ID_PRES0 & 0xF0))
00082                 EXC_THROW("Invalid _X-Message unique identifier!");
00083             UnpackSensorData(data);
00084         }CATCH_BLOCK("Failed to decode profile pressure X-Message!")
00085     }
00086
00087 };
00088
00089 } /* namespace xmsg */
00090 } /* namespace util */
00091 #endif /* XPROFILEPRESSURE_H_ */
```

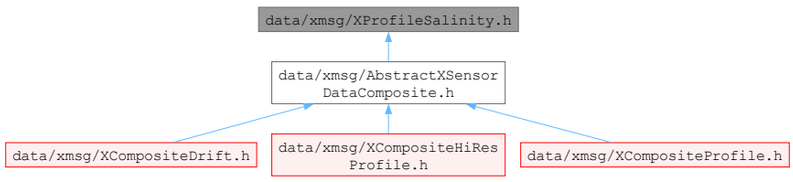
11.150 data/xmsg/XProfileSalinity.h File Reference

The util::xmsg::XProfileSalinity class header file.

`#include "AbstractXSensorData.h"`
Include dependency graph for XProfileSalinity.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XProfileSalinity

  Class representing a profile salinity X-Message.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.150.1 Detailed Description

The util::xmsg::XProfileSalinity class header file.

Version

0.6

Date

Jul 1, 2014 11:16:50 AM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.151 XProfileSalinity.h

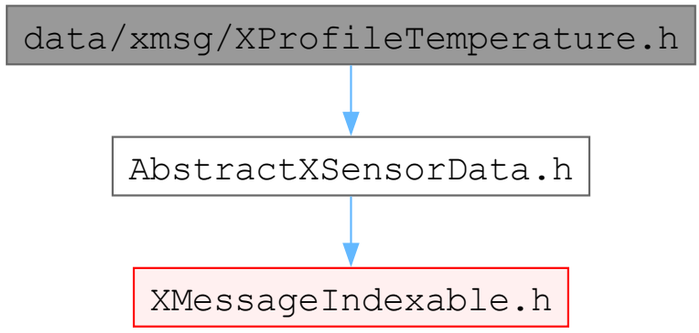
Go to the documentation of this file.

```
00001 /** The @ref util::xmsg::XProfileSalinity class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jul 1, 2014 11:16:50 AM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef XPROFILESALINITY_H_
00024 #define XPROFILESALINITY_H_
00025
00026 /* Includes */
00027
00028 #include "AbstractXSensorData.h"
00029
00030 namespace util {
00031 namespace xmsg {
00032
00033 /** Class representing a profile salinity X-Message.
00034  *
00035  */
00036 class XProfileSalinity:
00037         public AbstractXSensorData {
00038
00039 public:
00040
00041     /* Member Functions */
00042
00043     /** The parameterized constructor.
00044      *
00045      *  @param[in]      dive        The dive or cycle number of this message.
00046      *  @param[in]      whoi        The unique WHOI identifier.
00047      *  @param[in]      data        The binary profile salinity X-message data to decode.
00048      *
00049      *  @throw          util::log::MessageException if an decode error occurs.
00050      */
00051     XProfileSalinity(int16_t dive, uint16_t whoi, const vector<uint8_t>& data) :
00052                     AbstractXSensorData(dive, whoi, data) {
00053         DecodeXMsgData(data);    // Decode GPS message.
00054     }
00055
00056     /** The destructor.
00057      *
00058      */
00059     ~XProfileSalinity() {
00060         /* Do nothing. */
00061     }
00062
00063     /** Decode the bin-averaged profile salinity X-Message from the binary message data.
00064      *
00065      *  @param[in]      data        The binary message data to decode.
00066      *
00067      *  @throw          util::log::MessageException if the message is invalid or an error occurs during
00068      *                  decoding.
00069      */
00070     void DecodeXMsgData(const vector<uint8_t>& data) {
00071         try {
00072             const uint8_t* d = CheckXMsgData(data, (int) -1, (int) 5);
00073             if((d[0] & 0xF0) != (XMSG_TYPE_ID_SALN0 & 0xF0))
00074                 EXC_THROW("Invalid X-Message unique identifier!");
00075             UnpackSensorData(data);
00076         }CATCH_BLOCK("Failed to decode profile salinity X-Message!")
00077     }
00078 };
00079
00080 } /* namespace xmsg */
00081 } /* namespace util */
00082 #endif /* XPROFILESALINITY_H_ */
```

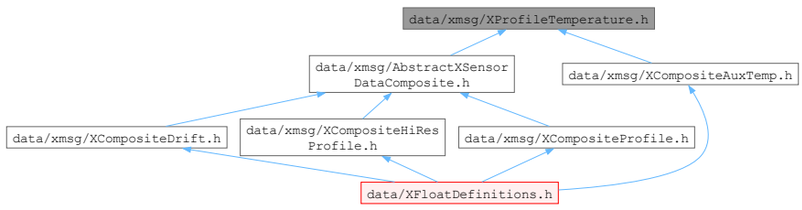
11.152 data/xmsg/XProfileTemperature.h File Reference

The util::xmsg::XProfileTemperature class header file.

```
#include "AbstractXSensorData.h"
```
Include dependency graph for XProfileTemperature.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XProfileTemperature

  Class representing a profile temperature X-Message.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.152.1 Detailed Description

The util::xmsg::XProfileTemperature class header file.

Version

0.6

Date

Jul 1, 2014 11:16:19 AM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.153 XProfileTemperature.h

Go to the documentation of this file.

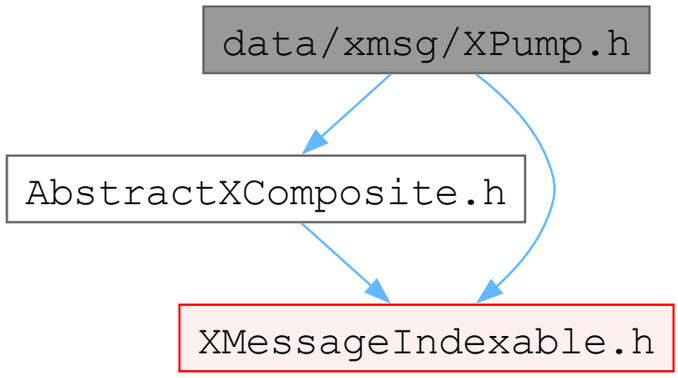
```
00001 /** The @ref util::xmsg::XProfileTemperature class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jul 1, 2014 11:16:19 AM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef XPROFILETEMPERATURE_H_
00024 #define XPROFILETEMPERATURE_H_
00025
00026 /* Includes */
00027
00028 #include "AbstractXSensorData.h"
00029
00030 /* Definitions */
00031
00032 namespace util {
00033   namespace xmsg {
00034
00035     /* Class Definitions */
00036
00037     /** Class representing a profile temperature X-Message.
00038      *
00039      */
00040     class XProfileTemperature:
00041         public AbstractXSensorData {
00042
00043
00044       public:
00045
00046         /* Member Functions */
00047
00048         /** Parameterized constructor.
00049          *
00050          *  @param[in]      dive        The dive or cycle number of this message.
00051          *  @param[in]      whoi        The unique WHOI identifier.
00052          *  @param[in]      data        The binary profile temperature X-message data to decode.
00053          *
00054          *  @throw          util::log::MessageException if an decode error occurs.
00055          */
00056         XProfileTemperature(int16_t dive, uint16_t whoi, const vector<uint8_t>& data) :
00057                 AbstractXSensorData(dive, whoi, data) {
00058           DecodeXMsgData(data);    // Decode GPS message.
00059         }
00060
00061         /** Destructor.
00062          *
00063          */
00064         ~XProfileTemperature() {
00065           /* Do nothing. */
00066         }
00067
00068         /** Decode the bin-averaged profile temperature X-Message from the binary message data.
00069          *
00070          *  @param[in]      data        The binary message data to decode.
00071          *
00072          *  @throw          util::log::MessageException if the message is invalid or an error occurs
      during
00073          *                  decoding.
00074          */
00075         void DecodeXMsgData(const vector<uint8_t>& data) {
00076           try {
00077             const uint8_t* d = CheckXMsgData(data, (int) -1, (int) 5);
00078             if((d[0] & 0xF0) != (XMSG_TYPE_ID_TEMP0 & 0xF0))
00079               EXC_THROW("Invalid X-Message unique identifier!");
00080             UnpackSensorData(data);
00081           }CATCH_BLOCK("Failed to decode profile temperature X-Message!")
00082         }
00083     };
00084
00085   } /* namespace xmsg */
00086 } /* namespace util */
00087 #endif /* XPROFILETEMPERATURE_H_ */
```

11.154 data/xmsg/XPump.h File Reference

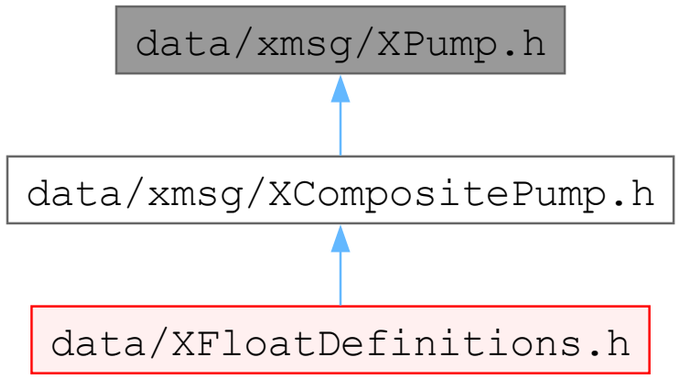
The util::xmsg::XPump class header file.

```
#include "AbstractXComposite.h"
#include "XMessageIndexable.h"
```

Include dependency graph for XPump.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XPump

  Class representing a pump data X-Message.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

Enumerations

- enum util::xmsg::XPumpPackingFormat_t {
  util::xmsg::XPUMP_PACK_LEGACY = 0 ,
  util::xmsg::XPUMP_PACK_STANDARD = 1 ,
  util::xmsg::XPUMP_PACK_STANDARD2 = 2 ,
  util::xmsg::XPUMP_PACK_UNKNOWN = 99 }

  Enumeration type definition of pump data X-Message packing formats.

11.154.1 Detailed Description

The util::xmsg::XPump class header file.

Version

0.6

Date

Jul 1, 2014 11:18:42 AM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.155 XPump.h

Go to the documentation of this file.

```
00001 /** The @ref util::xmsg::XPump class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jul 1, 2014 11:18:42 AM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef XPUMP_H_
00024 #define XPUMP_H_
00025
00026 /* Includes */
00027
00028 // local
00029 #include "AbstractXComposite.h"
00030 #include "XMessageIndexable.h"
00031
00032 /* Definitions */
00033
00034 namespace util {
00035 namespace xmsg {
00036
00037 /* Type Definitions */
00038
00039 /** Enumeration type definition of pump data X-Message packing formats. */
00040 typedef enum XPumpPackingFormat {
00041
00042     /** X-Message pump data packing format 0: The legacy packing format identifier. */
00043     XPUMP_PACK_LEGACY = 0,
00044
00045     /** X-Message pump data packing format 1: The standard packing format identifier. */
00046     XPUMP_PACK_STANDARD = 1,
00047
00048     /** X-Message pump data packing format 2: The (tentative) packing format identifier. */
00049     XPUMP_PACK_STANDARD2 = 2,
00050
00051     /** X-Message pump data packing format 99: The unknown/unset/invalid packing format identifier. */
00052     XPUMP_PACK_UNKNOWN = 99
00053 } XPumpPackingFormat_t;
00054
00055 /* Class Definitions */
00056
```

```
00057 /** Class representing a pump data X-Message.
00058  *
00059  */
00060 class XPump:
00061         public AbstractXComposite<XPumpCounts>,
00062         public XMessageIndexable {
00063
00064 private:
00065
00066     /* Private Data Members: */
00067
00068     /** The packing format of this pump data message. */
00069     XPumpPackingFormat_t pump_packing_format_;
00070
00071 public:
00072
00073     /* Public Member Functions */
00074
00075     /** The parameterized constructor.
00076      *
00077      *  @param[in]      dive        The dive or cycle number of this message.
00078      *  @param[in]      whoi        The unique WHOI identifier.
00079      *  @param[in]      data        The binary X-message data to decode.
00080      *
00081      *  @throw          util::log::MessageException if an decode error occurs.
00082      */
00083     XPump(int16_t dive, uint16_t whoi, const vector<uint8_t>& data) :
00084                     XMessageIndexable(dive, whoi, data),
00085                     pump_packing_format_(XPUMP_PACK_UNKNOWN) {
00086
00087         // Decode the message data.
00088         DecodeXMsgData(data);
00089     }
00090
00091     /** The destructor.
00092      *
00093      */
00094     ~XPump() {
00095         /* Do nothing. */
00096     }
00097
00098     /** Decode the pump data message from the binary data to decode.
00099      *
00100      *  @param[in]      data        The binary data to decode.
00101      */
00102     void DecodeXMsgData(const vector<uint8_t>& data) {
00103         try {
00104             const uint8_t* d = CheckXMsgData(data, -1, 3, false);
00105
00106             set_xmsg_index(0x0F & d[0]);       // Get the message index.
00107
00108             {    // Get the message packing format.
00109                 int f;
00110                 if((f = (d[1] & 0xF0) » 4) == 2) {
00111                     pump_packing_format_ = XPUMP_PACK_STANDARD2;
00112                     set_xmsg_size_bytes(((d[1] & 0x0F) « 8) | d[2]);
00113                 } else if(f == 1) {
00114                     pump_packing_format_ = XPUMP_PACK_STANDARD;
00115                     set_xmsg_size_bytes(((d[1] & 0x0F) « 8) | d[2]);
00116                 } else if(f == 0) {
00117                     pump_packing_format_ = XPUMP_PACK_LEGACY;
00118                     set_xmsg_size_bytes((d[1] « 8) | d[2]);
00119                 } else {
00120                     ostringstream os;
00121                     os « "Invalid pump packing format specifier! Expected 0 (legacy) or 1 "
00122                        "(standard), but received "
00123                        « f « "!";
00124                     BOOST_THROW_EXCEPTION(log::MessageException(os));
00125                 }
00126             }
00127
00128             // Check message size.
00129             if(xmsg_size_bytes() > data.size()) {
00130                 ostringstream os;
00131                 os « "Invalid XMessage size: message specifies at least "
00132                    « xmsg_size_bytes()
00133                    « " bytes, but only "
00134                    « data.size()
00135                    « " bytes were received.";
00136                 BOOST_THROW_EXCEPTION(log::MessageException(os));
00137             } else if(data.size() > xmsg_size_bytes()) {
00138                 lwrn() « "Extra bytes in XMessage: message specifies "
00139                           « xmsg_size_bytes()
00140                           « "bytes, but "
00141                           « data.size()
00142                           « " bytes were received.";
00143             }
00144
00145             // Check for terminator.
00146             if(d[xmsg_size_bytes() - 1] != 0x3B) {
00147                 ostringstream os;
00148                 os « "Invalid XMessage: expected terminator ';' (0x3B) at byte "
00149                    « (xmsg_size_bytes() - 1)
00150                    « ", but received 0x"
00151                    « hex
00152                    « d[xmsg_size_bytes() - 1];
00153                 BOOST_THROW_EXCEPTION(log::MessageException(os));
00154             }
00155
00156             {    // Iterate over the sensor data.
00157                 int32_t presCounts;
00158                 int16_t pumpSecs;
00159                 uint16_t voltage;
00160                 uint16_t current;
00161                 uint16_t vac0 = XMSG_BAD_U16;
00162                 uint16_t vac1;
00163                 if(pump_packing_format_ == XPUMP_PACK_LEGACY)
00164                     for(vector<uint8_t>::const_iterator it = data.begin() + 3; it < data.end() - 10;
```

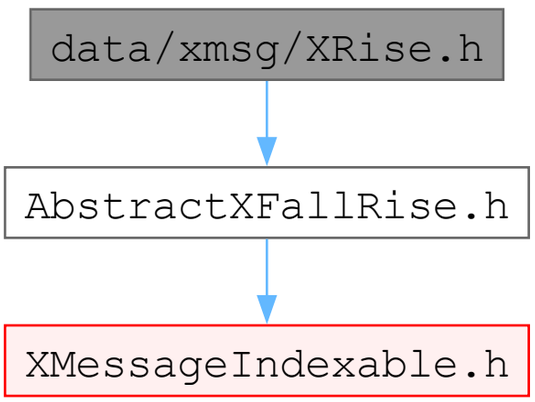
```
00165                           ) {
00166                       presCounts = ((*it++) « 8) | (*it++);
00167                       pumpSecs = ((*it++) « 8) | (*it++);
00168                       voltage = ((*it++) « 8) | (*it++);
00169                       current = ((*it++) « 8) | (*it++);
00170                       vac0 = *it++;
00171                       vac1 = *it++;
00172                       AppendElement(
00173                               XPumpCounts(
00174                                       XPHASE_CODE_UNKNOWN,
00175                                       presCounts,
00176                                       pumpSecs,
00177                                       voltage,
00178                                       current,
00179                                       vac0,
00180                                       vac1));
00181                   }
00182               else if(pump_packing_format_ == XPUMP_PACK_STANDARD)
00183                   for(vector<uint8_t>::const_iterator it = data.begin() + 3; it < data.end() - 11;
00184                           ) {
00185                       XPhaseCode_t phaseCode = (XPhaseCode_t) (*it » 4);
00186                       presCounts = (((*it++ & 0x0F) « 16)) | ((*it++) « 8) | (*it++);
00187                       pumpSecs = ((*it++) « 8) | (*it++);
00188                       voltage = ((*it++) « 8) | (*it++);
00189                       current = ((*it++) « 8) | (*it++);
00190                       vac0 = *it++;
00191                       vac1 = *it++;
00192                       AppendElement(
00193                               XPumpCounts(
00194                                       phaseCode,
00195                                       presCounts,
00196                                       pumpSecs,
00197                                       voltage,
00198                                       current,
00199                                       vac0 * 10,
00200                                       vac1 * 10));
00201                   }
00202               else if(pump_packing_format_ == XPUMP_PACK_STANDARD2)
00203                   for(vector<uint8_t>::const_iterator it = data.begin() + 3; it < data.end() - 11;
00204                           ) {
00205                       XPhaseCode_t phaseCode = (XPhaseCode_t) (*it » 4);
00206                       presCounts = (((*it++ & 0x0F) « 16)) | ((*it++) « 8) | (*it++);
00207                       pumpSecs = ((*it++) « 8) | (*it++);
00208                       voltage = ((*it++) « 8) | (*it++);
00209                       current = ((*it++) « 8) | (*it++);
00210                       vac1 = ((*it++) « 8) | (*it++);
00211                       AppendElement(
00212                               XPumpCounts(
00213                                       phaseCode,
00214                                       presCounts,
00215                                       pumpSecs,
00216                                       voltage,
00217                                       current,
00218                                       vac0,
00219                                       vac1));
00220                   }
00221           }
00222       }CATCH_BLOCK("Failed to decode pump data X-Message!")
00223   }
00224
00225   /** The the listing file comment string.
00226    *
00227    *  @return         The listing file comment string.
00228    */
00229   static const string GenerateListingComment() {
00230       ostringstream os;
00231
00232       try {
00233           const format fmt("#   %5s %4s %3s %4s %10s %10s %10s %10s %10s %10s\n"
00234                   "#   %5s %4s %3s %4s  %10s %10s %10s %10s %10s %10s");
00235           os « format(fmt) % "SN" % "dive" % "idx" % "nSam" % "pres" % "pumpT" % "pumpV"
00236                   % "pumpI" % "vac0" % "vac1" % "" % "" % "" % "" % "(dBar)" % "(sec)" % "(V)"
00237                   % "(A)" % "(inHg)" % "(inHg)";
00238       }CATCH_BLOCK("Failed to generate listing comment for pump data object!")
00239
00240       return os.str();
00241   }
00242
00243   /** Generate the listing file entry string for this pump data message.
00244    *
00245    *  @param[in]  pg      The pressure sensor sensitivity scale factor, in LSB/dBar.
00246    *  @param[in]  po      The pressure sensor offset, in dBar.
00247    *  @param[in]  pc      The pressure correction, in dBar, to be subtracted computed pressure
00248    *                      value.
00249    *
00250    *  @throw      log::MessageException if an error occurs while generating the listing string.
00251    *
00252    *  @return     The listing file entry string for this pump data message.
00253    */
00254   const string GenerateListingString(uint16_t pg, uint16_t po, float pc) const {
00255       ostringstream os;
00256       size_t sam = 0;
00257
00258       try {
00259           {
00260               const format fmt("PMP %5d %4d %3d %4d");
00261               os « format(fmt) % id_instrument() % id_dive() % (int) xmsg_index()
00262                       % num_elements();
00263           }
00264           {
00265               const format fmt("\npmp %5d %4d %3d %4d ");
00266               vector<XPumpCounts>::const_iterator it;
00267               for(it = elements().begin(); it < elements().end(); it++)
00268                   os « format(fmt) % id_instrument() % id_dive() % (int) xmsg_index() % sam++
00269                      « it->GenerateListingString(pg, po, pc);
00270           }
00271       }CATCH_BLOCK("Failed to generate listing string from pump data object!")
00272
```

```
00273         return os.str();
00274     }
00275
00276     /** Get the enumerated pump packing format identifier.
00277      *
00278      *  @return         The enumerated pump packing format identifier.
00279      */
00280     XPumpPackingFormat_t pump_packing_format() const {
00281         return pump_packing_format_;
00282     }
00283
00284 };
00285
00286 } /* namespace xmsg */
00287 } /* namespace util */
00288 #endif /* XPUMP_H_ */
```

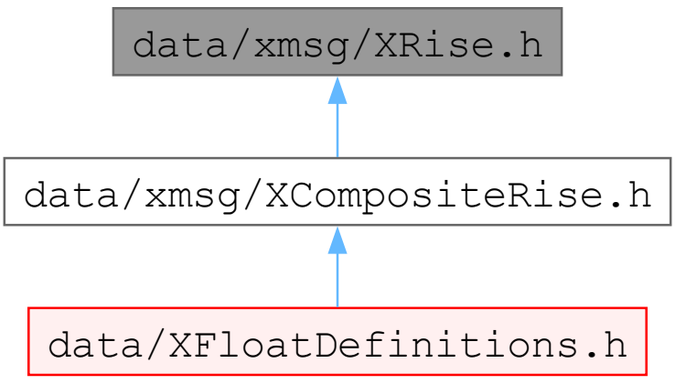
11.156 data/xmsg/XRise.h File Reference

The util::xmsg::XRise class header file.

`#include "AbstractXFallRise.h"`
Include dependency graph for XRise.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XRise

  Class representing a rise data X-Message.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.156.1 Detailed Description

The util::xmsg::XRise class header file.

Version

0.6

Date

Jul 1, 2014 11:18:24 AM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.157 XRise.h

Go to the documentation of this file.

```
00001 /** The @ref util::xmsg::XRise class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jul 1, 2014 11:18:24 AM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef XRISE_H_
00024 #define XRISE_H_
00025
00026 /* Includes */
00027
00028 #include "AbstractXFallRise.h"
00029
00030 /* Definitions */
00031
00032 namespace util {
00033 namespace xmsg {
00034
00035 /* Class Definitions */
00036
00037 /** Class representing a rise data X-Message.
00038  *
00039  */
00040 class XRise:
00041         public AbstractXFallRise {
00042
```

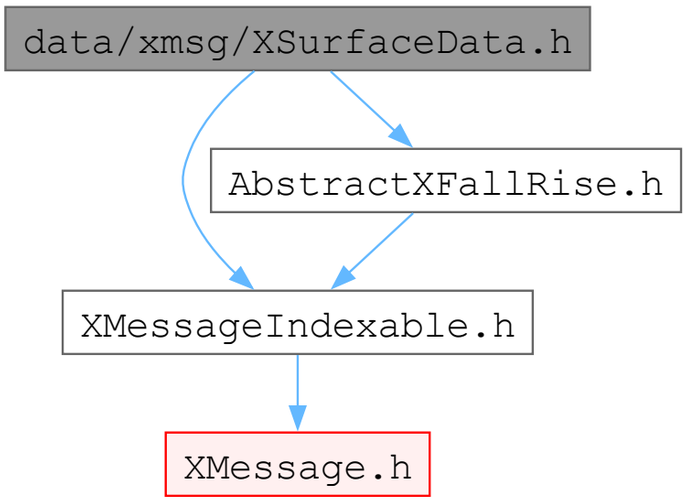
```
00043 public:
00044
00045     /* Member Functions */
00046
00047     /** The parameterized constructor.
00048      *
00049      *  @param[in]      dive        The dive or cycle number of this message.
00050      *  @param[in]      whoi        The unique WHOI identifier.
00051      *  @param[in]      data        The binary X-message data to decode.
00052      *
00053      *  @throw          util::log::MessageException if an decode error occurs.
00054      */
00055     XRise(int16_t dive, uint16_t whoi, const vector<uint8_t>& data) :
00056                     AbstractXFallRise(dive, whoi, data) {
00057         DecodeXMsgData(data);    // Decode GPS message.
00058     }
00059
00060     /** The destructor.
00061      *
00062      */
00063     ~XRise() {
00064         /* Do nothing. */
00065     }
00066
00067     /** Decode the rise data X-Message from the binary message data.
00068      *
00069      *  @param[in]      data        The binary message data to decode.
00070      */
00071     void DecodeXMsgData(const vector<uint8_t>& data) {
00072         try {
00073             if((data.data()[0] & 0xF0) != (XMSG_TYPE_ID_RISE0 & 0xF0))
00074                 EXC_THROW("Invalid X-Message unique identifier!");
00075             UnpackData(data);
00076         }CATCH_BLOCK("Failed to decode rise rate X-Message!")
00077     }
00078
00079     /** Get the listing entry string for this rise data X-Message.
00080      *
00081      *  @param[in]  pg      The pressure sensor sensitivity scale factor, in LSB/dBar.
00082      *  @param[in]  po      The pressure sensor offset, in dBar.
00083      *  @param[in]  sg      The salinity sensor sensitivity scale factor, in LSB/PSU.
00084      *  @param[in]  so      The salinity sensor offset, in PSU.
00085      *  @param[in]  tg      The temperature sensor sensitivity scale factor, in LSB/degC.
00086      *  @param[in]  to      The temperature sensor offset, in degC.
00087      *  @param[in]  pc      The pressure correction, in dBar, to be subtracted computed pressure
00088      *                      value.
00089      *
00090      *  @throw      log::MessageException if an error occurs while generating the listing string.
00091      *
00092      *  @return     The listing entry string for this rise data X-Message.
00093      */
00094     const string GenerateListingString(
00095             uint16_t pg,
00096             uint16_t po,
00097             uint16_t sg,
00098             uint16_t so,
00099             uint16_t tg,
00100             uint16_t to,
00101             float pc) const {
00102
00103         ostringstream os;
00104         size_t sam = 0;
00105
00106         try {
00107             {
00108                 const format fmt("R   %5d %4d %3d %4d %20s");
00109                 os « format(fmt) % id_instrument() % id_dive() % (int) xmsg_index() % num_samples()
00110                         % GenerateIso8601String(start_time());
00111             }
00112
00113             {
00114                 const format fmt("\nr   %5d %4d %3d %4d ");
00115                 for_each(
00116                         sensor_data().begin(),
00117                         sensor_data().end(),
00118                         [&](const XSensorTripletCodeCounts& sensor_data) {
00119                             os « format(fmt) % id_instrument() % id_dive() % (int) xmsg_index() % sam++
00120                             « sensor_data.GenerateListingString(pg, po, sg, so, tg, to, pc);
00121                         });
00122             }
00123         }CATCH_BLOCK("Failed to generate listing string for rise time-series data object!")
00124
00125         return os.str();
00126     }
00127 };
00128
00129 } /* namespace xmsg */
00130 } /* namespace util */
00131 #endif /* XRISE_H_ */
```

11.158 data/xmsg/XSurfaceData.h File Reference

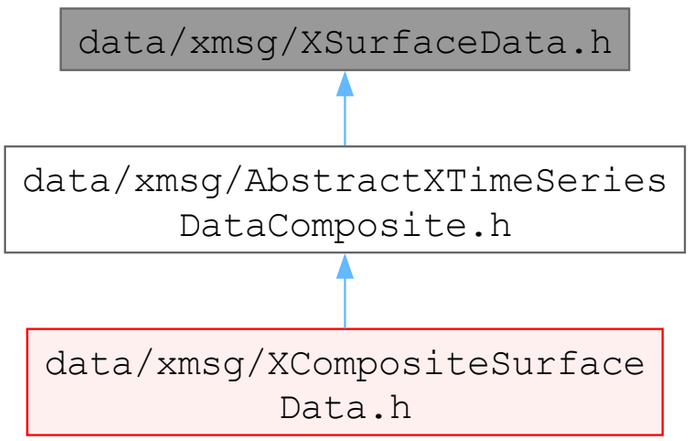
The util::xmsg::XSurfaceData class header file.

```
#include "XMessageIndexable.h"
#include "AbstractXFallRise.h"
```

Include dependency graph for XSurfaceData.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XSurfaceData

  The surface time-series data X-Message class.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.158.1 Detailed Description

The util::xmsg::XSurfaceData class header file.

Version

0.6

Date

Jun 16, 2014 11:31:04 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.159 XSurfaceData.h

Go to the documentation of this file.

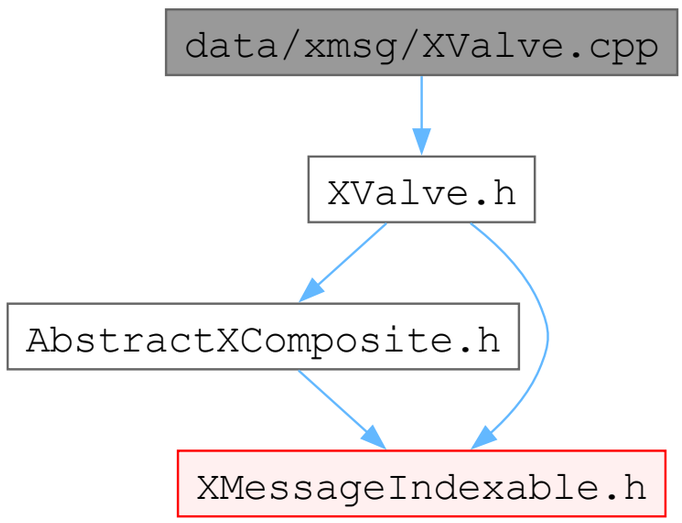
```
00001 /** The @ref util::xmsg::XSurfaceData class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jun 16, 2014 11:31:04 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef XSURFACEPRESSURE_H_
00024 #define XSURFACEPRESSURE_H_
00025
00026 /* Includes: */
00027
00028 // local
00029 #include "XMessageIndexable.h"
00030 #include "AbstractXFallRise.h"
00031
00032 namespace util {
00033 namespace xmsg {
00034
00035 /* Class Definition: */
00036
00037 /** The surface time-series data X-Message class.
00038  *
00039  *
00040  *
00041  */
00042 class XSurfaceData:
00043         public AbstractXFallRise {
00044
00045 public:
00046
00047     /* Member Functions: */
00048
00049     /** Parameterized constructor.
00050      *
00051      *  @param[in]      dive        The dive or cycle number of this message.
00052      *  @param[in]      whoi        The unique WHOI identifier.
00053      *  @param[in]      data        Binary surface pressure X-message data to decode.
00054      *
00055      *  @throw          util::log::MessageException if an decode error occurs.
00056      */
```

```
00057     XSurfaceData(int16_t dive, uint16_t whoi, const vector<uint8_t>& data) :
00058                     AbstractXFallRise(dive, whoi, data) {
00059
00060         // Decode the surface pressure message.
00061         DecodeXMsgData(data);
00062
00063     }
00064
00065     /** Destructor.
00066      *
00067      */
00068     ~XSurfaceData() {
00069
00070         /* Do nothing. */
00071
00072     }
00073
00074     /** Decode the surface pressure X-message.
00075      *
00076      *  @param[in]      data            Binary surface pressure X-message data to decode.
00077      *
00078      *  @throw          util::log::MessageException if an decode error occurs.
00079      */
00080     void DecodeXMsgData(const vector<uint8_t>& data) {
00081
00082         try {
00083
00084             // The the identifier.
00085             if((data.data()[0] & 0xF0) != (XMSG_TYPE_ID_PRES_SURF0 & 0xF0))
00086                 EXC_THROW("Invalid message type identifier.!");
00087
00088             // Unpack the message data.
00089             UnpackData(data);
00090
00091         }CATCH_BLOCK("Failed to decode surface data message!")
00092     }
00093
00094     /** Generate the listing entry string for this surface time-series data X-Message.
00095      *
00096      *  @param[in]  pg      The pressure sensor sensitivity scale factor, in LSB/dBar.
00097      *  @param[in]  po      The pressure sensor offset, in dBar.
00098      *  @param[in]  sg      The salinity sensor sensitivity scale factor, in LSB/PSU.
00099      *  @param[in]  so      The salinity sensor offset, in PSU.
00100      *  @param[in]  tg      The temperature sensor sensitivity scale factor, in LSB/degC.
00101      *  @param[in]  to      The temperature sensor offset, in degC.
00102      *  @param[in]  pc      The pressure correction, in dBar, to be subtracted computed pressure
00103      *                      value.
00104      *
00105      *  @throw      log::MessageException if an error occurs while generating the listing string.
00106      *
00107      *  @return     The listing entry string for this surface time-series data X-Message.
00108      */
00109     const string GenerateListingString(
00110             uint16_t pg,
00111             uint16_t po,
00112             uint16_t sg,
00113             uint16_t so,
00114             uint16_t tg,
00115             uint16_t to,
00116             float pc) const {
00117         ostringstream os;
00118         size_t sam = 0;
00119
00120         try {
00121             {
00122                 const format fmt("SFC %5d %4d %3d %4d %20s");
00123                 os « format(fmt) % id_instrument() % id_dive() % (int) xmsg_index() % num_samples()
00124                         % GenerateIso8601String(start_time());
00125             }
00126
00127             {
00128                 const format fmt("\nsfc %5d %4d %3d %4d ");
00129                 for_each(
00130                         sensor_data().begin(),
00131                         sensor_data().end(),
00132                         [&](const XSensorTripletCodeCounts& sensor_data) {
00133
00134                             os « format(fmt)
00135                                     % id_instrument() % id_dive() % (int) xmsg_index() % sam++
00136                                 « sensor_data.GenerateListingString(pg, po, sg, so, tg, to, pc);
00137
00138                         });
00139             }
00140         }CATCH_BLOCK("Failed to generate listing string for surface time-series data object!")
00141
00142         return os.str();
00143     }
00144 };
00145
00146 } /* namespace xmsg */
00147 } /* namespace util */
00148 #endif /* XSURFACEPRESSURE_H_ */
```

11.160 data/xmsg/XValve.cpp File Reference

The util::xmsg::XValve class source file.

```
#include "XValve.h"
```
Include dependency graph for XValve.cpp:



Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.160.1 Detailed Description

The util::xmsg::XValve class source file.

Version

0.6

Date

Mar 22, 2016 9:35:13 AM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

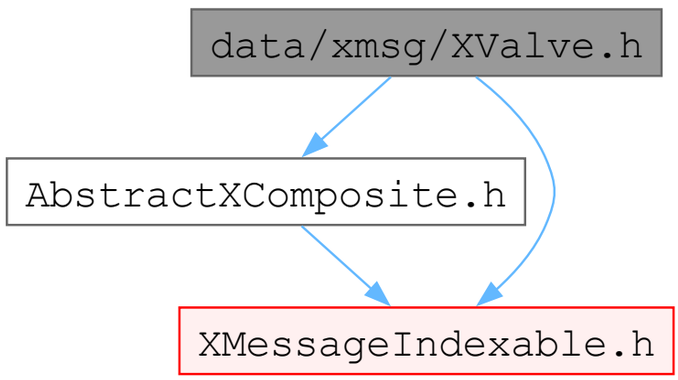
This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.161 data/xmsg/XValve.h File Reference

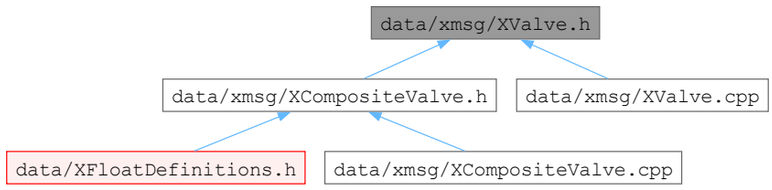
The util::xmsg::XValve class header file.

```
#include "AbstractXComposite.h"
#include "XMessageIndexable.h"
```

Include dependency graph for XValve.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XValve

  The valve data message class.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.161.1 Detailed Description

The util::xmsg::XValve class header file.

Version

0.6

Date

Mar 22, 2016 9:35:12 AM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.162 XValve.h

Go to the documentation of this file.

```
00001 /** The @ref util::xmsg::XValve class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Mar 22, 2016 9:35:12 AM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef XVALVE_H_
00024 #define XVALVE_H_
00025
00026 /* Includes: */
00027
00028 #include "AbstractXComposite.h"
00029 #include "XMessageIndexable.h"
00030
00031 /* Definitions: */
00032
00033 namespace util {
00034 namespace xmsg {
00035
00036 /* Types: */
00037
00038 /** The valve data message class.
00039  *
00040  *  An instance of this class represents a decoded valve data X-message, composed of the valve
00041  *  operation samples (see @ref XValveCounts) reported for a dive.
00042  */
00043 class XValve:
00044         public AbstractXComposite<XValveCounts>,
00045         public XMessageIndexable {
00046
00047 public:
00048
00049     /* Public Member Functions: */
00050
00051     /** Parameterized constructor.
00052      *
00053      *  @param[in]  dive    The dive or cycle number of this message.
00054      *  @param[in]  whoi    The unique WHOI instrument identifier.
00055      *  @param[in]  data    The binary X-message data to decode.
00056      *
00057      *  @throw      util::log::MessageException if the message cannot be decoded.
00058      */
00059     XValve(int16_t dive, uint16_t whoi, const vector<uint8_t>& data) :
00060                     XMessageIndexable(dive, whoi, data) {
00061
00062         // Decode the message data.
00063         DecodeXMsgData(data);
00064
00065
00066     }
00067
00068     ~XValve() {
00069
00070         /* Do nothing. */
00071
00072     }
00073
00074     /** Decode a valve data X-message (type 0xEB).
00075      *
00076      *  The low nibble of byte 3 is the message index. From byte 4, each valve operation is a
00077      *  7-byte sample: a phase code (high nibble of the first byte), a 20-bit pressure count (low
00078      *  nibble plus the next two bytes), the valve duration in seconds (2 bytes) and the case
```

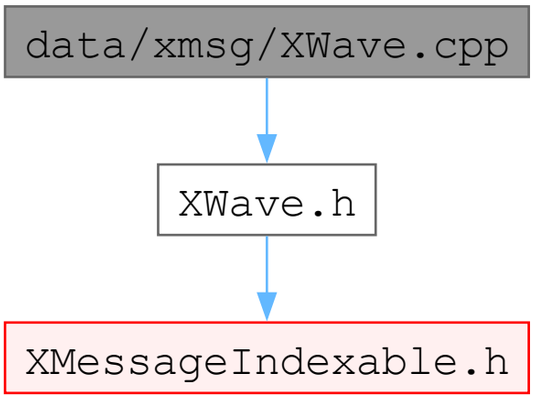
```
00079      *  vacuum after the operation (2 bytes). The case vacuum before the operation is not reported
00080      *  and is set to @c XMSG_BAD_U16. Each sample is stored as an @ref XValveCounts element.
00081      *
00082      *  @param[in]  data    The binary X-message data to decode.
00083      *
00084      *  @throw      util::log::MessageException if the message is shorter than its size field or
00085      *              the terminator is missing.
00086      */
00087     void DecodeXMsgData(const vector<uint8_t>& data) {
00088
00089         try {
00090
00091             const uint8_t* d = CheckXMsgData(data, 0xEB, 3, false);
00092
00093             //  TODO Get the message packing format.
00094
00095             // Check message size.
00096             if(xmsg_size_bytes() > data.size()) {
00097                 ostringstream os;
00098                 os « "Invalid XMessage size: message header specifies "
00099                    « xmsg_size_bytes()
00100                    « " bytes, but only "
00101                    « data.size()
00102                    « " bytes were received.";
00103                 BOOST_THROW_EXCEPTION(log::MessageException(os));
00104             } else if(data.size() > xmsg_size_bytes()) {
00105                 lwrn() « "Extra bytes in XMessage: message specifies "
00106                           « xmsg_size_bytes()
00107                           « "bytes, but "
00108                           « data.size()
00109                           « " bytes were received.";
00110             }
00111
00112             // Check for terminator.
00113             if(d[xmsg_size_bytes() - 1] != 0x3B) {
00114                 ostringstream os;
00115                 os « "Invalid XMessage: expected terminator ';' (0x3B) at byte "
00116                    « (xmsg_size_bytes() - 1)
00117                    « ", but received 0x"
00118                    « hex
00119                    « d[xmsg_size_bytes() - 1];
00120                 BOOST_THROW_EXCEPTION(log::MessageException(os));
00121             }
00122
00123             // Get the message index.
00124             set_xmsg_index(d[3] & 0x0F);
00125
00126             { // Iterate over the sensor data.
00127
00128                 int32_t presCounts;
00129                 int16_t valveSecs;
00130                 uint16_t vac0 = XMSG_BAD_U16;
00131                 uint16_t vac1;
00132                 XPhaseCode_t phaseCode;
00133                 for(vector<uint8_t>::const_iterator it = data.begin() + 4; it < data.end() - 7;) {
00134                     phaseCode = (XPhaseCode_t) (*it » 4);
00135                     presCounts = ((*it++ & 0x0F) « 16) | ((*it++) « 8) | (*it++);
00136                     valveSecs = ((*it++) « 8) | (*it++);
00137                     vac1 = ((*it++) « 8) | (*it++);
00138                     AppendElement(XValveCounts(phaseCode, presCounts, valveSecs, vac0, vac1));
00139                 }
00140             }
00141         }CATCH_BLOCK("Failed to decode pump data X-Message!")
00142     }
00143
00144     /** The the listing file comment string.
00145      *
00146      *  @return         The listing file comment string.
00147      */
00148     static const string GenerateListingComment() {
00149         ostringstream os;
00150
00151         try {
00152             const format fmt("#   %5s %4s %3s "
00153                     "%4s "
00154                     "%10s "
00155                     "%10s "
00156                     "%10s %10s\n"
00157                     "#   %5s %4s %3s "
00158                     "%4s  "
00159                     "%10s "
00160                     "%10s "
00161                     "%10s %10s");
00162             os « format(fmt) % "SN" % "dive" % "idx" //
00163                     % "nSam" //
00164                     % "pres" //
00165                     % "valveTime" //
00166                     % "vac0" % "vac1" //
00167                     % "" % "" % "" //
00168                     % "" //
00169                     % "(dBar)" //
00170                     % "(sec)" //
00171                     % "(inHg)" % "(inHg)";
00172         }CATCH_BLOCK("Failed to generate listing comment for pump data object!")
00173
00174         return os.str();
00175     }
00176
00177     const string GenerateListingString(uint16_t pGain, uint16_t pOff, float pres_correction) const {
00178         ostringstream os;
00179         size_t sam = 0;
00180
00181         try {
00182             {
00183                 const format fmt("VLV %5d %4d %3d %4d");
00184                 os « format(fmt) % id_instrument() % id_dive() % (int) xmsg_index()
00185                         % num_elements();
00186             }
```

```
00187             {
00188                 const format fmt("\nvlv %5d %4d %3d %4d ");
00189                 vector<XValveCounts>::const_iterator it;
00190                 for(it = elements().begin(); it < elements().end(); it++)
00191                     os « format(fmt) % id_instrument() % id_dive() % (int) xmsg_index() % sam++
00192                        « it->GenerateListingString(pGain, pOff, pres_correction);
00193             }
00194         }CATCH_BLOCK("Failed to generate listing string from pump data object!")
00195 
00196         return os.str();
00197     }
00198 
00199 };
00200 
00201 /* Functions: */
00202 
00203 } /* namespace xmsg */
00204 } /* namespace util */
00205 
00206 #endif /* XVALVE_H_ */
```

11.163 data/xmsg/XWave.cpp File Reference

The util::xmsg::XWave class source file.

`#include "XWave.h"`
Include dependency graph for XWave.cpp:



Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

11.163.1 Detailed Description

The util::xmsg::XWave class source file.

Version

0.6

Date

Jun 12, 2014 9:23:09 AM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

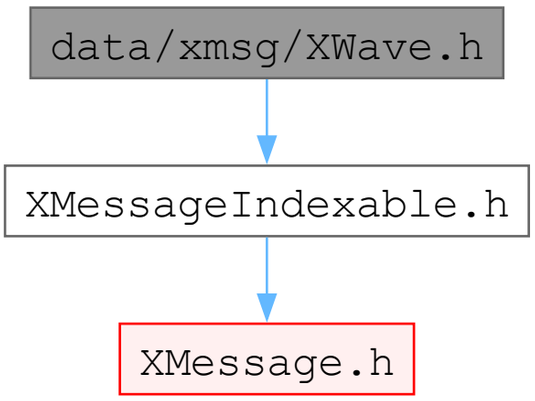
This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.164 data/xmsg/XWave.h File Reference

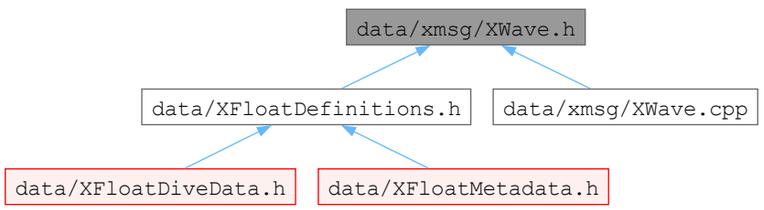
The util::xmsg::XWave class header file.

```
#include "XMessageIndexable.h"
```

Include dependency graph for XWave.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::xmsg::XWave

  Class representing measured ocean surface wave data X message.

Namespaces

- namespace util

  The utility namespace.
- namespace util::xmsg

  The X-Message utility namespace.

Macros

- #define XWAVE_INVALID_SF DBL_MAX

  The default/invalid sensitivity scale factor value for wave data.
- #define XWAVE_SIZE_ASSF_RAW (sizeof(util::xmsg::XWave::ASSF_RAW)/sizeof(double))

  The number of available accelerometer sensitivity scale factors for raw data.
- #define XWAVE_SIZE_ASSF_SPECTRAL (sizeof(util::xmsg::XWave::ASSF_SPECTRAL)/sizeof(double))

  The number of available accelerometer sensitivity scale factors for spectral.
- #define XWAVE_SIZE_FSAM (sizeof(util::xmsg::XWave::FSAM)/sizeof(double))

  The number of available sampling frequencies.
- #define XWAVE_SIZE_GSSF (sizeof(util::xmsg::XWave::GSSF)/sizeof(double))

  The number of available gyroscope sensitivity scale factors.
- #define XWAVE_SIZE_MSSF (sizeof(util::xmsg::XWave::MSSF)/sizeof(double))

  The number of available magnetometer sensitivity scale factors.

Enumerations

- enum util::xmsg::XWaveMode_t {
  util::xmsg::XWAVE_MODE_RAW = 0 ,
  util::xmsg::XWAVE_MODE_SPECTRAL = 1 ,
  util::xmsg::XWAVE_MODE_UNKNOWN = 99 }

  Wave data modes enumeration.
- enum **XWaveSpectralAxis_t** {
  XWAVE_SPECTRAL_AXIS_UNDEFINED = 0 ,
  XWAVE_SPECTRAL_AXIS_Z_PRIME = 1 ,
  XWAVE_SPECTRAL_AXIS_Y_PRIME = 2 ,
  XWAVE_SPECTRAL_AXIS_X_PRIME = 3 ,
  XWAVE_SPECTRAL_AXIS_Z = 4 ,
  XWAVE_SPECTRAL_AXIS_ZZ = 5 ,
  XWAVE_SPECTRAL_AXIS_ZX = 6 ,
  XWAVE_SPECTRAL_AXIS_ZY = 7 ,
  XWAVE_SPECTRAL_AXIS_X = 8 ,
  XWAVE_SPECTRAL_AXIS_XZ = 9 ,
  XWAVE_SPECTRAL_AXIS_XX = 10 ,
  XWAVE_SPECTRAL_AXIS_XY = 11 ,
  XWAVE_SPECTRAL_AXIS_Y = 12 ,
  XWAVE_SPECTRAL_AXIS_YZ = 13 ,
  XWAVE_SPECTRAL_AXIS_YX = 14 ,
  XWAVE_SPECTRAL_AXIS_YY = 15 ,
  XWAVE_NUM_SPECTRAL_AXES = 16 }
- enum **XWaveSpectralType_t** {
  XWAVE_SPECTRAL_TYPE_UNDEFINED = 0 ,
  XWAVE_SPECTRAL_TYPE_CO = 1 ,
  XWAVE_SPECTRAL_TYPE_QU = 2 ,
  XWAVE_SPECTRAL_TYPE_RE = 3 ,
  XWAVE_SPECTRAL_TYPE_IM = 4 ,
  XWAVE_SPECTRAL_TYPE_MAG = 5 ,
  XWAVE_SPECTRAL_TYPE_PHASE = 6 ,
  XWAVE_NUM_SPECTRAL_TYPES = 7 }

11.164.1 Detailed Description

The util::xmsg::XWave class header file.

Version

0.6

Date

Jun 12, 2014 9:23:09 AM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.164.2 Macro Definition Documentation

11.164.2.1 XWAVE_INVALID_SF

```
#define XWAVE_INVALID_SF DBL_MAX
```

The default/invalid sensitivity scale factor value for wave data.

11.164.2.2 XWAVE_SIZE_ASSF_RAW

```
#define XWAVE_SIZE_ASSF_RAW (sizeof(util::xmsg::XWave::ASSF_RAW)/sizeof(double))
```

The number of available accelerometer sensitivity scale factors for raw data.

11.164.2.3 XWAVE_SIZE_ASSF_SPECTRAL

```
#define XWAVE_SIZE_ASSF_SPECTRAL (sizeof(util::xmsg::XWave::ASSF_SPECTRAL)/sizeof(double))
```

The number of available accelerometer sensitivity scale factors for spectral.

11.164.2.4 XWAVE_SIZE_FSAM

```
#define XWAVE_SIZE_FSAM (sizeof(util::xmsg::XWave::FSAM)/sizeof(double))
```

The number of available sampling frequencies.

11.164.2.5 XWAVE_SIZE_GSSF

```
#define XWAVE_SIZE_GSSF (sizeof(util::xmsg::XWave::GSSF)/sizeof(double))
```

The number of available gyroscope sensitivity scale factors.

11.164.2.6 XWAVE_SIZE_MSSF

```
#define XWAVE_SIZE_MSSF (sizeof(util::xmsg::XWave::MSSF)/sizeof(double))
```

The number of available magnetometer sensitivity scale factors.

11.165 XWave.h

Go to the documentation of this file.

```
00001 /** The @ref util::xmsg::XWave class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jun 12, 2014 9:23:09 AM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef XMOTION_H_
00024 #define XMOTION_H_
00025
00026 /* Includes */
00027
00028 // local
00029 #include "XMessageIndexable.h"
00030
00031 /* Definitions */
00032
00033 /** The default/invalid sensitivity scale
00034 factor value for wave data.
00035  *
00036  */
00037 #define XWAVE_INVALID_SF DBL_MAX
00038
00039 /** The number of available accelerometer sensitivity scale factors for raw data. */
00040 #define XWAVE_SIZE_ASSF_RAW       (sizeof(util::xmsg::XWave::ASSF_RAW)/sizeof(double))
00041
00042 /** The number of available accelerometer sensitivity scale factors for spectral. */
00043 #define XWAVE_SIZE_ASSF_SPECTRAL  (sizeof(util::xmsg::XWave::ASSF_SPECTRAL)/sizeof(double))
00044
00045 /** The number of available sampling frequencies. */
00046 #define XWAVE_SIZE_FSAM           (sizeof(util::xmsg::XWave::FSAM)/sizeof(double))
00047
00048 /** The number of available gyroscope sensitivity scale factors. */
00049 #define XWAVE_SIZE_GSSF           (sizeof(util::xmsg::XWave::GSSF)/sizeof(double))
00050
00051 /** The number of available magnetometer sensitivity scale factors. */
00052 #define XWAVE_SIZE_MSSF           (sizeof(util::xmsg::XWave::MSSF)/sizeof(double))
00053
00054 namespace util {
00055 namespace xmsg {
00056
00057 /* Type Definitions */
00058
00059 /** Wave data modes enumeration.
00060  *
00061  *  @note           Currently, only raw wave motion measurements are supported.
00062  *
00063  */
00064 typedef enum XWaveMode {
00065
00066     /** Raw data mode. */
00067     XWAVE_MODE_RAW = 0,
00068
00069     /** Spectral data mode. */
00070     XWAVE_MODE_SPECTRAL = 1,
00071
00072     /** Unknown/invalid data mode. */
00073     XWAVE_MODE_UNKNOWN = 99
00074
00075 } XWaveMode_t;
00076
00077 typedef enum XWaveSpectralAxis {
00078
00079   XWAVE_SPECTRAL_AXIS_UNDEFINED = 0,
00080   XWAVE_SPECTRAL_AXIS_Z_PRIME   = 1,
00081   XWAVE_SPECTRAL_AXIS_Y_PRIME   = 2,
00082   XWAVE_SPECTRAL_AXIS_X_PRIME   = 3,
00083   XWAVE_SPECTRAL_AXIS_Z         = 4,
00084   XWAVE_SPECTRAL_AXIS_ZZ        = 5,
00085   XWAVE_SPECTRAL_AXIS_ZX        = 6,
00086   XWAVE_SPECTRAL_AXIS_ZY        = 7,
00087   XWAVE_SPECTRAL_AXIS_X         = 8,
00088   XWAVE_SPECTRAL_AXIS_XZ        = 9,
00089   XWAVE_SPECTRAL_AXIS_XX        = 10,
00090   XWAVE_SPECTRAL_AXIS_XY        = 11,
00091   XWAVE_SPECTRAL_AXIS_Y         = 12,
00092   XWAVE_SPECTRAL_AXIS_YZ        = 13,
00093   XWAVE_SPECTRAL_AXIS_YX        = 14,
00094   XWAVE_SPECTRAL_AXIS_YY        = 15,
00095   XWAVE_NUM_SPECTRAL_AXES       = 16,
00096
00097 } XWaveSpectralAxis_t;
00098
00099 typedef enum XWaveSpectralType {
00100
00101   XWAVE_SPECTRAL_TYPE_UNDEFINED = 0,
00102   XWAVE_SPECTRAL_TYPE_CO        = 1,
00103   XWAVE_SPECTRAL_TYPE_QU        = 2,
```

```
00104   XWAVE_SPECTRAL_TYPE_RE        = 3,
00105   XWAVE_SPECTRAL_TYPE_IM        = 4,
00106   XWAVE_SPECTRAL_TYPE_MAG       = 5,
00107   XWAVE_SPECTRAL_TYPE_PHASE     = 6,
00108   XWAVE_NUM_SPECTRAL_TYPES      = 7,
00109
00110 } XWaveSpectralType_t;
00111
00112 /** Class representing measured ocean surface wave data X message.
00113  *
00114  */
00115 class XWave: public XMessageIndexable {
00116
00117 private:
00118
00119     /* Data Members */
00120
00121     /** Available accelerometer sensitivity scale factors for raw wave data. */
00122     static const unsigned int ASSF_RAW[];
00123
00124     /** Available accelerometer sensitivity scale factors for spectral wave data -- message version 1. */
00125     static const unsigned int ASSF_SPECTRAL_V1[];
00126
00127     /** Available accelerometer sensitivity scale factors for spectral wave data -- message version 2. */
00128     static const unsigned int ASSF_SPECTRAL_V2[];
00129
00130     /** Available ample rate frequencies. */
00131     static const double FSAM[];
00132
00133     /** Available gyrscope sensitivity scale factors. */
00134     static const double GSSF[];
00135
00136     /** Available magenetometer sensitivity scale factors. */
00137     static const double MSSF[];
00138
00139 private:
00140
00141     /* Data Members */
00142
00143     /** The accelerometer sensitivity scale factor, in LSB/g.
00144      *
00145      *  @note           Dividing accelerometer counts by this number produces this measured
00146      *                  acceleration, in g.
00147      */
00148     unsigned int assf_;
00149
00150     /** Accelerometer software gain.
00151      *
00152      */
00153     uint16_t gain_accel_q3_;
00154
00155     /** The ocean surface wave field sampling mode.
00156      *
00157      *  This field indicates whether this message contains raw wave motion measurements or
00158      *  spectrally encoded data.
00159      */
00160     XWaveMode_t sample_mode_;
00161
00162     /** The base sample rate, in Hz.
00163      *
00164      */
00165     uint16_t sample_rate_base_;
00166
00167     /** The sample rate divider, in Hz/Hz.
00168      *
00169      */
00170     uint16_t sample_rate_div_;
00171
00172     /** The sample rate of the ocean surface wave measurements, in Hz.
00173      *
00174      */
00175     float sample_rate_effective_;
00176
00177     /** The power-of-two segment length for spectral data.
00178      *
00179      */
00180     uint16_t segment_len_;
00181
00182     /** The power-of-two segment length for spectral data.
00183      *
00184      */
00185     uint16_t segment_len_pow_;
00186
00187     /**
00188      *
00189      */
00190     uint8_t version_;
00191
00192     /** The spectral data samples.
00193      *
00194      */
00195     vector<XWaveSpectralSample> wave_data_spectral_;
00196
00197 public:
00198     /* Member Functions */
00199
00200     /** The parameterized constructor.
00201      *
00202      *  @param[in]      dive        The dive or cycle number of this message.
00203      *  @param[in]      whoi        The unique WHOI identifier.
00204      *  @param[in]      data        Binary surface pressure X-message data to decode.
00205      *
00206      *  @throw          util::log::MessageException if an decode error occurs.
00207      */
00208     XWave(int16_t dive, uint16_t whoi, const vector<uint8_t>& data) :
00209                     XMessageIndexable(dive, whoi, data),
00210                     assf_(0),
00211                     gain_accel_q3_(8),
```

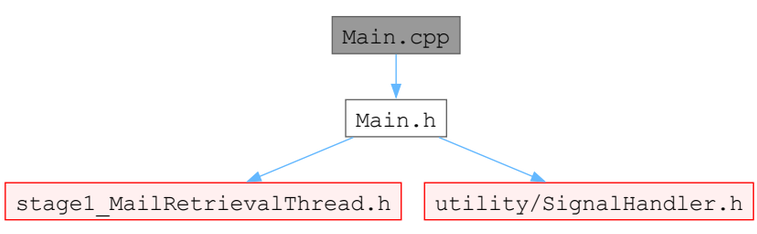
```
00212                     sample_mode_(XWAVE_MODE_UNKNOWN),
00213                     sample_rate_base_(0),
00214                     sample_rate_div_(0),
00215                     sample_rate_effective_(0),
00216                     segment_len_(0),
00217                     version_(0xFF),
00218                     wave_data_spectral_() {
00219         DecodeXMsgData(data);    // Decode the wave data message.
00220     }
00221
00222     /** The destructor.
00223      *
00224      */
00225     ~XWave() {
00226         /* Do noting. */
00227     }
00228
00229     void DecodeXMsgData(const vector<uint8_t>& data);
00230
00231     const string GenerateListingString() const;
00232
00233     unsigned int effective_ssf() const {
00234       return ( assf_ * gain_accel_q3_ * gain_accel_q3_ ) >> 6;
00235     }
00236
00237     /** Fix MRV's 9000th data encoding fuckup.
00238      *
00239      *  For select firmware versions, only every other spectral value is returned. The seconds half
00240      *  of the spectra is all zeros. Here we fix this by doubling the effective sample rate
00241      *
00242      */
00243     void FixFuckup9000() {
00244       sample_rate_effective_ = 2.0 * (float)sample_rate_base_ / (sample_rate_div_ + 1);
00245
00246       for(unsigned int i = 0; i < (unsigned int)(segment_len() >> 1); ++i) {
00247
00248         wave_data_spectral_.at(i).freq_
00249             = sample_rate_effective_*(i+1)/segment_len();
00250       }
00251     }
00252
00253 public:
00254
00255     /* Accessor Methods: */
00256
00257     float gain_accel() const {
00258       return (float) gain_accel_q3_ / 8;
00259     }
00260
00261     uint16_t segment_len() const {
00262       return segment_len_;
00263     }
00264
00265     uint16_t segment_len_pow() const {
00266       return segment_len_pow_;
00267     }
00268
00269 public:
00270
00271     /* Mutator Methods: */
00272
00273     void set_segment_len(uint16_t segment_len) {
00274       segment_len_ = segment_len;
00275     }
00276
00277     void set_segment_len_pow(uint16_t segment_len_pow) {
00278       set_segment_len(1 << (segment_len_pow_ = segment_len_pow));
00279     }
00280 };
00281
00282 } /* namespace xmsg */
00283 } /* namespace util */
00284
00285 #endif /* XMOTION_H_ */
```

11.166 Main.cpp File Reference

The main() function source file.

```
#include "Main.h"
```

Include dependency graph for Main.cpp:



Namespaces

- namespace util

  The utility namespace.
- namespace util::config

  The configuration utility namespace.
- namespace util::sig

  The system signals utility namespace.

Functions

- const util::config::LocalConfiguration & Initialize (int argc, char **argv)

  Initialize the application.
- int main (int argc, char **argv)

  The main program (daemon thread) entry point.
- void StartWorkerThreads () ATTR_NO_EXCEPTIONS

  Start worker threads.
- void TerminateWorkerThreads () ATTR_NO_EXCEPTIONS

  Terminate the worker threads.

11.166.1 Detailed Description

The main() function source file.

This file contains the main program entry point, main(), which defines the functional operation of the daemon thread. This thread is responsible for for performing the application initialization, instantiating and terminating the worker threads, maintaining the run-time lock, and finally invoking the exit handler.

Version

0.6

Date

May 12, 2014 4:42:20 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.166.2 Function Documentation

11.166.2.1 Initialize()

```
const util::config::LocalConfiguration & Initialize (
            int argc,
            char ** argv)
```

Initialize the application.

All startup initialization of the application is performed by this function. First, the local configuration initialization is invoked, which parses the configuration file and/or the command-line arguments and also acquires a run-time lock. Then the global logger is initialized, setting the sources, sinks, formats, and filters of the global logger. Next, the signal handlers are registered, defining the behavior of the application if a catchable signal is encountered from the system. Finally, requisite data and metadata is loaded from files on the local file system and from the SQL database.

Parameters

in	*argc*	The number of command-line arguments.
in	*argv*	The command-line argument list.

Exceptions

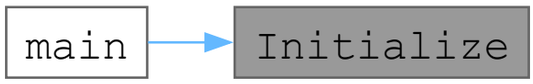
None.	Never throws.

Returns

The local configuration object.

Referenced by main().

Here is the caller graph for this function:



11.166.2.2 main()

```
int main (
            int argc,
            char ** argv)
```

The main program (daemon thread) entry point.

The operation of the daemon thread of the application is completely contained within this function. First, the program is initialized by calling Initialize() which parses the configuration options, acquires the run-time lock, sets up the global logger object, registers the signal handlers, and finally loads requisite data from both files on the local filesystem and the database. Then, the runtime lock will be checked and the thread will sleep for XFLT_THREAD_SLEEP_MS milliseconds. If the application is configured to run as a daemon, then the last step will be repeated indefinitely until a termination signal is received. Finally, the exit handler, util::sig::SignalHandler::ExitHandler(), is invoked to terminate the worker threads, save the locally cached data to the database, release the run-time lock, and flush the global logger.

Note

If, at any time during execution after initialization, a catchable signal is received, the execution of the daemon thread will be interrupted and the exit handler function util::sig::SignalHandler::ExitHandler() will be invoked. The exit handler terminates the worker threads, saves the locally cached data to the database, releases the run-time lock, and flushes the global logger.

Parameters

in	*argc*	The number of command-line arguments.
in	*argv*	The command-line argument list.

Exceptions

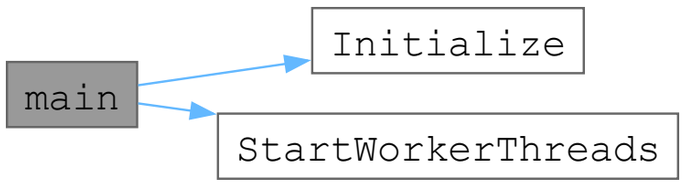
util::log::MessageException	if application initialization fails.

Returns

EXIT_SUCCESS on successful completion or non-zero value on failure.

References CATCH_NOTHROW, Initialize(), StartWorkerThreads(), XFLT_DAEMON_FLUSH_LOG_SEC, and XFLT_THREAD_SLEEP_MS.

Here is the call graph for this function:



11.166.2.3 StartWorkerThreads()

```
void StartWorkerThreads ()
```

Start worker threads.

A call to this function will instantiate all worker threads that comprise the pipeline of the application: (1) the mail retrieval thread, (2) the mail processing thread(s), and (3) the telemetry decoder threads.

Exceptions

None.	Never throws.

References CATCH_BLOCK, linf, and lout.

Referenced by main().

Here is the caller graph for this function:

11.166.2.4 TerminateWorkerThreads()

```
void TerminateWorkerThreads ()
```

Terminate the worker threads.

A call to this function will perform and orderly shutdown of the the worker threads. The threads will be shutdown in order of the pipeline of the application. Each thread or thread group will be allowed to complete its current task(s) before the shutdown occurs, and before shutting down the next stage of the threads in the pipeline. This way, all the data that is in the pipeline when the shutdown is invoked will be processes before the application terminates.

Warning

This function will block until all threads are shut down. If any of the worker threads hang, then this function may block indefinitely.

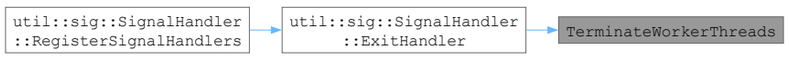
Exceptions

None.	Never throws.

References CATCH_BLOCK, linf, and lout.

Referenced by util::sig::SignalHandler::ExitHandler().

Here is the caller graph for this function:

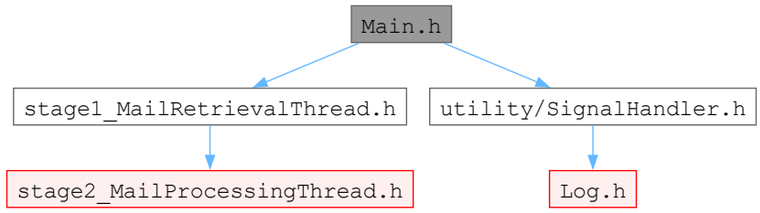


11.167 Main.h File Reference

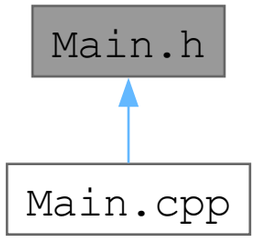
The main() function header file.

```
#include "stage1_MailRetrievalThread.h"
#include "utility/SignalHandler.h"
```

Include dependency graph for Main.h:



This graph shows which files directly or indirectly include this file:



Functions

- static const util::config::LocalConfiguration & Initialize (int argc, char **argv)

  Initialize the application.
- static void StartWorkerThreads () ATTR_NO_EXCEPTIONS

  Start worker threads.
- static void TerminateWorkerThreads () ATTR_NO_EXCEPTIONS

  Terminate the worker threads.

11.167.1 Detailed Description

The main() function header file.

This file contains includes and function prototypes that are needed for the main() source file.

Version

0.6

Date

May 12, 2014 4:42:32 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.167.2 Function Documentation

11.167.2.1 Initialize()

```
static const util::config::LocalConfiguration & Initialize (
            int argc,
            char ** argv)  [static]
```

Initialize the application.

All startup initialization of the application is performed by this function. First, the local configuration initialization is invoked, which parses the configuration file and/or the command-line arguments and also acquires a run-time lock. Then the global logger is initialized, setting the sources, sinks, formats, and filters of the global logger. Next, the signal handlers are registered, defining the behavior of the application if a catchable signal is encountered from the system. Finally, requisite data and metadata is loaded from files on the local file system and from the SQL database.

Parameters

in	*argc*	The number of command-line arguments.
in	*argv*	The command-line argument list.

Exceptions

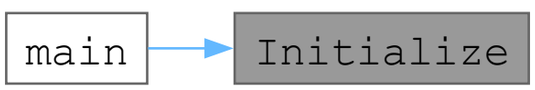
None.	Never throws.

Returns

The local configuration object.

Referenced by main().

Here is the caller graph for this function:



11.167.2.2 StartWorkerThreads()

```
static void StartWorkerThreads ()  [static]
```

Start worker threads.

A call to this function will instantiate all worker threads that comprise the pipeline of the application: (1) the mail retrieval thread, (2) the mail processing thread(s), and (3) the telemetry decoder threads.

Exceptions

None.	Never throws.

References CATCH_BLOCK, linf, and lout.

Referenced by main().

Here is the caller graph for this function:

11.167.2.3 TerminateWorkerThreads()

```
static void TerminateWorkerThreads ()  [static]
```

Terminate the worker threads.

A call to this function will perform and orderly shutdown of the the worker threads. The threads will be shutdown in order of the pipeline of the application. Each thread or thread group will be allowed to complete its current task(s) before the shutdown occurs, and before shutting down the next stage of the threads in the pipeline. This way, all the data that is in the pipeline when the shutdown is invoked will be processes before the application terminates.

Warning

This function will block until all threads are shut down. If any of the worker threads hang, then this function may block indefinitely.

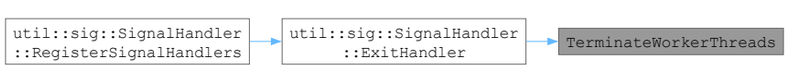
Exceptions

None.	Never throws.

References CATCH_BLOCK, linf, and lout.

Referenced by util::sig::SignalHandler::ExitHandler().

Here is the caller graph for this function:



11.168 Main.h

Go to the documentation of this file.

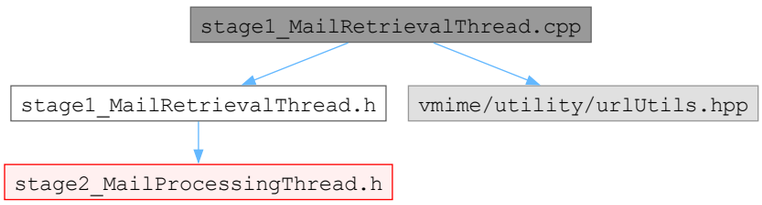
```
00001 /** The @ref main() function header file.
00002  *
00003  *  This file contains includes and function prototypes that are needed for the main() source file.
00004  *
00005  *  @file
00006  *
00007  *  @version    0.6
00008  *
00009  *  @date       May 12, 2014 4:42:32 PM
00010  *
00011  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00012  *              Engineer I,
00013  *              Department of Physical Oceanography,
00014  *              Woods Hole Oceanographic Institution
00015  *
00016  *  @copyright  2014 Woods Hole Oceanographic Institution.
00017  *              Released under the GNU General Public License.
00018  *
00019  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00020  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00021  *  General Public License for more details. You should have received a copy of the GNU General
00022  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00023  */
00024
00025 /* Includes: */
00026
00027 // local
00028 #include "stage1_MailRetrievalThread.h"
00029 #include "utility/SignalHandler.h"
00030
00031 namespace cfg = util::config;
00032
00033 /* Function Prototypes: */
00034
00035 static const util::config::LocalConfiguration& Initialize(int argc, char** argv) ATTR_NO_EXCEPTIONS;
00036
00037 static void StartWorkerThreads() ATTR_NO_EXCEPTIONS;
00038
00039 static void TerminateWorkerThreads() ATTR_NO_EXCEPTIONS;
00040
```

11.169 stage1_MailRetrievalThread.cpp File Reference

The argo::iridium::xfloat::XFloatMailRetrievalService class source file.

```
#include "stage1_MailRetrievalThread.h"
#include <vmime/utility/urlUtils.hpp>
```

Include dependency graph for stage1_MailRetrievalThread.cpp:



Namespaces

- namespace argo

  The Argo instrument namespace.
- namespace argo::iridium

  The Argo Iridium instrument namespace.
- namespace argo::iridium::xfloat

  The X-Float instrument namespace.

11.169.1 Detailed Description

The argo::iridium::xfloat::XFloatMailRetrievalService class source file.

This is the source file for the argo::iridium::xfloat::XFloatMailRetrievalService class, which defines the operation of the mail retrieval thread. This thread is the first stage in the processing pipeline and is is responsible for fetching email messages for the mail server and/or SQL database, and queuing them for consumption by the next stage in the processing pipeline, argo::iridium::xfloat::XFloatMailProcessingService.

The file contains the static member variables declared in the argo::iridium::xfloat::XFloatMailRetrievalService class. These include credentials and details needed for connecting to the mail server, which are set as a result of parsing the command-line and/or configuration file options during initialization.

Version

0.6

Date

Oct 16, 2014 2:21:06 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

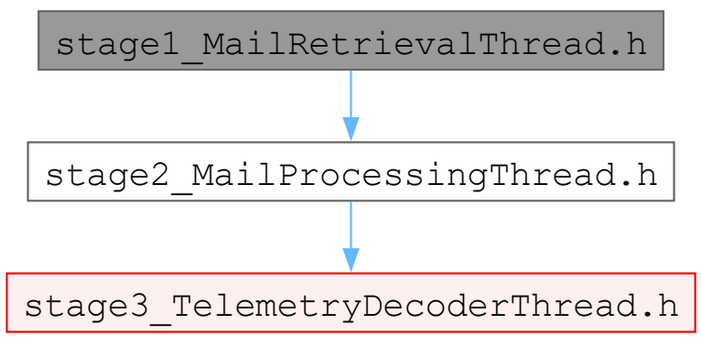
This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.170 stage1_MailRetrievalThread.h File Reference

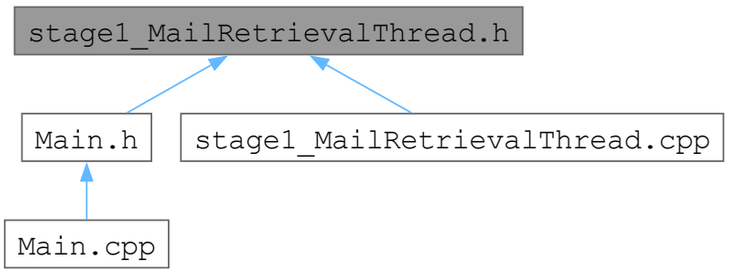
The argo::iridium::xfloat::XFloatMailRetrievalService class header file.

```
#include "stage2_MailProcessingThread.h"
```

Include dependency graph for stage1_MailRetrievalThread.h:



This graph shows which files directly or indirectly include this file:



Classes

- class argo::iridium::xfloat::XFloatMailRetrievalService
  The mail retrieval thread class.

Namespaces

- namespace argo
  The Argo instrument namespace.
- namespace argo::iridium
  The Argo Iridium instrument namespace.
- namespace argo::iridium::xfloat
  The X-Float instrument namespace.

Enumerations

- enum **XFloatMailFetchMode_t** {
  MAIL_FETCH_SERVER = 0 ,
  MAIL_FETCH_FILESYSTEM = 1 }

11.170.1 Detailed Description

The argo::iridium::xfloat::XFloatMailRetrievalService class header file.

This is the header file for the argo::iridium::xfloat::XFloatMailRetrievalService class, which defines the operation of the mail retrieval thread. This thread is the first stage in the processing pipeline and is is responsible for fetching email messages for the mail server and/or SQL database, and queuing them for consumption by the next stage in the processing pipeline, argo::iridium::xfloat::XFloatMailProcessingService.

Version

0.6

Date

Oct 16, 2014 2:21:06 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.171 stage1_MailRetrievalThread.h

Go to the documentation of this file.

```
00001 /** The @ref argo::iridium::xfloat::XFloatMailRetrievalService class header file.
00002  *
00003  *  This is the header file for the @ref argo::iridium::xfloat::XFloatMailRetrievalService
00004  *  class, which defines the operation of the mail retrieval thread. This thread is the first
00005  *  stage in the  processing pipeline and is is responsible for fetching email messages for the
00006  *  mail server and/or SQL database, and queuing them for consumption by the next stage in the
00007  *  processing pipeline, @ref argo::iridium::xfloat::XFloatMailProcessingService.
00008  *
00009  *  @file
00010  *
00011  *  @version    0.6
00012  *
00013  *  @date       Oct 16, 2014 2:21:06 PM
00014  *
00015  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00016  *              Engineer I,
00017  *              Department of Physical Oceanography,
00018  *              Woods Hole Oceanographic Institution
00019  *
00020  *  @copyright  2014 Woods Hole Oceanographic Institution.
00021  *              Released under the GNU General Public License.
00022  *
00023  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00024  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00025  *  General Public License for more details. You should have received a copy of the GNU General
00026  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00027  */
00028
00029 #ifndef XFLOATMAILRETRIEVALSERVICE_H_
00030 #define XFLOATMAILRETRIEVALSERVICE_H_
00031
00032 /* Includes: */
00033
00034 // local
00035 #include "stage2_MailProcessingThread.h"
00036
00037 namespace argo {
00038 namespace iridium {
00039 namespace xfloat {
00040
00041 /* Class Definition: */
00042
00043
00044
00045
00046 typedef enum MailFetchMode {
00047   MAIL_FETCH_SERVER = 0,
00048   MAIL_FETCH_FILESYSTEM = 1,
00049 } XFloatMailFetchMode_t;
00050
00051 /** The mail retrieval thread class.
00052  *
00053  *  This class defines the operation of the mail retrieval thread, which is responsible for
00054  *  fetching email messages for the mail server and/or SQL database, and queuing them for
00055  *  consumption by the next stage in the processing pipeline, @ref XFloatMailProcessingService.
00056  *  The mail retrieval thread is the first stage in the processing pipeline.
00057  *
00058  *  A single instance of this thread will be instantiated after application initialization. The
00059  *  function of the thread is defined by functor operator overload function. The thread enters
00060  *  a primary loop in which mail messages are queried from the database and mail server. If no
00061  *  messages are returned, the thread disconnects from the mail server and database, and sleeps for
00062  *  a period of time before resuming the loop. The loop continues until termination is signaled
00063  *  by the daemon thread, at which point the thread will disconnect from the mail server and die.
00064  *
00065  *  All the credentials for connecting to the mail server are stored as static members of this
00066  *  class. These values are set as a result of parsing the command-line and/or configuration
00067  *  options during application initialization.
00068  */
00069 class XFloatMailRetrievalService:
00070     public util::file::FileSystemUtils,
00071     public util::mysql::LocalMysqlConfiguration {
00072
00073     /* Friend Functions: */
00074
00075     /** Friend function for populating some of the static members of this class from the
00076      *  configuration file options.
00077      */
00078     friend options_description
      util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription();
00079
00080     /** Friend function for populating static members of this class from the command-line options.
00081      *
00082      */
00083     friend options_description util::config::LocalConfiguration::GenerateCommandLineOptionsDescription();
00084
00085   private:
00086
00087     /* Nested Classes: */
00088
00089 #   if defined(XMTPS_DEBUG_FETCH_MAIL) && XMTPS_DEBUG_FETCH_MAIL
00090     /** The MailTracer class.
00091      *
00092      *  This class is used to trace server-client communications between the mailserver and the
00093      *  application.
00094      *
00095      */
00096     class MailTracer:
00097     public vmime::net::tracer {
00098
00099       private:
00100
00101       /* Private Data Members: */
00102
```

```
00103       const vmime::string m_proto;
00104       const int m_connectionId;
00105
00106       public:
00107
00108       /* Public Member Functions: */
00109
00110       MailTracer(const vmime::string& proto, const int connectionId) :
00111           m_proto(proto),
00112           m_connectionId(connectionId) {
00113         /* Do nothing. */
00114       }
00115
00116       // Called by VMime to trace what is sent on the socket
00117       void traceSend(const vmime::string& line) {
00118
00119         ldbg() « "[tracer]"
00120         « "["
00121         « m_proto
00122         « ":"
00123         « m_connectionId
00124         « "] C: "
00125         « line
00126         « std::endl;
00127
00128       }
00129
00130       // Called by VMime to trace what is received from the socket
00131       void traceReceive(const vmime::string& line) {
00132
00133         ldbg() « "[tracer]"
00134         « "["
00135         « m_proto
00136         « ":"
00137         « m_connectionId
00138         « "] S: "
00139         « line
00140         « std::endl;
00141
00142       }
00143
00144     };
00145 #   endif // defined(XMTPS_DEBUG_FETCH_MAIL) && XMTPS_DEBUG_FETCH_MAIL
00146
00147 #   if defined(XMTPS_DEBUG_FETCH_MAIL) && XMTPS_DEBUG_FETCH_MAIL
00148     class MailTracerFactory:
00149     public vmime::net::tracerFactory {
00150
00151       public:
00152
00153       /* Public Member Functions: */
00154
00155       MailTracerFactory() {
00156
00157         /* Do nothing. */
00158
00159       }
00160
00161       vmime::shared_ptr<vmime::net::tracer> create(
00162           vmime::shared_ptr<vmime::net::service> serv,
00163           const int connectionId) {
00164
00165         return vmime::make_shared<MailTracer>(serv->getProtocolName(), connectionId);
00166
00167       }
00168
00169     };
00170 #   endif // defined(XMTPS_DEBUG_FETCH_MAIL) && XMTPS_DEBUG_FETCH_MAIL
00171
00172   private:
00173
00174     /* Static Data Members: */
00175
00176     static bool is_reprocess_active_;
00177
00178
00179     /** The reprocess-all-mail-messages boolean flag.
00180      *
00181      *  The value of this field is set as a result of parsing the command-line options. If
00182      *  unspecified in the command-line arguments, this field defaults to a the value
00183      *  @ref CFG_DEFAULT_REPROCESS_ALL.
00184      *
00185      *  The value of this boolean flag determines which messages are retrieved from the mail
00186      *  server upon startup of the application. If TRUE, then mail messages will be fetched from
00187      *  mail server starting with UID 0. Otherwise, if FALSE, the starting UID of mail messages
00188      *  to be fetched from the server will be determined by querying the maximum UID of the mail
00189      *  messages stored in the SQL database. Mail message will be fetched from the mail server,
00190      *  starting with the UID queried from the database plus one.
00191      */
00192     static bool is_reprocess_all_;
00193
00194     /** The mail fetch mode flag.
00195      *
00196      *  - 0: Fetch mail from mail server.
00197      *  - 1: Fetch mail from file system.
00198      *
00199      */
00200     static int mail_fetch_mode_;
00201
00202     static path mail_file_input_directory_;
00203
00204     static path mail_file_processed_directory_;
00205
00206     /** The mail server host qualified name or IP address string.
00207      *
00208      *  The value of this field is set as a result of parsing the command-line options. If
00209      *  unspecified, it defaults to "maila.whoi.edu".
00210      */
```

```
00211     static string mail_server_host_;
00212
00213     /** The mail server remote incoming mail message folder path string.
00214      *
00215      *  The value of this field is set as a result of parsing the command-line options. If
00216      *  unspecified, it defaults to "INBOX".
00217      */
00218     static string mail_server_inbox_folder_path_;
00219
00220     /** The mail server account password string.
00221      *
00222      *  The value of this field is set as a result of parsing the command-line options.
00223      */
00224     static string mail_server_password_;
00225
00226     /** The mail server protocol name string.
00227      *
00228      *  The value of this field is set as a result of parsing the command-line options. If
00229      *  unspecified, it defaults to "imap".
00230      */
00231     static string mail_server_protocol_;
00232
00233     /** The mail server account username string.
00234      *
00235      *  The value of this field is set as a result of parsing the command-line options. If
00236      *  unspecified, it defaults to "imap".
00237      */
00238     static string mail_server_user_;
00239
00240     /** The maximum number of messages to simultaneously fetch from the mail server.
00241      *
00242      *  The value of this field is set as a result of parsing the command-line options. If
00243      *  unspecified, it defaults to @ref CFG_DEFAULT_MAX_FETCH.
00244      */
00245     static size_t max_num_msgs_to_fetch_;
00246
00247     /** The starting index of the sequence of messages to fetch.
00248      *
00249      *  If messages are moved out of the INBOX folder after being processed, then the messages
00250      *  sequence start number will always be one. Otherwise, this field will contain the starting
00251      *  index of the first message not fetched from the mail server.
00252      *
00253      */
00254     static unsigned int message_start_index_;
00255
00256     /** Move mail messages flag.
00257      *
00258      *  If set, then the mail messages will be moved out of the inbox folder after being
00259      *  processed.
00260      *
00261      */
00262     static bool move_mail_messages_;
00263
00264     /** A vector containing the unique instrument identifier of messages to reprocess from the SQL
00265      *  database.
00266      *
00267      *  If the user specifies one or more instruments to reprocess by specifying their unique
00268      *  instrument identifiers, then those identifiers will be contained in this vector. Otherwise,
00269      *  this vector will be empty.
00270      *
00271      *  After the thread is instantiated, it will check the size of this vector. If the vector size
00272      *  is non-zero, then the thread will query the SQL database for messages with instrument
00273      *  identifiers matching those contained in this vector. Those messages will then be
00274      *  passed on to the mail processing thread.
00275      *
00276      *  Regardless of the size of this vector, the mail retrieval thread will then continue on to
00277      *  enter its primary loop until termination is signaled.
00278      *
00279      */
00280     static vector<unsigned int> reprocess_by_inst_ids_;
00281
00282     /** The time to sleep if no new mail messages are fetched, in seconds.
00283      *
00284      *  If no new mail messages are fetched, the mail retrieval thread will sleep for this period
00285      *  of time, in increments of @ref XFLT_THREAD_SLEEP_MS.
00286      *
00287      *  The value of this field is set as a result of parsing the command-line options. If
00288      *  unspecified, it defaults to @ref CFG_DEFAULT_MAIL_CHK_DLY.
00289      */
00290     static unsigned int sleep_seconds_if_no_new_msgs_;
00291
00292     /** The termination signaled flag.
00293      *
00294      *  This flag is used  to signal termination of the mail retrieval thread. During normal
00295      *  operation, the value of this field will be FALSE. Once termination is signaled, it will
00296      *  become TRUE causing all of the single instance of the mail retrieval to to exit its
00297      *  primary loop and die.
00298      */
00299     static std::atomic<bool> termination_signaled_;
00300
00301     /** The instance of the mail retrieval thread.
00302      *
00303      *  The thread operation is defined by the functor operator of the runnable object, @ref
00304      *  thread_object_, which is invoked when this thread is instantiated.
00305      */
00306     static thread thread_;
00307
00308     /** The mail retrieval thread object.
00309      *
00310      *  This is a reference to a runnable object whose functor operator is invoked during
00311      *  instantiation of the mail retrieval thread, @ref thread_.
00312      */
00313     static shared_ptr<XFloatMailRetrievalService> thread_object_;
00314
00315   private:
00316
00317     /* Dynamic Data Members: */
00318
```

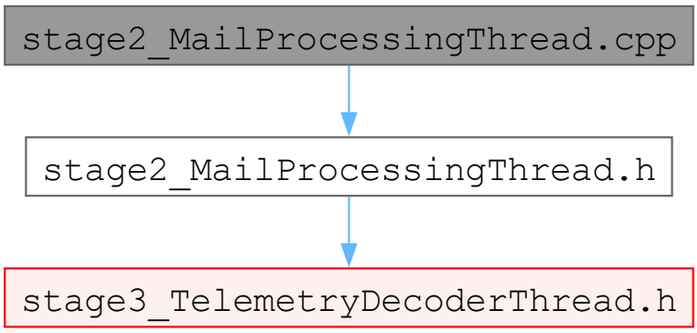
```
00319     /** Is propagation of mail messages to the mail processing thread enabled?
00320      *
00321      *  This field determines whether fetched messages are queued for consumption by the next
00322      *  thread in the processing pipeline, @ref XFloatMailProcessingService. If no instances of
00323      *  that thread are running, then this field will be FALSE and message will be discarded
00324      *  rather than queued. Otherwise, this field will be TRUE and messages will be queued for
00325      *  consumption by @ref XFloatMailProcessingService.
00326      */
00327     bool processing_thread_propagation_enabled_;
00328
00329     /** The VMime session object reference.
00330      *
00331      *  This object is used to connect to the mail server, and create the @ref mail_store_ object.
00332      *  It is created when the thread connects to the mail server and released when the thread
00333      *  disconnects from the mail server.
00334      */
00335     vmime::shared_ptr<vmime::net::session> mail_session_;
00336
00337     /** The VMime mail store object reference.
00338      *
00339      *  This object encapsulates the protocols that provide access to the mail drop. It is created
00340      *  when the thread connects to the mail server and released when the thread disconnects from
00341      *  the mail server.
00342      */
00343     vmime::shared_ptr<vmime::net::store> mail_store_;
00344
00345     /** The VMime inbox folder object reference.
00346      *
00347      *  This object represents the folder on the mail server that contains incoming messages.
00348      *  Typically, this folder is "INBOX".
00349      */
00350     vmime::shared_ptr<vmime::net::folder> inbox_folder_;
00351
00352   private:
00353
00354     /* Private Static Member Functions: */
00355
00356     static vector<path> ListRawMailFiles();
00357
00358     static void set_termination_signaled(bool termination_signaled = true);
00359
00360
00361   public:
00362
00363     /* Public Static Member Functions: */
00364
00365     static void StartWorkerThread() ATTR_NO_EXCEPTIONS;
00366
00367     static void TerminateWorkerThread() ATTR_NO_EXCEPTIONS;
00368
00369   private:
00370
00371     /* Private Member Functions: */
00372
00373     vmime::shared_ptr<vmime::net::store> ConnectToMailServer();
00374
00375     void DisconnectFromMailServer() ATTR_NO_EXCEPTIONS;
00376
00377     size_t FetchMailMessagesFromFileSystem();
00378
00379     size_t FetchMailMessagesFromServer();
00380
00381     string GenerateMailServerUrlString() const;
00382
00383     const vmime::shared_ptr<vmime::net::session>& mail_session();
00384
00385     const vmime::shared_ptr<vmime::net::folder>& OpenMailInboxFolder();
00386
00387   public:
00388
00389     /* Public Member Functions: */
00390
00391     XFloatMailRetrievalService(bool prop = (XFloatMailProcessingService::num_threads() > 0));
00392
00393     ~XFloatMailRetrievalService();
00394
00395     void operator()() ATTR_NO_EXCEPTIONS;
00396
00397 };
00398
00399 } /* namespace xfloat */
00400 } /* namespace iridium */
00401 } /* namespace argo */
00402
00403 #endif /* XFLOATMAILRETRIEVALSERVICE_H_ */
```

11.172 stage2_MailProcessingThread.cpp File Reference

The argo::iridium::xfloat::XFloatMailProcessingService class source file.

```
#include "stage2_MailProcessingThread.h"
```

Include dependency graph for stage2_MailProcessingThread.cpp:



Namespaces

- namespace argo

  The Argo instrument namespace.
- namespace argo::iridium

  The Argo Iridium instrument namespace.
- namespace argo::iridium::xfloat

  The X-Float instrument namespace.

11.172.1 Detailed Description

The argo::iridium::xfloat::XFloatMailProcessingService class source file.

This is the source file for the argo::iridium::xfloat::XFloatMailProcessingService class. This class defines the operation of the mail processing thread(s), which are the second stage in the processing pipeline. The mail processing thread consumes messages that were fetched from the mail server in the previous stage of the processing pipeline, the mail retrieval thread, argo::iridium::xfloat::XFloatMailRetrievalService. The messages are validated, saved to the SQL database, queued for consumption in the next stage of the processing pipeline, the telemetry decoder thread, argo::iridium::xfloat::XFloatTelemetryDecoderService, and their file representations are written to the local file system.

This file contains the static data members of the argo::iridium::xfloat::XFloatMailProcessingService class.

Version

0.6

Date

Oct 16, 2014 2:25:44 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

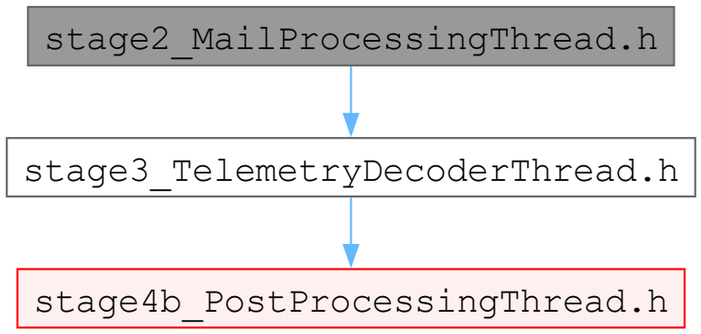
This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.173 stage2_MailProcessingThread.h File Reference

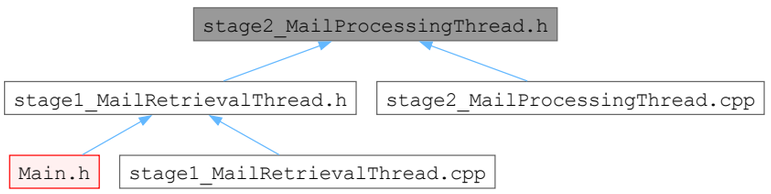
The argo::iridium::xfloat::XFloatMailProcessingService class header file.

```
#include "stage3_TelemetryDecoderThread.h"
```

Include dependency graph for stage2_MailProcessingThread.h:



This graph shows which files directly or indirectly include this file:



Classes

- class argo::iridium::xfloat::XFloatMailProcessingService

  The mail processing thread class.

Namespaces

- namespace argo

  The Argo instrument namespace.
- namespace argo::iridium

  The Argo Iridium instrument namespace.
- namespace argo::iridium::xfloat

  The X-Float instrument namespace.

11.173.1 Detailed Description

The argo::iridium::xfloat::XFloatMailProcessingService class header file.

This is the header file for the argo::iridium::xfloat::XFloatMailProcessingService class. This class defines the operation of the mail processing thread(s), which are the second stage in the processing pipeline. The mail processing thread(s) consume messages that were fetched from the mail server in the previous stage of the processing pipeline, the mail retrieval thread, argo::iridium::xfloat::XFloatMailRetrievalService. The messages are validated, saved to the SQL database, queued for consumption in the next stage of the processing pipeline, the telemetry decoder thread, argo::iridium::xfloat::XFloatTelemetryDecoderService, and their file representations are written to the local file system.

This file contains the class definition and the majority of the functionality of the class.

Version

0.6

Date

Oct 16, 2014 2:25:44 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.174 stage2_MailProcessingThread.h

Go to the documentation of this file.

```
00001 /** The @ref argo::iridium::xfloat::XFloatMailProcessingService class header file.
00002  *
00003  *  This is the header file for the @ref argo::iridium::xfloat::XFloatMailProcessingService class.
00004  *  This class defines the operation of the mail processing thread(s), which are the second stage
00005  *  in the processing pipeline. The mail processing thread(s) consume messages that were fetched
00006  *  from the mail server in the previous stage of the processing pipeline, the mail retrieval
00007  *  thread, @ref argo::iridium::xfloat::XFloatMailRetrievalService. The messages are validated,
00008  *  saved to the SQL database, queued for consumption in the next stage of the processing pipeline,
00009  *  the telemetry decoder thread, @ref argo::iridium::xfloat::XFloatTelemetryDecoderService, and
00010  *  their file representations are written to the local file system.
00011  *
00012  *  This file contains the class definition and the majority of the functionality of the class.
00013  *
00014  *  @file
00015  *
00016  *  @version    0.6
00017  *
00018  *  @date       Oct 16, 2014 2:25:44 PM
00019  *
00020  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00021  *              Engineer I,
00022  *              Department of Physical Oceanography,
00023  *              Woods Hole Oceanographic Institution
00024  *
00025  *  @copyright  2014 Woods Hole Oceanographic Institution.
00026  *              Released under the GNU General Public License.
00027  *
00028  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00029  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00030  *  General Public License for more details. You should have received a copy of the GNU General
00031  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00032  */
00033
00034 #ifndef XFLOATMAILPROCESSINGSERVICE_H_
00035 #define XFLOATMAILPROCESSINGSERVICE_H_
00036
00037 /* Includes: */
00038
00039 // local
00040 #include "stage3_TelemetryDecoderThread.h"
00041
00042 namespace argo {
00043 namespace iridium {
00044 namespace xfloat {
00045
00046 /* Class Definition: */
00047
00048 /** The mail processing thread class.
00049  *
00050  *  This class defines the operation of the mail processing thread(s), which are the second stage
00051  *  in the processing pipeline. The mail processing thread(s) consumes messages that were fetched
00052  *  from the mail server in the previous stage of the processing pipeline, @ref
00053  *  XFloatMailRetrievalService. The messages are validated, saved to the SQL database, queued for
00054  *  consumption in the next stage of the processing pipeline, the telemetry decoder thread(s),
00055  *  @ref XFloatTelemetryDecoderService, and finally their file representations are written to the
00056  *  local file system.
00057  *
00058  *  Zero or more instance of this thread may be instantiated after application initialization,
00059  *  depending on the configuration options. Each thread enters a primary loop in which messages
00060  *  are fetched from a common queue and processed. If the incoming queue is empty, then the thread
00061  *  will sleep for intervals of @ref XFLT_THREAD_SLEEP_MS milliseconds.
00062  *
00063  *  The primary loop will  continue until the daemon thread signals termination. When termination
00064  *  is signaled, the remaining messages in the incoming queue will be processed, then all instances
```

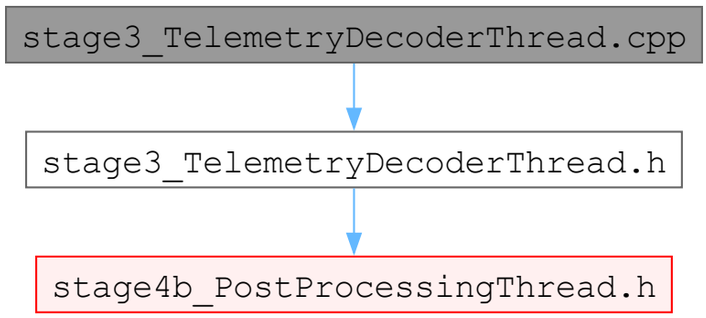
```
00065  *  of this thread will die. This ensures that processing pipeline will be thoroughly flushed
00066  *  before application termination, and that no messages will be partially or incompletely
00067  *  processed.
00068  */
00069 class XFloatMailProcessingService:
00070     public util::mysql::LocalMysqlConfiguration {
00071
00072     /* Friends Functions: */
00073
00074     /** Friend function for populating some of the static members of this class from the
00075      *  configuration file options.
00076      */
00077     friend po::options_description
      util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription();
00078
00079   private:
00080
00081     /* Static Data Members: */
00082
00083     /** The incoming mail message processing queue.
00084      *
00085      *  This synchronous FIFO queue is used to pass messages produced by the previous stage of the
00086      *  processing pipeline, the mail retrieval thread, @ref XFloatMailRetrievalService, to the
00087      *  mail processing thread(s). All the operations performed by on the queue are atomic.
00088      *  Depending on the number of mail processing threads, this queue may function as a
00089      *  one-to-one queue  or one-to-many queue.
00090      */
00091     static util::thread::SynchronousQueue<shared_ptr<XFloatMailMessage> > msg_processing_queue_;
00092
00093     /** The number of mail processing threads.
00094      *
00095      *  The value of this field determines the number of mail processing threads that are
00096      *  instantiated after initialization. Its value is set as a result of parsing the
00097      *  configuration file and/or command-line arguments during initialization. If the value is
00098      *  unspecified, then it defaults to a value of @ref CFG_DEFAULT_NPROC.
00099      */
00100     static unsigned int num_threads_;
00101
00102     /** The termination signaled flag.
00103      *
00104      *  This flag is used  to signal termination of the mail processing thread(s). During normal
00105      *  operation, the value of this field will be FALSE. Once termination is signaled, it will
00106      *  become TRUE causing all of the instances of the mail processing thread to process the
00107      *  remaining messages in the incoming queue, exiting their primary loop, and then die.
00108      */
00109     static std::atomic<bool> termination_signaled_;
00110
00111     /** The mail processing thread group.
00112      *
00113      *  This object represents the group of mail processing threads that are instantiated after
00114      *  initialization. Each new thread that is instantiated is added to this group. Upon
00115      *  termination, the thread group is joined.
00116      */
00117     static thread_group thread_group_;
00118
00119     /** The mail processing thread object(s).
00120      *
00121      *  Each instance of the mail processing thread is created by invoking the functor of an
00122      *  instance of this runnable class. Each instance of this object that is created during
00123      *  instantiation of a new thread is stored in this vector.
00124      */
00125     static vector<shared_ptr<XFloatMailProcessingService> > thread_objects_;
00126
00127   private:
00128
00129     /* Private Static Member Functions: */
00130
00131     static bool MsgProcessingQueuePop(shared_ptr<XFloatMailMessage>& msg, bool block = false);
00132
00133     static void set_termination_signaled(bool termination_signaled = true);
00134
00135   public:
00136
00137     /* Public Static Member Functions: */
00138
00139     static bool MsgProcessingQueuePush(const shared_ptr<XFloatMailMessage>& msg, bool block = true);
00140
00141     static void MsgProcessingQueuePushAll(
00142         vector<shared_ptr<XFloatMailMessage»::const_iterator first,
00143         vector<shared_ptr<XFloatMailMessage»::const_iterator last);
00144
00145     static unsigned int num_threads();
00146
00147     static void StartWorkerThreads() ATTR_NO_EXCEPTIONS;
00148
00149     static void TerminateWorkerThreads() ATTR_NO_EXCEPTIONS;
00150
00151   private:
00152
00153     /* Private Member Functions: */
00154
00155     void ProcessMailMsg(const shared_ptr<XFloatMailMessage>& msg) const;
00156
00157   public:
00158
00159     /* Public Member Functions: */
00160
00161     XFloatMailProcessingService();
00162
00163     ~XFloatMailProcessingService();
00164
00165     void operator()() ATTR_NO_EXCEPTIONS;
00166 };
00167
00168 } /* namespace xfloat */
00169 } /* namespace iridium */
00170 } /* namespace argo */
00171
```

```
00172 #endif /* XFLOATMAILPROCESSINGSERVICE_H_ */
```

11.175 stage3_TelemetryDecoderThread.cpp File Reference

The argo::iridium::xfloat::XFloatTelemetryDecoderService class source file.

`#include "stage3_TelemetryDecoderThread.h"`
Include dependency graph for stage3_TelemetryDecoderThread.cpp:



Namespaces

- namespace argo
  The Argo instrument namespace.
- namespace argo::iridium
  The Argo Iridium instrument namespace.
- namespace argo::iridium::xfloat
  The X-Float instrument namespace.

11.175.1 Detailed Description

The argo::iridium::xfloat::XFloatTelemetryDecoderService class source file.

This is the source file for the argo::iridium::xfloat::XFloatTelemetryDecoderService class. This class defines the operation of the telemetry decoder thread(s), which are the third stage in the processing pipeline. The telemetry decoder thread(s) consume mail messages that were processed by the previous stage of the processing pipeline, the mail processing thread(s), argo::iridium::xfloat::XFloatMailProcessingService. The processed mail messages are assigned to their corresponding dive data object. Then, the Iridium Short Burst Data (SBD) messages, that are supplied as attachments to the mail messages are decoded. The decoded data is contained in an instance of a class that inherits from util::xmsg::XMessage. The resultant object is also assigned to the dive data object.

This file contains the static data members of the argo::iridium::xfloat::XFloatTelemetryDecoderService class.

Version

0.6

Date

Oct 16, 2014 2:23:02 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

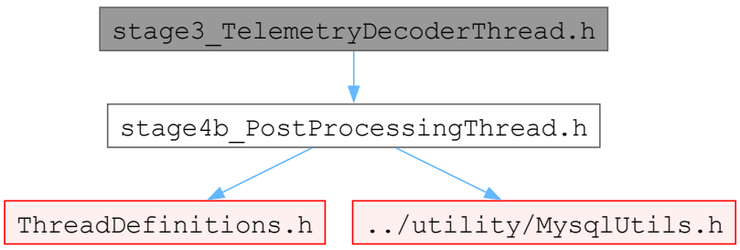
This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.176 stage3_TelemetryDecoderThread.h File Reference

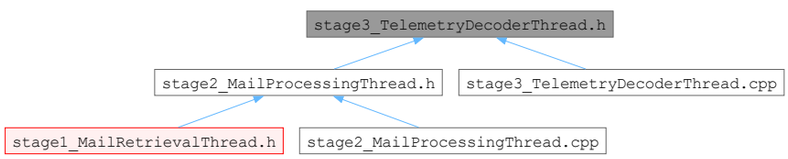
The argo::iridium::xfloat::XFloatTelemetryDecoderService class header file.

```
#include "stage4b_PostProcessingThread.h"
```

Include dependency graph for stage3_TelemetryDecoderThread.h:



This graph shows which files directly or indirectly include this file:



Classes

- class argo::iridium::xfloat::XFloatTelemetryDecoderService

  The telemetry decoder thread class.

Namespaces

- namespace argo

  The Argo instrument namespace.
- namespace argo::iridium

  The Argo Iridium instrument namespace.
- namespace argo::iridium::xfloat

  The X-Float instrument namespace.

Macros

- #define DBG_X_DECODE 0

  Set to true to print additional debug output for telemetry decoding.

11.176.1 Detailed Description

The argo::iridium::xfloat::XFloatTelemetryDecoderService class header file.

This is the header file for the argo::iridium::xfloat::XFloatTelemetryDecoderService class. This class defines the operation of the telemetry decoder thread(s), which are the third stage in the processing pipeline. The telemetry decoder thread(s) consume mail messages that were processed by the previous stage of the processing pipeline, the mail processing thread(s), argo::iridium::xfloat::XFloatMailProcessingService. The processed mail messages are assigned to their corresponding dive data object. Then, the Iridium Short Burst Data (SBD) messages, that are supplied as attachments to the mail messages are decoded. The decoded data is contained in an instance of a class that inherits from util::xmsg::XMessage. The resultant object is also assigned to the dive data object.

This file contains the class definition and the majority of the functionality of the class.

Version

0.6

Date

Oct 16, 2014 2:23:02 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.176.2 Macro Definition Documentation

11.176.2.1 DBG_X_DECODE

```
#define DBG_X_DECODE 0
```

Set to true to print additional debug output for telemetry decoding.

11.177 stage3_TelemetryDecoderThread.h

Go to the documentation of this file.

```
00001 /** The @ref argo::iridium::xfloat::XFloatTelemetryDecoderService class header file.
00002  *
00003  *  This is the header file for the @ref argo::iridium::xfloat::XFloatTelemetryDecoderService class.
00004  *  This class defines the operation of the telemetry decoder thread(s), which are the third stage
00005  *  in the processing pipeline. The telemetry decoder thread(s) consume mail messages that were
00006  *  processed by the previous stage of the processing pipeline, the mail processing thread(s),
00007  *  @ref argo::iridium::xfloat::XFloatMailProcessingService. The processed mail messages are
00008  *  assigned to their corresponding dive data object. Then, the Iridium Short Burst Data (SBD)
00009  *  messages, that are supplied as attachments to the mail messages are decoded. The decoded data
00010  *  is contained in an instance of a class that inherits from @ref util::xmsg::XMessage. The
00011  *  resultant object is also assigned to the dive data object.
00012  *
00013  *  This file contains the class definition and the majority of the functionality of the class.
00014  *
00015  *  @file
00016  *
00017  *  @version    0.6
00018  *
00019  *  @date       Oct 16, 2014 2:23:02 PM
00020  *
00021  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00022  *              Engineer I,
00023  *              Department of Physical Oceanography,
00024  *              Woods Hole Oceanographic Institution
00025  *
00026  *  @copyright  2014 Woods Hole Oceanographic Institution.
00027  *              Released under the GNU General Public License.
00028  *
00029  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00030  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00031  *  General Public License for more details. You should have received a copy of the GNU General
00032  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00033  */
00034
00035 #ifndef XFLOATTELEMETRYDECODERSERVICE_H_
00036 #define XFLOATTELEMETRYDECODERSERVICE_H_
00037
00038 /* Includes: */
00039
00040 // local
00041 //#include "stage4a_NCFileWriterThread.h"
00042 #include "stage4b_PostProcessingThread.h"
00043
00044 /* Definitions: */
00045
00046 /** Set to true to print additional debug output for telemetry decoding. */
00047 #define DBG_X_DECODE 0
00048
```

```
00049 namespace argo {
00050 namespace iridium {
00051 namespace xfloat {
00052
00053 /* Class Definition: */
00054
00055 /** The telemetry decoder thread class.
00056  *
00057  *  This class defines the operation of the telemetry decoder thread(s), which are the third stage
00058  *  in the processing pipeline. The telemetry decoder thread(s) consumes processed mail messages
00059  *  that are produced by  the previous stage of the processing pipeline, the mail processing
00060  *  thread, @ref XFloatMailProcessingService.
00061  *
00062  *  For each message received from the mail processing thread, the corresponding instrument object,
00063  *  @ref XFloat, is retrieved. Likewise, the dive data object, @ref XFloatDiveData, corresponding
00064  *  to the dive or cycle number of the mail message is retrieved from the instrument object. The
00065  *  mail message is assigned to the dive data object. Finally, the SBD message data attached to
00066  *  the mail message is decoded into one of the @ref util::xmsg::XMessage which is determined by
00067  *  the X-Message type identifier contained in the binary message data. The resulting @ref
00068  *  util::xmsg::XMessage object is assigned to the @ref XFloatDiveData.
00069  *
00070  *  Zero or more instance of this thread may be instantiated after application initialization,
00071  *  depending on the configuration options. Each thread enters a primary loop in which messages
00072  *  are fetched from a common queue and processed. If the incoming queue is empty, then the thread
00073  *  will sleep for intervals of @ref XFLT_THREAD_SLEEP_MS milliseconds.
00074  *
00075  *  The primary loop will  continue until the daemon thread signals termination. When termination
00076  *  is signaled, the remaining messages in the incoming queue will be processed, then all instances
00077  *  of this thread will die. This ensures that processing pipeline will be thoroughly flushed
00078  *  before application termination, and that no messages will be partially or incompletely
00079  *  processed.
00080  */
00081 class XFloatTelemetryDecoderService:
00082         public util::mysql::MysqlUtils {
00083
00084     /* Friends Functions: */
00085
00086     /** Friend function for populating some of the static members of this class from the
00087      *  configuration file options.
00088      */
00089     friend options_description
00090     util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription();
00091
00092 private:
00093
00094     /* Static Data Members: */
00095
00096     /** The telemetry decoder thread group.
00097      *
00098      *  This object represents the group of telemetry decoder threads that are instantiated after
00099      *  initialization. Each new thread that is instantiated is added to this group. Upon
00100      *  termination, the thread group is joined.
00101      */
00102     static thread_group thread_group_;
00103
00104     /** The telemetry decoder thread objects.
00105      *
00106      *  Each instance of the telemetry decoder threads is created by invoking the functor of an
00107      *  instance of this runnable class. Each instance of this object that is created during
00108      *  instantiation of a new thread is stored in this vector.
00109      */
00110     static vector<shared_ptr<XFloatTelemetryDecoderService> > thread_objects_;
00111
00112     /** The telemetry decode queue.
00113      *
00114      *  This synchronous FIFO queue is used to pass messages produced by the previous stage of the
00115      *  processing pipeline, the mail processing thread(s), @ref XFloatMailRetrievalService, to the
00116      *  telemetry decoder thread(s). All the operations performed by on the queue are atomic.
00117      *  Depending on the number of telemetry decoder threads, this queue may function as a
00118      *  one-to-one queue  or one-to-many queue.
00119      */
00120     static util::thread::SynchronousQueue<shared_ptr<XFloatMailMessage> > msg_decoding_queue_;
00121
00122     /** The number of telemetry decoder threads.
00123      *
00124      *  The value of this field determines the number of telemetry decoder threads that are
00125      *  instantiated after initialization. Its value is set as a result of parsing the
00126      *  configuration file and/or command-line arguments during initialization. If the value is
00127      *  unspecified, then it defaults to a value of @ref CFG_DEFAULT_NDECO.
00128      */
00129     static unsigned int num_threads_;
00130
00131     /** The termination signaled flag.
00132      *
00133      *  This flag is used  to signal termination of the telemetry decoder thread(s). During normal
00134      *  operation, the value of this field will be FALSE. Once termination is signaled, it will
00135      *  become TRUE causing all of the instances of the telemetry decoder thread to process the
00136      *  remaining messages in the incoming queue, exiting their primary loop, and then die.
00137      */
00138     static std::atomic<bool> termination_signaled_;
00139
00140 private:
00141
00142     /* Private Member Functions: */
00143
00144     /** Pop a mail message out of the telemetry decoder queue.
00145      *
00146      *  This function is used by the telemetry decoder thread(s) to receive mail messages produced
00147      *  by the previous stage in the processing pipeline, the mail processing thread, @ref
00148      *  XFloatMailProcessingService. The queue is synchronous, so the underlying pop function is
00149      *  atomic.
00150      *
00151      *  @param[out]     msg             The destination of the mail message to pop out of the
00152      *                                  queue.
00153      *  @param[in]      block           If TRUE, this function will block until the queue
00154      *                                  is not empty. If FALSE, this function will return FALSE
00155      *                                  if the queue is empty (and no element will be popped
00156      *                                  out of the queue).
```

```
00157      *
00158      *  @return         TRUE if the element was successfully popped out of the queue, or FALSE
00159      *                  otherwise.
00160      */
00161     static bool MsgProcessingQueuePop(shared_ptr<XFloatMailMessage>& msg,
00162     bool block = false) {
00163         return msg_decoding_queue_.pop(msg, block);
00164     }
00165
00166     /** Set the termination signaled flag.
00167      *
00168      *  This function is called by @ref TerminateWorkerThreads() in order to cause all currently
00169      *  running instances of the telemetry decoder thread to exit their primary loop and die.
00170      *
00171      *  @param[in]      termination_signaled        The termination signaled value to set.
00172      */
00173     static void set_termination_signaled(bool termination_signaled = true) {
00174         termination_signaled_ = termination_signaled;
00175     }
00176
00177 public:
00178
00179     /* Public Member Functions: */
00180
00181     /** The constructor.
00182      *
00183      */
00184     XFloatTelemetryDecoderService() {
00185         /* Do nothing. */
00186     }
00187
00188     /** The destructor.
00189      *
00190      */
00191     ~XFloatTelemetryDecoderService() {
00192         /* Do nothing. */
00193     }
00194
00195     /** Process a dive data object
00196      *
00197      *  @param[in]  dive_data   The dive data object to process.
00198      *
00199      *  This function is invoked each time a new message for this dive is decoded. The function
00200      *  first checks if the dive data object is complete and proceeds accordingly.
00201      *
00202      *  If the dive data object is complete, then PHY and listing files are written, the object is
00203      *  saved to the database, the KML-document-is-updated flag is set to false, the dive data
00204      *  object is pushed into the NetCDF file writing queue, and the parent instrument object is
00205      *  pushed into the float processing queue.
00206      *
00207      *  If the dive data object is incomplete, then the listing file is written, the dive data
00208      *  object is saved to the database, and the parent instrument is pushed into the float
00209      *  processing queue.
00210      */
00211     void ProcessDiveData(shared_ptr<XFloatDiveData>& dive_data) const {
00212
00213         namespace attr = blg::attributes;
00214
00215         bool dive_complete = false;
00216
00217         try {
00218
00219             if(!dive_data)
00220                 EXC_THROW("Pointer to dive data object is NULL!");
00221
00222             // Set scoped log attributes.
00223             BOOST_LOG_SCOPED_THREAD_ATTR("WhoiID", attr::constant<int>(dive_data->id_instrument()));
00224             BOOST_LOG_SCOPED_THREAD_ATTR("DacID", attr::constant<int>(dive_data->id_dac()));
00225             BOOST_LOG_SCOPED_THREAD_ATTR("WmoID", attr::constant<int>(dive_data->id_wmo()));
00226             BOOST_LOG_SCOPED_THREAD_ATTR("DiveID", attr::constant<int>(dive_data->id_dive()));
00227
00228             ldbg() « "Processing dive data object...";
00229
00230             try {
00231
00232                 ldbg() « "Checking dive data completeness...";
00233                 dive_complete = dive_data->CheckCompleteness();
00234
00235             }CATCH_BLOCK("Failed to check dive data completeness!")
00236
00237             // Check if the dive is complete.
00238             if(dive_complete) {
00239
00240                 lout() « "Dive complete.";
00241
00242                 // Get the parent instrument object.
00243                 shared_ptr<XFloat> parent_instrument(dive_data->parent_instrument());
00244
00245                 // Write file and save.
00246                 dive_data->WritePhyFile();
00247                 dive_data->WriteListingFile();
00248                 dive_data->SaveObjectToDatabase();
00249
00250 #               if defined(XMTPS_NC_FILE_WRITER_USES_QUEUE) && XMTPS_NC_FILE_WRITER_USES_QUEUE
00251
00252                 // Push the dive data object into the NC file writer queue.
00253                 XFloatNCFileWriterThread::NcFileProcessingQueuePush(dive_data);
00254
00255 #               else
00256
00257                 // Update the NetCDF file status flag.
00258                 dive_data->set_netcdf_file_updated(false);
00259
00260 #               endif // defined(XMTPS_NC_FILE_WRITER_USES_QUEUE) && XMTPS_NC_FILE_WRITER_USES_QUEUE
00261
00262
00263 #               if defined(XMTPS_POST_PROC_USES_QUEUE) && XMTPS_POST_PROC_USES_QUEUE
00264
```

```
00265                 // Update KML status flag.
00266                 parent_instrument->set_kml_document_updated(false);
00267 
00268                 // Push the instrument object into the post-processing queue.
00269                 XFloatPostProcessingService::FloatProcessingQueuePush(parent_instrument);
00270 
00271 #               else
00272 
00273                 // Update status flags.
00274                 parent_instrument->set_kml_document_updated(false);
00275                 parent_instrument->set_kml_file_updated(false);
00276                 parent_instrument->set_status_file_updated(false);
00277 
00278 #               endif // defined(XMTPS_POST_PROC_USES_QUEUE) && XMTPS_POST_PROC_USES_QUEUE
00279 
00280                 return;
00281 
00282             } else {
00283 
00284                 // Update the listing file and save after each new message.
00285                 dive_data->WriteListingFile();
00286                 dive_data->SaveObjectToDatabase();
00287 
00288 #               if defined(XMTPS_POST_PROC_USES_QUEUE) && XMTPS_POST_PROC_USES_QUEUE
00289 
00290                 // Push the instrument object into the post-processing queue.
00291                 XFloatPostProcessingService::FloatProcessingQueuePush(
00292                         dive_data->parent_instrument());
00293 
00294 #               else
00295 
00296                 // Update status flags.
00297                 dive_data->parent_instrument()->set_status_file_updated(false);
00298 
00299 #               endif // defined(XMTPS_POST_PROC_USES_QUEUE) && XMTPS_POST_PROC_USES_QUEUE
00300 
00301             }
00302 
00303         }CATCH_BLOCK("Failed to process dive data object!")
00304     }
00305 
00306     /** Decode the telemetry data from a processed mail message.
00307      *
00308      *  This function is used to decode the binary Iridium Short Burst Data (SBD) message that is
00309      *  supplied as an attachment to the email message. Mail messages that are received from the
00310      *  previous stage in the processing pipeline, the mail processing thread(s), @ref
00311      *  XFloatMailProcessingService, are decoded using this function.
00312      *
00313      *  First, the function checks the message reference @c msg. If it is NULL, then the function
00314      *  throws an exception. Otherwise, it continues on to set the global logger attributes:
00315      *   - DiveID,
00316      *   - MsgUID,
00317      *   - PcktID,
00318      *   - MomsnID,
00319      *   - WhoiID,
00320      *   - DacID, and
00321      *   - WmoID.
00322      *
00323      *  These attributes are used by the global logger object for filtering, etc. The values of the
00324      *  attributes are assumed to be known since, if any were unknown, then the mail processing
00325      *  threads would have queried them from the SQL database.
00326      *
00327      *  Next, the float object, @c xfloat, that corresponds to the instrument that transmitted
00328      *  this message is retrieved. If the float object does not already exist, it will be created.
00329      *  Also, the metadata object for this instrument is retrieved, which contains information
00330      *  about the start-of-mission and deployment date/times. The time of flight of the message
00331      *  is examined to ensure that the message is valid. Messages that were transmitted after the
00332      *  start-of-mission or deployment date/times are discarded.
00333      *
00334      *  Then, the dive data object, @c xdive, which contains all the data received for a single
00335      *  instrument and single dive/cycle is retrieved from the float object. If the dive data
00336      *  object for this dive/cycle does not already exist, it will be created. The mail message,
00337      *  @c msg, is then assigned to the dive data object.
00338      *
00339      *  The SBD message is passed to the @ref ProcessDiveData() function to be decoded.
00340      *
00341      *  @param[in]       msg             A shared reference to the mail message to decode.
00342      */
00343     void DecodeMessage(shared_ptr<XFloatMailMessage> msg) const {
00344 
00345         namespace attr = blg::attributes;
00346 
00347         shared_ptr<XFloat> xfloat;
00348         shared_ptr<XFloatDiveData> xdive;
00349         shared_ptr<vector<uint8_t> > xmsg_id_types;
00350 
00351         int msg_id_dive = XFLT_DIVE_UNKNOWN;
00352 
00353         try {
00354 
00355             /* Decode the mail message: */
00356 
00357             // Check the message pointer.
00358             if(!msg)
00359                 EXC_THROW("Null pointer to mail message object!");
00360 
00361             // Get the corresponding instrument and dive data objects
00362             xdive = (xfloat = XFloat::GetFloat(msg->id_instrument()))->GetDiveData(msg_id_dive =
    msg->id_dive());
00363 
00364             // Decode the mail message data.
00365             if(xdive->DecodeMailMessageData(msg))
00366 
00367                 // Ignore invalid messages.
00368                 return;
00369 
00370         }CATCH_BLOCK("Failed to decode telemetry data from mail message object!")
00371 
```

```
00372         try {
00373
00374             /* Do message-type specific pre-processing: */
00375
00376             if((xmsg_id_types = msg->xmsg_id_types()))
00377
00378                 // Iterate over the message type identifiers.
00379                 for(auto it = xmsg_id_types->begin(); it < xmsg_id_types->end(); ++it)
00380
00381                     switch(*it) {
00382
00383                     case x::XMSG_TYPE_ID_ARGO_DATA:
00384
00385                         /* Do type-specific pre-processing for Argo Data messages: */
00386
00387                         if(!xdive->is_surface_pres_valid())
00388
00389                             // Update the derived surface data fields.
00390                             xdive->UpdateDerivedSurfaceDataFields();
00391
00392                         // Compute the pressure offset.
00393                         xdive->ComputePressureOffsetFromSurfaceTimeSeries(3);
00394
00395                         break;
00396
00397                     case x::XMSG_TYPE_ID_GPS_ENDOFDIVE:
00398
00399                         /* Do type-specific pre-processing for GPS (end-of-dive) messages: */
00400
00401                         if(!xdive->is_surface_pres_valid())
00402
00403                             // Update the derived surface data fields.
00404                             xdive->UpdateDerivedSurfaceDataFields();
00405
00406                         // Compute the pressure offset.
00407                         xdive->ComputePressureOffsetFromSurfaceTimeSeries(3);
00408
00409                         break;
00410
00411                     case x::XMSG_TYPE_ID_PRES_SURF0:
00412                     case x::XMSG_TYPE_ID_PRES_SURF1:
00413                     case x::XMSG_TYPE_ID_PRES_SURF2:
00414                     case x::XMSG_TYPE_ID_PRES_SURF3:
00415                     case x::XMSG_TYPE_ID_PRES_SURF4:
00416                     case x::XMSG_TYPE_ID_PRES_SURF5:
00417                     case x::XMSG_TYPE_ID_PRES_SURF6:
00418                     case x::XMSG_TYPE_ID_PRES_SURF7:
00419                     case x::XMSG_TYPE_ID_PRES_SURF8:
00420                     case x::XMSG_TYPE_ID_PRES_SURF9:
00421                     case x::XMSG_TYPE_ID_PRES_SURFA:
00422                     case x::XMSG_TYPE_ID_PRES_SURFB:
00423                     case x::XMSG_TYPE_ID_PRES_SURFC:
00424                     case x::XMSG_TYPE_ID_PRES_SURFD:
00425                     case x::XMSG_TYPE_ID_PRES_SURFE:
00426                     case x::XMSG_TYPE_ID_PRES_SURFF:
00427
00428                         /* Do type-specific pre-processing for surface data messages: */
00429
00430                         // Update the derived surface data fields.
00431                         xdive->UpdateDerivedSurfaceDataFields();
00432
00433                         // Compute the pressure offset.
00434                         xdive->ComputePressureOffsetFromSurfaceTimeSeries(3);
00435
00436                         break;
00437
00438                     default:
00439
00440                         /* Do nothing. */
00441
00442                         break;
00443
00444                     } // switch(*it)
00445
00446         }CATCH_BLOCK("Failed to do message-type-specific pre-processing!")
00447
00448         try {
00449
00450             /* Process the dive data: */
00451
00452             ProcessDiveData(xdive);
00453
00454         } CATCH_BLOCK("Failed to process dive data!")
00455
00456         try {
00457
00458             /* Do message-type-specific post-processing: */
00459
00460             if(xmsg_id_types)
00461
00462                 // Iterate over the message type identifiers.
00463                 for(auto it = xmsg_id_types->begin(); it < xmsg_id_types->end(); ++it)
00464
00465                     switch(*it) {
00466
00467                     case x::XMSG_TYPE_ID_GPS_STARTOFDIVE:
00468
00469                         /* Do type-specific post-processing for GPS (start-of-dive) messages: */
00470
00471                         if(msg_id_dive < 0)
00472
00473                             break;
00474
00475                         try {
00476
00477                             shared_ptr<XFloatDiveData> previous_dive_data;
00478
00479 #                           if defined(BUILD_DEBUG) && BUILD_DEBUG
```

```
00480                             {
00481                                 // Set the scoped thread log attributes.
00482                                 BOOST_LOG_SCOPED_THREAD_ATTR(
00483                                         "WhoiID",
00484                                         attr::constant<int>(msg->id_instrument()));
00485                                 BOOST_LOG_SCOPED_THREAD_ATTR(
00486                                         "DacID",
00487                                         attr::constant<int>(msg->id_dac()));
00488                                 BOOST_LOG_SCOPED_THREAD_ATTR(
00489                                         "WmoID",
00490                                         attr::constant<int>(msg->id_wmo()));
00491                                 BOOST_LOG_SCOPED_THREAD_ATTR(
00492                                         "DiveID",
00493                                         attr::constant<int>(msg_id_dive));
00494                                 ldbg() « "Decoded start-of-dive GPS message. "
00495                                           « "Checking completeness of previous dive...";
00496                             }
00497 #                           endif // defined(BUILD_DEBUG) && BUILD_DEBUG
00498
00499                             // Reprocess the previous dive if it is complete.
00500                             if((previous_dive_data = (xfloat->GetDiveData(msg_id_dive - 1, true)))
00501                                     && !previous_dive_data->is_complete())
00502
00503                                 // Reprocess incomplete dive data.
00504                                 ProcessDiveData(previous_dive_data);
00505
00506                         }CATCH_BLOCK("Failed to do message-type-specific post-processing for "
00507                                 "start-of-dive GPS message!")
00508
00509                         break;
00510
00511                     default:
00512
00513                         /* Do nothing. */
00514
00515                         break;
00516
00517                     } // switch(*it)
00518
00519         }CATCH_BLOCK("Failed to do message-type-specific post-processing!")
00520
00521     }
00522
00523     /** Functor operator overload function: the main entry point for the telemetry decoder
00524      *  thread(s).
00525      *
00526      *  This function defines the behavior of the telemetry decoder thread(s), which are
00527      *  responsible for decoding the binary data from the processed mail messages that are
00528      *  produced by the mail processing thread, @ref XFloatMailProcessingService. After
00529      *  initialization, zero or more instances of this thread are instantiated. The static
00530      *  configuration options for this thread are set as a result of parsing the command-line
00531      *  arguments and/or configuration file.
00532      *
00533      *  First, the scope thread attribute "ThreadName" is set to "sbd-decode". This attribute is
00534      *  used by the global logger object for filtering, etc. Then the thread enters its primary
00535      *  loop, in which it pops mail messages from the incoming queue one at a time and decodes
00536      *  the messages before passing them to the next stage in the processing pipeline, the
00537      *  post-processing analysis thread(s), @ref XFloatPostProcessingService. If the incoming
00538      *  message queue is empty, then the connection o the SQL database is closed, and the thread
00539      *  sleeps for @ref XFLT_THREAD_SLEEP_MS milliseconds.
00540      *
00541      *  Once termination is signaled by the daemon process i.e. @ref termination_signaled_ is
00542      *  TRUE, the thread will exit its primary loop and die.
00543      *
00544      *  @exception      None. Never throws.
00545      */
00546     void operator ()() ATTR_NO_EXCEPTIONS {
00547
00548         namespace attr = blg::attributes;
00549
00550         shared_ptr<XFloatMailMessage> msg;
00551
00552         BOOST_LOG_SCOPED_THREAD_ATTR("ThreadName", attr::constant<string>("sbd-decode"));
00553         ldbg() « "Thread instantiated.";
00554
00555         // Enter the primary thread loop.
00556         do { // } while(!termination_signaled_);
00557
00558             try {
00559
00560                 // Pop message from queue. Do not block.
00561                 if(MsgProcessingQueuePop(msg, false)) {
00562
00563                     /*  Iterate over and decode all of the messages in the queue before checking
00564                      *  if terminations is signaled. If any of the messages in the queue are not
00565                      *  processed before termination, then there is a (slight) chance that they
00566                      *  will not be processed when the decoder is restarted!
00567                      *
00568                      */
00569
00570                     do { // } while(MsgProcessingQueuePop(msg, false));
00571
00572                         // Decode the mail message.
00573                         DecodeMessage(msg);
00574
00575                         // Reset the message pointer.
00576                         msg.reset();
00577
00578                     } while(MsgProcessingQueuePop(msg, false));
00579
00580                 } else {
00581
00582                     // Close SQL connection if open, for now, and sleep.
00583                     CloseSqlConnection();
00584                     sleep(bpt::millisec(XFLT_THREAD_SLEEP_MS));
00585                 }
00586
00587             }CATCH_NOTHROW("Exception caught in telemetry decoder thread!")
```

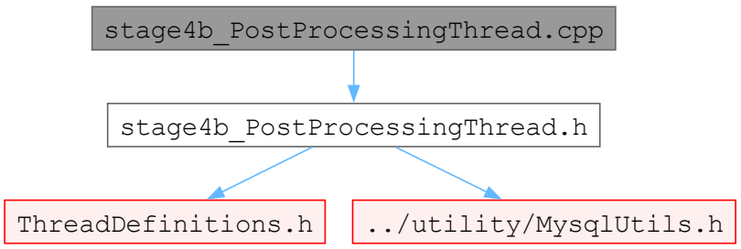
```
00588
00589         } while(!termination_signaled_);
00590
00591         // Terminate.
00592         ldbg() « "Termination signal received! Shutting down...";
00593
00594         // Drain messages queued while this thread was sleeping, so none are lost on shutdown.
00595         while(MsgProcessingQueuePop(msg, false)) {
00596
00597             try {
00598
00599                 DecodeMessage(msg);
00600
00601             }CATCH_NOTHROW("Exception caught while draining telemetry decoder queue!")
00602
00603             msg.reset();
00604         }
00605
00606         CloseSqlConnection();
00607
00608         return;
00609     }
00610
00611     /** Push a mail message into the telemetry decoder queue.
00612      *
00613      *  This function is used by the previous stage in the processing pipeline, the mail processing
00614      *  thread, @ref XFloatMailProcessingService, to pass a mail message to the telemetry decoder
00615      *  thread(s). The mail message processing queue is synchronous, so the underlying push
00616      *  function is atomic.
00617      *
00618      *  @param[in]      msg             The mail message to push into the  queue.
00619      *  @param[in]      block           If TRUE, this function will block until the queue
00620      *                                  is not full. If FALSE, this function will return FALSE
00621      *                                  if the queue is full (and the element will not be
00622      *                                  pushed into the queue).
00623      *
00624      *  @return         TRUE if the element was successfully pushed into the queue, or FALSE
00625      *                  otherwise.
00626      */
00627     static bool MsgDecodingQueuePush(const shared_ptr<XFloatMailMessage>& msg,
00628     bool block = true) {
00629         return msg_decoding_queue_.push(msg, block);
00630     }
00631
00632     /** Get the number of telemetry decoder threads.
00633      *
00634      *  @return         The number of telemetry decoder threads.
00635      */
00636     static unsigned int num_threads() {
00637         return num_threads_;
00638     }
00639
00640     /** Instantiate the telemetry decoder worker thread(s).
00641      *
00642      *  This function is used by the daemon thread to instantiate zero or more telemetry decoder
00643      *  threads after initialization. The number of threads that are instantiated by this function
00644      *  is determined by the value of @ref num_threads_, which is set as a result of parsing the
00645      *  command-line argument and/or configuration file during initialization.
00646      *
00647      *  Each thread is instantiated by creating an instance of this class, which is pushed into
00648      *  the @ref thread_objects_ vector, and invoking the functor of each object. Each thread
00649      *  that is created is added to the @ref thread_group_ object.
00650      *
00651      *  @exception          None. Never throws.
00652      */
00653     static void StartWorkerThreads() ATTR_NO_EXCEPTIONS {
00654
00655         try {
00656
00657             ldbg() « "Starting telemetry decoder thread(s)...";
00658
00659             for(unsigned int i = 0; i < num_threads_; i++) {
00660
00661                 shared_ptr<XFloatTelemetryDecoderService> thread(
00662                         make_shared<XFloatTelemetryDecoderService>());
00663                 thread_objects_.push_back(thread);
00664                 thread_group_.create_thread(*thread);
00665             }
00666
00667         }CATCH_BLOCK("Failed to start telemetry decoder worker thread(s)!")
00668     }
00669
00670     /** Terminate the telemetry decoder worker thread(s).
00671      *
00672      *  This function is used by the daemon thread to kill the telemetry decoder worker thread(s)
00673      *  upon application termination. The function signals termination, and then attempts to join
00674      *  the thread group.
00675      *
00676      *  @warning            If any of telemetry decoder threads fails to terminate properly, then
00677      *                      the @c join_all() function could block indefinitely. Precautions have
00678      *                      been take to ensure that this does not happen, but it may still be
00679      *                      possible.
00680      *
00681      *  @exception          None. Never throws.
00682      */
00683     static void TerminateWorkerThreads() ATTR_NO_EXCEPTIONS {
00684
00685         try {
00686
00687             ldbg() « "Terminating telemetry decoder thread(s)...";
00688
00689             set_termination_signaled();
00690
00691             if(num_threads_)
00692                 thread_group_.join_all();
00693
00694         }CATCH_BLOCK("Failed to terminate telemetry decoder worker thread(s)!")
00695
```

```
00696     }
00697
00698 };
00699
00700 } /* namespace xfloat */
00701 } /* namespace iridium */
00702 } /* namespace argo */
00703 #endif /* XFLOATTELEMETRYDECODERSERVICE_H_ */
```

11.178 stage4b_PostProcessingThread.cpp File Reference

The argo::iridium::xfloat::XFloatPostProcessingService class source file.

`#include "stage4b_PostProcessingThread.h"`
Include dependency graph for stage4b_PostProcessingThread.cpp:



Namespaces

- namespace argo

  The Argo instrument namespace.
- namespace argo::iridium

  The Argo Iridium instrument namespace.
- namespace argo::iridium::xfloat

  The X-Float instrument namespace.

11.178.1 Detailed Description

The argo::iridium::xfloat::XFloatPostProcessingService class source file.

This class is defines the operation of the post-processing analysis thread(s), which are the fourth stage of the processing pipeline. The post-processing analysis thread(s) are responsible for generating plots, Matlab, and NetCDF files as well as performing some analysis of the decoded telemetry fields in order to derive performance metrics of the instrument.

Version

0.6

Date

Oct 16, 2014 2:21:06 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

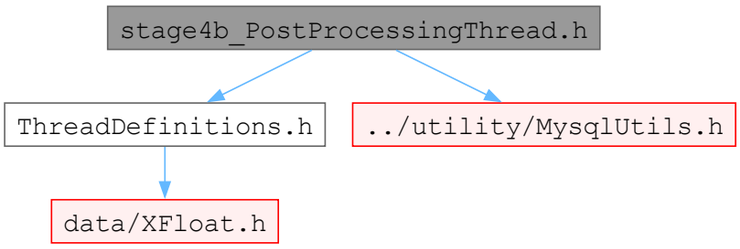
This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.179 stage4b_PostProcessingThread.h File Reference

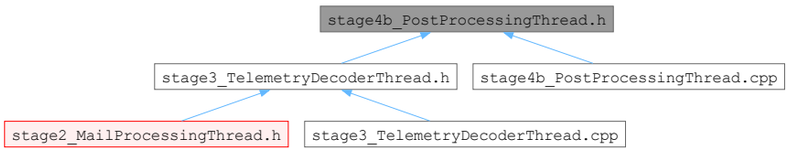
The argo::iridium::xfloat::XFloatPostProcessingService class header file.

```
#include "ThreadDefinitions.h"
#include "utility/MysqlUtils.h"
```

Include dependency graph for stage4b_PostProcessingThread.h:



This graph shows which files directly or indirectly include this file:



Classes

- class argo::iridium::xfloat::XFloatPostProcessingService

  The post-processing analysis thread class.

Namespaces

- namespace argo

  The Argo instrument namespace.
- namespace argo::iridium

  The Argo Iridium instrument namespace.
- namespace argo::iridium::xfloat

  The X-Float instrument namespace.

11.179.1 Detailed Description

The argo::iridium::xfloat::XFloatPostProcessingService class header file.

This class is defines the operation of the post-processing analysis thread(s), which are the fourth stage of the processing pipeline. The post-processing analysis thread(s) are responsible for generating plots, Matlab, and NetCDF files as well as performing some analysis of the decoded telemetry fields in order to derive performance metrics of the instrument.

Version

0.6

Date

Oct 16, 2014 2:21:06 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.180 stage4b_PostProcessingThread.h

Go to the documentation of this file.

```
00001 /** The @ref argo::iridium::xfloat::XFloatPostProcessingService class header file.
00002  *
00003  *  This class is defines the operation of the post-processing analysis thread(s), which are the
00004  *  fourth stage of the processing pipeline. The post-processing analysis thread(s) are
00005  *  responsible for generating plots, Matlab, and NetCDF files as well as performing some analysis
00006  *  of the decoded telemetry fields in order to derive performance metrics of the instrument.
00007  *
00008  *  @file
00009  *
00010  *  @version    0.6
00011  *
00012  *  @date       Oct 16, 2014 2:21:06 PM
00013  *
00014  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00015  *              Engineer I,
00016  *              Department of Physical Oceanography,
00017  *              Woods Hole Oceanographic Institution
00018  *
00019  *  @copyright  2014 Woods Hole Oceanographic Institution.
00020  *              Released under the GNU General Public License.
00021  *
00022  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00023  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00024  *  General Public License for more details. You should have received a copy of the GNU General
00025  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00026  */
00027
00028 #ifndef XFLOATPOSTPROCESSINGSERVICE_H_
00029 #define XFLOATPOSTPROCESSINGSERVICE_H_
00030
00031 /* Includes: */
00032
00033 // local
00034 #include "ThreadDefinitions.h"
00035 #include "utility/MysqlUtils.h"
00036
00037 namespace argo {
00038 namespace iridium {
00039 namespace xfloat {
00040
00041 /* Class Definition: */
00042
00043 /** The post-processing analysis thread class.
00044  *
00045  *  This class is defines the operation of the post-processing analysis thread(s), which are the
00046  *  fourth stage of the processing pipeline. The post-processing analysis thread(s) are
00047  *  responsible for generating plots, Matlab, and NetCDF files as well as performing some analysis
00048  *  of the decoded telemetry fields in order to derive performance metrics of the instrument.
00049  */
00050 class XFloatPostProcessingService:
00051         public util::kml::KmlUtil {
00052
00053     /* Friends Functions: */
00054
00055     /** Friend function for populating some of the static members of this class from the
00056      *  configuration file options.
00057      */
00058     friend options_description
00059     util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription();
00060
00061 private:
00062
00063     /* Static Data Members: */
00064
00065 #   if defined(XMTPS_POST_PROC_USES_QUEUE) && XMTPS_POST_PROC_USES_QUEUE
00066
00067     static util::thread::SynchronousQueue<shared_ptr<XFloat> > float_processing_queue_;
00068
00069 #   endif // defined(XMTPS_POST_PROC_USES_QUEUE) && XMTPS_POST_PROC_USES_QUEUE
00070
00071     /** Duration to sleep when not performing post-processing, in seconds.
00072      *
00073      *  The thread sleeps for this duration between periodically iterating over all cached
00074      *  instrument data objects and performing post-processing as needed.
00075      */
00076     static unsigned int idle_sleep_seconds_;
00077
00078     /** The post-processing analysis thread group.
```

```
00079      *
00080      *  This object represents the group of post-processing analysis threads that are instantiated after
00081      *  initialization. Each new thread that is instantiated is added to this group. Upon
00082      *  termination, the thread group is joined.
00083      */
00084     static thread thread_;
00085
00086     /** The post-processing analysis thread objects.
00087      *
00088      *  Each instance of the post-processing analysis threads is created by invoking the functor of an
00089      *  instance of this runnable class. Each instance of this object that is created during
00090      *  instantiation of a new thread is stored in this vector.
00091      */
00092     static shared_ptr<XFloatPostProcessingService > thread_object_;
00093
00094     /** The termination signaled flag.
00095      *
00096      *  This flag is used  to signal termination of the post-processing analysis thread. During
00097      *  normal operation, the value of this field will be FALSE. Once termination is signaled, it
00098      *  will become TRUE causing all of the single instance of the mail retrieval to to exit its
00099      *   primary loop and die.
00100      */
00101     static std::atomic<bool> termination_signaled_;
00102
00103 private:
00104
00105     /* Private Member Functions: */
00106
00107 #   if defined(XMTPS_POST_PROC_USES_QUEUE) && XMTPS_POST_PROC_USES_QUEUE
00108
00109     static bool FloatProcessingQueuePop(shared_ptr<XFloat>& flt,
00110     bool block = false) {
00111         return float_processing_queue_.pop(flt, block);
00112     }
00113
00114 #   endif // defined(XMTPS_POST_PROC_USES_QUEUE) && XMTPS_POST_PROC_USES_QUEUE
00115
00116     static void set_termination_signaled(bool termination_signaled = true) {
00117         termination_signaled_ = termination_signaled;
00118     }
00119
00120 public:
00121
00122     /* Public Member Functions: */
00123
00124     /** The constructor.
00125      *
00126      */
00127     XFloatPostProcessingService() {
00128
00129         /* Do nothing. */
00130
00131     }
00132
00133     /** The destructor.
00134      *
00135      */
00136     ~XFloatPostProcessingService() {
00137         /* Do nothing. */
00138     }
00139
00140 #   if defined(XMTPS_POST_PROC_USES_QUEUE) && XMTPS_POST_PROC_USES_QUEUE
00141
00142     static bool FloatProcessingQueuePush(const shared_ptr<XFloat>& flt, bool block = true) {
00143         return float_processing_queue_.push(flt, block);
00144     }
00145
00146 #   endif // defined(XMTPS_POST_PROC_USES_QUEUE) && XMTPS_POST_PROC_USES_QUEUE
00147
00148     /** Functor operator overload function: the main entry point for the post-processing analysis
00149      *  thread.
00150      *
00151      *  This function defines the behavior of the post-processing analysis thread which is
00152      *  responsible for generating tertiary files (e.g. Google Earth KML and status files).
00153      *
00154      *  During execution the primary loop consists of the following. First, sleep for
00155      *  @ref idle_sleep_seconds_. Then, iterate over all of the cached instrument objects and
00156      *  update files as needed. This loop continues until termination is signaled by the
00157      *  daemon/service thread.
00158      *
00159      *  Once termination is signaled by the daemon process i.e. @ref termination_signaled_ is
00160      *  TRUE, the thread will exit its primary loop and die.
00161      *
00162      *  @exception      None. Never throws.
00163      */
00164     void operator ()() ATTR_NO_EXCEPTIONS {
00165
00166         namespace attr = blg::attributes;
00167
00168         bool update_master;
00169         vector<shared_ptr<XFloat> > insts;
00170         shared_ptr<XFloat> inst;
00171         unsigned int sleep_ms;
00172
00173         BOOST_LOG_SCOPED_THREAD_ATTR("ThreadName", attr::constant<string>("post-process"));
00174         ldbg() << "Thread instantiated.";
00175
00176         // Enter the primary thread loop.
00177         do { // } while(!termination_signaled_);
00178
00179             try {
00180
00181 #               if defined(XMTPS_POST_PROC_USES_QUEUE) && XMTPS_POST_PROC_USES_QUEUE
00182
00183                 /* Read instrument objects from queue: */
00184
00185                 // Pop message from queue. Do not block.
00186                 if(FloatProcessingQueuePop(inst, false)) {
```

```
00187
00188                     do {
00189
00190                         // Write KML file.
00191                         inst->WriteKmlFile();
00192
00193                         // Write the status file.
00194                         inst->WriteStatusFile();
00195
00196                         // Reset the instrument pointer.
00197                         inst.reset();
00198
00199                     } while(FloatProcessingQueuePop(inst, false));
00200
00201                     // Update the master KML file.
00202                     XFloat::WriteKmlMasterFile();
00203
00204                 }
00205
00206                 // After all instruments are processed, sleep.
00207                 sleep(millisec(XFLT_THREAD_SLEEP_MS));
00208
00209 #           else
00210
00211                 /* Iterate over instrument objects and update as needed: */
00212
00213                 // Sleep unless terminated.
00214                 for(sleep_ms = 0; sleep_ms < idle_sleep_seconds_ * 1000 && !termination_signaled_;
00215                         sleep_ms += XFLT_THREAD_SLEEP_MS)
00216
00217                     sleep(millisec(XFLT_THREAD_SLEEP_MS));
00218
00219                 // Iterate over instrument objects.
00220                 update_master = false;
00221                 insts = XFloat::GetAllCachedFloats();
00222                 for(auto float_it = insts.begin();
00223                         float_it < insts.end() && !termination_signaled_; ++float_it) {
00224
00225                     if(!(inst = *float_it))
00226                         continue;
00227
00228                     // Set scoped thread log attributes.
00229                     BOOST_LOG_SCOPED_THREAD_ATTR("WhoiID", attr::constant<int>(inst->id_instrument()));
00230                     BOOST_LOG_SCOPED_THREAD_ATTR("DacID", attr::constant<int>(inst->id_dac()));
00231                     BOOST_LOG_SCOPED_THREAD_ATTR("WmoID", attr::constant<int>(inst->id_wmo()));
00232
00233 #                   if defined(XMTPS_DEBUG_POST_PROC_THREAD) \
00234                         && XMTPS_DEBUG_NC_FILE_WRITER_THREAD
00235
00236                     ldbg() « "Performing post-processing tasks for instrument #"
00237                             « inst->id_instrument() « "...";
00238
00239 #                   endif // defined(XMTPS_DEBUG_POST_PROC_THREAD) ...
00240
00241                     /* Catch per-float failures so one bad float does not skip the remaining
00242                      * floats, the master KML update, or closing the SQL connection. */
00243                     try {
00244
00245                         // Write KML file.
00246                         if(inst->WriteKmlFile())
00247                             update_master = true;
00248
00249                     }CATCH_NOTHROW("Failed to write KML file for instrument!")
00250
00251                     try {
00252
00253                         // Write the status file.
00254                         inst->WriteStatusFile();
00255
00256                     }CATCH_NOTHROW("Failed to write status file for instrument!")
00257
00258                     inst.reset();
00259                 }
00260                 insts.clear();
00261
00262                 if(update_master) {
00263
00264                     try {
00265
00266                         // Update the master KML file.
00267                         XFloat::WriteKmlMasterFile();
00268
00269                     }CATCH_NOTHROW("Failed to write KML master file!")
00270
00271                 }
00272
00273
00274                 // Close SQL connection when not in use.
00275                 util::mysql::MysqlUtils::CloseSqlConnection();
00276
00277 #           endif // defined(XMTPS_POST_PROC_USES_QUEUE) && XMTPS_POST_PROC_USES_QUEUE
00278
00279             }CATCH_NOTHROW("Exception caught in post-processing analysis thread!")
00280
00281         } while(!termination_signaled_);
00282
00283         // Terminate.
00284         ldbg() « "Termination signal received! Shutting down...";
00285
00286         return;
00287     }
00288
00289     /** Instantiate the post-processing analysis worker thread.
00290      *
00291      *  This function is invoked by the daemon thread after initialization to instantiate the
00292      *  post-processing analysis thread. It creates a new instance of
00293      *  @ref XFloatPostProcessingService, and instantiates a new thread that invokes the functor
00294      *  operator of the object.
```

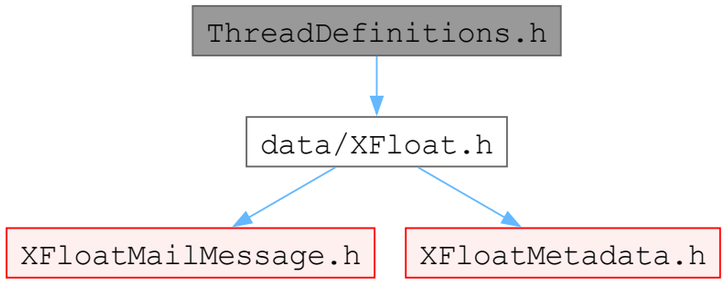
```
00295      *
00296      *  @exception          None. Never throws.
00297      */
00298     static void StartWorkerThread() ATTR_NO_EXCEPTIONS {
00299
00300         try {
00301
00302             ldbg() « "Starting post-processing analysis thread...";
00303             thread_object_ = make_shared<XFloatPostProcessingService>();
00304             thread_ = thread(*thread_object_);
00305
00306         }CATCH_BLOCK("Failed to start post-processing analysis worker thread!")
00307     }
00308
00309     /** Terminate the mail post-processing analysis thread.
00310      *
00311      *  This function is used by the daemon thread to kill the worker thread upon application
00312      *  termination. The function signals termination, and then attempts to join the child thread.
00313      *  If the child thread does not join, then the function will timeout after @ref
00314      *  XMTPS_THREAD_JOIN_SECONDS seconds, a warning will be issued, and the child thread will be
00315      *  abandoned.
00316      *
00317      *  @exception          None. Never throws.
00318      */
00319     static void TerminateWorkerThread() ATTR_NO_EXCEPTIONS {
00320
00321         try {
00322
00323             ldbg() « "Terminating post-processing analysis thread...";
00324             set_termination_signaled();
00325
00326             if(thread_.joinable()
00327                     && !thread_.try_join_for(boost::chrono::seconds(XMTPS_THREAD_JOIN_SECONDS)))
00328
00329                 lwrn() « "Failed to join post-processing analysis thread after waiting "
00330                        « XMTPS_THREAD_JOIN_SECONDS
00331                        « " seconds!";
00332             else
00333
00334                 ldbg() « "Thread terminated.";
00335
00336         }CATCH_BLOCK("Failed to terminate post-processing analysis worker thread!")
00337
00338     }
00339 };
00340
00341 } /* namespace xfloat */
00342 } /* namespace iridium */
00343 } /* namespace argo */
00344
00345 #endif /* XFLOATPOSTPROCESSINGSERVICE_H_ */
```

11.181 ThreadDefinitions.h File Reference

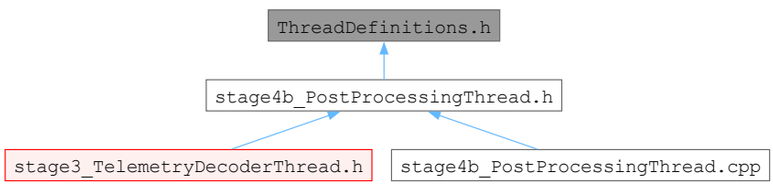
The thread definitions header file.

`#include "data/XFloat.h"`

Include dependency graph for ThreadDefinitions.h:



This graph shows which files directly or indirectly include this file:



Macros

- #define XMTPS_DEBUG_FETCH_MAIL (false)

  The debug-fetch-mail flag.
- #define **XMTPS_DEBUG_NC_FILE_WRITER_THREAD** (false)
- #define **XMTPS_DEBUG_POST_PROC_THREAD** (true)
- #define **XMTPS_NC_FILE_WRITER_USES_QUEUE** (false)
- #define **XMTPS_POST_PROC_USES_QUEUE** (false)
- #define XMTPS_THREAD_JOIN_SECONDS 120

  The maximum duration to allow for threads to join upon termination, in seconds.

11.181.1 Detailed Description

The thread definitions header file.

Version

0.6

Date

Dec 28, 2016 9:33:48 AM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.181.2 Macro Definition Documentation

11.181.2.1 XMTPS_DEBUG_FETCH_MAIL

```
#define XMTPS_DEBUG_FETCH_MAIL (false)
```

The debug-fetch-mail flag.

If TRUE, then additional debug messages will be printed during the mail fetch phase of thread operation.

11.181.2.2 XMTPS_THREAD_JOIN_SECONDS

```
#define XMTPS_THREAD_JOIN_SECONDS 120
```

The maximum duration to allow for threads to join upon termination, in seconds.

Referenced by argo::iridium::xfloat::XFloatMailRetrievalService::TerminateWorkerThread(), and argo::iridium::xfloat::XFloatPostProcessing

11.182 ThreadDefinitions.h

Go to the documentation of this file.

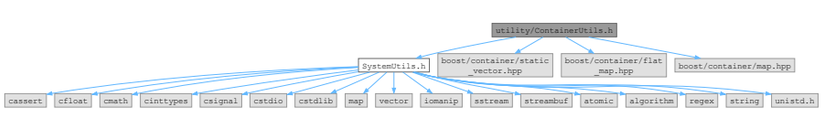
```
00001 /** The thread definitions header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Dec 28, 2016 9:33:48 AM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef THREADDEFINITIONS_H_
00024 #define THREADDEFINITIONS_H_
00025
00026 /* Includes: */
00027
00028 // local
00029 #include "data/XFloat.h"
00030
00031 /* Definitions: */
00032
00033 #if !defined(XMTPS_DEBUG_FETCH_MAIL)
00034
00035     /** The debug-fetch-mail flag.
00036      *
00037      *  If TRUE, then additional debug messages will be printed during the mail fetch phase of
00038      *  thread operation.
00039      */
00040 #   define XMTPS_DEBUG_FETCH_MAIL                   (false)
00041
00042 #endif // !defined(XMTPS_DEBUG_FETCH_MAIL)
00043
00044 #if !defined(XMTPS_DEBUG_NC_FILE_WRITER_THREAD)
00045
00046 #   define XMTPS_DEBUG_NC_FILE_WRITER_THREAD        (false)
00047
00048 #endif // !defined(XMTPS_DEBUG_NC_FILE_WRITER_THREAD)
00049
00050 #if !defined(XMTPS_DEBUG_POST_PROC_THREAD)
00051
00052 #   define XMTPS_DEBUG_POST_PROC_THREAD             (true)
00053
00054 #endif // !defined(XMTPS_DEBUG_NC_FILE_WRITER_THREAD)
00055
00056 #if !defined(XMTPS_NC_FILE_WRITER_USES_QUEUE)
00057
00058 #   define XMTPS_NC_FILE_WRITER_USES_QUEUE          (false)
00059
00060 #endif // !defined(XMTPS_NC_FILE_WRITER_USES_QUEUE)
00061
00062 #if !defined(XMTPS_POST_PROC_USES_QUEUE)
00063
00064 #   define XMTPS_POST_PROC_USES_QUEUE               (false)
00065
00066 #endif // !defined(XMTPS_POST_PROC_USES_QUEUE)
00067
00068 #if !defined(XMTPS_THREAD_JOIN_SECONDS) \
00069     || (defined(XMTPS_THREAD_JOIN_SECONDS) && XMTPS_THREAD_JOIN_SECONDS <= 0)
00070
00071     /** The maximum duration to allow for threads to join upon termination, in seconds.
00072      *
00073      */
00074 #   define XMTPS_THREAD_JOIN_SECONDS                120
00075
00076 #endif // !defined(XMTPS_THREAD_JOIN_SECONDS) ...
00077
00078 #endif /* THREADDEFINITIONS_H_ */
```

11.183 utility/ContainerUtils.h File Reference

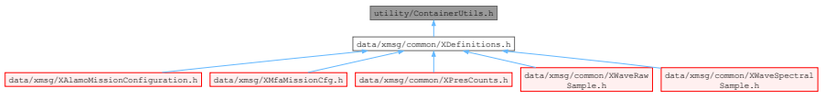
The ContainerUtils class header file.

```
#include "SystemUtils.h"
#include <boost/container/static_vector.hpp>
#include <boost/container/flat_map.hpp>
#include <boost/container/map.hpp>
```

Include dependency graph for ContainerUtils.h:



This graph shows which files directly or indirectly include this file:



11.183.1 Detailed Description

The ContainerUtils class header file.

Version

0.6

Date

Oct 18, 2014 12:36:48 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.184 ContainerUtils.h

Go to the documentation of this file.

```
00001 /** The ContainerUtils class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Oct 18, 2014 12:36:48 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef CONTAINERUTILS_H_
00024 #define CONTAINERUTILS_H_
00025
00026 /* Includes */
00027
00028 // local
00029 #include "SystemUtils.h"
00030
00031 // boost
00032 #include <boost/container/static_vector.hpp>
00033 #include <boost/container/flat_map.hpp>
00034 #include <boost/container/map.hpp>
00035 namespace bc = boost::container;
00036
00037
00038
00039 /* Definitions */
00040
00041 #endif /* CONTAINERUTILS_H_ */
```

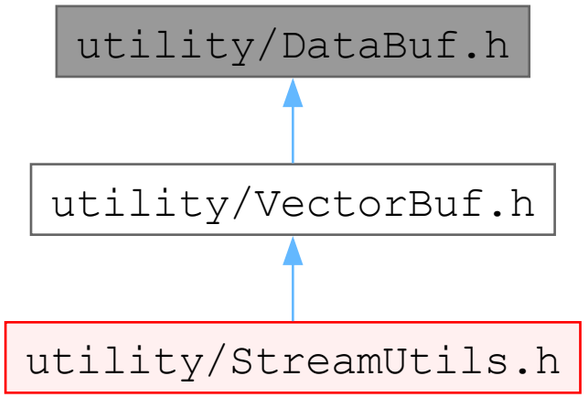
11.185 utility/DataBuf.h File Reference

The util::stream::DataBuf class header file.

```
#include "SystemUtils.h"
#include <boost/filesystem/fstream.hpp>
#include <boost/iostreams/device/file_descriptor.hpp>
#include <boost/iostreams/stream.hpp>
#include <boost/iostreams/stream_buffer.hpp>
```

Include dependency graph for DataBuf.h:

This graph shows which files directly or indirectly include this file:



Classes

- class util::stream::DataBuf< CharT, TraitsT >

  Template class for wrapping an arbitrary contiguous section of data as a std::basic_streambuf object.

Namespaces

- namespace util

  The utility namespace.
- namespace util::stream

  The stream utility namespace.

11.185.1 Detailed Description

The util::stream::DataBuf class header file.

Version

0.6

Date

Oct 18, 2014 12:34:52 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.186 DataBuf.h

Go to the documentation of this file.

```
00001 /** The @ref util::stream::DataBuf class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Oct 18, 2014 12:34:52 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef DATABUF_H_
00024 #define DATABUF_H_
00025
00026 /* Includes */
00027
00028 // local
00029 #include "SystemUtils.h"
00030
00031 // boost
00032 #include <boost/filesystem/fstream.hpp>
00033 #include <boost/iostreams/device/file_descriptor.hpp>
00034 #include <boost/iostreams/stream.hpp>
00035 #include <boost/iostreams/stream_buffer.hpp>
00036 namespace bis = boost::iostreams;
00037
00038 namespace util {
00039 namespace stream {
00040
00041 using std::char_traits;
00042 using std::basic_streambuf;
00043
00044 /* Class Definition */
00045
00046 /** Template class for wrapping an arbitrary contiguous section of data as a std::basic_streambuf
00047  *  object.
00048  *
00049  */
00050 template<typename CharT, typename TraitsT = char_traits<CharT> >
00051 class DataBuf: public basic_streambuf<CharT, TraitsT> {
00052
00053 public:
00054
00055     /* Member Functions */
00056
00057     /** Parameterized constructor.
00058      *
00059      *  @param[in]      data        A pointer to the start of the memory section of interest.
00060      *  @param[in]      size        The size of the memory section, in bytes.
00061      */
00062     DataBuf(const CharT* data, size_t size) {
00063         this->setg(const_cast<CharT*>(data), const_cast<CharT*>(data),
00064                    const_cast<CharT*>(data + size));
00065     }
00066 };
00067
00068 } /* namespace stream */
00069 } /* namespace util */
00070 #endif /* DATABUF_H_ */
```

11.187 utility/DateUtils.h File Reference

The util::date::DateUtils class header file.

```
#include "Log.h"
#include <boost/date_time.hpp>
#include <boost/date_time/gregorian/gregorian.hpp>
#include <boost/chrono.hpp>
```

Include dependency graph for DateUtils.h:

This graph shows which files directly or indirectly include this file:

Classes

- class util::date::DateUtils

  Class providing library of useful date and time functions.

Namespaces

- namespace util

  The utility namespace.
- namespace util::date

  Date utility namespace.

11.187.1 Detailed Description

The util::date::DateUtils class header file.

Version

0.6

Date

Oct 18, 2014 1:20:01 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.188 DateUtils.h

Go to the documentation of this file.

```
00001 /** The @ref util::date::DateUtils class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Oct 18, 2014 1:20:01 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef DATEUTILITIES_H_
00024 #define DATEUTILITIES_H_
00025
00026 /* Includes */
00027
00028 // local
00029 #include "Log.h"
00030
00031 // boost
00032 #include <boost/date_time.hpp>
00033 #include <boost/date_time/gregorian/gregorian.hpp>
00034 #include <boost/chrono.hpp>
00035 namespace bpt = boost::posix_time;
00036 namespace bgt = boost::gregorian;
00037
00038 namespace util {
00039 namespace date {
00040
00041 using std::ostringstream;
00042 using std::string;
00043 using boost::posix_time::not_a_date_time;
00044 using boost::posix_time::ptime;
00045 using boost::posix_time::second_clock;
00046 using boost::posix_time::time_duration;
00047 using boost::posix_time::to_iso_extended_string;
00048 using boost::posix_time::to_iso_string;
00049
00050 /* Class Definitions */
00051
00052 /** Class providing library of useful date and time functions.
00053  *
00054  */
00055 class DateUtils {
00056 public:
00057
00058     DateUtils() {
00059       /* Do nothing. */
00060     }
00061
00062     virtual ~DateUtils() {
00063       /* Do nothing. */
00064     }
00065
00066     /* Member Functions */
00067
00068     /** Convert a date/time to fractional days since a reference date/time.
00069      *
00070      *  @param[in]  dt          The date/time to convert.
00071      *  @param[in]  ref_dt      The reference date/time (day 0).
00072      *
00073      *  @throw      util::log::MessageException if either date/time is not valid.
00074      *
00075      *  @return     The number of days (with millisecond resolution) from @p ref_dt to @p dt.
00076      */
00077     static double GenerateJulianDay(const ptime& dt, const ptime& ref_dt) {
00078
00079         double juld = 0;
00080         long td_msec = 0;
00081
00082         try {
00083
00084             if(dt.is_not_a_date_time())
00085                 EXC_THROW("Invalid date/time!");
00086             if(ref_dt.is_not_a_date_time())
00087                 EXC_THROW("Invalid reference date/time!");
00088
00089             time_duration td = dt - ref_dt;
00090             td_msec = td.total_milliseconds();
00091             juld = (double)td_msec / (double) 1000 / (double) 3600 / (double) 24;
00092
00093         }CATCH_BLOCK("Failed to convert ptime to Julian day!")
00094
00095         return juld;
00096
00097     }
00098
00099     /** Generate an ISO8601 format date/time string representation of a POSIX time object.
00100      *
00101      *  @note           All date/time objects use the UTC time-zone.
00102      *
00103      *  @param[in]      p           POSIX time from which to generate string.
```

```
00104      *  @param[in]      extended    If TRUE, use the ISO 8601 extended format
00105      *                              (YYYY-MM-DDThh:mm:ssZ); otherwise, use the basic format
00106      *                              (YYYYMMDDThhmmss).
00107      *
00108      *  @return         An ISO8601 format date/time string representation of the POSIX time object. If
00109      *                  p is equal to boost::posix_time::not_a_date_time, then the string
00110      *                  "0000-00-00T00:00:00Z" (extended) or "00000000000000" (basic) will be
00111      *                  returned.
00112      */
00113     static string GenerateIso8601String(const ptime& p, bool extended = true) {
00114         static const string not_a_datetime_string_extended = "0000-00-00T00:00:00Z";
00115         static const string not_a_datetime_string = "00000000000000";
00116
00117         ostringstream oss;
00118
00119         if(extended) {
00120             if(p.is_not_a_date_time())
00121                 oss « not_a_datetime_string_extended;
00122             else
00123                 oss « to_iso_extended_string(p) « 'Z';
00124         } else {
00125             if(p.is_not_a_date_time())
00126                 oss « not_a_datetime_string;
00127             else
00128                 oss « to_iso_string(p);
00129         }
00130
00131         return oss.str();
00132     }
00133
00134     /** Generate an ISO8601 format date/time string representation of the current time, in UTC.
00135      *
00136      *  @note           All date/time objects use the UTC time-zone.
00137      *
00138      *  @return         An ISO8601 format date/time string representation of the current time.
00139      */
00140     static string GenerateCurrentUtcTimeIso8601String() {
00141         return GenerateIso8601String(GetCurrentUtcTime());
00142     }
00143
00144     /** Generate a POSIX time object representing the current date and time (rounded to the nearest
00145      *  second).
00146      *
00147      *  @return         A POSIX time object representing the current date and time (rounded to the
00148      *                  nearest second).
00149      */
00150     static ptime GetCurrentUtcTime() {
00151         return second_clock::universal_time();
00152     }
00153 };
00154
00155 } /* namespace date */
00156 } /* namespace util */
00157 #endif /* DATEUTILITIES_H_ */
```

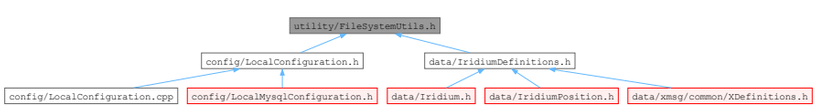
11.189 utility/FileSystemUtils.h File Reference

The util::file::FileSystemUtils class header file.

```
#include "Log.h"
#include "StreamUtils.h"
```

Include dependency graph for FileSystemUtils.h:

This graph shows which files directly or indirectly include this file:



Classes

- class util::file::FileSystemUtils

  Class providing library of useful filesystem functions.

Namespaces

- namespace util

  The utility namespace.
- namespace util::file

  The filesystem utility namespace.

11.189.1 Detailed Description

The util::file::FileSystemUtils class header file.

Version

0.6

Date

Oct 18, 2014 12:47:22 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.190 FileSystemUtils.h

Go to the documentation of this file.

```
00001 /** The @ref util::file::FileSystemUtils class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Oct 18, 2014 12:47:22 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef FILESYSTEMUTILITY_H_
00024 #define FILESYSTEMUTILITY_H_
00025
00026 /* Includes */
00027
00028 // local
00029 #include "Log.h"
00030 #include "StreamUtils.h"
00031
00032 namespace util {
00033 namespace file {
00034
00035 using boost::filesystem::create_directories;
00036 using boost::filesystem::directory_entry;
00037 using boost::filesystem::directory_iterator;
00038 using boost::filesystem::exists;
00039 using boost::filesystem::ifstream;
00040 using boost::filesystem::is_directory;
00041 using boost::filesystem::is_regular_file;
00042 using boost::filesystem::ofstream;
00043 using boost::filesystem::path;
00044 using std::string;
00045 using std::vector;
00046
00047 /* Class Definition */
00048
00049 /** Class providing library of useful filesystem functions.
00050  *
00051  */
00052 class FileSystemUtils {
00053
00054 public:
00055
00056     FileSystemUtils() {
```

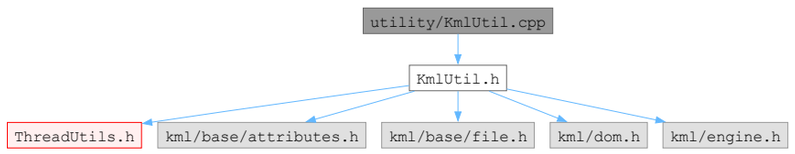
```
00057       /* Do nothing. */
00058     }
00059
00060     virtual ~FileSystemUtils() {
00061       /* Do nothing. */
00062     }
00063
00064     /* Member Functions */
00065
00066     /** Get a directory path.
00067      *
00068      *  @throws         MessageException if create == FALSE and the directory specified by path does
00069      *                  not exist.
00070      *
00071      *  @param[in]      dir             Directory path.
00072      *  @param[in]      create          If TRUE, the directory specified by path will be created if it
00073      *                                  does not already exists. If FALSE, an exception will be thrown
00074      *                                  if the directory does not exist.
00075      *
00076      *  @return         dir
00077      */
00078     static const path& GetDirectoryPath(const path& dir, bool create = false) {
00079
00080         try {
00081
00082             if(dir.empty())
00083                 EXC_THROW("Empty directory path!");
00084
00085             if(!is_directory(dir)) {
00086
00087                 if(exists(dir))
00088                     EXC_THROW("Invalid path to directory!");
00089
00090                 if(create) {
00091
00092                     try { // Create the directory.
00093
00094                         ldbg() « "Creating directory: " « dir « "...";
00095                         create_directories(dir);
00096
00097                     }CATCH_BLOCK("Failed to create directory!")
00098
00099                 } else {
00100
00101                     EXC_THROW("Directory does not exist!");
00102
00103                 }
00104
00105             }
00106
00107         }CATCH_BLOCK("Failed to get directory path" + dir.string() + "!")
00108
00109         return dir;
00110     }
00111
00112     /** Generate a boost::filesystem::path object from the directory path string.
00113      *
00114      *  @throws         MessageException if create == FALSE and the directory specified by path does
00115      *                  not exist.
00116      *
00117      *  @param[in]      dir             Directory path string.
00118      *  @param[in]      create          If TRUE, the directory specified by path will be created if it
00119      *                                  does not already exists. If FALSE, an exception will be thrown
00120      *                                  if the directory does not exist.
00121      *
00122      *  @return         A boost::filesystem::path object from the directory path string.
00123      */
00124     static path GetDirectoryPathFromString(const string& dir, bool create = false) {
00125
00126         path p;
00127
00128         try {
00129
00130             p = GetDirectoryPath(path(dir), create);
00131
00132         }CATCH_BLOCK("Failed to get path from directory string!")
00133
00134         return p;
00135     }
00136
00137     /** List all regular files contained in the given directory.
00138      *
00139      *  @param[in]      dir     The path to the directory of interest.
00140      *  @param[in]      n       If n>0, the maximum number of paths to return, else no effect.
00141      *
00142      *  @throw          log::MessageException if the directory does not exist, is not a directory, or
00143      *                  an error occurs.
00144      *
00145      *  @return         A vector of paths to all the regular files contained in the given directory.
00146      */
00147     static vector<path> ListFiles(path dir, size_t n = 0) {
00148         vector<path> files;
00149
00150         try {
00151             if(!exists(dir))
00152                 EXC_THROW("Directory does not exist: " + dir.string() + "!");
00153             if(!is_directory(dir))
00154                 EXC_THROW("Path is not a valid directory: " + dir.string() + "!");
00155
00156             copy_if(
00157                     directory_iterator(dir),
00158                     directory_iterator(),
00159                     back_inserter(files),
00160                     [](directory_entry & d) {
00161                         return is_regular_file(d.path());
00162                     });
00163
00164             if(n > 0)
```

```
00165               files.resize(n);
00166
00167           }CATCH_BLOCK("Failed to list files for directory!")
00168
00169           return files;
00170       }
00171
00172       /** List all directories contained in the given directory.
00173        *
00174        *  @param[in]      dir     The path to the directory of interest.
00175        *
00176        *  @throw          log::MessageException if the directory does not exist, is not a directory, or
00177        *                  an error occurs.
00178        *
00179        *  @return         A vector of paths to all the directories contained in the given directory.
00180        */
00181       static vector<path> ListDirectories(const path& dir) {
00182
00183           vector<path> dirs;
00184
00185           try {
00186               if(!exists(dir))
00187                   EXC_THROW("Directory does not exist: " + dir.string() + "!");
00188               if(!is_directory(dir))
00189                   EXC_THROW("Path is not a valid directory: " + dir.string() + "!");
00190
00191               copy_if(
00192                       directory_iterator(dir),
00193                       directory_iterator(),
00194                       back_inserter(dirs),
00195                       [](directory_entry & d) {
00196                           return is_directory(d.path());
00197                       });
00198
00199           }CATCH_BLOCK("Failed to list directories!")
00200
00201           return dirs;
00202       }
00203
00204       /** Copy a file into a directory.
00205        *
00206        *  @warning    Despite its name, this function copies the file and does not remove the source.
00207        *              If a file with the same name already exists in the destination directory, it is
00208        *              left unchanged. It is not currently called.
00209        *
00210        *  @param[in]  file        The file to copy.
00211        *  @param[in]  dir_dst     The destination directory.
00212        *  @param[in]  create_dir  If TRUE, create the destination directory if it does not exist.
00213        *
00214        *  @throw      util::log::MessageException if the file does not exist, is not a regular file,
00215        *              or cannot be copied.
00216        *
00217        *  @return     The path of the file in the destination directory.
00218        */
00219       static path MoveFileToDirectory(const path& file, const path& dir_dst, bool create_dir = false) {
00220
00221           path file_dst;
00222
00223           try{
00224
00225               if(!exists(file))
00226                   EXC_THROW("File not found!");
00227
00228               if(!is_regular_file(file))
00229                   EXC_THROW("Invalid file path!");
00230
00231               GetDirectoryPath(dir_dst, create_dir);
00232
00233               file_dst = dir_dst;
00234               file_dst /= file.filename();
00235
00236               if(!exists(file_dst)) {
00237 //                    copy_file(list_file, list_file_dest);
00238
00239                   ifstream src(file, std::ios::binary);
00240                   ofstream dst(file_dst, std::ios::binary);
00241
00242                   dst << src.rdbuf();
00243
00244               }
00245
00246           } CATCH_BLOCK("Failed to move file: " + file.string() + "!")
00247
00248
00249           return file_dst;
00250
00251       }
00252
00253
00254 };
00255
00256 } /* namespace file */
00257 } /* namespace util */
00258 #endif /* FILESYSTEMUTILITY_H_ */
```

11.191 utility/KmlUtil.cpp File Reference

The util::kml::KmlUtil class source file.

```
#include "KmlUtil.h"
```
Include dependency graph for KmlUtil.cpp:



Namespaces

- namespace util

  The utility namespace.

11.191.1 Detailed Description

The util::kml::KmlUtil class source file.

Version

0.6

Date

Aug 8, 2015 5:44:39 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

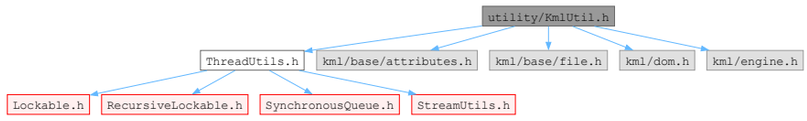
2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

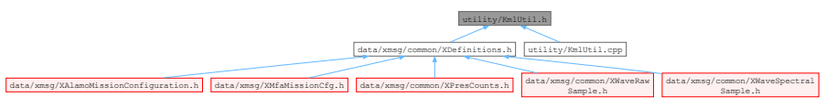
11.192 utility/KmlUtil.h File Reference

The util::kml::KmlUtil class header file.

```
#include "ThreadUtils.h"
#include <kml/base/attributes.h>
#include <kml/base/file.h>
#include <kml/dom.h>
#include <kml/engine.h>
```
Include dependency graph for KmlUtil.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::kml::KmlUtil

  The KML utility class.

Namespaces

- namespace util

  The utility namespace.

Macros

- #define **KML_NUM_DFLT_PATH_COLORS** (13)

11.192.1 Detailed Description

The util::kml::KmlUtil class header file.

Version

0.6

Date

Aug 8, 2015 5:44:39 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.193 KmlUtil.h

Go to the documentation of this file.

```
00001 /** The @ref util::kml::KmlUtil class header  file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Aug 8, 2015 5:44:39 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef KMLUTIL_H_
00024 #define KMLUTIL_H_
00025
00026 /* Includes: */
00027
00028 // local
00029 #include "ThreadUtils.h"
```

```
00030
00031 // KML
00032 #include <kml/base/attributes.h>
00033 #include <kml/base/file.h>
00034 #include <kml/dom.h>
00035 #include <kml/engine.h>
00036
00037 /* Definitions: */
00038
00039 #define KML_NUM_DFLT_PATH_COLORS    (13)
00040
00041 namespace util {
00042 namespace kml {
00043
00044 using boost::thread_specific_ptr;
00045 using boost::lexical_cast;
00046 using kmlbase::Color32;
00047 using kmldom::CoordinatesPtr;
00048 using kmldom::DocumentPtr;
00049 using kmldom::HotSpotPtr;
00050 using kmldom::IconStyleIconPtr;
00051 using kmldom::IconStylePtr;
00052 using kmldom::KmlPtr;
00053 using kmldom::KmlFactory;
00054 using kmldom::LabelStylePtr;
00055 using kmldom::LineStylePtr;
00056 using kmldom::PlacemarkPtr;
00057 using kmldom::PointPtr;
00058 using kmldom::StylePtr;
00059 using kmldom::UNITS_FRACTION;
00060
00061
00062 /** The KML utility class.
00063  *
00064  *  This class provides some useful functions for creating Google Earth KML files.
00065  *
00066  */
00067 class KmlUtil {
00068
00069 private:
00070
00071     /* Private Data Members: */
00072
00073
00074     /** The colour palette for instrument paths and placemarks, indexed by style index. */
00075     static const Color32 kml_default_path_colors_[KML_NUM_DFLT_PATH_COLORS];
00076
00077     /** The KML style IDs of the path (line) styles, one per palette colour. */
00078     static const string kml_default_path_style_ids_[KML_NUM_DFLT_PATH_COLORS];
00079
00080     /** The KML style IDs of the large placemark styles, one per palette colour. */
00081     static const string kml_default_placemark_style_ids_[KML_NUM_DFLT_PATH_COLORS];
00082
00083     /** The KML style IDs of the small placemark styles, one per palette colour. */
00084     static const string kml_small_placemark_style_ids_[KML_NUM_DFLT_PATH_COLORS];
00085
00086     /** The KML style IDs of the triangle placemark styles, one per palette colour. */
00087     static const string kml_triangle_placemark_style_ids_[KML_NUM_DFLT_PATH_COLORS];
00088
00089     /** The libkml factory used to create KML elements, held per thread.
00090      *
00091      *  @warning    This holds libkml's process-wide factory (@c KmlFactory::GetFactory()), and the
00092      *              default cleanup deletes it when the owning thread exits or
00093      *              @ref CloseKmlFactory() is called. Only the post-processing thread uses KML, so
00094      *              this currently happens only at shutdown.
00095      */
00096     static thread_specific_ptr<KmlFactory> kml_factory_;
00097
00098 public:
00099
00100     KmlUtil() {
00101       /* Do nothing. */
00102     }
00103
00104     virtual ~KmlUtil() {
00105       /* Do nothing. */
00106     }
00107
00108
00109     /* Protected Member Functions: */
00110
00111     /** Get the libkml factory, initializing it for this thread if necessary.
00112      *
00113      *  @throw      util::log::MessageException if the factory cannot be obtained.
00114      *
00115      *  @return     The thread's KML factory pointer (see @ref kml_factory_).
00116      */
00117     static thread_specific_ptr<KmlFactory>& GetKmlFactory() {
00118
00119         try {
00120
00121             // Create a new KML factory object, if it does not exist.
00122             if(!kml_factory_.get())
00123                 kml_factory_.reset(KmlFactory::GetFactory());
00124
00125         }CATCH_BLOCK("Failed to get KmlFactory object!")
00126
00127         return kml_factory_;
00128     }
00129
00130     /** Add the four default styles for one palette colour to a KML document.
00131      *
00132      *  Adds a 5-pixel path (line) style, a large dot placemark style, a small dot placemark style
00133      *  and a triangle placemark style, all in the colour @p style_idx (modulo the palette size).
00134      *  The icons are loaded from @c http://argo.whoi.edu/kml/img/. Errors are logged, not thrown.
00135      *
00136      *  @param[in]  document    The KML document to add the styles to.
00137      *  @param[in]  style_idx   The palette index of the colour.
```

```
00138      */
00139     static void SetDefaultKmlStyle(const DocumentPtr& document, unsigned int style_idx) {
00140
00141         try {
00142
00143             { /* Create Path Style: */
00144
00145                 // Create line style.
00146                 LineStylePtr line_style = GetKmlFactory()->CreateLineStyle();
00147                 line_style->set_color(GetDefaltKmlPathColor(style_idx));
00148                 line_style->set_width(5);
00149
00150                 // Create style
00151                 StylePtr style = GetKmlFactory()->CreateStyle();
00152                 style->set_id(GetDefaltKmlPathStyleId(style_idx));
00153                 style->set_linestyle(line_style);
00154
00155                 // Add style to document.
00156                 document->add_styleselector(style);
00157             }
00158
00159             { /* Create Icon Style: */
00160
00161                 // Create icon.
00162                 IconStyleIconPtr icon = GetKmlFactory()->CreateIconStyleIcon();
00163                 icon->set_href("http://argo.whoi.edu/kml/img/dot2.png");
00164
00165                 // Create hotspot.
00166                 HotSpotPtr hot_spot = GetKmlFactory()->CreateHotSpot();
00167                 hot_spot->set_x(0.5);
00168                 hot_spot->set_xunits(UNITS_FRACTION);
00169                 hot_spot->set_y(0.5);
00170                 hot_spot->set_yunits(UNITS_FRACTION);
00171
00172                 // Create icon style.
00173                 IconStylePtr icon_style = GetKmlFactory()->CreateIconStyle();
00174                 icon_style->set_color(GetDefaltKmlPathColor(style_idx));
00175                 icon_style->set_icon(icon);
00176                 icon_style->set_scale(1.5);
00177                 icon_style->set_hotspot(hot_spot);
00178
00179                 // Create label style;
00180                 LabelStylePtr label_style = GetKmlFactory()->CreateLabelStyle();
00181                 label_style->set_scale(1.5);
00182
00183                 // Create style.
00184                 StylePtr style = GetKmlFactory()->CreateStyle();
00185                 style->set_id(GetDefaltKmlPlacemarkStyleId(style_idx));
00186                 style->set_iconstyle(icon_style);
00187                 style->set_labelstyle(label_style);
00188
00189                 // Add style to document.
00190                 document->add_styleselector(style);
00191             }
00192
00193             { /* Create Small Icon Style: */
00194
00195                 // Create icon.
00196                 IconStyleIconPtr icon = GetKmlFactory()->CreateIconStyleIcon();
00197                 icon->set_href("http://argo.whoi.edu/kml/img/dot2.png");
00198
00199                 // Create hotspot.
00200                 HotSpotPtr hot_spot = GetKmlFactory()->CreateHotSpot();
00201                 hot_spot->set_x(0.5);
00202                 hot_spot->set_xunits(UNITS_FRACTION);
00203                 hot_spot->set_y(0.5);
00204                 hot_spot->set_yunits(UNITS_FRACTION);
00205
00206                 // Create icon style.
00207                 IconStylePtr icon_style = GetKmlFactory()->CreateIconStyle();
00208                 icon_style->set_color(GetDefaltKmlPathColor(style_idx));
00209                 icon_style->set_icon(icon);
00210                 icon_style->set_scale(.7);
00211                 icon_style->set_hotspot(hot_spot);
00212
00213                 // Create label style;
00214                 LabelStylePtr label_style = GetKmlFactory()->CreateLabelStyle();
00215                 label_style->set_scale(0.6);
00216
00217                 // Create style.
00218                 StylePtr style = GetKmlFactory()->CreateStyle();
00219                 style->set_id(GetSmallKmlPlacemarkStyleId(style_idx));
00220                 style->set_iconstyle(icon_style);
00221                 style->set_labelstyle(label_style);
00222
00223                 // Add style to document.
00224                 document->add_styleselector(style);
00225             }
00226
00227             { /* Create Triangle Icon Style: */
00228
00229                 // Create icon.
00230                 IconStyleIconPtr icon = GetKmlFactory()->CreateIconStyleIcon();
00231                 icon->set_href("http://argo.whoi.edu/kml/img/tri2.png");
00232
00233                 // Create hotspot.
00234                 HotSpotPtr hot_spot = GetKmlFactory()->CreateHotSpot();
00235                 hot_spot->set_x(0.5);
00236                 hot_spot->set_xunits(UNITS_FRACTION);
00237                 hot_spot->set_y(0);
00238                 hot_spot->set_yunits(UNITS_FRACTION);
00239
00240                 // Create icon style.
00241                 IconStylePtr icon_style = GetKmlFactory()->CreateIconStyle();
00242                 icon_style->set_color(GetDefaltKmlPathColor(style_idx));
00243                 icon_style->set_icon(icon);
00244 //                    icon_style->set_scale(1);
00245                 icon_style->set_hotspot(hot_spot);
```

```
00246
00247                 // Create label style;
00248                 LabelStylePtr label_style = GetKmlFactory()->CreateLabelStyle();
00249                 label_style->set_scale(0.6);
00250
00251                 // Create style.
00252                 StylePtr style = GetKmlFactory()->CreateStyle();
00253                 style->set_id(GetTriangleKmlPlacemarkStyleId(style_idx));
00254                 style->set_iconstyle(icon_style);
00255                 style->set_labelstyle(label_style);
00256
00257                 // Add style to document.
00258                 document->add_styleselector(style);
00259             }
00260
00261         }CATCH_NOTHROW("Failed to set default style " + lexical_cast<string>(style_idx) + "!")
00262
00263     }
00264
00265     /** Add the default styles for every palette colour to a KML document.
00266      *
00267      *  @param[in]  document    The KML document to add the styles to.
00268      *
00269      *  @throw      util::log::MessageException if an error occurs.
00270      */
00271     static void SetAllDefaultKmlSyles(const DocumentPtr& document) {
00272
00273         try {
00274
00275             for(int i = 0; i < KML_NUM_DFLT_PATH_COLORS; ++i)
00276
00277                 SetDefaultKmlStyle(document, i);
00278
00279         }CATCH_BLOCK("Failed to create default KML path styles!")
00280
00281     }
00282
00283     /** Release this thread's KML factory pointer.
00284      *
00285      *  @warning    This deletes libkml's shared factory (see @ref kml_factory_). It is not
00286      *              currently called.
00287      *
00288      *  @throw      util::log::MessageException if an error occurs.
00289      */
00290     static void CloseKmlFactory() {
00291
00292         try {
00293
00294             if(kml_factory_.get())
00295                 kml_factory_.reset();
00296
00297         }CATCH_BLOCK("Failed to close KmlFactory object!")
00298
00299     }
00300
00301     /** Get the palette colour for an index (an instrument ID, for example), modulo the palette size.
00302      *
00303      *  @param[in]  id      The index.
00304      *
00305      *  @return     The colour.
00306      */
00307     static const Color32& GetDefaltKmlPathColor(int id) {
00308         return kml_default_path_colors_[id % KML_NUM_DFLT_PATH_COLORS];
00309     }
00310
00311     /** Get the path (line) style ID for an index, modulo the palette size.
00312      *
00313      *  @param[in]  id      The index.
00314      *
00315      *  @return     The KML style ID.
00316      */
00317     static const string& GetDefaltKmlPathStyleId(int id) {
00318         return kml_default_path_style_ids_[id % KML_NUM_DFLT_PATH_COLORS];
00319     }
00320
00321     /** Get the large placemark style ID for an index, modulo the palette size.
00322      *
00323      *  @param[in]  id      The index.
00324      *
00325      *  @return     The KML style ID.
00326      */
00327     static const string& GetDefaltKmlPlacemarkStyleId(int id) {
00328         return kml_default_placemark_style_ids_[id % KML_NUM_DFLT_PATH_COLORS];
00329     }
00330
00331     /** Get the small placemark style ID for an index, modulo the palette size.
00332      *
00333      *  @param[in]  id      The index.
00334      *
00335      *  @return     The KML style ID.
00336      */
00337     static const string& GetSmallKmlPlacemarkStyleId(int id) {
00338         return kml_small_placemark_style_ids_[id % KML_NUM_DFLT_PATH_COLORS];
00339     }
00340
00341     /** Get the triangle placemark style ID for an index, modulo the palette size.
00342      *
00343      *  @param[in]  id      The index.
00344      *
00345      *  @return     The KML style ID.
00346      */
00347     static const string& GetTriangleKmlPlacemarkStyleId(int id) {
00348         return kml_triangle_placemark_style_ids_[id % KML_NUM_DFLT_PATH_COLORS];
00349     }
00350
00351 };
00352
00353 } /* namespace kml */
```

```
00354 } /* namespace util */
00355
00356 #endif /* KMLUTIL_H_ */
```

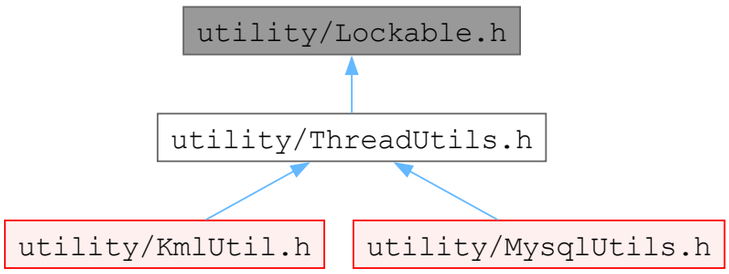
11.194 utility/Lockable.h File Reference

The util::thread::Lockable class header file.

```
#include "Log.h"
#include <boost/thread/locks.hpp>
#include <boost/thread/lock_guard.hpp>
#include <boost/thread/mutex.hpp>
```

Include dependency graph for Lockable.h:

This graph shows which files directly or indirectly include this file:



Classes

- class util::thread::Lockable
  A lockable class.

Namespaces

- namespace util
  The utility namespace.
- namespace util::thread
  The thread utility namespace.

11.194.1 Detailed Description

The util::thread::Lockable class header file.

Version

0.6

Date

Oct 18, 2014 12:38:49 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.195 Lockable.h

Go to the documentation of this file.

```
00001 /** The @ref util::thread::Lockable class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Oct 18, 2014 12:38:49 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef LOCKABLE_H_
00024 #define LOCKABLE_H_
00025
00026 /* Includes */
00027
00028 // local
00029 #include "Log.h"
00030
00031 // boost
00032 #include <boost/thread/locks.hpp>
00033 #include <boost/thread/lock_guard.hpp>
00034 #include <boost/thread/mutex.hpp>
00035
00036 namespace util {
00037 namespace thread {
00038
00039 using boost::mutex;
00040
00041 /* Class Definitions */
00042
00043 /** A lockable class.
00044  *
00045  *  @note           The mutex used for locking the object is not recursive, which means that it may
00046  *                  be lock at most one time, even by the same thread.
00047  */
00048 class Lockable {
00049
00050 protected:
00051
00052     /* Data Members */
00053
00054     /** Mutual exclusion lock for synchronizing operations. */
00055     mutable mutex mutex_;
00056
00057 public:
00058
00059     /* Member Functions */
00060
00061     /** The constructor.
00062      *
00063      */
00064     Lockable() {
00065         /* Do nothing. */
00066     }
00067
00068     /** The destructor.
00069      *
00070      */
00071     virtual ~Lockable();
00072
00073     /** Lock the object.
00074      *
00075      *  @param[in]      block       If TRUE, then the calling thread will block until the lock
00076      *                              becomes available. The default value is TRUE.
00077      *
00078      *  @throw          util::log::MessageException if an error occurs.
00079      *
00080      *  @return         TRUE if the lock was successfully obtained, or FALSE if block is FALSE and
00081      *                  the mutex is already locked.
00082      */
00083     bool LockObject(bool block) {
00084         try {
00085             if(block) mutex_.lock();
00086             else return mutex_.try_lock();
00087             return true;
00088         }CATCH_BLOCK("Failed to lock lockable object!")
00089         return false;
00090     }
00091
00092     /** Unlock the object.
00093      *
00094      *  @throw          util::log::MessageException if an error occurs.
00095      */
00096     void UnlockObject() {
00097         try {
00098             mutex_.unlock();
00099         }CATCH_BLOCK("Failed to unlock lockable object!")
00100     }
00101 };
00102
00103 } /* namespace thread */
```

```
00104 } /* namespace util */
00105 #endif /* LOCKABLE_H_ */
```

11.196 utility/Log.cpp File Reference

The util::log::Log class source file.

`#include "Log.h"`
Include dependency graph for Log.cpp:

Namespaces

- namespace util

  The utility namespace.
- namespace util::log

  The logging utility namespace.

11.196.1 Detailed Description

The util::log::Log class source file.

Version

0.6

Date

May 12, 2014 4:16:21 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.197 utility/Log.h File Reference

The util::log::Log class header file.

```
#include "MessageException.h"
#include <boost/filesystem.hpp>
#include <boost/date_time/posix_time/posix_time_types.hpp>
#include <boost/log/attributes/attribute.hpp>
#include <boost/log/attributes/attribute_cast.hpp>
#include <boost/log/attributes/attribute_name.hpp>
#include <boost/log/attributes/attribute_value.hpp>
#include <boost/log/attributes/scoped_attribute.hpp>
#include <boost/log/common.hpp>
#include <boost/log/core.hpp>
#include <boost/log/expressions.hpp>
#include <boost/log/sinks.hpp>
#include <boost/log/sources/severity_logger.hpp>
#include <boost/log/support/date_time.hpp>
#include <boost/log/trivial.hpp>
#include <boost/log/utility/setup/common_attributes.hpp>
#include <boost/log/utility/setup/console.hpp>
#include <boost/log/utility/setup/file.hpp>
#include <boost/log/utility/value_ref.hpp>
```

Include dependency graph for Log.h:

This graph shows which files directly or indirectly include this file:

Classes

- class util::log::Log

  The logger class.

Namespaces

- namespace util

  The utility namespace.
- namespace util::config

  The configuration utility namespace.
- namespace util::log

  The logging utility namespace.

Macros

- #define CATCH_BLOCK(msg)

  Standardized catch block for exception handling (throws exception).
- #define CATCH_NOTHROW(msg)

  Standardized catch block for exception handling (no throw).
- #define dbg1() LOG(util::log::LOG_DBG1) << __PRETTY_FUNCTION__ << ": "

  Returns an output stream for a debug trace level 1 message.
- #define dbg2() LOG(util::log::LOG_DBG2) << __PRETTY_FUNCTION__ << ": "

  Returns an output stream for a debug trace level 3 message.
- #define dbg3() LOG(util::log::LOG_DBG3) << __PRETTY_FUNCTION__ << ": "

  Returns an output stream for a debug trace level 2 message.
- #define DEFAULT_LOG_LEVEL util::log::LOG_LOG

  The default logging level to use, if none is specified in the run-time configuration.

- #define exc_catchAny(msg)

  Catch arbitrary exception (throws exception).
- #define exc_catchAny_noThrow(msg)

  Catch arbitrary exception (no throw).
- #define exc_catchStandard(e, msg)

  Catch standard exception (throws exception).
- #define exc_catchStandard_noThrow(e, msg)

  Catch standard exception (no throw).
- #define exc_diag(e) boost::diagnostic_information(e, true)

  Return a string containing diagnostic information for a given exception, e.
- #define EXC_THROW(msg) BOOST_THROW_EXCEPTION(util::log::MessageException(std::string() + msg))

  Throw an exception.
- #define ldbg() LOG(util::log::LOG_DBG)

  Returns an output stream for a debug message.
- #define lerr() LOG(util::log::LOG_ERR)

  Returns an output stream for an error message.
- #define lfat() LOG(util::log::LOG_FAT)

  Returns an output stream for a fatal error message.
- #define linf() LOG(util::log::LOG_INF)

  Returns an output stream for an info message.
- #define llog() LOG(util::log::LOG_LOG)

  Returns an output stream for a log message.
- #define LOG(level)

  Returns output stream for given log level.
- #define **LOG_INSTRUMENT_SPECIFIC** (false)
- #define **LOG_SET_SCOPED_THREAD_NAME**(NAME) BOOST_LOG_SCOPED_THREAD_ATTR("Thread↩ Name", boost::log::attributes::constant<string>(NAME));
- #define lout() LOG(util::log::LOG_LOG)

  Returns an output stream for a log message.
- #define lwrn() LOG(util::log::LOG_WRN)

  Returns an output stream for a warning message.
- #define N_LOG_LEVEL_NAMES 6

  The number of unique log level names.
- #define PP_STR(x) X_PP_STR(x)

  Returns a char of the name of a given preprocessor definition.*
- #define X_PP_STR(x) #x

  Returns a char of the value of a given preprocessor definition.*

Typedefs

- typedef blg::sources::severity_logger_mt< blg::trivial::severity_level > util::log::sev_logger_mt

  A thread-safe severity logger object type.

Enumerations

- enum util::log::LogLevel_t {
  util::log::LOG_FAT = 0 ,
  util::log::LOG_ERR = 1 ,
  util::log::LOG_WRN = 2 ,
  util::log::LOG_LOG = 3 ,
  util::log::LOG_INF = 4 ,
  util::log::LOG_DBG = 5 ,
  util::log::LOG_DBG1 = 6 ,
  util::log::LOG_DBG2 = 7 ,
  util::log::LOG_DBG3 = 8 }

  Logging levels enumeration.

11.197.1 Detailed Description

The util::log::Log class header file.

Version

0.6

Date

May 12, 2014 4:16:21 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.197.2 Macro Definition Documentation

11.197.2.1 CATCH_BLOCK

```
#define CATCH_BLOCK(
            msg)
```

Standardized catch block for exception handling (throws exception).

Referenced by argo::iridium::xfloat::XFloatDiveData::XFloatDiveData(), argo::iridium::xfloat::XFloatMailMessage::XFloatMailMessage(), argo::iridium::xfloat::XFloatMetadata::XFloatMetadata(), util::config::LocalConfiguration::AcquireRuntimeLock(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AggregateSensorData(), util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::AggregateSensorData(), util::xmsg::XCompositeSurfaceData::Aggreg util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignAuxTemperatureMe argo::iridium::xfloat::XFloatDiveData::AssignGps(), argo::iridium::xfloat::XFloatDiveData::AssignMailMessage(), util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::AssignMessage(), util::xmsg::XMfaMissionCfg::AssignMessage(), argo::iridium::xfloat::XFloatDiveData::AssignParentInstrument(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CL util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignSalinityMessage(), argo::iridium::xfloat::XFloatDiveData::AssignSurfaceTimeSeries(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_C util::xmsg::XAcousticPosition::AssignWindowIndices(), argo::iridium::xfloat::XFloatDiveData::CheckCompleteness(), util::config::LocalConfiguration::CheckRuntimeLock(), util::xmsg::XMessage::CheckXMsgData(), util::mysql::MysqlUtils::CleanupSqlConne util::mysql::MysqlUtils::CleanupSqlDriver(), util::kml::KmlUtil::CloseKmlFactory(), util::mysql::MysqlUtils::CloseSqlConnection(), util::xmsg::XCompositeIridiumPosition::ComputeAverageIrdiumPosition(), argo::iridium::xfloat::XFloatDiveData::ComputePressureOffsetF argo::iridium::xfloat::XFloatMailRetrievalService::ConnectToMailServer(), argo::iridium::xfloat::XFloatDiveData::DecodeMailMessageData argo::iridium::xfloat::XFloatTelemetryDecoderService::DecodeMessage(), argo::iridium::xfloat::XFloatDiveData::DecodeXMsgData(), util::xmsg::XAcousticCacst::DecodeXMsgData(), util::xmsg::XAcousticPosition::DecodeXMsgData(), util::xmsg::XAcousticWindow::Decod util::xmsg::XAlamoBit2::DecodeXMsgData(), util::xmsg::XAlamoBit::DecodeXMsgData(), util::xmsg::XAlamoMissionConfiguration::Decode util::xmsg::XArgo::DecodeXMsgData(), util::xmsg::XDriftPressure::DecodeXMsgData(), util::xmsg::XDriftSalinity::DecodeXMsgData(), util::xmsg::XDriftTemperature::DecodeXMsgData(), util::xmsg::XEngAbort::DecodeXMsgData(), util::xmsg::XEngAscent::DecodeXMsgDa util::xmsg::XEngBIT::DecodeXMsgData(), util::xmsg::XEngDescent::DecodeXMsgData(), util::xmsg::XEngDiagnostic::DecodeXMsgData(util::xmsg::XEngDrift::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngStandard::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XFall::DecodeXMsgData(), util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XHiResPressure::DecodeXMsgData(), util::xmsg::XHiResSalinity::DecodeXMsgData(), util::xmsg::XHiResTemperature::Decod util::xmsg::XMfaMissionCfg::DecodeXMsgData(), util::xmsg::XMissionCmdEcho::DecodeXMsgData(), util::xmsg::XMissionConfiguration:: util::xmsg::XProfilePressure::DecodeXMsgData(), util::xmsg::XProfileSalinity::DecodeXMsgData(), util::xmsg::XProfileTemperature::Deco util::xmsg::XPump::DecodeXMsgData(), util::xmsg::XRise::DecodeXMsgData(), util::xmsg::XSurfaceData::DecodeXMsgData(), util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), argo::iridium::xfloat::XFloatDiveData::DeleteFromDataba util::xmsg::XGPS::DeleteFromDatabase(), util::xmsg::XGPS::DeleteFromDatabase(), argo::iridium::xfloat::XFloatDiveData::DeleteObjects util::xmsg::XGPS::DeleteObjectsFromDatabase(), argo::iridium::xfloat::XFloatMailRetrievalService::DisconnectFromMailServer(), util::thread::ThreadUtils::executeAndPipe(), util::mysql::MysqlUtils::ExecuteQuery(), util::mysql::LocalMysqlConfiguration::executeSelectQ

util::mysql::MysqlUtils::ExecuteUpdate(), argo::iridium::xfloat::XFloatMailRetrievalService::FetchMailMessagesFromFileSystem(), argo::iridium::xfloat::XFloatMailRetrievalService::FetchMailMessagesFromServer(), util::config::LocalConfiguration::GenerateCommandLi argo::iridium::xfloat::XFloatDiveData::GenerateCompositeFallTimeSeriesData(), argo::iridium::xfloat::XFloatDiveData::GenerateComposit argo::iridium::xfloat::XFloatDiveData::GenerateCompositePumpTimeSeriesData(), argo::iridium::xfloat::XFloatDiveData::GenerateCompo util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription(), util::date::DateUtils::GenerateJulianDay(), argo::iridium::xfloat::XFloat::GenerateKmlMasterDocument(), argo::iridium::xfloat::XFloat::GenerateKmlMasterString(), argo::iridium::xfloat::XFloat::GenerateKmlString(), argo::iridium::xfloat::XFloatMailMessage::GenerateListingComment(), util::xmsg::AbstractXFallRise::GenerateListingComment(), util::xmsg::XCompositeIridiumPosition::GenerateListingComment(), util::xmsg::XGPS::GenerateListingComment(), util::xmsg::XPump::GenerateListingComment(), util::xmsg::XValve::GenerateListingComm argo::iridium::xfloat::XFloatDiveData::GenerateListingString(), argo::iridium::xfloat::XFloatMailMessage::GenerateListingString(), util::xmsg::XAcousticPosition::GenerateListingString(), util::xmsg::XAlamoMissionConfiguration::GenerateListingString(), util::xmsg::XArgo::GenerateListingString(), util::xmsg::XCompositeDrift::GenerateListingString(), util::xmsg::XCompositeFall::GenerateList util::xmsg::XCompositeHiResProfile::GenerateListingString(), util::xmsg::XCompositeIridiumPosition::GenerateListingString(), util::xmsg::XCompositeProfile::GenerateListingString(), util::xmsg::XCompositePump::GenerateListingString(), util::xmsg::XCompositeRis util::xmsg::XCompositeSurfaceData::GenerateListingString(), util::xmsg::XEngAbort::GenerateListingString(), util::xmsg::XEngBIT::Gener util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDiagnostic::GenerateListingString(), util::xmsg::XEngDrift::GenerateLis util::xmsg::XEngStandard::GenerateListingString(), util::xmsg::XEngSurface::GenerateListingString(), util::xmsg::XFall::GenerateListingSt util::xmsg::XGPS::GenerateListingString(), util::xmsg::XMfaMissionCfg::GenerateListingString(), util::xmsg::XMissionCmdEcho::Generate util::xmsg::XMissionConfiguration::GenerateListingString(), util::xmsg::XPresCodeCounts::GenerateListingString(), util::xmsg::XPresCounts::GenerateListingString(), util::xmsg::XPump::GenerateListingString(), util::xmsg::XPumpCounts::GenerateListing util::xmsg::XRise::GenerateListingString(), util::xmsg::XSensorTripletCodeCounts::GenerateListingString(), util::xmsg::XSurfaceData::Gen util::xmsg::XWave::GenerateListingString(), util::xmsg::XWaveRawSample::GenerateListingString(), argo::iridium::xfloat::XFloatDiveData: argo::iridium::xfloat::XFloatDiveData::GenerateOrderedPidList(), argo::iridium::xfloat::XFloatDiveData::GeneratePhyString(), util::xmsg::XCompositeDrift::GeneratePhyString(), util::xmsg::XCompositeHiResProfile::GeneratePhyString(), util::xmsg::XCompositeProf util::xmsg::XGPS::GeneratePhyString(), util::mysql::MysqlUtils::GeneratePreparedStatement(), argo::iridium::xfloat::XFloatDiveData::Gen argo::iridium::xfloat::XFloat::GetAllCachedFloats(), argo::iridium::xfloat::XFloat::GetAllDiveData(), util::file::FileSystemUtils::GetDirectoryP util::file::FileSystemUtils::GetDirectoryPathFromString(), argo::iridium::xfloat::XFloat::GetDiveData(), argo::iridium::xfloat::XFloatDiveData: argo::iridium::xfloat::XFloatDiveData::GetDiveDataObjectFromDatabase(), argo::iridium::xfloat::XFloat::GetFloat(), util::kml::KmlUtil::GetKmlFactory(), argo::iridium::xfloat::XFloat::GetListingDirectoryPath(), util::xmsg::XPresCodeCounts::GetPhaseCodeN util::mysql::MysqlUtils::GetSqlConnection(), util::mysql::LocalMysqlConfiguration::getSqlConnection(), util::mysql::MysqlUtils::GetSqlDrive util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::HasContiguousMessageIn argo::iridium::xfloat::XFloatDiveData::HasContiguousMomsns(), argo::iridium::xfloat::XFloatDiveData::HasContiguousPids(), argo::iridium::xfloat::XFloatDiveData::HasValidGpsPosition(), util::config::LocalConfiguration::InitializeConfiguration(), util::log::Log::InitializeLog(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS > util::file::FileSystemUtils::ListDirectories(), util::file::FileSystemUtils::ListFiles(), argo::iridium::xfloat::XFloatMetadata::ListMetadataFiles(), argo::iridium::xfloat::XFloatMailRetrievalService::ListRawMailFiles(), argo::iridium::xfloat::XFloatMetadata::LoadMetadataFromFiles(), util::thread::Lockable::LockObject(), util::thread::RecursiveLockable::LockObject(), util::file::FileSystemUtils::MoveFileToDirectory(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::num_pres_samples(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::num_saln_samples(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::num_temp_samples(), argo::iridium::xfloat::XFloatMailRetrievalService::OpenMailInboxFolder(), argo::iridium::xfloat::operator<<(), argo::iridium::xfloat::XFloatM argo::iridium::xfloat::XFloatMailMessage::ParseMailMsg(), argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgBody(), argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgHeader(), argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgSbdAttachmen util::config::LocalConfiguration::ParseProgramOptions(), argo::iridium::xfloat::XFloatTelemetryDecoderService::ProcessDiveData(), argo::iridium::xfloat::XFloatMailProcessingService::ProcessMailMsg(), util::sig::SignalHandler::RegisterSignalHandlers(), util::config::LocalConfiguration::ReleaseRuntimeLock(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TE util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::Reset(), util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CL argo::iridium::xfloat::XFloatDiveData::ResetData(), argo::iridium::xfloat::XFloat::ResetDiveData(), util::xmsg::AbstractXSensorDataCompo util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::SamplePresAsFloat(), util::xmsg::AbstractXSensorDataComposite< util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::SampleSalnAsFloat(), util::xmsg::AbstractXSensorDataComposite< util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::SampleTempAsFloat(), util::xmsg::AbstractXSensorData::SampleValu argo::iridium::xfloat::XFloat::SaveObjectToDatabase(), argo::iridium::xfloat::XFloatDiveData::SaveObjectToDatabase(), argo::iridium::xfloat::XFloatMailMessage::SaveObjectToDatabase(), argo::iridium::xfloat::XFloatMetadata::SaveObjectToDatabase(), util::mysql::SqlTableDescription< T >::SaveObjectToDatabase(), util::xmsg::XGPS::SaveObjectToDatabase(), argo::iridium::xfloat::XFloat argo::iridium::xfloat::XFloatMailMessage::SelectAllObjectsFromDatabase(), util::xmsg::XGPS::SelectAllObjectsFromDatabase(), argo::iridium::xfloat::XFloat::SelectObjectFromDatabase(), argo::iridium::xfloat::XFloatDiveData::SelectObjectFromDatabase(), argo::iridium::xfloat::XFloatMailMessage::SelectObjectFromDatabase(), util::mysql::SqlTableDescription< T >::SelectObjectFromDatabas util::xmsg::XGPS::SelectObjectFromDatabase(), argo::iridium::xfloat::XFloat::SelectObjectsFromDatabase(), argo::iridium::xfloat::XFloatD argo::iridium::xfloat::XFloatMailMessage::SelectObjectsFromDatabase(), util::mysql::SqlTableDescription< T >::SelectObjectsFromDatab util::xmsg::XGPS::SelectObjectsFromDatabase(), util::mysql::SqlTableDescription< T >::SelectValueFromDatabase(), util::kml::KmlUtil::SetAllDefaultKmlSyles(), argo::iridium::xfloat::XFloat::StartupTasks(), argo::iridium::xfloat::XFloatMailRetrievalService::S argo::iridium::xfloat::XFloatPostProcessingService::StartWorkerThread(), argo::iridium::xfloat::XFloatTelemetryDecoderService::StartWor StartWorkerThreads(), argo::iridium::xfloat::XFloatMailRetrievalService::TerminateWorkerThread(), argo::iridium::xfloat::XFloatPostProces argo::iridium::xfloat::XFloatTelemetryDecoderService::TerminateWorkerThreads(), TerminateWorkerThreads(), util::thread::Lockable::Unlo util::thread::RecursiveLockable::UnlockObject(), util::xmsg::AbstractXFallRise::UnpackData(), util::xmsg::AbstractXSensorData::UnpackSe util::xmsg::AuxiliarySensorData::UnpackSensorData(), argo::iridium::xfloat::XFloatDiveData::UpdateDerivedSurfaceDataFields(), argo::iridium::xfloat::XFloatMailMessage::WriteFiles(), argo::iridium::xfloat::XFloat::WriteKmlFile(), argo::iridium::xfloat::XFloat::WriteKmlM argo::iridium::xfloat::XFloatDiveData::WriteListingFile(), argo::iridium::xfloat::XFloatDiveData::WritePhyFile(), argo::iridium::xfloat::XFloatM argo::iridium::xfloat::XFloatMailMessage::WriteSbdFile(), and argo::iridium::xfloat::XFloat::WriteStatusFile().

11.197.2.2 CATCH_NOTHROW

```
#define CATCH_NOTHROW(
              msg)
```

Standardized catch block for exception handling (no throw).

Referenced by argo::iridium::xfloat::XFloatDiveData::AssignGps(), argo::iridium::xfloat::XFloatDiveData::DecodeXMsgData(), argo::iridium::xfloat::XFloatMailRetrievalService::DisconnectFromMailServer(), util::sig::SignalHandler::ExitHandler(), argo::iridium::xfloat::XFloatMailRetrievalService::FetchMailMessagesFromFileSystem(), argo::iridium::xfloat::XFloatMailRetrievalService:: argo::iridium::xfloat::XFloatDiveData::GenerateListingString(), argo::iridium::xfloat::XFloatMetadata::LoadFile(), main(), argo::iridium::xfloat::XFloatMailProcessingService::operator()(), argo::iridium::xfloat::XFloatMailRetrievalService::operator()(), argo::iridium::xfloat::XFloatPostProcessingService::operator()(), argo::iridium::xfloat::XFloatTelemetryDecoderService::operator()(), argo::iridium::xfloat::XFloat::ResetCachedDiveData(), argo::iridium::xfloat::XFloat::SaveLocalCacheToDatabase(), argo::iridium::xfloat::XFloatDiveData::SaveObjectToDatabase(), util::mysql::SqlTableDescription< T >::SaveObjectToDatabase(), util::kml::KmlUtil::SetDefaultKmlStyle(), util::xmsg::AbstractXSensorData::UnpackSensorData(), and util::xmsg::AuxiliarySensorData::Unp

11.197.2.3 dbg1

```
#define dbg1() LOG(util::log::LOG_DBG1) << __PRETTY_FUNCTION__ << ":  "
```

Returns an output stream for a debug trace level 1 message.

Referenced by util::xmsg::XWave::DecodeXMsgData(), util::thread::ThreadUtils::executeAndPipe(), argo::iridium::xfloat::XFloat::GetDiveD and argo::iridium::xfloat::XFloatMailRetrievalService::OpenMailInboxFolder().

11.197.2.4 dbg2

```
#define dbg2() LOG(util::log::LOG_DBG2) << __PRETTY_FUNCTION__ << ":  "
```

Returns an output stream for a debug trace level 3 message.

11.197.2.5 dbg3

```
#define dbg3() LOG(util::log::LOG_DBG3) << __PRETTY_FUNCTION__ << ":  "
```

Returns an output stream for a debug trace level 2 message.

11.197.2.6 DEFAULT_LOG_LEVEL

```
#define DEFAULT_LOG_LEVEL util::log::LOG_LOG
```

The default logging level to use, if none is specified in the run-time configuration.

Referenced by util::config::LocalConfiguration::ParseProgramOptions().

11.197.2.7 exc_catchAny

```
#define exc_catchAny(
              msg)
```

Value:

```
do {\
    exc_catchAny_noThrow(msg);\
    BOOST_THROW_EXCEPTION(util::log::MessageException(msg));\
} while(0)
```

Catch arbitrary exception (throws exception).

11.197.2.8 exc_catchAny_noThrow

```
#define exc_catchAny_noThrow(
                msg)
```

Value:

```
do {\
    lerr() « msg « std::endl\
        « " Caused by: " « __FILE__ « "(" « __LINE__ « "): "\
        « "Throw in " « __PRETTY_FUNCTION__;\
} while(0)
```

Catch arbitrary exception (no throw).

11.197.2.9 exc_catchStandard

```
#define exc_catchStandard(
                e,
                msg)
```

Value:

```
do {\
    BOOST_THROW_EXCEPTION(\
        util::log::MessageException(std::string(msg)+"\n\tCaused by:\n"+exc_diag(e))\
    );\
} while(0)
```

Catch standard exception (throws exception).

11.197.2.10 exc_catchStandard_noThrow

```
#define exc_catchStandard_noThrow(
                e,
                msg)
```

Value:

```
do {\
    lerr() « msg « std::endl\
        « __FILE__ « "(" « __LINE__ « "): Throw in " « __PRETTY_FUNCTION__ « std::endl\
        « "\tCaused by: " « std::endl\
        « exc_diag(e);\
} while(0)
```

Catch standard exception (no throw).

11.197.2.11 exc_diag

```
#define exc_diag(
                e) boost::diagnostic_information(e, true)
```

Return a string containing diagnostic information for a given exception, e.

Referenced by util::log::Log::Flush().

11.197.2.12 EXC_THROW

```
#define EXC_THROW(
              msg) BOOST_THROW_EXCEPTION(util::log::MessageException(std::string() + msg))
```

Throw an exception.

Referenced by argo::iridium::xfloat::XFloatDiveData::XFloatDiveData(), util::config::LocalConfiguration::AcquireRuntimeLock(), util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::AggregateSensorData(), util::xmsg::XCompositeSurfaceData::Aggreg util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignAuxTemperatureMe argo::iridium::xfloat::XFloatDiveData::AssignFallTimeSeries(), argo::iridium::xfloat::XFloatDiveData::AssignGps(), argo::iridium::xfloat::XFloatDiveData::AssignMailMessage(), util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::AssignM util::xmsg::XMfaMissionCfg::AssignMessage(), argo::iridium::xfloat::XFloatDiveData::AssignParentInstrument(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignPressureMessage(argo::iridium::xfloat::XFloatDiveData::AssignPumpTimeSeries(), argo::iridium::xfloat::XFloatDiveData::AssignRiseTimeSeries(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignSalinityMessage(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignTemperatureMessa argo::iridium::xfloat::XFloatDiveData::AssignValveData(), util::config::LocalConfiguration::CheckRuntimeLock(), util::xmsg::XMessage::CheckXMsgData(), argo::iridium::xfloat::XFloatDiveData::DecodeMailMessageData(), argo::iridium::xfloat::XFloatT argo::iridium::xfloat::XFloatDiveData::DecodeXMsgData(), util::xmsg::XAlamoBit2::DecodeXMsgData(), util::xmsg::XAlamoBit::DecodeXM util::xmsg::XAlamoMissionConfiguration::DecodeXMsgData(), util::xmsg::XDriftPressure::DecodeXMsgData(), util::xmsg::XDriftSalinity::D util::xmsg::XDriftTemperature::DecodeXMsgData(), util::xmsg::XEngAscent::DecodeXMsgData(), util::xmsg::XEngBIT::DecodeXMsgData util::xmsg::XEngDescent::DecodeXMsgData(), util::xmsg::XEngDrift::DecodeXMsgData(), util::xmsg::XEngIce::DecodeXMsgData(), util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XFall::DecodeXMsgData(), util::xmsg::XGPS::DecodeXMsgData(), util::xmsg::XHiResPressure::DecodeXMsgData(), util::xmsg::XHiResSalinity::DecodeXMsgData(), util::xmsg::XHiResTemperature::Decod util::xmsg::XMfaMissionCfg::DecodeXMsgData(), util::xmsg::XProfilePressure::DecodeXMsgData(), util::xmsg::XProfileSalinity::DecodeXI util::xmsg::XProfileTemperature::DecodeXMsgData(), util::xmsg::XRise::DecodeXMsgData(), util::xmsg::XSurfaceData::DecodeXMsgData util::xmsg::XWave::DecodeXMsgData(), util::thread::ThreadUtils::executeAndPipe(), argo::iridium::xfloat::XFloatMailRetrievalService::Fetc argo::iridium::xfloat::XFloatMailRetrievalService::FetchMailMessagesFromServer(), util::date::DateUtils::GenerateJulianDay(), argo::iridium::xfloat::XFloatDiveData::GenerateListingString(), util::xmsg::XAlamoBit2::GenerateListingString(), util::xmsg::XAlamoBit::Ger util::xmsg::XAlamoMissionConfiguration::GenerateListingString(), util::xmsg::XEngAscent::GenerateListingString(), util::xmsg::XEngDescent::GenerateListingString(), util::xmsg::XEngDrift::GenerateListingString(), util::xmsg::XEngSurface::GenerateListin util::xmsg::XMfaMissionCfg::GenerateListingString(), util::xmsg::XWave::GenerateListingString(), util::mysql::SqlTableDescription< T >::G argo::iridium::xfloat::XFloatDiveData::GeneratePhyString(), util::mysql::MysqlUtils::GeneratePreparedStatement(), util::file::FileSystemUtils::GetDirectoryPath(), argo::iridium::xfloat::XFloatDiveData::GetDiveDataMailMessagesFromDatabase(), argo::iridium::xfloat::XFloatDiveData::GetDiveDataObjectFromDatabase(), util::xmsg::XPresCodeCounts::GetPhaseCodeName(), util::log::Log::InitializeLog(), util::file::FileSystemUtils::ListDirectories(), util::file::FileSystemUtils::ListFiles(), argo::iridium::xfloat::XFloatMet argo::iridium::xfloat::XFloatMetadata::LoadFile(), util::file::FileSystemUtils::MoveFileToDirectory(), argo::iridium::xfloat::XFloatMailMessag argo::iridium::xfloat::XFloatMailMessage::ParseMailMsg(), argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgBody(), argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgHeader(), argo::iridium::xfloat::XFloatMailMessage::ParseMailMsgSbdAttachmen util::config::LocalConfiguration::ParseProgramOptions(), argo::iridium::xfloat::XFloatTelemetryDecoderService::ProcessDiveData(), argo::iridium::xfloat::XFloatMailProcessingService::ProcessMailMsg(), util::sig::SignalHandler::RegisterSignalHandlers(), util::config::LocalConfiguration::ReleaseRuntimeLock(), util::mysql::SqlTableDescription< T >::SaveObjectToDatabase(), util::mysql::SqlTableDescription< T >::SelectValueFromDatabase(), util::xmsg::AbstractXFallRise::UnpackData(), util::xmsg::AbstractXSensorData::UnpackSensorData(), util::xmsg::AuxiliarySensorData::UnpackSensorData(), and argo::iridium::xfloat::XFloatMailMessage::WriteFiles().

11.197.2.13 ldbg

```
#define ldbg() LOG(util::log::LOG_DBG)
```

Returns an output stream for a debug message.

Referenced by util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::AggregateSensorData(), util::xmsg::XCompositeSurfa util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignAuxTemperatureMe argo::iridium::xfloat::XFloatDiveData::AssignMailMessage(), util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::AssignM util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignPressureMessage(util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignSalinityMessage(), util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >::AssignTemperatureMessa argo::iridium::xfloat::XFloatDiveData::ComputePressureOffsetFromSurfaceTimeSeries(), argo::iridium::xfloat::XFloatMailRetrievalService argo::iridium::xfloat::XFloatTelemetryDecoderService::DecodeMessage(), argo::iridium::xfloat::XFloatDiveData::DecodeXMsgData(), util::xmsg::XAlamoBit2::DecodeXMsgData(), util::xmsg::XAlamoMissionConfiguration::DecodeXMsgData(), util::xmsg::XGPS::DecodeXM util::xmsg::XMfaMissionCfg::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), argo::iridium::xfloat::XFloatMailRetrievalService argo::iridium::xfloat::XFloatMailRetrievalService::FetchMailMessagesFromFileSystem(), argo::iridium::xfloat::XFloatMailRetrievalService:: argo::iridium::xfloat::XFloatDiveData::GenerateCompositeFallTimeSeriesData(), argo::iridium::xfloat::XFloatDiveData::GenerateComposit argo::iridium::xfloat::XFloatDiveData::GenerateCompositePumpTimeSeriesData(), argo::iridium::xfloat::XFloatDiveData::GenerateCompo util::file::FileSystemUtils::GetDirectoryPath(), argo::iridium::xfloat::XFloat::GetDiveData(), util::xmsg::AbstractXSensorDataComposite< P argo::iridium::xfloat::XFloatMetadata::LoadFile(), argo::iridium::xfloat::XFloatMailProcessingService::operator()(), argo::iridium::xfloat::XFloatMailRetrievalService::operator()(), argo::iridium::xfloat::XFloatPostProcessingService::operator()(), argo::iridium::xfloat::XFloatTelemetryDecoderService::operator()(), argo::iridium::xfloat::XFloatTelemetryDecoderService::ProcessDiveDa

argo::iridium::xfloat::XFloatMailProcessingService::ProcessMailMsg(), util::sig::SignalHandler::RegisterSignalHandlers(), util::config::LocalConfiguration::ReleaseRuntimeLock(), argo::iridium::xfloat::XFloat::SaveObjectToDatabase(), argo::iridium::xfloat::XFloa argo::iridium::xfloat::XFloatMetadata::SaveObjectToDatabase(), argo::iridium::xfloat::XFloatMailRetrievalService::StartWorkerThread(), argo::iridium::xfloat::XFloatPostProcessingService::StartWorkerThread(), argo::iridium::xfloat::XFloatMailProcessingService::StartWorker argo::iridium::xfloat::XFloatTelemetryDecoderService::StartWorkerThreads(), argo::iridium::xfloat::XFloatMailRetrievalService::TerminateW argo::iridium::xfloat::XFloatPostProcessingService::TerminateWorkerThread(), argo::iridium::xfloat::XFloatMailProcessingService::Termin argo::iridium::xfloat::XFloatTelemetryDecoderService::TerminateWorkerThreads(), util::xmsg::AbstractXSensorData::UnpackSensorData(util::xmsg::AuxiliarySensorData::UnpackSensorData(), and argo::iridium::xfloat::XFloatDiveData::UpdateDerivedSurfaceDataFields().

11.197.2.14 lerr

```
#define lerr() LOG(util::log::LOG_ERR)
```

Returns an output stream for an error message.

Referenced by util::xmsg::XWave::DecodeXMsgData().

11.197.2.15 lfat

```
#define lfat() LOG(util::log::LOG_FAT)
```

Returns an output stream for a fatal error message.

11.197.2.16 linf

```
#define linf() LOG(util::log::LOG_INF)
```

Returns an output stream for an info message.

Referenced by argo::iridium::xfloat::XFloatDiveData::DecodeXMsgData(), argo::iridium::xfloat::XFloat::LoadLocalCacheFromDatabase(), argo::iridium::xfloat::XFloat::LoadMetadataFromFiles(), argo::iridium::xfloat::XFloatMailRetrievalService::operator()(), argo::iridium::xfloat::XFloatMailProcessingService::ProcessMailMsg(), util::sig::SignalHandler::RegisterSignalHandlers(), util::config::LocalConfiguration::ReleaseRuntimeLock(), argo::iridium::xfloat::XFloat::SaveLocalCacheToDatabase(), StartWorkerThreads(), TerminateWorkerThreads(), argo::iridium::xfloat::XFloat::WriteKmlFile(), argo::iridium::xfloat::XFloat::WriteKmlMas argo::iridium::xfloat::XFloatDiveData::WriteListingFile(), argo::iridium::xfloat::XFloatDiveData::WritePhyFile(), argo::iridium::xfloat::XFloatM argo::iridium::xfloat::XFloatMailMessage::WriteSbdFile(), and argo::iridium::xfloat::XFloat::WriteStatusFile().

11.197.2.17 llog

```
#define llog() LOG(util::log::LOG_LOG)
```

Returns an output stream for a log message.

11.197.2.18 LOG

```
#define LOG(
              level)
```

Value:

```
if (level > util::log::Log::log_reporting_level()) { /* Do nothing. */ }\
else util::log::Log(level)
```

Returns output stream for given log level.

11.197.2.19 lout

```
#define lout() LOG(util::log::LOG_LOG)
```

Returns an output stream for a log message.

Referenced by util::sig::SignalHandler::ExitHandler(), argo::iridium::xfloat::XFloatMailRetrievalService::FetchMailMessagesFromServer(), argo::iridium::xfloat::XFloat::LoadLocalCacheFromDatabase(), argo::iridium::xfloat::XFloat::LoadMetadataFromFiles(), argo::iridium::xfloat::XFloatMailRetrievalService::operator()(), argo::iridium::xfloat::XFloatTelemetryDecoderService::ProcessDiveData(), util::sig::SignalHandler::RegisterSignalHandlers(), util::config::LocalConfiguration::ReleaseRuntimeLock(), argo::iridium::xfloat::XFloat::Sa argo::iridium::xfloat::XFloat::StartupTasks(), StartWorkerThreads(), and TerminateWorkerThreads().

11.197.2.20 lwrn

```
#define lwrn() LOG(util::log::LOG_WRN)
```

Returns an output stream for a warning message.

Referenced by util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >::AggregateSensorData(), util::xmsg::XCompositeSurfa
util::xmsg::XMessage::CheckXMsgData(), argo::iridium::xfloat::XFloatDiveData::ComputePressureOffsetFromSurfaceTimeSeries(),
argo::iridium::xfloat::XFloatDiveData::DecodeMailMessageData(), argo::iridium::xfloat::XFloatDiveData::DecodeXMsgData(),
util::xmsg::XAlamoBit2::DecodeXMsgData(), util::xmsg::XAlamoMissionConfiguration::DecodeXMsgData(), util::xmsg::XEngIce::DecodeX
util::xmsg::XEngSurface::DecodeXMsgData(), util::xmsg::XMfaMissionCfg::DecodeXMsgData(), util::xmsg::XPump::DecodeXMsgData(),
util::xmsg::XValve::DecodeXMsgData(), util::xmsg::XWave::DecodeXMsgData(), argo::iridium::xfloat::XFloatMailRetrievalService::FetchM
util::xmsg::XMfaMissionCfg::GenerateListingString(), argo::iridium::xfloat::XFloatMailProcessingService::ProcessMailMsg(),
util::config::LocalConfiguration::ReleaseRuntimeLock(), argo::iridium::xfloat::XFloatMailRetrievalService::TerminateWorkerThread(),
argo::iridium::xfloat::XFloatPostProcessingService::TerminateWorkerThread(), util::xmsg::AbstractXFallRise::UnpackData(),
and util::xmsg::AbstractXSensorData::UnpackSensorData().

11.197.2.21 N_LOG_LEVEL_NAMES

```
#define N_LOG_LEVEL_NAMES 6
```

The number of unique log level names.

Deprecated Log names are no long used.

11.197.2.22 PP_STR

```
#define PP_STR(
              x) X_PP_STR(x)
```

Returns a char* of the name of a given preprocessor definition.

11.197.2.23 X_PP_STR

```
#define X_PP_STR(
              x) #x
```

Returns a char* of the value of a given preprocessor definition.

11.198 Log.h

Go to the documentation of this file.

```
00001 /** The @ref util::log::Log class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       May 12, 2014 4:16:21 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef LOG_H_
00024 #define LOG_H_
00025
00026 /* Includes */
00027
00028 // local
00029 #include "MessageException.h"
00030
00031 // boost filesystem
```

```
00032 #include <boost/filesystem.hpp>
00033 namespace bfs = boost::filesystem;
00034
00035 // boost postix_time
00036 #include <boost/date_time/posix_time/posix_time_types.hpp>
00037
00038 // boost log
00039 #include <boost/log/attributes/attribute.hpp>
00040 #include <boost/log/attributes/attribute_cast.hpp>
00041 #include <boost/log/attributes/attribute_name.hpp>
00042 #include <boost/log/attributes/attribute_value.hpp>
00043 #include <boost/log/attributes/scoped_attribute.hpp>
00044 #include <boost/log/common.hpp>
00045 #include <boost/log/core.hpp>
00046 #include <boost/log/expressions.hpp>
00047 #include <boost/log/sinks.hpp>
00048 #include <boost/log/sources/severity_logger.hpp>
00049 #include <boost/log/support/date_time.hpp>
00050 #include <boost/log/trivial.hpp>
00051 #include <boost/log/utility/setup/common_attributes.hpp>
00052 #include <boost/log/utility/setup/console.hpp>
00053 #include <boost/log/utility/setup/file.hpp>
00054 #include <boost/log/utility/value_ref.hpp>
00055 namespace blg = boost::log;
00056
00057 /* Definitions */
00058
00059 #define LOG_INSTRUMENT_SPECIFIC ( false )
00060
00061 /** The default logging level to use, if none is specified in the run-time configuration. */
00062 #define DEFAULT_LOG_LEVEL       util::log::LOG_LOG
00063
00064 /** The number of unique log level names.
00065  *
00066  *  @deprecated     Log names are no long used.
00067  */
00068 #define N_LOG_LEVEL_NAMES       6
00069
00070 // boost log attribute keyword definitions
00071 #pragma GCC diagnostic push
00072 #pragma GCC diagnostic ignored "-Wmissing-field-initializers"
00073 BOOST_LOG_ATTRIBUTE_KEYWORD(attr_sevr, "Severity", blg::trivial::severity_level)
00074 BOOST_LOG_ATTRIBUTE_KEYWORD(attr_dac, "DacID", int)
00075 BOOST_LOG_ATTRIBUTE_KEYWORD(attr_dive, "DiveID", int)
00076 BOOST_LOG_ATTRIBUTE_KEYWORD(attr_momsn, "MomsnID", int)
00077 BOOST_LOG_ATTRIBUTE_KEYWORD(attr_muid, "MsgUID", int)
00078 BOOST_LOG_ATTRIBUTE_KEYWORD(attr_pckt, "PcktID", int)
00079 BOOST_LOG_ATTRIBUTE_KEYWORD(attr_xmid, "XmsgID", std::string)
00080 BOOST_LOG_ATTRIBUTE_KEYWORD(attr_whoi, "WhoiID", int)
00081 BOOST_LOG_ATTRIBUTE_KEYWORD(attr_wmo, "WmoID", int)
00082 BOOST_LOG_ATTRIBUTE_KEYWORD(attr_tname, "ThreadName", std::string)
00083 BOOST_LOG_ATTRIBUTE_KEYWORD(attr_tnum, "ThreadNum", int)
00084 #pragma GCC diagnostic pop
00085
00086 /*  Definitions - Macros:  */
00087
00088
00089 /** Returns a char* of the value of a given preprocessor definition. */
00090 #   define X_PP_STR(x)  #x
00091
00092 /** Returns a char* of the name of a given preprocessor definition. */
00093 #   define PP_STR(x)    X_PP_STR(x)
00094
00095 /*  Definitions - Macros - Log: */
00096
00097 #   define LOG_SET_SCOPED_THREAD_NAME(NAME) \
00098     BOOST_LOG_SCOPED_THREAD_ATTR("ThreadName", boost::log::attributes::constant<string>(NAME));
00099
00100 /** Returns output stream for given log level. */
00101 #   define LOG(level)  if (level > util::log::Log::log_reporting_level()) { /* Do nothing. */ }\
00102                         else util::log::Log(level)
00103
00104 /** Returns an output stream for a fatal error message. */
00105 #   define lfat()      LOG(util::log::LOG_FAT)
00106
00107 /** Returns an output stream for an error message. */
00108 #   define lerr()      LOG(util::log::LOG_ERR)
00109
00110 /** Returns an output stream for a warning message. */
00111 #   define lwrn()      LOG(util::log::LOG_WRN)
00112
00113 /** Returns an output stream for a log message. */
00114 #   define lout()      LOG(util::log::LOG_LOG)
00115
00116 /** Returns an output stream for a log message. */
00117 #   define llog()      LOG(util::log::LOG_LOG)
00118
00119 /** Returns an output stream for an info message. */
00120 #   define linf()      LOG(util::log::LOG_INF)
00121
00122 /** Returns an output stream for a debug message. */
00123 #   define ldbg()      LOG(util::log::LOG_DBG)
00124
00125 /** Returns an output stream for a debug trace level 1 message. */
00126 #   define dbg1()      LOG(util::log::LOG_DBG1) « __PRETTY_FUNCTION__ « ": "
00127
00128 /** Returns an output stream for a debug trace level 2 message. */
00129 #   define dbg3()      LOG(util::log::LOG_DBG3) « __PRETTY_FUNCTION__ « ": "
00130
00131 /** Returns an output stream for a debug trace level 3 message. */
00132 #   define dbg2()      LOG(util::log::LOG_DBG2) « __PRETTY_FUNCTION__ « ": "
00133
00134 // exception handling
00135
00136 /** Return a string containing diagnostic information for a given exception, e. */
00137 #   define exc_diag(e) boost::diagnostic_information(e, true)
00138
00139 /** Catch standard exception (no throw). */
```

```
00140 #   define exc_catchStandard_noThrow(e, msg)\
00141 do {\
00142     lerr() « msg « std::endl\
00143         « __FILE__ « "(" « __LINE__ « "): Throw in " « __PRETTY_FUNCTION__ « std::endl\
00144         « "\tCaused by: " « std::endl\
00145         « exc_diag(e);\
00146 } while(0)
00147
00148 /** Catch standard exception (throws exception). */
00149 #   define exc_catchStandard(e, msg)\
00150 do {\
00151     BOOST_THROW_EXCEPTION(\
00152         util::log::MessageException(std::string(msg)+"\n\tCaused by:\n"+exc_diag(e))\
00153     );\
00154 } while(0)
00155
00156 /** Catch arbitrary exception (no throw). */
00157 #   define exc_catchAny_noThrow(msg)\
00158 do {\
00159     lerr() « msg « std::endl\
00160         « " Caused by: " « __FILE__ « "(" « __LINE__ « "): "\
00161         « "Throw in " « __PRETTY_FUNCTION__;\
00162 } while(0)
00163
00164 /** Catch arbitrary exception (throws exception). */
00165 #   define exc_catchAny(msg)\
00166 do {\
00167     exc_catchAny_noThrow(msg);\
00168     BOOST_THROW_EXCEPTION(util::log::MessageException(msg));\
00169 } while(0)
00170
00171 /** Standardized catch block for exception handling (throws exception). */
00172 #   define CATCH_BLOCK(msg)\
00173 catch(std::regex_error& e) {\
00174     if(e.code() == std::regex_constants::error_backref) {\
00175         lerr() « "regex_error caught: error_backref";\
00176     } else if(e.code() == std::regex_constants::error_badbrace) {\
00177         lerr() « "regex_error caught: error_badbrace";\
00178     } else if(e.code() == std::regex_constants::error_badrepeat) {\
00179         lerr() « "regex_error caught: error_badrepeat";\
00180     } else if(e.code() == std::regex_constants::error_brace) {\
00181         lerr() « "regex_error caught: error_brace";\
00182     } else if(e.code() == std::regex_constants::error_brack) {\
00183         lerr() « "regex_error caught: error_brack";\
00184     } else if(e.code() == std::regex_constants::error_collate) {\
00185         lerr() « "regex_error caught: error_collate";\
00186     } else if(e.code() == std::regex_constants::error_complexity) {\
00187         lerr() « "regex_error caught: error_complexity";\
00188     } else if(e.code() == std::regex_constants::error_ctype) {\
00189         lerr() « "regex_error caught: error_ctype";\
00190     } else if(e.code() == std::regex_constants::error_escape) {\
00191         lerr() « "regex_error caught: error_escape";\
00192     } else if(e.code() == std::regex_constants::error_paren) {\
00193         lerr() « "regex_error caught: error_paren";\
00194     } else if(e.code() == std::regex_constants::error_range) {\
00195         lerr() « "regex_error caught: error_range";\
00196     } else if(e.code() == std::regex_constants::error_space) {\
00197         lerr() « "regex_error caught: error_space";\
00198     } else if(e.code() == std::regex_constants::error_stack) {\
00199         lerr() « "regex_error caught: error_stack";\
00200     }\
00201     exc_catchStandard(e, msg);\
00202 } catch(std::exception& e) {\
00203     exc_catchStandard(e, msg);\
00204 } catch(...) {\
00205     exc_catchAny(msg);\
00206 }
00207
00208 /** Standardized catch block for exception handling (no throw). */
00209 #   define CATCH_NOTHROW(msg)\
00210 catch(std::regex_error& e) {\
00211     if(e.code() == std::regex_constants::error_backref) {\
00212         lerr() « "regex_error caught: error_backref";\
00213     } else if(e.code() == std::regex_constants::error_badbrace) {\
00214         lerr() « "regex_error caught: error_badbrace";\
00215     } else if(e.code() == std::regex_constants::error_badrepeat) {\
00216         lerr() « "regex_error caught: error_badrepeat";\
00217     } else if(e.code() == std::regex_constants::error_brace) {\
00218         lerr() « "regex_error caught: error_brace";\
00219     } else if(e.code() == std::regex_constants::error_brack) {\
00220         lerr() « "regex_error caught: error_brack";\
00221     } else if(e.code() == std::regex_constants::error_collate) {\
00222         lerr() « "regex_error caught: error_collate";\
00223     } else if(e.code() == std::regex_constants::error_complexity) {\
00224         lerr() « "regex_error caught: error_complexity";\
00225     } else if(e.code() == std::regex_constants::error_ctype) {\
00226         lerr() « "regex_error caught: error_ctype";\
00227     } else if(e.code() == std::regex_constants::error_escape) {\
00228         lerr() « "regex_error caught: error_escape";\
00229     } else if(e.code() == std::regex_constants::error_paren) {\
00230         lerr() « "regex_error caught: error_paren";\
00231     } else if(e.code() == std::regex_constants::error_range) {\
00232         lerr() « "regex_error caught: error_range";\
00233     } else if(e.code() == std::regex_constants::error_space) {\
00234         lerr() « "regex_error caught: error_space";\
00235     } else if(e.code() == std::regex_constants::error_stack) {\
00236         lerr() « "regex_error caught: error_stack";\
00237     }\
00238     exc_catchStandard_noThrow(e, msg);\
00239 } catch(std::exception& e) {\
00240     exc_catchStandard_noThrow(e, msg);\
00241 } catch(...) {\
00242     exc_catchAny_noThrow(msg);\
00243 }
00244
00245 /** Throw an exception. */
00246 #   define EXC_THROW(msg) BOOST_THROW_EXCEPTION(util::log::MessageException(std::string() + msg))
00247
```

```
00248 namespace util {
00249 namespace config {
00250 // forward declaration
00251 class LocalConfiguration;
00252 } /* namespace config */
00253
00254 namespace log {
00255
00256 namespace attr = boost::log::attributes;
00257 namespace expr = boost::log::expressions;
00258 namespace keys = boost::log::keywords;
00259 using boost::algorithm::trim;
00260 using boost::filesystem::is_directory;
00261 using boost::filesystem::path;
00262 using boost::format;
00263 using boost::log::add_common_attributes;
00264 using boost::log::core;
00265 using boost::log::sinks::file::rotation_at_time_point;
00266 using boost::log::sinks::file::as_file_name_composer;
00267 using boost::log::sinks::synchronous_sink;
00268 using boost::log::sinks::text_multifile_backend;
00269 using std::make_shared;
00270 using boost::posix_time::ptime;
00271 using std::shared_ptr;
00272 using std::cout;
00273 using std::cerr;
00274 using std::endl;
00275 using std::exception;
00276 using std::ios;
00277 using std::setfill;
00278 using std::setw;
00279 using std::string;
00280 using std::ostream;
00281 using std::ostringstream;
00282
00283 /* Enumerations */
00284
00285 /** Logging levels enumeration.
00286  *
00287  */
00288 typedef enum LogLevel {
00289     LOG_FAT = 0,                    ///< Logging level 0: Fatal errors only.
00290     LOG_ERR = 1,                    ///< Logging level 1: Errors only.
00291     LOG_WRN = 2,                    ///< Logging level 2: Warnings, i.e. quiet.
00292     LOG_LOG = 3,                    ///< Logging level 3: Standard/Default output logging level.
00293     LOG_INF = 4,                    ///< Logging level 4: Informational messages, i.e. verbose.
00294     LOG_DBG = 5,                    ///< Logging level 5: Debugging messages.
00295     LOG_DBG1 = 6,                   ///< Logging level 6: More debugging messages.
00296     LOG_DBG2 = 7,                   ///< Logging level 7: Even more debugging messages.
00297     LOG_DBG3 = 8                    ///< Logging level 8: Most debugging messages.
00298 } LogLevel_t;
00299
00300 /* Type Definitions */
00301
00302 /** A thread-safe severity logger object type.
00303  *
00304  *  @note           This logger type uses the severity levels defined by
00305  *                  blg::trivial::severity_level in the boost log library.
00306  *
00307  */
00308 typedef blg::sources::severity_logger_mt<blg::trivial::severity_level> sev_logger_mt;
00309
00310 /* Class Definitions */
00311
00312 /** The logger class.
00313  *
00314  *  This class is a wrapper for the boost thread-safe severity logger object. It provides
00315  *  facilities for formatted console and file logging and supports filtering by message severity
00316  *  and other attributes.
00317  *
00318  */
00319 class Log {
00320
00321     /* Friends */
00322
00323     friend class config::LocalConfiguration;
00324
00325 private:
00326
00327     /* Static Data Members */
00328
00329     /** The global severity logger object.
00330      *
00331      *  This object is used as a global logger for all log messages generated by the application.
00332      *  It includes support for log message attributes that are used for filtering and formatting
00333      *  of the log messages. It automatically filters the destination (log files, standard output,
00334      *  etc.) of log messages, and handles daily rotation of log files.
00335      */
00336     static sev_logger_mt global_logger_;
00337
00338     /** The log directory path.
00339      *
00340      *  Log files generated as an output of this application will be written to this directory
00341      *  as well as sub-directories of this directory.
00342      *
00343      *  This field is set as a result of parsing the configuration options during initialization.
00344      *  During initialization, the application checks to ensure that this directory exists, and
00345      *  throws an error if it does not. Thus, during the subsequent duration of execution of the
00346      *  application, this directory is assumed to exist on the local filesystem.
00347      */
00348     static path log_directory_path_;
00349
00350     /** The global reporting level.
00351      *
00352      *  This field is used to filter the log messages by severity. Only log message with a severity
00353      *  that is less than or equal to this value will be reported by the global logger.
00354      *
00355      *  This field is set a result of parsing the configuration options during initialization of
```

```
00356      *  the application. If the log level is unspecified in the configuration options, then it will
00357      *  default to a value of @ref DEFAULT_LOG_LEVEL.
00358      */
00359     static LogLevel_t log_reporting_level_;
00360
00361 private:
00362
00363     /* Dynamic Data Members */
00364
00365     /** The severity level of this log message. */
00366     LogLevel_t log_msg_severity_level_;
00367
00368     /** The log message output stream. */
00369     ostringstream log_msg_stream_;
00370
00371 private:
00372
00373     /** Set the reporting level of the global logger.
00374      *
00375      *  @param[in]  level   The reporting level of the global logger to set.
00376      */
00377     static void set_log_reporting_level(LogLevel_t level) {
00378         log_reporting_level_ = level;
00379     }
00380
00381 public:
00382
00383     /* Member Functions */
00384
00385     /** The default constructor.
00386      *
00387      *  @param[in]  level   The message severity level.
00388      */
00389     Log(LogLevel_t level) ATTR_NO_EXCEPTIONS :
00390             log_msg_severity_level_(level) {
00391       /* Do nothing. */
00392     }
00393
00394     /** The destructor.
00395      *
00396      *  Flushes the global severity logger's output stream.
00397      */
00398     ~Log() {
00399         Flush(log_msg_severity_level_);
00400     }
00401
00402     /** Flush the log's message buffer buffer.
00403      *
00404      *  @exception      None. Never throws.
00405      *
00406      *  @param[in]      log         The logger.
00407      *
00408      *  @return         A const reference to this log's output stream.
00409      */
00410     template<typename T = sev_logger_mt> ostream& Flush(
00411             T& log = global_logger_) ATTR_NO_EXCEPTIONS {
00412         return Flush(log_msg_severity_level_, log);
00413     }
00414
00415     /** Flush the log's message buffer buffer.
00416      *
00417      *  @exception      None. Never throws.
00418      *
00419      *  @param[in]      lev         The severity level at which to flush the log.
00420      *  @param[in]      log         The logger.
00421      *
00422      *  @return         A const reference to this log's output stream.
00423      */
00424     template<typename T = sev_logger_mt> ostream& Flush(
00425             LogLevel_t lev,
00426             T& log = global_logger_) ATTR_NO_EXCEPTIONS {
00427
00428         using blg::trivial::severity_level;
00429
00430         try {
00431
00432             string msg(log_msg_stream_.str());
00433             log_msg_stream_.str("");
00434             log_msg_stream_.clear();
00435             trim(msg);
00436
00437             if(lev <= log_reporting_level_ && !msg.empty()) switch(lev) {
00438                 case LOG_FAT: { // fatal
00439
00440                     BOOST_LOG_SEV(log, severity_level::fatal) << msg;
00441
00442                 }
00443
00444                 break;
00445
00446                 case LOG_ERR: { // error
00447
00448                     BOOST_LOG_SEV(log, severity_level::error) << msg;
00449                 }
00450
00451                 break;
00452
00453                 case LOG_WRN: { // warning
00454
00455                     BOOST_LOG_SEV(log, severity_level::warning) << msg;
00456
00457                 }
00458
00459                 break;
00460
00461                 case LOG_LOG: // info
00462                 case LOG_INF: {
00463
```

```
00464                     BOOST_LOG_SEV(log, severity_level::info) « msg;
00465
00466                 }
00467
00468                 break;
00469
00470 #           if defined(BUILD_DEBUG) && BUILD_DEBUG
00471
00472                 case LOG_DBG: { // debug
00473
00474                     BOOST_LOG_SEV(log, severity_level::debug) « msg;
00475
00476                 }
00477
00478                 break;
00479
00480                 case LOG_DBG1: // trace
00481                 case LOG_DBG2:
00482                 case LOG_DBG3: {
00483
00484                     BOOST_LOG_SEV(log, severity_level::trace) « msg;
00485
00486                 }
00487
00488                 break;
00489
00490 #           endif // defined(BUILD_DEBUG) && BUILD_DEBUG
00491
00492                 default:
00493
00494                 /* Do nothing. */
00495
00496                 break;
00497
00498             }
00499
00500         } catch(exception& e) {
00501
00502             cerr « "LOG ERROR: Failed to Flush log object! Caused by:" « exc_diag(e);
00503
00504         } catch(...) {
00505
00506             cerr « "LOG ERROR: Failed to Flush log object! Caused unknown.";
00507
00508         }
00509
00510         return log_msg_stream_;
00511     }
00512
00513     /** Flush the global logger object.
00514      *
00515      *  @exception      None. Never throws.
00516      *
00517      */
00518     static void FlushGlobalLogger() ATTR_NO_EXCEPTIONS {
00519         blg::core::get()->flush();
00520     }
00521
00522     /** Initialize the global logger.
00523      *
00524      *  Adds common attributes to the global logger and defines the format of the log output.
00525      *
00526      *  Adds a log file at logPath/%Y-%m-%d.%3N.log that is rotated at midnight (local time) or every
00527      *  time each log file exceeds 10MB.
00528      *
00529      *  Also adds a console log.
00530      *
00531      *  Adds a float-specific log file.
00532      *
00533      *  @throw          MessageException if the directory specified by logPath does not exist, or an
00534      *                  error occurs while initializing the global logger.
00535      */
00536     static void InitializeLog() ATTR_NO_EXCEPTIONS {
00537
00538         try {
00539             cout « "Initializing logger...";
00540
00541             try { // Add logger attributes.
00542                 add_common_attributes();
00543             }CATCH_BLOCK("Failed to add common attributes!")
00544
00545             // Define the log format.
00546             auto fmt = expr::stream
00547             « expr::format_date_time<ptime>("TimeStamp", "%Y-%m-%dT%H:%M:%S.%f ")
00548             « setfill(' ') « setw(12) « attr_tname « " "
00549             « setfill(' ') « setw(7) « attr_sevr « " "
00550             « expr::if_(expr::has_attr(attr_whoi))
00551             [
00552             expr::stream  «  "Float[" «  setfill('0')  «  setw(5)  «  attr_whoi  «  "|"
00553                 «  expr::if_(expr::has_attr(attr_dac) && expr::is_in_range(attr_dac, 1000, 10000))
00554                     [
00555                      expr::stream « setfill('0') « setw(5) « attr_dac
00556                     ]
00557                     .else_
00558                     [
00559                      expr::stream « "?????"
00560                     ]
00561                 «  "|"
00562                 «  expr::if_(expr::has_attr(attr_wmo) && expr::is_in_range(attr_wmo, 1000000, 10000000))
00563                     [
00564                      expr::stream « setfill('0') « setw(7) « attr_wmo
00565                     ]
00566                     .else_
00567                     [
00568                     expr::stream  «  "???????"
00569                     ]
00570                 «  "] "
00571                 «  expr::if_(expr::has_attr(attr_dive) || expr::has_attr(attr_muid))
```

```
00572                     [
00573                     expr::stream <<  "Dive["
00574                         <<  expr::if_(expr::has_attr(attr_dive)  && expr::is_in_range(attr_dive, -1,
     10000))
00575                             [
00576                              expr::stream  <<  setfill(' ')  <<  setw(4)  <<  attr_dive
00577                             ]
00578                             .else_
00579                             [
00580                              expr::stream << "????"
00581                             ]
00582                         <<  "] "
00583                         <<  expr::if_(expr::has_attr(attr_muid))
00584                             [
00585                              expr::stream << "SBD["
00586                                 <<  expr::if_(expr::has_attr(attr_muid) && expr::is_in_range(attr_muid, 0,
     1000000))
00587                                     [
00588                                      expr::stream << setfill('0') << setw(6) << attr_muid
00589                                     ]
00590                                     .else_
00591                                     [
00592                                      expr::stream << "??????"
00593                                     ]
00594                                 << "|"
00595                                 <<  expr::if_(expr::has_attr(attr_momsn) && expr::is_in_range(attr_momsn,
     0, 1000000))
00596                                     [
00597                                      expr::stream << setfill('0') << setw(6) << attr_momsn
00598                                     ]
00599                                     .else_
00600                                     [
00601                                      expr::stream << "??????"
00602                                     ]
00603                                 <<  "|"
00604                                 <<  expr::if_(expr::has_attr(attr_pckt) && expr::is_in_range(attr_pckt, 0,
     256))
00605                                     [
00606                                      expr::stream  <<  setfill('0') << setw(3) << attr_pckt
00607                                     ]
00608                                     .else_
00609                                     [
00610                                      expr::stream  <<  "???"
00611                                     ]
00612                                 <<  "] "
00613                                 <<  expr::if_(expr::has_attr(attr_xmid))
00614                                     [
00615                                      expr::stream  <<  "Xmsg["  <<  attr_xmid  <<  "] "
00616                                     ]
00617                                     .else_
00618                                     [
00619                                      expr::stream  <<  setfill('-')  <<  setw(11)  <<  " "
00620                                     ]
00621                             ]
00622                             .else_
00623                             [
00624                              expr::stream << setfill('-') << setw(23) << " " << setw(11) << " "
00625                             ]
00626                     ]
00627                     .else_
00628                     [
00629                      expr::stream << setfill('-') << setw(11) << " " << setw(23) << " " << setw(11) << " "
00630                     ]
00631             ]
00632             .else_
00633             [
00634              expr::stream << setfill('-') << setw(27) << " " << setw(11) << " " << setw(23) << " " << setw(11) << "
     "
00635             ]
00636             << expr::message;
00637
00638             try {   // Add a global console log.
00639                 boost::log::add_console_log(cout, keys::format = (fmt));
00640             }CATCH_BLOCK("Failed to add console log!")
00641
00642             try {   // Add a global file log.
00643                 // Check the log path.
00644                 if(!is_directory(log_directory_path_))
00645                 EXC_THROW("Invalid path to log directory:" + log_directory_path_.string());
00646                 string log_path = log_directory_path_.string() + "/%Y-%m-%d.log";
00647                 add_file_log(
00648                         keys::file_name = log_path,//
00649                         keys::format = (fmt),//
00650                         keys::open_mode = (ios::out|ios::app),//
00651                         //               keys::rotation_size = 10 * 1024 * 1024, //
00652                         keys::time_based_rotation = rotation_at_time_point(0, 0, 0));
00653             }CATCH_BLOCK("Failed to add file log!")
00654
00655             try {   // Add a global file log.
00656                 // Check the log path.
00657                 if(!is_directory(log_directory_path_))
00658                 EXC_THROW("Invalid path to log directory:" + log_directory_path_.string());
00659                 string log_path = log_directory_path_.string() + "/%Y-%m-%d.error.log";
00660                 add_file_log(
00661                         keys::file_name = log_path,//
00662                         keys::filter = (attr_sevr >= blg::trivial::warning),
00663                         keys::format = (fmt),//
00664                         keys::open_mode = (ios::out|ios::app),//
00665                         //               keys::rotation_size = 10 * 1024 * 1024, //
00666                         keys::time_based_rotation = rotation_at_time_point(0, 0, 0));
00667             }CATCH_BLOCK("Failed to add file log!")
00668
00669 #if defined(LOG_INSTRUMENT_SPECIFIC) && LOG_INSTRUMENT_SPECIFIC
00670
00671             try {   // Add instrument specific log files.
00672                 shared_ptr<core> core = core::get();
00673                 shared_ptr<text_multifile_backend> backend ( make_shared<text_multifile_backend>());
00674
```

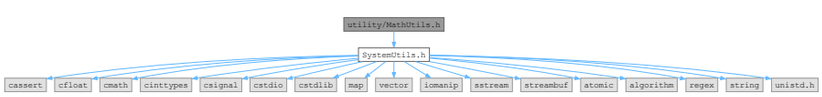
```
00675                 // Set file naming pattern.
00676                 backend->set_file_name_composer(
00677                         as_file_name_composer(
00678                                 expr::stream « log_directory_path_.string() « "/"
00679                                 « attr_whoi « "/"
00680                                 « expr::format_date_time<ptime>("TimeStamp", "%Y-%m-%d")
00681                                 « ".log"
00682                         )
00683                 );
00684
00685                 // Create the backend sink.
00686                 typedef synchronous_sink<text_multifile_backend> mf_sink_t;
00687                 shared_ptr<mf_sink_t> sink(new mf_sink_t(backend));
00688
00689                 // Set the log filter.
00690                 sink->set_filter(expr::has_attr(attr_whoi) && expr::is_in_range(attr_whoi, 1000, 10000));
00691
00692                 // Set the log formatter.
00693                 sink->set_formatter((fmt));
00694
00695                 // Add the sink.
00696                 core->add_sink(sink);
00697
00698             }CATCH_BLOCK("Failed to add instrument-specific log files!")
00699
00700 #endif // defined(LOG_INSTRUMENT_SPECIFIC) && LOG_INSTRUMENT_SPECIFIC
00701
00702             cout « " Done." « endl;
00703
00704         }CATCH_BLOCK("Failed to initialize logger!")
00705     }
00706
00707     /** Get the global logger message severity reporting level.
00708      *
00709      *  @return         The global logger message severity reporting level.
00710      */
00711     static LogLevel_t log_reporting_level() {
00712         return log_reporting_level_;
00713     }
00714
00715     /** The template input stream operator overload function.
00716      *
00717      *  @exception      None. Never throws.
00718      *
00719      *  @param[in]      x           A const reference to the object invoking the input stream
00720      *                              operation. This may be any object that supports the input
00721      *                              stream operation.
00722      *
00723      *  @return         A const reference to this log's output stream.
00724      */
00725     template<typename T> ostream& operator«(const T& x) ATTR_NO_EXCEPTIONS {
00726         log_msg_stream_ « x;
00727         return log_msg_stream_;
00728     }
00729
00730 };
00731
00732 }
00733 /* namespace log */
00734 } /* namespace util */
00735 #endif /* LOG_H_ */
```

11.199 utility/MathUtils.h File Reference

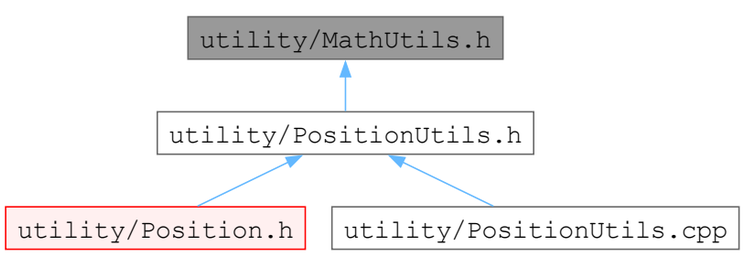
The util::math::MathUtils class header file.

`#include "SystemUtils.h"`

Include dependency graph for MathUtils.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::math::MathUtils

  Class containing commonly-used mathematical functions.

Namespaces

- namespace util

  The utility namespace.
- namespace util::math

  The math utility namespace.

11.199.1 Detailed Description

The util::math::MathUtils class header file.

Version

0.6

Date

Oct 18, 2014 1:24:27 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.200 MathUtils.h

Go to the documentation of this file.

```
00001 /** The @ref util::math::MathUtils class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Oct 18, 2014 1:24:27 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef MATHUTILS_H_
00024 #define MATHUTILS_H_
00025
00026 /* Includes */
00027
00028 // local
00029 #include "SystemUtils.h"
00030
00031 /* Definitions */
00032
00033 namespace util {
```

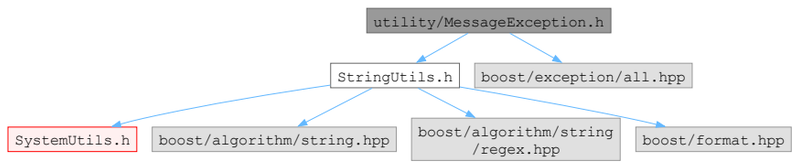
```
00034 namespace math {
00035
00036 /* Class Definitions */
00037
00038 /** Class containing commonly-used mathematical functions.
00039  *
00040  */
00041 class MathUtils {
00042 public:
00043
00044     MathUtils() {
00045       /* Do nothing. */
00046     }
00047
00048     virtual ~MathUtils() {
00049       /* Do nothing. */
00050     }
00051
00052     /** Convert angular units from from degrees to decimal radians.
00053      *
00054      *  @param[in]      deg         Angle in integer or decimal degrees to convert to radians.
00055      *
00056      *  @return         Angle in decimal radians.
00057      */
00058     template <typename T> static double degToRad(T deg) {
00059         return (double)deg*M_PI/180.;
00060     }
00061
00062     static inline float bitcastUint32ToFloat(uint32_t u) {
00063
00064       void* x = &u;
00065
00066       return *((float *) x);
00067
00068     }
00069
00070
00071 };
00072
00073 } /* namespace math */
00074 } /* namespace util */
00075 #endif /* MATHUTILS_H_ */
```

11.201 utility/MessageException.h File Reference

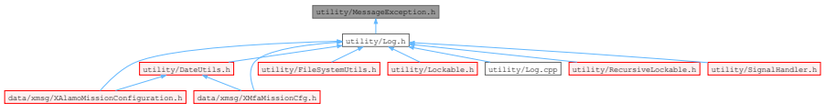
The util::log::MessageException class header file.

```
#include "StringUtils.h"
#include <boost/exception/all.hpp>
```

Include dependency graph for MessageException.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::log::MessageException

  Exception class with associated message.

Namespaces

- namespace util

  The utility namespace.
- namespace util::log

  The logging utility namespace.

Macros

- #define ATTR_NO_EXCEPTIONS throw()

  The "never throws" function attribute.

11.201.1 Detailed Description

The util::log::MessageException class header file.

Version

0.6

Date

May 12, 2014 4:18:48 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.201.2 Macro Definition Documentation

11.201.2.1 ATTR_NO_EXCEPTIONS

```
#define ATTR_NO_EXCEPTIONS throw()
```

The "never throws" function attribute.

11.202 MessageException.h

Go to the documentation of this file.

```
00001 /** The @ref util::log::MessageException class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       May 12, 2014 4:18:48 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022 #ifndef MESSAGEEXCEPTION_H_
00023 #define MESSAGEEXCEPTION_H_
00024
00025 /* Includes */
00026
00027 // local
00028 #include "StringUtils.h"
00029
00030 // boost
```

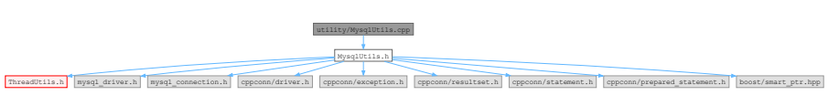
```
00031 #include <boost/exception/all.hpp>
00032
00033 #if defined(BUILD_NUMBER) && defined(__cplusplus) && __cplusplus >= 201103L
00034 /** The "never throws" function attribute.
00035  *
00036  */
00037 #   define ATTR_NO_EXCEPTIONS      noexcept
00038 #else
00039 /** The "never throws" function attribute.
00040  *
00041  */
00042 #   define ATTR_NO_EXCEPTIONS      throw()
00043 #endif //defined(BUILD_NUMBER) && defined(__cplusplus) && __cplusplus >= 201103L
00044
00045 /* Definitions */
00046
00047 namespace util {
00048 namespace log {
00049
00050 using std::string;
00051 using std::ostringstream;
00052
00053 /* Class Definition */
00054
00055 /** Exception class with associated message.
00056  *
00057  */
00058 class MessageException:
00059         public std::exception,
00060         public boost::exception {
00061
00062 private:
00063
00064     /* Data Members */
00065
00066     /** The exception message.
00067      *
00068      */
00069     string message_;
00070
00071 public:
00072
00073     /* Member Functions */
00074
00075     /** Default Constructor.
00076      *
00077      */
00078     MessageException() {
00079         /* Do nothing */
00080     }
00081
00082     /** Parameterized constructor.
00083      *
00084      * @param[in]       message         Exception message string.
00085      */
00086     MessageException(const string& message) :
00087                     message_(message) {
00088         /* Do nothing. */
00089     }
00090
00091     /** Parameterized constructor.
00092      *
00093      * @param[in]       message         Exception message string.
00094      */
00095     MessageException(const ostringstream& message) :
00096                     message_(message.str()) {
00097         /* Do nothing. */
00098     }
00099
00100     /** Destructor.
00101      *
00102      */
00103     virtual ~MessageException() ATTR_NO_EXCEPTIONS {
00104         /* Do nothing. */
00105     }
00106
00107     /** Return C string representing message exception.
00108      *
00109      *  @return         C string representing message exception.
00110      */
00111     virtual const char* what() const ATTR_NO_EXCEPTIONS {
00112         return message_.c_str();
00113     }
00114 };
00115
00116 } /* namespace mail */
00117 } /* namespace utility */
00118 #endif /* MESSAGEEXCEPTION_H_ */
```

11.203 utility/MysqlUtils.cpp File Reference

The util::mysql::MysqlUtils class source file.

`#include "MysqlUtils.h"`

Include dependency graph for MysqlUtils.cpp:



Namespaces

- namespace util

  The utility namespace.
- namespace util::mysql

  The MySQL utility namespace.

Variables

- thread_specific_ptr< Connection > MysqlUtils::sql_connection_ & **util::mysql::CleanupSqlConnection**
- thread_specific_ptr< MySQL_Driver > MysqlUtils::sql_driver_ & **util::mysql::CleanupSqlDriver**

11.203.1 Detailed Description

The util::mysql::MysqlUtils class source file.

Version

0.6

Date

May 12, 2014 4:19:03 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

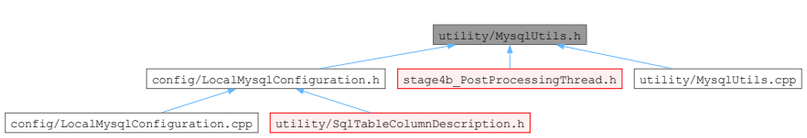
11.204 utility/MysqlUtils.h File Reference

The util::mysql::MysqlUtils class header file.

```
#include "ThreadUtils.h"
#include <mysql_driver.h>
#include <mysql_connection.h>
#include <cppconn/driver.h>
#include <cppconn/exception.h>
#include <cppconn/resultset.h>
#include <cppconn/statement.h>
#include <cppconn/prepared_statement.h>
#include <boost/smart_ptr.hpp>
```

Include dependency graph for MysqlUtils.h:

This graph shows which files directly or indirectly include this file:



Classes

- class util::mysql::MysqlUtils

  Class provides a library of function for accessing and manipulating a MySQL database.

Namespaces

- namespace util

  The utility namespace.
- namespace util::mysql

  The MySQL utility namespace.

11.204.1 Detailed Description

The util::mysql::MysqlUtils class header file.

Version

0.6

Date

May 12, 2014 4:19:03 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.205 MysqlUtils.h

Go to the documentation of this file.

```
00001 /** The @ref util::mysql::MysqlUtils class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       May 12, 2014 4:19:03 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef MYSQL_UTILS_H_
00024 #define MYSQL_UTILS_H_
00025
00026 /* Includes */
00027
00028 // local
00029 #include "ThreadUtils.h"
00030
00031 // mysql
00032 #include <mysql_driver.h>
00033 #include <mysql_connection.h>
```

```
00034 #include <cppconn/driver.h>
00035 #include <cppconn/exception.h>
00036 #include <cppconn/resultset.h>
00037 #include <cppconn/statement.h>
00038 #include <cppconn/prepared_statement.h>
00039
00040 // boost
00041 #include <boost/smart_ptr.hpp>
00042
00043 namespace util {
00044 namespace mysql {
00045
00046 using std::shared_ptr;
00047 using boost::thread_specific_ptr;
00048 using sql::Connection;
00049 using sql::mysql::MySQL_Driver;
00050 using sql::PreparedStatement;
00051 using sql::ResultSet;
00052 using std::endl;
00053 using std::ostringstream;
00054 using std::string;
00055
00056 /* Class Definition */
00057
00058 /** Class provides a library of function for accessing and manipulating a MySQL database.
00059  *
00060  */
00061 class MysqlUtils {
00062
00063 private:
00064
00065     /* Private Data Members: */
00066
00067     /** The thread-specific pointer to MySQL driver object. */
00068     static thread_specific_ptr<MySQL_Driver> sql_driver_;
00069
00070     /** Tthe thread-specific pointer to MySQL connection object. */
00071     static thread_specific_ptr<Connection> sql_connection_;
00072
00073 public:
00074
00075     MysqlUtils() {
00076       /* Do nothing. */
00077     }
00078
00079     virtual ~MysqlUtils() {
00080       /* Do nothing. */
00081     }
00082
00083
00084 private:
00085
00086     /* Private Member Functions: */
00087
00088     /** Cleanup an SQL connection.
00089      *
00090      *  This function for memory cleanup of the SQL connection on destruction. The thread-specific SQL
00091      *  connection object will be deleted if it exists.
00092      *
00093      *  @param[in]      con         The SQL connection to cleanup.
00094      *
00095      *  @throw          log::MessageException if an error occurs while cleaning up the SQL connection
00096      *                  object.
00097      */
00098     static void CleanupSqlConnection(Connection* con) {
00099
00100         try {
00101
00102             if(con)
00103                 // Delete the connection object.
00104                 delete (con);
00105
00106         }CATCH_BLOCK("Failed to cleanup SQL connection object!")
00107     }
00108
00109     /** Function for memory cleanup of MySQL driver on destruct.
00110      *
00111      *  Signals the SQL driver to end operation, if it exists.
00112      *
00113      *  @param[in]      driver      Driver to cleanup.
00114      *
00115      *  @throw          log::MessageException if an error occurs while cleaning up the SQL driver
00116      *                  object.
00117      */
00118     static void CleanupSqlDriver(MySQL_Driver* driver) {
00119
00120         (void) driver;
00121
00122         try {
00123
00124             //if(driver)
00125                 // XXX End this MySQL thread instance.
00126                 //driver->threadEnd();
00127
00128         }CATCH_BLOCK("Failed to cleanup the SQL driver object!")
00129     }
00130
00131 public:
00132
00133     /* Public Member Functions */
00134
00135     /** Close the thread-specific MySQL connection if it is currently open.
00136      *
00137      *  If the thread local MySQL connection is not currently open, then this function will have no
00138      *  effect.
00139      *
00140      *  @note           The MySQL connection should be closed when idle to prevent the connection from
00141      *                  timing out and throwing exceptions.
```

```
00142      *
00143      *  @throw          log::MessageException if an error occurs while attempting to close the MySQL
00144      *                  connection.
00145      */
00146     static void CloseSqlConnection() {
00147
00148         try {
00149
00150             // Check the SQL connection pointer.
00151             if(!sql_connection_.get())
00152                 return;
00153
00154             // Close the SQL connection object.
00155             if(!sql_connection_->isClosed())
00156                 sql_connection_->close();
00157
00158             // Reset the pointer.
00159             sql_connection_.reset();
00160
00161         }CATCH_BLOCK("Failed to close SQL connection!");
00162     }
00163
00164     /** Execute an SQL query and return a shared pointer to the ResultSet object.
00165      *
00166      *  @param[in]      con         A pointer to the SQL connection object. This must not be NULL.
00167      *  @param[in]      query       The SQL query to execute. This must be a well-formatted SQL query
00168      *                              string.
00169      *
00170      *  @throw          log::MessageException if 'con' is null, 'query' is a malfored SQL string,
00171      *                  or any other error occurs.
00172      *
00173      *  @return         A shared pointer to the query ResultSet object.
00174      */
00175     static shared_ptr<ResultSet> ExecuteQuery(Connection * con, const string& query) {
00176         shared_ptr<ResultSet> ptr;
00177
00178         try {
00179
00180             ptr = shared_ptr<ResultSet>(GeneratePreparedStatement(con, query)->executeQuery());
00181
00182         }CATCH_BLOCK("Failed to execute SQL query:\n " + query)
00183
00184         return ptr;
00185     }
00186
00187     /** Execute an SQL update and return the status code.
00188      *
00189      *  @param[in]      con         A pointer to the SQL connection object. This must not be NULL.
00190      *  @param[in]      update      The SQL update to execute. This must be a well-formatted SQL update
00191      *                              string.
00192      *
00193      *  @throw          log::MessageException if 'con' is null, 'update' is a malformed SQL string,
00194      *                  or any other error occurs.
00195      *
00196      *  @return         The update status code.
00197      */
00198     static int ExecuteUpdate(Connection * con, const string& update) {
00199         int stat = -1;
00200
00201         try {
00202
00203             stat = GeneratePreparedStatement(con, update)->executeUpdate();
00204
00205         }CATCH_BLOCK("Failed to execute SQL update: " + update + "!")
00206
00207         return stat;
00208     }
00209
00210     /** Get a thread-specific instance of a MySQL connection object.
00211      *
00212      *  A new connection will be created, if necessary.
00213      *
00214      *  @param[in]      url         The MySQL host server qualified address.
00215      *  @param[in]      user        The MySQL account username.
00216      *  @param[in]      password    The MySQL account password.
00217      *  @param[in]      schema      The MySQL account schema name.
00218      *
00219      *  @throw          log::MessageException if an error occurs while creating a new thread-specific
00220      *                  instance of the SQL connection object
00221      *
00222      *  @return         A thread-specific instance of a MySQL connection object.
00223      */
00224     static thread_specific_ptr<Connection>& GetSqlConnection(
00225             string url,
00226             string user,
00227             string password,
00228             string schema) {
00229
00230         try {
00231
00232             // Check if the connection already exists.
00233             if(!sql_connection_.get() || sql_connection_->isClosed()) {
00234
00235                 // Connect to SQL server - get a new instance of the SQL object for this thread.
00236                 sql_connection_.reset(GetSqlDriver()->connect(url, user, password));
00237                 sql_connection_->setSchema(schema);
00238             }
00239
00240         }CATCH_BLOCK("Failed to get MySQL connection object!")
00241
00242         return sql_connection_;
00243     }
00244
00245     /** Get a thread-specific instance of a MySQL driver object.
00246      *
00247      *  A new driver will created, if necessary.
00248      *
00249      *  @throw          log::MessageException if an error occurs while creating a new thread-specific
```

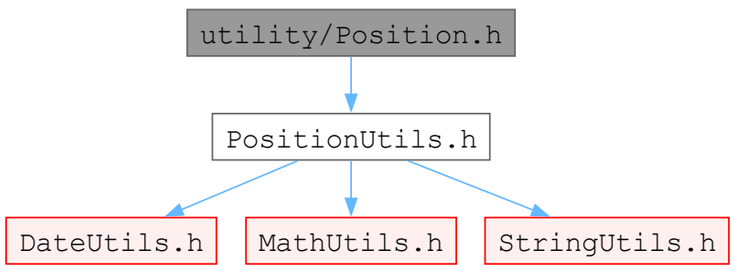
```
00250      *                  instance of the SQL driver object
00251      *
00252      *  @return         A thread-specific instance of a MySQL driver object.
00253      */
00254     static thread_specific_ptr<MySQL_Driver>& GetSqlDriver() {
00255
00256         try {
00257
00258             // Check if the driver already exists
00259             if(!(sql_driver_.get()))
00260
00261                 // Get a new instance of the MySQL driver for this thread.
00262                 sql_driver_.reset(sql::mysql::get_mysql_driver_instance());
00263
00264         }CATCH_BLOCK("Failed to get instance of MySQL driver object!")
00265
00266         return sql_driver_;
00267     }
00268
00269     /** Generate a prepared SQL statement object for the given SQL statement.
00270      *
00271      *  @param[in]      con         A pointer to the SQL connection object. This must not be NULL.
00272      *  @param[in]      statement   The SQL statement to prepare. This must be a well-formed SQL string.
00273      *
00274      *  @throw          log::MessageException if 'con' is NULL, 'statement' is not a valid SQL string,
00275      *                  or if an error occurs.
00276      *
00277      *  @return         A prepared SQL statement object generated from the given SQL statement.
00278      */
00279     static shared_ptr<PreparedStatement> GeneratePreparedStatement(
00280             Connection * con,
00281             const string& statement){
00282
00283         shared_ptr<PreparedStatement> stmt;
00284
00285         try {
00286
00287             // Check the SQL connection pointer.
00288             if(!con)
00289                 EXC_THROW("Pointer to SQL connection is NULL!");
00290
00291 //            dbg1() « "SQL: " « endl « statement;
00292
00293             // Generate the prepared statement.
00294             stmt.reset( con->prepareStatement(statement));
00295
00296         }CATCH_BLOCK("Failed to generate prepared SQL statement!")
00297
00298         return stmt;
00299     }
00300
00301     /** Generate an SQL select statement.
00302      *
00303      *  @param[in]      what        The SQL select statement select-expression clause.
00304      *  @param[in]      from        The SQL select statement table-references clause.
00305      *  @param[in]      where       The SQL select statement where-condition clause.
00306      *  @param[in]      groupBy     The SQL select statement group-by clause.
00307      *  @param[in]      orderBy     The SQL select statement order-by clause.
00308      *  @param[in]      limit       The SQL select statement result set row limit value.
00309      *  @param[in]      offset      The SQL select statement result set row offset.
00310      *
00311      *  @return         The SQL select statement.
00312      */
00313     static string GenerateSqlSelectStr(
00314             const string& what,
00315             const string& from,
00316             const string& where = "",
00317             const string& groupBy = "",
00318             const string& orderBy = "",
00319             unsigned int limit = 0,
00320             unsigned int offset = 0) {
00321
00322         ostringstream os;
00323
00324         os « "SELECT \n\t" « what « " \nFROM \n\t" « from;
00325
00326         if(!where.empty())
00327             os « " \nWHERE \n\t" « where;
00328
00329         if(!groupBy.empty())
00330             os « " \nGROUP BY \n\t" « groupBy;
00331
00332         if(!orderBy.empty())
00333             os « " \nORDER BY \n\t" « orderBy;
00334
00335         if(limit) {
00336             os « " \nLIMIT \n\t" « limit;
00337             if(offset)
00338                 os « " \nOFFSET \n\t" « offset;
00339         }
00340
00341         os « ";";
00342
00343         string select = os.str();
00344
00345         return select;
00346     }
00347
00348     /** Generate an SQL DELETE statement.
00349      *
00350      *  @warning    Without a @p where clause, the statement deletes every row in the table.
00351      *
00352      *  @param[in]  from        The name of the table to delete from.
00353      *  @param[in]  where       The SQL "where-condition" clause (empty = all rows).
00354      *
00355      *  @return     The SQL statement.
00356      */
00357     static string GenerateSqlDeleteStr(
```

```
00358             const string& from,
00359             const string& where = "") {
00360
00361         ostringstream os;
00362
00363         os « "DELETE FROM \n\t" « from;
00364
00365         if(!where.empty())
00366             os « " \nWHERE \n\t" « where;
00367
00368         os « ";";
00369
00370         return os.str();
00371
00372     }
00373
00374 };
00375
00376 } /* namespace iridium */
00377 } /* namespace argo */
00378 #endif /* MYSQL_UTILS_H_ */
```

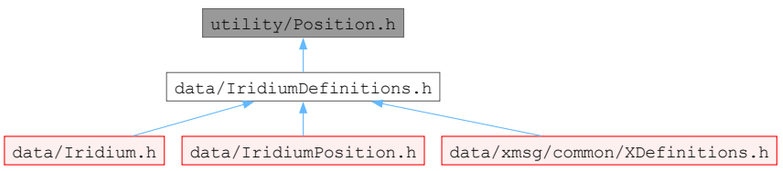
11.206 utility/Position.h File Reference

The util::position::Position class header file.

`#include "PositionUtils.h"`
Include dependency graph for Position.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::position::Position

  Abstract class representing a position, date, and time.

Namespaces

- namespace util

  The utility namespace.
- namespace util::position

  The position utility namespace.

11.206.1 Detailed Description

The util::position::Position class header file.

Version

0.6

Date

May 12, 2014 4:19:36 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.207 Position.h

Go to the documentation of this file.

```
00001 /** The @ref util::position::Position class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       May 12, 2014 4:19:36 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef POSITIONS_H_
00024 #define POSITIONS_H_
00025
00026 /* Includes */
00027
00028 // local
00029 #include "PositionUtils.h"
00030
00031 /* Definitions */
00032
00033 namespace util {
00034 namespace position {
00035
00036 using std::string;
00037 using boost::posix_time::ptime;
00038 using boost::posix_time::hours;
00039 using boost::posix_time::minutes;
00040 using boost::posix_time::seconds;
00041 using boost::gregorian::date;
00042 using boost::gregorian::days;
00043
00044 /** Abstract class representing a position, date, and time.
00045  *
00046  */
00047 class Position:
00048         protected PositionUtils {
00049
00050 private:
00051
00052     /* Private Data Members: */
00053
00054     /** The date and time of the position.
00055      *
00056      */
```

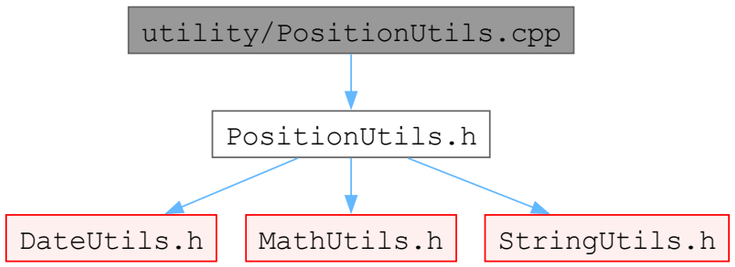
```
00057     ptime position_datetime_;
00058
00059     /** The latitude of the position, in decimal degrees.
00060      *
00061      */
00062     double position_latitude_;
00063
00064     /** The longitude of the position, in decimal degrees.
00065      *
00066      */
00067     double position_longitude_;
00068
00069     /** The valid position indicator.
00070      *
00071      */
00072     bool position_is_valid_;
00073
00074 protected:
00075
00076     /* Protected Member Functions: */
00077
00078     /** Set the date/time of the position.
00079      *
00080      *  @param[in]  datetime    The date/time of the position to set.
00081      */
00082     void set_position_datetime(const ptime& datetime) {
00083         position_datetime_ = datetime;
00084     }
00085
00086     /** Set the date and time of this position from the given GPS week, day, and time.
00087      *
00088      *  @param[in]  gpsWeek     The GPS week, in weeks since midnight of January 6, 1980.
00089      *  @param[in]  gpsDay      The GPS day of week (0 - 6).
00090      *  @param[in]  gpsHour     The UTC hour of day (0 - 23).
00091      *  @param[in]  gpsMinute   The UTC minute of hour (0 - 59)..
00092      *  @param[in]  gpsSecond   The UTC second of minute (0 - 59).
00093      */
00094     void set_gps_position_datetime(
00095             unsigned int gpsWeek,
00096             unsigned int gpsDay,
00097             unsigned int gpsHour = 0,
00098             unsigned int gpsMinute = 0,
00099             unsigned int gpsSecond = 0) {
00100         ptime time(
00101                 date(1980, boost::date_time::Jan, 6),
00102                 hours(gpsHour) + minutes(gpsMinute) + seconds(gpsSecond));
00103         set_position_datetime(time + days((gpsWeek * 7) + gpsDay));
00104     }
00105
00106     /** Set the position is valid flag.
00107      *
00108      *  @param[in]  is_valid    TRUE if the position is valid, or FALSE otherwise.
00109      */
00110     void set_position_is_valid(bool is_valid = true) {
00111         position_is_valid_ = is_valid;
00112     }
00113
00114     /** Set the latitude of the position, in decimal degrees.
00115      *
00116      *  @param[in]  lat         The latitude of the position, in decimal degrees.
00117      */
00118     void set_position_latitude(double lat) {
00119         set_position_is_valid(IsPositionValid(position_latitude_ = lat, position_longitude_));
00120     }
00121
00122     /** Set the longitude of the position, in decimal degrees.
00123      *
00124      *  @param[in]  lon         The longitude of the position, in decimal degrees.
00125      */
00126     void set_position_longitude(double lon) {
00127         position_longitude_ = lon;
00128         set_position_is_valid(IsPositionValid(position_latitude_, position_longitude_ = lon));
00129     }
00130
00131 public:
00132
00133     /* Public Member Functions: */
00134
00135     /** The default constructor.
00136      *
00137      *  @exception  None. Never throws.
00138      */
00139     Position() ATTR_NO_EXCEPTIONS :
00140                     position_datetime_(bpt::not_a_date_time),
00141                     position_latitude_(UTIL_POS_INVALID_LAT),
00142                     position_longitude_(UTIL_POS_INVALID_LON),
00143                     position_is_valid_(false) {
00144         /* Do nothing. */
00145     }
00146
00147     /** The destructor. */
00148     virtual ~Position() {
00149         /* Do nothing. */
00150     }
00151
00152     /** Get the date/time of the position.
00153      *
00154      *  @return         The date/time of the position.
00155      */
00156     ptime position_datetime() const {
00157         return position_datetime_;
00158     }
00159
00160     /** Get a formatted string representing the position date/time composed using the ISO
00161      *  8601 standard.
00162      *
00163      *  @return     The formatted string representation of the position date/time.
00164      */
```

```
00165     string position_datetime_as_iso8601_string() const {
00166         return GenerateIso8601String(position_datetime_);
00167     }
00168
00169     /** Is the position valid?
00170      *
00171      *  @return     TRUE if the position is valid, or false otherwise.
00172      */
00173     bool position_is_valid() const {
00174         return position_is_valid_;
00175     }
00176
00177     /** Get the latitude of the position, in decimal degrees.
00178      *
00179      *  @return     The latitude of the position, in decimal degrees.
00180      */
00181     double position_latitude() const {
00182         return position_latitude_;
00183     }
00184
00185     /** Get the longitude of the position, in decimal degrees.
00186      *
00187      *  @return     The longitude of the position, in decimal degrees.
00188      */
00189     double position_longitude() const {
00190         return position_longitude_;
00191     }
00192
00193     /** Get a formatted string representing the position composed using the ISO 6709
00194      *  standard.
00195      *
00196      *  @return     The formatted string representation of the position.
00197      */
00198     string postition_as_iso6709_string() const {
00199         return GenerateIso6709String(position_latitude_, position_longitude_);
00200     }
00201
00202 };
00203
00204 } /* namespace position */
00205 } /* namespace util */
00206
00207 #endif /* POSITIONS_H_ */
```

11.208 utility/PositionUtils.cpp File Reference

The util::position::PositionUtils class source file.

`#include "PositionUtils.h"`
Include dependency graph for PositionUtils.cpp:



Namespaces

- namespace util

  The utility namespace.
- namespace util::position

  The position utility namespace.

11.208.1 Detailed Description

The util::position::PositionUtils class source file.

Version

0.6

Date

May 12, 2014 4:20:02 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

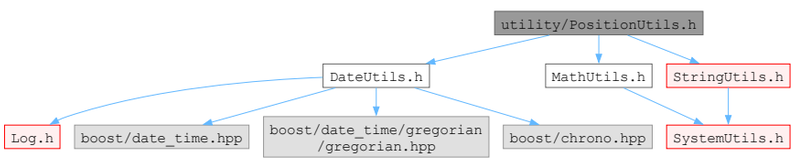
This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.209 utility/PositionUtils.h File Reference

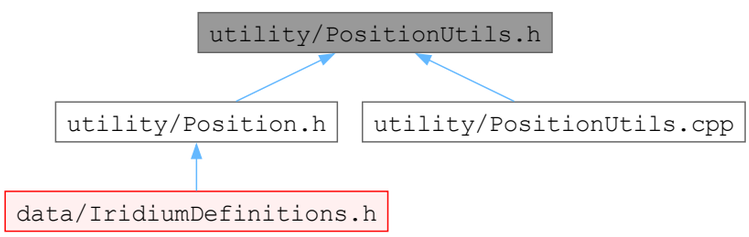
The util::position::PositionUtils class header file.

```
#include "DateUtils.h"
#include "MathUtils.h"
#include "StringUtils.h"
```

Include dependency graph for PositionUtils.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::position::PositionUtils

  Class providing library of useful functions for manipulating and processing geographic positions.

Namespaces

- namespace util

  The utility namespace.
- namespace util::position

  The position utility namespace.

Macros

- #define UTIL_POS_INVALID_LAT 99.99999
  Default/fill value for invalid position latitude.
- #define UTIL_POS_INVALID_LON 999.99999
  Default/fill value for invalid position longitude.
- #define UTIL_POS_INVALID_STDERR 9999.99
  Default/fill value for invalid position standard error.
- #define UTIL_POS_INVALID_VAR 99.99
  Default/fill value for invalid position variance.

11.209.1 Detailed Description

The util::position::PositionUtils class header file.

Version

0.6

Date

May 12, 2014 4:20:02 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.209.2 Macro Definition Documentation

11.209.2.1 UTIL_POS_INVALID_LAT

```
#define UTIL_POS_INVALID_LAT 99.99999
```

Default/fill value for invalid position latitude.

Referenced by util::xmsg::XGPS::DecodeXMsgData().

11.209.2.2 UTIL_POS_INVALID_LON

```
#define UTIL_POS_INVALID_LON 999.99999
```

Default/fill value for invalid position longitude.

Referenced by util::xmsg::XGPS::DecodeXMsgData().

11.209.2.3 UTIL_POS_INVALID_STDERR

```
#define UTIL_POS_INVALID_STDERR 9999.99
```

Default/fill value for invalid position standard error.

11.209.2.4 UTIL_POS_INVALID_VAR

```
#define UTIL_POS_INVALID_VAR 99.99
```

Default/fill value for invalid position variance.

11.210 PositionUtils.h

Go to the documentation of this file.

```
00001 /** The @ref util::position::PositionUtils class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       May 12, 2014 4:20:02 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef POSITIONUTILITIES_H_
00024 #define POSITIONUTILITIES_H_
00025
00026 /* Includes */
00027
00028 // local
00029 #include "DateUtils.h"
00030 #include "MathUtils.h"
00031 #include "StringUtils.h"
00032
00033 /* Definitions */
00034
00035 /** Default/fill value for invalid position latitude. */
00036 #define UTIL_POS_INVALID_LAT        99.99999
00037
00038 /** Default/fill value for invalid position longitude. */
00039 #define UTIL_POS_INVALID_LON        999.99999
00040
00041 /** Default/fill value for invalid position standard error. */
00042 #define UTIL_POS_INVALID_STDERR     9999.99
00043
00044 /** Default/fill value for invalid position variance. */
00045 #define UTIL_POS_INVALID_VAR        99.99
00046
00047 namespace util {
00048 namespace position {
00049
00050 using boost::format;
00051 using std::ostringstream;
00052 using std::string;
00053
00054 /* Class Definition */
00055
00056 /** Class providing library of useful functions for manipulating and processing geographic
00057  *  positions.
00058  *
00059  */
00060 class PositionUtils: public date::DateUtils, public math::MathUtils {
00061
00062 private:
00063
00064     /* Data Members */
00065
00066     /** The equatorial radius of the earth, in kilometers. */
00067     static const double EARTH_RADIUS_KM_EQUATORIAL;
00068
00069     /** The polar radius of the earth, in kilometers. */
00070     static const double EARTH_RADIUS_KM_POLAR;
00071
00072 public:
00073
00074     /* Member Functions */
00075
00076     /** Generate an IS06709 string representation of the position given by (@p lat, @p lon).
00077      *
00078      *  @param[in]      lat         The latitude of the position, in degrees North.
00079      *  @param[in]      lon         The longitude of the position, in degrees East.
00080      *
00081      *  @return         An IS06709 string representation of the position given by (@p lat, @p lon).
00082      */
00083     template<typename T> static std::string GenerateIso6709String(T lat, T lon) {
00084         ostringstream os;
00085         static const format fmt("%+.5f%+.5f");
00086         os << format(fmt) % lat % lon;
00087         return os.str();
00088     }
00089
00090     /** Get the approximate surface altitude or height of earth for a given latitude, in kilometers.
```

```
00091      *
00092      *  @warning        This method provides approximate surface height estimates only! This
00093      *                  approximation is calculated from a reference ellipsoid -- a
00094      *                  mathematically-defined surface that approximates the geoid, the truer
00095      *                  figure of the Earth.
00096      *
00097      *  @param[in]      lat         The geographic latitude of interest, in decimal degrees.
00098      *
00099      *  @return         The approximate surface altitude or height of earth for the given latitude,
00100     *                  in kilometers.
00101     */
00102    template<typename T> static double CalculateEllipsoidHeight(T lat) {
00103        if(lat > 90) lat = 90;
00104        if(lat < -90) lat = -90;
00105        return cos(degToRad(lat)) * (EARTH_RADIUS_KM_EQUATORIAL - EARTH_RADIUS_KM_POLAR)
00106                + EARTH_RADIUS_KM_POLAR;
00107    }
00108
00109    /** Get the geographical distance between two position, in kilometers, from the positions'
00110     *  locations, in decimal degrees.
00111     *
00112     *  @warning        This method provides approximate geographic estimates only! This
00113     *                  approximation is calculated from a reference ellipsoid -- a
00114     *                  mathematically-defined surface that approximates the geoid, the truer
00115     *                  figure of the Earth.
00116     *
00117     *  @note           This method uses the 'haversine' formula to calculate the great-circle
00118     *                  distance between two points - that is, the shortest distance over the
00119     *                  earth's surface - giving an 'as-the-crow-flies' distance between the points
00120     *                  (ignoring any hills they fly over, of course!).
00121     *
00122     *  @param[in]      lat1        The latitude of the geographic final position, in decimal
00123     *                              degrees North.
00124     *  @param[in]      lon1        The longitude of the geographic final position, in decimal
00125     *                              degrees East.
00126     *  @param[in]      lat0        The latitude of the geographic initial position, in decimal
00127     *                              degrees North.
00128     *  @param[in]      lon0        The longitude of the geographic initial position, in decimal
00129     *                              degrees East.
00130     *
00131     *  @return         The geographical distance between the two given positions, in kilometers.
00132     */
00133    template<typename T> static double CalculateGeographicDistance(T lat1, T lon1, T lat0, T lon0) {
00134        using namespace std;
00135
00136        // Convert to radians.
00137        double phi0 = degToRad(lat0);
00138        double phi1 = degToRad(lat1);
00139
00140        // Calculate angular changes.
00141        double delta_phi = degToRad(lat1 - lat0);
00142        double delta_lambda = degToRad(lon1 - lon0);
00143
00144        // Calculate distance.
00145        double a = sin(delta_phi / 2) * sin(delta_phi / 2)
00146                + cos(phi0) * cos(phi1) * sin(delta_lambda / 2) * sin(delta_lambda / 2);
00147        double c = 2 * atan2(sqrt(a), sqrt(1 - a));
00148        double d = CalculateEllipsoidHeight((lat1 + lat0) / 2) * c;
00149
00150        return d;
00151    }
00152
00153    /** Check position validity.
00154     *
00155     * @param[in]       lat         Latitude of position.
00156     * @param[in]       lon         Longitude of position.
00157     *
00158     * @return          TRUE if (lat, lon) represents a valid geographical position, or FALSE
00159     *                  otherwise.
00160     */
00161    template<typename T> static bool IsPositionValid(T lat, T lon) {
00162        return lat >= -90 && lat <= 90 && lon >= -180 && lon <= 180;
00163    }
00164 };
00165
00166
00167 } /* namespace position */
00168 } /* namespace util */
00169 #endif /* POSITIONUTILITIES_H_ */
```

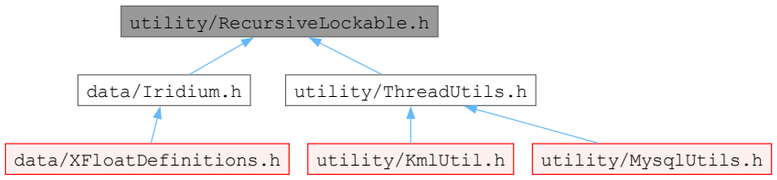
11.211 utility/RecursiveLockable.h File Reference

The util::thread::RecursiveLockable class header file.

```
#include "Log.h"
#include <boost/thread/locks.hpp>
#include <boost/thread/lock_guard.hpp>
#include <boost/thread/recursive_mutex.hpp>
```

Include dependency graph for RecursiveLockable.h:

This graph shows which files directly or indirectly include this file:



Classes

- class util::thread::RecursiveLockable

  A recursively lockable class.

Namespaces

- namespace util

  The utility namespace.
- namespace util::thread

  The thread utility namespace.

11.211.1 Detailed Description

The util::thread::RecursiveLockable class header file.

Version

0.6

Date

Oct 18, 2014 12:28:02 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.212 RecursiveLockable.h

Go to the documentation of this file.

```
00001 /** The @ref util::thread::RecursiveLockable class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Oct 18, 2014 12:28:02 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef RECURSIVELOCKABLE_H_
00024 #define RECURSIVELOCKABLE_H_
00025
00026 /* Includes */
00027
00028 // local
00029 #include "Log.h"
00030
00031 // boost
00032 #include <boost/thread/locks.hpp>
00033 #include <boost/thread/lock_guard.hpp>
00034 #include <boost/thread/recursive_mutex.hpp>
00035
00036 namespace util {
00037 namespace thread {
00038
00039 using std::recursive_mutex;
00040
00041 /* Class Definitions */
00042
00043 /** A recursively lockable class.
00044  *
00045  *  @note       This class uses a recursive mutex which means that it may be locked multiple times
00046  *              by the same thread.
00047  */
00048 class RecursiveLockable {
00049
00050 protected:
00051
00052     /* Data members */
00053
00054     /** Recursive mutual exclusion lock for synchronizing operations. */
00055     mutable recursive_mutex mutex_;
00056
00057 public:
00058
00059     /* Member Functions */
00060
00061     /** Default constructor.
00062      *
00063      */
00064     RecursiveLockable() {
00065         /* Do nothing. */
00066     }
00067
00068     /** Destructor.
00069      *
00070      */
00071     virtual ~RecursiveLockable() {
00072         /* Do nothing. */
00073     }
00074
00075     /** Lock the object.
00076      *
00077      *  @param[in]      block       If TRUE, then the calling thread will block until the lock
00078      *                              becomes available.
00079      *
00080      *  @throw          util::log::MessageException if an error occurs.
00081      *
00082      *  @return         TRUE if the lock was successfully obtained, or FALSE if block is FALSE and
00083      *                  another thread owns the lock on the mutex.
00084      */
00085     bool LockObject(bool block) {
00086         try {
00087             if(block) mutex_.lock();
00088             else return mutex_.try_lock();
00089             return true;
00090         }CATCH_BLOCK("Failed to lock recursive lockable object!")
00091         return false;
00092     }
00093
00094     /** Unlock the object.
00095      *
00096      *  @throw          util::log::MessageException if an error occurs.
00097      */
00098     void UnlockObject() {
00099         try {
00100             mutex_.unlock();
00101         }CATCH_BLOCK("Failed to unlock recursive lockable object!")
00102     }
00103 };
```

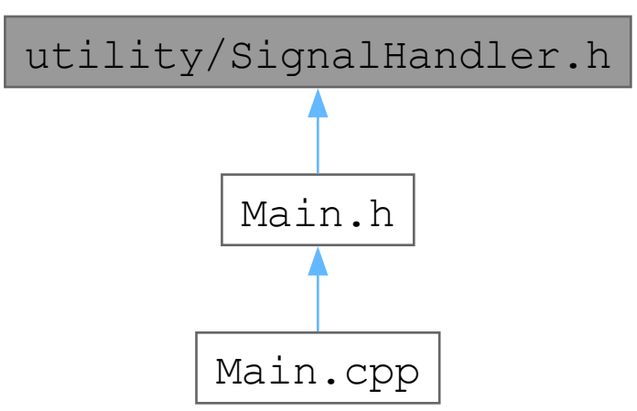
```
00104
00105 } /* namespace thread */
00106 } /* namespace util */
00107 #endif /* RECURSIVELOCKABLE_H_ */
```

11.213 utility/SignalHandler.h File Reference

The The util::sig::SignalHandler class header file.

`#include "Log.h"`
Include dependency graph for SignalHandler.h:

This graph shows which files directly or indirectly include this file:



Classes

- class util::sig::SignalHandler
  Class providing function for handling POSIX style system signals.

Namespaces

- namespace util
  The utility namespace.
- namespace util::sig
  The system signals utility namespace.

11.213.1 Detailed Description

The The util::sig::SignalHandler class header file.

Version

0.6

Date

Oct 18, 2014 12:49:07 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.214 SignalHandler.h

Go to the documentation of this file.

```
00001 /** The The @ref util::sig::SignalHandler class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Oct 18, 2014 12:49:07 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef SIGNALHANDLER_H_
00024 #define SIGNALHANDLER_H_
00025
00026 /* Includes */
00027
00028 #include "Log.h"
00029
00030 /* Definitions */
00031
00032 namespace util {
00033 namespace sig {
00034
00035 /* Class Definitions */
00036
00037 /** Class providing function for handling POSIX style system signals.
00038  *
00039  *  This class provides functions for graceful shutdown, triggered by catchable system signals.
00040  *
00041  */
00042 class SignalHandler {
00043
00044 public:
00045
00046     /* Member Functions */
00047
00048     /** Register signal handlers functions for UNIX-style system signals.
00049      *
00050      *  @exception      None. Never throws.
00051      *
00052      */
00053     static void RegisterSignalHandlers() ATTR_NO_EXCEPTIONS {
00054         try {
00055             /* Exit Signal Handlers */
00056             lout() << "Registering system signal handlers...";
00057
00058             ldbg() << "Registering SIGABRT handler...";
00059             if(signal(SIGABRT, SignalHandler::ExitHandler) == SIG_ERR)
00060                 EXC_THROW("Failed to register signal handler for SIGABRT!");
00061
00062             ldbg() << "Registering SIGINT handler...";
00063             if(signal(SIGINT, SignalHandler::ExitHandler) == SIG_ERR)
00064                 EXC_THROW("Failed to register signal handler for SIGINT!");
00065
00066             ldbg() << "Registering SIGQUIT handler...";
00067             if(signal(SIGQUIT, SignalHandler::ExitHandler) == SIG_ERR)
00068                 EXC_THROW("Failed to register signal handler for SIGQUIT!");
00069
00070             ldbg() << "Registering SIGTERM handler...";
00071             if(signal(SIGTERM, SignalHandler::ExitHandler) == SIG_ERR)
00072                 EXC_THROW("Failed to register signal handler for SIGTERM!");
00073
00074             ldbg() << "Registering SIGTSTP handler...";
00075             if(signal(SIGTSTP, SignalHandler::ExitHandler) == SIG_ERR)
00076                 EXC_THROW("Failed to register signal handler for SIGTSTP!");
00077
00078             linf() << "Done. Successfully registered system signal handlers.";
```

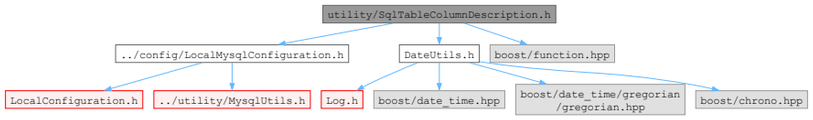
```
00079         }CATCH_BLOCK("Failed to register signal handlers!")
00080     }
00081
00082     static void ExitHandler(int signo);
00083 };
00084
00085 } /* namespace sig */
00086 } /* namespace util */
00087 #endif /* SIGNALHANDLER_H_ */
```

11.215 utility/SqlTableColumnDescription.h File Reference

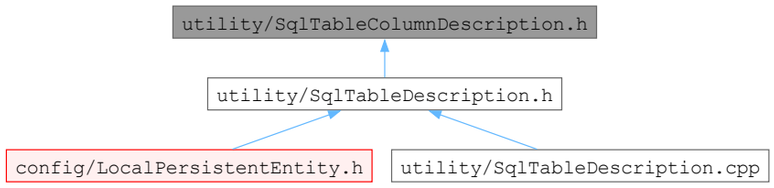
The util::mysql::SqlTableColumnDescription class header file.

```
#include "../config/LocalMysqlConfiguration.h"
#include "DateUtils.h"
#include <boost/function.hpp>
```

Include dependency graph for SqlTableColumnDescription.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::mysql::Column_bool< C >
  Template column description class for value of type bool.
- class util::mysql::Column_char< C >
  Template column description class for value of type double.
- class util::mysql::Column_double< C >
  Template column description class for value of type double.
- class util::mysql::Column_filesystem_path< C >
  Template column description class for value of type path.
- class util::mysql::Column_float< C >
  Template column description class for value of type float.
- class util::mysql::Column_int< C >
  Template column description class for value of type int.
- class util::mysql::Column_int16_t< C >
  Template column description class for value of type int16_t.
- class util::mysql::Column_long< C >
  Template column description class for value of type long.
- class util::mysql::Column_posix_time< C >
  Template column description class for value of type boost::posix_time::ptime.
- class util::mysql::Column_string< C >
  Template column description class for value of type string.
- class util::mysql::Column_uint16_t< C >
  Template column description class for value of type uint16_t.
- class util::mysql::Column_uint8_t< C >
  Template column description class for value of type uint8_t.
- class util::mysql::Column_vector_uint8_t< C >

Template column description class for value of type vector<uint8_t>.

- class util::mysql::SqlTableColumnDescription< C, T >

  The `SqlTableColumnDescription` class is a template class used to describe an SQL table column for class `C` and data member of type `T`.

- class util::mysql::SqlTableColumnDescriptionBase

  An abstract base class for the description of an SQL database table column.

Namespaces

- namespace util

  The utility namespace.

- namespace util::mysql

  The MySQL utility namespace.

Macros

- #define sql_ci16(T, ...) std::make_shared<util::mysql::Column_int16_t<T> >(__VA_ARGS__)

  Macro definition of SQL table column of type int16_t.

- #define sql_colb(T, ...) std::make_shared<util::mysql::Column_bool<T> >(__VA_ARGS__)

  Macro definition of SQL table column of type bool.

- #define sql_cold(T, ...) std::make_shared<util::mysql::Column_double<T> >(__VA_ARGS__)

  Macro definition of SQL table column of type double.

- #define sql_colf(T, ...) std::make_shared<util::mysql::Column_float<T> >(__VA_ARGS__)

  Macro definition of SQL table column of type float.

- #define sql_coli(T, ...) std::make_shared<util::mysql::Column_int<T> >(__VA_ARGS__)

  Macro definition of SQL table column of type int.

- #define sql_coll(T, ...) std::make_shared<util::mysql::Column_long<T> >(__VA_ARGS__)

  Macro definition of SQL table column of type long.

- #define sql_colp(T, ...) std::make_shared<util::mysql::Column_filesystem_path<T> >(__VA_ARGS__)

  Macro definition of SQL table column of type boost::filesystem::path.

- #define sql_cols(T, ...) std::make_shared<util::mysql::Column_string<T> >(__VA_ARGS__)

  Macro definition of SQL table column of type string.

- #define sql_colt(T, ...) std::make_shared<util::mysql::Column_posix_time<T> >(__VA_ARGS__)

  Macro definition of SQL table column of type boost::posix_time::ptime.

- #define sql_colv(T, ...) std::make_shared<util::mysql::Column_vector_uint8_t<T> >(__VA_ARGS__)

  Macro definition of SQL table column of type vector<uint8_t>.

- #define sql_cou8(T, ...) std::make_shared<util::mysql::Column_uint8_t<T> >(__VA_ARGS__)

  Macro definition of SQL table column of type uint8_t.

- #define sql_cu16(T, ...) std::make_shared<util::mysql::Column_uint16_t<T> >(__VA_ARGS__)

  Macro definition of SQL table column of type uint16_t.

Enumerations

- enum util::mysql::ColumDataType_t {
  COLUMN_DATA_TYPE_UNKOWN ,
  COLUMN_DATA_TYPE_BOOL ,
  COLUMN_DATA_TYPE_CHAR ,
  COLUMN_DATA_TYPE_DOUBLE ,
  COLUMN_DATA_TYPE_FILESYSTEM_PATH ,
  COLUMN_DATA_TYPE_FLOAT ,
  COLUMN_DATA_TYPE_INT ,
  COLUMN_DATA_TYPE_INT16_T ,
  COLUMN_DATA_TYPE_LONG ,
  COLUMN_DATA_TYPE_POSIX_TIME ,
  COLUMN_DATA_TYPE_STRING ,
  COLUMN_DATA_TYPE_TEST ,
  COLUMN_DATA_TYPE_UINT16_T ,
  COLUMN_DATA_TYPE_UINT8_T ,
  COLUMN_DATA_TYPE_VECTOR_UINT8_T }

  The ColumnDescription class is used to describe the attributes of a particular SQL table column including its name, type, and whether or not the column is part of the primary key for the SQL table.

11.215.1 Detailed Description

The util::mysql::SqlTableColumnDescription class header file.

Version

0.6

Date

Oct 18, 2014 12:37:26 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.215.2 Macro Definition Documentation

11.215.2.1 sql_ci16

```
#define sql_ci16(
              T,
              ...) std::make_shared<util::mysql::Column_int16_t<T> >(__VA_ARGS__)
```

Macro definition of SQL table column of type int16_t.

11.215.2.2 sql_colb

```
#define sql_colb(
              T,
              ...) std::make_shared<util::mysql::Column_bool<T> >(__VA_ARGS__)
```

Macro definition of SQL table column of type bool.

11.215.2.3 sql_cold

```
#define sql_cold(
              T,
              ...) std::make_shared<util::mysql::Column_double<T> >(__VA_ARGS__)
```

Macro definition of SQL table column of type double.

11.215.2.4 sql_colf

```
#define sql_colf(
              T,
              ...) std::make_shared<util::mysql::Column_float<T> >(__VA_ARGS__)
```

Macro definition of SQL table column of type float.

11.215.2.5 sql_coli

```
#define sql_coli(
              T,
              ...) std::make_shared<util::mysql::Column_int<T> >(__VA_ARGS__)
```

Macro definition of SQL table column of type int.

11.215.2.6 sql_coll

```
#define sql_coll(
              T,
              ...) std::make_shared<util::mysql::Column_long<T> >(__VA_ARGS__)
```

Macro definition of SQL table column of type long.

11.215.2.7 sql_colp

```
#define sql_colp(
              T,
              ...) std::make_shared<util::mysql::Column_filesystem_path<T> >(__VA_ARGS__)
```

Macro definition of SQL table column of type boost::filesystem::path.

11.215.2.8 sql_cols

```
#define sql_cols(
              T,
              ...) std::make_shared<util::mysql::Column_string<T> >(__VA_ARGS__)
```

Macro definition of SQL table column of type string.

11.215.2.9 sql_colt

```
#define sql_colt(
              T,
              ...) std::make_shared<util::mysql::Column_posix_time<T> >(__VA_ARGS__)
```

Macro definition of SQL table column of type boost::posix_time::ptime.

11.215.2.10 sql_colv

```
#define sql_colv(
              T,
              ...) std::make_shared<util::mysql::Column_vector_uint8_t<T> >(__VA_ARGS__)
```

Macro definition of SQL table column of type vector<uint8_t>.

11.215.2.11 sql_cou8

```
#define sql_cou8(
              T,
              ...) std::make_shared<util::mysql::Column_uint8_t<T> >(__VA_ARGS__)
```

Macro definition of SQL table column of type uint8_t.

11.215.2.12 sql_cu16

```
#define sql_cu16(
              T,
              ...) std::make_shared<util::mysql::Column_uint16_t<T> >(__VA_ARGS__)
```

Macro definition of SQL table column of type uint16_t.

11.216 SqlTableColumnDescription.h

Go to the documentation of this file.

```
00001 /** The @ref util::mysql::SqlTableColumnDescription class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Oct 18, 2014 12:37:26 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef COLUMNDESCRIPTION_H_
00024 #define COLUMNDESCRIPTION_H_
00025
00026 /* Includes */
00027
00028 // local
00029 #include "../config/LocalMysqlConfiguration.h"
00030 #include "DateUtils.h"
00031
00032 // boost
00033 #include <boost/function.hpp>
00034
00035 /** Macro definition of SQL table column of type bool. */
00036 #define sql_colb(T, ...) std::make_shared<util::mysql::Column_bool<T> >(__VA_ARGS__)
00037
00038 /** Macro definition of SQL table column of type double. */
00039 #define sql_cold(T, ...) std::make_shared<util::mysql::Column_double<T> >(__VA_ARGS__)
00040
00041 /** Macro definition of SQL table column of type boost::filesystem::path. */
00042 #define sql_colp(T, ...) std::make_shared<util::mysql::Column_filesystem_path<T> >(__VA_ARGS__)
00043
00044 /** Macro definition of SQL table column of type float. */
00045 #define sql_colf(T, ...) std::make_shared<util::mysql::Column_float<T> >(__VA_ARGS__)
00046
00047 /** Macro definition of SQL table column of type int. */
00048 #define sql_coli(T, ...) std::make_shared<util::mysql::Column_int<T> >(__VA_ARGS__)
00049
00050 /** Macro definition of SQL table column of type int16_t. */
00051 #define sql_ci16(T, ...) std::make_shared<util::mysql::Column_int16_t<T> >(__VA_ARGS__)
00052
00053 /** Macro definition of SQL table column of type long. */
00054 #define sql_coll(T, ...) std::make_shared<util::mysql::Column_long<T> >(__VA_ARGS__)
00055
00056 /** Macro definition of SQL table column of type boost::posix_time::ptime. */
00057 #define sql_colt(T, ...) std::make_shared<util::mysql::Column_posix_time<T> >(__VA_ARGS__)
00058
00059 /** Macro definition of SQL table column of type string. */
00060 #define sql_cols(T, ...) std::make_shared<util::mysql::Column_string<T> >(__VA_ARGS__)
00061
00062 /** Macro definition of SQL table column of type uint16_t. */
00063 #define sql_cu16(T, ...) std::make_shared<util::mysql::Column_uint16_t<T> >(__VA_ARGS__)
00064
00065 /** Macro definition of SQL table column of type uint8_t. */
00066 #define sql_cou8(T, ...) std::make_shared<util::mysql::Column_uint8_t<T> >(__VA_ARGS__)
00067
00068 /** Macro definition of SQL table column of type vector<uint8_t>. */
00069 #define sql_colv(T, ...) std::make_shared<util::mysql::Column_vector_uint8_t<T> >(__VA_ARGS__)
00070
00071 namespace util {
00072 namespace mysql {
00073
00074 using boost::filesystem::path;
00075 using boost::function;
00076 using boost::posix_time::ptime;
00077 using boost::posix_time::not_a_date_time;
00078 using boost::posix_time::time_from_string;
00079 using boost::posix_time::to_iso_string;
00080 using std::ostringstream;
00081 using std::vector;
00082 using sql::PreparedStatement;
00083 using sql::SQLString;
00084 using sql::ResultSet;
00085
00086 /* Class Definition */
00087
```

```
00088 /** The ColumnDescription class is used to describe the attributes of a particular SQL table column
00089  * including its name, type, and whether or not the column is part of the primary key for the SQL
00090  * table. A collection of ColumnDescrition object may be used to fully describe the attributes of
00091  * one or more SQL database tables.
00092  */
00093 typedef enum ColumnDataType {
00094     COLUMN_DATA_TYPE_UNKOWN,
00095     COLUMN_DATA_TYPE_BOOL,
00096     COLUMN_DATA_TYPE_CHAR,
00097
00098     COLUMN_DATA_TYPE_DOUBLE,
00099     COLUMN_DATA_TYPE_FILESYSTEM_PATH,
00100     COLUMN_DATA_TYPE_FLOAT,
00101     COLUMN_DATA_TYPE_INT,
00102     COLUMN_DATA_TYPE_INT16_T,
00103     COLUMN_DATA_TYPE_LONG,
00104     COLUMN_DATA_TYPE_POSIX_TIME,
00105     COLUMN_DATA_TYPE_STRING,
00106     COLUMN_DATA_TYPE_TEST,
00107     COLUMN_DATA_TYPE_UINT16_T,
00108     COLUMN_DATA_TYPE_UINT8_T,
00109     COLUMN_DATA_TYPE_VECTOR_UINT8_T,
00110 } ColumDataType_t;
00111
00112 /** An abstract base class for the description of an SQL database table column.
00113  *
00114  *  This base class is necessary for utilizing polymorphism such that a pointers to classes that
00115  *  inherit from this class may be stored in a single vector.
00116  *
00117  */
00118 class SqlTableColumnDescriptionBase {
00119
00120 private:
00121
00122     /* Data Members */
00123
00124     /** The name of the SQL table column. */
00125     string column_name_;
00126
00127     /** The index of the SQL table column.
00128      *
00129      */
00130     size_t column_index_;
00131
00132     /** The enumerated SQL table column data type. */
00133     ColumDataType_t data_type_;
00134
00135     /** Indicates whether the column is part of the primary key for the SQL table.
00136      *
00137      */
00138     bool is_primary_key_;
00139
00140 public:
00141
00142     /* Member Functions */
00143
00144     /** The parameterized constructor.
00145      *
00146      *  @param[in]  column_name     The name of the SQL table column.
00147      *  @param[in]  type            They enumerated SQL table column type.
00148      *  @param[in]  is_primary_key  TRUE if this column is a primary key for the SQL table, or
00149      *                              FALSE otherwise.
00150      */
00151     SqlTableColumnDescriptionBase(string column_name, ColumDataType_t type, bool is_primary_key) :
00152                     column_name_(column_name),
00153                     column_index_(0),
00154                     data_type_(type),
00155                     is_primary_key_(is_primary_key) {
00156         /* Do nothing. */
00157     }
00158
00159     /** The destructor.
00160      *
00161      */
00162     virtual ~SqlTableColumnDescriptionBase() {
00163         /* Do nothing. */
00164     }
00165
00166     /** Get the index of the SQL table column.
00167      *
00168      *  @return     The index of the SQL table column.
00169      */
00170     size_t column_index() const {
00171         return column_index_;
00172     }
00173
00174     /** Get the name of the SQL table column.
00175      *
00176      *  @return     The name of the SQL table column.
00177      */
00178     const string& column_name() const {
00179         return column_name_;
00180     }
00181
00182     /** Get the name of the SQL table column to be used in a @c SELECT query.
00183      *
00184      *  @return     The name of the SQL table column to be used in a @c SELECT query.
00185      */
00186     string column_select_name() const {
00187         return data_type_ == COLUMN_DATA_TYPE_VECTOR_UINT8_T ?
00188                 "HEX(" + column_name_ + ") AS " + column_name_ :
00189                 column_name_;
00190     }
00191
00192     /** Get the the enumerated SQL column data type.
00193      *
00194      *  @return     The the enumerated SQL column data type.
00195      */
```

```
00196     ColumDataType_t data_type() const {
00197         return data_type_;
00198     }
00199
00200     /** Is this column a primary key for the corresponding SQL database table?
00201      *
00202      *  @return     TRUE if this column is a primary key for the corresponding SQL database table,
00203      *              or FALSE otherwise.
00204      */
00205     bool is_primary_key() const {
00206         return is_primary_key_;
00207     }
00208
00209     /** Set the index of this SQL table column.
00210      *
00211      *  @param[in]  idx     The index to set.
00212      */
00213     void set_column_index(size_t idx) {
00214         column_index_ = idx;
00215     }
00216 };
00217
00218 /** The @p SqlTableColumnDescription class is a template class used to describe an SQL table
00219  *  column for class @p C and data member of type @p T.
00220  *
00221  */
00222 template<class C, typename T> class SqlTableColumnDescription:
00223         public SqlTableColumnDescriptionBase {
00224
00225 private:
00226
00227     /** The accessor function object wrapper.
00228      *
00229      *  This object is used as a wrapper for the accessor method of the template class C. It is
00230      *  used to retrieve the value of the SQL table column from the object for generating an
00231      *  INSERT or UPDATE statement.
00232      */
00233     function<T(const C&)> accessor_;
00234
00235     /** The mutator function object wrapper.
00236      *
00237      *  This object is used as a wrapper for the mutator method of the template class C. It is
00238      *  used to set the value of an object from the result set of an SQL SELECT query
00239      *  statement.
00240      */
00241     function<void(C&, T)> mutator_;
00242
00243 protected:
00244
00245     /** Get the object member value from the object.
00246      *
00247      *  @param[in]      obj         The object of type @p C.
00248      *
00249      *  @return         The value of type @p T of the object member corresponding to this SQL
00250      *                  column.
00251      */
00252     T GetValueFromObject(const C& obj) const {
00253         return accessor_(obj);
00254     }
00255
00256     /** Set the object member value.
00257      *
00258      *  @param[in]      obj         The object of type @p C.
00259      *  @param[in]      value       The value of type @p T, to set.
00260      */
00261     void SetObjectValue(C& obj, const T& value) const {
00262         mutator_(obj, value);
00263     }
00264
00265 public:
00266
00267     /** The parameterized constructor.
00268      *
00269      *  @param[in]  column_name     The name of the SQL table column.
00270      *  @param[in]  data_type       They enumerated SQL table column type.
00271      *  @param[in]  accessor        A pointer to the accessor method for acquiring the value of the
00272      *                              corresponding object field.
00273      *  @param[in]  mutator         A pointer to the mutator method for setting the value of the
00274      *                              corresponding object field.
00275      *  @param[in]  is_primary_key  TRUE if this column is a primary key for the SQL table, or
00276      *                              FALSE otherwise.
00277      */
00278     SqlTableColumnDescription(
00279             string column_name,
00280             ColumDataType_t data_type,
00281             T (C::*accessor)() const = NULL,
00282             void (C::*mutator)(T) = NULL,
00283             bool is_primary_key = false) :
00284                     SqlTableColumnDescriptionBase(column_name, data_type, is_primary_key),
00285                     accessor_(accessor),
00286                     mutator_(mutator) {
00287         /* Do nothing. */
00288     }
00289
00290     /** The parameterized constructor.
00291      *
00292      *  @param[in]  column_name     The name of the SQL table column.
00293      *  @param[in]  data_type       They enumerated SQL table column type.
00294      *  @param[in]  accessor        A pointer to the accessor method for acquiring the value of the
00295      *                              corresponding object field.
00296      *  @param[in]  mutator         A pointer to the mutator method for setting the value of the
00297      *                              corresponding object field.
00298      *  @param[in]  is_primary_key  TRUE if this column is a primary key for the SQL table, or
00299      *                              FALSE otherwise.
00300      */
00301     SqlTableColumnDescription(
00302             string column_name,
00303             ColumDataType_t data_type,
```

```
00304             T (C::*accessor)() const = NULL,
00305             void (C::*mutator)(const T&) = NULL,
00306             bool is_primary_key = false) :
00307                     SqlTableColumnDescriptionBase(column_name, data_type, is_primary_key),
00308                     accessor_(accessor),
00309                     mutator_(mutator) {
00310         /* Do nothing. */
00311     }
00312
00313     /** The parameterized constructor.
00314      *
00315      *  @param[in]  column_name     The name of the SQL table column.
00316      *  @param[in]  data_type       They enumerated SQL table column type.
00317      *  @param[in]  accessor        A pointer to the accessor method for acquiring the value of the
00318      *                              corresponding object field.
00319      *  @param[in]  mutator         A pointer to the mutator method for setting the value of the
00320      *                              corresponding object field.
00321      *  @param[in]  is_primary_key  TRUE if this column is a primary key for the SQL table, or
00322      *                              FALSE otherwise.
00323      */
00324     SqlTableColumnDescription(
00325             string column_name,
00326             ColumDataType_t data_type,
00327             const T& (C::*accessor)() const = NULL,
00328             void (C::*mutator)(const T&) = NULL,
00329             bool is_primary_key = false) :
00330                     SqlTableColumnDescriptionBase(column_name, data_type, is_primary_key),
00331                     accessor_(accessor),
00332                     mutator_(mutator) {
00333         /* Do nothing. */
00334     }
00335 };
00336
00337 /** Template column description class for value of type bool.
00338  *
00339  */
00340 template<class C> class Column_bool:
00341         public SqlTableColumnDescription<C, bool> {
00342
00343 public:
00344
00345     /** The parameterized constructor.
00346      *
00347      *  @param[in]  column_name     The name of the SQL table column.
00348      *  @param[in]  accessor        A pointer to the accessor method for acquiring the value of the
00349      *                              corresponding object field.
00350      *  @param[in]  mutator         A pointer to the mutator method for setting the value of the
00351      *                              corresponding object field.
00352      *  @param[in]  is_primary_key  TRUE if this column is a primary key for the SQL table, or
00353      *                              FALSE otherwise.
00354      */
00355     Column_bool(string column_name,
00356     bool (C::*accessor)() const = NULL, void (C::*mutator)(bool) = NULL,
00357     bool is_primary_key = false) :
00358                     SqlTableColumnDescription<C, bool>(
00359                             column_name,
00360                             COLUMN_DATA_TYPE_BOOL,
00361                             accessor,
00362                             mutator,
00363                             is_primary_key) {
00364         /* Do nothing. */
00365     }
00366
00367     /** Set the value in the field of the corresponding object from an SQL ResultSet Object.
00368      *
00369      *  @param[in,out]  obj         A reference to the object whose field value will be set.
00370      *  @param[in]      results     A reference to the ResultSet object.
00371      */
00372     void SetObjectValueFromResultSet(C& obj, const ResultSet& results) const {
00373         this->SetObjectValue(obj, results.getBoolean(this->column_name()));
00374     }
00375
00376     /** Set the value for the SQL PreparedStatement object from the value of the field of the
00377      *  corresponding object.
00378      *
00379      *  @param[in]      obj         A reference to the object whose field value will be read.
00380      *  @param[in,out]  stmt        A reference to the SQL PreparedStatement object whose column
00381      *                              value will be set.
00382      */
00383     void SetPreparedStatementValueFromObject(const C& obj, PreparedStatement& stmt) const {
00384         stmt.setBoolean(this->column_index(), this->GetValueFromObject(obj));
00385     }
00386 };
00387
00388 /** Template column description class for value of type double.
00389  *
00390  */
00391 template<class C> class Column_char:
00392         public SqlTableColumnDescription<C, char> {
00393
00394 public:
00395
00396     /** The parameterized constructor.
00397      *
00398      *  @param[in]  column_name     The name of the SQL table column.
00399      *  @param[in]  accessor        A pointer to the accessor method for acquiring the value of the
00400      *                              corresponding object field.
00401      *  @param[in]  mutator         A pointer to the mutator method for setting the value of the
00402      *                              corresponding object field.
00403      *  @param[in]  is_primary_key  TRUE if this column is a primary key for the SQL table, or
00404      *                              FALSE otherwise.
00405      */
00406     Column_char(
00407             string column_name,
00408             double (C::*accessor)() const = NULL,
00409             void (C::*mutator)(double) = NULL,
00410             bool is_primary_key = false) :
00411                     SqlTableColumnDescription<C, double>(
```

```
00412                               column_name,
00413                               COLUMN_DATA_TYPE_DOUBLE,
00414                               accessor,
00415                               mutator,
00416                               is_primary_key) {
00417         /* Do nothing. */
00418     }
00419
00420     /** Set the value in the field of the corresponding object from an SQL ResultSet Object.
00421      *
00422      *  @param[in,out]  obj         A reference to the object whose field value will be set.
00423      *  @param[in]      results     A reference to the ResultSet object.
00424      */
00425     void SetObjectValueFromResultSet(C& obj, const ResultSet& results) const {
00426         string str(results.getString(this->column_name()).asStdString());
00427
00428         if(str.length() > 0)
00429
00430             this->SetObjectValue(obj, str.at(0));
00431
00432         else
00433
00434             this->SetObjectValue(obj, '\0');
00435     }
00436
00437     /** Set the value for the SQL PreparedStatement object from the value of the field of the
00438      *  corresponding object.
00439      *
00440      *  @param[in]      obj         A reference to the object whose field value will be read.
00441      *  @param[in,out]  stmt        A reference to the SQL PreparedStatement object whose column
00442      *                              value will be set.
00443      */
00444     void SetPreparedStatementValueFromObject(const C& obj, PreparedStatement& stmt) const {
00445         char c[2];
00446
00447         memset(c, 0, sizeof(c));
00448
00449         c[0] = this->GetValueFromObject(obj);
00450
00451         stmt.setString(this->column_index(), string(c));
00452     }
00453 };
00454
00455 /** Template column description class for value of type double.
00456  *
00457  */
00458 template<class C> class Column_double:
00459         public SqlTableColumnDescription<C, double> {
00460
00461 public:
00462
00463     /** The parameterized constructor.
00464      *
00465      *  @param[in]  column_name     The name of the SQL table column.
00466      *  @param[in]  accessor        A pointer to the accessor method for acquiring the value of the
00467      *                              corresponding object field.
00468      *  @param[in]  mutator         A pointer to the mutator method for setting the value of the
00469      *                              corresponding object field.
00470      *  @param[in]  is_primary_key  TRUE if this column is a primary key for the SQL table, or
00471      *                              FALSE otherwise.
00472      */
00473     Column_double(
00474             string column_name,
00475             double (C::*accessor)() const = NULL,
00476             void (C::*mutator)(double) = NULL,
00477             bool is_primary_key = false) :
00478                     SqlTableColumnDescription<C, double>(
00479                             column_name,
00480                             COLUMN_DATA_TYPE_DOUBLE,
00481                             accessor,
00482                             mutator,
00483                             is_primary_key) {
00484         /* Do nothing. */
00485     }
00486
00487     /** Set the value in the field of the corresponding object from an SQL ResultSet Object.
00488      *
00489      *  @param[in,out]  obj         A reference to the object whose field value will be set.
00490      *  @param[in]      results     A reference to the ResultSet object.
00491      */
00492     void SetObjectValueFromResultSet(C& obj, const ResultSet& results) const {
00493         this->SetObjectValue(obj, results.getDouble(this->column_name()));
00494     }
00495
00496     /** Set the value for the SQL PreparedStatement object from the value of the field of the
00497      *  corresponding object.
00498      *
00499      *  @param[in]      obj         A reference to the object whose field value will be read.
00500      *  @param[in,out]  stmt        A reference to the SQL PreparedStatement object whose column
00501      *                              value will be set.
00502      */
00503     void SetPreparedStatementValueFromObject(const C& obj, PreparedStatement& stmt) const {
00504         stmt.setDouble(this->column_index(), this->GetValueFromObject(obj));
00505     }
00506 };
00507
00508 /** Template column description class for value of type path.
00509  *
00510  */
00511 template<class C> class Column_filesystem_path:
00512         public SqlTableColumnDescription<C, path> {
00513
00514 public:
00515
00516     /** The parameterized constructor.
00517      *
00518      *  @param[in]  column_name     The name of the SQL table column.
00519      *  @param[in]  accessor        A pointer to the accessor method for acquiring the value of the
```

```
00520      *                               corresponding object field.
00521      *  @param[in]  mutator          A pointer to the mutator method for setting the value of the
00522      *                               corresponding object field.
00523      *  @param[in]  is_primary_key   TRUE if this column is a primary key for the SQL table, or
00524      *                               FALSE otherwise.
00525      */
00526     Column_filesystem_path(
00527             string column_name,
00528             const path& (C::*accessor)() const = NULL,
00529             void (C::*mutator)(const path&) = NULL,
00530             bool is_primary_key = false) :
00531                     SqlTableColumnDescription<C, path>(
00532                             column_name,
00533                             COLUMN_DATA_TYPE_FILESYSTEM_PATH,
00534                             accessor,
00535                             mutator,
00536                             is_primary_key) {
00537         /* Do nothing. */
00538     }
00539
00540     /** The parameterized constructor.
00541      *
00542      *  @param[in]  column_name     The name of the SQL table column.
00543      *  @param[in]  accessor        A pointer to the accessor method for acquiring the value of the
00544      *                              corresponding object field.
00545      *  @param[in]  mutator         A pointer to the mutator method for setting the value of the
00546      *                              corresponding object field.
00547      *  @param[in]  is_primary_key  TRUE if this column is a primary key for the SQL table, or
00548      *                              FALSE otherwise.
00549      */
00550     Column_filesystem_path(
00551             string column_name,
00552             path (C::*accessor)() const = NULL,
00553             void (C::*mutator)(const path&) = NULL,
00554             bool is_primary_key = false) :
00555                     SqlTableColumnDescription<C, path>(
00556                             column_name,
00557                             COLUMN_DATA_TYPE_FILESYSTEM_PATH,
00558                             accessor,
00559                             mutator,
00560                             is_primary_key) {
00561         /* Do nothing. */
00562     }
00563
00564     /** Set the value in the field of the corresponding object from an SQL ResultSet Object.
00565      *
00566      *  @param[in,out]  obj         A reference to the object whose field value will be set.
00567      *  @param[in]      results     A reference to the ResultSet object.
00568      */
00569     void SetObjectValueFromResultSet(C& obj, const ResultSet& results) const {
00570         this->SetObjectValue(obj, path(results.getString(this->column_name()).asStdString()));
00571     }
00572
00573     /** Set the value for the SQL PreparedStatement object from the value of the field of the
00574      *  corresponding object.
00575      *
00576      *  @param[in]      obj         A reference to the object whose field value will be read.
00577      *  @param[in,out]  stmt        A reference to the SQL PreparedStatement object whose column
00578      *                              value will be set.
00579      */
00580     void SetPreparedStatementValueFromObject(const C& obj, PreparedStatement& stmt) const {
00581         stmt.setString(this->column_index(), this->GetValueFromObject(obj).string());
00582     }
00583 };
00584
00585 /** Template column description class for value of type float.
00586  *
00587  */
00588 template<class C> class Column_float:
00589         public SqlTableColumnDescription<C, float> {
00590
00591 public:
00592
00593     /** The parameterized constructor.
00594      *
00595      *  @param[in]  column_name     The name of the SQL table column.
00596      *  @param[in]  accessor        A pointer to the accessor method for acquiring the value of the
00597      *                              corresponding object field.
00598      *  @param[in]  mutator         A pointer to the mutator method for setting the value of the
00599      *                              corresponding object field.
00600      *  @param[in]  is_primary_key  TRUE if this column is a primary key for the SQL table, or
00601      *                              FALSE otherwise.
00602      */
00603     Column_float(
00604             string column_name,
00605             float (C::*accessor)() const = NULL,
00606             void (C::*mutator)(float) = NULL,
00607             bool is_primary_key = false) :
00608                     SqlTableColumnDescription<C, float>(
00609                             column_name,
00610                             COLUMN_DATA_TYPE_FLOAT,
00611                             accessor,
00612                             mutator,
00613                             is_primary_key) {
00614         /* Do nothing. */
00615     }
00616
00617     /** Set the value in the field of the corresponding object from an SQL ResultSet Object.
00618      *
00619      *  @param[in,out]  obj         A reference to the object whose field value will be set.
00620      *  @param[in]      results     A reference to the ResultSet object.
00621      */
00622     void SetObjectValueFromResultSet(C& obj, const ResultSet& results) const {
00623         this->SetObjectValue(obj, results.getDouble(this->column_name()));
00624     }
00625
00626     /** Set the value for the SQL PreparedStatement object from the value of the field of the
00627      *  corresponding object.
```

```
00628      *
00629      *  @param[in]      obj         A reference to the object whose field value will be read.
00630      *  @param[in,out]  stmt        A reference to the SQL PreparedStatement object whose column
00631      *                              value will be set.
00632      */
00633     void SetPreparedStatementValueFromObject(const C& obj, PreparedStatement& stmt) const {
00634         stmt.setDouble(this->column_index(), this->GetValueFromObject(obj));
00635     }
00636 };
00637
00638 /** Template column description class for value of type int.
00639  *
00640  */
00641 template<class C> class Column_int:
00642         public SqlTableColumnDescription<C, int> {
00643
00644 public:
00645
00646     /** The parameterized constructor.
00647      *
00648      *  @param[in]  column_name     The name of the SQL table column.
00649      *  @param[in]  accessor        A pointer to the accessor method for acquiring the value of the
00650      *                              corresponding object field.
00651      *  @param[in]  mutator         A pointer to the mutator method for setting the value of the
00652      *                              corresponding object field.
00653      *  @param[in]  is_primary_key  TRUE if this column is a primary key for the SQL table, or
00654      *                              FALSE otherwise.
00655      */
00656     Column_int(
00657             string column_name,
00658             int (C::*accessor)() const = NULL,
00659             void (C::*mutator)(int) = NULL,
00660             bool is_primary_key = false) :
00661                     SqlTableColumnDescription<C, int>(
00662                             column_name,
00663                             COLUMN_DATA_TYPE_INT,
00664                             accessor,
00665                             mutator,
00666                             is_primary_key) {
00667         /* Do nothing. */
00668     }
00669
00670     /** Set the value in the field of the corresponding object from an SQL ResultSet Object.
00671      *
00672      *  @param[in,out]  obj         A reference to the object whose field value will be set.
00673      *  @param[in]      results     A reference to the ResultSet object.
00674      */
00675     void SetObjectValueFromResultSet(C& obj, const ResultSet& results) const {
00676         this->SetObjectValue(obj, results.getInt(this->column_name()));
00677     }
00678
00679     /** Set the value for the SQL PreparedStatement object from the value of the field of the
00680      *  corresponding object.
00681      *
00682      *  @param[in]      obj         A reference to the object whose field value will be read.
00683      *  @param[in,out]  stmt        A reference to the SQL PreparedStatement object whose column
00684      *                              value will be set.
00685      */
00686     void SetPreparedStatementValueFromObject(const C& obj, PreparedStatement& stmt) const {
00687         stmt.setInt(this->column_index(), this->GetValueFromObject(obj));
00688     }
00689 };
00690
00691 /** Template column description class for value of type int16_t.
00692  *
00693  */
00694 template<class C> class Column_int16_t:
00695         public SqlTableColumnDescription<C, int16_t> {
00696
00697 public:
00698
00699     /** The parameterized constructor.
00700      *
00701      *  @param[in]  column_name     The name of the SQL table column.
00702      *  @param[in]  accessor        A pointer to the accessor method for acquiring the value of the
00703      *                              corresponding object field.
00704      *  @param[in]  mutator         A pointer to the mutator method for setting the value of the
00705      *                              corresponding object field.
00706      *  @param[in]  is_primary_key  TRUE if this column is a primary key for the SQL table, or
00707      *                              FALSE otherwise.
00708      */
00709     Column_int16_t(
00710             string column_name,
00711             int16_t (C::*accessor)() const = NULL,
00712             void (C::*mutator)(int16_t) = NULL,
00713             bool is_primary_key = false) :
00714                     SqlTableColumnDescription<C, int16_t>(
00715                             column_name,
00716                             COLUMN_DATA_TYPE_INT16_T,
00717                             accessor,
00718                             mutator,
00719                             is_primary_key) {
00720         /* Do nothing. */
00721     }
00722
00723     /** Set the value in the field of the corresponding object from an SQL ResultSet Object.
00724      *
00725      *  @param[in,out]  obj         A reference to the object whose field value will be set.
00726      *  @param[in]      results     A reference to the ResultSet object.
00727      */
00728     void SetObjectValueFromResultSet(C& obj, const ResultSet& results) const {
00729         this->SetObjectValue(obj, results.getInt(this->column_name()));
00730     }
00731
00732     /** Set the value for the SQL PreparedStatement object from the value of the field of the
00733      *  corresponding object.
00734      *
00735      *  @param[in]      obj         A reference to the object whose field value will be read.
```

```
00736      *  @param[in,out]  stmt        A reference to the SQL PreparedStatement object whose column
00737      *                              value will be set.
00738      */
00739     void SetPreparedStatementValueFromObject(const C& obj, PreparedStatement& stmt) const {
00740         stmt.setInt(this->column_index(), this->GetValueFromObject(obj));
00741     }
00742 };
00743
00744 /** Template column description class for value of type long.
00745  *
00746  */
00747 template<class C> class Column_long:
00748         public SqlTableColumnDescription<C, long> {
00749
00750 public:
00751
00752     /** The parameterized constructor.
00753      *
00754      *  @param[in]  column_name     The name of the SQL table column.
00755      *  @param[in]  accessor        A pointer to the accessor method for acquiring the value of the
00756      *                              corresponding object field.
00757      *  @param[in]  mutator         A pointer to the mutator method for setting the value of the
00758      *                              corresponding object field.
00759      *  @param[in]  is_primary_key  TRUE if this column is a primary key for the SQL table, or
00760      *                              FALSE otherwise.
00761      */
00762     Column_long(
00763             string column_name,
00764             long (C::*accessor)() const = NULL,
00765             void (C::*mutator)(long) = NULL,
00766             bool is_primary_key = false) :
00767                     SqlTableColumnDescription<C, long>(
00768                             column_name,
00769                             COLUMN_DATA_TYPE_LONG,
00770                             accessor,
00771                             mutator,
00772                             is_primary_key) {
00773         /* Do nothing. */
00774     }
00775
00776     /** Set the value in the field of the corresponding object from an SQL ResultSet Object.
00777      *
00778      *  @param[in,out]  obj         A reference to the object whose field value will be set.
00779      *  @param[in]      results     A reference to the ResultSet object.
00780      */
00781     void SetObjectValueFromResultSet(C& obj, const ResultSet& results) const {
00782         this->SetObjectValue(obj, results.getInt64(this->column_name()));
00783     }
00784
00785     /** Set the value for the SQL PreparedStatement object from the value of the field of the
00786      *  corresponding object.
00787      *
00788      *  @param[in]      obj         A reference to the object whose field value will be read.
00789      *  @param[in,out]  stmt        A reference to the SQL PreparedStatement object whose column
00790      *                              value will be set.
00791      */
00792     void SetPreparedStatementValueFromObject(const C& obj, PreparedStatement& stmt) const {
00793         stmt.setInt64(this->column_index(), this->GetValueFromObject(obj));
00794     }
00795 };
00796
00797 /** Template column description class for value of type boost::posix_time::ptime.
00798  *
00799  */
00800 template<class C> class Column_posix_time:
00801         public SqlTableColumnDescription<C, ptime> {
00802
00803 public:
00804
00805     /** The parameterized constructor.
00806      *
00807      *  @param[in]  column_name     The name of the SQL table column.
00808      *  @param[in]  accessor        A pointer to the accessor method for acquiring the value of the
00809      *                              corresponding object field.
00810      *  @param[in]  mutator         A pointer to the mutator method for setting the value of the
00811      *                              corresponding object field.
00812      *  @param[in]  is_primary_key  TRUE if this column is a primary key for the SQL table, or
00813      *                              FALSE otherwise.
00814      */
00815     Column_posix_time(
00816             string column_name,
00817             ptime (C::*accessor)() const = NULL,
00818             void (C::*mutator)(const ptime&) = NULL,
00819             bool is_primary_key = false) :
00820                     SqlTableColumnDescription<C, ptime>(
00821                             column_name,
00822                             COLUMN_DATA_TYPE_POSIX_TIME,
00823                             accessor,
00824                             mutator,
00825                             is_primary_key) {
00826         /* Do nothing. */
00827     }
00828
00829     /** The parameterized constructor.
00830      *
00831      *  @param[in]  column_name     The name of the SQL table column.
00832      *  @param[in]  accessor        A pointer to the accessor method for acquiring the value of the
00833      *                              corresponding object field.
00834      *  @param[in]  mutator         A pointer to the mutator method for setting the value of the
00835      *                              corresponding object field.
00836      *  @param[in]  is_primary_key  TRUE if this column is a primary key for the SQL table, or
00837      *                              FALSE otherwise.
00838      */
00839     Column_posix_time(
00840             string column_name,
00841             const ptime& (C::*accessor)() const = NULL,
00842             void (C::*mutator)(const ptime&) = NULL,
00843             bool is_primary_key = false) :
```

```
00844                   SqlTableColumnDescription<C, ptime>(
00845                           column_name,
00846                           COLUMN_DATA_TYPE_POSIX_TIME,
00847                           accessor,
00848                           mutator,
00849                           is_primary_key) {
00850         /* Do nothing. */
00851     }
00852
00853     /** Set the value in the field of the corresponding object from an SQL ResultSet Object.
00854      *
00855      *  @param[in,out]  obj         A reference to the object whose field value will be set.
00856      *  @param[in]      results     A reference to the ResultSet object.
00857      */
00858     void SetObjectValueFromResultSet(C& obj, const ResultSet& results) const {
00859         string time_str = results.getString(this->column_name()).asStdString();
00860         this->SetObjectValue(
00861                 obj,
00862                 time_str.empty() || time_str == "0000-00-00 00:00:00" ?
00863                         not_a_date_time :
00864                         time_from_string(time_str));
00865     }
00866
00867     /** Set the value for the SQL PreparedStatement object from the value of the field of the
00868      *  corresponding object.
00869      *
00870      *  @param[in]      obj         A reference to the object whose field value will be read.
00871      *  @param[in,out]  stmt        A reference to the SQL PreparedStatement object whose column
00872      *                              value will be set.
00873      */
00874     void SetPreparedStatementValueFromObject(const C& obj, PreparedStatement& stmt) const {
00875         ptime time(this->GetValueFromObject(obj));
00876         stmt.setDateTime(
00877                 this->column_index(),
00878                 time.is_not_a_date_time() ? "0000-00-00 00:00:00" : to_iso_string(time));
00879     }
00880 };
00881
00882 /** Template column description class for value of type string.
00883  *
00884  */
00885 template<class C> class Column_string:
00886         public SqlTableColumnDescription<C, string> {
00887
00888 public:
00889
00890     /** The parameterized constructor.
00891      *
00892      *  @param[in]  column_name     The name of the SQL table column.
00893      *  @param[in]  accessor        A pointer to the accessor method for acquiring the value of the
00894      *                              corresponding object field.
00895      *  @param[in]  mutator         A pointer to the mutator method for setting the value of the
00896      *                              corresponding object field.
00897      *  @param[in]  is_primary_key  TRUE if this column is a primary key for the SQL table, or
00898      *                              FALSE otherwise.
00899      */
00900     Column_string(
00901             string column_name,
00902             string (C::*accessor)() const = NULL,
00903             void (C::*mutator)(const string&) = NULL,
00904             bool is_primary_key = false) :
00905                     SqlTableColumnDescription<C, string>(
00906                             column_name,
00907                             COLUMN_DATA_TYPE_STRING,
00908                             accessor,
00909                             mutator,
00910                             is_primary_key) {
00911         /* Do nothing. */
00912     }
00913
00914     /** The parameterized constructor.
00915      *
00916      *  @param[in]  column_name     The name of the SQL table column.
00917      *  @param[in]  accessor        A pointer to the accessor method for acquiring the value of the
00918      *                              corresponding object field.
00919      *  @param[in]  mutator         A pointer to the mutator method for setting the value of the
00920      *                              corresponding object field.
00921      *  @param[in]  is_primary_key  TRUE if this column is a primary key for the SQL table, or
00922      *                              FALSE otherwise.
00923      */
00924     Column_string(
00925             string column_name,
00926             const string& (C::*accessor)() const = NULL,
00927             void (C::*mutator)(const string&) = NULL,
00928             bool is_primary_key = false) ATTR_DEPRECATED :SqlTableColumnDescription<C, string>(
00929             column_name,
00930             COLUMN_DATA_TYPE_STRING,
00931             accessor,
00932             mutator,
00933             is_primary_key) {
00934         /* Do nothing. */
00935     }
00936
00937     /** Set the value in the field of the corresponding object from an SQL ResultSet Object.
00938      *
00939      *  @param[in,out]  obj         A reference to the object whose field value will be set.
00940      *  @param[in]      results     A reference to the ResultSet object.
00941      */
00942     void SetObjectValueFromResultSet(C& obj, const ResultSet& results) const {
00943         this->SetObjectValue(obj, results.getString(this->column_name()).asStdString());
00944     }
00945
00946     /** Set the value for the SQL PreparedStatement object from the value of the field of the
00947      *  corresponding object.
00948      *
00949      *  @param[in]      obj         A reference to the object whose field value will be read.
00950      *  @param[in,out]  stmt        A reference to the SQL PreparedStatement object whose column
00951      *                              value will be set.
```

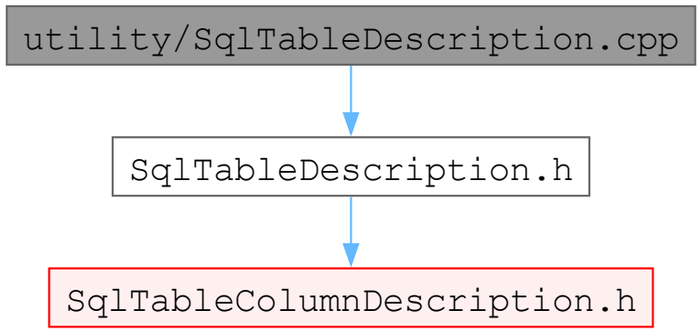
```
00952      */
00953     void SetPreparedStatementValueFromObject(const C& obj, PreparedStatement& stmt) const {
00954         stmt.setString(this->column_index(), this->GetValueFromObject(obj));
00955     }
00956 };
00957
00958 /** Template column description class for value of type uint16_t.
00959  *
00960  */
00961 template<class C> class Column_uint16_t:
00962         public SqlTableColumnDescription<C, uint16_t> {
00963
00964 public:
00965
00966     /** The parameterized constructor.
00967      *
00968      *  @param[in]  column_name     The name of the SQL table column.
00969      *  @param[in]  accessor        A pointer to the accessor method for acquiring the value of the
00970      *                              corresponding object field.
00971      *  @param[in]  mutator         A pointer to the mutator method for setting the value of the
00972      *                              corresponding object field.
00973      *  @param[in]  is_primary_key  TRUE if this column is a primary key for the SQL table, or
00974      *                              FALSE otherwise.
00975      */
00976     Column_uint16_t(
00977             string column_name,
00978             uint16_t (C::*accessor)() const = NULL,
00979             void (C::*mutator)(uint16_t) = NULL,
00980             bool is_primary_key = false) :
00981                     SqlTableColumnDescription<C, uint16_t>(
00982                             column_name,
00983                             COLUMN_DATA_TYPE_UINT16_T,
00984                             accessor,
00985                             mutator,
00986                             is_primary_key) {
00987         /* Do nothing. */
00988     }
00989
00990     /** Set the value in the field of the corresponding object from an SQL ResultSet Object.
00991      *
00992      *  @param[in,out]  obj         A reference to the object whose field value will be set.
00993      *  @param[in]      results     A reference to the ResultSet object.
00994      */
00995     void SetObjectValueFromResultSet(C& obj, const ResultSet& results) const {
00996         this->SetObjectValue(obj, results.getInt(this->column_name()));
00997     }
00998
00999     /** Set the value for the SQL PreparedStatement object from the value of the field of the
01000      *  corresponding object.
01001      *
01002      *  @param[in]      obj         A reference to the object whose field value will be read.
01003      *  @param[in,out]  stmt        A reference to the SQL PreparedStatement object whose column
01004      *                              value will be set.
01005      */
01006     void SetPreparedStatementValueFromObject(const C& obj, PreparedStatement& stmt) const {
01007         stmt.setInt(this->column_index(), this->GetValueFromObject(obj));
01008     }
01009 };
01010
01011 /** Template column description class for value of type uint8_t.
01012  *
01013  */
01014 template<class C> class Column_uint8_t:
01015         public SqlTableColumnDescription<C, uint8_t> {
01016
01017 public:
01018
01019     /** The parameterized constructor.
01020      *
01021      *  @param[in]  column_name     The name of the SQL table column.
01022      *  @param[in]  accessor        A pointer to the accessor method for acquiring the value of the
01023      *                              corresponding object field.
01024      *  @param[in]  mutator         A pointer to the mutator method for setting the value of the
01025      *                              corresponding object field.
01026      *  @param[in]  is_primary_key  TRUE if this column is a primary key for the SQL table, or
01027      *                              FALSE otherwise.
01028      */
01029     Column_uint8_t(
01030             string column_name,
01031             uint8_t (C::*accessor)() const = NULL,
01032             void (C::*mutator)(uint8_t) = NULL,
01033             bool is_primary_key = false) :
01034                     SqlTableColumnDescription<C, uint8_t>(
01035                             column_name,
01036                             COLUMN_DATA_TYPE_UINT8_T,
01037                             accessor,
01038                             mutator,
01039                             is_primary_key) {
01040         /* Do nothing. */
01041     }
01042
01043     /** Set the value in the field of the corresponding object from an SQL ResultSet Object.
01044      *
01045      *  @param[in,out]  obj         A reference to the object whose field value will be set.
01046      *  @param[in]      results     A reference to the ResultSet object.
01047      */
01048     void SetObjectValueFromResultSet(C& obj, const ResultSet& results) const {
01049         this->SetObjectValue(obj, results.getInt(this->column_name()));
01050     }
01051
01052     /** Set the value for the SQL PreparedStatement object from the value of the field of the
01053      *  corresponding object.
01054      *
01055      *  @param[in]      obj         A reference to the object whose field value will be read.
01056      *  @param[in,out]  stmt        A reference to the SQL PreparedStatement object whose column
01057      *                              value will be set.
01058      */
01059     void SetPreparedStatementValueFromObject(const C& obj, PreparedStatement& stmt) const {
```

```
01060         stmt.setInt(this->column_index(), this->GetValueFromObject(obj));
01061     }
01062 };
01063
01064 /** Template column description class for value of type vector<uint8_t>.
01065  *
01066  */
01067 template<class C> class Column_vector_uint8_t:
01068         public SqlTableColumnDescription<C, vector<uint8_t»,
01069         protected util::stream::StreamUtils {
01070
01071 public:
01072
01073     /** The parameterized constructor.
01074      *
01075      *  @param[in]  column_name     The name of the SQL table column.
01076      *  @param[in]  accessor        A pointer to the accessor method for acquiring the value of the
01077      *                              corresponding object field.
01078      *  @param[in]  mutator         A pointer to the mutator method for setting the value of the
01079      *                              corresponding object field.
01080      *  @param[in]  is_primary_key  TRUE if this column is a primary key for the SQL table, or
01081      *                              FALSE otherwise.
01082      */
01083     Column_vector_uint8_t(
01084             string column_name,
01085             vector<uint8_t> (C::*accessor)() const = NULL,
01086             void (C::*mutator)(const vector<uint8_t>&) = NULL,
01087             bool is_primary_key = false) :
01088                     SqlTableColumnDescription<C, vector<uint8_t»(
01089                             column_name,
01090                             COLUMN_DATA_TYPE_VECTOR_UINT8_T,
01091                             accessor,
01092                             mutator,
01093                             is_primary_key) {
01094         /* Do nothing. */
01095     }
01096
01097     /** Set the value in the field of the corresponding object from an SQL ResultSet Object.
01098      *
01099      *  @param[in,out]  obj         A reference to the object whose field value will be set.
01100      *  @param[in]      results     A reference to the ResultSet object.
01101      */
01102     void SetObjectValueFromResultSet(C& obj, const ResultSet& results) const {
01103         vector<uint8_t> data;
01104         string data_hex_str(results.getString(this->column_name()).asStdString());
01105
01106         for(string::const_iterator it = data_hex_str.begin(); it < data_hex_str.end() - 1;
01107                 it += 2) {
01108
01109             data.push_back((htoi(*it) « 4) | htoi(*(it + 1)));
01110
01111         }
01112
01113         this->SetObjectValue(obj, data);
01114     }
01115
01116     /** Set the value for the SQL PreparedStatement object from the value of the field of the
01117      *  corresponding object.
01118      *
01119      *  @param[in]      obj         A reference to the object whose field value will be read.
01120      *  @param[in,out]  stmt        A reference to the SQL PreparedStatement object whose column
01121      *                              value will be set.
01122      */
01123     void SetPreparedStatementValueFromObject(const C& obj, PreparedStatement& stmt) const {
01124         vector<uint8_t> data(this->GetValueFromObject(obj));
01125         ostringstream oss;
01126
01127         for_each(data.begin(), data.end(), [&](uint8_t byte) {
01128
01129             oss « byte;
01130
01131         });
01132
01133         stmt.setString(this->column_index(), oss.str());
01134     }
01135 };
01136
01137 } /* namespace mysql */
01138 } /* namespace util */
01139 #endif /* COLUMNDESCRIPTION_H_ */
```

11.217 utility/SqlTableDescription.cpp File Reference

The util::mysql::SqlTableDescription class source file.

```
#include "SqlTableDescription.h"
```
Include dependency graph for SqlTableDescription.cpp:



11.217.1 Detailed Description

The util::mysql::SqlTableDescription class source file.

Version

0.6

Date

Sep 30, 2017 4:26:11 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

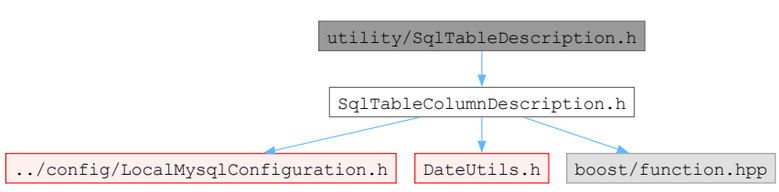
2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

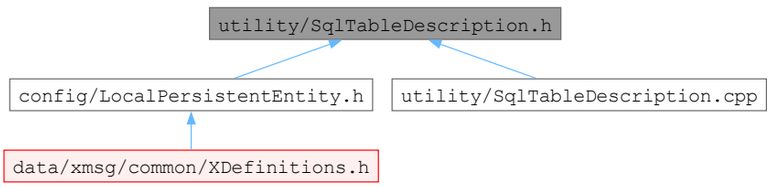
11.218 utility/SqlTableDescription.h File Reference

The util::mysql::SqlTableDescription class header file.

```
#include "SqlTableColumnDescription.h"
```
Include dependency graph for SqlTableDescription.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::mysql::SqlTableDescription< T >

  Template class representing the description of an SQL database table.

Namespaces

- namespace util

  The utility namespace.
- namespace util::mysql

  The MySQL utility namespace.

11.218.1 Detailed Description

The util::mysql::SqlTableDescription class header file.

Version

0.6

Date

Oct 18, 2014 12:57:33 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.219 SqlTableDescription.h

Go to the documentation of this file.

```
00001 /** The @ref util::mysql::SqlTableDescription class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Oct 18, 2014 12:57:33 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef TABLEDESCRIPTION_H_
00024 #define TABLEDESCRIPTION_H_
00025
00026 /* Includes */
00027
00028 // local
00029 #include "SqlTableColumnDescription.h"
00030
00031 namespace util {
00032 namespace mysql {
00033
00034 using boost::dynamic_pointer_cast;
00035 using boost::lexical_cast;
00036 using std::make_shared;
00037 using boost::posix_time::ptime;
00038 //using std::shared_ptr;
00039 using std::ostringstream;
00040 using std::string;
00041 using std::vector;
00042 using sql::PreparedStatement;
00043 using sql::SQLString;
00044 using sql::ResultSet;
00045
00046 /* Class Definition */
00047
00048 /** Template class representing the description of an SQL database table.
00049  *
00050  *  The template parameter T represents the class that the SQL database table represents.
00051  *
00052  */
00053 template<class T> class SqlTableDescription:
00054         public LocalMysqlConfiguration,
00055         protected util::stream::StreamUtils {
00056
00057     /* Template-Dependent Types */
00058
00059     /** A column of type @p bool.
00060      *
00061      */
00062     typedef Column_bool<T> cbol;
00063
00064     /** A column of type @p char.
00065      *
00066      */
00067     typedef Column_char<T> cchar;
00068
00069     /** A column of type @p double.
00070      *
00071      */
00072     typedef Column_double<T> cdbl;
00073
00074     /** A column of type @p boost::filesystem::path.
00075      *
00076      */
00077     typedef Column_filesystem_path<T> cpth;
00078
00079     /** A column of type @p float.
00080      *
00081      */
00082     typedef Column_float<T> cflt;
00083
00084     /** A column of type @p int.
00085      *
00086      */
00087     typedef Column_int<T> cint;
00088
00089     /** A column of type @p int16_t.
00090      *
00091      */
00092     typedef Column_int16_t<T> ci16;
00093
00094     /** A column of type @p long.
00095      *
00096      */
00097     typedef Column_long<T> clng;
00098
00099     /** A column of type @p boost::posix_time::ptime.
00100      *
00101      */
00102     typedef Column_posix_time<T> cptm;
00103
```

```
00104     /** A column of type @p std::string.
00105      *
00106      */
00107     typedef Column_string<T> cstr;
00108
00109     /** A column of type @p uint16_t.
00110      *
00111      */
00112     typedef Column_uint16_t<T> cu16;
00113
00114     /** A column of type @p uint8_t.
00115      *
00116      */
00117     typedef Column_uint8_t<T> cu8;
00118
00119     /** A column of type @p std::vector<uint8_t>.
00120      *
00121      */
00122     typedef Column_vector_uint8_t<T> cvu8;
00123
00124 private:
00125
00126     /* Data Members */
00127
00128     /** The insert statement for this SQL table. */
00129     string insert_on_duplicate_key_update_statement_;
00130
00131     /** The select all statement for this SQL table. */
00132     string select_statement_;
00133
00134     /** A vector of shared pointers to @ref SqlTableColumnDescription objects describing the
00135      *  columns of this SQL table.
00136      */
00137     vector<shared_ptr<SqlTableColumnDescriptionBase> > table_columns_;
00138
00139     /** The name of this SQL table. */
00140     string table_name_;
00141
00142 private:
00143
00144     /* Member Functions */
00145
00146     /** Generate a vector of object T from the given result set.
00147      *
00148      *  @param[in]      res         A shared pointer to the result set from which the vector of
00149      *                              objects is to be constructed.
00150      *
00151      *  @note           The columns of the result set must correspond in name, type, and order to
00152      *                  column descriptions contained in @ref
00153      *                  SqlTableDescription<T>::table_columns_.
00154      *
00155      *  @throw          log::MessageException if an error occurs while constructing the result
00156      *                  vector.
00157      *
00158      *  @return         A vector of object T constructed from the given result set.
00159      */
00160     vector<shared_ptr<T> > GenerateObjectList(shared_ptr<ResultSet> res) const {
00161         shared_ptr<T> obj;                      // An object constructed from the result set.
00162         vector<shared_ptr<T> > obj_list;        // The list of constructed object to return.
00163
00164         try {
00165             if(!res)
00166                 EXC_THROW("Pointer to result set is NULL!");
00167             obj_list.reserve(res->rowsCount());
00168
00169             // Iterate over the rows in result set and construct one object for each row.
00170             while(res->next()) {
00171                 obj = make_shared<T>();
00172
00173                 // Iterate over columns and set the corresponding values of the object members.
00174                 for_each(
00175                         table_columns_.begin(),
00176                         table_columns_.end(),
00177                         [&](const shared_ptr<SqlTableColumnDescriptionBase>& col) {
00178                             switch(col->data_type()) {
00179                                 case COLUMN_DATA_TYPE_BOOL: // Get bool value.
00180                                 (dynamic_pointer_cast<cbol>(col))->SetObjectValueFromResultSet(
00181                                         *obj,
00182                                         *res);
00183                                 break;
00184
00185                                 case COLUMN_DATA_TYPE_CHAR:// Get double value.
00186                                 (dynamic_pointer_cast<cchar>(col))->SetObjectValueFromResultSet(
00187                                         *obj,
00188                                         *res);
00189                                 break;
00190
00191                                 case COLUMN_DATA_TYPE_DOUBLE:// Get double value.
00192                                 (dynamic_pointer_cast<cdbl>(col))->SetObjectValueFromResultSet(
00193                                         *obj,
00194                                         *res);
00195                                 break;
00196
00197                                 case COLUMN_DATA_TYPE_FLOAT:// Get float value.
00198                                 (dynamic_pointer_cast<cflt>(col))->SetObjectValueFromResultSet(
00199                                         *obj,
00200                                         *res);
00201                                 break;
00202
00203                                 case COLUMN_DATA_TYPE_FILESYSTEM_PATH:// Get path value.
00204                                 (dynamic_pointer_cast<cpth>(col))->SetObjectValueFromResultSet(
00205                                         *obj,
00206                                         *res);
00207                                 break;
00208
00209                                 case COLUMN_DATA_TYPE_INT:// Get int value.
00210                                 (dynamic_pointer_cast<cint>(col))->SetObjectValueFromResultSet(
00211                                         *obj,
```

```
00212                                     *res);
00213                                 break;
00214
00215                                 case COLUMN_DATA_TYPE_INT16_T:// Get int16_t value.
00216                                 (dynamic_pointer_cast<ci16>(col))->SetObjectValueFromResultSet(
00217                                     *obj,
00218                                     *res);
00219                                 break;
00220
00221                                 case COLUMN_DATA_TYPE_LONG:// Get long value.
00222                                 (dynamic_pointer_cast<clng>(col))->SetObjectValueFromResultSet(
00223                                     *obj,
00224                                     *res);
00225                                 break;
00226
00227                                 case COLUMN_DATA_TYPE_POSIX_TIME:// Get boost::posix_time::ptime value.
00228                                 (dynamic_pointer_cast<cptm>(col))->SetObjectValueFromResultSet(
00229                                     *obj,
00230                                     *res);
00231                                 break;
00232
00233                                 case COLUMN_DATA_TYPE_STRING:// Get string value.
00234                                 (dynamic_pointer_cast<cstr>(col))->SetObjectValueFromResultSet(
00235                                     *obj,
00236                                     *res);
00237                                 break;
00238
00239                                 case COLUMN_DATA_TYPE_UINT16_T:// Get uint16_t value.
00240                                 (dynamic_pointer_cast<cu16>(col))->SetObjectValueFromResultSet(
00241                                     *obj,
00242                                     *res);
00243                                 break;
00244
00245                                 case COLUMN_DATA_TYPE_UINT8_T:// Get uint8_t value.
00246                                 (dynamic_pointer_cast<cu8>(col))->SetObjectValueFromResultSet(
00247                                     *obj,
00248                                     *res);
00249                                 break;
00250
00251                                 case COLUMN_DATA_TYPE_VECTOR_UINT8_T:// Get vector<uint8_t> value.
00252                                 (dynamic_pointer_cast<cvu8>(col))->SetObjectValueFromResultSet(
00253                                     *obj,
00254                                     *res);
00255                                 break;
00256
00257                                 case COLUMN_DATA_TYPE_UNKOWN:
00258                                 default:// If an unknown value type is encountered, throw exception.
00259                                 EXC_THROW("Failed to determine column type!");
00260                                 break;
00261                             }
00262
00263                         });
00264                 obj_list.push_back(obj);
00265             }
00266         }CATCH_BLOCK("Failed to generate vector of objects from the result set!")
00267
00268         return obj_list;
00269     }
00270
00271 public:
00272
00273     /* Member Functions */
00274
00275     /** The parameterized constructor.
00276      *
00277      *  @param[in]      name            The name of the SQL database table.
00278      *  @param[in]      columns         Vector of shared pointers to ColumnDescription objects
00279      *                                  describing the columns of this SQL table.
00280      */
00281     SqlTableDescription(
00282             const string& name,
00283             const vector<shared_ptr<SqlTableColumnDescriptionBase> >& columns) :
00284                     table_columns_(columns),
00285                     table_name_(name) {
00286         generateInsertStatment();
00287         generateSelectAllStatement();
00288
00289         // Set column indices.
00290         size_t idx = 0;
00291         for_each(
00292                 table_columns_.begin(),
00293                 table_columns_.end(),
00294                 [&](const shared_ptr<SqlTableColumnDescriptionBase>& col) {
00295                     col->set_column_index(++idx);
00296                 });
00297
00298     }
00299
00300     /** The destructor.
00301      *
00302      */
00303     ~SqlTableDescription() {
00304         /* Do nothing. */
00305     }
00306
00307     void DeleteObjectsFromDatabase(const string& where = "") const  {
00308
00309         try {
00310
00311             // Execute query.
00312             executeDeleteQuery(table_name_, where);
00313
00314         }CATCH_BLOCK("Failed to query objects from database!")
00315
00316     }
00317
00318     /** Save a persistent entity to the SQL table.
00319      *
```

```
00320      *  @throws         util::log::MessageException if an error occurs.
00321      *
00322      *  @param[in]      obj             The instance of the persistent entity to save to the
00323      *                                  SQL database table.
00324      */
00325     void SaveObjectToDatabase(const T& obj) const {
00326         shared_ptr<SqlTableColumnDescriptionBase> col;
00327         shared_ptr<PreparedStatement> stmt;
00328
00329         try {
00330             // Generate the SQL prepared statement.
00331             stmt = generatePreparedStatement(insert_on_duplicate_key_update_statement_);
00332
00333             // Iterate over the columns in the table.
00334             for(size_t i = 0; i < table_columns_.size(); ++i) {
00335                 col = table_columns_.at(i);
00336
00337                 // Set the appropriate prepared statement value, based on the column type!
00338                 switch(col->data_type()) {
00339                 case COLUMN_DATA_TYPE_BOOL:             // Set bool value.
00340                     (dynamic_pointer_cast<cbol>(col))->SetPreparedStatementValueFromObject(
00341                             obj,
00342                             *stmt);
00343                     break;
00344
00345                 case COLUMN_DATA_TYPE_CHAR:           // Set char value.
00346                     (dynamic_pointer_cast<cchar>(col))->SetPreparedStatementValueFromObject(
00347                             obj,
00348                             *stmt);
00349                     break;
00350
00351                 case COLUMN_DATA_TYPE_DOUBLE:           // Set double value.
00352                     (dynamic_pointer_cast<cdbl>(col))->SetPreparedStatementValueFromObject(
00353                             obj,
00354                             *stmt);
00355                     break;
00356
00357                 case COLUMN_DATA_TYPE_FILESYSTEM_PATH:  // Set path value.
00358                     (dynamic_pointer_cast<cpth>(col))->SetPreparedStatementValueFromObject(
00359                             obj,
00360                             *stmt);
00361                     break;
00362
00363                 case COLUMN_DATA_TYPE_FLOAT:            // Set float value.
00364                     (dynamic_pointer_cast<cflt>(col))->SetPreparedStatementValueFromObject(
00365                             obj,
00366                             *stmt);
00367                     break;
00368
00369                 case COLUMN_DATA_TYPE_INT:              // Set int value.
00370                     (dynamic_pointer_cast<cint>(col))->SetPreparedStatementValueFromObject(
00371                             obj,
00372                             *stmt);
00373                     break;
00374
00375                 case COLUMN_DATA_TYPE_INT16_T:          // Set int16_t value
00376                     (dynamic_pointer_cast<ci16>(col))->SetPreparedStatementValueFromObject(
00377                             obj,
00378                             *stmt);
00379                     break;
00380
00381                 case COLUMN_DATA_TYPE_LONG:             // Set long value.
00382                     (dynamic_pointer_cast<clng>(col))->SetPreparedStatementValueFromObject(
00383                             obj,
00384                             *stmt);
00385                     break;
00386
00387                 case COLUMN_DATA_TYPE_POSIX_TIME:       // Set ptime value.
00388                     (dynamic_pointer_cast<cptm>(col))->SetPreparedStatementValueFromObject(
00389                             obj,
00390                             *stmt);
00391                     break;
00392
00393                 case COLUMN_DATA_TYPE_STRING:           // Set string value.
00394                     (dynamic_pointer_cast<cstr>(col))->SetPreparedStatementValueFromObject(
00395                             obj,
00396                             *stmt);
00397                     break;
00398
00399                 case COLUMN_DATA_TYPE_UINT16_T:         // Set uint16_t value.
00400                     (dynamic_pointer_cast<cu16>(col))->SetPreparedStatementValueFromObject(
00401                             obj,
00402                             *stmt);
00403                     break;
00404
00405                 case COLUMN_DATA_TYPE_UINT8_T:          // Set uint8_t value.
00406                     (dynamic_pointer_cast<cu8>(col))->SetPreparedStatementValueFromObject(
00407                             obj,
00408                             *stmt);
00409                     break;
00410
00411                 case COLUMN_DATA_TYPE_VECTOR_UINT8_T:   // Set vector<uint8_t> value.
00412                     (dynamic_pointer_cast<cvu8>(col))->SetPreparedStatementValueFromObject(
00413                             obj,
00414                             *stmt);
00415                     break;
00416
00417                 case COLUMN_DATA_TYPE_UNKOWN:
00418                 default:  // If an unknown value type is encountered, throw exception.
00419                     EXC_THROW("Failed to set column value for unknown type!");
00420                     break;
00421                 }
00422             }
00423
00424             // Execule the insert statement.
00425             try {
00426                 stmt->executeUpdate();
00427                 return;
```

```
00428             }CATCH_NOTHROW("Failed to save object to database! Trying again...")
00429
00430             try {
00431                 stmt->executeUpdate();
00432                 return;
00433             }CATCH_NOTHROW("Failed to save object to database! Trying again...")
00434
00435             stmt->executeUpdate();
00436             return;
00437
00438         }CATCH_BLOCK("Failed to save object to database!")
00439
00440     }
00441
00442     /** Select all objects from this SQL database table.
00443      *
00444      *  @param[in]      groupBy     The SQL select statement group-by clause.
00445      *  @param[in]      orderBy     The SQL select statement order-by clause.
00446      *  @param[in]      limit       The SQL select statement result set row limit value.
00447      *  @param[in]      offset      The SQL select statement result set row offset.
00448      *
00449      *  @throw          util::log::MessageException if an error occurs while querying the database.
00450      *
00451      *  @return         A vector of shared pointers to the object selected from the database.
00452      */
00453     vector<shared_ptr<T> > SelectAllObjectsFromDatabase(
00454             const string& groupBy = "",
00455             const string& orderBy = "",
00456             const unsigned int& limit = 0,
00457             const unsigned int& offset = 0) const {
00458
00459         return SelectObjectsFromDatabase("", groupBy, orderBy, limit, offset);
00460
00461     }
00462
00463     /** Get a single object from the database.
00464      *
00465      *  @param[in]      where       The SQL select statement where-condition clause.
00466      *  @param[in]      groupBy     The SQL select statement group-by clause.
00467      *  @param[in]      orderBy     The SQL select statement order-by clause.
00468      *  @param[in]      offset      The SQL select statement result set row offset.
00469      *
00470      *  @throw          util::log::MessageException if an error occurs while querying the database.
00471      *
00472      *  @return         A shared pointer to the object selected from the database.
00473      */
00474     shared_ptr<T> SelectObjectFromDatabase(
00475             const string& where = "",
00476             const string& groupBy = "",
00477             const string& orderBy = "",
00478             const unsigned int& offset = 0) const {
00479
00480         shared_ptr<T> x;
00481
00482         try {
00483
00484             vector<shared_ptr<T> > res(
00485                     SelectObjectsFromDatabase(where, groupBy, orderBy, 1, offset));
00486
00487             if(!res.empty())
00488                 x = res.at(0);
00489
00490         }CATCH_BLOCK("Failed to get object from database!")
00491
00492         return x;
00493     }
00494
00495     /** Select objects from this SQL database table.
00496      *
00497      *  @param[in]      where       The SQL select statement where-condition clause.
00498      *  @param[in]      groupBy     The SQL select statement group-by clause.
00499      *  @param[in]      orderBy     The SQL select statement order-by clause.
00500      *  @param[in]      limit       The SQL select statement result set row limit value.
00501      *  @param[in]      offset      The SQL select statement result set row offset.
00502      *
00503      *  @throw          util::log::MessageException if an error occurs while querying the database.
00504      *
00505      *  @return         A vector of shared pointers to the object selected from the database.
00506      */
00507     vector<shared_ptr<T> > SelectObjectsFromDatabase(
00508             const string& where = "",
00509             const string& groupBy = "",
00510             const string& orderBy = "",
00511             const unsigned int& limit = 0,
00512             const unsigned int& offset = 0) const {
00513
00514         vector<shared_ptr<T> > resultVector;
00515
00516         try {
00517             // Execute query.
00518             resultVector = GenerateObjectList(
00519                     executeSelectQuery(
00520                             getColumnSelectNames(),
00521                             table_name_,
00522                             where,
00523                             groupBy,
00524                             orderBy,
00525                             limit,
00526                             offset));
00527         }CATCH_BLOCK("Failed to query objects from database!")
00528
00529         return resultVector;
00530     }
00531
00532
00533     /** Select a value from the SQL database table represented by this object.
00534      *
00535      *  @warning        The "select expression" clause must name a singular element when querying a
```

```
00536      *                  singular value from the database.
00537      *
00538      *  @note           The result set of this query is always limited to a single result.
00539      *
00540      *  @param[in]      what        The SQL select statement select-expression clause.
00541      *  @param[in]      where       The SQL select statement where-condition clause.
00542      *  @param[in]      groupBy     The SQL select statement group-by clause.
00543      *  @param[in]      orderBy     The SQL select statement order-by clause.
00544      *  @param[in]      offset      The SQL select statement result set row offset.
00545      *
00546      *  @throw          log::MessageException if an error occurs while querying the database or if
00547      *                  con == NULL.
00548      *
00549      *  @return         The selected value from the SQL database table represented by this object.
00550      */
00551     template<typename TT> TT SelectValueFromDatabase(
00552         const string& what,
00553         const string& where = "",
00554         const string& groupBy = "",
00555         const string& orderBy = "",
00556         const unsigned int& offset = 0) const {
00557
00558       TT val = TT();
00559
00560       try {
00561         // Get table column.
00562         vector<shared_ptr<SqlTableColumnDescriptionBase> >::const_iterator it;
00563         for(it = table_columns_.begin(); it < table_columns_.end(); it++)
00564           if((*it)->column_name() == what)
00565             break;
00566
00567         if(it >= table_columns_.end())
00568           EXC_THROW("'" + what + "' is not a column of " + table_name_ + "!");
00569         shared_ptr < SqlTableColumnDescriptionBase > column(*it);
00570
00571         // Query the database.
00572         string query = GenerateSqlSelectStr(
00573             column->column_select_name(),
00574             table_name_,
00575             where,
00576             groupBy,
00577             orderBy,
00578             1,
00579             offset);
00580         shared_ptr < ResultSet > results(executeQuery(query));
00581
00582         if(results->next())
00583           switch(column->data_type()) {
00584
00585             case COLUMN_DATA_TYPE_BOOL:             // Get boolean value.
00586
00587               val = static_cast<TT>(results->getBoolean(column->column_name()));
00588
00589               break;
00590
00591             case COLUMN_DATA_TYPE_VECTOR_UINT8_T:  // Get byte vector value.
00592
00593               try {
00594
00595                 /** @todo   Implement the functionality to parse values of type
00596                  *          vector<uint8_t> from the result set of an SQL query.
00597                  *
00598                  */
00599
00600                 EXC_THROW("The SelectValueFromDatabase function does not support values of"
00601                     " type vector<uint8_t>!");
00602
00603               }CATCH_BLOCK("Failed to parse by vector value from SQL result set!")
00604
00605               break;
00606
00607             case COLUMN_DATA_TYPE_DOUBLE:           // Get double value.
00608
00609               val = static_cast<TT>(results->getDouble(column->column_name()));
00610
00611               break;
00612
00613             case COLUMN_DATA_TYPE_FLOAT:            // Get float value.
00614
00615               val = static_cast<TT>(results->getDouble(column->column_name()));
00616
00617               break;
00618
00619             case COLUMN_DATA_TYPE_INT:              // Get integer value.
00620
00621               val = static_cast<TT>(results->getInt(column->column_name()));
00622
00623               break;
00624
00625             case COLUMN_DATA_TYPE_INT16_T:              // Get integer value.
00626
00627               val = static_cast<TT>(results->getInt(column->column_name()));
00628
00629               break;
00630
00631             case COLUMN_DATA_TYPE_LONG:             // Get long integer value.
00632
00633               val = static_cast<TT>(results->getInt64(column->column_name()));
00634
00635               break;
00636
00637             case COLUMN_DATA_TYPE_POSIX_TIME:       // Get POSIX time value.
00638
00639               try {
00640
00641                 bpt::ptime time = bpt::time_from_string(
00642                     results->getString(column->column_name()).asStdString());
00643                 val = *reinterpret_cast<TT*>(&time);
```

```
00644
00645               }CATCH_BLOCK("Failed to set POSIX time value from query result set!")
00646
00647               break;
00648
00649             case COLUMN_DATA_TYPE_STRING:           // Get string value.
00650
00651               try {
00652
00653                 val = lexical_cast < TT > (results->getString(column->column_name()).c_str());
00654
00655               }CATCH_BLOCK("Lexical cast to return type failed!")
00656
00657               break;
00658
00659             case COLUMN_DATA_TYPE_UINT16_T:         // Get unsigned short value.
00660
00661               val = static_cast<TT>(results->getUInt(column->column_name()));
00662
00663               break;
00664
00665             case COLUMN_DATA_TYPE_UINT8_T:          // Get unsigned short value.
00666
00667               val = static_cast<TT>(results->getUInt(column->column_name()));
00668
00669               break;
00670
00671             case COLUMN_DATA_TYPE_UNKOWN:
00672
00673               // If an unknown value type is encountered, throw exception.
00674
00675               /* No break. */
00676
00677             default:
00678
00679               EXC_THROW("Failed to get column value for unknown type!");
00680
00681               break;
00682
00683           }
00684
00685       }CATCH_BLOCK("Failed to query '" + what + "' from database!")
00686
00687       return val;
00688     }
00689
00690
00691
00692     /** Generate a string containing comma-separated, ordered list of column names for this SQL
00693      *  table from the column descriptions.
00694      *
00695      *  @note           This function is used to generate both INSERT and SELECT query
00696      *                  statements for the SQL table.
00697      *
00698      *  @return         A string containing comma-separated, ordered list of column names for
00699      *                  this SQL table.
00700      */
00701     string getColumnNames() const {
00702         ostringstream os;
00703
00704         for(size_t i = 0; i < table_columns_.size(); ++i) {
00705             os « (i ? ", " : "") « table_columns_.at(i)->column_name();
00706         }
00707
00708         return os.str();
00709     }
00710
00711     /** Generate a string containing comma-separated, ordered list of column names used for select
00712      *  statements for this SQL table from the column descriptions.
00713      *
00714      *  @note           This function is used to generate both INSERT and SELECT query
00715      *                  statements for the SQL table.
00716      *
00717      *  @return         A string containing comma-separated, ordered list of column names for
00718      *                  this SQL table.
00719      */
00720     string getColumnSelectNames() const {
00721         ostringstream os;
00722
00723         for(size_t i = 0; i < table_columns_.size(); ++i) {
00724             os « (i ? ", " : "") « table_columns_.at(i)->column_select_name();
00725         }
00726
00727         return os.str();
00728     }
00729
00730     /** Get the vector of shared pointers to ColumnDescription objects describing the columns
00731      *  of this SQL table.
00732      *
00733      *  @return         The vector of shared pointers to ColumnDescription objects describing
00734      *                  the columns of this SQL table.
00735      */
00736     vector<shared_ptr<SqlTableColumnDescriptionBase> > getColumns() const {
00737         return table_columns_;
00738     }
00739
00740     /** Generate an SQL INSERT statement for this SQL table.
00741      *
00742      *  @return         An SQL INSERT statement for this SQL table.
00743      */
00744     string generateInsertStatment() {
00745         ostringstream os;
00746         size_t nUpdate = 0;
00747
00748         os « "INSERT INTO \n\t" « table_name_ « "(" « getColumnNames() « ") \n";
00749         os « "VALUES\n\t(" « values(table_columns_.size()) « ") \n";
00750         os « "ON DUPLICATE KEY UPDATE \n\t";
00751
```

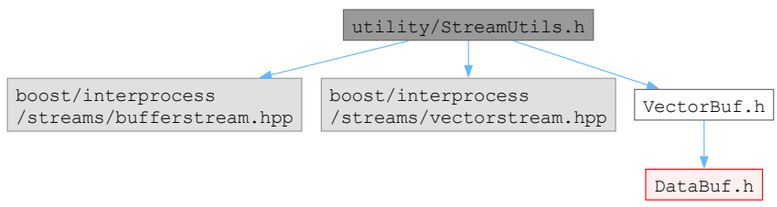
```
00752         // Iterate over columns.
00753         for(size_t i = 0; i < table_columns_.size(); ++i) {
00754             shared_ptr<SqlTableColumnDescriptionBase> column = table_columns_.at(i);
00755             if(!column->is_primary_key())
00756                 os « (nUpdate++ ? ", " : "")
00757                    « column->column_name()
00758                    « "=VALUES("
00759                    « column->column_name()
00760                    « ")";
00761         }
00762 
00763         os « ";";
00764 
00765         return insert_on_duplicate_key_update_statement_ = os.str();
00766     }
00767 
00768     /** Generate the "select all" SQL statement for this table.
00769      *
00770      *  @return         The "select all" SQL statement for this table.
00771      */
00772     string generateSelectAllStatement() {
00773         return select_statement_ = GenerateSqlSelectStr(getColumnSelectNames(), table_name_);
00774     }
00775 
00776     /** Generate a values string component of an SQL insert statement with n values.
00777      *
00778      *  @param[in]      n               The number of values in the SQL insert statement.
00779      *
00780      *  @return         A values string component of an SQL insert statement with n values.
00781      */
00782     static string values(size_t n) {
00783         ostringstream os;
00784 
00785         for(size_t i = 0; i < n; ++i)
00786             os « (i ? ",?" : "?");
00787 
00788         return os.str();
00789     }
00790 
00791 };
00792 
00793 } /* namespace mysql */
00794 } /* namespace util */
00795 #endif /* TABLEDESCRIPTION_H_ */
```

11.220 utility/StreamUtils.h File Reference

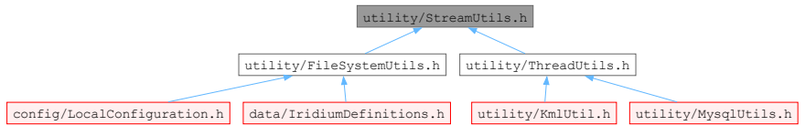
The util::stream::StreamUtils class source file.

```
#include <boost/interprocess/streams/bufferstream.hpp>
#include <boost/interprocess/streams/vectorstream.hpp>
#include "VectorBuf.h"
```

Include dependency graph for StreamUtils.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::stream::StreamUtils

  Class providing useful functions for manipulating I/O streams.

Namespaces

- namespace util

  The utility namespace.
- namespace util::stream

  The stream utility namespace.

11.220.1 Detailed Description

The util::stream::StreamUtils class source file.

Version

0.6

Date

Jul 1, 2014 5:34:26 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.221 StreamUtils.h

Go to the documentation of this file.

```
00001 /** The @ref util::stream::StreamUtils class source file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jul 1, 2014 5:34:26 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef STREAMUTILITIES_H_
00024 #define STREAMUTILITIES_H_
00025
00026 /* Includes */
00027
00028 // boost
00029 #include <boost/interprocess/streams/bufferstream.hpp>
00030 #include <boost/interprocess/streams/vectorstream.hpp>
00031
00032 // local
00033 #include "VectorBuf.h"
00034
00035 namespace util {
00036 namespace stream {
00037
00038 /** Class providing useful functions for manipulating I/O streams.
00039  *
00040  */
00041 class StreamUtils {
00042 public :
00043
```

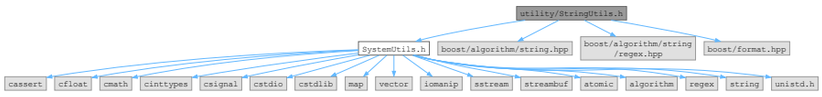
```
00044     /** Get the integer value of a hexadecimal character.
00045      *
00046      * @param[in]       c       The hexadecimal character.
00047      *
00048      * @return          The integer value of a hexadecimal character, or 0 if c is not a
00049      *                  hexadecimal character.
00050      */
00051     static uint8_t htoi(char c) {
00052         if(c >= '0' && c <= '9') return c - (uint8_t)'0';
00053         if(c >= 'a' && c <= 'f') return c - (uint8_t)'a' + 10;
00054         if(c >= 'A' && c <= 'F') return c - (uint8_t)'A' + 10;
00055         return 0;
00056     }
00057
00058 };
00059
00060 } /* namespace stream */
00061 } /* namespace util */
00062
00063 #endif /* STREAMUTILITIES_H_ */
```

11.222 utility/StringUtils.h File Reference

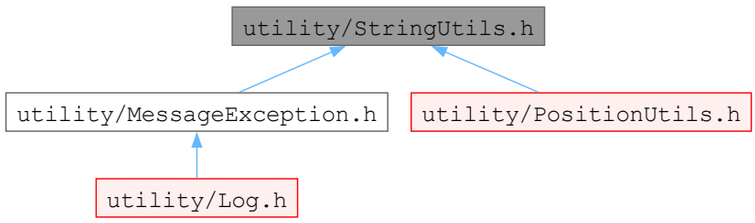
The StringUtils class header file.

```
#include "SystemUtils.h"
#include <boost/algorithm/string.hpp>
#include <boost/algorithm/string/regex.hpp>
#include <boost/format.hpp>
```

Include dependency graph for StringUtils.h:



This graph shows which files directly or indirectly include this file:



11.222.1 Detailed Description

The StringUtils class header file.

Version

0.6

Date

Jul 25, 2014 4:37:38 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.223 StringUtils.h

Go to the documentation of this file.

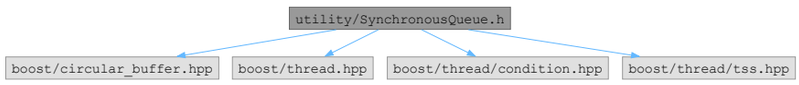
```
00001 /** The StringUtils class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Jul 25, 2014 4:37:38 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef STRINGUTILS_H_
00024 #define STRINGUTILS_H_
00025
00026 /* Includes */
00027
00028 // local
00029 #include "SystemUtils.h"
00030
00031 // boost
00032 #include <boost/algorithm/string.hpp>
00033 #include <boost/algorithm/string/regex.hpp>
00034 #include <boost/format.hpp>
00035
00036 #endif /* STRINGUTILS_H_ */
```

11.224 utility/SynchronousQueue.h File Reference

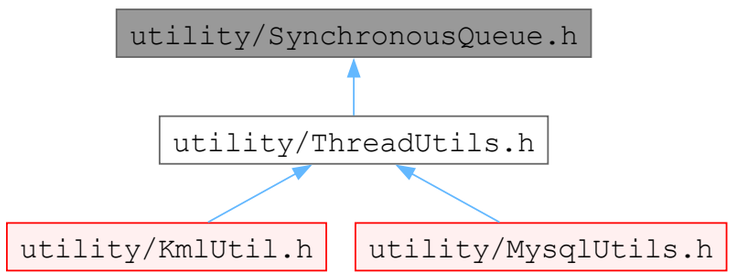
The The util::thread::SynchronousQueue class header file.

```
#include <boost/circular_buffer.hpp>
#include <boost/thread.hpp>
#include <boost/thread/condition.hpp>
#include <boost/thread/tss.hpp>
```

Include dependency graph for SynchronousQueue.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::thread::SynchronousQueue< T >

  Synchronous queue template.

Namespaces

- namespace util

  The utility namespace.
- namespace util::thread

  The thread utility namespace.

11.224.1 Detailed Description

The The util::thread::SynchronousQueue class header file.

Version

0.6

Date

Oct 18, 2014 1:44:13 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.225 SynchronousQueue.h

Go to the documentation of this file.

```
00001 /** The The @ref util::thread::SynchronousQueue class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Oct 18, 2014 1:44:13 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022 #ifndef SYNCHRONOUSQUEUE_H_
00023 #define SYNCHRONOUSQUEUE_H_
00024
00025 /* Includes */
00026
00027 // boost
00028 #include <boost/circular_buffer.hpp>
00029 #include <boost/thread.hpp>
00030 #include <boost/thread/condition.hpp>
00031 #include <boost/thread/tss.hpp>
00032
00033 namespace util {
00034 namespace thread {
00035
00036 /* Class Definition */
00037
00038 /** Synchronous queue template.
00039  *
00040  *  This template provides synchronized access to the underlying circular buffer and may be
00041  *  implemented for data of arbitrary type.
00042  *
```

```
00043  */
00044 template<typename T> class SynchronousQueue {
00045
00046 private:
00047
00048     /* Data Members */
00049
00050     /** Circular buffer for synchronized data storage.
00051      *
00052      */
00053     boost::circular_buffer<T> buffer;
00054
00055     /** Conditional variable indicating that the buffer is not full.
00056      *
00057      */
00058     boost::condition bufferNotFull;
00059
00060     /** Conditional variable indicating that the buffer is not empty.
00061      *
00062      */
00063     boost::condition bufferNotEmpty;
00064
00065     /** Mutual exclusion lock for synchronizing operations.
00066      *
00067      */
00068     mutable boost::mutex mutex;
00069
00070 public:
00071
00072     /* Member Functions */
00073
00074     /** Constructor.
00075      *
00076      *  @param[in]      n               Capacity of the underlying circular buffer to set.
00077      */
00078     SynchronousQueue(size_t n) :
00079                     buffer(n) {
00080
00081         /* Do nothing. */
00082
00083     }
00084
00085     ~SynchronousQueue() {
00086
00087         /* Do nothing. */
00088
00089     }
00090
00091     /** Push element into queue.
00092      *
00093      *  @param[in]      elem        Element to push into queue.
00094      *  @param[in]      block       If TRUE, this function will block until the queue is not full.
00095      *                              If FALSE, this function will return FALSE if the queue is full
00096      *                              (and the element will not be pushed into the queue).
00097      *
00098      *  @return         TRUE if the element was successfully pushed into the queue, or FALSE
00099      *                  otherwise.
00100      */
00101     bool push(const T& elem, bool block) {
00102
00103         {
00104             boost::unique_lock<boost::mutex> lock(mutex);
00105
00106             if(block)
00107
00108                 // If function is blocking, block until buffer is not full.
00109                 while(buffer.full())
00110
00111                     bufferNotFull.wait(lock);
00112
00113             else if(buffer.full())
00114
00115                 // Otherwise, return FALSE if the buffer is full.
00116                 return false;
00117
00118             // Push data into buffer.
00119             buffer.push_back(elem);
00120         }
00121
00122         // Notify waiting threads (if any), that buffer is no longer empty.
00123         bufferNotEmpty.notify_one();
00124
00125         return true;
00126     }
00127
00128     bool pushIfNotContained(const T& elem, bool block) ATTR_DEPRECATED {
00129
00130         if(!contains(elem))
00131         return push(elem, block);
00132         else
00133         return false;
00134
00135     }
00136
00137     /** Push multiple elements into queue.
00138      *
00139      *  @param[in]      first,last  Input iterators to the initial and final positions in a range.
00140      *                              The range used is @p [first,last), which includes all the
00141      *                              elements between first and last, including the element pointed
00142      *                              by first but not the element pointed by last.
00143      */
00144     template<typename Iter> void push(Iter first, Iter last) {
00145
00146         while(first != last) {
00147
00148             {
00149                 // Get an exclusive lock.
00150                 boost::unique_lock<boost::mutex> lock(mutex);
```

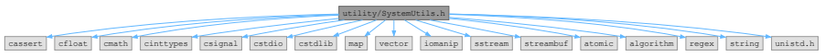
```
00151
00152                 while(buffer.full())
00153
00154                     bufferNotFull.wait(lock);
00155
00156                 // Push data into buffer.
00157                 buffer.push_back(*first++);
00158             }
00159
00160             // Notify waiting thread.
00161             bufferNotEmpty.notify_one();
00162         }
00163
00164         // Notify waiting thread.
00165 //        bufferNotEmpty.notify_all();
00166
00167     }
00168
00169     /** Is the queue empty?
00170      *
00171      *  @return         TRUE if the buffer is empty, or FALSE otherwise.
00172      */
00173     bool empty() const ATTR_DEPRECATED {
00174         mutex::scoped_lock lock(mutex);
00175         return buffer.empty();
00176     }
00177
00178     /** Is the queue full?
00179      *
00180      *  @return         TRUE if the buffer is full, or FALSE otherwise.
00181      */
00182     bool full() const ATTR_DEPRECATED {
00183         mutex::scoped_lock lock(mutex);
00184         return buffer.full();
00185     }
00186
00187     /** Get the number of elements currently in the queue.
00188      *
00189      *  @return         The number of elements currently in the queue.
00190      */
00191     size_t size() const ATTR_DEPRECATED {
00192         mutex::scoped_lock lock(mutex);
00193         return buffer.size();
00194     }
00195
00196     /** Pop element out of queue.
00197      *
00198      *  @param[out]     elem        The destination of element to pop out of the queue.
00199      *  @param[in]      block       If TRUE, this function will block until the queue is not empty.
00200      *                              If FALSE, this function will return FALSE if the queue is empty
00201      *                              (and no element will be popped out of the queue).
00202      *
00203      *  @return         TRUE if the element was successfully popped out of the queue, or FALSE
00204      *                  otherwise.
00205      */
00206     bool pop(T& elem, bool block) {
00207
00208         {
00209             boost::unique_lock<boost::mutex> lock(mutex);
00210             if(block)
00211
00212                 // If function is blocking, block until buffer is not empty.
00213                 while(buffer.empty())
00214
00215                     bufferNotEmpty.wait(lock);
00216
00217             else if(buffer.empty())
00218
00219                 // Otherwise, return FALSE if the buffer is empty.
00220                 return false;
00221
00222             // Pop element out of queue.
00223             elem = buffer.front();
00224             buffer.pop_front();
00225         }
00226
00227         // Notify waiting threads (if any), that buffer is no longer full.
00228         bufferNotFull.notify_one();
00229
00230         return true;
00231     }
00232
00233     bool contains(const T& elem) ATTR_DEPRECATED {
00234         mutex::scoped_lock lock(mutex);
00235
00236         bool contains_elem = false;
00237
00238         for_each(buffer.begin(), buffer.end(), [&](const T& obj) {
00239                     if(obj == elem)
00240                     contains_elem = true;
00241                 });
00242
00243         return contains_elem;
00244     }
00245
00246 };
00247
00248 } /* namespace thread */
00249 } /* namespace util */
00250 #endif /* SYNCHRONOUSQUEUE_H_ */
```

11.226 utility/SystemUtils.h File Reference

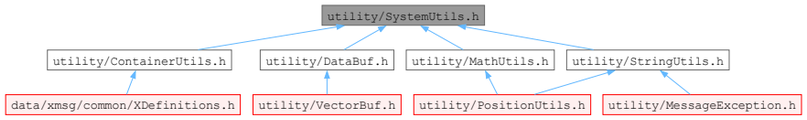
The SystemUtils class header file.

```
#include <cassert>
#include <cfloat>
#include <cmath>
#include <cinttypes>
#include <csignal>
#include <cstdio>
#include <cstdlib>
#include <map>
#include <vector>
#include <iomanip>
#include <sstream>
#include <streambuf>
#include <atomic>
#include <algorithm>
#include <regex>
#include <string>
#include <unistd.h>
```

Include dependency graph for SystemUtils.h:



This graph shows which files directly or indirectly include this file:



Macros

- #define ATTR_DEPRECATED __attribute__ ((__deprecated__))

  The deprecated function attribute macro.

11.226.1 Detailed Description

The SystemUtils class header file.

Version

0.6

Date

Aug 21, 2014 10:41:32 AM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.226.2 Macro Definition Documentation

11.226.2.1 ATTR_DEPRECATED

```
#define ATTR_DEPRECATED __attribute__ ((__deprecated__))
```

The deprecated function attribute macro.

11.227 SystemUtils.h

Go to the documentation of this file.

```
00001 /** The SystemUtils class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Aug 21, 2014 10:41:32 AM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef SYSTEMUTILITIES_H_
00024 #define SYSTEMUTILITIES_H_
00025
00026 /* Includes */
00027
00028 // STL C library
00029 #include <cassert>
00030 #include <cfloat>
00031 #include <cmath>
00032 #include <cinttypes>
00033 #include <csignal>
00034 #include <cstdio>
00035 #include <cstdlib>
00036 //#include <cstring>
00037
00038 // STL container library
00039 #include <map>
00040 #include <vector>
00041
00042 // STL I/O library
00043 #include <iomanip>
00044 #include <sstream>
00045 #include <streambuf>
00046
00047 // STL other libraries
00048 #include <atomic>
00049 #include <algorithm>
00050 #include <regex>
00051 #include <string>
00052
00053 // system libraries
00054 #include <unistd.h>
00055
00056 /* Definitions */
00057
00058 /** The deprecated function attribute macro.
00059  *
00060  */
00061 #define ATTR_DEPRECATED __attribute__ ((__deprecated__))
00062
00063 #endif /* SYSTEMUTILITIES_H_ */
```

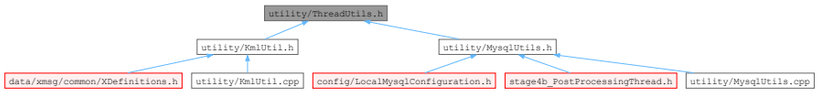
11.228 utility/ThreadUtils.h File Reference

The util::thread::ThreadUtils class header file.

```
#include "Lockable.h"
#include "RecursiveLockable.h"
#include "SynchronousQueue.h"
#include "StreamUtils.h"
```

Include dependency graph for ThreadUtils.h:

This graph shows which files directly or indirectly include this file:



Classes

- class util::thread::ThreadUtils

  Class providing useful functions for manipulating threads and processes.

Namespaces

- namespace util

  The utility namespace.
- namespace util::thread

  The thread utility namespace.

11.228.1 Detailed Description

The util::thread::ThreadUtils class header file.

Version

0.6

Date

Aug 21, 2014 3:20:37 PM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.229 ThreadUtils.h

Go to the documentation of this file.

```
00001 /** The @ref util::thread::ThreadUtils class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       Aug 21, 2014 3:20:37 PM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
```

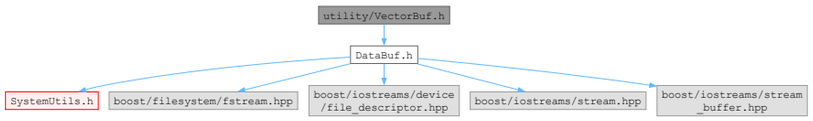
```
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022
00023 #ifndef THREADUTILS_H_
00024 #define THREADUTILS_H_
00025
00026 /* Includes */
00027
00028 // local
00029 #include "Lockable.h"
00030 #include "RecursiveLockable.h"
00031 #include "SynchronousQueue.h"
00032 #include "StreamUtils.h"
00033
00034 namespace util {
00035 namespace thread {
00036
00037 /* Class Definitions */
00038
00039 /** Class providing useful functions for manipulating threads and processes.
00040  *
00041  */
00042 class ThreadUtils {
00043 public:
00044     ThreadUtils() {
00045         /* Do nothing. */
00046     }
00047
00048     virtual ~ThreadUtils() {
00049         /* Do nothing. */
00050     }
00051
00052     /** Invoke an executable on the local filesystem, pipe the output of the command back to this
00053      *  application .
00054      *
00055      *  @param[in]      cmd         A pointer to the C-string containing the local system command to
00056      *                              execute.
00057      *
00058      *  @throw          log::MessageException if the pointer to the command string is null, the
00059      *                  command string is zero length, or an error occurs while executing the external
00060      *                  command or piping its output back to this application.
00061      *
00062      *  @return         The output of the executed command.
00063      */
00064     static std::string executeAndPipe(const char* cmd) {
00065         using bis::stream_buffer;
00066         using bis::file_descriptor_source;
00067         using std::string;
00068         using namespace std;
00069         ostringstream os;
00070
00071         FILE* cmdPipe = NULL;
00072         string line;
00073
00074         try {
00075             // Check command.
00076             if(cmd == NULL)
00077                 EXC_THROW("Pointer to command string is NULL!");
00078             if(strlen(cmd) < 1)
00079                 EXC_THROW("Command string is zero length!");
00080
00081             // Fork, execute the command, and pipe the output of the command back to the application.
00082             dbg1() « "Executing external system command: " « cmd « "...";
00083             if(!(cmdPipe = popen(cmd, "r")))
00084                 EXC_THROW("Failed to pipe stream from process: The call to popen() failed!");
00085
00086             // Buffer the piped output of the command into an istream.
00087             stream_buffer<file_descriptor_source> cmdStreamBuf(
00088                     file_descriptor_source(fileno(cmdPipe), bis::close_handle));
00089             istream cmdStream(&cmdStreamBuf);
00090
00091             // Capture the output of the external command.
00092             while(getline(cmdStream, line))
00093                 os « line « endl;
00094
00095         }CATCH_BLOCK("Failed to invoke process: '" + std::string(cmd) + "'!")
00096
00097         // Return the output of the external command.
00098         return os.str();
00099     }
00100
00101     /** Invoke an executable on the local filesystem, pipe the output of the command back to this
00102      *  application .
00103      *
00104      *  @param[in]      cmd         A reference to the string containing the local system command to
00105      *                              execute.
00106      *
00107      *  @throw          log::MessageException if the pointer to the command string is null, the
00108      *                  command string is zero length, or an error occurs while executing the external
00109      *                  command or piping its output back to this application.
00110      *
00111      *  @return         The output of the executed command.
00112      */
00113     static std::string executeAndPipe(const std::string& cmd) {
00114         return executeAndPipe(cmd.c_str());
00115     }
00116 };
00117
00118 } /* namespace thread */
00119 } /* namespace util */
00120 #endif /* THREADUTILS_H_ */
```

11.230 utility/VectorBuf.h File Reference

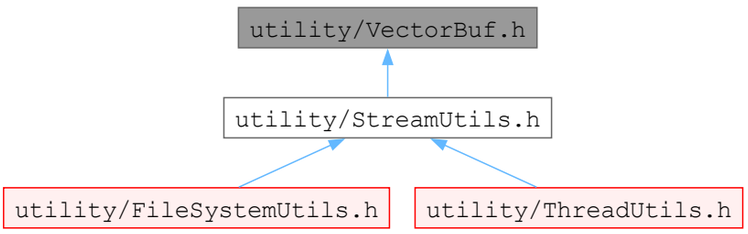
The util::stream::VectorBuf template class header file.

```
#include "DataBuf.h"
```

Include dependency graph for VectorBuf.h:



This graph shows which files directly or indirectly include this file:



Classes

- class util::stream::VectorBuf< CharT, TraitsT >

  Template class for wrapping a vector<T> as a std::basic_streambuf<T>.
- class util::stream::VectorBuf_c< CharT >

  Template class for wrapping a vector<T> as a std::basic_streambuf<char>.

Namespaces

- namespace util

  The utility namespace.
- namespace util::stream

  The stream utility namespace.

11.230.1 Detailed Description

The util::stream::VectorBuf template class header file.

Version

0.6

Date

May 14, 2014 9:11:06 AM

Author

Mr. Alexander K. Ekholm aekholm@whoi.edu, Engineer I, Department of Physical Oceanography, Woods Hole Oceanographic Institution

Copyright

2014 Woods Hole Oceanographic Institution. Released under the GNU General Public License.

This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this software. If not, please visit http://www.gnu.org/licenses/.

11.231 VectorBuf.h

Go to the documentation of this file.

```
00001 /** The @ref util::stream::VectorBuf template class header file.
00002  *
00003  *  @file
00004  *
00005  *  @version    0.6
00006  *
00007  *  @date       May 14, 2014 9:11:06 AM
00008  *
00009  *  @author     Mr. Alexander K. Ekholm <aekholm@whoi.edu>,
00010  *              Engineer I,
00011  *              Department of Physical Oceanography,
00012  *              Woods Hole Oceanographic Institution
00013  *
00014  *  @copyright  2014 Woods Hole Oceanographic Institution.
00015  *              Released under the GNU General Public License.
00016  *
00017  *  This is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
00018  *  the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
00019  *  General Public License for more details. You should have received a copy of the GNU General
00020  *  Public License along with this software. If not, please visit http://www.gnu.org/licenses/.
00021  */
00022 #ifndef VECTORSTREAMBUF_H_
00023 #define VECTORSTREAMBUF_H_
00024
00025 /* Includes */
00026
00027 // local
00028 #include "DataBuf.h"
00029
00030 namespace util {
00031 namespace stream {
00032
00033 /* Template Definitions*/
00034
00035 /** Template class for wrapping a vector<T> as a std::basic_streambuf<T>.
00036  *
00037  */
00038 template<typename CharT, typename TraitsT = std::char_traits<CharT> >
00039 class VectorBuf: public DataBuf<CharT, TraitsT> {
00040 public:
00041     /** Parameterized constructor.
00042      *
00043      *  @param[in]      vec         Vector to wrap.
00044      */
00045     VectorBuf(const std::vector<CharT> &vec) :
00046             DataBuf<CharT>(const_cast<CharT*>(vec.data()), vec.size()) {
00047         /* Do nothing. */
00048     }
00049 };
00050
00051 /** Template class for wrapping a vector<T> as a std::basic_streambuf<char>.
00052  *
00053  */
00054 template<typename CharT>
00055 class VectorBuf_c: public DataBuf<char> {
00056 public:
00057     /** Parameterized constructor.
00058      *
00059      *  @param[in]      vec         Vector to wrap.
00060      */
00061     VectorBuf_c(const std::vector<CharT> &vec) :
00062             DataBuf<char>((char*)vec.data(), vec.size()) {
00063         /* Do nothing. */
00064     }
00065 };
00066
00067 } /* namespace stream */
00068 } /* namespace util */
00069 #endif /* VECTORSTREAMBUF_H_ */
```

Index

~AbstractXComposite
util::xmsg::AbstractXComposite< T >, 62
~AbstractXFallRise
util::xmsg::AbstractXFallRise, 67
~AbstractXSensorData
util::xmsg::AbstractXSensorData, 86
~AbstractXSensorDataComposite
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >, 106
~AuxiliarySensorData
util::xmsg::AuxiliarySensorData, 139
~Iridium
argo::iridium::Iridium, 254
~IridiumPosition
argo::iridium::IridiumPosition, 263
~LocalConfiguration
util::config::LocalConfiguration, 291
~LocalMysqlConfiguration
util::mysql::LocalMysqlConfiguration, 302
~LocalPersistentEntity
util::mysql::LocalPersistentEntity, 315
~Lockable
util::thread::Lockable, 326
~Log
util::log::Log, 328
~MessageException
util::log::MessageException, 334
~Position
util::position::Position, 344
~RecursiveLockable
util::thread::RecursiveLockable, 366
~SqlTableColumnDescriptionBase
util::mysql::SqlTableColumnDescriptionBase, 379
~SqlTableDescription
util::mysql::SqlTableDescription< T >, 387
~XAcousticCacst
util::xmsg::XAcousticCacst, 419
~XAcousticPosition
util::xmsg::XAcousticPosition, 422
~XAcousticWindow
util::xmsg::XAcousticWindow, 434
~XAlamoBit
util::xmsg::XAlamoBit, 438
~XAlamoBit2
util::xmsg::XAlamoBit2, 447
~XAlamoEngineering
util::xmsg::XAlamoEngineering, 459
~XAlamoMissionConfiguration
util::xmsg::XAlamoMissionConfiguration, 484
~XArgo
util::xmsg::XArgo, 501
~XCompositeDrift
util::xmsg::XCompositeDrift, 546
~XCompositeFall
util::xmsg::XCompositeFall, 561
~XCompositeHiResProfile
util::xmsg::XCompositeHiResProfile, 567
~XCompositeIridiumPosition
util::xmsg::XCompositeIridiumPosition, 584
~XCompositeProfile
util::xmsg::XCompositeProfile, 605
~XCompositePump
util::xmsg::XCompositePump, 621
~XCompositeRise
util::xmsg::XCompositeRise, 624
~XCompositeSurfaceData
util::xmsg::XCompositeSurfaceData, 629
~XDriftPressure
util::xmsg::XDriftPressure, 641
~XDriftSalinity
util::xmsg::XDriftSalinity, 660
~XDriftTemperature
util::xmsg::XDriftTemperature, 679
~XEngAbort
util::xmsg::XEngAbort, 697
~XEngAscent
util::xmsg::XEngAscent, 709
~XEngBITFail
util::xmsg::XEngBITFail, 739
~XEngBITPass
util::xmsg::XEngBITPass, 751
~XEngDescent
util::xmsg::XEngDescent, 765
~XEngDiagnostic
util::xmsg::XEngDiagnostic, 785
~XEngDrift
util::xmsg::XEngDrift, 797
~XEngErrorCode
util::xmsg::XEngSurface::XEngErrorCode, 815
~XEngIce
util::xmsg::XEngIce, 823
~XEngIceErrorCode
util::xmsg::XEngIce::XEngIceErrorCode, 839
~XEngStandard
util::xmsg::XEngStandard, 846
~XEngSurface
util::xmsg::XEngSurface, 860
~XFall
util::xmsg::XFall, 883
~XFloat
argo::iridium::xfloat::XFloat, 911
~XFloatDiveData
argo::iridium::xfloat::XFloatDiveData, 982
~XFloatMailMessage
argo::iridium::xfloat::XFloatMailMessage, 1059
~XFloatMailProcessingService
argo::iridium::xfloat::XFloatMailProcessingService, 1139
~XFloatMailRetrievalService
argo::iridium::xfloat::XFloatMailRetrievalService, 1158
~XFloatMetadata
argo::iridium::xfloat::XFloatMetadata, 1188
~XFloatPostProcessingService
argo::iridium::xfloat::XFloatPostProcessingService, 1236
~XFloatTelemetryDecoderService
argo::iridium::xfloat::XFloatTelemetryDecoderService, 1246
~XGPS
util::xmsg::XGPS, 1263
~XHiResPressure
util::xmsg::XHiResPressure, 1309
~XHiResSalinity
util::xmsg::XHiResSalinity, 1328

~XHiResTemperature
util::xmsg::XHiResTemperature, 1347
~XMessage
util::xmsg::XMessage, 1366
~XMessageIndexable
util::xmsg::XMessageIndexable, 1376
~XMfaMissionCfg
util::xmsg::XMfaMissionCfg, 1398
~XMissionCmdEcho
util::xmsg::XMissionCmdEcho, 1416
~XMissionConfiguration
util::xmsg::XMissionConfiguration, 1427
~XPresCodeCounts
util::xmsg::XPresCodeCounts, 1440
~XPresCounts
util::xmsg::XPresCounts, 1445
~XProfilePressure
util::xmsg::XProfilePressure, 1449
~XProfileSalinity
util::xmsg::XProfileSalinity, 1468
~XProfileTemperature
util::xmsg::XProfileTemperature, 1487
~XPump
util::xmsg::XPump, 1505
~XPumpCounts
util::xmsg::XPumpCounts, 1520
~XRise
util::xmsg::XRise, 1527
~XSensorTripletCodeCounts
util::xmsg::XSensorTripletCodeCounts, 1547
~XSurfaceData
util::xmsg::XSurfaceData, 1555
~XValveCounts
util::xmsg::XValveCounts, 1589
~XWave
util::xmsg::XWave, 1593
~XWaveRawSample
util::xmsg::XWaveRawSample, 1608

AbstractXComposite
util::xmsg::AbstractXComposite< T >, 62
AbstractXFallRise
util::xmsg::AbstractXFallRise, 66
AbstractXSensorData
util::xmsg::AbstractXSensorData, 86
AbstractXSensorDataComposite
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >, 106
aca_
util::xmsg::XMfaMissionCfg, 1403
accel
util::xmsg::XWaveRawSample, 1608
accessor_
util::mysql::Column_bool< C >, 160
util::mysql::Column_char< C >, 166
util::mysql::Column_double< C >, 173
util::mysql::Column_filesystem_path< C >, 180
util::mysql::Column_float< C >, 186
util::mysql::Column_int< C >, 193
util::mysql::Column_int16_t< C >, 199
util::mysql::Column_long< C >, 206
util::mysql::Column_posix_time< C >, 213
util::mysql::Column_string< C >, 220
util::mysql::Column_uint16_t< C >, 226
util::mysql::Column_uint8_t< C >, 233
util::mysql::Column_vector_uint8_t< C >, 240
util::mysql::SqlTableColumnDescription< C, T >, 377
acg_
util::xmsg::XMfaMissionCfg, 1403
AcquireRuntimeLock
util::config::LocalConfiguration, 291
acr_
util::xmsg::XMfaMissionCfg, 1403
add_
util::xmsg::XAlamoMissionConfiguration, 489
util::xmsg::XMfaMissionCfg, 1404
AdditionalDocumentation.h, 1611
address_
argo::iridium::xfloat::XFloat, 960
argo::iridium::xfloat::XFloatDiveData, 1037
argo::iridium::xfloat::XFloatMailMessage, 1125
argo::iridium::xfloat::XFloatMailProcessingService, 1152
argo::iridium::xfloat::XFloatMailRetrievalService, 1176
argo::iridium::xfloat::XFloatMetadata, 1227
util::mysql::LocalMysqlConfiguration, 311
util::mysql::LocalPersistentEntity, 324
util::mysql::SqlTableDescription< T >, 404
util::xmsg::XGPS, 1302
adt_
util::xmsg::XAlamoMissionConfiguration, 489
util::xmsg::XMfaMissionCfg, 1404
aggregated_data_valid_
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >, 123
util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >, 136
util::xmsg::XCompositeDrift, 557
util::xmsg::XCompositeHiResProfile, 578
util::xmsg::XCompositeProfile, 616
util::xmsg::XCompositeSurfaceData, 636
aggregated_pres_data_
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >, 123
util::xmsg::XCompositeDrift, 557
util::xmsg::XCompositeHiResProfile, 578
util::xmsg::XCompositeProfile, 616
aggregated_saln_data_
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >, 124
util::xmsg::XCompositeDrift, 557
util::xmsg::XCompositeHiResProfile, 579
util::xmsg::XCompositeProfile, 617
aggregated_temp_data_
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >, 124
util::xmsg::XCompositeDrift, 557
util::xmsg::XCompositeHiResProfile, 579
util::xmsg::XCompositeProfile, 617
AggregateSensorData
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >, 106
util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >, 130
util::xmsg::XCompositeDrift, 546
util::xmsg::XCompositeHiResProfile, 567
util::xmsg::XCompositeProfile, 605
util::xmsg::XCompositeSurfaceData, 630
air_bladder_max_pres_mpsi_
util::xmsg::XEngSurface, 876
air_pump_curr_max_
util::xmsg::XAlamoBit2, 454
air_pump_current_ma_
util::xmsg::XEngSurface, 876
air_pump_cycle_count_
util::xmsg::XAlamoBit2, 454
air_pump_errors_

util::xmsg::XAlamoBit2, 454
air_pump_runtime_s_
util::xmsg::XEngSurface, 876
air_pump_test_duration_
util::xmsg::XAlamoBit2, 454
air_pump_v12_curr_max_
util::xmsg::XAlamoBit2, 454
air_pump_volt_max_
util::xmsg::XAlamoBit2, 454
air_valve_cal_open_
util::xmsg::XAlamoBit2, 454
air_valve_closed_cal_position_mv_
util::xmsg::XEngSurface, 877
air_valve_errors_
util::xmsg::XAlamoBit2, 454
air_valve_mid_cal_position_mv_
util::xmsg::XEngSurface, 877
air_valve_mid_position_at_end_mv_
util::xmsg::XEngSurface, 877
air_valve_middle_position_at_surface_mv_
util::xmsg::XEngDescent, 780
air_valve_open_
util::xmsg::XAlamoBit2, 454
air_valve_open_cal_position_mv_
util::xmsg::XEngSurface, 877
air_valve_open_position_at_start_mv_
util::xmsg::XEngSurface, 877
alamo_bit2_msg_
util::xmsg::XEngBIT, 736
util::xmsg::XEngBITFail, 748
util::xmsg::XEngBITPass, 760
alamo_bit_msg_
util::xmsg::XEngBIT, 736
util::xmsg::XEngBITFail, 748
util::xmsg::XEngBITPass, 760
alamo_mission_configuration_
util::xmsg::XMissionConfiguration, 1437
apf_
util::xmsg::XAlamoMissionConfiguration, 489
util::xmsg::XMfaMissionCfg, 1404
apn_
util::xmsg::XAlamoMissionConfiguration, 489
util::xmsg::XMfaMissionCfg, 1404
AppendAllElements
util::xmsg::AbstractXComposite< T >, 62, 63
util::xmsg::XCompositeAuxTemp, 540, 541
util::xmsg::XCompositeFall, 562
util::xmsg::XCompositeIridiumPosition, 584
util::xmsg::XCompositePump, 621
util::xmsg::XCompositeRise, 625
util::xmsg::XCompositeValve, 637, 638
util::xmsg::XPump, 1506
util::xmsg::XValve, 1576
AppendElement
util::xmsg::AbstractXComposite< T >, 63
util::xmsg::XCompositeAuxTemp, 541
util::xmsg::XCompositeFall, 562
util::xmsg::XCompositeIridiumPosition, 584
util::xmsg::XCompositePump, 621
util::xmsg::XCompositeRise, 625
util::xmsg::XCompositeValve, 638
util::xmsg::XPump, 1506
util::xmsg::XValve, 1576
apt_
util::xmsg::XAlamoMissionConfiguration, 489
util::xmsg::XMfaMissionCfg, 1404
argo, 37
argo::iridium, 37
argo::iridium::Iridium, 252
~Iridium, 254
id_dac, 254
id_dac_, 258
id_imei, 254
id_imei_, 258
id_instrument, 254
id_instrument_, 258
id_wmo, 255
id_wmo_, 259
Iridium, 253
LockObject, 255
mutex_, 259
set_id_dac, 256
set_id_imei, 256
set_id_instrument, 257
set_id_wmo, 257
UnlockObject, 258
argo::iridium::IridiumPosition, 260
~IridiumPosition, 263
CalculateEllipsoidHeight, 263
CalculateGeographicDistance, 264
degToRad, 265
EARTH_RADIUS_KM_EQUATORIAL, 281
EARTH_RADIUS_KM_POLAR, 281
GenerateCurrentUtcTimeIso8601String, 265
GenerateIso6709String, 266
GenerateIso8601String, 266
GenerateJulianDay, 267
GetCurrentUtcTime, 268
iridium_position_as_iso6709_string, 268
iridium_position_cep_radius, 268
iridium_position_cep_radius_, 281
iridium_position_datetime, 269
iridium_position_datetime_as_iso8601_string, 269
iridium_position_is_valid, 270
iridium_position_latitude, 270
iridium_position_longitude, 271
IridiumPosition, 263
IsPositionValid, 271
position_datetime, 272
position_datetime_, 281
position_datetime_as_iso8601_string, 272
position_is_valid, 273
position_is_valid_, 282
position_latitude, 273
position_latitude_, 282
position_longitude, 274
position_longitude_, 282
postition_as_iso6709_string, 274
set_gps_position_datetime, 275
set_iridium_position_cep_radius, 276
set_iridium_position_datetime, 276
set_iridium_position_is_valid, 277
set_iridium_position_latitude, 277
set_iridium_position_longitude, 277
set_position_datetime, 278
set_position_is_valid, 278
set_position_latitude, 279
set_position_longitude, 279
argo::iridium::xfloat, 37
operator<<, 39
operator>>, 40
XFloatMailRawFileFormat_t, 39
XMAIL_RAW_FORMAT_EXTENDED, 39
XMAIL_RAW_FORMAT_STANDARD, 39
XMailSbdByte_t, 39
XMailSbdData_t, 39
argo::iridium::xfloat::XFloat, 902
~XFloat, 911
address_, 960
argo_instrument_type, 911
argo_instrument_type_, 960
CalculateEllipsoidHeight, 911
CalculateGeographicDistance, 912
CleanupSqlConnection, 913

CleanupSqlDriver, 913
CloseKmlFactory, 913
CloseSqlConnection, 914
degToRad, 914
dive2data_map_, 960
EARTH_RADIUS_KM_EQUATORIAL, 960
EARTH_RADIUS_KM_POLAR, 961
ExecuteQuery, 914
executeQuery, 915
executeSelectQuery, 916
ExecuteUpdate, 917
executeUpdate, 917
fw_version, 918
fw_version_, 961
gain_pressure, 918
gain_pressure_, 961
gain_pressure_surface, 918
gain_pressure_surface_, 961
gain_salinity, 919
gain_salinity_, 961
gain_salinity_surface, 919
gain_salinity_surface_, 961
gain_temperature, 919
gain_temperature_, 962
gain_temperature_surface, 920
gain_temperature_surface_, 962
GenerateCurrentUtcTimeIso8601String, 920
GenerateIso6709String, 920
GenerateIso8601String, 921
GenerateJulianDay, 922
GenerateKmlMasterDocument, 922
GenerateKmlMasterString, 923
GenerateKmlString, 923
GeneratePreparedStatement, 924
generatePreparedStatement, 924
GenerateSqlDeleteStr, 925
GenerateSqlSelectStr, 925
GetAllCachedFloats, 926
GetAllDiveData, 926
GetCurrentUtcTime, 926
GetDefaltKmlPathColor, 927
GetDefaltKmlPathStyleId, 927
GetDefaltKmlPlacemarkStyleId, 928
GetDirectoryPath, 928
GetDirectoryPathFromString, 929
GetDiveData, 930
GetFloat, 931
GetKmlFactory, 931
GetListingDirectoryPath, 932
GetSmallKmlPlacemarkStyleId, 932
GetSqlConnection, 932
getSqlConnection, 933
GetSqlDriver, 934
GetTriangleKmlPlacemarkStyleId, 934
id2float_map_, 962
id2float_map_mutex_, 962
id_dac, 935
id_dac_, 962
id_imei, 935
id_imei_, 963
id_instrument, 935
id_instrument_, 963
id_wmo, 936
id_wmo_, 963
Initialize, 959
instrument_ids_to_disable_phy_generation, 936
instrument_ids_to_disable_phy_generation_, 963
instrument_ids_to_enable_phy_generation, 937
instrument_ids_to_enable_phy_generation_, 963
IsPositionValid, 937
kml_default_path_colors_, 963
kml_default_path_style_ids_, 964
kml_default_placemark_style_ids_, 964
kml_directory_path_, 964
kml_document_, 964
kml_document_updated_, 965
kml_factory_, 965
kml_file_generation_is_enabled_, 965
kml_file_path_, 965
kml_file_updated_, 965
kml_small_placemark_style_ids_, 965
kml_triangle_placemark_style_ids_, 966
latest_gps_position_datetime_, 966
latest_gps_position_dive_number_, 966
latest_gps_position_latitude_, 966
latest_gps_position_longitude_, 966
latest_iridium_position_cep_radius_, 966
latest_iridium_position_datetime_, 967
latest_iridium_position_dive_number_, 967
latest_iridium_position_latitude_, 967
latest_iridium_position_longitude_, 967
latest_iridium_sbd_momsn_, 967
latest_iridium_sbd_mtmsn_, 967
latest_iridium_startup_position_datetime_, 967
latest_iridium_startup_sbd_momsn_, 967
ListDirectories, 937
ListFiles, 938
LoadLocalCacheFromDatabase, 938
LoadMetadataFromFiles, 939
LockObject, 939
metadata, 940
metadata_, 967
MoveFileToDirectory, 940
mutex_, 968
netcdf_file_generation_is_enabled_, 969
offset_pressure, 941
offset_pressure_, 969
offset_pressure_surface, 941
offset_pressure_surface_, 969
offset_salinity, 941
offset_salinity_, 970
offset_salinity_surface, 942
offset_salinity_surface_, 970
offset_temperature, 942
offset_temperature_, 970
offset_temperature_surface, 942
offset_temperature_surface_, 970
password_, 970
phy_file_generation_is_enabled, 943
phy_file_generation_is_enabled_, 970
port_, 971
protocol_, 971
ResetCachedDiveData, 943
ResetDiveData, 944
SaveLocalCacheToDatabase, 944
SaveObjectToDatabase, 944
schema_, 971
SelectAllObjectsFromDatabase, 944
SelectObjectFromDatabase, 945
SelectObjectsFromDatabase, 945
SelectValueFromDatabase, 946
set_argo_instrument_type, 946
set_fw_version, 947
set_gain_pressure, 947
set_gain_pressure_surface, 948
set_gain_salinity, 948
set_gain_salinity_surface, 948
set_gain_temperature, 949
set_gain_temperature_surface, 949
set_id_dac, 949
set_id_imei, 950
set_id_instrument, 950
set_id_wmo, 951
set_metadata, 951

set_offset_pressure, 952
set_offset_pressure_surface, 952
set_offset_salinity, 953
set_offset_salinity_surface, 953
set_offset_temperature, 953
set_offset_temperature_surface, 954
set_phy_file_generation_is_enabled, 954
set_start_of_mission_datetime, 954
set_xmsg_version_major, 955
set_xmsg_version_minor, 955
SetAllDefaultKmlSyles, 955
SetDefaultKmlStyle, 956
sql_connection_, 971
sql_driver_, 971
sql_table_description_, 971
start_of_mission_datetime, 957
start_of_mission_datetime_, 972
StartupTasks, 957
status_directory_path_, 972
status_file_generation_is_enabled_, 972
status_file_path_, 972
status_file_updated_, 972
UnlockObject, 957
user_, 972
util::config::LocalConfiguration::GenerateCommandLineOptionsDescription, 960
util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription, 960
WriteKmlFile, 958
WriteKmlMasterFile, 958
WriteStatusFile, 958
XFloat, 910
xmsg_version_major, 958
xmsg_version_major_, 973
xmsg_version_minor, 959
xmsg_version_minor_, 973
argo::iridium::xfloat::XFloatDiveData, 973
~XFloatDiveData, 982
address_, 1037
argo_instrument_type, 983
argo_instrument_type_, 1037
AssignArgoData, 983
AssignAuxTemperatureBinned, 983
AssignDriftPressure, 983
AssignDriftSalinity, 984
AssignDriftTemperatre, 984
AssignEngineeringAbort, 984
AssignEngineeringAscent, 985
AssignEngineeringBitFail, 985
AssignEngineeringBitPass, 985
AssignEngineeringDescent, 985
AssignEngineeringDiagnostic, 986
AssignEngineeringDrift, 986
AssignEngineeringIce, 986
AssignEngineeringStandard, 986
AssignEngineeringSurface, 987
AssignFallTimeSeries, 987
AssignGps, 987
AssignMailMessage, 987
AssignMissionCmdEchoback, 988
AssignParentInstrument, 988
AssignProfilePressureBinned, 989
AssignProfilePressureUnbinned, 989
AssignProfileSalinityBinned, 990
AssignProfileSalinityUnbinned, 990
AssignProfileTemperatureBinned, 990
AssignProfileTemperatureUnbinned, 991
AssignPumpTimeSeries, 991
AssignRiseTimeSeries, 992
AssignSurfaceTimeSeries, 992
AssignValveData, 992
CheckCompleteness, 993
CleanupSqlConnection, 993
CleanupSqlDriver, 994
CloseSqlConnection, 994
complete_, 1037
ComputePressureOffsetFromSurfaceTimeSeries, 994
DecodeMailMessageData, 995
DecodeXMsgData, 996
DeleteFromDatabase, 996
DeleteObjectsFromDatabase, 997
ExecuteQuery, 997
executeQuery, 998
executeSelectQuery, 999
ExecuteUpdate, 1000
executeUpdate, 1000
GenerateCompositeFallTimeSeriesData, 1001
GenerateCompositeIridiumPosition, 1001
GenerateCompositePumpTimeSeriesData, 1002
GenerateCompositeRiseTimeSeriesData, 1002
GenerateCurrentUtcTimeIso8601String, 1003
GenerateIso8601String, 1003
GenerateJulianDay, 1004
GenerateListingString, 1005
GenerateOrderedMomsnList, 1005
GenerateOrderedPidList, 1006
GeneratePhyString, 1006
GeneratePreparedStatement, 1007
generatePreparedStatement, 1008
GenerateReadableHex, 1008
GenerateSqlDeleteStr, 1009
GenerateSqlSelectStr, 1009
GetCurrentUtcTime, 1009
GetDirectoryPath, 1010
GetDirectoryPathFromString, 1010
GetDiveDataMailMessagesFromDatabase, 1011
GetDiveDataObjectFromDatabase, 1012
GetGainsAndOffsets, 1012
GetGpsPositionEndOfDive, 1013
GetSqlConnection, 1013
getSqlConnection, 1014
GetSqlDriver, 1015
GetSurfaceGainsAndOffsets, 1015
HasBinnedProfileData, 1016
HasCompleteBinnedProfileData, 1016
HasCompleteProfileData, 1017
HasCompleteUnbinnedProfileData, 1018
HasContiguousMomsns, 1019
HasContiguousPids, 1019
HasDiveData, 1020
HasDriftData, 1020
HasEngineeringData, 1021
HasFallRiseData, 1021
HasPositionData, 1021
HasUnbinnedProfileData, 1022
HasValidGpsPosition, 1022
HasValidIridiumPosition, 1022
HasValidPosition, 1023
id_dac, 1023
id_dac_, 1038
id_dive, 1024
id_dive_, 1038
id_imei, 1024
id_imei_, 1038
id_instrument, 1025
id_instrument_, 1038
id_wmo, 1025
id_wmo_, 1038
intermediate_listing_directory_path_, 1038
intermediate_listing_file_generation_is_enabled_, 1039
intermediate_listing_file_path, 1026
iridium_position_datetime_first_, 1039

iridium_position_datetime_last_, 1039
ListDirectories, 1026
ListFiles, 1026
listing_file_path_, 1039
listing_file_updated_, 1039
LockObject, 1027
mail_messages_, 1039
MoveFileToDirectory, 1027
mutex_, 1040
netcdf_directory_path_, 1041
netcdf_file_generation_is_enabled_, 1041
netcdf_file_path_, 1041
netcdf_file_updated_, 1042
netcdf_reference_datetime_, 1042
num_sbd_msgs_, 1042
parent_instrument_, 1042
password_, 1042
phy_directory_path_, 1042
phy_file_generation_is_enabled_, 1043
phy_file_path, 1028
phy_file_path_, 1043
phy_file_updated_, 1043
port_, 1043
pres_offs_, 1043
pres_offs_num_dives_, 1043
pres_offs_num_samples_, 1044
pres_offs_valid_, 1044
pressure_offset_correction_enabled_list_file_, 1044
pressure_offset_correction_enabled_netcdf_file_, 1044
pressure_offset_correction_enabled_phy_file_, 1044
protocol_, 1044
ResetData, 1028
SaveObjectToDatabase, 1028
schema_, 1045
SelectObjectFromDatabase, 1029
SelectObjectsFromDatabase, 1029
SelectValueFromDatabase, 1030
SensorGainsAndOffsetsAreKnown, 1030
set_argo_instrument_type, 1031
set_id_dac, 1031
set_id_dive, 1031
set_id_imei, 1032
set_id_instrument, 1032
set_id_wmo, 1033
set_intermediate_listing_file_path, 1033
set_phy_file_path, 1034
sql_connection_, 1045
sql_driver_, 1045
sql_table_description_, 1045
std::make_shared, 1037
surface_pres_max_, 1045
surface_pres_mean_, 1045
surface_pres_min_, 1046
surface_pres_num_samples_, 1046
surface_pres_valid_, 1046
surface_pres_variance_, 1046
UnlockObject, 1034
UpdateDerivedSurfaceDataFields, 1034
user_, 1046
util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription, 1037
WriteListingFile, 1035
WritePhyFile, 1036
XFloatDiveData, 982
xmsg_acoustic_positions_, 1046
xmsg_argo_data_, 1046
xmsg_composite_drift_data_, 1046
xmsg_composite_profile_binned_, 1047
xmsg_composite_profile_unbinned_, 1047
xmsg_engineering_data_abort_, 1047
xmsg_engineering_data_ascent_, 1047
xmsg_engineering_data_bit_fail_, 1047
xmsg_engineering_data_bit_passed_, 1048
xmsg_engineering_data_descent_, 1048
xmsg_engineering_data_diagnostic_dive_, 1048
xmsg_engineering_data_drift_, 1048
xmsg_engineering_data_ice_, 1048
xmsg_engineering_data_standard_dive_, 1048
xmsg_engineering_data_surface_, 1049
xmsg_fall_time_series_, 1049
xmsg_gps_positions_, 1049
xmsg_mission_cfg_, 1049
xmsg_mission_cmd_echoback_, 1049
xmsg_pump_time_series_, 1050
xmsg_rise_time_series_, 1050
xmsg_surface_time_series_, 1050
xmsg_valve_data_, 1050
xmsg_wave_field_, 1050
argo::iridium::xfloat::XFloatMailMessage, 1051
~XFloatMailMessage, 1059
address_, 1125
CalculateEllipsoidHeight, 1059
CalculateGeographicDistance, 1060
CleanupSqlConnection, 1061
CleanupSqlDriver, 1061
CloseSqlConnection, 1061
degToRad, 1063
EARTH_RADIUS_KM_EQUATORIAL, 1125
EARTH_RADIUS_KM_POLAR, 1125
ExecuteQuery, 1063
executeQuery, 1064
executeSelectQuery, 1065
ExecuteUpdate, 1066
executeUpdate, 1066
GenerateCurrentUtcTimeIso8601String, 1067
GenerateIso6709String, 1067
GenerateIso8601String, 1068
GenerateJulianDay, 1069
GenerateListingComment, 1069
GenerateListingString, 1070
GeneratePreparedStatement, 1070
generatePreparedStatement, 1071
GenerateSqlDeleteStr, 1071
GenerateSqlSelectStr, 1072
GetCurrentUtcTime, 1072
GetDirectoryPath, 1073
GetDirectoryPathFromString, 1073
GetSqlConnection, 1074
getSqlConnection, 1075
GetSqlDriver, 1075
id_dac, 1076
id_dac_, 1125
id_dive, 1076
id_dive_, 1125
id_imei, 1077
id_imei_, 1125
id_instrument, 1077
id_instrument_, 1125
id_msg, 1078
id_msg_, 1126
id_msg_uid, 1078
id_msg_uid_, 1126
id_sbd_momsn, 1079
id_sbd_momsn_, 1126
id_sbd_mtmsn, 1079
id_sbd_mtmsn_, 1126
id_sbd_packet, 1080
id_sbd_packet_, 1127
id_wmo, 1080
id_wmo_, 1127
iridium_position_as_iso6709_string, 1081
iridium_position_cep_radius, 1081
iridium_position_cep_radius_, 1127

iridium_position_datetime, 1082
iridium_position_datetime_as_iso8601_string, 1082
iridium_position_is_valid, 1083
iridium_position_latitude, 1083
iridium_position_longitude, 1084
IsPositionValid, 1084
ListDirectories, 1085
ListFiles, 1085
LockObject, 1086
MoveFileToDirectory, 1086
msg_body_is_valid, 1087
msg_body_is_valid_, 1127
msg_body_text, 1087
msg_body_text_, 1127
msg_header_datetime, 1088
msg_header_datetime_, 1128
msg_header_from, 1088
msg_header_from_, 1128
msg_header_is_valid, 1088
msg_header_is_valid_, 1128
msg_header_raw_text, 1089
msg_header_raw_text_, 1128
msg_header_subject, 1089
msg_header_subject_, 1129
msg_header_valid_sender_regex_, 1129
msg_header_valid_subject_regex_, 1129
msg_is_valid, 1089
msg_is_valid_, 1129
msg_raw_text, 1090
msg_raw_text_, 1129
mutex_, 1130
operator<<, 1124
operator>>, 1124
ParseMailMsg, 1090, 1091
ParseMailMsgBody, 1092
ParseMailMsgHeader, 1093
ParseMailMsgSbdAttachment, 1094
password_, 1131
port_, 1131
position_datetime, 1095
position_datetime_, 1131
position_datetime_as_iso8601_string, 1095
position_is_valid, 1096
position_is_valid_, 1131
position_latitude, 1096
position_latitude_, 1132
position_longitude, 1097
position_longitude_, 1132
postition_as_iso6709_string, 1097
protocol_, 1132
raw_directory_path_, 1132
raw_file_format_, 1132
raw_file_generation_is_enabled_, 1132
raw_file_local_path, 1098
raw_file_local_path_, 1133
SaveObjectToDatabase, 1098
sbd_data, 1099
sbd_data_, 1133
sbd_data_is_valid, 1099
sbd_data_is_valid_, 1133
sbd_directory_path_, 1133
sbd_file_generation_is_enabled_, 1133
sbd_file_local_path, 1099
sbd_file_local_path_, 1134
sbd_file_name, 1100
sbd_file_name_, 1134
sbd_file_size_bytes, 1100
sbd_file_size_bytes_, 1134
sbd_file_valid_name_regex_, 1134
sbd_session_status_code, 1100
sbd_session_status_code_, 1134
sbd_session_status_string, 1101
sbd_session_status_string_, 1135
schema_, 1135
SelectAllObjectsFromDatabase, 1101
SelectObjectFromDatabase, 1101
SelectObjectsFromDatabase, 1103
SelectValueFromDatabase, 1103
set_gps_position_datetime, 1104
set_id_dac, 1105
set_id_dive, 1105
set_id_imei, 1106
set_id_instrument, 1106
set_id_msg, 1107
set_id_msg_uid, 1107
set_id_sbd_momsn, 1108
set_id_sbd_mtmsn, 1108
set_id_sbd_packet, 1108
set_id_wmo, 1109
set_iridium_position_cep_radius, 1109
set_iridium_position_datetime, 1110
set_iridium_position_is_valid, 1110
set_iridium_position_latitude, 1111
set_iridium_position_longitude, 1111
set_msg_body_is_valid, 1112
set_msg_body_text, 1112
set_msg_header_datetime, 1113
set_msg_header_from, 1113
set_msg_header_is_valid, 1113
set_msg_header_raw_text, 1114
set_msg_header_subject, 1114
set_msg_is_valid, 1114
set_msg_raw_text, 1115
set_position_datetime, 1115
set_position_is_valid, 1115
set_position_latitude, 1116
set_position_longitude, 1116
set_raw_file_local_path, 1117
set_sbd_data, 1117
set_sbd_data_is_valid, 1119
set_sbd_file_local_path, 1119
set_sbd_file_name, 1119
set_sbd_file_size_bytes, 1120
set_sbd_session_status_code, 1120
set_sbd_session_status_string, 1120
sql_connection_, 1135
sql_driver_, 1135
sql_table_description_, 1135
UnlockObject, 1120
user_, 1136
util::config::LocalConfiguration::GenerateConfigurationFileOptionsDesc 1124
WriteFiles, 1121
WriteRawFile, 1121
WriteSbdFile, 1122
XFloatMailMessage, 1059
argo::iridium::xfloat::XFloatMailProcessingService, 1136
~XFloatMailProcessingService, 1139
address_, 1152
CleanupSqlConnection, 1139
CleanupSqlDriver, 1139
CloseSqlConnection, 1139
ExecuteQuery, 1140
executeQuery, 1140
executeSelectQuery, 1141
ExecuteUpdate, 1142
executeUpdate, 1143
GeneratePreparedStatement, 1143
generatePreparedStatement, 1144
GenerateSqlDeleteStr, 1144
GenerateSqlSelectStr, 1145
GetSqlConnection, 1145
getSqlConnection, 1146
GetSqlDriver, 1147

msg_processing_queue_, 1152
MsgProcessingQueuePop, 1147
MsgProcessingQueuePush, 1148
MsgProcessingQueuePushAll, 1148
num_threads, 1149
num_threads_, 1152
operator(), 1149
password_, 1152
port_, 1153
ProcessMailMsg, 1149
protocol_, 1153
schema_, 1153
set_termination_signaled, 1150
sql_connection_, 1153
sql_driver_, 1153
StartWorkerThreads, 1151
TerminateWorkerThreads, 1151
termination_signaled_, 1153
thread_group_, 1154
thread_objects_, 1154
user_, 1154
util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescription, 1152
XFloatMailProcessingService, 1139
argo::iridium::xfloat::XFloatMailRetrievalService, 1155
~XFloatMailRetrievalService, 1158
address_, 1176
CleanupSqlConnection, 1158
CleanupSqlDriver, 1159
CloseSqlConnection, 1159
ConnectToMailServer, 1159
DisconnectFromMailServer, 1160
ExecuteQuery, 1160
executeQuery, 1161
executeSelectQuery, 1162
ExecuteUpdate, 1163
executeUpdate, 1163
FetchMailMessagesFromFileSystem, 1164
FetchMailMessagesFromServer, 1164
GenerateMailServerUrlString, 1165
GeneratePreparedStatement, 1165
generatePreparedStatement, 1166
GenerateSqlDeleteStr, 1166
GenerateSqlSelectStr, 1167
GetDirectoryPath, 1167
GetDirectoryPathFromString, 1168
GetSqlConnection, 1169
getSqlConnection, 1170
GetSqlDriver, 1170
inbox_folder_, 1176
is_reprocess_all_, 1177
ListDirectories, 1171
ListFiles, 1171
ListRawMailFiles, 1172
mail_fetch_mode_, 1177
mail_server_host_, 1177
mail_server_inbox_folder_path_, 1177
mail_server_password_, 1177
mail_server_protocol_, 1178
mail_server_user_, 1178
mail_session, 1172
mail_session_, 1178
mail_store_, 1178
max_num_msgs_to_fetch_, 1178
message_start_index_, 1178
move_mail_messages_, 1179
MoveFileToDirectory, 1173
OpenMailInboxFolder, 1173
operator(), 1174
password_, 1179
port_, 1179
processing_thread_propagation_enabled_, 1179
protocol_, 1179
reprocess_by_inst_ids_, 1179
schema_, 1180
set_termination_signaled, 1175
sleep_seconds_if_no_new_msgs_, 1180
sql_connection_, 1180
sql_driver_, 1180
StartWorkerThread, 1175
TerminateWorkerThread, 1176
termination_signaled_, 1180
thread_, 1181
thread_object_, 1181
user_, 1181
util::config::LocalConfiguration::GenerateCommandLineOptionsDescrip 1176
util::config::LocalConfiguration::GenerateConfigurationFileOptionsDesc 1176
XFloatMailRetrievalService, 1158
argo::iridium::xfloat::XFloatMetadata, 1182
~XFloatMetadata, 1188
address_, 1227
CalculateEffectiveHeight, 1188
CalculateGeographicDistance, 1188
CleanupSqlConnection, 1189
CleanupSqlDriver, 1190
CloseSqlConnection, 1190
comment, 1190
comment_, 1227
degToRad, 1191
deployer, 1191
deployer_, 1227
deployer_address, 1191
deployer_address_, 1228
deployment_cruise_id, 1192
deployment_cruiseId_, 1228
deployment_datetime, 1192
deployment_datetime_status, 1192
deployment_datetime_status_, 1228
deployment_latitude, 1192
deployment_longitude, 1193
deployment_platform, 1193
deployment_platform_, 1228
deployment_position_status, 1193
deployment_position_status_, 1228
deployment_type, 1194
deployment_type_, 1228
EARTH_RADIUS_KM_EQUATORIAL, 1228
EARTH_RADIUS_KM_POLAR, 1229
ExecuteQuery, 1194
executeQuery, 1194
executeSelectQuery, 1195
ExecuteUpdate, 1196
executeUpdate, 1196
GenerateCurrentUtcTimeIso8601String, 1197
GenerateIso6709String, 1197
GenerateIso8601String, 1198
GenerateJulianDay, 1199
GeneratePreparedStatement, 1199
generatePreparedStatement, 1200
GenerateSqlDeleteStr, 1200
GenerateSqlSelectStr, 1201
GetCurrentUtcTime, 1201
GetDirectoryPath, 1202
GetDirectoryPathFromString, 1202
GetSqlConnection, 1203
getSqlConnection, 1204
GetSqlDriver, 1204
id_dac, 1205
id_dac_, 1229
id_imei, 1205
id_imei_, 1229
id_instrument, 1206

id_instrument_, 1229
id_wmo, 1207
id_wmo_, 1229
IsPositionValid, 1207
ListDirectories, 1207
ListFiles, 1208
ListMetadataFiles, 1208
LoadFile, 1209
LoadMetadataFromFiles, 1209
local_path, 1210
local_path_, 1229
LockObject, 1210
manufacturer, 1210
manufacturer_, 1229
metadata_directory_path_, 1230
MoveFileToDirectory, 1211
mutex_, 1230
operator>>, 1227
password_, 1231
port_, 1231
position_datetime, 1211
position_datetime_, 1232
position_datetime_as_iso8601_string, 1212
position_is_valid, 1212
position_is_valid_, 1232
position_latitude, 1213
position_latitude_, 1232
position_longitude, 1213
position_longitude_, 1232
postition_as_iso6709_string, 1214
principal_investigators, 1214
principal_investigators_, 1232
principal_investigators_address, 1215
principal_investigators_address_, 1232
principal_investigators_originating_country, 1215
principal_investigators_originating_country_, 1232
protocol_, 1233
SaveObjectToDatabase, 1215
schema_, 1233
set_comment, 1215
set_deployer, 1216
set_deployer_address, 1216
set_deployment_cruise_id, 1216
set_deployment_datetime, 1217
set_deployment_datetime_status, 1217
set_deployment_latitude, 1217
set_deployment_longitude, 1217
set_deployment_platform, 1219
set_deployment_position_status, 1219
set_deployment_type, 1219
set_gps_position_datetime, 1219
set_id_dac, 1220
set_id_imei, 1220
set_id_instrument, 1221
set_id_wmo, 1221
set_local_path, 1222
set_manufacturer, 1222
set_position_datetime, 1223
set_position_is_valid, 1223
set_position_latitude, 1223
set_position_longitude, 1224
set_principal_investigators, 1225
set_principal_investigators_address, 1225
set_principal_investigators_originating_country, 1225
set_wmo_instrument_type, 1226
set_wmo_recorder_type, 1226
sql_connection_, 1233
sql_driver_, 1233
sql_table_description_, 1233
UnlockObject, 1226
user_, 1233
util::config::LocalConfiguration::GenerateConfigurationFileOptionsDesc 1227
wmo_instrument_type, 1226
wmo_instrument_type_, 1234
wmo_recorder_type, 1226
wmo_recorder_type_, 1234
XFloatMetadata, 1187
argo::iridium::xfloat::XFloatPostProcessingService, 1234
~XFloatPostProcessingService, 1236
CloseKmlFactory, 1236
GetDefaltKmlPathColor, 1236
GetDefaltKmlPathStyleId, 1236
GetDefaltKmlPlacemarkStyleId, 1237
GetKmlFactory, 1237
GetSmallKmlPlacemarkStyleId, 1238
GetTriangleKmlPlacemarkStyleId, 1238
idle_sleep_seconds_, 1241
kml_default_path_colors_, 1241
kml_default_path_style_ids_, 1242
kml_default_placemark_style_ids_, 1242
kml_factory_, 1242
kml_small_placemark_style_ids_, 1242
kml_triangle_placemark_style_ids_, 1243
operator(), 1238
SetAllDefaultKmlSyles, 1239
SetDefaultKmlStyle, 1240
StartWorkerThread, 1240
TerminateWorkerThread, 1241
termination_signaled_, 1243
thread_, 1243
thread_object_, 1243
util::config::LocalConfiguration::GenerateConfigurationFileOptionsDesc 1241
XFloatPostProcessingService, 1236
argo::iridium::xfloat::XFloatTelemetryDecoderService, 1244
~XFloatTelemetryDecoderService, 1246
CleanupSqlConnection, 1246
CleanupSqlDriver, 1246
CloseSqlConnection, 1247
DecodeMessage, 1247
ExecuteQuery, 1248
ExecuteUpdate, 1249
GeneratePreparedStatement, 1250
GenerateSqlDeleteStr, 1250
GenerateSqlSelectStr, 1251
GetSqlConnection, 1251
GetSqlDriver, 1252
msg_decoding_queue_, 1256
MsgDecodingQueuePush, 1252
MsgProcessingQueuePop, 1253
num_threads, 1253
num_threads_, 1256
operator(), 1254
ProcessDiveData, 1254
set_termination_signaled, 1255
sql_connection_, 1257
sql_driver_, 1257
StartWorkerThreads, 1255
TerminateWorkerThreads, 1256
termination_signaled_, 1257
thread_group_, 1257
thread_objects_, 1257
util::config::LocalConfiguration::GenerateConfigurationFileOptionsDesc 1256
XFloatTelemetryDecoderService, 1246
argo_instrument_type
argo::iridium::xfloat::XFloat, 911
argo::iridium::xfloat::XFloatDiveData, 983
util::xmsg::XArgo, 501
argo_instrument_type_
argo::iridium::xfloat::XFloat, 960

argo::iridium::xfloat::XFloatDiveData, 1037
util::xmsg::XArgo, 533
arr_
util::xmsg::XAlamoMissionConfiguration, 489
util::xmsg::XMfaMissionCfg, 1404
ascent_start_time_
util::xmsg::XAlamoMissionConfiguration, 489
util::xmsg::XMfaMissionCfg, 1404
ascent_state_start_time
util::xmsg::XArgo, 502
ascent_state_start_time_
util::xmsg::XArgo, 533
asd_
util::xmsg::XAlamoMissionConfiguration, 489
util::xmsg::XMfaMissionCfg, 1404
asi_
util::xmsg::XAlamoMissionConfiguration, 489
util::xmsg::XMfaMissionCfg, 1404
asn_
util::xmsg::XAlamoMissionConfiguration, 489
util::xmsg::XMfaMissionCfg, 1405
assf_
util::xmsg::XWave, 1604
ASSF_RAW
util::xmsg::XWave, 1604
ASSF_SPECTRAL_V1
util::xmsg::XWave, 1604
ASSF_SPECTRAL_V2
util::xmsg::XWave, 1604
AssignArgoData
argo::iridium::xfloat::XFloatDiveData, 983
AssignAuxTemperatureBinned
argo::iridium::xfloat::XFloatDiveData, 983
AssignAuxTemperatureMessage
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >, 107
util::xmsg::XCompositeDrift, 546
util::xmsg::XCompositeHiResProfile, 568
util::xmsg::XCompositeProfile, 606
AssignDriftPressure
argo::iridium::xfloat::XFloatDiveData, 983
AssignDriftSalinity
argo::iridium::xfloat::XFloatDiveData, 984
AssignDriftTemperatre
argo::iridium::xfloat::XFloatDiveData, 984
AssignElement
util::xmsg::AbstractXComposite< T >, 63
util::xmsg::XCompositeAuxTemp, 541
util::xmsg::XCompositeFall, 562
util::xmsg::XCompositeIridiumPosition, 585
util::xmsg::XCompositePump, 621
util::xmsg::XCompositeRise, 625
util::xmsg::XCompositeValve, 638
util::xmsg::XPump, 1506
util::xmsg::XValve, 1576
AssignEngineeringAbort
argo::iridium::xfloat::XFloatDiveData, 984
AssignEngineeringAscent
argo::iridium::xfloat::XFloatDiveData, 985
AssignEngineeringBitFail
argo::iridium::xfloat::XFloatDiveData, 985
AssignEngineeringBitPass
argo::iridium::xfloat::XFloatDiveData, 985
AssignEngineeringDescent
argo::iridium::xfloat::XFloatDiveData, 985
AssignEngineeringDiagnostic
argo::iridium::xfloat::XFloatDiveData, 986
AssignEngineeringDrift
argo::iridium::xfloat::XFloatDiveData, 986
AssignEngineeringIce
argo::iridium::xfloat::XFloatDiveData, 986
AssignEngineeringStandard
argo::iridium::xfloat::XFloatDiveData, 986
AssignEngineeringSurface
argo::iridium::xfloat::XFloatDiveData, 987
AssignFallTimeSeries
argo::iridium::xfloat::XFloatDiveData, 987
AssignGps
argo::iridium::xfloat::XFloatDiveData, 987
AssignMailMessage
argo::iridium::xfloat::XFloatDiveData, 987
AssignMessage
util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >, 130
util::xmsg::XCompositeSurfaceData, 630
util::xmsg::XMfaMissionCfg, 1399
AssignMissionCmdEchoback
argo::iridium::xfloat::XFloatDiveData, 988
AssignParentInstrument
argo::iridium::xfloat::XFloatDiveData, 988
AssignPressureMessage
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >, 108
util::xmsg::XCompositeDrift, 547
util::xmsg::XCompositeHiResProfile, 568
util::xmsg::XCompositeProfile, 606
AssignProfilePressureBinned
argo::iridium::xfloat::XFloatDiveData, 989
AssignProfilePressureUnbinned
argo::iridium::xfloat::XFloatDiveData, 989
AssignProfileSalinityBinned
argo::iridium::xfloat::XFloatDiveData, 990
AssignProfileSalinityUnbinned
argo::iridium::xfloat::XFloatDiveData, 990
AssignProfileTemperatureBinned
argo::iridium::xfloat::XFloatDiveData, 990
AssignProfileTemperatureUnbinned
argo::iridium::xfloat::XFloatDiveData, 991
AssignPumpTimeSeries
argo::iridium::xfloat::XFloatDiveData, 991
AssignRiseTimeSeries
argo::iridium::xfloat::XFloatDiveData, 992
AssignSalinityMessage
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >, 109
util::xmsg::XCompositeDrift, 547
util::xmsg::XCompositeHiResProfile, 568
util::xmsg::XCompositeProfile, 606
AssignSurfaceTimeSeries
argo::iridium::xfloat::XFloatDiveData, 992
AssignTemperatureMessage
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >, 110
util::xmsg::XCompositeDrift, 547
util::xmsg::XCompositeHiResProfile, 569
util::xmsg::XCompositeProfile, 607
AssignValveData
argo::iridium::xfloat::XFloatDiveData, 992
AssignWindowIndices
util::xmsg::XAcousticPosition, 423
ATTR_DEPRECATED
SystemUtils.h, 2207
ATTR_NO_EXCEPTIONS
MessageException.h, 2156
AuxiliarySensorData
util::xmsg::AuxiliarySensorData, 139
average_drift_pressure_counts_
util::xmsg::XEngDrift, 812
average_drift_temperature_counts_
util::xmsg::XEngDrift, 812

bat_volt_start_
util::xmsg::XAlamoBit2, 455
battery_voltage_mv_
util::xmsg::XEngSurface, 877
battery_voltage_v
util::xmsg::XEngSurface, 861
bit_passed_
util::xmsg::XAlamoBit2, 455
bit_status_code
util::xmsg::XAlamoBit, 439
bit_status_code_
util::xmsg::XAlamoBit, 443
util::xmsg::XAlamoBit2, 455
bit_version
util::xmsg::XAlamoBit, 439
util::xmsg::XAlamoBit2, 448
bit_version_
util::xmsg::XAlamoBit, 443
util::xmsg::XAlamoBit2, 455
boost, 41
boost::container, 41
boost::container::flat_map< C, D >, 251
elements, 251
index, 251
boost::container::static_vector< E >, 407
elements, 408
boost::shared_ptr< A >, 368
element, 369
boost::thread_specific_ptr< B >, 412
element, 413
btm_
util::xmsg::XAlamoMissionConfiguration, 490
util::xmsg::XMfaMissionCfg, 1405
buffer
util::thread::SynchronousQueue< T >, 412
bufferNotEmpty
util::thread::SynchronousQueue< T >, 412
bufferNotFull
util::thread::SynchronousQueue< T >, 412

CalculateEllipsoidHeight
argo::iridium::IridiumPosition, 263
argo::iridium::xfloat::XFloat, 911
argo::iridium::xfloat::XFloatMailMessage, 1059
argo::iridium::xfloat::XFloatMetadata, 1188
util::position::Position, 344
util::position::PositionUtils, 359
util::xmsg::XCompositeIridiumPosition, 585
util::xmsg::XGPS, 1264
CalculateGeographicDistance
argo::iridium::IridiumPosition, 264
argo::iridium::xfloat::XFloat, 912
argo::iridium::xfloat::XFloatMailMessage, 1060
argo::iridium::xfloat::XFloatMetadata, 1188
util::position::Position, 345
util::position::PositionUtils, 359
util::xmsg::XCompositeIridiumPosition, 585
util::xmsg::XGPS, 1264
case_vac_after_pumping_cinhg
util::xmsg::XPumpCounts, 1520
case_vac_after_pumping_cinhg_
util::xmsg::XPumpCounts, 1524
case_vac_after_pumping_inhg
util::xmsg::XPumpCounts, 1520
util::xmsg::XValveCounts, 1590
case_vac_after_valve_cinhg_
util::xmsg::XValveCounts, 1590
case_vac_at_end_cinhg_
util::xmsg::XEngAscent, 724
util::xmsg::XEngDescent, 780
util::xmsg::XEngDrift, 812
util::xmsg::XEngSurface, 878
case_vac_at_start_cinhg_
util::xmsg::XEngAscent, 724
util::xmsg::XEngDescent, 781
util::xmsg::XEngDrift, 812
util::xmsg::XEngSurface, 878
case_vac_before_pumping_cinhg
util::xmsg::XPumpCounts, 1521
case_vac_before_pumping_cinhg_
util::xmsg::XPumpCounts, 1524
case_vac_before_pumping_inhg
util::xmsg::XPumpCounts, 1521
util::xmsg::XValveCounts, 1590
case_vac_before_valve_cinhg_
util::xmsg::XValveCounts, 1590
case_vac_cingh_
util::xmsg::XEngSurface, 878
case_vac_cinhg_
util::xmsg::XEngDescent, 781
case_vac_inhg
util::xmsg::XEngDescent, 765
case_vac_start_
util::xmsg::XAlamoBit2, 455
case_vacuum_inhg
util::xmsg::XEngSurface, 861
CATCH_BLOCK
Log.h, 2140
CATCH_NOTHROW
Log.h, 2141
cbol
util::mysql::SqlTableDescription< T >, 385
cchar
util::mysql::SqlTableDescription< T >, 385
cdbl
util::mysql::SqlTableDescription< T >, 385
CFG_DEFAULT_ISDAEMON
LocalConfiguration.h, 1618
CFG_DEFAULT_MAIL_CHK_DLY
LocalConfiguration.h, 1618
CFG_DEFAULT_MAX_FETCH
LocalConfiguration.h, 1618
CFG_DEFAULT_MIN_UID
LocalConfiguration.h, 1618
CFG_DEFAULT_NDECO
LocalConfiguration.h, 1618
CFG_DEFAULT_NPROC
LocalConfiguration.h, 1618
CFG_DEFAULT_PATH
LocalConfiguration.h, 1618
CFG_DEFAULT_PATH_PID
LocalConfiguration.h, 1618
CFG_DEFAULT_REPROCESS_ALL
LocalConfiguration.h, 1619
CFG_FMT_DESC_LEN
LocalConfiguration.h, 1619
CFG_FMT_LINE_LEN
LocalConfiguration.h, 1619
CFG_MYSQL_DFLT_ADDRESS
LocalMysqlConfiguration.h, 1627
CFG_MYSQL_DFLT_PORT
LocalMysqlConfiguration.h, 1627
CFG_MYSQL_DFLT_PROTOCOL
LocalMysqlConfiguration.h, 1627
CFG_MYSQL_DFLT_SCHEMA
LocalMysqlConfiguration.h, 1627
cflt
util::mysql::SqlTableDescription< T >, 385
CheckCompleteness
argo::iridium::xfloat::XFloatDiveData, 993
CheckRuntimeLock
util::config::LocalConfiguration, 292
CheckXMsgData
util::xmsg::AbstractXFallRise, 67

util::xmsg::AbstractXSensorData, 86
util::xmsg::AuxiliarySensorData, 139
util::xmsg::XAcousticPosition, 423
util::xmsg::XAlamoEngineering, 459
util::xmsg::XArgo, 502
util::xmsg::XDriftPressure, 641
util::xmsg::XDriftSalinity, 660
util::xmsg::XDriftTemperature, 679
util::xmsg::XEngAbort, 698
util::xmsg::XEngAscent, 709
util::xmsg::XEngBIT, 728
util::xmsg::XEngBITFail, 740
util::xmsg::XEngBITPass, 752
util::xmsg::XEngDescent, 765
util::xmsg::XEngDiagnostic, 786
util::xmsg::XEngDrift, 797
util::xmsg::XEngIce, 823
util::xmsg::XEngStandard, 846
util::xmsg::XEngSurface, 861
util::xmsg::XFall, 883
util::xmsg::XGPS, 1265
util::xmsg::XHiResPressure, 1310
util::xmsg::XHiResSalinity, 1328
util::xmsg::XHiResTemperature, 1347
util::xmsg::XMessage, 1366
util::xmsg::XMessageIndexable, 1376
util::xmsg::XMissionCmdEcho, 1417
util::xmsg::XMissionConfiguration, 1427
util::xmsg::XProfilePressure, 1450
util::xmsg::XProfileSalinity, 1468
util::xmsg::XProfileTemperature, 1487
util::xmsg::XPump, 1507
util::xmsg::XRise, 1528
util::xmsg::XSurfaceData, 1556
util::xmsg::XValve, 1577
util::xmsg::XWave, 1594

ci16
util::mysql::SqlTableDescription< T >, 385

cint
util::mysql::SqlTableDescription< T >, 386

CleanupSqlConnection
argo::iridium::xfloat::XFloat, 913
argo::iridium::xfloat::XFloatDiveData, 993
argo::iridium::xfloat::XFloatMailMessage, 1061
argo::iridium::xfloat::XFloatMailProcessingService, 1139
argo::iridium::xfloat::XFloatMailRetrievalService, 1158
argo::iridium::xfloat::XFloatMetadata, 1189
argo::iridium::xfloat::XFloatTelemetryDecoderService, 1246
util::mysql::LocalMysqlConfiguration, 302
util::mysql::LocalPersistentEntity, 315
util::mysql::MysqlUtils, 336
util::mysql::SqlTableDescription< T >, 387
util::xmsg::XGPS, 1266

CleanupSqlDriver
argo::iridium::xfloat::XFloat, 913
argo::iridium::xfloat::XFloatDiveData, 994
argo::iridium::xfloat::XFloatMailMessage, 1061
argo::iridium::xfloat::XFloatMailProcessingService, 1139
argo::iridium::xfloat::XFloatMailRetrievalService, 1159
argo::iridium::xfloat::XFloatMetadata, 1190
argo::iridium::xfloat::XFloatTelemetryDecoderService, 1246
util::mysql::LocalMysqlConfiguration, 302
util::mysql::LocalPersistentEntity, 315
util::mysql::MysqlUtils, 336
util::mysql::SqlTableDescription< T >, 387
util::xmsg::XGPS, 1266

clng
util::mysql::SqlTableDescription< T >, 386

CloseKmlFactory
argo::iridium::xfloat::XFloat, 913
argo::iridium::xfloat::XFloatPostProcessingService, 1236
util::kml::KmlUtil, 284
util::xmsg::XGPS, 1267

CloseSqlConnection
argo::iridium::xfloat::XFloat, 914
argo::iridium::xfloat::XFloatDiveData, 994
argo::iridium::xfloat::XFloatMailMessage, 1061
argo::iridium::xfloat::XFloatMailProcessingService, 1139
argo::iridium::xfloat::XFloatMailRetrievalService, 1159
argo::iridium::xfloat::XFloatMetadata, 1190
argo::iridium::xfloat::XFloatTelemetryDecoderService, 1247
util::mysql::LocalMysqlConfiguration, 303
util::mysql::LocalPersistentEntity, 315
util::mysql::MysqlUtils, 337
util::mysql::SqlTableDescription< T >, 388
util::xmsg::XGPS, 1267

cms_
util::xmsg::XMfaMissionCfg, 1405

ColumDataType_t
util::mysql, 46

Column_bool
util::mysql::Column_bool< C >, 155

Column_char
util::mysql::Column_char< C >, 162

Column_double
util::mysql::Column_double< C >, 168

Column_filesystem_path
util::mysql::Column_filesystem_path< C >, 175

Column_float
util::mysql::Column_float< C >, 182

column_index
util::mysql::Column_bool< C >, 156
util::mysql::Column_char< C >, 163
util::mysql::Column_double< C >, 169
util::mysql::Column_filesystem_path< C >, 176
util::mysql::Column_float< C >, 183
util::mysql::Column_int< C >, 189
util::mysql::Column_int16_t< C >, 196
util::mysql::Column_long< C >, 202
util::mysql::Column_posix_time< C >, 209
util::mysql::Column_string< C >, 216
util::mysql::Column_uint16_t< C >, 223
util::mysql::Column_uint8_t< C >, 229
util::mysql::Column_vector_uint8_t< C >, 236
util::mysql::SqlTableColumnDescription< C, T >, 374
util::mysql::SqlTableColumnDescriptionBase, 380

column_index_
util::mysql::Column_bool< C >, 160
util::mysql::Column_char< C >, 166
util::mysql::Column_double< C >, 173
util::mysql::Column_filesystem_path< C >, 180
util::mysql::Column_float< C >, 186
util::mysql::Column_int< C >, 193
util::mysql::Column_int16_t< C >, 199
util::mysql::Column_long< C >, 206
util::mysql::Column_posix_time< C >, 213
util::mysql::Column_string< C >, 220
util::mysql::Column_uint16_t< C >, 226
util::mysql::Column_uint8_t< C >, 233
util::mysql::Column_vector_uint8_t< C >, 240
util::mysql::SqlTableColumnDescription< C, T >, 377
util::mysql::SqlTableColumnDescriptionBase, 382

Column_int
util::mysql::Column_int< C >, 188

Column_int16_t
util::mysql::Column_int16_t< C >, 195
Column_long
util::mysql::Column_long< C >, 201
column_name
util::mysql::Column_bool< C >, 157
util::mysql::Column_char< C >, 163
util::mysql::Column_double< C >, 170
util::mysql::Column_filesystem_path< C >, 177
util::mysql::Column_float< C >, 183
util::mysql::Column_int< C >, 190
util::mysql::Column_int16_t< C >, 196
util::mysql::Column_long< C >, 203
util::mysql::Column_posix_time< C >, 210
util::mysql::Column_string< C >, 217
util::mysql::Column_uint16_t< C >, 223
util::mysql::Column_uint8_t< C >, 230
util::mysql::Column_vector_uint8_t< C >, 236
util::mysql::SqlTableColumnDescription< C, T >, 375
util::mysql::SqlTableColumnDescriptionBase, 380
column_name_
util::mysql::Column_bool< C >, 160
util::mysql::Column_char< C >, 166
util::mysql::Column_double< C >, 173
util::mysql::Column_filesystem_path< C >, 180
util::mysql::Column_float< C >, 186
util::mysql::Column_int< C >, 193
util::mysql::Column_int16_t< C >, 199
util::mysql::Column_long< C >, 206
util::mysql::Column_posix_time< C >, 213
util::mysql::Column_string< C >, 220
util::mysql::Column_uint16_t< C >, 226
util::mysql::Column_uint8_t< C >, 233
util::mysql::Column_vector_uint8_t< C >, 240
util::mysql::SqlTableColumnDescription< C, T >, 377
util::mysql::SqlTableColumnDescriptionBase, 382
Column_posix_time
util::mysql::Column_posix_time< C >, 208
column_select_name
util::mysql::Column_bool< C >, 157
util::mysql::Column_char< C >, 164
util::mysql::Column_double< C >, 170
util::mysql::Column_filesystem_path< C >, 177
util::mysql::Column_float< C >, 184
util::mysql::Column_int< C >, 190
util::mysql::Column_int16_t< C >, 197
util::mysql::Column_long< C >, 203
util::mysql::Column_posix_time< C >, 210
util::mysql::Column_string< C >, 217
util::mysql::Column_uint16_t< C >, 224
util::mysql::Column_uint8_t< C >, 230
util::mysql::Column_vector_uint8_t< C >, 237
util::mysql::SqlTableColumnDescription< C, T >, 376
util::mysql::SqlTableColumnDescriptionBase, 381
Column_string
util::mysql::Column_string< C >, 215
Column_uint16_t
util::mysql::Column_uint16_t< C >, 222
Column_uint8_t
util::mysql::Column_uint8_t< C >, 228
Column_vector_uint8_t
util::mysql::Column_vector_uint8_t< C >, 235
comment
argo::iridium::xfloat::XFloatMetadata, 1190
comment_
argo::iridium::xfloat::XFloatMetadata, 1227
complete_
argo::iridium::xfloat::XFloatDiveData, 1037
ComputeAverageIrdiumPosition
util::xmsg::XCompositeIridiumPosition, 586
ComputePressureOffsetFromSurfaceTimeSeries
argo::iridium::xfloat::XFloatDiveData, 994
config/LocalConfiguration.cpp, 1615
config/LocalConfiguration.h, 1616, 1620
config/LocalMysqlConfiguration.cpp, 1625
config/LocalMysqlConfiguration.h, 1626, 1627
config/LocalPersistentEntity.h, 1628, 1630
configuration
util::config::LocalConfiguration, 292
configuration_
util::config::LocalConfiguration, 298
configuration_file_path_
util::config::LocalConfiguration, 298
ConnectToMailServer
argo::iridium::xfloat::XFloatMailRetrievalService, 1159
cpb_
util::xmsg::XAlamoMissionConfiguration, 490
util::xmsg::XMfaMissionCfg, 1405
cpn_
util::xmsg::XAlamoMissionConfiguration, 490
util::xmsg::XMfaMissionCfg, 1405
cpp_
util::xmsg::XAlamoMissionConfiguration, 490
util::xmsg::XMfaMissionCfg, 1405
cps_
util::xmsg::XAlamoMissionConfiguration, 490
util::xmsg::XMfaMissionCfg, 1405
cpth
util::mysql::SqlTableDescription< T >, 386
cptm
util::mysql::SqlTableDescription< T >, 386
crd_
util::xmsg::XMfaMissionCfg, 1405
crs_
util::xmsg::XMfaMissionCfg, 1405
csp_
util::xmsg::XAlamoMissionConfiguration, 490
util::xmsg::XMfaMissionCfg, 1406
cst_
util::xmsg::XAlamoMissionConfiguration, 490
util::xmsg::XMfaMissionCfg, 1406
cstr
util::mysql::SqlTableDescription< T >, 386
ctc_
util::xmsg::XMfaMissionCfg, 1406
ctd_status_code_
util::xmsg::XEngSurface, 878
ctf_
util::xmsg::XAlamoMissionConfiguration, 490
util::xmsg::XMfaMissionCfg, 1406
ctl_
util::xmsg::XAlamoMissionConfiguration, 491
util::xmsg::XMfaMissionCfg, 1406
ctu_
util::xmsg::XAlamoMissionConfiguration, 491
util::xmsg::XMfaMissionCfg, 1406
cu16
util::mysql::SqlTableDescription< T >, 386
cu8
util::mysql::SqlTableDescription< T >, 386
cvu8
util::mysql::SqlTableDescription< T >, 386

data/Iridium.h, 1630, 1632
data/IridiumDefinitions.h, 1634, 1636
data/IridiumPosition.h, 1637, 1638
data/XFloat.cpp, 1640
data/XFloat.h, 1641, 1643
data/XFloatDefinitions.h, 1658, 1662
data/XFloatDiveData.cpp, 1663
data/XFloatDiveData.h, 1665, 1666
data/XFloatMailMessage.cpp, 1684
data/XFloatMailMessage.h, 1685, 1689

data/XFloatMetadata.cpp, 1706
data/XFloatMetadata.h, 1707, 1709
data/xmsg/AbstractXComposite.h, 1717, 1718
data/xmsg/AbstractXFallRise.h, 1719, 1721
data/xmsg/AbstractXSensorData.h, 1724, 1726
data/xmsg/AbstractXSensorDataComposite.h, 1733, 1734
data/xmsg/AbstractXTimeSeriesDataComposite.h, 1745, 1747
data/xmsg/AuxiliarySensorData.cpp, 1757
data/xmsg/AuxiliarySensorData.h, 1759, 1760
data/xmsg/common/XCommon.h, 1765, 1766
data/xmsg/common/XDefinitions.h, 1766, 1779
data/xmsg/common/XPresCodeCounts.cpp, 1783
data/xmsg/common/XPresCodeCounts.h, 1784, 1786
data/xmsg/common/XPresCounts.h, 1788, 1789
data/xmsg/common/XPumpCounts.h, 1790, 1792
data/xmsg/common/XSensorTripletCodeCounts.h, 1794, 1795
data/xmsg/common/XValveCounts.cpp, 1797
data/xmsg/common/XValveCounts.h, 1798, 1799
data/xmsg/common/XWaveRawSample.h, 1801, 1802
data/xmsg/common/XWaveSpectralSample.cpp, 1803
data/xmsg/common/XWaveSpectralSample.h, 1805, 1806
data/xmsg/XAcousticPosition.h, 1808, 1809
data/xmsg/XAlamoBit.h, 1816, 1817
data/xmsg/XAlamoBit2.h, 1828
data/xmsg/XAlamoEngineering.h, 1841, 1843
data/xmsg/XAlamoMissionConfiguration.h, 1844, 1845
data/xmsg/XArgo.h, 1901, 1904
data/xmsg/XCompositeAuxTemp.h, 1915, 1917
data/xmsg/XCompositeDrift.h, 1918, 1919
data/xmsg/XCompositeFall.h, 1921, 1922
data/xmsg/XCompositeHiResProfile.h, 1923, 1924
data/xmsg/XCompositeIridiumPosition.h, 1926, 1929
data/xmsg/XCompositeProfile.h, 1931, 1932
data/xmsg/XCompositePump.h, 1934, 1936
data/xmsg/XCompositeRise.h, 1937, 1938
data/xmsg/XCompositeSurfaceData.cpp, 1939
data/xmsg/XCompositeSurfaceData.h, 1940, 1941
data/xmsg/XCompositeValve.cpp, 1946
data/xmsg/XCompositeValve.h, 1947, 1948
data/xmsg/XDriftPressure.h, 1949, 1951
data/xmsg/XDriftSalinity.h, 1951, 1953
data/xmsg/XDriftTemperature.h, 1953, 1955
data/xmsg/XEngAbort.h, 1955, 1957
data/xmsg/XEngAscent.h, 1958, 1959
data/xmsg/XEngBIT.h, 1963
data/xmsg/XEngBITFail.h, 1964, 1966
data/xmsg/XEngBITPass.h, 1967, 1968
data/xmsg/XEngDescent.h, 1969, 1970
data/xmsg/XEngDiagnostic.h, 1974, 1976
data/xmsg/XEngDrift.h, 1977, 1978
data/xmsg/XEngIce.cpp, 1983
data/xmsg/XEngIce.h, 1984, 1985
data/xmsg/XEngStandard.h, 1991, 1993
data/xmsg/XEngSurface.h, 1994, 1995
data/xmsg/XFall.h, 2012, 2014
data/xmsg/XGPS.cpp, 2015
data/xmsg/XGPS.h, 2017, 2018
data/xmsg/XHiResPressure.h, 2023, 2025
data/xmsg/XHiResSalinity.h, 2025, 2027
data/xmsg/XHiResTemperature.h, 2027, 2029
data/xmsg/XMessage.h, 2029, 2033
data/xmsg/XMessageIndexable.h, 2038, 2039
data/xmsg/XMfaMissionCfg.cpp, 2040
data/xmsg/XMfaMissionCfg.h, 2042, 2043
data/xmsg/XMissionCmdEcho.h, 2054, 2056
data/xmsg/XMissionConfiguration.h, 2057, 2058
data/xmsg/XProfilePressure.h, 2059, 2060
data/xmsg/XProfileSalinity.h, 2061, 2063
data/xmsg/XProfileTemperature.h, 2063, 2065
data/xmsg/XPump.h, 2065, 2067
data/xmsg/XRise.h, 2070, 2071
data/xmsg/XSurfaceData.h, 2072, 2074
data/xmsg/XValve.cpp, 2075
data/xmsg/XValve.h, 2077, 2078
data/xmsg/XWave.cpp, 2080
data/xmsg/XWave.h, 2081, 2084
data_pack_format
util::xmsg::AbstractXFallRise, 68
util::xmsg::XFall, 884
util::xmsg::XRise, 1528
util::xmsg::XSurfaceData, 1556
data_pack_format_
util::xmsg::AbstractXFallRise, 82
util::xmsg::XFall, 900
util::xmsg::XRise, 1544
util::xmsg::XSurfaceData, 1572
data_type
util::mysql::Column_bool< C >, 158
util::mysql::Column_char< C >, 164
util::mysql::Column_double< C >, 171
util::mysql::Column_filesystem_path< C >, 178
util::mysql::Column_float< C >, 184
util::mysql::Column_int< C >, 191
util::mysql::Column_int16_t< C >, 197
util::mysql::Column_long< C >, 204
util::mysql::Column_posix_time< C >, 211
util::mysql::Column_string< C >, 218
util::mysql::Column_uint16_t< C >, 224
util::mysql::Column_uint8_t< C >, 231
util::mysql::Column_vector_uint8_t< C >, 237
util::mysql::SqlTableColumnDescription< C, T >, 376
util::mysql::SqlTableColumnDescriptionBase, 381
data_type_
util::mysql::Column_bool< C >, 160
util::mysql::Column_char< C >, 167
util::mysql::Column_double< C >, 173
util::mysql::Column_filesystem_path< C >, 180
util::mysql::Column_float< C >, 187
util::mysql::Column_int< C >, 193
util::mysql::Column_int16_t< C >, 200
util::mysql::Column_long< C >, 206
util::mysql::Column_posix_time< C >, 213
util::mysql::Column_string< C >, 220
util::mysql::Column_uint16_t< C >, 227
util::mysql::Column_uint8_t< C >, 233
util::mysql::Column_vector_uint8_t< C >, 240
util::mysql::SqlTableColumnDescription< C, T >, 377
util::mysql::SqlTableColumnDescriptionBase, 382
DataBuf
util::stream::DataBuf< CharT, TraitsT >, 242
date_time
util::xmsg::XEngIce::XEngIceErrorCode, 839
util::xmsg::XEngSurface::XEngErrorCode, 816
date_time_
util::xmsg::XEngIce::XEngIceErrorCode, 844
util::xmsg::XEngSurface::XEngErrorCode, 820
DateTimeAs8601String
util::xmsg::XEngIce::XEngIceErrorCode, 839
util::xmsg::XEngSurface::XEngErrorCode, 816
dbg1
Log.h, 2142
dbg2
Log.h, 2142
dbg3
Log.h, 2142
DBG_X_DECODE
stage3_TelemetryDecoderThread.h, 2106
ddb_
util::xmsg::XMfaMissionCfg, 1407
ddd_
util::xmsg::XAlamoMissionConfiguration, 491
util::xmsg::XMfaMissionCfg, 1407

ddt_
util::xmsg::XAlamoMissionConfiguration, 491
util::xmsg::XMfaMissionCfg, 1407
DecodeMailMessageData
argo::iridium::xfloat::XFloatDiveData, 995
DecodeMessage
argo::iridium::xfloat::XFloatTelemetryDecoderService, 1247
DecodeXMsgData
argo::iridium::xfloat::XFloatDiveData, 996
util::xmsg::AbstractXFallRise, 68
util::xmsg::AbstractXSensorData, 87
util::xmsg::AuxiliarySensorData, 139
util::xmsg::XAcousticCacst, 419
util::xmsg::XAcousticPosition, 424
util::xmsg::XAcousticWindow, 434
util::xmsg::XAlamoBit, 439
util::xmsg::XAlamoBit2, 448
util::xmsg::XAlamoEngineering, 460
util::xmsg::XAlamoMissionConfiguration, 485
util::xmsg::XArgo, 503
util::xmsg::XDriftPressure, 642
util::xmsg::XDriftSalinity, 661
util::xmsg::XDriftTemperature, 680
util::xmsg::XEngAbort, 698
util::xmsg::XEngAscent, 710
util::xmsg::XEngBIT, 728
util::xmsg::XEngBITFail, 740
util::xmsg::XEngBITPass, 752
util::xmsg::XEngDescent, 766
util::xmsg::XEngDiagnostic, 786
util::xmsg::XEngDrift, 798
util::xmsg::XEngIce, 824
util::xmsg::XEngStandard, 847, 848
util::xmsg::XEngSurface, 862
util::xmsg::XFall, 884
util::xmsg::XGPS, 1267
util::xmsg::XHiResPressure, 1310
util::xmsg::XHiResSalinity, 1329
util::xmsg::XHiResTemperature, 1348
util::xmsg::XMessage, 1367
util::xmsg::XMessageIndexable, 1377
util::xmsg::XMfaMissionCfg, 1400
util::xmsg::XMissionCmdEcho, 1417
util::xmsg::XMissionConfiguration, 1428
util::xmsg::XProfilePressure, 1450
util::xmsg::XProfileSalinity, 1469
util::xmsg::XProfileTemperature, 1488
util::xmsg::XPump, 1507
util::xmsg::XRise, 1529
util::xmsg::XSurfaceData, 1557
util::xmsg::XValve, 1577
util::xmsg::XWave, 1594
DEFAULT_LOG_LEVEL
Log.h, 2142
degToRad
argo::iridium::IridiumPosition, 265
argo::iridium::xfloat::XFloat, 914
argo::iridium::xfloat::XFloatMailMessage, 1063
argo::iridium::xfloat::XFloatMetadata, 1191
util::math::MathUtils, 332
util::position::Position, 346
util::position::PositionUtils, 360
util::xmsg::XCompositeIridiumPosition, 587
util::xmsg::XGPS, 1268
DeleteFromDatabase
argo::iridium::xfloat::XFloatDiveData, 996
util::xmsg::XGPS, 1269
DeleteObjectsFromDatabase
argo::iridium::xfloat::XFloatDiveData, 997
util::xmsg::XGPS, 1269
deployer
argo::iridium::xfloat::XFloatMetadata, 1191
deployer_
argo::iridium::xfloat::XFloatMetadata, 1227
deployer_address
argo::iridium::xfloat::XFloatMetadata, 1191
deployer_address_
argo::iridium::xfloat::XFloatMetadata, 1228
deployment_cruise_id
argo::iridium::xfloat::XFloatMetadata, 1192
deployment_cruiseId_
argo::iridium::xfloat::XFloatMetadata, 1228
deployment_datetime
argo::iridium::xfloat::XFloatMetadata, 1192
deployment_datetime_status
argo::iridium::xfloat::XFloatMetadata, 1192
deployment_datetime_status_
argo::iridium::xfloat::XFloatMetadata, 1228
deployment_latitude
argo::iridium::xfloat::XFloatMetadata, 1192
deployment_longitude
argo::iridium::xfloat::XFloatMetadata, 1193
deployment_platform
argo::iridium::xfloat::XFloatMetadata, 1193
deployment_platform_
argo::iridium::xfloat::XFloatMetadata, 1228
deployment_position_status
argo::iridium::xfloat::XFloatMetadata, 1193
deployment_position_status_
argo::iridium::xfloat::XFloatMetadata, 1228
deployment_type
argo::iridium::xfloat::XFloatMetadata, 1194
deployment_type_
argo::iridium::xfloat::XFloatMetadata, 1228
Deprecated List, 17
descend_start_time_
util::xmsg::XAlamoMissionConfiguration, 491
util::xmsg::XMfaMissionCfg, 1407
descent_state_start_time
util::xmsg::XArgo, 504
descent_state_start_time_
util::xmsg::XArgo, 533
dft_
util::xmsg::XAlamoMissionConfiguration, 491
util::xmsg::XMfaMissionCfg, 1407
DisconnectFromMailServer
argo::iridium::xfloat::XFloatMailRetrievalService, 1160
dive2data_map_
argo::iridium::xfloat::XFloat, 960
dns_
util::xmsg::XAlamoMissionConfiguration, 492
util::xmsg::XMfaMissionCfg, 1407
drift_state_start_time
util::xmsg::XArgo, 504
drift_state_start_time_
util::xmsg::XArgo, 534
dsd_
util::xmsg::XAlamoMissionConfiguration, 492
util::xmsg::XMfaMissionCfg, 1407
dsi_
util::xmsg::XAlamoMissionConfiguration, 492
util::xmsg::XMfaMissionCfg, 1407
dst_
util::xmsg::XAlamoMissionConfiguration, 492
util::xmsg::XMfaMissionCfg, 1408
dt_end_
util::xmsg::XAlamoBit2, 455
dt_end_epoch_
util::xmsg::XAlamoBit2, 455
dt_start_
util::xmsg::XAlamoBit2, 455
dt_start_epoch_

util::xmsg::XAlamoBit2, 455
dtd_
util::xmsg::XAlamoMissionConfiguration, 492
util::xmsg::XMfaMissionCfg, 1408
duration_sec_
util::xmsg::XAlamoBit2, 456

EARTH_RADIUS_KM_EQUATORIAL
argo::iridium::IridiumPosition, 281
argo::iridium::xfloat::XFloat, 960
argo::iridium::xfloat::XFloatMailMessage, 1125
argo::iridium::xfloat::XFloatMetadata, 1228
util::position::Position, 357
util::position::PositionUtils, 364
util::xmsg::XCompositeIridiumPosition, 601
util::xmsg::XGPS, 1302
EARTH_RADIUS_KM_POLAR
argo::iridium::IridiumPosition, 281
argo::iridium::xfloat::XFloat, 961
argo::iridium::xfloat::XFloatMailMessage, 1125
argo::iridium::xfloat::XFloatMetadata, 1229
util::position::Position, 357
util::position::PositionUtils, 364
util::xmsg::XCompositeIridiumPosition, 601
util::xmsg::XGPS, 1302
elapsed_seconds_since_reference_time
util::xmsg::XPresCodeCounts, 1440
util::xmsg::XPresCounts, 1445
util::xmsg::XSensorTripletCodeCounts, 1548
elapsed_seconds_since_reference_time_
util::xmsg::XPresCodeCounts, 1443
util::xmsg::XPresCounts, 1447
util::xmsg::XSensorTripletCodeCounts, 1552
element
boost::shared_ptr< A >, 369
boost::thread_specific_ptr< B >, 413
vmime::shared_ptr< F >, 370
Element_t
util::xmsg::AbstractXComposite< T >, 62
util::xmsg::XCompositeAuxTemp, 540
util::xmsg::XCompositeFall, 561
util::xmsg::XCompositeIridiumPosition, 584
util::xmsg::XCompositePump, 621
util::xmsg::XCompositeRise, 624
util::xmsg::XCompositeValve, 637
util::xmsg::XPump, 1505
util::xmsg::XValve, 1575
elements
boost::container::flat_map< C, D >, 251
boost::container::static_vector< E >, 408
util::xmsg::AbstractXComposite< T >, 64
util::xmsg::XCompositeAuxTemp, 541
util::xmsg::XCompositeFall, 562
util::xmsg::XCompositeIridiumPosition, 587
util::xmsg::XCompositePump, 622
util::xmsg::XCompositeRise, 625
util::xmsg::XCompositeValve, 638
util::xmsg::XPump, 1508
util::xmsg::XValve, 1578
elements_
util::xmsg::AbstractXComposite< T >, 64
util::xmsg::XCompositeAuxTemp, 542
util::xmsg::XCompositeFall, 564
util::xmsg::XCompositeIridiumPosition, 601
util::xmsg::XCompositePump, 623
util::xmsg::XCompositeRise, 627
util::xmsg::XCompositeValve, 639
util::xmsg::XPump, 1518
util::xmsg::XValve, 1587
empty
util::thread::SynchronousQueue< T >, 410
engineering_version
util::xmsg::XAlamoEngineering, 460
util::xmsg::XEngAscent, 710
util::xmsg::XEngDescent, 767
util::xmsg::XEngDrift, 799
util::xmsg::XEngIce, 825
util::xmsg::XEngSurface, 863
engineering_version_
util::xmsg::XAlamoEngineering, 473
util::xmsg::XEngAscent, 724
util::xmsg::XEngDescent, 781
util::xmsg::XEngDrift, 813
util::xmsg::XEngIce, 837
util::xmsg::XEngSurface, 878
engineering_version_as_float
util::xmsg::XAlamoEngineering, 461
util::xmsg::XEngAscent, 711
util::xmsg::XEngDescent, 767
util::xmsg::XEngDrift, 799
util::xmsg::XEngIce, 825
util::xmsg::XEngSurface, 863
error_code
util::xmsg::XEngIce::XEngIceErrorCode, 840
util::xmsg::XEngSurface::XEngErrorCode, 816
error_code_
util::xmsg::XEngIce::XEngIceErrorCode, 844
util::xmsg::XEngSurface::XEngErrorCode, 820
error_codes_
util::xmsg::XEngIce, 837
util::xmsg::XEngSurface, 878
exc_catchAny
Log.h, 2142
exc_catchAny_noThrow
Log.h, 2142
exc_catchStandard
Log.h, 2143
exc_catchStandard_noThrow
Log.h, 2143
exc_diag
Log.h, 2143
EXC_THROW
Log.h, 2143
executeAndPipe
util::thread::ThreadUtils, 414
ExecuteQuery
argo::iridium::xfloat::XFloat, 914
argo::iridium::xfloat::XFloatDiveData, 997
argo::iridium::xfloat::XFloatMailMessage, 1063
argo::iridium::xfloat::XFloatMailProcessingService, 1140
argo::iridium::xfloat::XFloatMailRetrievalService, 1160
argo::iridium::xfloat::XFloatMetadata, 1194
argo::iridium::xfloat::XFloatTelemetryDecoderService, 1248
util::mysql::LocalMysqlConfiguration, 303
util::mysql::LocalPersistentEntity, 316
util::mysql::MysqlUtils, 337
util::mysql::SqlTableDescription< T >, 388
util::xmsg::XGPS, 1270
executeQuery
argo::iridium::xfloat::XFloat, 915
argo::iridium::xfloat::XFloatDiveData, 998
argo::iridium::xfloat::XFloatMailMessage, 1064
argo::iridium::xfloat::XFloatMailProcessingService, 1140
argo::iridium::xfloat::XFloatMailRetrievalService, 1161
argo::iridium::xfloat::XFloatMetadata, 1194
util::mysql::LocalMysqlConfiguration, 304
util::mysql::LocalPersistentEntity, 316
util::mysql::SqlTableDescription< T >, 389
util::xmsg::XGPS, 1270

executeSelectQuery
argo::iridium::xfloat::XFloat, 916
argo::iridium::xfloat::XFloatDiveData, 999
argo::iridium::xfloat::XFloatMailMessage, 1065
argo::iridium::xfloat::XFloatMailProcessingService, 1141
argo::iridium::xfloat::XFloatMailRetrievalService, 1162
argo::iridium::xfloat::XFloatMetadata, 1195
util::mysql::LocalMysqlConfiguration, 304
util::mysql::LocalPersistentEntity, 317
util::mysql::SqlTableDescription< T >, 390
util::xmsg::XGPS, 1271
ExecuteUpdate
argo::iridium::xfloat::XFloat, 917
argo::iridium::xfloat::XFloatDiveData, 1000
argo::iridium::xfloat::XFloatMailMessage, 1066
argo::iridium::xfloat::XFloatMailProcessingService, 1142
argo::iridium::xfloat::XFloatMailRetrievalService, 1163
argo::iridium::xfloat::XFloatMetadata, 1196
argo::iridium::xfloat::XFloatTelemetryDecoderService, 1249
util::mysql::LocalMysqlConfiguration, 305
util::mysql::LocalPersistentEntity, 318
util::mysql::MysqlUtils, 338
util::mysql::SqlTableDescription< T >, 391
util::xmsg::XGPS, 1272
executeUpdate
argo::iridium::xfloat::XFloat, 917
argo::iridium::xfloat::XFloatDiveData, 1000
argo::iridium::xfloat::XFloatMailMessage, 1066
argo::iridium::xfloat::XFloatMailProcessingService, 1143
argo::iridium::xfloat::XFloatMailRetrievalService, 1163
argo::iridium::xfloat::XFloatMetadata, 1196
util::mysql::LocalMysqlConfiguration, 306
util::mysql::LocalPersistentEntity, 319
util::mysql::SqlTableDescription< T >, 391
util::xmsg::XGPS, 1272
ExitHandler
util::sig::SignalHandler, 371

FetchMailMessagesFromFileSystem
argo::iridium::xfloat::XFloatMailRetrievalService, 1164
FetchMailMessagesFromServer
argo::iridium::xfloat::XFloatMailRetrievalService, 1164
first_pump_duration_adjustment_s_
util::xmsg::XEngDescent, 781
FixFuckup9000
util::xmsg::XWave, 1595
float_software_version
util::xmsg::XAlamoBit, 440
util::xmsg::XAlamoBit2, 449
float_software_version_
util::xmsg::XAlamoBit, 443
util::xmsg::XAlamoBit2, 456
Flush
util::log::Log, 329
FlushGlobalLogger
util::log::Log, 330
FSAM
util::xmsg::XWave, 1605
full
util::thread::SynchronousQueue< T >, 410
fw_version
argo::iridium::xfloat::XFloat, 918
fw_version_
argo::iridium::xfloat::XFloat, 961
util::xmsg::XAlamoMissionConfiguration, 492
util::xmsg::XMfaMissionCfg, 1408

gain_accel_q3_
util::xmsg::XWave, 1605
gain_pressure
argo::iridium::xfloat::XFloat, 918
util::xmsg::XArgo, 504
gain_pressure_
argo::iridium::xfloat::XFloat, 961
util::xmsg::XArgo, 534
gain_pressure_surface
argo::iridium::xfloat::XFloat, 918
util::xmsg::XArgo, 505
gain_pressure_surface_
argo::iridium::xfloat::XFloat, 961
util::xmsg::XArgo, 534
gain_salinity
argo::iridium::xfloat::XFloat, 919
util::xmsg::XArgo, 505
gain_salinity_
argo::iridium::xfloat::XFloat, 961
util::xmsg::XArgo, 534
gain_salinity_surface
argo::iridium::xfloat::XFloat, 919
util::xmsg::XArgo, 505
gain_salinity_surface_
argo::iridium::xfloat::XFloat, 961
util::xmsg::XArgo, 534
gain_temperature
argo::iridium::xfloat::XFloat, 919
util::xmsg::XArgo, 506
gain_temperature_
argo::iridium::xfloat::XFloat, 962
util::xmsg::XArgo, 535
gain_temperature_surface
argo::iridium::xfloat::XFloat, 920
util::xmsg::XArgo, 506
gain_temperature_surface_
argo::iridium::xfloat::XFloat, 962
util::xmsg::XArgo, 535
GenerateCommandLineOptionsDescription
util::config::LocalConfiguration, 292
GenerateCompositeFallTimeSeriesData
argo::iridium::xfloat::XFloatDiveData, 1001
GenerateCompositeIridiumPosition
argo::iridium::xfloat::XFloatDiveData, 1001
GenerateCompositePumpTimeSeriesData
argo::iridium::xfloat::XFloatDiveData, 1002
GenerateCompositeRiseTimeSeriesData
argo::iridium::xfloat::XFloatDiveData, 1002
GenerateConfigurationFileOptionsDescription
util::config::LocalConfiguration, 293
GenerateCurrentUtcTimeIso8601String
argo::iridium::IridiumPosition, 265
argo::iridium::xfloat::XFloat, 920
argo::iridium::xfloat::XFloatDiveData, 1003
argo::iridium::xfloat::XFloatMailMessage, 1067
argo::iridium::xfloat::XFloatMetadata, 1197
util::date::DateUtils, 244
util::position::Position, 346
util::position::PositionUtils, 361
util::xmsg::AbstractXFallRise, 68
util::xmsg::XAlamoBit2, 449
util::xmsg::XAlamoEngineering, 461
util::xmsg::XAlamoMissionConfiguration, 485
util::xmsg::XArgo, 506
util::xmsg::XCompositeIridiumPosition, 588
util::xmsg::XEngAscent, 711
util::xmsg::XEngDescent, 768
util::xmsg::XEngDrift, 799

util::xmsg::XEngIce, 826
util::xmsg::XEngIce::XEngIceErrorCode, 840
util::xmsg::XEngSurface, 863
util::xmsg::XEngSurface::XEngErrorCode, 817
util::xmsg::XFall, 885
util::xmsg::XGPS, 1273
util::xmsg::XMfaMissionCfg, 1400
util::xmsg::XRise, 1529
util::xmsg::XSurfaceData, 1557
generateInsertStatment
util::mysql::SqlTableDescription< T >, 392
GenerateIso6709String
argo::iridium::IridiumPosition, 266
argo::iridium::xfloat::XFloat, 920
argo::iridium::xfloat::XFloatMailMessage, 1067
argo::iridium::xfloat::XFloatMetadata, 1197
util::position::Position, 347
util::position::PositionUtils, 361
util::xmsg::XCompositeIridiumPosition, 588
util::xmsg::XGPS, 1273
GenerateIso8601String
argo::iridium::IridiumPosition, 266
argo::iridium::xfloat::XFloat, 921
argo::iridium::xfloat::XFloatDiveData, 1003
argo::iridium::xfloat::XFloatMailMessage, 1068
argo::iridium::xfloat::XFloatMetadata, 1198
util::date::DateUtils, 244
util::position::Position, 347
util::position::PositionUtils, 362
util::xmsg::AbstractXFallRise, 69
util::xmsg::XAlamoBit2, 449
util::xmsg::XAlamoEngineering, 462
util::xmsg::XAlamoMissionConfiguration, 486
util::xmsg::XArgo, 507
util::xmsg::XCompositeIridiumPosition, 589
util::xmsg::XEngAscent, 712
util::xmsg::XEngDescent, 768
util::xmsg::XEngDrift, 800
util::xmsg::XEngIce, 826
util::xmsg::XEngIce::XEngIceErrorCode, 841
util::xmsg::XEngSurface, 864
util::xmsg::XEngSurface::XEngErrorCode, 817
util::xmsg::XFall, 885
util::xmsg::XGPS, 1274
util::xmsg::XMfaMissionCfg, 1401
util::xmsg::XRise, 1530
util::xmsg::XSurfaceData, 1558
GenerateJulianDay
argo::iridium::IridiumPosition, 267
argo::iridium::xfloat::XFloat, 922
argo::iridium::xfloat::XFloatDiveData, 1004
argo::iridium::xfloat::XFloatMailMessage, 1069
argo::iridium::xfloat::XFloatMetadata, 1199
util::date::DateUtils, 245
util::position::Position, 348
util::position::PositionUtils, 363
util::xmsg::AbstractXFallRise, 70
util::xmsg::XAlamoBit2, 450
util::xmsg::XAlamoEngineering, 463
util::xmsg::XAlamoMissionConfiguration, 487
util::xmsg::XArgo, 508
util::xmsg::XCompositeIridiumPosition, 590
util::xmsg::XEngAscent, 713
util::xmsg::XEngDescent, 769
util::xmsg::XEngDrift, 801
util::xmsg::XEngIce, 827
util::xmsg::XEngIce::XEngIceErrorCode, 842
util::xmsg::XEngSurface, 865
util::xmsg::XEngSurface::XEngErrorCode, 818
util::xmsg::XFall, 886
util::xmsg::XGPS, 1275
util::xmsg::XMfaMissionCfg, 1402
util::xmsg::XRise, 1531
util::xmsg::XSurfaceData, 1559
GenerateKmlMasterDocument
argo::iridium::xfloat::XFloat, 922
GenerateKmlMasterString
argo::iridium::xfloat::XFloat, 923
GenerateKmlString
argo::iridium::xfloat::XFloat, 923
GenerateListingComment
argo::iridium::xfloat::XFloatMailMessage, 1069
util::xmsg::AbstractXFallRise, 71
util::xmsg::XCompositeIridiumPosition, 590
util::xmsg::XFall, 887
util::xmsg::XGPS, 1275
util::xmsg::XPump, 1508
util::xmsg::XRise, 1532
util::xmsg::XSurfaceData, 1560
util::xmsg::XValve, 1578
GenerateListingString
argo::iridium::xfloat::XFloatDiveData, 1005
argo::iridium::xfloat::XFloatMailMessage, 1070
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >, 111
util::xmsg::XAcousticPosition, 424
util::xmsg::XAlamoBit, 440
util::xmsg::XAlamoBit2, 451
util::xmsg::XAlamoMissionConfiguration, 488
util::xmsg::XArgo, 509
util::xmsg::XCompositeAuxTemp, 541
util::xmsg::XCompositeDrift, 548
util::xmsg::XCompositeFall, 563
util::xmsg::XCompositeHiResProfile, 569
util::xmsg::XCompositeIridiumPosition, 591
util::xmsg::XCompositeProfile, 607
util::xmsg::XCompositePump, 622
util::xmsg::XCompositeRise, 626
util::xmsg::XCompositeSurfaceData, 630
util::xmsg::XEngAbort, 699
util::xmsg::XEngAscent, 713
util::xmsg::XEngBIT, 729
util::xmsg::XEngBITFail, 741
util::xmsg::XEngBITPass, 753
util::xmsg::XEngDescent, 770
util::xmsg::XEngDiagnostic, 787
util::xmsg::XEngDrift, 802
util::xmsg::XEngStandard, 848
util::xmsg::XEngSurface, 866
util::xmsg::XFall, 887
util::xmsg::XGPS, 1276
util::xmsg::XMfaMissionCfg, 1402
util::xmsg::XMissionCmdEcho, 1418
util::xmsg::XMissionConfiguration, 1428
util::xmsg::XPresCodeCounts, 1440
util::xmsg::XPresCounts, 1445
util::xmsg::XPump, 1509
util::xmsg::XPumpCounts, 1522
util::xmsg::XRise, 1532
util::xmsg::XSensorTripletCodeCounts, 1548
util::xmsg::XSurfaceData, 1560
util::xmsg::XWave, 1595
util::xmsg::XWaveRawSample, 1608
GenerateMailServerUrlString
argo::iridium::xfloat::XFloatMailRetrievalService, 1165
GenerateObjectList
util::mysql::SqlTableDescription< T >, 392
GenerateOrderedMomsnList
argo::iridium::xfloat::XFloatDiveData, 1005
GenerateOrderedPidList
argo::iridium::xfloat::XFloatDiveData, 1006
GeneratePhyString

argo::iridium::xfloat::XFloatDiveData, 1006
util::xmsg::XCompositeDrift, 549
util::xmsg::XCompositeHiResProfile, 570
util::xmsg::XCompositeProfile, 608
util::xmsg::XGPS, 1276
GeneratePreparedStatement
argo::iridium::xfloat::XFloat, 924
argo::iridium::xfloat::XFloatDiveData, 1007
argo::iridium::xfloat::XFloatMailMessage, 1070
argo::iridium::xfloat::XFloatMailProcessingService, 1143
argo::iridium::xfloat::XFloatMailRetrievalService, 1165
argo::iridium::xfloat::XFloatMetadata, 1199
argo::iridium::xfloat::XFloatTelemetryDecoderService, 1250
util::mysql::LocalMysqlConfiguration, 306
util::mysql::LocalPersistentEntity, 319
util::mysql::MysqlUtils, 339
util::mysql::SqlTableDescription< T >, 393
util::xmsg::XGPS, 1277
generatePreparedStatement
argo::iridium::xfloat::XFloat, 924
argo::iridium::xfloat::XFloatDiveData, 1008
argo::iridium::xfloat::XFloatMailMessage, 1071
argo::iridium::xfloat::XFloatMailProcessingService, 1144
argo::iridium::xfloat::XFloatMailRetrievalService, 1166
argo::iridium::xfloat::XFloatMetadata, 1200
util::mysql::LocalMysqlConfiguration, 308
util::mysql::LocalPersistentEntity, 320
util::mysql::SqlTableDescription< T >, 393
util::xmsg::XGPS, 1277
GenerateReadableHex
argo::iridium::xfloat::XFloatDiveData, 1008
generateSelectAllStatement
util::mysql::SqlTableDescription< T >, 395
GenerateSqlDeleteStr
argo::iridium::xfloat::XFloat, 925
argo::iridium::xfloat::XFloatDiveData, 1009
argo::iridium::xfloat::XFloatMailMessage, 1071
argo::iridium::xfloat::XFloatMailProcessingService, 1144
argo::iridium::xfloat::XFloatMailRetrievalService, 1166
argo::iridium::xfloat::XFloatMetadata, 1200
argo::iridium::xfloat::XFloatTelemetryDecoderService, 1250
util::mysql::LocalMysqlConfiguration, 308
util::mysql::LocalPersistentEntity, 320
util::mysql::MysqlUtils, 339
util::mysql::SqlTableDescription< T >, 395
util::xmsg::XGPS, 1278
GenerateSqlSelectStr
argo::iridium::xfloat::XFloat, 925
argo::iridium::xfloat::XFloatDiveData, 1009
argo::iridium::xfloat::XFloatMailMessage, 1072
argo::iridium::xfloat::XFloatMailProcessingService, 1145
argo::iridium::xfloat::XFloatMailRetrievalService, 1167
argo::iridium::xfloat::XFloatMetadata, 1201
argo::iridium::xfloat::XFloatTelemetryDecoderService, 1251
util::mysql::LocalMysqlConfiguration, 309
util::mysql::LocalPersistentEntity, 321
util::mysql::MysqlUtils, 340
util::mysql::SqlTableDescription< T >, 396
util::xmsg::XGPS, 1278
GetAllCachedFloats
argo::iridium::xfloat::XFloat, 926
GetAllDiveData
argo::iridium::xfloat::XFloat, 926
getColumnNames
util::mysql::SqlTableDescription< T >, 396
getColumns
util::mysql::SqlTableDescription< T >, 397
getColumnSelectNames
util::mysql::SqlTableDescription< T >, 397
GetCurrentUtcTime
argo::iridium::IridiumPosition, 268
argo::iridium::xfloat::XFloat, 926
argo::iridium::xfloat::XFloatDiveData, 1009
argo::iridium::xfloat::XFloatMailMessage, 1072
argo::iridium::xfloat::XFloatMetadata, 1201
util::date::DateUtils, 246
util::position::Position, 349
util::position::PositionUtils, 363
util::xmsg::AbstractXFallRise, 71
util::xmsg::XAlamoBit2, 451
util::xmsg::XAlamoEngineering, 463
util::xmsg::XAlamoMissionConfiguration, 488
util::xmsg::XArgo, 509
util::xmsg::XCompositeIridiumPosition, 591
util::xmsg::XEngAscent, 714
util::xmsg::XEngDescent, 770
util::xmsg::XEngDrift, 802
util::xmsg::XEngIce, 828
util::xmsg::XEngIce::XEngIceErrorCode, 843
util::xmsg::XEngSurface, 866
util::xmsg::XEngSurface::XEngErrorCode, 819
util::xmsg::XFall, 888
util::xmsg::XGPS, 1279
util::xmsg::XMfaMissionCfg, 1403
util::xmsg::XRise, 1533
util::xmsg::XSurfaceData, 1561
GetDefaltKmlPathColor
argo::iridium::xfloat::XFloat, 927
argo::iridium::xfloat::XFloatPostProcessingService, 1236
util::kml::KmlUtil, 284
util::xmsg::XGPS, 1279
GetDefaltKmlPathStyleId
argo::iridium::xfloat::XFloat, 927
argo::iridium::xfloat::XFloatPostProcessingService, 1236
util::kml::KmlUtil, 284
util::xmsg::XGPS, 1280
GetDefaltKmlPlacemarkStyleId
argo::iridium::xfloat::XFloat, 928
argo::iridium::xfloat::XFloatPostProcessingService, 1237
util::kml::KmlUtil, 284
util::xmsg::XGPS, 1280
GetDirectoryPath
argo::iridium::xfloat::XFloat, 928
argo::iridium::xfloat::XFloatDiveData, 1010
argo::iridium::xfloat::XFloatMailMessage, 1073
argo::iridium::xfloat::XFloatMailRetrievalService, 1167
argo::iridium::xfloat::XFloatMetadata, 1202
util::config::LocalConfiguration, 293
util::file::FileSystemUtils, 247
GetDirectoryPathFromString
argo::iridium::xfloat::XFloat, 929
argo::iridium::xfloat::XFloatDiveData, 1010
argo::iridium::xfloat::XFloatMailMessage, 1073
argo::iridium::xfloat::XFloatMailRetrievalService, 1168
argo::iridium::xfloat::XFloatMetadata, 1202
util::config::LocalConfiguration, 294
util::file::FileSystemUtils, 247
GetDiveData

argo::iridium::xfloat::XFloat, 930
GetDiveDataMailMessagesFromDatabase
argo::iridium::xfloat::XFloatDiveData, 1011
GetDiveDataObjectFromDatabase
argo::iridium::xfloat::XFloatDiveData, 1012
GetFloat
argo::iridium::xfloat::XFloat, 931
GetGainsAndOffsets
argo::iridium::xfloat::XFloatDiveData, 1012
GetGpsPositionEndOfDive
argo::iridium::xfloat::XFloatDiveData, 1013
GetKmlFactory
argo::iridium::xfloat::XFloat, 931
argo::iridium::xfloat::XFloatPostProcessingService, 1237
util::kml::KmlUtil, 285
util::xmsg::XGPS, 1280
GetListingDirectoryPath
argo::iridium::xfloat::XFloat, 932
GetPhaseCodeName
util::xmsg::XPresCodeCounts, 1441
util::xmsg::XSensorTripletCodeCounts, 1549
GetSmallKmlPlacemarkStyleId
argo::iridium::xfloat::XFloat, 932
argo::iridium::xfloat::XFloatPostProcessingService, 1238
util::kml::KmlUtil, 285
util::xmsg::XGPS, 1282
GetSqlConnection
argo::iridium::xfloat::XFloat, 932
argo::iridium::xfloat::XFloatDiveData, 1013
argo::iridium::xfloat::XFloatMailMessage, 1074
argo::iridium::xfloat::XFloatMailProcessingService, 1145
argo::iridium::xfloat::XFloatMailRetrievalService, 1169
argo::iridium::xfloat::XFloatMetadata, 1203
argo::iridium::xfloat::XFloatTelemetryDecoderService, 1251
util::mysql::LocalMysqlConfiguration, 309
util::mysql::LocalPersistentEntity, 321
util::mysql::MysqlUtils, 340
util::mysql::SqlTableDescription< T >, 397
util::xmsg::XGPS, 1282
getSqlConnection
argo::iridium::xfloat::XFloat, 933
argo::iridium::xfloat::XFloatDiveData, 1014
argo::iridium::xfloat::XFloatMailMessage, 1075
argo::iridium::xfloat::XFloatMailProcessingService, 1146
argo::iridium::xfloat::XFloatMailRetrievalService, 1170
argo::iridium::xfloat::XFloatMetadata, 1204
util::mysql::LocalMysqlConfiguration, 310
util::mysql::LocalPersistentEntity, 322
util::mysql::SqlTableDescription< T >, 398
util::xmsg::XGPS, 1283
GetSqlDriver
argo::iridium::xfloat::XFloat, 934
argo::iridium::xfloat::XFloatDiveData, 1015
argo::iridium::xfloat::XFloatMailMessage, 1075
argo::iridium::xfloat::XFloatMailProcessingService, 1147
argo::iridium::xfloat::XFloatMailRetrievalService, 1170
argo::iridium::xfloat::XFloatMetadata, 1204
argo::iridium::xfloat::XFloatTelemetryDecoderService, 1252
util::mysql::LocalMysqlConfiguration, 311
util::mysql::LocalPersistentEntity, 323
util::mysql::MysqlUtils, 341
util::mysql::SqlTableDescription< T >, 399
util::xmsg::XGPS, 1284
GetSurfaceGainsAndOffsets
argo::iridium::xfloat::XFloatDiveData, 1015
GetTriangleKmlPlacemarkStyleId
argo::iridium::xfloat::XFloat, 934
argo::iridium::xfloat::XFloatPostProcessingService, 1238
util::kml::KmlUtil, 286
util::xmsg::XGPS, 1284
GetValueFromObject
util::mysql::Column_bool< C >, 158
util::mysql::Column_char< C >, 164
util::mysql::Column_double< C >, 171
util::mysql::Column_filesystem_path< C >, 178
util::mysql::Column_float< C >, 184
util::mysql::Column_int< C >, 191
util::mysql::Column_int16_t< C >, 197
util::mysql::Column_long< C >, 204
util::mysql::Column_posix_time< C >, 211
util::mysql::Column_string< C >, 218
util::mysql::Column_uint16_t< C >, 224
util::mysql::Column_uint8_t< C >, 231
util::mysql::Column_vector_uint8_t< C >, 237
util::mysql::SqlTableColumnDescription< C, T >, 376
global_logger_
util::log::Log, 331
gps_errors_
util::xmsg::XAlamoBit2, 456
GSSF
util::xmsg::XWave, 1605

HasBinnedProfileData
argo::iridium::xfloat::XFloatDiveData, 1016
HasCompleteBinnedProfileData
argo::iridium::xfloat::XFloatDiveData, 1016
HasCompleteProfileData
argo::iridium::xfloat::XFloatDiveData, 1017
HasCompleteUnbinnedProfileData
argo::iridium::xfloat::XFloatDiveData, 1018
HasContiguousMessageIndices
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >, 112
util::xmsg::XCompositeDrift, 550
util::xmsg::XCompositeHiResProfile, 571
util::xmsg::XCompositeProfile, 609
HasContiguousMomsns
argo::iridium::xfloat::XFloatDiveData, 1019
HasContiguousPids
argo::iridium::xfloat::XFloatDiveData, 1019
HasDiveData
argo::iridium::xfloat::XFloatDiveData, 1020
HasDriftData
argo::iridium::xfloat::XFloatDiveData, 1020
HasEngineeringData
argo::iridium::xfloat::XFloatDiveData, 1021
HasFallRiseData
argo::iridium::xfloat::XFloatDiveData, 1021
HasPositionData
argo::iridium::xfloat::XFloatDiveData, 1021
HasUnbinnedProfileData
argo::iridium::xfloat::XFloatDiveData, 1022
HasValidGpsPosition
argo::iridium::xfloat::XFloatDiveData, 1022
HasValidIridiumPosition
argo::iridium::xfloat::XFloatDiveData, 1022
HasValidPosition
argo::iridium::xfloat::XFloatDiveData, 1023
horizontal_dilution_of_precision_
util::xmsg::XGPS, 1302
htoi
util::mysql::Column_vector_uint8_t< C >, 237

util::mysql::SqlTableDescription< T >, 399
util::stream::StreamUtils, 408

i08
XDefinitions.h, 1770
i16
XDefinitions.h, 1770
i24
XDefinitions.h, 1770
i32
XDefinitions.h, 1770
iat_
util::xmsg::XMfaMissionCfg, 1408
ica_
util::xmsg::XMfaMissionCfg, 1408
icb_
util::xmsg::XMfaMissionCfg, 1408
icd_
util::xmsg::XMfaMissionCfg, 1408
icf_
util::xmsg::XMfaMissionCfg, 1408
icg_
util::xmsg::XMfaMissionCfg, 1408
icm_
util::xmsg::XMfaMissionCfg, 1409
icn_
util::xmsg::XMfaMissionCfg, 1409
ico_
util::xmsg::XMfaMissionCfg, 1409
icr_
util::xmsg::XMfaMissionCfg, 1409
ics_
util::xmsg::XMfaMissionCfg, 1409
ict_
util::xmsg::XMfaMissionCfg, 1409
icw_
util::xmsg::XMfaMissionCfg, 1409
icx_
util::xmsg::XMfaMissionCfg, 1409
id2float_map_
argo::iridium::xfloat::XFloat, 962
id2float_map_mutex_
argo::iridium::xfloat::XFloat, 962
id_dac
argo::iridium::Iridium, 254
argo::iridium::xfloat::XFloat, 935
argo::iridium::xfloat::XFloatDiveData, 1023
argo::iridium::xfloat::XFloatMailMessage, 1076
argo::iridium::xfloat::XFloatMetadata, 1205
id_dac_
argo::iridium::Iridium, 258
argo::iridium::xfloat::XFloat, 962
argo::iridium::xfloat::XFloatDiveData, 1038
argo::iridium::xfloat::XFloatMailMessage, 1125
argo::iridium::xfloat::XFloatMetadata, 1229
id_dive
argo::iridium::xfloat::XFloatDiveData, 1024
argo::iridium::xfloat::XFloatMailMessage, 1076
util::xmsg::AbstractXFallRise, 71
util::xmsg::AbstractXSensorData, 87
util::xmsg::AuxiliarySensorData, 140
util::xmsg::XAcousticPosition, 425
util::xmsg::XAlamoEngineering, 464
util::xmsg::XArgo, 509
util::xmsg::XDriftPressure, 643
util::xmsg::XDriftSalinity, 661
util::xmsg::XDriftTemperature, 680
util::xmsg::XEngAbort, 699
util::xmsg::XEngAscent, 715
util::xmsg::XEngBIT, 730
util::xmsg::XEngBITFail, 742
util::xmsg::XEngBITPass, 754
util::xmsg::XEngDescent, 771
util::xmsg::XEngDiagnostic, 787
util::xmsg::XEngDrift, 803
util::xmsg::XEngIce, 828
util::xmsg::XEngStandard, 848
util::xmsg::XEngSurface, 867
util::xmsg::XFall, 889
util::xmsg::XGPS, 1285
util::xmsg::XHiResPressure, 1311
util::xmsg::XHiResSalinity, 1329
util::xmsg::XHiResTemperature, 1348
util::xmsg::XMessage, 1367
util::xmsg::XMessageIndexable, 1377
util::xmsg::XMissionCmdEcho, 1418
util::xmsg::XMissionConfiguration, 1429
util::xmsg::XProfilePressure, 1451
util::xmsg::XProfileSalinity, 1469
util::xmsg::XProfileTemperature, 1488
util::xmsg::XPump, 1509
util::xmsg::XRise, 1533
util::xmsg::XSurfaceData, 1561
util::xmsg::XValve, 1579
util::xmsg::XWave, 1596
id_dive_
argo::iridium::xfloat::XFloatDiveData, 1038
argo::iridium::xfloat::XFloatMailMessage, 1125
util::xmsg::AbstractXFallRise, 82
util::xmsg::AbstractXSensorData, 101
util::xmsg::AuxiliarySensorData, 153
util::xmsg::XAcousticPosition, 431
util::xmsg::XAlamoEngineering, 473
util::xmsg::XArgo, 535
util::xmsg::XDriftPressure, 656
util::xmsg::XDriftSalinity, 675
util::xmsg::XDriftTemperature, 694
util::xmsg::XEngAbort, 705
util::xmsg::XEngAscent, 725
util::xmsg::XEngBIT, 737
util::xmsg::XEngBITFail, 749
util::xmsg::XEngBITPass, 761
util::xmsg::XEngDescent, 781
util::xmsg::XEngDiagnostic, 793
util::xmsg::XEngDrift, 813
util::xmsg::XEngIce, 837
util::xmsg::XEngStandard, 854
util::xmsg::XEngSurface, 879
util::xmsg::XFall, 900
util::xmsg::XGPS, 1302
util::xmsg::XHiResPressure, 1324
util::xmsg::XHiResSalinity, 1343
util::xmsg::XHiResTemperature, 1362
util::xmsg::XMessage, 1373
util::xmsg::XMessageIndexable, 1385
util::xmsg::XMissionCmdEcho, 1424
util::xmsg::XMissionConfiguration, 1437
util::xmsg::XProfilePressure, 1464
util::xmsg::XProfileSalinity, 1483
util::xmsg::XProfileTemperature, 1502
util::xmsg::XPump, 1518
util::xmsg::XRise, 1544
util::xmsg::XSurfaceData, 1572
util::xmsg::XValve, 1587
util::xmsg::XWave, 1605
id_imei
argo::iridium::Iridium, 254
argo::iridium::xfloat::XFloat, 935
argo::iridium::xfloat::XFloatDiveData, 1024
argo::iridium::xfloat::XFloatMailMessage, 1077
argo::iridium::xfloat::XFloatMetadata, 1205
id_imei_
argo::iridium::Iridium, 258
argo::iridium::xfloat::XFloat, 963

argo::iridium::xfloat::XFloatDiveData, 1038
argo::iridium::xfloat::XFloatMailMessage, 1125
argo::iridium::xfloat::XFloatMetadata, 1229
id_instrument
argo::iridium::Iridium, 254
argo::iridium::xfloat::XFloat, 935
argo::iridium::xfloat::XFloatDiveData, 1025
argo::iridium::xfloat::XFloatMailMessage, 1077
argo::iridium::xfloat::XFloatMetadata, 1206
util::xmsg::AbstractXFallRise, 72
util::xmsg::AbstractXSensorData, 88
util::xmsg::AuxiliarySensorData, 141
util::xmsg::XAcousticPosition, 426
util::xmsg::XAlamoEngineering, 465
util::xmsg::XArgo, 510
util::xmsg::XDriftPressure, 644
util::xmsg::XDriftSalinity, 662
util::xmsg::XDriftTemperature, 681
util::xmsg::XEngAbort, 700
util::xmsg::XEngAscent, 716
util::xmsg::XEngBIT, 731
util::xmsg::XEngBITFail, 743
util::xmsg::XEngBITPass, 755
util::xmsg::XEngDescent, 772
util::xmsg::XEngDiagnostic, 788
util::xmsg::XEngDrift, 804
util::xmsg::XEngIce, 829
util::xmsg::XEngStandard, 849
util::xmsg::XEngSurface, 868
util::xmsg::XFall, 890
util::xmsg::XGPS, 1286
util::xmsg::XHiResPressure, 1312
util::xmsg::XHiResSalinity, 1330
util::xmsg::XHiResTemperature, 1349
util::xmsg::XMessage, 1368
util::xmsg::XMessageIndexable, 1378
util::xmsg::XMissionCmdEcho, 1419
util::xmsg::XMissionConfiguration, 1430
util::xmsg::XProfilePressure, 1452
util::xmsg::XProfileSalinity, 1470
util::xmsg::XProfileTemperature, 1489
util::xmsg::XPump, 1510
util::xmsg::XRise, 1534
util::xmsg::XSurfaceData, 1562
util::xmsg::XValve, 1580
util::xmsg::XWave, 1597
id_instrument_
argo::iridium::Iridium, 258
argo::iridium::xfloat::XFloat, 963
argo::iridium::xfloat::XFloatDiveData, 1038
argo::iridium::xfloat::XFloatMailMessage, 1125
argo::iridium::xfloat::XFloatMetadata, 1229
util::xmsg::AbstractXFallRise, 83
util::xmsg::AbstractXSensorData, 101
util::xmsg::AuxiliarySensorData, 153
util::xmsg::XAcousticPosition, 431
util::xmsg::XAlamoEngineering, 473
util::xmsg::XArgo, 535
util::xmsg::XDriftPressure, 656
util::xmsg::XDriftSalinity, 675
util::xmsg::XDriftTemperature, 694
util::xmsg::XEngAbort, 705
util::xmsg::XEngAscent, 725
util::xmsg::XEngBIT, 737
util::xmsg::XEngBITFail, 749
util::xmsg::XEngBITPass, 761
util::xmsg::XEngDescent, 781
util::xmsg::XEngDiagnostic, 793
util::xmsg::XEngDrift, 813
util::xmsg::XEngIce, 837
util::xmsg::XEngStandard, 854
util::xmsg::XEngSurface, 879
util::xmsg::XFall, 901
util::xmsg::XGPS, 1302
util::xmsg::XHiResPressure, 1324
util::xmsg::XHiResSalinity, 1343
util::xmsg::XHiResTemperature, 1362
util::xmsg::XMessage, 1373
util::xmsg::XMessageIndexable, 1385
util::xmsg::XMissionCmdEcho, 1424
util::xmsg::XMissionConfiguration, 1437
util::xmsg::XProfilePressure, 1464
util::xmsg::XProfileSalinity, 1483
util::xmsg::XProfileTemperature, 1502
util::xmsg::XPump, 1518
util::xmsg::XRise, 1545
util::xmsg::XSurfaceData, 1573
util::xmsg::XValve, 1587
util::xmsg::XWave, 1605
id_msg
argo::iridium::xfloat::XFloatMailMessage, 1078
id_msg_
argo::iridium::xfloat::XFloatMailMessage, 1126
id_msg_uid
argo::iridium::xfloat::XFloatMailMessage, 1078
id_msg_uid_
argo::iridium::xfloat::XFloatMailMessage, 1126
id_sbd_momsn
argo::iridium::xfloat::XFloatMailMessage, 1079
id_sbd_momsn_
argo::iridium::xfloat::XFloatMailMessage, 1126
id_sbd_mtmsn
argo::iridium::xfloat::XFloatMailMessage, 1079
id_sbd_mtmsn_
argo::iridium::xfloat::XFloatMailMessage, 1126
id_sbd_packet
argo::iridium::xfloat::XFloatMailMessage, 1080
id_sbd_packet_
argo::iridium::xfloat::XFloatMailMessage, 1127
id_wmo
argo::iridium::Iridium, 255
argo::iridium::xfloat::XFloat, 936
argo::iridium::xfloat::XFloatDiveData, 1025
argo::iridium::xfloat::XFloatMailMessage, 1080
argo::iridium::xfloat::XFloatMetadata, 1207
id_wmo_
argo::iridium::Iridium, 259
argo::iridium::xfloat::XFloat, 963
argo::iridium::xfloat::XFloatDiveData, 1038
argo::iridium::xfloat::XFloatMailMessage, 1127
argo::iridium::xfloat::XFloatMetadata, 1229
id_xmsg_type
util::xmsg::AbstractXFallRise, 73
util::xmsg::AbstractXSensorData, 89
util::xmsg::AuxiliarySensorData, 142
util::xmsg::XAcousticPosition, 427
util::xmsg::XAlamoEngineering, 466
util::xmsg::XArgo, 511
util::xmsg::XDriftPressure, 645
util::xmsg::XDriftSalinity, 663
util::xmsg::XDriftTemperature, 682
util::xmsg::XEngAbort, 701
util::xmsg::XEngAscent, 717
util::xmsg::XEngBIT, 732
util::xmsg::XEngBITFail, 744
util::xmsg::XEngBITPass, 756
util::xmsg::XEngDescent, 773
util::xmsg::XEngDiagnostic, 789
util::xmsg::XEngDrift, 805
util::xmsg::XEngIce, 830
util::xmsg::XEngStandard, 850
util::xmsg::XEngSurface, 869
util::xmsg::XFall, 891
util::xmsg::XGPS, 1287

util::xmsg::XHiResPressure, 1313
util::xmsg::XHiResSalinity, 1331
util::xmsg::XHiResTemperature, 1350
util::xmsg::XMessage, 1369
util::xmsg::XMessageIndexable, 1379
util::xmsg::XMissionCmdEcho, 1420
util::xmsg::XMissionConfiguration, 1431
util::xmsg::XProfilePressure, 1453
util::xmsg::XProfileSalinity, 1471
util::xmsg::XProfileTemperature, 1490
util::xmsg::XPump, 1511
util::xmsg::XRise, 1535
util::xmsg::XSurfaceData, 1563
util::xmsg::XValve, 1581
util::xmsg::XWave, 1598
id_xmsg_type_
util::xmsg::AbstractXFallRise, 83
util::xmsg::AbstractXSensorData, 101
util::xmsg::AuxiliarySensorData, 153
util::xmsg::XAcousticPosition, 432
util::xmsg::XAlamoEngineering, 473
util::xmsg::XArgo, 535
util::xmsg::XDriftPressure, 656
util::xmsg::XDriftSalinity, 675
util::xmsg::XDriftTemperature, 694
util::xmsg::XEngAbort, 706
util::xmsg::XEngAscent, 725
util::xmsg::XEngBIT, 737
util::xmsg::XEngBITFail, 749
util::xmsg::XEngBITPass, 761
util::xmsg::XEngDescent, 782
util::xmsg::XEngDiagnostic, 794
util::xmsg::XEngDrift, 813
util::xmsg::XEngIce, 837
util::xmsg::XEngStandard, 855
util::xmsg::XEngSurface, 879
util::xmsg::XFall, 901
util::xmsg::XGPS, 1302
util::xmsg::XHiResPressure, 1324
util::xmsg::XHiResSalinity, 1343
util::xmsg::XHiResTemperature, 1362
util::xmsg::XMessage, 1374
util::xmsg::XMessageIndexable, 1385
util::xmsg::XMissionCmdEcho, 1425
util::xmsg::XMissionConfiguration, 1438
util::xmsg::XProfilePressure, 1464
util::xmsg::XProfileSalinity, 1483
util::xmsg::XProfileTemperature, 1502
util::xmsg::XPump, 1518
util::xmsg::XRise, 1545
util::xmsg::XSurfaceData, 1573
util::xmsg::XValve, 1587
util::xmsg::XWave, 1605
idle_sleep_seconds_
argo::iridium::xfloat::XFloatPostProcessingService, 1241
ilt_
util::xmsg::XMfaMissionCfg, 1409
imm_
util::xmsg::XMfaMissionCfg, 1410
imt_
util::xmsg::XMfaMissionCfg, 1410
inbox_folder_
argo::iridium::xfloat::XFloatMailRetrievalService, 1176
index
boost::container::flat_map< C, D >, 251
Initialize
argo::iridium::xfloat::XFloat, 959
Main.cpp, 2087
Main.h, 2091
InitializeConfiguration
util::config::LocalConfiguration, 295
InitializeLog
util::log::Log, 330
insert_on_duplicate_key_update_statement_
util::mysql::SqlTableDescription< T >, 404
instrument_ids_to_disable_phy_generation
argo::iridium::xfloat::XFloat, 936
instrument_ids_to_disable_phy_generation_
argo::iridium::xfloat::XFloat, 963
instrument_ids_to_enable_phy_generation
argo::iridium::xfloat::XFloat, 937
instrument_ids_to_enable_phy_generation_
argo::iridium::xfloat::XFloat, 963
intermediate_listing_directory_path_
argo::iridium::xfloat::XFloatDiveData, 1038
intermediate_listing_file_generation_is_enabled_
argo::iridium::xfloat::XFloatDiveData, 1039
intermediate_listing_file_path
argo::iridium::xfloat::XFloatDiveData, 1026
IRI_DIVE_UNKNOWN
IridiumDefinitions.h, 1635
IRI_GAIN_INVALID
IridiumDefinitions.h, 1635
IRI_ID_UNKNOWN
IridiumDefinitions.h, 1635
IRI_OFFS_INVALID
IridiumDefinitions.h, 1635
IRI_POS_INVALID_CEP
IridiumDefinitions.h, 1635
IRI_POS_INVALID_LAT
IridiumDefinitions.h, 1635
IRI_POS_INVALID_LON
IridiumDefinitions.h, 1636
Iridium
argo::iridium::Iridium, 253
iridium_position_as_iso6709_string
argo::iridium::IridiumPosition, 268
argo::iridium::xfloat::XFloatMailMessage, 1081
iridium_position_cep_radius
argo::iridium::IridiumPosition, 268
argo::iridium::xfloat::XFloatMailMessage, 1081
iridium_position_cep_radius_
argo::iridium::IridiumPosition, 281
argo::iridium::xfloat::XFloatMailMessage, 1127
iridium_position_datetime
argo::iridium::IridiumPosition, 269
argo::iridium::xfloat::XFloatMailMessage, 1082
iridium_position_datetime_as_iso8601_string
argo::iridium::IridiumPosition, 269
argo::iridium::xfloat::XFloatMailMessage, 1082
iridium_position_datetime_first_
argo::iridium::xfloat::XFloatDiveData, 1039
iridium_position_datetime_last_
argo::iridium::xfloat::XFloatDiveData, 1039
iridium_position_is_valid
argo::iridium::IridiumPosition, 270
argo::iridium::xfloat::XFloatMailMessage, 1083
iridium_position_latitude
argo::iridium::IridiumPosition, 270
argo::iridium::xfloat::XFloatMailMessage, 1083
iridium_position_longitude
argo::iridium::IridiumPosition, 271
argo::iridium::xfloat::XFloatMailMessage, 1084
IridiumDefinitions.h
IRI_DIVE_UNKNOWN, 1635
IRI_GAIN_INVALID, 1635
IRI_ID_UNKNOWN, 1635
IRI_OFFS_INVALID, 1635
IRI_POS_INVALID_CEP, 1635
IRI_POS_INVALID_LAT, 1635
IRI_POS_INVALID_LON, 1636
IridiumPosition

argo::iridium::IridiumPosition, 263
is_aggregated_data_valid
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >, 112
util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >, 131
util::xmsg::XCompositeDrift, 550
util::xmsg::XCompositeHiResProfile, 572
util::xmsg::XCompositeProfile, 610
util::xmsg::XCompositeSurfaceData, 631
is_daemon
util::config::LocalConfiguration, 295
is_daemon_
util::config::LocalConfiguration, 299
is_primary_key
util::mysql::Column_bool< C >, 158
util::mysql::Column_char< C >, 164
util::mysql::Column_double< C >, 171
util::mysql::Column_filesystem_path< C >, 178
util::mysql::Column_float< C >, 184
util::mysql::Column_int< C >, 191
util::mysql::Column_int16_t< C >, 197
util::mysql::Column_long< C >, 204
util::mysql::Column_posix_time< C >, 211
util::mysql::Column_string< C >, 218
util::mysql::Column_uint16_t< C >, 224
util::mysql::Column_uint8_t< C >, 231
util::mysql::Column_vector_uint8_t< C >, 238
util::mysql::SqlTableColumnDescription< C, T >, 376
util::mysql::SqlTableColumnDescriptionBase, 381
is_primary_key_
util::mysql::Column_bool< C >, 160
util::mysql::Column_char< C >, 167
util::mysql::Column_double< C >, 173
util::mysql::Column_filesystem_path< C >, 180
util::mysql::Column_float< C >, 187
util::mysql::Column_int< C >, 193
util::mysql::Column_int16_t< C >, 200
util::mysql::Column_long< C >, 206
util::mysql::Column_posix_time< C >, 213
util::mysql::Column_string< C >, 220
util::mysql::Column_uint16_t< C >, 227
util::mysql::Column_uint8_t< C >, 233
util::mysql::Column_vector_uint8_t< C >, 240
util::mysql::SqlTableColumnDescription< C, T >, 377
util::mysql::SqlTableColumnDescriptionBase, 382
is_reprocess_all_
argo::iridium::xfloat::XFloatMailRetrievalService, 1177
isa_
util::xmsg::XMfaMissionCfg, 1410
isd_
util::xmsg::XMfaMissionCfg, 1410
IsDataComplete
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >, 113
util::xmsg::XCompositeDrift, 551
util::xmsg::XCompositeHiResProfile, 572
util::xmsg::XCompositeProfile, 610
IsEmpty
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >, 114
util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >, 131
util::xmsg::XCompositeDrift, 551
util::xmsg::XCompositeHiResProfile, 572
util::xmsg::XCompositeProfile, 610
util::xmsg::XCompositeSurfaceData, 631
ism_
util::xmsg::XMfaMissionCfg, 1410
iso_
util::xmsg::XMfaMissionCfg, 1410
isp_
util::xmsg::XMfaMissionCfg, 1410
IsPositionValid
argo::iridium::IridiumPosition, 271
argo::iridium::xfloat::XFloat, 937
argo::iridium::xfloat::XFloatMailMessage, 1084
argo::iridium::xfloat::XFloatMetadata, 1207
util::position::Position, 349
util::position::PositionUtils, 364
util::xmsg::XCompositeIridiumPosition, 592
util::xmsg::XGPS, 1287
iss_
util::xmsg::XMfaMissionCfg, 1410
ist_
util::xmsg::XMfaMissionCfg, 1410
isv_
util::xmsg::XMfaMissionCfg, 1411
isw_
util::xmsg::XMfaMissionCfg, 1411
iue_
util::xmsg::XMfaMissionCfg, 1411

kml_default_path_colors_
argo::iridium::xfloat::XFloat, 963
argo::iridium::xfloat::XFloatPostProcessingService, 1241
util::kml::KmlUtil, 288
util::xmsg::XGPS, 1303
kml_default_path_style_ids_
argo::iridium::xfloat::XFloat, 964
argo::iridium::xfloat::XFloatPostProcessingService, 1242
util::kml::KmlUtil, 288
util::xmsg::XGPS, 1303
kml_default_placemark_style_ids_
argo::iridium::xfloat::XFloat, 964
argo::iridium::xfloat::XFloatPostProcessingService, 1242
util::kml::KmlUtil, 288
util::xmsg::XGPS, 1303
kml_directory_path_
argo::iridium::xfloat::XFloat, 964
kml_document_
argo::iridium::xfloat::XFloat, 964
kml_document_updated_
argo::iridium::xfloat::XFloat, 965
kml_factory_
argo::iridium::xfloat::XFloat, 965
argo::iridium::xfloat::XFloatPostProcessingService, 1242
util::kml::KmlUtil, 289
util::xmsg::XGPS, 1303
kml_file_generation_is_enabled_
argo::iridium::xfloat::XFloat, 965
kml_file_path_
argo::iridium::xfloat::XFloat, 965
kml_file_updated_
argo::iridium::xfloat::XFloat, 965
kml_small_placemark_style_ids_
argo::iridium::xfloat::XFloat, 965
argo::iridium::xfloat::XFloatPostProcessingService, 1242
util::kml::KmlUtil, 289
util::xmsg::XGPS, 1304
kml_triangle_placemark_style_ids_
argo::iridium::xfloat::XFloat, 966
argo::iridium::xfloat::XFloatPostProcessingService, 1243
util::kml::KmlUtil, 289

util::xmsg::XGPS, 1304

latest_gps_position_datetime_
argo::iridium::xfloat::XFloat, 966
latest_gps_position_dive_number_
argo::iridium::xfloat::XFloat, 966
latest_gps_position_latitude_
argo::iridium::xfloat::XFloat, 966
latest_gps_position_longitude_
argo::iridium::xfloat::XFloat, 966
latest_iridium_position_cep_radius_
argo::iridium::xfloat::XFloat, 966
latest_iridium_position_datetime_
argo::iridium::xfloat::XFloat, 967
latest_iridium_position_dive_number_
argo::iridium::xfloat::XFloat, 967
latest_iridium_position_latitude_
argo::iridium::xfloat::XFloat, 967
latest_iridium_position_longitude_
argo::iridium::xfloat::XFloat, 967
latest_iridium_sbd_momsn_
argo::iridium::xfloat::XFloat, 967
latest_iridium_sbd_mtmsn_
argo::iridium::xfloat::XFloat, 967
latest_iridium_startup_position_datetime_
argo::iridium::xfloat::XFloat, 967
latest_iridium_startup_sbd_momsn_
argo::iridium::xfloat::XFloat, 967
ldbg
Log.h, 2144
lerr
Log.h, 2145
lfat
Log.h, 2145
Libraries and Database Schema, 11
linf
Log.h, 2145
ListDirectories
argo::iridium::xfloat::XFloat, 937
argo::iridium::xfloat::XFloatDiveData, 1026
argo::iridium::xfloat::XFloatMailMessage, 1085
argo::iridium::xfloat::XFloatMailRetrievalService, 1171
argo::iridium::xfloat::XFloatMetadata, 1207
util::config::LocalConfiguration, 296
util::file::FileSystemUtils, 248
ListFiles
argo::iridium::xfloat::XFloat, 938
argo::iridium::xfloat::XFloatDiveData, 1026
argo::iridium::xfloat::XFloatMailMessage, 1085
argo::iridium::xfloat::XFloatMailRetrievalService, 1171
argo::iridium::xfloat::XFloatMetadata, 1208
util::config::LocalConfiguration, 296
util::file::FileSystemUtils, 248
listing_file_path_
argo::iridium::xfloat::XFloatDiveData, 1039
listing_file_updated_
argo::iridium::xfloat::XFloatDiveData, 1039
ListMetadataFiles
argo::iridium::xfloat::XFloatMetadata, 1208
ListRawMailFiles
argo::iridium::xfloat::XFloatMailRetrievalService, 1172
llog
Log.h, 2145
LoadFile
argo::iridium::xfloat::XFloatMetadata, 1209
LoadLocalCacheFromDatabase
argo::iridium::xfloat::XFloat, 938
LoadMetadataFromFiles
argo::iridium::xfloat::XFloat, 939
argo::iridium::xfloat::XFloatMetadata, 1209
local_path
argo::iridium::xfloat::XFloatMetadata, 1210
local_path_
argo::iridium::xfloat::XFloatMetadata, 1229
LocalConfiguration
util::config::LocalConfiguration, 291
LocalConfiguration.h
CFG_DEFAULT_ISDAEMON, 1618
CFG_DEFAULT_MAIL_CHK_DLY, 1618
CFG_DEFAULT_MAX_FETCH, 1618
CFG_DEFAULT_MIN_UID, 1618
CFG_DEFAULT_NDECO, 1618
CFG_DEFAULT_NPROC, 1618
CFG_DEFAULT_PATH, 1618
CFG_DEFAULT_PATH_PID, 1618
CFG_DEFAULT_REPROCESS_ALL, 1619
CFG_FMT_DESC_LEN, 1619
CFG_FMT_LINE_LEN, 1619
PROG_DESC, 1619
PROG_ID, 1619
PROG_MAJOR_VERS, 1619
PROG_MINOR_VERS, 1619
PROG_NAME, 1619
PROG_VERS, 1620
PROG_XVERS, 1620
LocalMysqlConfiguration
util::mysql::LocalMysqlConfiguration, 302
LocalMysqlConfiguration.h
CFG_MYSQL_DFLT_ADDRESS, 1627
CFG_MYSQL_DFLT_PORT, 1627
CFG_MYSQL_DFLT_PROTOCOL, 1627
CFG_MYSQL_DFLT_SCHEMA, 1627
LocalPersistentEntity
util::mysql::LocalPersistentEntity, 315
Lockable
util::thread::Lockable, 326
LockObject
argo::iridium::Iridium, 255
argo::iridium::xfloat::XFloat, 939
argo::iridium::xfloat::XFloatDiveData, 1027
argo::iridium::xfloat::XFloatMailMessage, 1086
argo::iridium::xfloat::XFloatMetadata, 1210
util::thread::Lockable, 326
util::thread::RecursiveLockable, 366
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >, 114
util::xmsg::XCompositeDrift, 551
util::xmsg::XCompositeHiResProfile, 573
util::xmsg::XCompositeProfile, 611
LOG
Log.h, 2145
Log
util::log::Log, 328
Log.h
CATCH_BLOCK, 2140
CATCH_NOTHROW, 2141
dbg1, 2142
dbg2, 2142
dbg3, 2142
DEFAULT_LOG_LEVEL, 2142
exc_catchAny, 2142
exc_catchAny_noThrow, 2142
exc_catchStandard, 2143
exc_catchStandard_noThrow, 2143
exc_diag, 2143
EXC_THROW, 2143
ldbg, 2144
lerr, 2145
lfat, 2145
linf, 2145

llog, 2145
LOG, 2145
lout, 2145
lwrn, 2145
N_LOG_LEVEL_NAMES, 2146
PP_STR, 2146
X_PP_STR, 2146
LOG_DBG
util::log, 44
LOG_DBG1
util::log, 44
LOG_DBG2
util::log, 44
LOG_DBG3
util::log, 44
log_directory_path_
util::log::Log, 331
LOG_ERR
util::log, 44
LOG_FAT
util::log, 44
LOG_INF
util::log, 44
LOG_LOG
util::log, 44
log_msg_severity_level_
util::log::Log, 331
log_msg_stream_
util::log::Log, 331
log_reporting_level
util::log::Log, 330
log_reporting_level_
util::log::Log, 332
LOG_WRN
util::log, 44
LogLevel_t
util::log, 44
lout
Log.h, 2145
lwrn
Log.h, 2145

magnt
util::xmsg::XWaveRawSample, 1608
mail_fetch_mode_
argo::iridium::xfloat::XFloatMailRetrievalService, 1177
mail_messages_
argo::iridium::xfloat::XFloatDiveData, 1039
mail_server_host_
argo::iridium::xfloat::XFloatMailRetrievalService, 1177
mail_server_inbox_folder_path_
argo::iridium::xfloat::XFloatMailRetrievalService, 1177
mail_server_password_
argo::iridium::xfloat::XFloatMailRetrievalService, 1177
mail_server_protocol_
argo::iridium::xfloat::XFloatMailRetrievalService, 1178
mail_server_user_
argo::iridium::xfloat::XFloatMailRetrievalService, 1178
mail_session
argo::iridium::xfloat::XFloatMailRetrievalService, 1172
mail_session_
argo::iridium::xfloat::XFloatMailRetrievalService, 1178
mail_store_
argo::iridium::xfloat::XFloatMailRetrievalService, 1178
main
Main.cpp, 2088
Main.cpp, 2086
Initialize, 2087
main, 2088
StartWorkerThreads, 2089
TerminateWorkerThreads, 2089
Main.h, 2090
Initialize, 2091
StartWorkerThreads, 2091
TerminateWorkerThreads, 2092
mam_
util::xmsg::XAlamoMissionConfiguration, 492
util::xmsg::XMfaMissionCfg, 1411
manufacturer
argo::iridium::xfloat::XFloatMetadata, 1210
manufacturer_
argo::iridium::xfloat::XFloatMetadata, 1229
mat_
util::xmsg::XAlamoMissionConfiguration, 492
util::xmsg::XMfaMissionCfg, 1411
max_ascent_duration
util::xmsg::XArgo, 512
max_ascent_duration_
util::xmsg::XArgo, 535
max_fall_to_drift_duration
util::xmsg::XArgo, 512
max_fall_to_drift_duration_
util::xmsg::XArgo, 536
max_fall_to_profle_duration
util::xmsg::XArgo, 512
max_fall_to_profle_duration_
util::xmsg::XArgo, 536
max_num_msgs_to_fetch_
argo::iridium::xfloat::XFloatMailRetrievalService, 1178
max_num_samples
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >, 115
util::xmsg::XCompositeDrift, 552
util::xmsg::XCompositeHiResProfile, 573
util::xmsg::XCompositeProfile, 611
max_surface_duration
util::xmsg::XArgo, 513
max_surface_duration_
util::xmsg::XArgo, 536
mds_
util::xmsg::XAlamoMissionConfiguration, 492
util::xmsg::XMfaMissionCfg, 1411
message_
util::log::MessageException, 335
message_start_index_
argo::iridium::xfloat::XFloatMailRetrievalService, 1178
MessageException
util::log::MessageException, 334
MessageException.h
ATTR_NO_EXCEPTIONS, 2156
metadata
argo::iridium::xfloat::XFloat, 940
metadata_
argo::iridium::xfloat::XFloat, 967
metadata_directory_path_
argo::iridium::xfloat::XFloatMetadata, 1230
mfd_power_
util::xmsg::XAcousticCacst, 420
mission_phase
util::xmsg::XArgo, 513
mission_phase_
util::xmsg::XArgo, 536
mnm_

util::xmsg::XMfaMissionCfg, 1411
mns_
util::xmsg::XMfaMissionCfg, 1411
move_mail_messages_
argo::iridium::xfloat::XFloatMailRetrievalService, 1179
MoveFileToDirectory
argo::iridium::xfloat::XFloat, 940
argo::iridium::xfloat::XFloatDiveData, 1027
argo::iridium::xfloat::XFloatMailMessage, 1086
argo::iridium::xfloat::XFloatMailRetrievalService, 1173
argo::iridium::xfloat::XFloatMetadata, 1211
util::config::LocalConfiguration, 296
util::file::FileSystemUtils, 250
mpc_
util::xmsg::XMfaMissionCfg, 1411
mps_
util::xmsg::XMfaMissionCfg, 1412
msg_body_is_valid
argo::iridium::xfloat::XFloatMailMessage, 1087
msg_body_is_valid_
argo::iridium::xfloat::XFloatMailMessage, 1127
msg_body_text
argo::iridium::xfloat::XFloatMailMessage, 1087
msg_body_text_
argo::iridium::xfloat::XFloatMailMessage, 1127
msg_data_
util::xmsg::XMfaMissionCfg, 1412
msg_decoding_queue_
argo::iridium::xfloat::XFloatTelemetryDecoderService, 1256
msg_header_datetime
argo::iridium::xfloat::XFloatMailMessage, 1088
msg_header_datetime_
argo::iridium::xfloat::XFloatMailMessage, 1128
msg_header_from
argo::iridium::xfloat::XFloatMailMessage, 1088
msg_header_from_
argo::iridium::xfloat::XFloatMailMessage, 1128
msg_header_is_valid
argo::iridium::xfloat::XFloatMailMessage, 1088
msg_header_is_valid_
argo::iridium::xfloat::XFloatMailMessage, 1128
msg_header_raw_text
argo::iridium::xfloat::XFloatMailMessage, 1089
msg_header_raw_text_
argo::iridium::xfloat::XFloatMailMessage, 1128
msg_header_subject
argo::iridium::xfloat::XFloatMailMessage, 1089
msg_header_subject_
argo::iridium::xfloat::XFloatMailMessage, 1129
msg_header_valid_sender_regex_
argo::iridium::xfloat::XFloatMailMessage, 1129
msg_header_valid_subject_regex_
argo::iridium::xfloat::XFloatMailMessage, 1129
msg_is_valid
argo::iridium::xfloat::XFloatMailMessage, 1089
msg_is_valid_
argo::iridium::xfloat::XFloatMailMessage, 1129
msg_processing_queue_
argo::iridium::xfloat::XFloatMailProcessingService, 1152
msg_raw_text
argo::iridium::xfloat::XFloatMailMessage, 1090
msg_raw_text_
argo::iridium::xfloat::XFloatMailMessage, 1129
msg_version_
util::xmsg::XAlamoMissionConfiguration, 493
util::xmsg::XMfaMissionCfg, 1412
MsgDecodingQueuePush
argo::iridium::xfloat::XFloatTelemetryDecoderService, 1252
MsgProcessingQueuePop
argo::iridium::xfloat::XFloatMailProcessingService, 1147
argo::iridium::xfloat::XFloatTelemetryDecoderService, 1253
MsgProcessingQueuePush
argo::iridium::xfloat::XFloatMailProcessingService, 1148
MsgProcessingQueuePushAll
argo::iridium::xfloat::XFloatMailProcessingService, 1148
msn_
util::xmsg::XAlamoMissionConfiguration, 493
util::xmsg::XMfaMissionCfg, 1412
MSSF
util::xmsg::XWave, 1605
mtd_
util::xmsg::XAlamoMissionConfiguration, 493
util::xmsg::XMfaMissionCfg, 1412
mtt_
util::xmsg::XAlamoMissionConfiguration, 493
util::xmsg::XMfaMissionCfg, 1412
mutator_
util::mysql::Column_bool< C >, 160
util::mysql::Column_char< C >, 167
util::mysql::Column_double< C >, 173
util::mysql::Column_filesystem_path< C >, 180
util::mysql::Column_float< C >, 187
util::mysql::Column_int< C >, 193
util::mysql::Column_int16_t< C >, 200
util::mysql::Column_long< C >, 206
util::mysql::Column_posix_time< C >, 213
util::mysql::Column_string< C >, 220
util::mysql::Column_uint16_t< C >, 227
util::mysql::Column_uint8_t< C >, 233
util::mysql::Column_vector_uint8_t< C >, 240
util::mysql::SqlTableColumnDescription< C, T >, 378
mutex
util::thread::SynchronousQueue< T >, 412
mutex_
argo::iridium::Iridium, 259
argo::iridium::xfloat::XFloat, 968
argo::iridium::xfloat::XFloatDiveData, 1040
argo::iridium::xfloat::XFloatMailMessage, 1130
argo::iridium::xfloat::XFloatMetadata, 1230
util::thread::Lockable, 327
util::thread::RecursiveLockable, 367
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >, 124
util::xmsg::XCompositeDrift, 558
util::xmsg::XCompositeHiResProfile, 579
util::xmsg::XCompositeProfile, 617

N_LOG_LEVEL_NAMES
Log.h, 2146
netcdf_directory_path_
argo::iridium::xfloat::XFloatDiveData, 1041
netcdf_file_generation_is_enabled_
argo::iridium::xfloat::XFloat, 969
argo::iridium::xfloat::XFloatDiveData, 1041
netcdf_file_path_
argo::iridium::xfloat::XFloatDiveData, 1041
netcdf_file_updated_
argo::iridium::xfloat::XFloatDiveData, 1042
netcdf_reference_datetime_
argo::iridium::xfloat::XFloatDiveData, 1042
num_elements
util::xmsg::AbstractXComposite< T >, 64
util::xmsg::XCompositeAuxTemp, 542

util::xmsg::XCompositeFall, 563
util::xmsg::XCompositeIridiumPosition, 592
util::xmsg::XCompositePump, 622
util::xmsg::XCompositeRise, 626
util::xmsg::XCompositeValve, 638
util::xmsg::XPump, 1512
util::xmsg::XValve, 1581
num_errors_
util::xmsg::XEngSurface, 879
num_pres_samples
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >, 115
util::xmsg::XCompositeDrift, 552
util::xmsg::XCompositeHiResProfile, 573
util::xmsg::XCompositeProfile, 611
num_saln_samples
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >, 116
util::xmsg::XCompositeDrift, 552
util::xmsg::XCompositeHiResProfile, 574
util::xmsg::XCompositeProfile, 612
num_samples
util::xmsg::AbstractXFallRise, 74
util::xmsg::AbstractXSensorData, 90
util::xmsg::AuxiliarySensorData, 142
util::xmsg::XDriftPressure, 645
util::xmsg::XDriftSalinity, 664
util::xmsg::XDriftTemperature, 683
util::xmsg::XFall, 891
util::xmsg::XHiResPressure, 1313
util::xmsg::XHiResSalinity, 1332
util::xmsg::XHiResTemperature, 1351
util::xmsg::XProfilePressure, 1453
util::xmsg::XProfileSalinity, 1472
util::xmsg::XProfileTemperature, 1491
util::xmsg::XRise, 1536
util::xmsg::XSurfaceData, 1564
num_satellites_
util::xmsg::XGPS, 1304
num_sbd_msgs_
argo::iridium::xfloat::XFloatDiveData, 1042
num_sbd_packets_for_dive_
util::xmsg::XEngSurface, 879
num_seek_attempts_
util::xmsg::XEngDescent, 782
num_seeks_to_drift_pressure
util::xmsg::XArgo, 513
num_seeks_to_drift_pressure_
util::xmsg::XArgo, 536
num_temp_samples
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >, 117
util::xmsg::XCompositeDrift, 553
util::xmsg::XCompositeHiResProfile, 574
util::xmsg::XCompositeProfile, 612
num_threads
argo::iridium::xfloat::XFloatMailProcessingService, 1149
argo::iridium::xfloat::XFloatTelemetryDecoderService, 1253
num_threads_
argo::iridium::xfloat::XFloatMailProcessingService, 1152
argo::iridium::xfloat::XFloatTelemetryDecoderService, 1256
num_transmit_attempts_for_previous_dive_
util::xmsg::XEngSurface, 879

offset_pressure
argo::iridium::xfloat::XFloat, 941
util::xmsg::XArgo, 514
offset_pressure_
argo::iridium::xfloat::XFloat, 969
util::xmsg::XArgo, 536
offset_pressure_surface
argo::iridium::xfloat::XFloat, 941
util::xmsg::XArgo, 514
offset_pressure_surface_
argo::iridium::xfloat::XFloat, 969
util::xmsg::XArgo, 536
offset_salinity
argo::iridium::xfloat::XFloat, 941
util::xmsg::XArgo, 514
offset_salinity_
argo::iridium::xfloat::XFloat, 970
util::xmsg::XArgo, 537
offset_salinity_surface
argo::iridium::xfloat::XFloat, 942
util::xmsg::XArgo, 515
offset_salinity_surface_
argo::iridium::xfloat::XFloat, 970
util::xmsg::XArgo, 537
offset_temperature
argo::iridium::xfloat::XFloat, 942
util::xmsg::XArgo, 515
offset_temperature_
argo::iridium::xfloat::XFloat, 970
util::xmsg::XArgo, 537
offset_temperature_surface
argo::iridium::xfloat::XFloat, 942
util::xmsg::XArgo, 515
offset_temperature_surface_
argo::iridium::xfloat::XFloat, 970
util::xmsg::XArgo, 537
oil_pump_curr_max_
util::xmsg::XAlamoBit2, 456
oil_pump_errors_
util::xmsg::XAlamoBit2, 456
oil_pump_test_duration_
util::xmsg::XAlamoBit2, 456
oil_pump_v12_curr_max_
util::xmsg::XAlamoBit2, 456
oil_pump_volt_max_
util::xmsg::XAlamoBit2, 456
oil_valve_cal_open_
util::xmsg::XAlamoBit2, 456
oil_valve_closed_cal_position_mv_
util::xmsg::XEngSurface, 879
oil_valve_closed_position_at_end_mv_
util::xmsg::XEngSurface, 879
oil_valve_closed_position_during_fall_pump_mv_
util::xmsg::XEngDescent, 782
oil_valve_errors_
util::xmsg::XAlamoBit2, 457
oil_valve_open_
util::xmsg::XAlamoBit2, 457
oil_valve_open_cal_position_mv_
util::xmsg::XEngSurface, 880
oil_valve_open_position_at_surface_mv_
util::xmsg::XEngDescent, 782
oil_valve_open_position_during_air_assist_mv_
util::xmsg::XEngSurface, 880
omega
util::xmsg::XWaveRawSample, 1609
OpenMailInboxFolder
argo::iridium::xfloat::XFloatMailRetrievalService, 1173
operator<<
argo::iridium::xfloat, 39
argo::iridium::xfloat::XFloatMailMessage, 1124
util::log::Log, 330

operator>>
argo::iridium::xfloat, 40
argo::iridium::xfloat::XFloatMailMessage, 1124
argo::iridium::xfloat::XFloatMetadata, 1227
operator()
argo::iridium::xfloat::XFloatMailProcessingService, 1149
argo::iridium::xfloat::XFloatMailRetrievalService, 1174
argo::iridium::xfloat::XFloatPostProcessingService, 1238
argo::iridium::xfloat::XFloatTelemetryDecoderService, 1254

p_end_
util::xmsg::XAcousticWindow, 435
p_start_
util::xmsg::XAcousticWindow, 435
parent_instrument_
argo::iridium::xfloat::XFloatDiveData, 1042
park_start_time_
util::xmsg::XAlamoMissionConfiguration, 493
util::xmsg::XMfaMissionCfg, 1412
ParseMailMsg
argo::iridium::xfloat::XFloatMailMessage, 1090, 1091
ParseMailMsgBody
argo::iridium::xfloat::XFloatMailMessage, 1092
ParseMailMsgHeader
argo::iridium::xfloat::XFloatMailMessage, 1093
ParseMailMsgSbdAttachment
argo::iridium::xfloat::XFloatMailMessage, 1094
ParseProgramOptions
util::config::LocalConfiguration, 297
password_
argo::iridium::xfloat::XFloat, 970
argo::iridium::xfloat::XFloatDiveData, 1042
argo::iridium::xfloat::XFloatMailMessage, 1131
argo::iridium::xfloat::XFloatMailProcessingService, 1152
argo::iridium::xfloat::XFloatMailRetrievalService, 1179
argo::iridium::xfloat::XFloatMetadata, 1231
util::mysql::LocalMysqlConfiguration, 311
util::mysql::LocalPersistentEntity, 324
util::mysql::SqlTableDescription< T >, 405
util::xmsg::XGPS, 1304
pdt_
util::xmsg::XAlamoMissionConfiguration, 493
util::xmsg::XMfaMissionCfg, 1412
pfd_
util::xmsg::XAlamoMissionConfiguration, 493
util::xmsg::XMfaMissionCfg, 1412
pfm_
util::xmsg::XAlamoMissionConfiguration, 493
util::xmsg::XMfaMissionCfg, 1413
phase_code
util::xmsg::XPresCodeCounts, 1441
util::xmsg::XSensorTripletCodeCounts, 1550
phase_code_
util::xmsg::XPresCodeCounts, 1443
util::xmsg::XPumpCounts, 1524
util::xmsg::XSensorTripletCodeCounts, 1552
util::xmsg::XValveCounts, 1590
phase_code_name
util::xmsg::XPresCodeCounts, 1441
util::xmsg::XSensorTripletCodeCounts, 1550
phase_code_names_
util::xmsg::XPresCodeCounts, 1443
util::xmsg::XSensorTripletCodeCounts, 1552
phy_directory_path_
argo::iridium::xfloat::XFloatDiveData, 1042
phy_file_generation_is_enabled
argo::iridium::xfloat::XFloat, 943
phy_file_generation_is_enabled_
argo::iridium::xfloat::XFloat, 970
argo::iridium::xfloat::XFloatDiveData, 1043
phy_file_path
argo::iridium::xfloat::XFloatDiveData, 1028
phy_file_path_
argo::iridium::xfloat::XFloatDiveData, 1043
phy_file_updated_
argo::iridium::xfloat::XFloatDiveData, 1043
pid_file_path_
util::config::LocalConfiguration, 299
pns_
util::xmsg::XAlamoMissionConfiguration, 493
util::xmsg::XMfaMissionCfg, 1413
pop
util::thread::SynchronousQueue< T >, 410
port_
argo::iridium::xfloat::XFloat, 971
argo::iridium::xfloat::XFloatDiveData, 1043
argo::iridium::xfloat::XFloatMailMessage, 1131
argo::iridium::xfloat::XFloatMailProcessingService, 1153
argo::iridium::xfloat::XFloatMailRetrievalService, 1179
argo::iridium::xfloat::XFloatMetadata, 1231
util::mysql::LocalMysqlConfiguration, 312
util::mysql::LocalPersistentEntity, 324
util::mysql::SqlTableDescription< T >, 405
util::xmsg::XGPS, 1305
Position
util::position::Position, 344
position_datetime
argo::iridium::IridiumPosition, 272
argo::iridium::xfloat::XFloatMailMessage, 1095
argo::iridium::xfloat::XFloatMetadata, 1211
util::position::Position, 350
util::xmsg::XCompositeIridiumPosition, 593
util::xmsg::XGPS, 1288
position_datetime_
argo::iridium::IridiumPosition, 281
argo::iridium::xfloat::XFloatMailMessage, 1131
argo::iridium::xfloat::XFloatMetadata, 1232
util::position::Position, 357
util::xmsg::XCompositeIridiumPosition, 601
util::xmsg::XGPS, 1305
position_datetime_as_iso8601_string
argo::iridium::IridiumPosition, 272
argo::iridium::xfloat::XFloatMailMessage, 1095
argo::iridium::xfloat::XFloatMetadata, 1212
util::position::Position, 350
util::xmsg::XCompositeIridiumPosition, 593
util::xmsg::XGPS, 1288
position_is_valid
argo::iridium::IridiumPosition, 273
argo::iridium::xfloat::XFloatMailMessage, 1096
argo::iridium::xfloat::XFloatMetadata, 1212
util::position::Position, 351
util::xmsg::XCompositeIridiumPosition, 594
util::xmsg::XGPS, 1289
position_is_valid_
argo::iridium::IridiumPosition, 282
argo::iridium::xfloat::XFloatMailMessage, 1131
argo::iridium::xfloat::XFloatMetadata, 1232
util::position::Position, 357
util::xmsg::XCompositeIridiumPosition, 601
util::xmsg::XGPS, 1305
position_latitude
argo::iridium::IridiumPosition, 273
argo::iridium::xfloat::XFloatMailMessage, 1096
argo::iridium::xfloat::XFloatMetadata, 1213
util::position::Position, 351

util::xmsg::XCompositeIridiumPosition, 594
util::xmsg::XGPS, 1289
position_latitude_
argo::iridium::IridiumPosition, 282
argo::iridium::xfloat::XFloatMailMessage, 1132
argo::iridium::xfloat::XFloatMetadata, 1232
util::position::Position, 357
util::xmsg::XCompositeIridiumPosition, 601
util::xmsg::XGPS, 1305
position_longitude
argo::iridium::IridiumPosition, 274
argo::iridium::xfloat::XFloatMailMessage, 1097
argo::iridium::xfloat::XFloatMetadata, 1213
util::position::Position, 352
util::xmsg::XCompositeIridiumPosition, 595
util::xmsg::XGPS, 1290
position_longitude_
argo::iridium::IridiumPosition, 282
argo::iridium::xfloat::XFloatMailMessage, 1132
argo::iridium::xfloat::XFloatMetadata, 1232
util::position::Position, 357
util::xmsg::XCompositeIridiumPosition, 601
util::xmsg::XGPS, 1305
position_standard_error
util::xmsg::XCompositeIridiumPosition, 595
position_standard_error_
util::xmsg::XCompositeIridiumPosition, 601
position_variance
util::xmsg::XCompositeIridiumPosition, 596
position_variance_
util::xmsg::XCompositeIridiumPosition, 602
PositionUtils.h
UTIL_POS_INVALID_LAT, 2168
UTIL_POS_INVALID_LON, 2168
UTIL_POS_INVALID_STDERR, 2168
UTIL_POS_INVALID_VAR, 2168
postition_as_iso6709_string
argo::iridium::IridiumPosition, 274
argo::iridium::xfloat::XFloatMailMessage, 1097
argo::iridium::xfloat::XFloatMetadata, 1214
util::position::Position, 352
util::xmsg::XCompositeIridiumPosition, 596
util::xmsg::XGPS, 1290
PP_STR
Log.h, 2146
pres_messages_
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >, 126
util::xmsg::XCompositeDrift, 559
util::xmsg::XCompositeHiResProfile, 581
util::xmsg::XCompositeProfile, 619
pres_offs_
argo::iridium::xfloat::XFloatDiveData, 1043
pres_offs_num_dives_
argo::iridium::xfloat::XFloatDiveData, 1043
pres_offs_num_samples_
argo::iridium::xfloat::XFloatDiveData, 1044
pres_offs_valid_
argo::iridium::xfloat::XFloatDiveData, 1044
pres_offset_
util::xmsg::XAlamoBit, 443
PresMsgPtr_t
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >, 105
util::xmsg::XCompositeDrift, 545
util::xmsg::XCompositeHiResProfile, 566
util::xmsg::XCompositeProfile, 604
PresMsgVector_t
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >, 105
util::xmsg::XCompositeDrift, 545
util::xmsg::XCompositeHiResProfile, 566
util::xmsg::XCompositeProfile, 604
pressure_counts_
util::xmsg::XPresCodeCounts, 1443
util::xmsg::XPresCounts, 1447
util::xmsg::XPumpCounts, 1524
util::xmsg::XSensorTripletCodeCounts, 1552
util::xmsg::XValveCounts, 1591
pressure_offset_correction_enabled_list_file_
argo::iridium::xfloat::XFloatDiveData, 1044
pressure_offset_correction_enabled_netcdf_file_
argo::iridium::xfloat::XFloatDiveData, 1044
pressure_offset_correction_enabled_phy_file_
argo::iridium::xfloat::XFloatDiveData, 1044
pressure_offset_reading_
util::xmsg::XEngSurface, 880
PressureValue
util::xmsg::XPresCodeCounts, 1442
util::xmsg::XPresCounts, 1446
util::xmsg::XPumpCounts, 1522
util::xmsg::XSensorTripletCodeCounts, 1550
util::xmsg::XValveCounts, 1590
principal_investigators
argo::iridium::xfloat::XFloatMetadata, 1214
principal_investigators_
argo::iridium::xfloat::XFloatMetadata, 1232
principal_investigators_address
argo::iridium::xfloat::XFloatMetadata, 1215
principal_investigators_address_
argo::iridium::xfloat::XFloatMetadata, 1232
principal_investigators_originating_country
argo::iridium::xfloat::XFloatMetadata, 1215
principal_investigators_originating_country_
argo::iridium::xfloat::XFloatMetadata, 1232
ProcessDiveData
argo::iridium::xfloat::XFloatTelemetryDecoderService, 1254
processing_thread_propagation_enabled_
argo::iridium::xfloat::XFloatMailRetrievalService, 1179
ProcessMailMsg
argo::iridium::xfloat::XFloatMailProcessingService, 1149
PROG_DESC
LocalConfiguration.h, 1619
PROG_ID
LocalConfiguration.h, 1619
PROG_MAJOR_VERS
LocalConfiguration.h, 1619
PROG_MINOR_VERS
LocalConfiguration.h, 1619
PROG_NAME
LocalConfiguration.h, 1619
PROG_VERS
LocalConfiguration.h, 1620
PROG_XVERS
LocalConfiguration.h, 1620
protocol_
argo::iridium::xfloat::XFloat, 971
argo::iridium::xfloat::XFloatDiveData, 1044
argo::iridium::xfloat::XFloatMailMessage, 1132
argo::iridium::xfloat::XFloatMailProcessingService, 1153
argo::iridium::xfloat::XFloatMailRetrievalService, 1179
argo::iridium::xfloat::XFloatMetadata, 1233
util::mysql::LocalMysqlConfiguration, 312
util::mysql::LocalPersistentEntity, 324
util::mysql::SqlTableDescription< T >, 405
util::xmsg::XGPS, 1305

psa_
util::xmsg::XAlamoMissionConfiguration, 494
util::xmsg::XMfaMissionCfg, 1413
psi_
util::xmsg::XAlamoMissionConfiguration, 494
util::xmsg::XMfaMissionCfg, 1413
psk_error_code_
util::xmsg::XAcousticCacst, 420
pth_
util::xmsg::XAlamoMissionConfiguration, 494
util::xmsg::XMfaMissionCfg, 1413
pto_
util::xmsg::XAlamoMissionConfiguration, 494
util::xmsg::XMfaMissionCfg, 1413
pump_current_a
util::xmsg::XEngDescent, 773
util::xmsg::XPumpCounts, 1523
pump_current_ma_
util::xmsg::XEngDescent, 782
util::xmsg::XPumpCounts, 1524
pump_duration_sec_
util::xmsg::XEngDescent, 782
util::xmsg::XPumpCounts, 1524
pump_packing_format
util::xmsg::XPump, 1512
pump_packing_format_
util::xmsg::XPump, 1518
pump_run_duration_
util::xmsg::XEngAscent, 725
pump_voltage_cv_
util::xmsg::XPumpCounts, 1525
pump_voltage_v
util::xmsg::XPumpCounts, 1523
push
util::thread::SynchronousQueue< T >, 411

raw_directory_path_
argo::iridium::xfloat::XFloatMailMessage, 1132
raw_file_format_
argo::iridium::xfloat::XFloatMailMessage, 1132
raw_file_generation_is_enabled_
argo::iridium::xfloat::XFloatMailMessage, 1132
raw_file_local_path
argo::iridium::xfloat::XFloatMailMessage, 1098
raw_file_local_path_
argo::iridium::xfloat::XFloatMailMessage, 1133
raw_sensor_data
util::xmsg::AbstractXSensorData, 90
util::xmsg::AuxiliarySensorData, 143
util::xmsg::XDriftPressure, 646
util::xmsg::XDriftSalinity, 664
util::xmsg::XDriftTemperature, 683
util::xmsg::XHiResPressure, 1314
util::xmsg::XHiResSalinity, 1332
util::xmsg::XHiResTemperature, 1351
util::xmsg::XProfilePressure, 1454
util::xmsg::XProfileSalinity, 1472
util::xmsg::XProfileTemperature, 1491
raw_sensor_data_
util::xmsg::AbstractXSensorData, 101
util::xmsg::AuxiliarySensorData, 153
util::xmsg::XDriftPressure, 657
util::xmsg::XDriftSalinity, 675
util::xmsg::XDriftTemperature, 694
util::xmsg::XHiResPressure, 1325
util::xmsg::XHiResSalinity, 1343
util::xmsg::XHiResTemperature, 1362
util::xmsg::XProfilePressure, 1465
util::xmsg::XProfileSalinity, 1483
util::xmsg::XProfileTemperature, 1502
RecursiveLockable
util::thread::RecursiveLockable, 366
RegisterSignalHandlers
util::sig::SignalHandler, 371
ReleaseRuntimeLock
util::config::LocalConfiguration, 298
reprocess_by_inst_ids_
argo::iridium::xfloat::XFloatMailRetrievalService, 1179
Reset
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >, 118
util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >, 132
util::xmsg::XCompositeDrift, 553
util::xmsg::XCompositeHiResProfile, 574
util::xmsg::XCompositeProfile, 612
util::xmsg::XCompositeSurfaceData, 632
ResetAggregatedData
util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >, 132
util::xmsg::XCompositeSurfaceData, 633
ResetCachedDiveData
argo::iridium::xfloat::XFloat, 943
ResetData
argo::iridium::xfloat::XFloatDiveData, 1028
ResetDiveData
argo::iridium::xfloat::XFloat, 944

s1970
XDefinitions.h, 1770
s2000
XDefinitions.h, 1770
salinity_counts_
util::xmsg::XSensorTripletCodeCounts, 1553
SalinityValue
util::xmsg::XSensorTripletCodeCounts, 1551
saln_messages_
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >, 126
util::xmsg::XCompositeDrift, 559
util::xmsg::XCompositeHiResProfile, 581
util::xmsg::XCompositeProfile, 619
SalnMsgPtr_t
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >, 105
util::xmsg::XCompositeDrift, 545
util::xmsg::XCompositeHiResProfile, 567
util::xmsg::XCompositeProfile, 605
SalnMsgVector_t
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >, 105
util::xmsg::XCompositeDrift, 545
util::xmsg::XCompositeHiResProfile, 567
util::xmsg::XCompositeProfile, 605
Sample
util::xmsg::AbstractXSensorData, 90
util::xmsg::AuxiliarySensorData, 143
util::xmsg::XDriftPressure, 646
util::xmsg::XDriftSalinity, 664
util::xmsg::XDriftTemperature, 683
util::xmsg::XHiResPressure, 1314
util::xmsg::XHiResSalinity, 1332
util::xmsg::XHiResTemperature, 1351
util::xmsg::XProfilePressure, 1454
util::xmsg::XProfileSalinity, 1472
util::xmsg::XProfileTemperature, 1491
sample_mode_
util::xmsg::XWave, 1605
sample_rate_base_

util::xmsg::XWave, 1606
sample_rate_div_
util::xmsg::XWave, 1606
sample_rate_effective_
util::xmsg::XWave, 1606
SamplePres
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >, 119
util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >, 133
util::xmsg::XCompositeDrift, 554
util::xmsg::XCompositeHiResProfile, 575
util::xmsg::XCompositeProfile, 613
util::xmsg::XCompositeSurfaceData, 633
SamplePresAsFloat
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >, 119
util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >, 133
util::xmsg::XCompositeDrift, 554
util::xmsg::XCompositeHiResProfile, 575
util::xmsg::XCompositeProfile, 613
util::xmsg::XCompositeSurfaceData, 633
SampleSaln
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >, 120
util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >, 134
util::xmsg::XCompositeDrift, 554
util::xmsg::XCompositeHiResProfile, 576
util::xmsg::XCompositeProfile, 614
util::xmsg::XCompositeSurfaceData, 633
SampleSalnAsFloat
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >, 120
util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >, 134
util::xmsg::XCompositeDrift, 555
util::xmsg::XCompositeHiResProfile, 576
util::xmsg::XCompositeProfile, 614
util::xmsg::XCompositeSurfaceData, 634
SampleTemp
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >, 121
util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >, 135
util::xmsg::XCompositeDrift, 555
util::xmsg::XCompositeHiResProfile, 577
util::xmsg::XCompositeProfile, 615
util::xmsg::XCompositeSurfaceData, 634
SampleTempAsFloat
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >, 122
util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >, 135
util::xmsg::XCompositeDrift, 556
util::xmsg::XCompositeHiResProfile, 577
util::xmsg::XCompositeProfile, 615
util::xmsg::XCompositeSurfaceData, 635
SampleValue
util::xmsg::AbstractXSensorData, 91
util::xmsg::AuxiliarySensorData, 143
util::xmsg::XDriftPressure, 646
util::xmsg::XDriftSalinity, 665
util::xmsg::XDriftTemperature, 684
util::xmsg::XHiResPressure, 1314
util::xmsg::XHiResSalinity, 1333
util::xmsg::XHiResTemperature, 1352
util::xmsg::XProfilePressure, 1454
util::xmsg::XProfileSalinity, 1473
util::xmsg::XProfileTemperature, 1492
SampleValueAsPressure
util::xmsg::AbstractXSensorData, 91
util::xmsg::AuxiliarySensorData, 144
util::xmsg::XDriftPressure, 647
util::xmsg::XDriftSalinity, 665
util::xmsg::XDriftTemperature, 684
util::xmsg::XHiResPressure, 1315
util::xmsg::XHiResSalinity, 1333
util::xmsg::XHiResTemperature, 1352
util::xmsg::XProfilePressure, 1455
util::xmsg::XProfileSalinity, 1473
util::xmsg::XProfileTemperature, 1492
SampleValueAsSalinity
util::xmsg::AbstractXSensorData, 92
util::xmsg::AuxiliarySensorData, 144
util::xmsg::XDriftPressure, 647
util::xmsg::XDriftSalinity, 666
util::xmsg::XDriftTemperature, 685
util::xmsg::XHiResPressure, 1315
util::xmsg::XHiResSalinity, 1334
util::xmsg::XHiResTemperature, 1353
util::xmsg::XProfilePressure, 1455
util::xmsg::XProfileSalinity, 1474
util::xmsg::XProfileTemperature, 1493
SampleValueAsTemperature
util::xmsg::AbstractXSensorData, 92
util::xmsg::AuxiliarySensorData, 145
util::xmsg::XDriftPressure, 648
util::xmsg::XDriftSalinity, 666
util::xmsg::XDriftTemperature, 685
util::xmsg::XHiResPressure, 1316
util::xmsg::XHiResSalinity, 1334
util::xmsg::XHiResTemperature, 1353
util::xmsg::XProfilePressure, 1456
util::xmsg::XProfileSalinity, 1474
util::xmsg::XProfileTemperature, 1493
SaveLocalCacheToDatabase
argo::iridium::xfloat::XFloat, 944
SaveObjectToDatabase
argo::iridium::xfloat::XFloat, 944
argo::iridium::xfloat::XFloatDiveData, 1028
argo::iridium::xfloat::XFloatMailMessage, 1098
argo::iridium::xfloat::XFloatMetadata, 1215
util::mysql::LocalPersistentEntity, 323
util::mysql::SqlTableDescription< T >, 400
util::xmsg::XGPS, 1291
sbd_data
argo::iridium::xfloat::XFloatMailMessage, 1099
sbd_data_
argo::iridium::xfloat::XFloatMailMessage, 1133
sbd_data_is_valid
argo::iridium::xfloat::XFloatMailMessage, 1099
sbd_data_is_valid_
argo::iridium::xfloat::XFloatMailMessage, 1133
sbd_directory_path_
argo::iridium::xfloat::XFloatMailMessage, 1133
sbd_file_generation_is_enabled_
argo::iridium::xfloat::XFloatMailMessage, 1133
sbd_file_local_path
argo::iridium::xfloat::XFloatMailMessage, 1099
sbd_file_local_path_
argo::iridium::xfloat::XFloatMailMessage, 1134
sbd_file_name
argo::iridium::xfloat::XFloatMailMessage, 1100
sbd_file_name_
argo::iridium::xfloat::XFloatMailMessage, 1134
sbd_file_size_bytes

argo::iridium::xfloat::XFloatMailMessage, 1100
sbd_file_size_bytes_
argo::iridium::xfloat::XFloatMailMessage, 1134
sbd_file_valid_name_regex_
argo::iridium::xfloat::XFloatMailMessage, 1134
sbd_session_status_code
argo::iridium::xfloat::XFloatMailMessage, 1100
sbd_session_status_code_
argo::iridium::xfloat::XFloatMailMessage, 1134
sbd_session_status_string
argo::iridium::xfloat::XFloatMailMessage, 1101
sbd_session_status_string_
argo::iridium::xfloat::XFloatMailMessage, 1135
sbm_
util::xmsg::XAlamoMissionConfiguration, 494
util::xmsg::XMfaMissionCfg, 1413
sbt_
util::xmsg::XAlamoMissionConfiguration, 494
util::xmsg::XMfaMissionCfg, 1413
schema_
argo::iridium::xfloat::XFloat, 971
argo::iridium::xfloat::XFloatDiveData, 1045
argo::iridium::xfloat::XFloatMailMessage, 1135
argo::iridium::xfloat::XFloatMailProcessingService, 1153
argo::iridium::xfloat::XFloatMailRetrievalService, 1180
argo::iridium::xfloat::XFloatMetadata, 1233
util::mysql::LocalMysqlConfiguration, 312
util::mysql::LocalPersistentEntity, 324
util::mysql::SqlTableDescription< T >, 405
util::xmsg::XGPS, 1305
sdc_
util::xmsg::XAlamoMissionConfiguration, 494
util::xmsg::XMfaMissionCfg, 1413
sds_
util::xmsg::XAlamoMissionConfiguration, 494
util::xmsg::XMfaMissionCfg, 1414
seek_to_drift_pressure_interval_duration
util::xmsg::XArgo, 516
seek_to_drift_pressure_interval_duration_
util::xmsg::XArgo, 537
segment_len_
util::xmsg::XWave, 1606
segment_len_pow_
util::xmsg::XWave, 1606
select_statement_
util::mysql::SqlTableDescription< T >, 405
SelectAllObjectsFromDatabase
argo::iridium::xfloat::XFloat, 944
argo::iridium::xfloat::XFloatMailMessage, 1101
util::mysql::SqlTableDescription< T >, 400
util::xmsg::XGPS, 1291
SelectObjectFromDatabase
argo::iridium::xfloat::XFloat, 945
argo::iridium::xfloat::XFloatDiveData, 1029
argo::iridium::xfloat::XFloatMailMessage, 1101
util::mysql::SqlTableDescription< T >, 401
util::xmsg::XGPS, 1292
SelectObjectsFromDatabase
argo::iridium::xfloat::XFloat, 945
argo::iridium::xfloat::XFloatDiveData, 1029
argo::iridium::xfloat::XFloatMailMessage, 1103
util::mysql::SqlTableDescription< T >, 402
util::xmsg::XGPS, 1292
SelectValueFromDatabase
argo::iridium::xfloat::XFloat, 946
argo::iridium::xfloat::XFloatDiveData, 1030
argo::iridium::xfloat::XFloatMailMessage, 1103
util::mysql::SqlTableDescription< T >, 403
util::xmsg::XGPS, 1293
sensor_data
util::xmsg::AbstractXFallRise, 74
util::xmsg::XFall, 892
util::xmsg::XRise, 1536
util::xmsg::XSurfaceData, 1564
sensor_data_
util::xmsg::AbstractXFallRise, 83
util::xmsg::XFall, 901
util::xmsg::XRise, 1545
util::xmsg::XSurfaceData, 1573
sensor_data_format
util::xmsg::AbstractXSensorData, 93
util::xmsg::AuxiliarySensorData, 145
util::xmsg::XDriftPressure, 648
util::xmsg::XDriftSalinity, 667
util::xmsg::XDriftTemperature, 686
util::xmsg::XHiResPressure, 1316
util::xmsg::XHiResSalinity, 1335
util::xmsg::XHiResTemperature, 1354
util::xmsg::XProfilePressure, 1456
util::xmsg::XProfileSalinity, 1475
util::xmsg::XProfileTemperature, 1494
sensor_data_format_
util::xmsg::AbstractXSensorData, 102
util::xmsg::AuxiliarySensorData, 154
util::xmsg::XDriftPressure, 657
util::xmsg::XDriftSalinity, 676
util::xmsg::XDriftTemperature, 695
util::xmsg::XHiResPressure, 1325
util::xmsg::XHiResSalinity, 1344
util::xmsg::XHiResTemperature, 1363
util::xmsg::XProfilePressure, 1465
util::xmsg::XProfileSalinity, 1484
util::xmsg::XProfileTemperature, 1503
sensor_type_
util::xmsg::XAlamoBit2, 457
util::xmsg::XAlamoMissionConfiguration, 494
util::xmsg::XMfaMissionCfg, 1414
SensorGainsAndOffsetsAreKnown
argo::iridium::xfloat::XFloatDiveData, 1030
set_
util::xmsg::XAlamoMissionConfiguration, 495
util::xmsg::XMfaMissionCfg, 1414
set_aggregated_data_valid
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >, 122
util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >, 136
util::xmsg::XCompositeDrift, 556
util::xmsg::XCompositeHiResProfile, 578
util::xmsg::XCompositeProfile, 616
util::xmsg::XCompositeSurfaceData, 635
set_alamo_bit2_msg
util::xmsg::XEngBIT, 732
util::xmsg::XEngBITFail, 744
util::xmsg::XEngBITPass, 756
set_alamo_bit_msg
util::xmsg::XEngBIT, 733
util::xmsg::XEngBITFail, 745
util::xmsg::XEngBITPass, 757
set_argo_instrument_type
argo::iridium::xfloat::XFloat, 946
argo::iridium::xfloat::XFloatDiveData, 1031
util::xmsg::XArgo, 516
set_ascent_state_start_time
util::xmsg::XArgo, 516
set_bit_passed
util::xmsg::XAlamoBit2, 452
set_bit_status_code
util::xmsg::XAlamoBit, 441
util::xmsg::XAlamoBit2, 452
set_bit_version

util::xmsg::XAlamoBit, 441
util::xmsg::XAlamoBit2, 452
set_column_index
util::mysql::Column_bool< C >, 158
util::mysql::Column_char< C >, 165
util::mysql::Column_double< C >, 171
util::mysql::Column_filesystem_path< C >, 178
util::mysql::Column_float< C >, 185
util::mysql::Column_int< C >, 191
util::mysql::Column_int16_t< C >, 198
util::mysql::Column_long< C >, 204
util::mysql::Column_posix_time< C >, 211
util::mysql::Column_string< C >, 218
util::mysql::Column_uint16_t< C >, 225
util::mysql::Column_uint8_t< C >, 231
util::mysql::Column_vector_uint8_t< C >, 238
util::mysql::SqlTableColumnDescription< C, T >, 376
util::mysql::SqlTableColumnDescriptionBase, 381
set_comment
argo::iridium::xfloat::XFloatMetadata, 1215
set_data_pack_format
util::xmsg::AbstractXFallRise, 75
util::xmsg::XFall, 892
util::xmsg::XRise, 1537
util::xmsg::XSurfaceData, 1565
set_date_time
util::xmsg::XEngIce::XEngIceErrorCode, 843
util::xmsg::XEngSurface::XEngErrorCode, 819
set_deployer
argo::iridium::xfloat::XFloatMetadata, 1216
set_deployer_address
argo::iridium::xfloat::XFloatMetadata, 1216
set_deployment_cruise_id
argo::iridium::xfloat::XFloatMetadata, 1216
set_deployment_datetime
argo::iridium::xfloat::XFloatMetadata, 1217
set_deployment_datetime_status
argo::iridium::xfloat::XFloatMetadata, 1217
set_deployment_latitude
argo::iridium::xfloat::XFloatMetadata, 1217
set_deployment_longitude
argo::iridium::xfloat::XFloatMetadata, 1217
set_deployment_platform
argo::iridium::xfloat::XFloatMetadata, 1219
set_deployment_position_status
argo::iridium::xfloat::XFloatMetadata, 1219
set_deployment_type
argo::iridium::xfloat::XFloatMetadata, 1219
set_descent_state_start_time
util::xmsg::XArgo, 517
set_drift_state_start_time
util::xmsg::XArgo, 517
set_engineering_version
util::xmsg::XAlamoEngineering, 466
util::xmsg::XEngAscent, 717
util::xmsg::XEngDescent, 774
util::xmsg::XEngDrift, 805
util::xmsg::XEngIce, 830
util::xmsg::XEngSurface, 869
set_error_code
util::xmsg::XEngIce::XEngIceErrorCode, 843
util::xmsg::XEngSurface::XEngErrorCode, 819
set_float_software_version
util::xmsg::XAlamoBit, 442
util::xmsg::XAlamoBit2, 453
set_fw_version
argo::iridium::xfloat::XFloat, 947
set_gain_pressure
argo::iridium::xfloat::XFloat, 947
util::xmsg::XArgo, 517
set_gain_pressure_surface
argo::iridium::xfloat::XFloat, 948
util::xmsg::XArgo, 518
set_gain_salinity
argo::iridium::xfloat::XFloat, 948
util::xmsg::XArgo, 518
set_gain_salinity_surface
argo::iridium::xfloat::XFloat, 948
util::xmsg::XArgo, 518
set_gain_temperature
argo::iridium::xfloat::XFloat, 949
util::xmsg::XArgo, 519
set_gain_temperature_surface
argo::iridium::xfloat::XFloat, 949
util::xmsg::XArgo, 519
set_gps_position_datetime
argo::iridium::IridiumPosition, 275
argo::iridium::xfloat::XFloatMailMessage, 1104
argo::iridium::xfloat::XFloatMetadata, 1219
util::position::Position, 353
util::xmsg::XCompositeIridiumPosition, 596
util::xmsg::XGPS, 1294
set_id_dac
argo::iridium::Iridium, 256
argo::iridium::xfloat::XFloat, 949
argo::iridium::xfloat::XFloatDiveData, 1031
argo::iridium::xfloat::XFloatMailMessage, 1105
argo::iridium::xfloat::XFloatMetadata, 1220
set_id_dive
argo::iridium::xfloat::XFloatDiveData, 1031
argo::iridium::xfloat::XFloatMailMessage, 1105
util::xmsg::AbstractXFallRise, 75
util::xmsg::AbstractXSensorData, 93
util::xmsg::AuxiliarySensorData, 146
util::xmsg::XAcousticPosition, 427
util::xmsg::XAlamoEngineering, 467
util::xmsg::XArgo, 519
util::xmsg::XDriftPressure, 649
util::xmsg::XDriftSalinity, 667
util::xmsg::XDriftTemperature, 686
util::xmsg::XEngAbort, 702
util::xmsg::XEngAscent, 718
util::xmsg::XEngBIT, 733
util::xmsg::XEngBITFail, 745
util::xmsg::XEngBITPass, 757
util::xmsg::XEngDescent, 774
util::xmsg::XEngDiagnostic, 790
util::xmsg::XEngDrift, 806
util::xmsg::XEngIce, 831
util::xmsg::XEngStandard, 851
util::xmsg::XEngSurface, 870
util::xmsg::XFall, 893
util::xmsg::XGPS, 1294
util::xmsg::XHiResPressure, 1317
util::xmsg::XHiResSalinity, 1335
util::xmsg::XHiResTemperature, 1354
util::xmsg::XMessage, 1369
util::xmsg::XMessageIndexable, 1380
util::xmsg::XMissionCmdEcho, 1421
util::xmsg::XMissionConfiguration, 1431
util::xmsg::XProfilePressure, 1457
util::xmsg::XProfileSalinity, 1475
util::xmsg::XProfileTemperature, 1494
util::xmsg::XPump, 1512
util::xmsg::XRise, 1537
util::xmsg::XSurfaceData, 1565
util::xmsg::XValve, 1582
util::xmsg::XWave, 1598
set_id_imei
argo::iridium::Iridium, 256
argo::iridium::xfloat::XFloat, 950
argo::iridium::xfloat::XFloatDiveData, 1032
argo::iridium::xfloat::XFloatMailMessage, 1106
argo::iridium::xfloat::XFloatMetadata, 1220

set_id_instrument
argo::iridium::Iridium, 257
argo::iridium::xfloat::XFloat, 950
argo::iridium::xfloat::XFloatDiveData, 1032
argo::iridium::xfloat::XFloatMailMessage, 1106
argo::iridium::xfloat::XFloatMetadata, 1221
util::xmsg::AbstractXFallRise, 76
util::xmsg::AbstractXSensorData, 93
util::xmsg::AuxiliarySensorData, 146
util::xmsg::XAcousticPosition, 428
util::xmsg::XAlamoEngineering, 467
util::xmsg::XArgo, 520
util::xmsg::XDriftPressure, 649
util::xmsg::XDriftSalinity, 667
util::xmsg::XDriftTemperature, 686
util::xmsg::XEngAbort, 702
util::xmsg::XEngAscent, 718
util::xmsg::XEngBIT, 733
util::xmsg::XEngBITFail, 745
util::xmsg::XEngBITPass, 757
util::xmsg::XEngDescent, 775
util::xmsg::XEngDiagnostic, 790
util::xmsg::XEngDrift, 806
util::xmsg::XEngIce, 831
util::xmsg::XEngStandard, 851
util::xmsg::XEngSurface, 870
util::xmsg::XFall, 893
util::xmsg::XGPS, 1295
util::xmsg::XHiResPressure, 1317
util::xmsg::XHiResSalinity, 1335
util::xmsg::XHiResTemperature, 1354
util::xmsg::XMessage, 1370
util::xmsg::XMessageIndexable, 1380
util::xmsg::XMissionCmdEcho, 1421
util::xmsg::XMissionConfiguration, 1432
util::xmsg::XProfilePressure, 1457
util::xmsg::XProfileSalinity, 1475
util::xmsg::XProfileTemperature, 1494
util::xmsg::XPump, 1513
util::xmsg::XRise, 1538
util::xmsg::XSurfaceData, 1566
util::xmsg::XValve, 1582
util::xmsg::XWave, 1599
set_id_msg
argo::iridium::xfloat::XFloatMailMessage, 1107
set_id_msg_uid
argo::iridium::xfloat::XFloatMailMessage, 1107
set_id_sbd_momsn
argo::iridium::xfloat::XFloatMailMessage, 1108
set_id_sbd_mtmsn
argo::iridium::xfloat::XFloatMailMessage, 1108
set_id_sbd_packet
argo::iridium::xfloat::XFloatMailMessage, 1108
set_id_wmo
argo::iridium::Iridium, 257
argo::iridium::xfloat::XFloat, 951
argo::iridium::xfloat::XFloatDiveData, 1033
argo::iridium::xfloat::XFloatMailMessage, 1109
argo::iridium::xfloat::XFloatMetadata, 1221
set_id_xmsg_type
util::xmsg::AbstractXFallRise, 76
util::xmsg::AbstractXSensorData, 94
util::xmsg::AuxiliarySensorData, 146
util::xmsg::XAcousticPosition, 428
util::xmsg::XAlamoEngineering, 468
util::xmsg::XArgo, 520
util::xmsg::XDriftPressure, 649
util::xmsg::XDriftSalinity, 668
util::xmsg::XDriftTemperature, 687
util::xmsg::XEngAbort, 702
util::xmsg::XEngAscent, 719
util::xmsg::XEngBIT, 733
util::xmsg::XEngBITFail, 745
util::xmsg::XEngBITPass, 757
util::xmsg::XEngDescent, 775
util::xmsg::XEngDiagnostic, 790
util::xmsg::XEngDrift, 807
util::xmsg::XEngIce, 832
util::xmsg::XEngStandard, 851
util::xmsg::XEngSurface, 871
util::xmsg::XFall, 894
util::xmsg::XGPS, 1295
util::xmsg::XHiResPressure, 1317
util::xmsg::XHiResSalinity, 1336
util::xmsg::XHiResTemperature, 1355
util::xmsg::XMessage, 1370
util::xmsg::XMessageIndexable, 1380
util::xmsg::XMissionCmdEcho, 1421
util::xmsg::XMissionConfiguration, 1432
util::xmsg::XProfilePressure, 1457
util::xmsg::XProfileSalinity, 1476
util::xmsg::XProfileTemperature, 1495
util::xmsg::XPump, 1513
util::xmsg::XRise, 1538
util::xmsg::XSurfaceData, 1566
util::xmsg::XValve, 1582
util::xmsg::XWave, 1599
set_intermediate_listing_file_path
argo::iridium::xfloat::XFloatDiveData, 1033
set_iridium_position_cep_radius
argo::iridium::IridiumPosition, 276
argo::iridium::xfloat::XFloatMailMessage, 1109
set_iridium_position_datetime
argo::iridium::IridiumPosition, 276
argo::iridium::xfloat::XFloatMailMessage, 1110
set_iridium_position_is_valid
argo::iridium::IridiumPosition, 277
argo::iridium::xfloat::XFloatMailMessage, 1110
set_iridium_position_latitude
argo::iridium::IridiumPosition, 277
argo::iridium::xfloat::XFloatMailMessage, 1111
set_iridium_position_longitude
argo::iridium::IridiumPosition, 277
argo::iridium::xfloat::XFloatMailMessage, 1111
set_local_path
argo::iridium::xfloat::XFloatMetadata, 1222
set_log_reporting_level
util::log::Log, 331
set_manufacturer
argo::iridium::xfloat::XFloatMetadata, 1222
set_max_ascent_duration
util::xmsg::XArgo, 521
set_max_fall_to_drift_duration
util::xmsg::XArgo, 522
set_max_fall_to_profle_duration
util::xmsg::XArgo, 522
set_max_surface_duration
util::xmsg::XArgo, 522
set_metadata
argo::iridium::xfloat::XFloat, 951
set_mission_phase
util::xmsg::XArgo, 523
set_msg_body_is_valid
argo::iridium::xfloat::XFloatMailMessage, 1112
set_msg_body_text
argo::iridium::xfloat::XFloatMailMessage, 1112
set_msg_header_datetime
argo::iridium::xfloat::XFloatMailMessage, 1113
set_msg_header_from
argo::iridium::xfloat::XFloatMailMessage, 1113
set_msg_header_is_valid
argo::iridium::xfloat::XFloatMailMessage, 1113
set_msg_header_raw_text
argo::iridium::xfloat::XFloatMailMessage, 1114

set_msg_header_subject
argo::iridium::xfloat::XFloatMailMessage, 1114
set_msg_is_valid
argo::iridium::xfloat::XFloatMailMessage, 1114
set_msg_raw_text
argo::iridium::xfloat::XFloatMailMessage, 1115
set_num_seeks_to_drift_pressure
util::xmsg::XArgo, 523
set_offset_pressure
argo::iridium::xfloat::XFloat, 952
util::xmsg::XArgo, 523
set_offset_pressure_surface
argo::iridium::xfloat::XFloat, 952
util::xmsg::XArgo, 524
set_offset_salinity
argo::iridium::xfloat::XFloat, 953
util::xmsg::XArgo, 524
set_offset_salinity_surface
argo::iridium::xfloat::XFloat, 953
util::xmsg::XArgo, 524
set_offset_temperature
argo::iridium::xfloat::XFloat, 953
util::xmsg::XArgo, 525
set_offset_temperature_surface
argo::iridium::xfloat::XFloat, 954
util::xmsg::XArgo, 525
set_phy_file_generation_is_enabled
argo::iridium::xfloat::XFloat, 954
set_phy_file_path
argo::iridium::xfloat::XFloatDiveData, 1034
set_position_datetime
argo::iridium::IridiumPosition, 278
argo::iridium::xfloat::XFloatMailMessage, 1115
argo::iridium::xfloat::XFloatMetadata, 1223
util::position::Position, 354
util::xmsg::XCompositeIridiumPosition, 597
util::xmsg::XGPS, 1296
set_position_is_valid
argo::iridium::IridiumPosition, 278
argo::iridium::xfloat::XFloatMailMessage, 1115
argo::iridium::xfloat::XFloatMetadata, 1223
util::position::Position, 354
util::xmsg::XCompositeIridiumPosition, 597
util::xmsg::XGPS, 1296
set_position_latitude
argo::iridium::IridiumPosition, 279
argo::iridium::xfloat::XFloatMailMessage, 1116
argo::iridium::xfloat::XFloatMetadata, 1223
util::position::Position, 354
util::xmsg::XCompositeIridiumPosition, 598
util::xmsg::XGPS, 1297
set_position_longitude
argo::iridium::IridiumPosition, 279
argo::iridium::xfloat::XFloatMailMessage, 1116
argo::iridium::xfloat::XFloatMetadata, 1224
util::position::Position, 356
util::xmsg::XCompositeIridiumPosition, 598
util::xmsg::XGPS, 1297
set_position_standard_error
util::xmsg::XCompositeIridiumPosition, 599
set_position_variance
util::xmsg::XCompositeIridiumPosition, 599
set_principal_investigators
argo::iridium::xfloat::XFloatMetadata, 1225
set_principal_investigators_address
argo::iridium::xfloat::XFloatMetadata, 1225
set_principal_investigators_originating_country
argo::iridium::xfloat::XFloatMetadata, 1225
set_raw_file_local_path
argo::iridium::xfloat::XFloatMailMessage, 1117
set_raw_sensor_data
util::xmsg::AbstractXSensorData, 95
util::xmsg::AuxiliarySensorData, 147
util::xmsg::XDriftPressure, 650
util::xmsg::XDriftSalinity, 669
util::xmsg::XDriftTemperature, 688
util::xmsg::XHiResPressure, 1318
util::xmsg::XHiResSalinity, 1337
util::xmsg::XHiResTemperature, 1356
util::xmsg::XProfilePressure, 1458
util::xmsg::XProfileSalinity, 1477
util::xmsg::XProfileTemperature, 1496
set_sbd_data
argo::iridium::xfloat::XFloatMailMessage, 1117
set_sbd_data_is_valid
argo::iridium::xfloat::XFloatMailMessage, 1119
set_sbd_file_local_path
argo::iridium::xfloat::XFloatMailMessage, 1119
set_sbd_file_name
argo::iridium::xfloat::XFloatMailMessage, 1119
set_sbd_file_size_bytes
argo::iridium::xfloat::XFloatMailMessage, 1120
set_sbd_session_status_code
argo::iridium::xfloat::XFloatMailMessage, 1120
set_sbd_session_status_string
argo::iridium::xfloat::XFloatMailMessage, 1120
set_seek_to_drift_pressure_interval_duration
util::xmsg::XArgo, 525
set_sensor_data_format
util::xmsg::AbstractXSensorData, 95
util::xmsg::AuxiliarySensorData, 148
util::xmsg::XDriftPressure, 651
util::xmsg::XDriftSalinity, 669
util::xmsg::XDriftTemperature, 688
util::xmsg::XHiResPressure, 1319
util::xmsg::XHiResSalinity, 1337
util::xmsg::XHiResTemperature, 1356
util::xmsg::XProfilePressure, 1459
util::xmsg::XProfileSalinity, 1477
util::xmsg::XProfileTemperature, 1496
set_start_of_mission_datetime
argo::iridium::xfloat::XFloat, 954
set_start_time
util::xmsg::AbstractXFallRise, 77
util::xmsg::XAlamoEngineering, 469
util::xmsg::XEngAscent, 720
util::xmsg::XEngDescent, 776
util::xmsg::XEngDrift, 808
util::xmsg::XEngIce, 833
util::xmsg::XEngSurface, 872
util::xmsg::XFall, 895
util::xmsg::XRise, 1539
util::xmsg::XSurfaceData, 1567
set_surface_state_start_time
util::xmsg::XArgo, 526
set_target_ascent_rate
util::xmsg::XArgo, 526
set_target_drift_duration
util::xmsg::XArgo, 526
set_target_drift_pressure
util::xmsg::XArgo, 527
set_target_profile_pressure
util::xmsg::XArgo, 527
set_termination_signaled
argo::iridium::xfloat::XFloatMailProcessingService, 1150
argo::iridium::xfloat::XFloatMailRetrievalService, 1175
argo::iridium::xfloat::XFloatTelemetryDecoderService, 1255
set_total_satellite_comm_time_seconds
util::xmsg::XCompositeIridiumPosition, 600
set_wmo_instrument_type
argo::iridium::xfloat::XFloatMetadata, 1226

set_wmo_recorder_type
argo::iridium::xfloat::XFloatMetadata, 1226
set_x_version_major
util::xmsg::XArgo, 527
set_x_version_minor
util::xmsg::XArgo, 528
set_xmsg_index
util::xmsg::AbstractXFallRise, 78
util::xmsg::AbstractXSensorData, 96
util::xmsg::AuxiliarySensorData, 148
util::xmsg::XDriftPressure, 651
util::xmsg::XDriftSalinity, 670
util::xmsg::XDriftTemperature, 689
util::xmsg::XFall, 896
util::xmsg::XHiResPressure, 1319
util::xmsg::XHiResSalinity, 1338
util::xmsg::XHiResTemperature, 1357
util::xmsg::XMessageIndexable, 1381
util::xmsg::XMissionConfiguration, 1433
util::xmsg::XProfilePressure, 1459
util::xmsg::XProfileSalinity, 1478
util::xmsg::XProfileTemperature, 1497
util::xmsg::XPump, 1514
util::xmsg::XRise, 1540
util::xmsg::XSurfaceData, 1568
util::xmsg::XValve, 1583
util::xmsg::XWave, 1600
set_xmsg_size_bytes
util::xmsg::AbstractXFallRise, 78
util::xmsg::AbstractXSensorData, 97
util::xmsg::AuxiliarySensorData, 149
util::xmsg::XAcousticPosition, 429
util::xmsg::XAlamoEngineering, 470
util::xmsg::XArgo, 528
util::xmsg::XDriftPressure, 652
util::xmsg::XDriftSalinity, 671
util::xmsg::XDriftTemperature, 690
util::xmsg::XEngAbort, 703
util::xmsg::XEngAscent, 721
util::xmsg::XEngBIT, 734
util::xmsg::XEngBITFail, 746
util::xmsg::XEngBITPass, 758
util::xmsg::XEngDescent, 777
util::xmsg::XEngDiagnostic, 791
util::xmsg::XEngDrift, 809
util::xmsg::XEngIce, 834
util::xmsg::XEngStandard, 852
util::xmsg::XEngSurface, 873
util::xmsg::XFall, 896
util::xmsg::XGPS, 1298
util::xmsg::XHiResPressure, 1320
util::xmsg::XHiResSalinity, 1339
util::xmsg::XHiResTemperature, 1358
util::xmsg::XMessage, 1371
util::xmsg::XMessageIndexable, 1382
util::xmsg::XMissionCmdEcho, 1422
util::xmsg::XMissionConfiguration, 1434
util::xmsg::XProfilePressure, 1460
util::xmsg::XProfileSalinity, 1479
util::xmsg::XProfileTemperature, 1498
util::xmsg::XPump, 1515
util::xmsg::XRise, 1540
util::xmsg::XSurfaceData, 1568
util::xmsg::XValve, 1584
util::xmsg::XWave, 1601
set_xmsg_version_major
argo::iridium::xfloat::XFloat, 955
set_xmsg_version_minor
argo::iridium::xfloat::XFloat, 955
SetAllDefaultKmlSyles
argo::iridium::xfloat::XFloat, 955
argo::iridium::xfloat::XFloatPostProcessingService, 1239
util::kml::KmlUtil, 286
util::xmsg::XGPS, 1299
SetDefaultKmlStyle
argo::iridium::xfloat::XFloat, 956
argo::iridium::xfloat::XFloatPostProcessingService, 1240
util::kml::KmlUtil, 287
util::xmsg::XGPS, 1300
SetObjectValue
util::mysql::Column_bool< C >, 159
util::mysql::Column_char< C >, 165
util::mysql::Column_double< C >, 172
util::mysql::Column_filesystem_path< C >, 179
util::mysql::Column_float< C >, 185
util::mysql::Column_int< C >, 192
util::mysql::Column_int16_t< C >, 198
util::mysql::Column_long< C >, 205
util::mysql::Column_posix_time< C >, 212
util::mysql::Column_string< C >, 219
util::mysql::Column_uint16_t< C >, 225
util::mysql::Column_uint8_t< C >, 232
util::mysql::Column_vector_uint8_t< C >, 238
util::mysql::SqlTableColumnDescription< C, T >, 377
SetObjectValueFromResultSet
util::mysql::Column_bool< C >, 159
util::mysql::Column_char< C >, 165
util::mysql::Column_double< C >, 172
util::mysql::Column_filesystem_path< C >, 179
util::mysql::Column_float< C >, 185
util::mysql::Column_int< C >, 192
util::mysql::Column_int16_t< C >, 198
util::mysql::Column_long< C >, 205
util::mysql::Column_posix_time< C >, 212
util::mysql::Column_string< C >, 219
util::mysql::Column_uint16_t< C >, 225
util::mysql::Column_uint8_t< C >, 232
util::mysql::Column_vector_uint8_t< C >, 239
SetPreparedStatementValueFromObject
util::mysql::Column_bool< C >, 159
util::mysql::Column_char< C >, 166
util::mysql::Column_double< C >, 172
util::mysql::Column_filesystem_path< C >, 179
util::mysql::Column_float< C >, 186
util::mysql::Column_int< C >, 192
util::mysql::Column_int16_t< C >, 199
util::mysql::Column_long< C >, 205
util::mysql::Column_posix_time< C >, 212
util::mysql::Column_string< C >, 219
util::mysql::Column_uint16_t< C >, 226
util::mysql::Column_uint8_t< C >, 232
util::mysql::Column_vector_uint8_t< C >, 239
sev_logger_mt
util::log, 44
sfn_
util::xmsg::XAlamoMissionConfiguration, 495
util::xmsg::XMfaMissionCfg, 1414
sgt_
util::xmsg::XAlamoMissionConfiguration, 495
util::xmsg::XMfaMissionCfg, 1414
signal_to_noise_max_
util::xmsg::XGPS, 1306
signal_to_noise_mean_
util::xmsg::XGPS, 1306
signal_to_noise_min_
util::xmsg::XGPS, 1306
size
util::thread::SynchronousQueue< T >, 411
sleep_seconds_if_no_new_msgs_
argo::iridium::xfloat::XFloatMailRetrievalService, 1180

snr_in_
util::xmsg::XAcousticCacst, 420
snr_out_
util::xmsg::XAcousticCacst, 420
SPLASH
util::config::LocalConfiguration, 299
SPLASH_LIS
util::config::LocalConfiguration, 299
splash_lis
util::config::LocalConfiguration, 298
sql_ci16
SqlTableColumnDescription.h, 2177
sql_colb
SqlTableColumnDescription.h, 2177
sql_cold
SqlTableColumnDescription.h, 2177
sql_colf
SqlTableColumnDescription.h, 2177
sql_coli
SqlTableColumnDescription.h, 2177
sql_coll
SqlTableColumnDescription.h, 2178
sql_colp
SqlTableColumnDescription.h, 2178
sql_cols
SqlTableColumnDescription.h, 2178
sql_colt
SqlTableColumnDescription.h, 2178
sql_colv
SqlTableColumnDescription.h, 2178
sql_connection_
argo::iridium::xfloat::XFloat, 971
argo::iridium::xfloat::XFloatDiveData, 1045
argo::iridium::xfloat::XFloatMailMessage, 1135
argo::iridium::xfloat::XFloatMailProcessingService, 1153
argo::iridium::xfloat::XFloatMailRetrievalService, 1180
argo::iridium::xfloat::XFloatMetadata, 1233
argo::iridium::xfloat::XFloatTelemetryDecoderService, 1257
util::mysql::LocalMysqlConfiguration, 312
util::mysql::LocalPersistentEntity, 324
util::mysql::MysqlUtils, 342
util::mysql::SqlTableDescription< T >, 405
util::xmsg::XGPS, 1306
sql_cou8
SqlTableColumnDescription.h, 2178
sql_cu16
SqlTableColumnDescription.h, 2178
sql_driver_
argo::iridium::xfloat::XFloat, 971
argo::iridium::xfloat::XFloatDiveData, 1045
argo::iridium::xfloat::XFloatMailMessage, 1135
argo::iridium::xfloat::XFloatMailProcessingService, 1153
argo::iridium::xfloat::XFloatMailRetrievalService, 1180
argo::iridium::xfloat::XFloatMetadata, 1233
argo::iridium::xfloat::XFloatTelemetryDecoderService, 1257
util::mysql::LocalMysqlConfiguration, 312
util::mysql::LocalPersistentEntity, 324
util::mysql::MysqlUtils, 342
util::mysql::SqlTableDescription< T >, 405
util::xmsg::XGPS, 1306
sql_table_description_
argo::iridium::xfloat::XFloat, 971
argo::iridium::xfloat::XFloatDiveData, 1045
argo::iridium::xfloat::XFloatMailMessage, 1135
argo::iridium::xfloat::XFloatMetadata, 1233
SqlTableColumnDescription
util::mysql::SqlTableColumnDescription< C, T >, 373, 374
SqlTableColumnDescription.h
sql_ci16, 2177
sql_colb, 2177
sql_cold, 2177
sql_colf, 2177
sql_coli, 2177
sql_coll, 2178
sql_colp, 2178
sql_cols, 2178
sql_colt, 2178
sql_colv, 2178
sql_cou8, 2178
sql_cu16, 2178
SqlTableColumnDescriptionBase
util::mysql::SqlTableColumnDescriptionBase, 379
SqlTableDescription
util::mysql::SqlTableDescription< T >, 386
src_
util::xmsg::XAcousticCacst, 420
ssi_
util::xmsg::XAlamoMissionConfiguration, 495
util::xmsg::XMfaMissionCfg, 1414
stage1_MailRetrievalThread.cpp, 2093
stage1_MailRetrievalThread.h, 2094
stage2_MailProcessingThread.cpp, 2099
stage2_MailProcessingThread.h, 2101
stage3_TelemetryDecoderThread.cpp, 2104
stage3_TelemetryDecoderThread.h, 2105
DBG_X_DECODE, 2106
stage4b_PostProcessingThread.cpp, 2113
stage4b_PostProcessingThread.h, 2114
start_of_fall_to_profile_pressure_counts_
util::xmsg::XEngDrift, 813
start_of_fall_to_profile_pressure_time_
util::xmsg::XEngDrift, 813
start_of_mission_datetime
argo::iridium::xfloat::XFloat, 957
start_of_mission_datetime_
argo::iridium::xfloat::XFloat, 972
start_time
util::xmsg::AbstractXFallRise, 80
util::xmsg::XAlamoEngineering, 471
util::xmsg::XEngAscent, 722
util::xmsg::XEngDescent, 778
util::xmsg::XEngDrift, 810
util::xmsg::XEngIce, 835
util::xmsg::XEngSurface, 874
util::xmsg::XFall, 898
util::xmsg::XRise, 1542
util::xmsg::XSurfaceData, 1570
start_time_
util::xmsg::AbstractXFallRise, 83
util::xmsg::XAlamoEngineering, 474
util::xmsg::XEngAscent, 725
util::xmsg::XEngDescent, 783
util::xmsg::XEngDrift, 813
util::xmsg::XEngIce, 837
util::xmsg::XEngSurface, 880
util::xmsg::XFall, 901
util::xmsg::XRise, 1545
util::xmsg::XSurfaceData, 1573
StartupTasks
argo::iridium::xfloat::XFloat, 957
StartWorkerThread
argo::iridium::xfloat::XFloatMailRetrievalService, 1175
argo::iridium::xfloat::XFloatPostProcessingService, 1240
StartWorkerThreads

argo::iridium::xfloat::XFloatMailProcessingService, 1151
argo::iridium::xfloat::XFloatTelemetryDecoderService, 1255
Main.cpp, 2089
Main.h, 2091
status_directory_path_
argo::iridium::xfloat::XFloat, 972
status_file_generation_is_enabled_
argo::iridium::xfloat::XFloat, 972
status_file_path_
argo::iridium::xfloat::XFloat, 972
status_file_updated_
argo::iridium::xfloat::XFloat, 972
std::make_shared
argo::iridium::xfloat::XFloatDiveData, 1037
util::xmsg::XGPS, 1302
surface_pres_max_
argo::iridium::xfloat::XFloatDiveData, 1045
surface_pres_mean_
argo::iridium::xfloat::XFloatDiveData, 1045
surface_pres_min_
argo::iridium::xfloat::XFloatDiveData, 1046
surface_pres_num_samples_
argo::iridium::xfloat::XFloatDiveData, 1046
surface_pres_valid_
argo::iridium::xfloat::XFloatDiveData, 1046
surface_pres_variance_
argo::iridium::xfloat::XFloatDiveData, 1046
surface_start_time_
util::xmsg::XAlamoMissionConfiguration, 495
util::xmsg::XMfaMissionCfg, 1414
surface_state_start_time
util::xmsg::XArgo, 529
surface_state_start_time_
util::xmsg::XArgo, 538
SynchronousQueue
util::thread::SynchronousQueue< T >, 410
system_current_a
util::xmsg::XEngAscent, 723
util::xmsg::XEngDescent, 779
util::xmsg::XEngSurface, 875
system_current_ma_
util::xmsg::XEngAscent, 725
util::xmsg::XEngDescent, 783
util::xmsg::XEngSurface, 880
system_voltage_mv_
util::xmsg::XEngDescent, 783
system_voltage_v
util::xmsg::XEngDescent, 779
SystemUtils.h
ATTR_DEPRECATED, 2207

table_columns_
util::mysql::SqlTableDescription< T >, 406
table_name_
util::mysql::SqlTableDescription< T >, 406
target_ascent_rate
util::xmsg::XArgo, 529
target_ascent_rate_
util::xmsg::XArgo, 538
target_drift_duration
util::xmsg::XArgo, 530
target_drift_duration_
util::xmsg::XArgo, 538
target_drift_pressure
util::xmsg::XArgo, 530
target_drift_pressure_
util::xmsg::XArgo, 538
target_profile_pressure
util::xmsg::XArgo, 530
target_profile_pressure_
util::xmsg::XArgo, 538
temp_messages_
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >, 127
util::xmsg::XCompositeDrift, 560
util::xmsg::XCompositeHiResProfile, 581
util::xmsg::XCompositeProfile, 619
temperature_counts_
util::xmsg::XSensorTripletCodeCounts, 1553
TemperatureValue
util::xmsg::XSensorTripletCodeCounts, 1551
TempMsgPtr_t
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >, 106
util::xmsg::XCompositeDrift, 545
util::xmsg::XCompositeHiResProfile, 567
util::xmsg::XCompositeProfile, 605
TempMsgVector_t
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >, 106
util::xmsg::XCompositeDrift, 546
util::xmsg::XCompositeHiResProfile, 567
util::xmsg::XCompositeProfile, 605
TerminateWorkerThread
argo::iridium::xfloat::XFloatMailRetrievalService, 1176
argo::iridium::xfloat::XFloatPostProcessingService, 1241
TerminateWorkerThreads
argo::iridium::xfloat::XFloatMailProcessingService, 1151
argo::iridium::xfloat::XFloatTelemetryDecoderService, 1256
Main.cpp, 2089
Main.h, 2092
termination_signaled_
argo::iridium::xfloat::XFloatMailProcessingService, 1153
argo::iridium::xfloat::XFloatMailRetrievalService, 1180
argo::iridium::xfloat::XFloatPostProcessingService, 1243
argo::iridium::xfloat::XFloatTelemetryDecoderService, 1257
thread_
argo::iridium::xfloat::XFloatMailRetrievalService, 1181
argo::iridium::xfloat::XFloatPostProcessingService, 1243
thread_group_
argo::iridium::xfloat::XFloatMailProcessingService, 1154
argo::iridium::xfloat::XFloatTelemetryDecoderService, 1257
thread_object_
argo::iridium::xfloat::XFloatMailRetrievalService, 1181
argo::iridium::xfloat::XFloatPostProcessingService, 1243
thread_objects_
argo::iridium::xfloat::XFloatMailProcessingService, 1154
argo::iridium::xfloat::XFloatTelemetryDecoderService, 1257
ThreadDefinitions.h, 2118
XMTPS_DEBUG_FETCH_MAIL, 2119
XMTPS_THREAD_JOIN_SECONDS, 2119
time_to_acquire_fix_
util::xmsg::XGPS, 1306

toa_sec_
util::xmsg::XAcousticCacst, 420
toa_usec_
util::xmsg::XAcousticCacst, 420
Todo List, 19
total_satellite_comm_time_seconds
util::xmsg::XCompositeIridiumPosition, 600
total_satellite_comm_time_seconds_
util::xmsg::XCompositeIridiumPosition, 602
ts_end_
util::xmsg::XAcousticWindow, 435
type_
util::xmsg::XAlamoBit2, 457

u08
XDefinitions.h, 1771
u16
XDefinitions.h, 1771
u24
XDefinitions.h, 1771
u32
XDefinitions.h, 1771
UnlockObject
argo::iridium::Iridium, 258
argo::iridium::xfloat::XFloat, 957
argo::iridium::xfloat::XFloatDiveData, 1034
argo::iridium::xfloat::XFloatMailMessage, 1120
argo::iridium::xfloat::XFloatMetadata, 1226
util::thread::Lockable, 326
util::thread::RecursiveLockable, 366
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >, 123
util::xmsg::XCompositeDrift, 557
util::xmsg::XCompositeHiResProfile, 578
util::xmsg::XCompositeProfile, 616
UnpackData
util::xmsg::AbstractXFallRise, 80
util::xmsg::XFall, 898
util::xmsg::XRise, 1542
util::xmsg::XSurfaceData, 1570
UnpackSensorData
util::xmsg::AbstractXSensorData, 98
util::xmsg::AuxiliarySensorData, 150
util::xmsg::XDriftPressure, 653
util::xmsg::XDriftSalinity, 672
util::xmsg::XDriftTemperature, 691
util::xmsg::XHiResPressure, 1321
util::xmsg::XHiResSalinity, 1340
util::xmsg::XHiResTemperature, 1359
util::xmsg::XProfilePressure, 1461
util::xmsg::XProfileSalinity, 1480
util::xmsg::XProfileTemperature, 1499
UpdateDerivedSurfaceDataFields
argo::iridium::xfloat::XFloatDiveData, 1034
user_
argo::iridium::xfloat::XFloat, 972
argo::iridium::xfloat::XFloatDiveData, 1046
argo::iridium::xfloat::XFloatMailMessage, 1136
argo::iridium::xfloat::XFloatMailProcessingService, 1154
argo::iridium::xfloat::XFloatMailRetrievalService, 1181
argo::iridium::xfloat::XFloatMetadata, 1233
util::mysql::LocalMysqlConfiguration, 312
util::mysql::LocalPersistentEntity, 325
util::mysql::SqlTableDescription< T >, 406
util::xmsg::XGPS, 1306
util, 41
util::config, 42
util::config::LocalConfiguration, 290
~LocalConfiguration, 291
AcquireRuntimeLock, 291
CheckRuntimeLock, 292
configuration, 292
configuration_, 298
configuration_file_path_, 298
GenerateCommandLineOptionsDescription, 292
GenerateConfigurationFileOptionsDescription, 293
GetDirectoryPath, 293
GetDirectoryPathFromString, 294
InitializeConfiguration, 295
is_daemon, 295
is_daemon_, 299
ListDirectories, 296
ListFiles, 296
LocalConfiguration, 291
MoveFileToDirectory, 296
ParseProgramOptions, 297
pid_file_path_, 299
ReleaseRuntimeLock, 298
SPLASH, 299
SPLASH_LIS, 299
splash_lis, 298
util::config::LocalConfiguration::GenerateCommandLineOptionsDescription
argo::iridium::xfloat::XFloat, 960
argo::iridium::xfloat::XFloatMailRetrievalService, 1176
util::config::LocalConfiguration::GenerateConfigurationFileOptionsDescriptio
argo::iridium::xfloat::XFloat, 960
argo::iridium::xfloat::XFloatDiveData, 1037
argo::iridium::xfloat::XFloatMailMessage, 1124
argo::iridium::xfloat::XFloatMailProcessingService, 1152
argo::iridium::xfloat::XFloatMailRetrievalService, 1176
argo::iridium::xfloat::XFloatMetadata, 1227
argo::iridium::xfloat::XFloatPostProcessingService, 1241
argo::iridium::xfloat::XFloatTelemetryDecoderService, 1256
util::date, 42
util::date::DateUtils, 243
GenerateCurrentUtcTimeIso8601String, 244
GenerateIso8601String, 244
GenerateJulianDay, 245
GetCurrentUtcTime, 246
util::file, 43
util::file::FileSystemUtils, 246
GetDirectoryPath, 247
GetDirectoryPathFromString, 247
ListDirectories, 248
ListFiles, 248
MoveFileToDirectory, 250
util::ftp, 43
util::kml::KmlUtil, 282
CloseKmlFactory, 284
GetDefaltKmlPathColor, 284
GetDefaltKmlPathStyleId, 284
GetDefaltKmlPlacemarkStyleId, 284
GetKmlFactory, 285
GetSmallKmlPlacemarkStyleId, 285
GetTriangleKmlPlacemarkStyleId, 286
kml_default_path_colors_, 288
kml_default_path_style_ids_, 288
kml_default_placemark_style_ids_, 288
kml_factory_, 289
kml_small_placemark_style_ids_, 289
kml_triangle_placemark_style_ids_, 289
SetAllDefaultKmlSyles, 286
SetDefaultKmlStyle, 287
util::log, 43
LOG_DBG, 44
LOG_DBG1, 44

LOG_DBG2, 44
LOG_DBG3, 44
LOG_ERR, 44
LOG_FAT, 44
LOG_INF, 44
LOG_LOG, 44
LOG_WRN, 44
LogLevel_t, 44
sev_logger_mt, 44
util::log::Log, 327
~Log, 328
Flush, 329
FlushGlobalLogger, 330
global_logger_, 331
InitializeLog, 330
Log, 328
log_directory_path_, 331
log_msg_severity_level_, 331
log_msg_stream_, 331
log_reporting_level, 330
log_reporting_level_, 332
operator<<, 330
set_log_reporting_level, 331
util::log::MessageException, 333
~MessageException, 334
message_, 335
MessageException, 334
what, 335
util::mail, 44
util::math, 45
util::math::MathUtils, 332
degToRad, 332
util::mysql, 45
ColumDataType_t, 46
util::mysql::Column_bool< C >, 154
accessor_, 160
Column_bool, 155
column_index, 156
column_index_, 160
column_name, 157
column_name_, 160
column_select_name, 157
data_type, 158
data_type_, 160
GetValueFromObject, 158
is_primary_key, 158
is_primary_key_, 160
mutator_, 160
set_column_index, 158
SetObjectValue, 159
SetObjectValueFromResultSet, 159
SetPreparedStatementValueFromObject, 159
util::mysql::Column_char< C >, 161
accessor_, 166
Column_char, 162
column_index, 163
column_index_, 166
column_name, 163
column_name_, 166
column_select_name, 164
data_type, 164
data_type_, 167
GetValueFromObject, 164
is_primary_key, 164
is_primary_key_, 167
mutator_, 167
set_column_index, 165
SetObjectValue, 165
SetObjectValueFromResultSet, 165
SetPreparedStatementValueFromObject, 166
util::mysql::Column_double< C >, 167
accessor_, 173
Column_double, 168
column_index, 169
column_index_, 173
column_name, 170
column_name_, 173
column_select_name, 170
data_type, 171
data_type_, 173
GetValueFromObject, 171
is_primary_key, 171
is_primary_key_, 173
mutator_, 173
set_column_index, 171
SetObjectValue, 172
SetObjectValueFromResultSet, 172
SetPreparedStatementValueFromObject, 172
util::mysql::Column_filesystem_path< C >, 174
accessor_, 180
Column_filesystem_path, 175
column_index, 176
column_index_, 180
column_name, 177
column_name_, 180
column_select_name, 177
data_type, 178
data_type_, 180
GetValueFromObject, 178
is_primary_key, 178
is_primary_key_, 180
mutator_, 180
set_column_index, 178
SetObjectValue, 179
SetObjectValueFromResultSet, 179
SetPreparedStatementValueFromObject, 179
util::mysql::Column_float< C >, 181
accessor_, 186
Column_float, 182
column_index, 183
column_index_, 186
column_name, 183
column_name_, 186
column_select_name, 184
data_type, 184
data_type_, 187
GetValueFromObject, 184
is_primary_key, 184
is_primary_key_, 187
mutator_, 187
set_column_index, 185
SetObjectValue, 185
SetObjectValueFromResultSet, 185
SetPreparedStatementValueFromObject, 186
util::mysql::Column_int< C >, 187
accessor_, 193
column_index, 189
column_index_, 193
Column_int, 188
column_name, 190
column_name_, 193
column_select_name, 190
data_type, 191
data_type_, 193
GetValueFromObject, 191
is_primary_key, 191
is_primary_key_, 193
mutator_, 193
set_column_index, 191
SetObjectValue, 192
SetObjectValueFromResultSet, 192
SetPreparedStatementValueFromObject, 192
util::mysql::Column_int16_t< C >, 194
accessor_, 199

column_index, 196
column_index_, 199
Column_int16_t, 195
column_name, 196
column_name_, 199
column_select_name, 197
data_type, 197
data_type_, 200
GetValueFromObject, 197
is_primary_key, 197
is_primary_key_, 200
mutator_, 200
set_column_index, 198
SetObjectValue, 198
SetObjectValueFromResultSet, 198
SetPreparedStatementValueFromObject, 199
util::mysql::Column_long< C >, 200
accessor_, 206
column_index, 202
column_index_, 206
Column_long, 201
column_name, 203
column_name_, 206
column_select_name, 203
data_type, 204
data_type_, 206
GetValueFromObject, 204
is_primary_key, 204
is_primary_key_, 206
mutator_, 206
set_column_index, 204
SetObjectValue, 205
SetObjectValueFromResultSet, 205
SetPreparedStatementValueFromObject, 205
util::mysql::Column_posix_time< C >, 207
accessor_, 213
column_index, 209
column_index_, 213
column_name, 210
column_name_, 213
Column_posix_time, 208
column_select_name, 210
data_type, 211
data_type_, 213
GetValueFromObject, 211
is_primary_key, 211
is_primary_key_, 213
mutator_, 213
set_column_index, 211
SetObjectValue, 212
SetObjectValueFromResultSet, 212
SetPreparedStatementValueFromObject, 212
util::mysql::Column_string< C >, 214
accessor_, 220
column_index, 216
column_index_, 220
column_name, 217
column_name_, 220
column_select_name, 217
Column_string, 215
data_type, 218
data_type_, 220
GetValueFromObject, 218
is_primary_key, 218
is_primary_key_, 220
mutator_, 220
set_column_index, 218
SetObjectValue, 219
SetObjectValueFromResultSet, 219
SetPreparedStatementValueFromObject, 219
util::mysql::Column_uint16_t< C >, 221
accessor_, 226
column_index, 223
column_index_, 226
column_name, 223
column_name_, 226
column_select_name, 224
Column_uint16_t, 222
data_type, 224
data_type_, 227
GetValueFromObject, 224
is_primary_key, 224
is_primary_key_, 227
mutator_, 227
set_column_index, 225
SetObjectValue, 225
SetObjectValueFromResultSet, 225
SetPreparedStatementValueFromObject, 226
util::mysql::Column_uint8_t< C >, 227
accessor_, 233
column_index, 229
column_index_, 233
column_name, 230
column_name_, 233
column_select_name, 230
Column_uint8_t, 228
data_type, 231
data_type_, 233
GetValueFromObject, 231
is_primary_key, 231
is_primary_key_, 233
mutator_, 233
set_column_index, 231
SetObjectValue, 232
SetObjectValueFromResultSet, 232
SetPreparedStatementValueFromObject, 232
util::mysql::Column_vector_uint8_t< C >, 234
accessor_, 240
column_index, 236
column_index_, 240
column_name, 236
column_name_, 240
column_select_name, 237
Column_vector_uint8_t, 235
data_type, 237
data_type_, 240
GetValueFromObject, 237
htoi, 237
is_primary_key, 238
is_primary_key_, 240
mutator_, 240
set_column_index, 238
SetObjectValue, 238
SetObjectValueFromResultSet, 239
SetPreparedStatementValueFromObject, 239
util::mysql::LocalMysqlConfiguration, 300
~LocalMysqlConfiguration, 302
address_, 311
CleanupSqlConnection, 302
CleanupSqlDriver, 302
CloseSqlConnection, 303
ExecuteQuery, 303
executeQuery, 304
executeSelectQuery, 304
ExecuteUpdate, 305
executeUpdate, 306
GeneratePreparedStatement, 306
generatePreparedStatement, 308
GenerateSqlDeleteStr, 308
GenerateSqlSelectStr, 309
GetSqlConnection, 309
getSqlConnection, 310
GetSqlDriver, 311
LocalMysqlConfiguration, 302

password_, 311
port_, 312
protocol_, 312
schema_, 312
sql_connection_, 312
sql_driver_, 312
user_, 312
util::mysql::LocalPersistentEntity, 313
~LocalPersistentEntity, 315
address_, 324
CleanupSqlConnection, 315
CleanupSqlDriver, 315
CloseSqlConnection, 315
ExecuteQuery, 316
executeQuery, 316
executeSelectQuery, 317
ExecuteUpdate, 318
executeUpdate, 319
GeneratePreparedStatement, 319
generatePreparedStatement, 320
GenerateSqlDeleteStr, 320
GenerateSqlSelectStr, 321
GetSqlConnection, 321
getSqlConnection, 322
GetSqlDriver, 323
LocalPersistentEntity, 315
password_, 324
port_, 324
protocol_, 324
SaveObjectToDatabase, 323
schema_, 324
sql_connection_, 324
sql_driver_, 324
user_, 325
util::mysql::MysqlUtils, 335
CleanupSqlConnection, 336
CleanupSqlDriver, 336
CloseSqlConnection, 337
ExecuteQuery, 337
ExecuteUpdate, 338
GeneratePreparedStatement, 339
GenerateSqlDeleteStr, 339
GenerateSqlSelectStr, 340
GetSqlConnection, 340
GetSqlDriver, 341
sql_connection_, 342
sql_driver_, 342
util::mysql::SqlTableColumnDescription< C, T >, 372
accessor_, 377
column_index, 374
column_index_, 377
column_name, 375
column_name_, 377
column_select_name, 376
data_type, 376
data_type_, 377
GetValueFromObject, 376
is_primary_key, 376
is_primary_key_, 377
mutator_, 378
set_column_index, 376
SetObjectValue, 377
SqlTableColumnDescription, 373, 374
util::mysql::SqlTableColumnDescriptionBase, 378
~SqlTableColumnDescriptionBase, 379
column_index, 380
column_index_, 382
column_name, 380
column_name_, 382
column_select_name, 381
data_type, 381
data_type_, 382
is_primary_key, 381
is_primary_key_, 382
set_column_index, 381
SqlTableColumnDescriptionBase, 379
util::mysql::SqlTableDescription< T >, 382
~SqlTableDescription, 387
address_, 404
cbol, 385
cchar, 385
cdbl, 385
cflt, 385
ci16, 385
cint, 386
CleanupSqlConnection, 387
CleanupSqlDriver, 387
clng, 386
CloseSqlConnection, 388
cpth, 386
cptm, 386
cstr, 386
cu16, 386
cu8, 386
cvu8, 386
ExecuteQuery, 388
executeQuery, 389
executeSelectQuery, 390
ExecuteUpdate, 391
executeUpdate, 391
generateInsertStatment, 392
GenerateObjectList, 392
GeneratePreparedStatement, 393
generatePreparedStatement, 393
generateSelectAllStatement, 395
GenerateSqlDeleteStr, 395
GenerateSqlSelectStr, 396
getColumnNames, 396
getColumns, 397
getColumnSelectNames, 397
GetSqlConnection, 397
getSqlConnection, 398
GetSqlDriver, 399
htoi, 399
insert_on_duplicate_key_update_statement_, 404
password_, 405
port_, 405
protocol_, 405
SaveObjectToDatabase, 400
schema_, 405
select_statement_, 405
SelectAllObjectsFromDatabase, 400
SelectObjectFromDatabase, 401
SelectObjectsFromDatabase, 402
SelectValueFromDatabase, 403
sql_connection_, 405
sql_driver_, 405
SqlTableDescription, 386
table_columns_, 406
table_name_, 406
user_, 406
values, 404
util::position, 46
util::position::Position, 342
~Position, 344
CalculateEllipsoidHeight, 344
CalculateGeographicDistance, 345
degToRad, 346
EARTH_RADIUS_KM_EQUATORIAL, 357
EARTH_RADIUS_KM_POLAR, 357
GenerateCurrentUtcTimeIso8601String, 346
GenerateIso6709String, 347
GenerateIso8601String, 347
GenerateJulianDay, 348

GetCurrentUtcTime, 349
IsPositionValid, 349
Position, 344
position_datetime, 350
position_datetime_, 357
position_datetime_as_iso8601_string, 350
position_is_valid, 351
position_is_valid_, 357
position_latitude, 351
position_latitude_, 357
position_longitude, 352
position_longitude_, 357
postition_as_iso6709_string, 352
set_gps_position_datetime, 353
set_position_datetime, 354
set_position_is_valid, 354
set_position_latitude, 354
set_position_longitude, 356
util::position::PositionUtils, 358
CalculateEllipsoidHeight, 359
CalculateGeographicDistance, 359
degToRad, 360
EARTH_RADIUS_KM_EQUATORIAL, 364
EARTH_RADIUS_KM_POLAR, 364
GenerateCurrentUtcTimeIso8601String, 361
GenerateIso6709String, 361
GenerateIso8601String, 362
GenerateJulianDay, 363
GetCurrentUtcTime, 363
IsPositionValid, 364
util::sig, 47
util::sig::SignalHandler, 371
ExitHandler, 371
RegisterSignalHandlers, 371
util::stream, 47
util::stream::DataBuf< CharT, TraitsT >, 241
DataBuf, 242
util::stream::StreamUtils, 408
htoi, 408
util::stream::VectorBuf< CharT, TraitsT >, 415
VectorBuf, 416
util::stream::VectorBuf_c< CharT >, 417
VectorBuf_c, 417
util::string, 47
util::sys, 47
util::thread, 48
util::thread::Lockable, 325
∼Lockable, 326
Lockable, 326
LockObject, 326
mutex_, 327
UnlockObject, 326
util::thread::RecursiveLockable, 365
∼RecursiveLockable, 366
LockObject, 366
mutex_, 367
RecursiveLockable, 366
UnlockObject, 366
util::thread::SynchronousQueue< T >, 409
buffer, 412
bufferNotEmpty, 412
bufferNotFull, 412
empty, 410
full, 410
mutex, 412
pop, 410
push, 411
size, 411
SynchronousQueue, 410
util::thread::ThreadUtils, 413
executeAndPipe, 414
util::types, 48
util::xmsg, 48
XFallRiseDataPackFormat_t, 54
XMSG_TYPE_ID_ALAMO_WAVE_C1, 57
XMSG_TYPE_ID_ALAMO_WAVE_C3, 58
XMSG_TYPE_ID_ARGO_DATA, 58
XMSG_TYPE_ID_AUX_SENSOR_DATA, 58
XMSG_TYPE_ID_ENG_ABORT, 58
XMSG_TYPE_ID_ENG_BIT_FAIL, 58
XMSG_TYPE_ID_ENG_BIT_PASS, 58
XMSG_TYPE_ID_ENG_DIAGNOSTIC, 58
XMSG_TYPE_ID_ENG_DIVE, 58
XMSG_TYPE_ID_FALL0, 56
XMSG_TYPE_ID_FALL1, 56
XMSG_TYPE_ID_FALL2, 56
XMSG_TYPE_ID_FALL3, 56
XMSG_TYPE_ID_FALL4, 56
XMSG_TYPE_ID_FALL5, 56
XMSG_TYPE_ID_FALL6, 56
XMSG_TYPE_ID_FALL7, 56
XMSG_TYPE_ID_FALL8, 56
XMSG_TYPE_ID_FALL9, 56
XMSG_TYPE_ID_FALLA, 56
XMSG_TYPE_ID_FALLB, 56
XMSG_TYPE_ID_FALLC, 56
XMSG_TYPE_ID_FALLD, 56
XMSG_TYPE_ID_FALLE, 56
XMSG_TYPE_ID_FALLF, 56
XMSG_TYPE_ID_GPS_ABORT, 55
XMSG_TYPE_ID_GPS_BITEST, 55
XMSG_TYPE_ID_GPS_DIAGNOSTIC, 55
XMSG_TYPE_ID_GPS_ENDOFDIVE, 55
XMSG_TYPE_ID_GPS_STARTOFDIVE, 55
XMSG_TYPE_ID_MISS_CMD_ECHO, 58
XMSG_TYPE_ID_MISS_CONF0, 58
XMSG_TYPE_ID_MISS_CONF1, 58
XMSG_TYPE_ID_MISS_CONF2, 58
XMSG_TYPE_ID_MISS_CONF3, 58
XMSG_TYPE_ID_MISS_CONF4, 58
XMSG_TYPE_ID_MISS_CONF5, 58
XMSG_TYPE_ID_MISS_CONF6, 58
XMSG_TYPE_ID_MISS_CONF7, 58
XMSG_TYPE_ID_MIZ_ACOUSTIC, 58
XMSG_TYPE_ID_PRES0, 55
XMSG_TYPE_ID_PRES1, 55
XMSG_TYPE_ID_PRES2, 55
XMSG_TYPE_ID_PRES3, 55
XMSG_TYPE_ID_PRES4, 55
XMSG_TYPE_ID_PRES5, 55
XMSG_TYPE_ID_PRES6, 55
XMSG_TYPE_ID_PRES7, 55
XMSG_TYPE_ID_PRES8, 55
XMSG_TYPE_ID_PRES9, 55
XMSG_TYPE_ID_PRES_DRIFT8, 57
XMSG_TYPE_ID_PRES_DRIFT9, 57
XMSG_TYPE_ID_PRES_DRIFTA, 57
XMSG_TYPE_ID_PRES_DRIFTB, 57
XMSG_TYPE_ID_PRES_DRIFTC, 57
XMSG_TYPE_ID_PRES_DRIFTD, 57
XMSG_TYPE_ID_PRES_DRIFTE, 57
XMSG_TYPE_ID_PRES_DRIFTF, 57
XMSG_TYPE_ID_PRES_HI_RES0, 57
XMSG_TYPE_ID_PRES_HI_RES1, 57
XMSG_TYPE_ID_PRES_HI_RES2, 57
XMSG_TYPE_ID_PRES_HI_RES3, 57
XMSG_TYPE_ID_PRES_HI_RES4, 57
XMSG_TYPE_ID_PRES_HI_RES5, 57
XMSG_TYPE_ID_PRES_HI_RES6, 57
XMSG_TYPE_ID_PRES_HI_RES7, 57
XMSG_TYPE_ID_PRES_SURF0, 56
XMSG_TYPE_ID_PRES_SURF1, 56
XMSG_TYPE_ID_PRES_SURF2, 56
XMSG_TYPE_ID_PRES_SURF3, 56

XMSG_TYPE_ID_PRES_SURF4, 57
XMSG_TYPE_ID_PRES_SURF5, 57
XMSG_TYPE_ID_PRES_SURF6, 57
XMSG_TYPE_ID_PRES_SURF7, 57
XMSG_TYPE_ID_PRES_SURF8, 57
XMSG_TYPE_ID_PRES_SURF9, 57
XMSG_TYPE_ID_PRES_SURFA, 57
XMSG_TYPE_ID_PRES_SURFB, 57
XMSG_TYPE_ID_PRES_SURFC, 57
XMSG_TYPE_ID_PRES_SURFD, 57
XMSG_TYPE_ID_PRES_SURFE, 57
XMSG_TYPE_ID_PRES_SURFF, 57
XMSG_TYPE_ID_PRESA, 55
XMSG_TYPE_ID_PRESB, 55
XMSG_TYPE_ID_PRESC, 55
XMSG_TYPE_ID_PRESD, 55
XMSG_TYPE_ID_PRESE, 55
XMSG_TYPE_ID_PRESF, 55
XMSG_TYPE_ID_PUMP0, 56
XMSG_TYPE_ID_PUMP1, 56
XMSG_TYPE_ID_PUMP2, 56
XMSG_TYPE_ID_PUMP3, 56
XMSG_TYPE_ID_PUMP4, 56
XMSG_TYPE_ID_PUMP5, 56
XMSG_TYPE_ID_PUMP6, 56
XMSG_TYPE_ID_PUMP7, 56
XMSG_TYPE_ID_PUMP8, 56
XMSG_TYPE_ID_PUMP9, 56
XMSG_TYPE_ID_PUMPA, 56
XMSG_TYPE_ID_PUMPB, 56
XMSG_TYPE_ID_PUMPC, 56
XMSG_TYPE_ID_PUMPD, 56
XMSG_TYPE_ID_PUMPE, 56
XMSG_TYPE_ID_PUMPF, 56
XMSG_TYPE_ID_RISE0, 56
XMSG_TYPE_ID_RISE1, 56
XMSG_TYPE_ID_RISE2, 56
XMSG_TYPE_ID_RISE3, 56
XMSG_TYPE_ID_RISE4, 56
XMSG_TYPE_ID_RISE5, 56
XMSG_TYPE_ID_RISE6, 56
XMSG_TYPE_ID_RISE7, 56
XMSG_TYPE_ID_RISE8, 56
XMSG_TYPE_ID_RISE9, 56
XMSG_TYPE_ID_RISEA, 56
XMSG_TYPE_ID_RISEB, 56
XMSG_TYPE_ID_RISEC, 56
XMSG_TYPE_ID_RISED, 56
XMSG_TYPE_ID_RISEE, 56
XMSG_TYPE_ID_RISEF, 56
XMSG_TYPE_ID_SALN0, 55
XMSG_TYPE_ID_SALN1, 55
XMSG_TYPE_ID_SALN2, 55
XMSG_TYPE_ID_SALN3, 55
XMSG_TYPE_ID_SALN4, 55
XMSG_TYPE_ID_SALN5, 55
XMSG_TYPE_ID_SALN6, 55
XMSG_TYPE_ID_SALN7, 56
XMSG_TYPE_ID_SALN8, 56
XMSG_TYPE_ID_SALN9, 56
XMSG_TYPE_ID_SALN_DRIFT8, 57
XMSG_TYPE_ID_SALN_DRIFT9, 57
XMSG_TYPE_ID_SALN_DRIFTA, 57
XMSG_TYPE_ID_SALN_DRIFTB, 57
XMSG_TYPE_ID_SALN_DRIFTC, 57
XMSG_TYPE_ID_SALN_DRIFTD, 57
XMSG_TYPE_ID_SALN_DRIFTE, 57
XMSG_TYPE_ID_SALN_DRIFTF, 57
XMSG_TYPE_ID_SALN_HI_RES0, 57
XMSG_TYPE_ID_SALN_HI_RES1, 57
XMSG_TYPE_ID_SALN_HI_RES2, 57
XMSG_TYPE_ID_SALN_HI_RES3, 57
XMSG_TYPE_ID_SALN_HI_RES4, 57
XMSG_TYPE_ID_SALN_HI_RES5, 57
XMSG_TYPE_ID_SALN_HI_RES6, 57
XMSG_TYPE_ID_SALN_HI_RES7, 57
XMSG_TYPE_ID_SALNA, 56
XMSG_TYPE_ID_SALNB, 56
XMSG_TYPE_ID_SALNC, 56
XMSG_TYPE_ID_SALND, 56
XMSG_TYPE_ID_SALNE, 56
XMSG_TYPE_ID_SALNF, 56
XMSG_TYPE_ID_TEMP0, 55
XMSG_TYPE_ID_TEMP1, 55
XMSG_TYPE_ID_TEMP2, 55
XMSG_TYPE_ID_TEMP3, 55
XMSG_TYPE_ID_TEMP4, 55
XMSG_TYPE_ID_TEMP5, 55
XMSG_TYPE_ID_TEMP6, 55
XMSG_TYPE_ID_TEMP7, 55
XMSG_TYPE_ID_TEMP8, 55
XMSG_TYPE_ID_TEMP9, 55
XMSG_TYPE_ID_TEMP_DRIFT8, 57
XMSG_TYPE_ID_TEMP_DRIFT9, 57
XMSG_TYPE_ID_TEMP_DRIFTA, 57
XMSG_TYPE_ID_TEMP_DRIFTB, 57
XMSG_TYPE_ID_TEMP_DRIFTC, 57
XMSG_TYPE_ID_TEMP_DRIFTD, 57
XMSG_TYPE_ID_TEMP_DRIFTE, 57
XMSG_TYPE_ID_TEMP_DRIFTF, 57
XMSG_TYPE_ID_TEMP_HI_RES0, 57
XMSG_TYPE_ID_TEMP_HI_RES1, 57
XMSG_TYPE_ID_TEMP_HI_RES2, 57
XMSG_TYPE_ID_TEMP_HI_RES3, 57
XMSG_TYPE_ID_TEMP_HI_RES4, 57
XMSG_TYPE_ID_TEMP_HI_RES5, 57
XMSG_TYPE_ID_TEMP_HI_RES6, 57
XMSG_TYPE_ID_TEMP_HI_RES7, 57
XMSG_TYPE_ID_TEMPA, 55
XMSG_TYPE_ID_TEMPB, 55
XMSG_TYPE_ID_TEMPC, 55
XMSG_TYPE_ID_TEMPD, 55
XMSG_TYPE_ID_TEMPE, 55
XMSG_TYPE_ID_TEMPF, 55
XMSG_TYPE_ID_TEST, 58
XMSG_TYPE_ID_UNKNOWN, 58
XMsgTypeId_t, 55
XPHASE_CODE_BEGINING_OF_FALL_TO_PROFILE, 58
XPHASE_CODE_BEGINNING_OF_DRIFT, 58
XPHASE_CODE_BUOYANCY_AT_100DBAR, 58
XPHASE_CODE_DRIFTING, 58
XPHASE_CODE_END_OF_RISE, 58
XPHASE_CODE_FALLING_TO_PROFILE, 58
XPHASE_CODE_ICE_TURNAROUND, 58
XPHASE_CODE_INVALID, 58
XPHASE_CODE_RISING, 58
XPHASE_CODE_SEEK, 58
XPHASE_CODE_SEEK_DURING_DRIFT, 58
XPHASE_CODE_SINKING, 58
XPHASE_CODE_START_OF_RISE, 58
XPHASE_CODE_START_OF_SINK, 58
XPHASE_CODE_SURFACE, 58
XPHASE_CODE_UNKNOWN, 58
XPhaseCode_t, 58
XPUMP_PACK_LEGACY, 59
XPUMP_PACK_STANDARD, 59
XPUMP_PACK_STANDARD2, 59
XPUMP_PACK_UNKNOWN, 59
XPumpPackingFormat_t, 58
XRISEFALL_FMT_LEGACY, 54
XRISEFALL_FMT_STNDRD, 54
XRISEFALL_FMT_STNDRD_WITH_TEMP_AND_SALN, 55

XRISEFALL_FMT_UNKNOWN, 55
XSENS_FMT_CURVATURE, 59
XSENS_FMT_DIFFERENCE, 59
XSENS_FMT_UNKNOWN, 59
XSensorDataFormat_t, 59
XWAVE_MODE_RAW, 59
XWAVE_MODE_SPECTRAL, 59
XWAVE_MODE_UNKNOWN, 59
XWaveMode_t, 59
util::xmsg::AbstractXComposite< T >, 61
~AbstractXComposite, 62
AbstractXComposite, 62
AppendAllElements, 62, 63
AppendElement, 63
AssignElement, 63
Element_t, 62
elements, 64
elements_, 64
num_elements, 64
util::xmsg::AbstractXFallRise, 65
~AbstractXFallRise, 67
AbstractXFallRise, 66
CheckXMsgData, 67
data_pack_format, 68
data_pack_format_, 82
DecodeXMsgData, 68
GenerateCurrentUtcTimeIso8601String, 68
GenerateIso8601String, 69
GenerateJulianDay, 70
GenerateListingComment, 71
GetCurrentUtcTime, 71
id_dive, 71
id_dive_, 82
id_instrument, 72
id_instrument_, 83
id_xmsg_type, 73
id_xmsg_type_, 83
num_samples, 74
sensor_data, 74
sensor_data_, 83
set_data_pack_format, 75
set_id_dive, 75
set_id_instrument, 76
set_id_xmsg_type, 76
set_start_time, 77
set_xmsg_index, 78
set_xmsg_size_bytes, 78
start_time, 80
start_time_, 83
UnpackData, 80
xmsg_index, 81
xmsg_index_, 83
xmsg_size_bytes, 81
xmsg_size_bytes_, 83
util::xmsg::AbstractXSensorData, 84
~AbstractXSensorData, 86
AbstractXSensorData, 86
CheckXMsgData, 86
DecodeXMsgData, 87
id_dive, 87
id_dive_, 101
id_instrument, 88
id_instrument_, 101
id_xmsg_type, 89
id_xmsg_type_, 101
num_samples, 90
raw_sensor_data, 90
raw_sensor_data_, 101
Sample, 90
SampleValue, 91
SampleValueAsPressure, 91
SampleValueAsSalinity, 92
SampleValueAsTemperature, 92
sensor_data_format, 93
sensor_data_format_, 102
set_id_dive, 93
set_id_instrument, 93
set_id_xmsg_type, 94
set_raw_sensor_data, 95
set_sensor_data_format, 95
set_xmsg_index, 96
set_xmsg_size_bytes, 97
UnpackSensorData, 98
xmsg_index, 99
xmsg_index_, 102
xmsg_size_bytes, 100
xmsg_size_bytes_, 102
util::xmsg::AbstractXSensorDataComposite< PRES_CLASS, SALN_CLASS, TEMP_CLASS, MAX_MSGS >, 103
~AbstractXSensorDataComposite, 106
AbstractXSensorDataComposite, 106
aggregated_data_valid_, 123
aggregated_pres_data_, 123
aggregated_saln_data_, 124
aggregated_temp_data_, 124
AggregateSensorData, 106
AssignAuxTemperatureMessage, 107
AssignPressureMessage, 108
AssignSalinityMessage, 109
AssignTemperatureMessage, 110
GenerateListingString, 111
HasContiguousMessageIndices, 112
is_aggregated_data_valid, 112
IsDataComplete, 113
IsEmpty, 114
LockObject, 114
max_num_samples, 115
mutex_, 124
num_pres_samples, 115
num_saln_samples, 116
num_temp_samples, 117
pres_messages_, 126
PresMsgPtr_t, 105
PresMsgVector_t, 105
Reset, 118
saln_messages_, 126
SalnMsgPtr_t, 105
SalnMsgVector_t, 105
SamplePres, 119
SamplePresAsFloat, 119
SampleSaln, 120
SampleSalnAsFloat, 120
SampleTemp, 121
SampleTempAsFloat, 122
set_aggregated_data_valid, 122
temp_messages_, 127
TempMsgPtr_t, 106
TempMsgVector_t, 106
UnlockObject, 123
util::xmsg::AbstractXTimeSeriesDataComposite< MSG_CLASS >, 127
aggregated_data_valid_, 136
AggregateSensorData, 130
AssignMessage, 130
is_aggregated_data_valid, 131
IsEmpty, 131
Reset, 132
ResetAggregatedData, 132
SamplePres, 133
SamplePresAsFloat, 133
SampleSaln, 134
SampleSalnAsFloat, 134
SampleTemp, 135

SampleTempAsFloat, 135
set_aggregated_data_valid, 136
util::xmsg::AuxiliarySensorData, 137
~AuxiliarySensorData, 139
AuxiliarySensorData, 139
CheckXMsgData, 139
DecodeXMsgData, 139
id_dive, 140
id_dive_, 153
id_instrument, 141
id_instrument_, 153
id_xmsg_type, 142
id_xmsg_type_, 153
num_samples, 142
raw_sensor_data, 143
raw_sensor_data_, 153
Sample, 143
SampleValue, 143
SampleValueAsPressure, 144
SampleValueAsSalinity, 144
SampleValueAsTemperature, 145
sensor_data_format, 145
sensor_data_format_, 154
set_id_dive, 146
set_id_instrument, 146
set_id_xmsg_type, 146
set_raw_sensor_data, 147
set_sensor_data_format, 148
set_xmsg_index, 148
set_xmsg_size_bytes, 149
UnpackSensorData, 150
xmsg_index, 151
xmsg_index_, 154
xmsg_size_bytes, 152
xmsg_size_bytes_, 154
util::xmsg::XAcousticCacst, 418
~XAcousticCacst, 419
DecodeXMsgData, 419
mfd_power_, 420
psk_error_code_, 420
snr_in_, 420
snr_out_, 420
src_, 420
toa_sec_, 420
toa_usec_, 420
XAcousticCacst, 419
util::xmsg::XAcousticPosition, 421
~XAcousticPosition, 422
AssignWindowIndices, 423
CheckXMsgData, 423
DecodeXMsgData, 424
GenerateListingString, 424
id_dive, 425
id_dive_, 431
id_instrument, 426
id_instrument_, 431
id_xmsg_type, 427
id_xmsg_type_, 432
set_id_dive, 427
set_id_instrument, 428
set_id_xmsg_type, 428
set_xmsg_size_bytes, 429
XAcousticPosition, 422
xmsg_size_bytes, 430
xmsg_size_bytes_, 432
util::xmsg::XAcousticWindow, 432
~XAcousticWindow, 434
DecodeXMsgData, 434
p_end_, 435
p_start_, 435
ts_end_, 435
XAcousticWindow, 434
util::xmsg::XAlamoBit, 435
~XAlamoBit, 438
bit_status_code, 439
bit_status_code_, 443
bit_version, 439
bit_version_, 443
DecodeXMsgData, 439
float_software_version, 440
float_software_version_, 443
GenerateListingString, 440
pres_offset_, 443
set_bit_status_code, 441
set_bit_version, 441
set_float_software_version, 442
XAlamoBit, 438
util::xmsg::XAlamoBit2, 443
~XAlamoBit2, 447
air_pump_curr_max_, 454
air_pump_cycle_count_, 454
air_pump_errors_, 454
air_pump_test_duration_, 454
air_pump_v12_curr_max_, 454
air_pump_volt_max_, 454
air_valve_cal_open_, 454
air_valve_errors_, 454
air_valve_open_, 454
bat_volt_start_, 455
bit_passed_, 455
bit_status_code_, 455
bit_version, 448
bit_version_, 455
case_vac_start_, 455
DecodeXMsgData, 448
dt_end_, 455
dt_end_epoch_, 455
dt_start_, 455
dt_start_epoch_, 455
duration_sec_, 456
float_software_version, 449
float_software_version_, 456
GenerateCurrentUtcTimeIso8601String, 449
GenerateIso8601String, 449
GenerateJulianDay, 450
GenerateListingString, 451
GetCurrentUtcTime, 451
gps_errors_, 456
oil_pump_curr_max_, 456
oil_pump_errors_, 456
oil_pump_test_duration_, 456
oil_pump_v12_curr_max_, 456
oil_pump_volt_max_, 456
oil_valve_cal_open_, 456
oil_valve_errors_, 457
oil_valve_open_, 457
sensor_type_, 457
set_bit_passed, 452
set_bit_status_code, 452
set_bit_version, 452
set_float_software_version, 453
type_, 457
XAlamoBit2, 447
util::xmsg::XAlamoEngineering, 457
~XAlamoEngineering, 459
CheckXMsgData, 459
DecodeXMsgData, 460
engineering_version, 460
engineering_version_, 473
engineering_version_as_float, 461
GenerateCurrentUtcTimeIso8601String, 461
GenerateIso8601String, 462
GenerateJulianDay, 463
GetCurrentUtcTime, 463

id_dive, 464
id_dive_, 473
id_instrument, 465
id_instrument_, 473
id_xmsg_type, 466
id_xmsg_type_, 473
set_engineering_version, 466
set_id_dive, 467
set_id_instrument, 467
set_id_xmsg_type, 468
set_start_time, 469
set_xmsg_size_bytes, 470
start_time, 471
start_time_, 474
XAlamoEngineering, 459
xmsg_size_bytes, 472
xmsg_size_bytes_, 474
util::xmsg::XAlamoMissionConfiguration, 474
~XAlamoMissionConfiguration, 484
add_, 489
adt_, 489
apf_, 489
apn_, 489
apt_, 489
arr_, 489
ascent_start_time_, 489
asd_, 489
asi_, 489
asn_, 489
btm_, 490
cpb_, 490
cpn_, 490
cpp_, 490
cps_, 490
csp_, 490
cst_, 490
ctf_, 490
ctl_, 491
ctu_, 491
ddd_, 491
ddt_, 491
DecodeXMsgData, 485
descend_start_time_, 491
dft_, 491
dns_, 492
dsd_, 492
dsi_, 492
dst_, 492
dtd_, 492
fw_version_, 492
GenerateCurrentUtcTimeIso8601String, 485
GenerateIso8601String, 486
GenerateJulianDay, 487
GenerateListingString, 488
GetCurrentUtcTime, 488
mam_, 492
mat_, 492
mds_, 492
msg_version_, 493
msn_, 493
mtd_, 493
mtt_, 493
park_start_time_, 493
pdt_, 493
pfd_, 493
pfm_, 493
pns_, 493
psa_, 494
psi_, 494
pth_, 494
pto_, 494
sbm_, 494
sbt_, 494
sdc_, 494
sds_, 494
sensor_type_, 494
set_, 495
sfn_, 495
sgt_, 495
ssi_, 495
surface_start_time_, 495
XAlamoMissionConfiguration, 484
xas_, 495
util::xmsg::XArgo, 496
~XArgo, 501
argo_instrument_type, 501
argo_instrument_type_, 533
ascent_state_start_time, 502
ascent_state_start_time_, 533
CheckXMsgData, 502
DecodeXMsgData, 503
descent_state_start_time, 504
descent_state_start_time_, 533
drift_state_start_time, 504
drift_state_start_time_, 534
gain_pressure, 504
gain_pressure_, 534
gain_pressure_surface, 505
gain_pressure_surface_, 534
gain_salinity, 505
gain_salinity_, 534
gain_salinity_surface, 505
gain_salinity_surface_, 534
gain_temperature, 506
gain_temperature_, 535
gain_temperature_surface, 506
gain_temperature_surface_, 535
GenerateCurrentUtcTimeIso8601String, 506
GenerateIso8601String, 507
GenerateJulianDay, 508
GenerateListingString, 509
GetCurrentUtcTime, 509
id_dive, 509
id_dive_, 535
id_instrument, 510
id_instrument_, 535
id_xmsg_type, 511
id_xmsg_type_, 535
max_ascent_duration, 512
max_ascent_duration_, 535
max_fall_to_drift_duration, 512
max_fall_to_drift_duration_, 536
max_fall_to_profle_duration, 512
max_fall_to_profle_duration_, 536
max_surface_duration, 513
max_surface_duration_, 536
mission_phase, 513
mission_phase_, 536
num_seeks_to_drift_pressure, 513
num_seeks_to_drift_pressure_, 536
offset_pressure, 514
offset_pressure_, 536
offset_pressure_surface, 514
offset_pressure_surface_, 536
offset_salinity, 514
offset_salinity_, 537
offset_salinity_surface, 515
offset_salinity_surface_, 537
offset_temperature, 515
offset_temperature_, 537
offset_temperature_surface, 515
offset_temperature_surface_, 537
seek_to_drift_pressure_interval_duration, 516
seek_to_drift_pressure_interval_duration_, 537

set_argo_instrument_type, 516
set_ascent_state_start_time, 516
set_descent_state_start_time, 517
set_drift_state_start_time, 517
set_gain_pressure, 517
set_gain_pressure_surface, 518
set_gain_salinity, 518
set_gain_salinity_surface, 518
set_gain_temperature, 519
set_gain_temperature_surface, 519
set_id_dive, 519
set_id_instrument, 520
set_id_xmsg_type, 520
set_max_ascent_duration, 521
set_max_fall_to_drift_duration, 522
set_max_fall_to_profle_duration, 522
set_max_surface_duration, 522
set_mission_phase, 523
set_num_seeks_to_drift_pressure, 523
set_offset_pressure, 523
set_offset_pressure_surface, 524
set_offset_salinity, 524
set_offset_salinity_surface, 524
set_offset_temperature, 525
set_offset_temperature_surface, 525
set_seek_to_drift_pressure_interval_duration, 525
set_surface_state_start_time, 526
set_target_ascent_rate, 526
set_target_drift_duration, 526
set_target_drift_pressure, 527
set_target_profile_pressure, 527
set_x_version_major, 527
set_x_version_minor, 528
set_xmsg_size_bytes, 528
surface_state_start_time, 529
surface_state_start_time_, 538
target_ascent_rate, 529
target_ascent_rate_, 538
target_drift_duration, 530
target_drift_duration_, 538
target_drift_pressure, 530
target_drift_pressure_, 538
target_profile_pressure, 530
target_profile_pressure_, 538
x_version_float, 531
x_version_major, 531
x_version_major_, 538
x_version_minor, 532
x_version_minor_, 538
XArgo, 500
xmsg_size_bytes, 532
xmsg_size_bytes_, 539
util::xmsg::XCompositeAuxTemp, 539
AppendAllElements, 540, 541
AppendElement, 541
AssignElement, 541
Element_t, 540
elements, 541
elements_, 542
GenerateListingString, 541
num_elements, 542
util::xmsg::XCompositeDrift, 543
∼XCompositeDrift, 546
aggregated_data_valid_, 557
aggregated_pres_data_, 557
aggregated_saln_data_, 557
aggregated_temp_data_, 557
AggregateSensorData, 546
AssignAuxTemperatureMessage, 546
AssignPressureMessage, 547
AssignSalinityMessage, 547
AssignTemperatureMessage, 547
GenerateListingString, 548
GeneratePhyString, 549
HasContiguousMessageIndices, 550
is_aggregated_data_valid, 550
IsDataComplete, 551
IsEmpty, 551
LockObject, 551
max_num_samples, 552
mutex_, 558
num_pres_samples, 552
num_saln_samples, 552
num_temp_samples, 553
pres_messages_, 559
PresMsgPtr_t, 545
PresMsgVector_t, 545
Reset, 553
saln_messages_, 559
SalnMsgPtr_t, 545
SalnMsgVector_t, 545
SamplePres, 554
SamplePresAsFloat, 554
SampleSaln, 554
SampleSalnAsFloat, 555
SampleTemp, 555
SampleTempAsFloat, 556
set_aggregated_data_valid, 556
temp_messages_, 560
TempMsgPtr_t, 545
TempMsgVector_t, 546
UnlockObject, 557
XCompositeDrift, 546
util::xmsg::XCompositeFall, 560
∼XCompositeFall, 561
AppendAllElements, 562
AppendElement, 562
AssignElement, 562
Element_t, 561
elements, 562
elements_, 564
GenerateListingString, 563
num_elements, 563
XCompositeFall, 561
util::xmsg::XCompositeHiResProfile, 564
∼XCompositeHiResProfile, 567
aggregated_data_valid_, 578
aggregated_pres_data_, 578
aggregated_saln_data_, 579
aggregated_temp_data_, 579
AggregateSensorData, 567
AssignAuxTemperatureMessage, 568
AssignPressureMessage, 568
AssignSalinityMessage, 568
AssignTemperatureMessage, 569
GenerateListingString, 569
GeneratePhyString, 570
HasContiguousMessageIndices, 571
is_aggregated_data_valid, 572
IsDataComplete, 572
IsEmpty, 572
LockObject, 573
max_num_samples, 573
mutex_, 579
num_pres_samples, 573
num_saln_samples, 574
num_temp_samples, 574
pres_messages_, 581
PresMsgPtr_t, 566
PresMsgVector_t, 566
Reset, 574
saln_messages_, 581
SalnMsgPtr_t, 567
SalnMsgVector_t, 567

SamplePres, 575
SamplePresAsFloat, 575
SampleSaln, 576
SampleSalnAsFloat, 576
SampleTemp, 577
SampleTempAsFloat, 577
set_aggregated_data_valid, 578
temp_messages_, 581
TempMsgPtr_t, 567
TempMsgVector_t, 567
UnlockObject, 578
XCompositeHiResProfile, 567
util::xmsg::XCompositeIridiumPosition, 581
~XCompositeIridiumPosition, 584
AppendAllElements, 584
AppendElement, 584
AssignElement, 585
CalculateEllipsoidHeight, 585
CalculateGeographicDistance, 585
ComputeAverageIrdiumPosition, 586
degToRad, 587
EARTH_RADIUS_KM_EQUATORIAL, 601
EARTH_RADIUS_KM_POLAR, 601
Element_t, 584
elements, 587
elements_, 601
GenerateCurrentUtcTimeIso8601String, 588
GenerateIso6709String, 588
GenerateIso8601String, 589
GenerateJulianDay, 590
GenerateListingComment, 590
GenerateListingString, 591
GetCurrentUtcTime, 591
IsPositionValid, 592
num_elements, 592
position_datetime, 593
position_datetime_, 601
position_datetime_as_iso8601_string, 593
position_is_valid, 594
position_is_valid_, 601
position_latitude, 594
position_latitude_, 601
position_longitude, 595
position_longitude_, 601
position_standard_error, 595
position_standard_error_, 601
position_variance, 596
position_variance_, 602
postition_as_iso6709_string, 596
set_gps_position_datetime, 596
set_position_datetime, 597
set_position_is_valid, 597
set_position_latitude, 598
set_position_longitude, 598
set_position_standard_error, 599
set_position_variance, 599
set_total_satellite_comm_time_seconds, 600
total_satellite_comm_time_seconds, 600
total_satellite_comm_time_seconds_, 602
XCompositeIridiumPosition, 584
util::xmsg::XCompositeProfile, 602
~XCompositeProfile, 605
aggregated_data_valid_, 616
aggregated_pres_data_, 616
aggregated_saln_data_, 617
aggregated_temp_data_, 617
AggregateSensorData, 605
AssignAuxTemperatureMessage, 606
AssignPressureMessage, 606
AssignSalinityMessage, 606
AssignTemperatureMessage, 607
GenerateListingString, 607
GeneratePhyString, 608
HasContiguousMessageIndices, 609
is_aggregated_data_valid, 610
IsDataComplete, 610
IsEmpty, 610
LockObject, 611
max_num_samples, 611
mutex_, 617
num_pres_samples, 611
num_saln_samples, 612
num_temp_samples, 612
pres_messages_, 619
PresMsgPtr_t, 604
PresMsgVector_t, 604
Reset, 612
saln_messages_, 619
SalnMsgPtr_t, 605
SalnMsgVector_t, 605
SamplePres, 613
SamplePresAsFloat, 613
SampleSaln, 614
SampleSalnAsFloat, 614
SampleTemp, 615
SampleTempAsFloat, 615
set_aggregated_data_valid, 616
temp_messages_, 619
TempMsgPtr_t, 605
TempMsgVector_t, 605
UnlockObject, 616
XCompositeProfile, 605
util::xmsg::XCompositePump, 619
~XCompositePump, 621
AppendAllElements, 621
AppendElement, 621
AssignElement, 621
Element_t, 621
elements, 622
elements_, 623
GenerateListingString, 622
num_elements, 622
XCompositePump, 621
util::xmsg::XCompositeRise, 623
~XCompositeRise, 624
AppendAllElements, 625
AppendElement, 625
AssignElement, 625
Element_t, 624
elements, 625
elements_, 627
GenerateListingString, 626
num_elements, 626
XCompositeRise, 624
util::xmsg::XCompositeSurfaceData, 627
~XCompositeSurfaceData, 629
aggregated_data_valid_, 636
AggregateSensorData, 630
AssignMessage, 630
GenerateListingString, 630
is_aggregated_data_valid, 631
IsEmpty, 631
Reset, 632
ResetAggregatedData, 633
SamplePres, 633
SamplePresAsFloat, 633
SampleSaln, 633
SampleSalnAsFloat, 634
SampleTemp, 634
SampleTempAsFloat, 635
set_aggregated_data_valid, 635
XCompositeSurfaceData, 629
util::xmsg::XCompositeValve, 636
AppendAllElements, 637, 638

AppendElement, 638
AssignElement, 638
Element_t, 637
elements, 638
elements_, 639
num_elements, 638
util::xmsg::XDriftPressure, 639
~XDriftPressure, 641
CheckXMsgData, 641
DecodeXMsgData, 642
id_dive, 643
id_dive_, 656
id_instrument, 644
id_instrument_, 656
id_xmsg_type, 645
id_xmsg_type_, 656
num_samples, 645
raw_sensor_data, 646
raw_sensor_data_, 657
Sample, 646
SampleValue, 646
SampleValueAsPressure, 647
SampleValueAsSalinity, 647
SampleValueAsTemperature, 648
sensor_data_format, 648
sensor_data_format_, 657
set_id_dive, 649
set_id_instrument, 649
set_id_xmsg_type, 649
set_raw_sensor_data, 650
set_sensor_data_format, 651
set_xmsg_index, 651
set_xmsg_size_bytes, 652
UnpackSensorData, 653
XDriftPressure, 641
xmsg_index, 655
xmsg_index_, 657
xmsg_size_bytes, 655
xmsg_size_bytes_, 657
util::xmsg::XDriftSalinity, 658
~XDriftSalinity, 660
CheckXMsgData, 660
DecodeXMsgData, 661
id_dive, 661
id_dive_, 675
id_instrument, 662
id_instrument_, 675
id_xmsg_type, 663
id_xmsg_type_, 675
num_samples, 664
raw_sensor_data, 664
raw_sensor_data_, 675
Sample, 664
SampleValue, 665
SampleValueAsPressure, 665
SampleValueAsSalinity, 666
SampleValueAsTemperature, 666
sensor_data_format, 667
sensor_data_format_, 676
set_id_dive, 667
set_id_instrument, 667
set_id_xmsg_type, 668
set_raw_sensor_data, 669
set_sensor_data_format, 669
set_xmsg_index, 670
set_xmsg_size_bytes, 671
UnpackSensorData, 672
XDriftSalinity, 659
xmsg_index, 673
xmsg_index_, 676
xmsg_size_bytes, 674
xmsg_size_bytes_, 676
util::xmsg::XDriftTemperature, 677
~XDriftTemperature, 679
CheckXMsgData, 679
DecodeXMsgData, 680
id_dive, 680
id_dive_, 694
id_instrument, 681
id_instrument_, 694
id_xmsg_type, 682
id_xmsg_type_, 694
num_samples, 683
raw_sensor_data, 683
raw_sensor_data_, 694
Sample, 683
SampleValue, 684
SampleValueAsPressure, 684
SampleValueAsSalinity, 685
SampleValueAsTemperature, 685
sensor_data_format, 686
sensor_data_format_, 695
set_id_dive, 686
set_id_instrument, 686
set_id_xmsg_type, 687
set_raw_sensor_data, 688
set_sensor_data_format, 688
set_xmsg_index, 689
set_xmsg_size_bytes, 690
UnpackSensorData, 691
XDriftTemperature, 678
xmsg_index, 692
xmsg_index_, 695
xmsg_size_bytes, 693
xmsg_size_bytes_, 695
util::xmsg::XEngAbort, 696
~XEngAbort, 697
CheckXMsgData, 698
DecodeXMsgData, 698
GenerateListingString, 699
id_dive, 699
id_dive_, 705
id_instrument, 700
id_instrument_, 705
id_xmsg_type, 701
id_xmsg_type_, 706
set_id_dive, 702
set_id_instrument, 702
set_id_xmsg_type, 702
set_xmsg_size_bytes, 703
XEngAbort, 697
xmsg_size_bytes, 704
xmsg_size_bytes_, 706
util::xmsg::XEngAscent, 706
~XEngAscent, 709
case_vac_at_end_cinhg_, 724
case_vac_at_start_cinhg_, 724
CheckXMsgData, 709
DecodeXMsgData, 710
engineering_version, 710
engineering_version_, 724
engineering_version_as_float, 711
GenerateCurrentUtcTimeIso8601String, 711
GenerateIso8601String, 712
GenerateJulianDay, 713
GenerateListingString, 713
GetCurrentUtcTime, 714
id_dive, 715
id_dive_, 725
id_instrument, 716
id_instrument_, 725
id_xmsg_type, 717
id_xmsg_type_, 725
pump_run_duration_, 725

set_engineering_version, 717
set_id_dive, 718
set_id_instrument, 718
set_id_xmsg_type, 719
set_start_time, 720
set_xmsg_size_bytes, 721
start_time, 722
start_time_, 725
system_current_a, 723
system_current_ma_, 725
XEngAscent, 708
xmsg_size_bytes, 723
xmsg_size_bytes_, 725
util::xmsg::XEngBIT, 726
alamo_bit2_msg_, 736
alamo_bit_msg_, 736
CheckXMsgData, 728
DecodeXMsgData, 728
GenerateListingString, 729
id_dive, 730
id_dive_, 737
id_instrument, 731
id_instrument_, 737
id_xmsg_type, 732
id_xmsg_type_, 737
set_alamo_bit2_msg, 732
set_alamo_bit_msg, 733
set_id_dive, 733
set_id_instrument, 733
set_id_xmsg_type, 733
set_xmsg_size_bytes, 734
XEngBIT, 727
xmsg_size_bytes, 736
xmsg_size_bytes_, 737
util::xmsg::XEngBITFail, 738
∼XEngBITFail, 739
alamo_bit2_msg_, 748
alamo_bit_msg_, 748
CheckXMsgData, 740
DecodeXMsgData, 740
GenerateListingString, 741
id_dive, 742
id_dive_, 749
id_instrument, 743
id_instrument_, 749
id_xmsg_type, 744
id_xmsg_type_, 749
set_alamo_bit2_msg, 744
set_alamo_bit_msg, 745
set_id_dive, 745
set_id_instrument, 745
set_id_xmsg_type, 745
set_xmsg_size_bytes, 746
XEngBITFail, 739
xmsg_size_bytes, 748
xmsg_size_bytes_, 749
util::xmsg::XEngBITPass, 750
∼XEngBITPass, 751
alamo_bit2_msg_, 760
alamo_bit_msg_, 760
CheckXMsgData, 752
DecodeXMsgData, 752
GenerateListingString, 753
id_dive, 754
id_dive_, 761
id_instrument, 755
id_instrument_, 761
id_xmsg_type, 756
id_xmsg_type_, 761
set_alamo_bit2_msg, 756
set_alamo_bit_msg, 757
set_id_dive, 757
set_id_instrument, 757
set_id_xmsg_type, 757
set_xmsg_size_bytes, 758
XEngBITPass, 751
xmsg_size_bytes, 760
xmsg_size_bytes_, 761
util::xmsg::XEngDescent, 762
∼XEngDescent, 765
air_valve_middle_position_at_surface_mv_, 780
case_vac_at_end_cinhg_, 780
case_vac_at_start_cinhg_, 781
case_vac_cinhg_, 781
case_vac_inhg, 765
CheckXMsgData, 765
DecodeXMsgData, 766
engineering_version, 767
engineering_version_, 781
engineering_version_as_float, 767
first_pump_duration_adjustment_s_, 781
GenerateCurrentUtcTimeIso8601String, 768
GenerateIso8601String, 768
GenerateJulianDay, 769
GenerateListingString, 770
GetCurrentUtcTime, 770
id_dive, 771
id_dive_, 781
id_instrument, 772
id_instrument_, 781
id_xmsg_type, 773
id_xmsg_type_, 782
num_seek_attempts_, 782
oil_valve_closed_position_during_fall_pump_mv_, 782
oil_valve_open_position_at_surface_mv_, 782
pump_current_a, 773
pump_current_ma_, 782
pump_duration_sec_, 782
set_engineering_version, 774
set_id_dive, 774
set_id_instrument, 775
set_id_xmsg_type, 775
set_start_time, 776
set_xmsg_size_bytes, 777
start_time, 778
start_time_, 783
system_current_a, 779
system_current_ma_, 783
system_voltage_mv_, 783
system_voltage_v, 779
XEngDescent, 765
xmsg_size_bytes, 779
xmsg_size_bytes_, 783
util::xmsg::XEngDiagnostic, 784
∼XEngDiagnostic, 785
CheckXMsgData, 786
DecodeXMsgData, 786
GenerateListingString, 787
id_dive, 787
id_dive_, 793
id_instrument, 788
id_instrument_, 793
id_xmsg_type, 789
id_xmsg_type_, 794
set_id_dive, 790
set_id_instrument, 790
set_id_xmsg_type, 790
set_xmsg_size_bytes, 791
XEngDiagnostic, 785
xmsg_size_bytes, 792
xmsg_size_bytes_, 794
util::xmsg::XEngDrift, 794
∼XEngDrift, 797

average_drift_pressure_counts_, 812
average_drift_temperature_counts_, 812
case_vac_at_end_cinhg_, 812
case_vac_at_start_cinhg_, 812
CheckXMsgData, 797
DecodeXMsgData, 798
engineering_version, 799
engineering_version_, 813
engineering_version_as_float, 799
GenerateCurrentUtcTimeIso8601String, 799
GenerateIso8601String, 800
GenerateJulianDay, 801
GenerateListingString, 802
GetCurrentUtcTime, 802
id_dive, 803
id_dive_, 813
id_instrument, 804
id_instrument_, 813
id_xmsg_type, 805
id_xmsg_type_, 813
set_engineering_version, 805
set_id_dive, 806
set_id_instrument, 806
set_id_xmsg_type, 807
set_start_time, 808
set_xmsg_size_bytes, 809
start_of_fall_to_profile_pressure_counts_, 813
start_of_fall_to_profile_pressure_time_, 813
start_time, 810
start_time_, 813
XEngDrift, 797
xmsg_size_bytes, 811
xmsg_size_bytes_, 814
util::xmsg::XEngIce, 820
~XEngIce, 823
CheckXMsgData, 823
DecodeXMsgData, 824
engineering_version, 825
engineering_version_, 837
engineering_version_as_float, 825
error_codes_, 837
GenerateCurrentUtcTimeIso8601String, 826
GenerateIso8601String, 826
GenerateJulianDay, 827
GetCurrentUtcTime, 828
id_dive, 828
id_dive_, 837
id_instrument, 829
id_instrument_, 837
id_xmsg_type, 830
id_xmsg_type_, 837
set_engineering_version, 830
set_id_dive, 831
set_id_instrument, 831
set_id_xmsg_type, 832
set_start_time, 833
set_xmsg_size_bytes, 834
start_time, 835
start_time_, 837
XEngIce, 823
xmsg_size_bytes, 836
xmsg_size_bytes_, 837
util::xmsg::XEngIce::XEngIceErrorCode, 838
~XEngIceErrorCode, 839
date_time, 839
date_time_, 844
DateTimeAs8601String, 839
error_code, 840
error_code_, 844
GenerateCurrentUtcTimeIso8601String, 840
GenerateIso8601String, 841
GenerateJulianDay, 842
GetCurrentUtcTime, 843
set_date_time, 843
set_error_code, 843
XEngIceErrorCode, 839
util::xmsg::XEngStandard, 844
~XEngStandard, 846
CheckXMsgData, 846
DecodeXMsgData, 847, 848
GenerateListingString, 848
id_dive, 848
id_dive_, 854
id_instrument, 849
id_instrument_, 854
id_xmsg_type, 850
id_xmsg_type_, 855
set_id_dive, 851
set_id_instrument, 851
set_id_xmsg_type, 851
set_xmsg_size_bytes, 852
XEngStandard, 846
xmsg_size_bytes, 853
xmsg_size_bytes_, 855
util::xmsg::XEngSurface, 855
~XEngSurface, 860
air_bladder_max_pres_mpsi_, 876
air_pump_current_ma_, 876
air_pump_runtime_s_, 876
air_valve_closed_cal_position_mv_, 877
air_valve_mid_cal_position_mv_, 877
air_valve_mid_position_at_end_mv_, 877
air_valve_open_cal_position_mv_, 877
air_valve_open_position_at_start_mv_, 877
battery_voltage_mv_, 877
battery_voltage_v, 861
case_vac_at_end_cinhg_, 878
case_vac_at_start_cinhg_, 878
case_vac_cingh_, 878
case_vacuum_inhg, 861
CheckXMsgData, 861
ctd_status_code_, 878
DecodeXMsgData, 862
engineering_version, 863
engineering_version_, 878
engineering_version_as_float, 863
error_codes_, 878
GenerateCurrentUtcTimeIso8601String, 863
GenerateIso8601String, 864
GenerateJulianDay, 865
GenerateListingString, 866
GetCurrentUtcTime, 866
id_dive, 867
id_dive_, 879
id_instrument, 868
id_instrument_, 879
id_xmsg_type, 869
id_xmsg_type_, 879
num_errors_, 879
num_sbd_packets_for_dive_, 879
num_transmit_attempts_for_previous_dive_, 879
oil_valve_closed_cal_position_mv_, 879
oil_valve_closed_position_at_end_mv_, 879
oil_valve_open_cal_position_mv_, 880
oil_valve_open_position_during_air_assist_mv_, 880
pressure_offset_reading_, 880
set_engineering_version, 869
set_id_dive, 870
set_id_instrument, 870
set_id_xmsg_type, 871
set_start_time, 872
set_xmsg_size_bytes, 873
start_time, 874
start_time_, 880

system_current_a, 875
system_current_ma_, 880
XEngSurface, 860
xmsg_size_bytes, 875
xmsg_size_bytes_, 880
util::xmsg::XEngSurface::XEngErrorCode, 814
~XEngErrorCode, 815
date_time, 816
date_time_, 820
DateTimeAs8601String, 816
error_code, 816
error_code_, 820
GenerateCurrentUtcTimeIso8601String, 817
GenerateIso8601String, 817
GenerateJulianDay, 818
GetCurrentUtcTime, 819
set_date_time, 819
set_error_code, 819
XEngErrorCode, 815
util::xmsg::XFall, 881
~XFall, 883
CheckXMsgData, 883
data_pack_format, 884
data_pack_format_, 900
DecodeXMsgData, 884
GenerateCurrentUtcTimeIso8601String, 885
GenerateIso8601String, 885
GenerateJulianDay, 886
GenerateListingComment, 887
GenerateListingString, 887
GetCurrentUtcTime, 888
id_dive, 889
id_dive_, 900
id_instrument, 890
id_instrument_, 901
id_xmsg_type, 891
id_xmsg_type_, 901
num_samples, 891
sensor_data, 892
sensor_data_, 901
set_data_pack_format, 892
set_id_dive, 893
set_id_instrument, 893
set_id_xmsg_type, 894
set_start_time, 895
set_xmsg_index, 896
set_xmsg_size_bytes, 896
start_time, 898
start_time_, 901
UnpackData, 898
XFall, 882
xmsg_index, 899
xmsg_index_, 901
xmsg_size_bytes, 899
xmsg_size_bytes_, 901
util::xmsg::XGPS, 1258
~XGPS, 1263
address_, 1302
CalculateEllipsoidHeight, 1264
CalculateGeographicDistance, 1264
CheckXMsgData, 1265
CleanupSqlConnection, 1266
CleanupSqlDriver, 1266
CloseKmlFactory, 1267
CloseSqlConnection, 1267
DecodeXMsgData, 1267
degToRad, 1268
DeleteFromDatabase, 1269
DeleteObjectsFromDatabase, 1269
EARTH_RADIUS_KM_EQUATORIAL, 1302
EARTH_RADIUS_KM_POLAR, 1302
ExecuteQuery, 1270
executeQuery, 1270
executeSelectQuery, 1271
ExecuteUpdate, 1272
executeUpdate, 1272
GenerateCurrentUtcTimeIso8601String, 1273
GenerateIso6709String, 1273
GenerateIso8601String, 1274
GenerateJulianDay, 1275
GenerateListingComment, 1275
GenerateListingString, 1276
GeneratePhyString, 1276
GeneratePreparedStatement, 1277
generatePreparedStatement, 1277
GenerateSqlDeleteStr, 1278
GenerateSqlSelectStr, 1278
GetCurrentUtcTime, 1279
GetDefaltKmlPathColor, 1279
GetDefaltKmlPathStyleId, 1280
GetDefaltKmlPlacemarkStyleId, 1280
GetKmlFactory, 1280
GetSmallKmlPlacemarkStyleId, 1282
GetSqlConnection, 1282
getSqlConnection, 1283
GetSqlDriver, 1284
GetTriangleKmlPlacemarkStyleId, 1284
horizontal_dilution_of_precision_, 1302
id_dive, 1285
id_dive_, 1302
id_instrument, 1286
id_instrument_, 1302
id_xmsg_type, 1287
id_xmsg_type_, 1302
IsPositionValid, 1287
kml_default_path_colors_, 1303
kml_default_path_style_ids_, 1303
kml_default_placemark_style_ids_, 1303
kml_factory_, 1303
kml_small_placemark_style_ids_, 1304
kml_triangle_placemark_style_ids_, 1304
num_satellites_, 1304
password_, 1304
port_, 1305
position_datetime, 1288
position_datetime_, 1305
position_datetime_as_iso8601_string, 1288
position_is_valid, 1289
position_is_valid_, 1305
position_latitude, 1289
position_latitude_, 1305
position_longitude, 1290
position_longitude_, 1305
postition_as_iso6709_string, 1290
protocol_, 1305
SaveObjectToDatabase, 1291
schema_, 1305
SelectAllObjectsFromDatabase, 1291
SelectObjectFromDatabase, 1292
SelectObjectsFromDatabase, 1292
SelectValueFromDatabase, 1293
set_gps_position_datetime, 1294
set_id_dive, 1294
set_id_instrument, 1295
set_id_xmsg_type, 1295
set_position_datetime, 1296
set_position_is_valid, 1296
set_position_latitude, 1297
set_position_longitude, 1297
set_xmsg_size_bytes, 1298
SetAllDefaultKmlSyles, 1299
SetDefaultKmlStyle, 1300
signal_to_noise_max_, 1306
signal_to_noise_mean_, 1306

signal_to_noise_min_, 1306
sql_connection_, 1306
sql_driver_, 1306
std::make_shared, 1302
time_to_acquire_fix_, 1306
user_, 1306
XGPS, 1263
xmsg_size_bytes, 1301
xmsg_size_bytes_, 1307
util::xmsg::XHiResPressure, 1307
~XHiResPressure, 1309
CheckXMsgData, 1310
DecodeXMsgData, 1310
id_dive, 1311
id_dive_, 1324
id_instrument, 1312
id_instrument_, 1324
id_xmsg_type, 1313
id_xmsg_type_, 1324
num_samples, 1313
raw_sensor_data, 1314
raw_sensor_data_, 1325
Sample, 1314
SampleValue, 1314
SampleValueAsPressure, 1315
SampleValueAsSalinity, 1315
SampleValueAsTemperature, 1316
sensor_data_format, 1316
sensor_data_format_, 1325
set_id_dive, 1317
set_id_instrument, 1317
set_id_xmsg_type, 1317
set_raw_sensor_data, 1318
set_sensor_data_format, 1319
set_xmsg_index, 1319
set_xmsg_size_bytes, 1320
UnpackSensorData, 1321
XHiResPressure, 1309
xmsg_index, 1323
xmsg_index_, 1325
xmsg_size_bytes, 1323
xmsg_size_bytes_, 1325
util::xmsg::XHiResSalinity, 1326
~XHiResSalinity, 1328
CheckXMsgData, 1328
DecodeXMsgData, 1329
id_dive, 1329
id_dive_, 1343
id_instrument, 1330
id_instrument_, 1343
id_xmsg_type, 1331
id_xmsg_type_, 1343
num_samples, 1332
raw_sensor_data, 1332
raw_sensor_data_, 1343
Sample, 1332
SampleValue, 1333
SampleValueAsPressure, 1333
SampleValueAsSalinity, 1334
SampleValueAsTemperature, 1334
sensor_data_format, 1335
sensor_data_format_, 1344
set_id_dive, 1335
set_id_instrument, 1335
set_id_xmsg_type, 1336
set_raw_sensor_data, 1337
set_sensor_data_format, 1337
set_xmsg_index, 1338
set_xmsg_size_bytes, 1339
UnpackSensorData, 1340
XHiResSalinity, 1327
xmsg_index, 1341
xmsg_index_, 1344
xmsg_size_bytes, 1342
xmsg_size_bytes_, 1344
util::xmsg::XHiResTemperature, 1345
~XHiResTemperature, 1347
CheckXMsgData, 1347
DecodeXMsgData, 1348
id_dive, 1348
id_dive_, 1362
id_instrument, 1349
id_instrument_, 1362
id_xmsg_type, 1350
id_xmsg_type_, 1362
num_samples, 1351
raw_sensor_data, 1351
raw_sensor_data_, 1362
Sample, 1351
SampleValue, 1352
SampleValueAsPressure, 1352
SampleValueAsSalinity, 1353
SampleValueAsTemperature, 1353
sensor_data_format, 1354
sensor_data_format_, 1363
set_id_dive, 1354
set_id_instrument, 1354
set_id_xmsg_type, 1355
set_raw_sensor_data, 1356
set_sensor_data_format, 1356
set_xmsg_index, 1357
set_xmsg_size_bytes, 1358
UnpackSensorData, 1359
XHiResTemperature, 1346
xmsg_index, 1360
xmsg_index_, 1363
xmsg_size_bytes, 1361
xmsg_size_bytes_, 1363
util::xmsg::XMessage, 1364
~XMessage, 1366
CheckXMsgData, 1366
DecodeXMsgData, 1367
id_dive, 1367
id_dive_, 1373
id_instrument, 1368
id_instrument_, 1373
id_xmsg_type, 1369
id_xmsg_type_, 1374
set_id_dive, 1369
set_id_instrument, 1370
set_id_xmsg_type, 1370
set_xmsg_size_bytes, 1371
XMessage, 1365
xmsg_size_bytes, 1372
xmsg_size_bytes_, 1374
util::xmsg::XMessageIndexable, 1374
~XMessageIndexable, 1376
CheckXMsgData, 1376
DecodeXMsgData, 1377
id_dive, 1377
id_dive_, 1385
id_instrument, 1378
id_instrument_, 1385
id_xmsg_type, 1379
id_xmsg_type_, 1385
set_id_dive, 1380
set_id_instrument, 1380
set_id_xmsg_type, 1380
set_xmsg_index, 1381
set_xmsg_size_bytes, 1382
XMessageIndexable, 1375, 1376
xmsg_index, 1383
xmsg_index_, 1385
xmsg_size_bytes, 1384

xmsg_size_bytes_, 1386
util::xmsg::XMfaMissionCfg, 1386
~XMfaMissionCfg, 1398
aca_, 1403
acg_, 1403
acr_, 1403
add_, 1404
adt_, 1404
apf_, 1404
apn_, 1404
apt_, 1404
arr_, 1404
ascent_start_time_, 1404
asd_, 1404
asi_, 1404
asn_, 1405
AssignMessage, 1399
btm_, 1405
cms_, 1405
cpb_, 1405
cpn_, 1405
cpp_, 1405
cps_, 1405
crd_, 1405
crs_, 1405
csp_, 1406
cst_, 1406
ctc_, 1406
ctf_, 1406
ctl_, 1406
ctu_, 1406
ddb_, 1407
ddd_, 1407
ddt_, 1407
DecodeXMsgData, 1400
descend_start_time_, 1407
dft_, 1407
dns_, 1407
dsd_, 1407
dsi_, 1407
dst_, 1408
dtd_, 1408
fw_version_, 1408
GenerateCurrentUtcTimeIso8601String, 1400
GenerateIso8601String, 1401
GenerateJulianDay, 1402
GenerateListingString, 1402
GetCurrentUtcTime, 1403
iat_, 1408
ica_, 1408
icb_, 1408
icd_, 1408
icf_, 1408
icg_, 1408
icm_, 1409
icn_, 1409
ico_, 1409
icr_, 1409
ics_, 1409
ict_, 1409
icw_, 1409
icx_, 1409
ilt_, 1409
imm_, 1410
imt_, 1410
isa_, 1410
isd_, 1410
ism_, 1410
iso_, 1410
isp_, 1410
iss_, 1410
ist_, 1410
isv_, 1411
isw_, 1411
iue_, 1411
mam_, 1411
mat_, 1411
mds_, 1411
mnm_, 1411
mns_, 1411
mpc_, 1411
mps_, 1412
msg_data_, 1412
msg_version_, 1412
msn_, 1412
mtd_, 1412
mtt_, 1412
park_start_time_, 1412
pdt_, 1412
pfd_, 1412
pfm_, 1413
pns_, 1413
psa_, 1413
psi_, 1413
pth_, 1413
pto_, 1413
sbm_, 1413
sbt_, 1413
sdc_, 1413
sds_, 1414
sensor_type_, 1414
set_, 1414
sfn_, 1414
sgt_, 1414
ssi_, 1414
surface_start_time_, 1414
wmt_, 1414
xas_, 1414
XMfaMissionCfg, 1398
util::xmsg::XMissionCmdEcho, 1415
~XMissionCmdEcho, 1416
CheckXMsgData, 1417
DecodeXMsgData, 1417
GenerateListingString, 1418
id_dive, 1418
id_dive_, 1424
id_instrument, 1419
id_instrument_, 1424
id_xmsg_type, 1420
id_xmsg_type_, 1425
set_id_dive, 1421
set_id_instrument, 1421
set_id_xmsg_type, 1421
set_xmsg_size_bytes, 1422
XMissionCmdEcho, 1416
xmsg_size_bytes, 1423
xmsg_size_bytes_, 1425
util::xmsg::XMissionConfiguration, 1425
~XMissionConfiguration, 1427
alamo_mission_configuration_, 1437
CheckXMsgData, 1427
DecodeXMsgData, 1428
GenerateListingString, 1428
id_dive, 1429
id_dive_, 1437
id_instrument, 1430
id_instrument_, 1437
id_xmsg_type, 1431
id_xmsg_type_, 1438
set_id_dive, 1431
set_id_instrument, 1432
set_id_xmsg_type, 1432
set_xmsg_index, 1433
set_xmsg_size_bytes, 1434

XMissionConfiguration, 1426
xmsg_index, 1435
xmsg_index_, 1438
xmsg_size_bytes, 1436
xmsg_size_bytes_, 1438
util::xmsg::XPresCodeCounts, 1438
~XPresCodeCounts, 1440
elapsed_seconds_since_reference_time, 1440
elapsed_seconds_since_reference_time_, 1443
GenerateListingString, 1440
GetPhaseCodeName, 1441
phase_code, 1441
phase_code_, 1443
phase_code_name, 1441
phase_code_names_, 1443
pressure_counts_, 1443
PressureValue, 1442
XPresCodeCounts, 1439
util::xmsg::XPresCounts, 1444
~XPresCounts, 1445
elapsed_seconds_since_reference_time, 1445
elapsed_seconds_since_reference_time_, 1447
GenerateListingString, 1445
pressure_counts_, 1447
PressureValue, 1446
XPresCounts, 1445
util::xmsg::XProfilePressure, 1447
~XProfilePressure, 1449
CheckXMsgData, 1450
DecodeXMsgData, 1450
id_dive, 1451
id_dive_, 1464
id_instrument, 1452
id_instrument_, 1464
id_xmsg_type, 1453
id_xmsg_type_, 1464
num_samples, 1453
raw_sensor_data, 1454
raw_sensor_data_, 1465
Sample, 1454
SampleValue, 1454
SampleValueAsPressure, 1455
SampleValueAsSalinity, 1455
SampleValueAsTemperature, 1456
sensor_data_format, 1456
sensor_data_format_, 1465
set_id_dive, 1457
set_id_instrument, 1457
set_id_xmsg_type, 1457
set_raw_sensor_data, 1458
set_sensor_data_format, 1459
set_xmsg_index, 1459
set_xmsg_size_bytes, 1460
UnpackSensorData, 1461
xmsg_index, 1463
xmsg_index_, 1465
xmsg_size_bytes, 1463
xmsg_size_bytes_, 1465
XProfilePressure, 1449
util::xmsg::XProfileSalinity, 1466
~XProfileSalinity, 1468
CheckXMsgData, 1468
DecodeXMsgData, 1469
id_dive, 1469
id_dive_, 1483
id_instrument, 1470
id_instrument_, 1483
id_xmsg_type, 1471
id_xmsg_type_, 1483
num_samples, 1472
raw_sensor_data, 1472
raw_sensor_data_, 1483
Sample, 1472
SampleValue, 1473
SampleValueAsPressure, 1473
SampleValueAsSalinity, 1474
SampleValueAsTemperature, 1474
sensor_data_format, 1475
sensor_data_format_, 1484
set_id_dive, 1475
set_id_instrument, 1475
set_id_xmsg_type, 1476
set_raw_sensor_data, 1477
set_sensor_data_format, 1477
set_xmsg_index, 1478
set_xmsg_size_bytes, 1479
UnpackSensorData, 1480
xmsg_index, 1481
xmsg_index_, 1484
xmsg_size_bytes, 1482
xmsg_size_bytes_, 1484
XProfileSalinity, 1467
util::xmsg::XProfileTemperature, 1485
~XProfileTemperature, 1487
CheckXMsgData, 1487
DecodeXMsgData, 1488
id_dive, 1488
id_dive_, 1502
id_instrument, 1489
id_instrument_, 1502
id_xmsg_type, 1490
id_xmsg_type_, 1502
num_samples, 1491
raw_sensor_data, 1491
raw_sensor_data_, 1502
Sample, 1491
SampleValue, 1492
SampleValueAsPressure, 1492
SampleValueAsSalinity, 1493
SampleValueAsTemperature, 1493
sensor_data_format, 1494
sensor_data_format_, 1503
set_id_dive, 1494
set_id_instrument, 1494
set_id_xmsg_type, 1495
set_raw_sensor_data, 1496
set_sensor_data_format, 1496
set_xmsg_index, 1497
set_xmsg_size_bytes, 1498
UnpackSensorData, 1499
xmsg_index, 1500
xmsg_index_, 1503
xmsg_size_bytes, 1501
xmsg_size_bytes_, 1503
XProfileTemperature, 1486
util::xmsg::XPump, 1503
~XPump, 1505
AppendAllElements, 1506
AppendElement, 1506
AssignElement, 1506
CheckXMsgData, 1507
DecodeXMsgData, 1507
Element_t, 1505
elements, 1508
elements_, 1518
GenerateListingComment, 1508
GenerateListingString, 1509
id_dive, 1509
id_dive_, 1518
id_instrument, 1510
id_instrument_, 1518
id_xmsg_type, 1511
id_xmsg_type_, 1518
num_elements, 1512

pump_packing_format, 1512
pump_packing_format_, 1518
set_id_dive, 1512
set_id_instrument, 1513
set_id_xmsg_type, 1513
set_xmsg_index, 1514
set_xmsg_size_bytes, 1515
xmsg_index, 1516
xmsg_index_, 1518
xmsg_size_bytes, 1517
xmsg_size_bytes_, 1518
XPump, 1505
util::xmsg::XPumpCounts, 1519
~XPumpCounts, 1520
case_vac_after_pumping_cinhg, 1520
case_vac_after_pumping_cinhg_, 1524
case_vac_after_pumping_inhg, 1520
case_vac_before_pumping_cinhg, 1521
case_vac_before_pumping_cinhg_, 1524
case_vac_before_pumping_inhg, 1521
GenerateListingString, 1522
phase_code_, 1524
pressure_counts_, 1524
PressureValue, 1522
pump_current_a, 1523
pump_current_ma_, 1524
pump_duration_sec_, 1524
pump_voltage_cv_, 1525
pump_voltage_v, 1523
XPumpCounts, 1520
util::xmsg::XRise, 1525
~XRise, 1527
CheckXMsgData, 1528
data_pack_format, 1528
data_pack_format_, 1544
DecodeXMsgData, 1529
GenerateCurrentUtcTimeIso8601String, 1529
GenerateIso8601String, 1530
GenerateJulianDay, 1531
GenerateListingComment, 1532
GenerateListingString, 1532
GetCurrentUtcTime, 1533
id_dive, 1533
id_dive_, 1544
id_instrument, 1534
id_instrument_, 1545
id_xmsg_type, 1535
id_xmsg_type_, 1545
num_samples, 1536
sensor_data, 1536
sensor_data_, 1545
set_data_pack_format, 1537
set_id_dive, 1537
set_id_instrument, 1538
set_id_xmsg_type, 1538
set_start_time, 1539
set_xmsg_index, 1540
set_xmsg_size_bytes, 1540
start_time, 1542
start_time_, 1545
UnpackData, 1542
xmsg_index, 1543
xmsg_index_, 1545
xmsg_size_bytes, 1543
xmsg_size_bytes_, 1545
XRise, 1527
util::xmsg::XSensorTripletCodeCounts, 1546
~XSensorTripletCodeCounts, 1547
elapsed_seconds_since_reference_time, 1548
elapsed_seconds_since_reference_time_, 1552
GenerateListingString, 1548
GetPhaseCodeName, 1549
phase_code, 1550
phase_code_, 1552
phase_code_name, 1550
phase_code_names_, 1552
pressure_counts_, 1552
PressureValue, 1550
salinity_counts_, 1553
SalinityValue, 1551
temperature_counts_, 1553
TemperatureValue, 1551
XSensorTripletCodeCounts, 1547
util::xmsg::XSurfaceData, 1553
~XSurfaceData, 1555
CheckXMsgData, 1556
data_pack_format, 1556
data_pack_format_, 1572
DecodeXMsgData, 1557
GenerateCurrentUtcTimeIso8601String, 1557
GenerateIso8601String, 1558
GenerateJulianDay, 1559
GenerateListingComment, 1560
GenerateListingString, 1560
GetCurrentUtcTime, 1561
id_dive, 1561
id_dive_, 1572
id_instrument, 1562
id_instrument_, 1573
id_xmsg_type, 1563
id_xmsg_type_, 1573
num_samples, 1564
sensor_data, 1564
sensor_data_, 1573
set_data_pack_format, 1565
set_id_dive, 1565
set_id_instrument, 1566
set_id_xmsg_type, 1566
set_start_time, 1567
set_xmsg_index, 1568
set_xmsg_size_bytes, 1568
start_time, 1570
start_time_, 1573
UnpackData, 1570
xmsg_index, 1571
xmsg_index_, 1573
xmsg_size_bytes, 1571
xmsg_size_bytes_, 1573
XSurfaceData, 1555
util::xmsg::XValve, 1574
AppendAllElements, 1576
AppendElement, 1576
AssignElement, 1576
CheckXMsgData, 1577
DecodeXMsgData, 1577
Element_t, 1575
elements, 1578
elements_, 1587
GenerateListingComment, 1578
id_dive, 1579
id_dive_, 1587
id_instrument, 1580
id_instrument_, 1587
id_xmsg_type, 1581
id_xmsg_type_, 1587
num_elements, 1581
set_id_dive, 1582
set_id_instrument, 1582
set_id_xmsg_type, 1582
set_xmsg_index, 1583
set_xmsg_size_bytes, 1584
xmsg_index, 1585
xmsg_index_, 1588
xmsg_size_bytes, 1586

xmsg_size_bytes_, 1588
XValve, 1575
util::xmsg::XValveCounts, 1588
~XValveCounts, 1589
case_vac_after_pumping_inhg, 1590
case_vac_after_valve_cinhg_, 1590
case_vac_before_pumping_inhg, 1590
case_vac_before_valve_cinhg_, 1590
phase_code_, 1590
pressure_counts_, 1591
PressureValue, 1590
valve_duration_sec_, 1591
XValveCounts, 1589
util::xmsg::XWave, 1591
~XWave, 1593
assf_, 1604
ASSF_RAW, 1604
ASSF_SPECTRAL_V1, 1604
ASSF_SPECTRAL_V2, 1604
CheckXMsgData, 1594
DecodeXMsgData, 1594
FixFuckup9000, 1595
FSAM, 1605
gain_accel_q3_, 1605
GenerateListingString, 1595
GSSF, 1605
id_dive, 1596
id_dive_, 1605
id_instrument, 1597
id_instrument_, 1605
id_xmsg_type, 1598
id_xmsg_type_, 1605
MSSF, 1605
sample_mode_, 1605
sample_rate_base_, 1606
sample_rate_div_, 1606
sample_rate_effective_, 1606
segment_len_, 1606
segment_len_pow_, 1606
set_id_dive, 1598
set_id_instrument, 1599
set_id_xmsg_type, 1599
set_xmsg_index, 1600
set_xmsg_size_bytes, 1601
wave_data_spectral_, 1606
xmsg_index, 1602
xmsg_index_, 1606
xmsg_size_bytes, 1603
xmsg_size_bytes_, 1606
XWave, 1593
util::xmsg::XWaveRawSample, 1607
~XWaveRawSample, 1608
accel, 1608
GenerateListingString, 1608
magnt, 1608
omega, 1609
XWaveRawSample, 1608
util::xmsg::XWaveSpectralSample, 1609
UTIL_POS_INVALID_LAT
PositionUtils.h, 2168
UTIL_POS_INVALID_LON
PositionUtils.h, 2168
UTIL_POS_INVALID_STDERR
PositionUtils.h, 2168
UTIL_POS_INVALID_VAR
PositionUtils.h, 2168
utility/ContainerUtils.h, 2120, 2121
utility/DataBuf.h, 2122, 2123
utility/DateUtils.h, 2124, 2125
utility/FileSystemUtils.h, 2126, 2127
utility/KmlUtil.cpp, 2129
utility/KmlUtil.h, 2130, 2131
utility/Lockable.h, 2135, 2136
utility/Log.cpp, 2137
utility/Log.h, 2138, 2146
utility/MathUtils.h, 2153, 2154
utility/MessageException.h, 2155, 2156
utility/MysqlUtils.cpp, 2157
utility/MysqlUtils.h, 2158, 2159
utility/Position.h, 2163, 2164
utility/PositionUtils.cpp, 2166
utility/PositionUtils.h, 2167, 2169
utility/RecursiveLockable.h, 2170, 2172
utility/SignalHandler.h, 2173, 2174
utility/SqlTableColumnDescription.h, 2175, 2179
utility/SqlTableDescription.cpp, 2189
utility/SqlTableDescription.h, 2190, 2192
utility/StreamUtils.h, 2199, 2200
utility/StringUtils.h, 2201, 2202
utility/SynchronousQueue.h, 2202, 2203
utility/SystemUtils.h, 2205, 2207
utility/ThreadUtils.h, 2207, 2208
utility/VectorBuf.h, 2210, 2211

values
util::mysql::SqlTableDescription< T >, 404
valve_duration_sec_
util::xmsg::XValveCounts, 1591
VectorBuf
util::stream::VectorBuf< CharT, TraitsT >, 416
VectorBuf_c
util::stream::VectorBuf_c< CharT >, 417
vmime, 59
vmime::shared_ptr< F >, 370
element, 370

wave_data_spectral_
util::xmsg::XWave, 1606
what
util::log::MessageException, 335
wmo_instrument_type
argo::iridium::xfloat::XFloatMetadata, 1226
wmo_instrument_type_
argo::iridium::xfloat::XFloatMetadata, 1234
wmo_recorder_type
argo::iridium::xfloat::XFloatMetadata, 1226
wmo_recorder_type_
argo::iridium::xfloat::XFloatMetadata, 1234
wmt_
util::xmsg::XMfaMissionCfg, 1414
WriteFiles
argo::iridium::xfloat::XFloatMailMessage, 1121
WriteKmlFile
argo::iridium::xfloat::XFloat, 958
WriteKmlMasterFile
argo::iridium::xfloat::XFloat, 958
WriteListingFile
argo::iridium::xfloat::XFloatDiveData, 1035
WritePhyFile
argo::iridium::xfloat::XFloatDiveData, 1036
WriteRawFile
argo::iridium::xfloat::XFloatMailMessage, 1121
WriteSbdFile
argo::iridium::xfloat::XFloatMailMessage, 1122
WriteStatusFile
argo::iridium::xfloat::XFloat, 958

X_PP_STR
Log.h, 2146
x_version_float
util::xmsg::XArgo, 531
x_version_major
util::xmsg::XArgo, 531
x_version_major_
util::xmsg::XArgo, 538

x_version_minor
util::xmsg::XArgo, 532
x_version_minor_
util::xmsg::XArgo, 538
XAcousticCacst
util::xmsg::XAcousticCacst, 419
XAcousticPosition
util::xmsg::XAcousticPosition, 422
XAcousticWindow
util::xmsg::XAcousticWindow, 434
XAlamoBit
util::xmsg::XAlamoBit, 438
XAlamoBit2
util::xmsg::XAlamoBit2, 447
XAlamoEngineering
util::xmsg::XAlamoEngineering, 459
XAlamoMissionConfiguration
util::xmsg::XAlamoMissionConfiguration, 484
XArgo
util::xmsg::XArgo, 500
XArgo.h
XARGO_INSRUMENT_TYPE_ALAMO, 1904
XARGO_INSRUMENT_TYPE_DEEP_ARGO, 1904
XARGO_INSRUMENT_TYPE_SOLO2, 1904
XARGO_INSRUMENT_TYPE_UNKNOWN, 1904
XArgoInstrumentType_t, 1903
XARGO_INSRUMENT_TYPE_ALAMO
XArgo.h, 1904
XARGO_INSRUMENT_TYPE_DEEP_ARGO
XArgo.h, 1904
XARGO_INSRUMENT_TYPE_SOLO2
XArgo.h, 1904
XARGO_INSRUMENT_TYPE_UNKNOWN
XArgo.h, 1904
XArgoInstrumentType_t
XArgo.h, 1903
xas_
util::xmsg::XAlamoMissionConfiguration, 495
util::xmsg::XMfaMissionCfg, 1414
XCompositeDrift
util::xmsg::XCompositeDrift, 546
XCompositeFall
util::xmsg::XCompositeFall, 561
XCompositeHiResProfile
util::xmsg::XCompositeHiResProfile, 567
XCompositeIridiumPosition
util::xmsg::XCompositeIridiumPosition, 584
XCompositeIridiumPosition.h
XMSG_POS_INVALID_LAT, 1928
XMSG_POS_INVALID_LON, 1928
XMSG_POS_INVALID_STDERR, 1928
XMSG_POS_INVALID_VAR, 1928
XCompositeProfile
util::xmsg::XCompositeProfile, 605
XCompositePump
util::xmsg::XCompositePump, 621
XCompositeRise
util::xmsg::XCompositeRise, 624
XCompositeSurfaceData
util::xmsg::XCompositeSurfaceData, 629
XDefinitions.h
i08, 1770
i16, 1770
i24, 1770
i32, 1770
s1970, 1770
s2000, 1770
u08, 1771
u16, 1771
u24, 1771
u32, 1771
XMSG_BAD_DBL, 1771
XMSG_BAD_FLT, 1772
XMSG_BAD_I08, 1772
XMSG_BAD_I16, 1772
XMSG_BAD_I32, 1772
XMSG_BAD_U08, 1772
XMSG_BAD_U16, 1772
XMSG_BAD_U32, 1772
XMSG_DEBUG_AGGREGATE_TIMESERIES_DATA, 1772
XMSG_DEBUG_ALAMO_MISS_GFG, 1772
XMSG_DEBUG_BIST, 1773
XMSG_DEBUG_ENG_SURF, 1773
XMSG_DEBUG_MFA_MISS_GFG, 1773
XMSG_DEBUG_SENSOR_CURVATURE, 1773
XMSG_DEBUG_SURF_PRES_AGGR_DATA, 1773
XMSG_DIVE_UNKNOWN, 1773
XMSG_ENABLE_MULTI_PROF, 1773
XMSG_ENG_BAD_VER, 1774
XMSG_GAIN_INVALID, 1774
XMSG_HEADER, 1774
XMSG_HEADER2, 1774
XMSG_ID_UNKNOWN, 1774
XMSG_INIT_SAM_N_RESERVE, 1775
XMSG_MAX_DRFT, 1775
XMSG_MAX_FALL, 1775
XMSG_MAX_GPS, 1775
XMSG_MAX_MAIL, 1775
XMSG_MAX_MFA_MISS, 1775
XMSG_MAX_MISS, 1775
XMSG_MAX_PROF, 1775
XMSG_MAX_PROF_HIRES, 1776
XMSG_MAX_PUMP, 1776
XMSG_MAX_RISE, 1776
XMSG_MAX_SURF_PRES, 1776
XMSG_MAX_VALVE, 1776
XMSG_MAX_WAVE, 1776
XMSG_OFFS_INVALID, 1776
XMSG_PHY, 1776
XMSG_SBD_MIN_BYTES, 1777
XMSG_SECTION, 1777
XMSG_SECTION_BREAK, 1777
XMSG_SENS_BAD, 1777
XMSG_SENS_BAD_PRES, 1777
XMSG_SENS_BAD_SALN, 1778
XMSG_SENS_BAD_TEMP, 1778
XMSG_SENS_MAX_PRES, 1778
XMSG_SENS_MAX_SALN, 1778
XMSG_SENS_MAX_SAMPLES, 1778
XMSG_SENS_MAX_SUB, 1778
XMSG_SENS_MAX_TEMP, 1778
XMSG_SENS_MIN_PRES, 1779
XMSG_SENS_MIN_SALN, 1779
XMSG_SENS_MIN_TEMP, 1779
XMSG_VERSION_UNKNOWN, 1779
XDriftPressure
util::xmsg::XDriftPressure, 641
XDriftSalinity
util::xmsg::XDriftSalinity, 659
XDriftTemperature
util::xmsg::XDriftTemperature, 678
XEngAbort
util::xmsg::XEngAbort, 697
XEngAscent
util::xmsg::XEngAscent, 708
XEngBIT
util::xmsg::XEngBIT, 727
XEngBITFail
util::xmsg::XEngBITFail, 739
XEngBITPass
util::xmsg::XEngBITPass, 751
XEngDescent
util::xmsg::XEngDescent, 765

XEngDiagnostic
util::xmsg::XEngDiagnostic, 785
XEngDrift
util::xmsg::XEngDrift, 797
XEngErrorCode
util::xmsg::XEngSurface::XEngErrorCode, 815
XEngIce
util::xmsg::XEngIce, 823
XEngIceErrorCode
util::xmsg::XEngIce::XEngIceErrorCode, 839
XEngStandard
util::xmsg::XEngStandard, 846
XEngSurface
util::xmsg::XEngSurface, 860
XFall
util::xmsg::XFall, 882
XFallRiseDataPackFormat_t
util::xmsg, 54
XFloat
argo::iridium::xfloat::XFloat, 910
XFloatDefinitions.h
XFLT_DAEMON_FLUSH_LOG_SEC, 1660
XFLT_DIVE_UNKNOWN, 1660
XFLT_GAIN_INVALID, 1660
XFLT_ID_UNKNOWN, 1660
XFLT_OFFS_INVALID, 1660
XFLT_PID_INVALID, 1661
XFLT_POS_INVALID_LAT, 1661
XFLT_POS_INVALID_LON, 1661
XFLT_QUEUE_SIZE_MAIL_PROCESSING, 1661
XFLT_QUEUE_SIZE_TELEMETRY, 1661
XFLT_SBD_SESSION_STATUS_CODE_UNKNOWN, 1661
XFLT_SBD_SIZE_BYTES_UNKNOWN, 1661
XFLT_THREAD_SLEEP_MS, 1661
XFloatDiveData
argo::iridium::xfloat::XFloatDiveData, 982
XFloatMailMessage
argo::iridium::xfloat::XFloatMailMessage, 1059
XFloatMailMessage.h
XMAIL_DFLT_MSG_HEADER_VALID_SNDR_REGEX, 1687
XMAIL_DFLT_MSG_HEADER_VALID_SUBJ_REGEX, 1687
XMAIL_DFLT_RAW_FILE_FORMAT, 1688
XMAIL_DFLT_RAW_FILE_GENERATRION_ENABLE, 1688
XMAIL_DFLT_SBD_FILE_GENERATRION_ENABLE, 1688
XMAIL_DFLT_SBD_FILE_VALID_NAME_REGEX, 1688
XFloatMailProcessingService
argo::iridium::xfloat::XFloatMailProcessingService, 1139
XFloatMailRawFileFormat_t
argo::iridium::xfloat, 39
XFloatMailRetrievalService
argo::iridium::xfloat::XFloatMailRetrievalService, 1158
XFloatMetadata
argo::iridium::xfloat::XFloatMetadata, 1187
XFloatPostProcessingService
argo::iridium::xfloat::XFloatPostProcessingService, 1236
XFloatTelemetryDecoderService
argo::iridium::xfloat::XFloatTelemetryDecoderService, 1246
XFLT_DAEMON_FLUSH_LOG_SEC
XFloatDefinitions.h, 1660
XFLT_DIVE_UNKNOWN
XFloatDefinitions.h, 1660
XFLT_GAIN_INVALID
XFloatDefinitions.h, 1660
XFLT_ID_UNKNOWN
XFloatDefinitions.h, 1660
XFLT_OFFS_INVALID
XFloatDefinitions.h, 1660
XFLT_PID_INVALID
XFloatDefinitions.h, 1661
XFLT_POS_INVALID_LAT
XFloatDefinitions.h, 1661
XFLT_POS_INVALID_LON
XFloatDefinitions.h, 1661
XFLT_QUEUE_SIZE_MAIL_PROCESSING
XFloatDefinitions.h, 1661
XFLT_QUEUE_SIZE_TELEMETRY
XFloatDefinitions.h, 1661
XFLT_SBD_SESSION_STATUS_CODE_UNKNOWN
XFloatDefinitions.h, 1661
XFLT_SBD_SIZE_BYTES_UNKNOWN
XFloatDefinitions.h, 1661
XFLT_THREAD_SLEEP_MS
XFloatDefinitions.h, 1661
XGPS
util::xmsg::XGPS, 1263
XHiResPressure
util::xmsg::XHiResPressure, 1309
XHiResSalinity
util::xmsg::XHiResSalinity, 1327
XHiResTemperature
util::xmsg::XHiResTemperature, 1346
XMAIL_DFLT_MSG_HEADER_VALID_SNDR_REGEX
XFloatMailMessage.h, 1687
XMAIL_DFLT_MSG_HEADER_VALID_SUBJ_REGEX
XFloatMailMessage.h, 1687
XMAIL_DFLT_RAW_FILE_FORMAT
XFloatMailMessage.h, 1688
XMAIL_DFLT_RAW_FILE_GENERATRION_ENABLE
XFloatMailMessage.h, 1688
XMAIL_DFLT_SBD_FILE_GENERATRION_ENABLE
XFloatMailMessage.h, 1688
XMAIL_DFLT_SBD_FILE_VALID_NAME_REGEX
XFloatMailMessage.h, 1688
XMAIL_RAW_FORMAT_EXTENDED
argo::iridium::xfloat, 39
XMAIL_RAW_FORMAT_STANDARD
argo::iridium::xfloat, 39
XMailSbdByte_t
argo::iridium::xfloat, 39
XMailSbdData_t
argo::iridium::xfloat, 39
XMessage
util::xmsg::XMessage, 1365
XMessageIndexable
util::xmsg::XMessageIndexable, 1375, 1376
XMfaMissionCfg
util::xmsg::XMfaMissionCfg, 1398
XMissionCmdEcho
util::xmsg::XMissionCmdEcho, 1416
XMissionConfiguration
util::xmsg::XMissionConfiguration, 1426
xmsg_acoustic_positions_
argo::iridium::xfloat::XFloatDiveData, 1046
xmsg_argo_data_
argo::iridium::xfloat::XFloatDiveData, 1046
XMSG_BAD_DBL
XDefinitions.h, 1771
XMSG_BAD_FLT
XDefinitions.h, 1772
XMSG_BAD_I08
XDefinitions.h, 1772
XMSG_BAD_I16
XDefinitions.h, 1772
XMSG_BAD_I32

XDefinitions.h, 1772
XMSG_BAD_U08
XDefinitions.h, 1772
XMSG_BAD_U16
XDefinitions.h, 1772
XMSG_BAD_U32
XDefinitions.h, 1772
xmsg_composite_drift_data_
argo::iridium::xfloat::XFloatDiveData, 1046
xmsg_composite_profile_binned_
argo::iridium::xfloat::XFloatDiveData, 1047
xmsg_composite_profile_unbinned_
argo::iridium::xfloat::XFloatDiveData, 1047
XMSG_DEBUG_AGGREGATE_TIMESERIES_DATA
XDefinitions.h, 1772
XMSG_DEBUG_ALAMO_MISS_GFG
XDefinitions.h, 1772
XMSG_DEBUG_BIST
XDefinitions.h, 1773
XMSG_DEBUG_ENG_SURF
XDefinitions.h, 1773
XMSG_DEBUG_MFA_MISS_GFG
XDefinitions.h, 1773
XMSG_DEBUG_SENSOR_CURVATURE
XDefinitions.h, 1773
XMSG_DEBUG_SURF_PRES_AGGR_DATA
XDefinitions.h, 1773
XMSG_DIVE_UNKNOWN
XDefinitions.h, 1773
XMSG_ENABLE_MULTI_PROF
XDefinitions.h, 1773
XMSG_ENG_BAD_VER
XDefinitions.h, 1774
xmsg_engineering_data_abort_
argo::iridium::xfloat::XFloatDiveData, 1047
xmsg_engineering_data_ascent_
argo::iridium::xfloat::XFloatDiveData, 1047
xmsg_engineering_data_bit_fail_
argo::iridium::xfloat::XFloatDiveData, 1047
xmsg_engineering_data_bit_passed_
argo::iridium::xfloat::XFloatDiveData, 1048
xmsg_engineering_data_descent_
argo::iridium::xfloat::XFloatDiveData, 1048
xmsg_engineering_data_diagnostic_dive_
argo::iridium::xfloat::XFloatDiveData, 1048
xmsg_engineering_data_drift_
argo::iridium::xfloat::XFloatDiveData, 1048
xmsg_engineering_data_ice_
argo::iridium::xfloat::XFloatDiveData, 1048
xmsg_engineering_data_standard_dive_
argo::iridium::xfloat::XFloatDiveData, 1048
xmsg_engineering_data_surface_
argo::iridium::xfloat::XFloatDiveData, 1049
xmsg_fall_time_series_
argo::iridium::xfloat::XFloatDiveData, 1049
XMSG_GAIN_INVALID
XDefinitions.h, 1774
xmsg_gps_positions_
argo::iridium::xfloat::XFloatDiveData, 1049
XMSG_HEADER
XDefinitions.h, 1774
XMSG_HEADER2
XDefinitions.h, 1774
XMSG_ID_UNKNOWN
XDefinitions.h, 1774
xmsg_index
util::xmsg::AbstractXFallRise, 81
util::xmsg::AbstractXSensorData, 99
util::xmsg::AuxiliarySensorData, 151
util::xmsg::XDriftPressure, 655
util::xmsg::XDriftSalinity, 673
util::xmsg::XDriftTemperature, 692
util::xmsg::XFall, 899
util::xmsg::XHiResPressure, 1323
util::xmsg::XHiResSalinity, 1341
util::xmsg::XHiResTemperature, 1360
util::xmsg::XMessageIndexable, 1383
util::xmsg::XMissionConfiguration, 1435
util::xmsg::XProfilePressure, 1463
util::xmsg::XProfileSalinity, 1481
util::xmsg::XProfileTemperature, 1500
util::xmsg::XPump, 1516
util::xmsg::XRise, 1543
util::xmsg::XSurfaceData, 1571
util::xmsg::XValve, 1585
util::xmsg::XWave, 1602
xmsg_index_
util::xmsg::AbstractXFallRise, 83
util::xmsg::AbstractXSensorData, 102
util::xmsg::AuxiliarySensorData, 154
util::xmsg::XDriftPressure, 657
util::xmsg::XDriftSalinity, 676
util::xmsg::XDriftTemperature, 695
util::xmsg::XFall, 901
util::xmsg::XHiResPressure, 1325
util::xmsg::XHiResSalinity, 1344
util::xmsg::XHiResTemperature, 1363
util::xmsg::XMessageIndexable, 1385
util::xmsg::XMissionConfiguration, 1438
util::xmsg::XProfilePressure, 1465
util::xmsg::XProfileSalinity, 1484
util::xmsg::XProfileTemperature, 1503
util::xmsg::XPump, 1518
util::xmsg::XRise, 1545
util::xmsg::XSurfaceData, 1573
util::xmsg::XValve, 1588
util::xmsg::XWave, 1606
XMSG_INIT_SAM_N_RESERVE
XDefinitions.h, 1775
XMSG_MAX_DRFT
XDefinitions.h, 1775
XMSG_MAX_FALL
XDefinitions.h, 1775
XMSG_MAX_GPS
XDefinitions.h, 1775
XMSG_MAX_MAIL
XDefinitions.h, 1775
XMSG_MAX_MFA_MISS
XDefinitions.h, 1775
XMSG_MAX_MISS
XDefinitions.h, 1775
XMSG_MAX_PROF
XDefinitions.h, 1775
XMSG_MAX_PROF_HIRES
XDefinitions.h, 1776
XMSG_MAX_PUMP
XDefinitions.h, 1776
XMSG_MAX_RISE
XDefinitions.h, 1776
XMSG_MAX_SURF_PRES
XDefinitions.h, 1776
XMSG_MAX_VALVE
XDefinitions.h, 1776
XMSG_MAX_WAVE
XDefinitions.h, 1776
xmsg_mission_cfg_
argo::iridium::xfloat::XFloatDiveData, 1049
xmsg_mission_cmd_echoback_
argo::iridium::xfloat::XFloatDiveData, 1049
XMSG_OFFS_INVALID
XDefinitions.h, 1776
XMSG_PHY
XDefinitions.h, 1776
XMSG_POS_INVALID_LAT

XCompositeIridiumPosition.h, 1928
XMSG_POS_INVALID_LON
XCompositeIridiumPosition.h, 1928
XMSG_POS_INVALID_STDERR
XCompositeIridiumPosition.h, 1928
XMSG_POS_INVALID_VAR
XCompositeIridiumPosition.h, 1928
xmsg_pump_time_series_
argo::iridium::xfloat::XFloatDiveData, 1050
xmsg_rise_time_series_
argo::iridium::xfloat::XFloatDiveData, 1050
XMSG_SBD_MIN_BYTES
XDefinitions.h, 1777
XMSG_SECTION
XDefinitions.h, 1777
XMSG_SECTION_BREAK
XDefinitions.h, 1777
XMSG_SENS_BAD
XDefinitions.h, 1777
XMSG_SENS_BAD_PRES
XDefinitions.h, 1777
XMSG_SENS_BAD_SALN
XDefinitions.h, 1778
XMSG_SENS_BAD_TEMP
XDefinitions.h, 1778
XMSG_SENS_MAX_PRES
XDefinitions.h, 1778
XMSG_SENS_MAX_SALN
XDefinitions.h, 1778
XMSG_SENS_MAX_SAMPLES
XDefinitions.h, 1778
XMSG_SENS_MAX_SUB
XDefinitions.h, 1778
XMSG_SENS_MAX_TEMP
XDefinitions.h, 1778
XMSG_SENS_MIN_PRES
XDefinitions.h, 1779
XMSG_SENS_MIN_SALN
XDefinitions.h, 1779
XMSG_SENS_MIN_TEMP
XDefinitions.h, 1779
xmsg_size_bytes
util::xmsg::AbstractXFallRise, 81
util::xmsg::AbstractXSensorData, 100
util::xmsg::AuxiliarySensorData, 152
util::xmsg::XAcousticPosition, 430
util::xmsg::XAlamoEngineering, 472
util::xmsg::XArgo, 532
util::xmsg::XDriftPressure, 655
util::xmsg::XDriftSalinity, 674
util::xmsg::XDriftTemperature, 693
util::xmsg::XEngAbort, 704
util::xmsg::XEngAscent, 723
util::xmsg::XEngBIT, 736
util::xmsg::XEngBITFail, 748
util::xmsg::XEngBITPass, 760
util::xmsg::XEngDescent, 779
util::xmsg::XEngDiagnostic, 792
util::xmsg::XEngDrift, 811
util::xmsg::XEngIce, 836
util::xmsg::XEngStandard, 853
util::xmsg::XEngSurface, 875
util::xmsg::XFall, 899
util::xmsg::XGPS, 1301
util::xmsg::XHiResPressure, 1323
util::xmsg::XHiResSalinity, 1342
util::xmsg::XHiResTemperature, 1361
util::xmsg::XMessage, 1372
util::xmsg::XMessageIndexable, 1384
util::xmsg::XMissionCmdEcho, 1423
util::xmsg::XMissionConfiguration, 1436
util::xmsg::XProfilePressure, 1463
util::xmsg::XProfileSalinity, 1482
util::xmsg::XProfileTemperature, 1501
util::xmsg::XPump, 1517
util::xmsg::XRise, 1543
util::xmsg::XSurfaceData, 1571
util::xmsg::XValve, 1586
util::xmsg::XWave, 1603
xmsg_size_bytes_
util::xmsg::AbstractXFallRise, 83
util::xmsg::AbstractXSensorData, 102
util::xmsg::AuxiliarySensorData, 154
util::xmsg::XAcousticPosition, 432
util::xmsg::XAlamoEngineering, 474
util::xmsg::XArgo, 539
util::xmsg::XDriftPressure, 657
util::xmsg::XDriftSalinity, 676
util::xmsg::XDriftTemperature, 695
util::xmsg::XEngAbort, 706
util::xmsg::XEngAscent, 725
util::xmsg::XEngBIT, 737
util::xmsg::XEngBITFail, 749
util::xmsg::XEngBITPass, 761
util::xmsg::XEngDescent, 783
util::xmsg::XEngDiagnostic, 794
util::xmsg::XEngDrift, 814
util::xmsg::XEngIce, 837
util::xmsg::XEngStandard, 855
util::xmsg::XEngSurface, 880
util::xmsg::XFall, 901
util::xmsg::XGPS, 1307
util::xmsg::XHiResPressure, 1325
util::xmsg::XHiResSalinity, 1344
util::xmsg::XHiResTemperature, 1363
util::xmsg::XMessage, 1374
util::xmsg::XMessageIndexable, 1386
util::xmsg::XMissionCmdEcho, 1425
util::xmsg::XMissionConfiguration, 1438
util::xmsg::XProfilePressure, 1465
util::xmsg::XProfileSalinity, 1484
util::xmsg::XProfileTemperature, 1503
util::xmsg::XPump, 1518
util::xmsg::XRise, 1545
util::xmsg::XSurfaceData, 1573
util::xmsg::XValve, 1588
util::xmsg::XWave, 1606
xmsg_surface_time_series_
argo::iridium::xfloat::XFloatDiveData, 1050
XMSG_TYPE_ID_ALAMO_WAVE_C1
util::xmsg, 57
XMSG_TYPE_ID_ALAMO_WAVE_C3
util::xmsg, 58
XMSG_TYPE_ID_ARGO_DATA
util::xmsg, 58
XMSG_TYPE_ID_AUX_SENSOR_DATA
util::xmsg, 58
XMSG_TYPE_ID_ENG_ABORT
util::xmsg, 58
XMSG_TYPE_ID_ENG_BIT_FAIL
util::xmsg, 58
XMSG_TYPE_ID_ENG_BIT_PASS
util::xmsg, 58
XMSG_TYPE_ID_ENG_DIAGNOSTIC
util::xmsg, 58
XMSG_TYPE_ID_ENG_DIVE
util::xmsg, 58
XMSG_TYPE_ID_FALL0
util::xmsg, 56
XMSG_TYPE_ID_FALL1
util::xmsg, 56
XMSG_TYPE_ID_FALL2
util::xmsg, 56
XMSG_TYPE_ID_FALL3

util::xmsg, 56
XMSG_TYPE_ID_FALL4
util::xmsg, 56
XMSG_TYPE_ID_FALL5
util::xmsg, 56
XMSG_TYPE_ID_FALL6
util::xmsg, 56
XMSG_TYPE_ID_FALL7
util::xmsg, 56
XMSG_TYPE_ID_FALL8
util::xmsg, 56
XMSG_TYPE_ID_FALL9
util::xmsg, 56
XMSG_TYPE_ID_FALLA
util::xmsg, 56
XMSG_TYPE_ID_FALLB
util::xmsg, 56
XMSG_TYPE_ID_FALLC
util::xmsg, 56
XMSG_TYPE_ID_FALLD
util::xmsg, 56
XMSG_TYPE_ID_FALLE
util::xmsg, 56
XMSG_TYPE_ID_FALLF
util::xmsg, 56
XMSG_TYPE_ID_GPS_ABORT
util::xmsg, 55
XMSG_TYPE_ID_GPS_BITEST
util::xmsg, 55
XMSG_TYPE_ID_GPS_DIAGNOSTIC
util::xmsg, 55
XMSG_TYPE_ID_GPS_ENDOFDIVE
util::xmsg, 55
XMSG_TYPE_ID_GPS_STARTOFDIVE
util::xmsg, 55
XMSG_TYPE_ID_MISS_CMD_ECHO
util::xmsg, 58
XMSG_TYPE_ID_MISS_CONF0
util::xmsg, 58
XMSG_TYPE_ID_MISS_CONF1
util::xmsg, 58
XMSG_TYPE_ID_MISS_CONF2
util::xmsg, 58
XMSG_TYPE_ID_MISS_CONF3
util::xmsg, 58
XMSG_TYPE_ID_MISS_CONF4
util::xmsg, 58
XMSG_TYPE_ID_MISS_CONF5
util::xmsg, 58
XMSG_TYPE_ID_MISS_CONF6
util::xmsg, 58
XMSG_TYPE_ID_MISS_CONF7
util::xmsg, 58
XMSG_TYPE_ID_MIZ_ACOUSTIC
util::xmsg, 58
XMSG_TYPE_ID_PRES0
util::xmsg, 55
XMSG_TYPE_ID_PRES1
util::xmsg, 55
XMSG_TYPE_ID_PRES2
util::xmsg, 55
XMSG_TYPE_ID_PRES3
util::xmsg, 55
XMSG_TYPE_ID_PRES4
util::xmsg, 55
XMSG_TYPE_ID_PRES5
util::xmsg, 55
XMSG_TYPE_ID_PRES6
util::xmsg, 55
XMSG_TYPE_ID_PRES7
util::xmsg, 55
XMSG_TYPE_ID_PRES8
util::xmsg, 55
XMSG_TYPE_ID_PRES9
util::xmsg, 55
XMSG_TYPE_ID_PRES_DRIFT8
util::xmsg, 57
XMSG_TYPE_ID_PRES_DRIFT9
util::xmsg, 57
XMSG_TYPE_ID_PRES_DRIFTA
util::xmsg, 57
XMSG_TYPE_ID_PRES_DRIFTB
util::xmsg, 57
XMSG_TYPE_ID_PRES_DRIFTC
util::xmsg, 57
XMSG_TYPE_ID_PRES_DRIFTD
util::xmsg, 57
XMSG_TYPE_ID_PRES_DRIFTE
util::xmsg, 57
XMSG_TYPE_ID_PRES_DRIFTF
util::xmsg, 57
XMSG_TYPE_ID_PRES_HI_RES0
util::xmsg, 57
XMSG_TYPE_ID_PRES_HI_RES1
util::xmsg, 57
XMSG_TYPE_ID_PRES_HI_RES2
util::xmsg, 57
XMSG_TYPE_ID_PRES_HI_RES3
util::xmsg, 57
XMSG_TYPE_ID_PRES_HI_RES4
util::xmsg, 57
XMSG_TYPE_ID_PRES_HI_RES5
util::xmsg, 57
XMSG_TYPE_ID_PRES_HI_RES6
util::xmsg, 57
XMSG_TYPE_ID_PRES_HI_RES7
util::xmsg, 57
XMSG_TYPE_ID_PRES_SURF0
util::xmsg, 56
XMSG_TYPE_ID_PRES_SURF1
util::xmsg, 56
XMSG_TYPE_ID_PRES_SURF2
util::xmsg, 56
XMSG_TYPE_ID_PRES_SURF3
util::xmsg, 56
XMSG_TYPE_ID_PRES_SURF4
util::xmsg, 57
XMSG_TYPE_ID_PRES_SURF5
util::xmsg, 57
XMSG_TYPE_ID_PRES_SURF6
util::xmsg, 57
XMSG_TYPE_ID_PRES_SURF7
util::xmsg, 57
XMSG_TYPE_ID_PRES_SURF8
util::xmsg, 57
XMSG_TYPE_ID_PRES_SURF9
util::xmsg, 57
XMSG_TYPE_ID_PRES_SURFA
util::xmsg, 57
XMSG_TYPE_ID_PRES_SURFB
util::xmsg, 57
XMSG_TYPE_ID_PRES_SURFC
util::xmsg, 57
XMSG_TYPE_ID_PRES_SURFD
util::xmsg, 57
XMSG_TYPE_ID_PRES_SURFE
util::xmsg, 57
XMSG_TYPE_ID_PRES_SURFF
util::xmsg, 57
XMSG_TYPE_ID_PRESA
util::xmsg, 55
XMSG_TYPE_ID_PRESB
util::xmsg, 55
XMSG_TYPE_ID_PRESC

util::xmsg, 55
XMSG_TYPE_ID_PRESD
util::xmsg, 55
XMSG_TYPE_ID_PRESE
util::xmsg, 55
XMSG_TYPE_ID_PRESF
util::xmsg, 55
XMSG_TYPE_ID_PUMP0
util::xmsg, 56
XMSG_TYPE_ID_PUMP1
util::xmsg, 56
XMSG_TYPE_ID_PUMP2
util::xmsg, 56
XMSG_TYPE_ID_PUMP3
util::xmsg, 56
XMSG_TYPE_ID_PUMP4
util::xmsg, 56
XMSG_TYPE_ID_PUMP5
util::xmsg, 56
XMSG_TYPE_ID_PUMP6
util::xmsg, 56
XMSG_TYPE_ID_PUMP7
util::xmsg, 56
XMSG_TYPE_ID_PUMP8
util::xmsg, 56
XMSG_TYPE_ID_PUMP9
util::xmsg, 56
XMSG_TYPE_ID_PUMPA
util::xmsg, 56
XMSG_TYPE_ID_PUMPB
util::xmsg, 56
XMSG_TYPE_ID_PUMPC
util::xmsg, 56
XMSG_TYPE_ID_PUMPD
util::xmsg, 56
XMSG_TYPE_ID_PUMPE
util::xmsg, 56
XMSG_TYPE_ID_PUMPF
util::xmsg, 56
XMSG_TYPE_ID_RISE0
util::xmsg, 56
XMSG_TYPE_ID_RISE1
util::xmsg, 56
XMSG_TYPE_ID_RISE2
util::xmsg, 56
XMSG_TYPE_ID_RISE3
util::xmsg, 56
XMSG_TYPE_ID_RISE4
util::xmsg, 56
XMSG_TYPE_ID_RISE5
util::xmsg, 56
XMSG_TYPE_ID_RISE6
util::xmsg, 56
XMSG_TYPE_ID_RISE7
util::xmsg, 56
XMSG_TYPE_ID_RISE8
util::xmsg, 56
XMSG_TYPE_ID_RISE9
util::xmsg, 56
XMSG_TYPE_ID_RISEA
util::xmsg, 56
XMSG_TYPE_ID_RISEB
util::xmsg, 56
XMSG_TYPE_ID_RISEC
util::xmsg, 56
XMSG_TYPE_ID_RISED
util::xmsg, 56
XMSG_TYPE_ID_RISEE
util::xmsg, 56
XMSG_TYPE_ID_RISEF
util::xmsg, 56
XMSG_TYPE_ID_SALN0
util::xmsg, 55
XMSG_TYPE_ID_SALN1
util::xmsg, 55
XMSG_TYPE_ID_SALN2
util::xmsg, 55
XMSG_TYPE_ID_SALN3
util::xmsg, 55
XMSG_TYPE_ID_SALN4
util::xmsg, 55
XMSG_TYPE_ID_SALN5
util::xmsg, 55
XMSG_TYPE_ID_SALN6
util::xmsg, 55
XMSG_TYPE_ID_SALN7
util::xmsg, 56
XMSG_TYPE_ID_SALN8
util::xmsg, 56
XMSG_TYPE_ID_SALN9
util::xmsg, 56
XMSG_TYPE_ID_SALN_DRIFT8
util::xmsg, 57
XMSG_TYPE_ID_SALN_DRIFT9
util::xmsg, 57
XMSG_TYPE_ID_SALN_DRIFTA
util::xmsg, 57
XMSG_TYPE_ID_SALN_DRIFTB
util::xmsg, 57
XMSG_TYPE_ID_SALN_DRIFTC
util::xmsg, 57
XMSG_TYPE_ID_SALN_DRIFTD
util::xmsg, 57
XMSG_TYPE_ID_SALN_DRIFTE
util::xmsg, 57
XMSG_TYPE_ID_SALN_DRIFTF
util::xmsg, 57
XMSG_TYPE_ID_SALN_HI_RES0
util::xmsg, 57
XMSG_TYPE_ID_SALN_HI_RES1
util::xmsg, 57
XMSG_TYPE_ID_SALN_HI_RES2
util::xmsg, 57
XMSG_TYPE_ID_SALN_HI_RES3
util::xmsg, 57
XMSG_TYPE_ID_SALN_HI_RES4
util::xmsg, 57
XMSG_TYPE_ID_SALN_HI_RES5
util::xmsg, 57
XMSG_TYPE_ID_SALN_HI_RES6
util::xmsg, 57
XMSG_TYPE_ID_SALN_HI_RES7
util::xmsg, 57
XMSG_TYPE_ID_SALNA
util::xmsg, 56
XMSG_TYPE_ID_SALNB
util::xmsg, 56
XMSG_TYPE_ID_SALNC
util::xmsg, 56
XMSG_TYPE_ID_SALND
util::xmsg, 56
XMSG_TYPE_ID_SALNE
util::xmsg, 56
XMSG_TYPE_ID_SALNF
util::xmsg, 56
XMSG_TYPE_ID_TEMP0
util::xmsg, 55
XMSG_TYPE_ID_TEMP1
util::xmsg, 55
XMSG_TYPE_ID_TEMP2
util::xmsg, 55
XMSG_TYPE_ID_TEMP3
util::xmsg, 55
XMSG_TYPE_ID_TEMP4

util::xmsg, 55
XMSG_TYPE_ID_TEMP5
util::xmsg, 55
XMSG_TYPE_ID_TEMP6
util::xmsg, 55
XMSG_TYPE_ID_TEMP7
util::xmsg, 55
XMSG_TYPE_ID_TEMP8
util::xmsg, 55
XMSG_TYPE_ID_TEMP9
util::xmsg, 55
XMSG_TYPE_ID_TEMP_DRIFT8
util::xmsg, 57
XMSG_TYPE_ID_TEMP_DRIFT9
util::xmsg, 57
XMSG_TYPE_ID_TEMP_DRIFTA
util::xmsg, 57
XMSG_TYPE_ID_TEMP_DRIFTB
util::xmsg, 57
XMSG_TYPE_ID_TEMP_DRIFTC
util::xmsg, 57
XMSG_TYPE_ID_TEMP_DRIFTD
util::xmsg, 57
XMSG_TYPE_ID_TEMP_DRIFTE
util::xmsg, 57
XMSG_TYPE_ID_TEMP_DRIFTF
util::xmsg, 57
XMSG_TYPE_ID_TEMP_HI_RES0
util::xmsg, 57
XMSG_TYPE_ID_TEMP_HI_RES1
util::xmsg, 57
XMSG_TYPE_ID_TEMP_HI_RES2
util::xmsg, 57
XMSG_TYPE_ID_TEMP_HI_RES3
util::xmsg, 57
XMSG_TYPE_ID_TEMP_HI_RES4
util::xmsg, 57
XMSG_TYPE_ID_TEMP_HI_RES5
util::xmsg, 57
XMSG_TYPE_ID_TEMP_HI_RES6
util::xmsg, 57
XMSG_TYPE_ID_TEMP_HI_RES7
util::xmsg, 57
XMSG_TYPE_ID_TEMPA
util::xmsg, 55
XMSG_TYPE_ID_TEMPB
util::xmsg, 55
XMSG_TYPE_ID_TEMPC
util::xmsg, 55
XMSG_TYPE_ID_TEMPD
util::xmsg, 55
XMSG_TYPE_ID_TEMPE
util::xmsg, 55
XMSG_TYPE_ID_TEMPF
util::xmsg, 55
XMSG_TYPE_ID_TEST
util::xmsg, 58
XMSG_TYPE_ID_UNKNOWN
util::xmsg, 58
xmsg_valve_data_
argo::iridium::xfloat::XFloatDiveData, 1050
xmsg_version_major
argo::iridium::xfloat::XFloat, 958
xmsg_version_major_
argo::iridium::xfloat::XFloat, 973
xmsg_version_minor
argo::iridium::xfloat::XFloat, 959
xmsg_version_minor_
argo::iridium::xfloat::XFloat, 973
XMSG_VERSION_UNKNOWN
XDefinitions.h, 1779
xmsg_wave_field_
argo::iridium::xfloat::XFloatDiveData, 1050
XMsgTypeId_t
util::xmsg, 55
XMTPS — X-Message Telemetry Processing Service, 1
XMTPS_DEBUG_FETCH_MAIL
ThreadDefinitions.h, 2119
XMTPS_THREAD_JOIN_SECONDS
ThreadDefinitions.h, 2119
XPHASE_CODE_BEGINING_OF_FALL_TO_PROFILE
util::xmsg, 58
XPHASE_CODE_BEGINNING_OF_DRIFT
util::xmsg, 58
XPHASE_CODE_BUOYANCY_AT_100DBAR
util::xmsg, 58
XPHASE_CODE_DRIFTING
util::xmsg, 58
XPHASE_CODE_END_OF_RISE
util::xmsg, 58
XPHASE_CODE_FALLING_TO_PROFILE
util::xmsg, 58
XPHASE_CODE_ICE_TURNAROUND
util::xmsg, 58
XPHASE_CODE_INVALID
util::xmsg, 58
XPHASE_CODE_RISING
util::xmsg, 58
XPHASE_CODE_SEEK
util::xmsg, 58
XPHASE_CODE_SEEK_DURING_DRIFT
util::xmsg, 58
XPHASE_CODE_SINKING
util::xmsg, 58
XPHASE_CODE_START_OF_RISE
util::xmsg, 58
XPHASE_CODE_START_OF_SINK
util::xmsg, 58
XPHASE_CODE_SURFACE
util::xmsg, 58
XPHASE_CODE_UNKNOWN
util::xmsg, 58
XPhaseCode_t
util::xmsg, 58
XPresCodeCounts
util::xmsg::XPresCodeCounts, 1439
XPresCounts
util::xmsg::XPresCounts, 1445
XProfilePressure
util::xmsg::XProfilePressure, 1449
XProfileSalinity
util::xmsg::XProfileSalinity, 1467
XProfileTemperature
util::xmsg::XProfileTemperature, 1486
XPump
util::xmsg::XPump, 1505
XPUMP_PACK_LEGACY
util::xmsg, 59
XPUMP_PACK_STANDARD
util::xmsg, 59
XPUMP_PACK_STANDARD2
util::xmsg, 59
XPUMP_PACK_UNKNOWN
util::xmsg, 59
XPumpCounts
util::xmsg::XPumpCounts, 1520
XPumpPackingFormat_t
util::xmsg, 58
XRise
util::xmsg::XRise, 1527
XRISEFALL_FMT_LEGACY
util::xmsg, 54
XRISEFALL_FMT_STNDRD
util::xmsg, 54

XRISEFALL_FMT_STNDRD_WITH_TEMP_AND_SALN
util::xmsg, 55
XRISEFALL_FMT_UNKNOWN
util::xmsg, 55
XSENS_FMT_CURVATURE
util::xmsg, 59
XSENS_FMT_DIFFERENCE
util::xmsg, 59
XSENS_FMT_UNKNOWN
util::xmsg, 59
XSensorDataFormat_t
util::xmsg, 59
XSensorTripletCodeCounts
util::xmsg::XSensorTripletCodeCounts, 1547
XSurfaceData
util::xmsg::XSurfaceData, 1555
XValve
util::xmsg::XValve, 1575
XValveCounts
util::xmsg::XValveCounts, 1589
XWave
util::xmsg::XWave, 1593
XWave.h
XWAVE_INVALID_SF, 2083
XWAVE_SIZE_ASSF_RAW, 2083
XWAVE_SIZE_ASSF_SPECTRAL, 2083
XWAVE_SIZE_FSAM, 2083
XWAVE_SIZE_GSSF, 2083
XWAVE_SIZE_MSSF, 2083
XWAVE_INVALID_SF
XWave.h, 2083
XWAVE_MODE_RAW
util::xmsg, 59
XWAVE_MODE_SPECTRAL
util::xmsg, 59
XWAVE_MODE_UNKNOWN
util::xmsg, 59
XWAVE_SIZE_ASSF_RAW
XWave.h, 2083
XWAVE_SIZE_ASSF_SPECTRAL
XWave.h, 2083
XWAVE_SIZE_FSAM
XWave.h, 2083
XWAVE_SIZE_GSSF
XWave.h, 2083
XWAVE_SIZE_MSSF
XWave.h, 2083
XWaveMode_t
util::xmsg, 59
XWaveRawSample
util::xmsg::XWaveRawSample, 1608